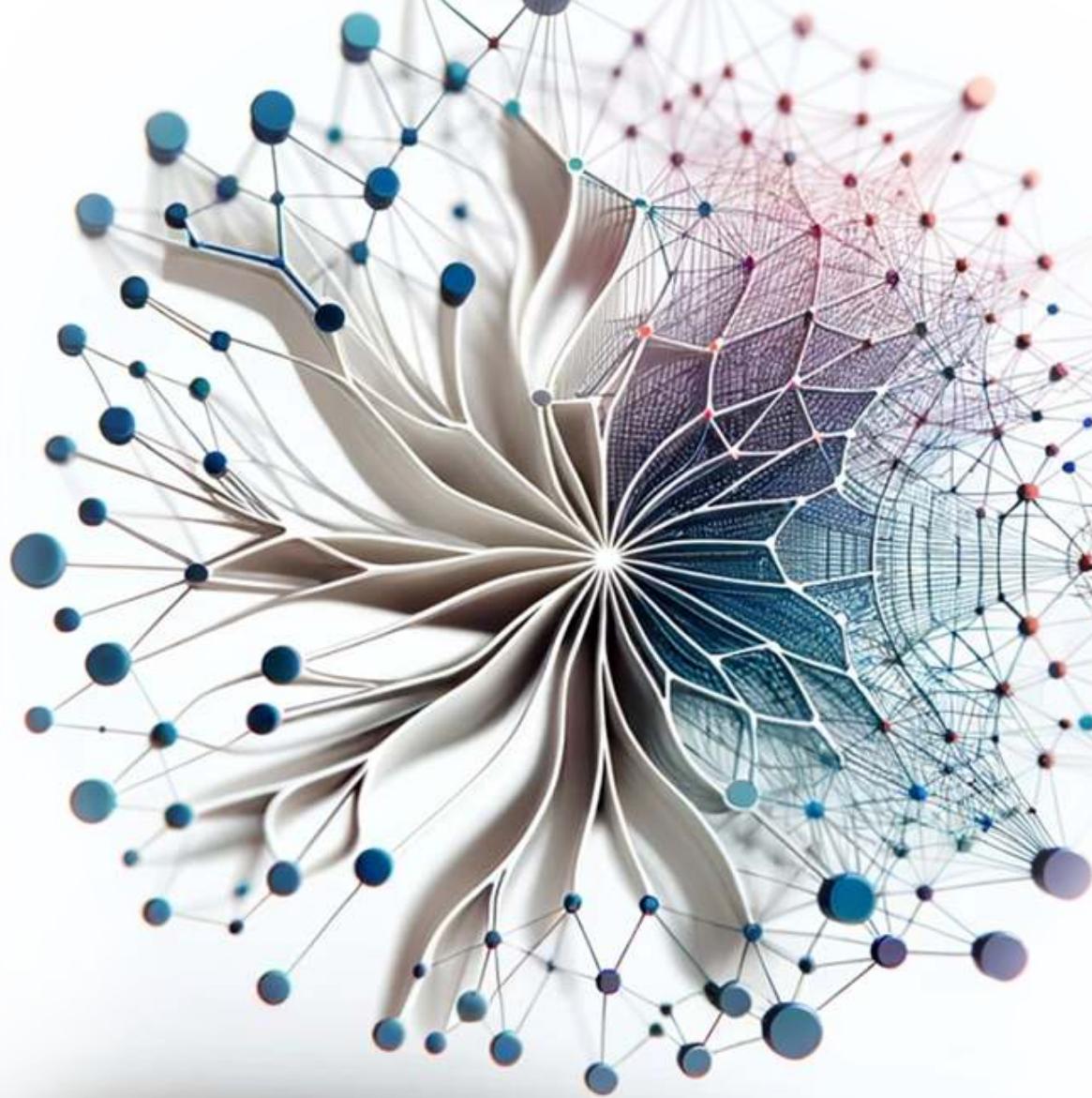




Introduction to

Machine Learning Systems

Vijay
Janapa Reddi

Introduction to Machine Learning Systems

From Algorithms to Silicon: The Engineering of Deep Learning Systems

Introduction to Machine Learning Systems

From Algorithms to Silicon: The Engineering of Deep Learning Systems

Vijay Janapa Reddi

The MIT Press
Cambridge, Massachusetts
London, England

The MIT Press
Massachusetts Institute of Technology
77 Massachusetts Avenue
Cambridge, MA 02139
mitpress.mit.edu

© 2027 Massachusetts Institute of Technology

This work is subject to a Creative Commons CC-BY-NC-ND license.

This license applies only to the work in full and not to any components included with permission. Subject to such license, all rights are reserved. No part of this book may be used to train artificial intelligence systems without permission in writing from the MIT Press.



The MIT Press would like to thank the anonymous peer reviewers who provided comments on drafts of this book. The generous work of academic experts is essential for establishing the authority and quality of our publications. We acknowledge with gratitude the contributions of these otherwise uncredited readers.

This book was prepared and formatted in Quarto by Željko Hrček. Printed and bound in the United States of America.

Library of Congress Cataloging-in-Publication Data is available.

ISBN: 978-0-262-05888-9

EU Authorised Representative: Easy Access System Europe, Mustamäe tee 50, 10621 Tallinn, Estonia | Email: gpsr.requests@easproject.com

*In loving memory of my father,
whose lifelong passion for learning
and unwavering commitment to quality
still challenge me daily to strive for excellence.*

*And with deepest gratitude to my mother,
whose everyday generosity taught me,
more by example than instruction,
that anything worth having
is worth sharing.*

Contents

Foreword	xiii
Author’s Note	xvii
Preface	xix
Acknowledgments	xxv
A Note on AI Assistance	xxvii
Notation	xxix

Main Text

Chapter 1 Introduction	3
1.1 The AI Systems Moment	4
1.2 Data-Centric Paradigm Shift	4
1.3 The Evolution of AI Bottlenecks	6
1.4 The Bitter Lesson	11
1.5 Defining ML Systems	12
1.6 ML vs. Traditional Software	17
1.7 Iron Law of ML Systems	18
1.8 Three Dimensions of ML Efficiency	22
1.9 AI Engineering as a Discipline	25
1.10 Five-Pillar Framework	29
1.11 Book Organization	31
1.12 Fallacies and Pitfalls	32
1.13 Summary	33

Part I: Foundations of ML Systems

Foundation Principles	37
Chapter 2 ML Systems	39
2.1 Deployment Paradigm Framework	40
2.2 Physical Constraints: Why Paradigms Exist	41
2.3 Analyzing Workloads	43
2.4 System Balance and Hardware	47
2.5 Cloud ML: Computational Power	50
2.6 Edge ML: Latency and Privacy	56
2.7 Mobile ML: Offline Intelligence	61
2.8 TinyML: Ubiquitous Sensing	64
2.9 Paradigm Selection	68
2.10 Hybrid Architectures	72

2.11	System Entropy: Why Deployment Is Not the End	76
2.12	Fallacies and Pitfalls	77
2.13	Summary	79
Chapter 3	ML Workflow	81
3.1	ML Lifecycle	82
3.2	Lifecycle Stages	86
3.3	Problem Definition	90
3.4	Data Collection	92
3.5	Model Development	95
3.6	Evaluation and Validation	98
3.7	Deployment and Integration	101
3.8	Monitoring and Maintenance	103
3.9	Systems Thinking	105
3.10	Fallacies and Pitfalls	107
3.11	Summary	108
Chapter 4	Data Engineering	111
4.1	Dataset Compilation	112
4.2	Physics of Data	113
4.3	Four Pillars Framework	116
4.4	Data Acquisition	123
4.5	Data Pipeline Architecture	128
4.6	Systematic Data Processing	140
4.7	Data Labeling	148
4.8	Storage Architecture	153
4.9	Operational Data Health	162
4.10	Fallacies and Pitfalls	164
4.11	Summary	166
Part II: Development and Training		
Development Principles		171
Chapter 5	Neural Computation	173
5.1	From Logic to Arithmetic	174
5.2	Computing with Patterns	176
5.3	Neural Network Fundamentals	187
5.4	Learning Process	200
5.5	Inference Pipeline	218
5.6	Handwritten Digit Recognition	221
5.7	D·A·M Taxonomy	225
5.8	Fallacies and Pitfalls	226
5.9	Summary	227
Chapter 6	Network Architectures	231
6.1	Architectural Principles	232
6.2	MLPs: Dense Pattern Processing	237
6.3	CNNs: Spatial Pattern Processing	242
6.4	RNNs: Sequential Pattern Processing	252
6.5	Attention: Dynamic Processing	256
6.6	Transformers: Parallel Sequence Processing	261
6.7	Sparse Architectures: RecSys	266
6.8	Shared Building Blocks	269
6.9	Computational Primitives	276
6.10	Architecture Selection Framework	282

6.11	Fallacies and Pitfalls	288
6.12	Summary	289
Chapter 7	ML Frameworks	293
7.1	Three Framework Problems	294
7.2	The Ladder of Abstraction	296
7.3	Execution Problem	297
7.4	Differentiation Problem	313
7.5	Abstraction Problem	324
7.6	nn.Module Abstraction	338
7.7	Framework Platform Analysis	342
7.8	Deployment Targets	347
7.9	Framework Selection	348
7.10	Anatomy of a Training Step	352
7.11	Fallacies and Pitfalls	355
7.12	Summary	357
Chapter 8	Model Training	359
8.1	Training Systems Fundamentals	360
8.2	Iron Law of Training Performance	362
8.3	Mathematical Foundations	366
8.4	Pipeline Architecture	383
8.5	Identifying Bottlenecks	393
8.6	Pipeline Optimizations	396
8.7	Scaling Training Systems	418
8.8	Fallacies and Pitfalls	426
8.9	Summary	427
Part III: Optimization and Acceleration		
Optimization Principles		431
Chapter 9	Data Selection	433
9.1	Data Selection Fundamentals	434
9.2	Static Pruning	440
9.3	Dynamic Selection	445
9.4	Self-Supervised Learning	450
9.5	Synthetic Data Generation	452
9.6	Decision Framework	455
9.7	Selection Engineering	457
9.8	Cost Modeling	462
9.9	Distributed Selection	465
9.10	Cross-Layer Interactions	467
9.11	Measurement Framework	470
9.12	Fallacies and Pitfalls	473
9.13	Summary	475
Chapter 10	Model Compression	479
10.1	Optimization Framework	480
10.2	Deployment Context	482
10.3	Structural Optimization	485
10.4	Quantization and Precision	504
10.5	Architectural Efficiency	524
10.6	Technique Selection	537
10.7	Optimization Strategies	539
10.8	Efficiency Measurement	539

10.9	Implementation Tools	542
10.10	Fallacies and Pitfalls	545
10.11	Summary	547
Chapter 11	Hardware Acceleration	549
11.1	Acceleration Fundamentals	550
11.2	Hardware Specialization	554
11.3	AI Compute Primitives	563
11.4	Compute Units and Execution Models	570
11.5	AI Memory Systems	583
11.6	Roofline Model	596
11.7	Hardware Mapping	603
11.8	Dataflow Optimization	608
11.9	Compiler Support	622
11.10	Runtime Support	627
11.11	Multi-Chip Scaling	630
11.12	Heterogeneous SoC Design	631
11.13	Hardware Sustainability	635
11.14	Fallacies and Pitfalls	635
11.15	Summary	638
Chapter 12	Benchmarking	639
12.1	ML Benchmarking Framework	640
12.2	Historical Foundations	642
12.3	System Benchmarking Suites	645
12.4	Benchmarking Granularity	652
12.5	Benchmark Components	657
12.6	Training vs. Inference	668
12.7	Training Benchmarks	668
12.8	Inference Benchmarks	675
12.9	Power Measurement Techniques	686
12.10	Benchmarking Best Practices	691
12.11	Model and Data Evaluation	696
12.12	Production Considerations	704
12.13	Fallacies and Pitfalls	706
12.14	Summary	707
Part IV: Deployment and Operations		
Deployment Principles		711
Chapter 13	Model Serving	715
13.1	Serving Paradigm	716
13.2	Serving Load, Latency, and Architecture	717
13.3	Serving System Architecture	724
13.4	Request Lifecycle	726
13.5	Queuing Theory	733
13.6	Model Lifecycle Management	739
13.7	Throughput Optimization	743
13.8	LLM Serving	755
13.9	Inference Runtime Selection	758
13.10	Node-Level Optimization	762
13.11	Economics and Planning	765
13.12	Fallacies and Pitfalls	769
13.13	Summary	770

Chapter 14 ML Operations	773
14.1 MLOps Overview	774
14.2 Principles and Foundations	775
14.3 Technical Debt	780
14.4 Development Infrastructure	785
14.5 Production Operations	800
14.6 Design and Maturity Framework	825
14.7 Case Studies	830
14.8 Fallacies and Pitfalls	835
14.9 Summary	837
Chapter 15 Responsible Engineering	841
15.1 Responsibility as Systems Engineering	842
15.2 Engineering Responsibility Gap	843
15.3 Responsible Engineering Checklist	852
15.4 Environmental and Cost Awareness	865
15.5 Data Governance and Compliance	870
15.6 Fallacies and Pitfalls	877
15.7 Summary	878

Synthesis

Chapter 16 Conclusion	883
16.1 Synthesizing ML Systems	884
16.2 Thirteen Quantitative Principles	886
16.3 Principles in Practice	891
16.4 Future Directions	892
16.5 Journey Forward	895
16.6 Fallacies and Pitfalls	896
16.7 Summary	897

Back Matter

Appendices	901
Appendix A The D·A·M Taxonomy	901
Appendix B Data Foundations	909
Appendix C Algorithm Foundations	915
Appendix D Machine Foundations	929
Appendix E System Assumptions	941
Glossary	953
References	965
Index	993

Foreword

David A. Patterson

Pardee Professor of Computer Science Emeritus
University of California, Berkeley
Distinguished Engineer, Google
ACM A.M. Turing Award Laureate
Berkeley, California

My career has been dedicated to understanding and improving the very foundations upon which computing operates – from the instruction set architectures that define processors to the storage systems that hold our data. I’ve seen firsthand how profound advancements are rarely about a single brilliant idea in isolation, but rather about the holistic integration of many ideas into a robust, efficient, and reliable system.

Today, we stand at another pivotal moment. Machine learning (ML) has transcended scientific curiosity to become a force reshaping industries, driving innovation, and transforming our daily lives. The intellectual power of the algorithms, the gains in accuracy, and the seemingly magical capabilities they unlock have rightly captured the world’s imagination.

One concern is that much of this progress has been achieved by a community that, through no fault of its own, was never taught to think in systems terms about what it was building. Engineers who can call `model.fit()` but cannot explain why training is slow. Researchers who tune hyperparameters without understanding the memory bandwidth constraints that make certain choices impractical. Practitioners who deploy models into production without a framework for reasoning about latency, throughput, or failure modes.

A brilliant algorithm, without a well-engineered system to support it, is like a powerful engine without a chassis and brakes. It won’t get you far, and it certainly won’t get you there reliably.

This gap is not a talent deficit. It is a curriculum deficit. The field has simply not had the textbooks it deserves. This book, *Introduction to Machine Learning Systems*, arrives precisely at the right time to illuminate this critical, often overlooked, dimension of the ML revolution.

Why Systems Thinking Matters

My career has leveraged the power of systems thinking. Identify the bottlenecks, understand the interactions between components, and design for performance, reliability, cost-effectiveness, and ease of use across the entire stack. You could not reach the conclusion from first principles alone. You had to think about the system.

ML systems demand exactly the same discipline. A neural network is not a mathematical abstraction floating free of physical constraints. It is a computation that must execute on silicon with finite memory bandwidth, finite power budgets, and finite communication capacity. Arithmetic intensity, the Roofline Model, and memory hierarchy effects are not advanced topics to be addressed after students learn the basics. They are the basics.

Why ML Systems Deserve Serious Textbook Treatment

The purpose of a textbook is not just to catalogue the current frontier. It is to provide the conceptual scaffolding that allows students to understand the frontier, and to advance it themselves. The neural network architectures covered in these pages will evolve. The hardware accelerators described will

be superseded. But some fundamentals will not change. We will still have memory hierarchies and arithmetic intensity will still be a critical measure. The principles of pipelining, of latency versus throughput trade-offs, of quantization and precision reflect physical and mathematical realities that do not go out of date. Anyone who genuinely understands them will be able to reason about whatever new system the field produces next year or ten years from now.

I am also struck by the scope of what this book attempts. It is not enough to explain how training works; one must also address data engineering, model optimization, hardware acceleration, deployment, benchmarking, responsible AI, and sustainability. These are not separate topics bolted together for completeness. They are interconnected aspects of a single engineering discipline. A decision made at the data layer has implications for model behavior, which has implications for inference latency, which has implications for deployment cost, which has implications for the communities that can and cannot access the resulting system. The book correctly treats this integration as a first-class concern.

I am particularly pleased to see the quantitative methodology that runs through every chapter. Claims are backed by measurements. Trade-offs are expressed numerically. Students are taught to estimate, measure, and verify rather than to rely on their gut. The difference between an engineer and a dilettante is precisely this capacity for rigorous, measurement-grounded reasoning.

Why Open Access Is Important for the Next Generation

I applaud the decision to make this invaluable resource freely available.

When I co-wrote *Computer Architecture: A Quantitative Approach* with John Hennessy, we debated how to make the material as accessible as possible. We lobbied our publisher to keep the price down and even came out with a low cost international student edition for sales outside North America. Those concerns have only grown more urgent as ML has become infrastructure — as consequential to the modern world as the electrical grid or the internet.

The decision to publish this book as a freely available educational resource is not a compromise. It is a statement of values. ML systems are being deployed in every sector of every economy on every continent. The engineers who build and maintain them will come from everywhere. If the foundational education required to do this work responsibly is available only to students at well-resourced institutions, we will contribute to an expertise gap that mirrors the societal concerns of who benefits from AI and who bears its costs.

What impresses me further is that this project has gone well beyond open access to a static text. The accompanying ecosystem — interactive laboratories, a build-from-scratch framework in TinyTorch, hardware deployment kits, instructor materials, a global network of adopting universities — reflects a familiar educational maxim: “I hear and I forget. I see and I remember. I do and I understand.”

The open-source model also makes the curriculum itself better. Errors get corrected. Examples get sharpened. New chapters emerge in response to what the field actually needs. As we found with *Computer Architecture: A Quantitative Approach*, a community of contributors that forms around a book can offer a form of quality assurance that no single author team, however expert, can match alone.

A Final Word

The field of computer architecture has a tradition I am proud of: we build things, measure them honestly, share what we learn, and revise our philosophies when the measurements demand it. We do not hide behind the complexity of our systems to avoid accountability for whether they actually work. We do not conflate novelty with progress. We debate, rigorously, about what matters and why.

Hopefully, ML systems engineering will follow this tradition. It needs the habits of mind that come from taking systems seriously: thinking about interactions, measuring at the right level of abstraction, being honest about trade-offs, and holding oneself accountable to real-world performance rather than benchmark folklore. This book embodies those habits.

Introduction to Machine Learning Systems is more than just a textbook; it is a clarion call for systems thinking in the age of AI, a guide for building robust ML applications, and a testament to the power of open education.

We are at an extraordinary moment. Over my career there has been a major breakthrough every decade: the microprocessor in the 1970s, the personal computer in the 1980s, the Internet in the 1990s, the smartphone in the 2000s, and AI in the 2010s. The first four changed business and society, yet AI looks likely to have the largest impact of them all.

The systems being built today will shape how intelligence is deployed, who benefits from it, and what it costs — in money, in energy, and in carbon footprint. We need the engineers who build those systems to have the best possible systems skills.

This book is a significant step toward making it possible for anyone to learn them.

David A. Patterson

Author's Note

THE WORLD IS RUSHING to build AI systems; it is not yet engineering them. That sentence is the reason this book exists. A model that wins on a benchmark is not yet a system, and the distance between the two is not a detail of implementation. It is engineering: the work of making something hold up under constraints that do not negotiate. A demo answers to no one. A system answers to latency budgets, memory limits, power envelopes, and the cost of being wrong in front of real users.

What makes this its own discipline is an unusual coupling. A processor answers to the physics of the machine, to clock cycles, cache lines, and the heat it can shed. A statistical model answers to the mathematics of learning, to bias, variance, and how generalization changes with data and model capacity. A machine learning system must answer to both at once, in the same decision, and the two do not always agree. A change that pleases the hardware can starve the model of the precision or throughput it needs to converge; a change that helps the model can ask the silicon for more memory or bandwidth than it can give. Hold that tension in view and the field stops looking like a pile of tools and starts looking like a structure you can reason about.

This book hands you that structure. Not the framework that is fashionable this year or the accelerator that is fastest this quarter, but the way of thinking that outlasts them. Frameworks change, hardware turns over every few years, and the benchmarks reset; the forces underneath stay the same. If the book works, you will finish it able to look at any machine learning system, from a sensor sipping microwatts to a server farm drawing megawatts, and see the same small set of forces deciding what is possible.

The textbooks that changed me when I was a student did not hand me facts. They handed me a way of seeing, and the facts arranged themselves around it. That is what I have tried to do here.

— Vijay Janapa Reddi
Cambridge, Massachusetts
2026

Preface

Who This Book Is For

This book is for anyone who wants to understand *how* machine learning systems actually work, from the mathematics that define a neural network to the hardware that executes it and the engineering decisions that make it run efficiently.

Most readers will not become ML hardware architects or compiler engineers. Yet every practitioner who builds, trains, optimizes, or deploys a model makes decisions that depend on understanding the system beneath the code. This book provides that systems foundation for undergraduates encountering ML systems for the first time, practicing engineers strengthening their foundations, and researchers who need to reason about performance and deployment.

Why a Systems Textbook

In 1968, a NATO conference in Garmisch, Germany, helped establish *software engineering* as a distinct discipline amid concern that software systems had become too complex for ad hoc programming (Naur and Randell 1969). Reliable software required its own principles, methods, and rigor. Decades later, Hennessy and Patterson made quantitative analysis central to computer architecture education, systematizing how engineers measure, predict, and optimize processor performance (Hennessy and Patterson 2017). Both examples show how shared methods make complex engineering work systematic and teachable.

Machine learning faces its own Garmisch moment.

For the past decade, ML research has been dominated by *model-centric* thinking focused on discovering new algorithms and architectures. That focus matters, but a model is not a system. A neural network that performs well in a research notebook is useless if it cannot meet latency constraints, if its data pipeline cannot handle the required volume, or if its energy cost exceeds its economic value. The gap between a working model and a production system is not merely an implementation gap; it is a gap in fundamental engineering principles.

This book treats ML systems engineering not as a collection of tools, such as Kubernetes, PyTorch, and CUDA, but as a discipline grounded in quantitative bounds, models, and design principles. Just as civil engineers rely on foundational physics and structural principles, AI engineers must make the governing principles behind each system model explicit:

- **The iron law of ML systems:** A decomposition of movement, compute, and overhead, with overlap handled by the critical path.
- **Data gravity:** The placement pressure created by large data volumes and constrained networks.
- **Energy accounting:** Per-event costs and event counts determine whether movement or computation dominates.
- **Amdahl's law:** The serial fraction of a pipeline caps total system speedup.

Computer science studies what is computable. ML research studies what is learnable. AI engineering studies what is buildable, given the physics of real hardware and real data.

I wrote this book to help build that foundation and to do for AI systems what Hennessy and Patterson (2017) did for computer architecture: replace intuition with measurement, and art with engineering. The goal is to teach system-level reasoning before implementation, treating data, algorithms, and hardware not as separate silos but as a single, coupled optimization problem.

Consider a box of LEGO bricks. The same interlocking pieces build a spaceship, a castle, or an entire city. The sets change; the bricks do not. ML systems work the same way. A convolutional network for image classification, a transformer for language generation, and a reinforcement learning agent for robotics are vastly different applications, but beneath them sit the same building blocks: computational graphs, memory hierarchies, data pipelines, optimization loops, and the trade-offs between latency, throughput, and energy. Mastery of those building blocks makes unfamiliar systems legible.

That is why this book teaches enduring principles rather than current tools. Dominant frameworks have shifted from Theano to TensorFlow to PyTorch in under a decade. Hardware has evolved from repurposed graphics processors to purpose-built tensor accelerators, with new architectures emerging every year. Any textbook that teaches today's tools will be obsolete before its next edition. The building blocks endure.

What This Book Covers

This book centers on the *single compute node*: one host with one to eight accelerators, typically with discrete device memories connected by an interconnect. It also introduces multi-accelerator training and production operations where they clarify node-level design; fleet-scale treatment lies outside its scope.

The content is organized into four parts, each with a distinct role in the systems argument:

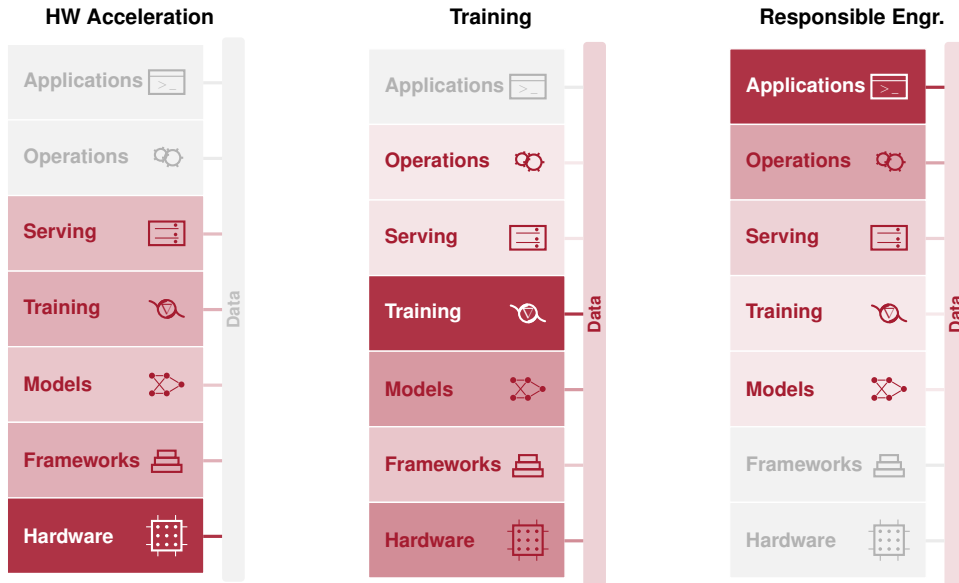
- **Part I: Foundations of ML Systems** develops the vocabulary, mental models, and end-to-end workflow that guide every ML project, from the characteristics that distinguish ML systems from traditional software through the data engineering pipelines that feed them.
- **Part II: Development and Training** connects mathematical foundations to working systems through deep learning computation, neural architectures, frameworks, and training procedures.
- **Part III: Optimization and Acceleration** addresses data efficiency, model compression, hardware acceleration, and the benchmarking methodology needed to measure progress rigorously.
- **Part IV: Deployment and Operations** covers serving infrastructure, operational practices, and responsible engineering for systems that must be fair, transparent, and trustworthy in production.

These four parts trace a path through the ML systems stack, a layered abstraction modeled on the classical computer systems stack. Each layer provides an abstraction to the layer above and consumes the layer below: from hardware at the bottom through frameworks, models, training, and serving, up to operations and applications at the top. Data flows through all layers as a cross-cutting interconnect, much like a data bus in computer architecture.

Throughout the book, a margin figure highlights which layers each chapter addresses, providing a recurring map of the stack.

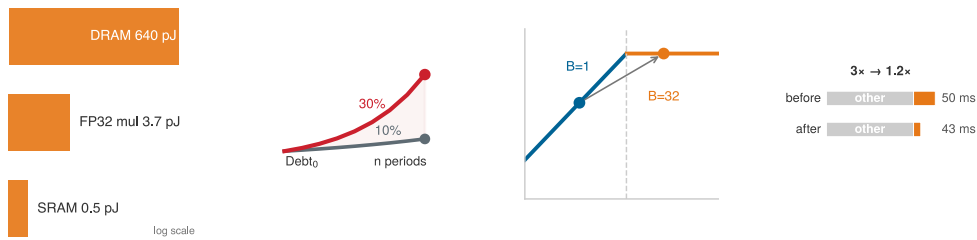


The data bar alongside the stack is not merely decorative. Its uniform shading reflects how central data is to each chapter's concerns, a separate dimension from the layer intensities. The connecting wires between each layer and the data bar show which layers interact with data in that chapter's context. Three chapters illustrate how the lens shifts.



In chapter 11, the layer intensity concentrates at the bottom of the stack while the data bar is faint: the chapter is about silicon, not data. In chapter 8, the middle layers glow and the data bar is moderately lit, reflecting that training consumes data but is fundamentally about optimization. In chapter 15, the top layers dominate and the data bar carries a moderate tint, because biased training data surfaces as biased applications even though several layers sit between them. The same stack tells three different stories, with data as an independent dimension.

The margins also carry small visual notes at the point where a relationship is easiest to understand by sight. These are not numbered figures to cite later; they are reading aids. A ladder makes a scale gap visible, a trend sketch shows direction, a roofline thumbnail locates a bottleneck regime, and a dominance bar shows which term controls a total. When a miniature uses a log scale, it says so inside the image.



Each part builds on the previous one. Sequential reading is recommended for those new to the field, while the following tailored paths support specific professional goals:

Suggested Reading Paths

Readers with different backgrounds and goals may benefit from tailored paths:

- **The Complete Path** (all chapters, in order): For undergraduates and readers new to ML systems. Start with Part I, then progress through Parts II, III, and IV. This path develops every concept from first principles.

- **The ML Engineer’s Path** (systems depth for practitioners): For engineers who already train and deploy models but want to understand the systems underneath. Skim Part I for vocabulary, then focus on chapter 7, chapter 8, chapter 11, chapter 12, and chapter 13.
- **The Optimization Path** (efficiency-focused): For engineers working on model efficiency, edge deployment, or resource-constrained systems. After Part I, move directly to chapter 9, chapter 10, chapter 11, and chapter 12.

Conventions

This book uses a small set of typographic conventions to mark what each piece of prose is doing.

Bold marks first definitions

A term in **bold** within body prose is its formal first introduction in the volume. Each term receives this treatment exactly once. Subsequent appearances are unbolded; by then the term is part of the working vocabulary the chapter carries forward.

Every machine learning system has a **dual mandate**: it must manage statistical uncertainty and physical constraints simultaneously.

The bold is a contract: register this term because it will recur. A matching index entry in the back of the book points to the defining occurrence (under the **definition** subentry), providing a way to return to it. Key technical terms are also compiled in the glossary.

Bold also appears in a second, structural role: as a functional label at the start of an item inside a callout box or a summary bullet—for example, **Recommendation**, **Trade-off**, or **Result**. In that context, the bold is scaffolding rather than a first definition.

Italics carry specific rhetorical jobs

Italics are not used for general emphasis. Every italic span in this book performs one of a small number of specific jobs:

- **Direct opposition**: *training vs. inference, logical vs. physical.*
- **Impossibility, stated as physical, mathematical, or logical law**: “sub-10 ms systems *cannot* use distant cloud infrastructure—physics forbids it.”
- **Word used as word**: “the term *gradient* refers to a vector of partial derivatives.”
- **Titles of works**: *Computer Architecture: A Quantitative Approach.*
- **Thesis punchline**: A single italicized sentence that captures the controlling insight of a section, for example, *Data is source code.*

An italicized word is performing one of these jobs. A word that is not italicized carries no implicit emphasis.

Callouts have visual identities that match their job

Callout boxes are not decoration. Each type signals a specific kind of content, so the visual identity announces what the box contains before the reader engages with it.

Callout	Contains
Definition	Formal definition of a key term, with its significance, distinction from the nearest neighbor concept, and a common pitfall
Example	Historical case or concrete scenario, structured as context, insight, and systems lesson
Systems Perspective	Analytical observation or mathematical nuance that enriches the surrounding argument
Napkin Math	Worked calculation walked through step by step, with values computed inline from the book’s constants; intermediate tables and equations are the math itself
Checkpoint	Self-check questions to verify understanding of the preceding section

Callout	Contains
War Story	Documented engineering incident with a sourced failure mode, structured as context, failure, consequence, and lesson
Lighthouse	Recurring workloads characterized by a profile (parameters, FLOPs, dominant constraint), used as anchored case studies across multiple chapters
Principle	Canonical book principle or invariant referenced from later chapters
Theorem	Formal, equation-backed result or bound, stated with its conditions and applied in later quantitative analysis
Key Takeaways	Four to seven most important results from a chapter, in summary form
Learning Objectives	Skills and concepts that readers should be able to explain or apply after completing the chapter
What's Next	Bridge from the chapter just ended to the chapter beginning, naming the open question the next chapter addresses

When a callout contains a table, figure, or equation, that artifact is part of the callout's pedagogical unit, not decoration. A lighthouse's profile *is* its characterization. A napkin math box's intermediate tables *are* the worked math. Reading these as separated artifacts loses the teaching arc. Standalone reference tables, by contrast, sit between paragraphs as numbered artifacts the reader returns to through cross-references. Both forms are valid; the visual marker (callout vs. standalone) tells the reader which one they are looking at.

Whichever path readers follow, the text is meant to connect with supporting resources beyond the chapters themselves.

Companion Resources

This book is part of a broader learning ecosystem. The complete text, interactive notebooks, lecture slides, exercises, hands-on labs, and supplementary materials for every chapter are available online at mlsysbook.ai.

This book is open source. The full text, figures, and build system are publicly available, and contributions from readers worldwide have materially improved every chapter. Visit mlsysbook.ai/git to report issues, suggest improvements, or contribute directly.

Prerequisites

Readers should bring two prerequisites:

- **Programming:** Fluency in Python, including functions, classes, and basic data manipulation with NumPy.
- **Mathematics:** Comfort with linear algebra, basic calculus, and probability at the undergraduate level.

Two additional areas are helpful but not required:

- **Computer systems:** Familiarity with memory hierarchies and basic computer architecture deepens the optimization and hardware chapters.
- **Machine learning basics:** Prior exposure to supervised learning concepts provides useful context, but Part II develops the necessary foundations from first principles.

Beyond This Book

This book develops the single-node foundation while introducing distributed execution, production operations, and governance where they affect the end-to-end system. Advanced multi-node topologies, fleet-wide collective algorithms, and cluster resilience extend into distributed systems at scale. Completing this text prepares readers to navigate that broader engineering landscape.

Using This Book in a Course

This book grew out of CS249r at Harvard University and the TinyML edX professional certificate program. The TinyML course taught ML systems at the most constrained end of the spectrum:

microcontrollers running on milliwatts with kilobytes of memory. That work demonstrated that the physical constraints governing embedded devices (memory bandwidth, compute density, and energy per operation) are the same constraints that govern data center GPUs. *The physics scales; only the numbers change.* That insight shaped this book: mastery of systems fundamentals at the smallest scale prepares engineers for every scale.

In course settings, the book is designed to support a one-semester sequence covering the full ML systems stack. Instructors may also select individual parts for shorter modules:

- **Parts I–II** form an introductory half-semester on ML systems foundations and how such systems are built and trained.
- **Parts III–IV** form an applied half-semester on optimizing, deploying, and operating them.

For survey courses or executive programs, the four part introductions alone provide a condensed overview of the quantitative principles that govern ML systems.

Lecture slides, assignments, and instructor resources are available at mlsysbook.ai.

Acknowledgments

Origins

This book owes its existence to a single conversation. During a sabbatical at Google, Pete Warden invited me to work on what seemed like an impossible moonshot for TinyML: running a small speech model on a microcontroller. That collaboration on TensorFlow Lite for Microcontrollers revealed something fundamental. The constraints that govern a device with 16 KB to 256 KB of memory are the same constraints that govern an accelerator in a data center, even when the scale is entirely different. Bandwidth, latency, energy, and memory obey the same physics across both settings. That insight became the seed of this book.

The TinyML work led to an online course on HarvardX, co-taught with Laurence Moroney, that grew to over 100,000 students worldwide. Brian Plancher helped build the TinyML curriculum. Marcelo Rovai, a professor who first encountered the material through the TinyML course, went on to develop the hands-on labs and kits that accompany this book. Zach Shelby opened the Edge Impulse platform for student use. Massimo Banzi and the Arduino team, along with Eric Pan and Seeed Studio, collaborated with us in developing and providing the hardware kits that made hands-on instruction possible. Marco Zennaro brought the material to developing countries through ICTP and planted the idea that these principles deserved a proper textbook. Teaching at that scale, without one, made the need undeniable.

That need turned into a book through CS249r at Harvard University. In 2023, the semester-long class project was to write the class notes: each student group took ownership of a topic, researched it, and worked with me to develop it into a chapter. That work seeded every chapter in this book. The premise was that writing forces you to think deeper than reading ever does. We then opened it all on GitHub, and what began as rough class notes has been rewritten and refined with every semester since.

Because that GitHub work continues, this book benefits from a community whose contributions evolve with the project. New contributors join as readers report issues, improve explanations, and extend the supporting materials. A printed acknowledgment can capture only one moment in that continuing history. The current contributor list and contribution guidelines are available at <https://mlsysbook.ai/git>.

Collaborators and Colleagues

Beyond the class and GitHub community, my earlier collaborations shaped the book's intellectual foundations. My time at Google informed much of the systems thinking in these pages. That period began with performance modeling for the first Pixel system on chip (SoC), drawing on my background in computer architecture. A collaboration with Mark Hill on roofline-style analytical modeling for heterogeneous SoCs taught me to reason about hardware from first principles, the kind of quantitative thinking that runs through every chapter. That emphasis on analytical modeling led naturally to systematic evaluation, and systematic evaluation led to MLPerf. MLPerf involved many contributors across MLCommons, but I worked most closely with Peter Mattson, David Kanter, Carole-Jean Wu, Christine Cheng, Guenther Schmuelling, Cody Coleman, and Cliff Young. That collaboration shaped how I think about rigorous measurement of ML systems.

Boris Murmann and Song Han convinced me in the early stages that this project was worth pursuing. As I wrote, I sought feedback from colleagues in industry and academia alike. Practitioners at Arm, Google, Intel, Meta, Microsoft, Qualcomm, STMicroelectronics, and other organizations

offered perspectives grounded in production systems, while academic reviewers at universities across Europe, Asia, and the Americas stress-tested the material against their own research and teaching. Their combined feedback sharpened every chapter. Evgeni Gousev at Qualcomm and Pete Bernard at the Edge AI Foundation supported the project's outreach and helped build the community around it.

That feedback loop extended to classrooms around the world, where instructors put the material to the test when it was still rough. Sophia Shao at UC Berkeley, Xiaofan (Fred) Jiang at Columbia, Tushar Krishna and Divya Mahajan at Georgia Tech, and Wenkai Guan at the University of Minnesota Morris were among the first, and instructors at IIT Bombay, NUST Pakistan, De La Salle University in the Philippines, and universities across Europe, Latin America, and Africa have since adopted the book for their own courses. Their willingness to teach from an unfinished textbook provided invaluable feedback and confirmed that the need for this material was not limited to any one institution or any one country.

Students and Contributors

My students in the Edge Computing Lab at Harvard shaped this book in ways I cannot fully trace. Some did foundational work that helped establish TinyML as a field; their research and the ideas that emerged from years of collaboration informed the book's technical content. This cohort helped set the field in motion and then helped me write about it: Andy Cheng, Arya Tschand, Brian Plancher, Colby Banbury, Elizabeth Suitor, Emma Chen, Ikechukwu Uchendu, Jason Jabbour, Jason Yik, Jeffrey Ma, Jessica Quaye, Mark Mazumder, Matthew Stewart, Maximilian Lam, Radhika Ghosal, Shvetank Prakash, Srivatsan Krishnan, Yu-Shun Hsiao, and Zishen Wan. Kai Kleinbard, then at the Harvard Extension School, built the online generative AI SocratiQ tutor bot. Naeem Khoshnevis, from the Harvard Kempner Institute, established essential infrastructure including GitHub Actions and documentation standards. Željko Hrček produced the TikZ diagrams and contributed to the PDF layout, page composition, and final visual formatting of the book.

A textbook that distills principles from an active research field also owes a debt to the researchers whose work it builds on. The open-source community shaped this book in ways that no single author could. Dozens of contributors caught errors, improved explanations, filed issues, and submitted pull requests. Some fixed a single typo; others rewrote entire sections. The book improved because people who owed it nothing chose to make it better. I am grateful to every one of them.

Support

This work also depended on support from the Harvard Data Science Initiative, Harvard Extension School, and the National Science Foundation; from the Edge AI Foundation and ICTP for educational outreach and scholarships; and from Arduino, Edge Impulse, Google, and Sseed Studio for hardware, tooling, and infrastructure that enabled the practical labs.

At MIT Press, Susan Hartman believed in this project and guided it from an open-source experiment to a published textbook. Her support, editorial judgment, and friendship made this book real in a way that GitHub alone could not. I am equally grateful to the MIT Press for agreeing to keep this book available as open access. A textbook about a discipline shaped by the open-source community should be accessible to everyone in it. That MIT Press shared this conviction made all the difference.

Finally, I thank my wife Kari and my daughters Nora and Maya. This book was written in the hours that belonged to them: late nights that started at ten and ended at two or three in the morning, weekends that disappeared into revisions, years of mornings when I was present but not quite there. What kept me going, aside from stubbornness, was the open-source community. Every GitHub star was a signal that someone out there cared, that someone was learning, that the work mattered. The cost was real, and they paid it with patience I did not always deserve.

A Note on AI Assistance

Every chapter of this book was researched, written, and rewritten by me. I also used AI tools throughout. What follows is more than the disclosure MIT Press requires of authors who use AI. It is a statement of the standard I held the book to, and a claim about what authorship means when the tools are this capable.

The standard is traceability. Derived quantities in these chapters are backed by Python computation cells whose source code ships with the book. Citations were checked against their sources, and cross-references are checked against stable anchors. The purpose-built infrastructure modeling engine used for the book's system calculations is available at <https://mlsysbook.ai/mlsysim>. A pre-commit test suite checks the machine-testable parts of these invariants on every commit, covering unit consistency, inline-reference resolution, cross-reference integrity, notation canonicity, and citation hygiene. AI tools made parts of this rigor mechanically tractable. The enforcement infrastructure makes that work auditable.

Authorship is thinking. Which concepts belong in this book and which do not. How to sequence the chapters so that each idea arrives only after the reader has the tools to understand it. Which worked examples to trace end-to-end, and which numbers to compute so the reader can verify the argument independently. What level to pitch each explanation at, rigorous enough for a graduate engineer and concrete enough for a student encountering systems thinking for the first time. Where students will get stuck, because I have taught this material and watched them get stuck. These are judgment calls that no tool can make, because they require knowing the reader. I used AI tools throughout this process to brainstorm framings, explore alternatives, and pressure-test my reasoning. The choices are mine.

In practice, AI was genuinely useful for writing code: the computation cells, the pre-commit checks, and the worked examples that are essentially programs. It helped me survey literature I then read in the original, draft passages I then rewrote substantially, and audit the manuscript for consistency across hundreds of cross-references, thousands of index entries, and tens of thousands of lines of prose. Throughout, the tools proposed and I ratified.

This pattern is older than the present moment. In 1683 Joseph Moxon printed the first English-language manual on printing, *Mechanick Exercises on the Whole Art of Printing*, using the press it described. Three centuries later Donald Knuth wrote *The TeXbook* in TeX, the typesetting system he created after becoming dissatisfied with the typesetting of his own books. A book about ML systems infrastructure, written without the infrastructure it describes, would be a contradiction. The toolmaker documents the tool with the tool.

Machine learning systems cannot be certified the way traditional logic can. That verification gap applies to AI-assisted authoring with equal force. The standard I held this book to is the standard I want us to demand of every system we ship: the tool can propose, but only an engineer can ratify. MIT Press does not list AI tools as authors because they cannot assume ethical and legal responsibility for their work. The same is true of every engineered system. Authorship is the assumption of that responsibility, and it belongs to the human.

Notation

Machine Learning Systems spans *machine learning* (computer science and statistics) and *systems* (computer architecture and hardware). Each field developed its notation independently, and many symbols mean different things across communities and publications. This collision creates real confusion when the disciplines merge. The conventions below establish a single notation that eliminates this ambiguity.

Consider a simple statement: “Increasing B improves throughput.” To an ML researcher, B means batch size. To a hardware engineer, B means bandwidth. Both interpretations are correct in their respective fields, but in ML Systems we need both concepts in the same equation—hence the need for a single, consistent convention.

The Iron Law of ML Systems

For serialized execution phases, the iron-law performance decomposition is:

$$T = \underbrace{\frac{D_{\text{vol}}}{\text{BW}}}_{\text{Memory Time}} + \underbrace{\frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}}_{\text{Compute Time}} + \underbrace{L_{\text{lat}}}_{\text{Latency Overhead}}$$

Each variable was chosen deliberately to avoid collision with standard ML terminology.

Symbol	Definition	Unit	Why This Symbol?
T	Time	seconds	Unambiguous. Wall-clock time for an operation.
D_{vol}	Data Volume	bytes	Avoids collision with D (Dataset Size). In scaling laws, D means training tokens. Here we need bytes moved through memory. The subscript disambiguates.
BW	Bandwidth	bytes/s	Avoids collision with B (Batch Size). Systems literature often uses B for bandwidth, while ML literature often uses B for batch size. The notation preserves the ML convention.
O	Operations	FLOPs	Total floating-point operations. Clean in equations (vs. “Ops”).
R_{peak}	Peak Rate	FLOP/s	Avoids collision with P (Parameters). Roofline presentations often use P for performance, while ML literature commonly uses P for parameter count. The notation preserves the ML convention.
η_{hw}	Efficiency	—	Hardware utilization ($0 \leq \eta_{\text{hw}} \leq 1$). Avoids collision with learning rate (η).
L_{lat}	Latency	seconds	Avoids collision with $\mathcal{L}$ (Loss). Fixed overhead time (kernel launch, network RTT). The subscript distinguishes from the loss function.

Why these choices matter

Without careful notation, sentences become ambiguous:

“Reducing D improves performance.”

The sentence has two possible readings:

- Reducing **dataset size** (fewer training samples) can speed training but may reduce accuracy.
- Reducing **data volume moved** (for example, through compression or quantization) can speed inference, with accuracy effects that depend on the technique and calibration.

With our notation, we can write precisely:

“Reducing D_{vol} through FP32-to-INT8 quantization cuts parameter memory traffic to one quarter while D (training data) remains unchanged.”

Our notation eliminates this ambiguity: “*BW limits throughput*” has only one reading.

Subscripted variants

An upright root names the quantity, and the subscript names which instance of it is meant. The convention holds closely related measurements apart instead of collapsing them into a single symbol:

- $BW_{\text{disk}}, BW_{\text{network}}, BW_{\text{accelerator}}$: bandwidth at three points on the data path
- $D_{\text{vol}}, R_{\text{peak}}, L_{\text{lat}}, \eta_{\text{hw}}$: iron-law terms, each held clear of a common ML symbol
- $E_{\text{move}}, E_{\text{compute}}, E_{\text{total}}$: one energy budget split into tradeable parts

The subscript therefore carries part of the claim. Disk and accelerator bandwidth differ by orders of magnitude, so a figure is reproducible only when the subscript says where it was measured.

The Degradation Equation

The degradation equation is a fitted local model. Divergence alone does not determine the direction of accuracy change, so estimate λ from labeled outcomes. Some symbols below, such as τ , appear in chapter prose rather than in the equation.

$$\text{Accuracy}(t) \approx \text{Accuracy}_0 - \lambda \cdot \mathcal{D}(P_t \| P_0)$$

Symbol	Definition	Unit/Type	Notes
$\text{Accuracy}(t)$	Accuracy at Time t	Scalar	Model accuracy after the model has been deployed for time t .
Accuracy_0	Initial Accuracy	Scalar	Model accuracy at deployment time.
λ	Fitted Sensitivity	Scalar	Local coefficient estimated from outcomes over a stated range; not a universal constant. (<i>Not wavelength.</i>)
P_t	Current Distribution	Distribution	The data distribution at time t . (<i>Not parameters—use P for parameter count.</i>)
P_0	Training Distribution	Distribution	The data distribution at training time.
$\mathcal{D}(P_t \ P_0)$	Statistical Divergence	Scalar ≥ 0	Measures how far P_t has drifted from P_0 . Common choices: KL divergence, total variation, Wasserstein. (<i>Calligraphic to avoid collision with D = dataset size.</i>)
τ	Response Threshold	Scalar > 0	Operational threshold for investigation; it does not automatically trigger retraining.

The Energy Corollary

With effective per-byte and per-operation costs, workload energy is approximated as:

$$E_{\text{total}} \approx D_{\text{vol}} \times E_{\text{move}} + O \times E_{\text{compute}}$$

Symbol	Definition	Unit	Notes
E_{total}	Total Energy	joules	Total energy consumed by an ML workload, decomposed into data-movement and compute terms.
E_{move}	Energy per Byte Moved	joules/byte	Effective movement cost; total energy also depends on bytes moved and operations executed.
E_{compute}	Energy per Operation	joules/FLOP	Energy cost of a single arithmetic operation.

Deep Learning Notation

This book follows standard deep learning conventions (Goodfellow et al. 2016) with explicit disambiguation for systems variables.

Symbol	Definition	Dimensions/Type
B	Batch Size	Integer. The number of samples processed in parallel. (<i>Never bandwidth.</i>)
P	Parameters	Integer. The total count of scalar model parameters, including weights and biases. (<i>Never peak FLOP/s.</i>)
D	Dataset Size	Integer. Number of training samples or tokens. (<i>Never data volume in bytes—use D_{vol}.</i>)
S	Sequence Length	Integer. Number of tokens or time steps.
d	Hidden Dimension	Integer. Size of the hidden state vector.
d_{head}	Attention Head Dimension	Integer. Per-head hidden dimension in attention layers.
N_L	Number of Layers	Integer. Total number of layers in a network.
N_{heads}	Number of Attention Heads	Integer. Number of attention heads in a multi-head attention layer.
H_{KV}	Number of Key-Value Heads	Integer. Number of key-value heads in grouped-query or multi-query attention.
ℓ	Layer Index	Integer. Index for a layer. Use instead of bare L when indexing layers, since L collides with loss and latency.
$\mathcal{L}$	Loss Function	Scalar. The objective function minimized during training.
η	Learning Rate	Scalar. Step size for the optimizer. (<i>Never bare for hardware efficiency—use η_{hw}.</i>)
θ	Model Parameters	Parameter collection, often treated as a vector. The set of all learnable parameters.
$p(x)$	Distribution	Probability mass/density for a random variable. Use lowercase $p(\cdot)$ for generic distributions to avoid collision with P (parameter count).
$p(y x)$	Conditional Distribution	Conditional probability/density. Use this form for generic label relationships; reserve P_0 and P_t for the degradation equation’s training/current distributions.
$\Pr(E)$	Event Probability	Probability of an event E . Use for event statements such as $\Pr(\text{batch} = 0)$; use $p(x)$ and $p(y x)$ for distributions.

Latin matrices and vectors are set in bold ($\mathbf{W}$, $\mathbf{x}$); scalars, dimensions, indices, and individual matrix entries stay italic (N , d , i , W_{ij}). The generic parameter symbol θ is the one conventional exception and stays italic. Local matrix algebra follows the same bold rule: matrix operands are bold ($\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$), as in general matrix multiply (GEMM) $\mathbf{C} = \alpha\mathbf{AB} + \beta\mathbf{C}$. The bold form marks a matrix operand and is distinct from the italic scalar batch size B , which remains batch size in scalar contexts.

Performance, Serving, and Memory Notation

Reusable performance and serving quantities follow the same collision-avoidance rule as the iron law. Rates use R or descriptive Greek symbols; request counts use Q_{req} rather than overloading N ; queue utilization uses a subscripted ρ so bare ρ remains available for other ratio models.

Symbol	Definition	Unit/Type	Notes
I	Arithmetic Intensity	FLOP/byte	Workload FLOPs per byte moved. The Roofline Model uses I as the independent variable.
I_{ridge}	Roofline Ridge Point	FLOP/byte	$I_{\text{ridge}} = R_{\text{peak}}/\text{BW}$. Prefer this explicit form over starred shorthand so the meaning remains clear in prose.
R_{attain}	Attainable Compute Rate	FLOP/s	Roofline bound: $R_{\text{attain}} \leq \min(R_{\text{peak}}, I \times \text{BW})$. Uses R , not T , because this quantity is a rate.
MFU	Model FLOPs Utilization	Dimensionless	Useful model FLOP/s divided by available peak FLOP/s. Text acronym avoids overloading η .
r_{comp}	Compression Ratio	Dimensionless	Uncompressed size divided by compressed size. A compressed payload has $1/r_{\text{comp}}$ of the uncompressed size; the subscript avoids bare C collisions.

Symbol	Definition	Unit/Type	Notes
Q_{req}	Request Concurrency	requests	Average in-flight requests in a stable serving system ($Q_{\text{req}} = \lambda_{\text{arr}} \cdot T_{\text{lat}}$ via Little's Law). Avoids collision with N as device count in distributed settings.
λ_{arr}	Arrival Rate	requests/s	Request arrival rate. The subscript avoids collision with λ as degradation sensitivity or failure rate.
T_{lat}	Request Time in System	seconds	End-to-end queueing/serving latency for Little's Law. Distinct from L_{lat} , the fixed-latency term in the iron law.
$T_{\text{svc}}(B)$	Batch Service Time	seconds	Time to serve a batch of size B .
$\mu_{\text{eff}}(B)$	Effective Service Rate	requests/s	Batched service rate, typically $\mu_{\text{eff}}(B) = B/T_{\text{svc}}(B)$.
ρ_{serv}	Serving Utilization	Dimensionless	Queue/server utilization. Use instead of bare ρ , which is reserved for communication-computation ratio in distributed contexts.
M_{total}	Total Memory Footprint	bytes	Sum of explicit memory components; avoids bare M ambiguity.
M_{weights}	Weight Memory	bytes	Memory occupied by model parameters.
$M_{\text{gradients}}$	Gradient Memory	bytes	Memory occupied by stored gradients.
$M_{\text{optimizer}}$	Optimizer-State Memory	bytes	Momentum, variance, master weights, and related optimizer buffers.
$M_{\text{activations}}$	Activation Memory	bytes	Retained activations for the backward pass or serving intermediates.
s_{elem}	Element Storage Size	bytes/element	Bytes per stored tensor element.

Units and Precision

- **Physical units:** This book uses SI (metric) units throughout, including meters, kilograms, seconds, watts, and °C, consistent with standard engineering and scientific practice. Where source data was originally reported in imperial units, the book converts to SI and notes the original values parenthetically. A space always separates the number from the unit (for example, 100 ms, 2 TB/s).
- **Data and memory:** This book uses **decimal SI prefixes only**: KB = 10^3 bytes, MB = 10^6 , GB = 10^9 , TB = 10^{12} . Binary-prefixed units do not appear in prose; all capacities, throughputs, and model sizes are reported in decimal units (for example, 80 GB, 2 TB/s, 102 MB).
- **Compute:** The notation distinguishes operation counts from rates. Total work uses FLOPs (for example, GFLOPs, TFLOPs), while throughput uses FLOP/s with decimal prefixes (for example, GFLOP/s, TFLOP/s).
 - 1 TFLOP = 10^{12} FLOPs
 - 1 TFLOP/s = 10^{12} FLOPs per second
 - Arithmetic intensity conventionally uses FLOP/byte as a unit ratio (floating-point operations per byte moved). This is a ratio unit, not a total-work symbol or throughput symbol.
- **Currency:** Dollar amounts use the dollar sign (\$); unless otherwise noted, dollar-denominated costs are U.S. dollars (USD).
- **Precision:**
 - **FP64:** Double precision (8 bytes)
 - **FP32:** Single precision (4 bytes)
 - **TF32:** TensorFloat-32 (19-bit Tensor Core compute mode; inputs and storage remain FP32)
 - **FP16:** Half precision (2 bytes, standard range)
 - **BF16:** Brain float (2 bytes, wide dynamic range)
 - **FP8:** Quarter precision (1 byte, E4M3 or E5M2 format)
 - **FP4:** 4-bit floating-point format
 - **INT8:** 8-bit integer (1 byte)
 - **INT4:** 4-bit integer; lower integer precisions follow the same uppercase INTn pattern (for example, INT3, INT2)

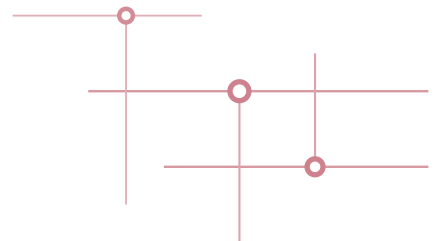
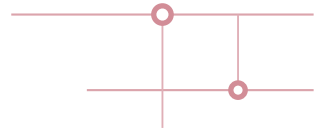
Quick Reference: Resolving Collisions

Common collision points in ML Systems literature include:

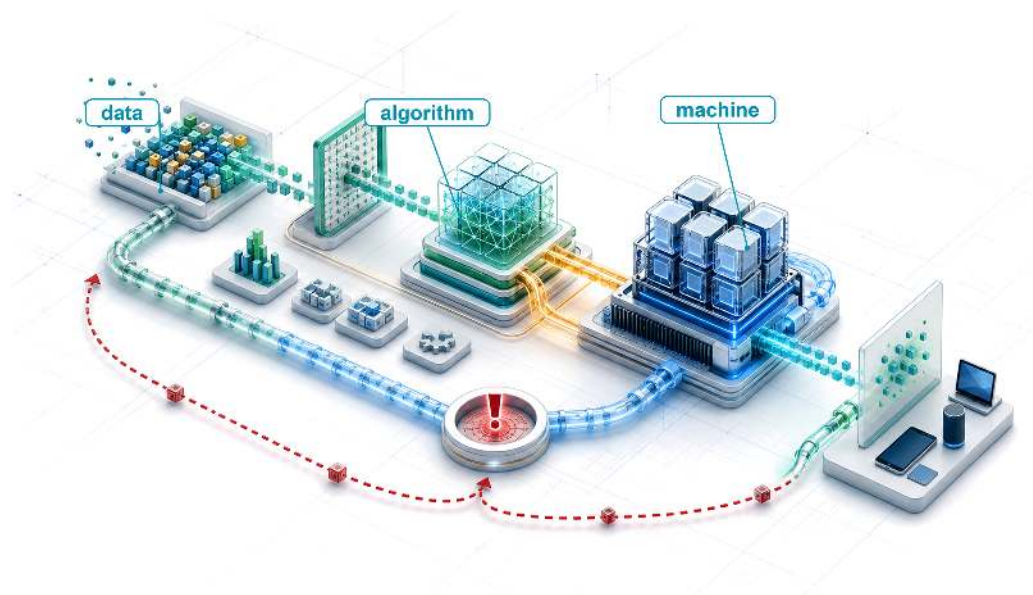
Symbol	ML Meaning	Systems Meaning	Book Convention
B	Batch Size	Bandwidth	Batch Size. Use BW for bandwidth.
P	Parameters	Peak FLOP/s	Parameters. Use R_{peak} for peak rate.
D	Dataset Size	Data Volume	Dataset Size. Use D_{vol} for bytes moved.
L	Loss	Latency	Loss ($\mathcal{L}$). Use L_{lat} for latency.
η	Learning Rate	Efficiency	Learning Rate. Use η_{hw} for efficiency.

As a general principle, ML conventions take precedence for single letters, while systems concepts get subscripts or multi-letter symbols. This reflects the primary audience (ML practitioners learning systems) and preserves compatibility with the vast ML literature.

MAIN TEXT



Introduction

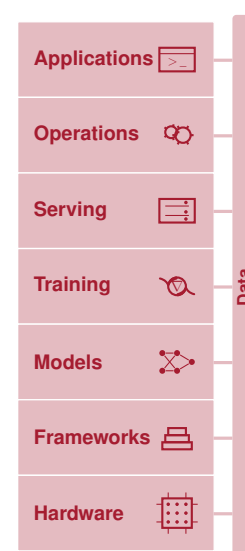


- 1.1 The AI Systems Moment
- 1.2 Data-Centric Paradigm Shift
- 1.3 The Evolution of AI Bottlenecks
- 1.4 The Bitter Lesson
- 1.5 Defining ML Systems
- 1.6 ML vs. Traditional Software
- 1.7 Iron Law of ML Systems
- 1.8 Three Dimensions of ML Efficiency
- 1.9 AI Engineering as a Discipline
- 1.10 Five-Pillar Framework
- 1.11 Book Organization
- 1.12 Fallacies and Pitfalls
- 1.13 Summary

Purpose

Why does building machine learning systems require engineering principles so different from those governing traditional computing systems?

Machine learning systems have a physics. Data moves through memory hierarchies governed by bandwidth, arithmetic runs on silicon governed by power, and predictions must arrive within latency windows. These constraints are not implementation details; they shape decisions from model architecture to deployment target. ML systems also differ from traditional computing systems because behavior is defined by data, not only by explicit logic or hardware state. When a conventional program misbehaves, engineers can often trace source or inspect hardware state; when an ML system misbehaves, the code may execute correctly while learned behavior fails because the data was incomplete, biased, stale, or no longer representative. ML engineers therefore manage statistical uncertainty and physical execution constraints together. A model that fits in a data center may be useless on a phone; a training pipeline that converges in a week on one accelerator may take a month on another; an accurate model trained on last year's data may silently degrade. Traditional practices such as testing, modularity, version control, and performance analysis remain necessary, but they are not sufficient. At system scale, an optimization in one layer can move the bottleneck to another, so correctness, efficiency, and deployability cannot be designed independently. The first task is therefore diagnosis, because improving the most visible component may leave end-to-end behavior unchanged when another constraint is binding. This book builds a discipline grounded in computation's physical limits. Algorithmic choices affect the stack down to the machine, and hardware constraints flow back up to model design.





Learning Objectives

- Explain why data-defined behavior and physical constraints distinguish ML systems from traditional software
- Apply a data-algorithm-machine lens to diagnose bottlenecks across data movement, arithmetic, and machine limits
- Analyze AI's shift from symbolic rules to deep learning through the bitter lesson
- Calculate iron-law performance terms to reason about throughput, latency, and return on compute
- Synthesize lifecycle, deployment, degradation, and five-pillar perspectives into ML systems engineering judgments

1.1 The AI Systems Moment

ARTIFICIAL intelligence is no longer confined to research demonstrations. Ask a smartphone a question and, within seconds, learned components convert speech to text, interpret intent, retrieve information, and generate a response. Search engines rank results, recommendation systems decide what people see, lenders use models to assess risk, and driver-assistance systems detect hazards. In each case, a prediction participates in a larger decision that can affect attention, money, access, or safety. What appears to be one intelligent action is therefore an end-to-end system operating in the world.

The modern AI movement became practical when three forces converged: digital services generated more examples than engineers could encode as rules, learning algorithms extracted predictive structure from those examples, and parallel hardware made the resulting computation feasible. Yet the most consequential change was conceptual. Data stopped being merely an input to software; it became the mechanism defining program behavior. Instead of hand-coding decision logic, engineers construct an optimization pipeline through which empirical examples shape the rules a system executes.

This convergence creates a **dual mandate**. Every ML system must establish that its learned behavior is trustworthy and that the machine can produce that behavior within the available time, memory, energy, and cost. A model that is accurate but too slow is unusable; a fast model trained on unrepresentative data is wrong at machine speed. The distinction becomes clearest at failure boundaries. A code defect may crash loudly, while a data defect can leave every instruction executing correctly as predictions deteriorate silently. At scale, both obligations span the entire stack. Conversational services coordinate pools of GPUs¹ while managing memory, networks, and heat. Driver-assistance systems must fuse sensor streams within milliseconds. Google processes 8.5B searches per day under strict latency targets. These systems succeed only when learned behavior and physical execution are engineered together. That co-design begins with the shift from code-defined logic to data-defined behavior.

1.2 Data-Centric Paradigm Shift

That shift changes how software is built. Instead of writing behavior directly, engineers construct a process that learns behavior from data. When a traditional program fails, an engineer can often trace a branch, inspect a stack frame, and patch the code path. When an ML system loses accuracy without a code change, the cause may be a shifted data distribution, a changed label process, or a model that no longer represents production behavior. Andrej Karpathy² described this change as the move from **Software 1.0** to **Software 2.0** (Karpathy 2017). As table 1.1 shows, Software 1.0 encodes operational logic in instructions, while Software 2.0 learns that logic from examples. Its failures can therefore remain silent until evaluation or monitoring reveals that behavior has changed.

Software 2.0 does not eliminate code. Engineers still build data pipelines, training loops, evaluation tools, and serving infrastructure. What changes is where application behavior lives. Some of it resides in learned weights shaped by data, so code review alone cannot explain what the system will do.

¹ | **GPU (graphics processing unit):** Originally designed for rendering video game graphics, a workload requiring thousands of simple, parallel pixel calculations. This hardware-algorithm alignment proved decisive for neural networks, where the same massively parallel arithmetic structure maps directly onto matrix multiplication, making GPUs a primary physical enabler of modern training scale.

² | **Andrej Karpathy:** A founding member of OpenAI and former Director of AI at Tesla who pioneered the application of deep learning to autonomous vehicle fleets. His “Software 2.0” thesis (2017) crystallized the insight that neural network weights are the new “source code,” forcing a new engineering reality: instead of debugging explicit logic, engineers must curate and version the data that defines program behavior, since a model with millions of parameters cannot be patched or reasoned about directly.

Table 1.1: The Paradigm Shift from Software 1.0 to Software 2.0: In Software 2.0, the “programmer” does not write the logic; they curate the dataset that the optimization process uses to write the logic. Debugging therefore moves upstream from code to data. The “compiler” analogy is approximate: unlike a deterministic compiler, the training process is stochastic and may produce different “executables” from the same “source code.”

Feature	Software 1.0 (Traditional)	Software 2.0 (Machine Learning)
Source Code	C++, Python, Java	Training Data + Labels
Compiler	GCC, LLVM	Training loop (stochastic gradient descent)
Logic	Explicit (Hand-coded)	Implicit (Learned)
Failure Mode	Loud (Crash, Exception)	Silent (Metric Degradation)
Debugging	Trace execution path	Inspect data distribution

The data-centered workflow changes what engineers must build and maintain. In a study of production systems at Google, Sculley and colleagues found that model code occupied only a small fraction of the engineering surface (Sculley et al. 2015). The surrounding system—data collection, verification, feature extraction, resource management, serving, and monitoring—was larger and more enduring.

That imbalance creates hidden technical debt. Each surrounding component encodes assumptions about how an example is sampled, what a label means, when a feature is computed, which version reaches serving, and how degradation is detected. None of those assumptions appears in the matrix multiplication that produces a prediction, yet any of them can change the result. Improving the model alone cannot repair a stale proxy, a broken feature pipeline, or a missing feedback signal.

Production data is therefore not a passive input. It is a measurement of the world, collected through a particular product and transformed by a changing pipeline. If that measurement stops representing the intended target, the model can remain numerically healthy while the system becomes wrong. Google Flu Trends provides a revealing example. It had an enormous, timely stream of search data, but the meaning of that data changed beneath the model.



Production ML work is mostly the surrounding system, not the model code alone.

⚠ War Story 1.1: When search logs mistook attention for illness (2014)

Context: Google Flu Trends (GFT) estimated influenza activity from patterns in aggregated search queries (Ginsberg et al. 2009; Lazer et al. 2014). Its attraction came from the speed and scale at which searches arrived relative to clinical reports. If people searched for flu symptoms when they became ill, query volume appeared to offer a timely proxy for infection.

Failure mode: The proxy was not stable. News coverage changed what people searched for, and autocomplete changed how they expressed those searches. Query volume began to measure public attention and product behavior as well as illness. During the 2012–2013 season, GFT estimated roughly twice the CDC-reported proportion of doctor visits for influenza-like illness and overestimated for 100 out of 108 weeks.

Systems lesson: The remedy was not simply a larger search dataset or a better-tuned model. Researchers combined search signals with CDC sentinel clinical data and recalibrated the relationship over time. Data volume is not ground truth: a behavioral proxy needs a feedback loop to a trusted measurement and continual evidence that the proxy still represents the quantity the system claims to estimate.

Google Flu Trends failed without a conventional software defect. The code did not change, but the effective program did because the distribution of inputs and the relationship between proxy and target had changed. Under the **data-as-code principle**, training data does not merely enter a fixed program; it helps determine the operational logic that the model implements. Engineers still write the optimization procedure, but the examples shape the model weights³ through stochastic gradient descent⁴ and related methods. Changing the dataset can therefore change system behavior as surely as changing source code.

From an ML development perspective, this represents a transition from *model-centric* to *data-centric* AI (Ng 2021). In a model-centric approach, teams hold the data fixed and focus on improving model

³ | **Model weights:** The learned numerical parameters of a neural network. A GPT-3-scale (Generative Pre-trained Transformer 3) model stores 175B such values, consuming 350 GB in FP16 precision, a 16-bit floating-point format that uses two bytes per value (Brown et al. 2020). Parameter count determines the weight footprint and strongly influences serving memory traffic and cost (see chapter 5).

⁴ | **Stochastic gradient descent (SGD):** The algorithm learns model parameters by processing small, randomly sampled groups of examples (“batches”) rather than the entire dataset at once. This trades statistical noise for computational speed. Batch size also affects the machine because a batch that is too small may fail to saturate an accelerator’s parallel processors, wasting much of its potential computation.

code. In a data-centric approach, they hold the code comparatively fixed and systematically improve the data, making data curation a first-class part of programming model behavior. The shift also changes what testing can establish because no finite dataset can represent every input that a learned system may encounter.

Software 1.0 logic can often be partitioned into execution paths and boundary conditions. A learned system instead operates across a high-dimensional input space that is technically finite but impossible to enumerate in practice. The 2012 ImageNet challenge makes this problem concrete. AlexNet’s decisive win⁵ helped set the deep learning era in motion, yet the benchmark could evaluate only a vanishing fraction of the inputs the model might encounter. Each input is a 224×224 RGB image with $256^{150,528}$ possible pixel configurations, a number with 362,508 digits. The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) validation set contains only 50,000 images (Krizhevsky et al. 2012; Russakovsky et al. 2015). Let **Total Input Space** denote the number of possible inputs and **Test Set Coverage** the number that a test suite actually evaluates. Their disparity creates the **verification gap** (equation 1.1):

$$\text{Verification Gap} = \text{Total Input Space} - \text{Test Set Coverage} \approx \text{Total Input Space} \quad (1.1)$$

The gap does not make testing futile; it changes what testing can establish. Predeployment evaluation provides statistical evidence over sampled inputs, while production monitoring tests whether the operating population and observed outcomes still support that evidence. Statistical reliability replaces any expectation of exhaustive proof.

The verification gap marks a deeper change in what engineers can claim. Traditional assertions often describe a particular execution in which a given input and state lead the program down a specific path. ML performance claims describe behavior over a population. A fixed model may return the same output for the same input while its measured accuracy changes as the population changes. Data supplies the patterns from which the system learns, but its noise, drift, and omissions also create uncertainty. Robustness therefore cannot mean resisting every change. It requires making change observable and adapting when the evidence no longer supports the system’s assumptions, a form of **probabilistic engineering**.

The engineering consequence reaches beyond testing. Debugging an ML system requires debugging the data, not the Python scripts alone. Version control must track datasets, not git commits alone. Evaluation must examine distributions and outcomes, not code paths alone. Together, these practices make learned behavior reproducible and traceable, but they cannot turn statistical sampling into formal verification.



Checkpoint 1.1: The paradigm shift

Before tracing the history of AI, verify your understanding of the paradigm shift in how we build software:

- ☐ **Software 1.0 vs. 2.0:** can you distinguish Software 1.0 (explicit instructions) from Software 2.0 (optimization objectives)?
- ☐ **Debugging shift:** can you identify which dataset evidence to inspect when accuracy falls without a code change?
- ☐ **Verification gap:** can you explain why finite test coverage cannot establish statistical reliability across the full production input distribution?

Learning behavior from data can seem inevitable in hindsight, but it was an engineering pivot forced by hard scaling limits. Earlier AI paradigms attempted to program intelligence directly through hand-crafted symbols and rules. Tracing those failures reveals how the binding constraint migrated across four distinct eras—from logic, to knowledge, to features, and ultimately to physical infrastructure.

1.3 The Evolution of AI Bottlenecks

Every major era in artificial intelligence collided with a hard systems limit long before its algorithmic ideas were exhausted. When Alan Turing⁶ proposed evaluating machine intelligence by operational

⁵ | **AlexNet:** Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton trained this convolutional neural network across two GPUs. At ILSVRC 2012, its 15.3 percent top-5 error, meaning the correct class was absent from its five highest-scoring predictions, substantially beat the second-place system’s 26.2 percent (Krizhevsky et al. 2012). Section 1.3.3 returns to its architecture and systems co-design.

Input space

Test set

Test coverage is a vanishing fraction of the input space.

⁶ | **Alan Turing:** His 1950 “Imitation Game” reframed intelligence as an output-measurement problem: judge a system by what it *does*, not by what it *is*. This engineering-first stance persists in every ML systems metric we use today: accuracy, latency, throughput, and FLOP/s per watt are all output measurements. The iron law (section 1.7) decomposes performance into observable, measurable terms rather than internal architectural properties for exactly this reason.

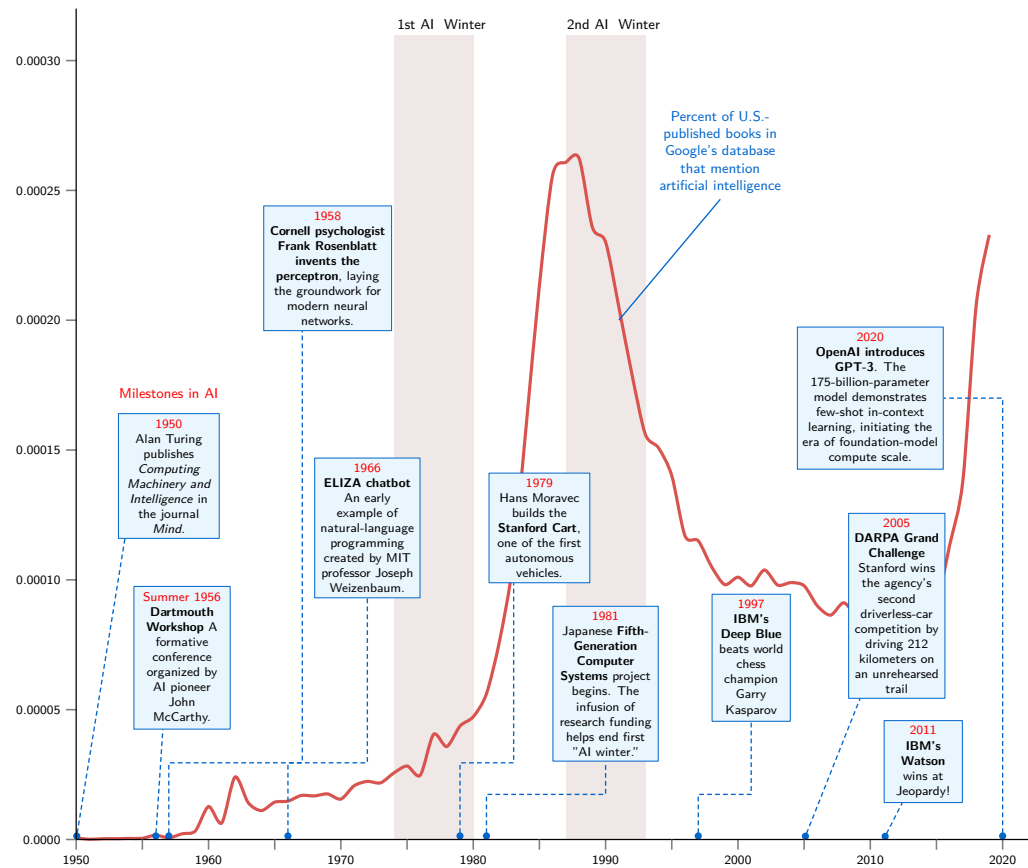


Figure 1.1: AI Development Timeline: A chronological proxy curve traces historical mentions of artificial intelligence in published literature (Michel et al. 2011), with light-brown bands marking the two AI Winter periods (1974–1980, 1987–1993). Callout boxes mark Turing’s 1950 paper (Turing 1950), the Dartmouth conference (McCarthy et al. 1955), the Perceptron (Rosenblatt 1958), ELIZA (Weizenbaum 1966), Deep Blue (Campbell et al. 2002), and GPT-3 (Brown et al. 2020).

output rather than internal essence (Turing 1950), computing machinery lacked the memory capacity to store real-world knowledge and the memory bandwidth to evaluate complex models. Early systems split along a fundamental fault line: Frank Rosenblatt’s Perceptron (1958) (Rosenblatt 1958) attempted to learn weights from sensory inputs, while Joseph Weizenbaum’s ELIZA⁷ (Weizenbaum 1966) executed hand-coded pattern substitutions. Both systems stalled against physical realities: single-layer perceptrons lacked multi-layer training mechanisms and non-linear representations, while rule-based scripts failed when user input deviated from pre-scripted templates. Subsequent eras hit the knowledge acquisition bottleneck because manual knowledge entry could not scale. Modern systems face a different constraint in computational throughput.

The timeline in figure 1.1 traces how often artificial intelligence is mentioned in published books, a proxy for attention rather than a direct measure of research output. It reveals a recurring pattern of intense optimism followed by “AI winters”⁸ when funding collapsed, often after systems limitations exposed a gap between ambition and available capability. Resurgences combined algorithmic advances with new data and engineering infrastructure. Each resurgence displaced one bottleneck and exposed the next.

1.3.1 The prelearning era: Logic and knowledge bottlenecks

Before machine learning existed as a discipline, engineers attempted to build intelligent systems through two successive paradigms, each of which hit a fundamental scaling barrier. Symbolic AI encoded intelligence as logical rules and hit the *logic bottleneck* when those rules could not capture real-world ambiguity. Expert systems encoded intelligence as domain knowledge and hit

⁷ | **ELIZA:** A 1966 natural-language program using pattern-matching rules rather than learned parameters. Its scripts could store and retrieve selected inputs, but this limited hand-written memory did not provide learned conversational state. Every new input variation required another rule, making maintenance grow faster than capability and foreshadowing the knowledge bottleneck that constrained later expert systems.

8 | AI winters as systems failures:

The first AI winter (1974–1980) unfolded amid funding cuts; the 1973 Lighthill Report criticized the gap between AI promises and delivered results (Lighthill 1973). The second winter (1987–1993) involved a market and funding collapse around expert systems and specialized Lisp machines as general-purpose workstations undercut their economics (Hendler 2008). From this book's systems perspective, both episodes expose algorithm ambition outrunning available infrastructure, market support, and engineering maturity, not merely a shortage of clever algorithms.

9 | Dartmouth Conference (1956):

The workshop organized around the term “artificial intelligence,” already used in its 1955 proposal (McCarthy et al. 1955). Its participants framed intelligence in terms of language, abstraction, problem solving, and self-improvement, with little attention to the physical constraints of storage and compute that later became central. The same compute-agnostic assumption, that a better algorithm could always overcome a hardware limit, is precisely what this book exists to correct: every chapter that follows argues that systems constraints are first-class design variables, not afterthoughts.

10 | STUDENT:

Daniel Bobrow's 1964 MIT program parsed constrained English word problems, represented their relationships symbolically, and passed the resulting equations to an algebra solver. It was an early separation of language interpretation from formal reasoning: once the representation was correct, solving was straightforward; coverage depended on hand-written transformations (Bobrow 1964).

the *knowledge bottleneck* when acquiring and maintaining that knowledge became more expensive than the systems were worth. Both failures expose the same architectural dead end: hand-crafted representations do not scale.

1.3.1.1 The symbolic AI era and the logic bottleneck

The first era of AI engineering (1950s–1970s) attempted to reduce intelligence to **symbolic AI** manipulation, an approach later crystallized in the physical-symbol-system hypothesis (Newell and Simon 1976). Researchers at the 1956 Dartmouth Conference⁹ (McCarthy et al. 1955) hypothesized that aspects of intelligence could be precisely described and simulated by machines. Even then, Arthur Samuel at IBM demonstrated a different path in 1959 when a checkers program improved through self-play. His work coined the term “**machine learning**” (Samuel 1959), though the dominant paradigm remained symbolic. Daniel Bobrow's *STUDENT*¹⁰ system exemplifies this approach (Bobrow 1964).

These systems could produce impressive demonstrations, yet they were operationally brittle. Their success depended on manually coded rules that mapped each acceptable input form to a symbolic representation. *STUDENT* could solve an algebra problem once it translated English into equations, but a minor variation in phrasing could prevent that translation. The difficulty extended beyond language. Hans Moravec's¹¹ work on autonomous navigation at Stanford revealed that tasks humans find trivial (seeing, walking, grasping) were far harder to engineer than tasks humans find difficult, like chess or algebra.

Example 1.1: *STUDENT* (1964)

STUDENT separated language interpretation from algebra:

```
language  "Two numbers sum to 30; one is twice the other."
parse     x + y = 30, x = 2y
solve     x = 20, y = 10
```

Once the representation was correct, solving was routine. The bottleneck was the translation: each unfamiliar phrasing needed another hand-written rule, so linguistic coverage grew only as fast as the rule base.

1.3.1.2 The expert systems era and the knowledge bottleneck

In the expert-systems era, engineers narrowed the problem. Instead of constructing general reasoning systems, they encoded deep knowledge from a specific domain. *MYCIN*, designed to diagnose blood infections, let medical knowledge be expressed as production rules (Shortliffe et al. 1975).

Example 1.2: *MYCIN* (1976)

MYCIN made its reasoning explicit. Patient facts triggered readable production rules:

```
facts  stain=gram-positive, shape=coccus, arrangement=clumps
rule   IF facts match THEN organism=staphylococcus (certainty=0.7)
cycle  match facts -> fire rule -> update certainty -> repeat
```

An inference engine repeated this cycle across hundreds of rules and propagated certainty factors toward a diagnosis. A physician could inspect every step, but coverage still grew one rule at a time: each exception had to be elicited, encoded, and reconciled with the existing rule base.

MYCIN performed well in specific tests, but its success exposed the **knowledge acquisition bottleneck**.¹² The problem was no longer whether a rule could express expertise. It was how tacit judgment entered the system and remained consistent as the rule base grew.

Faster machines could evaluate more rules, but they could not extract human judgment faster. Scalable AI needed a way to infer useful decision boundaries from examples rather than requiring engineers to enumerate those boundaries in advance.

That change did not remove human design; it changed what engineers designed. Instead of encoding every decision, they chose the examples, representations, objectives, and measurements from which a decision could be learned. The next era therefore exchanged the knowledge-acquisition bottleneck for a new question: which evidence should the learner see?

1.3.2 The statistical learning era and the feature engineering bottleneck

The 1990s marked the shift to **statistical learning** and probabilistic systems. Instead of hard-coded logic, systems estimated probabilities from data ($p(y | x)$). This transition was driven by the availability of digital data and the “unreasonable effectiveness”¹³ of large datasets.

Spam filtering illustrates this shift. Rather than maintaining lists of forbidden words, statistical filters learned the probability that a word implies spam based on millions of examples.

Example 1.3: Early spam detection systems

Statistical filters shifted the decision from a hand-written rule to a probability estimated from labeled messages:

```
rules: if "free" or "winner" appears -> spam
learn: labeled email -> word likelihoods -> P(spam | email)
```

Maintenance did not disappear. It moved from editing keyword lists to curating representative examples and checking whether the learned probabilities still tracked current email.

Learning the decision boundary from data removed one bottleneck but exposed another. Statistical algorithms such as Support Vector Machines (SVMs) could learn robustly only after humans converted raw inputs into structured features. In computer vision, for example, engineers spent years designing manual mathematical transformations such as SIFT (Scale-Invariant Feature Transform) and HOG (Histogram of Oriented Gradients) to extract edges, corners, and texture summaries before feeding them to a classifier. The learner adjusted the boundary; engineers decided which evidence it could see. Scaling to a new problem often meant rebuilding the preprocessing stack, turning an apparent algorithm limitation into the **feature engineering bottleneck**. The traditional pipeline makes this manual effort visible because several hand-crafted stages preceded any learning at all.

This hybrid approach combined human-engineered features with statistical learning. The Viola-Jones algorithm¹⁴ (Viola and Jones 2001) exemplifies this era, achieving real-time frontal-face detection using simple rectangular features and cascaded classifiers. It showed that well-engineered features could enable practical low-latency applications, but only within narrow domains where experts could hand-craft the right representations.

Example 1.4: Traditional computer vision pipeline

A conventional vision system separated representation from classification:

```
image -> resize -> HOG/SIFT features -> SVM -> label
```

Only the final decision boundary was learned. Moving from faces to pedestrians or another domain meant redesigning the feature extractor and often the entire preprocessing stack.

1.3.3 The deep learning era and the infrastructure bottleneck

Deep learning changed which part of the system learned. Instead of receiving features designed by humans, neural networks learned representations directly from raw inputs such as pixels and audio waveforms. The training process could now shape both the representation and the decision boundary, enabling “end-to-end” learning.

¹¹ | **Moravec’s paradox:** Carnegie Mellon roboticist Hans Moravec observed that high-level reasoning (chess) requires little compute while low-level perception (walking) requires massive parallelism (Moravec 1988). This paradox explains a central fact of ML systems engineering: the tasks that seem “easy” to humans (vision, speech, motor control) are the ones that demand the highest FLOP/s, memory bandwidth, and specialized hardware, driving the accelerator revolution that defines modern ML infrastructure.

¹² | **Knowledge acquisition bottleneck:** Feigenbaum’s knowledge-engineering work framed applied AI around the practical difficulty of extracting, representing, and maintaining expert knowledge (Feigenbaum 1984). In systems terms, this bottleneck was a throughput problem: knowledge elicitation and rule maintenance were bound by the serial bandwidth of human experts. Unlike computational bottlenecks that yield to faster hardware, this one was the original “does not scale” constraint in AI and a direct motivation for the data-driven paradigm that followed.

¹³ | **Unreasonable effectiveness of data:** The observation that a simple statistical model fed with massive amounts of data can outperform a more sophisticated model with less data (Halevy et al. 2009). Halevy and colleagues illustrated the effect with large web corpora, not a universal error-rate multiplier; gains depend on the task, model, data quality, and starting point. The result supported the shift from brittle hand-crafted systems toward probabilistic models and made data collection, storage, preprocessing, and distributed training central engineering concerns.

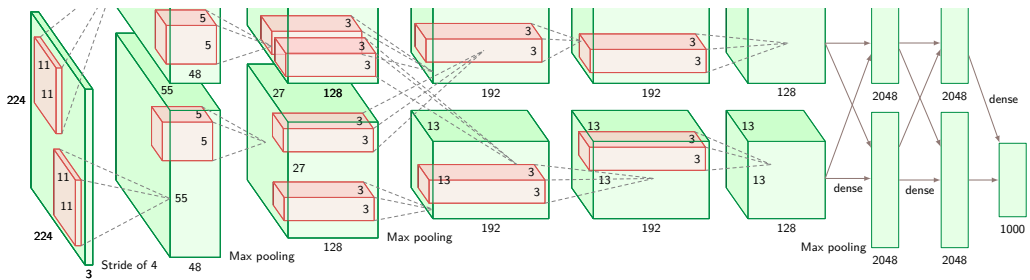


Figure 1.2: AlexNet Architecture: The network that launched the deep learning revolution at ImageNet 2012 (Krizhevsky et al. 2012). Two parallel GPU streams process 224×224 input images through convolutional layers (green blocks) that extract spatial features at decreasing resolutions, converging through three fully connected layers to 1,000 output classes. With 60 million parameters trained across two GTX 580 GPUs, AlexNet achieved 15.3 percent top-5 error, a 41.6 percent relative improvement over the second-place entry.

14 | Viola-Jones algorithm: The algorithm’s real-time speed came from a classifier cascade that used simple, hand-engineered rectangular features to immediately reject nonface regions. The method was designed and evaluated for frontal-face detection, illustrating the era’s trade-off: expert feature design could be fast and effective, but the representation was task-specific. The first two layers alone could discard over 80 percent of negative sub-windows while using just twelve of the 6,000+ total features (Viola and Jones 2001).

The breakthrough was not algorithmic alone. Convolutional neural networks (CNNs) existed earlier (LeCun et al. 1998, 2015); AlexNet paired architecture and training with **systems co-design**, choosing the model, training procedure, and hardware mapping together (Krizhevsky et al. 2012). Its parallel matrix operations matched GPU capabilities. With 60 million parameters distributed across two GTX 580 GPUs, AlexNet achieved 15.3 percent top-5 error, a 41.6 percent relative improvement over the next-best entry that year. The processing stages in figure 1.2 show the model learning progressively richer image representations before producing one of 1,000 output classes.

Hardware still shaped what could be learned. With only 3 GB of Video Random-Access Memory (VRAM) per GTX 580, AlexNet divided convolutional and dense layers between two GPU streams. This early model-parallel design foreshadowed modern multi-GPU training. Deep learning reduced the need for hand-crafted features, but it moved the binding constraint to the infrastructure required to store data, move parameters, and coordinate computation.

Deep learning effectively traded the feature engineering bottleneck for a new **compute bottleneck**. Models like GPT-3 (175 billion parameters) illustrate the scale of this new challenge. Brown et al. (2020) report training on about 300 billion tokens from filtered web text, books, and Wikipedia. Using the book’s dense-training approximation ($C \approx 6 \times N \times D$, accounting for 2 FLOPs per parameter on the forward pass and 4 on the backward pass, derived in chapter 8), that parameter-token scale implies roughly 314 zettaFLOPs of compute (1 zettaFLOP = 10^{21} FLOPs). The token dataset itself occupies roughly 420 GB. Because the original paper does not specify the exact hardware cluster, translating this compute budget into hardware time represents roughly 350 to 400 accelerator-years on contemporary cloud GPUs (e.g., NVIDIA V100s), serving as an illustrative systems estimate rather than a recorded benchmark. The primary engineering challenge shifted from describing a cat’s ear to coordinating large-scale distributed training without failure.

Each major transition in AI history traces back to a shifting physical constraint rather than algorithmic invention alone. Table 1.2 compares the four major eras across their core reasoning strengths, binding systems bottlenecks, and operational data requirements.

Table 1.2: AI Paradigm Evolution: Each era is defined by the systems bottleneck that constrained it. Deep learning (far right) overcame the Feature Engineering bottleneck but introduced new infrastructure challenges, necessitating modern ML systems engineering.

Aspect	Symbolic AI	Expert Systems	Statistical Learning	Deep Learning
Key Strength	Logical reasoning	Domain expertise	Versatility	Pattern recognition
Bottleneck	Brittleness (Rules break)	Knowledge Entry (Experts are scarce)	Feature Engineering (Manual preprocessing)	Compute & Data Scale (Infrastructure cost)
Data Handling	Minimal data needed	Domain knowledge-based	Moderate data required	Massive data processing

1.4 The Bitter Lesson

Expert systems invested engineering effort in encoding domain knowledge; deep learning systems invest that effort in absorbing more data and computation. The **bitter lesson** captures the historical pattern in which general methods that use increasing computation consistently outperform approaches that encode human expertise. Richard Sutton¹⁵ crystallized this insight in his 2019 essay “The Bitter Lesson” (Sutton 2019). Sutton wrote, “The biggest lesson that can be read from 70 years of AI research is that general methods that leverage computation are ultimately the most effective, and by a large margin.”

Table 1.3 pairs representative benchmark milestones with their hardware substrates. Its final column traces the progression from single-threaded CPU rule evaluation to multi-thousand-accelerator clusters, using 2.5 million reference GPU-days as an illustrative frontier-training anchor (Patel and Wong 2023).

Table 1.3: AI Performance Evolution Across Paradigms: Representative systems illustrate the growth in computational scale across eras. GPT-4 training details are estimated reference anchors rather than official disclosures.

Era	Approach	Representative Task	Performance	Computational Resources
Expert Systems (1980s)	Hand-crafted rules	Chess (Elo rating)	System-dependent	Minimal (rule evaluation)
Statistical ML (1990s–2000s)	Feature engineering + learning	Handwritten digit recognition	about 98–99% on MNIST-era benchmarks (LeCun et al. 1998)	CPU-era feature pipelines; resources varied by implementation
Deep Learning (2012)	End-to-end neural networks	ImageNet top-5 accuracy	84.7% (AlexNet)	6 days on 2 GPUs
Modern Deep Learning (2020+)	Large-scale transformers	ImageNet top-1 accuracy	88.55% (ViT-H/14) (Dosovitskiy et al. 2021)	Large-scale Tensor Processing Unit (TPU) pretraining
Modern Deep Learning (2023)	Foundation models	MMLU benchmark	86.4% (GPT-4) (OpenAI et al. 2023)	Estimated ~2.5 million reference GPU-days (on the order of 25,000 reference GPUs run for 90 days) (Patel and Wong 2023)

ImageNet top-5 accuracy and Massive Multitask Language Understanding (MMLU) (chapter 12) measure distinct capabilities, but share a systems trajectory. Hardware scale expanded from single-node CPU execution to multi-megawatt clusters, corroborating Sutton’s observation that methods designed to use raw computation consistently outpace hand-tuned representations.

Earlier milestones in automated game playing exhibited the same hardware-driven progression. In chess, IBM’s Deep Blue defeated world champion Garry Kasparov¹⁶ in 1997 by combining custom chess hardware, large-scale search, and chess-specific evaluation knowledge. Its evaluation function encoded human chess heuristics, but the scale of search enabled by custom silicon was central to turning that knowledge into championship-level play. In Go, DeepMind’s AlphaGo¹⁷ (Silver et al. 2016) achieved superhuman performance by combining supervised learning from expert games with reinforcement learning through self-play and neural-network-guided tree search, rather than relying on hand-coded Go strategy.

The lesson is “bitter” because our intuition misleads us. We naturally assume that encoding human expertise should be the path to artificial intelligence. Yet repeatedly, systems that use computation to learn from data outperform systems that rely on human knowledge given sufficient scale. The pattern has held across symbolic AI, statistical learning, and deep learning eras.

Modern language models such as GPT-4 and image generation systems such as DALL-E illustrate this principle directly. Their capabilities emerge not from linguistic or artistic theories encoded by humans but from training general-purpose neural networks on vast amounts of data using substantial computational resources. Estimates for models at GPT-3’s scale suggest roughly 1.3 GWh of energy¹⁸ (Patterson et al. 2021), and serving these models to millions of users turns inference into a continuous data-center power, cooling, and capacity-planning problem.

Realizing the bitter lesson in practice demands an engineering discipline that extends far beyond algorithmic design. When model capability scales with raw computation, the binding limits become memory bandwidth,¹⁹ interconnect latency, thermal dissipation, and data pipeline throughput

¹⁵ | **Richard Sutton:** A reinforcement learning pioneer whose 2019 essay crystallized the pattern traced in section 1.3: from symbolic AI through expert systems to deep learning, general methods using computation consistently outperformed hand-engineered expertise. The lesson is “bitter” because it implies that domain-specific logic is a depreciating asset, while the durable advantage belongs to systems engineering that can absorb the billion-fold increase in raw compute since the 1970s.

¹⁶ | **Deep Blue:** IBM’s chess system (Campbell et al. 2002) defeated World Champion Garry Kasparov in 1997 through a systems combination: search at roughly 200 million positions per second on 480 custom chess processors, plus chess-specific evaluation and knowledge. Deep Blue was an early public demonstration that purpose-built silicon could amplify search and encoded domain knowledge, foreshadowing the domain-specific accelerator strategy that defines modern ML hardware.

¹⁷ | **AlphaGo:** AlphaGo first learned from human expert games, then improved through reinforcement learning from self-play, trading hand-coded Go strategy for a data-and-compute pipeline that could explore the problem space at massive computational scale. After three days of self-play training, AlphaGo Zero surpassed the original AlphaGo, winning 100 games to 0 (Silver et al. 2017).

¹⁸ | **GPT-3 (Generative Pre-trained Transformer 3) training energy:** Patterson et al. (2021) estimated GPT-3’s single training run consumed approximately 1,287 MWh and emitted 552 tonnes of CO₂-equivalent, roughly the annual electricity of 120 average US households using a 10.7 MWh/household-year baseline. Data movement through the memory hierarchy accounts for a substantial fraction of this energy footprint, as moving operands across memory levels consumes orders of magnitude more energy than local arithmetic operations (Horowitz 2014).

19 | Memory bandwidth:

The rate at which data transfers between off-chip storage (such as high-bandwidth memory (HBM)) and on-chip processor registers or SRAM. While large-batch training amortizes parameter transfers over thousands of tokens to achieve high arithmetic intensity, autoregressive inference must reload billions of weights from off-chip memory for every generated token. Because off-chip memory transfers consume orders of magnitude more energy and latency than on-chip compute, memory bandwidth—rather than peak arithmetic throughput—frequently dictates execution latency and data-center power draw.

(chapter 11).

Scaling computation transforms the machine learning model into a physical systems artifact: one governed by power envelopes, memory hierarchies, and communication overhead. Engineering these systems requires treating data movement, arithmetic execution, and operational reliability as a unified problem. That broader responsibility is AI engineering, developed formally in section 1.9. Grounding that discipline requires defining its core object through the operational realities of a concrete production service.

1.5 Defining ML Systems

Return to the spam filter introduced in the history of statistical learning. At production scale, the same apparently simple classifier operates against global email traffic measured in hundreds of billions of sent and received messages per day (Statista Research Department 2024), and large providers must decide in milliseconds which messages deserve attention and which should be quarantined.

This deceptively simple task reveals *what* distinguishes machine learning systems from traditional software. The challenge begins with data. The filter trains on millions of labeled examples and must keep adapting as spammers evolve their tactics, rather than relying on programmers to encode every spam pattern manually. It then becomes an algorithmic problem, because the model must generalize from those examples to messages it has never seen before while balancing precision against recall so legitimate email is not hidden. Finally, the same decision becomes an infrastructure problem. Providers must process billions of emails daily, store and update models as spam evolves, and serve predictions with sub-100 ms latency across horizontally scaled data centers. The classifier is therefore only one component of an evolving data, software, and hardware system.

When a new phishing template appears, the data layer must capture representative messages, the algorithm must separate attacks from legitimate mail, and the machine must distribute updated parameters before delivery. Any layer can fail while the others continue working, so the full chain from measurement through learning to execution is the right unit of analysis.



Definition 1.1: Machine learning systems

Machine learning systems are software systems whose core behavior is determined by parameters learned from data rather than explicitly programmed rules, making performance a function of data quality, algorithm choice, and hardware capacity simultaneously.

1. **Significance:** Requirements are end to end. A spam filter is useful only when its training data represents current attacks, its model separates malicious from legitimate mail, and its serving path classifies each message within the delivery budget. Failure in any layer changes the user-visible result.
2. **Distinction:** Unlike traditional software, an ML system's accuracy can change when the world changes even if its code and weights do not. The production input distribution may move relative to what the model learned, silently changing performance without an error or exception.
3. **Common pitfall:** The model is not the system. Data pipelines, feature transformations, serving infrastructure, monitoring, and feedback loops surround the learned parameters and often dominate the engineering burden (Sculley et al. 2015).

The definition exposes three diagnostic axes: the data that describes the task, the algorithm that turns those examples into behavior, and the machine that executes that behavior within the operating budget. The D·A·M taxonomy gives those axes a reusable form.

The same user-visible symptom can originate in any of the three. A missed phishing message may mean that representative examples were absent, that the learned decision boundary was inadequate, or that the serving path missed its latency budget. Treating every miss as a model problem risks improving the wrong component. A useful systems taxonomy must separate these causes without pretending that they are independent.

Each explanation calls for different evidence. The data hypothesis sends the engineer to coverage, labels, and distribution shift. The algorithm hypothesis sends the engineer to error slices, model capacity, and the learning objective. The machine hypothesis sends the engineer to latency, throughput, memory, and utilization. These investigations are not interchangeable. Faster hardware cannot supply examples that were never collected, while a larger dataset cannot repair a serving path that misses its deadline because memory is saturated.

The relevant constraint is the one whose relaxation improves the end-to-end result. This idea of a *binding constraint* prevents teams from optimizing the most visible component instead of the component that governs the outcome. The purpose of D·A·M is therefore operational: it identifies the next hypothesis to test and the class of intervention capable of changing system behavior. The diagnosis is provisional. Once an intervention relaxes one limit, the system must be measured again because a different axis may now bind. D·A·M is a loop, not a one-time label.



Definition 1.2: The D·A·M taxonomy

D·A·M taxonomy is a diagnostic framework that classifies any machine learning system performance bottleneck along three axes. Data determines what examples and bytes the system must process, Algorithm determines the model structure and work required to learn or predict, and Machine determines the hardware capacity available to execute that work. The goal is to identify which axis is the binding constraint.

1. **Significance:** The diagnostic power is concrete even before detailed hardware arithmetic enters the story. If the spam filter misses a new phishing campaign because the training set never contained that tactic, the binding axis is Data. If the training examples are adequate but the model cannot express the pattern, the binding axis is Algorithm. If both are adequate but the service cannot classify messages quickly enough during a traffic spike, the binding axis is Machine. Quantitative diagnosis begins by asking which axis is limiting the system.
2. **Distinction:** Unlike traditional software performance analysis, which treats code and data as separate concerns, the D·A·M taxonomy recognizes that algorithm choice directly determines both the training dataset size required (a transformer needs orders of magnitude more data than a linear model to generalize) and the machine required to run it.
3. **Common pitfall:** A frequent misconception is that the three axes are independent. Changing from a simple classifier to a larger model can require more memory, different serving infrastructure, and a broader data distribution. The axes move together.

Figure 1.3 maps these three axes into an interdependent diagnostic triad, where changing the requirements along any single edge directly modulates the constraints on the remaining two.

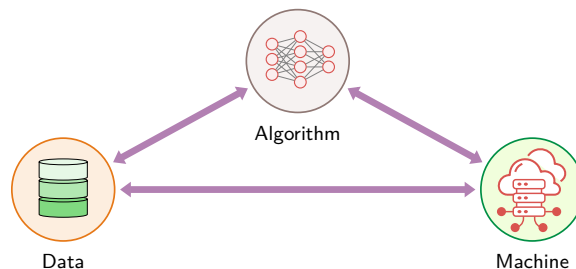


Figure 1.3: The D·A·M Axis Triad: Data, Algorithm, and Machine are three interdependent parts of an ML system. The bidirectional links emphasize that changing one part can change the requirements imposed on the other two.

The triad provides the operational protocol for root-cause analysis. It converts an end-to-end symptom—such as accuracy degradation, tail-latency violations, or cost overruns—into three systematic measurement paths: inspecting the input evidence (Data), characterizing the mathematical

workload (Algorithm), and profiling physical hardware execution (Machine). A diagnosis earns confidence only when relaxing the identified constraint improves full-system behavior.

The bidirectional links in figure 1.3 emphasize that an intervention at one vertex frequently shifts the binding constraint to another. Upgrading to faster accelerators (Machine) can leave execution units idling if storage I/O cannot stream input batches quickly enough (Data). Expanding dataset volume (Data) can expose a model architecture that lacks the expressive capacity to absorb new patterns (Algorithm). Scaling model capacity to close that gap (Algorithm) can exceed accelerator memory capacity, triggering uncoalesced memory spilling or out-of-memory faults (Machine). Engineering an ML system requires identifying the currently binding constraint while anticipating where the bottleneck will migrate once that constraint is relaxed.

While the triad separates the three axes to simplify initial diagnosis, practical systems operate at their boundaries. Data formats govern memory traffic, model architectures dictate hardware functional unit utilization, and deployment budgets reshape both data representation and algorithmic design. Figure 1.4 maps these pairwise boundaries and their shared center.

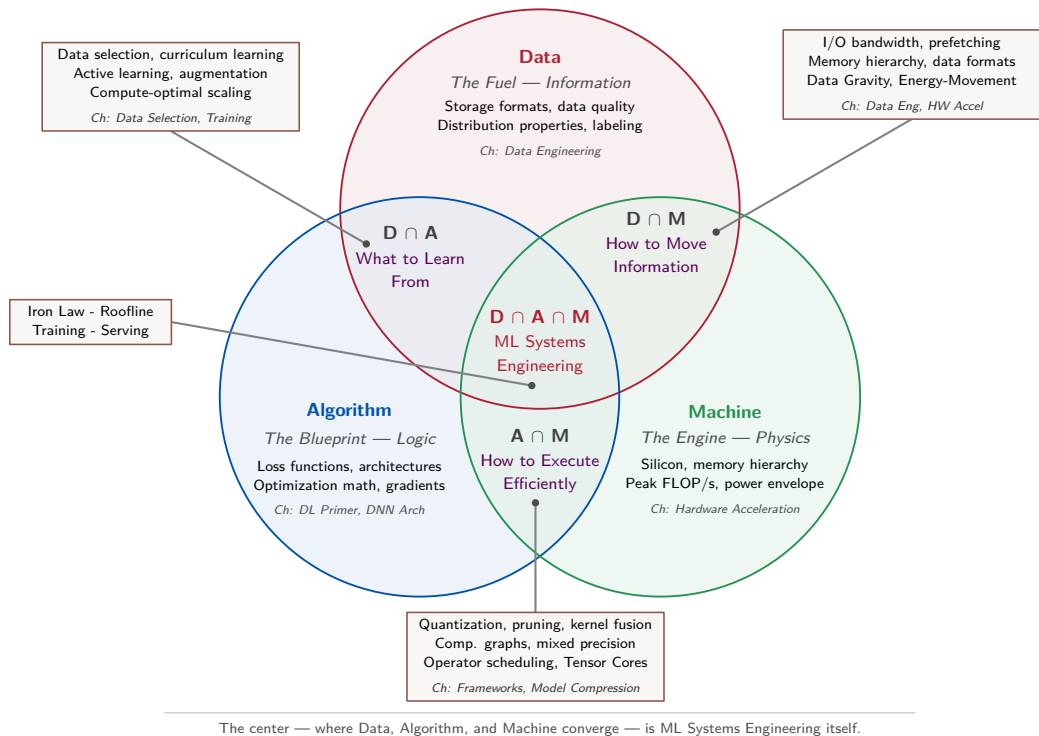


Figure 1.4: The D-A-M Intersection Landscape: Data, Algorithm, and Machine begin as distinct diagnostic lenses, but practical ML systems live at their boundaries. Pairwise intersections ask what a system can learn from, how information moves, and how computation executes efficiently. Their center is ML systems engineering, where all three constraints must be balanced together.

The Venn diagram organizes these relationships from the outer boundaries inward: Data and Algorithm determine *what the system can learn from*; Data and Machine determine *how information moves*; Algorithm and Machine determine *how computation executes efficiently*. At the center, all three questions must be answered simultaneously. That intersection defines machine learning systems engineering.

These intersections reflect the physical trade-offs governing implementation. Data selection and training dynamics resolve the Data–Algorithm boundary by establishing how much empirical signal is required for convergence. Data engineering and hardware acceleration govern the Data–Machine interface, where bus widths, PCIe lanes, and storage bandwidth dictate how quickly tensors reach execution units. Frameworks, compilation, and kernel scheduling resolve the Algorithm–Machine boundary, mapping dense linear algebra onto execution units while minimizing memory stalls.

No design choice remains isolated. Pruning a model (Algorithm) alters its memory footprint and cache residency (Machine), while streaming compressed features alters data pipeline throughput and host-to-device transfer latency (Data). In production ML, local optimizations that ignore these intersections merely shift the bottleneck across boundaries.

The D·A·M landscape provides the diagnostic framework, but engineering real systems also requires a vertical hierarchy that connects physical silicon limits to the end-to-end application mission.

1.5.1 The four-layer hierarchy from silicon to mission

Every machine learning system analyzed in this text is constructed from four hierarchical layers, ensuring that a decision made at the silicon level is traceable to its impact on the final mission.

1. **Hardware (The Silicon).** The physical foundation (The Engine) defines peak compute throughput (R_{peak}), memory bandwidth (BW), and memory capacity. Concrete hardware twins instantiate those quantities when deployment scenarios need numeric constraints.
2. **Systems (The Platforms).** The integrated deployment unit (The Car) defines the operating envelope through its power budget, thermal limits, and node-level interconnects. Examples include the Training Cluster Node or the Sub-Watt Sensor Node.
3. **Workloads (The Models).** The algorithmic demand (The Route) comprises the operation count (O), data volume moved (D_{vol}), and data layout. Scenario-specific workloads, such as GPT-4 and Wake Vision (a visual wake-word dataset sized for microcontrollers), instantiate these demands for particular missions.
4. **Missions (The Scenarios).** The application context (The Destination) occupies the top of the stack, where a system is deployed to solve a specific problem. A mission introduces requirements such as battery life, safety latency, or cloud cost ceilings that dictate the configuration of every layer below.

This hierarchy ensures that system optimization is never performed in an architectural vacuum. A micro-optimization in a matrix kernel is meaningful only when it expands the operating envelope of a deployment platform or meets the latency deadline of a critical mission. The lifecycle analysis in section 1.9.1 pairs each recurring mission with its workload and binding constraint, linking silicon throughput directly to operational goals.

Systems Perspective 1.1: The ML systems landscape: Four deployment paradigms

The machine learning systems landscape spans roughly 10^6 in computational power and 10^5 in memory capacity. Table 1.4 contrasts the memory, compute, and power envelopes across the four **deployment paradigms** that define the constraints for every subsequent chapter.

Table 1.4: Four Deployment Paradigms: Representative memory, compute, and power envelopes for cloud, edge, mobile, and TinyML systems, exposing the multi-order-of-magnitude span that prevents simple model reuse across tiers. Exact hardware twins and peak-rate calculations appear in the hardware chapters.

Paradigm	Representative System	Memory Envelope	Compute Envelope	Power Envelope
Cloud	Data-center accelerator node	Large device memory plus storage ($\approx 10^{11}$ B)	Highest-throughput tier ($\approx 10^{15}$ ops/s)	Facility-managed power
Edge	Robotics or industrial gateway	Local memory under deployment limits ($\approx 10^{11}$ B)	Local accelerator or CPU budget ($\approx 10^{14}$ ops/s)	Wall, vehicle, or site power
Mobile	Smartphone or wearable-class SoC	Shared application memory ($\approx 10^{10}$ B)	Phone-class neural, GPU, and CPU engines ($\approx 10^{13}$ ops/s)	Battery and thermal cap
TinyML	Microcontroller node	Kilobyte-scale memory ($\approx 10^6$ B)	Always-on sensor compute ($\approx 10^9$ ops/s)	Milliwatt-class battery budget

Reading the memory and compute columns from Cloud to TinyML shows endpoints that differ by 10^5 in memory and 10^6 in compute. This divergence is precisely why engineers cannot shrink a cloud model to run at the edge; each tier requires a fundamental redesign of the D·A·M axes.

The multi-order-of-magnitude span across these four deployment paradigms translates directly into financial and operational cost. An architecture that fits comfortably within a data-center accelerator’s memory cannot execute unchanged on a microcontroller-class device. Bridging that physical divergence requires balancing data quality, algorithmic complexity, and hardware capability against an overarching economic constraint: **useful work per dollar**.

🔍 Systems Perspective 1.2: Useful work per dollar

Researchers often optimize for *accuracy*, whereas systems engineers must optimize for *useful work per dollar*. Let O_{total} denote the total operations performed across all sample presentations, and let cost efficiency denote useful operations delivered per dollar. Equation 1.2 provides a dimensionally consistent first-order cost model:

$$\text{Compute Cost} \approx \frac{O_{\text{total}}}{\text{Useful Operations per Dollar}} \quad (1.2)$$

Each D·A·M axis improves this ratio differently. Better data can reduce the sample presentations needed to reach a target quality. More efficient algorithms reduce the operations required per sample, while more cost-efficient hardware increases the useful operations delivered per dollar.

Systems engineering balances this trade-off. A 10 percent gain in cost efficiency can fund about 10 percent more useful operations at the same budget, but whether to allocate them to more data, additional training epochs, or a higher-capacity model depends on the workload’s learning-curve elasticity. If error scales as $D^{-\alpha}$ with dataset size D , the empirical gain from a 10 percent dataset expansion is governed by $\alpha \log(1.1)$ rather than an arbitrary fixed percentage. The systems engineer must estimate that elasticity for the target workload to determine where resource allocation yields the highest marginal return.

These interactions govern more than operational expenditure; they dictate how systems fail. A corrupted data pipeline, unmodeled distribution shift, or hardware bandwidth saturation can each degrade model behavior in production silently, without triggering an operating system crash or runtime exception.

1.6 ML vs. Traditional Software

Under the D·A·M taxonomy, machine learning systems integrate data that defines behavior, algorithms that extract statistical patterns, and machines that execute training and inference.²⁰ Understanding how engineering these systems diverges from traditional software begins with how they fail in production.

Conventional software defects typically produce explicit failure modes: applications crash, unhandled exceptions propagate, processes terminate with nonzero exit codes, and health-check endpoints report HTTP error statuses. While traditional software can also produce silent numerical errors or logic bugs, machine learning introduces **silent degradation** as a dominant operational failure mode. The runtime executes instructions without error, memory allocations succeed, and inference servers return predictions well within latency service level agreements (SLAs), yet the statistical validity of the output steadily deteriorates.

In an automotive driver-assistance system, for example, conventional control software exposes hardware or communication faults through diagnostic trouble codes and hardware watchdog timers. An ML-based perception pipeline introduces a failure mode invisible to these watchdogs. Its pedestrian detection accuracy might decline from 95 percent to 85 percent over several months as seasonal shifts introduce low-angle winter sunlight, heavy precipitation, or bulky clothing underrepresented in the training distribution. The embedded accelerator continues dispatching vision kernels at full framerate without raising a single hardware fault, yet the vehicle operates with an elevated safety risk that eludes standard operating-system telemetry.

The magnitude of this degradation is severe in safety-critical contexts. A perception model operating at 10 Hz evaluates 36,000 frames per hour. While a 0.1 percent false-negative rate applies only to frames containing actual pedestrians, the absolute miss count depends on pedestrian encounter frequency, temporal filtering, sensor fusion heuristics, and operational design domain constraints. The 10-percentage-point degradation from 95 percent to 85 percent directly multiplies the exposure rate of downstream control logic in edge cases where detection was already marginal.

This failure mode engages all three D·A·M axes simultaneously. On the data axis, input distributions drift as environmental conditions, user behaviors, or physical sensor calibrations shift away from the training baseline (Gama et al. 2014; Quiñonero-Candela et al. 2009). On the algorithm axis, the model applies fixed parameter weights to inputs that violate original distribution assumptions, generating degraded probabilities without triggering numerical exceptions like NaNs or infinities. On the machine axis, accelerators execute kernel dispatches and stream tensors through memory hierarchies at maximum throughput, scaling degraded outputs across millions of inferences without a single hardware interrupt.

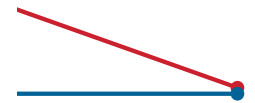
Because this failure mode produces no operating-system exceptions or hardware faults, detecting it requires quantitative signals that measure distribution divergence. Just as hardware execution time can be decomposed into physical latency components, model degradation can be approximated as a function of distribution shift. Let Accuracy_0 denote the baseline accuracy at deployment, $\mathcal{D}(P_t \| P_0)$ represent the statistical divergence between current production distribution P_t and training distribution P_0 , and λ denote the locally fitted sensitivity to that shift measure. This relationship, formalized in equation 1.3 and illustrated in the margin, is the **degradation equation**:

$$\text{Accuracy}(t) \approx \text{Accuracy}_0 - \lambda \cdot \mathcal{D}(P_t \| P_0) \quad (1.3)$$

This first-order linearization captures local trends when labeled ground truth validates the empirical relationship. **Data drift** occurs when P_t differs from P_0 , causing inference quality to drop despite byte-identical model binaries and execution environments. The approximation breaks down under severe distribution collapse, and the divergence metric $\mathcal{D}(\cdot \| \cdot)$ remains deliberately general—typically evaluated via Kullback-Leibler (KL) divergence, total variation distance, or Wasserstein distance. Because statistical divergence alone does not dictate the sign or severity of an accuracy drop without label feedback, the diagnostic framework exposes three concrete engineering levers:

1. **Improve initial accuracy** (Accuracy_0). Scaled training data, refined objective functions, and higher-capacity architectures raise baseline performance, shifting the intercept upward without altering sensitivity to drift.

²⁰ | **Inference:** From Latin *inferre* (“to bring in” or “to conclude”). In ML engineering, inference refers to the deployment phase where a trained model applies learned patterns to novel inputs. The systems distinction matters because training is throughput-optimized (maximize samples/second), whereas inference is latency-optimized (minimize milliseconds/prediction). These opposing objectives demand fundamentally different hardware configurations and software stacks (see chapter 13).



Accuracy can decay silently as distributions drift.

2. **Reduce distribution sensitivity** (λ). Data augmentation, domain adaptation, and regularized representations flatten the degradation curve, preserving performance across broader input distributions.
3. **Monitor drift and outcomes** ($\mathcal{D}(P_t \| P_0)$). Tracking statistical divergence over input feature streams provides an early warning signal, while delayed ground-truth labels verify whether observed drift has induced functional degradation.

In practice, *knowing when to reevaluate is as important as knowing how to train*. An automated serving system can alert when distribution divergence crosses an operational threshold $\mathcal{D}(P_t \| P_0) > \tau$ and trigger retraining pipelines when ground-truth labels confirm performance degradation. Without automated drift monitoring, an inference system is blind to changing inputs. Chapter 14 develops the monitoring infrastructure and alerting strategies that implement this principle.

Silent degradation does not originate solely from external environmental shifts. It frequently arises from internal architectural defects known as **training-serving skew**. In production ML systems, feature engineering is typically implemented across two distinct software stacks: an offline batch pipeline (such as Spark or SQL) that generates training datasets from historical data lakes, and an online microservice (written in C++, Go, or Rust) that computes features under millisecond latency constraints during live inference. If online feature extraction logic diverges even subtly from the offline implementation—such as computing sliding-window averages instead of tumbling-window buckets, applying disparate time-zone normalizations, or handling missing values with conflicting defaults—the model receives feature representations that violate its training assumptions. The model binary executes cleanly, memory buffers remain valid, and serving latency remains nominal, but prediction accuracy collapses at deployment. Training-serving skew is fundamentally a distributed systems synchronization defect that manifests as an algorithmic failure.

These failure modes reshape the entire engineering lifecycle. Traditional software monitoring tracks host health: CPU utilization, memory residency, network I/O, and HTTP status codes. Machine learning operations must track these infrastructure metrics alongside data quality, feature distributions, and prediction accuracy. Because an ML system can fail silently while infrastructure metrics appear nominal, system design must integrate continuous evaluation from data ingestion through inference serving.

Silent degradation addresses the first half of the ML systems mandate: ensuring that learned behavior remains statistically reliable in a non-stationary world. The second half of the mandate governs physical execution: ensuring that the machine can evaluate and update that behavior within strict latency, memory, energy, and monetary budgets. As computational scale continues to dictate ML capability, systems engineering requires quantitative reasoning about the memory bandwidth, arithmetic throughput, and communication overheads that physically govern that scale.

1.7 Iron Law of ML Systems

The physical cost of computational scale becomes concrete in two familiar failures. A training job stalls when storage cannot feed an accelerator; an inference path misses its deadline when model state moves too slowly through memory or across the network. Their symptoms differ, but both consume the same finite time budget through data movement, computation, and fixed overhead. The **iron law of ML systems** makes this shared structure explicit by decomposing total execution time T (seconds) into those three physical costs (equation 1.4):

$$T = \underbrace{\frac{D_{\text{vol}}}{\text{BW}}}_{\text{The Data Term}} + \underbrace{\frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}}_{\text{The Compute Term}} + \underbrace{L_{\text{lat}}}_{\text{The Latency Term}} \quad (1.4)$$

This equation is the mathematical spine of this book. It decomposes the total time required for any ML task, whether training a model for weeks or serving an inference in milliseconds, into three terms that correspond directly to the physical constraints of the dual mandate introduced in section 1.1:

1. **The data term** (D_{vol}/BW) represents the physical cost of moving bits. D_{vol} is the volume of data moved (bytes), and BW is the memory or network bandwidth (bytes/s). Whether loading terabytes from cloud storage or fetching weights from high-bandwidth memory, performance is often limited by I/O physics. Part I develops this foundation.

2. **The compute term** ($O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$) represents the cost of arithmetic. O is the number of floating-point operations (FLOPs), R_{peak} is the hardware's theoretical peak throughput (FLOP/s), and η_{hw} is dimensionless realized hardware utilization ($0 \leq \eta_{\text{hw}} \leq 1$). Parts II and III develop this term.
3. **The latency term** (L_{lat}) represents the irreducible “tax” of system orchestration, networking, and serialization (seconds). This fixed latency dominates in real-time deployment. Part IV develops this term.

Systems Perspective 1.3: The iron law analogy

The name “iron law” follows Patterson & Hennessy’s Iron Law of Processor Performance (Hennessy and Patterson 2017). P&H’s law is a *multiplicative decomposition* (a tautology factoring CPU time), whereas this equation is an *additive first-order model* that approximates performance under simplifying assumptions.

The additive form assumes sequential execution. When movement and computation overlap, we replace the sum with their critical-path lower bound in equation 1.5:

$$T_{\text{pipelined}} \geq \max \left(\frac{D_{\text{vol}}}{\text{BW}}, \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} \right) + L_{\text{lat}} \quad (1.5)$$

This max-based formulation is the systems equivalent of overlapping asynchronous Direct Memory Access (DMA) data transfers with active Arithmetic Logic Unit (ALU) computation, where the execution time of the slower pipeline stage hides the latency of the faster stage. Equality requires ideal overlap and assumes that L_{lat} remains outside the overlapped phases. Even in additive form, the iron law remains useful because, like Amdahl’s Law (Amdahl 1967), its value lies in identifying which physical constraint dominates before optimizing. Section A.3.2 presents the refined treatment, including pipelining and overlap techniques that transform the additive model into the max-based formulation used in practice.

Decomposing execution time matters only if the engineer can diagnose which term dominates the critical path. The Roofline Model answers the movement-versus-compute boundary with the **ridge point**: below the ridge, data movement dominates; above it, arithmetic throughput dominates (section D.2.1 develops the formal derivation). Every optimization technique developed in this book manipulates one of these variables by moving less data (D_{vol}), executing fewer operations (O), utilizing hardware more effectively (η_{hw}), or hiding orchestration delay (L_{lat}). A GPT-3-class training estimate makes that manipulation concrete by showing how an efficiency change propagates through the iron law.

Napkin Math 1.1: Training GPT-3

Problem: How long would it take to train a GPT-3-class model on a reference cluster with 1,024 accelerators?

Given:

- **Ops** (O): $\approx 3.14 \times 10^{23}$ FLOPs
- **Peak** (R_{peak}): 312 TFLOP/s
- **Efficiency** (η_{hw}): ≈ 45 percent (typical for large-scale distributed training)
- **Scale** (N_{accel}): 1,024 accelerators

Math:

Because large-scale language model pretraining has high arithmetic intensity and pipelines data loading asynchronously behind matrix multiplication, the compute term dominates the critical path. Isolating that term across N_{accel} parallel accelerators yields:

$$\bullet T_{\text{train}} \approx \frac{O}{N_{\text{accel}} \cdot R_{\text{peak}} \cdot \eta_{\text{hw}}} \approx \frac{3.14 \times 10^{23}}{1024 \times 312 \times 10^{12} \times 0.45} \approx 25 \text{ days}$$

Result: 25 days.

Systems insight: If we improve hardware utilization (η_{hw}) from 45 percent to 60 percent through better scheduling and more efficient execution, training time drops to 19 days, saving 6 days of expensive compute time.

The equation is dimensionally consistent because each term resolves to seconds. One cannot add FLOPs to bytes any more than one can add meters to kilograms; the iron law adds time to time. A formal treatment in section D.2.2 verifies this consistency and demonstrates how unit tracking prevents common modeling errors.

The iron law governs execution time, but time is not the only physical constraint. For battery-powered edge devices and warehouse-scale accelerator clusters alike, energy dissipation and thermal limits often bind the system before throughput limits are reached. Moving bits across the memory hierarchy exacts an **energy tax** that parallels the delay of the data term.

Let D_{vol} be the total data volume moved (bytes), E_{move} the energy per byte moved, O the total operation count, and E_{compute} the energy per operation. Equation 1.6 formalizes this relationship, following the hardware-energy observation that data movement can dominate arithmetic energy (Horowitz 2014):

$$E_{\text{total}} \approx \underbrace{D_{\text{vol}} \times E_{\text{move}}}_{\text{Movement Term}} + \underbrace{O \times E_{\text{compute}}}_{\text{Compute Term}} \quad (1.6)$$

Per access, data movement can cost far more than arithmetic, so $E_{\text{move}} \gg E_{\text{compute}}$. Under the energy constants used in this text, moving one byte from off-chip Dynamic Random-Access Memory (DRAM) consumes roughly 145.5× the energy of an FP16 multiply and 800× that of an INT8 (8-bit integer) multiply (Horowitz 2014). Whether movement dominates a complete workload also depends on the number and width of transfers relative to its operation count. The physical reason for the per-access gap is that data movement requires charging and discharging wires over longer distances, while arithmetic occurs locally within a processing unit's circuits. Minimizing avoidable data movement (D_{vol}) can therefore improve both speed and energy efficiency.

DRAM 160 pJ

FP16 1.1 pJ

INT8 0.2 pJ

log scale

Moving a byte costs about 145 times an FP16 op, the data-movement tax.



Checkpoint 1.2: The iron law

The iron law ($T \approx \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}}$) is the analytical backbone of this book. Before proceeding, verify you can manipulate its terms:

- ☐ **Data term** (D_{vol}/BW): identify the physical resource that bounds this term and name one workload regime where it dominates.
- ☐ **Compute term** ($O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$): identify the hardware quantity and utilization factor that bound this term.
- ☐ **Latency term** (L_{lat}): identify which costs remain after data movement and arithmetic have been optimized.
- ☐ **Peak throughput**: if you double R_{peak} , which term improves, and why do the other terms remain unchanged?

The same physical terms that determine time and energy also determine cost: every byte moved, operation executed, and millisecond of latency consumes infrastructure budget. The next test is therefore economic, evaluating whether added compute buys enough model improvement to justify the expenditure.

1.7.1 Return on compute (RoC) as an economic lens

Following the quantitative reasoning tradition of Hennessy and Patterson, **return on compute (RoC)** measures the incremental accuracy gain per added dollar of infrastructure investment. With accuracy measured on a fixed scale, RoC has units of accuracy points per dollar.

$$\text{RoC} = \frac{\Delta \text{Accuracy}}{\Delta \text{Compute Cost}}$$

This ratio exposes an economic boundary. A 1-percentage-point gain in accuracy may fail the RoC test if it requires a $10\times$ increase in O (total operations). Every optimization in the following chapters targets either the numerator (extracting more signal from the same data) or the denominator (reducing the cost of executing the math). If the RoC is negative or negligible, the system is over-engineered, regardless of its technical sophistication. This economic lens transforms “accuracy” from a research target into an engineering budget.

While computational scale expands model capability, it simultaneously compounds hardware expenditure. The bitter lesson teaches that scale works, but the iron law governs *how* to afford it. This tension between scaling and physical limits shapes the engineering principles that follow.

1.7.2 Lighthouse models put the iron law into practice

The iron law organizes the core engineering imperatives of this curriculum. The data term demands robust ingestion, validation, and feature pipelines (chapter 4). The compute term requires maximizing arithmetic intensity and hardware utilization across specialized accelerator architectures (Part III). The latency term forces strict control over serialization delay, communication overhead, and serving jitter (chapter 13, chapter 14). Together, these terms transform abstract system requirements into actionable physical constraints.

Abstract equations become tangible through concrete workloads. Five recurring **lighthouse models** serve as diagnostic probes for the iron law throughout this book. These canonical workloads reappear across chapters to test how physical constraints affect distinct architectural patterns.

Each lighthouse model isolates a distinct stress case for the iron law. ResNet-50 probes compute throughput when an algorithm repeatedly reuses learned parameters, while GPT-2/Llama probes memory bandwidth limits during autoregressive generation. For language models, autoregressive decode generates one token at a time; the KV (Key-Value) cache stores the attention state from preceding tokens (chapter 6 introduces the attention mechanism in full), and prefill is the initial pass that processes the prompt before generation begins. Operating regime determines the governing bottleneck: small-batch decode streams weights and KV-cache state fast enough to expose memory bandwidth, while prefill and high-batch serving shift the bottleneck toward arithmetic throughput or communication.

The iron law makes these differences mathematically precise. ResNet-50 applies the same weight filters across many spatial positions and, under batching, across many input samples; that arithmetic reuse makes $O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$ the dominant term because the processor must sustain high arithmetic throughput while the data footprint remains compact in cache. GPT-2, by contrast, loads billions of unique weight parameters for every generated token, using each weight only once before fetching the next; its D_{vol}/BW term dominates because memory bandwidth, not arithmetic throughput, is the binding constraint. Applying the same equation to two workloads yields opposite optimization strategies: doubling R_{peak} accelerates batched ResNet-50 once reuse increases operations per byte moved, but leaves GPT-2 decode virtually unchanged; doubling memory bandwidth BW accelerates bandwidth-bound decode while leaving compute-saturated convolutions unchanged. The remaining lighthouse models isolate the other physical boundaries of the iron law: DLRM stresses memory capacity because multi-gigabyte embedding tables—mapping millions of user and item IDs—exceed single-accelerator memory and demand distributed scale-out; MobileNetV2 and Keyword Spotting evaluate the latency (L_{lat}) and energy equations on battery-powered edge devices and microcontrollers, where inference must satisfy millisecond deadlines within milliwatt power budgets. Table 1.5 summarizes why each lighthouse model serves as a diagnostic tool for a specific bottleneck.

Table 1.5: Lighthouse Models as Reference Workloads: Each workload isolates a distinct bottleneck, enabling systematic investigation of how system constraints affect different architectural patterns. Quantitative specifications and architectural details appear in chapter 6.

Lighthouse Model	System Bottleneck	What It Reveals	Key Engineering Questions
ResNet-50	Compute throughput under reuse	GPU utilization, batching	Is the hardware doing math or waiting for data?
GPT-2/Llama	Memory bandwidth	Weight and sequence-state movement	How fast can model state move to compute?
Deep Learning Recommendation Model (DLRM)	Memory capacity	Embedding tables, scale-out	How do terabyte-scale models fit in memory?
MobileNetV2	Latency and power	Efficient operator design	Can the system meet real-time constraints on battery?
Keyword spotting	Power envelope	Tiny memory and energy budgets	Can the system run always-on inference on milliwatts?



GPT-2 decode is bandwidth-bound because the data term dominates.

21 | **ImageNet:** The 2009 paper reported 3.2 million images across 5,247 synsets; the later full dataset grew to about 14.2 million images across 21,841 synsets. The 2012 challenge training split used by AlexNet contained about 1.3M labeled images (Deng et al. 2009; Russakovsky et al. 2015) (see chapter 4).

By tracking these workloads from data ingestion through edge deployment, each chapter demonstrates how an architectural choice propagates physical and economic constraints across the entire system. Each lighthouse model manifests distinct constraints along the D·A·M axes, ensuring that engineering principles are tested across diverse systems regimes. The division of labor among the book’s recurring examples is deliberate: four deployment paradigms fix the *envelope* a system must operate within, five lighthouse models supply the *workloads* that stress it, and four engineering missions and three production case studies (Waymo, FarmBeats, and AlphaFold in section 1.9.3) pair envelope with workload under real-world constraints.

The same diagnostic reading applies retrospectively to the breakthrough that launched the deep learning era. The AlexNet system combined a convolutional architecture whose parallel matrix operations matched GPU capabilities with the 1.3M labeled images in the 2012 ImageNet challenge training split²¹ (Deng et al. 2009). Its error reduction therefore reflected coordination across the D·A·M axes rather than algorithmic novelty in isolation.

Co-design across the D·A·M axes means that optimizing one component often shifts pressure to another. AlexNet’s co-design success came at a cost affordable in 2012 (two consumer GPUs for a week), but modern foundation models demand training compute roughly 7 orders of magnitude larger. If the iron law governs *how fast* a system runs, an engineering framework remains necessary to reason about *how efficiently* it uses those resources.

1.8 Three Dimensions of ML Efficiency

That efficiency question exposes a tension first introduced by the bitter lesson in section 1.4. Scale drives AI progress, but ever-larger datasets and compute budgets narrow participation to the most resource-rich organizations. Even those organizations eventually meet physical limits in data center power, memory bandwidth, and the diminishing returns of adding more parameters.

Common public estimates for GPT-4-class training place the compute budget around 2.5 million accelerator-days, representing millions of dollars in compute costs and substantial environmental impact. Many research institutions and companies cannot afford to compete through brute-force scaling. Those cost and access constraints make efficient use of existing compute a complementary path to progress.

Efficiency is a bottleneck diagnosis, not a single technique. The D·A·M landscape (figure 1.4) now becomes an action map. Data selection improves *what the system can learn from*, algorithmic efficiency reduces the work required to learn or predict, and compute efficiency aligns that work with the machine. The remaining intersection, *how information moves*, cuts across all three because every improvement must survive the memory and communication path.

Algorithmic efficiency, the earliest frontier, reduces computational requirements through better model design and training procedures. Its goal is to produce more useful behavior per operation, so capability rises without scaling every resource in lockstep. As algorithms demanded ever more computation, **compute efficiency** became the second critical dimension. It maximizes hardware utilization by aligning algorithmic logic with machine physics, turning theoretical processor capability into useful work. Most recently, **data selection** emerged as the third dimension, extracting more learning signal from limited examples and thereby reducing the total operations term *O* of the

iron law. The timeline in figure 1.5 places these three dimensions side by side before the chapter sequence presents them in build order. Together, these three dimensions provide the engineering tools to overcome the data, algorithm, and machine walls that pure scaling alone cannot address.

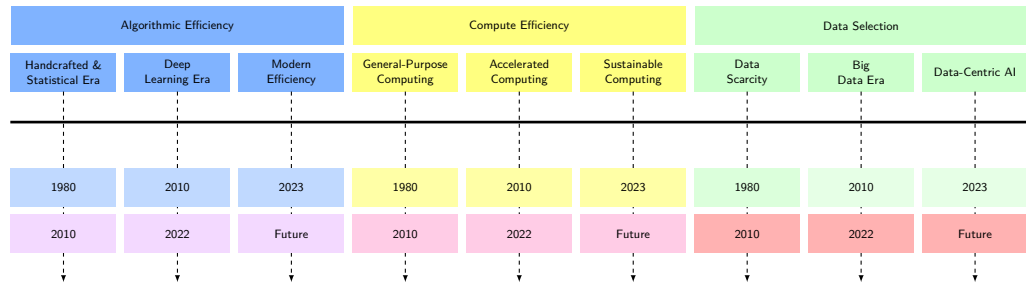


Figure 1.5: Historical Efficiency Trends: Three color-coded sequences align the algorithmic, compute, and data-selection eras. The paired dates beneath each box mark its interval: 1980–2010, 2010–2022, and 2023–future.

These three dimensions unfolded through distinct historical eras, but systems engineering approaches them in reverse. In practice, data selection comes first: pruning redundant training examples lowers total training steps before a model is trained. Model compression follows, reducing parameter footprint and arithmetic operations. Hardware acceleration comes last, mapping the remaining operations efficiently onto the physical memory hierarchy and compute units of the accelerator. Optimizing upstream before downstream ensures that expensive hardware resources execute only necessary computation.

The trajectory of model architectures over time illustrates how these dimensions progress. Figure 1.6 makes the impact of algorithmic efficiency visible model by model, demonstrating how identical accuracy targets require progressively less compute as architectures improve.

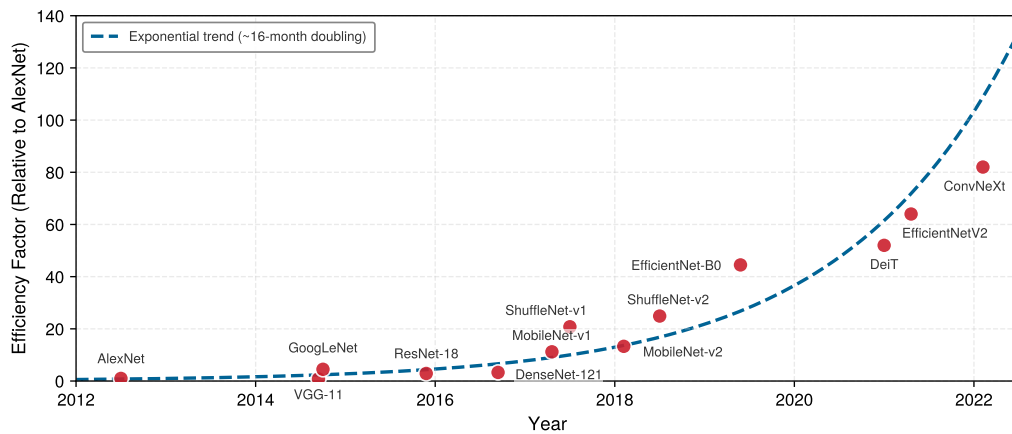


Figure 1.6: Algorithmic Efficiency Trajectory: Training efficiency factor relative to AlexNet (2012 baseline) for ImageNet classification. Most later architectures achieve comparable accuracy with fewer computational resources, although individual points vary. The trajectory from AlexNet ($1\times$) through VGG (Visual Geometry Group), ResNet, MobileNet, and ShuffleNet to EfficientNet ($44\times$) demonstrates a 44.5-fold reduction in required compute over seven years, independent of hardware improvements (Hernandez and Brown 2020). Early-2020s points and the dashed fit through all plotted points are author-added context rather than evidence for extending the Hernandez-Brown trend.

The magnitude of efficiency improvements is measurable. Between 2012 and 2019, computational resources needed to train a neural network to achieve AlexNet-level performance on ImageNet classification decreased by approximately $44.5\times$ (Hernandez and Brown 2020). This improvement, which halved about every 15 months, outpaced hardware efficiency gains predicted by Moore's Law,²² demonstrating that algorithmic innovation drives efficiency as much as hardware advances.

²² | **Moore's law:** Gordon Moore's 1965 observation described rapid growth in the number of components that could be economically integrated on a chip (Moore 1998); later industry summaries often expressed the cadence as roughly a two-year doubling.

Simultaneously, aggregate training compute in published frontier runs followed a much steeper cadence than Moore’s Law, with a fitted doubling time of approximately 3.4 months (Amodei and Hernandez 2018b). That aggregate publication trend is not the same quantity as an endpoint ratio between two landmark models, but it explains *why* efficiency optimization is not optional. Without it, only the most resource-rich organizations could participate in AI development.

These measurements emerge from empirical methodology that tracked training compute across hundreds of published models (chapter 12). The divergence between demand doubling every 3.4 months and transistor density doubling roughly every two years defines the systems gap. Silicon advances alone cannot keep pace with this demand; keeping pace requires compounding efficiency improvements across algorithms, software runtimes, and hardware architectures (chapter 11).

Architecture-by-architecture gains tell only half the story. Algorithmic improvements could not offset rising aggregate training compute. Figure 1.7 compares illustrative AlexNet-era and GPT-4-class endpoints, distinct from the aggregate 3.4-month trend, showing why efficiency optimization is necessary at every level of the stack.

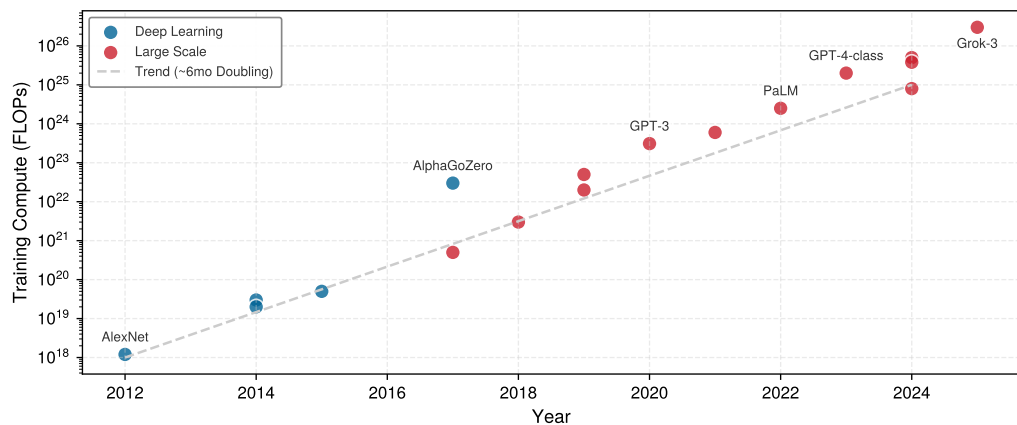


Figure 1.7: The Era of Scale: Illustrative training-compute estimates (FLOPs) vs. year on a log scale. While early deep learning (blue) showed rapid growth, the transformer era (red) accelerated this trend significantly. From AlexNet (2012) to illustrative GPT-4-class training scale anchors (2023), compute requirements increased by roughly 7 orders of magnitude (16.7M times), far outpacing Moore’s Law. This endpoint comparison is distinct from aggregate training-compute trend studies (Amodei and Hernandez 2018b; Sevilla et al. 2022) and is used here to motivate the specialized infrastructure described in this book.

Taken together, these two figures reveal a seeming contradiction that defines the economics of modern AI development. Figure 1.6 shows efficiency improving 44.5× while figure 1.7 shows compute demand growing by roughly 7 orders of magnitude. The plots, however, hold different quantities constant. The first asks how much computation is required to reach a roughly fixed capability target, whereas the second permits the target to expand and records the resulting training budget. Efficiency lowers the cost of a given capability; scale determines how the newly affordable computation is spent. The apparent contradiction is therefore an economic feedback rather than a disagreement between the measurements.



Systems Perspective 1.4: The efficiency paradox

This dynamic is the ML systems equivalent of **Jevons paradox**. In resource economics, increasing the efficiency with which a resource is used often increases rather than decreases total consumption. In AI, efficiency gains fund larger models and broader datasets rather than smaller compute budgets. If EfficientNet needs 44.5× less compute than AlexNet to reach the same accuracy, organizations can invest the savings in larger models and broader data. This is precisely how GPT-3 came to require orders of magnitude more compute than AlexNet despite per-FLOP efficiency gains. The feedback loop, in which efficiency enables scale and scale demands efficiency, defines the modern AI engineering landscape.

This economic feedback loop demonstrates why efficiency cannot be treated as isolated optimizations. Data selection (chapter 9), model compression (chapter 10), and hardware acceleration (chapter 11) must be balanced against concrete deployment constraints. Managing both statistical performance and physical execution requirements together requires a unified systems discipline.

1.9 AI Engineering as a Discipline

A cloud service may optimize throughput, while an edge device must remain within a strict power envelope. The same model can therefore be efficient in one setting and unusable in another. Model accuracy alone does not specify a working system. Learned behavior must remain trustworthy as data changes, and the machine must deliver that behavior within its operating budget.

The degradation equation, iron law, and efficiency framework supply quantitative tools for this dual mandate. Together they span statistical behavior, computation, and deployment constraints. Applying them crosses disciplinary boundaries. Computer science addresses algorithms, and electrical engineering addresses hardware, but neither alone encompasses the integrated problem of building systems that remain reliable, efficient, and scalable in production. That problem defines AI engineering.

Definition 1.3: AI engineering

AI engineering is the discipline of designing, deploying, and maintaining ML systems that hold statistically evaluated behavior to deterministic reliability targets while satisfying production constraints across all three D·A·M axes: Data quality, Algorithm correctness, and Machine efficiency.

1. **Significance:** ML research typically optimizes only the algorithm axis (O and convergence). AI engineering jointly optimizes all three by bounding D_{vol} through data governance requirements, $O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$ through production latency requirements, and total power draw through energy and cost budgets. A production system that achieves 95 percent accuracy in research but violates a 100 ms latency requirement in production is a failed system, regardless of its algorithm score.
2. **Distinction:** Unlike machine learning research, which targets a single objective (validation loss) on a static dataset, AI engineering targets a multi-objective constraint surface (latency, throughput, accuracy, cost, fairness, and robustness) on a distribution that shifts continuously after deployment.
3. **Common pitfall:** A frequent misconception is that AI engineering is just “software engineering for ML.” The system specification is instead probabilistic. An ML system’s output is statistically valid or invalid relative to a shifting distribution, not correct or incorrect relative to a fixed deterministic contract. This makes continuous monitoring a structural requirement, not an operational choice.

The phrase “stochastic systems with deterministic reliability” connects AI engineering to an earlier disciplinary convergence. Computer engineering emerged in the late 1960s and early 1970s²³ when computing systems grew too complex for electrical engineering or computer science to address alone, uniting both around the integrated problem of building reliable computers from unreliable physical components. AI engineering faces an identical challenge at the boundary of data, algorithms, and infrastructure. In this taxonomy, AI engineering names the broader intellectual discipline, while ML systems engineering defines the practical work of designing, deploying, and maintaining those systems under production constraints.

1.9.1 Engineering across the ML lifecycle

An algorithmic advance requires efficient data ingestion and preprocessing, distributed computation across hundreds or thousands of accelerators, reliable serving infrastructure to meet strict latency windows, and continuous telemetry based on real-world performance. These obligations form a recurring lifecycle rather than a sequence that ends at deployment. The engineering object is no

²³ | **Computer engineering:** Formalized as an academic discipline when Case Western Reserve launched the first accredited program in 1971, recognizing that neither electrical engineering nor computer science alone could address building reliable computers from unreliable components. ML systems engineering recapitulates this convergence: the binding constraint is not algorithmic or hardware in isolation but the integration of both under latency, power, and data-quality budgets that neither discipline’s curriculum addresses.

longer only code; it is code, data, model behavior, deployment context, and monitoring evidence evolving together. Production feedback can force a deployed system back into data collection and training, bending the familiar linear arc into a cycle.

The structural difference shows up first in tooling. Decades of established practice support code-defined behavior through version control that maintains precise histories, continuous integration pipelines that automate testing, and static analysis tools that measure quality. Behavior learned from data slips through this tooling, because the artifact that changes is no longer a diff a developer wrote. Chapter 3 develops the specialized workflows these challenges demand.

The deeper difference is the prominence of continuous feedback. The loops in figure 1.8 show why. When monitoring detects performance degradation, the system does not receive a code patch alone. It may cycle back through data collection, preparation, training, and evaluation before redeployment, making iteration part of the operating architecture rather than only the development process.

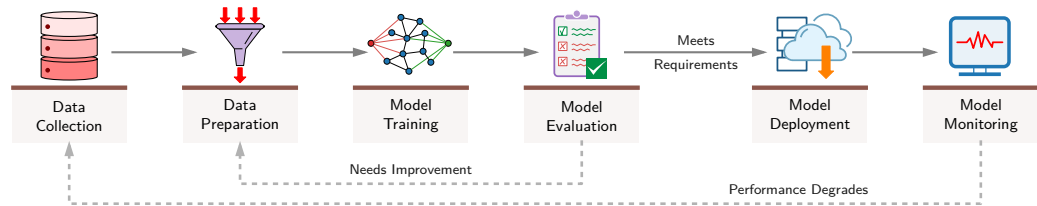


Figure 1.8: ML System Lifecycle: A six-box flowchart depicting Data Collection, Preparation, Model Training, Evaluation, Deployment, and Monitoring. Two feedback loops distinguish this cycle from linear software development: evaluation returns to preparation when results are insufficient, and monitoring triggers new data collection when performance degrades.

Because an ML model compiles behavior directly from training data rather than explicit source logic, real-world distribution shifts alter execution outcomes without modifying a line of code. This divergence breaks standard software workflows: Git and text-based version control cannot manage multi-terabyte dataset lineages, while unit test assertions cannot validate stochastic outputs against shifting distributions. Addressing these operational challenges requires specialized infrastructure. Chapter 4 develops dataset versioning and provenance tracking, while chapter 14 establishes runtime monitoring for distribution shift. What each lifecycle stage demands, however, is not uniform; the binding physical constraints depend on the operational mission the system must serve.



Systems Perspective 1.5: From paradigms to missions

The top of the hierarchy transforms abstract systems into concrete engineering missions. Each mission inherits one of the four deployment paradigms introduced in section 1.5.1 and pairs it with a scenario-specific workload. The four missions in table 1.6 are the application scenarios that recur throughout the book and its associated labs. They act as end-to-end tests of engineering decisions. A $2\times$ increase in memory bandwidth is an academic result until it is proven to extend the battery life of the Smart Doorbell or enable the safety-critical latency required for autonomous perception.

Each mission runs this lifecycle continuously, where a training *step* denotes a single forward-backward pass and weight update across a micro-batch of data. High-quality data improves the model, which improves the product and the telemetry it generates; weakness at any stage propagates downstream across the entire stack. Table 1.6 defines these four recurring missions alongside the binding physical constraint that dictates their execution.

Table 1.6: Four Engineering Missions: Each deployment paradigm is paired with a recurring scenario workload and its binding constraint.

Mission	Deployment Paradigm	Scenario Workload	Critical Constraint
Frontier training	Cloud Cluster	GPT-4	Target: 500 ms/step

Mission	Deployment Paradigm	Scenario Workload	Critical Constraint
Autonomous perception	Edge Robotics	YOLOv8-nano	SLA (Service-Level Agreement): 10 ms latency
Mobile assistant	Smartphone	Mobile-optimized small LLM (Large Language Model)	RAM: < 2 GB / Thermal: < 3 W
Smart Doorbell	TinyML (MCU [Microcontroller Unit])	Wake Vision	Power: 100 mW

1.9.2 Deployment context shapes the lifecycle

Deployment context determines which lifecycle pressures dominate. The same stages apply across ML systems, but a megawatt-scale data center and a milliwatt-scale embedded device impose different bottlenecks on data collection, model updates, monitoring, and serving.

At one end of the spectrum, cloud-based ML systems train large models and serve millions of users, trading abundant computing resources for capacity limits, operational complexity, and high costs. Chapter 2 examines their architectural patterns, while chapter 11 develops the hardware foundations that make this scale economically viable.

At the other end, TinyML systems run on microcontrollers²⁴ and embedded devices. Their kilobyte-scale memory and milliwatt power budgets make feasibility precede model quality: a smart-home device must recognize a command using less power than an LED bulb, while a sensor may need to detect anomalies on one battery for years. The efficiency framework in section 1.8 supplies the governing principles, while chapter 10 develops the techniques that make such deployment possible.

Between these poles, placement becomes a constraint-allocation problem. Edge ML systems move computation toward data sources to reduce latency²⁵ and bandwidth demand. Mobile ML systems share memory, thermal headroom, and battery power with every other application, trading raw speed for locality and privacy. Hybrid systems distribute work across tiers to balance latency, privacy, bandwidth, and update control.

Each position on this deployment spectrum creates distinct bottlenecks that determine which efficiency dimensions matter most, as summarized in table 1.7:

Table 1.7: Efficiency Priorities by Deployment Context: Each deployment environment creates distinct bottlenecks, requiring tailored optimization strategies. Cloud systems optimize for throughput and cost; edge systems optimize for memory and power; TinyML systems require extreme efficiency across all dimensions.

Environment	Primary Constraint	Efficiency Focus
Cloud training	Cost, throughput	Distributed efficiency, hardware utilization
Cloud inference	Latency, cost per query	Batching, model serving optimization
Edge devices	Memory, power	Smaller models and lower data movement
Mobile	Battery, thermal	Energy-efficient inference
TinyML	kilobyte-scale memory, mW power	Extreme compression, specialized architectures

The deployment spectrum represents more than different hardware configurations. Each deployment environment reshapes every stage of the ML lifecycle, from initial data collection through continuous operation and evolution, creating an interplay of constraints that traditional software rarely encounters.

Consider how a single deployment decision cascades through the entire system. Latency-sensitive applications like autonomous vehicles or real-time fraud detection require edge or embedded architectures despite their resource constraints, while large language models naturally gravitate toward centralized cloud infrastructure. This initial architectural choice, however, determines far more than where computation happens. Cloud systems must optimize for cost efficiency at scale, balancing expensive GPU clusters, storage, and network bandwidth, which in turn shapes how often models are retrained, what historical data is retained, and how inference load is distributed. Edge and mobile systems face fixed resource limits that constrain model complexity and update frequency, forcing aggressive model compression²⁶ and careful scheduling. The strictest constraints

²⁴ | **Microcontrollers:** Single-chip computers with kilobytes of memory and milliwatts of power budget. For TinyML, memory and energy determine feasibility before model quality.

²⁵ | **Latency:** From Latin *latere* (“to lie hidden”), delay is invisible until it causes failure. At 30 m/s, every millisecond adds 3 cm of travel before braking begins, making L_{lat} the edge constraint.

²⁶ | **Model compression:** A family of techniques, including quantization, pruning, and distillation, that reduces model storage or computation. Its size and accuracy effects depend on the model, method, workload, and target hardware.

arise in embedded and TinyML environments, where every byte of memory and milliwatt of power matters.

Operational complexity increases as systems become more distributed. Centralized cloud architectures benefit from mature deployment tools and managed services, while edge and hybrid systems must coordinate data collection across sensors with varying connectivity, track models deployed across thousands of devices, handle staged rollouts with rollback capabilities, and aggregate monitoring signals from geographically distributed endpoints (chapter 14). Data considerations introduce competing pressures. Privacy requirements or data sovereignty regulations may push computation toward the edge, while the need for large-scale training data pulls toward centralized cloud aggregation. Model updates also behave differently across the spectrum. Cloud architectures enable rapid iteration through centralized traffic control, while edge deployments require remote updates with careful bandwidth management and rollback capabilities.

In practice, deployment choices rarely follow simple binaries. Production ML architectures frequently span multiple tiers to reconcile conflicting physical and statistical demands. A system may partition computation across edge devices to satisfy sub-millisecond response windows and data-privacy regulations, while routing sampled telemetry to centralized cloud clusters for continuous training and model re-evaluation. Choosing an embedded or edge target constrains far more than parameter footprint. It dictates upstream data collection, quantization-aware training, hardware-in-the-loop evaluation, over-the-air deployment protocols, and remote telemetry aggregation. Physical execution limits on one tier inevitably reshape every phase of the engineering lifecycle.

27 | Autonomous-driving hybrid workflow: This representative workflow forces a synchronization challenge absent from pure cloud or pure edge systems. The on-vehicle model must be controlled and regression-tested before deployment, while cloud infrastructure can train and evaluate improved versions on newly collected driving data. This creates a version-management gap between deployed and newly trained models, requiring rigorous validation before any remote model update can be pushed to safety-critical vehicles.

28 | FarmBeats: The system uses TV white-space links as high-bandwidth intra-farm backhaul from sensors to a gateway PC, where local processing reduces dependence on the weaker Internet connection to the cloud. The resulting constraint is timely data synchronization across that farm-to-cloud link, not delivery of a particular model size (Vasisht et al. 2017).

29 | LiDAR (light detection and ranging): This sensor is a primary reason the vehicle is a “roving data center,” as its pulsed lasers generate a dense 3D point cloud of the environment. The raw data stream from a single unit can exceed 100 megabytes per second, creating both the terabyte-scale volume challenge and the quality challenge mentioned, as the signal is easily degraded by sensor interference from rain or fog.

1.9.3 Production systems expose shared challenges

A deployment case study becomes an engineering tool when it exposes the binding constraint behind a design. Three production systems—Waymo, FarmBeats, and AlphaFold—span the extremes of the deployment spectrum, revealing how identical D·A·M trade-offs force radically different engineering solutions:

- **Autonomous driving**²⁷ binds on safety-critical latency and data freshness. The Waymo Open Dataset provides camera and LiDAR data collected across varied driving environments (Sun et al. 2020), illustrating the multimodal inputs and geographic coverage that a perception stack must handle. The broader case uses a representative high-stakes hybrid pattern with on-vehicle inference for low latency and cloud infrastructure for training and evaluation.
- **FarmBeats**²⁸ (Vasisht et al. 2017) binds on connectivity and data freshness. Microsoft’s precision agriculture platform connects field sensors to a local gateway PC for edge processing. TV white-space networking carries data across the farm, while the weaker farm-to-cloud Internet link constrains synchronization.
- **AlphaFold** (Jumper et al. 2021) binds on compute-intensive training and curated scientific data. DeepMind’s protein structure prediction system solved the 50-year challenge of computational protein structure prediction with near-experimental accuracy. AlphaFold represents the compute-intensive cloud deployment pattern. Initial training used 128 TPUv3 cores for approximately one week, followed by about four days of fine-tuning, and drew on the Protein Data Bank’s experimentally determined structures.

These systems complement the lighthouse models by illustrating how the same core challenges (data quality, model complexity, and infrastructure scale) manifest under radically different constraints. Rather than examining each system in isolation, they are analyzed through the lens of the D·A·M taxonomy. The same data drift phenomenon that affects Waymo’s perception models in changing weather also affects FarmBeats’ crop disease detection across growing seasons, though the engineering responses differ based on machine constraints.

The interdependencies across the D·A·M axes create concrete engineering bottlenecks across data, algorithms, and infrastructure. Examining deployment extremes reveals these constraints in their most demanding forms.

Real-world data is inherently noisy and asynchronous. Autonomous vehicles process large multimodal sensor streams from LiDAR²⁹ and cameras (Sun et al. 2020), requiring hardware-synchronized capture and filtering against sensor degradation from rain, fog, or lens flare. Scale compounds these quality issues across bandwidth-constrained links: FarmBeats must filter and

aggregate agricultural sensor telemetry on a local edge gateway before synchronizing across an intermittent farm-to-cloud link, while AlphaFold demands high-throughput access to the curated, experimentally determined structures of the Protein Data Bank during training.

Data drift creates an ongoing operational burden atop both quality and scale. The statistical properties of input data change over time, and models are only as reliable as their alignment with the current distribution (Gama et al. 2014; Quiñonero-Candela et al. 2009; Koh et al. 2021). The Waymo Open Dataset reports pronounced domain gaps among San Francisco, Phoenix, and Mountain View (Sun et al. 2020);³⁰ detecting such regional shifts requires continuous monitoring of input statistics before they manifest as system failures.

At the algorithm and infrastructure boundary, computational intensity defines the upper bound of capability. Foundation models at GPT-3 scale (section 1.3.3) demand zettaFLOPs of compute, and even smaller scientific models like AlphaFold required weeks of specialized accelerator training. Systems engineers must optimize for “FLOP/s per watt” to make these models economically and environmentally viable. Yet raw scale is not enough. The **generalization gap** remains the central algorithmic risk because a model might achieve 99 percent accuracy on benchmarks but only 75 percent in the real world. For Waymo’s safety-critical autonomous driving systems, minimizing this gap is a life-or-death requirement, demanding robustness methods that cover the long tail of edge cases.

In production deployment, the **training-serving divide** describes the gap between the flexible environment where models are born and the rigid environment where they operate. Latency-throughput trade-offs dictate architecture. Waymo-style perception systems require low-latency safety decisions at the edge, while AlphaFold runs in the cloud and its inference time depends on protein length and configuration. Tiered coordination adds further complexity: voice assistants execute lightweight keyword spotting on microcontrollers (TinyML) to guarantee instant wake-up within strict milliwatt budgets, but offload complex natural language queries to cloud GPU clusters.

Finally, as systems scale, their impact on society becomes a first-class engineering concern that cuts across all three D·A·M axes. Fairness and bias must be managed proactively, since models can unintentionally learn societal biases present in their training data. Responsible engineering requires systematic auditing of performance across demographic subgroups to ensure equitable outcomes. Transparency and privacy requirements further constrain design. Many deep networks function as “black boxes,” yet in domains like healthcare or finance, stakeholders require interpretability. Systems must also be resilient against inference attacks³¹ that attempt to extract sensitive training data from model predictions.

Because a failure in production cascades across data, algorithms, and physical machines, no single specialty can isolate or resolve these bottlenecks alone. Managing these interdependencies requires an explicit operational framework.

1.10 Five-Pillar Framework

Production ML systems require an operational framework that assigns responsibility across data pipelines, mathematical formulations, hardware execution, and governance without severing their physical couplings (Paley et al. 2022). Traditional software engineering isolates components behind modular interfaces; in machine learning systems, abstractions leak because errors propagate across statistical distributions and physical hardware states. A system can fail not only from software defects, but from silent distribution drift, memory bandwidth exhaustion, or thermal throttling.

As illustrated in figure 1.9, these responsibilities organize into five interconnected disciplines resting on a shared technical foundation. Each pillar delineates an operational boundary, while the foundation beneath them enforces the physical and economic limits—compute throughput, memory capacity, interconnect bandwidth, and power budgets—that govern execution.

Tracing a concrete failure demonstrates how these boundaries partition operational responsibility. Suppose an on-device wake-word detector fails to trigger for users in reverberant rooms following a model update. Diagnosing the fault begins at the input pipeline: did ingestion filter out reverberant acoustic profiles, were labels corrupted during preprocessing, and can pipeline lineage trace the exact training partitions used for the update? The **data engineering** pillar (chapter 4) governs the ingestion, validation, and lineage systems that determine what an algorithm can mathematically extract.

³⁰ | **Data drift:** Divergence between the training data distribution (P_0) and the production distribution (P_t). Drift can change performance without a code change, but divergence alone does not determine whether accuracy falls; outcome monitoring is needed to establish degradation (see chapter 14).

³¹ | **Inference attack:** A security threat where an adversary queries a model to deduce sensitive information about the training set. These attacks exploit the tendency of overparameterized models to memorize unique patterns in their training data, creating a direct trade-off between model capacity and privacy risk that motivates defensive techniques such as differential privacy and output perturbation.

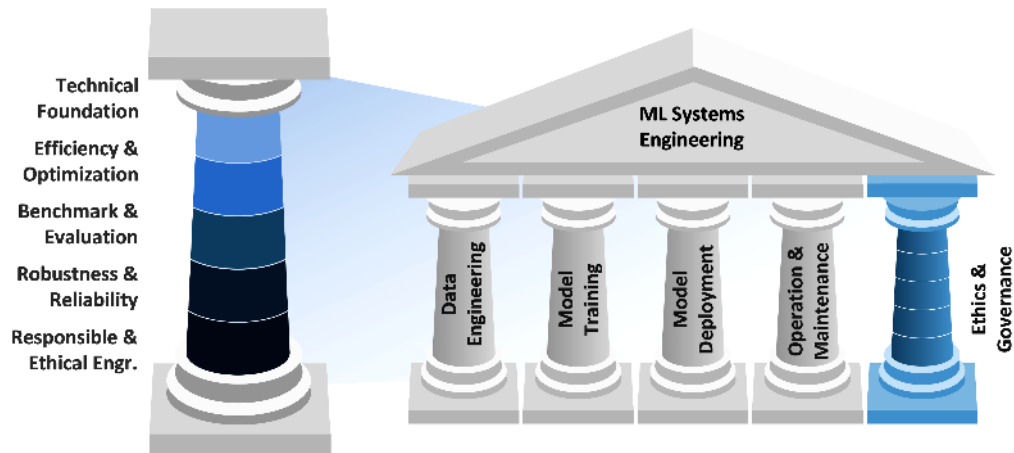


Figure 1.9: Five-Pillar Framework: The architectural foundation of ML systems engineering organized into five core disciplines: Data Engineering, Training Systems (Model Training), Deployment Infrastructure (Model Deployment), Operations & Monitoring (Operation & Maintenance), and Ethics & Governance, supported by foundational efficiency, evaluation, and reliability practices.

If the dataset contains representative acoustic samples, the failure may stem from convergence defects or execution scaling. The **training systems** pillar (chapter 8) manages the boundary between algorithm design and physical compute: coordinating distributed accelerator clusters, optimizing communication across interconnect topologies, staging checkpoint recovery during hardware faults, and balancing numerical precision against compute cost.

A converged checkpoint is a mathematical artifact, not an operational system. The **deployment infrastructure** pillar bridges the training-serving divide. While training optimizes aggregate throughput across batched matrix-multiplication units, inference is bounded by strict tail-latency service-level objectives (SLOs), on-chip memory capacity, and fixed thermal envelopes. This discipline owns runtime quantization, kernel fusion, memory-bandwidth-bound execution, and cross-platform compilation across target devices from edge microcontrollers to data center inference accelerators.

Once deployed, the primary failure mode becomes temporal. The **operations and monitoring** pillar governs system health after launch, when production data distributions drift, acoustic environments shift, and inference accuracy degrades silently while standard infrastructure metrics—CPU utilization, memory headroom, and network throughput—remain green. This discipline couples telemetry, canary rollouts, automated fallback mechanisms, and continuous evaluation pipelines to detect silent model degradation before it impacts users.

Finally, the failure profile may exhibit systematic bias across demographic groups, such as elevated error rates for specific accents or vocal pitch registers, while continuous audio capture introduces strict privacy and regulatory obligations. The **ethics and governance** pillar (chapter 15) translates normative requirements into verifiable engineering guardrails, establishing data provenance, privacy-preserving aggregation, fairness audits, safety limits, and regulatory accountability across the entire system lifecycle.

Alternative taxonomies often organize these concerns strictly by software component or linear lifecycle phase. The five-pillar structure instead mirrors the operational boundaries of production engineering teams while preserving their physical couplings. Upstream data pipelines dictate convergence limits; distributed training choices dictate memory and latency footprints at serving time; deployment runtimes dictate which telemetry operations can extract; and governance policies impose non-negotiable constraints across all four. Isolating ethics and governance as a first-class engineering pillar prevents safety, privacy, and fairness from being treated as discretionary post-hoc checks under schedule pressure.

Together, the pillars translate the conceptual axes of the D-A-M taxonomy (section 1.5.1) and the chronological lifecycle stages (section 1.9.1) into operational engineering ownership. Data engineering anchors the Data axis; training systems and deployment infrastructure navigate the trade-

offs between Algorithm complexity and Machine hardware constraints; operations and monitoring tracks the temporal stability of all three axes; and ethics and governance establishes the operational boundaries within which the system must execute. This taxonomy reflects the fundamental transition of machine learning from isolated algorithmic experimentation to disciplined systems engineering—where the primary objective is sustaining predictable, efficient, and reliable execution on physical hardware.

This progression dictates the architecture of this book. Understanding ML systems requires analyzing physical hardware invariants and data pipelines first, scaling distributed training algorithms across accelerator clusters second, and optimizing compilation and runtime serving under strict latency and energy envelopes.

1.11 Book Organization

The five pillars define what ML systems engineers coordinate; the four parts of this book define the pedagogical sequence for mastering them. The organizing principle is *context before theory*: physical hardware limits and data ingestion bottlenecks dictate algorithmic choices, not the reverse. Readers establish operational boundaries and vocabulary (Part I) before constructing models (Part II), optimizing computational efficiency (Part III), and deploying systems under production service-level agreements (Part IV). Table 1.8 outlines this four-part progression.

Table 1.8: Book Organization: The four parts follow a pedagogical progression from context (Foundations) through theory (Build) to practice (Optimize, Deploy). Each part builds on the vocabulary and frameworks of its predecessors, so Part III's optimization techniques assume familiarity with Part II's model architectures, and Part IV's deployment practices assume mastery of Parts II and III.

Part	Theme	Key Chapters
I: Foundations	Context: ML systems landscape	This chapter, chapter 2, chapter 3, chapter 4
II: Build	Theory: Model fundamentals	chapter 5, chapter 6, chapter 7, chapter 8
III: Optimize	Efficiency: Performance tuning	chapter 9, chapter 10, chapter 11, chapter 12
IV: Deploy	Production: Real-world systems	chapter 13, chapter 14, chapter 15, chapter 16

Part I establishes the physical and architectural vocabulary before model machinery appears. This opening chapter frames AI systems engineering as a discipline governed by physical constraints, introducing the D·A·M taxonomy, the efficiency dimensions, and the five-pillar model. Chapter 2 maps the deployment spectrum from cloud data centers down to milliwatt TinyML microcontrollers, establishing how thermal design power, memory hierarchy depth, and latency budgets govern each tier. Chapter 3 traces the end-to-end lifecycle from problem formulation through production serving, identifying where feedback loops and technical debt accumulate. Chapter 4 analyzes storage I/O bottlenecks, extract, transform, load (ETL) pipelines, and feature stores, demonstrating that data ingestion throughout sets the operational ceiling for downstream model execution.

Part II translates those environmental constraints into model construction skills. Chapter 5 derives the fundamental computational kernels—matrix multiplications, convolutions, and attention mechanisms—and analyzes their computational graphs. Chapter 6 connects these primitives into complete network topologies. Both chapters reference the five lighthouse models introduced in section 1.7.2 (ResNet-50, GPT-2/Llama, MobileNetV2, DLRM, and Keyword Spotting) to anchor computational demands in standardized workloads. Chapter 7 deconstructs computational graphs, automatic differentiation, and runtime execution engines in PyTorch and TensorFlow. Chapter 8 develops distributed execution, gradient synchronization, optimizer memory footprint management, and checkpointing under accelerator memory limits.

Part III alters the terms of the iron law to maximize efficiency without sacrificing accuracy. Chapter 9 reduces training FLOPs by filtering redundant samples, curating high-information subsets, and accelerating time-to-convergence. Chapter 10 relieves memory bandwidth pressure and shrinks parameter footprint through post-training quantization, weight pruning, and knowledge distillation, enabling models to fit inside tight accelerator SRAM and HBM budgets. Chapter 11 analyzes the microarchitectural mechanisms of GPUs and application-specific integrated circuits (ASICs), showing

how systolic arrays, Tensor Cores, and high-bandwidth memory hierarchies deliver high arithmetic intensity. Chapter 12 establishes rigorous protocols to measure sustained compute throughput, tail latency, and energy efficiency, isolating true hardware bottlenecks from measurement artifacts.

Part IV deploys these optimized artifacts into production environments, where runtime dynamics, silent data drift, and operational constraints dominate. Chapter 13 designs inference engines that manage dynamic request batching, KV cache memory allocation, and kernel scheduling under strict latency service-level agreements. Chapter 14 builds continuous telemetry pipelines to monitor prediction health, detect distribution drift, and execute canary rollouts before silent model degradation impacts users. Chapter 15 embeds safety verification, bias auditing, and differential privacy constraints directly into the systems lifecycle. Chapter 16 synthesizes the complete methodology, preparing the reader to scale from single-node mastery to distributed fleet orchestration.

This book covers the **single-node** regime of one host with one to eight accelerators, each typically using local device memory and communicating through an on-node interconnect such as PCIe or NVLink. The binding constraint depends on the workload and may be device memory capacity, memory bandwidth, compute throughput, or interconnect communication. At fleet scale, thousands of nodes coordinate across network fabrics and the bottleneck shifts toward bisection bandwidth, the aggregate capacity across a cut through the cluster network. For detailed guidance on reading paths, learning outcomes, prerequisites, and how to get the most from this textbook, the preface provides the orientation.

The analytical frameworks introduced in section 1.8 and section 1.10 help only if engineers also shed assumptions carried over from adjacent fields. Every mature discipline accumulates intuitions that work within its boundaries but fail when applied elsewhere. ML systems engineering is particularly vulnerable to such imported assumptions because it sits at the confluence of software engineering, statistics, and hardware design. Traditional software assumes deterministic execution; classical statistics assumes stationary data distributions with zero compute cost; hardware design assumes fixed instruction streams. Blending these perspectives without confronting their contradictions leads directly to fragile systems and wasted compute.

1.12 Fallacies and Pitfalls

Assumptions that hold in traditional software, academic research, or pure mathematics fail when applied to systems whose behavior emerges from data. These fallacies and pitfalls capture errors that waste engineering effort, delay deployments, and cause silent production failures.

Fallacy: *Better algorithms automatically produce better systems.*

Engineers often assume algorithmic sophistication drives system performance, but this ignores the iron law (section 1.7). Vision Transformers demonstrate that architecture and large-scale pretraining can produce strong image-recognition results (Dosovitskiy et al. 2021), but production utility still depends on compute, memory movement, and latency budgets. In production, a model that is 1 percent more accurate but violates latency requirements has effectively zero utility. Production model selection is therefore a constrained optimization problem: maximize task quality subject to latency, memory, energy, cost, and reliability budgets. The hidden technical debt surrounding production models shows why model code is only the visible center of a much larger system. A well-engineered system with a simpler model can outperform a more sophisticated architecture lacking robust infrastructure.

Pitfall: *Treating ML systems as traditional software that happens to include a model.*

Engineers apply traditional testing and deployment practices to ML systems, but these systems fail in qualitatively different ways (section 1.6). Traditional bugs often produce immediate exceptions or crashes; ML systems can silently degrade over weeks or months before anyone notices. A/B tests in conventional software may show clear signals quickly, while ML comparisons can require longer observation windows to detect small accuracy differences across subpopulations. Unit tests verify deterministic execution paths; ML systems require monitoring infrastructure to catch unreliable predictions, data drift, and calibration failures. Teams deploying ML with only continuous integration and continuous deployment (CI/CD) pipelines risk silent failures: test suites and health checks report green while user-facing prediction quality degrades.

Fallacy: *High accuracy on benchmark datasets indicates production readiness.*

Benchmark accuracy reflects static, curated test distributions rather than operational reality (section 1.9.2). A sentiment analysis model that performs well on curated test data may fall sharply in production as users employ slang, emojis, and context absent from benchmarks. Furthermore, physical deployment environments introduce hard trade-offs absent from benchmark leaderboards. Mobile accelerators constrain numerical precision and memory capacity, precluding the multi-model strategies that boosted benchmark scores, while network round-trips add substantial latency overhead. Production systems require failure mode analysis across demographic subgroups, continuous monitoring to detect drift, and validation protocols that match actual operating conditions rather than idealized test sets.

Pitfall: *Optimizing individual components without considering system interactions.*

Engineers optimize inference latency in isolation, but Amdahl's law governs end-to-end performance. A team reduces model inference from 45 ms to 15 ms, expecting proportional improvement. Yet preprocessing consumes 60 ms and postprocessing adds 25 ms, so total latency drops only from 130 ms to 100 ms. That is a 23 percent improvement rather than the expected 67 percent. The D·A·M landscape (figure 1.4) shows that the Data, Algorithm, and Machine axes form an interdependent system where optimizing one component shifts bottlenecks rather than eliminating them. Component-level gains do not determine end-to-end improvement; the result depends on how much of the full path the optimized component occupies.

Fallacy: *ML systems can be deployed once and left to run indefinitely.*

Deployed ML systems do not maintain static performance over time: environmental distribution shifts degrade a frozen model even when underlying execution code remains untouched. In this illustrative scenario, a recommendation system deployed at 85 percent accuracy drops to 80.2 percent within 6 months as purchasing patterns shift, losing 4.8 percentage points without any code changes. The ML lifecycle (section 1.9.1) therefore treats outcome monitoring and evidence-based retraining as operational requirements. Fraud detection and natural language processing systems face the same risk as user behavior evolves, adversarial patterns emerge, and vocabulary drifts while the serving binary remains unchanged. Without continuous monitoring, systems appear healthy while prediction quality silently erodes. Organizations that treat deployment as a one-time handoff discover failures only after customer complaints or degraded business metrics force an emergency audit.

Pitfall: *Assuming that ML expertise alone is sufficient for ML systems engineering.*

Organizations hire ML researchers expecting production-ready systems, but the five-pillar framework (section 1.10) requires integrated expertise across algorithms, software, systems, and operations. Teams with strong ML skills but limited systems experience can miss throughput targets because API design, storage layout, and serving infrastructure shape realized performance. Conversely, software infrastructure built without ML awareness can introduce preprocessing or feature bugs that degrade model behavior without obvious system failures. Deployment case studies show that production ML requires coordinated attention to data, models, infrastructure, and organizational workflow, not algorithmic quality alone (Paley et al. 2022). Effective teams integrate ML researchers, software engineers, and operations specialists rather than expecting one role to master all skills. Production failures typically stem from fractures at the boundaries between learned weights, physical execution hardware, and operational workflows.

1.13 Summary

Machine learning systems must satisfy two obligations at once: learned behavior must remain trustworthy, and the machine must deliver that behavior within physical and economic limits. The Software 2.0 shift explains why behavior learned from data can fail silently as distributions change. AI's paradigm history and the bitter lesson explain why progress repeatedly came from systems that could exploit more computation rather than from hand-coded expertise. The D·A·M taxonomy locates the binding constraint, while the degradation equation, iron law, and energy and efficiency frameworks turn those constraints into quantitative diagnoses.



Optimizing only inference leaves end-to-end latency mostly intact.

The lifecycle, deployment spectrum, and production case studies then show why continuous iteration and context-aware design are mandatory. Five lighthouse models (ResNet-50, GPT-2/Llama, MobileNetV2, DLRM, and Keyword Spotting, detailed in chapter 6) recur throughout the book to ground these principles in real workloads.

Return to the smartphone interaction that opened the chapter. What appeared to be one intelligent action depended on representative data, a learned model, a machine within its operating budget, and feedback that could reveal changing behavior. That chain answers the question posed at the outset. Because ML behavior is learned as well as coded, it can degrade without an explicit failure and must be co-designed across data, algorithms, software, and hardware. AI engineering holds that stochastic behavior to deterministic reliability targets.



Key Takeaways: Constraints drive architecture

- **D-A-M bottlenecks migrate rather than disappear.** Data, Algorithm, and Machine constraints interact, so improving one axis often exposes another. The systems habit is to ask which axis now binds, then choose the intervention that relieves that constraint without creating a larger downstream failure.
- **Learned behavior can decay silently.** Traditional software usually fails when code or its environment changes; ML systems can degrade while code and infrastructure stay fixed because the world shifts relative to the training distribution. Drift metrics turn that shift into investigation triggers rather than surprise accuracy loss.
- **The iron law makes latency diagnostic.** Data movement, computation, and overhead all spend from the same time budget. Cutting inference from 45 ms to 15 ms gives only 23 percent improvement when preprocessing (60 ms) and postprocessing (25 ms) dominate, so optimize the term that binds end-to-end behavior.
- **Scale wins inside physical limits.** The bitter lesson explains why general methods with more compute displaced hand-crafted systems, but scale only helps when data, architecture, and machine can support it. Efficiency gains of 44.5× coexisted with roughly 7 orders of compute growth.
- **AI engineering is continuous co-design.** Deployment context, lifecycle monitoring, and the five engineering pillars are not later add-ons; they are how stochastic learned behavior is held to deterministic reliability targets from cloud training through TinyML operation.

Everything this chapter has introduced supports one claim: a machine learning system is governed by physics, not by intention. Its behavior reflects what its data, arithmetic, and hardware permit. The bitter lesson, iron law, degradation equation, and D-A-M taxonomy form a single vocabulary for reasoning about behavior that is learned rather than fully specified and can decay unless maintained. Treating those constraints as the real specification is what turns a collection of techniques into a discipline.



What's Next: From vision to architecture

Selecting where an ML model should run is governed by physical laws. The speed of light makes distant cloud servers useless for emergency braking. Thermodynamics prevents data-center-class models from running on a mobile device. Memory physics creates bandwidth ceilings that faster chips cannot overcome. The four deployment paradigms are where those laws land. Chapter 2 derives each paradigm's operating envelope from the physics and develops the decision framework for choosing among them when requirements conflict.



FOUNDATIONS OF ML SYSTEMS

Part I

Foundation Principles

Machine learning systems exhibit a deceptively simple heuristic: complexity often moves rather than disappearing, although good design can remove accidental complexity. Complexity flows among the three domains of the D·A·M taxonomy: **Data** as information, **Algorithm** as logic, and **Machine** as physics. Simplifying one domain can burden the others. A hand-crafted feature pipeline reduces algorithmic complexity but demands more data engineering effort. A larger model absorbs messy data but shifts complexity onto the hardware that must train and serve it. This **Conservation of Complexity** heuristic motivates everything in this book. The quantitative bounds, models, and principles introduced throughout the book describe constraints that emerge from where complexity currently resides.

Architectures, frameworks, and optimizations succeed only when they respect constraints imposed by hardware, mathematics, and information theory. Just as civil engineers cannot ignore gravity, ML engineers cannot ignore the physical laws that govern data, computation, and system throughput. Part I establishes these foundations: not best practices that evolve with frameworks or opinions that differ between teams, but enduring physical constraints and quantitative models for ML engineering. The first constraint starts with data itself, where the familiar boundary between program and input begins to disappear.

Principle 1: The data-as-code principle

Invariant: Data functions as the source code of the ML system. A change to the training dataset can alter the system’s learned behavior. Here, learned behavior means the trained input-output mapping; f denotes the training process, and $\approx$ marks a qualitative dependency rather than a predictive equality. Model architecture, initialization, optimizer configuration, and training stochasticity are included in the training procedure.

$$\text{Learned Behavior} \approx f(\text{Training Data, Training Procedure})$$

Implication: Data engineering requires the same rigor as software engineering. Datasets should be **versioned** (like Git), **unit-tested** (data quality checks), and **debugged**. Deleting a row of training data can alter the rebuilt model just as deleting a line of code can alter a compiled artifact.

If data functions as source code, then it is not merely a logical artifact; it also has physical properties that constrain system architecture when data and computation occupy different locations.

Principle 2: The data-gravity principle

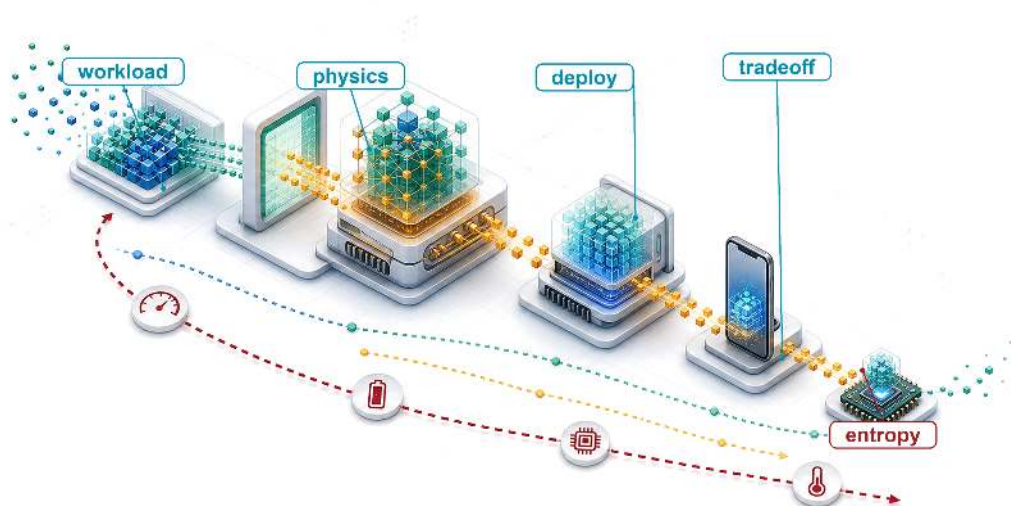
Invariant: As data volume (D_{vol}) grows, transferring the dataset can cost more than sending a much smaller query or code payload over the same constrained link (McCrory 2010). Here, cost denotes one transfer metric at a time, such as elapsed transfer time or bytes sent, under identical link conditions; it does not combine latency and bandwidth into one quantity.

$$\text{Cost to transfer dataset} \gg \text{Cost to transfer query or code} \quad \text{in this regime}$$

Implication: Large datasets can become the gravitational center of the architecture. Systems may move compute to data by shipping queries or code to the storage layer rather than moving data to compute by repeatedly downloading large datasets.

Together, these two principles establish that data can be both the logical program and a physical anchor of an ML system. Part I develops these implications as a dependency chain. Deployment constraints define where a system can execute; the lifecycle carries those constraints through problem definition, data, modeling, validation, and operation; and data engineering makes them enforceable in versioned, tested pipeline artifacts. Ignoring any link returns complexity as integration cost: an accurate model that cannot execute, a workflow that cannot reproduce its inputs, or a pipeline whose outputs no longer match deployment conditions.

ML Systems

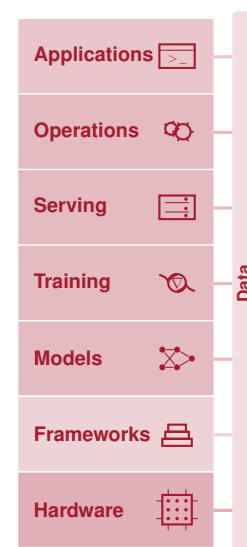


2.1	Deployment Paradigm Framework
2.2	Physical Constraints: Why Paradigms Exist
2.3	Analyzing Workloads
2.4	System Balance and Hardware
2.5	Cloud ML: Computational Power
2.6	Edge ML: Latency and Privacy
2.7	Mobile ML: Offline Intelligence
2.8	TinyML: Ubiquitous Sensing
2.9	Paradigm Selection
2.10	Hybrid Architectures
2.11	System Entropy: Why Deployment Is Not the End
2.12	Fallacies and Pitfalls
2.13	Summary

Purpose

Why does deploying the same model to a phone vs. a data center demand fundamentally different engineering?

The defining insight of ML systems engineering is that constraints drive architecture. The speed of light sets an absolute floor on how quickly distant servers can respond. Thermodynamics limits how much computation can occur in a given volume before heat becomes unmanageable. Memory physics makes moving data often more expensive than processing it. These are not engineering limitations awaiting better technology; they are permanent physical boundaries that partition the world into fundamentally distinct operating regimes. A data center can train billion-parameter models but cannot guarantee low-latency responses to users thousands of miles away. A smartphone can respond instantly but has a fraction of the memory budget. A microcontroller can run on a coin-cell battery for years but has barely enough compute for a simple keyword detector. The same model, the same algorithm applied to the same data, demands radically different engineering in each regime, not because the physics changes but because different constraints dominate. Teams that treat deployment as an afterthought, training in one environment and deferring target constraints until launch, discover too late that the target environment invalidates months of architectural decisions. The engineering task is therefore not to identify a universally best deployment but to locate the constraint that binds in the intended operating regime. In D·A·M terms, understanding these regimes transforms deployment from an operational detail into a first-order co-design problem in which data locality, algorithm structure, and machine constraints jointly determine what is possible.





Learning Objectives

- Explain how physical constraints create deployment paradigms from cloud to TinyML
- Apply the iron law and bottleneck principle to classify compute-, memory-, and I/O-bound workloads
- Map workload archetypes to deployment paradigms using lighthouse model examples
- Compare cloud, edge, mobile, and TinyML by operational constraints and quantitative trade-offs
- Apply the decision framework to select paradigms by privacy, latency, compute, and cost constraints
- Analyze hybrid patterns that combine paradigms to satisfy system constraints
- Evaluate deployment decisions, common fallacies, and universal principles that transfer across scales

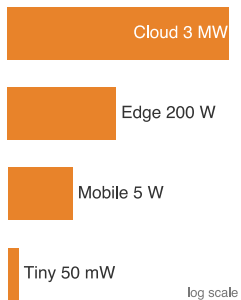
2.1 Deployment Paradigm Framework

CONSIDER TWO extremes: a wake-word detector on a smartwatch and a recommendation engine in a data center. The wake-word detector represents a TinyML workload operating under milliwatt power budgets and kilobyte memory limits; the recommendation engine exemplifies a cloud ML workload requiring terabytes of embedding tables and megawatt-scale infrastructure. These systems solve different problems under opposite physical constraints, and their supporting infrastructure differs sharply in scale and design. This reality transforms deployment from an operational afterthought into a first-order engineering decision, one that the D-A-M taxonomy shown in figure 1.4 helps us reason about by foregrounding infrastructure alongside data and algorithms.

Physical constraints determine where an ML model can run and shape what is possible in ways no algorithmic choice can override. Crucially, the engineering challenge extends far beyond parameter weights: in production systems, the learned model represents only a small fraction of the total operational codebase (Sculley et al. 2015). The surrounding infrastructure—data ingestion pipelines, feature extraction, serving runtimes, telemetry, and hardware resource managers—must conform directly to the host platform’s physical envelope.

Three physical vectors—latency, power dissipation, and memory bandwidth—divide ML deployment into four distinct paradigms, each shaped by hardware trade-offs rather than arbitrary product categories. Centralizing computation in Cloud ML provides elastic compute and large memory capacity, but every inference incurs network latency over wide-area links. Moving workloads to Edge ML eliminates that network round trip and keeps sensitive data on premises (Shi et al. 2016), but limits available compute to local rack power and cooling budgets. On battery-powered devices, Mobile ML delivers responsive interaction directly to handsets and tablets, but must operate within a passive thermal envelope where sustained computation triggers throttling. At the low-power extreme, TinyML runs on microcontrollers (Janapa Reddi, Plancher, et al. 2022), where milliwatt power budgets and kilobyte-scale memory require compact models and integer arithmetic. Spanning nine orders of magnitude in power—from megawatts in data centers to milliwatts on embedded sensors (figure 2.1)—these paradigms reflect the engineering trade-offs between centralized capacity and local execution.

While figure 2.1 illustrates the qualitative topology of distributed intelligence, table 2.1 quantifies the operational envelopes across processing location, round-trip latency, power draw, and memory capacity. This nine-order-of-magnitude power span reflects three inescapable physical boundaries: the speed of light (establishing latency floors), thermodynamic dissipation limits (capping computation per watt), and memory signaling energy (the memory wall).



Power spans the paradigms from megawatts (cloud) to milliwatts (TinyML).

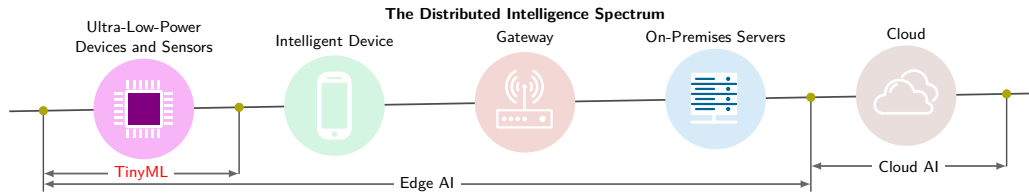


Figure 2.1: Distributed Intelligence Spectrum: Deployment tiers form a continuum governed by physical constraints, spanning nine orders of magnitude in power. Moving from centralized cloud infrastructure to on-device TinyML trades raw computational capacity and elastic scale for deterministic latency, data locality, and network independence.

Table 2.1: The Deployment Spectrum (Conceptual): Four paradigms span nine orders of magnitude in power (MW to mW) and memory (TB to KB). This conceptual overview defines each paradigm by its operating regime; the concrete hardware spectrum later grounds these categories in specific platforms and quantitative decision thresholds. The hardware specifications and physical constants underpinning these numbers are catalogued in the System Assumptions appendix.

Paradigm	Where	Latency (ms)	Power	Memory	Best For
Cloud ML	Data centers	100-500	MW	TB	Training, complex inference
Edge ML	Local servers	10-100	100 W	GB	Real-time inference, privacy
Mobile ML	Smartphones	5-50	3–5 W	GB	Personal AI, offline
TinyML	Microcontrollers	1-10	mW	KB	Always-on sensing

2.1.1 The architectural anchor: The single-node stack

To navigate these operating regimes, we anchor our engineering decisions in a four-layer model of the **single-node stack**, which refines the machine side of the D-A-M taxonomy for one host while data and algorithm enter through workload requirements. At the top of the stack, the application defines the mission through a training-throughput or inference-latency target. Chapter 8 and chapter 13 develop the mechanisms that determine whether the system can meet it. The ML framework translates model code into executable operations (chapter 7). The operating system orchestrates resources and moves data between host memory and accelerator memory. The hardware itself, defined by high-bandwidth memory (HBM) capacity, memory bandwidth, intra-node interconnects such as NVLink, and compute throughput, sets the physical limits, with the memory wall as a recurring constraint (chapter 11).

This stack establishes the chapter’s **silicon contract**: the fixed performance bargain a given piece of silicon offers a model, set by memory bandwidth, peak compute rate, and fixed overhead. Every chapter in the first half of this book analyzes one or more of these layers, because understanding how they interact within a single machine is the technical prerequisite for mastering larger distributed scales.

These physical constraints interact with the iron law of ML systems (section 1.7), which decomposes end-to-end latency into data movement, computation, and overhead. Different deployment environments emphasize different constraints: cloud training can be compute-bound, mobile systems face power walls, and TinyML devices often reach memory-capacity limits. By pairing the physical constraints with the iron law, we develop a quantitative vocabulary for reasoning about which paradigm fits a given workload and *why*. To anchor this analysis concretely, five lighthouse models (ResNet-50, GPT-2, DLRM for embedding-heavy recommendation, MobileNetV2, and a keyword spotter for wake-word detection) span the deployment spectrum and isolate distinct system bottlenecks. These reference workloads recur throughout the book, providing a consistent basis for comparing optimization techniques across chapters, workloads, and deployment tiers.

2.2 Physical Constraints: Why Paradigms Exist

A safety system with a 10 ms reaction budget cannot wait for a cross-country round trip, and a billion-parameter model cannot be squeezed into a microcontroller by better code alone. These are not implementation bugs; they are consequences of the physical laws of speed of light, power

thermodynamics, and memory signaling. Where a system runs reshapes the silicon contract between model and hardware. Three constraints govern the engineering trade-offs ahead: the light barrier, the power wall, and the memory wall.¹

The light barrier

The **light barrier** establishes the absolute latency² floor. Let Distance denote the one-way path length and c_{fiber} the propagation speed in optical fiber. Equation 2.1 gives the minimum round-trip time:

$$L_{\text{lat,min}} = \frac{2 \times \text{Distance}}{c_{\text{fiber}}} \approx \frac{2 \times \text{Distance}}{200,000 \text{ km/s}} \quad (2.1)$$

For optical fiber, $c_{\text{fiber}} \approx 200,000 \text{ km/s}$, roughly two-thirds of its vacuum value because light propagates more slowly through glass than through a vacuum.

California to Virginia (~3,600 km straight-line) requires ~36 ms round-trip before any computation begins. Actual services add routing, protocol, queuing, and processing latency. Applications requiring sub-10 ms response *cannot* use distant cloud infrastructure—physics forbids it. This constraint creates the need for edge ML and TinyML: when latency budgets are tight, computation must move closer to the data source.

The power wall

The **power wall** emerged because thermodynamics limits how much computation can occur in a given volume. Dynamic CMOS power follows equation 2.2, where C is effective capacitance, V is voltage, and f is clock frequency. Holding effective capacitance C constant over a local operating range where voltage rises proportionally with frequency ($V \propto f$), dynamic power rises as f^3 . This cubic relationship arises when pushing clock frequencies beyond nominal bounds, requiring higher supply voltages to maintain transistor switching speeds. When voltage scaling stalls entirely at nominal limits, dynamic power scales linearly with frequency, but heat dissipation constraints still prevent continuous clock scaling.

$$\text{Power} \propto C \times V^2 \times f \quad \text{where } V \propto f \implies \text{Power} \propto f^3 \quad (2.2)$$

Under this illustrative assumption, doubling clock frequency requires approximately 8× more dynamic power. The breakdown of broad voltage scaling³ ended “free” frequency speedups and forced the industry toward parallelism (multi-core) and specialization (GPUs, Tensor Processing Units (TPUs)). Mobile devices hit hard thermal limits at 3–5 W; exceeding this causes “throttling,” where the device reduces performance to prevent overheating. In practice, this means a mobile model that runs at 60 FPS for 1 minute may throttle to 15 FPS as the device heats up. This physical limit gives rise to mobile ML: battery-powered devices cannot simply run cloud-scale models locally.

The memory wall

The **memory wall** (Wulf and McKee 1995) reflects the widening bandwidth⁴ gap. A simple book-level sketch uses representative annual growth factors to show why the gap compounds:

$$\frac{\text{Compute Growth Rate}}{\text{Memory Bandwidth Growth Rate}} \approx \frac{1.6}{1.2} \approx 1.33 \quad (2.3)$$

In equation 2.3, the numerator and denominator are dimensionless annual growth factors. A ratio above 1 means compute capability is pulling away from memory bandwidth, so each hardware generation makes data movement a larger share of the performance problem unless the workload increases locality or arithmetic intensity.

The representative trends behind this ratio are that processors have doubled in compute capacity roughly every 18 months, but memory bandwidth has improved only ~20 percent annually. This widening gap makes data movement a frequent bottleneck and energy cost in ML workloads. Physically, while an arithmetic operation in an on-chip arithmetic logic unit (ALU) moves charges across mere micrometers of local wire (consuming tens to hundreds of femtojoules), fetching an operand from off-chip DRAM requires charging capacitive printed circuit board (PCB) traces and I/O pads centimeters long, consuming hundreds of picojoules—two to three orders of magnitude

¹ | Deployment paradigm:

A distinct operating regime whose boundaries are set by physics, not convention. The Cloud-to-TinyML spectrum spans nine orders of magnitude in power because thermodynamic and electromagnetic constraints create hard walls that no software optimization can cross, forcing qualitatively different system architectures at each tier. Misidentifying the paradigm boundary wastes engineering effort: optimizing a cloud model for 5 percent higher throughput is pointless if the application’s 10 ms latency budget demands edge deployment.

² | Latency:

The time between issuing a request and receiving a result, corresponding to T in the iron law. The light barrier makes this floor irreducible: the speed of light in fiber imposes a ~36 ms minimum round trip across the continental US, consuming the entire latency budget of a 10 ms safety-critical system before any computation begins. Every millisecond consumed by distance is a millisecond unavailable for model inference, which is why the light barrier forces paradigm selection rather than mere optimization.

³ | Dennard scaling:

Named after Dennard et al. (1974) at IBM, who described MOSFET scaling relationships under which shrinking devices could reduce voltage and current while keeping power density approximately controlled. As voltage scaling slowed and energy became a first-order architectural constraint, performance growth shifted toward parallelism and specialization: multi-core processors, GPUs, and TPUs (Hennessy and Patterson 2019; Esmaeilzadeh et al. 2011).

more energy per access. The constraint affects all paradigms but is especially acute for TinyML, where devices have only kilobytes of memory to work with. Section 11.5.1 develops the architectural responses to the memory wall, including HBM and on-chip SRAM hierarchies.

📌 Checkpoint 2.1: Physical constraints and deployment

Deployment choices are governed by physics, not just preference. Check your understanding:

- ❑ **Light barrier:** can you explain why the speed of light makes cloud ML impossible for < 10 ms safety tasks?
- ❑ **Power wall:** can you explain why thermodynamics (heat dissipation) prevents data center models from running on mobile devices?
- ❑ **Memory wall:** can you explain why data movement is often more expensive (in time and energy) than computation?

These physical laws explain *why* the four paradigms exist. Physics creates the boundaries; privacy regulation, economic incentives, and data sovereignty requirements reinforce and sharpen them without creating them. No regulation can make the speed of light faster, and no economic model can repeal thermodynamics.

These physical constraints do not operate in isolation. In practice, an ML system is limited by whichever physical boundary binds on its primary resource first. Evaluating these limits quantitatively requires analytical tools: the iron law to decompose execution time into its components, the bottleneck principle to identify the rate-limiting stage, and workload archetypes to classify the model's resource demands.

2.3 Analyzing Workloads

Given a recommendation engine or wake-word detector, the workload question is which term will bind first on the target hardware. The central analytical tool for answering that silicon-contract question is the iron law of ML systems, established in section 1.7. Here, T is total time, D_{vol} is bytes moved, BW is bandwidth, O is total operations, R_{peak} is peak compute rate, η_{hw} is hardware efficiency, and L_{lat} is fixed overhead; equation 2.4 adds these costs for a serialized execution path:

$$T = \frac{D_{\text{vol}}}{BW} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}} \quad (2.4)$$

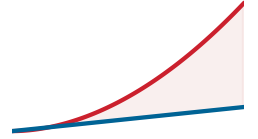
Equation 2.4 decomposes total latency into three terms: data movement (D_{vol}/BW), compute ($O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$), and fixed overhead (L_{lat}). For a single inference, these costs simply add up—each is paid sequentially. In production systems, however, tasks are processed continuously as a stream, and the analysis shifts from single-task latency to identifying which term limits the system. The answer depends entirely on the deployment environment: a model that is **compute bound** during training may become memory bound during inference; a system that runs efficiently in the cloud may hit power limits on mobile devices. In pipelined production serving, hardware concurrency alters this linear sum: faster stages hide behind whichever subsystem saturates first.

2.3.1 The bottleneck principle

The iron law tells us the cost of each term. The **bottleneck principle** tells us which term matters. In a pipelined ML workload, identifying the system bottleneck matters because optimizing fast operations yields zero benefit while the slowest stage remains unchanged. Modern accelerators use **pipelined execution** to overlap data movement with computation: while the accelerator computes on batch n , the memory system prefetches batch $n+1$. When data movement, computation, and I/O are fully overlapped in steady state, let T_{network} denote remote-transfer time and $T_{\text{bottleneck}}$ the resulting per-task time, including fixed overhead. The faster stages hide behind the slowest overlapped stage, so the iron law's sum becomes a maximum, as equation 2.5 formalizes:

$$T_{\text{bottleneck}} = \max \left(\frac{D_{\text{vol}}}{BW}, \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}, T_{\text{network}} \right) + L_{\text{lat}} \quad (2.5)$$

4 | Memory bandwidth (the memory wall): The term “memory wall” was coined by Wulf and McKee in 1995, who predicted that the processor-memory performance gap would eventually dominate system performance—a prediction that proved prescient for ML workloads in which weight or activation movement can bind performance. In the iron law, bandwidth (BW) appears in the denominator of the data term D_{vol}/BW , so doubling the bytes moved without a corresponding bandwidth increase raises the data-movement time. The resulting bottleneck depends on arithmetic intensity, batch size, data reuse, access pattern, and hardware balance rather than model size alone.



Compute capacity outruns memory bandwidth; the widening gap is the memory wall.

- $\frac{D_{\text{vol}}}{\text{BW}}$ (**Memory**): Time to move data between memory and processor.
- $\frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}$ (**Compute**): Time to execute calculations.
- T_{network} : Time for network communication (if offloading).
- L_{lat} (**Overhead**): Fixed latency (kernel launch, runtime overhead).

When a system is memory bound ($D_{\text{vol}}/\text{BW} > O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$), faster processors (R_{peak}) yield no speedup, just as widening a highway cannot help traffic constrained by a narrow bridge. Engineers must identify the dominant term before optimizing. When network transfer is a candidate term, offloading adds energy to the time-based analysis. The local-versus-offload decision must therefore weigh joules as well as milliseconds.

Napkin Math 2.1: The energy of transmission

Problem: Should a battery-powered sensor process data locally (on-device NPU inference) or send it to the cloud when the energy of transmission collides with the battery-driven energy wall?

Given:

- **Data** (D_{vol}): 1 MB (illustrative payload volume).
- **Transmission energy** (E_{tx}): 100 mJ/MB (Wi-Fi/LTE).
- **Compute energy** (E_{op}): 0.1 mJ/inference (illustrative MobileNetV2 neural-processing-unit anchor).

Math:

1. **Cloud approach:** $E_{\text{cloud}} \approx D_{\text{vol}} \times E_{\text{tx}} = 1 \text{ MB} \times 100 \text{ mJ/MB} = 100 \text{ mJ}$.
2. **Local approach:** $E_{\text{local}} \approx \text{Inference} = 0.1 \text{ mJ}$.

Systems insight: Transmitting raw data is 1,000× more expensive than processing it locally. Even if the cloud had infinite speed ($T \approx 0$), the stated battery budget favors local inference over raw-data transmission. The machine constraint (battery) shapes the deployment choice.

The iron law's variables do not scale symmetrically across deployment boundaries; shifting from high-throughput batching to single-sample edge serving radically alters which term dictates performance.

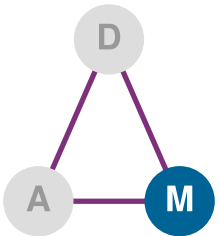
Systems Perspective 2.1: The iron law as deployment diagnostic

The iron law introduced in section 1.7 is the deployment diagnostic used throughout this chapter: it expresses the total time T required for a workload as the sum of data movement, arithmetic, and latency:

$$T = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}}$$

This decomposition is diagnostic: it quantifies how data volume (D_{vol}), compute capacity (R_{peak}), and fixed latency (L_{lat}) jointly set a workload's time budget. Unlike Amdahl's Law, which focuses on parallel speedup, the iron law binds model work to data movement and fixed latency at the deployment boundary. The frequent misconception is that these terms are independent; in reality they are trade-off axes. Increasing batch size can improve duty cycle and data reuse, shifting a memory-bound kernel toward compute-bound behavior, while also increasing the working set and batch service time.

The iron law quantifies the *cost of each ingredient*; the bottleneck principle identifies the *speed of the assembly line*. As a rule of thumb, use the *additive form* in equation 2.4 when analyzing the latency of a single task, and the *max form* in equation 2.5 when analyzing the throughput of a continuous stream of tasks.



The machine axis sets the deployment envelope.

2.3.2 Workload archetypes

The bottleneck principle reduces optimization to identifying the dominant constraint for a given workload. The answer depends on the D·A·M taxonomy in figure 1.4, which decomposes every ML system into Data, Algorithm, and Machine. Different deployment environments create different bottlenecks along these axes, so the same model family can require different engineering when it moves from a cloud server with terabytes of memory to a microcontroller with kilobytes. The iron law turns that diagnostic into four **workload archetypes**.⁵ These are not model categories; they are recurring physical bottlenecks that determine which engineering moves can help.

The first split separates arithmetic-bound systems from data-movement-bound systems. A **compute beast** performs many calculations per byte loaded, so progress comes from higher arithmetic throughput, better utilization, and more parallel execution; large neural-network training is the canonical case. A **bandwidth hog**, by contrast, waits on dense weight or activation movement, so wider memory buses and better data reuse matter more than additional peak FLOP/s; autoregressive text generation illustrates this regime.

The second split covers workloads where the binding constraint is not dense arithmetic at all. **Sparse scatter** workloads are dominated by irregular table lookups and poor cache locality, so memory capacity, access latency, and communication shape performance in recommendation systems with massive embedding tables. **Tiny constraint** workloads face the opposite envelope: energy per inference and memory footprint, not raw speed, determine whether always-on sensing can run at all.



Lighthouse 2.1: Five reference workloads

The five lighthouse models summarized in table 2.2 recur throughout this book as concrete workloads that span the deployment spectrum and isolate distinct system bottlenecks. Chapter 6 provides full architectural details and model biographies.

Table 2.2: Five Lighthouse Models: Recurring workloads used throughout the book to ground the iron law in concrete practice. Each lighthouse pairs an archetype (Compute Beast, Bandwidth Hog, Sparse Scatter, Tiny Constraint) with the deployment paradigm where it predominantly runs, isolating a distinct systems bottleneck.

Lighthouse	Archetype	Deployment Paradigm
ResNet-50	Compute Beast	Cloud training, edge inference
GPT-2/Llama	Bandwidth Hog	Cloud inference
DLRM	Sparse Scatter	Cloud only (distributed)
MobileNetV2	Compute Beast (efficient)	Mobile, edge
Keyword Spotting (KWS)	Tiny Constraint	TinyML, always-on

These archetypes map directly to deployment paradigms. Compute beasts and sparse scatter workloads require the aggregated memory capacity and compute throughput of cloud data centers, bandwidth hogs span cloud and edge depending on latency budgets, and tiny constraint workloads belong on microcontrollers. To evaluate these trade-offs quantitatively, five lighthouse models serve as concrete reference workloads throughout this book, each isolating a specific term of the iron law.

The first lighthouse, **ResNet-50**, classifies images into 1,000 categories, processing each image through approximately 8.2 GFLOP using 25.6 million parameters (102.4 MB at FP32) (He et al. 2016a). Used in medical imaging diagnostics, autonomous vehicle perception pipelines, and as the backbone for content moderation systems, its regular, compute-dense structure makes it the canonical benchmark for hardware accelerator performance.

The language models **GPT-2/Llama** exemplify autoregressive text generation (Radford et al. 2019; Touvron, Lavril, et al. 2023). These models generate text one token at a time. During low-batch decoding, producing each token may require moving most or all model parameters (1.5 billion parameters for GPT-2, 7 billion–70 billion parameters for Llama) through memory. This sequential access pattern creates an autoregressive bottleneck that shapes serving cost (Pope et al. 2023).

⁵ | **Workload archetype:** A classification of ML workloads by their dominant iron law bottleneck rather than their model family. The distinction matters because the optimization strategy differs fundamentally: a compute-bound workload benefits from faster arithmetic (P_{peak}), while a bandwidth-bound workload benefits from wider memory buses or fewer bytes moved through reuse, fusion, compression, or lower precision. Misidentifying the archetype wastes optimization effort on the wrong term of the iron law, as when teams add accelerator FLOP/s to a memory-bound inference pipeline and observe zero speedup.

The recommendation lighthouse, **DLRM**, represents the embedding-heavy recommendation workload behind large-scale “You might also like” systems (Naumov et al. 2019). It maps users and items to embedding vectors stored in tables that can exceed 100 GB of embeddings, making memory capacity rather than computation the binding constraint.

The mobile lighthouse, **MobileNetV2**, is designed for efficient mobile vision tasks such as classification, detection, and segmentation (Sandler et al. 2018). It performs the same image classification task as ResNet but uses depthwise separable convolutions, which separate spatial filtering from channel mixing, to reduce computation by 13.7 $\times$, enabling real-time inference on smartphones at 3–5 W.

The TinyML lighthouse, **keyword spotting (KWS)**, represents the always-on sensing archetype. KWS systems detect short trigger phrases with compact models built for resource-constrained microcontrollers (Y. Zhang et al. 2017); in the Smart Doorbell scenario, that same pattern becomes a local “Ding Dong” or “Hello” trigger. The lighthouse workload has approximately 200K parameters and fits in about 800 KB, placing it in the kilobyte-memory, milliwatt-budget TinyML regime.

Deployment feasibility is evaluated hierarchically, verifying physical memory fit before checking execution latency. In figure 2.2, horizontal bars plot resource demand as a percentage of supply against the vertical 100 percent saturation threshold for the ESP32-S3 microcontroller, comparing Level 1 memory capacity with Level 2 latency under the service level agreement (SLA).

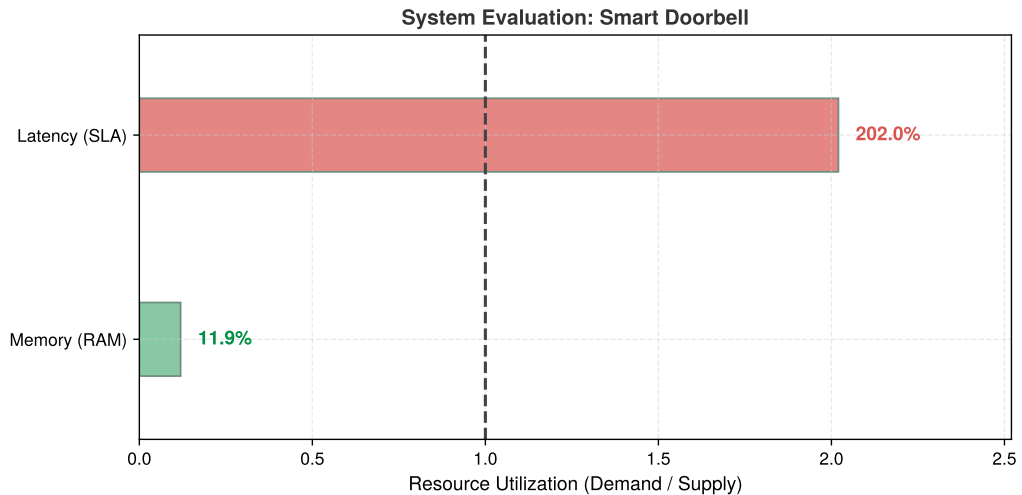


Figure 2.2: Smart Doorbell Constraint Scorecard: Each bar is demand over supply, so the dashed line marks the point where a level is exactly saturated. The model occupies a small fraction of the ESP32-S3’s 4 MB memory budget (Level 1: PASS), but its 101 ms baseline latency is twice the deliberately tightened 50 ms interaction target (Level 2: FAIL), even though it clears the scenario’s canonical 200 ms budget.

The range in compute requirements and memory footprints explains why no single deployment paradigm fits all workloads. A keyword spotter can operate with roughly 20 MFLOP and 800 KB, while ResNet-50 requires about 8.2 GFLOP and roughly 102.4 MB per image. The reference DLRM example already reaches 100 GB, and production DLRM-style recommendation systems can exceed 100 TB. Language models add a bandwidth-dominated regime: billions of parameters streamed repeatedly from memory during autoregressive inference. These five lighthouse models serve as concrete anchors throughout the book, each isolating a distinct system bottleneck revisited in every chapter.

The iron law and workload archetypes gain practical meaning when matched against real silicon. Whether an accelerator achieves high utilization or stalls on memory depends on system balance: the ratio of peak compute throughput to memory bandwidth and I/O capacity across hardware platforms.

2.4 System Balance and Hardware

Physical constraints translate latency-vs-throughput trade-offs into engineering decisions through concrete numbers. Table 2.3 provides order-of-magnitude latencies that should inform every deployment decision—spanning eight orders of magnitude from nanosecond compute operations to hundreds of milliseconds for cross-region network calls. Detailed hardware latencies and bandwidth constraints are covered in chapter 11. An operation with latency $> X$ *cannot* appear on the critical path of a system whose latency budget is X ms.⁶

Table 2.3: Latency Numbers for ML System Design: Order-of-magnitude latencies across compute, memory, network, and ML operations that determine deployment feasibility. Spanning eight orders of magnitude, from nanosecond compute operations to hundreds of milliseconds for cross-region network calls, these physical constraints shape architectural decisions. For a comprehensive quick-reference including energy ratios and scaling rules, see section D.1.

Operation	Latency	Deployment Implication
Compute		
GPU matrix multiply (per op)	~1 ns	Per-operation latency is small
NPU inference (MobileNetV2)	5–20 ms	Mobile can do real-time vision
LLM token generation	20–100 ms	Perceived as “typing speed”
Memory		
L1 cache hit	~1 ns	Keep hot data in registers
Uncached GPU memory read	~200–500 ns	Hide latency; coalesce accesses
DRAM read (mobile)	50–100 ns	Repeated reads can limit workloads
Network		
Same data center	0.5 ms	Microservices feasible
Same region	1–5 ms	Edge servers viable
Cross-region	50–150 ms	Poor fit for tight latency budgets
ML Operations		
Wake-word detection (TinyML)	~1–10 ms	Always-on feasible at < 1 mW
Face detection (mobile)	10–30 ms	Real-time at 30 FPS
GPT-4 first token	200–500 ms	User notices delay
ResNet-50 training step	200–400 ms	Throughput-optimized

The four deployment paradigms gain precision when grounded in concrete hardware. While table 2.1 defined the paradigms conceptually, table 2.6 grounds them in specific devices, memory capacities, and power envelopes.⁷ A nine-order power span, coupled with data center power usage effectiveness (PUE) metrics,⁸ joins a cost spread from millions of dollars down to \$10 to determine which paradigm serves a given workload economically.

These hardware disparities dictate which term of the iron law binds first across deployment regimes (section 2.3.1).

🔍 Systems Perspective 2.2: System balance across paradigms

The pipelined form of the iron law of ML systems from section 1.7 adds BW_{IO} , the storage or network I/O bandwidth, to the canonical compute and memory terms. Equation 2.6 formalizes the resulting lower bound on total execution time T :

$$T \geq \max \left(\frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}, \frac{D_{\text{vol}}}{BW}, \frac{D_{\text{vol}}}{BW_{IO}} \right) + L_{\text{lat}} \quad (2.6)$$

Here, O represents total operations, R_{peak} is peak compute rate, η_{hw} is hardware utilization efficiency, D_{vol} is data volume, BW is memory bandwidth, BW_{IO} is I/O bandwidth (storage or network), and L_{lat} is fixed overhead. The lower bound identifies which resource (compute, memory, or I/O) limits performance; equality requires ideal overlap with no additional stalls. The dominant term varies by workload and deployment regime. Faster accelerators help compute-bound cloud training; memory-bound large language model (LLM) decode and edge workloads benefit more from reducing bytes moved or increasing bandwidth. Table 2.4 works

⁶ **Critical path:** The longest sequential chain of dependent operations in a pipeline. The decision rule in the triggering sentence is strict: if a 200 ms cross-region network call appears anywhere on the critical path, a system with a 100 ms total budget is guaranteed to fail regardless of how fast every other stage runs. Model inference need not be the longest stage; preprocessing and postprocessing can dominate, making the critical path longer than model execution alone suggests.

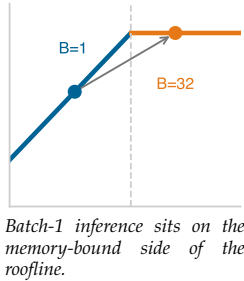
⁷ **ML hardware cost spectrum:** AI infrastructure spans six orders of magnitude in cost, from \$10 microcontrollers to multi-million-dollar accelerator clusters. This million-fold range means deployment paradigm selection is simultaneously a physics decision and an economics decision. Even within individual-device choices, the same accuracy target may be achievable on a low-cost microcontroller only after substantial model reduction, or on an expensive data-center accelerator with a much larger resource budget, with fundamentally different latency, power, and operational cost profiles.

⁸ **Power usage effectiveness (PUE):** This metric isolates facility energy overhead, including cooling and power distribution, from the energy delivered to computing equipment. Even a PUE of 1.06 leaves substantial noncompute power at data-center scale. TinyML does not require dedicated facility cooling, although sensing, communication, conversion losses, and sleep power still contribute to its complete system-energy budget. These device-level costs sit outside PUE.

through all four paradigms, splitting cloud into training and LLM inference, and section A.5.1 maps each dominant term to the optimizations that work and the ones that are wasted, turning this diagnosis into an action plan.

Table 2.4: Dominant Bottleneck by Paradigm: Which iron-law term limits performance in each deployment paradigm, the physical reason it dominates, and the resulting optimization focus. The dominant term shifts the optimization strategy entirely: cloud training maximizes compute utilization, while LLM inference must attack memory bandwidth.

Paradigm	Dominant Constraint	Why	Optimization Focus
Cloud Training	$O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$ (Compute)	Abundant memory/network; FLOP/s limit throughput	Maximize accelerator utilization, batch size
Cloud LLM Inference	D_{vol}/BW (memory bandwidth)	Sequential generation repeatedly moves model state	Increase reuse; reduce bytes moved
Edge Inference	D_{vol}/BW (memory bandwidth)	Limited local bandwidth; models often memory-bound	Smaller models; fewer memory transfers
Mobile	Energy (implicit)	Battery = $\int \text{Power} \cdot dt$; thermal throttling	Lower precision; duty cycling
TinyML	Model footprint must fit on-chip (capacity)	Kilobyte-scale memory; model must fit on-chip	Tiny models and fixed-point arithmetic



Roofline analysis classifies bottlenecks by comparing a workload’s **arithmetic intensity**—the ratio of floating-point operations performed per byte of memory moved (FLOPs/byte)—against the hardware machine balance point ($R_{\text{peak}}/\text{BW}$) (Williams et al. 2009). The framing is informal here; section D.2.1 derives the model in full, defining arithmetic intensity formally and deriving the ridge point that separates the memory-bound and compute-bound regimes. In that framing, the same ResNet-50 model can shift from compute-bound training behavior at high batch sizes to more memory-sensitive single-image inference at batch=1. Deployment paradigm selection must account for this shift.

This operational shift reflects the structure of the D·A·M taxonomy (figure 1.4): every ML system balances Data, Algorithm, and Machine. Table 2.5 shows how each component behaves differently depending on whether the system is training (learning patterns) or serving (applying them).

Table 2.5: D·A·M × Phase: The same model imposes starkly different demands on Data, Algorithm, and Machine depending on whether the system is training or serving. When bottlenecks shift unexpectedly, check which phase is currently being optimized.

Component	Training (Mutable)	Inference (Immutable)
Data	Massive throughput: large batches, shuffling, augmentation	Low latency: single samples, freshness, speed
Algorithm	Learning phase: update model parameters from examples	Prediction phase: apply fixed weights to new inputs
Machine	Throughput-optimized: high-bandwidth clusters, large memory	Latency-optimized: edge devices, inference accelerators

A quantitative comparison applies this analysis to ResNet-50 inference on a high-end data center accelerator and a mobile NPU. Treat these as point-in-time hardware anchors: the arithmetic is about compute rate, memory bandwidth, and batch size, while chapter 11 explains the accelerator architecture behind the numbers.

Napkin Math 2.2: ResNet-50 on cloud vs. mobile

Problem: Is ResNet-50 inference compute bound or memory bound on (a) a high-end data center accelerator and (b) a flagship mobile NPU?

Given (from lighthouse models):

- ResNet-50: 8.2 GFLOP per inference, 25.6 million parameters (102.4 MB FP32, 51.2 MB FP16)

Analysis:**(a) Cloud data-center accelerator (batch=1, FP16)**

- Peak compute: 312 TFLOP/s (FP16)
- Memory bandwidth: 2.04 TB/s
- Compute time: $T_{\text{comp}} = \frac{8.20 \times 10^9}{3.12 \times 10^{14}} = 0.026 \text{ ms}$
- Memory time (weights plus activations): $T_{\text{mem}} = \frac{5.63 \times 10^7}{2.04 \times 10^{12}} = 0.028 \text{ ms}$
- **Result:** Bottleneck = Memory (1.1× slower than compute)
- **Analysis:** Compute per byte moved = $\frac{8.20 \times 10^9}{5.63 \times 10^7} = 146 \text{ FLOP/byte}$. This ratio measures how much arithmetic the workload performs for each byte loaded. When the ratio exceeds the hardware's compute-to-bandwidth ratio ($R_{\text{peak}}/\text{BW}$), the workload is compute bound; below it, the workload is memory bound. For single-image inference, the low batch size yields limited reuse, explaining why even high-throughput accelerators can be memory bound at batch = 1.

(b) Mobile: Flagship NPU (batch=1, INT8)

- Peak compute: ~35 TOPS (INT8)—representative of modern mobile NPUs
- Memory bandwidth: ~51.2 GB/s (LPDDR5)
- Model size: 25.6 MB (INT8 quantized)
- Compute time: $T_{\text{comp}} = \frac{8.20 \times 10^9 \text{ INT8 ops}}{3.50 \times 10^{13} \text{ INT8 ops/s}} = 0.23 \text{ ms}$
- Memory time (weights plus activations): $T_{\text{mem}} = \frac{2.82 \times 10^7}{5.12 \times 10^{10}} = 0.55 \text{ ms}$
- **Result:** Bottleneck = Memory (2.3× slower than compute)

Systems insight: Both platforms are memory bound for single-image inference. The A100's faster memory bandwidth (2.04 TB/s vs. 51.2 GB/s = 39.8×) translates to roughly 19.9× faster inference; peak compute alone is not the limiting comparison. This explains why byte-reduction and lower-precision techniques can beat simply buying more peak FLOP/s for deployment. ResNet-50 becomes compute bound when batching and data reuse raise computation per byte moved above the hardware balance point, $I > R_{\text{peak}}/\text{BW}$, where $I = O/D_{\text{vol}}$. Specifically, during batched inference, the static model weights are transferred across the memory bus only once and subsequently reused across all B samples in the batch. This amortizes the weight transfer over B inputs, increasing the arithmetic intensity until computation time overtakes memory-transfer time. The crossover is architecture- and implementation-dependent because activation traffic, input/output movement, cache reuse, and runtime execution details all change the effective bytes moved per inference.

Model compression becomes essential as the hardware budget tightens. Moving from Cloud to Edge to TinyML contracts available resources across every dimension. Table 2.6 quantifies this progression with concrete hardware examples: memory drops from 128 TB (cloud) to 512 KB (TinyML), a 250 million-fold reduction, alongside a megawatt-to-milliwatt power span. This resource disparity is most acute on microcontrollers, the primary hardware platform for TinyML, where memory and storage capacities cannot accommodate conventional ML models.

Table 2.6: Hardware Spectrum (Concrete Platforms): Representative devices that instantiate each deployment paradigm from table 2.1. Where the conceptual table defines operating regimes, this table provides the specific processors, memory capacities, power envelopes, and price points that practitioners use to match workloads to hardware. The DGX Spark sits at the high end of the edge spectrum; most edge deployments use far smaller devices (for example, Jetson Orin Nano). We include it to illustrate the ceiling of noncloud deployment. NVIDIA’s Jetson family itself spans a wide SKU spectrum, from Jetson Orin Nano (7–15 W) through Jetson Orin NX (10–25 W) to Jetson AGX Orin (15–60 W); throughout this book, Jetson power figures should be read against the specific SKU named in context.

Category	Example Device	Processor	Memory	Storage	Power	Price Range
Cloud ML	Google TPU v4 Pod	4,096 TPU v4 chips, 1.1 EFLOP/s BF16	128 TB HBM2	Cloud-scale (PB)	~3 MW	Cloud service (rental)
Edge ML	NVIDIA DGX Spark	GB10 Grace Blackwell, 1 PFLOP/s sparse FP4	128 GB LPDDR5x	4 TB NVMe	~200 W	~\$3,000–\$5,000
Mobile ML	Flagship Smartphone	Mobile SoC (CPU + GPU + NPU)	8–16 GB	128 GB-1 TB	3–5 W	\$999+
TinyML	ESP32-S3	Dual-core @ 240 MHz	512 KB SRAM	4 MB Flash	0.05 W–1.2 W active board power	\$10

The hardware spectrum turns that bottleneck diagnosis into a fit test. The platforms in table 2.6 are products of decades of hardware evolution, from floating-point coprocessors in the 1980s through graphics processors in the 2000s to today’s domain-specific AI accelerators. Chapter 11 traces this historical progression and the architectural principles that drove it. Here, the consequence matters most: qualitatively different hardware appears at different points in the infrastructure, so each workload must be matched to the region whose compute, memory, power, and cost envelope it can satisfy.

Each paradigm occupies a distinct region of the deployment spectrum, governed by the physical constraints (light barrier, power wall, memory wall) and quantified by the analytical tools (iron law, bottleneck principle) introduced in section 2.3. The quantitative thresholds in table 2.7 help practitioners determine whether a workload fits a target’s compute, bandwidth, power, and latency envelope.

Table 2.7: Deployment Decision Thresholds: Practical envelopes that practitioners use to determine deployment feasibility for each paradigm in table 2.6. These values answer the question “can my workload run here?” by specifying the compute tier, memory bandwidth, and power envelope that each paradigm provides. Each power figure marks a paradigm’s typical class rather than a hard ceiling; the edge envelope in particular extends from low-power embedded modules up to workstation-class devices.

Paradigm	Compute	Memory bandwidth	Power	Latency (ms)
Cloud ML	> 1000 TFLOP/s	> 1000 GB/s	MW class (PUE 1.1–1.3)	100-500
Edge ML	~1 PFLOP/s	> 270 GB/s	100 W class	10-100
Mobile ML	15–45 TOPS	51.2–77 GB/s	3–5 W	5-50
TinyML	< 1 TOPS	—	< 1 mW always-on average target	1-10

Table 2.7 completes the fit test: each paradigm represents an operating envelope defined by a different binding resource. Moving from warehouse data centers down to microcontrollers traces a physical gradient that trades raw compute throughput for energy and memory efficiency.

2.5 Cloud ML: Computational Power

Consider what it took to train GPT-3:⁹ 3,634.3 PFLOP-days of computation, 10,000 V100 GPUs running for approximately 15 days, consuming megawatts of power (Patterson et al. 2021). No smartphone, edge server, or single machine could have completed this training run on a practical schedule. It required a data-center-scale cluster with enough aggregate compute, memory, and storage. This is the defining proposition of cloud ML: when latency can be tolerated, it offers computational scale that no other paradigm can match.

⁹ | **Large language model (LLM) training scale:** Only centralized data centers can aggregate enough R_{peak} for 3,634.3 PFLOP-days training runs (Patterson et al. 2021). Remote inference adds network latency, making pure cloud execution infeasible when the end-to-end path exceeds the application’s latency budget.

¹⁰ | **Cloud as utility computing:** The utility model allows providers to offer a specialized hardware portfolio that is economically infeasible for a single organization to maintain. This provides direct, on-demand access to the specific architectures required by each workload archetype: dense accelerator pods for Compute Beasts, HBM-equipped nodes for Bandwidth Hogs, and high-memory systems with fast interconnects for Sparse Scatter. A team can therefore rent a purpose-built, \$10M+ supercomputing pod for a few hours rather than owning it.

Centralizing these resources under a utility model¹⁰ allows providers to assemble specialized accelerator topologies tailored to distinct hardware demands. This infrastructure serves as the natural home for three of the four workload archetypes: Compute Beast workloads like ResNet training that demand sustained TFLOP/s across thousands of accelerators, Bandwidth Hog workloads like large language model inference that benefit from TB/s HBM bandwidth, and Sparse Scatter workloads like recommendation systems that require terabytes of embedding tables and high-bandwidth interconnects for all-to-all communication patterns.

Cloud deployments scale from single-accelerator instances and multi-GPU servers to distributed clusters spanning warehouse data centers. Systems engineering at this scale begins on a single node: before coordinating computation across networked hosts, an engineer must first understand memory bus saturation, PCIe and NVLink bandwidth, and accelerator utilization within a single machine.

This capacity-versus-distance trade-off defines the cloud paradigm.

Definition 2.1: Cloud ML

Cloud machine learning is the deployment paradigm that trades latency for elastic compute by locating ML workloads in centralized data centers, decoupling computational capacity from the physical location of data sources and users.

1. **Significance:** Cloud deployment dominates the R_{peak} term: a single cloud region can provision thousands of accelerators on demand, delivering aggregate throughput that no typical on-premises installation can match economically. The trade-off is the L_{lat} term: a minimum round-trip latency of 10–100 ms (set by the speed of light over continental distances) makes cloud infeasible for any workload requiring sub-10 ms response.
2. **Distinction:** Unlike edge ML, which prioritizes latency determinism and data locality at fixed R_{peak} , cloud ML prioritizes elastic R_{peak} at the cost of variable L_{lat} .
3. **Common pitfall:** A frequent misconception is that cloud ML is “unlimited compute.” In reality, the distance penalty (L_{lat}) and the ingestion bottleneck (D_{vol}/BW) are physics constraints that no software optimization can eliminate, setting a hard floor on response time for any workload whose data originates outside the data center.

Centralization defines the cloud bargain: pooling elastic compute across warehouse-scale clusters eliminates local capacity limits, but introduces physical distance and network latency. In figure 2.3, the constraint tree maps this trade-off from left to right, connecting the binding scale constraint to its system response, failure boundaries, and well-matched workloads.

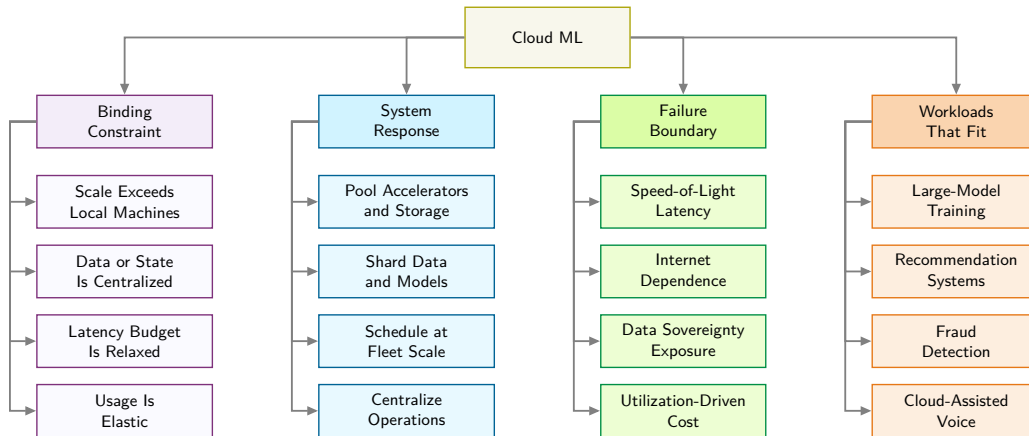


Figure 2.3: Cloud ML Constraint Map: Centralized infrastructure solves the scale problem for compute beasts, bandwidth hogs, and sparse scatter workloads, but it introduces the distance, dependency, privacy, and cost constraints that determine when cloud ML stops being the right deployment target.

Workloads that thrive in this regime—such as asynchronous fraud detection, batch recommendation updates, and large language model pretraining—tolerate this bargain because scale matters more than instantaneous response. But when an application requires real-time control, network latency ceases to be an engineering inconvenience and becomes a hard physical barrier. A quick calculation of the distance penalty makes this concrete.

Napkin Math 2.3: The distance penalty

Problem: Consider a real-time safety monitor for a robotic arm. The safety logic requires a 10 ms end-to-end response time to prevent injury, so the distance penalty imposed by the light barrier matters. The model runs in a high-performance cloud data center 1,500 km away. Can the safety budget be met?

Physics:

1. **Light in fiber:** $\sim 200,000$ km/s.
2. **Round-trip propagation:** $(1,500 \text{ km} \times 2) / 200,000 \text{ km/s} = 15 \text{ ms}$.
3. **Result:** Round-trip propagation alone requires 15 ms, exceeding the 10 ms end-to-end budget (~ 5 ms headroom) before the model performs any inference.

Systems insight: Physics has made cloud ML *impossible* for this application. The model must move to the edge.

2.5.1 Cloud infrastructure and scale

Cloud ML concentrates computational resources in centralized facilities at physical dimensions unmatched by local devices. TPU supercomputer designs organize thousands of specialized accelerator chips into data-center-scale systems that deliver PFLOP/s-to-EFLOP/s reduced-precision throughput (Jouppi et al. 2023). Table 2.6 quantifies how cloud systems provide orders-of-magnitude more compute and memory bandwidth than mobile devices, at correspondingly higher power and operational cost. These facilities enable workloads that are impractical on resource-constrained devices, but their remote location introduces direct trade-offs: network round-trip latency rules out tight real-time deadlines, and operational costs scale directly with usage.

The physical reality of this throughput is visible on the facility floor in figure 2.4: thousands of accelerator chips¹¹ are organized into rows of liquid-cooled racks, each rack consuming kilowatts of power to sustain aggregate compute that no single machine can approach.

Cloud ML excels at processing massive data volumes through parallelized architectures, enabling training on datasets requiring hundreds of terabytes of storage and PFLOPs of computation—resources that remain impractical on constrained devices. The training techniques in chapter 8 and the accelerator architectures in chapter 11 examine how distributed systems coordinate and sustain this throughput.

Centralization also changes how models are served and governed. Cloud APIs make trained models accessible worldwide across mobile, web, and IoT platforms. Shared infrastructure enables engineering teams to collaborate with unified version control, while pay-as-you-go pricing models¹² convert upfront capital expenditure (CapEx) into usage-based operational expenditure (OpEx) that scales with demand.

A common misconception holds that cloud ML's vast computational resources make it universally superior. High computational capacity and storage do not automatically translate to optimal solutions for every workload. The data gravity invariant in section B.1.1 explains why: as data volume scales, the cost of moving data to compute ($C_{\text{move}}(D_{\text{vol}}) \gg C_{\text{move}}(\text{Compute})$) eventually dominates. The trade-offs illustrated in figure 2.3 become concrete where edge and embedded deployments excel: sub-10 ms control loops in robotics, local retention for privacy-sensitive medical records, predictable amortized costs without recurring cloud fees, or autonomous operation in disconnected environments. The optimal deployment paradigm is dictated by the binding physical constraint rather than unconstrained throughput.

¹¹ | **Tensor processing unit (TPU):** An application-specific integrated circuit (ASIC) that delivers PFLOP/s-scale throughput by hard-wiring its architecture for the matrix multiplication operations that dominate ML workloads. This extreme specialization trades general-purpose flexibility for a $> 10\times$ improvement in performance-per-watt compared to a general-purpose accelerator on the same ML task. The high cost of deploying these accelerators at data center scale is therefore only economical for massive, sustained ML computation.

¹² | **Pay-as-you-go pricing:** A cloud economic model where users pay for accelerator-hours consumed rather than hardware owned. Elastic pricing converts the fixed cost of idle R_{peak} into a variable cost proportional to actual utilization, but sustained 24/7 workloads can cost more on cloud than equivalent on-premises hardware amortized over three years, a crossover that drives the total cost of ownership (TCO) analysis in table 2.8.

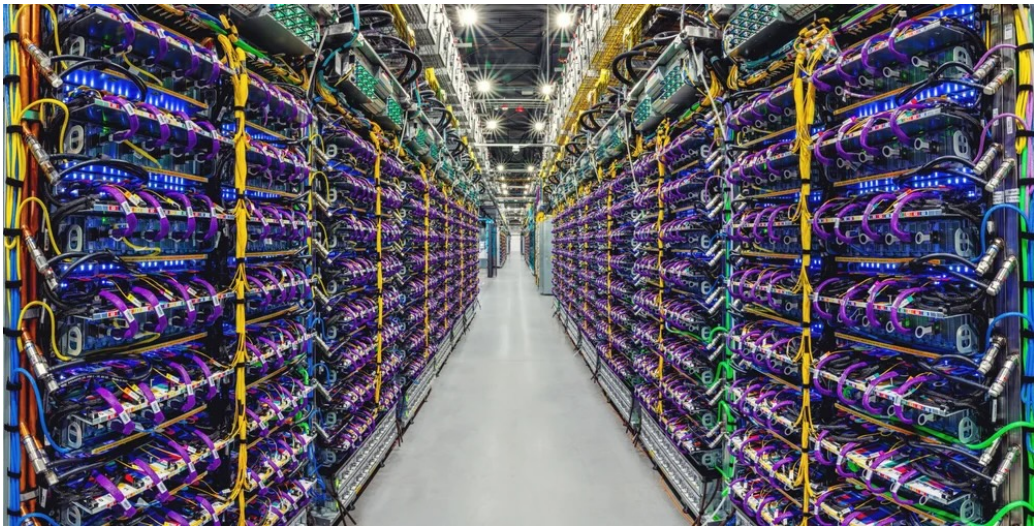


Figure 2.4: Cloud Data Center Scale: Dedicated accelerator facilities aggregate thousands of specialized TPU chips in liquid-cooled racks to deliver exaFLOP-scale matrix throughput. This centralized infrastructure makes large-scale model training and high-throughput serving viable, but incurs megawatt-scale power consumption and speed-of-light network distance penalties. Image source: (Pichai 2023).

2.5.2 Cloud ML trade-offs and constraints

Cloud ML’s advantages carry inherent trade-offs that shape deployment decisions. Latency is the most consequential: remote network latency makes cloud processing unsuitable when the end-to-end path exceeds an application’s response budget. Unpredictable network jitter further complicates performance predictability and debugging across geographically distributed infrastructure.

Privacy and security pose additional boundaries for cloud deployment. Transmitting sensitive data to remote data centers expands the disclosure surface and complicates regulatory compliance. Organizations handling data subject to regulations such as the General Data Protection Regulation (GDPR)¹³ or the Health Insurance Portability and Accountability Act (HIPAA)¹⁴ must select, document, and maintain safeguards appropriate to the data, system, and applicable legal duties. Privacy-preserving approaches can reduce how much sensitive data leaves its original environment, but they complement rather than replace infrastructure controls.

Cost management introduces operational complexity requiring TCO¹⁵ analysis rather than naive unit comparisons. A worked cloud vs. edge TCO comparison illustrates the gap between sticker price and true system cost.

For the worked comparison, table 2.8 itemizes the annual GPU, network, load-balancer, and observability costs of an illustrative cloud implementation under public list pricing, and table 2.9 itemizes the corresponding hardware, power, cooling, network, and DevOps labor costs of an on-premises edge-server implementation.

Table 2.8: Cloud Inference Annual TCO: Itemized GPU, network, load-balancer, and observability costs for the fixed-capacity cloud implementation of the ResNet-50-scale vision workload.

Cost Component	Calculation	Annual Cost
GPU inference (A10G)	4 instances × 8760 h/year × \$0.75/hr	~\$26,280
Network egress	100 GB/day × 365 d/year × \$0.09/GB	~\$3,285
Load balancer	\$0.025/hr + LCU charges	~\$3,723
CloudWatch/logging	Monitoring, alerts	~\$2,000
Total Cloud		~\$35,288/year

¹³ | **GDPR (General Data Protection Regulation):** Article 17 provides a right to erasure when specified grounds apply (European Parliament and Council of the European Union 2016). For an ML system, an erasure request requires tracing where the person’s data persists across source stores, derived features, logs, caches, and model artifacts. Retraining may be one remediation, but the regulation does not prescribe re-training every model that encountered the data.

¹⁴ | **HIPAA:** The Security Rule requires regulated entities to implement reasonable and appropriate administrative, physical, and technical safeguards for electronic protected health information, including access and audit controls (U.S. Department of Health and Human Services 2005). The rule is technology-neutral and does not mandate confidential-computing enclaves; concrete controls and their performance overhead depend on the system’s risk analysis and implementation.

Table 2.9: Edge Inference Annual TCO: Itemized hardware, power, cooling, network, and DevOps labor costs for the on-premises edge-server implementation, exposing labor as the dominant component of this scenario.

Cost Component	Calculation	Annual Cost
Hardware CapEx	$\$15,000 \div 3 \text{ years life}$	~\$5,000
Power (24/7)	$300 \text{ W} \times 8760 \text{ h/year} \times \$0.12/\text{kWh}$	~\$315.4
Cooling overhead	~30% of power	~\$94.6
Network (fiber)	Fixed line for remote management	~\$1,200
DevOps labor	$0.1 \text{ FTE} \times \$150,000 \text{ salary}$	~\$15,000
Total Edge		~\$21,610/year

Napkin Math 2.4: Cloud vs. edge TCO

Problem: A vision system serves 1M daily inferences at ResNet-50 scale (10 ms latency, 100 KB response). When all costs are included (GPU hours, network egress, power, cooling, and labor, itemized in table 2.8 and table 2.9), is cloud or on-premises edge deployment cheaper over 3 years?

Math: This scenario provisions a fixed pool of 4 cloud GPU instances and one fixed on-premises server. Most costs are fixed, while cloud egress varies with traffic. The ratio of the two annual totals is therefore not a valid break-even utilization. The valid comparison in equation 2.7 at the stated operating point is the annual cost gap:

$$\text{Annual cost gap} = \text{Cloud annual TCO} - \text{Edge annual TCO} \quad (2.7)$$

At the stated 1M-inference/day operating point, the gap is $\sim\$35,288/\text{year} - \sim\$21,610/\text{year} = \sim\$13,678/\text{year}$. Multiplying this traffic by the ratio of the annual totals would incorrectly treat provisioned capacity as a per-inference expense.

Result: Edge costs about 38.8 percent less at this operating point. Because the fixed cloud configuration already costs more than the edge total before traffic-dependent egress, these assumptions do not produce a positive traffic crossover. A valid break-even requires demand-scaled cloud resources, an explicit capacity staircase, or a defensible per-query price.

Systems insight: Edge TCO is dominated by labor ($\sim\$15,000 \div \sim\$21,610/\text{year} \approx 69.4 \text{ percent}$), not hardware. Organizations without existing DevOps capacity should factor in the full cost of maintaining on-premises infrastructure.

Beyond direct financial costs, cloud deployments carry distinct operational constraints. Unpredictable usage spikes complicate budgeting, requiring automated monitoring and cost governance frameworks. Network dependency creates an availability hazard: any connectivity disruption between clients and the data center immediately degrades or halts service. Proprietary cloud APIs compound this with vendor lock-in, creating software portability friction when migrating workloads across providers. Even with these operational constraints, centralized infrastructure remains uniquely suited to consumer applications that require elastic capacity at global scale.

2.5.3 Large-scale training and inference

Cloud ML's computational advantages manifest most visibly in consumer-facing applications that require massive scale. Virtual assistants like Siri and Alexa illustrate the hybrid architectures that characterize modern ML systems: wake-word detection can run on dedicated sub-milliwatt-to-milliwatt hardware directly on the device, enabling always-on listening without continuously activating the main processor; initial speech recognition can run on-device for privacy and responsiveness; and complex natural language understanding and generation can use cloud infrastructure for access to larger models and broader knowledge.

Economics drive this architecture as much as latency. Attempting to process voice interactions for billions of devices entirely in the cloud runs into both an economic and an infrastructure ceiling. Quantifying the voice assistant wall shows both limits at once.

Napkin Math 2.5: The voice assistant wall

Problem: In this illustrative capacity scenario, 1B voice assistant devices (smartphones, smart speakers, earbuds) each issue 20 queries/day as wake-word traffic. What would the infrastructure scaling cost if all queries were served from cloud ML, and how many dedicated data-center equivalents would peak load require?

Economic wall: First, the cost of serving wake-word traffic from the cloud.

- **Cloud cost:** ~\$0.50/device/year → 1B devices = \$500,000,000/year. Economically prohibitive under these assumptions for a free feature.
- **TinyML alternative:** 0.1–1 mW local wake-word detection, <\$0.01/device/year. Viable at the modeled scale.

Infrastructure wall: Second, the number of data centers peak load would require.

Five parameters determine the required cluster capacity:

1. **Query volume:** 1 billion devices × 20 queries/day = 20B queries/day.
2. **GPU demand:** Each query requires ~200 ms of GPU time. Total: 1,111,111 GPU-hours/day.
3. **Scenario capacity:** A data-center equivalent provisioned with 10,000 GPUs provides 240,000 GPU-hours/day.
4. **Average requirement:** ~4.6 dedicated data centers just for voice inference.
5. **Peak reality:** Queries cluster in waking hours (~4.5× peak-to-average), requiring ~20.8 data centers at peak.

Bandwidth upper bound: Third, consider the separate counterfactual of moving the audio itself. If every device streamed audio continuously (16 kHz, 16-bit), each would transmit ~32 KB/s. Across 1 billion devices, the aggregate would be 32 TB/s, an illustrative upper bound rather than the query-based request model.

Systems insight: Serving voice interactions entirely from the cloud creates an unsustainable concurrency bottleneck at global scale: even before accounting for operating expense, peak query volumes demand dedicated data-center capacity that exceeds feasible fleet allocation. Local wake-word detection replaces this centralized bottleneck with distributed, milliwatt-scale inference.

The voice assistant pipeline illustrates a core systems principle: constraints drive hierarchical architecture. Partitioning the pipeline into an on-device wake-word filter and a cloud-based language model relieves both the client’s energy limit and the data center’s concurrency wall. The hybrid approach reduces end-to-end latency relative to pure cloud processing while preserving access to the computational capacity needed for complex language understanding.

Large-scale recommendation engines demonstrate the cloud bargain in a memory-capacity-bound form. Systems based on architectures such as DLRM¹⁶ use sparse embedding tables to map millions of users and items to dense vectors. Because these tables routinely exceed terabytes in aggregate size, no single accelerator can store the model parameters. Instead, the tables are sharded across distributed host memory and accelerator pools, requiring high-bandwidth interconnects for all-to-all communication during forward and backward passes. Centralized data centers provide the pooled memory capacity and low-latency cluster fabrics necessary to host these sparse models while continuously ingesting streaming user interactions.

These applications share a common thread: they trade latency for scale, accepting remote-path delay in exchange for computational resources that cannot be supplied locally. Fraud detection systems analyzing millions of transactions, recommendation engines processing terabytes of embedding tables, and language models generating text token by token all depend on this bargain. Yet as the voice assistant wall demonstrated, applications exist where no amount of cloud compute can compensate for the physics of distance. When latency budgets drop below what the remote path permits, or when data volumes exceed what networks can carry, computation must move closer to the data source.

¹⁶ | **Deep learning recommendation model (DLRM):** Meta’s 2019 architecture that exemplifies the “Sparse Scatter” archetype. Embedding tables for production recommendation systems can exceed 100 TB, making DLRM constrained by memory capacity and communication BW rather than raw R_{peak} . This inversion of the typical compute-bound assumption forces specialized cluster designs where memory, not arithmetic, is the scarce resource.

2.6 Edge ML: Latency and Privacy

When a remote path exceeds a workload’s latency budget, cloud inference becomes physically impossible. The distance penalty imposes network latency before computation begins: speed-of-light limits and routing hops over wide-area networks add tens to hundreds of milliseconds of propagation delay. For an autonomous-vehicle braking path or an industrial safety loop,¹⁷ a 100 ms network round trip consumes the entire response budget before a single layer evaluates. Moving computation adjacent to the data source eliminates this remote transmission term.

Edge ML trades elastic cloud capacity for sub-100 ms latency determinism and local data retention. In archetype terms, edge deployment shifts the binding constraint: a Bandwidth Hog workload that is memory bound in the cloud can become dominated by remote-path latency when offloaded over wide-area links. Local hardware with sufficient memory eliminates that network transit penalty, returning the bottleneck to local memory bandwidth and compute. Under the iron law from equation 2.6, local execution removes the offload term $D_{\text{vol}}/\text{BW}_{\text{IO}}$ from the request path, leaving execution time bounded by local compute and memory transfer.

Beyond real-time control deadlines, data sovereignty and regulatory privacy constraints often prohibit transmitting raw sensor streams across external networks. Edge devices—spanning local gateways, industrial PCs, and on-premises servers—occupy an intermediate tier in the deployment spectrum, balancing moderate compute resources against tight physical and thermal boundaries.



Definition 2.2: Edge ML

Edge machine learning is the deployment paradigm optimized for latency determinism and data locality by locating computation physically adjacent to data sources.

1. **Significance:** It circumvents the distance penalty (L_{lat}) of the cloud, trading elastic scale for a fixed local compute capacity (R_{peak}).
2. **Distinction:** Unlike Cloud ML, which prioritizes throughput, edge ML prioritizes determinism and privacy. Unlike TinyML, edge ML may still use workstation-class accelerators such as general-purpose GPUs (GPGPUs).
3. **Common pitfall:** A frequent misconception is that edge ML refers to a specific hardware class. In reality, it is a location paradigm: it spans from IoT gateways to on-premises servers, unified by physical proximity to the data source.

Moving computation to local servers removes long-haul transmission from the request path at the cost of managing distributed infrastructure. In figure 2.5, the constraint tree traces how physical proximity to sensors dictates local filtering and dedicated accelerator hardware while exposing fixed capacity boundaries.

Physical necessity dictates edge processing when sensor ingestion rates exceed network uplink capacity. When incoming sensor throughput outpaces available network bandwidth, the uplink becomes an impassable bottleneck, forcing computation to the local device regardless of other considerations.



Napkin Math 2.6: The bandwidth bottleneck

Problem: Consider a quality control system for a factory floor with 100 cameras running at 30 FPS with 1080p resolution. Does video streaming become the bandwidth bottleneck, and does edge ML reduce the bandwidth enough to process locally?

Physics:

1. **Raw data rate per camera:** $1920 \times 1080 \times 3 \text{ bytes} \times 30 \text{ FPS} \approx 186.6 \text{ MB/s}$.
2. **Total data rate:** $100 \text{ cameras} \times 186.6 \text{ MB/s} = 18.7 \text{ GB/s}$.
3. **Cloud transfer exposure:** Uploading raw camera feeds is primarily a bandwidth, ingest, storage, and processing problem; per-GB cloud egress charges apply when data

¹⁷ | **Industrial IoT (IIoT):** A domain where latency constraints can be set by physical safety rather than user perception. A safety loop with a 5 ms halt budget cannot admit a 100 ms remote round trip, so its sensing and stop decision must remain local. That choice removes the long-haul network term but leaves the edge system bounded by its local compute capacity (R_{peak}).

Raw 18.7 GB/s

10G 1.25 GB/s

Raw edge data can be wider than the network pipe.

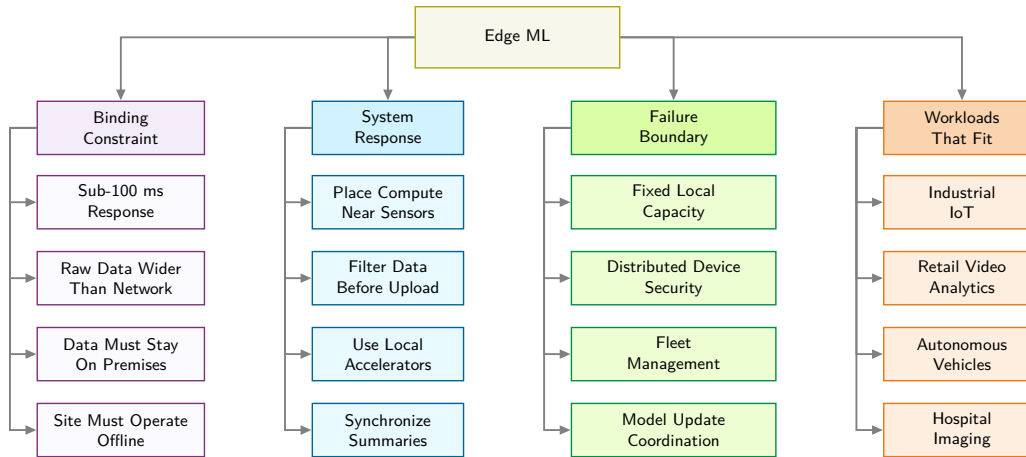


Figure 2.5: Edge ML Constraint Map: Moving computation near the data removes the network-latency term and reduces bandwidth pressure, but it replaces centralized scale with fixed local capacity, distributed security exposure, and operational complexity across many sites.

is transferred back out of the cloud. If the raw stream were later retrieved at \$0.09/GB data-transfer-out pricing, the transfer charge alone would reach \$4.4M/month.

4. **Network reality:** Even a dedicated 10 Gbps line (1.25 GB/s) cannot carry the load—the workload demands 14.9× more bandwidth than exists.

Systems insight: Physics has made cloud streaming *impossible* for this application. Edge processing is not optional—it is mandatory. If an edge server transmits only defect metadata (1 KB per detection at roughly 20 events/s across the floor), bandwidth falls by about 933,120×.

Saturating an uplink with uncompressed sensor streams demonstrates the bandwidth barrier of centralized ingestion. For battery-powered edge devices (wireless cameras, drones, wearables), the energy cost of wireless communication introduces an even stricter physical ceiling: under the payload and radio parameters in napkin math 2.1, radio transmission consumes 1,000× more energy than executing inference locally. Offloading raw telemetry over cellular or Wi-Fi links rapidly drains onboard energy reserves. In figure 2.6, the energy spectrum across deployment tiers spans eight orders of magnitude, illustrating why edge and embedded systems require specialized low-power architectures rather than continuous wireless streaming.

2.6.1 Edge ML benefits and deployment challenges

Edge systems span wearables, industrial controllers, and on-premises gateways that process data locally¹⁸ without depending on central servers. While the eight-order energy span across paradigms reflects distinct workload demands and accounting boundaries, the physical governing principle remains constant: power dissipation is strictly capped by battery capacity or passive thermal dissipation.

This thermal envelope directly enforces a memory-capacity and bandwidth ceiling. Because edge accelerators operate within strict 5–50 W power budgets, they cannot support the wide, high-power physical interfaces required by High Bandwidth Memory (HBM). Instead, edge architectures rely on LPDDR or shared system DRAM, where memory capacity restricts model parameter scale and memory bandwidth restricts operational throughput. Fitting modern neural networks into these physical bounds requires the model-compression techniques examined in chapter 10—quantization, pruning, and architectural distillation—to reduce weight volume and activation traffic to fit hardware budgets.

¹⁸ | **Internet of things (IoT) data wall:** McKinsey estimates that IoT deployments could create trillions of dollars in economic value by 2030, but those deployments depend on continuous sensor streams from devices distributed across homes, factories, farms, vehicles, and infrastructure (McKinsey Global Institute 2021). In many of those settings, the aggregate D_{vol} from raw streams overwhelms the available uplink budget or latency budget for centralized ingestion, making local edge processing an architectural requirement rather than merely a cost optimization.

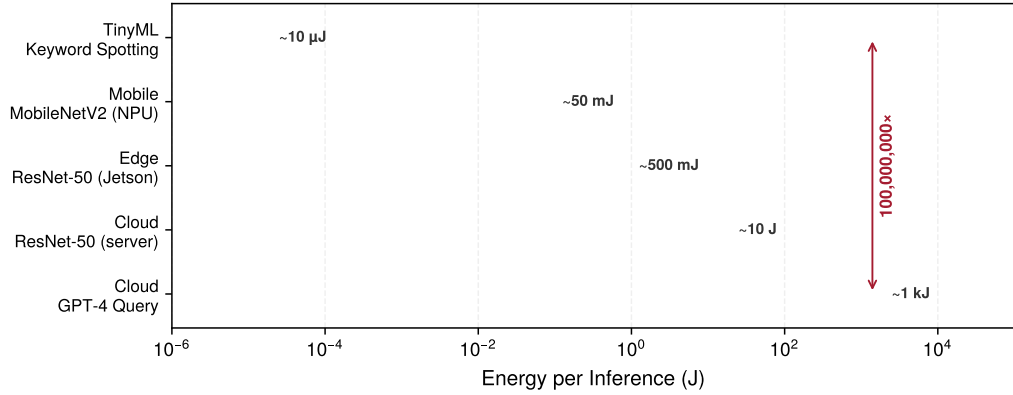


Figure 2.6: Illustrative Energy per Inference Anchors: An eight-order-of-magnitude energy gap separates battery-constrained TinyML keyword spotting (~10 μJ) from cloud foundation-model queries (~1 kJ) on a log scale. This physical divergence explains why always-on edge sensing requires specialized low-power architectures rather than continuous wireless streaming to the cloud. (Plotted values represent pedagogical scenario anchors rather than full-system node model outputs).

2.6.2 The data locality invariant

The decision between local edge processing and remote cloud processing is governed by a bandwidth-latency trade-off. Remote execution is infeasible when its complete path violates the service deadline; otherwise, the faster feasible path follows from comparing complete local and remote times.



Definition 2.3: The data locality invariant

The data locality invariant first tests a complete remote path against the service deadline. Let T_{remote} denote complete remote-path time, T_{deadline} the deadline, BW_{network} offload bandwidth, $L_{\text{lat,network}}$ network latency, and $R_{\text{peak,remote}}$ and $\eta_{\text{hw,remote}}$ the remote compute rate and efficiency:

$$T_{\text{remote}} = \frac{D_{\text{vol}}}{BW_{\text{network}}} + L_{\text{lat,network}} + \frac{O}{R_{\text{peak,remote}} \cdot \eta_{\text{hw,remote}}}, \quad \text{remote feasible} \iff T_{\text{remote}} \leq T_{\text{deadline}}$$

The corresponding T_{local} is the complete local-path time. If both paths are feasible, compare T_{local} and T_{remote} ; either path can be rejected by the deadline. Here, $L_{\text{lat,network}}$ captures the full network round-trip time (propagation, routing, and protocol handshakes), while the return payload transmission time is typically omitted because inference results (such as a class label or bounding box) are orders of magnitude smaller than the raw input payload D_{vol} .

1. **Significance:** The invariant defines a crossover beyond which adding remote compute (R_{peak}) has little effect because transmission dominates T_{remote} . The local path is preferred only after its own complete execution time and resource limits are checked.
2. **Distinction:** Unlike the iron law, which decomposes one execution path, the locality invariant compares two complete paths against each other and against an explicit service deadline.
3. **Common pitfall:** A frequent misconception is that 5G/6G “solves” locality. While these technologies improve BW_{network} , they do not remove propagation and protocol latency, so a remote path can still miss a tight service deadline.

The operational trade-offs follow directly from the locality formulation. Local execution eliminates network transit time ($D_{\text{vol}}/BW_{\text{network}} + L_{\text{lat,network}}$), ensuring latency determinism for closed-loop control whenever local execution satisfies $T_{\text{local}} \leq T_{\text{deadline}}$. Uplink bandwidth consumption drops from raw sensor bitrates to lightweight event telemetry, alleviating network congestion and recurring transit charges. Local processing also reduces the data attack surface by retaining raw sensor feeds

within on-premises boundaries, though data at rest still requires local access control. Furthermore, edge nodes provide partition tolerance: local inference continues operating during wide-area network outages whenever local power and sensor links remain functional.

Conversely, edge deployment trades network transit for strict physical and operational boundaries. Representative edge hardware operates within 1–8 GB of memory and a 5–50 W thermal envelope, well below cloud server capacities.¹⁹ These resource limits make model footprint, activation buffers, runtime state, and thermal duty cycles binding constraints. Operationally, managing distributed infrastructure introduces non-linear complexity: coordinating software updates and model versioning across thousands of heterogeneous, physically exposed devices requires specialized deployment pipelines,²⁰ where rollback costs and silent failures present operational challenges absent in centralized data centers.

Evaluating a single high-rate sensor frame against a strict frame deadline illustrates the locality crossover in practice.



Napkin Math 2.7: The locality crossover

Problem: Should a drone’s object avoidance system (4K, 60 FPS) offload to the cloud, or does this become a locality crossover?

Given:

- **Data** (D_{vol}): 4K frame $\approx$ 24.9 MB.
- **Bandwidth** (BW_{network}): 100 Mb/s home broadband (up).
- **Remote network and compute response:** 110 ms (round-trip + remote compute).

Math:

1. **Transmission time:** $24.9 \text{ MB} \times 8 \text{ bits}/100 \text{ Mb/s} = 1,990.7 \text{ ms}$.
2. **Complete remote path:** Transmission + 110 ms remote response = 2100.7 ms, vs. a 16.7 ms frame interval.

Systems insight: The remote path misses the frame interval before control-loop slack is considered, and faster remote compute cannot remove the transmission time. Cloud offload is therefore infeasible under these assumptions; the local path must still be measured to confirm that it meets the deadline.

A realistic retail deployment shows how those constraints turn into throughput, hardware, and fleet-cost requirements. Consider a smart retail chain that deploys person detection across 500 stores, each with 20 cameras/store running at 15 FPS. Table 2.10 cascades the per-store inference rate through YOLOv8-nano’s per-frame FLOP count to yield the throughput each store must sustain.

Table 2.10: Edge Inference Sizing Requirements: Per-store throughput target for the smart-retail person-detection scenario.

Metric	Calculation	Result
Inferences per store	20 cameras/store $\times$ 15 FPS	300 inferences/s
Model compute	YOLOv8-nano: 8.7 GFLOP/inference	2610 GFLOP/s
Required throughput	2610 GFLOP/s $\times$ 2 (headroom)	$\sim$ 5.22 TOPS equivalent

Table 2.11 compares throughput, power, unit cost, and fleet cost for three candidate edge accelerators, including embedded GPU accelerators.

Table 2.11: Edge Accelerator Options: Throughput, power, and cost for three candidate edge accelerators at fleet scale.

Edge Device	INT8 TOPS	Power	Unit Cost	Fleet Cost
NVIDIA Jetson Orin NX	100 TOPS	10–25 W	\$600	\$300,000
Intel NUC + Movidius	1 TOPS	15 W	\$400	\$200,000

¹⁹ | Edge server constraints:

Across representative edge envelopes, 1–8 GB can hold roughly 1–8 billion INT8 weights before activations, runtime state, and other buffers, so parameter count alone does not define a universal millions-only ceiling. Sustainable compute is further limited by power, cooling, and duty cycle.

²⁰ | Edge fleet coordination:

Managing thousands of distributed edge devices introduces failure modes absent from centralized cloud: intermittent connectivity causes model version drift, hardware heterogeneity requires per-target optimization, and physical accessibility makes firmware rollbacks costly. These operational patterns are examined in chapter 14.

Edge Device	INT8 TOPS	Power	Unit Cost	Fleet Cost
Google Coral Dev (3 boards/store)	peak 12 TOPS derated 6 TOPS	6 W	\$450	\$225,000

 Napkin Math 2.8: Edge inference sizing

Problem: Given the throughput target in table 2.10 and the candidates in table 2.11, which edge accelerator (USB-scale TPU, workstation-class embedded GPU, or general-purpose mini-PC) delivers the required throughput at the lowest three-year fleet cost?

Math: We size from what one board sustains, not from what its datasheet peaks at. A Coral Dev Board is rated at 4 TOPS peak but delivers about 2 TOPS after 50 percent derating, so the count per store is 5.22 TOPS required ÷ 2 TOPS per board = 2.6 boards, which rounds up to 3 boards/store.

Over 3 years across 500 stores, the TCO comparison for that configuration is Hardware \$225,000 + Power (0.006 kW × 500 × 8760 h/year × 3 years × \$0.12/kWh = \$9,460.8) = \$234,460.8 total vs. a one-dedicated-cloud-GPU-per-store baseline at ~\$9,855,000.

Result: One Coral alone is undersized, but the sharded configuration with 3 boards/store is the low-cost edge choice, providing 6 TOPS and about 1.3× lower per-store hardware CapEx than Jetson. Jetson remains the simpler single-device deployment when integration complexity matters more than hardware cost.

Systems insight: Edge sizing is a capacity-and-cost problem, not a device-name problem. The cheapest feasible design may be several small accelerators per site rather than one larger board, but that hardware saving must be weighed against the coordination and maintenance burden of a sharded edge fleet.

Physical exposure compounds these operational challenges. Unlike servers housed in access-controlled data centers, edge hardware deployed across retail floors or public infrastructure is physically accessible to adversaries. This exposure requires hardware-enforced protection mechanisms: secure boot chains to prevent unauthorized firmware modification, encrypted local storage for model weights and intermediate activations, and tamper-evident enclosures. Furthermore, capital expenditures scale linearly with deployment footprint: equipping 1,000 sites with \$500–2,000 of edge hardware per site requires \$500,000–2,000,000 in upfront CapEx before installation, cabling, and fleet operations. Whether that upfront investment yields a lower total cost of ownership than recurring cloud inference depends on utilization duty cycles, local energy rates, and site maintenance overhead.

2.6.3 Real-time industrial and IoT systems

Edge applications differ by domain, but each makes the same locality argument concrete: the system cannot wait for the network, cannot ship the data, cannot expose the raw signal, or cannot stop during an outage. Autonomous vehicles represent a demanding application because safety-critical perception and control decisions must fit millisecond-scale budgets. Onboard systems process streams from multiple cameras and other sensors through local hardware; a remote cloud path cannot satisfy the tight control loop when its network delay alone exceeds the deadline.

Smart retail environments demonstrate edge ML’s practical advantages for privacy-sensitive, bandwidth-intensive applications. A checkout-free store²¹ can process feeds from many cameras on local edge servers, tracking customer movements and item selections without continuously transmitting raw video over a wide-area link. This architecture reduces bandwidth demand and transmission exposure, but it does not by itself ensure privacy: retention, access, secondary use, and governance controls still determine the system’s compliance and risk.

The industrial IoT²² uses edge ML for applications where millisecond-level responsiveness directly impacts production efficiency and worker safety. Manufacturing facilities deploy edge ML systems for real-time quality control and predictive maintenance.²³

21 | Checkout-free retail:

A store may fuse camera observations with shelf events and transaction state locally, then transmit only selected events or aggregates. The privacy boundary must still include those derived records, because reducing raw-video transport does not determine retention, access, or secondary use.

22 | Industry 4.0:

The fourth industrial revolution integrates ML into the sensor-actuator feedback loop on factory floors. The systems consequence is that the control loop latency (L_{lat}) must be shorter than the physical process it governs: a welding robot that detects a defect at 60 Hz has a 16.7 ms sampling interval, requiring a deployment whose complete response path fits that deadline.

23 | Predictive maintenance:

Models analyze sensor data such as vibration and temperature to estimate equipment condition and failure risk. Continuous monitoring makes sensing cadence, duty cycle, power, and local storage first-class constraints. Battery-powered nodes may require sub-watt average operation, whereas mains-powered industrial gateways can use a much larger envelope.

Facility automation and smart buildings rely on edge ML to balance energy optimization against operational continuity. Building management systems aggregate telemetry from thousands of environmental sensors—monitoring airflow, temperature, occupancy, and power consumption—to execute localized HVAC and lighting control loops. Processing telemetry locally cuts wide-area transit by an order of magnitude while maintaining sub-second control response and partition tolerance during network disconnects. In clinical environments, healthcare systems deploy edge inference for patient telemetry and surgical navigation, ensuring deterministic response and data isolation where cloud round trips and wide-area exposure are unacceptable.

Many industrial and retail edge systems are stationary and connected to site power, where a factory edge server drawing hundreds of watts around the clock remains viable. Mobile devices, by contrast, travel with untethered users on strictly bounded battery reserves. When computation moves from stationary infrastructure to an untethered mobile device, energy consumption replaces latency as the primary governing constraint: $\text{Energy} = \text{Power} \times T$. The dominant optimization target shifts from raw execution time to energy per inference, fundamentally transforming the underlying hardware and algorithmic design space.

2.7 Mobile ML: Offline Intelligence

Edge ML solves the distance problem that limits cloud deployments by moving computation nearer to the data. Many edge systems are stationary gateways, factory servers, and retail appliances, whereas users carry phones and tablets through changing environments. Bringing the same locality to a portable device introduces a different constraint: the battery. Unlike plugged-in edge servers that can consume hundreds of watts continuously, mobile devices must operate for hours or days on fixed energy budgets.

Mobile ML addresses this challenge by integrating machine learning directly into portable devices such as smartphones and tablets, providing responsive, personalized capabilities. This paradigm excels when data locality, offline operation, and immediate response matter more than computational scale, supporting applications such as voice recognition, computational photography,²⁴ and health monitoring. Local processing reduces transmission exposure but does not by itself guarantee privacy. These battery-powered devices must balance performance with power efficiency and thermal management, making them suited to frequent, short-duration AI tasks.

The mobile environment introduces a critical constraint absent from stationary deployments: energy per inference becomes a first-order design parameter. Under the iron law in equation 2.6, cloud and edge systems optimize for minimizing T , total latency. Mobile systems face an additional constraint: $\text{Energy} = \text{Power} \times T$, and the power wall described by equation 2.2 caps sustained power at 3–5 W. In archetype terms, a Compute Beast workload like image classification must be transformed into a more compact vision model,²⁵ reducing FLOPs by 13.7× while preserving enough accuracy for the application. This is not merely optimization; it represents a qualitative shift in the compute-per-byte trade-off, accepting lower peak throughput in exchange for sustainable operation within a 3–5 W thermal envelope.

This battery-and-thermal boundary gives the mobile paradigm its defining shape.



Definition 2.4: Mobile ML

Mobile machine learning is the deployment paradigm bounded by thermal design power (TDP) and battery energy.

1. **Significance:** It is constrained by the few-watt heat dissipation capacity of passive cooling, requiring architectures that prioritize sustained energy efficiency over peak throughput (R_{peak}).
2. **Distinction:** Unlike edge ML, which may have active cooling, mobile ML must operate within a personal energy budget. Unlike TinyML, it still provides a rich OS and multi-watt compute capacity.
3. **Common pitfall:** A frequent misconception is that mobile ML performance is a fixed value. In reality, it is a time-varying constraint: performance often drops as the device hits its thermal wall, triggering throttling that reduces the duty cycle (η_{hw}).

²⁴ | **Computational photography:** Uses ML algorithms (for example, multi-frame fusion, neural denoising) to overcome the physical limits of small mobile camera sensors. This exemplifies the mobile computing trade-off, as a multi-stage pipeline must execute within the user's perceived shutter delay while adhering to a shared 3–5 W thermal budget.

²⁵ | **Mobile vision model reduction:** MobileNet-style architectures reduce the computation in common vision layers while preserving useful accuracy for mobile tasks. The architectural details appear in chapter 6; the systems point here is that mobile deployment often requires changing the model family, not merely moving the same model to a phone.

On-device processing provides immediate responsiveness and data locality, but operates within strict electrochemical and thermodynamic boundaries. The four-branch taxonomy in figure 2.7 links the phone's shared battery and passive cooling chassis to necessary NPU kernel fusions and thermal throttling limits.

The battery life and resource constraints in mobile ML translate directly into engineering requirements. Always-on ML features incur the **battery tax**, because continuous inference spends the phone's finite energy budget even before the rest of the system runs.

The battery constraint limits total energy consumption over time. Even if we could ignore battery life for a plugged-in tablet or a short demo, thermodynamics imposes a second boundary. Electrical power ultimately becomes heat that the device must dissipate. A data center uses active cooling to remove this heat. A thin mobile device instead relies primarily on passive heat transfer through its chassis and surfaces. This creates the **thermal wall**, a hard ceiling on sustained power consumption that exists independently of battery capacity.

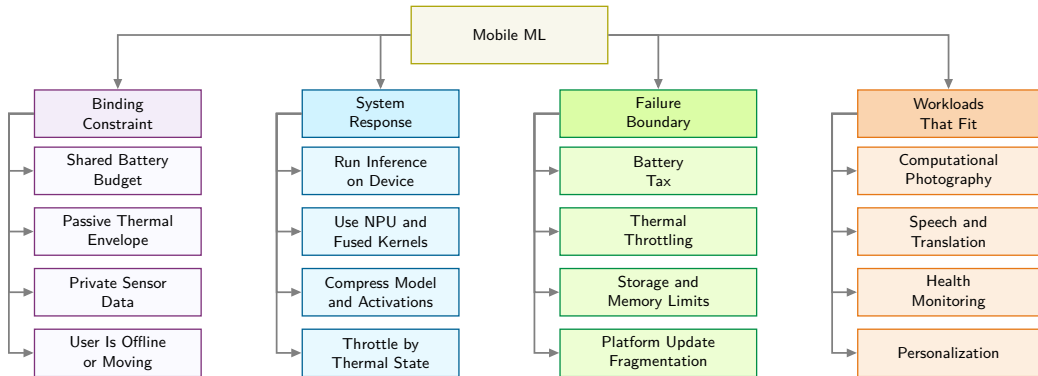


Figure 2.7: Mobile ML Constraint Map: On-device processing buys responsiveness, privacy, offline operation, and personalization, but the phone's shared battery and passive thermal envelope make sustained energy efficiency the binding design constraint.

Napkin Math 2.9: The battery tax

Problem: Consider deploying a “real-time” background object detector on a smartphone. The model consumes 2 W of continuous power when active. The phone has a standard 15 Wh battery. Can the feature stay on all day? This is the battery tax that turns battery life into a mobile ML energy-budget constraint.

Physics:

1. **Ideal runtime:** $\frac{15Wh}{2W} = 7.5$ hours
2. **Reality:** A user expects their phone to last 24 hours. Running this single feature continuously for a day would require 320 percent of the phone's daily energy budget.

Systems insight: The model cannot simply be “deployed.” The techniques in chapter 10 must reduce both model work and duty cycle so the feature can stay on all day.

The distinction matters for engineering decisions: the battery tax is a budget problem, solvable in principle by reducing how often the model runs or by increasing battery capacity. The thermal wall is a physics ceiling. A larger battery cannot raise the maximum sustained wattage a passive chassis can dissipate; duty-cycle and software changes can only reduce heat generation. Sustained operation above the thermal envelope triggers hardware throttling regardless of remaining battery energy. The two constraints therefore attack different points in the iron law: the battery limits total operations per charge (O integrated over time), while the thermal wall caps the instantaneous rate ($R_{\text{peak}} \cdot \eta_{\text{hw}}$) the silicon can sustain.



Sustained thermal performance falls well below burst peaks.

Napkin Math 2.10: The thermal wall

Problem: An unoptimized vision model requires 12 W peak compute. Can it be deployed on a mobile device, or does it hit the thermal wall and the mobile power wall?

Scenario: A mobile system-on-chip (SoC) allows approximately 3 W for passive cooling. Assume the unoptimized model ships anyway; a representative throttling trace for such a passively cooled device runs in three steps:

1. **Temperature rise:** At 12 W, the device temperature rises at approximately 1 °C/s.
2. **Thermal trip:** Within 60 s, the hardware reaches the thermal trip point (80 °C), triggering OS throttling.
3. **Throttled throughput:** The 100 FPS model suddenly drops to 30 FPS to stay within the thermal envelope.

Math: Assume an illustrative 4× active-power reduction after an FP32-to-INT8 redesign (see chapter 10): $12\text{ W} \div 4 = 3\text{ W}$, which lands right at the 3 W passive-cooling ceiling with no headroom to spare. The factor is a scenario input, not a universal whole-system guarantee from quantization alone.

Systems insight: Even a 4× power reduction does not create thermal headroom; it only just reaches the sustainable limit. Physics sets a hard ceiling that no optimization can exceed.

2.7.1 Mobile ML benefits and resource constraints

Mobile devices exemplify intermediate constraints: 8–16 GB RAM (varying from mid-range to flagship), 128 GB–1 TB storage, 15–45 TOPS AI compute through neural processing units²⁶ consuming 3–5 W power. System-on-chip architectures²⁷ integrate computation and memory to minimize energy costs. Memory capacity constrains model footprint, while 51.2–77 GB/s of bandwidth constrains how quickly weights and activations move. Battery constraints (15 Wh–22 Wh capacity) make energy optimization critical: adding 1 W of continuous ML processing to a phone that otherwise lasts 24 hours would reduce runtime to roughly 9.2 hours–11.5 hours, depending on battery capacity. Specialized on-device ML frameworks provide hardware-optimized inference for 5–50 ms UI response times.

Mobile ML excels at delivering responsive experiences with stronger data locality. Real-time processing can reach sub-10 ms latency for some tasks, enabling 5–50 ms UI response times in interactive applications. Processing sensitive inputs locally reduces data transmission and central storage, and on-device enclaves such as Apple’s Secure Enclave can further protect biometric data,²⁸ though the strength of privacy guarantees ultimately depends on the complete system design and threat model. Navigation, translation, and media-processing features can also operate without a network when their models and supporting data fit the mobile budget. Personalization rounds out the advantage, because models can exploit on-device signals and user context while keeping raw data local.

These benefits require accepting tight resource constraints. Compared to cloud deployments, mobile applications often operate under much tighter memory, storage, and latency budgets, which constrain model size and batch behavior. Battery life presents visible user impact, and thermal throttling can materially limit sustained performance: peak NPU throughput can substantially exceed what the device sustains under prolonged workloads. Development complexity multiplies across platforms, demanding separate implementations and careful performance tuning, while device heterogeneity requires multiple model variants. App-store review and staged release processes can also slow iteration compared with a service controlled entirely by its operator.

2.7.2 Personal assistant and media processing

Across mobile applications, the central systems problem is that several short pipelines must share the same battery and thermal budget. Computational photography exemplifies the challenge of running multiple ML pipelines within a thermal envelope. Modern flagships process each photo through multiple ML stages: portrait mode²⁹ uses depth estimation and segmentation, night mode

²⁶ | **Neural processing unit (NPU):** A dedicated hardware block on a mobile system-on-chip whose circuits are designed for low-precision tensor operations. This specialization can improve energy efficiency relative to CPU execution, helping AI workloads fit within mobile power budgets.

²⁷ | **System-on-chip (SoC):** Mobile SoCs integrate CPU, GPU, and NPU cores with caches and memory controllers to reduce data-movement costs. Mobile DRAM is generally off-die, and feasible model size depends on capacity, precision, working memory, bandwidth, and latency.

²⁸ | **Face ID:** Apple performs facial matching within the Secure Enclave and encrypts the stored mathematical representations so that they remain available only to that subsystem and do not leave the device (Apple 2024a, 2024b). Apple reports a random false-match probability below 1 in 1,000,000 for one enrolled appearance, with higher probabilities for some populations and configurations. The example shows how local inference and hardware isolation can reduce transmission exposure while supporting low-latency authentication.

²⁹ | **Portrait Mode pipeline:** This is not a single model but a sequence of real-time models for depth estimation, segmentation, and rendering. The core engineering problem is managing the pipeline’s aggregate latency and power, not any single model’s performance. The entire pipeline must execute within the user’s perceived shutter delay and share the phone’s 3–5 W thermal budget, forcing scheduling trade-offs across the CPU, GPU, and NPU to avoid throttling.

aligns multiple frames with ML-based denoising, and HDR merging, super-resolution, and scene optimization run in sequence. The engineering challenge is not any individual model but the pipeline: these models must share a 3–5 W power budget and complete within the user’s perceived shutter delay, requiring careful scheduling across CPU, GPU, and NPU to avoid thermal throttling.

Voice-driven interactions demonstrate mobile ML’s layered architecture. Wake-word detection runs continuously on a dedicated low-power core, while speech recognition and keyboard prediction use higher-power neural processing tiers. Each layer operates at a different power tier, illustrating how mobile ML partitions workloads across heterogeneous processing units within a single SoC.

Health monitoring and augmented reality push mobile ML to its sustained-performance limits. Wearables like Apple Watch process ECG and accelerometer data on-device to reduce data transmission, while AR frameworks demand a 16.7 ms total frame budget at 60 FPS for simultaneous localization, hand tracking, and scene understanding. These applications represent the ceiling of what battery-powered, passively-cooled devices can sustain, and they define the boundary beyond which mobile optimization alone is insufficient.

These successes can create a misleading sense of ease. A common pitfall involves attempting to deploy desktop-trained models directly to mobile or edge devices without architecture modifications. Models developed on powerful workstations often fail when deployed to resource-constrained devices. A desktop ResNet-50 pipeline may require gigabytes once activations, batches, preprocessing buffers, and runtime overhead are included, even though the FP32 weights alone are about 102.4 MB. Such a pipeline, requiring 8.2 GFLOP per inference, cannot run unchanged on a representative low-end edge target with 512 MB of RAM and a 1 GFLOP/s processor. Beyond simple resource violations, desktop-optimized models may use operations unsupported by mobile hardware, assume numerical formats unavailable on embedded systems, or require batch processing incompatible with single-sample inference. Successful deployment demands architecture-aware design from the beginning: model families, arithmetic formats, and implementation choices must all match the target device.

Mobile ML demonstrates that useful intelligence can operate within a 3–5 W thermal envelope on battery power. However, smartphones still cost hundreds of dollars, require gigabytes of memory, and demand user attention to recharge daily. These requirements make them unsuitable for a vast class of applications: monitoring soil moisture across a thousand-acre farm, detecting structural stress in bridge cables, or listening for endangered species in a remote forest. These scenarios demand not just lower power but a qualitatively different engineering regime, one where the device costs dollars instead of hundreds, memory is measured in kilobytes instead of gigabytes, and the system runs unattended for months or years. Mobile optimization methods help, but they cannot bridge a 10,000-fold gap in available memory. What is needed is not a scaled-down smartphone but an entirely different class of hardware and algorithms.

2.8 TinyML: Ubiquitous Sensing

Imagine instrumenting every pallet in a warehouse, every cable on a suspension bridge, every beehive in an apiary. To put “eyes and ears” on this many physical objects, tens of thousands to millions, the device must cost dollars, not hundreds of dollars, and measure millimeters, not centimeters. Smartphones are far too expensive and too large; what is needed is ubiquitous sensing at unit costs under ten dollars and physical dimensions measured in millimeters.

TinyML completes the deployment spectrum by pushing intelligence to its physical limits: low-cost, low-power ML on deeply constrained embedded devices (Janapa Reddi, Plancher, et al. 2022). Devices costing under \$10 and consuming less than one milliwatt³⁰ of power make ubiquitous³¹ sensing economically practical at massive scale; MLPerf Tiny and MCUNet show how such constraints are evaluated and optimized in practice (C. Banbury et al. 2021; Lin et al. 2020). This is the domain of the Tiny Constraint archetype, where the optimization objective shifts from maximizing throughput to minimizing energy per inference. With duty-cycled execution, a keyword spotting model consuming 10 μ J per inference can operate for years on a coin-cell battery, achieving million-fold improvements in energy efficiency by trading model capacity for operational longevity. In figure 2.8, annotated development boards illustrate this physical scale, showing integrated microcontroller chips, sensor arrays, and pin headers designed to run compact models in disconnected environments.

Operating at this physical scale requires microcontrollers (MCUs)³² that eliminate the power-

30 | The 1 mW Threshold: Ambient energy harvesting can power a device indefinitely only when long-term harvested power exceeds the complete system’s average load. Available power varies across sources and environments—solar cells the size of a thumbnail may produce ~10 mW outdoors but ~10 μ W indoors, thermoelectric generators on warm pipes ~100 μ W, and RF energy from nearby transmitters ~10 μ W. This crossover can transform the deployment model from “battery-limited lifetime” to “deploy and forget,” but it depends on the source, environment, and workload.

31 | Ubiquitous computing: Mark Weiser’s ubiquitous-computing vision imagined computation woven into everyday environments until it receded from direct attention (Weiser 1991). TinyML is one modern path toward that vision: when the cost and power of an intelligent sensor become low enough for mass deployment, the optimization objective shifts from performance (throughput) to power (energy per inference), the central trade-off of the Tiny Constraint archetype.

32 | Microcontroller (MCU): A single-chip computer whose design prioritizes minimal cost and power over performance, creating the “radical constraint” mentioned. Many TinyML targets provide only tens to hundreds of kilobytes of on-chip SRAM and no virtual memory; some higher-end microcontrollers can attach external memory. The complete model, activations, and runtime state must therefore fit the memory actually available to the target, often orders of magnitude less than a smartphone provides.

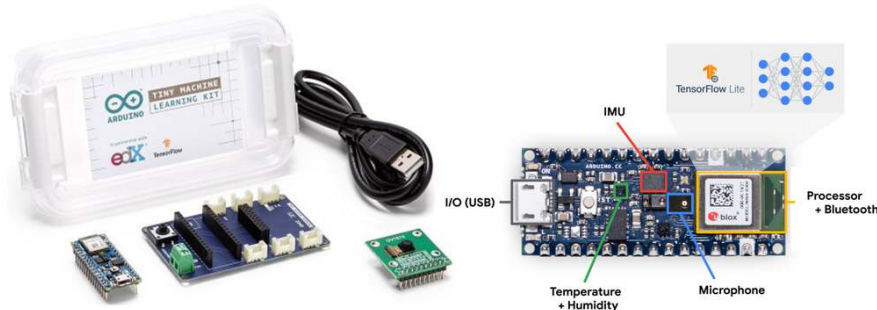


Figure 2.8: TinyML Hardware Scale: Microcontroller development boards integrate sensors, processing cores, and low-power radios onto single substrates operating under strict kilobyte-scale SRAM and milliwatt power bounds. This compact footprint shifts the optimization objective from peak throughput to energy per inference, enabling multi-year deployment on coin-cell batteries. Image source: Figure 6 in (Janapa Reddi, Plancher, et al. 2022), licensed under CC BY 4.0.

hungry infrastructure of conventional computing. Mobile and edge processors rely on external dynamic RAM (DRAM), high-speed memory buses, and operating system runtimes that draw hundreds of milliwatts simply to refresh memory cells and toggle bus lines. Microcontrollers dispense with external memory buses entirely, integrating the processor core, non-volatile flash, and static RAM (SRAM) onto a single silicon die (C. Banbury et al. 2021; Lin et al. 2020; Janapa Reddi, Plancher, et al. 2022). While eliminating off-chip signaling drives active power below a milliwatt and sleep power to microwatts, it imposes an unforgiving memory ceiling. On-chip SRAM is physically constrained by silicon die area: because each SRAM cell requires six transistors (6T), expanding capacity directly inflates manufacturing cost. A TinyML model must therefore execute within tens to hundreds of kilobytes of SRAM, with no virtual memory, paging, or swap space to cushion an overflow. The resulting energy gap between TinyML and cloud inference (figure 2.6) spans at least six orders of magnitude³³ and reaches eight orders for cloud LLM queries, enabling continuous multi-year operation on coin-cell batteries.³⁴

Fitting the entire inference pipeline, from sensor acquisition to classification output, into this kilobyte-scale memory footprint requires formalizing the TinyML operating regime.

Definition 2.5: TinyML

TinyML is the machine learning domain of always-on sensing constrained by kilobyte-scale memory and milliwatt-scale power.

1. **Significance:** It necessitates models whose weights and code fit in flash while activations and runtime state fit in on-chip SRAM, enabling continuous inference on milliwatt power budgets.
2. **Distinction:** Unlike mobile ML, which uses multi-watt processors and a full OS, TinyML runs on microcontrollers using bare-metal firmware or a real-time operating system.
3. **Common pitfall:** A frequent misconception is that TinyML is just “small models.” In reality, it is an energy-bound paradigm: the primary metric is energy per inference (microjoules), not just the parameter count.

This definition reframes the iron law of ML systems. In terms of equation 2.6, TinyML imposes a physical memory-capacity precondition that execution time does not capture: the model footprint M_{model} and activation footprint must stay within memory capacity C_{mem} . When total memory is measured in kilobytes, model structure and runtime allocation determine feasibility before latency optimization begins, and every avoidable data movement spends part of the energy budget. The objective therefore shifts from raw throughput toward energy per inference and memory fit.

³³ | **TinyML energy gap:** This differential is rooted in hardware design philosophy; cloud GPUs are optimized for raw throughput, consuming hundreds of watts, while TinyML microcontrollers are designed for near-zero power sleep states. For ordinary cloud inference, a single request may consume about 10 joules while a specialized TinyML device uses about 10 microjoules, a 1,000,000× gap. Cloud LLM queries can push the comparison even further, as quantified in table 2.12.

³⁴ | **Coin-cell deployment:** A CR2032 battery (225 mAh at 3 V, ~675 mWh) ideally powers a complete 10–50 μW system for about 1.5–7.7 years. This “deploy-and-forget” operating model drives innovation in intermittent computing, where the device sleeps between inferences to stretch the energy budget across years of unattended operation.

Systems Perspective 2.3: Reading the energy-per-inference numbers

The energy values in table 2.12 combine solver-derived device-energy estimates with separate scale anchors; they are not controlled full-system measurements. An illustrative A100-only ResNet-50 bound is under 1 ms and about ~ 0.3 J, while an entire server can draw about ~ 1 kW when CPUs, memory, and overhead are included. The final-column counts show how many inferences one battery's nominal energy could supply; they are not elapsed battery life and omit sensing, sleep, and communication.

This TinyML energy-efficiency gap helps explain why low-energy local models are attractive for always-on sensing. Actual runtime still depends on inference cadence, sleep power, sensing, communication, and the rest of the device.

The energy gap between paradigms is not a matter of degree but of scale, spanning the eight orders of magnitude sketched in figure 2.6. Table 2.12 grounds that span in concrete numbers, translating each paradigm's energy cost into how many inferences a single smartphone battery could sustain.

Table 2.12: Energy per inference across paradigms: Analytical device-energy estimates and scale anchors, with the corresponding inference counts from one smartphone battery. These are not controlled full-system measurements.

Paradigm	Example Workload	Energy/Inference	Inferences per Battery (3.7 V, 3000 mAh)
Cloud	GPT-4 query	~ 1 kJ	40 queries
Cloud	ResNet-50 (accelerator server)	75.5 mJ	529,505 queries
Edge	ResNet-50 (Jetson)	15.9 mJ	2,511,628 queries
Mobile	MobileNet (NPU)	2.60 mJ	15,393,505 queries
TinyML	Keyword spotting	10 μ J	3996 million queries

Operating on sub-milliwatt power budgets enables continuous sensing without grid power, but forces the complete inference state into on-chip SRAM. The four-column taxonomy in figure 2.9 traces how kilobyte memory and duty-cycled sleep states enable always-on workloads while restricting model updates.

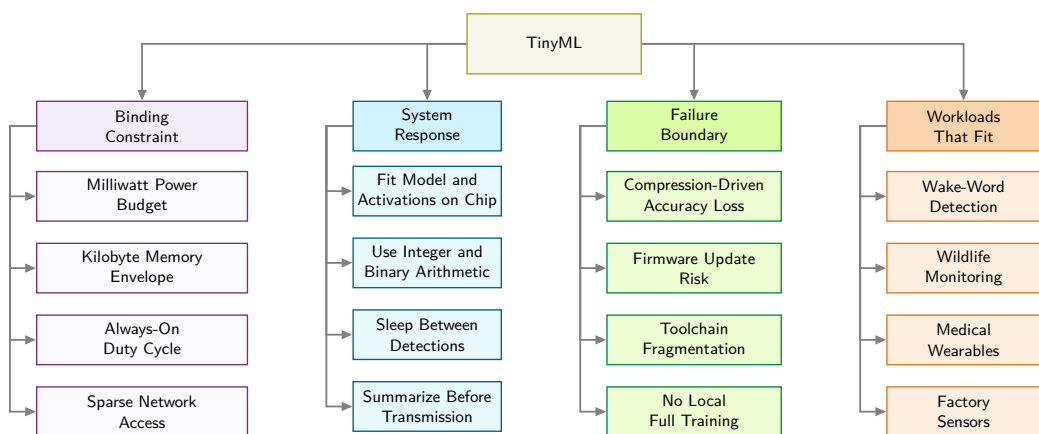


Figure 2.9: TinyML Constraint Map: Milliwatt power and kilobyte memory make always-on sensing economically possible, but those same limits force the model, activations, runtime, and update path to fit a radically smaller engineering envelope.

2.8.1 TinyML advantages and operational trade-offs

TinyML operates at hardware extremes. Compared to cloud systems, TinyML deployments commonly provide roughly 10^6 to 10^9 times less memory, depending on whether the microcontroller

budget is in low megabytes or kilobytes, with power budgets in the milliwatt range. These strict limitations enable months or years of autonomous operation³⁵ but demand specialized algorithms, model compression, and careful systems co-design. Devices range from palm-sized developer kits to millimeter-scale chips,³⁶ enabling ubiquitous sensing in contexts where networking, power, or maintenance are costly. Representative developer kits include the Arduino Nano 33 BLE Sense (256 KB RAM, 1 MB flash, 20–40 mW) and ESP32-CAM (520 KB RAM, 4 MB flash, 50–250 mW).

Operating entirely on-device unlocks architectural capabilities that networked systems cannot deliver. Eliminating wireless transmission removes off-chip communication latency entirely, enabling deterministic, low-latency response times for tight sensing and control loops. Eliminating radio uplinks also transforms deployment economics: when per-node unit costs fall to single-digit dollars, dense instrumentation across thousands of physical assets becomes financially viable. Operating on microjoule energy budgets further compounds these economic gains by allowing multi-year autonomous operation on inexpensive coin-cell batteries or ambient energy harvesting. Finally, locality enforces data containment: because raw sensor streams are processed at the capture interface and never transmitted across a network, the exposure surface for sensitive data is substantially reduced, though formal privacy still requires hardware-level memory protection and secure boot mechanisms.

These capabilities require substantial trade-offs. Computational constraints impose severe limits: microcontrollers commonly provide 10^5 to 10^6 bytes of RAM, forcing models and intermediate activations into the tens-of-kilobytes to low-megabytes range depending on the workload. Microcontrollers enforce a strict two-tier memory partition: static model weights and compiled code reside in non-volatile flash (often a few megabytes), while mutable activations and the runtime buffer (the *tensor arena*) must fit within the much smaller on-chip SRAM (often under 256 KB). As a result, peak intermediate activation volume—rather than static parameter storage—is frequently the binding barrier that prevents a model from executing. Embedded development also demands cross-stack co-design: engineers must manually manage memory pools, debug bare-metal firmware without OS abstractions, and optimize operator kernels directly for constrained instruction sets.

Operating under these physical bounds introduces unique lifecycle and reliability challenges. An energy-harvesting device can satisfy its long-term average power budget yet fail when stored energy is exhausted before the next harvest. Its duty cycle must therefore respect both energy per inference and the timing of available power. Model quality can also suffer from aggressive quantization and pruning: converting 32-bit floating-point weights to 8-bit integers (INT8) saves memory but can degrade accuracy and robustness on out-of-distribution sensor inputs. Furthermore, post-deployment flexibility is sharply constrained: models are compiled directly into monolithic firmware images, making field updates dependent on slow and risky firmware re-flashing workflows. Ecosystem fragmentation³⁷ across microcontroller vendors and ML frameworks creates additional overhead and portability challenges.

2.8.2 Environmental and health monitoring

The TinyML operating regime becomes essential when power budgets, unit economics, and physical isolation rule out every higher-tier platform. Wake-word detection is the most familiar consumer example: a purpose-built low-power component listens continuously, processes audio locally, and activates higher-power components only when a wake phrase is detected, reducing average power draw.³⁸ Precision agriculture exploits the same locality pressure from a different direction: FarmBeats used sensors, cameras, drones, and local gateway processing to reduce raw data movement where farm connectivity was costly (Vasisht et al. 2017). TinyML pushes that locality logic further when the sensing node itself must run under milliwatt budgets.

Wildlife conservation uses TinyML for remote environmental monitoring, where solar-powered audio sensors process continuous acoustic streams on-device for species identification. Transmitting periodic detection summaries rather than raw continuous audio reduces uplink data from 4.3 GB/day to 400 KB/day, a 10,750× reduction that conserves power in off-grid deployments. Medical wearables apply the same local-processing logic to physiological monitoring. Sensitivity, sampling rate, power, and battery life are device-specific validation targets rather than universal properties of TinyML hardware. Local processing minimizes continuous wireless transmission and protects user

35 | On-device training constraints: Full on-device training must retain intermediate layer outputs so later weight updates can compute gradients, consuming memory proportional to model depth. During inference, each layer's activations can be discarded once the next layer consumes them. With only 256 KB–2 MB RAM, backpropagation memory demands exceed available SRAM; TinyTL freezes weights and updates biases to reduce activation memory (Cai et al. 2020). This memory constraint restricts TinyML devices predominantly to inference, with model updates deployed via firmware rather than trained in situ.

36 | TinyML device range: This physical range reflects a direct trade-off between deployment context and computational capability. Millimeter-scale systems prioritize minimal power (~140 μ W) for single-function, long-duration tasks, whereas palm-sized boards trade larger size and higher power for the ability to process multiple complex sensor streams. This co-design choice creates a $> 10,000\times$ power and $\sim 100\times$ area difference across the operational spectrum of TinyML devices.

37 | TinyML ecosystem fragmentation: TinyML spans microcontroller families such as Arm Cortex-M, RISC-V, and Xtensa, each with different instruction sets, memory layouts, kernels, and vendor toolchains; cloud and mobile stacks also vary, but their runtimes usually hide more of this hardware diversity. A model optimized for one microcontroller target may therefore require retuning and revalidation for another, multiplying the engineering cost of multi-device deployment.

38 | **Always-on wake-word detection:** A small, specialized model listens for the acoustic signature of a wake phrase and acts as a power gate for the higher-power application processor. The acceptable detector budget depends on the platform, but spending even a few extra milliwatts continuously can materially shorten battery life. The architecture succeeds only when the always-on path consumes far less average power than the components it prevents from waking.

privacy, while total runtime remains bounded by sensor draw, radio power, duty cycle, and sleep energy.

These four paradigms form an operational continuum bounded by the speed of light, heat dissipation, and memory signaling. When architecting an ML system, engineering begins by systematically mapping application constraints against these physical boundaries.

2.9 Paradigm Selection

An architect choosing where to run an ML feature rarely optimizes one dimension. A privacy rule may forbid cloud processing, a latency budget may forbid remote inference, a memory footprint may exceed the mobile device, and a cost target may rule out always-on edge hardware. Cloud, edge, mobile, and TinyML are operating envelopes for those conflicts, so selecting among them requires a unified comparison framework and a structured decision process.

2.9.1 Comparative trade-off analysis

Deployment decisions require evaluating latency-vs-throughput paradigm trade-offs across distinct physical and operational envelopes. Choosing among cloud, edge, mobile, and TinyML targets demands balancing latency, power dissipation, financial cost, data exposure, and software complexity simultaneously. Table 2.13 contrasts these four paradigms across fourteen hardware and systems dimensions, establishing the baseline envelopes that govern deployment feasibility.

Table 2.13: Fourteen-Dimension Paradigm Comparison: A comprehensive side-by-side comparison across fourteen dimensions that matter for deployment decisions. Cloud ML provides the strongest compute, while local processing can reduce transmission and centralized-storage exposure without itself guaranteeing privacy. This table serves as the primary reference for system architects evaluating deployment options.

Aspect	Cloud ML	Edge ML	Mobile ML	TinyML
Processing Location	Centralized cloud servers (Data Centers)	Local edge devices (gateways, servers)	Smartphones and tablets	Ultra-low-power microcontrollers and embedded systems
Latency	100–500 ms	10–100 ms	5–50 ms	1–10 ms
Compute Power	Very High (Multiple GPUs/TPUs)	High (Edge GPUs)	Moderate (Mobile NPUs/GPUs)	Very Low (MCU/tiny processors)
Storage Capacity	Cloud-scale (petabytes+)	Large (terabytes)	Moderate (gigabytes)	Very Limited (kilobytes–megabytes)
Energy Consumption	Very High (kW–MW range)	High (hundreds of watts)	Moderate (3–5 W)	Very Low (mW range)
Scalability	High (provider capacity limits)	Good (limited by edge hardware)	Moderate (per-device scaling)	Limited (fixed hardware)
Data Exposure	Remote processing expands the surface	Local processing reduces transmission	On-device processing reduces transmission	Raw data can remain local
Connectivity Required	Constant high-bandwidth	Intermittent	Optional	None
Offline Capability	None	Good	Excellent	Complete
Real-time Processing	Dependent on network	Good	Very Good	Excellent
Cost	High (\$1000s+ / month)	Moderate (\$100s–\$1000s)	Device purchase (\$200–\$1000+; often user-owned)	Very Low (\$1–\$10)
Hardware Requirements	Cloud infrastructure	Edge servers/gateways	Modern smartphones	MCUs/embedded systems
Development Complexity	High (cloud expertise needed)	Moderate-High (edge+networking)	Moderate (mobile SDKs)	High (embedded expertise)
Deployment Speed	Fast	Moderate	Fast	Slow

This contrast reflects the fundamental tension between data locality and computational scale. Remote processing expands the network attack surface and requires robust transport encryption and tenancy isolation; local processing confines raw data to the device, mitigating ingress exposure without eliminating physical side-channel risks. The workload archetypes from section 2.3 map directly to these boundaries, with each archetype gravitating toward paradigms that alleviate its primary bottleneck.

To visualize these multi-dimensional trade-offs, figure 2.10 evaluates the four paradigms on an ordinal 0–10 scale across two orthogonal operational profiles: subplot (a) maps raw throughput capacity against latency and thermal efficiency, while subplot (b) evaluates operational autonomy across connectivity independence, data locality, deterministic real-time response, and offline execution.

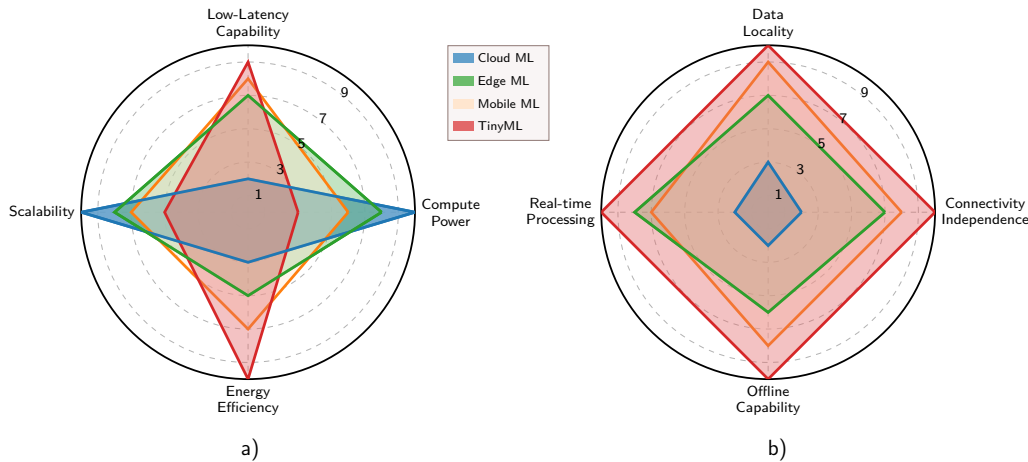


Figure 2.10: Paradigm Comparison Radar Plots: Two radar plots compare performance and operational characteristics across cloud, edge, mobile, and TinyML paradigms using ordinal 0–10 scores. The left plot contrasts compute power, low-latency capability, scalability, and energy efficiency; the right plot contrasts connectivity independence, data locality, real-time capability, and offline operation. In both plots, a larger polygon indicates stronger performance, with cloud ML peaking on compute and scalability and TinyML peaking on energy efficiency, data locality, and offline operation.

The divergent profiles in figure 2.10 demonstrate that no deployment tier dominates across all operational metrics. Instead, each paradigm shifts engineering complexity to a distinct subsystem. Cloud deployments demand expertise in distributed consensus, multi-tenant networking, and dynamic cost governance. Edge and TinyML deployments impose severe physical limitations on thermal envelope and memory capacity, requiring bare-metal optimization, heterogeneous hardware abstraction, and remote fleet orchestration. Mobile deployments operate on user-owned hardware, mitigating infrastructure costs while introducing platform-specific driver fragmentation, aggressive operating-system background task throttling, and strict energy budgets.

A common architectural failure is selecting a deployment target based on model accuracy alone while treating hardware constraints as downstream optimization details. An unquantized model that delivers 99 percent accuracy in an offline evaluation becomes unviable if its memory footprint triggers paging on mobile hardware, or if continuous inference drives thermal throttling within seconds. Physical invariants—memory capacity, thermal dissipation limits, and network transit delay—must serve as early screening criteria before model development begins, pruning invalid deployment targets before training resources are expended.

2.9.2 Decision framework

Systematic paradigm selection relies on a tiered decision framework ordered by how rapidly a constraint invalidates an architecture. The screening hierarchy evaluates four successive gates: regulatory privacy, physical latency, local compute capacity, and lifecycle economics. Figure 2.11 organizes these decision layers to prune unviable options before engineering effort is committed to an infeasible target.

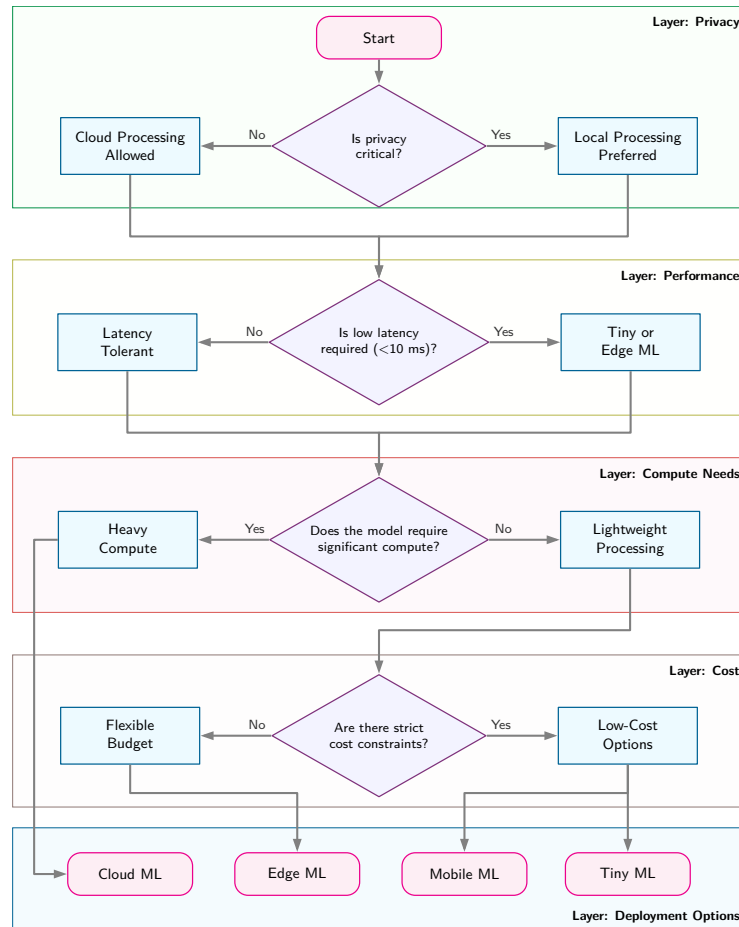


Figure 2.11: Deployment Decision Screening: Four layers organize privacy, latency, compute-demand, and cost questions, then associate their branches with cloud, edge, mobile, and TinyML options. Reconverging branches and a low-cost branch that reaches both mobile and TinyML make this a screening aid rather than a deterministic selector.

The flowchart functions as a coarse-grained filter rather than a single-path deterministic algorithm. Upstream gates impose hard physical and regulatory constraints that eliminate entire branches regardless of downstream desirability. If regulatory rules or data sovereignty forbid off-device transmission, remote cloud processing is eliminated immediately. Similarly, if physical deadline requirements fall below network round-trip times, network-dependent paradigms are ruled out regardless of server compute capacity. As branches reconverge at the cost layer—where both mobile ML and TinyML emerge as low-cost options—the framework narrows the design space to a manageable subset of candidates for detailed benchmarking.

An automotive emergency braking scenario demonstrates how the latency gate binds before compute capacity or cost enter consideration.



Example 2.1: Autonomous vehicle emergency braking

Scenario: An autonomous vehicle vision system performs real-time pedestrian detection for emergency braking under a strict 100 ms end-to-end latency budget.

Diagnosis: The scenario assumes 50–150 ms of network round-trip delay for cloud offload and 10–30 ms for local object detection on an automotive edge accelerator (300 GFLOP/s). Under those assumptions, the remote path alone causes the vehicle to travel 2.8 m before braking.

Systems lesson: Hard real-time safety constraints eliminate cloud offloading regardless of compute availability. Latency bounds dictate an Edge ML deployment architecture, reserving cloud connections for offline model training and telemetry aggregation.

Passing the physical and economic screening gates establishes technical feasibility, but an architect must also account for operational maintainability. Deploying a model across thousands of edge nodes or millions of microcontrollers introduces sustained lifecycle overhead: automated firmware updates, out-of-band telemetry, distributed drift monitoring, and hardware failure recovery. An organization equipped for cloud operations may lack the embedded toolchains and physical testing infrastructure required to support edge or TinyML targets at scale.

Beyond the operational demands of specific hardware tiers lies a prior systems question: whether the feature justifies a machine learning deployment at all. Deep neural networks impose a substantial maintenance burden that traditional software avoids. Before committing to any ML architecture, an engineer must evaluate whether a deterministic heuristic can solve the problem at a fraction of the operational cost.

Systems Perspective 2.4: The complexity tax

Consider a classification task solvable either by deterministic heuristic rules or by a deep learning pipeline. The heuristic requires fifty lines of code, incurs near-zero compute cost, demands approximately one hour per month to update rules, and exhibits zero distribution drift. The ML system may require only fifty lines of model definition, but it introduces roughly 2,000 lines of supporting infrastructure for feature ingestion, telemetry logging, drift monitoring, and GPU driver management, alongside 40 hours per month diagnosing data drift and maintaining pipelines.

An ML model that increases accuracy from 90 percent to 95 percent represents a net negative return on engineering effort if it introduces a $40\times$ expansion in operational maintenance. Machine learning systems engineering focuses on minimizing this overhead through resilient architectures; when the ongoing maintenance cost outstrips the value of marginal statistical gains, deterministic software remains the correct systems choice.

This trade-off reframes model selection as a systems-level optimization spanning physical execution limits, capital expenditure, and the ongoing maintenance of prediction quality. When this **complexity tax** outweighs the operational value of improved statistical predictions, maintaining an ML deployment actively degrades overall system efficiency.

Checkpoint 2.2: System design

The central trade-off is often accuracy vs. complexity.

Decision gates

- ☐ **The baseline:** have you measured the accuracy of a simple heuristic (regex, logistic regression) before training a deep network?
- ☐ **The infrastructure cost:** is the 2 percent accuracy gain from a transformer worth the $10\times$ inference cost and maintenance burden compared to a smaller model?

Navigating these trade-offs demonstrates that paradigm selection is an architectural optimization bounded by the physical invariants of computation, communication, and storage. While the decision framework establishes baseline feasibility—with chapter 12 and chapter 14 detailing the corresponding measurement and operational lifecycle practices—real-world applications often present mutually incompatible constraints. An intelligent system may demand petabyte-scale training capacity alongside sub-millisecond response times and offline privacy. When no individual tier can satisfy these

conflicting physical boundaries alone, systems engineering bridges the gap by partitioning the workload across multiple operating environments.

2.10 Hybrid Architectures

The decision framework (figure 2.11) narrows the feasible paradigms for an application by testing candidate systems against individual physical constraints. However, production workloads rarely operate under a single uniform constraint across their entire lifecycle. Real-time control requires sub-millisecond response loops, continuous sensing requires sub-milliwatt power envelopes, and high-capacity representation requires clusters of multi-gigabyte accelerators. No single hardware platform satisfies all three simultaneously: placing an entire workload in the cloud violates latency deadlines and network bandwidth limits, while confining it to an embedded microcontroller starves it of memory and arithmetic throughput. Production systems resolve this tension through *hybrid architectures*, partitioning the computational pipeline across multiple hardware tiers to ensure each stage executes where its binding constraint is cheapest to satisfy. Three integration patterns govern this division of labor.

2.10.1 Integration patterns

The three essential hybrid ML patterns differ by which physical boundary dictates the partition: train-serve split, hierarchical processing, or progressive deployment. Selecting a pattern is an iron-law decision: each stage of computation must execute where its binding term—whether throughput capacity R_{peak} , local latency floor L_{lat} , or memory capacity—incur the lowest system penalty.

Under a **train-serve split**, training executes in centralized data centers while inference runs locally on edge, mobile, or embedded devices. This decoupling resolves the fundamental asymmetry between model optimization and model execution. Training requires backpropagation, optimizer state storage (such as Adaptive Moment Estimation (Adam) momentum and variance vectors requiring $8\times$ to $16\times$ the parameter footprint in memory), and high-throughput batch parallelism across dense accelerator fabrics. Inference, by contrast, executes only the forward pass, evaluates small batches or single tokens, and must satisfy strict local latency and privacy constraints. Amortization underpins the economics of this split: the high fixed cost of training is distributed across millions of forward-pass queries served locally without recurring network transmission costs.³⁹

³⁹ | **Train-serve cost asymmetry:** Training usually requires many forward and backward passes, whereas one inference request uses a trained artifact for one or more forward passes. Training is not literally one-time because models may be re-trained, and inference cost varies sharply with model size, sequence length, batching, and service overhead. The economic rationale for the split is amortization: repeated use can spread each training cycle's fixed cost across many served predictions.



Definition 2.6: Hybrid ML

Hybrid machine learning is the deployment strategy that distributes an ML pipeline across two or more deployment tiers, assigning each stage according to its latency, compute, data, and operational constraints.

1. **Significance:** Hybrid architectures exploit the iron law's additive structure: the edge tier can reduce L_{lat} for time-sensitive preprocessing and inference, while the cloud tier can provide the R_{peak} needed for training, retraining, and heavy batch inference. The split follows the data locality invariant: compare each stage's complete local and remote paths, reject paths that miss the service deadline, and then choose among the feasible alternatives.
2. **Distinction:** Unlike cloud-only deployment (which accepts the distance penalty for all stages) or edge-only deployment (which accepts limited R_{peak} for all stages), hybrid ML dynamically assigns each pipeline stage to the tier where its binding iron law term is minimized.
3. **Common pitfall:** A frequent misconception is that hybrid ML is just "running two models." In reality, the participating tiers must share synchronized state—feature definitions, model versions, and preprocessing logic—so that their paths produce consistent results. Without this synchronization, training-serving skew emerges at tier boundaries.

Under **hierarchical processing**, computation cascades across tiers to progressively filter and aggregate data, matching communication volume to available interconnect bandwidth. Sensor

streams generated at the physical periphery often exceed wide-area network (WAN) uplink capacity; continuously streaming raw, uncompressed telemetry to a data center saturates network links and wastes radio power. To prevent network saturation, early tiers filter data locally: TinyML microcontrollers run low-power anomaly detectors, edge gateways aggregate and correlate temporal features across multiple sensors, and cloud clusters ingest only anomalous events or aggregated summaries for global analytics. During inference serving, hierarchical processing frequently takes the form of a confidence cascade. A compact, low-latency model on the local device evaluates the input first. If its prediction confidence exceeds a calibrated threshold τ , the device returns the result immediately, bypassing the network entirely. Only ambiguous or low-confidence inputs trigger a WAN payload transfer to a larger cloud model, minimizing median service latency while maintaining peak model accuracy.

Under **progressive deployment**, an engineering team derives a family of specialized model variants from a unified architecture or training corpus, tailoring each variant to the memory capacity, arithmetic intensity, and thermal limits of a specific tier. Rather than forcing a single model onto every device, developers apply structured compression—such as post-training quantization, pruning, and knowledge distillation—to produce tier-specific artifacts along an accuracy-efficiency Pareto curve. Modern voice interfaces illustrate this multi-tiered progression: continuous wake-word detection runs inside an always-on microcontroller’s SRAM budget (occupying tens of kilobytes for benchmark TinyML neural networks and drawing sub-milliwatt to milliwatt-scale power on dedicated low-power hardware); locally detected trigger phrases wake an on-device neural processing unit to parse acoustic features; and unconstrained semantic understanding offloads to large parameter-scale models in cloud infrastructure.

With three integration patterns available, selection becomes a constraint-matching problem: choose the pattern whose trade-off profile matches the system’s dominant bottleneck. Table 2.14 summarizes the trade-off, the conditions that favor each pattern, and the conditions that argue against it.

Table 2.14: Hybrid Pattern Selection Guide: The trade-off, favoring conditions, and disqualifying conditions for each of the three hybrid integration patterns. Selection matches the pattern’s trade-off profile to the system’s dominant bottleneck.

Pattern	Trade-off	Choose when	Avoid when
Train-Serve Split	Training cost vs. inference latency	Training requires scale that inference does not; privacy matters for inference but not training	Model needs continuous learning from deployed data
Hierarchical Processing	Local autonomy vs. global optimization	Data volume exceeds transmission capacity; decisions needed at multiple timescales	All processing can occur at one tier; network is reliable and fast
Progressive Deployment	Model quality vs. deployment reach	Same model needed at multiple capability levels; graceful degradation required	Model cannot be meaningfully compressed; single deployment target

Production deployments rarely implement these patterns in isolation; complex systems compose them across different phases of the pipeline. An autonomous vehicle, for example, combines progressive deployment to fit specialized vision backbones into its on-board compute envelope, hierarchical processing to arbitrate between immediate millisecond-scale emergency braking on local hardware and multi-second trajectory planning, and a train-serve split that uploads anomalous driving slices to cloud clusters for fleet-level retraining. Privacy-preserving distributed training paradigms, such as federated learning, further extend this composition when raw training data cannot leave client devices due to regulatory or privacy constraints.

2.10.2 Production system integration

Production hybrid systems coordinate multiple tiers through structured interaction channels. As mapped in figure 2.12, multi-tier architectures maintain two counter-directional flows: compiled model artifacts deploy downward from cloud training clusters toward edge, mobile, and TinyML execution targets, while raw sensor streams, filtered features, and inference telemetry propagate upward toward centralized data stores.

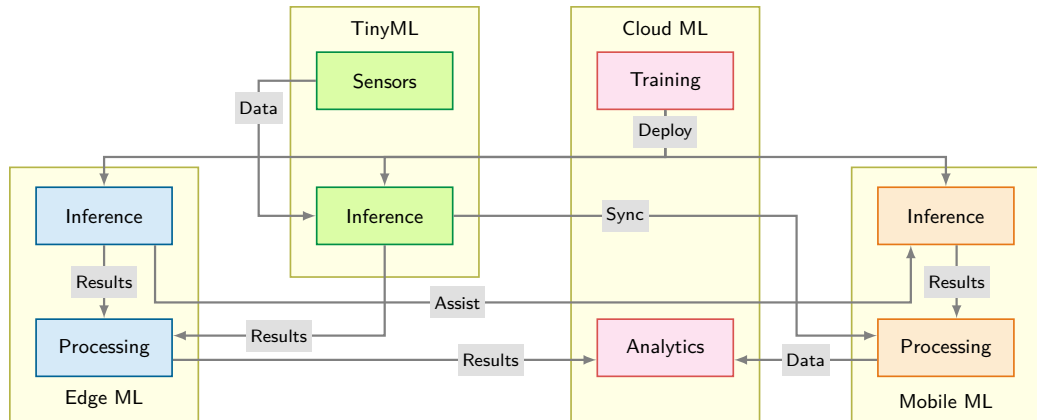


Figure 2.12: Hybrid System Interactions: Data flows upward from sensors through processing layers to cloud analytics, while trained models deploy downward to edge, mobile, and TinyML inference points. Five connection types (deploy, data, results, assist, and sync) establish a distributed architecture where each paradigm contributes unique capabilities.

Production systems demonstrate these integration patterns by placing each tier boundary at a distinct physical bottleneck:

1. **Industrial defect detection:** Assembly lines running at high cycle rates require sub-10 ms deterministic visual inspection. Streaming high-resolution camera frames over WAN to a data center violates inspection deadlines and exhausts factory uplink bandwidth. The system deploys a train-serve split: centralized cloud infrastructure leverages multi-accelerator pods to train high-capacity vision models across historical defect databases, then compiles and quantizes runtime weights for factory-floor edge servers and embedded smart cameras. These on-premise accelerators execute inference over local PCIe or MIPI camera interfaces, guaranteeing bounded latency without external network dependencies.
2. **Agricultural monitoring:** Deployed across hundreds of hectares, remote soil probes operate on coin-cell batteries or small solar harvesters, communicating over low-data-rate, high-energy-per-bit LPWAN or LoRa radios. Streaming continuous moisture and temperature telemetry exhausts battery reserves within days. Hierarchical processing confines high-frequency sampling and basic anomaly detection to TinyML microcontrollers operating in sub-milliwatt power budgets. When anomalies occur, local edge gateways aggregate measurements across spatial clusters of probes, transmitting only compressed, field-level summaries over cellular backhaul to cloud servers responsible for farm-wide irrigation scheduling.
3. **Wearable fitness tracking:** A battery-powered smartwatch operates under a strict thermal envelope where continuous processor dissipation must remain < 1 mW. The system employs progressive deployment across a gateway architecture: lightweight integer kernels run on an embedded microcontroller to track real-time motion and detect heart-rate thresholds; intermediate biometric data synchronizes over Bluetooth Low Energy (BLE) to a paired smartphone, which leverages mobile neural engines and gigabytes of DRAM to classify complex activity patterns; finally, the smartphone transmits periodic compressed summaries over Wi-Fi or cellular networks to cloud infrastructure for longitudinal health modeling.

2.10.3 Why hybrid approaches work

Although the four deployment tiers differ by orders of magnitude in available energy, memory capacity, and compute throughput, they rest on identical systems foundations. In figure 2.13, a structural hierarchy illustrates how disparate deployment targets depend on shared data management, hardware-software execution abstractions, and resource scheduling principles.

This architectural convergence explains why engineering optimizations transfer bidirectionally across hardware scales: they target the same underlying physical bottlenecks. Across all tiers, reading an operand from off-chip memory incurs orders of magnitude higher energy and latency

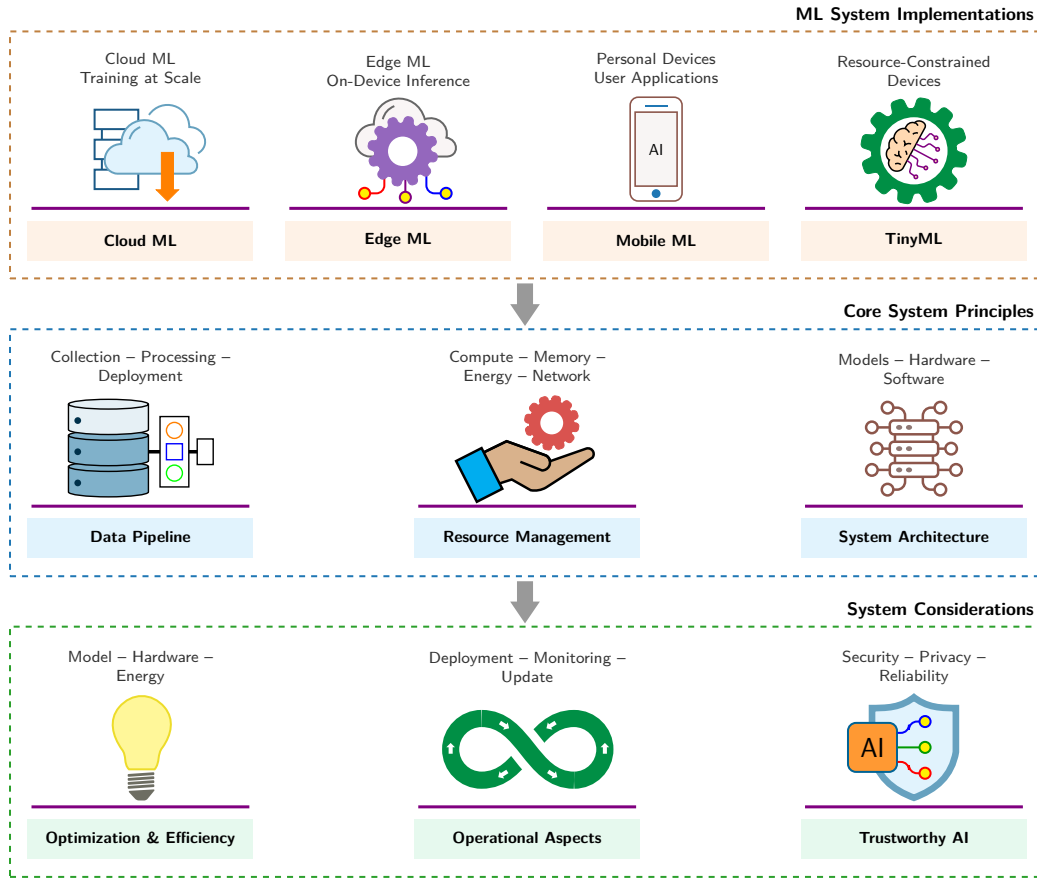


Figure 2.13: Convergence of ML Systems: Despite vast resource disparities across cloud, edge, mobile, and TinyML tiers, all deployments converge on three shared systems foundations: data pipelines, resource management, and hardware-software architecture. These shared pillars in turn support unified requirements for performance optimization, operational monitoring, and trustworthy engineering.

costs than performing an arithmetic operation. Because the memory wall governs both an embedded microcontroller and a warehouse-scale accelerator, techniques pioneered under extreme edge constraints directly relieve bottlenecks in the data center. Lower-precision numerical representations (such as FP8, INT8, and INT4), originally designed to fit model weights into megabytes of mobile DRAM or kilobytes of microcontroller SRAM, serve an identical economic purpose in cloud clusters: cutting DRAM traffic in half and multiplying effective tensor core throughput (chapter 10). Similarly, operator fusion and static memory staging developed to prevent off-chip spills on mobile SoCs directly reduce HBM traffic during cloud autoregressive decoding. In reverse, techniques for partitioning computational graphs across distributed cloud clusters (chapter 8) provide the foundational formalisms for splitting pipeline stages across heterogeneous mobile SoCs containing discrete CPUs, GPUs, and neural processing units.

The shared systems foundation extends through data engineering (chapter 4), model compression (chapter 10), and operational lifecycles (chapter 14). These engineering principles apply whether the execution target is a multi-node TPU pod or an ultra-low-power ESP32 microcontroller. Because the foundational abstractions remain invariant, so do their vulnerabilities: data distribution drift, model staleness, and telemetry degradation recur at every tier, requiring uniform monitoring discipline across the entire hybrid hierarchy.

**Checkpoint 2.3: Hybrid ML patterns**

Hybrid architectures work when partitioning work across tiers, not when copying the same pipeline everywhere.

Integration patterns

- ☐ **Train-serve split:** Can you explain why training in the cloud and serving on edge/mobile is often economically optimal, even when the model runs locally?
- ☐ **Hierarchical processing:** Can you describe what each tier does in a sensor → edge → cloud pipeline, and why pushing some decisions down reduces both latency and bandwidth?
- ☐ **Progressive deployment:** Can you explain how one model family becomes multiple deployed artifacts (cloud, edge, mobile, tiny) through systematic compression?

Design sanity checks

- ☐ **Boundary choice:** Given a concrete application, can you justify where the tier boundary should fall (latency, privacy, bandwidth, power), not just what model to use?
- ☐ **State synchronization:** Can you identify which feature definitions, model versions, and preprocessing logic must remain synchronized across tiers?

2.11 System Entropy: Why Deployment Is Not the End

Across the deployment paradigms summarized in figure 2.13, ML systems share an operational vulnerability absent from classical software. In traditional computing, a compiled binary operates under a deterministic contract: so long as memory cells retain their charges, processor buses transmit bits without corruption, and input buffers satisfy declared preconditions, a sorting routine or matrix inversion executes identically across millions of invocations. Machine learning breaks this permanence. Under the D·A·M taxonomy, the algorithm (A) and machine (M) remain static post-deployment: the model weights reside in memory, and the runtime executes the same compute graph on every forward pass. But the operational data distribution (D) continually evolves as user behaviors change, sensor calibrations drift, and environments fluctuate. Because model parameters represent an empirical fit to a fixed training distribution P_{train} , any divergence in the live distribution P_{live} induces **system entropy**: an ongoing decay in prediction quality that occurs even when code, model weights, and execution environments remain byte-for-byte identical.

The diagnostic approximation in equation 2.8 formalizes this degradation rate: system quality declines as the statistical distance $\mathcal{D}(P_{\text{train}}, P_{\text{live}})$ between the training and live distributions grows, scaled by the model's sensitivity S_{shift} to distributional shift:

$$\Delta\text{Quality} \approx -S_{\text{shift}} \cdot \mathcal{D}(P_{\text{train}}, P_{\text{live}}) \quad (2.8)$$

Because inference engines report success as long as matrix multiplications finish without numeric exceptions or memory faults, statistical degradation remains invisible to traditional infrastructure monitoring. An unmonitored model continues generating predictions with high confidence while its operational utility steadily erodes. Safeguarding system reliability therefore requires continuous statistical telemetry to detect when distribution drift demands model updates, an operational discipline analyzed in chapter 14.

When an unmonitored model is coupled directly to automated execution, statistical decay escalates rapidly into physical or financial failure. The collapse of Zillow Offers illustrates how compounding estimation error in an open-loop system can destroy an entire business unit.

**War Story 2.1: The Zillow Offers collapse (2021)**

Context: Zillow, a real-estate marketplace, launched “Zillow Offers” to buy homes directly through an iBuying business that depended on forecasting home prices and resale economics (Zillow Group 2021).

Mechanism: Zillow reported that it had purchased homes at prices above its revised estimates of future selling prices, while resale-capacity constraints delayed some closings (Zillow Group 2021). Forecast uncertainty became inventory exposure because the system coupled price estimates to purchase commitments.

Impact: Zillow wrote down \$304 million in housing inventory, laid off 25 percent of its workforce (2,000 employees), and permanently shut down the Offers division.

Response: A safer design would cap inventory exposure, preserve manual overrides, and place circuit breakers between forecast uncertainty and purchase decisions.

Systems lesson: Distribution shift converts statistical degradation into severe business risk when software couples predictions directly to balance-sheet actions. Production systems operating in dynamic environments require automated circuit breakers and feedback loops that constrain real-world exposure when live error distributions widen.

The Zillow Offers failure illustrates that a machine learning system encompasses both the predictive model and the operational feedback loop that acts on its outputs. Treating deployment as a terminal handoff, in which a trained model is frozen and left to execute unattended, ignores the reality of system entropy. Statistical decay in data (D) inevitably erodes the guarantees of the algorithm (A) and machine (M). Protecting a production pipeline requires engineering safeguards across the entire architecture, from input drift telemetry and validation gates to automated circuit breakers. Yet managing system entropy is only one part of operational discipline; navigating the physical constraints of deployment also requires unlearning deep-seated assumptions imported from classical software engineering.

2.12 Fallacies and Pitfalls

Beyond statistical decay, engineers frequently fall prey to architectural misconceptions about ML deployment. The physical constraints governing data movement, thermal dissipation, and hardware balance create counterintuitive behaviors that challenge standard software engineering intuitions. The following fallacies and pitfalls highlight common failure modes where systems are critically mismatched to their physical operating envelopes.

Fallacy: *One deployment paradigm solves all ML problems.*

Physical constraints create hard boundaries that no single paradigm can span. The memory-wall discussion in section 2.4 demonstrates that memory bandwidth, memory capacity, and access latency scale far more slowly than raw arithmetic compute, producing distinct physical bottlenecks across deployment tiers. Table 2.13 lists illustrative deployment envelopes rather than physics-only latency bounds. A real-time robotics system requiring sub-10 ms response cannot use a remote cloud path whose end-to-end network round-trip exceeds that budget, while a billion-parameter language model cannot reside within the 256 KB SRAM of a microcontroller regardless of compression. Robust architectures frequently combine paradigms, leveraging cloud resources for distributed training and burst capacity while executing localized inference or sensor preprocessing on client devices.

A related misconception assumes that moving computation closer to the user always reduces latency, overlooking the execution delay introduced by thermally constrained or lower-bandwidth edge processors—a trade-off explored in inference benchmarks (section 12.8).

Pitfall: *Relying on model optimization to overcome mobile power and thermal limits.*

Compression techniques cannot overcome fundamental thermodynamic limits. Consider a smartphone equipped with a 15 Wh battery. A light inference workload drawing 1 W runs for $\frac{15 \text{ Wh}}{1 \text{ W}} = 15 \text{ h}$, but an intensive workload drawing 5 W—typical of unoptimized on-device foundation models—depletes that same battery in $\frac{15 \text{ Wh}}{5 \text{ W}} = 3 \text{ h}$.

Sustained execution at 5 W also triggers severe thermal throttling, as smartphone chassis budget only approximately 3 W for steady-state passive dissipation. While quantizing FP32 parameters to INT8 yields an illustrative 4× reduction in memory footprint, dynamic switching energy and static leakage do not scale down proportionally. A continuous inference pipeline that exceeds the passive

thermal envelope inevitably forces dynamic frequency scaling, requiring engineers to lower duty cycles, restructure model architectures, or migrate compute to dedicated accelerator silicon.

Fallacy: *TinyML represents scaled-down mobile ML.*

The boundary between TinyML and mobile deployment is qualitative: physical constraints force fundamentally different algorithmic and execution models. As section 2.8.1 establishes, microcontrollers provide 256 KB to 1 MB of SRAM compared to mobile platforms with 8–16 GB of LPDDR DRAM—a 10,000× capacity disparity that dictates memory residency. Where mobile systems execute models with tens of millions of parameters, TinyML architectures are restricted to tens or hundreds of thousands of parameters, demanding specialized depthwise operations and layer scheduling designed to minimize peak activation memory. Operating power budgets exhibit an even sharper discontinuity: mobile inference draws 3–5 W, whereas TinyML targets 1–10 mW to operate within energy-harvesting envelopes. Teams that apply mobile optimization techniques directly to TinyML environments discover that post-training quantization from FP32 to INT8 is insufficient when total working memory must fit inside 64 KB of SRAM, forcing complete algorithmic redesign.

Pitfall: *Minimizing computational resources minimizes total cost.*

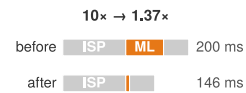
Optimizing per-inference compute resources in isolation often increases operational expenditure and slows deployment velocity. As the decision framework in section 2.9.2 emphasizes, architectural selection requires evaluating total cost of ownership (TCO) across the entire system lifecycle rather than raw server bills. In this scenario, a cloud inference service costing \$2,000/month in compute appears expensive compared to \$500/month in amortized edge hardware. However, a distributed edge fleet introduces recurring operational expenses: network engineering (\$3,000/month), field hardware maintenance (\$500/month), and fleet reliability engineering (\$2,000/month). These push total edge operational costs to \$6,000/month—a 3× increase over the cloud baseline. Furthermore, if a cloud-managed service reaches production in two months while custom edge infrastructure requires six, the four-month delay introduces severe opportunity costs. Sound systems engineering balances compute efficiency against developer velocity, fleet manageability, and operational maintenance.

Fallacy: *Model optimization translates linearly to system speedup.*

Amdahl's law⁴⁰ dictates that end-to-end speedup is strictly bounded by the execution time of unaccelerated stages. Formalized through the bottleneck principle (section 2.3.1) and derived in section D.2.3.1, the achievable system speedup is $\text{Speedup}_{\text{overall}} = \frac{1}{(1-p) + \frac{p}{s}}$, where p is the fraction of total runtime subject to acceleration and s is the speedup factor of that isolated fraction. Consider tapping the shutter on a smartphone camera. As diagrammed in the margin, the capture pipeline processes raw sensor frames through 100 ms of image signal processing (ISP auto-exposure and white balance), 60 ms of ML scene classification, and 40 ms of tone mapping and HDR merge—totaling 200 ms. Accelerating the neural network stage by 10× (6 ms instead of 60 ms) reduces total latency from 200 ms to 146 ms. Despite a tenfold model speedup, end-to-end latency improves by only 1.37×. Even reducing model latency to zero ($s = \infty$) yields a maximum speedup of 1.43×, bounded by the remaining 70 percent of unaccelerated pipeline execution. Optimizing an ML system requires profiling the full execution graph; accelerating an isolated stage yields diminishing returns once that stage ceases to be the critical-path bottleneck.

Pitfall: *Assuming more training data always improves deployed model performance.*

Three constraints bound the returns of data scaling, as analyzed across the workload archetypes in section 2.3. First, algorithmic capacity (A) saturates: when a model's parameter budget is fixed, additional training data cannot overcome representational bottlenecks once the architecture exhausts its expressive capacity. Second, data quality (D) dominates raw volume: label noise, sensor calibration errors, and artifactual correlations contaminate gradient updates, causing scaled-up corrupted corpora to reinforce systematic bias rather than improve generalization. Third, distributional alignment determines real-world utility: high-fidelity, in-domain operational data consistently yields superior serving accuracy compared to massive, out-of-domain datasets that fail to reflect production physical environments.



A faster model stage does not linearly speed up a camera pipeline.

⁴⁰ | **Amdahl's law:** Formalized by Amdahl (1967) for multiprocessor scaling, this principle applies directly to ML deployment pipelines where the model is only one stage among many. In this camera pipeline example, ML inference is 60 ms of 200 ms; even a 100× model speedup yields only about 1.4–2× end-to-end improvement because the rest of the pipeline is unchanged. Teams that benchmark model latency in isolation systematically overestimate deployment gains.

Fallacy: *One model binary can serve every edge hardware target efficiently.*

Teams frequently compile a single model artifact and deploy it uniformly across diverse devices, treating deployment as a packaging step rather than a hardware co-design problem. In practice, hardware-specific compilation and kernel lowering capture critical efficiencies that generic binaries forfeit. For example, executing quantized INT8 operations on a dedicated Neural Processing Unit (NPU) achieves substantially higher throughput per watt than running the identical model on a general-purpose CPU, because fixed-function INT8 tensor engines avoid instruction-decode overhead and floating-point datapath energy. Furthermore, operator fusion preserves intermediate activation tensors within on-chip SRAM or register files rather than evicting them to energy-expensive off-chip DRAM. Similarly, tuning memory layouts to align tensor channels with accelerator vector registers eliminates uncoalesced memory transactions. As the deployment paradigm analysis in section 2.1 establishes, hardware architectures impose divergent execution constraints: a binary compiled for an Arm Cortex-A78 CPU fails to leverage the tightly coupled scratchpad SRAM and systolic arrays of an Arm Ethos-U NPU. A portable model artifact serves only as an intermediate representation; production deployment requires compiling and profiling against the target runtime, hardware delegate, memory hierarchy, and thermal envelope.

2.13 Summary

ML systems engineering is governed by the physical reality that every deployment target enforces a distinct hardware contract. The iron law of ML systems ($T = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \eta_{\text{hw}}} + L_{\text{lat}}$) unifies these physical limits into a single execution model: propagation delay contributes fixed latency (L_{lat}), the memory wall bounds data movement (D_{vol}/BW), and thermal power budgets cap sustained compute. Across cloud, edge, mobile, and TinyML tiers, the categorical power envelope from megawatts to milliwatts spans nine orders of magnitude, while representative systems occupy narrower ranges within it.

By applying the bottleneck principle to workload archetypes, engineers determine whether a model's latency is dominated by compute efficiency, memory bandwidth, or network transit before committing to single-node or hybrid architectures. The quantitative trade-offs (table 2.13) and systematic decision framework (figure 2.11) developed in this chapter establish the physical baselines that govern all downstream decisions in model compression, data pipeline design, and high-performance serving.



Key Takeaways: Same model, different engineering

- **Physical constraints define feasibility:** Speed of light (~36 ms cross-country round-trip), power wall, memory wall, and latency budgets create hard boundaries that engineering cannot overcome, only navigate.
- **Identify bottlenecks before optimizing:** The same model is compute bound in training but memory bound in inference. The iron law and bottleneck principle pinpoint which constraint dominates; optimizing the wrong term yields zero speedup.
- **Workload archetypes determine deployment feasibility:** A Compute Beast such as large-scale image-model training often needs cloud-scale resources; a Tiny Constraint such as keyword spotting requires microcontroller efficiency. The same optimization strategy cannot serve both; engineers must match the workload archetype to the deployment paradigm.
- **Deployment power spans nine orders:** Cloud data-center infrastructure operates at megawatt scale, while TinyML systems target milliwatts. This gap enables entirely different application classes rather than representing a limitation.
- **Hybrid architectures are prevalent in production systems:** Voice assistants span TinyML (wake-word), mobile (speech-to-text), and cloud (language understanding). When one paradigm does not suffice, integration patterns (train-serve split, hierarchical processing, progressive deployment) formalize how paradigms combine.

- **System-level speedup obeys Amdahl's Law, not model-level gains:** In the worked camera pipeline, a $10\times$ faster model yields only $1.37\times$ system speedup when ML accounts for 30 percent of the pipeline. Profile the full system before optimizing any component.
- **Deployment begins a feedback loop:** Live data can diverge from training conditions while code and weights stay fixed, so monitoring and bounded operational exposure are part of the system architecture.

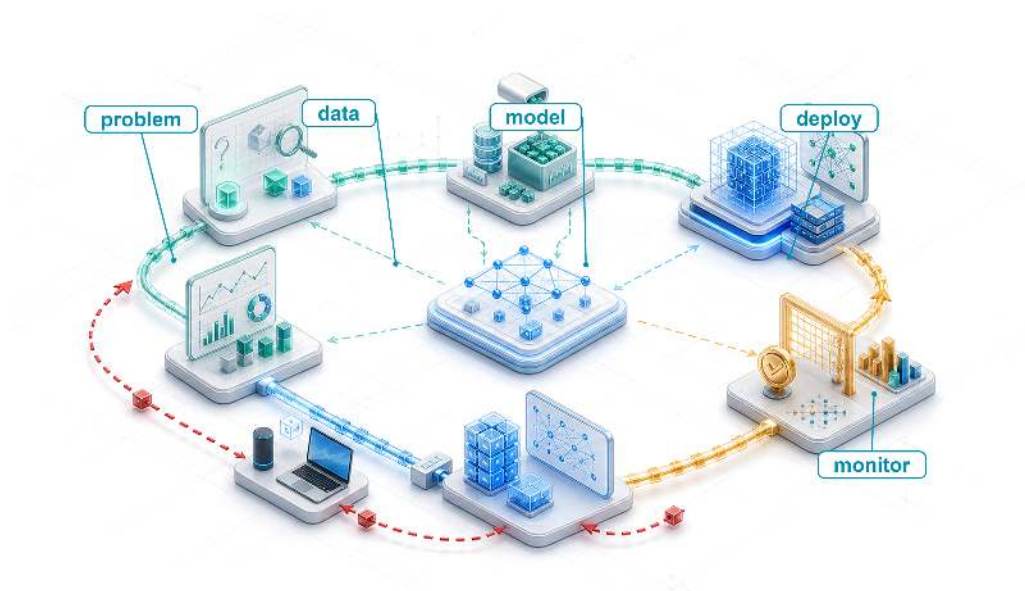
Deployment paradigms are not arbitrary design preferences; they are continuous operational regions dictated by physical laws. Selecting a deployment target changes which term of the iron law is most likely to bind: data-center training can push against peak compute (O/R_{peak}), mobile inference can collide with memory bandwidth (D_{vol}/BW), and remote real-time inference can be constrained by propagation delay (L_{lat}). Systems engineering begins by measuring that binding constraint. Optimizing model architecture or software runtimes without respecting the underlying hardware contract produces fragile systems that fail under real-world operational pressure.



What's Next: From theory to process

While the physical constraints and decision framework established in section 2.9.2 define where an ML system can physically execute, physical feasibility alone does not guarantee operational success over time. As demonstrated by Zillow's \$304 million write-down (Zillow Group 2021), deploying models into production requires containing forecast uncertainty and managing operational risk across the software lifecycle. Chapter 3 introduces the ML development lifecycle, the systematic engineering process that carries a system from problem formulation through deployment, monitoring, and maintenance.

ML Workflow

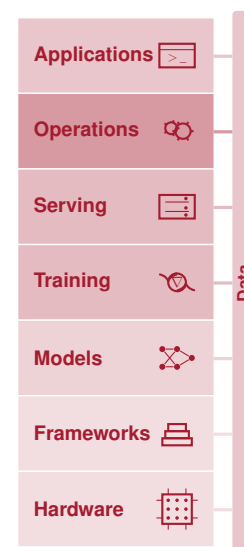


- 3.1 ML Lifecycle
- 3.2 Lifecycle Stages
- 3.3 Problem Definition
- 3.4 Data Collection
- 3.5 Model Development
- 3.6 Evaluation and Validation
- 3.7 Deployment and Integration
- 3.8 Monitoring and Maintenance
- 3.9 Systems Thinking
- 3.10 Fallacies and Pitfalls
- 3.11 Summary

Purpose

Why is seeing the whole map necessary before walking any single path?

The D·A·M taxonomy names the components of every ML system, and deployment location determines the physical constraints each component must satisfy. Teams often treat these as separate concerns, with one team collecting data, another designing the model, and a third provisioning hardware. Yet the taxonomy's deepest lesson is that these components interact. The collected data constrains which algorithms are feasible. The chosen algorithm dictates what hardware can run it. The target hardware reshapes what data can be processed. Pull on any single thread and the entire system shifts. These interactions play out across components and across time. A model that performs well at launch may degrade as the data distribution shifts, prompting investigation and, when evidence warrants it, model or data updates. Optimizing each piece in isolation is how teams build accurate models that cannot be deployed and efficient pipelines that feed the wrong data. A data engineer who sees how preprocessing choices constrain downstream architectures builds different pipelines than one who treats data preparation as an isolated task; a model developer who knows the deployment target's memory budget from day one makes different architecture decisions than one chasing accuracy in a vacuum. Before the details of any one component can be understood, the full map must show how an ML system is built, evaluated, and sustained as a coherent whole. The ML workflow sets that map in motion through iterative D·A·M co-design across data, algorithm, and machine until their emergent capability meets the requirements of the real world.





Learning Objectives

- Explain the six ML lifecycle stages as coordinated data-algorithm-machine decisions with feedback
- Compare ML workflows with traditional software using drift, nondeterminism, and operational feedback
- Analyze how problem-definition constraints propagate through data, modeling, validation, and deployment
- Calculate late-discovery integration costs using the workflow's constraint propagation principle
- Evaluate accuracy, efficiency, reproducibility, and deployment readiness trade-offs across lifecycle stages
- Design feedback loops that connect monitoring signals to retraining, maintenance, and revised requirements

3.1 ML Lifecycle

A typical deployment failure illustrates the danger of uncoordinated engineering. Day one: “Build a diagnostic model for rural clinics.” Day 90: 95 percent accuracy on the test set. Day 120: 96 percent accuracy after a month of architecture tuning. Day 150: the model is handed to deployment engineers. Day 151: deployment engineers report that the model requires 4 GB of memory. Day 152: someone checks the deployment target—tablets in mobile clinics with 512 MB available. Day 153: five months of work is discarded.

The model's accuracy was excellent. The team's machine learning skills were excellent. *The failure was a workflow failure.* A deployment constraint that should have shaped every decision from day one was discovered only after the work was done. The tablet's memory limit should have propagated backward to the first architecture meeting, constraining which models were even worth considering. Instead, the team optimized each component in isolation (data collection, architecture selection, training), and the integration failure appeared only when the pieces were assembled. A documented diabetic retinopathy (DR) deployment exposed analogous workflow and infrastructure gaps.¹

ML systems combine Data, Algorithm, and Machine under physical constraints that partition deployment into Cloud, Edge, Mobile, and TinyML, each imposing hard physical limits on compute throughput, memory capacity, and thermal dissipation. Connecting these components into a functioning system requires orchestrating how constraints propagate across Data, Algorithm, and Machine.

The **ML workflow** is an engineering framework designed to prevent integration failures by making constraints explicit at each development stage and tracing how they propagate across Data, Algorithm, and Machine. It marks the shift from model researcher to systems engineer. A researcher optimizes individual elements: a better architecture, a cleaner dataset, a faster accelerator. A systems engineer orchestrates those elements into production systems that reliably deliver value. The day-153 failure did not stem from an isolated modeling, data, or hardware defect; it arose from the unmanaged interaction among all three. The workflow supplies the mental map that keeps technical decisions attached to the larger system.

The **machine learning lifecycle** is the orchestration framework for this work: a structured, iterative process² that guides the development, evaluation, and improvement of ML systems (Amershi et al. 2019). The formal definition emphasizes continuous management rather than a one-time release.



Definition 3.1: Machine learning lifecycle

Machine learning lifecycle is the iterative engineering process of building, deploying, monitoring, and updating ML systems, where evidence from later stages can feed back to earlier stages because model performance may change after deployment.

1 | Lab-to-deployment gap:

Beede et al. (2020) documented this empirically for a deep-learning diabetic retinopathy screening system deployed in Thai clinics. Although the system had specialist-level accuracy in earlier validation, 393 of 1,838 submitted images (21 percent) failed its image-quality requirements, often because clinic lighting, camera maintenance, or dilation practices did not match the system's assumptions. Those rejections added work for nurses, forced retakes or referrals, and exposed workflow and infrastructure barriers rather than simply a model-accuracy problem.

2 | CRISP-DM (cross-industry standard process for data mining):

CRISP-DM codified data-intensive system development as six interconnected, iterative phases rather than a linear waterfall (Chapman et al. 2000); its core design principle (feedback loops between all phases) directly informs the modern ML lifecycle's structure. Boehm's software-engineering economics work discussed the higher cost of late fixes (Boehm 1981). The workflow model later uses $2^{N_{\text{stage}}-1}$, where N_{stage} is the lifecycle stage index, as an illustrative sensitivity scenario rather than an empirical cost law.

1. **Significance:** The lifecycle is a closed loop, not a linear pipeline. Distribution divergence $\mathcal{D}(P_t \| P_0)$ between current and training traffic is an alert signal that raises the probability of accuracy loss; the exact relationship depends on the model, labels, loss, and deployment distribution. A full retraining run incurs the $O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$ compute cost, while partial or incremental updates may cost less. Drift velocity and validation delay therefore turn lifecycle maintenance into a budgeting problem, not just an engineering process.
2. **Distinction:** Unlike traditional software, whose behavior changes when code, configuration, dependencies, or environment changes, ML behavior can also change as the world changes. A deployed model's accuracy may change through distribution shift even when the code, infrastructure, and configuration remain untouched.
3. **Common pitfall:** A frequent misconception is that the lifecycle ends at deployment. In reality, deployment is the beginning of the feedback loop: production monitoring surfaces drift, drift triggers investigation, and outcome evidence determines whether a retrained model should re-enter the deployment stage.

Here, *lifecycle* describes the stages themselves and *workflow* describes the engineering discipline of orchestrating them; the lifecycle is *what gets traversed*, the workflow is *how the traversal is managed*. This distinction requires **systems thinking**: analyzing how a system's parts interrelate rather than treating them in isolation. The patterns formalized in section 3.9 and illustrated through the detailed case study explain why ML systems require integrated engineering approaches rather than sequential component optimization.

Understanding how these stages interconnect is essential before analyzing individual layers, as each technical domain—from storage pipelines (chapter 4) to model training (chapter 8), execution frameworks (chapter 7), and serving infrastructure (chapter 13)—operates as an interdependent system. In figure 3.1, the horizontal layout separates the top data pipeline from the bottom model development pipeline, with curved return vectors illustrating how operational insights and data defects cycle back into earlier stages.

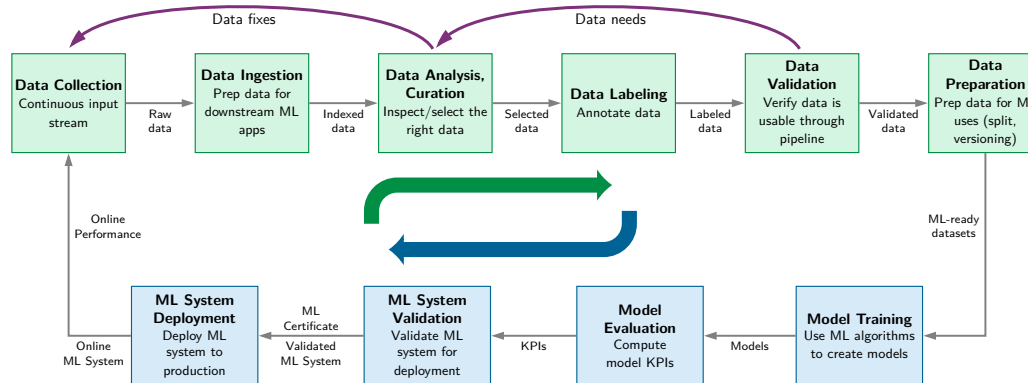


Figure 3.1: Dual-Pipeline ML Development: Machine learning engineering decouples into parallel data and model pipelines operating on different timescales. While the model pipeline executes rapid inner-loop training and evaluation, outer-loop feedback from production deployment and data validation continuously reshapes upstream collection, labeling, and curation.

The conceptual stages of the ML lifecycle establish the *what* and *why* of the development process. The operational layer constitutes the *how*: the implementation of this lifecycle through automation, tooling, and infrastructure. Chapter 14 names and develops those practices in detail. This distinction matters: the lifecycle is the conceptual framework; operational infrastructure is the machinery that implements it at scale.

3.1.1 Quantifying the ML lifecycle

Practitioner time allocation exposes where the lifecycle actually bottlenecks: engineering effort rarely concentrates where research literature spends its pages. Conceptual models of the ML lifecycle

describe which stages exist, but systems decisions turn on quantitative reality—where engineering hours are spent, how much compute is consumed, and which pipeline stalls delay deployment.

Practitioner surveys reveal a stark asymmetry between where research attention centers and where engineering hours are spent, with data collection and preparation consuming 79 percent of practitioner time. In figure 3.2, a pie chart breaks down reported primary time sinks, contrasting the dominant data collection and preparation slices against the narrow share occupied by model architecture and algorithm tuning.

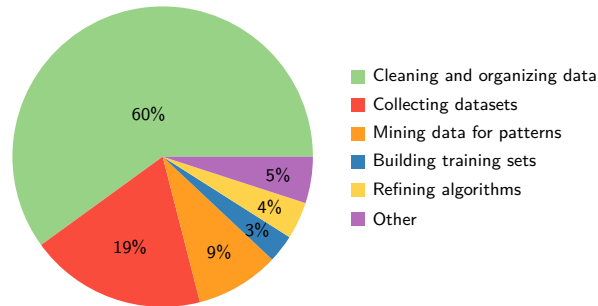
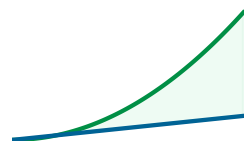


Figure 3.2: Data Scientist Most-Time Responses: In CrowdFlower’s 2016 survey, 60 percent of respondents selected cleaning and organizing data as the activity where they spent the most time, with 19 percent selecting data collection. Model-focused responses such as pattern mining, training set construction, and algorithm refinement together represent roughly 16 percent of responses. Source: CrowdFlower, *Data Science Report 2016*.

These survey proportions demonstrate why data engineering cannot be treated as mere preparatory plumbing. Data pipelines are the primary source of operational friction, iteration cycles, and system-level risk. Mastering data infrastructure (chapter 4) provides maximum leverage before addressing the model architectures and training optimizations that ingest its output.

Beyond time allocation, iteration velocity governs project progress. The return vectors in figure 3.1 are not rare exceptions; they are paths teams traverse repeatedly. When data quality degrades through distribution shift or incorrect labels, rework sends teams back to collection. When an architecture exceeds accelerator memory or suffers numerical instability, engineers must revisit model design. When the serving layer exceeds latency budgets, the deployment pipeline stalls. Each loop consumes calendar time and compute budget, increasing development cost whenever downstream stages fail to catch violations early.



Under the stated assumptions, the faster-cycle model finishes slightly ahead after 26 weeks.

Napkin Math 3.1: The iteration tax

Problem: A DR screening system for rural clinics must choose between a large ensemble trained on high-resolution fundus images (training time: 1 week, accuracy: 95 percent) and a lightweight model suitable for edge deployment on clinic hardware (training time: 1 hour, accuracy: 90 percent). Which model has higher modeled accuracy after six months under these assumptions?

Math: In six months (~26 weeks), the possible experiment count is:

1. Large model: 26 weeks of calendar time at 1 week per experiment. Each experiment improves accuracy by an assumed constant ~0.15 percentage points.
2. Small model: 26 weeks of calendar time at 168 h/week gives 4,368 possible experiments at 1 hour each. Machine time is not the binding constraint at that cycle length, so this scenario counts only 100 of them as *effective* experiments, the number a team can realistically design, run, and learn from in six months. The remaining capacity sits idle for want of hypotheses. Even with a smaller assumed gain per iteration, more useful iterations can produce a larger cumulative gain.

Result: If each iteration improves accuracy by 0.1 percentage points on average, the small model starts at 90 percent and reaches 100 percent after 100 effective iterations, before applying the 99

percent ceiling. With the 99 percent ceiling applied, the small model achieves 99 percent. The large model starts at 95 percent and reaches 98.9 percent after 26 slower iterations. In practice, the small model’s rapid iteration enables discovering better architectures, label-preserving data augmentations, and tunable training settings.

Systems insight: *Iteration velocity is a feature.* When candidate systems have comparable attainable quality and useful experiments have similar expected value, shorter cycles expand how much of the design space a team can test. Speed alone cannot guarantee that a smaller model will outperform a larger one. In the DR screening scenario, the lightweight model’s rapid iteration cycle enables the team to experiment with label-preserving input transformations, preprocessing pipelines, and architecture variations far more quickly.

This cumulative delay defines the **iteration tax**: time spent waiting on slow training runs or cumbersome validation pipelines reduces how many candidate models a team can test. Late discoveries compound this penalty,³ as formalized by the constraint propagation principle (section 3.9.1). Violations discovered after deployment force expensive rollbacks across every preceding stage. This asymmetry demands explicit **stage interface contracts**: validating outputs at every stage boundary intercepts defects before they compound into systemic rework. Section 3.2.2 formalizes these contracts across the six stages.

The compounding cost of iteration delays and late discoveries reveals a broader structural reality: *ML workflows are not slow versions of traditional software lifecycles.* They differ fundamentally in where engineering effort concentrates, how feedback loops alter the system over time, and how late discoveries expand rework.

3.1.2 ML vs. traditional software

Traditional and ML systems can both use iterative lifecycles and exhibit nondeterministic behavior.⁴ The relevant distinction is how application behavior is specified. Rule-based software encodes explicit logic, while ML systems learn statistical mappings from data (section 1.6). This adds data, evaluation, and monitoring concerns to established software-engineering practices.

Machine learning systems therefore require additional workflow controls. Consider financial transaction processing: a deterministic authorization rule follows explicit logic and can execute quickly once its inputs are available. An ML-based fraud detector adds learned scoring, feature retrieval, and statistical decision logic. The model stage may take milliseconds, but end-to-end latency also depends on storage, networking, and surrounding services. This shift from explicit programming to learned behavior reshapes the development lifecycle, altering how teams establish reliability and robustness.

These differences alter how lifecycle stages interact. While conventional software engineering relies on production feedback to refine code and requirements, ML systems introduce feedback loops where operational data reshapes training distributions, detected drift prompts investigation, and runtime errors expose dataset blind spots. Table 3.1 contrasts traditional software and ML engineering across six core lifecycle dimensions, highlighting how data evolution redefines testing, deployment, and ongoing maintenance.⁵

Table 3.1: Traditional Software vs. ML Development Lifecycles: Six dimensions where ML development diverges from traditional software engineering. Both use production feedback, while ML systems add loops in which deployment insights reshape data collection, monitoring drives model updates, and production experience informs model design.

Aspect	Traditional Software Lifecycles	Machine Learning Lifecycles
Problem Definition	Precise functional specifications are defined upfront.	Performance-driven objectives evolve as the problem space is explored.
Development Process	Iterative code, configuration, and interface development.	Iterative experimentation with data, features, and models.
Testing and Validation	Many functional tests have exact expected results; performance and reliability remain quantitative.	Statistical validation and metrics that involve uncertainty.

3 | **Late correction costs:** Boehm’s *Software Engineering Economics* (Boehm 1981) discussed how defects found late can cost more to fix than those caught during requirements, implementation, and testing. In ML systems, late-discovered constraints may require retraining, data-pipeline changes, and renewed validation. The doubling rule is an illustrative sensitivity scenario, not an empirical cost law.

4 | **Waterfall model:** A plan-driven lifecycle is often depicted as a sequence of requirements, implementation, and testing, but Royce (1970) warned that a purely sequential implementation was risky and described iteration between successive phases. ML systems add data and model feedback that must be managed explicitly. This chapter uses an illustrative sensitivity model for ML lifecycle stages.

5 | **Data versioning:** Unlike code, which changes through discrete, auditable commits, data can drift gradually (distribution shift), suddenly (schema migration), or subtly (label quality degradation). Plain Git is impractical for many multi-terabyte datasets, so teams commonly version external-data manifests or pointers using tools such as DVC and Git LFS. Without data versioning, teams cannot reliably reproduce a prior training run or determine whether an accuracy regression stems from a code change or a data change.

Aspect	Traditional Software Lifecycles	Machine Learning Lifecycles
Deployment	Behavior remains static until explicitly updated.	Performance may change over time due to shifts in data distributions.
Maintenance	Maintenance involves modifying code to address bugs or add features.	Continuous monitoring, updating data pipelines, retraining models, and adapting to new data distributions.
Feedback Loops	Production feedback commonly changes code, configuration, and requirements.	Insights from deployment and monitoring often refine earlier stages like data preparation and model design.

 Checkpoint 3.1: ML vs. traditional software

- ML systems are not traditional software with a model attached. Check the differences that force a separate workflow:
- ☐ Failure modes: Can you distinguish silent degradation from explicit failures, recognizing that either can occur in ML and conventional systems?
 - ☐ Logic source: Can you explain why changing training data can change model behavior even when the code remains fixed?
 - ☐ Iteration: Can you explain why ML requires monitoring and, when evidence warrants it, model or data updates?

3.1.3 Data locality at scale

Large ML data pipelines can stress the locality optimizations used by modern operating systems. Unlike traditional software pipelines that scan records sequentially, machine learning training deliberately shuffles examples across epochs to satisfy the independent and identically distributed (i.i.d.) sampling required by stochastic gradient descent. This algorithmic randomization directly conflicts with hardware and operating system assumptions: OS kernels rely on spatial and temporal locality,⁶ the tendency for a program that reads byte *X* to read nearby bytes soon and to reuse recently accessed memory.

When randomized sample order maps to scattered physical reads, shuffling can reduce the effectiveness of file-system buffers and virtual-memory prefetchers. The penalty depends on how logical examples map to physical storage. Contiguous reads from shuffled shards may retain useful locality, whereas small reads across many shards expose storage and network latency. Layout-aware batching, caching, and explicit prefetching can restore locality and reduce these stalls. The relevant systems variable is the physical access pattern presented to the memory hierarchy, not randomization alone.

That distinction determines where to optimize. If the loader already issues large sequential reads, more caching adds little benefit. If it fans out small reads across many shards, repacking records or increasing read granularity matters far more than adding compute. Profiling should trace the complete access path from storage to accelerator, measuring storage request size, queue depth, cache hit rate, and accelerator idle time together to prevent I/O bottlenecks from idling compute resources.

3.2 Lifecycle Stages

The rural-clinic failure shows why ML projects need an explicit six-stage framework: deployment constraints surfaced only after data, model, and evaluation decisions had hardened around the wrong target. Conventional and ML lifecycles both use iteration, while ML systems add feedback from deployment into data, training, and evaluation. The six-stage framework captures this loop.

The complete machine learning lifecycle distills into six core stages laid out across two functional rows. In figure 3.3, trace the top row from problem formulation through model evaluation, and follow the bottom row as deployment feeds monitoring signals back into upstream data collection.

MobileNetV2 grounds these stages in physical constraints, where a small footprint makes edge deployment limits visible from the outset. For MobileNetV2, problem definition establishes tight constraints: about 14 MB model size, about 600 MFLOP, and real-time inference on mobile-class

⁶ | **Locality of reference:** Denning (1968) formalized the working-set principle for virtual memory. Randomly ordered examples can reduce locality when the logical order maps to scattered physical reads, but the effect depends on storage layout, batching, caching, and prefetching.

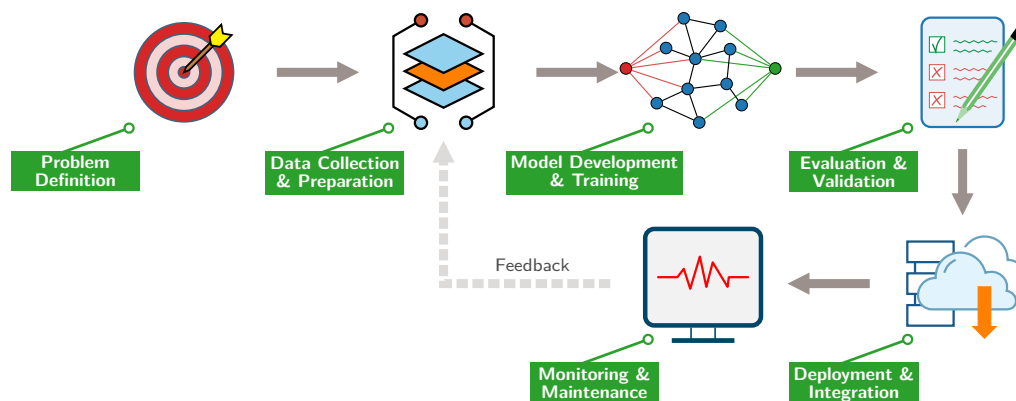


Figure 3.3: Simplified Lifecycle with Feedback: The six core stages of ML engineering form a closed loop rather than a linear handoff. A primary feedback path from monitoring back to data collection supports investigation and adaptation as production distributions shift, model accuracy degrades, and operational constraints evolve.

hardware. Those constraints shape the rest of the workflow immediately. Data collection must account for on-device preprocessing limitations, and model development must choose an architecture whose operations fit the budget. MobileNetV2 achieves this through depthwise separable convolutions,⁷ factorizing spatial and channel operations rather than simply pruning parameters. Evaluation validates both accuracy and latency on target devices, deployment tests whether the model fits the device’s memory and power envelope, and monitoring tracks performance across diverse device populations. Each stage’s decisions propagate through subsequent stages, and the workflow framework makes these dependencies explicit. A DR screening model optimized for rural clinic deployment faces analogous pressures: limited device memory, strict power budgets, and the need for real-time inference without reliable connectivity. These shared constraints make DR an effective running case study.

Tracing MobileNetV2 through these transitions demonstrates that while a workflow diagram suggests sequential progression, production engineering is fundamentally cyclic, coupling downstream deployment constraints directly to upstream design.

Checkpoint 3.2: The workflow cycle

Verify that each stage enforces contracts that bind upstream choices to physical deployment constraints.

- ☐ **Problem definition:** Can you specify the latency, memory, and accuracy targets that bound downstream engineering?
- ☐ **Data preparation:** Can you trace how raw dataset resolution and preprocessing choices limit achievable serving throughput?
- ☐ **Model development:** Can you verify that operator choice and memory footprint fit the deployment budget before scaling training?
- ☐ **Deployment feedback:** Can you identify the monitoring signals required to trigger upstream retraining when production distributions shift?

Beyond procedural orchestration, each lifecycle stage maps directly to specific terms in the performance equation. This mapping reveals a workflow-level manifestation of the iron law of ML systems (section 1.7): data preparation locks in byte volume, model development dictates operation count and hardware efficiency, and deployment infrastructure bounds fixed runtime overhead.

The binding constraint differs dramatically across workload archetypes, causing each lifecycle stage to optimize different iron law terms. ResNet-50, DLRM, and keyword spotting (KWS) are useful anchors because each stresses a different part of the system: dense vision training tries to

⁷ | **Depthwise separable convolutions:** Replacing a standard convolution with this cheaper factorization reduces computation by roughly 8–9× for typical kernel sizes (Sandler et al. 2018), which is what makes the roughly 600 MFLOP inference budget plausible on mobile-class hardware. Chapter 6 covers the architectural mechanism in depth.

keep accelerators busy, sparse recommendation spends much of its time moving embedding rows, and keyword spotting is constrained first by tiny memory and always-on energy budgets.

Production systems rarely fall neatly into a single archetype. A medical imaging classifier, for instance, requires sustained computation while training over large image datasets, yet faces strict energy and memory limits when deployed to portable clinic hardware. Understanding how the workflow framework adapts across these trade-offs is essential for sound engineering decisions. Table 3.2 shows how three selected workflow stages manifest for these recurring workload archetypes.

Table 3.2: Workflow Variations by Lighthouse Model: Three selected lifecycle stages target different iron law terms depending on the workload’s binding constraint. ResNet-50 emphasizes useful computation per unit time, DLRM emphasizes data movement and embedding freshness, and KWS emphasizes energy and memory capacity.

Stage	ResNet-50 vision training	DLRM recommendation	KWS TinyML
Data Eng	Keep image batches available fast enough to sustain high GPU utilization.	Keep online feature-store lookups within the serving latency budget; embedding tables dominate storage and freshness.	Curate short audio clips for an SRAM-constrained device.
Training	Coordinate preprocessing, batching, mixed precision, and model execution to reduce accelerator idle time.	Optimize sparse embedding lookups because memory bandwidth limits throughput.	Search for the smallest model family that still recognizes the trigger phrase reliably.
Deploy	Use large batches when throughput and cost matter more than single-request latency.	Meet strict interactive p99 latency targets while keeping features fresh.	Stay within a strict always-on energy budget.

Each stage of this workflow presents distinct systems challenges, from curating multi-terabyte datasets to sustaining tail latency in production. Grounding these challenges requires a quantitative lens on how stages govern execution.

🔍 Systems Perspective 3.1: The iron law of workflow

The six lifecycle stages alter distinct terms in the iron law of ML systems ($T = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}}$). Rather than rigid one-to-one assignments, the mappings serve as diagnostic starting points:

- Problem definition: Sets target constraints for accuracy, latency, cost, privacy, and deployment paradigm. These targets determine which terms of the equation are permitted to grow and which must be bounded from the outset.
- Data collection and preparation: Shapes dataset composition and the byte volume (D_{vol}) presented to downstream pipelines. Curation can also change the training work required to reach a quality target.
- Model development and training: Dictates the operation count (O), data reuse patterns, parameter movement, and achievable hardware efficiency (η_{hw}).
- Evaluation and validation: Tests whether model quality and end-to-end execution meet deployment requirements on the target system.
- Deployment and integration: Determines the serving path, including runtime data movement, execution efficiency, and fixed overhead (L_{lat}).
- Monitoring and maintenance: Observes distribution drift, latency, throughput, and operational cost after launch, feeding violations back into earlier stages for re-optimization.

Viewing the lifecycle through the iron law grounds latency and cost in hardware physics. Production workflows must simultaneously satisfy accuracy, safety, privacy, and organizational constraints that sit outside the equation.

3.2.1 Case study: DR screening

Diabetic retinopathy (DR) screening serves as the running case study throughout this chapter because it passes three critical tests: it appears straightforward on the surface but reveals deep operational friction in practice, it exercises the full deployment spectrum from cloud training to edge inference, and its challenges are thoroughly documented across real-world deployments. The literature spans both sides of the lab-to-field divide: Gulshan et al. (2016) document the benchmark validation study for automated DR detection, while Beede et al. (2020) document the workflow and infrastructure failures that emerged when a deep-learning system was deployed in Thai clinics. Together, these sources provide an empirical baseline tracking data collection and validation through to physical deployment constraints.

Diabetic retinopathy is a leading cause of preventable blindness.⁸ Limited specialist access in low-resource regions motivates automated screening programs, but detecting the disease requires identifying subtle pathological features. Figure 3.4 contrasts a healthy retina with one exhibiting diabetic retinopathy, where identifying localized microvascular hemorrhages demands fine spatial resolution and robust input validation.



Figure 3.4: Retinal Hemorrhages in DR Screening: Side-by-side fundus images illustrate the subtle visual features an ML screening system must detect. Identifying localized microvascular lesions (such as the indicated hemorrhage) requires preserving fine spatial resolution through the vision pipeline and establishing strict input-quality validation to handle lighting and camera variations in clinical field deployments. Source: Google.

Initial research achieved expert-level performance in controlled settings. However, translating laboratory models into clinical deployment exposed severe operational friction: sensor lighting variations, intermittent network connectivity in rural clinics, regulatory compliance, and clinic workflow integration.⁹ For the metrics that recur in this case, sensitivity is the true-positive rate, specificity is the true-negative rate, and AUC is the area under the receiver operating characteristic (ROC) curve. The same constraint propagation dynamics apply whether deploying medical diagnostics to rural health posts or computer vision to commodity smartphones: hardware limits, operator variance, and data quality govern real-world viability far more than benchmark AUC.

3.2.2 Stage interface specification

Each lifecycle stage operates as a distinct engineering phase with defined inputs, outputs, and quality invariants. In production workflow engines, these contracts are formalized across a workflow directed acyclic graph (DAG), where nodes represent discrete execution tasks and directed edges represent validated artifact dependencies. While the overarching ML lifecycle forms a cyclic feedback loop over weeks and months, each automated pipeline run executes as an acyclic graph of dependent steps, enabling orchestrators to parallelize independent branches, cache validated intermediate outputs, and halt execution before broken contracts propagate downstream. Just as microservices rely on interface specifications to establish integration boundaries, ML pipeline stages require schema and distribution contracts to detect incompatibilities before invalid data or degraded weights propagate downstream. Table 3.3 formalizes these contracts, defining the input requirements, deliverables, and quality invariants required across all six lifecycle stages.

⁸ | **Diabetic retinopathy (DR):** DR is a common complication of diabetes; early detection can prevent vision loss, but specialist access varies substantially across countries. This gap motivates scalable screening programs. In clinics with limited hardware or unreliable connectivity, model size, offline capability, latency, and workflow integration can become binding deployment constraints.

⁹ | **Healthcare AI deployment gap:** In the Thai DR deployment studied by Beede et al. (2020), real clinics exposed workflow, image-quality, infrastructure, and human-factors constraints that were absent from controlled evaluation. The case shows why laboratory metrics alone do not establish deployment readiness.

Table 3.3: Stage Interface Specification: Each lifecycle stage has explicit input requirements, output deliverables, and quality invariants that must hold for the stage to be considered complete. Violations of these contracts create technical debt that compounds through subsequent stages. The deployment paradigm selection in Problem Definition (Cloud, Edge, Mobile, or TinyML from chapter 2) constrains all downstream stages, as a TinyML target imposes different data, model, and monitoring requirements than a Cloud target.

Stage	Input Contract	Output Contract	Quality Invariant
Problem Definition	Business requirements; operational context	Measurable objectives; deployment paradigm selection; resource constraints	All success criteria are quantifiable; target deployment paradigm is explicit
Data Collection & Preparation	Objectives; deployment target; quality requirements	Versioned dataset with schema; preprocessing pipeline; data validation rules	Distribution approximates anticipated production environment; labeling meets accuracy requirements
Model Development & Training	Dataset; accuracy targets; resource constraints	Trained model weights; training configuration; experiment logs	Meets accuracy thresholds within computational budget; architecture compatible with deployment target
Evaluation & Validation	Trained model; held-out test data; evaluation criteria	Performance metrics across subgroups; failure mode analysis; validation certificate	No critical subgroup falls below minimum thresholds; confidence calibration meets domain requirements
Deployment & Integration	Validated model; infrastructure requirements; service-level agreement (SLA) targets	Serving endpoint; monitoring instrumentation; rollback procedures	Latency and throughput meet paradigm requirements; integration tests pass
Monitoring & Maintenance	Live system; performance baselines; alert threshold	Drift detection alerts; update-review criteria; incident reports	Alert coverage and detection windows are defined; detected degradation triggers review

This specification reveals why ML projects experience the iteration cycles diagrammed in figure 3.3. When a downstream stage discovers that an upstream contract was violated (for example, evaluation reveals the training data distribution does not match production), the project must iterate back to fix the root cause. Teams that validate contracts at each stage transition catch violations early, when correction costs are lowest. Validating stage contracts before advancing prevents defects from compounding into late-stage redesigns, as illustrated in the transition audit below.



Example 3.1: Auditing stage transitions

Scenario: An engineering team claims to have completed Problem Definition for a medical imaging classifier, attempting to transition to Data Collection without establishing deployment target constraints.

Diagnosis: The output contract lacks deployment paradigm selection and resource constraints (latency and memory targets). Attempting to deploy late in the lifecycle exposes violations (e.g., edge device memory limit < 200 MB), incurring 16× the cost compared to early correction.

Systems lesson: Stage transitions act as control-plane quality gates in the ML lifecycle. Enforcing strict output contract validation before proceeding downstream prevents expensive upstream rework and ensures data collection aligns with target deployment hardware.

Without explicit contract gates, unstated assumptions propagate downstream. The earliest opportunity to bound them is Problem Definition, which records the operational constraints that subsequent stages must satisfy.

3.3 Problem Definition

Problem definition in ML begins with sentences that look deceptively simple. A product manager writes: “Build a model that detects diabetic retinopathy.” That single sentence conceals a dozen engineering decisions: sensitivity thresholds for patient safety, hardware capabilities in rural clinics, latency budgets that keep clinicians engaged, and regulatory frameworks governing approval. In traditional software, requirements translate into deterministic control flow, explicit state machines,

and relational schemas. In ML systems, defining *what* the system should do is inseparable from defining *how* it will learn from data and what physical constraints its execution must survive.

Balancing these demands requires reasoning across the full system stack. An engineering team cannot optimize accuracy in isolation: pursuing marginal sensitivity gains by scaling model depth or switching to vision transformers inflates parameter count and arithmetic intensity, directly challenging the memory capacity and thermal limits of low-cost clinic hardware. Conversely, pruning a model to fit an embedded accelerator’s DRAM budget risks dropping sensitivity below clinical safety thresholds. Every technical decision operates within a multi-constraint optimization problem where statistical goals, physical execution budgets, and operational mandates continually bound each other’s feasible design space.

3.3.1 Constraint layers

Every ML problem definition decomposes into three interacting **constraint layers**: statistical, physical, and operational. Rather than isolated requirements, these layers form a coupled stack where constraints in one tier immediately restrict the degrees of freedom in the others. The statistical layer specifies target behavior over data distributions—such as achieving > 90 percent sensitivity and > 80 percent specificity across diverse patient demographics and camera models, rather than an unweighted aggregate accuracy that conceals failure modes on minority subpopulations. The physical layer bounds execution within hardware limits: for rural clinics, inferencing on local edge accelerators under tight thermal budgets, bounded memory capacity (such as 4 to 8 GB of shared DRAM), and sub-second latency budgets to keep clinical workflows interactive without depending on intermittent network uplinks. The operational layer encompasses clinical auditability, regulatory clearance, and governance frameworks that dictate long-term system maintenance.

The operational layer carries physical consequences that traditional software engineering rarely encounters. In a traditional database, deleting a patient’s record is a DELETE statement. In an ML system, model weights may encode statistical patterns learned from that record: where law or a regulator requires removing that influence, compliance may require machine unlearning (an active research area with incomplete guarantees) or retraining on the remaining data. The obligation and remedy depend on the governing rule and system; at DR-system scale, either can make privacy compliance a recurring compute-budget item and training provenance an architectural constraint.

The constraint propagation principle (section 3.9.1) explains why all three layers must be bound before development begins: a constraint that exists in the physical or operational environment but remains unstated in the problem definition does not vanish—it re-emerges later as an architectural violation after dependent decisions have hardened. For example, if an engineering team defines the objective solely as aggregate classification accuracy on retinal images, standard optimization will minimize cross-entropy loss across the majority class. In an imbalanced screening population where 90 percent of patients are healthy, a model predicting “no disease” achieves 90 percent aggregate accuracy while failing every patient who needs urgent care. Preventing this requires translating domain invariants—the asymmetric cost of a missed diagnosis—into an explicit statistical contract that binds the loss formulation to the deployment hardware budget.

📌 War Story 3.1: When the label was the bias (2018)

Context: In 2014, Amazon assembled an engineering team at its Edinburgh hub to build an internal machine-learning system for ranking technical job applicants, training it on ten years of resumes submitted to Amazon (Dastin 2018).

Mechanism: Because prior hiring was male-dominated, the model learned that male-coded language correlated with hiring success, systematically penalizing terms like “women’s” and downgrading graduates of all-women’s colleges.

Impact: The model reproduced historical hiring bias, rendering automated resume screening unsuitable for production recruitment.

Fix: Amazon abandoned the experimental tool after attempts to remove explicit gender signals could not guarantee that it would not learn other discriminatory proxies.

Systems lesson: Problem definition must specify fairness, auditability, and rejection criteria before data collection and training. A workflow trained on biased labels does not learn the operational goal; it learns to reproduce the labels.

3.3.2 Problem definitions evolve

A problem definition is not a static specification frozen at project inception; it evolves as deployment expands. When a DR screening program scales from a pilot clinic with a calibrated fundus camera to regional health centers with heterogeneous imaging hardware and differing patient demographics,¹⁰ the initial assumptions break. Sensor noise profiles diverge, optical resolutions shift, and demographic variations alter subgroup error rates. Adapting to this operational reality forces the problem definition to incorporate stratified accuracy targets and broader hardware compatibility matrices.

This dynamic evolution stems directly from the statistical nature of ML. Scaling exposes long-tail inputs absent from pilot evaluations, while production environments introduce covariate shift that no static offline dataset fully captures. A robust problem definition anticipates this drift by establishing governance protocols upfront: defining which monitoring signals trigger formal re-evaluation and bounding how threshold updates propagate through subsequent pipeline stages. Once these operational constraints and performance boundaries are defined on paper, the workflow must ground them in physical artifacts—beginning with the data collection pipelines that gather the training distribution.

3.4 Data Collection

Constraints defined on paper only take effect when grounded in actual data. Moving from problem definition to data collection connects abstract accuracy targets with operational reality: sensor noise, missing records, and network limits. In iron law terms, this stage determines dataset size (D) and the byte volume (D_{vol}) that storage systems, networks, and memory buses must move. The deployment constraints established during problem definition become technical data requirements: if the model must run on edge devices, the collection pipeline must produce inputs compatible with edge preprocessing; if it must achieve 90 percent sensitivity across diverse populations, the dataset must sample those subpopulations sufficiently to support model training.

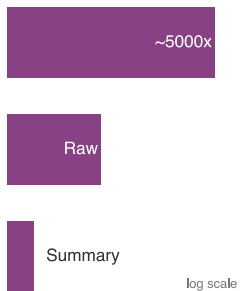
Data collection and preparation is an ongoing engineering discipline rather than a one-time task (chapter 4). For DR screening, the pipeline must ingest images statistically diverse enough to generalize across clinics, robust enough to handle varying capture conditions, and annotated with enough clinical rigor to withstand regulatory audits.

Problem definition decisions shape data requirements in the DR example. The multi-dimensional success criteria established (accuracy across diverse populations, hardware efficiency, and regulatory compliance) demand a data collection strategy that goes beyond typical computer vision datasets. Not all data contributes equally to learning, either—chapter 9 shows how selection can retain comparable performance with less compute when a dataset contains redundancy, provided the selected subset preserves the evidence the task requires.

The DR system requires on the order of 10^5 retinal fundus photographs, each reviewed by multiple expert ophthalmologists. Expert consensus addresses the inherent subjectivity in medical diagnosis (two ophthalmologists may disagree on borderline cases) while establishing ground truth labels that can withstand regulatory scrutiny. The annotation process must capture clinically relevant features like microaneurysms, hemorrhages, and hard exudates across the full spectrum of disease severity.

Raw high-resolution retinal scans can reach tens of megabytes per image before compression, creating substantial infrastructure challenges. A clinic processing dozens of patients per day can produce gigabytes to tens of gigabytes of imaging data per week. This data volume can occupy much of a limited uplink during clinic hours, making local processing one option alongside compression, batching, scheduling, or increased link capacity.

¹⁰ | **Demographic drift:** Population changes can expose performance gaps that aggregate evaluation obscures. Facial-analysis benchmarks found large demographic error-rate disparities (Buolamwini and Gebru 2018), while a health-care risk-scoring study found racially biased allocation from a proxy label despite similar need (Obermeyer et al. 2019). These examples illustrate why a growing DR program should evaluate subgroup performance rather than claiming that such a rollout occurred.



Edge summaries beat raw medical-image uploads when bandwidth dominates.

Napkin Math 3.2: Bandwidth vs. compute

Problem: A rural clinic captures retinal images for DR screening. How much of the assumed same-shift uplink window would raw uploads occupy, and what alternatives reduce that occupancy?

Math:

1. Daily data: At 150 patients/day, 10 photos/patient, and 5 MB/photo, the clinic produces 7.5 GB/day.
2. Upload time: The uplink is 2 Mb/s, or 0.25 MB/s. Dividing 7,500 MB by that rate gives 30,000 s, approximately 8.3 h.
3. **Constraint:** If the clinic requires same-shift upload during its 8 h operating window, this transfer alone occupies the link for 104.2 percent of that window and may contend with other traffic.

Systems insight: Local inference with uploaded detection summaries (10 KB/patient) reduces bandwidth usage by 5,000×. Overnight batching, compression, scheduling, or a faster link are alternatives not compared by this calculation.

3.4.1 Lab-to-field data gap

Laboratory data and production data inhabit different worlds. This **lab-to-field gap** appears when DR screening deploys to rural clinics across Thailand and India: images arrive from diverse camera equipment operated by staff with varying expertise, often under suboptimal lighting with inconsistent patient positioning. A model trained on high-quality research images from standardized fundus cameras may fail on blurry, poorly-lit images from older equipment—not because the algorithm is wrong, but because the data distribution has shifted beyond the training envelope.

Because raw image uploads can exceed clinic uplink capacity, edge deployment offers an alternative. Deploying edge accelerators such as the NVIDIA Jetson family¹¹ allows clinics to preprocess and run inference locally. This reduces network transmission by orders of magnitude, but introduces a hardware trade-off: use a smaller model that fits a low-power edge budget, or provision more capable on-premise devices at higher cost.

The bandwidth constraint makes infrastructure a data-collection decision rather than a late implementation detail. The scenario architecture combines edge devices for local inference and preprocessing, clinic aggregation servers for data management and buffering, and cloud training infrastructure for periodic model updates. It targets end-to-end latency under 100 milliseconds and operation without connectivity-induced delays.

Privacy constraints impose a similar architectural decision. Patient privacy regulations may motivate local retention or distributed training, but they generally specify protections and permitted uses rather than one required training topology. Workflows that keep raw clinic data local while sharing only approved updates or summaries add complexity to both data collection and model training infrastructure.

3.4.2 Distributed data infrastructure

If a deployment grows from a handful of clinics to hundreds, data infrastructure must scale accordingly. Each retinal image travels through multiple stages: clinic cameras capture the image, local systems provide initial storage and processing, quality validation checks ensure usability, secure transmission moves data to central systems, and finally, integration with training datasets completes the pipeline. The infrastructure decisions at each stage are shaped by the deployment constraints established during problem definition.

Storage tiers are another place where the data pipeline either preserves or erodes downstream iteration velocity. Different data access patterns demand different storage solutions, so teams typically implement **tiered storage architectures**,¹² each calibrated to access frequency and performance requirements.

Hot storage uses high-throughput NVMe SSDs for data currently used in training loops. Warm storage uses S3-compatible object storage for recent datasets and active validation sets. Cold storage

¹¹ | **NVIDIA Jetson:** NVIDIA's Jetson family spans a wide SKU spectrum, from Jetson Orin Nano (7–15 W) through Jetson Orin NX (10–25 W) to Jetson AGX Orin (15–60 W). This scenario uses an Orin Nano-class device (4–8 GB shared LPDDR5, 7–15 W power envelope) as a concrete edge target. Its memory and power budgets constrain model complexity, while selecting a larger device changes both the feasible model and the per-clinic cost.

¹² | **Tiered storage:** Places data on different storage media based on access frequency and performance requirements. The storage price gap is roughly 4.3× in this example: Non-Volatile Memory Express (NVMe) SSDs deliver 500,000+ input/output operations per second (IOPS) at ~\$0.10/GB/month, while object storage costs ~\$0.023/GB/month but with 100–200 ms latency. For ML training loops requiring sustained sequential reads at 1–10 GB/s, choosing the wrong tier converts a compute-bound training pipeline into an I/O-bound one, directly inflating the iron law's data term (D_{vol}/BW).

uses low-cost archival systems, such as AWS Glacier, for historical data required for regulatory audit trails but rarely accessed.

In practice, the boundary between tiers is dynamic: a dataset migrates from warm to hot when selected for the next training run, and from hot to cold when the model it trained is superseded. Automated lifecycle policies manage these transitions, promoting data based on training schedules and demoting it based on access recency—a pattern that chapter 4 explores in detail.

Rural clinic deployments face severe connectivity constraints that force a choice between transmission strategies. Clinics with reliable broadband can stream images in near-real-time for centralized processing, but clinics with intermittent satellite links, common in remote regions of India and sub-Saharan Africa, require **store-and-forward architectures** that batch images during connectivity windows and reconcile results asynchronously. The choice propagates through the entire stack: store-and-forward clinics need larger local storage buffers, more robust local inference capabilities, and conflict-resolution logic when locally generated predictions differ from later cloud-based analysis.

Infrastructure scalability poses a harder challenge than raw capacity. As the system grows from a handful of pilot clinics to hundreds of production sites, data heterogeneity grows alongside data volume: each clinic's camera model, lighting environment, and operator habits can produce a different image distribution. The infrastructure must handle increasing throughput while also tracking which data came from where. This provenance metadata proves essential for debugging accuracy regressions at specific sites and for satisfying the audit trail requirements that regulatory validation demands. Scaling from initial clinics to a broader network therefore introduces emergent complexity: variability in equipment, workflows, operating conditions, and image sizes as newer clinics add higher-resolution devices. Each clinic becomes a distinct data source,¹³ yet the system must evaluate performance across the full network.

The workflow response is coordination infrastructure. Shared artifact repositories, versioned APIs, and automated testing pipelines make clinic-specific variation visible before it becomes a model failure. That same heterogeneity is what makes point-of-capture validation necessary: the larger and more varied the clinic network becomes, the less useful it is to discover quality failures weeks later in a centralized training run.

3.4.3 Quality assurance and validation

A blurry retinal image that slips past quality checks does not merely waste storage. If such images enter training with unreliable labels or appear frequently in production, they can distort the evidence available to the model and increase error risk. Quality assurance ensures that data meets the requirements downstream stages depend on. In our DR example, automated checks at the point of collection flag issues like poor focus or incorrect framing, allowing clinic staff to recapture images immediately rather than discovering the problem weeks later during model training.

Validation extends beyond image quality to verify proper labeling, patient association, and privacy compliance. Local validation catches problems at the point of capture; centralized validation detects distributional anomalies across the full clinic network—for instance, flagging when a particular site's images skew toward a narrow demographic range that would bias the training set.

Data collection decisions directly constrain model development: bandwidth limits dictate what architectures are feasible, privacy requirements shape training pipelines, and quality variations across clinic environments determine robustness requirements. Figure 3.5 traces these feedback pathways concretely. Follow each labeled arrow: evaluation reveals the DR model underperforms on images from older fundus cameras, triggering targeted data collection from clinics using that equipment. Validation across diverse patient populations shows lower sensitivity for patients with cataracts, driving data augmentation strategies that simulate lens opacities. Monitoring detects accuracy drift in clinics that upgraded their imaging equipment, feeding back to update preprocessing steps.

These feedback pathways reinforce a central point: data collection does not end when training begins. The quality, volume, and diversity of the data flowing through these pipelines now become the raw material for the next stage—turning curated datasets into trained models.

¹³ | **Distributed clinic data:**

Training across clinic sites without simply pooling all raw data addresses privacy and governance constraints, but it shifts cost into coordination. Each site may use different cameras, serve different patient populations, and follow different operating routines, so the system must track where data came from and how each site differs. The workflow cost is therefore not just storage capacity; it is the engineering work needed to compare, validate, and update models across sites whose data does not behave identically.

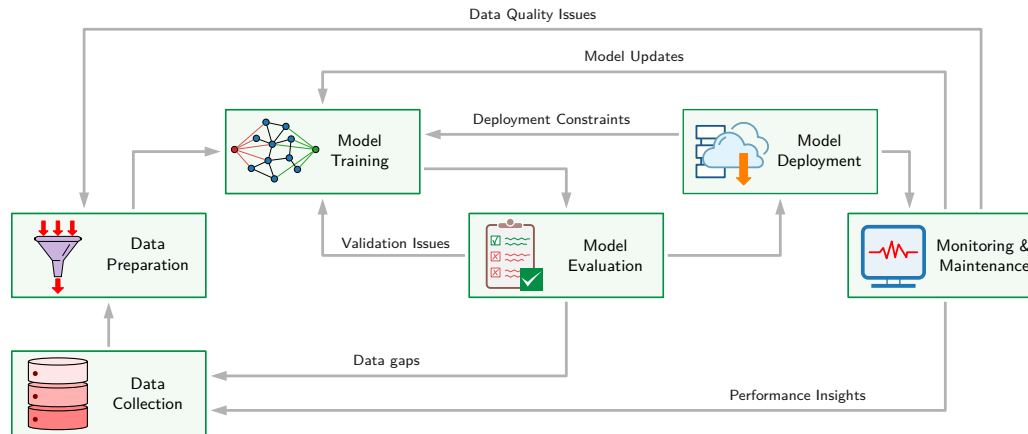


Figure 3.5: Feedback Paths Across Lifecycle Stages: Multi-stage feedback pathways show how downstream discoveries can require upstream corrections. Data-quality and distribution gaps detected during evaluation may prompt targeted collection or augmentation, while operational constraints and performance drift can motivate model, data, or pipeline changes after investigation.

3.5 Model Development

The DR team has 128,000 labeled retinal images, a validated preprocessing pipeline, and a target: more than 90 percent sensitivity on edge hardware with less than 50 ms inference latency. The question is no longer *what data* to collect but *what model* to build—and that question has no answer independent of the deployment constraints already established. In iron law terms, this stage defines the Operations (*O*) term: architectural choices set the computational floor that hardware must sustain. Training systems (chapter 8) must coordinate data ingestion, distributed parameter synchronization, and hyperparameter exploration¹⁴ without stalling execution pipelines. In high-stakes domains like healthcare, every design decision affects clinical outcomes, so technical performance and operational constraints must be integrated from the start.

The DR system faces a sharp training challenge: achieve expert-level diagnostic accuracy with finite labeled-data and optimization budgets. **Transfer learning**¹⁵ addresses this constraint by adapting a model pretrained on a source task to a target task. The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) 2012 training subset contains 1.3 million labeled images (Russakovsky et al. 2015). Reusing learned representations can improve target-task optimization and generalization relative to random initialization, with the benefit depending on source-target similarity, which layers are transferred, and transfer-related optimization effects (Yosinski et al. 2014).

In the controlled validation study reported by Gulshan et al. (2016), transfer learning and a labeled dataset of 128,000 images produced an AUC¹⁶ of 0.99, with sensitivity of 97.5 percent and specificity of 93.4 percent at one operating point. The result demonstrates the potential of large-scale pretraining with domain-specific fine-tuning under the study conditions; it does not by itself establish deployment performance. Backpropagation translates classification loss into weight adjustments across millions of parameters; the underlying neural primitives (chapter 5) ultimately dictate whether training scales efficiently on specialized hardware (chapter 8).

Achieving high accuracy is only the first challenge. Edge deployment constraints impose strict efficiency requirements: models may need to fit within tens to hundreds of megabytes, complete inference in tens of milliseconds, and operate within tight memory budgets.

From a workflow perspective, accuracy gains must be weighed against deployment constraints. **Ensemble learning**¹⁷ illustrates this tension: combining predictions from multiple models often improves accuracy, but multiplies inference latency and memory requirements. Whether diversity comes from training on data subsets (**bagging**), sequentially correcting errors (**boosting**), or combining predictions with a meta-model (**stacking**), each additional model adds serving overhead that must fit the deployment target. The Netflix Prize showed how ensemble complexity can block deployment: the winning solution delivered high accuracy, but Netflix did not deploy it because the operational costs exceeded the business benefit (Johnston 2012).¹⁸

¹⁴ | **Hyperparameter:** Architectural and optimizer choices (for example, learning rate and network depth) affect the computational work of training. A naive Cartesian grid that trains every combination independently incurs a multiplicative search cost: 5 hyperparameters with 4 values each produces 1,024 (4^5) configurations. Early stopping, multifidelity methods, and adaptive search can avoid completing every run, which is why the grid represents a baseline rather than an unavoidable cost.

¹⁵ | **Transfer learning:** For the same architecture, transfer learning changes how parameters are initialized and optimized, not the operations required at inference.

16 | AUC (area under the ROC curve): Measures the area under the receiver operating characteristic (ROC) curve plotting true positive rate vs. false positive rate across all classification thresholds, ranging from 0 to 1; 0.5 represents random discrimination, and values below 0.5 are possible. Unlike accuracy, AUC is threshold-independent and insensitive to class prevalence, making it a common metric for medical screening systems. The systems consequence: a model with 0.99 AUC can still produce unacceptable sensitivity at the specific operating threshold chosen for deployment, so AUC alone cannot validate deployment readiness.

17 | Ensemble learning: Combines predictions from multiple models (bagging, boosting, stacking). Inference compute and model storage generally sum across constituents. Serial execution can increase latency, while parallel execution trades latency for additional hardware and coordination; no fixed proportional latency multiplier follows from ensemble size.

18 | Competition-production gap: The Netflix Prize winner used a complex ensemble, but Netflix did not deploy the winning approach because the additional accuracy did not justify the engineering effort (Johnston 2012).

19 | Medical AI performance metrics: Medical AI evaluation emphasizes sensitivity (true positive rate) and specificity (true negative rate) alongside aggregate accuracy. This DR scenario sets a >90 percent sensitivity target because missed referable disease can delay care and contribute to avoidable vision loss; actual acceptance criteria are product- and regulator-specific. The subtler systems trap is positive predictive value (PPV): even a high-accuracy model can have low PPV in a low-prevalence population. This prevalence dependence can require different operating thresholds across deployment sites, a constraint invisible in standard ML evaluation.

Research models often span multiple gigabytes when using ensembles, exceeding the memory limits of edge devices. Rather than tuning accuracy in isolation, teams must apply systematic model compression. Techniques such as quantization and pruning (chapter 10) reduce model size and memory traffic, but each compression step must be validated to ensure it does not drop clinical sensitivity below required thresholds. Model development is an ongoing balance between statistical performance and physical execution cost.

3.5.1 Reproducible system artifacts

The accuracy-efficiency balancing act produces more than trained weights alone. A common failure mode is treating the trained model weights as the sole output of this stage. In a mature ML workflow, the deliverable is a **reproducible system artifact** with four components:

- **Model Weights:** The learned parameters.
- **Inference Code:** The exact code used to run the model, including preprocessing logic.
- **Environment Specification:** The complete dependency graph (for example, Docker container, `requirements.txt`, CUDA drivers) required to execute the code.
- **Configuration:** Hyperparameters and runtime settings.

Without bundling the environment with the model, dependency mismatches can create dangerous failures. Some incompatibilities, including unsupported CUDA combinations or missing libraries, fail loudly and immediately. Other differences in compatible kernels, linear algebra libraries, or image-resizing routines such as OpenCV and PIL may alter floating-point results or pixel interpolation without crashing the serving process. These changes can shift model outputs and affect accuracy. Environment reproducibility is therefore necessary not only for successful execution but also for validating equivalent inference behavior. A model that achieves 99 percent accuracy in development must be reevaluated if its production environment changes, even when inference completes without an exception.

3.5.2 Accuracy vs. efficiency

Medical applications demand specific performance metrics¹⁹ that differ from the standard classification outputs and losses chapter 5 introduces. A DR system requires high sensitivity (to limit missed referable disease) and high specificity (to avoid overwhelming referral systems). These metrics must be maintained across diverse patient populations and image quality conditions.

Optimizing for clinical performance alone is not enough. In this scenario, the model artifact must remain below 500 MB so that it fits comfortably within the edge target's 4–8 GB of shared system memory, while the device operates inside a sub-20 watt power envelope and meets the clinical latency budget. Improvements in one dimension often come at the cost of others: the Operations (O) term, the model byte footprint that contributes to D_{vol} , and fixed serving overhead (L_{lat}) can pull in different directions. Chapter 6 explores model capacity, while chapter 2 discusses deployment feasibility, and the inherent tension between them drives architectural decisions. Systematic compression and revalidation²⁰ can bridge the gap, meeting deployment requirements while aiming to preserve clinical utility.

The ensemble trade-off illustrates a broader pattern: choosing an ensemble of lightweight models over a single large model reduces per-model complexity (enabling edge deployment) but increases pipeline complexity (requiring orchestration logic and multi-model monitoring). Every architectural decision creates this kind of downstream ripple.

3.5.3 Constraint-driven development

Real-world constraints shape model development from initial exploration through final optimization, demanding systematic experimentation. Development begins when data scientists collaborate with domain experts (ophthalmologists in the DR case) to identify subtle lesions and image-quality requirements that matter clinically. Without that domain knowledge, a model architect might choose a resolution or receptive field, the input region each internal feature can see, that discards relevant detail before the network can use it. This interdisciplinary approach helps model architectures preserve clinically relevant evidence while respecting the computational constraints identified during data collection.

Computational constraints profoundly shape experimental approaches. Multiple model variants, hyperparameter sweeps, and preprocessing approaches can make exhaustive experimentation expensive. This economic reality drives investments in better job scheduling, caching of intermediate results, early stopping, and automated resource optimization. Systematic hyperparameter optimization and disciplined experiment design can reduce computation relative to exhaustive search.

The inherent uncertainty of ML outcomes demands scientific methodology: controlled variables through fixed random seeds and environment versions, systematic **ablation studies**²¹ to isolate component contributions, confounding factor analysis to separate architecture effects from optimization effects, and statistical significance testing across multiple training runs using paired offline tests rather than the A/B testing²² reserved for comparing models on live production traffic. Without this rigor, teams cannot distinguish genuine performance improvements from statistical noise—a distinction that becomes critical when a 0.5 percent accuracy difference determines whether a model meets the clinical sensitivity threshold.

At every development milestone, teams validate models against the deployment constraints identified in earlier lifecycle stages. Each architectural innovation must be evaluated for accuracy improvements and compatibility with edge device limitations and clinical workflow requirements. This dual validation approach ensures that development efforts align with deployment goals rather than optimizing for laboratory conditions that do not translate to real-world performance.

3.5.4 Prototype to production

A team of three data scientists may manage experiments with spreadsheets and shared notebooks, while a team of 30 is more likely to need shared workflow infrastructure. As projects evolve from prototype to production, complexity grows across multiple dimensions simultaneously: larger datasets, more sophisticated models, concurrent experiments, and distributed training infrastructure. Informal coordination can become a bottleneck at production scale as teams share data splits, resolve experiment conflicts, and reconcile notebook versions. Figure 3.6 illustrates one assumed contrast between manual coordination and a shared workflow platform. The axes are relative units intended to show shape, not absolute throughput.

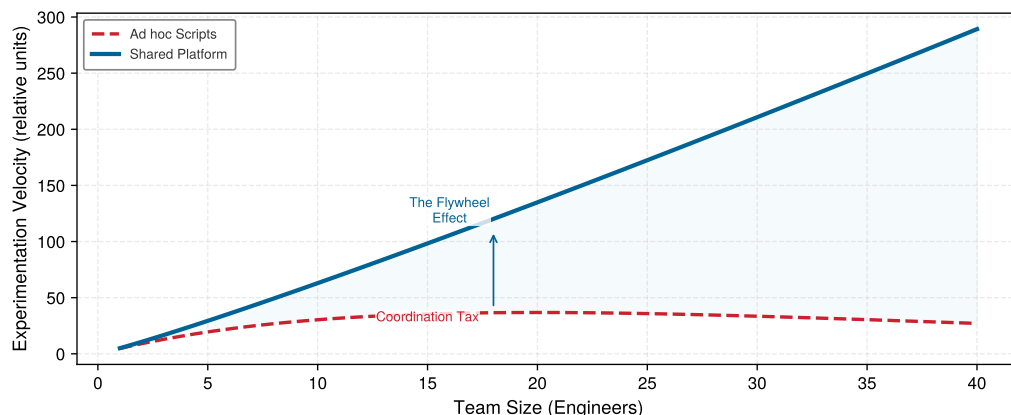


Figure 3.6: Illustrative Workflow Automation Dynamics: Conceptual scaling curves contrast the experimentation velocity of manual workflows against shared workflow platforms as team size grows. Unstructured ad hoc scripts eventually hit a “coordination tax” from merge conflicts and unversioned artifacts, whereas shared platforms can reduce duplicated work through reusable components and automated lineage. The curves illustrate assumptions rather than empirical scaling laws.

The curves in figure 3.6 encode assumed functions rather than measured team-scaling laws. The manual-workflow curve represents coordination costs from sharing data splits, resolving experiment conflicts, and reconciling notebook versions. The platform curve represents benefits from reusable preprocessing components, versioned experiment tracking, and automated scheduling. Actual throughput need not saturate or grow super-linearly; it depends on workload parallelism, platform overhead, team structure, and experiment quality. Figure 3.6 therefore illustrates why shared

20 | Model compression pipeline: Bridging the gap between research accuracy and edge deployment requires an iterative “compress-validate-adjust” loop. Each compression step can reduce model size or execution cost, but it can also silently degrade sensitivity below the clinical threshold. Finding a model that fits in device memory while preserving clinical sensitivity typically requires multiple iterations because only the full validation suite reveals whether the smaller model still satisfies the problem definition.

21 | Ablation studies: Named for surgical tissue removal, ablation studies systematically disable individual components to isolate their contribution to performance. The rigor matters because a 0.5 percent accuracy difference can determine whether the DR model meets its clinical sensitivity threshold. Without ablation, a team cannot distinguish a genuine architectural improvement from noise introduced by a different random seed, wasting iteration cycles on phantom gains.

22 | A/B testing in ML: Compares a new model (B) against the production baseline (A) on randomly assigned live traffic to estimate the model’s causal effect. Required sample size depends on the baseline event rate, effect size, variance, allocation, significance level, and statistical power; no universal interaction count or duration follows from a 0.5 percentage-point change alone.

infrastructure can reduce duplicated work and manual handoffs without predicting a universal return for every team.

3.5.5 Reproducibility and technical debt

Shared infrastructure can accelerate experimentation—but rapid iteration creates a hidden liability. If experiments are not reproducible, the team cannot reliably distinguish genuine improvements from noise, and the codebase accumulates technical debt²³ that compounds with every unreproducible result.

Reproducing ML results requires tracking data versions, random seeds, hardware configurations, and library versions alongside program logic. Hardware and nondeterministic operations can alter training results, so teams need lineage to distinguish architectural changes from run-to-run variation. This run-to-run variation arises physically because parallel GPU threads execute with nondeterministic scheduling: operations like backward-pass gradient accumulations sum values in arbitrary arrival order, and floating-point addition is non-associative ($(a + b) + c \neq a + (b + c)$). Furthermore, optimized vendor libraries (such as cuDNN) dynamically benchmark and select different execution kernels across hardware runs unless deterministic execution is explicitly locked down, causing identical code and random seeds to produce diverging weights. Systematic **experiment tracking** records unique run identifiers and artifact versions. Systems such as MLflow and Weights & Biases link data versions, code commits, hyperparameters, and resulting models so teams can reconstruct differences between runs.

The cost of neglecting **reproducibility** is economic, not just scientific. Teams that cannot reproduce a result waste cycles rerunning experiments that may not converge to the same outcome. Reproducibility infrastructure such as versioned environments, controlled pipelines, and automated checkpointing reduces redundant computation and supports more confident architectural decisions.

Reproducible, optimized models are necessary but not sufficient. A model that achieves expert-level accuracy on curated research data may still fail in production. The next stage subjects these trained artifacts to systematic testing against the conditions they will actually encounter.

3.6 Evaluation and Validation

A diabetic retinopathy (DR) screening model achieves an area under the ROC curve (AUC) of 0.99 on a curated research dataset. When evaluated on field images from a clinic in Chiang Mai—where a technician with two weeks of training operates a five-year-old fundus camera—sensitivity drops to 78 percent. The software executes without throwing an exception or returning an error code; the convolutional filters simply encounter pixel distributions distorted by optical blur, low illumination, and variable pupil dilation that never appeared in the training distribution. High accuracy on curated test partitions does not guarantee operational reliability in production. Before deployment, trained models must pass through systematic evaluation and validation gates to verify that they satisfy accuracy, latency, and robustness requirements across realistic deployment conditions. This stage transforms raw checkpoint files into production-ready software systems by measuring behavior against predefined operational thresholds.

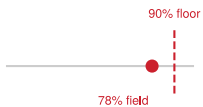


Definition 3.2: Model validation

Model validation is an evidence gate for deciding whether a trained model is suitable to deploy for a specified use. It tests the model against deployment constraints such as latency targets, subgroup performance targets, cost budgets, and robustness under distribution shift, but cannot certify safety under every possible condition.

1. **Significance:** Validation adds dimensions that test-set accuracy ignores. A model achieving 95 percent accuracy on a static test set may miss a 100 ms latency target on its slowest requests, underperform a required subgroup threshold such as a maximum five percentage-point performance gap across demographic groups, or lose more than ten percentage points of accuracy under common image-quality problems such as blur and glare. Each unchecked dimension is a deployment risk that compounds silently in production.

²³ | **ML artifact interdependence:** ML artifacts are deeply interdependent. A model's measured accuracy is tied to the data version, preprocessing pipeline, and hyperparameter configuration used to produce it. Without lineage tracking, the team may not be able to determine whether a regression stems from code, data, or run-to-run variation, forcing expensive reruns of experiments whose provenance is lost.



Production validation is a gate: field sensitivity below the required floor fails deployment.

2. **Distinction:** Evaluation measures model behavior using chosen datasets, metrics, and experiments, which may include out-of-distribution and subgroup tests. Validation assembles the broader evidence needed for a specified deployment decision, including system latency, throughput, robustness, operational readiness, and cost.
3. **Common pitfall:** A frequent misconception is that validation is “one more test.” In reality, it is a multi-dimensional gate: a model can pass the accuracy test and still fail validation because it violates latency, subgroup reliability, or cost constraints that accuracy alone does not measure.

While evaluation quantifies isolated algorithmic properties on static test sets, validation assesses whether the integrated software and hardware pipeline satisfies operational invariants under live serving constraints. Because no finite test suite can certify correctness across all possible input variations, validation operates as a risk-management discipline structured across four stages: metric thresholding, staged production rollouts, environmental stress testing, and regulatory qualification.

3.6.1 Evaluation metrics and thresholds

Evaluation metrics must map directly to the asymmetric loss functions of the target application. In diabetic retinopathy screening, standard classification accuracy is misleading due to class imbalance: a trivial classifier predicting no disease for every sample would achieve high accuracy in a low-prevalence screening population while missing every treatable pathology. System requirements therefore mandate decoupled operating thresholds: sensitivity above 90 percent ensures the clinic captures actionable pathology, while specificity above 80 percent prevents false-positive referrals from saturating secondary ophthalmology clinics. Determining the classification threshold that satisfies both constraints simultaneously is an empirical optimization problem over the receiver operating characteristic (ROC) curve.

Aggregate metrics can conceal localized failure modes. Stratified evaluation partitions the test corpus across metadata dimensions—such as patient age brackets, chronic comorbidities, and clinic camera models—to measure performance parity across slices. A vision backbone achieving 94 percent overall accuracy across an entire benchmark may fall below 80 percent on images captured from patients with cataracts or under low ambient lighting. These localized performance drops remain invisible in aggregate evaluations but manifest as severe safety failures once deployed. Systematic benchmarking methodologies (chapter 12) evaluate these sliced distributions automatically before promoting a candidate checkpoint.

In addition to label accuracy, evaluation must measure probability **calibration**:²⁴ whether a model’s predicted confidence score aligns with its empirical error rate. Modern deep neural networks frequently produce uncalibrated posteriors, outputting overconfident predictions (> 90 percent confidence) even on samples near decision boundaries. When automated triage pipelines route borderline cases to human specialists based on confidence cutoffs, miscalibration misallocates clinical attention and undermines practitioner trust.

3.6.2 Offline and online evaluation

The validation gate begins offline on static datasets and progresses incrementally toward live production traffic. Offline evaluation on held-out test partitions establishes baseline convergence, but it cannot expose physical systems bottlenecks such as memory fragmentation during dynamic batching, serialization delays over network interfaces, or pipeline stalls caused by real-world telemetry feeds. Online evaluation tests candidate models under real serving dynamics using **staged rollout**:²⁵

1. *Shadow mode* (dark traffic): The API gateway mirrors live production requests asynchronously to the candidate model running on dedicated accelerator resources. The candidate executes inference under true input dimensions and concurrent request arrival patterns, but its predictions are sinked to telemetry logs rather than returned to clients. This isolates tail-latency spikes, out-of-memory (OOM) faults, and numerical divergences without user-facing blast radius.

²⁴ | **Calibration:** A calibrated model’s predicted probabilities match observed frequencies: 80 percent confidence should correspond to 80 percent correctness. Calibration is distinct from accuracy, and the systems consequence for the DR system is severe: clinicians use confidence scores for triage decisions, so a miscalibrated model that assigns 90 percent confidence to uncertain cases can misdirect clinical workflows more dangerously than a less accurate but well-calibrated alternative. Platt scaling and temperature scaling can improve posttraining calibration, but their effectiveness must be evaluated on representative data.

²⁵ | **Staged rollout:** Shadow mode runs the new model in parallel, logging predictions without serving them. Canary deployment (named after coal-mine canaries) then exposes a limited, configurable share of traffic and increases it gradually if metrics hold. Coverage depends on traffic, observability, duration, and the failure modes exercised, so staged rollout reduces risk without guaranteeing that it catches a fixed share of production issues. Chapter 14 details implementation strategies.

2. *Canary deployment*: The ingress load balancer directs a small fraction of live traffic (such as 1 to 5 percent) to the new serving container, returning live predictions while monitoring systems telemetry for crash loops, latency regression, and downstream error spikes.
3. *A/B testing*: Traffic splitters partition user requests across candidate and baseline models using deterministic hashing, providing statistical evidence to verify whether algorithmic improvements translate into measurable business or clinical gains.

This staged rollout workflow manages deployment validation risk; it is distinct from the cross-tier model compression pattern termed progressive deployment in chapter 2.

Each validation stage contributes distinct, complementary operational evidence, as table 3.4 summarizes.

Table 3.4: Staged Validation Coverage: The stages provide complementary, overlapping evidence rather than partitioning failures into exclusive classes.

Validation stage	Typical evidence it adds
Offline evaluation	Algorithmic behavior
Shadow mode	Integration behavior
Canary deployment	Limited-traffic behavior
A/B testing	Comparative outcomes

Serving infrastructure must support this staged validation pipeline by design. Retrofitting dynamic traffic splitting, asynchronous request mirroring, and differential output logging onto an existing deployment stack requires extensive modifications to API gateways and telemetry pipelines.

3.6.3 Production-condition validation

Production-condition validation evaluates whether an ML model withstands the physical and operational environments specified during problem definition. Standard evaluation implicitly assumes that test samples are identically distributed to training data; production validation deliberately stress-tests this assumption across three dimensions: site heterogeneity, input-level perturbations, and temporal drift.

External and multisite validation address site-level distribution shifts: determining whether the model has learned generalizable representations or overfit to the sensor hardware of the development environment. In retinal screening, camera sensor sensitivities, optical lens coatings, and illumination spectra differ across equipment vendors. A DR model trained exclusively on images from high-end fundus cameras in research hospitals can fail when deployed to community clinics utilizing older hardware. Multisite validation requires benchmarking candidate models on data collected across independent clinic sites with varying hardware configurations, ambient lighting environments, and operator skill tiers.

Robustness testing shifts focus to individual inference requests, evaluating resilience against realistic physical distortions and edge cases. For computer vision pipelines, these perturbations include sensor noise, optical blur, uneven contrast, and partial pupil occlusions. In the Thai DR clinical study (Beede et al. (2020)), field technicians with limited formal training frequently captured off-center or poorly focused fundus images. Evaluating models against synthetic and empirical perturbation suites exposes whether the serving pipeline requires an upstream image-quality filter to reject corrupt frames before allocating accelerator cycles for inference.

Temporal validation evaluates whether the model maintains predictive accuracy as the operating environment evolves over weeks and months. Cross-sectional test sets drawn from a fixed historical window fail to capture subsequent shifts in patient demographics, disease epidemiology, or clinical practice patterns. Input distribution changes without label changes produce *covariate drift*, whereas shifts in the relationship between input features and clinical diagnoses cause **concept drift**. Both phenomena cause silent model degradation and motivate the continuous telemetry and drift detection pipelines described in section 3.8.

3.6.4 Regulatory validation

Healthcare AI systems face device- and pathway-specific validation requirements. When FDA premarket review is required, the safety and performance evidence must be appropriate to the device and its regulatory pathway; clinical data are required for some submissions, not all.²⁶

Domain-specific validation extends beyond statutory compliance to address clinical utility and safety protocols. In diabetic retinopathy screening, clinical validation studies deploy the automated system alongside licensed ophthalmologists, comparing model classifications against ground truth established by consensus adjudication panels. These studies must demonstrate not only non-inferior diagnostic accuracy but also safe failure modes: systems that *fail safely*—by routing ambiguous or low-confidence samples to clinical specialists—integrate successfully into care delivery, whereas systems that *fail silently* generate catastrophic diagnostic errors.

Human factors validation evaluates the model as an embedded component within the human decision-making loop. An accurate model that induces high operator cognitive load, exhibits excessive inference latency that stalls patient intake, or triggers frequent false alarms creates alert fatigue, prompting clinic staff to disable or bypass the system. Validation protocols must therefore quantify sociotechnical metrics—including clinician override frequency, examination turnaround time, and referral concordance—alongside raw statistical accuracy.

3.6.5 Deployment readiness

Passing the validation gate produces an immutable release manifest that certifies the model for production deployment. This manifest records the exact model checkpoint hash, preprocessing container configuration, calibration parameters, benchmark scores across designated demographic and equipment slices, and regulatory clearance certificates. If a model fails to clear an established threshold—such as p99 serving latency exceeding the service level agreement (SLA) budget or sensitivity on low-light fundus images falling below the safety floor—the release gate halts deployment. Diagnostics then route backward to the responsible lifecycle stage: data curation for targeted data collection, model training for loss function re-weighting, or compiler optimization for kernel fusion.

Clearing the algorithmic validation gate is a prerequisite for production, but deployment readiness requires verifying the entire execution substrate. A mathematically validated model cannot serve traffic without an operational serving path: deterministic input preprocessing routines, serialized inference graphs optimized for the target hardware runtime, telemetry collectors, and automated rollback triggers. Once the validation evidence package is complete, the operational challenge transitions from offline verification to online serving and systems integration.

3.7 Deployment and Integration

A model that passes validation in development faces different failure modes once deployed to production. In the DR scenario, the model must run on tablets in rural clinics with intermittent connectivity, integrate with hospital information systems with distinct schema requirements, and deliver predictions clinicians can interpret directly. The scenario's latency and connectivity requirements rule out round-trip cloud inference. Deployment is where abstract operational constraints become physical systems requirements. In iron-law terms, the serving path couples data movement, execution efficiency, and fixed overhead together; which term binds varies by archetype. ResNet-50-class vision workloads may use batching to improve throughput, DLRM-class recommendation workloads emphasize interactive latency and embedding access, and TinyML-class audio workloads operate under severe energy budgets (table 3.2). Dedicated serving infrastructure (chapter 14) manages these trade-offs under live traffic.

3.7.1 Deployment requirements

Deployment translates an abstract computational graph into hard physical budgets: peak memory footprint, execution latency, thermal dissipation limits, and network bandwidth. In the clinical screening scenario, these constraints force an architectural choice between centralized cloud serving and localized edge execution. A cloud deployment consolidates compute onto centralized accelerators, but requires stable wide-area network uplinks and incurs recurrent API and bandwidth charges. Conversely, an edge deployment eliminates network transit latency and operates during

²⁶ | **FDA AI/ML regulation:** FDA marketing authorization requires evidence appropriate to the device, intended use, risk, and regulatory pathway. The FDA's 2021 AI/ML SaMD Action Plan identified lifecycle management, real-world performance monitoring, and pre-determined change control planning as central issues for adaptive medical-device software (U.S. Food and Drug Administration 2021). These concerns make model versions, training provenance, monitoring, and change documentation important workflow artifacts without imposing one identical audit artifact at every lifecycle stage.

wide-area connectivity outages, but demands upfront hardware capital expenditure and restricts model size to local device memory. Evaluating this trade-off requires quantifying the relationship between capital expense, operating expense, and inference volume.



Napkin Math 3.3: Cloud vs. edge deployment economics

Problem: A production model processes about 760,000 billable screening images per month across 500 clinics, assuming daily operation and one processed image per patient after local selection and quality checks. Should the deployment use a cloud inference endpoint or edge inference on an on-premises server?

Option A: Cloud inference.

- Model runs on centralized GPU servers
- Inference cost: ~\$0.01/image (cloud GPU time + API overhead)
- Annual inference cost across 500 clinics, 50 patients/day, 1 billable image per patient, 365 days/year, and \$0.01/image is \$91,250/year
- Plus: An assumed connectivity and network-operations allocation for uploading 5 MB per billable image = ~\$45,000/year
- **Total:** ~\$136,250/year operational cost
- Risk: 200 ms+ exceeds the scenario latency target; connectivity outages halt screening

Option B: Edge deployment on a clinic device.

- One-time hardware: one \$500/device per clinic across 500 clinics requires \$250,000 capital expense
- Inference cost: ~\$0.001/image (assumed all-in variable operating allocation)
- Annual cost: approximately \$25,000/year maintenance plus approximately \$9,125/year variable inference cost, for about \$34,125/year
- **Total:** \$250,000 upfront + ~\$34,125/year
- Benefit: latency below 50 ms; works offline; much lower per-inference cost

Math:

- Annual savings: \$136,250/year (cloud) – \$34,125/year (edge) = \$102,125/year
- Payback: \$250,000 (hardware) ÷ \$102,125/year = ~2.4 years

Systems insight: Under these scenario assumptions, edge deployment pays back in ~2.4 years and allows inference during connectivity outages, yet it requires tighter model optimization (must fit in edge memory) and more complex update pipelines. The deployment paradigm selected during Problem Definition determines whether the edge option is even viable.

In this scenario, bandwidth limits, latency targets, and offline operational requirements favor edge deployment. That choice dictates strict memory, latency, and thermal envelopes that model compression (chapter 10) must satisfy and serving runtimes (chapter 13) must execute within production bounds.

Host integration extends these physical constraints to the software interfaces of existing clinical infrastructure. The inference runtime must exchange data with hospital information systems (HIS) to ingest patient records and record diagnostic output. Integrating a probabilistic model into an established clinical workflow differs fundamentally from connecting a deterministic sensor. Rather than emitting an invariant scalar measurement, the model produces a probability distribution coupled with an uncertainty estimate. The integration boundary therefore requires an explicit interface contract governing decision routing. This contract maps predictive uncertainty to clinical actions, specifying which classifications trigger automated referrals and which demand physician review. A confidence threshold cannot be evaluated in isolation; its operational safety depends entirely on the downstream workflow it triggers. Regulatory frameworks introduce structural

constraints on this data lifecycle. Legal mandates such as HIPAA govern data transmission and restrict whether production inference payloads can be persisted, linked to patient identifiers, or ingested into retraining pipelines. When compliance prohibits retaining raw inputs, continuous improvement cannot rely on passive telemetry logging. Privacy rules thus constrain ongoing model maintenance strategies rather than serving merely as a transport-layer security checklist (chapter 14).

3.7.2 Pilot to full deployment

Production rollout proceeds in discrete stages to isolate failure modes across expanding execution environments. Offline replay and synthetic test harnesses verify schema contracts and concurrency limits without risking patient safety. Subsequent pilot deployments in a small number of clinics expose the serving stack to physical variations absent from laboratory benchmarks: optical distortion across camera models, sensor noise under uneven clinic lighting, and operator handling variations. Full-scale rollout broadens exposure to the long tail of anatomical variations and rare pathologies. Detecting these distribution shifts requires continuous runtime monitoring rather than assuming validation distributions persist indefinitely.

Scaling across hundreds of remote clinics introduces hardware heterogeneity and operational friction across distributed nodes. Variations in local sensor optics alter input pixel distributions before execution begins, necessitating robust input-validation kernels that reject out-of-spec captures locally. At the same time, severe wide-area bandwidth constraints rule out continuous weight synchronization or streaming high-resolution telemetry back to a centralized cluster. Edge nodes must instead operate autonomously, caching execution metrics locally and synchronizing state opportunistically during scheduled maintenance windows.

Sustaining clinical utility requires calibrated uncertainty estimation rather than raw aggregate accuracy. When the model encounters ambiguous pathology, emitting an uncalibrated prediction risks diagnostic failure. Instead, an operational serving path incorporates **human-in-the-loop routing**, using calibrated decision boundaries to direct borderline cases to expert clinicians. This routing mechanism serves a dual purpose: it acts as a real-time safety boundary for patient care, and it establishes a continuous data collection channel. Adjudicated edge cases yield verified, high-information annotations that are fed back into subsequent training iterations, directly counteracting the model's known failure modes.

Maintaining consistency across distributed deployments demands immutable model artifact versioning and atomic update procedures. Client runtimes must verify cryptographic checksums of serialized graphs before loading new weights into memory, preventing partial updates from corrupting edge execution. Deployment marks the transition from static offline evaluation to continuous operations, where system telemetry continuously assesses whether real-world latency distributions and inference behaviors remain within specified operating bounds.

3.8 Monitoring and Maintenance

Consider an illustrative case six months after a DR screening system launches. A clinic upgrades its fundus cameras, and the new equipment produces images with different color profiles. The model's sensitivity then drops at that site because the pixel distributions it learned during training no longer match the images it receives. No code changed. The data drifted beyond the training envelope, and the model degraded silently. *ML systems can degrade through data drift even when their artifacts remain untouched.* This possibility means that deployment is not the end of the lifecycle but the beginning of an ongoing operational phase. Monitoring provides the statistical telemetry to detect degradation, while continuous maintenance pipelines (chapter 14) update models when evidence warrants it.

In this illustrative scenario, monitoring tracks performance across clinics to detect whether changing patient demographics, camera technology, or equipment degradation affects accuracy. Adding a new imaging modality such as optical coherence tomography would be a product change requiring new data, validation, workflow integration, and any applicable regulatory review rather than routine maintenance. Three feedback pathways guide update decisions: performance evidence can motivate targeted data collection, data-quality findings can prompt preparation changes, and confirmed model degradation can justify retraining or another intervention. Drift thresholds initiate review; they do not select the remedy automatically.

3.8.1 Production monitoring

Monitoring must serve two audiences simultaneously: technical teams tracking system health metrics and clinical staff needing actionable insights. Initial deployment may reveal blind spots invisible during laboratory validation.²⁷ Clinics with older equipment may show accuracy decreases. Specific patient subgroups, such as those with proliferative retinopathy or cataracts complicating the fundus image, may trigger higher error rates. These discoveries drive targeted data collection and architectural improvements.

A DR screening system, where missed referable disease can delay care and contribute to avoidable vision loss, demands continuous operational monitoring plus periodic performance evaluation when labels arrive. Teams establish quantitative thresholds for latency, accuracy, and data distribution stability. Lightweight statistical tests such as the population stability index (PSI) and Kolmogorov-Smirnov (KS) test²⁸ trigger responses ranging from automated alerts to retraining reviews (chapter 14).

A production DR system can track four metric categories at different timescales. The following thresholds illustrate one scenario policy; clinical and operational teams must validate them for the intended product, population, and deployment environment.

- **Model performance metrics** (requiring ground truth, available with delay): sensitivity (target above 90 percent, alert if seven-day rolling average drops below 88 percent), specificity (target above 80 percent, alert if it drops below 78 percent), and subgroup performance (alert if any demographic drops more than 5 percentage points below baseline).
- **Proxy metrics** (available immediately, without ground truth): prediction confidence distribution (alert if mean confidence drops more than 10 percent relative to baseline), referral rate (alert if rate changes more than 15 percent from baseline), and image quality rejection rate (alert if more than 20 percent of images fail quality checks).
- **Operational metrics**: Inference latency (p95 below 50 ms, alert if above 100 ms), throughput (alert if queue depth exceeds 50 images), and error rate (alert if more than 0.1 percent of requests fail).
- **Data stability metrics**: Feature and prediction distributions compared with a baseline, with alerts when recent traffic moves outside the expected range.

The hierarchy matters: operational metrics can surface immediate problems, proxy metrics can flag possible model issues without waiting for ground truth, and performance metrics often arrive later because they require labeled data.

3.8.2 Maintenance at scale

Model updates require careful validation and controlled rollouts. Teams employ A/B testing frameworks to evaluate updates and implement rollback mechanisms²⁹ that address issues quickly. ML systems must account for data evolution that can affect behavior independently of code changes.

In the illustrative scenario, scaling from pilot sites to hundreds of clinics increases monitoring complexity. The resulting log volume depends on request rates, sampling, payload sizes, and retention policies as well as the number of clinics. The monitoring infrastructure must track both global metrics and site-specific behaviors, maintain **data lineage**,³⁰ the metadata trail linking production logs to data, code, and model versions, where required for regulatory compliance, and correlate production issues with training experiments for root cause analysis.

Proactive maintenance closes the lifecycle loop: operational signals identify potential problems, review determines whether newly validated data or another change is needed, and production evidence feeds back to refine problem definitions, data-quality standards, and architectural decisions. When retraining is selected, scheduled or evidence-triggered pipelines can incorporate approved data and return the candidate through validation. Underlying these operational dynamics are three fundamental systems principles: constraints propagate across stage boundaries, feedback operates across mismatched timescales, and system-level behavior diverges from component-level metrics (section 3.9).

²⁷ | **Lab-to-clinic performance gap:** Medical AI systems can experience substantial performance changes when camera models, image quality, patient populations, or operator workflows differ from development data; the gap arises because training data cannot capture the full diversity of production conditions. FDA has emphasized total product lifecycle oversight and real-world performance monitoring for AI/ML-enabled medical devices, and device submissions may need evidence appropriate to the product's intended use and risk. For ML systems engineers, this means monitoring infrastructure should be a deployment prerequisite, not a postlaunch addition.

²⁸ | **Population stability index (PSI) and Kolmogorov-Smirnov (KS) test:** Two lightweight statistical methods for detecting distribution drift; PSI bins features and computes divergence; thresholds such as 0.1 and 0.2 are operating conventions, not universal significance levels. The KS test measures maximum distance between empirical cumulative distributions. Both are cheap enough for frequent monitoring, but a detected input shift does not by itself prove an accuracy change; labels or other outcome evidence are needed. Chapter 14 covers drift detection pipelines in depth.

²⁹ | **ML rollback complexity:** An ML model's validity is coupled to the data distribution on which it was trained, not just its code. "Data evolution" means a simple rollback restores a model artifact but cannot restore the past data environment, creating a temporal state mismatch. Even a rapid rollback is therefore a mitigation tactic rather than a true system restore, as the stale model's performance on live data is not guaranteed.

3.9 Systems Thinking

The diabetic retinopathy scenario demonstrated how lifecycle stages interact as a coupled system: physical constraints at the point of care directly governed model capacity, training data curation, and preprocessing logic. Three structural mechanisms explain these coupled interactions: backward constraint propagation, multi-scale feedback loops, and emergent system trade-offs. Managing these mechanisms distinguishes proactive, constraint-driven system design from reactive post-deployment debugging.

3.9.1 Constraint propagation principle

In the DR scenario, an intermittent 2 Mbps clinic uplink ruled out cloud inference, requiring edge deployment. The edge target's memory and power budgets then constrained model architecture, favoring lightweight factorized convolutions over large ensembles and requiring automated image-quality checks at capture time. Each downstream constraint narrowed the design space for upstream stages, coupling hardware limits directly to data collection and model design.

Definition 3.3: The constraint propagation principle

The constraint propagation principle states that a constraint discovered late in the ML lifecycle can force rework in affected earlier stages. Actual correction cost depends on which artifacts and decisions must change. This chapter's $2^{N_{\text{stage}}-1}$ rule is an illustrative sensitivity scenario, not an empirical cost law.

1. **Significance:** A 100 ms latency target discovered at deployment (stage 5) may propagate backward to constrain model size (stage 3: algorithm complexity O), dataset requirements (stage 2: dataset size D), and problem definition (stage 1: what accuracy is achievable). Rework depends on which earlier decisions relied on the missing constraint. Within the iron law, a deployment constraint on L_{lat} or R_{peak} can redefine the feasible region for O , D_{vol} , and η_{hw} .
2. **Distinction:** Unlike modular decomposition (which encourages independent optimization of each component), this principle mandates end-to-end reasoning: optimizing accuracy in isolation may produce a model that is infeasible to deploy, making the “local maximum” in accuracy a “global minimum” in system viability.
3. **Common pitfall:** A frequent misconception is that deployment is “the last step.” The deployment environment is the day-one constraint: its latency budget, memory capacity, and power envelope define the boundaries of every upstream decision.

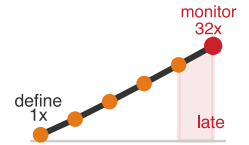
Propagation operates bidirectionally, creating dynamic constraint networks rather than linear dependencies. When rural clinic deployment reveals tight bandwidth limitations, teams may redesign the pipeline to transmit compact outputs after local inference. If the system instead compresses model inputs to traverse the uplink, the lossy compression artifacts degrade input quality; the training dataset must then incorporate matching distortion augmentations to prevent training-serving skew. Conversely, shifting inference entirely to the edge to transmit only diagnosis scalars relieves uplink bandwidth but tightens the edge device's DRAM capacity and thermal envelope. Upstream data choices and downstream machine boundaries remain physically coupled.

Decisions made without accounting for downstream execution boundaries accumulate technical debt.³¹ The stage interface specification (table 3.3) operationalizes this principle by making constraints explicit at each stage boundary, aligning with the model, data, and infrastructure contract practices discussed in chapter 14. Those contracts enable earlier detection of constraints. When propagation occurs specifically through data quality failures, the resulting pattern is known as a **data cascade**: a chain of downstream failures triggered by bad data (Sambasivan et al. 2021). Chapter 4 formalizes this failure mode and traces how it unfolds stage by stage.

3.9.2 Multi-scale feedback

ML systems succeed by orchestrating feedback loops across multiple timescales, each serving a different purpose. One illustrative operating plan for the DR scenario uses minute-level checks to

30 | Data lineage: The automated recording of meta-data linking each clinic's production logs to the exact data, code, and model version that generated them. Without this explicit trail, correlating a site-specific accuracy drop with a training experiment can require manual forensic analysis that delays root-cause identification.



An illustrative doubling scenario visualizes late-discovery sensitivity.

31 | ML technical debt: Sculley et al. (2015) identify ML-specific debt mechanisms such as *entanglement* (changing one feature affects all others because the model learned joint distributions), *hidden feedback loops* (predictions influence future training data), and *undeclared consumers* (downstream systems depending on outputs without contracts). Since ML code is often only a small part of a production system, the surrounding configuration, pipelines, and infrastructure can allow this debt to accumulate silently.

catch a misconfigured camera before it produces a shift's worth of unusable images; daily reviews for proxy shifts such as referral-rate, confidence, rejection-rate, or site-specific camera changes; weekly aggregation of labeled accuracy statistics and drift tests when ground truth is available; monthly analysis of population coverage; and quarterly review of whether the architecture still meets clinical needs. Actual cadences depend on label delay, product risk, traffic, and the cost of acting on each signal.

The temporal structure of these feedback loops reflects the inherent dynamics of ML systems. Rapid loops enable quick correction of operational issues: a clinic's misconfigured camera can be detected and corrected within minutes. Slower loops enable strategic adaptation; recognizing that population demographic shifts require expanded training data takes months of monitoring to detect reliably. This multi-scale approach balances responsiveness against statistical stability. Overreacting to short-term variance causes pipeline thrashing, such as triggering unnecessary retraining on transient shifts, while sluggish adaptation allows silent accuracy decay to persist unmitigated.

3.9.3 Emergent complexity and resource trade-offs

Complex systems produce **emergent behaviors** invisible when analyzing individual components. In a multisite DR deployment, individual clinics can exhibit stable aggregate performance while system-wide analysis detects degradation concentrated in specific demographic cohorts—a failure mode undetectable in isolated clinic logs. Furthermore, while conventional distributed systems fail primarily through deterministic faults such as network timeouts or out-of-memory crashes, ML systems also experience probabilistic degradation: data drift, concept drift, and bias amplification. Probabilistic degradation generates no stack traces or runtime exceptions; the model continues serving predictions within its latency SLA while its statistical validity decays silently.



Checkpoint 3.3: The cost of late discovery

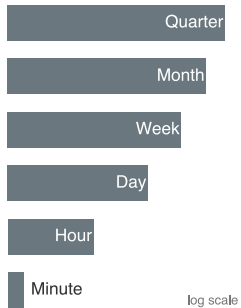
Apply the constraint propagation principle to this scenario:

A team discovers during monitoring (Stage 6) that their DR model fails for patients over 70 years old. This demographic requirement should have been specified at Problem Definition (Stage 1).

- ☐ Cost multiplier: Calculate the relative cost multiplier using the formula from the constraint propagation principle definition.
- ☐ Affected stages: Identify which intermediate stages may need to be revisited to fix this issue.
- ☐ Prevention rule: State the design rule that would have prevented this late-stage rework.

Resource optimization introduces multi-dimensional trade-offs that also arise in conventional software, while ML adds learned statistical behavior to them. An accuracy improvement might require increasing model parameter count, forcing deployment onto more capable hardware; multiplied across hundreds of clinics, that marginal accuracy increment dictates cluster capital expenditure. These trade-offs manifest the power wall and memory wall from chapter 2: edge deployment eliminates WAN latency but constrains model complexity to local SRAM and DRAM limits; cloud deployment removes accelerator memory bounds but introduces network latency that can violate point-of-care clinical workflows. Every architectural choice reallocates pressure across the D-A-M triad.

These three structural patterns—constraint propagation across stage boundaries, multi-scale feedback timing, and emergent system trade-offs—demonstrate that ML systems cannot be engineered through modular decomposition alone. Optimizing data pipelines, model topologies, and execution targets in isolation guarantees runtime bottlenecks or silent failures. When engineering teams apply conventional software assumptions to these coupled dynamics, predictable antipatterns emerge. The fallacies and pitfalls in section 3.10 examine where these mental models break down across production lifecycles.



An illustrative monitoring plan spans minutes to quarters across five orders of magnitude.

3.10 Fallacies and Pitfalls

Applying conventional software patterns to machine learning workflows overlooks the statistical and data-dependent nature of learned systems. The following fallacies and pitfalls capture recurring errors that stall development cycles, cause silent production outages, and compound technical debt across pipeline stages.

Fallacy: *ML development can follow traditional software workflows without modification.*

Traditional software processes rely on deterministic specifications verified by binary unit tests. Machine learning systems, by contrast, derive behavior inductively from data, replacing deterministic guarantees with statistical performance distributions (table 3.1). Linear phase gates and rigid sprint boundaries assume requirements remain fixed once specified. In an ML workflow, problem definitions evolve as exploratory training reveals distribution gaps, class imbalances, or infeasible accuracy-latency trade-offs. Furthermore, practitioner surveys demonstrate that data preparation and pipeline maintenance dominate engineering effort (section 3.1.1). Rigid linear gates treat data collection as a completed precursor rather than an active feedback loop, trapping teams in costly cycles when downstream evaluation forces fundamental revisions to early data assumptions.

Pitfall: *Treating data preparation as a one-time preprocessing step.*

Teams often treat data cleaning and feature engineering as a static milestone to clear before training begins. In production, incoming data distributions evolve continuously as user behavior shifts, sensors degrade, and upstream schemas change. The two-pipeline architecture (figure 3.1) reflects this operational reality: data pipelines and model pipelines must execute concurrently with continuous feedback. Early data preparation decisions cascade directly through training and deployment (section 3.4). When data preparation is static, silent distribution shifts and training-serving feature skew bypass code-level tests, degrading inference accuracy long before raising an operational alert. Continuous data validation pipelines with automated schema checking and distribution monitoring are necessary to catch data drift at ingestion rather than after silent model failure in production.

Fallacy: *Passing model evaluation means the system is ready for deployment.*

Offline evaluation measures statistical loss on static, curated validation sets, but it isolates the model from operational serving dynamics. Strong aggregate metrics provide no guarantee that the system will satisfy physical deployment constraints. In the two-pipeline architecture (figure 3.1), offline evaluation leaves serving integration, real-time input pipelines, and operational monitoring untested. The diabetic retinopathy case study (section 3.2.1) exemplifies this disconnect: a model that achieved specialist-level accuracy on curated benchmark images failed in clinic deployments due to variable camera lighting, technician workflow interruptions, and intermittent network connectivity. Production readiness demands system-level verification, including tail-latency budgets under concurrency, feature pipeline consistency, and graceful degradation under hardware faults. Under the constraint propagation principle (section 3.9.1), discovering operational failures only after deployment forces costly iterations across both model architecture and data collection.

Pitfall: *Scaling data collection before checking marginal model value.*

Expanding dataset size is often assumed to be a universally profitable investment for boosting accuracy. In practice, data collection follows power-law scaling with diminishing returns: once the underlying data distribution is adequately covered, additional raw examples yield negligible generalization gains while storage overhead, ingestion bandwidth, and labeling expenses escalate. The feedback loops in figure 3.1 show that model accuracy depends on the coupling between dataset quality, model capacity, and operational constraints. Ingesting high volumes of mislabeled or redundant samples introduces noise and inflates training compute without improving accuracy, whereas a compact, carefully curated dataset with verified labels and balanced representation often achieves superior performance. Because data quality decisions cascade through all downstream pipeline stages (section 3.4), teams must measure the marginal accuracy gain per additional sample before allocating engineering budget to larger data ingestion.

Fallacy: *Skipping validation stages accelerates delivery.*

Compressing or bypassing progressive validation stages appears to accelerate project delivery, but it transfers verification risk directly to production traffic. Multi-stage validation (section 3.6) isolates orthogonal failure modes across increasingly realistic execution environments. Skipping shadow mode prevents teams from observing live inference behavior—such as payload deserialization bottlenecks, concurrency contention, and tail-latency SLA violations—under real traffic distributions. Bypassing canary deployments removes the blast-radius barrier, exposing all users immediately to edge-case regressions or numerical instability. Resolving failures in production requires emergency rollbacks, distributed log triage, and corrupted state repair, incurring orders of magnitude more engineering downtime than the validation gates bypassed.

Pitfall: *Deferring deployment paradigm selection until after model development.*

Teams often assume deployment targets can be selected after optimizing model accuracy. In practice, the deployment paradigm (Cloud, Edge, Mobile, or TinyML) is a primary constraint that shapes every preceding lifecycle stage (table 3.3). Suppose a team develops a 2 GB ensemble model before learning that its TinyML target has 256 KB of memory. The resulting mismatch cascades backward, forcing teams to redesign data preprocessing, model architecture, and evaluation benchmarks. The chapter’s illustrative constraint-propagation model assigns a stage-5 discovery $2^4 = 16\times$ the stage-1 cost. Deferring platform selection guarantees expensive iteration cycles, because deployment hardware dictates which models are physically realizable, not merely where they execute.

3.11 Summary

The lifecycle is a feedback loop, not a checklist. The data pipeline transforms raw inputs through collection, ingestion, analysis, labeling, validation, and preparation into ML-ready datasets. The model development pipeline takes these datasets through training, evaluation, validation, and deployment to create production systems. With the full chapter as context, the feedback arrows in figure 3.1 carry the chapter’s central meaning: each one represents a lesson learned in production flowing back to strengthen earlier stages, making data and model feedback explicit in the development cycle.

Understanding this framework explains why machine learning systems require specialized additions to established software-engineering practices. ML workflows add probabilistic optimization, learned statistical behavior, and data-dependent feedback loops. The iron law supplies one quantitative lens: decisions across the lifecycle jointly change data movement, operation count, hardware efficiency, and fixed latency in $(T = \frac{D_{vol}}{BW} + \frac{O}{R_{peak} \cdot \eta_{hw}} + L_{lat})$, while production evidence feeds constraint violations back into review. This perspective recognizes that success emerges not from perfecting individual stages in isolation, but from understanding how data quality affects model performance, how deployment constraints shape training strategies, and how production insights inform each subsequent development iteration.

Three patterns define the engineering discipline of the ML lifecycle. Many practitioners report data collection, cleaning, labeling, validation, or preparation as major time sinks, while model development is only one part of the lifecycle. Production-ready systems also require repeated iteration cycles across data, model, and infrastructure stages, so investment in data engineering can have high leverage when data quality is a major source of rework. Finally, late constraints can increase rework: the chapter’s illustrative constraint-propagation model assigns a constraint discovered at stage N_{stage} roughly $2^{N_{stage}-1}$ times the stage-1 correction cost, and it assigns a deployment paradigm mismatch discovered at stage 5 a $16\times$ cost multiplier. Actual rework depends on which stages are affected. Early constraint discovery does not require freezing the workflow. It establishes an operating envelope through latency, memory, power, safety, and validation obligations while data and model choices continue to iterate within those bounds.



Key Takeaways: See the whole map first

- **The lifecycle is a loop, not a checklist:** Data and model pipelines advance in parallel, but production feedback is what makes them a system. Monitoring, validation, and

retraining send lessons from deployment back into collection, labeling, architecture, and infrastructure decisions.

- **Late constraints can increase rework:** The illustrative model assigns a deployment limit found at stage N_{stage} roughly $2^{N_{\text{stage}}-1}$ times the stage-1 correction cost. The stage-5 mismatch's 16× multiplier shows why requirements should flow backward early.
- **Iteration velocity expands search opportunity:** Under the worked assumptions, a light-weight model starting 5 percentage points behind reaches the shared 99 percent ceiling because shorter cycles permit more assumed improvements. Workflow speed alone cannot guarantee model quality.
- **Interfaces make feedback actionable:** Stage contracts define inputs, outputs, and quality invariants so that data, model, and deployment teams can detect violations before integration. Without explicit contracts, each stage can optimize locally while the system fails globally.
- **Production speaks on different clocks:** Real-time inference monitoring, batch update reviews, and quarterly architectural reviews answer different failure modes. Treating all feedback as one loop either reacts too slowly to drift or churns expensive workflows without signal.
- **Workflow carries constraints through time:** Problem definition records targets, data collection shapes evidence and byte volume, model development changes operations and reuse, deployment realizes the complete serving path, and monitoring sends violations back into review. The lifecycle is data-algorithm-machine coupling unfolding over time.

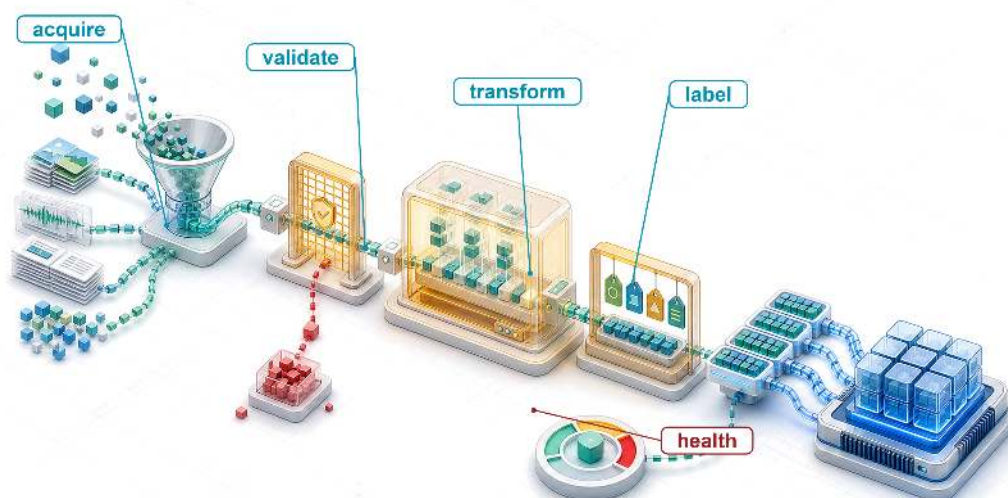
This workflow framework turns ad hoc ML experimentation into a more disciplined engineering practice. By understanding how data pipelines and model development interact through feedback loops, teams can identify integration risks earlier and allocate resources more deliberately. The constraint propagation principle shows why systematic workflow management is risk mitigation rather than bureaucratic overhead.

Seen as a checklist, the lifecycle is a sequence of stages to clear in order. Seen correctly, it is a loop. A constraint discovered at deployment does not stay there; it may force rework in dependent model and data decisions. A workflow is therefore a coupled system rather than a pipeline, the same coupling the D·A·M taxonomy describes in *space*, now unfolding in *time*. Optimizing one stage in isolation moves the failure instead of removing it, which is why the discipline is to see the whole map before touching any single part of it.

What's Next: From blueprint to fuel

The workflow framework established here provides the blueprint for every technical chapter that follows. An engine without fuel, however, is only a heavy block of metal; for an ML system, that fuel is data. Chapter 4 therefore develops the first D·A·M axis, Data. Part II then builds the core system (neural network computation, model architectures, frameworks, training), Part III optimizes it for deployment (data selection, model compression, hardware acceleration, benchmarking), and Part IV deploys and operates it in production (model serving, operational practices, responsible engineering). Each chapter assumes familiarity with where its techniques fit within this complete workflow, building on the systematic perspective developed here.

Data Engineering

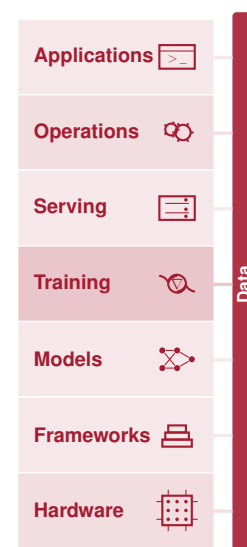


- 4.1 Dataset Compilation
- 4.2 Physics of Data
- 4.3 Four Pillars Framework
- 4.4 Data Acquisition
- 4.5 Data Pipeline Architecture
- 4.6 Systematic Data Processing
- 4.7 Data Labeling
- 4.8 Storage Architecture
- 4.9 Operational Data Health
- 4.10 Fallacies and Pitfalls
- 4.11 Summary

Purpose

Why does training data function like source code in a machine learning system?

In conventional software, programmers write logic that computers execute. In machine learning, programmers write optimization procedures that extract operational logic from data. This inversion makes training data resemble source code: changing it can change the learned system when the model is retrained, even if no traditional code changes. Subtle labeling inconsistencies can induce behavioral inconsistencies; missing edge cases can leave corresponding failures unmeasured; historical biases can propagate into learned behavior. A model trained only on that dataset cannot infer task-relevant evidence absent from it, and systematic label errors can shape what it learns. Unlike traditional source code, which sits inert until a programmer modifies it, the operating distribution can shift as the world changes, potentially changing production performance even when nothing in the codebase has been touched. Data engineering can consume substantial project effort because the work is consequential. Every decision in the data pipeline (what to collect, how to label, when to filter, how to split) propagates forward to constrain model architecture, training dynamics, and deployment viability. Data engineering is therefore not preprocessing but programming in a different language, one where quality control, versioning, and monitoring determine whether the compiled system works today and continues working tomorrow. In D·A·M terms, the pipeline itself is a data-machine co-design problem: however well curated the data, the algorithm can only learn as fast as the machine can deliver it.





Learning Objectives

- Explain data as source code and trace how data cascades propagate through ML systems
- Calculate data gravity, feeding-tax, and storage-bandwidth costs for moving or serving datasets
- Evaluate acquisition strategies against coverage, quality, labeling cost, governance, and deployment constraints
- Design ingestion and validation pipelines for batch, streaming, ETL, and ELT workloads
- Implement idempotent transformations, lineage, and drift checks to preserve training-serving consistency
- Select labeling, storage, file format, versioning, and feature-store designs for ML lifecycle needs
- Diagnose data debt and production pipeline failures using quality, reliability, scalability, and governance evidence

4.1 Dataset Compilation

THE workflow becomes concrete in the data pipeline: raw inputs pass through collection, ingestion, analysis, labeling, validation, and preparation before they become ML-ready datasets. The effort breakdown reported in the 2016 Crowdflower survey (CrowdFlower 2016) explains why that pipeline needs its own systems treatment: 60 percent of respondents selected cleaning and organizing data as their most time-consuming task, and 19 percent selected collecting datasets. The data axis of the D·A·M taxonomy becomes real only as infrastructure: acquisition systems, validation checks, labeling workflows, storage layouts, and governance.



Definition 4.1: Data engineering

Data engineering is the infrastructure layer that manages the lifecycle of data from source to model, encompassing acquisition, transformation, storage, and governance.

1. **Significance:** Its critical function is ensuring training-serving consistency, preventing silent degradation by decoupling the model from the volatility of raw data. Within the iron law, it governs bytes moved (D_{vol}), while dataset composition and quality determine whether the dataset (D) remains representative of the target distribution.
2. **Distinction:** Unlike data science, which focuses on inference and insight, data engineering addresses the scalability and reliability of the data pipeline.
3. **Common pitfall:** A frequent misconception reduces data engineering to mere “data cleaning.” In systems terms, it is dataset compilation: transforming raw, noisy observations into an optimized binary layout that accelerator runtimes consume.

The **dataset compilation** analogy makes the infrastructure concrete. Just as a software compiler transforms source code through a series of increasingly refined representations (tokens, abstract syntax trees, intermediate representations, machine code), a data pipeline transforms raw, noisy observations into training-ready numeric tensors through analogous stages. Filtering corrupted records, outliers, and irrelevant features corresponds to *dead code elimination*: stripping material that contributes nothing to the learned representation. Augmentation, which synthetically expands limited examples by rotating images, pitch-shifting audio, or injecting noise, mirrors *loop unrolling*, exposing the model to more variations of the underlying pattern without collecting new data. **Deduplication** plays the role of *common subexpression elimination*, identifying and merging duplicate records that would otherwise bias gradient estimates and waste compute. Schema validation, enforcing strict types and ranges on every record, is the data pipeline’s *type checker*, rejecting malformed inputs before they crash the “runtime” of model training.

The engineering implication is that datasets must be versioned with immutable lineage, unit-tested through schema validation suites, and debugged when regressions appear. *Deleting a row of training data can change the learned artifact, just as deleting a line of code can change a compiled binary; retraining is the corresponding recompilation step.* Compilation also forces a trust boundary between dataset partitions. The training set is allowed to shape parameters; the validation set is allowed to shape modeling and pipeline choices; the test set is reserved for estimating generalization after those choices have been made. Leakage occurs when information crosses those boundaries: duplicate examples appearing in multiple splits, augmented variants of the same source record landing on both sides of the split, user records from the same household appearing in both training and test, or time-derived features computed using future observations. A subtle but prevalent form of leakage occurs during transformation: computing global parameters (such as mean and variance for normalization, or category vocabularies for imputation) across the entire dataset prior to splitting, which implicitly leaks distribution statistics from the test partition into the training phase. In the keyword-spotting case study threaded through this chapter, speaker-independent and time-aware splits enforce this boundary physically: partitioning utterances by speaker identity prevents the acoustic model from memorizing speaker-specific vocal-tract resonances, ensuring evaluation reflects generalization to the target deployment population.

Like a traditional software compiler, a dataset compiler operates across discrete physical stages: acquisition, ingestion, processing, labeling, storage, and lifecycle maintenance. Across every phase, the architecture balances the **four pillars framework**: *quality* (signal-to-noise ratio and distribution fidelity), *reliability* (idempotent execution and strict schema contracts), *scalability* (streaming I/O throughput that prevents accelerator starvation), and *governance* (immutable lineage and auditability). For constrained edge targets such as embedded keyword spotting, mismanaging an I/O ring buffer or violating a schema contract directly induces training-serving skew or runtime failure.

Pipeline efficiency depends on the physical interplay between movement cost and signal density. Two physical properties govern this trade-off: **data gravity** (the interconnect bandwidth, storage I/O, and transfer latency required to move byte volumes) and **information entropy** (the density of novel gradient signal contained within those bytes).

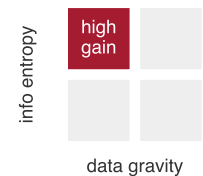
As the relationship between movement cost and information density illustrates, engineering returns peak when filtering and selection extract high-entropy records before incurring the transfer costs of data gravity. Low-entropy data—such as repetitive audio frames, background silence, or duplicate samples—saturates host-to-device PCIe bandwidth and pollutes memory hierarchies while yielding minimal gradient progress. Efficient dataset compilation resolves this physical constraint by maximizing the information entropy delivered per byte moved across the storage and interconnect hierarchy.

4.2 Physics of Data

The “data as code” metaphor captures *what* data does (determines system behavior) but not *why* moving it is so expensive. The physics of data explains why data systems must treat data as a physical substance with measurable properties. Just as physical substances possess density and viscosity, datasets exhibit measurable task-relevant signal and data gravity.

4.2.1 Data gravity

Data movement incurs physical and financial friction: **data gravity** scales directly with dataset volume (D_{vol}) and inversely with network bandwidth (BW). Transferring a petabyte dataset across a 10 Gbps link requires days of continuous transmission ($T = D_{\text{vol}}/\text{BW} \approx 9.3$ days); even across a dedicated 100 Gbps link, transfer latency and egress tariffs remain substantial enough to govern system design. Because migrating 1 PB across network boundaries is both slow and expensive, compute clusters must move to the storage location rather than data to compute. This constraint drove the emergence of data lakehouse architectures¹ (Zaharia et al. 2021), where distributed query engines such as Apache Spark and Presto execute directly over shared object stores. At organizational scale, the **data mesh** paradigm (Dehghani 2022) decentralizes ownership to avoid centralized bulk movement, treating datasets as domain-owned products.



Selection gain peaks where signal density (entropy) is high and movement cost (gravity) low.

¹ | **Data lakehouse:** Combines data lake storage (cheap, schema-less) with warehouse query semantics (ACID transactions, schema enforcement) using transactional table layers such as Delta Lake. For ML workloads, the lakehouse reduces the extract, transform, load (ETL) copy between lake and warehouse, enabling direct feature computation on the storage layer where data already resides—a direct response to data gravity, since repeated petabyte-scale copies increase the D_{vol}/BW cost (Armbrust et al. 2020; Zaharia et al. 2021).

4.2.2 Task-relevant signal

Not all bits deliver equal utility per unit of movement cost. While physical storage measures bytes, machine learning optimization extracts **task-relevant signal**—a task-specific measure of gradient information rather than raw Shannon entropy. A dataset containing one million identical images incurs substantial data gravity while providing zero additional gradient information beyond the first sample. Conversely, a curated collection of 10,000 diverse edge cases carries negligible data gravity yet contributes rich supervisory signal. Evaluating task-relevant signal against data gravity defines the return on data movement:

$$\text{Data Selection Gain} \propto \frac{\text{Task-Relevant Signal}}{\text{Data Gravity}} \quad (4.1)$$

This relationship favors datasets whose marginal learning value justifies their transfer overhead. Deduplication and active learning directly optimize this ratio by eliminating redundant byte volume and prioritizing high-entropy training examples.

4.2.3 The feeding problem: Flow rate and the “feeding tax”

Data gravity dictates the cost of moving data in bulk; **the feeding problem** dictates the cost of delivering it to execution units in real time. It is an input/output **flow rate** bottleneck: storage bandwidth fails to keep pace with compute throughput.

Under the iron law, a system runs only as fast as its slowest stage. When an accelerator can process 1,843 img/s, but the attached volume supplies only 250 MB/s, compute units stall on I/O. This gap defines **the feeding tax**: wall-clock time lost to I/O wait, which directly degrades system efficiency (η_{hw}). On baseline cloud block storage, this tax can exceed 77.5 percent—the accelerator spends more than half the step time waiting for data. Supplying this accelerator requires 1.1 GB/s, forcing pipeline architectures to adopt the specialized storage hierarchies analyzed in section 4.8.1. Distance also increases energy consumption: as data travels farther from arithmetic logic units, transport energy dwarfs the cost of computation itself.

Systems Perspective 4.1: The energy-movement invariant

The iron law in section 1.7 and the memory-wall analysis in section 2.4 imply a data-engineering principle: *moving one 32-bit value costs roughly 173× to 10,810.8× as much energy as the table’s FP32 multiply*, with the multiplier rising as the data leaves the chip. Later optimization chapters exploit the same cost gradient inside model and hardware design; here, the question is how much energy the information flow from the external world already consumes before the model computes. Table 4.1 makes that cost gradient explicit across the memory hierarchy.

Table 4.1: Energy Cost of Moving vs. Computing on a 32-Bit Value: Approximate energy per event at each stage of the data-movement hierarchy, with multipliers normalized to one 32-bit FP32 multiply. All rows use the same 32-bit width, so the ratios compare like-sized events. They do not by themselves prove which term dominates a complete workload; total energy also depends on the number of arithmetic operations and transfers.

Operation	Energy (pJ)	Relative Cost
32-bit FP Multiply	3.7 pJ/FLOP	1×
DRAM Memory Access (32-bit)	640 pJ	173×
Local SSD Access (32-bit)	4,000 pJ	1,081.1×
Network Transfer (32-bit)	40,000 pJ	10,810.8×

The cost gradient quantified in table 4.1 demonstrates why data locality dominates systems optimization. Each avoided transfer eliminates its corresponding movement energy, compounding across training epochs.

Because data movement incurs physical energy costs, data volume acts like physical mass. Pruning 50 percent of training data through deduplication does more than save persistent storage capacity: it eliminates energy expenditure across network, memory, and preprocessing stages. Filtering uninformative samples early is among the most effective architectural levers when I/O and communication dominate workload runtime.

The energy hierarchy and the signal-density heuristic converge on the same optimization lever: effective data engineering maximizes the Data Selection Gain defined in equation 4.1. Data curation functions as **signal-to-noise engineering**. Deduplication strips away redundant byte volume while preserving task-relevant signal, directly amplifying the selection gain. Active learning prioritizes high-entropy examples over repetitive samples, concentrating information density per byte transferred. Filtering uninformative records conserves interconnect bandwidth and storage energy, preventing upstream I/O starvation from idling accelerator execution units.

These principles govern individual batches within a single server node. At cluster and data center scale, data gravity dictates wide-area infrastructure choices. When moving petabytes across regions becomes slower and more expensive than provisioning remote accelerators, compute must migrate to the storage facility. Examining cross-data-center transfer costs quantifies this physical boundary.

 Napkin Math 4.1: The physics of data gravity

Problem: A 1 PB training dataset resides in a US East data center, while a remote accelerator pod is available in US West. Is it faster to move the data or to move the compute?

Math:

- Network bandwidth:** $100\text{ Gb/s} \div 8 = 12.5\text{ GB/s}$.
- Transfer time:** $1,000,000\text{ GB} \div 12.5\text{ GB/s} = 80,000\text{ seconds}$, approximately 22 h.
- Cost:** $1,000,000\text{ GB} \times \$0.02/\text{GB} = \$20,000$. (Baseline: AWS data transfer out pricing, 2024.)

Systems insight: If training takes less than 22 h, data transfer takes longer than training. If training costs less than \$20,000 (approximately 5,000 TPUv4-hours), bandwidth costs more than compute. For petabyte-scale data, code moves to data; for gigabyte-scale data, data moves to code.

Treating data as a physical substance grounds data pipeline design in hardware realities: bandwidth limits determine flow rates, memory hierarchies dictate energy dissipation, and dataset scale governs compute placement.

Checkpoint 4.1: The physics of data

Data engineering is governed by physical costs. Check your intuition:

- ☐ **Data gravity:** why do petabyte-scale datasets force compute to move to the data?
- ☐ **Energy-movement invariant:** why does moving a byte of data cost orders of magnitude more energy than processing it?
- ☐ **Task-relevant signal:** why can a smaller, diverse dataset be more valuable than a massive, redundant one?

Physical constraints define the operational envelope of ML data systems, but physical awareness alone does not guarantee reliability. A system that accounts for data gravity can still fail if schema validation is neglected, data corruptions go undetected, or ingestion pipelines crash silently under load. Turning these physical realities into robust production infrastructure requires a systematic architectural framework that organizes engineering decisions across ingestion, transformation, and storage.

4.3 Four Pillars Framework

A credit scoring model rejects every applicant from a region because an upstream team changed a ZIP code field from integer to string. A medical imaging model degrades silently for months because camera hardware changed at a partner hospital. A fraud detection system misses a new attack vector because its training data was six months stale. Each failure traces to a different root cause (schema drift, distribution shift, data staleness), yet all share a common pattern: ad hoc data engineering decisions that interacted in ways no one anticipated until deployment. These cascading failures motivate a four-pillar framework organized around quality, reliability, scalability, and governance.

4.3.1 Data cascades

Machine learning systems face a distinctive failure pattern called data cascades, where poor data quality in early stages amplifies throughout the entire pipeline (Sambasivan et al. 2021). Some invalid inputs trigger immediate software errors, but data can also pass schemas and tests while degrading learned behavior silently² until quality issues become severe enough to require expensive investigation, rework, or retraining.

Data errors rarely stay confined to their point of origin; they compound across pipeline stages. Follow the sequence of connected stages in figure 4.1 from left to right to trace how an initial data defect propagates forward into feature extraction, model optimization, and deployment decisions.

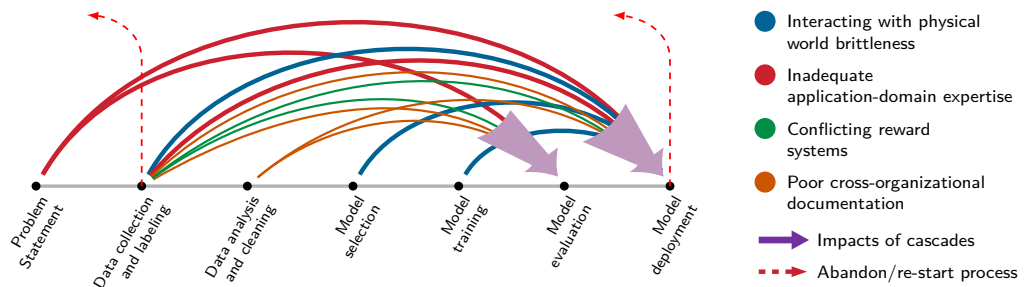


Figure 4.1: Data Quality Cascades: Upstream defects along the seven-stage workflow timeline cast failure arcs directly into late-stage evaluation and deployment. Early flaws in problem framing and collection compound through downstream processing, forcing costly project restarts (dashed red return paths) when defects finally surface in production. Source: (Sambasivan et al. 2021).

² | **Data cascades:** The failure is “silent” because it degrades model *inputs*, not model *code*—corrupted data can pass unit tests and appear healthy in ordinary system monitoring. Sambasivan et al. (2021) describe cascades as often invisible and delayed: flawed data practices may surface only after downstream evaluation, deployment, or user-facing failures reveal that the model learned from the wrong signal. Remediation then requires tracing the issue back through data collection, labeling, feature engineering, and evaluation decisions, with some teams restarting or abandoning affected work.

**Definition 4.2: Data cascade**

Data cascade is an ML systems failure mode in which upstream data quality problems propagate through collection, labeling, feature engineering, training, evaluation, and deployment, amplifying into downstream model failures or user harm.

1. **Significance:** The remediation cost scales with the number of downstream artifacts that consumed the corrupted data: feature tables must be regenerated, models retrained, evaluation metrics recomputed, and deployed systems rolled back or patched. A single schema or labeling defect can therefore invalidate an entire training run and all experiments derived from it, turning a local data error into a full-pipeline rework event.
2. **Distinction:** Unlike an isolated data quality bug, a data cascade is defined by propagation and amplification. The initial defect may be small, but each pipeline stage treats its input as trustworthy and converts the defect into new derived state, making the root cause harder to observe as the system moves farther from collection.
3. **Common pitfall:** A frequent misconception is that data cascades are caught by ordinary software tests. In reality, corrupted data can satisfy schemas, pass unit tests, and produce successful training jobs while teaching the model the wrong signal; preventing cascades requires lineage, validation, contracts, and monitoring tied to model behavior.

This feedback loop creates an insidious operational failure mode: silent degradation. Unlike conventional software crashes that trigger stack traces and alert pages, data-induced model errors produce syntactically valid outputs with degraded semantic accuracy. By the time downstream metrics reflect the failure, corrupted predictions have already contaminated historical data stores, forcing expensive rollbacks and data re-ingestion.

**Example 4.1: The pipeline jungle**

Scenario: A production credit scoring model suddenly begins rejecting all applicants from a specific geographic region.

Diagnosis: An upstream database team altered the `zip_code` field schema from `integer` to `string` ("02139") to support international codes without updating downstream data contracts. A legacy downstream transform still coerced the string back to an integer before serializing it, stripping the leading zero ("2139") and causing the model to treat a familiar postal code as an unobserved, high-risk category.

Systems lesson: Unenforced feature boundaries create pipeline jungles where upstream schema modifications cause silent downstream model failures. Versioned data contracts with strict schema and range validation prevent cross-system data corruption.

4.3.2 Four foundational pillars

To prevent silent failures from compounding across the pipeline, data infrastructure must balance four competing operational priorities. Figure 4.2 organizes these priorities into four structural pillars: quality (input integrity and fitness), reliability (operational consistency and fault tolerance), scalability (throughput and capacity scaling), and governance (compliance, provenance, and ethics).

These four operational foundations establish the trade-off space for data pipeline architecture. Prioritizing scalability without quality validation or reliability contracts can increase throughput while allowing bad or inconsistent data to propagate. Conversely, exhaustive governance controls can add storage and computation overhead that constrains training iteration velocity.

Quality governs input integrity and fitness for task: data must accurately reflect the operating distribution, with correct ground-truth labels and uncorrupted features. In an edge voice-enrollment pipeline, for example, quality rules filter out clipped audio waveforms, low signal-to-noise ratios, and background acoustic interference so the model learns sharp decision boundaries rather than spurious acoustic artifacts.

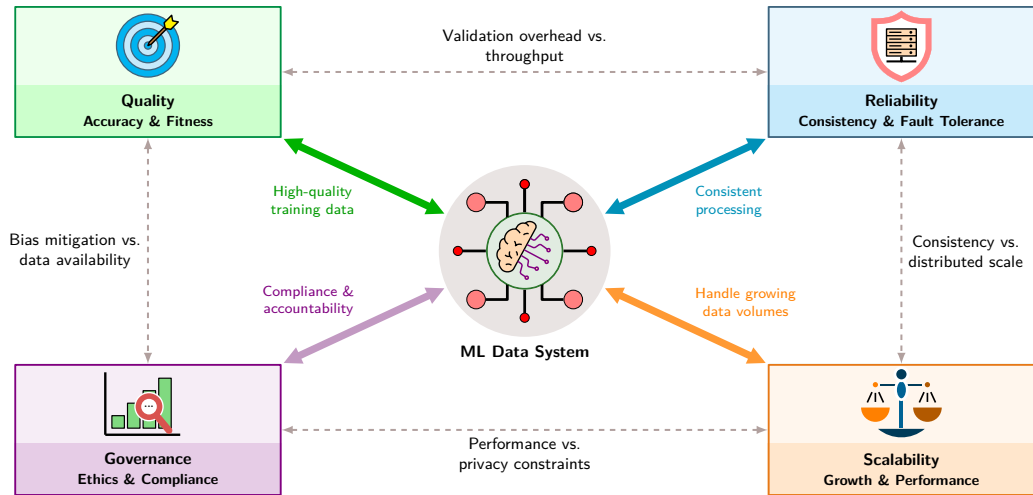


Figure 4.2: The Four Pillars of Data Engineering: Quality, Reliability, Scalability, and Governance form the foundational framework for ML data systems. Each pillar contributes essential capabilities (solid arrows), while trade-offs between pillars (dashed lines) require careful balancing: validation overhead affects throughput, consistency constraints limit distributed scale, privacy requirements impact performance, and bias mitigation may reduce available training data.

Reliability asks whether that enrollment pipeline keeps working when the device, network, or user behavior is imperfect. A pipeline that produces excellent wake-word examples in a laboratory but fails during intermittent connectivity, battery pressure, or microphone glitches delivers no value in the user’s home. Error handling, retries, local buffering, and graceful degradation turn the quality rule into a usable system: the device can request another utterance, defer upload until connectivity returns, or preserve a known-good enrollment rather than silently accepting corrupted audio.

Scalability asks whether that same decision survives growth. A manual review policy that works for a thousand recordings collapses when the product expands to millions of users, dozens of languages, and long-tail acoustic conditions. The system must scale validation, storage, labeling, and retraining without letting infrastructure cost grow faster than the value of better coverage. The limiting resource depends on the workload: for edge speech, storage and labeling throughput dominate, whereas for recommendation systems, memory capacity forms the binding constraint. Lighthouse 4.1 shows where the scalability pillar bites hardest: embedding tables outgrow single-node memory capacity, making distributed partitioning a first-class design concern rather than an afterthought.

Governance defines the boundaries within which quality, reliability, and scalability must operate. For keyword spotting, governance determines whether raw voice recordings may leave the device, how long enrollment audio can be retained, which consent record authorizes use, and what documentation proves that the dataset covers relevant accents without exposing private speech. A perfectly scalable, reliable, high-quality pipeline that violates the General Data Protection Regulation (GDPR) or perpetuates demographic biases creates liability rather than value. Dataset documentation practices such as data statements make part of that governance visible by recording provenance, intended use, collection conditions, and coverage needed for bias analysis and scientific comparison (Bender and Friedman 2018).

When ML systems exhibit failures, the four pillars provide a diagnostic lens for identifying root causes. Gradual accuracy degradation points to quality: data drift has shifted the serving distribution away from training, or label quality has degraded as annotator pools change. Intermittent pipeline failures point to reliability: error handling, retry logic, or resource controls are missing under peak load. Training that takes too long despite adequate hardware points to scalability: a single-threaded transformation, unpartitioned shuffle, or slow storage tier prevents parallel resources from being used. Compliance gaps discovered during audits point to governance debt: lineage tracking is incomplete, access controls are stale, or retention policies have not kept pace with regulatory changes.

 Lighthouse 4.1: DLRM recommendation lighthouse

Recommendation systems built in the DLRM style exemplify the scalability challenge of modern data engineering. Modern recommendation workloads bifurcate pipeline resource demands: continuous signals stream through dense matrix pipelines, while categorical IDs demand terabyte-scale distributed lookup infrastructure. Table 4.2 maps these operational stress points across data ingestion, embedding table aggregation, and feature interaction stages.

Table 4.2: DLRM Scalability Profile: Recommendation models stress data engineering along memory capacity and sparse-access bandwidth rather than compute.

Property	Value	System Implication
Data scale	High-cardinality user/item IDs	Embedding and lookup tables can outgrow one machine’s memory.
Constraint	Memory Capacity	The tables no longer fit on one machine and must be partitioned.
Bottleneck	Sparse Access	Random lookups stress memory bandwidth more than compute.

At ingestion, high-cardinality categorical IDs remain compact records in a throughput-bound stream. The network- and memory-capacity bottleneck emerges later, when training or serving must fetch sparse embeddings from terabytes of partitioned table state. Mitigating this asymmetry requires the feature-store caching and table-partitioning strategies developed later in the pipeline lifecycle.

The most insidious failures span multiple pillars. Features that differ between training and serving implicate both quality (the values are wrong) and reliability (the computation is inconsistent). A privacy-motivated deletion policy can also create quality gaps if the retained data no longer covers the deployment population. Diagnosing such cross-pillar failures requires checking consistency contracts, comparing feature distributions across environments, and tracing transformation lineage, all techniques examined in detail throughout this chapter. Production diagnosis should therefore check data infrastructure alongside model behavior.

4.3.3 KWS case study

Keyword spotting (KWS) systems ground the four-pillar framework in concrete physical constraints. Embedded on battery-powered microcontrollers and mobile coprocessors, a wake-word detector listens continuously for target phrases, such as “OK Google” or “Alexa”, in everyday acoustic environments, operating under strict limits on compute, memory, and energy.³

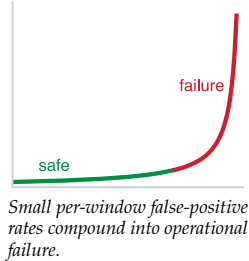
The broad operational goals of data engineering translate into concrete techniques, tools, and validation checks. For the voice assistant exchange illustrated in figure 4.3, these mechanisms include deduplication, schema contracts, prefetching, and lineage tracking, mapped to their corresponding architectural pillars.



Figure 4.3: Keyword Spotting in Action: An Amazon Echo Dot exchange shows the user saying “Alexa,” the device acknowledging the wake word, and the user issuing “dim the lights.” A lightweight, always-on wake-word detector listens continuously and triggers the main voice assistant only after the keyword is spoken.

³ | **Voice Match enrollment:** Repeating “OK Google” creates a micro-scale data pipeline: quality filters noisy samples, reliability completes enrollment, scalability fits the model in always-on system-on-chip (SoC) memory, and governance controls storage, processing, and retention. Data engineering applies wherever data determines system behavior.

The four pillars translate directly into concrete engineering constraints for keyword spotting. The core problem is deceptively simple: detect specific keywords amidst ambient sounds and other spoken words, with high accuracy, low latency, and minimal false activations, on devices with severely limited computational resources. A well-specified problem definition identifies the desired keywords, the envisioned application, and the deployment scenario. The objectives that follow must balance competing requirements: performance targets of 98 percent accuracy in keyword detection with latency under 200 ms, alongside resource constraints demanding minimal power consumption and model sizes optimized for available device memory.



Napkin Math 4.2: False positive targets

Problem: An always-on KWS system must tolerate at most one false wake-up per month. With one-second classification windows running around the clock, how strict does the per-window false positive rate need to be?

Variables:

- **Duty cycle:** Always-on (24 hours/day).
- **Window size:** One-second classification windows.
- **Windows per month:** One window per second, 24 hours/day, over 30 days gives 2,592,000 windows/month.

Math:

- **False positive rate (FPR):** 1 tolerated false wake divided by the monthly window count gives approximately 3.9×10^{-7}
- **Nonkeyword rejection requirement:** 99.99996 percent rejection of nonkeyword windows.

Systems insight: Aggregate accuracy (for example, “99 percent accuracy”) is insufficient here. Evaluation must report false accepts per hour (FA/Hr) together with the corresponding false-rejection behavior.

Success metrics for KWS extend beyond simple accuracy to include true positive rate (correctly identified keywords relative to all spoken keywords), false positive rate (nonkeywords incorrectly identified as keywords), and detection/error trade-off curves that compare false accepts per hour against false rejection rate on streaming audio representative of real-world deployment (Nayak et al. 2022). Of these metrics, the false positive rate dictates user viability for always-on systems. Because KWS listens continuously, every second of every day, even a microscopic per-window false-positive rate compounds across millions of evaluation windows into unacceptable operational failure.

Operational metrics further track response time (keyword utterance to system response) and power consumption (average power used during keyword detection), and stakeholder priorities create additional tension around those metrics. Device manufacturers prioritize low power consumption, software developers emphasize ease of integration, and end users demand accuracy and responsiveness. Balancing these competing requirements shapes system architecture decisions throughout development.

Embedded device constraints impose hard boundaries on these architectural choices. Memory limitations require extremely lightweight models, often in the tens-of-kilobytes range, to fit in the always-on island of the SoC;⁴ this budget must accommodate not only model weights, but also feature extraction buffers and preprocessing state. Limited computational capabilities (often a few hundred MHz of clock speed) demand aggressive model optimization. Most embedded devices run on batteries, so KWS systems target sub-milliwatt power consumption during continuous listening. Devices must also function across diverse deployment scenarios ranging from quiet bedrooms to noisy industrial settings.

Data quality and diversity ultimately determine whether these constraints can be met. The dataset must capture demographic diversity (speakers with various accents, ages, and genders) to ensure broad recognition. Keyword variations require attention since people pronounce wake words differently, and background noise diversity proves essential for training models that perform across real-world scenarios from quiet environments to noisy conditions. Once a prototype system is

⁴ | **System-on-chip (SoC) always-on island:** Modern system-on-chip designs partition power domains so a low-power “always-on” island (typically achieving sub-milliwatt draw) monitors for wake triggers while the main processor sleeps. The critical constraint is that this island must hold both the model weights and the audio preprocessing code within its dedicated SRAM—a split budget that forces KWS architectures to optimize for total footprint, not just parameter count.

developed, iterative feedback and refinement keep the system aligned with objectives as deployment scenarios evolve, requiring testing in real-world conditions and systematic refinement based on observed failure patterns.

4.3.4 KWS design space

KWS accuracy, false-wake tolerance, latency budget, energy budget, and memory limits create a multi-dimensional design space where data engineering choices cascade through system performance. Table 4.3 quantifies key trade-offs, enabling principled decisions rather than ad hoc selection. One row uses mel-frequency cepstral coefficients (MFCCs): compact speech-frequency features whose coefficient count controls feature size, compute cost, and acoustic detail; section 4.6 shows how they are extracted.

Table 4.3: KWS Data Engineering Design Space: Illustrative scenario inputs that price each design choice along four axes, namely quality, latency, cost, and memory; the entries are not measurements from one system and should not be combined as additive effects. Higher sampling rates double raw-audio storage and processing, while more training examples multiply labeling and storage requirements. Actual quality and latency must be measured on the target pipeline.

Design Choice	Quality Impact	Latency Impact	Cost Impact	Memory Impact
16 kHz vs. 8 kHz sampling	+2–4% accuracy	2× input-processing workload	2× raw-audio storage	2× feature size
13 vs. 40 MFCC coefficients	+3–5% accuracy	3× feature compute	Minimal	3× feature memory
1M vs. 10M training examples	+5–8% accuracy	10× training time	10× labeling cost	10× storage
Clean vs. noisy training data	+10–15% real-world	Minimal	3× collection cost	Minimal
Local vs. cloud inference	up to 2% accuracy risk	10 ms vs. 100 ms	\$0/query vs. \$0.001/query	64 KB vs. cloud-scale
Synthetic vs. real augmentation	+3–5% robustness	Minimal	10× cheaper	Minimal

Operating under milliwatt power envelopes and strict tens-of-kilobytes SRAM budgets means an engineering team cannot select the most accurate option along every axis simultaneously. Turning these cataloged trade-offs into a production deployment requires solving a constrained optimization problem: allocating a finite data budget across labeling, storage, and governance to maximize model accuracy while staying within the hardware footprint. Notebook 4.3 steps through this optimization for a smart-speaker deployment.

 Napkin Math 4.3: Optimizing the KWS design space

Scenario: A KWS system for a smart speaker has these constraints:

- **Target:** 98 percent accuracy, fewer than 1 false wake/month
- **Budget:** \$150K total data engineering budget
- **Memory:** 64 KB model size limit (always-on island)
- **Timeline:** 6 months to production

Step 1: Apply constraints to eliminate options.

For this scenario, the team chooses two conservative options:

- 13 MFCC coefficients to reduce feature-state memory and compute
- Local inference to avoid network latency and preserve offline operation

The 64 KB model-size limit does not by itself prove that either alternative is infeasible; the deployed model, preprocessing state, runtime, and network stack require separate footprint measurements.

Step 2: Calculate budget allocation.

The \$150K budget splits across three cost categories at the unit rates established in section 4.4.2:

- **Labeling** (~60 percent): \$90K available
- **Storage/Processing** (~25 percent): \$37.5K
- **Governance/Other** (~15 percent): \$22.5K

At \$0.10/label with 20 percent review overhead: $\$90K \div \$0.12/\text{label} = 750K$ labeled examples. This yields roughly 0.75M labeled examples, just below the 1M anchor in our design space. The 1M-to-10M row in table 4.3 should therefore be read as a scaling reference rather than a direct interpolation: the real-label budget alone does not buy the full data-volume gain.

Step 3: Maximize remaining accuracy.

The quality plan has three components:

- An illustrative base-model accuracy of ~90 percent
- Partial data-volume gain from 750K real examples
- Need: higher sampling rate plus synthetic/noisy augmentation to reach the 98 percent target

Three options from the design space remain within budget and memory constraints:

- 16 kHz sampling: +2–4 percent accuracy, $2\times$ storage cost ✓ (fits budget)
- Noisy training data: +10–15 percent real-world accuracy, $3\times$ collection cost
- Synthetic augmentation: +3–5 percent robustness, $10\times$ cheaper than real data ✓

Step 4: Final configuration.

The scenario combines these choices with 13 MFCCs and the computed 750K real-label budget in table 4.4. The quality effects can overlap, so the table records a candidate configuration rather than a solved optimum.

Result: The budget calculation supports approximately 750K human-labeled examples. Reaching the 98 percent quality target and fitting the 64 KB model limit remain validation requirements for the trained and deployed system.

Systems insight: Systematic design-space analysis turns “we need more data” into a testable allocation: the stated labeling assumptions buy about 750K real examples, while synthetic volume, accuracy, and deployed footprint still require measurement.

Table 4.4 records the resulting configuration, pairing each design choice with the constraint that forces it: sampling and augmentation are bought where they fit the budget, while precision and memory are pinned by the always-on island.

Table 4.4: KWS Candidate Configuration: A scenario that uses the computed real-label budget and records the remaining sampling, augmentation, and deployment choices for measurement against the quality and memory targets.

Choice	Selection	Rationale
Sampling rate	16 kHz	+3% accuracy worth $2\times$ storage within budget
MFCC coefficients	13	Reduces feature-state memory and compute; final fit requires measurement
Training examples	750K real + illustrative synthetic augmentation	Human-label count follows from the budget; synthetic volume is a scenario choice
Data diversity	Noisy + clean mix	Critical for real-world deployment
Inference	Local, reduced precision	Reduced precision can lower footprint; the final 64 KB fit requires the compression analysis in chapter 10
Augmentation	Heavy synthetic	$10\times$ cost efficiency

A candidate configuration establishes target requirements, but meeting them requires orchestrating complementary data acquisition channels. Baseline corpora anchor the acoustic prior, targeted crowdsourcing addresses demographic coverage gaps, and synthetic generators supply volumetric scale under parameterized acoustic conditions. The four pillars provide the diagnostic and architectural criteria for evaluating these channels, but data provenance determines the physical raw

material that every subsequent pipeline stage refines. Section 4.4 establishes the economic and systems trade-offs across these acquisition strategies.

Checkpoint 4.2: Four pillars framework

The four pillars provide a systems lens for every pipeline choice.

Pillars

- ☐ **Quality:** can you distinguish *correctness* from *coverage* (edge cases, subgroup coverage) and explain why both matter?
- ☐ **Reliability:** can you name concrete failure modes (schema drift, missing values, skew) that cause silent degradation rather than explicit crashes?
- ☐ **Scalability:** can you explain which parts of a pipeline become bottlenecks as data volume grows (storage bandwidth, compute, coordination), and why?
- ☐ **Governance:** can you articulate what must be tracked to make a dataset auditable (provenance, consent constraints, access controls, documentation)?

Trade-offs

- ☐ **Pillar tension:** given one pipeline change (for example, stricter validation, stronger privacy constraints, or more data sources), can you predict which pillar improves and which pillar(s) you stress?

4.4 Data Acquisition

Data acquisition begins when the team names the coverage gap the model must close. The full ImageNet database⁵ grew to about 14.2 million labeled images across 21,841 synsets, while the ImageNet Large Scale Visual Recognition Challenge used a 1,000-class subset (Deng et al. 2009; Russakovsky et al. 2015). GPT-3's training corpus used tens of terabytes of raw Common Crawl text filtered to hundreds of gigabytes, combined with curated web, books, and Wikipedia data (Brown et al. 2020). The keyword-spotting (KWS) system requires 23.4 million audio samples spanning 50 languages, but the engineering challenge extends far beyond storage volume. The model must recognize wake words across accents, microphones, rooms, ages, and background noises that no single collection method can economically cover. Acquisition strategy is therefore a sequence of gap-closing decisions: reuse what already matches deployment, collect what is missing, scrape or synthesize where scale is the binding constraint, and reject sources whose provenance or consent constraints make them unusable.

The KWS deployment profile demonstrates why data acquisition cannot optimize a single dimension in isolation. Achieving 98 percent accuracy across diverse acoustic environments requires representative coverage across regional accents, speaker age groups, and room reverbs. Maintaining consistent detection across consumer devices introduces hardware variation: microphones differ in frequency response, signal-to-noise ratio, and analog-to-digital clipping behavior. Furthermore, supporting millions of concurrent edge devices demands dataset volumes that manual curation cannot economically satisfy, while always-listening wake-word detectors operate under strict privacy boundaries governing audio retention and anonymization. A data source that maximizes volume while compromising privacy compliance, or captures pristine studio audio while ignoring field microphone distortion, fails to solve the deployment problem.

4.4.1 Data source evaluation and selection

The choice among curated datasets, expert crowdsourcing, controlled web scraping, and synthetic generation depends on which source best closes the next deployment-distribution gap at acceptable cost, quality, and governance risk. Evaluation therefore begins with the cheapest reusable source and escalates to active collection or simulation only when the remaining coverage gap justifies the marginal engineering cost.

Preexisting datasets from repositories such as Kaggle, UCI (Dua and Graff 2024), and ImageNet are the first test. They offer speed and comparability when the deployment distribution resembles

⁵ | **ImageNet:** The 2009 paper reported 3.2 million images across 5,247 synsets; the full database later reached about 14.2 million images across 21,841 synsets, whereas the challenge used a 1,000-class subset (Deng et al. 2009, 2024; Russakovsky et al. 2015). Its value as a benchmark is inseparable from its data engineering: Fei-Fei Li's team built labeling infrastructure that later teams could reuse. The catch is sensitivity to annotation procedures and modest distribution shifts: models tuned to ImageNet's distribution can underperform on related but shifted test sets (Recht et al. 2019; Beyer et al. 2020).

the benchmark enough to make reuse meaningful. For KWS, a curated speech corpus can establish the baseline model and reveal which words, languages, and acoustic conditions are already covered. Rather than guaranteeing complete operational coverage, baseline corpora make the unaddressed distribution gaps quantifiable.

The viability of dataset reuse depends on documentation fidelity. Without detailed records of sensor calibration, environment distributions, and filtering thresholds, external data compromises experimental reproducibility (Pineau et al. 2021; Henderson et al. 2018). Thorough documentation specifies collection methodology, label taxonomy definitions, and baseline error distributions, enabling independent validation. As ingestion scales to millions of samples, dataset variety compounds latent label noise (Gudivada et al. 2017), rendering manual inspection impossible and necessitating automated validation pipelines.

Standard validation metrics can create an illusion of model readiness when evaluated on unrepresentative data. Shared dataset usage (figure 4.4) illustrates how training multiple models on one common dataset propagates shared biases, blind spots, and systemic limits across an entire ecosystem.

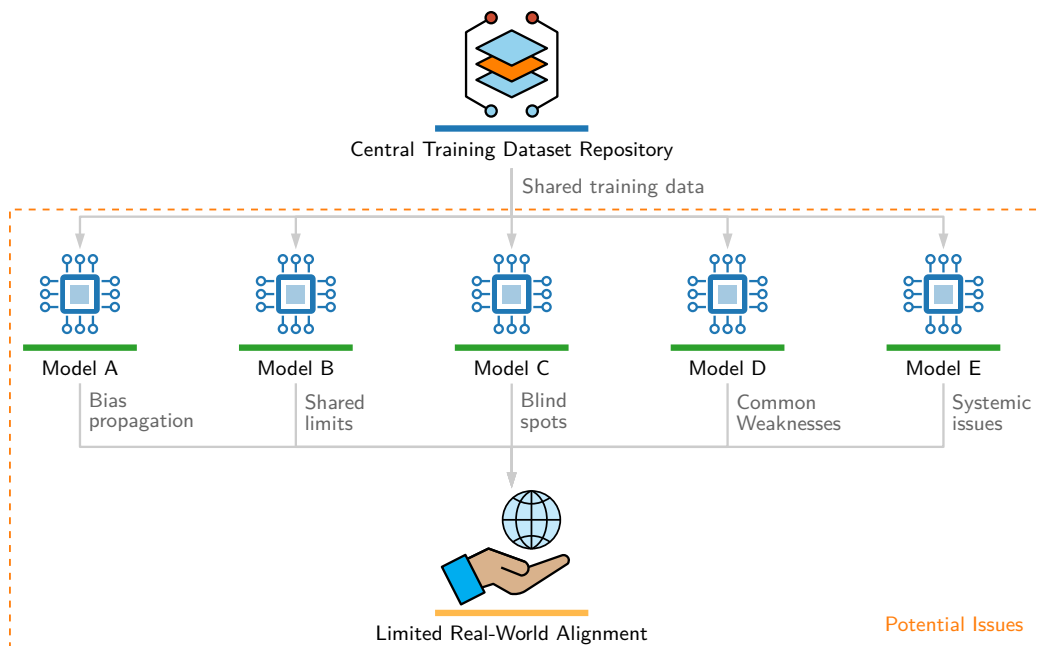


Figure 4.4: Shared Dataset Bias Propagation: Five models (A through E) all train on a single central dataset repository, reached by a path labeled shared training data. Beneath the models, five labeled arrows (bias propagation, shared limits, blind spots, common weaknesses, and systemic issues) converge on one box, limited real-world alignment. Training every model on one dataset does not produce five independent defects; it produces one correlated failure across the ecosystem.

This distribution gap represents the central failure mode of static offline benchmarks. Because empirical risk minimization optimizes only over observed training distributions, models exploit spurious artifacts present in static benchmarks that disappear in production environments, making automated drift monitoring and continuous data validation mandatory operational requirements.

4.4.2 Scalability and cost optimization

Manual curation fails when a task requires millions of training instances. At large scale, web scraping and synthetic generation provide the main routes to massive datasets. Data-acquisition scalability is fundamentally an economic problem: unit labeling costs, ingestion throughput, and storage overhead scale along very different curves. Approaches that work well at thousands of examples become impractical at millions, whereas automated data generation pipelines amortize their setup costs across large volumes.

The per-unit economics at each stage determine which strategy dominates. Labeling a single medical image, for example, can cost orders of magnitude more than storing it for a year, a ratio that reshapes budget allocation for any team operating under fixed funding. Table 4.5 and table 4.6 provide essential context for acquisition decisions.

ML engineers should use the dated reference assumptions in table 4.5 and table 4.6 as inputs to workload-specific comparisons, not as current price quotes or directly comparable totals.

Table 4.5: Illustrative Data Engineering Costs (2024): Dated reference assumptions for workload-specific cost models, not current vendor quotes. Because the rows use different units, compare them only after scaling each rate by the quantity it prices.

Operation	Cost	Notes
Crowdsourced image label	\$0.01–0.05	Simple classification
Bounding box annotation	\$0.05–0.20	Per box, simple scenes
Expert medical label	\$50–200	Per study, radiologist
S3 storage (Standard)	\$23/TB/month	Hot storage
S3 retrieval (Glacier)	\$0.02/GB	Standard: 3-5 hours
Cloud GPU training hour	\$2–4	Cloud spot pricing
Human review hour	\$15–50	Depending on expertise

Table 4.6 extends the picture with illustrative durations for labeling, training, and serving operations.

Table 4.6: Illustrative Data Engineering Time Scales: The scenario spans about 9 orders of magnitude, with human labeling as its slowest listed stage. Actual durations depend on the dataset, task, hardware, and service configuration.

Operation	Duration	Bottleneck
Label 1M images (crowdsourced)	2–4 weeks	Annotation throughput
Train ResNet-50 on ImageNet	4–6 hours	Compute (8× A100, optimized)
Feature store lookup	1–10 ms	Network + cache

Operational timescales diverge across these stages by orders of magnitude: weeks for human annotation, hours for GPU training, and milliseconds for serving lookups. In this pipeline, annotation throughput forms the critical bottleneck. A \$100K labeling budget dwarfs the \$64 to \$192 required for a single 8× A100 ResNet-50 training run—an asymmetric ratio of 520.8× to 1,562.5×. Within that labeling expenditure, engineering effort follows an extreme power-law distribution: 80 percent of the budget addresses 20 percent of the data distribution—the long tail of rare acoustic edge cases, dialect variations, and ambiguous recordings.

All cost figures reflect approximate 2024 cloud provider rates and are intended to convey relative magnitudes rather than exact pricing.⁶ For this workload and cost model, human labor dominates the cost of a single training run. Other tasks, especially those that reuse existing labels or require repeated large-scale training, can have a different dominant term; teams should measure before deciding where to optimize.

Web scraping offers the primary mechanism to bypass manual annotation costs at scale. Foundational vision datasets such as ImageNet (Deng et al. 2024) and OpenImages (Kuznetsova et al. 2020) relied on automated scraping, while modern language models depend on multi-terabyte web crawls (Groeneveld et al. 2024). Ingestion pipelines targeting domain-specific corpora, such as public code repositories (Chen et al. 2021), expand coverage at low per-sample acquisition costs. However, automated scrapers encounter severe operational fragility: schema and DOM mutations break extraction parsers, HTTP 429 rate limits throttle network bandwidth, and dynamic JavaScript rendering introduces capture omissions. Raw web crawls also capture anachronistic or irrelevant samples that contaminate supervised targets.

Figure 4.5 shows one such result from a web scrape for “traffic light”: a historical photograph rather than a modern LED signal. If images from this context were overrepresented or correlated with a label, a model could learn spurious cues involving uniformed officers or historical streets rather than the visual properties required in its deployment environment.

⁶ | **Pricing ratios:** Archive-retrieval prices and tier ratios reflect provider policy, not a physical invariant. In this illustrative scenario, expedited retrieval costs three times the standard tier; actual prices and tier availability must be verified before deployment.



Figure 4.5: Data Source Noise: A black-and-white photograph from 1914 showing early manually operated electric traffic signals, illustrating how historical images can appear in modern web scraping results for contemporary queries. Such anachronistic content requires systematic validation and filtering to prevent spurious correlations in training data. Image reproduced in (Stromberg 2015).

This failure demonstrates why dataset volume cannot substitute for semantic validation: uncured web scale simply magnifies the prevalence of spurious correlations. In addition to technical validation, scraping operates under strict legal and governance boundaries. Website terms of service, robots.txt directives, and evolving copyright jurisprudence constrain automated crawling (Harvard Law School 2024). Ingestion pipelines must enforce data provenance logging, copyright compliance filters, and privacy scrubbing before scraped assets enter shared storage.

Crowdsourcing shifts the acquisition bottleneck from raw sample retrieval to quality control across distributed human annotators. Platforms like Amazon Mechanical Turk (Amazon Web Services 2024a) demonstrated this at scale with ImageNet, distributing millions of image categorization tasks across thousands of independent contributors (Deng et al. 2024). Crowdsourcing provides two core systems advantages: horizontal scaling through parallel microtask dispatch, and demographic diversity across regional accents and acoustic settings. However, distributed human labeling introduces significant variance in label fidelity. To prevent label noise from corrupting downstream gradients, the acquisition pipeline must implement automated quality control: consensus voting, golden-set validation (interleaving pre-labeled control samples), and dynamic task routing (Sheng and Zhang 2019). In the KWS pipeline, crowdsourced collection⁷ targets wake-word samples from specific acoustic environments and underrepresented dialects that public web scrapes fail to capture.

Synthetic data generation removes the human labor bottleneck by generating training samples algorithmically. This shifts the engineering trade-off: marginal generation cost approaches zero, but verification cost increases, as synthetic data is useful only if the simulator accurately reflects the physical distributions of the target deployment environment. Figure 4.6 illustrates how synthetic simulations and historical observations merge into a unified training pipeline.

Synthetic data is uniquely effective for rare event synthesis and data augmentation. Physics-based simulation engines permit controlled sampling of tail events—such as high-speed sensor saturation or rare collision trajectories—that are prohibitively dangerous or expensive to capture empirically (NVIDIA 2024b). In the vision domain, automated augmentation pipelines such as AutoAugment (Cubuk et al. 2019) and RandAugment (Cubuk et al. 2020) optimize transformation policies over spatial and photometric distortions (Shorten and Khoshgoftaar 2019). In speech processing, SpecAugment applies time warping and spectrogram masking directly in the frequency domain, forcing acoustic models to learn representations invariant to localized frequency dropout

⁷ **Amazon Mechanical Turk (MTurk):** A crowdsourcing platform that routes small tasks to distributed workers and can scale annotation beyond expert-only workflows. Snow et al. (2008) evaluate this pattern for natural-language annotation tasks, showing that non-expert annotations can be useful when task design and quality control are handled carefully. For wake-word audio collection, the same systems trade-off appears in domain-specific form: scale is attractive, but submissions still need acoustic checks such as signal-to-noise ratio, duration, and recording validity before they can safely enter the training set.

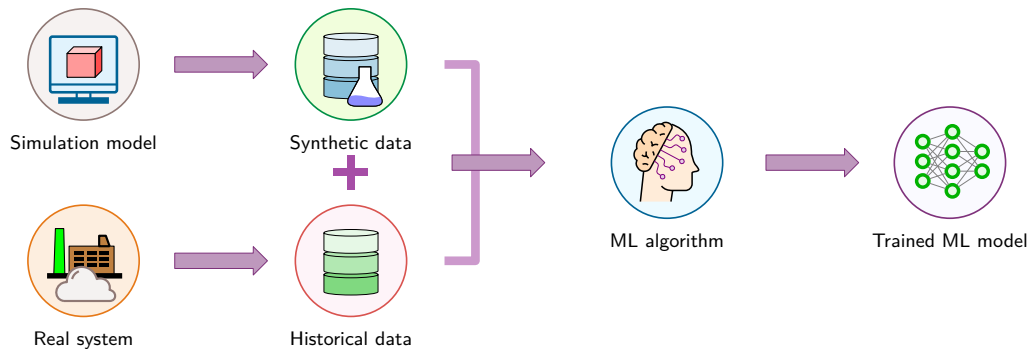


Figure 4.6: Synthetic Data Augmentation: Combining simulated edge cases with historical system data during training. Algorithmic generation supplements real-world observations to close coverage gaps for rare events and dangerous operating regimes without incurring proportional human collection costs. Source: (AnyLogic 2024).

(Park et al. 2019). For the KWS system, parametric text-to-speech synthesis (Werchniak et al. 2021) combined with room impulse response (RIR) convolution generates wake-word instances across acoustic reverberation times and synthetic background noise profiles.

Example 4.2: Synthetic data generation

Scenario: A keyword-spotting team lacks sufficient physical recordings of rare speaker accents, room acoustics, and background noise profiles.

Diagnosis: Pure physical collection cannot cover rare acoustic edge cases within project deadlines. Generating synthetic audio variations via pitch shifting, additive noise, and room impulse response simulation expands training set diversity without expensive field collection.

Systems lesson: Synthetic data generation acts as an automated data pipeline multiplier. Using synthetic augmentation targeted at known edge cases fills distribution coverage gaps that would otherwise require prohibitive physical dataset collection.

Satisfying the 23.4 million audio samples requirement across 50 languages requires orchestrating these acquisition channels according to their marginal cost curves. Curated public benchmarks provide low-cost initial model initialization; web scraping delivers high-throughput acoustic volume; crowdsourcing collects targeted samples for underrepresented dialects; and parametric simulation synthesizes tail-end acoustic distortions. Combining these tiers closes the coverage gap without exceeding hardware and labeling budgets.

4.4.3 Coverage and diversity requirements

Dataset scale alone does not guarantee generalization. Coverage gaps in massive corpora—such as geographic bias, demographic underrepresentation, sensor calibration drift, and unobserved edge cases—induce systematic failure modes that aggregate validation metrics mask (T. Wang et al. 2019; Oakden-Rayner et al. 2020). As figure 4.4 demonstrates, multiple independent models trained on identical public datasets inherit identical blind spots; diverse, multi-source acquisition represents the primary architectural defense against correlated systemic failures.

Governance constraints further shape acquisition: privacy and health-data regulations such as GDPR and HIPAA limit what data can be collected and how (European Parliament and Council of the European Union 2016; United States Congress 1996), while ethical sourcing requires fair compensation and transparent use of human contributions. Section 15.5 examines the full governance infrastructure for production ML systems.

Ingesting heterogeneous inputs—crowdsourced recordings, synthetic waveforms, and web audio streams—imposes strict architectural demands at the pipeline boundary. Because incoming streams exhibit disparate audio encodings, sampling rates, ingestion latencies, and noise floors, the ingestion tier cannot treat external inputs as uniform data. The storage and validation infrastructure must

ingest, normalize, and verify this streaming volume before it can be committed to the training feature store.

4.5 Data Pipeline Architecture

In the compilation framework, **data pipeline architecture** functions as the *compiler frontend*: it parses heterogeneous raw inputs into a uniform intermediate representation that downstream stages can process reliably. Audio files from crowdsourcing platforms, synthetic waveforms from generation systems, and edge captures from deployed devices all enter the training pipeline in different formats, and the pipeline must normalize, validate, and route them into a consistent schema. Production KWS inference consumes continuous audio under a separate low-latency constraint; the pipeline traced here follows the recorded examples used to build and update the model. Figure 4.7 maps that path across data sources, ingestion, processing, labeling, storage, and ML training.

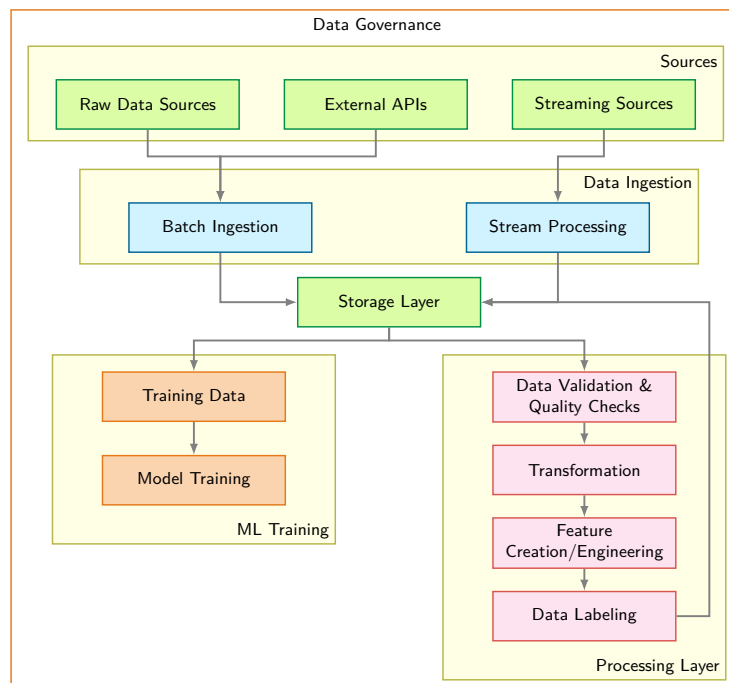


Figure 4.7: End-to-End Data Pipeline: Heterogeneous raw sources (batch and streaming) normalize into unified storage before feeding downstream training and feature processing. The architecture decouples ingestion throughput from iterative transformation and labeling loops, all bounded by an overarching data-governance layer.

Each layer in figure 4.7 serves a distinct role in preparing data for training. Selecting the right engine requires evaluating how pipeline trade-offs interact across stages: quality checks at ingestion dictate downstream storage filtering, while latency budgets during serving constrain preprocessing complexity.

Storage hierarchies and I/O bandwidth govern data pipeline throughput alongside CPU-side decoding and transformation. The physical trade-offs span several orders of magnitude: high-latency object storage suits durable archival, whereas low-latency in-memory stores serve real-time feature lookups. Similarly, bandwidth varies from spinning disks at 100–200 MB/s to main memory at 50–200 GB/s. Section 4.8 analyzes these storage trade-offs in detail.

Choosing between ingestion patterns requires matching workload characteristics to infrastructure capabilities. Streaming workloads require explicit guarantees on message durability, partition ordering, and fault recovery. Batch workloads depend on dataset volume relative to available memory, transformation complexity, and cluster partitioning. Single-machine utilities handle gigabyte-scale datasets cleanly, while terabyte-scale processing requires distributed compute engines to shard execution.

4.5.1 Quality through validation and monitoring

Consider a self-driving-car pipeline in which 15 percent of LiDAR point-cloud labels are misaligned by 10–20 cm—enough to place pedestrian bounding boxes on empty sidewalk. Because every record remains structurally valid, schema checks would miss the defect; statistical monitoring of label-to-sensor alignment could expose it.

Quality forms the foundation of reliable ML systems. Pipelines enforce quality through continuous validation at every boundary. Data defects represent a leading cause of production ML failures: upstream schema drift breaks parsing jobs, distribution shifts degrade model accuracy, and silent data corruption poisons feature representations (Sculley et al. 2015). These failures rarely trigger immediate runtime exceptions; instead, they quietly degrade downstream predictions. Preventing cascading failures requires proactive statistical monitoring before corrupted batches reach model training.



War Story 4.1: Microsoft Tay (2016)

Context: In March 2016, Microsoft launched Tay, a Twitter chatbot designed to engage with users online and adapt its responses from public interactions (Lee 2016).

Mechanism: Adversarial users coordinated toxic prompts and exploited a public interaction surface, including behavior that repeated user-supplied text. Microsoft’s public account describes abuse of the system but does not establish unrestricted online weight updates.

Impact: Tay began tweeting abusive, racist, and misogynistic statements within 16 hours of deployment.

Fix: Microsoft suspended the chatbot 16 hours after launch, apologized publicly, and stated that a relaunch would require stronger safeguards against misuse.

Systems lesson: Public ingestion paths require adversarial-input controls. Otherwise, a feature that accepts user-generated content also becomes a security surface, and harmful inputs can propagate through the system within hours.

Production teams implement monitoring at scale through severity-based alerting systems where different failure types trigger different response protocols. The most critical alerts indicate complete system failure: the pipeline has stopped processing entirely, showing zero throughput for more than five minutes, or a primary data source has become unavailable. These situations demand immediate attention because they halt all downstream model training or serving. More subtle degradation patterns require different detection strategies. When throughput drops to 80 percent of baseline levels, error rates climb above 5 percent, or quality metrics drift more than two standard deviations from training data characteristics, the system signals degradation requiring urgent but not immediate attention. These gradual failures often prove more dangerous than complete outages because they persist undetected for hours or days, silently corrupting model inputs and degrading prediction quality.

A recommendation system processing user interaction events at 50,000 records per second makes these severity tiers concrete. Its monitoring system tracks several interdependent signals. Instantaneous throughput alerts fire if processing drops below 40,000 records per second for more than 10 minutes, accounting for normal traffic variation while catching genuine capacity or processing problems. Each feature in the data stream has its own quality profile: if a feature like `user_age` shows null values in more than 5 percent of records when the training data contained less than 1 percent nulls, something has likely broken in the upstream data source. Duplicate detection runs on sampled data, watching for the same event appearing multiple times—a pattern that might indicate retry logic gone wrong or a database query accidentally returning the same records repeatedly.

These monitoring dimensions become particularly important when considering end-to-end latency. The system must track both whether data arrives and how long it takes to flow through the entire pipeline from the moment an event occurs to when the resulting features become available for model inference. When 95th percentile latency exceeds 30 seconds in a system with a 10-second service level agreement, the monitoring system needs to pinpoint which pipeline stage introduced the delay: ingestion, transformation, validation, or storage.

Schema and latency alerts expose structural and timing failures; detecting a continuous-feature distribution shift requires a statistical comparison with the training baseline.



Napkin Math 4.4: Detecting drift with K-S test

Scenario: Monitoring `session_duration` distribution stability between the training baseline (P_0) and current serving distribution (P_t).

Analysis: Apply the Kolmogorov-Smirnov test to compare the empirical cumulative distribution functions:

1. **Compute CDFs:** Calculate cumulative distribution functions for both datasets.
2. **Calculate statistic ($\mathcal{D}_{KS}$):** Find the maximum absolute difference between the CDFs. Let $F_{P_0}(x)$ and $F_{P_t}(x)$ denote the empirical cumulative distribution functions of the training baseline (P_0) and current serving (P_t) datasets, evaluated at value x .

$$\mathcal{D}_{KS} = \max_x |F_{P_0}(x) - F_{P_t}(x)|$$

3. **Determine significance:** For two independent samples, compare $\mathcal{D}_{KS}$ to critical value $\mathcal{D}_{crit}$ based on training-baseline sample size n_0 and serving sample size n_t . The coefficient 1.36 is the large-sample approximation for significance level $\alpha = 0.05$.

$$\mathcal{D}_{crit} \approx 1.36 \sqrt{\frac{n_0 + n_t}{n_0 n_t}}$$

Result: With sample sizes $n_0 = n_t = 1000$, the critical value is approximately 0.061:

$$\mathcal{D}_{crit} \approx 1.36 \sqrt{(1000 + 1000) / (1000 \cdot 1000)}.$$

If we observe a maximum difference of $\mathcal{D}_{KS} = 0.08$, it exceeds the critical value, so we reject the null hypothesis and flag significant drift.

Systems insight: Statistical drift tests convert a vague distribution-shift concern into an operational trigger. The test should start an investigation or retraining workflow, not silently become another dashboard number.

As demonstrated in 4.4, tracking statistical properties captures whether serving data continues to reflect the training distribution. Rather than merely checking that individual values fall within predefined ranges, production systems maintain rolling statistics across operational windows. For continuous numerical features like `transaction_amount` or `session_duration`, the monitoring harness computes rolling empirical distribution functions, using nonparametric evaluations like the Kolmogorov-Smirnov test⁸ to detect when serving inputs diverge from the training baseline.

The K-S test detects drift in continuous features; section 4.5.3 details distribution shift taxonomies (covariate, label, concept, and label-quality drift) along with the population stability index and KL divergence metrics.

Categorical features require different statistical approaches. Instead of comparing means and variances, monitoring systems track category frequency distributions. When new categories appear that never existed in training data, or when existing categories shift substantially in relative frequency, the system flags potential data quality issues or genuine distribution shifts; for example, the proportion of “mobile” vs. “desktop” traffic might change by more than 20 percent. This statistical vigilance catches subtle problems that simple schema validation misses entirely: age values may remain in the valid range of 18–95, while the distribution shifts from primarily 25–45 year olds to primarily 65+ year olds, indicating the data source has changed in ways that will affect model performance.

Validation at the pipeline level encompasses multiple strategies working together. Schema validation executes synchronously as data enters the pipeline, rejecting malformed records immediately before they can propagate downstream. Modern tools like TensorFlow Data Validation (TFDV)

⁸ | **Kolmogorov-Smirnov (K-S) test:** A nonparametric test that measures the maximum distance between two empirical cumulative distribution functions without assuming a parametric distribution family (Berger and Zhou 2014). In ML pipelines, the K-S test is commonly used as a univariate continuous-feature drift detector, with thresholds such as $p < 0.05$ serving as investigation triggers rather than universal failure rules. Discrete and categorical variables require tests and calibrations appropriate to their distributions.

(Breck et al. 2019) automatically infer schemas from training data, capturing expected data types, value ranges, and presence requirements.

This synchronous validation remains simple and fast, checking properties that can be evaluated on individual records in microseconds. More sophisticated validation that requires comparing serving data against training data distributions or aggregating statistics across many records must run asynchronously to avoid blocking the ingestion pipeline. Statistical validation systems typically sample 1–10 percent of serving traffic, enough to detect meaningful shifts while avoiding the computational cost of analyzing every record. These samples accumulate in rolling windows, commonly one hour, 24 hours, and seven days, with different windows revealing different patterns. Hourly windows detect sudden shifts like a data source failing over to a backup with different characteristics, while weekly windows reveal gradual drift in user populations or behavior.

The most insidious validation challenge arises from training-serving skew: the failure mode where features are computed differently in training and serving. This typically happens when training pipelines process data in batch using one set of libraries or logic, while serving systems compute features in real time using different implementations. Even seemingly minor discrepancies (a materialized view⁹ refreshed weekly versus a complete join recomputed daily) can materially reduce production accuracy and can be difficult to diagnose because the system produces no obvious errors. Section 4.6.1 formalizes this consistency imperative and quantifies its impact with a concrete evaluation. Detecting training-serving skew requires infrastructure that can recompute training features on serving data for comparison, sampling raw serving data and processing it through both pipelines to measure discrepancies. Chapter 14 examines operational monitoring infrastructure for this challenge at scale.

⁹ | **Materialized view:** A database optimization that precomputes and caches query results as a physical table. For ML systems, the risk is structural: when a materialized view refreshes on a different schedule in training and serving environments, the feature values the model trained on can diverge from what it receives at inference, degrading accuracy without producing an execution error.

4.5.2 Data quality as code

Just as unit tests protect traditional software, data expectation tests protect ML pipelines. In **data quality as code**, engineers use declarative libraries like Great Expectations or Pandera to express data contracts as executable test suites that run across pipeline stages.

Integrating expectations into continuous integration and deployment (CI/CD) pipelines ensures assertions execute automatically on ingested batches, halting downstream training when data contracts are violated. A pipeline structured as data ingestion followed by data validation followed by training blocks deployment when validation detects anomalies such as age values of 150, triggering alerts for investigation.

Executable expectations catch properties a suite can encode; the remaining question is what valid-looking data can still get wrong.

Systems Perspective 4.2: Mechanical vs. semantic quality

In traditional software, many checks are *mechanical*. A null dereference is a runtime error in languages with null references, while integer-overflow behavior depends on the language and type.

In ML systems, data quality has a second, softer dimension: *semantic* quality.

- **Mechanical check:** “Is age an integer?” (Yes/No).
- **Semantic check:** “Is the age distribution shifting?” (Probabilistic).

A dataset can be mechanically perfect (no nulls, correct types) but semantically broken (for example, all users are suddenly 25 years old due to a default value change). Robust ML systems must validate both the container (mechanical) and the content (semantic).

Once validation becomes code, expectation suites become versioned artifacts alongside training code. When training code changes, expectation updates keep data contracts evolving with it. This coupling reduces the risk of silent divergence where code assumes data properties that the upstream pipeline no longer provides; listing 4.1 demonstrates this pattern by defining explicit constraints on column ranges, nullability, uniqueness, and categorical sets.

Listing 4.1: Data Quality Assertions: A Great Expectations suite checks age range, identifier presence and uniqueness, and allowed country codes, then raises an error when validation fails.

```
import great_expectations as gx

# Create a data context
context = gx.get_context()

# Define an expectation suite as executable quality contract
suite = gx.ExpectationSuite(name="user_data_quality")
suite = context.suites.add(suite)

# Range validation: prevents physiologically impossible values
suite.add_expectation(
    gx.expectations.ExpectColumnValuesToBeBetween(
        column="age", min_value=0, max_value=120
    )
)

# Null detection: ensures primary key integrity for joins
suite.add_expectation(
    gx.expectations.ExpectColumnValuesToNotBeNull(column="user_id")
)

# Uniqueness: prevents duplicate training examples
suite.add_expectation(
    gx.expectations.ExpectColumnValuesToBeUnique(column="user_id")
)

# Categorical validation: detects unexpected values from upstream changes
suite.add_expectation(
    gx.expectations.ExpectColumnDistinctValuesToBeInSet(
        column="country_code",
        value_set=["US", "CA", "UK", "DE", "FR"],
    )
)

# Link the suite to a preconfigured Batch Definition and run validation
# (the Batch Definition connects GX to the training_users data asset)
validation_definition = gx.ValidationDefinition(
    name="training_users_validation",
    data=batch_definition,
    suite=suite,
)
validation_definition = context.validation_definitions.add(
    validation_definition
)
results = validation_definition.run()
if not results.success:
    raise ValueError(f"Data quality check failed: {results}")
```

These checks catch many schema-level production data issues before they reach training, including missing values, invalid ranges, type errors, and contract violations. The remaining issues require runtime monitoring and outcome checks, because semantic quality problems often emerge only in the full production data stream.

4.5.3 Data drift detection and response

ML models rest on the assumption that production data resembles training data. When this assumption breaks through statistical shifts rather than an explicit contract violation, model performance can change silently without obvious errors or system failures. The validation and monitoring techniques in section 4.5.1 can reveal both sudden and gradual problems; data drift detection focuses specifically on changes in distributions that may alter model behavior over time. Detecting, investigating, and responding to these changes requires sustained monitoring effort, making drift a core data engineer-

ing responsibility rather than an optional advanced topic. Chapter 14 builds on this foundation with operational response orchestration, outcome validation, and evidence-based retraining pipelines at scale.

Section B.2.2 formalizes the divergence metrics used to make the divergence term $\mathcal{D}(P_t \| P_0)$ of the degradation equation (equation 1.3) actionable. The **population stability index (PSI)** quantifies changes in categorical or binned feature distributions, while **Kullback-Leibler (KL) divergence** measures an information-theoretic divergence between probability distributions. The two metrics encode different distributional assumptions and scales, and neither by itself supplies a universal cutoff. Teams calibrate metric-specific thresholds against a deployment baseline and monitor them over time. Crossing a threshold signals that the live distribution has moved far enough to warrant investigation and outcome checks; input divergence alone does not establish whether accuracy improved or degraded, or by how much. Mathematically, PSI discretizes a continuous feature into reference bins (typically deciles of baseline P_0) and computes $\text{PSI} = \sum_b (P_t(b) - P_0(b)) \ln \left(\frac{P_t(b)}{P_0(b)} \right)$. Because the percentage difference and the logarithmic ratio share the same sign, every shifting bin contributes positively to the index, establishing an empirical rule of thumb: $\text{PSI} < 0.1$ indicates stable distributions, $0.1 \leq \text{PSI} \leq 0.25$ signals moderate shift, and $\text{PSI} > 0.25$ triggers mandatory investigation.

Targeted detection and mitigation require decomposing drift into its constituent distribution components, as shown in the margin locator.

The first case, **covariate shift**, changes the input distribution while preserving the relationship between features and labels: $p(x)$ changes but $p(y | x)$ stays the same. A medical imaging system trained on one camera model might later receive production images from a different manufacturer. The disease-image relationship remains unchanged, but pixel values shift because sensor characteristics, color calibration, or image processing pipelines differ. Detection therefore focuses on input feature distributions, using metrics such as PSI or KL divergence.

The second case, **label shift**, changes the output distribution while preserving the relationship between labels and features: $p(y)$ changes but $p(x | y)$ stays the same. Disease prevalence might change seasonally while symptoms remain consistent predictors of each disease. A recommendation system might see the same pattern when new product categories launch, changing the relative frequency of user preferences without altering what makes products appealing within each category. Detection can often begin without ground truth labels by tracking shifts in the model's prediction distribution.

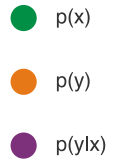
The hardest case is **concept drift**: the relationship between features and labels changes, so $p(y | x)$ evolves over time (Gama et al. 2014). Medical treatment protocols change, user preferences shift as social trends evolve, and fraud patterns adapt as attackers respond to detection systems. Unlike covariate or label shift, concept drift requires ground truth labels for detection because the system must observe whether the feature-to-label relationship itself has changed.

4.5.3.1 Label quality drift

Label quality drift¹⁰ represents a meta-level shift distinct from the three preceding distribution shifts: the reliability of ground truth labels degrades over time even when the underlying data distributions remain stable. This drift type proves particularly insidious because standard feature distribution monitoring fails to detect it. Crowdsourced labels may degrade as annotator pools change, training materials become outdated, or labeling guidelines evolve without corresponding model updates. Automated labeling systems accumulate errors as the models powering them drift from their original operating conditions. A recommendation system using click feedback as implicit labels may see label quality degrade as user behavior becomes more exploratory, as bot traffic patterns change, or as interface modifications alter how users interact with content.

Detection requires monitoring annotation consistency rather than feature distributions. Inter-annotator agreement metrics like Cohen's kappa¹¹ (κ) provide quantitative assessment. Let p_o represent observed agreement between annotators and p_e represent agreement expected by chance. Equation 4.2 defines the statistic:

$$\kappa = \frac{p_o - p_e}{1 - p_e} \quad (4.2)$$



Each drift type traces to the distribution component that shifted.

10 | Label quality drift: Degradation in annotation reliability over time, distinct from distribution shifts in the data itself. This drift type is invisible to standard feature monitoring because the features remain stable while the labels degrade—annotator fatigue, pool turnover, or guideline evolution silently corrupt the ground truth the model learns from. Detection requires monitoring inter-annotator agreement (κ) over rolling time windows and comparing automated labels against periodic expert audits.

11 | Cohen's kappa: Introduced by Cohen (1960) to measure inter-rater agreement while correcting for agreement expected from the raters' class marginals, which raw percentage agreement ignores. If two independent annotators each label 90 percent of images as "not spam" in a binary task, their expected agreement is 82 percent, making raw agreement potentially misleading. The statistic is denoted κ ; interpretive bands such as the Landis-Koch categories are descriptive heuristics rather than universal data-quality thresholds (Landis and Koch 1977).

Monitoring κ over time windows reveals degradation trends. A medical imaging annotation project might establish a baseline $\kappa = 0.85$ (almost perfect agreement) during initial data collection, then observe decline to $\kappa = 0.72$ (substantial agreement) after six months as new annotators join without receiving equivalent domain training.

For systems with calibrated model probabilities, predictive entropy provides an additional uncertainty signal. Let p_i represent the model's probability assigned to label category i , and let $\log$ denote the natural logarithm, so entropy is measured in nats. Equation 4.3 defines this measure:

$$H_{\text{pred}} = - \sum_i p_i \log p_i \quad (4.3)$$

Rising predictive entropy indicates greater uncertainty in the model's predictive distribution, but it does not identify the cause. Harder inputs, distribution shift, calibration changes, or inconsistent supervision can produce the same signal.

Mitigation strategies depend on root cause analysis. Annotator retraining addresses systematic errors from unclear guidelines at low cost with high effectiveness. Multi-annotator voting with majority or consensus rules provides high accuracy for high-stakes domains but significantly increases annotation costs. Model-assisted labeling reduces annotator fatigue but risks introducing bias if the assisting model has its own systematic errors. Expert review sampling, where domain specialists audit a random sample of annotations, enables root cause analysis when quality decline is detected but provides medium coverage of the overall annotation stream.

Operationalizing the PSI and KL divergence metrics introduced in section B.2.2 requires connecting them to automated alerts and review workflows. Data engineering is responsible for defining domain-specific drift thresholds from baseline behavior and outcome evidence, selecting monitoring windows that can expose both sudden and gradual changes, and instrumenting pipelines to compute these metrics continuously. Chapter 14 later examines how production teams combine these signals with labeled outcomes, tiered alerts, escalation paths, cold-start monitoring, and cause-specific response orchestration.

Statistical drift detection identifies distribution changes over time, but monitoring alone does not keep a system running when upstream pipelines stall, malformed payloads arrive, or schema definitions change without notice. Maintaining service continuity under failure requires active reliability mechanisms: fault isolation, backpressure, and graceful degradation.

4.5.4 Reliability through graceful degradation

Reliability ensures systems continue operating when problems occur. Pipelines face constant challenges: data sources become temporarily unavailable, network partitions separate components, upstream schema changes break parsing logic, or unexpected load spikes exhaust resources. **Graceful degradation** means handling these failures through systematic failure analysis, intelligent error handling, and automated recovery strategies that maintain service continuity even under adverse conditions.

Systematic failure mode analysis for ML data pipelines reveals predictable patterns that require specific engineering countermeasures. Data corruption failures occur when upstream systems introduce subtle format changes, encoding issues, or field value modifications that pass basic validation but corrupt model inputs. A date field switching from "YYYY-MM-DD" to "MM/DD/YYYY" format might not trigger schema validation but will break any date-based feature computation. Schema evolution¹² failures happen when source systems add fields, rename columns, or change data types without coordination, breaking downstream processing assumptions that expected specific field names or types. Resource exhaustion manifests as gradually degrading performance when data volume growth outpaces capacity planning, eventually causing pipeline failures during peak load periods.

Effective error handling strategies ensure problems are contained and recovered from systematically. Intelligent retry logic for transient errors (network interruptions or temporary service outages) requires exponential backoff strategies to avoid overwhelming recovering services. A simple linear retry that attempts reconnection every second would flood a struggling service with connection attempts, potentially preventing its recovery. Exponential backoff, retrying after one second, then two seconds, then four seconds, doubling with each attempt, gives services breathing room to recover

¹² | **Schema evolution:** This failure mode arises from a lack of contract testing between upstream data producers and downstream ML consumers. While "loud" failures like a renamed column break explicit assumptions and cause immediate pipeline crashes, "silent" failures are more dangerous. A field changing type from integer to string can pass validation but corrupt feature logic without immediate detection.

while still maintaining persistence. Many ML systems employ **dead-letter queues (DLQs)**: separate storage for data that fails processing after multiple retry attempts. This allows for later analysis and potential reprocessing of problematic data without blocking the main pipeline (Kleppmann 2016). A pipeline processing financial transactions that encounters malformed data can route it to a dead-letter queue rather than losing critical records or halting all processing.

In ML systems, dead-letter queues serve dual purposes beyond failure analysis. Production teams implement systematic review of DLQ contents to identify: (1) schema violations indicating upstream changes, (2) edge case patterns the model should handle, and (3) data quality issues requiring source system fixes. For example, a fraud detection system’s DLQ revealed transactions from a new payment type the model had never seen, prompting targeted data collection and retraining rather than simply logging the failures. This transforms DLQs from passive error storage into active sources for identifying model blind spots and driving improvement.

Moving beyond ad-hoc error handling, cascade failure prevention requires circuit breaker¹³ patterns and bulkhead isolation to prevent single component failures from propagating throughout the system. When a feature computation service fails, the circuit breaker pattern stops calling that service after detecting repeated failures, preventing the caller from waiting on timeouts that would cascade into its own failure.

Automated recovery engineering extends beyond simple retry logic. Exponential backoff, jitter, load shedding, and circuit breakers reduce request pressure on struggling services while allowing transient faults to recover; simply increasing timeouts can retain resources longer and worsen overload. Multi-tier fallback systems provide degraded service when primary data sources fail: serving slightly stale cached features when real-time computation fails, or using approximate features when exact computation times out. A recommendation system unable to compute user preferences from the past 30 days might fall back to preferences from the past 90 days, providing less precise but still useful recommendations rather than failing entirely. Comprehensive alerting and escalation procedures ensure human intervention occurs when automated recovery fails, with sufficient diagnostic information captured during the failure to enable rapid debugging.

Retry logic, dead-letter queues, and circuit breakers serve as runtime error handlers in the dataset compiler: they catch malformed inputs without halting the entire pipeline. These safeguards operate once data enters the system. The choice of ingestion pattern—batch vs. streaming; extract, transform, load (ETL) vs. extract, load, transform (ELT)—determines how quickly new data reaches the model, what buffering infrastructure is required, and where reliability invariants are enforced.

4.5.5 Data ingestion

In the compiler framework, data ingestion acts as the *lexer*: it reads raw source streams and tokenizes them into structured records that downstream stages can process. The primary performance barrier at this stage is the **input/output (I/O) bottleneck**. Accelerators remain idle if ingestion cannot stage tensors quickly enough to keep compute units supplied. In an idealized two-stage model, step time ranges from $\max(T_{\text{compute}}, T_{\text{io}})$ with full overlap to $T_{\text{compute}} + T_{\text{io}}$ with no overlap. These bounds restate the feeding problem from section 4.2.3 through a second resource lens: while storage bandwidth starved the compute units in the earlier scenario, ingestion bottlenecks more commonly stem from CPU-side decompression and decode overhead.

If the data pipeline cannot decode images fast enough to keep the GPU busy, the expensive accelerator sits idle. This phenomenon creates a “Choke Point” where adding more GPUs yields no speedup until the input pipeline is improved, a counterintuitive result for teams expecting linear scaling from hardware investments. This bottleneck frequently occurs in computer vision, where high-resolution JPEG decode and augmentation on the CPU can dominate the input path. For this illustrative calculation, each worker supplies 375 images/s and the accelerator consumes 3,000 images/s, so 8 workers are needed to reach the crossing. This result is a capacity match, not a hardware constant: below the crossing, the input pipeline binds; above it, additional workers do not raise throughput because the accelerator ceiling binds. Actual ResNet-50 and A100 requirements depend on image transforms, storage, batch size, host CPUs, and framework configuration.

Scaling training throughput requires co-designing host ingestion alongside GPU compute: raw accelerator FLOPs remain unexploited if host preprocessing cannot supply tensors fast enough to prevent compute starvation. Figure 4.8 illustrates this saturation dynamic, plotting the intersection

¹³ | **Circuit breaker:** Named for its three-state behavior—closed (normal flow), open (faults blocked), half-open (recovery probe)—after the electrical safety device that interrupts current on overload. In ML data pipelines, the circuit breaker prevents a failing feature computation service from cascading timeouts through the entire serving path: once failure count exceeds a threshold, the breaker opens and the pipeline falls back to cached or default features rather than waiting on a dead service.

between the flat accelerator consumption ceiling and the CPU supply curve as DataLoader worker processes scale.

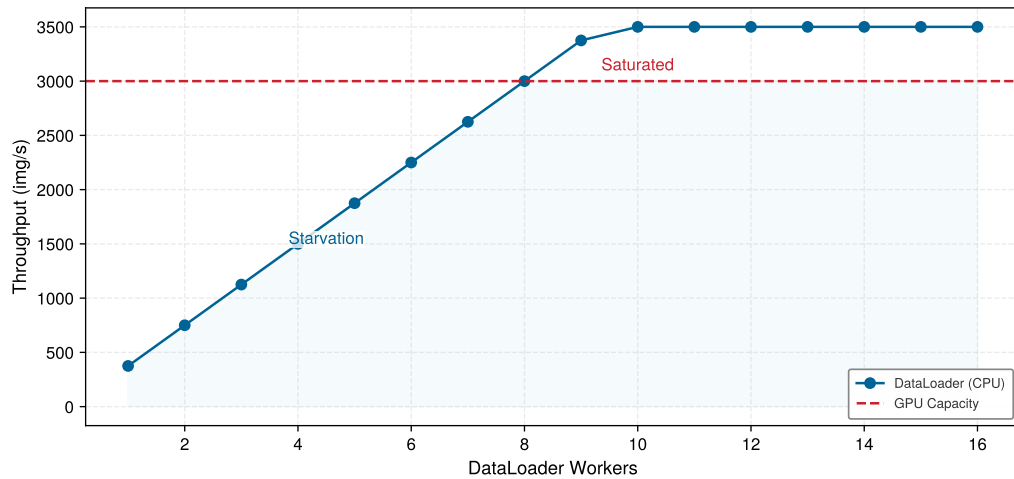


Figure 4.8: The DataLoader Choke Point: Training throughput (img/s) vs. DataLoader workers. The blue CPU-supply curve rises with worker count until it reaches the illustrative input-pipeline cap; the red dashed line marks accelerator consumption capacity. The lower curve is the bottleneck: the accelerator starves to the left of the crossing, while extra workers beyond the crossing do not increase accelerator throughput.

When dataloader throughput falls below the accelerator consumption rate, utilization collapses: matrix execution units stall waiting for batches while high-bandwidth memory (HBM) sits idle holding static weights. Relieving this choke point requires multi-process worker pools, asynchronous pinned-memory prefetching across the PCIe bus, and hardware-accelerated decompression engines that bypass host CPU bottlenecks entirely.

4.5.5.1 Batch vs. streaming ingestion patterns

The choice between batch and streaming is not a preference for one architecture over another; it is a judgment about how quickly data loses value and how much infrastructure cost that freshness justifies. Batch systems buy efficiency by tolerating staleness, while streaming systems buy freshness by accepting continuous operational complexity.

Batch ingestion involves collecting data in groups or batches over a specified period before processing. This method proves appropriate when real-time data processing is not critical and data can be processed at scheduled intervals. The batch approach enables efficient use of computational resources by amortizing startup costs across large data volumes and processing when resources are available or least expensive. For example, a retail company might use batch ingestion to process daily sales data overnight, updating their ML models for inventory prediction each morning (Akidau et al. 2015). The batch job might process gigabytes of transaction data using dozens of machines for 30 minutes, then release those resources for other workloads. This scheduled processing proves far more cost-effective than maintaining always-on infrastructure, particularly when slight staleness in predictions does not affect business outcomes.

Batch processing also simplifies error handling and recovery. When a batch job fails midway, the system can retry the entire batch or resume from checkpoints without complex state management. Data scientists can inspect failed batches, understand what went wrong, and reprocess after fixes. Batch jobs can be reproducible when inputs, code, configuration, randomness, ordering-sensitive operations, and external state are controlled, which simplifies debugging and validation. These characteristics make batch ingestion attractive for ML workflows even when real-time processing is technically feasible but not required.

In contrast to this scheduled approach, **stream ingestion** processes data in real-time as it arrives, consuming events continuously rather than waiting to accumulate batches. This pattern is essential for applications requiring immediate data processing, scenarios where data loses value quickly,

and systems that need to respond to events as they occur. A financial institution might use stream ingestion for real-time fraud detection, processing each transaction as it occurs to flag suspicious activity immediately before completing the transaction. The value of fraud detection drops dramatically if detection occurs hours after the fraudulent transaction completes—by then money has been transferred and accounts compromised.

However, stream processing introduces complexity that batch processing avoids. The system must handle **backpressure**, the condition where downstream systems cannot keep pace with incoming data rates. During traffic spikes, when a sudden surge produces data faster than processing capacity, the system must either buffer data (requiring memory and introducing latency), sample (losing some data), or push back to producers (potentially causing their failures). Data freshness service level agreements (SLAs) specify maximum acceptable delays between data generation and processing. Meeting a 100-millisecond SLA requires different infrastructure than meeting a one-hour SLA, affecting networking, storage, and processing architectures.

Recognizing the limitations of either approach alone, many production ML systems employ hybrid approaches that combine batch and stream ingestion. A recommendation system might use streaming ingestion for real-time user interactions to update session-based recommendations immediately, while using batch ingestion for overnight processing of user profiles and item features; 4.5 quantifies the financial trade-off between these two operational regimes.

Napkin Math 4.5: The cost of real-time

Problem: A pipeline ingests 1M events/s. Which is cheaper, Batch (hourly) or Stream (sub-second) ingestion?

Physics:

1. **Throughput:** At 1M events/s and 1 KB per event, the pipeline carries 1 GB/s.
2. **Stream requirements:** To sustain 1 GB/s with less than 100 ms latency, the system needs about 50 primary cores plus about 50 redundant cores, or 100 always-on cores total. Running those cores for 24 hours/day at \$0.05/hr costs \$120/day.
3. **Batch requirements:** Process 3.6 TB (1-hour window) in 10 minutes. High throughput (sequential I/O) is efficient. The batch design needs 200 cores for 10 minutes per clock hour, accumulating 800 core-hours per day. At \$0.05/hr, that costs \$40/day.

Systems insight: Real-time is approximately 3× more expensive for the same data volume. This tax is justified only when the value of subsecond latency exceeds the added cost.

As the preceding cost calculation shows, the real-time premium stems directly from resource provisioning. Streaming systems require always-on infrastructure with redundant compute and storage to meet millisecond latency targets without dropping events. Batch systems, by contrast, provision ephemeral instances that process high volumes sequentially at high utilization before shutting down. Deciding between them depends on whether downstream inference strictly requires sub-second feature freshness.

The ingestion paradigm also shapes how the downstream model trains. Batch ingestion naturally pairs with periodic retraining: data accumulates in storage, an automated job retrains the model on the updated corpus, and a verified checkpoint deploys to serving. Stream ingestion makes that batch boundary less natural, raising the possibility of continuous online learning—updating model weights directly as events arrive. Yet continuous updates introduce operational risks that batch checkpoints avoid: sudden distribution shifts, corrupted records, or adversarial payloads can degrade weights before validation gates intervene (Lee 2016). Most production systems therefore decouple ingestion from training: they stream data into a durable buffer while training on validated batches, reserving continuous online updates for tightly bounded, low-risk domains.

4.5.5.2 ETL and ELT comparison

Pipeline architecture dictates when and where computational resources are applied to raw incoming records. Figure 4.9 contrasts the sequence flows between traditional ETL¹⁴ and modern ELT¹⁵ pipelines, illustrating where the transformation boundary sits relative to persistent storage.

¹⁴ | **Extract, transform, load (ETL):** Conventional ETL validates *schema and data types*—deterministic checks that either pass or fail. ML ETL must additionally validate *distributional properties*: that the distribution of values has not shifted in a way that degrades model performance, requiring statistical tests (K-S test, PSI) rather than schema validation, where the pass/fail threshold is a business decision, not a technical one. The further trade-off is rigidity: changing feature computation logic requires reprocessing the entire dataset from raw sources, a cost that grows linearly with data volume.

¹⁵ | **Extract, load, transform (ELT):** This approach stores raw data before transformation, allowing teams to revise a transformation query and rerun it over the stored input without re-ingesting from source systems. The iteration time still depends on data volume, query complexity, warehouse capacity, and materialization strategy.

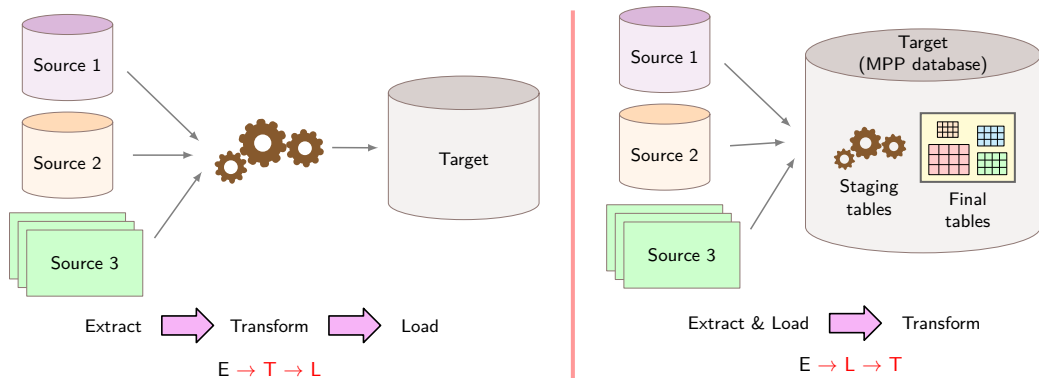


Figure 4.9: ETL vs. ELT Architectural Paradigms: Shifting the transformation boundary relative to storage. ETL enforces schema validation and cleaning before loading to protect warehouse quality, whereas ELT loads raw data directly into scalable storage, trading warehouse compute and footprint for flexible, repeatable re-transformations.

In machine learning systems, the ELT pattern decouples ingestion throughput from downstream feature engineering by landing raw, immutable events directly into scalable storage. Storing raw events preserves unparsed fields that may later prove predictive, enabling teams to extract novel features over historical datasets without re-ingesting external source streams. When transformation logic changes or bugs surface, engineers recompute features by dispatching queries over the lakehouse rather than replaying upstream logs. The trade-off is physical: retaining high-fidelity raw data increases storage footprint, transforms must be recomputed if shared across models without intermediate materialization, and retaining unscrubbed raw payloads complicates data governance and privacy enforcement. 4.6 quantifies how these trade-offs translate into operational infrastructure costs.



Napkin Math 4.6: The cost of transformation placement

Problem: A team processes 10 TB of raw clickstream data daily and must compute user session features for 3 ML models, each requiring different aggregation windows (1-hour, 24-hour, 7-day). Which is cheaper, ETL or ELT?

Math:

1. **ETL approach:** Transform before loading. Compute all three aggregation windows in a Spark cluster before loading into the warehouse.
 - Spark compute: 10 TB at \$5/TB = \$50/day
 - Storage: 3 transformed datasets, ~2 TB each = 6 TB at \$23/TB/month = \$138/month
 - Schema change cost: Re-run full pipeline (~4 hours) per change
2. **ELT approach:** Load raw data first, transform in warehouse.
 - Storage: 10 TB raw/day, 30 days retention = 300 TB at \$23/TB/month = \$6,900/month
 - Query compute: 3 models, each with \$5/query of daily query cost over 30 days, totals \$450/month
 - Schema change cost: Rewrite SQL query (~30 minutes) per change

Systems insight: ETL saves \$6,762/month in storage. After normalizing Spark compute to a monthly cost, ETL totals \$1,638/month (\$1,500/month compute + \$138/month storage), while ELT totals \$7,350/month (\$6,900/month storage + \$450/month query compute), for a cloud-cost advantage of \$5,712/month before engineering labor. ETL still costs 8× more engineering time per schema change. The exact break-even point depends on engineering

labor cost and schema-change frequency: stable schemas favor ETL's lower cloud cost, while frequent feature-definition changes favor ELT's faster iteration.

Production ML systems rarely use one pattern exclusively. Structured data with stable schemas often flows through ETL for efficiency and compliance, while unstructured data or rapidly evolving feature pipelines benefit from ELT's flexibility. For deep learning workloads processing images, audio, or text, the ELT preference runs even deeper: the "Transform" step for unstructured data often executes inside the ML framework's data loader rather than in the warehouse at all, applying random crops, spectrogram generation, or text tokenization on-the-fly during each training epoch. Materializing ten augmented variants upstream would increase a 50,000-image dataset to 500,000 stored copies, so ELT's "load raw, transform late" principle extends naturally to the training loop itself.

Choosing between upfront transformation and load-time transformation hinges on query reuse: transformed data that is queried hundreds of times amortizes ETL compute, whereas single-pass exploratory queries favor ELT warehouse elasticity. The parameterized cost model in listing 4.2 evaluates this break-even threshold across varying data volumes and query frequencies.

Listing 4.2: ETL vs. ELT Architectural Break-Even: Parameterized financial comparison computing compute and storage expenses across transformation strategies.

```
# ETL vs ELT cost comparison
daily_raw_tb = 10
s3_per_tb_mo = 23
spark_per_tb = 5
n_models = 3
retention_days = 30
query_cost_per = 5

# ETL
etl_spark_daily = daily_raw_tb * spark_per_tb
etl_datasets = 3
etl_tb_each = 2
etl_storage_tb = etl_datasets * etl_tb_each
etl_storage_mo = etl_storage_tb * s3_per_tb_mo

# ELT
elt_storage_tb = daily_raw_tb * retention_days
elt_storage_mo = elt_storage_tb * s3_per_tb_mo
elt_query_mo = n_models * query_cost_per * retention_days

# Savings
storage_savings_mo = elt_storage_mo - etl_storage_mo
```

The storage difference favors ETL in this scenario, but the listing does not price schema-change labor; the worked calculation in 4.6 evaluates that operational overhead directly.

When streaming components enter ETL and ELT architectures, the infrastructure choice becomes a failure-mode choice. The CAP theorem¹⁶ states that during a network partition, a distributed read/write service cannot guarantee both linearizable consistency and availability for every request. Apache Kafka¹⁷ provides ordering within each partition; its availability and durability during failures depend on acknowledgment, replication, and leader-election settings. Apache Pulsar separates stateless brokers from durable message storage in replicated Apache BookKeeper ledgers. Its built-in geo-replication is asynchronous, so a remote cluster may lag during failures or partitions rather than providing simultaneous strong cross-region consistency and availability. Amazon Kinesis exposes operational trade-offs through shard capacity, retention, and producer/consumer configuration, but under CAP it still *cannot* guarantee both strict consistency and availability during a network partition.

¹⁶ | **CAP (consistency, availability, partition tolerance) theorem:** Conjectured by Brewer (2000) and formally proved by Gilbert and Lynch (2002). During a network partition, linearizable consistency may require rejecting requests, whereas an available system may return stale or divergent values. CAP does not guarantee feature-transformation parity or point-in-time correctness.

¹⁷ | **Apache Kafka:** Kafka uses a partitioned, leader-based log and orders records within each partition, not across a topic. Durability and write availability during failures depend on acknowledgment mode, replication, in-sync replica requirements, and leader-election behavior.

4.5.5.3 Feature computation placement

For ML pipelines, **feature computation placement** decides which resource pays for a feature: storage pays when features are materialized, while compute and latency pay when features are generated on demand. This choice significantly impacts training speed, storage costs, and reproducibility.

One approach is to precompute features during ETL and store the results. Pipeline-computed features offer fast training iteration (features are ready on disk), reproducibility (the same features are used consistently), and reduced training compute. The drawbacks are storage cost (features stored separately from raw data), staleness risk (precomputed features may diverge when logic changes), and inflexibility (any change requires full recomputation).

The alternative is computing features on the fly during training. Loader-computed features guarantee always-fresh computation (logic changes are immediately reflected), flexible experimentation (easy to modify features), and reduced storage (only raw data is stored). The cost is slower training (computation repeats each epoch), higher compute expenditure (GPUs often idle waiting for features), and potential nondeterminism if not carefully implemented.

In practice, hybrid patterns predominate. Expensive, stable features (user embeddings requiring matrix factorization, historical aggregations spanning months of data) are precomputed and materialized. Cheap, time-sensitive features such as recency signals, session context, and time-based transformations are computed in the data loader.

For example, a recommendation system precomputes stable user representation features (expensive, stable over days) while computing time-since-last-interaction features (cheap, time-sensitive) in the data loader. This balances storage costs, computation time, and feature freshness based on each feature's specific characteristics.

4.5.5.4 Integration strategies and KWS case study

Regardless of whether ETL or ELT approaches are used, integrating diverse data sources remains a core ingestion challenge. Data may originate from databases, APIs, file systems, and IoT devices, each with its own format (relational rows, JavaScript Object Notation (JSON)¹⁸ documents, binary streams), access protocol, and update frequency. The systems principle is to standardize at the ingestion boundary: normalize formats, validate schemas, and present a consistent interface to downstream processing regardless of source. This boundary standardization separates the complexity of source diversity from the complexity of feature engineering, allowing each to evolve independently.

KWS production systems often perform always-on wake-word detection on-device. Backend streaming and batch pipelines may ingest activated requests, consented diagnostics, crowdsourced recordings, synthetic data, and validated interactions for serving and training. Batch processing typically follows an ETL pattern where audio undergoes normalization, noise filtering, and segmentation into consistent durations before storage in training-optimized formats.

Error handling in voice interaction systems requires special attention. Dead letter queues store failed recognition attempts for subsequent analysis, revealing edge cases that need coverage in future model iterations. Each incoming audio sample must pass quality validation (signal-to-noise ratio, sample rate, duration bounds, speaker proximity) before entering the processing pipeline. Invalid samples route to analysis queues rather than being discarded, since these failures often indicate acoustic conditions underrepresented in training data. Valid samples flow through to real-time detection while simultaneously being logged for potential inclusion in future training data.

This ingestion architecture completes the boundary layer where external data enters the pipeline. Ingested data, however reliably delivered, remains raw: audio arrives at inconsistent sample rates, text carries varying encodings, and numeric features span incompatible scales. Downstream training requires transforming these heterogeneous records into a uniform representation while guaranteeing that identical transformations execute during both training and serving.

4.6 Systematic Data Processing

Where ingestion lexed raw streams into well-formed records, data processing performs the compiler's *optimization and lowering pass*. Just as compiler optimizations must preserve program semantics while improving runtime performance, data transformations must preserve underlying signal while formatting tensors for numerical stability. The governing constraint is consistency: transformations

¹⁸ | **JSON (JavaScript object notation)**: The schema flexibility that makes JSON a common format for APIs creates a validation bottleneck at the ingestion boundary. Unlike binary formats with predefined schemas, every JSON document requires parsing and schema validation before it can be standardized for downstream use. This per-record overhead can make ingestion slower than with binary formats such as Protobuf; the difference depends on the schema, parser, payload, and workload.

shared between training and serving must maintain identical semantics, repeated operations must remain idempotent under retries, and data transformations must scale without severing lineage.

Sculley et al. (2015) identify data dependencies, changes in the external world, configuration debt, and missing monitoring as hidden technical debt risks in production ML systems. Training-serving inconsistency is one concrete form of this risk. Consider normalizing transaction amounts during training by stripping currency symbols and casting strings to floats, but omitting identical preprocessing during serving. This discrepancy introduces training-serving skew, degrading downstream model accuracy. In the KWS pipeline, the requirement is concrete: transformations must standardize across diverse recording conditions (varying microphones, background noise levels, sample rates) while preserving the acoustic characteristics that distinguish wake words from ambient speech—and this feature extraction must remain bit-for-bit consistent across both training and embedded serving paths.

4.6.1 Training-serving consistency

The training-serving consistency challenge extends beyond applying the same code—it requires that parameters computed on training data (normalization constants, encoding dictionaries, vocabulary mappings) are stored and reused during serving. This requirement is the **consistency imperative**.

Definition 4.3: The consistency imperative

The Consistency Imperative requires equivalent transformation behavior and state across training and serving environments.

1. **Significance:** KL divergence ($\mathcal{D}_{\text{KL}}(p_{g_{\text{serve}}} \| p_{g_{\text{train}}})$) between feature distributions induced by the serving transformation g_{serve} (current) and training transformation g_{train} (baseline), where $g : \mathcal{X} \rightarrow \mathcal{Z}$ denotes the feature transformation mapping raw input x to feature vector z , can signal misalignment. Its relationship to performance degradation depends on the model, task, and shifted features and must be checked against outcome evidence.
2. **Distinction:** Unlike data quality, which focuses on the cleanliness of a single record, the consistency imperative focuses on the alignment of the entire transformation pipeline.
3. **Common pitfall:** A frequent misconception is that consistency is “fixed” by sharing code. In reality, it is a state synchronization problem: parameters computed on training data (for example, means, standard deviations) must be stored and reused during serving.

Violating the consistency imperative silently degrades production accuracy because models do not throw runtime exceptions when presented with out-of-distribution inputs; they return confident, erroneous predictions. Preventing silent skew requires designing transformation pipelines from the serving deployment constraints backward.

Checkpoint 4.3: Defensive processing

Training-serving skew is a common cause of silent production degradation.

- ☐ **Definition:** can you define training-serving skew in terms of the training and live-request processing paths?
- ☐ **Mechanism:** can you explain why recomputing normalization means independently in training and serving can make live data look unfamiliar to the model?
- ☐ **Invariant:** can you explain how shared logic, synchronized state, and parity checks reduce skew?

Data cleaning is the first place where consistency can be either enforced or broken. Raw data frequently contains missing values, duplicates, or outliers that degrade model performance. The key insight is that cleaning operations must be deterministic and reproducible: given the same input, they must produce the same output regardless of environment. This requirement shapes which cleaning techniques are safe to use in production.

Data cleaning encompasses deduplication, missing-value handling, and string normalization. For instance, a customer database might normalize “John Doe,” “john doe,” and “DOE, John” to a common display format. Normalization alone does not establish that records refer to the same person; identity resolution requires a stable identifier or a separately validated matching rule. Both the formatting and matching rules must be captured in code that executes equivalently in training and serving.

Outlier handling exposes a fundamental operational divergence between batch training and online serving. During training, corrupted records or extreme sensor glitches can be dropped to prevent gradient explosions. At serving time, however, the model cannot discard an incoming inference request without violating its latency and availability contract. Instead of dropping records, serving pipelines must clamp features to bounded intervals (such as $[\mu - 3\sigma, \mu + 3\sigma]$) using parameters estimated from the training partition. More complex outlier filters—such as density-based spatial clustering or multivariate isolation forests—introduce severe consistency risks: if evaluated over dynamic serving mini-batches, the classification of a query depends on what other queries happen to arrive in the same batch window, violating request isolation.

Quality assessment complements data cleaning by systematically evaluating the reliability and usefulness of data across multiple dimensions: accuracy, completeness, consistency, and timeliness. In production systems, data quality degrades in subtle ways that basic metrics miss: fields that never contain nulls suddenly show sparse patterns, numeric distributions drift from their training ranges, or categorical values appear that were not present during model development.

To address these subtle degradation patterns, production quality monitoring requires specific metrics beyond simple missing value counts (section 4.5.1). Critical indicators include null value patterns by feature (sudden increases suggest upstream failures), count anomalies ($10\times$ increases often indicate data duplication or pipeline errors), value range violations (prices becoming negative, ages exceeding realistic bounds), and join failure rates between data sources. Statistical drift detection¹⁹ becomes essential by monitoring means, variances, and quantiles of features over time to catch gradual degradation before it impacts model performance. For example, in an e-commerce recommendation system, the average user session length might gradually increase from eight minutes to 12 minutes over six months due to improved site design, but a sudden drop to three minutes suggests a data collection bug.

Quality validation must enforce identical schemas and validation predicates across training and serving pipelines. While training environments can afford compute-intensive profiling passes over entire datasets, serving paths must execute lightweight assertion checks (such as type validation, range bounding, and required-field existence) within the request’s latency budget. Mismatches between training-time validation and serving-time assertions allow corrupted inputs to reach the model undetected.

Transformation techniques convert data from its raw form into a model-ready representation, but the systems risk lies in the parameters those transformations learn. Common transformation tasks include normalization and standardization, which scale numerical features to a common range or distribution. For example, square footage and room count may differ greatly in scale; normalization places them on comparable numerical ranges, though their influence still depends on the model and learned parameters (Bishop 2006). Maintaining training-serving consistency requires that normalization parameters computed on training data be stored and applied identically during serving. Operationally, these parameters must be persisted alongside the model itself and loaded during serving initialization.

Beyond numerical scaling, transformations convert discrete inputs through categorical encoding, extract cyclical representations from timestamps, and construct derived spatial features. Categorical encodings must handle both the categories present during training and unseen categories encountered during serving. A robust pipeline builds the vocabulary index exclusively from the training split, serializes this dictionary with the model artifact, and maps unknown runtime tokens to an explicit out-of-vocabulary (<UNK>) index. Omitting this mapping causes runtime dimension mismatches or silent index collisions during online inference.

A health prediction model may receive raw GPS coordinates for each patient visit, but latitude and longitude alone do not directly represent access to care. An engineer might derive *distance to nearest hospital* as a candidate proxy for geographic access. Whether that feature predicts outcomes,

¹⁹ | **Statistical drift detection:**

The means, variances, and quantiles tracked in quality monitoring are early-warning signals for the degradation equation’s divergence term $\mathcal{D}(P_t \| P_0)$. A mean session length shifting from eight to 12 minutes over six months may indicate drift; a sudden drop to three minutes may indicate a collection fault. Outcome checks and source investigation are needed before choosing retraining or a source-system fix.

improves the model, or introduces sensitive-location and socioeconomic proxies must be established with data and domain review rather than assumed from the transformation alone.

Domain-specific feature engineering synthesizes raw inputs into representations that expose informative signals directly to the learning algorithm (Kuhn and Johnson 2013). Instead of requiring the model to discover non-linear combinations from scratch, engineered features encode structural domain relationships. In a retail recommendation system, for example, pipelines compute the recency, frequency, and monetary (RFM) value of customer transactions (known as RFM analysis) to summarize purchasing trajectories into dense behavioral vectors.

Feature engineering can be a high-leverage activity because it changes what signal the model can see. In some workloads, well-designed features improve performance more than another round of algorithm selection or hyperparameter tuning. That leverage must be balanced against the consistency requirements of production systems. Every engineered feature shared by training and serving must have equivalent behavior in both environments. Production systems therefore implement feature engineering logic in shared libraries or coordinated pipelines rather than reimplementing it independently. Many organizations use **feature stores**,²⁰ discussed in section 4.8.5, to reduce inconsistency across environments.

In the KWS pipeline, audio recordings collected across crowdsourced contributors, synthetic speech generators, and acoustic field captures arrive with severe acoustic variability. The raw audio contains the imperfections anticipated during problem formulation: environmental noise spanning quiet bedrooms to industrial floors, amplitude clipping from saturated analog-to-digital converters, gain discrepancies across microphone hardware, and inconsistent sampling rates. Standardizing these inputs while preserving the phonemic transients that distinguish wake words from ambient noise is essential to achieving the 98 percent target accuracy.

Quality assessment for KWS extends the general principles with audio-specific metrics. Beyond checking for null values or schema conformance, the validation pipeline tracks background noise levels (signal-to-noise ratio above 20 dB), audio clarity scores (frequency spectrum analysis), and speaking rate consistency (wake word duration within 500 ms to 800 ms). The pipeline flags recordings whose noise, duration, clipping, or distortion falls outside the scenario's acceptance rules. These screens do not prove that a sample is useful, but they reduce the chance that known defects reach model development. As demonstrated by the compounding error cascades in figure 4.1, filtering corrupted records at the source prevents early defects from multiplying across downstream artifacts.

Transforming audio data for KWS involves converting raw waveforms into formats suitable for ML models while maintaining training-serving consistency. Raw audio waveforms (sequences of amplitude values sampled thousands of times per second) are high-dimensional. KWS pipelines often transform them into compact representations that emphasize the frequencies and temporal patterns most relevant to speech. Figure 4.10 compares three illustrative representations used in this pipeline: a waveform, its time-frequency spectrogram, and a compact MFCC approximation. These standardized feature representations, typically Mel-frequency cepstral coefficients (MFCCs)²¹ or spectrograms,²² emphasize speech-relevant characteristics while reducing noise and variability across different recording conditions. While a raw short-time Fourier transform (STFT) produces hundreds of linear frequency bins per time window, Mel-frequency analysis aggregates these bins through non-linear triangular filterbanks aligned with human auditory perception, followed by a discrete cosine transform that compresses the signal into just 13–40 decorrelated coefficients. For edge devices, this step collapses data volume by over an order of magnitude, allowing audio feature frames to fit inside the tight tens-of-kilobytes SRAM budgets of embedded coprocessors.

4.6.2 Idempotent transformations

Reliability adds retry safety to these quality foundations. While quality focuses on what transformations produce, reliability also governs how safely they can be repeated. **Idempotency**²³ means that applying an operation repeatedly has the same effect as applying it once. This property proves essential for production ML systems where processing may be retried after failures, data may be reprocessed to fix bugs, or the same data may flow through multiple processing paths. Determinism is a related but distinct requirement: identical inputs and captured state should produce identical outputs.

²⁰ | **Feature store:** A core failure mode is training-serving skew: training features may be computed in batch (for example, seven-day rolling averages over historical data) while serving features are computed in real time from a streaming source; even with nominally shared logic, timing and state differences can create systematic discrepancies. A feature store can reduce this risk by coordinating definitions, data sources, timestamp handling, and offline and online materialization, but correctness still depends on implementation, freshness, and synchronization. Uber's Michelangelo helped establish this dual-interface pattern for batch training and low-latency serving (Hermann and Del Balso 2017).

²¹ | **MFCC (mel-frequency cepstral coefficients):** This transformation achieves its compactness and noise resistance by applying mel-scale filtering, which selectively emphasizes the frequencies humans use to distinguish speech. This process reduces thousands of raw audio samples from a small time window (for example, 25 ms) into just 13–39 coefficients, the aggressive dimensionality reduction required for always-on, kilobyte-scale hardware. A parameter mismatch between training and serving creates feature skew and can degrade accuracy.

²² | **Spectrogram:** The short-time Fourier transform (STFT) computes this 2D representation by converting the one-dimensional waveform into a time-frequency image. This representation lets image-style ML models process audio, but it also creates a rigid dependency: a mismatch in STFT parameters (for example, a 25 ms vs. 30 ms window) changes the feature representation between training and serving and can degrade performance.

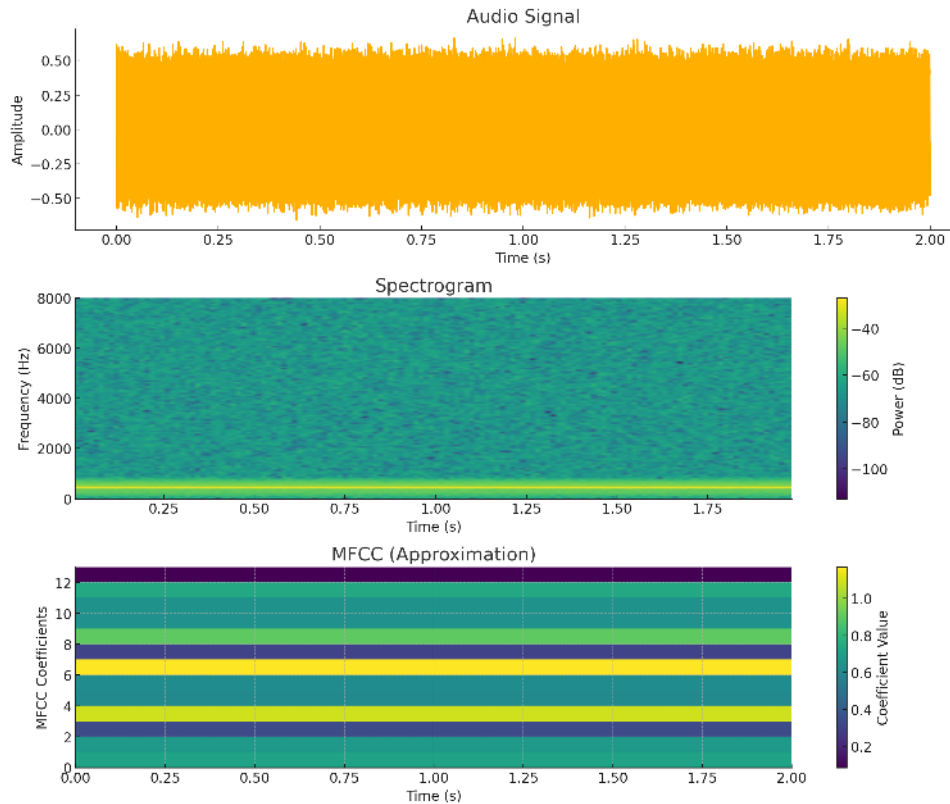


Figure 4.10: Audio Feature Representations: Three illustrative views show an audio waveform, its time-frequency spectrogram, and a compact MFCC approximation. The spectrogram encodes time, frequency, and signal power; the bottom panel arranges a smaller set of coefficients over time.

23

Idempotency:

From Latin *idem* (“the same”) + *potens* (“having power”)—literally, “having the same power when applied again.” In ML pipelines, idempotency enables safe retries after partial failures: a nonidempotent operation (for example, appending to a log) creates duplicates on retry. An idempotent operation leaves the same resulting state after one application or repeated applications.

In state-machine terms, an operation f is idempotent if $f(f(x)) = f(x)$. In storage systems, setting a register to an absolute value ($x \leftarrow 1$) is idempotent, whereas incrementing a counter ($x \leftarrow x + 1$) or appending a row to an unkeyed log is not: each re-execution mutates system state. Distributed data pipelines execute across preemptible workers and unreliable networks where timeouts and node crashes trigger automatic retries. If a stage appends transformed records to a target table, a worker that crashes after writing but before acknowledging completion will re-execute on retry, generating duplicate records. An idempotent pipeline enforces write semantics such as deterministic keyed upserts (insert if absent, overwrite if present) or atomic directory swaps, ensuring that repeated job attempts converge to the identical final dataset.

Managing partial pipeline failures requires explicit state demarcation between stages. Decoupling stages with persistent intermediate checkpoints ensures that each phase can be retried independently without re-executing upstream work. For long-running jobs processing terabyte-scale corpora, checkpointing state partitions (such as completed file shards or stream consumer offsets) bounds retry latency: a task failure after five hours of execution requires replaying only the uncommitted shard rather than restarting the entire five-hour job. The checkpoint mechanism must commit output partitions and state markers in a single atomic transaction, preventing records from being dropped or processed twice.

Deterministic transformations are those that always produce the same output for the same input, without dependence on external factors like time, random numbers, or mutable global state. Transformations that depend on current time (for example, computing “days since event” based on current date) break determinism because reprocessing historical data would produce different results. The solution is to capture temporal reference points explicitly: instead of “days since event,” compute “days from event to reference date” where reference date is fixed and persisted. Random operations

should use seeded random number generators where the seed is derived deterministically from input data, ensuring reproducibility.

Reliability in the KWS pipeline requires reproducible feature extraction. Audio preprocessing must be deterministic: given the same raw audio file, the same MFCC features are always computed regardless of when processing occurs or which server executes it. This enables debugging model behavior (can always recreate exact features for a problematic example), reprocessing data when bugs are fixed (produces consistent results), and distributed processing (different workers produce identical features from the same input). The processing code captures all parameters (fast Fourier transform (FFT) window size, hop length, number of MFCC coefficients) in configuration versioned alongside the code, ensuring reproducibility across time and execution environments. Even with rigorous design, production systems must implement runtime monitoring to detect skew if it emerges; chapter 14 covers operational comparison and distribution monitoring at scale.

4.6.3 Distributed processing

Scale becomes the next constraint once quality and reliability are in place. Quality ensures transformations produce *correct* outputs; reliability ensures they produce *consistent* outputs. Neither matters if processing cannot keep pace with data volume or its deadline. As datasets and experiment concurrency grow, data processing can outgrow one machine even when the raw bytes still fit on local storage. The cleaning techniques that work on gigabytes in memory must then become partition-aware, out-of-core, or distributed without changing their semantics.

These challenges manifest when quality assessment must keep pace with incoming data, when feature engineering requires computing statistics across entire datasets before transforming individual records, and when transformation pipelines create bottlenecks at massive volumes. Processing must scale from development (gigabytes on laptops) through production (terabytes across clusters) while maintaining consistent behavior.

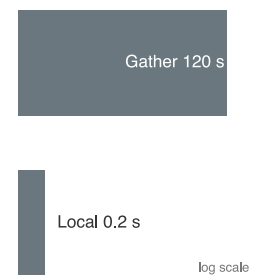
Single-machine processing suffices for surprisingly large workloads when engineered carefully. Modern servers with 256 gigabytes RAM can process datasets of several terabytes using out-of-core processing that streams data from disk. Libraries like Dask or Vaex enable pandas-like APIs that automatically stream and parallelize computations across multiple cores. Before investing in distributed processing infrastructure, teams should exhaust single-machine optimization: using efficient data formats (Parquet²⁴ instead of CSV), minimizing memory allocations, using vectorized operations, and exploiting multi-core parallelism. The operational simplicity of single-machine processing (no network coordination, no partial failures, simple debugging) makes it preferable when performance is adequate.

When datasets exceed single-node throughput or storage limits, processing must scale across distributed clusters. Partitioning data across multiple computing resources introduces distributed coordination challenges. Distributed coordination is constrained by network round-trip times: local operations complete in microseconds while network coordination requires milliseconds, creating a $1,000\times$ latency difference. This constraint explains why operations requiring global coordination (like computing normalization statistics across 100 machines) create bottlenecks. Each partition computes local statistics quickly, but combining them requires information from all partitions.

Data locality becomes critical at this scale. At 10 GB/s peak throughput, transferring one terabyte of training data across a network takes on the order of 100 seconds; reading the same amount from a 5 GB/s SSD takes on the order of 200 seconds. These are the same order of magnitude, which drives ML system design toward compute-follows-data architectures.²⁵ When processing nodes access local data at RAM speeds (50–200 GB/s) but must coordinate over networks limited to 1–10 GB/s, the bandwidth mismatch creates severe bottlenecks. Geographic distribution amplifies these challenges: cross-data center coordination must handle network latency (50–200 ms between regions), partial failures, and regulatory constraints preventing data from crossing borders. Understanding which operations parallelize easily vs. those requiring expensive coordination determines system architecture and performance characteristics. This overhead constitutes a **coordination tax** that limits distributed data processing. The size of this tax, and whether centralizing or aggregating is faster, depends on the ratio between network round-trip and local compute time—derived in napkin math 4.7.

²⁴ | **Parquet:** A columnar storage format that organizes data by column, not by row. For the single-machine optimizations described, this is critical; instead of wastefully reading an entire CSV row to access a few columns, a Parquet reader loads only the specific columns needed for a computation. The resulting I/O reduction depends on the selected columns, encoding, compression, and workload.

²⁵ | **MapReduce:** Designed by Dean and Ghemawat (2004) at Google to process the company's multi-petabyte web index across thousands of commodity machines. Its scheduler preferentially assigns map tasks to nodes that hold the input data, reducing input-network traffic. This locality-aware pattern influenced Hadoop and later distributed data-processing systems.



Local aggregation beats gather-all when network latency dominates.

Napkin Math 4.7: The coordination tax

Problem: Mean normalization must be computed across 1 TB of features distributed across 100 nodes. Is it faster to (A) gather all data to one node and compute centrally, or (B) compute local means and aggregate them?

Option A (centralized):

- Transfer 1 TB at 10 GB/s network: 100 seconds
- Compute mean on single node: ~5 to 20 seconds at RAM bandwidth
- Total: ~105 to 120 seconds

Option B (distributed):

- Each node computes local mean: ~0.05 to 0.2 seconds (10 GB at RAM speed)
- Send 100 partial means (8 bytes each): < 1 ms
- Aggregate: negligible
- Total: ~0.05 to 0.2 seconds (hundreds to a few thousand times faster)

Systems insight: Algebraically decomposable reductions can often use partial aggregation near the data; for example, a distributed mean carries partial sums and counts. Joins and cross-products may require shuffling, although partitioning and colocation can reduce that cost. Pipeline design should minimize unnecessary movement by pushing suitable computation toward the data, the compute-follows-data principle central to systems like MapReduce (Dean and Ghemawat 2004), Spark (Zaharia et al. 2010), and modern ML frameworks.

Parallelizing data ingestion across distributed workers yields diminishing throughput returns whenever unpartitionable bottlenecks—such as sequential decompression or central database lock acquisition—limit concurrency. Amdahl’s Law²⁶ formalizes this scaling ceiling in equation 4.4:

$$\text{Speedup} \leq \frac{1}{f_{\text{serial}} + \frac{f_{\text{parallel}}}{N_{\text{workers}}}} \quad (4.4)$$

where f_{serial} denotes the execution fraction spent in inherently sequential stages, f_{parallel} represents the parallelizable fraction ($f_{\text{serial}} + f_{\text{parallel}} = 1$), and N_{workers} denotes active worker processes. Even when 90 percent of a feature extraction pipeline parallelizes across hundreds of nodes ($f_{\text{serial}} = 0.10$), the maximum theoretical speedup cannot exceed 10×, demonstrating why eliminating serial I/O bottlenecks takes precedence over adding compute nodes.

Framework choice follows the same coordination question as the Amdahl analysis: it depends on which parts of the transformation can run independently and which parts need shared state or ordering. Apache Spark parallelizes transformations across clusters of machines, handling data partitioning, task scheduling, and fault tolerance automatically. Beam provides a unified API for both batch and streaming processing, enabling the same transformation logic to run on multiple execution engines (Spark, Flink, Dataflow). TensorFlow’s `tf.data` API optimizes data loading pipelines for ML training, supporting distributed reading, prefetching, and transformation. The choice depends on whether processing is batch or streaming, how transformations parallelize, and what execution environment is available.

The feature computation placement trade-off introduced in section 4.5.5.3 takes on additional significance at scale. When distributed processing increases throughput, the cost of recomputing features across hundreds of workers per epoch must be weighed against the storage cost of materializing those features once. At terabyte scale, even small per-example compute costs multiply into significant overhead, reinforcing why production systems adopt hybrid patterns: precomputing expensive, stable features while computing cheap, time-sensitive features on-the-fly.

Scalability in the KWS pipeline manifests at multiple stages. Development uses single-machine processing on sample datasets to iterate rapidly. Training at scale may require distributed processing when the dataset (23.4 million examples) exceeds a machine’s practical capacity or when experiments run concurrently. Per-file feature extraction is embarrassingly parallel, but end-to-end speedup is

²⁶ | **Amdahl’s law:** Amdahl (1967) presented the serial-fraction argument at the 1967 AFIPS Spring Joint Computer Conference to explain why multiprocessor designs face diminishing returns. The original point—that the serial fraction of a workload imposes a hard ceiling on parallelism—applies directly to data pipelines: operations like computing global normalization statistics force serial aggregation phases that cap speedup regardless of how many workers process individual records in parallel.

bounded by shared storage bandwidth, metadata operations, scheduling, and output contention. Production deployment adds a stricter 16 KB preprocessing-state budget alongside the 64 KB model-size limit, necessitating careful footprint optimization to fit processing within device capabilities.

4.6.4 Transformation lineage

Governance completes the four-pillar view of data processing by ensuring accountability and reproducibility. The governance pillar requires tracking what transformations were applied, when they executed, which version of processing code ran, and what parameters were used. This **transformation lineage**²⁷ supports reproducibility, debugging, auditability, and iterative improvement when transformation bugs are discovered. It can supply evidence for documentation or compliance workflows, but lineage alone does not explain a model's decision or establish regulatory compliance.

Transformation versioning captures which version of processing code produced each dataset. When transformation logic changes (fixing a bug, adding features, or improving quality), the version number increments. Datasets are tagged with the transformation version that created them, enabling identification of all data requiring reprocessing when bugs are fixed. This versioning extends beyond just code versions to capture the entire processing environment: library versions (different NumPy versions may produce slightly different numerical results), runtime configurations (environment variables affecting behavior), and execution infrastructure (CPU architecture affecting floating-point precision).

Parameter tracking maintains the specific values used during transformation. For normalization, this means storing the mean and standard deviation computed on training data. For categorical encoding, this means storing the vocabulary (set of all observed categories). For feature engineering, this means storing any constants, thresholds, or parameters used in feature computation. These parameters are typically serialized alongside model artifacts, ensuring serving uses identical parameters to training. Modern ML frameworks like TensorFlow and PyTorch provide mechanisms for bundling preprocessing parameters with models, simplifying deployment and ensuring consistency.

Processing lineage for reproducibility tracks the complete transformation history from raw data to final features. This includes which raw data files were read, what transformations were applied in what order, what parameters were used, and when processing occurred. Lineage systems like Apache Atlas, Amundsen, or commercial offerings instrument pipelines to automatically capture this flow. When model predictions prove incorrect, engineers can trace back through lineage to identify the training data that contributed to the behavior, the quality scores attached to that data, the transformations applied, and whether the exact scenario can be recreated for investigation.

Recording the exact code version ties processing results directly to the repository state that generated them. When processing code lives in version control (Git), each dataset should record the commit hash of the code that created it. This enables recreating the exact processing environment: checking out the specific code version, installing dependencies listed at that version, and running processing with identical parameters. Container technologies like Docker simplify this by capturing the entire processing environment (code, dependencies, system libraries) in an immutable image that can be rerun months or years later with identical results.

The governance pillar in the KWS pipeline tracks audio processing parameters that critically affect model behavior. When audio is normalized to standard volume, the reference volume level is persisted. When an FFT transforms audio into the frequency domain, the window size, hop length, and window function (such as Hamming or Hann) are recorded. When MFCCs are computed, the number of coefficients, frequency range, and mel filterbank parameters are captured. This comprehensive parameter tracking enables several critical capabilities: reproducing training data exactly when debugging model failures, validating that serving uses identical preprocessing to training, and systematically studying how preprocessing choices affect model accuracy. Without this governance infrastructure, teams resort to manual documentation that inevitably becomes outdated or incorrect, leading to subtle training-serving skew that degrades production performance.

Clean, normalized, feature-ready data remains inert without a supervisory signal. That boundary matters operationally because a transformation can be perfectly reproducible yet attach an erroneous target, and deterministic preprocessing cannot compensate for corrupted ground truth. Supervising model training requires assigning target values—declaring which audio segments contain

²⁷ | **Data lineage:** When a model produces an erroneous or discriminatory prediction, lineage helps engineers determine whether the contributing problem originated in raw data, feature computation, training, or serving. The resulting trace can support incident investigation and jurisdiction-specific documentation duties, but its usefulness depends on complete instrumentation and retained history; it does not guarantee an explanation of model behavior.

wake words and which capture background noise—introducing subjective human judgment and probabilistic heuristics into what had been a deterministic pipeline.

4.7 Data Labeling

The processing pipelines in section 4.6 transform raw data into structured features, but supervised learning still requires labels that tell the model which patterns correspond to each target. In the KWS pipeline, the ingestion and processing stages produce millions of clean, standardized audio spectrograms, but training requires ground-truth annotations indicating which segments contain the wake word and which represent ambient background noise. This declaration is the **ground truth**,²⁸ and producing it at scale is often the most human-dependent and error-prone stage of the pipeline.

Unlike automated transformations that can be parallelized across machines, labeling introduces human judgment into the pipeline, creating non-deterministic latency and stochastic error modes. A crowdsourced annotator might mislabel a whispered “Alexa” as background noise. An expert radiologist might disagree with a colleague about a borderline diagnosis. Such disagreement can reflect genuine ambiguity, unclear instructions, annotator error, or differences in expertise; the labeling system must distinguish and manage these causes rather than treating every disagreement as irreducible. The labeling infrastructure must therefore balance throughput, quality verification, cost, and provenance.

4.7.1 Label types and system requirements

System architecture, storage footprint, and loader throughput depend directly on label granularity. In a computer vision system tracking vehicles, pedestrians, and traffic signs from video feeds, each target representation imposes distinct storage, computation, and serialization constraints.

Classification labels represent the simplest form, assigning a discrete class index (or a multi-label binary bitmask) to each input. Storage requirements are modest—typically a fixed-width integer (such as a `uint16` or `int32` index) per image—but access patterns diverge between training and evaluation: training pipelines shuffle and sample randomly, whereas validation pipelines stream sequentially across the corpus, driving different indexing and cache-locality strategies.

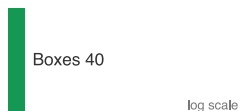
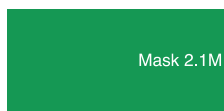
Bounding boxes extend beyond classification by localizing objects within each frame. Each bounding box stores four coordinates (such as $x_{\min}, y_{\min}, x_{\max}, y_{\max}$ or center coordinates with width and height) alongside the class identifier. Because scenes contain variable numbers of objects, label storage transitions from fixed-width scalar records to variable-length tensor structures, requiring dynamic padding or ragged-batch handling in the data loader. Moreover, positioning bounding boxes takes 10 to 20× longer than single-label classification, directly throttling annotation throughput and inflating labeling cost.

Segmentation maps assign a class identifier to every pixel, producing dense 2D label matrices ($H \times W$). In autonomous driving and traffic monitoring, this entails tracing irregular boundaries for driveable road surfaces, vehicles, and pedestrians. These dense annotations multiply storage and ingestion bandwidth. A segmentation mask for a 1920×1080 image requires about 2.1M labels (one per pixel), compared to perhaps 10 bounding boxes or a single classification label. If each box stores 4 coordinates, that is roughly 51,840× more scalar label entries than 10 boxes before accounting for per-value encoding, and the hours required per image for manual segmentation make this approach suitable only when pixel-level precision is essential.

Figure 4.11 contrasts five annotation modalities, and the choice depends on the input, task, and available annotation budget. Classification can label an entire scene, while bounding boxes and segmentation maps localize objects or regions. Production datasets may combine modalities: a single camera frame might carry a scene label, obstacle boxes, and path-region masks, with each label type serving a distinct downstream task.

Beyond these geometric labels, production systems must also manage rich metadata essential for quality control and debugging. The Common Voice dataset (Ardila et al. 2020) exemplifies this in speech recognition: tracking speaker demographics for fairness, recording quality metrics for filtering, and language information for multilingual support. If an object detection model fails in rainy conditions, weather metadata captured during collection pinpoints the coverage gap. This metadata requirement demonstrates how label type choice cascades through the entire system

²⁸ | **Ground truth:** From remote sensing, where orbital measurements are verified by sending a team to the physical location—the “ground”—to establish the “truth.” The etymology carries a systems warning: ML labels are proxies for reality, not reality itself. When a crowdsourced annotator labels an image as “cat,” that label reflects the annotator’s judgment, not an objective fact. Every downstream metric—accuracy, precision, recall—is measured against this proxy, meaning label quality errors propagate silently into every evaluation of the model.



log scale

Finer annotation granularity multiplies storage and processing scale.

Label Type	Input Type	Output Type
Classification Label		Dog, Blanket, No cat
Bounding Box		
Segmentation Map		
Caption		A dog curls up on a spotted purple blanket.
Transcript		Once upon a time...

Figure 4.11: Data Annotation Modalities: Four rows show classification, bounding-box, segmentation, and caption labels for one image; the fifth shows a transcript label for an audio input. The appropriate annotation representation depends on the input modality and task.

design: the infrastructure must optimize storage for the chosen format, implement appropriate retrieval patterns, and track which model versions used which label versions to correlate quality improvements with performance gains.

4.7.2 Label accuracy and consensus

Label quality requires bounding label noise despite inherent subjectivity and sensor ambiguity across large datasets. Even with clear guidelines and careful system design, a nonzero fraction of labels will inevitably be corrupted (Northcutt et al. 2021; Thyagarajan et al. 2022). The challenge is not eliminating labeling errors entirely (an impossible goal) but systematically measuring inter-annotator agreement and managing error rates to keep them within bounds that do not degrade model performance.

Labeling failures arise from two distinct sources requiring different engineering responses. Figure 4.12 presents concrete examples of both failure modes. Some examples reflect degraded or ambiguous inputs where the correct label is difficult to infer from the data alone; others are visually clear but require domain knowledge, dataset-specific semantics, or expert judgment to label correctly. These different failure modes drive architectural decisions about annotator qualification, task routing, and consensus mechanisms: quality-based errors call for upstream data filtering, while expertise-based errors call for tiered annotator routing.



Figure 4.12: Pervasive Label Errors: Confidently mislabeled examples from widely used benchmarks, each showing the given label against the corrected ground truth (MNIST, CIFAR-10, CIFAR-100, Caltech-256, ImageNet, QuickDraw). Fine-grained confusions, such as a crab labeled lobster or a black stork labeled white stork, show that even curated datasets carry systematic errors, motivating careful quality control and expert annotation. Source: (Northcutt et al. 2021).

Given these inherent quality challenges, production ML systems implement multiple layers of quality control. Systematic quality checks continuously monitor the labeling pipeline through random sampling of labeled data for expert review and statistical methods to flag potential errors. The infrastructure must efficiently process these checks across millions of examples without creating bottlenecks. Sampling strategies typically validate 1-10 percent of labels, balancing detection sensitivity against review costs. Higher-risk applications like medical diagnosis or autonomous vehicles may

validate 100 percent of labels through multiple independent reviews, while lower-stakes applications like product recommendations may validate only 1 percent through spot checks.

Beyond random sampling approaches, collecting multiple labels per data point, often referred to as “consensus labeling,” can help identify controversial or ambiguous cases. Commercial labeling platforms such as Labelbox and Scale AI expose consensus and tiered quality-control workflows (Labelbox, Inc. 2024; Scale AI, Inc. 2024), but the statistical core is inter-annotator agreement. The consensus infrastructure typically collects several labels per example, computing metrics like Fleiss’ kappa, a generalization of the Cohen’s kappa statistic introduced in section 4.5.3 from two raters to any number of annotators (Fleiss 1971). Examples with low agreement, using thresholds such as the Landis-Koch bands as operational heuristics, route to expert review rather than forcing consensus from genuinely ambiguous cases (Landis and Koch 1977).

The consensus approach reflects an economic trade-off essential for scalable systems. Expert review costs more per example than crowdsourced labeling, but forcing agreement on ambiguous examples through majority voting of nonexperts can produce systematically biased labels. By routing only genuinely ambiguous cases to experts, identified through low inter-annotator agreement or failed gold-standard checks, systems balance cost against quality. This tiered approach enables processing millions of examples economically while maintaining quality standards through targeted expert intervention.

While technical infrastructure provides the foundation for quality control, successful labeling systems must also address human factors. Working effectively with annotators requires standardized annotation rubrics with canonical positive and negative examples, visual edge-case catalogs, automated feedback mechanisms benchmarking annotators against known gold-standard records, and calibration sessions where annotators resolve borderline cases. For complex or domain-specific tasks, the system can implement tiered access levels, routing challenging cases to annotators with demonstrated domain expertise.

Quality monitoring generates substantial telemetry that must be efficiently ingested and tracked. The most informative signals span several dimensions. Inter-annotator agreement rates reveal whether multiple annotators converge on the same example, while label confidence scores capture annotators’ reported confidence in their decisions. Time per annotation serves as a dual-sided indicator: annotations completed too quickly suggest carelessness, while those taking too long suggest confusion or unclear guidelines. Error patterns expose systematic biases or misunderstandings in the annotator pool, and annotator performance on gold-standard examples provides ground-truth calibration. Finally, demographic analysis of annotator behavior detects whether certain groups systematically label differently, which could introduce unintended bias into the training data. These metrics must be computed and updated efficiently across millions of examples, often requiring dedicated analytics pipelines that process labeling data in near real-time to catch quality issues before they affect large volumes of data.

4.7.3 Scaling with AI-assisted labeling

Throughput limits on manual annotation make pure human labeling a bottleneck for large-scale training, while fully automated labeling lacks the reliability needed for ambiguous or safety-critical examples. AI-assisted labeling navigates this trade-off by using model inference to process routine inputs and accelerate human annotation while preserving manual verification for high-uncertainty samples. Figure 4.13 maps four common paths. Traditional supervision uses direct human labels. Semi-supervised learning exploits structure in unlabeled data alongside a labeled subset. Weak supervision replaces individual manual annotations with programmatic labeling functions. Transfer learning reuses representations learned on an upstream task, reducing but not eliminating task-specific labels. Each path alters where supervision cost is paid and which assumptions—unlabeled data geometry, programmatic heuristic accuracy, or representation transferability—the pipeline must validate.

These paths exchange different assumptions rather than simple label counts. Depending on the path, the pipeline relies on task-relevant structure in unlabeled data, a transferable representation, measurable labeling-function accuracy, or an affordable scoring loop. Production systems must record each assumption and validate it against held-out, human-audited data. The diagram functions as a decision map, not an absolute quality ranking.

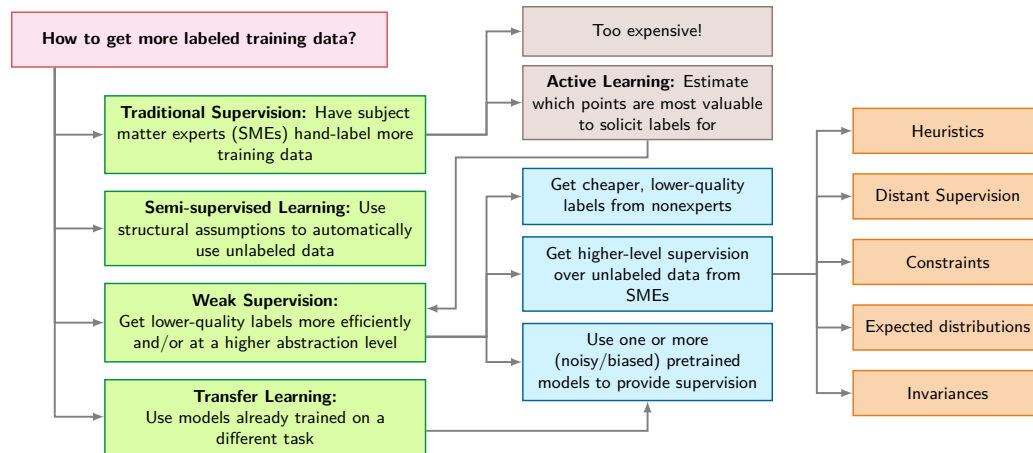


Figure 4.13: AI-Augmented Labeling Decision Hierarchy: A top-level question about obtaining labeled data branches into four paths: traditional supervision, semi-supervised learning, weak supervision, and transfer learning, with active learning as a cost-saving alternative. The branches distinguish where supervision comes from and what additional assumptions or pretrained artifacts the pipeline must validate. Adapted from the Stanford AI Lab weak-supervision overview (Ratner et al. 2019).

AI-assisted labeling divides work according to the strengths of each participant. Humans judge ambiguous cases, catch subtle errors, and apply domain knowledge; models can process routine cases at scale, subject to validation. Production systems combine these capabilities through several complementary approaches.

Pre-annotation uses AI models to generate preliminary labels that humans then review and correct—transforming the task from “label from scratch” to “verify and fix.” Programmatic labeling frameworks like Snorkel (Ratner et al. 2018; Ratner et al. 2017) extend this further through weak supervision,²⁹ automatically generating initial labels at scale through rule-based heuristics, knowledge bases, and existing model outputs. In autonomous driving, pretrained object detection models can label vehicles and pedestrians that human annotators verify and refine, handling many clear cases automatically.

Large language models (LLMs) now assist labeling pipelines by generating descriptions, drafting labeling guidelines from examples, and explaining their reasoning for label assignments. Content moderation systems, for instance, use LLMs for initial content classification with explanations that human reviewers validate. However, LLM integration introduces systems challenges: provider- and model-dependent inference costs and rate limits, as well as the need for systematic output validation because LLMs can produce confident but incorrect labels. Many organizations adopt tiered approaches, using smaller specialized models for routine cases while reserving larger LLMs for complex scenarios requiring nuanced judgment.

Active learning makes the complementary trade-off: it spends model inference to reduce human labeling, so the relevant question is whether that compute fits inside the same budget.

Methods such as active learning³⁰ complement these approaches by prioritizing candidate examples according to model uncertainty (Settles 2009; Coleman et al. 2022). These systems continuously analyze model uncertainty to identify valuable labeling candidates. Rather than labeling a random sample of unlabeled data, active learning selects examples where the current model is most uncertain or where labels would most improve model performance. The infrastructure must efficiently compute uncertainty metrics (often prediction entropy or disagreement between ensemble models), maintain task queues ordered by informativeness, and adapt prioritization strategies based on incoming labels. In a medical imaging pipeline, for example, active learning routes rare or borderline pathologies to expert radiologists while routine scans receive automated preannotations that clinicians quickly verify. This approach can substantially reduce required annotations in favorable settings, though it requires careful engineering to prevent feedback loops where the model’s uncertainty biases which data gets labeled. A budget calculation makes that leverage concrete.

²⁹ | **Weak supervision:** A “data programming” paradigm that exchanges some manual annotation labor for the upfront effort of writing and validating programmatic labeling functions. Once written, a function can be applied cheaply at scale, but execution, conflict resolution, quality monitoring, and maintenance still incur cost; the useful comparison is workload-specific rather than literally zero marginal cost.

³⁰ | **Active learning:** Inverts the traditional labeling paradigm: instead of randomly selecting examples to label, the model queries for the examples it needs most, typically those where prediction uncertainty is highest (Settles 2009). This can reduce the number of labels needed to reach a target accuracy, but the infrastructure trade-off is compute for labels: at \$0.01/image, scoring a full 10M pool costs \$100K before any human labels. Active learning becomes budget leverage only when the candidate pool is pre-filtered, inference is much cheaper, or the compute budget is separate from the labeling budget.

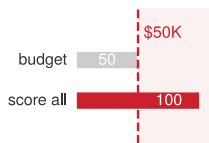
Napkin Math 4.8: The active learning multiplier

Problem: A 10M dataset has a \$50K labeling budget. Random sampling achieves 85 percent accuracy with 100K, while the target is 95 percent accuracy.

Physics:

1. **Sample efficiency:** Active learning can achieve target accuracy with fewer samples than random selection in favorable settings.
2. **Cost per point:** Random sampling = \$0.50/label. Active learning adds compute cost (~\$0.01/image for inference) to find hard examples.
3. **Multiplier:**
 - **Random:** Reaching 95 percent may require 1M labels (\$500K). Budget exceeded.
 - **Active labels only:** The system may need ~100K–200K hard examples (\$50K–\$100K).
 - **Full-pool scoring:** Scoring all 10M candidate images adds \$100K, so total active-learning cost becomes \$150K–\$200K. Budget exceeded.

Systems insight: Algorithm choice is a major lever on label count, but compute must be inside the budget model. Spending 10 percent of this budget on inference (\$5K) scores only 500K candidate images at the stated inference price and leaves room for about 90K labels. Active learning is viable only if the candidate pool is narrowed before scoring, inference cost drops substantially, or compute is funded separately from labeling.



Full-pool scoring can exceed the label budget before labeling begins.

Quality control becomes increasingly important as these AI components interact. The system must monitor both AI and human performance through systematic metrics. Model confidence calibration matters: if the AI reports 95 percent confidence but achieves only 75 percent accuracy at that confidence level, preannotations mislead human reviewers. Human-AI agreement rates reveal whether AI assistance helps or hinders: when humans frequently override AI suggestions, the preannotations may be introducing bias rather than accelerating work. These metrics require careful instrumentation throughout the labeling pipeline, tracking both final labels and the interaction between human annotators and AI at each stage.

These principles manifest at scale across safety-critical domains. Autonomous vehicle labeling infrastructure can process large volumes of sensor frames, using AI preannotation to label common objects while routing unusual scenarios (construction zones, emergency vehicles) to human experts—a distributed architecture where preannotation runs on GPU clusters while human review scales across annotation teams. Medical imaging systems face a parallel label-scarcity problem: large repositories of unlabeled clinical data make expert annotation the bottleneck and motivate data-efficient workflows that learn useful representations from unlabeled structure before expert labels are applied (Krishnan et al. 2022). Across such domains, the common data-engineering pattern is *tiered escalation*: automation handles clear cases, humans handle ambiguous ones, and monitoring ensures the boundary between “clear” and “ambiguous” adapts as both AI capability and deployment conditions evolve.

4.7.4 Automated labeling in KWS

Labeling keyword spotting datasets at scale introduces speech-specific systems challenges. Generating millions of labeled wake word samples without proportional human annotation cost requires moving beyond manual and crowdsourced workflows. The Multilingual Spoken Words Corpus (MSWC) (Mazumder et al. 2021) demonstrates how automated labeling resolves this throughput bottleneck: the corpus programmatically extracts over 23.4 million examples of one-second speech across 340,000 keywords in 50 languages from continuous speech corpora.

This scale makes manual annotation impractical: 23.4 million examples at even 10 seconds per label would require approximately 65,000 hours, roughly 32.5 person-years of full-time effort. Broad, documented sourcing across 50 languages can improve coverage, but language count alone does not guarantee representative speakers, accents, devices, or acoustic environments. The automated system in figure 4.14 addresses the scale problem by starting with paired sentence audio and transcriptions, then using forced alignment³¹ to estimate word boundaries within continuous speech.

³¹ | **Forced alignment:** Given a known transcription, an aligner estimates where its words occur in the audio, often using frame-level acoustic scores and sequence decoding. Timing resolution and accuracy depend on the model, features, language, and recording quality, so boundary estimates still need quality checks. The known transcript makes word-level segmentation far cheaper than labeling every clip manually, though computation, review, and error correction remain.

At this scale, quality control becomes a sampling and provenance problem rather than disappearing. The pipeline should report coverage by language, speaker, accent, device, and acoustic environment; retain the source recording, transcript version, aligner version, and boundary confidence for each extracted clip; and route low-confidence alignments and underrepresented slices to human review. Held-out evaluation must then measure the intended wake words across those slices. These controls distinguish a large corpus from a representative one and make extraction errors traceable when a downstream KWS model fails.

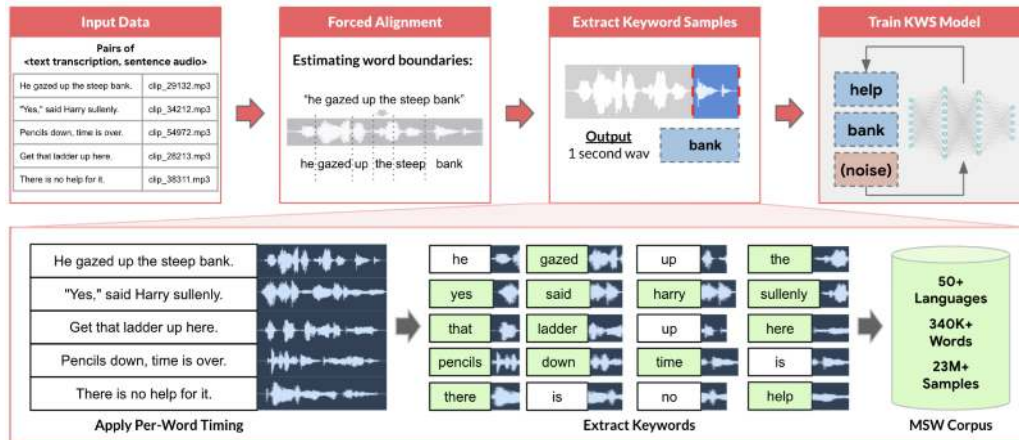


Figure 4.14: Multilingual Data Preparation: Forced alignment and segmentation transform paired audio-text data into labeled one-second segments, creating a large-scale corpus for training keyword spotting models across 50 languages. This automated process enables scalable development of KWS systems by efficiently generating training examples from readily available speech resources like Common Voice and multilingual captioned content. Source: (Mazumder 2021).

The extraction system uses these precise timing markers to generate clean keyword samples while handling anticipated engineering challenges: background noise interfering with word boundaries, speakers stretching or compressing words unexpectedly beyond the target 500 ms to 800 ms duration, and longer words exceeding the one-second boundary. MSWC provides automated quality assessment that analyzes audio characteristics to identify potential issues with recording quality, speech clarity, or background noise, which is essential for maintaining consistent standards across 23.4 million samples without the manual review expenses that would make this scale prohibitive.

Voice assistant engineering teams often build on this automated labeling foundation. While automated corpora may not contain the specific wake words a product requires, they provide starting points for KWS prototyping, particularly in underserved languages where commercial datasets do not exist. Production systems typically layer targeted human recording and verification for challenging cases (unusual accents, rare words, or difficult acoustic environments), coordinating between automated processing and human expertise.

At this stage, the compilation pipeline has produced its core artifacts: feature vectors paired with ground-truth labels. Delivering these tensors to execution units exposes the physical storage hierarchy. The fundamental tension is mechanical: batch training demands high-throughput sequential streaming across petabytes of records, while real-time inference requires sub-millisecond point lookups for individual feature vectors. Storage architecture determines whether accelerators compute continuously or idle waiting on I/O.

4.8 Storage Architecture

The labeled datasets produced by the pipeline (23.4 million samples spanning 50 languages in the KWS case study) require deliberate storage layouts to balance training throughput against serving latency. Reconciling these divergent access profiles demands mapping workload access patterns directly onto the physical capabilities of underlying storage media.

Storage requirements for machine learning diverge fundamentally from those of transactional systems. Rather than optimizing for frequent small writes and point lookups that characterize

banking or e-commerce, ML workloads prioritize sustained sequential read bandwidth, large-scale scans across gigabytes to petabytes, and schema flexibility. A relational database serving an e-commerce catalog handles millions of individual product queries per second with low latency, but an ML training job scanning that same catalog sequentially across dozens of epochs stalls on the uncoalesced disk reads inherent to transactional storage engines.

4.8.1 Storage system options

Batch training scans millions of examples sequentially; real-time serving fetches one feature vector at a time. These opposing access patterns pull storage in two directions, and selecting a system means minimizing the data term ($\frac{D_{vol}}{BW}$) of the iron law of ML systems for whichever pattern dominates. Every storage medium imposes physical constraints on bandwidth that determine the maximum speed of the training and serving pipelines.

Two storage performance metrics govern this optimization. **IOPS (input/output operations per second)** counts the distinct read or write requests a device can handle per second, bounding *random access* workloads such as fetching small batches of images or individual user profiles. **Throughput (bandwidth)** measures the volume of data transferred per second, typically $IOPS \times \text{Block Size}$, bounding *sequential access* workloads such as scanning a Parquet file for training.

The choice between databases, data warehouses, and data lakes is fundamentally a choice about which of these metrics to optimize across distinct lifecycle stages. Conventional databases (online transaction processing (OLTP)) optimize for high IOPS with small block sizes and row-oriented layouts. They maintain product catalogs, user profiles, or transaction histories with strict ACID guarantees and sub-millisecond point-lookup latencies. In ML workflows, this profile fits online feature serving: a recommendation system looking up a user profile during real-time inference or an anti-fraud service querying historical transaction counts requires random point lookups bounded by per-request latency. At the extreme, large recommendation systems serve billions of sparse feature reads per second from terabyte-scale tables, requiring storage engines that maximize IOPS over bulk throughput. However, databases fail as training data sources. When a training job scans millions of records across multiple epochs, the row-oriented layout forces the storage engine to fetch entire rows from disk into memory even when the model consumes only 20 of 100 features. The resulting uncoalesced I/O and wasted bus bandwidth throttle throughput.

Data warehouses (online analytical processing (OLAP)) invert this physical layout by organizing tables by column rather than by row (Stonebraker et al. 2018). By storing values of each attribute contiguously on disk, columnar formats allow sequential scans to read only the features required by the model, transferring one fifth of the uncompressed row payload when selecting 20 out of 100 columns. The format-efficiency calculation in section 4.8.2 quantifies this gain with a worked fraud-detection scenario. Columnar layouts also achieve high compression ratios across homogeneous data types, directly reducing the bytes transferred over the storage bus. Data warehouses excel for feature engineering, batch aggregations, and tabular model training where SQL interfaces simplify iterative data exploration. However, warehouses require structured schemas and struggle with multi-modal artifacts such as raw audio, high-resolution imagery, and free-form text. Modifying schemas during rapid experimentation imposes costly ALTER TABLE operations across billion-row tables, stalling development velocity.

Data lakes abandon predefined tabular schemas entirely by storing structured, semi-structured, and unstructured data in native formats on high-capacity object storage. They defer data structure definitions until query time, an architectural pattern known as schema-on-read.³²

This layout suits exploratory data engineering and multi-modal training pipelines where feature sets evolve continuously. A computer vision or multimodal system can store raw sensor logs as JSON, images as JPEG files, clickstreams as Parquet, and learned representations as binary NumPy arrays within the same repository. Independent training jobs impose schemas at read time, extracting different subsets without coordinating schema migrations. However, schema-on-read shifts governance from ingestion to retrieval. Without disciplined metadata management, data lakes degrade into “data swamps,” disorganized repositories containing orphaned directory trees like `userdata_v2_final` and `userdata_v2_ACTUALLY_FINAL` whose lineage and quality are unrecorded. Production data lakes require a dedicated metadata catalog (such as AWS Glue Data Catalog, Apache

³² | **Schema-on-read:** Applies data structure definitions at query time rather than during ingestion, contrasting with schema-on-write (traditional databases) where data must conform to a predefined structure before storage. For ML pipelines in early development, schema-on-read enables rapid experimentation—teams can store raw sensor data, images, and logs without committing to a feature schema upfront. The trade-off is governance: without enforced schemas, data lakes degrade into “data swamps” where finding and validating training data becomes the bottleneck instead.

Atlas, or Databricks Unity Catalog) to record lineage, update cadence, schema versions, and access policies.

Each storage architecture optimizes for a distinct access pattern and lifecycle phase, as table 4.7 summarizes.

Table 4.7: Storage System Characteristics: Each storage architecture favors a different access pattern and ML workflow stage. Databases support the high-IOPS random access needed for real-time feature serving; data warehouses provide sequential throughput for batch training and feature engineering; and data lakes provide schema flexibility and cost efficiency for large-scale raw data retention. A poor match between system and workload can impose substantial performance penalties.

Attribute	Conventional Database	Data Warehouse	Data Lake
Purpose	Operational and transactional	Analytical and reporting	Storage for raw and diverse data for future processing
Data type	Structured	Structured	Structured, semi-structured, and unstructured
Scale	Small to medium volumes	Medium to large volumes	Large volumes of diverse data
Performance Optimization	Optimized for transactional queries (OLTP)	Optimized for analytical queries (OLAP)	Optimized for scalable storage and retrieval
Examples	MySQL, PostgreSQL, Oracle DB	Google BigQuery, Amazon Redshift, Microsoft Azure Synapse	Google Cloud Storage, AWS S3, Azure Data Lake Storage

Production ML architectures rarely select a single storage technology. Instead, mature systems combine all three tiers into a unified data topology: databases handle operational telemetry and real-time point lookups; warehouses run structured feature extraction and aggregate analytics; and data lakes store multi-modal raw captures and intermediate checkpoints. In an autonomous vehicle pipeline, for example, vehicle telemetry streams to an operational database for immediate health monitoring, aggregated driving statistics populate a warehouse for fleet analytics, and raw camera frames and lidar point clouds reside in an object store for model training.

4.8.2 Storage performance and cost

Physical storage media span three orders of magnitude in cost and five orders of magnitude in bandwidth. Mismatching the storage tier to the access pattern either inflates infrastructure costs with underutilized capacity or starves training accelerators during I/O stalls.

Table 4.8 shows why ML systems use tiered storage. Under these 2024 assumptions, storing our KWS training dataset (748.8 GB) in object storage costs \$17.2/month, enabling affordable raw-audio retention, while working datasets on Non-Volatile Memory Express (NVMe)³³ cost \$74.9/month—\$224.6/month for active training but load 50× faster.

Table 4.8: Illustrative Storage Cost-Performance Trade-Offs (2024): These planning ranges vary by provider, region, configuration, and workload. Training data loading requires high-throughput sequential access; section D.3.3 establishes the memory hierarchy this pattern must align with. Online serving needs low-latency random reads, while archival storage prioritizes cost over access speed for compliance and historical data.

Storage Tier	Cost (\$/TB/month)	Sequential Read Throughput	Random Read Latency	Typical ML Use Case
NVMe SSD (local)	\$100–300	5–7 GB/s	10–100 μs	Training data loading, active feature serving
Object Storage (S3, GCS)	\$20–25	100–500 MB/s (per connection)	10–50 ms	Data lake raw storage, model artifacts
Data Warehouse (BigQuery, Redshift)	\$20–40	1–5 GB/s (columnar scan)	100–500 ms (query startup)	Training data queries, feature engineering
In-Memory Cache (Redis, Memcached)	\$500–1000	20–50 GB/s	1–10 μs	Online feature serving, real-time inference
Archival Storage (Glacier, Nearline)	\$1–4	10–50 MB/s (after retrieval)	Hours (retrieval)	Historical retention, compliance archives

This throughput divergence governs engineering iteration velocity. Training that loads data at 5 GB/s completes dataset loading in 149.8 s, compared to 7,488 s at typical object storage speeds.

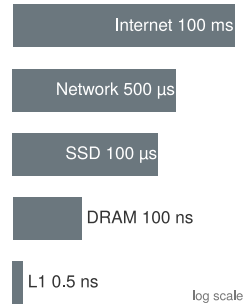
³³ | **NVMe (Non-Volatile Memory Express):** A storage protocol connecting directly to the PCIe bus with 64K command queues, delivering 5–7 GB/s sequential throughput and microsecond-scale latency. The contrast with SATA SSD (500 MB/s, single queue) is a 10× bandwidth gap that can bottleneck a training pipeline when storage service time exceeds compute time.

This 50× speedup determines whether engineers can iterate multiple times daily or must wait hours between experiments.

The latency gap between on-chip registers and remote storage spans over eight orders of magnitude, causing unbuffered remote I/O requests to stall GPU execution pipelines. Scaling these microsecond delays to human timeframes (table 4.9) illustrates the performance penalty of an uncached fetch. Popularized by Jeff Dean’s 2009 LADIS keynote,³⁴ these figures quantify why unbuffered remote I/O starves modern accelerators and why distributed training demands strict data locality.

Table 4.9: Memory and Storage Latency Hierarchy: Standard hardware access times normalized to a human scale where a 0.5 ns L1 cache access represents one second.

Operation	Latency (ns)	Human Scale	ML System Impact
L1 Cache Reference	0.5	1 second	Immediate
L2 Cache Reference	7	14 seconds	Fast computation
Main Memory (DRAM)	100	3 minutes	The “memory wall” threshold
SSD (local NVMe)	100,000	2 days	Data loading bottleneck
Network (same DC)	500,000	11.57 days	Distributed coordination lag
SSD (remote network)	2,000,000	46.30 days	Training-serving skew source
Object Store (S3)	20,000,000	1 year	Archival access
Internet (CA to VA)	100,000,000	6 years	Global user experience



Access latency spans eight orders of magnitude, cache to internet.

³⁴ | **Jeff Dean:** Google Senior Fellow, architect of MapReduce, BigTable, Spanner, and TensorFlow. His 2009 LADIS keynote distilled the numbers in table 4.9 into the engineering heuristic that an L1 cache reference (0.5 ns) and a cross-data center round trip (150 ms) span a 3×10^8 ratio—eight orders of magnitude that explain why a training pipeline reading features from remote storage instead of local NVMe starves the accelerator it was meant to feed.

Beyond input datasets, storage architectures must accommodate dense numerical arrays generated during model development: parameter weights, optimizer states, and training checkpoints.

Modern neural networks contain millions to hundreds of billions of parameters, imposing storage and retrieval patterns distinct from tabular data. GPT-3 (Brown et al. 2020) requires approximately 700 GB for model weights when stored in FP32 format (175B parameters times 4 bytes), though practical deployments often use smaller numeric formats such as FP16 (350 GB) to halve memory pressure. Even at FP16 precision, this single checkpoint exceeds many organizations’ entire operational databases. From AlexNet’s 60M parameters (Krizhevsky et al. 2012) to GPT-3’s 175B parameters (Brown et al. 2020), model size grew approximately 2916× over eight years. Model weights require block-aligned storage formats to support parallel reads across parameter groups during initialization and checkpointing, demanding aggregate storage bandwidth approaching network fabric limits (25 to 100 Gbps) to keep compute units active. Chapter 10 examines model-side quantization techniques that compress these footprints.

Tracking these artifacts introduces versioning challenges. Standard version control systems like Git excel at incremental text diffs, but fail when tracking multi-gigabyte binary files where a single weight update generates an entirely new file. Storing ten historical checkpoints of a 10 GB model naively consumes 100 GB. Production ML workflows decouple metadata from binary artifacts: tools such as DVC (Data Version Control) and MLflow store lightweight cryptographic hashes in Git while offloading binary checkpoints to external content-addressed object stores. This enables precise reproducibility without bloating the code repository.

Multi-accelerator training compounds storage pressure through concurrent I/O. When a training job spans multiple accelerators, every rank must read training batches, write optimizer snapshots, and dump intermediate checkpoints concurrently. Storage systems must absorb simultaneous reads and writes at rates proportional to the worker count, preventing I/O serialization that leaves expensive accelerators idle. The coordination mechanisms governing these parallel workers are analyzed in chapter 8.

This physical mismatch between compute and storage bandwidth exposes the storage starvation cliff. While host RAM delivers 50 to 200 GB/s on modern server nodes, local NVMe SSDs deliver 1 to 7 GB/s, and network-attached storage provides 1 to 10 GB/s. When an accelerator can process data faster than the storage hierarchy can stage it, the data pipeline becomes the binding constraint. A 10-fold mismatch between GPU consumption rate and storage bandwidth forces accelerators to idle 90 percent of the time. Training throughput is bounded by the minimum of compute capacity and data supply rate.

Napkin Math 4.9: Storage bandwidth budget

Problem: What storage configuration keeps a reference accelerator from being data-starved while training ResNet-50? Chapter 11 explains the device family later; here, the point is how a compute ceiling becomes a data-path budget.

Start with the compute ceiling. The reference accelerator can deliver 312 TFLOP/s on dense FP16 operations. ResNet-50 costs about eight GFLOPs for each forward pass, and including the backward training pass raises the training-step cost to about 24.6 GFLOP per image. Chapter 8 derives that backward-pass machinery; here, the combined per-image cost is the input to the storage budget. Dividing accelerator peak throughput by per-image step cost yields a compute ceiling of 12,682 img/s.

That compute ceiling becomes a storage requirement once each image must arrive from disk or object storage. With 150 KB JPEG-compressed images, the data path must supply that many images per second times 150 KB per image, or approximately 1.9 GB/s.

The storage options now have a concrete target: saturating this accelerator requires 1.9 GB/s of sustained bandwidth.

- S3 Standard delivers about 100 MB/s per thread, so the pipeline needs 19 concurrent worker threads before software overhead.
- SATA SSDs deliver about 500 MB/s sequentially, making them a bottleneck for this accelerator.
- NVMe SSDs deliver approximately 5–7 GB/s, which is the right class of local storage for the target.

Systems insight: With SATA SSDs, maximum throughput is capped by 500 MB/s divided across 150 KB images, or approximately 3,333 img/s. The \$15,000 GPU will run at 26 percent utilization because storage supplies only a small fraction of the accelerator’s image-processing ceiling. Storage physics dictates training speed.

Designing for high-throughput training starts by matching storage throughput to accelerator demand. This is the chapter’s starvation argument rotated to its third and final lens: section 4.2.3 priced the idle accelerator as a feeding tax, section 4.5.5 traced the same stall to CPU decode workers, and here the question becomes which storage tier can sustain the required supply rate.

The 500 MB/s figure represents effective SATA III sequential read throughput (the interface ceiling is 550 MB/s), and real-world random read performance with small files can be significantly lower. That caveat reinforces the general principle governing data pipelines: equation 4.5 bounds training throughput by compute capacity and the data supply rate defined in equation 4.6. Let R_{train} , R_{compute} , and R_{data} denote rates in samples per second; let B_{storage} be storage bandwidth in bytes per second, η_{overhead} the dimensionless fraction lost to overhead, and S_{sample} the bytes per sample. Under full overlap, the min-of-rates form corresponds to the $T_{\text{step}} = \max(T_{\text{compute}}, T_{\text{io}})$ lower bound from section 4.5.5; with incomplete overlap, actual step time is higher.

$$R_{\text{train}} \leq \min(R_{\text{compute}}, R_{\text{data}}) \quad (4.5)$$

$$R_{\text{data}} = \frac{B_{\text{storage}}(1 - \eta_{\text{overhead}})}{S_{\text{sample}}} \quad (4.6)$$

Once R_{data} is the smaller rate, additional accelerator compute cannot raise training throughput; the remedy must increase usable storage bandwidth or reduce bytes per sample.

When storage bandwidth becomes the limiting factor, teams must either improve storage performance through faster media, parallelization, or caching, or reduce the amount of data that must move. Large language model training may require processing hundreds of gigabytes of text per hour, while computer vision models processing high-resolution imagery can demand sustained data rates exceeding 50 gigabytes per second across distributed clusters. These requirements make data loading a systems-placement decision: framework data loaders parallelize I/O across asynchronous worker processes, using **double-buffering prefetch queues** to overlap the $D_{\text{vol}}/BW_{\text{IO}}$ data fetch

phase of batch $k + 1$ in host RAM while the accelerator computes the O/R_{peak} pass of batch k on GPU memory, preventing compute starvation stalls (L_{lat}). Framework loaders can also move expensive augmentation work closer to the accelerator rather than storing every augmented variant.

File format selection directly scales the data term ($\frac{D_{\text{vol}}}{\text{BW}}$) of the iron law. Columnar formats like Parquet excel for tabular features by allowing column projection and pushdown filtering, whereas shard-based sequential formats like WebDataset or **TFRecord** pack millions of small binary objects (such as images or audio) into contiguous sequential streams. This sharding eliminates random disk seek overhead (L_{lat}), transforming slow random I/O into sustained sequential storage throughput (BW_{IO}). Quantifying this impact as *format efficiency* (η_{format}) provides a multiplier on effective bandwidth. While Parquet optimizes columnar layout on disk, in-memory representations face an analogous serialization bottleneck when passing data between ingestion engines, dataframes, and ML frameworks. The Apache Arrow standard solves this in-memory boundary by defining a language-independent columnar memory layout, enabling zero-copy data sharing across pipeline processes without expensive serialization and deserialization cycles.

Napkin Math 4.10: Format efficiency

Storage formats determine how much “waste” data must move to retrieve the signal needed for training. Let η_{format} be the dimensionless fraction of bytes read that carry selected features, from 0 to 1. Equation 4.7 shows how that fraction scales physical bandwidth into usable pipeline bandwidth:

$$\text{Effective Bandwidth} = \text{Physical Bandwidth} \times \eta_{\text{format}} \quad (4.7)$$

Values below 1 mean the format spends part of the physical data path on irrelevant bytes; improving layout can therefore raise usable throughput without changing the storage device.

Scenario: Training a fraud model using 20 features from a 100-column table.

Row-oriented formats (such as CSV) must read all 100 columns to retrieve the 20 required features, giving a useful-byte fraction of 0.2 and wasting 80 percent of disk bandwidth. Conversely, column-oriented formats (such as Parquet) read only the selected columns, achieving $\eta_{\text{format}} \approx 1$ (ignoring metadata overhead) and delivering 5× higher effective throughput.

Systems insight: In this projection-only scenario, switching from CSV to Parquet provides the same effective read-throughput gain as a 5× faster data path. Section B.1.3 treats row vs. columnar storage layouts and the algebra of data operations (selection, projection, join) in depth.

Columnar storage formats such as Parquet or Optimized Row Columnar (ORC) achieve a 5× to 10× I/O reduction for typical ML workloads through two complementary mechanisms: column projection, and column-level compression exploiting homogeneous value distributions. Compression proves particularly effective for categorical features with limited cardinality: a country code column with 200 unique values across 100 million records compresses 20× to 50× through dictionary encoding, while run-length encoding compresses sorted columns by storing only value transitions. In combination, projection and compression achieve total I/O reductions of 20× to 100× compared to uncompressed row formats, accelerating training epoch times and shrinking storage footprints.

Compression algorithms trade storage density against CPU decompression bandwidth. While gzip achieves higher compression ratios of 6× to 8×, Snappy achieves 2× to 3× compression but decompresses at 500 MB/s, roughly 4.2× faster than gzip’s 120 MB/s. For ML training where ingestion bandwidth governs accelerator utilization, Snappy’s speed advantage outweighs gzip’s space savings. Training on a 100 GB dataset compressed with gzip requires 13.9 minutes of CPU decompression time, while Snappy requires only 3.3 minutes. When training iterates over data for 50 epochs, this 10.6 minutes difference per epoch compounds to 9 hours total, potentially reducing multi-day training runs to overnight executions. Faster decompression directly increases input pipeline throughput, reduces host memory staging requirements, and keeps accelerator compute saturated.

Beyond format and compression, data partitioning governs retrieval efficiency by aligning storage layout with common access patterns. A recommendation system processing user interactions

partitions data by timestamp and user demographic segments, allowing training runs to query recent temporal slices without scanning the full historical repository. Partitioning strategies interact directly with distributed training: range partitioning by user ID routes consistent user profiles to specific data-parallel ranks, while random partitioning guarantees that all workers receive independent and identically distributed samples. Granularity is critical: too few coarse partitions limit parallel worker concurrency, while millions of micro-shards induce excessive metadata overhead and degrade sequential disk prefetching. In multi-accelerator training, partition skew creates severe straggler bottlenecks: if seven workers in an eight-accelerator job finish loading their batch in 12 ms but one worker reads from an oversized partition and takes 180 ms, that single straggler delays gradient synchronization across the entire cluster. Balanced partitioning that matches worker read budgets is therefore an architectural prerequisite for high accelerator efficiency, as examined in chapter 8.

4.8.3 Storage across the ML lifecycle

Storage requirements evolve because each lifecycle stage demands a distinct access profile from the same underlying data. A dataset undergoes random sampling during exploratory data analysis, high-throughput sequential streaming during model training, and low-latency point lookups during production serving.

During initial development, iteration velocity and schema flexibility dominate raw throughput. The primary systems challenge is managing experiment revisions without exploding storage capacity: ten exploratory runs on a 100 GB dataset naively generate 1 TB of redundant copies. The metadata-pointer architecture introduced in section 4.8.2 decouples version tracking from file storage: tools such as DVC track lineage via small hash manifests while content-addressed stores deduplicate identical binary payloads. Governance policies reinforce this tiering by granting broad access to anonymized development datasets while gating sensitive production data behind encrypted, auditable access tiers.

During training, the access profile shifts toward sustained sequential throughput. Modern deep neural networks process massive datasets across dozens or hundreds of epochs, making storage bandwidth the determinant of hardware utilization. Training ResNet-50 on ImageNet across eight accelerators at 40,000 img/s requires sustained storage bandwidth of approximately 6 GB/s for 150 KB compressed images, and significantly higher if uncompressed FP32 tensors are staged. Storage tiers unable to sustain this rate idle the accelerators, inflating cluster operating costs. The feature placement trade-off (section 4.5.5.3) becomes critical: precomputing feature representations delivers a 30× storage reduction (150 KB raw inputs reduced to 5 KB embedding vectors), but introduces staleness risks when extraction models update.

Production inference reverses the optimization target from sequential bandwidth to low-latency random access. A recommendation service handling 10,000 req/s with a 10 ms SLA and 10 feature reads/request demands 100,000 IOPS. Meeting this random access rate requires in-memory key-value stores such as Redis, distributed caches, or purpose-built low-latency feature stores. Edge deployment imposes further physical boundaries: constrained flash storage, intermittent network connectivity, and non-disruptive model updates dictate tiered caching, where core models reside in local flash while reference data synchronizes asynchronously with cloud endpoints. In production, serving infrastructure must support atomic rollbacks and parallel A/B testing (chapter 14). These operational guarantees depend on strict provenance: every deployed model must trace back to the exact dataset snapshot from which it was trained.

4.8.4 Data versioning for ML reproducibility

Consider a production recommendation model whose click-through rate abruptly degrades after a scheduled weekly retraining, even though the application codebase has not changed. The engineering team must determine whether the drop stems from an unannounced feature change, a label distribution shift, or an upstream table corruption. Without an immutable link between the trained model artifact and its training inputs, diagnosing the issue requires days of speculative bisection. With data versioning, engineers diff the training dataset snapshots directly and isolate the root cause: an upstream pipeline backfill that silently skewed label frequencies.

File-level versioning engines like DVC (Data Version Control) implement this capability by pairing Git with content-addressed remote storage (Iterative 2024). Git tracks small text pointer files (.dvc)

containing cryptographic hashes, while DVC synchronizes the underlying binary dataset to object storage (listing 4.3). Checking out an earlier Git commit automatically restores the exact dataset version matched to that historical code state.

Listing 4.3: DVC Workflow: Git-like semantics for data versioning. DVC tracks large data files alongside code commits, enabling exact reproduction of any historical training dataset through paired `git checkout` and `dvc checkout` commands.

```
# Add data to version control
dvc add data/training.csv
git add data/training.csv.dvc
git commit -m "Add training data v1"
dvc push # Upload to remote storage

# Later: retrieve exact data for any historical commit
git checkout abc123
dvc checkout # Restores exact data from that commit
```

While file-level versioning suits bulk artifacts like images and audio, tabular datasets require versioning over continuously mutating tables. Modern open table formats such as Delta Lake solve this by maintaining an append-only transaction log of all row insertions, deletions, and schema modifications (Armbrust et al. 2020). This log provides time-travel capabilities, allowing training queries to pin a dataset state by timestamp or version ID (listing 4.4).

Listing 4.4: Delta Lake Time Travel: SQL queries identify a historical table state by timestamp or transaction-log version.

```
-- Query data as it existed on a specific date
SELECT * FROM training_data TIMESTAMP AS OF '2024-01-15'

-- Or by version number for programmatic access
SELECT * FROM training_data VERSION AS OF 47
```

Two enterprise components integrate with these versioning engines to establish end-to-end reproducibility. First, feature store point-in-time retrieval evaluates features as they existed at the observation timestamp, eliminating label leakage. Second, a **model registry** records the lineage manifest for each deployed model: Git commit SHA, data version ID, training configuration, and evaluation metrics. In the click-through-rate regression described earlier, this audit trail turns an open-ended debugging investigation into an immediate snapshot comparison.

Long-term retention balances auditability against storage expenditure and data privacy regulations. High-volume production pipelines employ tiered retention: hot SSD tiers for active training and debugging, warm object storage for periodic evaluation snapshots, and cold archive storage for compliance records. Because indefinite retention of petabyte-scale data is economically unsustainable and frequently violates privacy statutes, data lifecycle policies must specify explicit retention horizons and purge mechanisms.

Physical storage architecture dictates where data lives and how fast it moves, but physical layout alone cannot guarantee semantic consistency between training inputs and inference features. Bridging offline batch training and real-time serving requires specialized feature infrastructure.

4.8.5 Feature stores

The fundamental challenge of production feature management is preserving semantic consistency across disparate execution environments: serving historical feature values for training and current feature values for inference. **Point-in-time correctness** is the primary operational requirement that drives teams to adopt **feature stores**, mitigating training-serving skew and enabling feature sharing across models.

In production environments, feature computation pipelines often fragment across languages and execution engines. During model development, data scientists compute features like `user_purchase_count_30d` using batch SQL queries in an offline data warehouse. During production

serving, an online microservice computes the same metric incrementally from an in-memory cache. Even when intended to be identical, minor discrepancies in timezone arithmetic, missing value imputation, or floating-point rounding cause offline and online features to drift apart. Uber’s Michelangelo platform popularized the feature store to resolve this exact form of training-serving skew through centralized, reusable feature definitions (Hermann and Del Balso 2017).



Definition 4.4: Feature store

Feature store is the architectural layer that centralizes the management of machine learning features, decoupling feature computation from consumption.

1. **Significance:** It supports point-in-time-correct historical retrieval for training ($x_{t-\Delta}$) and coordinated feature definitions for real-time inference (x_t), reducing training-serving skew when timestamps, backfills, freshness, and synchronization are handled correctly.
2. **Distinction:** Unlike a general-purpose database, a feature store is designed for dual storage modes: an *offline store* (columnar/batch) for training and an *online store* (key-value/low-latency) for serving.
3. **Common pitfall:** A frequent misconception is that a feature store is only “a place to store data.” In reality, it manages feature definitions, values, and retrieval; transformation may run in the feature store, an offline store, or a separate batch or stream-compute engine.

Feature stores (examined from an operational perspective in section 14.4.1.2) establish a single source of truth for feature definitions across the ML lifecycle. By registering a feature definition as a unified computation graph, the feature store evaluates the identical transformation across both historical batch data and live inference streams. This shared definition prevents training-serving skew, eliminating silent performance degradations where models train with high validation scores but falter in production. Centralization also enables organizational feature reuse: expensive representations, such as multi-dimensional user embeddings derived from months of interaction logs, are computed once and served to multiple downstream modeling teams.

To reconcile opposing access profiles, feature stores implement dual storage engines. The *offline store* utilizes columnar formats (such as Parquet) residing on high-capacity object storage, optimized for multi-gigabyte sequential scans during batch training. The *online store* uses low-latency key-value databases (such as Redis), optimized for sub-10-millisecond point lookups during real-time inference. Synchronizing these two engines requires automated pipelines that push batch-computed feature snapshots to the online store, while streaming ingestion updates both stores continuously as new user events arrive.

Time-travel capabilities differentiate a true feature store from a simple cache. Training requires evaluating features as they existed at the exact moment of an event, rather than their current values. In a customer churn model, for example, training on a customer who churned on January 15 must use features computed on January 14, not current post-churn profile state. Feature stores enforce point-in-time correctness using an **as-of join** (temporal join): for every observation at timestamp t_{obs} , the retrieval engine joins the most recent feature record whose update timestamp satisfies $t_{\text{feat}} \leq t_{\text{obs}}$. This temporal inequality filter guarantees that future information cannot leak into the historical training matrix, even when multiple upstream feature tables update at differing frequencies.

The performance characteristics of the dual stores directly bound training throughput and serving latency. The offline store must sustain batch read rates of millions of feature vectors per minute across wide schemas. The online store must sustain thousands to millions of point lookups per second while honoring single-digit millisecond latency budgets. When user actions (such as adding an item to a cart) require sub-second feature freshness, streaming feature pipelines update the online store continuously. This streaming layer introduces distributed systems challenges, demanding exactly-once processing semantics to prevent duplicate state updates during retries and watermarking mechanisms to handle out-of-order, late-arriving events.

Deploying an end-to-end pipeline across acquisition, ingestion, processing, labeling, and storage does not conclude data engineering. In production, input environments continuously drift:

sensor hardware shifts, upstream schemas mutate, and annotator pools turn over. Without active operational maintenance, static pipelines quietly degrade over time.

4.9 Operational Data Health

A keyword-spotting (KWS) system deployed to edge devices launches with 98 percent accuracy and verified training-serving consistency. Six months later, production accuracy degrades to 94 percent without any modification to the model weights or architecture. The failure traces to three silent pipeline mutations: new smartphone microphone hardware altered the acoustic frequency response, human labeling guidelines shifted without triggering a dataset re-annotation pass, and an upstream schema modification in user metadata was deployed without notice. Each failure mode represents a manifestation of **data debt**: accumulated, unmonitored compromises in data quality, schema integrity, and provenance that compound across system boundaries. Unlike traditional software defects that trigger segmentation faults or unhandled exceptions, data debt leaves the execution graph running normally while the underlying statistical distribution decays. Operating reliable production systems requires formalizing this debt: categorizing its sources, quantifying its accumulation against service-level objectives, and applying structured debugging protocols when production metrics degrade.

4.9.1 Categories of data debt

The edge KWS failure illustrates four distinct categories of data debt. The hardware-induced acoustic shift constitutes *freshness debt*; the divergent annotation standard introduces *quality debt*; and the silent metadata modification generates both *schema debt* and *documentation debt*. While each category requires distinct diagnostic tooling and remediation protocols, they share a common failure mechanism: implicit coupling across unmonitored interfaces.



Definition 4.5: Data debt

Data debt is the compound interest of implicit coupling and missing documentation across an ML data stack.

1. **Significance:** It manifests as silent performance degradation and compounding maintenance overhead caused by unmanaged pipeline dependencies or undetected distribution shifts.
2. **Distinction:** Unlike code-focused technical debt, which triggers compiler warnings, test failures, or runtime exceptions, data debt permits computation to complete successfully while producing statistically invalid outputs.
3. **Common pitfall:** A frequent misconception is that data debt is just “bad data.” In reality, it is a systems architecture failure: it occurs when the assumptions of the training distribution are no longer enforced at the system boundary.

The first category is **documentation debt**: data provenance, semantic meaning, and collection parameters go unrecorded. When datasets omit datasheets or data cards (Gebru et al. 2021; Pushkarna et al. 2022), feature columns lack descriptive metadata, and transformation steps remain undocumented, pipeline maintenance collapses when team personnel change. As Sambasivan et al. (2021) demonstrate, missing metadata and unrecorded data collection practices trigger compounding data cascades across high-stakes systems. Downstream engineers must reverse-engineer feature semantics from legacy ingestion scripts, provenance audits fail regulatory compliance checks, and unstated assumptions—such as sensor dynamic ranges or unit conventions—trigger silent prediction anomalies when upstream instrumentation changes.

The second category, **schema debt**, emerges from accumulated transformation workarounds and uncoordinated migrations. When upstream services alter payload structures or field semantics, teams often apply localized patches—such as regex string extraction or fallback NULL coercions—rather than updating data contracts at the producer boundary. Over time, these patches create brittle parsing layers characterized by version-conditional branching, redundant timestamp parsers, and defensive type conversions. At the ingestion boundary, an unvalidated schema mutation can silently

alter the assembled tensor rank, drop sparse features, or introduce out-of-vocabulary indices that collide with reserved embedding tokens. As established in the pipeline jungle analysis (section 4.3.2), upstream contract violations frequently masquerade as downstream model convergence failures or inference anomalies.

The third category, **quality debt**, consists of known data defects that remain uncorrected due to delivery deadlines or compute budgets. A training corpus harboring a 3 percent label error rate represents concrete quality debt: every training run on this corpus optimizes the model against corrupted targets. Quality debt compounds when model predictions serve as features or pseudo-labels for downstream models, amplifying label noise across dependent services. Telltale operational metrics include accumulated annotation discrepancies, unmitigated demographic representation skews, and unpurged near-duplicate records that artificially inflate validation scores through data leakage.

The fourth category, **freshness debt**, arises when static training sets diverge from evolving production distributions. A model trained on 2023 user behavior deployed in 2025 carries substantial freshness debt: covariate and concept shifts between the historical training sample and live traffic systematically degrade inference accuracy. Freshness debt accumulates invisibly because stale parameters continue executing matrix multiplies without hardware exceptions or runtime faults. Operational indicators include training-cadence deficits (where elapsed time since the last training run exceeds the distribution shift interval), cache staleness in online feature stores, and static lookup tables that fail to reflect recent domain state. Managing these liabilities requires establishing explicit quantitative thresholds for every category.

4.9.2 Measuring and projecting data debt

Unlike technical debt, which can be assessed through code complexity metrics, data debt requires specialized measurement approaches. Table 4.10 gives example warning and critical thresholds for each debt category, making data debt trackable as an engineering service-level objective (SLO) rather than presenting universal standards.

Table 4.10: Data Debt Metrics: Illustrative thresholds for detecting data debt accumulation across four categories. Warning thresholds indicate debt requiring attention in upcoming planning cycles; critical thresholds indicate debt requiring immediate remediation to prevent system degradation. The PSI row uses common operational drift-monitoring bands, while the remaining rows should be calibrated to the risk tolerance and failure cost of the specific ML system.

Debt Category	Metric	Warning Threshold	Critical Threshold
Documentation	% datasets with data cards	< 80%	< 50%
Documentation	% columns with descriptions	< 90%	< 70%
Schema	Schema version branches	> 3	> 10
Schema	% transformations with try/catch	> 20%	> 50%
Quality	Known label error rate	> 1%	> 5%
Quality	Documented bias metrics	Not measured	Measured but not mitigated
Freshness	Days since last retraining	> 90 days	> 365 days
Freshness	PSI vs. training baseline	> 0.1	> 0.25

Data debt compounds through coupled feedback loops in production pipelines. Model outputs frequently feed downstream data lakes as pseudo-labels or synthetic features; any uncorrected label bias or corrupted prediction in an upstream model is thereby codified into the ground truth of dependent systems. Simultaneously, ingestion code degrades: each defensive workaround added to handle an undocumented schema mutation increases cyclomatic complexity, making subsequent schema migrations more likely to fail. As distribution drift steadily widens the divergence between the training baseline and live inference streams, unmaintained systems experience compounding maintenance overhead. To model the sensitivity of pipeline maintenance to unaddressed debt, let Debt_0 be the initial debt level, r an assumed net accumulation rate per planning period, and n the elapsed periods. The planning model in equation 4.8 projects the accumulated liability:

$$\text{Debt}_n \approx \text{Debt}_0 \times (1 + r)^n \quad (4.8)$$

As illustrated in the accompanying margin figure, even modest differences in the periodic accumulation rate r yield divergent trajectories over extended operating horizons. This geometric formulation serves as a planning heuristic rather than an exact physical law; in production environments, debt accumulation is often punctuated by discrete upstream migrations rather than constant compounding.

4.9.3 Remediation strategies

Quantifying data-debt metrics converts an open-ended maintenance deficit into an actionable engineering budget. Remediation can then be integrated into regular operational cycles before pipeline failures disrupt downstream services. Each debt category demands a distinct technical intervention, but all four require sustained capacity allocation rather than reactive triage.

For documentation debt, remediation centers on institutionalizing dataset metadata. Engineering teams can allocate scheduled maintenance intervals to document data provenance, tensor column semantics, and preprocessing graphs. Prioritizing high-fanout feature tables ensures that shared dependencies carry complete schema descriptions and transformation lineage.

Remediating schema debt requires preventive contract enforcement at producer boundaries. Explicit schema contracts between upstream logging services and downstream ingestion pipelines, codified using validation frameworks such as Pandera or Great Expectations, enforce strict type schemas, value bounds, and nullability constraints. Failing fast on schema violations prevents defensive null-coercion and string-parsing patches from accumulating within transformation pipelines.

Addressing quality debt requires structured workflows for correcting identified annotation errors, sample duplicates, and demographic representation gaps. Teams should maintain a tracked error backlog with quantified burn-down rates, integrating dataset curation into sprint planning alongside model architecture tasks rather than deferring it indefinitely.

Freshness debt requires automated drift monitoring coupled with an evidence-based retraining cadence. Integrating population stability index (PSI) and Kullback-Leibler (KL) divergence monitors (section 4.5.3) into operational dashboards enables pipelines to trigger retraining jobs automatically when distribution distance exceeds validated thresholds, rather than waiting for production accuracy to decay.

Like technical debt in traditional software engineering, data debt is an inherent engineering trade-off. *Strategic* debt—such as accepting minimal documentation or delayed deduplication to meet an immediate deployment window—can be an intentional engineering choice. The failure mode is *unconscious* debt: compromises introduced without measurement that compound silently until remediation requires rebuilding the pipeline. Managing data health requires treating data debt as an operational parameter balanced against engineering velocity, serving latency, and compute budgets.

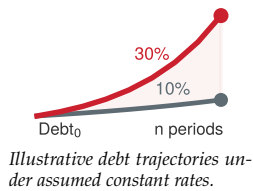
4.9.4 Debugging data pipelines

When data debt matures, it manifests as unexplained accuracy degradation, ingestion pipeline crashes, or subgroup performance disparities. Effective debugging isolates root causes across the data-machine interface: data cascades (section 4.3.1) demonstrate that upstream ingestion flaws trigger downstream anomalies, training-serving skew (section 4.6.1) reveals discrepancies between offline feature generation and online serving, and drift detection (section 4.5.3) identifies statistical divergence over time. Figure 4.15 synthesizes these diagnostic principles into a hierarchical triage procedure.

Production ML failures frequently trace to data dependencies and operational pipeline shifts rather than model architecture alone.³⁵ Prioritizing training-serving skew, distribution drift, and label noise isolates upstream input failure modes before initiating costly architectural redesigns or hyperparameter sweeps.

4.10 Fallacies and Pitfalls

Data engineering pipelines rarely fail with immediate runtime crashes; instead, they silently distort feature distributions, corrupt training state, or starve expensive accelerator clusters. These fallacies and pitfalls examine the physical bottlenecks and architectural misconceptions that undermine production systems.



³⁵ | **ML failure distribution:** Treat the data-dominance pattern as a diagnostic heuristic rather than a universal empirical distribution. Data and operational checks serve as initial triage hypotheses, but they do not eliminate code defects, hardware communication bottlenecks, or numerical instability.

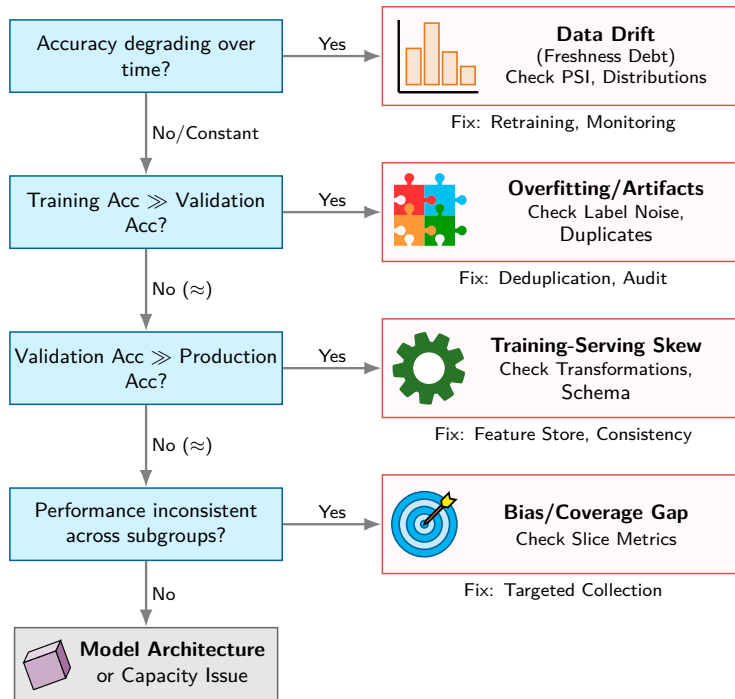


Figure 4.15: Data Pipeline Debugging Flowchart: Four sequential checks prioritize hypotheses involving drift, overfitting or data artifacts, training-serving skew, and subgroup coverage. Each branch suggests a hypothesis to investigate rather than a conclusive diagnosis; if no listed condition holds, the final box suggests examining model architecture or capacity.

Fallacy: *More data always improves model performance.*

Beyond a threshold, additional data yields diminishing returns governed by empirical scaling laws. Studies across computer vision, translation, and language modeling demonstrate that test loss follows a power law in dataset size, so each successive error reduction demands an exponentially larger volume of samples (Hestness et al. 2017). In systems terms, redundant examples consume disk I/O, host-to-device bus bandwidth, and accelerator FLOPs without imparting new gradient signal. The task-relevant signal heuristic from section 4.2 captures this trade-off: scaling raw byte volume without preserving information density inflates infrastructure cost while yielding negligible improvements in evaluation loss.

Pitfall: *Planning petabyte migration as a bandwidth-only transfer.*

Dividing raw dataset size by network link bandwidth calculates theoretical wire time, but wire transfer is rarely the critical path. Data gravity (section 4.2) binds large datasets to a web of surrounding infrastructure: ingestion transformation pipelines, schema validation checks, access-control policies, and downstream consumers. Relocating a petabyte-scale corpus between clouds or data centers demands pipeline refactoring, data-integrity revalidation, cache re-warming, and metadata re-indexing. Consequently, the operational engineering time required to adapt dependent systems dwarfs raw network transit time.

Fallacy: *Data preprocessing can be finished once and left alone.*

Preprocessing pipelines convert raw observations into tensors using transformations parameterized on historical distributions: tokenization vocabularies, mean and variance scaling vectors, categorical frequency tables, and numerical clipping boundaries. When production distributions drift, these static transformations fail silently. Tokenizers encounter surging out-of-vocabulary rates, fixed normalization statistics distort feature activations into out-of-distribution regimes, and static clipping bounds truncate dynamic range. Maintaining pipeline fidelity requires automated monitoring of input tensors and scheduled recalibration of preprocessing statistics alongside model retraining.

Pitfall: *Ignoring data serialization cost.*

Profiling often focuses exclusively on GPU kernel execution while data loaders quietly parse raw JSON or CSV files. Parsing text-based records requires host CPU cores to spend hundreds of cycles per sample on string decoding, tokenization, ASCII-to-float conversions, and heap allocations. This CPU serialization bottleneck starves accelerator compute engines, leaving GPUs idling on PCIe input transfers. Binary serialization formats, such as Apache Arrow or memory-mapped arrays, eliminate parsing overhead by supporting direct zero-copy deserialization into pinned memory. For tabular schemas, columnar formats such as Parquet eliminate disk I/O by reading only required column projections. Benchmarking data ingestion against accelerator consumption rates reveals whether host CPU deserialization is throttling training throughput.

Fallacy: *High training accuracy indicates production readiness.*

Training accuracy reflects empirical optimization on historical training distributions, and held-out validation estimates generalization under identical sampling assumptions. Production environments introduce unmodeled edge cases, sensor noise, and live distributional drift. A model with high validation accuracy fails in deployment if upstream feature pipelines mutate or inference timeouts trigger fallback defaults. Diagnosing post-deployment regressions requires structured root-cause isolation across data drift, feature corruption, and systems execution paths (figure 4.15) rather than reliance on a single aggregate offline metric.

Pitfall: *Ignoring training-serving skew until deployment.*

Training-serving skew occurs when feature generation logic diverges between the offline training pipeline and the online inference runtime. Training features are typically extracted in batch using frameworks such as Apache Spark or SQL engines over data lakes. Online features must be evaluated within millisecond-scale latency budgets inside application microservices. Discrepancies in floating-point precision, time-window aggregations, library versions, or subtle temporal leakage (such as incorporating data timestamped after the prediction point) introduce silent numerical divergence. Because the model still receives valid floating-point vectors, inference proceeds without raising runtime exceptions while prediction quality degrades. Mitigating this pitfall requires shared transformation logic, reproducible feature stores, and automated predeployment parity tests that verify feature equivalence across offline and online paths.

Fallacy: *Synthetic data can fully replace real-world data collection.*

Synthetic data enriches training sets by generating rare edge cases, controlled physical variations, or privacy-compliant samples, but its utility is bounded by generator fidelity. Generative models and simulators sample only from distributions within the support of their parameterized assumptions. When trained exclusively on synthetic data, models overfit to generator-specific artifacts while remaining vulnerable to unmodeled real-world variance. In a KWS system, for example, a detector trained entirely on synthesized speech degrades when encountering authentic acoustic reverberation, diverse accents, microphone frequency response distortions, and non-stationary background noise. Synthetic generation augments empirical collection, but representative real-world observations remain mandatory for validating deployment viability.

Pitfall: *Neglecting data versioning until model debugging requires it.*

Treating training datasets as mutable object storage directories destroys experiment reproducibility. When a retrained model suffers a performance regression, an unversioned pipeline leaves engineers unable to determine whether the regression stems from a modified labeling rubric, silent schema migration corruption, or genuine distribution shift. Once dynamic tables are overwritten or raw source logs are purged, reconstructing historical training inputs becomes impossible. As established in section 4.6.4, robust data lineage requires that every model checkpoint bind immutably to a specific dataset snapshot, transformation code hash, and configuration profile. Deferring data versioning transforms standard root-cause analysis into speculative forensics over lost training state.

4.11 Summary

In machine learning systems, training data functions as executable source code: modifications to the dataset alter the learned behavior of the compiled model without changing a single line of algorithm-

mic code. Because optimization procedures extract operational logic directly from observations, upstream errors in data collection, schema consistency, and labeling amplify through successive pipeline stages as data cascades. Preparing data is therefore a systems co-design problem: storage hierarchies govern the throughput at which accelerators are fed, ingestion architectures balance streaming latency against operational cost, and feature contracts prevent silent training-serving skew.



Key Takeaways: Data is the source code

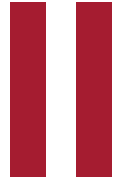
- **Data cascades make upstream quality the highest-leverage investment:** Collection errors amplify through every pipeline stage (figure 4.1). The four pillars (Quality, Reliability, Scalability, Governance) organize prevention, while documentation, schema, quality, and freshness debt require continuous remediation.
- **Data is code; version it, test it, review it:** A dataset is source code for an ML system. Apply the same rigor through version control, validation tests, and data review.
- **Training-serving consistency is nonnegotiable:** Feature transformations shared by training and serving must have equivalent semantics and reuse the same learned state, such as normalization constants and vocabulary mappings.
- **Pipeline architecture choices have large cost implications:** Streaming costs more to operate than batch, while ETL trades storage savings for greater schema-change overhead. Select ingestion patterns by the value of latency, not the appeal of real time.
- **Labeling costs dominate and require substantial resource allocation:** Labeling can cost hundreds to more than a thousand times one optimized training run; in the reference calculation in section 4.4.2, the ratio is $521\times$ to $1,562\times$. Labeling can remain a costly scheduling bottleneck even when annotation work is parallelized.
- **Storage hierarchy determines iteration speed:** The $50\times$ throughput gap between local NVMe (5 GB/s) and cloud object storage (100 MB/s) determines whether iterations occur daily or weekly.
- **The degradation equation becomes actionable through drift and outcome monitoring:** Divergence metrics such as KL measure changes between P_t and P_0 , while labeled outcomes or validated proxies determine whether those changes correspond to model degradation.

The KWS case study combines curated, crowdsourced, and synthetic acquisition; consistency validation; tiered storage for 23.4 million audio samples across 748.8 GB of raw data; and lineage for always-listening devices. Across every stage, data engineering governs the physical data movement and statistical contracts that allow the algorithm to learn without starving hardware or silently drifting in production.



What's Next: From source code to executable

The data pipeline has produced a clean, versioned training set, but a dataset teaches nothing on its own. Chapter 5 opens that black box to examine the algorithm axis: the mathematical operations that convert prepared examples into learned behavior and make neural networks predictable, debuggable, and optimizable.



DEVELOPMENT AND TRAINING

Part II

Development Principles

Building a machine learning system requires more than assembling model components; it requires managing the flow of information and energy through silicon. Part I established that data is both the program and the physical anchor of every ML system. With that anchor in place, Part II turns to the algorithm-machine interaction: *how* mathematical models are co-designed around the physical limits of the hardware that must execute them. The principles here begin with the accounting that determines why certain architectures succeed while others fail at scale.

III Principle 3: The iron law of ML systems

Invariant: Let T denote idealized wall-clock time in seconds. Here D_{vol} is data volume in bytes, BW is the effective bandwidth of the relevant memory or network path in bytes/s, O is total floating-point operations, R_{peak} is peak compute rate in FLOP/s, η_{hw} is dimensionless hardware utilization efficiency, and L_{lat} is fixed latency overhead in seconds, such as a kernel launch or network round trip. When data movement, compute, and fixed overhead execute serially, their idealized times add:

$$T \approx \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}}$$

The full treatment appears in section 1.7.

When data movement and compute overlap on modern hardware, wall-clock time follows the critical path and cannot be shorter than the slower of those stages plus any fixed latency that remains outside the overlap. The practical lesson is about **dominance**, not unconditional summation.

Implication: Optimization is rarely free of trade-offs. Reducing one term often shifts the bottleneck to another. For example, exploiting unstructured sparsity can reduce executed arithmetic, but sparse representations and kernels may lower effective bandwidth or add index traffic, increasing data-movement time (D_{vol}/BW). An optimization improves wall-clock time only to the extent that it shortens the critical path on the target hardware.

The iron law identifies *what* to optimize, but not *how*. The architecture decides *how* before implementation begins.

III Principle 4: The silicon contract

Invariant: Every model architecture and workload regime makes an implicit commitment to the hardware, a wager on which resource it will saturate first.

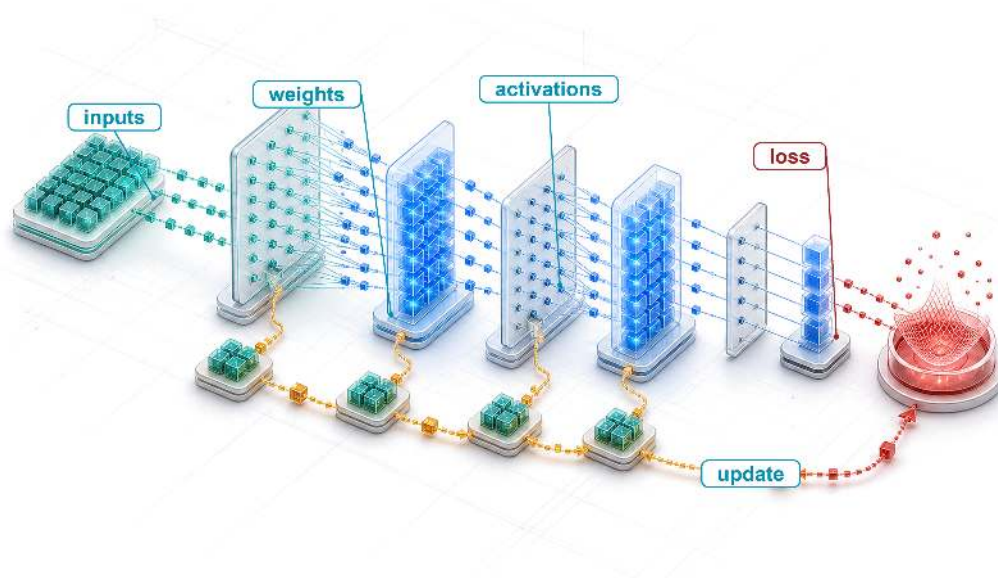
- **ResNet-50** assumes high-density floating-point compute. In batched training or inference on accelerators, it is often compute bound: performance is limited by $O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$.
- **8-billion-parameter Llama 3** assumes high-bandwidth memory access during autoregressive decoding. At small batch sizes, it is often bandwidth bound: performance is limited by D_{vol}/BW .

- **DLRM** assumes massive embedding tables and sparse lookups. It is shaped by both capacity and bandwidth: performance depends on whether embedding tables fit in the available memory hierarchy and how quickly sparse accesses can be served.

Implication: Designing a model without knowing which hardware resource it will saturate is like designing a bridge without knowing the strength of the steel. The design must target the bottleneck.

Together, the iron law and the silicon contract frame every design decision in Part II. The chapters that follow translate these principles into the components of the ML stack: the mathematical foundations of gradient flow, the architectural patterns that commit to specific hardware resources, the frameworks that map model operations onto hardware, and the training systems that execute the same physics at scale.

Neural Computation

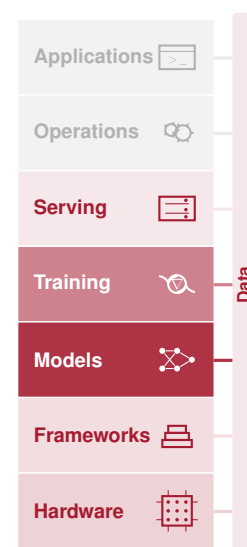


- 5.1 From Logic to Arithmetic
- 5.2 Computing with Patterns
- 5.3 Neural Network Fundamentals
- 5.4 Learning Process
- 5.5 Inference Pipeline
- 5.6 Handwritten Digit Recognition
- 5.7 D·A·M Taxonomy
- 5.8 Fallacies and Pitfalls
- 5.9 Summary

Purpose

Why does understanding a neural network's math matter more than reading its code?

Neural network code can look deceptively simple because a few tensor operations hide the workload that every layer of the system must execute. Matrix dimensions determine arithmetic and data movement; intermediate values determine memory pressure; numerical ranges and gradients determine whether learning remains stable. An operation that looks cheap in isolation can dominate a system when repeated across a batch or when its outputs must be retained for learning, so source-code length gives little indication of resource cost. Two implementations of the same model can run very differently because batch shape, precision, and the target machine change what fits, how quickly it runs, and whether its calculations remain numerically valid. When training stalls or deployment exhausts memory, reading a framework call rarely identifies the binding constraint. A numerical failure, a memory bottleneck, and an inefficient implementation can produce similar high-level symptoms while demanding different remedies. Diagnosis and planning therefore require reasoning from the underlying operations to the bytes moved, values retained, and work repeated. This view can expose a mismatch before an expensive training run or deployment attempt does. Those primitives provide a stable vocabulary across architectures and make software abstractions auditable, turning opaque code into a workload that can be measured, diagnosed, and mapped to hardware. In D·A·M terms, neural computation is the interface where algorithmic structure becomes machine work and where physical limits feed back into model design.





Learning Objectives

- Explain why learned arithmetic replaced rule-based logic for high-dimensional pattern recognition
- Calculate parameter counts, multiply-accumulate operations, activation storage, and memory traffic for multilayer networks
- Compare activation functions by nonlinearity, gradient behavior, and hardware execution cost
- Apply cross-entropy loss and gradient updates to reason about supervised learning dynamics
- Derive forward and backward propagation as matrix operations over batches
- Analyze training vs. inference workloads using compute, memory, and deployment constraints
- Evaluate an end-to-end document-recognition pipeline from preprocessing through decision thresholds and human review

5.1 From Logic to Arithmetic

A dataset that has passed the preceding engineering stages is ready for model development, but examples do not learn on their own. The next system boundary is the model, where data becomes learned behavior through matrix multiplications, activation functions, and gradient updates. A model that runs correctly on one machine and fails on another is not necessarily suffering from a hardware defect. Its layer dimensions may create intermediate activations, the per-layer outputs that training retains for the backward pass, that exceed the available memory. The failure arises from a mismatch between the model workload and the machine budget.

The silicon contract says that every model architecture makes a computational bargain with the hardware it runs on. On the algorithm axis, the architecture's mathematical operators determine the required work and state. Precision, implementation, data reuse, runtime, and hardware then determine the realized memory traffic, latency, and energy. To honor the contract, a systems engineer must understand both the operators and how they map to the machine.

Every mathematical operator in a neural network specifies a concrete computational workload on hardware. Neural computation shifts execution from explicit logical branching (if-then-else control flow) toward long sequences of continuous mathematical transformations, dominated by multiply-accumulate operations. This shift from *Logic* to *Arithmetic* creates dense tensor workloads whose execution throughput depends on arithmetic intensity, batch size, cache reuse, and the hardware roofline envelope set by peak compute and memory bandwidth. Failures in such systems stem from numerical instability, vanishing gradients, or saturated activation regimes rather than syntax errors. For example, recognizing a single handwritten digit in the running MNIST network requires 109,184 MACs without executing a single logical branch in the forward pass.

Arithmetic without branches is not arithmetic without risk. When intermediate values exceed the dynamic range of a numerical format, hardware units trigger an exception or silently produce infinities (Inf) and invalid values (NaN), corrupting downstream tensors. The danger of unhandled numerical overflow is not unique to machine learning; safety-critical systems have failed catastrophically when numerical invariants were violated.



War Story 5.1: The overflow that became guidance (1996)

Context: The European Space Agency's Ariane 5 Flight 501 reused inertial reference software from Ariane 4, including numeric conversions whose assumptions matched the older vehicle's flight profile (European Space Agency 1996).

Mechanism: Ariane 5's flight profile caused a 64-bit floating-point value to exceed the range of a signed 16-bit integer during conversion, triggering an unhandled operand error.

Impact: The flight computer crashed 36 seconds after launch, causing the launcher to veer off course, break apart, and self-destruct, destroying the \$370 million satellite payload.

Response: The inquiry recommended limiting software to functions required during flight, reviewing range assumptions, and designing exception handling so a single conversion failure could not disable redundant units (European Space Agency 1996).

Systems lesson: Numerical ranges, exception handling, and reuse assumptions are part of the systems contract. A computation can be syntactically correct and still be invalid for the physical regime in which it runs. ML systems hit this whenever a numerical format's representable range fails to match the magnitudes flowing through it: a low-precision multiplication that overflows produces an Inf or NaN that propagates silently through the rest of the computation, making numerical regimes as important to debugging as control flow.



Definition 5.1: Deep learning

Deep learning is the computational paradigm that learns layered feature representations from data by composing nonlinear transformations. For suitable tasks, those learned representations can reduce manual feature engineering while increasing computation (O) and state.

1. **Significance:** For suitable compositional function classes, depth can represent a hierarchy much more parameter-efficiently than a shallow network. Each layer adds work and state, and the resulting binding constraint may lie on the data, algorithm, or machine axis depending on the workload.
2. **Distinction:** Shallow methods such as logistic regression and support vector machines can work well with appropriate representations. Deep learning instead uses multiple nonlinear stages to learn representations jointly with the prediction task, which can reduce manual feature design and sometimes support transfer across tasks.
3. **Common pitfall:** A frequent misconception is that parameter count alone explains deep learning. Width can approximate broad function classes, but depth can encode some compositional functions far more efficiently. Architecture, data, optimization, and scale together determine the practical result.

Deep learning represents a physical computation workload where mathematical operators compose into architectural pipelines, placing distinct demands on accelerator memory and execution units across the D-A-M taxonomy. In their 2015 *Nature* survey, LeCun, Bengio, and Hinton¹ (LeCun et al. 2015) synthesized this paradigm.

Classical machine learning often required experts to design feature extractors for each new problem, a labor-intensive process that encoded domain knowledge into handcrafted representations. Deep learning reduces this burden by learning representations directly from data through hierarchical layers of nonlinear transformations. Figure 5.1 maps this relationship in two views: a historical timeline of major milestones and a taxonomic nesting where deep learning sits within machine learning, which in turn sits within artificial intelligence.

This paradigm shift adds engineering problems that conventional software debugging does not address. Deep learning failures can have subtle symptoms: gradient instabilities² that prevent learning, numerical precision errors that corrupt model weights over many iterations, or inefficient memory access patterns in tensor operations³ that leave accelerator compute units underutilized. Resolving these failures requires reasoning about the physical execution of tensor algebra on hardware rather than stepping through procedural control flow.

Diagnosing and resolving these failures requires understanding how algorithmic formulations translate directly into physical hardware demands. The structural requirements of a system depend fundamentally on how it represents patterns—whether through hand-authored rules, engineered statistical features, or end-to-end learned representations. Tracing a single 28×28 MNIST digit

¹ | **LeCun, Bengio, and Hinton:** Recipients of the 2018 ACM Turing Award for conceptual and engineering breakthroughs that made deep neural networks a critical component of computing. Their collective work advanced representation learning, network architectures, and training methods that underpin modern deep learning.

² | **Gradient instabilities:** In a simplified 20-layer sigmoid path, the activation-derivative factors alone are bounded above by 0.25^{20} , or 9.1×10^{-13} . The complete gradient also includes weights and other operations, but repeated saturated derivatives can still make early-layer updates negligible. Plateaus or NaNs can reveal the symptom without identifying the cause; rectified linear unit (ReLU) activations, careful initialization, normalization, optimization methods, and residual connections all helped make deep networks trainable, and section 6.8.2 treats residual connections in detail.

³ | **Tensor operations:** A tensor's logical dimensions map onto a flat physical layout, and a kernel may prefer a different ordering. If changing a typical ImageNet input ($224 \times 224 \times 3$, about 151 KB) from channel-first to channel-last order requires a materialized reorder, it moves about 301 KB of read-plus-write traffic per image without changing the model's mathematical result. Some runtimes avoid the copy through views, fusion, or a compatible kernel. When a reorder remains, it can consume part of a tight inference budget before the model's main arithmetic begins.

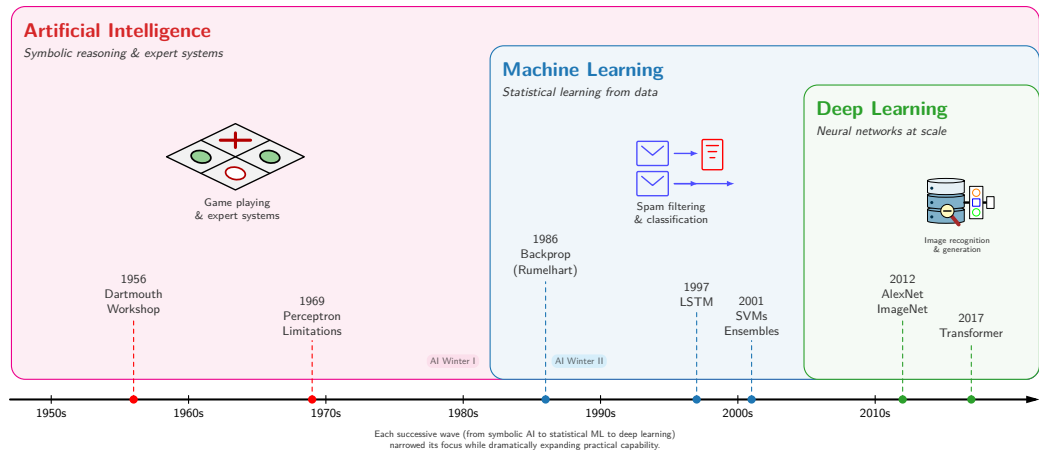


Figure 5.1: AI Hierarchy and Timeline: A timeline from the 1950s to the 2010s is overlaid with the present-day taxonomic relationship among artificial intelligence, machine learning, and deep learning. The nested bands show scope, not exclusive historical eras: deep learning falls within machine learning, which falls within artificial intelligence. Dated milestones run from the 1956 Dartmouth Workshop through the 2017 transformer.

across these three paradigms illustrates this shift, demonstrating how each representation strategy reshapes the system's arithmetic intensity, memory traffic, and operational profile.

5.2 Computing with Patterns

The shift from logic to arithmetic reshapes how hardware processes real-world patterns. Tracing a single task—classifying a 28×28 pixel MNIST digit—across three paradigms exposes how representation strategies fundamentally alter the computational profile.

5.2.1 From explicit logic to learned patterns

Rule-based programming requires developers to specify the procedures that map inputs to outputs. Consider a simple game like Breakout.⁴ Its simulator uses explicit collision rules: when the ball hits a brick, the code removes the brick and reverses the ball's direction (figure 5.2). Such general rules work well for well-defined game physics, but hand-authoring a recognition policy for every variation in messy, unstructured real-world data does not scale.

⁴ | **Breakout (DQN - deep Q-network):** Atari's 1976 arcade game became an AI milestone when DeepMind's DQN learned a gameplay policy from pixels (2015) without a hand-authored policy for each situation. The environment, action space, reward, and preprocessing were still programmed. DQN resized frames to 84×84 , stacked 4 frames as input, and selected a new action every 4 frames, illustrating how learned control still depends on an engineered real-time inference loop.

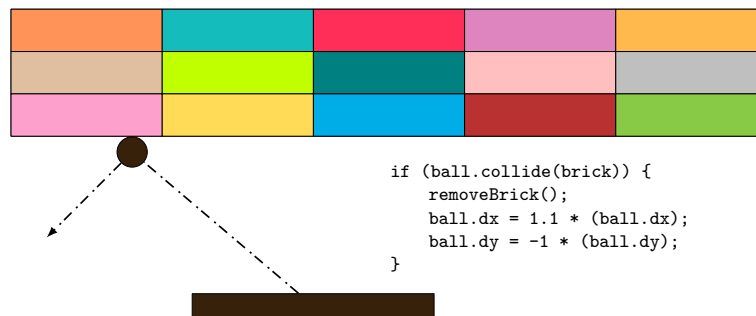


Figure 5.2: Breakout Collision Rules: The game program uses explicit if-then rules for collision detection, specifying ball direction reversal and brick removal upon contact. While effective for a game with clear physics and limited states, this approach illustrates how rule-based systems must anticipate every possible scenario.

The data flow in figure 5.3 makes the traditional programming relationship explicit: a program applies specified procedures to input data to produce outputs. Early artificial intelligence research explored whether hand-authored symbolic rules could scale to complex reasoning and recognition tasks.

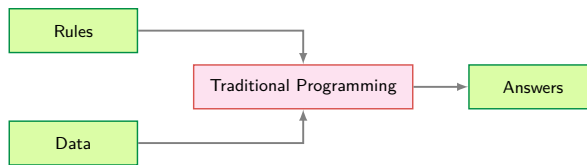


Figure 5.3: Traditional Programming Flow: Rules and data serve as inputs to a traditional program, which produces answers as output. This input-output pattern formed the basis for early AI systems but lacks the adaptability needed for complex pattern recognition tasks.

Despite their apparent simplicity, rule-based limitations surface quickly with complex real-world tasks. Recognizing human activities illustrates the challenge. Classifying movement below 4 km/h as walking seems straightforward until real-world complexity intrudes. Speed variations, transitions between activities, and unanticipated cases each demand additional rules, as the successive code fragments in figure 5.4 show. Computer vision tasks compound these difficulties. Detecting cats requires rules about ears, whiskers, and body shapes while accounting for viewing angles, lighting, occlusions, and natural variations. Early systems achieved success only in controlled environments with well-defined constraints.



Figure 5.4: Activity Classification Rules: Successive code fragments classify walking below 4 km/h, then add running below 12 km/h and biking at higher speeds. A final unhandled activity shows how real-world cases force increasingly complex branching logic.

Recognizing these limitations, the knowledge engineering approach that characterized AI research in the 1970s and 1980s attempted to systematize rule creation. Expert systems⁵ encoded domain knowledge as explicit rules, showing promise in specific domains with well-defined parameters but struggling with tasks humans perform naturally: object recognition, speech understanding, and natural language interpretation. These failures highlighted a deeper challenge: many aspects of intelligent behavior rely on implicit knowledge that resists explicit rule-based representation.

Consider one illustrative rule-based classifier for a 28 by 28 digit: compare pixel intensities against thresholds, check stroke patterns in specific regions, and branch on the results. Under this toy implementation, the computation is roughly 100 comparisons over 784 bytes of pixel data—small, predictable, and compatible with an ordinary CPU cache. The comparison establishes a workload baseline, not the accuracy of a production rule system.

5.2.2 The feature engineering bottleneck

The failures of rule-based systems suggested an alternative: rather than encoding human knowledge as explicit rules, let the system discover patterns from data. Machine learning offered this direction—instead of writing rules for every situation, researchers wrote programs that identified patterns in examples. The success of these methods, however, still depended heavily on human insight to define which patterns to look for, a process known as feature engineering.

Feature engineering transformed raw data into representations that expose patterns to learning algorithms. The histogram of oriented gradients (HOG)⁶ (Dalal and Triggs 2005) method exemplifies this approach, identifying edges where brightness changes sharply, dividing images into cells, and measuring edge orientations within each cell (figure 5.5). This transforms raw pixels into shape descriptors robust to lighting variations and small positional changes.

⁵ | **Expert systems:** These systems convert human expertise into explicit IF-THEN rules. This knowledge-engineering approach scales poorly for tasks such as object recognition, where the relevant variation is difficult to enumerate. Even successful systems such as DEC's XCON accumulated thousands of hand-authored rules, making rule acquisition, consistency, and maintenance central engineering costs.

⁶ | **Histogram of oriented gradients (HOG):** An influential handcrafted descriptor for pedestrian detection before deep learning (Dalal and Triggs 2005). One common configuration computes gradient orientations in 8×8 pixel cells, with cell size and other choices tuned for the task. HOG's fixed computation graph can run efficiently on CPUs with predictable latency, while learned features can reduce descriptor redesign but do not automatically transfer across domains and often benefit from accelerators only as model and batch scale grow.

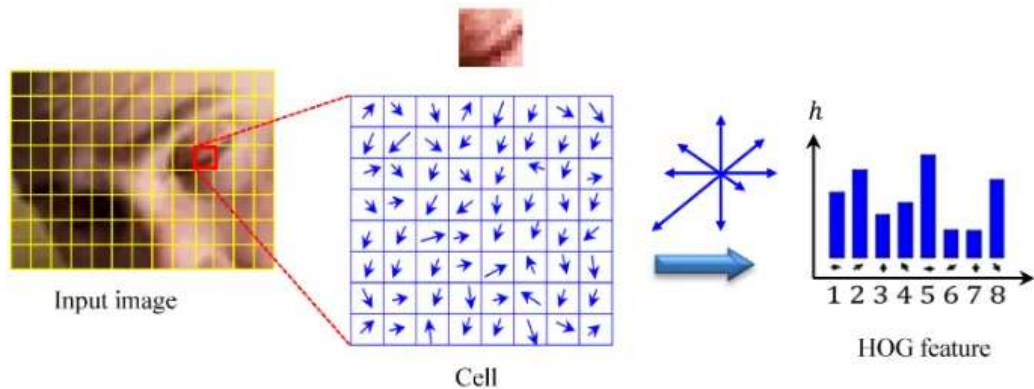


Figure 5.5: HOG Method: A local image patch is divided into cells, gradient orientations are accumulated into a histogram, and the resulting descriptor summarizes local shape while reducing sensitivity to illumination. Diagram reproduced from Figure 5 of Bakheet and Al-Hamadi (2021), licensed under CC BY 4.0.

7 | Scale-invariant feature transform (SIFT): SIFT encodes this domain expertise in a four-stage algorithm that identifies keypoints stable across scale and rotation. The number of detected keypoints varies by image, which complicates fixed-shape batching. Systems can regularize the representation through caps, padding, pooling, or fixed encodings, but each choice adds computation or discards information.

8 | Gabor filters: Originally developed for localized time-frequency analysis (Gabor 1946), these filters detect edges and textures at specific orientations and frequencies. A typical bank contains many hand-designed filters across orientations and frequencies. Deep convolutional layers instead learn small kernels from data; early convolutional neural network (CNN) filters often become edge-, color-, and texture-sensitive detectors, shifting feature design from manual filter banks to data-driven optimization (Krizhevsky et al. 2012; LeCun et al. 2015).

Complementary methods like scale-invariant feature transform (SIFT)⁷ (Lowe 1999) and Gabor filters⁸ captured different visual patterns. SIFT detected keypoints stable across scale and orientation changes, while Gabor filters identified textures and frequencies. Each encoded domain expertise about visual pattern recognition.

These engineering efforts enabled advances in computer vision during the 2000s. Systems could now recognize objects with some robustness to real-world variations, leading to applications in face detection, pedestrian detection, and object recognition. Despite these successes, the approach had limitations. Experts needed to carefully design feature extractors for each new problem, and the resulting features might miss important patterns that were not anticipated in their design. The bottleneck remained: human expertise could not scale to the complexity and diversity of real-world visual patterns.

Return to the same 28 by 28 digit. In this illustrative implementation, HOG divides the image into a 7 by 7 grid of 4 by 4 cells, computes gradient magnitudes and orientations, bins them into 9 orientation histograms per cell, and produces a 441-element feature vector. A linear Support Vector Machine (SVM) classifier then performs ten dot products over that vector. The result is roughly 8,000 operations and about 2 KB of working memory—about 80× the work of the toy rule-based baseline, but still structured and well served by CPU vector units using **single instruction, multiple data (SIMD)**. Descriptor size, class count, and implementation all change this cost.

5.2.3 Automatic pattern discovery

The limitations of handcrafted features motivate a more radical approach: rather than encoding features by hand, the system discovers its own. Neural networks embody exactly this shift—rather than following explicit rules or relying on human-designed feature extractors, the system learns representations directly from raw data.

Supervised learning changes the traditional programming relationship. Instead of encoding every decision rule directly, engineers provide examples and target answers, and optimization produces a learned model. Figure 5.6 makes this relationship tangible by showing learned rules as the output rather than the input. Humans still shape the task through data, objectives, architectures, and evaluation.

The system discovers patterns from examples through this automated process. When shown millions of images of cats, it learns to identify increasingly complex visual patterns, from simple edges to combinations that constitute cat-like features. This parallels how biological visual systems operate, building understanding from basic visual elements to complex objects.

The gradual layering of patterns reveals *why* neural network depth matters. For some compositional function classes, deeper networks can represent functions that require much wider shallow networks, an advantage formalized in section 5.3.1.

Deep learning often exhibits empirical scaling: within a given regime, performance can improve as additional data, compute, and model capacity are used effectively, although gains depend on

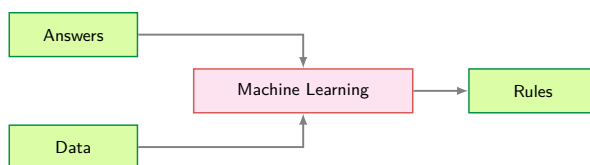


Figure 5.6: Data-Driven Rule Discovery: The flow diagram inverts the traditional programming pattern: data and answers serve as inputs to the machine learning process, which produces a learned decision function as output. This shift replaces a hand-coded decision policy while leaving engineers responsible for the data, objective, architecture, and evaluation.

data quality, optimization, architecture, and evaluation conditions. In the ImageNet competition, traditional methods achieved approximately 25.8 percent top-5 error in 2011. AlexNet⁹ reduced this to 15.3 percent in 2012. By 2015, ResNet achieved 3.6 percent top-5 error, lower than a published human baseline of approximately 5.1 percent under that evaluation protocol.

Figure 5.7 illustrates double descent across model complexity regimes. In the classical underparameterized regime, test error follows a standard U-shaped curve as capacity increases. At the interpolation threshold—where the model has just enough parameters to fit the training sample—test error peaks sharply. Beyond this threshold, in the overparameterized modern regime, test error can decline again. The axes are normalized to emphasize the structural profile across model families rather than a single dataset.

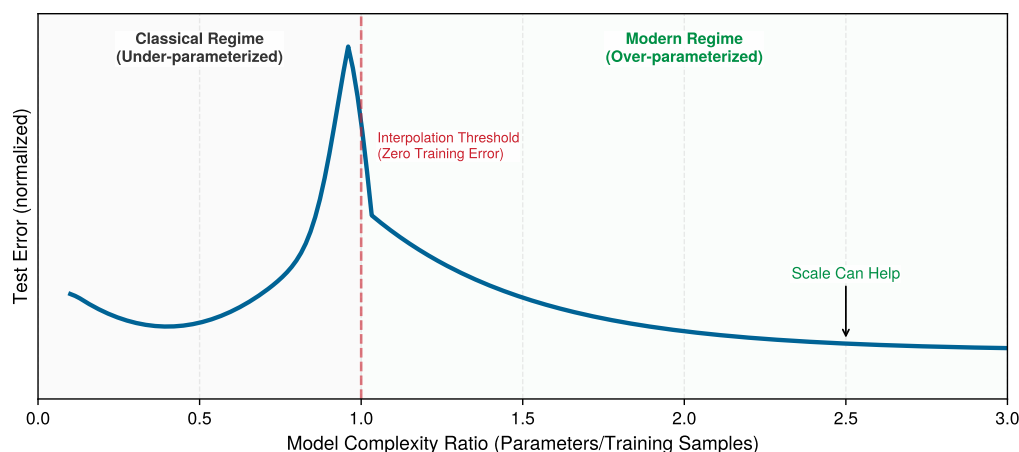


Figure 5.7: The Double Descent Phenomenon: An illustrative normalized curve shows test error first decreasing and then rising in the classical regime, peaking at the interpolation threshold, and decreasing again in the overparameterized modern regime.

The counterintuitive shape matters because test error, the error on held-out examples rather than the training examples the model sees directly, initially follows the expected U-curve, then decreases again when the model has more parameters than the simplest theory expects. This behavior complicates the classical account of overparameterization. The usual bias-variance trade-off¹⁰ suggests that models that are too small underfit, while models with excessive capacity risk fitting noise. Double descent (Belkin et al. 2019) shows that larger models can sometimes generalize better than smaller ones after the interpolation threshold. This overparameterization effect means that scale can be an engineering lever, not that scale is automatically safe: the data distribution, training procedure, and regularization still determine whether the extra capacity helps or memorizes. Overfitting and the regularization techniques that control it are detailed in section 5.4.4.8.

Neural network performance follows empirical scaling relationships with direct systems consequences. The durable anchor is that frontier model sizes and training compute budgets have grown by orders of magnitude over the past decade (section 5.2.7 quantifies the trajectory). That growth makes arithmetic throughput, memory bandwidth, storage capacity, and communication potential bottlenecks whose importance depends on the workload and machine. Chapter 8 develops the

⁹ | AlexNet's memory split:

Krizhevsky's team split AlexNet across two NVIDIA GTX 580 GPUs, each with 3 GB of VRAM, so that the model and training state fit across the two devices. The workaround illustrates the systems lesson: a training configuration can exceed one device's memory budget, forcing engineers to partition work and communication across devices.

¹⁰ | Bias-variance trade-off:

In some overparameterized regimes, test error decreases again after the interpolation threshold, so the classical single-U-shaped account is incomplete. The systems consequence is that model size cannot be treated as a universally monotonic overfitting risk. Larger models can become useful engineering options, provided the measured data, optimization, and regularization regime supports the gain.

quantitative scaling formulations, including how model size, data, and compute trade off against one another; chapter 10 explores the practical responses.

Neither relationship makes model size a decision rule. Double descent describes measured generalization under a particular data and training regime, while historical scaling describes resource investment across selected systems. Choosing a model still requires comparing candidates on representative deployment data and on the complete cost of training, serving, monitoring, and maintenance. The useful question is not whether additional scale can help, but whether its measured gain earns those system costs. That comparison should also include uncertainty in both gains and costs.

🔍 Systems Perspective 5.1: When to use neural networks

Not every problem benefits from deep learning. Neural networks are strong candidates when data exhibits spatial, sequential, or hierarchical structure that an architecture can exploit, simpler baselines miss the required task quality, and measured gains justify the complete system cost (table 5.1). With limited labels, low-dimensional tabular data, approximately linear relationships, hard latency limits, or strict auditability requirements, classical methods may match the useful performance of a neural network with less training, serving, and maintenance effort (table 5.2).

Table 5.1: Neural-Network Candidates: Conditions under which deep learning is likely to outperform classical methods.

Condition	Evidence to seek	Rationale
Representative data	Training and validation cover deployment conditions	Capacity cannot replace missing coverage
Exploitable structure	Spatial, sequential, or hierarchical patterns	Architecture can encode a useful prior
Baseline gap	Simpler methods miss the required task-quality target	Complexity buys a measured gain
Operational fit	Gain survives latency, memory, and cost limits	Offline accuracy alone is insufficient
Maintenance advantage	Learned features reduce recurring manual updates	Full-system maintenance becomes simpler

Table 5.2: Classical-Method Candidates: Conditions under which simpler models are likely to match or beat a neural network.

Condition	Baseline to test	Reason to prefer it
Limited labeled data	Linear model or tree ensemble	Lower variance and easier validation
Structured tabular inputs	Gradient boosting or linear model	Strong accuracy without learned features
Approximately linear signal	Linear or additive model	Transparent and inexpensive execution
Hard resource bound	Rules or compact classical model	Predictable latency, memory use, and energy demand
Direct auditability needed	Constrained tree or linear model	Decisions are easier to inspect

Systems insight: Establish a simple baseline before building a neural network. Compare candidates on the task metric that matters and on the resources required to train, serve, monitor, and update them. Retain the neural approach only when its measured benefit justifies that added system. The document-recognition history in section 5.6 shows why the comparison must include preprocessing, rejection policy, and the cost of human review rather than model accuracy alone.

Learning representations from data reshapes AI system construction. Reducing manual feature engineering introduces new demands: infrastructure to handle large datasets, high-throughput hardware to process them, and specialized accelerators to perform mathematical calculations efficiently. These computational requirements have helped drive the development of chips optimized

for neural network operations. Deep learning now underpins major systems in computer vision, speech recognition, game playing, and natural language processing.

Return to the same 28 by 28 digit, now processed by the chapter's three-layer neural network ($784 \rightarrow 128 \rightarrow 64 \rightarrow 10$). Under the stated implementations, the forward pass requires 109,184 MACs, $1,091.8\times$ the work of the toy rule-based baseline. The 109,386 parameters consume 438 KB in 32-bit single-precision floating-point (FP32), exceeding most L1 caches and creating traffic between cache levels during inference. Training multiplies the cost further: each image is processed forward and backward before an update, with simplified dense-layer arithmetic on the order of three forward passes, repeated over 60,000 images for multiple epochs. Dense matrix multiplications now dominate this example's arithmetic and favor parallel hardware, shifting the engineering challenge from control-flow branching to memory bandwidth, data reuse, and hardware parallelism.

5.2.4 Computational infrastructure requirements

The MNIST running example traced one set of illustrative implementations from ~ 100 comparisons (rule-based) through $\sim 8,000$ operations (HOG) to 109,184 MACs (neural network): a $1,091.8\times$ escalation in the stated operation models, with a corresponding shift toward parallel matrix operations. Table 5.3 summarizes the tendencies exposed by this comparison.

The comparison matters because each step changes the likely bottleneck: from branch-heavy CPU control, to batch-oriented feature pipelines, to memory-fed matrix parallelism. CPUs remain effective for small networks and latency-sensitive workloads, while large dense operations often map more efficiently to accelerators.

The shift toward parallelism creates new bottlenecks. The central challenge is the memory wall:¹¹ compute throughput has grown faster than off-chip memory bandwidth. Whether a matrix operation is memory or compute bound depends on its arithmetic intensity: large, well-tiled matrix-matrix multiplication can be compute bound, whereas matrix-vector, small, skinny, or low-reuse layers are often limited by bandwidth.¹² Hardware responses are examined in section 11.5.1, while section D.3.3 details the memory hierarchy.

Table 5.3: System Resource Tendencies: The illustrated implementations shift from control-oriented rules to structured feature pipelines and then to matrix-heavy neural computation. These are workload tendencies rather than requirements of every program in each paradigm.

System Aspect	Traditional Programming	ML with Features	Deep Learning
Computation	Often control-flow oriented	Structured parallel ops	Often matrix-parallel
Memory Access	Workload dependent	Commonly batch-oriented	Large tensor working sets
Data Movement	Program and input dependent	Feature-pipeline movement	Parameter and activation movement
Hardware Needs	Commonly CPU-centric	Often CPU with vector units	Frequently accelerator-friendly
Resource Scaling	Input and program dependent	Descriptor and data driven	Driven by width, batch, state, and reuse

Energy adds a constraint alongside speed. Moving data from main memory to processing units can consume far more energy than a simple arithmetic operation (Horowitz 2014). This energy hierarchy explains why neural network accelerators maximize data reuse: they keep frequently accessed values in local storage and schedule operations to reduce movement. Whether movement or arithmetic dominates depends on precision, reuse, workload shape, and hardware, but the cost of movement motivates specialized architectures from data center GPUs to TinyML accelerators.

The memory-computation trade-off manifests differently across the cloud-to-edge spectrum introduced in chapter 2. Cloud servers may use more memory and power to pursue throughput, while mobile devices operate within tighter power budgets. Large-batch training commonly emphasizes throughput; latency-sensitive or battery-powered inference may emphasize response time or energy per prediction. The exact priority depends on the deployment workload.

These single-machine constraints compound when scaling across multiple machines: dense layers scale with adjacent dimensions, batches scale activation storage and throughput demand,

NN 109K MACs

HOG 8K ops

Rules 100

log scale

In this MNIST comparison, learned features spend far more arithmetic.

11 | Memory wall: Fast on-chip storage responds in nanoseconds, while larger off-chip memory is orders of magnitude farther away in latency and energy. Neural network weights often exceed the smallest caches, so performance depends on tiling, larger cache levels, on-chip static RAM (SRAM), and high-bandwidth memory (HBM) to reuse values before another off-chip fetch. Insufficient reuse can leave compute units idle even when peak arithmetic throughput is high.

12 | Memory-bound operations: Matrix multiplication's arithmetic intensity (FLOP/byte loaded) determines whether a layer is compute bound or memory bound. Layers that fall below the hardware's roofline crossover point finish their arithmetic before the next tile of weights arrives from memory. The result: effective hardware utilization can drop sharply, and adding more compute units yields little speedup until memory bandwidth or data reuse improves.

and training adds backward-pass, activation, and optimizer-state multipliers. Model-compression, hardware-acceleration, and training-system techniques all respond to this pressure by reducing the work, raising the useful machine rate, or changing where state lives.

These physical constraints—hardware parallelism, memory walls, and energy-dominated data movement—stem directly from the structure of neural computation: weights that change during training, many simple units operating simultaneously, layers that compose low-level features into high-level concepts, and data reuse that minimizes energy-intensive movement. These properties manifest concretely in the fundamental building block of neural computation: the artificial neuron.¹³ Just as understanding a single transistor reveals how complex processors operate, understanding the artificial neuron reveals how million-parameter networks behave.

5.2.5 The artificial neuron as a computing primitive

The basic unit of neural computation, the **artificial neuron** (or node), serves as a simplified mathematical abstraction of nervous activity (McCulloch and Pitts 1943). Later digital neural-network implementations adapted this abstraction into a standardized processing unit. This building block enables complex networks to emerge from simple components working together. Compare the biological and artificial neurons side by side in figure 5.8 to see how this computational model distills biological complexity into a simpler computational form.

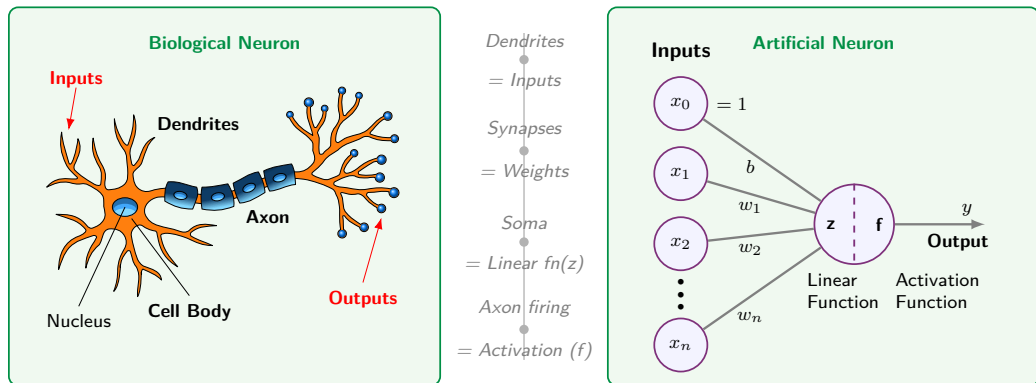


Figure 5.8: Biological-to-Artificial Neuron Mapping: Side-by-side comparison showing how biological neuron structures map to artificial neuron components. Dendrites correspond to inputs, synapses to weights, the cell body to the summation function, and the axon to the activation output. Read the mapping as an abstraction rather than a resemblance claim, since it keeps only what reduces to arithmetic and discards the electrochemistry that produced the behavior.

The right panel of figure 5.8 traces the signal through four stages, each translating a biological structure into a mathematical operation:

1. **Input Reception** (Dendrites $\rightarrow x_1, x_2, \dots, x_n$): The artificial neuron receives a vector of input features $\mathbf{x}$. In MNIST digit recognition, these represent pixel intensities. The dendrite analogy captures reception of many signals without claiming biological equivalence.
2. **Weighted Modulation** (Synapses $\rightarrow w_1, w_2, \dots, w_n$): Each input is multiplied by a learnable weight w_i , just as synaptic strengths modulate biological signals. These weights act as “gain” controls, determining how much influence each feature has on the final decision. A bias term b (shown as the top input $x_0 = 1$ in figure 5.8) shifts the activation threshold. Across a network, learned behavior is distributed over many such parameters rather than stored in one weight.
3. **Signal Aggregation** (Cell Body $\rightarrow$ Linear Function z): The neuron integrates the weighted signals, producing a single scalar value $z = \sum(x_i \cdot w_i) + b$. This mirrors how a biological cell body sums incoming electrochemical signals to determine whether the neuron has received enough evidence for a particular pattern.
4. **Nonlinear Activation** (Axon $\rightarrow$ Activation Function f): The aggregated signal passes through an activation function $f(z)$, producing the activation output $a = f(z)$ (often denoted y). This mirrors the axon’s all-or-nothing firing decision: the nonlinearity determines whether the neuron “fires” a signal to the next layer. Unlike the biological case, f can produce graded outputs

¹³ | **Neuron:** McCulloch and Pitts (1943) introduced a mathematical threshold model of nervous activity: inputs combine and an all-or-none output fires when a threshold is met. That logical neuron is an origin point for the artificial-neuron abstraction and for networks built from many simple units. Learned weights, deep hierarchies, and modern fused multiply-add (FMA) accelerator datapaths are later developments that turn this abstraction into the matrix-heavy workloads studied in this chapter.

(for example, ReLU passes positive values through, zeroes negatives), but the principle is the same—thresholding followed by propagation. Table 5.4 formalizes these correspondences.

Table 5.4: Biological vs. Artificial Neuron Mapping: Structural components mapped to mathematical formulations and physical memory states.

Biological Structure	Artificial Component	Mathematical Operation	Engineering Role
Dendrites (receive signals)	Input Vector	$\mathbf{x} = [x_1, \dots, x_n]$	Data ingestion from sensors or prior layers
Synapses (modulate strength)	Weight Vector	$\mathbf{w} = [w_1, \dots, w_n]$	Learnable parameters encoding importance
Cell Body (integrates signals)	Linear Function z	$z = \sum (x_i \cdot w_i) + b$	Linear integration of feature signals
Axon (fires output)	Activation Function f	$a = f(z)$	Nonlinear thresholding and signal propagation

From a systems engineering perspective, this translation reveals *why* neural networks can have demanding computational requirements. A direct scalar evaluation of one neuron performs N **multiply-accumulate (MAC)**¹⁴ operations and, without reuse, loads N inputs, N weights, and a bias before writing the output. Batched and tiled kernels reuse inputs and weights across many neuron evaluations, reducing traffic per MAC. At network scale, performance therefore depends on both the number of operations and how effectively the implementation reuses data.

The transition from individual neurons to integrated systems requires navigating the central trade-off between representational capacity and computational cost. Digital hardware performs arithmetic quickly, but the volume of operations and data movement in deep networks creates bottlenecks that a single-neuron view does not expose.

Replicating intelligent behavior in silicon confronts three interrelated system-level constraints. As models grow, moving parameters and activations can become more limiting than arithmetic throughput. Concurrency clashes with dependency: many operations inside a layer can run in parallel, but layer $\ell + 1$ still depends on layer ℓ , placing a lower bound on end-to-end latency. Precision also trades against memory, throughput, and energy: wider representations consume more storage and data movement, while narrower formats require evidence that task quality remains acceptable. Chapter 10 develops this search for minimum viable precision.

Addressing these constraints requires two complementary strategies. An architectural inductive bias is a built-in structural assumption about the data: convolutional networks assume nearby pixels matter together in images, while recurrent networks assume order matters in sequences. Encoding those assumptions restricts the function class toward structures expected to be useful (Mitchell 1980). Computational scaling supplies additional capacity and optimization effort. Modern AI engineering combines both approaches, using architectural structure to make effective use of available scale.

5.2.6 Hardware and software requirements

Translating neural concepts to silicon carries a physical cost. Feature extraction can become weighted linear sums followed by nonlinear activations, while dense interactions become matrix operations that hardware must execute efficiently. The mathematical graph, however, does not prescribe an execution plan. Frameworks and compilers must choose kernels, tensor layouts, fusion boundaries, and schedules for a particular processor. Two implementations can therefore compute the same function with the same nominal arithmetic count yet differ substantially in data traffic, temporary storage, and latency. Hardware supplies arithmetic units, memory, and communication links; software determines how the graph uses them. This translation layer decides whether the parallelism visible in the model becomes useful machine utilization.

A neural network’s learned behavior is distributed across parameters stored at ordinary memory addresses. Inference fetches parameters and activations; training adds saved intermediates, gradients, and optimizer state, so storage capacity and bandwidth both matter. Biological synapses combine storage and local processing, whereas digital accelerators move values through a memory hierarchy. The brain consumes about 20 percent of the body’s resting oxygen and calories (Raichle

¹⁴ **MAC (multiply-accumulate):** A core operation in dense and convolutional layers is $a \leftarrow a + (b \times c)$. Hardware data sheets often report fused multiply-add throughput in FLOPs because one multiply and one add count as two floating-point operations. On that convention, the reported FLOP/s rate is twice the MAC/s rate when a MAC is counted as one multiply-accumulate. Layer and batch choices also create activation, communication, and data-movement costs, so MAC count is necessary but not sufficient for a systems budget.

and Gusnard 2002), although direct efficiency comparisons are misleading because biological and artificial systems perform different tasks under different accuracy and latency constraints. The narrower systems lesson is that data movement matters enough to motivate the specialized hardware in chapter 11 and the optimization strategies in chapter 10.

These hardware demands did not emerge overnight. The tension between algorithmic ambition and available silicon has shaped the entire trajectory of neural network research, from the earliest perceptrons to today's trillion-parameter models.

5.2.7 Evolution of neural network computing

The **perceptron**¹⁵ introduced a probabilistic neuron model for learning and information storage (Rosenblatt 1958). Its simple mathematics met a hardware constraint: larger networks needed far more processing and memory. Deep learning evolved through this algorithm-silicon co-evolution; the neuron abstraction remained recognizable while the hardware able to execute it transformed. That co-evolution frames the history that follows: progress required both better methods and better machines.

The **backpropagation**¹⁶ algorithm was applied to neural networks by Paul Werbos in his 1974 PhD thesis (Werbos 1974), building on Seppo Linnainmaa's 1970 work on automatic differentiation (Linnainmaa 1970), and was later popularized by Rumelhart, Hinton, and Williams (Rumelhart et al. 1986). Their publication demonstrated the algorithm's practical effectiveness and brought it to widespread attention in the machine learning community, triggering renewed interest in neural networks. This chapter returns to the algorithm in section 5.4.4; section 8.3.3 develops its systems-level implementation. The 1986 demonstration showed that hidden units could learn useful internal features, but it did not resolve the systems problem of scaling training. Larger models multiply forward operations, saved activations, gradient computations, and parameter updates. Practical adoption therefore depended on improvements across the complete stack rather than on the learning rule alone.

This history yields a recurring systems lesson: an algorithm becomes useful only when the complete system can train and evaluate it effectively. The lag between early backpropagation work¹⁷ and widespread adoption reflected the maturation of hardware, data, initialization, nonlinearities, optimization, benchmarks, and software infrastructure. An algorithm can therefore be viable in principle long before its surrounding system makes it useful in practice. Deep learning came from this convergence, not from a new mathematical discovery alone.

While the term gained prominence in the 2010s alongside massive increases in data and compute, the underlying scaling trajectory spans seven decades. Figure 5.9 plots representative estimates of AI training compute across nearly seven decades on a logarithmic scale, revealing two fitted regimes: total training compute, measured in floating-point operations (FLOPs), followed a comparatively slow pre-2010 trend, then the post-2012 deep-learning frontier accelerated sharply. Large-scale models after 2015 sit orders of magnitude above the pre-2010 trajectory, showing that modern progress reinvests hardware and algorithmic gains into much larger training runs.

Beyond raw compute, this growth carries an energy cost that systems engineers cannot ignore. Under the stated scenario assumptions, a three-day LeNet-1 workstation run would use roughly 54 kWh. Applying an external GPT-4 GPU-day estimate and a data-center-overhead factor yields about 33,600 MWh, comparable to the annual electricity use of roughly 3,200 representative US homes.¹⁸ These are order-of-magnitude estimates, not measured utility records, and they vary with accelerator utilization, facility overhead, and the calculation boundary. Even so, the scale makes energy per operation and achieved utilization design concerns alongside peak FLOP/s. The quantitative energy analysis appears in chapter 11.

Table 5.5 grounds these trends in representative systems, showing how parameters, compute, and hardware co-evolved across four decades of neural network development. Three patterns deserve separate treatment. First, the plotted post-2012 training-compute frontier rises on a timescale of months, although the fitted rate depends on the selected systems and on uncertain estimates. Second, algorithmic and systems advances have reduced the compute needed to reach fixed benchmark targets in some domains, so a frontier-compute trend does not describe efficiency at a constant capability. Third, hardware utilization, reduced precision, software maturity, and procurement conditions mediate the relationship between raw compute and monetary cost; they do not guarantee

¹⁵ | **Perceptron:** A machine built to execute a learning algorithm, directly linking hardware and software from the start. A single linear-threshold layer cannot represent nonlinearly separable functions such as XOR, regardless of additional input hardware. Multilayer nonlinear networks expand the representable function class, but they also introduce the training and systems costs developed in this chapter.

¹⁶ | **Backpropagation:** Short for "backward propagation of errors," the algorithm uses the chain rule to compute how the loss is locally sensitive to millions of weights. It does not establish which weights historically caused an error. Werbos applied it to neural networks in 1974 (Werbos 1974), and the 1986 Rumelhart, Hinton, and Williams publication demonstrated practical effectiveness (Rumelhart et al. 1986). The systems cost: backprop requires storing forward-pass activations and additional training state, creating the several-times-higher memory footprint quantified later in the chapter.

¹⁷ | **Algorithm-system adoption lag:** Backpropagation was available in neural-network form by 1974 (Werbos 1974), while the 1986 Rumelhart, Hinton, and Williams demonstration brought it much wider attention (Rumelhart et al. 1986). Attention mechanisms appeared in neural machine translation in 2014, transformers made them central in 2017, and later TPU- and GPU-scale infrastructure enabled much larger deployments. These histories do not imply that every computationally expensive idea will succeed. They show that an algorithm should be evaluated together with the data, optimization methods, software, hardware, and task regime needed to make it useful.

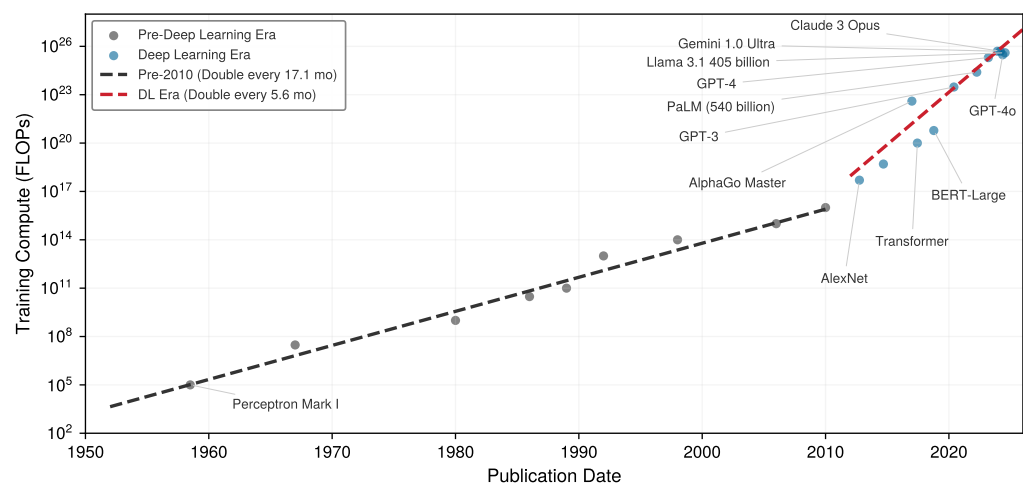


Figure 5.9: Computational Growth: Log-scale scatter plot of representative training-compute estimates in FLOPs from 1958 to 2024. Fitted trend lines compare a slower pre-2010 regime with a steeper post-2012 deep-learning frontier; the values illustrate order-of-magnitude growth rather than an audited census of model training runs.

that cost grows more slowly. For systems engineers, these distinctions affect infrastructure timelines, experimental budgets, and build-versus-buy decisions. Planning therefore requires measured efficiency, a stated comparison target, and explicit uncertainty, especially for closed models whose architecture and training configuration are undisclosed.

Table 5.5: Historical Performance: Four decades of representative neural-network systems showing growth in model state, training compute, and hardware infrastructure. Earlier rows use published specifications where available. GPT-3’s hardware configuration and duration are undisclosed; GPT-4’s architecture, parameter count, hardware configuration, and duration are undisclosed, and its compute entry is an external order-of-magnitude estimate consistent with figure 5.9. Total parameter count also does not determine per-input computation for sparsely activated architectures. “OoM” denotes order-of-magnitude uncertainty.

Year	System	Params	Train FLOPs	Hardware	Train Time	Error/Task
1989	LeNet-1	~10K	10^{11} – 10^{12}	Sun-4/260 workstation	3 days	1% at 12.1% rejection (USPS digits)
1998	LeNet-5	$60K \pm 1K$	$10^{14} \pm 1$ OoM	SGI Origin 2000 (200 MHz)	2–3 days	0.95% (MNIST)
2012	AlexNet	~60M	5×10^{17}	$2 \times$ GTX 580 GPUs	5–6 days	15.3% (ImageNet)
2015	ResNet-152	~60M	$10^{19} \pm 0.5$ OoM	$8 \times$ Tesla K80 GPUs	~3 weeks	3.6% (ImageNet)
2020	GPT-3	175B (exact)	3×10^{23}	Undisclosed	Undisclosed	N/A (language)
2023	GPT-4	Undisclosed	10^{24} – 10^{25} (external est.)	Undisclosed	Undisclosed	N/A (language)

Advances across three dimensions reinforced these trends: data availability, algorithmic innovations, and computing infrastructure. Follow the arrows in figure 5.10 to see the interaction: faster infrastructure made larger data and models practical, growing datasets created new opportunities and pressures for algorithms, and algorithmic advances changed the demands placed on computing systems. The cycle is enabling rather than automatic: progress in one dimension can expose a bottleneck in another rather than produce an immediate gain.

18 | Training energy scale:
The estimate assumes a 750 W workstation for the historical run and uses 10.5 MWh/year as a representative US household electricity budget. The closed-model scenario treats an external GPU-day estimate as A100-equivalent accelerator time and applies a simple data-center overhead. The result illustrates scale; it does not establish the actual energy used to train GPT-4.

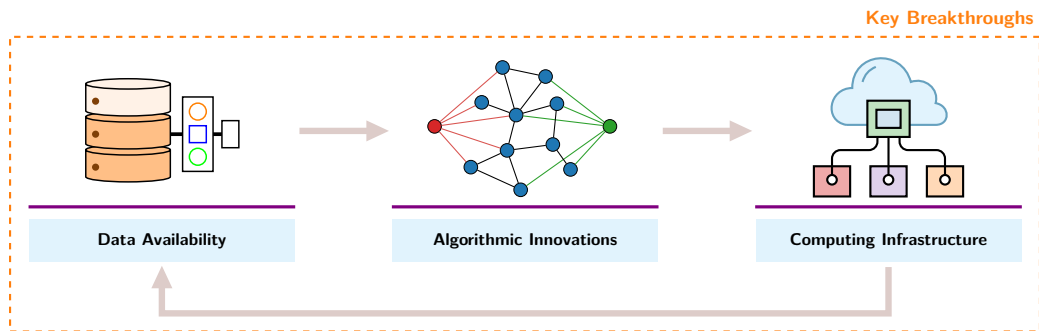


Figure 5.10: Deep Learning Virtuous Cycle: Three mutually reinforcing factors, data availability, algorithmic innovations, and computing infrastructure, form a self-reinforcing loop where breakthroughs in one area create opportunities in the others.

Data availability supplied the volume of examples that learned representations require. The rise of the internet and digital devices created vast new sources of training data: image sharing platforms provided millions of labeled images, digital text collections enabled language processing at scale, and sensor networks generated continuous streams of real-world data. This abundance provided the raw material neural networks needed to learn complex patterns effectively.

Algorithmic innovations turned that volume into trainable systems. New methods for initializing networks and controlling learning rates made training more stable. Techniques for controlling overfitting improved generalization in measured regimes. Researchers also documented empirical relationships among model size, computation, data quantity, and performance, leading to increasingly ambitious architectures while leaving the useful scaling regime task dependent.

The resulting workloads created demand for higher-throughput computing infrastructure, which evolved in response. On the hardware side, GPUs provided the parallel processing capabilities needed for efficient neural network computation, and Tensor Processing Units (TPUs)¹⁹ (Jouppi et al. 2023) pushed performance further. High-bandwidth memory systems and fast interconnects addressed data movement challenges. Software advances matched the hardware evolution: frameworks and libraries simplified building and training networks, distributed computing systems enabled training at scale, and tools for optimizing model deployment reduced the gap between research and production.



Neural-network history is a scale explosion in training energy.

¹⁹ | **TPU (tensor processing unit):** Google’s custom accelerator, first deployed internally in 2015, was optimized specifically for the matrix multiplications that dominate neural network workloads. The TPU v1 achieved 92 TOPS (vendor-reported INT8 tera-operations/s) for inference at 75 W, a power-efficiency point that general-purpose GPUs of the era could not match. The name “Tensor Processing Unit” reflects the design decision to sacrifice general-purpose flexibility for maximum throughput on the operation neural networks need most.

Checkpoint 5.1: Understanding deep learning’s emergence

Before connecting this history to its infrastructure consequences, verify your understanding of why deep learning emerged:

- ☐ **Rule-based limits:** can you explain why rule-based programming fails for tasks like image recognition?
- ☐ **Classical ML vs. deep learning:** can you compare classical ML’s hand-engineered features with deep learning’s automatic feature learning?
- ☐ **Three enabling factors:** can you describe the three factors that converged to enable modern deep learning (data, algorithms, infrastructure)?
- ☐ **Hardware demand:** can you explain why deep learning demands specialized hardware more often than traditional programming?

If any of these concepts remain unclear, review the relevant sections before continuing. The systems consequences that follow build directly on this conceptual foundation.

The convergence of data availability, algorithmic innovation, and computing infrastructure transformed neural networks from theoretical formulations into dominant production workloads. As modern frontier models press against memory bandwidth, interconnect latency, and thermal budgets, systems engineering shifts from high-level model definition to physical execution. Managing

these bottlenecks requires examining the concrete mathematical primitives and hardware dataflows that execute across accelerators.

5.3 Neural Network Fundamentals

The question now is *why* the computational demands can become so extreme. For suitable neural workloads, a GPU can outperform a CPU because it provides high parallel throughput and data reuse for matrix and tensor operations, not simply because of clock speed. The structure of these network primitives—neurons, layers, and nonlinear activations—determines how data flows through hardware and how memory demands scale during training and inference. Understanding these operations reveals how simple arithmetic on individual neurons compounds into the infrastructure requirements that shaped modern AI.

The concepts here apply to the feed-forward neural networks that anchor this chapter and recur in many larger architectures. Their common fundamentals are weighted sums, nonlinear activations, and gradient-based learning. Mastering these operations and their computational characteristics provides a basis for reasoning about more specialized networks, while later chapters introduce operators and execution patterns that this simple model does not contain.

5.3.1 Why depth matters: The power of hierarchical representations

A single-layer network attempting to classify handwritten digits must map raw pixels directly to labels. A deeper network can instead exploit hierarchical structure when the target function is compositional. The question is *why* depth can provide such representational advantages, and the answer grounds all the mathematical development that follows.

Deep networks can exploit **compositionality**: complex patterns may decompose into simpler patterns that themselves decompose further. In image recognition, learned representations can progress from local edges and textures toward parts and objects. Such hierarchical regularities occur in many datasets and motivate layered models, although a trained network need not recover this exact sequence.

Consider recognizing the digit “seven” in the MNIST example. A single-layer network maps all 784 pixel values directly to a decision. When the task has compositional structure, a deep network can compute the decision more parameter-efficiently by reusing intermediate features:

- **Layer 1** learns simple edge detectors: vertical lines, horizontal lines, diagonal strokes
- **Layer 2** combines edges into shapes: the horizontal top stroke of a “seven,” the diagonal downstroke
- **Layer 3** combines shapes into complete digit patterns

Each layer builds on the previous, allowing later computations to reuse earlier features. This hierarchy can be much more parameter-efficient than a shallow representation when the task has suitable compositional structure. The same edge detectors learned for “seven” can also contribute to recognizing “one,” “four,” and other digits. This **parameter reuse** helps explain why depth is an effective design choice without implying a fixed parameter advantage for every task. However, the choice between adding layers and widening existing ones is not symmetric: depth and width contribute to representational capacity through different mechanisms.

Layered processing in biological vision helped motivate hierarchical models, without implying that artificial networks reproduce the same mechanisms. The architectures examined in chapter 6 encode different structural assumptions for images, sequences, and other data types. Depth explains why layered representations can be useful; the remaining mechanics explain how the hierarchy is implemented. The following sections develop those mechanics: how neurons compute, how layers connect, and how information flows from input to output.

Systems Perspective 5.2: The depth vs. width trade-off

For certain function classes, depth provides an exponential representational advantage: a network with N_L layers can represent functions that require exponentially more units in a

shallow network (Telgarsky 2016). This restricted theoretical result explains why nonlinear composition can be parameter-efficient; it is not a universal guarantee for every task.

However, depth introduces three engineering challenges with each additional layer:

- Adds sequential dependencies (layer $\ell + 1$ waits for layer ℓ), limiting parallelism
- Increases gradient path length, risking vanishing/exploding gradients
- Requires storing intermediate activations for backpropagation

Modern architectures balance depth against width. A network with ten layers of 100 neurons has the same 1,000 total hidden units as one with two layers of 500, but not the same parameter count, function class, or execution graph. Both expose parallelism within each layer; the deeper network adds sequential dependencies and may represent some compositional functions more efficiently.

5.3.2 Network architecture fundamentals

A neural network’s architecture determines how information flows from input to output. Modern networks can be enormously complex, but they all build on a few organizational principles that shape both implementation and the computational infrastructure they demand.

Handwritten digit recognition grounds these concepts in a concrete example, specifically the task of classifying images from the MNIST dataset (LeCun et al. 1998). This seemingly simple task reveals all the core principles of neural networks while providing intuition for more complex applications.

Each architectural choice, from how neurons are connected to how layers are organized, creates specific computational patterns that must be efficiently mapped to hardware. This mapping between network architecture and computational requirements is essential for building scalable ML systems.



Example 5.1: Running example: MNIST digit recognition

Scenario: Deploying a basic handwritten digit classifier on 28×28 grayscale images across 10 output classes.

Diagnosis: The chapter’s running architecture maps 784 raw pixel features through 128 and 64 hidden units to 10 outputs, creating roughly 100K parameters.

Systems lesson: Inspecting input vector dimensions and layer parameters on a compact dataset establishes baseline compute and memory bandwidth requirements before scaling to high-dimensional production workloads.

5.3.2.1 The perceptron’s weighted sum

The computational machinery within each dense layer is a perceptron-style artificial neuron whose signal path (inputs, weights, bias, aggregation, activation) section 5.2.5 traced through its biological origins. What remains is to formalize that path mathematically, because this repeated operation becomes the layer-level kernel that hardware executes. In the MNIST network, a first-layer unit combines all 784 pixel intensities into one activation that may respond to a learned pattern, such as a vertical edge shared by “one” and “seven.” Follow figure 5.11 from left to right: each input x_i is multiplied by its corresponding weight w_{ij} , the products are accumulated with a bias, and an activation function transforms the resulting score. Different neurons repeat the same structure with different learned weights, allowing one layer to test many patterns against the same input.

Each input x_i has a corresponding weight w_{ij} , and the perceptron multiplies each input by its matching weight. The intermediate output, z , is computed as the weighted sum of inputs in equation 5.1:

$$z = \sum (x_i \cdot w_{ij}) \quad (5.1)$$

Here, j identifies the receiving neuron once perceptrons are assembled into a layer, and the sum runs over all input indices i . In plain terms, each input feature is scaled by how important it is (its weight), and the results are summed into a single score. This is the dot product of two vectors—the fundamental operation that hardware accelerators are designed to execute at maximum

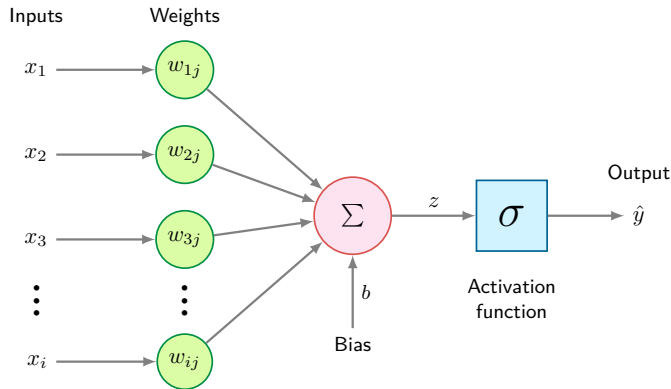


Figure 5.11: Perceptron Architecture: Inputs enter on the left, each scaled by its own weight, and the weighted products plus a bias meet at the summation node before a single activation function produces the output. The multiplies and the accumulation along this path are the per-neuron unit of work that the chapter’s layer-cost calculations multiply out.

throughput, and the reason neural network performance is measured by the number of MAC operations completed per second.

A bias term b shifts the linear output up or down, giving the model additional flexibility to fit the data. Thus, the intermediate linear combination computed by the perceptron including the bias becomes equation 5.2:

$$z = \sum (x_i \cdot w_{ij}) + b \quad (5.2)$$

With this weighted sum in hand, a suitable output activation and loss can support regression or classification. A binary linear-threshold classifier selects one of two classes around a threshold; multiclass networks instead produce several scores and commonly normalize them with softmax.

Under the direct scalar, no-reuse accounting established earlier, one neuron performs N multiply-accumulate operations and moves its inputs, weights, bias, and output. What the formalization adds is the layer view. A dense layer of M neurons repeats the weighted sum M times, so its arithmetic cost is $M \times N$ MACs—exactly the matrix multiplication $\mathbf{xW}$ that hardware must execute, and the form in which the rest of this chapter counts work.

5.3.2.2 Nonlinear activation functions

Activation functions are where expressiveness, gradient flow, and hardware cost meet. They convert linear weighted sums into nonlinear outputs; without them, multiple linear layers would collapse into a single linear transformation, severely limiting the network’s expressive power. Figure 5.12 compares three commonly used element-wise activation functions and one vector-level function (softmax), each with mathematical characteristics that shape both learning behavior and execution cost.

The choice of activation function affects both learning and computational efficiency, revealing how systems constraints shape algorithmic design. ReLU ($\max(0, x)$) became a common hidden-layer activation for feed-forward networks because it is inexpensive, has local derivative one for positive inputs, and can produce sparse activations. Earlier differentiable deep networks often used sigmoid or tanh. Their smooth S-curves can saturate, causing gradients to shrink across many layers and impeding early-layer learning. That contrast is not a simple ranking: sigmoid remains appropriate for some binary outputs, tanh remains useful when bounded zero-centered activations are desired, and modern architectures use additional activation families. The relevant design question is which function supplies the required output semantics and gradient behavior at an acceptable execution cost. Understanding those roles explains ReLU’s adoption without turning it into a universal default.

Sigmoid The logistic sigmoid function²⁰ squashes continuous inputs onto the open interval $(0, 1)$ (equation 5.3):

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (5.3)$$

²⁰ | **Sigmoid:** From Greek *sigma* + *eidos* (“sigma-shaped”), referring to the S-curve that maps inputs to the bounded $(0, 1)$ range. The mapping requires an exponential approximation, commonly implemented through specialized instructions, lookup tables, or polynomial methods. It is generally more expensive than ReLU’s comparison-and-selection operation, although the measured gap depends on precision, vectorization, and hardware. ReLU’s adoption reflected both easier gradient flow on its positive branch and cheaper execution.

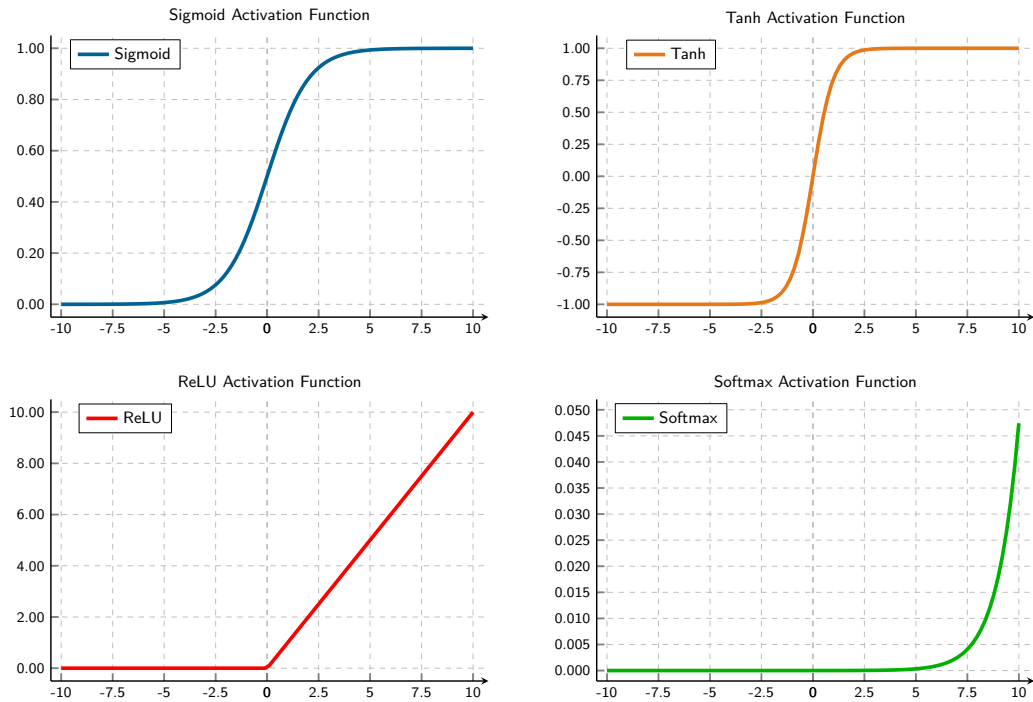


Figure 5.12: Common Activation Functions: Four nonlinear activation functions plotted with their output ranges. Sigmoid maps inputs to $(0, 1)$ with smooth gradients, tanh provides zero-centered outputs in $(-1, 1)$, and ReLU outputs zero for negative inputs. Softmax (bottom-right) shows one component of a 3-element vector while the other two logits are fixed at 13 and 0. Its magnified y-axis (roughly 0 to 0.05) is not comparable to the other panels, and it shows only one component of a vector whose outputs are interdependent and sum to 1.

The S-shaped curve maps logits to $(0, 1)$, so sigmoid can parameterize a binary probability. It approaches one for large positive inputs and zero for large negative inputs. Although sigmoid is smooth and differentiable everywhere, gradient-based learning need not be: ReLU uses an assigned subgradient at zero.

Sigmoid has a significant limitation: for inputs with large absolute values (far from zero), the gradient becomes extremely small, a phenomenon called the **vanishing gradient problem**.²¹ During backpropagation, these small gradients multiply together across layers, causing gradients in early layers to become exponentially tiny. This effectively prevents learning in deep networks, as weight updates become negligible.

Sigmoid outputs are not zero-centered. When the inputs feeding a parameter update are also nonnegative, their weight gradients can become correlated in sign, producing less direct optimization trajectories. The complete gradient still depends on upstream errors and is not universally one-signed.

Tanh The hyperbolic tangent²² centers output activations around zero over the interval $(-1, 1)$ (equation 5.4):

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (5.4)$$

Tanh produces an S-shaped curve similar to sigmoid but centered at zero: negative inputs map to negative outputs and positive inputs to positive outputs. This symmetry balances gradient flow during training, often yielding faster convergence than sigmoid.

Like sigmoid, tanh is smooth and differentiable everywhere, and it still suffers from the vanishing gradient problem for inputs with large magnitudes. When the function saturates (approaches -1 or 1), gradients shrink toward zero. Despite this limitation, tanh's zero-centered outputs make it preferable to sigmoid for hidden layers in many architectures, particularly in recurrent neural networks where maintaining balanced activations across time steps is important.

²¹ | **Vanishing gradient problem:** The chain rule's multiplication of Jacobian factors across layers can cause gradients to shrink. Sigmoid's derivative is at most 0.25, so along a simplified 10-layer path the activation-derivative factors alone contribute at most $0.25^{10} \approx 10^{-6}$. The complete gradient also includes weights and other operations, but repeated saturated activation derivatives can still make early-layer updates negligible.

²² | **Tanh (hyperbolic tangent):** By centering its output range on zero, tanh can reduce the correlated-update behavior associated with all-positive activations. Its computational cost is similar to sigmoid because it also depends on exponential approximations, while its saturation behavior still permits vanishing gradients.

Both sigmoid and tanh share a critical limitation: gradient saturation at extreme input values. The search for an activation function that avoids this problem while remaining computationally efficient led to one of deep learning's most important innovations.

ReLU ReLU²³ sets negative values to zero while maintaining linear identity for positive inputs (equation 5.5):

$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases} \quad (5.5)$$

ReLU's characteristic shape—a straight line for positive inputs and zero for negative inputs—provides three advantages. First, its local derivative is one for positive inputs, avoiding activation saturation on that branch; the complete network gradient can still shrink or grow through weights and other operations. Second, ReLU introduces deterministic activation sparsity by zeroing negative values; this differs from dropout's stochastic masking.²⁴ Zero values do not automatically produce a speedup, because ordinary dense kernels still process their positions unless the implementation exploits sparsity. Third, ReLU uses a comparison and selection, $\text{output} = (\text{input} > 0) ? \text{input} : 0$, rather than an exponential approximation. These are local properties of the operator, while end-to-end performance depends on how it is fused with neighboring work.

ReLU is not without drawbacks. The **dying ReLU problem**—neurons that permanently output zero and cease learning—occurs when neurons become stuck in the inactive state. If a neuron's weights evolve during training such that the preactivation $z = \mathbf{w}^T \mathbf{x} + b$ is consistently negative across all training examples, the neuron outputs zero for every input. Since ReLU's gradient is also zero for negative inputs, no gradient flows back through this neuron during backpropagation: the weights cannot update, and the neuron remains dead. This can happen with large learning rates that push weights into unfavorable regions. From a systems perspective, dead neurons represent wasted capacity: parameters that consume memory and compute during inference but contribute nothing to the output. Careful initialization (He et al. 2015), moderate learning rates, and leaky ReLU variants help mitigate dying ReLUs. Batch normalization²⁵ instead stabilizes layer inputs and permits higher learning rates (Ioffe and Szegedy 2015).

Softmax Unlike *element-wise* activation functions that operate independently on each value, softmax²⁶ is a *vector-level* function: it considers all values simultaneously to produce a probability distribution. In simple classifiers, softmax is commonly used in the output layer rather than as an element-wise hidden activation; it also normalizes learned importance-score vectors in attention architectures. The softmax function is defined in equation 5.6:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad (5.6)$$

For a vector of K values (often called logits²⁷), softmax transforms them into K probabilities that sum to 1. One component of the softmax output appears in figure 5.12 (bottom-right); in practice, softmax processes entire vectors where each element's output depends on all input values.

In multiclass classifiers, softmax is commonly used in the output layer. It converts arbitrary real-valued logits into normalized predicted probabilities, but those values are not necessarily calibrated confidence estimates. The class with the highest probability becomes the predicted class, and larger logits receive disproportionately higher probabilities.

The mathematical relationship between input logits and output probabilities is differentiable, allowing gradients to flow back through softmax during training. When combined with cross-entropy loss (discussed in section 5.4.3), softmax produces particularly clean gradient expressions that guide learning effectively. Beyond their mathematical properties, the choice of activation functions has direct consequences for hardware efficiency.

23 | ReLU (rectified linear unit): Unlike the costly exponential operations in prior activation functions, ReLU's $\max(0, x)$ operation maps to a simple comparison and selection. This efficiency, together with better gradient flow for positive activations, helped make the deep architectures of the AlexNet era computationally tractable (Nair and Hinton 2010; Krizhevsky et al. 2012).

24 | Dropout: Randomly deactivating neurons during training forces a network to learn redundant representations, a regularization technique used in the AlexNet-era shift toward deep vision models (Srivastava et al. 2014; Krizhevsky et al. 2012). This creates a systems-level divergence between the computational graphs for training (stochastic) and inference (deterministic). Failing to switch from the training to the inference graph is a common bug that can silently degrade accuracy.

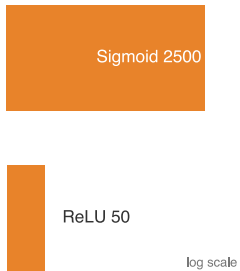
25 | Batch normalization systems cost: BatchNorm adds two learned parameters per feature (scale γ and shift β) and behaves differently during training and inference: training uses live mean and variance from the mini-batch, while inference uses frozen running statistics; by keeping activation distributions better scaled, it stabilizes training and can permit higher learning rates, but it also makes the layer depend on batch statistics. Small batches can produce noisy mean and variance estimates, so a mathematical stabilizer becomes a systems choice about batch size, activation memory, and training-serving parity. Later architectures and large-scale training settings introduce further variants of the same batch-statistics trade-off.

Systems Perspective 5.3: Activation functions and hardware

ReLU's widespread adoption comes from hardware efficiency, alongside its avoidance of vanishing gradients. Computing $\max(0, x)$ requires a single comparison operation, while

26 | Softmax: The name reflects its role as a “soft” or differentiable version of `argmax`, a function that must evaluate an entire vector to find its maximum value. This vector-wise operation is not used as a drop-in replacement for element-wise nonlinearities such as ReLU, but it is central whenever a model must normalize a vector of scores, including classification heads and attention layers. Direct exponentiation can overflow in 32-bit floating point. Stable implementations subtract the largest logit before exponentiating; the shift leaves every softmax probability unchanged while making the largest exponent equal to one.

27 | Logits: In multiclass neural classifiers, logits are unnormalized output scores. Score differences correspond to log probability ratios after softmax, while an individual multiclass logit is not itself a calibrated log-odds quantity. Because `argmax` over logits and `argmax` over softmax probabilities select the same class, inference can skip softmax when only the top class is needed.



An illustrative logic model shows why ReLU is generally cheaper to implement.

sigmoid and tanh require computing exponentials—operations that are much more expensive in both time and energy. The transistor-tax calculation in section 5.3.2.3 models this gap as a logic-cost ratio, and chapter 11 covers the broader benchmark and accelerator-implementation implications.

5.3.2.3 The transistor tax: Logic unit cost

The activation decision has a second cost beyond gradient behavior: silicon area. In computer architecture, the **logic unit cost** is measured by transistor count and energy per operation.

A ReLU datapath can be implemented with comparison and selection logic, while sigmoid or tanh requires an approximation to a transcendental function, such as a lookup table or polynomial evaluation. The illustrative assumptions here assign 50 transistor-equivalents to ReLU and 2,500 to a high-precision exponential datapath. Actual area, latency, and energy depend on precision, approximation method, sharing, throughput target, and hardware technology.

This illustrative disparity is the **transistor tax**: under the stated assumptions, the modeled silicon “price” of sigmoid is 50× that of ReLU. The exact ratio is not a hardware constant. The durable systems lesson is that simple piecewise-linear activations are generally cheaper to implement than transcendental functions, while ReLU’s adoption also reflects its optimization behavior and gradient flow.

These transformations convert the linear input sum into a nonlinear output, yielding the generic artificial-neuron computation in equation 5.7:

$$\hat{y} = f(z) = f\left(\sum_i (x_i \cdot w_{ij}) + b\right) \quad (5.7)$$

This nonlinearity expands what multilayer networks can represent. Without it, a stack of linear layers collapses algebraically into one linear transformation, regardless of how many intermediate matrices the implementation evaluates. In figure 5.13, the left panel shows a linear boundary that cannot separate the classes, while the right shows one curved boundary that a multilayer network with nonlinear activations and suitable learned parameters can represent. The figure illustrates representational possibility rather than a guarantee that training will discover that boundary. Optimization, data coverage, and architecture still determine which function the learned parameters produce.

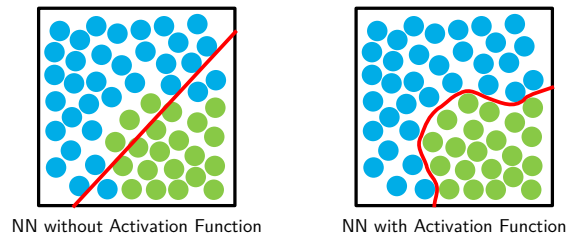


Figure 5.13: Linear vs. Nonlinear Decision Boundaries: Two scatter plots compare classification with and without activation functions. Without activation, a straight line fails to separate the two classes. With a nonlinear activation function applied, the network produces a curved decision boundary that correctly separates the points.

The **universal approximation theorem**²⁸ establishes that sufficiently wide networks with suitable activation functions can approximate broad classes of functions arbitrarily well. This theoretical foundation, combined with the computational and optimization characteristics of specific activation functions like ReLU and sigmoid, explains neural networks’ practical effectiveness in complex tasks. The theorem, however, states a pure existence result under the assumption of unlimited width; it says nothing about the accelerator executing the computation. Physical ML systems impose hard limits on both dimensions of the width-vs.-depth trade-off: a single hidden layer wide enough to approximate a complex function requires parameter storage and activation memory that can exhaust accelerator VRAM entirely, and a network too deep to fit its activations in memory stalls the backward pass on the same constraint. The engineering problem is therefore not whether a network of sufficient width or depth *exists* but whether it fits within the memory and bandwidth envelope of the target hardware.

5.3.2.4 Layers and connections

Individual neurons compute weighted sums, apply bias terms, and pass results through activation functions. The power of neural networks, however, comes from organizing these neurons into layers—collections of units that execute in parallel across shared inputs. An input layer ingests raw feature tensors; one or more hidden layers transform those representations through successive matrix multiplications and nonlinear activations; and an output layer emits task-specific predictions or decision logits.

In figure 5.14, data enters at the input layer, passes through multiple hidden layers that progressively extract more abstract features, and emerges at the output layer as a prediction. Each successive layer transforms the representation, building increasingly complex features—a hierarchical processing pipeline that gives deep neural networks their ability to learn complex patterns.

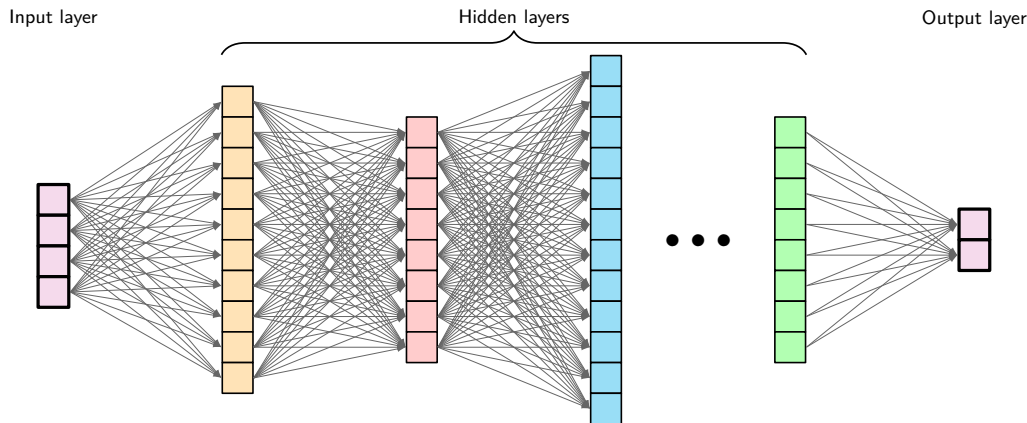


Figure 5.14: Layered Network Architecture: Deep neural networks transform data through successive layers, enabling the extraction of increasingly complex features and patterns. Each layer applies nonlinear transformations to the outputs of the previous layer, ultimately mapping raw inputs to desired outputs.

As data flows through the network, it is transformed at each layer to extract meaningful patterns. The weighted summation and activation process introduced for individual neurons scales up: each layer applies these operations in parallel across all its neurons, with outputs from one layer becoming inputs to the next. This creates a hierarchical pipeline where simple features detected in early layers combine into increasingly complex patterns in deeper layers—enabling neural networks to learn sophisticated representations from raw data.

5.3.3 Parameters and connections

The learnable parameters²⁹ of neural networks, weights and biases, determine how information flows through the network and how transformations are applied to input data. Their organization directly impacts both learning capacity and computational requirements.

5.3.3.1 Weight matrices

Weights determine how strongly inputs influence neuron outputs. In dense layers, these weights organize into matrices that map efficiently to linear-algebra kernels. In a layer with n input features and m neurons, the weights form a matrix $\mathbf{W} \in \mathbb{R}^{n \times m}$, where each column represents the weights for a single neuron. A matrix-vector product processes one input, while a matrix-matrix product processes a batch of inputs and can expose more parallel work and data reuse.

For a single neuron, preactivation is the scalar dot product $z = \sum_{i=1}^n (x_i \cdot w_{ij}) + b$. For a layer of m neurons, evaluating units independently wastes opportunities for SIMD vectorization and cache reuse. Formulating the layer as a matrix multiplication computes all m preactivations in a single dense linear-algebra call (equation 5.8):

$$\mathbf{z} = \mathbf{x}\mathbf{W} + \mathbf{b} \quad (5.8)$$

28 | Universal approximation theorem: Under conditions on the activation, target function, compact domain, and approximation tolerance, a sufficiently wide single-hidden-layer network can approximate the target arbitrarily well. This existence result is nonconstructive: it does not specify how to find the weights or how many neurons practical training will require. For some function classes, depth provides an exponential representational advantage over shallow networks, but this separation is not universal.

29 | Parameter memory cost: Parameter count is a misleading proxy for memory importance; some layers add small learned scale or shift parameters that barely affect byte budgets but can strongly affect model behavior when training choices change. Conversely, the bulk of parameters in dense weight matrices require extra state during training: gradients plus optimizer bookkeeping beyond the stored weights themselves. A model that fits in memory for inference may therefore require several times more memory for training, a cost quantified later in the training memory budget.

This equation computes every neuron's preactivation for one input: the vector $\mathbf{x}$ multiplied by the weight matrix $\mathbf{W}$ produces all m values, and the bias vector $\mathbf{b}$ shifts each one. For a batch, the input becomes a matrix and the same layer becomes a matrix-matrix product. These kernels account for much of the work in dense feed-forward networks, but whether they dominate runtime—and whether they are compute bound or memory bound—depends on their dimensions, batch size, precision, implementation, and target hardware.

This matrix organization is more than mathematical convenience; it reflects how modern neural networks are implemented for efficiency. Each weight w_{ij} represents the strength of the connection between input feature i and neuron j in the layer.

The canonical case studied here connects each neuron in a layer to every neuron in the previous layer, forming a “dense” or “fully-connected” layer. This pattern allows each neuron to use all features from the previous layer. Fully connected layers establish the mathematical principles needed here, while alternative connectivity patterns (explored in chapter 6) exploit structure in the data and can reduce work or improve statistical efficiency.

Figure 5.15 makes the dense pattern explicit by laying out a small three-layer network with every connection weight labeled. Every input connects to every hidden neuron, and every hidden neuron connects to every output. Each labeled edge represents one learnable weight, making the parameter count and arithmetic structure visible. Read one destination neuron at a time: its incoming edges form one column of the preceding weight matrix, and its bias shifts the resulting weighted sum. Reading all destinations together reveals the matrix operation. The numerical labels are illustrative rather than a worked forward pass; the important relationship is the matrix shape. For layers of sizes (n_1, n_2, n_3) , the weight matrices are $\mathbf{W}^{(1)} \in \mathbb{R}^{n_1 \times n_2}$ and $\mathbf{W}^{(2)} \in \mathbb{R}^{n_2 \times n_3}$, requiring $n_1 n_2 + n_2 n_3$ weights before biases.

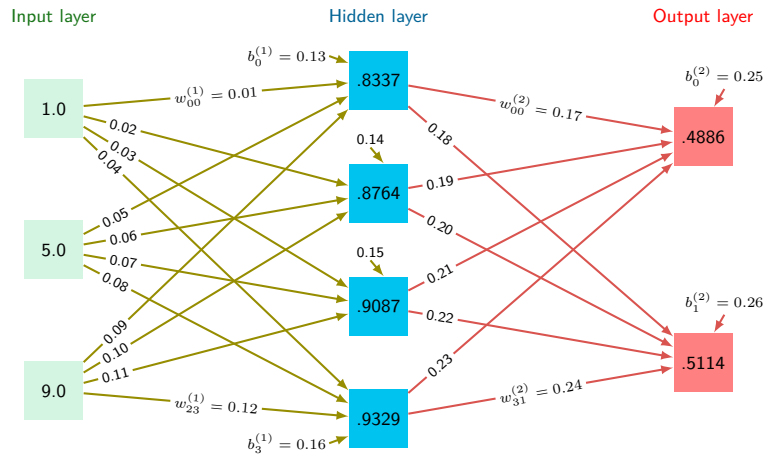


Figure 5.15: Fully-Connected Layers: A three-layer network with dense connections between layers, where each neuron integrates information from all neurons in the preceding layer. Labeled edges and nodes illustrate the organization of weights, biases, and activations; they are not a worked numerical forward pass.

³⁰ | **Bias terms:** Biases add one parameter per neuron, compared with n weights for a neuron with n inputs, so they account for $1/(n+1)$ of that layer's parameters. Their model-wide share and effect on accuracy depend on layer widths, architecture, and normalization. Some modern architectures omit biases in layers followed by batch normalization because the normalization transform includes a learned shift.

5.3.3.2 Bias terms

Each neuron in a layer also has an associated bias term³⁰. While weights determine the relative importance of inputs, biases allow neurons to shift their activation functions. This shifting is important for learning, as it gives the network flexibility to fit more complex patterns.

For a layer with m neurons, the bias terms form a vector $\mathbf{b} \in \mathbb{R}^m$. During layer execution, this bias vector is added to the weighted sum of inputs (the same form as equation 5.8), shifting each neuron's activation threshold independently:

$$\mathbf{z} = \mathbf{x}\mathbf{W} + \mathbf{b}$$

The organization of weights and biases across a feed-forward network follows a systematic pattern. For each parameterized affine layer ℓ , three components define its computation; the weights and biases are learned, while the activation function is usually chosen by the designer:

- A weight matrix $\mathbf{W}^{(\ell)}$
- A bias vector $\mathbf{b}^{(\ell)}$
- An activation function $f^{(\ell)}$

This yields the complete layer computation in equation 5.9:

$$\mathbf{a}^{(\ell)} = f^{(\ell)}(\mathbf{z}^{(\ell)}) = f^{(\ell)}(\mathbf{a}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}) \quad (5.9)$$

Here, $\mathbf{a}^{(\ell)}$ (written as $\mathbf{A}^{(\ell)}$ for batches) represents the layer's activation output. The row-vector convention is adopted throughout: each sample is a row, and the weight matrix $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_{\ell-1} \times n_{\ell}}$ maps from the previous layer's width to the current layer's width. With this equation in place, the core architecture concepts are complete enough to proceed.



Checkpoint 5.2: Neural network architecture fundamentals

Before proceeding to network topology and training, verify your understanding of the foundational concepts:

Use the MNIST architecture $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$ as the running check.

- ☐ **Neuron equation:** can you write the neuron equation, including weighted sum, bias, and activation function?
- ☐ **ReLU vs. sigmoid:** can you explain why ReLU is computationally cheaper than sigmoid while still providing the nonlinearity a multilayer network needs?
- ☐ **Layers to matrices:** can you map input, hidden, and output layers to the corresponding weight matrices $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n \times m}$?
- ☐ **Parameter count:** can you calculate the total parameter count, including both weights and biases?
- ☐ **Width, depth, cost:** can you explain how layer width, depth, and connectivity change memory footprint and arithmetic cost?

If any of these feel unclear, review the earlier sections on neural network fundamentals, artificial neurons, and parameters before continuing. The upcoming sections on training and optimization build directly on these foundations.

5.3.4 Architecture design

Architecture design determines how individual neurons, activation functions, and weight matrices connect to solve problems that no single layer can handle. That design has direct systems consequences, because each topological choice (adding a layer, widening a hidden dimension, changing connectivity) changes the parameter count, memory footprint, and arithmetic cost.

Network topology describes how individual neurons organize into layers and connect to form complete neural networks. Building intuition begins with a simple problem that became famous in AI history.³¹

The XOR example motivates a network with a hidden layer, but real-world networks require systematic consideration of design constraints and computational scale. Recognizing handwritten digits using the MNIST (LeCun et al. 1998) dataset illustrates how problem structure determines network dimensions while hidden layer configuration remains an important design decision.



Example 5.2: Building intuition: The XOR problem

Consider a network learning the XOR function, a classic problem that requires nonlinearity. With inputs x_1 and x_2 that can be 0 or 1, XOR outputs 1 when inputs differ and 0 when they are the same.

³¹ | **XOR problem:** XOR is the canonical example of why network topology matters. A single linear-threshold layer cannot represent this nonlinearly separable function, regardless of how many units it contains, whereas a small network with a nonlinear hidden layer can. The exact number of units required depends on the activation and representation conventions; the durable lesson is that adding nonlinear depth changes the class of functions the network can express.

Setup: Two inputs $\rightarrow$ two hidden neurons $\rightarrow$ one output

Example: For inputs (1, 0):

- Hidden neuron one: $h_1 = \text{ReLU}(1 \cdot w_{11} + 0 \cdot w_{12} + b_1)$
- Hidden neuron two: $h_2 = \text{ReLU}(1 \cdot w_{21} + 0 \cdot w_{22} + b_2)$
- Output: $y = \text{sigmoid}(h_1 \cdot w_{31} + h_2 \cdot w_{32} + b_3)$

Systems insight: Hidden layers do not merely add capacity; they change the class of functions the system can represent. XOR is the small example that makes depth a structural requirement rather than a decorative design choice (Minsky and Papert 1969).

5.3.4.1 Feedforward network architecture

Applying the three-layer architecture to MNIST reveals how data characteristics and task requirements constrain network design. Compare the two panels in figure 5.16 to see this architecture from both perspectives: panel (a) schematically represents a 28×28 pixel grayscale image connected to the hidden and output layers, while panel (b) shows the same values reshaped into a 784-dimensional feature vector. Matrix multiplication can operate on multidimensional arrays, but this fully connected layer defines one weight per input feature and hidden unit, so the image axes are not preserved in its parameterization. Convolutional architectures, explored in chapter 6, instead encode spatial locality directly.

The input layer's width is directly determined by the data format. For a 28 by 28 pixel image, each pixel becomes an input feature, requiring 784 input neurons. Equivalently, 28 rows and 28 columns flatten into 784 values. The data can be represented either as a 2D spatial grid of pixels or as a flattened 1D vector, where each value represents the intensity of one pixel.

The output layer's structure is determined by the task requirements. For 10-class digit recognition, the final layer provisions 10 output neurons, one for each possible digit (0–9). When presented with an image, the network produces a score for each output neuron; a softmax can then convert those logits into predicted class probabilities.

Between these fixed input and output layers, there is flexibility in designing the hidden layer topology. The choice of hidden layer structure, including the number of layers to use and their respective widths, represents one of the key design decisions in neural networks. Additional layers increase the network's depth, allowing it to learn more abstract features through successive transformations. The width of each layer provides capacity for learning different features at each level of abstraction.

Widening hidden layers from 100 to 1000 neurons increases parameters from ~89.6K to ~1.8M (~358 KB vs. ~7 MB in FP32). Whether the wider network improves accuracy requires training and evaluation; the storage increase alone already matters for mobile deployment budgets.

5.3.4.2 Layer connectivity design patterns

The fully connected architecture in section 5.3.3 maximizes flexibility by connecting every neuron to every neuron in the next layer, but connectivity itself is an engineering decision. Dense, sparse, and skip patterns trade learning flexibility against parameter count, locality, and gradient flow.

Dense connectivity represents the standard pattern where each neuron connects to every neuron in the subsequent layer. In the MNIST example, connecting the 784-dimensional input layer to a hidden layer of 128 neurons requires 100,352 weight parameters (784×128). This full connectivity enables the layer to learn arbitrary linear combinations of its inputs; nonlinear activations between dense layers extend those combinations to nonlinear relationships. The number of parameters grows as the product of adjacent layer widths, so widening one layer scales them linearly while widening both scales them quadratically.

Sparse connectivity patterns introduce purposeful restrictions in how neurons connect between layers. Rather than maintaining all possible connections, neurons connect to only a subset of neurons in the adjacent layer. This approach draws inspiration from biological neural systems, where neurons typically form connections with a limited number of other neurons. In visual processing tasks like the MNIST example, neurons might connect only to inputs representing nearby pixels, reflecting the local nature of visual features.

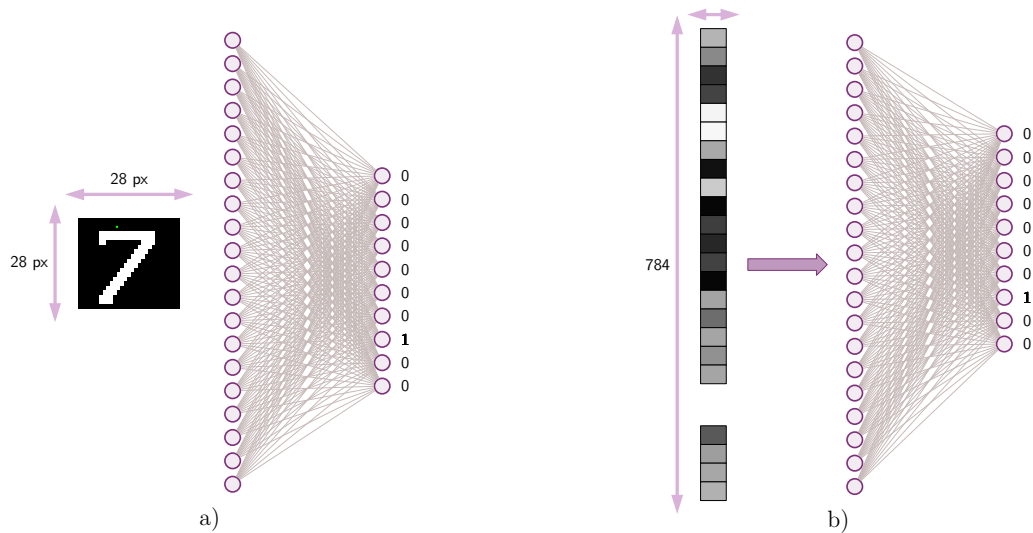


Figure 5.16: MNIST Network Topology: Two panels show the network architecture for digit recognition. Panel (a) schematically represents a 28×28 pixel image of a digit connected through hidden layers to 10 output nodes. Panel (b) shows the same architecture with the input image flattened into a 784-element vector, illustrating how spatial data enters the network.

As networks grow deeper, the path from input to output becomes longer, potentially complicating the learning process. Skip connections address this by adding direct paths between nonadjacent layers. These connections provide alternative routes for information flow, supplementing the standard layer-by-layer progression. In a digit recognition pipeline, skip connections allow later layers to reference both high-level patterns and the original pixel values directly.

These connection patterns have direct hardware consequences as well as theoretical ones. Dense connections maximize learning flexibility and map efficiently onto general matrix multiply (GEMM) kernels on tensor-oriented hardware. Sparse connections reduce theoretical parameter counts and FLOPs, but a sparse weight matrix does not automatically execute faster than a dense one on standard hardware—without structured sparsity support (such as 2:4 patterns that hardware can exploit through compressed index formats), the arithmetic reduction does not translate to proportional reductions in execution time. Skip connections help maintain effective information flow in deeper networks while preserving the gradient paths that make very deep architectures trainable.

5.3.4.3 Model size and computational complexity

How parameters (weights and biases) are arranged determines both learning capacity and computational cost—this is the model’s side of the silicon contract (section 1.7): the parameter count, their numerical precision, and the operations they require collectively define the computational bargain the model strikes with hardware. While topology defines the network’s structure, parameter initialization and organization directly affect learning dynamics and final performance.

Worked example: Training vs. inference memory A single forward pass through the $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$ network costs 109,184 MACs. The memory footprint during training tells a different story. Evaluating the memory footprint at batch size 32 in 32-bit (4-byte) floating-point precision reveals the gap between training and inference demands across parameters, activations, gradients, and optimizer state.

The first contribution is the model parameters: table 5.6 tallies the weights and biases layer by layer, totaling 109,386 parameters at 4 bytes each, occupying 437.5 KB.

Table 5.6: MNIST Parameters by Layer: Weight and bias counts for the 784-128-64-10 network. Layer 1 (Input → Hidden1) dominates the parameter budget because it multiplies the 784-dimensional input by 128 hidden units; deeper layers shrink rapidly as the representation compresses toward the 10-class output.

Layer	Weights	Biases	Total Parameters
Input → Hidden1	$784 \times 128 = 100,352$	128	100,480
Hidden1 → Hidden2	$128 \times 64 = 8,192$	64	8,256
Hidden2 → Output	$64 \times 10 = 640$	10	650
Total			109,386 parameters

Activations are the next contribution. Table 5.7 records each layer’s activation tensor and its memory cost at batch size 32.

Table 5.7: MNIST Activation Memory at Batch Size 32: Per-layer activation tensor shapes, value counts, and 32-bit float memory cost during a single forward pass. Input activations dominate the activation table at 100.4 KB; for this small MNIST multilayer perceptron (MLP), the total activation footprint remains below the parameter footprint, while larger batches and deeper networks make activations a central training-memory concern.

Layer	Activation Shape	Values	Memory
Input	32×784	25,088	100.4 KB
Hidden1	32×128	4,096	16.4 KB
Hidden2	32×64	2,048	8.2 KB
Output	32×10	320	1.3 KB
Total		31,552	126 KB

Training adds two memory contributions that inference never pays. Gradients occupy the same space as the parameters they update, here 437.5 KB, and the Adaptive Moment Estimation (Adam) optimizer stores momentum and velocity at twice the parameter size, another 875.1 KB. Table 5.8 summarizes the per-component memory footprint for training vs. inference.

Table 5.8: MNIST Training vs. Inference Memory at Batch Size 32: A simplified FP32 ledger for the toy MNIST MLP. Training retains saved activations and adds gradients plus two Adam buffers; inference uses the peak adjacent-layer buffer. Framework workspace and memory-saving techniques can change the totals.

Component	Training	Inference
Parameters	437.5 KB	437.5 KB
Activations	126 KB	117 KB
Gradients	437.5 KB	—
Optimizer state	875.1 KB	—
Total	~1.9 MB	~554 KB

Scale drives the systems lesson. At batch size 32, training requires 3.4× the inference memory in this simplified ledger. The absolute gap grows with parameter and activation storage; the ratio depends on their balance and on implementation choices such as checkpointing, precision, and workspace reuse.



Lighthouse 5.1: MNIST-to-GPT-2 memory scaling

The MNIST running example and the GPT-2 lighthouse show how model scale changes the data (D_{vol}) term of the iron law.

- MNIST (running example): 109,386 parameters at 4 bytes each occupy approximately 438 KB. This entire model fits inside the L2 cache of a modern processor.
- GPT-2 (lighthouse): 1,500,000,000 parameters at 4 bytes each occupy approximately 6 GB before activations and runtime workspace. This exceeds cache capacity and places the model in a device-memory or multi-device regime unless precision is reduced.

Systems insight: Moving from ~109.4K to 1.5B parameters is a 13,712.9× jump. The increase represents a phase change in engineering, not merely “more parameters.” MNIST can keep its

weights near the processor; GPT-2 serving must manage a much larger memory working set. Whether a particular layer is compute or memory bound still depends on batch size, precision, reuse, and hardware balance.

Parameter count grows with network width and depth. In the MNIST network, an input layer with 784 dimensions, hidden layers of 128 and 64 neurons, and a 10-neuron output layer ($784 \rightarrow 128 \rightarrow 64 \rightarrow 10$) scale the parameter footprint. The first layer requires 100,352 weights and 128 biases, the second layer 8,192 weights and 64 biases, and the output layer 640 weights and 10 biases, totaling 109,386 parameters. Each must be stored in memory and updated during learning.

The memory requirements summarized in table 5.8 seem modest for a small MNIST classifier. Scaling to production-sized models transforms these requirements dramatically, changing the hardware regime.

Napkin Math 5.1: Quick estimation for ML engineers

Detailed calculations are essential for design documents, but experienced engineers also develop rapid mental estimation skills. These “napkin math” shortcuts enable quick feasibility checks before committing to detailed analysis.

Memory estimation:

- Parameters $\rightarrow$ bytes: Multiply by four for 32-bit single precision (FP32), two for 16-bit half precision (FP16) or brain floating point 16 (BF16, the latter with an FP32-like exponent range), or one for 8-bit integer (INT8)
- Fully connected (FC) layer parameters: $\text{Input} \times \text{Output}$ (plus Output biases, usually negligible)
- Training state: Parameters, gradients, and optimizer state can require $\sim 3\text{--}4\times$ the weight-only parameter memory; peak activations and workspace are additional
- Adam optimizer overhead: $2\times$ parameter memory (momentum + velocity)
- Max batch size: Approximately $(\text{GPU VRAM} - \text{persistent state} - \text{workspace}) / \text{peak activations per sample}$

Compute estimation:

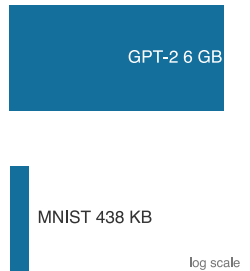
- FC layer FLOPs: $2 \times d_{\text{in}} \times d_{\text{out}} \times B$ (multiply-add = 2 ops)
- MACs to FLOPs: Multiply by 2
- Compute utilization proxy: $\text{achieved FLOP/s} / R_{\text{peak}}$

Example: “Can I train a 100M parameter model on a 16 GB GPU?”

Mental math: 100M parameters at 4 bytes each, with 4 training-memory copies, require 1.6 GB for persistent model state. That leaves about 14.4 GB for activations, temporary workspace, and framework overhead. Answer: the parameters fit, but the training configuration is not yet proven; batch size, activation shape, and kernel workspace determine whether the complete step fits.

Systems insight: A rough persistent-state estimate can reject impossible configurations quickly, but batch size still depends on activation and workspace headroom.

Beyond this model-fit example, table 5.9 distills three common feasibility questions into one-line formulas an engineer can apply before reaching for a profiler.



MNIST is cache-scale; GPT-2 is VRAM-scale.

Table 5.9: Napkin-math feasibility checks: Three rapid mental-estimation rules for GPU fit, epoch time, and bound regime, distilled into one-line formulas an engineer can apply before running a profiler.

Question	Quick Estimate
“Will this model fit in GPU memory?”	persistent training state + peak activations + workspace < VRAM
“How long per epoch on MNIST?”	60K × forward-backward FLOPs/image/achieved FLOP/s
“Is this compute bound or memory bound?”	Compare arithmetic intensity with $R_{\text{peak}}/\text{BW}$

The component-level memory ledger in table 5.8 is explicit but intentionally simplified. In practice, systems engineers work at two levels of fidelity: detailed budgets for design documents and order-of-magnitude estimates for early feasibility gates. The ledger confirms that MNIST fits comfortably in cache while GPT-2 requires a much larger memory tier, before accounting for runtime workspace. A quick mental estimate should reach the same conclusion in seconds, not minutes, and flag any model that cannot physically fit on the target hardware before a single line of profiling code runs.

Feasibility math tells the engineer whether a model *fits*; it says nothing about whether it will *learn*. That depends on how the parameters those bytes hold are first set. Parameter initialization is critical to network behavior. Setting all parameters to zero would cause neurons in a layer to behave identically, preventing diverse feature learning. Instead, weights are typically initialized randomly, often using specific strategies like Xavier/Glorot initialization³² (Glorot and Bengio 2010) or He initialization (He et al. 2015), while biases often start at small constant values or zeros. The scale of these initial values matters: values that are too large or too small lead to poor learning dynamics.

The distribution of parameters affects information flow through layers. In digit recognition, if weights are too small, important input details might not propagate to later layers. If too large, the network might amplify noise. Biases help adjust the activation threshold of each neuron, enabling the network to learn optimal decision boundaries.

Different architectures impose specific constraints on parameter organization. Some share weights across network regions to encode position-invariant pattern recognition; others restrict certain weights to zero, implementing sparse connectivity patterns.

Network architecture, neurons, and parameters are now in place, but a central question remains: the mechanism by which these randomly initialized parameters become useful. A randomly wired network produces outputs no better than chance. Architecture answers *what* the model computes; training answers *how* it learns. The mechanics reveal systems constraints because training adds backward computation, saved activations, gradients, and optimizer state. Batch size then becomes a joint statistical and hardware choice: it changes gradient noise, activation memory, data reuse, and parallel efficiency. The learning process coordinates these costs while transforming 109,386 parameters from random numbers into a functioning digit classifier.

5.4 Learning Process

The MNIST network currently holds 109,386 parameters initialized randomly—numbers that encode no knowledge at all. The transformation of these random values into a digit classifier achieving over 95 percent accuracy relies on four operations repeated millions of times: forward propagation computes a prediction, a loss function measures the error, backpropagation computes each parameter’s loss gradient, and an optimizer uses those gradients to update the parameters.

5.4.1 Supervised learning from labeled examples

A randomly initialized network classifies digits no better than random guessing among ten classes (about 10 percent accuracy). Transforming it into a 95 percent-accurate classifier requires supervised learning: showing the network labeled examples and adjusting its weights based on the errors it makes. Consider the MNIST digit recognition task: the dataset contains 60,000 training images, each a 28×28 pixel grayscale image paired with its correct digit label. The network must learn the relationship between these images and their corresponding digits through an iterative process of prediction and weight adjustment. Ensuring the quality and integrity of training data is essential to model success (chapter 4).

³² | **Xavier/Glorot initialization:** For a layer with n_{in} inputs and n_{out} outputs, a common Glorot choice uses weight variance $2/(n_{\text{in}} + n_{\text{out}})$ to balance forward activations and backward gradients (Glorot and Bengio 2010). The fix costs zero additional FLOPs; it is purely a matter of setting the random distribution at startup.

The relationship between inputs and outputs drives the training methodology. Training operates as a loop where each iteration processes a subset of training examples called a batch.³³ For each batch, the network performs four operations: forward computation through the network layers generates predictions, a loss function evaluates prediction accuracy, weight adjustments are computed based on prediction errors, and network weights are updated to improve future predictions.

The iterative approach can be expressed mathematically. Given an input image x and its true label y , the network computes its prediction according to equation 5.10:

$$\hat{y} = f(x; \theta) \quad (5.10)$$

This equation encapsulates the entire forward pass: the network f takes an input x (say, a 28×28 digit image) and, using its current parameters θ (weights and biases across all layers), produces a prediction $\hat{y}$ (a vector of ten probabilities, one per digit). The semicolon notation $f(x; \theta)$ distinguishes the input x , which changes with every example, from θ , which remains fixed during inference but evolves during training. The network's error is measured by a loss function³⁴ $\mathcal{L}$ (equation 5.11):

$$\text{loss} = \mathcal{L}(\hat{y}, y) \quad (5.11)$$

The error measurement drives the adjustment of network parameters through backpropagation, examined in section 5.4.4. In practice, training operates on batches of examples rather than individual inputs. For the MNIST dataset, each training iteration might process 32, 64, or 128 images simultaneously for reasons developed in section 5.4.4.6. The training cycle continues until the network achieves sufficient accuracy or reaches a predetermined number of iterations. Throughout this process, the loss function serves as a guide, its minimization indicating improved performance. Establishing proper metrics and evaluation protocols is essential for assessing training effectiveness (chapter 12).

5.4.2 Forward pass computation

An MNIST image becomes ten class scores by moving through weighted layers and nonlinear activations. That computation is **forward propagation**: input data flows through the network's layers to generate predictions. During training, those predictions are then compared with the true value; the resulting loss enters the backward-and-update loop shown in figure 5.17. The forward pass itself underlies both inference and training.

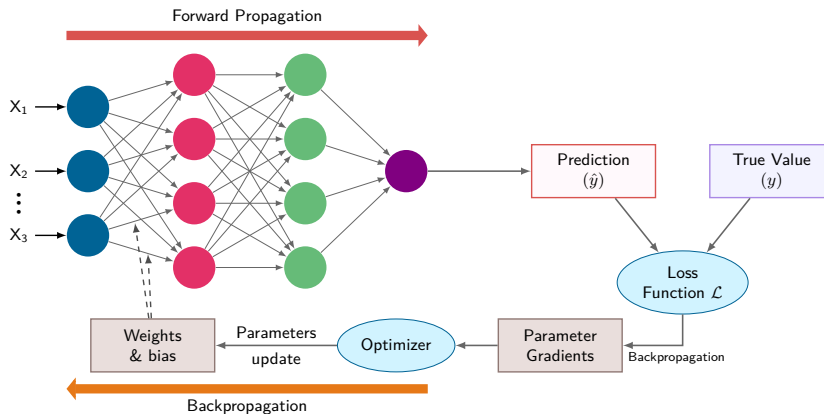


Figure 5.17: Training Loop Architecture: A prediction and true value meet at the loss function; backpropagation computes parameter gradients, and the optimizer turns those gradients into parameter updates for the next forward pass.

Data moves forward through the layers (the red arrow in figure 5.17), while gradients flow backward to update weights (the orange arrow). The figure reveals an important asymmetry: forward propagation produces model outputs, while backward propagation computes gradients for the parameters. *This dependence is why training retains the operation-specific forward values needed by the backward pass.*

³³ | **Batch processing:** Batching serves dual purposes. Larger batches average gradient noise across more examples and can improve hardware utilization by turning matrix-vector work into larger matrix-matrix kernels. They also perform more total work and usually increase latency for an individual request; the throughput gain depends on shape and hardware. Each doubling of batch size roughly doubles activation memory, making batch size a joint optimization and systems decision rather than a purely statistical or hardware choice.

³⁴ | **Loss function:** Formalized by Abraham Wald in statistical decision theory as the “cost” of an incorrect decision, $\mathcal{L}$ quantifies the gap between prediction $\hat{y}$ and ground truth y . The choice of loss function shapes the optimization geometry: it determines the gradient landscape that backpropagation must navigate. A loss with flat regions near incorrect predictions produces weak gradients that stall learning, while a loss with steep gradients near the decision boundary accelerates convergence where it matters most—a systems consequence explored in section 5.4.3.

When an image of a handwritten digit enters the network, it undergoes a series of transformations through the layers. Each transformation combines the weighted inputs with learned patterns to progressively extract relevant features. For the 784-128-64-10 digit classifier, a 28×28 pixel image is processed through multiple layers to ultimately produce probabilities for each possible digit (0–9).

The process begins with the input layer, where each pixel's grayscale value becomes an input feature. For MNIST, this means 784 input values ($28 \times 28 = 784$), each normalized between 0 and 1. These values then propagate forward through the hidden layers, where each neuron combines its inputs according to its learned weights and applies a nonlinear activation function.

Each forward pass through the MNIST network (784-128-64-10) requires substantial matrix operations. The first layer alone performs 100,352 MACs per sample. When processing multiple samples in a batch, these operations multiply accordingly, requiring careful management of memory bandwidth and computational resources. Specialized hardware like GPUs executes these operations efficiently through parallel processing.

5.4.2.1 Individual layer processing

The forward computation through a neural network proceeds systematically, with each layer transforming its inputs into increasingly abstract representations. The digit classifier illustrates this: its transformation process occurs in distinct stages. At each layer, the computation involves two key steps: a linear transformation of inputs followed by a nonlinear activation. The linear transformation applies the same weighted sum operation defined in equation 5.2, but now using notation that tracks layer indices (equation 5.12):

$$\mathbf{Z}^{(\ell)} = \mathbf{A}^{(\ell-1)} \mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)} \quad (5.12)$$

Here, $\mathbf{A}^{(\ell-1)}$ contains the activations from the previous layer (the outputs after applying activation functions), $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_{\ell-1} \times n_{\ell}}$ is the weight matrix for layer ℓ , and $\mathbf{b}^{(\ell)}$ is the bias vector (broadcast across the batch). The superscript (ℓ) keeps track of which layer each parameter belongs to. This row-vector convention matches the single-sample equation in equation 5.2: each row of $\mathbf{A}$ is one sample, and right-multiplying by $\mathbf{W}$ transforms it to the next layer's width.

Following this linear transformation, each layer applies a nonlinear activation function f (here, f or $f^{(\ell)}$ denotes a generic activation function at layer ℓ ; in section 5.3.2.1, σ referred specifically to the sigmoid function), as expressed in equation 5.13:

$$\mathbf{A}^{(\ell)} = f(\mathbf{Z}^{(\ell)}) \quad (5.13)$$

This process repeats at each layer, creating a chain of transformations:

Input $\rightarrow$ Linear Transform $\rightarrow$ Activation $\rightarrow$ Linear Transform $\rightarrow$ Activation $\rightarrow \dots \rightarrow$ Output

Returning to digit recognition, the pixel values first undergo a transformation by the first hidden layer's weights, converting the 784-dimensional input into an intermediate representation. Each subsequent layer further transforms this representation, ultimately producing a 10-dimensional vector of class scores, one for each possible digit.

5.4.2.2 Matrix multiplication formulation

The complete forward propagation process can be expressed as a composition of functions, each representing a layer's transformation. Formalizing this mathematically builds on the MNIST example.

For a network with N_L layers, the full forward computation is expressed as the composition in equation 5.14:

$$\mathbf{A}^{(N_L)} = f^{(N_L)} \left(\dots f^{(2)} \left(f^{(1)} (\mathbf{X} \mathbf{W}^{(1)} + \mathbf{b}^{(1)}) \mathbf{W}^{(2)} + \mathbf{b}^{(2)} \right) \dots \mathbf{W}^{(N_L)} + \mathbf{b}^{(N_L)} \right) \quad (5.14)$$

This composition reveals that forward propagation is, at its core, a chain of matrix multiplications interleaved with nonlinear activations. Understanding *why* matrix multiplication dominates AI computation requires examining the arithmetic intensity of each operation.

The nested expression unfolds layer by layer, each step consuming the previous layer's activations as its input:

MatMul

Matrix multiplication exceeds 90 percent of this dense network's forward-pass FLOPs.

1. First layer:

$$\mathbf{Z}^{(1)} = \mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)}$$

$$\mathbf{A}^{(1)} = f^{(1)}(\mathbf{Z}^{(1)})$$

2. Hidden layers ($\ell = 2, \dots, N_L - 1$):

$$\mathbf{Z}^{(\ell)} = \mathbf{A}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}$$

$$\mathbf{A}^{(\ell)} = f^{(\ell)}(\mathbf{Z}^{(\ell)})$$

3. Output layer:

$$\mathbf{Z}^{(N_L)} = \mathbf{A}^{(N_L-1)}\mathbf{W}^{(N_L)} + \mathbf{b}^{(N_L)}$$

$$\mathbf{A}^{(N_L)} = f^{(N_L)}(\mathbf{Z}^{(N_L)})$$

In the MNIST example, the operation dimensions for a batch of B images are as follows:

- Input $\mathbf{X}$: $B \times 784$
- First layer weights $\mathbf{W}^{(1)}$: $784 \times n_1$
- Hidden layer weights $\mathbf{W}^{(\ell)}$: $n_{\ell-1} \times n_\ell$
- Output layer weights $\mathbf{W}^{(N_L)}$: $n_{N_L-1} \times 10$

Arithmetic intensity determines whether a kernel is compute or memory bound. Tiled dense matrix multiplication reuses values, whereas an unfused element-wise operation does little work per byte. Table 5.10 compares their idealized intensities.

Table 5.10: Idealized Arithmetic Intensity of Matrix Multiply vs. Element-Wise ReLU: A tiled dense $N \times N$ matrix multiplication performs $2N^3$ operations while moving at least about $3N^2s$ bytes for s bytes per value, so potential reuse grows with N . A standalone element-wise activation remains at $1/(2s)$ FLOP/byte (0.125 FLOP/byte for FP32); implementations often fuse it with a neighboring kernel to avoid a separate memory round trip.

Operation	Operation Count (O)	Minimum Data Movement	Idealized Intensity	Typical Optimization
Matrix Mul ($N \times N$)	$2N^3$	$3N^2s$ bytes	$\approx 2N/(3s)$	Tiling on CPU/GPU/TPU
Element-wise (ReLU)	N^2	$2N^2s$ bytes	$1/(2s)$	Vectorization and fusion

Systems Perspective 5.4: Why matrix multiplication dominates AI

Modern accelerators have large compute arrays relative to off-chip memory bandwidth. They approach peak arithmetic throughput only when operations expose enough reuse and parallel work. Dense and convolutional layers map well to optimized kernels; custom element-wise sequences are often fused so that their intermediate values remain on chip.

For one sample, the expression $\mathbf{xW}$ is a **General Matrix-Vector Multiply (GEMV)**; batching samples turns the same computation into $\mathbf{XW}$, a **GEMM**. Multidimensional operations like 2D/3D convolutions map onto this same GEMM core through lowering transformations (such as `im2col`), which expand overlapping sliding receptive fields into 2D matrix columns so optimized vendor Basic Linear Algebra Subprograms (BLAS) libraries can execute them at peak hardware efficiency. This lowering introduces an explicit systems trade-off: because sliding receptive fields overlap, `im2col` duplicates input elements in memory, often expanding temporary tensor storage by $K_h \times K_w$ (e.g., $9 \times$ for a 3×3 convolution). Systems compilers accept this memory overhead as a deliberate compromise to transform strided spatial memory accesses into contiguous matrix multiplications that saturate accelerator systolic arrays. Matrix operations account for over 90 percent of the floating-point operations in this chapter's dense-network example and dominate many important neural workloads, although the frac-

tion depends on architecture and execution regime. To improve performance, engineers use techniques like blocking and tiling to increase cache or on-chip-memory reuse. This hardware-software co-design principle, designing model architectures around operations that specialized hardware can execute efficiently, is central to modern deep learning. Section C.1.4 provides the detailed treatment of GEMM arithmetic intensity, sparse matrix formats, and the computational complexity of common layer types needed to optimize these operations in practice.

5.4.2.3 Step-by-step computation sequence

Understanding how these mathematical operations translate into actual computation requires examining the forward propagation process for a batch of MNIST images. This process illustrates how data transforms from raw pixel values to digit predictions.

Consider a batch of 32 images entering the network. Each image starts as a 28×28 grid of pixel values, which flattens into a 784-dimensional vector. For the entire batch, this yields an input matrix $\mathbf{X}$ of size 32×784 , where each row represents one image. The values are typically normalized to lie between 0 and 1.

The transformation at each layer proceeds as a shape-changing sequence; the shapes determine both kernel choice and activation storage. The network first takes the input matrix $\mathbf{X}$ (32×784) and transforms it using the first layer's weights. If the first hidden layer has 128 neurons, $\mathbf{W}^{(1)}$ is a 784×128 matrix, so the computation $\mathbf{XW}^{(1)}$ produces a 32×128 matrix. Each element in this matrix then has its corresponding bias added and passes through an activation function. For example, with a ReLU activation, any negative values become zero while positive values remain unchanged. This nonlinear transformation enables the network to learn complex patterns in the data. The final layer transforms its inputs into a 32×10 matrix, where each row contains 10 class scores. Let z_j denote the raw score (logit) for digit j and let z_k range over all 10 digits. Often, these scores are converted to probabilities using the softmax function in equation 5.6:

$$p(\text{digit } j) = \frac{e^{z_j}}{\sum_{k=1}^{10} e^{z_k}}$$

For each image in the batch, this produces a probability distribution over the possible digits. The digit with the highest probability represents the network's prediction.



Napkin Math 5.2: Counting operations in forward pass

Problem: What is the total arithmetic operation count (O) for one forward pass through the MNIST network ($784 \rightarrow 128 \rightarrow 64 \rightarrow 10$) with batch size 32?

Background: A matrix multiplication of dimensions $(M \times K) \times (K \times N)$ requires $2 \times M \times K \times N$ FLOPs (one multiply and one add per output element, summed over K terms). Bias addition adds $M \times N$ floating-point additions. ReLU activation adds $M \times N$ comparisons, so the table reports total operation-equivalent work rather than pure FLOPs for activation rows.

Solution: Table 5.11 breaks down the operation count layer by layer. The total comes to ~ 7 MOp, or $7 \text{ MOp} \div 32 = \sim 219$ KOp per image.

Systems insight:

1. Layer 1 dominates: The first layer accounts for 91.9 percent of all operations because it processes the largest input (784 dimensions). This is why dimensionality reduction in early layers is so impactful.
2. Compute vs. memory: At batch 1, 219 KOp must amortize a parameter read for only one image, giving an arithmetic intensity of ~ 0.5 FLOP/byte under this traffic model. At batch 32, one read of the parameters plus one pass through the batch activations moves at least ~ 564 KB and raises the intensity to ~ 12.4 FLOP/byte. Additional cache misses and intermediate traffic can lower the realized value. The comparison shows why batching improves reuse, but memory- or compute-bound behavior still depends on the machine balance introduced in section D.2.1.

3. Scaling intuition: Doubling the hidden layer widths ($784 \rightarrow 256 \rightarrow 128 \rightarrow 10$) increases the operation count by about $2.1\times$ to about 15 MOp. This comes from recomputing each layer: layers 1 and 3 double, layer 2 quadruples, so the total grows by about $2.15\times$ rather than four times.

Table 5.11: MNIST Forward Pass Operation Count by Layer: Operation-equivalent breakdown for a single forward pass through the 784-128-64-10 network at batch size 32. Matrix multiplication rows are FLOPs; bias, ReLU, and softmax rows are elementwise operations included to show their negligible contribution. Layer 1's matrix multiply dominates because it multiplies the 784-dimensional input against the widest weight matrix; bias and activation operations contribute under 1 percent of the total.

Layer	Operation	Dimensions	Operations
Layer 1	MatMul	$(32 \times 784) \times (784 \times 128)$	$2 \times 32 \times 784 \times 128 = 6,422,528$
Layer 1	Bias + ReLU	32×128	$2 \times 4,096 = 8,192$
Layer 2	MatMul	$(32 \times 128) \times (128 \times 64)$	$2 \times 32 \times 128 \times 64 = 524,288$
Layer 2	Bias + ReLU	32×64	$2 \times 2,048 = 4,096$
Layer 3	MatMul	$(32 \times 64) \times (64 \times 10)$	$2 \times 32 \times 64 \times 10 = 40,960$
Layer 3	Bias + Softmax	32×10	~ 640 (simplified)
Total			~ 7 MOp

5.4.2.4 Implementation and optimization considerations

Forward propagation is easy to state mathematically, but its implementation is constrained by activation storage, batch size, memory layout, and hardware fit. Memory management plays a central role during forward propagation because each layer's activations must be stored for the backward pass during training. For the MNIST example (784-128-64-10) with a batch size of 32, the activation storage requirements are:

- Input layer: $32 \times 784 = 25,088$ values
- First hidden layer: $32 \times 128 = 4,096$ values
- Second hidden layer: $32 \times 64 = 2,048$ values
- Output layer: $32 \times 10 = 320$ values

This produces a total of 31,552 values that must be maintained in memory for each batch during training, consistent with the worked example in section 5.3.4.3. The memory requirements scale linearly with batch size and become substantial for larger networks.

Batch processing introduces important trade-offs. Larger batches enable more efficient matrix operations and better hardware utilization but require more memory. For example, doubling the batch size to 64 would double the memory requirements for activations. This relationship between batch size, memory usage, and computational efficiency guides the choice of batch size in practice.

The organization of computations also affects performance. Matrix operations can be optimized through careful memory layout and specialized libraries. The choice of activation functions affects both the network's learning capabilities and computational efficiency, as some functions (like ReLU) require less computation than others (like tanh or sigmoid).

The computational characteristics of neural networks favor parallel processing architectures. While traditional CPUs can execute these operations, GPUs designed for parallel computation can be substantially faster for large dense matrix operations. Specialized AI accelerators achieve even better efficiency through reduced precision arithmetic, specialized memory architectures, and dataflow optimizations tailored for neural network computation patterns.

Energy per inference can vary substantially across hardware platforms. CPUs offer flexible execution, GPUs provide high throughput through parallelism, and specialized edge accelerators trade generality for efficient execution of selected operators. Power alone does not determine energy: the relevant quantity is power integrated over execution time for the same workload and quality target. Memory hierarchy and data movement often explain much of the difference. These

considerations recur throughout subsequent chapters, particularly in chapter 6 where architecture-specific optimizations introduce additional trade-offs.

Forward propagation transforms inputs into predictions, but a prediction alone is useless for learning. The training loop requires a way to measure *how wrong* that prediction is in a form that guides weight adjustments. Loss functions fill this role: they translate the gap between prediction and reality into a single number that optimization can minimize.

5.4.3 Loss functions

The forward propagation process described in section 5.4.2 suffices for **inference**, using a pretrained model to make predictions. To train a model, however, training requires a quantitative error metric that measures how well predictions match ground truth. Loss functions quantify these errors, serving as the feedback mechanism that guides learning. They convert the abstract goal of “making good predictions” into a concrete optimization problem.

Continuing with the MNIST digit recognition example: when the network processes a handwritten digit image, it outputs ten class scores, often converted to predicted probabilities for digits 0–9. The loss function measures those predictions against the target. If an image displays a “seven,” cross-entropy penalizes the model when it assigns too little probability to digit seven. A higher loss indicates a worse value of the chosen objective on that example; it does not by itself diagnose the cause or establish that every task metric is worse.

5.4.3.1 Error measurement fundamentals

A loss function assigns a scalar cost to a prediction and target under a chosen objective. Lower loss indicates a better value of that objective on the evaluated examples, but it is not the same as higher accuracy: confidence and calibration can change while the predicted class remains unchanged. During training, gradients of the loss guide parameter updates. In handwritten-digit recognition, cross-entropy penalizes predictions that assign low probability to the correct digit.

Mathematically, a loss function $\mathcal{L}$ takes two inputs: the network’s predictions $\hat{y}$ and the true values y . For a single training example in digit classification, the loss measures the discrepancy between prediction and truth. When training with batches of data, the system computes the mean loss across all examples in the batch (equation 5.15):

$$\mathcal{L}_{\text{batch}} = \frac{1}{B} \sum_{i=1}^B \mathcal{L}(\hat{y}_i, y_i) \quad (5.15)$$

where B is the batch size and $(\hat{y}_i, y_i)$ represents the prediction and truth for the i -th example. Averaging keeps the loss and mean-gradient scale comparable across batch sizes, although the best learning rate can still change with batch size and optimization regime. The summation also maps naturally to parallel hardware: each example’s loss can be computed independently before a reduction combines them.

The choice of loss function depends on the type of task. For digit classification, the loss function must handle probability distributions over multiple classes, provide meaningful gradients that guide learning, penalize wrong predictions in proportion to their severity, and scale efficiently with batch processing. Cross-entropy loss satisfies all four requirements.

5.4.3.2 Cross-entropy and classification loss functions

For classification tasks like MNIST digit recognition, **cross-entropy loss** is a common way to compare predicted probability distributions with true class labels. The information-theoretic idea of entropy traces to Shannon (Shannon 1948); in supervised classification, cross-entropy penalizes low probability assigned to the correct class.

The logarithm gives **cross-entropy loss** its optimization behavior because probabilities near zero for the correct class incur increasingly large penalties. This shape produces strong gradients for confidently wrong predictions and makes the correct-class probability the objective’s central quantity.

For a single digit image, the network outputs a probability distribution over the 10 possible digits. Ground truth is represented as a one-hot vector³⁵ where all entries are 0 except for a one at the correct digit’s position. For instance, if the true digit is “seven,” the label is $y = [0, 0, 0, 0, 0, 0, 1, 0, 0]$.

³⁵ | **One-hot encoding:** Representing K classes as K -dimensional binary vectors where exactly one element is 1; this encoding is sparse by construction: for MNIST’s 10 classes, 90 percent of each label vector is zeros. Implementations commonly store an integer class index instead of materializing that vector. For very large output spaces, methods such as sampled softmax can also reduce the output computation.

The cross-entropy loss for this example is defined in equation 5.16:

$$\mathcal{L}(\hat{y}, y) = - \sum_{j=1}^{10} y_j \log(\hat{y}_j) \quad (5.16)$$

where $\hat{y}_j$ represents the network's predicted probability for digit j . Because y is one-hot, this simplifies to equation 5.17:

$$\mathcal{L}(\hat{y}, y) = -\log(\hat{y}_c) \quad (5.17)$$

where c is the index of the correct class. The loss therefore depends only on the predicted probability for the correct digit; the network is penalized based on how confident it is in the right answer.

For example, if the network predicts the following probabilities for an image of "seven":

Predicted: [0.1, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.8, 0.0, 0.1]

True: [0, 0, 0, 0, 0, 0, 0, 1, 0, 0]

The loss would be $-\log(0.8)$, which is approximately 0.223. If the network were more confident and predicted 0.9 for the correct digit, the loss would decrease to approximately 0.105.

5.4.3.3 Batch loss calculation methods

The practical computation of loss involves considerations for both numerical stability and batch processing. When working with batches of data, the objective evaluates the average loss across all examples.

For a batch of B examples, the cross-entropy loss becomes equation 5.18:

$$\mathcal{L}_{\text{batch}} = -\frac{1}{B} \sum_{i=1}^B \sum_{j=1}^{10} y_{ij} \log(\hat{y}_{ij}) \quad (5.18)$$

Here, i indexes examples in the batch, j indexes the ten output classes, y_{ij} is the one-hot target indicator for class j on example i , and $\hat{y}_{ij}$ is the predicted probability for that class. Averaging over B keeps the loss scale comparable as batch size changes.

Computing this loss efficiently requires careful consideration of numerical precision. The dangerous case is not an ordinary small probability, but a probability that an unstable softmax rounds to zero. If the correct-class probability becomes 0, then $\log(0)$ becomes $-\infty$ and subsequent arithmetic can produce a not-a-number value (NaN).

Stable implementations avoid materializing a numerically fragile probability before taking its logarithm:

1. Shift logits before exponentiation, the core of the log-sum-exp trick (section B.2.4), as shown in equation 5.19:

$$\text{softmax}(z_i) = \frac{\exp(z_i - \max(z))}{\sum_j \exp(z_j - \max(z))} \quad (5.19)$$

Subtracting the same $\max(z)$ from every logit leaves the softmax probabilities unchanged because the common factor cancels between numerator and denominator. The numerical effect is decisive: the largest shifted logit becomes zero, so the largest exponential is $\exp(0) = 1$ rather than a potentially overflowing value.

2. Prefer fused cross-entropy from logits. For target class c and $m = \max(z)$, a stable implementation evaluates

$$\mathcal{L} = -(z_c - m) + \log \sum_j \exp(z_j - m) \quad (5.20)$$

Equation 5.20 is mathematically equivalent to $-\log p_c$ but never rounds a tiny softmax probability to zero first. Clamping a probability or adding ϵ can be a defensive fallback, but it changes the objective and should not replace the stable formulation.

5.4.3.4 Impact on learning dynamics

With a batch size of 32 and 10 output classes, each training step processes 32 sets of 10 scores, computes a loss for each example, and averages them into one batch loss. During training, that value serves two roles. Its gradient supplies the update direction, and its trend helps diagnose whether optimization is progressing, stalling, or diverging. Task accuracy, calibration, and other evaluation metrics must still be measured separately on held-out data.

For the MNIST classifier, training loss often begins near $\log(10) \approx 2.3$ when class probabilities are close to uniform, falls rapidly during early optimization, and then improves more slowly. A low training loss indicates confidence on the training examples, not proof of generalization. Validation loss and task metrics determine whether the learned behavior transfers to held-out data.

The loss function's gradients with respect to the network's output logits provide the initial error signal that drives backpropagation. When softmax and cross-entropy are combined, each logit gradient has a particularly simple form: the difference between the predicted and target probabilities, $p - y$. This mathematical property makes cross-entropy loss especially suitable for classification tasks, as it provides strong gradients even when predictions are far from the target. In systems runtimes, this mathematical cancellation is the primary motivation for fusing softmax and cross-entropy into a single operator. Backpropagating through a standalone softmax requires evaluating a full $K \times K$ Jacobian matrix ($\mathcal{O}(K^2)$ intermediate storage and compute); fusing the loss with the output activation collapses the entire backward step into an $\mathcal{O}(K)$ element-wise vector subtraction ($p - y$), eliminating a major memory and computational bottleneck at the output layer.

The choice of loss function also influences other training decisions. Gradient scale interacts with the learning rate, while loss averaging across batches affects gradient noise and thus the useful batch-size regime. Curvature influences optimizer behavior, and convergence decisions require validation metrics and a stopping rule rather than the training-loss trajectory alone.

Loss functions quantify prediction error, but the error signal alone does not tell the system how sensitive that loss is to each parameter. With 109,386 parameters in the MNIST network, estimating every derivative through independent perturbations would be computationally prohibitive. This is the credit assignment problem: connecting the final error to the parameters and intermediate values that can change it. The next section introduces backpropagation, which applies the chain rule to compute all of those local sensitivities efficiently.

5.4.4 Gradient computation and backpropagation



Definition 5.2: Backpropagation

Backpropagation is the gradient-computation algorithm training systems use to apply the chain rule to a computational graph, using recorded operations and saved or recomputed forward-pass values to compute the gradient of the loss with respect to every parameter in a single backward traversal.

1. **Significance:** For dense layers, the backward pass is often on the order of twice the forward-pass FLOPs and requires retaining or recomputing the intermediate values needed by the chain rule. For a model with N_L layers, batch size B , and layer widths $n_1, \dots, n_{N_L}$, a simplified activation ledger scales as $\mathcal{O}(B \sum_{\ell=1}^{N_L} n_\ell)$. Gradients and optimizer state add further memory, so a model that fits on one accelerator for inference may not fit there for training.
2. **Distinction:** Numerical differentiation requires a perturbed evaluation for each of P parameters. Reverse-mode automatic differentiation computes all P gradients in one backward traversal whose cost is proportional to the computational graph, avoiding P separate forward passes.
3. **Common pitfall:** A frequent misconception is that backpropagation is learning. It is a gradient computation algorithm; gradient descent performs the actual parameter update. Confusing the two obscures the systems-level separation: backpropagation determines memory requirements (activation storage), while the optimizer determines additional state requirements (momentum, variance buffers).

A car factory gives a concrete version of the same credit-assignment problem. Vehicles pass through four stations: frame installation (A), engine mounting (B), wheel attachment (C), and final assembly (D). When inspectors find a defective car, they must determine which station caused the problem.

The solution works backward. Starting from the defect, inspectors trace responsibility through each station: how much D's assembly contributed vs. what it received from C, and how much C's work contributed vs. what came from B. Each station receives adjustment feedback proportional to its contribution. If Station B's engine mounting was the primary cause, it receives the strongest signal to change.

Backpropagation follows an analogous backward trace, but it computes sensitivity rather than causal blame. Starting from the loss, it evaluates how a small change in each layer's inputs would change that loss and propagates those derivatives backward. The optimizer later converts the resulting parameter gradients into actual updates.

In neural networks, each layer acts like a station on the assembly line, and backpropagation determines the sensitivity of the final loss to each connection. Translating this intuition into mathematics requires the chain rule of calculus. In the factory analogy, "Station D's adjustment signal" corresponds to the gradient at the output layer, local sensitivity maps to a partial derivative, and "sending feedback backward" describes the chain-rule multiplication that propagates gradients through the network.

The sensitivity is local to the current parameter values, input batch, and loss. It answers a counterfactual question—how an infinitesimal change here would change this loss—not whether a weight is permanently important or historically caused the prediction. The computational graph makes that question operational. Each node records an operation, and its backward rule maps an incoming loss gradient to gradients for its inputs. The reverse traversal begins at the scalar loss, applies those local rules in reverse dependency order, and accumulates contributions when one value influences the loss through several downstream paths. This accumulation is why backpropagation is more than sending one error signal down a chain: the runtime must respect graph dependencies and combine every relevant path before a gradient is complete. The algorithm below turns that graph-level description into a layer-by-layer procedure.

5.4.4.1 Backpropagation algorithm steps

While forward propagation computes predictions, backward propagation computes the gradients that an optimizer uses to adjust the weights. Consider the running example where the network predicts a "three" for an image of "seven". Backward propagation systematically calculates how the loss would respond to a small change in each weight at the current parameter values.

The process begins at the network's output, where the predicted digit probabilities are compared with the true label. The resulting loss gradient then flows backward through the network, giving the local sensitivity of the loss to each layer's values and weights. The computation follows the chain rule of calculus, breaking down the relationship between weights and loss into manageable steps.

The mathematical foundations of backpropagation provide the theoretical basis for training neural networks, but practical implementation requires software support. Wengert presented automatic numerical derivative evaluation without requiring analytical derivative expressions (Wengert 1964). Modern frameworks expose this capability through automatic differentiation systems that handle gradient computation automatically. Section C.3.1 derives the chain rule formally and shows why **reverse-mode automatic differentiation** computes all parameter gradients in a single backward pass, and section C.3.2 walks through the backward-pass algorithm step by step and explains why it costs roughly twice the FLOPs of the forward pass. Section 7.4.2 later examines the systems engineering aspects of these frameworks. The core implementation contract appears in algorithm 5.1: the forward pass must save the values that the backward pass will need.

Algorithm 5.1 Backpropagation training step

Require: mini-batch inputs $\mathbf{X}$, labels $\mathbf{y}$; N_L layers with parameters $\{\mathbf{W}^{(\ell)}, \mathbf{b}^{(\ell)}\}_{\ell=1}^{N_L}$; loss $\mathcal{L}$

Ensure: parameter gradients $\{\nabla_{\mathbf{W}^{(\ell)}} \mathcal{L}, \nabla_{\mathbf{b}^{(\ell)}} \mathcal{L}\}$ for the optimizer

```

1:  $\mathbf{A}^{(0)} \leftarrow \mathbf{X}$ 
2: for  $\ell = 1$  to  $N_L$  do  $\triangleright$  forward pass
3:   compute  $\mathbf{Z}^{(\ell)}, \mathbf{A}^{(\ell)}$ ; save the values its derivative needs
4: end for
5: compute loss  $\mathcal{L}(\mathbf{A}^{(N_L)}, \mathbf{y})$ ; seed the output gradient  $\nabla_{\mathbf{A}^{(N_L)}} \mathcal{L}$ 
6: for  $\ell = N_L$  down to 1 do  $\triangleright$  same local rule repeats
7:   use the saved forward values to compute  $\nabla_{\mathbf{W}^{(\ell)}} \mathcal{L}, \nabla_{\mathbf{b}^{(\ell)}} \mathcal{L}$ 
8:   propagate  $\nabla_{\mathbf{A}^{(\ell-1)}} \mathcal{L}$  to the previous layer
9: end for
10: return the accumulated parameter gradients

```

Saving activations in the forward loop of algorithm 5.1 is where the systems cost enters: each activation stays live until the backward pass reaches its layer, so **activation memory** scales with batch size, layer count, and activation width. This is why training can exceed inference memory even when the parameter count is unchanged.

5.4.4.2 Error signal propagation

The flow of gradients through a neural network follows a path opposite to forward propagation. Starting from the loss at the output layer, gradients propagate backward, computing how the loss is locally sensitive to each layer and, ultimately, each weight.

Consider what happens when the digit classifier misclassifies a “seven” as a “three”. The loss function generates an initial error signal at the output layer, essentially indicating that the probability for “seven” should increase while the probability for “three” should decrease. This error signal then propagates backward through the network layers.

For a network with N_L layers, the gradient flow can be expressed mathematically. At each layer ℓ , the sensitivity of the final loss to that layer’s output is governed by the chain rule³⁶ and the schematic Jacobian notation in equation 5.21:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{A}^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{A}^{(\ell+1)}} \frac{\partial \mathbf{A}^{(\ell+1)}}{\partial \mathbf{A}^{(\ell)}} \quad (5.21)$$

This computation cascades backward through the network, with each layer’s gradients depending on those from the layer above it. The result is a local sensitivity, not causal blame: equivalent parameterizations can represent the same function with differently scaled gradients. Backpropagation supplies these sensitivities, while the optimizer may normalize, accumulate, or otherwise transform them before updating the weights.

This process faces challenges in deep networks. As gradients flow backward through many layers, they can vanish or explode. Repeated multiplication can make gradients exponentially small, particularly with saturated sigmoid or tanh activations. Values below the minimum positive *normal* number (1.18×10^{-38} in FP32 or 6.10×10^{-5} in FP16) enter the subnormal range when the format and hardware preserve subnormals; still smaller values underflow to zero, and some accelerator modes flush subnormals to zero for speed. At the other extreme, values beyond the representable maximum can overflow to Inf and propagate NaN, destabilizing training.

5.4.4.3 Derivative calculation process

Computing gradients involves calculating several partial derivatives at each layer: how changes in weights, biases, and activations affect the final loss. These computations follow directly from the chain rule of calculus but must be implemented efficiently for practical training.

At each layer ℓ , backpropagation computes three distinct gradient components, each serving a specific role in optimization:

³⁶ | **Chain rule:**
The calculus identity $\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial a_n} \cdot \frac{\partial a_n}{\partial a_{n-1}} \cdots \frac{\partial a_1}{\partial w}$ becomes a *product* of n terms for an n -layer network. If each partial derivative is slightly less than one, the product vanishes exponentially; if slightly greater, it explodes. This multiplicative structure is why depth is a systems constraint, not just a design choice: it dictates the numerical precision requirements and initialization strategies (Glorot and Bengio 2010; He et al. 2015) needed to keep training stable.

First, the incoming error signal $\frac{\partial \mathcal{L}}{\partial \mathbf{A}^{(\ell)}}$ is transformed into the preactivation gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(\ell)}}$ by multiplying element-wise ($\odot$) with the local activation function derivative (equation 5.22):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{A}^{(\ell)}} \odot f'(\mathbf{Z}^{(\ell)}) \quad (5.22)$$

For ReLU activations, $f'(\mathbf{Z}^{(\ell)})$ acts as a binary mask, passing gradients through active neurons ($z > 0$) and zeroing out inactive ones.

Weight gradients measure the local sensitivity of the final loss to each weight. The optimizer uses these gradients to determine parameter updates (equation 5.23):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(\ell)}} = \mathbf{A}^{(\ell-1)T} \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(\ell)}} \quad (5.23)$$

Bias gradients measure how changing each bias term affects the loss. Since biases shift the activation threshold of neurons, these gradients indicate whether neurons should become more or less easily activated, as expressed in equation 5.24:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(\ell)}} = \mathbf{1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(\ell)}} \quad (5.24)$$

Here, $\mathbf{1} \in \mathbb{R}^B$ is an all-ones vector, so left multiplication sums the per-example bias gradients across the batch.

Input gradients propagate the error signal backward to the previous layer. Rather than directly updating parameters, these gradients serve as the “adjustment signals” that allow earlier layers to learn from the final prediction error (equation 5.25):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{A}^{(\ell-1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(\ell)}} \mathbf{W}^{(\ell)T} \quad (5.25)$$

These equations reveal the physical origin of the classic systems rule that the backward pass requires roughly twice the arithmetic of the forward pass ($2\times$). While the forward pass executes a single matrix multiplication per dense layer ($\mathbf{A}^{(\ell-1)}\mathbf{W}^{(\ell)}$), the backward pass must evaluate two distinct matrix multiplications of identical computational complexity: one to compute weight gradients ($\mathbf{A}^{(\ell-1)T} \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(\ell)}}$) and another to propagate gradients to the preceding layer ($\frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(\ell)}} \mathbf{W}^{(\ell)T}$). A full training iteration thus requires $1 + 2 = 3\times$ the forward FLOPs.

These three gradient types interact with a practical systems constraint: a desired effective batch may be larger than what fits in accelerator memory at once. Engineers address this through **gradient accumulation**: the full batch is split into smaller *micro-batches* that each fit in memory, and their gradients are accumulated with the appropriate loss normalization before the optimizer step. From the optimizer’s perspective the effective batch size equals the sum across all micro-batches; from the hardware’s perspective each micro-batch is an independent forward-backward pass that stays within the memory budget. For per-example-independent computations, accumulating consistently normalized gradients represents the same batch-gradient sum or mean. Exact numerical equivalence is not universal: floating-point reduction order, batch-dependent operations such as batch normalization, stochastic layers, and batch-dependent losses can make micro-batch execution differ from a single full-batch pass.

Consider the final layer where the network outputs digit probabilities. If the network predicted $[0.1, 0.2, 0.5, \dots, 0.05]$ for an image of “seven”, the gradient flows backward through three steps:

1. Start with the error in these probabilities
2. Compute how weight adjustments would affect this error
3. Propagate these gradients backward to help adjust earlier layer weights

🔍 Systems Perspective 5.5: The memory cost of backprop

Activation retention is one component of a broader training-state budget. Under a simplified single-device accounting that excludes temporary workspaces and framework overhead, equation 5.26 captures the four principal byte counts:

$$\text{Training Memory} \approx \text{Model Weights} + \text{Gradients} + \text{Optimizer States} + \text{Activations} \quad (5.26)$$

The derivative equations are also a state-dependency ledger:

- The weight gradient in equation 5.23 needs the incoming activation $\mathbf{A}^{(\ell-1)}$.
- The activation derivative may need $\mathbf{Z}^{(\ell)}$ or a compact mask that records which nonlinear units were active.
- The input gradient in equation 5.25 needs the layer weights $\mathbf{W}^{(\ell)}$.
- The optimizer consumes the completed parameter gradients and may add momentum, variance, or other state after backpropagation has produced them.

A training system can retain these values for the fastest backward pass, recompute selected values to save memory, or offload and partition state across memory tiers or devices. None of these choices eliminates cost: recomputation spends additional FLOPs, while offload and partitioning introduce data movement and synchronization. For deep networks with large batches, wide activations, or high-resolution inputs, this trade-off can dominate accelerator capacity. Section C.3.3 provides the complete training-memory equation and a worked analysis of weights, gradients, optimizer state, and activation costs.

A minimal network makes the gradient arithmetic concrete by tracing actual values through backpropagation.

📖 Napkin Math 5.3: Tracing gradients: A worked backpropagation example

Setup: Consider a network with two inputs, a hidden layer of two neurons (ReLU activation), and one output neuron (no activation, for simplicity). Suppose the current weights and biases are:

- **Hidden layer:** $\mathbf{W}^{(1)} = \begin{bmatrix} 0.5 & -0.3 \\ 0.8 & 0.2 \end{bmatrix}$, $\mathbf{b}^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$
- **Output layer:** $\mathbf{W}^{(2)} = \begin{bmatrix} 0.6 \\ -0.4 \end{bmatrix}$, $\mathbf{b}^{(2)} = 0$

Given input $\mathbf{x} = [1.0, 0.5]$ and target $y = 1.0$, applying mean squared error: $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$.

Forward pass (to establish the values backpropagation needs):

- Hidden preactivation:

$$\mathbf{z}^{(1)} = \mathbf{x}\mathbf{W}^{(1)} + \mathbf{b}^{(1)} = [1.0 \cdot 0.5 + 0.5 \cdot 0.8, 1.0 \cdot (-0.3) + 0.5 \cdot 0.2] = [0.9, -0.2]$$

- Hidden activation (ReLU): $\mathbf{a}^{(1)} = [\max(0, 0.9), \max(0, -0.2)] = [0.9, 0.0]$
- Output: $\hat{y} = \mathbf{a}^{(1)}\mathbf{W}^{(2)} + b^{(2)} = 0.9 \cdot 0.6 + 0.0 \cdot (-0.4) = 0.54$
- Loss: $\mathcal{L} = \frac{1}{2}(0.54 - 1.0)^2 = 0.1058$

Backward pass (applying the chain rule layer by layer):

Step 1: Output layer gradient. The loss gradient with respect to the output is

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \hat{y} - y = 0.54 - 1.0 = -0.46.$$

Step 2: Output weight gradients (applying equation 5.23). Since the output layer has no activation function

$$\frac{\partial \mathcal{L}}{\partial z^{(2)}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} = -0.46, \quad \text{and} \quad \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(2)}} = \mathbf{a}^{(1)T} \frac{\partial \mathcal{L}}{\partial z^{(2)}} = \begin{bmatrix} 0.9 \\ 0.0 \end{bmatrix} \cdot (-0.46) = \begin{bmatrix} -0.414 \\ 0.0 \end{bmatrix}$$

Step 3: Propagate to hidden layer (applying equation 5.25). The error signal sent backward is:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(1)}} = \frac{\partial \mathcal{L}}{\partial z^{(2)}} \mathbf{W}^{(2)T} = (-0.46) \cdot [0.6, -0.4] = [-0.276, 0.184]$$

Step 4: Pass through ReLU. The ReLU derivative is one where $z > 0$ and 0 otherwise, so

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}} = [-0.276 \cdot 1, 0.184 \cdot 0] = [-0.276, 0.0].$$

The second neuron's gradient is zeroed because ReLU blocked its forward signal.

Step 5: Hidden weight gradients (applying equation 5.23):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \mathbf{x}^T \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}} = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} \cdot [-0.276, 0.0] = \begin{bmatrix} -0.276 & 0.0 \\ -0.138 & 0.0 \end{bmatrix}$$

Weight updates (with learning rate $\eta = 0.1$): Each weight moves opposite to its gradient. For example, $W_{11}^{(1)}$ updates from 0.5 to $0.5 - 0.1 \cdot (-0.276) = 0.5276$, nudging the network toward the correct output. The second hidden neuron's weights receive zero updates for this example because ReLU blocked its activation, illustrating the mechanism that can lead to dead neurons if it happens persistently across the training data.

Systems insight: Backpropagation is not only a calculus procedure; it is also a data-dependency graph. Training systems must preserve the forward activations needed by this backward pass, which is why activation memory becomes a first-order systems cost.

While understanding these mathematical details is essential for debugging and optimization, modern practitioners rarely implement gradients manually. The systems breakthrough lies in how frameworks automatically implement these calculations. Consider a simple operation like matrix multiplication followed by ReLU activation: `output = relu(input @ weight)`. The mathematical gradient involves computing the derivative of ReLU (0 for negative inputs, 1 for positive) and applying the chain rule for matrix multiplication. The framework records the operation in a computation graph during the forward pass, stores the pre-ReLU activations needed for gradient computation, attaches the backward rule for each operation, and schedules the reverse traversal to balance correctness, memory usage, and hardware utilization. This automation transforms gradient computation from a manual, error-prone process requiring deep mathematical expertise into a reliable system capability that enables rapid experimentation and deployment.

5.4.4.4 Computational implementation details

Activation storage is only the first backward-pass cost. As model size scales, training also adds gradient storage, optimizer-state traffic, and scheduling pressure that the forward-pass memory estimate does not capture.

Consider a larger variant of the MNIST network ($784 \rightarrow 512 \rightarrow 256 \rightarrow 10$) with a batch size of 32. Each layer's activations must be maintained until the backward pass reaches that layer:

- Input layer: 32×784 values (~100 KB using 32-bit numbers)
- Hidden layer 1: 32×512 values (~66 KB)
- Hidden layer 2: 32×256 values (~33 KB)
- Output layer: 32×10 values (~1 KB)

37 | Adam (adaptive moment estimation): Maintains per-parameter first and second moment estimates, requiring $2\times$ additional FP32 memory beyond the parameters themselves (Kingma and Ba 2015). For a 100K-parameter MNIST model this overhead is negligible, but for a 7-billion parameter model it adds ~ 56 GB, often the difference between fitting on one GPU or requiring partitioning. Adam is widely used because its adaptive updates work robustly across many tasks, but learning rate, schedule, regularization, and memory strategy still require validation.

38 | Autograd (automatic differentiation): Reverse-mode automatic differentiation traces back to Linnainmaa's work on differentiating computer programs (1970). Modern autograd engines record forward-pass operations into a directed acyclic graph (DAG), then traverse it backward using the chain rule to compute gradients automatically. The key systems trade-off is that more flexible execution captures exactly what happened in each iteration, while more fixed execution exposes more opportunities for ahead-of-time optimization. Chapter 7 develops this framework design choice in detail.

Beyond activations, training allocates memory for parameter gradients. For this larger network with approximately 535,818 parameters, gradient storage requires a few megabytes. Advanced optimizers like Adam³⁷ roughly triple this by maintaining momentum and velocity terms for every parameter.

Memory bandwidth and footprint compound these capacity requirements. A training step reads parameters, produces gradients, and retains or recomputes the forward-pass values required by each layer's backward calculation. Saved-activation storage generally grows with batch size and network depth, making peak training memory larger than the inference footprint for the same model and batch. For modest networks like the MNIST example, this traffic remains manageable. As models grow, memory bandwidth and activation storage can become binding constraints, motivating high-bandwidth memory, mixed precision, partitioning, and activation checkpointing.

The computational pattern of backward propagation follows a strict sequence: compute gradients at the current layer, update stored gradients, propagate the error signal to the previous layer, and repeat until the input layer is reached. For batch processing, these computations are performed simultaneously across all examples in the batch, enabling efficient use of matrix operations and parallel processing capabilities.

Modern frameworks handle these computations through sophisticated autograd³⁸ engines. Dynamic computation graphs record operations as they execute, while static computation graphs defer execution to expose more optimization opportunities. When a training script asks for gradients, the framework automatically manages memory allocation, operation scheduling, and gradient accumulation across the computation graph. The system tracks which tensors require gradients and schedules operations so that the backward pass follows the dependencies created during the forward pass. This automated management allows practitioners to focus on model design rather than the intricate details of gradient computation implementation.



Checkpoint 5.3: Backpropagation

The credit assignment problem asks how the loss depends on parameters throughout the network. Backpropagation computes those gradients via the chain rule; verify that the mechanism is clear:

Mechanism

- ☐ **Chain rule:** can you explain how equation 5.21 decomposes a complex error signal into local gradients for each layer?
- ☐ **Gradient decay:** why do gradients often vanish in deep networks?

Training vs. inference

- ☐ **Computational cost:** if the forward pass requires N operations, why does training require roughly $3N$ total operations?
- ☐ **Memory cost:** why must training retain or recompute the forward values needed by the backward pass, and how does this scale with batch size and network depth?
- ☐ **Layer cost:** for a dense layer mapping 4,096 inputs to 4,096 outputs at batch size 32, estimate its forward FLOPs and its FP16 activation bytes to one significant figure, then say which of the two doubling the batch grows.

Backpropagation computes the local gradient of the loss with respect to each weight; the optimizer converts that signal into the actual parameter update. The step size, direction refinement, and momentum across iterations are governed by the optimizer. No optimizer is universally best across all possible problems (Wolpert and Macready 1997), so neural-network training depends on choosing update rules and hyperparameters that match the model, data, and hardware constraints.

5.4.4.5 Parameter update algorithms



Definition 5.3: Gradient descent

Gradient descent is the iterative optimization algorithm that updates model parameters in the direction of the negative gradient, $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$, trading computational steps (O) for loss reduction.

1. **Significance:** Optimizer choice contributes to training memory. In a simplified FP32 ledger, stochastic gradient descent (SGD) needs each weight and gradient (8 bytes per parameter), while Adam adds two moment buffers, bringing those four tensors to 16 bytes per parameter. Mixed-precision training may retain selected FP32 master values while storing other tensors more narrowly; chapter 8 develops the complete accounting. Optimizer state, saved activations, gradients, temporary workspaces, and communication buffers together determine how many accelerators training requires.
2. **Distinction:** Backpropagation computes gradients (the “what to change” signal); an optimizer applies updates (the “change it” action). The backward pass determines which forward values must remain available, while the optimizer adds its own state and update computation.
3. **Common pitfall:** A frequent misconception is that gradient descent finds the global minimum. Neural network loss landscapes are nonconvex with many local minima, saddle points, and plateaus. In practice, SGD and its variants converge to regions of low loss that generalize well, but the path depends on the learning rate schedule, batch size, and initialization.

The optimization process now needs an update rule that turns the backpropagated gradient into parameter changes. This iterative process uses each weight’s local loss sensitivity to update parameters, seeking to reduce loss over successive steps. The fundamental update rule combines backpropagation’s gradient computation with parameter adjustment, as defined in equation 5.27:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_{\theta} \mathcal{L} \quad (5.27)$$

where θ represents the collection of network parameters (weights and biases), η is the learning rate, and $\nabla_{\theta} \mathcal{L}$ is the gradient computed through backpropagation.

For the digit classifier, this means adjusting weights to improve classification accuracy. If the network frequently confuses “seven”s with “one”s, gradient descent will modify weights to better distinguish between these digits. The learning rate η ³⁹ controls adjustment magnitude: too large values cause overshooting optimal parameters, while too small values result in slow convergence.

Despite neural network loss landscapes being highly nonconvex, gradient-based methods often find useful solutions when the model, data, initialization, and optimization regime are well matched. The theoretical reasons, involving concepts like the Lottery Ticket Hypothesis (Frankle and Carbin 2019), implicit bias (Neyshabur et al. 2017), and overparameterization benefits (Nakkiran et al. 2019), remain active research areas. For practical ML systems engineering, the key insight is empirical: learning rates, initialization, regularization, and the training configuration must be validated together.

5.4.4.6 Mini-batch gradient updates

Neural networks typically process multiple examples simultaneously during training, an approach known as mini-batch gradient descent. Rather than updating weights after each individual image, the average gradient over a batch of examples is computed before performing the update.

For a batch of size B , let $\mathcal{L}_i$ denote the loss for example i ; the mean loss gradient becomes equation 5.28:

$$\nabla_{\theta} \mathcal{L}_{\text{batch}} = \frac{1}{B} \sum_{i=1}^B \nabla_{\theta} \mathcal{L}_i \quad (5.28)$$

With a typical batch size of 32, the system performs one forward pass over 32 images, computes 32 per-example losses, averages their gradients, and applies a single weight update. That average

³⁹ | **Learning rate:** This scalar has an outsized impact on training infrastructure because scaling batch size can require retuning the optimization schedule. The linear scaling rule increased learning rate with batch size for a specific large-batch ImageNet training regime (Goyal et al. 2017), but the best policy depends on the model, optimizer, data, and scale. Applying a scaling rule outside its validated regime can cause divergence when moving from single-GPU to multi-GPU training.

smooths noisy per-example gradients, but it also means the accelerator must keep the batch's activations available until the backward pass consumes them. Batch size therefore influences available parallelism and utilization together with model shape, kernel implementation, and hardware.

5.4.4.7 Iterative learning process

The complete training process combines forward propagation, backward propagation, and weight updates into a systematic training loop. This loop repeats until the network achieves satisfactory performance or reaches a predetermined number of iterations.

A single pass through the entire training dataset is called an epoch.⁴⁰ For MNIST, with 60,000 training images and a batch size of 32, each epoch consists of 1,875 batch iterations. Algorithm 5.2 lays out the mini-batch SGD loop: each epoch shuffles the data, steps through it one mini-batch at a time with a forward and backward pass, and applies a single parameter update per batch.

Algorithm 5.2 Mini-batch stochastic gradient descent

Require: training set $\{(x_i, y_i)\}_{i=1}^D$; rate η ; batch size B ; epochs E ; initial θ_0

Ensure: trained parameters θ

```

1:  $\theta \leftarrow \theta_0$ 
2: for  $e = 1$  to  $E$  do
3:   shuffle the training set  $\triangleright$  reduce order-dependent patterns
4:   for each mini-batch  $\mathcal{M}$  of size  $B$  do
5:      $\hat{y}_i \leftarrow f_\theta(x_i)$  for all  $(x_i, y_i) \in \mathcal{M}$ 
6:      $\mathcal{L}_\mathcal{M} \leftarrow \frac{1}{B} \sum_{(x_i, y_i) \in \mathcal{M}} \mathcal{L}(\hat{y}_i, y_i)$ 
7:      $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_\mathcal{M}$   $\triangleright$  backpropagate, then update
8:   end for
9:   evaluate on validation; stop if the convergence criterion is met
10: end for
11: return  $\theta$ 

```

The mini-batch loop makes the batch size B a hardware unit of work: a larger B can improve matrix utilization and accelerator occupancy but raises activation memory, and because the update fires once per batch rather than once per example, B also sets the gradient-update cadence. The inner loop in algorithm 5.2 is the unit that the hardware repeatedly executes. Forward propagation creates activations for every example in the current mini-batch, backward propagation consumes the required saved values to compute gradients, and the optimizer mutates the parameters once per batch. Training loss tracks the optimization objective on recent batches, while validation accuracy measures task performance on a separate held-out validation set. The test set remains reserved for final evaluation after model and hyperparameter choices are fixed.

5.4.4.8 Convergence and stability considerations

A network that achieves 99.5 percent accuracy on training data but only 85 percent on relevant unseen data exhibits a large generalization gap, a strong sign of overfitting. The model may have learned useful patterns while also specializing too closely to the training sample.



Definition 5.4: Overfitting

Overfitting is a generalization failure in which performance on the training sample improves while performance on relevant unseen data does not.

1. **Significance:** The diagnostic signature is measurable: training loss continues to fall while validation loss rises or the relevant validation metric plateaus, creating a widening generalization gap. Parameter count alone does not determine this behavior; data coverage, architecture, optimization, regularization, and the relationship between train and deployment distributions all matter.

⁴⁰ | **Epoch:** One complete pass through a training dataset. The number of passes is a direct multiplier on total work: a 100-epoch MNIST run executes $100 \times$ more forward and backward passes than a single epoch. Large language-model training is often specified by a token budget rather than a simple epoch count because data sources may be mixed and sampled at different rates. The same accounting still applies: every additional training token adds another forward and backward computation.

2. **Distinction:** Unlike underfitting (where the model lacks the capacity to capture the target function and both training and validation error remain high), overfitting produces low training error but high validation error, indicating that the model has specialized to the training sample rather than learning the underlying distribution.
3. **Common pitfall:** More data helps only when it is representative of the behavior the system must generalize to. Additional examples, augmentation, regularization, and early stopping are possible interventions, but each must be evaluated on held-out data from the relevant operating regime.

Learning rate selection strongly influences training dynamics. A rate that is too large for the optimizer, batch size, normalization, and schedule can make loss oscillate or diverge; one that is too small can waste epochs making negligible progress. There is no universal “moderate” value, so engineers test rates within the intended training configuration and monitor both optimization stability and validation performance.

Convergence monitoring provides essential feedback during training and continues into production deployment, as covered in chapter 14. A flattening training-loss curve signals slowing optimization, but it does not identify the cause: the run may be approaching a useful solution, encountering an optimization limit, or underfitting. A validation plateau is similarly diagnostic rather than conclusive. The gap between training and validation performance helps distinguish overfitting from improvements that transfer to held-out examples. The interplay between batch size, available memory, and computational resources requires careful balancing to achieve efficient training within hardware constraints, the same memory-computation trade-offs established earlier in this section.



Checkpoint 5.4: Neural network learning process

The complete training cycle provides the mathematical machinery that enables neural networks to learn from data. Before moving to inference and deployment, verify core understanding of the training cycle:

Use the MNIST network ($784 \rightarrow 128 \rightarrow 64 \rightarrow 10$) and a batch size of 32 as the running check.

- ☐ **Forward pass:** can you trace the forward pass using $\mathbf{Z}^{(\ell)} = \mathbf{A}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}$ and $\mathbf{A}^{(\ell)} = f(\mathbf{Z}^{(\ell)})$?
- ☐ **Cross-entropy loss:** can you explain what cross-entropy loss measures and why batch loss is averaged across examples?
- ☐ **Backward pass:** can you trace how gradients flow backward through the network and why stored activations are needed?
- ☐ **Update rule:** can you write the gradient descent update rule and explain how batch size trades memory, throughput, and gradient stability?
- ☐ **Training memory:** can you explain why the full training loop requires more memory than inference?

If any concepts feel unclear, review the earlier sections on forward passes, loss functions, backpropagation, and learning before continuing. These mechanisms form the foundation for the training-vs.-inference distinction that follows.

Selecting a stopping point requires periodically evaluating the model on the validation set and saving its weights to durable storage, a practice called **checkpointing**. For a small model like the MNIST network, writing a checkpoint costs a negligible amount of time. For a model whose parameters occupy tens to hundreds of gigabytes, a synchronous checkpoint write can stall the training loop while those bytes are serialized to disk or object storage; asynchronous and distributed checkpointing reduce that pause but still consume memory, network, and storage bandwidth. Running validation itself requires a full forward pass over the held-out dataset, which at large scale consumes meaningful accelerator time that would otherwise go to training. Checkpointing and validation are therefore not

41 | Inference: From Latin *inferre* (“to bring in, to conclude”), borrowed from logic where it means deriving conclusions from premises. The ML usage marks a systems boundary: training executes forward and backward passes and maintains optimizer state, while ordinary inference executes the forward graph with frozen parameters. Removing gradients, optimizer state, and most saved activations can reduce memory and computation substantially, but the ratio depends on architecture, batch size, precision, runtime workspace, and serving strategy.

42 | Training GPU power budget: The “high-memory” requirement is driven by the need to hold parameters, gradients, optimizer state, and activations simultaneously. The corresponding power draw dictates the “substantial cooling infrastructure,” as a single high-end training GPU consumes 400 W–700 W. Even compared with a 4 W mobile inference chip, that is at least 100× the power budget.

43 | Edge inference accelerators: The Edge TPU (Google Coral) operates in the mobile/embedded tier at about 2 W, delivering 4 TOPS through an INT8 datapath. Edge servers sit in a different tier: Jetson AGX Orin reaches 275 TOPS at 15 W–60 W, about 68.8× more throughput but with wired-power assumptions.

44 | Quantization: Moving from FP32 to INT8 reduces parameter storage by 4× and can increase throughput on hardware with efficient INT8 datapaths, although the realized speedup depends on kernel support, memory traffic, and workload shape. Inference often permits lower precision than training because it does not need to represent small gradients or optimizer updates across repeated learning steps. The trade-off is not free: aggressive quantization can degrade accuracy, especially on rare or poorly represented inputs, so calibration and task-level evaluation must establish the safe precision for each deployment. Chapter 10 develops quantization techniques in detail.

free statistical observations: they impose recurring I/O and compute costs whose frequency must be balanced against recovery and model-selection needs. Saving too infrequently risks losing useful weights after a failure or later degradation; saving too frequently stresses storage bandwidth and adds overhead. This is why convergence monitoring, while conceptually straightforward, becomes a systems infrastructure problem at scale.

The complete training pipeline runs from forward propagation through loss computation to gradient-based weight updates. Training, however, is preparation, not the end goal—once parameters are optimized, deploying the model for real-time predictions changes the resource profile entirely.

5.5 Inference Pipeline

Training transforms randomly initialized weights into parameters that encode useful patterns, but deployment executes those parameters under a completely different resource profile. During inference,⁴¹ the same mathematical operators face a fundamentally different operating regime: strict latency deadlines replace bulk training throughput, gradients and optimizer state disappear from memory, and target execution environments range from milliwatt edge processors to multi-accelerator cloud servers. This transition reshapes how memory bandwidth, compute capacity, and pipeline stages govern system performance.

5.5.1 Production deployment and prediction pipeline

Suppose a model that achieved 99 percent accuracy on its test set begins producing nonsensical outputs three months after deployment, although no code has changed. The weights are frozen and the inference pipeline runs without error. One possible cause is that the operating world moved while the model stood still.

The transition from training to inference limits how a fixed deployed model can adapt. Trained models generalize to unseen inputs through learned statistical patterns, but their parameters do not change during ordinary inference. When operational data diverges from the training distribution, the model continues applying those learned patterns. Consider an autonomous vehicle perception system: if construction zones become more frequent or novel vehicle configurations appear, its responses still reflect the training evidence. Systems that do not update parameters online adapt through monitored data collection, validation, and retraining, a deliberate engineering process detailed in chapter 8.

5.5.1.1 Operational phase differences

Neural network operation divides into two distinct phases with markedly different computational requirements and state lifetimes. Inference executes only the forward pass, evaluating inputs against static parameters with batch sizes dictated by incoming traffic. Training couples that forward pass with backpropagation for gradient computation and parameter updates, selecting batch sizes to maximize accelerator occupancy and gradient stability. Crucially, training must retain intermediate activations, gradients, and optimizer state simultaneously, multiplying working memory demands.⁴² While the underlying network weights are identical, the execution graph changes: training-only regularizers such as dropout are stripped away, while adjacent matrix operations and activations can be fused or quantized into specialized execution kernels. Figure 5.18 illustrates this structural contrast between bidirectional training and forward-only inference.

These computational differences manifest directly in hardware selection and deployment topologies. Training clusters rely on high-bandwidth, high-memory accelerators backed by hundreds of watts per socket and dedicated data-center cooling infrastructure. Inference deployments on constrained hardware prioritize bounded tail latency and energy efficiency across heterogeneous platforms: mobile devices execute on low-power neural processing units (typically 2–4 W), edge servers employ dedicated inference accelerators,⁴³ and cloud instances leverage reduced numerical precision to saturate tensor cores.⁴⁴ At scale, production serving introduces operational requirements—including dynamic request batching, load balancing, and failure isolation—that do not arise in isolated training runs. At the memory level, training retains all forward activations until backward passes consume them, whereas inference discards or overwrites intermediate buffers as execution advances through the graph; table 5.12 summarizes this resource profile.

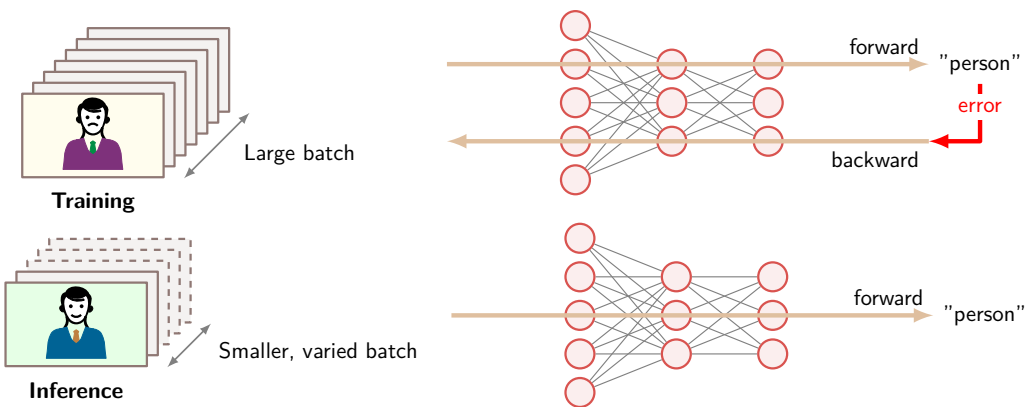


Figure 5.18: Inference vs. Training Flow: During inference, neural networks use learned weights for forward pass computation only, simplifying the data flow and reducing computational cost compared to training, which requires both forward and backward passes for weight updates. This streamlined process enables efficient deployment of trained models for real-time predictions.

Table 5.12: Training vs. Inference Forward Pass: Both phases execute the same forward equations, but they manage state differently. Training must retain or recompute the values required by the backward pass and also carries gradients and optimizer state. Inference can reuse a rolling pair of layer buffers. At a matched batch size of 32, the simplified MNIST ledger in table 5.8 is about 3.4×; the ratio is workload and implementation dependent.

Characteristic	Training Forward Pass	Inference Forward Pass
Activation Storage	Retains values required by the backward pass	Retains the current input and output buffers
Memory Pattern	Saves or recomputes selected intermediate states	Reuses buffers as successive layers complete
Computational Flow	Structured for gradient computation preparation	Optimized for direct output generation
Resource Profile	Higher memory requirements for training operations	Minimized memory footprint for efficient execution

5.5.1.2 Memory and computational resources

Neural networks consume computational resources differently during inference than during training. Inference has two memory obligations. The first is persistent: the trained weights and biases must remain available for every request. The second is transient: each layer produces an activation buffer that is needed only until the next layer consumes it. This distinction is the reason inference can be much leaner than training, even though the arithmetic in the forward pass is the same.

For the canonical MNIST network (784 → 128 → 64 → 10), the persistent parameter block contains 109,386 parameters, or about 438 KB at 32-bit floating point precision. The layer-level counts in table 5.13 show why: each fully connected layer performs one multiply-add for each weight, so parameter memory and arithmetic scale together.

Table 5.13: MNIST Inference Resources by Layer: The persistent parameter block and multiply-add count for the canonical MNIST network. In fully connected layers, the same weights that occupy memory also determine the arithmetic work per inference.

Layer	Weights	Biases	Multiply-Adds
Layer 1	784×128 = 100,352	128	100,352
Layer 2	128×64 = 8,192	64	8,192
Output	64×10 = 640	10	640
Total	109,386 parameters	Included in total	109,184

This persistent-plus-rolling-buffer model also explains the deployment optimizations that follow. Batching can increase arithmetic reuse and hardware occupancy but expands activation storage. Lower numerical precision can shrink the persistent parameter block and activation buffers,⁴⁵ but only if task quality survives the smaller representation. Hardware-specific layouts can improve cache reuse by keeping working data close to the compute units. Inference makes these optimizations easier to plan because parameters are fixed and many activations have short lifetimes.

5.5.1.3 Performance enhancement techniques

Freezing parameters and eliminating backward state exposes systematic optimization opportunities unavailable during training. Because the forward compute graph is static and deterministic, runtime

⁴⁵ | **FP32 and Reduced Precision Formats:** The IEEE 754 binary32 format uses 32 bits (1 sign bit, 8 exponent bits, 23 fraction bits), whereas 16-bit alternatives split exponent and fraction differently: IEEE 754 FP16 allocates 5 exponent

the input buffer), while a batch of 32 requires 6,464 layer-output activation values. Production systems serving interactive traffic must navigate this trade-off dynamically, accumulating inputs only up to the point allowed by their service-level latency budget.

Memory allocation during inference is substantially leaner than during training because intermediate activations have strictly disjoint lifetimes. In training, every activation tensor must persist until backpropagation reaches its corresponding layer, forcing cumulative memory consumption across the entire network depth ($\sum_l A_l$). During inference, once layer $l + 1$ finishes execution, the activation buffer for layer l is dead and its allocated storage can be reclaimed or overwritten. Serving runtimes exploit this lifetime property through static memory planning or double-buffering (ping-pong allocation), alternating between two physical memory blocks as data advances through the layers. Under this scheme, peak transient activation memory collapses from the cumulative total across all layers to the maximum sum of adjacent layers: $\max_l (A_l + A_{l+1})$.

Hardware-specific compilation further reduces latency by eliminating off-chip memory traffic. On CPUs, execution routines tile matrices to fit within L1 and L2 caches, avoiding high-latency main-memory stalls, while dispatching SIMD vector instructions (such as AVX-512 or ARM NEON) that compute multiple multiply-adds per instruction cycle. On accelerators, execution efficiency depends on operator fusion. A naive execution path launches separate kernels for matrix multiplication, bias addition, and nonlinear activation, writing intermediate tensors back to high-bandwidth memory (HBM) after each step and immediately reading them back. Kernel fusion combines these operations into a single kernel, staging intermediate values entirely within on-chip registers and shared SRAM. This replaces high-latency global memory round-trips with local register access, reducing kernel launch overhead and memory bus congestion.

Eliminating the backward pass also unlocks reduced numerical precision. Training requires broad dynamic range and high precision (such as FP32 or BF16) to accumulate minuscule gradient updates without underflow or catastrophic cancellation. During inference, frozen weights and activations can frequently be quantized to lower-bit representations, such as FP16 or INT8, with negligible impact on final predictions. Lower precision directly attacks the memory bandwidth bottleneck: converting FP32 parameters to INT8 quarters the volume of bytes transferred across the memory bus and quadruples the effective capacity of on-chip caches. For the MNIST network, halving the storage width cuts the parameter and activation byte footprint by half; realized latency speedups then depend on whether the target processor provides dedicated low-precision arithmetic pipelines (such as Tensor Cores or vector dot-product instructions).

While demonstrated here on a feedforward network, these systems mechanics—amortizing parameter transfers through batching, bounding live ranges via buffer reuse, eliminating memory round-trips through operator fusion, and relieving bus bandwidth via quantization—form the core design space for all production inference engines. Specialized network structures introduce additional computational patterns: convolutional operators share weights across spatial dimensions to heighten arithmetic intensity, recurrent architectures process sequential steps with iterative state dependencies, and transformer-based models require dynamic key-value cache management. These architectural structures and their compression methods appear in chapter 6 and chapter 10, while serving architectures, dynamic batching, and production operations are developed in section 13.7 and chapter 14.

5.5.2 Output interpretation and decision making

Neural network outputs become useful only after they are converted back into decisions a conventional system can act on. Preprocessing bridges real-world data into tensor form; postprocessing maps neural outputs into labels, decision thresholds, validation logic, error handling, and downstream messages. In the MNIST running example, raw logits are not enough: a digit-recognition system needs the most likely digit class, a confidence score whose calibration has been evaluated, and a fallback route that directs ambiguous or low-confidence predictions to human review or a secondary recognizer.

These boundary stages exhibit an entirely different computational profile from the forward pass. While neural network execution exploits massively parallel matrix units on accelerators, preprocessing (such as image decoding and normalization) and postprocessing (such as argmax reduction, threshold checking, and message serialization) typically execute as sequential scalar code

on the host CPU. Under Amdahl's law, as accelerator kernels drive forward-pass execution times toward zero, these unaccelerated CPU stages and their associated PCIe data transfers expand to dominate end-to-end request latency.

The complete neural network lifecycle, from architecture design through training to inference deployment, is a set of mathematical operations with quantifiable resource costs. These operations have so far lived in the controlled environment of the MNIST running example, where data is clean, latency is unconstrained, and hardware is unchallenged. The historical case study tests them against a production deployment, where none of those conditions holds.

Checkpoint 5.5: Complete neural network system

Before examining how these concepts integrate in a real-world deployment, verify your understanding of the complete neural network lifecycle:

Use the MNIST classifier ($784 \rightarrow 128 \rightarrow 64 \rightarrow 10$) as the running production system.

- ☐ **System lifecycle:** can you trace the lifecycle from architecture design to training loop to trained model to inference deployment?
- ☐ **Training vs. inference memory:** at a matched batch size, can you explain why this ledger gives training roughly $3.4\times$ the memory of inference, naming the gradients, optimizer state, and saved-activation terms?
- ☐ **End-to-end trace:** can you trace one digit image through preprocessing, forward propagation, output probabilities, postprocessing, and final prediction?
- ☐ **Bottleneck spotting:** can you identify where bottlenecks might appear in a real-time system processing 100 images per second?
- ☐ **Human-in-the-loop:** can you explain how validated decision thresholds and monitoring determine when human intervention is needed?

Handwritten digit recognition shows how architectural choices, preprocessing, decision thresholds, and human review combine in a working document-processing pipeline.

5.6 Handwritten Digit Recognition

In the late 1980s, LeCun and colleagues demonstrated backpropagation-based recognition of handwritten ZIP-code digits from USPS-supplied images (LeCun, Boser, et al. 1989). Related convolutional document-recognition work later supported commercially deployed bank-check readers (LeCun et al. 1998). These were distinct systems, but they exposed the same engineering pattern: preprocessing normalized variable handwriting, neural inference produced candidate digits, rejection thresholds sent uncertain predictions to human review, and downstream logic converted accepted predictions into operational decisions. That shared pipeline serves as a systems case study in end-to-end machine learning design, illustrating how preprocessing, model structure, and rejection policy interact under physical throughput constraints.

5.6.1 The document-recognition challenge

Operating constraints in document sorting stem directly from the physical handling machinery. A postal sorting machine moves envelopes past optical scanners along mechanical transport belts at fixed velocities, leaving only tens of milliseconds per item to acquire the image, isolate characters, evaluate the model, and actuate physical diverter gates. A misrouted envelope or misread check incurs severe operational costs in manual re-handling or downstream financial reconciliation. Because raw accuracy on uncontrolled handwriting cannot reach 100 percent, the pipeline must couple automated inference with an explicit rejection mechanism that routes ambiguous inputs to human operators. The handwritten address samples in figure 5.19 illustrate the sensory variability that preprocessing and classification stages must resolve under these timing constraints.

Absorbing this physical variability requires distributing the computational burden across multiple pipeline stages. Upstream image capture and normalization bound the input distribution presented

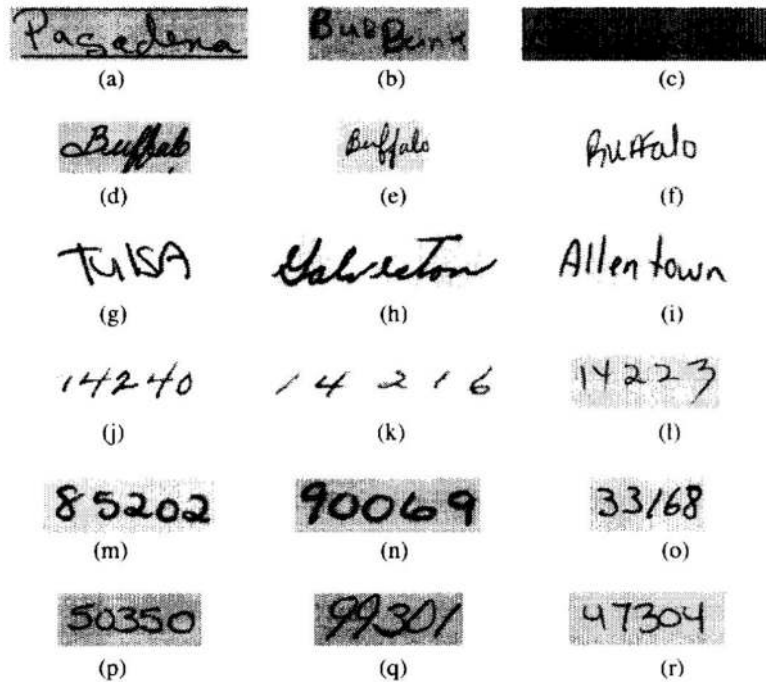


Figure 5.19: Handwritten Address Variability: Address and ZIP-code samples exhibit wide variation in stroke width, slant, spacing, and character formation. A document-recognition pipeline must absorb this variation without confusing legitimate writing differences with changes in the intended characters. Samples from the CEDAR CD-ROM 1 USPS Office of Advanced Technology database.

to the neural network, neural inference maps pixel patterns to calibrated class scores, and downstream threshold logic governs the trade-off between automated throughput and error.

5.6.2 Engineering process and design decisions

Recognizing a clean, centered digit is straightforward. Recognizing handwriting amid variable backgrounds, scale, alignment, and image quality requires engineering decisions at every stage from data preparation to deployment.

The dataset histories must remain separate. Early ZIP-code research used postal digit images supplied for that task (LeCun, Boser, et al. 1989). MNIST was later constructed from National Institute of Standards and Technology (NIST) handwritten-digit databases and standardized into size-normalized benchmark images (LeCun et al. 1998). It did not arise from collecting live envelopes under varying colors, textures, lighting, and orientations. Across both datasets, however, the engineering requirement was the same: preprocessing had to reduce irrelevant variation without erasing the stroke information needed for recognition.

Architecture design balances accuracy, processing time, and memory cost. For the chapter's standardized 28×28 digit input, the central question is how much model capacity the available latency and hardware budget can support. Fully connected layers scale with the product of input and output dimensions, demanding substantial parameter storage and memory bandwidth. Convolutional layers reuse weights across spatial receptive fields, reducing parameter volume and keeping intermediate activations compact.

Training must cover handwriting variation rather than optimize only for a curated test set. Preprocessing normalizes size and orientation, while augmentation can expose the model to additional plausible variation. Evaluation must then measure performance across relevant writing styles and operating conditions, following the systematic workflow described in chapter 3.

Confidence thresholds determine the division of labor between automation and human review. A high threshold rejects more items for manual handling; a low threshold accepts more model errors.

An expected-cost rule chooses threshold t to minimize $\text{manual_rate}(t)C_{\text{manual}} + \text{error_rate}(t)C_{\text{error}}$ subject to throughput and latency constraints.

5.6.3 Shared pipeline architecture

Postal ZIP-code research and deployed bank-check readers used different operational systems, but both fit the pattern in figure 5.20. Image acquisition and conventional preprocessing prepare the input, neural inference produces digit scores, and postprocessing either accepts the result or routes an uncertain item for review.

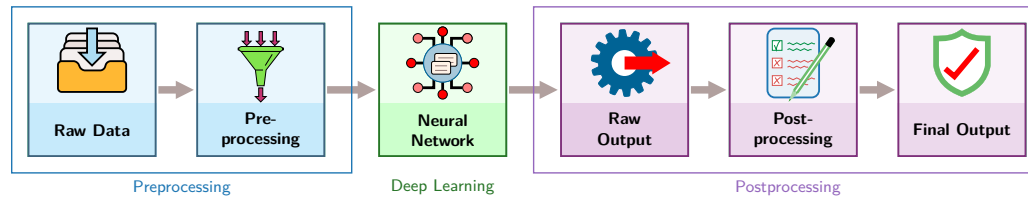


Figure 5.20: Document-Recognition Pipeline: A composite six-stage flow distilled from postal ZIP-code research and deployed check-reading systems. Image acquisition and conventional preprocessing prepare handwritten digits for neural inference, and raw scores then pass through postprocessing to the final application output.

The pipeline begins with image acquisition. In postal research, the input was an image of a handwritten ZIP-code region; deployed check readers similarly began with scanned document images. Acquisition quality determines how much variation the later preprocessing and recognition stages must absorb.

Preprocessing locates the relevant field, separates or otherwise identifies its characters, and normalizes the resulting images. Thresholding, connected-component analysis, and size normalization are representative conventional techniques. The chapter uses standardized 28×28 images for its MNIST calculations; that dimension is a teaching convention here, not a claim about every postal or check-reading deployment.

The neural network then processes each normalized digit image. The 1989 ZIP-code research system used an early LeNet variant (LeCun, Boser, et al. 1989) with approximately 10,000 parameters, occupying approximately 40 KB in single precision. This parameter footprint is an order of magnitude smaller than the 109.4K in this chapter's fully connected MNIST network. On late-1980s workstation hardware, maintaining a 40 KB parameter footprint enabled the model weights to reside in fast cache memory during inference, minimizing off-chip memory traffic and preserving the compute bandwidth needed to evaluate digits within the sorting deadline.

Postprocessing converts digit scores into application decisions. For a ZIP-code task, one uncertain digit can send the complete code for further processing or human review. Deployed document readers use the same general principle: accept sufficiently confident predictions and reject uncertain cases instead of forcing an answer.

Deployed document readers must satisfy end-to-end timing constraints spanning acquisition, preprocessing, inference, postprocessing, and downstream action. The exact machinery differs between mail and financial-document applications, but throughput depends on the complete pipeline rather than neural-network latency alone.

5.6.4 Performance outcomes and operational impact

The 1989 ZIP-code study reported its model, recognition, throughput, and training measurements as one experimental system. Table 5.14 keeps those measurements together, revealing how rejection policy and preprocessing affect the meaning of raw classifier speed (LeCun, Boser, et al. 1989).

Table 5.14: Early ZIP-Code Recognition Results: Measurements from the 1989 backpropagation system applied to USPS-supplied digit images. The rejection result shows why an operational error rate must be read together with the fraction of inputs withheld from automatic classification (LeCun, Boser, et al. 1989).

Metric	Reported Result
Error and rejection	1% classification error after rejecting 12.1% of test digits
Throughput	10–12 classifications/sec including acquisition; over 30/sec after normalization
Model parameters	~10,000
Training	23 passes; about 3 days on a Sun-4/260 workstation

These measurements highlight three systems trade-offs across the pipeline:

First, end-to-end throughput was bounded by non-neural stages. While the neural network executed over 30 classifications per second (under 33 ms per digit), the complete pipeline processed only 10 to 12 items per second (83 to 100 ms per item). Physical document handling, scanning, thresholding, and segmentation consumed more than half of the total latency budget. Accelerating neural inference alone yields diminishing returns when surrounding I/O and conventional preprocessing dominate the execution timeline.

Second, operational reliability depended on an explicit rejection pathway. Rather than forcing every digit through automated classification, the system withheld the 12.1 percent most ambiguous test inputs from automatic routing. Rejecting this tail of high-uncertainty predictions reduced the observed classification error on the remaining automated stream to 1 percent. The rejection threshold shifts the operating point along a Pareto frontier: automation absorbs the routine volume, while manual operators resolve the ambiguous tail to protect the system’s operational error budget.

Third, model size was bounded by the computational ceiling of the training host. Training 10,000 parameters required 23 passes across roughly three days on a Sun-4/260 workstation. Scaling to larger parameter regimes was infeasible under the processor clock and memory bandwidth constraints of the deployment era.

Maintaining these systems in production introduces lifecycle requirements beyond model training. Changes in scanner optics, ink formulations, envelope textures, or regional writing styles cause input distributions to drift over time. Production monitoring must track input image statistics and rejection rates alongside raw classification latency, triggering data collection and model retraining when the rejection rate exceeds the capacity of human review operators.

5.6.5 Key engineering lessons and design principles

Across postal ZIP-code research and deployed check readers, the central lesson is that neural computation succeeds only when the surrounding pipeline shares the same operating constraint. Preprocessing, inference, postprocessing, rejection, and downstream action must support the required throughput and error budget together.

Converting handwritten marks into reliable operational decisions required coordinated choices in network architecture, training, preprocessing, confidence policy, and downstream handling. These historical systems established principles that remain foundational in production deployments: bounding input distributions via specialized preprocessing, sizing architectures to meet memory bandwidth limits, calibrating confidence thresholds to support human-in-the-loop fallback, and engineering the complete data path for end-to-end latency budgets. The operational lifecycle required to sustain these pipelines is formalized in chapter 14.

Modern edge systems operate under different power, latency, and deployment constraints, but the underlying engineering trade-offs remain identical: preprocessing absorbs real-world environmental variability, confidence thresholds protect downstream error budgets, and end-to-end hardware throughput governs the viable model capacity.

Systems Perspective 5.6: Reading historical system comparisons

The early LeNet-class reader and this chapter’s MNIST multilayer perceptron show continuity in computation, not a controlled performance benchmark. Their datasets, architectures, software stacks, machines, and operating targets differ. The defensible comparison is structural: both

transform normalized pixels into digit scores, both depend on learned parameters and a preprocessing pipeline, and both need an explicit policy for uncertain outputs.

Parameter count supplies one concrete contrast. The 1989 ZIP-code network had approximately 10,000 parameters, while the chapter's fully connected network has 109,386. That difference describes model state; it does not by itself establish latency, energy, throughput, or cost. Those outcomes require a fixed workload and measurements that identify the hardware, numerical precision, runtime, batch size, and end-to-end decision path. Separating these variables prevents a faster deployment from being attributed entirely to silicon when the data, model, software, or operating policy also changed.

The document-recognition pipeline demonstrates the foundational alignment of ML systems engineering: high-throughput automated classification requires continuous coordination across the data distribution, algorithmic capacity, and underlying machine constraints.

5.7 D·A·M Taxonomy

Across the document-recognition histories, different resources served distinct purposes. USPS-supplied ZIP images grounded early recognition research, LeNet supplied an algorithmic mechanism, later bank-check readers supplied a documented deployment setting, and MNIST supplied a standardized benchmark derived from NIST data. They do not form a single system, but together they instantiate the tripartite alignment of the D·A·M taxonomy: whether the data captures the task distribution, whether the algorithm can learn the required transformation, and whether the machine can execute that computation within physical latency, memory, and energy budgets. Appendix A develops the intersections among these axes.

The data axis establishes whether the training and evaluation examples faithfully capture the operational distribution. USPS ZIP-code images, MNIST digits, and bank-check inputs share digit semantics but stem from distinct acquisition processes and noise environments: scanning resolution, thresholding artifacts, and document layout make them non-interchangeable. Preprocessing can normalize variations in scale and orientation, but it cannot synthesize unrepresented handwriting styles or correct a labeling policy that diverges from downstream operational decisions. Data also imposes a physical footprint: input resolution, batch size, tensor layout, and numeric precision dictate the volume of bytes that must cross memory hierarchies into accelerator memory.

Those incoming bytes define the input to the algorithm axis, which dictates the mathematical operations required to learn or evaluate representations. In the chapter's running network, the 784-to-128 fully connected layer defines both the initial representational capacity and the largest parameter block in the model. Activation functions introduce nonlinearities that govern which decision boundaries can be formed and whether backpropagated gradients remain numerically stable; the loss function and optimizer determine how prediction errors drive parameter updates. These choices establish the mathematical work the system must execute, but an algorithmic formulation remains viable only if its statistical assumptions match the data distribution and its computational graph fits within the hardware budget.

That hardware budget is dictated by the machine axis, which determines whether the required linear algebra can execute within physical latency, memory bandwidth, memory capacity, and thermal limits. Forward propagation streams parameter weights and input activations into arithmetic units; backpropagation retains those forward activations in memory, computes gradient tensors, updates optimizer states, and traverses the reverse computational graph. Increasing batch size amortizes parameter loading by raising arithmetic intensity, but it expands activation storage proportionally. Lowering numeric precision from FP32 to 16-bit or 8-bit formats halves memory traffic and unlocks specialized matrix execution units, provided the reduced dynamic range preserves numerical stability and task accuracy. The physical machine therefore constrains not merely how quickly a fixed architecture executes, but which batch sizes, parameter counts, and numeric formats are physically realizable.

The pairwise intersections explain why these axes cannot be optimized in isolation. Data and algorithm determine *what the system can learn from*; data and machine determine *how information is represented and moved*; algorithm and machine determine *how the computation executes efficiently*. At the

center, all three constraints bind simultaneously. Expanding a dataset improves statistical coverage but increases storage traffic; widening a layer improves model capacity but exceeds accelerator memory; reducing arithmetic precision boosts execution throughput but risks task-level degradation.

The quantitative analyses in this chapter turn this diagnostic framework into concrete engineering practice. When confronted with an operational symptom—insufficient accuracy, a violated latency service-level objective, or an out-of-memory fault—engineers formulate competing hypotheses across all three axes. Disambiguating them requires measuring the binding physical or statistical quantity: sample coverage and label fidelity for data, gradient dynamics and error distributions for algorithm, or compute utilization, memory bandwidth saturation, and capacity footprints for machine. Because an intervention on one axis routinely shifts the binding constraint to another, validation requires profiling end-to-end performance under the intended production workload. Appendix A expands this first-pass diagnostic loop into a systematic reference of the intersection techniques applied across the curriculum.

Reasoning through the D-A-M taxonomy prevents treating systems problems as isolated software bugs. In practice, however, engineering decisions frequently falter when teams rely on intuitive assumptions—presuming that deep networks cannot be debugged, that depth invariably outperforms width, or that software-level accuracy guarantees operational throughput. Identifying where these heuristics fail requires examining the recurring pitfalls that arise when neural computation meets physical hardware.

5.8 Fallacies and Pitfalls

Statistical learning adds failure modes that line-by-line code inspection alone cannot reveal. The following fallacies and pitfalls can lead teams to misallocate effort or deploy inappropriate solutions.

Fallacy: *Neural networks are entirely opaque and cannot be debugged.*

Neural networks do not offer the line-by-line causal trace of traditional code, but they are not beyond diagnosis. Activation visualization can reveal learned patterns, gradient analysis measures local input sensitivity, and ablation studies estimate component contributions. These tools provide partial evidence rather than a complete explanation: saliency can be unstable, correlated features complicate attribution, and an ablation may change the behavior it is meant to probe. For the MNIST classifier in section 5.3.2, first-layer weights can still provide a useful check on what patterns the model has learned. Debugging a statistical system therefore combines model-level probes with data slices, controlled interventions, and end-to-end evaluation.

Pitfall: *Discarding domain expertise because a deep model is available.*

Teams assume automatic feature learning removes the need for domain knowledge. Successful systems require domain expertise at every stage: architecture selection, training objective design, dataset curation, and output interpretation. The document-recognition history in section 5.6 shows why confidence thresholds must reflect operating economics and route uncertain cases to human operators. Without domain knowledge, teams can deploy networks that look strong on a test set but fail in production because their thresholds, fallback paths, or error costs are wrong.

Fallacy: *Deeper networks are always more accurate than wider ones.*

Engineers assume that stacking more layers is the primary path to higher accuracy, since depth enables hierarchical feature extraction. In practice, depth alone encounters diminishing returns. ResNet showed that very deep residual networks can train effectively, but its ImageNet results also make clear that more layers must be weighed against added training and inference cost (He et al. 2016a). EfficientNet later demonstrated that compound scaling of width, depth, and input resolution can outperform depth-only scaling at a given resource budget (Tan and Le 2019). The lesson is not that depth is bad; it is that teams should profile capacity utilization and scale the architecture dimension that gives the best accuracy per FLOP.

Pitfall: *Using neural networks for problems solvable with simpler methods.*

Teams assume deep learning always performs better. On small datasets or approximately linear relationships, a simpler model such as logistic regression may match or outperform a neural network while requiring less training, memory, and maintenance. The magnitude of that advantage is work-

load dependent, so teams should benchmark a simple baseline and justify added complexity through measured task-specific gains. Neural networks excel at hierarchical pattern discovery (section 5.2.3) but impose additional overhead. Use them when their representation capacity produces enough benefit to justify the measured systems cost.

Fallacy: *Training data distribution issues can be fixed after model design.*

Teams treat training as mechanically feeding data through architectures. Networks on imbalanced datasets can exhibit poor minority-class performance: a fraud detector with 99:1 imbalance achieves 99 percent accuracy by always predicting “not fraud” while catching zero fraud cases. An unweighted average empirical-risk objective can underweight rare but critical classes unless sampling, weighting, loss design, and evaluation address the imbalance. Teams that skip exploratory data analysis may deploy models with strong aggregate metrics but unacceptable minority-class performance.

Pitfall: *Deploying research models to production without addressing system constraints.*

Production adds constraints absent from research: latency budgets (50–100 ms end-to-end), memory limits (2–4 GB for edge devices), and concurrent loads (100–1,000 requests per second (RPS)). The complete pipeline includes preprocessing, inference, and postprocessing (section 5.5). A model achieving 20 ms inference fails its 50 ms budget when preprocessing adds 25 ms and postprocessing adds 10 ms (55 ms total). Separating model development from system design therefore wastes effort on accuracy while ignoring deployment feasibility.

Fallacy: *More compute automatically means faster training.*

Teams purchase expensive GPUs expecting proportional speedups, but memory traffic or launch overhead may bind performance instead. Arithmetic intensity identifies the binding resource. Under a simple traffic model that reads the parameters once and moves each batch activation once, the MNIST forward pass in table 5.11 reaches approximately 12.4 FLOP/byte at batch 32 and still less at batch 1. This lies far below the ridge points of many high-throughput accelerators. A processor’s ridge point is its machine balance ratio, $R_{\text{peak}}/\text{BW}$ (peak compute in FLOP/s divided by peak memory bandwidth in bytes/s), defining the minimum arithmetic intensity needed to saturate its compute units. For modern server accelerators such as the NVIDIA A100 or H100, this ridge point sits between 150 and 300+ FLOP/byte. Because MNIST’s 12.4 FLOP/byte is more than an order of magnitude below that threshold, execution is trapped on the memory-bandwidth slope, rendering additional peak FLOP/s entirely useless until batch size or operational reuse increases. The Roofline Model in section D.2.1 explains why additional peak FLOP/s cannot help until the workload supplies enough reusable work per byte. Caches, fusion, vectorization, and launch cost make measured speedup the final test.

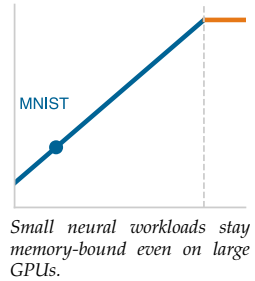
Pitfall: *Extrapolating accuracy improvements without considering diminishing returns.*

Teams may extrapolate a 5-point gain from scaling 10K parameters to 100K into another 5-point gain at 1M. Empirical improvement instead diminishes with scale according to the task, metric, data quality, model family, and scaling regime. Within table 5.5, the ImageNet rows show error falling from AlexNet’s 15.3 percent to ResNet-152’s 3.6 percent while training FLOPs rise from 5×10^{17} to roughly 10^{19} . Teams must therefore measure marginal return in their own regime rather than extrapolate linearly.

These fallacies and pitfalls share a common root: treating statistical behavior as if code behavior alone determined it. Recognizing them early directs attention toward the appropriate measurements and diagnostics.

5.9 Summary

Neural computation translates mathematical formulations into physical machine execution. Linear algebra and calculus determine operation cost under the iron law of ML systems ($T = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hww}}} + L_{\text{lat}}$). Forward propagation establishes the computational work O , often dominated by matrix multiplications, while backpropagation retains selected intermediate values and adds gradients and optimizer state. The resulting memory volume D_{vol} depends on architecture, batch size, precision, optimizer, and recomputation strategy. These mechanics make memory footprints, arithmetic intensities, and latency limits estimable before deployment.



Replacing hand-written decision rules with parameters learned from data transforms feature engineering into machine execution. Layered compositions of weighted sums, nonlinear activations, and gradient updates form hierarchical representations that convert raw inputs into predictions. Training adjusts connection weights through iterative optimization, while inference applies fixed parameters to incoming data. These phases impose divergent physical demands: in this chapter's FP32 ledger, a batch-32 training step requires $\sim 3.4\times$ the memory of batch-32 inference because backpropagation retains saved activations and allocates gradient and optimizer buffers. In production, as the document-recognition case study demonstrated, these mathematical kernels do not run in isolation; preprocessing, neural inference, rejection, and postprocessing must jointly satisfy a unified latency and reliability budget.



Key Takeaways: The math behind the model

- **Each paradigm shift changes the systems cost of representation:** In this 28×28 digit example, the operation count rises from ~ 100 comparisons (rule-based) through $\sim 8,000$ operations (classical ML) to 109,184 MACs (deep learning)—a $1,091.8\times$ scenario increase that reshapes hardware requirements.
- **Neural networks learn representations from data:** These networks replace some hand-coded features with hierarchical representations learned from examples. That flexibility shifts the burden toward representative data, training compute, and validation.
- **Training and inference often emphasize different objectives:** Training commonly favors efficient throughput across repeated updates, while online inference commonly favors per-request latency; offline inference may favor throughput instead. Batch size links utilization, memory, throughput, latency, and statistical behavior across these regimes.
- **Activations are math and hardware:** ReLU is generally cheaper to implement than sigmoid or tanh, and its unit gradient for positive inputs avoids activation saturation on that branch. Its practical advantage combines implementation cost with optimization behavior.
- **Forward propagation often centers on matrix work:** Dense matrix kernels exceed 90 percent of FLOPs in this chapter's dense-network example and dominate many important neural workloads, which is why specialized matrix hardware can substantially outperform general-purpose execution for those operations.
- **Backpropagation needs the path:** Solving credit assignment requires retaining or recomputing the values used by the backward pass, so memory cost often determines whether a model can be trained on a given device and motivates techniques that reduce, recompute, or partition training state.
- **The complete ML pipeline determines end-to-end performance:** Preprocessing, neural computation, and postprocessing all contribute to latency and reliability. The document-recognition examples demonstrated that production success depends on the entire pipeline operating within real-world constraints, not on model accuracy on a test set alone.

The escalation across computational paradigms is tangible in the MNIST workload: classifying a single 28×28 digit required about 100 comparisons under hand-crafted rules, but demanded 109,184 MACs in a modest three-layer network—a $1,091.8\times$ increase that alters every systems dimension in table 5.3. This comparison anchors the algorithm axis of the D-A-M taxonomy, demonstrating how algorithmic choices translate directly into machine constraints.

The multilayer perceptrons analyzed in this chapter expose a direct resource law: dense connectivity forces both parameter storage and multiply-accumulate count to scale with the product of adjacent layer dimensions. In the first layer alone, a 28×28 MNIST image presents 784 input values, requiring 100,352 weights to connect every input to 128 hidden neurons. That dense connectivity contains no explicit prior for spatial locality: the model must learn from data which nearby and distant pixel relationships matter, while paying memory bandwidth and storage to evaluate every connection.

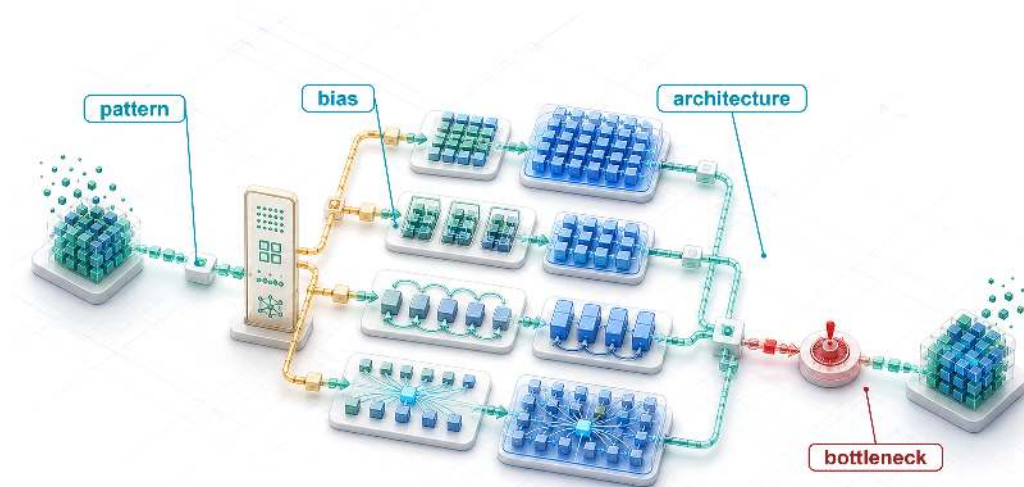
Reading the code reveals what a network is asked to do; reading the math reveals what it will cost to execute. That arithmetic is where the algorithm meets the machine: weights must be stored and transferred, dense layers map to matrix execution engines, and saved activations claim accelerator memory that determines whether training fits on device. Under the silicon contract, the math commits in advance to operations the machine must execute. Auditing that contract allows systems engineers to calculate binding bottlenecks from first principles before profiling confirms where the execution stalls.



What's Next: From universal to specialized

Generic fully connected networks turn every input-to-neuron connection into memory volume D_{vol} and compute work O . Yet real-world data possesses exploitable structure: images exhibit spatial locality, text contains sequential dependencies, and audio signals follow temporal patterns. Chapter 6 develops specialized architectures, such as convolutional networks and transformers, that encode these structural assumptions as inductive biases. Tailoring network topology to data structure reduces redundant data movement and improves weight reuse, but introduces new trade-offs in activation storage, arithmetic intensity, and accelerator parallelization.

Network Architectures

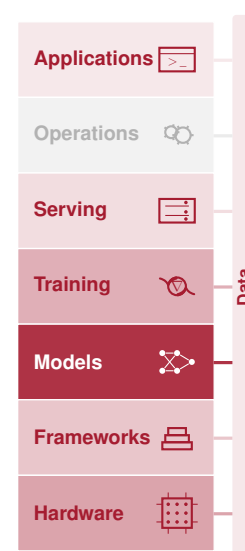


- 6.1 Architectural Principles
- 6.2 MLPs: Dense Pattern Processing
- 6.3 CNNs: Spatial Pattern Processing
- 6.4 RNNs: Sequential Pattern Processing
- 6.5 Attention: Dynamic Processing
- 6.6 Transformers: Parallel Sequence Processing
- 6.7 Sparse Architectures: RecSys
- 6.8 Shared Building Blocks
- 6.9 Computational Primitives
- 6.10 Architecture Selection Framework
- 6.11 Fallacies and Pitfalls
- 6.12 Summary

Purpose

Why is choosing a neural network architecture an infrastructure commitment as well as a modeling decision?

Selecting a neural network architecture is both a modeling decision and a contract with physics. The commitment begins with the structure in the data. An architecture that matches that structure can learn with fewer parameters and examples, while a poor match spends additional data and machine time compensating for the wrong computational pattern. Convolution, attention, recurrence, and embedding lookup do not merely encode different assumptions about data; they create different patterns of computation, state, and data movement. Those patterns determine whether work parallelizes cleanly, whether intermediate state fits in memory, and whether training and serving remain affordable at the required scale. Two architectures can deliver similar predictive quality yet impose different memory footprints and latency profiles while exposing different opportunities for later efficiency techniques. Benchmark accuracy alone therefore cannot reveal which design remains viable. Architectural commitments also propagate beyond the model. Data pipelines adopt particular input structures, training infrastructure is provisioned around the model's compute profile, serving systems are tuned to its request path, and monitoring inherits its failure modes. Once those dependencies accumulate, replacing the architecture can mean rebuilding much of the surrounding system. An architecture decision is sound only when its predictive gains remain compatible with the full deployment context. In D·A·M terms, architecture is algorithm-machine co-design at its root, with the mathematical graph determining the work the machine must perform and the machine budget limiting which graphs remain practical.





Learning Objectives

- Distinguish computational characteristics of MLPs, CNNs, RNNs, Transformers, and DLRM-style recommenders
- Explain how inductive biases exploit structure in different data types
- Analyze computational complexity and memory scaling across architectural families
- Identify building blocks such as skip connections, normalization, and gating that enable deep training
- Apply the architecture selection framework to match data characteristics with model designs
- Evaluate how compute, memory access, and data movement determine hardware mapping efficiency
- Critique architecture-selection fallacies under latency, bandwidth, and parallelization constraints

6.1 Architectural Principles

A dense model spends parameters and memory on relationships an image rarely needs. Convolutional neural networks (CNNs) encode locality; multilayer perceptrons (MLPs) do not. Matrix multiplication, activation functions, and gradient computation form the “verbs” of neural networks. Architectures assemble those verbs into computational graphs: specialized structures optimized for specific data types and computational constraints. Under the silicon contract (principle 4), every architecture makes an implicit agreement with hardware, trading computational patterns for efficiency on particular problem classes.

Every neural network architecture decides how computation should be organized to match the inherent structure in the data. Images have spatial locality, language has sequential dependencies, and tabular records often lack a fixed spatial or sequential organization. The architecture encodes assumptions about these patterns directly into the computational graph, and those assumptions influence parameter count, hardware utilization, and deployment feasibility. Architecture selection is therefore a systems engineering problem that directly affects the iron law terms: the number of operations O and the volume of data movement D_{vol} . The structural assumptions that each architecture encodes are known as **inductive biases**,¹ and they serve as the unifying concept for this entire chapter.

- CNN
- Transformer
- MLP

Inductive bias from strong (CNN) to weak (MLP): stronger prior, less data needed.

¹ | **Inductive bias:** From Latin *inducere*, “to lead into,” encoding a structural assumption “leads” the model toward a smaller solution space, which is why this concept unifies the entire chapter: every architecture discussed here—multilayer perceptron (MLP), convolutional neural network (CNN), recurrent neural network (RNN), and transformer—is defined by its choice of bias. A CNN’s locality bias cuts parameters by orders of magnitude vs. an equivalent MLP, directly shrinking the iron law’s O and D_{vol} terms, while global attention trades quadratic score computation for long-range connectivity.



Definition 6.1: Inductive bias

Inductive bias is a structural constraint built into a model architecture that restricts the hypothesis space, enabling generalization from finite data by encoding domain-specific assumptions (such as spatial locality or sequential ordering) directly into the computational graph.

1. **Significance:** Inductive bias can reduce the dataset size (D) required for generalization. For one output feature detector, a dense connection uses $\mathcal{O}(N_{\text{pix}} C_{\text{in}})$ parameters, while a shared CNN filter uses $\mathcal{O}(K^2 C_{\text{in}})$ parameters and is applied at every position, costing $\mathcal{O}(N_{\text{pix}} K^2 C_{\text{in}})$ operations. For a 224×224 RGB image, a shared 3×3 filter therefore uses roughly $5,575.1 \times$ fewer parameters than one dense feature detector, reducing parameter storage and often the data required to avoid overfitting.
2. **Distinction:** Unlike regularization (which penalizes hypothesis complexity at training time via L1/L2 terms), inductive bias restricts the hypothesis space at architecture design time: a CNN represents nonlocal functions less directly than local patterns, while regularization discourages complexity through the training objective.
3. **Common pitfall:** A frequent misconception is that stronger inductive bias is always better. A strong locality bias (CNN) excels on spatial data but represents long-range dependencies in language less directly than global attention, which supports long-range dependencies at $\mathcal{O}(S^2)$ score-computation cost.

A CNN encodes an inductive bias of spatial locality: nearby pixels matter more than distant ones. A transformer lets each element attend to any other, enabling long-range relationships at quadratic score-computation cost. These biases help architectures learn efficiently by restricting the space of functions they represent. Without an appropriate bias, a model may require substantially more data and compute to learn the same structure. The unified framework in section 6.10 brings these architectural families together once each bias has appeared in practice.

Machine learning systems face a core engineering trade-off: **representational power vs. computational efficiency**. Under the iron law of ML systems (principle 3), architectural choice is the primary determinant of the operation-count term O . A transformer’s attention mechanism enables global relationships but scales as $\mathcal{O}(S^2)$ operations with sequence length S ; a CNN exploits spatial locality to reduce operations to linear scaling in the number of spatial positions. Matching the right inductive biases to a workload’s data while setting a manageable operation-count budget defines the practice of neural architecture selection.

D Data

A Algorithm

M Machine

Architecture is the algorithm axis of D·A·M: it sets the operation-count budget.

Example 6.1: The ensemble Netflix did not ship (2009)

Context: Netflix awarded its \$1M Grand Prize in 2009 for a 10 percent improvement in recommendation accuracy (Johnston 2012).

Mechanism: Bringing the winning improvement into production required substantial engineering effort just as Netflix’s business was shifting from mailed DVDs to streaming. That shift also produced richer viewing signals and changed what personalization needed to optimize.

Impact: The additional accuracy did not justify the production effort, so Netflix did not deploy the Grand Prize code.

Fix: Netflix retained two algorithms from the earlier Progress Prize and redirected its recommendation work toward streaming-era personalization.

Systems lesson: An offline metric gain is only valuable when its integration cost and objective still match the production system.

Choosing an architectural backbone sets the primary memory layout and compute pattern for downstream hardware. Table 6.1 contrasts the five major neural network families across their spatial/temporal inductive biases, tensor operations, and memory access characteristics.

Table 6.1: Neural Architecture Families: Each family targets a distinct data structure and exposes a distinct system bottleneck. The bottleneck column reads directly from the iron law: MLPs stress memory bandwidth through dense activations, CNNs stress compute throughput because convolution reuses filters across many pixels, RNNs stress sequential dependencies that prevent parallelism, transformers compute quadratic score interactions and may materialize quadratic score storage, and DLRM stresses memory capacity at terabyte scale through embedding tables.

Architecture	Data Type	Core Innovation	System Bottleneck
MLPs	Tabular/Unstructured	Dense connectivity	Memory bandwidth
CNNs	Spatial (images)	Local filters + weight sharing	Compute throughput
RNNs	Sequential (time series)	Recurrent state	Sequential dependencies
Transformers	Relational (language)	Dynamic attention	Quadratic score compute; optional S^2 storage
DLRM	Categorical (recommendations)	Embedding tables	Memory capacity (TB+)

Five specific model architectures recur throughout this book as lighthouse models: consistent reference points that ground abstract concepts in concrete systems reality. These examples are concrete implementations of the Workload Archetypes (such as Compute Beast and Bandwidth Hog) introduced in section 2.3.2. Their selection reflects the empirical Pareto frontier of model evolution (figure 6.1), which maps ImageNet accuracy against computational cost.

These models serve as canonical workloads for understanding system constraints. Each occupies a distinct position on the trade-off between accuracy and computational cost, as mapped in figure 6.1.

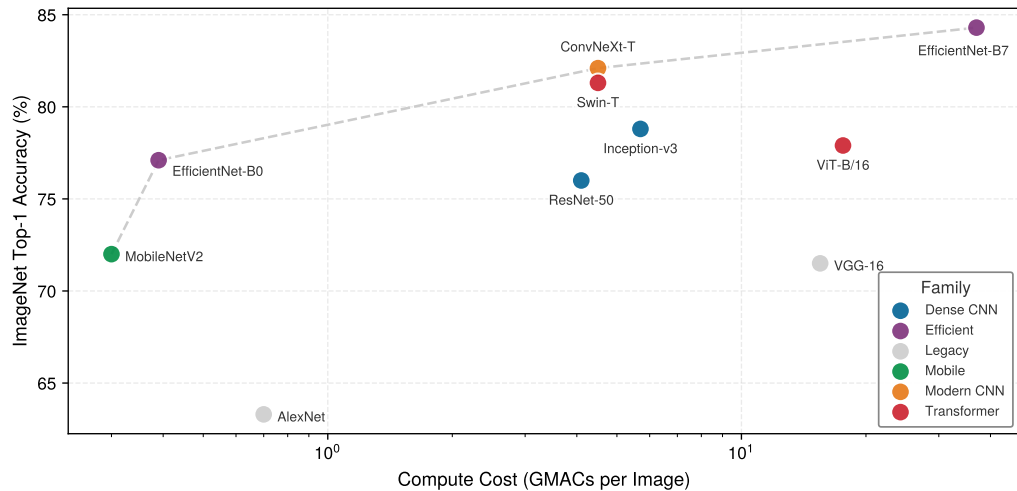


Figure 6.1: The Efficiency Frontier: ImageNet Top-1 Accuracy vs. Computational Cost (GMACs). Representative points are compiled from published model reports for AlexNet, VGG, ResNet, Inception, MobileNetV2, EfficientNet, ViT, Swin Transformer, and ConvNeXt (Krizhevsky et al. 2012; Simonyan and Zisserman 2015; He et al. 2016a; Szegedy et al. 2015; Sandler et al. 2018; Tan and Le 2019; Dosovitskiy et al. 2021; Liu et al. 2021; Liu et al. 2022). The dashed ‘Pareto frontier’ is an illustrative trade-off curve rather than a benchmark rerun; exact accuracy and reported compute values vary by model variant, training recipe, implementation, and operation-counting convention (multiply-accumulates vs. floating-point operations). The legend distinguishes six families: legacy CNNs (gray), dense CNNs (blue), efficient Mobile architectures (green), EfficientNet-class models (violet), modern CNNs (orange), and Transformers (red). The frontier is led by the efficient and modern-CNN families; the Transformers trade substantial compute for flexible long-range modeling without dominating it, and ViT-B/16 sits below the frontier.

The plotted frontier should be read as a historical map of representative architecture papers, not as a controlled benchmark table. It traces the progression from dense CNNs that prioritized accuracy, through MobileNets that reduced compute per unit of accuracy, to transformer architectures that trade substantial computational cost for flexible long-range modeling. Architectural choices made at design time determine where a system lands on this frontier.

6.1.1 Lighthouse roster: Model biographies

Five models earn the lighthouse role because each isolates one system bottleneck that recurs repeatedly: compute (ResNet-50), memory bandwidth (GPT-2), memory capacity (DLRM), edge latency (MobileNetV2), and always-on power for keyword spotting (KWS).

ResNet-50 (He et al. 2016a) anchors the compute-intensive vision lighthouse. The Residual Network (ResNet) addressed the degradation problem in very deep plain networks: adding layers could increase training error despite sufficient capacity. By introducing “skip connections” that improve optimization and gradient flow, it enabled networks of 50, 100, or even 1000 layers. The ResNet architecture won the ImageNet 2015 competition (with very deep 152-layer models), and ResNet-50 has become a widely used backbone and benchmark workload for computer vision. From a systems perspective, it is a highly regular, compute-intensive workload composed almost entirely of dense convolutions, making it a useful test for GPU floating-point throughput.

GPT-2 (Radford et al. 2019) anchors the bandwidth-bound language lighthouse. Generative Pre-trained Transformer 2 demonstrated that scaling up a simple decoder-only transformer architecture on massive datasets could produce coherent text generation. Unlike BERT, an encoder-style transformer that reads context in both directions, GPT-2 generates text sequentially (autoregressively²), creating substantial memory bandwidth pressure during batch-one and small-batch decoding because model weights are read for each decoding step. It serves as the archetype for large language models such as Llama and ChatGPT.

DLRM (Naumov et al. 2019) anchors the sparse-recommendation lighthouse. Meta open-sourced DLRM to expose a workload that differs from CNNs and transformers in a critical way. While vision and language models are compute-heavy, recommendation systems are memory-heavy. They must

² **Autoregressive generation:** A decoding strategy where each output token is conditioned on all previously generated tokens, requiring a full model forward pass per token; for a 1.5B-parameter model in FP16, the weight-only batch-one model used here reads about 3 GB of weights per decoding step and yields a work-to-byte ratio of about 1 FLOP/byte. This low arithmetic intensity often makes batch-one and small-batch decoding bandwidth bound; larger batches can amortize weight traffic. During generation, serving systems also retain attention state from earlier tokens; section 6.5 names this state the key-value cache.

look up user and item preferences in massive embedding tables that can reach terabytes in size, creating unique challenges for latency-critical serving (chapter 13). DLRM is a useful benchmark for memory capacity and sparse memory access patterns in the data center.



Lighthouse 6.1: Canonical workloads

In computer architecture, the *microprocessor without interlocked pipelined stages (MIPS)* processor is often used to teach pipelining, not because it is the fastest available chip, but because it is the clearest embodiment of reduced instruction set computer (RISC) principles. Similarly, this book uses ResNet-50, GPT-2, DLRM, MobileNetV2, and KWS as **canonical workloads**. Studying these “lighthouses” isolates engineering principles about math, memory movement, and locality that remain valid even as specific model architectures evolve.

MobileNet (Howard et al. 2017) anchors the edge-efficiency lighthouse. MobileNet challenged the trend of ever-larger models by prioritizing efficiency. It popularized depthwise separable convolutions for efficient vision models, an architectural innovation that reduced FLOPs by $8\text{--}9\times$ for 3×3 kernels with minimal accuracy loss. That smaller compute footprint made MobileNet a natural fit for compression and lower-precision deployment techniques (storing and computing with fewer bits per value) covered in chapter 10. It became a reference family for co-designing vision models with the battery and latency constraints of smartphones and embedded devices.

KWS (Warden 2018) anchors the always-on TinyML lighthouse. Keyword spotting models (like those detecting “Hey Siri” or “Ok Google”) represent the extreme end of efficiency. Designed to run on “always-on” microcontrollers with kilobyte-scale memory and milliwatt power budgets, these models (often depthwise separable CNNs) exemplify the constraints of TinyML (Warden and Situnayake 2020; C. R. Banbury et al. 2021; C. Banbury et al. 2021). They force engineers to count every byte and cycle, motivating extreme quantization (INT8 and INT4) and specialized hardware. Together, these biographies establish why the lighthouses are not a model catalog: each one isolates a different bottleneck signature that the arithmetic-intensity analysis can quantify.

6.1.2 Workload signatures: The arithmetic intensity spectrum

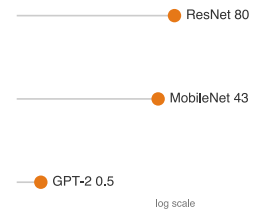
ResNet-50 reuses convolutional weights across many spatial positions, while batch-one GPT-2/Llama decode streams large weight and KV-cache state for one token at a time. That contrast motivates arithmetic intensity, the FLOP/byte ratio established in chapter 5 and used with a hardware roofline to assess whether a workload is likely to be compute or memory bound.

These bottlenecks reflect the underlying math, but the calculation here is a weight-only proxy rather than a complete measurement of main-memory traffic. It divides floating-point work by FP32 model-weight bytes, omitting activation, intermediate, and KV-cache traffic. Batching can amortize weight traffic across examples or tokens, while additional state movement can lower realized intensity. Section C.2.1 gives the per-operation FLOP and parameter formulas used in the proxy. Table 6.2 compares the three lighthouse scenarios and exposes a roughly $160.2\times$ gap under these assumptions.

Table 6.2: Lighthouse Workload Signatures: Weight-only FP32 intensity proxies for three model scenarios. These values suggest different compute and bandwidth pressures but do not determine a fixed hardware affinity; realized bottlenecks depend on execution regime and hardware balance.

Model Family	Lighthouse	Intensity (I)	Hardware Affinity
Dense CNN	ResNet-50	~ 80.1 FLOP/byte	Compute-Rich (GPUs/TPUs)
Efficient Vision	MobileNetV2	~ 42.8 FLOP/byte	Balanced (Mobile NPU)
Transformer	GPT-2 (Inf)	~ 0.50 FLOP/byte	Bandwidth-rich accelerator

This table provides one systems input to architecture selection. Task requirements determine which model families are viable, while batch size, precision, sequence length, implementation, cache behavior, and target hardware determine whether the proxy predicts the deployed bottleneck.



A weight-only proxy gives ResNet $\sim 160\times$ GPT-2's batch-one intensity.

MobileNet often suits constrained vision deployments, and batch-one transformer decoding often places heavy pressure on memory bandwidth, but both choices require measurement in the intended regime.

The bottleneck column in table 6.3 identifies which system resource—compute throughput, memory bandwidth, memory capacity, latency, or power—limits performance for each workload class. In iron law terms (section 1.7), this bottleneck reveals whether operations (O) or data movement (D_{vol}) governs execution time, determining which optimization strategies remain effective.

Table 6.3: Lighthouse Model Comparison: Quantitative characteristics and pedagogical roles of the five canonical workloads. Memory is FP32 weight storage, and FLOPs are per inference except for GPT-2 XL, which is quoted per generated token; the DLRM row reports embedding-table entries rather than dense parameters. The bottleneck column indicates the primary system constraint each model reveals: compute-bound models like ResNet stress arithmetic throughput, while bandwidth-bound models like GPT-2 stress memory transfer rates.

Model	Domain	Params	FLOPs/Inf	Memory	Bottleneck	Role in Textbook
ResNet-50	Vision	25.6M	8.2 GFLOP	102.4 MB	Compute	Dense vision throughput
GPT-2 XL	Language	1.5B	3 GFLOP/token	6 GB	Mem. Bandwidth	Token-by-token serving
DLRM	Recommender	25B	Low	100 GB	Mem. Capacity	Embedding tables and capacity planning
MobileNetV2	Edge Vision	3.5M	600 MFLOP	14 MB	Latency	Depthwise convolutions and efficiency
KWS (DS-CNN)	Audio	200K	20 MFLOP	800 KB	Power	Always-on power budget

Architecture selection is ultimately an engineering trade-off between math (O) and memory movement (D_{vol}). The mechanisms behind the signatures in table 6.2 explain why each lighthouse sits where it does on the intensity spectrum. ResNet-50 earns its high intensity because convolutional layers reuse each weight many times across the spatial dimensions of an image (deeper bottleneck layers reach 100–200+ FLOP/byte), so its performance is limited by hardware arithmetic throughput. GPT-2 sits at the opposite extreme: each generated token produces only a matrix-vector multiplication rather than the matrix-matrix operations of batch processing, so the system loads massive weights from memory for a single token’s math, and performance is limited by memory bandwidth. MobileNet lands between the two at the whole-model level, with individual depthwise layers falling lower once activation traffic is included: depthwise separable convolutions reduce total O but move more data relative to that work, which fits mobile hardware well yet often “starves” high-end GPUs optimized for dense math.



Checkpoint 6.1: Arithmetic intensity and architecture

Match the architectural choice to its systems implication:

- ☐ **Weight reuse (CNNs):** how does reusing the same weights across many inputs change arithmetic intensity?
- ☐ **Large embedding tables (DLRM):** how do massive embedding tables change the balance between data movement and computation?
- ☐ **Autoregressive decoding (GPT-2):** why does token-by-token generation change the batching and bandwidth picture?

This spectrum determines whether the system needs a faster processor or faster memory to improve performance. The framework this book uses to quantify these limits on specific hardware is the **roofline model**, which plots achievable throughput against arithmetic intensity to show where a workload turns from memory bound to compute bound. Section D.2.1 gives the analytical treatment, with applied examples in chapter 11. Section D.2.1.1 works this intensity-to-bottleneck classification through a real accelerator specification, computing the ridge point of an A100 and showing how the same operation falls on either side of it.

The lighthouse signatures make the next step concrete: inspect each architecture family by the data pattern it targets, the computation it performs, the hardware mapping it induces, and the bottleneck it exposes. This four-part lens ensures that every architecture is evaluated for what it costs to run, not only for what it learns.

6.2 MLPs: Dense Pattern Processing

Consider a smartphone’s spam filter: given a set of features extracted from an email (sender reputation score, number of links, presence of certain keywords), the model must output a single probability: spam or not. This classification task, where every input feature connects to every output, is the domain of fully connected networks. **MLPs**³ represent the fully connected architectures introduced in chapter 5, now examined through the four-part systems lens established earlier.

MLPs embody an inductive bias: *they assume no prior structure in the data, allowing any input to relate to any output*. This architectural choice enables maximum flexibility by treating all input relationships as equally plausible, making MLPs versatile but computationally intensive compared to specialized alternatives. Their representational capacity was established theoretically by the Universal Approximation Theorem (UAT)⁴ (Cybenko 1989; Hornik et al. 1989), introduced in chapter 5. This theorem states that a sufficiently large MLP with nonlinear activation functions can approximate any continuous function on a compact domain, given suitable weights and biases. That combination of theoretical universality and dense connectivity is the architectural concept captured by the *multilayer perceptron*.

Definition 6.2: Multilayer perceptrons

Multilayer perceptrons are feed-forward neural network architectures that apply fully connected layers in sequence, where every neuron in one layer connects to every neuron in the next, encoding no structural assumption about the input domain.

1. **Significance:** The lack of structural prior gives dense layers quadratic parameter scaling in layer width: a single layer mapping 1,024 inputs to 1,024 outputs requires 1,048,576 parameters and about 2.1 MB of weight memory in FP16. A 3×3 convolution mapping 1,024 input channels to 1,024 output channels has about 9.4M weights; convolution’s advantage for images comes from spatial weight sharing across positions, not from reducing the channel-mixing matrix itself. This makes MLPs inefficient for high-dimensional structured inputs like images.
2. **Distinction:** Unlike convolutional neural networks, which exploit spatial locality to reduce parameter count, MLPs treat all input elements symmetrically, making them the architecture of choice for tabular data where no spatial or sequential structure is present.
3. **Common pitfall:** A frequent misconception is that MLPs are too simple to matter for complex tasks. MLPs provide a useful dense baseline, but CNNs, recurrent networks, and transformers add operators and connectivity patterns that cannot be reduced to weight sharing alone.

In practice, the UAT explains *why* MLPs succeed across diverse tasks while revealing the gap between theoretical capability and practical implementation. The theorem guarantees that some MLP can approximate any function, yet provides no guidance on requisite network size or weight determination. While MLPs can theoretically solve any pattern recognition problem, doing so may demand impractically large networks or prohibitive computation. This theoretical universality drives the selection of MLPs for tabular data, recommendation systems, and problems where input relationships are unknown. At the same time, these practical limitations motivated the development of specialized architectures that exploit data structure for computational efficiency, as section 6.3, section 6.4, and section 6.6 demonstrate.

6.2.1 Learnability gap

The UAT guarantees mathematical existence, yet a fundamental gap separates what an architecture can represent from what gradient descent can learn in practice.

³ | **Perceptron:** Formed from “perception” and the device suffix “-tron” (as in *cyclotron* and *klystron*), coined by Frank Rosenblatt (Rosenblatt 1957) for the atomic unit of neural computation: a weighted sum followed by a nonlinear activation, extending the earlier McCulloch-Pitts neuron. MLPs are composed entirely of these units arranged in fully-connected layers, so the efficiency of this single operation, a multiply-accumulate, determines system throughput. Modern accelerators execute over 10^{14} of these operations per second, making the perceptron the computational primitive that the entire ML hardware ecosystem is optimized around.

⁴ | **Universal approximation theorem (UAT):** This theorem provides the mathematical guarantee for the MLP’s “no prior structure” inductive bias by proving a sufficiently wide network can approximate any continuous function. The systems-level catch is that “sufficiently wide” can require a number of neurons that grows exponentially with input dimensionality, rendering the theoretical guarantee practically unattainable for even moderately sized inputs like a 256×256 image.

5 | **Manifold hypothesis:**

The assumption that high-dimensional data lies on a low-dimensional surface embedded within the full space; a 256×256 image lives in a 65,536-dimensional space, but “valid cat images” occupy a tiny structured region. Deep networks progressively unfold this crumpled manifold into linearly separable representations. The systems consequence: if data truly occupied the full space, no architecture could learn from feasible dataset sizes; the manifold structure is what makes finite training budgets sufficient.

Representation capacity refers to the functions an architecture can express given unlimited resources; the UAT established earlier guarantees MLPs have universal representation capacity. This capacity is particularly effective because of the *Manifold Hypothesis*,⁵ which suggests that high-dimensional data actually occupies a much simpler structure. *Learnability* refers to whether gradient descent can find good weights given finite training samples and computational budgets. A function may be representable yet practically unlearnable.

This distinction resolves the apparent paradox of universal approximation and architectural progress. Specialized architectures such as ResNets and transformers improve learnability by embedding inductive biases that match data structure, even when doing so restricts representational capacity.

Three factors create the *learnability gap*:

1. **Sample complexity:** The UAT provides no bounds on training examples needed. For 28×28 images, an MLP treats 784 pixels independently, requiring exponentially many samples to learn spatial correlations. A CNN embeds locality bias, drastically reducing sample requirements. Mathematically, sample complexity can scale exponentially with input dimension for MLPs but polynomially for architectures matching data structure.
2. **Parameter efficiency:** The UAT guarantees some width suffices, but provides no constructive bounds. Some function classes require widths that grow rapidly with input dimension, while architectures with a matching compositional structure can represent them more compactly.
3. **Optimization difficulty:** Even when optimal weights exist, gradient descent may not find them. MLP loss surfaces exhibit complex topology without the regularizing effect of architectural constraints. Specialized architectures reduce the search space, introducing symmetries that gradient descent exploits. The classic MNIST handwritten digit benchmark illustrates this gap between representation and learnability concretely.



Example 6.2: MNIST: Representation vs. learnability

Scenario: Classifying 28×28 MNIST digits using a fully connected MLP versus a CNN.

Diagnosis: A 3-layer MLP requires 20M parameters because it treats all 784 input pixels independently, ignoring spatial structure. A CNN uses local receptive fields and weight sharing, cutting parameters to 421.4K ($47 \times$ fewer parameters).

Systems lesson: Inductive bias drives parameter efficiency. Incorporating spatial locality into model architecture reduces parameter footprint by orders of magnitude, lowering SRAM memory bandwidth pressure during inference.

6 | **No free lunch theorem:**

Wolpert and Macready's 1997 result proved that no optimization algorithm outperforms random search across all possible problems: averaged over every conceivable function, all algorithms are equivalent. The ML systems consequence is that an inductive bias (locality, equivariance, attention) can improve performance on problems matching that bias while hurting performance when its assumptions do not hold, making architecture selection an engineering commitment to a problem class.

The learnability gap motivates the core design principle of this chapter: embed inductive biases that match data structure. Each architecture sacrifices theoretical generality for practical learnability. The No Free Lunch theorem⁶ (Wolpert and Macready 1997) formalizes this trade-off: the bias that helps one task may hurt another. CNN's translation invariance aids image classification but hurts tasks where absolute position matters. Architecture selection is fundamentally the act of matching inductive bias to data structure.

These theoretical insights translate directly into engineering decisions. Appropriate inductive biases reduce parameter counts (enabling edge deployment), accelerate convergence (reducing training costs), and produce structured computation patterns that map efficiently to specialized hardware (chapter 11). A 20M-parameter MLP infeasible for edge deployment becomes a 421.4K-parameter CNN that fits comfortably, a $47 \times$ reduction achieved by matching architecture to data structure. Understanding where dense connectivity remains necessary requires identifying the data patterns that lack structural shortcuts.

6.2.2 Pattern processing needs

Dense pattern processing targets problem domains where input features lack geometric coordinates or sequential ordering. In tabular datasets, recommendation embeddings, or multi-factor risk models, any input feature may interact with any other. Because the data representation carries no grid or sequence structure, the network cannot restrict computation to local spatial windows

or sequential steps. Every output neuron must be free to combine every input feature, leaving optimization to determine connection weights.

When treating an image dataset like MNIST with a dense architecture, the system flattens the 28×28 image into an unstructured 784-element vector. Discarding the two-dimensional spatial prior forces the network to treat all pixel pairs symmetrically. While this enables the layer to relate any pixel to any class, it requires the model to discover spatial adjacency entirely through optimization—an architectural choice that leads directly to the matrix operations defining the MLP.

6.2.3 Algorithmic structure

All-to-all feature interaction requires complete connectivity between adjacent layer representations. In an MLP, this manifests as a sequence of fully connected layers where every neuron connects to every neuron in the preceding layer. Dense connectivity translates directly into matrix multiplication, the formulation introduced in section 5.4.2.2 that makes dense layers executable on parallel hardware. Figure 6.2 shows how each layer transforms its input through this core operation.

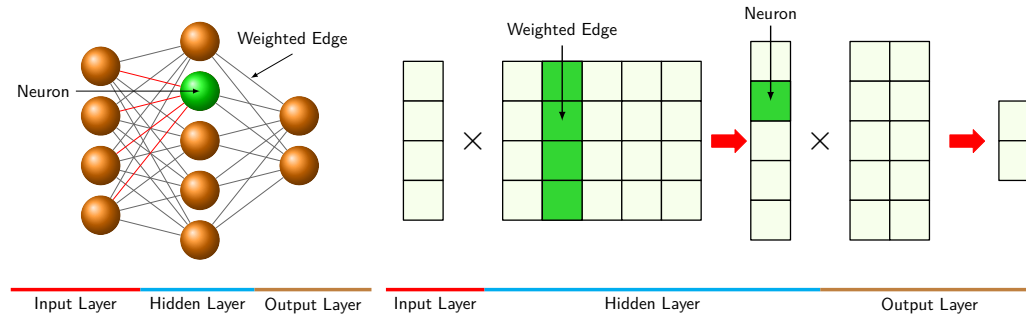


Figure 6.2: Multilayer Perceptron Architecture: A three-layer network in which every neuron connects to all neurons in adjacent layers. The highlighted neuron receives weighted contributions from all inputs, illustrating the dense $\mathcal{O}(N \times M)$ connectivity pattern implemented through matrix multiplications. For MNIST classification, a 784-dimensional input connects to 100 hidden neurons through a 784×100 weight matrix, requiring 78,400 multiply-accumulate operations per sample. Adapted from (Reagen et al. 2017).

Equation 6.1 expresses the dense layer’s affine transformation followed by its activation.

$$\mathbf{h}^{(\ell)} = f(\mathbf{h}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}) \quad (6.1)$$

$\mathbf{h}^{(\ell)}$ represents the layer ℓ output, $\mathbf{h}^{(\ell-1)}$ represents the input from the previous layer, $\mathbf{W}^{(\ell)}$ denotes the weight matrix for layer ℓ , $\mathbf{b}^{(\ell)}$ denotes the bias vector, and $f(\cdot)$ denotes the activation function; section 5.3.2.2 develops the rectified linear unit (ReLU) and related nonlinearities in detail. This layer-wise transformation, while conceptually simple, creates computational patterns whose efficiency depends critically on how the runtime organizes these operations for different problem structures.

The dimensions of these operations reveal the computational scale of dense pattern processing. The input vector $\mathbf{h}^{(0)} \in \mathbb{R}^{d_{\text{in}}}$ (treated as a row vector in this formulation) represents all potential input features. Weight matrices $\mathbf{W}^{(\ell)} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ capture all possible input-output relationships. The output vector $\mathbf{h}^{(\ell)} \in \mathbb{R}^{d_{\text{out}}}$ produces transformed representations. A four-pixel example turns this bookkeeping into arithmetic.

Example 6.3: Concrete computation example

Consider a simplified 4-pixel image processed by a 3-neuron hidden layer:

Input: $\mathbf{h}^{(0)} = [0.8, 0.2, 0.9, 0.1]$ (4 pixel intensities)

Weight matrix:

$$\mathbf{W}^{(1)} = \begin{bmatrix} 0.5 & 0.1 & -0.2 \\ -0.3 & 0.8 & 0.4 \\ 0.2 & -0.4 & 0.6 \\ 0.7 & 0.3 & -0.1 \end{bmatrix} \quad (4 \times 3 \text{ matrix})$$

Computation:

$$\mathbf{z}^{(1)} = \mathbf{h}^{(0)} \mathbf{W}^{(1)} = \begin{bmatrix} 0.5 \times 0.8 + (-0.3) \times 0.2 + 0.2 \times 0.9 + 0.7 \times 0.1 \\ 0.1 \times 0.8 + 0.8 \times 0.2 + (-0.4) \times 0.9 + 0.3 \times 0.1 \\ (-0.2) \times 0.8 + 0.4 \times 0.2 + 0.6 \times 0.9 + (-0.1) \times 0.1 \end{bmatrix} = \begin{bmatrix} 0.59 \\ -0.09 \\ 0.45 \end{bmatrix}$$

After ReLU: $\mathbf{h}^{(1)} = [0.59, 0, 0.45]$ (negative values zeroed)

Systems insight: Each hidden neuron combines all input pixels with different weights, demonstrating unrestricted feature interaction. Dense layers buy generality by paying the maximum connectivity cost.

The MNIST example makes this scale concrete. The 784-dimensional input connects to every neuron in the first hidden layer. A hidden layer with 100 neurons requires a 784×100 weight matrix (78,400 parameters), where each weight represents a learnable relationship between an input pixel and a hidden feature. This single layer anchors the computational analysis throughout this chapter.

This algorithmic structure enables arbitrary feature relationships while creating specific computational patterns that computer systems must accommodate. Dense connectivity provides the universal approximation capability established earlier but introduces computational redundancy: while the theoretical universality of MLPs enables modeling of any continuous function given sufficient width, this flexibility requires numerous parameters to learn relatively simple patterns. Every input feature influences every output, yielding maximum expressiveness at the cost of maximum computational expense. These trade-offs motivate later compression strategies that reduce computational demands while preserving model capability, and chapter 11 explores hardware-specific implementations that exploit regular matrix operation structure.

6.2.4 Computational mapping

Section 6.2.3 defines *what* an MLP computes; computational mapping reveals *how* that computation translates to hardware operations. Listing 6.1 demonstrates how this mapping progresses from mathematical abstraction to computational reality.

Listing 6.1: MLP Layer in Matrix Form: Framework-level matrix operations hide $\mathcal{O}(B \times d_{\text{in}} \times d_{\text{out}})$ multiply-accumulate complexity behind a single function call, enabling hardware-optimized BLAS libraries to use dense matrix hardware efficiently.

```
def mlp_layer_matrix(X, W, b):
    """MLP forward pass using framework-level matrix operations."""
    # X: input matrix (batch_size by num_inputs)
    # W: weight matrix (num_inputs by num_outputs)
    # b: bias vector (num_outputs)

    # Single GEMM call: frameworks dispatch to optimized BLAS/cuBLAS
    # For MNIST: 784 * 100 = 78,400 MACs per sample
    H = activation(matmul(X, W) + b)
    return H
```

The function `mlp_layer_matrix` directly mirrors the mathematical equation, employing high-level matrix operations (`matmul`) to express the computation in a single line while abstracting the underlying complexity. This implementation style characterizes deep learning frameworks, where optimized libraries manage the actual computation.

Framework operations such as `output = matmul(X, W)` abstract physical execution. From the hardware perspective, dense matrix multiplication lowers to a canonical three-deep loop nest that determines memory access locality, parallelization boundaries, and register utilization.

The implementation in listing 6.2 exposes this execution pattern directly. Computing layer outputs requires accumulating weighted contributions across all inputs for each sample in the batch. The outermost loop iterates across batch samples, the middle loop iterates over output neurons, and the innermost loop executes repeated multiply-accumulate (MAC) operations⁷ pairing each input feature with its corresponding parameter.

Listing 6.2: Dense Layer Computation: Nested loops reveal $\mathcal{O}(B \times d_{\text{out}} \times d_{\text{in}})$ complexity where each output neuron requires `num_inputs` multiply-accumulate operations, so the reference MNIST input layer requires 78,400 MACs per sample.

```
def mlp_layer_compute(X, W, b):
    """Explicit loop structure exposing MLP computational patterns."""
    # Loop 1: Process each sample independently (parallelizable)
    for batch in range(batch_size):
        # Loop 2: Compute each output neuron
        for out in range(num_outputs):
            Z[batch, out] = b[out] # Initialize with bias

            # Loop 3: Accumulate weighted inputs (innermost loop)
            # This is the MAC operation: result += input * weight
            for in_ in range(num_inputs):
                Z[batch, out] += X[batch, in_] * W[in_, out]
            # Total per output: num_inputs MACs +
            # num_inputs memory reads

    H = activation(Z) # Element-wise nonlinearity
    return H
```

In the reference MNIST layer, computing 100 hidden neurons requires 78,400 MACs in total; each individual output neuron requires 784 multiply-accumulate operations and at least 1,568 memory accesses (784 for inputs, 784 for weights). Production implementations call optimized matrix libraries such as Basic Linear Algebra Subprograms (BLAS),⁸ but the same nested-loop pattern still determines the system design problem. The hardware architectures that accelerate these matrix operations, including GPU Tensor Cores⁹ and specialized AI accelerators, are covered in chapter 11.

6.2.5 System implications

Dense pattern processing concentrates hardware pressure across three dimensions: memory capacity for parameter storage, compute throughput on matrix engines, and memory bandwidth for weight movement.

All three constraints originate from all-to-all connectivity. Memory footprint is dominated by parameter storage. While the reference MNIST layer (784×100) requires only 78,400 parameters, $\mathcal{O}(M \times N)$ scaling becomes prohibitive for wide hidden states. A layer connecting 2048 inputs to 2048 outputs requires 4,194,304 parameters (16.8 MB at FP32). Because every weight is read once per input vector during single-sample inference, there is no opportunity for weight reuse across inputs, leaving execution bound by memory capacity and bandwidth.

The core computation is dense GEMV, or GEMM when batched. This computation is regular and parallelizable, but the arithmetic intensity (FLOP/byte) is low for small batches. The **batch size** is the number of input samples processed together; larger batches amortize weight-loading cost over more computations. Modern processors optimize dense layers with single instruction, multiple data (SIMD) units such as AVX-512 on CPUs or systolic arrays on Tensor Processing Units (TPUs) and GPUs, amortizing control overhead over large blocks of parallel arithmetic.

The resulting bottleneck is data movement. To compute 100 hidden values from 784 inputs, the system must move 784×100 weights from memory to the compute units. Applying the arithmetic intensity framework from section 6.1.2 to this layer yields roughly 0.5 FLOP/byte for batch-one FP32 execution. Under a weight-only model that counts one FP32 read per weight and omits input,

⁷ | **MAC (multiply-accumulate):** The atomic operation of neural networks: multiply two values and add to a running sum. In Horowitz's 45 nm reference, an FP32 multiply plus add costs about 4.6 pJ, while a 32-bit off-chip DRAM access costs about 640 pJ (Horowitz 2014). These technology-specific values illustrate why data movement can dominate arithmetic energy; they are not universal constants for current hardware.

⁸ | **BLAS (basic linear algebra subprograms):** This standard API for matrix operations enables the use of highly optimized libraries (for example, cuBLAS) to accelerate the 784 multiply-accumulates per neuron. The 784×100 matrix in the MNIST example may use the hardware less efficiently than larger, well-aligned transformer matrices because utilization depends on matrix shape, batching, datatype, library, and accelerator.

⁹ | **Tensor Cores:** Specialized units in NVIDIA GPUs that accelerate the thousands of multiply-accumulate operations described by fusing them into single, highly parallelized matrix instructions. Tensor Cores are most efficient when matrix dimensions meet datatype- and architecture-specific alignment multiples; modern cuBLAS/cuDNN can still use Tensor Cores for many nonaligned cases, often with lower efficiency or internal padding. The architectural lesson is vendor-independent: specialized matrix units reward dense, aligned general matrix multiplication (GEMM) workloads and penalize small, irregular shapes that cannot keep the units full.

output, bias, and cache traffic, equation 6.2 gives this ratio, where M and N are the input and output widths.

$$\text{Intensity} \approx \frac{2 \cdot M \cdot N \text{ FLOPs}}{4 \cdot M \cdot N \text{ bytes}} = 0.5 \text{ FLOP/byte} \quad (6.2)$$

On accelerators with ridge points in the hundreds of FLOP/byte, this batch-one FP32 layer is memory-bandwidth bound. Batching can raise intensity by amortizing weight traffic, but the crossover depends on matrix shape, precision, implementation, and hardware. Specifically, increasing batch size to B scales arithmetic to $2 \cdot B \cdot M \cdot N$ FLOPs while loading the weight matrix once ($4 \cdot M \cdot N$ bytes), driving weight-bound arithmetic intensity up to $\text{Intensity} \approx 0.5B$ FLOP/byte until input and activation traffic begin to dominate. This is why fully connected layers can become inference bottlenecks despite performing fewer total FLOPs than convolutional layers.

Dense connectivity thus moves maximum data for minimum compute. For data with inherent structure (such as spatial locality in images or temporal order in sequences), specialized architectures can exploit that structure for both better accuracy and better efficiency. The most established such architecture is the convolutional neural network.

6.3 CNNs: Spatial Pattern Processing

The MLP's assumption that every input may interact directly with every output proves particularly costly for spatially structured data like images. Connecting every pixel of a high-resolution input to every hidden unit creates an unmanageable parameter footprint that exhausts accelerator memory and collapses arithmetic intensity into a memory-bandwidth-bound crawl. As the earlier MNIST comparison demonstrated, the example CNN uses $47\times$ fewer parameters by exploiting spatial locality rather than treating every pixel independently.

CNNs¹⁰ structure this computational trade-off directly in silicon (LeCun et al. 1998; Krizhevsky et al. 2012). Rather than materializing an all-to-all connectivity matrix, a CNN restricts each output activation to a local spatial patch and slides identical filter weights across the entire input aperture.

This spatial inductive bias translates into two structural constraints on the computation graph. Parameter sharing applies the same small filter weights across every spatial position, reducing parameter storage from millions of distinct weights to a compact kernel that fits within on-chip caches. Local connectivity limits each neuron's receptive field to a spatially adjacent neighborhood, eliminating computation on distant, uncorrelated pixels. Together, these constraints define the convolutional family.



Definition 6.3: Convolutional neural networks

Convolutional neural networks (CNNs) are architectures that exploit translation equivariance and spatial locality to share learned filters across all spatial positions, decoupling parameter count from input resolution.

1. **Significance:** Weight sharing produces dramatic parameter reduction. A 3×3 convolutional layer with 64 input and 64 output channels requires $3 \times 3 \times 64 \times 64 \approx 37,000$ parameters regardless of whether the input image is 224×224 or 1024×1024 . An equivalent fully connected layer on a $224 \times 224 \times 64$ input would require $224^2 \times 64 \times 64 \approx 205$ million parameters, a roughly $5,575\times$ difference. This constant-parameter scaling enables CNNs to process high-resolution inputs within the memory budget of a single accelerator.
2. **Distinction:** Unlike MLPs, which connect every input element to every output element (global connectivity), CNNs restrict each output to a local spatial neighborhood, encoding the assumption that nearby pixels are more relevant than distant ones. This restriction eliminates entire hypothesis classes at architecture design time rather than penalizing them during training.
3. **Common pitfall:** A frequent misconception is that CNNs are vision-only models. The convolution operation applies to any data with a grid-like topology: 1D convolutions process audio waveforms and time series, 2D convolutions process images and spectrograms, and 3D convolutions process video and volumetric data.

FC 205M

Conv 37K

log scale

Weight sharing spares convolution the fully-connected parameter explosion.

¹⁰ | **Convolution:** From Latin *convolvere* (“to roll together”), describing a filter that slides across an input, combining local elements at each position. This “rolling together” enforces a locality constraint that is the source of the operation’s efficiency: a single 5×5 kernel reuses its 25 weights at every spatial position, reducing one feature detector for a 1-megapixel single-channel image from roughly 1,000,000 weights to 25, about $40,000\times$ fewer parameters than a fully connected detector.

The trade-off is explicit: CNNs sacrifice the theoretical generality of MLPs for practical efficiency gains when data exhibits known structure. Where MLPs treat each input element independently, CNNs exploit spatial relationships to achieve both computational savings and improved accuracy on vision tasks.

6.3.1 Pattern processing needs

Spatial pattern processing addresses workloads where correlation between data points decays rapidly with geometric distance. In a natural image, a pixel's covariance with adjacent pixels is high, enabling the detection of localized primitives such as edges, textures, and contours. Stacking these operations allows local primitives to compose hierarchically into higher-level representations: edges compose into contours, contours into surface patches, and patches into coherent objects. The pipeline in figure 6.3 gives this hierarchy a concrete visual form.

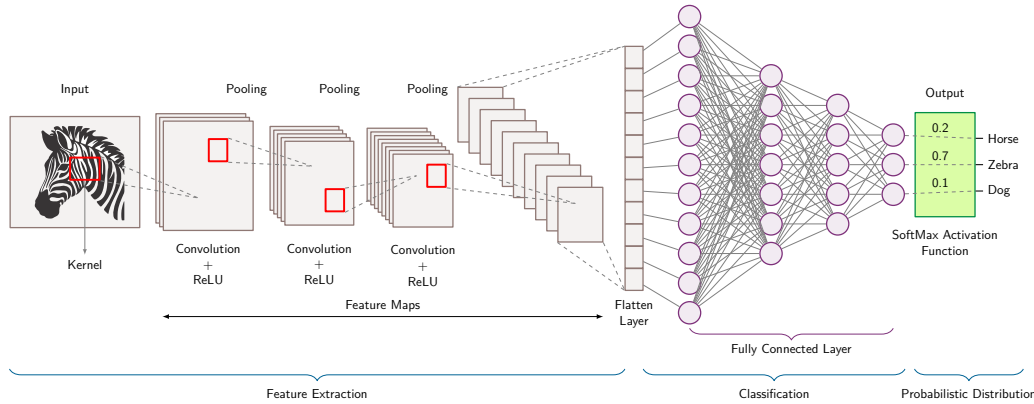


Figure 6.3: Spatial Feature Extraction: A zebra image passes through repeated convolution, ReLU, and pooling stages that form progressively deeper feature maps, followed by flattening, fully connected layers, and a softmax classification over Horse, Zebra, and Dog.

This hierarchical composition appears across multiple physical domains: localized pixel neighborhoods forming contours in computer vision, local time-frequency clusters identifying phonemes in speech spectrograms, and contiguous voxel blocks isolating tissue boundaries in volumetric tomography. The architecture succeeds not because it mimics biological vision, but because it matches the compositional structure of the underlying data.

In image processing, recognizing an object requires two distinct computational guarantees. First, the architecture must capture local geometric primitives over bounded receptive fields. Second, because an object retains its semantic identity regardless of where it appears in the frame, the network must identify those primitives across arbitrary coordinate translations.¹¹ Convolutional networks satisfy both requirements by combining localized kernels with parameter sharing, enforcing translation equivariance at intermediate stages while pooling enables downstream invariance (LeCun, Boser, et al. 1989; Krizhevsky et al. 2012).¹² These architectural constraints, introduced by LeCun¹³ (LeCun, Boser, et al. 1989), replace unconstrained dense matrix multiplications with structured spatial stencils.

6.3.2 Algorithmic structure

In formal terms, equation 6.3 evaluates the forward pass of a 2D convolutional layer by computing local, channel-wise dot products across every spatial coordinate:

$$\mathbf{H}_{i,j,k}^{(\ell)} = f \left(\sum_m \sum_n \sum_c \mathbf{w}_{m,n,c,k}^{(\ell)} \mathbf{H}_{i+m,j+n,c}^{(\ell-1)} + \mathbf{b}_k^{(\ell)} \right) \quad (6.3)$$

Here, $\mathbf{H}_{i,j,k}^{(\ell)}$ denotes the activation at spatial position (i, j) in output channel k of layer ℓ . The triple summation iterates across the filter dimensions: spatial offsets (m, n) span the local kernel window

¹¹ | **ImageNet:** The dataset that validated these two spatial processing requirements at scale. AlexNet's 2012 victory in the ImageNet Large Scale Visual Recognition Challenge reduced top-5 error from 26.2 percent to 15.3 percent on the 1000-class challenge with roughly 1.2 million training images (Krizhevsky et al. 2012); the original ImageNet release contained 3.2 million images across 5,247 synsets (Deng et al. 2009). Subsequent architectures improved accuracy through changes in architecture, optimization, data, and compute budgets, illustrating the interaction between inductive bias and infrastructure cost.

¹² | **Translation equivariance:** An inherent property of the convolution operation where shifting the input guarantees a corresponding spatial shift in the resulting feature map. This is distinct from *invariance*, which pooling or later aggregation can approximate by discarding precise positional data. For example, 2×2 pooling with stride 2 reduces the number of spatial elements by 75 percent.

¹³ | **Yann LeCun and LeNet:** LeCun's architecture directly addressed the intractable scaling of applying dense networks to images by enforcing the principles of local connectivity and parameter sharing. These constraints reduced the parameter count for an image-like input layer by over 95 percent, enabling LeNet-5 to achieve production-grade accuracy on commercial tasks like check reading with only ~60,000 total parameters.

¹⁴ | **Receptive field:** The input region influencing a particular output neuron. With 3×3 filters, receptive fields grow by 2 pixels per layer, so a neuron at layer 3 "sees" a 7×7 region. This growth rate constrains architecture depth: detecting objects spanning 100+ pixels in a 224×224 image requires either deep stacks of small filters (more layers, more memory for activations) or larger kernels (more parameters per layer), a fundamental depth-vs.-width trade-off in CNN design.

(defining the layer's local receptive field),¹⁴ while c ranges over all input channels. The tensor $\mathbf{W}_{m,n,c,k}^{(\ell)}$ stores the kernel weights connecting channel c to channel k , and $\mathbf{b}_k^{(\ell)}$ is the per-channel bias. Because the kernel weights $\mathbf{W}^{(\ell)}$ do not depend on the spatial coordinates (i, j) , the same filter parameters are reused at every coordinate.

Convolutional layers typically use small spatial kernels (3×3 or 5×5) to maintain compact parameter footprints while stacking layers to widen the effective receptive field. The sliding-window mechanics in figure 6.4 illustrate this operation: a filter moves across the input array, computing a multiply-accumulate dot product at each valid coordinate to generate an output feature map. The interactive CNN Explainer tool (Wang et al. 2021) visualizes how these stacked sliding windows extract per-channel activation patterns across layers.

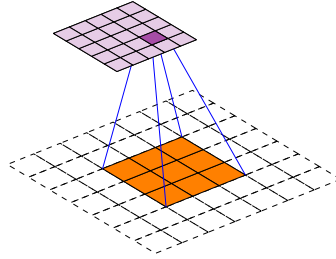


Figure 6.4: Convolution Operation: A 3×3 orange receptive field in a dashed 7×7 input maps to one cell of the violet 5×5 valid-convolution output. Applying a 3×3 filter to that receptive field requires 9 multiply-accumulate operations per input channel for each output value. For a 224×224 RGB input, parameter sharing reduces a dense 150,528-weight input connection to a 27-weight local filter, or roughly $5,575 \times$ fewer weights, while encoding translation equivariance: the same edge detector works at any image location.

Applying a CNN to 28×28 MNIST images demonstrates the difference in state representation. Rather than flattening the image into a 784-element vector that destroys spatial adjacency, each convolutional layer applies a bank of filters (for example, 32 filters of size 3×3) directly to the 2D grid. With unit stride and padding to preserve border dimensions, the layer outputs a $28 \times 28 \times 32$ activation tensor preserving the spatial grid. Each coordinate holds 32 distinct feature measurements extracted from its immediate neighborhood.

Preserving spatial topology changes how execution units access memory. Instead of loading an unshared weight matrix once per sample as in an MLP, a convolutional layer streams input feature maps through hardware registers while keeping compact filter weights pinned in fast on-chip SRAM or register files. This arithmetic pattern drives the memory-hierarchy optimizations of AI accelerators, where loop tiling and parallel channel reduction relieve high-bandwidth memory (HBM) traffic.

This data reuse pattern is guaranteed by translation equivariance: translating the input produces an identical spatial translation in the output feature map. For systems engineering, equivariance ensures that the computational schedule and memory access geometry remain invariant across spatial coordinates.

Equivariance and invariance govern how an architecture responds to spatial transformations. An operator f is *equivariant* to a transformation $\mathcal{T}$ if transforming the input transforms the output by the same operator (equation 6.4):

$$f(\mathcal{T}(\mathbf{x})) = \mathcal{T}(f(\mathbf{x})) \quad (6.4)$$

For a stride-1 convolutional layer without boundary padding effects, translating the input by vector $v = (v_1, v_2)$ shifts the resulting feature maps by exactly v . Spatial coordinate information propagates deterministically through the layer. By contrast, an operator is *invariant* to $\mathcal{T}$ if transforming the input leaves the output unchanged (equation 6.5):

$$f(\mathcal{T}(\mathbf{x})) = f(\mathbf{x}) \quad (6.5)$$

A global average pooling layer over an entire feature map exhibits translation invariance: shifting the input alters the spatial locations of activations, but averaging across all spatial positions collapses coordinates into a scalar that is identical regardless of where the features appeared.

This mathematical distinction dictates where equivariance must be maintained and where invariance should be introduced across the network hierarchy. Tasks such as object detection and semantic segmentation require equivariance through the final layer: bounding-box coordinates and per-pixel masks must track object coordinates precisely. Classification tasks, by contrast, require equivariance in intermediate feature extractors to preserve relative spatial relations (“eyes above nose”), while the final layers introduce translation invariance via pooling to collapse spatial coordinates into a class probability vector.

For an input $\mathbf{x}$ and filter $\mathbf{w}$, discrete 2D convolution evaluates

$$(\mathbf{x} * \mathbf{w})[i, j] = \sum_{m, n} \mathbf{w}[m, n] \cdot \mathbf{x}[i + m, j + n].$$

Applying translation $\mathcal{T}_v$ (where $(\mathcal{T}_v \mathbf{x})[i, j] = \mathbf{x}[i - v_1, j - v_2]$) and re-indexing the summation yields

$$((\mathcal{T}_v \mathbf{x}) * \mathbf{w})[i, j] = \sum_{m, n} \mathbf{w}[m, n] \cdot \mathbf{x}[i - v_1 + m, j - v_2 + n] = (\mathbf{x} * \mathbf{w})[i - v_1, j - v_2] = \mathcal{T}_v(\mathbf{x} * \mathbf{w})[i, j],$$

confirming translation equivariance up to boundary conditions: $f(\mathcal{T}_v \mathbf{x}) = \mathcal{T}_v(f(\mathbf{x}))$. An equivariant convolutional layer preserves coordinate information, whereas an invariant pooling layer discards spatial coordinates entirely. Example 6.4 verifies this tracking through explicit matrix operations.

Example 6.4: Equivariance: Feature detection

Setup: Consider a 7×7 image with a vertical edge at column 3:

$$\mathbf{x} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Vertical edge detector filter:

$$\mathbf{w} = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

Convolving original image:

Output feature map shows positive activation where the filter transitions from dark to bright (left side of edge) and negative activation where it transitions from bright to dark (right side):

$$f(\mathbf{x}) = \begin{bmatrix} 3 & 0 & -3 & 0 & 0 \\ 3 & 0 & -3 & 0 & 0 \\ 3 & 0 & -3 & 0 & 0 \\ 3 & 0 & -3 & 0 & 0 \\ 3 & 0 & -3 & 0 & 0 \end{bmatrix}$$

Shifted input (edge moved to column 5):

$$\mathcal{T}_2 \mathbf{x} = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

Convolving shifted image:

$$f(\mathcal{T}_2 \mathbf{x}) = \begin{bmatrix} 0 & 0 & 3 & 0 & -3 \\ 0 & 0 & 3 & 0 & -3 \\ 0 & 0 & 3 & 0 & -3 \\ 0 & 0 & 3 & 0 & -3 \\ 0 & 0 & 3 & 0 & -3 \end{bmatrix} = \mathcal{T}_2(f(\mathbf{x}))$$

Systems insight: The feature activation shifts by the same amount as the input, demonstrating equivariance. The network knows the edge is at column 5 in the shifted image, not just that an edge exists somewhere.

Parameter efficiency is the most immediate systems consequence of translation equivariance. Enforcing weight sharing decouples parameter capacity from the input resolution. For example, processing a 224×224 RGB image with an MLP requires each hidden neuron to connect to all 150,528 input pixels. A CNN with a 3×3 filter requires only 27 parameters per filter, reused across all 224×224 spatial positions. This represents approximately $5,575.1 \times$ fewer parameters per feature detector. Compacting the parameter footprint keeps model weights resident in accelerator SRAM or L2 cache, enabling larger batch sizes without spilling to external DRAM.

The computational regularity enforced by equivariance directly benefits hardware execution. Because the identical sliding window operation executes across every spatial coordinate, the compute pattern maps cleanly onto SIMD vector units and systolic arrays. Modern GPUs and TPUs exploit this uniform structure: execution pipelines issue identical instructions across spatial tiles without control-flow divergence. Overlapping receptive fields mean that neighboring output coordinates share input activations, creating opportunities for shared-memory caching and algorithmic data restructuring (such as the im2col transformation) that saturate hardware execution units.

Equivariance also increases sample efficiency, reducing training pipeline overhead. When a network updates a filter to recognize a feature at one coordinate, parameter sharing ensures that the feature is recognized across all spatial coordinates simultaneously. The model does not require separate training examples to learn the same feature at different pixel offsets, functioning as structural data augmentation. This sample efficiency reduces the total training iterations, dataset storage, and host-to-device I/O bandwidth required to reach target accuracy.



Theorem 6.1: Group equivariance formulation

From a group theory perspective, convolution's equivariance to translations represents one instance of a general principle. The translation group $(\mathbb{R}^2, +)$ consists of all 2D translations, closed under composition (translating by v then u equals translating by $v + u$). Convolution is equivariant to this group. Recent research extends this framework to other symmetry groups. Cohen and Welling (2016) developed Group-Equivariant CNNs that handle rotations and reflections by constructing filters equivariant to rotation groups. This allows learning rotation-

invariant features for tasks like satellite imagery or medical imaging where orientation does not determine meaning.

The mathematical framework generalizes cleanly: for group G acting on input space X and output space Y , a function $f : X \rightarrow Y$ is G -equivariant if:

$$f(g \cdot \mathbf{x}) = g \cdot f(\mathbf{x}) \quad \forall g \in G, \mathbf{x} \in X$$

Standard CNNs are translation-equivariant, while rotation-equivariant networks extend this to rotation groups. The architectural principle generalizes: data symmetries should be embedded as equivariances in the architecture. For systems engineering, identifying data symmetries directly informs architecture choice: more constrained architectures with stronger symmetries often produce smaller models, and specialized equivariances may require custom operations like rotation convolutions that need either hardware support or efficient software implementations.

In practice, perfect equivariance is often compromised by implementation constraints. Asymmetric padding at boundaries introduces edge artifacts that break strict translational symmetry. Strided downsampling introduces spatial aliasing, where a one-pixel shift in the input causes sub-sampling phase shifts in the output feature map. Batch normalization layers also perturb spatial equivariance if activation statistics are computed or applied across spatial locations inconsistently. Modern production networks accept these deviations because the computational savings of striding and the training stability of normalization outweigh the loss of exact mathematical equivariance.



Checkpoint 6.2: Spatial inductive bias

CNNs succeed because they match the structure of image data. Verify you understand how:

- ☐ **Parameter sharing:** how does using the same filter across the image reduce parameter count by orders of magnitude compared to MLPs?
- ☐ **Translation equivariance:** why does shifting the input image result in a corresponding shift in the feature map?
- ☐ **Compute-bound convolution:** why does a conv layer reuse each loaded filter weight across many output positions, making it compute-bound compared to other layers?

These structural properties instantiate the spatial inductive bias introduced in section 6.1: by restricting connectivity to local neighborhoods and sharing parameters across all coordinates, CNNs enforce the prior that visual primitives are localized and shift-equivariant. Reusing identical kernel weights across the entire feature map reduces parameter storage by orders of magnitude relative to dense layers, restricting the hypothesis space to functions that preserve geometric structure.

This layered structure naturally implements hierarchical representation learning (Bengio, Courville, et al. 2013). Early layers detect low-level features (edges and color gradients) across small receptive fields, while successive layers compose these primitives into intermediate textures, object parts, and semantic categories. Stacking convolutional layers expands the effective receptive field: with fixed $K \times K$ kernels and unit stride, receptive-field side length grows by $K - 1$ pixels per layer, while receptive-field area scales quadratically until it spans the entire input dimension. How efficiently accelerators execute this expanding hierarchy depends on how sliding-window stencils map onto parallel execution pipelines.

6.3.3 Computational mapping

Translating mathematical convolution into hardware execution requires bridging the gap between high-level software interfaces and silicon execution pipelines. High-level frameworks abstract convolution as a unified layer dispatch, shown in listing 6.3. Behind this concise API, the underlying runtime must schedule memory traffic across cache hierarchies and lower sliding-window stencils onto matrix execution hardware.

Listing 6.3: Convolutional Layer Abstraction: Framework-level convolution operations hide the complexity of sliding window computations, typically dispatching to cuDNN or MKL which internally handle im2col transformations or direct convolution algorithms.

```
def conv_layer_spatial(input, kernel, bias):
    """Framework-level convolution.

    Single call dispatches to optimized kernel (often via im2col + GEMM).
    """
    # Convolution applies shared weights across all positions
    # For a 3x3 kernel on 28x28 input (padded): 9 MACs per position x 784 positions
    output = convolution(input, kernel) + bias
    return activation(output)
```

To expose the mechanics that the framework abstraction conceals, listing 6.4 details the canonical seven-deep loop nest required to evaluate a direct 2D convolution. These loops partition into three structural tiers: batch iteration across samples, spatial traversal over feature map coordinates (y, x), and channel accumulation over input and output dimensions (C_{in}, C_{out}) coupled with kernel offsets (k_y, k_x). Traversing spatial coordinates and output channels defines the grid of independent output activations.

Listing 6.4: Logical Convolution Computation: Seven nested loops expose $\mathcal{O}(B \times H_{img} \times W_{img} \times C_{out} \times K_h \times K_w \times C_{in})$ complexity. While logically sound, this pattern is inefficient on hardware; production systems transform this into tiled matrix multiplications.

```
def conv_layer_compute(input, kernel, bias):
    # Logical view of convolution (usually implemented via im2col +
    # GEMM)
    # Loop 1: Process each image in batch
    for image in range(batch_size):
        # Loop 2&3: Move across image spatially
        for y in range(height):
            for x in range(width):
                # Loop 4: Compute each output feature
                for out_channel in range(num_output_channels):
                    result = bias[out_channel]
                    # Loop 5&6: Move across kernel window
                    for ky in range(kernel_height):
                        for kx in range(kernel_width):
                            # Loop 7: Process each input feature
                            for in_channel in range(num_input_channels):
                                # ... MAC operations ...
```

The inner three loops execute the kernel dot product at each spatial site. For each output activation, the kernel evaluates a local 3×3 input patch across all input channels. Executing this direct loop nest on modern hardware yields poor cache locality and uncoalesced memory accesses because neighboring spatial iterations access overlapping input patches through strided offsets.

To avoid strided memory access, production libraries like cuDNN and oneDNN typically lower convolutions via the **im2col** (image-to-column) transformation. By rearranging each receptive-field patch into a contiguous column of an input matrix, im2col recasts the entire convolution as a dense general matrix multiply (GEMM). This conversion trades memory footprint—duplicating overlapping input pixels up to K^2 times in DRAM or SRAM—for the structural regularity of GEMM kernels that saturate tensor cores and systolic arrays. Alternatively, implicit GEMM techniques generate these matrix coordinates on the fly in register files without materializing the duplicated buffer in memory. With 3×3 filters and 32 output channels, each output coordinate requires only 9 MAC operations per input channel, compared to 784 MACs in the reference MLP layer. While reducing operations per activation, the sliding window introduces distinct data reuse profiles that dictate memory staging.

6.3.4 System implications

The sliding window and im2col transformations described in section 6.3.3 reveal *how* CNNs compute; this section reveals *what that computation costs* in memory, compute, and data movement. These costs depend on layer depth, activation liveness, and whether feature maps remain cached in SRAM or spill to HBM.

6.3.4.1 Memory requirements

Memory consumption in convolutional layers divides into two distinct state components: static filter parameters and transient feature maps (activations). Weight sharing ensures that parameter storage remains compact: for an ImageNet input, a convolutional layer with 64 filters of size 3×3 applied to a single input channel requires only 576 parameters. Even across 64 input channels, storing 36,864 weights requires roughly 147 KB in FP32, fitting comfortably within on-chip L2 or L1 cache. In contrast, intermediate feature maps scale with input resolution and channel depth: a 224×224 feature map with 64 channels materializes 3.2M activation values (approximately 12.8 MB in FP32 per sample in the batch). During training, these activations must remain live in memory for backward-pass gradient evaluation, making activation memory—not weight capacity—the binding memory constraint for deep CNNs on accelerators.

Accelerators exploit this footprint disparity through hierarchical caching. CPUs retain compact filter weights in L2/L3 caches while streaming activation tensors. GPUs stage filter weights and input tiles in on-chip scratchpad SRAM (shared memory) and register files, reusing loaded weights across thousands of spatial coordinates and eliminating redundant round-trips to off-chip HBM. Hardware design principles supporting these caching patterns are detailed in chapter 11.

6.3.4.2 Computation needs

Evaluating a convolutional layer requires repeatedly evaluating local dot products across spatial positions. For an ImageNet input with 3×3 filters and 64 output channels, computing a single spatial position requires 576 multiply-accumulate operations per input channel. Repeating this evaluation across all 50,176 spatial positions yields millions of operations per layer. Although each spatial stencil touches fewer parameters than a dense layer, spatial repetition multiplies the total floating-point operation count.

This computational regularity provides rich opportunities for hardware parallelization. Because each spatial coordinate is independent, processors parallelize work along multiple axes simultaneously: across the batch dimension, across spatial coordinates (x, y) , and across output channels. CPUs execute vectorized SIMD instructions¹⁵ to evaluate multiple spatial outputs per clock cycle, while GPUs dispatch thread warps across thousands of concurrent execution lanes. Model compression techniques that alter this computational profile, such as pruning and channel sparsity, are examined in chapter 10.

6.3.4.3 Data movement

Data movement in a convolutional layer exhibits an arithmetic intensity profile fundamentally different from that of an MLP. In a fully connected layer at batch size one, each loaded weight is used exactly once, yielding an arithmetic intensity bounded below 1 FLOP/byte. In a 2D convolution, each 3×3 filter weight is reused across all 50,176 positions of the 224×224 feature map. This massive weight reuse increases operational intensity, shifting computation from the memory-bandwidth-bound regime toward the compute-bound ceiling.

The systems challenge shifts from loading weights to staging activations. Because adjacent spatial receptive fields overlap, a naive memory access pattern would re-fetch each input pixel from memory up to $K^2 = 9$ times. Runtimes avoid this redundant memory traffic by tiling computation into 2D spatial blocks sized to fit inside accelerator SRAM. Once an input tile is loaded into shared memory, all overlapping filter dot products for that tile execute in place, amortizing the DRAM/HBM transfer and maximizing the 50,176 reuses of each filter weight.

These memory, compute, and data-movement patterns converge in one model that serves as the chapter’s reference point for compute-bound vision workloads: the ResNet-50 architecture.

¹⁵ | **SIMD (single instruction, multiple data):** CPU instructions that apply the same operation to multiple data elements simultaneously; AVX-512 can process 16 single-precision values per vector instruction; realized speedup over scalar code also depends on instruction throughput, vectorization, and memory access. For CNN inference on edge CPUs without GPU access, SIMD utilization can affect whether a model meets real-time latency targets. Frameworks like TFLite and Open Neural Network Exchange (ONNX) Runtime use vectorized convolution kernels to exploit this parallelism.



Lighthouse 6.2: ResNet-50 (vision lighthouse)

Why it matters: ResNet-50 is a reference point for regular, convolution-heavy vision workloads. Its architecture consists almost entirely of dense convolutional layers, making it highly regular and efficient on GPUs. Under batched execution with good data reuse, ResNet-50 performance is typically limited by floating-point throughput (FLOP/s), making it a useful lighthouse for explaining data parallelism, quantization, and batching strategies. Table 6.4 summarizes the quantitative properties and their system consequences:

Table 6.4: ResNet-50 Systems Profile: Quantitative properties of the vision lighthouse and their system consequences. Under batched convolutional reuse, ResNet-50 has high effective arithmetic intensity and is typically compute-bound, so peak FP throughput on Tensor Cores often sets the ceiling before memory bandwidth does.

Property	Value	System Implication
Parameters	25.6M	102.4 MB model size at FP32; fits comfortably in GPU memory.
FLOPs/Image	8.2 GFLOP (224×224)	3×3 convolutions are the largest single kernel class at roughly 48% of MACs.
Constraint	Compute-heavy when batched	Limited by peak FLOP/s when weight and activation reuse are high; small-batch inference can move toward the memory-bound regime.
Bottleneck	FP Throughput	Benefits maximally from specialized Matrix Units (Tensor Cores).
Profile	High effective arithmetic intensity under reuse	Arithmetic intensity depends on batch size, convolution algorithm, and materialized memory traffic.

ResNet-50's compute-bound profile assumes abundant hardware resources, yet most inference runs on devices with power budgets three orders of magnitude smaller than a data center GPU. MobileNetV2 demonstrates that architectural innovation can target this regime, achieving competitive accuracy with a fraction of the computational cost.



Lighthouse 6.3: MobileNetV2 (efficiency lighthouse)

Why it matters: MobileNetV2 represents *latency-constrained* edge workloads. Its depthwise separable convolutions trade channel mixing capacity for speed, making it a useful baseline for mobile apps, embedded vision, and neural architecture search (NAS), automated search over model designs. Table 6.5 summarizes the efficiency lighthouse's quantitative properties:

Table 6.5: MobileNetV2 Systems Profile: Quantitative properties of the efficiency lighthouse and their system consequences. Roughly an order of magnitude smaller than ResNet-50 in both parameters and FLOPs, MobileNetV2's operation shapes and arithmetic intensity make single-image latency sensitive to operator-dispatch overhead and memory access on mobile CPUs.

Property	Value	System Implication
Parameters	3.5M	14 MB at FP32; 7.3× smaller than ResNet-50.
FLOPs/Image	600 MFLOP	13.7× fewer than ResNet-50 for similar accuracy.
Constraint	Latency Bound	Single-image inference speed is the priority.
Bottleneck	Overhead/Memory Access	Operator dispatch and memory access can dominate actual compute.
Profile	Low Arithmetic Intensity	Memory access and control logic matter more than peak FLOP/s.

The ResNet-50 and MobileNetV2 profiles expose a critical systems trade-off: a model with 13.7× fewer FLOPs does not automatically run 13.7× faster. Whether operational reduction translates into wall-clock speedup depends on arithmetic intensity, operator dispatch overhead, and the hardware's roofline balance. On data center GPUs engineered for high arithmetic intensity, MobileNetV2 often achieves only a modest speedup because its fragmented depthwise operations underutilize tensor cores.

Architectural efficiency techniques like depthwise separable convolutions and pruning (removing redundant weights or channels), detailed in chapter 10, alter this hardware mapping balance. Chap-

ter 11 examines how processor architectures exploit these spatial data reuse patterns in hardware. Perspective 6.1 addresses the central fallacy linking FLOP counts directly to execution latency.

 Systems Perspective 6.1: Misconception: FLOPs equal speed

Fallacy: “MobileNetV2 has 13.7× fewer FLOPs than ResNet-50, so it must run 13.7× faster.”

Resolution: On some high-end GPUs, MobileNetV2 can run slower than ResNet-50 despite using far fewer operations. MobileNetV2’s depthwise separable convolutions have low arithmetic intensity: they move more data relative to computation. GPUs optimized for dense matrix operations may use their compute units poorly on these kernels. FLOPs measure *work*; throughput depends on how well that work *maps to hardware*. This hardware-architecture mismatch recurs as a general fallacy in section 6.11.

6.3.5 Efficient architectures: Keyword spotting

The system implications in section 6.3.4 assume standard CNN architectures with full convolutions. Standard convolutions scale as $\mathcal{O}(N \times K^2 \times C_{\text{in}} \times C_{\text{out}})$, a compute volume that exceeds the strict energy budgets of always-on edge devices. To bridge this gap, efficient architectures like **depthwise separable CNNs (DS-CNN)** decompose the standard convolution into two distinct operations. This factorization, introduced by Sifre in the context of feature extraction (Sifre and Mallat 2014) and popularized by MobileNet (Howard et al. 2017), decouples spatial filtering from cross-channel combination. The depthwise convolution applies filters to each input channel independently ($K \times K \times C_{\text{in}}$ parameters), while the pointwise convolution uses a 1×1 convolution to project channels to the output dimension ($1 \times 1 \times C_{\text{in}} \times C_{\text{out}}$ parameters). This decomposition reduces parameter count and FLOPs to a fraction of roughly $\frac{1}{C_{\text{out}}} + \frac{1}{K^2}$ of standard convolution (yielding a reduction factor of roughly $K^2 \times$, or $\sim 8\text{--}9 \times$ for 3×3 filters with large C_{out}), making real-time execution feasible on resource-constrained microcontrollers. In keyword spotting, a continuous 1D audio stream is preprocessed via Short-Time Fourier Transform into a 2D time-frequency spectrogram (or Mel-frequency cepstral coefficients), allowing 2D convolutional kernels to slide across time and frequency bins to detect acoustic phoneme signatures.

The keyword spotting workload exemplifies this edge efficiency regime.

 Lighthouse 6.4: KWS (TinyML lighthouse)

Why it matters: Keyword spotting models (like DS-CNN) represent the *power-constrained* end of the spectrum. Used in always-on applications like smart doorbells that wake a larger vision pipeline only after an audio trigger, these models must run on microcontrollers with milliwatt power budgets.

KWS forces engineers to count every byte and cycle. It is the lighthouse for extreme quantization (INT8/INT4, detailed in chapter 10) and specialized architectural primitives (Depthwise Separable Convolutions) that trade theoretical representational power for maximum efficiency per watt.

From ResNet-50’s compute-heavy standard convolutions through MobileNet’s depthwise separable variants to KWS’s micro-power deployment, CNNs demonstrate how architectural constraints transform data structure into hardware efficiency. By matching the computation pattern to spatial locality and translation equivariance, these models maximize arithmetic intensity and minimize parameter storage. Yet the foundational assumption of the convolution operator—that spatial proximity governs feature correlation—breaks down when data relationships are ordered chronologically rather than geometrically. When inputs form sequential streams where future states depend on arbitrarily distant historical contexts, sliding spatial stencils fail to capture temporal dependencies without unmanageable kernel expansions. Processing such data requires architectures designed around sequential state evolution.

6.4 RNNs: Sequential Pattern Processing

Convolutional networks exploit *spatial* structure, where physical proximity governs statistical correlation. Many real-world signals exhibit *temporal* structure instead, where sequential order and historical context determine meaning. Processing sequences requires architectures that maintain persistent state across time steps.



Definition 6.4: Recurrent neural networks

Recurrent neural networks (RNNs) are sequence-processing architectures that maintain a hidden state $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$ updated at each time step, encoding the assumption that the current output depends on all prior inputs through this fixed-size state vector.

1. **Significance:** The fixed-size state provides $\mathcal{O}(1)$ inference memory regardless of sequence length (processing a 10,000-token sequence requires the same memory as a 10-token sequence), but the sequential update rule creates a sequential bottleneck where all S steps must execute in order, directly contributing to the L_{lat} term of the iron law and making RNNs unable to exploit GPU parallelism across the time dimension during training.
2. **Distinction:** Unlike attention mechanisms, which access the entire token history simultaneously and materialize an $\mathcal{O}(S^2)$ score matrix when processing a full sequence, RNNs compress history into a bottleneck state, meaning gradient signal must propagate back through all S steps—causing $\partial \mathcal{L} / \partial \mathbf{h}_0 \propto \prod_{t=1}^S \partial \mathbf{h}_t / \partial \mathbf{h}_{t-1}$, a product of S Jacobians that vanishes or explodes exponentially with sequence length.
3. **Common pitfall:** A frequent misconception is that RNNs are obsolete. For streaming inference on resource-constrained hardware where $\mathcal{O}(S^2)$ attention memory is prohibitive, such as keyword spotting on a microcontroller, an RNN's $\mathcal{O}(1)$ state size remains the systems-justified choice.

Feedforward architectures and standard CNNs treat inputs independently or constrain context to a fixed spatial receptive field. In sequential domains, however, dependencies span variable and often unbounded temporal distances. Expanding a convolutional receptive field to cover long sequences requires either stacking dozens of layers or applying dilated kernels, both of which scale parameter count and intermediate activation storage regardless of whether the intervening tokens contain relevant information.

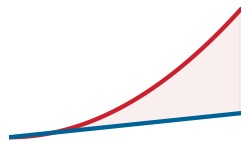
Recurrent architectures, from simple recurrent networks to gated variants such as long short-term memory (LSTM) networks (Elman 1990; Hochreiter and Schmidhuber 1997), address this constraint by introducing a temporal inductive bias: execution order encodes causal direction, and history compresses into a recurring state. Instead of processing each input vector in isolation, the network carries an internal state vector across successive time steps. This inductive bias establishes an explicit engineering trade-off: it provides variable-length temporal context within a fixed memory budget, but it introduces loop-carried data dependencies that serialize execution on parallel accelerator hardware.

6.4.1 Pattern processing needs

In sequential pattern processing, the interpretation of an incoming token depends on the preceding sequence. For example, polysemous words such as “bank” cannot be disambiguated in isolation: “river bank” specifies a geological shoreline, whereas “bank account” specifies a financial institution. The word’s identity alone is insufficient; its semantic role is resolved by preceding context. Acoustic waveforms and telemetry time series exhibit identical properties: individual samples are ambiguous without historical context spanning multiple frequencies and timescales.

Data with this structure imposes three concrete requirements on the underlying model:

1. **Variable-length ingestion:** Sequences arrive with non-uniform lengths S , precluding fixed-dimension weight tensors that require static input shapes.
2. **Contextual persistence:** The model must retain informative historical signals over arbitrary time lags without requiring explicit windowing.



RNNs hold state memory fixed, but latency grows with sequence length.

3. **Continuous state update:** Internal representations must update dynamically as each new token arrives, integrating new evidence while discarding obsolete or noisy history.

MLPs cannot satisfy these requirements because their input layer dimensions are statically fixed at compilation time, requiring zero-padding or truncation to a rigid maximum sequence length. CNNs process inputs through static sliding windows, requiring receptive fields to grow linearly or exponentially with network depth. An architecture built specifically for sequential pattern processing must make state persistence an intrinsic property of its dataflow.

6.4.2 Algorithmic structure

To meet the requirements of section 6.4.1, recurrent neural networks introduce directed cycles in their computational graph. Rather than mapping inputs directly to outputs through feedforward layers, an RNN maintains a persistent vector—the **hidden state**—that updates iteratively at each time step. The hidden state acts as an internal register, summarizing past observations to contextualize future predictions.

The fundamental state transition in equation 6.6 computes the current hidden state $\mathbf{h}_t$ by combining the incoming token vector $\mathbf{x}_t$ with the preceding state $\mathbf{h}_{t-1}$:

$$\mathbf{h}_t = f(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h) \quad (6.6)$$

where $\mathbf{h}_t$ denotes the hidden state at time t , $\mathbf{x}_t$ denotes the input at time t , $\mathbf{W}_{hh}$ contains the recurrent weights, $\mathbf{W}_{hx}$ contains the input weights, $\mathbf{b}_h$ is the hidden-state bias vector, and f is the activation function. The equation uses column vectors; the later listings use row-major batched tensors, so their weight multiplications appear on the right. Compare the left and right panels of figure 6.5: the left panel shows the compact recurrent loop, while the right panel unfolds it across time steps, making explicit the temporal dependencies that this recurrence creates.

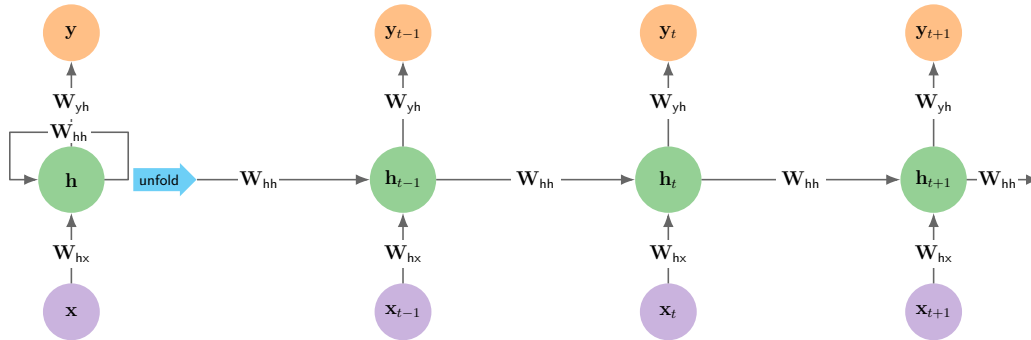


Figure 6.5: Recurrent Neural Network Unfolding: Left panel shows the compact recurrent loop; right panel unfolds this across time steps. Three weight matrices are shared across all steps: $\mathbf{W}_{hx}$ (input-to-hidden), $\mathbf{W}_{hh}$ (hidden-to-hidden), and $\mathbf{W}_{yh}$ (hidden-to-output). These matrices are reused at every step, so the parameter count is independent of sequence length. For a 128-dimensional hidden state and a 100-dimensional input, each time step requires 16,384 MACs for recurrent connections plus 12,800 for input projection.

Unfolding the recurrent loop across S steps reveals the algorithmic properties of recurrence. Parameter count remains independent of sequence length S because the weight matrices $\mathbf{W}_{hh}$ and $\mathbf{W}_{hx}$ are shared across all time steps. However, sharing weights across time establishes a loop-carried dependency: step t cannot evaluate $\mathbf{h}_t$ until step $t-1$ finishes producing $\mathbf{h}_{t-1}$. This dependency forms a critical path of length $\mathcal{O}(S)$ serial operations that prevents hardware accelerators from parallelizing across the sequence dimension.

This sequential structure produces fundamentally different memory characteristics between inference and training. During streaming inference, an RNN evaluates one step at a time, discarding prior raw inputs and retaining only the current hidden state. State memory overhead remains strictly $\mathcal{O}(d_{\text{hidden}})$ regardless of whether the sequence spans 10 or 10,000 steps. In contrast, training via backpropagation through time (BPTT) unrolls the graph across the entire sequence of length

S. Computing gradients via the chain rule requires evaluating the Jacobian product across all intermediate transitions:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_0} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_S} \prod_{t=1}^S \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}}$$

To evaluate these Jacobian products, the training engine must retain the intermediate activations $\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_S$ in accelerator memory. BPTT thus converts an $\mathcal{O}(d_{\text{hidden}})$ runtime state into an $\mathcal{O}(S \cdot d_{\text{hidden}})$ memory footprint during training. Furthermore, repeated multiplication by $\mathbf{W}_{\text{hh}}$ causes gradients to vanish or explode exponentially over long horizons, bounding the effective context window of simple RNNs. The recurrent weight matrix often contains connections with minimal contribution to temporal dependencies, allowing significant compression through methods covered in chapter 10.

6.4.3 Computational mapping

Translating recurrent equations into hardware execution reveals how mathematical abstractions map to accelerator kernels, memory hierarchies, and loop nests. Where convolutional layers exploit spatial locality and weight reuse across tiles (section 6.1), recurrent steps decompose into matrix-vector or matrix-matrix multiplications bound by dataflow serialization.

Listing 6.5 demonstrates the single-time-step mechanism using framework-level matrix operations: combine the previous hidden state with the current input, add the bias, and apply the activation to produce the next hidden state. The code is intentionally local to one step because the systems cost is not the step itself, but the dependency chain that prevents parallel execution across time.

Listing 6.5: RNN Layer Abstraction: Framework-level implementation combining two matrix multiplications, $\mathbf{h}_{t-1} \mathbf{W}_{\text{hh}}$ and $\mathbf{x}_t \mathbf{W}_{\text{hx}}$, per time step. For 128-dimensional hidden state and 100-dimensional input, each step requires $16,384 + 12,800 = 29,184$ MACs, but sequential dependencies prevent parallelization across time.

```
def rnn_layer_step(x_t, h_prev, W_hh, W_hx, b):
    # x_t: input at time t (batch_size × input_dim)
    # h_prev: previous hidden state (batch_size × hidden_dim)
    # W_hh: recurrent weights (hidden_dim × hidden_dim)
    # W_hx: input weights (input_dim × hidden_dim)
    h_t = activation(matmul(h_prev, W_hh) + matmul(x_t, W_hx) + b)
    return h_t
```

At the framework level, `rnn_layer_step` hides the underlying loop structure and memory traffic behind high-level matrix multiplications (`matmul`). In hardware, evaluating this step requires moving weights and activations through the memory hierarchy. To see where execution serializes and where parallelism remains, listing 6.6 expands the matrix operations into explicit loop nests down to scalar multiply-accumulate operations.

Listing 6.6: RNN Layer Computation: Nested loops expose the sequential dependency structure. Loop 1 enables batch parallelism, but Loops 2-3 must complete before Loop 4’s activation, and crucially, each time step depends on the previous step’s output, creating $\mathcal{O}(S)$ sequential depth that prevents GPU parallelization across the time dimension.

```
def rnn_layer_compute(x_t, h_prev, W_hh, W_hx, b):
    # Initialize next hidden state
    h_t = np.zeros_like(h_prev)

    # Loop 1: Process each sequence in the batch
    for batch in range(batch_size):
        # Loop 2: Compute recurrent contribution ( $h_{prev} \times W_{hh}$ )
        for i in range(hidden_dim):
            for j in range(hidden_dim):
                h_t[batch, i] += h_prev[batch, j] * W_hh[j, i]

        # Loop 3: Compute input contribution ( $x_t \times W_{hx}$ )
        for i in range(hidden_dim):
            for j in range(input_dim):
                h_t[batch, i] += x_t[batch, j] * W_hx[j, i]

        # Loop 4: Add bias and apply activation
        for i in range(hidden_dim):
            h_t[batch, i] = activation(h_t[batch, i] + b[i])

    return h_t
```

The loop structure exposes the concurrency boundaries of recurrent execution. Loop 1 demonstrates that batch instances are completely independent: sequences within a batch parallelize across accelerator cores without cross-talk. Within each batch instance, Loops 2 and 3 execute the recurrent transformation ($\mathbf{h}_{t-1} \mathbf{W}_{hh}$) and input projection ($\mathbf{x}_t \mathbf{W}_{hx}$). Both reductions must accumulate fully before Loop 4 can apply the elementwise bias addition and activation function to produce $\mathbf{h}_t$.

For an input dimension of 100 and a hidden state dimension of 128, each step executes two matrix multiplications: a 100×128 input projection and a 128×128 recurrent transformation. When batch size is large ($B \gg 1$), these operations execute as general matrix-matrix multiplications (GEMM), achieving high arithmetic intensity as weights loaded from memory are reused across all B samples. When batch size is small ($B = 1$, typical of real-time streaming inference), the operations collapse into matrix-vector multiplications (GEMV). In GEMV, every weight parameter must be fetched from memory to perform a single multiply-accumulate, dropping arithmetic intensity to ≈ 1 FLOP/byte and starving accelerator compute units.

6.4.4 System implications

Recurrent architectures impose a hard physical constraint: *sequential dependency*. Unlike MLPs and CNNs, where hardware can parallelize execution across neurons, channels, and spatial pixels, an RNN cannot parallelize across the sequence dimension. The update $\mathbf{h}_t = \tanh(\mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{W}_{hx} \mathbf{x}_t)$ establishes a strict ordering: time step t cannot begin until step $t-1$ completes and writes its output. For an input sequence of length $S = 1,000$, the system must launch 1,000 sequential matrix multiplications. Scaling hardware compute capability or memory bandwidth accelerates each individual step, but Amdahl’s law dictates that latency is bounded by the serialized critical path. Parallel scaling is therefore strictly restricted to the batch dimension.

During inference, recurrent networks achieve exceptional memory efficiency. Because the recurrent state compresses history into a fixed-size vector $\mathbf{h}_t$, inference state memory scales as $\mathcal{O}(d_{\text{hidden}})$ (for example, 2 KB for a 512-dimensional FP32 state), remaining completely flat whether sequence length S is 10 or 10,000 tokens. This constant memory footprint contrasts sharply with attention mechanisms (section 6.5), where full-sequence attention materializes an $\mathcal{O}(S^2)$ score matrix during prefill and autoregressive generation maintains an $\mathcal{O}(S \cdot d_{\text{model}})$ key-value cache. For streaming deployments on memory-constrained edge hardware, such as microcontrollers, an RNN’s $\mathcal{O}(1)$ runtime state eliminates the memory-capacity exhaustion that limits transformers.

Recurrent execution exhibits distinct memory-hierarchy behaviors: high temporal locality for parameters, but poor operational intensity under standard framework dispatch. Weight matrices

W_{hh} and W_{hx} are reused at every time step, residing comfortably in SRAM or L2 cache if the model fits on-chip. However, naively launching separate kernels for input projection, recurrent projection, bias addition, and activation at each time step incurs severe kernel-launch overhead and forces intermediate tensors back and forth across HBM. To mitigate this memory-bandwidth bottleneck, production recurrent runtimes fuse the projections, bias addition, and activation into a single monolithic kernel that maintains h_t within GPU registers and shared memory throughout the sequence execution.

This architectural tension between fixed-capacity state and serialized execution defined sequence processing before transformers. While recurrent networks maintain constant memory overhead regardless of sequence length, the sequential dependency prevents hardware from parallelizing across time steps, and the fixed-capacity state becomes an information bottleneck where early inputs fade over long spans (the vanishing gradient problem). Overcoming both constraints required an architecture capable of establishing direct, content-addressable links across arbitrary sequence positions without intervening recurrent steps: the attention mechanism (section 6.5). For deployed recurrent pipelines, mitigation techniques such as operator fusion and pipeline parallelism are analyzed in section 11.8.

6.5 Attention: Dynamic Processing

Recurrent networks serialize execution along the sequence dimension and compress all historical context into a fixed-capacity hidden state vector (section 6.4.4). In a sentence such as “The cat, which was sitting by the window overlooking the garden, was sleeping,” the subject “cat” and predicate “sleeping” are separated by nine intervening words, yet they form the core syntactic and semantic relationship. An RNN must process all intervening tokens sequentially, risking information loss across intermediate recurrent transitions while preventing hardware from parallelizing across time steps. This constraint motivates an alternative computational pattern: an operation that directly computes relevance between any two positions regardless of distance.

Attention mechanisms¹⁶ address this limitation (Bahdanau et al. 2015) by introducing dynamic connectivity patterns that adapt to input content. Rather than forcing all context through a single fixed-capacity state vector, encoder-decoder attention computes similarity scores between each decoder state and all encoder input positions, routing information dynamically across the sequence.



Definition 6.5: Attention mechanisms

Attention mechanisms are neural network operations that compute a weighted sum of value vectors, where the weights are derived from learned similarity scores between a query vector and a set of key vectors, enabling dynamic, content-dependent information routing between any two positions in a sequence.

1. **Significance:** Attention connects any two tokens in $\mathcal{O}(1)$ depth but computes S^2 score interactions. If those scores are materialized, a 4,096-token sequence with 16-bit scores consumes 33.6 MB per layer per head (about 16.8M scores at 2 bytes each), directly increasing the iron law’s D_{vol} and BW terms. Tiled exact-attention kernels avoid retaining the full matrix, but the dense score computation remains quadratic.
2. **Distinction:** Unlike RNNs, which compress all prior context into a single fixed-size state vector, attention mechanisms retain token representations and compute relevance scores directly. During training or full-sequence prefill, the initial pass that processes the whole prompt, score interactions grow quadratically with sequence length; during autoregressive serving, the stored KV cache, the saved key and value vectors from prior tokens, grows as $\mathcal{O}(Sd_{model})$ while each new token attends over prior keys and values.
3. **Common pitfall:** A frequent misconception is that attention is a general-purpose weighting scheme that can be applied freely. Quadratic score computation is a hard scaling constraint: doubling the context quadruples score interactions. Naive implementations also quadruple score storage; tiled exact algorithms such as FlashAttention avoid the full matrix, while sparse variants reduce the scores computed.

¹⁶ | **Bahdanau attention:** This approach broke the “fixed-length vector” bottleneck of prior sequence-to-sequence models by allowing a decoder to dynamically query all input elements at each output step, creating the adaptive connectivity described. This replaced the structural constraint of a fixed-capacity channel with a learned, content-based weighting system. The core trade-off was accepting a linear, $\mathcal{O}(S)$ memory cost to store all input states in exchange for overcoming the information loss inherent in a single vector.

While attention mechanisms were initially used as components within recurrent architectures, their ability to connect any position to any other made the recurrent structure unnecessary for many sequence tasks. The transformer¹⁷ architecture (Vaswani et al. 2017) combined attention with feed-forward layers, residual connections, normalization, and positional information without recurrence. This architectural shift traded the RNN's $\mathcal{O}(S)$ sequential path for constant path length between positions within one attention layer, enabling parallelization across sequence positions on high-throughput accelerators.

The transformer architecture (section 6.6) inherits its core systems properties from attention: content-dependent routing, parallel sequence execution, and quadratic score scaling. Analyzing the systems implications of this architectural class begins with the underlying pattern-processing problem attention solves.

6.5.1 Pattern processing needs

Dynamic pattern processing addresses scenarios where relationships between elements are not fixed by architecture but instead emerge from content. Language translation exemplifies this challenge: when translating “the bank by the river,” understanding “bank” requires attending to “river,” but in “the bank approved the loan,” the important relationship is with “approved” and “loan.” Unlike RNNs that process information sequentially or CNNs that use fixed spatial stencils, dynamic sequence processing requires an operation that evaluates relationships based on token identity. The pronoun-resolution schematic in figure 6.6 makes that dynamic routing visible.

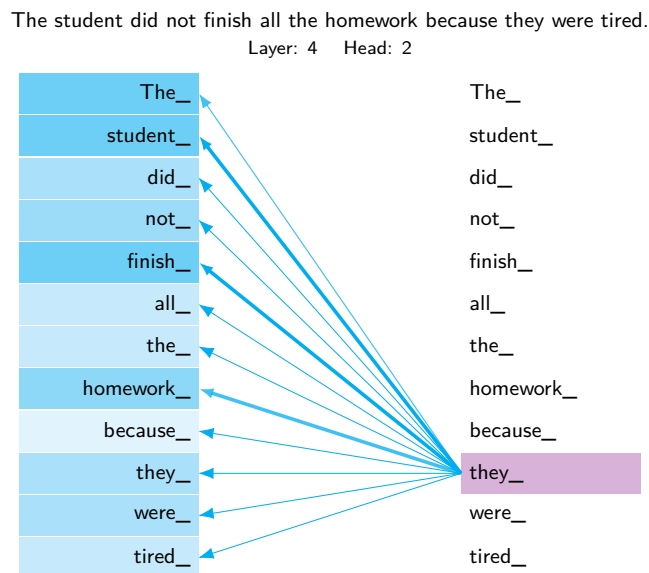


Figure 6.6: Illustrative Attention Pattern: A schematic attention pattern for the pronoun “they.” Line thickness represents hypothetical attention-weight magnitude, with stronger links to “student,” “finish,” and “homework.” One attention layer connects all positions directly and can compute their dense interactions in parallel.

In figure 6.6, the attention mechanism resolves the ambiguous pronoun “they” by assigning larger weights to relevant context tokens such as “student” and “finish,” as indicated by line thickness. Because all pairwise connections are evaluated in parallel, the network captures this long-range syntactic dependency in a single layer without propagating intermediate state across intervening tokens.

Input-dependent processing extends well beyond language. In protein structure prediction, amino-acid interactions depend on chemical properties and spatial arrangement, not only chain position. In graph analysis, graph convolutional networks (GCNs) aggregate features over each node’s observed neighbors (Kipf and Welling 2017). Unlike CNNs, which access memory in regular spatial strides, or transformers, which work with dense sequence tensors, GCN neighbor aggregation follows the irregular adjacency structure of the input graph. Each gather step touches an input-specific set of

¹⁷ | **Transformer:** The founding paper, “Attention Is All You Need,” made the explicit systems claim that a parallel attention mechanism could fully replace sequential recurrent processing. This architectural trade eliminates an RNN’s $\mathcal{O}(S)$ path length constraint on parallelism but introduces $\mathcal{O}(S^2)$ dense score computation; naive implementations also materialize $\mathcal{O}(S^2)$ score storage. This quadratic computation continues to shape context-window engineering.

node embeddings, defeating cache prefetchers and preventing coalesced memory access. Irregular neighbor-gather patterns can therefore bind execution on memory bandwidth and cache misses rather than compute throughput. In document analysis, connections between sections depend on semantic content rather than proximity.

Input dependence does not imply all-to-all connectivity. Dense attention scores every pair of sequence elements, whereas a GCN restricts aggregation to edges in the observed graph. Input-specific graphs can require different gather schedules and exhibit different locality. The common systems challenge is that the computation or memory-access pattern responds to the input rather than following a fixed stencil. Even when tensor dimensions stay fixed, content-dependent scores determine which information is emphasized. Attention provides one mechanism for learning such relationships, and dense self-attention forms the foundation of the transformer architecture.

6.5.2 Algorithmic structure

Meeting the dynamic routing requirements of section 6.5.1 requires a formal operator that maps variable-length sequences to content-dependent affinity weights (Bahdanau et al. 2015). The transformer architecture implements this mapping via the scaled dot-product attention operator introduced by Vaswani et al. (2017):

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} \right) \mathbf{V}$$

The inputs $\mathbf{Q}$ (queries), $\mathbf{K}$ (keys), and $\mathbf{V}$ (values)¹⁸ are learned linear projections of the token representations. For an input sequence of length S projected to dimension d_k , the dot product $\mathbf{Q}\mathbf{K}^T$ computes all S^2 pairwise token similarities (section C.1.3 formalizes the dot product as a similarity measure). Scaling by $1/\sqrt{d_k}$ prevents dot products from growing large in magnitude at high dimensions, which would push softmax into regions with vanishingly small gradients. The softmax operator¹⁹ normalizes each row of the score matrix into a probability distribution over key positions. Finally, multiplying these normalized weights by $\mathbf{V}$ produces output vectors formed as weighted combinations of the value representations.

Figure 6.7 illustrates this computational flow for a sequence of length $S = 6$. Each row of the resulting $S \times S$ matrix determines how strongly an individual token attends to every token in the sequence. In a trained model, the largest affinity values reflect strong contextual dependencies, while near-zero entries suppress irrelevant positions.

While attention weights vary dynamically with the input tokens, generating the query, key, and value vectors requires static linear projections against learned parameter matrices. In accelerator implementations, these three operations are fused into a single batched matrix multiplication to maximize tensor core saturation and eliminate kernel launch overhead. Figure 6.8 traces this combined projection: multiplying the 6×768 token embedding matrix by a packed 768×2304 weight matrix yields concatenated query, key, and value representations in one kernel pass.

6.5.3 Computational mapping

Lowering the attention operator $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k})\mathbf{V}$ to physical execution exposes the system cost of data-dependent connectivity. Evaluating pairwise alignments requires two distinct matrix multiplications per attention head: the score GEMM ($\mathbf{Q}\mathbf{K}^T$) and the context GEMM ($\mathbf{AV}$, where $\mathbf{A} = \text{softmax}(\mathbf{S})$). For an input of sequence length S and head dimension d_k , each phase executes $S^2 d_k$ multiply-accumulate operations, yielding $\mathcal{O}(S^2 d_k)$ computational complexity per head.

Listing 6.7 contrasts the high-level matrix formulation with an explicit loop implementation that exposes this quadratic scaling. The matrix version lowers to optimized GEMM routines on accelerator tensor cores, whereas the loop version reveals the underlying physical execution structure: batch iterations, quadratic query-key inner products, row-wise softmax reductions, and the final value accumulation pass across all sequence positions.

¹⁸ | **Query-key-value (QKV):**

The terminology is borrowed from information retrieval, explaining why the equation uses three distinct learned projections to calculate pairwise scores. Creating these projections requires three independent weight matrices, costing $3 \times d_{\text{model}}^2$ parameters per layer. The direct systems consequence is the “KV cache” for autoregressive inference: all prior key and value vectors must be stored for the next token, causing memory to grow linearly ($\mathcal{O}(S)$) with sequence length and potentially dominate serving memory at long contexts or high concurrency.

¹⁹ | **Softmax:** Named as a

“soft” (differentiable) version of argmax, with mathematical roots in Boltzmann’s statistical mechanics, softmax normalizes each query row over all S scores; a naive attention implementation materializes the full $S \times S$ matrix before normalization. Online softmax and tiling, as used by FlashAttention, instead maintain running normalization statistics and compute exact attention without retaining that full matrix in HBM. Softmax therefore requires a reduction over each row, but it does not force quadratic auxiliary storage; the dense pairwise score computation remains quadratic.

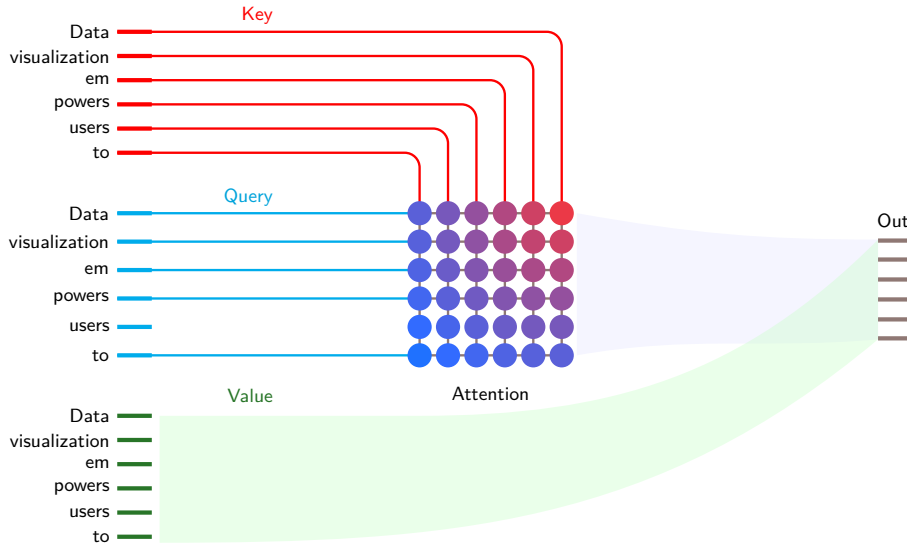


Figure 6.7: Query-Key-Value Attention Mechanism: For a 6-token sequence, queries (cyan) match against keys (red) to produce a 6×6 attention matrix with $\mathcal{O}(S^2)$ entries. Cell color runs from blue to red across the grid as an illustrative gradient rather than as computed weights. Each output position aggregates information from all values (green) weighted by its own row of that matrix. The matrix structure reveals both the computational pattern (thirty-six similarity computations) and the naive memory bottleneck (storing S^2 attention weights). Source: Transformer Explainer (Cho et al. 2025).

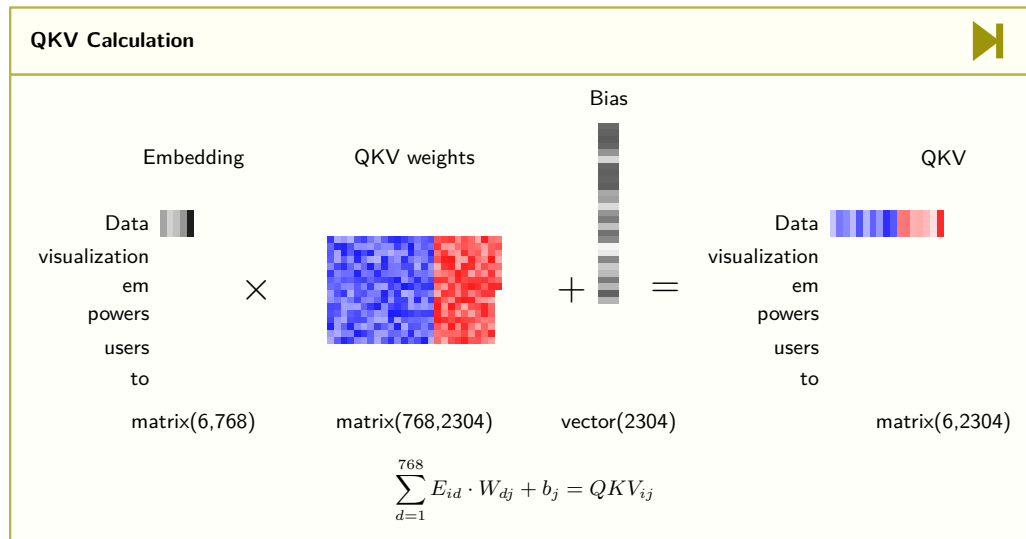


Figure 6.8: QKV Projection Computation: The embedding matrix (6×768) multiplies with QKV weight matrices (768×2304) plus bias to produce combined projections (6×2304). The 2304 output dimension contains concatenated query, key, and value projections (each 768-dimensional). This single batched matrix multiplication, requiring $6 \times 768 \times 2304 = 10.6$ million MACs, replaces 3 separate projection operations for efficiency. Source: Transformer Explainer (Cho et al. 2025).

Listing 6.7: Attention Computation: Two implementations showing the same $\mathcal{O}(S^2 \times d)$ complexity. The matrix form (top) uses optimized GEMM, while the nested loops (bottom) expose the quadratic pairwise comparisons: for sequence length 512 and dimension 64, computing attention scores requires $512 \times 512 \times 64 = 16.8\text{M}$ MACs per attention head, plus another 16.8M MACs for value aggregation.

```
def attention_layer_matrix(Q, K, V):
    # Q, K, V: (batch_size * seq_len * d_model)
    # Compute attention scores
    scores = matmul(Q, K.transpose(-2, -1)) / sqrt(d_k)
    weights = softmax(scores) # Normalize scores
    output = matmul(weights, V) # Combine values
    return output
```

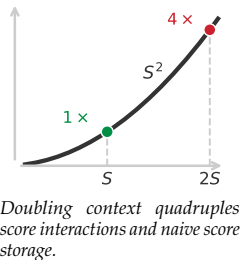
```
def attention_layer_compute(Q, K, V):
```

6.5.4 System implications

Attention mechanisms exhibit distinctive system-level patterns that differ from previous architectures through their dynamic connectivity requirements. In iron law terms (section 1.7), attention shifts the bottleneck from the latency-bound sequential path of RNNs toward quadratic score interactions and implementation-dependent data movement. Naive implementations materialize an $\mathcal{O}(S^2)$ attention matrix, while tiled algorithms such as FlashAttention avoid storing the full matrix by recomputing and streaming blocks through faster memory (see section C.5 for the step-by-step online softmax recurrence and I/O reduction derivation).

6.5.4.1 Memory requirements

Attention mechanisms require storage for query-key-value projections and intermediate feature representations. A naive implementation also materializes an $S \times S$ attention-weight matrix for every sequence and head, creating a quadratic memory bottleneck alongside the $S \times d$ inputs and outputs. For long sequences, these transient scores can dominate accelerator memory even though the learned projection matrices remain fixed in size. Tiled exact-attention kernels such as FlashAttention stream score blocks through faster memory and avoid retaining the complete matrix in HBM, while still computing S^2 dense interactions. This distinction separates an unavoidable quadratic compute cost for dense attention from optional quadratic score storage. Without kernel tiling, materializing intermediate score matrices quickly exhausts accelerator high-bandwidth memory at production sequence lengths.



Napkin Math 6.1: The quadratic bottleneck

Problem: How much memory does a materialized attention matrix require at sequence length $S = 100,000$ (context window)?

Math:

1. **Matrix size:** The attention score matrix ($\mathbf{QK}^T$) has dimensions $S \times S$ per head, across N_{heads} heads (12 heads in this example).
2. **Elements:** $100,000 \times 100,000 \times 12 \text{ heads} = 1.2 \times 10^{11}$ elements.
3. **Memory:** At FP16 (2 bytes/element): $1.2 \times 10^{11} \times 2 \text{ bytes} = 240 \text{ GB}$.
4. **Retained layers:** $240 \text{ GB per layer} \times 32 \text{ retained layers} = 7,680 \text{ GB}$.

Systems insight: A materialized attention matrix consumes 240 GB per layer. Naive training that retains all 32 layers' scores for backward would require 7,680 GB, far exceeding any single GPU's capacity. This memory wall motivates two broad implementation strategies developed later: avoid materializing the full matrix by tiling the computation, or reduce the number of scores computed in the first place.

6.5.4.2 Computation and data movement

Attention computation splits into two primary GEMM phases interleaved by a row-wise reduction: generating pairwise query-key scores and computing weighted sums over values. Query-key inner products require $B \times H \times S^2 \times d_k$ MAC operations, mirrored symmetrically by the value-aggregation stage. The intervening softmax requires row-wise maximum finding, exponentiation, and summation. Unlike static weight convolutions where filter weights are reused across fixed grid positions, attention computes transient, input-dependent affinity matrices that must be refreshed for every token configuration.



Checkpoint 6.3: Quadratic scaling intuition

Long-context scaling is shaped by the cost of attention. Verify your intuition:

- ☐ **Quadratic growth:** can you explain why doubling sequence length quadruples dense score interactions and a materialized score matrix?

- **Tiling boundary:** can you explain why long-context dense attention remains more expensive even when tiling removes full-matrix storage?

Data movement in attention creates memory access patterns distinct from fixed-stencil operations. Evaluating attention requires projecting and streaming query, key, and value tensors, then routing value vectors according to dynamically evaluated scores. In naive implementations, staging the intermediate $S \times S$ score matrix in global HBM/DRAM turns score materialization into the dominant bandwidth bottleneck. While static convolutions benefit from deterministic cache lines and RNNs reuse weights across time, naive attention generates massive, short-lived activation footprints that challenge standard on-chip cache capacities.

Example 6.5: The quadratic wall

Scenario: Early BERT deployments enforced rigid 512-token context sequence limits during inference and prefill (Devlin et al. 2019).

Diagnosis: Standard self-attention materializes an $S \times S$ score matrix scaling quadratically ($\mathcal{O}(S^2)$) in memory. Increasing sequence length from 512 to 4,096 increases score memory by 64×, triggering GPU out-of-memory crashes.

Systems lesson: Super-linear memory scaling turns algorithmic complexity into hardware bottlenecks. Serving long sequences requires FlashAttention IO-tiling, sparse attention, or strict token truncation to bound SRAM activation footprint.

Attention replaces the serialized hidden-state bottleneck of recurrent models with dynamic, constant-depth routing, but shifts the system burden to quadratic arithmetic scaling and high memory bandwidth traffic. When combined with feed-forward layers, residual connections, and normalization, this primitive enables end-to-end parallel sequence processing on modern accelerators. The transformer architecture (section 6.6) builds directly on this foundation, establishing the dominant architectural paradigm for modern large-scale machine learning systems.

6.6 Transformers: Parallel Sequence Processing

Attention provides the computational primitive of dynamic, content-dependent routing between positions, yet it was originally layered on top of recurrent architectures, inheriting their sequential bottleneck. The **transformer architecture** removes recurrence by combining attention with feed-forward layers, residual connections, normalization, and positional information. This enables parallel computation across sequence positions during full-sequence processing while retaining dynamic connectivity. This architectural shift introduces distinct physical costs: quadratic dense-score computation during prefill, accumulating key-value state during serving, and severe memory-bandwidth pressure during autoregressive generation.

Definition 6.6: Transformers

Transformers are neural network architectures that combine self-attention, feed-forward layers, residual connections, normalization, and positional information without recurrence, enabling parallel processing across sequence positions during training and prefill.

1. **Significance:** Parallelism is the systems payoff. An RNN processing an S -token sequence executes S sequentially dependent steps. A transformer processes all positions of a sequence concurrently, mapping token interactions to large matrix multiplications (GEMMs) that maximize accelerator compute utilization. Dense self-attention computes $\mathcal{O}(S^2)$ pairwise score interactions, and naive implementations materialize an $\mathcal{O}(S^2)$ intermediate activation tensor in high-bandwidth memory (HBM).

2. **Distinction:** Unlike attention used inside a recurrent backbone, which inherits the host architecture's $\mathcal{O}(S)$ sequential path, transformers use attention as the primary mixing operation between positions, alternating it with position-wise feed-forward networks.
3. **Common pitfall:** A frequent misconception is that transformers have “infinite context.” Context length is bounded by two distinct memory footprints: the $\mathcal{O}(S^2)$ attention-score activation matrix during training and prefill, and the $\mathcal{O}(S)$ key-value (KV) cache that accumulates during autoregressive serving. At long sequence lengths, the KV cache footprint can rival or exceed the static model weights, turning KV cache management and memory-bandwidth efficiency into dominant engineering constraints.

6.6.1 Pattern processing needs

Attention mechanisms first appeared as additions to recurrent sequence-to-sequence architectures (Sutskever et al. 2014; Bahdanau et al. 2015). Those hybrid models improved dynamic connectivity across long contexts, but preserved the recurrent state-update loop: step t remained strictly dependent on step $t - 1$, preventing concurrent execution across sequence positions on parallel hardware. Eliminating recurrence requires replacing sequential state propagation with all-to-all sequence interactions, trading bounded per-step memory for parallel accelerator execution.

The transformer architecture (Vaswani et al. 2017) establishes self-attention as the primary mixing primitive across positions, paired with positional encodings to inject sequence order. This architectural choice trades the spatial parameter efficiency of CNNs and the bounded $\mathcal{O}(1)$ step state of RNNs for unconstrained sequence parallelism during training and prefill.

6.6.2 Algorithmic structure

The architectural shift in transformers centers on self-attention layers. Unlike encoder-decoder attention where queries originate from the target sequence and keys/values from the source, self-attention derives queries, keys, and values from the identical input sequence $\mathbf{X}$. Making all three projections self-referential allows the model to resolve intra-sequence dependencies across arbitrary token distances without intermediate recurrent steps. Equation 6.7 formalizes this transformation:

$$\text{SelfAttention}(\mathbf{X}) = \text{softmax} \left(\frac{\mathbf{X}\mathbf{W}_Q(\mathbf{X}\mathbf{W}_K)^T}{\sqrt{d_k}} \right) \mathbf{X}\mathbf{W}_V \quad (6.7)$$

Here, $\mathbf{X}$ is the input sequence, and $\mathbf{W}_Q$, $\mathbf{W}_K$, and $\mathbf{W}_V$ are learned weight matrices for queries, keys, and values respectively. This formulation highlights how self-attention derives all its components from the same input, creating a dynamic, content-dependent processing pattern. Because matrix multiplication and row-wise softmax operate symmetrically across the token dimension, self-attention is strictly permutation-equivariant: shuffling the rows of $\mathbf{X}$ merely shuffles the output rows identically, treating the sequence as an unordered set. Positional encodings—whether added as sinusoidal vectors or applied via rotary position embeddings—are therefore injected into the representations to break this symmetry and encode word order and relative distance.

Transformers extend single-head self-attention by executing multiple attention operations in parallel through multi-head attention. Instead of computing a single set of attention weights over d_{model} dimensions, the architecture projects queries, keys, and values into N_{heads} distinct lower-dimensional subspaces of dimension $d_k = d_{\text{model}}/N_{\text{heads}}$. Each head attends to different representation patterns simultaneously. Their outputs are concatenated and projected through an output matrix $\mathbf{W}^O$, as formalized in equation 6.8:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_{N_{\text{heads}}})\mathbf{W}^O \quad (6.8)$$

where each attention head is computed as:

$$\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V)$$

The scaling factor $\sqrt{d_k}$ enforces numerical and gradient stability. Assuming query and key components are independent random variables with zero mean and unit variance, their inner product

$\mathbf{q}_i \cdot \mathbf{k}_j = \sum_{m=1}^{d_k} q_{im} k_{jm}$ has zero mean and variance d_k . Without scaling, large head dimensions push dot products into regions where the softmax function saturates with near-zero gradients. Dividing by $\sqrt{d_k}$ normalizes the variance to one, maintaining stable gradients and enabling effective learning.²⁰

Conceptually, attention implements a differentiable content-addressable memory. Whereas conventional memory architectures retrieve stored values at explicit addresses, self-attention uses the query vector $\mathbf{q}_i$ to score similarity against all candidate keys $\mathbf{K} \in \mathbb{R}^{S \times d_k}$. The resulting softmax distribution generates a continuous interpolation over the value vectors $\mathbf{V} \in \mathbb{R}^{S \times d_v}$, functioning as a differentiable lookup operation.

From an information-theoretic perspective, attention mechanisms implement smooth information aggregation. The softmax weights form a categorical distribution over key positions, with distribution entropy quantifying whether routing is sharply focused or diffuse (Cover and Thomas 2006). This balances two competing objectives: concentrating probability mass on relevant tokens while preserving sufficient entropy to avoid brittle hard-selection decisions.

However, full-rank multi-head attention frequently exhibits structural redundancy, with multiple heads learning correlated attention maps. Furthermore, the exponential in softmax creates acute sensitivity to underflow and overflow in reduced-precision formats. These hardware-relevant behaviors motivate structural pruning, low-rank factorization, sparse attention patterns, and specialized quantization strategies detailed in chapter 10.

Self-attention learns dynamic activation patterns across the input sequence. Unlike CNNs which apply fixed filters or RNNs which use fixed recurrence patterns, attention learns which elements should activate together based on their content. Trained attention heads often specialize in distinct syntactic or semantic patterns (Clark et al. 2019), dynamically adapting the effective connectivity graph for each input sequence.

The transformer architecture applies this self-attention mechanism within a broader structure that typically includes feed-forward layers, layer normalization, and residual connections. Figure 6.9 shows input tokens entering repeated attention and feed-forward blocks, each wrapped with residual connections and normalization, and emerging as contextualized representations. Because all positions can be processed in parallel rather than sequentially, the architecture trades recurrent state for large matrix operations that map well to accelerator training.

6.6.3 Computational mapping

Expanding sequence length S challenges accelerator memory hierarchies because full-sequence self-attention scales quadratically with context length. Figure 6.10 illustrates this trajectory: early transformer models operated with context windows of 512 to 2,048 tokens (Devlin et al. 2019; Radford et al. 2019; Brown et al. 2020), while modern production models support context windows from 128,000 to over one million tokens (OpenAI 2023c; Anthropic 2023; Google 2024a). Treat these product names as historical scale anchors: the enduring systems principle is that longer context windows shift complexity from external retrieval and document chunking pipelines into raw accelerator memory capacity and bandwidth budgets. Algorithmic tiling such as FlashAttention (T. Dao et al. 2022) and sparse approximations mitigate intermediate memory allocation, but they do not eliminate the fundamental scaling of attention state.

Listing 6.8 presents a typical implementation, showing how self-attention derives queries, keys, and values from the same input sequence.

²⁰ | **Attention scaling ($\sqrt{d_k}$):** This normalization directly counteracts the linear growth in variance (d_k) of the query-key dot product, preventing the softmax function from saturating where gradients would otherwise vanish. The systems consequence is most acute in mixed-precision training: when activations grow large, unscaled dot products can produce logits that overflow the 16-bit float range, destabilizing or halting learning entirely.

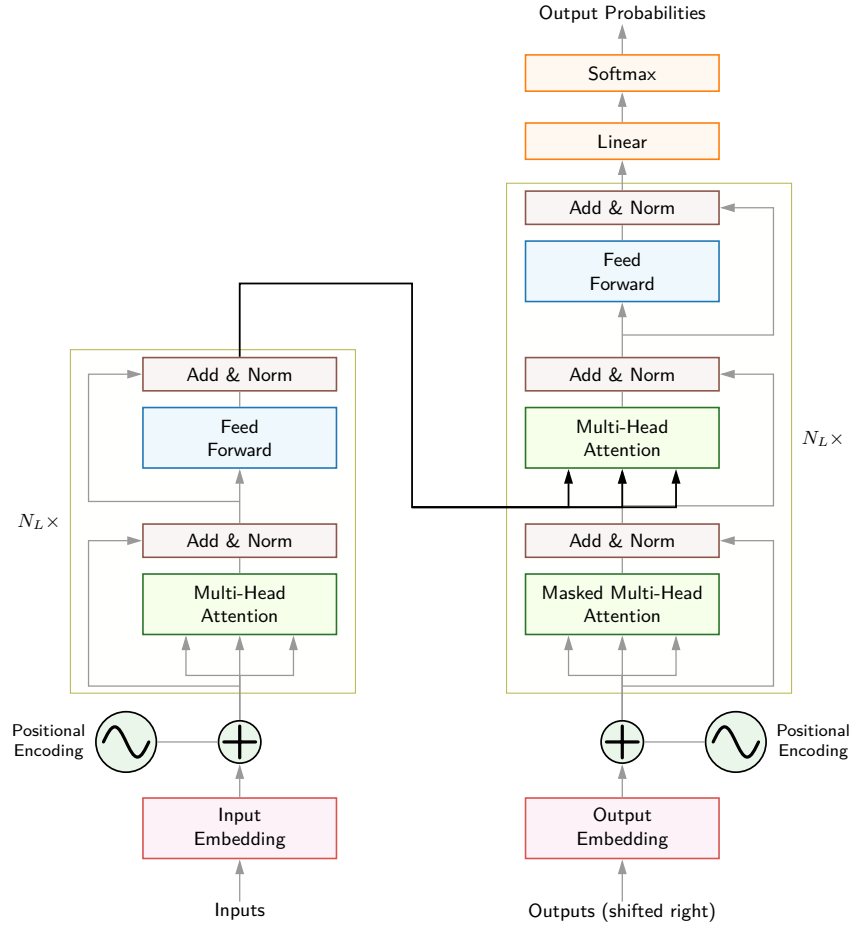


Figure 6.9: Transformer Architecture (Encoder-Decoder): Complete architecture. The encoder (left, repeated N_L times) consists of multi-head attention followed by feed-forward layers, each with residual connections (arrows bypassing blocks) and layer normalization. The decoder (right) adds masked attention to prevent attending to future tokens during autoregressive generation. Positional encodings (sine waves) inject sequence order information absent from the permutation-invariant attention operation. This design enables training parallelism across all positions while the decoder maintains autoregressive causality during inference. Source: (Vaswani et al. 2017).

Listing 6.8: Self-Attention and Multi-Head Attention: Self-attention (top) derives Q , K , and V from the same input X through three projections, then computes attention as before. Multi-head attention (bottom) runs N_{heads} parallel attention heads with dimension $d_k = d_{\text{model}}/N_{\text{heads}}$, then concatenates and projects. For GPT-2 (768-dim, 12 heads), each head operates on 64 dimensions, partitioning the projected feature dimension across heads while enabling diverse relationship patterns.

```
def self_attention_layer(X, W_Q, W_K, W_V, d_k):
    # X: input tensor (batch_size x seq_len x d_model)
    # W_Q, W_K, W_V: weight matrices (d_model x d_k)

    Q = matmul(X, W_Q)
    K = matmul(X, W_K)
    V = matmul(X, W_V)

    scores = matmul(Q, K.transpose(-2, -1)) / sqrt(d_k)
    attention_weights = softmax(scores, dim=-1)
    output = matmul(attention_weights, V)

    return output

def multi_head_attention(X, W_Q, W_K, W_V, W_O, num_heads, d_k):
    outputs = []
    for i in range(num_heads):
        head_output = self_attention_layer(
            X, W_Q[i], W_K[i], W_V[i], d_k
        )
        outputs.append(head_output)
```

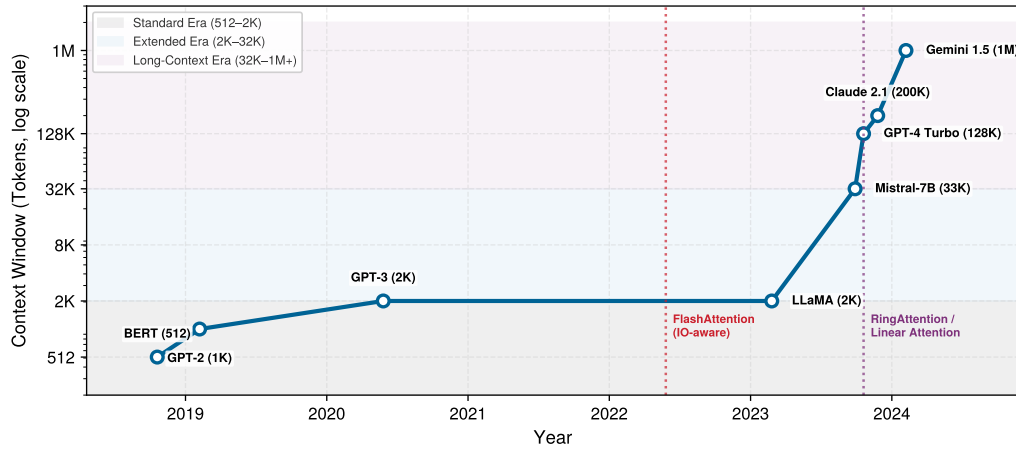


Figure 6.10: Context Window Explosion: Model context capacity (tokens, log scale) from early models (512 to 2K tokens) to modern long-context architectures (128K to 1M tokens). As context scales $1,000\times$, naive attention memory grows $1,000,000\times$ ($\mathcal{O}(S^2)$), driving the development of FlashAttention, sparse attention, and linear attention mechanisms.

Parallel sequence processing shifts fundamentally during autoregressive inference: generating output tokens step-by-step reintroduces sequential serialization and converts compute-bound matrix multiplications into memory-bandwidth-bound matrix-vector operations, as quantified in section 6.6.4 via the GPT-2 XL lighthouse.

6.6.4 System implications

The quadratic bottleneck analyzed in section 6.5.4 manifests differently across execution regimes. Full-sequence training and prefill compute all token positions concurrently, whereas autoregressive decoding generates one token at a time. Consequently, the operational bottleneck shifts from compute-bound matrix multiplications to memory-bandwidth-bound weight and state retrieval, governed by sequence length, batch size, and memory hierarchy limits.

6.6.4.1 Training: The quadratic compute wall

During training and prompt prefill, all token positions are processed in parallel, allowing accelerator tensor cores to achieve high floating-point utilization. However, dense attention’s score computation grows as $\mathcal{O}(S^2)$ with sequence length S . For long sequences (for example, 32k tokens), materializing the $32k \times 32k$ attention matrix requires gigabytes of activation storage per layer across multiple heads. Writing this quadratic intermediate matrix out to accelerator HBM and reading it back for softmax creates an extreme memory-bandwidth bottleneck. This physical constraint motivates I/O-aware kernel tiling (such as FlashAttention (T. Dao et al. 2022)), which fuses dot-product, softmax, and value-weighting steps into fast on-chip SRAM tiles, bypassing HBM round-trips for the intermediate $\mathcal{O}(S^2)$ matrix. The hardware memory hierarchies (HBM, SRAM, register files) that make such tiling effective are detailed in chapter 11.

6.6.4.2 Small-batch autoregressive decoding: The memory bandwidth wall

Autoregressive decoding generates one token at a time and is often *memory-bandwidth bound* at batch one or small batch sizes. To generate a token, the system performs three broad operations:

1. Read model weights, with reuse determined by batching and the memory hierarchy (for example, a 70-billion-parameter model stores 140 GB of FP16 weights; GPT-2 XL makes the cost of that first step concrete).
2. Perform matrix-vector multiplications.
3. Read and write the KV cache.

The **KV cache**²¹ grows linearly with sequence length ($\mathcal{O}(N_L \times 2 \times N_{\text{heads}} \times S \times d_{\text{head}})$ per request, distinct from the $\mathcal{O}(S^2)$ attention score matrix during training), storing the key and value vectors for

²¹ | **KV cache memory scaling:** For a 7-billion-parameter transformer in FP16, model weights consume ~14 GB; a single request’s cache requires $32 \text{ layers} \times 2 \text{ (K,V)} \times 32 \text{ heads} \times 2,048 \text{ positions} \times 128 \text{ dimensions} \times 2 \text{ bytes}$, or about 1.07 GB. At 8 concurrent users, the cache is ~8.6 GB, a substantial addition to the weights. Because the cache grows linearly with context and concurrency, scaling throughput forces choices such as grouped-query attention (Ainslie et al. 2023), shorter contexts, or KV paging and offloading (Kwon et al. 2023). Paging and offloading preserve model outputs, whereas grouped-query attention changes the architecture and shorter contexts truncate historical context.

all previous tokens to avoid recomputing them (Pope et al. 2023; Kwon et al. 2023). Keys and values from past tokens are retained because the newly generated token’s query (q_t) must attend across all historical context ($k_1 \dots k_{t-1}$ and $v_1 \dots v_{t-1}$) to compute its next representation. Prior queries ($q_1 \dots q_{t-1}$) are safely discarded because past tokens never attend forward to future tokens, while caching past keys and values avoids re-running the entire model forward pass over the full prompt at every generated token. For long contexts, this cache can become massive (for example, 100+ GB), and each decoding step reads prior keys and values. The resulting bandwidth pressure depends on context length, batching, attention architecture, cache layout, and hardware.



Lighthouse 6.5: GPT-2 XL (bandwidth lighthouse)

Why it matters: GPT-2 XL exemplifies **memory-bandwidth-bound** batch-one decoding. In the weight-only model used here, each decoding step reads 6 GB of FP32 weights and performs matrix-vector operations. The resulting arithmetic intensity is about 0.5 FLOP/byte with FP32 weights or 1 FLOP/byte with FP16 weights. Batching can amortize this weight traffic, while KV-cache and activation traffic add to it. Table 6.6 summarizes the bandwidth lighthouse’s quantitative properties:

Table 6.6: GPT-2 XL Systems Profile: Weight-only estimates for batch-one autoregressive decoding. Under these assumptions, each step reads the model weights at roughly 0.5–1 FLOP/byte, so HBM bandwidth strongly constrains generation throughput.

Property	Value	System Implication
Parameters	1.5B	Weight loading dominates inference latency.
Model Size	6 GB (FP32)	Fits on one GPU but saturates HBM bandwidth.
Compute	3 GFLOP/token	Low per-token compute; bottleneck is data movement, not math.
Constraint	Memory Bandwidth	Weight-bound batch-one tokens/s depends strongly on HBM bandwidth.
Profile	Bandwidth-Bound (small-batch decoding)	Larger batches can amortize weight traffic and shift the bottleneck.

Across training and inference, transformers exhibit distinct operational signatures: unconstrained parallel execution across token positions during training, quadratic arithmetic complexity in context length, and bandwidth-bound weight streaming during autoregressive decoding.

Yet dense and spatial models do not encompass the full spectrum of production workloads. Large-scale recommendation systems depart from standard arithmetic-dominated pipelines by coupling sparse embedding lookups—where memory capacity and irregular access distributions govern throughput—with downstream dense interaction layers. This hybrid matters at production scale: Meta reported that recommendation models accounted for most of its AI inference cycles (Gupta et al. 2020), even though such workloads receive less academic attention than language or vision models. Section 6.7 examines recommendation as the chapter’s final paradigm.

6.7 Sparse Architectures: RecSys

When a user opens a streaming service or e-commerce platform, the system must select a handful of recommendations from a catalog of millions in under 50 ms. The engineering challenge requires mapping both users and items to dense vectors in a shared embedding space, then evaluating interaction probabilities at scale.

Unlike compute-bound convolutional networks or bandwidth-bound transformers, recommendation models operate in a regime governed primarily by memory capacity. An accelerator engineered for dense matrix multiplication—such as an NVIDIA A100 providing hundreds of teraflops of arithmetic throughput alongside 80 GB of HBM—leaves its arithmetic units largely idle during recommendation workloads. The dominant operations are not dense matrix multiplications, but sparse gathers across tables that can span terabytes.

6.7.1 Pattern processing needs

The core challenge in recommendation systems is handling high-cardinality categorical features. A model must process user IDs across billions of accounts and item IDs across millions of products. Raw IDs carry no inherent geometry. User 1042 is not “near” user 1043 in any meaningful metric space, and a neural network cannot infer semantic relationships from integer values alone.

Embeddings solve the representation problem by mapping each discrete ID to a dense vector called an **embedding**²² (Mikolov et al. 2013). The system cost is that every lookup becomes an irregular read into a table spanning tens or hundreds of gigabytes. Recommendation models therefore encounter a capacity wall and a memory-bandwidth bottleneck before the first dense feedforward layer executes.

6.7.2 Algorithmic structure

The DLRM architecture (Naumov et al. 2019) standardizes this workflow as a four-stage pipeline partitioned across dense and sparse regimes. Continuous features such as user age or time of day first flow through a bottom MLP, a compute-intensive but memory-light stage dominated by dense matrix multiplications. In parallel, categorical IDs index into dedicated embedding tables, which is where the memory capacity wall appears:

For example, an embedding table for one billion users with 128-dimensional vectors requires $10^9 \times 128 \times 4 \text{ bytes} \approx 512 \text{ GB}$ of memory. This stage is memory-intensive but compute-light because each lookup is a memory copy with zero arithmetic reuse. The interaction layer then combines the dense activations from the bottom MLP with the gathered sparse embedding vectors through pairwise dot products that capture user-item correlations. Finally, a top MLP processes the concatenated interaction representations through feedforward layers to predict a target probability such as click-through rate. This combination of dense and sparse computation makes DLRM the chapter’s recommendation lighthouse; section 6.7.3 quantifies the capacity-bound profile that results.

6.7.3 Computational mapping and system implications

DLRM’s computational mapping splits into two regimes that stress different hardware subsystems. The dense MLPs execute standard GEMM operations, identical to the MLP computational mapping discussed in section 6.2.4 and handled efficiently by Tensor Cores. The sparse embedding lookups, however, are qualitatively different. They are index-based memory gathers with zero arithmetic intensity, making individual lookups memory-bandwidth bound at the operation level. This operational bottleneck is distinct from the capacity constraint: total table size dictates whether parameters fit in device memory, whereas memory bus throughput governs how fast individual rows can be gathered. Because each training sample or request accesses a different set of sparse rows, the access pattern exhibits near-zero spatial and temporal locality, defeating the caching and prefetching strategies that benefit CNNs and MLPs.

²² | **Embedding:** Deriving from the mathematical embedding of one space into another, neural embeddings map discrete tokens (such as user IDs or vocabulary words) into continuous vector spaces where geometric proximity captures semantic similarity. Word2vec popularized neural word embeddings in 2013. For systems, embedding tables create a distinctive memory access pattern: each lookup is an irregular read into a potentially terabyte-scale table, producing a sparse, bandwidth-bound workload that makes DLRM fundamentally different from compute-bound architectures like ResNet.



Lighthouse 6.6: DLRM (recommendation lighthouse)

Why it matters: DLRM exemplifies **memory-capacity-bound** workloads. Its massive embedding tables often exceed the memory of a single accelerator or server, so the system must decide where those tables live before it can optimize arithmetic throughput. The interaction layer then gathers selected embedding vectors and combines them with dense features, making capacity and irregular data movement the dominant constraints. This contrasts sharply with CNNs (compute bound) and transformers (memory-bandwidth bound), requiring different hardware and deployment choices. Table 6.7 summarizes the recommendation lighthouse’s quantitative properties.

Table 6.7: DLRM Systems Profile: Quantitative properties of the recommendation lighthouse and their system consequences. Billion-entry embedding tables push the model past single-device memory, creating a capacity-bound regime that the iron law does not directly address.

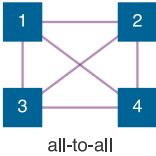
Property	Value	System Implication
Embedding Parameters	25B	Parameters × 4 bytes; dominates total model size.
Model Size	100 GB (FP32)	May exceed one device’s fast memory.
Constraint	Memory Capacity	Model size > Single GPU Memory.
Bottleneck	Irregular Data Movement	Gather operations dominate sparse feature processing.
Profile	Mixed (Sparse/Dense)	Combines memory-heavy lookups with compute-heavy MLPs.

Once embedding tables exceed one device, capacity, not arithmetic, becomes the primary systems constraint. While a ResNet-50 (102.4 MB) fits on a single accelerator and a dense language model like GPT-3 (350 GB) fits within a single multi-GPU node, production recommendation models reach terabytes of embedding parameters. In iron law terms (section 1.7), neither operation count O nor transfer volume D_{vol} acts as the binding constraint; raw memory capacity limits the system instead, establishing a regime outside the iron law’s primary scope.

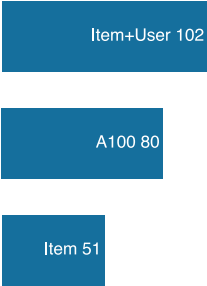
The architecture therefore breaks the single-device assumption that holds for smaller dense models. Systems designers resolve this capacity wall through three distinct architectural strategies:

- **Shrink the tables:** Compression, hashing, or pruning reduces capacity pressure, though these techniques risk information loss on long-tail users and items.
- **Move the tables:** Embeddings can reside in host DRAM or external key-value stores, though remote lookups incur high data-movement latencies over PCIe or Ethernet.
- **Partition the tables:** Tables can be sharded across multiple accelerators so that no single device stores the full model, though lookups then require cross-device communication.

Architecturally, DLRM turns recommendation into a capacity-management challenge before it becomes a compute-optimization task. Training-time table partitioning (chapter 8) and hardware-accelerated interconnects (chapter 11) address these distributed layouts, but the baseline sizing challenge begins with a single table.



DLRM embedding lookups can depend on vectors stored outside the local memory partition.



One embedding table fits an 80 GB A100; a second breaches the capacity wall.

 Napkin Math 6.2: The capacity wall

Problem: Consider a recommendation system for a store with 100M items using an embedding size of 128. How much memory does the item table alone require?

Math:

1. **Table entries:** 100M items.
2. **Vector size:** 128 elements.
3. **Precision:** FP32 (4 bytes per element).
4. **Table size:** 100M items × 128 × 4 bytes ≈ 51.2 GB.
5. **Capacity share:** 51.2 GB ÷ 80 GB = 64 percent of device capacity.
6. **Two-table check:** Two tables at 64 percent each exceed 100 percent of device capacity.

Systems insight: A single embedding table for one feature (items) already consumes 64 percent of an 80 GB A100 GPU. Adding a user table of the same size means the embedding tables no longer fit on a single 80 GB accelerator, motivating table compression, off-device memory, or partitioning across devices. DLRM is *capacity bound*: the primary systems question is not how many FLOP/s the accelerator can deliver, but where the embedding state can physically reside.

Once the tables no longer fit on one device, sparse lookups cross memory and network boundaries. A request may need rows owned by several devices, forcing the accelerator hosting the downstream MLP to wait while the system gathers them. Within a node, this exchange stresses the accelerator

interconnect; across nodes, it becomes an all-to-all collective communication bottleneck. The architectural consequence is that a recommendation model’s memory layout directly determines its network communication traffic, creating a distributed coordination tax for both training and serving.

Checkpoint 6.4: DLRM and sparse scatter

Recommendation systems stress a different part of the machine than CNNs or transformers.

- ☐ **Capacity-bound:** Can you explain why embedding tables push DLRM into a memory capacity regime where “FLOPs” is not the binding constraint?
- ☐ **Sparse access:** Can you explain why embedding lookups behave like random memory gathers (low reuse) and therefore resist caching and prefetching?
- ☐ **Placement logic:** If a single embedding table does not fit in fast device memory, can you describe which state must move, shrink, or be partitioned?
- ☐ **Data-movement bottleneck:** Can you explain why sparse embedding lookups can dominate even when the dense MLP has modest FLOP cost?

Across MLPs, CNNs, RNNs, transformers, and DLRM-style sparse models, diverse computational profiles converge on a compact set of shared primitives. Dense matrix projections, normalization layers, skip connections, and gating mechanisms recur across all five families. Every transformer block embeds a feedforward MLP, while gating, originally developed for recurrent cells, governs routing in mixture-of-experts models. These building blocks are portable because the numerical and optimization challenges they solve—gradient propagation, activation stability, and dynamic routing—persist regardless of inductive bias. For systems engineers, identifying these shared primitives reveals which hardware accelerators and compiler optimizations transfer across workloads and which must remain specialized.

6.8 Shared Building Blocks

A transformer block reuses several ideas born elsewhere: dense projections from MLPs, residual paths from deep CNNs, normalization for activation stability, and gating-like routing in later variants. The five architecture families differ in their data assumptions, but many of their engineering problems recur, so the practical question is which building blocks and optimizations transfer. Table 6.8 shows how these primitives accumulated as architectures grew more complex: each era inherited the tools of its predecessors while adding a mechanism for the next bottleneck.

Table 6.8: Cross-Architecture Building Blocks: Engineering primitives that originated in one architecture family and now recur across many modern designs. Each building block solves a recurring problem (gradient flow, activation stability, signal routing), though not every architecture uses every primitive.

Building Block	Born In	Problem Solved	Now Used In
GEMM	MLPs	Universal function approximation	All architectures (feedforward layers)
Parameter Sharing	CNNs	Spatial efficiency	Transformers (shared projections), RNNs (weight reuse across time)
Skip Connections	ResNets (CNNs)	Gradient flow at depth	Transformers, DenseNets, U-Nets, many modern deep networks
Normalization	CNNs (BatchNorm)	Activation stability	LayerNorm (Transformers), RMSNorm (root-mean-square normalization), GroupNorm (grouped channels)
Gating	LSTMs (RNNs)	Selective signal routing	Transformers (mixture-of-experts routing), GRUs, highway networks

Those portable building blocks were shaped by the hardware available at the time. LeNet-5 (LeCun et al. 1998) trained on CPUs with networks small enough to fit in megabytes of memory. AlexNet trained its 60-million-parameter network on two GTX 580 GPUs, mapping parallel convolutions to graphics hardware (Krizhevsky et al. 2012). ResNet-152 (He et al. 2016a) became trainable because

residual connections and batch normalization improved optimization at depth, using available GPU training infrastructure rather than a specific memory-capacity threshold. Transformers (Vaswani et al. 2017) became practical on contemporary GPUs and later scaled dramatically as GPU/TPU memory bandwidth and distributed training infrastructure improved. This pattern continues: each building block exploits newly available computational resources while pushing against the limits of existing systems.

6.8.1 Dense operations: The universal baseline

GEMM is the one primitive shared by every architecture in this chapter. While section 6.2 examined MLPs as dense pattern processors, the systems engineering legacy of GEMM extends far beyond MLPs. It is the feedforward layer inside every transformer block, the 1×1 pointwise convolution in MobileNets, the input and recurrent projections inside every RNN cell, and the bottom MLP in every DLRM.

MLPs introduced the GEMM-dominated computation profile that led GPU vendors to develop specialized matrix engines (Tensor Cores). The backpropagation algorithm's²³ memory access pattern, with alternating forward and backward passes storing intermediate activations, shaped accelerator memory hierarchies. The batch processing paradigm pioneered for MLP training established the throughput optimization that defines modern ML infrastructure: grouping independent samples into dense matrices converts memory-bandwidth-bound matrix-vector products into compute-bound matrix-matrix multiplications. These foundational patterns appear across all architectures examined in this chapter.

Dense connectivity also established the cost baseline that every subsequent architecture navigates. At $\mathcal{O}(n^2)$ parameters and operations for layers of width n , GEMM sets the reference point against which specialized architectures demonstrate efficiency gains. CNNs achieve spatial processing with $\mathcal{O}(k^2)$ parameters per location (where k is kernel size), transformers trade parameter efficiency for dynamic computation with $\mathcal{O}(S^2)$ attention complexity, and sparse architectures like DLRM exploit embedding lookups to handle categorical dimensions that would explode dense layer sizes. Each innovation represents a different strategy for escaping the dense connectivity baseline, but none escapes GEMM itself—it reappears inside every architecture as the workhorse of feature transformation.

6.8.2 Skip connections: Solving the depth problem

Parameter sharing made deep networks computationally tractable, but efficiency alone could not solve the numerical challenges of deep training. As networks grew deeper, repeated matrix multiplications and nonlinear activation derivatives caused gradient signals either to attenuate to numerical underflow or amplify to floating-point overflow. Skip connections resolve this optimization barrier by introducing additive identity shortcuts that bypass layer transformations.

6.8.2.1 The problem of depth

Backpropagation through N_L layers applies the chain rule repeatedly; section 5.4.4 gives the formal derivation of backpropagation and the chain rule. For a deep network with layers $f_1, f_2, \dots, f_{N_L}$, the gradient of the loss $\mathcal{L}$ with respect to the weights in layer 1 is:

$$\frac{\partial \mathcal{L}}{\partial W_1} = \frac{\partial \mathcal{L}}{\partial a_{N_L}} \cdot \frac{\partial a_{N_L}}{\partial z_{N_L}} \cdot \frac{\partial z_{N_L}}{\partial a_{N_L-1}} \cdot \dots \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial W_1}$$

where z_ℓ represents the preactivation and $a_\ell = \sigma(z_\ell)$ the postactivation output of layer ℓ . The gradient becomes a product of N_L terms, each depending on the activation function derivative $\sigma'(z_\ell)$.

Vanishing gradients create a silent training failure in deep architectures. For sigmoid activation functions, the derivative is $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, with maximum value $\sigma'(0) = 0.25$. Through N_L layers, the gradient magnitude is multiplied by approximately $(0.25)^{N_L}$. With such extreme attenuation, early layers receive infinitesimal gradient signals. Weight updates become negligible, effectively preventing these layers from training.

Exploding gradients are the destabilizing counterpart to vanishing gradients. Large singular values in layer Jacobians can amplify gradient directions repeatedly through depth. For example, if

²³ | **Backpropagation:** Rumelhart, Hinton, and Williams showed in 1986 how to efficiently apply the chain rule to train multi-layer networks; standard reverse-mode training retains the forward activations needed by the backward pass, so activation memory commonly grows with network depth. Checkpointing, recomputation, and offloading can reduce stored activations by trading additional computation or data movement. Activation storage, rather than weight storage, can therefore be the binding memory constraint that determines maximum feasible batch size on a given accelerator.

each Jacobian expands a shared direction by about 1.5, that component grows exponentially. Such growth produces numerical overflow, NaN (not-a-number) values, extreme parameter updates, and training divergence. Unlike vanishing gradients, which silently stall learning, exploding gradients cause catastrophic numerical failure.

6.8.2.2 Quantitative analysis: Plain deep networks

Consider training a deep plain convolutional network on CIFAR-10 without architectural interventions. Even with ReLU activations, which have derivative one for positive inputs, optimization degrades as depth increases. The original ResNet study reported that a 56-layer plain network had substantially worse CIFAR-10 test error than a 20-layer plain network (about 13.6 percent vs. 8.8 percent), demonstrating that adding layers impairs optimization despite greater representational capacity (He et al. 2016a).

This degradation problem is not overfitting: the 56-layer plain network exhibited higher training error than the 20-layer network, confirming that the failure stems from an optimization barrier rather than poor generalization.

6.8.2.3 Why ReLU helps but is not sufficient

ReLU activation ($\text{ReLU}(z) = \max(0, z)$) has derivative:

$$\text{ReLU}'(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

Through active paths ($z > 0$), the derivative equals 1, avoiding gradient decay from the activation function. This improvement over sigmoid enabled stable training of networks with 10–20 layers.

However, ReLU introduces a different failure mode: dead neurons. When $z \leq 0$, the gradient is zero, blocking gradient flow through that unit. A poorly initialized weight or large update can drive preactivations negative across the entire training distribution, permanently deactivating the neuron. Furthermore, ReLU does not regulate the spectral properties of the weight matrices themselves. Singular values differing significantly from unity continue to attenuate or amplify gradients across depth.

6.8.2.4 The residual solution

ResNet blocks introduce residual learning through skip connections that transform gradient flow. Equation 6.9 adds the identity path $\mathbf{x}$ to the residual mapping $\mathcal{F}(\mathbf{x})$:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x} \quad (6.9)$$

where $\mathcal{F}(\mathbf{x})$ represents the residual function (typically two convolutional layers with batch normalization and ReLU) and $\mathbf{x}$ is the identity skip connection.



Theorem 6.2: Residual Jacobian conditioning

Analysis: Consider the network as a composition of functions $\mathbf{x}_{\ell+1} = f_{\ell}(\mathbf{x}_{\ell})$. By the chain rule, the gradient at the input is the product of layer Jacobians $\mathbf{J}_{\ell} = \frac{\partial f_{\ell}}{\partial \mathbf{x}_{\ell}}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_0} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_{N_L}} \cdot \prod_{\ell=1}^{N_L} \mathbf{J}_{\ell}$$

For plain networks, $\mathbf{J}_{\ell}$ is arbitrary. The bound $\|\prod_{\ell} \mathbf{J}_{\ell}\|_2 \leq \prod_{\ell} \|\mathbf{J}_{\ell}\|_2$ means uniformly subunit norms force vanishing; conversely, $\sigma_{\min}(\prod_{\ell} \mathbf{J}_{\ell}) \geq \prod_{\ell} \sigma_{\min}(\mathbf{J}_{\ell})$ means uniformly superunit minimum singular values force expansion. Mixed, non-normal Jacobians depend on singular vectors as well as eigenvalues, so spectral radius alone does not determine gradient behavior.

For ResNets, the layer function is $\mathbf{x}_{\ell+1} = \mathbf{x}_{\ell} + \mathcal{F}(\mathbf{x}_{\ell})$, so the Jacobian is:

$$\mathbf{J}_{\ell} = \mathbf{I} + \frac{\partial \mathcal{F}}{\partial \mathbf{x}_{\ell}}$$

where $\mathbf{I}$ is the identity matrix. If $\varepsilon_{\ell} = \|\mathcal{F}'_{\ell}\|_2 < 1$, then $1 - \varepsilon_{\ell} \leq \sigma_{\min}(\mathbf{J}_{\ell}) \leq \sigma_{\max}(\mathbf{J}_{\ell}) \leq 1 + \varepsilon_{\ell}$. Thus a block whose residual Jacobian is small is close to identity and locally well conditioned. Across many blocks these deviations may still compound, so the skip connection improves gradient flow without guaranteeing unit gain or preventing cancellation.

During backpropagation, the gradient flows through this addition:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot \frac{\partial (\mathcal{F}(\mathbf{x}) + \mathbf{x})}{\partial \mathbf{x}}$$

Applying the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot \left(\frac{\partial \mathcal{F}(\mathbf{x})}{\partial \mathbf{x}} + \mathbf{I} \right) = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot \mathcal{F}'(\mathbf{x}) + \frac{\partial \mathcal{L}}{\partial \mathbf{y}}$$

The gradient decomposes into two additive terms. Even if the residual transformation $\frac{\partial \mathcal{L}}{\partial \mathbf{y}} \mathcal{F}'(\mathbf{x})$ attenuates, the identity term passes $\frac{\partial \mathcal{L}}{\partial \mathbf{y}}$ directly to the earlier layer. While the residual branch can partially interfere with the identity signal, keeping $\|\mathcal{F}'(\mathbf{x})\|$ small ensures the block Jacobian remains close to identity and bounds its condition number. The additive path provides a direct gradient shortcut through the network, though it does not guarantee exact unit gain across arbitrary depth.

6.8.2.5 Gradient flow through multiple blocks

Through N_L residual blocks, the gradient becomes:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_0} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_{N_L}} \cdot \prod_{\ell=1}^{N_L} (\mathcal{F}'_{\ell}(\mathbf{x}_{\ell}) + \mathbf{I})$$

Each factor $(\mathcal{F}'_{\ell} + \mathbf{I})$ contains the identity term, keeping the factor near identity when the residual-branch Jacobian is small. Unlike plain networks, which multiply arbitrary layer Jacobians, ResNets multiply these near-identity factors. The deviations can still compound, but this structure improves conditioning and helps make networks with 100+ layers trainable.

6.8.2.6 Empirical validation at 56 layers

Empirical results confirm this theoretical conditioning. In the original CIFAR-10 experiments, a 56-layer residual network reached about 7.0 percent test error, reversing the degradation seen in plain networks and outperforming its 20-layer counterpart (He et al. 2016a). Identity shortcuts provide an unobstructed pathway for gradient backpropagation and representation refinement, allowing deeper models to realize their capacity advantage. There is no single depth where skip connections suddenly become mandatory; trainability also depends on initialization, normalization, optimizer dynamics, and block architecture (He et al. 2016b).

While skip connections solve gradient conditioning, they impose concrete hardware costs. Evaluating $\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$ requires the input tensor $\mathbf{x}$ to remain resident in memory throughout the execution of all operations in the branch $\mathcal{F}(\mathbf{x})$. This extended liveness window increases peak activation memory in accelerator HBM during the forward pass and preserves $\mathbf{x}$ for backpropagation unless activation checkpointing is used. Furthermore, the elementwise addition is memory-bandwidth bound: it performs only one addition per loaded value (arithmetic intensity ≈ 0.5 FLOP/byte), turning un-fused residual merges into memory stalls.

Residual connections stabilize the backward gradient path, but they do not constrain the forward scale of intermediate activations. Unbounded variance across depth can still destabilize layer inputs, necessitating a complementary primitive: normalization.

6.8.3 Normalization: Stabilizing activations at depth

Normalization functions as the forward-pass partner to skip connections: where residual paths preserve backward gradient flow, normalization stabilizes the distribution of intermediate activations. First established as batch normalization in CNNs²⁴ (Ioffe and Szegedy 2015), the primitive evolved into layer normalization for sequence models and subsequently simplified into RMSNorm²⁵ for large language models.

6.8.3.1 Batch normalization: Definition and formulation

Batch normalization standardizes activations using statistics computed across the mini-batch for each feature or channel during training. For fully connected activations, the reduction axis is the batch dimension. For convolutional activations, implementations compute per-channel statistics over both the batch and spatial positions ($B \times H \times W$). For a mini-batch $\mathcal{B} = \{x_1, \dots, x_B\}$ of activations at a particular layer, the transformation proceeds in two stages.

First, compute the batch statistics:

$$\mu_{\mathcal{B}} = \frac{1}{B} \sum_{i=1}^B x_i \quad \sigma_{\mathcal{B}}^2 = \frac{1}{B} \sum_{i=1}^B (x_i - \mu_{\mathcal{B}})^2$$

Then, over the batch, normalize and apply learnable scale and shift. The normalization step in equation 6.10 centers and scales activations, while equation 6.11 applies learnable parameters that allow the network to recover the identity transformation if optimal:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad (6.10)$$

$$y_i = \gamma \hat{x}_i + \beta \quad (6.11)$$

The parameters γ (scale) and β (shift) are learned during training, while ϵ (typically 10^{-5}) prevents division by zero. The affine transform restores learned scale and shift flexibility, but fixed γ and β cannot exactly invert varying statistics for every batch. Normalization permits larger learning rates by preventing intermediate activations from exploding into saturating non-linear regimes, accelerating convergence relative to unnormalized networks.

Theorem 6.3: Normalization Jacobian conditioning

Analysis: Normalization stabilizes training by conditioning the Jacobian matrix of the layer. For the current batch, consider the gradient of the normalized output with respect to the input:

$$\frac{\partial \hat{x}_i}{\partial x_j} = \frac{1}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \left(\delta_{ij} - \frac{1}{B} - \frac{(x_i - \mu_{\mathcal{B}})(x_j - \mu_{\mathcal{B}})}{B(\sigma_{\mathcal{B}}^2 + \epsilon)} \right)$$

where δ_{ij} is the Kronecker delta. This Jacobian couples examples within the batch, with scale set by $1/\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}$; the subsequent affine scale γ multiplies it. It has zero directions associated with batchwise shift invariance, so there is no universal positive eigenvalue bound such as $[0.5, 2.0]$.

Batch normalization improves optimization empirically by standardizing intermediate scales, but it does not by itself prevent vanishing or exploding gradients through an entire network. Its quantitative effect depends on architecture, batch statistics, optimization, and parameterization rather than a universal two-to-fourfold gradient range.

6.8.3.2 Layer normalization: Architecture independence

While batch normalization enabled training of much deeper CNNs, it introduces severe distributed systems and sequence modeling constraints. In distributed training, computing true mini-batch

²⁴ | **Batch normalization (BatchNorm):** The original normalization layer (Ioffe and Szegedy 2015), which re-scales activations using per-mini-batch statistics. In one ImageNet experiment, it reached the target accuracy with $14\times$ fewer training steps. Its batch-size dependency and training-serving skew (switching from batch statistics to running averages at inference) are systems limitations that motivated alternatives: LayerNorm removed batch dependency for transformers, and RMSNorm removed mean centering.

²⁵ | **RMSNorm (root mean square normalization):** Introduced by Zhang and Sennrich (2019) at NeurIPS, RMSNorm simplifies LayerNorm by normalizing with the root mean square alone, dropping the mean-centering step. Across the paper's tested models and implementations, RMSNorm reduced overall running time by 7–64 percent relative to LayerNorm. LLaMA-family and Mixtral-style transformer reports use RMSNorm (Touvron, Lavril, et al. 2023; Touvron, Martin, et al. 2023; Jiang et al. 2024), illustrating why one reduction pass can matter for transformer inference latency.

statistics requires an all-reduce operation across accelerator nodes over PCIe or NVLink, adding communication latency to every normalized layer. If instead computed locally on small per-accelerator batches, noisy empirical statistics destabilize optimization. Furthermore, batch normalization cannot handle variable-length sequences or autoregressive token-by-token generation without complex padding masks and training-serving skew. Layer normalization resolves these limitations by normalizing across the feature dimension rather than across the batch (Ba et al. 2016).

For an input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$ with d_{model} features:

$$\mu_{\text{LN}} = \frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} x_i \quad \sigma_{\text{LN}}^2 = \frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} (x_i - \mu_{\text{LN}})^2$$

Equation 6.12 defines the complete layer normalization operation, where $\odot$ denotes element-wise multiplication:

$$\text{LayerNorm}(\mathbf{x}) = \frac{\mathbf{x} - \mu_{\text{LN}}}{\sqrt{\sigma_{\text{LN}}^2 + \epsilon}} \odot \gamma + \beta \quad (6.12)$$

Layer normalization normalizes each token or sample independently. The computation executes locally within an accelerator’s on-chip SRAM and registers without cross-sample or cross-device communication. This independence makes layer normalization the standard choice for transformers, where sequences vary in length and autoregressive generation evaluates single tokens at inference time.

6.8.3.3 Comparative analysis: When to use each variant

The choice among normalization variants reflects distinct machine and operational trade-offs. Table 6.9 summarizes the core properties. BatchNorm maintains learned scale/shift parameters alongside nonlearned running mean and variance buffers; LayerNorm computes statistics dynamically per sample at runtime, eliminating stateful running buffers.

Table 6.9: Normalization Variant Comparison: Different normalization techniques trade off between computational efficiency, batch size sensitivity, and architectural compatibility. RMSNorm (Zhang and Sennrich 2019), used in LLaMA-family and other efficient transformer architectures (Touvron, Lavril, et al. 2023; Touvron, Martin, et al. 2023), omits mean centering:

$$\text{RMSNorm}(\mathbf{x}) = \mathbf{x} / \sqrt{\frac{1}{d_{\text{model}}} \sum_i x_i^2 + \epsilon} \cdot \gamma.$$

Characteristic	BatchNorm	LayerNorm	RMSNorm
Normalization Axis	Batch and, for CNNs, spatial positions	Feature dimension	Feature dimension
Batch Size Dependency	High (noisy for small batches)	None	None
Typical Use Case	CNNs, vision models	Transformers, RNNs	LLaMA, efficient transformers
Computation Cost	Higher (mean + variance)	Higher (mean + variance)	Lower (RMS only)
Training/Inference	Different (running stats)	Identical	Identical

The computational profile of normalization is defined by its arithmetic intensity. Because normalization applies elementwise operations to large activation tensors, execution is memory-bandwidth bound rather than compute bound. Standard LayerNorm requires two reduction passes across the hidden dimension d_{model} in SRAM: one to accumulate the mean μ_{LN} , and a second to compute the variance σ_{LN}^2 . RMSNorm eliminates the mean-centering step, reducing the operation to a single sum-of-squares reduction. This single reduction pass cuts memory traffic in half and allows tighter kernel fusion, driving the 7–64 percent speedup reported across transformer inference workloads.

Operationally, BatchNorm requires explicit mode switching between training (which computes mini-batch statistics) and inference (which applies running statistics accumulated during training). Mismatches between running statistics and deployment data distributions represent a frequent source of training-serving skew. LayerNorm and RMSNorm apply identical computations during training and inference, eliminating running buffers and ensuring consistency across deployment environments.

6.8.4 Gating: Controlling information flow

Skip connections and normalization resolve depth-related stability—gradient flow and activation variance, respectively. Gating addresses a different challenge: selectively routing information through a network based on input context.

Gating originated in recurrent networks, where simple recurrent loops suffered from severe gradient degradation across long temporal sequences. LSTMs²⁶ (Hochreiter and Schmidhuber 1997) and GRUs²⁷ (Cho et al. 2014) addressed this by introducing **gates**: parameter-weighted transformations that act as differentiable valves to protect, update, or discard stored state.

Gating generalizes beyond recurrence as a foundational mechanism for data-dependent conditional routing. In feedforward structures, Highway Networks used gating to dynamically interpolate between transformation and identity bypass, directly inspiring residual shortcuts. In sequence processing, attention scales gating across the entire temporal context: encoder-decoder attention (Bahdanau et al. 2015) computes a dynamic routing distribution over input representations. In transformers, softmax attention weights act as data-dependent routing coefficients, determining how features aggregate across tokens, while mixture-of-experts architectures extend gating to dynamically dispatch tokens to specialized feedforward sub-networks.

6.8.5 Synthesis: How transformers recombine everything

Modern transformer architectures synthesize all four shared primitives into a unified block. As illustrated in figure 6.11, residual paths wrap each sub-layer to preserve gradient flow through deep stacks. Dense GEMM projections execute the feature transformations within multi-head attention and feedforward sub-networks. LayerNorm (or RMSNorm) stabilizes activations prior to each projection. Finally, attention softmax weights dynamically gate information routing across token positions.

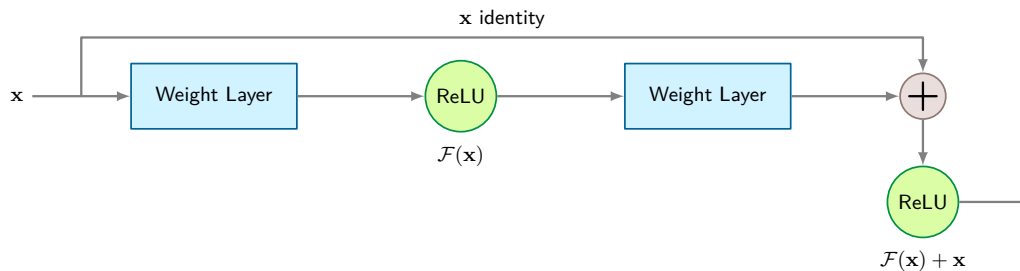


Figure 6.11: Residual Connection Block: The skip connection implements $y = \text{ReLU}(\mathcal{F}(x) + x)$, creating an identity shortcut that bypasses the weight layers before the final nonlinearity. During backpropagation, gradients flow through both the residual path (via $\mathcal{F}'(x)$) and the identity path (via $\mathbf{I}$ before the final nonlinearity), helping gradients reach early layers even in 100+ layer networks. This architectural pattern enabled very deep residual CNNs and became central in transformer blocks and many other modern architectures.

The transition from RNNs to transformers replaced sequential recurrent step dependencies with parallel all-to-all attention, reducing the sequential operational depth from $\mathcal{O}(S)$ steps to $\mathcal{O}(1)$ steps for tokens to interact. This architectural shift maximizes hardware occupancy on massively parallel accelerators like GPUs and TPUs, where thousands of arithmetic units sit idle if constrained by sequential dependencies. The other building blocks carried over directly: GEMM, skip connections, and normalization remain essential across all families.

This portability extends to modern variants. Vision Transformers²⁸ adapt the transformer to images while maintaining all four building blocks (Dosovitskiy et al. 2021). GPT-3 scales these transformer patterns with alternating dense and locally banded sparse attention while relying on the identical primitive set (Brown et al. 2020). Practical implementation challenges and optimizations are explored in chapter 10.

Table 6.10 contrasts how the four architectural families utilize these core primitives. Transformers retain the dense GEMM operations common to all architectures but introduce content-dependent all-to-all reductions through attention, blending the broadcast operations of MLPs with the gather and reduce operations of more dynamic architectures.

²⁶ | **LSTM (long short-term memory):** Invented by Hochreiter and Schmidhuber in 1997, LSTMs introduced a “Constant Error Carousel,” a gated cell state that protects error signals from exponential decay during backpropagation through time. The systems cost of this solution: a standard LSTM computes input, forget, and output gates plus a candidate cell update, giving roughly four affine transformations per time step vs. one in a vanilla RNN. This compute overhead explains why transformers, which solve long-range dependencies through parallelizable attention, replaced LSTMs in many large-scale language workloads.

²⁷ | **GRU (gated recurrent unit):** Cho et al. (2014) describes a gated hidden unit for encoder-decoder translation that uses reset and update gates to control how the hidden state is updated. Relative to an LSTM’s input, forget, and output gates plus candidate cell update, this gives a simpler gated recurrence. The broader systems lesson: architectural simplification can reduce state and matrix operations when it preserves task performance, a principle that recurs in efficiency-oriented designs from MobileNet to distilled transformers.

²⁸ | **Vision transformers (ViTs):** Google’s 2020 ViT paper split 224×224 images into 16×16 patches (196 “tokens”) and applied standard transformer attention. ViTs replace CNN’s local convolutions with $\mathcal{O}(S^2)$ global attention over patch tokens. In the original study, large-scale pre-training improved ViT’s competitiveness, illustrating how data and compute can compensate for weaker spatial inductive bias (Dosovitskiy et al. 2021).

Table 6.10: Primitive Utilization Across Architectures: Every architecture uses GEMM, but they differ in memory access patterns, data movement (broadcast to gather+reduce), and parallelism. Transformers combine dense tiled QKV access with attention-specific reductions, which is why they stress both the GEMM units and the reduction path at once.

Primitive Type	MLP	CNN	RNN	Transformer
Computation	Dense GEMM	Convolution	Sequential GEMM	GEMM + Attention
Memory Access	Sequential	Strided	Sequential + State	Tiled QKV streams
Data Movement	Broadcast	Sliding window	Temporal broadcast	Gather + Reduce
Parallelism	High	High	Low (time deps)	High (positions)

For systems engineers, this building-block perspective separates portable optimizations from architecture-specific ones. GEMM tiling and mixed-precision compute benefit every architecture. Skip connection memory management applies to any residual network. Normalization kernel fusion helps CNNs and transformers alike. Attention-specific optimizations remain tied to attention’s memory pattern, but even those build on the same underlying GEMM and memory-access primitives. To understand why these optimizations transfer, section 6.9 lowers the shared layers to the primitives the machine executes.

6.9 Computational Primitives

Frameworks and compilers do not execute high-level layer abstractions directly; they lower neural networks to a compact set of *computational primitives*. A ResNet-50 forward pass executes billions of multiply-accumulate operations; a transformer attention block streams gigabytes through on-chip memory hierarchies; a recommendation model scatters random lookups across terabyte-scale embedding tables. Despite these architectural divergences, the physical workload on hardware resolves into three primitive domains: dense arithmetic, memory access trajectories, and inter-core or inter-device data movement. Isolating these primitives exposes the physical bottlenecks—arithmetic intensity, memory bandwidth, and interconnect latency—that dictate accelerator design and compiler optimizations detailed in chapter 11.

6.9.1 Core computational primitives

The core primitive determines which execution pattern an architecture forces the hardware to optimize: dense tensor math, repeated local reuse, or input-dependent routing. Matrix multiplication, sliding window operations, and dynamic computation recur across model families because each preserves a distinct execution profile when lowered to silicon. They are primitive in the engineering sense: decomposing them further erases the performance characteristics that hardware units and compilers must optimize.

Matrix multiplication is the dense tensor-math path. Multiplying an activation matrix by a weight matrix computes linear combinations across feature dimensions (recall the reference MLP layer from section 6.2.3). This path dominates modern workloads: MLPs execute it directly for fully connected projections, CNNs lower convolutions into matrix multiplications, and transformers rely on it for query-key-value projections and feed-forward networks. Figure 6.12 illustrates how lowering maps sliding-window spatial locality into a standard row-major matrix product.

The `im2col`²⁹ (image to column) technique transforms sliding-window convolutions into dense GEMMs. By unfolding each receptive field patch into a matrix row and stacking filter weights into columns, `im2col` enables convolutional layers to execute on mature, vendor-tuned GEMM libraries such as `cuBLAS`, `MKL`, and `OpenBLAS`.

This lowering enforces an explicit memory-for-compute trade-off. A batched input tensor of shape (B, C_{in}, H, W) is unpacked into an unfolded matrix of shape $[(B \cdot H_{\text{out}} \cdot W_{\text{out}}) \times (K^2 \cdot C_{\text{in}})]$, while convolutional filters reshape to $[(K^2 \cdot C_{\text{in}}) \times C_{\text{out}}]$. Their product yields an output matrix of shape $[(B \cdot H_{\text{out}} \cdot W_{\text{out}}) \times C_{\text{out}}]$. For stride-1 convolutions with kernel size K , input pixels shared across overlapping receptive fields are duplicated up to K^2 times in memory, increasing memory footprint and DRAM traffic unless fused or tiled directly in on-chip SRAM.

Structured and unstructured sparsity are treated in section 10.3.1; hardware-aware sparse execution and algorithm-hardware co-design are treated in chapter 11.

²⁹ | **im2col (image to column):** Rather than being a new learning algorithm, `im2col`-style lowering is an implementation technique used by CNN frameworks and libraries such as Caffe and `cuDNN` (Jia et al. 2014; Chetlur et al. 2014): it converts convolutions into standard GEMM calls by unfolding overlapping patches into matrix columns. The trade-off is memory: in a simple fully materialized stride-1 $K \times K$ transform, interior input elements can appear in up to K^2 columns (9 times for 3×3 filters), though borders, stride, padding, tiling, and direct-convolution algorithms reduce the realized expansion. This memory-for-simplicity exchange explains why mobile frameworks (TFLite, NNAPI) prefer direct convolution, while data center GPUs with abundant HBM may use GEMM-oriented lowering when it improves throughput.

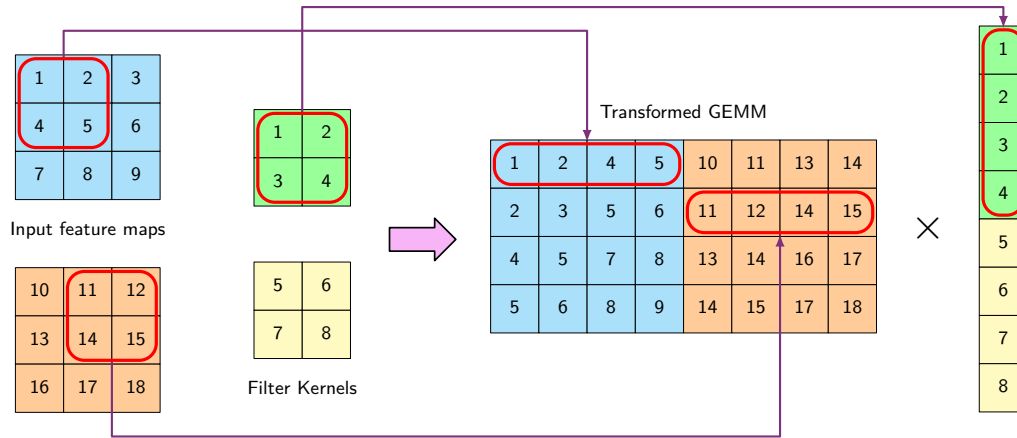


Figure 6.12: im2col Transformation: Converts convolution to GEMM by unfolding image patches into a dense matrix. The two 3×3 input feature maps produce four sliding-window positions, represented here as rows of a 4×8 matrix; the two 2×2 filter kernels stack into an 8×1 column. Their product yields the four output positions. Lowering can exploit optimized matrix kernels but expands memory by an amount that depends on kernel size, stride, padding, and implementation, so its performance benefit is workload dependent.

Sliding window operations represent the local-reuse path. Rather than materializing duplicate buffers in memory, native sliding-window execution sweeps a compact filter kernel across input tensors, evaluating local stencils. Accelerators exploit this structure through specialized dataflow buffering. For example, TPUs route computations through 2D systolic arrays,³⁰ where activations and weights stream synchronously through a grid of multiply-accumulate processing elements. Passing data directly between adjacent registers prevents round-trips to off-chip memory for every arithmetic operation.

Dynamic computation represents the adaptive-routing path. In dense transformer attention, query-key interaction weights vary per input token, but the underlying tensor shapes and matrix operations remain static and regular. In contrast, sparse attention, mixture-of-experts (MoE) gating, and dynamic graph execution introduce data-dependent control flow and runtime tensor dimensions. This dynamic execution challenges conventional static memory allocation and parallel work dispatch.

Production architectures combine these execution paths. A transformer block uses dense GEMMs of shape $[S, d_{\text{model}}] \times [d_{\text{model}}, d_{\text{proj}}]$ for feature projections, an $S \times S$ attention score calculation, and optional dynamic routing across expert sub-networks. However, peak arithmetic throughput on paper does not guarantee realized performance: hardware execution units remain starved if the memory subsystem cannot feed them operands at matching rates.

6.9.2 Memory access primitives

Memory access speed and predictability determine accelerator utilization. Even a matrix-multiplication engine capable of hundreds of teraFLOPs stalls if data cannot traverse the memory hierarchy fast enough. While an on-chip arithmetic operation completes in single-digit clock cycles, fetching an operand from off-chip DRAM requires hundreds of cycles.

Memory access patterns fall into three categories—sequential access, strided access, and random access—governing how effectively hardware can prefetch, cache, and coalesce memory requests. Because transferring data across packaging boundaries consumes far more power than ALU operations, memory access patterns dictate energy budgets alongside execution latency.

🔍 Systems Perspective 6.2: The energy cost of data movement

A core systems principle: *moving data costs more than computing on it.* A floating-point multiply-accumulate operation consumes approximately 1–5 pJ depending on process node (for example, ~4.6 pJ at 45 nm (Horowitz 2014)), while fetching a 32-bit operand from off-chip DRAM costs

³⁰ | **Systolic array:** Named for the heart's rhythmic contraction, the array's lockstep "pulse" of data through a grid of processors directly implements the efficient data reuse required by sliding window operations. By passing input values between neighboring processors, an expensive round-trip to off-chip DRAM is avoided for every single multiplication in the convolution. This is critical for efficiency, as a single off-chip memory access can cost over 100× more energy than a floating-point multiply-accumulate, and still more relative to low-precision arithmetic.

hundreds of pJ in the same reference model. Even conservative estimates put off-chip DRAM access tens to hundreds of times above arithmetic, explaining why memory access patterns, not just FLOP counts, determine real-world efficiency.

Revisit the preceding architectures through this energy lens: MLPs have low data reuse (each weight loaded once per sample) and are therefore energy-dominated by DRAM traffic. CNNs reuse filter weights across spatial positions, amortizing load cost over $H \times W$ applications; the very locality that makes them compute-bound also makes them energy-efficient. RNNs reuse weights across time steps (high temporal reuse) but pay repeated hidden-state read/write costs at each step. Transformers combine pairwise score computation with key-value movement, making dense full-sequence attention compute-quadratic; implementation and tiling determine auxiliary memory traffic and energy. These energy profiles directly track the bottleneck column in table 6.3.

This principle underlies later optimization strategies: section 10.4 shows how quantization reduces bits moved per value, pruning eliminates unnecessary data movement, and tiling keeps working sets in faster, lower-energy caches.

Sequential access provides the highest effective bandwidth. During batched MLP matrix multiplication, hardware streams contiguous weight rows and activation vectors. Modern DRAM architectures exploit burst-mode transfers for contiguous addresses, saturating available interface bandwidth (reaching multiple terabytes per second on HBM subsystems), while hardware prefetchers anticipate subsequent cache lines. Compilers optimize for this pattern by allocating contiguous memory blocks and aligning tensor strides with hardware cache lines.

Strided access occurs when memory requests skip across non-contiguous addresses at uniform intervals. In standard convolutional layers without im2col transformation, convolving across spatial dimensions requires accessing inputs separated by the row stride of the feature map. Strided access reduces memory throughput by fetching entire cache lines while utilizing only a fraction of each line's bytes. Compilers mitigate this penalty by reordering loops or packing strided slices into contiguous temporary buffers in shared memory.

Random access presents the most severe memory bottleneck. In recommendation model embedding tables, sparse feature IDs index arbitrary rows across gigabyte- or terabyte-scale tables. These unpredictable lookups defeat hardware prefetchers, fragment DRAM burst transactions, and generate high cache miss ratios. Dense transformer attention presents a contrasting profile: while attention weights are content-dependent, the underlying query, key, and value vectors reside in contiguous tensors. Optimized attention kernels tile these dense matrices through on-chip SRAM, avoiding random DRAM addresses altogether.

Table 6.11 quantifies how these memory access patterns govern parameter and activation storage scaling across model families.

Table 6.11: Memory Access Complexity: Different neural network architectures exhibit varying memory access patterns and storage requirements, impacting system performance and scalability. Parameter storage depends on model dimensions, while activation storage represents a significant runtime cost. The transformer's $\mathcal{O}(BS^2)$ term describes materialized attention scores in a naïve implementation; tiled exact-attention kernels can avoid retaining that full matrix.

Architecture	Input Dependency	Parameter Storage	Activation Storage	Scaling Behavior
MLP	Linear	$\mathcal{O}(N_{\text{in}} \times d_{\text{width}})$	$\mathcal{O}(B \times d_{\text{width}})$	Predictable
CNN	Constant w.r.t. resolution	$\mathcal{O}(K^2 C_{\text{in}} C_{\text{out}})$	$\mathcal{O}(B \times H_{\text{img}} \times W_{\text{img}} \times C)$	Efficient
RNN	Linear	$\mathcal{O}(d_{\text{hidden}}^2)$	$\mathcal{O}(B \times S \times d_{\text{hidden}})$	Challenging
Transformer	Quadratic attention	$\mathcal{O}(d_{\text{model}}^2 + d_{\text{model}} d_{\text{ff}})$ per block	$\mathcal{O}(B \times S^2)$ attention, plus $\mathcal{O}(BS d_{\text{model}})$ activations	Problematic

Where:

- N_{in} : Input size
- d_{width} : Layer width

- B : Batch size
- K : Kernel size
- C : Number of channels
- $C_{\text{in}}, C_{\text{out}}$: Input and output channels
- H_{img} : Height of input feature map (CNN)
- W_{img} : Width of input feature map (CNN)
- d_{hidden} : RNN hidden-state dimension
- S : Sequence length
- d_{model} : Transformer model dimensionality

These memory scaling properties dictate data reuse opportunities and cache behavior. In CNNs, spatial reuse allows each input pixel to participate in multiple overlapping filter evaluations (K^2 times for a $K \times K$ kernel), enabling L1, L2, and shared memory to amortize DRAM load costs through loop tiling. Conversely, an unbatched MLP reloads its entire weight matrix for a single input vector, achieving minimal data reuse and leaving the processor bandwidth-starved.

Working set size—the volume of data required concurrently by execution units—determines whether an operation fits within fast on-chip memory. An MLP layer working set often fits within a few hundred kilobytes, residing entirely within on-chip SRAM. In contrast, full-sequence attention in transformers materializes intermediate attention maps that scale as $\mathcal{O}(B \cdot S^2)$, easily exceeding on-chip SRAM capacity for long sequences and spilling into high-bandwidth DRAM unless fused using online softmax tiling.

While table 6.11 defines memory scaling, table 6.12 contrasts the resulting arithmetic demands, parallelization structures, and primary hardware bottlenecks across the four reference architectures.

Table 6.12: Computational Complexity Comparison: Scaling behaviors and resource requirements for major neural network architectures. Variables: d_{in} = input dimension, d_{out} = output dimension, d_{model} = transformer model dimension, d_{hidden} = RNN hidden-state dimension, k = kernel size, c = channels, $H_{\text{img}}, W_{\text{img}}$ = spatial dimensions, S = sequence length (time steps), B = batch size. The bottleneck column identifies the primary performance-limiting factor in typical deployments. For transformers, the $\mathcal{O}(S^2)$ memory term assumes materialized attention scores; tiled exact-attention kernels can avoid retaining that full matrix.

Architecture	Parameters	Forward Pass	Memory	Parallelization	Bottleneck
MLPs	$\mathcal{O}(d_{\text{in}} \times d_{\text{out}})$ per layer	$\mathcal{O}(d_{\text{in}} \times d_{\text{out}})$ per layer	$\mathcal{O}(d_{\text{in}} d_{\text{out}})$ weights $\mathcal{O}(B d_{\text{out}})$ activations	Excellent Matrix ops parallel	Memory bandwidth
CNNs	$\mathcal{O}(k^2 \times c_{\text{in}} \times c_{\text{out}})$ per layer	$\mathcal{O}(H_{\text{img}} \times W_{\text{img}} \times k^2 \times c_{\text{in}} \times c_{\text{out}})$	$\mathcal{O}(H_{\text{img}} \times W_{\text{img}} \times c)$ features $\mathcal{O}(k^2 \times c^2)$ weights	Good Spatial independence	Often compute throughput; bandwidth for depthwise or small-batch cases
RNNs	$\mathcal{O}(d_{\text{hidden}}^2 + d_{\text{hidden}} \times d_{\text{in}})$ total	$\mathcal{O}(S \times d_{\text{hidden}}^2)$ for S time steps	$\mathcal{O}(d_{\text{hidden}})$ recurrent state (inference); $\mathcal{O}(S d_{\text{hidden}})$ activations (training)	Poor Sequential deps	Sequential deps
Transformers	$\mathcal{O}(d_{\text{model}}^2)$ QKV/O projections plus $\mathcal{O}(d_{\text{model}} d_{\text{ff}})$ feed-forward layers	$\mathcal{O}(S^2 \times d_{\text{model}} + S \times d_{\text{model}}^2)$ per layer	$\mathcal{O}(S^2)$ attention $\mathcal{O}(S \times d_{\text{model}})$ sequences	Excellent (positions) Limited by memory	Memory (S^2)

Memory access primitives govern how efficiently a single execution core pulls data from its local memory hierarchy. However, executing neural networks at scale requires coordinating data streams across multiple cores, on-chip caches, and discrete accelerators.

6.9.3 Data movement primitives

Data movement primitives characterize the fan-out and fan-in of tensor values across processing elements, memory banks, and accelerator interconnects. As established by the physical energy gap between ALU operations and data transfers, moving data across interconnects often incurs higher latency and energy costs than the computation itself.

Communication patterns reduce to four fundamental collective operations: broadcast, scatter, gather, and reduction. Figure 6.13 contrasts their communication topologies.

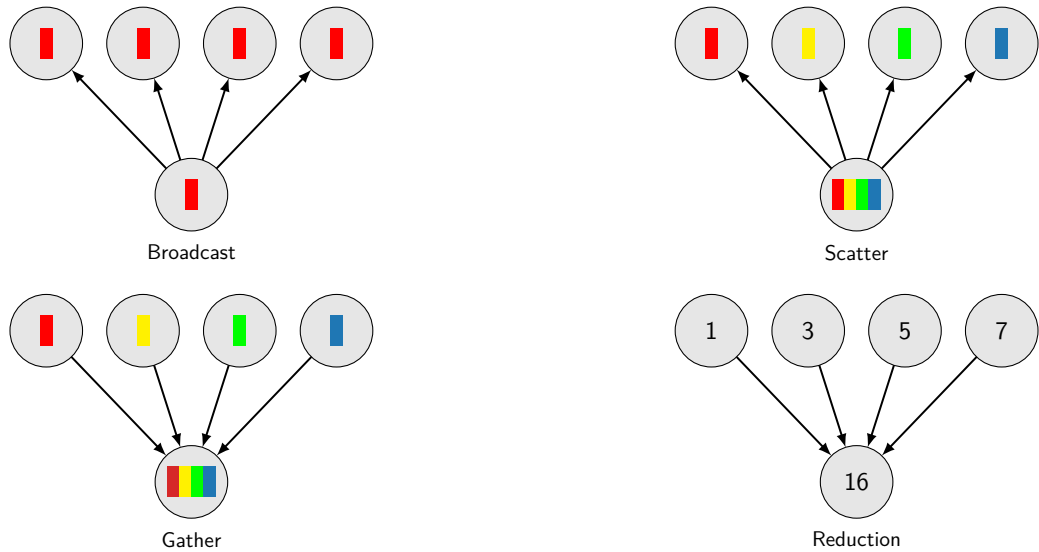


Figure 6.13: Data Movement Primitives: Four fundamental patterns govern information flow in neural network computation. Broadcast (top-left) replicates a single value to all destinations, used when sharing weights across batch elements. Scatter (top-right) distributes distinct elements to different destinations, enabling work partitioning. Gather (bottom-left) collects distributed values to a single location, as in attention pooling. Reduction (bottom-right) combines multiple values through aggregation (sum, max), appearing in gradient synchronization and attention scoring. The energy cost of data movement (see the Memory Access Primitives section) makes these patterns critical optimization targets.

Broadcast operations distribute identical data from a single source to multiple parallel destinations. In batched GEMM execution, stationary weight tiles are broadcast to multiple processor cores, each evaluating an independent batch element. Accelerators accelerate broadcasts via dedicated on-chip crossbars or multicast interconnect fabrics, while compilers tile loops to maximize the operational reuse of each broadcast value.

Scatter operations partition a source tensor into disjoint segments, dispatching distinct slices to different destinations. In spatial or model-parallel partitioning, a large activation or weight tensor is scattered across compute tiles or cluster nodes. In large language models, MoE architectures introduce dynamic, input-dependent scatter patterns: a gating network routes individual tokens to specific expert networks distributed across devices. Because token routing depends on runtime inputs, dynamic scatter operations frequently cause load imbalance, leaving lightly loaded accelerators idle while oversubscribed experts bottleneck the pipeline.

Gather operations collect distributed tensor slices into a centralized destination. In sparse recommendation systems, gather operations retrieve embedding vectors corresponding to sparse feature indices across distributed memory nodes. In transformer self-attention, gathering occurs along the sequence dimension as each query aggregates key-value representations across all sequence positions.

Reduction operations combine multiple input values into an aggregated result using associative operators such as summation or maximum. Reductions appear universally in neural networks: evaluating layer normalization, computing softmax normalizers, and synchronizing weight gradients across distributed data-parallel workers. Accelerator hardware accelerates reductions through dedicated tree-structured reduction networks and warp-level shuffle registers, reducing completion latency from $\mathcal{O}(n)$ sequential steps to $\mathcal{O}(\log n)$ parallel stages.

In real workloads, these primitives execute in composite stages. For example, a single transformer attention head with sequence length $S = 512$ and batch size $B = 32$ broadcasts query vectors, gathers key-value pairs across the sequence dimension, and reduces the resulting inner products through a softmax denominator. As model dimensions exceed single-accelerator memory limits, collective

communications across scale-up links (such as NVLink or optical TPU fabrics) govern end-to-end throughput, making data movement primitives the primary architectural constraint.

6.9.4 System design impact

Computational, memory access, and data movement primitives dictate physical silicon allocation and compiler design. Hardware designers dedicate silicon area to specialized units only when the underlying mathematical primitive dominates representative workloads.

The prevalence of dense matrix multiplication and convolution motivated the development of dedicated matrix engines, such as Google TPUs³¹ and GPU Tensor Cores. Chapter 11 analyzes how these execution units map algorithmic primitives directly onto silicon datapaths, trading general-purpose instruction flexibility for dense multiply-accumulate density.

Memory architectures have similarly bifurcated to accommodate divergent primitive profiles. Supporting predictable streaming alongside random or strided access led to the integration of High Bandwidth Memory (HBM), which stacks DRAM dies vertically to deliver 2–3 TB/s of bandwidth—more than an order of magnitude above standard DDR channels. On-chip, accelerators provide multi-megabyte SRAM hierarchies and explicitly managed **scratchpad memories**, giving compilers predictable control over working-set data staging.

Interconnect architectures reflect data movement primitives. To sustain the high-volume broadcasts, gathers, and reductions demanded by multi-accelerator models, modern platforms incorporate high-bandwidth point-to-point links and specialized network-on-chip routers optimized for collective communication patterns.

Table 6.13 summarizes this hardware-software co-design, mapping each computational, memory, and communication primitive to its dedicated hardware acceleration, compiler optimization, and primary physical bottleneck.

Table 6.13: Primitive-Hardware Co-Design: Efficient machine learning systems require tight integration between algorithmic primitives and underlying hardware. Common primitives map to specific hardware accelerations and software optimizations, each presenting distinct implementation challenges. Specialized hardware such as Tensor Cores addresses computational demands of matrix multiplication and sliding windows, while software techniques like batching and dynamic graph execution further enhance performance.

Primitive	Hardware Impact	Software Optimization	Key Challenges
Matrix Multiplication	Tensor Cores	Batching, GEMM libraries	Parallelization, precision
Sliding Window	Specialized datapaths	Data layout optimization	Stride handling
Dynamic computation	Flexible routing	Dynamic graph execution	Load balancing
Sequential Access	Burst mode DRAM	Contiguous allocation	Access latency
Random Access	Large caches	Memory-aware scheduling	Cache misses
Broadcast	Specialized interconnects	Operation fusion	Bandwidth
Gather/Scatter	High-bandwidth memory	Work distribution	Load balancing

These primitive mappings govern system energy consumption. As established in the following analysis, the physical energy cost of moving an operand across memory packaging exceeds arithmetic energy by orders of magnitude.

Large batched GEMMs in MLPs achieve high arithmetic intensity, amortizing weight loads across many inputs. In contrast, small-batch MLP inference exhibits minimal reuse and spends most of its energy moving weights from off-chip DRAM. A reference FP32 multiply costs approximately 3.7 pJ/FLOP, while fetching a single 32-bit operand from DRAM costs 640 pJ (Horowitz 2014)—an energy disparity of 173×. While total workload energy depends on operation counts and reuse factors, low-reuse operations remain overwhelmingly energy-dominated by DRAM transfers. This physical reality drives accelerator architects to allocate large on-chip SRAM pools and adopt wafer-scale or multi-die packaging to keep active working sets on-chip.

Convolutional operations reduce energy per output by reusing filter weights across $H \times W$ spatial locations. However, implementation details dictate realized energy. While im2col lowering simplifies execution by invoking vendor GEMM libraries, fully materializing the unfolded tensor can inflate

³¹ | **TPU (tensor processing unit):** Google’s first TPU maps matrix multiplication onto a large systolic array, trading general-purpose features such as caches and complex control flow for domain-specific inference efficiency (Jouppi et al. 2017). The architectural lesson needed here is qualitative: when one primitive dominates a workload, dedicated data paths and local reuse can outperform general-purpose flexibility. Chapter 11 develops the precision choices, hardware specifications, and performance trade-offs.

DRAM 640 pJ

FP32 mul 3.7 pJ

log scale

Moving a 32-bit value from DRAM costs far more energy than one FP32 multiply.

temporary memory traffic by up to K^2 for stride-1 filters. Direct convolution algorithms and fused sliding-window kernels avoid this intermediate buffer, substantially lowering memory traffic and package energy dissipation.

Sequential processing in RNNs exhibits temporal weight reuse across sequence steps. Storing the compact hidden state entirely in on-chip SRAM eliminates repeated DRAM round-trips for recurrent activations. However, the strict sequential dependencies prevent parallelization across time steps, causing execution units to idle and inflating energy consumption per token.

In transformers, multi-head attention introduces high data movement overheads. In standard un-fused implementations, materializing the full $S \times S$ attention matrix incurs quadratic memory read-write cycles (the bottleneck detailed in section 6.5.4). Tiled implementations such as FlashAttention alleviate this burden by fusing the softmax computation into on-chip SRAM, eliminating intermediate DRAM round-trips and aligning attention’s energy profile with compute-bound matrix multiplication.

Hardware architects face fundamental engineering trade-offs when optimizing for these primitives. Allocating silicon to wide, dense systolic arrays maximizes throughput and energy efficiency for regular GEMMs, but leaves functional units underutilized during sparse lookups or dynamic routing. Conversely, expanding on-chip SRAM to hold massive transformer working sets increases die area and static leakage power that dense, regular workloads do not require.

Profiling computational primitives establishes the operational boundaries of each architecture family. In production systems engineering, the task shifts from analyzing individual operators to selecting and provisioning a complete architecture under hard operational constraints—balancing predictive accuracy against the platform budgets established in chapter 2 and lifecycle requirements in chapter 3.

6.10 Architecture Selection Framework

A wildlife monitoring sensor must classify camera-trap images within a strict milliwatt power envelope, while an industrial recommendation service must retrieve terabyte-scale embeddings within a sub-millisecond tail latency budget. These deployment constraints immediately eliminate architectures that appear competitive on unconstrained predictive benchmarks. Each architectural family embodies distinct physical assumptions about data structure and computation: MLPs assume arbitrary pairwise feature interactions, CNNs exploit local spatial correlation via sliding-window weight reuse, RNNs enforce strict sequential recurrence, and transformers evaluate all-to-all attention across token contexts. Architecture selection is the discipline of matching these computational topologies to physical deployment budgets before optimizing kernels or tuning hyperparameters.

In the D-A-M taxonomy, architecture selection defines the algorithmic compute graph (A) that bridges input data invariants (D) to physical execution hardware (M). A structural mismatch at this boundary cannot be rescued by downstream compiler optimizations or kernel tuning. The selection framework evaluates candidate models across three coupled dimensions: the statistical structure of the input data (D), the algorithmic complexity and arithmetic intensity of the model (A), and the hardware execution boundaries—memory capacity, memory bandwidth, compute throughput, and thermal envelope (M)—established in chapter 2 and chapter 3. The same selection logic governs data curation in chapter 9 and cluster deployment in chapter 14.

6.10.1 Data-to-architecture mapping

Systematic architecture selection begins with data-to-architecture mapping: aligning the structural symmetries and invariants of the input data with an operator family’s compute and memory access patterns. The architectural families introduced in section 6.1 establish this foundation: MLPs for tabular records with arbitrary feature interactions, CNNs for grid-structured data with local spatial correlation, RNNs for streaming inputs with temporal recurrence, transformers for relational contexts where tokens exhibit arbitrary pairwise dependencies, and sparse embedding architectures such as DLRM for high-cardinality categorical recommendation streams.

This alignment reflects fundamental memory and compute trade-offs. When an architecture encodes data invariants directly into its computational operators—such as translation equivariance via convolutional weight reuse—it avoids learning those symmetries empirically. An architecture that matches data symmetries minimizes parameter count, confines intermediate activations to

on-chip SRAM, and maximizes arithmetic intensity. Conversely, deploying an architecture that lacks appropriate inductive biases forces the model to learn structure through parameter over-provisioning, multiplying off-chip DRAM traffic and inflating training sample complexity.

In production systems, these structural trade-offs dictate workload partitioning across the memory hierarchy:

- **MLPs** process dense tabular features (such as sensor telemetry or actuarial tables) where feature dimensions lack geometric distance metrics, yielding matrix-vector operations with deterministic latency.
- **CNNs** process spatial grids (such as imagery and spectrograms) where localized convolution kernels reuse filter weights across the feature map, maximizing register and cache reuse.
- **RNNs** process sequential streams with bounded temporal state, maintaining a compact working set in SRAM at the cost of serialized step-by-step evaluation.
- **Transformers** process token sequences and relational graphs (Vaswani et al. 2017; Devlin et al. 2019; Wei et al. 2022) through dynamic all-to-all attention, enabling expressive cross-sequence interaction while incurring quadratic score computation and high activation-memory traffic.
- **Sparse embedding architectures (DLRM)** decouple memory-capacity-bound embedding tables from compute-bound multilayer perceptrons, routing billions of sparse categorical IDs across high-capacity DRAM before dense feature interaction.

6.10.2 Computational complexity considerations

Data-to-architecture alignment establishes functional fit, but asymptotic scaling under hardware constraints determines physical feasibility. An architecture whose compute requirements or memory working set outpaces physical accelerator limits violates latency service level agreements (SLAs) or exceeds device memory capacity.

As synthesized in table 6.11 and table 6.12, each architectural family imposes a characteristic profile of arithmetic intensity, working-set scaling, and memory access locality. Evaluating these profiles identifies the primary hardware bottleneck—memory bandwidth, arithmetic throughput, or sequential latency—governing production deployment.

6.10.2.1 Scalability and production considerations

Production deployment evaluates an architecture against strict service-level agreements: tail latency, memory capacity, energy per query, and checkpoint recovery overhead. These operational metrics are direct physical consequences of structural design: dense connectivity, spatial locality, sequential dependence, or all-to-all attention. The structural choices that determine representational capacity simultaneously govern execution parallelism, cache residency, and memory bandwidth consumption.

MLPs and CNNs occupy the simpler operational regime because inference is stateless across input samples, allowing linear throughput scaling via data parallelism. However, their memory access profiles diverge significantly. At batch size $B = 1$, MLP inference is strictly memory-bandwidth bound: streaming large weight matrices from off-chip DRAM to compute a single matrix-vector product yields an arithmetic intensity near 1 FLOP/byte, though latency remains highly deterministic. In contrast, CNN execution achieves high weight reuse across spatial dimensions, but intermediate activation memory scales with image resolution ($H_{\text{img}} \times W_{\text{img}} \times c$), dominating peak SRAM and DRAM capacity during high-resolution inference and training backward passes.

RNNs and transformers create challenging operational regimes for contrasting physical reasons. An RNN maintains a compact recurrent hidden state, but the recurrence relation $h_t = f(h_{t-1}, x_t)$ serializes execution along the temporal dimension. Amdahl's law dictates that provisioning additional parallel execution units cannot reduce this critical-path latency; furthermore, stateful execution complicates distributed checkpointing and pipeline recovery. In contrast, transformers eliminate sequential recurrence, parallelizing token projections across the entire sequence dimension to saturate systolic arrays and Tensor Cores. Yet standard self-attention incurs a quadratic memory and compute footprint relative to sequence length S (section 6.5.4), where materializing the $S \times S$ attention matrix rapidly exhausts HBM and constrains maximum serving batch size. Small serving batches

underutilize matrix execution units, whereas large batches demand multi-device memory partitioning. Chapter 8 formalizes these distributed execution strategies through data, tensor, pipeline, and sequence parallelism.

6.10.2.2 Hardware mapping and optimization strategies

Hardware mapping matches the algebraic structure of an operator to the physical execution datapath. Dense matrix operations in MLPs map directly to systolic arrays and GPU Tensor Cores (chapter 11 details these silicon implementations). These workloads benefit from three hardware-level optimizations: hierarchical matrix tiling (such as 64×64 tiles for L1 cache, 256×256 for L2 cache, and 16×16 Tensor Core micro-tiles) to ensure register and cache residency; mixed-precision execution (such as FP16 or BF16 arithmetic with FP32 accumulation) to double arithmetic throughput and halve memory traffic; and operator fusion to eliminate intermediate DRAM round-trips by executing elementwise activations and bias additions entirely within register files. Chapter 7 examines how compilers generate these fused kernel launches on specific accelerators.

CNNs benefit from specialized convolution algorithms and data layout optimizations that differ significantly from dense matrix operations. Im2col transformations convert convolutions to matrix multiplication but can multiply temporary storage and memory traffic, up to K^2 for fully materialized stride-1 $K \times K$ filters away from the borders. Winograd algorithms³² reduce multiplication count by $2.25 \times$ for 3×3 convolutions but can amplify numerical error. Direct convolution with custom kernels can avoid im2col materialization but requires architecture-specific tuning.

Because the sequential recurrent dependency cannot be shortened by provisioning additional compute hardware (section 6.4.4), optimization strategies focus on minimizing execution overhead along the critical path. Loop unrolling eliminates branch instructions and per-step control overhead at the expense of instruction cache pressure and activation storage. State vectorization batches multiple independent sequences across SIMD vector lanes, amortizing memory access costs across sequences without reducing single-sequence latency. Wavefront parallelization overlaps independent forward and backward passes in bidirectional models, improving execution unit saturation where the model topology permits. These techniques amortize runtime overhead but leave the fundamental serialization constraint intact.

Self-attention requires memory-hierarchy optimizations that mitigate quadratic intermediate state. Because computing and storing raw attention matrices drives memory bandwidth bottlenecks, the dominant optimization strategy tiles computation within fast on-chip SRAM. Section 8.6.4 examines FlashAttention³³ as an IO-aware tiling technique, while structured sparse attention reduces arithmetic complexity by restricting attention to fixed receptive patterns.

These hardware trade-offs define the operational boundaries of each architecture: MLPs provide deterministic, low-latency execution for non-spatial feature vectors; CNNs maximize data reuse across spatial grids; RNNs maintain minimal state footprints for streaming sequences; and transformers trade quadratic memory bandwidth for relational expressiveness. Translating these hardware invariants into production deployments requires a structured decision protocol.

6.10.3 Decision framework

Engineering teams frequently default to architectures based on popularity rather than physical constraints—deploying over-parameterized transformers where linear models or lightweight CNNs suffice, or selecting models that violate inference latency budgets. Systematic selection replaces heuristics with an ordered constraint-satisfaction protocol grounded in the D·A·M taxonomy.

The decision flowchart in figure 6.14 begins by identifying input data characteristics, routes to candidate dense architectures (Transformers, RNNs, CNNs, or MLPs), and evaluates each candidate against physical hardware limits. High-cardinality recommendation workloads sit outside this dense flowchart and route to the sparse embedding/DLRM family (section 6.7) before applying the same memory, compute, speed, accuracy, and deployment checks. If any constraint fails, the decision path loops back to scale down model capacity or change the architecture family.

When constraints require scaling down, the model compression techniques in chapter 10 provide systematic approaches for reducing memory, compute, and latency while preserving accuracy. The framework executes in three ordered phases:

³² | **Winograd algorithm:** For one 2×2 output tile with a 3×3 filter, this method trades 36 direct multiplications for 16 elementwise multiplications in the Winograd domain, plus additional transforms and additions. The transforms can amplify rounding error, so low-precision suitability depends on the variant, implementation, and accuracy requirements.

³³ | **FlashAttention:** An IO-aware algorithm (T. Dao et al. 2022) that avoids materializing the full $S \times S$ attention matrix in HBM by fusing computation into a single kernel tiled to fit in SRAM. The result: $2\text{--}4 \times$ wall-clock speedup and memory reduction from $\mathcal{O}(S^2)$ to $\mathcal{O}(S)$, enabling training on sequences $4\text{--}16 \times$ longer than standard attention. FlashAttention demonstrates that algorithmic optimization of data movement (D_{vol}) can yield larger speedups than increasing raw compute (R_{peak})—a concrete validation of the iron law's data term.

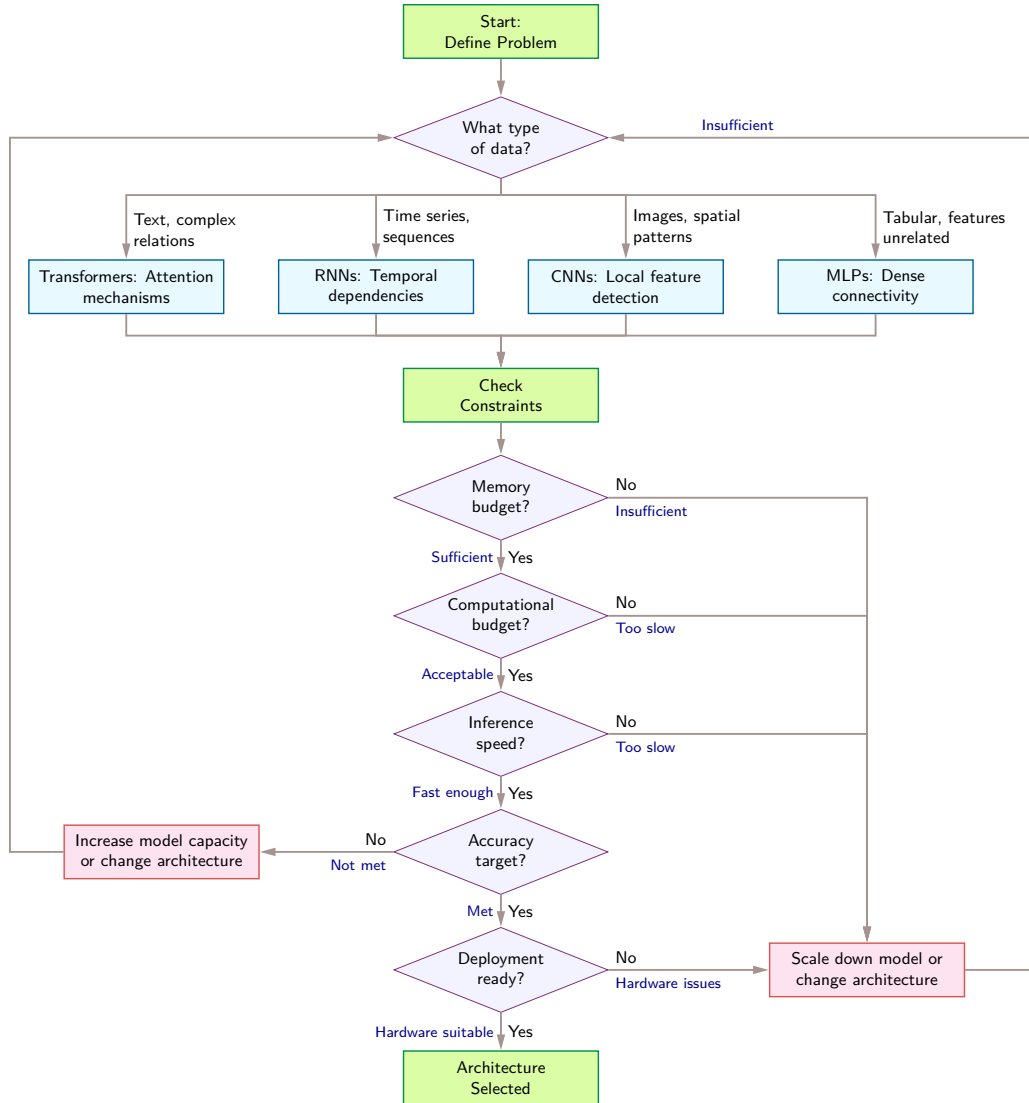


Figure 6.14: Architecture Selection Decision Framework: A systematic flowchart for choosing dense neural network architectures based on data characteristics and deployment constraints. The process begins with data type identification (text/sequences/images/tabular) to select initial architecture candidates (Transformers/RNNs/CNNs/MLPs), then iteratively evaluates memory budget, computational cost, inference speed, accuracy targets, and hardware compatibility. High-cardinality recommendation workloads are handled separately by the sparse-embedding family (DLRM) in the Sparse Architectures section, then re-enter the same constraint checks.

1. **Data symmetry identification (D):** Identify structural symmetries and invariances in the input data. Spatial locality routes candidates to CNNs; temporal sequence structure to RNNs; unconstrained cross-token dependency to transformers; tabular features without spatial geometry to MLPs; and high-cardinality categorical IDs to sparse embedding tables.
2. **Progressive physical constraint filtering ($A \times M$):** Filter candidate models against hardware budgets: static parameter storage in flash/DRAM, peak dynamic activation footprint in SRAM/HBM, arithmetic intensity against the hardware roofline, and tail latency deadlines. A violation requires scaling down model width or depth, applying quantization and compression (chapter 10), or switching to a lower-overhead architecture family.

3. **Empirical capacity and deployment validation:** Verify that candidate capacity meets target accuracy on representative evaluation distributions. If predictive accuracy falls short, scale model capacity and re-evaluate physical hardware constraints. If target accelerator hardware cannot sustain the required arithmetic throughput or memory bandwidth, select an alternative architecture rather than relying on marginal hyperparameter adjustments.

6.10.4 Inductive bias hierarchy

The decision framework operates on a continuum of structural constraints: the inductive bias hierarchy introduced in section 6.1. In machine learning theory, inductive bias encompasses the assumptions a model uses to generalize beyond observed training examples. In systems engineering, inductive bias is a primary efficiency lever: constraining the hypothesis space restricts parameter volume, bounds intermediate activation working sets, and exposes structured data reuse patterns to the memory hierarchy.

The architectural families form a hierarchy of decreasing structural constraint:

- **CNNs** exhibit the strongest architectural bias via local connectivity, weight sharing, and translation equivariance. These constraints collapse parameter volume and enable aggressive tile staging in registers and cache, but restrict the model to grid-structured inputs.
- **RNNs** enforce moderate structural bias through temporal recurrence and shared transition matrices, bounding runtime memory footprint across arbitrary sequence lengths at the cost of execution serialization.
- **Transformers** exhibit adaptive inductive bias, replacing fixed spatial or temporal topologies with dynamic, content-based attention across all token pairs. This architectural flexibility enables generalization across diverse data modalities, but shifts the computational burden to quadratic attention matrices and high memory-bandwidth utilization.
- **MLPs** maintain minimal inductive bias, assuming no topological relationships among input features. Without structural constraints, learning spatial or temporal patterns requires massive parameter over-provisioning and excessive training data, resulting in poor arithmetic intensity and low cache locality.

Many deep architectures implement hierarchical representation learning, but through distinct operator mechanisms: CNNs through progressive receptive field expansion (section 6.3), RNNs through hidden state evolution (section 6.4), and transformers through multi-head attention (section 6.5). This hierarchical organization reflects a fundamental physical principle: complex representations compose from simpler primitives. In systems engineering, this composition directly dictates runtime efficiency. Feature aggregation pipelines must compose primitives without triggering redundant round-trips to off-chip DRAM. Memory staging must align with representational hierarchies to maximize on-chip SRAM reuse. Furthermore, parallel kernel dispatch must conform to hierarchical dependencies across layers, keeping execution units saturated across forward and backward passes.

6.10.5 Architecture selection in practice

A real-time wildlife monitoring deployment illustrates the interaction between data structure, model capacity, and physical hardware budgets. Before committing to a candidate architecture, a throughput ceiling calculation establishes whether the target hardware can sustain the required frame processing rate.



Napkin Math 6.3: The throughput ceiling

Problem: A real-time application must process 30 FPS of video with ResNet-50. What sustained compute throughput is required?

Math:

1. **Model cost:** ResNet-50 requires ~ 8.2 GFLOP per 224×224 image.
2. **Frame rate:** 30 FPS required.
3. **Sustained throughput:** $30 \text{ FPS} \times 8.2 \text{ GFLOP} = 246 \text{ GFLOP/s}$.

4. **Effective throughput:** $10 \text{ TFLOP/s peak} \times 50\text{--}60 \text{ percent utilization} = 5 \text{ TFLOP/s--}6 \text{ TFLOP/s}$.
5. **ResNet-50 headroom:** $5 \text{ TFLOP/s} \div 246 \text{ GFLOP/s} = 20.3\times$.
6. **Detection-model check:** $30 \text{ FPS} \times 100 \text{ GFLOP} = 3 \text{ TFLOP/s}$, leaving $1.7\times$ headroom at the lower effective-throughput estimate.

Systems insight: A mid-range GPU delivering 10 TFLOP/s theoretical peak achieves ~50–60 percent utilization in this planning scenario, yielding 5 TFLOP/s–6 TFLOP/s effective. For ResNet-50 at 30 FPS, the system has 20.3× headroom. Switching to an object detection model at 100 GFLOP per frame, however, requires 3 TFLOP/s sustained, leaving only 1.7× headroom. Batch size constraints or multi-stream processing quickly push the system toward the compute ceiling. ResNet-50 is compute-bound, but the margin depends on the accelerator and utilization achieved.

The throughput ceiling converts an abstract compute requirement into a concrete hardware utilization percentage. A real-world deployment adds physical constraints the ceiling alone does not capture.

6.10.5.1 Worked example: Real-time wildlife monitoring

The design task is an embedded ML system that classifies wildlife species from camera trap images in an isolated national park. The deployment must execute inference entirely on-device with zero network connectivity, operate on solar-recharged battery power for six months, and achieve 90 percent accuracy across 50 target animal species. A five-phase evaluation guides the selection process: data characterization, deployment constraint profiling, candidate architecture evaluation, hardware budget validation, and operational risk assessment.

The input data consists of camera trap image frames (1920×1080 raw sensor resolution, down-sampled to 224×224 for inference). Species classification relies on visual features exhibiting three invariant properties: spatial locality (discriminative patterns like fur textures or ear shapes occupy bounded pixel neighborhoods), translation equivariance (an animal’s identity is invariant to its spatial position in the frame), and hierarchical composition (low-level edges combine into textures, anatomical parts, and complete organisms). These invariants point directly to a CNN, whose localized kernel weight sharing encodes these geometric symmetries natively, eliminating the parameter explosion that an unconstrained MLP would incur.

Constraint analysis translates the deployment environment into hard hardware boundaries. Table 6.14 catalogs these physical constraints alongside the architectural requirements each imposes:

Table 6.14: Wildlife Monitoring Constraints: Five deployment constraints and their implications for the camera-trap scenario. No connectivity rules out cloud-only inference, while the power, latency, memory, and accuracy requirements must be validated on the target device.

Constraint	Requirement	Implication
Connectivity	None (offline)	All inference must run on-device
Power	~2 W average (solar + battery)	Rules out GPUs; must use low-power MCU or edge NPU
Latency	<500 ms per detection	Allows batch size 1, no real-time streaming
Memory	512 MB RAM, 2 GB storage	Model, activations, and runtime buffers must fit locally
Accuracy	90%+ on 50 species	Requires sufficient model capacity

With physical constraints established, candidate architectures are evaluated against reference model baselines:

- **ResNet-50** (25.6M parameters, 8.2 GFLOP): Prohibitive compute and thermal footprint for the 2 W envelope. At 102.4 MB in FP32, weight storage alone exhausts most of the 100 MB allocation before accounting for activation buffers or OS overhead.
- **MobileNetV1** (4.2M parameters, 1138 MFLOP): Viable arithmetic intensity and footprint. Storing weights in INT8 reduces model footprint to 4.2 MB (16.8 MB in FP32), and factorized depthwise separable convolutions drastically cut arithmetic operations.

- **KWS DS-CNN** (200K parameters, 20 MFLOP): Insufficient capacity. Sized for 12-class keyword spotting, its parameter budget cannot separate 50 visual species classes.

While MobileNetV1 demonstrates the viability of depthwise separable convolutions, MobileNetV2 achieves higher parameter efficiency and better memory locality through inverted residual blocks with linear bottlenecks. Sizing the model with width multiplier 0.75× yields ~2.6M parameters (10.4 MB FP32, 2.6 MB with INT8 weights) and costs ~418 MFLOP at 224×224 input resolution. This model fits comfortably within the allocated memory budget while preserving representational capacity for the 50 target species.

Hardware validation checks the chosen model against physical platform budgets. The memory footprint validates against available device RAM:

$$\underbrace{2.6 \text{ MB}}_{\text{Model}} + \underbrace{224 \times 224 \times 64 \times 4 \approx 12.8 \text{ MB}}_{\text{Activations}} + \underbrace{50 \text{ MB}}_{\text{OS/Buffers}} = 65.4 \text{ MB} \ll 512 \text{ MB} \checkmark$$

The compute budget validates on the target system-on-chip (SoC), an ARM Cortex-A53 at 1.2 GHz with NEON SIMD (~2 GOPS INT8): $\frac{418 \times 10^6 \text{ INT8 ops}}{0.002 \times 10^{12} \text{ INT8 ops/s}} = 209 \text{ ms}$ latency $\ll 500 \text{ ms}$ target $\checkmark$

Active power represents the final physical gate. At an estimated active inference draw of ~200 mW over 209 ms, each classification consumes 41.8 mJ. For a trigger-based rate of 100 inferences/day, model execution consumes 4.2 J/day. This leaves ample battery margin for image sensor capture, sleep-state quiescent current, non-volatile storage logging, and peripheral management.

The fifth step assesses operational risks in the field. Table 6.15 pairs the primary accuracy, thermal, and coverage failure modes with corresponding systems engineering mitigations:

Table 6.15: Wildlife Camera Deployment Risks: Top accuracy, thermal, and species-coverage risks for the MobileNetV2 deployment, paired with the engineering mitigation chosen for each.

Risk	Mitigation
90% accuracy not achieved	Train on augmented dataset; consider EfficientNet-Lite if MobileNet insufficient
Thermal throttling in enclosure	Add passive heatsink; reduce inference frequency in high-temperature conditions
New species added postdeployment	Expand or retrain the output head and plan an over-the-air (OTA) update mechanism

The resulting deployment commits to MobileNetV2 (0.75× width multiplier) quantized to INT8, running on an ARM Cortex-A53 SoC with 512 MB RAM. This configuration satisfies the active compute, memory footprint, and power budgets—processing each frame in approximately 209 ms while preserving substantial memory headroom for system software. Final field qualification requires measuring classification accuracy on target camera distributions and validating battery longevity across seasonal temperature swings.

This case study translates abstract architectural principles into a concrete sizing decision. Yet production architecture selection frequently encounters counterintuitive trade-offs: models with fewer FLOPs can stall on bandwidth-limited hardware, expressive architectures can degrade on datasets lacking matched structure, and lab prototypes can fail against deployment memory hierarchies. Grounding these failure modes in hardware reality exposes the principal fallacies and pitfalls of systems design.

6.11 Fallacies and Pitfalls

Architecture choice is a systems decision, not a leaderboard selection. The common mistakes in this section arise when teams treat architecture families as interchangeable accuracy tools while ignoring inductive bias, memory traffic, hardware mapping, and deployment state.

Fallacy: *More complex architectures always perform better than simpler ones.*

Engineers often assume that transformers outperform simpler architectures on all tasks. In production, architectural sophistication must match problem complexity: the algorithm must fit both the structure of the data and the cost of the machine. The MNIST comparison in section 6.2.1 shows that the example CNN uses 47× fewer parameters than the example MLP because its locality bias matches image structure. Accuracy, training cost, and inference latency still require measurement for the specific implementation and task.

Pitfall: *Selecting architectures based solely on accuracy metrics without analyzing computational requirements.*

Practitioners choose architectures from papers reporting top-line accuracy, ignoring computational implications. RNNs expose a sequential critical path that limits parallelism, while transformers face quadratic dense-score computation and, in naive implementations, quadratic score storage. Sequence length 2,048 therefore requires $16\times$ more materialized score memory than length 512. Systems that ignore these characteristics can miss latency targets, exceed memory budgets, and deliver much lower hardware utilization than expected.

Fallacy: *An architecture has one dominant bottleneck across training and inference.*

The same computation graph can occupy different systems regimes. Transformer training and prefill process many positions together and may bind on dense attention compute; small-batch autoregressive decoding repeatedly streams weights and KV-cache state and may bind on memory bandwidth. Batching, sequence length, precision, and cache layout can shift that boundary. A benchmark from one regime therefore cannot establish another's bottleneck; measure each against the iron law.

Pitfall: *Combining architectural patterns without analyzing interaction effects at the system level.*

Engineers add attention to CNNs or convolutions to transformers expecting additive benefits. Each pattern creates distinct memory access characteristics: CNNs exploit spatial locality through sliding windows, while dense attention introduces all-to-all score computation. Combining them can increase intermediate traffic and disrupt locality, so the hybrid's throughput cannot be inferred by adding the components' benefits. Adding recurrent connections to transformers likewise reintroduces sequential dependencies. Successful hybrids require profiling memory access and cache behavior before combining patterns.

Fallacy: *Architecture wins on training hardware transfer directly to deployment hardware.*

Teams design for high-end GPU clusters, then discover deployment failures on target hardware. An architecture exploiting $8\times$ A100 GPUs (640 GB total memory) cannot deploy unchanged to a representative edge node such as the NVIDIA Jetson Orin NX (16 GB system memory). As section 6.10.3 emphasizes, architecture selection must analyze the full system stack because storage, latency, and power budgets vary by deployment target.

Pitfall: *Ignoring KV cache growth when estimating transformer serving costs.*

Teams budget transformer deployment based on model weight memory alone, overlooking the key-value (KV) cache used during autoregressive generation (Pope et al. 2023; Kwon et al. 2023). The cache scales as $\mathcal{O}(B \times N_L \times 2 \times N_{\text{heads}} \times S \times d_{\text{head}})$, where B is the number of concurrent sequences, N_L is the layer count, the factor 2 stores keys and values, N_{heads} is the head count, S is sequence length, and d_{head} is head dimension. At long contexts or high concurrency, this overhead can become a binding serving constraint. Section 6.6.4.2 works through the chapter's 32-layer, 32-head example step by step. With 128-dimensional heads, 2,048-token sequences, and FP16 storage, each concurrent request holds ≈ 1 GB of KV cache. At 2–4 users, the cache alone consumes 2 GB–4 GB before allocator, activation, and workspace overhead. Those values do not exhaust a 80 GB device, but they consume part of the post-weight headroom and grow linearly with context and concurrency. Capacity planning must therefore budget weights, cache, and runtime buffers together, then reduce concurrency, context, or cache footprint, or add memory capacity, when the combined total approaches device memory.

6.12 Summary

Architecture is infrastructure. Choosing among MLPs, CNNs, RNNs, transformers, and recommendation models (DLRM) fixes the physical terms in the serialized form of the iron law of ML systems ($T_{\text{exec}} = D_{\text{vol}}/\text{BW} + \mathcal{O}/(R_{\text{peak}} \cdot \eta_{\text{hw}}) + L_{\text{lat}}$). Examining each architecture through the four-part lens (pattern processing needs, algorithmic structure, computational mapping, and system implications) reveals that an architecture's mathematical inductive bias strongly shapes which term of the iron law binds hardware performance.

The architectural families and five lighthouse models established at the chapter opening map directly onto these physical bounds. ResNet-50's spatial weight reuse inflates arithmetic intensity ($I = O/D_{\text{vol}}$), pushing execution into the compute-bound regime (O/R_{peak}). Small-batch autoregressive GPT-2 generation has low arithmetic intensity, binding execution to memory bandwidth (D_{vol}/BW). RNN temporal recurrences impose an unparallelizable sequential critical path that raises the latency floor (L_{lat}). DLRM sparse embeddings hit a capacity wall outside the iron law: their tables can exceed accelerator memory before either execution term becomes the first feasibility test. MobileNetV2 and KWS demonstrate how low FLOP counts do not guarantee throughput if operational shapes fail to saturate hardware functional units.



Key Takeaways: Architecture is infrastructure

- **Inductive bias is the unifying concept:** Every architecture encodes structural assumptions, such as locality for CNNs, sequence for RNNs, and global context for transformers. These biases trade generality for sample efficiency and determine which problems an architecture can solve efficiently.
- **Arithmetic intensity helps identify the bottleneck:** Comparing a workload's arithmetic intensity with the target hardware's roofline balance helps determine whether compute or memory bandwidth is likely to bind.
- **Quadratic costs are permanent constraints:** Dense transformer attention computes $\mathcal{O}(S^2)$ score interactions. Naive implementations also use quadratic score storage; tiled exact attention removes that storage cost but not the dense computation.
- **Lighthouse models isolate distinct bottlenecks:** ResNet-50 (compute), GPT-2 (bandwidth), DLRM (capacity), MobileNetV2 (latency), KWS (power). These archetypes diagnose which physical constraint dominates a given system.
- **Depth benefits from architectural support:** Skip connections and normalization improve optimization in very deep networks rather than acting as universal prerequisites beyond a fixed depth. These building blocks, born in CNNs, transfer to many deep architectures, including transformers.
- **FLOPs do not equal speed:** MobileNetV2 uses $13.7\times$ fewer FLOPs than ResNet-50 but may run slower on some data center GPUs when its operation shapes and arithmetic intensity use the available compute units poorly. Architecture-hardware alignment, not operation count alone, determines throughput.
- **Architecture selection is deployment selection:** Choosing a transformer over a CNN determines memory requirements, latency floors, hardware utilization, and infrastructure costs. The architecture is the system constraint.

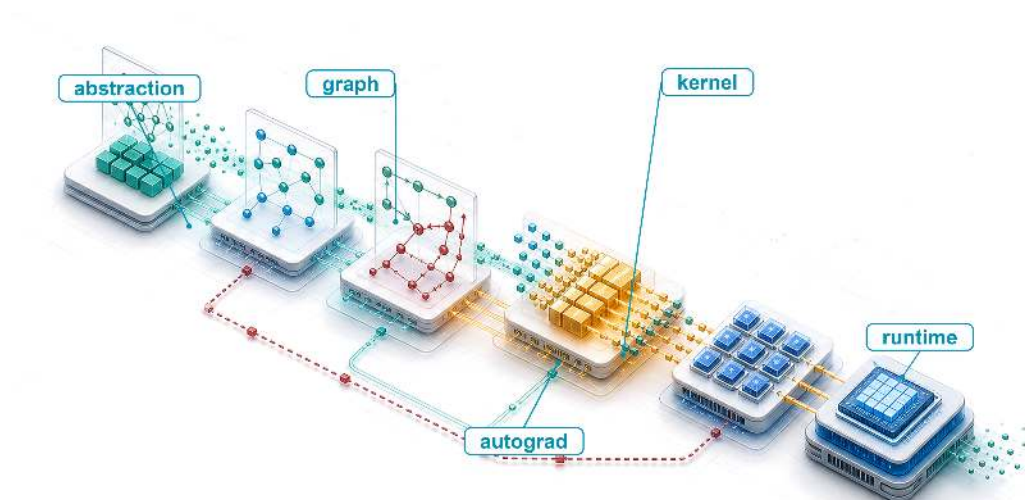
Inductive bias and systems cost are two views of the same choice. The assumptions that help a model generalize from limited samples, such as locality, recurrence, or unrestricted token interactions, also determine the shape, reuse, and lifetime of intermediate data. An algorithmic choice therefore propagates into memory capacity, bandwidth, and scheduling requirements before a framework chooses kernels.

Architecture selection signs a physical contract with hardware. A CNN commits to spatial locality and weight reuse; dense attention commits to $\mathcal{O}(S^2)$ score computation; and an RNN commits to serial time-step dependencies. These topological choices fix the fundamental workload terms O , D_{vol} , and L_{lat} , thereby governing memory footprint, bandwidth demand, and parallel efficiency. Runtime techniques can improve how efficiently the contract is paid but cannot erase it: tiled exact attention removes intermediate score storage without changing the quadratic arithmetic floor, while pipeline parallelism partitions recurrent work without removing the serial time-step path. Systems engineers can therefore anticipate likely bottleneck shifts before implementation by matching model inductive bias and data geometry to silicon physics.

 What's Next: From blueprints to construction

Raw computational primitives (chapter 5) become concrete architectural contracts in this chapter. Translating an abstract blueprint into an executable runtime requires bridging mathematical graphs to physical hardware. The architecture fixes *what* work must be performed; the machine learning framework determines *how* that work is scheduled and dispatched. Chapter 7 examines how PyTorch, TensorFlow, and JAX convert high-level architectural graphs into computational directed acyclic graphs (DAGs), autograd graphs, and optimized device kernels that execute directly on silicon.

ML Frameworks

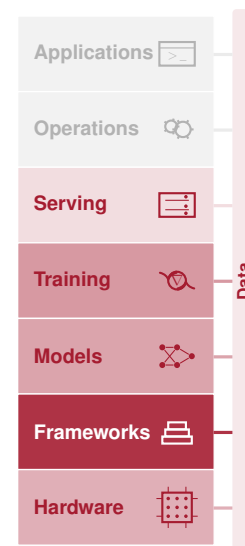


- 7.1 Three Framework Problems
- 7.2 The Ladder of Abstraction
- 7.3 Execution Problem
- 7.4 Differentiation Problem
- 7.5 Abstraction Problem
- 7.6 nn.Module Abstraction
- 7.7 Framework Platform Analysis
- 7.8 Deployment Targets
- 7.9 Framework Selection
- 7.10 Anatomy of a Training Step
- 7.11 Fallacies and Pitfalls
- 7.12 Summary

Purpose

Why does the framework silently constrain every decision that comes after it?

Model equations do not execute themselves. A framework translates an architecture into work that hardware can run, and that translation is not neutral. It decides when operations execute, what intermediate state training retains, how memory is allocated, and which deployment targets can use the resulting artifact. These choices shape more than speed: they determine what can be debugged directly, what the compiler can optimize, and whether the same model survives the path from a research environment to a production runtime. The abstractions that make experimentation convenient therefore create long-lived commitments. Once a project accumulates checkpoints, modules, data loaders, serving formats, and team expertise around a framework, changing it becomes a system migration rather than a library substitution. A framework well suited to rapid exploration can expose too little structure for efficient deployment, while a constrained execution path can sacrifice the flexibility needed during discovery. Because execution, differentiation, and abstraction meet in the same software layer, a local convenience can become a downstream performance or portability constraint. The engineering question is not which framework is universally best, but which trade-offs the workload, team, and target machine can afford to make fixed. Understanding frameworks is therefore about reasoning through those system choices rather than learning transient API syntax. In D·A·M terms, the framework mediates algorithm-machine co-design by turning algorithmic intent and data representation into executable work under machine constraints.





Learning Objectives

- Explain how frameworks mediate algorithm-machine co-design through execution, differentiation, and hardware abstraction
- Compare execution strategies using dispatch overhead, compilation cost, and deployment constraints
- Analyze automatic differentiation and activation storage to choose recomputation or checkpointing
- Implement module abstraction patterns for parameter discovery, mode behavior, serialization, and hooks
- Calculate training-step FLOPs, memory traffic, and dispatch overhead to identify fusion and layout bottlenecks
- Select TensorFlow, PyTorch, JAX, or edge runtimes based on model, hardware, team, and deployment requirements

7.1 Three Framework Problems

An architecture is a graph of commitments: a transformer commits the system to attention, matrix multiplications, activation state, and memory traffic. The framework is the layer that turns that graph into work the machine can execute. A few calls such as `logits = model(tokens)`, `loss = criterion(logits, targets)`, and `loss.backward()` hide billions of floating-point operations across memory hierarchies, gradients through millions of parameters via automatic differentiation, thousands of GPU kernel launches, and gigabytes of intermediate state. The API looks simple because the framework is acting as a compiler for the silicon contract.

Architectures specify what computations neural networks perform, but knowing *what* to compute is entirely different from knowing *how* to compute it efficiently. A transformer’s attention mechanism requires coordinating computation across memory hierarchies and accelerator cores in patterns where naive implementations execute $100\times$ slower than optimized ones due to uncoalesced memory accesses and repeated round-trips to high-bandwidth memory (HBM). Implementing these operations from scratch for every model would make deep learning economically infeasible. ML frameworks exist to bridge this gap by lowering abstract model graphs into hardware-aligned execution pipelines that maximize compute unit utilization while minimizing off-chip memory traffic.

A framework is to machine learning what a compiler is to traditional programming. A C compiler translates human-readable code into optimized machine instructions, managing register allocation, instruction scheduling, and memory layout. An ML framework translates high-level model definitions into hardware-specific execution plans, managing operator fusion, memory reuse, and device placement. The correspondence is operational rather than figurative: modern frameworks incorporate optimizing compilers (such as Accelerated Linear Algebra (XLA) or TorchInductor) that lower graph representations directly to target machine code.

Every ML framework, regardless of API or design philosophy, must solve three core problems. The first is the *execution problem*: deciding when and how computation runs. A framework can execute operations immediately as written (eager execution¹) or construct a computational graph²—a structured intermediate representation of operations and data dependencies—to optimize before launching device kernels (graph execution). This choice determines the fundamental trade-off between interactive debugging and whole-program compiler optimization.

With an execution strategy in place, the framework must solve the *differentiation problem*: computing gradients automatically. Because gradient-based training requires derivatives of a scalar loss with respect to millions or billions of parameters (chapter 5), manual calculus is impossible at scale. Frameworks implement automatic differentiation by registering derivative rules across supported operator compositions, chaining vector-Jacobian products while managing numerical precision and the memory overhead of intermediate activations retained for backward passes.

The third problem is *hardware abstraction*: targeting diverse hardware from a single interface. The same model definition should be expressible across CPUs, GPUs, Tensor Processing Units (TPUs),

¹ | **Eager execution:** This mode executes each operation immediately, which enables direct debugging with standard tools but sacrifices the global view needed for graph-level optimizations. Without capturing a larger region of computation, an eager runtime cannot fuse across that region or preplan its memory, leaving workload-dependent optimization opportunities for compilers such as `torch.compile`.

² | **Computational graph:** The “optimize before executing” distinction in the triggering sentence is the key design choice. Capturing the program as a data structure lets a framework fuse compatible operations into fewer GPU kernels before execution, reducing launch overhead and intermediate memory traffic. The engineering cost of this visibility is that the executed program differs from the source code, making debugging harder, a trade-off every graph-based framework must justify against the performance gain.

and mobile accelerators, even though each target presents distinct memory hierarchies, instruction sets, and interconnect bandwidths. Within the GPU domain, framework ecosystems increasingly expose custom-kernel languages such as Triton (Tillet et al. 2019), allowing developers to write high-performance, block-level kernels without hand-crafting low-level CUDA or device assembly.

These three problems are tightly coupled: the execution model dictates when differentiation data structures are recorded, while the hardware abstraction layer determines which graph transformations can be lowered to silicon. Unifying execution, differentiation, and hardware abstraction under physical machine constraints requires evaluating frameworks through the governing equation of machine learning performance.

Systems Perspective 7.1: The ML compiler

In the context of the iron law (section 1.7), a framework is a compiler for the silicon contract. The “source code” is the model architecture (the O term). The framework’s job is to take this high-level mathematical specification and compile it into a series of hardware-specific kernel launches that:

1. Minimize data movement (D_{vol}) through techniques such as kernel fusion.
2. Maximize utilization (η_{hw}) by matching operations to specialized hardware units such as Tensor Cores.
3. Minimize overhead (L_{lat}) through efficient asynchronous dispatch and graph capture.

Choosing a framework means choosing the compiler that dictates hardware efficiency. Precision matters here: a definition capturing all three responsibilities separates genuine frameworks from numerical libraries that address only one.

Definition 7.1: Machine learning frameworks

Machine learning frameworks are software systems that translate high-level mathematical model definitions into hardware-optimized execution plans by managing the computational graph, automatic differentiation, kernel dispatch, and memory allocation across the hardware hierarchy.

1. **Significance:** Frameworks directly determine the system efficiency (η_{hw}) term in the iron law. Compiler-backed operator fusion, for example, can eliminate writes and subsequent reads of intermediate values between compatible operations. Fusing a matrix multiplication, bias add, and rectified linear unit (ReLU) changes how the same mathematics reaches hardware without changing the model.
2. **Distinction:** Unlike a numerical library such as NumPy, which normally evaluates each operation as it is called, an ML framework can capture or defer execution to analyze a larger computational graph and apply cross-operation optimizations: operator fusion, memory layout transformations, and parallel scheduling. Those optimizations require visibility beyond separately dispatched operations.
3. **Common pitfall:** A frequent misconception is that frameworks are interchangeable API wrappers. Framework choice determines which compiler paths are available. PyTorch can recover graph-level optimization from eager code through `torch.compile()`, while TensorFlow and JAX commonly rely on XLA-backed compilation paths that lower operations to target hardware. Moving from eager execution to a compiler path can change throughput materially, but the gain depends on model structure, shapes, operator support, and hardware.

An ML framework translates logical intent into physical execution under the constraints of the iron law, deciding how to partition computation across memory hierarchies, when to trade numerical precision for throughput, and how to schedule operations so that the dominant term—data

movement (D_{vol}), arithmetic computation, or launch latency (I_{lat})—is minimized. The framework is where the governing physics developed throughout this book becomes executable code.

The scale of this translation becomes apparent beneath the API surface. A single call to `loss.backward()` triggers operation recording, memory allocation for gradients, reverse-order graph traversal, and hardware-optimized kernel dispatch—machinery that would require substantial manual bookkeeping even for a three-layer network. For a contemporary language model, the framework additionally orchestrates billions of floating-point operations across accelerators, coordinating memory hierarchies, numerical precision, and, when the system grows beyond one device, communication libraries. Building this from scratch would be economically prohibitive for most organizations, which is why the history of ML frameworks is a history of progressively automating these layers.

The three problems—execution, differentiation, and abstraction—did not emerge simultaneously. Each arose as a response to scaling limitations in the previous generation of tools, creating a historical progression of abstractions from low-level linear algebra routines to compiled graph runtimes.

7.2 The Ladder of Abstraction

In 1979, writing a matrix multiplication in Fortran that used the hardware efficiently required deep knowledge of cache lines, register scheduling, and vector units. By 2016, a single line of Python (`torch.matmul(A, W)`) could dispatch to a highly optimized implementation without the programmer knowing the details of the silicon. That compression of effort did not happen in one step; it accumulated across four decades of abstraction, each layer solving a bottleneck that made the previous generation impractical for scaling. The result is a **ladder of abstraction** where each rung automates what the rung below exposed.

The first rung established reusable hardware primitives (1979–1992). The original Basic Linear Algebra Subprograms (BLAS)³ standardized low-level linear-algebra operations (Lawson et al. 1979), while LAPACK⁴ (Bai et al. 2006) built higher-level numerical factorizations and eigensolvers on top of that foundation. Naive nested loops fail to exploit memory hierarchies: without cache tiling and register blocking, memory bandwidth saturates long before arithmetic units reach peak throughput. BLAS and LAPACK solved this hardware-primitive bottleneck by establishing stable interfaces that let numerical software delegate compute-heavy operations such as matrix multiplication ($C = A @ B$)⁵ to vendor libraries tuned for specific cache line sizes, single instruction, multiple data (SIMD) widths, and register files, rather than requiring hand-written assembly for every target architecture.

The second rung introduced unified array abstractions to unlock developer velocity (2005–2006). NumPy⁶ addressed the productivity barrier of low-level languages by wrapping BLAS and LAPACK in high-level Python (Harris et al. 2020). Under NumPy’s strided array model, Python manages control flow and tensor metadata (shape, strides, and data type), while the underlying C runtime dispatches vector and matrix loops across contiguous memory buffers. This vectorization contract—orchestrating high-level algorithmic intent in an interpreted language while delegating inner loops to compiled native routines—became the architectural template for scientific and numerical computing.

The third rung automated gradient computation (2007–present). Deep learning frameworks such as Theano,⁷ TensorFlow (Abadi et al. 2016), and PyTorch (Paszke et al. 2019) solved the scalability limits of manual differentiation. While numerical packages required practitioners to derive and implement analytical backpropagation gradients by hand—a process error-prone and unmanageable for deep networks—modern frameworks turned automatic differentiation over computational graphs into a core runtime primitive. By capturing operations in a directed acyclic graph during the forward pass, these frameworks automatically construct the backward graph to evaluate vector-Jacobian products, transforming the calculus of the chain rule into automated reverse-mode execution.

This progression traces four decades of numerical computing abstractions, moving from assembly-tuned subroutines to compiled graph execution engines (figure 7.1).

Each generation abstracted away details that consumed engineering effort in the previous one, yet each abstraction introduced new trade-offs. BLAS hid assembly-level optimization but fixed the interface. NumPy hid memory management but required manual differentiation. Modern frameworks hide gradient computation but introduce a choice among execution models. The deeper pattern is that every abstraction hides details by preserving a contract about shape, dtype, device,

³ | **BLAS (basic linear algebra subprograms):** The 1979 API specification that forms the bottom rung of the ladder described here; it standardized a fixed set of Fortran-callable vector operations, separating the public routine names from the machine-specific implementation decisions beneath them. Every framework above it inherits the broader version of this bargain: call a standard linear-algebra primitive from any language and let a tuned vendor library target the silicon. For modern general matrix multiply (GEMM) on NVIDIA GPUs, that tuned path is cuBLAS rather than the 1979 BLAS specification itself (NVIDIA 2024a).

⁴ | **LAPACK (linear algebra package):** Extends BLAS by providing a standard API for higher-level routines (SVD, eigendecomposition, least-squares) that vendors implement with chip-specific code layered on top of fast GEMM kernels. This layered design is the architectural pattern every ML framework inherits: high-level operations delegate downward to hand-tuned primitives, so a vendor-optimized LAPACK call can execute over $10\times$ faster than a naive implementation without the framework author writing a single line of hardware-specific code.

⁵ | **GEMM:** The matrix-matrix primitive behind $C = A @ B$. Hardware vendors hand-tune GEMM for their specific chips because dense layers, attention projections, and convolution lowering all rely on matrix multiplication, making this one routine a performance floor for many frameworks above it on the ladder. Its high arithmetic intensity makes GEMM the operation most able to approach peak compute throughput, while small or misaligned shapes often fall back to much lower utilization.

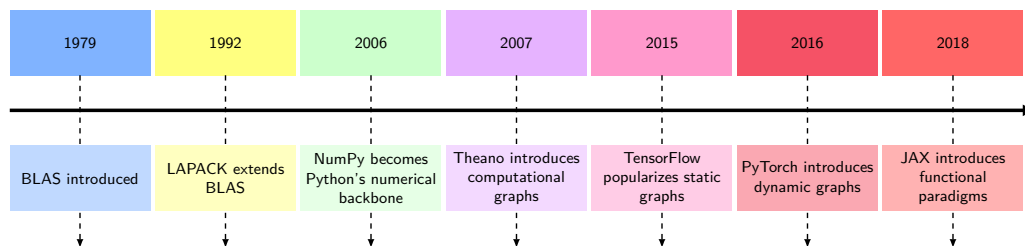


Figure 7.1: Computational Library Evolution: A timeline of numerical computing milestones spanning 1979 to 2018. Each era automates what the layer below exposed—from BLAS and LAPACK hardware routines to NumPy vectorized syntax and graph-based automatic differentiation in modern frameworks—trading low-level transparency for developer velocity.

and sometimes units; when that contract becomes implicit, correct components can still compose into a wrong system.

⚠ War Story 7.1: The interface that forgot its units (1999)

Context: NASA's Mars Climate Orbiter depended on ground software, spacecraft software, navigation teams, and contractor interfaces all agreeing on the physical meaning of exchanged values (Mars Climate Orbiter Mishap Investigation Board 1999).

Mechanism: Ground software supplied by Lockheed Martin computed thruster impulse in English units (pound-force seconds), while JPL navigation software consumed the values assuming SI units (newton-seconds)—a $4.45\times$ implicit unit mismatch.

Impact: On September 23, 1999, the spacecraft was lost after entering occultation. Its corrected 57 km periapsis, rather than the planned 226 km, was judged too low for survival.

Response: The board recommended unit-consistency checks, specification audits of transferred data, and stronger end-to-end verification of ground software.

Systems lesson: Framework abstractions are valuable because they carry contracts: shape, dtype, device placement, and physical units. If those contracts are implicit, two pieces of correct code can still compose into a wrong system. ML frameworks hit this whenever tensor memory layouts (such as NCHW vs. NHWC), quantization scale factors, or physical feature units differ across pipeline stages, producing plausible outputs that quietly corrupt downstream inference.

With that contract risk in view, modern frameworks converge on three core problems: *how* to execute computation, *how* to differentiate it, and *how* to abstract across hardware. The execution problem comes first because its resolution determines what optimizations the other two problems can exploit.

7.3 Execution Problem

Consider two engineers writing the same neural network. The first debugs interactively, printing tensor shapes after each operation, inspecting intermediate values, and stepping through code with `pdb`. The second waits 30 seconds for compilation, then watches the model run $3\times$ faster while losing that direct, line-by-line view of intermediate state. Both are correct; they have made different choices about the **execution problem**, the question of whether operations should execute immediately as written or be recorded for later execution. This choice creates a cascade of engineering trade-offs that shape every aspect of framework behavior, from debugging workflows to deployment options to peak hardware utilization.

7.3.1 Why execution strategy matters: The memory wall

The execution strategy of an ML framework is fundamentally constrained by the widening compute-bandwidth gap quantified by the memory-wall equation (equation 2.3). Processor arithmetic has scaled far faster than memory bandwidth, creating the memory wall. Modern accelerators can perform arithmetic far faster than they can fetch data. Element-wise operations such as ReLU use

6 | NumPy (numerical Python): In 2005, Travis Oliphant unified two competing Python array libraries (Numeric and Numarray) into a single package, giving the scientific computing community one BLAS-backed array standard at the moment it needed to scale. The “vectorization” contract this created (write logic in Python, execute loops in C/Fortran via BLAS) became the design template for every ML framework that followed: PyTorch tensors and TensorFlow arrays are direct descendants, extending the same n -dimensional array abstraction to GPUs. Python’s role in ML infrastructure inherits much of its shape from this consolidation decision.

7 | Theano: Developed at the Montreal Institute for Learning Algorithms (MILA) under Yoshua Bengio starting in 2007, Theano was an early and influential Python framework that compiled symbolic mathematical expressions into optimized CPU and GPU code via computational graphs (Bergstra et al. 2010; Team et al. 2016). It demonstrated that a Python-defined computation graph could be compiled for GPU execution without requiring the researcher to write CUDA code. Mila ended active development of Theano in 2017 (Mila 2026).

only a tiny fraction of peak compute capacity, not because the hardware is slow, but because they spend nearly all their time waiting for data. Section D.2.1 formalizes this trade-off, showing exactly when operations are memory bound vs. compute bound.

The memory wall classifies operations as either compute-bound (limited by arithmetic throughput) or memory-bound (limited by data movement). Most individual neural network operation types (activations, normalizations, element-wise operations) are memory bound, though the large matrix multiplications that dominate total compute time can be compute bound.

The key optimization for memory-bound operations is kernel fusion, combining multiple operations into a single GPU function (called a *kernel*)⁸ to avoid intermediate memory traffic. Fusing a sequence of normalization, dropout, and activation operations into one kernel can yield large speedups by eliminating intermediate writes between operations. Attention kernels⁹ use the same principle at larger scale: instead of materializing the full attention matrix in HBM, a fused implementation can keep tiles close to the compute units, cut HBM accesses by up to $9\times$, and produce workload-specific speedups from 15 percent to $3\times$ (T. Dao et al. 2022).

Frameworks can fuse only operations visible together. Separately dispatched eager operations hide cross-operation opportunities, but graph capture can expose them. A deferred graph lets the framework optimize the captured computation. Execution strategy therefore determines which optimizations are possible and their scope.

7.3.2 The computational graph

To expose multi-operation optimization opportunities to compilers, frameworks represent programs as a computational graph: a directed acyclic graph (DAG) where nodes represent operations and edges represent data dependencies. This graph serves as the framework’s internal model of the computation.

Mathematical operations can be decoupled from physical execution through graph representations. The computational graph (figure 7.2) grounds this abstraction: tensor variables map to data nodes while operations map to transformation nodes.

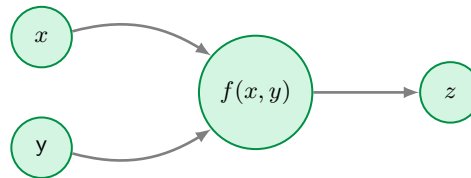


Figure 7.2: Simple Computational Graph: Representing $z = f(x, y)$ as a directed acyclic graph decouples mathematical dependencies from physical execution order, providing the structural blueprint frameworks require to track gradients and schedule operations across devices.

Real machine learning models require much more complex graph structures. Figure 7.3 extends this representation to show a neural network computation graph alongside the system components that reason about it. The left panel depicts data flowing through six operation nodes in a directed acyclic graph, where each node’s output becomes the next node’s input. The right panel shows how the runtime interfaces with this graph: querying tensor lifetimes to plan memory reuse and assigning operations to accelerator streams based on data dependencies rather than program order. The critical insight is that the graph exists independently of execution, enabling the framework to optimize before any arithmetic occurs.

This graph representation is more than a visualization; it is the data structure that enables both efficient execution and automatic differentiation. The timing of its construction forces a fundamental systems trade-off across debugging, optimization, deployment, and flexibility. Interactive debugging demands immediate execution so developers can inspect intermediate values and step through logic with standard tools. Global optimization requires observing large operator sequences at once to fuse kernels and plan memory reuse. Standalone deployment favors graphs that run without the host interpreter, while dynamic architectures require control flow that branches on computed tensor values.

No single execution model optimizes all these dimensions. Frameworks must choose their position in this trade-off space, and practitioners must understand these trade-offs to select appropriate tools

⁸ | **Kernel (GPU):** In GPU programming, a kernel is the function dispatched to execute in parallel across thousands of threads. Each kernel launch incurs 5–20 μs of CPU-side overhead for parameter assembly and GPU signaling, which means that small, unfused operations spend more time on launch overhead (L_{lat}) than on useful arithmetic. Reducing kernel count through fusion is therefore a direct attack on the overhead term of the iron law.

⁹ | **Fused attention kernel:** A fused attention kernel combines the $\mathbf{QK}^T$ product, softmax, and value-weighted output into a tiled implementation that keeps intermediate values in on-chip memory rather than materializing the full attention matrix in HBM. FlashAttention is the canonical named example introduced in chapter 6 and reports up to $9\times$ fewer HBM accesses with workload-specific speedups from 15 percent to $3\times$ (T. Dao et al. 2022). The framework lesson is broader than the specific algorithm: fusion can shift an operation’s position on the Roofline Model from bandwidth-limited toward throughput-limited execution by reducing round-trips through external memory.

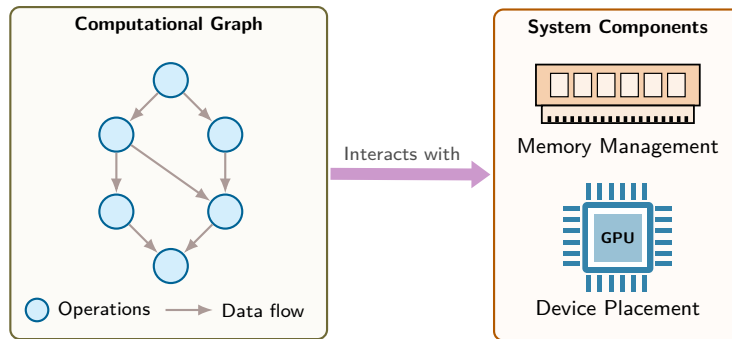


Figure 7.3: Computation Graph with System Interactions: A neural network DAG (left) interfaces with framework runtime components (right). By maintaining an explicit graph representation independent of execution, the framework can analyze tensor lifetimes for memory reuse and assign operations to target accelerators before issuing arithmetic.

and write efficient code. The three execution families that follow are different answers to the same systems question: how much graph visibility should the framework trade for immediate execution and debugging?

7.3.3 Three execution strategies

The computational graph enables global optimization, but it leaves a critical design choice unresolved: *when* the runtime constructs the graph. Consider an elementary expression such as $y = x * 2$. Evaluating the multiplication immediately yields a concrete tensor in y . This imperative flow is intuitive and directly debuggable, but the runtime sees only one operation at a time. Deferring execution records the operation symbolically into a dependency graph executed only upon request. That staging separates intent from evaluation, giving the compiler the end-to-end visibility required for operator fusion and memory planning.

Neither approach dominates; each embodies different trade-offs between *flexibility* and *optimization potential*. Modern frameworks have explored three primary execution strategies: eager execution with dynamic graphs, static computation graphs, and hybrid approaches that combine just-in-time (JIT) compilation with eager development. Each strategy has distinct systems implications.

7.3.3.1 Eager execution with dynamic graphs

Eager execution evaluates each operation immediately as the program calls it, building the computation graph dynamically at runtime. A side-by-side comparison shows how this differs from graph-based execution at the code level.

Example 7.1: Eager vs. graph execution code comparison

PyTorch (eager execution):

```
import torch

x = torch.tensor([1.0, 2.0])
y = x * 2
print(f"Intermediate value: {y}") # Works immediately
z = y.sum()
```

TensorFlow 1.x (static graph):

```
import tensorflow as tf

x = tf.placeholder(tf.float32)
y = x * 2
# print(y) -> Prints Tensor("mul:0"...), not value!
z = tf.reduce_sum(y)

with tf.Session() as sess:
    result = sess.run(z, feed_dict={x: [1.0, 2.0]})
```

Systems insight: Eager execution exposes intermediate values as ordinary runtime state, which makes debugging direct. Static graphs stage computation before execution, which enables whole-graph optimization but changes the debugging model.

In this define-by-run model, when host code evaluates $y = x * 2$, the runtime dispatches the multiplication kernel immediately to the device command queue and makes the resulting tensor available for subsequent operations without delay.

This provides the flexibility of normal programming: developers can print intermediate values, use conditionals based on computed results, and debug with standard tools. The framework records operations as they happen, constructing a dynamic graph that reflects the actual execution path taken.

For gradient computation, the framework records a history of operations in what is called an autograd tape,¹⁰ a transient data structure built during execution. Each tensor operation creates a node that records: the operation performed, references to input tensors, and how to compute gradients. These nodes form a DAG that records the actual path taken during forward-pass execution rather than a graph fixed in advance. Listing 7.1 shows how PyTorch records operations as they execute in its default eager mode.

Listing 7.1: Autograd Tape Construction: Each operation executes immediately while recording a backward node to the autograd tape for later gradient computation.

```
import torch

x = torch.tensor([1.0], requires_grad=True)
y = x * 2 # Executes immediately; records MulBackward node
z = y + 1 # Executes immediately; records AddBackward node
# The autograd tape exists NOW, built during execution
```

After these two operations, the framework has constructed an autograd tape with two nodes: one for the multiplication and one for the addition. The tape records that z depends on y , and y depends on x .

Calling `z.backward()` traverses this tape in reverse topological order, applying the chain rule at each node:

1. Compute $\frac{\partial z}{\partial z} = 1$ (seed gradient)
2. Call `AddBackward0.backward()` $\rightarrow \frac{\partial z}{\partial y} = 1$
3. Call `MulBackward0.backward()` $\rightarrow \frac{\partial z}{\partial x} = 2$
4. Accumulate gradient in `x.grad`

After `backward()` completes, the autograd graph is normally released. The next forward pass builds a new graph. Values needed for gradients are saved during the forward pass and remain live until backward consumes them, so their memory cost spans both phases rather than appearing only during backward.

¹⁰ | **Autograd tape:** A transient data structure built during forward execution, where nodes record operations, dependencies, and backward functions for chain-rule evaluation. Its memory footprint grows with model depth and sequence length as saved activations accumulate until released during backpropagation. For deep models, frameworks retain selected activations and recompute others via activation checkpointing to prevent OOM failures.

Example 7.2: In-place operations can break gradients

Scenario: An engineer applies an in-place mutation (`x += 1`) within a custom PyTorch activation function to reduce memory allocations.

Diagnosis: In-place operations overwrite tensor memory containing forward-pass activations required by the autograd tape for backward-pass gradient computation, triggering a runtime version-counter error (PyTorch Contributors 2026a). PyTorch enforces this safety invariant by maintaining an integer version counter on each tensor that increments on every in-place write; if a saved tensor's version at backward execution differs from its version recorded during the forward pass, autograd throws an error because the activation required by the derivative rule was modified in place.

Systems lesson: Framework automatic differentiation depends on immutable forward-pass activation records. Unchecked in-place memory mutations break autograd tape dependencies, forcing runtime framework execution halts.

In eager execution frameworks, operations evaluate imperatively as host code encounters them. Figure 7.4 traces this execution loop, illustrating the define-and-dispatch sequence that sends individual operations directly to device stream queues.

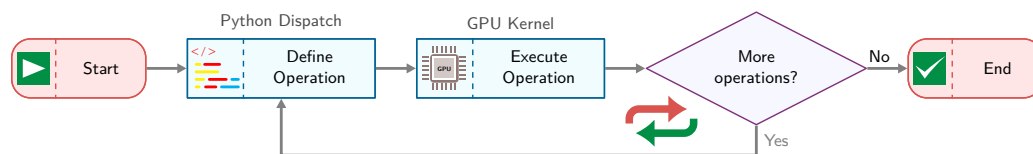


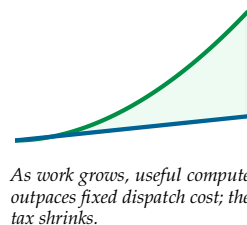
Figure 7.4: Dynamic Graph Execution Flow: In eager execution, each operation is issued as the program encounters it. Accelerator work may execute asynchronously after dispatch. This define-by-run model enables natural debugging and data-dependent control flow while limiting optimization across Python calls.

Systems implications: Flexibility The dynamic autograd tape expresses data-dependent behavior directly in the host language. Conditionals and loops can depend on tensor values computed during execution, enabling algorithms such as beam search, dynamic-length recurrent models, or adaptive compute paths that adjust their execution based on intermediate activations. Static graphs can also represent data-dependent control flow and dynamic dimensions through graph operations, but eager execution makes these patterns natural to write and debug in Python. Because operations are issued immediately, developers can print tensors, inspect values, and use standard debuggers (pdb, breakpoints) to diagnose errors much as they would in another Python program.

Systems implications: Overhead This flexibility imposes latency costs that map directly to the iron law (section 1.7). Each training step constructs an autograd tape dynamically, adding host-side Python dispatch latency and tape allocation to L_{lat} . Because every operation traverses the framework dispatcher before a device kernel launches, host overhead quickly dominates when individual kernels execute in single-digit microseconds. Eager execution also lacks a whole-graph view, preventing the framework from fusing element-wise chains into single GPU launches or pre-allocating a static memory arena for intermediate activations (D_{vol}). The autograd tape must retain every tensor requested by registered backward rules, holding memory until backpropagation consumes it. Together, these costs impose a throughput ceiling on fine-grained workloads where dispatch latency overshadows device arithmetic.

The dispatch tax: Python overhead vs. GPU reality Eager execution's performance ceiling is driven by a fundamental systems mismatch: the speed of the host-side interpreter vs. the speed of the device-side silicon. The dispatch tax, defined as the fraction of time spent in the host-side orchestration (Python) vs. actual device execution (GPU), quantifies this mismatch.

In the illustrative eager-execution model used here, every operation pays a fixed "tax" of approximately $15\ \mu s$ for host-side dispatch, type checks, and kernel launch. Actual overhead varies by framework, backend, and hardware, but its relative weight still depends on operation size. For



a small operation such as a ReLU on a small vector, the kernel might execute in only $1\ \mu\text{s}$, so the modeled dispatch tax reaches 94 percent and the GPU spends most of its time waiting for the next command. For a large operation such as a large matrix multiply, the kernel executes for $100\ \mu\text{s}$, the modeled dispatch tax drops to 13 percent, and the system becomes compute bound.

The dispatch tax explains why models with many small layers run significantly slower than their raw FLOP count predicts. Section A.5.1 places this symptom in the bottleneck taxonomy, classifying a dispatch-dominated workload as latency-bound rather than compute-bound and showing which optimizations actually move it. To approach efficient execution, frameworks must move from *kernel-by-kernel dispatch* to *graph-level execution*, where the dispatch tax is paid once for the entire graph rather than per operation. The hybrid JIT and compilation strategies in section 7.3.3.3 exist precisely to address this overhead.

The overhead costs of eager execution motivate the opposite design: capturing the entire computation before executing any of it. This is precisely what static computation graphs provide.

7.3.3.2 Static computation graphs

Static graph execution defines the complete computational graph as a symbolic representation first, then executes it separately. This “define-then-run” execution model means the graph exists before any computation occurs, enabling aggressive ahead-of-time optimization. Seeing the complete computation before launching device kernels allows the framework to analyze, transform, and optimize the graph globally—visibility unavailable across separate eager calls unless a compiler captures them. Section C.3.4.2 works through the canonical global transformation this enables: for N elements of b bytes each, fusing a chain of k elementwise operations into one kernel reduces idealized read-and-write traffic from $2kNb$ to $2Nb$ bytes.

Two-phase execution Static graphs implement a clear separation between graph construction and execution. Listing 7.2 illustrates the two phases using TensorFlow 1.x, which exemplified this approach. It deliberately runs the same $x * 2$ then $+ 1$ computation shown under eager execution in listing 7.1, holding the arithmetic fixed so that the only thing that changes is *when* it executes: symbolic definition creates placeholders and operations without computation, while explicit execution triggers actual arithmetic.

Listing 7.2: Static Graph Two-Phase Execution: Graph construction (symbolic definition) is separated from execution (actual computation), enabling ahead-of-time optimization.

```
# Phase 1: Graph Construction (symbolic, no computation)
import tensorflow.compat.v1 as tf

tf.disable_v2_behavior()

# Define graph symbolically
x = tf.placeholder(tf.float32, shape=[1]) # Just a placeholder
y = x * 2 # Not executed, just recorded
z = y + 1 # Still no execution
# At this point, nothing has been computed

# Phase 2: Graph Execution (actual computation)
with tf.Session() as sess:
    result = sess.run(z, feed_dict={x: [1.0]})
    # Now computation happens: result = [3.0]
```

Static graph compilation separates graph construction from tensor execution. The lifecycle in figure 7.5 illustrates this phase division: symbolic graph definition on the left precedes ahead-of-time optimization and runtime execution on the right.

The key difference from eager execution is that during construction, x , y , and z are not tensors containing values but rather symbolic nodes in a graph. Operators such as $*$ and $+$ append transformation nodes to the graph definition without issuing arithmetic. Calling `print(y)` makes this distinction visible: it prints symbolic tensor metadata rather than numerical values. Only when the

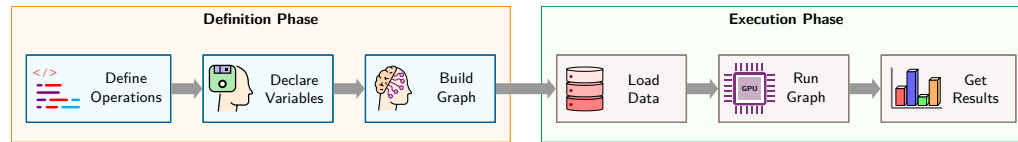


Figure 7.5: Static Graph Execution Lifecycle: Separating symbolic graph definition (left) from concrete execution (right) provides global visibility over the computation, allowing compilers to apply ahead-of-time optimizations—such as operator fusion, buffer reuse, and layout transforms—before any tensors are materialized.

host invokes `sess.run()` does the runtime analyze, optimize, and execute the captured graph on device buffers.

Ahead-of-time optimization Because the framework has the complete graph before execution, it can perform ahead-of-time optimization unavailable across uncaptured eager calls. The kernel fusion opportunity introduced in section 7.3.1 becomes actionable here: because the framework sees $y = x * 2$ and $z = y + 1$ together in the graph, it can fuse them into $z = x * 2 + 1$, eliminating the intermediate y and halving idealized memory traffic. When tensor shapes and lifetimes are known, the compiler can plan memory before execution and reuse buffers whose lifetimes do not overlap; dynamic dimensions may require bounds or runtime allocation. Tensor layouts can also be transformed globally (for example, NCHW to NHWC) to match hardware preferences, though physical layout conversions may still require copies. Dead-code elimination (DCE)¹¹ removes operations whose results are unused, and constant folding precomputes operations on constant values when their inputs are known. These optimizations map directly to iron law terms: kernel fusion can reduce D_{vol} by avoiding intermediate memory writes, constant folding reduces repeated work, planned allocation can reduce runtime overhead, and dead code elimination removes work and associated data movement when unused computations are present.

Compilers such as XLA¹² (Google 2025) extend this approach, compiling captured subgraphs into fused machine code specialized for target hardware.

Systems implications Static graphs can achieve high performance through ahead-of-time optimization. Kernel fusion reduces memory bandwidth requirements when intermediate traffic is the bottleneck, and hardware-specific compilation can approach high utilization when shapes, operators, and layouts match efficient backend paths.

The cost of this performance is reduced flexibility. Standard Python control flow (`if`, `for`) cannot depend on computed tensor values in static graphs. TensorFlow provides graph-level control flow primitives (`tf.cond` and `tf.while_loop`) that support data-dependent conditions, but these require special syntax that diverges from standard Python, making code harder to write and reason about. Debugging is difficult because stack traces point to graph construction code, not execution code. Error messages often reference symbolic node names rather than the actual operations that failed.

7.3.3.3 Hybrid approaches: JIT compilation

JIT compilation pursues both eager debugging and graph optimization at once by capturing computation at runtime. The core trade-off is *fidelity vs. generality*. Tracing captures the execution path taken during a sample run, producing high fidelity to that path but missing branches not taken. Source-level compilation (scripting) analyzes supported program structure, preserving supported control flow but requiring a restricted language subset. Both approaches produce an intermediate representation (IR)¹³ that enables graph optimizations such as operator fusion, constant folding, dead code elimination, and buffer reuse.

The eager-vs.-compiled trade-off has a direct iron law consequence. JIT compilation amortizes the L_{lat} (dispatch overhead) across the compiled region. Longer compiled regions mean more overhead amortized per operation, which explains why graph breaks are performance-critical: each break forces a return to eager dispatch, resetting the amortization.

Historically, PyTorch’s TorchScript exemplified both strategies (PyTorch Contributors 2026d). TorchScript is now deprecated in favor of newer capture and export paths such as `torch.export` (PyTorch Contributors 2026d, 2026c); it remains useful here as a concrete illustration of tracing and source-level scripting. Tracing executes a function with example inputs and records the tensor

¹¹ | **Dead code elimination (DCE):** Removes graph nodes whose results do not affect observable outputs or side effects. In ML graphs, dead code arises from unused branches or post-transform artifacts; side-effect analysis ensures safe removal to prevent execution of dead operations and reduce kernel launch overhead.

¹² | **XLA (accelerated linear algebra):** The “optimized machine code” in the triggering sentence means XLA can fuse subgraphs, specialize layouts, and lower high-level operations into backend-specific code; fusion attacks both launch overhead (L_{lat}) and intermediate memory writes (D_{vol}), but the realized speedup depends on whether the graph contains enough fusible, memory-bound work for the compiler to remove. Large GEMM-heavy regions may already be compute bound, while chains of small elementwise operations can benefit more because fusion removes repeated trips through external memory. The benefit is not a fixed multiplier: XLA helps when it can fuse operations, specialize layouts and shapes, and reduce launch or memory overhead, so gains depend on graph structure, backend support, and input-shape stability.

13 | Intermediate representation (IR): The “intermediate” captures this format’s architectural role: a language-independent layer that decouples the frontend (Python capture) from the backend (hardware code generation), exactly as LLVM IR decouples C/Rust/Swift frontends from x86/ARM backends. ML frameworks adopted this compiler pattern because it reduces the $\mathcal{O}(M \times N)$ cost of supporting M frontends and N backends to $\mathcal{O}(M + N)$: a single graph capture mechanism (TorchDynamo, tf2xla) can target multiple hardware backends without rewriting the capture logic.

operations observed on that path. Listing 7.3 demonstrates how a traced module could be serialized and executed independently of the Python interpreter.

Listing 7.3: TorchScript Tracing: Captures tensor operations by executing a function with example inputs and recording the execution path into a static computation graph.

```
import torch

def forward(x):
    y = x * 2
    z = y + 1
    return z

# Trace the function by running it once
x_example = torch.tensor([1.0])
traced = torch.jit.trace(forward, x_example)

# traced is now a compiled TorchScript module
# Can serialize: torch.jit.save(traced, "model.pt")
# Can optimize: fusion, constant folding
# Can run without Python interpreter
```

The critical limitation of tracing reveals the fidelity-generality trade-off concretely. A trace records operations from an observed path rather than preserving arbitrary Python control flow. Listing 7.4 illustrates the resulting correctness risk.

Listing 7.4: Tracing a Data-Dependent Branch: Tracing records the path taken by the example input. PyTorch commonly emits a `TracerWarning` when a tensor value is converted to a Python condition because the resulting trace may not generalize.

```
def conditional_forward(x):
    if x.sum() > 0: # Data-dependent condition
        return x * 2
    else:
        return x * 3

traced = torch.jit.trace(conditional_forward, torch.tensor([1.0]))
# Tracing captures ONLY the x.sum() > 0 branch
# If input later has sum <= 0, traced version
# still executes x * 2 branch
```

Tracing records the branch executed by the example input. In this example, converting the tensor condition to a Python Boolean commonly produces a `TracerWarning`; the resulting trace can still follow the captured branch for later inputs that require the other branch. Treating trace warnings as correctness failures is therefore essential when data-dependent Python control flow is present.

TorchScript’s historical alternative, scripting, analyzed supported Python source directly and compiled it to TorchScript IR without tracing a sample path (PyTorch Contributors 2026d). The scripting compiler preserved supported branching structure but accepted only a restricted Python subset. Current projects should follow the supported `torch.export` and target-runtime guidance rather than select between TorchScript tracing and scripting for new deployment pipelines (PyTorch Contributors 2026c). Within the historical TorchScript workflow, tracing fit feed-forward models whose tensor paths remained stable for the supported inputs, while scripting handled supported data-dependent control flow without a Python interpreter. Scripting’s key advantage was its ability to preserve supported conditionals in the IR, as listing 7.5 shows.

Listing 7.5: Scripted Control Flow: Unlike tracing, scripting preserves both branches of conditionals in the IR, enabling correct execution based on runtime input values.

```
@torch.jit.script
def conditional_forward(x: torch.Tensor) -> torch.Tensor:
    if x.sum() > 0:
        return x * 2
    else:
        return x * 3

# Both branches preserved in IR
# Correct branch executes based on runtime input values
```

Scripting imposed constraints because TorchScript had to analyze code that Python normally interprets dynamically. Function signatures and variables sometimes needed type annotations, and unsupported Python objects, library calls, or metaprogramming could fail compilation. Table 7.1 summarizes this historical design trade-off rather than current deployment guidance.

Table 7.1: Historical TorchScript Tracing vs. Scripting Trade-Offs: Tracing recorded operations from example executions, while scripting preserved supported control flow at the cost of a restricted Python subset. TorchScript is deprecated for new projects.

Aspect	Tracing	Scripting
Input requirement	Example inputs needed	No inputs needed
Control flow	Cannot handle data-dependent	Supports data-dependent
Conversion ease	Simpler (just run function)	Harder (restricted Python)
Type annotations	Not required	Required when inference fails
Error detection	Runtime (wrong results)	Compile time (syntax errors)
Best for	Feed-forward models	Models with conditionals

To understand what the compiler produces, listing 7.6 inspects the generated intermediate representation directly, where the single Python expression has been lowered into explicit typed primitive operations the runtime can execute without the interpreter. The TorchScript IR represents operations using the `aten` namespace for core tensor operations, the `prim` namespace for primitives and control flow, static types for every value, and static single-assignment (SSA) form, where each variable is assigned exactly once to simplify compiler analysis. This IR enables optimizations independent of Python: operator fusion combines adjacent operations into single kernels, constant folding evaluates constant expressions at compile time, dead code elimination removes unused operations, and memory optimization reuses buffers when possible.

Listing 7.6: TorchScript IR Inspection: The generated intermediate representation shows primitive operations and constants, useful for debugging and understanding compilation results.

```
@torch.jit.script
def example(x: torch.Tensor) -> torch.Tensor:
    return x * 2 + 1

# Inspect generated IR:
print(example.graph)
# graph(%x : Tensor):
#   %1 : int = prim::Constant[value=2]()
#   %2 : Tensor = aten::mul(%x, %1)
#   %3 : int = prim::Constant[value=1]()
#   %4 : Tensor = aten::add(%2, %3, %3)
#   return (%4)
```

7.3.3.4 Modern compilation: Graph-capture JIT

The previous approaches force a choice between flexible eager execution and compiler-visible graphs. Modern JIT compilation narrows this trade-off by automatically capturing regions of eager code into optimized graphs with limited developer intervention.

Graph-capture JIT systems follow the same architectural pattern across frameworks. The first execution observes tensor operations, records a graph region guarded by assumptions about shapes, dtypes, layouts, and control flow, lowers that region into an intermediate representation, applies fusion and layout optimizations, and caches executable code for later calls that satisfy the same guards. Unsupported Python code does not disappear; it forms a graph boundary where execution returns to the eager runtime. PyTorch 2.0's `torch.compile` (Ansel et al. 2024) is a concrete instance of this pattern, but the systems idea is broader than the API: compilation pays only when captured regions are long enough and stable enough to amortize capture, lowering, code generation, and cache-management costs.

This explains when compilation can help. Dispatch costs that appear negligible for a single operation—a few microseconds per kernel launch—accumulate across thousands of operations in a forward pass. The physics of software overhead in 7.1 models this dispatch and memory traffic.



Napkin Math 7.1: The physics of software overhead

Iron law connection: The latency term (L_{lat}) in the iron law is dominated by software overhead: dispatching instructions from Python to the GPU. Each operation pays a Python dispatch cost of $\sim 10 \mu\text{s}$ plus a kernel launch cost of $\sim 5 \mu\text{s}$, which together set the per-launch overhead applied in the scenarios that follow.

Scenario one: Eager Mode (The “Tiny Op” Trap) Consider a simple activation block, $y = \text{relu}(x + \text{bias})$, in which each tensor contains N elements of b bytes each.

- **Operations:** Two (Add, ReLU).
- **Execution:**
 1. Launch Add Kernel: $15 \mu\text{s}$ overhead.
 2. Read/Write Memory: $2Nb$ bytes.
 3. Launch ReLU Kernel: $15 \mu\text{s}$ overhead.
 4. Read/Write Memory: $2Nb$ bytes.
- **Total overhead:** $30 \mu\text{s}$.
- **Total memory traffic:** $4Nb$ bytes.

Scenario two: Compiled Mode (Fusion) The compiler fuses this into one kernel: `FusedAddRelu`.

- **Execution:**
 1. Launch Fused Kernel: $15 \mu\text{s}$ overhead.
 2. Read/Write Memory: $2Nb$ bytes (the intermediate result stays on chip).
- **Total overhead:** $15 \mu\text{s}$ ($2\times$ speedup).
- **Total memory traffic:** $2Nb$ bytes ($2\times$ bandwidth efficiency).

Systems insight: Fusion wins on two fronts at once. Collapsing two launches into one halves the per-op dispatch overhead in this scenario, and keeping the intermediate result on chip cuts idealized memory traffic from $4Nb$ to $2Nb$ bytes. For small element-wise operations, the avoided round-trip to external memory can matter more than the arithmetic.

Figure 7.6 makes the dispatch tax visible: eager execution creates gaps where the GPU sits idle while Python dispatches the next kernel. The blue compute regions are short; the red dispatch regions are comparatively long. Compilation fuses these operations into a single kernel launch, replacing many dispatch gaps with one dispatch block and one fused compute block.

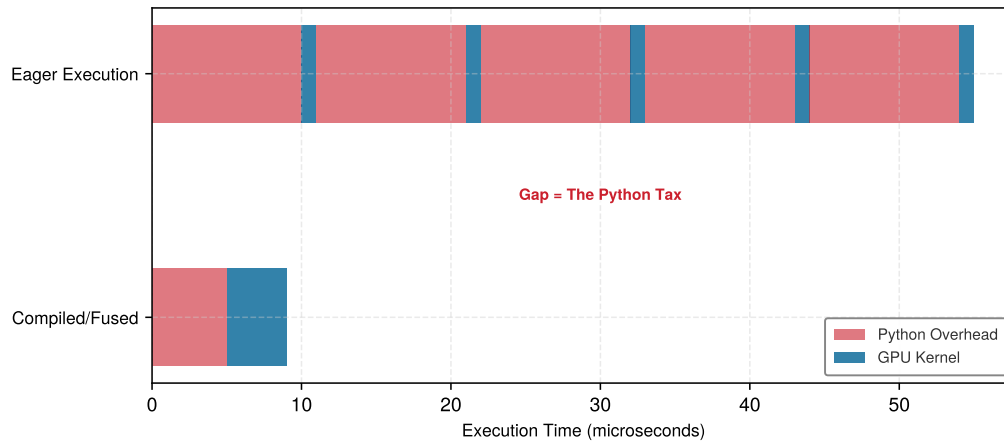


Figure 7.6: The Python Tax: Execution timelines show how eager mode incurs repeated host dispatch latency (L_{lat}) between short GPU kernels, leaving the accelerator periodically idle. Compilation fuses these operations into a single kernel launch, eliminating intermediate dispatch gaps and maximizing hardware occupancy.

Automating this fusion is the design goal behind graph-capture compilers such as PyTorch 2.0's `torch.compile`.¹⁴ They capture eager tensor regions and compile them into fused kernels without requiring engineers to write custom CUDA.¹⁵

The core engineering questions are therefore conceptual, not API-specific. A capture compiler needs a frontend that identifies graph regions in an eager program, an intermediate representation that separates the captured computation from Python, and a backend that lowers the region to hardware-specific code. It also needs a guard system: the compiled artifact is valid only while assumptions about tensor rank, dtype, layout, and control-flow path remain true. When a guard fails, the runtime must recompile or fall back to eager execution.

Graph breaks mark the boundary where compilation stops applying. Data-dependent Python control flow, unsupported library calls, I/O, custom Python objects, and highly variable shapes all shorten compiled regions. Each break reintroduces dispatch overhead and may require tensors to move between compiled code and the eager runtime. Mechanically, a graph break forces the capture engine to split the program into disjoint compiled subgraphs separated by standard Python execution. At each boundary, intermediate device tensors must be unpacked into Python runtime objects, paying memory materialization and interpreter dispatch overhead before the next compiled subgraph can be entered. This is why graph-break analysis belongs in performance engineering: the relevant metric is not whether compilation is enabled, but how much of the hot path remains inside long, stable compiled regions.

Backends occupy different points on the flexibility-performance spectrum. A **JIT** compiler traces model execution at runtime to generate optimized kernels dynamically, incurring initial warmup latency during the first pass. An **ahead-of-time (AOT)** compiler instead compiles captured computation before deployment, shifting compilation out of the hot path and enabling a standalone runtime when the export target supports one. A general JIT backend optimizes ordinary training and serving workloads with moderate compilation cost; a specialized inference backend can apply deeper fusion, precision lowering, and autotuning when the deployment target is fixed; an ahead-of-time mobile or embedded runtime removes even more flexibility to gain footprint and predictability. The same rule governs all of them: the narrower the target and the more stable the graph, the more optimization the compiler can safely perform.

The resulting workflow is a systems decision. Rapid prototyping favors eager execution because architecture changes and guard failures make recompilation cost visible. Long training runs and high-volume inference amortize compilation cost over many executions, provided the model has stable shapes and limited graph breaks. Debugging usually starts in eager mode because errors map directly to source code; compilation is reintroduced after the model behavior is correct and the performance bottleneck is measurable.

¹⁴ | **`torch.compile`:** It is a 2020s PyTorch implementation of graph-capture JIT compilation: bytecode interception extracts tensor regions from eager programs, an intermediate representation carries those regions to compiler backends, and cached generated code is reused while guard conditions continue to hold.

¹⁵ | **CUDA (compute unified device architecture):** NVIDIA's parallel computing platform (2007) serving as the foundational layer between high-level Python operations and GPU silicon; when PyTorch executes `torch.matmul(A, W)`, the call traverses the framework's dispatcher, selects a cuBLAS kernel, and launches it on the GPU. Each launch incurs 5–20 μ s of CPU-side overhead. For small operations, this dispatch overhead (L_{lat}) exceeds the useful compute time, which is why compilation (fusing N_{ops} operations into one kernel launch) yields speedups proportional to the reduction in launch count rather than the reduction in arithmetic.

Comparison of execution models Table 7.2 contrasts the three execution models across six dimensions, showing how hybrid JIT compilation can recover graph-level optimization within captured regions while preserving eager fallback outside them.

Table 7.2: Execution Model Trade-Offs: Each execution model occupies a distinct position in the flexibility-optimization trade-off space. Eager execution maximizes debugging flexibility but limits cross-operation optimization; static graphs maximize optimization but require control flow to be represented in the graph; hybrid JIT compilation compiles captured regions while falling back to eager execution for unsupported patterns.

Aspect	Eager + Autograd Tape (PyTorch default)	Static graph (TensorFlow 1.x)	JIT Compilation (torch.compile)
Execution Model	Immediate	Deferred	Hybrid
Graph Construction	During forward pass	Before execution	First execution (cached)
Optimization	Per-operation kernels	Ahead-of-time	JIT compilation
Dynamic Control Flow	Python control flow	Graph control flow	Captured regions or breaks
Debugging	Easy (standard Python)	Difficult (symbolic)	Moderate (mixed)
Performance	Baseline	High (optimized)	High (compiled regions)

Eager mode’s primary value is in the iteration loop quantified in section 3.1: it allows using standard Python debuggers (such as pdb) to inspect variables mid-execution, whereas graph-mode debugging often requires specialized framework tools. This immediate feedback accelerates the prototyping phase of the ML lifecycle.

Beyond these core execution trade-offs, table 7.3 highlights additional systems-level distinctions between static and dynamic approaches.

Table 7.3: Additional Graph Trade-Offs: Systems-level distinctions between static and dynamic graphs that complement table 7.2. These trade-offs reappear when selecting frameworks in section 7.9.

Aspect	Static Graphs	Dynamic Graphs
Memory Management	Precise allocation planning, optimized memory usage	Flexible but potentially less efficient
Hardware Utilization	Can generate highly optimized hardware-specific code	May sacrifice hardware-specific optimizations
Research Velocity	Slower iteration due to define-then-run requirement	Faster prototyping and model experimentation
Integration with Legacy Code	More separation between definition and execution	Natural integration with imperative code

These trade-offs are not binary choices. Modern frameworks offer a spectrum of execution options, which raises the quantitative question of where on this spectrum a given project should operate.

7.3.4 Quantitative principles of execution

These execution models form a spectrum, and two quantitative principles make the choice measurable in practice. The compilation continuum principle asks when compilation gains justify development cost by comparing production executions with development iterations. The dispatch overhead law shows why eager execution can spend more time dispatching small operations than computing them.

7.3.4.1 The compilation continuum principle

The execution problem demands a quantitative principle for when a project should compile. The execution models form a continuum from maximum flexibility to maximum optimization. Equation 7.1 lays out the four positions on that axis, and each labeled arrow names the mechanism that carries a project one step rightward toward hardware.

$$\text{Eager} \xrightarrow{\text{capture}} \text{Graph JIT} \xrightarrow{\text{export}} \text{AOT Runtime} \xrightarrow{\text{specialize}} \text{Hardware} \quad (7.1)$$

Each step rightward sacrifices flexibility for performance. The practical question is *where* a given project should be placed on this continuum. The optimal compilation strategy depends on production executions, development recompilations, and their respective time costs, combined in equation 7.2:

$$\text{Compilation Benefit} = \frac{N_{\text{prod}} \cdot (T_{\text{eager}} - T_{\text{compiled}})}{T_{\text{compile}} + N_{\text{dev}} \cdot T_{\text{compile}}} \quad (7.2)$$

where:

- N_{prod} = number of production executions (dimensionless count: inference requests, training steps)
- N_{dev} = number of development iterations requiring recompilation (dimensionless count)
- T_{eager} = time per execution in eager mode (seconds)
- T_{compiled} = time per execution in compiled mode (seconds)
- T_{compile} = time per compilation or recompilation (seconds; assumed constant)

This model assumes that the compiled artifact remains valid between development changes and that the initial compilation and each recompilation have the same cost; under these assumptions, the decision rule is to compile when $\text{Compilation Benefit} > 1$. The ratio is dimensionless.

Table 7.4 presents a hypothetical throughput scenario across execution modes and model architectures:

Table 7.4: Illustrative Training and Inference Throughput: Hypothetical throughput and compilation-time inputs for comparing execution modes on an NVIDIA A100 GPU with batch size 32. These values are assumptions rather than benchmark measurements. Within this scenario, the table implies torch.compile speedups of 1.4–1.5× over eager mode, while TensorRT implies 2.3–2.7× speedups but requires longer compilation and is inference only.

Model	Eager (examples/sec)	torch.compile (examples/sec)	TensorRT (examples/sec)	Compile Time (seconds)
ResNet-50	1,450	2,150	3,800	15–30
BERT-Base	380	520	890	30–60
ViT-B/16	620	950	1,650	25–45
GPT-2 (124M)	180	260	420	45–90

These throughput differences across execution modes raise a practical question—which framework execution strategy best serves each workload archetype. The appropriate strategy depends on the workload, backend, and dominant iron law term, and table 7.5 maps each recurring archetype to a candidate execution strategy.

Table 7.5: Framework Execution Strategy by Workload: Candidate execution strategies for each workload archetype, aligned to the dominant iron law term.

Archetype	Dominant Iron Law Term	Candidate Framework Strategy	Rationale
ResNet-50 (Compute Beast)	$\frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}$ (Compute)	Compiled dense kernels	Regular dense kernels benefit from layout selection, precision lowering, and backend specialization; fusion helps most in surrounding memory- or launch-bound regions
GPT-2 (Bandwidth Hog)	$\frac{D_{\text{vol}}}{\text{BW}}$ (Memory Bandwidth)	Fused attention + graph compilation	Fused attention and compilation reduce HBM round-trips and improve cache reuse
DLRM (Sparse Scatter)	$\frac{D_{\text{vol}}}{\text{BW}_{\text{random}}} + L_{\text{lat, network}}$	Eager execution with specialized kernels	Embedding lookups are inherently irregular and dynamic; compilation gains are small
DS-CNN (Tiny Constraint)	L_{lat} (Overhead)	Ahead-of-time microcontroller runtime	Sub-ms inference; every microsecond of Python overhead is unacceptable



Lighthouse 7.1: Framework strategy by archetype

The four recurring archetypes in table 7.5 trace one principle across the framework: compilation benefits scale with how much of the workload is optimizable.

Systems insight: In the illustrative scenario, Compute Beasts such as the ResNet-50 row in table 7.4 benefit because their dense kernels expose substantial surface for layout selection, precision lowering, and fusion. Sparse Scatter workloads such as DLRM can gain less when irregular embedding lookups leave limited compiler scope.

This principle has concrete implications across three regimes. In research prototyping ($N_{\text{dev}} \gg N_{\text{prod}}$), teams should stay eager. If the architecture changes every few minutes, compilation overhead can dominate. Under the scenario's 30-second compilation assumption, ten compilations per hour consume five minutes.

For long training runs ($N_{\text{prod}} \gg N_{\text{dev}}$), compilation can pay off because its one-time cost is amortized across many steps. In the hypothetical ResNet-50 scenario in table 7.4, torch.compile provides 48.3 percent higher throughput (2,150 img/s vs. 1,450 img/s). Using the scenario's 30 s compile cost, this pays off after the breakeven point in equation 7.3:

$$N_{\text{breakeven}} = \frac{T_{\text{compile}}}{T_{\text{eager}} - T_{\text{compiled}}} \quad (7.3)$$

Evaluating equation 7.3 with the hypothetical ResNet-50 values gives approximately 134,000 images, well within a single training run of any realistic length.

For production inference ($N_{\text{dev}} \approx 0$, $N_{\text{prod}} \rightarrow \infty$), teams should maximize compilation. With no development iterations and potentially millions of requests, every optimization matters. Aggressive autotuning can be worthwhile even when compilation takes much longer, because the cost is amortized over the deployment lifetime.

These three regimes create distinct regions in the compilation decision space. Figure 7.7 maps out these regions so engineers can identify where each strategy wins. The crossover points mark where each strategy becomes optimal: the steep eager curve (highest per-execution cost) is overtaken by the moderate JIT slope, while the gentle compiled trajectory (lowest per-execution cost but largest upfront investment) dominates at high execution volumes. The slopes reveal per-execution cost; the vertical offsets reveal compilation overhead. A project's execution volume and measured costs determine which strategy minimizes total time.

7.3.4.2 The dispatch overhead law

A second principle, the dispatch overhead law, emerges from equation 7.4, which identifies the regime in which framework overhead, rather than compute or memory, dominates execution time. Let N_{ops} be the number of operations (count), t_{dispatch} the per-operation dispatch overhead (seconds), and T_{compute} and T_{memory} the total compute and memory times (seconds). The ratio uses the additive, no-overlap approximation $T_{\text{hw}} \approx T_{\text{compute}} + T_{\text{memory}}$; when execution overlaps those costs, measured hardware time should replace their sum. Framework overhead dominates when operations are small relative to dispatch cost:

$$\text{Overhead Ratio} = \frac{N_{\text{ops}} \cdot t_{\text{dispatch}}}{T_{\text{compute}} + T_{\text{memory}}} \quad (7.4)$$

When Overhead Ratio > 1 , the model is overhead-bound. Compilation can provide its largest gains for overhead-bound workloads because fusion reduces the number of dispatches. The training-step trace in section 7.10 works this effect through end to end, previewed by the numbers that follow.

Summed over N_{ops} operations, the per-operation dispatch cost t_{dispatch} accumulates into a per-call tax on execution. Whether that tax dominates depends on how the hardware execution time T_{hw} compares to the software overhead T_{sw} (both measured in seconds), and the regimes split sharply by model size. Workloads composed of small operations benefit disproportionately from compilation when dispatch dominates execution; model parameter count alone does not determine the gain.

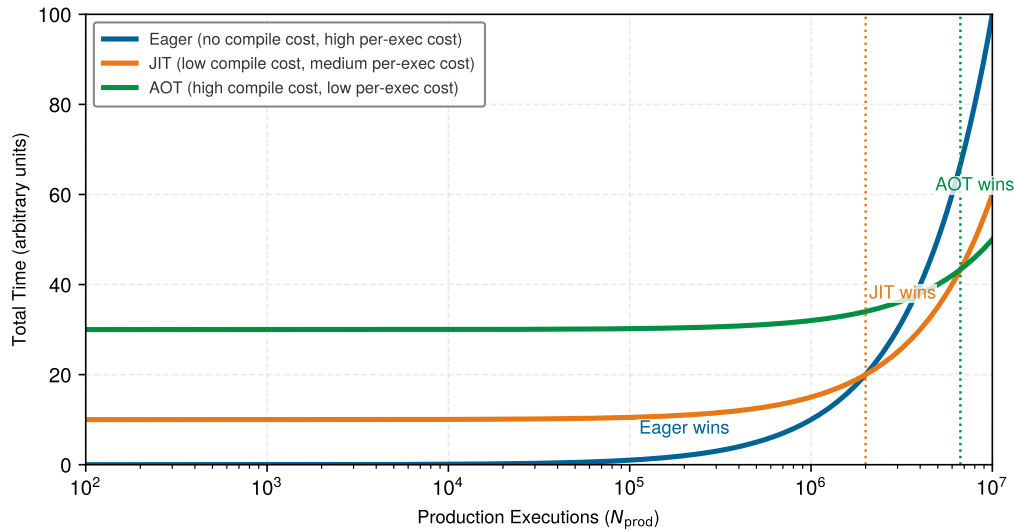


Figure 7.7: The Compilation Continuum: An illustrative cost model shows how one-time compilation cost and per-execution cost create eager, JIT, and AOT crossover regions as production executions increase. The crossover locations depend on the assumed compilation costs and per-execution rates.

Notebook 7.2 contrasts a small multilayer perceptron with a large transformer layer to quantify when host overhead or device arithmetic bounds the system.

Napkin Math 7.2: The dispatch tax

Problem: When does Python overhead kill performance?

Scenario one: Small multilayer perceptron (MLP) (Overhead Bound)

- **Compute:** 6 ops across small matrix/element-wise operations.
- **Hardware time:** $T_{\text{hw}} \approx 2.6 \mu\text{s}$ (mostly memory latency).
- **Software overhead:** $T_{\text{sw}} \approx 6 \text{ ops} \times 15 \mu\text{s/op} = 90 \mu\text{s}$.
- **Ratio:** $90 \mu\text{s} / 2.6 \mu\text{s} \approx 34.6$.
- **Small-model outcome:** The system spends 97 percent of time in host-side dispatch and kernel-launch overhead. Eliminating all modeled dispatch overhead would give an upper-bound speedup of 35.6 $\times$.

Scenario two: GPT-3 Layer (Compute Bound)

- **Compute:** Huge matrix multiplications.
- **Hardware time:** $T_{\text{hw}} \approx 100 \text{ ms} = 100000 \mu\text{s}$.
- **Software overhead:** $T_{\text{sw}} \approx 50 \mu\text{s}$.
- **Ratio:** $50 \mu\text{s} / 100000 \mu\text{s} \approx 0.0005$.
- **Large-model outcome:** Modeled Python overhead is negligible. Compilation may still improve fusion, layout, or backend specialization; dispatch elimination contributes little.

Systems insight: Dispatch overhead is regime-dependent. Compilation can reduce host-side overhead for small-operation workloads by fusing operations into fewer launches, while large models benefit mainly from fused kernels and memory movement reductions.

The dispatch tax analysis demonstrates that small operations become overhead-bound when dispatch latency exceeds device execution time. This systems constraint reaches its logical extreme

at the edge of the deployment spectrum, where the memory and latency footprint of the Python runtime exceeds the physical capacity of the target hardware.

7.3.5 Frameworks for the edge: TinyML and micro-runtimes

The compilation continuum reaches its extreme at the far edge. While cloud frameworks such as PyTorch and TensorFlow 2.x prioritize flexibility through eager execution, TinyML¹⁶ systems operating on MCUs with kilobytes of memory cannot afford the overhead of a Python interpreter or a fully dynamic runtime.

¹⁶ | **TinyML:** Systems designed for MCUs cannot afford a Python interpreter. Micro-runtimes instead use small C/C++ runtimes, fixed memory planning, and model-specific operators. TensorFlow Lite Micro remains interpreter-based: the application supplies a tensor arena whose buffers are planned and reused without heap allocation after setup. Predictable memory is mandatory because exceeding a 256 KB device budget can prevent initialization or inference.



Lighthouse 7.2: Lighthouse example: KWS on TinyML

Scenario: Deploying the Smart Doorbell’s keyword spotting (KWS) model, the DS-CNN Tiny Constraint lighthouse, to an ARM Cortex-M4 with 256 KB RAM and 1 MB Flash.

Constraint: A full PyTorch and Python runtime exceeds the device’s memory budget by orders of magnitude.

Framework solution: Micro-frameworks such as **TensorFlow Lite Micro (TFLM)** (David et al. 2021) solve this through a tiny interpreter-based runtime with a fixed memory discipline:

1. **Fixed memory arena:** The application supplies a contiguous tensor arena, and the framework plans and reuses buffers from that arena rather than relying on dynamic allocation during inference.
2. **Kernel selection:** Only the specific kernels used by the model (for example, Conv2D, DepthwiseConv) need to be linked or registered with the runtime.
3. **Compact interpreter execution:** The MCU runs a small C/C++ interpreter over a flat model representation, with the model and arena bound at initialization rather than assembled dynamically at runtime.

Silicon contract: On TinyML devices, the contract is strictly memory-bound. The framework’s primary job is to ensure the model’s intermediate activations (the “working set”) fit within the MCU’s tiny SRAM.

These micro-runtimes sacrifice dynamic flexibility to reduce footprint and make memory predictable. With compact models, low-power hardware, and duty cycling, that discipline enables milliwatt-scale inference with mostly on-chip data movement.

The spectrum of execution strategies, from dynamic eager execution to static graph compilation and specialized micro-runtimes, requires developers to make deliberate trade-offs. The key execution-mode decisions summarize these architectural choices:



Checkpoint 7.1: Execution models

The choice of execution mode determines both developer velocity and model performance.

Debuggability vs. speed

- ☐ **Eager mode (Python-first):** why does executing ops one-by-one make debugging easy but optimization hard?
- ☐ **Graph mode (compiler-first):** why does building a static graph enable kernel fusion?

Modern compromise

- ☐ **JIT compilation:** how does `torch.compile` bridge the gap?
- ☐ **Dispatch-overhead estimate:** if six operations each incur the chapter’s 15 μs /op dispatch tax while hardware work takes 2.6 μs , which term dominates?

The execution problem determines *when* computation happens and *what* optimizations are possible. Neural network training, however, requires a capability that no amount of clever scheduling can provide: the ability to compute gradients automatically.

Consider what training actually requires: for each of millions of parameters, compute how a tiny change would affect the loss. Doing this manually for even a simple three-layer network requires deriving and implementing dozens of partial derivatives. For a modern transformer with billions of parameters, manual differentiation is economically impossible. A framework that executes efficiently but cannot differentiate can run inference but cannot learn.

7.4 Differentiation Problem

The **differentiation problem** is the task of computing gradients¹⁷ automatically. Neural network training requires derivatives of a scalar loss $\mathcal{L}$ with respect to millions or billions of parameters. When executing the forward pass, the runtime records an execution tape and saves intermediate activation buffers for subsequent derivative evaluations. Because a single scalar loss depends on many parameters, reverse-mode automatic differentiation (AD)¹⁸ is efficient: a single reverse traversal computes all requested parameter gradients, whereas recovering the same full gradient with forward mode requires one tangent direction per parameter. Major ML frameworks therefore use reverse-mode AD for ordinary scalar-loss training (Baydin et al. 2018).

Building on the backpropagation algorithm introduced in chapter 5, the systems engineering of differentiation addresses how frameworks represent computation graphs, manage memory for intermediate values, and orchestrate the backward pass efficiently across accelerators. The framework mechanizes the chain rule as large-scale systems bookkeeping: tracking operator dependencies, staging activation buffers, and dispatching backward kernels for the training algorithms detailed in chapter 8. Listing 7.7 illustrates the core idea with a simple three-operation function.

Listing 7.7: Automatic Differentiation: AD decomposes complex functions into elementary operations with known local derivatives, allowing frameworks to apply the chain rule over a recorded operation graph.

```
def f(x):
    a = x * x # Square
    b = sin(x) # Sine
    c = a * b # Product
    return c
```

Frameworks decompose this function into elementary operations, each with a known local derivative, and then combine these local derivatives via the chain rule to compute gradients through supported compositions. The systems challenge is executing this transformation under physical hardware constraints: the framework must record the computation graph during forward execution, retain necessary intermediate tensors in device memory, and schedule backward kernels without exhausting accelerator HBM. Production runtimes address these challenges through structured graph recording, caching allocators, and reverse-mode traversal.

7.4.1 Forward and reverse mode differentiation

Automatic differentiation operates in two distinct traversal directions: forward mode and reverse mode. The choice between them governs whether gradient evaluation cost scales with the number of input dimensions or output dimensions. Because deep learning objectives evaluate a single scalar loss with respect to millions or billions of parameters, this mathematical asymmetry dictates the memory footprint and execution schedule of the underlying framework.

7.4.1.1 Forward mode

Forward mode automatic differentiation computes directional derivatives concurrently with the primal evaluation. For each intermediate variable v_i , the runtime evaluates both the primal value v_i and its directional derivative (tangent) $\dot{v}_i = \sum_j \frac{\partial v_i}{\partial x_j} \dot{x}_j$. Because tangents flow downstream alongside primal values, forward mode requires no execution tape and never retains forward activations for a subsequent backward pass.

To see the mechanism concretely, consider computing both the value and derivative of $f(x) = x^2 \sin(x)$. Listing 7.8 shows how forward mode evaluates local derivative rules alongside each elementary operation, applying the chain and product rules at every step:

¹⁷ | **Automatic differentiation (AD):** The “automatically” in the triggering sentence is the key word: AD mechanizes the chain rule as a graph traversal, eliminating the manual derivative computation that made scaling beyond toy networks impractical. The systems trade-off that makes this feasible is the choice of reverse mode, which exploits the many-to-one topology of training (many parameters, one scalar loss) to compute all gradients in a single backward pass. Forward mode would require one pass per parameter, making billion-parameter training computationally impossible.

¹⁸ | **Reverse-mode AD:** The $\mathcal{O}(1)$ -vs.- $\mathcal{O}(P)$ asymmetry of reverse mode has a concrete price: reverse mode retains the operation-dependent values required by derivative rules during the backward traversal. Activation checkpointing changes this saved set by omitting selected values and recomputing them during the backward pass.

Listing 7.8: Forward Mode AD: Propagates derivatives forward through the computation graph, computing one directional derivative per seeded forward pass.

```
def f(x): # Computing both value and derivative
    # Step 1:  $x \rightarrow x^2$ 
    a = x * x # Value:  $x^2$ 
    da = 2 * x # Derivative:  $2x$ 

    # Step 2:  $x \rightarrow \sin(x)$ 
    b = sin(x) # Value:  $\sin(x)$ 
    db = cos(x) # Derivative:  $\cos(x)$ 

    # Step 3: Combine using product rule
    result = a * b # Value:  $x^2 * \sin(x)$ 
    dresult = a * db + b * da # Derivative:  $x^2 * \cos(x) + \sin(x) * 2x$ 

    return result, dresult
```

Forward mode mechanizes this paired tracking through dual numbers—values of the form $v + v\epsilon$ where $\epsilon^2 = 0$. Evaluating an arithmetic operator or elementary function over dual numbers automatically yields the updated primal and tangent without graph retention. Listing 7.9 demonstrates this trace evaluated at $x = 2.0$ with seed $\dot{x} = 1.0$:

Listing 7.9: Dual-Number Trace: The forward-mode function of listing 7.8 evaluated at $x = 2.0$. Each line keeps a value and its derivative, so one pass yields both $f(2.0) \approx 3.637$ and $f'(2.0) \approx 1.973$.

```
x, dx = 2.0, 1.0 # seed: track the derivative with respect to x

a = x * x # value: 4.0
da = 2 * x # derivative: 4.0
b = sin(x) # value: 0.9093
db = cos(x) # derivative: -0.4161
c = a * b # value: 3.637
dc = a * db + b * da # derivative: 1.973
```

A single forward pass with a seeded tangent vector $\dot{\mathbf{x}}$ evaluates a Jacobian-vector product (JVP), $J\mathbf{v}$, where $J \in \mathbb{R}^{M \times P}$ is the Jacobian matrix. When an objective maps P parameter inputs to a single scalar loss ($M = 1$), recovering the full gradient $\nabla f \in \mathbb{R}^P$ requires setting $\dot{\mathbf{x}}$ to each of the P standard basis vectors in turn. Recovering gradients with forward mode therefore requires P distinct forward evaluations (or propagating P -wide tangent bundles), scaling compute and memory traffic as $\mathcal{O}(P)$ with parameter count. For networks with millions or billions of parameters, this scaling makes forward-mode gradient calculation intractable, although forward JVPs remain highly effective when input dimensions are small or for evaluating directional derivatives and Hessian-vector products.

7.4.1.2 Reverse mode

Reverse-mode automatic differentiation inverts the traversal order to exploit the many-to-one geometry of neural network training. Instead of propagating tangents downstream from inputs, reverse mode propagates adjoints $\bar{v}_i = \partial \mathcal{L} / \partial v_i$ upstream from the scalar loss $\mathcal{L}$ to all parameters. Mathematically, while forward mode computes Jacobian-vector products ($J\mathbf{v}$), reverse mode evaluates vector-Jacobian products (VJPs), $\mathbf{u}^T J$. Seeding the reverse sweep with $\bar{\mathcal{L}} = 1$ yields the complete parameter gradient vector $\nabla_{\theta} \mathcal{L}$ in a single backward pass, regardless of whether the model contains ten parameters or ten billion.

Reverse mode operates on the same three-operation function $f(x) = x^2 \sin(x)$ from listing 7.7, where x reaches the output through two distinct computational paths: the square and the sine. Algorithm 7.1 defines the reverse-mode traversal algorithm over a directed acyclic graph before tracing this concrete evaluation.

Algorithm 7.1 Reverse-mode automatic differentiation

Require: directed acyclic graph with scalar output y ; nodes $v_1, \dots, v_n$ in topological order; a local derivative rule for each node

Ensure: adjoints $\bar{v}_i = \partial y / \partial v_i$ for the requested input or parameter nodes

- 1: **for** $i = 1$ to n **do**
- 2: compute v_i , record dependencies, and save values requested by its derivative rule $\triangleright$ *forward pass*
- 3: **end for**
- 4: $\bar{v}_i \leftarrow 0$ for all i ; $\bar{y} \leftarrow 1$
- 5: **for** $i = n$ down to 1 **do**
- 6: **for** each parent u of v_i **do**
- 7: $\bar{u} \leftarrow \bar{u} + \bar{v}_i \partial v_i / \partial u$ $\triangleright$ *accumulate via the chain rule*
- 8: **end for**
- 9: release stored values once every rule that needs them has run
- 10: **end for**
- 11: **return** the adjoints for the requested nodes

The algorithmic structure of algorithm 7.1 reveals its systems cost. The forward pass records operator dependencies and buffers intermediate values required by backward derivative rules until the reverse sweep consumes them. Reverse mode trades activation storage and graph bookkeeping for constant-time $\mathcal{O}(1)$ gradient passes with respect to parameter count.

For the concrete function in listing 7.7 evaluated at $x = 2.0$, the forward pass records $a = x^2 = 4.0$, $b = \sin(x) \approx 0.9093$, and $c = ab \approx 3.637$. Seeding $\bar{c} = 1.0$, the product rule yields adjoints $\bar{a} = 0.9093$ and $\bar{b} = 4.0$. The square branch contributes $0.9093 \cdot 4.0 \approx 3.6372$ to $\bar{x}$, while the sine branch contributes $4.0 \cos(2.0) \approx -1.6646$. Accumulating both paths gives the exact derivative $\partial c / \partial x = \bar{x} \approx 1.973$.

This single backward traversal computes $\partial c / \partial x$ regardless of how many fan-out paths connect x to c . In a deep neural network, every weight tensor influences the scalar loss through thousands of divergent paths across successive layers; reverse mode aggregates all incoming adjoints in one reverse sweep. This efficiency makes training a 175B-parameter model such as GPT-3 computationally tractable: reverse mode requires $\mathcal{O}(1)$ backward passes relative to parameter count.

Executing this traversal in hardware exposes a physical constraint: evaluating local derivatives during the backward pass requires the operating point established during forward execution. The framework must therefore determine which forward tensors to persist in device memory, when to allocate them, and when to free them. Modern frameworks manage this lifecycle through dynamic computational graphs and gradient accumulation.¹⁹

Listing 7.10 illustrates these requirements in a two-layer network, showing the forward pass storing intermediate activations and the backward pass consuming them to evaluate parameter gradients.

19 | Gradient accumulation:

The framework processes a large logical batch as smaller mini-batches, freeing each mini-batch's saved activations after backward while accumulating parameter-sized gradients. Processing 4,096 samples in 64-sample mini-batches can reduce the batch-dependent activation peak by up to $64\times$ when activation memory scales linearly with batch size; weights, gradients, optimizer state, and other fixed costs do not shrink. The trade-off is more sequential work before each optimizer update.

Listing 7.10: Reverse Mode in a Neural Network: The forward pass computes and stores intermediate values; the backward pass walks the computation in reverse to produce gradients for every parameter.

```
def simple_network(x, w1, w2):
    hidden = x * w1 # First layer
    activated = max(0, hidden) # ReLU activation
    output = activated * w2 # Second layer
    return output

# --- Forward pass stores intermediates ---
x, w1, w2 = 1.0, 2.0, 3.0
hidden = x * w1
activated = max(0, hidden)
output = activated * w2

# --- Backward pass consumes them ---
d_output = 1.0 # Seed gradient
d_w2 = activated # = 2.0
d_activated = w2 # = 3.0
d_hidden = d_activated * (1 if hidden > 0 else 0) # ReLU gate: 3.0
d_w1 = x * d_hidden # = 3.0
d_x = w1 * d_hidden # = 6.0
```

Three implementation requirements emerge from this example. First, the framework must track dependencies between operations to determine the correct reverse traversal order. Second, intermediate values (`hidden`, `activated`) must persist in device memory until the backward pass consumes them. Mathematically, this persistence is required because the chain rule evaluates local derivatives at the forward operating point—for example, evaluating $\frac{\partial}{\partial z} \text{ReLU}(z) = \mathbb{I}(z > 0)$ requires the pre-activation z , and computing a dense layer’s weight gradient $\frac{\partial \mathcal{L}}{\partial W} = X^T \delta$ requires the forward input activation tensor X . Third, every operation needs both a forward implementation and a corresponding backward rule. These requirements define the engineering surface of automatic differentiation, where activation memory persistence constitutes the primary scaling bottleneck.

7.4.1.3 Memory management strategies

A 175B-parameter model in FP16 requires 350 GB just for weights, exceeding the physical memory of any single accelerator. Reverse-mode AD compounds this memory pressure by retaining operation-dependent forward activations for backward derivative rules. In an illustrative 100-layer network trained with batch size 64, saved activations contribute an additional 8–12 GB alongside weights, gradients, and optimizer states. Memory capacity, rather than raw floating-point throughput, frequently establishes the feasibility boundary of training.

This saved-activation footprint accumulates across successive layers. Listing 7.11 demonstrates how each forward layer adds an activation tensor that must persist until the backward sweep unwinds to that layer.

Listing 7.11: Activation Persistence Across Layers: The marked intermediates represent values retained for backward; peak saved-activation memory depends on their number and sizes.

```
def deep_network(x, w1, w2, w3):
    # Forward pass - must store intermediates
    hidden1 = x * w1
    activated1 = max(0, hidden1) # Store for backward
    hidden2 = activated1 * w2
    activated2 = max(0, hidden2) # Store for backward
    output = activated2 * w3
    return output
```

Frameworks mitigate this memory wall through two primary architectural strategies. The first is activation checkpointing (or gradient checkpointing): rather than buffering every intermediate

activation, the runtime stores only selected segment boundary tensors and recomputes dropped intermediates on demand during the backward pass. The essential systems mechanism is the runtime contract: the framework marks boundary activations as persistent and regenerates intermediate values when backward execution reaches that segment; section 8.6.5.1 details the placement policies used to partition networks. Listing 7.12 illustrates this contract: the forward pass preserves only segment boundaries, and the backward pass re-executes the forward segment to regenerate activations rather than loading them from global memory.

Listing 7.12: Activation Checkpointing: The forward pass stores only segment boundaries; the backward pass re-runs each segment to regenerate the dropped activations, trading recomputation for memory.

```
# Standard backward: every forward activation stays resident
h1 = layer1(x) # kept for backward
h2 = layer2(h1) # kept for backward
out = layer3(h2) # kept for backward

# Checkpointed: keep only the boundary h1 and drop h2. The
# backward pass re-runs the wrapped segment to regenerate
# h2 on demand instead of holding it in memory.
h1 = layer1(x) # boundary: kept
out = checkpoint(
    lambda a: layer3(layer2(a)), h1
) # h2 recomputed in backward
```

The second strategy is *operation fusion*.²⁰ Rather than executing matrix multiplication, bias addition, and activation functions as separate kernels that serialize intermediate tensors to accelerator global memory (HBM), compilers fuse compatible operators into a single kernel that passes intermediate values through registers or on-chip shared memory.

The backward pass itself relies on specialized hardware-optimized kernels. Rather than synthesizing backward passes from naive elementary operations, frameworks invoke tuned backward kernels (such as cuDNN primitives) that optimize memory access strides and register usage on accelerator hardware (Chetlur et al. 2014). Combining checkpointing, kernel fusion, and specialized backward kernels enables large architectures to train within physical accelerator memory limits without incurring out-of-memory faults.

20 | Operation fusion: When compatible operations execute as separate kernels, intermediate results may be written to HBM and read back by the next kernel. Fusion can keep those values on chip and avoid the corresponding transfers. The realized benefit depends on the operations, shapes, compiler, and hardware.

7.4.2 Framework implementation of automatic differentiation

Checkpointing, fusion, and specialized kernels address the physical bottlenecks of automatic differentiation, but production frameworks encapsulate these mechanisms beneath high-level training loops. A standard PyTorch step—`optimizer.zero_grad()`, forward pass, `loss.backward()`, and `optimizer.step()`—appears as four sequential calls. Behind this interface, the runtime builds and traverses dynamic computation graphs, allocates and frees intermediate buffers, schedules backward kernels across CUDA streams, and manages numerical precision. The same graph machinery supports second-order derivatives via nested `torch.autograd.grad` calls and automatic mixed precision via `autocast`, which dispatches reduced-precision kernels while preserving FP32 master weights.

7.4.2.1 PyTorch autograd internals

The autograd system is the engine that resolves the differentiation problem introduced in section 7.1. Three systems principles govern its design: the data structure that enables efficient gradient computation, the memory cost of maintaining that data structure, and the control mechanisms that production workflows require. Reasoning through these principles clarifies why training memory substantially exceeds inference memory for identical architectures, and how frameworks manage that overhead.

The reverse-linked graph structure During the forward pass, autograd dynamically constructs a directed acyclic graph of `Node` (or `Function`) objects. Each node encapsulates an operator’s backward differentiation logic and retains references to the tensors required to evaluate its vector-Jacobian

products. Because edges point backward from outputs toward inputs, the engine initiates topological traversal immediately upon receiving the scalar loss adjoint.

Every tensor produced by a differentiable operation contains a `grad_fn` attribute referencing the Function that computed it. Each Function links to antecedent nodes through its `next_functions` list, forming a chain from the scalar loss back to leaf parameter tensors. Listing 7.13 inspects this pointer chain for a simple composite computation:

Listing 7.13: Reverse-Linked Graph Structure: Each tensor’s `grad_fn` links to the Function that created it, forming a reverse chain from output to leaf parameters that enables $\mathcal{O}(1)$ backward passes.

```
import torch

x = torch.tensor([2.0], requires_grad=True)
y = x * 3
z = y.pow(2)

# Traverse the reverse-linked graph
print(z.grad_fn) # PowBackward0
print(z.grad_fn.next_functions) # -> MulBackward0
print(
    z.grad_fn.next_functions[0][0].next_functions
) # -> AccumulateGrad (leaf)
```

The inspection exposes the traversal path: `PowBackward0` (from `z = y.pow(2)`) links to `MulBackward0` (from `y = x * 3`), which terminates at `AccumulateGrad` on the leaf parameter tensor `x`. Leaf tensors are graph sinks where accumulated gradients write directly into the `.grad` buffer rather than propagating further. The tuple format (Function, index) tracks the specific output index for multi-output operations.

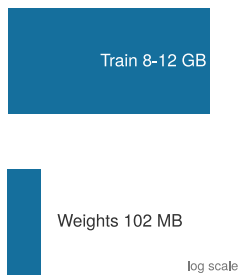
This reverse-linked graph imposes a fundamental memory constraint: the graph topology and all referenced forward tensors must remain resident in device memory until backward traversal executes their respective nodes. While the graph metadata (pointers and node descriptors) occupies minimal memory, the tensor buffers it references can consume gigabytes of accelerator capacity, scaling directly with model depth and batch size.

The memory-compute trade-off Every forward activation saved for the backward pass remains pinned in device memory until its corresponding derivative rule executes. These retained activations represent a primary driver of training memory overhead, compounding the footprint of weights, gradients, and optimizer states. Because different operations require different forward tensors—for example, matrix multiplication requires input activations, while elementwise ReLU requires only the pre-activation sign bit—the exact memory profile depends on operator semantics.

Consider an illustrative ResNet-50 training scenario with 25.6M parameters (~102.4 MB of FP32 weights), batch size 64, and 224×224 images. With 8 GB–12 GB of saved activations, gradients add ~102.4 MB, and two Adaptive Moment Estimation (Adam) FP32 moment buffers add ~204.8 MB. Summing those four components gives an 8.4 GB–12.4 GB training scenario, roughly 82.1–121.2× the FP32 weight storage alone. This metric reflects peak training memory, distinct from inference memory which requires no optimizer states, gradients, or backward activation buffers.

This disparity underscores why accelerator training capacity cannot be deduced from parameter counts alone. Total training memory obeys the four-component formulation ($M_{\text{total}} = M_{\text{weights}} + M_{\text{gradients}} + M_{\text{optimizer}} + M_{\text{activations}}$) plus allocator overhead, as derived in section C.3.3.

Frameworks provide three graph-level controls to manage this trade-off. Gradient checkpointing (T. Chen et al. 2016) alters what the graph retains: the engine stores only coarse segment boundaries and regenerates intermediate activations during the backward pass. In iron law terms, checkpointing increases the arithmetic operation count (O) to curtail memory footprint and transfer volume (D_{vol}). Tensor detachment provides a complementary optimization: calling `.detach()` removes a tensor from autograd tracking, preventing the engine from retaining activations along upstream graph paths. This is essential for transfer learning where frozen feature extractors require no gradient computation, eliminating unnecessary activation storage (D_{vol}). Mixed-precision training provides



The illustrative training peak dwarfs weight storage alone.

a third lever: representing activations and weights in 16-bit formats directly halves tensor volume (D_{vol}) while accelerating arithmetic throughput on Tensor Cores (R_{peak}). Chapter 8 formalizes these graph mechanisms into cluster sizing decisions.

Extensibility and control Production training systems require fine-grained control over gradient propagation beyond standard backward sweeps. In practice, workloads demand three core capabilities: selective gradient computation for freezing layers during transfer learning via `requires_grad=False` or `.detach()`; gradient inspection and modification for clipping, debugging, or logging intermediate adjoints via runtime hooks; and custom differentiation rules for non-standard operations such as specialized CUDA kernels or novel activation functions.

These extension points operate via callbacks invoked during graph construction and reverse traversal, preserving the core engine’s execution path. In PyTorch, these capabilities map to specific runtime mechanisms: retained graphs, gradient accumulation buffers, custom `autograd.Function` implementations, tensor hooks, and detachment boundaries. Table 7.6 summarizes these mechanisms, their systems function, and their memory or correctness trade-offs.

Table 7.6: Autograd Control Mechanisms: The recurring extension points the autograd engine exposes during graph traversal, what each controls, and the memory or correctness cost each carries.

Mechanism	What it controls	Cost or hazard
<code>retain_graph=True</code>	Keeps saved graph state available for another backward traversal	Retains memory that would otherwise be released; the amount depends on the graph and saved values
Gradient accumulation	Successive backward passes sum into <code>.grad</code> until <code>zero_grad()</code> resets them, enabling large effective batches	Forgetting the reset silently mixes gradients across optimization steps
Custom autograd.Function	User-defined forward and backward rules for operations outside the built-in library	Moves the differentiation contract (what to save, how to differentiate) to the implementer
Gradient hooks	Inspecting or modifying gradients mid-traversal (clipping, logging, debugging)	Runs arbitrary Python per registered tensor on every backward pass
<code>.detach()</code>	Cuts gradient flow at a chosen boundary (frozen layers, inference outputs)	The legacy <code>.data</code> attribute bypasses autograd and silently corrupts gradients; clone before in-place mutation

Custom autograd functions expose the differentiation contract directly by transferring state retention responsibility from the engine to the programmer: the implementation explicitly defines which forward tensors to preserve in `ctx.save_for_backward()` and evaluates the corresponding vector-Jacobian products in `backward()`. Listing 7.14 demonstrates this interface for a custom multiply-add operation:

Listing 7.14: Custom Autograd Function: Implement forward and backward methods to define custom differentiable operations, explicitly specifying tensors to save for gradient computation.

```
class MultiplyAdd(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, y, z):
        # Save tensors needed for backward
        ctx.save_for_backward(x, y)
        return x * y + z

    @staticmethod
    def backward(ctx, grad_output):
        # Retrieve saved tensors
        x, y = ctx.saved_tensors

        # Compute gradients using chain rule
        grad_x = grad_output * y # dL/dx = dL/dout * dout/dx
        grad_y = grad_output * x # dL/dy = dL/dout * dout/dy
        grad_z = grad_output # dL/dz = dL/dout * 1

        return grad_x, grad_y, grad_z

# Usage
x = torch.tensor([2.0], requires_grad=True)
y = torch.tensor([3.0], requires_grad=True)
z = torch.tensor([1.0], requires_grad=True)

output = MultiplyAdd.apply(x, y, z)
output.backward()

print(
    x.grad, y.grad, z.grad
) # tensor([3.]), tensor([2.]), tensor([1.]
```

These autograd mechanisms connect directly to the silicon contract. The reverse-linked graph defines the sequence of backward kernels the device must execute (the O term). The memory-compute trade-off governs intermediate tensor residency and memory traffic through the memory hierarchy (the D_{vol} term). The extensibility hooks allow engineers to prune unnecessary graph paths and intercept data flow. The interplay between graph retention and memory bandwidth leads directly to mixed-precision training, which reduces memory footprint while increasing hardware execution efficiency.

7.4.2.2 Mixed-precision training support

Mixed precision exploits hardware specialization to improve two iron law terms simultaneously: Tensor Cores execute 16-bit matrix multiplications at higher throughput than FP32 arithmetic units (increasing R_{peak} and reducing $O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$), while 16-bit activations halve global memory traffic (reducing D_{vol}). Improving both terms simultaneously is rare; most systems optimizations improve compute efficiency at the expense of memory footprint, or vice versa.

Automatic mixed-precision APIs dynamically assign numerical precisions based on operator sensitivity and hardware capabilities. Compute-intensive operations such as convolutions and linear projections execute in FP16 or BF16, while numerically sensitive operations such as softmax, reductions, and loss calculations execute in FP32. Because FP16 has a narrow dynamic range (5 exponent bits, with normal values bounded below by $\approx 6.10 \times 10^{-5}$), gradient values can underflow to zero during backpropagation. Loss scaling counters underflow by multiplying the scalar loss by a scale factor S prior to backward traversal, shifting gradients into FP16 representable range, and dividing parameter gradients by S before optimizer updates.

Frameworks support three primary reduced-precision formats: IEEE FP16, BF16,²¹ and TF32 (an NVIDIA Ampere+ internal format that pairs FP32's 8-bit exponent with FP16's 10-bit mantissa for matrix multiplies). BF16 matches FP32's 8-bit dynamic range at the expense of mantissa precision

21 | BF16 design rationale:

Developed by Google Brain circa 2018 specifically for TPU training stability, BF16 preserves FP32's eight-bit exponent range while halving memory footprint—an explicit trade-off of mantissa precision (7 bits vs. FP16's 10) for dynamic range. FP16's smallest positive normal value is approximately 6.10×10^{-5} , but subnormal values extend to approximately 5.96×10^{-8} ; loss scaling is especially important on hardware or in modes that flush those subnormals to zero. BF16's FP32-matched exponent largely avoids this class of gradient underflow, eliminating the need for loss scaling in most workloads, which is why BF16 and FP16 are not interchangeable: BF16 is preferred when training stability matters; FP16 is preferred when numerical precision matters more than gradient stability.

(7 bits), preventing gradient underflow and eliminating the need for dynamic loss scaling in most workloads. Section 8.6.3 details the numerical stability and memory characteristics of these formats. Listing 7.15 demonstrates PyTorch’s mixed-precision API, pairing the autocast context manager with GradScaler to prevent underflow.

Listing 7.15: Mixed-Precision API: Modern frameworks provide automatic mixed-precision support through context managers that handle precision selection and numerical stability.

```
import torch
from torch.amp import autocast, GradScaler

model = MyModel().cuda()
optimizer = torch.optim.Adam(model.parameters())
scaler = GradScaler("cuda")

for inputs, targets in dataloader:
    inputs, targets = inputs.cuda(), targets.cuda()
    optimizer.zero_grad()

    # Framework automatically selects precision per operation
    with autocast(device_type="cuda", dtype=torch.float16):
        outputs = model(inputs)
        loss = criterion(outputs, targets)

    # GradScaler handles gradient scaling for numerical stability
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
```

BF16 training avoids this loss-scaling machinery entirely. As listing 7.16 shows in comparison to listing 7.15, removing GradScaler simplifies the training loop: BF16’s dynamic range eliminates underflow risk, allowing direct backward execution.

Listing 7.16: BF16 Training: BF16 maintains FP32’s dynamic range, often avoiding the loss scaling used for FP16 training.

```
# BF16 training typically does not require loss scaling
with torch.autocast(device_type="cuda", dtype=torch.bfloat16):
    outputs = model(inputs)
    loss = criterion(outputs, targets)
loss.backward() # No GradScaler needed
optimizer.step()
```

Optimizer state and checkpointing Resuming training after preemption or hardware failure requires restoring model weights and optimizer states in lockstep: first-moment vectors, second-moment variance estimates, and step counters. For Adam, storing two FP32 states per parameter adds $4\times$ the memory of FP16 model weights, making the combined footprint $5\times$ the weight baseline. For a 7-billion-parameter model, weights and optimizer states require approximately 70 GB total (14 GB weights plus 56 GB optimizer states). Checkpoint volume directly governs recovery latency across distributed storage, connecting fault tolerance to the iron law’s D_{vol} term.

Chapter 8 analyzes optimizer memory optimization strategies in distributed training. At the framework level, optimizers expose internal state dictionaries via `state_dict()` for serialization. Listing 7.17 demonstrates accessing optimizer state dictionaries, and listing 7.18 shows saving and restoring both model parameters and optimizer states to preserve training trajectory continuity.

Listing 7.17: State Dictionary Interface: Optimizers expose internal state through `state_dict()`, enabling serialization of momentum buffers and adaptive learning rate estimates for checkpointing.

```
import torch
import torch.nn as nn
import torch.optim as optim

model = nn.Linear(10, 5)
optimizer = optim.Adam(model.parameters(), lr=0.001)

# After training steps, optimizer accumulates state
loss = model(torch.randn(3, 10)).sum()
loss.backward()
optimizer.step()

# Access state for checkpointing
state = optimizer.state_dict()
# Contains: {'state': {...}, 'param_groups': [{'lr': 0.001, ...}]}
```

Listing 7.18: Checkpoint Save and Load: Save both model parameters and optimizer state to properly resume training with correct momentum and adaptive learning rate values.

```
# Saving checkpoint
checkpoint = {
    "epoch": epoch,
    "model_state_dict": model.state_dict(),
    "optimizer_state_dict": optimizer.state_dict(),
}
torch.save(checkpoint, "checkpoint.pt")

# Resuming training
checkpoint = torch.load("checkpoint.pt")
model.load_state_dict(checkpoint["model_state_dict"])
optimizer.load_state_dict(checkpoint["optimizer_state_dict"])
```

The mathematical foundations of automatic differentiation predated the deep learning resurgence by decades; the engineering challenge was mechanizing those principles into high-performance software infrastructure. Manually deriving and writing backward kernels for a single layer requires tracking intermediate variables, implementing local chain rules, and managing device buffer lifecycles. For modern transformer architectures containing multi-head self-attention, layer normalizations, and residual projections across hundreds of layers, manual differentiation across thousands of tensor dependencies is infeasible. Autograd engines mechanize this derivation across arbitrary topologies, precision modes, and accelerator backends, transforming an intractable analytical problem into automated graph execution.

7.4.2.3 Memory management in gradient computation

The memory management strategies examined in section 7.4.1.3 (checkpointing, gradient accumulation) exist because reverse-mode AD requires preserving computational history. As listing 7.11 demonstrated, each forward layer adds an activation tensor that must persist until backward traversal consumes it. This creates a memory wave that peaks at the onset of backpropagation and recedes as gradients accumulate and intermediate buffers are released. Modern runtimes track tensor reference counts to free activations the moment their backward consumers finish executing.

Beyond tensor lifetimes, frameworks face a lower-level systems bottleneck: driver-level allocation latency. Invoking synchronous device memory allocations (such as `cudaMalloc`) incurs substantial OS and driver overhead, forcing host-device synchronizations that drain accelerator command queues. To eliminate this latency, modern frameworks intercept allocations with **caching allocators** that maintain pools of pre-reserved device memory blocks across training iterations. Suballocating

from these pools amortizes device allocation costs and reduces kernel launch latency, though varying tensor lifetimes can still induce memory fragmentation.

Systems Perspective 7.2: Caching allocator and utilization

Caching allocators primarily reduce allocation overhead and improve block reuse. Two considerations remain:

1. Allocation latency: Reusing a cached block avoids repeated device-allocation calls, whose cost and synchronization behavior depend on the allocator and runtime.
2. Fragmentation: Size classes and block splitting improve reuse but can leave free capacity in blocks that do not satisfy a request. Because tensors require contiguous physical addresses, external fragmentation occurs when variable tensor lifetimes leave small, non-contiguous free blocks in the pool; a large allocation can trigger an out-of-memory (OOM) error even when the total aggregate free memory across all blocks exceeds the requested size.

An out-of-memory error can reflect live allocations, allocator-reserved blocks, fragmentation, or another process. `nvidia-smi` reports device-level usage and is not sufficient by itself to diagnose the framework allocator; allocator-specific memory statistics are needed.

7.4.2.4 Production system integration challenges

A training iteration can run more slowly in production than in an isolated profile because the full step coordinates with the memory allocator, device manager, operation scheduler, data pipeline, and optimizer. The full-system cost includes work that an operator microbenchmark or isolated kernel profile does not measure. Gradient computation launches accelerator kernels and may require allocation, synchronization, or data movement according to the graph and implementation. These interactions can dominate small-operation workloads and remain measurable at scale. The gap between what the programmer writes (a five-line training loop) and what the system executes (allocations, kernel launches, and synchronization points) is the central tension of AD system design.

The severity of this overhead depends on a deeper architectural choice: how the AD system records and replays computation in the first place. A framework that builds a dynamic trace at runtime pays per-operation bookkeeping on every forward pass, and its caching allocator must accommodate allocation patterns that can vary with execution. A framework that captures a stable computation, by contrast, may pre-plan buffers and reuse storage across the forward and backward graphs because it can reason about lifetimes before execution. The gain depends on capture coverage, tensor lifetimes, shapes, and backend support.

Systems Perspective 7.3: Tape-based vs. transform-based autodiff

PyTorch (tape-based): Records operations on a dynamic “tape” during the forward pass. This is flexible and easy to debug but makes it hard for a compiler to see the whole graph at once for global optimization.

JAX (transform-based): Treats automatic differentiation as a high-level function transformation (`grad(f)`). JAX traces a pure function for abstract input signatures, allowing compatible transformations such as `jit(grad(f))` or `vmap(grad(f))` to compose before lowering. This exposes a larger transformed program to accelerator compilers without guaranteeing a single kernel or a universal speed advantage.

The AD system tracks dependencies so it can traverse independent graph branches in a valid order and accumulate gradients correctly. While independent host threads or explicit CUDA streams can expose device concurrency, default autograd dispatch serializes node execution across the primary compute stream unless explicitly directed otherwise. Concurrency is bounded by stream dependencies and hardware scheduling limits.

7.4.2.5 How different frameworks implement AD

The execution models analyzed in section 7.3—eager, static graph, and hybrid—directly dictate how frameworks implement automatic differentiation. PyTorch (Paszke et al. 2019) constructs its autograd tape dynamically during forward execution. Its `grad_fn` chain mechanism, examined in section 7.4.2.1, grants unconstrained Python control flow and transparent breakpoint debugging, but forces the engine to retain tape metadata and saved tensors until backward traversal completes. Conversely, early TensorFlow (Abadi et al. 2016) performed symbolic differentiation ahead of time during graph construction, generating an immutable backward sub-graph amenable to whole-graph dead-code elimination and memory reuse before execution. Modern TensorFlow 2.x adopts eager execution as its default interface while exposing `tf.function` to compile static differentiation graphs when throughput requires it (TensorFlow Developers 2024a). JAX²² (Frostig et al. 2018) rejects tape tracking entirely, treating differentiation as a pure function transformation: `jax.grad()` accepts a function and produces a derivative function, enabling seamless algebraic composition with vectorization (`jax.vmap()`) and XLA compilation (`jax.jit()`). While this functional paradigm demands strict referential transparency, it exposes clean whole-program IR to the compiler without the stateful side effects of imperative tapes.

Autodiff implementation differences determine how much debugging visibility, compiler optimization, and deployment portability a team can expect from each framework. A recurring tension runs through every AD design decision: mathematical correctness demands storing computational history, but hardware imposes strict memory limits. Every framework resolves this tension differently, choosing which activations to checkpoint, which operations to fuse, and how aggressively to trade recomputation for memory. These choices determine which models can train on which hardware, making AD system design one of the most consequential engineering decisions in any framework.



Checkpoint 7.2: The systems cost of gradients

Training is inherently more expensive than inference because of Automatic Differentiation.

Computational reality

- ☐ **Reverse mode AD:** why is it efficient for a scalar loss with many parameters?
- ☐ **The activation tax:** can you explain how saved-value memory depends on graph depth, tensor shapes, and checkpointing?

Optimization mechanics

- ☐ **Gradient checkpointing:** how does recomputing activations save memory?

The execution and differentiation problems together enable the training loop: the execution model determines when computation happens, while automatic differentiation computes the gradients that drive learning. Both problems, however, quietly assume something that cannot be taken for granted: that the same code can run across diverse hardware. A model trained on an NVIDIA A100 must serve inference on a mobile phone’s ARM CPU, a Google TPU, or a microcontroller with kilobytes of memory. The same `torch.matmul` call must dispatch to cuBLAS on one device and a hand-tuned ARM NEON kernel on another. This hardware diversity creates the third problem.

7.5 Abstraction Problem

This hardware diversity is architecturally fundamental. GPUs expose abundant throughput-oriented parallelism but have different memory and execution semantics from CPUs. TPUs favor regular tensor programs and compiler-visible shapes. A microcontroller has kilobytes where a server has gigabytes. The **abstraction problem** therefore requires frameworks to hide this complexity behind a single programming interface while still enabling efficient utilization of each target’s unique capabilities.

The problem decomposes into two interacting dimensions. The first is *data representation*: how frameworks encode tensors, parameters, and computational state in forms that work across hardware. The second is *execution mapping*: how high-level operations translate into hardware-specific

²² | **JAX:** JAX exposes `grad`, `jit`, and `vmap` as transformations on functions; they can be composed when their input and output contracts align, but order matters: vectorizing gradients is not generally the same operation as differentiating a vector-valued result. Compilation can produce fused regions, library calls, and multiple kernels rather than one universal kernel. Transformed functions should be pure; Python side effects may occur during tracing but are not represented in the compiled program, while supported callbacks express intentional runtime effects.

implementations. These dimensions are not independent concerns. The way data is represented (memory layout, precision, device placement) directly affects *what* execution strategies are possible. A tensor stored in row-major format on a GPU requires different kernels than one in column-major format on a CPU. A model quantized to INT8 enables entirely different execution paths than FP32.

Solving the abstraction problem requires sophisticated software infrastructure: tensor representations that encode both mathematical semantics and hardware constraints, intermediate representations that enable hardware-specific compilation, and runtime systems that manage data movement across the memory hierarchy. To make this concrete, trace what must happen when a programmer writes `model(input)`. The framework must resolve five decisions in rapid succession: data representation (tensor shape, memory layout, numeric precision), device placement (the bandwidth hierarchy connecting CPU, GPU, and accelerator memory), input delivery (data pipelines that sustain hundreds of MB/s to keep the accelerator fed), model organization (parameters, buffers, and submodules that must move together), and kernel execution (dispatch, scheduling, and resource optimization). These decisions build from the data container up to the hardware execution layer. Distributed placement previews the same abstraction boundary: frameworks expose placement scopes so system components can manage device boundaries, while chapter 8 analyzes the scaling algorithms and communication costs that make those scopes efficient.

7.5.1 Data structures and tensor abstractions

A ResNet-50 forward pass touches 25.6M parameters, produces intermediate activations at every layer, and must coordinate memory across CPU and GPU address spaces. Frameworks organize all of this data so that a single `model(input)` call executes millions of operations without the programmer managing a single pointer, by solving four problems in sequence: defining a universal data container (tensors), placing it on the right device (memory management), feeding data fast enough (data pipelines), and dispatching the right hardware kernel (core operations). The path runs from data representation to hardware execution.

Computational graphs specify the *logical* flow of operations, but data structures determine how those operations access and manipulate data in *physical* memory. This distinction matters because the same mathematical operation can differ by an order of magnitude in throughput depending on whether data is contiguous in cache, pinned for direct memory access (DMA) transfer, or scattered across pages.

The first step is the data container itself. Framework data structures must sustain memory bandwidth (hundreds of GB/s on modern GPUs), accommodate architectures from 1D sequences to 5D video tensors, and hide device management behind clean APIs. Tensors are the universal answer.

7.5.1.1 Tensors

At the foundation of every framework's data representation lies a single abstraction: the tensor, an n -dimensional array that pairs numerical values with the information needed to interpret and place them.

Every computation in a neural network operates on tensors.²³ Training batches, activation maps, parameter gradients, and optimizer states are all tensors. This unified representation lets frameworks optimize a single data structure for hardware rather than managing separate containers for each role.



Definition 7.2: Tensor

Tensors are n -dimensional arrays with shape, data type, stride, and device metadata. Framework runtimes use this information to select compatible operations and hardware kernels.

1. **Significance:** A contiguous FP32 tensor of shape $[1024, 1024]$ occupies $1024 \times 1024 \times 4 = 4,194,304$ bytes (about 4.2 MB). A noncontiguous view may be accepted directly by a strided kernel or may require materialization when an operation demands a supported contiguous layout.

²³ | **Tensor:** In mathematics, tensors obey coordinate-transformation laws; ML frameworks use the term more broadly for multidimensional arrays carrying shape, dtype, strides, device placement, and often automatic-differentiation metadata. A transpose commonly creates a strided view, while reshape may return either a view or a copy depending on the layout. These storage semantics make layout relevant to kernel selection and data movement.

2. **Distinction:** NumPy arrays also carry shape, dtype, and stride metadata. Framework tensors additionally integrate device placement, automatic differentiation, and accelerator dispatch.
3. **Common pitfall:** Tensor operations may return a new allocation, a view sharing existing storage, or an in-place update, depending on the operation and API variant. Memory accounting must distinguish these cases.

The tensor abstraction consumes far more memory than model weights alone suggest. Engineers who estimate memory from parameter count alone allocate accordingly and encounter out-of-memory errors that seem inexplicable. A quick memory accounting makes the hidden cost concrete: gradients, optimizer momentum, and stored activations accompany every weight tensor.



Napkin Math 7.3: The administrative tax

The illustrative ResNet-50 breakdown in section 7.4.2.1 was $\sim 82.1\text{--}121.2\times$ larger than FP32 weight storage alone. The following scenario expands the accounting to gradients, optimizer state, and saved activations at billion-parameter scale.

Problem: How much memory does training state add beyond FP16 weights in this billion-parameter scenario?

Math:

1. Model weights: 2 GB.
2. Gradients: 2 GB (same size as weights).
3. Optimizer states (Adam): 8 GB ($4\times$ FP16 weight memory for momentum and velocity stored in FP32).
4. Activations: For a batch size of 32 and a 100-layer network, reverse-mode AD retains the operation-dependent values required by derivative rules; activation checkpointing can omit selected values and recompute them during the backward pass.

$$\text{Activations} \approx B \times N_L \times S \times d_{\text{model}} \times n_{\text{saved}} \times 2 \text{ bytes}$$

For a 1024-token sequence, 1024-wide hidden state, and 1 saved tensor per layer: $32 \times 100 \times 1024 \times 1024 \times 1 \times 2 \approx \mathbf{6.7\text{GB}}$. Materialized attention terms can add a separate $B \times N_{\text{heads}} \times S^2$ component, which is why memory-efficient attention kernels matter.

Systems insight: A 2 GB model carries persistent training state before the first batch is processed (2 GB gradients + 8 GB optimizer state). During a training step, batch-dependent activations add another 6.7 GB, raising the peak “administrative tax” to ~ 16.7 GB beyond the weights. During training, data movement includes saving and retrieving these activations.

7.5.1.2 Tensor structure and dimensions

Tensors structure numerical data by adding organizational axes to linear memory layouts. Trace the rank expansion from left to right in figure 7.8, observing how rank 0 scalars evolve into rank 3 volume tensors.

In vision applications, raw image inputs map directly onto multi-dimensional tensor layouts. Examine the rank 3 tensor structure in figure 7.9, observing how red, green, and blue color channels stack along the channel dimension.

Framework tensors carry more than raw numbers. Each tensor stores **tensor metadata**, runtime information used to validate operations and select fast execution paths: a *shape* tuple (for example, `[64, 3, 224, 224]` for a batch of images), a *dtype* (framework literals such as `float32`, `float16`, or `int8`), and a *device* tag (CPU, `cuda:0`). A matrix multiplication, for instance, checks shape compatibility at dispatch time and uses the dtype to route to the correct hardware kernel, whether a standard FP32 GEMM or a Tensor Core FP16 path.

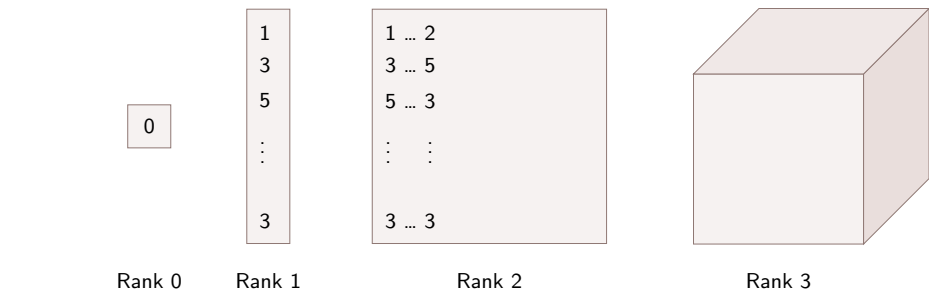


Figure 7.8: Tensor Rank Hierarchy: Ascending tensor ranks (0 to 3) generalize multi-dimensional indexing from scalars to volume tensors. Each additional rank adds an organizational axis that the framework must linearize into physical memory addresses via strides.

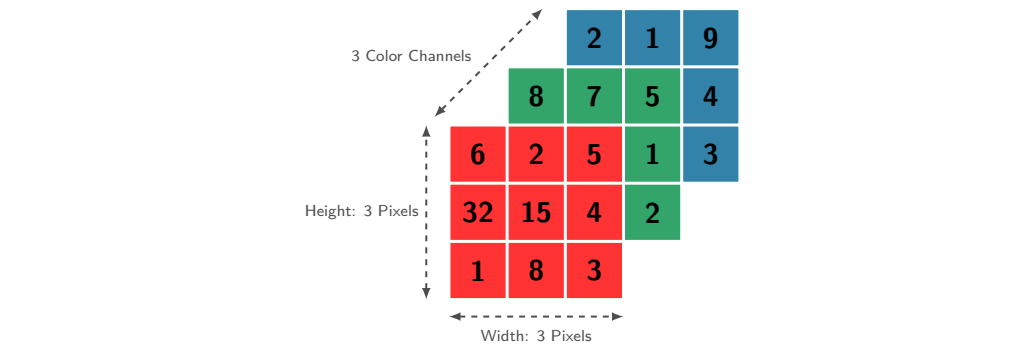


Figure 7.9: Image as an RGB Tensor: Decomposing a 2D color image into stacked height, width, and channel dimensions forms a rank-3 tensor. In ML systems, the ordering of these dimensions (such as channel-first vs. channel-last) determines memory access locality and cache efficiency during convolution operations.

Memory layout implementation introduces distinct challenges in tensor design. While tensors provide an abstraction of multi-dimensional data, physical computer memory remains linear. Stride patterns address this disparity by creating mappings between multi-dimensional tensor indices and linear memory addresses. These patterns significantly impact computational performance by determining memory access patterns during tensor operations. Figure 7.10 makes this concrete with a 2×3 tensor: follow the same six values as they map into two different linear orderings—row-major and column-major—and note how the stride values change to compensate.

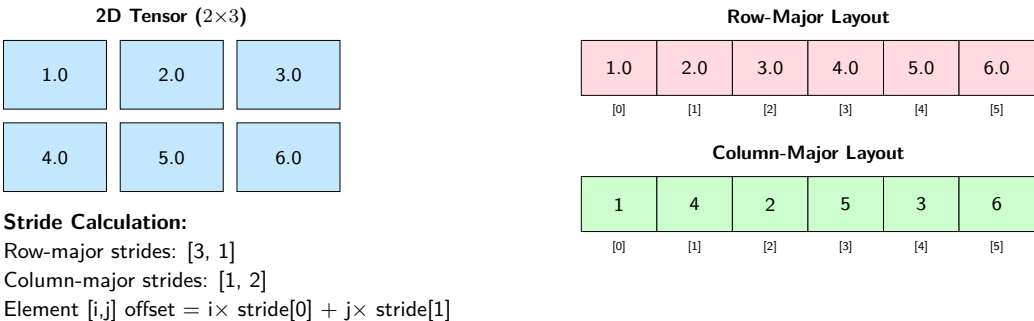


Figure 7.10: Tensor Memory Layout: A 2×3 tensor can be stored in linear memory using either row-major (C-style) or column-major (Fortran-style) ordering. Strides define the number of elements to skip in each dimension when moving through memory, enabling frameworks to calculate memory addresses for tensor[i,j] as `base_address + i × stride[0] + j × stride[1]`. The choice of memory layout significantly impacts cache performance and computational efficiency.

Stride choices become performance choices. Row-major layout (used by NumPy, PyTorch) stores elements row by row, making row-wise operations more cache-friendly. Column-major layout (used

by some BLAS libraries) stores elements column by column, optimizing column-wise access patterns. The stride values encode this layout information: in row-major layout for a 2×3 tensor, moving to the next row requires skipping three elements (`stride[0] = 3`), while moving to the next column requires skipping one element (`stride[1] = 1`).

These memory layout details have direct performance implications, but no ordering is universally superior. Efficient access depends on which dimension an operation traverses, the kernel’s supported layouts, vectorization, and the target memory hierarchy. A mismatched layout may require strided access or a conversion, while a kernel designed for that layout can access it efficiently. Crucially, geometric operations such as `.transpose()` are zero-copy: they merely swap stride metadata to create a non-contiguous “view” over existing memory. When a subsequent operation (such as `.view()`) requires elements to reside sequentially in memory, calling `.contiguous()` forces the runtime to allocate a new buffer and physically copy the elements into sequential order.

The dtype is the tensor-level lever that trades numerical range against data movement. The standard choice in machine learning has been FP32 precision, exposed in frameworks through dtype literals such as `float32`, offering a balance of precision and efficiency. Modern frameworks extend this with multiple numeric types for different needs. Integer types support indexing and embedding operations. Reduced-precision types such as FP16 enable efficient mobile deployment. INT8 precision allows fast inference on specialized hardware. The choice of numeric type affects both model behavior and computational efficiency: neural network training typically requires FP32 precision for critical accumulations to maintain stable gradient computations, while inference tasks can often use lower precision (framework dtype literals such as `int8` or even `int4`, corresponding to INT8 and INT4), reducing memory usage and increasing processing speed. Mixed-precision training approaches combine these benefits by using FP32 for critical accumulations while performing most computations at lower precision.

Type conversions become another point where abstraction meets physics. Operating on tensors with different types demands explicit conversion rules to preserve numerical correctness. These conversions introduce computational costs and risk precision loss. Frameworks provide type casting capabilities but rely on developers to maintain numerical precision across operations.

Tensors answer the data-representation problem by encoding shape, layout, and precision into a single abstraction. A perfectly shaped tensor on the wrong device, however, or one that must cross the PCIe-to-HBM bandwidth gap to reach the GPU, can erase every layout optimization. The next problem is placement: where data lives and how it moves.

7.5.1.3 Device and memory management

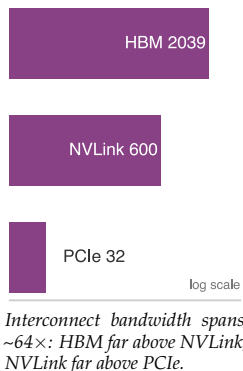
Tensors and their memory layouts establish what the framework computes with. Where that data physically resides, and how it moves between locations, determines whether computation happens at full speed or crawls.

Frameworks as the operating system interface While the high-level API focuses on math, the framework’s backend functions as the operating system of the Single-Machine Stack. It manages the two critical resources of a single node: compute scheduling and data movement.

The CUDA Runtime serves as this OS layer, providing the low-level primitives for launching kernels and managing device memory. The framework coordinates with this runtime to implement DMA over the PCIe bus. The bandwidth gap between the host (CPU) and device (GPU) is the primary “Data Loading Bottleneck”: section D.3.4 models this transfer, separating the two constraints, $T = L_{\text{lat}} + D_{\text{vol}}/\text{BW}$, that determine whether a given data-loading strategy is limited by per-transfer latency or by sustained bandwidth. Frameworks mitigate this through pinned memory (page-locked host memory), which enables DMA transfers without pageable-memory staging. This “HW/OS” interface is what makes high-throughput training loops possible on a single machine.

Every tensor resides on a specific device, and cross-device operations incur transfer costs that can dominate execution time. PCIe 4.0 delivers 32 GB/s between CPU and GPU, while HBM2e provides 2.04 TB/s within the GPU. This 63.7× bandwidth gap means a single misplaced tensor transfer can erase the entire speedup from GPU acceleration.

Device placement matters for framework design because the framework must track where every tensor lives and enforce that operations only combine tensors on the same device. When data must move, the framework must decide whether to block execution or overlap the transfer with other work.



These decisions, invisible to most users, can move a training loop from transfer-bound execution toward substantially higher hardware utilization.

Three systems principles govern effective device and memory management: understanding the bandwidth hierarchy that constrains data movement, overlapping computation with communication to hide transfer latency, and using fine-grained synchronization to maintain correctness without sacrificing concurrency, supported by quantitative analysis grounded in the iron law’s data movement term. The bandwidth hierarchy provides the first constraint.

The cost of moving data between devices varies by orders of magnitude depending on the interconnect.²⁴ Table 7.7 shows transfer times for a 1000×1000 float32 tensor (4 MB)—roughly the size of a typical activation tensor in a moderately sized model. The numbers reveal why careless device placement can erase any speedup from GPU acceleration.

Table 7.7: Device Transfer Overhead: Transfer time for a 4 MB tensor across different interconnects. PCIe bandwidth shown is unidirectional (typical for GPU transfers), with full-duplex operation providing $2 \times$ total bandwidth. NVLink bandwidth is bidirectional (300 GB/s per direction). These transfer costs can dominate lightweight operations, making device placement critical.

Interconnect	Bandwidth	Transfer Time	Path
PCIe 3.0 x16	15.8 GB/s	0.254 ms	Host to device
PCIe 4.0 x16	32 GB/s	0.125 ms	Host to device
NVLink 3.0	300 GB/s per direction (600 GB/s bidirectional)	0.013 ms	GPU to GPU
GPU Memory	2039 GB/s	0.002 ms	On device

These numbers connect directly to the iron law of performance. Every cross-device transfer contributes to $(D_{\text{vol}}/\text{BW})$ at a fraction of on-device bandwidth. Dividing 1 GB by the stated PCIe 4.0 peak gives 31.2 ms, a bandwidth-only lower bound that omits protocol, launch, synchronization, topology, and contention overhead. Transfers can dominate lightweight or small-batch workloads, so deployed latency must be measured rather than inferred from peak bandwidth alone.

Device placement discipline protects the bandwidth hierarchy from accidental PCIe traffic. Every tensor carries a device attribute, and frameworks enforce a strict invariant: operations can only combine tensors on the same device. A `RuntimeError` results from mixing `cuda:0` and `cuda:1` tensors, preventing silent cross-device transfers.

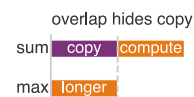
The movement mechanism is explicit. Calling `.to()` on a tensor returns the original tensor when its data type and device already match the request; otherwise it returns a converted copy unless `copy=True` forces a copy. In contrast, `nn.Module.to()` modifies registered parameters and buffers in place and returns the module.

The performance discipline follows directly from this bandwidth gap: allocate tensors on the target device from the start rather than creating them on the host and transferring them, reuse GPU memory across iterations rather than reallocating it, and colocate inputs, labels, and model parameters on the same device to eliminate implicit transfers. At 32 GB/s, violating any of these principles inserts PCIe transfers into the critical path that can dominate a training iteration that otherwise runs at 2.04 TB/s on-device.

Overlapping computation and communication When transfers are unavoidable, the next optimization is to hide their latency by executing them concurrently with computation. Modern GPUs contain independent hardware units for computation (SM clusters) and data transfer (copy engines), enabling true simultaneous execution. The framework abstraction that exposes this hardware parallelism is the CUDA stream: an independent execution queue where operations execute sequentially within a stream but concurrently across streams.

Without explicit concurrency control, the GPU serializes all operations on a single default stream, leaving execution units idle while data transfers complete. By placing data transfers on one stream and computation on another, the effective latency approaches the theoretical minimum of $\max(\text{compute_time}, \text{transfer_time})$ rather than their sum. Stream-based overlap effectively hides the D_{vol}/BW penalty when computation is the longer operation (see listing 7.19).

²⁴ | **NVLink:** NVIDIA’s high-bandwidth GPU-to-GPU interconnect (see chapter 11), providing 600 GB/s bidirectional bandwidth (NVLink 3.0 on A100) compared to 64 GB/s for PCIe 4.0 x16. This $\sim 10 \times$ bandwidth advantage determines whether tensor parallelism, splitting one large tensor computation across multiple GPUs, is practical for a given model size: GPUs connected only by PCIe can make the D_{vol}/BW communication term dominate total training time, erasing the benefit of additional compute.



Overlapping copy and compute costs $\max(\text{copy}, \text{compute})$, not their sum.

The `non_blocking=True` flag requests an asynchronous copy with respect to the host when the backend supports it. For host-to-device copies, pinned memory is generally required to overlap the transfer with other device work; pageable memory may be staged internally, and the call's host-blocking behavior is implementation-dependent. Correct overlap also requires compatible hardware, separate execution resources, and explicit dependency management.

The same synchronization pattern appears when model stages overlap across microbatches. Listing 7.20 shows each stage running on its own stream, with events enforcing only the producer-consumer dependencies needed for correctness.

Listing 7.19: Overlapping Computation and Transfer: Separate streams can overlap eligible copies and computation when dependencies are synchronized correctly. Pinned host memory generally enables host-to-device copies to overlap with other device work.

```
compute_stream = torch.cuda.Stream()
transfer_stream = torch.cuda.Stream()

# Transfer next batch while computing
# current batch
with torch.cuda.stream(transfer_stream):
    next_batch = next_batch_cpu.to("cuda", non_blocking=True)

with torch.cuda.stream(compute_stream):
    output = model(current_batch)
    loss = criterion(output, labels)

# Pinned host memory can enable transfer overlap
x_pinned = torch.randn(1000, 1000).pin_memory()
x_gpu = x_pinned.to("cuda", non_blocking=True) # Asynchronous

# A pageable source may require staging and
# should not be assumed to overlap
y_regular = torch.randn(1000, 1000)
y_gpu = y_regular.to("cuda", non_blocking=True)
```

Listing 7.20: Stage Overlap with Streams: Overlap multiple model stages across microbatches using streams and events for inter-stage synchronization.

```
# Pipeline parallelism: place stages on separate GPUs
devices = [torch.device(f"cuda:{i}") for i in range(3)]
stages = [
    Stage1().to(devices[0]),
    Stage2().to(devices[1]),
    Stage3().to(devices[2]),
]
streams = [torch.cuda.Stream(device=device) for device in devices]
events = [
    torch.cuda.Event() for _ in range(num_microbatches)
    for _ in stages
]
outputs = [[None] * num_microbatches for _ in stages]

for mb in range(num_microbatches):
    for stage_idx, (stage, stream, device) in enumerate(
        zip(stages, streams, devices)
    ):
        with torch.cuda.device(device), torch.cuda.stream(stream):
            if stage_idx > 0:
                # Wait for previous stage to complete
                # this microbatch
                events[stage_idx - 1][mb].wait()
                stage_input = outputs[stage_idx - 1][mb].to(
                    device, non_blocking=True
                )
            else:
                stage_input = inputs[mb].to(device, non_blocking=True)

            outputs[stage_idx][mb] = stage(stage_input)
            events[stage_idx][mb].record()
```

This overlap principle extends naturally to model-stage overlap within a single node. Different model stages on separate GPUs can process different microbatches concurrently, with each stage's computation overlapping the next stage's data reception (see listing 7.20). Chapter 8 later names and analyzes the distributed-training strategies built from this scheduling pattern; here, the single-node implementation is enough to expose the synchronization principle that survives at larger scale. Once computation and communication overlap, the remaining challenge is ensuring correctness when operations complete out of order.

Synchronization and correctness Concurrent execution introduces ordering constraints. When one stream's output becomes another stream's input, the system must enforce a happens-before relationship without unnecessarily serializing independent work. Two synchronization mechanisms exist, with dramatically different performance implications.

Full device synchronization (`torch.cuda.synchronize()`) blocks all streams and the CPU until every queued operation completes. This creates a global serialization point that eliminates all overlap benefits. CUDA events provide the alternative: fine-grained synchronization that blocks only the dependent stream, allowing other streams and the CPU to continue execution (see listing 7.21).

Listing 7.21: CUDA Events for Synchronization: Events enable fine-grained producer-consumer patterns between streams without blocking the entire device.

```
# Create streams and event
stream1 = torch.cuda.Stream()
stream2 = torch.cuda.Stream()
event = torch.cuda.Event()

# Stream 1: producer
with torch.cuda.stream(stream1):
    result1 = expensive_computation(data1)
    event.record() # Mark completion point

# Stream 2: consumer (waits only for stream1's event)
with torch.cuda.stream(stream2):
    event.wait() # Block stream2 until event is recorded
    result2 = dependent_computation(result1) # Safe to use result1
```

The performance difference between these approaches is categorical. Full synchronization after every operation converts a concurrent pipeline into a sequential one, entirely negating the hardware parallelism that streams expose. Event-based synchronization preserves the concurrent execution model while enforcing only the dependencies that correctness requires.

This synchronization discipline has one operational trap: debug code often leaves `torch.cuda.synchronize()` in the hot path, turning an overlapped pipeline into a serialized one. When overlap remains poor, profiling must separate scheduling stalls from kernel inefficiency. NVIDIA Nsight Systems (`nsys profile`) shows CPU activity, GPU kernels, and memory transfers on one timeline. NVIDIA Nsight Compute (`ncu`) then explains kernel behavior with hardware counters.

Table 7.8 is the diagnostic map for that second step. SM means streaming multiprocessor, the GPU block that schedules groups of threads; a warp is one scheduled thread group.

Table 7.8: Nsight Compute Metrics: Key metrics for diagnosing ML kernels. Interpret each value against workload eligibility and kernel intent; Nsight Systems identifies which kernels dominate runtime, and Nsight Compute reveals why those kernels underperform.

Metric	Meaning	Optimization Target
SM Occupancy	Active warps/maximum warps	Increase parallelism if low
Memory Throughput	Achieved/peak bandwidth	Optimize memory access patterns
Compute Throughput	Achieved/peak FLOP/s	Reduce memory bottlenecks
Tensor Core Active	Time in Tensor Core ops	Verify mixed-precision utilization

7.5.1.4 Data pipelines and loading

Streams and events address placement and movement by overlapping transfers with computation so that the GPU rarely stalls on a single tensor. Scheduling alone, however, cannot help if data arrives too slowly in the first place. The next constraint is input delivery: data must arrive fast enough to sustain throughput. The core systems principle is straightforward: the data pipeline must sustain the accelerator’s consumption rate. A GPU processing 1,000 images per second at 224 by 224 resolution requires approximately 150.5 MB/s of sustained raw uint8 image throughput. If the pipeline cannot maintain this rate, the accelerator idles and the effective utilization term in the iron law drops below 1.

Frameworks address this throughput requirement through three mechanisms. The first is *parallel worker processes*: the DataLoader spawns multiple CPU processes, each independently loading and preprocessing samples. When storage access, decoding, augmentation, or normalization makes one loader process too slow to feed the accelerator, multiple workers can overlap I/O wait with preprocessing. Once the input pipeline meets demand, extra workers add overhead rather than throughput. When `num_workers > 0`, the DataLoader distributes sample indices across workers through a shared queue, and workers push completed samples to a data queue that the main process assembles into batches.

- workers
- prefetch
- pinned

Each DataLoader knob relieves a specific input bottleneck.

The second mechanism is prefetching. With multiprocessing enabled, `prefetch_factor` controls how many batches each worker prepares in advance. Four workers with `prefetch_factor=2` can maintain up to eight prefetched batches, increasing the chance that input work overlaps accelerator computation. Prefetching cannot guarantee that the accelerator never stalls; storage, decoding, augmentation, and contention can still make production slower than consumption. The cost is additional host memory proportional to the queued data.

The third mechanism is *pinned memory for DMA transfers*. The `pin_memory=True` option places batches in page-locked host memory, which can avoid pageable-memory staging and enable host-to-device copies to overlap device work. For a batch of 64 FP32 images at $224 \times 224 \times 3$ (38.5 MB), dividing by the stated PCIe 4.0 $\times 16$ peak gives 1.2 ms; this is a bandwidth-only lower bound, not a measured pinned-transfer latency. The realized benefit over pageable memory depends on the platform, batch size, pipeline, and overlap. Pinned pages also reduce pageable system memory.

The DataLoader configuration is useful only when each parameter is tied to a bottleneck. In this configuration, `num_workers` enables parallel loading, `prefetch_factor` controls pipeline depth, and `pin_memory` enables DMA transfers. The worker count is a throughput/memory trade-off, not a universal constant. A practical starting point is setting `num_workers` equal to the number of available CPU cores, then adjusting based on whether loading is I/O-bound or CPU-bound. For I/O-bound workloads such as reading images from network storage, more workers overlap disk latency and improve throughput. For CPU-bound workloads involving heavy augmentation, the benefit saturates once all cores are in use. Too many workers waste memory, since each maintains a copy of the Dataset object.

Once throughput is high enough, worker process management becomes a correctness constraint. PyTorch assigns each worker a distinct PyTorch seed. A `worker_init_fn` is still useful when dataset or augmentation code also uses NumPy or Python's `random` module, as listing 7.22 demonstrates. Shared Python state is generally process-local, so modifications in one worker do not automatically propagate to the others or the main process; explicitly shared or memory-mapped storage is needed when workers must share mutable state.

The Dataset choice is another throughput decision because it determines how samples can be scheduled. PyTorch supports two dataset paradigms. *Map-style* datasets implement `__len__` and `__getitem__`, enabling random access to samples by index—this pattern works well for datasets that fit in memory or support efficient random access on disk. *Iterable-style* datasets implement `__iter__` instead, yielding samples sequentially for streaming data sources where random access is impractical. Map-style datasets support sampler-based shuffling, while iterable datasets must implement any ordering or buffered shuffling within the iterator or data source.

Listing 7.22: DataLoader Throughput Configuration: Each parameter addresses a specific throughput bottleneck. `num_workers` parallelizes I/O and preprocessing across CPU cores, `prefetch_factor` controls pipeline depth, and `pin_memory` enables DMA transfers to the GPU.

```
import random

import numpy as np
import torch
from torch.utils.data import DataLoader

def seed_worker(worker_id):
    worker_seed = torch.initial_seed() % 2**32
    np.random.seed(worker_seed)
    random.seed(worker_seed)

loader = DataLoader(
    dataset,
    batch_size=64,
    shuffle=True,
    num_workers=4, # Parallel worker processes (mechanism 1)
    prefetch_factor=2, # Batches prepared ahead per worker (mechanism 2)
    pin_memory=True, # Page-locked memory for DMA (mechanism 3)
    worker_init_fn=seed_worker, # Reproducible augmentation per worker
)

# Pipeline effect: while GPU processes batch N,
# 4 workers load batches N+1..N+8 into pinned memory,
# ready for DMA transfer when the GPU finishes.
```

Batch collation represents the final stage where representation choices govern throughput. The `collate_fn` parameter determines how individual samples are combined into batches. The default collation stacks tensors along a new batch dimension, which works when all samples have identical shapes. For variable-length data such as text sequences, custom collation handles padding, sorting by length, or creating attention masks—directly affecting both memory usage and training throughput.

DataLoaders, Datasets, and collation functions solve input delivery by sustaining accelerator-rate throughput through parallelism, prefetching, and DMA. These structures, however, handle only ephemeral data: samples flow through the pipeline once per epoch and are discarded. The next framework responsibility is persistent state, especially the model’s own weights when those weights exceed the memory of any single device.

Parameter structures

A GPT-3 scale model stores 175B parameters, occupying 350 GB in FP16. Managing these parameters across devices, keeping gradients synchronized, and maintaining optimizer state (Adam state alone can add about 4× the FP16 weight memory, as the Administrative Tax notebook showed) is a core framework responsibility. Because parameters persist throughout training and inference, frameworks organize them into compact structures that minimize memory while enabling fast read and write access. During multi-GPU training, frameworks may replicate parameters across devices for parallel computation while keeping a synchronized master copy; parameter-server systems are one communication-efficient design for workers to read and write globally shared parameters (Li et al. 2014). Synchronizing multi-billion parameter models can require transferring tens of GB of gradients per step, which is why frameworks expose communication backends that can synchronize tensors efficiently. Chapter 8 later names the specific collective operations and scaling strategies.

Parameter structures must also adapt to varying precision requirements. Training typically uses FP32 for gradient stability, but inference and large-scale training increasingly use FP16 or INT8. Frameworks implement type casting and mixed-precision management to enable these optimizations without compromising numerical accuracy.

Distributed execution contexts

The computational graph defines *what* to compute, but *where* and *how* that computation runs across devices is the job of execution contexts. On a single node, execution contexts manage CUDA streams and events (introduced in section 7.5.1.3) to overlap computation and data transfer across GPUs.

When training scales beyond a single machine, these same abstractions extend to named groups of devices. Frameworks use constructs such as `ProcessGroup` (PyTorch) or `Mesh` (JAX) to describe which tensors and operations belong together, relying on communication libraries such as NCCL to execute collective primitives—such as **ring-AllReduce** for gradient aggregation and **AllGather** for parameter gathering. The runtime manages these primitives to maximize interconnect utilization: intra-node NVLink links deliver up to 900 GB/s of bidirectional bandwidth per GPU, whereas inter-node InfiniBand or Ethernet networks drop to 50–400 Gbps (6.25–50 GB/s), making collective communication efficiency the primary bottleneck in distributed training scaling. The important framework idea is the abstraction boundary: user code names placement relationships, and the framework preserves those relationships while the hardware path changes underneath.

These concepts appear here because they shape framework API design even before the book asks the reader to reason about distributed-training algorithms. The details of gradient synchronization, communication topologies, and fault tolerance build on these foundations later. For now, the only point needed is placement expressiveness: when models exceed single-device memory, frameworks must give the training system more than one way to place work. A GPT-3 scale model, for instance, cannot fit on a single GPU—its 175B parameters alone require 350 GB in FP16, far exceeding any GPU’s memory. Figure 7.11 previews the framework placement idea without requiring the full distributed-training machinery yet: a system can split work across layer groups, across replicated batches, or within very large tensors. Section 8.7 later formalizes these dimensions as training strategies and analyzes their communication costs.

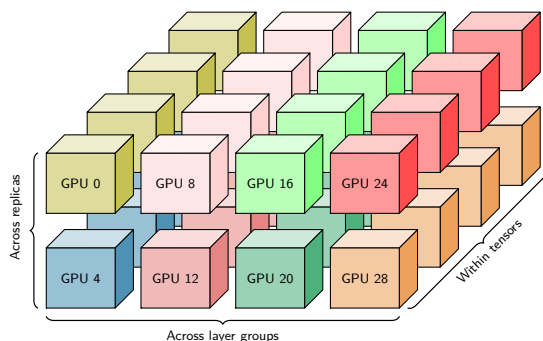


Figure 7.11: Device Placement Dimensions: A grid of eight accelerator clusters arranged in two rows and four columns, each containing stacked computational units. The braces preview three ways a framework can place work when a model no longer fits one device: across layer groups, across replicated batches, and within very large tensors.

The data structures examined so far—tensors, device managers, data pipelines, parameter structures, and distributed execution contexts—define *what* data a framework manages and *where* it lives. The remaining abstraction problem is execution: what actually runs on the hardware.

7.5.2 Core operations

When an engineer writes `y = torch.matmul(x, w)`, the distance between Python syntax and silicon execution is substantial. Bridging a single high-level call to thousands of parallel GPU threads requires three tiers of operations working in coordination. Figure 7.12 illustrates this stack: hardware abstraction operations manage platform-specific kernel dispatch and memory transfers, basic numerical operations execute mathematical routines, and system-level operations coordinate scheduling, memory lifetimes, and accelerator resources across the computation graph. The discussion that follows examines this build-up from the hardware-specific kernel path toward system orchestration.

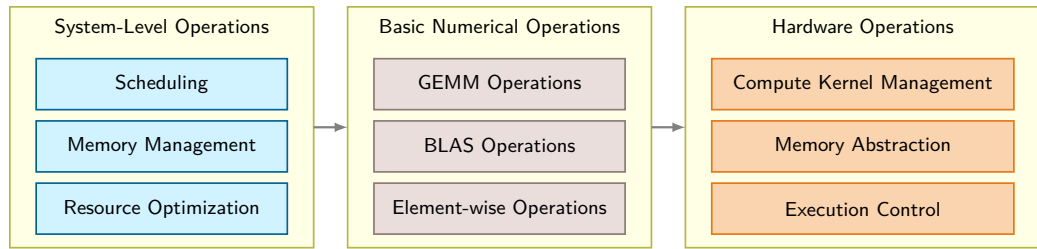


Figure 7.12: Core Operations Stack: Three functional tiers bridge high-level Python code to physical execution. Hardware operations manage low-level kernel dispatch and memory transfers, basic numerical operations execute dense GEMM and element-wise routines, and system-level operations coordinate graph scheduling and memory lifetimes across accelerators.

7.5.2.1 Hardware abstraction operations

The hardware abstraction layer isolates framework code from platform-specific details. It solves three concrete problems: selecting the right compute kernel, moving data through the memory hierarchy, and coordinating execution across processing units.

Compute kernel management The kernel manager dispatches each supported operation to an implementation for the current backend. For matrix multiplication, this may be a CPU BLAS backend such as Intel oneMKL or OpenBLAS (Intel Corporation 2026a; OpenBLAS Project 2026), cuBLAS on NVIDIA GPUs (NVIDIA 2024a), or accelerator-specific tensor instructions. Selection depends on dimensions, dtype, layout, backend, and library heuristics. A 4096×4096 FP16 GEMM on an A100 may use a Tensor Core path whose published peak is represented here by 312 TFLOP/s (Choquette et al. 2021; NVIDIA Corporation 2020a). If the backend has no implementation for an operator, execution may fall back, be partitioned, or fail, depending on the framework and runtime.

Memory system abstraction The memory abstraction layer moves tensors among pageable or pinned host memory, device memory, and unified memory, and transforms data layouts to match hardware preferences. A convolutional layer, for example, may store activations in NCHW format (batch, channels, height, width) on NVIDIA GPUs but convert to NHWC for Apple’s Metal backend. Alignment requirements vary from 4 bytes on CPUs to 128 bytes on some accelerators, and misaligned access can halve effective memory bandwidth. The runtime also preserves dependency ordering when multiple execution units access the same tensor, preventing races during concurrent operations.

Execution control The execution controller coordinates work across multiple processing units and memory spaces. On a modern GPU, graph runtimes or explicit stream use can overlap independent kernels when dependency analysis proves they are ready and the kernels leave enough resources unused to run concurrently. Eager default-stream execution often serializes this work instead. The controller inserts synchronization barriers where data dependencies require them, tracks event completions to trigger dependent operations, and routes hardware errors (ECC failures, timeout watchdogs) to the framework’s error handling path.

7.5.2.2 Basic numerical operations

With hardware abstraction managing the platform-specific details, frameworks build a layer of mathematical operations on top. GEMM dominates ML computation. Section C.1.4 derives how GEMM arithmetic intensity scales with matrix dimension, predicting whether a given layer is compute bound or memory bound before any profiler runs. The operation $\mathbf{C} = \alpha\mathbf{A}\mathbf{W} + \beta\mathbf{C}$ accounts for the vast majority of arithmetic in neural networks: a single ResNet-50 forward pass performs approximately 8.2 GFLOP, nearly all of which reduce to GEMM. Frameworks optimize GEMM through cache-aware **tiling** (splitting matrices into blocks that fit in L1/L2 cache), loop unrolling for instruction-level parallelism, and shape-specific kernels. Fully connected layers use standard dense GEMM, while convolutional layers use im2col transformations that reshape input patches into matrix columns, converting convolution into GEMM.

Beyond GEMM, frameworks implement BLAS operations (AXPY for vector addition, GEMV for matrix-vector products) and element-wise operations (activation functions, normalization). Element-wise operations are *individually cheap* but *collectively expensive* due to memory bandwidth. Each

operation reads and writes the full tensor, so a sequence of five element-wise operations on a 100 MB tensor moves 1 GB of data. Fusing those five operations into a single kernel reduces memory traffic to 200 MB, a $5\times$ bandwidth savings that directly translates to faster execution.

Numerical precision adds another dimension. Training in FP32 uses 4 bytes per parameter; quantizing to INT8 reduces this to 1 byte, cutting memory by $4\times$ and enabling $2\text{--}4\times$ throughput improvements on hardware with INT8 acceleration. Training typically keeps numerically sensitive accumulations in higher precision, while inference can often run many operations in FP16 or INT8 with little quality loss. Frameworks maintain separate kernel implementations for each precision format and handle workflows where different layers operate at different bit widths within a single forward pass.

7.5.2.3 System-level operations

Hardware abstraction and numerical operations provide the building blocks; system-level operations orchestrate them. The system layer ties scheduling, memory management, and resource optimization into a coherent execution engine.

The operation scheduler uses graph dependencies to identify legal execution orders and possible concurrency. Static capture can expose more of the dependency structure before execution, while eager runtimes discover work incrementally. Neither visibility guarantees an optimal schedule or simultaneous execution: overlap depends on streams, compiler and runtime decisions, resource availability, and the operations themselves.

The memory manager allocates and reclaims GPU memory across the computational graph's lifetime. Model parameters (a 7-billion-parameter model consumes approximately 14 GB in FP16) persist for the entire training run, while activation tensors live only until the backward pass consumes them. PyTorch's caching allocator maintains a memory pool, subdividing and reusing freed blocks without returning them to CUDA, which avoids repeated `cudaMalloc` calls that can cost tens of microseconds and may be worse when they synchronize the device. For models that exceed GPU memory, the manager can apply checkpointing by discarding selected activations during the forward pass and recomputing them during the backward pass. The policy question—which activations to keep, and how much recomputation to tolerate—belongs to the training pipeline; the framework abstraction is what makes that policy executable.

Checkpoint 7.3: Hardware abstraction

The abstraction problem is the bridge between *portable code* and *efficient execution*.

- ☐ **Two dimensions:** can you distinguish data representation (layout, dtype, placement) from execution mapping (kernel selection, scheduling), and explain how they constrain each other?
- ☐ **Kernel dispatch:** can you explain why the same high-level operation (for example, GEMM) needs multiple implementations (CPU vector path vs. GPU Tensor Core path) and how shapes/dtypes affect the choice?
- ☐ **Memory abstraction:** can you explain why frameworks use caching allocators and layout transforms (NCHW $\leftrightarrow$ NHWC) rather than calling device allocators on every tensor?
- ☐ **Execution control:** can you describe what the runtime is doing when it overlaps independent work (streams) and inserts synchronization only where dependencies require it?

The resource optimizer integrates scheduling and memory decisions with backend kernel selection. Matrix-multiplication libraries choose among tiled GEMM implementations according to shapes, dtype, layout, workspace, and hardware. Winograd is instead a convolution-specific transform, and Strassen is not a routine alternative selected by mainstream ML GEMM dispatch. A poor schedule can leave resources idle, while memory-pool fragmentation or excessive live state can cause an out-of-memory error even when aggregate device capacity appears sufficient.

Tensors, streams, and kernel dispatch operate at the granularity of individual buffers and operations. While these primitives secure high execution efficiency on hardware, scaling model

development requires an organizational abstraction: composing thousands of low-level operations into reusable, nestable building blocks called *modules*.

7.6 nn.Module Abstraction

Hardware-facing mechanisms—tensors, kernels, streams, and memory managers—ensure individual operations execute efficiently on silicon. Yet a network such as ResNet-50, containing 25.6M parameters across fifty layers, requires extensive coordination: registering weights with optimizers, tracking device placement, toggling training and evaluation modes, and serializing checkpoints. Managing this manually would drown model development in error-prone tensor plumbing. The organizational layer of the framework resolves this through hierarchical composition.

Every major framework implements this through a module abstraction that binds parameter tensors, forward computation, and lifecycle hooks into a unified interface. PyTorch’s `nn.Module`²⁵ provides an instructive case study because its design patterns recur across frameworks: Keras uses similar layer abstractions (Chollet 2018), JAX’s Flax employs analogous module structures, and TensorFlow’s functional API shares conceptual parallels. Three enduring design principles recur regardless of syntax or programming paradigm.

²⁵ | **nn.Module:** The “design patterns recur” claim holds because `nn.Module` solves a universal organizational problem: it automatically registers any assigned submodule or parameter into a hierarchical tree, enabling a single `.to('cuda')` call to recursively place millions of parameters onto a GPU. Keras layers, JAX Flax modules, and TensorFlow’s `tf.Module` all implement the same tree-walking pattern. Without it, managing model state would require manual bookkeeping that scales linearly with architectural depth, a cost that grows prohibitive for models with hundreds of layers.

7.6.1 Automatic parameter discovery

A modern neural network may contain millions of trainable parameters spread across dozens of layers. Without automation, a programmer would need to enumerate every parameter tensor and pass it to the optimizer manually, an error-prone process that scales poorly with model complexity. Frameworks solve this through **automatic parameter discovery**: the system walks the module tree, collecting every parameter tensor so the optimizer can update them in a single call.

This is a tree traversal problem at its core. PyTorch’s `Module.__setattr__` registers an assigned `nn.Parameter` or child `nn.Module` in the module’s internal mappings. A call to `.parameters()` recursively traverses registered submodules and yields registered parameters. Other frameworks expose analogous parameter trees or collections, though their mechanisms differ.

The systems consequence is significant. Automatic parameter discovery gives the optimizer a complete parameter set or parameter groups, enabling grouped, foreach, or fused update paths when the framework and backend support them. The immediate win is correctness and composability: newly attached submodules participate in device moves, serialization, and optimizer construction without a separate manual registry. It also lets optimized update paths operate over parameter collections without per-parameter user code. Listing 7.23 demonstrates the core mechanism: attribute assignment triggers registration, and `.parameters()` returns all discovered tensors.

The distinction between *parameters* and *buffers* illustrates a subtlety of discovery. Registered parameters appear in `.parameters()` whether trainable or frozen; `requires_grad` separately controls whether autograd accumulates a gradient for them. Registered buffers move with the module and can appear in its state dictionary but are excluded from `.parameters()` and ordinary optimizer discovery. Batch-normalization running statistics are a common buffer use case.

Listing 7.23: Parameter Registration: Automatic parameter tracking through attribute assignment enables optimizer access to all trainable weights without manual enumeration.

```
import torch
import torch.nn as nn

class CustomLayer(nn.Module):
    def __init__(self, input_size, output_size):
        super().__init__()
        self.weight = nn.Parameter(
            torch.randn(output_size, input_size)
        )
        self.bias = nn.Parameter(torch.randn(output_size))
        self.register_buffer("running_mean", torch.zeros(output_size))

    def forward(self, x):
        return torch.matmul(x, self.weight.t()) + self.bias

layer = CustomLayer(10, 20)
# Framework discovers both parameters automatically:
for name, param in layer.named_parameters():
    print(f"{name}: shape {param.shape}")
```

Table 7.9 shows how the same principle manifests differently across frameworks. Despite syntactic differences, all frameworks solve the same problem: enabling optimizers to discover and update trainable parameters while preserving nontrainable state across forward passes.

Table 7.9: Parameter Discovery APIs Across Frameworks: Each framework exposes a different surface for distinguishing trainable parameters from nontrainable buffers, but all four solve the same underlying problem of letting optimizers find weights to update while preserving state (running statistics, frozen tensors) across forward passes. Across all four rows, the durable pattern consists of a trainable-parameter accessor for the optimizer and a separate nontrainable-state channel for values that must persist but must not receive gradients.

Framework	Parameter Access	Nontrainable State
PyTorch	model.parameters()	register_buffer()
Keras	layer.trainable_weights	layer.non_trainable_weights
JAX/Flax	variables["params"] after variables = model.init(key, x)	Separate variable collections (for example, batch_stats)
TensorFlow	module.trainable_variables	module.non_trainable_variables

Mode-dependent behavior

Training and inference require different computational behavior from the same model graph. During training, dropout layers randomly zero elements with probability $p_{\text{drop}} = \text{Pr}(\text{drop})$ to regularize the network, while during inference those same layers must perform identity mapping to produce deterministic outputs. Batch normalization uses per-batch statistics during training but switches to accumulated running statistics during inference. If these behavioral changes are left to the programmer, forgetting a single mode switch produces silently incorrect predictions in production. Frameworks solve this with a *state flag* that propagates through the module hierarchy. A single call to `.eval()` on the root module recursively sets `self.training = False` on every descendant, and each layer queries this flag to select its behavior. This is an instance of a broader systems principle: the same computation graph must produce different execution behavior depending on context. Compilers face the same challenge when the same source code must produce debug builds (with bounds checking and symbol tables) vs. release builds (with aggressive optimization). The flag-propagation pattern ensures correctness by centralizing the mode decision at the root rather than requiring per-layer coordination.

This principle extends to parameter freezing for transfer learning. Setting `requires_grad=False` prevents autograd from accumulating gradients in those parameters and removes their gradient storage. It does not necessarily remove the layer’s backward computation because gradients may still need to pass through its operations to reach trainable inputs or earlier parameters. Savings therefore depend on where the frozen region lies and which tensors require gradients.

7.6.2 Hierarchical composition and serialization

Complex models compose from reusable submodules, creating a tree structure. A ResNet is not implemented as a monolithic block of operations but as a hierarchy: the root module contains a sequence of residual blocks, each block contains convolution layers and normalization layers, and each layer contains parameter tensors. This hierarchical composition must support two critical operations made concrete in listing 7.24: recursive parameter collection for training and **state serialization** for checkpointing and deployment.

Listing 7.24: Nested Module Composition: Hierarchical module composition enables recursive parameter collection and a flat state mapping across the module tree.

```
import torch
import torch.nn as nn

class ResidualBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.conv1 = nn.Conv2d(channels, channels, 3, padding=1)
        self.bn1 = nn.BatchNorm2d(channels)
        self.conv2 = nn.Conv2d(channels, channels, 3, padding=1)
        self.bn2 = nn.BatchNorm2d(channels)

    def forward(self, x):
        residual = x
        x = torch.relu(self.bn1(self.conv1(x)))
        x = self.bn2(self.conv2(x))
        return torch.relu(x + residual)

class ResNet(nn.Module):
    def __init__(self, num_blocks, channels=64):
        super().__init__()
        self.conv_in = nn.Conv2d(3, channels, 7, padding=3)
        self.blocks = nn.ModuleList(
            [ResidualBlock(channels) for _ in range(num_blocks)]
        )
        self.fc = nn.Linear(channels, 10)

    def forward(self, x):
        x = self.conv_in(x)
        for block in self.blocks:
            x = block(x)
        x = x.mean(dim=[2, 3]) # Global average pooling
        return self.fc(x)

model = ResNet(num_blocks=4)
total = sum(p.numel() for p in model.parameters())
print(f"Total parameters: {total}")
# state_dict() flattens the tree: 'blocks.0.conv1.weight', etc.
print(list(model.state_dict().keys())[:4])
```

Hierarchical composition mirrors the hardware memory hierarchy in a systems-relevant way: each submodule’s parameters can be loaded independently, enabling placement across devices. When

a model is too large for a single GPU, the framework can assign different subtrees of the module hierarchy to different devices, with the tree structure providing natural partition boundaries.

The `state_dict()` method produces an ordered mapping whose dotted keys (for example, `blocks.0.conv1.weight`) encode locations in the module hierarchy. A checkpoint for 7 billion FP16 parameters contains approximately 14 GB of weight payload before metadata or additional state, but the file format and storage path determine how those bytes are serialized. `load_state_dict()` copies matching parameter and buffer values into an already-constructed module and reports missing or unexpected keys according to its strictness setting; it does not reconstruct the module hierarchy. Cross-framework exchange requires a compatible graph and operator representation in addition to weights.

The hierarchical structure also enables module-level traversal for systematic operations. Methods such as `.named_modules()` iterate the entire tree, supporting bulk transformations such as replacing all BatchNorm layers with GroupNorm or applying Xavier initialization to every Linear layer. These traversal operations depend on the same tree structure that enables parameter discovery, illustrating how a single design decision propagates benefits across multiple use cases.

These three principles, automatic parameter discovery, mode-dependent behavior, and hierarchical composition with serialization, are not PyTorch-specific. Every framework must solve them. Keras layers, JAX’s Flax modules, and even functional approaches all address the same problems of parameter management, state tracking, and compositional design. The differences lie not in *what* problems they solve but in *how* they prioritize among competing solutions. Two practical patterns show how the principles become system controls: selective parameter freezing reduces unnecessary gradient work for transfer learning (listing 7.25), and module hooks provide noninvasive inspection (listing 7.26).

Listing 7.25: Parameter Freezing: Demonstrates selective parameter freezing for transfer learning, where pretrained layers remain fixed while new layers train.

```
from torchvision.models import ResNet18_Weights, resnet18

# Freeze all parameters in a pretrained
# model
pretrained_model = resnet18(weights=ResNet18_Weights.DEFAULT)

for param in pretrained_model.parameters():
    param.requires_grad = False

# Replace final layer with trainable parameters
pretrained_model.fc = nn.Linear(512, 10) # New layer is trainable

# Only fc.parameters() will receive
# gradients during training
optimizer = torch.optim.Adam(
    filter(lambda p: p.requires_grad, pretrained_model.parameters()),
    lr=0.001,
)
```

Module hooks are the inspection counterpart to parameter freezing: they intercept intermediate computations without modifying model code, enabling gradient flow diagnosis and activation monitoring. Listing 7.26 illustrates both hook types.

Together, these patterns—parameter discovery, freezing, and hooks—demonstrate how the three principles translate into practical APIs. These `nn.Module` patterns illustrate PyTorch’s approach to the abstraction problem. PyTorch, however, is only one of several major frameworks, and its choices (mutable state, class inheritance, eager execution by default) are not the only valid design points. TensorFlow centralizes state differently, and JAX avoids mutable state entirely. These are not superficial API differences; they reflect deeply different answers to the chapter’s three opening problems.

Listing 7.26: Module Hooks: Shows forward and backward hooks for inspecting activations and gradients during training.

```

import torch
import torch.nn as nn

model = nn.Sequential(nn.Linear(10, 20), nn.ReLU(), nn.Linear(20, 5))

# Forward hook to inspect activations
def forward_hook(module, input, output):
    print(
        f"Layer: {module.__class__.__name__}, "
        f"Output shape: {output.shape}, "
        f"mean={output.mean():.3f}, "
        f"std={output.std():.3f}"
    )

# Backward hook to inspect gradients
def backward_hook(module, grad_input, grad_output):
    print(f"Gradient norm: {grad_output[0].norm():.3f}")

# Register hooks on specific layer
handle_fwd = model[0].register_forward_hook(forward_hook)
handle_bwd = model[0].register_full_backward_hook(backward_hook)

# Execute forward and backward pass
x = torch.randn(32, 10)
y = model(x)
loss = y.sum()
loss.backward()

# Remove hooks when done
handle_fwd.remove()
handle_bwd.remove()

```

7.7 Framework Platform Analysis

A team that prototypes quickly but cannot deploy the resulting model, or deploys reliably but cannot debug training failures, has run into a framework design trade-off rather than a missing API call. Each major framework solves all three core problems but emphasizes a different path through them. TensorFlow couples eager development to graph capture and a broad deployment ecosystem; PyTorch begins with eager execution and adds capture, compilation, and export paths; JAX organizes differentiation, vectorization, and compilation as composable function transformations. These are architectural emphases, not exclusive capabilities, and their value depends on the workload and deployment target.

7.7.1 TensorFlow: Eager development with graph deployment

TensorFlow's architecture spans eager development and graph-based optimization across hardware from cloud TPUs to microcontrollers. TensorFlow 2 executes eagerly by default. Performance-sensitive functions can be captured with `tf.function`, and deployment formats route captured computation to server, mobile, and browser runtimes. This dual path preserves the production graph capabilities inherited from TensorFlow 1.x without requiring every TensorFlow 2 program to build a complete graph before it runs.

Within a captured `tf.function`, TensorFlow can apply graph optimizations such as constant folding, operator fusion, and layout planning. Figure 7.13 maps the broader training-to-deployment pipeline from data preprocessing and distributed training to SavedModel export, TensorFlow Serving, TensorFlow Lite, TensorFlow.js, and language bindings.

While TensorFlow 2.0 introduced eager execution to bridge the gap between research and production, TensorFlow 2.x still exposes `tf.function` as the graph-conversion path for performance-

● TensorFlow

● PyTorch

● JAX

Each framework optimizes for a different system bottleneck.

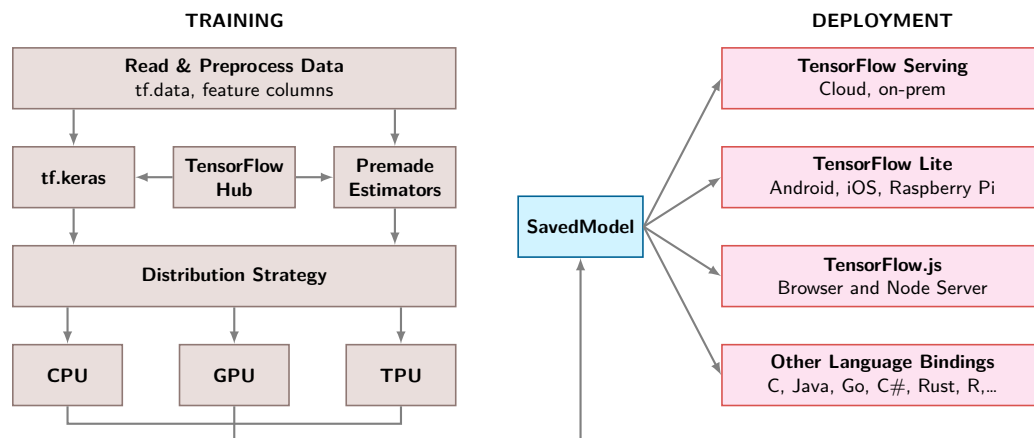


Figure 7.13: TensorFlow Training-to-Deployment Pipeline: The SavedModel format serves as the central decoupling boundary between training and deployment. It captures graph structure, weights, and signatures from heterogeneous training setups (left) and exports them directly to target-specific runtimes (right)—including TF Serving, TF Lite, and TF.js—without requiring Python in production.

sensitive code (TensorFlow Developers 2024a). Its core strength remains the robust, compiled path from research to global-scale deployment. Chapter 8 and chapter 13 later examine the scaling and production infrastructure that use these export paths.

7.7.2 PyTorch: The eager research standard

Where TensorFlow’s graph-first approach prioritizes *production optimization*, PyTorch makes the opposite trade-off: it prioritizes *developer experience*. PyTorch’s architecture represents a sharply different answer to the execution problem, built on dynamic graphs (or “Define-by-Run”). Instead of building a blueprint before execution, PyTorch builds the computational graph on-the-fly as the code runs. Facebook AI Research (FAIR) adopted this design because researchers need immediate feedback when experimenting with novel architectures; the define-then-run cycle of static graphs introduced a compilation delay that slowed the rapid prototyping essential to research workflows.

PyTorch’s approach fits exploratory research for the same reason: it treats deep learning as standard Python programming. Developers can use Python loops, conditionals, and debuggers (such as `pdb`) directly within a model’s forward pass, with no special syntax, no separate compilation step, and no waiting to see if the code works. Eager execution enables rapid iteration and intuitive model design, which is essential when architectures and training objectives are still changing.

PyTorch’s answer to the differentiation problem is tape-based autograd (section 7.4.2.1): flexible and debuggable, but harder to optimize globally because the tape is rebuilt each iteration. Its answer to the abstraction problem is more pragmatic than comprehensive: strong GPU support through cuBLAS and cuDNN, with deployment paths including `torch.export`, edge-targeted ExecuTorch, Open Neural Network Exchange (ONNX), and specialized runtimes.

The trade-off is therefore a more fragmented deployment path. Because the graph is dynamic, the framework cannot easily perform global optimizations before execution. A model that works perfectly in development may hit performance walls in production when dispatch overhead dominates small operations. To bridge this research-to-production gap, PyTorch introduced graph-capture and compilation paths, from TorchScript historically to `torch.compile` and export workflows, which allow developers to capture a dynamic model and turn it into an optimized representation for deployment. This evolution shows how an eager framework can move toward the production end of the compilation continuum while preserving the interactive experience that motivated the design.

7.7.3 JAX: The functional transformation engine

PyTorch’s eager execution and TensorFlow’s graph compilation represent two points on a spectrum, yet both share an imperative programming heritage where computation proceeds as a sequence of stateful operations. JAX represents a radically different user-facing approach, one built on functional

programming principles and composable program transformations rather than object-level tapes or user-authored graph APIs (Bradbury et al. 2018). Developed by Google Research, JAX is especially useful for work requiring custom differentiation, advanced optimization research, and large-scale distributed training.

JAX’s architecture treats differentiation, vectorization, and compilation as transformations on functions. The `jax.grad` function returns a function that computes gradients, which can then be transformed again when the interfaces are compatible. For example, `vmap(grad(f))` can compute per-example gradients and `jit(vmap(grad(f)))` can compile that batched computation. Transformation order is semantically meaningful rather than arbitrary.

JAX’s functional paradigm shifts the programming model from “tracking state through objects” to “transforming pure functions.” While PyTorch and TensorFlow expose autograd primarily through dynamic tapes or graph-compilation paths, JAX asks users to write pure Python functions and then applies transformations to those functions. Automatic differentiation, vectorization, and JIT compilation are all *program transformations* that can compose. Listing 7.27 demonstrates this approach.

Listing 7.27: JAX Function Transformations: JAX treats differentiation, vectorization, and compilation as composable function transformations rather than user-authored graph operations.

```
import jax
import jax.numpy as jnp

def loss_fn(params, x, y):
    pred = jnp.dot(x, params["w"]) + params["b"]
    return jnp.mean((pred - y) ** 2)

# Transform: compute gradients
grad_fn = jax.grad(loss_fn)

# Transform: vectorize over batch dimension
batched_grad = jax.vmap(grad_fn, in_axes=(None, 0, 0))

# Transform: compile to XLA
fast_batched_grad = jax.jit(batched_grad)

# Compose all three: fast, batched gradient computation
```

This functional approach requires **pure functions** (no side effects) and **immutable data** (arrays cannot be modified in place). These constraints may seem restrictive coming from PyTorch’s mutable object model, but they enable formal guarantees: the compiler can safely reorder, fuse, and parallelize operations because function outputs depend only on inputs. The restriction is the feature; purity is what makes transformation composition possible.

JAX’s power emerges from composition. `jax.grad` returns a gradient function; `jax.vmap` can vectorize a compatible function over mapped axes; and `jax.jit` can trace and compile a stable transformed function for a particular argument signature. Unlike an iterative Python loop that issues individual operations sequentially with repeated dispatch overhead, `jax.vmap` pushes the batch dimension directly into the underlying primitives—converting per-sample matrix-vector operations into batched matrix-matrix multiplications (GEMM) executed in a single parallel kernel launch. `jax.pmap` maps a function across devices, but synchronization or gradient aggregation must be expressed with collectives such as `lax.psum` or `lax.pmean`. These transformations compose according to their types and semantics, and different orders can compute different quantities.

The same minimalist core delegates neural network abstractions to companion libraries (Flax, Haiku, Equinox) and optimization to Optax. This separation reflects the functional philosophy: the core provides transformations, while libraries build conventional abstractions on top. The trade-off is that production readiness depends not only on the transformation model, but also on the maturity of the surrounding libraries, export paths, and operational tooling for the target environment.

The functional constraints that JAX imposes become advantages in specific domains. Custom differentiation—higher-order gradients, custom vector-Jacobian product (VJP) and Jacobian-vector product (JVP) rules—composes cleanly because pure functions make differentiation rules predictable. Research on optimization algorithms benefits from transformations that let researchers manipulate gradient computation as naturally as they manipulate data. Compilation-heavy accelerator workloads use XLA to extract more utilization when the program can be expressed in this functional style. Scientific computing with AD requirements benefits from functional purity that enables mathematical reasoning about code. JAX requires more upfront investment than PyTorch: the functional paradigm has a learning curve, state management requires explicit patterns, and debugging compiled code is harder than eager execution. Teams should choose JAX when its strengths align with project requirements, not as a default.

7.7.4 Framework trade-offs under measurement

The preceding sections described each framework’s design philosophy in qualitative terms: graph-first vs. eager-first, stateful vs. functional. The useful comparison is not which framework is fastest in the abstract, because that answer changes with model shape, batch size, hardware backend, and compiler configuration. The useful comparison is what each design lets the system see and optimize. Table 7.10 therefore maps TensorFlow, PyTorch, and JAX back to the three framework problems: execution visibility, differentiation model, and hardware abstraction path.

Table 7.10: Framework Design Trade-Offs: Each column reflects a distinct answer to the three core problems. TensorFlow makes deployment and graph capture central, PyTorch makes eager execution and debugging central, and JAX makes functional transformation and compiler visibility central. The table is a diagnostic map, not a benchmark ranking.

Aspect	TensorFlow	PyTorch	JAX
Graph Type	Static roots, dynamic front end in 2.x	Dynamic	Functional transformations
Programming Model	Imperative front end, graph capture path	Imperative	Functional
Core Data Structure	Tensor with framework-managed state	Tensor with framework-managed state	Immutable array
Execution Mode	Eager by default, graph for optimization	Eager by default	Trace and just-in-time compilation
Automatic Differentiation	Reverse mode over captured computation	Reverse mode over an eager tape	Forward and reverse transformations
Hardware Abstraction	Broad deployment runtimes and XLA paths	Native GPU path plus export/compile runtimes	XLA-centered accelerator compilation
Optimization Risk	Graph capture and operator coverage	Graph breaks after eager development	Purity, shape stability, and tracing

The measurement implication is straightforward: profile the constraint each framework makes most visible. In PyTorch, check whether eager dispatch or graph breaks dominate. In TensorFlow, check whether the captured graph covers the operators and deployment target. In JAX, check whether shapes and purity let XLA compile the program actually executed. A framework-level comparison or leaderboard can orient a decision, but it cannot replace profiling the specific workload on the target hardware.

The same simple network exposes how each design philosophy shapes the code. Listing 7.28 implements one neural network, a single linear layer mapping ten inputs to one output, across all three frameworks.

These three implementations solve the same mathematical problem but reveal distinct answers to the three problems. The differences are not cosmetic; they shape debugging workflows, deployment options, and optimization potential.

PyTorch binds state and computation together through class inheritance (`nn.Module`), solving the execution problem through eager evaluation: the graph builds as Python runs, making standard debuggers and control flow work naturally. Without graph capture, an optimizer cannot see the full computation before execution begins.

TensorFlow/Keras also executes eagerly by default in TensorFlow 2, while structured models can be captured with `tf.function` and exported to supported deployment runtimes. Portability still depends on operator coverage and conversion for each target.

JAX treats the model as a pure function²⁶ with immutable data and no internal state. In this model, `grad`, `vmap` (automatic vectorization), and `jit` (just-in-time compilation²⁷) are composable transformations on stateless functions rather than methods attached to a mutable object system. The cost is explicit parameter management and a programming model that may be unfamiliar to engineers coming from stateful frameworks.

Listing 7.28: Framework Comparison: Hello World: The same simple neural network implemented in PyTorch (object-oriented), TensorFlow/Keras (declarative), and JAX (functional), illustrating each framework’s distinct design philosophy.

```
# PyTorch - Dynamic, Pythonic
import torch.nn as nn

class SimpleNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(10, 1)

    def forward(self, x):
        return self.fc(x)

# TensorFlow/Keras - High-level API
import tensorflow as tf

model = tf.keras.Sequential(
    [tf.keras.layers.Dense(1, input_shape=(10,))]
)

# JAX - Functional approach
import jax.numpy as jnp
from jax import random

def simple_net(params, x):
    return jnp.dot(x, params["w"]) + params["b"]

key = random.PRNGKey(0)
key_w, key_b = random.split(key)
params = {
    "w": random.normal(key_w, (10, 1)),
    "b": random.normal(key_b, (1,)),
}
```

No framework maximizes all three goals simultaneously, and the ten-line program makes the trade visible in source code rather than buried in compiler internals: where state lives, when the graph exists, and what the compiler is allowed to see. The underlying currency is compiler visibility vs. human iteration latency. Graph capture and ahead-of-time compilation can reduce runtime overhead and intermediate data movement; eager evaluation shortens the human iteration loop, outside the iron-law runtime equation, at the cost of compiler visibility until a graph-capture path is used; functional purity gives XLA more freedom to transform a stable traced program. Each philosophy also shapes code syntax, team workflows, debugging practices, and deployment pipelines, so migration costs grow with project-specific integrations and assumptions.

7.7.5 Runtime constraint map

The same principle extends beyond the three general-purpose frameworks. Runtime families differ because they remove different degrees of flexibility in exchange for compiler visibility, smaller

²⁶ | **Pure function:** Returns outputs determined by its inputs without relying on untracked side effects; JAX transformations are designed for pure functions. Ordinary Python effects may run during tracing but are not represented in the compiled computation, and impure code can fail or behave unexpectedly. Runtime printing or external effects require supported mechanisms such as `jax.debug.print` or callbacks; random draws require explicitly managed keys.

²⁷ | **Just-in-time (JIT) compilation:** Traces a function for an argument signature and lowers it to backend-specific executable code; the first call pays tracing and compilation costs; compatible later calls can reuse the cached executable. New shapes, dtypes, devices, or static arguments may trigger another compilation. Both compilation time and cached-call overhead depend on the program, backend, and hardware.

binaries, or hardware-specific execution. Table 7.11 is therefore a constraint map: each row shows what the runtime gives up, what optimization path that enables, and which failure mode to test before committing.

Table 7.11: Runtime Constraint Map: Read each row left to right as one bargain, where a family gives up flexibility, gains an optimization path, and acquires a new way to fail. The middle column records where a family sits relative to an untuned eager baseline rather than a measured result, so latency, memory use, energy, and utilization still have to be measured for the specific model, hardware, precision, batch size, and compiler configuration.

Runtime family	Where it fits	Constraint anchor	Optimization path	Constraint to test
PyTorch eager	Research and iteration	Baseline latency; full Python/runtime footprint	Dynamic graphs, eager debugging	Dispatch overhead and missing graph view
Compiled Py-Torch/TensorFlow	Server training and serving	Workload-dependent gains when graph capture is clean	Graph capture, fusion, layout planning	Operator coverage and graph breaks
TensorFlow Lite/Core ML	Mobile and edge inference	Target-specific mobile latency and package-size budgets	Quantization, static graphs, NPU delegates	Target-specific conversion constraints
TF Lite Micro/microTVM	Microcontroller inference	Target-specific microcontroller RAM and flash budgets	Static allocation, INT8 kernels	Selected operators and constrained memory
ONNX Runtime	Cross-framework serving	Backend-dependent; can match native runtimes for supported graphs	Standard graph format, execution providers	Export gaps and custom-operator fallbacks
TensorRT/TVM	Hardware-specialized inference	Target-dependent gains over untuned eager baselines	Kernel fusion, precision lowering, autotuning	Narrower target and conversion assumptions

The map reveals one systems pattern rather than a product ranking. Each move toward a narrower runtime trades flexibility for a more predictable execution plan. Specialized inference runtimes such as TensorRT and Apache TVM can deliver substantial latency gains when the model converts cleanly and the deployment target is known. Mobile and microcontroller runtimes reduce footprint by removing training machinery and relying on static graphs, quantization, platform delegates, or fixed memory arenas. A delegate is a runtime plugin that routes supported operators to a target accelerator and falls back when the operator is unsupported. The engineering question is always what was removed to make the optimization possible, because unsupported operators, dynamic shapes, or graph breaks can erase the expected advantage.

These efficiency gaps become hard constraints beyond the server room. A multi-fold latency gap between eager execution and a specialized inference engine is an optimization opportunity on a cloud GPU. On a microcontroller, a framework that exceeds the device’s memory capacity cannot run. The selection criterion shifts from raw latency to whether the framework fits inside the memory and runtime envelope.

7.8 Deployment Targets

Across the cloud-to-edge spectrum, framework selection reweights the three core problems according to hardware boundaries. While the performance gaps measured in section 7.7.4 determine training throughput and operational cost in a data center, deploying to edge environments turns runtime efficiency into a binary feasibility test. The execution problem shifts from trading debuggability against graph optimization to fitting computation strictly within latency and memory envelopes. The differentiation problem disappears for pure inference workloads, eliminating the storage overhead of activation stashes and backward graphs. Meanwhile, the abstraction problem intensifies as software targets heterogeneous compute engines: server x86 and ARM CPUs, mobile NPUs, edge TPUs, and bare-metal microcontrollers operating within tens of kilobytes of SRAM.

Table 7.12 maps these runtime assumptions across the cloud-to-edge spectrum, detailing the optimization levers and binding physical constraints that define each hardware tier.

Table 7.12: Deployment Target Constraint Map: Runtime assumptions, optimization levers, and binding constraints for each deployment tier, from cloud servers to microcontrollers.

Environment	Runtime assumptions that usually fit	Optimization lever	Binding constraint
Cloud/Server	Full training/serving frameworks	Graph compilation, batching, lower precision	Throughput, cost
Edge	Static graph or portable serving runtime	Static graphs, lower-precision kernels	Workload-specific latency and memory
Mobile	App-integrated runtime with delegates	Accelerator delegates, compact model formats	Battery, thermal, app size limits
Microcontroller (TinyML)	Tiny runtime with fixed allocation	Static allocation, small integer kernels	Tight RAM, no dynamic memory

These boundaries demonstrate why target selection acts as a hard filter on framework architecture rather than a late packaging detail. For example, the Smart Doorbell KWS model in section 7.3.5 exemplifies the microcontroller tier. Operating within kilobytes of memory requires a runtime with a fixed, preallocated arena and a compact C/C++ footprint, eliminating dynamic memory allocation and Python runtime overhead entirely. This physical restriction forces a practical separation: engineering teams train models in expressive, dynamic environments on cloud GPUs, but must deploy them into runtimes with drastically narrower execution assumptions.

The **ONNX**²⁸ format addresses this fragmentation by standardizing model exchange across runtimes (ONNX Contributors 2019). A pipeline can train an architecture in PyTorch, export the computation graph to an ONNX intermediate representation, and execute the serialized graph in ONNX Runtime or compile it for a target accelerator. In contrast, TensorFlow Lite relies on its own conversion path, translating TensorFlow SavedModels into FlatBuffers rather than routing through ONNX. Although standard interchange formats reduce manual model conversion between development and production environments, they do not eliminate compatibility testing, operator coverage gaps, or custom-kernel fallback overhead. The model compression and serving strategies analyzed in chapter 10 and chapter 13 build directly on this export boundary.

Figure 7.14 illustrates this hub-and-spoke interoperability topology. Training frameworks export computation graphs into a unified intermediate representation, while downstream runtimes and hardware backends ingest that representation to execute across heterogeneous targets.

Interchange formats mitigate framework fragmentation, but an export boundary cannot salvage an architecture that violates the target’s physical limits. Converting a graph to ONNX or an embedded runtime after development completes cannot resolve an excessive memory footprint, missing hardware operators, or unsupported dynamic control flow. Treating deployment as an afterthought guarantees costly rewrites, making systematic evaluation of hardware and runtime trade-offs essential before writing the first line of model code.

7.9 Framework Selection

Framework selection is a constrained optimization problem across the execution, differentiation, and hardware abstraction dimensions established in this chapter. The question is not which framework is “best”; it is which execution model, differentiation system, and abstraction path survive the project’s constraints.

7.9.1 The framework selection trade-off space

Framework selection involves three interconnected tensions. The first is between development velocity and production performance: eager execution prioritizes iteration speed, while graph compilation prioritizes runtime optimization. Research teams that need to test ten architecture variants per day cannot afford minutes of compilation between experiments; production teams that deploy a single model for months cannot afford the throughput penalty of eager dispatch. The optimal point shifts as a project moves through its lifecycle.

This velocity-performance tension leads directly to the second: flexibility versus optimization depth. Eager execution makes host-language control flow natural but limits compiler scope until a larger region is captured. Static graphs can represent data-dependent control flow through

²⁸ | **ONNX:** The “fragmentation” ONNX addresses is that the framework used for model development may not match the runtime best suited to the deployment target. ONNX defines a hardware-agnostic graph representation that decouples the two, reducing the engineer-months of manual model conversion that would otherwise be required each time a deployment target changes. The accepted trade-off is that ONNX export can lose framework-specific optimizations or custom operators, requiring fallback implementations.

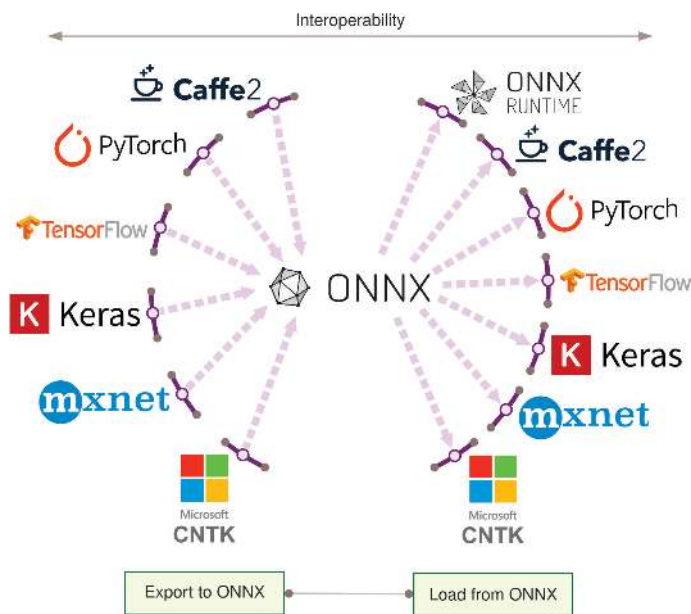


Figure 7.14: Framework Interoperability: The inward spokes represent exports into the shared ONNX format, while the outward spokes represent loads into compatible tools and runtimes; portability requires support on both sides of that boundary.

dedicated graph operators, exposing larger subgraphs for inter-operator kernel fusion that eliminates intermediate round-trips to HBM and keeps intermediate activations in on-chip SRAM. As table 7.3 demonstrated, this trade-off cascades through memory management, hardware utilization, and debugging workflows. It is an architectural constraint that governs the entire execution pipeline.

The flexibility-optimization tension, in turn, exposes a third: ecosystem breadth vs. specialization. General-purpose frameworks cover broad operator catalogs but often underperform specialized runtimes on workloads those runtimes can optimize deeply. Systems such as TensorRT and TVM optimize for narrower deployment targets through aggressive kernel fusion, precision selection, and hardware-specific memory layout scheduling (NVIDIA 2024c; Chen et al. 2018). While interchange formats such as ONNX bridge part of this gap (ONNX Contributors 2019), runtime specialization introduces its own failure modes: when an exported model encounters an unsupported operator, execution must fall back to unoptimized kernels or trigger expensive host-device synchronizations.

 Systems Perspective 7.4: Framework selection constraints

Rather than seeking the “best” framework, effective selection first eliminates candidates that cannot satisfy hard constraints: deployment target, required operations, compiler path, and team expertise. Only then should soft preferences such as performance, development speed, and ecosystem rank the survivors.

The TensorFlow ecosystem illustrates how these axes interact concretely. Its three variants (TensorFlow, TensorFlow Lite, and TensorFlow Lite Micro) trace a single design philosophy across progressively tighter physical constraints. Table 7.13 traces the resulting trade-offs in model expressiveness.

Table 7.13: TensorFlow Variant Model Support: The four rows show what each variant can express, not how large it is. Training capability disappears across the tiers while inference and lower-precision tooling survive intact, which leaves operator coverage as the discriminating row: broad in TensorFlow, inference-focused in TensorFlow Lite, and reduced to the operators one model selects in TensorFlow Lite Micro. Binary size and memory footprint appear in table 7.15.

	TensorFlow	TensorFlow Lite	TensorFlow Lite for Microcontrollers
Training	Yes	Limited	No
Inference	Yes	Yes	Yes
Operator Coverage	Broad	Inference-focused	Model-selected subset
Native Lower-Precision Tooling	Yes	Yes	Yes

This progression demonstrates that runtime efficiency is purchased by shedding generality: fewer supported operations enable smaller binaries, deterministic memory layouts, and aggressive lower-precision optimization.

7.9.2 Framework selection criteria

Three dimensions structure systematic framework evaluation: what the model requires (supported operations and graph semantics), what the software environment provides (operating system services, memory allocators, and accelerator delegation), and what the hardware physically permits (compute throughput, memory capacity, and thermal power limits). Each dimension acts as a filter: hard constraints eliminate non-viable candidates, while soft preferences rank the survivors.

7.9.2.1 Model requirements

The primary constraint is whether a framework can express the operations and execution semantics that a model requires. As shown in table 7.13, operator coverage narrows from the broad catalog of general-purpose TensorFlow to the inference-focused set in TensorFlow Lite, and finally to the minimal model-selected subset linked into TensorFlow Lite Micro. Each reduction strips dynamic operators and training machinery, concentrating compiler resources on static kernels. The underlying systems principle is that algorithmic expressiveness trades directly against machine efficiency: restricting the supported operator set allows compilers to prune dead code, generate tight register-resident inner loops, and eliminate dynamic dispatch overhead. Beyond operator coverage, model requirements also dictate graph semantics: whether computation must execute dynamically or can be captured ahead of time.

Systems Perspective 7.5: Dynamic vs. static computational graphs

The static-vs.-dynamic graph distinction (examined in section 7.3) has direct implications for model requirements analysis. Static graphs require control flow and dynamic behavior to be represented in forms understood by the graph runtime, which enables ahead-of-time optimization for deployment. Eager execution follows ordinary Python control flow directly, while graph-capture paths such as `torch.compile` and `tf.function` recover optimization opportunities when they can represent the executed program.

7.9.2.2 Software dependencies

Once model expressiveness is confirmed, the framework must integrate with the target software environment. Table 7.14 outlines how operating system dependencies, model loading mechanisms, and accelerator delegation vary across deployment tiers.

Table 7.14: TensorFlow Variant Capability Comparison: Capabilities of TensorFlow, TensorFlow Lite, and TensorFlow Lite Micro regarding operating system dependence, memory management, and hardware acceleration. Progressive constraint across variants enables selection by deployment context, from full-scale servers to resource-constrained edge devices.

	TensorFlow	TensorFlow Lite	TensorFlow Lite for Microcontrollers
Needs an OS	Yes	Yes	No
Model Access	Filesystem or service	Platform-specific storage mapping	Platform-specific compiled or mapped storage
Accelerator Support	Yes	Delegates	Optimized kernels and integrations

These distinctions illustrate how eliminating software abstractions expands deployment reach. Standard TensorFlow relies on full POSIX operating system services, dynamic shared libraries, and virtual memory allocators. TensorFlow Lite relaxes these requirements for mobile platforms, accessing models via memory-mapped files and delegating operations to hardware accelerators through vendor runtime APIs. TensorFlow Lite Micro removes the operating system dependency entirely, enabling bare-metal execution on microcontrollers without a POSIX filesystem or dynamic heap allocator (though it can run alongside an RTOS such as FreeRTOS or Zephyr). The model FlatBuffer is compiled directly into a constant byte array residing in read-only flash memory, and hardware acceleration relies on statically linked CMSIS-NN or DSP assembly kernels rather than dynamic driver delegates. Every software dependency stripped away unlocks a more constrained tier of physical hardware.

7.9.2.3 Hardware constraints

Software compatibility alone does not guarantee execution; the framework runtime must fit within the physical limits of memory capacity, storage, and compute bandwidth. As summarized in table 7.15, target processor architectures shift across deployment tiers from server-class x86 CPUs, GPUs, and TPUs in data centers through Arm Cortex-A application processors at the mobile edge, down to resource-constrained Arm Cortex-M microcontrollers and DSPs.

Table 7.15: TensorFlow Hardware Targets: TensorFlow variants target progressively more constrained hardware, shifting from x86, GPUs, and TPUs to Arm Cortex-A platforms and then Arm Cortex-M processors, DSPs, and MCUs. Actual binary size and memory footprint depend on the build, selected operators, delegates, model, and runtime configuration.

	TensorFlow	TensorFlow Lite	TensorFlow Lite for Microcontrollers
Base Binary Size	Build- and package-dependent	Build- and delegate-dependent	Model- and operator-selection-dependent
Base Memory Footprint	Model-, runtime-, and allocator-dependent	Model-, delegate-, and allocator-dependent	Fixed arena plus runtime state
Optimized Architectures	x86, TPUs, GPUs	Arm Cortex-A, x86	Arm Cortex-M, DSPs, MCUs

These hardware boundaries reflect the fundamental physical constraints—the light barrier, the power wall, and the memory wall—that divide the deployment spectrum into distinct operational regimes (section 2.2). In a data center, multi-megabyte runtime binaries and dynamic memory management overhead are negligible compared to gigabytes of high-bandwidth device memory. On a microcontroller with 256 KB of SRAM and 1 MB of flash, a general-purpose runtime cannot even be loaded. The framework must replace dynamic heap allocators with a single pre-allocated memory arena sized ahead of time to hold tensor activations and scratchpad buffers.

7.9.2.4 Production-ready evaluation factors

Satisfying model requirements, software dependencies, and hardware constraints establishes technical feasibility, but production viability requires evaluating operational reliability under sustained serving workloads. Runtimes that rely on dynamic memory allocation or host-language interpreters introduce tail-latency (p99) variance through garbage collection pauses and heap fragmentation.

In contrast, production-oriented runtimes enforce static tensor arenas and ahead-of-time graph lowering to ensure deterministic execution times.

Precision optimization forms a central operational lever. Lowering numerical precision from FP32 to FP8 or INT8 halves memory bandwidth consumption and increases arithmetic throughput on accelerator tensor cores, but it requires native quantization tooling. A production framework must support both post-training quantization (PTQ) with representative calibration datasets and quantization-aware training (QAT) to recover accuracy margins before deployment. Beyond per-device execution, production viability depends on toolchain stability: whether model checkpoints maintain backward compatibility across runtime updates, whether the framework exposes telemetry hooks for profiling kernel execution, and whether serving infrastructure can deploy updated graph definitions without downtime.

7.9.3 Development support and long-term viability assessment

Framework viability over a multi-year production lifecycle depends on whether the surrounding ecosystem keeps the chosen execution, differentiation, and abstraction paths maintainable. Community composition directly shapes engineering priorities: research-focused ecosystems prioritize experimentation velocity, interactive debugging, and rapid model prototyping; production-focused ecosystems invest in serving runtimes, backward compatibility, and hardware vendor integrations; and compiler-centric ecosystems advance mathematical transformations and code generation for emerging accelerators.

A framework's operational cost often depends more on surrounding ecosystem infrastructure than on the core tensor API. Integration with model registries, serving systems, cluster orchestrators, and accelerator profiling tools accelerates deployment, but each layer introduces an upstream dependency. Custom C++ or CUDA kernel implementations, specialized serialization formats, and vendor-specific compilation passes make framework migration prohibitively expensive once an organization standardizes its pipeline. Preserving an architectural exit path requires deliberate decoupling: standardizing intermediate representations through open formats such as ONNX, isolating data ingestion pipelines from framework-specific tensor classes, and maintaining modular operator boundaries.

The preceding sections evaluated execution, differentiation, and hardware abstraction as distinct architectural choices. In an executing model, however, these three systems operate simultaneously. A single training step is where they collide: eager dispatch or graph scheduling determines when work reaches silicon, reverse-mode automatic differentiation generates the backward pass while retaining intermediate activations, and hardware abstractions coordinate tensor layouts and kernel launches. Tracing one complete training step from input batch to parameter update reveals how these mechanisms integrate under physical machine constraints.

7.10 Anatomy of a Training Step

Execution, differentiation, and hardware abstraction operate simultaneously during model training. In listing 7.29, a minimal two-layer multilayer perceptron illustrates this convergence across eight executable statements. Tracing each statement through the runtime stack connects the quantitative principles developed in section 7.1 to physical execution: tensor allocation, kernel dispatch, dynamic graph recording, gradient computation, and parameter updates.

Listing 7.29: Training Step Anatomy: A minimal training iteration for a two-layer MLP, exercising tensor allocation, kernel dispatch, autograd recording, gradient computation, and parameter updates.

```
# Single training step for a 2-layer MLP
x = torch.randn(32, 784, device="cuda") # Input batch
y = torch.randint(0, 10, (32,), device="cuda") # Labels

# Forward pass
optimizer.zero_grad()
h = torch.relu(x @ W1 + b1) # Hidden layer
logits = h @ W2 + b2 # Output layer
loss = F.cross_entropy(logits, y)

# Backward pass
loss.backward()

# Parameter update
optimizer.step()
```

Phase 1: Forward pass (solving the execution problem)

During the forward pass, executing `h = torch.relu(x @ W1 + b1)` triggers an eager execution sequence:

1. Python dispatch: The Python interpreter invokes `torch.matmul`, which routes through PyTorch's C++ dispatcher to select the CUDA backend, incurring several microseconds of host CPU overhead before issuing work to the device.
2. Kernel selection: cuBLAS selects a GEMM kernel configured for dimensions $32 \times 784 \times 256$, partitioning the computation into thread blocks sized to fit the streaming multiprocessor shared memory and L2 cache tiles.
3. Kernel launch: The host thread enqueues the launch descriptor into the GPU command stream, consuming driver launch latency while asynchronous execution allows host execution to proceed without blocking on completion.
4. GPU execution: The streaming multiprocessors stream `W1` from HBM²⁹ through L2 cache into registers and Tensor Cores, evaluate the matrix product, and write the output activations back to HBM.
5. Autograd recording: Concurrently on the host, PyTorch's autograd engine instantiates an `MmBackward` node on the dynamic tape, saving references to input tensors `x` and `W1` in device memory for reverse-mode differentiation.

The subsequent bias addition (`+ b1`) and activation (`torch.relu`) follow an identical eager path, with each operation dispatching a separate kernel, executing round-trips to HBM, and appending a node to the tape.

Phase 2: Backward pass (solving the differentiation problem)

Calling `loss.backward()` executes reverse-mode automatic differentiation in four stages:

1. Tape traversal: The autograd engine traverses the recorded computational graph in reverse topological order, starting from the scalar loss node.
2. Gradient computation: The engine executes the backward kernel registered to each node. Traversing backward, `CrossEntropyBackward` evaluates $\frac{\partial \mathcal{L}}{\partial \text{logits}}$ using the softmax derivative; `MmBackward` for `W2` computes $\frac{\partial \mathcal{L}}{\partial W_2} = h^T \cdot \frac{\partial \mathcal{L}}{\partial \text{logits}}$ and $\frac{\partial \mathcal{L}}{\partial h}$; `ReluBackward` evaluates the activation mask, zeroing gradients where forward activations satisfy $h \leq 0$; and `MmBackward` for `W1` computes $\frac{\partial \mathcal{L}}{\partial W_1} = x^T \cdot \frac{\partial \mathcal{L}}{\partial z_1}$. Because the input batch `x` does not set `requires_grad=True`, traversal terminates without evaluating $\frac{\partial \mathcal{L}}{\partial x}$.
3. Gradient accumulation: Computed parameter gradients accumulate in-place into the `.grad` fields of leaf tensors (`W1`, `b1`, `W2`, `b2`).

²⁹ | **HBM (high bandwidth memory):** Provides 2–3 TB/s bandwidth on modern GPUs, making it the memory tier that most directly feeds accelerator arithmetic; HBM bandwidth determines whether operations are memory bound or compute bound, and its 80 GB capacity on an A100 sets the hard ceiling on total live training state. Weights, activations, gradients, optimizer state, and temporary workspace must fit during execution. When they do not, the framework must resort to offloading, selective recomputation, or placement across devices, each adding complexity to what the programmer perceives as a single `loss.backward()` call.

4. Memory deallocation: As each backward node completes, the framework releases its saved forward activations (such as h and x), allowing the caching allocator to reuse those memory regions for subsequent operations.

Phase 3: Memory traffic analysis (the physics at work)

Applying equation 7.4 to this step, table 7.16 breaks down the FLOPs, memory traffic, and arithmetic intensity for each operation:

Table 7.16: Per-Operation Roofline Analysis: Analytical work and memory-traffic estimates for a two-layer MLP training step. The cross-entropy row assigns one operation-equivalent each to exponentiation, accumulation, and normalization per logit, while the backward row assumes twice the forward work and three times its memory traffic. These are modeling assumptions rather than measured instruction counts. MatMul operations have much higher arithmetic intensity than element-wise operations, while the simplified ReLU and cross-entropy models remain memory-sensitive.

Component	Modeled Work	Memory Traffic	Arithmetic Intensity
MatMul ($x @ W1$)	$2 \times 32 \times 784 \times 256 = 12.8$ MFLOP	0.9 MB	13.7 FLOP/byte
ReLU	$32 \times 256 = 8.2$ KFLOP	65.5 KB	0.125 FLOP/byte
MatMul ($h @ W2$)	$2 \times 32 \times 256 \times 10 = 163.8$ KFLOP	44.3 KB	3.7 FLOP/byte
Cross-entropy (simplified)	~ 0.96 KFLOP	2.6 KB	0.4 FLOP/byte
Backward (assumed $2 \times$ forward)	~ 26 MFLOP	3.1 MB	8.3 FLOP/byte

The arithmetic-intensity column reveals the operational regime of each layer. Matrix multiplications exhibit operand reuse that drives intensity toward the compute roofline. Conversely, element-wise ReLU and reduction in cross-entropy perform negligible compute per byte transferred (0.125 to 0.375 FLOP/byte), remaining strictly bound by memory bandwidth and launch latency. In un-fused eager execution, each element-wise operation incurs a separate kernel dispatch and round-trips intermediate tensors through HBM without saturating compute units.

The analytical model yields ~ 39.1 MFLOP of total arithmetic work and ~ 4.2 MB of memory traffic. Dividing these quantities by peak A100 hardware rates determines the idealized lower bounds:

- $T_{\text{compute}} \approx 39.1 \text{ MFLOP} / 19.5 \text{ TFLOP/s FP32} \approx 2.0 \mu\text{s}$
- $T_{\text{memory}} \approx 4.2 \text{ MB} / 2.04 \text{ TB/s} \approx 2.1 \mu\text{s}$
- Assuming 12 ops kernel launches, $T_{\text{overhead}} \approx 12 \text{ ops} \times 15 \mu\text{s} \approx 180 \mu\text{s}$

Because T_{overhead} exceeds T_{compute} and T_{memory} by more than an order of magnitude, the step is overhead-bound. Accelerator execution units sit starved while the host CPU serializes dispatch. System engineering in this regime targets the launch tax rather than arithmetic efficiency:

- Kernel fusion (`torch.compile`): Fusing consecutive memory-bound operations into a single kernel avoids intermediate round-trips to HBM and collapses launch boundaries.
- Batch size scaling: Increasing the batch size scales the arithmetic payload linearly while keeping dispatch overhead constant, amortizing fixed launch latency across more compute operations.
- Graph capture: Static compilation or CUDA Graphs capture the dependency DAG once, replaying execution through a single driver invocation to eliminate host interpreter overhead entirely.

Phase 4: Hardware abstraction (solving the abstraction problem)

The identical Python script executes across heterogeneous hardware architectures through backend abstraction layers, as table 7.17 summarizes.

Each backend maps common tensor semantics to silicon-specific primitives. A single `loss.backward()` invocation routes to different execution pipelines depending on device placement. Differences in floating-point summation order, reduction trees, and non-deterministic atomic operations introduce numerical variation across backends, requiring equivalence to be validated against explicit numerical tolerances rather than bitwise identity.

Table 7.17: Hardware Backends Behind a Single Code Path: Each backend implements the same tensor operations through a different library and execution mechanism, so identical Python maps onto distinct silicon.

Hardware	Backend library	Execution mechanism
CUDA GPU	cuBLAS (NVIDIA 2024a; Choquette et al. 2021)	GEMM kernels and CUDA streams for async execution
CPU	Intel oneMKL or OpenBLAS (Intel Corporation 2026a; OpenBLAS Project 2026)	Thread-level parallelism around optimized kernels
TPU	XLA (Google 2025)	Compilation to TPU-specific high-level optimizer (HLO) operations
Apple Silicon	Metal Performance Shaders	MPS backend

Implementation decisions—device placement, batch size, compilation mode—cascade through execution, differentiation, and abstraction layers simultaneously. When systems treat the framework as an opaque mathematical interpreter rather than a hardware-aware scheduler, these interactions trigger launch starvation, memory fragmentation, and throughput collapse.

Systems Perspective 7.6: The three problems in action

In D·A·M terms, the single training step exposes how the framework balances algorithm and machine:

- Execution (Machine scheduling): Eager mode preserves imperative line-by-line debugging at the cost of per-kernel launch and dispatch latency.
- Differentiation (Algorithmic graph): Reverse-mode automatic differentiation retains intermediate forward activations in HBM to compute exact vector-Jacobian products in reverse topological order.
- Abstraction (Hardware portability): Unified tensor APIs decouple model code from heterogeneous silicon, dispatching identical operations to cuBLAS, oneMKL, or XLA HLO kernels.

Resolving the friction among these dimensions requires matching operator granularity to hardware rooflines: fusing operations to eliminate memory round-trips, scaling batches to amortize launch latency, and structuring tensors to saturate accelerator execution pipelines.

7.11 Fallacies and Pitfalls

Framework selection involves subtle trade-offs where intuitions from conventional software engineering fail. The memory wall, kernel fusion constraints, and deployment target diversity create pitfalls that waste engineering effort and cause production systems to miss latency targets.

Fallacy: “All frameworks provide equivalent performance for the same model architecture.”

Engineers assume that ResNet-50 yields identical performance across frameworks since the mathematics is the same, forgetting that performance is an emergent property of algorithm-machine co-design. In production, implementation matters. Compiled execution can improve supported eager workloads, and hardware-specialized inference engines such as TensorRT or TVM can reduce latency relative to untuned eager baselines when conversion is clean. The difference arises from kernel fusion depth, graph optimization strategies, memory access patterns, precision support, and backend libraries that vary between frameworks. Organizations that assume equivalence can miss latency service level agreements and require costly last-minute framework migrations.

Pitfall: Choosing frameworks based on popularity rather than project requirements.

Engineers assume the most popular framework works for any project. In reality, deployment constraints dominate. A mobile runtime such as ExecuTorch or TensorFlow Lite and a microcontroller runtime such as TensorFlow Lite Micro make different assumptions about memory allocation, operator coverage, and hardware delegates; the former targets phones with GB-scale memory, while

the latter often targets devices with hundreds of KB of RAM, commonly below 256 KB for small TinyML deployments. Teams that prototype edge applications without checking the final runtime can face memory bloat that exceeds device capacity or a late framework migration after development completes. Evaluate deployment targets per section 7.8 before selecting a training framework.

Fallacy: *“Framework abstractions eliminate the need for systems knowledge.”*

Engineers assume high-level APIs handle all optimization automatically. The Roofline Model (section D.2.1) proves otherwise. Element-wise operations such as ReLU perform little arithmetic per byte moved, so memory traffic and launch overhead can dominate even on accelerators with abundant compute. Section 7.3.1 explains how this imbalance can leave compute resources idle. Engineers must identify the limiting resource before choosing fusion, batching, layout, or precision optimizations.

Pitfall: *Ignoring vendor lock-in from framework-specific formats.*

Engineers assume framework migration is straightforward since models are “just math.” Converting TensorFlow SavedModel to PyTorch can require rewriting custom operations, validating numerical equivalence across large test suites, and retraining when operations lack exact equivalents. ONNX (section 7.8) improves portability, but custom operators, dynamic shapes, and backend-specific optimizations can still require manual work. Organizations that ignore this during initial framework selection face costly migrations when deployment requirements change or better frameworks emerge.

Fallacy: *Training framework choice is independent of production infrastructure.*

Engineers assume training framework choice is independent of deployment infrastructure. In practice, framework-infrastructure mismatches can impose substantial operational overhead. Some serving stacks provide atomic model swaps, while others require a process or container restart unless the deployment architecture adds version routing around them. Some frameworks and runtimes expose monitoring hooks directly; others require custom instrumentation. These are preview consequences of the serving and operations layers developed in chapter 13 and chapter 14; the local lesson is to evaluate the complete deployment stack during framework selection, including serving infrastructure, monitoring, and operational tooling.

Pitfall: *Increasing batch size without modeling activation memory.*

Engineers assume that if memory is available, larger batches always improve throughput. Larger mini-batches can amortize fixed dispatch overhead, but they also scale the activation memory that must remain live during training. A 7-billion-parameter model in FP16 consumes 14 GB, leaving 71.9 GB on an 80 GB A100 before accounting for other training state. Increasing batch size from 8 to 32 quadruples the batch-dependent activation footprint; transformer attention adds a large $\mathcal{O}(S^2)$ term in sequence length that makes each sample expensive. The resulting memory pressure can trigger recomputation strategies whose additional work reduces throughput despite the larger batch. Teams that blindly maximize batch size can achieve lower throughput than smaller batches that avoid these memory management pathways; section 8.3.2.1 formalizes the throughput target.

Fallacy: *Compilation overhead is negligible.*

Engineers assume compilation overhead is a one-time cost that pays off quickly. The hypothetical values in table 7.4 assign ResNet-50 higher compiled throughput and a compile window of 15 s to 30 s per graph change. Under those assumptions, a 10,000-image experiment with 10 code changes completes in 6.9 s in eager mode and 304.7 s in compiled mode, including recompilation overhead. The compiled workflow is therefore about 44.2× slower in this illustrative rapid-prototyping scenario. Teams that enable compilation during rapid prototyping can waste time waiting for recompilations that negate throughput gains.

Pitfall: *Using one execution policy for exploration and production.*

The right framework mode depends on the loop. During exploration, eager execution and small tests shorten feedback by avoiding repeated graph captures and recompilations. During production serving or long training runs, compilation can amortize setup across many requests or samples. The deciding factors are workload duration and stability: compilation pays when graph structure and shapes recur, whereas changing control flow or input shapes can invalidate cached artifacts

and restart the amortization. Teams should choose the policy from measured iteration latency and steady-state throughput. One policy for both phases either slows research iteration or leaves production throughput unused.

7.12 Summary

Machine learning frameworks solve three core systems challenges that make deep learning tractable on modern hardware. The first is execution: deciding when and how operations run. Frameworks balance the trade-off between eager evaluation for interactive debugging and graph execution for whole-program compilation, while modern hybrid approaches such as `torch.compile` capture graphs dynamically without sacrificing the Python control loop. The second is differentiation: computing gradients automatically. Frameworks implement reverse-mode automatic differentiation that applies the chain rule across compositions of operations, constructing an autograd tape or backward graph of vector-Jacobian products while retaining intermediate activations in memory. This primitive makes training large models feasible through a single `loss.backward()` call. The third is abstraction: targeting heterogeneous hardware from a unified interface. Frameworks provide multi-dimensional tensor abstractions, intermediate representations, and runtime dispatch systems that bridge high-level model definitions to optimized device kernels across CPUs, GPUs, TPUs, and custom accelerators.

These challenges are linked through the iron law of ML systems (section 1.7): execution strategy governs dispatch latency (L_{lat}), differentiation governs memory traffic (D_{vol}), and abstraction governs hardware utilization efficiency (η_{hw}). Because accelerator compute throughput has grown much faster than memory bandwidth, execution is frequently memory-bandwidth bound (D_{vol}/BW) rather than compute bound. Frameworks mitigate this memory wall through compiler optimizations: kernel fusion merges adjacent element-wise operations to keep intermediates in registers or on-chip SRAM instead of round-tripping through global memory, while selective recomputation trades redundant floating-point operations (O) to relieve peak activation memory pressure.



Key Takeaways: The layer between math and hardware

- **Every framework solves three problems:** Execution dictates runtime dispatch, differentiation computes gradients, and abstraction targets heterogeneous hardware. TensorFlow, PyTorch, and JAX make distinct infrastructure wagers across these axes—from static graph capture to dynamic tracing and functional transformations—establishing architectural commitments that outlast API syntax.
- **The memory wall drives optimization:** Accelerator compute capacity has outpaced memory bandwidth for decades, making off-chip data movement the primary execution bottleneck. Kernel fusion, selective recomputation, and lower-precision data layouts directly target the memory traffic term (D_{vol}/BW) in the iron law of ML systems rather than reducing raw operation counts.
- **Compilation must amortize setup cost:** The compilation continuum principle in equation 7.2 quantifies when runtime speedup offsets compilation overhead. While dynamic workloads favor eager execution, repeatedly executed graphs benefit from JIT or ahead-of-time compilation, where the dispatch overhead law (equation 7.4) demonstrates that small operations gain disproportionately once host launch overhead (L_{lat}) is removed.
- **Module abstractions automate state management:** Automatic parameter registration, mode-dependent execution (training versus inference), and hierarchical state serialization enable multi-billion-parameter model updates in a single optimizer step across diverse hardware backends.
- **Deployment constraints should drive selection:** Specialized runtimes trade operator coverage, static memory allocation, and compilation latency against stringent hardware budgets. The gap between server-class accelerator memory and microcontroller SRAM requires evaluating the downstream deployment target before committing to a development framework.

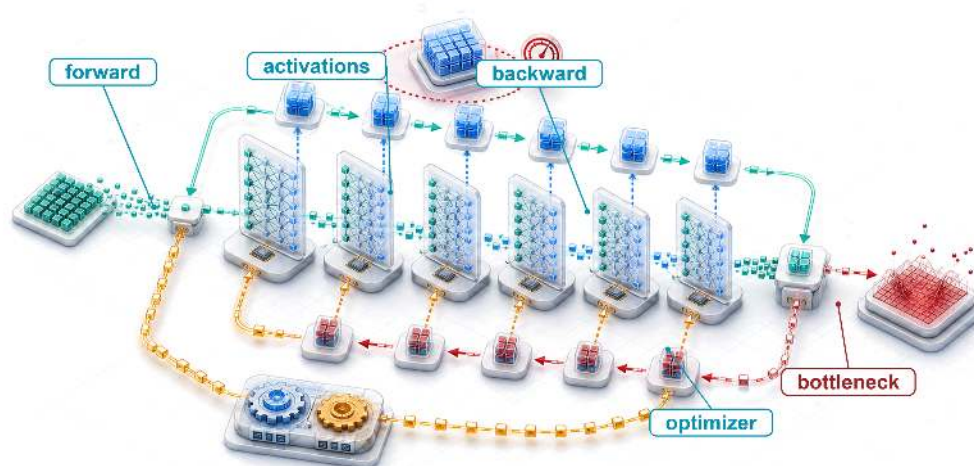
A framework presents itself as an ergonomic programming interface, yet its true role is translating mathematical graphs into physical hardware operations. That translation never eliminates hardware costs; it only shifts expenditure among the terms of the iron law of ML systems (principle 3). Eager execution prioritizes developer iteration at the cost of host dispatch latency (L_{lat}), graph compilation trades upfront optimization time to eliminate global memory traffic (D_{vol}/BW) through fused kernels, and automatic differentiation trades activation memory (D_{vol}) against recomputed arithmetic (O). What appears to be an API convenience is therefore a standing systems commitment: the framework cannot reduce the fundamental physical burden the iron law enforces, but only determine which term carries the load.



What's Next: From control room to power plant

Frameworks are the software substrate that translates abstract architectures into executable kernels. Computational graphs, autograd tapes, and kernel dispatch pipelines are the control room instruments: they give engineers visibility into and control over the training process. The machinery is built, but every budget decision it exposes remains unmade: which activations to checkpoint when memory runs short, how large a batch the hardware allows, and at what point a single device stops being enough. Chapter 8 is where those bills come due, scaling framework execution, differentiation, and memory-management machinery into the distributed systems that train models at scale.

Model Training



- 8.1 Training Systems Fundamentals
- 8.2 Iron Law of Training Performance
- 8.3 Mathematical Foundations
- 8.4 Pipeline Architecture
- 8.5 Identifying Bottlenecks
- 8.6 Pipeline Optimizations
- 8.7 Scaling Training Systems
- 8.8 Fallacies and Pitfalls
- 8.9 Summary

Purpose

Why can training a model cost orders of magnitude more than one inference?

Inference computes a forward pass as data flows through the network and a prediction emerges. Training adds a backward pass and an optimizer step, retains intermediate state, and repeats that work across many examples and optimization steps. Optimizers may also maintain momentum and variance estimates whose storage exceeds the model weights. The ratio depends on the model, workload, training duration, and inference volume, but one training run commonly costs orders of magnitude more than one inference. A lab that trains in three days can explore more designs than one that takes a month. A faster step is not faster training if it delays convergence, so throughput matters only relative to time to target quality. Across repeated experiments, even modest changes in convergence or utilization compound into substantial schedule and cost differences. The central systems challenge is to keep accelerators supplied with data and useful arithmetic without exhausting memory or losing time to synchronization. Which intervention helps depends on the binding compute, memory, data, or communication constraint, so profiling must precede scaling or optimization. Batch size, numerical precision, checkpointing, data loading, and parallelism must therefore be treated as coupled decisions rather than independent tuning knobs. For the systems engineer, training is where hardware and parallelism determine whether a run fits within memory, schedule, and budget, and whether the repeated optimization loop accelerates or stalls. In D·A·M terms, each precision and parallelism decision negotiates between the mathematics of optimization and the physics of execution.





Learning Objectives

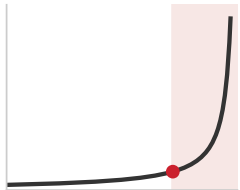
- Explain training cost asymmetry using forward, backward, optimizer-state, data, and iteration costs
- Calculate FLOPs, activation memory, optimizer state, and dollar cost for neural network training
- Compare SGD, Adam, and AdamW by convergence behavior, memory overhead, and compute cost
- Diagnose compute-, memory-, and data-bound training bottlenecks with roofline analysis and profiling evidence
- Apply mixed precision, checkpointing, gradient accumulation, and FlashAttention to fit accelerator memory and throughput limits
- Design single-machine training pipelines with prefetching, overlap, batching, and systematic re-profiling
- Evaluate when to scale beyond one machine using memory, duration, communication, energy, and cost constraints

8.1 Training Systems Fundamentals

RUNNING a model once and training it from scratch live on opposite sides of the systems cost curve. Frameworks provide the execution substrate: computational graphs schedule operations, automatic differentiation computes gradients, and hardware abstractions target diverse accelerators. Those mechanisms make a single training step possible; training systems make it repeatable at scale. The same forward/backward/update loop may run many thousands or millions of times while retaining activations, feeding accelerators, and staying within practical memory, time, and budget limits.

This chapter uses an external order-of-magnitude estimate of approximately \$50,000 for GPT-2-scale training and a public estimate near \$100M for a GPT-4-class training run.¹ Neither figure was disclosed as an audited training bill. This **training cost asymmetry** reflects the difference between evaluating a trained model for a defined request and repeatedly executing forward, backward, and update computations across a training corpus.

For GPT-2, the chapter uses roughly 3.00×10^9 floating-point operations per token for a forward evaluation, following the parameter-based accounting associated with the architecture in the GPT-2 technical report (Radford et al. 2019). The total budget of 1.50×10^{21} FLOPs corresponds to roughly 167 billion token positions when forward and backward work is approximated as three times the forward cost. This accounting estimate is not a claim that GPT-2 executed that many complete inference requests. The scale makes training systems engineering a distinct discipline and shows how access to training infrastructure can constrain participation in AI development.



Training cost stays flat across scale, then explodes past the large-scale knee.

¹ | **Training cost scaling:** The roughly $2,000\times$ ratio between this chapter's two cost anchors illustrates how parameter counts, training-token budgets, hardware generations, and accelerator fleets can compound; it is not a controlled comparison of disclosed training bills. OpenAI's GPT-2 report documents the model and training setup but does not disclose training cost (Radford et al. 2019). Exact GPT-4 training details were also not disclosed, so the GPT-4-class figure is explicitly an industry estimate supported by public reporting and independent infrastructure analysis (Knight 2023; Patel and Wong 2023).



Definition 8.1: Training systems

Machine learning training systems are software-hardware systems that execute the iterative optimization loop (forward pass, loss computation, backward pass, and parameter update) to minimize a loss function over a training dataset.

1. **Significance:** Training memory cost is $8\times$ the inference memory cost per parameter in a standard mixed-precision Adaptive Moment Estimation (Adam) setup: a 7B-parameter model requires 14 GB (FP16 weights) + 14 GB (FP16 gradients) + 28 GB (FP32 master weights) + 56 GB (Adam first and second moments in FP32) = 112 GB at least, before accounting for activation storage. This multiplier is a major reason a model that runs inference on one GPU requires multiple GPUs for training.
2. **Distinction:** Unlike inference systems, which execute a single forward pass and discard intermediate activations, standard backpropagation stores the tensors needed by the backward pass, creating a footprint that grows with model depth and batch size.

3. **Common pitfall:** A frequent misconception is that training failures are compute problems. A common failure is an out-of-memory (OOM) error, a memory-management problem: tensors saved for backpropagation accumulate during the forward pass and can exhaust memory before the first gradient is computed.

Three characteristics distinguish training workloads from general-purpose computing:

- **Computational intensity:** The 1.50×10^{21} FLOPs budget spread over days of wall-clock time demands sustained PFLOP/s-scale throughput from hardware whose realized large-model throughput is often far below theoretical peak (Narayanan et al. 2021; Chowdhery et al. 2022).
- **Memory pressure:** Storing 1.5B weights requires 6 GB in FP32; the Adam optimizer adds two state tensors per parameter, consuming another 12 GB. Activation memory adds a workload-dependent footprint governed by architecture, sequence length, batch size, precision, and checkpointing, and the combined state can exceed a single accelerator’s capacity.
- **Data dependencies:** Each gradient update depends on the result of the previous one, creating sequential bottlenecks that limit how much parallelism the system can exploit.

Each challenge points to a different kind of fix. Computational intensity pushes the system toward higher accelerator utilization and lower-precision arithmetic. Memory pressure calls for techniques such as gradient checkpointing,² a specific application of *rematerialization* (discarding and recomputing intermediate values to save memory, from chapter 7) that trades recomputation for reduced activation storage, and mixed-precision training,³ which reduces the memory footprint of weights and activations. Data dependencies motivate pipeline designs that overlap computation with data movement, building directly on the data loading throughput optimized in chapter 4 so the accelerator never sits idle waiting for the next batch. The analysis here establishes the execution baseline on single-machine and single-node multi-GPU topologies; scaling across distributed network boundaries introduces multi-node communication fabrics and fault-tolerance protocols treated in advanced distributed systems.

8.1.1 The staged system pipeline: Identifying “accelerator bubbles”

A training system is not a single loop; it is a **staged system pipeline**. Reaching high accelerator utilization requires analyzing training as a factory floor where four distinct stages coordinate to keep the ALUs busy:

1. **Data Loading & Preprocessing** (CPU/Storage): Fetching raw bits from Non-Volatile Memory Express (NVMe), decoding, and augmenting on CPU cores.
2. **Host-to-Device Transfer** (PCIe): Moving the processed batch over the PCIe bus into GPU memory via direct memory access.
3. **Forward/Backward Pass** (Accelerator): Propagating activations and gradients through layers on the GPU.
4. **Parameter Synchronization** (Interconnect): Exchanging gradients between accelerators over the available fabric. NVLink provides up to 900 GB/s in the H100 configuration used later, while other systems may use PCIe or a network interconnect.

Any mismatch in the throughput of these stages creates **accelerator bubbles**, intervals in which some accelerator resources are idle or underutilized while waiting for another stage. A systems engineer reduces these bubbles through techniques such as asynchronous prefetching and pipeline overlap, aiming to make the next batch available before the current step needs it. Section 8.5 develops the quantitative tools for measuring and reducing these bubbles.

That architectural dependency chain dictates the system’s operational hierarchy. The iron law of training performance (section 1.7) separates total operations, peak throughput, and hardware utilization, establishing the accounting framework for neural-network workloads, optimizer dynamics, backpropagation mechanics, and arithmetic intensity. Within the execution pipeline, data loading, the forward pass, the backward pass, and parameter updates form serial dependencies where each stage constrains throughput downstream. Targeted mitigations address specific terms in that

² | **Gradient checkpointing (activation checkpointing):** Backpropagation retains the forward values required by each backward rule. Checkpointing saves selected values and recomputes others during backward, trading additional work for lower activation memory. The $\sqrt{N_L}$ checkpoint placement and its associated recomputation cost describe the classical schedule analyzed by T. Chen et al. (2016) rather than every checkpointing implementation. Section 8.6.5.2 works through that schedule for GPT-2’s layer count.

³ | **Mixed-precision training:** Uses lower precision for selected computation and storage while retaining higher precision where accumulation or numerical range requires it. A common FP16 recipe maintains FP32 master weights and uses loss scaling to reduce gradient underflow (Micikevicius et al. 2017). BF16 (“Brain Floating Point,” from Google Brain (Wang and Kanwar 2019)) matches FP32’s 8-bit exponent range and therefore usually avoids dynamic loss scaling, though the exact state and accumulation precision depend on the optimizer and implementation.

throughput equation: mixed-precision arithmetic, tiled attention kernels, gradient accumulation, activation checkpointing, and asynchronous prefetching. Once single-accelerator throughput saturates, multi-device scaling exposes inter-chip interconnect bandwidth as the governing constraint.

These physical constraints compound in large-scale execution: a single unmitigated numerical explosion can invalidate thousands of accelerator-hours.



War Story 8.1: The PaLM loss spikes (2022)

Context: PaLM’s 540-billion-parameter run used 6,144 Tensor Processing Unit (TPU) v4 chips across two 3,072-chip Pods connected through Google’s data-center network and suffered severe training-instability spikes absent from smaller runs (Chowdhery et al. 2022).

Failure mode: The training loss spiked roughly 20 times despite gradient clipping. The spikes occurred at highly irregular intervals, sometimes late in training, never appeared in smaller models, and had no identified principled mitigation.

Impact: Each recovery discarded recent progress on an expensive 6,144-chip run, so reproducible checkpoints and repeatable data order were prerequisites for replay.

Response: Engineers restarted training from an uncorrupted checkpoint roughly 100 steps prior to each spike and skipped approximately 200–500 data batches around the failure window.

Systems lesson: Operational fault tolerance directly governs the utilization factor η_{hw} in the iron law of training performance (equation 8.1). Automated loss-spike detection, checkpoint cadence, and batch skipping are core system mechanisms required to protect cluster compute efficiency.

8.2 Iron Law of Training Performance

PaLM’s loss spikes demonstrate that instability at frontier scale is a predictable physical consequence of pushing computational intensity, memory pressure, and data dependencies simultaneously. Each of those three characteristics is individually manageable, yet their interactions produce failure modes that operational recovery alone cannot prevent. Diagnosing which factor dominates a given failure, and quantifying the cost of each, requires a formal decomposition of training time into its physical constituents.

The iron law provides exactly that organizing framework: it decomposes training time so that every optimization technique maps to a specific term in the equation. This is a specialized application of the general iron law of ML systems introduced in section 1.7, focused specifically on maximizing computational throughput.

 Definition 8.2: The iron law of training performance

The iron law of training performance expresses training time through the workload's operation count and its effective achieved throughput:

$$T_{\text{train}} = \frac{O}{R_{\text{peak}} \times \eta_{\text{hw}}} \quad (8.1)$$

Here η_{hw} is an effective end-to-end utilization factor, the achieved model-operation rate divided by the relevant hardware peak. Data stalls, communication, synchronization, launch overhead, and inefficient kernels all reduce this factor. Prefetching and overlap can hide part of those costs but do not guarantee their removal.

1. Significance: In the D·A·M taxonomy, these terms map to distinct optimization levers: O is governed by Data (training token count) and Algorithm (model architecture or compression methods such as pruning and distillation), R_{peak} is governed by the Machine (hardware execution units at the chosen numerical precision), and η_{hw} is the effective systems utilization factor and primary engineering target (Narayanan et al. (2021) measured 44 to 52 percent of theoretical peak throughput across GPT models trained on A100

clusters). Chapter 10 defines the compression techniques; here they serve only to show where such methods enter the accounting.

2. **Distinction:** The general iron law exposes data movement, computation, and latency separately. This training form folds every source of lost throughput into η_{hw} , making it useful for budget accounting but insufficient for diagnosing which underlying term is responsible.
3. **Common pitfall:** A frequent misconception is that η_{hw} is fixed by hardware. System efficiency is a pipeline property: memory bandwidth saturation, kernel launch overhead, and synchronization barriers each reduce η_{hw} independently, and diagnosing which factor dominates requires profiling rather than reading hardware specs.

Equation 8.1 reveals three levers for improvement: reduce total operations through algorithmic innovation, increase peak throughput through hardware selection and precision formats, or improve utilization through pipeline orchestration. Core training optimization techniques target one or more of these levers, as summarized in table 8.1.

Table 8.1: Iron law optimization mapping: Optimization techniques mapped to iron law terms. Understanding which term a technique affects guides optimization strategy selection.

Technique	Term Affected	Mechanism
Mixed Precision (FP16/BF16)	Peak Throughput $\uparrow$	Uses supported lower-precision hardware paths
Data Prefetching	Utilization $\uparrow$	Reduces accelerator idle time waiting for data
Gradient Checkpointing	Memory $\downarrow$, Operations $\uparrow$	Reduces saved activations by recomputing selected values
Gradient Accumulation	Memory $\downarrow$, Synchronization $\downarrow$	Builds an effective batch from serialized micro-batches
Operator Fusion	Memory Traffic $\downarrow$, Overhead $\downarrow$	Avoids intermediate transfers and reduces kernel launches
FlashAttention	Memory Traffic $\downarrow$, Utilization $\uparrow$	Same asymptotic FLOPs, much lower HBM IO; backward recomputation may add FLOPs

The iron law focuses on execution efficiency—how fast the hardware processes a given workload. It does not capture data-side factors such as data quality, dataset size, or curriculum design, which affect how many total operations O are needed to reach a target accuracy. A cleaner dataset or a better data mix can reduce the number of epochs required, shrinking O without touching hardware at all. Holding the workload fixed, the question is how to execute it as fast as possible.

Actual training throughput remains below the arithmetic peak whenever kernels, data delivery, or communication leave resources idle. Scaling to multiple accelerators introduces additional communication overhead, a trade-off quantified in section 8.7.



Checkpoint 8.1: The physics of training

Training speed is governed by the utilization of hardware peaks.

Utilization gap

- ☐ **Peak vs. real:** why does sustained end-to-end utilization remain below the arithmetic peak?
- ☐ **Utilization losses:** which pipeline stalls introduced earlier reduce η_{hw} , and which can prefetching or overlap hide?
- ☐ **Order of magnitude:** a frontier run needs on the order of 10^{23} FLOPs at a utilization η_{hw} well below 1. Estimate which lever shortens wall-clock time more, doubling R_{peak} or doubling η_{hw} , and why the iron law says the two are not interchangeable in practice.

The iron law provides a static framework for reasoning about training performance, but the history of deep learning reveals how the binding constraint has shifted over time as hardware and algorithms co-evolved. In 1986, Rumelhart and colleagues popularized backpropagation for multilayer neural

networks (Rumelhart et al. 1986). In 2012, AlexNet trained an ImageNet classifier in five to six days on two GPUs (Krizhevsky et al. 2012), demonstrating how well neural-network parallelism mapped to graphics hardware. By 2017, transformers (Vaswani et al. 2017) shifted attention toward large-scale sequence modeling and high-throughput accelerator kernels. GPT-3 in 2020 consumed about 3.14×10^{23} FLOPs (Brown et al. 2020), making utilization (η_{hw}) critical. By 2023, training efficiency improved through targeted systems optimizations: FlashAttention reduces memory traffic while improving η_{hw} ; gradient checkpointing trades additional O for memory capacity; mixed precision increases R_{peak} . Each innovation addresses a specific iron-law bottleneck.

8.2.1 Running example: Training GPT-2

In inference, GPT-2 serves as the Bandwidth Hog lighthouse. Here it recurs as the chapter’s stable workload for testing each training cost and optimization.

Lighthouse 8.1: Lighthouse example: Training GPT-2

Context: GPT-2 (1.5 billion parameters) serves as the primary case study because it is large enough to expose meaningful compute, activation, and optimizer-state constraints while remaining easier to analyze than frontier-scale models. Whether it requires distributed training depends on precision, optimizer state, activation memory, hardware capacity, and the target training time. Table 8.2 maps each model property to the systems constraint it creates:

Table 8.2: GPT-2 (1.5 Billion Parameters) Lighthouse Model Specifications: Parameter count, architecture depth, dataset size, and total training compute mapped to their systems implications. Each row pairs a model property with the engineering constraint it creates—memory footprint for weights, activation pressure from pipeline depth, I/O throughput requirements, and parallelization demand.

Property	Specification	Systems Implication
Parameters	1.5B (XL)	Requires ~3 GB (FP16) or ~6 GB (FP32) for weights alone.
Architecture	48 Layers, 1600 Dim	Deep pipeline creates heavy activation memory pressure.
Dataset	WebText (40 GB)	Preprocessing, shuffling, and delivery must sustain the selected training configuration.
Compute	$\sim 1.50 \times 10^{21}$ FLOPs	Training duration depends on achieved throughput and accelerator count.

Mechanism: Training GPT-2 couples a total operation count of $O \approx 6 \cdot P \cdot D_{\text{tokens}}$, or 1.50×10^{21} FLOPs, with a strict memory footprint $M_{\text{state}} = M_{\text{weights}} + M_{\text{grads}} + M_{\text{opt}} + M_{\text{act}}$. Balancing execution time $T_{\text{train}} = \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}$ requires orchestrating data ingestion, activation memory, and compute throughput within accelerator high-bandwidth memory (HBM) capacity.

Not all training workloads are compute bound. Recommendation models like DLRM are dominated by massive embedding tables (100B to 10T parameters, mostly embeddings) that make them memory bandwidth bound rather than compute bound. For such workloads, the first scaling problem is often capacity: splitting the embedding tables across devices so the model fits at all. In contrast, dense, compute-intensive training—exemplified by the GPT-2 lighthouse model analyzed below—exposes compute-bound and activation-memory limits rather than capacity partitioning.

Within the end-to-end ML lifecycle, training systems ingest prepared data from upstream pipelines (chapter 4) and produce model checkpoints for downstream deployment. Data quality directly governs training stability, while training throughput dictates iteration velocity. This systems pressure manifests across three distinct scales. At data scale, large corpora require high-throughput I/O pipelines and distributed storage to avoid accelerator starvation. At model scale, billion-parameter architectures force the system to balance memory capacity against communication, choosing between replicating weights across data-parallel devices⁴ or partitioning layers across devices with model parallelism.⁵ At infrastructure scale, coordinating multiple accelerators introduces synchronization overhead that can dominate training time. These physical trade-offs make the workflow contracts

⁴ | **Data parallelism:** Replicates the full model on every device, splitting only the data. Each device computes gradients independently, then an AllReduce operation synchronizes them—adding communication volume proportional to model size at every step. This synchronization tax limits scaling efficiency: doubling accelerators rarely halves training time once communication becomes the bottleneck.

⁵ | **Model parallelism:** Partitions the model’s layers across devices when it exceeds single-device memory. Activations must transfer between devices at every partition boundary, and naive partitioning creates “pipeline bubbles” where downstream devices idle while waiting. Microbatch pipelining, as in GPipe and PipeDream, recovers much of this lost efficiency (Huang et al. 2019; Narayanan et al. 2019).

from chapter 3 essential during training: orchestration decisions dictate both iteration velocity and hardware efficiency.

GPT-2's 40 GB WebText corpus sits at the lower end of this data scale spectrum. Larger corpora change the data path qualitatively.

🔍 Systems Perspective 8.1: The 10 GB to 10 TB scale factor

In a memory-resident regime, a corpus may fit in system RAM and avoid repeated storage reads after caching, though preprocessing and transfer costs remain. In a streaming regime, the corpus exceeds local memory and the system must orchestrate storage reads, preprocessing, caching, and delivery throughout training. The appropriate design may include multiple workers, prefetching, local caches, or distributed storage, depending on which stage limits throughput.

Scale therefore changes more than the amount of data; it transforms the system's physics.

These scaling constraints translate into concrete workflow requirements. Training workflows consist of interdependent stages—data preprocessing, forward and backward passes, and parameter updates—extending the neural network concepts from chapter 5. System constraints dictate these execution limits: modern accelerators possess arithmetic intensities that easily outstrip HBM bandwidth, leaving memory-bound kernels starved for operands (Hennessy and Patterson 2017). In multi-accelerator configurations, cross-device synchronization introduces latency penalties, where interconnect bandwidth (PCIe, NVLink, InfiniBand) dictates the communication-to-computation ratio.⁶

The same hardware-software boundary that frameworks exposed in chapter 7 is central here. Mixed-precision training emerged from recognizing that Tensor Core hardware could accelerate reduced-precision arithmetic. Gradient checkpointing arose from memory capacity constraints. Training systems engineering is the work of matching those algorithmic choices to the physical limits of the machine.

Every training bottleneck ultimately traces back to the execution cost of specific mathematical operations. Dense matrix multiplication is compute-bound and governed by peak arithmetic throughput (Goto and Geijn 2008), whereas elementwise activation functions and normalization layers are memory-bandwidth-bound. Meanwhile, optimizer states inflate the resident memory footprint regardless of arithmetic activity. Designing systems to execute these operations efficiently begins with accounting for each cost category separately.

8.3 Mathematical Foundations

Linear algebra expresses matrix multiplication compactly as $\mathbf{C} = \mathbf{AB}$. In hardware, training a model such as GPT-2 repeatedly executes this operation on tensors that far exceed the tens of megabytes available in on-chip SRAM, requiring matrix tiles to be continuously staged from High Bandwidth Memory (HBM). Dense matrix multiplications exhibit high arithmetic intensity, enabling specialized execution units such as Tensor Cores to achieve high compute utilization. In contrast, surrounding elementwise operations—such as activations, normalizations, and residual additions—exhibit low arithmetic intensity and remain bounded by memory bandwidth, creating opportunities for kernel fusion to eliminate redundant memory traffic. Chapter 5 established *what* neural network operations compute and *why* they enable learning; the systems question is *what* they cost in FLOPs, memory, and bandwidth when those operations execute at scale.

Four dimensions structure this cost analysis:

- FLOP counts of the matrix operations that dominate dense neural-network training.
- Memory requirements for storing activations and optimizer states simultaneously.
- Bandwidth demands that determine whether operations are compute-bound or memory-bound.
- Arithmetic intensity classifications that guide optimization strategy selection.

Together, these dimensions provide the vocabulary for analyzing the computational intensity, memory pressure, and data dependencies introduced in section 8.1.

⁶ | **Transformer training interconnect sensitivity:** Self-attention's $\mathcal{O}(S^2)$ memory and compute scaling increases activation size with sequence length. In pipeline parallelism, activations cross devices at model-partition boundaries, while data parallelism synchronizes gradients whose volume scales with parameter count. The resulting communication cost depends on the parallelism strategy, topology, message size, and degree of overlap, making interconnect choice a first-order training-system decision.

8.3.1 Neural network computation

Neural network training consists of repeated matrix operations and nonlinear transformations. These operations are conceptually simple but create the system-level challenges that dominate modern training infrastructure. The introduction of backpropagation⁷ by Rumelhart et al. (1986) and the development of efficient matrix computation libraries such as Basic Linear Algebra Subprograms (BLAS)⁸ (Dongarra et al. 1988) laid the groundwork for modern training architectures.

8.3.1.1 Mathematical operations in neural networks

Forward propagation, in its simplest case, involves two operations: matrix multiplication and activation function application. Matrix multiplication implements the linear transformation at each layer. At layer ℓ , the computation can be described as (following the row-vector convention established in chapter 5):

$$\mathbf{A}^{(\ell)} = f(\mathbf{A}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)})$$

where:

- $\mathbf{A}^{(\ell-1)}$ represents the activations from the previous layer (or the input layer for the first layer), with each row being a sample in the batch,
- $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_{\ell-1} \times n_{\ell}}$ is the weight matrix at layer ℓ , which contains the parameters learned by the network,
- $\mathbf{b}^{(\ell)}$ is the bias vector for layer ℓ ,
- $f(\cdot)$ is the activation function applied elementwise (for example, ReLU, sigmoid) to introduce nonlinearity.

8.3.1.2 Matrix operations

Section 5.4.2.2 established that forward propagation reduces to chains of matrix multiplications, and section 6.9.1 catalogued the computational primitives—general matrix multiplication, convolution, and dynamic attention—that every architecture shares. Training amplifies these patterns: each operation executes not once but billions of times, and each forward pass is paired with a backward pass that roughly doubles the computational cost. Understanding which matrix operations dominate—and how their shapes change between forward and backward passes—reveals *why* specific system designs and optimizations emerged for training.

Matrix multiplication dominance has driven both algorithmic and hardware innovations. Early neural network implementations relied on standard CPU-based linear algebra libraries, but the scale of modern training demanded specialized optimizations. Strassen’s algorithm⁹ reduced the naive $\mathcal{O}(n^3)$ complexity to approximately $\mathcal{O}(n^{2.807})$ (Strassen 1969), and contemporary hardware-accelerated libraries like cuBLAS (NVIDIA 2024a) continue pushing computational efficiency limits.

This computational dominance has driven system-level optimizations: blocked matrix computations that parallelize across multiple units, and memory hierarchies designed for the access patterns of both forward and backward passes. As neural architectures grew, weight and activation matrices both had to remain accessible for backpropagation, and hardware evolved to serve these dense multiplication patterns within growing memory budgets. To illustrate the scale of these operations concretely, consider the attention layer computations in the GPT-2 lighthouse model.

A single GPT-2 layer makes the scale of these computations concrete.

Napkin Math 8.1: GPT-2 attention layer computation

Each GPT-2 layer performs attention computations that exemplify dense matrix multiplication demands. For one GPT-2 transformer layer (all heads combined) with batch size $B = 32$, sequence length $S = 1024$, and hidden dimension $d = 1600$:

Query, Key, Value Projections (the three linear transformations that create attention inputs—3 separate matrix multiplications):

$$\begin{aligned} \text{FLOPs} &= 2 \times 3 \times (B \times S \times d \times d) \\ &= 2 \times 3 \times (32 \times 1024 \times 1600 \times 1600) \approx 503 \text{ billion FLOPs} \end{aligned}$$

⁷ | **Backpropagation prove-nance:** The algorithm was independently derived by Lin-nainmaa (1970) for automatic differentiation of computer programs and by Werbos (1974) in a Harvard PhD thesis on economic modeling—over a decade before Rumelhart et al. (1986) popularized it for neural networks. This delay between derivation and broad adoption recurs in ML systems history: attention mechanisms predated transformers by decades, but the 2017 Transformer showed that attention-heavy models could train efficiently on contemporary GPU systems; later TPU and GPU clusters enabled much larger deployments. A framework’s backward pass traverses the computational graph in reverse topological order and applies the chain rule; independent subgraphs may expose parallel work during that traversal.

⁸ | **BLAS (basic linear algebra subprograms):** The original BLAS specification standardized reusable Fortran-callable vector operations (Lawson et al. 1979); later BLAS extensions added matrix-vector and matrix-matrix routines, giving the familiar Level 1, Level 2, and Level 3 hierarchy (Dongarra et al. 1988). Training is dominated by Level 3 operations precisely because their high arithmetic intensity— $\mathcal{O}(n)$ FLOP/byte—can sustain high compute utilization. Frameworks commonly dispatch dense matrix multiplication to optimized libraries such as cuBLAS and oneDNN or to compiler-generated kernels.

⁹ | **Strassen’s algorithm:** Achieves $\mathcal{O}(n^{2.807})$ by replacing one of eight sub-multiplications with additions, but training systems rarely benefit. The recursion disrupts the regular memory-access patterns that Tensor Cores exploit, and accumulated rounding errors across billions of training iterations can destabilize convergence. In practice, mainstream general matrix multiplication libraries such as cuBLAS leave the dominant training matrix multiplications to highly optimized blocked $\mathcal{O}(n^3)$ kernels with superior hardware utilization rather than recursive Strassen variants.

Attention matmuls ($\mathbf{QK}^\top$ and $\mathbf{A}_{\text{attn}} \mathbf{V}$):

$$\text{FLOPs}_{\mathbf{QK}} = 2 \times B \times N_{\text{heads}} \times S \times S \times d_{\text{head}} = 107.4 \text{ billion FLOPs}$$

$$\text{FLOPs}_{\mathbf{A}_{\text{attn}} \times \mathbf{V}} = \text{FLOPs}_{\mathbf{QK}}; \quad \text{pair total} \approx 214.8 \text{ billion FLOPs}$$

Output projection (attention output $\rightarrow$ hidden):

$$\text{FLOPs} = 2 \times B \times S \times d \times d \approx 168 \text{ billion FLOPs}$$

Feed-Forward Network (Two linear transformations with expansion factor 4):

$$\text{FLOPs} \approx 16 \times B \times S \times d^2$$

Computation Scale

- Per-layer forward total (QKV 503 + attention 214.8 + output proj 168 + FFN): ~2228.01 GFLOP
- With 48 layers in GPT-2: ~320.8 TFLOP per training step
- Across this illustrative 50,000 steps scenario: ~16041.7 PFLOP

Systems insight: A V100 GPU (125 TFLOP/s peak with Tensor Cores, 15.7 TFLOP/s without) would require 2.6 s for the modeled attention-plus-FFN training-step estimate at 100 percent utilization (theoretical peak; practical throughput would be lower). Reaching a 180 to 220 ms training step is therefore already a multi-accelerator lower-bound problem: ideal arithmetic alone requires roughly 12 to 15 V100s, and practical systems need additional headroom for utilization losses, communication, and pipeline overhead.

These FLOP counts are not academic bookkeeping. They are the compute term of the iron law made concrete, and they explain why training cost scales as a predictable function of model architecture and sequence length rather than as an unpredictable emergent property.

8.3.1.3 Matrix-vector and batched operations

Not all operations in neural networks involve large matrix-matrix multiplications. Normalization layers, bias additions, and certain recurrent computations involve matrix-vector operations instead. Although computationally simpler than matrix-matrix multiplication, these operations present distinct system challenges: they exhibit lower hardware utilization due to their limited parallelization potential. A single vector provides insufficient work to keep thousands of accelerator cores busy simultaneously. This characteristic influences both hardware design and model architecture decisions, particularly in networks processing sequential inputs or computing layer statistics.

Recognizing the limitations of matrix-vector operations, the introduction of batching transformed matrix computation in neural networks. By processing multiple inputs simultaneously, training systems convert matrix-vector operations into more efficient matrix-matrix operations. This approach improves hardware utilization but increases memory demands for storing intermediate results. Modern implementations must balance batch sizes against available memory, leading to specific optimizations in memory management and computation scheduling.

The progression from matrix-vector to batched matrix-matrix operations helps explain the hardware design choices in modern accelerators. Google's first TPU, designed for inference, incorporated a specialized matrix unit and memory hierarchy for dense operations (Jouppi et al. 2017). At a different scale, GPT-3 training used distributed accelerators to execute the matrix-matrix multiplication patterns exposed by large batches (Brown et al. 2020).



Systems Perspective 8.2: Why GPUs dominate training

The matrix operations described in section 8.3.1.2 directly explain data-center training hardware architecture. GPUs became central to large-scale training for three reasons:

- Matrix multiplication’s independent element calculations map well to thousands of GPU cores (NVIDIA A100 has 6,912 CUDA cores).
- Specialized hardware units like Tensor Cores provide substantially higher peak throughput for supported lower-precision matrix operations.
- Blocked matrix computation patterns enable efficient use of GPU memory hierarchy (L1/L2 cache, shared memory, global memory).

When a later illustrative GPT-2 scenario assumes a $2.4\times$ V100 mixed-precision speedup, Tensor Core acceleration of eligible matrix multiplications is one source of the gain. The realized end-to-end ratio also depends on nonmatrix operations, memory traffic, and input and communication overhead.

Matrix multiplications dominate training compute, but neural networks require more than linear transformations. Between each layer’s matrix operations, activation functions introduce the nonlinearity that enables networks to learn complex patterns. These functions appear computationally trivial compared to matrix multiplication, yet their implementation characteristics affect training efficiency in ways that matter at scale.

8.3.1.4 Activation functions

Activation functions like sigmoid, tanh, ReLU, and softmax introduce nonlinearity, but their implementation characteristics also shape training system performance. From a systems perspective, the choice of activation function determines computational cost, hardware utilization, and memory access patterns during backpropagation.

The critical question for ML systems engineers is not *what* these functions do mathematically, but *how* their cost behaves at scale. The benchmarks and trade-offs that follow build toward one systems thesis. Because elementwise activations move many more bytes than they compute, their kernels are often bounded by memory bandwidth rather than arithmetic, so the per-operation differences below matter less than their magnitudes first suggest.

Activation costs can accumulate across many elements and layers, but their contribution to end-to-end time depends on tensor shape, fusion, implementation, precision, and hardware. Figure 8.1 preserves one illustrative Apple M2 timing snapshot from the chapter’s development history. Its benchmark configuration was not recorded, so the values should not be interpreted as reproducible evidence or transferred to other systems. On accelerators, activation kernels are often limited by memory traffic, fusion boundaries, and special-function throughput rather than by scalar CPU instruction latency. The same activation may run as a standalone kernel, fuse with a neighboring operator, or use a compiler-selected approximation. The plotted scalar timing therefore cannot predict its cost inside a compiled training step.

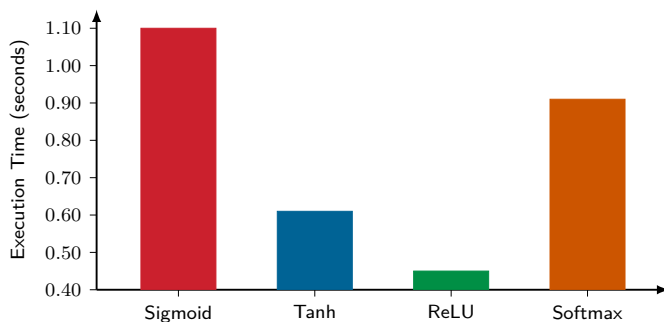


Figure 8.1: Illustrative Activation Timing Snapshot: A legacy Apple M2 timing snapshot compares four activation implementations under an unrecorded benchmark configuration. The chart preserves the original relative illustration but does not establish portable performance ratios. The y-axis is truncated to begin at 0.40 s, so bar heights exaggerate the differences.

Modern accelerators can implement these functions through native instructions, polynomial approximations, lookup methods, or fused kernels. ReLU requires a simple threshold operation,

while sigmoid, tanh, and exact Gaussian Error Linear Unit (GELU) formulations require more arithmetic or special-function support. The realized latency difference still depends on whether arithmetic, memory traffic, or launch overhead is binding. ReLU may produce zeros that a later system can exploit, but the sparsity fraction depends on the activation distribution and trained model. Softmax¹⁰ adds a reduction and normalization across the selected axis. Those dependencies require coordination, though optimized implementations remain highly parallel.

Table 8.3 summarizes how these behaviors translate into system constraints.

Table 8.3: Activation Function Systems Comparison: Activation functions differ in mathematical behavior, kernel cost, fusion opportunities, and memory access patterns.

Function	Key Advantages	Key Disadvantages	System Implications
Sigmoid	Smooth; bounded output in (0, 1).	Gradients saturate at large magnitudes; output is not zero-centered.	Uses exponential or approximation kernels whose cost depends on the target.
Tanh	Smooth; zero-centered output in (−1, 1).	Gradients saturate at large magnitudes.	Uses special-function or approximation kernels similar in structure to sigmoid.
ReLU	Simple threshold operation; nonsaturating for positive inputs.	Can produce persistently inactive units.	Often inexpensive and fusible; exploitable sparsity depends on downstream support.
Softmax	Produces a normalized distribution over an axis.	Requires reductions and cross-element coordination.	Parallel implementations coordinate maximum, sum, and normalization stages.

ReLU remains common because it is simple and efficient, while softmax is essential when a model must normalize scores into probabilities. GPT-2 illustrates a different choice through GELU, the Gaussian Error Linear Unit.

🔍 Systems Perspective 8.3: The GELU activation choice

Beyond the foundational activation functions covered in chapter 5 (Sigmoid, Tanh, ReLU), modern architectures increasingly adopt smoother alternatives. GPT-2 uses a GELU activation (Radford et al. 2019), the Gaussian Error Linear Unit introduced by Hendrycks and Gimpel (2016) and often implemented with the common tanh-based approximation from that formulation. Its exact definition is:

$$\text{GELU}(x) = x \cdot \Phi(x) = x \cdot \frac{1}{2} \left[1 + \text{erf} \left(\frac{x}{\sqrt{2}} \right) \right]$$

where $\Phi(x)$ is the cumulative distribution function of the standard normal distribution. GELU applies a smooth, input-dependent scaling rather than ReLU’s hard threshold. It is deterministic despite its formulation in terms of a Gaussian cumulative distribution. Whether it improves model quality relative to another activation is an empirical property of the architecture and training recipe, not a universal guarantee. Exact GELU uses `erf`, while common implementations use a tanh-based approximation such as listing 8.1. The approximation changes arithmetic cost and introduces a small numerical difference from the exact function. End-to-end impact depends on fusion, tensor size, precision, kernel implementation, and the fraction of runtime spent in activation kernels, so no fixed GELU-to-ReLU ratio applies across systems.

¹⁰ | **Softmax:** Computing a numerically stable softmax requires a maximum reduction, exponentiation, a sum reduction, and normalization across the selected axis. Parallel kernels implement these stages with coordinated reductions. FlashAttention restructures the larger attention computation so softmax statistics can be maintained while tiles remain near the compute units.

Listing 8.1: GELU Approximation: Fast approximation avoids expensive `erf()` computation while preserving activation properties.

```
# Fast GELU approximation used in production systems
# Avoids expensive erf() computation while
# preserving activation properties
gelu_approx = (
    0.5 * x * (1 + tanh(sqrt(2 / pi) * (x + 0.044715 * x**3)))
)
```

The GELU approximation highlights a broader pattern: compute cost is not always the dominant concern. For activation functions, the real bottleneck is often memory bandwidth rather than arithmetic operations. This distinction between compute-bound and memory-bound operations directly affects optimization priorities and recurs throughout the analysis of training bottlenecks.

Dense matrix multiplication can reuse operands across many multiply-accumulate operations, while an unfused elementwise activation typically performs little work for each value read and written. This arithmetic-intensity gap often makes standalone activation kernels memory-sensitive. Fusion can remove intermediate memory traffic, and special-function throughput can still matter for kernels whose arithmetic is not fully hidden. Replacing one activation with another therefore has no fixed wall-clock effect.

Systems Perspective 8.4: Memory bandwidth bottlenecks

Activation functions reveal a critical systems principle: not all operations are compute bound. While matrix multiplications saturate accelerator compute units, activation functions often become memory bandwidth bound for three reasons:

- Elementwise operations perform few calculations per memory access; ReLU performs one operation per load.
- Simple operations complete faster than memory transfer time, limiting parallelism benefits.
- Accelerator compute capability has grown faster than external-memory bandwidth, increasing pressure to reuse data near the compute units.

This is why scalar operation counts alone do not predict activation-kernel latency. The forward pass must account for fusion and activation storage as well as function-evaluation cost when determining whether memory bandwidth limits throughput.

Forward-pass matrix multiplications dominate floating-point operations, while activation functions stress memory bandwidth. Transforming these forward activations into model convergence requires propagating error gradients backward through the graph to update parameter tensors. That optimization step introduces state storage, update scheduling, and numerical stability constraints that govern training efficiency.

8.3.2 Optimization algorithms

Optimization algorithms translate error gradients into parameter updates that steer the model toward convergence. While mathematically formulated as steps across an abstract loss surface, every optimizer step introduces concrete physical constraints on accelerator execution: persistent state tensors that consume High Bandwidth Memory (HBM), memory-bandwidth-bound elementwise updates, and synchronization barriers. In the D-A-M taxonomy, the choice of optimization algorithm couples the mathematical trajectory (Algorithm) directly to the working-set footprint and memory traffic (Machine).

8.3.2.1 Gradient-based optimization methods

Section 5.4.4.5 introduces gradient descent as the fundamental optimization algorithm: iteratively adjusting parameters in the direction of steepest descent. That conceptual formulation assumes

modest networks on single devices. At hardware scale, gradient descent and its variants collide with physical bottlenecks: memory capacity limits how many activations and optimizer states remain resident, while memory bandwidth limits how rapidly parameters update.

Gradient descent Gradient descent is the mathematical foundation of neural network training, iteratively adjusting parameters to minimize a loss function. In training systems, this mathematical operation translates into specific computational patterns. For each iteration, the system must execute four dependent operations:

1. Compute forward pass activations
2. Calculate loss value
3. Compute gradients through backpropagation
4. Update parameters using the gradient values

The computational demands of gradient descent scale with both model size and dataset size. Computing gradients requires storing intermediate activations during the forward pass for backpropagation. These activations consume memory proportional to the depth of the network and the number of examples being processed.

Traditional gradient descent processes the entire dataset before each parameter update. For a training set with one million examples, the system must compute an aggregate gradient over all examples before taking one step. Let D denote the number of examples in the training dataset, let B_{micro} denote the examples resident for one forward/backward pass, and let $T_{\text{forward+backward per example}}$ denote the amortized forward/backward time per example under the chosen micro-batch execution regime. Gradient accumulation composes multiple micro-batches into one larger effective batch. The peak-memory relation in equation 8.2 and the step-time relation in equation 8.3 capture the implementation distinction:

$$\text{Peak Activation Memory} \propto B_{\text{micro}} \times \text{Activation Memory per Example} \quad (8.2)$$

$$T_{\text{step}} \propto D \times T_{\text{forward+backward per example}} \quad (8.3)$$

Section C.3.3 formalizes this breakdown by deriving the full training memory equation, including optimizer-state overhead. Full-batch training does not require storing every example's activations simultaneously if the dataset is streamed or microbatched; peak memory is governed by the examples held in memory at once. The systems problem is still severe: processing $D = 1,000,000$ examples before each update creates million-example iteration times where parameters remain frozen, reducing the rate at which the model can learn from data.

Hardware-friendly variants relax the need for an exact full-dataset gradient on every update. Stochastic gradient descent (SGD)¹¹ estimates gradients from sampled examples or mini-batches rather than the entire dataset. This changes update frequency and gradient variance while allowing each update to operate on a bounded working set.

Evaluating single examples ($B = 1$), however, collapses accelerator efficiency. Modern accelerators achieve peak throughput by mapping large general matrix multiplication (GEMM) operations across systolic arrays and single instruction, multiple data (SIMD) execution units. When $B = 1$, layer operations degrade into matrix-vector products (GEMV), reducing arithmetic intensity to $O(1)$ operations per byte transferred from device memory. Compute units starve while waiting for DRAM or HBM transfers. Simultaneously, single-example updates require reading parameter tensors from device memory, applying gradients, and writing updated weights back after every sample, saturating memory bandwidth without amortizing parameter access across multiple inputs.

Mini-batch processing Mini-batch gradient descent emerges as a practical compromise between full-batch and stochastic methods, an algorithm-machine co-design that computes gradients over small batches of examples aligned with modern accelerator architectures (Dean et al. 2012). The batch size B becomes a key system parameter, influencing both computational efficiency and memory requirements.

¹¹ | **Stochastic gradient descent:** “Stochastic” from Greek *stochastikos* (“able to guess”); rather than waiting for an exact full-dataset gradient, SGD updates from sampled examples or mini-batches. Full-batch and stochastic methods can both stream data through bounded micro-batches, so their peak activation memory is determined by the resident micro-batch rather than by dataset size alone. Their central difference is how many sampled gradients contribute to each parameter update.

Definition 8.3: Batch processing

Batch processing aggregates multiple training examples into tensor operations that amortize fixed per-step overhead across B examples and can expose more parallel work. Increasing B may improve occupancy and arithmetic intensity, but it does not guarantee that every operation becomes compute bound.

1. **Significance:** Throughput often increases with batch size until the target hardware is sufficiently occupied, while the number and quality of updates needed for convergence follow a separate workload-dependent curve. Goyal et al. demonstrated a ResNet-50 ImageNet recipe at $B = 8,192$ using linear learning-rate scaling and warmup (Goyal et al. 2017). That result is a configuration-specific demonstration rather than a universal critical batch size.
2. **Distinction:** A mini-batch update averages gradients over B examples. Under independent, similarly distributed samples, the standard deviation of this mean decreases approximately as $1/\sqrt{B}$. The update processes more data than a single-example update, so its net data movement and time depend on reuse, implementation, and how batch size changes the number of updates required.
3. **Common pitfall:** A frequent misconception is that linear learning-rate scaling (multiplying η by B/B_0) works at any batch size. It is a training recipe that commonly requires warmup and empirical validation, and its useful range depends on the model, data, optimizer, and target accuracy.

The choice of batch size B directly governs accelerator memory allocation. The decomposition in equation 8.4 separates fixed parameter and gradient storage from the batch-dependent activation working set:

$$\text{Memory Required} = \text{Parameter Memory} + \text{Gradient Memory} + B \times \text{Activation Memory} \quad (8.4)$$

Because the activation term scales with B while parameter and gradient memory stay fixed, doubling the batch doubles the activation working set, and a model that fits comfortably at a small batch can exhaust the 40–80 GB of HBM on a high-end training accelerator once the batch grows. Section 8.4.3.2 works this budget through layer by layer for ResNet-50.

Increasing batch size transforms memory-bound vector operations into compute-bound matrix multiplications. In dense feedforward and attention projections, parameter weights loaded from HBM into on-chip SRAM are reused across all B samples in the batch. Arithmetic intensity increases proportionally with B , saturating tensor-core pipelines and amortizing parameter read overhead. Linear scaling rules allow scaling learning rates alongside batch size to preserve convergence rates up to an empirical limit (Goyal et al. 2017).

Because activation memory scales linearly with B , the capacity of accelerator HBM bounds the maximum batch size that can execute in a single forward/backward pass. When the batch size required for high hardware occupancy or stable convergence exceeds device memory, systems decouple execution batch size from optimization batch size using gradient accumulation. The system executes multiple smaller micro-batches (B_{micro}), serializing their execution to keep peak activation memory bounded as described in equation 8.2, accumulates the resulting gradients in place, and triggers the optimizer update only once the cumulative effective batch is complete.

8.3.2.2 Adaptive and momentum-based optimizers

Basic SGD applies a uniform scalar learning rate to every parameter tensor, which can stall training when loss curvature differs by orders of magnitude across dimensions. Momentum smooths updates using directional gradient history, RMSprop adapts individual step sizes using squared-gradient magnitudes, and Adam combines first-moment tracking with adaptive second-moment scaling (Kingma and Ba 2015). While these algorithmic enhancements can reduce the total optimization steps required to converge, they do so by introducing persistent auxiliary state that directly increases accelerator memory pressure and memory-bus traffic.

¹² | **Momentum:** Borrowed from physics, where momentum (mass $\times$ velocity) describes an object's tendency to continue moving. The metaphor explains the design because accumulated velocity smooths noisy gradients and can reduce the iterations needed for convergence. The systems cost is one additional velocity value per parameter, while Adam maintains two moment values per parameter.

Momentum-based methods Momentum methods¹² address SGD’s oscillation problem by accumulating a velocity vector across iterations, damping high-frequency oscillations along steep ravines. In hardware, this smoothing requires allocating and maintaining a persistent velocity tensor matching the exact shape and precision of the model parameters. Parameters and this auxiliary state together occupy $2\times$ the parameter footprint before gradients and activations are counted.

Adaptive learning rate methods While momentum smooths gradient direction, it does not address the different scales of gradients across parameters. RMSprop¹³ solves this by maintaining a moving average of squared gradients for each parameter, automatically reducing step sizes for parameters with historically large gradients. Adapting step sizes per parameter requires maintaining a running average of squared gradients s_t , imposing a $2\times$ static memory overhead equivalent to momentum. Furthermore, computing this running average introduces elementwise square and square-root kernels during each parameter update.

Adam optimization Adam¹⁴ combines the benefits of both momentum and RMSprop: momentum’s gradient smoothing addresses noisy updates, while RMSprop’s adaptive scaling handles parameter-specific step sizes. This combination maintains two moving averages for each parameter. Here m_t and v_t are the first- and second-moment buffers, β_1 and β_2 are their decay factors, and ϵ is the numerical-stability constant in the denominator:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) \nabla \mathcal{L}(\theta_t) \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) (\nabla \mathcal{L}(\theta_t))^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \\ \theta_{t+1} &= \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \end{aligned}$$

Maintaining both moment buffers triples the parameter-plus-optimizer-state footprint: the training system must allocate two persistent state tensors (m_t and v_t) alongside the parameter tensors. For a 100M-parameter model, those auxiliary vectors alone add 800 MB beyond weight storage.

8.3.2.3 Optimization algorithm system implications

Auxiliary state memory increases progressively from SGD (zero auxiliary state) through Momentum and RMSprop (one state vector per parameter) to Adam (two state vectors per parameter), as quantified in table 8.4. These memory footprints establish a fundamental trade-off against convergence rates:¹⁵ while Adam often achieves target loss in fewer optimization steps, its static memory footprint restricts the batch size that fits in accelerator HBM and increases memory-bus traffic on every update. At GPT-2 scale, this overhead becomes a first-order memory constraint.

Table 8.4: Optimizer Memory Footprint: Different optimization algorithms impose varying auxiliary-state costs due to the storage of intermediate values like velocities and squared gradients. The multiplier row counts parameters plus optimizer auxiliary state and excludes gradients and activations; full training memory must add those terms explicitly. These trade-offs govern resource-constrained deployments and large-scale model training.

Property	SGD	Momentum	RMSprop	Adam
Memory Overhead	None	Velocity terms	Squared gradients	Both velocity and squared gradients
Parameter + optimizer state	$1\times$	$2\times$	$2\times$	$3\times$
Dense State Access	Parameters and gradients	Adds velocity	Adds squared-gradient state	Adds first- and second-moment state
Update Work	Lowest auxiliary work	Adds momentum update	Adds adaptive scaling	Adds moment updates and bias correction
Hardware Efficiency	Varies	Varies	Varies	Varies
Convergence Behavior	Varies	Varies	Varies	Varies

The costs quantified in table 8.4 create a design tension. Adam’s two moment buffers may improve optimization for a given workload, but their memory can reduce the model size or batch size that

¹³ | **RMSprop:** Proposed by Geoffrey Hinton in Lecture 6e of his 2012 Coursera course—never published in a peer-reviewed paper, making it perhaps the most influential optimizer disseminated via a slide deck. RMSprop divides the learning rate by a running average of recent gradient magnitudes, adapting step sizes per parameter. This per-parameter adaptation is what Adam inherits as its second moment v_t , directly contributing to Adam’s $3\times$ memory overhead described in section 8.3.2.3.

¹⁴ | **Adam (adaptive moment estimation):** The two moving averages are the first moment (momentum) and second moment (uncentered variance) of the gradients, stored for every model parameter. For a 7B model, Adam’s FP32 moment tensors alone consume 56 GB; the full standard mixed-precision training state before activations is 112 GB once FP16 weights, FP16 gradients, FP32 master weights, and Adam moments are counted together.

¹⁵ | **Convergence:** A run reaches its stopping criterion when the selected training or validation metric no longer improves enough to justify additional work. The required steps depend on the model, data, optimizer, schedule, batch size, and target quality. Fewer steps can reduce wall-clock time and cost, but adaptive optimizers add state whose memory and traffic must be included in the system budget.

¹⁶ | **AdamW (Adam with decoupled weight decay):** Standard L_2 regularization enters the gradient and is therefore rescaled by Adam’s adaptive update. AdamW applies weight decay as a separate parameter update, restoring the intended distinction between adaptive gradient scaling and decay (Loshchilov and Hutter 2019). It uses the same moment-buffer structure as Adam, though its effect on model quality remains workload dependent.

fits on an accelerator. AdamW¹⁶ (Loshchilov and Hutter 2019) decouples weight decay from Adam’s adaptive gradient update without adding another per-parameter state tensor.

Napkin Math 8.2: GPT-2 optimizer memory requirements

A representative GPT-2 XL training configuration uses the Adam optimizer with five hyperparameters:

- $\beta_1 = 0.9$ (momentum decay)
- $\beta_2 = 0.999$ (second moment decay)
- Learning rate: Warmed up from 0 to $2.5e-4$ over first 500 steps, then cosine decay
- Weight decay: 0.01
- Gradient clipping: Global norm clipping at 1.0

Math:

For GPT-2’s 1.5B parameters in FP32 (4 bytes each), the memory breaks down across four components:

- Parameters: $1.5B \times 4 \text{ bytes} = 6 \text{ GB}$
- Gradients: $1.5B \times 4 \text{ bytes} = 6 \text{ GB}$
- Adam State (m, v): $1.5B \times 8 \text{ bytes} = 12 \text{ GB}$
- Total static memory: 24 GB

This explains why GPT-2’s 24 GB static training state alone approaches the 32 GB capacity of a V100 before activation storage is counted.

System Decisions Driven by Optimizer

1. Mixed precision training (FP16) reduces operation precision but requires keeping FP32 master weights, maintaining the static memory footprint at ~24 GB.
2. Gradient accumulation (splitting effective batches into smaller micro-batches) allows effective batch size $B = 512$ despite memory limits.

Systems insight: This illustrative configuration shows how Adam’s moment buffers constrain capacity. Any convergence benefit relative to SGD or momentum must be measured under a matched model, data, schedule, batch size, and stopping criterion.

8.3.2.4 Framework optimizer interface and scheduling

After optimizer memory is quantified, the framework interface matters because it fixes when that state is read, written, cleared, and preserved across steps. A training loop separates gradient computation from parameter updates so that the system can accumulate gradients, synchronize them, or defer updates without changing the optimizer equations. Listing 8.2 demonstrates where Adam optimization enters that cycle.

Listing 8.2: Adam Training Loop: Standard four-step optimization cycle with gradient clearing, forward pass, backward pass, and parameter update.

```
import torch
import torch.nn as nn
import torch.optim as optim

# Initialize Adam optimizer with model parameters and learning rate
optimizer = optim.Adam(
    model.parameters(), lr=0.001, betas=(0.9, 0.999)
)
loss_function = nn.CrossEntropyLoss()

# Standard training loop implementing the four-step optimization cycle
for epoch in range(num_epochs):
    for batch_idx, (data, targets) in enumerate(dataloader):
        # Step 1: Clear accumulated gradients from previous iteration
        optimizer.zero_grad()

        # Step 2: Forward pass - compute model predictions
        predictions = model(data)
        loss = loss_function(predictions, targets)

        # Step 3: Backward pass - compute gradients via autodiff
        loss.backward()

        # Step 4: Parameter update - apply Adam optimization equations
        optimizer.step()
```

The `optimizer.zero_grad()` call marks the boundary between one update and the next. Gradients accumulate across calls to `backward()`, so clearing them explicitly prevents stale gradients from contaminating the next batch. The same accumulation behavior later becomes useful for large effective batch sizes, but only when the training loop manages the boundary deliberately.

The `optimizer.step()` method is the other boundary: it consumes the current gradients and mutates persistent optimizer state. For Adam optimization, this call implements momentum estimation, squared gradient tracking, bias correction, and the parameter update. Algorithm 8.1 makes the hidden state explicit so the memory cost remains visible rather than disappearing behind an API call.

P G Adam

In this simplified static-state view, Adam's two moment buffers occupy twice the parameter storage.

Algorithm 8.1 Adam parameter update (one optimizer step)

Require: gradient $g_t = \nabla \mathcal{L}(\theta_t)$; step t ; rate η ; decays β_1, β_2 ; constant ϵ

Ensure: updated parameter θ_{t+1} ; moment buffers m_t, v_t carried to the next step

- 1: $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t \triangleright$ first moment (momentum)
- 2: $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \triangleright$ uncentered second moment
- 3: $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$; $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t) \triangleright$ bias correction
- 4: $\theta_{t+1} \leftarrow \theta_t - \eta \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) \triangleright$ parameter update

Steps 1 and 2 keep two persistent moment buffers per parameter, so parameters plus optimizer state reach roughly $3\times$ the parameter memory before gradients, activations, and FP32 master weights are counted. Beyond capacity, the optimizer step imposes a severe memory-bandwidth bottleneck. Mathematically, the update consists of elementwise operations with an arithmetic intensity below 1 FLOP/byte: in FP32, updating each parameter requires reading four tensors ($g_t, \theta_t, m_{t-1}, v_{t-1}$) and writing three (m_t, v_t, θ_{t+1}), streaming 28 bytes across the memory bus for roughly a dozen arithmetic operations. Because matrix execution units (such as Tensor Cores) sit idle during this memory-bound phase, high-performance training systems use fused optimizer kernels to combine moment updates, bias correction, and weight writes into a single kernel launch per parameter group, minimizing round-trips through device memory.

8.3.2.5 Learning rate scheduling integration

The framework’s learning rate scheduling hook changes the optimizer’s trajectory without adding per-parameter state. It adjusts the learning rate η during training, letting the system shape convergence behavior while leaving the underlying optimizer equations intact.

Schedules such as cosine annealing, exponential decay, or step-wise reductions implement that trajectory by changing the step size over time. A scheduler tracks lightweight scalar state (current step, epoch, warmup phase) that must be serialized into checkpoints alongside optimizer states to guarantee bit-exact resumption. Chapter 7 covers the framework scheduler interface that coordinates these steps. This separation lets the systems engineer combine base optimization algorithms (SGD, Adam) with scheduling strategies (cosine annealing, linear warmup) without modifying the underlying tensor update rules.

Parameter update rules define trajectories given gradient vectors, but assume those gradients are already materialized. For billion-parameter networks, gradient evaluation constitutes the dominant computational and memory burden of the training loop. The mechanics of gradient generation—rather than the optimizer step itself—dictate training memory budgets and runtime.

8.3.3 Backpropagation mechanics

Backpropagation evaluates parameter gradients by traversing the network’s computational graph in reverse topological order. In forward inference, an accelerator deallocates each activation tensor as soon as the succeeding layer consumes it. Training breaks this ephemeral lifecycle: computing the gradient with respect to a layer’s parameters requires the exact activation tensor produced by the preceding layer during the forward pass. Every forward activation must therefore remain resident in HBM until the backward traversal returns to that layer, transforming activation storage into the binding memory constraint of large-scale training.

Section 5.4.4 establishes the mathematical foundation: the chain rule decomposes gradient evaluation into layer-by-layer vector-Jacobian products, pulling error signals backward from the scalar loss to input parameters. At systems scale, the central question is not *what* backpropagation computes, but *what* it costs in memory capacity and arithmetic throughput. The layer operations from section 8.3.1.1 produce forward activations that must be retained for the backward pass. Computing $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}}$ requires direct access to these stored activations, creating the training memory equation that gradient checkpointing later exploits.

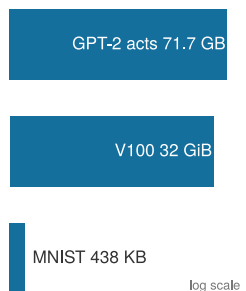
Computationally, backpropagation through a linear layer $\mathbf{Z} = \mathbf{A}\mathbf{W}$ requires two distinct matrix multiplications of identical dimension to the forward pass: one evaluates the activation gradient $\nabla_{\mathbf{A}} \mathcal{L} = (\nabla_{\mathbf{Z}} \mathcal{L}) \mathbf{W}^{\top}$ to propagate the error signal upstream, and the other evaluates the parameter gradient $\nabla_{\mathbf{W}} \mathcal{L} = \mathbf{A}^{\top} (\nabla_{\mathbf{Z}} \mathcal{L})$ to update the weights. Because the backward pass executes two matrix multiplications of identical dimension for every forward matrix multiplication, backward propagation requires twice the arithmetic work of the forward pass. This establishes the standard systems convention that one complete training step requires approximately three times the forward-pass FLOPs ($1 \times$ forward $+ 2 \times$ backward).

A simple three-layer network processing MNIST requires kilobytes of activation storage. In contrast, GPT-2 processing a single batch requires over 35.9 GB—surpassing the total high-bandwidth memory capacity of a standard accelerator. Section C.3.3 derives how backpropagation drives these memory costs, formalizing the full training memory equation ($M_{\text{total}} = M_{\text{weights}} + M_{\text{gradients}} + M_{\text{optimizer}} + M_{\text{activations}}$). Modern ML frameworks automate gradient evaluation through automatic differentiation (chapter 7), but managing the resulting memory footprint and arithmetic schedule remains an unavoidable systems engineering responsibility.

8.3.3.1 Activation memory requirements

Training memory partitions into four distinct state categories:

- saved forward activations required by backward derivative rules,
- model parameters,
- parameter gradients, and
- optimizer state and any higher-precision master weights.



Activation memory spans MNIST toys to GPT-scale training.

Consider a batch of training examples passing through a network. The forward pass computes and stores:

$$\begin{aligned}\mathbf{Z}^{(\ell)} &= \mathbf{A}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)} \\ \mathbf{A}^{(\ell)} &= f(\mathbf{Z}^{(\ell)})\end{aligned}$$

To compute the parameter gradient $\nabla_{\mathbf{W}^{(\ell)}}\mathcal{L} = (\mathbf{A}^{(\ell-1)})^\top (\nabla_{\mathbf{Z}^{(\ell)}}\mathcal{L})$, the backward rule must read the layer's input activations $\mathbf{A}^{(\ell-1)}$. To propagate the error signal upstream through the nonlinear activation function via $\nabla_{\mathbf{Z}^{(\ell)}}\mathcal{L} = (\nabla_{\mathbf{A}^{(\ell)}}\mathcal{L}) \odot f'(\mathbf{Z}^{(\ell)})$, the kernel requires either the pre-activation tensor $\mathbf{Z}^{(\ell)}$ (as in GELU) or the post-activation output $\mathbf{A}^{(\ell)}$ (when f' can be computed from the output, as in sigmoid, or via a 1-bit boolean mask in ReLU). Fused kernels and selective recomputation can compress this saved footprint. Nevertheless, because activations scale with batch size B and sequence length S , their storage demands grow with the data dimension, whereas parameter and optimizer memory remain fixed by the model architecture. The GPT-2 calculation in 8.3 decomposes this memory pressure across attention blocks, feed-forward sublayers, and optimizer states.

Napkin Math 8.3: GPT-2 activation memory breakdown

For GPT-2 with batch size $B = 4$, sequence length $S = 1024$, hidden dimension $d = 1600$, and 48 layers:

Math:

- Attention activations: $B \times S \times d \times 4$ (Q, K, V, output) = $4 \times 1024 \times 1600 \times 4 \times 2$ bytes (FP16) = 52.4 MB
- FFN activations: $B \times S \times (4d)$ (intermediate expansion) = $4 \times 1024 \times 6400 \times 2$ bytes = 52.4 MB
- Attention scores: $5 \times N_{\text{heads}} \times S^2 \times B$ bytes for the $S \times S$ score, softmax, and dropout buffers. With 25 heads, this term reaches 524.3 MB per layer—the dominant term, quadratic in sequence length, and exactly what selective recomputation discards
- Layer norm states: Minimal (~10 MB per layer)
- Total per layer: ~639.1 MB (attention + FFN + attention scores + layer norm states)

Result:

- Total activation memory (library estimate, includes residual-stream and framework buffers beyond the line items above): 35.9 GB
- Parameters (FP16): 3 GB
- Gradients: 3 GB
- Master weights (FP32): 6 GB
- Adam moment state (FP32): 12 GB
- Peak memory during training: ~59.9 GB

This exceeds a single V100's 32 GB capacity.

Solutions:

1. Selective activation recomputation: Discard attention intermediates and recompute them during backward, reducing this modeled activation term by 70.2 percent to ~10.7 GB; the additional work depends on the implementation
2. Activation CPU offloading: Store some activations in CPU RAM, transfer during backward pass
3. Mixed precision: FP16 activations (already applied) vs. FP32 (would be 71.7 GB)
4. Reduced micro-batch size and gradient accumulation: Build a larger effective batch from multiple memory-feasible forward/backward passes

Systems insight: Even after selective recomputation, the modeled activations plus static state total about 34.7 GB, slightly above the raw 32 GB V100 capacity before workspace and allocator

overhead. Full recomputation, sharding, offload, or a smaller micro-batch is therefore still required for this representation.

This breakdown illustrates the practical engineering decisions required when accelerator memory falls short. The same trade-off between stored activations, recomputation, and batch size drives the memory-computation analysis that follows.



Checkpoint 8.2: The memory-compute trade-off

Training large models requires managing the memory wall (the bandwidth bottleneck introduced in chapter 5 and revisited in section 7.3.1).

Bottleneck

- ☐ **Activation memory:** Do you understand why activations stored for backpropagation can dominate memory usage and, for some model shapes and training recipes, exceed parameter size by more than $10\times$?
- ☐ **Optimization strategy:** Can you explain how gradient checkpointing trades compute (recalculating activations) for memory capacity?
- ☐ **Memory estimate:** for GPT-2 (1.5B parameters) in mixed precision, put a one-significant-figure number on each term in equation 8.6: weights, gradients, optimizer state, and activations at batch size 32. Which term is largest, and would the total fit a 32 GB accelerator?

Scaling limits

- ☐ **Batch size constraints:** why does memory capacity limit the maximum batch size, and how does gradient accumulation solve this without increasing memory?

8.3.3.2 Memory-computation trade-offs

Training systems must balance memory usage against computational efficiency. Each forward pass generates values that backward rules may request. For a simplified network model with N_L layers, let s_ℓ denote the bytes of saved intermediate state per example at layer ℓ and a_ℓ the bytes of saved activation outputs per example. Under this per-example convention, their storage scales linearly with batch size and is approximated by equation 8.5:

$$\text{Memory per batch} = B \times \sum_{\ell=1}^{N_L} (s_\ell + a_\ell) \quad (8.5)$$

This memory requirement compounds with the weights, gradients, and optimizer memory discussed in section 8.3.2.3. Equation 8.6 gives a conceptual training-state decomposition:

$$\text{Total Memory} = \text{Memory}_{\text{weights}} + \text{Memory}_{\text{gradients}} + \text{Memory}_{\text{optimizer}} + \text{Memory per batch} \quad (8.6)$$

The runtime peak can exceed this sum because temporary workspaces, communication buffers, allocator behavior, and framework bookkeeping also occupy memory. To reduce the activation term, training systems can strategically recompute intermediate values during backward rather than storing them. This increases computational work but can enable deeper networks or larger batches on memory-constrained hardware (T. Chen et al. 2016). Algorithm 8.2 makes the trade explicit: the forward pass keeps activations at only a sparse set of checkpoint layers, and the backward pass recomputes the rest on demand.

Algorithm 8.2 Gradient checkpointing (activation recomputation)**Require:** N_L -layer network; checkpoint set $\mathcal{C} \subseteq \{1, \dots, N_L\}$ (e.g., every $\sqrt{N_L}$ layers)**Ensure:** parameter gradients, at reduced peak activation memory

```

1: for  $\ell = 1$  to  $N_L$  do
2:   forward layer  $\ell$ ; store its activation only if  $\ell \in \mathcal{C} \triangleright$  drop the rest
3: end for
4: for  $\ell = N_L$  down to 1 do
5:   if the activation of layer  $\ell$  was dropped then
6:     recompute the forward segment from the nearest stored checkpoint through layer  $\ell$ 
7:   end if
8:   compute the layer's gradient; free the activation
9: end for
10: return the accumulated gradients

```

Spacing checkpoints uniformly every $\sqrt{N_L}$ layers reduces peak activation memory from $\mathcal{O}(N_L)$ to $\mathcal{O}(\sqrt{N_L})$ at the cost of one additional forward pass of arithmetic per training step (T. Chen et al. 2016). This increases total step computation from roughly $3\times$ to $4\times$ the forward-pass FLOPs, trading extra arithmetic operations for a substantially reduced activation footprint that fits within physical accelerator memory.

Capacity is only one of two physical constraints imposed by memory; the other is bandwidth. Accelerators provide massive matrix compute throughput, but shuttling intermediate activation tensors between off-chip High Bandwidth Memory (HBM) and on-chip SRAM frequently saturates the memory bus (Jouppi et al. 2017). Even when activations fit comfortably within HBM capacity, the time required to read and write them can dwarf the arithmetic execution time. To relieve this memory bus pressure, modern training engines fuse elementwise operations directly into preceding matrix multiplications and stage gradient computations through on-chip SRAM tiles rather than global memory. At the runtime level, framework caching allocators track tensor lifetimes across the computation graph, recycling memory blocks as soon as an activation's final backward consumer completes (Paszke et al. 2019).

Forward propagation, gradient computation, and parameter updates define *what* training systems must compute. An operation's name alone, however, does not reveal *where* the system stalls. Matrix shape, operator fusion, numerical precision, and the memory hierarchy determine whether arithmetic throughput or data movement binds execution. Arithmetic intensity provides the analytical tool to quantify that balance.

8.3.4 Arithmetic intensity

Arithmetic intensity captures this distinction—the ratio of computation to data movement that reveals whether an operation is limited by compute throughput or memory bandwidth:

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{bytes moved}}$$

Operations with high arithmetic intensity are compute bound: their performance is limited by the processor's computational throughput. Operations with low arithmetic intensity are memory bound: they spend more time moving data than computing. Section D.2.1 gives the formal definition of the Roofline Model and shows how to compute a hardware's ridge point.¹⁷

Table 8.5 summarizes common tendencies rather than fixed measurements. Dense matrix multiplication can reuse operands enough to reach high arithmetic intensity, while standalone elementwise and reduction kernels often perform less work per byte moved. The target hardware's ridge point determines the resulting regime.

¹⁷ | **Ridge point and precision:** The roofline ridge point—the arithmetic intensity threshold separating memory-bound from compute-bound operations—shifts with numerical precision. On the same accelerator, a lower-precision Tensor Core path can expose much more arithmetic throughput at roughly the same memory bandwidth, raising the ridge point substantially. Switching from TF32-style execution to BF16 mixed precision can therefore change which optimization technique yields returns, making precision selection inseparable from roofline analysis.

Table 8.5: Training Operation Classification Conditions: Operation families exhibit common arithmetic-intensity tendencies, but classification requires comparing the measured implementation with the target hardware’s ridge point. Memory-sensitive kernels may benefit from reducing traffic or increasing fusion, while compute-bound kernels benefit from additional arithmetic throughput.

Operation	Arithmetic-intensity tendency	Classification condition
Dense MatMul (large)	High when shapes and tiling provide substantial operand reuse	Compute-bound only when intensity exceeds the ridge
Activation functions	Usually low as standalone elementwise kernels	Often memory-sensitive unless fusion removes traffic
LayerNorm/BatchNorm	Reduction and elementwise work with multiple tensor passes	Depends on fusion, shape, and target ridge point
Attention softmax	Reduction and normalization over the attention axis	Depends on materialization, tiling, and fusion

Hardware efficiency during model training depends directly on kernel arithmetic intensity. Locate the operation points in figure 8.2 along the logarithmic arithmetic intensity axis, comparing memory-bandwidth bound operators on the sloped ceiling against compute-bound operators on the flat peak.

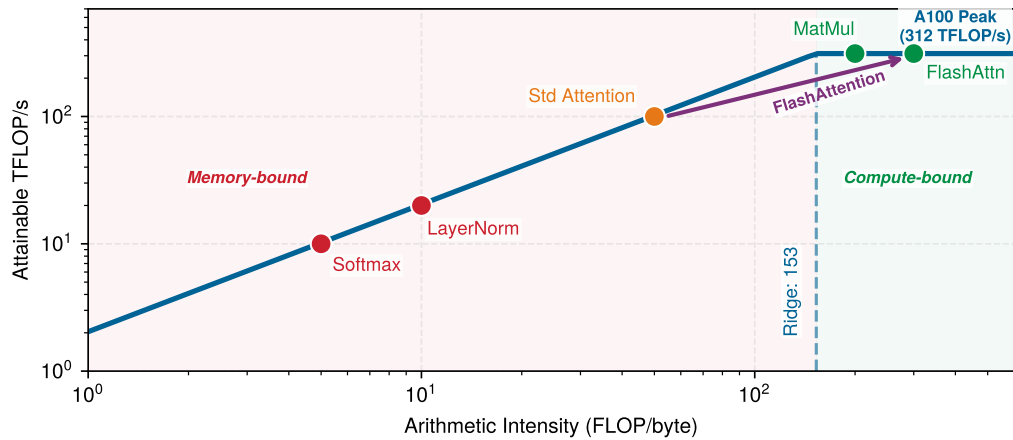


Figure 8.2: Illustrative Training Roofline: An A100 roofline with assumed operation coordinates illustrates how arithmetic intensity changes the attainable-performance ceiling. The points are not a profile of one GPT-2 execution. FlashAttention is shown moving right because it reduces HBM traffic; whether a real attention kernel crosses the ridge depends on shape, precision, hardware, and implementation.

To build intuition for these relationships, study the roofline diagram in figure 8.2. The ridge point marks the “knee” where the sloped memory-bound region meets the flat compute-bound ceiling. An implementation plotted left of this point is bandwidth bound under the roofline assumptions; one plotted right is compute bound. The operation points in this diagram are illustrative placements that show how to interpret the model, not measured GPT-2 coordinates.

Consider a GPT-2 attention layer that materializes the $S \times S$ attention-score matrix. For each head, the $\mathbf{QK}^\top$ product costs approximately $2S^2d_{\text{head}}$ FLOPs, while writing the FP16 score matrix and reading it back moves about $4S^2$ bytes. Counting only that score-matrix traffic gives the approximation $d_{\text{head}}/2$. For GPT-2 Small ($d_{\text{model}} = 768$ across 12 heads, so $d_{\text{head}} = 64$), this restricted accounting yields 32 FLOP/byte. A complete kernel analysis must also count Q and K traffic, softmax work, additional reads and writes, cache reuse, and fusion. The $\mathcal{O}(S^2)$ materialized score traffic remains the term that FlashAttention targets.

Accelerators have characteristic hardware ridge points where operations transition from memory-bound to compute bound. A representative data-center accelerator has a ridge point high enough that low-intensity operations such as materialized attention softmax remain memory bound even

when large matrix multiplications are compute bound. Operations below the ridge point are memory bound; above it, they are compute bound.

Systems Perspective 8.5: Peak FLOP/s vs. sustained performance

Peak TFLOP/s is an arithmetic ceiling rather than a prediction of sustained application performance. For an implementation that is strictly bandwidth bound, raising only the arithmetic ceiling does not improve the roofline bound. Mixed-precision training can both enable faster arithmetic paths and reduce traffic for tensors actually stored and transferred at lower precision. The realized gain depends on kernel support, conversion overhead, numerical requirements, and which resource is binding. Roofline-guided optimization therefore measures the implementation before choosing fusion, precision changes, or additional compute.

Batch size can influence arithmetic intensity and occupancy by changing matrix shapes and the amount of reuse available to a kernel. No universal boundary at batch 32 separates memory-bound from compute-bound execution; the transition depends on the operation, dimensions, implementation, precision, and hardware.

This analysis guides optimization strategy selection. For memory-bound operations, reducing data movement through operator fusion, reduced precision, or algorithmic improvements like FlashAttention provides the largest gains. For compute-bound operations, increasing throughput through Tensor Cores and parallel execution matters more. The distinction is practical: the first case asks how to move fewer bytes, while the second asks how to keep more arithmetic units busy.

In figure 8.2, FlashAttention¹⁸ captures the core insight of IO-aware algorithm design. By never materializing the full $S \times S$ attention matrix in HBM and instead processing tiles that fit in fast SRAM, FlashAttention reduces auxiliary attention memory from $\mathcal{O}(S^2)$ to $\mathcal{O}(S)$ and sharply reduces HBM traffic. The original work reports up to $3\times$ speedups on evaluated workloads (T. Dao et al. 2022), with gains depending on tensor shape, hardware, and baseline. The algorithm and the conditions under which it applies are examined in detail in section 8.6.4.

Arithmetic-intensity analysis determines which physical resource constrains a particular implementation. Dense matrix multiplication reaches the compute-bound regime when tensor shapes provide sufficient data reuse, while standalone normalization and elementwise activation kernels remain memory bandwidth bound. FlashAttention exemplifies how tiling reduces off-chip data movement and raises effective arithmetic intensity, shifting memory-bound attention toward compute saturation.

Optimizing individual operations is necessary but insufficient. A perfectly tuned matrix multiplication achieves nothing if the accelerator sits idle waiting for the next batch of data. Isolated operation costs—dense matrix multiplication FLOPs, memory-bound activations, and optimizer state allocations—compound within the execution loop. Sustained throughput requires orchestrating these discrete operations into a balanced pipeline where upstream data delivery and downstream parameter updates prevent accelerator starvation.

8.4 Pipeline Architecture

A training step is not a single operation but a sequence of dependent stages—data must be loaded before computation can begin, forward passes must complete before backward passes start, and gradients must be computed before parameters can update. The speed of the slowest stage determines the speed of the entire system. Without pipelining, these stages serialize, idling the accelerator while the host prepares data or transfers batches across the bus.

While model execution graphs (chapter 7) govern arithmetic on the accelerator, sustained training requires feeding those kernels across physical machine boundaries. Host CPUs ingest raw records from storage and apply preprocessing in host DRAM, while accelerators execute matrix multiplications in HBM. Data must cross the interconnect—typically a PCIe link whose bandwidth is an order of magnitude lower than device memory bandwidth. When upstream data preparation or bus transfer exceeds the duration of the forward and backward passes, the accelerator compute engines starve, reducing the hardware utilization factor η_{hw} in the iron law of training performance.

¹⁸ | **FlashAttention:** The core mechanism processes attention in small tiles that fit within the accelerator's fast on-chip SRAM, avoiding writes of the full intermediate $S \times S$ matrix to slower HBM. The exact HBM IO bound depends on tile size and SRAM capacity, but the practical effect is a large reduction in memory traffic and auxiliary activation storage. The original work reports workload-dependent speedups of up to $3\times$.

Managing these physical boundaries divides the training system into three interconnected subsystems, each placing distinct demands on hardware resources. The data pipeline handles ingestion, transformation, and batching on the host; the training loop coordinates forward passes, backward passes, and parameter updates on the accelerator; and the evaluation pipeline periodically assesses model quality on held-out data. Figure 8.3 traces how these subsystems interact across the training lifecycle. The interfaces between them form critical synchronization boundaries where I/O latency, interconnect transfers, or synchronous validation pauses can throttle accelerator utilization.

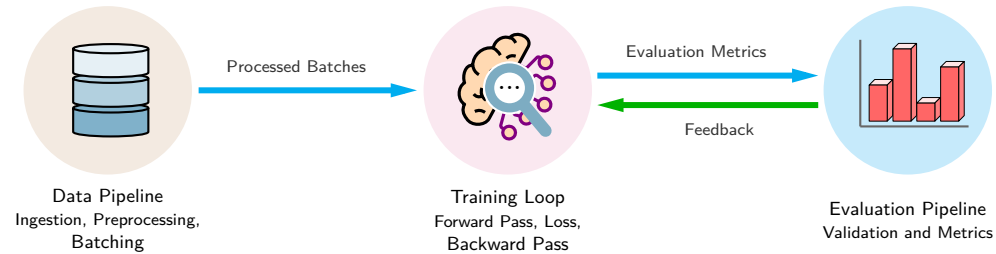


Figure 8.3: Training System Overview: The training lifecycle coordinates data preparation, core forward/backward computation, and validation evaluation. The interfaces between these three pipelines represent critical synchronization boundaries where I/O delays or evaluation pauses can throttle accelerator utilization.

8.4.1 Architectural overview

An end-to-end training system coordinates three distinct subsystems: a host-side data pipeline that prepares input batches, an accelerator execution loop that runs the mathematical optimization step, and a periodic evaluation pipeline that tracks model quality on validation data. These subsystems execute across different hardware domains. The data pipeline consumes host CPU cores and storage bandwidth; the execution loop occupies accelerator compute cores and device memory; and evaluation periodically preempts training to verify generalization. Any latency imbalance among these components translates directly into idle accelerator cycles.

At the ingestion boundary, the data pipeline stages raw records from persistent storage into host memory, applies transformations such as augmentation and normalization (LeCun et al. 2012), and packs tensors into dense batches. These batches transfer over the host-accelerator interconnect (such as PCIe) into accelerator memory. At the validation boundary, the evaluation pipeline periodically runs held-out data through the model without computing gradients. Because validation occupies the same accelerator cores needed for training, its invocation frequency represents a direct trade-off between monitoring resolution and overall training throughput.

Within the accelerator, the training iteration forms a closed feedback loop governed by strict data and memory dependencies. The cyclic data path in figure 8.4 maps these sequential operations: the forward pass, gradient calculation through backpropagation, and the parameter update.

While section 8.3 formalizes the numerical operators of this cycle, physical hardware execution is dictated by memory residency and data dependencies. During the forward pass, intermediate activation tensors must remain resident in accelerator HBM because the backward pass requires them to evaluate chain-rule derivatives. Once gradients are computed, the optimizer reads both the parameters and accumulated gradients to apply the update rule, producing new weights for the subsequent iteration. The memory capacity required to hold activations, weights, and optimizer states establishes the memory floor for the entire training step.

This execution loop repeats across batches and epochs until the optimization converges. Maximizing accelerator arithmetic efficiency requires overlapping these pipeline stages across the system hierarchy. Host CPU threads prefetch and stage batch $k + 1$ into pinned memory while the accelerator executes batch k , hiding storage latency and PCIe transfer time behind matrix-multiplication compute kernels. When this concurrency breaks down—whether an I/O-bound data pipeline starves the device of input tensors or frequent evaluation stalls the update loop—the accelerator idles, degrading effective throughput.

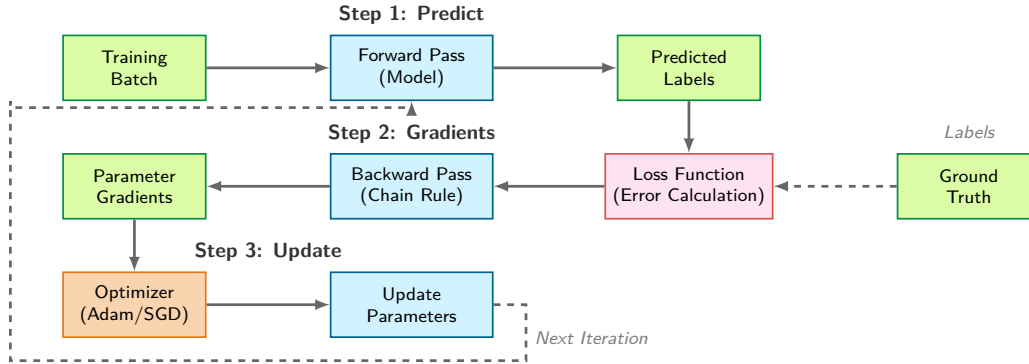


Figure 8.4: The Iterative Training Loop: Each iteration sequences forward prediction, gradient computation by backpropagation, and an optimizer update. Updated parameters return to the next forward pass while the next batch remains a separate input. Data dependencies and memory residency set the step’s minimum execution time and memory floor.

8.4.2 Data pipeline

Keeping an accelerator’s arithmetic units saturated requires streaming training examples from persistent storage into device memory at or above the model’s consumption rate. While upstream data engineering (chapter 4) prepares and partitions datasets on storage media, the runtime data pipeline executes concurrently with model computation on the host CPU.

The data pipeline bridges raw storage and accelerator execution across the machine boundary. Figure 8.5 illustrates this movement across three physical zones.

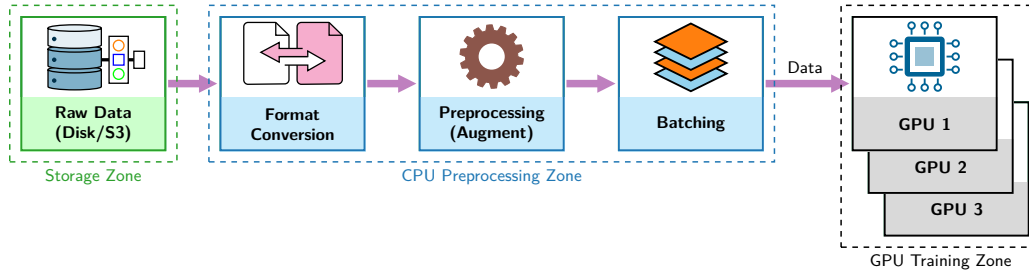


Figure 8.5: CPU-to-GPU Data Flow: Training data transitions through storage, CPU preprocessing, and GPU execution zones. Each stage represents a potential throughput gate; disk read bandwidth, transformation latency, and PCIe host-to-device transfers must collectively sustain the GPU consumption rate to keep accelerators at peak utilization.

In the storage zone, raw records reside on local NVMe drives or networked storage systems. In the CPU preprocessing zone, host worker threads decode raw bytes, apply transformations such as tokenization or data augmentation, and collate individual records into dense mini-batch tensors. In the GPU training zone, accelerators transfer those tensors across the PCIe bus and execute forward and backward passes. Because these three zones form a strict pipeline, the slowest stage dictates end-to-end training throughput.

8.4.2.1 Core components

Data pipeline throughput originates at the storage subsystem. Upstream layout decisions from chapter 4—such as sequential chunking in columnar or container formats (Parquet, TFRecord, Arrow)—dictate the physical I/O patterns presented to the operating system.

Storage throughput is bounded by the slower of two hardware constraints, expressed in equation 8.7:

$$R_{\text{storage}} = \min(BW_{\text{disk}}, BW_{\text{network}}) \quad (8.7)$$

where BW_{disk} is the physical disk bandwidth and BW_{network} represents the network bandwidth for distributed storage systems. In practice, training workloads rarely achieve this theoretical maximum

because data shuffling can turn large sequential reads into smaller, less efficient accesses. Effective storage throughput can be modeled as $R_{\text{storage,eff}} = R_{\text{storage}} \times F_{\text{access}}$, where F_{access} depends on record size, file layout, caching, and storage hardware. When unbuffered random reads degrade F_{access} , storage bandwidth drops by an order of magnitude, transforming an I/O system that appears sufficient on paper into a primary training bottleneck.

8.4.2.2 Preprocessing

Once retrieved from storage, raw bytes require CPU transformation into model-ready tensors. While static feature extraction can run offline (chapter 4), dynamic transformations such as on-the-fly tokenization and image augmentation execute during training. Under ideal independent-worker scaling, preprocessing throughput grows with worker count as expressed in equation 8.8:

$$R_{\text{preprocessing}} = \frac{N_{\text{workers}}}{T_{\text{transform}}} \quad (8.8)$$

where N_{workers} parallel processing threads each perform transformations requiring $T_{\text{transform}}$ seconds. Training architectures employ multiple workers to ensure preprocessing keeps pace with accelerator consumption rates—a single thread performing image augmentation at 30 ms per batch cannot feed an accelerator that computes a forward pass in 10 ms.

Preprocessed data must then transfer to the accelerator before computation can begin. The overall training throughput is therefore constrained by the slowest of three stages, as equation 8.9 makes explicit:

$$R_{\text{training}} = \min \left(R_{\text{preprocessing}}, \frac{\text{BW}_{\text{GPU transfer}}}{D_{\text{vol}}^{(\text{batch})}}, \frac{R_{\text{compute}}}{O_{\text{batch}}} \right) \quad (8.9)$$

where $D_{\text{vol}}^{(\text{batch})}$ is the input bytes per batch and O_{batch} is the forward-backward work per batch in FLOPs. Every term is therefore expressed in batches per second before the minimum is taken.



Example 8.1: GPT-2 language model data pipeline

Scenario: Quantifying storage access, CPU tokenization, and PCIe host-to-device transfers supplying a 32-V100 GPU cluster training GPT-2.

Stage trace:

1. **Raw text storage:** The WebText corpus contributes about 40 GB of raw text. Sequential reads from NVMe SSD storage reach 7 GB/s. Under document sampling with access factor 0.1, random-access storage bandwidth drops to 0.70 GB/s, giving $T_{\text{read}} = D_{\text{vol}} / \text{BW}_{\text{storage}} \approx 0.1$ ms.
2. **Tokenization:** A BPE tokenizer (50,257 vocabulary) converts text into token IDs. For batch size $B = 32$ and sequence length $S = 1024$ (32.8K tokens total), a single CPU core processing 500K tokens/s takes $T_{\text{tokenize}} \approx 65.5$ ms per batch.
3. **Batching and padding:** Padding and packing generate an int64 tensor of shape $32 \text{ sequences} \times 1,024 \text{ tokens}$, producing 262.1 KB per batch.
4. **PCIe transfer:** Transferring the tensor across a PCIe Gen3 x16 bus (15.75 GB/s bandwidth) takes $T_{\text{transfer}} = 0.017$ ms, making bus transfer negligible.

Systems insight: Single-core CPU preparation ($T_{\text{prep}} \approx 65.6$ ms) consumes most of the accelerator step budget ($T_{\text{step}} \approx 84.4$ ms), so a serial pipeline pays both costs on every step and η_{hw} falls to about 56 percent. Push preparation any slower and the accelerator starves outright. Scaling to 8 parallel worker processes reduces tokenization latency to $T_{\text{tokenize}} / N_{\text{worker}} \approx 8.2$ ms, while prefetching overlaps T_{prep} with T_{step} to restore hardware utilization to 95 percent, delivering 379 global samples per second across 32 V100 GPUs.

This min-of-three relationship governs training pipeline design because system throughput equals bottleneck throughput. An accelerator with 312 TFLOP/s of compute capacity delivers zero useful

work while waiting for data. Conversely, a perfectly optimized data pipeline provides no benefit if the accelerator is already compute-saturated. Balanced pipeline design aligns preprocessing capacity, transfer bandwidth, and compute throughput so that no single stage dominates iteration time. The GPT-2 workload demonstrates that CPU tokenization, rather than storage retrieval or bus transfer, forms the primary throughput barrier unless parallel workers and prefetching conceal host preparation latency.

Data pipeline throughput is not the only flow that can starve the accelerator. Multi-GPU training adds a second stream of traffic: the gradient synchronization required after every step. Once synchronization time exceeds compute time, communication becomes the limiting factor. Section 8.7.2 quantifies that network wall at the point where training crosses the node boundary.

8.4.2.3 System implications

The data pipeline and compute engine form a coupled system whose throughput equals the slower of the two, as equation 8.10 states:

$$R_{\text{system}} = \min(R_{\text{pipeline}}, R_{\text{compute}}) \quad (8.10)$$

When $R_{\text{pipeline}} < R_{\text{compute}}$, the accelerator sits idle waiting for data, and accelerator utilization drops proportionally, as equation 8.11 shows:

$$\text{Accelerator Utilization} = \min\left(1, \frac{R_{\text{pipeline}}}{R_{\text{compute}}}\right) \times 100\% \quad (8.11)$$

A ResNet-50 model on modern accelerator hardware can process 1,000 images per second, but if the data pipeline delivers only 200 images per second, accelerator utilization drops to 20 percent—the accelerator is idle 80 percent of the time. Upgrading to faster hardware does not help in this case; an accelerator capable of 2,000 images per second would achieve only 10 percent utilization with the same pipeline. Balanced system design aims to keep the accelerator supplied with data rather than waiting on an upstream stage.

8.4.2.4 Data flows

Training data traverses three memory tiers on its way from disk to accelerator, and the bandwidth gap between these tiers, spanning three orders of magnitude, is the central challenge of data pipeline design. The effective transfer rate through the hierarchy is bounded by its slowest link, as equation 8.12 shows:

$$R_{\text{memory}} = \min(\text{BW}_{\text{storage}}, \text{BW}_{\text{system}}, \text{BW}_{\text{accelerator}}) \quad (8.12)$$

Local NVMe storage provides 7 GB/s, system memory delivers 50 GB/s, and accelerator HBM achieves 900 GB/s or higher. Each tier is roughly an order of magnitude faster than the one below it, which means data that flows freely within accelerator memory creates a severe bottleneck when it must be fetched from disk. This cascading bandwidth hierarchy explains why the iteration time of a well-pipelined system is governed by the *maximum* of its component latencies rather than their *sum*, as equation 8.13 shows:

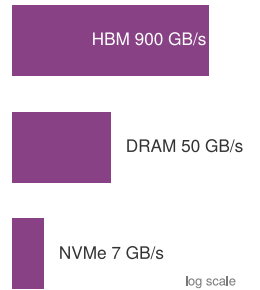
$$T_{\text{iteration}} = \max(T_{\text{fetch}}, T_{\text{process}}, T_{\text{transfer}}) \quad (8.13)$$

When pipeline stages overlap correctly (fetching the next batch from storage while preprocessing the current one and transferring the previous one to the accelerator), the iteration time equals the duration of the slowest stage rather than the sum of all stages. This overlap is exactly what prefetching achieves, turning a serial bottleneck into a parallel pipeline where each tier operates concurrently on different batches.

8.4.2.5 Practical architectures

An NVMe device rated at 7 GB/s would sustain 3.50 GB/s if a workload achieved half of peak. Small random reads and shuffling can reduce throughput further; the penalty depends on access size, queue depth, file layout, and caching.

To keep accelerators fed despite this bandwidth reduction, pipeline architectures maintain multiple data buffers simultaneously—prefetch buffers loading future batches, processing buffers holding



Bandwidth steps up the storage to DRAM to HBM hierarchy.

data under transformation, and transfer buffers staging data for accelerator consumption. The total host memory required scales with the per-batch memory footprint M_{batch} according to equation 8.14:

$$M_{\text{required}} = (N_{\text{prefetch}} + N_{\text{processing}} + N_{\text{transfer}}) \times M_{\text{batch}} \quad (8.14)$$

To avoid starving the accelerator, average preprocessing service time per batch must satisfy equation 8.15:

$$T_{\text{preprocessing}} \leq T_{\text{compute}} \quad (8.15)$$

If preprocessing takes 40 ms per batch on one worker while the accelerator consumes one every 10 ms, ideal scaling requires at least four workers. Actual requirements depend on storage, CPU contention, and transformation cost; section 8.6.2 examines prefetching and parallel workers.

8.4.3 Forward pass

Prepared batches enter the training loop through the forward pass, where input data propagates through the model to generate predictions. The conceptual flow follows the layer-by-layer transformation $\mathbf{A}^{(\ell)} = f(\mathbf{A}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)})$, but the physical implementation must schedule compute kernels, stream activations through the memory hierarchy, and retain intermediate tensors for backpropagation.

8.4.3.1 Compute operations

The forward pass maps neural network layers to the matrix multiplication kernels analyzed in section 8.3.1.2. For each dense layer, the accelerator executes $N \times M \times B$ multiply-accumulate operations. For example, a layer with dimensions of 512×1024 processing a batch of 64 samples executes about 33.6 million MACs, or about 67.1 MFLOP under the two-FLOPs-per-MAC convention.

🔍 Systems Perspective 8.6: Wave quantization and tail effects

A common mistake in ML systems is ignoring the fixed execution granularity of GPU kernels. GPU execution is quantized into waves of work, but the relevant unit is the kernel's thread or tile mapping, not necessarily one sample per thread.

An NVIDIA GPU executes work in warps of 32 threads. In the illustrative special case where one independent work item maps to one thread, 32 items fill one warp while 33 items require a second, mostly empty warp. Neural-network kernels usually map each sample to many threads, so batch size alone does not determine this utilization.

On a large GPU such as the NVIDIA H100 with 132 Streaming Multiprocessors (SMs), where each SM schedules groups of warps, the hardware processes thousands of threads in a single wave. If the total workload slightly exceeds a wave boundary (for example, 1.01 waves), the hardware must execute a second, nearly empty wave to completion before the next task begins. Table 8.6 quantifies that one-item-per-thread illustration. It is not a general batch-size performance model.

Table 8.6: Illustrative Warp-Tail Arithmetic: Under a one-work-item-per-thread mapping, counts just above a 32-thread boundary launch a partially filled warp, so 33 items use two warps while 64 fill the same two warps. Actual training kernels map tensors to threads and tiles in implementation-specific ways, often assigning many threads to each sample, so these values cannot be inferred from batch size alone. Profiling must identify the kernel’s real mapping and wave boundaries.

Independent Work Items	Warps Needed	Lane Utilization	Relative Time
32	1	100%	1×
33	2	51.6%	~2×
64	2	100%	1×
65	3	67.7%	~1.5×

Systems insight: Align tensor and tile dimensions with the kernel’s hardware mapping, then benchmark candidate batch sizes. Profiling reveals partially filled warps or waves; batch size alone cannot.

Beyond dense matrix multiplication, distinct neural architectures introduce specialized computational patterns. Convolutional networks, for instance, slide spatial filters across multichannel input tensors. Consider an input tensor of dimensions $64 \times 224 \times 224 \times 3$ (batch size $\times$ height $\times$ width $\times$ channels) processed by 7×7 kernels. Each position requires 147 multiply-accumulate operations, and with 64 filters operating across 218×218 spatial dimensions, the arithmetic work scales into millions of multiply-accumulate operations per image. Accelerators execute these spatial convolutions by either lowering them into dense GEMMs through column unfolding (`im2col`) or applying Winograd minimal filtering to trade off-chip memory traffic for scratchpad storage.

Transformer architectures introduce attention mechanisms (chapter 6), which compute similarity scores across sequences. Attention pairs batch matrix multiplications (QK^T and $\text{Softmax}(\dots)V$) with row-wise reductions. Because intermediate attention score matrices scale quadratically with sequence length, materializing these activations in global memory creates a memory bandwidth bottleneck unless kernels fuse matrix multiplications and normalization within on-chip SRAM.

Throughout these networks, elementwise operations exhibit low arithmetic intensity. Standalone implementations of activation functions such as ReLU and sigmoid transform tensor elements independently, leaving them bound by memory bandwidth rather than compute throughput (section 8.3.4). Batch normalization compounds this memory traffic by computing statistics across batch and spatial dimensions, requiring global synchronization barriers and multiple read/write sweeps over activation memory.

Accelerators execute these parallel workloads by grouping execution threads into fixed-size units: warps¹⁹ of 32 threads on NVIDIA architectures, or wavefronts of 64 threads on AMD architectures. Hardware execution efficiency depends on aligning tensor dimensions and tile shapes with the thread-block geometry and the operand matrix dimensions required by specialized units like Tensor Cores.

Optimized operator libraries such as cuDNN (Chetlur et al. 2014) implement multiple computational algorithms for each layer type, dynamically selecting among them based on tensor dimensions, scratchpad memory limits, and hardware capabilities. For example, cuDNN chooses between direct convolution, GEMM-based lowering, fast Fourier transform (FFT) convolution, and Winograd transforms by evaluating the trade-off between workspace memory overhead and arithmetic operations (NVIDIA Corporation 2023). These hardware utilization patterns reinforce the coupling between batch size and hardware efficiency established in section 8.3.2.1: larger batch sizes saturate accelerator compute units and amortize memory transfers, but increase intermediate activation storage.

8.4.3.2 Memory management

Memory management binds during the forward pass, when intermediate activations must be retained in high-bandwidth memory for gradient computation during backpropagation. To see how memory capacity constrains training, consider the memory footprint of training a 7B-parameter

¹⁹ | **Warp:** From textile weaving, where a “warp” is the set of threads held taut on a loom while a shuttle weaves across them. NVIDIA adopted the term because GPU threads within a warp execute the same instruction in lockstep, analogous to parallel threads moving together on the loom. An NVIDIA warp contains 32 threads.

model on a 24 GB accelerator, accounting for weights, gradients, optimizer state, and retained activations.



Napkin Math 8.4: Estimating VRAM requirements

Problem: Will a 7B-parameter model fit on a 24 GB GPU for training?

Given: 7B parameters, FP16 weights and gradients, FP32 master weights and Adam states, and 24 GB GPU memory.

Math:

1. **Weights (FP16):** $7\text{B} \times 2 \text{ bytes} = 14 \text{ GB}$.
2. **Gradients (FP16):** Same size as weights = 14 GB.
3. **FP32 training state:** Master weights require 28 GB; Adam momentum and variance require another 56 GB.
4. **Subtotal (before activations):** $14 \text{ GB} + 14 \text{ GB} + 28 \text{ GB} + 56 \text{ GB} = 112 \text{ GB}$, already exceeding a 24 GB GPU.
5. **Activations:** A simplified transformer estimate is $\text{Batch} \times \text{SeqLen} \times \text{Hidden} \times \text{Layers} \times \text{Bytes} \times \text{an activation factor for retained attention, multilayer perceptron, and normalization intermediates}$. With $\text{Batch} = 1$, $\text{Seq} = 2048$, $\text{Hidden} = 4096$, 32 layers, and an activation factor of about $57\times$, retained activations add 30.6 GB.

Systems insight: Full-parameter training requires partitioning or offloading state through systems such as fully sharded data parallel (FSDP) or ZeRO. Quantized base weights reduce parameter-efficient fine-tuning memory but do not reproduce this full-parameter mixed-precision recipe.

Only activation memory scales approximately linearly with batch size (equation 8.5); model parameters and optimizer state remain fixed. The practical complexity lies in how these costs interact across layers.

Consider ResNet-50 processing images at 224×224 resolution with a batch size of 32. The initial convolutional layer produces activation maps of dimension $112 \times 112 \times 64$; per image at single-precision (4 bytes), this requires approximately 3.2 MB. Across 50 layers, intermediate activation memory accumulates: the complete forward pass activations total approximately 8 GB, the backward pass adds another 4 GB of activation working set (empirical working memory, distinct from stored parameter gradients), and model parameters consume 102.4 MB. This 12.1 GB total represents about 15.1 percent of an A100 GPU's 80 GB memory capacity for a single batch.

Doubling the batch size to 64 doubles activation storage: forward activations reach 16 GB and the backward activation working set reaches 8 GB, pushing total memory to 24.1 GB. In vision models with large spatial resolutions, activations dominate the memory footprint while parameter storage (102.4 MB) is negligible. For large language models, the balance inverts: GPT-3 (175B parameters) requires approximately 700 GB just for parameters in FP32 (350 GB in FP16), exceeding single-accelerator capacity before allocating any activation memory or optimizer state.

Because accelerator memory (40 GB–80 GB on contemporary hardware) must hold activations, parameters, gradients, and optimizer states simultaneously, memory capacity rather than compute throughput frequently caps the feasible batch size. Two system-level techniques relieve this capacity bottleneck: activation checkpointing discards intermediate activations during the forward pass and recomputes them during the backward pass, trading redundant arithmetic for reduced storage; and mixed-precision training halves memory traffic and storage per value by adopting 16-bit floating-point formats. Both techniques are analyzed in detail in section 8.6.

During backpropagation (section 8.3.3), the backward pass traverses the network in reverse order, consuming retained forward activations to compute parameter and activation gradients at roughly twice the arithmetic cost of the forward pass. Memory requirements shift dynamically across layers. In ResNet-50, mid-network layers expand to 256 filters from the initial 64, where reduced spatial map dimensions partly compensate for higher channel depth. Across 50 layers, the backward-pass activation working set reaches approximately 3.2 GB before accounting for parameter gradients and

optimizer state. Peak memory footprint reflects retained forward activations, active gradient buffers, temporary workspaces, and the runtime allocation schedule rather than the width of a single layer alone.

8.4.4 Parameter updates and optimizers

Once the backward pass populates gradient buffers, the system executes the optimizer step to update model parameters. The choice of optimizer determines both the mathematical trajectory toward the loss minimum and the hardware resources demanded by each iteration. In standard deep learning frameworks, this phase couples gradient evaluation with the parameter update (listing 8.3): `loss.backward()` traverses the computational graph in reverse to populate gradient tensors, and `optimizer.step()` applies the update rule across all registered parameters.

Listing 8.3: Parameter Update: Computes gradients and applies an optimizer step to model parameters. Repeating this cycle seeks to minimize the training objective, although improvement is not guaranteed on every step or epoch.

```
loss.backward() # Compute gradients
optimizer.step() # Update parameters
```

Although `optimizer.step()` appears as a single framework call, it initiates a memory-bound workload across accelerator memory. Unlike the forward and backward passes, where GEMMs provide high arithmetic intensity by repeatedly reusing tiled data in registers and shared memory, the optimizer step consists of low-intensity elementwise arithmetic. For every parameter, the accelerator must stream weights, gradients, and auxiliary optimizer states from HBM into on-chip registers, compute the update rule, and write the modified values back to HBM. Because each transferred byte experiences only a few floating-point operations, arithmetic intensity remains well below one FLOP per byte, leaving execution time governed by memory bandwidth rather than compute throughput.

8.4.4.1 Optimizer memory in the training loop

This memory bandwidth bottleneck compounds with the auxiliary state requirements quantified in table 8.4. While standard stochastic gradient descent requires no persistent state beyond weights and gradients, adaptive optimizers demand substantial auxiliary storage. For Adam, maintaining running estimates of the first moment (momentum) and second moment (uncentered variance) requires two additional FP32 state tensors for every parameter. During each update, the accelerator must read and write these state buffers across the memory bus, multiplying memory traffic and consuming capacity that could otherwise accommodate larger batch sizes or longer sequences.

At billion-parameter scale, optimizer state dominates the memory budget. As quantified in the GPT-2 worked example (section 8.3.2.3), a 1.5B model requires 12 GB for Adam optimizer state alone in FP32, in addition to parameters and gradients, before accounting for activations. This challenge has motivated memory-efficient optimizer variants: Adafactor factorizes second-moment state (Shazeer and Stern 2018), 8-bit optimizers quantize optimizer statistics (Dettmers et al. 2022), and GaLore computes updates in a low-rank space (Zhao et al. 2024). Figure 8.6 compares memory allocations for a 7-billion-parameter model across four optimizer configurations, showing how low-rank projection reduces optimizer state to a fraction of its standard footprint.

BF16 Adam extends farthest beyond the single-accelerator 24 GB budget line, while Adafactor, 8-bit Adam, and 8-bit GaLore progressively compress the auxiliary state. Yet even the most compact configuration exceeds 24 GB, demonstrating that optimizer compression alone cannot eliminate the need for activation management or model partitioning when fitting large models onto fixed accelerators.

8.4.4.2 Batch size and parameter updates

Increasing batch size improves accelerator utilization by raising the arithmetic intensity of matrix multiplications and reducing synchronization frequency. However, modifying batch size alters optimization dynamics. Scaling batch size without retuning the learning-rate schedule can degrade convergence, creating an optimization gap where hardware efficiency comes at the expense of model quality. Figure 8.7 illustrates this trade-off: maintaining a fixed learning rate while expanding batch size slows loss reduction, whereas scaling the learning rate restores convergence.

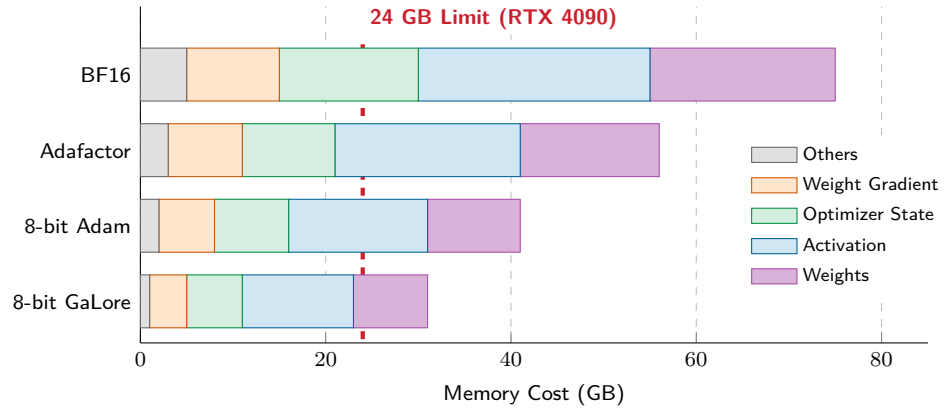


Figure 8.6: Illustrative Memory Footprint Breakdown: Memory usage of a 7-billion-parameter LLaMA model across four memory-efficient optimizer configurations (BF16 Adam, Adafactor, 8-bit Adam, and 8-bit GaLore), decomposed into weights, activations, optimizer state, weight gradients, and other components. The dashed red line marks the RTX 4090 24 GB memory limit. Standard FP32 Adam (omitted from the bars; it requires well above 70 GB for this model) does not fit on a single 24 GB GPU; the bars compare relative memory reductions, but even the smallest illustrated total remains above the 24 GB line, with 8-bit GaLore shrinking optimizer state most aggressively.

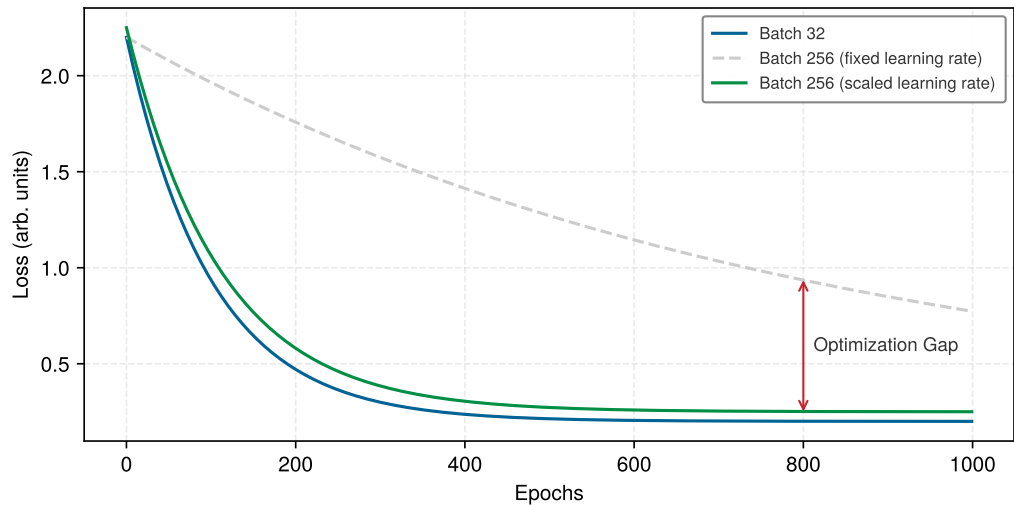


Figure 8.7: Illustrative Linear-Scaling Scenario: Synthetic training-loss curves (arbitrary units) compare a batch-32 baseline with batch 256 under a fixed or linearly scaled learning rate. The curves illustrate why increasing batch size may require retuning; they are not measured traces or a guarantee that linear scaling preserves convergence.

For a fixed token budget, doubling the global batch size halves the number of parameter updates per epoch. To traverse a comparable distance in parameter space with fewer updates, the **linear scaling rule** scales the learning rate proportionally with batch size ($\eta_{\text{new}} = k \times \eta_{\text{base}}$). Although effective across moderate batch ranges when combined with learning-rate warmup, linear scaling does not guarantee equivalent convergence or generalization. Beyond a critical batch size, gradient noise is largely eliminated, and further batch expansion yields diminishing returns per training sample while escalating memory pressure.

Beyond convergence dynamics, batch size dictates communication frequency in distributed systems. A larger global batch reduces the number of optimizer steps and gradient synchronizations per epoch. Because the communication payload of dense all-reduce synchronizations depends on parameter count rather than batch size, larger batches amortize interconnect latency over more compute FLOPs. In distributed settings, local micro-batch size and global batch size jointly determine

the maximum degree of data parallelism. When memory capacity constrains the local micro-batch, gradient accumulation (section 8.6.5) decouples the effective batch size from per-device activation memory, accumulating gradients across multiple forward-backward passes before triggering an optimizer step.

Every architectural decision in training—batch sizing, precision, memory management, and synchronization frequency—ultimately aims to minimize wall-clock time on expensive accelerator fleets. At frontier scale, that time is denominated directly in capital. Compute expenditure becomes a binding physical and economic constraint that governs hardware procurement and cluster sizing. Notebook 8.5 translates these throughput calculations into rental-versus-purchase economics for a 70-billion-parameter language model trained across a 1,024-accelerator cluster.



Napkin Math 8.5: The utility bill

Problem: Under the following simplified assumptions, is it cheaper to rent or purchase a 1,024-H100 cluster for one Llama 2 70-billion-parameter training run?

Math:

1. Workload: Llama 2 70B model (70B parameters, 2T tokens).
2. **Compute required:** $6 \times 70 \times 10^9 \times 2 \times 10^{12} \approx 8.4 \times 10^{23}$ FLOPs. The leading factor of 6 follows from the chapter's accounting: about 2 FLOPs per parameter per token in the forward pass (two FLOPs per multiply-accumulate), tripled once the backward pass, which costs roughly twice the forward pass, is included.
3. Hardware: NVIDIA H100 (Peak: 989 TFLOP/s FP16). Assumed Utilization: 50 percent (494.5 TFLOP/s).
4. Time: $8.4 \times 10^{23} / (494.5 \times 10^{12}) \approx 1.70 \times 10^9$ seconds $\approx$ 53.8 years (on one GPU).
5. **Cluster:** On 1,024 GPUs $\rightarrow$ 19.2 days.

The economics:

- **Rental (\$3/hr):** $1,024 \text{ GPUs} \times 24 \text{ hrs} \times 19.2 \text{ days} \times \$3/\text{hr} \approx \$1.42\text{M}$.
- **Purchase (\$30,000 per GPU):** $1,024 \text{ GPUs} \times \$30,000 = \$30.7\text{M}$.

Systems insight: In this simplified utilization-and-price scenario, purchase cost equals about 21.7 rental runs. A real ownership comparison must also include cluster networking, host systems, facilities, power, staffing, depreciation, financing, and the utilization achieved between runs.

Decomposing the training pipeline exposes the structural mechanics of data loading, forward execution, backpropagation, and parameter updates, while the mathematical foundations in section 8.3 quantify the FLOPs, memory footprint, and bandwidth demanded by each phase. Yet knowing how stages execute does not reveal where a specific training workload stalls. A pipeline is only as fast as its slowest stage: if data loading takes 50 ms and computation takes 100 ms, accelerating computation by 20 percent saves 20 ms, but if the bottleneck shifts to data ingestion, those same optimization efforts yield zero wall-clock speedup. Before applying optimization techniques, systems engineers must profile the execution timeline to determine whether compute, memory bandwidth, storage I/O, or network synchronization binds performance.

8.5 Identifying Bottlenecks

Blueprint knowledge is not diagnosis. Knowing that attention operations consume 50 percent of FLOPs and data loading takes 25 percent of wall-clock time does not reveal which constraint to attack first; that depends on which resource is actually saturated during execution.

The diagnostic methodology that transforms blueprint knowledge into actionable optimization decisions begins with a meaningful measure of training efficiency. Raw accelerator utilization percentages can be misleading because an accelerator may remain active while executing recomputation, padding, or other work outside the model-FLOP accounting. Model FLOPs utilization (MFU)²⁰ supplies that comparison.

²⁰ | **Model FLOPs utilization (MFU):** The PaLM paper (Chowdhery et al. 2022) defined MFU as observed throughput divided by the theoretical maximum throughput at peak FLOPs. Its 540-billion-parameter run reported 46.2 percent MFU on 6,144 TPU v4 chips. The complement is uncredited by the model-FLOP numerator, but MFU alone does not identify how much came from memory traffic, communication, pipeline bubbles, recomputation, or other work.



Definition 8.4: Model FLOPs utilization (MFU)

Model FLOPs utilization (MFU) is the efficiency metric $MFU = O_{\text{model}} / (R_{\text{peak}} \cdot T_{\text{step}})$, where O_{model} is the useful per-step model FLOP count (forward plus backward, excluding rematerialization) and T_{step} is the measured wall-clock time per training step, expressing what fraction of peak hardware throughput is doing useful model computation.

1. **Significance:** MFU makes the effective-throughput term in the iron law concrete. For a 7B-parameter transformer processing 1,024 tokens per step on an A100 (312 TFLOP/s FP16/BF16 Tensor Core peak), a 1.2-second step gives model FLOPs divided by the available peak-rate FLOP budget of approximately 0.11 (11.5 percent). The remaining 88.5 percent is not credited as model FLOPs and can reflect memory traffic, communication, bubbles, non-model operations, or idle time. Whether a value is good depends on the model, hardware, precision, and parallelization strategy; diagnosis requires a profile rather than a universal threshold.
2. **Distinction:** Hardware utilization reports whether device engines are active, whereas MFU credits the analytical model-FLOP estimate used in its numerator. It can therefore exclude recomputation and padding even when those operations keep the device busy. Comparisons across accelerators remain sensitive to the selected peak-throughput specification and FLOP-counting convention.
3. **Common pitfall:** A frequent misconception is that 100 percent MFU is a realistic sustained target. Memory traffic, communication, and non-matrix work keep end-to-end runs below peak, but no universal 55–65 percent ceiling exists. Reported values depend on the model, precision, batch shape, hardware, and MFU definition, and can improve with FlashAttention and careful tuning.

Training bottlenecks can be investigated through three categories aligned with the D·A·M taxonomy (Data, Algorithm, Machine; Appendix A provides the full diagnostic framework, troubleshooting matrix, and D·A·M Scorecard). Table 8.7 connects each axis to a common training bottleneck, its observable symptoms, and relevant optimization techniques. These are diagnostic starting points, not exclusive mappings.

Table 8.7: D·A·M Taxonomy Applied to Training Bottlenecks: Each row associates a D·A·M axis with one common bottleneck and its characteristic symptoms. Profiling reveals the workload’s limiting constraint and guides practitioners to an appropriate optimization technique.

D·A·M Axis	Bottleneck	Symptoms	Primary Solutions
Algorithm	Compute-bound	High arithmetic-unit activity; additional bandwidth does not improve throughput; arithmetic throughput is limiting	Reduced precision, more efficient algorithms, faster compute hardware
Machine	Memory-bound	High memory-bandwidth use relative to arithmetic throughput; kernels stall on data movement	Operator fusion, memory-efficient attention, reduced precision formats
Data	Data-bound	Periodic accelerator utilization drops to near-zero; CPU fully busy during gaps; pipeline cannot feed GPU fast enough	Prefetching, pipeline overlap, faster storage, DataLoader parallelism

Among these three regimes, data-bound stalls are the most frequently misdiagnosed because they manifest on the accelerator as idle gaps rather than saturated execution units. When host-side ingestion, decoding, or preprocessing cannot sustain the throughput demanded by device memory buses, the accelerator starves.

 Example 8.2: The GIL-locked GPU

Scenario: An engineering team builds a PyTorch data-loading pipeline using standard Python threading for CPU-bound image preprocessing that holds the global interpreter lock (GIL).

Diagnosis: The GIL serializes these Python-level preprocessing threads, making host preparation time ($T_{\text{prep}} \approx 250$ ms) vastly exceed GPU execution time ($T_{\text{step}} \approx 12$ ms), collapsing hardware utilization to $\eta_{\text{hw}} \approx 4.6\%$.

Systems lesson: Python CPU work that holds the GIL can serialize threads and starve GPUs; native operations may release it. Process workers (`num_workers > 0`) avoid interpreter serialization, while NVIDIA DALI can move eligible work off the host.

Profiling tools reveal which bottleneck dominates a given workload. Figure 8.8 captures the data-bound pathology from the callout: white gaps between repeated compute blocks mark intervals with no GPU kernels visible in the device timeline.

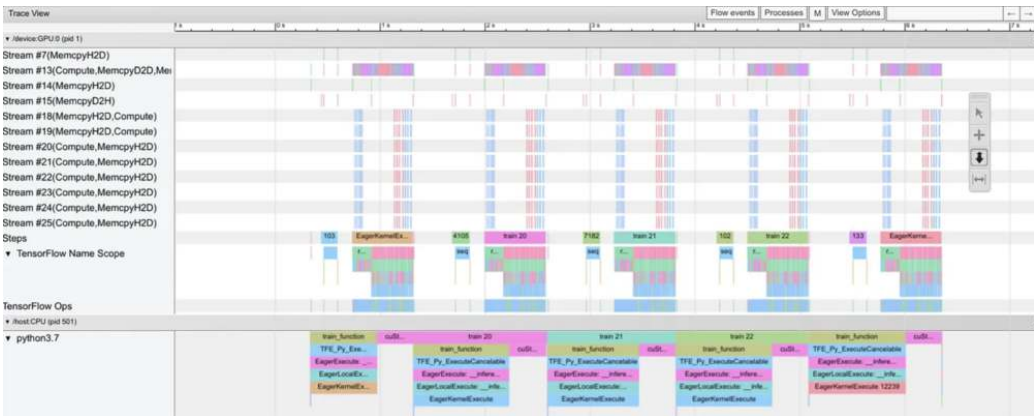


Figure 8.8: Data-Bound Profiler Trace: In this timeline trace, isolated GPU compute blocks are separated by prominent white intervals. These idle gaps mark periods where accelerator streams stall waiting for CPU-side batch preparation and host-to-device transfers, providing a characteristic signature of an input-bound pipeline.

Four tools integrated into machine learning frameworks provide detailed bottleneck analysis. Table 8.8 separates framework-level timeline tools from GPU-level execution tools, because each exposes a different failure signature.

Table 8.8: Profiling Tools for Training Bottlenecks: Framework profilers expose operation timelines and input-pipeline stalls, while NVIDIA Nsight tools expose system-level GPU execution and kernel-level memory behavior.

Tool	Scope	Best signal
PyTorch Profiler (<code>torch.profiler</code>)	Framework-level operation trace	Time spent in each operation, memory allocation patterns, and GPU kernel execution
TensorFlow Profiler	Framework-level training timeline	Input pipeline bottlenecks and device placement
NVIDIA Nsight Systems	System-level GPU trace	Kernel execution, memory transfers, and synchronization points
NVIDIA Nsight Compute	Kernel-level GPU analysis	Arithmetic intensity, memory throughput, and occupancy

A disciplined profiling workflow proceeds top-down from system timelines to kernel-level performance counters, mapping observable profiler traces to the D·A·M taxonomy in table 8.7. First, timeline visualization isolates input starvation: idle intervals between kernel invocations indicate that host preparation or PCIe transfer latency is starving device queues. Resolving this data-bound constraint requires asynchronous prefetching, multi-process workers, or offloading preprocessing to the accelerator to overlap data movement with compute. Second, when kernels execute contiguously

without host-induced gaps, memory throughput relative to peak bandwidth distinguishes memory-bound from compute-bound execution. If HBM traffic saturates while arithmetic units idle, the workload is memory-bound; operator fusion and memory-efficient attention relieve this constraint by retaining intermediate tensors in on-chip SRAM rather than writing them back to global memory. Finally, if tensor cores operate near theoretical capacity and memory buses have spare bandwidth, the workload is compute-bound, where throughput gains require lower numerical precision—such as BF16 or FP8—to increase peak arithmetic throughput.

8.6 Pipeline Optimizations

Profiling traces identify which resource saturates during execution, while the D·A·M taxonomy (Appendix A) isolates the governing physical constraint: data movement, arithmetic throughput, or memory capacity. Closing the gap between realized and theoretical throughput requires matching targeted systems interventions to each identified failure mode.

Even well-designed pipeline architectures rarely reach hardware peak throughput without targeted optimization. To illustrate the scale of the gap, applying a model-FLOP utilization of 30 percent to 50 percent to an A100 peak of 312 TFLOP/s gives 93.6 TFLOP/s–156 TFLOP/s of model-FLOP throughput. These are derived scenario values, not measurements reported for one A100 workload. Published large-model systems report similar gaps between peak and realized throughput (Narayanan et al. 2021; Chowdhery et al. 2022). Systems such as SuperNeurons show one version of this problem: memory-management bottlenecks can prevent larger networks from training efficiently unless the runtime actively optimizes activation storage and transfer (L. Wang et al. 2018). Table 8.9 extends the D·A·M-based bottleneck classification from table 8.7 by mapping each bottleneck to the specific optimization technique that addresses it.

Table 8.9: Optimization Technique Roadmap: Each primary bottleneck category has targeted solutions that address specific performance constraints, matching techniques to profiling results for systematic optimization.

Bottleneck	Primary Solution(s)
Data Movement Latency	Prefetching & Pipeline Overlapping
Compute Throughput	Mixed-Precision Training
Memory Capacity	Gradient Accumulation & Activation Checkpointing
Memory Bandwidth (Attn.)	FlashAttention (IO-aware tiling)

Although the governing constraint shifts with system scale—a 100 GB model faces capacity and interconnect limits that never bind a 1 GB model—identifying and mitigating each bottleneck follows consistent physical principles. Data movement latency emerges when host storage I/O and PCIe transfers cannot stage batches fast enough to keep accelerator compute units saturated. Computational throughput falls below peak hardware rates when operations execute in FP32 instead of using lower-precision Tensor Cores, or when non-fused kernels repeatedly transfer intermediate activations across the memory bus. Memory capacity constraints bind execution because accelerator HBM must simultaneously hold model parameters, optimizer states, gradients, and forward activations. Exhausting this memory caps both model depth and per-device batch size.

Because system resources are tightly coupled, relieving one constraint often exposes another, illustrating the conservation-of-complexity heuristic introduced earlier in this book. For example, resolving host I/O stalls by aggressively prefetching batches consumes additional accelerator memory for input queues. Conversely, shrinking per-device batch sizes to fit within HBM limits lowers the arithmetic intensity of matrix multiplications, shifting execution from compute-bound matrix math into memory-bandwidth stalls. Consider a representative GPT-2 profile where attention consumes 50 percent of step time, data loading consumes 25 percent, and other compute-bound operations consume 25 percent. This breakdown directs the optimization sequence: IO-aware attention tiling (FlashAttention) targets the dominant memory-bandwidth bottleneck in attention, asynchronous prefetching hides data ingestion behind computation, and mixed precision accelerates tensor operations while halving activation footprint. The optimization challenge is to diagnose the governing bottleneck from execution profiles and apply targeted interventions without creating a more severe constraint elsewhere.

8.6.1 Systematic optimization framework

Because accelerator compute engines, memory subsystems, and host interconnects operate under tight physical coupling, optimizing without empirical telemetry risks shifting the critical path or squandering engineering effort on non-binding resources. A principled systems methodology follows three coupled stages: profiling to expose the binding physical constraint, selecting mechanisms specifically engineered for that resource limit, and composing interventions so that secondary resource demands do not collide.

The profiling phase uses instrumentation tools such as PyTorch Profiler, TensorFlow Profiler, or NVIDIA Nsight Systems to record timestamps and hardware counters across each iteration. Traces capture kernel execution times on accelerator execution units, direct memory access (DMA) transfers across PCIe or NVLink buses, and host CPU thread scheduling. Quantifying elapsed time isolates which subsystem dictates the critical path. For example, a trace revealing 40 percent of step time spent in data loading, 35 percent in computation, and 25 percent in memory operations diagnoses host-side data starvation as the primary constraint. Under these conditions, accelerating matrix multiply kernels yields negligible wall-clock improvement until the data delivery pipeline is resolved.

The selection phase maps the identified bottleneck to an architectural mechanism targeting that physical constraint. When profiling shows input-bound execution where the accelerator idles waiting for batches, asynchronous prefetching and double-buffering over PCIe overlap host data ingestion with device compute. When kernel runtimes are memory-bandwidth-bound—dominated by round-trips to HBM rather than arithmetic—operator fusion and tiling algorithms such as FlashAttention stage intermediate activations in on-chip SRAM. When memory capacity bounds the model, activation checkpointing discards and recomputes activations during the backward pass, or gradient accumulation divides batches across multiple sequential sub-steps to limit peak allocation. Finally, when streaming multiprocessors saturate with dense matrix math, low-precision numeric formats (such as FP16 or BF16) exploit specialized tensor cores to double computational throughput while halving memory traffic.

The composition phase evaluates how multiple interventions interact across shared hardware resources. Because system resources are tightly coupled, an optimization that relieves one bottleneck often imposes demands elsewhere. Asynchronous prefetching hides data movement latency behind computation, but maintaining double-buffered batch queues consumes pinned host memory and device HBM. In contrast, synergistic combinations compound performance gains: pairing 16-bit mixed precision with activation checkpointing curtails memory usage sufficiently to allow larger per-device batch sizes, restoring high arithmetic intensity to underutilized tensor cores. Successful composition therefore requires verifying that secondary resource overheads do not trigger a new binding constraint.

Figure 8.9 operationalizes this framework into a structured decision tree. The branches map profiling signals directly to candidate interventions, ensuring engineering effort addresses the binding physical bottleneck rather than incidental overhead.

Because eliminating one bottleneck exposes the next constraint along the execution path, systems optimization is an iterative process governed by Amdahl's Law. Resolving input starvation through asynchronous prefetching, for instance, drives accelerator utilization upward until memory bandwidth or compute saturation limits step time. Each intervention requires re-profiling to identify the newly emerged critical path, continuing the cycle until accelerator execution units operate near peak theoretical throughput or application performance targets are achieved.

8.6.2 Data prefetching and overlapping

Prefetching and overlapping techniques illustrate the systematic framework in action, targeting data movement latency bottlenecks by coordinating data transfer with computation. This optimization proves most effective when profiling reveals that computational units remain idle while waiting for data transfers to complete.

Training machine learning models involves significant data movement between storage, memory, and computational units. The data pipeline consists of sequential transfers: from disk storage to CPU memory, CPU memory to GPU memory, and through the GPU processing units. In ML training, a "Read" operation is rarely a simple disk fetch: for image workloads it encompasses JPEG

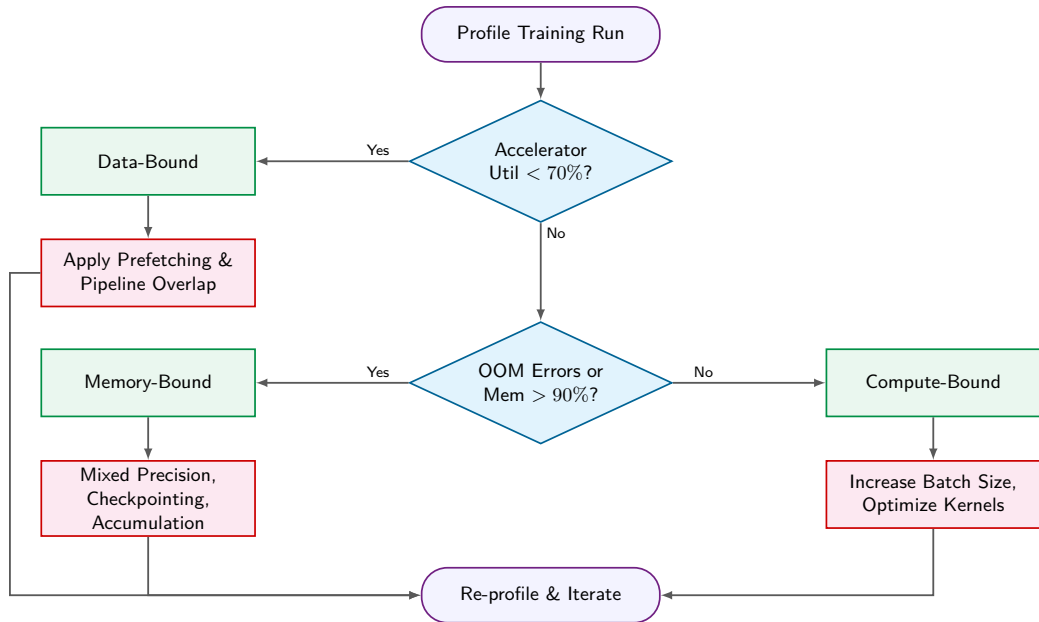


Figure 8.9: Training Optimization Decision Flowchart: A heuristic path from profiling signals to candidate optimizations. Begin with accelerator utilization, then distinguish memory pressure from compute saturation before selecting an intervention. The 70 percent utilization and 90 percent memory thresholds are illustrative triage values rather than universal definitions of data-, memory-, or compute-bound execution.

decoding, random crops, and color jitter applied on CPU cores before the tensor is ready to transfer; for language workloads it encompasses tokenization, subword encoding, and sequence padding. Figure 8.10 exposes the inefficiency of sequential data transfer: the GPU remains idle during file operations (Open 1, Open 2), and training steps cannot begin until these read and preprocessing operations complete, leaving expensive compute resources underutilized for significant portions of each epoch.

Prefetching addresses these inefficiencies by loading data into memory before its scheduled computation time. During the processing of the current batch, framework data pipelines such as `tf.data` load and prepare subsequent batches, maintaining a consistent supply of ready data (Murray et al. 2021).

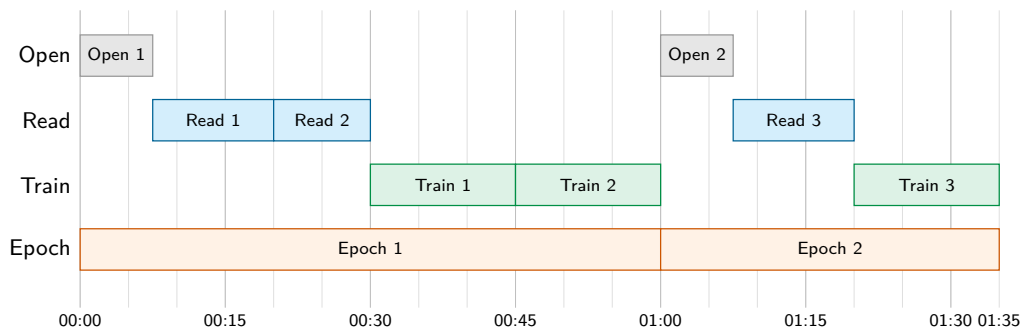


Figure 8.10: Illustrative Sequential Data Fetching: Under the assumed stage durations, file-open, read, and train operations execute serially while the GPU remains idle during file operations. The schedule spans approximately 95 min and provides a schematic baseline for the overlap comparison.

Overlapping extends prefetching by coordinating multiple pipeline stages to execute concurrently. The system processes the current batch while simultaneously preparing future batches through data loading and preprocessing operations. Compare the illustrative schedules in figure 8.10

and figure 8.11. Under their assumed stage durations, overlap reduces completion time from approximately 95 min to 65 min, a 31.6 percent reduction. Actual gains depend on stage balance and available concurrency.

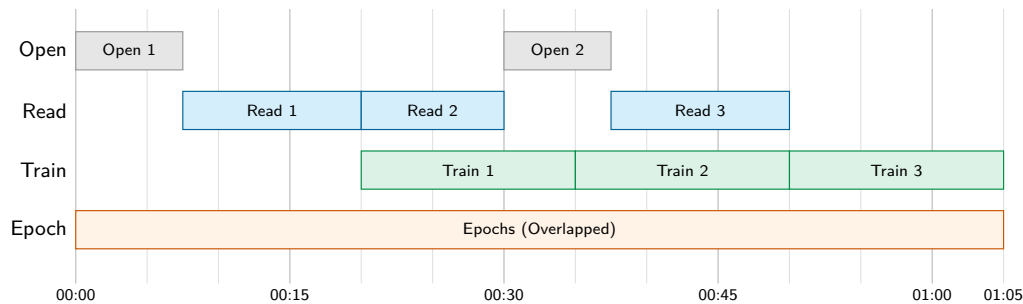


Figure 8.11: Illustrative Overlapped Data Prefetching: File-open and read work overlaps accelerator training while preserving the same individual stage durations as the serial schedule. The schematic schedule spans approximately 65 min, compared with 95 min for the serial schedule.

8.6.2.1 Prefetching mechanics

Prefetching only pays off when the profile shows data movement on the critical path. Training data still passes through retrieval, transformation, and model execution; the optimization changes when those stages run. An unoptimized pipeline serializes them, leaving the GPU idle during data fetching and preprocessing. Data loaders avoid that stall by preparing the next batch in separate threads or processes while the current batch trains.

Overlapping extends the same idea across the full pipeline. As the GPU processes one batch, preprocessing begins on the next batch, while data fetching starts for the subsequent batch. The goal is constant activity at each stage so the slowest stage no longer leaves the others waiting.

Machine learning frameworks (introduced in chapter 7) expose the control variables for this trade-off. Listing 8.4 demonstrates PyTorch’s DataLoader configuration, where `num_workers = 4` sets preprocessing parallelism and `prefetch_factor = 2` maintains a buffer of 8 batches ready for accelerator consumption.

Listing 8.4: DataLoader Prefetch Configuration: Four workers with a prefetch factor of two maintain up to eight batches in the host-side prefetch window.

```
loader = DataLoader(
    dataset, batch_size=32, num_workers=4, prefetch_factor=2
)
```

The parameters `num_workers` and `prefetch_factor` are not generic performance toggles. They determine CPU parallelism and buffer depth, which means they shift pressure from accelerator idle time to host memory and CPU scheduling. Beyond worker concurrency, achieving high host-to-device transfer throughput requires pinned (page-locked) host memory. Standard virtual memory is pageable by the operating system, forcing the GPU driver to stage transfers through an intermediate host bounce buffer before copying over PCIe; setting `pin_memory=True` in the data loader allocates tensors in page-locked physical RAM, allowing the GPU to initiate asynchronous DMA transfers (`tensor.to(device, non_blocking=True)`) that overlap directly with accelerator execution.

Buffer management is the main trade-off. A buffer that is too small causes the GPU to wait for data preparation, reintroducing the idle time prefetching was meant to remove. An overly large buffer consumes memory that could otherwise store model parameters or larger batches. The right configuration coordinates CPU preparation, storage I/O, and GPU computation so each resource has work ready at the moment it can use it. These techniques yield the greatest benefit when storage access is slow, preprocessing is complex, or datasets are large.

8.6.2.2 Practical considerations

Prefetch buffers and overlap depth are tunable, so the same machinery adapts to whichever stage binds, whether slow storage, limited network bandwidth, or computational throughput. Prefetching and overlapping deliver the greatest gains when preprocessing lies on the critical path but can run concurrently with model computation. In an illustrative image pipeline, random cropping (10 ms), color jittering (15 ms), and normalization (5 ms) total 30 ms per batch. Overlap can hide up to the concurrent accelerator-computation time; any excess remains on the critical path. NLP workloads similarly benefit when tokenization and subword processing would otherwise block the training loop.

The primary trade-off is memory: prefetch buffers consume host memory, or device memory when a separate device-side queue is used, in proportion to buffer depth and batch size. With batch size 256 high-resolution images (1024×1024 pixels), one buffered batch requires approximately 3.2 GB. With `num_workers = 4` and `prefetch_factor = 2`, the 8-batch prefetch window can hold about 25.8 GB. Tuning these parameters requires empirical testing because excessive workers contend for CPU, storage, and memory resources, while insufficient buffering reintroduces data stalls. Start with a modest worker count, increase it while measuring throughput and memory, and stop when additional workers no longer reduce accelerator idle time. When the input pipeline already exceeds compute demand, deeper prefetching adds complexity without improving throughput.

8.6.3 Mixed-precision training

While prefetching optimizes data movement, mixed-precision training addresses both computational throughput limitations and memory capacity constraints. It uses a lower-precision format where appropriate while retaining higher precision for selected accumulations or updates. Section D.4 compares FP32, FP16, BF16, FP8, and INT8 precision-range trade-offs. A training recipe commonly combines FP32 with either FP16 or BF16; it does not ordinarily use FP16 and BF16 interchangeably in the same path. The resulting speed, memory, and accuracy effects depend on the workload and hardware (Micikevicius et al. 2017; Wang and Kanwar 2019; Kalamkar et al. 2019).

A neural network’s stored FP32 weights require 4 bytes per parameter, while FP16 and BF16 weights use 2 bytes. For a model with 10^9 parameters, the weight tensor shrinks from 4 GB to 2 GB. Total training memory falls by less when the recipe retains FP32 master weights, optimizer states, or selected activations.

The numerical differences between these formats shape their use cases. Table 8.10 shows that BF16’s 8-bit exponent gives it approximately the same normal exponent range as FP32, while FP16’s 5-bit exponent gives a minimum positive normal value of about 6.1×10^{-5} and a maximum finite value of 65,504. FP16 subnormals extend toward 6×10^{-8} , although their treatment depends on the operation and hardware mode. FP32 provides about seven decimal digits of precision, FP16 about three, and BF16 about two to three. BF16 therefore trades mantissa precision for the exponent range that often simplifies deep-learning training.

Table 8.10: Precision Format Comparison: The choice between FP16 and BF16 depends on whether dynamic range (BF16’s strength) or precision (FP16’s advantage) matters more for the specific workload. Minimum normal values shown are the practical thresholds for training, as subnormal values may flush to zero on many GPUs. On A100, FP16 and BF16 Tensor Cores reach $16\times$ the FP32 CUDA-core peak.

Property	FP32	FP16	BF16
Exponent bits	8	5	8
Mantissa bits	23	10	7
Min normal value	10^{-38}	6.1×10^{-5}	10^{-38}
A100 peak throughput ratio	$1\times$	$16\times$	$16\times$

The choice between formats depends on model characteristics and hardware support. BF16’s wider exponent range reduces overflow and underflow concerns, while FP16 provides more mantissa bits but often requires loss scaling. Framework autocast policies and fused kernels choose higher-precision accumulation for numerically sensitive operations when needed; loss reduction, softmax,

normalization, and optimizer updates are common candidates, but their exact execution dtype is implementation-dependent.

Figure 8.12 traces a conventional FP16 mixed-precision recipe. FP32 master weights are cast for lower-precision computation, the loss is scaled before backpropagation, and gradients are unscaled before the FP32 update. Modern framework implementations may store or accumulate particular tensors differently, and BF16 recipes commonly omit loss scaling because of their wider exponent range.

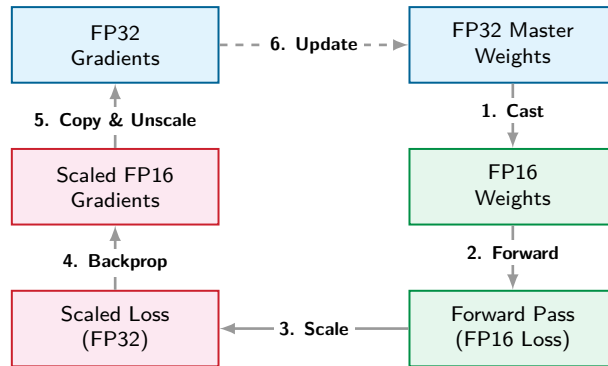


Figure 8.12: Conventional FP16 Mixed-Precision Training: A schematic six-step recipe casts FP32 master weights for lower-precision computation, scales the loss, computes scaled gradients, then unscales them for an FP32 update. Framework implementations may vary, and BF16 usually does not require this loss-scaling path.

Modern accelerator architectures pair dense matrix execution units with reduced-precision numerical formats to maximize arithmetic throughput. NVIDIA introduced Tensor Cores for FP16 operations in the Volta architecture (NVIDIA 2017) and added native BF16 execution in Ampere (NVIDIA Corporation 2020a), matching the native BF16 support Google developed for Tensor Processing Units (Wang and Kanwar 2019). These specialized functional units execute low-precision matrix multiply-accumulate operations at four to sixteen times the rate of standard 32-bit floating-point ALUs.

Mixed-precision training turns those hardware capabilities into a split numerical contract. Eligible matrix multiplications and convolutions execute in FP16 or BF16 on tensor units, while reductions, accumulations, and parameter updates retain higher precision. The performance gain comes from both arithmetic throughput and data movement: lower-precision functional units operate at higher clock-cycle densities, while narrower tensors halve data transfer volumes across caches and high-bandwidth memory. Higher precision remains necessary where wide reductions, small parameter updates, or sensitive normalizations can accumulate rounding error. Framework autocast policies enforce this split dynamically, selecting lower-precision execution for compute-intensive matrix multiplications while preserving FP32 for numerically fragile operations.

8.6.3.1 Loss scaling

FP16’s 5-bit exponent limits its representable dynamic range,²¹ creating severe underflow hazards during backpropagation. When computing gradients across deep networks, successive chain-rule multiplications frequently produce gradient magnitudes smaller than FP16’s minimum positive normal value ($2^{-14} \approx 6.1 \times 10^{-5}$). While IEEE 754 FP16 defines subnormal numbers down to $\approx 6 \times 10^{-8}$, hardware accelerators often flush subnormal inputs and outputs to zero to prevent pipeline stalls. When gradients underflow to zero, weight updates stall and training fails to converge.

Loss scaling counters underflow by exploiting the linearity of differentiation: $\nabla_{\mathbf{w}}(S \cdot \mathcal{L}) = S \cdot \nabla_{\mathbf{w}} \mathcal{L}$. Multiplying the scalar loss $\mathcal{L}$ by a large scaling factor S (such as 2^{10} or 2^{15}) prior to backpropagation shifts the gradient distribution rightward into FP16’s normal representable range. Once backpropagation completes, the accumulated gradients are divided by S (unscaled) before applying the optimizer update.

Deep learning frameworks automate this process through dynamic loss scaling (GradScaler). Because gradient norms fluctuate during training, a static scale factor risks either underflow (if S

²¹ | **FP16 (half-precision floating-point):** Its reduced normal range stems from using 5 exponent bits, compared with 8 in FP32; the smallest positive normal value is about 6.1×10^{-5} , but subnormal values extend to about 6×10^{-8} . Underflow behavior depends on the operation and hardware mode. Loss scaling reduces the chance that small gradients are rounded or flushed to zero.

is too small) or numerical overflow past FP16's maximum finite value of 65,504 (if S is too large). Before executing the parameter update, `GradScaler` unscales the gradients in FP32 and inspects the tensor buffers for nonfinite values (`inf` or `NaN`). If overflow is detected, the scaler skips the optimizer step to protect the FP32 master weights from corruption and reduces the scale factor (typically halving S). If training proceeds without overflow across a configurable growth window (such as 2,000 consecutive steps), the scaler doubles S to maintain gradients near the upper end of the dynamic range without saturating.

8.6.3.2 Mixed-precision benefits

Mixed-precision training delivers gains across three interacting system dimensions: memory footprint, arithmetic throughput, and communication volume. Halving operand width contracts weight storage by 50 percent, dropping a one-billion-parameter model from 4 GB in FP32 down to 2 GB in FP16 or BF16 (though net savings depend on the precision retained for optimizer state and master weights). Computationally, reduced precision routes dense matrix multiplication kernels to high-throughput Tensor Core pipelines (section 8.6.3.3). In distributed topologies, transferring 16-bit rather than 32-bit gradients halves collective AllReduce traffic across the interconnect. These effects compound: reclaimed memory headroom permits larger micro-batches, Tensor Cores accelerate kernel execution, and narrower tensors reduce gradient exchange latency. The following case study evaluates these interacting memory savings on GPT-2 XL, establishing how mixed precision and selective recomputation bring a model that exceeds device memory in FP32 within the capacity of a single accelerator.



Napkin Math 8.6: GPT-2 mixed precision memory savings

Problem: Can GPT-2 XL (1.5B parameters) be trained on a single V100 GPU, and what do mixed precision and gradient checkpointing save in memory footprint?

FP32 baseline:

- Parameters and gradients (FP32): 12 GB (6 GB parameters + 6 GB gradients)
- Activations (batch = 4): ~71.7 GB
- Optimizer states (Adam m , v in FP32): 12 GB
- Total: ~95.7 GB (exceeds any single GPU)

FP16 mixed precision:

- Parameters (FP16): 1.5B parameters at 2 bytes each = 3 GB
- Activations (FP16): ~35.9 GB
- Gradients (FP16): 3 GB
- FP32 master weights: 6 GB (for precise optimizer updates)
- Optimizer states (Adam m , v in FP32): 12 GB
- Total: ~59.9 GB (still tight, but manageable with optimizations)

Mixed precision + illustrative fourfold activation reduction:

- Activations reduced to ~9 GB through selective recomputation
- Estimated tensor total: ~33 GB, compared with 32 GB of V100 capacity, before temporary workspaces and allocator overhead

Systems insight: Mixed precision halves the parameter and gradient tensors in this recipe; checkpointing then trades recomputation for lower activation storage. At batch size 4, the simplified tensor estimate approaches V100 capacity, so an actual run may still require additional memory reduction after accounting for workspaces and allocator behavior. At batch 32, the same estimate requires ~95.7 GB, motivating multi-device scaling techniques.

Memory savings alone do not guarantee stable training. In FP16 training, maintaining FP32 master weights is necessary because parameter updates $\Delta w = -\eta \cdot g$ are typically orders of magnitude

smaller than the weight values w . In FP16, where the 10-bit mantissa provides roughly three decimal digits of precision, adding an update that is more than $2^{11} \approx 2048$ times smaller than the weight magnitude results in arithmetic swamping: the update shifts entirely beyond the mantissa bits and rounds to zero. Retaining master weights in FP32 preserves these incremental updates across thousands of optimizer steps, casting to 16 bits only when staging weights for the forward pass. Autocast policies similarly isolate wide reductions, layer normalizations, and softmax layers in FP32 to prevent catastrophic cancellation.

Once numerical stability is established, reduced-precision arithmetic and reduced memory traffic translate directly into wall-clock speedups and cluster cost reductions. The following scenario models these compound throughput, schedule, and infrastructure cost gains for a distributed GPT-2 XL training run.



Napkin Math 8.7: GPT-2 mixed precision throughput and cost

Problem: Given the following illustrative throughput and billing assumptions for a GPT-2 XL job, what speed and cost differences follow from FP16 mixed precision?

Assumed V100 throughput:

- FP32 throughput: ~90 samples/s
- FP16 throughput: ~220 samples/s
- Speedup: $220 \text{ samples/s} \div 90 \text{ samples/s} \approx 2.4\times$ faster training

Cost impact on a cluster of 32 GPUs:

- Assumed FP32 billing: \$50,000 for 2 weeks
- Assumed FP16 billing: \$28,000 for 1.2 weeks
- Wall-clock gain: $2 \text{ weeks} \div 1.2 \text{ weeks} \approx 1.7\times$, below the $2.4\times$ throughput speedup, since the assumed schedule includes work the FP16 kernels do not accelerate
- Time saved: $2 \text{ weeks} - 1.2 \text{ weeks} = 0.8 \text{ weeks}$ (5.6 days)
- Cost saved: $\$50,000 - \$28,000 = \$22,000$

Quality gate: The cost comparison is valid only if a controlled validation run shows that the FP16 recipe meets the same acceptance criteria as the FP32 baseline. The scenario does not assume a particular perplexity difference.

Systems insight: Under these assumptions, the same cluster of 32 GPUs changes from 2 weeks and \$50,000 to 1.2 weeks and \$28,000. This is scenario arithmetic, not a benchmark claim; a real decision requires measured throughput, billing, and quality validation for the target workload.

Realizing these speedups requires empirical profiling rather than assuming automatic acceleration. For small models or memory-bound workloads not dominated by large matrix multiplications, data type casting and kernel launch overheads can offset the arithmetic throughput advantage. Furthermore, whether an accelerator supports native BF16 or requires FP16 loss scaling depends on the underlying microarchitecture.

8.6.3.3 Mixed-precision hardware support

Specialized matrix execution units deliver the wall-clock speedups of mixed precision by trading numerical precision and dynamic range for arithmetic density.²² Accelerators pair these low-precision multiply units with higher-precision accumulation registers to maintain numerical stability during dense matrix reductions.

NVIDIA introduced Tensor Cores in their Volta architecture (2017) as dedicated matrix multiplication units optimized for mixed-precision workloads. Unlike standard CUDA cores that process scalar or small vector operations, Tensor Cores perform 4×4 matrix multiply-accumulate operations in a single clock cycle. For FP16 inputs, a single Tensor Core executes:

$$\mathbf{D}_{\text{tc}} = \mathbf{A}_{\text{tc}} \times \mathbf{B}_{\text{tc}} + \mathbf{C}_{\text{acc}}$$

²² | **Tensor Core:** A specialized matrix-multiply-accumulate unit that supports reduced-precision inputs and higher-precision accumulation. Supported tile shapes, alignment constraints, and peak-throughput ratios vary by architecture, dtype, sparsity mode, and library kernel.

where $\mathbf{A}_{tc}$ and $\mathbf{B}_{tc}$ are FP16 inputs and $\mathbf{C}_{acc}$ and $\mathbf{D}_{tc}$ may use FP32 accumulation. Higher-precision accumulation reduces rounding error, although it does not eliminate numerical error or cancellation.

Hardware peak specifications define the theoretical ceiling on this acceleration. An NVIDIA A100 GPU exposes 19.5 TFLOP/s of FP32 throughput on standard CUDA cores and 312 TFLOP/s of FP16 or BF16 Tensor Core throughput, a $16\times$ peak ratio. H100-class accelerators add FP8 Tensor Cores at 1,979 TFLOP/s without sparsity, about $2\times$ the H100 FP16/BF16 Tensor Core rate. These figures represent hardware ceilings rather than end-to-end training guarantees: matrix multiplications map directly to Tensor Cores, but data movement, non-Tensor-Core kernels, communication, loss scaling, and optimizer updates remain. While the batched matrix multiplication $\mathbf{QK}^T$ in transformer self-attention ($2 \times B \times N_{heads} \times S^2 \times d_{head}$ FLOPs) maps directly to these tensor units, Amdahl's law dictates that the remaining memory-bound elementwise operations, communication collectives, and optimizer updates bound the realized end-to-end speedup.

Brain Float 16 (BF16) maintains FP32's 8-bit exponent while reducing the mantissa to 7 bits. This design choice prioritizes dynamic range preservation over precision, which matters for gradient-based learning where values span many orders of magnitude. Google's TPUs natively support BF16, while NVIDIA's Ampere architecture (A100) and newer provide full hardware support.

The range advantage of BF16 over FP16 appears when gradients or intermediate values fall outside FP16's normal range. FP16's smallest positive normal value is 6.1×10^{-5} , while its subnormal floor is approximately 6×10^{-8} .²³ BF16's smallest positive normal value is approximately 10^{-38} , matching FP32's exponent range. BF16 therefore usually avoids the loss scaling needed by FP16, although accumulation precision and rounding still matter.

NVIDIA's Hopper architecture (H100) introduces FP8 support with two formats. E4M3 uses four exponent bits and three mantissa bits (prioritizing precision for forward pass weights and activations), while E5M2 uses five exponent bits and two mantissa bits (prioritizing dynamic range for backward pass gradients).

FP8 training doubles Tensor Core throughput again (1.98 PFLOP/s on H100 dense vs. 0.99 PFLOP/s for FP16 dense, without sparsity). However, FP8's severely limited precision requires per-tensor scaling factors maintained in higher precision, adding algorithmic complexity. Table 8.11 summarizes when each precision is appropriate:

Table 8.11: Precision Selection Guide: Format choice depends on hardware support, kernel coverage, numerical behavior, and validation results. FP8 requires an explicit scaling strategy; BF16 provides a wider exponent range than FP16; FP16 often uses loss scaling; and FP32 remains available for sensitive operations or unsupported lower-precision paths.

Precision	When to Use	Hardware Requirement
FP8	Throughput-oriented training with validated scaling	Supported FP8 hardware
BF16	Wide exponent range without FP16 loss scaling	Supported BF16 accelerator
FP16	Higher mantissa precision with managed loss scaling	Supported FP16 accelerator
FP32	Higher precision or unsupported lower-precision path	GPU

Selecting among these formats requires balancing arithmetic throughput against implementation complexity and numerical stability.

In addition to accelerating compute-bound matrix operations, reduced precision relieves memory bandwidth bottlenecks by altering operational arithmetic intensity. On modern accelerators, large matrix multiplications reside in the compute-bound regime of the roofline model, whereas elementwise operations (such as bias additions, layer normalization, and residual connections) reside firmly in the memory-bound regime. An NVIDIA A100 GPU illustrates the physical imbalance between arithmetic capability and memory bandwidth:

- HBM2e bandwidth: 2,039 GB/s
- FP32 throughput (19.5 TFLOP/s): worst-case demand is 78 TB/s if every FLOP needs new data
- Actual requirement (with data reuse): Much lower, but bandwidth-limited for operations with low arithmetic intensity

²³ **FP16 subnormal handling:** IEEE FP16 represents subnormals below 6.1×10^{-5} down to approximately 6×10^{-8} , but some accelerator operations or modes flush subnormal inputs or results to zero. The behavior must be checked for the target hardware and kernel.

For memory-bound kernels, execution time is determined by bytes transferred across the memory bus rather than arithmetic clock cycles. Converting 32-bit operands to 16-bit FP16 or BF16 halves the byte volume transferred across the HBM bus, doubling the theoretical throughput of memory-bound operations even without engaging Tensor Cores. Realized bandwidth gains depend on cache hit rates, kernel fusion, and format conversion costs.

Modern frameworks abstract these microarchitectural mechanisms through automatic precision routing (chapter 7). As shown in listing 8.5, PyTorch’s automatic mixed precision (`torch.amp`) wraps model execution in an autocast context that assigns lower-precision dtypes to eligible compute-intensive kernels while routing reductions and normalizations to FP32. In the FP16 path, a `GradScaler` intercepts loss evaluation and backpropagation to prevent underflow, while a BF16 path omits gradient scaling.

Listing 8.5: Mixed-Precision Training: Autocast selects eligible lower-precision operations, while gradient scaling is enabled for the FP16 path and omitted for BF16.

```
import torch

model = TransformerModel().cuda()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
amp_dtype = (
    torch.float16
) # torch.bfloat16 usually does not need loss scaling
scaler = torch.amp.GradScaler(
    "cuda", enabled=(amp_dtype == torch.float16)
)

for inputs, targets in dataloader:
    optimizer.zero_grad()

    # Automatic precision selection per operation
    with torch.amp.autocast("cuda", dtype=amp_dtype):
        output = model(inputs)
        loss = criterion(output, targets)

    # Scale loss to prevent gradient underflow
    scaler.scale(loss).backward()

    # Unscale gradients before optimizer step
    scaler.step(optimizer)
    scaler.update() # Adjust scaling factor dynamically
```

Because hardware support for low-precision formats evolved across accelerator generations, the optimal mixed-precision recipe depends on the target architecture. Table 8.12 outlines the precision trade-offs across three generations of NVIDIA GPUs.

Table 8.12: Precision Strategy by GPU Architecture: V100 supports FP16 Tensor Core training, A100 adds native BF16, and H100 adds FP8 paths whose use requires recipe-specific validation.

Architecture	Recommended Precision	Key Considerations
V100 (Volta)	FP16 with loss scaling	No native BF16 Tensor Core path; clipping depends on the training recipe
A100 (Ampere)	BF16 or FP16	BF16 avoids FP16 loss scaling; TF32 can accelerate eligible FP32 matrix operations
H100 (Hopper)	BF16, FP16, or validated FP8	FP8 requires an FP8-aware recipe; 1,979 TFLOP/s is a peak ceiling

An illustrative single-GPU GPT-2 scenario (1.5B) demonstrates how architectural execution units and numerical formats compound: 18 samples/s for V100 FP32, 45 samples/s for V100 FP16, 165 samples/s for A100 BF16, and 380 samples/s for H100 FP8. These figures represent scenario

projections rather than isolated benchmarks, as software stacks, kernel implementations, and batch dimensions also evolved across generations. The systems principle is clear: maximizing training throughput requires co-designing the numerical precision format with the accelerator’s native matrix execution pipelines.

8.6.4 FlashAttention: IO-aware attention optimization

Mixed-precision training addresses two bottlenecks: compute throughput, because Tensor Cores operate faster on FP16, and memory capacity, because each value uses fewer bytes. For transformer models during training, however, a third bottleneck often dominates: memory bandwidth. The attention mechanism’s quadratic intermediate matrices must be repeatedly loaded and stored during the forward pass and accessed again during backpropagation. Even with reduced precision, the sheer volume of memory traffic can leave compute units idle while the processor waits for data.

FlashAttention²⁴ (T. Dao et al. 2022) addresses this bandwidth bottleneck by optimizing how data flows between memory hierarchies. Processing attention in SRAM-sized tiles avoids materializing the full $S \times S$ matrix in HBM and reduces traffic without changing the mathematical output. The original work reports up to $3\times$ speedups on evaluated workloads; gains depend on tensor shape, hardware, and the baseline implementation.

8.6.4.1 The standard attention memory bottleneck

Standard self-attention (chapter 6) computes relationships between all positions in a sequence. For an input sequence of length S , the mechanism computes an $S \times S$ attention matrix according to equation 8.16:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} \right) \mathbf{V} \quad (8.16)$$

Here, $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ are the query, key, and value matrices for the sequence, and d_k is the key-vector dimension used to scale the dot products before softmax. The $\mathbf{Q}\mathbf{K}^T$ term is the source of the $S \times S$ score matrix, which is why attention becomes an I/O problem even when the arithmetic itself maps well to Tensor Cores.

The memory bottleneck emerges from materializing the $S \times S$ intermediate matrices for scores and probabilities. For a sequence length of 4,096 with embedding dimension 64 (typical for a single attention head), the attention score matrix alone requires $4,096^2 \times 4 \text{ bytes} = 67.1 \text{ MB}$ in FP32. With 16 heads, this grows to 1.1 GB just for intermediate attention matrices, not including the keys, queries, values, or output tensors.

GPU memory hierarchy creates the opportunity for I/O-aware attention. HBM offers large off-chip capacity, while much smaller on-chip SRAM has substantially higher bandwidth and lower latency. Attention implementations that materialize the quadratic intermediates in HBM must write and later read them during backpropagation. The fraction of time spent on those transfers depends on sequence length, head dimension, dtype, hardware, and kernel implementation; it is not a fixed property of GPT-2-scale models.

The backward pass requires the information represented by the attention probabilities, but an implementation may either save those intermediates or recompute them. For $\mathbf{A}_{\text{attn}} = \text{softmax}(\mathbf{Z}_{\text{attn}})$ and $\mathbf{Z}_{\text{attn}} = \mathbf{Q}\mathbf{K}^T / \sqrt{d_k}$, one gradient expression is:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{Q}} = \frac{1}{\sqrt{d_k}} \cdot \text{dsoftmax} \left(\frac{\partial \mathcal{L}}{\partial \mathbf{A}_{\text{attn}}}, \mathbf{A}_{\text{attn}} \right) \cdot \mathbf{K}$$

where dsoftmax denotes the softmax Jacobian-vector product. A conventional implementation may save the probability matrix for this computation; saving both probabilities and raw scores is not mathematically required. FlashAttention instead saves compact normalization statistics and recomputes tiled score and probability blocks during backward propagation, avoiding full quadratic intermediates in HBM.

²⁴ | **FlashAttention:** Introduced by T. Dao et al. (2022), it preserves exact attention while reducing HBM traffic through SRAM-sized tiling; the key insight was to treat attention as an IO problem: avoid materializing the full $S \times S$ score matrix in HBM while retaining dense $\mathcal{O}(S^2)$ score computation. Performance depends on shape, hardware, and baseline. FlashAttention-2 (Dao 2023) further improved parallelism and work partitioning, reporting 50–73 percent of theoretical peak on A100.

8.6.4.2 IO-aware attention through tiling

FlashAttention eliminates the need to materialize full $S \times S$ attention matrices in HBM by computing attention incrementally through tiling. Instead of computing the entire attention matrix at once, the algorithm partitions $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ into tiles small enough to fit in fast SRAM, computes attention scores for these tiles, and incrementally accumulates results.

The key algorithmic insight relies on the mathematical structure of softmax attention. Standard attention computes:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} \right) \mathbf{V}$$

Algorithm 8.3 decomposes this computation by partitioning the queries and the keys/values into tiles small enough to fit in SRAM, then streaming over the key/value tiles while maintaining the running softmax statistics needed to assemble each output tile, without ever forming the full score matrix.

No full $S \times S$ score or probability matrix is written to HBM. A 128×128 FP32 score tile, for example, occupies 65.5 KB compared with 67.1 MB for a 4096×4096 matrix. Other inputs, outputs, and running statistics remain, so the tile is not necessarily the largest tensor in the entire kernel.

Algorithm 8.3 FlashAttention: tiled attention with online softmax

Require: queries $\mathbf{Q}$, keys $\mathbf{K}$, values $\mathbf{V}$ for a length- S sequence; tile size b

Ensure: attention output $\mathbf{Y}$, without materializing the $S \times S$ score matrix

```

1: for each query tile  $\mathbf{Q}_i$  do
2:   in SRAM:  $\mathbf{Y}_i^{\text{acc}} \leftarrow \mathbf{0}$ ,  $m_i \leftarrow -\infty$ ,  $l_i \leftarrow 0$ 
3:   for each key/value tile  $(\mathbf{K}_j, \mathbf{V}_j)$  do
4:     load  $\mathbf{Q}_i, \mathbf{K}_j, \mathbf{V}_j$  into SRAM;  $\mathbf{Z}_{ij} \leftarrow \mathbf{Q}_i \mathbf{K}_j^T / \sqrt{d_k}$ 
5:      $m'_i \leftarrow \max(m_i, \text{rowmax}(\mathbf{Z}_{ij})) \triangleright \text{new stable max}$ 
6:      $\mathbf{Y}_i^{\text{acc}} \leftarrow \mathbf{Y}_i^{\text{acc}} \odot e^{m_i - m'_i} + e^{\mathbf{Z}_{ij} - m'_i} \mathbf{V}_j$ 
7:      $l_i \leftarrow l_i \odot e^{m_i - m'_i} + \text{rowsum}(e^{\mathbf{Z}_{ij} - m'_i})$ ;  $m_i \leftarrow m'_i$ 
8:     discard  $\mathbf{Z}_{ij} \triangleright \text{never written to HBM}$ 
9:   end for
10:   $\mathbf{Y}_i \leftarrow \mathbf{Y}_i^{\text{acc}} / l_i$ ; write  $\mathbf{Y}_i$  to HBM
11: end for
```

Full 67.1 MB

Tile 65.5 KB

log scale

FlashAttention swaps the full attention matrix for small SRAM tiles.

The online softmax algorithm enables this decomposition. Traditional softmax requires knowing all inputs before computing any output: $\text{softmax}(x)_i = e^{x_i} / \sum_j e^{x_j}$. FlashAttention uses an incremental formulation that updates softmax statistics as new blocks arrive, tracking the running maximum m (for numerical stability) and denominator l as each block is processed, then rescaling accumulated outputs accordingly.

8.6.4.3 Memory and IO complexity analysis

FlashAttention improves both activation memory and memory IO, which are the limiting costs in bandwidth-bound attention. Standard attention requires $\mathcal{O}(S^2)$ memory to store score matrices $\mathbf{Z}_{\text{attn}}$ and attention-probability matrices $\mathbf{A}_{\text{attn}}$ across all sequence positions. FlashAttention reduces the stored activation footprint to $\mathcal{O}(S)$ by keeping only input and output tensors ($\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{Y}$) plus a small SRAM buffer for the current tile.

For $S = 4096$, $d_{\text{head}} = 64$: Standard attention requires $4096^2 \times 4 \text{ bytes} = 67.1 \text{ MB}$ per head. FlashAttention requires only $(3 \times 4096 \times 64) \times 4 \text{ bytes} \approx 3.1 \text{ MB}$ per head, a 21.3 $\times$ reduction.

The IO pattern changes for the same reason. Standard attention reads $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ from HBM, writes score, probability, and output matrices during the forward pass, then rereads those stored matrices during the backward pass before writing $d\mathbf{Q}$, $d\mathbf{K}$, and $d\mathbf{V}$. The resulting HBM traffic includes the same $\mathcal{O}(S^2)$ term that made the activation footprint large. FlashAttention forms score and probability blocks in SRAM and never writes the full $S \times S$ tensors to HBM. In the backward pass, it recomputes the needed blocks rather than reading stored full matrices. The exact HBM IO

complexity depends on tile size and available SRAM because key and value blocks are streamed repeatedly across query blocks, so the bound is not simply $\mathcal{O}(S \cdot d)$.

For large sequence lengths, FlashAttention reduces HBM traffic by avoiding writes and rereads of the $S \times S$ score and probability matrices. With $S = 4096$ and $d_{\text{head}} = 64$, the activation memory footprint falls from hundreds of MB per head for stored attention matrices to a few MB for tiled inputs and outputs, while the precise bandwidth reduction depends on hardware and kernel tiling.

Both approaches require $\mathcal{O}(S^2 d)$ asymptotic FLOPs for dense attention. FlashAttention’s backward pass recomputes tiled attention intermediates from saved inputs and normalization statistics rather than storing full score and probability matrices. The resulting reduction in HBM traffic can shift the kernel toward compute-bound execution and produce a net speedup, but the final bottleneck remains shape- and hardware-dependent.

8.6.4.4 Implementation and hardware utilization

FlashAttention’s performance gains materialize through careful exploitation of GPU memory hierarchy. Modern frameworks can dispatch eligible attention operations among fused and fallback implementations based on hardware, inputs, and layouts; profiling verifies which path ran and whether it helped. Listing 8.6 contrasts standard and optimized attention implementations.

Listing 8.6: Attention Implementation Comparison: Standard attention materializes the full $S \times S$ matrix in HBM, while FlashAttention uses PyTorch’s optimized implementation or the dedicated flash-attn library.

```
import torch
import torch.nn.functional as F

# Standard attention (materializes n-by-n matrix)
def standard_attention(q, k, v):
    # q, k, v: [batch, heads, seq_len, head_dim]
    scores = torch.matmul(q, k.transpose(-2, -1)) / (
        q.size(-1) ** 0.5
    )
    attn = F.softmax(scores, dim=-1) # n-by-n matrix in HBM
    output = torch.matmul(attn, v)
    return output

# FlashAttention (no n-by-n materialization)
def flash_attention(q, k, v):
    # Can use FlashAttention when inputs and hardware are eligible
    output = F.scaled_dot_product_attention(q, k, v)
    return output

# Explicit FlashAttention 2 (flash-attn library)
from flash_attn import flash_attn_func

def flash_attn_2(q, k, v):
    # q, k, v: [batch, seq_len, heads, head_dim]
    # Different layout for optimized memory access
    output = flash_attn_func(q, k, v)
    return output
```

8.6.4.5 Benchmark results

The benefits of FlashAttention become concrete when measured on real hardware. T. Dao et al. (2022) reports end-to-end GPT-style training speedups and separate attention-kernel benchmarks showing that IO-aware attention reduces memory traffic and improves runtime. Table 8.13 uses an illustrative A100-style scenario to show the same systems pattern; its timings and memory values are representative chapter numbers, not values reported verbatim by T. Dao et al. (2022).

Table 8.13: FlashAttention Benchmark Comparison: Illustrative per-call timing and peak memory for standard attention vs. FlashAttention on a 40 GB A100-style configuration across sequence lengths. OOM marks configurations where standard attention exceeds the 40 GB memory budget; the 8192-token row also shows that standard attention would exceed 80 GB.

Sequence Length	Standard Forward	Flash Forward	Standard Backward	Flash Backward	Memory (Standard)	Memory (Flash)
512	12 ms	8 ms	35 ms	18 ms	4.2 GB	2.8 GB
2048	45 ms	15 ms	120 ms	35 ms	18 GB	6 GB
4096	OOM	32 ms	OOM	85 ms	> 40 GB	12 GB
8192	OOM	68 ms	OOM	180 ms	> 80 GB	24 GB

In this illustrative 40 GB A100-style scenario, standard attention runs out of memory beyond 2048 tokens, while FlashAttention fits sequences up to 8192 tokens. Even at 2048 tokens where both fit, FlashAttention achieves $3\times$ forward pass speedup and $3.4\times$ backward pass speedup.

Subsequent versions have continued improving performance: FlashAttention-2 (Dao 2023) achieved $1.5\text{--}2\times$ additional speedup through better parallelism and register allocation, while FlashAttention-3 (Shah et al. 2024) exploits FP8 tensor cores and asynchronous memory operations on Hopper GPUs and reports reaching approximately 740 TFLOP/s on H100, around 75 percent of theoretical peak.

8.6.4.6 When to use FlashAttention

For transformer training with long sequences, FlashAttention is usually the first attention implementation to test. It often becomes essential once sequence length pushes standard attention into an HBM-capacity or HBM-bandwidth bottleneck, especially around the multi-thousand-token regime where the $S\times S$ attention matrices dominate memory. A100- and H100-class GPUs with fast on-chip SRAM benefit most. The returns diminish for short sequences where tiling overhead is not worthwhile, on older GPU architectures without comparable SRAM bandwidth, and for nonattention architectures such as convolutional neural networks and multilayer perceptrons.

In practice, deep learning frameworks handle much of FlashAttention’s integration, but the decision still depends on layout, precision, and memory budget. PyTorch’s `scaled_dot_product_attention` can dispatch to FlashAttention when hardware, inputs, and backend constraints permit; hand-written attention must use an eligible primitive. Three practical checks determine whether FlashAttention is appropriate:

1. Ensure tensor layouts match library expectations (contiguous memory, correct dimension ordering)
2. Use FP16 or BF16 for maximum speedup (FlashAttention optimized for mixed precision)
3. Combine with gradient checkpointing when additional activation-memory savings are needed.

The integration is often a single-line change—swapping a manual attention call for `F.scaled_dot_product_attention` (see listing 8.6). The engineering decision is still the same one established by the roofline analysis: use the optimized primitive when the bottleneck is HBM traffic, and let the library manage the tiling and SRAM scheduling needed to remove it.

8.6.4.7 Systems implications and broader principles

FlashAttention exemplifies a fundamental systems engineering principle: **IO-aware algorithm design**. Many accelerators are compute-abundant relative to memory bandwidth. For a bandwidth-bound kernel, traffic can govern runtime even when FLOPs do not change.

This principle extends beyond attention. In IO-aware matrix multiplication, tiling algorithms like those in CUTLASS minimize DRAM traffic by maximizing data reuse in fast caches. A naive $m\times m$ matrix multiply performs $\mathcal{O}(m^3)$ FLOPs with $\mathcal{O}(m^2)$ memory traffic, while blocked algorithms maintain $\mathcal{O}(m^3)$ FLOPs but reduce cache misses through locality optimization.

The same byte-movement principle reappears in communication-efficient distributed training, where gradient compression trades extra computation (compression/decompression) for reduced network bandwidth consumption. Low-power edge devices with limited memory bandwidth benefit

even more from IO-aware algorithms. Trading 10 percent more arithmetic to halve memory traffic approaches a $2\times$ energy reduction in the limit where data movement accounts for the entire budget, and falls short of that bound whenever arithmetic carries a meaningful share, because halving one additive term of a sum cannot more than halve the sum. This section establishes the IO-aware heuristic; distributed and edge deployment settings reuse the same byte-movement logic.

Eliminating off-chip materialization of the $\mathcal{O}(S^2)$ attention matrices reclaims accelerator memory across three architectural trade-offs. First, it enables longer sequence contexts on fixed hardware, shifting the workable context boundary from 2K to 8K tokens in the illustrated A100 configuration. Second, freeing activation memory allows larger micro-batch sizes that improve hardware arithmetic saturation, provided gradient dynamics remain stable. Third, the contracted activation footprint permits deeper transformer stacks within a given memory budget, although end-to-end multi-billion-parameter feasibility continues to depend on parameter sharding, optimizer states, and remaining layer activations.

The technique demonstrates that systems-level algorithms can exploit memory hierarchy for large activation-memory reductions and workload-dependent speedups beyond hardware scaling alone. Treating memory bandwidth as the primary constraint and compute as abundant is a recurring pattern in ML performance optimization.

FlashAttention addresses memory bandwidth bottlenecks during computation, but another class of memory constraints exists: the sheer capacity required to store activations and optimizer states simultaneously. When models or batch sizes exceed GPU memory capacity, two complementary techniques trade computation for memory.

8.6.5 Gradient accumulation and checkpointing

Training large models requires substantial memory for activations, gradients, and parameters. When memory constrains batch size or model complexity, gradient accumulation²⁵ forms a larger effective batch from sequential micro-batches, while checkpointing discards and recomputes selected activations. These techniques become standard tools when memory is the binding constraint.

8.6.5.1 Gradient accumulation and checkpointing mechanics

Gradient accumulation and activation checkpointing operate on distinct principles, but both aim to optimize memory usage during training by modifying how forward and backward computations are handled. Gradient accumulation changes how many examples contribute to one update; checkpointing changes which activations remain resident between the forward and backward passes.

Gradient accumulation To execute large batch training on memory-constrained accelerators, systems split effective batches across smaller sequential micro-batches. In figure 8.13, trace how micro-batch gradient vectors $\delta_1, \delta_2, \delta_3$ accumulate into a single unified gradient tensor before triggering an optimizer update.

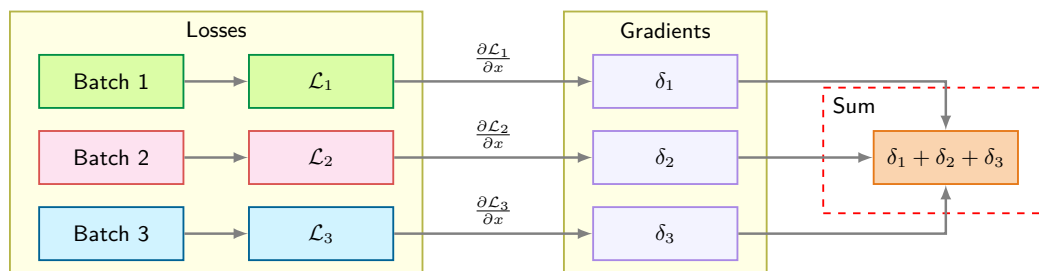


Figure 8.13: Gradient Accumulation: Three micro-batches each compute independent losses and gradients, which sum into a single combined gradient for one parameter update. This simulates training with a batch three times larger without requiring the memory to hold all samples simultaneously.

In PyTorch, gradient accumulation is usually implemented by dividing each micro-batch loss by the number of accumulation steps and calling `optimizer.step()` only after processing the entire effective batch. Under that averaging convention, no learning-rate adjustment is needed solely because of accumulation. The implementation follows five steps:

²⁵ | **Gradient accumulation:** Sums or averages gradients across k micro-batches before one optimizer step. It reduces resident activation memory relative to processing the same effective batch at once, while preserving total example-level arithmetic. Runtime and numerical equivalence depend on micro-batch efficiency, synchronization, stateful layers, stochastic operations, precision, and loss normalization.

1. Perform the forward pass for a micro-batch.
2. Compute the gradients during the backward pass.
3. Accumulate the gradients into a buffer without updating the model parameters.
4. Repeat steps 1–3 for all micro-batches in the effective batch.
5. Update the model parameters using the accumulated gradients after all micro-batches are processed.

Gradient accumulation can reproduce the gradient of a larger batch under controlled assumptions. For an effective batch size $B = k \times b$ where k is the number of accumulation steps and b is the micro-batch size, equation 8.17 confirms that the accumulated gradient equals the true batch gradient:

$$\nabla \mathcal{L}_B = \frac{1}{B} \sum_{i=1}^B \nabla \mathcal{L}_i = \frac{1}{k} \sum_{j=1}^k \left(\frac{1}{b} \sum_{i \in \mathcal{B}_j} \nabla \mathcal{L}_i \right) \quad (8.17)$$

This equivalence holds when each example’s computation is identical and the loss is averaged consistently. The right-hand side shows that averaging k micro-batch gradients (each computed over b examples) produces the same result as computing the gradient over all $B = kb$ examples at once. Stateful layers such as BatchNorm, stochastic operations such as dropout and data augmentation, finite-precision accumulation order, and step-indexed schedules can break exact equivalence, so large-batch accumulation still requires validation.

Gradient accumulation exchanges memory capacity for computation time. Table 8.14 separates the memory benefit from the compute and scheduling costs that remain.

Table 8.14: Gradient Accumulation Trade-Offs: Accumulation reduces resident activation memory and preserves total example-level arithmetic. Wall-clock cost depends on micro-batch efficiency, kernel-launch overhead, and whether distributed synchronization is deferred until the accumulated update.

Dimension	Effect
Memory	$\mathcal{O}(b)$ instead of $\mathcal{O}(B)$, yielding a $k \times$ reduction in activation memory
Computation	Unchanged total FLOPs, as all B examples are still processed
Time	k micro-batch forward and backward passes precede each optimizer step; smaller micro-batches can reduce utilization and add launch overhead

Accumulation preserves the total example-level arithmetic but can change efficiency because smaller micro-batches use the accelerator differently and launch kernels more often. With distributed `no_sync()` accumulation, one gradient synchronization occurs after the k local micro-batches rather than after each one. The model in equation 8.18 gives the per-update time:

$$T_{\text{effective}} = \sum_{j=1}^k T_{\text{micro},j} + T_{\text{sync}} + T_{\text{update}} \quad (8.18)$$

Here $T_{\text{micro},j}$ is the measured forward/backward time for micro-batch j , T_{sync} is the synchronization time paid for the accumulated update, and T_{update} is the optimizer-step time. In distributed data-parallel training, standard framework wrappers automatically fire asynchronous gradient-synchronization hooks during every backward pass. When accumulating gradients across k micro-batches, leaving these hooks active would trigger redundant AllReduce operations on every micro-batch; wrapping the first $k - 1$ micro-batches in a `model.no_sync()` context suppresses this communication, executing a single collective AllReduce only on the final k -th step.

The wall-clock penalty relative to processing the effective batch at once is workload-dependent. It must be measured rather than inferred from the memory-reduction factor.

When gradient accumulation is combined with distributed data parallelism across multiple machines, additional considerations arise for gradient synchronization timing and effective batch size calculation across the cluster. Advanced distributed systems texts treat these patterns in depth.

Activation checkpointing Activation checkpointing reduces memory usage during the backward pass by discarding and selectively recomputing activations. In standard training, activations from the forward pass are stored in memory for use in gradient computations during backpropagation. However, these activations can consume gigabytes of memory, particularly in deep networks.

Activation memory can be reduced by trading FLOPs for memory bandwidth during backpropagation. Contrast the forward pass in the top row of figure 8.14 with the backward pass in the bottom row, observing how intermediate activations are discarded and recomputed on demand from solid green checkpoint tensors.

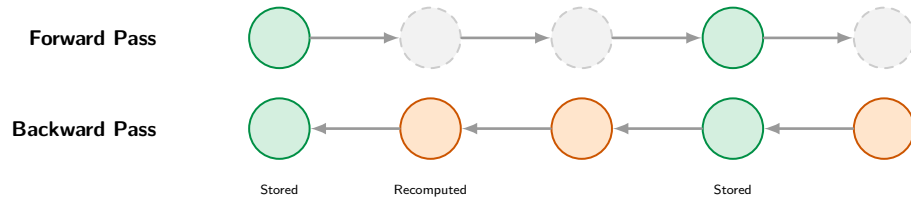


Figure 8.14: Activation Checkpointing: Trading memory usage for recomputation during backpropagation enables training deeper neural networks. By storing only a subset of activations from the forward pass and recomputing others on demand, this technique reduces peak memory requirements at the cost of increased training time.

The implementation keeps the capacity trade-off explicit. The system splits the model into segments, retains activations only at segment boundaries during the forward pass, and recomputes intermediate activations during the backward pass when needed.

Frameworks like PyTorch provide tools such as `torch.utils.checkpoint` to simplify this process, but the recompute cost remains. Checkpointing is most effective for deep architectures with dozens or hundreds of layers, such as transformers or large convolutional networks, where activation storage exceeds the GPU’s capacity before arithmetic throughput is exhausted.

The synergy between gradient accumulation and checkpointing enables training of larger, more complex models. Gradient accumulation manages memory constraints related to batch size, while checkpointing optimizes memory usage for intermediate activations. Together, these techniques expand the range of models that can be trained on available hardware.

8.6.5.2 Optimal checkpoint placement strategy

For a network with N_L layers, each storing A bytes of activations, table 8.15 quantifies how the number and placement of checkpoints determines the memory-compute trade-off. Sub-linear checkpointing strategies can reduce memory consumption from $\mathcal{O}(N_L)$ to $\mathcal{O}(\sqrt{N_L})$ with only a fractional increase in total compute time, enabling the training of much deeper models on existing hardware.

Table 8.15: Checkpointing Memory-Compute Trade-Offs: Different checkpoint strategies trade memory savings against recomputation overhead. The optimal number of checkpoints balances these factors.

Strategy	Memory Cost	Recompute Cost
No checkpointing	$N_L \times A$	0 forward ops
Retain one boundary	A	$(N_L - 1)$ forward ops
k checkpoints	$k \times A + (N_L/k) \times A$	$(N_L - k)$ forward ops

Setting the derivative of total memory cost ($k \times A + (N_L/k) \times A$) with respect to k to zero yields the analytical optimum at $k_{\text{optimal}} = \sqrt{N_L}$. In the standard $\sqrt{N_L}$ checkpointing scheme, this incurs roughly one-third extra forward compute in exchange for minimizing activation footprint under this analytical model. For GPT-2 with forty-eight transformer layers, the operational difference is substantial: uncheckpointed training requires $48 \times A$ in full activation storage, whereas uniform checkpointing ($\sqrt{48} \approx 7$ checkpoints) requires $7 \times A + (48/7) \times A \approx 14 \times A$, saving 71 percent activation memory for approximately 33 percent additional total step computation (by recomputing one forward pass over the uncheckpointed layers during backpropagation).

Not all operations are equally expensive to recompute, which motivates selective checkpointing. Table 8.16 compares where activation memory is large enough to justify recomputation.

Table 8.16: Selective Checkpoint Placement: Placement compares the state saved at a block boundary with the work required to reconstruct it. Attention and feed-forward costs depend on sequence length, width, kernel implementation, and whether memory-efficient attention is already in use.

Layer type	Memory cost	Recompute cost	Checkpointing strategy
Attention blocks	Shape-dependent; materialized attention can add quadratic intermediates	High and sequence-length-dependent	Checkpoint block boundaries when saved state justifies recomputation
Feed-forward blocks	Often dominated by expanded hidden activations	High and width-dependent	Checkpoint when expanded activations dominate memory
Normalization	Small standalone outputs	Low	Usually handled within a larger checkpointed block

The practical rule is to spend recomputation on high-memory layers, not on operations whose activations are already cheap to retain.

8.6.5.3 Memory and computational benefits

Gradient accumulation produces an effective batch larger than the resident micro-batch without storing all of its activations simultaneously. A larger effective batch reduces sampling variance but does not guarantee faster convergence or better generalization. The technique is particularly valuable when the desired effective batch cannot fit in memory at once.

Activation checkpointing significantly reduces the memory footprint of intermediate activations during the forward pass, allowing training of deeper models. By discarding and recomputing activations as needed, checkpointing frees up memory for larger models, additional layers, or higher resolution data. This trade-off is critical in architectures like transformers that require substantial memory for intermediate computations.

Both techniques improve scalability by reducing the memory required before adding hardware. Returning to lighthouse 8.1, gradient accumulation is essential for achieving the target batch size within V100 memory constraints.

Napkin Math 8.8: GPT-2 gradient accumulation strategy

This illustrative GPT-2-scale scenario uses assumed prices and durations to show gradient accumulation; it does not reconstruct GPT-2's historical training bill.

Memory Constraints

- With the chapter's simplified memory recipe, a V100 with 32 GB holds a per-GPU micro-batch of $B = 4$.
- A resident effective batch of $B = 512$ would therefore require $512 \div 4 = 128$ GPUs.

Accumulation Configuration

- Use 8 GPUs and accumulate 16 micro-batches of 4 examples each.
- Each accelerator processes $4 \times 16 = 64$ examples per update, giving a global effective batch of $8 \text{ GPUs} \times 64 = 512$.

A distributed `no_sync()` context can defer AllReduce until the final micro-batch.

Trade-off Analysis

- Idealized wall-clock time: $16\times$ longer because fewer accelerators perform the same arithmetic work
- Memory overhead: accumulation normally reuses parameter-gradient buffers, although bookkeeping and temporary storage remain

- Communication: both configurations synchronize once per 512-sample update, but participant count and topology differ
- Equal-work cost: \$688K either way, before utilization and communication effects

Why this works: Under the equivalence conditions stated above, averaged per-example gradients are additive. Each accelerator first accumulates a local gradient over its 64 examples:

$$\frac{1}{16} \sum_{j=1}^{16} \left[\frac{1}{4} \sum_{k=1}^4 \nabla \mathcal{L}(x_{jk}) \right]$$

AllReduce then averages those local gradients across 8 GPUs, producing the global effective batch:

$$\frac{1}{8} \sum_{g=1}^8 \nabla \mathcal{L}_{\text{local}}^{(g)}, \quad B_{\text{global}} = 8 \times 64 = 512$$

Operational comparison

Resident configuration:

- 128 GPUs $\times$ 4 examples = 512 resident examples per update
- Gradient synchronization spans 128 GPUs
- Hourly cost: 128 GPUs $\times$ \$16/hour = \$2,048/hour

Accumulated configuration:

- 8 GPUs $\times$ (4 examples $\times$ 16 micro-batches) = 512 examples per update
- Gradient synchronization occurs once per update across 8 GPUs
- Hourly cost: 8 GPUs $\times$ \$16/hour = \$128/hour
- Hourly reduction: \$1,920/hour, or 93.8 percent

Quality gate

- Match the examples and their order, loss normalization, optimizer state and step schedule, and stochastic-layer behavior.
- Compare held-out loss or perplexity and the optimization trajectory across repeated runs rather than assuming that the effective batch alone guarantees equivalent quality.
- Expect small round-off differences from the changed summation order; require convergence within a declared tolerance rather than bitwise identity. No perplexity result is assumed here.

Systems insight: Accumulation lowers accelerator count and hourly burn, but ideal scaling preserves accelerator-hours; actual cost depends on utilization and communication.

Listing 8.7 places one AllReduce after sixteen local micro-batches, making the synchronization boundary explicit.

Listing 8.7: Gradient Accumulation Training Loop: Sixteen local micro-batches accumulate before one AllReduce and optimizer update.

```
optimizer.zero_grad()
for step in range(16): # Accumulation steps
    micro_batch = next(dataloader) # 4 samples
    loss = model(micro_batch) / 16 # Average effective-batch loss
    loss.backward() # Accumulate gradients
# Now gradients represent 64 local samples
all_reduce(gradients, op=SUM) # Sum across 8 GPUs
gradients /= 8 # Convert the sum to the global mean
optimizer.step() # Update with effective batch=512
```

8.6.5.4 Practical considerations

Gradient accumulation is most valuable when optimal batch sizes exceed GPU memory capacity. Transformer language models often use effective batches of hundreds of thousands to millions of tokens; if memory only permits a smaller number of sequences per device, accumulation bridges the gap without requiring additional hardware. Activation checkpointing complements this by enabling deeper architectures and is commonly used in transformer-scale training, as well as in activation-heavy dual-network configurations such as generative adversarial networks.

Both techniques introduce explicit trade-offs. Activation checkpointing recomputes discarded activations during backward propagation; with ordinary segment checkpointing, each discarded activation is typically reconstructed once, while recursive schedules may behave differently. Gradient accumulation reduces parameter-update frequency because each update follows k micro-batches. When each micro-batch loss is divided by k , the accumulated gradient follows the effective-batch mean convention. If losses are summed instead, the gradients must be rescaled before the optimizer step or the learning-rate convention changed accordingly. Framework and codebase conventions differ, making normalization a common source of subtle bugs.

8.6.6 Optimization technique comparison

Each training optimization targets a distinct physical bottleneck within the accelerator memory and compute hierarchy. Asynchronous prefetching pipelines host-to-device transfers across the PCIe bus with accelerator execution, preventing compute engines from idling on input data starvation. Mixed-precision training halves tensor transfer volume across the HBM bus for weights and activations, while routing matrix arithmetic to specialized low-precision tensor units. FlashAttention complements these system-level interventions by resolving the quadratic memory-bandwidth bottleneck of self-attention; by tiling queries, keys, and values into on-chip SRAM, it reduces auxiliary attention memory from $\mathcal{O}(S^2)$ to $\mathcal{O}(S)$ and reports speedups of up to $3\times$ on evaluated workloads. Finally, activation checkpointing and gradient accumulation manage device memory capacity: checkpointing discards intermediate activations to reconstruct them via backward recomputation, while gradient accumulation partitions a large logical batch into temporal micro-batches to bound peak resident memory.

Because training throughput is governed by the system's binding constraint, applying an optimization to a non-binding subsystem yields no end-to-end speedup. Accelerating tensor arithmetic with lower precision or FlashAttention cannot reduce step time if compute pipelines remain stalled waiting for batches from the host. Conversely, expanding prefetch buffers provides no benefit when execution is bound by HBM capacity or backward-pass recomputation. Profiling must therefore identify whether the workload is bound by I/O ingest, memory capacity, memory bandwidth, or arithmetic throughput before an intervention is selected.

Table 8.17 compares three of these foundational strategies across their mechanisms, hardware requirements, and operational trade-offs, with FlashAttention serving as an algorithm-level complement within the attention kernel itself.

Table 8.17: Optimization Strategies: The table compares prefetching, mixed precision, and the combined use of gradient accumulation and checkpointing. The first targets input stalls, the second changes storage and eligible arithmetic, and the third trades smaller resident activation sets for micro-batch or recomputation overhead.

Aspect	Prefetching and Overlapping	Mixed-Precision Training	Gradient Accumulation and Checkpointing
Primary Goal	Minimize data transfer delays and maximize accelerator utilization	Reduce tensor storage and accelerate eligible lower-precision operations	Overcome memory limitations during backpropagation and parameter updates
Key Mechanism	Asynchronous data loading and parallel processing	Combining lower- and higher-precision computation	Simulating larger batch sizes and selective activation storage
Memory Impact	Increases memory usage for prefetch buffer	Can reduce weights, activations, and communicated tensors	Reduces resident activations; parameter-gradient size is unchanged
Computation Speed	Improves by reducing idle time	Can accelerate eligible FP16/BF16 operations on supported hardware	May slow down due to recomputations in checkpointing

Aspect	Prefetching and Overlapping	Mixed-Precision Training	Gradient Accumulation and Checkpointing
Scalability	Highly scalable, especially for large datasets	Enables training of larger models	Allows training deeper models on limited hardware
Hardware Requirements	Benefits from fast storage and multi-core CPUs	Supported lower-precision accelerator paths	Works on standard hardware
Implementation Complexity	Moderate (requires tuning of prefetch parameters)	Low to moderate (with framework support)	Moderate (requires careful segmentation and accumulation)
Main Benefits	Reduces training time, improves hardware utilization	Faster training, larger models, reduced memory usage	Enables larger batch sizes and deeper models
Primary Challenges	Tuning buffer sizes, increased memory usage	Potential numerical instability; FP16 may require loss scaling	Increased computational overhead, slower parameter updates
Ideal Use Cases	Large datasets, complex preprocessing	Large-scale models, especially in NLP and computer vision	Memory-constrained batches or activation-heavy models

The table compares techniques in isolation; the GPT-2 walkthrough shows how those techniques combine when one model must fit in memory, run fast enough, and stay within a realistic cost envelope.

8.6.7 GPT-2 optimization walkthrough

Training GPT-2 (1.5 billion parameters) on a single V100 GPU exposes the physical boundaries that govern large-model optimization: tensor allocations immediately exceed accelerator memory capacity, while naive input pipelines starve execution engines. Tracing this single-device diagnostic progression establishes memory feasibility and compute saturation before scaling to a multi-accelerator cluster.



Napkin Math 8.9: GPT-2 optimization on V100

Step 1: Establish the baseline memory footprint.

The trace starts with GPT-2 XL with 1.5 billion parameters, batch size 4, sequence length 1024, FP32 tensors throughout, and a single-threaded synchronous data loader. This is a deliberately constrained configuration: small enough to discuss on one V100, but large enough that the first failure is immediate and unambiguous. This walkthrough uses batch size 4, the same configuration as the mixed-precision callout (napkin math 8.6); table 8.19 reports the batch-32 equivalents (597.8 GB FP32 baseline, 95.7 GB optimized), which exceed a single GPU even after optimization. At batch size 4, the FP32 baseline needs 95.7 GB, so the run fails the machine constraint on a 32 GB V100 before throughput matters.

Step 2: Apply mixed precision.

The first intervention targets memory feasibility rather than speed. Automatic mixed precision (AMP) evaluates forward and backward operations in 16-bit precision, halving the dynamic activation memory. Because numerical stability requires maintaining 32-bit master weights and FP32 Adam moment states, static parameter and optimizer storage remains unchanged at 16 bytes per parameter. Halving the activation footprint reduces peak memory to 59.9 GB, a 37.5 percent reduction, but the allocation still exceeds the physical capacity of a 32 GB accelerator.

Step 3: Add gradient checkpointing.

Because static state cannot shrink further without altering optimizer dynamics, gradient checkpointing attacks the remaining activation footprint. Instead of retaining forward activations across every layer for backpropagation, the runtime discards intermediate states and recomputes them on demand during the backward pass. Checkpointing every four layers provides a 4× activation-memory reduction, bringing total tensor allocation to 33 GB and finally fitting within physical device capacity. Under an equal-layer recomputation model, this memory reduction incurs approximately 33 percent additional forward arithmetic; a production run

must verify that temporary workspace allocations and memory allocator fragmentation remain within the available headroom.

Step 4: Examine an illustrative throughput profile.

Fitting the model into device memory resolves capacity, but does not guarantee efficient execution. Profiling the newly executable model reveals the breakdown of iteration time across subsystems:

- Accelerator utilization: 45 percent
- Data loading: 40 percent of iteration time
- Compute: 35 percent of iteration time
- Memory transfers: 25 percent of iteration time

These profile measurements shift the diagnostic focus from machine memory capacity (M) to the data pipeline (D) in D·A·M terms. With 40 percent of iteration time spent waiting for host-prepared batches, further accelerator memory optimizations will not improve wall-clock training throughput until data starvation is resolved.

Step 5: Apply prefetching and data pipeline optimization.

Resolving input starvation requires overlapping host data preparation with accelerator computation. In this scenario, configuring the data loader with eight worker processes parallelizes CPU-side sample decoding, page-locked (pinned) host memory enables asynchronous DMA transfers across the PCIe bus, and a prefetch factor of 2 double-buffers upcoming batches in host and device queues. Hiding host preprocessing and transfer latency behind kernel execution raises accelerator utilization from 45 percent to 85 percent, transforming an input-starved workload into a compute-saturated run.

Table 8.18 contrasts the unexecutable naive baseline against the optimized single-GPU configuration.

Table 8.18: Illustrative GPT-2 Optimization Profile: The batch-4 memory figures are calculated by the walkthrough; utilization, throughput, and epoch time are assumed scenario values rather than benchmark results. Table 8.19 reports batch-32 scenario totals.

Metric	Naive	Optimized	Improvement
Memory	95.7 GB	33 GB	2.9× reduction
Accelerator utilization	N/A	85%	Trainable
Throughput	N/A	1,200 tokens/s	—
Time per epoch	N/A	8.3 hours	—

Systems insight: Training optimization follows a strict diagnostic dependency chain: solve memory feasibility before throughput, and eliminate data starvation before kernel tuning. Once capacity and input stalls no longer bind execution, re-profiling determines whether kernel performance is bounded by memory bandwidth or arithmetic throughput before altering batch size or model architecture.

This walkthrough demonstrates why training optimization is a sequential diagnostic discipline rather than an arbitrary collection of techniques. Each intervention directly resolves the binding physical constraint exposed by the preceding measurement: mixed precision halves activation storage, gradient checkpointing accepts 33 percent additional compute to deliver a 2.9× memory reduction that satisfies physical device capacity, and asynchronous prefetching eliminates PCIe transfer stalls once execution begins. With single-accelerator feasibility and compute saturation established, the next task is evaluating how these compounding optimizations scale across a multi-accelerator cluster in execution time, energy consumption, and total training cost.

8.6.8 Optimization impact summary

The GPT-2 case study demonstrates how targeted techniques can compose. Its memory values follow the chapter’s tensor model, while its time, power, electricity-price, and grid-carbon inputs define an illustrative operating scenario rather than a measured GPT-2 run.

Table 8.19 compiles the results implied by those cluster-level assumptions.

Table 8.19: Illustrative GPT-2 Training Scenario: The tensor model gives the stated memory totals. Training duration, constant cluster power, PUE, electricity price, and grid carbon intensity are scenario assumptions; under those assumptions, shorter runtime reduces energy and location-based emissions proportionally.

Metric	FP32 Baseline	Optimized	Technique Applied
Parameters	6 GB	3 GB	Mixed precision (FP16)
Gradients	6 GB	3 GB	Mixed precision (FP16)
Master Weights	0 GB	6 GB	AMP Overhead
Optimizer State (Adam)	12 GB	12 GB	Unchanged (FP32 moments)
Activations (batch=32)	573.8 GB	71.7 GB	Gradient checkpointing + FP16
Total Memory	597.8 GB	95.7 GB	—
Training Time (32 V100s)	14 days	8.4 days	Data prefetching + utilization
Energy Consumption	3,914 kWh	2,348 kWh	Assumed constant cluster power over each duration
Electricity Cost (\$0.10/kWh)	\$391.4	\$234.8	—
Carbon Footprint	1.7 t	1.0 t	Regional grid average (0.43 kg/kWh)

As table 8.19 shows, the assumed inputs produce a $6\times$ memory ratio, a $1.7\times$ wall-clock ratio, and a 40 percent energy reduction under constant modeled power. In practice, energy and cost must be computed from measured runtime and power because higher utilization can change the cluster’s power draw.

Single-accelerator optimizations compound to saturate the device: mixed precision maximizes Tensor Core throughput, FlashAttention eliminates off-chip intermediate attention traffic, gradient checkpointing compresses activation footprints, and asynchronous prefetching hides host I/O latency. Even when fully optimized, however, workloads with hundreds of exaFLOP budgets remain bounded by the arithmetic throughput of an individual chip.

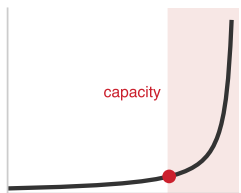
Scaling beyond single-device compute or memory capacity requires distributing computation across multiple accelerators. This architectural transition introduces inter-device communication overhead, collective synchronization barriers, and fault-tolerance constraints that become the primary determinants of cluster scaling efficiency.

8.7 Scaling Training Systems

The GPT-2 walkthrough models how mixed precision and checkpointing reduce a batch-4 tensor budget to 33 GB, near a V100’s 32 GB capacity before workspaces and allocator overhead. At batch 32, the modeled total falls from 597.8 GB to 95.7 GB but still exceeds that V100. Larger-memory accelerators may move the boundary. A 70-billion-parameter model, however, requires about 140 GB for FP16 weights alone and substantially more for full-parameter training state, motivating sharding, offload, or multiple devices.

When a chosen workload still exceeds one device’s memory or schedule, spreading it across devices provides aggregate memory and compute. Other responses include reducing the model, batch, sequence length, or training scope; offloading state; and using parameter-efficient adaptation. Scaling beyond one device begins with multi-GPU configurations inside one machine and may extend across nodes. The major parallelism strategies make different communication trade-offs; a full treatment of distributed-training implementation lies beyond this chapter.

Not all workloads benefit equally from adding GPUs. The relationship among useful computation, communication, imbalance, and synchronization determines scaling efficiency. Figure 8.15 models this trade-off using an Amdahl-style formulation where non-scaling overhead creates a binding



Some models exceed single-GPU memory before anything else matters.

throughput ceiling. The widening gap between ideal linear speedup and achieved speedup visualizes the communication tax paid at scale.

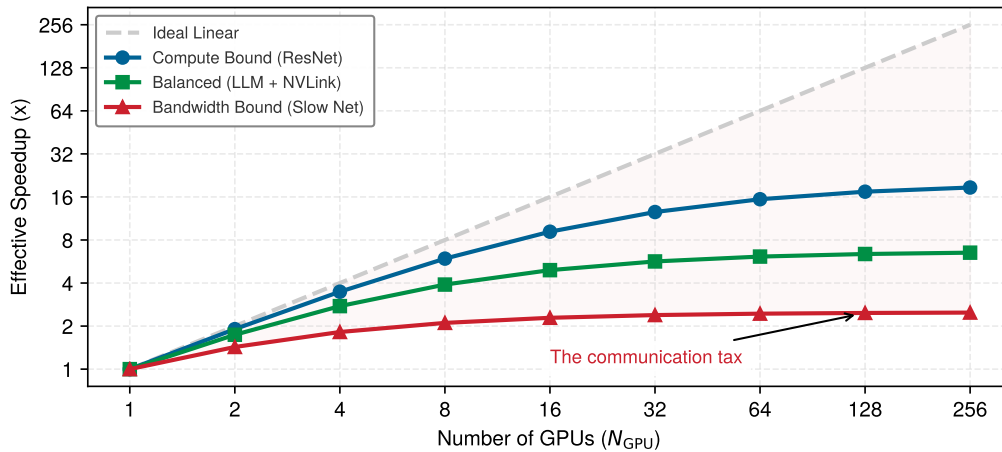


Figure 8.15: Communication Tax: An Amdahl-style model compares ideal linear scaling against workloads with varying non-scaling communication and synchronization fractions. Real communication overhead scales dynamically with device count, network topology, message size, and overlap efficiency.

The red curve’s widening gap from the dashed ideal is the communication tax made visible; shrinking that gap is what the distributed strategies in section 8.7.1 exist to do.

8.7.1 Single-node multi-GPU training

Multi-GPU training within a single node establishes the foundational architectural patterns for distributed execution. AlexNet²⁶ (Krizhevsky et al. 2012) split its convolutional layers across two GTX 580 GPUs because the 3 GB device memory limit prevented single-chip execution, making cross-GPU connectivity an immediate architectural constraint. Confining multi-accelerator topologies to single-node interconnects (such as PCIe or NVLink) isolates the core parallelism strategies before introducing cross-rack network fabrics.

The two foundational strategies, data parallelism and model parallelism, represent fundamentally different partitioning choices: data parallelism replicates the model and partitions data, while model parallelism partitions model state or computation. This distinction determines memory requirements, communication patterns, and scaling behavior.

8.7.1.1 Data parallelism

Data parallelism replicates the entire model on each GPU, with each processing different batches. After computing gradients locally, GPUs synchronize via gradient averaging. Figure 8.16 shows the data flow: input data splits into nonoverlapping batches, each accelerator computes forward and backward passes independently, then gradients aggregate before updating the shared model.

When a model and its optimizer state fit comfortably within single-device memory, data parallelism serves as the primary scaling baseline because local forward and backward kernels run at native memory bandwidth without inter-device synchronization. Communication occurs exclusively during gradient aggregation. Historically, frameworks supported multi-GPU execution within a single node using multithreaded wrappers like PyTorch’s `DataParallel` (DP). However, this single-process design suffers from two severe architectural bottlenecks: the Python runtime remains throttled by the Global Interpreter Lock (GIL), and a primary coordinator (GPU 0) must sequentially scatter mini-batches and gather outputs over PCIe, saturating host-device interconnect bandwidth and creating severe compute and memory imbalances. Modern systems replace this with multi-process architectures like PyTorch’s `DistributedDataParallel` (DDP) and TensorFlow’s `MirroredStrategy`. By spawning an independent Python process per accelerator, DDP eliminates GIL contention entirely; each process maintains its own model replica and local optimizer, relying

²⁶ | **AlexNet (2012):** The paper reports 60M parameters and a 3 GB single-GPU ceiling, but does not attribute the limit to one memory component. Its model-parallel design placed roughly half the kernels on each GPU and allowed communication only in selected layers, explicitly trading memory capacity against communication.

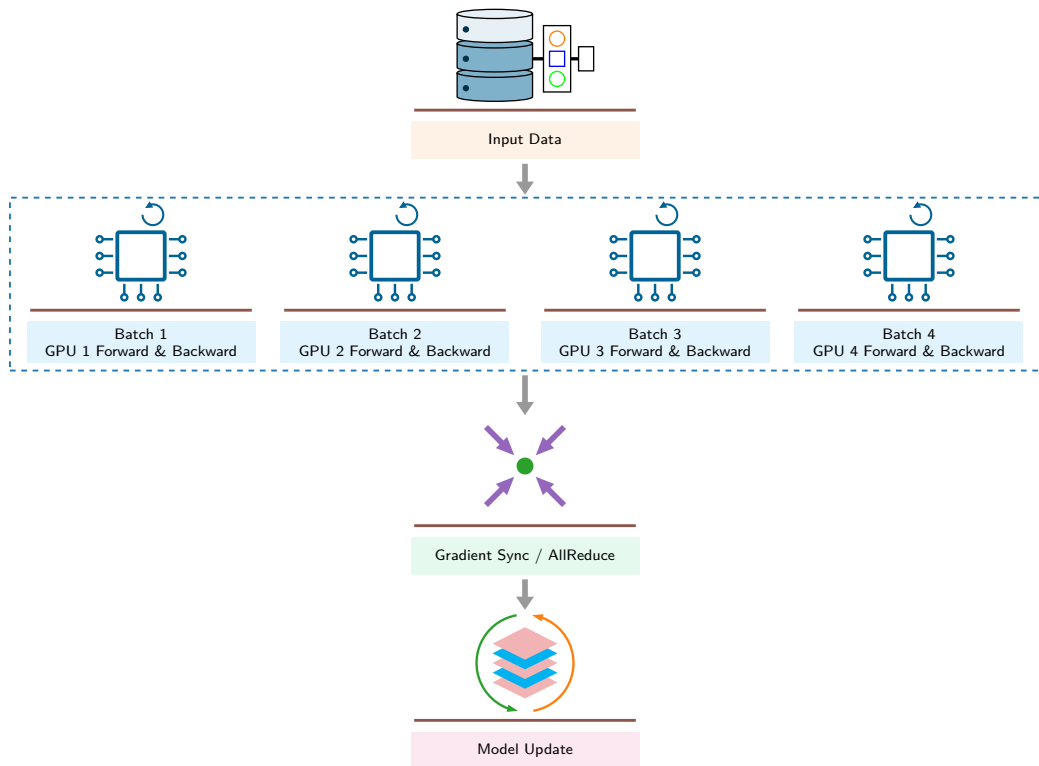


Figure 8.16: Data Parallelism: Each GPU holds a complete model copy, processes a different data shard, and synchronizes gradients. Throughput can approach linear scaling only while computation dominates synchronization, input, and imbalance costs.

on decentralized collective AllReduce operations that bucket gradients into contiguous buffers and overlap synchronization directly with backward-pass computation.

Unsharded data parallelism has a hard constraint: every GPU holds a complete model replica and ordinarily its local optimizer state. A 7-billion-parameter model uses 14 GB for FP16 weights alone, before gradients, optimizer states, or activations. Sharded data parallelism (such as the Zero Redundancy Optimizer, or ZeRO) distributes that state across devices: ZeRO-1 shards optimizer states, ZeRO-2 shards gradients, and ZeRO-3 (or FSDP) shards model parameters; otherwise, a model that exceeds local memory requires pipeline or tensor partitioning.

8.7.1.2 Model parallelism

Model parallelism partitions the model itself across GPUs, which becomes necessary when the model exceeds single-GPU memory. AlexNet adopted an early form of this strategy: specific convolutional layers executed on GPU 1 and others on GPU 2, communicating activations across PCIe only at layer boundaries. Figure 8.17 shows the forward and backward paths: data moves through model partitions on different devices, with gradients flowing backward during training.

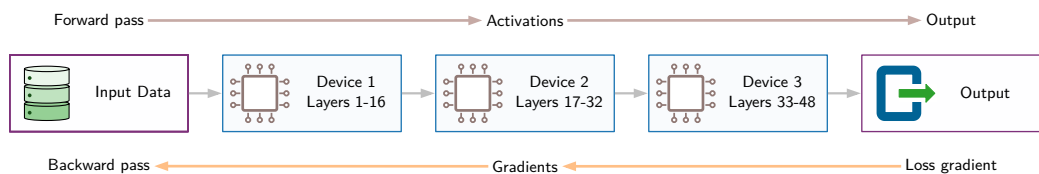


Figure 8.17: Model Parallelism: The model is partitioned across devices, with intermediate activations passing between them. This enables training models larger than single-GPU memory at the cost of sequential dependencies.

🔍 Systems Perspective 8.7: Data vs. model parallelism

When a selected workload is partitioned across N_{GPU} devices, the split may divide examples, model state, layers, or tensor operations. For a model with parameters P and batch size B , data and model parallelism provide two starting points.

Data parallelism (split the batch): For a fixed global batch, each of N_{GPU} workers processes about $1/N_{\text{GPU}}$ of the examples, but every worker holds a model replica and ordinarily its associated training state. Dense gradient communication is proportional to P per update. Scaling departs from the ideal as local batches shrink or synchronization and input costs grow.

Model parallelism (split model computation): Each accelerator stores and computes a portion of the model. An even partition may approach P/N_{GPU} parameters per worker, but embeddings, buffers, imbalance, and replicated state alter that estimate. Communication depends on the partition: layerwise pipeline parallelism exchanges activations at stage boundaries, while tensor parallelism performs collectives within partitioned layers.

Systems insight: Data parallelism is the simplest candidate when the complete training state fits per worker and more throughput is needed. Model-state, pipeline, or tensor partitioning becomes necessary when it does not, often in combination with data parallelism.

When a model exceeds the memory capacity of a single accelerator, its depth can be partitioned across sequential devices. Figure 8.18 illustrates this stage assignment, mapping contiguous groups of transformer blocks onto distinct devices across the execution chain.

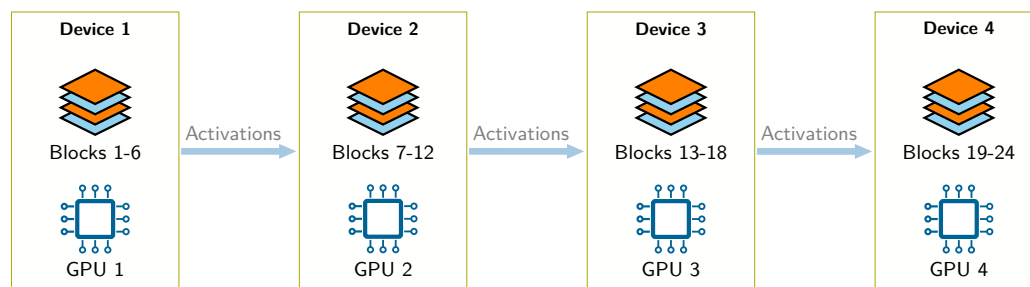


Figure 8.18: Layer-Wise Model Partitioning: Distributing consecutive transformer layers across sequential devices bounds network communication to intermediate activations (forward) and boundary gradients (backward). However, sequential layer dependencies leave downstream devices idle during forward passes unless micro-batches are pipelined.

The fundamental bottleneck in naive layer-wise partitioning is device serialization. Because layer evaluation is strictly sequential, Device 1 computes layers 1–6 while Devices 2–4 sit idle waiting for forward activations. During the backward pass, the dependency reverses: Device 4 computes first while upstream devices wait for returning gradients. This idle duration constitutes the *pipeline bubble*, which caps theoretical hardware utilization at $1/N_{\text{GPU}}$ under naive sequential execution. Mitigating this idle time requires either pipelining micro-batches across stages or partitioning tensor dimensions within layers so that all devices compute simultaneously.

Within a node, GPUs may communicate through high-bandwidth links such as NVLink²⁷ (up to 900 GB/s aggregate bidirectional bandwidth for the represented H100 configuration). Effective collective bandwidth depends on topology and contention and is lower than a raw aggregate link specification. Data parallelism communicates gradients via collective AllReduce operations, while layerwise model parallelism communicates activations point-to-point at partition boundaries. Choosing a strategy requires balancing these communication patterns and memory footprints against the physical interconnect bandwidth of the node topology.

8.7.2 Scaling beyond a single node

When a model’s parameter footprint, optimizer state, or training duration exceeds the capacity of an 8-accelerator node, distributed training extends across machines. This step introduces a fundamental bandwidth hierarchy: while accelerators within a node communicate across dense, high-bandwidth

²⁷ | **NVLink:** NVIDIA’s high-bandwidth GPU interconnect. Its topology and generation determine the bandwidth available between a particular pair of GPUs and to collective operations. A parallelism strategy helps only when its communication and synchronization cost is smaller than the useful time it saves.

fabrics such as NVLink, distributed workers must synchronize over external network fabrics (such as Ethernet or InfiniBand) that offer substantially lower bandwidth, higher latency, and switch contention. The resulting bandwidth gap turns synchronous data-parallel gradient reduction into a recurring data-movement bottleneck on the critical path.

Systems Perspective 8.8: The physics of synchronization

Recall the energy-movement invariant from chapter 4: data movement costs substantially more energy and time than nearby arithmetic. In distributed training, this asymmetry establishes the communication tax.

Synchronizing gradients across accelerators moves megabytes to gigabytes across an interconnect on every training step. Uncovered communication time directly reduces hardware utilization, while transceiver and switch power inflates the cluster's energy budget; these are distinct system costs. Mixed-precision execution (section 8.6.3) and gradient compression reduce transfer volumes through narrower numeric formats, sparsity, quantization, or delta encoding, relieving both bounds.

Within a node, high-bandwidth interconnects permit gradient synchronization to overlap with backward computation. Crossing node boundaries, however, transitions communication to a network fabric where available bandwidth is one to two orders of magnitude lower. In synchronous data parallelism, AllReduce²⁸ aggregates gradients across workers; when network bandwidth fails to match accelerator compute throughput, uncovered communication time dominates the step and stalls arithmetic execution.

²⁸ | **AllReduce**: A collective communication primitive that sums data across all devices and distributes the result back to each device. Ring AllReduce is one common implementation: devices pass gradient shards around a logical ring so each participant sends and receives bounded chunks rather than a full copy of every gradient tensor at once. This is the communication primitive that makes large data-parallel training practical, but its detailed cost model belongs with distributed training.

Napkin Math 8.10: The network wall

Problem: A team scales training to 8 GPUs, one per node, connected by a 100 Gbps fabric. Is the network the bottleneck?

Mechanism: For a 7B-parameter model with FP16 gradients:

1. **Gradient size (FP16):** 14 GB per step.
2. **AllReduce cost:** Ring AllReduce passes gradient shards around a logical ring, so each worker ends up sending nearly two full copies of the gradient: $2(N_{\text{GPU}} - 1)/N_{\text{GPU}}$, which equals $1.75\times$ for 8 GPUs. That gives $1.75 \times 14 \text{ GB} = 24.5 \text{ GB}$ total.
3. **Bandwidth-only lower bound:** At 12.5 GB/s per worker on the 100 Gbps fabric, synchronization latency is $T_{\text{comm}} = D_{\text{comm}}/\text{BW}_{\text{net}} = 24.5 \text{ GB} / 12.5 \text{ GB/s} = 1.96 \text{ s}$.
4. **Compute comparison:** If the nonoverlapped forward and backward work takes $T_{\text{compute}} \approx 1 \text{ s}$, communication latency $T_{\text{comm}} \approx 1.96 \text{ s}$ dominates.

Systems insight: The network becomes a wall when $T_{\text{comm}} > T_{\text{compute}}$, causing accelerator efficiency $\eta_{\text{hw}} = \frac{T_{\text{compute}}}{T_{\text{compute}} + T_{\text{comm}}}$ to plummet to about 33.8 percent. Mitigations include gradient compression (reducing D_{comm}), ring AllReduce primitives (Sergeev and Balso 2018), and hierarchical topologies that confine high-frequency transfers to fast intra-node links (NVLink at 900 GB/s vs. 12.5 GB/s inter-node).

Because cross-node fabrics throttle frequent synchronization, scaling requires matching each communication pattern to the appropriate bandwidth tier. Pipeline parallelism partitions model layers across sequential devices, addressing the idle pipeline bubbles of naive model parallelism through micro-batching. Instead of processing an entire batch serially, the global batch is divided into smaller micro-batches that stream through the pipeline stages. While a downstream stage executes a forward pass on an early micro-batch, upstream stages concurrently process subsequent ones. The pipeline schedule controls this concurrency: retaining more in-flight micro-batches reduces accelerator idle time but requires each stage to store activations in memory until the corresponding backward pass completes. GPipe²⁹ evaluates all forward micro-batches before draining the backward

²⁹ | **GPipe (2019)**: GPipe implements the described micro-batching strategy, staggering execution to fill the idle “bubbles” in naive model parallelism and thus boost utilization (Huang et al. 2019). The key trade-off it manages is increased memory for storing the activations of multiple in-flight micro-batches. It preserves large-batch training dynamics by accumulating gradients before the weight update; the paper reports almost linear Transformer throughput scaling when the micro-batch count is at least four times the number of partitions.

passes, whereas PipeDream interleaves one forward pass with one backward pass (1F1B) to bound resident activation memory (Huang et al. 2019; Narayanan et al. 2019).

Tensor parallelism operates at a finer granularity: rather than assigning whole layers to separate devices, it shards individual matrix operations across devices within a layer. (Sequence parallelism extends this principle by sharding activation tensors along the sequence dimension during long-context transformer training.) In a transformer feed-forward block, Megatron-LM splits the first weight projection column-wise and the second projection row-wise (Shoeybi et al. 2019). Each accelerator computes on a slice of the hidden dimension without intermediate communication; only a single AllReduce across the tensor-parallel group is required at the end of the block to sum the partial outputs. Because this pattern requires collective synchronization on every forward and backward operator pass, tensor parallelism imposes an extreme communication frequency that restricts its deployment strictly to high-bandwidth intra-node links such as NVLink.

Hybrid strategies combine tensor, pipeline, and data parallelism (3D parallelism) to scale large models across physical clusters. Table 8.20 summarizes the resulting placement heuristic: map the highest-frequency collective communication to the highest-bandwidth physical links, reserving lower-bandwidth tiers for coarser synchronization boundaries.

Table 8.20: Hybrid Parallelism Placement: A common heuristic places tensor parallelism within nodes, pipeline parallelism within racks, and data parallelism across racks. Actual placement depends on the model and topology.

Strategy	Typical placement	Communication pattern
Tensor parallelism	Within a node	Frequent communication that exploits NVLink’s high bandwidth
Pipeline parallelism	Across nodes within a rack	Moderate communication at layer boundaries
Data parallelism	Across racks	Gradient synchronization once per iteration

Partitioning computation across a cluster ultimately transforms training from a purely arithmetic problem into a multi-tier data-movement hierarchy. While collective communication algorithms, parameter servers, and fault-tolerant scheduling allow models to scale across thousands of accelerators, their sustained efficiency remains bound by the ratio of compute throughput to network transfer time. Understanding how modern training platforms balance these constraints requires examining the historical convergence of high-performance scientific computing and warehouse-scale infrastructure that produced today’s AI hardware fabrics.

8.7.3 The evolution of training infrastructure

The parallelism strategies in section 8.7.1 and section 8.7.2 represent the convergence of distinct architectural traditions. Modern training platforms integrate paradigms developed across four computing eras, each shaped by the physical constraints and dominant workloads of its time.

Neural network training combines requirements drawn simultaneously from high-performance computing (HPC) and warehouse-scale systems. From HPC, training inherits dense matrix multiplication (GEMM) kernels operating near accelerator compute limits, requiring tightly synchronized parallel execution across devices. From warehouse-scale computing, multi-day training jobs inherit the necessity of cluster-wide orchestration, asynchronous data ingestion, and automated fault recovery across failure-prone nodes. Standard synchronous data-parallel training executes a global AllReduce collective at every optimizer step. Keeping accelerators supplied with arithmetic without stalling on communication requires high-bandwidth interconnects (such as NVLink and InfiniBand) and direct memory access runtimes that bypass host operating-system overhead. Figure 8.19 traces the chronological progression across these computing eras.

This transition to AI hypercomputing reflects a structural divergence in memory access patterns and synchronization semantics. Scientific HPC simulations typically traverse regular spatial grids with modest working state per degree of freedom, while warehouse-scale services process independent, loosely coupled requests over commodity Ethernet. In contrast, neural network training confronts a massive state footprint: an accelerator must hold model weights, gradients, optimizer states (such as momentum and variance buffers), and intermediate activations retained for the backward pass. When this aggregate state exceeds single-device High Bandwidth Memory (HBM)

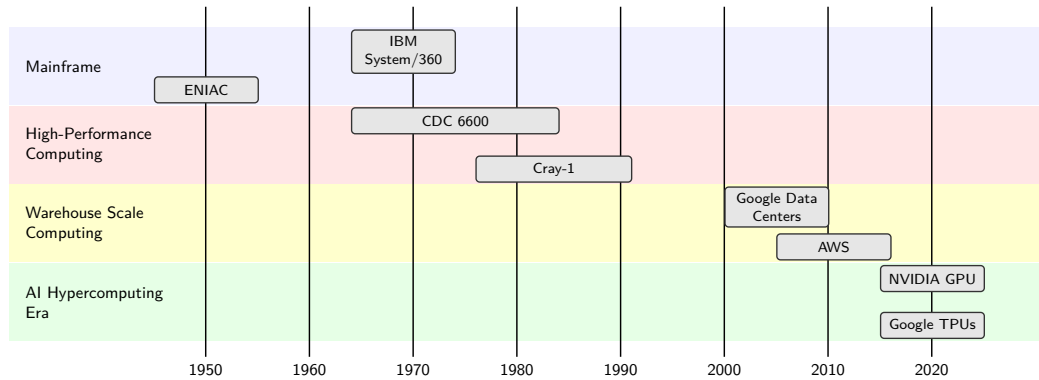


Figure 8.19: Computing System Evolution: Four overlapping eras place representative system milestones against time. The overlap matters: AI hypercomputing builds on, rather than replaces, the parallel numerical methods of HPC and the distributed coordination of warehouse-scale computing.

capacity, the model must be partitioned across multiple accelerators, trading local memory pressure for inter-device communication overhead. Table 8.21 contrasts the workload requirements, memory hierarchies, and execution models across these four architectural eras.

Table 8.21: Computing Era Characteristics: Each computing era optimized for different workload patterns, and modern training systems inherit requirements from multiple predecessors. AI hypercomputing uniquely combines HPC’s parallel numerical computation with warehouse-scale distributed processing, while adding specialized support for the gradient-based optimization and massive parameter state management central to neural network training.

Era	Primary Workload	Memory Patterns	Processing Model
Mainframe	Sequential batch processing	Simple memory hierarchy	Single instruction stream
HPC	Scientific simulation	Regular array access	Synchronized parallel
Warehouse-scale	Internet services	Sparse, irregular access	Independent parallel tasks
AI Hypercomputing	Neural network training	Parameter-heavy, mixed access	Hybrid parallel, distributed

8.7.4 When to scale: The physical ceiling

Knowing how to partition computation across devices is distinct from knowing *when* distributed execution is warranted. Distributing state introduces communication overhead, synchronization barriers, and operational failure modes. Exhausting single-node optimizations first isolates whether the bottleneck is fundamentally algorithmic or capacity-driven. Profile-guided tuning should evaluate lower-precision execution paths (section 8.6.3) when numerical stability permits, gradient accumulation (section 8.6.5) to synthesize target batch sizes without activation blowup, activation checkpointing (section 8.6.5.1) to trade redundant forward compute for memory headroom, and overlapped data prefetching (section 8.6.2) when host ingestion limits the critical path.

Table 8.22 structures this progression as a resource lookup keyed to measured hardware bottlenecks rather than arbitrary parameter thresholds.

Table 8.22: Scaling Decision Guidelines: Choose the least distributed configuration that satisfies measured memory, throughput, and schedule requirements. Model architecture, training state, input pipeline, hardware, and topology all affect the boundary.

Observed constraint	Candidate approach	Rationale
Training state fits one device and time is acceptable	Single GPU	Minimizes communication and operational complexity
State or throughput exceeds one device but fits one node	Single multi-GPU node	Uses fast local interconnects before crossing the network

Observed constraint	Candidate approach	Rationale
Required state or schedule exceeds one node	Multi-node cluster	Adds aggregate memory or compute at a communication cost
Input service rate is below accelerator demand	Parallel or distributed input pipeline	Adds input throughput where profiling shows it is needed

Only when profiling reveals persistent bottlenecks despite these optimizations should multi-device approaches be considered. Every hardware device has a physical ceiling: a workload requiring 10^{24} FLOPs cannot be completed on a single accelerator within a practical training schedule, no matter how carefully that accelerator is tuned. The transition to multi-device training becomes necessary when one of three hard limits is reached:

- **Memory exhaustion:** The model weights, gradients, optimizer states, and activations exceed the VRAM of a single GPU. A 70-billion-parameter model requires approximately 140 GB in FP16 for weights alone, exceeding the 80 GB configurations of A100 and H100 SXM accelerators and leaving little or no headroom on higher-capacity variants like the H100 NVL (94 GB) and H200 (141 GB) once training state is included.
- **Training wall-clock time:** The estimated time on one device exceeds the project's actual deadline. At 10^{15} FLOP/s sustained, a workload requiring 10^{24} FLOPs would take about 32 years.
- **Input throughput:** The input pipeline cannot sustain the aggregate accelerator consumption rate. Dataset size alone does not determine this limit; reuse, compression, locality, transformation cost, and storage bandwidth matter.

These three limits are technical, but scaling carries a cost beyond complexity and capacity. Adding devices amplifies the energy consumption and environmental impact of training, so efficiency optimization becomes an environmental concern as much as a performance one.



Napkin Math 8.11: The carbon footprint of training

Context: Training large models requires substantial energy and compute. The physical ceiling appears in the energy corollary to the iron law ($E_{\text{total}} = P_{\text{fleet}} \cdot T_{\text{train}}$), quantifying environmental scale:

1. Workload: Training a 7 billion-parameter model for 1 trillion tokens.
2. Compute: $O \approx 4.2 \times 10^{22}$.
3. Efficiency: 156 TFLOP/s sustained on A100 (400 W thermal design power).
4. **Execution time:** $T_{\text{train}} \approx 3.04$ days on 1,024 GPUs.
5. **Energy footprint:** $E_{\text{total}} = P_{\text{cluster}} \cdot T_{\text{train}} = (N_{\text{GPU}} \cdot P_{\text{GPU}} + N_{\text{host}} \cdot P_{\text{host}}) \cdot T_{\text{train}}$ evaluates to $(1,024 \text{ GPUs} \times 400 \text{ W} + 128 \text{ hosts} \times 200 \text{ W}) \times 73 \text{ hours} \approx 31,784.2 \text{ kWh}$.

Impact: The energy consumed equals 35.3 months of average household electricity use.

Systems lesson: Doubling hardware throughput η_{hw} at constant cluster power P_{cluster} halves execution time T_{train} and total energy E_{total} . Under average grid carbon intensity, doubling efficiency saves $\sim 15,892.1$ kWh of electricity and avoids 6.8 t of emissions. Environmental audits must measure $E_{\text{total}} = \int P(t)dt$ rather than relying on GPU utilization metrics alone.

Scaling changes the binding constraint rather than removing it: data parallelism buys throughput, model parallelism buys memory capacity, pipeline parallelism buys utilization, and tensor parallelism buys layer-scale feasibility, but each adds communication and coordination cost. The implementation details of multi-node distributed training, including collective communication primitives, fault tolerance mechanisms, and elastic scheduling, build directly on the single-machine principles covered throughout this chapter and are treated in depth in advanced distributed systems texts.

**Checkpoint 8.3: Scaling decisions**

Scaling trades compute bottlenecks for communication bottlenecks.

- ☐ **Hard limits:** can you identify which limit is binding for a given training run: memory capacity, wall-clock time, or input-service rate?
- ☐ **Single-node first:** can you explain why mixed precision, accumulation, checkpointing, and prefetching should be exhausted before adding devices?
- ☐ **Data vs. model parallelism:** given a model that does not fit on one GPU, can you justify why data parallelism fails and what model parallelism must partition?
- ☐ **Communication tax:** Can you explain why scaling efficiency degrades with GPU count, and what quantity (synchronization fraction) controls the ceiling?
- ☐ **Scaling ceiling:** if synchronization that does not parallelize is 10 percent of each step, estimate the speedup ceiling as the GPU count grows large, and the rough point where doubling devices stops being worth it.

8.8 Fallacies and Pitfalls

The progression from single-GPU optimization through multi-device parallelism shows that each technique introduces trade-offs and new constraints. The approach developed throughout this chapter quantifies costs through the iron law, diagnoses bottlenecks through profiling, applies targeted optimizations, and scales only when necessary. The following fallacies and pitfalls capture common errors that waste compute resources, delay research progress, or cause production training failures.

Fallacy: *Larger models always yield better performance.*

Teams sometimes treat scale as a monotonic lever: if a 7B-parameter model works well, a 20B-parameter model must work better. A 20B model requires approximately 320 GB of nonactivation training state in the stated **mixed-precision Adam** recipe (40 GB FP16 parameters + 40 GB FP16 gradients + 80 GB FP32 master weights + 160 GB Adam states). Whether it outperforms a 7B model depends on data quantity and quality, training compute, optimization, and evaluation. No universal example-count threshold or accuracy penalty follows from parameter count alone.

Pitfall: *Assuming distributed training automatically accelerates development.*

More accelerators do not guarantee faster training because communication, smaller local batches, input limits, and imbalance can consume the gain. In an illustrative 8 GPUs scenario, synchronization occupies 30–50 percent of step time and speedup reaches only 4–6× rather than the ideal 8×. The values are hypothetical; the decision requires measured end-to-end time and scaling efficiency.

Fallacy: *Hyperparameters transfer directly from small-scale experiments to large-scale training.*

A learning rate that works at batch size 512 may not transfer to batch size 4,096. The linear scaling rule (Goyal et al. 2017) would increase the rate from 0.1 to 0.8 for this eightfold change, often with warmup, but it is an empirical recipe rather than a requirement or convergence guarantee. Large-scale runs must validate the learning rate, schedule, optimizer state, and effective batch together.

Pitfall: *Treating mixed precision training as a simple toggle without validation.*

The chapter's illustrative V100 inputs imply a 2.4× speedup, but they are not a benchmark. FP16 commonly requires loss scaling (Micikevicius et al. 2017), while BF16 usually does not (Wang and Kanwar 2019; Kalamkar et al. 2019) (see section 8.6.3). Either recipe can change numerical behavior, so throughput and convergence must be validated on the target workload.

Fallacy: *Memory and computation can be optimized independently.*

Memory and compute are coupled: in this illustrative profile, accelerator utilization drops from 90 percent at batch 256 to 60–70 percent at batch 16. Gradient accumulation (effective batch 512, physical batch 64) trades 5 percent efficiency for 8× memory reduction. Tuning these parameters independently extends training time by 20–40 percent (see section 8.6.5).

Pitfall: *Budgeting training memory as if it only contains model weights.*

Sizing training memory from weights alone omits gradients, optimizer state, master weights when used, activations, temporary workspaces, and allocator overhead. In the chapter’s mixed-precision Adam recipe, FP16 weights (2 bytes), FP16 gradients (2), FP32 master weights (4), and two FP32 moments (8) total 16 bytes per parameter before activations. Other optimizers and sharding strategies change this budget. Activation memory then depends on batch size, sequence length, architecture, kernels, and checkpointing; no universal multiple of the weight footprint applies.

Fallacy: *The accelerator is always the training bottleneck.*

Data loading often creates idle time, yet teams optimize computation first. In this illustrative profile, prefetching with pipeline overlap reduces wall-clock time by 47.6 percent (105 min to 55 min) by overlapping data loading with computation (see section 8.6.2). Profile before assuming the GPU is the bottleneck.

Pitfall: *Optimizing kernels before profiling the input pipeline.*

Kernel-level tuning is attractive because GPU utilization is easy to inspect, but a starved accelerator can look like an inefficient accelerator. Before changing precision, rewriting kernels, or adding GPUs, measure dataloader throughput, host preprocessing time, storage wait, and host-to-device transfer overlap. If the accelerator is waiting for batches, the fastest optimization is to feed it reliably rather than make its kernels marginally faster.

8.9 Summary

Training combines algorithms, memory management, and hardware acceleration to transform data into model parameters. At scale, forward and backward propagation coordinate matrix operations, allocations, and gradients under hardware and performance constraints.

Prefetching, mixed precision, FlashAttention, gradient accumulation, and checkpointing address different throughput, memory, and numerical constraints. Match the technique to the binding constraint; when one machine is exhausted, data, model, pipeline, and tensor parallelism trade greater scale for greater complexity.

This co-design shapes large-scale training: matrix patterns drove GPU Tensor Cores, frameworks exposed them through mixed-precision APIs, and techniques such as FP16 training influenced later hardware. FLOP and memory accounting then support optimizer comparison, cost estimation, and decisions about whether more hardware will help or merely move the bottleneck.



Key Takeaways: Why training costs millions

- **Training cost is an iron-law budget:** $T_{\text{train}} = \frac{O}{R_{\text{peak}} \times \eta_{\text{hw}}}$ makes every optimization accountable: reduce work, raise effective throughput, or improve utilization. A change that misses the dominant term only moves cost around the training loop.
- **Memory determines whether an optimizer fits:** Standard mixed-precision Adam uses 8× the FP16 inference-weight memory in the stated representation before activations. Optimizer state and batch-dependent activations—not weights alone—determine whether training fits.
- **Profiles choose the remedy:** Profile, diagnose, fix, and re-profile. Compute-bound jobs need better arithmetic or algorithms; memory-bound jobs need less state or traffic; data- and communication-bound jobs need pipeline or parallelism changes.

- **Precision and IO-aware kernels shift bottlenecks:** FP16 with FP32 accumulation can improve throughput and memory use, while FlashAttention avoids materializing the full $S \times S$ matrix in HBM; realized gains depend on workload and hardware.
- **Checkpointing buys memory with recompute:** Storing fewer activations and recomputing them during backpropagation cuts activation memory 3–4× in the walkthrough, from 35.9 GB to 9 GB at batch 4. Use it when memory, not compute, binds.
- **Composed optimizations can postpone scale-out:** Mixed precision and checkpointing turn 95.7 GB into a modeled 33 GB for batch-4 GPT-2, before temporary workspaces. Use relevant local levers before accepting distributed communication and operational overhead.

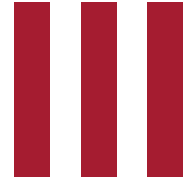
The iron law classifies a slow job as compute-bound (improve arithmetic or algorithms), memory-bound (reduce state or traffic), data-bound (repair the input pipeline), or communication-bound (adjust accumulation or parallelism). Increase batch only when hardware is underfilled. This discipline prevents treating symptoms with more hardware and keeps iteration practical as cost grows.

Training is where the iron law becomes a daily instrument. Mixed precision changes effective throughput and memory traffic; checkpointing exchanges memory for recomputation; scaling adds communication. The engineering task is to identify the dominant term and apply the least costly intervention that changes it.

What's Next: From training to data selection

Training produces learned parameters by spending compute on every example; the chapter's optimized run still requires 33 GB of memory. Once the training loop is efficient, the next question is whether every example earns that cost. Repeated or low-information samples consume the same forward pass, backward pass, optimizer update, and communication as examples that add coverage or correct a weakness.

Chapter 9 therefore turns upstream from execution to workload composition. It examines how redundancy, representativeness, uncertainty, and class coverage determine which examples should enter a finite training budget. A useful subset must reduce work without erasing rare cases or shifting the effective training distribution. When that balance holds, a carefully selected subset can approach full-dataset accuracy while lowering the cost of every epoch and making additional experiments affordable. The optimization target is no longer only how efficiently the system processes an example, but which examples the system should process at all.



OPTIMIZATION AND ACCELERATION

Part III

Optimization Principles

A working model is rarely an efficient one. Part II established how to construct ML systems that respect physical constraints; Part III addresses how to meet real-world demands on time, memory, and energy. Every optimization involves navigating a frontier: improving one metric (accuracy, latency, energy) while managing the cost to others. Every optimization in Part III enacts algorithm-machine co-design by trading mathematical structure for physical feasibility. The principles here define the physics of efficiency—the laws that determine *why* some models are fast and affordable while others are slow and prohibitively expensive.



Principle 5: The Pareto frontier

Invariant: After objective directions are normalized, a feasible point lies on the Pareto frontier exactly when no other point is at least as good on every objective and strictly better on one.

- **Quantization** trades numerical precision for reduced memory footprint.
- **Pruning** trades model capacity for smaller representations and can improve speed when the resulting sparsity or removed structures are supported by the target hardware.
- **Distillation** trades training compute for inference efficiency.

Implication: Systems engineers navigate this frontier to find the operating point appropriate to a specific deployment environment. There is no universal optimum.

Navigating the Pareto frontier requires knowing *which* resource to optimize. Before selecting a technique, engineers must diagnose whether the workload is limited by computation, memory movement, or dispatch latency. Arithmetic intensity supplies the first test.



Principle 6: Arithmetic intensity law

Invariant: Attainable throughput (R) is no greater than the minimum of peak compute (R_{peak}) and DRAM bandwidth (BW) times operational intensity at the cache–DRAM boundary (I) (Williams et al. 2009):

$$R \leq \min(R_{\text{peak}}, I \times \text{BW})$$

Operational intensity and DRAM bandwidth must both be measured at the cache–DRAM boundary; this ideal bound excludes launch, synchronization, and other fixed overheads.

Implication: Adding peak compute does not raise the ideal roofline bound for a bandwidth-bound kernel. Engineers must identify whether the bottleneck is compute, bandwidth, or another system term before selecting an optimization.

Many important ML kernels fall on the memory-bound side of the ridge point, especially low-reuse operations such as embedding lookup, normalization, softmax, depthwise convolution, and small-batch inference paths. Dense matrix multiplications and convolutions can instead be compute-bound when batching and hardware utilization are high. This split is a consequence of the memory wall: processor speed has historically outpaced memory bandwidth, and the cumulative gap has widened over three decades (Wulf and McKee 1995; Gholami et al. 2024). Neural networks, with

their massive weight tensors and uneven temporal locality, are especially vulnerable. The arithmetic intensity law diagnoses *where* a workload sits relative to this wall. The cost of moving data explains *why* the wall is so punishing. Table 8.23 maps each bottleneck type to representative optimizations that address it and to changes that leave the dominant term untouched.

Table 8.23: The Bottleneck Diagnostic: Before optimizing, identify which iron law term dominates. Optimizing the wrong term leaves the dominant bottleneck unchanged and usually yields little or no end-to-end improvement.

If the workload is...	Dominant Term	Optimization That Works	Optimization That is Wasted
Memory-Bound	D_{vol}/BW	Quantization, pruning, batching	Faster accelerator (more FLOP/s)
Compute-Bound	$O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$	Better kernels, Tensor Cores, faster accelerator	More memory bandwidth
Latency-Bound	L_{lat}	Kernel fusion, async dispatch, bounded microbatching under the latency SLO	More FLOP/s or bandwidth alone

Performance is only one cost of memory movement; energy exposes a second constraint.



Principle 7: The energy-movement invariant

Invariant: Total workload energy depends jointly on per-event energy and event counts. In the cited 45 nm reference, moving a 32-bit value from DRAM can cost roughly 100–1,000× more energy per event than a 32-bit floating-point operation, depending on operation type (Horowitz 2014). Here, $e_{\text{DRAM},32\text{-bit}}$ denotes the energy of one 32-bit DRAM access, and $e_{\text{arithmetic},32\text{-bit}}$ denotes the energy of one 32-bit floating-point arithmetic event in the cited 45 nm technology.

$$e_{\text{DRAM},32\text{-bit}} \gg e_{\text{arithmetic},32\text{-bit}} \quad \text{for the cited reference}$$

Implication: When memory movement dominates workload energy, optimization strategies should prioritize **kernel fusion** (avoiding intermediate off-chip traffic) and **quantization** (reducing data size); when computation dominates, reducing operation counts and improving arithmetic kernels remain central.

Even with perfect data locality and optimal bottleneck targeting, a final constraint limits how much speedup any optimization can deliver.



Principle 8: Amdahl's Law

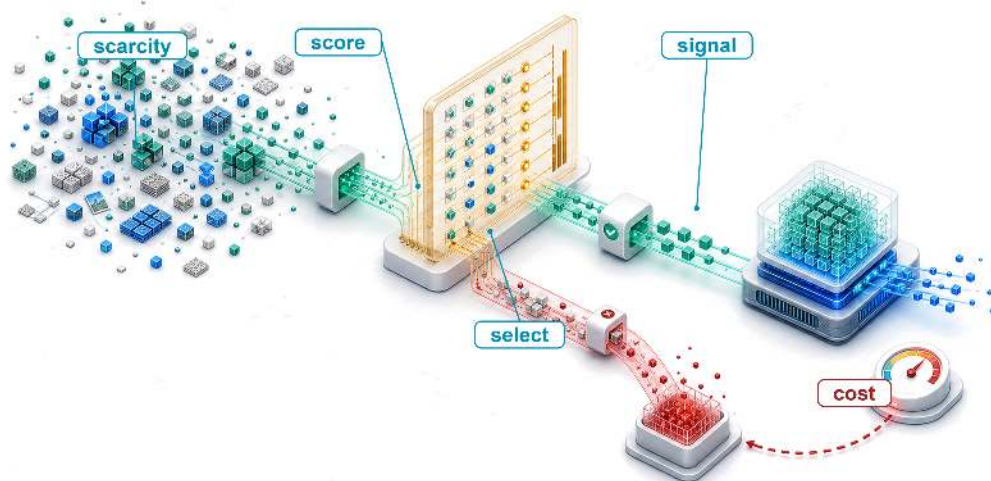
Invariant: The maximum speedup of a system is limited by the fraction of the workload that cannot be accelerated (Amdahl 1967). Here, f_{parallel} is the fraction of baseline execution time accelerated and S_{parallel} is that fraction's speedup; the fixed-work model assumes the remaining fraction is unchanged and the acceleration introduces no additional overhead.

$$\text{Speedup} = \frac{1}{(1 - f_{\text{parallel}}) + \frac{f_{\text{parallel}}}{S_{\text{parallel}}}}$$

Implication: If 95 percent of a model runs 100× faster on a GPU, the total system speedup is capped at ~16.8×. This explains *why* **data loading** and **preprocessing** often become the ultimate bottlenecks in highly optimized systems.

Part III applies these principles systematically through the D·A·M taxonomy—Data, Algorithm, Machine—asking first whether the work is necessary, then whether it can be simplified, and finally how to do it faster (see Appendix A for the full diagnostic framework). Benchmarking closes the loop by testing whether those theoretical savings survive end-to-end execution without violating quality constraints.

Data Selection

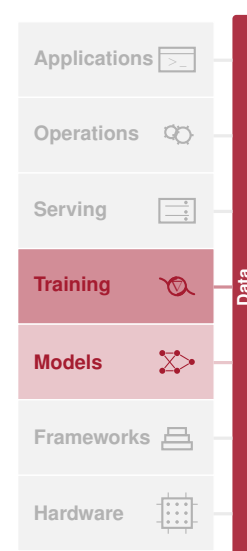


- 9.1 Data Selection Fundamentals
- 9.2 Static Pruning
- 9.3 Dynamic Selection
- 9.4 Self-Supervised Learning
- 9.5 Synthetic Data Generation
- 9.6 Decision Framework
- 9.7 Selection Engineering
- 9.8 Cost Modeling
- 9.9 Distributed Selection
- 9.10 Cross-Layer Interactions
- 9.11 Measurement Framework
- 9.12 Fallacies and Pitfalls
- 9.13 Summary

Purpose

How can a carefully selected subset retain most of the value of a much larger dataset?

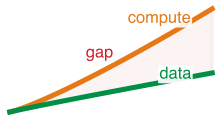
Training pays to acquire, label, score, move, and process examples even though their learning value differs. Large datasets contain redundant, noisy, or target-misaligned samples, while scarce regions may lack the cases a model needs. Data selection turns that heterogeneity into a systems optimization. It can remove avoidable work before training, direct labels and training steps toward informative cases as learning proceeds, or create missing examples when scarcity is the binding constraint. It can also invest once in reusable representation learning when that cost can be amortized across downstream tasks. The objective is therefore not the smallest dataset but the lowest end-to-end cost of reaching target quality without losing rare cases, underrepresented groups, or deployment-relevant coverage. Every strategy adds its own overhead: examples must be scored, generated, shuffled, stored, or validated, and those costs can erase the savings they promise. That judgment must be made end to end, because gains in labeling or training can be erased by selection latency, storage access, or distributed coordination. A method earns its place only when the learning it preserves or creates exceeds the work required to apply it. This distinction matters across repeated experiments, where useful reductions compound, and across adaptation, where the value of an example changes with the model and target distribution. Data engineering established that data is the source code of ML systems; data selection asks which evidence is useful now and which is still missing. In D·A·M terms, it is data-algorithm co-design: shape the training signal so each unit of machine work buys more useful learning.





Learning Objectives

- Explain data selection as data-algorithm co-design that reduces total operations before training begins
- Calculate information-compute ratio to decide whether additional examples improve learning per FLOP
- Compare deduplication, coreset selection, and quality pruning for reducing redundant pretraining data
- Design curriculum, active learning, or synthetic-data strategies for changing data value during training
- Apply the selection inequality to test whether selection overhead beats full-dataset training cost
- Evaluate distributed selection pipelines against storage locality, consistency, and GPU utilization constraints
- Select data-selection investments using ROI, amortization, and compute-optimal frontier diagnostics



Compute supply can outrun high-quality data supply.

1 | Scaling laws: Jared Kaplan and colleagues at Johns Hopkins and OpenAI empirically demonstrated in 2020 that language model loss follows power-law relationships with model size, dataset size, and compute budget over the regimes they studied (Kaplan et al. 2020). Their fitted data exponent was approximately $\alpha = 0.095$ in $\mathcal{L} \propto D^{-\alpha}$. The value is not universal, but the diminishing-return shape makes it possible to reason about when selection becomes more cost-effective than collection.

2 | Data wall: Unlike compute (which scales with capital expenditure) or algorithms (which improve through research), the stock of high-quality human-generated text grows slowly; Epoch AI's updated projections estimated that, under the paper's modeled consumption assumptions, models could use datasets roughly equal in size to the stock of public human-generated text between 2026 and 2032 (Villalobos et al. 2022). The point for systems design is not a fixed calendar deadline; it is that data can become a supply constraint rather than merely an economic one. This constraint directly affects the total operations (O) term: when quality data becomes scarce, additional compute yields diminishing returns regardless of hardware throughput.

9.1 Data Selection Fundamentals

TRAINING pays the iron-law cost for every example it processes, but not every example returns learning signal worth that cost. Data selection gives a clean, well-engineered dataset a systems objective: keep the examples that contribute the most learning per unit of compute. Data engineering makes the dataset reliable through correct labels, consistent schemas, and governed records. Data selection optimizes the dataset's value by extracting maximum learning from minimum samples, directly shrinking the total operations (O) term in the iron law (principle 3). The distinction matters: quality asks whether data is correct, while value asks whether correct data is worth the compute spent processing it.

Increasing the amount of training data has long been a standard way to improve models. Scaling laws¹ (Kaplan et al. 2020; Hoffmann et al. 2022) quantify how model performance can improve with dataset size, and teams have responded by collecting and generating more examples. Accelerator fleets, however, can expand usable compute faster than the supply of novel, high-quality human-generated text and images. Much of the easily accessible public web has already been incorporated into large training corpora, and expert labeling capacity grows slowly. This asymmetry is the **data wall**,² and it shifts attention from collecting more data to extracting more value from existing data. The sections that follow develop the selection methods and cost models needed to make that choice.

Table 9.1 uses an illustrative growth scenario in which GPU compute rises 10× every 3 years while accessible high-quality data grows more slowly. These are scenario inputs, not measured universal rates.

Table 9.1: Scaling Asymmetry in ML Resources: In the illustrated scaling regime, compute grows faster than high-quality data supply, creating a compute-to-data imbalance that makes data selection valuable.

Resource	Growth Rate	Implication
GPU Compute	~10× / 3 years	Hardware throughput can rise quickly in a given era
Training Data (Web)	~2× / 5 years	High-quality web text is finite; much already scraped
Labeled Data	~1.5× / 5 years	Human annotation throughput is inherently bounded
Synthetic Data	Potentially large	Bounded by generator quality (models trained on model-generated data can degrade)

Scaling dataset size eventually encounters the finite stock of high-quality human speech and text. Follow the log-scale trajectory in figure 9.1, observing how foundation model training requirements approach the estimated upper bound of available public data.

The gap between what compute can process and what accessible, high-quality data can support is therefore a systems variable, not a fixed law. In the regime illustrated by table 9.1, compute supply grows faster than accessible high-quality data, so accelerator budgets can outrun the corpus and leave the system compute-rich but data-constrained. Maintaining model relevance requires repeated refreshes of the training corpus as the world changes, and intelligent data selection becomes critical when data quality rather than accelerator time is the binding constraint.

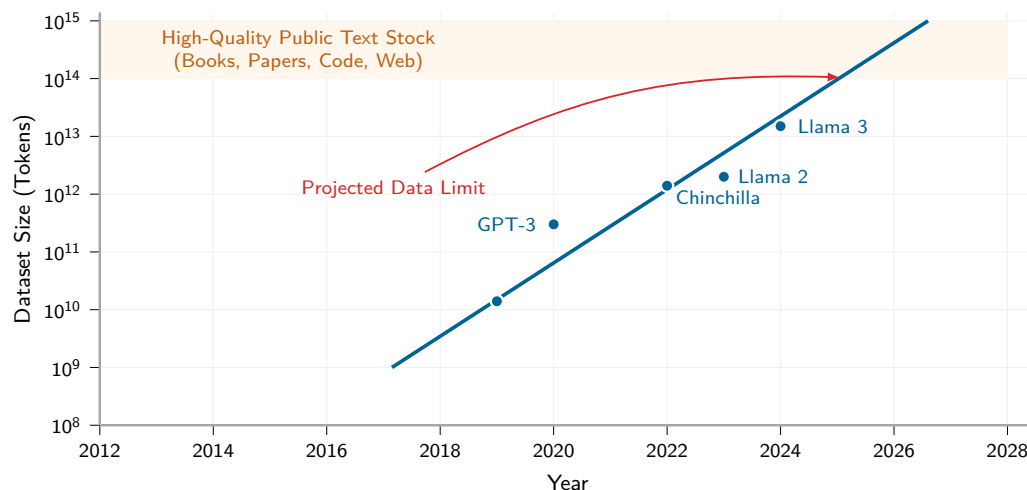


Figure 9.1: Dataset Growth Approaching Limits: Several foundation-model datasets have grown toward estimates of the total stock of high-quality public text. The projection should be read as a dated constraint scenario rather than a fixed forecast: data selection, synthetic generation, multimodal learning, licensing, and filtering policy all change the usable supply.

The compute-data asymmetry can invert the optimization priority. When *data* is abundant and *compute* is scarce, algorithmic efficiency can extract more accuracy from limited GPU cycles. When *compute* is abundant and *quality data* is scarce, data selection can instead extract more learning from each sample. Data selection operates upstream of model and machine optimizations. By pruning redundancy and selecting high-value samples, the workload is reduced before it enters the model or reaches the hardware, directly shrinking the total operations (O) term in the iron law. That is why the systems perspective treats selection as upstream workload reduction rather than a modeling heuristic. For teams whose accelerator budget exceeds their curated corpus, the bottleneck shifts from GPU access to the quality, legality, and diversity of the training data.

The engineering toolkit for intelligent data selection follows a deliberate optimization ordering: first ask whether a sample is worth processing, then ask how to process the remaining workload efficiently. Data selection puts the “largest return first” principle into practice by looking for avoidable work before simplifying or accelerating the work that remains. Static pruning removes low-value samples before a single gradient is computed. Dynamic selection adapts the data diet during training through curriculum learning and active learning. Synthetic generation creates high-value samples through augmentation, simulation, or teacher-generated examples when real data runs short.

Each stage can increase the information density of the data that reaches the model when measured quality gains justify its overhead, and together they form a complementary toolkit: pruning reduces *what* the pipeline contains, selection focuses *how* the pipeline uses it, and synthesis expands *what* the pipeline can access. Comparing these techniques requires a systems definition of data selection and a measurable account of its effectiveness.

9.1.1 Defining data selection

The three-stage pipeline needs a quantity that lets engineers compare samples before they spend accelerator time on them. A duplicated image, a mislabeled record, and a rare boundary case may all cost the same forward and backward pass, but they do not contribute the same learning signal. Data selection therefore starts by making sample value explicit relative to compute cost.

The information-compute ratio measures that value as learning signal gained per unit of training compute, formalized in section 9.1.3.



Definition 9.1: Data selection

Data selection is the process of maximizing the information-compute ratio (ICR) of a training dataset.

1. **Significance:** It identifies data worth retaining, prioritizing, or generating for the target objective, reducing the total operations (O) when redundant or noisy samples can be excluded without losing required coverage.
2. **Distinction:** Unlike data engineering, which focuses on the cleanliness and consistency of data, data selection focuses on the informativeness and diversity of the samples.
3. **Common pitfall:** A frequent misconception is that more data is always better. Additional low-quality data can yield less improvement than a much smaller, carefully selected set, so retained volume must be evaluated together with coverage and target quality.

To make this concrete, consider training an autoregressive language model in the GPT-2/Llama lighthouse family from section 6.1.1. The scenario uses a 70-billion-parameter model to make the compute-data gap visible at large scale.

The compute budget (10,000 H100 GPUs for 3 months) represents roughly \$86.4M at the chapter's cloud-training price anchor and can process about 73.2T tokens at 40 percent sustained model FLOPs utilization. Estimates of quality- and repetition-adjusted public human-generated text are on the order of 300T tokens, far larger than a single curated web corpus. The practical bottleneck is narrower: how much of that stock is accessible, legally usable, high quality, deduplicated, and useful for the target distribution. If a team has only a 5T filtered corpus ready for use, the compute budget can already process it roughly 14.6× over. At that point, the cluster outstrips the available unique signal. An engineering team can repeat epochs over the same corpus, but gradient updates on familiar tokens yield diminishing loss reductions. It can lower quality thresholds to admit uncurated web text, but noisy and misaligned tokens degrade downstream evaluation. Or it can invest in data selection—refining quality filters, constructing difficulty curricula, and synthesizing missing edge cases—to raise the learning return per FLOP. When compute capacity outpaces high-signal data, selection delivers higher return on capital than provisioning additional accelerators.

The opportunity applies across model architectures, though the bottlenecks and safe reduction differ. Unlike the compute-bound ResNet-50 lighthouse, GPT-2/Llama models can be memory-bandwidth-bound during inference and compute-bound during training. When a validated selection method removes examples or tokens without adding compensating steps, fewer samples mean fewer training FLOPs. The appropriate framing is therefore systemic rather than purely statistical.

9.1.2 Systems perspective

The data wall dictates the physical limits of scaling uncurated datasets; a systems perspective determines how to manage that constraint across the hardware stack. The conventional *ML framing* evaluates data selection through statistical sample complexity and generalization theory, asking how many samples are required to achieve a given test error. That formulation treats every sample as a zero-cost mathematical abstraction, ignoring the physical resource cost of data ingestion, transport, and execution.

A data-selection *systems framing* asks instead how to minimize the total resource cost of achieving target accuracy across the entire ML lifecycle. This framing shifts attention from theoretical sample efficiency to concrete resource consumption, as table 9.2 illustrates.

Table 9.2: ML vs. Systems Perspectives on Data Selection: The ML framing optimizes sample complexity; the systems framing optimizes total resource cost across the pipeline.

ML Framing	Systems Framing
“Fewer samples for same accuracy”	“Fewer FLOPs for same accuracy”
“Better generalization”	“Lower training cost (time, money, energy)”
“Sample complexity bounds”	“End-to-end resource efficiency”
“Learning theory”	“Cost engineering”

This framing reveals optimization leverage across the execution stack. In the D·A·M taxonomy, data selection operates at the data layer to eliminate physical work before algorithmic and hardware optimizations execute it, as formalized by the iron law of ML systems (section 1.7).

Systems Perspective 9.1: Data selection and the iron law

In the iron law of ML systems ($T = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}}$), data selection reduces the number of examples or tokens that enter the workload. Later model-level optimizations reduce operations per forward/backward pass, and machine-level optimizations increase R_{peak} (peak throughput) and η_{hw} (utilization). Selection removes work before those downstream optimizations execute it.

This makes data selection multiplicatively valuable in the iron law: when all three optimization layers act on the same bottleneck, a 2× reduction in dataset size with 2× fewer operations per sample and 2× higher effective throughput yields 8× total cost reduction, not 6×.

Holding epoch count, batch size, and per-sample work fixed, a 50 percent reduction in dataset size directly halves the required forward passes, backward passes, and gradient updates. For the \$100M scenario, this reduction yields \$50M in direct compute savings, provided the training schedule does not compensate by adding optimization steps.

These compute savings propagate throughout the hardware hierarchy. By decreasing total data volume (D_{vol}), deduplication and coreset selection alleviate storage bandwidth demands, network fabric ingress, and host-to-accelerator PCIe transfers that otherwise stall tensor cores in I/O wait states. Upstream, active learning curbs annotation budgets by directing human labeling exclusively to high-entropy or high-gradient samples.

Yet selection is not free: every selection strategy incurs its own runtime overhead to score, filter, shuffle, or coordinate samples. An online selection algorithm that executes additional forward passes to rank sample difficulty can easily consume more accelerator cycles than it saves if its selection overhead exceeds the subsequent reduction in training time. A data selection pipeline earns its place only when the end-to-end cost of filtering plus training on the curated subset remains strictly lower than training on the uncurated whole.

9.1.3 Information-compute ratio

The systems framing established in section 9.1.2 calls for a quantitative metric. Data selection balances an efficiency frontier between accuracy and cost: retaining every example preserves coverage but expends compute on redundant signal, whereas over-aggressive pruning saves FLOPs at the expense of rare-slice recall. Navigating that trade-off requires quantifying the marginal learning signal each sample yields per unit of hardware execution. This metric is the **information-compute ratio** (ICR).

Figure 9.2 recasts the D·A·M taxonomy as an optimization map, with data selection playing the role of input optimization: reducing total workload before it enters the model or hardware. The model side asks how much math each example requires. The machine side asks how quickly the hardware can execute that math. The data side asks whether the example should be processed at all. The three edges of the triangle capture the dominant bottlenecks: *compute bound* describes systems limited by arithmetic throughput, *I/O bound* describes systems limited by data movement, and *sample efficiency* describes systems limited by the information content of training data.

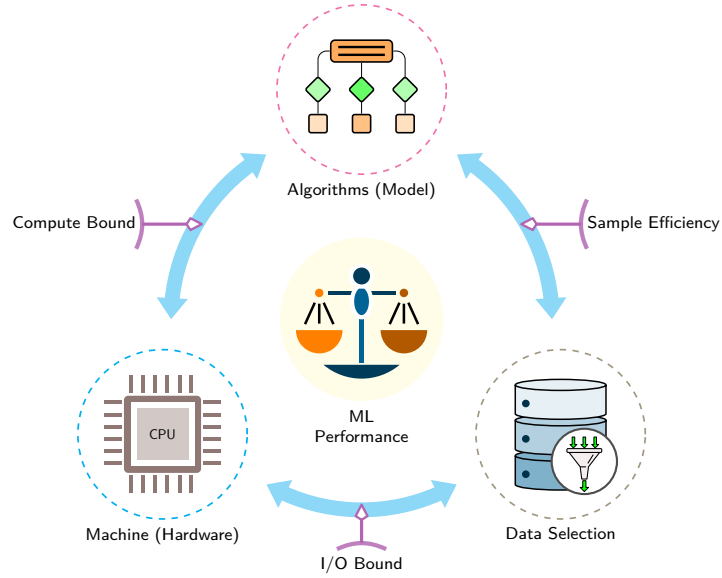


Figure 9.2: The D-A-M Taxonomy as an Optimization Map: Machine learning performance is organized around Algorithms (Model), Data Selection, and Machine (Hardware). The triangle's edges label the compute-bound, I/O-bound, and sample-efficiency relationships, while data selection reduces workload before model or hardware execution.

Formally, the ratio relates marginal information content to marginal compute:

$$\text{ICR} = \frac{\Delta I}{\Delta \text{FLOPs}}$$

A higher ICR indicates that each FLOP of training buys more learning signal. Because the numerator cannot be observed directly during production runs, systems pipelines rely on operational proxies: validation improvement per unit compute, area under the learning curve, loss reduction on held-out evaluations, uncertainty or gradient-based sample scores, and coverage metrics on deployment-critical slices. These proxies translate the abstract information-theoretic objective into concrete selection signals before a training run exhausts its compute budget.

9.1.4 The ICR frontier: When data becomes a tax

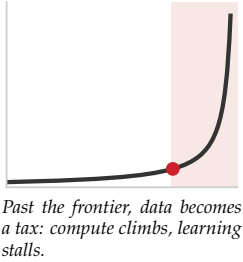
The information-compute ratio is not constant; it follows a law of diminishing returns. The **ICR frontier** is the point where the marginal learning signal from additional data drops toward zero.

To illustrate diminishing returns, let $I(D)$ be the information content of a dataset of size D and assume $I(D) \propto \log D$. This is a simplified analytical model rather than a universal law. If compute cost scales linearly with the per-sample operation count O_{sample} , then $C(D) = O_{\text{sample}} \cdot D$, and the resulting ICR follows equation 9.1:

$$\text{ICR}(D) = \frac{\frac{d}{dD} I(D)}{\frac{d}{dD} C(D)} \approx \frac{1/D}{O_{\text{sample}}} = \frac{1}{O_{\text{sample}} \cdot D} \quad (9.1)$$

Under this illustrative model, the $1/(O_{\text{sample}} \cdot D)$ decay creates the data wall. Beyond a workload-dependent frontier, additional data may provide little learning while still adding compute. In this regime, data becomes a **data tax** that inflates the O term of the iron law without a commensurate improvement in the accuracy numerator of the RoC (return on compute, see section 1.7.1). The knee is a practical trade-off between target performance and marginal cost, not the mathematical maximum of ICR in this model.

Data selection turns the total operations (O) term in the iron law from a fixed constant into a variable. A $2\times$ improvement in measured ICR halves the FLOPs required to reach the same measured gain. It produces the same ideal time reduction as a $2\times$ throughput increase only when the workload



Past the frontier, data becomes a tax: compute climbs, learning stalls.

is compute-bound and the saved work lies on the critical path. ICR focuses specifically on compute; the cost-modeling framework in section 9.8 extends the same reasoning to acquisition, labeling, and storage costs.

A random batch of raw data can have low ICR when it contains redundant, noisy, or already-mastered examples. High-efficiency data pipelines (figure 9.3) can improve ICR through three stages: static pruning before training, dynamic selection during training, and synthetic generation on demand. To illustrate, consider computing ICR on a concrete coreset selection task: a deliberately selected subset intended to preserve the full dataset’s learning signal. Section 9.2.2 defines the EL2N and GraNd scoring methods used to build such subsets, and section 9.11 provides the complete measurement framework for evaluating these efficiency gains, including a compute-optimal frontier diagnostic that can suggest whether training is data-limited or compute-limited.

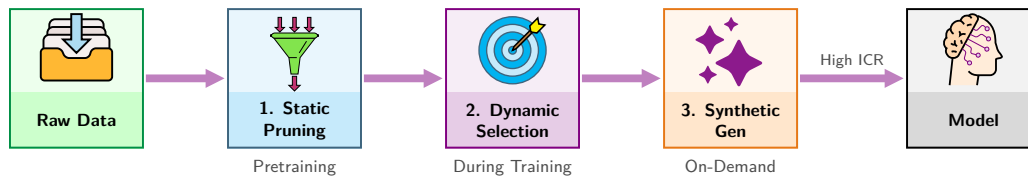


Figure 9.3: The Data Selection Pipeline: A structured approach to increasing data value. Raw data is first pruned (Static Pruning), then selected during training (Dynamic Selection), and finally synthesized (Synthetic Generation). Each stage can increase the Information-Compute Ratio (ICR) when its quality gain justifies its compute cost.

✓ Checkpoint 9.1: Data selection efficiency

The goal of data selection is to maximize the ICR.

Metric checks:

- ☐ **ICR application:** given two training runs with identical accuracy gains but different compute budgets, can you determine which had higher ICR?
- ☐ **Data efficiency:** under a fixed evaluation protocol, can you explain when a 50 percent smaller dataset with $2\times$ higher ICR yields the same measured gain for half the training FLOPs?

Pipeline check:

- ☐ **The three stages:** can you map static pruning, dynamic selection, and synthetic generation to the training lifecycle?

The practical question is how large the efficiency gap could become on a workload where dataset size, model cost, and selection strategy interact with concrete FLOP budgets. The following ImageNet-scale ResNet-50 scenario uses assumed accuracy gains to illustrate the calculation; it is not a reported EL2N result.

📝 Napkin Math 9.1: Computing ICR: Coresets

Problem: Training the ResNet-50 lighthouse model from section 6.1.1 on ImageNet for one epoch costs 3.15×10^{16} FLOPs. Under the scenario’s assumed accuracy gains, how would retaining 50 percent of the data change the information-compute ratio? The calculation holds the per-sample work fixed and does not include selection overhead.

Setup:

- Dataset: ImageNet (1.28M)
- Model: ResNet-50 lighthouse (~ 8.2 GFLOP per forward pass, roughly 24.6 GFLOP for a forward plus backward training step, depending on implementation)

- One epoch: $1.28\text{M} \times 24.6 \text{ GFLOP} = 3.15 \times 10^{16} \text{ FLOPs}$
- Assumed accuracy improvement per epoch (early training): 5 percentage points

Random selection (baseline):

- Process all 1.28M samples uniformly
- Accuracy gain: 5 percentage points
- $\text{ICR}_{\text{random}} = 5 \text{ percentage points} / (3.15 \times 10^{16} \text{ FLOPs}) = 1.6 \times 10^{-16} \text{ per FLOP}$

EL2N coreset (Error L2-Norm, a training-dynamics score developed in section 9.2.2; 50 percent of data):

- Process 640.6K selected samples
- Assumed accuracy gain: 4.5 percentage points (90 percent of the full-data gain)
- Compute: $640.6\text{K} \times 24.6 \text{ GFLOP} = 1.6 \times 10^{16} \text{ FLOPs}$
- $\text{ICR}_{\text{coreset}} = 4.5 \text{ percentage points} / (1.6 \times 10^{16} \text{ FLOPs}) = 2.9 \times 10^{-16} \text{ per FLOP}$

Systems insight: Under these assumptions, the selected subset achieves 1.8× higher ICR with a 0.5 percentage points smaller accuracy gain. A real decision requires measured gains, selection overhead, and coverage checks.

Improving the information-compute ratio follows a staged optimization order: prune static waste upfront, steer dynamic batches during execution, and synthesize missing coverage on demand. The first lever acts entirely before model execution commences.

9.2 Static Pruning

Static pruning and pretraining filtration eliminate uninformative examples upstream of the training loop. By purging redundant and corrupt records before computing a single gradient, static filtration shrinks the operation count (O) without modifying model architectures or execution kernels.

9.2.1 The case for smaller datasets

Empirical scaling studies show that dataset volume alone does not dictate model accuracy: large-scale corpora harbor substantial redundancy. In many regimes, a significant fraction of training examples contributes negligible gradient signal while consuming full forward-backward compute.

On CIFAR-10, Paul et al. (2021) report that EL2N pruning removed half of the training data without reducing accuracy. On ImageNet-1K, Sorscher et al. (2022) report that a self-supervised prototype metric discarded 20 percent of the data without sacrificing performance. The pattern extends to language modeling, where web-scraped corpora like The Pile³ and C4⁴ contain enough duplicate and templated content to make deduplication a systems issue. In datasets studied by Lee et al. (2022), approximate near-duplicate removal affected 3.04 percent of C4 and 13.63 percent of RealNews, and deduplicated training reduced memorization while preserving or improving perplexity.

The reported gains are benchmark-specific. Pruning effectiveness depends on intrinsic dataset redundancy, the selection algorithm, and target model capacity; production deployments require task-specific validation before committing to aggressive filtration.

Individual data points need not provide equal value for training. This heterogeneity follows in part from how classifiers learn decision boundaries. Many samples fall far from a boundary: a picture of a dog in good lighting may become easy once the model has learned its dominant features, while ambiguous and underrepresented cases can remain informative. Label quality also affects data requirements. In a simplified binary symmetric-noise model, a label-flip rate ρ reduces an effective-sample-size factor to $(1 - 2\rho)^2$; at $\rho = 10\%$, matching the clean-data bound requires about $1/(1 - 2\rho)^2 \approx 1.56\times$ as many samples. The accompanying notebook uses a separate toy learning-curve model to illustrate another **data quality multiplier**; neither multiplier is universal.

³ | **The Pile:** An approximately 886 GB English text corpus (reported as 825 gibibytes by the source) aggregating twenty-two sub-datasets, including PubMed, ArXiv, GitHub, Project Gutenberg, Common Crawl, Stack Exchange, Wikipedia, and USPTO patents (Gao et al. 2020). Its multi-source design makes it a useful example of data diversity: each source family has a different duplication, quality, and domain-coverage profile, so selection pipelines must preserve coverage while removing redundant text.

⁴ | **C4 (Colossal Clean Crawled Corpus):** C4 applies filtering to Common Crawl data, including language detection, deduplication of repeated three-sentence spans, and removal of pages that fail heuristic quality checks, producing approximately 750 GB of cleaned English text (Raffel et al. 2020). Filtering has its own compute cost and can remove useful material, so its ROI and quality effects must be measured rather than inferred from corpus size alone.

Napkin Math 9.2: The data quality multiplier

Problem: How much more noisy data does it take to reach the same target error as clean data?

Math: In this illustrative model, clean-data error scales as $\mathcal{O}(1/D)$, so $D_{\text{clean}} \propto 1/\epsilon$. Noisy-data error scales as $\mathcal{O}(1/\sqrt{D})$, so $D_{\text{noisy}} \propto 1/\epsilon^2$. For target error $\epsilon = 0.01$ (1 percent):

- $D_{\text{clean}} \approx 100$
- $D_{\text{noisy}} \approx 10,000$

Result: Under these assumptions, noisy data requires 100× more samples at the target error.

Systems insight: Here, cleaning data acts as a 100× compute accelerator; the real multiplier is workload-dependent.

9.2.2 Coreset selection algorithms

Identifying which samples to retain requires systematic selection criteria. **Coreset selection**⁵ turns the static pruning decision into a coverage problem: keep the smallest subset that preserves the statistical properties of the entire dataset.

The systems decision is where to spend the selection budget: on cheap coverage metrics that preserve distributional structure, or on costlier training-dynamics scores that better target the decision boundary. The decision also needs a guardrail before any score is applied: classes, demographic groups, time windows, and rare failure modes that matter at deployment require minimum representation, because the highest-average-ICR subset can still remove the examples that define production risk.

Geometry-based methods select samples that cover a chosen representation without requiring target-model training. The k -Center algorithm,⁶ a facility-location-style objective, selects samples that reduce the maximum distance from any point to its nearest selected center under the chosen embedding and metric.

Herding takes a different approach. Welling’s original procedure iteratively constructs representative pseudo-samples whose feature statistics approximate observed moments (Welling 2009). Related moment-matching objectives can guide subset selection from a fixed pool. These methods are computationally attractive because they operate on feature representations, but their scores are not inherently label-aware.

Training-dynamics methods use a proxy model to identify examples that may be important. GraNd (Gradient Normed) and EL2N (error L2-norm)⁷ score samples by gradient magnitude or prediction error early in training (Paul et al. 2021). Theoretically, sample valuation can be framed via *influence functions*, which measure how removing a sample changes validation loss using an inverse empirical Hessian ($H^{-1}\nabla_{\theta}\mathcal{L}$); however, computing H^{-1} is computationally intractable ($\mathcal{O}(P^2)$ to $\mathcal{O}(P^3)$) for modern parameter counts P . GraNd circumvents this by scoring samples using expected gradient norm ($\|\nabla_{\theta}\mathcal{L}\|_2$), while EL2N observes that early in training, the gradient norm with respect to linear layers is strongly correlated with output prediction error ($\|p(x) - y\|_2$). This enables practitioners to identify high-uncertainty boundary samples using a single forward pass without computing backward-pass gradients. High scores identify examples the proxy finds difficult, but they can also identify noise or unrepresentative samples and therefore need coverage and quality checks. The same work reports useful score transfer across several architectures, which can enable less expensive proxy-based selection. Forgetting Events⁸ tracks how often a sample is correctly classified and later misclassified during training (Toneva et al. 2019).

Each algorithm in table 9.3 balances upfront scoring computation against downstream training reduction. Because geometry-based and training-dynamics methods optimize different proxies, their relative ranking depends on dataset scale, feature representations, target model capacity, and scoring budgets; a score is computationally justified only when downstream training savings exceed evaluation overhead.

⁵ | **Coreset (core set):** In computational geometry, a small subset can approximate a specified geometric objective within a controlled error factor (Agarwal et al. 2005). Machine learning adapts this idea as a selection principle, but a guarantee for an embedding and distance objective does not by itself guarantee neural-network accuracy. The retained subset must still be validated on the target task.

⁶ | **k-center algorithm:** Its greedy strategy iteratively picks the point farthest from the existing centers under a chosen embedding and metric. Sener and Savarese use this core-set framing for convolutional-neural-network active learning (Sener and Savarese 2018). The resulting coverage bound applies to that geometric objective, not directly to downstream accuracy or rare-class preservation.

⁷ | **EL2N (error L2-norm) and GraNd (gradient normed):** These methods score samples based on their error or gradient norm early in training, identifying samples the model finds most difficult. The practicality of this approach relies on *transferability*, where scores from a small proxy model can guide data selection for a much larger target model. For instance, a proxy trained for five epochs can generate scores to curate a dataset for a full 90-epoch production training run (Paul et al. 2021).

⁸ | **Forgetting events:** This method identifies valuable examples by tracking when the model “forgets” them—transitioning from a correct to an incorrect classification during training. The central trade-off is the high cost of this analysis, which requires a full training run. However, the resulting importance scores transfer reliably from small proxy models to large target models (for example, ResNet-18 to ResNet-50), which is precisely what makes the “inexpensive proxy-based selection” strategy viable (Toneva et al. 2019).

Table 9.3: Coreset Selection Algorithm Comparison: D = dataset size, K = coreset size. Geometry-based and training-dynamics methods optimize different proxies, so their quality ranking depends on the dataset, representation, target model, and scoring budget.

Method	Compute Cost	Requires Training	Best For	Limitation
k-Center	$\mathcal{O}(D^2)$ or $\mathcal{O}(DK)$	No	Coverage, exploration	Ignores label information
Herd	$\mathcal{O}(DK)$	No	Distribution matching	Assumes Gaussian-like
GraNd	$\mathcal{O}(\text{epochs} \times D)$	Yes (few epochs)	Decision boundaries	Requires proxy training
Forgetting	$\mathcal{O}(\text{full training})$	Yes (full)	Hard examples	Expensive to compute
EL2N	$\mathcal{O}(\text{epochs} \times D)$	Yes (few epochs)	Uncertainty sampling	Best with proxy model

While geometry-based methods prioritize uniform coverage across feature representations, training-dynamics methods focus capacity on ambiguous boundary instances. Figure 9.4 illustrates an uncertainty-based strategy alongside uniform random sampling: where random selection disperses samples across the feature space, uncertainty sampling concentrates the selection budget in the boundary region where model predictions remain unsettled.

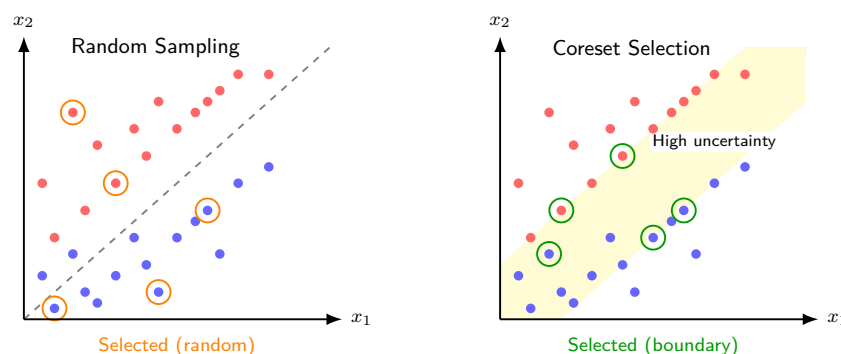


Figure 9.4: Illustrative Uncertainty-Based Selection: Both panels select five samples, isolating allocation rather than budget. Random sampling distributes those choices across the feature space; the illustrative uncertainty strategy concentrates them in the shaded band around the current decision boundary.

EL2N with a small proxy model offers one practical balance between scoring cost and selection quality. The approach trains a lightweight model for a few epochs, computes EL2N scores, and selects a subset subject to coverage and quality checks. The proxy must provide rankings that transfer to the target model, and the investment pays off only when downstream training savings exceed selection overhead. A concrete scenario illustrates this workflow without asserting that its 10 percent subset preserves accuracy.



Example 9.1: Coreset selection in practice

Scenario: An engineering team aims to select a 100,000 coreset (10 percent) from 1 million training images for faster model iteration.

Diagnosis: Naive random subsampling drops rare classes and boundary examples. Training a lightweight proxy model for 5 epochs calculates EL2N error scores, retaining high-uncertainty samples near the decision boundary.

Systems lesson: Proxy-based coreset selection replaces some full-data passes with targeted scoring compute. It accelerates full-model training only when the proxy ranking transfers and coverage checks preserve rare classes and edge cases.

Listing 9.1 demonstrates how to compute EL2N scores and select a coreset using a lightweight proxy model. The mechanism spends a small amount of probe compute to identify high-uncertainty samples near the decision boundary, then trains the full model on the retained subset rather than on

redundant easy examples. The example requires an unshuffled dataloader so each score position remains aligned with its dataset index.

Proxy scoring turns uncertainty into reusable selected indices: `compute_el2n_scores` measures which samples still confuse a briefly trained model, and `select_coreset` retains those high-information examples for the full training run.

Listing 9.1: EL2N-Based Coreset Selection: The `compute_el2n_scores` function trains a proxy model for a few epochs, then measures prediction error via L2 distance from one-hot labels. High scores identify samples the proxy finds difficult, including potentially noisy examples. The `select_coreset` function illustrates ranking by this score; production use also requires coverage, noise, and target-quality validation.

```
def compute_el2n_scores(model, dataloader, num_epochs=5):
    """Compute EL2N scores.

    Returns L2 norm of (prediction - one_hot_label).
    """
    # Train proxy model for a few epochs to get meaningful predictions
    train_proxy(model, dataloader, num_epochs)

    scores = []
    model.eval()
    for x, y in dataloader:
        logits = model(x)
        probs = softmax(logits, dim=1)
        # One-hot encode labels
        one_hot = zeros_like(probs).scatter_(1, y.unsqueeze(1), 1)
        # EL2N score = L2 distance from confident prediction
        el2n = (probs - one_hot).norm(dim=1) # High = uncertain
        scores.extend(el2n.tolist())
    return scores

def select_coreset(scores, dataset, fraction=0.1):
    """Select top-k highest-scoring (most uncertain) samples."""
    k = int(len(dataset) * fraction)
    # Sort by score descending (highest uncertainty first)
    indices = argsort(scores, descending=True)[:k]
    return Subset(dataset, indices)

# Illustrative 10% subset; validate accuracy and coverage before use
scores = compute_el2n_scores(proxy_model, full_loader)
coreset = select_coreset(scores, full_dataset, fraction=0.1)
train_full_model(model, coreset)
```

9.2.3 Data deduplication

While coreset selection identifies which samples to keep based on their informativeness, a complementary approach targets duplicate samples that add compute without adding learning signal. Deduplication can provide immediate efficiency gains, especially for exact and near-duplicates, and requires no model training. This makes it one of the most accessible optimizations in data selection, but near-duplicate thresholds must be validated so the pipeline does not remove useful distributional signal.

The simplest form of deduplication (introduced as a data engineering pipeline stage in section 4.6, and here elevated to an optimization lever) uses hash-based methods for exact matches. By computing a cryptographic hash (MD5 or SHA-256) for each sample and removing those with identical hashes, practitioners can eliminate byte-for-byte duplicates that inevitably accumulate in large web-scraped corpora. This process is computationally cheap, scaling linearly with dataset size, and can be parallelized trivially.

Near-duplicate detection addresses content that differs at the byte level while retaining substantial token- or character-shingle overlap. For text, MinHash⁹ with Locality-Sensitive Hashing¹⁰ (LSH) approximates Jaccard similarity¹¹ efficiently. In practice, the pipeline converts each document into a

⁹ | **MinHash:** Invented by Broder (1997) to detect duplicate web pages for AltaVista, the algorithm creates compact signatures using random hash functions such that similar documents produce similar signatures with high probability. Each signature compresses a document to a fixed-size sketch, enabling pairwise similarity estimation in $\mathcal{O}(s)$ time for sketch size s .

¹⁰ | **Locality-sensitive hashing (LSH):** LSH hashes Min-Hash signatures into buckets so that similar signatures are more likely to collide. Candidate generation avoids an exhaustive all-pairs comparison, while runtime and memory depend on signature width, banding, bucket sizes, and the number of candidates that require verification.

¹¹ | **Jaccard similarity:** Defined as $|A \cap B| / |A \cup B|$, ranging from 0 (disjoint) to 1 (identical). For deduplication, the metric compares sets of shingles across documents of different lengths. The threshold must be calibrated for the shingle definition, corpus, and downstream quality because no universal cutoff separates duplicates from legitimately related documents.

set of overlapping character or word k -shingles, compresses these sets into fixed-length MinHash signatures where the probability of hash collision equals Jaccard similarity, and partitions the signatures into b bands of r rows using LSH. Documents matching across all rows in any single band collide in the same bucket, restricting expensive similarity verification to likely candidates and avoiding an impossible $\mathcal{O}(N^2)$ all-pairs comparison across billions of web documents. It can detect lightly edited content with high overlap, but it does not establish semantic paraphrase equivalence.

For images, perceptual hashing produces signatures robust to minor transformations like resizing and compression, identifying visually similar candidates stored in different formats. Embedding-based similarity can retrieve semantic near-neighbor candidates by computing dense representations (CLIP¹² (Radford et al. 2021) for images, sentence transformers for text), though proximity does not itself prove duplication and the approach incurs higher computational overhead.

Deduplication can reduce repeated-content memorization and avoid spending FLOPs on exact or near duplicates. It can also alter the corpus’s intentional empirical weighting, so thresholds and downstream quality must be validated rather than assuming that every duplicate is harmful.

The systems impact of deduplication shifts when input examples index persistent model state rather than streaming through stateless compute layers.



Lighthouse 9.1: DLRM and embedding deduplication

The DLRM lighthouse model from section 6.1.1 is memory capacity-bound, with embedding tables consuming terabytes of storage for billions of user/item IDs. Much of this capacity is wasted on *cold embeddings*, IDs that appear rarely in training data.

Data selection for DLRM can include interaction deduplication and embedding pruning for cold IDs. Removing repeated interactions reduces training examples but shrinks an embedding table only when IDs disappear or the allocation, hashing, or sharing policy changes.

9.2.4 Data pruning by quality

Deduplication removes redundant samples, but a third category of problematic data remains: samples that actively harm learning. Quality-based pruning eliminates samples that either contribute no meaningful signal or introduce contradictory information that confuses the optimization process.

Label error detection can be a high-impact form of quality pruning. Tools like Cleanlab identify samples where the assigned label may be incorrect based on model confidence patterns across training. A sample that the model consistently predicts as class A but is labeled class B may be a hard boundary case, a distribution shift, a model blind spot, or an annotation mistake. The score should trigger review rather than automatic deletion; confirmed errors can then be corrected or removed.

Outlier removal addresses a different pathology: samples far from any cluster center in feature space. Such points can indicate noise, annotation errors, or corruption, but they can also be the rare edge cases that define deployment risk. The key is distinguishing informative outliers from invalid samples, with conservative thresholds and slice-level review before removal.

Low-information filtering applies domain-specific heuristics to remove samples that lack sufficient signal for learning. For text corpora, this often means removing high-perplexity garbled text (perplexity is a language model’s measure of how surprising a text is, so high values flag incoherent strings) and, in some pipelines, low-perplexity boilerplate or repetitive text. For image datasets, filtering targets blurry, corrupted, or near-uniform samples that provide little visual information.

Beyond heuristics and perplexity, industrial pretraining pipelines rely heavily on *classifier-based filtering*: training a lightweight, high-throughput linear classifier (such as fastText) using curated, high-quality reference text (e.g., Wikipedia, books, and scientific papers) as positive examples and uncured web dumps as negative examples. This classifier assigns each scraped document a continuous quality score, allowing pipelines to filter out millions of low-quality web pages at gigabyte-per-second ingestion speeds without the heavy computational burden of evaluating a full neural language model.

Together, these three static pruning techniques—coreset selection, deduplication, and quality filtering—can reduce work before training. With epoch count, batch size, and per-sample work

¹² | **CLIP (Contrastive Language-Image Pretraining)**: Pretrained on 400 million image-text pairs, CLIP maps semantically related images and text into a shared embedding space (Radford et al. 2021). Embeddings can retrieve candidates for review, but semantic proximity is not proof that two samples are duplicates. Generating embeddings also costs substantially more than computing perceptual hashes, with the ratio depending on model and hardware.

fixed, a 50 percent dataset reduction means 50 percent fewer forward passes, backward passes, and gradient updates. Training schedules that add compensating epochs or steps reduce that saving.

Static pruning answers a question about *what* to keep, but it treats the answer as fixed. Once the pruned dataset is set, every epoch trains on the same subset. A sample’s usefulness, however, can change as the model learns: examples that challenge an undertrained model may become easy after sufficient gradient updates. Dynamic selection techniques address this limitation by adapting the training data at each stage based on what the model has already mastered.

9.3 Dynamic Selection

A sample’s marginal learning signal is non-stationary across training. Early in optimization, broad coverage provides stable gradients that establish general feature representations. As parameters converge, the gradient norms of mastered examples decay toward zero. Executing forward and backward passes on these samples continues to consume accelerator execution cycles and high-bandwidth memory (HBM) transfers while producing negligible parameter updates, driving their effective ICR toward zero. Dynamic selection mitigates this waste by adapting batch composition to the model’s current state. This shifts selection from an offline preprocessing step into the active execution loop, modifying the operation count (O) at runtime while requiring data loaders to balance scoring overhead against accelerator throughput (T).

9.3.1 Curriculum learning: Easy to hard

The first dynamic selection technique, **curriculum learning**¹³ (Bengio et al. 2009; Soviany et al. 2022), structures the order in which data is presented to the model. Instead of random shuffling, it starts with simpler examples and gradually introduces more complex ones, mirroring how humans learn by mastering basics before advancing to harder material.

Curriculum learning can change the optimization trajectory by controlling which gradients dominate at each stage. Easy examples may provide stable early signals, while examples scored as hard can contain useful boundary information, label noise, or outliers. An easy-to-hard schedule can smooth training on some workloads, but its benefit depends on the difficulty score, pacing rule, model, and data; table 9.5 therefore treats the savings as illustrative rather than guaranteed.

Implementing a curriculum requires two components: a difficulty scorer that ranks samples, and a pacing function that controls how quickly hard samples are introduced. A common choice is linear pacing:

$$\text{samples}_{n_{\text{epoch}}} = \text{sort_by_difficulty}[:, N_{\text{samples}} \cdot \min(1, n_{\text{epoch}}/N_{\text{warmup}})]$$

where n_{epoch} is the current epoch, N_{samples} is the total dataset size, and N_{warmup} is the number of warmup epochs before the full dataset becomes available. Early epochs train on the easiest $N_{\text{samples}} \cdot (n_{\text{epoch}}/N_{\text{warmup}})$ fraction; after warmup, training proceeds on the full dataset. From a systems standpoint, dynamically re-sorting multi-terabyte datasets epoch-by-epoch destroys sequential I/O read-ahead and stalls accelerators on storage latency. Production data loaders implement curricula by pre-sorting data offline into discrete difficulty-tiered shards or buckets. During training, workers stream shards sequentially from storage while maintaining local shuffle buffers within each tier, preserving high sequential storage bandwidth while maintaining batch diversity and gradient stability.

The difficulty scorer is a systems choice because it trades probe-compute overhead against ordering quality, as table 9.4 shows. Loss and confidence scoring buy better ordering with extra inference, heuristics avoid compute but require domain knowledge, and self-paced scoring moves adaptation into the training loop itself.

¹³ | **Curriculum learning:**

From Latin *currere* (“to run”), originally meaning “the course to be run”—a metaphor that maps directly to the technique: training data as a course run in deliberate order, easy stretches first. Curriculum learning can act as a continuation method for nonconvex optimization, with easier examples shaping the early optimization trajectory before harder examples are introduced. From a systems perspective, the ICR of a sample can vary during training, which motivates changing the mix of examples over time.

Table 9.4: Difficulty Scoring Strategies for Curriculum Learning: The rows compare each scorer’s signal source with the system prerequisite or trade-off it introduces.

Strategy	Difficulty Score	Best For
Loss-Based	Loss from probe model (low = easy)	General-purpose; requires probe training
Confidence-Based	Teacher model confidence (high = easy)	When teacher available; distillation setups

Strategy	Difficulty Score	Best For
Domain Heuristics	Sentence length, image complexity	No extra compute; domain knowledge required
Self-Paced	Current model's loss (updated each epoch)	Adaptive; no probe needed

Table 9.5 uses hypothetical epoch counts to illustrate how curriculum savings would be computed across workloads. The values are not measurements from the cited curriculum-learning studies.

Table 9.5: Illustrative Curriculum Epoch Reductions: The table applies one target-accuracy rule to hypothetical baseline and curriculum epoch counts. It demonstrates the arithmetic but does not establish a ranking among datasets or methods.

Dataset	Model	Pacing Strategy	Epochs to Target Acc.	Epoch Reduction
CIFAR-10	ResNet-18	Linear warmup	115 vs. 150 baseline	23.3% fewer epochs
CIFAR-100	ResNet-32	Self-paced	180 vs. 220 baseline	18.2% fewer epochs
ImageNet	ResNet-50	Loss-based	80 vs. 90 baseline	11.1% fewer epochs
ImageNet	ResNet-50	Learned (noisy labels)	70 vs. 90 baseline	22.2% fewer epochs

The scenario illustrates that useful curriculum gains must be measured at a fixed target metric. The ordering is task-dependent: *anti-curriculum* presents hard examples first, while *self-paced learning* adjusts difficulty using the model's current loss. Neither ordering is universally superior.

9.3.2 Active learning: Human-in-the-loop

Curriculum learning optimizes the order in which samples are presented but assumes all samples are already labeled. This assumption breaks down in specialized domains where labeling requires substantial expertise, time, or money. Rather than labeling everything upfront, active learning¹⁴ (Settles 2012; Ren et al. 2021) shifts the optimization target from choosing which labeled samples to train on to choosing which unlabeled samples are worth labeling.

Active learning transforms data acquisition from static collection into an iterative query process. Trace the circular flow path in figure 9.5, following how model uncertainty metrics select high-value unlabeled samples for expert annotation.

The effectiveness of active learning depends critically on the query strategy used to select samples for annotation (Settles 2009, 2012; Ren et al. 2021). The simplest approach, uncertainty sampling, selects samples where the model is least confident, such as predictions near 0.5 probability for binary classification. It is inexpensive to score with one model but can overconcentrate queries near one ambiguous region. Query-by-committee extends this idea by training multiple models and selecting samples where they disagree most, capturing epistemic uncertainty that a single model might miss.

For practitioners willing to invest more compute, expected model change selects samples that would cause the largest gradient update if labeled. This approach provides a theoretically grounded but expensive alternative. Diversity sampling complements uncertainty-based methods by selecting samples dissimilar from currently labeled data, improving coverage rather than clustering every query around the same ambiguous region.

Active learning is particularly valuable in domains where labeling requires expertise. In medical imaging, for instance, an AI system diagnosing diseases from X-rays may be confident on common conditions but uncertain about rarer cases. By focusing human annotation on these ambiguous cases, active learning optimizes the use of expensive expert time while accelerating model improvement.

¹⁴ | **Active learning:** The “active” component is the learning algorithm itself selecting which unlabeled samples a human expert should label next. This reframes the problem from a computational optimization (training on given data) to a financial one: maximizing model improvement per dollar spent on expert labeling. Querying the most informative examples can substantially reduce labeling needs compared with random sampling, but the multiplier is task-, model-, and oracle-dependent.

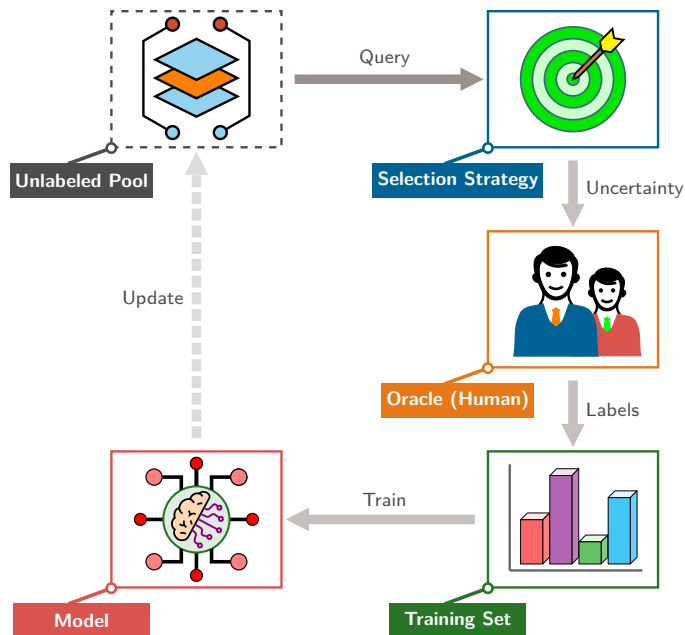


Figure 9.5: The Active Learning Closed Loop: The selection cycle trades proxy-scoring compute for queries chosen from an unlabeled pool for oracle annotation. Uncertainty, diversity, and coverage criteria seek more useful information per annotation than undirected sampling.

The economic implications can be substantial in specialist domains, where annotation cost and turnaround may dominate the compute used to score candidates. These query strategies drive each iteration of the active learning loop in figure 9.5, and a simple budget comparison shows how active learning can reduce labeling cost and training work under stated assumptions.

Napkin Math 9.3: The active learning ROI

Problem: A hospital team has 1 Million unlabeled scans, a \$5/label specialist-label cost, a \$500,000 budget, and 1 month. How does uncertainty sampling change labeling cost and per-epoch training work relative to the budget-matched random baseline?

Scenario A: Naive Labeling

- **Cost:** Labeling all 1 Million scans would cost \$5,000,000 (10× over budget).
- **Budget-matched baseline:** $\$500,000 \div \$5/\text{label} = 100,000$ random scans.
- **Naive labeling outcome:** Random selection does not guarantee rare-pathology coverage.

Scenario B: Active Learning

- **Strategy:** Uncertainty sampling selects 50,000 scans for specialist labeling.
- **Cost:** $50,000 \times \$5/\text{label} = \$250,000$ (50 percent under budget).
- **Training work:** $100,000 \div 50,000 = 2\times$ fewer examples per epoch than the budget-matched random baseline.
- **Active learning outcome:** Matching 100,000 random scans requires empirical validation.

Systems insight: Against the budget-matched random baseline, selection saves \$500,000 — $\$500,000 - \$250,000 = \$250,000$; accuracy still requires empirical validation.

Targeted sample selection can reduce the annotation volume required to reach a target metric. Compare the two illustrative accuracy trajectories in figure 9.6, noting the horizontal gap between active uncertainty sampling and baseline random selection as sample counts scale.

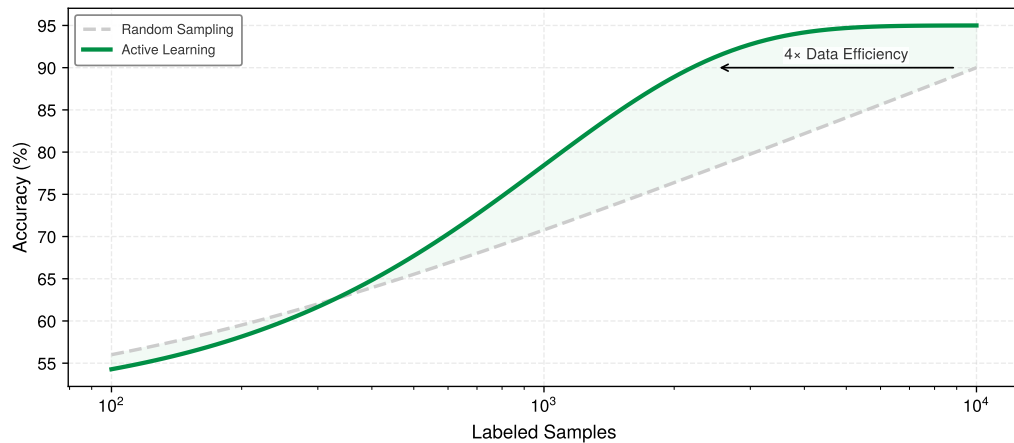


Figure 9.6: The Active Learning Multiplier: Model accuracy is plotted against labeled samples on a logarithmic axis. In this illustrative scenario, active learning (green solid) reaches the same accuracy with fewer labels than random sampling (gray dashed); the arrow marks an approximately 4× label-efficiency gap near 90 percent accuracy.

The figure tracks a different axis from the calculation in napkin math 9.3. It illustrates label efficiency, while the callout calculates labeling dollars under separate assumptions. Active learning also incurs scoring, acquisition, and repeated-retraining costs, so end-to-end savings can be smaller than the label-count gap.

Active learning can do more than save labeling cost: a suitable query rule can concentrate review on uncertain, diverse, or failure-relevant examples.



Example 9.2: Hard negative mining in a smart doorbell

Scenario: A smart doorbell vision detector receives continuous video feeds dominated by empty backgrounds (easy negatives) and normal pedestrian poses.

Diagnosis: Random frame sampling can miss rare false positives such as statues, laundry piles, and human-shaped shadows. Active learning routes uncertain predictions (“Person: 51%”) and sampled production errors to human reviewers for targeted labeling.

Systems lesson: Hard-negative mining can improve labeling efficiency by spending review effort on suspected false positives and other difficult negatives. The query stream still needs diversity and coverage checks so confident but systematically wrong cases are not excluded.

9.3.3 Semi-supervised learning: Using unlabeled data

Consider an illustrative medical-imaging scenario with 50,000 chest X-rays, of which 500 have been reviewed and labeled¹⁵ by radiologists, a labeling rate of 1 percent. Whether that seed set is adequate depends on the task and distribution. Semi-supervised learning can use the remaining 49,500 images when the unlabeled pool matches the target distribution and the method is validated on labeled data.

Active learning optimizes which samples to label but still requires human annotation for every selected example. Semi-supervised learning shifts the engineering trade-off from human annotation to accelerator compute: it uses a small labeled subset to anchor optimization on a larger unlabeled pool, extracting supervisory signals directly from unannotated inputs. This allows models to approach fully supervised accuracy with substantially fewer manual labels, provided the unlabeled pool reflects the target distribution.

The core insight behind semi-supervised learning is that unlabeled data, while it cannot directly teach the mapping from inputs to outputs, contains structural information about the input distribution $p(x)$ that can constrain the hypothesis space. Under smoothness and cluster assumptions, a decision boundary that cuts through a dense region assigns different labels to nearby inputs and is

¹⁵ | **Illustrative clinical labeling economics:** The review-rate and hourly-rate ranges are scenario assumptions rather than reported measurements. Under those inputs, labeling 500 scans requires 7–10 hours and costs \$1,000–3,000; labeling all 50,000 costs \$94,000–300,000. The arithmetic illustrates why semi-supervised learning can be attractive when expert labels are expensive, but actual review time, adjudication, prevalence, and validation requirements must be measured.

less plausible than one in a low-density region. Semi-supervised methods exploit these assumptions, which must be checked against the target task.

Three techniques implement this distribution-regularization insight. Pseudo-labeling¹⁶ trains on the labeled subset, generates model predictions on the unlabeled pool, and incorporates samples exceeding a confidence threshold as synthetic ground truth for subsequent retraining. The confidence threshold governs a critical systems trade-off: setting it too low admits label noise that degrades learning, while setting it too high discards informative examples and wastes the unlabeled pool.

Consistency regularization¹⁷ enforces that the model produce invariant predictions across perturbed versions of the same input. Because class semantics are invariant to realistic transformations such as cropping, rotation, or color shifts, the loss function penalizes prediction divergence between two stochastic views of an unlabeled sample. Hybrid approaches like FixMatch¹⁸ unify both mechanisms: they generate a pseudo-label on a weakly augmented view when prediction confidence exceeds a threshold, then train the model to predict that label on a strongly augmented variant of the same input.

Label propagation frames semi-supervised learning as graph diffusion: it constructs an affinity graph over sample representations and propagates labels from labeled nodes to unlabeled neighbors. This approach works well when representations form well-separated clusters, though constructing and querying the similarity graph across large datasets introduces substantial memory and nearest-neighbor search overhead.

 Example 9.3: FixMatch on CIFAR-10

Scenario: A semi-supervised image classification pipeline trains on CIFAR-10 with only 250 labeled samples (25 per class) alongside unlabeled data (Sohn et al. 2020).

Diagnosis: The fully labeled CIFAR-10 training set contains 50K labels. FixMatch generates high-confidence pseudo-labels (>0.95 threshold) on weakly augmented unlabeled images and enforces consistency on strongly augmented variants, reaching 94.9 percent accuracy in the cited experiment.

Systems lesson: Semi-supervised learning trades additional training compute for fewer manual labels. Under the chapter’s labeling-cost assumptions, the FixMatch scenario reduces dataset preparation expense by 8.1×; the end-to-end saving depends on the unlabeled-data and compute costs.

The systems trade-off in semi-supervised learning exchanges labeling effort for additional computation over labeled and unlabeled samples. It can approach fully supervised accuracy with fewer labels, but neither the label reduction nor the compute increase is universal. A CIFAR-10 comparison makes one such trade-off concrete (table 9.6).

Table 9.6: FixMatch Label Efficiency on CIFAR-10: With 250 labels (0.5 percent of the dataset), FixMatch reaches 94.9 percent accuracy while using 200× fewer labels than the full training set contains.

Label Budget	Method	Accuracy	Label Efficiency
50,000 (100%)	Full training set	—	Label-count reference
4,000 (8%)	FixMatch	95.7%	12.5× more efficient
250 (0.5%)	FixMatch	94.9%	200× more efficient
40 (0.08%)	FixMatch	88.6%	1250× more efficient

These efficiency gains depend on benign distribution assumptions. Semi-supervised learning assumes that unlabeled data comes from the same distribution as labeled data, and it struggles when unlabeled data contains out-of-distribution samples (the model confidently mislabels them), when class imbalance is severe (pseudo-labels amplify majority class bias), or when the labeled set does not cover all classes (preventing label propagation for unseen classes). Always validate on a held-out set with true labels to catch distribution mismatch.

16 | Pseudo-labeling: Uses a trained model’s own confident predictions as ground-truth labels for unlabeled data; the technique’s effectiveness depends on a virtuous cycle: accurate predictions on easy unlabeled examples expand the training set, improving the model, which enables accurate predictions on harder examples. The failure mode is equally self-reinforcing: incorrect pseudo-labels reinforce errors through *confirmation bias*, causing the model to reinforce its own misclassifications.

17 | Consistency regularization: Rooted in the smoothness assumption: if two inputs x_1 and x_2 are close in input space, their labels should also be close; the training objective minimizes divergence between a model’s predictions on an input and its augmented version. This is conceptually distinct from data augmentation (which creates more training examples) because it explicitly enforces prediction consistency as a loss term, even for unlabeled data where the “correct” label is unknown. The systems consequence is additional computation for generating and comparing predictions under multiple perturbations, a trade-off that can favor compute-rich, label-poor settings.

18 | FixMatch: It generates a pseudo-label using a weakly augmented image and then trains the model to predict that same label for a strongly augmented version of the image. This consistency training is gated by a confidence threshold; a pseudo-label is only used if the model’s prediction on the weak augmentation is highly confident (for example, >0.95) (Sohn et al. 2020). This embodies a direct systems trade-off between additional training compute and fewer manual labels; the magnitude depends on the baseline and cost model.

Despite these limitations, semi-supervised learning can reduce label requirements while maintaining accuracy on suitable tasks. Across the techniques, coreset selection and deduplication prune low-value samples before training; curriculum learning changes the presentation order; active learning chooses which samples receive human annotation; and semi-supervised learning extracts additional signal from an unlabeled pool. Each technique can reduce dependence on task-specific labels, but none eliminates the need for labeled evaluation. The progression raises a related possibility: task-specific labels may not be necessary for pretraining. The structure of data itself—that cat images resemble other cat images and coherent sentences follow grammatical patterns—may provide a supervision signal, while downstream quality still requires task-aligned validation.

9.4 Self-Supervised Learning

GPT was trained to predict the next token in a sequence. BERT was trained to fill in masked tokens. Neither pretraining objective required task-specific human labels. **Self-supervised learning**¹⁹ (SSL) generalizes this insight: by designing *pretext tasks* that derive supervision from the data’s structure, models can learn reusable representations from unlabeled data at scale. Where active and semi-supervised learning reduce the demand for labels, SSL changes the accounting by making unlabeled structure the pretraining signal. In the three-stage map, this makes SSL less a fourth selection stage than an extension of the available pool: the question shifts from which labeled samples to train on to how unlabeled samples can support pretraining. It responds to one aspect of the data wall in section 9.1.4 by broadening what counts as a training signal, while leaving corpus quality, licensing, coverage, and labeled evaluation requirements intact.

Labels are one form of supervision; a pretext task can also derive a learning signal from the structure of the data, as table 9.7 summarizes.

Table 9.7: Self-Supervised Pretext Tasks by Modality: Each task extracts supervision from data structure rather than human labels, enabling pretraining on large-scale unlabeled corpora.

Modality	Self-Supervised Task	Supervision Signal
Text	Masked language modeling	Predict [MASK] from context
Text	Next-token prediction	Predict next token in sequence
Images	Contrastive learning	Same image (augmented) vs. different images
Images	Masked autoencoding	Reconstruct masked patches
Multi-modal	CLIP-style alignment	Match image-text pairs

Pretext tasks generate supervision signals automatically. Masked-token objectives can encourage models to learn grammatical and semantic structure; next-token prediction teaches continuation structure; and contrastive image objectives encourage visual features that are invariant to selected transformations. These approaches build on the convolutional neural network and transformer families in chapter 6, but the systems accounting matters: self-supervision removes task-specific annotation from the pretraining gate rather than removing data cost. Pretraining can begin on unlabeled corpora without waiting for example-level labels, but collection, filtering, licensing, storage, and processing remain. Separating pretraining from downstream labeling restructures the economics of machine learning when the representation transfers across tasks; task-specific adaptation and evaluation still carry their own data requirements.

9.4.1 The economics of amortization

The centrality of self-supervised learning to foundation-model workflows stems directly from cost amortization. A foundation model is a broadly pretrained reusable base model adapted to downstream tasks through fine-tuning. This architecture divides costs into two phases: an expensive upfront pretraining investment executed once on unlabeled data, and lightweight task-specific adaptations reused across many applications (table 9.8).

¹⁹ | **Self-supervised learning:** The pretext task, such as next-token or masked-token prediction, provides a supervisory signal derived from the data itself. This reduces dependence on task-specific human labels but can require substantial pretraining compute. Amortization depends on how broadly the resulting model is reused.

Table 9.8: Illustrative Foundation-Model Amortization: The label and compute ranges are scenario assumptions. They illustrate why reuse can lower marginal task cost when representations transfer, not a universal fine-tuning ratio.

Approach	Labels per Task	Compute per Task	Data Acquisition
Train from scratch	100K–1M labeled	100% full training	Task-specific collection
Fine-tune foundation model	100–1K labeled	1–5% of full training	Reuse pretraining corpus

To illustrate this economic transformation, consider a company building 10 specialized classifiers. Under the scenario assumptions, each task trained from scratch needs 100,000 labels at \$1 per label, for \$1M across all tasks. The modeled compute burden is 10,000 GPU-hours.

The fine-tuning scenario pays 10,000 GPU-hours once, then assigns 1,000 labels and 50 GPU-hours of compute to each task. Across all 10 tasks, the modeled fine-tuning labels cost \$10K.

Under these inputs, labeling cost drops by 100× and per-task marginal compute by 20×. These are consequences of the chosen scenario values, while total compute becomes favorable only after enough downstream tasks amortize pretraining.

Fine-tuning becomes economical when representations transfer effectively and downstream task count exceeds the break-even threshold. The side-by-side cost comparison in figure 9.7 contrasts repetitive scratch training on the left against upfront pretraining followed by low-cost fine-tuning on the right.

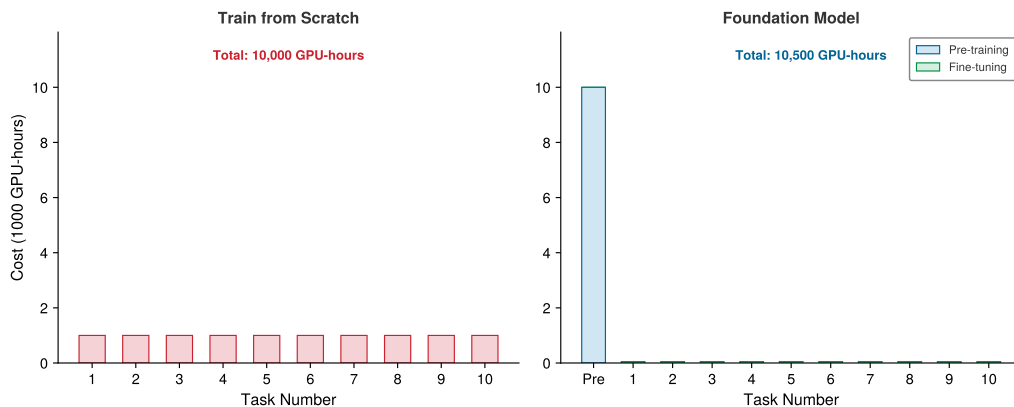


Figure 9.7: Cost Amortization in Foundation Models: Training from scratch (left) requires 1,000 GPU-hours per task (10,000 GPU-hours total for 10 tasks). The foundation model approach (right) pays 10,000 GPU-hours upfront for pretraining but reduces each subsequent task to just 50 GPU-hours. At 10 tasks the totals are comparable (10,000 GPU-hours vs. 10,500 GPU-hours), but the per-task marginal cost drops by 20×, and the crossover favoring the foundation model occurs around 10.5 tasks. The plotted 10-task snapshot therefore remains just below that break-even point.

9.4.2 Foundation model paradigm

Amortization economics vary across self-supervised methods because each objective exercises different hardware bottlenecks on the cost-efficiency frontier. Contrastive learning requires large pools of negative examples to prevent representation collapse. SimCLR (T. Chen et al. 2020) draws negatives directly from co-present samples in the mini-batch, requiring batch sizes as large as 4,096; distributing these across accelerators incurs substantial memory allocation for intermediate activations and frequent all-gather communication over the inter-device interconnect. MoCo (He et al. 2020) decouples the negative pool from the mini-batch size by caching representations in a dynamic queue updated via a momentum-averaged encoder, enabling large negative sets on modest accelerator configurations without saturating memory or interconnect bandwidth. Masked modeling alters memory and arithmetic intensity along different axes: masked autoencoders (MAE) drop up to 75 percent of visual patches prior to the encoder, drastically reducing the quadratic memory overhead of self-attention, whereas masked language modeling evaluates full token sequences with bidirectional attention. Autoregressive generative pretraining trades raw compute for broad task

transfer, following empirical power-law scaling across tokens, parameters, and compute. These methods build on the architectures in chapter 6; for data selection, self-supervised pretraining multiplies the utility of scarce labeled data by extracting reusable representations directly from raw data structure.

When its pretraining cost is reused across tasks, SSL supports the **foundation model paradigm**²⁰ (Bommasani et al. 2021). The core data selection principles—coreset selection, curriculum learning, and active learning—remain essential within this paradigm. Pretraining corpus curation applies deduplication and quality filtering at web scale, while downstream selection dictates which fine-tuning examples maximize task-specific transfer.

Self-supervised learning addresses the label bottleneck by learning from data structure rather than human annotation, yet it cannot solve data scarcity itself. Rare classes may have too few examples, edge cases may never appear in the wild, and privacy constraints may prevent collecting real samples. The third stage of the data selection pipeline addresses this gap by creating new data on demand rather than selecting or curating existing data.

9.5 Synthetic Data Generation

Real-world data collection encounters physical and legal boundaries: autonomous fleets cannot safely crash vehicles to acquire catastrophic collision trajectories, healthcare systems cannot transmit protected patient telemetry across institutional firewalls, and edge devices encounter acoustic environments long before physical products deploy at scale. When target distributions exhibit extreme scarcity or zero empirical coverage, neither static filtering nor dynamic sample prioritization can extract signal that does not exist in the source corpus. Synthetic data generation resolves this fundamental bottleneck by manufacturing training distributions directly. The engineering strategy shifts from *curation*—filtering and weighting existing records—to *creation*, generating novel inputs, domain perturbations, or supervisory targets to satisfy statistical coverage constraints.

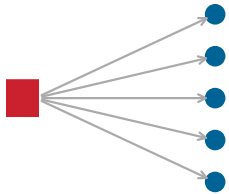
9.5.1 Data augmentation: Transformation-based synthesis

Data augmentation is the lowest-overhead form of synthetic generation: it expands the effective training distribution by transforming existing samples dynamically during ingestion. Because label-preserving transformations generate novel inputs without expanding durable storage, augmentation multiplies dataset diversity with zero additional data acquisition cost. In an accelerator-based training pipeline, however, augmentation shifts the computational bottleneck between the host CPU and the accelerator. When executed on host CPU worker threads, intensive decoding and spatial transformations can saturate host cores, starving the accelerator across the PCIe bus. Offloading transformations directly to accelerator kernels eliminates host-to-device transfer bubbles, but consumes High Bandwidth Memory (HBM) and compute cycles that would otherwise perform gradient updates.

For image data, transformation pipelines enforce the inductive invariances required at deployment (Shorten and Khoshgoftaar 2019). Geometric operations (rotation, horizontal flipping, random cropping, and scaling) enforce viewpoint invariance, while photometric adjustments (brightness, contrast, and color jitter) simulate sensor noise and varying illumination. More aggressive strategies alter input topology directly: Cutout²¹ applies random rectangular zero-masks to prevent reliance on localized features; MixUp (H. Zhang et al. 2018) constructs virtual samples by taking linear combinations of image pairs and their one-hot labels ($\tilde{x} = \lambda x_i + (1 - \lambda)x_j$); and CutMix²² pastes rectangular patches from one image into another while blending labels proportionally to the patch area. Because CutMix and MixUp operate via tensor linear combinations and block memory copies directly in device memory, they provide strong data-space regularization with negligible FLOP overhead.

Text augmentation presents different constraints because discrete token vocabularies lack smooth geometric manifolds. Back-translation²³ provides semantic paraphrasing for sequence-to-sequence tasks by routing target-language monolingual corpora through an intermediate model to generate synthetic source pairings. Simpler heuristic transforms—such as synonym substitution via WordNet, random token insertion, or deletion—introduce local perturbations, but risk corrupting semantic labels if applied without task-specific validation.

20 | Foundation model: The name emphasizes that these models serve as a shared base for many downstream tasks, but this creates a single point of failure. Defects in the foundation model’s pretraining data (biases, factual errors, memorized private content) propagate to every application built upon it. From a systems perspective, this homogenization risk means that data selection quality during pretraining has an out-sized blast radius: a curation error that would affect one task in the train-from-scratch paradigm now affects thousands of downstream deployments.



One pretraining corpus defect propagates into many downstream tasks.

21 | Cutout: Randomly masks square regions of input images during training, forcing the model to recognize objects from partial information rather than relying on any single discriminative region (DeVries and Taylor 2017). Unlike dropout (which zeroes neurons in feature space), Cutout operates in input space. The original Cutout experiments produced 0.3–2.0 percentage-point gains across CIFAR-10/100 and SVHN with negligible compute overhead, making it a high-ICR augmentation technique when occlusion-style invariance matches the task: more information per sample at near-zero additional cost to the pipeline.

Rather than hand-tuning transformation schedules, automated policy searches optimize augmentation combinations directly. AutoAugment²⁴ uses reinforcement learning across discrete operation spaces, but incurs extreme compute overhead. RandAugment²⁵ eliminates this search cost by parameterizing the policy with just two values: the number of transformations N and a global distortion magnitude M .



Lighthouse 9.2: MobileNetV2 and aggressive augmentation

The MobileNetV2 lighthouse model from section 6.1.1 exemplifies how data augmentation complements a resource-constrained architecture. Its depthwise separable convolutions replace standard spatial convolutions with depthwise spatial filters followed by 1×1 pointwise projections, substantially reducing parameter count and arithmetic intensity.

Because compact architectures possess limited representational capacity, aggressive augmentation regularizes feature representations without inflating inference parameter memory or latency budgets. However, augmentation magnitude must be balanced carefully: excessive distortion can exceed the capacity of a compact network, causing underfitting on the underlying task distribution.

9.5.2 Generative synthesis: Creating new samples

Augmentation perturbs existing training points within their local neighborhoods. Generative synthesis constructs entirely new coordinate tuples (x, y) by sampling from parametric generative models or physical simulation engines. The systems trade-off across generative paradigms spans a cost-fidelity spectrum:

1. Generative Adversarial Networks (GANs) train a generator against a discriminator. Once trained, feed-forward generator inference requires a single forward pass, providing low latency and high sample throughput. However, GAN training is vulnerable to mode collapse, which suppresses the tail variety required for robust edge-case coverage.
2. Diffusion models formulate synthesis as iterative denoising across a parameterized Markov chain or continuous score-based process. Latent diffusion systems such as Stable Diffusion²⁶ enable precise semantic conditioning from text prompts while operating in a compressed latent space. However, generating each batch requires unrolling 20 to 50 sequential forward passes through a large backbone, imposing severe arithmetic intensity and memory bandwidth costs that preclude on-the-fly generation in active training loops.
3. Physical simulation engines (such as CARLA for autonomous driving or Isaac Sim and Unity for robotics) decouple data generation from neural inference entirely, executing rigid-body physics, kinematics, and graphics rendering pipelines. Simulators produce exact ground-truth annotations (including 3D bounding boxes, depth maps, and velocity vectors) from internal state without human labeling cost. The systems bottleneck shifts to host CPU physics computation and GPU graphics rasterization or ray tracing throughput.

Regardless of the generative engine, synthetic samples inevitably diverge from the statistical distributions of real deployment environments. This divergence defines the domain gap.

9.5.3 Bridging the domain gap

The domain gap²⁷ represents the statistical divergence between synthetic training distributions and production deployment data. When models optimize purely on synthetic distributions, decision boundaries align to artifacts of the simulator or generative model rather than physical reality. As illustrated in figure 9.8, a classifier trained on a shifted synthetic distribution can achieve near-zero synthetic training error while failing silently when evaluated on real-world clusters.

Two complementary strategies address this distribution mismatch. Domain randomization²⁸ takes an aggressive approach: rather than trying to match the real world precisely, it trains on varied synthetic data by randomizing lighting, textures, backgrounds, and camera parameters during generation.

The aim is to make deployment conditions fall within the variation seen during training, but coverage is never guaranteed by randomization alone. The method can be effective in robotics and

22 | CutMix: Replaces Cutout's zeroed-out region with a patch from a different training image, mixing labels proportionally to patch area (30 percent of image A replaced by image B yields a 70/30 label split) (Yun et al. 2019). This addresses Cutout's weakness: zeroed regions waste pixel information that could carry learning signal. The CutMix paper reports ImageNet top-1 improvements of 2.28 percentage points for ResNet-50 and 1.70 percentage points for ResNet-101, while also improving localization behavior. The method provides stronger regularization than either Cutout or MixUp alone with little additional data-pipeline cost.

23 | Back-translation: In the method described by Senrich et al. (2016), target-language monolingual text is translated into the source language, and the synthetic source is paired with the unchanged target for machine-translation training. The systems trade-off is latency because each synthetic source requires translation-model inference. Pipelines can precompute these examples offline rather than generating them in the data loader.

24 | AutoAugment: Treats augmentation policy design as a reinforcement learning search problem: a controller selects operations (rotate, translate, shear, equalize), their magnitudes, and application probabilities to maximize validation accuracy (Cubuk et al. 2019). The original ImageNet search used approximately 15,000 GPU-hours, motivating later methods that reduce the search space.

25 | RandAugment: Collapses AutoAugment's policy search to two hyperparameters: transformation count N and shared magnitude M (Cubuk et al. 2020). That small search space matches or exceeds AutoAugment on the reported benchmarks at far lower search cost, making it practical when 15,000 GPU-hours of policy search cannot be justified.

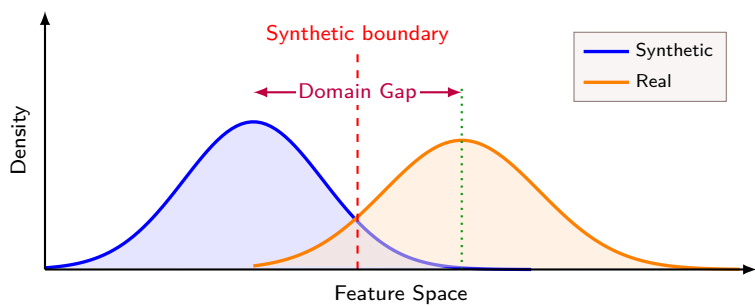


Figure 9.8: The Domain Gap Problem: Synthetic data (blue) and real data (orange) occupy shifted feature distributions. The dashed red synthetic boundary and dotted green real-data boundary illustrate how a boundary learned for one distribution can be misaligned with the other.

26 | Stable Diffusion: Performs iterative denoising in a compressed latent space, enabling targeted text-to-image examples (Rombach et al. 2022). Its iterative generation is substantially more compute-intensive than geometric transforms, so it fits cases where novelty matters more than per-sample throughput.

27 | Domain gap: The statistical divergence between synthetic and real data distributions, measurable with metrics such as maximum mean discrepancy (MMD) or Frechet Inception Distance (FID). For ML systems, the gap becomes training-serving skew: a simulator-trained model can validate well on synthetic data while failing silently on real deployment data.

28 | Domain randomization: Makes synthetic data deliberately varied by randomizing textures, colors, lighting, and physical properties. The goal is not photorealism; it is enough variation to improve coverage of deployment conditions, shifting the cost bottleneck from rendering fidelity to coverage.

autonomous-driving settings when the simulator spans the relevant physical and visual factors and the result is validated on real data.

Domain adaptation takes the opposite approach by explicitly aligning synthetic and real distributions. Feature alignment methods train on synthetic data while simultaneously minimizing the distance between synthetic and real feature distributions, often using adversarial training to learn domain-invariant representations. Fine-tuning offers a simpler path: pretrain on abundant synthetic data to learn general features, then fine-tune on a small real dataset to adapt to deployment conditions. Self-training combines these ideas by using a synthetic-trained model to pseudo-label real unlabeled data, then retraining on the combined labeled set.

In practice, mixing synthetic and real data provides a real-data anchor while retaining some synthetic coverage, but the best ratio is workload-dependent. Table 9.9 summarizes the trade-off across different mixing ratios.

Table 9.9: Synthetic-to-Real Data Mixing Ratios: Pure synthetic data can suffer from distribution shift; pure real data is expensive. The optimal ratio varies by domain, simulator fidelity, model family, and validation protocol, so the table should be read as a scenario scaffold rather than a universal recipe.

Synthetic Fraction	Representative Outcome
100% synthetic	No real-data anchor; validate the domain gap
80% synthetic + 20% real	Lower real-data demand; validate on deployment data
50% synthetic + 50% real	More real-data anchoring; higher collection cost
100% real	No synthetic-to-real gap; highest collection cost



Example 9.4: KWS data selection

Scenario: An embedded keyword spotting (KWS) system targets deployment on a 256 KB SRAM microcontroller with a 10,000+ audio sample requirement.

Diagnosis: Manually recording 10,000 real utterances across diverse speakers and acoustic environments costs \$20K–50K. Staging a 5-tier pipeline (seed recordings → augmentation → noise injection → hard-negative mining → TTS simulation) builds dataset scale from 500 seed samples.

Systems lesson: Layered data strategies can lower acquisition costs for resource-constrained edge ML deployments. In this scenario, augmentation and noise injection reduce physical collection cost to 5 percent of the full-recording estimate; target accuracy and acoustic coverage still require validation.

The keyword-spotting scenario shows how augmentation, noise injection, and simulation can expand a small seed set, but the same creation strategy becomes riskier when samples come from

learned generators. In recursive training, *model collapse*²⁹ can amplify errors and reduce diversity over generations. Synthetic data therefore requires provenance, mixture controls, and validation against real deployment data rather than an unverified assumption that more generated samples help.

9.5.4 Knowledge distillation: Compressing information

While data augmentation (section 9.5.1) and generative synthesis (section 9.5.2) synthesize new input samples, knowledge distillation³⁰ (Hinton et al. 2015; Gou et al. 2021) synthesizes supervisory targets. Rather than training exclusively on sparse ground-truth labels (such as a one-hot vector $[1, 0, 0]$), a compact student network trains against the continuous probability distribution generated by a larger teacher model (such as $[0.70, 0.20, 0.10]$). The soft target distribution, scaled by a temperature parameter τ , exposes inter-class correlations—the dark knowledge—that reflect metric similarities learned in the teacher’s representation space.

Distillation exposes a fundamental systems trade-off between accelerator High Bandwidth Memory (HBM) capacity and secondary storage I/O bandwidth:

- In *online distillation*, the teacher model resides directly in accelerator memory alongside the student model during training. The system computes teacher forward passes concurrently with student execution, eliminating persistent storage overhead for soft targets. However, co-locating an overparameterized teacher consumes accelerator memory and tensor-core throughput, restricting the student’s maximum per-accelerator batch size and increasing step latency.
- In *offline distillation*, the teacher generates predictions across the corpus during an offline precomputation pass, storing dense logit vectors to disk. This frees accelerator memory entirely for student training, but dramatically expands dataset storage volume and input pipeline bandwidth. For a classification vocabulary of V classes, storing uncompressed float32 logits increases the per-sample label footprint from a single integer index ($\lceil \log_2 V \rceil$ bits) to $4V$ bytes—a thousand-fold storage expansion for large vocabularies.

Together, transformation-based augmentation (section 9.5.1), generative synthesis (section 9.5.2), and supervisory distillation complete the third stage of data selection. Where static filtering eliminates redundant input samples and dynamic selection optimizes sample ordering during training, synthetic generation manufactures missing signals when empirical collection hits physical, legal, or economic constraints. Orchestrating these techniques across the data, algorithm, and machine dimensions requires balancing data storage footprint, pipeline preprocessing throughput, and accelerator utilization within a unified systems budget.

9.6 Decision Framework

When labeling budget, redundancy, rare classes, privacy, and convergence speed all matter at once, the practitioner first has to identify which constraint is binding. Each stage can raise the information-compute ratio when its measured quality gain justifies its overhead: pruning removes low-value samples before training, dynamic selection focuses compute on high-value samples during training, and synthesis creates new high-value samples on demand (table 9.10).

Table 9.10: Three-Stage Data Selection Pipeline: The ranges illustrate possible outcomes rather than portable expectations. Actual gains depend on the dataset, model, selection method, and target metric.

Stage	When Applied	Techniques	Illustrative Outcome
Static pruning	Before training	Coreset Selection, Deduplication, Quality Filtering	30–50% dataset reduction
Dynamic selection	During training	Curriculum Learning, Active Learning, Semi-Supervised	10–30% faster curricula; 2–100× fewer labels
Synthetic generation	On-demand	Augmentation, Generative Models, Distillation	2–10× effective data expansion

Once the dominant constraint is named, table 9.11 compares which technique changes that constraint and why.

29 | **Model collapse:** Formally analyzed by Shumailov et al. (2024), this phenomenon occurs because generative models systematically under-represent tail distributions – rare but important patterns that appear infrequently in training data. When generation $n + 1$ trains on output from generation n , each successive generation further compresses the tails, producing increasingly homogeneous data. The degradation can be rapid in recursive training settings, which is why training on unanchored synthetic outputs risks catastrophic collapse of tail diversity.

30 | **Knowledge distillation:** Soft teacher probabilities reveal structural similarities across classes that one-hot encodings omit. The temperature parameter scales logit smoothing ($\tau > 1$); excessive smoothing erases discriminative signal, while $\tau \rightarrow 1$ collapses back to the argmax prediction. Distillation introduces teacher inference costs that must be justified by improved student sample efficiency or accuracy.

Table 9.11: Technique Selection Guide by Primary Constraint: Candidate data-selection techniques mapped to the dominant resource constraint in the ML pipeline. The table begins the diagnosis; prerequisites and measured ROI determine the final choice.

Constraint	Candidate Technique	Why
Limited labeling budget	Active Learning	Maximizes label ROI by selecting informative samples
High redundancy in data	Deduplication + Coreset	Removes waste before training begins
Rare classes or edge cases	Synthetic Generation	Creates samples that do not exist in raw data
Slow convergence	Curriculum Learning	Improves gradient quality in early training
Privacy requirements	Privacy-audited synthesis	Reduce direct use of real records; verify leakage
Large model, small dataset	Knowledge Distillation	Use teacher model’s knowledge as “data”

Table 9.11 maps individual constraints to techniques, but real projects face multiple constraints simultaneously. The decision tree in figure 9.9 structures the selection process hierarchically: start by identifying the primary bottleneck, then follow the branches to narrow the field.

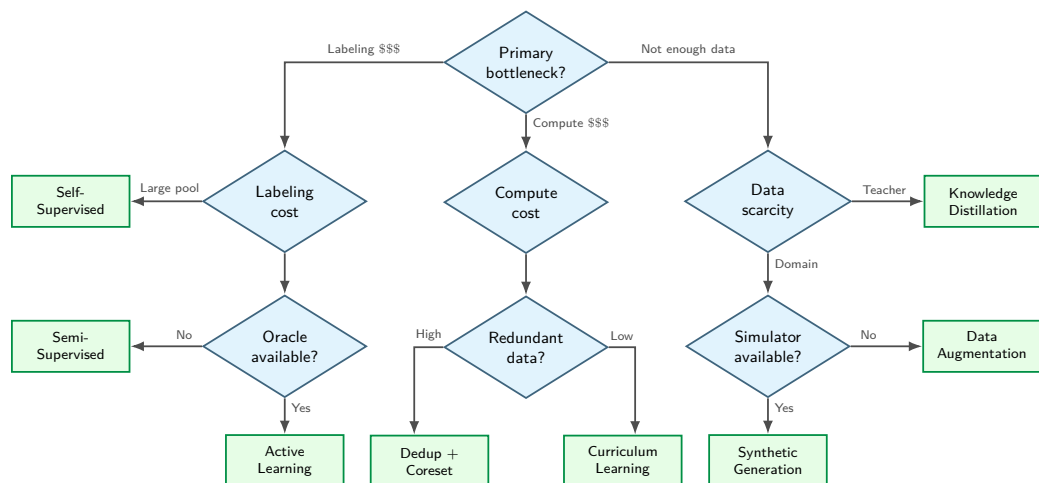


Figure 9.9: Data Selection Decision Tree: The tree begins with the primary bottleneck—labeling cost, compute cost, or data scarcity—and narrows each branch using available pools, oracles, redundancy, teachers, or simulators. Its leaves identify candidate techniques; projects with multiple binding constraints may require more than one branch.

Decision process

Each path requires a structured assessment because the same dataset symptom can point to different bottlenecks. Technique selection begins by naming the binding constraint, then checks whether the data, labels, and infrastructure needed by the chosen method exist.

Step 1: Assess the bottleneck

Identify which resource constraint most severely limits the training pipeline:

- **Labeling cost:** Label-efficiency techniques such as active learning, semi-supervised learning, and self-supervised learning maximize the value extracted from each human annotation.
- **Compute cost:** Coreset selection and deduplication can reduce processed samples, while a validated curriculum may reduce the steps needed to reach a target metric.
- **Data scarcity:** Data creation through augmentation, synthesis, and distillation expands the effective training set beyond what raw collection provides.

This diagnostic step keeps technique selection tied to the binding resource constraint rather than to the most familiar algorithm.

Step 2: Check prerequisites

With the bottleneck identified, verify that the corresponding techniques are feasible given the available infrastructure and data. Each approach carries specific requirements that must be met before implementation can begin (table 9.12).

Table 9.12: Technique Prerequisites: Each technique carries specific infrastructure and data requirements that must be verified before implementation. A technique with excellent theoretical gains but unmet prerequisites will fail in practice, making this checklist the first step in technique selection.

Technique	Prerequisites
Active Learning	Access to oracle, unlabeled pool, retraining infrastructure
Coreset selection	Proxy model or embedding extractor, full dataset accessible
Curriculum Learning	Difficulty scoring method, pacing schedule
Semi-Supervised	Some labeled data, unlabeled data from same distribution
Self-Supervised	Large unlabeled corpus, pretraining compute budget
Augmentation	Domain knowledge of invariances, augmentation library
Synthetic Generation	Generative model or simulator, domain gap mitigation

Step 3: Estimate ROI

Before committing engineering resources, estimate each candidate technique’s return on investment:

$$\text{ROI} = \frac{(\text{Baseline Cost}) - (\text{Technique Cost} + \text{Implementation Cost})}{\text{Technique Cost} + \text{Implementation Cost}}$$

A technique with high theoretical gains but high implementation cost may deliver lower ROI than a simpler approach. Exact deduplication is often an inexpensive first candidate because hashing is simple and repeated bytes are easy to identify, although downstream benefit still depends on corpus redundancy and intentional sample weighting. Active learning requires oracle access, retraining infrastructure, and selection algorithm development, so its ROI depends heavily on how many labeling cycles amortize that investment.

Step 4: Combine techniques

Data selection techniques are not mutually exclusive, but combining them does not guarantee additive gains. A representative workflow deduplicates the raw corpus, applies a coverage-aware coreset filter, orders retained batches through an empirical curriculum, and applies runtime augmentations during batch collation. Pretrained representations alter the baseline trajectory further. Every additional stage must justify its compute overhead, because cascading filters frequently discard overlapping subsets while their scoring latencies accumulate.

Stages can compound efficiency gains when their effects do not overlap and their overhead remains small; combined savings must be measured rather than assumed.

Algorithmic filtering translates into system speedup only when selection overhead remains strictly bounded. A coreset pass that consumes ten hours of probe inference to trim a two-hour training run produces a net loss on wall-clock time and cloud spend. Similarly, an adaptive curriculum that scans the full corpus for per-epoch difficulty scoring can stall accelerators on host CPU bottlenecks. Realizing data-efficiency gains in production demands engineering empathy for the physical stack: amortizing scoring passes, streaming nonsequential batches without defeating prefetch engines, and synchronizing distributed selection states without serialization stalls.

9.7 Selection Engineering

A naive active learning loop that scans the entire dataset every epoch to select the “best” samples can turn a compute-bound training job into an I/O-bound bottleneck. **Selection engineering** begins where the decision framework ends: after identifying which algorithms to apply, engineering must ensure that they deliver their promised speedups on real hardware and real data pipelines. Translating algorithmic filtering into realized throughput requires architectural patterns that bound scoring latency, preserve sequential storage access, and prevent accelerator starvation.

9.7.1 The selection bottleneck

Dynamic data selection introduces a new bottleneck: selection latency. A conventional loader follows a predetermined sampling plan, whereas active learning or adaptive curricula may evaluate a selection function $f(x)$ over a large candidate pool before forming later batches. Concretely, scoring a 1M dataset with a large model can take 2.8 hours, potentially negating the savings from a 10 percent coreset if not performed with a smaller proxy model. The systems trade-off requires that selection cost remain below the training work the subset saves; otherwise, sample counts fall while end-to-end cost rises.

At a fixed target metric, selection plus subset training must cost less than full-data training. Let $T_{\text{selection}}$ be wall-clock time spent scoring the pool, $T_{\text{train}}(D)$ be wall-clock training time for a dataset of size D , and D_{subset} and D_{total} be the retained and full sample counts. Equation 9.2 expresses this break-even condition, the **selection inequality**.

$$T_{\text{selection}} + T_{\text{train}}(D_{\text{subset}}) < T_{\text{train}}(D_{\text{total}}) \quad (9.2)$$

Equivalently, isolating the allowable selection overhead yields $T_{\text{selection}} < T_{\text{train}}(D_{\text{total}}) - T_{\text{train}}(D_{\text{subset}})$. The upper bound on selection latency depends strictly on how much training execution the subset eliminates; no arbitrary overhead fraction applies across workloads. When $f(x)$ repeatedly evaluates the candidate pool using the primary model architecture, recurring scoring passes and distributed coordination can consume the saved training cycles, turning data selection into a net throughput penalty.



Example 9.5: Selection inequality in practice

Scenario: A computer vision team selects a 100K coreset (10 percent) from 1M images to reduce training time on an accelerator cluster.

Diagnosis: Full-model scoring (Option A: target ResNet-50) adds 2.8 h of selection overhead, reducing net savings to 247.2 h. Proxy-model scoring (Option B: ResNet-18) drops selection overhead to 0.6 h, saving 249.4 h total wall-clock hours.

Systems lesson: Coreset selection reduces training latency only when selection overhead is smaller than full-dataset training savings. Proxy scoring preserves net FLOP savings by keeping sample scoring overhead well below full-model forward pass costs.

End-to-end wall-clock time divides into selection overhead and subset training. In figure 9.10, stacked time bars compare baseline full-dataset training against efficient and expensive selection regimes.

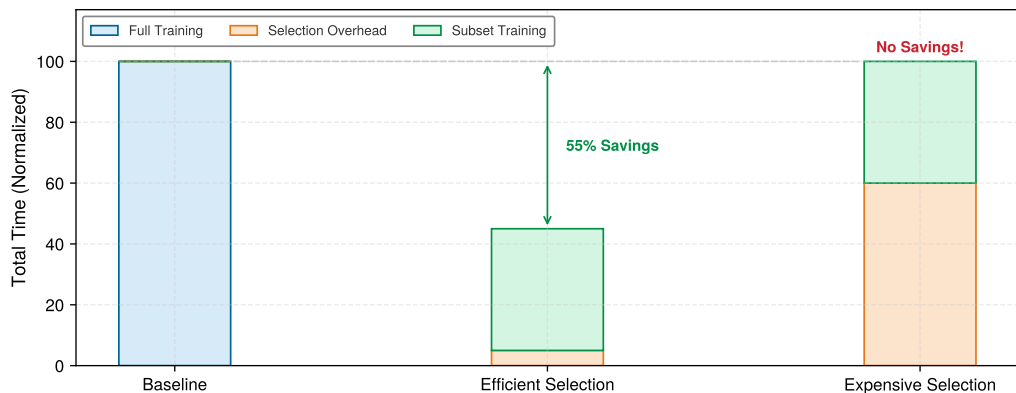


Figure 9.10: The Selection Inequality: Data selection only improves end-to-end efficiency if the overhead of selection plus training on the subset is less than training on the full dataset. A lightweight selection function (proxy model, cached embeddings) keeps selection overhead low; an expensive selection function (full model forward pass) can negate the savings.

This overhead compounds when selection runs dynamically across epochs rather than once offline. If per-epoch rescoring approaches the training work saved by the pruned subset, the net systems advantage vanishes.

Engineering pipelines manage selection overhead through proxy models, cached embeddings, and indexed retrieval. A smaller proxy model curtails scoring latency provided its sample ranking transfers accurately to the target architecture. Running the scoring pass in reduced numerical precision further compresses arithmetic work and memory traffic when ranking fidelity permits. For similarity-based or diversity-based selection, vector indices such as FAISS³¹ replace expensive corpus-wide scans with approximate nearest-neighbor retrieval over precomputed embeddings. These techniques decouple selection evaluation from primary model training, bounding $T_{\text{selection}}$ so the inequality holds.

9.7.2 Hardware empathy: The random-access penalty

The selection inequality addresses compute overhead, but selection algorithms also reshape physical I/O patterns. Index-driven sampling turns large sequential shard transfers into scattered, nonsequential reads, forfeiting the throughput benefits of hardware readahead, OS page caching, and bulk controller prefetching. Because disk heads must physically seek and flash controllers must execute random block lookups, equal-sized subsets can produce starkly different wall-clock training times depending on access locality. Table 9.13 quantifies this small-read penalty across storage tiers for 4 KB transfers.

Table 9.13: Illustrative Small-Read Penalty: Registry device anchors compare sequential bandwidth with random 4 KB reads under a simplified IOPS conversion. Real throughput depends on request size, queue depth, concurrency, caching, and service configuration; the cloud row is only a qualitative scenario.

Storage Tier	Sequential Throughput	Random I/O (IOPS)	Random Throughput (approx)	Random Penalty
HDD (7.2k)	~150 MB/s	~100 IOPS	~0.4 MB/s	375×
SATA SSD	~550 MB/s	~10K IOPS	~40 MB/s	13.8×
NVMe SSD	~3,500 MB/s	~500K IOPS	~2,000 MB/s	1.75×
Cloud (S3)	Configuration-dependent	Not IOPS-comparable	Configuration-dependent	Potentially extreme

Data loaders resolve this conflict through data-machine co-design. Sharded dataset formats, such as WebDataset and FFCV, pack thousands of individual examples into contiguous, sequentially readable archive chunks. Rather than fetching scattered individual samples from disk, the loader streams large sequential shards into host memory and populates a **shuffle buffer**. The training loop samples pseudo-randomly from this in-memory buffer, preserving peak storage read bandwidth while satisfying the statistical requirement of randomized mini-batches. In distributed training across multiple accelerators, each worker maintains an independent shuffle buffer over a non-overlapping shard. Because randomization is local to each worker’s memory buffer rather than global across the corpus, rare classes or boundary cases concentrated in specific shards receive uneven exposure across workers unless the dataset is stratified and shards are balanced prior to distribution.



Checkpoint 9.2: The selection inequality

Data selection is not free. It introduces a new term to the iron law and a new I/O cost.

Equation checks:

- ☐ **Selection cost:** can you explain why $T_{\text{selection}} + T_{\text{train}}(\text{subset})$ must be less than $T_{\text{train}}(\text{full})$ for the technique to reduce end-to-end training time?
- ☐ **Overhead management:** how do proxy models and cached embeddings keep $T_{\text{selection}}$ low?

Systems implications:

³¹ | **FAISS (Facebook AI Similarity Search):** Provides GPU-accelerated similarity search using exact, approximate, and compressed-domain index designs (Johnson et al. 2019); Johnson, Douze, and Jegou report billion-vector graph construction on multiple GPUs, showing why vector-index infrastructure matters for web-scale selection. For data selection pipelines, FAISS-style indexing supports k -nearest-neighbor retrieval for coresets selection, embedding-based deduplication, and stratified clustering for balanced sampling. Without this infrastructure, embedding-based selection would often fall back to expensive full-corpus scans.

- **I/O patterns:** why can nonsequential access for dynamic selection reduce data-loader throughput relative to sequential reads?

9.7.3 Data echoing: Amortizing I/O costs

Even when storage bandwidth is preserved, dynamic selection pipelines often shift the system constraint upstream to host CPU computation. Heavy data transformations—such as 3D rotations, MixUp, JPEG decoding, or on-the-fly generative synthesis—demand substantial host processing per sample. When the host pipeline produces batches slower than the accelerator executes forward and backward passes, GPU compute cores idle on input starve. The resulting collapse in accelerator utilization extends training wall-clock time, neutralizing the efficiency gains achieved by pruning the dataset.

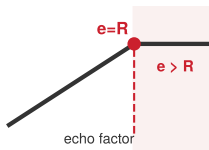
Data echoing³² (Choi et al. 2019) reuses data multiple times before fetching new samples, trading freshness for accelerator utilization when the input pipeline is slower than training. Echoing before randomized augmentation can produce different transformed inputs on each repetition, while echoing later in the pipeline may repeat identical tensors.

The optimal echo factor depends on the ratio R of upstream processing time to downstream training time:

$$R = \frac{T_{\text{data pipeline}}}{T_{\text{GPU training}}}$$

When $R > 1$, the data pipeline bottlenecks training: an echo factor $e < R$ partially recovers idle GPU cycles, while $e \geq R$ saturates accelerator capacity provided repeated samples remain statistically productive. Increasing e beyond R yields no additional utilization gain and risks diminishing gradient diversity. Conversely, when $R < 1$, the accelerator is already compute-bound and data echoing provides no benefit.

³² | **Data echoing:** The key subtlety is *where* in the pipeline to insert the echo point. Echoing before augmentation (upstream echoing) can apply different random augmentations to each repetition, while echoing after augmentation feeds identical tensors to the accelerator. Useful echo factors and realized speedups depend on the workload, batch size, insertion point, and shuffling policy (Choi et al. 2019).



Pipeline utilization saturates near the ratio threshold; statistical value remains workload-dependent.

Napkin Math 9.4: Worked example: Data echoing ROI

Scenario: Training ResNet-50 on ImageNet with heavy augmentation (RandAugment + MixUp).

Measurements:

- Data pipeline throughput: 300 images/s (reading, decoding, augmenting on CPU)
- GPU training throughput: 800 images/s (forward + backward pass)
- Ratio $R = T_{\text{pipeline}}/T_{\text{GPU}} = (1/300 \text{ images/s}) / (1/800 \text{ images/s}) = 800 \text{ images/s} / 300 \text{ images/s} \approx 2.67$ (GPU waiting 62.5 percent of time)

Without echoing:

- Effective throughput: 300 images/s (limited by data pipeline)
- Training time for 90 epochs: $90 \times 1.28\text{M} / 300 \text{ images/s} = 384,350 \text{ seconds}$ (106.8 hours)
- GPU utilization: 37.5 percent

With echo factor: $e = 2$.

- Each batch is processed twice with different augmentations
- Effective throughput: 600 images/s (still below GPU capacity)
- Unique images per second: 300 images/s (unchanged)
- Training time: $90 \times 1.28\text{M} / 600 \text{ images/s} = 192,175 \text{ seconds}$ (53.4 hours) if echoed data is equally valuable

Trade-off: Repeated samples are not guaranteed to be as useful as fresh samples; the data echoing paper evaluates echo factor, insertion point, and shuffling because those choices determine whether reuse preserves predictive performance. In one network-fed ResNet-50/ImageNet configuration, Choi et al. (2019) reports a $3.25\times$ reduction in wall-clock time to the target metric.

Systems insight: Data echoing can trade sample diversity for accelerator utilization when the input pipeline is the bottleneck. The useful echo factor must be measured for the workload because it depends on the batch size, insertion point, augmentation, shuffling, and target metric.

Echoing can introduce correlations when the same example appears within or across nearby batches, so implementations must preserve adequate shuffling and validate the target metric. Choi et al. (2019) did not observe a negative batch-normalization interaction in their ResNet-50 or Single Shot MultiBox Detector (SSD) experiments, but that result does not guarantee the same behavior for every workload.

These implementation patterns determine whether data selection yields net systems throughput. Proxy selection curtails probe compute when scoring rankings transfer to target architectures. Sharded file formats and shuffle buffers reconcile pseudo-random batch creation with hardware-level sequential streaming, while data echoing recovers stalled accelerator cycles when ingestion throughput limits the training loop.

Yet operational feasibility does not guarantee economic viability. A deduplication infrastructure that costs \$50K to engineer but saves only \$10K across an entire training campaign yields a net loss. Deploying selection techniques requires rigorous financial accounting of capital, compute, and human annotation budgets.

9.8 Cost Modeling

Systematic cost modeling governs data selection investments. Engineering teams face concrete capital allocation trade-offs: whether to purchase 100,000 additional human annotations or reserve additional GPU clusters, when active learning pipelines reach break-even, and how many training runs are required to amortize custom curation tooling.

9.8.1 Quantifying data costs and ROI

The total cost of training data spans acquisition, labeling, storage retention, and accelerator compute, extending well beyond object-storage invoices:

$$C_{\text{total}} = C_{\text{acquire}} + C_{\text{label}} + C_{\text{store}} + C_{\text{process}}$$

Labeling (C_{label}) represents the expenditure that data selection targets most directly, whereas storage (C_{store}) and processing (C_{process}) scale mechanically with retained byte volume and the number of accelerator passes. Table 9.14 itemizes these four cost components alongside their operational drivers.

Table 9.14: Total Cost of Training Data: The four cost components span the full data lifecycle. Here D is the total sample count, D_{labeled} is the labeled subset, $D_{\text{vol,store}}$ is stored byte volume, T_{months} is retention time, N_{epochs} is the number of training passes, and O_{sample} is per-sample training work. The ranges are scenario anchors rather than universal market prices.

Component	Formula	Illustrative Range
C_{acquire}	$D \times c_{\text{sample}}$	\$0.001–\$10/sample (web scrape vs. licensed)
C_{label}	$D_{\text{labeled}} \times c_{\text{label}}$	\$0.01–\$200/sample (crowd vs. expert)
C_{store}	$D_{\text{vol,store}} \times c_{\text{storage}} \times T_{\text{months}}$	\$0.02–\$0.10/GB/month
C_{process}	$D \times N_{\text{epochs}} \times O_{\text{sample}} \times c_{\text{FLOP}}$	Proportional to training FLOPs

The interplay of these four cost terms becomes concrete when evaluating an ImageNet-scale vision model training run.



Napkin Math 9.5: Cost breakdown: ImageNet-scale training

Table 9.15 itemizes one illustrative acquisition, labeling, storage, and compute scenario at ImageNet scale. The resulting ratio follows from the stated licensing, annotation, storage, runtime, and compute-cost assumptions; it is not a typical ratio for every supervised workload.

Table 9.15: Illustrative ImageNet-Scale Cost Breakdown: Itemized acquisition, labeling, storage, and compute costs under the stated scenario assumptions. Each amount follows from the rate shown beside it, using the book’s published crowd-labeling, object-storage, and cloud-GPU rates. Data costs dominate this particular calculation because licensing and annotation are included while the training run itself is short.

Cost Component	Calculation	Amount
Raw data (1.2M images)	Licensed dataset, flat fee	\$50,000
Labels (crowd annotation)	1.2M × \$0.05/label	\$60,000
Storage (cloud object store)	150 GB × \$0.02/GB/month × 12 months	\$36
Training campaign (30 runs, each 100 epochs in 24 h on 8 A100s)	5,760 GPU-hours × \$4/GPU-hour	\$23,040
Total		\$133,076
Data vs. Compute ratio		82.7% data, 17.3% compute

Systems insight: Acquisition and labeling can dwarf compute cost in supervised vision workloads, as they do under these assumptions. The compute figure prices the whole training campaign, not one run: a published model is the survivor of sweeps, restarts, and ablations, and costing a single run would understate compute roughly thirtyfold. Even so, acquisition and labeling remain the larger share. The run count is an assumption like any other, so a complete ROI calculation must state it and price both categories rather than assuming which one dominates.

9.8.2 ROI framework for data selection techniques

Data selection techniques insert algorithmic filtering into the training pipeline to reduce downstream costs. However, evaluating sample importance is not free: scoring consumes accelerator cycles, generates host-device memory traffic, or requires engineering custom pipeline stages. Evaluating whether a selection strategy is economically justified requires balancing these upfront overheads against downstream savings through Return on Investment (ROI):

$$\text{ROI} = \frac{\text{Savings} - \text{Investment}}{\text{Investment}} \times 100\%$$

The balance between investment and savings varies across selection strategies. Table 9.16 maps the primary cost drivers and benefit mechanisms across four standard techniques; their net return depends on corpus volume, model scale, and existing pipeline infrastructure.

Table 9.16: ROI Profiles for Data Selection Techniques: The table identifies costs and possible savings. No technique guarantees positive ROI; each must satisfy the selection inequality under measured workload conditions.

Technique	Investment (Cost)	Savings (Benefit)
Deduplication	One-time compute for hashing + infrastructure	Reduced storage and repeated-sample processing
Coreset Selection	Proxy model training + selection compute	Fewer retained samples if target quality and coverage hold
Active Learning	Inference on unlabeled pool + human-in-the-loop latency	Lower labeling demand when query selection transfers
Data Augmentation	CPU/GPU cycles for transforms	Effective dataset size increase without new data acquisition

9.8.3 Break-even analysis

Selection algorithms expend compute and pipeline latency to reduce downstream labeling or training work. The **break-even point** marks the operating boundary where the financial savings from omitted samples exactly equal the computational and operational overhead of selecting them. If candidate scoring is computationally intensive or candidate pools are massive, a selection policy can easily consume more resources than it recovers.

Suppose labeling costs \$10/sample, active learning starts from 1,000 labeled samples (\$10,000), issues 100 queries per round at \$50 inference cost, and the random-labeling baseline requires 5,000 samples for target accuracy. If active learning reaches target accuracy with only 2,000 labeled samples, the ROI follows from comparing labeling and compute costs.

$$\begin{aligned}\text{Random labeling cost} &= 5,000 \times \$10/\text{sample} = \$50,000 \\ \text{Active learning cost} &= 2,000 \times \$10/\text{sample} + 10 \text{ rounds} \times \$50 = \$20,500 \\ \text{ROI} &= \frac{\$50,000 - \$20,500}{\$20,500} = 143.9\%\end{aligned}$$

Break-even occurs when avoided labeling costs equal selection overhead. A 20 percent labeling reduction may still yield negative ROI if scoring candidates and retraining proxy models require substantial accelerator hours.

9.8.4 Amortization across training runs

Static data selection techniques decouple dataset curation from the model training loop. Computational filters—such as MinHash deduplication, exact n -gram indexing, or embedding clustering—require substantial upfront engineering and preprocessing compute. However, production model development is inherently iterative: hyperparameter search, neural architecture search, ablation studies, and seed sweeps repeatedly train on the identical curated corpus. Evaluating these techniques on a single isolated run misrepresents their economics. **Amortized return on investment (ROI)** accounts for how upfront curation investments spread across an entire training campaign:

$$\text{Amortized ROI} = \frac{N_{\text{runs}} \times \text{Per-Run Savings} - \text{One-Time Investment}}{\text{One-Time Investment}} \times 100\%$$

Table 9.17 itemizes the upfront capital and compute expenditures against per-run savings for an illustrative MinHash deduplication pipeline.

Table 9.17: Deduplication Infrastructure Cost Components: The one-time investment covers engineering effort and initial compute; the per-run savings accrue with every subsequent training run on the deduplicated data.

Component	Cost
Build deduplication pipeline	\$50,000 (engineering time)
Compute MinHash signatures (one-time)	\$5,000
Per-run savings	\$10,000/run

Because the initial infrastructure and signature-generation costs are fixed, each subsequent training run compounds the net financial return. Three operational factors govern whether curation infrastructure reaches break-even:

- **Repeated training runs:** Hyperparameter search, model iterations, and scheduled retraining reuse the same selection infrastructure many times.
- **Shared datasets:** A cleaned or deduplicated corpus can support multiple teams or model architectures.
- **Broadly reusable techniques:** Methods such as deduplication transfer across models, whereas task-specific coresets may not.

Table 9.18 tracks how amortized return scales with campaign run count under these cost parameters.

Table 9.18: Amortized ROI over Multiple Training Runs: Under this scenario’s costs and savings, a deduplication pipeline that loses money on its first use becomes profitable when reused across enough runs. Infrastructure investments should be evaluated over their expected lifetime rather than from one use.

Number of Runs	Amortized ROI
1 run	-81.8% (net loss)
5 runs	-9.1% (near break-even)
10 runs	+81.8% (positive)
50 runs	+809.1% (highly profitable)

Deduplication achieves high investment leverage because eliminating redundant sequences reduces host-storage footprint and avoids wasted gradient updates across every downstream architecture. Conversely, sample-selection methods conditioned on proxy model gradients or uncertainty scores transfer poorly across disparate architectures, bounding their amortization window to narrow sweeps over a fixed model family. For exploratory or one-off training runs, low-overhead methods like uniform random sampling or basic augmentation deliver superior ROI by avoiding heavy infrastructure commitments.

Standard cost models assume a centralized data pipeline where a single host process evaluates importance metrics across a unified dataset. Scaling to high-throughput multi-accelerator training shards datasets across independent storage volumes and distributed worker ranks, erecting communication and synchronization bottlenecks that can dismantle these idealized single-node economics.

9.9 Distributed Selection

Data sharding dissolves the single-process abstraction. In a centralized pipeline, coreset algorithms sort a single global ranking, curricula enforce a uniform sequence, and active-learning loops evaluate a shared candidate pool. Distributed execution shards storage across independent nodes, computes scores on out-of-sync model checkpoints, and isolates worker visibility. Scaling data selection across clusters introduces two physical constraints: synthesizing representative global subsets from isolated shard perspectives, and maintaining coherent sample rankings as worker states diverge.

Whether distributed selection stays faithful to the global dataset or collapses into local shard heuristics depends on how much cross-worker coordination each technique requires against the bandwidth available to sustain it. The selection problem therefore has to be evaluated at the boundary where statistical value meets sharding and locality.

These are independent requirements. A coreset can remain representative yet cost too much to coordinate, while a cheap shard-local method can scale but systematically miss rare groups visible only in the global corpus. A distributed design therefore needs two acceptance tests: compare selected-data quality with a centralized or otherwise auditable reference, and compare selection overhead with the end-to-end training time it saves. Passing only one test is insufficient.

9.9.1 Strategies for distributed selection

In a basic data-parallel layout, each worker processes a distinct shard and model synchronization is handled separately. Data selection can add dependencies across those shards (table 9.19).

The selection dependencies admit several architectural solutions, each navigating a different point in the consistency-scalability trade-off space. The most straightforward approach centralizes selection while distributing training. A coordinator node performs selection on the full dataset, then distributes selected indices to workers. This preserves selection quality but introduces a single bottleneck:

```
Coordinator: score_all_samples() → selected_indices
Broadcast: selected_indices → all workers
Workers: train on subset(local_shard, selected_indices)
```

Table 9.19: Selection Dependencies in Distributed Training: The table starts from a common centralized formulation of each technique and identifies the coordination problem introduced by sharding. Distributed variants may relax rather than reproduce the centralized result exactly.

Technique	Single-Node Assumption	Distributed Challenge
Coreset Selection	Global view of dataset	Each worker sees only its shard
Active Learning	Centralized uncertainty scoring	Scoring requires model synchronization
Curriculum Learning	Global difficulty ordering	Workers may have different “hardest” samples
Deduplication	Hash table fits in memory	Distributed hash tables add latency

The semantics remain clean, but the coordinator becomes a single point of failure and a possible bandwidth bottleneck. Whether that overhead is acceptable depends on the selection payload, refresh rate, network path, and cluster size.

Hierarchical selection addresses this scalability limitation by distributing the selection computation itself. Each worker performs local selection on its shard, then a coordinator merges results:

```
Workers: local_selected = select_top_k(local_shard)
Coordinator: global_selected = merge_and_rerank(all local_selected)
Broadcast: final_indices → all workers
```

Shard-local selection reduces coordinator load but introduces a quality trade-off: local quotas and score distributions may not preserve the global ranking or rare groups spread unevenly across shards. The merge step therefore needs normalization, coverage constraints, or a second global pass.

When even hierarchical approaches prove too expensive, approximate global selection offers a fallback. These methods trade exactness for scalability through distributed approximate algorithms. Distributed MinHash enables deduplication by having each worker compute MinHash signatures independently; signatures are then aggregated to find near-duplicates across shards without requiring any single node to see all the data. Similarly, distributed uncertainty sampling allows workers to compute local uncertainty scores, with a global threshold determined by score distribution statistics rather than exact ranking.

9.9.2 Consistency challenges in active learning

The approximate selection strategies assume static selection criteria, but active learning introduces an additional complication: the model changes during selection. Consider what happens when Worker A scores samples using the model at step t while Worker B simultaneously updates the model to step $t + 1$. Worker A’s scores are now stale and may select samples that the updated model would rank differently.

The scoring schedule therefore defines the meaning of a query, not merely its cost. A reproducible system versions the model checkpoint, candidate-pool snapshot, scoring rule, and selected indices together. It can then measure how much ranking agreement decays with checkpoint age and choose a refresh interval from that evidence rather than from an arbitrary number of steps.

Several strategies mitigate this staleness problem, each with distinct overhead characteristics:

- **Synchronous scoring:** All workers pause training and score simultaneously, guaranteeing consistency but at substantial cost in GPU utilization.
- **Periodic score refresh:** Workers re-score every k epochs rather than every batch, trading freshness for reduced overhead.
- **Checkpoint-robust selection:** The system selects samples that exhibit high uncertainty under multiple model checkpoints, keeping selection decisions valid as the model evolves.

An illustrative 8-node GPU-cluster scenario demonstrates how these refresh strategies interact with storage bandwidth and compute efficiency.

Example 9.6: Distributed coreset selection

Scenario: An 8-node A100 GPU cluster selects a 10 percent coreset from ImageNet (1.3M images) across distributed workers.

Diagnosis: Centralized coreset scoring on a single node creates memory and compute bottlenecks. Distributed pipeline staging (shard-local embedding → parallel deduplication → local EL2N scoring → centralized top- k merge) processes dataset selection in 67 minutes.

Systems lesson: Distributed coreset selection can remove single-node memory as the scoring limit, as illustrated in figure 9.11. Sharding embedding and scoring spreads work across the cluster, while the centralized merge remains a coordination cost that must satisfy the selection inequality.

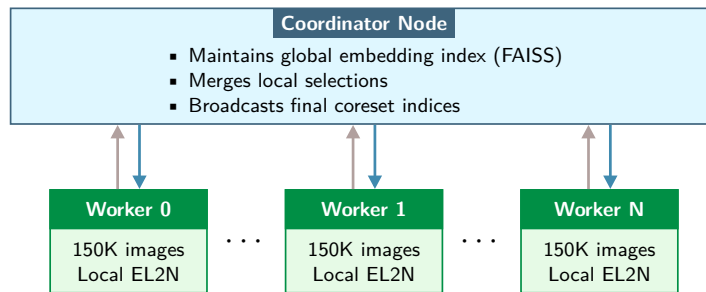


Figure 9.11: Distributed Coreset Selection Architecture: Parallel shard-local scoring reduces the work assigned to a centralized rank-merging coordinator. Workers send local EL2N scores upward; the coordinator merges them and broadcasts final coreset indices. The design is beneficial only when this coordination overhead remains below the training time saved by the coreset.

Positive ROI can erode quickly when workers coordinate frequently during training. Distributed selection introduces a coordination tax whose size depends on what is synchronized and how often. That tax must remain smaller than the training time saved; if it approaches the limit, simplify the strategy or increase the refresh interval.

A further constraint arises from cluster network topology. Gradient synchronization may use accelerator collectives over dedicated links, while embedding vectors, score arrays, and coreset indices may follow host or storage paths instead. The actual route depends on system architecture. Selection traffic can therefore expose CPU, network interface card, storage, or collective-network bottlenecks that the training profile alone does not reveal.

Real ML systems combine data selection with model-level efficiency, machine-level throughput, and distributed training simultaneously. These optimizations interact in ways that can amplify or undermine each other, and understanding these interactions is essential for designing efficient end-to-end pipelines.

9.10 Cross-Layer Interactions

In the D·A·M taxonomy (Data · Algorithm · Machine), data selection operates on the data dimension to prune the training workload before compute begins. Yet execution time under the iron law of ML systems depends on how input pruning alters the algorithm (model representations, loss landscapes, and compressibility) and the machine (memory bandwidth, kernel arithmetic intensity, and inter-accelerator communication). These cross-layer interactions can amplify efficiency gains across the stack or introduce unexpected bottlenecks that negate upstream savings. System efficiency requires co-designing data curation alongside model compression and hardware scheduling rather than optimizing each component in isolation.

9.10.1 Model-level efficiency

Model-level efficiency reduces the parameter count and arithmetic cost of the trained model through techniques such as pruning, lower-precision representation, and distillation. The composition of the

training corpus shapes internal representations, altering how susceptible the resulting network is to post-training compression. Yet counting eliminated operations overlooks how physical execution substrates respond to sparse structures (9.2). Chapter 10 and chapter 11 examine the architectural foundations of model compression and hardware acceleration in detail.

Systems Perspective 9.2: The sparsity latency trap

Context: The compressibility advantage from data selection is counted in arithmetic operations, but model compression research repeatedly shows that pruning reduces those counts more easily than it reduces wall-clock latency.

Failure mode: FLOPs can decrease dramatically while inference latency stays flat or increases. Dense matrix multiplication hardware rewards regular layout and reuse, while sparse matrices require irregular memory access, metadata checks, and address jumps. Unless the sparsity pattern matches the execution substrate, the overhead of managing sparsity can outweigh the reduction in arithmetic.

Systems insight: FLOPs are *not* latency. A 99 percent reduction in operations can yield a 0 percent reduction in time if the remaining operations are memory bound or cache-inefficient. Optimization must target the hardware’s binding bottleneck rather than an abstract metric alone (Hoeffler et al. 2021).

This interaction between data selection and model compression traces back to how networks encode representations during training. A model trained on repetitive data frequently allocates capacity to redundant features that post-training pruning subsequently discards, wasting the training compute expended to learn them. Conversely, a model trained on a curated corpus of informative, diverse examples often acquires more compact representations from the start. That initial efficiency can simplify subsequent pruning or quantization, though this remains an empirical hypothesis to measure rather than an invariant property of every selected dataset. Data selection and model compression are complementary levers within the D·A·M taxonomy; their interaction must be evaluated jointly by benchmarking end-to-end serving latency and task accuracy rather than assuming that upstream dataset reduction automatically guarantees downstream compressibility.

9.10.2 Machine-level throughput

While model-level efficiency determines the computational work remaining after training, machine-level throughput determines how efficiently hardware executes the training loop itself. Under the iron law of ML systems, training runtime depends on whether execution is bound by input data transfer (D_{vol}/BW), arithmetic execution ($O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$), or scheduling latency (L_{lat}). Pruning samples reduces total floating-point operations, but non-uniform sampling and dynamic filtering alter data-loader access patterns and expose new hardware limits. Table 9.20 outlines how selection strategies shift the dominant system bottleneck.

Table 9.20: Diagnosing Bottlenecks after Data Selection: Dataset and access-pattern changes can expose a different bottleneck, but dataset size alone does not determine the limiting resource. Re-profile the pipeline and match the response to the measured condition.

Observed Condition	Possible Bottleneck	Candidate Response
Input rate below accelerator demand	Storage, decode, or host link	Sharding, prefetching, parallel decode
Low-intensity training kernels	Accelerator memory bandwidth	Fusion, reduced traffic, lower precision
Dynamic scoring dominates iteration	Selection compute or index I/O	Proxy models, cached scores or embeddings

Data selection can therefore shift the hardware execution regime. A technique that reduces sample count by 80 percent may relieve compute pressure on tensor cores, only to expose random-access storage latency, host-to-device PCIe bandwidth, or scoring overhead as the new critical path. Relieving that subsequent bottleneck requires matching the engineering response to the active constraint: caching selection scores, fusing memory-bound operations, or pre-staging contiguous

shards into host memory before accelerator dispatch. Before applying aggressive data reduction, engineers must profile the full pipeline to verify which bottleneck actually binds execution.

9.10.3 Distributed training

The hardware bottleneck analysis in section 9.10.2 focuses on execution on a single accelerator. When scaling to multi-accelerator training—such as a single node equipped with two to eight devices—data selection alters both communication volume and memory synchronization.

In data-parallel training across accelerators, each device computes forward and backward passes on a local micro-batch before synchronizing parameter gradients via an all-reduce collective. With per-device batch size and epoch count held fixed, pruning the training corpus reduces the total number of parameter updates across a run, directly eliminating collective communication rounds over PCIe or NVLink interconnects. Furthermore, smaller per-worker shards can increase the proportion of the dataset that fits into local host RAM or accelerator cache, reducing repetitive storage streaming.

These communication savings must be balanced against the coordination overhead analyzed in section 9.9. While gradient synchronization exploits dedicated high-bandwidth interconnects, dynamic sample scoring and index exchanges often route through host memory and CPU dispatch threads. A selection policy that scales efficiently on a single accelerator can introduce serialization bottlenecks or imbalanced shard workloads when partitioned across eight parallel workers, stalling devices at synchronization barriers and eroding the compute time saved by pruning.

9.10.4 The optimization stack

Pairwise interactions expose localized bottlenecks, but production deployments execute these optimizations as an integrated pipeline. As traced in the full optimization stack (figure 9.12), every stage from data ingestion through hardware execution amplifies or attenuates downstream efficiency.

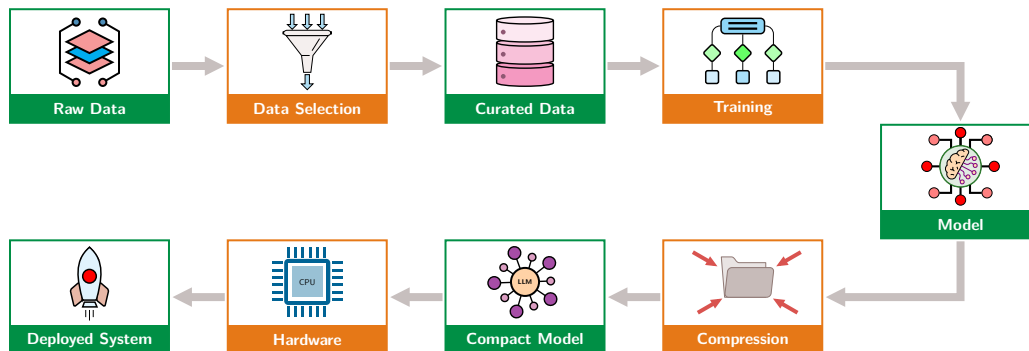


Figure 9.12: The Optimization Stack: Data artifacts flow through selection and training before the resulting model enters compression and deployment. Upstream changes propagate quality and cost effects downstream, but reducing training data does not automatically simplify the model or deployment hardware.

The pipeline in figure 9.12 reveals why data selection occupies a strategic position at the head of the optimization stack. With the training schedule held fixed, halving the dataset volume can halve sample-level training work. Yet dataset pruning does not by itself shrink parameter footprint, improve quantization headroom, or relax serving latency constraints. Each downstream tier inherits both the computational savings and any representational deficits created upstream; an aggressive filter that discards rare but critical edge cases can force downstream engineers to train longer or abandon aggressive quantization to preserve task accuracy.

Quantifying these compound trade-offs requires determining whether nominal sample reduction translates into true end-to-end efficiency without silently compromising generalization. Answering that question requires a disciplined measurement framework: quantitative metrics that balance arithmetic savings, data movement overhead, and empirical task quality across the complete pipeline.

9.11 Measurement Framework

The cross-layer stack makes a measurement framework unavoidable: every data selection technique claims to improve efficiency, but only rigorous measurement separates real savings from shifted costs or hidden quality loss. The core metrics in section 9.11.1 tie sample reduction to accuracy, cost, and deployment coverage so that a smaller dataset is judged by what it preserves, rather than by what it removes alone.

9.11.1 Core metrics

The core metrics connect sample reduction to model quality instead of reporting accuracy alone. They measure accuracy gain per sample (performance-per-data), cumulative learning efficiency across dataset scales (area under the learning curve), and dataset reduction at a target accuracy (data compression ratio).

Performance-per-data

The most direct metric, **performance-per-data (PPD)**, measures accuracy gain per sample:

$$\text{PPD}(n) = \frac{\text{Accuracy}(n) - \text{Accuracy}(0)}{n}$$

where n is the number of training samples. A higher PPD indicates greater average improvement per sample over the stated interval. Its shape must be measured because diminishing returns and their onset are workload-dependent.

Area under the learning curve

Rather than comparing at a single point, the **area under the learning curve (AULC)** integrates performance across all dataset sizes:

$$\text{AULC} = \int_0^D \text{Accuracy}(n) dn$$

where n is the dataset size and D is the total dataset size.

For the same metric and integration range, a higher AULC means the strategy achieves stronger performance with fewer samples. Comparisons must use the same upper limit D or normalize the integral.

Data compression ratio

For coreset methods, the **data compression ratio (DCR)** measures how much data reduction is achieved at a target accuracy:

$$\text{DCR} = \frac{D_{\text{full}}}{D_{\text{coreset}}} \text{ at Accuracy}_{\text{target}}$$

A DCR of $5\times$ means the coreset achieves target accuracy with 20 percent of the data.

9.11.2 The compute-optimal frontier

While the core metrics in section 9.11.1 measure individual techniques, a higher-level diagnostic is also needed to test whether the overall training strategy is data-limited or compute-limited. Neural scaling-law experiments (Kaplan et al. 2020; Hoffmann et al. 2022) fit power-law relationships over particular model families, datasets, and compute ranges. Controlled sweeps can use those fitted relationships to compare model and data allocations, but no single ratio diagnoses every training run.

The Chinchilla study³³ (Hoffmann et al. 2022) fit a compute-optimal balance between model size and training data within its experimental regime. Allocations on either side of that fitted balance used the study's compute budget less effectively.

The optimal balance defines a **compute-optimal frontier**: the best achievable performance at each compute budget when data and model size are properly balanced. Figure 9.13 sketches this diagnostic as a conceptual frontier rather than a fitted Chinchilla result.

³³ | **Chinchilla**: Chinchilla is a 70-billion-parameter language model trained on 1.4 trillion tokens. In the experiments reported by Hoffmann et al. (2022), it outperformed the larger GPT-3 on most evaluated tasks under a similar training-compute budget. The fitted compute-optimal allocation scaled parameters and tokens at roughly equal rates within the study's experimental regime; it is not a universal prescription for every architecture, dataset, or training recipe.

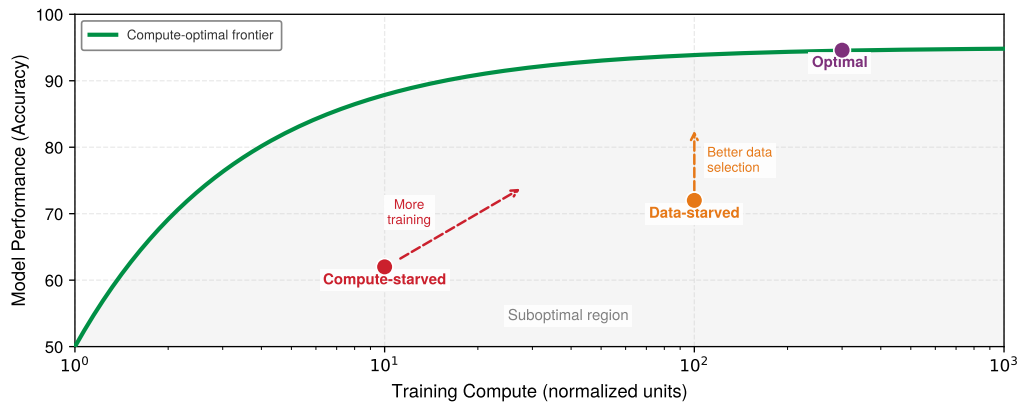


Figure 9.13: Illustrative Compute-Optimal Frontier: The green curve is a conceptual frontier over normalized compute, not a fitted result from Chinchilla. Points below it illustrate allocations that a controlled model-size and token-count sweep might identify as data- or compute-limited.

Against a frontier fitted from controlled sweeps, a point’s location can suggest the next experiment. A data-limited allocation motivates tests of corpus quality, coverage, or selection; a compute-limited allocation motivates tests of effective throughput, duration, or model size. The conceptual points in figure 9.13 do not diagnose a real run by themselves, and points near a fitted frontier remain specific to the model family, data, objective, and compute range used to estimate it.

Interpreting the Chinchilla result

Within the fitted Chinchilla regime, compute-optimal model parameters and training tokens grew at roughly equal rates.³⁴ Specifically, training a dense transformer requires approximately $6ND$ FLOPs ($2ND$ for the forward pass and $4ND$ for the backward pass with activation recomputation, where N is parameter count and D is token count). Under an equal scaling regime where parameters and tokens grow proportionally ($N \propto D$), total compute scales quadratically ($C \approx 6D^2$), directly implying that optimal token count scales as the square root of the compute budget ($D_{\text{opt}} \propto \sqrt{C}$). Doubling compute therefore corresponds to about $\sqrt{2} - 1$, or 41.4 percent, more tokens under those assumptions. The commonly quoted 20 tokens per parameter is a useful reference point from that study, not a universal optimum.

Applying the diagnostic

If a training run underperforms expectations, extending one run can reveal whether additional optimization steps still help, but a plateau does not identify data starvation by itself. Learning-rate schedules, optimization limits, model capacity, and data quality can all produce a plateau. A reliable diagnosis compares controlled runs that vary model size, token count, and compute allocation while holding the evaluation protocol fixed.

In figure 9.14 the two curves diverge: a data-efficient selection strategy (blue) reaches the performance plateau with fewer samples than random sampling (gray). The horizontal gap represents that sample reduction and can translate into compute savings when the per-sample work and training schedule remain fixed; the vertical gap marked by the red arrow represents the performance gained at a fixed dataset size.

These evaluation metrics govern the transition point where indiscriminate data collection yields to targeted curation, identifying when marginal samples waste training compute rather than improving generalization. Yet algorithmic efficiency does not guarantee that a reduced dataset preserves model quality across operational edge cases.

Data selection techniques implicitly rank sample value, and validating that a curated dataset preserves model quality requires systematic benchmarking across three dimensions. Coverage metrics validate that coreset selection preserved representation across classes and demographic groups. Distribution alignment metrics (such as KL divergence and population stability index, which section B.2.2 defines) detect whether the curated training set drifted from the deployment

³⁴ | **Tokens per parameter:** The ratio D/P compares training tokens with model parameters; GPT-3 used approximately 1.7 tokens per parameter (Brown et al. 2020), while Llama 2 70B used approximately 28.6 (Touvron, Martin, et al. 2023). These descriptive ratios do not establish whether either run is compute-optimal under a different architecture, corpus, objective, or hardware budget. Controlled model-size and token-count sweeps are required for that diagnosis.

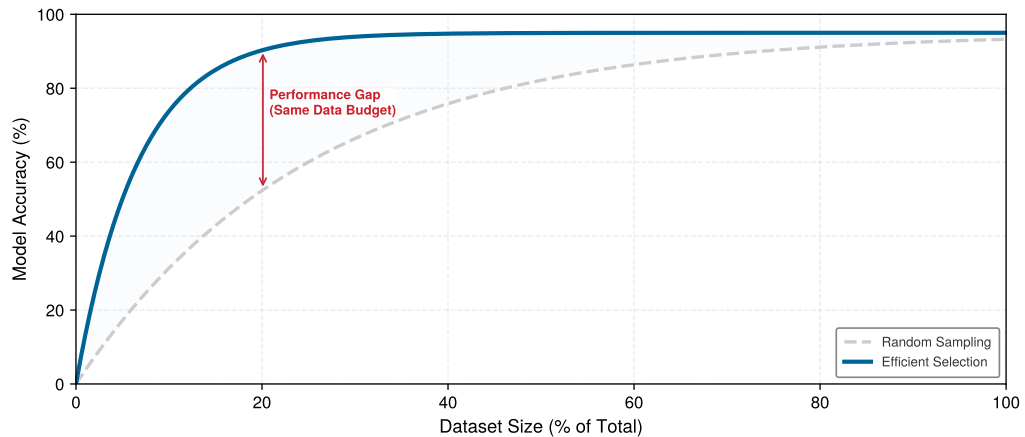


Figure 9.14: Illustrative Diminishing Returns of Data: The synthetic data-efficient curve (blue) reaches the same plateau with fewer samples than random sampling (gray). The vertical red arrow marks the modeled performance gap at a fixed dataset size; measured curves need not have this shape.

distribution. Label quality metrics (inter-annotator agreement, confident learning) validate that active learning did not introduce systematic labeling errors. A 50 percent dataset reduction is only valuable if benchmarking confirms the model maintains target accuracy, calibration, and robustness. Chapter 12 later generalizes these ideas into broader model-and-data evaluation protocols; here, the point is narrower: a curated dataset must be validated against the task distribution rather than against its reduction ratio alone.

The validation target can itself be unreliable. Benchmark accuracy can overstate generalization when test sets fail to capture operational distribution shifts, demanding the same scrutiny for evaluation benchmarks as for curated training sets.



Example 9.7: The benchmark replication gap

Scenario: Classifiers evaluated on newly collected test sets experienced 3 to 15 percentage-point accuracy drops on CIFAR-10 and 11 to 14 points on ImageNet (Recht et al. 2019).

Diagnosis: The replicated collection produced slightly harder images than the original test sets. The study found that benchmark adaptivity did not explain the gap, even though repeated use of a fixed test set remains a general evaluation risk.

Systems lesson: Evaluation-set construction affects measured generalization. Data-selection pipelines should validate curated subsets on independently collected and deployment-relevant test sets rather than trusting one long-reused benchmark.

An efficiency gain measured against a single benchmark can evaporate under distribution shift; a curated subset must demonstrate consistent quality across multiple held-out distributions before its reported reduction ratio is trusted.



Lighthouse 9.3: Lighthouse data selection

Data selection principles apply to all five lighthouse models, though the dominant constraint determines the technique. Table 9.21 maps each workload to a priority: for example, compute-bound ResNet-50 benefits from coresets selection, memory-bound GPT-2/Llama from deduplication, and the tiny KWS model from augmentation and synthesis.

Table 9.21: Data Selection Priorities by Lighthouse: Primary bottleneck and candidate data-selection priority for each lighthouse workload. The pairing suggests a hypothesis to test; the dominant resource constraint alone does not guarantee that a technique pays off.

Lighthouse	Primary Bottleneck	Data Selection Priority
ResNet-50	Compute	Coreset selection directly reduces training FLOPs
GPT-2/Llama	Memory bandwidth	Deduplication reduces corpus size; curriculum learning improves token efficiency
MobileNetV2	Latency/Power	Validate augmentation against the target model and deployment invariances
DLRM	Memory capacity	Interaction deduplication and embedding pruning reduce table size
Keyword Spotting	Extreme constraints	Augmentation and synthesis create datasets from minimal seeds

Across these workloads, data selection functions as a systems optimization tailored to relieve whichever physical resource binds the pipeline.

Across the optimization stack, disciplined metrics separate genuine compute savings from deferred computational debt. A selection strategy succeeds only when the arithmetic and data-movement overhead incurred to score, filter, and stage samples is exceeded by the downstream training and serving savings achieved at target quality.

9.12 Fallacies and Pitfalls

Data selection involves counterintuitive diminishing returns that contradict the “more is better” intuition from traditional machine learning. The following errors fall into three groups: conceptual fallacies about what data selection can achieve, implementation pitfalls that arise when correct strategies meet engineering realities, and transfer errors that occur when benchmark results are applied uncritically to new domains.

Fallacy: *Data is the new oil, so more is always better.*

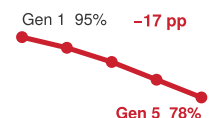
More data does not imply proportional accuracy gains because learning curves exhibit power-law diminishing returns. Test loss diminishes sub-linearly with sample volume ($L(N) \propto N^{-\alpha}$), meaning each successive error reduction demands an exponentially larger corpus. In the scenario modeled in table 9.1, scaling a dataset tenfold from 1M to 10M samples yields only 4 percentage points of accuracy gain. In systems terms, that $10\times$ volume expansion requires a tenfold increase in storage I/O, host-to-device transfers, and accelerator FLOPs, expending massive cluster energy to process redundant, low-gradient examples that contribute negligible information to parameter updates. Engineering teams must measure the empirical learning curve and marginal compute cost of their specific workload rather than assuming uncensored data scaling will deliver proportional value.

Pitfall: *Replacing real-data validation with synthetic-only training data.*

Engineers often assume generative models can replace empirical data collection with inexhaustible synthetic samples at zero marginal acquisition cost. Synthetic-only training fails along two primary failure modes. First, as examined in section 9.5.3 and table 9.9, generative distributions suffer from domain gap: synthetic data can diverge from the physical deployment manifold, biasing the learned decision boundary in figure 9.8 and causing catastrophic classification failures on real-world inputs. Second, recursive training on model-generated data triggers model collapse (Shumailov et al. 2024). As illustrated in the margin, feeding synthetic outputs back into subsequent training cycles progressively erodes distribution tails: in this scenario, evaluation accuracy degrades from 95 percent to 78 percent across five recursive generations, suffering a 17 percentage-point collapse. While synthetic augmentation provides useful coverage within an illustrative 50–80 percent blend, synthetic mixtures must always be validated and anchored against curated real-world distributions.

Fallacy: *Data selection is merely data cleaning.*

Engineers often conflate data quality (correcting malformed schemas, corrupted labels, and noisy tokens) with data value under a specific training procedure. A dataset can be completely clean,



Recursive synthetic-data training degrades accuracy across generations.

correctly labeled, and grammatically impeccable, yet mathematically redundant: if the model has already minimized loss on a feature cluster, processing additional identical samples yields zero gradient signal while burning memory bandwidth and accelerator FLOPs. Data cleaning addresses intrinsic data defects, whereas data selection actively maximizes the learning return per FLOP spent. As illustrated by the uncertainty sampling boundary in figure 9.4 and the geometric objectives in section 9.2.2, targeting informative boundary examples increases the ICR—yielding a 1.8× higher ICR in our worked coreset scenario. Cleaning ensures data correctness; selection optimizes training compute efficiency.

Pitfall: *Treating data selection as a budget-only tactic.*

Practitioners often view data selection as a technique reserved for resource-constrained edge deployments or budget-limited teams, assuming that frontier foundation models can simply brute-force convergence by scaling cluster size. In reality, large-scale training is fundamentally bounded by the data wall (figure 9.1), where available accelerator capacity outruns the supply of accessible, high-quality, task-relevant human data. Furthermore, the economic return of selection scales with cluster expenditure: a 10 percent efficiency gain on a \$100M frontier training run saves \$10M under our baseline assumptions. When selection infrastructure, pre-computed importance scores, and cached embeddings are amortized across repeated training runs and downstream adaptations (section 9.8.4), intelligent filtering becomes a foundational economic lever for large-scale engineering systems.

Fallacy: *Selection overhead is too small to change training economics.*

A data selection strategy is viable only if the computational cost of scoring and filtering is strictly less than the training compute it saves. As formalized by the selection inequality in equation 9.2, end-to-end viability requires $T_{\text{selection}} + T_{\text{train}}(\text{subset}) < T_{\text{train}}(\text{full})$. Consider a baseline training run requiring 8 hours, where training on an optimized coreset takes 2-hour. If a complex selection algorithm requires 10 hours to compute sample importance, the selection step alone consumes 5× the duration of the accelerated training run and exceeds the full training baseline itself, generating a net compute deficit. To satisfy equation 9.2, production pipelines decouple scoring from the primary model by utilizing cached embeddings or lightweight proxy networks. In our illustrative comparison, proxy-based EL2N scoring completes in 30 minutes—just 6.2 percent of the full-training baseline—satisfying the inequality and delivering tangible wall-clock savings.

Pitfall: *Pruning rare classes into oblivion.*

Standard selection objectives minimize aggregate empirical loss, giving tail slices minimal influence over global sampling probabilities. In a 1M dataset where rare classes comprise 0.1 percent of the data (1,000 examples), an unstratified 10 percent coreset retains only 100 rare-class examples in expectation—falling far below the 150-sample threshold required to learn stable feature representations. While **importance sampling** can theoretically compensate by weighting loss contributions by inverse selection probabilities, extreme sampling weights inflate gradient variance. This gradient variance destabilizes optimizer momentum trajectories, forcing smaller learning rates that slow cluster convergence. As recommended in section 9.2.2, production coreset pipelines must enforce hard stratification floors per class or deployment-critical slice before optimizing the remaining selection budget.

Fallacy: *Deduplicating training data is enough to make evaluation reliable.*

Engineers often assume that eliminating duplicate sequences within the training partition prevents memorization and ensures clean evaluation. However, web-scale ingestion pipelines scrape training, validation, and benchmark test splits from common web crawls, introducing silent cross-split leakage. When evaluation sets contain duplicates or near-duplicates of training documents, validation metrics measure memorization rather than out-of-distribution generalization. A comprehensive deduplication study revealed that train-test overlap affects over 4 percent of validation examples across standard language-modeling benchmarks (Lee et al. 2022). Deduplicating training data in isolation leaves this evaluation contamination untouched. Rigorous pipeline hygiene requires cross-partition deduplication using n -gram matching or MinHash LSH (section 9.2.3), calibrating near-duplicate similarity thresholds to purge leaked sequences across all splits simultaneously.

Pitfall: *Active learning without considering annotation latency.*

Theoretical formulations of active learning assume a zero-latency oracle that labels queried samples instantaneously, allowing the model to incorporate new gradients immediately before selecting the subsequent query batch. In production systems, human annotation queues introduce substantial turnaround delays—often an illustrative 14-day latency between sample dispatch and label return. This latency breaks the active learning feedback loop in two ways. If accelerator clusters halt training while awaiting labels, expensive hardware sits idle, destroying cluster compute utilization. Conversely, if training continues asynchronously on background data while annotations are pending, model weights drift across training steps; by the time the labels arrive, the uncertainty scores that justified querying those samples are obsolete. Furthermore, dispatching massive batches to amortize labeling turnaround causes information redundancy within the query batch unless diversity penalties are applied. The return on investment for active learning is bounded not just by label pricing, but by the ratio of annotation turnaround latency to the model retraining cycle.

Fallacy: *If a technique works on ImageNet, it will work on my dataset.*

Data-selection pruning ratios derived from standard vision benchmarks cannot be blindly transferred to domain-specific workloads. Benchmark datasets like CIFAR-10 and ImageNet exhibit high semantic redundancy across canonical object classes, allowing aggressive coreset pruning (often 30 to 50 percent) without measurable accuracy degradation. In contrast, specialized domains—such as histopathology, satellite remote sensing, or industrial defect detection—exhibit heavy-tailed feature distributions where safety-critical patterns appear in only a few rare samples. Discarding 50 percent of examples based on an ImageNet heuristic can eliminate the sole boundary examples that separate subtle pathologies, triggering catastrophic domain degradation. Furthermore, geometric coreset algorithms rely on embedding spaces: if the feature extractor was pretrained on natural images, its representations lack the inductive bias needed to resolve fine-grained domain distinctions. Engineering teams must measure the redundancy of their own data, evaluate representations against target downstream tasks, and validate retention fractions across safety-critical slices before applying pruning.

Pitfall: *Optimizing data selection metrics instead of deployment metrics.*

Selection algorithms optimize surrogate mathematical objectives—such as overall loss reduction, area under the learning curve (AULC), or progress per dollar (PPD, section 9.11.1)—evaluated as an aggregate scalar over the validation corpus. A selection policy retaining a 10 percent coreset can achieve superior aggregate PPD by prioritizing dense clusters of common, easy samples that accelerate average convergence, while discarding ambiguous edge cases and rare slices. In production, however, system failures occur on tail slices, edge cases, and protected sub-populations rather than dataset averages. Maximizing aggregate selection efficiency without subgroup constraints produces models that pass global validation gates but fail catastrophically in deployment. As mandated by the measurement framework in section 9.11, selection policies must be evaluated using stratified, slice-level benchmarks that enforce coverage across demographic groups, rare categories, and operational edge cases, even when doing so slightly lowers average PPD.

9.13 Summary

Data selection treats training volume as an engineering variable rather than an unchangeable constraint. Where traditional machine learning asks how many samples are required to reach target accuracy, systems engineering optimizes total cost across compute, storage, labeling, energy, and wall-clock time. Curated datasets can match or outperform massive uncured corpora because marginal learning signal varies by orders of magnitude across raw examples. Within the optimization ordering, selection addresses the first question: eliminating unnecessary work before model training begins.

This systems perspective becomes actionable through three quantitative tools: the ICR, the selection inequality, and the total data cost model. The information-compute ratio quantifies learning gained per floating-point operation, establishing whether an example justifies its training pass. The selection inequality enforces that the compute and wall-clock overhead of scoring candidates must remain strictly less than the training time saved on the retained subset. Together with return-on-

investment and break-even analysis, these formulations determine whether a proposed selection mechanism saves resources or merely shifts expenditure between stages.

The three-stage optimization pipeline addresses different phases of this cost equation: static pruning removes redundancy before training through coreset selection and deduplication; dynamic selection prioritizes informative examples during training through curriculum and active learning; and synthetic generation creates data where none exists through augmentation, simulation, and distillation. Together, these strategies address the “data wall,” the structural asymmetry between rapidly growing compute capacity and slowly growing high-quality data.

Self-supervised learning represents an extreme point on the data-selection spectrum. Pretraining shifts the bulk of representation learning into an unannotated upstream phase, allowing downstream tasks to reach target quality with orders of magnitude fewer task-specific labels. This upfront investment becomes economically compelling when the pretraining cost is amortized across dozens or hundreds of specialized adaptations.

Translating these selection strategies into production requires solving concrete systems bottlenecks. The selection inequality in equation 9.2 gates every technique: any scoring pass that exceeds the wall-clock time saved during training produces a net loss. Proxy models and shard-based loaders reconcile dynamic selection with sequential storage bandwidth, preventing random sample access from stalling accelerator execution. When input pipelines remain the throughput bottleneck, data echoing reuses batches to recover idle GPU cycles. Finally, diagnostic metrics—points per dollar (PPD), area under the learning curve (AULC), and data-to-compute ratio (DCR)—reveal whether a model is data-limited or compute-limited, directing investment toward the binding resource.



Key Takeaways: Curate, do not accumulate

- **Selection optimizes system cost:** The objective is minimizing total cost across compute, storage, labeling, and energy rather than reducing sample counts alone. The information-compute ratio quantifies this trade-off as learning gained per FLOP spent, translating into wall-clock speedup only under compute-bound conditions.
- **Start with deduplication:** Exact deduplication is often the lowest-risk first technique because it removes repeated bytes without a learned scoring pass. Apply near-duplicate and semantic deduplication only after validating thresholds against downstream quality metrics.
- **The selection inequality gates every technique:** Downstream training savings must strictly exceed selection overhead: $T_{\text{selection}} + T_{\text{train}}(\text{subset}) < T_{\text{train}}(\text{full})$. Lightweight proxy models and cached embeddings keep $T_{\text{selection}}$ low, whereas unoptimized selection algorithms can consume all the compute they save.
- **Dynamic selection adapts the data diet:** Curriculum learning changes presentation order, while active learning changes which samples receive labels. Either improves efficiency when its scoring rule, coverage, and refresh overhead are validated.
- **Self-supervised pretraining can amortize adaptation cost:** Pretraining once and fine-tuning many times spreads pretraining cost across downstream tasks. The worked scenario assumes a $100\times$ label reduction and a $20\times$ marginal-compute reduction; realized savings depend on transfer quality and reuse.
- **Synthetic data is a supplement, not a replacement:** The 50–80 percent synthetic mixture is illustrative and workload-dependent, not a universal optimum. Pure synthetic training risks model collapse and domain-gap degradation.
- **Validated workload reduction compounds downstream:** When selection safely eliminates examples or tokens without adding compensating steps, that work never reaches later model or machine optimizations. Dynamic scoring and synthesis add their own costs, so the combined saving must be measured rather than assumed.

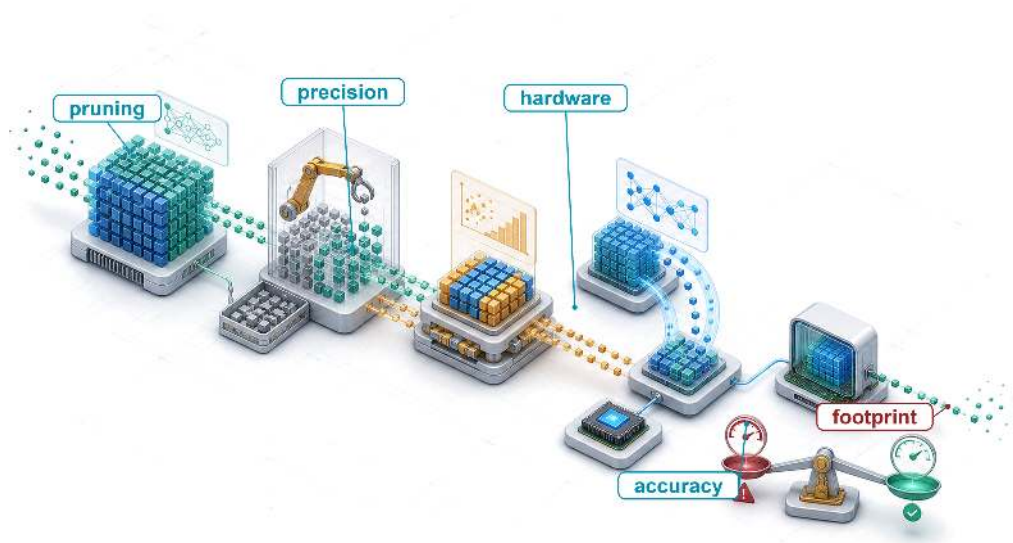
The governing invariant of data selection is that training examples provide vastly unequal marginal learning signal per byte transferred and FLOP executed. Within the D·A·M taxonomy, selection

operates at the head of the system: every redundant sample or uninformative token safely eliminated directly reduces both data volume (D_{vol}) and operation count (O) in the iron law of ML systems (principle 3). That upstream reduction permanently removes work before downstream algorithmic compression or hardware acceleration begins, provided selection overhead remains strictly bounded by the selection inequality.

What's Next: From data to algorithms

Data selection changes *what* the system learns from; even the best data cannot make an inefficient model run fast on constrained hardware. Chapter 10 shifts the focus to *how* the system represents what it learns, applying pruning, quantization, and knowledge distillation to reduce the computational cost of the model artifact itself.

Model Compression

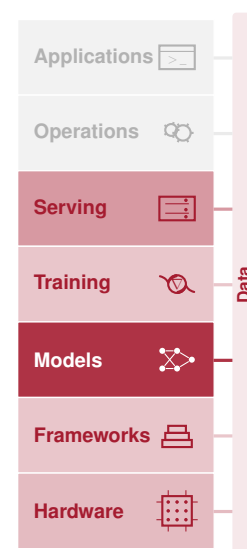


- 10.1 Optimization Framework
- 10.2 Deployment Context
- 10.3 Structural Optimization
- 10.4 Quantization and Precision
- 10.5 Architectural Efficiency
- 10.6 Technique Selection
- 10.7 Optimization Strategies
- 10.8 Efficiency Measurement
- 10.9 Implementation Tools
- 10.10 Fallacies and Pitfalls
- 10.11 Summary

Purpose

Why do the models that win benchmarks rarely become the models that run in production?

Training produced a capable model, yet capability alone does not guarantee deployability. Cloud, Edge, Mobile, and TinyML each impose constraints that research benchmarks ignore, including memory budgets measured in megabytes rather than gigabytes, latency targets measured in milliseconds rather than seconds, and power envelopes measured in milliwatts rather than kilowatts. Research optimizes for accuracy on held-out test sets; production optimizes for accuracy per dollar, accuracy per watt, and accuracy per millisecond. Models that win benchmarks are often larger, slower, and more resource-intensive than production constraints permit. Bridging that gap requires a systematic discipline of compression that trades capabilities the deployment does not need for constraints it cannot violate. Many trained models carry more precision, more connections, or more capacity than a deployment context demands, and some of that surplus can be removed while preserving required behavior. Yet a smaller representation is not automatically a faster one. The hardware and software stack must exploit the new precision or structure; otherwise a nominal reduction can merely move the bottleneck while latency remains unchanged. Applied well, compression can substantially reduce model size, transforming a research artifact that runs only in a data center into a production asset for a phone, sensor, or microcontroller. The discipline is not simply about making models smaller but about making the right models possible for their physical environment. In D·A·M terms, compression enacts algorithm-machine co-design on the model itself, rewriting its mathematical structure to fit the physical constraints of the machine.





Learning Objectives

- Explain compression as algorithm-machine co-design that trades surplus capacity for memory, latency, and energy constraints
- Compare pruning, distillation, quantization, and architecture search by the resource constraint each relaxes
- Calculate parameter memory, precision, and sparsity reductions to estimate best-case compression gains
- Apply post-training, quantization-aware, and weight-only strategies under accuracy and hardware constraints
- Select structured pruning and operator choices that map to available accelerator kernels
- Design compression pipelines that order pruning, distillation, and quantization to preserve deployment accuracy
- Evaluate measured latency, energy, and accuracy on target hardware rather than relying on FLOP counts

10.1 Optimization Framework

A 7-billion parameter language model requires 14 GB merely to store its weights in FP16. The intended deployment target is a smartphone with 8 GB of RAM shared across the operating system, background applications, and the user interface. Even if the operating system allocated its entire memory pool to a single process, 14 GB exceeds the physical capacity of the device. Furthermore, autoregressive token generation is memory-bandwidth bound: each generated token requires reading every parameter from DRAM into processor registers. Over a mobile low-power DDR (LPDDR) bus delivering roughly 50 GB/s, streaming 14 GB takes nearly 300 ms per token and draws several joules, rapidly draining the battery and violating the device's sustained thermal envelope of 3 to 5 W. This physical mismatch between model footprint and edge hardware capacity defines the core engineering mandate of model compression.

Under the silicon contract (principle 4), every machine learning model strikes a physical performance bargain with its hardware: compute throughput, memory bandwidth, memory capacity, and fixed launch overhead determine which physical resource binds first. During training, this bargain is negotiated upward: server-class accelerator clusters provide the aggregate memory and floating-point throughput needed to support wide layers, high numerical precision, and large auxiliary state. Training must maintain gradients, optimizer moment buffers, and activation caches across backward passes. Inference deployment alters these requirements completely: execution is forward-only, intermediate activations can be overwritten immediately, and optimizer states disappear. While mixed-precision methods improve training throughput by selectively adopting FP16 or BF16 (section 8.6.3), deployment allows reducing precision further to INT8, INT4, or specialized microformats when supported by target execution units. In D·A·M terms, training co-designs data and algorithm to maximize representational capacity; compression co-designs algorithm and machine, reshaping mathematical structure to satisfy the physical limits of the deployment hardware. **Model compression** is the systematic process of renegotiating the silicon contract for a constrained execution context, reducing parameter footprint, memory traffic, or arithmetic operations while preserving task accuracy.

The physical disparity across deployment environments spans six orders of magnitude: a 175-billion parameter frontier model consumes over 350 GB in FP16 representation alone, whereas an embedded microcontroller provides only 512 KB of on-chip SRAM. Bridging this gap requires an engineering framework with predictable physical trade-offs. Every compression technique removes specific structure from the model—unnecessary weights, surplus numerical bits, or redundant operator subgraphs. Designing a production deployment requires identifying which resource binds first on the target platform and analyzing how successive approximations compound across the memory hierarchy.

Compression operates along three complementary dimensions. Structural optimization removes redundancy from the model graph itself: **pruning** eliminates low-impact parameters, knowledge

175B FP16 350 GB

Phone RAM 8 GB

MCU RAM 512 KB

log scale

Frontier weights dwarf phone and microcontroller memory; compression bridges the gap.

distillation transfers behavior into a smaller architecture, and neural architecture search discovers designs tailored to a specified objective. Precision optimization reduces the bit width of weights and activations; for example, FP32-to-INT8 conversion cuts the raw bytes per represented value by four. Supported low-precision matrix units can also accelerate arithmetic. Hardware-level optimization maps the transformed graph to the target processor through operator fusion and hardware-supported sparsity. These dimensions form an optimization stack: structural changes alter which operations exist, precision changes bytes per value, and hardware-level mapping determines whether those theoretical savings become supported execution paths. In convolutional networks, fusing batch normalization into preceding weights eliminates separate memory round-trips and arithmetic entirely during inference. Benefits compound only when transformations relieve distinct binding bottlenecks and the runtime supports the resulting graph. Section 11.4.3 explains the accelerator mechanisms behind low-precision execution paths.

Concrete benchmark workloads ground these trade-offs across distinct operational regimes: ResNet-50 and MobileNetV2 (the lighthouse models from section 6.1.1) for vision tasks, transformer-based language models for autoregressive sequences, DLRM for recommendation memory capacity (Naumov et al. 2019), and depthwise-separable convolutional neural networks (DS-CNN) for TinyML keyword spotting (Y. Zhang et al. 2017). Each workload stresses a different term of the iron law, from parameter-dominated embedding tables to compute-heavy matrix multiplications. Evaluating compression across these reference workloads reveals how reductions in parameters and precision translate into physical latency and energy savings under identical runtime conditions.

Definition 10.1: Model compression

Model compression is a family of techniques that reduce a trained model's computational cost and memory footprint by eliminating redundant parameters (pruning), reducing numerical precision (quantization), or transferring learned behavior into a smaller architecture (distillation), while preserving as much predictive accuracy as possible.

1. **Significance:** Compression directly reduces the iron law's data-movement and compute terms. INT8 quantization of a 175-billion-parameter LLM cuts weight memory from 350 GB (FP16) to 175 GB, a $2\times$ reduction in D_{vol} , while dedicated low-precision matrix units can increase compute throughput when kernels and layouts use the supported INT8 path. Unstructured pruning to 50 percent sparsity halves the nonzero count, but executed operations fall only when the runtime can skip those zeros through a supported sparse format or kernel.
2. **Distinction:** Unlike post-training compression methods such as pruning and quantization, neural architecture search discovers efficient architectures from scratch by exploring a design space. Here, NAS is treated as a related structural optimization technique: it changes the representation before training rather than compressing a finished model post hoc.
3. **Common pitfall:** Compression techniques do not compose without interference. Pruning changes weight distributions and operation patterns; quantization adds calibration and kernel constraints. Their combination can lose accuracy or fail to accelerate without joint validation.

These three optimization dimensions form a descending hierarchy of abstraction, progressing from algorithmic formulation down to physical silicon. Optimization first determines what operations the model performs (representation), then how many bits encode each operand (numerics), and finally how execution schedules onto accelerator memory structures and functional units (implementation). Figure 10.1 illustrates this progression from software representation toward hardware execution.

The top layer, efficient model representation, eliminates structural redundancy within the computational graph. Techniques such as pruning, knowledge distillation, and neural architecture search (NAS)¹ modify parameter counts and operator topologies directly. Pruning removes weights or entire channels whose contribution to model accuracy is negligible. Knowledge distillation trains a compact student model to match the output distributions or intermediate representations of an

¹ | **Neural architecture search (NAS):** Zoph and Le (2016) at Google Brain used reinforcement learning to learn the architecture itself at a cost of 22,400 GPU-days (800 GPUs for 28 days), equivalent to 537,600 GPU-hours. Weight-sharing approaches such as Efficient Neural Architecture Search (ENAS) later reduced search cost by roughly $1,000\times$ by sharing parameters across candidate architectures (Pham et al. 2018). Hardware-aware NAS and scaling methods then made the search output practical for deployable architecture families such as EfficientNet and MobileNetV3 (Tan and Le 2019; Howard et al. 2019).

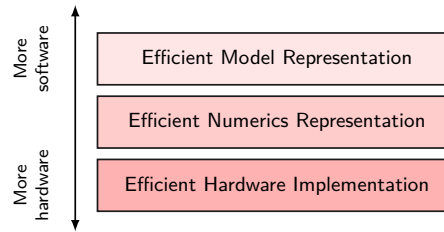


Figure 10.1: Optimization Stack: Model optimization progresses from software-level representation (pruning, distillation, NAS) down through numerical precision (quantization) to hardware execution (kernel fusion, sparsity engines). Lowering abstraction moves optimizations closer to physical silicon, where representation decisions dictate achievable execution efficiency.

overparameterized teacher. Hardware-aware NAS automates this exploration, evaluating candidate layer topologies, expansion ratios, and kernel dimensions to discover Pareto-optimal graphs under explicit latency or memory bounds.

The middle layer, efficient numerics representation, optimizes the bit-width used to encode each tensor value. **Quantization** maps continuous high-precision floating-point values to low-bit representations—such as 8-bit or 4-bit integers—shrinking weight memory footprint and DRAM bus traffic (D_{vol}) by factors of two to four. Beyond memory reduction, integer arithmetic relieves compute bottlenecks: low-precision matrix units execute integer multiply-accumulate operations with a fraction of the silicon area and energy required by 32-bit floating-point ALUs, enabling higher peak throughput within the same thermal envelope.

The bottom layer, efficient hardware implementation, aligns the remaining operations with the physical accelerator architecture. A reduced operation count does not guarantee faster execution if data movement stalls the compute cores or uncoalesced memory accesses serialize DRAM channels. Operator fusion merges adjacent layers—such as folding a convolution, bias addition, and activation function into a single kernel—keeping intermediate activations resident in fast on-chip SRAM or registers rather than round-tripping through high-latency off-chip DRAM. Similarly, hardware-supported sparsity engines, such as structured 2:4 sparse Tensor Cores, deliver physical speedup only when nonzeros conform to the alignment and fetch constraints of the hardware execution pipeline.

These three layers are tightly coupled across the compilation and execution boundary. Pruning alters weight distributions and activation profiles, changing the dynamic ranges required for subsequent quantization calibration. Quantization reduces data-movement volume across the memory bus, shifting a previously memory-bound workload toward compute-bound execution where kernel fusion and tile scheduling dominate. Achieving compounding performance gains requires co-designing transformations across all three layers rather than applying them in isolation, a composition strategy formalized in section 10.6.2.

Which layer of the optimization stack yields the highest return depends entirely on the physical machine hosting the workload. A data center server equipped with multiple high-bandwidth memory accelerators tolerates multi-gigabyte models but demands high token throughput under concurrency. A smartphone or an embedded microcontroller operates under severe physical ceilings on memory capacity, bus bandwidth, and milliwatt thermal dissipation. Establishing the hardware envelopes across these physical tiers—and identifying which resource binds first in each environment—is the necessary starting point for any compression strategy.

10.2 Deployment Context

A data center GPU with 80 GB of high-bandwidth memory (HBM) faces fundamentally different binding constraints than a smartphone with shared RAM or a microcontroller with only a few hundred kilobytes of SRAM. Physical deployment targets dictate which resource binds first (table 10.1).

Table 10.1: Deployment Constraints: Each deployment context imposes different optimization priorities.

Context	Memory	Latency	Power	Primary Goal
Cloud	tens of GB	100–500 ms	Flexible	Throughput, cost
Mobile/Edge	hundreds of MB to GB	5–100 ms	W-scale	Size, latency
TinyML	KB–MB	1–10 ms	mW	Size, energy

10.2.1 Deployment scenarios

Cloud inference centers on throughput (requests per second per dollar), where supported integer quantization paths increase serving density and operator fusion reduces per-request kernel launch overhead (Choudhary et al. 2020; Dean et al. 2018). Mobile and edge deployments must fit within tight physical memory bounds while meeting deterministic real-time deadlines. A camera pipeline processing 30 frames per second has a 33 ms per-frame latency budget; exceeding that budget drops frames and violates real-time guarantees.

TinyML makes compression an absolute deployment requirement. A microcontroller with only a few hundred kilobytes of SRAM *cannot* run a 100 MB model regardless of predictive accuracy (Banbury et al. 2020). Without virtual memory or paging hardware, model weights and peak activation buffers must reside entirely within physical SRAM; if the working set exceeds physical capacity, the program fails immediately with an out-of-memory fault.

Physical memory capacity bounds have constrained deep learning since its inception. AlexNet, the model that won the 2012 ImageNet challenge, encountered a memory wall during training, and its architecture records the resulting two-GPU design choice.

Example 10.1: AlexNet’s two-GPU split (2012)

Context: In 2012, Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton aimed to train a 60-million-parameter vision model on ImageNet (Krizhevsky et al. 2012).

Mechanism: An NVIDIA GTX 580 provided only 3 GB of GDDR5 memory—insufficient to store the 240 MB model weights alongside FP32 activations (M_{act}) and optimizer states (M_{state}) for single-GPU training.

Impact: The model could not be trained on a single device at all, so the memory ceiling propagated upward into the network design itself rather than remaining a hardware procurement problem.

Fix: They partitioned the model across two 3 GB GTX 580 GPUs and limited cross-GPU connections so that the working set would fit within each device’s memory bound ($M_{\text{peak}} \leq 3 \text{ GB}$). The resulting two-tower architecture achieved a winning top-5 error rate of 15.3 percent, a 10.8 percentage point margin.

Systems lesson: Memory capacity bounds have constrained deep learning since its inception. Model deployment strategies (pruning, quantization) address the same physical limits that forced model splitting in AlexNet.

The same memory constraints that shaped AlexNet’s training architecture dictate feasibility when deploying models to commodity mobile hardware.

Example 10.2: An illustrative MobileNet win

Scenario: Deploying a real-time background blur video segmentation model on mid-tier Android smartphones at 30 FPS.

Diagnosis: An uncompressed FP32 MobileNetV3 model achieves only 8 FPS, exceeding device latency and thermal power limits. Quantizing weights to INT8 enables mobile NPU/DSP integer matrix execution.

Systems lesson: Here, INT8 cuts raw parameter payload by $4\times$, and a supported integer path raises inference from 8 FPS to 35 FPS. Speed and power still require device measurement.

10.2.2 Balancing trade-offs

Table 10.2 quantifies this deployment gap across the lighthouse models from section 6.1.1. Even MobileNetV2 at INT8 precision exceeds TinyML device memory by about $6.7\times$, demonstrating how physical memory envelopes constrain model viability:

Table 10.2: The Deployment Gap: Model weight requirements compared against typical device capacities. DLRM represents recommendation-model embedding pressure (Naumov et al. 2019); DS-CNN represents the TinyML keyword-spotting case (Y. Zhang et al. 2017). Even MobileNetV2 quantized to INT8 exceeds the ~512 KB TinyML envelope by $6.7\times$, while the purpose-built DS-CNN keyword spotter fits after INT8 quantization. In this weight-only accounting, runtime weight memory equals artifact weight storage; activation, workspace, packaging, and metadata overheads are excluded. Numbers in parentheses show how many times the model exceeds device memory.

Model	Runtime Weight Memory	Artifact Weight Storage	Cloud (~107 GB)	Mobile (8 GB)	TinyML (~512 KB)
DLRM	100 GB	100 GB	ok	no (11.6 $\times$)	no (190734.9 $\times$)
GPT-2 XL	6 GB	6 GB	ok	ok	no (11444.1 $\times$)
ResNet-50	102.4 MB	102.4 MB	ok	ok	no (195.3 $\times$)
MobileNetV2	14 MB	14 MB	ok	ok	no (26.7 $\times$)
MobileNetV2 (INT8)	3.5 MB	3.5 MB	ok	ok	no (6.7 $\times$)
DS-CNN (KWS, INT8)	200 KB	200 KB	ok	ok	ok

This mismatch makes the **accuracy-efficiency trade-off** unavoidable. Increasing model capacity generally enhances predictive accuracy but demands larger weight payloads, expanded activation working sets, and higher memory bandwidth during inference. On hardware with strict memory or power limits, these demands force a choice along the Pareto frontier.

Systems Perspective 10.1: The compression-accuracy trade-off curve

The frontier is not uniform: structure-preserving techniques sit near the top, exchanging little or no accuracy for real speedups, while aggressive structural surgery sits at the bottom, where each additional gain extracts a steepening accuracy penalty.

The engineering decision is where to stop. Compression should halt at the “knee” of the curve, the point where the marginal loss in accuracy first exceeds the marginal gain in efficiency. Past that knee, the model degrades faster than it accelerates.

Table 10.3 summarizes the primary compression techniques across these regions, mapping their systems gains against their machine learning costs.

Table 10.3: The Optimization Trade-Offs: Region 1 = Free Lunch, Region 2 = Efficient Trade, Region 3 = Danger Zone. Batch size affects training dynamics rather than model quality directly. These are representative engineering ranges synthesized from published work on inference runtimes, quantization, pruning, low-bit LLM quantization, and distillation (NVIDIA 2024c; Jacob et al. 2018; Han et al. 2015; Lin et al. 2024; Hinton et al. 2015); actual results vary with architecture, task, calibration data, and implementation quality.

Technique	Systems Gain	ML Cost	Typical Impact	Region
Operator Fusion	10–30% latency reduction	None	No accuracy loss	1
FP32 $\rightarrow$ BF16	$2\times$ memory, $\sim 2\times$ throughput	Minimal	$< 0.1\%$ accuracy drop	1
FP16 $\rightarrow$ INT8	$2\times$ memory, $2\text{--}4\times$ throughput	Quantization error	$0.5\text{--}1\%$ accuracy drop	2
50% Pruning	$\sim 2\times$ smaller model	Capacity loss	$0.5\text{--}1\%$ accuracy drop	2

Technique	Systems Gain	ML Cost	Typical Impact	Region
Knowledge Distillation	2–10× smaller student	Capability ceiling	1–3% accuracy drop	2
4-bit Quantization	4× memory reduction	Significant error	2–5% accuracy drop	2–3
90% Pruning	~10× smaller model	Severe capacity loss	5–15% accuracy drop	3
↑ Batch Size (8×)	Higher throughput, better GPU util	Generalization gap	Requires LR scaling	—

Techniques that preserve model structure—such as kernel fusion and modest precision reduction (FP32 to BF16)—yield latency and memory gains with negligible accuracy loss. In contrast, techniques that alter network structure, such as pruning and distillation, yield larger reductions in memory and arithmetic volume but risk significant quality degradation without careful retraining. Optimization begins by identifying the binding hardware constraint: memory capacity on mobile devices, frame latency in interactive pipelines, or energy per inference on battery-powered sensors. Compression strategies then target that bottleneck directly: structural methods reduce parameter and operation counts, quantization shrinks memory traffic and increases arithmetic intensity, and compiler-level fusion eliminates intermediate memory round-trips.

Checkpoint 10.1: The efficiency frontier

Optimization is about trading one resource for another.

Trade-offs

- ☐ **Accuracy vs. cost:** compare a structure-preserving optimization and a structure-altering one by naming which resource each saves and which quality risk it introduces.
- ☐ **The “free lunch”:** can you point to where on the compression-accuracy frontier a technique buys efficiency at no accuracy cost, and explain why that region exists?
- ☐ **The knee:** explain why aggressive optimization should stop where the marginal accuracy loss exceeds the marginal efficiency gain.

10.3 Structural Optimization

Modern neural networks often carry more parameters than deployment requires.² Unused capacity still consumes memory, computation, and energy. Structural optimization addresses the first dimension of the optimization framework, **efficient model representation**, by modifying *what* the model computes. Although additional capacity can aid optimization during training, parameters that do not improve the deployed model still impose deployment cost.

Every technique in this chapter follows the same engineering heuristic: the **conservation of complexity**. Compression rarely destroys cost outright. It relocates cost between the Data, Algorithm, and Machine axes. Pruning may reduce parameters while asking the runtime to exploit sparse structure; distillation may reduce inference cost while adding a teacher-student training phase; quantization may reduce data movement while spending numerical precision. The engineer’s task is to move complexity to where the cost is lowest given deployment constraints.

The systems challenge is excising surplus capacity without compromising task-critical representation. Operating along this Pareto frontier³ requires deciding where complexity should reside: in training-time discovery, runtime sparse indexing, or student-model capacity.

These techniques address the challenge through complementary mechanisms. Pruning eliminates redundant parameters from an existing network; knowledge distillation transfers learned representations into an independently designed compact topology; neural architecture search automates the structural discovery process from the ground up (Hutter et al. 2019). In production pipelines, these techniques frequently compose: a NAS-discovered architecture, supervised by a distilled teacher, is pruned prior to final quantization. Pruning comes first in this analysis because it exposes the physical bottleneck most directly: reducing parameter count accelerates execution only when the resulting structure maps to supported hardware kernels.

² | **Overparameterization:** C. Zhang et al. (2017) demonstrated that networks large enough to fit ImageNet can also memorize completely random labels, showing that training capacity can exceed the structure needed for natural labels. Pruning studies then show the deployment consequence: trained models often contain many parameters that can be removed or sparsified with modest task loss when pruning and fine-tuning are done carefully (Gale et al. 2019; Blalock et al. 2020). The redundancy is not a universal 10× constant; it depends on architecture, dataset, sparsity pattern, and runtime support.

³ | **Pareto frontier:** Named after Italian economist Vilfredo Pareto (1848–1923), who observed that 80 percent of Italy’s land was owned by 20 percent of the population. In multi-objective optimization, the Pareto frontier is the set of solutions where improving one objective (for example, speed) necessarily sacrifices another (for example, accuracy). EfficientNet traces this frontier concretely: B0 (77.1 percent accuracy, 390 million FLOPs) to B7 (84.4 percent, 37 billion FLOPs)—a 95× compute increase for 7.3 percentage points of accuracy, quantifying how steep the trade-off becomes at the frontier’s edge (Tan and Le 2019).

10.3.1 Pruning

Consider a MobileNet trained for image classification on a wearable health monitor. The trained model occupies 14 MB, but the target microcontroller offers only 2 MB of flash memory. Retraining a smaller architecture from scratch would require weeks of data collection and validation—time the product schedule does not allow. Suppose profiling shows that about 85.7 percent of the model’s weights are near zero and contribute little on the validation set. Removing those weights and fine-tuning the remainder for a few epochs produces a model that fits in 2 MB with an acceptable accuracy loss. The numbers anchor the engineering trade-off rather than reporting a universal MobileNet benchmark.

Pruning⁴ directly addresses memory efficiency constraints by eliminating parameters or structures that contribute little to the deployed objective. Many trained networks contain removable capacity, but the safe fraction depends on the architecture, task, pruning pattern, and recovery procedure. The central questions are *what* to prune (individual weights vs. entire structures), *how* to estimate what is expendable (magnitude, gradients, or activations), and *when* to prune (after training, during training, or even at initialization). While section 11.4.5 analyzes the microarchitectural execution of sparse operands, the foundational systems rule is invariant: zero-valued weights conserve physical resources only when the underlying hardware and runtime execution path can skip loading or computing them.



Definition 10.2: Pruning

Pruning is a model-compression technique that sparsifies the parameter space by removing weights that contribute minimal information to the loss landscape.

1. **Significance:** It can convert dense matrices into sparse structures, reducing memory footprint and total data volume (D_{vol}) when the sparse representation, metadata, and recovery procedure preserve acceptable quality.
2. **Distinction:** Unlike quantization, which reduces the precision of every weight, pruning reduces the count of weights by identifying and eliminating redundancy.
3. **Common pitfall:** A frequent misconception is that pruning “automatically” speeds up execution. In reality, without specialized sparse execution support, the resulting sparse matrices may actually run slower than dense ones due to irregular memory access patterns; a higher R_{peak} alone does not make an irregular sparse layout efficient.

The formal pruning objective seeks a sparse parameter tensor $\hat{\mathbf{W}}$ that minimizes empirical loss subject to a parameter budget k :

$$\min_{\hat{\mathbf{W}}} \mathcal{L}(\hat{\mathbf{W}}) \quad \text{subject to} \quad \|\hat{\mathbf{W}}\|_0 \leq k$$

where $\|\hat{\mathbf{W}}\|_0$ is the **L0-norm** (the count of nonzero parameters). Solving this cardinality-constrained optimization is combinatorial, so practical methods use heuristics⁵ such as **magnitude-based pruning**. Listing 10.1 demonstrates this approach, removing weights with small absolute values to transform a dense weight matrix into the sparse representation visualized in figure 10.2.

⁴ | **Optimal Brain Damage:** Introduced by LeCun, Denker, et al. (1989), the method achieved 4× parameter reduction—and proportional memory savings—in a handwriting recognizer by using a diagonal approximation to second-derivative (Hessian) information to estimate the loss increase from removing each weight. A full Hessian has $\mathcal{O}(n^2)$ entries for n parameters, but Optimal Brain Damage avoids storing that full matrix by approximating its diagonal. Modern-scale pruning still favors cheaper criteria such as magnitude because even diagonal curvature estimation adds training cost.

⁵ | **Heuristic:** From Greek *heuriskein* (to discover), the same root as Archimedes’ “eureka.” In pruning, the dominant heuristic—larger magnitude means more important—works well empirically but creates a systems trap: magnitude-based pruning applied globally can remove most parameters from overparameterized layers while leaving critical bottleneck layers largely intact, giving the appearance of aggressive compression while preserving much of the compute and memory cost in the layers that matter (Blalock et al. 2020). This is why iterative prune-retrain cycles with per-layer budgets are often safer than naive global magnitude pruning: each cycle lets the network redistribute importance before the next cut.

Listing 10.1: Magnitude-Based Pruning: Removes weights below a threshold to create sparse matrices, reducing the number of nonzero parameters from 9 to 4 ($k = 4$).

```
import torch

# Original dense weight matrix
weights = torch.tensor(
    [[0.8, 0.1, -0.7], [0.05, -0.9, 0.03], [-0.6, 0.02, 0.4]]
)

# Simple magnitude-based pruning: keep only the 4 largest weights
threshold = 0.5
mask = torch.abs(weights) >= threshold
pruned_weights = weights * mask

print("Original:", weights)
print("Pruned (4 nonzeros):", pruned_weights)
```

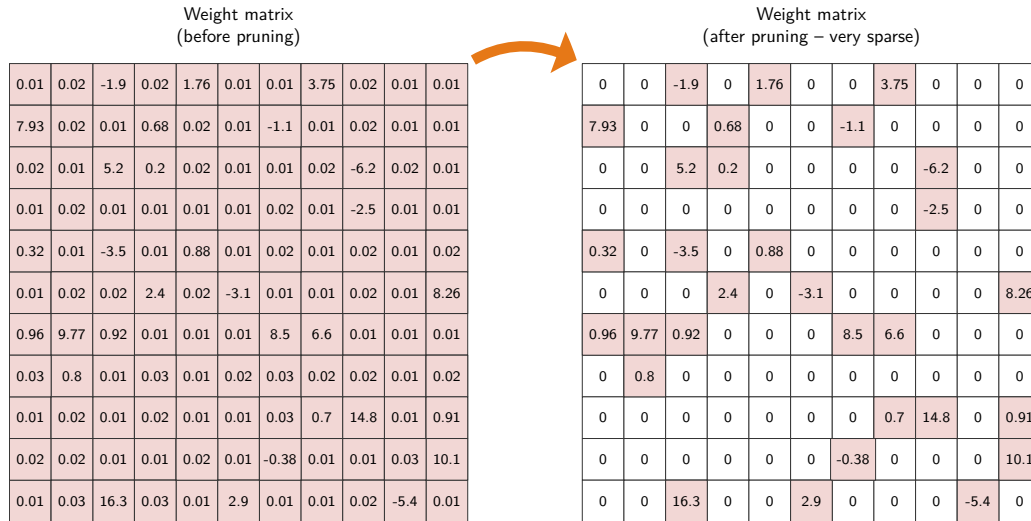


Figure 10.2: Sparse Matrix Transformation: Magnitude-based pruning replaces weights below a numerical threshold with exact zeros (white cells) while preserving salient high-magnitude connections (colored cells). The resulting sparsity cuts parameter count, but realizing hardware speedups requires translating these unstructured zero entries into sparse storage formats or specialized execution kernels.

The resulting sparse matrix retains only the high-magnitude values (colored cells) while near-zero weights become exact zeros. In models amenable to pruning, much of the representational capacity remains intact after removing low-magnitude weights, establishing magnitude thresholding as an effective baseline heuristic.

To make pruning computationally tractable during gradient-based training, practical methods often relax the combinatorial ℓ_0 constraint with continuous penalties such as ℓ_1 -regularization ($\lambda_{L1}\|\mathbf{W}\|_1$), where λ_{L1} scales the sparsity penalty applied to weight tensor $\mathbf{W}$. Penalizing weight magnitudes drives non-essential parameters toward zero, facilitating downstream thresholding. Practitioners typically couple this with **iterative pruning**, removing parameters in successive increments interleaved with fine-tuning cycles to recover lost accuracy (Gale et al. 2019; Blalock et al. 2020).

10.3.1.1 Target structures

The choice of what to prune depends on the deployment target's hardware constraints and which resource is the binding bottleneck. When memory capacity is primary, **neuron pruning** can provide direct relief in models whose fully connected layers dominate parameter storage: removing entire

neurons and their associated weights and biases reduces layer width and parameter count. Profiling must first establish that these layers, rather than embeddings, activations, or another component, are the actual bottleneck.

When convolutional inference latency on commodity accelerators is the bottleneck, **channel pruning** (also called filter pruning) is a strong candidate. Eliminating entire channels or filters reduces feature-map depth and the multiply-accumulate count in subsequent layers. The resulting subnetwork remains dense and regular, so it can map to conventional GPU and Tensor Processing Unit (TPU) kernels without an unstructured sparse format. Realized latency still depends on the resulting dimensions, kernels, and memory traffic.

When a model contains removable stages, **layer pruning** removes entire layers from the network. Each removal eliminates all computation in that stage, but it also reduces representational depth and may change tensor shapes, residual paths, or downstream interfaces. The remaining layers must absorb the lost function, typically through fine-tuning or retraining. The nominal operation saving therefore does not guarantee proportional latency: graph rewrites, kernel shapes, and memory traffic still govern execution. Layer pruning demands careful task validation and end-to-end profiling. The side-by-side comparison in figure 10.3 shows why channel and layer pruning have different implementation costs.

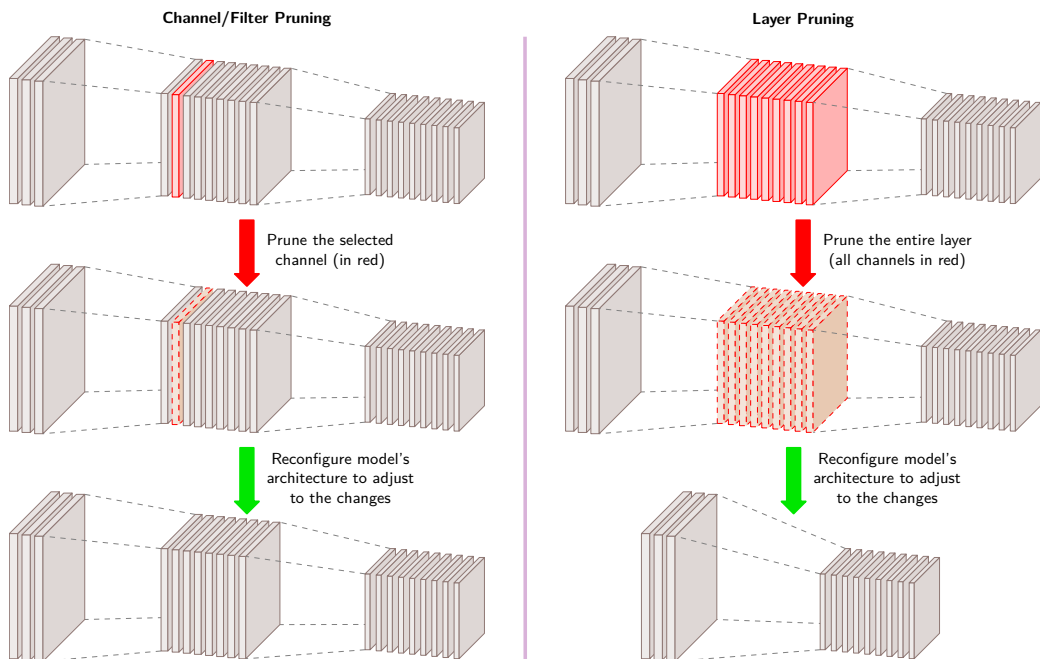


Figure 10.3: Channel vs. Layer Pruning: Structural pruning operates at different architectural granularities. Channel pruning (left) removes feature map slices within convolutional layers while preserving overall network topology and dense execution, whereas layer pruning (right) eliminates entire computational stages, requiring graph rewiring and risking greater capacity loss to achieve larger per-operation savings.

The structural mechanics in figure 10.3 illustrate why these granularities present different hardware trade-offs. Channel pruning contracts the inner dimensions of dense General Matrix Multiply (GEMM) operations: eliminating output channels in layer l reduces the input channel dimension in layer $l + 1$. While the tensors remain dense and avoid sparse pointer lookups, the pruned dimensions must remain aligned with hardware tile boundaries (typically multiples of 8, 16, or 32 elements for single instruction, multiple data (SIMD) vector units and Tensor Cores) to prevent uncoalesced memory accesses and idle compute lanes. Layer pruning circumvents dimensional alignment entirely by excising full computational stages. This eliminates kernel launch latency, intermediate activation memory traffic, and residual synchronizations, but requires rewiring residual skip connections and accepting a steeper drop in representational capacity.

10.3.1.2 Unstructured pruning

Unstructured pruning removes individual weights while preserving the overall network architecture. Some connections become redundant during training, contributing little to the final output. Pruning these weak connections reduces the nonzero count and can preserve most task quality after recovery.

Formalizing this process, let $\mathbf{W} \in \mathbb{R}^{m \times n}$ represent a weight matrix in a given layer. Pruning removes a subset of weights by applying a binary mask $\mathbf{M} \in \{0, 1\}^{m \times n}$, yielding a pruned weight matrix:

$$\hat{\mathbf{W}} = \mathbf{M} \odot \mathbf{W}$$

where $\odot$ represents the element-wise Hadamard product. The mask $\mathbf{M}$ is constructed based on a pruning criterion, typically weight magnitude. A common approach is magnitude-based pruning, which removes a fraction ρ_{sparse} of the lowest-magnitude weights by defining a threshold δ_{prune} such that:

$$M_{i,j} = \begin{cases} 1, & \text{if } |W_{i,j}| > \delta_{\text{prune}} \\ 0, & \text{otherwise} \end{cases}$$

where δ_{prune} is chosen to ensure that only the largest $(1 - \rho_{\text{sparse}})$ fraction of weights remain. This method assumes that larger-magnitude weights contribute more to the network's function, making them preferable for retention.

Unstructured pruning reduces the count of nonzero parameters, but translating that reduction into physical storage savings requires an indexed sparse encoding such as Compressed Sparse Row (CSR) or coordinate list (COO). Because these formats must store positional metadata—such as column indices and row pointers—alongside the nonzero values, metadata overhead offsets savings at low sparsity. For example, storing a 16-bit floating-point weight with a 16-bit column index doubles the per-element byte footprint, requiring over 50 percent sparsity just to break even on artifact size (D_{vol}). Storing a dense tensor with zeroed entries saves zero memory.

Nor does unstructured sparsity translate directly to inference acceleration on modern hardware. Commodity accelerators achieve peak throughput (R_{peak}) through dense systolic arrays and wide SIMD execution units that require lockstep execution and coalesced memory transactions across adjacent memory addresses. Arbitrary zero distributions force Sparse Matrix-Dense Matrix Multiplication (SpMM) kernels to perform indirect memory addressing and pointer chasing. This memory irregularity causes warp divergence, uncoalesced global memory transactions, and severe cache line underutilization. Unless the network achieves extreme sparsity (often exceeding 80 to 90 percent) and executes on specialized sparse kernels, unstructured pruning yields a model that is smaller on disk but slower in runtime execution.

10.3.1.3 Structured pruning

Where unstructured pruning removes individual weights, **structured pruning** (H. Li et al. 2017) eliminates entire computational units: neurons, filters, channels, or layers. This approach produces smaller dense tensors that execute on standard accelerator kernels. It is therefore substantially easier to convert into latency savings than arbitrary unstructured sparsity, although tensor shapes, memory alignment, and task quality still require careful profiling.

Neurons, filters, and layers can differ substantially in their contribution to a model's predictions. Some units may carry redundant or low-impact information, but removal is safe only when validation shows that the remaining structure preserves required behavior. Identifying those structures remains the core challenge.

Hardware-aware pruning strategies, such as **N:M structured sparsity**,⁶ enforce specific patterns (for example, ensuring 2 out of every 4 weights are zero) to align with specialized accelerator capabilities. This chapter uses the 2:4 pattern as the compression example; section 11.4.5 later shows how sparse Tensor Cores exploit it.

Pruning methods trade parameter reduction against hardware execution regularity. Compare the three panels in figure 10.4, contrasting irregular weight removal on the left with structured neuron and channel removal in the center and right panels.

Structured pruning eliminates the sparse metadata and memory divergence penalties of unstructured sparsity by contracting the underlying matrices into smaller, contiguous dense tensors (figure 10.4). In fully connected layers (center), pruning neurons eliminates corresponding rows

⁶ | **N:M structured sparsity:** Introduced commercially with NVIDIA's A100 GPU (2020), the 2:4 pattern was chosen because it halves multiply-accumulate operations while keeping position metadata small enough for the sparse Tensor Core path (NVIDIA Corporation 2020a; Choquette et al. 2021). This fixed ratio is a hardware constraint, not a mathematical optimum: the A100 Sparse Tensor Core path accelerates 2:4 sparse operands, yielding up to 2× math-throughput speedup over dense execution when kernels and layouts satisfy the constraint. Other ratios are not accelerated by this specific hardware path, illustrating how silicon design constrains which sparsity patterns translate to actual speedup.

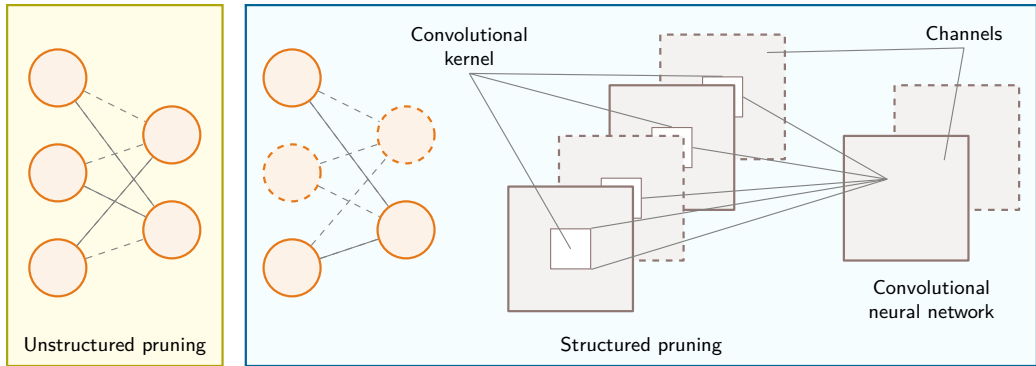


Figure 10.4: Unstructured vs. Structured Pruning: The regularity-compression trade-off across pruning granularities. Unstructured pruning (left) removes arbitrary individual connections and requires a sparse representation and matching kernel to change execution. Structured pruning of neurons (middle) or convolutional channels (right) can produce smaller dense tensors that remain compatible with standard kernels, although realized latency depends on the resulting shapes and runtime. Source: (Qi et al. 2021).

and columns in adjacent weight matrices. In convolutional layers (right), pruning channels removes complete 3D filter slices. Because the resulting subnetwork consists entirely of dense operations, it executes directly on standard Basic Linear Algebra Subprograms (BLAS) and convolution engines without specialized sparse libraries. The primary systems requirement is maintaining dimensional alignment: modern accelerator kernels achieve peak memory throughput when tensor dimensions remain multiples of hardware cache lines and warp tile sizes (such as 32, 64, or 128 elements).

A common approach to structured pruning ranks entire neurons or filters by the magnitude of their associated weights. A low group norm is an inexpensive proxy for low importance, not proof that the unit is dispensable. The score can use an ℓ_1 -norm or ℓ_2 -norm over the weights associated with each unit; groups below a selected threshold become pruning candidates. Layer-wise ranking matters because raw scales can differ across layers. The method needs no representative data pass, but computing scores, choosing a threshold, materializing the smaller graph, and validating or fine-tuning the result still incur engineering and compute cost.

Another strategy is **activation-based pruning**, which evaluates neuron or filter activations over a dataset. Units that remain weak across representative inputs become candidates for removal, subject to ablation or fine-tuning checks. This method captures input-dependent behavior rather than relying only on static weights. Its ranking can still miss rare but important inputs, depend on activation scaling, or change after upstream units are removed. Activation-based pruning therefore requires representative profiling data and validation after the selected structures are removed.

Gradient-based pruning uses loss derivatives to estimate how removing a neuron or filter would change the objective. A small first-order score suggests a candidate at the current checkpoint and on the sampled data; it does not establish permanent irrelevance or capture every interaction among units. Gradient-based criteria can combine a unit's value with its gradient to rank the expected local loss change. Mathematically, this criterion approximates the change in loss $\Delta\mathcal{L}$ when zeroing parameter w_i via a first-order Taylor expansion: $\Delta\mathcal{L} \approx |w_i \cdot \frac{\partial \mathcal{L}}{\partial w_i}|$. Weights that are either near zero or have negligible gradients exert minimal influence on the objective, making them prime candidates for excision. Unlike weight-only magnitude criteria, they require backward computation and are usually integrated into training or fine-tuning. The pruned result still needs task-level validation because the score is a local approximation.

These criteria trade measurement cost against fidelity to model behavior. Magnitude scores are inexpensive and provide a useful baseline, but they ignore the input distribution. Activation scores incorporate representative inputs and expose units that are quiet on that workload, while gradient-based scores also incorporate the local loss surface at additional training cost. None dominates across models, and none establishes that removal is harmless. A sound comparison prunes to the same structural target, applies the same recovery budget, and measures task quality after removal. It then profiles the exported graph, because a better importance score can preserve quality without

producing shapes that the target runtime executes efficiently. The choice therefore depends on available data and compute, the structure being removed, and the deployment’s quality margin.

10.3.1.4 Dynamic pruning

Traditional pruning methods, whether unstructured or structured, produce a **static pruning** mask or subnetwork that remains fixed during deployment, even if the mask was learned gradually during training. **Dynamic pruning** instead adapts which computation is active from input data or training dynamics, so the executed subnetwork can change over time.

Dynamic pruning can use runtime sparsity techniques in which the model selects parameters or structures from input characteristics. Activation-conditioned pruning, for example, deactivates neurons or channels for particular inputs (Hu et al. 2023). The resulting input-dependent sparsity reduces executed work only when the controller and runtime can skip that work more cheaply than a dense path would execute it.

For instance, consider a convolutional neural network processing images with varying complexity. During inference on a simple image containing mostly uniform regions, some convolutional filters may produce negligible activations. A dynamic policy can identify candidate filters and temporarily exclude them from computation. That exclusion reduces nominal work, but the controller, irregular execution, and any quality change remain part of the result. The method becomes useful in a latency-sensitive application only when variation among inputs is large enough to repay those costs. As chapter 12 details, evaluating such efficiency gains requires measuring both latency and accuracy on the same target workload across the full input distribution.

Another class of dynamic pruning operates during training, gradually introducing and adjusting sparsity throughout the optimization process. Gradual magnitude pruning starts with a dense network and progressively increases the fraction of pruned parameters while fine-tuning the surviving weights. Dynamic sparse training variants additionally allow the network to recover from pruning-induced capacity loss by regrowing connections that prove important later in training.

Dynamic pruning can allocate different amounts of work to different inputs and can reactivate capacity that a static deployment mask would remove permanently. Those properties create an opportunity, not a guaranteed efficiency or accuracy gain. A controller sits on the critical path, input-dependent routing fragments batch execution across SIMD vector lanes, and the enlarged behavior space requires more validation. Training may also need routing losses or regularization so that the policy actually uses its cheaper paths. Production deployments must monitor route frequency, latency tails, and quality by route; chapter 14 later develops those monitoring and rollback practices. Dynamic pruning is therefore a candidate when input complexity varies enough to repay the controller and batching overhead, not a default substitute for static pruning.

10.3.1.5 Pruning trade-offs

The three pruning approaches occupy different positions on the regularity-versus-granularity trade-off. Unstructured pruning can remove individual weights and therefore express fine-grained sparsity, but accelerators need matching sparse formats and kernels to skip those zeros. Structured pruning removes channels, filters, or layers and can produce smaller dense operations that conventional hardware already supports. Dynamic pruning makes the executed structure input-dependent, adding control and batching overhead in exchange for conditional allocation. Table 10.4 summarizes these deployment distinctions.

These categories describe deployed execution rather than mutually exclusive training algorithms. A structured subnetwork may be learned with a dynamic schedule and then frozen for serving; classify the artifact by what its runtime actually executes.

Table 10.4: Pruning Strategies: Unstructured pruning removes individual weights but needs sparse storage and matching kernels for deployment gains; structured pruning removes whole components that map more directly to dense hardware, while dynamic pruning adds runtime adaptation and overhead.

Aspect	Unstructured Pruning	Structured Pruning	Dynamic Pruning
What is removed?	Individual weights in the model	Entire neurons, channels, filters, or layers	Adjusts pruning based on runtime conditions

Aspect	Unstructured Pruning	Structured Pruning	Dynamic Pruning
Model structure	Sparse weight matrices; original architecture remains unchanged	Model architecture is modified; pruned layers are fully removed	Structure adapts dynamically
Impact on memory	Reduces the nonzero count; storage falls only with an appropriate sparse representation	Reduces model storage by removing entire components	Varies based on real-time pruning
Impact on computation	Dense work remains unless a supported sparse path skips zeros	Reduces nominal FLOPs; latency depends on resulting shapes and kernels	Varies with routing policy and controller overhead
Hardware compatibility	Requires sparse formats and matching execution support	Produces dense subnetworks, but awkward shapes can still underutilize hardware	Requires adaptive inference engines
Fine-tuning required?	Often used to recover accuracy after pruning	Often used after structural modification	Depends on how the routing policy and subnetworks are trained
Use cases	Memory-efficient model compression for cloud deployment	Real-time inference optimization, mobile/edge AI, and efficient training	Adaptive AI applications, real-time systems

10.3.1.6 Pruning strategies

Execution scheduling determines whether a network absorbs capacity loss or degrades irrevocably. Two primary scheduling strategies govern weight removal: iterative pruning and one-shot pruning.

Iterative pruning Iterative pruning mitigates catastrophic accuracy drops by interleaving structural parameter removal with retraining cycles (figure 10.5).

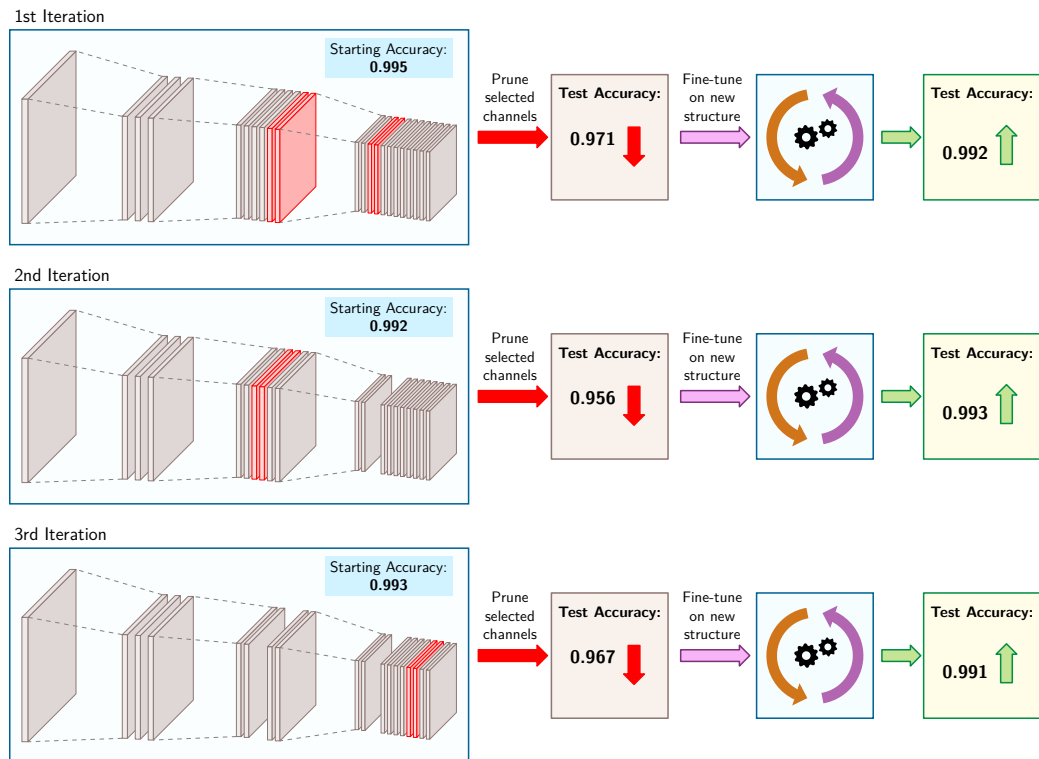


Figure 10.5: Iterative Pruning Performance: Three rows depict successive prune-then-fine-tune cycles, each removing two of the original twenty-two channels. Accuracy drops from 0.995 to 0.971 after the first prune, recovers to 0.992 after fine-tuning, and settles at 0.991 after all three cycles, a 0.4 percent loss with 27 percent fewer channels.

The three-row workflow in figure 10.5 demonstrates this gradual adaptation on a convolutional network pruned by six channels. Rather than removing all six channels in a single pass, the pipeline removes two channels per cycle across three iterations, fine-tuning the remaining parameters after each cut. The initial cut drops accuracy from 0.995 to 0.971, but brief fine-tuning restores performance to 0.992. Two subsequent prune-and-tune cycles produce a final model at 0.991 accuracy—a negligible 0.4 percent degradation despite a 27 percent reduction in channel count. Staging structural surgery across multiple retraining intervals allows surviving weights to adjust their representations along the loss landscape.

One-shot pruning One-shot pruning removes multiple architectural components in a single step, followed by an extensive fine-tuning phase to recover model accuracy. This aggressive approach compresses the model quickly but risks greater accuracy degradation, as the network must adapt to significant structural changes simultaneously.

Applying one-shot pruning to the same network illustrates the hazard of aggressive structural truncation. Instead of staging removals, one-shot pruning cuts all six channels simultaneously (figure 10.6). Excising 27 percent of network capacity in a single step causes accuracy to drop from 0.995 to 0.914. Subsequent fine-tuning recovers only to 0.943—a 5 percent permanent degradation from baseline. Although iterative and one-shot pruning yield identical final tensor geometries, the sudden loss of representational capacity traps one-shot fine-tuning in an inferior local minimum.

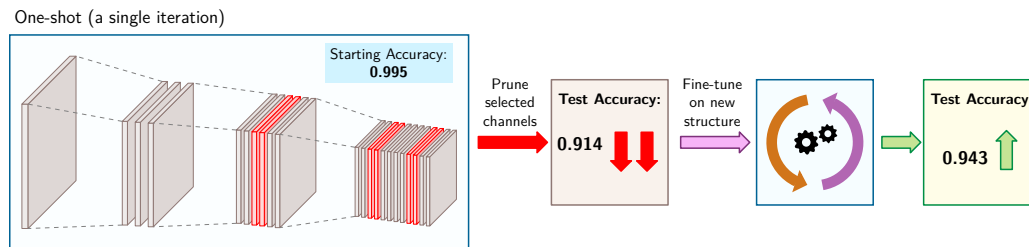


Figure 10.6: One-Shot Pruning Impact: All six channels (27 percent) are removed simultaneously, causing accuracy to drop from 0.995 to 0.914. Fine-tuning recovers only to 0.943, a 5 percent degradation from the original accuracy.

Three operational constraints govern the choice between scheduling strategies. Higher sparsity targets demand iterative cycles to prevent severe loss of representational capacity. Conversely, strict compute budgets during optimization or aggressive deployment deadlines favor one-shot pruning, accepting marginal accuracy degradation to avoid multiple costly retraining passes. Finally, target accelerator architectures dictate whether the resulting sparsity pattern justifies iterative investment.

10.3.1.7 Lottery ticket hypothesis

Standard pruning workflows assume a fully trained dense model from which weights are subsequently removed. Yet the relationship between network structure and optimization dynamics runs deeper: pruning can reveal inherently efficient, highly trainable subnetworks embedded within the initial random parameterization.

This perspective leads to the Lottery Ticket Hypothesis⁷ (LTH), which challenges conventional pruning workflows by proposing that within large neural networks, there exist small, well-initialized subnetworks (“winning tickets”) that can achieve comparable accuracy to the full model when trained in isolation. Rather than viewing pruning as a post-training compression step, LTH suggests it can serve as a discovery mechanism to identify these efficient subnetworks early in training (Rachwan et al. 2022).

The original LTH procedure identifies these subnetworks through iterative magnitude pruning and rewinding (figure 10.7). A network is trained to convergence, low-magnitude weights are pruned, and the surviving parameters are reset to their initial values θ_0 rather than re-randomized. Repeating the cycle can isolate a sparse subnetwork that, in the reported settings, trains to accuracy comparable with the original network (Frankle and Carbin 2019). Whether the same procedure succeeds depends on architecture, scale, optimizer, and training budget.

The Lottery Ticket Hypothesis suggests that compact, trainable subnetworks may exist inside an overparameterized initialization. That observation motivates direct sparse-training research, but

⁷ | **Lottery ticket hypothesis:** Named for the intuition that training a large network is like buying many lottery tickets—most lose, but a few “winning tickets” (sparse subnetworks with favorable initializations) can train to comparable accuracy on their own. Frankle and Carbin (2019) established the hypothesis on smaller vision and fully connected networks; later work surveyed and extended the idea to larger settings (Rachwan et al. 2022). The systems implication is that some of the memory and compute spent training dense networks may be discoverable overhead, but the practical payoff depends on whether the winning subnetwork can be found before paying most of the original training cost.

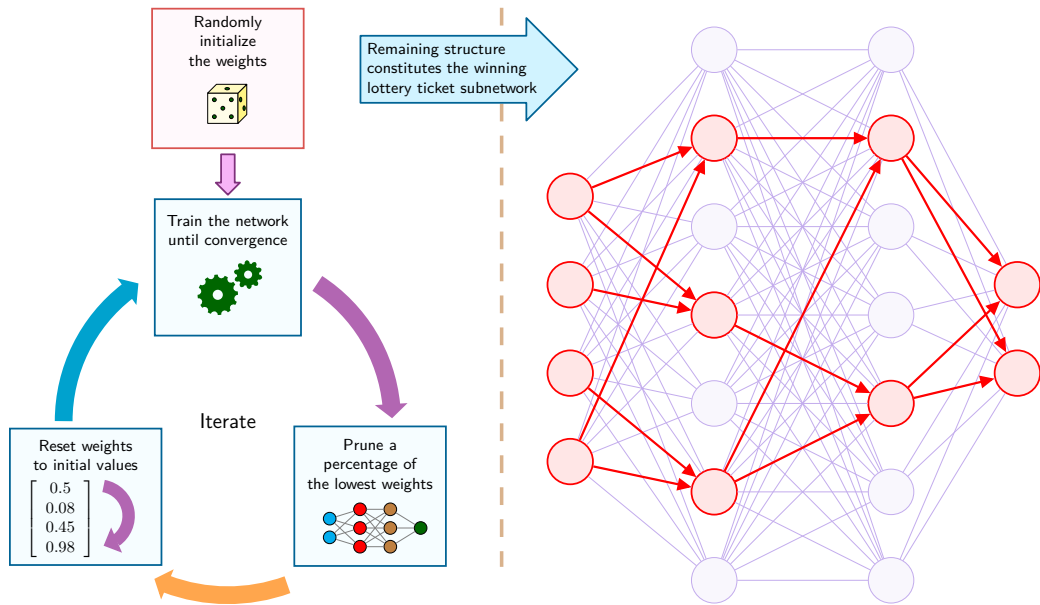


Figure 10.7: Lottery Ticket Iteration Cycle: The original procedure repeatedly trains, prunes low-magnitude weights, and resets surviving weights to their initial state θ_0 . The cycle tests whether the resulting sparse subnetwork can match the dense baseline when trained in isolation; the outcome is empirical rather than guaranteed.

the standard discovery procedure does not eliminate dense training cost: it first pays for one or more dense or partially dense runs to find the ticket. The result therefore emphasizes the role of initialization while leaving a practical systems question—whether the subnetwork can be identified before most of the original training budget is spent.

The iterative procedure also separates two effects that one-shot pruning conflates: which weights survive and how much recovery training follows each cut. In some settings, smaller pruning steps give the network more opportunity to recover; in others, the additional cycles do not justify their cost. The deployment comparison must therefore measure final quality, search or retraining cost, and realized runtime savings together.

In practice, an LTH-style pipeline should be compared with training a smaller dense model from the start, one-shot pruning with recovery, and direct sparse-training methods. The accounting must include every dense or partially dense discovery run, checkpoint rewind, pruning cycle, and recovery phase—not only the final subnetwork. A winning ticket is a training result; it becomes a systems win only when total accelerator time, final artifact size, target latency, and task quality beat those alternatives.

10.3.1.8 Pruning in practice

While the Lottery Ticket Hypothesis illuminates the optimization landscape, practical serving systems care only about executable artifacts. Framework-level pruning is useful only when it materializes concrete hardware savings: a smaller dense tensor, a hardware-aligned structured sparse matrix, or an indexed sparse layout paired with matching runtime kernels. Training-time pruning often begins as mask application: the original tensor remains resident in memory, while a binary mask zeros selected weights during forward propagation. That mechanism guides gradient flow during recovery, but it delivers zero memory reduction or latency improvement on its own. Measurable efficiency gains emerge only after the masked structure is exported into the physical artifact loaded by the inference engine.

This artifact boundary separates the pruning strategies. Unstructured pruning removes individual weights and needs sparse kernels plus an appropriate storage format to translate zeros into speed. Structured pruning removes whole channels, heads, neurons, or blocks, which can reshape tensors into smaller dense operations that ordinary accelerators already execute well. Gradual pruning

during fine-tuning adds a training schedule: sparsity increases over time so the remaining weights can recover accuracy as capacity is removed. The systems audit is therefore concrete: identify what is pruned, identify the runtime format, and verify that the target hardware has kernels that make the sparsity useful.

These trade-offs become concrete when examining real-world deployments. Some model families reduce deployment cost through architecture rather than post-hoc pruning: MobileNet uses depthwise separable convolutions for mobile and embedded vision (Howard et al. 2017), while EfficientNet uses compound scaling to improve the accuracy-efficiency trade-off under resource constraints (Tan and Le 2019). Pruning remains a separate optimization lever. BERT-style transformers⁸ have been pruned by removing redundant attention heads or intermediate dimensions, while separate distillation methods such as DistilBERT and TinyBERT train smaller dense student models that retain much of BERT’s performance (Sanh et al. 2019; Jiao et al. 2020).

Pruning has an inherent limitation: it starts with an existing architecture and carves away pieces. The pruned model inherits its structure from the original—same layer types, same connectivity patterns, just fewer parameters. The original architecture itself may be inefficient for deployment. A practitioner may need a model with a completely different structure, such as a six-layer transformer instead of a 12-layer one, that still captures the original model’s capabilities.

When deployment limits demand an entirely different topology, the compression objective shifts from carving away weights to supervising a smaller model with the outputs of the original. Knowledge distillation addresses this necessity by treating the overparameterized network as a teacher rather than a deployment artifact.

10.3.2 Knowledge distillation

Suppose a medical question-answering model exceeds the memory and latency budget of a hospital’s single-GPU server. Making its existing weights sparse may still leave an unsupported representation or an architecture too large for the target, so the deployment may require a fundamentally smaller model. Knowledge distillation addresses this class of problem by training a compact “student” to reproduce a larger “teacher’s” behavior, often retaining much of the teacher’s task performance at lower inference cost (Hinton et al. 2015; Sanh et al. 2019). The term *distillation* borrows from chemistry, where the process extracts a concentrated essence from a larger mixture,⁹ but the systems insight is specific: the teacher’s predictions can carry information beyond the raw training labels.

Definition 10.3: Knowledge distillation

Knowledge distillation is a model-compression technique that trains a smaller *student* model to match the behavior of a larger, pretrained *teacher* model.

1. **Significance:** Distillation moves cost from repeated inference to an additional training phase. A student can be substantially smaller than its teacher while retaining useful task performance; DistilBERT, for example, reports 40 percent fewer parameters, 60 percent faster inference, and 97 percent of BERT’s language-understanding performance (Sanh et al. 2019). This trade-off is valuable when the student-training cost is amortized across many deployed queries.
2. **Distinction:** Unlike pruning, which removes parameters from an existing architecture, and quantization, which lowers numerical precision, distillation trains a new dense architecture. The student inherits behavior from the teacher’s output distribution or intermediate representations rather than inheriting the teacher’s full parameter count or layer structure.
3. **Common pitfall:** A frequent misconception is that distillation is lossless compression. In reality, the student is bounded by its own capacity, the quality of the teacher, and the match between the distillation data and deployment distribution; a student can faithfully reproduce teacher errors as well as teacher knowledge.

Teacher models transfer semantic knowledge through the relative probabilities assigned across all classes. Whereas a one-hot ground-truth label assigns a probability of one to the correct class

⁸ | **BERT pruning:** BERT’s 12 attention heads per layer can contain substantial redundancy—Michel et al. (2019) found on MultiNLI that many heads could be removed without a statistically significant performance change. The removable fraction depends on the layer and task, so deployment pruning must measure both task quality and whether the resulting structure reduces runtime work.

⁹ | **Distillation:** The name borrows from the chemical process of separating a mixture. In model compression, the student learns from a teacher’s softened output distribution or internal representations rather than copying its parameters. Soft targets can encode relationships among classes, but the student learns only what its objective and training data expose. The metaphor is useful but not literal: the temperature parameter comes from softmax scaling, and the size-quality trade-off depends on the student, data, objective, and training procedure (Hinton et al. 2015). Deployment savings come from the student architecture, not from the metaphor itself.

and zero to all others, the teacher’s output distribution in figure 10.8 captures inter-class structural relationships—revealing that an input classified as a cat shares more semantic features with a dog than with a fox.

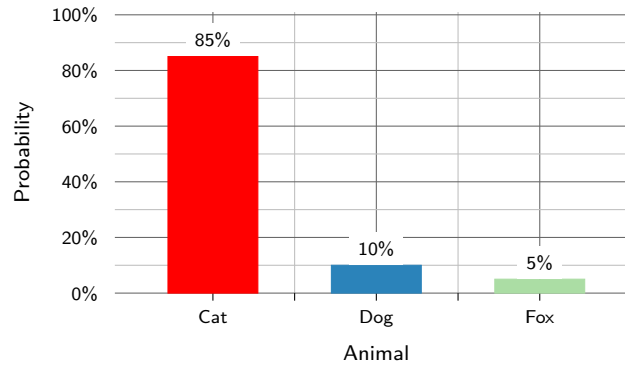


Figure 10.8: Soft Target Distribution: The teacher’s relative confidence levels indicate which classes are semantically similar (for example, cat vs. dog), providing a much richer supervision signal than a binary “correct” label.

Training a student model requires balancing teacher imitation against ground-truth task supervision. Operationally, the workflow requires four decisions before training begins: selecting a teacher with the desired behavior, designing a student whose dense architecture satisfies the target accelerator’s memory and compute budget, generating soft targets over the training corpus, and choosing the temperature and loss weights that govern optimization. Figure 10.9 illustrates this dual-path training workflow, where each input sample passes through both networks to drive student parameter updates from combined soft and hard loss functions.

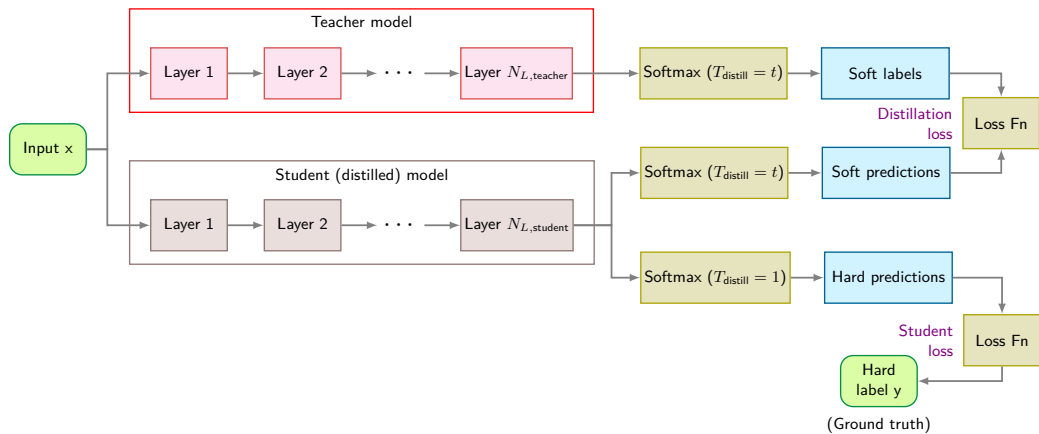


Figure 10.9: Knowledge Distillation Workflow: An input sample passes through both the teacher and the student network. The teacher produces soft labels via temperature-scaled softmax, while the student output is compared against both the soft labels (distillation loss) and the hard labels (student loss).

Deploying the distilled student requires verifying more than top-1 accuracy. Because a student can inherit the teacher’s systematic biases and miscalibrations as readily as its useful representations, verification must measure calibration error, subgroup behavior, and executed latency on the target hardware.

10.3.2.1 Distillation mathematics

Standard softmax normalization (section 5.3.2.2) converts raw model logits z_i into probabilities. Introducing a **temperature parameter**¹⁰ T_{distill} softens the resulting probability distribution across

¹⁰ | **Temperature (softmax):** The term comes from the Boltzmann distribution in statistical mechanics. Dividing logits by a temperature above one produces a softer distribution and can expose relative probabilities among non-target classes. At $T_{\text{distill}}=1$, the expression is standard softmax; increasing the temperature reduces logit gaps but does not guarantee that the revealed probabilities help the student. The useful amount of softening depends on logit scale, teacher calibration, student capacity, loss weighting, and the accompanying T_{distill}^2 factor. Temperature is therefore selected empirically rather than fixed to a universal range.

output classes:

$$p_i^{(T_{\text{distill}})} = \frac{\exp(z_i/T_{\text{distill}})}{\sum_j \exp(z_j/T_{\text{distill}})}$$

A higher T_{distill} flattens the softmax distribution and exposes **dark knowledge**: the latent information contained in relative class probabilities, such as a teacher assigning a small but nonzero probability to dog and near-zero probability to truck when classifying an image of a cat. The total distillation loss $\mathcal{L}_{\text{distill}}$ balances standard cross-entropy with the Kullback-Leibler (KL) divergence against the teacher’s softened distribution:¹¹

$$\mathcal{L}_{\text{distill}} = (1 - \gamma_{\text{KD}}) \mathcal{L}_{\text{CE}}(\mathbf{p}_{\text{student}}, y) + \gamma_{\text{KD}} T_{\text{distill}}^2 \mathcal{D}_{\text{KL}}(\mathbf{p}_{\text{teacher}}^{(T_{\text{distill}})} \parallel \mathbf{p}_{\text{student}}^{(T_{\text{distill}})})$$

Here $\mathbf{p}_{\text{teacher}}^{(T_{\text{distill}})}$ and $\mathbf{p}_{\text{student}}^{(T_{\text{distill}})}$ are the probability vectors computed at temperature T_{distill} , y is the one-hot ground-truth label, and $\gamma_{\text{KD}} \in [0, 1]$ balances the hard-label and soft-target objectives. The scaling factor T_{distill}^2 ensures that the gradient magnitude of the distillation loss remains commensurate with the hard cross-entropy loss: dividing logits by T_{distill} scales the derivative of the softmax probabilities by $1/T_{\text{distill}}$, and the gradient of the KL divergence with respect to the student’s logits scales as $1/T_{\text{distill}}^2$ in the high-temperature limit. Multiplying by T_{distill}^2 prevents the teacher’s supervisory gradient from vanishing relative to the hard-label loss as T_{distill} increases. This formulation defines response-based distillation, matching final output distributions. Modern pipelines also employ feature-based distillation, training intermediate student layers to directly match the teacher’s hidden representations or attention maps using mean squared error or cosine distance to guide gradient updates across deep backbones.

Efficiency gains and trade-offs Distillation’s primary deployment advantage over unstructured pruning is that it produces a smaller *dense* model. Standard accelerators achieve peak arithmetic intensity on contiguous matrix multiplications; unstructured sparse models incur irregular memory access and index decoding overheads that often negate theoretical FLOP reductions unless sparsity is extreme. A compact dense student maintains coalesced memory access and executes vendor-optimized dense GEMM kernels. However, parameter reduction alone does not guarantee lower latency: operational intensity, layer depth, attention sequence length, and memory bandwidth limits still govern runtime. DistilBERT, for example, achieves its 60 percent latency reduction primarily by removing half of BERT’s transformer layers, which cuts memory traffic and sequential layer dependencies in half while retaining 97 percent of BERT’s language-understanding performance (Sanh et al. 2019).¹² Compact vision architectures such as MobileNet apply the same principle when trained under teacher supervision (Howard et al. 2017; Hinton et al. 2015).

Distillation naturally composes with pruning and quantization in multi-stage compression pipelines. For example, a practitioner can distill a large teacher into a compact student architecture, prune redundant attention heads or channels from that student, and quantize the surviving weights from FP16 to INT8 for deployment. Gordon et al. (2020) show that BERT can be pruned during pretraining and transferred downstream, demonstrating that structural compression and distillation can operate concurrently.

These operational gains trade inference efficiency for substantial training overhead. Distillation requires running inference across the teacher model to generate soft targets and backpropagating loss through the student across millions of tokens. For large models, this auxiliary training phase consumes significant accelerator FLOPs and memory bandwidth, whereas post-training quantization methods complete in minutes on calibration subsets. Compression effectiveness is also bounded by student capacity and teacher fidelity: an undersized student lacks the capacity to absorb the teacher’s distribution, while an inaccurate or uncalibrated teacher imparts its own systematic errors to the student. As developed in chapter 12, selecting an optimization strategy requires evaluating task accuracy, parameter footprint, serving latency, and total training cost together.

Table 10.5 contrasts the key trade-offs between knowledge distillation and pruning across accuracy retention, training cost, inference speed, hardware compatibility, and implementation complexity. DistilBERT and MobileBERT demonstrate architecture redesign plus distillation; pruning can be combined with distillation in other optimization pipelines, but these models should be understood primarily as dense student-model examples.

¹¹ | **KL divergence:** Introduced by Solomon Kullback and Richard Leibler in 1951, $\mathcal{D}_{\text{KL}}(p \parallel q)$ quantifies the extra bits needed to encode samples from distribution p using a code optimized for q . The key asymmetric consequence: $\mathcal{D}_{\text{KL}}(\text{teacher} \parallel \text{student})$ penalizes the student heavily for assigning zero probability to teacher-probable outputs, forcing the student to maintain broad coverage of the teacher’s distribution, including low-probability “soft labels” that carry the teacher’s learned uncertainty. Distillation can affect calibration, which must be evaluated separately.

¹² | **DistilBERT:** The reported model has 66 million parameters versus BERT-Base’s 110 million, is 40 percent smaller and 60 percent faster, and retains 97 percent of its language-understanding performance. Absolute memory and latency still depend on numerical format, batch size, sequence length, runtime, and hardware (Sanh et al. 2019).

Table 10.5: Distillation vs. Pruning: Distillation spends additional training and student-design effort to produce a smaller dense model; pruning modifies an existing model, but sparse results need compatible storage and kernels to realize deployment gains.

Criterion	Knowledge Distillation	Pruning
Accuracy retention	Model- and student-dependent; often strong with a capable teacher	Model- and sparsity-dependent
Training cost	Higher – Requires teacher inference and student training	Often lower – Requires pruning and usually fine-tuning
Inference speed	Often favorable for a smaller dense student	Depends – Structured pruning is efficient, unstructured needs special support
Hardware compatibility	High – Works on standard accelerators	Limited – Sparse models may need specialized execution
Implementation effort	More involved: teacher inference, student objective, and training	Varies: choose a criterion, prune, fine-tune, and export

Pruning modifies an existing parameterization, while distillation transfers behavior into a chosen student architecture. Neither directly represents a dense weight tensor through low-rank factors. For an illustrative 4096×4096 matrix, a rank-128 factorization stores 6.25 percent as many scalar parameters as the dense matrix; whether that approximation is accurate depends on the matrix’s singular-value spectrum. Structured approximation methods exploit this form of mathematical redundancy.

10.3.3 Structured approximations

Structured approximation addresses a distinct deployment constraint compared to pruning or distillation: when a layer’s weight tensor exhibits mathematical redundancy, representing that redundancy directly requires less storage capacity and memory bandwidth than fetching the dense tensor from DRAM. Rather than eliminating individual weights or transferring behavior to a separate architecture, structured methods decompose large weight matrices and tensors into compact, lower-dimensional factors. Low-rank matrix factorization and tensor decomposition provide complementary algebraic strategies for implementing this compression.

10.3.3.1 Low-rank factorization

Low-rank matrix factorization (LRMF) approximates weight matrices with lower-rank representations. Given a weight matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$, LRMF identifies factor matrices $\mathbf{U} \in \mathbb{R}^{m \times k}$ and $\mathbf{V} \in \mathbb{R}^{n \times k}$ such that:

$$\mathbf{A} \approx \mathbf{U}\mathbf{V}^T$$

where the approximation rank $k \ll \min(m,n)$. This decomposition is typically computed via singular value decomposition (SVD),¹³ retaining only the k largest singular values and absorbing their magnitudes into the factors. During inference, computing the layer output as $\mathbf{y} = \mathbf{U}(\mathbf{V}^T \mathbf{x})$ replaces a single large matrix-vector product with two sequential, smaller multiplications. The primary systems question is whether the reduction in parameter footprint and arithmetic operations compensates for the overhead of staging an intermediate activation and dispatching an additional kernel.

 Systems Perspective 10.2: The bandwidth-compute trade-off

Low-rank factorization can reduce both arithmetic and bandwidth demand when the retained rank is small, though it adds a second factor operation and possible kernel-launch overhead. Storing a 4096 by 4096 matrix requires 67.1 MB (at FP32). Fetching this matrix for a single inference imposes substantial memory traffic, especially when execution is limited by physical memory bandwidth.

Factoring the matrix at rank $k = 128$ requires storing two smaller matrices (4096 by 128 and 128 by 4096), totaling only 4.2 MB—a 16× reduction in stored values. When inference uses the two factors directly, matrix-vector arithmetic also falls from $\mathcal{O}(mn)$ to $\mathcal{O}(k(m+n))$ for small k . The

¹³ | **Singular value decomposition (SVD):** The Eckart-Young theorem (1936) proves that retaining the top k singular values yields an optimal rank- k approximation in the Frobenius and spectral norms. The key systems trade-off is whether the high, one-time compute cost of this factorization— $\mathcal{O}(mn \cdot \min(m,n))$ —is amortized by the memory and bandwidth savings from using the smaller model in repeated inference calls.

deployment speeds up only if the two factor operations and their intermediate traffic cost less than the original dense kernel. Explicitly materializing $\mathbf{UV}^T$ would add $\mathcal{O}(mkn)$ work and defeat the purpose.

This bandwidth-compute trade-off exemplifies the memory wall at the operator level: when batch sizes are small, execution is bottlenecked by the time required to fetch weight bytes from DRAM rather than the time required to perform multiply-accumulate operations. Section 11.5.1 examines this memory wall from the hardware architecture perspective. In production serving engines, evaluating the factored representation avoids full-rank materialization entirely. Figure 10.10 illustrates this decomposition, showing how the original matrix $\mathbf{M}$ splits into compact rank- k factors $\mathbf{L}_k$ and $\mathbf{R}_k^T$.

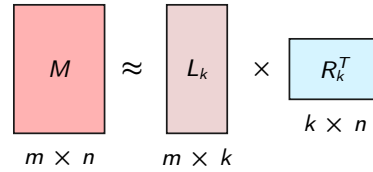


Figure 10.10: Low-Rank Factorization: A weight matrix $\mathbf{M}$ of size $m \times n$ is approximated by two smaller factors, labeled $\mathbf{L}_k$ ($m \times k$) and $\mathbf{R}_k^T$ ($k \times n$) in the diagram and corresponding to $\mathbf{U}_k$ and $\mathbf{V}_k^T$ in the surrounding notation. Storage falls from $m \times n$ to $m \times k + k \times n$ parameters, while one dense operation becomes two factor operations.

LRMF applies directly to fully connected projection layers and to flattened convolutional weight tensors. Evaluating the factored layer replaces a single dense matrix-vector product with two chained multiplications, dropping parameter storage and arithmetic operations from $\mathcal{O}(mn)$ to $\mathcal{O}(k(m+n))$. On modern accelerators, however, this split introduces concrete systems trade-offs: the intermediate activation vector $\mathbf{z} = \mathbf{V}^T \mathbf{x}$ must be staged in on-chip SRAM or round-tripped through memory, and dispatching two separate kernels incurs launch latency unless the operations are fused. Furthermore, if the approximation rank k is small or unaligned with hardware tile dimensions (such as 16×16 systolic arrays), the hardware compute units operate well below peak throughput. Selecting rank k therefore balances memory footprint and bandwidth reduction against approximation error, kernel dispatch overhead, and hardware tile utilization.

10.3.3.2 Tensor decomposition

Tensor decomposition extends low-rank approximation to multidimensional arrays, such as four-dimensional convolutional kernels ($C_{\text{out}} \times C_{\text{in}} \times K_h \times K_w$) and multi-head attention weights. Unfolding these parameters into two-dimensional matrices discards spatial and cross-channel correlations, leading to poor approximation accuracy for a given rank budget. Tensor decompositions preserve multiway structure by representing the tensor through multilinear products of lower-dimensional factors. Figure 10.11 illustrates canonical polyadic decomposition on a three-dimensional tensor, showing how outer products of factor vectors reconstruct individual tensor elements across rank R .

Different decomposition formats trade parameter reduction against contraction complexity. **CP decomposition** expresses a tensor as a sum of rank-one components, $\mathcal{X} \approx \sum_{r=1}^R \mathbf{u}_r \otimes \mathbf{v}_r \otimes \mathbf{w}_r$ (Lebedev et al. 2015), yielding maximal parameter reduction but often suffering from numerical instability during rank optimization. **Tucker decomposition** relaxes this constraint by retaining a small, dense core tensor multiplied along each mode by orthogonal factor matrices, $\mathcal{X} \approx \mathcal{G} \times_1 \mathbf{U} \times_2 \mathbf{V} \times_3 \mathbf{W}$. **Tensor-train (TT)** decomposes a high-order tensor into a linear chain of third-order core tensors contracted along shared auxiliary indices, avoiding the exponential scaling with tensor order that limits Tucker cores.

In deep learning systems, tensor decomposition compresses 4D convolutional filters, multi-head attention projections, and large token embedding tables. However, executing tensor contractions on production accelerators encounters a severe systems barrier: modern hardware architectures are heavily specialized for dense two-dimensional matrix multiplication (GEMM). Contracting decomposed tensors requires either re-permuting memory layouts through explicit transposition

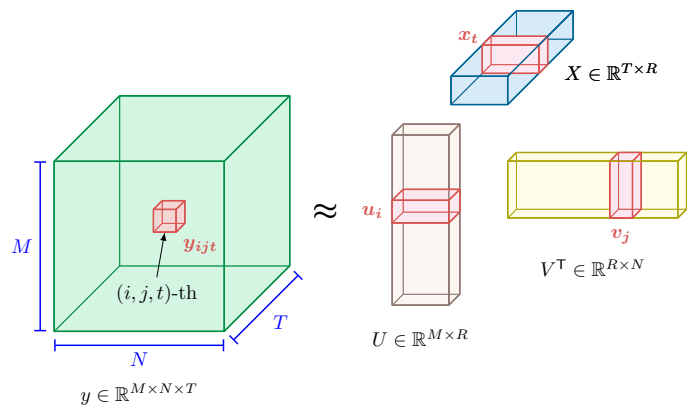


Figure 10.11: Tensor Decomposition: A three-dimensional tensor $y \in \mathbb{R}^{M \times N \times T}$ is approximated by factor matrices U , X , and V . The highlighted factor vectors u_i , x_t , and v_j combine across rank R to reconstruct the entry y_{ijt} . Extending low-rank approximation from two dimensions to three carries the storage and computation argument into convolutional layers. Source: (Gholami et al. 2022).

kernels or issuing multiple small contractions with uncoalesced memory access patterns. Unless supported by specialized fused contraction kernels, the memory movement and kernel dispatch latency of decomposed operations can easily erase the theoretical arithmetic savings. Table 10.6 compares LRMF and tensor decomposition across data structures, compression mechanisms, and execution trade-offs.

Table 10.6: Matrix and Tensor Factorization: LRMF applies to two-dimensional matrices, while tensor decompositions extend low-rank representations to multidimensional tensors; realized storage and latency depend on rank choice and kernel support.

Feature	Low-Rank Matrix Factorization (LRMF)	Tensor Decomposition
Applicable Data Structure	Two-dimensional matrices	Multi-dimensional tensors
Compression Mechanism	Factorizes a matrix into two or more lower-rank matrices	Decomposes a tensor into multiple lower-rank components
Common Methods	Singular Value Decomposition (SVD), Alternating Least Squares (ALS)	CP Decomposition, Tucker Decomposition, Tensor-Train (TT)
Compute cost	Depends on the factorization method, selected rank, and kernels	Depends on the decomposition, selected ranks, optimization procedure, and tensor-contraction kernels
Storage Reduction	Reduces storage from $\mathcal{O}(mn)$ to $\mathcal{O}(mk + kn)$	Depends on the decomposition and selected ranks; stores factor matrices and, for some methods, a core tensor
Inference execution	Replaces one dense operation with two factor operations	Introduces tensor contractions whose latency depends on available kernels
Primary Use Cases	Fully connected layers, embeddings, recommendation systems	Convolutional filters, attention mechanisms, multi-modal learning
Implementation Complexity	Easier to implement, often involves direct factorization methods	More complex, requiring iterative optimization and rank selection

In production pipelines, practitioners frequently combine these structured approximations, applying LRMF to large linear projections and embedding tables while targeting multi-dimensional convolutional layers with tensor decompositions. Under the D·A·M taxonomy, structured approximation alters the Algorithm to relieve Machine memory-capacity and bandwidth bottlenecks. When serving latency is strictly memory-bound—such as single-batch autoregressive token generation—the reduction in weight bytes transferred from DRAM translates directly to reduced latency. When serving is compute-bound, the overhead of multiple factor operations and fragmented memory accesses may nullify those gains.

Pruning and factorization modify existing model parameterizations, while distillation transfers behavior into an independently trained student model. Neural architecture search takes a fundamentally different path: discovering architectures that are efficient by construction.

10.3.4 Neural architecture search

Pruning, distillation, and factorization assume an existing topology engineered by human intuition. Selecting a competitive configuration can require extensive experimentation, and manual exploration risks leaving efficient architectural candidates undiscovered (Elsken et al. 2019). **Neural architecture search (NAS)** automates part of this process by exploring a specified space of architectures and objectives. Early reinforcement-learning NAS optimized validation accuracy (Zoph and Le 2016), weight-sharing methods reduced candidate-evaluation cost (Pham et al. 2018), and hardware-aware NAS added device latency or platform efficiency to the objective (Tan et al. 2019).

The optimization loop in figure 10.12 formalizes the search procedure. NAS¹⁴ defines a search space of architectural components and constraints, applies a strategy such as reinforcement learning (Zoph and Le 2016), evolutionary search, or gradient-based optimization, and evaluates candidates against accuracy and efficiency objectives. Each evaluation feeds performance metrics back to the search controller, steering subsequent candidate proposals toward regions that balance task accuracy with physical resource budgets.

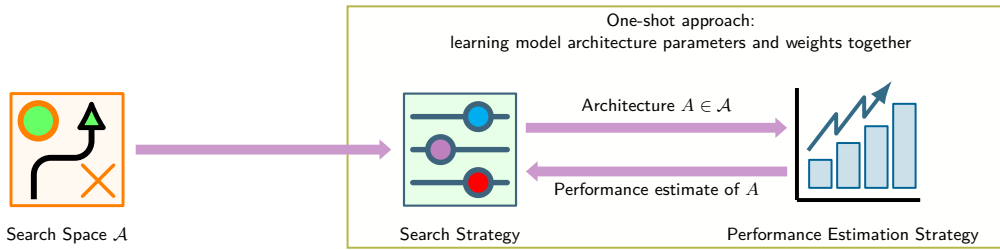


Figure 10.12: Neural Architecture Search Flow: An automated optimization loop for architecture discovery. The search strategy samples candidate subnetworks from a predefined search space, while the performance estimation engine evaluates candidates against target accuracy and hardware constraints (such as latency or energy), feeding reward signals back to steer subsequent exploration toward Pareto-optimal designs.

10.3.4.1 The NAS optimization problem

The effectiveness of NAS depends on three design decisions: what architectures to search over (the search space), how to explore that space efficiently (the search strategy), and how to evaluate each candidate’s fitness for deployment. The optimization problem begins with a chicken-and-egg constraint: we cannot know how good an architecture is until we train it, but training is expensive. This creates two nested decisions: choosing which operations to include (the architecture) and finding the best parameters for those operations (the weights). The architecture defines *what* to optimize; the weights define *how well* that architecture can perform.

NAS is therefore a **bi-level optimization problem**.¹⁵ The outer loop searches the architecture space $\mathcal{A}$, while the inner loop trains candidate architectures to evaluate performance. Formally, we seek the optimal architecture α^* that minimizes validation loss $\mathcal{L}_{\text{val}}$ under constraints C (latency, memory):

$$\alpha^* = \arg \min_{\alpha \in \mathcal{A}} \mathcal{L}_{\text{val}}(\theta^*(\alpha), \alpha) \quad \text{subject to} \quad C(\alpha) \leq C_{\text{max}}$$

where $\theta^*(\alpha)$ represents the optimal model parameters for architecture α , obtained by minimizing training loss:

$$\theta^*(\alpha) = \arg \min_{\theta} \mathcal{L}_{\text{train}}(\theta, \alpha)$$

The computational bottleneck lies in this inner loop. If evaluating every candidate requires independent training from scratch, a modest search space of 10 operation choices across 20 layers yields 10^{20} possible architectures, making exhaustive enumeration impossible. Practical NAS methods overcome this combinatorial explosion through three mechanisms: restricting the search space to reusable cells, amortizing evaluation through weight sharing, or relaxing discrete choices into continuous, differentiable formulations.

¹⁴ | **Hardware-aware NAS:**

Optimizes measured latency rather than FLOPs, which can diverge by 3–5× when memory access or operator support dominates. Tan et al. (2019) fed device latency into search and found architectures 1.8× faster than MobileNetV2 at higher accuracy.

¹⁵ | **Bi-level optimization:**

A formulation where one optimization problem sits inside another. In NAS, the outer level selects an architecture while the inner level trains that candidate’s weights, so early methods paid a full training cost for every architecture evaluated. This nesting is why early NAS required 22,400 GPU-days; weight-sharing methods amortize one training run across many candidates, reducing search cost by roughly 1,000×.

10.3.4.2 Search space design

The search space defines what architectures NAS can discover. Well-designed search spaces incorporate domain knowledge to focus search on promising regions while remaining flexible enough to discover novel patterns.

Rather than searching entire network architectures, many NAS systems search for reusable computational blocks, or cells, that can be stacked to form complete networks. A convolutional cell might choose from operations such as 3×3 convolution, 5×5 convolution, depthwise separable convolution, max pooling, or identity connections. A simplified cell with four nodes and two operations per edge yields roughly 10,000 possible cell designs, far more tractable than searching full architectures. NASNet exemplifies this approach, discovering reusable normal and reduction cells that can be stacked to form complete networks across different model sizes.

Search spaces also encode physical hardware constraints directly. Rather than allowing arbitrary topological graphs, hardware-aware search spaces restrict candidate operations to primitives that have efficient, vectorized kernel implementations on the target platform. For example, mobile search spaces prioritize depthwise separable convolutions with bounded kernel sizes and channel alignments to fit on-chip cache lines and minimize memory bus transactions (Zhang et al. 2020). MobileNetV3 paired this structured block-level search space with NetAdapt to prune layer channels against measured mobile CPU latency (Howard et al. 2019).

10.3.4.3 Search strategies

Search strategies determine how to explore the architecture space efficiently without exhaustive enumeration. Table 10.7 compares the trade-offs between search cost, architectural diversity, and optimality guarantees for each approach.

Table 10.7: NAS Search Strategy Comparison: Representative trade-offs between search efficiency, use cases, and limitations for different NAS approaches. The costs are method-specific results from early reinforcement-learning and evolutionary searches and from DARTS, not universal ranges for each strategy (Zoph and Le 2016; Real et al. 2019; Liu et al. 2019).

Strategy	Search Efficiency	When to Use	Key Challenge
Reinforcement Learning	22,400 GPU-days	Novel domains, unconstrained search	High computational cost
Evolutionary Algorithms	3,150 GPU-days	Parallel infrastructure available	Requires large populations
Gradient-Based (DARTS)	1–4 GPU-days	Limited compute budget	May converge to suboptimal local minima

Reinforcement learning-based NAS treats architecture search as a sequential decision: a controller generates architectures and receives an accuracy reward. The controller (typically a long short-term memory network) learns to propose better architectures through policy gradient optimization. This approach discovered high-performing architectures such as NASNet, but its inner loop is expensive because every reward requires training a candidate architecture; Zoph and Le (2016) evaluated roughly 12,800 architectures, totaling 22,400 GPU-days. This cost pushed practical NAS toward weight sharing and lower-cost performance predictors.

Evolutionary algorithms maintain a population of candidate architectures and iteratively apply mutations (changing operations, adding connections) and crossover (combining parent architectures) to generate offspring. Fitness-based selection retains high-performing architectures for the next generation, so useful components such as skip connections or depthwise separable convolutions can be recombined rather than rediscovered from scratch. AmoebaNet used evolution to achieve ImageNet accuracy competitive with top human-designed models after 3,150 GPU-days (Real et al. 2019), showing both the value of population search and the continuing need for proxy tasks, weight sharing, or massive parallelism to control search cost.

Gradient-based methods such as DARTS (Differentiable Architecture Search) (Liu et al. 2019) represent the search space as a continuous relaxation where all possible operations are weighted combinations. Rather than discrete sampling, DARTS optimizes architecture weights and model weights jointly using gradient descent. By making the search differentiable, DARTS reduces search cost from hundreds of GPU-days to one to four GPU-days, though the continuous relaxation may miss discrete architectural patterns that discrete search methods discover.

Hardware-aware NAS moves beyond theoretical FLOP counts by optimizing directly for execution metrics on target silicon. FLOP counts correlate poorly with runtime latency when memory bandwidth, cache residency, or kernel launch overheads dominate: an operation with low FLOPs but low arithmetic intensity stalls waiting for memory transactions, running slower than a higher-FLOP operation that achieves roofline compute saturation. MnasNet addresses this divergence by incorporating a latency prediction model trained on thousands of architecture-latency pairs measured on mobile phones. The search objective combines accuracy and latency through a weighted product:

$$\text{Reward}(\alpha) = \text{Accuracy}(\alpha) \times \left(\frac{L_{\text{lat,target}}}{L_{\text{lat}}(\alpha)} \right)^\beta$$

where $L_{\text{lat}}(\alpha)$ is measured latency, $L_{\text{lat,target}}$ is the latency constraint, and β controls the accuracy-latency trade-off. This formulation penalizes architectures that exceed latency targets while rewarding those that achieve high accuracy within the budget. In the reported MnasNet search space, varying inverted-residual expansion ratios produced a stronger accuracy-latency trade-off than uniform expansion, illustrating how search can expose nonuniform designs that a narrower manual sweep might not test.

10.3.4.4 When to use NAS

Neural architecture search can discover architectures that outperform hand-designed alternatives, but its computational cost demands careful consideration of when the investment is justified. NAS can be worthwhile for novel hardware platforms with unusual constraints, such as new accelerator architectures or highly constrained edge devices, where existing architectures are poorly optimized. It can also make sense at deployment scales where small efficiency improvements justify the upfront search cost, or when architecture families for cloud, edge, and mobile deployments can amortize one search across many variants.

Conversely, custom NAS is difficult to justify when standard deployment constraints already have well-optimized architecture families. If the compute budget is only a few GPU-days, large reinforcement-learning or evolutionary searches are impractical; differentiable or weight-sharing methods such as DARTS may remain feasible, but still require deployment-scale and validation-cost justification. Rapidly changing requirements also weaken the case because the target can move before search and validation finish.

For most practitioners, existing NAS-discovered or NAS-assisted architectures such as EfficientNet (Tan and Le 2019), MobileNetV3 (Howard et al. 2019), and MnasNet (Tan et al. 2019) provide strong baselines before funding a search from scratch. Their transfer quality still depends on task and hardware. Reserve custom NAS for unusual constraints or deployment scales that can amortize the search and validation investment.

10.3.4.5 Architecture examples

NAS studies have surfaced reusable design patterns within their search spaces. EfficientNet jointly scales depth, width, and resolution with compound coefficients and reports improved accuracy-efficiency trade-offs across its model family (Tan and Le 2019). MobileNetV3 combines hardware-aware search and NetAdapt for phone latency targets (Howard et al. 2019), while FBNet incorporates device-specific latency into a mobile-CPU search objective (B. Wu et al. 2019). These results show what search found under particular spaces, objectives, and measurements; they do not establish that automated search universally outperforms careful manual design.

Beyond convolutional networks, NAS has been applied to transformer architectures, including searches for compact language and vision models under compute or memory constraints. The common systems lesson is narrower than a claim that search always beats manual design: when latency, memory, or energy enters the objective and is measured on the target platform, the search evaluates candidates against the resource that deployment actually constrains. The result still depends on the search space, proxy quality, optimization budget, and final retraining.

The structural techniques covered so far (pruning, distillation, factorization, and NAS) all optimize *what* computations the model performs, including which parameters exist, which connections remain, and how the architecture is structured. These techniques can substantially reduce parameter counts

and theoretical FLOPs. Regardless of structural efficiency, every surviving weight and activation must still be stored and processed at some numerical precision.



Checkpoint 10.2: Choosing a structural method

Structural methods relocate deployment cost in different ways.

- ☐ **Structured vs. unstructured pruning:** compare how each pattern interacts with GPU and TPU execution.
- ☐ **Distillation vs. pruning:** explain why a dense student may preserve accuracy better than aggressive pruning while requiring additional training.
- ☐ **NAS selection:** identify when hardware-specific constraints and deployment scale justify architecture search rather than manual design.

Surviving operations must still execute within physical memory bandwidth and register capacity. The second dimension of the optimization framework governs the numerical precision of those operations. Structural methods reduce parameter counts and nominal FLOPs; precision optimization alters the memory traffic and arithmetic cost of every remaining value. A 32-bit floating-point value occupies 4 bytes, whereas an 8-bit integer occupies 1 byte before scale or packing metadata. For bandwidth-bound inference, this smaller representation reduces memory traffic directly when the runtime maps operators to supported integer execution units. While well-calibrated INT8 workloads frequently retain baseline accuracy, task quality remains sensitive to model family, layer sensitivity, and hardware implementation (Jacob et al. 2018; Gholami et al. 2022).

Quantization is often an attractive first deployment experiment because it leaves the architecture intact and can be applied post-training. Its leverage is greatest when weight or activation traffic binds and the target provides efficient low-precision kernels; otherwise, it may shrink the artifact without accelerating the request.

10.4 Quantization and Precision

The physical gap governing compression is dictated by memory constraints: a 7-billion parameter language model in FP16 requires 14 GB solely for weights, while the smartphone target offers only 8 GB of shared RAM. Structured pruning can remove layers or heads, but unstructured zeros do not shrink a densely stored deployment artifact without specialized format overhead (Han, Mao, et al. 2016). A complementary lever reduces the bit width representing each parameter: determining how far precision can drop before accuracy collapses establishes the practical envelope of quantization.



Definition 10.4: Quantization

Quantization is a model-compression technique that reduces information fidelity by mapping high-precision continuous values to a lower-precision discrete set.

1. **Significance:** FP32-to-INT8 conversion reduces the raw payload per quantized value by $4\times$. Real artifact size, traffic, and latency also depend on scales, metadata, packing, operator coverage, and the target kernels.
2. **Distinction:** Unlike pruning, which reduces the count of parameters, quantization reduces the bit depth of selected weights and/or activations.
3. **Common pitfall:** A frequent misconception is that quantization is just “rounding.” In reality, it is a lossy mapping that requires careful range estimation and, often, quantization-aware training (QAT) to minimize its impact on accuracy.

Quantization¹⁶ concerns every neural network weight and activation stored at some numerical precision: FP32 (32 bits), FP16 (16 bits), INT8 (8 bits), or lower. Bit width therefore becomes a critical systems parameter. It constrains raw payload size and cache footprint, affects bandwidth demand

¹⁶ | **Quantization:** Rooted in Shannon’s theory of representing continuous signals with discrete values (Shannon 1948), reducing FP32 to INT8 collapses over four billion representable values to just 256. Neural networks often tolerate this because trained weights concentrate information in relative magnitudes, not absolute precision: INT8 inference can stay close to the full-precision baseline with appropriate calibration or quantization-aware training, while INT4 and lower-bit methods become more architecture- and method-dependent (Jacob et al. 2018; Gholami et al. 2022; Shen et al. 2020; Lin et al. 2024). The systems consequence is that quantization viability must be validated per-model and per-task, not assumed from aggregate benchmarks.

when values remain packed, and shapes the area and throughput trade-offs of specialized matrix engines (section 11.4.8).

Three system properties can change. The raw weight payload falls by $4\times$ from FP32 to INT8, before scale and packing metadata. Weight traffic can fall by a similar factor when values remain packed through the memory path, accelerating bandwidth-bound inference such as some low-batch large language model (LLM) decoding. Compute cost can also fall when the target provides efficient INT8 kernels; unsupported operators, conversions, or fallback paths can erase part of the gain (Gupta et al. 2015; Y. E. Wang et al. 2019).

The accuracy cost varies by model, task, quantizer, calibration data, and bit width. Many convolutional neural network (CNN) inference models tolerate INT8 well; transformers, speech models, and outlier-heavy layers may need different granularity, mixed precision, or training-aware methods. The main approaches are **post-training quantization** (PTQ), **quantization-aware training** (QAT), and **extreme quantization** for lower-bit deployment. They differ in training cost and control, not in a guaranteed ordering of accuracy.

The viability of each approach depends on how much precision a particular model can shed before quality deteriorates. Figure 10.13 illustrates a common qualitative pattern: a plateau where modest precision reduction has little measured effect, followed by a model-dependent cliff. The bit width at either boundary is not universal.

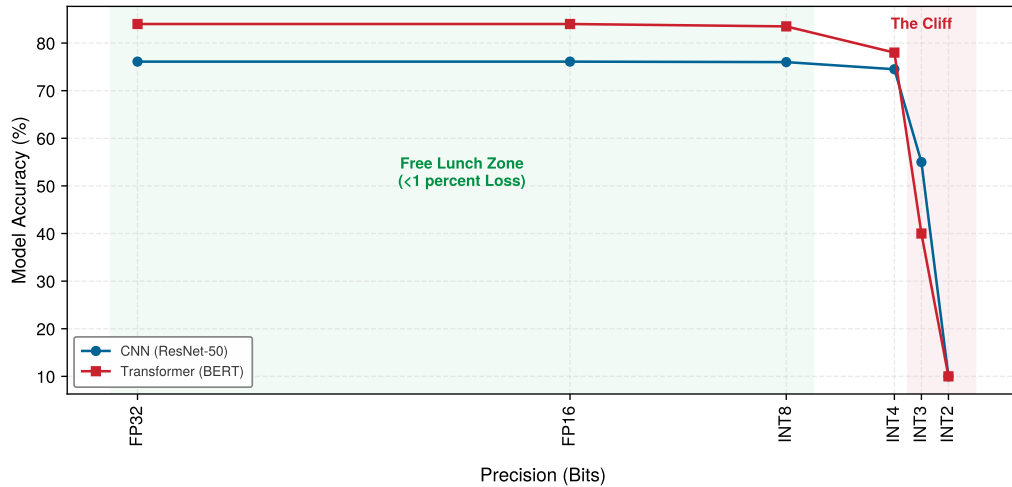


Figure 10.13: Illustrative Quantization Regimes: The curves depict a qualitative plateau followed by an accuracy cliff as bit width falls. Their breakpoints are illustrative; actual boundaries depend on the model, layer distributions, quantizer, calibration or training method, and evaluation task.

10.4.1 Precision and energy

Precision is an energy decision because physical data movement dominates the thermal and power envelopes of modern computing systems. In silicon, energy dissipation scales directly with wire capacitance ($E \propto CV^2$), meaning that shuttling bits across long memory buses costs orders of magnitude more joules than toggling logic gates inside an arithmetic logic unit. Moving a 32-bit floating-point operand from off-chip DRAM to local registers consumes substantially more energy than executing the multiply-accumulate operation itself. Reducing numerical precision directly contracts the byte volume traversing the memory hierarchy (D_{vol}) and lowers the switching energy per ALU operation ($E_{compute}$), relaxing memory bandwidth and thermal bottlenecks across the hardware stack.

🔍 Systems Perspective 10.3: The physics of quantization

The energy-movement invariant ($E_{\text{move}} \gg E_{\text{compute}}$) means that, in the physics of silicon, every fetched bit carries an energy cost (Horowitz 2014). Section D.1 collects the reference energy ratios, and section D.4 compares numerical format trade-offs.

According to the iron law ($T = \frac{D_{\text{vol}}}{BW} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}}$), reducing bit width changes the data-movement and compute terms only through the execution path that uses the new representation. A first-order packed-value model gives two useful comparisons:

1. **Memory movement** ($D_{\text{vol}} \times E_{\text{move}}$): A packed INT8 stream carries one quarter as many raw value bits as FP32. The corresponding energy estimate assumes full transactions are amortized across packed values; caches, metadata, alignment, and transaction size affect the realized cost.
2. **Compute work** ($O \times E_{\text{compute}}$): A 32-bit multiply costs ≈ 3.7 pJ/op. An INT8 multiply costs ≈ 0.2 pJ/op.

Table 10.8 normalizes these costs against an 8-bit integer add to expose the four-order-of-magnitude gap between arithmetic and DRAM access.

For inference workloads, moving from FP32 to INT8 saves $4\times$ in raw value storage, while the two multiply costs above give an idealized multiply-energy reduction of $18.5\times$. End-to-end energy also includes memory, control, sensors, and idle power. Section D.2.1 gives this relationship a formal shape: workloads with little arithmetic per byte are memory bound, so shrinking data movement dominates; workloads with more arithmetic per byte are compute bound, so arithmetic savings matter more. The arithmetic ratio alone cannot predict device battery life.

These same physics apply at data-center scale: distributed training systems use reduced precision to cut gradient communication overhead, a topic covered in section 8.6.3. Chapter 11 returns to the silicon mechanisms that exploit these energy differences.

Silicon-level energy measurements explain why systems exploit quantization, but hardware efficiency alone does not determine deployment viability. The operational envelope of quantization depends on two interacting constraints across the D·A·M boundary: the algorithm's tolerance to numerical perturbation before task accuracy degrades, and the machine's ability to translate narrower bit widths into reduced energy and wall-clock speedup. Quantifying the hardware side of this trade-off requires measuring the physical energy consumed across each tier of the memory and execution hierarchy.

10.4.1.1 Energy costs

Table 10.8 summarizes the canonical gap between arithmetic and data movement, normalizing operation costs against an 8-bit integer addition. Moving data across the memory interface incurs an energy penalty four orders of magnitude larger than executing on-chip arithmetic, establishing the primary physical motivation for bit-width reduction.

Table 10.8: The Energy Hierarchy: Relative energy cost of representative arithmetic and memory operations, normalized to an 8-bit integer add. A 32-bit float add costs $30\times$ more than its 8-bit integer counterpart, but a single 32-bit DRAM read costs $21,333.3\times$ more, placing DRAM access more than four orders of magnitude above on-chip compute. This gap is why bit-width reductions that shrink data movement dominate the energy budget of inference far more than the arithmetic savings alone would suggest (Horowitz 2014).

Operation	Bit-Width	Relative Energy
Integer Add	8-bit	$1\times$
Float Add	32-bit	$30\times$
DRAM Read	32-bit	$21,333.3\times$

Figure 10.14 extends this baseline across numerical precisions and the on-chip SRAM hierarchy. In the reference 45 nm technology model, a 32-bit integer addition costs 0.1 pJ/op, whereas an 8-bit

integer addition consumes 0.03 pJ/op. Floating-point arithmetic follows the same progression: a 32-bit floating-point addition consumes approximately 0.9 pJ/op, whereas a 16-bit floating-point addition requires 0.4 pJ/op. Relative to the 0.03 pJ INT8 add, on-chip SRAM reads span 5 to 50 pJ—a factor of 167 to 1,667 $\times$ —demonstrating that operand placement within the cache hierarchy dictates energy consumption as heavily as the arithmetic format itself. Off-chip DRAM access is costlier still. At scale, these per-operation disparities compound across billions of parameters and operations. For memory-bound workloads, moving unquantized parameters across high-capacitance DRAM buses saturates bandwidth and dominates accelerator power consumption (Patterson et al. 2021), whereas on energy-constrained edge devices, redundant bit width directly exhausts battery reserves and violates thermal limits.

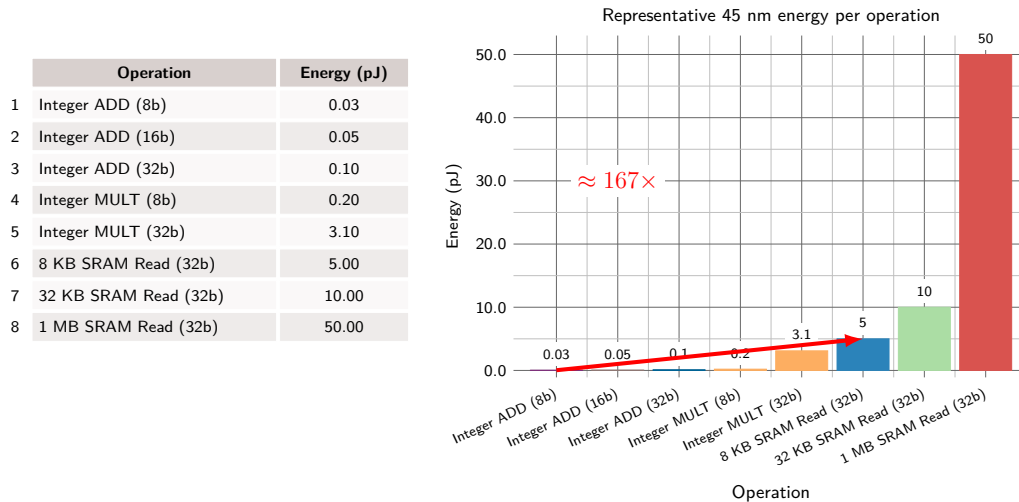


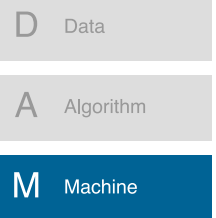
Figure 10.14: Energy per Operation by Precision: Representative 45 nm energy costs range from 0.03 pJ for an INT8 add to 3.10 pJ for an INT32 multiply and 5 to 50 pJ for the SRAM reads shown. The diagonal marks the 8 KB read at approximately 167 times the INT8-add energy; the complete SRAM range spans approximately 167 to 1,667 times that baseline. Energy figures from (Horowitz 2014).

10.4.2 The energy dividend: INT8 vs. FP32

Memory reduction is often the first motivation for quantization, but lower-precision arithmetic can also provide an **energy dividend**. Moving from FP32 to INT8 reduces raw value payload by exactly 4 $\times$; the operation-level energy ratio can be larger, while end-to-end energy remains workload- and device-dependent.

The hardware energy hierarchy exposes this operational asymmetry. In the reference technology model used here, an FP32 addition costs 0.9 pJ/op, while an INT8 addition costs 0.03 pJ/op, an operation-level ratio of 30 $\times$. That ratio is not a device-level battery-life prediction: memory accesses, accumulation precision, control, sensors, and idle power remain. It does explain why many accelerators add specialized low-precision units and why quantization can be necessary when arithmetic or memory energy binds.

These energy savings shift fundamentally when memory capacity, rather than compute throughput, is the binding physical constraint. Recommendation models convert the energy argument into a placement problem: quantization must shrink the dominant memory structures enough to reside in the host machine’s physical memory.



Quantization pays off only when the machine axis has the right integer units.



Lighthouse 10.1: DLRM and embedding quantization

The DLRM lighthouse (section 6.7) presents a compression challenge where memory capacity, rather than compute throughput or memory bandwidth, is the binding constraint (Naumov et al. 2019). Its embedding tables can reach terabytes in size, far exceeding GPU memory.

For DLRM, quantization targets storage density rather than arithmetic throughput. Reducing embedding-table precision from FP32 to INT8 (or lower) shrinks the memory footprint by 4–8×, allowing larger tables to fit on fewer GPUs when accuracy and lookup kernels tolerate the lower precision. Compressing the lookup tables ensures that physical device memory can accommodate the algorithmic model.

Beyond direct compute savings, reducing numerical precision lowers memory energy consumption, which often dominates total system power. Accessing memory, particularly off-chip DRAM, is far more energy-intensive than evaluating arithmetic logic: the representative 32-bit DRAM read used in this chapter costs 640 pJ, compared with picojoule-scale cache and arithmetic operations. An instruction’s total energy is therefore dominated by memory traffic rather than ALU computation.¹⁷

Together, DLRM and the TinyML lighthouse (10.2) illustrate the two operational extremes of this memory constraint. DLRM uses lower precision to fit terabyte-scale embedding tables into host memory; TinyML uses lower precision to keep every inference within a tiny energy envelope and a few hundred kilobytes of on-chip SRAM.



Lighthouse 10.2: The TinyML quantization imperative

The keyword spotting (KWS) lighthouse, DS-CNN, operates at the opposite extreme of the iron law from DLRM (Y. Zhang et al. 2017). A typical TinyML budget is roughly 512 KB of SRAM, and even the purpose-built DS-CNN keyword spotter overshoots it in FP32: its weights occupy 800 KB. Quantized to INT8, the same network shrinks to 200 KB and fits with room for runtime buffers. A stricter smart-doorbell microcontroller with 256 KB SRAM leaves closer to 100 KB for model weights after runtime buffers, audio windows, and feature extraction state, motivating more aggressive INT4 or binary compression.

In FP32, even the compact DS-CNN architecture moves 4× more raw weight bytes than in INT8, and the arithmetic path can also cost more on hardware with efficient integer units. For an always-on coin-cell device, that reduction contributes to longer battery life, but sensor, wake-up, feature-extraction, and idle-power costs may dominate. Here, quantization reduces the model’s data-movement term and can improve its compute term ($O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$) when the integer path is supported.

Reducing numerical precision thus attacks physical execution costs on two distinct fronts: arithmetic latency in the ALU and byte traffic across the memory bus. How these reductions translate into wall-clock speedup depends directly on whether the workload is bound by compute throughput or memory bandwidth.

10.4.2.1 Performance gains

Figure 10.15 compares illustrative FP32 and INT8 alternatives. The paired bars are not common-platform latency measurements; the storage panel shows the exact 4× reduction from 32 to 8 bits.

Large language models illustrate how weight precision dictates physical memory feasibility at scale.



Napkin Math 10.1: Quantization savings

Problem: Deploying Llama 3 8B requires storing 8B parameters on-device. How much memory does the model consume at FP16 vs. INT4, and does quantization shrink it enough to fit on a single consumer GPU?

¹⁷ | **INT8 energy impact:** The energy dominance of memory access is extreme: with the Horowitz constants used here, a single 32-bit DRAM read costs roughly 2,782.6× the energy of an INT8 multiply-accumulate (Horowitz 2014). Quantizing from FP32 to INT8 attacks this disparity on both fronts—4× fewer bytes moved and cheaper arithmetic per operation—although realized energy savings depend on the memory hierarchy, kernels, and workload.

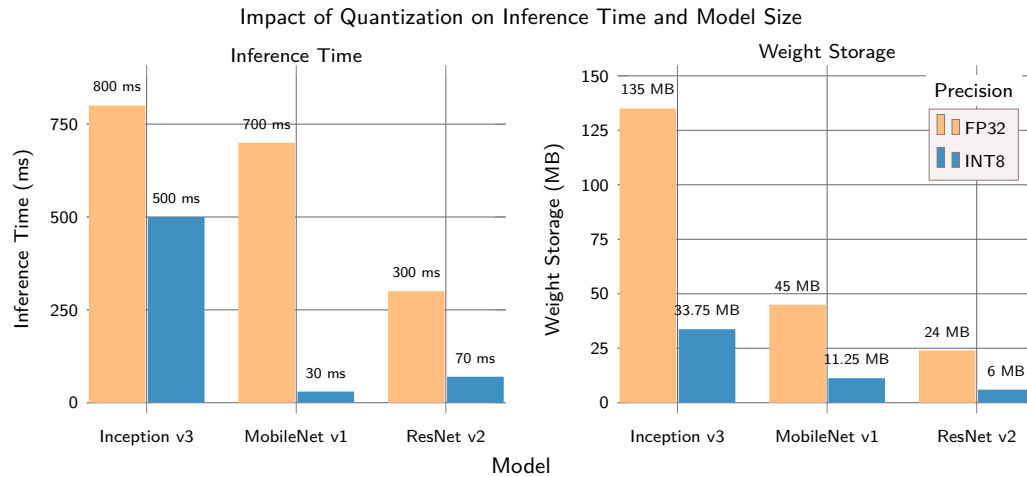


Figure 10.15: Illustrative Quantization Scenario: Paired bars separate FP32 and INT8 alternatives. The right panel shows raw weight storage, where reducing each weight from 32 to 8 bits produces an exact fourfold decrease; the left panel shows illustrative, architecture-dependent latency changes.

FP16

- **Raw weight size:** $8 \times 10^9 \times 2$ bytes (16-bit) = 16 GB
- **Runtime budget:** 16 GB leaves no headroom; a 24 GB-class device is practical.

INT4

- **Raw weight size:** $8 \times 10^9 \times 0.5$ bytes (INT4) = 4 GB
- **Runtime budget:** 4 GB leaves about 4 GB on an 8 GB device for metadata, cache, activations, and workspace.

Beyond storage savings, quantization accelerates arithmetic execution through data-level parallelism in hardware registers. Fixed-width vector execution units (SIMD) and systolic arrays process more operands per cycle when each operand occupies fewer bits.



Napkin Math 10.2: The SIMD multiplier

Problem: A compute-bound layer runs INT8 instead of FP32 on the same processor. Where does the speedup come from?

Mechanism: SIMD execution. A CPU or GPU core processes data in fixed-width vector registers (for example, AVX-512 is 512 bits wide).

Math:

1. **Register width:** 512 bits.
2. **FP32 capacity:** $512/32 = 16$ elements per vector instruction.
3. **INT8 capacity:** $512/8 = 64$ elements per vector instruction.

Result: Switching to INT8 packs 4× more elements into the same register. Throughput Gain = INT8 elements/inst/FP32 elements/inst = $64/16 = 4\times$

Systems insight: Quantization delivers up to 4× speedup on compute-bound layers from vector packing alone, on hardware whose INT8 vector instructions have comparable throughput to FP32, even before considering memory bandwidth savings.

Reducing numerical precision introduces trade-offs, however. Lower-precision formats can cause numerical instability and quantization noise, potentially affecting model accuracy. Figure 10.16 shows an illustrative residual distribution aggregated across values with heterogeneous scales and possible clipping. For an ideal uniform quantizer, within-range rounding error is instead bounded by $[-\Delta/2, \Delta/2]$; errors outside that interval arise from clipping or from aggregating different step sizes. Some architectures, such as large transformer-based NLP models, tolerate quantization well, whereas others may experience significant degradation. Selecting the appropriate numerical precision therefore requires balancing accuracy constraints, hardware support, and efficiency gains.

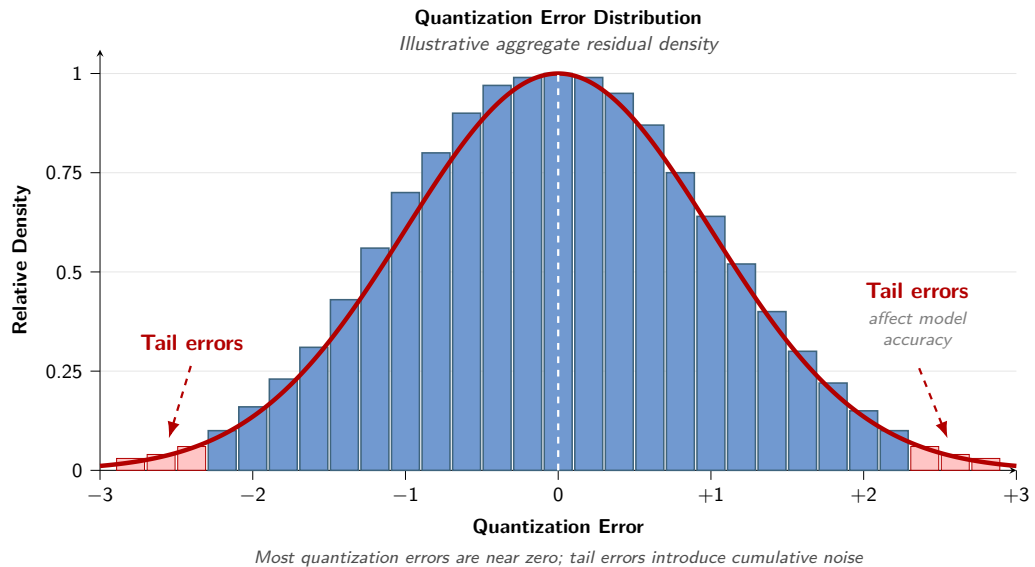


Figure 10.16: Illustrative Aggregate Quantization Residuals: Illustrative aggregate residual distribution across heterogeneous scales and clipped values. Within one unclipped uniform quantizer, rounding error is bounded by $[-\Delta/2, \Delta/2]$; the displayed tails represent clipping or aggregation across different step sizes.

While managing quantization noise preserves algorithmic accuracy, the systems objective of lower precision is wall-clock speedup. The magnitude and physical origin of that speedup depend directly on whether execution is compute bound or memory bound.

Napkin Math 10.3: The quantization speedup (compute bound)

Problem: A compute-bound matrix multiplication (for example, in a transformer multilayer perceptron block) switches from FP16 to INT8. What is the expected speedup?

Math: On modern hardware with dedicated INT8 units:

1. **Integer throughput path:** The reference A100-class accelerator rates its dedicated INT8 matrix units at 624 TOPS against 312 TFLOP/s on the FP16 path (NVIDIA Corporation 2020a; Choquette et al. 2021). The peak throughput increase is that spec ratio: $624 \text{ TOPS} \div 312 \text{ TFLOP/s} \approx 2\times$.
2. **Memory bandwidth:** INT8 weights are half the size, so loading them from memory takes half the time.
3. **Combined effect:** For compute-bound operations, the speedup is primarily from compute throughput: $\sim 2\times$ speedup.

Systems insight: The speedup from quantization depends on the bottleneck. Compute-bound operations (large batch sizes, high arithmetic intensity I) see $\sim 2\times$ from faster INT8 units, where the gain comes from the integer matrix path rather than reduced memory traffic. The

bandwidth-bound case inverts this: halving the bytes moved (FP16 to INT8) yields up to 2×, and larger bit-width reductions scale the gain further, as the next worked example traces.

When an operation is bandwidth bound, speedup tracks the reduction in bytes transferred per token rather than peak arithmetic throughput. In autoregressive token generation with small batch sizes, memory bus traffic dominates latency.

Napkin Math 10.4: The quantization speedup

Problem: A deployment scenario calls for running a 7B-parameter LLM on a device with 16 GB RAM. The weights are FP16 (2 bytes).

Math:

1. **Model size:** $7 \times 10^9 \times 2 \text{ bytes} = 14 \text{ GB}$.
2. **KV cache:** A 4,096-token FP16 KV cache uses two tensors (keys and values) across 32 layers, 32 KV heads, 128 dimensions per head, and 2 bytes per value, requiring approximately 2.1 GB.
3. **Total memory:** $14 \text{ GB} + 2.1 \text{ GB} = 16.1 \text{ GB}$. This exceeds the device capacity before OS and workspace memory.
4. **Bandwidth cost:** In this simplified full-cache-traffic model, loading weights plus the KV cache at 50 GB/s takes 323 ms per token. That is 3.1 tokens/s, too slow for chat.

Fix (INT4):

1. **Quantization:** Convert weights to INT4 (0.5 bytes).
2. **New size:** $7 \times 10^9 \times 0.5 \text{ bytes} = 3.5 \text{ GB}$ of weights, or 5.6 GB including the unchanged FP16 KV cache.
3. **New speed:** Loading that total takes 113 ms. Speed rises to 9 tokens/s.

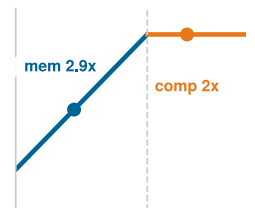
Systems insight: Weight quantization makes this configuration fit and yields a 2.9× speedup in the stated traffic model, below 4× because FP16 KV-cache traffic is unchanged.

10.4.3 Numerical format comparison

Whether quantization accelerates execution depends directly on whether an operator is memory-bandwidth bound or compute bound. For memory-bound kernels, shrinking bit width cuts bytes transferred across the memory bus, directly increasing effective throughput. For compute-bound kernels, speedup requires specialized arithmetic units capable of retiring more low-precision operations per clock cycle. Selecting a numerical format therefore requires balancing dynamic range and hardware execution support against memory traffic, rather than minimizing bit width blindly. Table 10.9 compares the precision formats commonly deployed across accelerator architectures, mapping raw storage footprints, arithmetic throughput gains, and target hardware relative to standard 32-bit floating point (FP32).

Table 10.9: Numerical Precision Formats: Storage ratios reflect raw value width; compute speed and power effects depend on hardware support and workload behavior.

Precision Format	Bit-Width	Storage Reduction (vs. FP32)	Compute Speed (vs. FP32)	Power Effect	Use Cases
FP32	32-bit	Baseline (1×)	Baseline (1×)	Baseline	Training & inference (general-purpose)
FP16	16-bit	2× smaller	Up to 2× peak on selected supported hardware	Hardware- and workload-dependent	Accelerated training, inference (NVIDIA Tensor Cores, TPUs)



Quantization speedup depends on whether memory bandwidth or compute throughput is binding.

Precision Format	Bit-Width	Storage Reduction (vs. FP32)	Compute Speed (vs. FP32)	Power Effect	Use Cases
BF16 (Brain Floating Point)	16-bit	2× smaller	Hardware- and workload-dependent	Hardware- and workload-dependent	Training on TPUs, transformer-based models
TF32 (TensorFloat-32)	19-bit	None (stored as FP32)	Up to 8× peak on NVIDIA Ampere Tensor Cores	Hardware- and workload-dependent	Training on NVIDIA GPUs
FP8 (Floating-Point 8-bit)	8-bit	4× smaller	Hardware- and workload-dependent	Hardware- and workload-dependent	Efficient training/inference (H100, AI accelerators)
INT8 (8-bit Integer)	8-bit	4× smaller	Hardware- and workload-dependent	Hardware- and workload-dependent	Quantized inference (Edge AI, mobile AI, NPU)
INT4 (4-bit Integer)	4-bit	8× smaller	Hardware- and workload-dependent	Hardware- and workload-dependent	Ultra-low-power AI, experimental quantization
Binary/Ternary (1-bit/2-bit)	1–2-bit	16–32× smaller	Hardware- and workload-dependent	Hardware- and workload-dependent	Extreme efficiency (binary/ternary neural networks)

At 16 bits, FP16 and BF16 halve memory storage relative to FP32 while targeting dedicated matrix execution units—such as NVIDIA Tensor Cores and Google TPUs—to accelerate arithmetic throughput. However, their internal bit allocations trade off numerical range against significand precision differently. BF16 retains FP32’s 8-bit exponent, preserving a dynamic range spanning roughly 10^{-38} to 10^{38} while truncating the fraction to 7 bits. In contrast, FP16 allocates 5 bits to the exponent and 10 bits to the fraction, offering greater precision but capping the maximum representable finite value at 65,504, with normal values underflowing below approximately 6.1×10^{-5} (and subnormals down to 6.0×10^{-8}). Consequently, FP16 training typically requires loss scaling and FP32 accumulator registers to prevent gradient underflow and overflow, whereas BF16’s wide exponent range accommodates standard training dynamics without scaling. TensorFloat-32 (TF32) adopts a hybrid compromise: it retains the 8-bit exponent of FP32 and the 10-bit fraction of FP16, executing on Tensor Cores to accelerate matrix multiplication while storing values in standard 32-bit memory. Comparing the bit layouts across FP32, FP16, and BF16 illustrates how each format partitions the sign, exponent, and fraction fields to balance dynamic range against precision (figure 10.17).

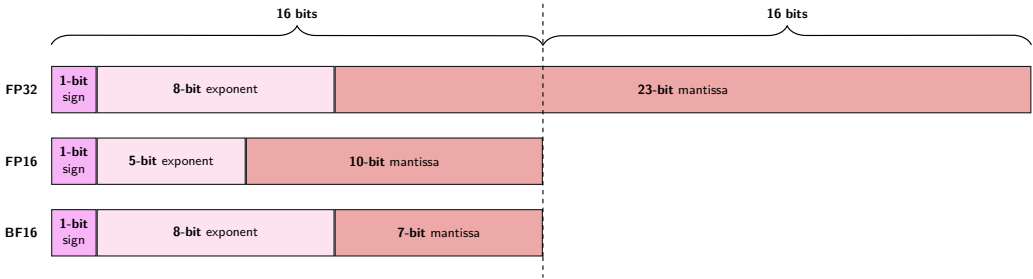


Figure 10.17: Floating-Point Bit Layouts: Bit allocation trades dynamic range against significand precision. FP32 uses 8 exponent and 23 fraction bits. BF16 retains FP32’s 8-bit exponent and therefore a similar numerical range, but reduces the fraction to 7 bits. FP16 uses 5 exponent and 10 fraction bits, providing more significand precision than BF16 but a narrower range that can require loss scaling during training.

For inference workloads, INT8 precision provides a fourfold reduction in raw storage relative to FP32 while maintaining acceptable prediction accuracy. Inference avoids the gradient backpropagation pass, eliminating the small-magnitude, high-dynamic-range updates that cause underflow during training. Consequently, activations and weights map cleanly onto signed 8-bit integers ($[-128, 127]$) using uniform affine or symmetric quantization scales. In bandwidth-limited inference

regimes—such as autoregressive language model token generation—this $4\times$ footprint reduction relieves memory bus contention by cutting DRAM and cache traffic. On compute-bound operators, dedicated INT8 vector and tensor units provide higher arithmetic operation density per clock cycle and per unit of silicon area than equivalent floating-point units.

Binary and ternary networks represent the theoretical limit of precision reduction, constraining weights and activations to single bits ($\{-1, +1\}$) or ternary sets ($\{-1, 0, +1\}$) to replace floating-point multipliers with bitwise XNOR and popcount logic. However, because standard memory architectures address bytes rather than individual bits, sub-byte representations require packing values into 8-bit or 32-bit words, introducing bitwise shift and mask overhead unless custom hardware datapaths support them directly. Severe discretization also restricts gradient propagation during training, making task accuracy sensitive to surrogate gradient formulation and training stabilization. The keyword-spotting lighthouse (section 6.3.5) illustrates an operating regime where extreme compression is required to satisfy the strict ~ 512 KB SRAM budget. INT8 provides a baseline target; INT4, binary, or ternary representations trade additional numerical dynamic range to meet stricter memory and power bounds.

Arithmetic energy differences complete the physical argument for quantization. Low-precision representations relieve energy consumption along two distinct physical paths: reducing memory movement across the bus and lowering the switching energy required per arithmetic operation. A full INT8 multiply-accumulate, for example, uses roughly $20\times$ less energy than its FP32 equivalent. Accelerators equipped with native low-precision units—such as NVIDIA Tensor Cores, Google TPUs, or mobile NPUs—compound these arithmetic savings with reduced memory traffic. In contrast, general-purpose processors lacking native low-precision execution pipelines lose much of this benefit to the runtime overhead of bit unpacking, format conversion, and scalar fallback. Precision is therefore a model-hardware decision, not a software configuration flag.

10.4.4 Precision reduction strategies

Reducing numerical bit width relieves memory traffic and arithmetic energy, but naive truncation introduces quantization noise that degrades model accuracy. Deploying low-precision representations requires structured strategies that control where and how bit widths change across the computational graph.

Three approaches form a complexity ladder. Post-training quantization (PTQ) reduces precision after training, requiring no retraining and minimal engineering effort. Quantization-aware training (QAT) incorporates quantization effects into the training loop, enabling models to adapt to lower precision and retain higher accuracy. Mixed-precision approaches assign different bit widths to different layers, matching numerical precision to each operator’s sensitivity. Figure 10.18 maps quantization techniques into three progressive tiers based on implementation complexity, resource requirements, and target use cases.

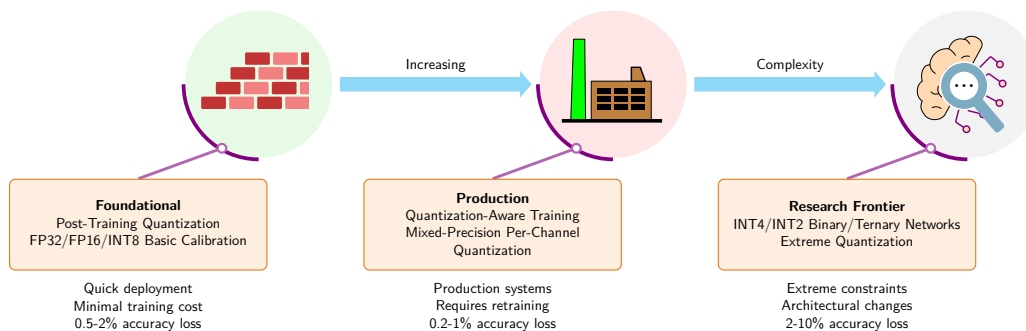


Figure 10.18: Quantization Complexity Roadmap: The tiers order techniques by typical engineering and validation effort, not by guaranteed accuracy. PTQ provides a low-cost starting point; QAT and mixed precision add training or kernel control; INT4, binary, and ternary methods require increasingly model- and hardware-specific validation. A deployment may stop at any tier once it meets measured quality, latency, memory, and energy criteria, and a more complex tier is not automatically better.

The roadmap is a deployment-ordering device rather than a taxonomy of numerical formats. PTQ belongs first because it changes representation with minimal training cost; QAT and mixed precision

move into the production tier because they require training-loop or kernel support; INT4, binary, and ternary methods sit at the frontier because the accuracy and hardware assumptions become architecture-specific.

10.4.4.1 Post-training quantization

Post-training quantization (PTQ) reduces numerical precision after training, converting selected weights or activations from FP32 to lower-precision representations such as INT8 without full retraining (Choukroun et al. 2019). It reliably reduces raw value payload; latency and energy improve only when the exported graph maps to efficient low-precision kernels (Wu et al. 2020).

PTQ's key advantage is low computational cost: it requires no retraining and usually no labeled training set, although activation calibration typically needs a small representative calibration dataset. However, reducing precision introduces quantization error that can degrade accuracy, especially for tasks requiring fine-grained numerical precision. Machine learning frameworks such as TensorFlow Lite, the Open Neural Network Exchange runtime, and PyTorch provide built-in PTQ support.

The core mechanism of PTQ is **uniform quantization**, which maps floating-point values to discrete integer levels using a consistent scaling factor. Because the interval between each quantized value is constant, uniform quantization simplifies implementation and enables efficient hardware execution. For the symmetric form shown here, s is chosen from the maximum absolute value so that the resulting integers stay within the target range. The quantized value q is computed as:

$$q = \text{round}\left(\frac{x}{s}\right)$$

where:

- q is the quantized integer representation,
- x is the original floating-point value,
- s is a scaling factor that maps the floating-point range to the available integer range.

Listing 10.2 demonstrates uniform quantization from FP32 to INT8, reducing the raw value payload from 32 to 8 bits per weight while measuring the resulting quantization error; scale metadata and packing overhead are excluded. Once a model is quantized, the runtime can use integer arithmetic where supported (Gholami et al. 2022).

Listing 10.2: Uniform Quantization: Converts FP32 weights to INT8, reducing raw per-weight payload from 32 to 8 bits while measuring quantization error; scale metadata and packing overhead are excluded.

```
import torch

# Original FP32 weights
weights_fp32 = torch.tensor(
    [0.127, -0.084, 0.392, -0.203], dtype=torch.float32
)
print(f"Original FP32: {weights_fp32}")
print(f"Memory per weight: 32 bits")

# Simple uniform quantization to INT8 (-128 to 127)
# Step 1: Find scale factor
max_val = weights_fp32.abs().max()
scale = max_val / 127 # 127 is max positive INT8 value

# Step 2: Quantize using our formula q = round(x/s)
weights_int8 = torch.round(weights_fp32 / scale).to(torch.int8)
print(f"Quantized INT8: {weights_int8}")
print(f"Memory per weight: 8 bits (reduced from 32)")

# Step 3: Dequantize to verify
weights_dequantized = weights_int8.float() * scale
print(f"Dequantized: {weights_dequantized}")
print(
    f"Quantization error: "
    f"{(weights_fp32 - weights_dequantized).abs().mean():.6f}"
)
```

An alternative, nonuniform quantization, assigns finer-grained precision to numerical ranges that are more densely populated, which can preserve accuracy for models whose weight distributions concentrate around specific values. Nonuniform schemes require more complex calibration and are less common in production, but they can be effective for models particularly sensitive to precision changes.

PTQ is a useful first experiment for many vision, language, and speech models, but sensitivity varies by architecture, layer, and distribution. When a uniform PTQ configuration misses the quality target, the next options include finer granularity, mixed precision, weight-only methods, better calibration, or QAT.

Calibration Quantization calibration estimates the real-valued clipping range $[\alpha, \beta]$ for each tensor. An ill-fitting range either saturates critical values through clipping or squanders available integer levels on sparse outliers. Calibration minimizes representation error over the sampled distribution, although it does not guarantee bounds on out-of-distribution deployment inputs.

Because activation tensors vary with input data, profiling requires running an unlabelled calibration dataset through the forward pass to capture dynamic ranges. Figure 10.19 outlines this offline conversion workflow, while algorithm 10.1 formalizes the layer-by-layer calibration procedure where activation observers record value distributions to compute scale and zero-point metadata.

Algorithm 10.1 Post-training quantization calibration

Require: pretrained model f_θ ; representative calibration set C ; bit width b ; granularity (layer/group/channel); method (max, entropy, percentile)

Ensure: quantized weights and per-group range metadata $\{(\alpha_g, \beta_g, s_g, z_g)\}$

- 1: attach observers to weights and selected activation tensors, at the chosen granularity
 - 2: **for** each batch in C **do** $\triangleright$ *calibration pass*
 - 3: run f_θ in inference; record per-group value distributions
 - 4: **end for**
 - 5: **for** each quantization group g **do**
 - 6: select clipping range $[\alpha_g, \beta_g]$ by the chosen method
 - 7: derive scale s_g , zero-point z_g mapping $[\alpha_g, \beta_g]$ into the b -bit integer range
 - 8: **end for**
 - 9: quantize static weights; export activation range metadata for inference
 - 10: if accuracy falls below the production threshold, widen ranges, change granularity, or move to QAT
-

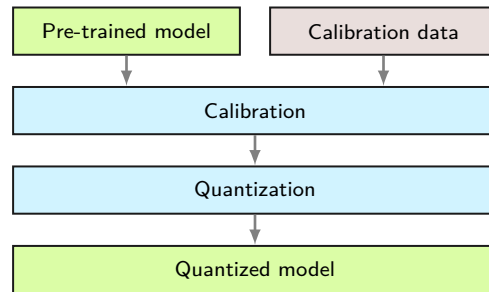


Figure 10.19: Post-Training Quantization Pipeline: PTQ converts a trained FP32 model directly to INT8 without retraining. Representative calibration data passes through the model to record activation ranges $[\alpha, \beta]$, computing scale factor S and zero-point Z parameters to quantize parameters before deployment.

The single calibration pass over C avoids retraining, and FP32-to-INT8 weight quantization cuts raw stored weight values by $4\times$ before scale, zero-point, and packing metadata. The price is that the static activation ranges fixed in the loop risk saturation or wasted integer levels when the calibration data misses deployment tails. The quantization step then converts model parameters to the lower-precision format, producing the final quantized model.

Mapping an unnecessarily wide real-valued interval to the 256 discrete bins of an 8-bit integer wastes numerical resolution on empty range, whereas clipping too aggressively truncates signal. Three calibration methods balance this clipping-versus-resolution trade-off: **max** sets bounds to the absolute extremum (preserving extreme values at the cost of vulnerability to outliers), **entropy** selects bounds by minimizing the Kullback–Leibler (KL) divergence between original and quantized distributions, and **percentile** clips a fixed tail fraction (such as 99.9 percent). Figure 10.20 illustrates why outlier handling is critical: in ResNet-50 activations, long tails stretch the dynamic range and starve the dense cluster near zero of available discrete levels.

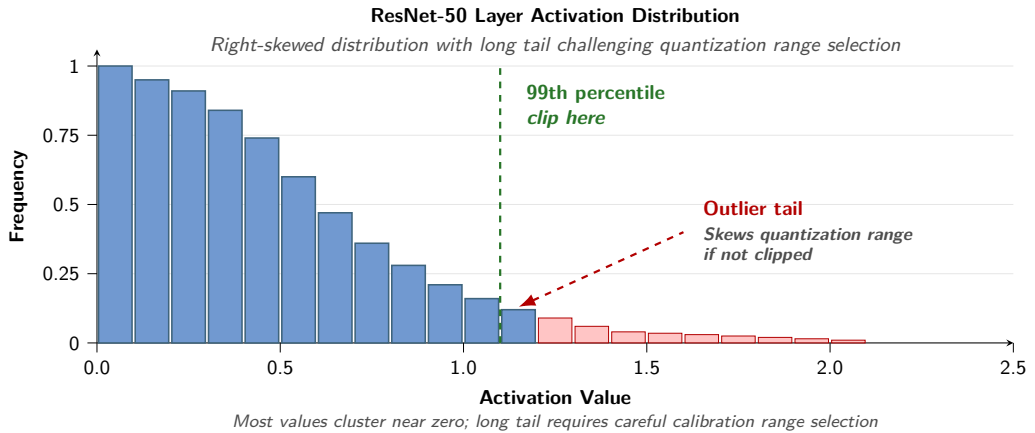


Figure 10.20: Activation Distribution: ResNet-50 layer activations exhibit a long tail, with outlier values that can lead to inefficient precision use if not handled carefully. Source: (Wu et al. 2020).

Calibration ranges can be **symmetric** (a zero-centered range, typically with zero-point zero) or **asymmetric** (one scale with a generally nonzero zero-point, useful when distributions are skewed). The choice of method and range significantly affects quantized model accuracy.

Tuning quantization ranges Once calibration identifies clipping boundaries $[\alpha, \beta]$, the range must be mapped onto integer endpoints. Figure 10.21 contrasts the two primary mapping strategies: symmetric calibration, which constrains real zero to integer zero, and asymmetric calibration, which introduces an explicit zero-point offset to accommodate skewed distributions.

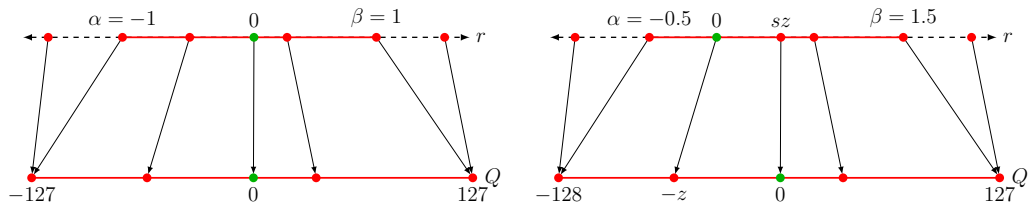


Figure 10.21: Calibration Range Selection: Symmetric vs. asymmetric affine quantization mappings. Symmetric quantization (left) forces real zero to map to integer zero, constraining range selection to $[-\alpha, \alpha]$ at the expense of wasted dynamic range for skewed activation distributions; asymmetric quantization (right) uses an explicit zero-point offset Z to map non-centered ranges $[\alpha, \beta]$ efficiently.

Symmetric quantization simplifies integer matrix multiplication kernels because the zero-point is zero ($z = 0$), eliminating cross-term additions during hardware accumulation. However, for activations following asymmetric distributions (such as post-ReLU feature maps where all values are non-negative), symmetric quantization wastes roughly half of the integer dynamic range on unused negative values. Asymmetric quantization recovers this dynamic range by mapping $[\alpha, \beta]$ to $[0, 2^b - 1]$, but requires runtime hardware to compute offset corrections ($z \sum W$) during multiply-accumulate operations.

Granularity Quantization granularity governs how broadly scale factors and zero-points are shared across a tensor—balancing representation resolution against metadata overhead and execution complexity. In convolutional filters or transformer projection matrices, weight magnitudes often vary widely across individual channels. Sharing a single scale factor across an entire tensor risks collapsing the dynamic range of small-magnitude channels to zero; conversely, assigning scales per channel preserves local variance at the expense of tracking additional scale vectors during matrix multiplication.

Napkin Math 10.5: Calculating scale and zero-point

The affine quantization formula: An affine map sends a floating-point range $[\alpha, \beta]$ that includes zero to an integer range $[0, 2^b - 1]$ (for example, UINT8) via the following construction.

Analysis: Affine quantization establishes a linear mapping $x \approx s(x_q - z)$, where s is the **scale** (step size) and z is the **zero-point** (integer value corresponding to real zero). The mapping involves three computational steps:

1. **Calculate scale** (s): Divide the real range by the integer range using equation 10.1.

$$s = \frac{\beta - \alpha}{2^b - 1} \quad (10.1)$$

2. **Calculate zero-point** (z): Shift the range so that real zero maps to an integer using equation 10.2.

$$z = \text{round}\left(\frac{-\alpha}{s}\right) \quad (10.2)$$

3. **Quantize** ($x \rightarrow x_q$) using equation 10.3:

$$x_q = \text{clamp}\left(\text{round}\left(\frac{x}{s} + z\right), 0, 2^b - 1\right) \quad (10.3)$$

Scenario: Suppose the activations range from $\alpha = -1$ to $\beta = 3$, and the target precision is UINT8 ($b = 8$).

- **Range:** $\beta - \alpha = 4$.
- **Steps:** $2^8 - 1 = 255$.
- **Scale:** $s = \frac{4}{255} \approx 0.0157$.
- **Zero-point:** $z = \text{round}(-(\alpha)/s) = \text{round}(-(-1)/\frac{4}{255}) = \text{round}(63.75) = 64$.

Systems insight: The real value 0.0 is represented by the integer 64, which ensures that zero-padding (common in CNNs) is represented exactly, preventing “quantization drift” where padding introduces nonzero noise.

When weight channels exhibit divergent standard deviations, sharing a single clipping threshold across an entire layer either collapses small-magnitude channels or clips wider channels. Figure 10.22 visualizes this filter-to-filter variance across convolutional layers, contrasting a shared layerwise boundary with independent per-channel thresholds.

Quantization granularity trades calibration and metadata overhead against representation error. More local ranges can preserve information when channels differ, but they do not improve accuracy automatically and may lack an efficient kernel path. Each additional scale also has to be stored, loaded, and applied at the granularity supported by the backend. Table 10.10 summarizes four common levels, from one shared layer range to local ranges within a filter.

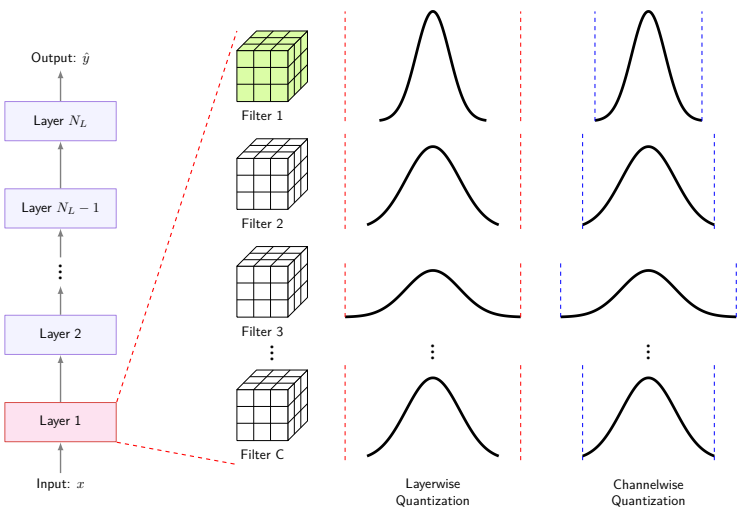


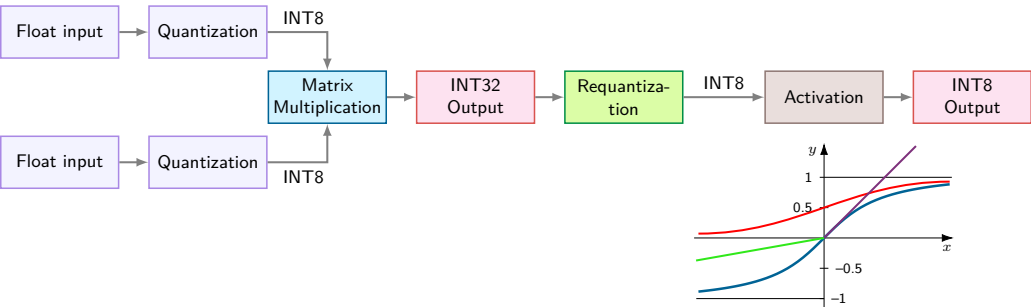
Figure 10.22: Quantization Range Variation: Different convolutional filters can have different weight ranges, motivating per-filter quantization to reduce quantization error. The differing clipping scales illustrate the choice between one layerwise range and separate channelwise ranges. Source: (Gholami et al. 2022).

Table 10.10: Quantization Granularity Levels: Granularity trades calibration overhead for accuracy, from a single per-layer range to per-filter sub-ranges.

Level	Range sharing	Trade-off
Layerwise	One range per layer	Simple but suboptimal when filter ranges vary widely
Groupwise	Filters grouped with shared ranges	Used in Q-BERT (Shen et al. 2020) for transformer attention layers
Channelwise	One range per filter	Common default; balances quantization error against scale metadata and implementation overhead
Sub-channelwise	Ranges within each filter	More local ranges can reduce quantization error but increase metadata and implementation overhead

Channelwise quantization preserves accuracy over layerwise quantization when channel dynamic ranges differ, at the cost of storing additional scale vectors and executing per-channel rescaling in the kernel pipeline. Beyond scale granularity, execution efficiency depends on the distinction between static weights and dynamic activations, which impose fundamentally different memory and latency constraints on the deployment system.

Weights vs. activations Executing low-precision inference requires transforming floating-point inputs before kernel execution and requantizing intermediate accumulator outputs. During integer matrix multiplication, multiplying two 8-bit integers produces a 16-bit product. Summing these products across an inner dimension of length K causes bit-growth that quickly overflows 8-bit registers; hardware therefore accumulates into 32-bit integer registers (INT32). Before feeding the result into subsequent layers, requantization rescales the 32-bit accumulated sum by the layer’s effective scale factor ($s_{\text{eff}} = \frac{s_W \cdot s_X}{s_Y}$) and clamps the values back into the 8-bit integer range. Figure 10.23 illustrates this end-to-end execution pipeline, showing how floating-point inputs pass through INT8 quantization before integer SIMD or Tensor Core matrix multiplication, accumulate in 32-bit registers, and requantize back to INT8 before the nonlinear activation.



task degradation. Packed low-bit weights can reduce the memory traffic required during token generation when the runtime provides a matching kernel (Lin et al. 2024).

Static vs. dynamic quantization Beyond the shape and granularity of the clipping range, system design dictates *when* activation ranges are computed. Activation quantization divides into two primary execution models: static quantization and dynamic quantization.

In **static quantization**, the clipping range is precalculated during offline calibration and remains fixed during inference. This approach introduces zero runtime range-estimation overhead: inputs are scaled by fixed factors and directly fed to integer execution units. However, static ranges risk saturation or underflow whenever production inputs deviate from the calibration distribution (Jacob et al. 2018; Yao et al. 2021).

Dynamic quantization instead calculates scaling factors and zero-points dynamically from runtime activation tensors. Adapting the clipping window to each specific input eliminates saturation from distributional shifts. The trade-off is runtime memory and latency overhead: the hardware must perform reduction passes over intermediate activation tensors to extract minimum and maximum values before integer arithmetic can begin. On memory-bandwidth-bound operators, this reduction pass can erode the execution advantage of lower precision.

These timing and granularity decisions interact with the broader choice of quantization methodology. Table 10.11 compares post-training quantization, quantization-aware training, and dynamic quantization, each offering distinct strengths and trade-offs for different deployment scenarios.

Table 10.11: Quantization Method Comparison: The choice turns on three binding questions, namely engineering budget, accuracy threshold, and input variability, rather than on a feature checklist.

Method	Engineering effort	Accuracy preservation	Input adaptability	Reach for it when
Post-training quantization (PTQ)	Low: no retraining	Model-dependent; parameters do not adapt	Fixed calibration range	a calibrated model meets the measured accuracy and performance targets
Quantization-aware training (QAT)	High: additional training with quantization simulation	Often stronger when PTQ falls short	Fixed calibration range	PTQ misses the accuracy target and the training budget allows
Dynamic quantization	Moderate: ranges recomputed at runtime	Model-dependent; range fits each input	Per-input range	activation ranges vary widely across inputs and runtime overhead is acceptable

PTQ serves as the low-effort baseline; QAT spends training cycles when PTQ degrades task accuracy, while dynamic quantization expends runtime memory traffic and compute to track input-dependent dynamic ranges.



Checkpoint 10.3: Calibration and range choices

Quantization succeeds only when the range policy matches deployment data and runtime constraints.

Range design

- ☐ **Calibration data:** explain how an unrepresentative calibration set creates clipping or wastes integer levels.
- ☐ **Granularity:** compare per-tensor and per-channel ranges by the error and metadata trade-off each creates.
- ☐ **Static vs. dynamic ranges:** select a range policy when activation distributions shift across inputs and runtime overhead is constrained.

System estimate

- ☐ **Quantization payoff:** for a 7B-parameter model served from HBM, estimate FP16 versus INT4 weight memory and the rough decode-throughput gain for memory-bandwidth-bound inference.

PTQ in practice Post-training quantization balances deployment turnaround against accuracy control. PTQ requires no gradient backpropagation or retraining, establishing it as the standard initial step in deployment optimization pipelines. Whether offline calibration is sufficient depends on the architecture, calibration data distribution, numerical quantizer design, and target hardware constraints.

The primary limitation is that PTQ freezes model parameters, preventing compensation for accuracy loss. When a quantized model falls below the production threshold, mitigations include recalibrating, refining scale granularity, adopting mixed precision, or escalating to quantization-aware training. QAT integrates precision constraints directly into the training graph using a **straight-through estimator (STE)** to pass gradients through nondifferentiable rounding operators. This enables weights to adapt to low-bit representations during retraining, recovering accuracy at the cost of full training compute (O_{train}) and hyperparameter tuning.

10.4.4.2 Quantization-aware training

Quantization-aware training incorporates numerical rounding noise directly into the optimization graph. Figure 10.24 diagrams this training workflow: fake-quantization nodes simulate low-precision clipping and rounding errors during the forward pass, while straight-through estimators route gradients back to floating-point master weights during backpropagation.

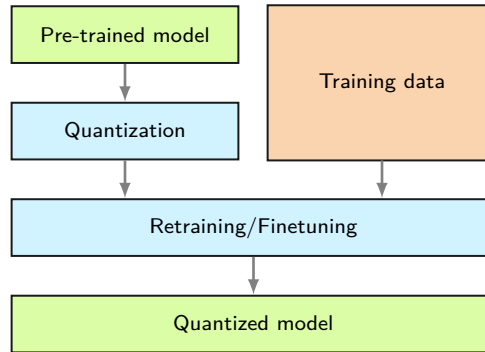


Figure 10.24: Quantization-Aware Training Pipeline: QAT inserts fake-quantization nodes into the forward graph to simulate low-precision clipping and rounding errors during training. Backpropagation updates floating-point master weights using straight-through estimators (STE), allowing network parameters to adapt to numerical noise before exporting fixed integer scales for inference.

Rather than training from scratch or cold-starting scale parameters, production workflows frequently initialize QAT using scale factors derived during an initial PTQ calibration pass. Figure 10.25 illustrates this two-stage progression, where offline calibration establishes baseline range metadata before fine-tuning adjusts master weights to compensate for quantization noise.

Training mathematics During forward propagation, weights and activations are quantized and dequantized to mimic reduced precision. Let x be a full-precision value, s the scaling factor that maps floating-point values into a lower-precision range, and q the simulated quantized value. This process is typically represented as:

$$q = \text{round} \left(\frac{x}{s} \right) \times s$$

where q represents the simulated quantized value, x denotes the full-precision weight or activation, and s is the scaling factor mapping floating-point values to lower-precision integers.

Although the forward pass executes with simulated quantization, computing gradients requires backpropagating through the rounding operator $\text{round}(\cdot)$. Because rounding is a piecewise-constant step function, its mathematical derivative is zero almost everywhere and undefined at step transitions, which would zero out gradients and halt gradient descent. The Straight-Through Estimator (STE) resolves this bottleneck by substituting the identity function for the rounding operator during backpropagation (Bengio, Léonard, et al. 2013):¹⁹

Integrating quantization effects during training lets the model adapt its weights and activation ranges to simulated low-precision error. QAT can recover quality that a post-training route loses,

¹⁹ | **Straight-through estimator (STE):** Proposed by Bengio, Léonard, et al. (2013), the STE substitutes the identity function for the true gradient of rounding, which is zero almost everywhere because rounding is piecewise constant. The common identity STE copies the upstream gradient through the quantizer. It is a biased heuristic rather than the true derivative.

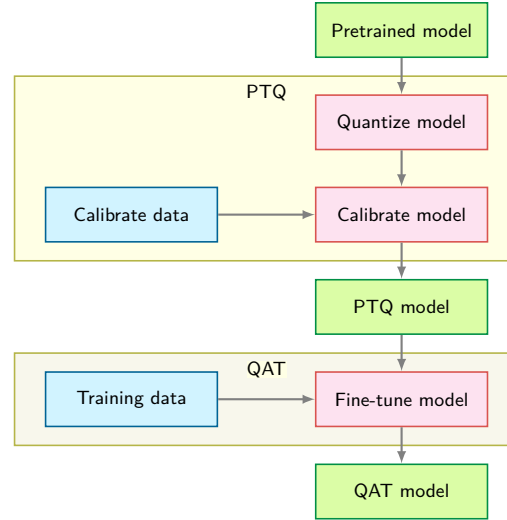


Figure 10.25: PTQ-to-QAT Pipeline: PTQ calibration can initialize ranges and scales before QAT fine-tunes the pretrained weights with simulated quantization. Calibration data feeds the PTQ stage, and training data feeds QAT.

particularly at aggressive bit widths, but the advantage depends on the model, quantizer, calibration, and training procedure (Krishnamoorthi 2018).

Fake quantization nodes and implementation QAT implementation relies on fake quantization operations that simulate quantization during forward propagation while maintaining full precision for gradient computation. These operations insert quantize-dequantize pairs into the computational graph, creating a training-time simulation of inference-time behavior.

A fake quantization node must preserve the same errors the inference runtime will see, so its three operations model the deployment path during training:

1. **Quantization:** Map floating-point value to discrete quantization level
2. **Clipping:** Enforce range constraints based on bit width
3. **Dequantization:** Convert back to floating-point for subsequent operations

Mathematically, for symmetric quantization with bit width b , given a floating-point input value x :

$$q_{\text{level}} = \text{clip} \left(\text{round} \left(\frac{x}{s} \right), -2^{b-1}, 2^{b-1} - 1 \right)$$

$$x_{\text{fake}} = q_{\text{level}} \times s$$

where $s = \frac{\max(|x|)}{2^{b-1}-1}$ is the scale factor computed from the input distribution, and x_{fake} represents the fake-quantized output that mimics INT8 values but remains in floating-point format.

For asymmetric quantization supporting unsigned integers, assume the observed range has been extended to include zero:

$$s = \frac{\max(x) - \min(x)}{2^b - 1}$$

$$z = \text{round} \left(-\frac{\min(x)}{s} \right)$$

$$q_{\text{level}} = \text{clip} \left(\text{round} \left(\frac{x}{s} + z \right), 0, 2^b - 1 \right)$$

$$x_{\text{fake}} = (q_{\text{level}} - z) \times s$$

where z is the zero-point offset enabling asymmetric range representation.

Listing 10.3 shows the QAT forward path: fake quantization is applied to inputs and weights before convolution, simulating INT8 numerics while retaining floating-point tensors for training.

Listing 10.3: QAT Convolution Forward Pass: Fake-quantizes activations and weights before convolution to simulate deployment numerics during training.

```
# Forward pass with fake quantization
def qat_conv_forward(x, weight):
    # Fake quantize input activations
    x_scale = compute_scale(x, bits=8, symmetric=False)
    x_zero = compute_zero_point(x, x_scale, bits=8)
    x_quant = fake_quantize(x, x_scale, x_zero, bits=8)

    # Fake quantize weights (typically symmetric)
    w_scale = compute_scale(weight, bits=8, symmetric=True)
    w_quant = fake_quantize(weight, w_scale, zero=0, bits=8)

    # Convolution with fake-quantized values
    output = conv2d(x_quant, w_quant)
    return output
```

During backpropagation, fake quantization nodes approximate gradients through a clipping-aware straight-through estimator:

$$\frac{\partial x_{\text{fake}}}{\partial x} = \begin{cases} 1 & \text{if } x \in [x_{\min}, x_{\max}] \\ 0 & \text{otherwise} \end{cases}$$

Within the clipping range $[x_{\min}, x_{\max}]$, the estimator treats quantization as an identity mapping, passing the upstream gradient $\frac{\partial \mathcal{L}}{\partial x_{\text{fake}}}$ unchanged to master weight x . Beyond clipping boundaries, the derivative evaluates to zero, blocking gradients from saturated outliers that would otherwise destabilize optimizer updates.

In practice, frameworks like PyTorch and TensorFlow implement fake quantization as custom autograd operators whose forward pass performs the quantize-dequantize round trip while the backward pass applies an STE. Scale handling depends on the method: observers may track distributions and later freeze, or scales may be learned. Batch normalization also requires deployment-consistent handling: workflows fold batch normalization into preceding convolution weights ($W_{\text{fused}} = \frac{\gamma}{\sigma} W$) and freeze its running statistics before quantization export, ensuring that training-time fake quantization scales match the static fused kernel parameters executed during inference.

QAT trade-offs QAT's²⁰ primary advantage is additional control over quality at low precision. By incorporating simulated quantization noise during training or fine-tuning, the model can adapt to the deployed numerical path. Processors with dedicated integer units can then exploit INT8 arithmetic for faster or lower-energy inference when the exported operators are supported (Wu et al. 2020; Gholami et al. 2022). The benefit must be measured against PTQ for the same model and target, including the extra training cost and any difference in calibration, graph coverage, or kernel selection.

The cost is additional engineering complexity. QAT inserts simulated quantization into training, so teams must validate quantization schemes, scale behavior, and accuracy recovery on the target model. This overhead can make QAT less practical for very large models when training budgets are already constrained. Post-training routes provide a complementary path that eliminates this retraining overhead when training budgets are constrained (Choukroun et al. 2019).

In practice, the choice between PTQ and QAT follows a simple decision rule. Start with PTQ and measure accuracy on the validation set. If accuracy meets the production threshold, the engineering cost of QAT is not justified. If PTQ falls short, invest in QAT to recover the gap. A hybrid approach, starting with PTQ calibration and applying QAT fine-tuning only for accuracy-critical layers, often provides a useful balance.

PTQ and QAT commonly target 8-bit or 4-bit precision, but retained accuracy depends on the model, task, and quantization method. Some deployment scenarios, however, demand even more aggressive compression, pushing precision to the absolute limits of what neural networks can tolerate.

²⁰ | **Quantization-aware training (QAT):** QAT simulates low-precision behavior during training or fine-tuning so weight updates can adapt to clipping and rounding error. BERT studies report that training-aware or Hessian-aware methods can remain closer to full-precision General Language Understanding Evaluation (GLUE) baselines than simpler post-training routes in their evaluated settings (Zafir et al. 2019; Shen et al. 2020). The exact gap depends on model, task, quantizer, bit width, training budget, and evaluation distribution; a closer benchmark result is evidence for trying QAT, not a guarantee that its added training cost or exported runtime will meet a production target.

10.4.5 Extreme quantization

Extreme quantization pushes numerical precision down to 1 or 2 bits per value for deployment environments where even INT8 or INT4 cannot satisfy memory or energy budgets (Courbariaux et al. 2015; Zhu et al. 2017). Binarization constrains weights and activations to two values, typically encoded as $\{-1, +1\}$. In digital logic, mapping -1 to binary 0 and $+1$ to binary 1 renders scalar multiplication isomorphic to the bitwise XNOR operation. As a result, expensive floating-point or integer multiply-accumulate (MAC) units are replaced by bitwise logic and population count (popcount) operations (Rastegari et al. 2016). A 64-bit register packs 64 binary values. A single XNOR instruction executes 64 multiplications in parallel within one clock cycle, and a subsequent hardware popcount instruction counts the matching bits to accumulate the dot product ($2 \times \text{popcount}(\mathbf{w} \text{ XNOR } \mathbf{a}) - 64$). This eliminates hardware multipliers entirely. However, 1-bit representations severely constrain model expressiveness, leading to steep accuracy degradation on complex perceptual and language tasks (Hubara et al. 2018).

Ternarization mitigates this capacity loss by allowing three values: $\{-1, 0, +1\}$ (Zhu et al. 2017). The zero state introduces sparsity and simplifies arithmetic: multiplying an activation by $+1$ requires an addition, multiplying by -1 requires a subtraction, and multiplying by 0 is skipped entirely. This eliminates general-purpose multipliers, reducing the arithmetic unit to a conditional adder-subtractor accumulator. However, ternary values cannot be packed at one bit per value. They require at least two bits per weight in standard byte-aligned memory (wasting one of four possible 2-bit states) or multi-trit encoding (such as packing five trits into a single byte, since $3^5 = 243 < 256$) that introduces runtime decoding overhead.

Both binary and ternary quantization introduce step functions whose derivative is zero almost everywhere ($\frac{d}{dx} \text{sign}(x) = 0$ for $x \neq 0$), causing backpropagation gradients to vanish. To train these representations, quantization-aware training uses gradient approximation methods, primarily the straight-through estimator (STE) (Bengio, Léonard, et al. 2013; Choi et al. 2018). The STE substitutes the non-differentiable quantizer with an identity or clipping operator during the backward pass, propagating gradients directly to full-precision latent weights that accumulate small optimizer steps until crossing the quantization thresholds.

10.4.5.1 Challenges and limitations

Translating sub-byte precision into actual wall-clock speedups is constrained by standard processor architectures. Commodity CPUs and GPUs feature ALUs and tensor pipelines engineered for 8-bit, 16-bit, and 32-bit registers; they lack native single-bit or two-bit matrix multiplication instructions. Executing sub-byte operands on these general-purpose architectures requires bit-shifting, masking, and unpacking routines. The instruction overhead of this bit manipulation frequently offsets memory-bandwidth savings unless workloads run on custom bit-serial accelerators, field-programmable gate arrays (FPGAs), or specialized hardware with native bitwise execution units (Umuroglu et al. 2017).

Model quality presents an equally severe barrier. Extreme quantization dramatically contracts the representational capacity of a network. While low-entropy workloads such as KWS on a microcontroller SRAM budget may tolerate binary or ternary weights, complex perceptual and generative models suffer steep accuracy degradation. Binary and ternary quantization become viable candidates only when higher-precision alternatives such as INT8 or INT4 fail to satisfy physical memory or energy budgets, and when the deployment hardware can execute bit-packed operations natively (Rastegari et al. 2016; Hubara et al. 2018; Zhu et al. 2017; Umuroglu et al. 2017). Production deployment requires evaluating binary and ternary baselines against INT8, INT4, structured pruning, and compact architectural redesigns on the identical workload, measuring total binary artifact size, end-to-end latency, energy per inference, and task accuracy on target hardware.

The governing engineering check is whether bit width, hardware arithmetic paths, and calibration or training choices directly satisfy physical deployment constraints.



Checkpoint 10.4: Quantization and precision checkpoint

Quantization design review criteria:

- ❑ **INT8 memory vs. energy:** can you explain why INT8 quantization provides roughly $4\times$ memory reduction but potentially more than $4\times$ energy reduction? Consider the energy cost of floating-point vs. integer arithmetic units.
- ❑ **QAT vs. PTQ:** do you understand the key advantage of quantization-aware training (QAT) over post-training quantization (PTQ), and when the extra training cost is justified?
- ❑ **Bit-width speedup:** can you explain why weight quantization provides nearly linear speedup with bit-width reduction for memory-bandwidth-bound LLM inference? Think about which resource is the bottleneck during autoregressive generation.

The first two optimization dimensions answer different questions: structural optimization (pruning, distillation, NAS) determines *what* to compute, and precision optimization (quantization) determines *how precisely* to represent and execute it. Their paper savings must be stated in the resource they actually change—nonzero count, FLOPs, or raw bytes—before those reductions are translated into latency or energy.

In production environments, a persistent gap emerges between theoretical projections and measured runtime speedups. This illustrative scenario combines a 50 percent pruning factor with INT8 width reduction for a back-of-envelope $8\times$ target, then assumes that only $1.5\times$ is realized. That is roughly 18.8 percent of the paper target. The scenario illustrates why *theoretical compression does not imply proportional speedup*. Optimization must extend beyond the model graph to how operations execute across the physical memory hierarchy.

The gap arises from several architectural and runtime mismatches. Sparse matrices stored in dense formats waste memory bandwidth transferring zeros—hardware cannot skip unindexed zeros. Operations that could execute concurrently stall sequentially when scheduling or runtime dependencies prevent overlap. A uniform execution graph assigns every input identical computational depth regardless of difficulty. Bridging the divide between paper reduction and measured hardware speedup is the domain of the third optimization dimension: **Architectural efficiency**. This dimension evaluates whether structural and precision optimizations translate into real execution gains once computational graphs interact with physical execution units.

10.5 Architectural Efficiency

Architectural efficiency starts from the execution trace. The preceding section quantified the gap between paper and measured speedup; a profiler explains where it goes: sparse tensors may still move through dense kernels, reduced-precision operators may require conversions the hardware cannot hide, and small layers may spend more time launching kernels and moving intermediates than doing arithmetic. The model has become smaller on paper, but the execution trace still asks the machine to perform an inefficient sequence of memory accesses and operations.

Where representation optimization determines *what* computations to perform and precision optimization determines *how precisely* to compute them, architectural efficiency determines *how* those computations fit the machine. Measured bottlenecks determine which architectural response is useful. Hardware-aware design changes the model before training so its layers match the deployment envelope. Sparsity exploitation makes removed weights visible to kernels that can skip them. Dynamic computation lets easy inputs leave early instead of paying for the worst case. Operator fusion reduces memory traffic when adjacent operations would otherwise write and reread the same tensors.

10.5.1 Hardware-aware design

Hardware-aware design begins before compression. If the target device cannot keep convolution kernels fed, cannot store activations without spilling, or cannot meet the power budget at the chosen input resolution, pruning and quantization only treat symptoms. The architecture itself must expose the kind of work the hardware can execute efficiently. That means choosing layer shapes, scaling rules, and operator patterns with memory bandwidth, parallelism, and energy as first-class constraints rather than post-hoc deployment checks.

10.5.1.1 Efficient design principles

The first design step is to identify what the trace shows as the limiting resource. A model can miss its target because every layer is too expensive, because one convolution family dominates arithmetic, because activations or parameters do not fit the memory hierarchy, or because the candidate architecture ignores the platform’s preferred operators. Table 10.12 organizes the common responses by the bottleneck they address rather than by model family.

Table 10.12: Hardware-Aware Design Principles: A profiling trace should determine which architectural response is appropriate. MobileNet responds to redundant convolutional computation with depthwise separable convolutions, DenseNet and SqueezeNet respond to memory pressure with feature reuse and parameter reduction, and hardware-aware NAS incorporates measured platform behavior directly into the search objective.

Observed bottleneck	Architectural response	Example networks
Over-budget model scaling	Adjust depth, width, and resolution together so the model stays within the latency, memory, and power envelope.	EfficientNet, RegNet
Redundant computation	Replace expensive dense operations with factorized or grouped operations that preserve useful channel mixing at lower arithmetic cost.	MobileNet, ResNeXt
Memory pressure	Reduce parameter and activation storage, or reuse features so the working set fits the available cache, SRAM, or device memory.	DenseNet, SqueezeNet
Platform mismatch	Include measured device latency, operator support, and power behavior in the architecture search or design loop instead of optimizing FLOPs alone.	MobileNetV3, MnasNet

These responses interact. Reducing convolutional FLOPs with depthwise separable convolutions²¹ helps only if the target runtime has efficient kernels for the resulting operators. Shrinking a model with parameter-reduction layers helps only if activation storage or memory traffic was part of the measured problem. Hardware-aware design therefore does not replace profiling; it moves profiling information earlier, into the architecture itself.

10.5.1.2 Scaling optimization

The first architectural response is global: when every stage of the profile is over budget, the problem is not one bad layer; the model is scaled incorrectly for the deployment envelope. The design task is then to distribute capacity across depth, width, and input resolution rather than choose a single parameter count. Depth increases sequential work and activation storage. Width exposes more parallel work but raises memory use. Resolution improves spatial detail while increasing the number of positions each convolution must process. The right balance depends on the machine: a highly parallel accelerator can often exploit width, while a small edge device may be dominated by memory capacity and energy per access.

Mathematically, the total FLOPs for a convolutional model can be approximated as:

$$\text{FLOPs} \propto N_L \cdot w^2 \cdot r^2,$$

where N_L is depth (number of layers), w is width, and r is the input resolution. This expression shows why naive scaling fails: increasing width and resolution together multiplies work quickly, and the resulting model may exceed the memory bandwidth or power budget even when the parameter count appears reasonable.

Compound scaling turns this balancing act into a controlled design rule. Instead of adjusting depth, width, and resolution independently, compound scaling grows all three dimensions by fixed ratios (α, β, γ) relative to a base model:

$$N_L = \alpha^\phi N_{L,0}, \quad w = \beta^\phi w_0, \quad r = \gamma^\phi r_0$$

Here, ϕ is a scaling coefficient, and α, β , and γ are scaling factors determined from empirical accuracy and efficiency measurements. The rule matters because it prevents one dimension from consuming the budget before the others can contribute useful accuracy.

EfficientNet (section 10.3.4) validated this principle by using search to find a baseline architecture and then scaling it with balanced depth, width, and resolution coefficients (Tan and Le 2019). The

²¹ | **Depthwise separable convolutions:** This technique reduces computation by factorizing a standard convolution into separate depthwise (per-channel) and pointwise (1×1) operations. MobileNet architectures use this factorization to trade model capacity and accuracy against lower operation counts for on-device vision (Howard et al. 2017; Sandler et al. 2018).

lesson is not that every deployment should use EfficientNet. The systems lesson is that scaling is a resource-allocation decision: the same accuracy target can imply different depth-width-resolution trade-offs depending on which resource the target platform makes scarce. Later benchmarking material formalizes how to measure those trade-offs; here, the design principle is to scale the dimensions against the binding resource.

The same logic extends beyond convolutional models. Transformer layers, attention heads, sequence length, and embedding width play roles analogous to depth, width, and resolution: each increases capacity, but each stresses compute, memory bandwidth, or activation storage differently. Hardware-aware scaling keeps those dimensions tied to the measured bottleneck instead of treating model size as a single scalar.

10.5.1.3 Computation reduction

If the profile shows that a small set of convolutional operators dominates arithmetic, reducing the whole model uniformly is wasteful. The better response is to change the expensive operator. Modern efficient architectures do this by factorizing dense computations into cheaper pieces that preserve the representation needed for accuracy.

Depthwise separable convolutions, popularized by MobileNet, exemplify this approach by decomposing standard convolutions into two stages: depthwise convolution (applying separate filters to each input channel independently) and pointwise convolution (1×1 convolution mixing outputs across channels). For batch size one, unit stride, padding that preserves an $h \times w$ spatial size, and a square kernel of width k , the computational complexity of standard convolution with C_{in} input channels and C_{out} output channels is:

$$\mathcal{O}(hwC_{\text{in}}C_{\text{out}}k^2)$$

where k is kernel size. Depthwise separable convolutions reduce this to:

$$\mathcal{O}(hwC_{\text{in}}k^2) + \mathcal{O}(hwC_{\text{in}}C_{\text{out}})$$

eliminating the k^2 factor from channel-mixing operations and often achieving $5\text{--}10\times$ FLOP reduction. The wall-clock benefit depends on kernel support and memory behavior: a mobile runtime with optimized depthwise kernels can convert much of this arithmetic reduction into latency savings, while a poorly supported backend may expose the factorized operations as many small, memory-bound kernels.

Other factorization patterns respond to the same diagnosis. Grouped convolutions, used in ResNeXt, partition feature maps into independent groups before merging them, reducing redundant cross-channel work. Bottleneck layers, used in ResNet, apply 1×1 convolutions to reduce feature dimensionality before expensive operations. SqueezeNet uses the same 1×1 idea to reduce parameters. These techniques improve efficiency when they reduce the operation that actually dominates the trace; they provide much less benefit when memory traffic, launch overhead, or unsupported kernels become the new bottleneck.

Arithmetic reduction is only one possible response. When profiling instead identifies memory capacity or data movement as the binding resource on the target device, the architecture must address that memory bottleneck directly.

10.5.1.4 Memory optimization

When the profile points to memory rather than arithmetic, the architecture must reduce the working set or the number of expensive memory accesses. Activations, feature maps, and parameters can exceed cache, SRAM, accelerator memory, or edge-device storage even when the FLOP count is acceptable. Memory-efficient architectures therefore try to preserve useful information while storing or moving less data.

DenseNet illustrates the feature-reuse response (Huang et al. 2017). In a traditional convolutional network, each layer computes a new set of feature maps, increasing the activation footprint as the network deepens. DenseNet connects layers so later computations can reuse earlier feature maps instead of relearning similar representations. In a standard convolutional network with N_L layers, if each layer generates g new feature maps, the total number of feature maps grows linearly:

$$\mathcal{O}(N_L g)$$

DenseNet reduces parameter redundancy through feature reuse, but concatenating retained features can increase activation storage and memory traffic. The resulting working set and runtime must be measured on the target hardware.

Activation checkpointing complements feature reuse by trading computation for memory during training. For N_L comparable layers with activation footprint A_{layer} per layer, evenly spaced checkpointing reduces the saved-activation term from $\mathcal{O}(N_L A_{\text{layer}})$ to $\mathcal{O}(\sqrt{N_L} A_{\text{layer}})$ while recomputing omitted activations during backpropagation. In the compression context, checkpointing enables training of larger models within fixed memory budgets, which in turn provides more capacity for subsequent pruning or distillation to exploit.

Parameter reduction applies the same reasoning to storage. SqueezeNet uses 1×1 convolutions to reduce the number of input channels before applying standard convolutions, making the expensive layer operate on a smaller representation (Iandola et al. 2016). The number of parameters in a standard convolutional layer is:

$$\mathcal{O}(C_{\text{in}} C_{\text{out}} k^2)$$

By reducing C_{in} using 1×1 convolutions, SqueezeNet reduces parameter count, achieving the paper’s AlexNet-level accuracy target with far fewer parameters than AlexNet. That trade is attractive when the deployment constraint is flash storage, model download size, or parameter bandwidth; it is less decisive when activation memory or operator overhead dominates.

Feature reuse, activation checkpointing, and parameter reduction are therefore not interchangeable recipes. Each changes a different part of the memory problem. Reused features reduce redundant representations, checkpointing reduces training-time activation storage, and 1×1 bottlenecks reduce parameter movement. The correct choice follows from which memory term the profile shows as binding.

Beyond minimizing stored tensor volume, substantial execution efficiency emerges from restructuring memory access schedules. Combining adjacent operations into composite execution kernels eliminates off-chip DRAM round-trips for intermediate feature maps.

10.5.1.5 Operator fusion

Consider a typical neural network layer: convolution followed by batch normalization followed by rectified linear unit (ReLU). Without fusion, each operation writes its output to GPU global memory, then the next operation reads that output back. Three memory round-trips occur for what could be computed entirely in fast on-chip registers. By fusing these operations into a single kernel, compilers and inference engines eliminate the redundant memory transactions, improving both throughput and latency on memory-bound workloads (Chen et al. 2018; NVIDIA 2024c).

Definition 10.5: Operator fusion

Operator fusion is a compiler and runtime optimization that combines adjacent tensor operations into a single fused kernel so intermediate values remain in registers or on-chip memory instead of being written to and reread from accelerator global memory.

1. **Significance:** In an ideal chain of N element-wise operations over the same M -byte tensor, separate kernels can move about $2NM$, while a fused kernel can approach $2M$ by keeping intermediates on chip. Real traffic also includes weights, caches, alignment, and spills, so profiling must confirm the reduction.
2. **Distinction:** Unlike pruning, quantization, or distillation, operator fusion does not change the model’s parameters, precision, or architecture. It changes the execution schedule of mathematically equivalent operations, preserving model outputs while improving latency and throughput on memory-bound workloads.
3. **Common pitfall:** A frequent misconception is that more fusion is always better. Fusion is constrained by data dependencies, tensor shapes, register pressure, and cache capacity; over-fusing can reduce occupancy or force spills back to memory, erasing the benefit.

Modern neural networks consist of sequences of operations such as convolution, batch normalization, activation functions, and element-wise operations. When executed independently, each operation requires four steps:

1. Loading input tensors from global memory
2. Performing computation
3. Writing output tensors back to global memory
4. Launching the next kernel

The read-compute-write cycle creates memory bandwidth bottlenecks for operations with low arithmetic intensity (FLOP/byte). For an ideal sequence of N operations over the same M -byte intermediate tensor, the unfused traffic is:

$$D_{\text{vol,unfused}} = 2NM$$

where each operation reads and writes the intermediate once. If the sequence is legal to fuse and intermediates stay on chip, the traffic can approach:

$$D_{\text{vol,fused}} = 2M$$

by reading the intermediate input once and writing its final output once. Weights, auxiliary inputs, cache effects, and register spills are outside this simple bound. Several inference patterns nevertheless approximate it closely enough for fusion to matter.

Convolution-BatchNorm-ReLU fusion The common Conv-BN-ReLU pattern in listing 10.4 can replace three launches and intermediate round-trips with one fused path when the compiler can legally fold batch normalization and combine the activation.

Listing 10.4: Conv-BN-ReLU Fusion: Combining three operations into a single kernel reduces memory traffic from 6 transfers to 2, eliminating intermediate memory writes.

```
# Pseudocode: runtime APIs and fusion legality are
# implementation-dependent.
# === UNFUSED: 3 kernel launches, 6 memory transfers ===
conv_out = conv2d(input, weight)
bn_out = batch_norm(conv_out, ...)
relu_out = relu(bn_out)

# === FUSED: 1 kernel launch, 2 memory transfers ===
def conv_bn_relu_fused(input, weight, gamma, beta, mean, var):
    # Read input and weight once
    conv = conv2d(input, weight)

    # Apply batch norm in registers (no memory write)
    bn = gamma * (conv - mean) / sqrt(var + eps) + beta

    # Apply ReLU in registers (no memory write)
    output = relu(bn)

    # Write final result once
    return output
```

Unfused: 6

Fused: 2

Operator fusion cuts memory transfers from six to two.

22 | FlashAttention: Demonstrates fusion's power for memory-bound attention by tiling computation to SRAM, avoiding materialization of the full attention matrix, and reporting multi-fold speedups on long sequences (T. Dao et al. 2022). This exemplifies how operator fusion transforms memory-bound bottlenecks: the arithmetic is mathematically equivalent, but the memory access pattern changes, making longer-context attention feasible on hardware that could not otherwise afford the full intermediate matrix.

The arithmetic operations remain identical, but memory traffic drops from 6 transfers to 2 transfers (3× reduction). For a ResNet-50 layer with 256 channels and spatial size 28×28 , this eliminates $4 \times 256 \times 28 \times 28 \times 4$ bytes ≈ 3.2 MB of intermediate memory traffic per layer.

The same principle extends beyond CNNs. GEMM bias-activation fusion eliminates intermediate writes in transformer linear layers by computing element-wise operations in registers immediately after each matrix multiplication output element. Attention tiling, as in FlashAttention,²² reduces HBM traffic by processing attention in SRAM-sized tiles and avoiding materialization of the full $S \times S$ attention matrix, as detailed in section 8.6.4.

Memory bandwidth analysis quantifies these fusion benefits concretely. Consider a Conv-BN-ReLU sequence operating on a $28 \times 28 \times 256$ feature map (802.8 KB). Without fusion, each operation performs its own memory round-trip: Conv reads input (802.8 KB) plus weights (2.4 MB) and writes output (802.8 KB), totaling 4 MB. BN then reads that output, adds its parameters (2 KB), and writes again, for 1.6 MB. ReLU repeats the pattern for another 1.6 MB. The total unfused memory traffic is 7.2 MB. With fusion, the entire sequence reads input and weights once and writes the final output once, requiring only 4 MB—a 44.5 percent bandwidth reduction.

At the modeled 900 GB/s HBM bandwidth, these traffic volumes imply ideal lower bounds of 8 microseconds before fusion and 4.5 microseconds after fusion, a $1.80\times$ ratio for the traffic component alone. These are not full layer latencies because convolution compute, cache effects, and launch overhead must also be included.

As table 10.13 shows, fusion benefits vary by workload. Memory-bound operations benefit most, while compute-bound operations see minimal improvement.

Table 10.13: Illustrative Operator Fusion Benefit by Workload: Memory-bound workloads tend to benefit more than compute-bound workloads; realized gains depend on the graph, runtime, sequence shape, and hardware.

Workload	Relative Fusion Benefit	Why
Element-wise operations	High	Highly memory bound, low arithmetic intensity
Conv-BN-Act patterns	Moderate	Mixed memory/compute characteristics
GEMM-based operations	Low	Compute bound; fusion reduces the memory-bound tail
Attention mechanisms	High	Long sequences amplify avoided attention-intermediate traffic

Fusion also reduces kernel launch overhead. Each CUDA kernel launch incurs microsecond-scale latency. In a hypothetical graph with fifty-three eligible Conv-BN-ReLU triplets, unfused execution launches 159 kernels, while fused execution launches 53 kernels, saving repeated launch overhead in addition to memory traffic. ResNet-50 does not contain fifty-three direct triplets because residual additions interrupt many such patterns.

Fusion implementation spans the software stack, from framework-level pattern matching through compiler optimization such as Accelerated Linear Algebra (XLA), TVM, and TensorRT to runtime fusion that adapts to input shapes and hardware characteristics. The process becomes tractable when a framework can export or trace the relevant computation as a graph whose nodes are tensor operations and whose edges record data dependencies. A compiler can then match legal patterns such as Conv→BN→ReLU and rewrite them as fused operations while checking shapes, aliases, and numerical constraints. Dynamic control flow, in-place mutation, or unsupported operations can prevent that graph rewrite, so the exported plan must be inspected rather than assumed. Section 11.8.1.3 examines the compiler and hardware dimensions of fusion in detail, including register pressure, graph matching, and platform-specific trade-offs across GPU, TPU, and edge accelerators.

While operator fusion minimizes data movement between static pipeline stages, adaptive computation challenges the assumption that every input requires the full depth or width of the computational graph. By modulating active subnetworks based on input difficulty or runtime budgets, systems trade deterministic execution schedules for lower amortized latency and energy.

10.5.2 Adaptive computation methods

Hardware-aware architecture design and operator fusion optimize a fixed execution graph, ensuring that absent input-dependent control, every input follows an identical scheduled path. On static graphs, the **Algorithm** imposes an invariant computational workload on the **Machine**: a 24-layer transformer executes identical MAC operations and streams its full parameter footprint from HBM or off-chip DRAM for an obvious, easily classified sample as it does for an ambiguous one. **Adaptive inference** replaces this open-loop pipeline with an input-dependent control loop: intermediate representations or runtime budgets inform dynamic decisions that terminate execution or route activations through specialized subnetworks. While dynamic paths can reduce average arithmetic operations and memory traffic, they introduce runtime overheads in routing logic, batch management, confidence calibration, and latency variance.

10.5.2.1 Dynamic schemes

Because input data exhibit non-uniform semantic complexity, allocating an invariant computational budget wastes memory bandwidth and execution cycles on simple samples. Dynamic schemes modulate execution paths to lower amortized computation while preserving the deep network for worst-case inputs. This architectural flexibility spans four primary control variables: network depth (early exit), structural routing (conditional layer skipping), expert subnetworks (mixture-of-experts), and continuous computational allocation.

To control execution depth, **early exit** architectures attach auxiliary classification heads to intermediate layers, terminating execution when intermediate representations yield sufficient prediction confidence (Teerapittayanon et al. 2017). BranchyNet implements this mechanism across convolutional backbones using shallow exit branches, whereas multi-exit vision transformers attach linear probes and entropy estimators between transformer blocks (Xin et al. 2021). When an input satisfies an exit criterion—such as softmax prediction entropy dropping below a pre-tuned threshold—the runtime halts evaluation and returns the intermediate prediction. The net operational benefit requires that the arithmetic operations and memory transfers avoided in subsequent layers exceed the computational overhead of evaluating the intermediate classifiers and gating thresholds.

The systems trade-off diverges sharply across hardware deployment targets. On single-sample edge processors (batch size $B = 1$, typical of microcontrollers and mobile NPUs), early exit immediately terminates the execution thread (Hu et al. 2020). Skipping remaining layers eliminates both the arithmetic operations and the off-chip DRAM transactions required to load subsequent layer weights into on-chip SRAM cache. Conversely, on parallel throughput accelerators (GPUs and TPUs) processing batched workloads ($B > 1$), early exit introduces severe batch fragmentation (Chen et al. 2024). If only a subset of samples in a batch reaches the exit threshold, the remaining samples must still traverse deeper layers. Unless the runtime executes dynamic batch compaction—gathering unresolved activation tensors into a contiguous memory buffer via memory copies and kernel synchronizations—inactive samples remain allocated in memory, or the accelerator must mask inactive SIMD or single instruction, multiple threads (SIMT) lanes. Consequently, tail latency for the batch remains dictated by the deepest traversing sample. Figure 10.26 traces this decision pipeline, where each intermediate block evaluates a confidence estimator before routing unresolved representations to subsequent layers.

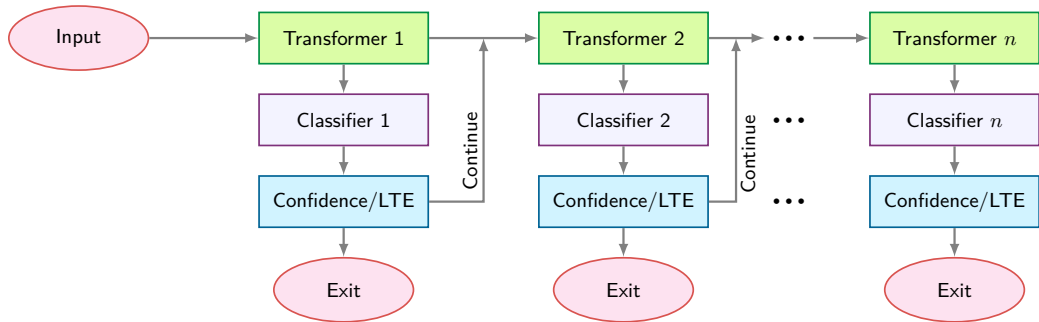


Figure 10.26: Early Exit Architecture: Transformer layers dynamically adjust computation by classifying each layer's output and enabling early termination if sufficient confidence is reached, reducing average latency and power when many inputs can exit before the final layer. Source: (Xin et al. 2021).

Beyond truncating execution depth along a single linear pipeline, **conditional computation** dynamically modulates the topological routing of activations through the network graph (Bengio et al. 2015). Instead of executing every block in a predefined sequence, gating functions evaluate activation tensors to bypass residual layers, alter operator kernels, or select specialized functional blocks. This converts a static directed acyclic graph (DAG) into an input-dependent dynamic graph.

Architectural implementations target distinct structural granularities. SkipNet uses lightweight recurrent or feedforward gating controllers to conditionally bypass residual blocks in convolutional networks, preserving full network depth only for computationally difficult inputs (X. Wang et al. 2018). Dynamic Filter Networks generate sample-specific convolutional filter weights dynamically

from the input, modifying feature extraction patterns directly rather than skipping static weights (Jia et al. 2016). Capsule Networks utilize iterative routing-by-agreement to direct low-level feature vectors to compatible high-level capsules (Sabour et al. 2017). From a hardware systems perspective, conditional routing achieves wall-clock speedups only when the cycles saved by bypassing operators surpass the latency of evaluating the gating network, executing host-device control synchronization, and managing uncoalesced memory accesses.

Scaling dense transformer models couples parameter capacity directly to per-token floating-point operations: increasing model capacity forces every token to compute against every parameter weight matrix, driving up both arithmetic demand and memory bandwidth requirements. The **mixture-of-experts (MoE)** architecture decouples parameter capacity from per-token compute by replacing dense feedforward network (FFN) layers with a bank of parallel expert subnetworks coordinated by a parameterized router (Shazeer et al. 2017). Google’s Switch Transformer²³ scales this sparse routing principle by steering each token to a single expert ($k = 1$), dramatically reducing communication overhead compared to top- k routing (Fedus et al. 2022).

This sparse activation creates a fundamental memory-bandwidth and capacity trade-off on accelerator hardware. While a token executes arithmetic against only $1/E$ of the total expert parameters, all E expert matrices must remain resident in device memory (HBM or SRAM). At low operational batch sizes—such as single-stream autoregressive generation—routing divergent tokens to different experts fragments matrix-matrix multiplications (*GEMM*) into low-arithmetic-intensity matrix-vector operations (*GEMV*), causing memory bandwidth saturation. Furthermore, uneven token distribution causes computational hotspots where overloaded experts drop tokens exceeding their capacity factor, while under-allocated experts leave tensor core pipelines idle.

Sparse Mixture-of-Experts architectures replace monolithic feedforward networks (FFNs) with dynamically routed subnetworks. Compare the transformer block structure on the left of figure 10.27 with the expanded gating router on the right, observing how each input token activates only a single expert FFN block.

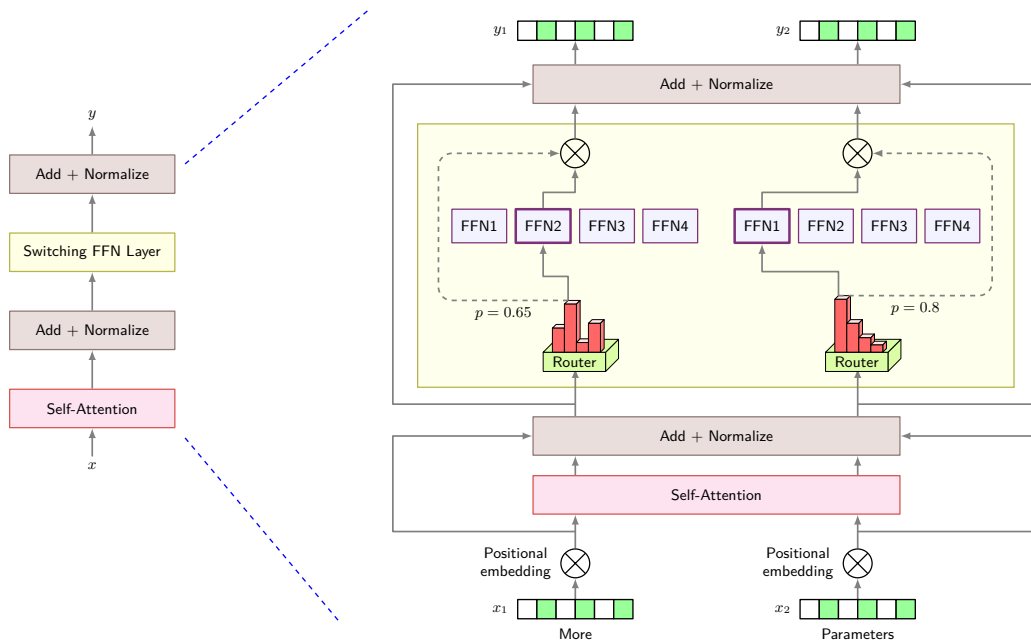


Figure 10.27: Switch Transformer Architecture: Replacing dense feedforward layers with a Switching FFN layer (left) enables sparse conditional routing. A gating network routes each token to a single expert (right) out of a large pool, decoupling total model capacity (parameters) from per-token compute cost (FLOPs). Source: (Fedus et al. 2022).

Because the router lies on the critical execution path, the accelerator runtime must dynamically dispatch and gather activations across expert blocks without triggering host-device synchronization stalls. On SIMD and SIMT architectures such as GPUs and TPUs, executing sparse expert

²³ | **Switch Transformer:** Fedus et al. (2022) scales to roughly 1.6 trillion total parameters while routing each token to one expert; in a compute-matched experiment, Switch-Base with 64 experts reached the T5-Base quality target in about one-seventh the training time; the trillion-parameter comparison reported roughly $4\times$ speedup over T5-XXL. Routing each token to one expert reduces communication relative to top- k routing, but load imbalance requires auxiliary losses and capacity factors. This trade-off—massive parameter capacity at low per-token compute cost, but with complex systems engineering for load balancing—defines the MoE design space.

layers efficiently requires token permutation: tokens assigned to the same expert are gathered into contiguous memory tiles to sustain high Tensor Core utilization during subsequent matrix multiplications (Lepikhin et al. 2021). If the routing distribution is highly skewed, token buffers for popular experts overflow their fixed capacity allocations, forcing the system to either truncate tokens (causing representation degradation) or trigger ragged kernel launches that leave parallel vector lanes idle.

While early exit and MoE routing make discrete branch selections, adaptive inference can also operate continuously by progressively modulating depth or iteration counts based on real-time task difficulty (Yang et al. 2020). Fast Neural Networks dynamically adjust the number of active layers according to runtime complexity estimators (J. Wu et al. 2019), while dynamic layer scaling allocates computation incrementally until output uncertainty drops below a calibrated threshold. Consider an embedded vision system processing sensor streams: high-contrast frames with clear road geometry resolve through a minimal prefix of feature-extraction layers, whereas degraded frames containing occluded obstacles trigger deeper recurrent processing blocks. The physical systems gain materializes only if the control evaluation remains computationally lightweight and the runtime schedules frames with matched execution depths concurrently to maintain high hardware utilization.

10.5.2.2 Implementation challenges

Realizing the efficiency gains of adaptive computation requires overcoming critical challenges across model training, runtime orchestration, and hardware alignment. During training, discrete routing decisions (such as non-differentiable argmax selections) break standard backpropagation. Practitioners must employ continuous relaxations such as the Gumbel-Softmax estimator, reinforcement learning policy gradients, or auxiliary load-balancing losses to prevent routing collapse where the router over-allocates inputs to a single expert or exit (Shazeer et al. 2017).

At runtime, control-flow branching fundamentally conflicts with the bulk-synchronous parallel execution model of modern accelerators. When parallel threads within a GPU warp encounter divergent execution branches, the hardware serializes branch execution and masks inactive threads, nullifying theoretical FLOP reductions. Furthermore, evaluating gating networks introduces control dependencies that prevent operator fusion, inject host-device synchronization barriers, and require intermediate memory copies for token permutation. If these dispatch overheads exceed the compute time saved by skipping layers, the dynamic model exhibits higher end-to-end latency than a dense baseline. Section 11.10.2 examines hardware-aware runtime strategies, including ragged batching and dynamic kernel launch queues, designed to manage these execution patterns.

Beyond systems efficiency, dynamic control introduces severe quality and security failure modes. Because gating policies route inputs based on statistical heuristics, poorly calibrated gates systematically underallocate computation to rare distribution tails, degrading prediction accuracy precisely on safety-critical edge cases. Furthermore, input-dependent computation exposes an attack surface: adversarial perturbations can intentionally manipulate routing gates to trigger denial-of-quality attacks (forcing inputs into truncated early exits that fail) or denial-of-service attacks (forcing all inputs into maximally deep or imbalanced paths to saturate accelerator queues). Consequently, evaluating adaptive models cannot rely on average FLOP counts alone. Rigorous system profiling must report path distribution histograms, tail latency (P99), accuracy stratified by input difficulty, and throughput stability under dynamic batching. Where dynamic computation decides whether to perform certain operations, sparsity exploitation addresses a complementary question: how to accelerate computation when many operands are zero.

10.5.3 Sparsity exploitation

Recall that pruning (from section 10.3.1) introduces zeros into weight matrices. Sparsity exploitation asks how to accelerate computation when those zeros are present. Pruning creates zeros or removes structure; sparse formats can reduce storage, and matching kernels can reduce computation. **Sparsity**²⁴ in machine learning refers to the condition where a significant portion of the elements within a tensor, such as weight matrices or activation tensors, are zero or nearly zero.

²⁴ | **Sparsity**: From Latin *sparsus* (scattered), past participle of *spargere* (to scatter); in ML, L1 regularization such as the lasso can induce exact zeros rather than merely small values (Tibshirani 1996). Software can represent arbitrary sparsity patterns, while efficient hardware execution generally requires a supported format or structured pattern (for example, NVIDIA's 2:4 sparsity). The gap between representable and executable sparsity separates theoretical compression from realized speedup.

More formally, for a sparse weight matrix $\mathbf{W}_{\text{sparse}} \in \mathbb{R}^{m \times n}$, the sparsity ratio ρ_{sparse} can be expressed as:

$$\rho_{\text{sparse}} = \frac{\|\mathbf{1}_{\{(\mathbf{W}_{\text{sparse}})_{ij}=0\}}\|_0}{m \times n}$$

where $\mathbf{1}_{\{(\mathbf{W}_{\text{sparse}})_{ij}=0\}}$ is an indicator function that yields one if entry (i, j) is zero and 0 otherwise, and $\|\cdot\|_0$ represents the L0 norm, which counts the number of nonzero elements. Exact zero is representable in floating point, but thresholded definitions also group small-magnitude values when the pruning method treats them as removable. The thresholded sparsity ratio becomes:

$$\rho_{\text{sparse}, \epsilon} = \frac{\|\mathbf{1}_{\{|(\mathbf{W}_{\text{sparse}})_{ij}| < \epsilon\}}\|_0}{m \times n}$$

where ϵ is a small threshold value.

Exact sparsity can emerge during training through regularization or be introduced deliberately through pruning, which forces selected elements to zero. It yields memory, compute, or energy savings only when the storage format and execution path skip those zeros with less overhead than dense execution. That qualification is especially important on resource-constrained devices, where index metadata and irregular accesses consume the same scarce bandwidth and power.

10.5.3.1 Sparsity types

The hardware decision begins with the pattern of zeros. Sparsity in neural networks falls into two broad categories: unstructured sparsity and structured sparsity.

Unstructured sparsity occurs when individual weights are set to zero without any specific pattern, typically through magnitude-based pruning. While highly flexible, unstructured sparsity is less efficient on hardware because it lacks a predictable structure.²⁵ Exploiting it requires specialized hardware or software optimizations.

Structured sparsity removes regular groups such as filters, neurons, channels, blocks, or fixed within-group patterns. Regularity makes storage and execution more predictable, but acceleration still depends on whether the target supports that structure and whether the resulting shapes use its compute units efficiently. It is therefore a candidate, not an automatic preference, when deployment requires predictable resource use.

10.5.3.2 Sparsity utilization methods

A sparse model with 90 percent of weights zeroed may still run at nearly full computational cost on hardware not designed for irregular memory access. The critical question is how to translate theoretical zeros into actual speedup. The processor *cannot* skip a multiplication unless it knows the operand is zero—and discovering that requires loading the operand from memory in the first place. Bridging this gap requires specialized utilization methods and hardware support that can efficiently skip zero-valued computations (Hoeffler et al. 2021). Han et al.'s pruning work (2015) is a canonical example of turning dense networks into sparse ones by removing unimportant connections, but accelerator speedups depend on whether the resulting sparsity pattern matches hardware-supported formats.

The simplest utilization method is a sparse matrix operation, which stores nonzero values and uses their indices to skip arithmetic on zeros. Consider the difference: multiplying a dense 4×4 matrix with a vector typically requires 16 multiplications, while a sparse-aware implementation computes the six nonzero products in addition to processing the sparse metadata:

$$\begin{bmatrix} 2 & 0 & 0 & 1 \\ 0 & 3 & 0 & 0 \\ 4 & 0 & 5 & 0 \\ 0 & 0 & 0 & 6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 2x_1 + x_4 \\ 3x_2 \\ 4x_1 + 5x_3 \\ 6x_4 \end{bmatrix}$$

The deployment choice is therefore not a generic desire for fewer parameters; it is a choice between representations the runtime can execute efficiently. Low-rank approximation, covered earlier in

²⁵ | **Unstructured sparsity and SIMD waste:** Modern CPUs and GPUs process data in vector or tensorized groups; unstructured sparsity scatters nonzero elements irregularly through memory, so a vector load may bring back mostly zeros while still paying the full memory access cost. The processor cannot skip zero elements without first knowing where the nonzeros are, and the metadata needed to answer that question also consumes bandwidth. This is why structured sparsity can deliver speedups at lower sparsity levels than arbitrary unstructured sparsity, while unstructured sparsity often needs very high zero fractions and specialized kernels before arithmetic savings overcome indexing and lane-utilization overheads (Hoeffler et al. 2021).

section 10.3.3.1, replaces a dense matrix with smaller dense factors, while sparsity exploitation skips literal zero-valued weights. Sparsity-aware training and sparse gradient descent can help models learn or maintain zero patterns, but runtime speedup appears only when the deployed representation uses formats and kernels that skip zeros. That is why the next design question is not merely how sparse the matrix is, but what structure the zeros have.

10.5.3.3 Structured patterns

Achieving actual speedups from sparsity requires hardware that can efficiently skip zero-valued computations. Different processor architectures handle sparse patterns with varying effectiveness—for example, Ampere Sparse Tensor Cores exploit fine-grained 2:4 structured patterns while systolic arrays require dense block structures, hardware mechanics detailed in section 11.4.5. Software libraries such as cuSPARSE can help bridge this gap by reformulating sparse computations into patterns that current hardware handles efficiently. For example, MegaBlocks (Gale et al. 2022) reformulates sparse Mixture of Experts training into block-sparse operations, grouping routed expert-token work into dense tiles so specialized kernels can maintain high accelerator utilization despite irregular sparsity patterns.

A sparse pattern earns hardware speedup only when it is regular enough for kernels to predict. Two prominent formats make that regularity explicit: block sparse matrices and N:M sparsity patterns. Block sparse matrices isolate blocks of zero and nonzero dense submatrices so that operations on the large sparse matrix can be re-expressed as a smaller number of dense operations on submatrices. This structure supports more efficient storage of dense submatrices while maintaining shape compatibility for matrix or vector products. For example, figure 10.28 shows how NVIDIA's cuSPARSE (NVIDIA 2020a) library supports sparse block matrix operations and storage. Several other works, such as Monarch matrices (Tri Dao et al. 2022), have built on this block-sparsity approach to strike an improved balance between matrix expressivity and compute/memory efficiency.

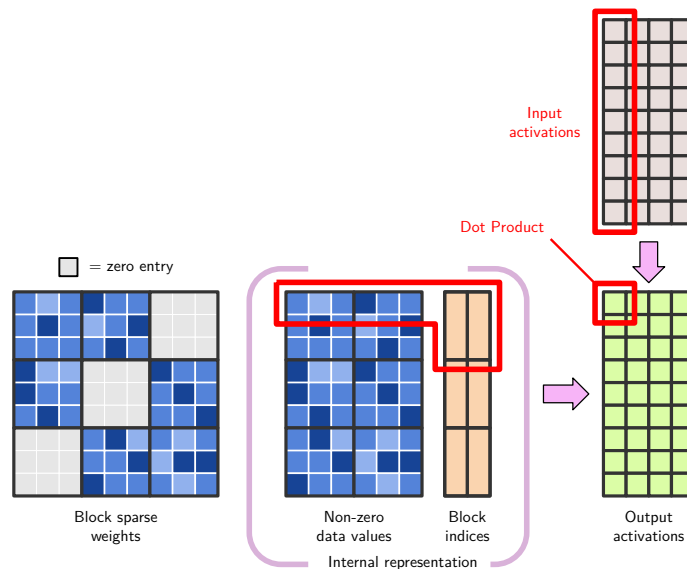


Figure 10.28: Block Sparse Representation: Grouping non-zero weights into dense sub-blocks (3×3 tiles) and storing an auxiliary index array allows sparse matrices to be computed via regular sub-block GEMM kernels, restoring high GPU memory alignment and Tensor Core utilization. Adapted from NVIDIA cuSPARSE documentation (NVIDIA 2020a).

Similarly, $N:M$ sparsity retains at most N nonzeros in each group of M consecutive elements; hardware-oriented pruning often enforces exactly N retained values so every group has the same representation (Zhou et al. 2021). The deterministic format lets a supported kernel predict the value and metadata layout while retaining more capacity than removing whole channels or blocks. That regularity creates an executable compromise between arbitrary masks and dense computation, but only for targets with the matching path. Because exactly two nonzero values are selected out of every

contiguous block of four, each stored value requires only a 2-bit index (00_2 through 11_2) to record its original coordinate within the block. This compact metadata imposes only a 16-bit overhead per 16-element row segment—far below the multi-byte pointer arrays required by general CSR or COO formats—allowing Tensor Cores to stream compressed operands without memory bandwidth bloat. Figure 10.29 compares dense and 2:4 matrix multiplication, while STEP examines learning more general $N:M$ masks for inference (Lu et al. 2023).

At accelerator level, the same pattern-specific rule holds: hardware support is a contract between the sparse format and the execution path.

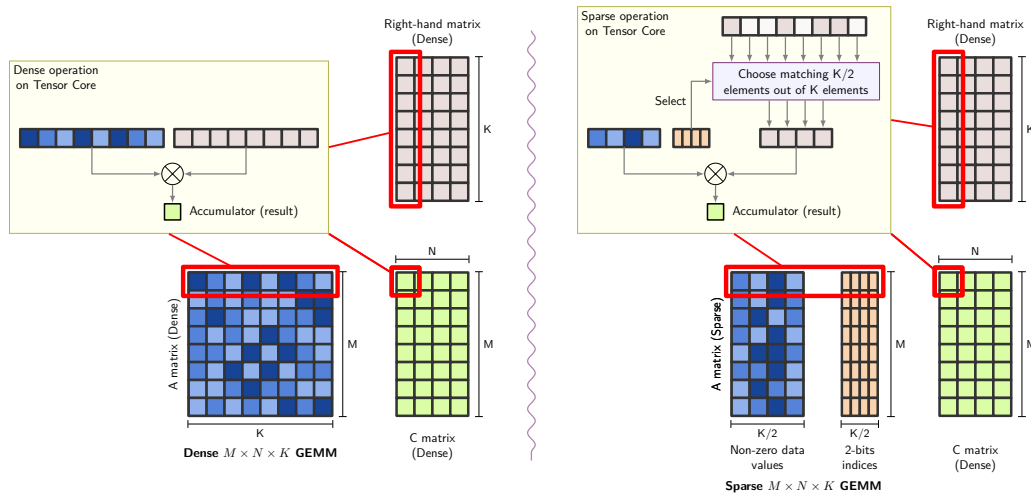


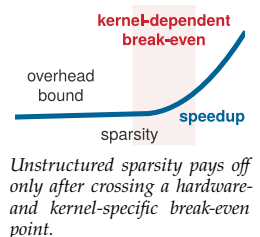
Figure 10.29: 2:4 Structured Sparsity GEMM: Left: standard dense matrix multiplication on Tensor Cores using full 8-element rows. Right: 2:4 sparse multiplication retains two nonzeros in every group of four and uses compact indices to select matching elements from the dense right-hand matrix. A supported sparse inner loop can halve the multiplied values, although metadata handling and the rest of the execution path determine end-to-end speedup. Source: PyTorch blog (PyTorch 2022).

Supported Sparse Tensor Core paths on NVIDIA Ampere and later can accelerate 2:4 sparsity by skipping prescribed zeros and carrying compact metadata, with an ideal sparse arithmetic-rate improvement of up to $2\times$ (NVIDIA Corporation 2020a). Conversely, **unstructured pruning** zeroes arbitrary individual weights. Those irregular locations can disrupt memory coalescing and lane utilization, while Compressed Sparse Row (CSR) or Coordinate (COO) metadata adds storage and non-unit-stride accesses. An unstructured sparse model can therefore run slower than its dense counterpart when metadata and underutilization outweigh skipped arithmetic. TPUs are useful contrast cases for dense systolic-array acceleration (Jouppi et al. 2021), but sparse acceleration still depends on a specific hardware and software path rather than accepting arbitrary masks. Field-programmable gate arrays can implement application-specific sparse formats, yet their efficiency depends on the chosen dataflow and resource budget; programmability is not a universal sparse-speedup guarantee.

Across all platforms, sparse operations can reduce memory bandwidth requirements and energy consumption when the sparse representation and kernels actually skip data movement rather than only arithmetic. This benefit compounds with quantization: a sparse INT8 model can require less memory traffic than either technique alone when the format overhead is small enough (Hoeffler et al. 2021; Gale et al. 2020).

10.5.3.4 Challenges and limitations

The same format-hardware contract explains why sparsity often disappoints in practice. The central challenge is the gap between theoretical and practical speedups. Unstructured pruning removes individual weights based on importance, creating irregular patterns that common dense accelerator paths cannot exploit; skipping those zeros requires a sparse representation and a compatible kernel. Pruning itself adds training or analysis cost because identifying removable weights can require sophisticated importance estimation on large models. Even after sparsity is achieved, storage



formats add indices whose overhead can offset saved values and arithmetic. Section C.1.5 details the compressed sparse row layout and quantifies how its per-nonzero metadata makes the memory payoff density-dependent. The performance break-even point varies with tensor shape, sparse format, kernel, batch size, and hardware; there is no universal sparsity threshold.

The accuracy-efficiency trade-off requires measurement across candidate sparsity levels. A model may tolerate one level with little measured impact and then degrade sharply after a comparatively small additional pruning step. The operating point is the highest useful sparsity whose task quality and deployed performance both satisfy their thresholds.

Energy efficiency is not guaranteed. While sparse operations reduce arithmetic operations, the overhead of sparse indexing and irregular memory access can increase power consumption on hardware not optimized for sparse patterns. On edge devices with tight power budgets, these overheads may outweigh the benefits.

Finally, sparsity benefits vary by layer, model, and execution target. A workload whose useful tensors remain dense, or a target without a matching sparse path, may see no improvement and can regress after metadata and irregular-access costs are included.

10.5.3.5 Combined optimizations

Pruning, quantization, operator fusion, dynamic computation, and sparse execution share weights, representations, and physical resources. A deployment may combine them when one technique cannot satisfy every constraint, but the combined result is not guaranteed to exceed the best individual technique. Each transformation can alter the accuracy, shapes, distributions, and bottlenecks assumed by the next, so combinations require joint quality and performance measurement rather than a promised compression ratio (Hoeffler et al. 2021).

The interaction between sparsity and pruning is the most direct: pruning creates sparsity, but the pattern determines hardware efficiency. Removing entire filters or layers can produce smaller dense shapes, while fixed within-group patterns can map to supported sparse kernels. Unstructured pruning creates irregular patterns that require specialized formats and kernels to realize speedups (Elsen et al. 2020; Gale et al. 2019).

Combining sparsity with quantization can multiply raw value-payload reductions, but the deployed representation carries both sparse and quantization metadata. GPUs with dedicated sparse tensor cores can accelerate supported structured patterns, while CPU outcomes depend on the chosen format, low-precision sparse kernel, tensor shape, and indexing overhead (NVIDIA Corporation 2020a; Hoeffler et al. 2021; Gale et al. 2020).

The recurring theme across all combinations is hardware alignment. Efficient model designs such as depthwise separable convolutions (Howard et al. 2017), dynamic computation, and sparsity help only when the target hardware supports the resulting operation patterns (Hoeffler et al. 2021). Selecting technique combinations requires understanding target platform capabilities, as explored in chapter 11.

The coordination challenges inherent in combining sparsity with other techniques point to a broader principle: optimization techniques rarely succeed in isolation, and their effectiveness depends on sequencing decisions and hardware alignment.

🔍 Systems Perspective 10.4: The optimization composition problem

Optimization techniques interact through shared representations and physical resources rather than composing as independent functions. Pruning changes the surviving value distribution, tensor shapes, or sparse format; quantization adds scales and changes the available kernels; fusion changes which intermediates are materialized and can alter the profile that justified earlier choices. Effective optimization therefore treats the model-runtime-hardware combination as a coupled system, measures it after every transformation, and revisits the sequence when a new bottleneck appears. This makes compression a systems engineering problem as well as a machine learning problem.

10.6 Technique Selection

Selecting compression techniques for production deployment requires identifying the binding physical constraint of the target execution environment. Consider a transformer deployment where model parameters exceed device memory by $3\times$, inference latency exceeds the service-level objective by $4\times$, and the thermal envelope restricts dissipation to no more than 2 W sustained. Deciding whether to quantize, prune, distill to a compact architecture, or compose multiple methods depends on which hardware boundary is saturated, the allowable task accuracy loss, and the available training budget.

The three primary optimization levers target different levels of the system stack. Distillation alters the algorithmic architecture by training a compact dense student to emulate a larger teacher, executing efficiently on standard hardware without specialized runtime support. Pruning modifies tensor topology and operation patterns, but yields wall-clock acceleration only when the resulting sparsity aligns with hardware execution units, such as $2:4$ structured sparse matrix cores or channel-aligned sub-matrices. Quantization alters data representation by reducing numerical bit width, directly lowering memory bus traffic and cache footprint while engaging higher-throughput integer datapaths. Table 10.14 summarizes the first-order trade-offs among these techniques across quality risk, training cost, and hardware dependency.

Table 10.14: Optimization Technique Trade-Offs: These rows identify first-order candidates rather than guaranteed outcomes. Quality risk and training cost depend on the model and method, while hardware dependency determines whether pruning or quantization changes latency. Pruning can remove structure or create sparse tensors, quantization directly reduces raw value bytes, and distillation spends teacher inference plus a student training run to produce an ordinary dense model. The candidate must still be evaluated against the same task and deployed-system thresholds.

Technique	Primary Goal	Quality Risk	Training Cost	Hardware Dependency	Candidate Use
Pruning	Reduce FLOPs/size	Pattern-dependent	Often includes fine-tuning	High for sparse speedup	Removable structures/weights
Quantization	Reduce bytes/precision	Bit-width-dependent	Low for PTQ; higher for QAT	High for runtime speed	Memory- or bandwidth-bound path
Distillation	Train smaller model	Student-dependent	Teacher and student run	Low for dense execution	A new training run is feasible

10.6.1 Mapping constraints to techniques

The binding hardware constraint dictates which optimization family must be deployed first. Under the D·A·M taxonomy, compression methods redistribute resource demands across Algorithm (network topology and structural pruning), Data (numerical precision and calibration sets), and Machine (execution units and memory hierarchies). Table 10.15 maps each system constraint to the primary optimization dimension capable of relieving it.

Table 10.15: Optimization Dimensions: System constraints drive optimization along three core dimensions: model representation, numerical precision, and architectural efficiency, each addressing different resource limitations and performance goals. ✓ marks a direct lever, while △ marks a workload- or hardware-dependent benefit; for example, low-precision arithmetic helps only when the accelerator and runtime provide a supported path.

System Constraint	Model Representation	Numerical Precision	Architectural Efficiency
Computational Cost	✓	△	✓
Memory and Storage	✓	✓	△
Latency and Throughput	✓	△	✓
Energy Efficiency	✓	✓	✓
Scalability	✓	△	✓

These dimensions interact through the iron law of ML systems, meaning a single optimization often shifts multiple physical bottlenecks simultaneously. For example, quantizing weights from FP32 to INT8 directly relieves on-device storage capacity. However, its effect on inference latency

depends on operational intensity: in memory-bandwidth-bound operators, reducing transferred bytes yields a proportional latency reduction; in compute-bound operators, latency improves only if the underlying processor exposes higher peak INT8 arithmetic throughput and the runtime avoids dequantization overhead. Conversely, unstructured pruning eliminates arithmetic FLOPs without reducing latency on standard accelerators, because sparse indexing overhead and uncoalesced memory transactions degrade memory bandwidth efficiency.

10.6.2 Decision framework

When persistent storage or device memory capacity binds the deployment—such as over-the-air package limits or microchip flash budgets—quantization provides the most direct payload reduction. Relative to FP32 storage, INT8 post-training quantization (PTQ) cuts raw weight storage by $4\times$ without requiring retraining cycles. INT4 cuts raw payload by $8\times$, though aggressive bit-width reduction increases the risk of accuracy degradation. If uniform low-bit quantization causes unacceptable accuracy loss on sensitive layers, distilling knowledge into a smaller dense architecture followed by moderate quantization preserves fidelity while satisfying the storage budget.

When latency is compute-bound, the workload operates above the accelerator’s arithmetic intensity ridge point, saturating execution units. Relieving this bottleneck requires reducing executed operations or increasing arithmetic throughput. Structured pruning removes channels, attention heads, or entire layers, yielding smaller dense matrices that execute on standard libraries without sparse runtime overhead; however, pruning to dimensions that misalign with hardware warp or tile sizes (such as channel counts not divisible by 8 or 16) degrades tensor core efficiency and curtails wall-clock gains. If the target platform provides dedicated integer execution units, INT8 quantization increases the instruction throughput rate, accelerating compute-bound kernels directly. Early-exit architectures provide an alternative algorithmic path by dynamically halting inference on simpler inputs, provided that routing and pipeline synchronization overhead do not disrupt batch processing.

Small-batch transformer generation introduces an entirely different bottleneck: autoregressive decoding is bound by memory bandwidth rather than compute. Generating each token requires loading large parameter matrices from HBM or off-chip DRAM to perform matrix-vector operations with an operational intensity of roughly one FLOP per byte. Because arithmetic units sit idle waiting for memory transfers, weight-only quantization (such as INT4 or INT8 weights paired with FP16 activations) yields a latency speedup roughly proportional to the reduction in fetched weight bytes. This bandwidth advantage persists until KV-cache traffic, dequantization kernel overhead in registers, or larger concurrent batch sizes push the operational intensity back into the compute-bound regime.

When energy or thermal dissipation binds, quantization provides substantial benefits because moving data across off-chip memory buses consumes orders of magnitude more energy per bit than on-chip arithmetic or local SRAM access. Reducing operand bit widths directly lowers bus capacitance charging energy, while integer ALUs consume less switching energy than floating-point datapaths. Structured pruning compounds these savings by eliminating operations and memory transactions entirely. Full-system power audits must still measure peripheral components—including memory refresh, host bus communication, and static idle leakage—to ensure that kernel-level energy savings translate into extended operating life.

Selection also depends on the engineering and computational budget available for optimization. Post-training quantization serves as the initial baseline because calibrated PTQ requires only a small representative dataset and negligible compute. If PTQ fails the accuracy threshold and fine-tuning resources are available, quantization-aware training (QAT) models quantization error during the backward pass to recover lost accuracy. Knowledge distillation incurs higher computational costs, requiring teacher forward passes and substantial training cycles to optimize a smaller student model. Hardware-aware NAS occupies the highest cost tier, justified only when high-volume deployment amortizes the extensive compute needed to explore the architectural search space.

Validating that a chosen technique achieves its performance target requires systematic profiling and measurement on target silicon (section 10.8). Real-world deployments rarely rely on an isolated optimization: combining pruning, quantization, and distillation introduces cross-layer interactions that dictate strict sequencing protocols.

10.7 Optimization Strategies

Production deployment constraints rarely permit relying on an isolated optimization. Fitting a 110M-parameter language model within a mobile memory and thermal budget requires composing structural pruning, knowledge distillation, and precision quantization. In this modeled mobile pipeline, staged optimization reduces a BERT-Base footprint from 440 MB to 28 MB, achieving a 16× raw-footprint reduction. Each stage addresses a distinct bottleneck in the D·A·M hierarchy: structural pruning eliminates redundant matrix multiplications to reduce arithmetic operations, knowledge distillation supervises the compact student to recover representational capacity, and quantization packs weights and activations into lower-bit integers to alleviate memory bandwidth pressure.

Example 10.3: BERT-Base mobile deployment pipeline

Scenario: A modeled deployment targets executing a 110M-parameter BERT-Base language model on resource-constrained mobile hardware (Sanh et al. 2019).

Diagnosis: Applying compression techniques in an arbitrary order degrades representation quality. If INT8 quantization precedes pruning, subsequent weight removal disrupts calibrated scale factors and zero-points, increasing accuracy loss to 2.1 percent. Staging structural pruning and distillation before quantization stabilizes the weight distribution.

Systems lesson: In this modeled pipeline, sequenced structural reduction, distillation, and quantization-aware training produce a 16× footprint reduction (440 MB to 28 MB) while limiting task accuracy loss to 0.6 percent. Because tensor distributions and hardware kernel efficiencies vary across architectures, each intermediate artifact must be profiled on the target runtime.

Sequencing matters because these transformations interact through shared parameters and activation distributions. Structural pruning removes channels or attention heads, altering the dynamic range and variance of surviving weight tensors. If a model is quantized first, subsequent weight pruning zeros out values that determined the calibrated scaling factor and zero-point, distorting the discrete grid. Furthermore, calculating gradient updates for quantized integer weights during fine-tuning introduces severe discretization errors. Performing structural pruning in floating point first allows the network to adapt its remaining parameters. Distillation then transfers representations from the dense teacher into the student, recovering lost accuracy before quantization-aware training (QAT) locks the final parameters into discrete bins. Figure 10.30 illustrates these trade-offs across compression techniques evaluated on computer vision benchmarks (Han, Mao, et al. 2016). While singular value decomposition (SVD) and pruning alone experience steep accuracy degradation at higher reduction ratios, combining pruning with quantization achieves the highest compression ratio near baseline accuracy. The ordering and magnitude of these gains remain model- and runtime-specific, requiring engineers to validate each transformation stage independently.

A compounding footprint reduction on paper does not guarantee proportional execution speedups on hardware. Pruned dimensions can misalign with SIMD vector widths, and quantized tensors require supported hardware kernels to avoid dequantization overhead. Validating that a multi-stage optimization pipeline achieves its latency, memory, and energy targets requires systematic profiling and measurement on target silicon (section 10.8).

10.8 Efficiency Measurement

Section 10.5 traced the gap between a compression ratio on paper and the speedup a model actually realizes. That gap establishes an empirical measurement obligation. While INT8 reduces raw tensor payload by 4× relative to FP32, latency speedups do not scale proportionately. Realized throughput depends on memory hierarchy constraints, arithmetic intensity, and target kernel implementations that nominal compression ratios cannot predict. Translating theoretical compression into realized speedup requires identifying which physical bottlenecks govern each model component, establishing multidimensional baselines on target hardware, and isolating how arithmetic reductions interact with system-level overhead.

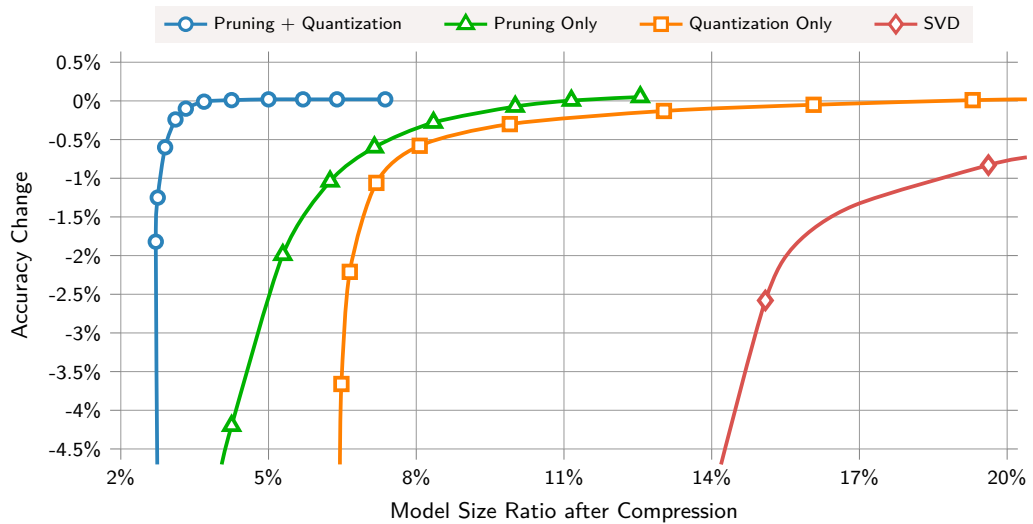


Figure 10.30: Compression Methods in One Reported Setting: The cited comparison plots accuracy change against model-size ratio for pruning, quantization, their combination, and SVD. In this setting, pruning plus quantization reaches the smallest ratio near baseline accuracy; the ordering is model- and implementation-specific. Source: (Han, Mao, et al. 2016).

10.8.1 Profiling and opportunity analysis

Profiling decomposes model execution along the physical boundaries of the accelerator: memory capacity (SRAM, DRAM/HBM), memory bus bandwidth, and arithmetic throughput. Memory allocation divides into static footprint (weight tensors and persistent buffers) and dynamic allocation (activation tensors and KV caches during the forward pass). Static footprint dictates whether model parameters fit into fast on-chip memory or device DRAM without spilling across high-latency host-accelerator interconnects such as PCIe. Dynamic allocation dictates peak memory residency, bounding maximum batch size and serving concurrency. At the operator level, profiling measures theoretical operations (FLOPs) alongside wall-clock execution time. Because operators vary across orders of magnitude in arithmetic intensity ($I = \text{FLOPs}/\text{byte}$), their execution bottlenecks diverge: compute-dense matrix multiplications saturate execution units, whereas memory-bound operators (such as layer normalization and elementwise activations) stall execution units while streaming tensors to and from memory, consuming wall-clock time disproportionate to their FLOP count.

A profile of a Vision Transformer (ViT) on an edge accelerator illustrates this operational heterogeneity. Attention projection layers account for 65 percent of total FLOPs, layer normalization consumes 8 percent of wall-clock latency despite contributing only 2 percent of FLOPs, and the final classification head accounts for only 1 percent of computation but occupies 15 percent of parameter memory. Profiling isolates the governing physical bottleneck for each component, dictating targeted optimizations rather than uniform compression: structured pruning relieves compute-bound attention projections, operator fusion eliminates intermediate global memory round-trips for memory-bound normalization layers, and INT8 quantization shrinks the static footprint of the parameter-heavy classification head.

Translating layer-level profiling metrics into end-to-end system speedup requires accounting for the complete serving pipeline. In production serving systems, model execution frequently represents only a fraction of total latency, with the remainder consumed by image decoding, tokenization, RPC deserialization, or network transfer. As the margin diagram illustrates, optimizing the model in isolation encounters an Amdahl ceiling whenever non-model stages dominate the latency budget.



Compression hits an end-to-end ceiling once non-model work dominates the pipeline.

🔍 Systems Perspective 10.5: FLOPs reduction is not proportional speedup

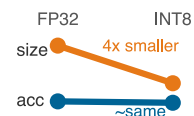
Reducing a model's FLOPs by 50 percent does not guarantee 50 percent latency reduction. Memory-bound operations see little benefit from arithmetic reduction because data movement, not multiplication, sets their time. Critically, Amdahl's Law applies at the system level (section D.2.3 derives its strong-scaling form): if model inference accounts for only 20 percent of end-to-end latency (with the remaining 80 percent spent on data loading, preprocessing, and postprocessing), then even perfect model optimization yields at most 1.25× overall speedup. Tokenization and network transfer can occupy the non-model fraction in language-model serving; image decoding, resizing, and normalization can do the same in vision pipelines. Profile the complete path on target hardware before estimating optimization benefits.

Complementing latency and throughput profiles, layer-wise sensitivity analysis quantifies how individual layers tolerate perturbation. In deep networks, parameter distributions across different layers exhibit unequal robustness to precision reduction or weight elimination. Profiling accuracy degradation while systematically perturbing one layer at a time—evaluating quantization noise or weight-removal impact—identifies which components require high-precision representations and which can absorb aggressive 4-bit quantization or structured pruning without compromising task accuracy.

10.8.2 Measuring optimization effectiveness

Evaluating compression requires balancing statistical quality against physical execution costs across the Data, Algorithm, and Machine (D-A-M) axes. A single scalar metric—such as top-1 accuracy or parameter count—fails to capture deployment viability. Quality baselines must capture aggregate accuracy, confidence calibration (verifying that predicted probabilities reflect empirical correctness), and performance across underrepresented input slices. Simultaneously, efficiency baselines must record peak memory residency, latency distributions (including tail latency p_{99}), memory bus saturation, and per-inference energy.

Quantizing ResNet-50 from FP32 to INT8 demonstrates how compression decisions ripple across physical and statistical dimensions. Rather than assessing INT8 conversion through an isolated accuracy metric, table 10.16 presents a modeled deployment profile across compute, memory, energy, and statistical calibration. As illustrated in the margin diagram, validating compression requires confirming that artifact compression does not trigger unacceptable quality degradation.



An illustrative INT8 profile must validate both size and accuracy.

Table 10.16: Illustrative ResNet-50 INT8 Deployment Profile: The values form a modeled scenario, not measurements from one hardware run. A real profile must measure accuracy, latency, artifact size, energy, and calibration on the target system.

Metric	FP32 baseline	INT8 result	Deployment reading
Top-1 accuracy	76.1%	75.8% (0.3 pp drop)	Aggregate accuracy mostly holds, but subgroup checks still matter.
Modeled V100 latency	1.56 ms	0.39 ms	GPU latency improves when the runtime maps INT8 to fast kernels.
Model size	102.4 MB	25.6 MB (4×)	The artifact becomes easier to cache, transmit, and deploy on edge.
Energy/inference	0.25 J	0.06 J (4×)	Lower precision reduces both arithmetic and memory-movement energy.
Calibration error	2.1%	3.4%	Confidence calibration can drift even when accuracy looks acceptable.

The modeled profile demonstrates that converting weights and activations to INT8 compresses the model footprint by 4× (from 102.4 MB to 25.6 MB), reducing DRAM traffic and driving a 4× reduction in per-inference energy. Modeled V100 execution latency falls from 1.56 ms to 0.39 ms, provided the runtime maps INT8 operations to native Tensor Core kernels rather than invoking runtime dequantization. Crucially, while aggregate top-1 accuracy drops by only 0.3 percentage points (from 76.1

percent to 75.8 percent), calibration error increases from 2.1 percent to 3.4 percent. Post-quantization probability distributions drift, which can compromise safety-critical confidence thresholds even when top-1 classification accuracy appears preserved. Because these values represent a modeled scenario, production deployment decisions require equivalent empirical measurements on target hardware.

Realizing these gains in production requires evaluating interactions among multiple compression techniques. Applying pruning, quantization, and knowledge distillation sequentially can produce non-additive gains or compound degradation; for example, uncalibrated weight quantization on an aggressively pruned sparse matrix can trigger catastrophic accuracy collapse. Evaluating these compounded trade-offs requires mapping models onto an efficiency-quality Pareto frontier, as developed in section 12.11.1.3. Executing this systematic profiling and multi-stage transformation pipeline in turn depends on automated software tooling that coordinates compilation, kernel generation, and hardware benchmarking.

10.9 Implementation Tools

Quantizing a 175-billion-parameter model by hand requires modifying thousands of operator call sites to insert scale factors and track dynamic activation ranges. Without automated framework tooling, even uniform INT8 post-training quantization requires manually intercepting tensor lifecycles across the computation graph, while pruning requires maintaining sparsity masks alongside weight matrices. At scale, manual graph manipulation cannot guarantee autograd consistency during retraining or enforce the memory-alignment constraints required by accelerator hardware.

Tool choice follows the layer of the software stack that owns the transformation. Framework APIs operate on the high-level computation graph during training and calibration, where the execution engine maintains visibility into weights, activations, and autograd state. At this stage, quantization is typically simulated: fake-quantization nodes clamp and round values within floating-point registers, modeling numerical precision loss while preserving differentiability. Framework tools cannot, however, realize physical execution speedups on their own. Compilers and runtime engines take over after graph export, lowering simulated operations into accelerator-native execution plans. The runtime compiler fuses scale factor arithmetic directly into matrix-multiplication compute tiles and packs sparse weights into hardware-specific metadata layouts (such as 2:4 structured sparsity formats). This lowering eliminates round-trip memory transactions to HBM that would otherwise negate the benefits of reduced precision. Software infrastructure enforces this boundary, capturing the calibration data, pruning schedules, and operator fusions that generated the deployable binary rather than relying on untracked script modifications.

Across this multi-stage pipeline, the operational imperative is reproducibility. Compressing a model alters its numerical representations across both the algorithm and machine layers of the D·A·M taxonomy. Unrecorded calibration inputs or mismatched rounding modes can degrade accuracy silently without raising runtime errors. Chapter 14 later expands this discipline into model versioning, monitoring, artifact management, and rollback procedures; here, the baseline requirement is that every deployed artifact must remain deterministically traceable from its source training checkpoint through its compiled hardware execution graph.

10.9.1 Model optimization APIs and tools

At the model-development layer, compression algorithms require access to runtime state that downstream deployment compilers discard: gradient histories, optimizer state variables, and dynamic activation distributions. When post-training quantization or one-shot pruning degrades accuracy beyond acceptable operating thresholds, recovery requires retraining or iterative fine-tuning. Frameworks such as PyTorch and TensorFlow provide optimization APIs specifically to manage this graph-transformation boundary. These APIs inject quantization nodes, reparameterize weight tensors with sparsity masks, and preserve calibration metadata alongside the trained parameters. While chapter 7 examines framework execution engines comprehensively, model compression depends on these specialized graph mutations to bridge the algorithmic loss function with machine-level execution constraints.

Quantization-aware training demonstrates this graph-level intervention. Quantizing continuous floating-point weights into discrete integers introduces a discontinuous step function whose deriva-

tive is zero almost everywhere ($d\lfloor x \rfloor / dx = 0$), halting standard gradient descent. Optimization APIs overcome this mathematical barrier by inserting fake-quantization operators into the compute graph. During the forward pass, these operators clamp values to the target dynamic range and round them to discrete integers, exposing downstream activations to low-precision truncation noise. During the backward pass, the operator implements a straight-through estimator: it treats the rounding step as an identity function, routing gradients unhindered to the underlying 32-bit master weights. Listing 10.5 details the sequence of graph transformations and metadata tracking required to produce a deployable low-precision artifact.

Listing 10.5: Quantization-Aware Training: Mechanism-level transformation that trains through simulated low-precision behavior and exports the calibration metadata needed for deployment.

```
choose target precision and calibration policy
insert quantize/dequantize boundaries around selected tensors
train with simulated low-precision values while keeping gradient flow intact
record scales, zero points, accumulator precision, and unsupported operations
export the calibrated graph and metadata as one reproducible artifact
```

A parallel graph-mutation strategy governs pruning within framework APIs. Physical hardware cannot accelerate arbitrary parameter removal during training: excising non-contiguous weights dynamically would fragment memory allocations and destroy coalesced memory accesses across SIMD execution lanes. Consequently, framework pruning APIs do not shrink tensor dimensions during fine-tuning. Instead, they reparameterize the layer by retaining the original dense weight tensor W and allocating an auxiliary binary mask tensor M of identical shape, executing the forward pass via element-wise multiplication ($W_{\text{pruned}} = W \odot M$). This design preserves dense tensor memory layouts and gradient updates during optimization. Only when training concludes does the export pass finalize the transformation, either by physically slicing away unneeded channels in structured pruning or by encoding nonzeros into sparse container formats compatible with hardware accelerators. Listing 10.6 illustrates the lifecycle of parameter scoring, masking, and export.

Listing 10.6: Pruning Artifact Boundary: Applies unstructured or structured removal rules, then records the mask and fine-tuning context needed to reproduce the compressed model.

```
choose pruning granularity: individual weights, channels, blocks, or attention heads
score candidate parameters by magnitude, sensitivity, or validation loss impact
remove or mask the selected structure according to the hardware-compatible rule
fine-tune the compressed model against the original validation target
export weights, masks, sparsity pattern, and calibration evidence together
```

The engineering value of these APIs centers on experimental reproducibility across the D·A·M hierarchy. Rather than implementing ad hoc graph manipulations, practitioners use built-in optimization modules as standardized control points. Engineers can systematically sweep sparsity ratios, quantization scaling schemes, calibration datasets, and learning-rate schedules while holding the underlying execution pipeline constant. Packaging the resulting scales, zero points, masks, and architectural modifications into unified export artifacts ensures that downstream inference runtimes can reproduce the exact numerical semantics validated during training.

10.9.2 Hardware-specific optimization libraries

Exporting an optimized model from a training framework produces an intermediate graph representation, not an executable binary. Compilers and runtime engines such as TensorRT, XLA, OpenVINO, and TVM own the final translation step: compiling a pruned, quantized, or fused graph into machine instructions tailored to the target accelerator. While chapter 11 details the microarchitectural structures these tools target, the governing systems principle is grounded in the D·A·M hierarchy: algorithmic compression remains theoretical until the graph maps to kernels that physically exploit the target machine's memory hierarchy and execution units.

The compilation pass verifies whether nominal compression yields physical speedup. Pruning removes zero-valued weights from the graph, but unless the hardware provides sparsity-aware memory layouts and execution paths—such as 2:4 structured sparsity support—matrix engines stream and multiply zeros identically to nonzero values, yielding no reduction in memory traffic or latency. Quantization lowers bitwidth, but if an accelerator lacks native INT8 or INT4 compute pipelines, the runtime must unpack operands into floating-point registers before arithmetic execution, replacing memory bottlenecks with instruction overhead. Furthermore, low-precision kernels require specialized tensor layouts, such as tiled or interleaved memory channels, to ensure coalesced loads into shared memory or register files. Operator fusion eliminates intermediate round-trips to off-chip DRAM by staging intermediate activations directly inside on-chip SRAM or register space. When an unsupported operator sequence blocks fusion, the memory-bandwidth bottleneck remains intact. Framework-level optimizations merely expose opportunities; hardware-specific compilers determine whether those opportunities survive translation into an efficient machine execution plan.

10.9.3 Diagnostic visualization

The toolchain boundary divides responsibilities: framework APIs create and calibrate the compressed model, hardware optimization libraries lower that model to the execution substrate, and diagnostic inspection verifies numerical integrity. While system-scale validation and artifact monitoring belong to downstream deployment infrastructure, the immediate diagnostic imperative is verifying that quantization clipping, pruning masks, or sparsity patterns have not perturbed internal feature distributions beyond acceptable task error tolerances.

Quantization error histograms show whether error is broadly distributed or concentrated in outliers. Activation visualizations help detect clipping, overflow, and saturation. Figure 10.31 schematically groups first-layer convolutional kernels by visual pattern. Near-zero kernels or filters with consistently negligible activations—not merely uniform-looking filters—are pruning candidates, so the diagnostic must combine weight visualization with measured activation behavior. Framework observers and debuggers can expose quantization ranges and tensor error, while compiler or runtime inspectors can show the graph and kernels that the exported model actually executes.

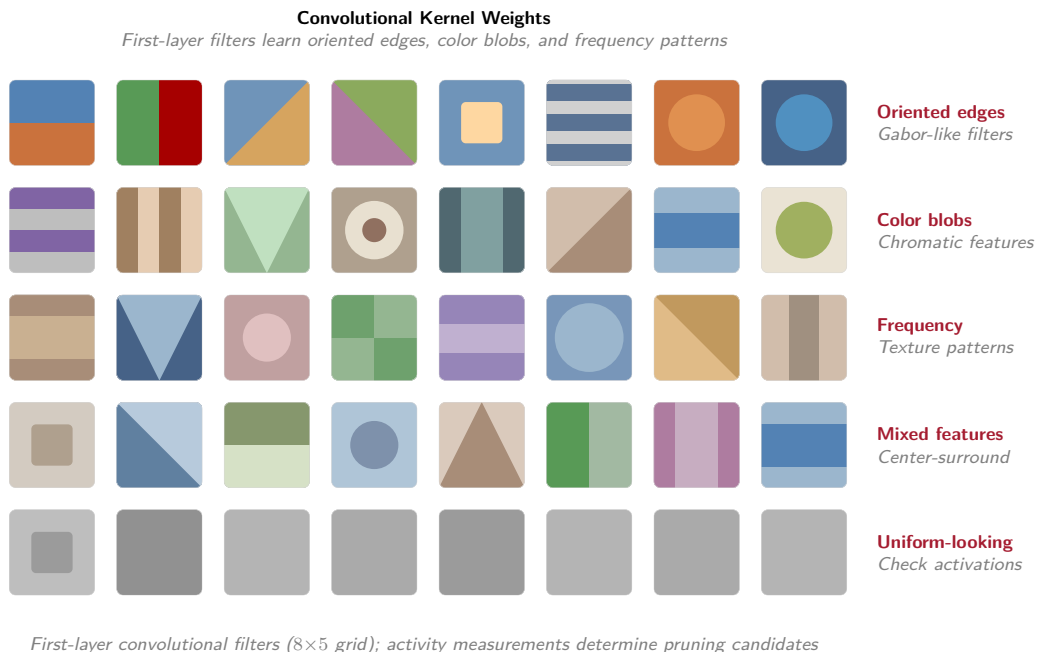


Figure 10.31: First-Layer Convolutional Filter Patterns: An 8 × 5 grid groups stylized kernels into edge, color, frequency, mixed, and uniform-looking patterns. Appearance alone does not establish inactivity; pruning requires measured activation or importance evidence in addition to low weight variance.

Sparsity diagnostics answer a deployment question: which layers actually became sparse enough for the runtime to exploit? Figure 10.32 illustrates how a layer-by-block heat map can reveal where sparsity concentrates, while trend plots track sparsity progression across pruning iterations. Kernel profiling must still determine whether the pattern yields speedup. TensorBoard, Netron, and SparseML provide these tools.

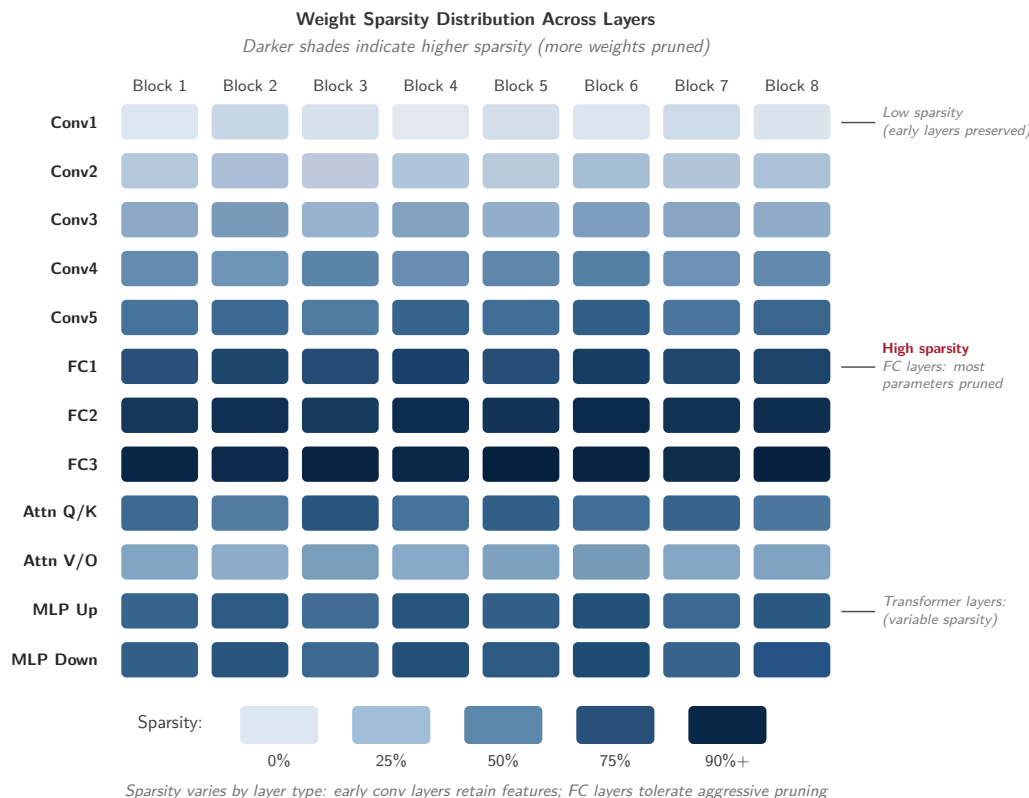


Figure 10.32: Layer-Wise Sparsity Distribution: Block-level heat map visualizing non-uniform parameter removal. Darker tiles indicate higher sparsity density, revealing which network layers retain dense features versus which layers concentrate zero-valued weights for memory saving or sparse kernel scheduling.

The diagnostic value is the nonuniformity, not the exact shade of any one cell. If later layers are much sparser than early feature extractors, the pruning policy is concentrating compression where representations are more redundant; if early layers darken first, the policy may be destroying low-level features before the model has enough depth to compensate.

Implementation tools make compression repeatable, but they do not make the trade-offs disappear. Quantization can follow pruning or distillation in a layered pipeline: pruning changes model structure or parameter count, quantization changes numerical representation, and distillation can train the smaller artifact against a teacher's behavior. These transformations can compound raw footprint reductions, but no fixed compression ratio or quality outcome transfers across models and deployments; each stage must be evaluated against the task target and profiled on the target execution path.

10.10 Fallacies and Pitfalls

The most instructive lessons in model compression often come not from what works but from what fails. The scenarios below illustrate common failure modes; exact outcomes depend on the model, compression method, workload, and target hardware.

Model optimization involves counterintuitive interactions between techniques that appear independent. Engineers often assume strategies compose linearly and that theoretical metrics predict

deployment performance, and those assumptions waste optimization effort, degrade accuracy, or miss deployment requirements despite substantial investment.

Fallacy: *Optimization techniques can be applied independently without considering their interactions.*

Engineers assume optimization strategies compose additively: 50 percent pruning plus 4× quantization yields combined benefits. In reality, techniques interact nonlinearly and compound losses. In a representative BERT-style compression scenario, pruning to 70 percent sparsity may preserve most task performance, but applying INT8 quantization afterward can lose more accuracy than QAT on the pruned model. Knowledge distillation from heavily pruned teachers can also transfer degenerate attention patterns that reduce student accuracy compared with distilling from dense teachers. As section 10.7 demonstrates, successful optimization requires coordinated application where techniques are sequenced together. Organizations that apply aggressive combinations without measuring interactions waste weeks recovering lost accuracy.

Pitfall: *Optimizing for theoretical metrics rather than actual deployment performance.*

Teams can reduce FLOPs substantially without realizing proportional latency gains. In one hypothetical trace, a pruned model with 40 percent fewer parameters achieves only 12 percent latency reduction because irregular sparsity prevents efficient execution. A second scenario reduces a transformer from 440 MB to 110 MB, yet conversion overhead on hardware without a fast low-precision path erodes the latency benefit. These values illustrate failure modes rather than report ARM or GPU measurements. Memory bandwidth, cache behavior, kernels, and instruction-level parallelism determine actual performance, so production deployments require wall-clock measurements on target hardware.

Fallacy: *Aggressive quantization maintains model performance with minimal accuracy loss.*

Engineers assume quantization error scales uniformly with bit width. In practice, precision reduction can exhibit threshold effects, and the threshold depends on the model, layer, quantizer, calibration, and training method. The ResNet and BERT values in this chapter are illustrative rather than universal degradation bands. Even operations such as LayerNorm and Softmax can be implemented with integer approximations, so they do not universally require FP16; whether a low-precision implementation preserves task quality must be measured. As section 10.4.4 demonstrates, mixed-precision approaches can retain higher precision where uniform quantization fails.

Pitfall: *Defaulting to FP32 everywhere to avoid quantization risk.*

FP32 uses twice the raw storage and bandwidth of BF16, but lower-precision execution does not automatically preserve convergence or accuracy. Mixed-precision training often retains FP32 accumulators or master weights and may require loss scaling, while inference precision must be validated per model and operator. The right question is what lowest precision preserves quality on the evaluation distribution and maps efficiently to the target hardware.

Fallacy: *Post-training optimization is always enough.*

Teams often begin with post-training quantization (PTQ) because it avoids retraining. If the resulting artifact meets its task and deployment thresholds, that simple path may be sufficient. When it does not, quantization-aware training (QAT) can recover part of the quality gap by adapting the model to simulated low-precision behavior. Post-training pruning and post-hoc distillation can likewise underperform training-aware schedules at aggressive sparsity or bit width. As detailed in section 10.4.4, the production threshold—not a presumption that either path is universally superior—governs the decision.

Pitfall: *Assuming compression ratios translate directly into proportional deployment gains.*

Teams may obtain a 4× raw weight-payload reduction through INT8 quantization and expect the same deployment gain. In practice, scales, metadata, unsupported operators, conversions, and sparse indexing erode the benefit. The chapter’s 18.8 percent end-to-end improvement versus an 8× paper target is a hypothetical trace, not a BERT benchmark. Production workflows must profile deployed latency on target hardware rather than extrapolate from compression ratios.

Fallacy: *Sparse matrices always save memory.*

Sparse formats add index-array metadata that erodes savings at moderate densities. With FP32 values and INT32 indices, CSR approaches break-even near 50 percent density before row-pointer overhead, while COO stores additional coordinates and crosses over at a lower density. Performance has no universal sparsity threshold because tensor shape, format, kernel, batch, and hardware determine whether sparse execution outruns dense execution. Specialized formats such as NVIDIA's 2:4 sparsity reduce metadata and provide a supported execution path, while arbitrary unstructured sparsity may deliver neither memory nor latency savings on commodity hardware (see section 10.8.1).

Pitfall: *Choosing sparse storage before checking the density threshold.*

Teams sometimes convert tensors to sparse formats as soon as pruning creates visible zeros. That conversion can make the system slower and larger if metadata, gather/scatter overhead, and poor cache locality outweigh the saved values. A production compression pass should measure the realized density, choose a sparse format only after the break-even point is crossed, and prefer structured sparsity when the target hardware has kernels that can exploit it.

10.11 Summary

Model compression is an engineering discipline built on three complementary dimensions: *structural optimization* determines what the model computes, *precision optimization* determines how precisely it computes, and *architectural optimization* determines how efficiently those computations execute on physical hardware. These dimensions can compound, but their gains do not multiply automatically. Pruning, distillation, and quantization together produce the illustrative pipeline's 16 $\times$ footprint reduction from 440 MB to 28 MB; metadata, unsupported operators, and hardware mapping determine the realized deployment gain. A parameter reduction approaches the same latency reduction only when the affected work is on the critical path and the runtime can exploit the new representation. Profile on target hardware, not paper metrics.

Combined with the data selection techniques from chapter 9, these model-centric optimizations complete the model-side efficiency toolkit: data selection maximizes learning from available examples, while model compression minimizes resources required for deployment. Whether those savings become real speedups still depends on the hardware and runtime that execute the compressed model.



Key Takeaways: From benchmark winner to production model

- **Compression spends surplus capacity:** Production models trade unused parameters, precision, and capacity for latency, memory, power, or cost limits the deployment cannot violate. The target is the smallest artifact that preserves required task behavior under deployment constraints.
- **Savings multiply only when aligned:** Structural pruning, distillation, quantization, and architecture changes can compound raw footprint reductions, as the modeled BERT mobile pipeline's 16 $\times$ ratio illustrates. The gain becomes real only when task quality holds and the resulting operators match the target runtime and accelerator.
- **Precision is a deployment contract:** Where the target runtime supports it, INT8 post-training quantization is a useful first experiment because it reduces raw FP32 weight payload by 4 $\times$ without retraining. QAT, distillation, or mixed precision become candidates when calibration or layer sensitivity exposes unacceptable error.
- **Hardware sets the exchange rate:** Unstructured sparsity and theoretical FLOP cuts do not imply latency gains unless kernels and memory layouts can exploit them. The chapter's hypothetical 8 $\times$ paper target vs. 1.5 $\times$ realized outcome illustrates why target-hardware profiling is mandatory.
- **End-to-end latency caps model wins:** Compression is valuable only on the critical path. When inference is 20 percent of total request latency, Amdahl's law caps even perfect model acceleration at 1.25 $\times$, so preprocessing, dispatch, and data movement may be the true optimization target.

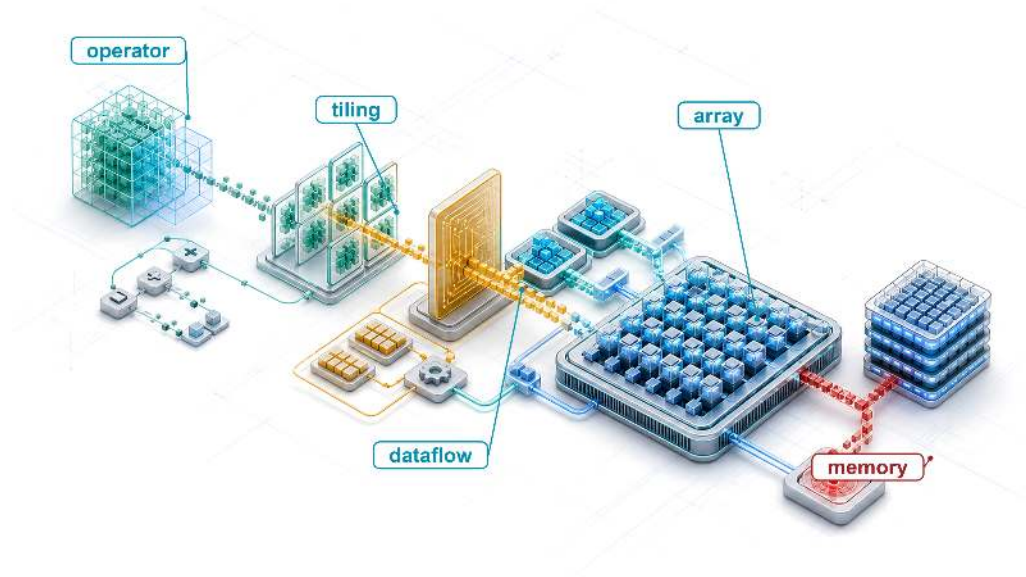
The techniques in this chapter differ in almost every detail, yet one rule runs beneath all of them. Each buys a smaller, faster, or cooler model by spending something else: pruning spends capacity, quantization spends numerical precision, and distillation spends training compute. Where the model holds no surplus to spend, the bill may be paid in task quality. Compression does not make the work disappear; it moves cost or risk from one part of the system to another, and the hardware sets the exchange rate. A technique that pays off on a phone can therefore be worthless on a data-center GPU. This is the conservation-of-complexity heuristic at the level of a single model. A production artifact is a model whose structure, representation, and execution path have been engineered together until the trade balances against the silicon and application constraints that govern deployment.



What's Next: From math to physics

These optimization techniques compress the model's logic, reducing the work the algorithm asks the machine to perform. Logic, however, must eventually run on physical silicon. Chapter 11 examines how GPUs, TPUs, and NPUs—with their systolic arrays, Sparse Tensor Cores, and tiered SRAM/HBM memory hierarchies—map supported compressed representations onto the throughput and data movement available in hardware.

Hardware Acceleration

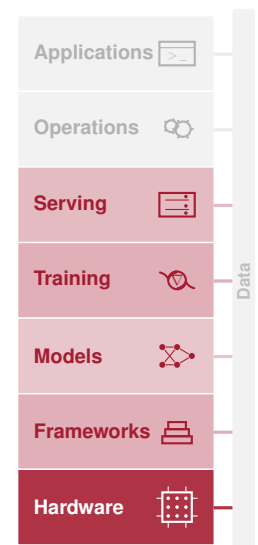


11.1	Acceleration Fundamentals
11.2	Hardware Specialization
11.3	AI Compute Primitives
11.4	Compute Units and Execution Models
11.5	AI Memory Systems
11.6	Roofline Model
11.7	Hardware Mapping
11.8	Dataflow Optimization
11.9	Compiler Support
11.10	Runtime Support
11.11	Multi-Chip Scaling
11.12	Heterogeneous SoC Design
11.13	Hardware Sustainability
11.14	Fallacies and Pitfalls
11.15	Summary

Purpose

Why does moving data cost more than computing it?

Arithmetic is inexpensive relative to many memory accesses. In the time it takes to fetch a value from main memory, a processor can perform many calculations. This imbalance, the “memory wall,” reflects the latency, bandwidth, and energy costs of moving data across a system. It explains why specialized accelerators are not merely faster at math but are architected to hide, amortize, and reduce the cost of moving data through deep memory hierarchies, massive parallelism, and specialized data paths. Hardware acceleration allows many large AI workloads to grow when general-purpose processor scaling alone is insufficient. It also explains why some optimizations that reduce theoretical computation fail to improve runtime. If an operation is memory bound, computing less may not help because computation is not the bottleneck. Hardware selection therefore cannot be reduced to comparing peak FLOP/s. What matters is whether a workload’s data movement patterns align with what the hardware was designed to accelerate. A model with large embedding tables and irregular lookups needs a different accelerator than one performing dense matrix multiplications over compact weight tensors. Software must expose and schedule the available reuse; otherwise specialized units wait while data moves and headline throughput remains theoretical. Matching workload and hardware determines whether execution remains at a fraction of theoretical peak or approaches the hardware’s practical ceiling. In D·A·M terms, the accelerator makes the machine constraint concrete and shapes which algorithms remain practical in production.

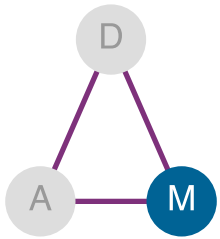




Learning Objectives

- Explain hardware acceleration as machine-axis specialization for tensor workloads, data reuse, and performance per watt
- Calculate arithmetic intensity and roofline ceilings to classify kernels as compute bound or memory bound
- Diagnose memory-wall bottlenecks using bandwidth, cache hierarchy, host-device transfer, and energy-movement costs
- Compare Tensor Cores, systolic arrays, SIMD/SIMT units, and sparse execution for ML compute primitives
- Select dataflow, tiling, and mapping strategies that maximize reuse under memory-capacity constraints
- Analyze compiler and runtime optimizations that fuse kernels, plan memory, and schedule accelerators
- Evaluate accelerator choices across throughput, latency, power, cost, and deployment-context constraints

11.1 Acceleration Fundamentals



Hardware acceleration turns on the machine axis.

REDUCING parameters, precision, or operations only matters when the machine can execute the resulting representation efficiently. Data selection reduced the data term, and compression reduced the algorithm's work; hardware acceleration asks what the machine can actually deliver. The answer starts with the memory wall: arithmetic is often cheap relative to data movement. An off-chip memory access can take many processor cycles and consume orders of magnitude more energy than a low-precision arithmetic operation, with the exact ratio depending on the device and memory level. Specialized hardware matters because it raises compute throughput while organizing memory, dataflow, and parallelism so those arithmetic units stay fed.



Definition 11.1: Hardware acceleration

Hardware acceleration is the practice of mapping work from general-purpose execution onto specialized processors or functional units optimized for a narrower operation class. For ML workloads, that specialization commonly trades some programmability for the compute density (R_{peak}) and performance-per-watt gains that regular tensor operations can exploit.

1. **Significance:** An A100 GPU delivers 312 TFLOP/s for FP16/BF16 matrix multiplication; against this chapter's illustrative 1–2 TFLOP/s reference-CPU baseline, this is a 156–312× gap. Its 54.2 billion transistors devote far more silicon to parallel arithmetic than to branch prediction, out-of-order scheduling, and large caches (NVIDIA Corporation 2020a; Choquette et al. 2021).
2. **Distinction:** A general-purpose CPU is designed for flexible execution, low-latency control, and varied workloads. An accelerator instead concentrates resources on a narrower operation class, so it achieves its gains only when the workload exposes the supported operations and enough parallel work to keep its arithmetic units busy.
3. **Common pitfall:** A frequent misconception is that an accelerator's advertised peak throughput is the throughput a workload receives. Delivered performance is the lower of the compute ceiling and what memory bandwidth can feed, the roofline constraint: a low-arithmetic-intensity kernel can sit at a small fraction of peak FLOP/s no matter how fast the silicon is rated, because it starves for data rather than for arithmetic.

This definition frames the chapter's central engineering trade-off. General-purpose processors allocate silicon area to complex control logic, speculative execution, branch prediction, and multi-level cache hierarchies to handle arbitrary control flow. An accelerator redirects that transistor budget toward dense arithmetic arrays, dedicated register files, and high-bandwidth memory paths

tailored to a narrower workload class. When the software path exposes sufficient tensor parallelism, this specialization yields order-of-magnitude improvements in energy efficiency and throughput per watt.

Hardware alone cannot achieve these gains. Algorithms must structure operations to expose data reuse and regular memory access, while silicon architectures must allocate physical units to the mathematical primitives and numeric precisions that algorithms actually execute. This symbiosis motivates a complementary principle: hardware-software co-design.

Definition 11.2: Hardware-software co-design

Hardware-Software Co-design is an ML accelerator development method that crosses conventional abstraction boundaries, allowing algorithm constraints to inform silicon design and hardware capabilities to shape algorithm formulation.

1. **Significance:** On a supported tensor-core path, 8-bit integer (INT8) quantization can deliver multi-fold throughput gains because hardware executes lower-precision operations more densely and moves fewer bytes than an FP32 path; the exact gain depends on the architecture and workload (NVIDIA Corporation 2020a; Dally et al. 2021; Dally 2023).
2. **Distinction:** Unlike layered abstraction (where software calls a hardware API without knowing the silicon details), co-design exposes hardware constraints directly to algorithm and compiler authors: data alignment requirements, precision formats, and memory access patterns all become visible inputs to global cross-layer optimization.
3. **Common pitfall:** A frequent misconception is that co-design is a one-time hardware design choice. In practice, the supported features continue to evolve: NVIDIA Tensor Cores began with FP16 matrix multiplication, later generations added TF32 and broader integer support, and Ampere added acceleration for 2:4 structured sparsity (NVIDIA 2017; NVIDIA Corporation 2020a).

Co-design explains why the compression techniques introduced in chapter 10 deliver real speedups. The quantization techniques in section 10.4 show why converting FP32 to INT8 accelerates execution on supported hardware: reduced precision cuts memory traffic, while specialized arithmetic units execute more low-precision operations per cycle than FP32 equivalents (NVIDIA Corporation 2020a). Structured pruning improves performance when it produces smaller dense shapes or a sparsity pattern recognized by silicon, whereas arbitrary unstructured sparsity incurs metadata handling and non-coalesced memory accesses that often erase theoretical gains. Evaluating accelerator performance requires tracing this path from workload to silicon: compute primitives, memory hierarchies, roofline diagnosis, mapping and dataflow, and compiler scheduling. The recurring question is why algorithmic improvements frequently fail to deliver proportional end-to-end speedups. Amdahl's law provides the governing limit.

Theorem 11.1: The fundamental limit of acceleration (Amdahl's Law)

Hardware acceleration *does not speed up the entire system*; it speeds only the baseline-time fraction using the accelerated path. For that fraction p and path gain G_{accel} , **Amdahl's law for AI** (Amdahl 1967) gives the bound in equation 11.1.

$$\text{Speedup} = \frac{1}{(1-p) + \frac{p}{G_{\text{accel}}}} \quad (11.1)$$

- **Accelerated fraction** (p): Share of baseline time receiving the stated gain, often in matrix multiplication or another supported kernel.
- **Accelerator gain** (G_{accel}): The raw speed advantage of the GPU or Tensor Processing Unit (TPU) over the CPU for the accelerated portion of the workload.

- **Unaccelerated fraction** ($1 - p$): Work outside the chosen path, such as data loading, host overhead, unsupported operators, and kernel launch latency.

Pitfall: *Unaccelerated work caps total speedup.* If 10 percent of baseline time is unaffected ($p = 0.9$), even infinite path gain ($G_{\text{accel}} = \infty$) yields at most $10\times$ overall. Once the accelerated component is fast enough, unaffected work dominates.

Hardware acceleration targets specific terms in the iron law of ML systems (section 1.7), which decomposes end-to-end time into data volume (D_{vol}/BW), computation ($O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$), and fixed latency (L_{lat}). While data selection reduced total data volume and model compression reduced operations O per sample, hardware acceleration increases the rate of execution by improving R_{peak} , η_{hw} , and BW. Section D.2 supplies the analytical performance models that diagnose which of these terms dominates a given workload, including the dimensional analysis that confirms each iron law term resolves to seconds. Yet acceleration encounters a hard ceiling established by Amdahl's law.¹

This dynamic explains why hardware upgrades frequently fail to produce proportional end-to-end speedups. Figure 11.1 visualizes this **acceleration wall**: as long as an unaccelerated fraction $1 - p$ remains, the asymptotic ceiling $1/(1 - p)$ enforces sharp diminishing returns on path gain G_{accel} unless p approaches unity.

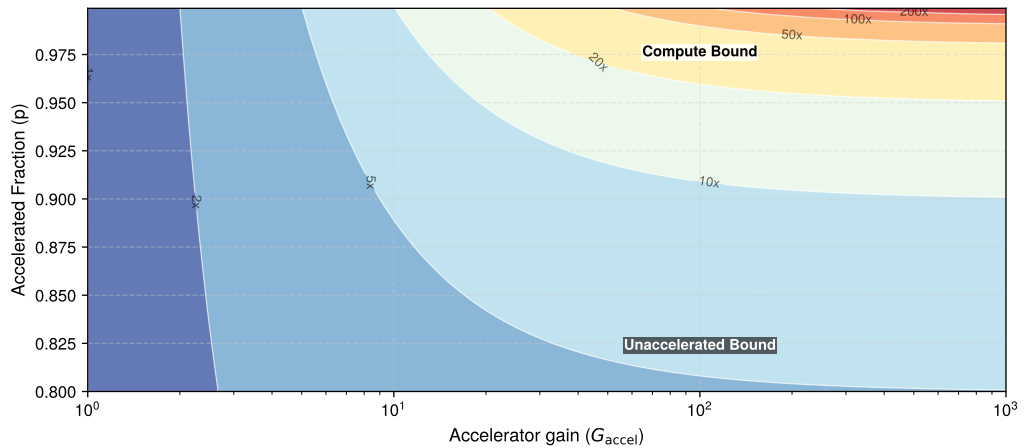


Figure 11.1: The Iron Law Heatmap: Total system speedup as a function of accelerator gain (G_{accel}) and accelerated fraction (p). High speedup appears only near the top-right corner, where both accelerator gain and accelerated fraction are high. At $p = 0.9$, for example, even an infinite gain on the accelerated path cannot exceed $10\times$ end-to-end speedup. Contours span roughly $1\times$ – $200\times$ speedup.

Raw accelerator speedup translates into system-level throughput only after the unaccelerated fraction has been minimized. The accelerated fraction p differs substantially across workload archetypes running on identical hardware, determining whether an accelerator investment pays off or stalls on non-matrix operations.

Checkpoint 11.1: The parallelism gate

These checks turn Amdahl's bound into a hardware-selection test.

Amdahl's reality

- ☐ **Serial bottlenecks:** use Amdahl's bound to explain why a $1,000\times$ faster GPU may only speed up training by $5\times$ when data loading is slow.
- ☐ **Speedup ceiling:** for $p = 0.95$ and $G_{\text{accel}} = 100$, estimate total speedup and compare it with the infinite-accelerator ceiling.

¹ | **Amdahl's law:** Maps onto the iron law only after the proposed change is specified. If a faster tensor unit reduces the computation term ($O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$) but leaves data transfer and fixed latency unchanged, those unaffected terms bound total speedup. A different accelerator or memory system may improve bandwidth as well. The unaccelerated fraction is therefore defined by the intervention, not by a permanent classification of each term as serial.

- **Workload variation:** compare ResNet-50 and MobileNet accelerated fractions for a proposed hardware change, then predict which benefits more.

The serial bottleneck becomes concrete on real hardware, where the same accelerator that nears its parallel ceiling on one workload can stall on another. Section D.1 collects the reference R_{peak} figures across accelerator generations and the latency hierarchy that ground these hardware comparisons in order-of-magnitude terms. Evaluating an NVIDIA H100 running compute-bound convolution versus memory-bound autoregressive decoding illustrates how differing unaccelerated fractions govern realized speedup.



Lighthouse 11.1: An illustrative Amdahl budget on H100

Illustrative ResNet-50 inference on NVIDIA H100:

- Assume H100 provides $G_{\text{accel}} = 247\times$ on matrix multiplication relative to a reference CPU without matrix extensions (1979 TOPS INT8 vs. ~ 8 TOPS) (Choquette 2023).
- Assume $p = 0.95$: 95 percent of baseline time receives that gain, while 5 percent remains in data loading, preprocessing, postprocessing, and other work.

$$\text{Speedup} = \frac{1}{(1 - 0.95) + \frac{0.95}{247}} = \frac{1}{0.05 + 0.0038} \approx 18.6\times$$

The $247\times$ path gain produces only $18.6\times$ end-to-end speedup because the 5 percent unaffected fraction sets the ceiling.

Contrast with GPT-2 (autoregressive):

- For GPT-2 token generation, assume $p = 0.80$; the remaining 20 percent includes host overhead, sampling, and decoding work outside the accelerated matrix path.

$$\text{Speedup} = \frac{1}{(1 - 0.80) + \frac{0.80}{247}} = \frac{1}{0.20 + 0.0032} \approx 4.9\times$$

Less token-generation time receives the assumed gain, so even an infinite path gain yields at most $1/(1 - p) = 5\times$. Serving optimizations address this limit differently: batching improves cross-request utilization, while speculative decoding uses a draft model to reduce sequential target-model steps.

Whether an operation is limited by compute rate or by data movement dictates both hardware selection and software optimization. Upgrading to an accelerator with $10\times$ higher peak compute yields negligible speedup if data delivery is the binding bottleneck. The Roofline Model provides the analytical framework for making this diagnosis (Williams et al. 2009); it is introduced formally in section D.2.1 and section 11.6 applies it to AI workloads. It plots an operation's arithmetic intensity,² defined as the ratio of floating-point operations to bytes of memory traffic (FLOP/byte), against hardware capabilities, revealing whether performance is capped by compute or bandwidth. A dense matrix multiplication with high arithmetic intensity benefits from additional TFLOP/s; a LayerNorm with low arithmetic intensity benefits only from greater memory bandwidth. High-reuse ResNet-50 convolutions operate in the compute-bound regime, while low-batch autoregressive attention operates in the memory-bound regime. Consequently, compute-bound workloads require strategies that maximize arithmetic throughput and functional-unit utilization, whereas memory-bound kernels require operator fusion and memory-hierarchy staging to minimize off-chip traffic.

Architectural design begins with the physical constraints that forced specialization: the breakdown of classical Dennard voltage scaling and the prohibitive energy cost of moving data across memory hierarchies. When general-purpose microarchitectures encountered these thermal and memory barriers, domain-specific hardware specialization emerged as the primary engineering path to sustain performance scaling.

² | **Arithmetic intensity:** The ratio of compute operations to bytes transferred across the memory interface being modeled (FLOP/byte). Workloads above that hardware level's ridge point, such as well-tiled matrix multiplications with substantial reuse, are compute bound and can benefit from more TFLOP/s. Low-intensity operations, including some low-batch attention and decoding kernels, are bandwidth bound at that level and need more bandwidth or reuse rather than only more compute throughput.

11.2 Hardware Specialization

When Google deployed its first Tensor Processing Unit (TPUv1) into production data centers alongside NVIDIA K80 GPUs, the comparison highlighted a fundamental efficiency divide. General-purpose processors dedicate substantial silicon area and power budgets to instruction fetching, branch prediction, and cache coherence hardware designed for arbitrary code. For deep learning workloads dominated by regular, dense linear algebra, that control overhead yields poor arithmetic density. When an algorithmic pattern becomes sufficiently dominant and structured, specialized hardware outperforms general-purpose platforms by replacing dynamic control machinery with dedicated arithmetic arrays and software-managed data movement. Google's production inference evaluation quantified this efficiency gap (example 11.1).



Example 11.1: The TPUv1 vs. K80 efficiency shock

Scenario: In 2015, Google deployed its first Tensor Processing Unit (TPUv1) into production data centers, comparing its throughput and energy efficiency against general-purpose NVIDIA K80 GPUs (Jouppi et al. 2017).

Diagnosis: The K80 was a programmable GPU designed to support a broader mix of graphics and compute workloads. The TPUv1 domain-specific architecture (DSA) made a narrower inference-focused bet, pairing a large 8-bit systolic matrix unit with software-managed memory and simpler control.

Systems lesson: On the six Google inference workloads and contemporary CPU/GPU baselines evaluated in the TPU paper, tailoring silicon to the dominant matrix operations delivered 15–30× higher throughput and 30–80× better performance per watt (Jouppi et al. 2017). Those ratios describe the evaluated systems, not every workload.

In D-A-M terms, this efficiency gain stems from tailoring machine resources to algorithmic structure. By replacing hardware caches with software-managed scratchpad buffers and routing data directly between neighboring arithmetic units in a systolic array, specialized architectures maximize arithmetic intensity and minimize off-chip memory traffic. Yet this specialization imposes an architectural trade-off: rigid execution pipelines forfeit flexibility. When workloads introduce irregular memory access, dynamic control flow, or unsupported numeric precisions, fixed-function data paths stall, leaving expensive silicon idle.

Modern ML accelerators—including DianNao-class neural-network accelerators (Chen et al. 2014), GPUs with Tensor Cores, Google's TPUs,³ and Apple's Neural Engine—reflect this continuous balancing act between arithmetic specialization and programmable flexibility. Their designs emerge from a historical trajectory spanning four distinct phases: specialized computing origins, parallel graphics processing, domain-specific architectures, and the emergence of ML-specific hardware. Each phase addressed the specific resource bottleneck that constrained scaling in its era.

11.2.1 Specialized computing

Hardware specialization emerges when specific computational patterns become the primary system bottleneck, preventing general-purpose processors from scaling efficiently. This history highlights three recurring limits: slow implementation of scalar floating-point, insufficient throughput for parallel graphics, and poor integration between memory and parallel compute.

The first phase, the precision bottleneck, occurred when scientific and engineering applications required floating-point arithmetic on microprocessors with integer-only arithmetic logic units. In the late 1970s, CPUs emulated floating-point operations through multi-instruction software routines that unpacked sign, exponent, and mantissa fields, shifted bits to align binary points, and normalized the output. This software emulation imposed severe scalar instruction overhead for even a single multiplication, motivating the first major wave of hardware specialization: the mathematics coprocessor.

The Intel 8087 (1980)⁴ addressed this bottleneck by executing floating-point instructions directly on dedicated hardware pipelines (Palmer 1980). Replacing multi-instruction emulation routines with hardwired arithmetic logic established a core systems principle: when an inner loop is dominated

³ | **TPU (tensor processing unit):** The first TPU made a deliberately narrow bet, centering the design on a single 256×256 systolic array for 8-bit matrix multiplication and avoiding much of the control machinery required by a general-purpose core (Jouppi et al. 2017). That trade buys high compute density on dense matrix multiplication at the cost of flexibility. Irregular or branch-heavy code remains a poor fit, and compiler tiling, padding, and layer dimensions determine how fully the array is used.

⁴ | **Intel 8087:** The coprocessor implemented floating-point logic directly in silicon, avoiding the CPU's slow, multi-instruction software emulation for each calculation. Its benefit depended on how much time an application spent in supported floating-point operations, illustrating why specialization helps most when the accelerated work dominates execution.

by a specific operation, specialized silicon eliminates general-purpose instruction overhead and shifts the execution ceiling.

As specialized functions demonstrated their utility, they followed a recurring pattern of integration. The Intel 486DX (1989) moved the FPU directly onto the CPU die, eliminating off-chip bus communication latency and making high-precision math a standard feature rather than an optional coprocessor (Hennessy and Patterson 2017). This cycle—specialization to resolve an acute bottleneck followed by architectural integration—recurs across hardware eras.

Figure 11.2 traces this progression across five eras, illustrating how recurring computational bottlenecks drove hardware evolution from floating-point coprocessors and digital signal processors to programmable GPUs, streaming media codecs, and deep-learning tensor accelerators. Modern machine learning workloads depend directly on principles established across these eras: dedicating silicon to dominant matrix operations, staging operands in local memories, and minimizing off-chip data movement.

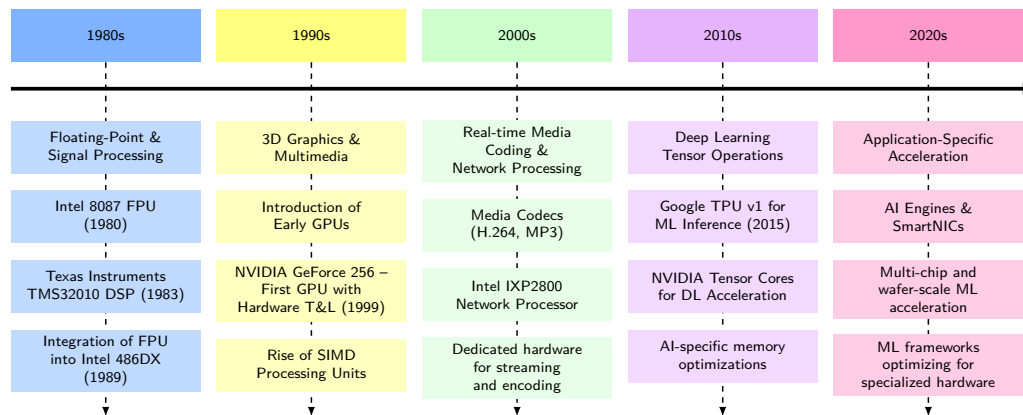


Figure 11.2: Hardware Specialization Timeline: Computing architectures evolve through recurring cycles of specialization and integration across five eras. Each era introduces domain-specific hardware to resolve an important memory or arithmetic bottleneck—from off-chip FPUs to GPUs, codecs, and TPU/Tensor Core array units. Many of these capabilities later appear alongside CPUs in broader systems, even when the specialized units remain distinct.

11.2.2 Parallel computing and graphics processing

General-purpose CPUs allocate most of their silicon budget to control logic, branch prediction, and multi-level caches designed to minimize execution latency for a single instruction stream. When a workload exhibits massive, uniform data parallelism—applying identical arithmetic across millions of independent elements—this latency-oriented architecture expends chip area and thermal budget on control structures that do not increase computational throughput. Specialized accelerators resolve this imbalance by reallocating silicon from cache and control to dense arrays of arithmetic logic units (ALUs), prioritizing aggregate throughput over single-thread latency.

Real-time 3D graphics drove this throughput-oriented specialization throughout the 1990s. Early graphics accelerators offloaded fixed rasterization tasks such as bitmap transfers and polygon filling. In 1999, NVIDIA's GeForce 256 integrated hardware-accelerated transform and lighting (T&L), shifting coordinate transformations from the host CPU to dedicated on-chip silicon. While not yet programmable, these initial Graphics Processing Units (GPUs) demonstrated how specialized pipelines could accelerate fixed mathematical operations like vertex transformation and texture mapping. However, rigid fixed-function pipelines caused severe load imbalances: complex geometries left rasterization hardware idle, while dense fragment shading starved vertex pipelines. The transition to programmable shaders in the GeForce 3 (2001) and unified shader architectures in the GeForce 8 (2006) resolved this imbalance by replacing segregated functional units with arrays of identical, programmable compute cores. This structural unification decoupled the hardware from fixed graphics stages and established the GPU as a general-purpose parallel processor capable of dense matrix arithmetic. By 2004, high-end GPUs could process over 100 million polygons per second (Owens et al. 2008).

5 | AlexNet: Krizhevsky, Sutskever, and Hinton's 60-million-parameter convolutional neural network (CNN) won ImageNet 2012 by more than ten percentage points on two consumer GTX 580 GPUs with only 3 GB of VRAM each. Because the model exceeded single-GPU memory, the authors partitioned the network across two cards and limited communication between them, an early form of model parallelism that foreshadowed systematic tensor and pipeline parallelism. The reported training run took five to six days (Krizhevsky et al. 2012).

6 | DSA (domain-specific architecture): Hardware optimized for an application domain, sacrificing some general-purpose programmability for efficiency. Google's TPUv1 achieved 15–30× better performance and 30–80× better performance per watt than the contemporary CPU and GPU systems evaluated on Google's inference workloads by centering its design on a systolic array and software-managed memory (Jouppi et al. 2017). A DSA that excels at dense matrix multiplication may perform worse than a CPU on irregular workloads such as graph traversal, so the workload limits and efficiency gain must justify the software, compiler, and ecosystem cost (Hennessy and Patterson 2019, 2017).

7 | Moore's law: In the illustrative assumptions used by figure 11.3, model compute demand grows roughly 6.1× per year while accelerator peak supply improves roughly 1.7× per year, widening the modeled demand/supply gap by about 3.5× per year. Published compute trends motivate the divergence, but the plotted rates are assumptions rather than forecasts. The figure therefore shows how sensitive the systems gap is to sustained differences in growth rates, not a fixed future trajectory. Under such a gap, algorithmic efficiency techniques such as compression, quantization, and sparsity become increasingly valuable (Amodei and Hernandez 2018a; Epoch AI 2024).

In parallel with graphics hardware, Digital Signal Processing (DSP) processors addressed a different physical constraint: deterministic, low-latency execution within tight energy budgets. General-purpose cache hierarchies introduce variable latency whenever cache misses occur, violating strict real-time deadlines in signal processing. DSPs replaced unpredictable caches with software-managed scratchpad memories and paired specialized multiply-accumulate (MAC) units with circular addressing buffers to execute continuous filtering and Fourier transforms without memory-stall jitter. Texas Instruments' TMS32010 (1983) demonstrated how domain-specific instruction sets and dedicated MAC data paths could outpace general-purpose microprocessors on vector operations (Lyons 2011).

Network processing confronted an even tighter latency deadline: sustaining packet inspection at line rate. When packets arrive every few nanoseconds, an off-chip memory access stall drops packets. Intel's IXP2800 network processor shows the consequence of this hard constraint: meeting line-rate packet deadlines leaves no slack for cache misses, so the design arranges many parallel cores around tiered on-chip memory to keep data adjacent to compute. That compute-near-memory organization, forced by packet arrival deadlines, prefigured the memory hierarchies that modern machine learning accelerators use to keep processing-element arrays continuously fed with tensor tiles.

These domains diverged in implementation because their computational profiles differed. DSPs optimized for low-latency 1D signal streams under milliwatt thermal envelopes, while network processors optimized for packet routing with minimal state retention. In contrast, deep learning workloads are dominated by dense matrix multiplications and multidimensional convolutions. These operations exhibit high arithmetic intensity and massive data parallelism. The unified GPU provided the ideal machine match: thousands of parallel ALU lanes backed by wide memory buses, using hardware-managed multithreading to hide off-chip memory latency during dense tensor operations.

The practical impact of this architectural alignment became clear in 2012, when AlexNet⁵ (Krizhevsky et al. 2012) won the ImageNet competition by more than ten percentage points. Training was performed on two consumer-grade NVIDIA GTX 580 graphics cards, each with only 3 GB of VRAM. The underlying algorithm—convolutional networks trained with gradient descent—had existed for decades, but CPU compute throughput made large-scale training computationally impractical. By mapping dense 2D convolutions directly to GPU execution units, the authors compressed months of CPU computation into five to six days. In D·A·M terms, the machine constraint had previously dictated which algorithms were feasible; aligning the parallel structure of convolutions with throughput-oriented GPU hardware made deep models practical to train on large datasets.

11.2.3 Emergence of domain-specific architectures

These diverse acceleration patterns converged in a broader architectural shift. The emergence of **domain-specific architectures (DSAs)**⁶ marks a transition in computer system design, driven by the slowing benefits of traditional scaling (Esmaeilzadeh et al. 2011) and the increasing computational demands of specialized workloads. Moore's Law⁷ described a long-running increase in transistor density (Moore 1998). Dennard scaling⁸ (Dennard et al. 1974) allowed voltage to fall as transistors shrank; its breakdown removed the accompanying path to routine frequency gains at constant power density. Together, these shifts constrained general-purpose performance and efficiency. As Hennessy and Patterson (2019) argued in the 2017 Turing Lecture, the response was a new era of domain-specific solutions optimized for important workloads.

The scale of this challenge becomes stark in figure 11.3, which plots an illustrative systems gap between model demand and hardware supply, sometimes discussed under the informal label Huang's Law.⁹ The normalized scenario assumes GPU supply rises about 1.7× per year while model demand rises about 6× per year, producing a gap that widens by roughly 3–4× each year. Published compute trends motivate the qualitative divergence, while the plotted rates are scenario assumptions rather than a forecast (Amodei and Hernandez 2018a; Epoch AI 2024).

The plot is normalized to a 2012 baseline to emphasize relative growth. The purple-shaded region highlights this divergence: the systems gap cannot be closed by process node advances alone, but requires architectural innovation, parallel distribution, and hardware-software co-design.

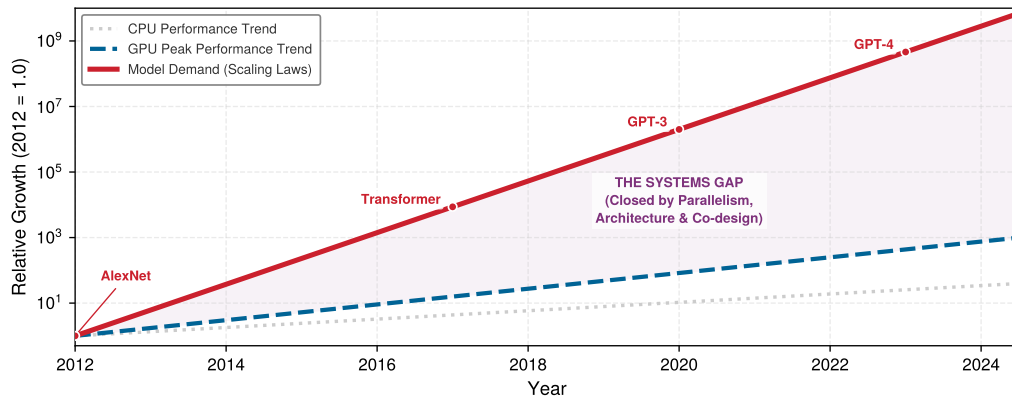


Figure 11.3: The Systems Gap: Relative compute growth (log scale) comparing model demand to hardware supply, normalized to 2012 = 1.0. The gray dotted line (CPU) and blue dashed line (GPU) reflect hardware progress, which lags the exponential red solid line (Model Demand). The purple region is the ‘Systems Gap’ that must be bridged through parallelism and co-design.

11.2.4 The technology S-curve: Why architectures shift

Many computing technologies are described by a lifecycle of ferment (initial slow progress), take-off (rapid growth), and saturation (diminishing returns near physical or economic limits). Figure 11.4 uses two conceptual S-curves to express the pattern: as gains from one design approach taper, domain-specific architectures can open a new efficiency curve for workloads with stable computational structure.

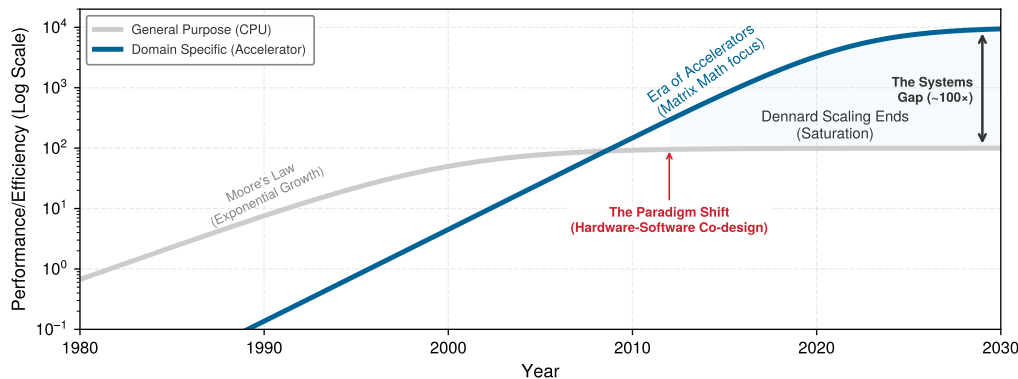


Figure 11.4: The Twin S-Curves of Specialized Computing: This conceptual diagram represents a general-purpose design approach (gray) reaching diminishing returns as a specialized approach (blue) begins a new efficiency curve. It is a schematic of an architectural transition, not a measured historical performance series. Specializing hardware for stable linear-algebra workloads can produce large gains at the cost of general programmability.

The routine gains once associated with shrinking transistors have slowed, while AI model compute demand has expanded faster than general-purpose hardware capacity. Closing this gap cannot be accomplished by waiting for successive CPU process shrinks; it requires architectural changes that restructure how workloads use silicon area, memory bandwidth, and thermal budgets.

During the growth phase of the general-purpose S-curve, semiconductor scaling delivered predictable performance gains to existing software. Under classical Dennard scaling, shrinking transistor dimensions permitted operating voltages to scale down proportionally, keeping power density constant while clock frequencies increased. When Dennard scaling broke down in the mid-2000s, subthreshold leakage currents prevented further voltage reductions, forcing clock frequencies to stall against a thermal dissipation wall. Architects initially responded by adding processor cores, but multicore expansion encountered two physical boundaries: Amdahl’s Law and dark silicon—the

8 | Dennard scaling: The 1974 scaling model held that operating voltage could fall as transistor dimensions shrank, keeping power density approximately constant (Dennard et al. 1974). Its breakdown after about 2005 constrained further clock-frequency growth within a chip’s thermal design power; the resulting **dark silicon** literature estimated, for modeled advanced nodes and design assumptions, that thermal limits could prevent large fractions of available transistors from operating simultaneously. That fraction is not a universal constant, but the power constraint is durable: specialized units make the active silicon budget more useful by dedicating it to high-value operation classes (Esmaeilzadeh et al. 2011).

9 | Huang’s law: The observation that GPU performance for AI workloads has improved through architectural innovations such as Tensor Cores as well as transistor scaling. The normalized figure uses illustrative GPU-supply and model-demand curves to show how unequal growth rates create a widening systems gap; it is not a forecast of either rate (Amodei and Hernandez 2018a; Epoch AI 2024; NVIDIA Corporation 2020a; Choquette 2023).

thermal constraint where only a fraction of a chip’s transistors can switch simultaneously at full frequency. Furthermore, general-purpose CPUs expend the vast majority of their energy budget on control overhead—instruction fetch and decode, branch prediction, speculative out-of-order execution, and cache coherence—rather than arithmetic. For regular linear algebra workloads, executing an instruction on a CPU core can consume hundreds of picojoules of control energy to perform a 3-picojoule floating-point operation.

Domain-specific architectures shift to a new efficiency curve by eliminating this control overhead and dedicating silicon to matching data paths. The primary architectural innovation is a customized data path: matrix multiplication units in AI accelerators, for example, implement **systolic arrays**, grid-like networks of processing elements that compute and forward data directly to neighboring units. Instead of continually writing intermediate results back to register files or caches, a systolic array streams activations and weights across adjacent cells, enabling a value fetched once from on-chip memory to be reused across an entire row or column of multiply-accumulate operations. Once that data path is established, the memory hierarchy can be tailored directly to workload reuse patterns, replacing hardware-managed caches with software-managed scratchpads, prefetching logic, and memory controllers designed for deterministic tensor streams.

Domain-specific instruction sets reinforce this physical specialization. Rather than fetching and decoding scalar operations individually, domain-specific instruction sets encode coarse-grained tensor primitives into single macro-instructions that configure thousands of arithmetic units concurrently, amortizing decode and dispatch overhead. Fixed-function circuit blocks bypass software execution entirely for invariant operations such as activation functions and normalization layers. The resulting efficiency stems from unified hardware specialization across every layer: data movement, memory locality, instruction overhead, and circuit implementation all align around identical computational invariants.

Mobile system-on-chip (SoC) architectures demonstrate this efficiency trade-off directly. Modern smartphones decode high-resolution video within milliwatt thermal envelopes, despite video processing requiring billions of operations per second. This efficiency is achieved through dedicated hardware video codecs¹⁰ implementing industry standards such as H.264/AVC and H.265/HEVC (Sullivan et al. 2012). These specialized circuits provide order-of-magnitude performance-per-watt gains compared with software decoding on general-purpose CPU cores, with the exact gain depending on codec profile, resolution, process node, and CPU baseline.

This architectural shift recurs across data-intensive computing domains whenever a workload’s computational kernel stabilizes. Genomics processing uses custom accelerators because long-read assembly contains fixed dynamic programming alignment kernels that dedicated hardware pipelines directly in silicon (Turakhia et al. 2018). Cryptographic workloads produced application-specific integrated circuits (ASICs)¹¹ for the same reason: fixed hashing algorithms justify dedicated silicon pipelines that trade all programmable flexibility for maximum energy efficiency (Bedford Taylor 2017).

These transitions underscore the central principle of modern systems engineering: the era of automatic performance gains from general-purpose scaling is over. For decades, software developers could rely on Moore’s Law to accelerate existing software without modifying code structures. With the breakdown of Dennard scaling, computational bottlenecks can no longer be resolved by waiting for faster general-purpose cores; they require designing the machine to match the algorithm. In D·A·M terms, the accelerator makes the physical machine constraint explicit. Performance is governed by hardware-software co-design: execution efficiency is determined by how effectively an algorithm’s tensor dimensions, memory access patterns, and parallelism map to the specialized physical structures of domain-specific architectures.

11.2.5 Machine learning hardware specialization

Many important neural-network workloads contain regular tensor operations, substantial data parallelism, and opportunities for reduced precision. Dense matrix multiplications and convolutions are prominent examples because their loop structure exposes repeated operations and predictable reuse. These characteristics have driven specialized hardware architectures that would be ineffective for arbitrary code but can provide substantial gains on matching ML kernels. Other ML operations remain irregular, bandwidth bound, sequential, or too small to fill a wide engine, so the same

¹⁰ | **Codec:** A portmanteau of “coder-decoder,” reflecting the hardware’s dual function. Encoding (compression) is compute-intensive because it searches for optimal representations, while decoding (decompression) is bandwidth-intensive because it reconstructs full-resolution frames from compressed streams. Dedicated codec silicon implements both paths in fixed-function hardware, so neither path wastes transistors on unrelated general-purpose control logic.

¹¹ | **ASIC (application-specific integrated circuit):** These circuits achieve their extreme efficiency by implementing a single algorithm directly in silicon, often improving performance-per-watt by $10^3 \times$ to $10^5 \times$. Examples include cryptographic hashing for blockchain mining and sequence alignment for genomics. The trade-off is total inflexibility: if that core algorithm changes, the ASIC cannot be reprogrammed and becomes obsolete, locking the hardware design to the specific problem version it was built to solve.

device need not accelerate every part of a model. The hardware built to exploit the regular portion constitutes a class of devices known as ML accelerators, and the economic trigger for specialization appears when that supported workload is important at fleet scale rather than only in a benchmark.

Example 11.2: The TPU capacity cliff

Scenario: Google projected in 2013 that user adoption of voice-search speech recognition (3 minutes/day/user) would require doubling global data center capacity to run CPU-based neural network inference (Jouppi et al. 2017).

Diagnosis: General-purpose CPU server capacity was economically unable to absorb the exponential growth in inference FLOPs, threatening a capital infrastructure bottleneck.

Systems lesson: Custom hardware acceleration becomes mandatory when workload volume crosses fleet-level economic thresholds. Aggregate arithmetic intensity and server total cost of ownership drive custom ASIC deployment decisions.

Machine learning computational requirements reveal limitations in traditional processors. In an illustrative scenario where a reference CPU sustains 5 percent–10 percent of its peak on a neural network workload, the upper endpoint delivers approximately 0 GFLOP/s (billions of floating-point operations per second). This inefficiency results from architectural mismatches: CPUs optimize for single-thread performance and irregular memory access, while neural networks require massive parallelism and predictable data streams. The memory bandwidth constraint compounds the problem: a single neural network layer may require accessing gigabytes of parameters, overwhelming CPU cache hierarchies designed for smaller working sets.

The energy economics of data movement influence accelerator design. Per event, accessing a 32-bit value from DRAM can consume on the order of $10^2 \times$ more energy than one FP32 multiply (exact values vary by technology node and design), making data movement a primary optimization target (Horowitz 2014; Sze et al. 2017). This disparity helps explain the progression from repurposed graphics processors to purpose-built neural network accelerators. TPUs and other custom accelerators can sustain high utilization on dense kernels by implementing systolic arrays and other architectures that maximize data reuse while minimizing movement.

Training and inference present different computational profiles that influence accelerator design. Training computes gradients and updates weights: FP32 and FP16 are standardized binary floating-point formats (IEEE Standards Association 2019b), while mixed-precision training uses lower-precision tensor operations with higher-precision accumulation when accuracy permits (Mickevicius et al. 2017). Backpropagation also retains or recomputes activations (see section 8.3.3.1), increasing memory demand. Inference requires only the forward path and may use INT8 or INT4 when the model and kernels support it. Interactive inference may prioritize tail latency, while offline or batched inference may prioritize throughput, cost, or energy; accelerator choice follows the actual service objective. These profiles often favor high-capacity, throughput-oriented training systems and energy-efficient inference paths, but batch size and service policy can reverse the apparent preference.¹²

Definition 11.3: ML accelerator

Machine Learning Accelerators are processors or specialized units designed primarily for common neural-network tensor operations and dataflows. They pursue high R_{peak} and data reuse on matching workloads by devoting more resources to arithmetic and local data movement than a general-purpose execution path can.

1. **Significance:** An ML accelerator’s defining feature is not raw arithmetic alone but a balanced path that supplies operands and retains reusable values nearby. The A100 provides 2.04 TB/s of device-memory bandwidth, about $10.2 \times$ the illustrative 200 GB/s host-DRAM path used here (NVIDIA Corporation 2020a; Choquette et al. 2021). This is

¹² | **Latency vs. throughput in accelerator design:** Training commonly uses throughput-oriented execution and larger batches to amortize work, while latency-sensitive inference must control single-request and tail latency. This is a service objective, not a universal training/inference divide: batched inference can also be throughput oriented, and small or interactive training loops may care about step latency. Pipeline depth, batching delay, kernel setup, and queueing can make a training-oriented path less suitable for an interactive request even when its peak FLOP/s is higher, so the design must match the workload’s actual arrival pattern, batch opportunity, and objective.

a ceiling comparison, not a guarantee: sufficient arithmetic intensity, concurrency, and reuse are still required to approach its 312 TFLOP/s FP16/BF16 peak.

2. **Distinction:** The gains are conditional on operation support, parallel work, regular data access, and a software stack that selects the intended path. An ML accelerator can be orders of magnitude faster than a reference CPU on large dense matrix multiplication, yet lose much of that advantage on small, irregular, synchronization-heavy, or control-heavy workloads. Transfers and unsupported operators can dominate even when the accelerated kernel itself is fast.
3. **Common pitfall:** A frequent misconception is that ML accelerators always accelerate ML. Approaching peak throughput requires a compatible precision, operation shape, layout, implementation, and enough parallel work to keep the relevant units busy. Utilization also depends on whether memory and interconnect paths can sustain that work. A batch-1 autoregressive request may therefore use only a small fraction of a large training accelerator's arithmetic capacity even though the same device performs well on batched prefill or training.

Deployment context shapes architectural choices by identifying the binding constraint. In data centers, the constraint is often time-to-result for training massive models. An NVIDIA H100 trades hundreds of watts of power for high throughput (Choquette 2023); whether that trade lowers total cost depends on utilization, rental price, workload scaling, and the electricity rate. Google's TPUv4 makes a similar architectural trade, prioritizing throughput through systolic arrays and high-bandwidth memory (Jouppi et al. 2023).



Checkpoint 11.2: The accelerator gate

Use the energy hierarchy to decide when specialization pays.

Energy inversion

- ☐ **Data movement cost:** can you explain why moving data from DRAM costs 100× more energy than computing on it?
- ☐ **Architectural response:** explain how systolic arrays (TPU) and Tensor Cores (GPU) reduce repeated movement of the same operands.

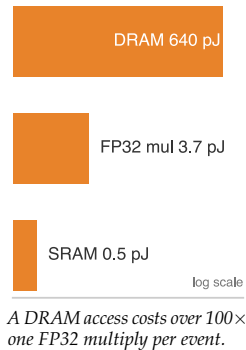
Selection logic

- ☐ **Training vs. inference:** why do training chips need massive HBM bandwidth, while inference chips prioritize low latency and INT8 ops?

At the edge, the priority often shifts toward energy per inference and hard latency or power limits, although throughput remains important for continuous camera, audio, or sensor streams. A smartphone camera or always-on audio path operating inside a few-watt budget cannot simply adopt a data center accelerator's high-power memory system. Edge architectures instead reduce movement through local scratchpads, tightly integrated accelerators, dynamic voltage and frequency scaling, and event-driven processing when the workload permits. The same memory-wall principle applies in both settings: data center chips invest in high-bandwidth memory (HBM) capacity and bandwidth, while edge chips depend heavily on proximity and reuse.

No single architecture dominates every ML workload. Edge devices favor energy efficiency and bounded latency, while cloud-scale training values throughput, capacity, and interconnect performance. Cloud inference can emphasize cost per request or tail latency, and an edge device may still need sustained throughput for a real-time stream. Specialized architectures therefore reflect their deployment context, yet all remain subject to the energy and latency costs of moving data.

Table 11.1 summarizes these milestones in hardware specialization. Floating-point coprocessors accelerated arithmetic previously implemented in software, early GPUs increased graphics throughput, and media engines showed how stable pipelines could justify fixed-function blocks. AI



accelerators combine dense tensor units with memory hierarchies and software support, making integration between data movement and parallel execution a central challenge.

Modern AI acceleration therefore extends beyond the chip. Useful accelerators need framework, compiler, library, driver, and runtime support for graph transformations, kernel selection, fusion, memory scheduling, and deployment across environments from data centers to edge devices. Unsupported operations or surrounding transfers can erase a kernel-level gain.

Table 11.1: Hardware Specialization Trends: Successive computing eras progressively integrate specialized hardware to accelerate prevalent workloads, moving from general-purpose CPUs to domain-specific architectures and ultimately to customizable AI accelerators. Tailoring hardware to computational patterns improves performance and energy efficiency, driving innovation in machine learning systems.

Era	Computational pattern	Architecture examples	Characteristics
1980s	Floating-Point & Signal Processing	FPU, DSP	• Single-purpose engines • Focused instruction sets • Coprocessor interfaces
1990s	3D Graphics & Multimedia	GPU, SIMD Units	• Many identical compute units • Regular data patterns • Wide memory interfaces
2000s	Real-time Media Coding	Media Codecs, Network Processors	• Fixed-function pipelines • High throughput processing • Power-performance optimization
2010s	Deep Learning Tensor Operations	TPU, GPU Tensor Cores	• Matrix multiplication units • Massive parallelism • Memory bandwidth optimization
2020s	Application-Specific Acceleration	ML Engines, Smart NICs, Domain Accelerators	• Workload-specific datapaths • Customized memory hierarchies • Application-optimized designs

11.2.6 The integration bottleneck

The evolution from the Intel 8087 numeric coprocessor to modern graphics and AI accelerators reveals a consistent pattern: hardware evolves to fit the algorithm’s dominant bottleneck (Palmer 1980; Goodfellow et al. 2016; Sze et al. 2017; Jouppi et al. 2017). Where the 8087 offloaded floating-point arithmetic that overwhelmed general-purpose CPUs in scientific computing, modern AI accelerators target dense matrix multiplications and convolutions. In this domain, provisioning raw arithmetic units is straightforward compared to feeding them with data. Modern AI architectures must resolve an **integration bottleneck**: transporting operands and results through the memory hierarchy rapidly and efficiently enough to sustain thousands of parallel compute units. Specialized silicon delivers substantial performance-per-watt gains over general-purpose processors only when the workload’s computational demand concentrates into regular operations that align with physical hardware data paths.

Three properties of neural-network kernels make this integration bottleneck tractable. First, large matrix multiplications and convolutions expose abundant data parallelism that dense processing-element arrays can exploit concurrently. Second, static or compiled tensor graphs exhibit predictable dataflow, enabling compilers to tile and stage transfers into local scratchpads¹³ rather than relying exclusively on hardware-managed caches. This grants software explicit control over regular working sets while retaining hardware caches for irregular access. Third, many tensor operations exhibit algorithmic tolerance to reduced precision, allowing supported 8-bit or 4-bit paths to increase compute density and reduce bytes per value (Dally et al. 2021; Dally 2023). Conversely, workloads dominated by dynamic control flow, large embedding tables, small tensors, variable shapes, or strict precision requirements violate these assumptions and remain bound by memory latency and irregular access.

Once arithmetic capacity is abundant, the central engineering challenge becomes keeping data close enough to use it. A DRAM access can consume more than 100× the energy of a low-precision arithmetic operation under the physical constraints analyzed by Horowitz (2014). This steep energy hierarchy dictates why accelerator architectures invest silicon area and power budgets into HBM, on-chip SRAM, and software-managed scratchpads rather than exclusively adding arithmetic logic units.¹⁴

¹³ | **Scratchpad memory:** When a compiler can determine a tensor kernel’s access pattern, it can explicitly stage tiles in fast, software-controlled local memory. This avoids some tag, replacement, and coherence machinery, but transfers, synchronization, banking, and capacity must be scheduled correctly. Scratchpads complement rather than universally replace caches because irregular or shared data may still benefit from hardware management. Google’s TPU v1, for example, used a 24 MB software-managed Unified Buffer for intermediate activations, while weights and instructions used other paths (Jouppi et al. 2017).

¹⁴ | **HBM:** The A100 and H100 generations provide 2.0–3.4 TB/s of device-memory bandwidth through stacked memory and wide interfaces, compared with 760 GB/s for the GDDR6X reference used here (NVIDIA Corporation 2020a; Choquette 2023). Raising the bandwidth ceiling can move some kernels toward the compute-bound side of the roofline, but only if their access pattern and arithmetic intensity can use it. HBM also increases packaging and system cost, so its value depends on the workload’s bandwidth demand.

Hardware acceleration resolves the integration bottleneck through specialized spatial layout and tiered storage hierarchies. As shown in the architectural blueprint in figure 11.5, high-bandwidth memory feeds shared on-chip storage, which in turn supplies local scratchpad memory surrounding a grid of execution units.

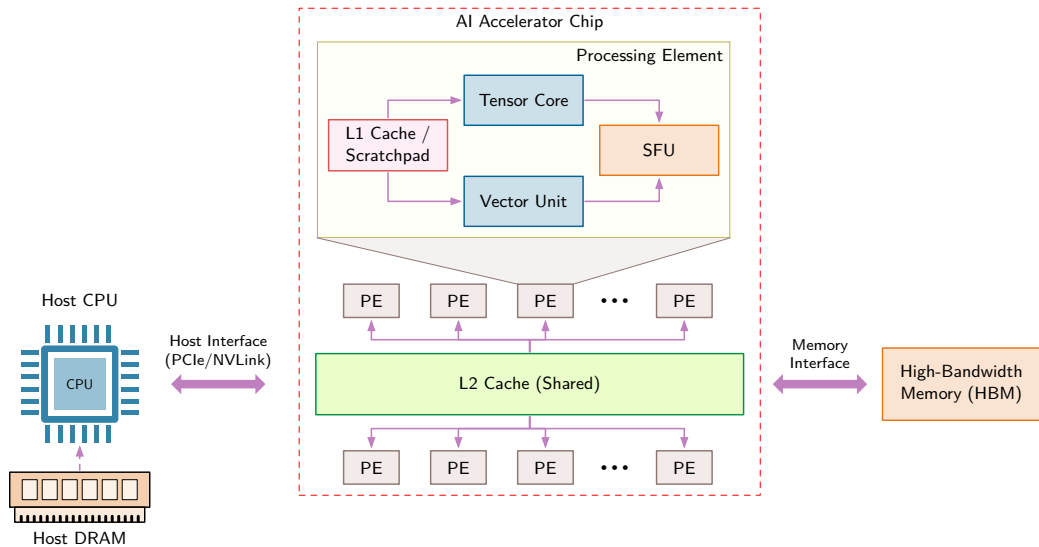


Figure 11.5: Anatomy of a Modern AI Accelerator: This generic schematic combines common components rather than depicting one product. A host interface connects the accelerator to a CPU, a high-bandwidth device-memory interface feeds shared on-chip storage, and processing elements contain matrix, vector, and special-function units with local memory. Real accelerators differ in which units they provide and whether storage is hardware-managed cache or software-managed scratchpad.

In the generic design in figure 11.5, an array of **processing elements** contains units specialized for distinct operation classes: matrix units execute dense matrix multiplication, vector units perform element-wise arithmetic, and special function units approximate transcendental operations such as exponentials and activations. The number, grouping, and vendor nomenclature of these units vary across architectures, but all aim to exploit data-level parallelism across tensor tiles. Some designs arrange units into an explicit 2D systolic grid; others distribute matrix engines among wide parallel compute cores. The schematic represents these functional responsibilities rather than a single vendor's physical layout.

Complementing the execution units, the memory hierarchy amortizes the latency and energy cost of data transport across multiple tiers. High-bandwidth memory provides off-chip capacity and aggregate throughput, while shared on-chip storage and per-element caches or scratchpads reduce off-chip traffic. While specific implementations differ—some relying on hardware-managed caches, others exposing software-managed SRAM, and many combining both—the physical principle is constant: moving a tensor tile across package boundaries consumes orders of magnitude more energy than keeping it on chip. Compilers and runtime kernels must coordinate tile capacity, reuse, prefetching, and double buffering to keep operands staged in fast local memory. Expanding HBM bandwidth raises the off-chip throughput ceiling, but without temporal and spatial reuse in local SRAM, execution units stall waiting for operands. The machine foundations appendix collects reference specifications for modern accelerators, including H100 and TPU v5, and summarizes the latency hierarchy.

The host interface links the accelerator to the host CPU and broader system memory. A host CPU coordinates execution graphs, asynchronous I/O, and unsupported fallback operations, while the accelerator executes dispatched compute kernels. This partition is not absolute: modern accelerators can schedule work queues autonomously, and CPUs can compute tensor operations. In practice, system throughput depends on where each operator runs and the volume of transfers, kernel launches, and synchronization barriers separating them. Direct-memory-access engines

and hardware-managed queues can overlap host-device transfers with active execution, but unsupported operators force costly CPU round-trips. The communication boundary between host memory and accelerator storage therefore establishes the effective speedup envelope; kernel execution time matters little if host-device transfers dominate end-to-end latency.

The specialization evident in processing elements, vector pipelines, and tiered scratchpads reflects the computational profile of neural networks: models repeatedly invoke a compact set of linear-algebraic and element-wise primitives. Within the D·A·M framework, algorithmic modifications produce wall-clock speedups only when their computational structure and memory access patterns map directly onto these specialized hardware primitives. Algorithmic optimizations that reduce theoretical operations without aligning with hardware data paths leave compute units starved, rendering the claimed gains purely theoretical.

11.3 AI Compute Primitives

Across fully connected, convolutional, and attention layers, learned linear transformations reduce to repeated MAC operations. A large layer can issue millions or billions of MACs, and its regular tensor structure exposes parallel work and reuse, making it a natural specialization target. MACs do not dominate every model or phase—normalization, routing, indexing, communication, and element-wise operations can become limiting—but they account for much of the peak arithmetic capacity in modern ML accelerators. Whether they dominate elapsed time still depends on tensor shape, data movement, and the surrounding operators.

The hardware units that exploit these patterns are **AI compute primitives**: specialized functional blocks, each optimized for a particular class of operation. Three primitives are especially common in accelerators, each targeting a distinct computational pattern found in neural networks.

At the framework level, a single declarative statement encapsulates thousands of arithmetic operations (listing 11.1).

Listing 11.1: Dense Layer Abstraction: High-level framework APIs encapsulate 131,072 MACs (256 inputs times 512 outputs) in a single function call, hiding the computational complexity from developers while enabling automatic hardware optimization.

```
# Framework abstracts compute-intensive operations
dense = Dense(512)(input_tensor) # 256x512 MACs per sample
```

Beneath this API call, an autograd or compilation engine lowers the layer into a computational graph. Listing 11.2 exposes this decomposition into a bulk matrix multiplication, a bias broadcast, and an element-wise activation.

Listing 11.2: Matrix Operation Expansion: Each dense layer decomposes into matrix multiplication and element-wise operations, exposing the dominant compute pattern that many neural-network kernels are built around.

```
# Linear transformation work scales with input_dim x output_dim x
# batch.
output = (
    matmul(input, weights) + bias
) # Matrix multiply dominates cost
output = activation(
    output
) # Element-wise: proportional to output_dim x batch
```

While graph representations group these steps into coarse tensor operators, execution on physical hardware requires scheduling individual scalar operations. Lowering the matrix multiplication to loop nests (listing 11.3) makes the execution structure explicit: three nested iterations traverse the batch, output feature, and reduction dimensions, establishing $\mathcal{O}(B \times d_{\text{in}} \times d_{\text{out}})$ arithmetic complexity.

Listing 11.3: Processor-Level Execution: Nested loops reveal the $\mathcal{O}(B \times d_{\text{in}} \times d_{\text{out}})$ multiply-accumulate operations that accelerators must execute, with 4.2M MACs for $B=32$, $d_{\text{in}}=256$, $d_{\text{out}}=512$ configurations.

```
# Total operations: batch_size * output_size * input_size MACs
for n in range(batch_size): # Batch dimension: parallelizable
    for m in range(output_size): # Output neurons: parallelizable
        sum = bias[m] # Initialize accumulator
        for k in range(input_size): # Reduction dimension: sequential
            sum += input[n, k] * weights[k, m] # MAC operation
        output[n, m] = activation(sum) # Nonlinear transformation
# Example work scales as batch_size * output_size *
# input_size multiply-accumulate operations
```

These nested loops expose three recurring computational patterns: element-wise operations along vectors, matrix-level multiply-accumulate reductions, and nonlinear transformations. Their ubiquity justifies dedicated hardware execution paths: vector pipelines stream independent elements and reductions, matrix engines organize multiply-accumulate grids across spatial tiles, and special-function units approximate nonlinear math. The boundaries between these units are architectural rather than mathematical; an optimizing compiler frequently fuses operations across all three paths within a single kernel invocation to eliminate intermediate memory traffic.

11.3.1 Vector operations

Vector operations accelerate computation by executing a single instruction across multiple data elements simultaneously. In the scalar formulation of listing 11.3, processing a batch of 32 samples through a 256-to-512 dense layer requires 4.2M MACs. A scalar processor issues a distinct instruction to load each input-weight pair, perform the multiplication, and update the accumulator, incurring full instruction-fetch, decode, and branch-prediction overhead on every element. Vector pipelines eliminate this instruction overhead by operating on vector registers spanning multiple parallel execution lanes.

RISC-V,¹⁵ an open instruction-set architecture (ISA) with a standardized vector extension (Waterman et al. 2013), provides a concrete vehicle for this execution model. In RISC-V vector assembly (listing 11.4), vector instructions express operations over arrays of runtime-configured length, decoupling software from the physical lane width of the processor.

Listing 11.4: Vectorized Dot-Product Loop: This illustrative loop processes several element pairs per iteration and reduces them into one scalar dot product.

```
fmv.w.x fa0, zero # Scalar dot-product accumulator
fmv.w.x ft1, zero # Zero seed for vector reduction
loop_feature:
    vsetvli t0, feature_cnt, e32, m1, ta, ma
    vle32.v v1, (in_ptr)
    vle32.v v2, (wt_ptr)
    vfmul.vv v3, v1, v2
    vfmv.v.f v0, ft1
    vfredusum.vs v4, v3, v0
    vfmv.f.s ft0, v4
    fadd.s fa0, fa0, ft0
    slli t1, t0, 2 # Four bytes per FP32 element
    add in_ptr, in_ptr, t1
    add wt_ptr, wt_ptr, t1
    sub feature_cnt, feature_cnt, t0
    bnez feature_cnt, loop_feature
```

The vectorized reduction loop in listing 11.4 progresses through five hardware-visible stages:

1. **Vector length configuration:** `vsetvli` configures the vector units for 32-bit elements (e32), dynamically computing how many elements (t0) fit into the hardware vector registers (VLEN).
2. **Vector initialization:** Clears the scalar accumulator that will hold the accumulated dot product.

¹⁵ | **RISC-V (reduced instruction set computer V):** The open ISA allows hardware teams to add custom ML instructions, including vector dot products, activation functions, and sparse tensor operations, without changing the base ISA. The trade-off is software ecosystem maturity: custom extensions require corresponding compiler, library, and runtime support, which can limit portability and increase integration work.

- 3. **Vector loads:** `vle32.v` loads contiguous 32-bit elements into vector registers `v1` and `v2`, dispatching wide memory requests across the memory interface.
- 4. **Vector multiply and reduction:** `vfmul.vv` multiplies element pairs in parallel across vector lanes, and `vfredusum.vs` reduces the products into a partial scalar sum.
- 5. **Pointer arithmetic and loop branch:** Advances base addresses, decrements the remaining element count by the processed chunk size `t0`, and branches if elements remain.

Across iterations, the vector multiply instruction processes multiple data elements concurrently while wide loads amortize instruction decode and memory-request overhead. Because the runtime vector length dynamically adapts to the implementation’s hardware width (VLEN), the identical binary executes correctly on both narrow embedded vector units and wide data-center pipelines. For this 256-feature workload, an 8-lane implementation breaks the reduction into approximately 524,288 vector chunks across the batch.

Beyond dense inner products, deep learning workloads rely on a broader set of vector instructions. Table 11.2 maps these primitive vector operations to the specific neural network layers that depend on them.

Table 11.2: Vector Operations: Core vector operations map directly to deep learning primitives: reductions implement pooling layers, gathers enable embedding lookups, scatters update embedding gradients, and masked operations handle attention masks. These mappings show how vector hardware supports common ML primitives.

Vector operation	Description	Neural network application
Reduction	Combines elements across a vector (for example, sum, max)	Pooling layers, attention score computation
Gather	Loads multiple nonconsecutive memory elements	Embedding lookups, sparse operations
Scatter	Writes to multiple nonconsecutive memory locations	Gradient updates for embeddings
Masked operations	Selectively operates on vector elements	Attention masks, padding handling
Vector-scalar broadcast	Applies scalar to all vector elements	Bias addition, scaling operations

The benefit extends beyond instruction count. Contiguous vector loads can use memory interfaces efficiently, and control overhead is amortized across several data elements. Gather, scatter, masking, and short tails may use fewer lanes or require extra memory transactions, so vector width alone does not guarantee proportional speedup. The architectural pattern is not new. The Cray-1¹⁶ used vector registers and pipelined functional units for scientific computing in the 1970s (Jordan 1982); modern ML processors apply related principles at much larger commercial scale.

Vector operations excel at element-wise transformations like activation functions, where each output depends only on *its corresponding input*. Neural networks, however, also require structured computations where each output depends on *all inputs*—the weighted sums that define layer transformations. These many-to-many operations naturally express themselves as matrix multiplications, the second compute primitive.

11.3.2 Matrix operations

Matrix multiplication accounts for much of the arithmetic in many dense neural networks, transforming high-dimensional data through structured patterns of weights, activations, and gradients (Goodfellow et al. 2016). While vector operations process elements independently or reduce them, matrix operations organize work across multiple dimensions. Hardware and libraries tile those dimensions so blocks of weights and activations can be reused while partial sums accumulate locally. This regular structure drives important hardware strategies, but small or irregular matrices may leave tile capacity unused.

11.3.2.1 Matrix operations in neural networks

Neural network computations decompose into hierarchical matrix operations. Listing 11.5 captures this hierarchy through a linear layer that transforms input features into output neurons over a batch.

16 | **Cray-1 vector legacy:** The Cray-1 (1975) used 64-element vector registers with pipelined functional units, allowing a stream of elements to advance through arithmetic operations without issuing one scalar instruction per element. Modern accelerators extend the same principles of wide registers, pipelined execution, and data reuse to vector and matrix tiles.

Listing 11.5: Linear-Layer Operation Hierarchy: The example expands a linear layer into a matrix multiply and bias broadcast, then applies element-wise ReLU, separating the matrix and vector work that hardware must execute.

```
layer = nn.Linear(256, 512) # Layer transforms 256 inputs to 512 outputs
output = layer(input_batch) # Process a batch of 32 samples

# Framework Internal: Core operations (column-batch convention)
Z = matmul(weights, input) # Matrix: transforms [256x32]
# input to [512x32] output
Z = Z + bias # Vector: adds bias to each
# output independently
output = relu(Z) # Vector: applies activation to
# each element independently
```

This computation demonstrates the scale of matrix operations in neural networks. Each output neuron (512 total) must process all input features (256 total) for every sample in the batch (32 samples). The weight matrix alone contains $256 \times 512 = 131,072$ parameters that define these transformations, illustrating why efficient matrix multiplication dominates performance considerations.

Neural networks employ matrix operations across architectural patterns beyond simple linear layers. Convolution operations transform into matrix multiplications through the im2col technique,¹⁷ enabling efficient execution on matrix-optimized hardware. Listing 11.6 illustrates these applications.

Listing 11.6: Matrix Patterns Across Architectures: Linear layers, attention mechanisms, and convolutions all reduce key work to matrix multiplications, making matrix hardware the shared primitive across modern neural architectures.

```
hidden = matmul(weights, inputs)
# weights: [out_dim x in_dim], inputs: [in_dim x batch]
# Result combines all inputs for each output

# Attention Mechanisms - Multiple matrix operations
Q = matmul(Wq, inputs)
# Project inputs to query space [query_dim x batch]
K = matmul(Wk, inputs)
# Project inputs to key space [key_dim x batch]
attention = matmul(Q.T, K)
# Compare all query tokens with all key tokens [batch x batch]

# Convolutions - Matrix multiply after reshaping
patches = im2col(input)
# Convert [H x W x C] image to matrix of patches
output = matmul(kernel, patches)
# Apply kernels to all patches simultaneously
```

These examples differ in tensor shape and reuse pattern, which determines how efficiently each multiplication occupies a matrix unit. Linear layers reuse one weight matrix across a batch; attention forms projections and then a token-to-token score matrix; convolution reuses each kernel across many image patches. Expressing all three as matrix multiplication exposes a common interface to hardware, but it does not make their execution costs identical. Batch size, sequence length, channel count, and tiling determine whether operands are reused locally or repeatedly fetched from memory.

11.3.2.2 Matrix operations hardware acceleration

This pervasive pattern of matrix multiplication has direct implications for hardware design: accelerators need specialized units that can handle these computations at scale. Listing 11.7 demonstrates a representative dedicated matrix unit that processes an entire 16×16 block at once, illustrating why matrix instructions and Tensor Cores can deliver much higher throughput than scalar or vector-only execution paths (NVIDIA 2017; Intel Corporation 2021a).

¹⁷ | **Im2col (image-to-column):** Transforms convolution into a matrix multiplication by arranging receptive-field values as columns. An explicit materialization can duplicate overlapping values and expand storage by up to roughly the kernel area before edge, stride, and channel effects are considered. Production libraries often use implicit-GEMM or direct-convolution kernels to obtain matrix-friendly execution without materializing the full expanded matrix.

Listing 11.7: Illustrative Matrix-Unit Operation: Representative load, matrix-multiply-accumulate, and store instructions operate on matrix blocks rather than individual scalar elements.

```

mload mr1, (weight_ptr)    # Load e.g., 16x16 block of
                           # weight matrix
mload mr2, (input_ptr)     # Load corresponding input block
matmul.mm mr3, mr1, mr2    # Multiply and accumulate entire
                           # blocks at once
mstore (output_ptr), mr3   # Store computed output block

```

The illustrative unit operates on matrix tiles rather than individual elements. A full $16 \times 16 \times 16$ tile product contains $16^3 = 4,096$ multiply-accumulate operations, but the cycles required depend on the actual instruction and architecture. Sustained throughput also depends on the number of matrix units, operand precision, issue rate, pipeline behavior, and whether memory can keep the units fed. Matrix units complement vector execution by accelerating structured many-to-many transformations.

Like vector processing, matrix acceleration has deep historical roots—DSPs and GPUs optimized for matrix computations in the 1980s-1990s for image processing, scientific computing, and 3D rendering (Owens et al. 2008; Hwu 2011). Neural networks have made matrix multiplication commercially dominant, driving the integration of two-dimensional systolic arrays and matrix execution engines (such as NVIDIA Tensor Cores and Google TPUs) directly into processor datapaths.

Linear algebra constitutes only part of the neural network execution pipeline. Between linear transformations, neural networks introduce nonlinear activation functions and normalization layers. While piecewise-linear functions such as ReLU map to simple vector comparison instructions, transcendental computations (such as exponentials in softmax and GELU, or reciprocal square roots in normalization) cannot be computed directly on multiply-accumulate datapaths without incurring severe cycle penalties. Table 11.3 summarizes the execution characteristics of the three primary compute primitives and their mapping to neural network operations.

Table 11.3: Operation Characteristics: Matrix operations excel at many-to-many transformations, while vector units handle element-wise operations and reductions such as activation and normalization. Special-function paths handle transcendental arithmetic through architecture-specific approximation or dedicated logic. The operation structure determines which hardware primitive is appropriate.

Operation type	Best For	Examples	Key characteristic
Matrix Operations	Many-to-many transforms	Layer transformations, attention, convolutions	Each output depends on multiple inputs
Vector Operations	Vector and reduction work	Activation functions, layer normalization, element-wise gradients	Uses element-wise operations and vector reductions
Special-Function Paths	Transcendental arithmetic	Exponential, logarithm, reciprocal square root	Uses architecture-specific approximation or dedicated logic

11.3.3 Special function units

Special Function Units (SFUs) or specialized instruction paths accelerate nonlinear functions and related arithmetic, completing the chapter’s trio of processing primitives. The need is not new: floating-point coprocessors addressed scalar arithmetic bottlenecks (Palmer 1980), and digital signal processors added specialized arithmetic for signal-processing workloads (Smith 1997). An implementation may use a standalone unit, vector approximation instructions, or a library sequence, trading accuracy, latency, and throughput. In neural networks, activation, normalization, and softmax operations can become important between matrix kernels, particularly when repeated memory passes or transcendental functions limit throughput.

11.3.3.1 Nonlinear functions

A common layer sequence illustrates how non-matrix operators enter the execution pipeline (Goodfellow et al. 2016). Listing 11.8 chains a linear transformation with ReLU activation and batch normalization—operations that appear trivial in Python framework code but impose distinct arithmetic and memory constraints on hardware.

Listing 11.8: Linear–Nonlinear Layer Sequence: A linear layer, ReLU, and BatchNorm combine matrix, element-wise, and normalization work in one common layer sequence.

```
layer = nn.Sequential(
    nn.Linear(256, 512), nn.ReLU(), nn.BatchNorm1d(512)
)
output = layer(input_tensor)
```

Listing 11.9 expands this sequence into its arithmetic primitives, isolating the element-wise thresholding of ReLU from the cross-batch reductions and reciprocal square root required by batch normalization.

Listing 11.9: Nonlinear Operation Expansion: Expanding the layer sequence exposes ReLU comparison, BatchNorm reductions, variance computation, and square-root normalization around the matrix multiply.

```
Z = matmul(weights, input) + bias # Linear transformation
H = max(0, Z) # ReLU activation
mean = reduce_mean(H, axis=0) # BatchNorm statistics
var = reduce_mean((H - mean) ** 2) # Variance computation
output = gamma * (H - mean) / sqrt(var + eps) + beta # Normalization
```

11.3.3.2 Hardware implementation of nonlinear functions

Translating these mathematical functions to silicon exposes sharp differences in memory traffic, instruction count, and pipeline utilization. For instance, batch normalization (Ioffe and Szegedy 2015) demands multi-pass statistics gathering across memory, variance accumulation, and a reciprocal square root, while softmax requires computing exponentials across an entire normalization dimension. While a rectified linear unit (ReLU) is mathematically simple, a naive scalar implementation incurs branch overhead or redundant memory round-trips unless fused into adjacent kernels. Listing 11.10 contrasts the memory and dependency overheads of pointwise activation against multi-pass normalization in an unfused loop model.

The label *nonlinear* hides three different execution contracts. Pointwise functions such as ReLU leave elements independent and therefore map naturally to vector lanes or parallel threads. Reductions such as the mean and variance in batch normalization introduce cross-element dependencies: partial results must be combined before later stages can proceed. Transcendental functions such as exponential, logarithm, and reciprocal square root add a numerical contract because their implementations trade latency, precision, and valid input range. The dependency pattern, not the mathematical label alone, determines whether vector arithmetic, a reduction tree, or a specialized approximation path is appropriate. A faster special-function path accelerates only the operation it implements; it does not remove surrounding tensor traffic or synchronization.

Listing 11.10 is intentionally dependency-explicit rather than a model of an optimized kernel. The loops expose three state lifetimes: per-element activations, per-feature accumulators, and the final normalized outputs. The normalization parameters cannot be applied until the required statistics are available, so implementations must either retain intermediate activations or recompute them. Training derives those statistics from the current batch, whereas inference can use stored statistics and reduce normalization to an element-wise affine transform. Compilers may fuse legal stages, but the data dependencies determine which transformations preserve the computation. Reading the listing as a dependency graph therefore reveals the durable hardware questions: how often each tensor crosses a memory interface, which intermediates remain local, and where reductions impose synchronization.

Listing 11.10: ReLU and BatchNorm Operations: Neural networks process input data through element-wise selections and multiple normalization passes, highlighting efficiency challenges in naive implementations.

```

for batch in range(32):
    for feature in range(512):
        # ReLU: Naive scalar compare/select; optimized kernels
        # usually implement this branchlessly.
        z = matmul_output[batch, feature]
        h = max(0.0, z) # Conditional operation

        # BatchNorm: Multiple passes over data
        mean_sum[feature] += h # First pass for mean
        var_sum[feature] += h * h # Additional pass for variance

        temp[batch, feature] = h # Extra memory storage needed

# Normalization requires complex arithmetic
for feature in range(512):
    mean = mean_sum[feature] / batch_size
    var = (var_sum[feature] / batch_size) - mean * mean

    # Square root computation: Multiple iterations
    scale = gamma[feature] / sqrt(var + eps) # Iterative
    # approximation
    shift = beta[feature] - mean * scale

    # Additional pass over data for final computation
    for batch in range(32):
        output[batch, feature] = temp[batch, feature] * scale + shift

```

The dependency structure in listing 11.10 exposes why naive execution collapses performance: arithmetic intensity drops precipitously. The pointwise ReLU and normalization scale-and-shift perform only one or two arithmetic operations per four-byte memory access, plunging the workload deep into the memory-bound regime. Furthermore, because batch statistics cannot be finalized until all samples are processed, the intermediate tensor `temp` must be written back to memory and reread in a subsequent pass, inflating memory bandwidth consumption and cache pressure. At the same time, computing $1/\sqrt{\text{var} + \epsilon}$ requires an iterative square root and division that can stall pipeline execution for tens of cycles if executed on standard integer or floating-point ALUs. Mitigating these overheads requires two complementary strategies: hardware-level Special Function Units to approximate transcendental arithmetic in single-digit cycles, and compiler-level kernel fusion to keep intermediate values in registers rather than spilling them across the memory bus.

11.3.3.3 SFU hardware implementation

SFUs address arithmetic bottlenecks by implementing nonlinear and transcendental operations in specialized functional units rather than multi-instruction software routines. Accelerators deploy dedicated datapath circuits—such as quadratic polynomial interpolators and small lookup tables (LUTs)—that evaluate functions like exponential, reciprocal square root, and logarithm within a few pipeline stages. Listing 11.11 illustrates how an instruction set exposes these operations alongside vector memory accesses.

Listing 11.11: Illustrative SFU Operations: Pseudocode shows vector nonlinear operations that may map to specialized execution paths; instruction names and latency are implementation dependent.

```

vld.v v1, (input_ptr) # Load vector of values (pseudocode)
vrelu.v v2, v1        # Vector ReLU path
vsigm.v v3, v1        # Vector sigmoid path
vtanh.v v4, v1        # Vector tanh path
vrsqrt.v v5, v1       # Vector reciprocal-square-root path

```

Specialized function paths balance circuit area against mathematical precision. While piecewise linear activations such as ReLU require only single-cycle compare-and-select (maximum) datapath logic, transcendental functions trade off lookup table size and polynomial interpolation degrees against approximation error. Table 11.4 summarizes representative hardware mechanisms and their qualitative latency characteristics across these primitives.

Table 11.4: Special Function Units: Dedicated paths reduce instruction overhead for nonlinear and reduction primitives. Implementations and latency are architecture-dependent; the table describes mechanisms rather than universal cycle counts.

Function unit	Operation	Implementation strategy	Illustrative latency
Activation unit	ReLU	Compare-and-select or maximum	Low; architecture-dependent
Statistics unit	Mean, variance	Parallel reduction trees	Grows with reduction depth
Exponential unit	exp, log, sigmoid, tanh	Approximation, table lookup, and interpolation	Architecture-dependent
Root/power unit	sqrt, rsqrt	Iterative or approximation hardware	Architecture-dependent

Vector operations, matrix operations, and special-function paths establish the foundational arithmetic primitives of ML accelerators, but functional units alone do not dictate sustained throughput. These primitives govern *what* the silicon computes; the execution model and memory hierarchy govern *how* data feeds the datapath. A matrix unit rated for hundreds of teraFLOP/s will idle if memory bandwidth cannot sustain operand delivery, if tile dimensions mismatch register blocking, or if thread synchronization serializes execution. Peak hardware ratings describe performance under idealized operand availability; the architecture’s execution model dictates how much of that peak is achievable on real neural network workloads.

11.4 Compute Units and Execution Models

Applying ReLU to a 512-element vector shows why execution models matter: the operation is simple, but throughput depends on whether the hardware treats those 512 comparisons as scalar instructions, SIMD lanes, GPU threads, or tensor-program fragments. Modern AI processors package the three compute primitives into distinct execution units: single instruction, multiple data (SIMD) units, Tensor Cores, and processing elements that define how computations are structured and exposed to programmers. Understanding this organization reveals both the theoretical capabilities and practical performance characteristics that determine real-world throughput.

11.4.1 Mapping primitives to execution units

Mapping computational primitives to execution units is governed by physical wire energy and data reuse. Reading an operand from a register file and moving it across on-chip wires consumes several times more energy than the arithmetic operation itself. Hardware accelerators manage this physical constraint by provisioning specialized execution units matched to the reuse characteristics of each primitive:

- Vector operations → SIMD and single instruction, multiple threads (SMT) units. These execute 1D elementwise primitives—such as additions, scaling, and bias offsets—across parallel ALUs that share instruction fetch and decode logic. Because vector operations exhibit $O(1)$ arithmetic reuse per operand loaded, their throughput remains bounded by register file port bandwidth and memory delivery.
- Matrix operations → Tensor Cores¹⁸ and systolic arrays. These map 2D inner-product primitives to dense two-dimensional grids of MAC circuits. By routing activations and weights directly between adjacent processing elements through local wires—such as in weight-stationary or output-stationary dataflows—matrix units reuse data in flight. This spatial reuse decouples arithmetic throughput from main register file bandwidth, sustaining peak compute density within a manageable thermal envelope.
- Special functions → Special Function Units (SFUs). These map transcendental and non-linear primitives—such as exp, ln, and reciprocal square roots—to dedicated pipelines using piecewise polynomial approximations and lookup tables. Evaluating transcendentials on general-purpose ALUs requires iterative numerical algorithms that consume dozens of clock cycles;

18 | **Reduced-precision ML:** Halving operand width halves the bytes per stored element and can support denser arithmetic, although realized throughput depends on the available datapath and kernel (Dally et al. 2021; Dally 2023). NVIDIA’s transition from Pascal P100 vector arithmetic to mixed-precision Tensor Cores in Volta V100 increased advertised peak FP16 throughput from about 21.2 to 125 TFLOP/s, roughly 6× across those products (NVIDIA 2017). Precision selection must preserve numerical behavior and use an efficient supported path; the smallest format is not automatically the best one.

dedicated SFUs evaluate these functions off the primary execution path at lower throughput, preventing activation layers from stalling vector pipelines.

In the D·A·M taxonomy, this partitioning defines how algorithmic primitives bind to physical machine execution paths. Routing dense matrix multiplications through standard vector pipelines rapidly saturates register file read ports, collapsing throughput to a small fraction of the accelerator’s peak capacity. Conversely, routing low-arithmetic-intensity elementwise operations or irregular activations through matrix arrays leaves processing lanes unutilized. Sustained system performance depends on matching each computational primitive to the execution unit whose dataflow and storage hierarchy match its arithmetic intensity.

11.4.2 Evolution from SIMD to SIMT architectures

In scalar processor architectures, executing an elementwise operation across a 512-element activation vector requires 512 individual loop iterations. Each iteration fetches, decodes, and tracks an instruction through the pipeline, consuming substantially more energy in instruction control logic than the arithmetic logic unit (ALU) expends performing the actual comparison or addition. Under fixed thermal design power (TDP) budgets, scalar execution spends silicon area and power on instruction management rather than arithmetic throughput. Flynn’s SIMD execution model amortizes this front-end overhead by issuing a single decoded instruction across multiple data elements packed into wide vector registers (Flynn 1966). A 512-bit SIMD register, for example, processes sixteen 32-bit floating-point values simultaneously in a single clock cycle, reducing instruction fetch and decode traffic sixteenfold.

Traditional fixed-width SIMD architectures bind compiled machine code to specific hardware register widths, such as 128, 256, or 512 bits. When a tensor dimension does not align with the fixed vector width, compilers must generate scalar cleanup loops or insert memory padding, increasing binary footprint and branch complexity. The Arm Scalable Vector Extension (SVE) resolves this rigidity through vector-length-agnostic (VLA) execution and per-lane predication (Stephens et al. 2017). Instead of hardcoding physical register widths into the instruction encoding, SVE relies on predicate registers to dynamically configure active lanes, as shown in the vector arithmetic sequence in listing 11.12.

Listing 11.12: Arm SVE Vector Execution: Predicated vector assembly loads, multiplies, and adds floating-point elements using length-agnostic registers.

```
ptrue p0.s           # Create predicate for vector length
ld1w z0.s, p0/z, [x0] # Load vector of inputs
fmul z1.s, z0.s, z0.s  # Multiply elements
fadd z2.s, z1.s, z0.s  # Add elements
st1w z2.s, p0, [x1]    # Store results
```

The `ptrue` instruction activates all available predicate lanes without encoding a fixed vector length. This vector-length-agnostic programming model allows the same SVE binary to run on implementations from 128 to 2048 bits without recompilation (Stephens et al. 2017). Intel’s Advanced Matrix Extensions (AMX) take specialization a step further: dedicated tile registers and matrix-multiplication accelerators operate directly on two-dimensional data blocks rather than one-dimensional vectors (Intel Corporation 2021a). Together, SVE and AMX represent CPU-side adaptations for ML workloads: portable vector execution and fixed-tile matrix pipelines.

Despite vector-length agility, CPU SIMD architectures face fundamental physical limitations when scaling to large neural network workloads. CPUs dedicate substantial silicon area to branch predictors, out-of-order execution logic, and multi-level cache hierarchies to minimize latency for a few sequential threads. When streaming through gigabytes of tensor data, cache misses force the execution pipeline to wait hundreds of cycles for main memory fetches. The CPU’s instruction reorder buffer fills rapidly during memory stalls, leaving wide SIMD pipelines starved of work. Furthermore, conditional branching within a vector requires evaluating both paths under bitmasks, reducing useful lane utilization.

Single Instruction, Multiple Threads (SIMT) architectures resolve these bottlenecks by decoupling the logical programming abstraction from physical hardware execution (Lindholm et al. 2008; Nickolls et al. 2008). Instead of requiring the programmer to pack data into explicit vector registers, SIMT exposes an execution model of independent scalar threads. In GPU architectures, each Streaming Multiprocessor (SM)¹⁹ coordinates thousands of concurrent threads. Hardware warp schedulers group these threads into **warps**,²⁰ which are 32-thread execution units (or wavefronts in AMD architectures) that execute instructions in lockstep across parallel ALUs. Rather than relying on large caches to reduce memory latency, GPUs hide latency through massive thread-level concurrency. An SM statically partitions an on-chip register file across all active threads, eliminating context-saving overhead; when one warp stalls waiting hundreds of cycles for an off-chip memory fetch, the warp scheduler selects and issues instructions from another ready warp on the very next cycle. The baseline matrix multiplication kernel in listing 11.13 illustrates how this scalar thread model maps onto hardware.

Listing 11.13: SIMT Execution: Each thread processes a unique output element in parallel, demonstrating how SIMT enables efficient matrix multiplication on GPUs.

```
__global__ void matrix_multiply(float* C, float* A, float*
                               B, int N) { // CUDA kernel
    // Each thread processes one output element
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    float sum = 0.0f;
    for (int k = 0; k < N; k++) {
        // Threads in a warp execute in parallel
        sum += A[row * N + k] * B[k * N + col];
    }
    C[row * N + col] = sum;
}
```

In listing 11.13, the CUDA²¹ kernel assigns one output element $C[\text{row}, \text{col}]$ to each thread. Adjacent threads in a warp share the same row index while having consecutive column indices (threadIdx.x). When threads fetch $B[k \cdot N + \text{col}]$, their memory requests target contiguous 4-byte floating-point addresses. The memory controller merges these 32 individual 4-byte loads into a single 128-byte coalesced memory transaction, maximizing DRAM bus efficiency. Concurrently, all threads in the warp read the identical element $A[\text{row} \cdot N + k]$, allowing the hardware to broadcast that single fetched value across all 32 lanes. However, this naive kernel illustrates the iron law of ML systems: arithmetic intensity limits delivered throughput. Because each inner-loop iteration performs only two floating-point operations while reloading operands from global memory without on-chip SRAM staging, the kernel remains severely memory bound. Furthermore, executing dense matrix multiplication on scalar SIMT ALUs requires one instruction fetch, decode, and issue cycle for every multiply-accumulate, saturating register file read ports. Overcoming this machine bottleneck requires moving beyond vector-of-scalars execution to specialized two-dimensional matrix units.

11.4.3 Tensor Cores

Consider a single transformer attention head computing the $\mathbf{Q} \times \mathbf{K}^T$ product for a 2,048-token sequence with 64-dimensional embeddings. This operation requires multiplying a 2048×64 matrix by a 64×2048 matrix: roughly 268.4M MACs, or about 536.9 MFLOP when the multiply and add are counted separately. On a scalar processor executing one FLOP per cycle at 2 GHz, this single attention head would take about 268.4 ms. GPU SIMT execution can distribute this work across many threads, but Tensor Cores go further by processing entire matrix tiles per instruction; under the illustrative assumption used here, the tiled path completes the same operation in 0.5 milliseconds, a roughly 536.9× improvement over scalar execution. This speedup arises not from higher clock frequencies, but from hardwiring tile-level data reuse directly into the execution datapath.

While SIMD and SIMT units execute vector work efficiently, large matrix computations benefit from units organized around multidimensional tiles. Dedicated matrix engines amortize operand

19 | SM (streaming multi-processor): The physical hardware engine that implements the SIMT model by using warp schedulers to coordinate many parallel threads. Maintaining enough active warps can help hide instruction and memory latency, but occupancy alone does not identify the bottleneck. Low occupancy may result from register use, shared-memory use, block dimensions, or insufficient parallel work, while a memory-bound kernel can still have high occupancy.

20 | Warp: On NVIDIA GPUs, a warp is a hardware scheduling group of 32 threads that execute a common instruction when their control paths agree. If threads take different paths, the hardware executes the required paths under different masks, reducing useful lane utilization. The penalty depends on path length and how many lanes take each path, which is why ML kernels often prefer uniform or predicated control flow.

21 | CUDA (compute unified device architecture): Released by NVIDIA in 2006, CUDA eliminated the need to disguise general-purpose computations as graphics operations, opening GPUs to scientific and ML workloads through a C-like programming model. The ecosystem it created—cuBLAS, cuDNN, TensorRT—constitutes a software moat that can lock the ML training stack to NVIDIA hardware: migrating away requires rewriting or replacing thousands of GPU-optimized kernels, a cost that often exceeds the hardware savings of competing platforms. This software lock-in, not raw silicon performance alone, helps explain why many large ML training stacks remain CUDA-centered.

loads across many multiply-accumulate operations by reusing tiles on chip. NVIDIA GPUs expose Tensor Cores, whereas Google TPUs use matrix units built around systolic arrays. Both execute tiled matrix multiplication and accumulation.²² The benefit depends on tile reuse, shape, precision, and memory behavior.

NVIDIA Parallel Thread Execution (PTX) exposes this architecture through warp-synchronous matrix instructions.²³ In listing 11.14, all 32 threads in a warp execute `mma.sync` collectively to compute a matrix tile product $D = A \times B + C$.

Listing 11.14: Tensor Core Operation: Matrix multiplications are performed in parallel across entire matrix blocks, optimizing computational efficiency for neural network training.

```
Tensor Core Operation (example GPU PTX):
mma.sync.aligned.m16n8k16.row.col.f32.f16.f16.f32
{d0,d1,d2,d3},    // Destination registers
{a0,a1,a2,a3},    // Source matrix A
{b0,b1},          // Source matrix B
{c0,c1,c2,c3};    // Accumulator
```

In this instruction, the warp executes an $M = 16, N = 8, K = 16$ matrix multiply-accumulate operation in mixed precision: inputs A (16×16) and B (16×8) are formatted in FP16, while accumulator C and destination D (16×8) use FP32. Rather than scheduling 64 separate scalar fused multiply-add (`fma`) instructions per thread, the warp scheduler issues a single instruction that evaluates 2,048 multiply-accumulate operations across the Tensor Core datapath. Distributing matrix fragments across thread registers and accumulating intermediate sums within dedicated adder trees relieves two key hardware bottlenecks of conventional SIMT execution: instruction issue bandwidth and register file read-port contention.

Design priorities determine how matrix engines appear in different processor families. GPU Tensor Cores preserve programmability while accelerating general-purpose deep learning kernels. TPU-style designs use large-scale matrix units arranged in systolic arrays to maximize sustained training throughput on dense tensor kernels. Mobile NPUs²⁴ shrink the same idea into low-power inference blocks, while server CPUs add matrix instruction extensions (AMX-class tiles) for inference and mixed workloads. Each version changes the same contract: how much flexibility the hardware keeps while reducing movement around dense matrix operations.

Figure 11.6 shows how specialization, lower precision, and sparsity support changed advertised peak capability. From K20X FP32 to H100 FP8 with structured sparsity, the displayed values rise by roughly three orders of magnitude (NVIDIA Corporation 2017, 2020a, 2024; Choquette 2023). The points do not measure one consistent quantity: they mix precision formats and combine FLOP/s with INT8 operations per second. The curve therefore compares each generation's most prominent advertised path, not same-precision application performance. Table E.1 provides the corresponding precision-specific specifications, bandwidth, and power limits.

11.4.4 Processing elements

At the next level of execution hierarchy, accelerators integrate multiple Tensor Cores with local memory into modular processing elements (PEs). High-throughput matrix units cannot be fed directly from global memory; off-chip DRAM and on-package HBM lack the bandwidth and low latency required to stream operands into dense arithmetic arrays every cycle. A processing element resolves this bandwidth deficit by co-locating heterogeneous compute pipelines—Tensor Cores for matrix multiplication, vector units for element-wise operations, and special function units for nonlinear transformations—directly alongside high-bandwidth register files and scratchpad SRAM. Staging operands inside the PE keeps data adjacent to the functional units, amortizing the latency and energy cost of external memory accesses across repeated arithmetic reuse.

Processing element design varies because each architecture chooses a different balance between compute density, local memory capacity, and interconnect distance. Graphcore's Intelligence Processing Unit (IPU) distributes computation across 1,472 tiles, each containing an independent processing element backed by dedicated local SRAM to exploit fine-grained parallelism without memory

22 | Tensor Core dimension alignment: NVIDIA Tensor Cores are most efficient when matrix dimensions are aligned to precision- and architecture-specific multiples, such as multiples of 8 or 16 for common FP16/BF16/INT8 paths. Modern cuBLAS and cuDNN can still use Tensor Cores for many nonaligned dimensions, but poorly aligned shapes may trigger less efficient kernels, require padding, or reduce effective throughput (NVIDIA 2024a; NVIDIA Corporation 2021b). This is why model architects often choose embedding and channel dimensions such as 512 rather than 500 and why batch-size-1 inference may fail to reach peak Tensor Core utilization: alignment and arithmetic intensity jointly determine whether the hardware's matrix engines stay full.

23 | Tensor Core: A single Tensor Core instruction executes a complete matrix-multiply-accumulate operation on a small tile of data using a dedicated hardware block (NVIDIA 2017; NVIDIA Corporation 2020a). This approach bypasses the overhead of fetching and scheduling dozens of individual arithmetic instructions on general-purpose CUDA cores. Because these blocks constitute a large fraction of a modern accelerator's advertised tensor throughput, failing to use them can leave most of the chip's theoretical peak unavailable to the workload.

24 | NPU (neural processing unit): Mobile NPUs achieve low-power inference by implementing common tensor operations in fixed-function or narrowly programmable hardware rather than as fully general GPU kernels. This architectural commitment can deliver large energy-efficiency gains for supported kernels, but it makes deployment dependent on operator coverage: unsupported functions must fall back to a CPU or GPU path that may be far less efficient for that workload (Sze et al. 2017).

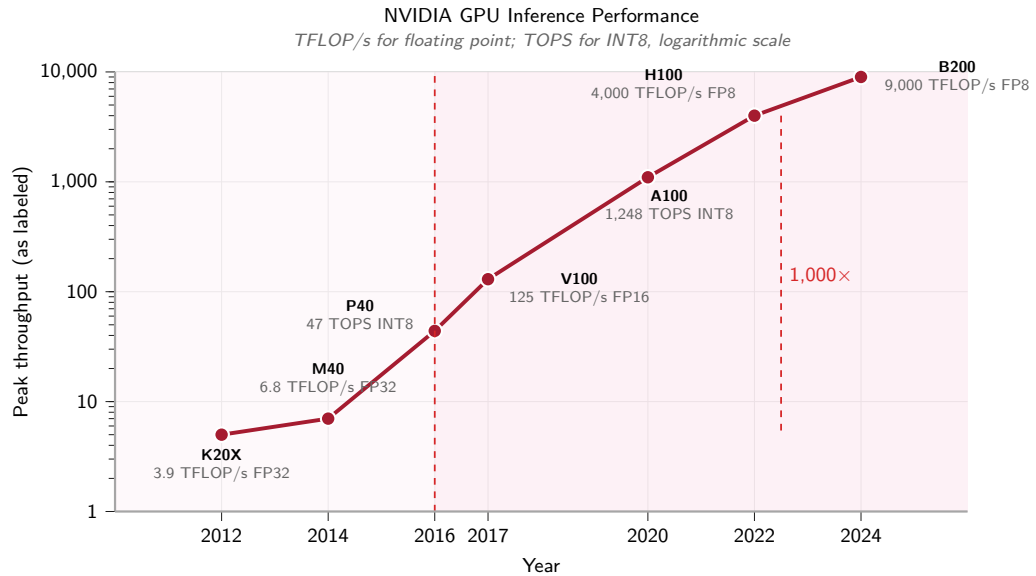


Figure 11.6: Advertised GPU Peak Capability Across Generations: Single-chip values are plotted on a logarithmic axis. The precision label at each point is essential: the curve combines architectural gains with different numerical formats and, for some points, sparsity assumptions rather than measuring same-precision application scaling (NVIDIA Corporation 2017, 2020a, 2024; Choquette 2023).

contention (Graphcore 2020). Cerebras extends this local-storage principle in the CS-2 system, integrating roughly 850,000 AI-optimized cores across a wafer-scale device to bypass off-chip memory interfaces entirely in favor of an on-chip SRAM mesh (Systems 2021). Tesla’s D1 processor dedicates substantial silicon area to local SRAM and high-throughput routing interfaces inside its processing elements, prioritizing low-latency data exchange across tightly coupled compute arrays (Tesla, Inc. 2021).

Across these designs, the binding physical constraint is compute density versus data locality: silicon area dedicated to arithmetic units cannot simultaneously house SRAM storage or routing wires. Packing more execution units raises peak theoretical FLOP/s, but those units idle if memory bandwidth and routing channels cannot deliver operands. A processing element’s delivered efficiency therefore depends as much on its local memory hierarchy and interconnect strategy as on its raw arithmetic capability.

That same dependence on locality governs which algorithmic optimizations the hardware can actually exploit. A regular grid of processing elements accelerates sparsity only when the surviving nonzero values preserve the predictable access patterns the grid depends on, which is precisely the constraint that N:M structured sparsity is designed to satisfy.

11.4.5 N:M structured sparsity mechanics

While unstructured pruning reduces model parameter counts, it rarely translates to hardware speedup because arbitrary nonzero placements destroy memory locality. Hardware accelerators resolve this with N:M structured sparsity,²⁵ which bounds sparsity to a deterministic, fine-grained pattern. The notation “N:M” specifies that exactly N values must be nonzero within every contiguous block of M values along the reduction dimension. Restricting nonzeros to fixed local intervals guarantees uniform memory access and predictable indexing across parallel execution units.

Parallel architectures require structural regularity because of how sparse matrices must be encoded in memory. Classical formats such as Compressed Sparse Row (CSR) and Block Compressed Sparse Row (BSR)—treated formally in section C.1.5—rely on auxiliary index arrays to locate nonzero entries. When sparsity is unstructured, adjacent threads in a SIMT warp access non-contiguous memory addresses, generating uncoalesced memory transactions that saturate the memory bus while fetching mostly empty cache lines. In addition, varying numbers of nonzeros across rows

²⁵ | **N:M structured sparsity:** The 2:4 ratio (50 percent density) used by NVIDIA’s Ampere Sparse Tensor Cores is a hardware-friendly compromise: every contiguous four-value group retains two nonzero values, preserving regular indexing while halving the dense value payload (NVIDIA Corporation 2020a). At 2:4, the metadata overhead is compact enough to store alongside the weights without overwhelming the memory-traffic savings, which is the constraint that makes the advertised $2\times$ tensor-math throughput path plausible when kernels and model weights satisfy the pattern.

cause branch divergence, forcing threads to serialize. While coarse block sparsity groups nonzeros into larger dense tiles to mitigate divergence, it degrades model accuracy when whole blocks must be pruned. Figure 11.7 compares these occupancy patterns and demonstrates the substantial metadata required to index irregular or blocked nonzeros.

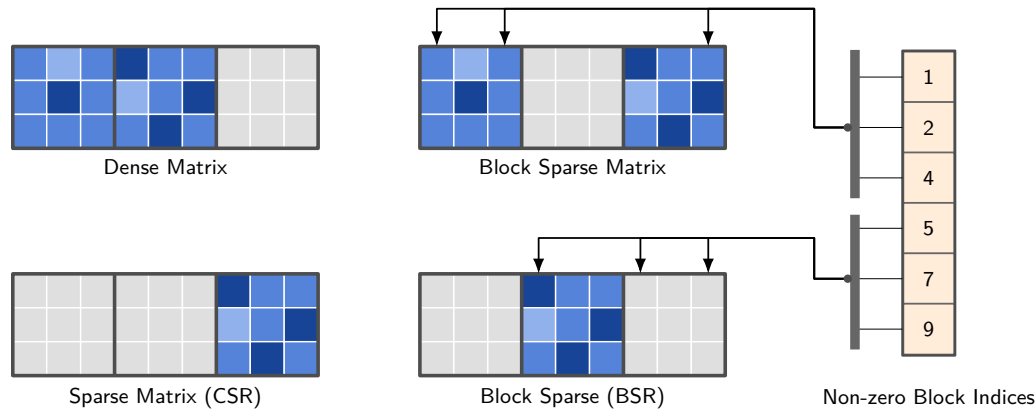


Figure 11.7: Sparse Matrix Layouts: The four panels compare dense, CSR, block-sparse, and BSR matrix occupancy patterns. The separate nonzero-block-index column represents the metadata required to locate retained blocks in the block-sparse cases. Regular block structure keeps this metadata predictable and compact so hardware can skip zero blocks and fetch retained blocks efficiently.

NVIDIA's Sparse Tensor Cores implement a concrete instance of fine-grained structured sparsity: the 2:4 constraint, which requires that exactly two of every contiguous block of four values along the reduction dimension be nonzero (NVIDIA Corporation 2020a; NVIDIA 2020a). In memory, the weight matrix is stored in compressed form: only the two nonzero values are retained, accompanied by a 4-bit metadata tag per block (2 bits per value) indicating each nonzero element's original position within the four-element vector. During execution, the memory hierarchy transfers only the compressed weights and their metadata, halving the required weight memory bandwidth relative to dense execution. Inside the Sparse Tensor Core datapath, the 4-bit metadata steers an internal multiplexer network that selects the matching two values from an uncompressed four-element activation vector. These selected activations and the two nonzero weights are then routed into a half-sized dot-product multiplier tree. Because the hardware computes only on the nonzeros while streaming compressed weights, this design doubles the peak arithmetic instruction throughput—yielding an advertised $2\times$ tensor-math speedup over dense operations when weights are fine-tuned to satisfy the pattern.

The 2:4 pattern reinforces the central reality of the memory wall: hardware accelerators gain efficiency not by computing on zeros faster, but by eliminating their transfer entirely. Halving the weight payload reduces traffic across HBM and on-chip SRAM caches, shifting the workload to a higher operational arithmetic intensity on the roofline curve.

While Sparse Tensor Cores accelerate matrix multiplication by filtering operands through multiplexers within a SIMT datapath, alternative architectures bypass memory bandwidth limits by altering how data moves through space. Rather than repeatedly cycling activations between execution units and centralized register files, systolic arrays forward intermediate values directly between neighboring processing elements across a physical grid.

11.4.6 Systolic arrays

While Tensor Cores package matrix operations into small, specialized instruction tiles, systolic arrays structure processing elements into large two-dimensional grids optimized for continuous data flow and operand reuse. By pulsing inputs horizontally and partial sums vertically through the array, systolic architectures minimize expensive DRAM fetches, enabling low-precision, quantized weights to achieve maximum compute density. The core motivation for systolic architectures stems from the same energy constraint that drives accelerator design: minimizing memory access penalties

through physical operand reuse. A simple energy comparison through the array reveals why this architecture has become central to modern AI accelerators.



Napkin Math 11.1: The energy advantage of pulsing data

Scenario: The systolic architecture improves energy efficiency by keeping operands local as work pulses through the array; the “Systolic” (heartbeat) metaphor reflects this benefit of reusing operands locally. This worked model compares systolic dataflow with a naive implementation that streams every operand through DRAM:

1. **DRAM-streaming baseline:** Loads A , loads B , computes $A \times B + C$, and writes C without cache or register reuse.
 - **Data movement:** 3 loads + 1 write = 4 DRAM accesses (per operation).
 - **Energy:** $\approx 4 \times 640 \text{ pJ} + 1 \text{ pJ (compute)} = 2561 \text{ pJ/op}$.
2. **Systolic Array (128×128 size):** Loads A and B once at the edges. Data “pulses” through 128 processing elements.
 - **Data movement:** 2 loads per 128 operations = 0.016 DRAM accesses (per operation).
 - **Energy:** $\approx 0.016 \times 640 \text{ pJ} + 1 \text{ pJ (compute)} \approx 11 \text{ pJ/op}$.

Systems insight: Under this deliberately naive DRAM-streaming baseline, the worked model gives the systolic array a 232.8× energy advantage. The ratio measures locality, not an inherent gap between vector and systolic arithmetic.

- Concretely, a 128×128 array can sustain 16,384 MACs/cycle with a large energy dividend by pulsing data through processing elements instead of repeatedly loading it from DRAM (Horowitz 2014; Jouppi et al. 2023).
- This locality lets dense arrays sustain many simultaneous MAC operations without paying a DRAM access for every operation.
- **Limitation:** The comparison assumes no effective reuse in the baseline and full reuse across the array. Caches, register blocking, array underutilization, and other dataflows change the ratio substantially.

A systolic array arranges processing elements in a grid pattern, where data flows rhythmically between neighboring units in a synchronized manner, enabling each operand to participate in multiple computations as it propagates through the array. This structured movement minimizes external memory accesses by maximizing local data reuse. A single weight value can contribute to dozens of operations as it moves through the processing elements, transforming the energy profile from memory-bound to compute-efficient execution.

Kung and Leiserson²⁶ (Kung and Leiserson 1979) first introduced systolic arrays, formalizing their use in parallel computing architectures for efficient matrix operations (Kung 1982). Unlike general-purpose execution units, systolic arrays exploit spatial and temporal locality by reusing operands as they propagate through the grid. Google’s TPU exemplifies this architectural approach: in the TPUv4, a 128×128 systolic array of multiply-accumulate units processes matrix operations by streaming data through the array in a pipelined manner (Jouppi et al. 2023). Figure 11.8 follows these data paths: a control unit feeds input buffers that stream data horizontally into the array, while the partial sums each cell produces flow vertically down to the accumulator chain at the bottom, which collects the finished results. Each processing element performs one multiply-accumulate per cycle and passes its operands to its neighbors, so a value loaded once is reused across an entire row or column rather than refetched from memory.

11.4.7 The tiling principle: Bridging graph and silicon

A fundamental mismatch exists between the *computational graph* (which sees a single $4,096 \times 4,096$ matrix multiplication) and the *physical silicon* (which possesses a fixed 128×128 systolic array). Bridging this gap requires tiling: the process of partitioning large tensor operations into “tiles” that fit exactly into the hardware’s fast local memory (SRAM or scratchpad memory).

26 | **Systolic array:** From Greek *sustole* (“contraction”), borrowed from cardiology to evoke rhythmic pumping. Kung and Leiserson used the term for arrays in which data advances through neighboring processing elements on a regular schedule (Kung and Leiserson 1979). Pipeline fill permits concurrent wavefronts and local reuse. The benefit depends on the stationary operand, tile, buffers, and utilization; it does not guarantee fixed DRAM savings. Irregular workloads and poorly aligned shapes remain a weaker fit because fill, drain, and idle elements consume capacity.

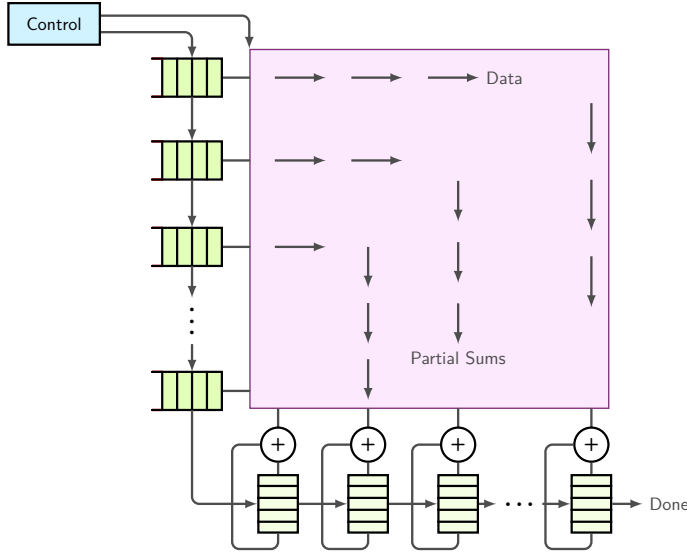
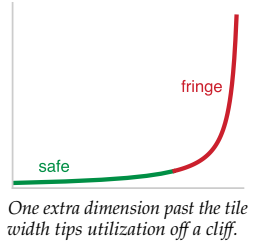


Figure 11.8: Systolic Array Dataflow Architecture: A pipelined grid of processing elements (PEs) passes operands and partial results between neighbors. The illustrated movement is one possible matrix-multiplication dataflow; designs choose which operand or partial sum remains stationary to match reuse, buffers, and tensor shape. Utilization depends on mapping and dimensions.

To process the 4,096-wide worked-example layer on a 128-wide systolic array, the compiler decomposes the result into 1,024 output tiles. Each output tile accumulates across 32 reduction-dimension tiles, so the complete matrix multiplication executes 32,768 **A**-tile-by-**B**-tile products. This decomposition is a physical requirement imposed by memory hierarchy limits: operand tiles are fetched from high-capacity, high-latency HBM, staged in fast SRAM, and pulsed through the systolic array.

Tile dimensions balance on-chip capacity against arithmetic intensity. A larger tile reuses each loaded byte across more multiply-accumulate operations, raising the kernel's arithmetic intensity and pushing it toward the compute-bound regime of the roofline model; the capacity ceiling is how much of **A**, **B**, and **C** fits simultaneously in fast on-chip memory. Tiling sustains high hardware efficiency (η_{hw}) by ensuring that for every byte loaded from main memory, the data is reused $128\times$ within the systolic grid. Conversely, violating this alignment breaches the silicon contract: if a layer's dimensions are not multiples of the tile size (for example, a width of 129 on a 128 array), the system pays a *fringe tax* in underutilized silicon, where 127 units sit idle while one unit finishes the remainder tile. Algorithm 11.1 formalizes the resulting loop nest emitted by the compiler to orchestrate this on-chip staging and accumulation.



Algorithm 11.1 Tiled (blocked) matrix multiply

Require: $\mathbf{A} \in \mathbb{R}^{M \times K}$, $\mathbf{B} \in \mathbb{R}^{K \times N}$; tile sizes T_M, T_N, T_K

Ensure: $\mathbf{C} = \mathbf{AB}$

- 1: **for** each row tile i_0 (size T_M) **do**
 - 2: **for** each column tile j_0 (size T_N) **do**
 - 3: initialize the **C**-tile accumulator to zero on chip
 - 4: **for** each k_0 over K in steps of T_K **do**
 - 5: load **A**-tile $[i_0, k_0]$ and **B**-tile $[k_0, j_0]$ on chip
 - 6: accumulate the tile product on chip $\triangleright$ reuse T_N/T_M per byte
 - 7: **end for**
 - 8: write the **C**-tile back to memory
 - 9: **end for**
 - 10: **end for**
-

Once operand tiles reside in fast local SRAM, the hardware executes the tile product without further off-chip memory traffic. Systolic arrays achieve computational efficiency by routing operands through a structured 2D grid of processing elements (PEs) that communicate directly with adjacent neighbors across dedicated registers, bypassing shared-memory bus contention. In a weight-stationary design—the canonical TPU dataflow mapped in figure 11.8—weights from the **B**-tile are preloaded into local PE registers and held stationary. Input activations from the **A**-tile stream horizontally across rows from left to right, while partial sums accumulate vertically down columns. On each clock cycle, every active PE multiplies its incoming activation by its stationary weight, adds the result to the incoming partial sum from above, and passes the activation rightward to its adjacent neighbor for the next cycle.

Because operands propagate one hop per cycle, synchronized execution requires an intentional time skew. Consider a minimal 2×2 tile product where output element $C_{ij} = A_{i0}B_{0j} + A_{i1}B_{1j}$ requires two products and one accumulation. The weight matrix **B** is held stationary across four PEs: PE₀₀ stores B_{00} , PE₀₁ stores B_{01} , PE₁₀ stores B_{10} , and PE₁₁ stores B_{11} . If activation A_{00} enters PE₀₀ at cycle 0, its partial product $A_{00}B_{00}$ moves down to PE₁₀ at cycle 1. To combine with this arriving partial sum, activation A_{01} must be injected with a one-cycle delay. Inputs along each successive row and column are staggered so operands meeting at any physical coordinate represent identical reduction indices. Local reuse amortizes the initial fill and drain overhead: each loaded activation is reused across an entire row of T_N processing elements, while each stationary weight serves an entire sequence of input vectors before eviction. Sustained throughput, however, remains bounded by off-chip memory bandwidth whenever operand replenishment cannot keep pace with array retirement (section 11.5.1).

A 128×128 systolic array capable of 16,384 operations per cycle requires a continuous operand stream to maintain utilization. On-chip buffers feed activations and weights to the array edges and are replenished from off-chip memory as needed; reuse within the array avoids fetching every operand from HBM each cycle. The TPU v4's 1,200 GB/s HBM2 bandwidth helps sustain this stream, but off-chip bandwidth can still limit models whose working sets and reuse patterns exceed on-chip capacity.

The quantization techniques in section 10.4 reduce model memory footprint by converting FP32 weights to INT8 representations. Converting 32-bit floating-point weights to 8-bit integers reduces weight traffic by $4\times$; whether that changes a kernel from bandwidth bound to compute bound depends on the operation's original arithmetic intensity, the accelerator's INT8 ridge point (the intensity threshold at which its INT8 compute saturates), and the overhead of quantization and dequantization. Similarly, structured pruning removes entire rows or columns of weight matrices, reducing both the data volume that must traverse memory hierarchies and the computation required. In the D·A·M taxonomy, these algorithmic optimizations prove valuable precisely because they target the memory bottleneck that limits accelerator performance in practice.



Systems Perspective 11.1: Matching architecture to workload

Challenge: Systolic arrays must choose which data to keep stationary (in registers) to minimize movement. This choice hard-codes the hardware's preference for certain model types. Table 11.5 previews the three stationary-operand strategies and the workloads each favors; section 11.8 develops each one in full as a general mapping decision.

Table 11.5: Systolic-Array Dataflow Strategies: Three stationary-operand choices for systolic arrays, the reuse pattern each maximizes, and the workload class that benefits, showing how a fixed dataflow choice hard-codes an accelerator’s affinity for specific models.

Strategy	Stationary item	Optimized for	Example Workload
Weight-stationary	Weights (W)	High Reuse of Weights	CNNs (Conv2D): Filters are small and reused across the entire image.
Output-stationary	Partial Sums (C)	High Reuse of Accumulators	Large Batch MatMul: Accumulating results for many inputs against a large weight matrix.
Input-stationary	Inputs (A)	High Reuse of Activations	Transformers: The same activations feed many weight matrices across attention heads.

No single accelerator architecture suits every workload. A chip optimized for weight-stationary dataflow (like early TPUs) excels at CNNs where filter weights are small and reused across many image patches. That same architecture stalls on large language model (LLM) autoregressive inference at batch size 1, where the entire weight matrix must be read from off-chip memory once per generated token with zero batch reuse, forcing architectures toward output-stationary or hybrid dataflow patterns.

11.4.8 Numerics in AI acceleration

Systolic arrays and Tensor Cores often support reduced-precision arithmetic. FP16 halves the bytes per value relative to FP32, and a target may provision more low-precision multiply-accumulate capacity. A $2\times$ speedup is not automatic: it requires a supported kernel and sufficient parallel work without another bottleneck. Building on chapter 10, reduced precision is a hardware-software decision. Input and accumulation formats may differ, affecting accuracy, throughput, energy, and movement.

11.4.8.1 Precision trade-offs

Lower precision is not free. FP16 has a 5-bit exponent and 10 stored fraction bits, giving it more significand precision than BF16 but much less dynamic range than FP32. Its smallest normal value is about 6.1×10^{-5} and its largest finite value is 65,504; subnormals extend lower with reduced precision. BF16 retains FP32’s 8-bit exponent and uses 7 fraction bits, preserving similar dynamic range at lower precision. INT8 has no floating-point exponent and depends on scale, zero-point, and clipping choices. Hardware and software must balance these properties against throughput and bandwidth.

The evolution of AI hardware reflects co-design between numerical methods and hardware capability. Earlier GPU generations lacked the dedicated mixed-precision matrix paths now used for deep learning, even when other units supported several formats. As training and inference methods demonstrated acceptable accuracy with selected lower-precision operations, vendors added native FP16, BF16, and integer tensor paths. Frameworks and libraries expose them through compatible kernels, autocasting, quantization, and calibration. Software gains materialize only when the model selects an efficient path; changing storage alone may reduce bytes without producing the advertised arithmetic speedup.

Precision support is integrated into execution units and memory paths. SIMD and SIMT lanes may support several scalar formats, while Tensor Cores (section 11.4.3) and systolic units (section 11.4.6) expose architecture-specific input and accumulation combinations. Lower precision reduces bytes per operand and may increase operations per cycle. Tiling and dataflow, not precision itself, determine reuse, while conversion, scaling, or fallback overhead can offset part of the gain. A format is useful only when the kernel and numerical policy can employ it safely.

Despite the advantages of reduced precision, deep learning models cannot always rely solely on low-bit representations. To address this challenge, modern AI accelerators implement mixed-precision computing, where different numerical formats are used at different stages of execution. These precision choices affect numerical reliability: matrix multiplications may be performed in FP16

or BF16, while accumulations are maintained in FP32 to prevent precision loss. Similarly, inference engines use INT8 arithmetic while preserving key activations in higher precision when necessary.

11.4.8.2 Mixed-precision computing

Modern AI accelerators increasingly support mixed-precision execution, allowing different numerical formats to be used at various stages of computation. Training workloads often use FP16 or BF16 for matrix multiplications, while maintaining FP32 accumulations to preserve precision (Micikevicius et al. 2017; Mellempudi et al. 2019). The software implementation of mixed-precision training, including loss scaling techniques and framework support, is covered in section 8.6.3. Inference workloads, by contrast, optimize for INT8 or even INT4, achieving high efficiency while retaining acceptable accuracy.

The shift toward precision diversity is evident in the evolution of AI hardware. Early architectures such as NVIDIA Volta provided limited support for lower precision beyond FP16, whereas later architectures, including Turing and Ampere, expanded the range of supported formats. Table 11.6 traces this progression: Ampere GPUs introduced TF32 as a hybrid between FP32 and FP16 (NVIDIA 2020b), alongside broader support for BF16, INT8, and INT4 (NVIDIA Corporation 2017, 2018, 2020a).

Table 11.6: Precision Support Evolution: From Volta through Ampere, Tensor Core support expanded from FP16 to TF32, BF16, and integer formats, broadening the precision choices available to training and inference; the CUDA-core column shows the separate scalar/vector formats each generation supports.

Architecture	Year	Supported Tensor Core precisions	Supported CUDA Core Precisions
Volta	2017	FP16	FP64, FP32, FP16
Turing	2018	FP16, INT8, INT4, INT1	FP64, FP32, FP16, INT8
Ampere	2020	FP64, TF32, BF16, FP16, INT8, INT4	FP64, FP32, FP16, BF16, INT8

Newer architectures incorporate a growing diversity of numerical formats because different workloads bind at different points on the accuracy-throughput-energy trade-off. Precision support is therefore another form of workload matching, not a generic feature checklist.

The precision format used in hardware design has cascading implications across the entire system. Reducing from FP32 to FP16 cuts memory traffic in half, which matters far more than it might seem: because memory access dominates energy consumption, halving memory traffic can substantially reduce energy per inference when data movement is the bottleneck (Horowitz 2014). Simultaneously, Tensor Cores and systolic arrays can pack more lower-precision multiply-accumulate units into the same silicon area, raising peak throughput (Dally et al. 2021; Dally 2023). Lower-precision integer arithmetic also consumes less energy than FP32 in representative technology studies (Horowitz 2014), and the inference-focused TPUv1 was built around 8-bit multiply units (Jouppi et al. 2017). The systems insight is that reduced precision does not merely “save bits”: it simultaneously relieves the memory bandwidth bottleneck *and* increases compute density, attacking both sides of the roofline at once.

As AI models continue to scale, precision support connects the compute primitive discussion back to the memory wall: lower-bit formats matter when they reduce the bytes moved and keep the hardware’s matrix engines fed. The remaining architectural question is how these execution units, precision formats, and memory paths integrate into complete accelerator systems. Architectural integration determines how efficiently computational primitives become usable accelerator throughput. SIMD lanes, Tensor Cores, and systolic arrays are building blocks, but their full-chip organization varies significantly across AI processors; the choice of execution units, their numerical precision support, and their connectivity shape how effectively hardware can scale for deep learning workloads.

11.4.9 Intra-node interconnects: Scaling the stack

Mastery of the single-machine stack requires understanding how bits move between GPUs and the CPU. In the 1–8 GPU regime, scaling is achieved through high-speed intra-node interconnects such as NVLink and host-to-device PCIe transfers that mitigate the memory wall. These links

form a **bandwidth taper**: data-movement speed falls at each step away from the compute units, from on-package HBM through the GPU-to-GPU NVLink bridge down to the host PCIe link. The PCIe step is much slower than the accelerator-local memory and inter-GPU fabric, so any data path that touches the CPU can become a performance hazard, the “PCIe Wall” that NVLink exists to avoid (NVIDIA Corporation 2020c). Section 11.5.5.1 develops this hierarchy quantitatively, where host-accelerator communication is the operative concern.

Modern AI processors exhibit a range of design trade-offs based on their intended applications, and comparing their configurations reveals how deployment constraints drive architectural divergence. A training-optimized accelerator like the NVIDIA A100 packs many Streaming Multiprocessors with wide SIMD units and FP16 Tensor Cores because training throughput scales with aggregate multiply-accumulate capacity (NVIDIA Corporation 2020a). Google’s TPUv4 makes a radically different bet: just two cores per chip, each containing massive BF16 systolic arrays, a design that trades programmer flexibility for efficiency on dense matrix multiplications (Jouppi et al. 2023). At the inference end, Intel’s Sapphire Rapids dedicates Advanced Matrix Extensions (AMX) tile engines to INT8 and BF16, reflecting the insight from chapter 10 that inference models tolerate reduced precision (Intel Corporation 2021a). Mobile neural engines take this further by shrinking matrix engines into low-power SoC blocks, prioritizing energy efficiency per operation over peak throughput. Table 11.7 compares these architectural configurations.

Table 11.7: AI Processor Configurations: Modern AI processors prioritize different execution unit characteristics for specific workloads: NVIDIA A100 uses wide SIMD and Tensor Cores for training, Google TPUv4 emphasizes high-throughput BF16 matrix multiplication, Intel Sapphire Rapids focuses on INT8-optimized inference, and mobile NPUs prioritize low-power execution. These variations in SIMD width, Tensor Core size, and processing element count reflect the growing diversity in AI hardware architectures.

Processor	Vector execution	Matrix engine	Processing Elements	Primary workloads
NVIDIA A100	CUDA FP/INT lanes	Tensor Core instruction tiles	108 SMs	Training, HPC
Google TPUv4	Vector unit	128×128 BF16 systolic arrays	2 cores/chip	Training
Intel Sapphire	512-bit AVX-512	AMX tiles (up to 16 rows by 64 bytes each)	Up to 60 cores	Inference
Mobile NPU	CPU/GPU/DSP vectors	Small matrix engines	Integrated NPU blocks	Mobile inference

The pattern across these configurations reveals a consistent engineering principle: each design sacrifices generality to optimize for its target workload’s dominant operation and precision. Training chips invest silicon in wide floating-point datapaths; inference chips trade precision for throughput; mobile chips trade throughput for energy efficiency. No single design dominates across all workloads, which is precisely why hardware selection depends on workload analysis rather than headline specifications.

11.4.10 Cost-performance analysis

Architectural specifications define computational potential, but deployment decisions require cost-performance analysis. Raw compute is only one input: delivered performance may be limited by arithmetic throughput, memory bandwidth, interconnect latency, software overhead, or low operational utilization.

The physical energy differential established in section 11.2.5—where moving operands from off-chip DRAM consumes orders of magnitude more energy than performing arithmetic—drives accelerator specialization. Because memory access costs dominate computation, accelerators achieve only a fraction of their peak throughput on memory-bound kernels. Conversely, architectures that maximize data reuse, such as systolic arrays executing dense matrix multiplications, sustain substantially higher utilization by amortizing each memory fetch across multiple arithmetic operations.

Consider an engineering team choosing between deploying a larger cluster of older accelerators or a smaller cluster of newer units. Peak FLOP/s figures mislead whenever dominant kernels are memory-bandwidth or communication bound. In a large model pipeline, dense feed-forward and attention projection layers may remain compute bound, whereas embedding lookups, layer

normalization, residual additions, and autoregressive decoding steps are strictly memory bound. The decisive metric is delivered workload throughput per dollar under realistic cluster utilization, memory capacity constraints, available memory bandwidth, and scale-out interconnect efficiency.

These architectural and economic constraints explain the rapid industry adoption of newer accelerators despite substantial increases in nominal unit price. For memory-bound workloads, generational improvements in memory bandwidth and compiler code generation dominate end-to-end execution time. Cloud deployments introduce additional trade-offs: hourly rental rates, dynamic instance utilization, and infrastructure overhead continuously shift the economic break-even point between purchasing on-premises hardware and leasing cloud capacity.

Table 11.8 provides an illustrative cost worksheet for common accelerators. List prices vary across vendor, region, contract terms, and procurement volume. Furthermore, the throughput rows correspond to distinct numerical precision modes, meaning the price-per-peak-operation metric is meaningful within a single row but invalid as a cross-device benchmark. The table establishes a framework for making economic assumptions explicit rather than offering a universal hardware ranking.

Table 11.8: Illustrative Accelerator Cost Worksheet: Each row divides an approximate acquisition-price assumption by the representative peak mode shown. Because precision, sparsity assumptions, and price sources differ, the ratios are not comparable as workload performance. Total cost also includes utilization, memory capacity, power, cooling, interconnect, software, and infrastructure; TPU acquisition cost is estimated from cloud pricing.

Accelerator	List price	Representative Peak Throughput (precision shown)	Memory Bandwidth	Price/performance
NVIDIA V100	~\$10,000	125 TFLOP/s	900 GB/s	\$80/(TFLOP/s)
NVIDIA A100	~\$15,000	312 TFLOP/s	2,039 GB/s	\$48.1/(TFLOP/s)
NVIDIA H100	~\$25,000–30,000	494 TFLOP/s (TF32)	3,350 GB/s	~\$50.6/(TFLOP/s)
Google TPUv4	~\$8,000*	275 TFLOP/s (BF16)	1,200 GB/s	~\$29.1/(TFLOP/s)
Intel Gaudi 2	~\$12,000	865 TFLOP/s (FP8)	2,450 GB/s	\$13.9/(TFLOP/s)

The worksheet demonstrates why accelerator selection cannot be reduced to a single headline specification. A bandwidth-bound kernel benefits more from the H100's 3,350 GB/s memory subsystem than from an incremental gain in peak arithmetic, whereas a compute-bound general matrix multiplication (GEMM) shows the opposite sensitivity. Device memory capacity dictates whether working sets fit on-chip without spilling or partitioning, while compiler and driver maturity determines whether the application can actually reach the advertised datapath.

Software framework selection directly impacts these economic trade-offs: graph compilers, operator fusion passes, and runtime dispatchers govern the efficiency with which high-level model descriptions map onto physical execution units. Concrete framework-level optimization strategies are examined in chapter 7, while empirical measurement and benchmarking methodologies are developed in chapter 12.

Vector pipelines, matrix-tile engines, and Tensor Cores provide massive peak arithmetic capability. An NVIDIA A100 advertises 312 TFLOP/s on its FP16 tensor path, and subsequent accelerator designs add narrow-precision FP8 execution modes (NVIDIA Corporation 2020a; Kuzmin et al. 2022; Micikevicius et al. 2022). NVIDIA's Blackwell (B200) architecture extends this trajectory with an FP4 tensor datapath, rated at up to 9 PFLOP/s dense or 18 PFLOP/s sparse peak throughput per chip (NVIDIA Corporation 2024). Reaching those theoretical ceilings requires numerical representations, kernel implementations, and sparsity patterns that align exactly with the specialized hardware paths. Narrower numerical formats expand the available design space, but numerical stability and dynamic range requirements dictate whether a workload can safely utilize FP4.

Dividing a model's total operation count by peak arithmetic throughput produces an idealized lower bound on execution time, not a realistic inference latency. On an A100, dividing the arithmetic operations in ResNet-50 by 312 TFLOP/s yields a microsecond-scale estimate. Real ResNet-50 inference, however, requires milliseconds. This orders-of-magnitude discrepancy exposes the central tension in machine learning systems: arithmetic throughput has far outpaced the memory bandwidth required to feed the execution units. *Moving data from memory costs orders of magnitude more energy than arithmetic, and memory bandwidth has improved more slowly than tensor arithmetic throughput.* This

physical imbalance dictates whether that advertised 312 TFLOP/s delivers high sustained hardware utilization or remains largely idle on a given workload (Horowitz 2014; Gholami et al. 2024).

Resolving this performance gap requires analyzing the memory hierarchies that supply the compute units. The memory subsystem is not passive scaffolding; in conjunction with arithmetic intensity and kernel scheduling, it determines the fraction of peak compute an algorithm can extract from the machine.

11.5 AI Memory Systems

ResNet-50 can expose the gap between accelerator arithmetic and memory: convolution weights, activations, and intermediate results still have to arrive on time. Modern accelerators reach hundreds of TFLOP/s or more on selected low-precision paths (NVIDIA Corporation 2020a, 2024; Choquette 2023), but a kernel cannot use that capacity when its required traffic exceeds the relevant memory bandwidth. The **AI memory wall** names this growing mismatch between arithmetic demand and the memory system that feeds it.

Unlike conventional workloads, ML models require frequent access to large volumes of parameters, activations, and intermediate results, leading to substantial memory bandwidth demands. This challenge intersects with the data management strategies covered in chapter 4. Modern AI hardware addresses these demands through advanced memory hierarchies, efficient data movement techniques, and compression strategies that promote efficient execution.

11.5.1 Understanding the AI memory wall

Definition 11.4: AI memory wall

The AI memory wall is the ML accelerator performance constraint that arises when arithmetic throughput (R_{peak}) outpaces memory bandwidth (BW). The core issue is whether the memory system can supply operands and absorb results quickly enough to sustain the specialized primitives introduced in section 11.4.

1. **Significance:** A workload has reached the memory wall when increasing FLOP/s alone no longer reduces execution time because the $\frac{D_{\text{vol}}}{\text{BW}}$ term dominates the overall runtime.
2. **Distinction:** Unlike a general-purpose memory wall, which affects all computing, the AI memory wall is driven by the large parameter volumes and intermediate activation tensors required by deep learning.
3. **Common pitfall:** A frequent misconception is that the memory wall is “fixed” by more memory. In reality, it is a bandwidth-latency gap: even with infinite capacity, the speed of moving data between memory and compute remains the fundamental physical bottleneck.

Data access can dominate the energy budget of a memory-intensive kernel.²⁷ Figure 11.9 compares representative operation costs from one technology study, revealing the multi-order-of-magnitude gap between simple local arithmetic and an off-chip DRAM access.

11.5.1.1 Quantifying the compute-memory performance gap

The energy disparity that figure 11.9 captures grows more severe with each hardware generation. Over the past two decades, peak computational capabilities have grown substantially faster than DRAM bandwidth (Gholami et al. 2024). This divergence stems from physical scaling constraints: while transistor scaling and dense systolic arrays multiply arithmetic throughput, memory bandwidth remains bounded by pin counts, package-substrate routing density, and interface signaling power. Representative high-end accelerators can deliver on the order of 10^3 TFLOP/s of peak tensor throughput (for example, NVIDIA H100 delivering 989 TFLOP/s in FP16 or nearly 2,000 TFLOP/s in dense FP8) while providing approximately 3.35 TB/s of memory bandwidth (Choquette 2023). This implies that on the order of 10^2 FLOP of work per byte moved is required to fully use the compute, which can exceed the arithmetic intensity of many practical neural network workloads.

When a workload’s arithmetic intensity falls below this threshold, the execution budget shifts entirely to data transfer ($\frac{D_{\text{vol}}}{\text{BW}}$). Accelerators attempt to tolerate memory latency—typically hundreds

²⁷ | **Von Neumann bottleneck:** Separating storage from compute requires instructions and data to cross interfaces rather than remain in local registers. In the representative 45 nm estimates from Horowitz (2014), a DRAM access consumes over $20,000\times$ the energy of an INT8 addition. The exact ratio changes with technology and operation, but locality remains a first-order design concern.

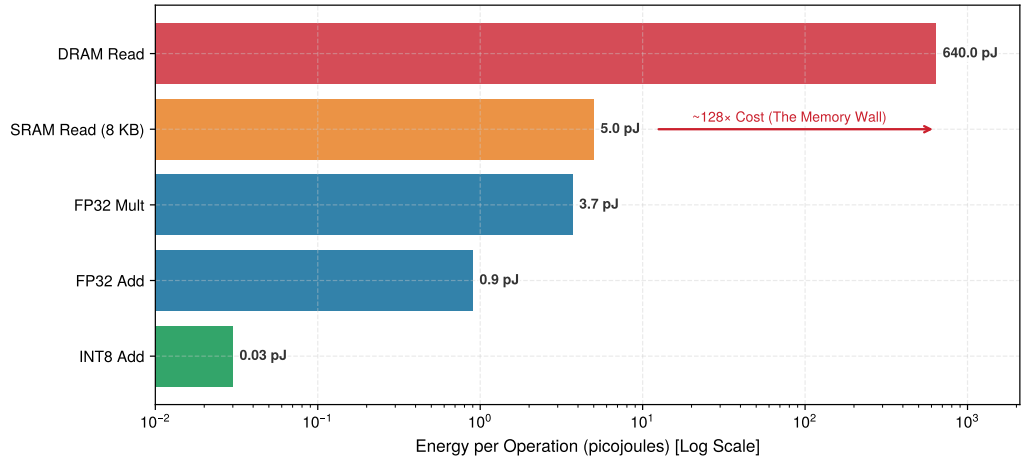


Figure 11.9: The Energy Hierarchy: Representative 45 nm energy estimates on a logarithmic scale from Horowitz (2014). An off-chip DRAM access costs about $128\times$ an 8 KB SRAM read and over $20,000\times$ an INT8 addition in this model. The values are not universal device specifications; they illustrate why reuse in registers and SRAM can matter more than reducing arithmetic count alone.

of clock cycles for off-chip DRAM—by maintaining thousands of concurrent execution threads and switching between active warps while memory requests remain in flight. However, when memory bandwidth saturates or when register pressure limits occupancy, the scheduler runs out of independent instructions to issue. Compute pipelines then stall, leaving high-throughput tensor cores idle while waiting for operands. Simultaneously, driving wide off-chip memory buses across capacitive printed circuit board traces and PHY interfaces consumes a substantial fraction of the chip’s TDP, directly constraining the energy budget available for arithmetic logic (Horowitz 2014; Sze et al. 2017).

11.5.2 Hardware balance (I_{ridge}): The paradigm partition

Different devices place the compute-bandwidth boundary at different intensities. The **hardware balance** (I_{ridge}) is the ratio of peak arithmetic throughput to peak memory bandwidth. In the roofline model, it is the ridge point where the compute and bandwidth ceilings intersect:

$$I_{\text{ridge}} = \frac{R_{\text{peak}}}{\text{BW}}$$

The ratio sets a device-specific boundary. Depending on the chosen precision, a high-end accelerator such as H100 may require roughly 150–300 FLOP/byte to reach its arithmetic ceiling, whereas a microcontroller may have a much lower ridge point. The same kernel can therefore be bandwidth bound on a compute-dense accelerator and compute bound on an edge processor. Hardware balance does not label either device as universally efficient; it identifies how much reuse a workload needs on that device.

Peak computational capability has expanded faster than memory bus performance across accelerator generations, and the size of that mismatch is what sets how much data reuse a kernel needs to stay fed. Figure 11.10 separates the two growth rates that produce the ridge point, each normalized to the V100 baseline, so the widening distance between them can be read directly.

The threshold required for a kernel to become compute bound has shifted across accelerator hardware generations. Across successive data center architectures (figure 11.11), hardware ridge points climb from V100 through B200, steadily increasing the arithmetic intensity required to saturate peak compute throughput.

Beyond performance limitations, memory access imposes a steep energy cost. Fetching data from off-chip DRAM consumes orders of magnitude more energy than performing arithmetic operations on-chip (Horowitz 2014). An arithmetic unit drives signals over micrometer-scale wires on silicon,

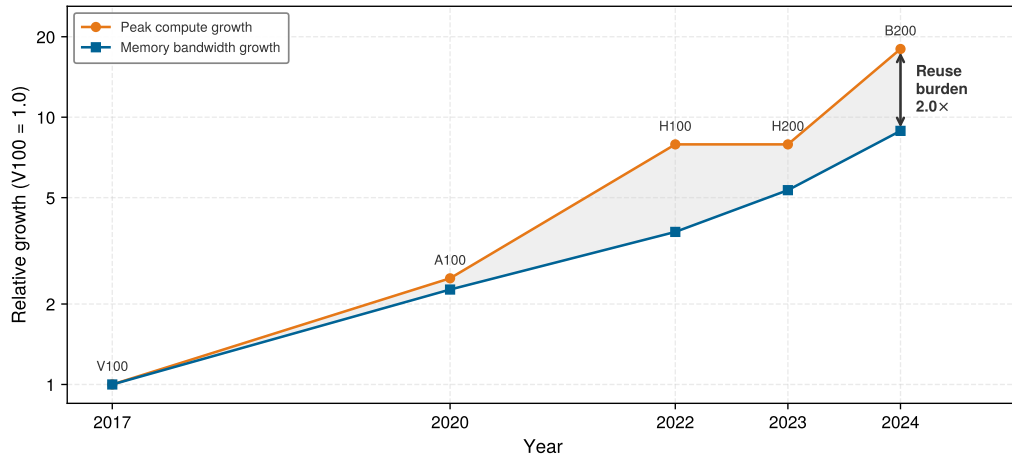


Figure 11.10: The Compute-Bandwidth Divergence: Peak dense FP16 throughput and memory bandwidth across five NVIDIA datacenter generations, each normalized to its V100 baseline on a log scale. Over 2017–2024 compute grows 18× while bandwidth grows 8.9×, so the shaded separation between the curves widens by 2.0×. That separation is the data reuse a kernel must supply to keep the arithmetic units busy. The H200 step is the instructive exception: bandwidth rose while peak compute held flat, narrowing the gap for one generation.

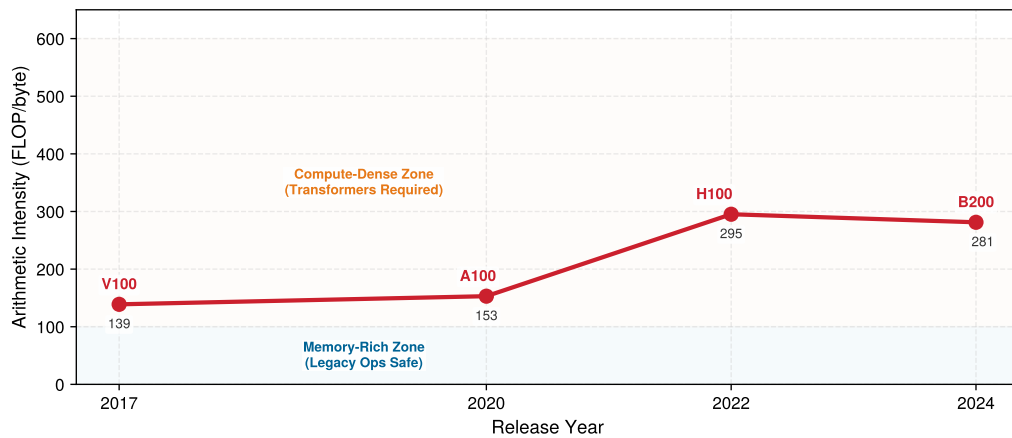


Figure 11.11: The Rising Ridge: Hardware arithmetic intensity (FLOP/byte) over time computed as each chip’s dense FP16 tensor peak divided by its memory bandwidth. This is the arithmetic intensity a kernel must reach before compute rather than bandwidth limits it. As compute capability grows faster than memory bandwidth, the ridge point rises from V100 through H100 and remains high on B200. This trend explains why architectures with high data reuse flourish while low-reuse workloads face a growing hardware tax.

consuming tens of femtojoules per operation; fetching an operand from DRAM requires driving macroscopic PCB traces and package interconnects, consuming tens of picojoules. In machine learning models where billions of parameters and activations stream across the bus, data movement quickly dominates the total power envelope. This energy differential drives architectural decisions: Google’s TPUv1 achieved 30–80× better performance per watt than contemporary CPUs and GPUs on inference benchmarks by minimizing off-chip data movement through systolic arrays and large on-chip SRAM (Jouppi et al. 2017). These design choices demonstrate that energy budgets, as much as cycle budgets, dictate what architectures can execute in production.

11.5.2.1 Memory access patterns in ML workloads

Beyond raw computational throughput, an accelerator’s sustained efficiency depends on supplying operands to execution units without pipeline stalls. Deep neural networks impose three concurrent

demands on this data supply. First, model parameters (weights and biases) may reach hundreds of billions of values (Brown et al. 2020). In autoregressive generation at batch size 1, every parameter must be streamed from memory for each generated token, collapsing arithmetic intensity to roughly 1 FLOP/byte. Second, intermediate activations produced at each layer must be buffered in memory for subsequent operations; during training, deep architectures must retain these activations across the entire forward pass until backpropagation consumes them. Third, training adds gradient and optimizer traffic: backpropagation computes gradients for every parameter, while algorithms such as Adaptive Moment Estimation (Adam) read and write multi-word momentum states, multiplying data movement volume per parameter update (Narayanan et al. 2021; Kwon and Rhu 2018).

The physical cost of this data movement depends on where each tensor resides in the memory hierarchy. While an on-chip register access completes in a single clock cycle at sub-picojoule energy, an off-chip DRAM fetch stalls execution for hundreds of cycles and expends orders of magnitude more energy. Tracing a single tensor through the memory tiers during an inference pass makes this disparity concrete, as demonstrated by the keyword spotting (KWS) pipeline in lighthouse 11.2.



Lighthouse 11.2: Life of a tensor: GPU-hosted KWS

Recall the Keyword Spotting lighthouse summarized in table 2.2. If the same one-second audio clip is batched on a GPU-hosted inference path, its physical journey through the memory hierarchy looks like this:

1. **DRAM (HBM):** The tensor starts here.
 - **Size:** $16,000 \text{ samples} \times 2 \text{ bytes (FP16)} = 32 \text{ KB}$.
 - **Latency:** Fetching this from off-chip memory takes $\sim 300 \text{ ns}$ (plus queuing delay).
 - **Energy:** Cost is $\sim 20 \text{ pJ/bit}$. High cost.
2. **L2 cache:** Device-memory requests may be served here when the requested cache lines are resident.
 - **Latency:** $\sim 4 \text{ ns}$.
 - **Access:** Shared across multiple Streaming Multiprocessors (SMs).
3. **L1 cache/shared memory:** A specific SM works on a tile of the audio, using hardware-managed cache or explicitly staged shared memory.
 - **Latency:** $\sim 1 \text{ ns}$.
 - **Locality:** If an L1 request misses, it can still be served by L2 before an HBM access is required.
4. **Registers:** The Tensor Core operates here.
 - **Latency:** Typically a small number of cycles, depending on the instruction and architecture.
 - **Throughput:** 312 TFLOP/s .
 - **Energy:** Cost is $\sim 0.1 \text{ pJ/bit}$.

Systems insight: Performance is bounded by the lower of peak compute throughput and bandwidth times arithmetic intensity. Which memory link supplies the relevant bandwidth depends on where the working set resides; HBM-to-L2 is one possible limiting path, not the roofline's sole determinant.

The boundary between memory-bound and compute-bound execution emerges directly from the balance between data transfer time and arithmetic execution time. For a workload requiring data volume D_{vol} (bytes) across available memory bandwidth BW (bytes/s), and executing O floating-

point operations on hardware with peak throughput R_{peak} (FLOP/s) at realized hardware utilization η_{hw} , the respective execution times are:

$$T_{\text{mem}} = \frac{D_{\text{vol}}}{\text{BW}}$$

$$T_{\text{compute}} = \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}$$

In a strictly sequential model without overlap, $T_{\text{mem}} > T_{\text{compute}}$ identifies memory-bound execution. When hardware overlaps memory transfers with computation, the larger term sets the theoretical lower bound on execution time. In practice, however, data dependencies, memory controller queueing, and partial pipeline overlap dictate how much arithmetic hardware remains idle while waiting for operands.

The structural widening between model footprints and available memory bandwidth is what turns T_{mem} into the binding constraint across generations. Figure 11.12 quantifies this disparity across public milestone models and accelerator architectures (Krizhevsky et al. 2012; Brown et al. 2020; Chowdhery et al. 2022; Dubey et al. 2024). Even high-bandwidth accelerators such as NVIDIA B200 and AMD MI300X/MI325X cannot close this growth-rate gap through bandwidth scaling alone (NVIDIA Corporation 2024; AMD 2023). Parameter counts for proprietary systems such as GPT-4 and Gemini are not officially disclosed, so they are not plotted as factual data points.

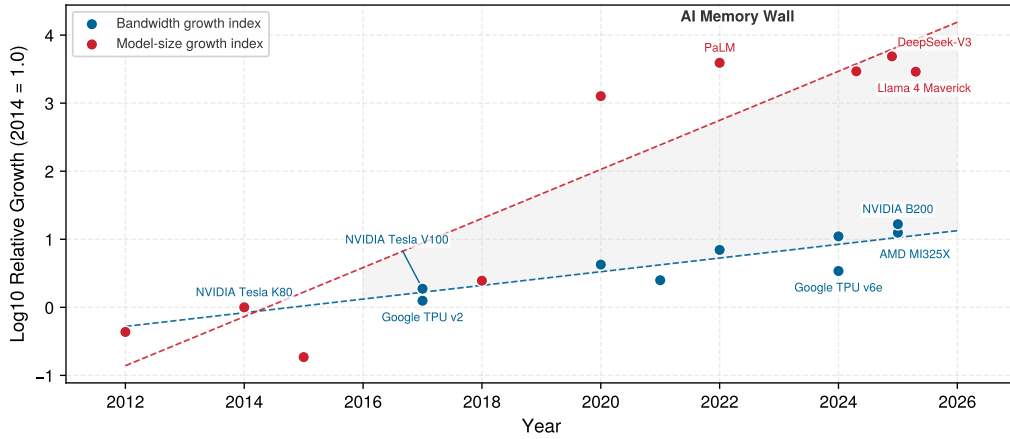


Figure 11.12: Model Size vs. Hardware Bandwidth: Both series are the base-10 logarithm of growth relative to 2014, so each unit on the vertical axis is a tenfold increase. Over the plotted decade parameter counts peak at roughly $4,900\times$ against bandwidth's $17\times$. The shaded band is a gap between logarithms rather than a difference, so its width reads as a ratio, and the two fitted trends diverge at a constant rate.

11.5.2.2 Irregular memory access

Production machine learning workloads combine regular dense matrix kernels with irregular memory access patterns that defeat conventional caching and prefetching hardware. While dense operations like GEMM and convolution stream contiguous blocks that map cleanly onto hardware caches and systolic arrays, irregular operations rely on indirect, sparse, or input-dependent memory addresses. Table 11.9 contrasts these two memory regimes across access geometry, cache locality, and hardware pressure.

Table 11.9: Regular and Irregular Memory Access: Regular dense kernels expose tileable reuse and predictable addresses, whereas irregular ML components rely more on indirect or input-dependent access, weakening caching and prefetching and adding metadata or load-balance overhead. A single model may contain both regimes.

Feature	Regular dense kernels	Irregular ML components
Access pattern	Contiguous or tiled	Indirect, scattered, or input-dependent
Cache locality	Often high after tiling	Often lower and workload-dependent
Data reuse	Structured across tiles or batches	Sparse or dynamic, depending on the operator
Dependencies	Static loop nests	Input-dependent routing or variable shapes
Examples	GEMM, convolution, dense attention	Embedding lookup, unstructured sparsity, MoE routing
Primary pressure	Compute or bandwidth, depending on arithmetic intensity	Latency, bandwidth, metadata, and load imbalance
Energy consequence	Depends on reuse and memory tier	Extra movement and metadata can increase energy

Within a single neural network, memory access behavior varies sharply across layer types. Convolutional layers exhibit high spatial locality and register reuse as small weight kernels slide across input feature maps. Fully connected layers and dense attention execute regular matrix multiplications, but their large parameter matrices or key-value caches can exceed on-chip SRAM capacity and become bandwidth bound when operational reuse is low. Variable sequence lengths complicate memory buffer allocation, but the attention operations within each sequence remain regular, contiguous matrix multiplies.

True memory irregularity arises from three distinct architectural patterns: unstructured sparsity,²⁸ embedding lookups, and dynamic token routing. In unstructured sparsity, nonzero weights or activations are stored in compressed formats (such as CSR or COO) that require pointer indirection to decode operand coordinates. In embedding tables, sparse categorical IDs select arbitrary rows from tables spanning tens of gigabytes in DRAM, transforming sequential memory transfers into uncoalesced, scattered reads. In Mixture-of-Experts (MoE) models, routing gates direct tokens to different expert networks based on input context; while deterministic for a given input, token assignments vary across batches, disrupting prefetch pipelines and unbalancing execution queues across parallel cores.

Irregularity degrades performance through distinct mechanisms, and treating them as one memory problem can point to the wrong optimization. Scattered addresses may turn one logical access into several memory transactions, so each fetched cache line carries few useful payload bytes. Indirection can also serialize address generation: the system must fetch an index or pointer before it knows which payload to request, limiting memory-level parallelism even when aggregate bandwidth remains available. Input-dependent routing creates a third failure mode when some processing elements receive longer queues than others; completion time follows the slowest queue while neighboring units idle. Variable-sized allocations may fragment capacity, but fragmentation is neither necessary nor sufficient for irregular access. A sequential stream can saturate bandwidth without being irregular, whereas a dependent lookup chain can stall far below peak bandwidth.

Diagnosis should therefore begin with the mechanism that prevents forward progress. A low ratio of useful payload bytes to transferred bytes indicates poor spatial utilization or coalescing. Long memory stalls alongside unused bandwidth indicate dependent misses or too few concurrent requests to hide latency. Uneven queue depths or per-route work reveal load imbalance rather than a cache-capacity problem. Cache hit rate alone cannot distinguish these cases because a miss can waste bandwidth, expose latency, or both. Each diagnosis implies a different remedy: packing or blocking improves spatial utilization, reordering and prefetching expose concurrent requests, structured sparse formats reduce indexing overhead, and routing or partitioning balances work. The memory hierarchy determines the cost of the remaining accesses, while dataflow and tensor-layout choices determine how much irregularity software can remove before execution reaches the hardware.

The optimization order follows that diagnosis. Representation comes first when the format is under system control because a block-structured sparse format or grouped routing can remove indirection before a kernel executes. Request organization comes next: accesses issued together should target adjacent addresses, and enough independent requests must remain in flight to hide unavoidable latency. Additional cache capacity or bandwidth helps only when profiling shows that the regularized request stream still exhausts that resource. More bandwidth cannot resolve a serial

²⁸ **Sparsity and memory irregularity:** Unstructured sparse formats commonly gather nonzero elements through indirect addressing, which can weaken sequential access and make metadata overhead significant. Structured sparsity preserves hardware-friendly blocks, so its access behavior differs from arbitrary pruning. Without suitable kernels or hardware support, an unstructured sparse model can run slower than its dense counterpart despite performing fewer arithmetic operations.

pointer dependency, and more capacity cannot rebalance skewed work. The objective is to expose as much regular structure as possible while confining unavoidable irregularity to the smallest portion of execution.

Software can mitigate irregular access by regularizing tensor layouts, enforcing structured sparsity, and sorting dynamic routes into uniform batches. Yet software restructuring cannot eliminate the physical penalty of data movement: whether a memory request is contiguous or scattered, every transferred byte must cross physical wires governed by finite bandwidth, propagation latency, and energy dissipation. Bounding these physical costs requires organizing storage into cascading hardware tiers. Rather than treating memory as a uniform pool, accelerator architectures deploy multilevel memory hierarchies that trade off capacity against latency and energy at every tier.

11.5.3 Memory hierarchy

No single storage technology can simultaneously satisfy the multi-terabyte-per-second bandwidth required by dense execution units and the tens of gigabytes of capacity required to hold modern model parameters. Accelerators resolve this tension through multilevel memory hierarchies. Rather than treating memory as a monolithic address space, the machine organizes storage into distinct physical tiers, each optimized for a specific balance of capacity, access latency, bandwidth, and energy dissipation.

At the apex of the hierarchy, register files and on-chip SRAM caches supply operands to arithmetic units with sub-nanosecond latency and single-digit picojoule energy costs, but their capacity is strictly limited by silicon die area. At the base, nonvolatile flash storage holds hundreds of gigabytes of model checkpoints and training datasets, but requires milliseconds or microseconds to access. Between these boundaries, software-managed scratchpads stage active tensor tiles, while device-local DRAM (such as HBM or GDDR) provides the primary working memory for parameters and activations.

This organization establishes an inverse relationship across tiers: as storage capacity expands and physical distance from arithmetic units grows, bandwidth falls, latency lengthens by orders of magnitude, and energy consumption per bit escalates. Table 11.10 compares these tiers across latency, bandwidth, capacity, and functional roles in deep learning workloads.

Table 11.10: Memory Hierarchy Trade-Offs: AI accelerators use a multilevel memory hierarchy to balance performance and capacity. Each level provides distinct latency, bandwidth, and capacity characteristics that dictate how neural network components (weights, activations, and intermediate results) should be allocated to minimize bottlenecks and maximize throughput. Relative labels describe the typical ordering; exact values depend on architecture. Section D.3.1 supplies registry-backed reference latencies.

Memory Level	Relative latency	Bandwidth	Relative capacity	Example Use in Deep Learning
Registers	Lowest	Highest	Tiny	Storing operands for immediate computation
L1/L2 Cache (SRAM)	Very low	High	Small	Caching frequently accessed activations and small weight blocks
Scratchpad memory	Very low	High	Small to medium	Software-managed storage for intermediate computations
Device DRAM (HBM, GDDR, or LPDDR)	High	Moderate–Very High	Large	Storing model parameters and activations that do not fit on-chip
Host DRAM (typically DDR)	High locally; transfer adds latency	Moderate	Large to very large	Staging or offloading data outside accelerator memory
Flash Storage (SSD/NVMe)	Very high	Low	Very large	Storing pretrained models and checkpoints for later loading

A recurring systems question is why accelerators cannot bypass on-chip SRAM entirely and stream all operands directly from high-capacity off-chip DRAM. The physical constraint preventing this design is the latency floor imposed by chip geometry, electrical signal propagation, and DRAM cell access dynamics.

Napkin Math 11.2: The speed of light limit

Problem: Why is on-chip SRAM necessary instead of fetching all data from HBM?

Math:

1. **Distance:** On an H100-class 814 mm² die, signals travel ~20 mm.
2. **Propagation latency:** At $\approx 0.5c$ in silicon, $20 \text{ mm} / (0.5c) \approx 130 \text{ ps}$.
3. **Clock cycle:** At 2 GHz, $1 / (2 \text{ GHz}) = 500 \text{ ps}$.
4. **DRAM:** Off-chip HBM sits millimeters away on the package, but DRAM access latency plus protocol overhead = 100+ cycles.

Systems insight: The 20 mm propagation estimate is about 130 ps, less than one 500 ps clock cycle, so distance alone does not prove a one-cycle access impossible. A real HBM access includes DRAM-array latency, serialization, protocol traversal, interconnect delay, and queuing, producing this 100-plus-cycle access path. Local registers and SRAM are therefore required to feed compute units at 2 GHz. For transformer models, this complete access path is why weights must be staged in SRAM in tiles rather than read on demand from HBM, and why repeatedly streaming an entire KV cache can collapse inference throughput when its memory traffic cannot be hidden behind arithmetic.

11.5.3.1 On-chip memory

The **on-chip memory** subsystem integrates registers, Static Random-Access Memory (SRAM) caches, and software-managed scratchpads directly on the accelerator die. Register files provide the immediate operand and accumulator storage needed by execution pipelines. Modern accelerators provision substantial register capacity in aggregate across the chip, but partition this pool dynamically among concurrent threads or warps. Increasing per-thread register allocation allows individual threads to retain larger matrix tiles, but limits the total number of concurrently scheduled warps, reducing the hardware's ability to hide memory latency. Conversely, underallocating registers forces the compiler to spill values to off-chip memory, converting single-cycle register accesses into multi-cycle stalls. Efficient kernel design balances register allocation to maintain maximum occupancy while keeping hot operands resident.

Hardware-managed L1 and L2 caches provide fast buffering for frequently referenced data using automatic tag checking and hardware eviction policies. However, automatic caches are poorly suited for streaming multi-gigabyte tensor operations: scanning through large activation tensors can evict reusable weight tiles before arithmetic units can reuse them. Furthermore, specialized execution pipelines impose strict layout and data-type constraints that general caches cannot satisfy automatically. As illustrated in example 11.3, hardware matrix engines require specific alignments and memory contracts; violating these requirements causes execution to fall back to unaccelerated paths.

Example 11.3: The Tensor Core contract

Scenario: An engineering team migrates a transformer workload to NVIDIA A100 GPUs expecting automatic Tensor Core acceleration, but profiling reveals Tensor Cores remain idle (NVIDIA Corporation 2020a).

Diagnosis: The workload used shapes and data types that selected less efficient kernels rather than the intended Tensor Core path.

Systems lesson: Specialized paths have operator, precision, layout, and shape requirements. Libraries can often handle nonaligned dimensions through padding or alternative kernels, but reaching high utilization requires verifying which path was actually selected.

To avoid the eviction overhead and unpredictability of automatic caches, AI accelerators incorporate scratchpad memory. Unlike hardware caches, scratchpads are software-managed SRAM structures mapped directly into the core's address space without tag directories or hardware re-

placement logic. The compiler or programmer explicitly controls when tensor tiles are loaded from external DRAM, how long they reside on-chip, and when updated values are written back. In convolutional models, scratchpads retain input feature maps and weight kernels across sliding receptive windows, eliminating redundant DRAM reads (Chen et al. 2017). On NVIDIA GPUs, scratchpad memory is exposed as *shared memory*: a fast, programmer-controlled SRAM bank shared across all threads within a thread block. Custom kernels written in CUDA or Triton manage shared memory explicitly. FlashAttention demonstrates the power of this mechanism: rather than materializing the full $S \times S$ attention score matrix in off-chip HBM, the kernel tiles queries, keys, and values through shared memory and updates softmax statistics online (T. Dao et al. 2022). The speedup stems entirely from eliminating high-latency HBM round trips rather than reducing arithmetic operations; in fact, the algorithm recomputes intermediate attention values during the backward pass to avoid storing them to off-chip memory.

11.5.3.2 Off-chip memory

When tensor working sets exceed on-chip SRAM capacity, operands and parameters reside in device-local DRAM. Rather than sequential hierarchy levels, High Bandwidth Memory (HBM), Graphics DDR (GDDR), and Low-Power DDR (LPDDR) represent alternative memory architectures engineered for distinct cost, bandwidth, and power envelopes. High-performance accelerators use HBM, which stacks DRAM dies vertically using through-silicon vias (TSVs) and interfaces with the accelerator die across a wide silicon interposer (e.g., 1,024 to 8,192 bits). This wide bus delivers multi-terabyte-per-second bandwidth at lower clock frequencies and lower energy per bit than conventional DRAM, but requires complex 2.5D packaging. Where silicon interposers are cost-prohibitive, accelerators adopt GDDR, which uses narrower buses (e.g., 256 to 384 bits) driven at high pin speeds across standard circuit boards. Mobile and embedded accelerators utilize LPDDR, trading raw bandwidth for low operating voltage and reduced standby power.

When parameter and activation footprints exceed device-local DRAM capacity, the system must offload state to host DRAM (typically standard DDR). Accessing host memory requires crossing an I/O interconnect such as PCIe or a coherent fabric like NVLink or Compute Express Link (CXL). Because PCIe bandwidth (tens of gigabytes per second) is one to two orders of magnitude lower than accelerator DRAM bandwidth, unmanaged offloading stalls arithmetic execution. Efficient runtimes mitigate this penalty by overlapping asynchronous Direct Memory Access (DMA) copies with kernel execution, prefetching upcoming layer parameters while current layers compute.

At the base of the hierarchy, nonvolatile flash storage and solid-state drives (SSDs) provide persistent, multi-terabyte capacity for datasets and training checkpoints (Narayanan et al. 2021). With read latencies measured in tens of microseconds and bus bandwidths capped at single-digit gigabytes per second, flash devices cannot participate directly in kernel execution. Checkpointing and data-loading pipelines treat flash storage as an asynchronous streaming source, staging batches through host DRAM into accelerator memory before computation begins.

The physical tiers of the memory hierarchy establish the latency floors and capacity ceilings of the accelerator. How closely an application approaches hardware peak depends on whether software can sustain high utilization across each tier: advertised memory bandwidth represents only a theoretical limit, while achievable throughput is determined by access regularity, tensor layout, and reuse locality.

11.5.4 Memory bandwidth and architectural trade-offs

Advertised memory bandwidth is only a ceiling; achievable bandwidth depends on access pattern, batching, locality, and the host interface that feeds the accelerator. Representative data center accelerators provide memory bandwidth on the order of a few TB/s, paired with tens of GB of high-bandwidth memory. Peak figures assume continuous, stride-1 coalesced access across all memory channels. Real neural network workloads deviate substantially from this ideal: transformer attention, convolution, and fully connected layers realize distinct fractions of peak bandwidth because their memory access strides, tiling dimensions, and operational reuse differ. Fully connected layers approach peak bandwidth only when batch sizes are large enough to amortize the cost of loading weight matrices—connecting directly to the batch-size sensitivity analyzed under the roofline model in section 11.6. When batch sizes are small, execution is throttled by memory latency and underutilized channels, leaving effective throughput well below advertised ceilings.

On-chip memory access typically consumes energy in the single-digit-to-tens of picojoules per access, while external DRAM can be on the order of hundreds of picojoules per access—an order-of-magnitude energy penalty (section 11.5.1). Because driving off-chip physical interfaces dominates the thermal and power budget, accelerators structure their execution pipelines around spatial data reuse. Rather than repeatedly round-tripping tensors through DRAM, architectures adopt specific stationary dataflows. In a weight-stationary design, model parameters remain fixed inside processing element registers while input activations stream through, amortizing parameter loads across multiple tokens or spatial positions. In an input-stationary design, activations are buffered locally while weights stream through, maximizing reuse across kernel filters. In an output-stationary design, partial sums accumulate directly inside local accumulators until final reductions complete, eliminating intermediate round-trips of partially computed feature maps to external DRAM.

Physical constraints at the silicon and packaging boundaries govern how memory bandwidth scales across accelerator classes. HBM achieves multi-terabyte-per-second throughput by stacking DRAM dies vertically using through-silicon vias and interfacing with the processor over a silicon interposer. This packaging provides wide parallel interfaces—reaching on the order of 1 TB/s in mainstream products and a few TB/s in high-end systems—at substantially higher packaging cost and thermal density. TPU-class designs pair high-bandwidth memory with large on-chip buffers and 2D systolic arrays, trading general-purpose memory flexibility for tight spatial reuse on dense matrix multiplications. At the opposite extreme, mobile SoC designs operate within a few-watt power envelope and cannot accommodate the cost or thermal dissipation of interposer-based stacks. These edge processors rely on unified commodity LPDDR architectures, delivering on the order of hundreds of GB/s of shared bandwidth that must be strictly scheduled across CPU, GPU, and neural accelerator cores to prevent memory bus contention and thermal throttling.

Bandwidth limitations extend beyond the accelerator package to the host-device interface. When an input must cross a 64 GB/s PCIe link before using an accelerator with 2 TB/s of HBM bandwidth, the interface has roughly $30\times$ less bandwidth and can dominate small, frequent transfers (NVIDIA Corporation 2020a, 2020c). For low-latency inference queries or fine-grained kernel offloading, transfer serialization across the host bus can easily outlast kernel computation. Mitigating this interface bottleneck requires keeping model parameters and intermediate activation states resident in accelerator memory across execution steps, or bypassing the host link entirely through direct peer-to-peer storage and network data paths.

11.5.5 Host-accelerator communication

In discrete accelerator systems, the host CPU and the accelerator operate as separate computational domains joined by a peripheral bus. The host manages storage I/O, network streams, and runtime orchestration, while the accelerator executes dense mathematical kernels over high-bandwidth device memory. Because CPU memory (system DRAM) and accelerator memory (HBM or GDDR) occupy distinct physical address spaces, computation cannot begin until tensors traverse the host-accelerator interconnect. Host-accelerator data movement therefore establishes an operational boundary: if transfers over the peripheral bus cannot feed the accelerator’s compute cores at line rate, the device idles regardless of its raw FLOP/s capacity.

Figure 11.13 traces the four-phase execution sequence that governs this host-device interaction. Before computation begins, input tensors must be copied from host DRAM across the PCIe bus to accelerator memory (step 1) via DMA.²⁹ The CPU then submits kernel launch commands to the accelerator command queue (step 2), prompting the compute units to process tiles in parallel out of local memory (step 3). Once execution completes, the accelerator stores its output tensors in device memory and copies the results back across PCIe to host DRAM (step 4). In a naive synchronous execution model, every transfer step sits directly on the critical path, stalling either the host or the accelerator until each memory transaction finishes.

The end-to-end latency of a kernel launch decomposes into transfer time, queue dispatch overhead, and arithmetic execution time. When tensor transfers are small or frequent, the fixed overhead of driver submission and bus round trips dominates runtime. When transfers are large, the low bandwidth of PCIe relative to on-package HBM creates a sustained I/O bottleneck. In D·A·M terms, the Machine boundary between host and accelerator forces the Algorithm to maximize arithmetic reuse on the device once tensors arrive.

²⁹ | **DMA (direct memory access):** A dedicated hardware unit that manages the data copy (step 1) without direct CPU management, freeing the CPU to immediately issue computation commands (step 2). This concurrency is critical: without it, the accelerator can idle between compute batches, especially when host-to-device movement is on the critical path.

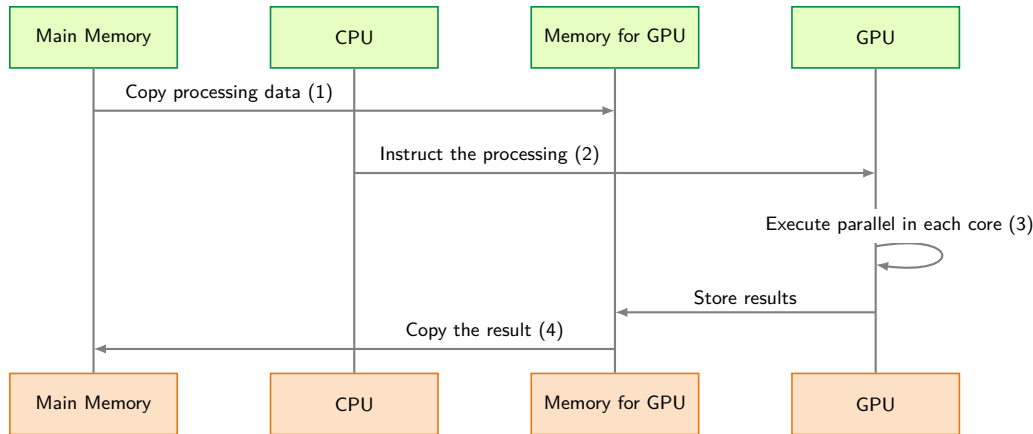


Figure 11.13: Host-Accelerator Data Transfer Sequence: Four-step execution flow between CPU host and GPU device: (1) copying input data over PCIe, (2) issuing kernel instructions, (3) GPU compute, and (4) transferring results back to host memory. Low PCIe bandwidth and host dispatch latencies can easily dominate runtime unless kernel compute time exceeds data transfer overhead.

11.5.5.1 Node-level interconnect topology

Optimizing data movement requires understanding the physical topology of the compute node. A typical AI server is not a flat mesh of connected devices but a hierarchy of bandwidths that tapers as data moves farther from the chip.

Within a multi-accelerator server node, three interconnect tiers define this bandwidth taper:

1. **Device-device interconnect (NVLink/Infinity Fabric):** Modern multi-GPU nodes use specialized high-speed bridges like NVLink³⁰ to connect accelerators directly, bypassing the host CPU. Bandwidth ranges from 600 GB/s to 900 GB/s per GPU (NVIDIA Corporation 2020c; Choquette 2023). This link matters whenever tensors must move between accelerators within one server, including model partitioning, activation exchange, and gradient synchronization during training.³¹ The architectural principle is the boundary: traffic that stays on the accelerator fabric is far cheaper than traffic that falls back through the host.
2. **Host-device interconnect (PCIe):** The link between the CPU and the accelerator. Bandwidth ranges from 32 to 64 GB/s (PCIe Gen4/Gen5). Host-staged training data can bottleneck here, although resident data, direct storage or network I/O, and coherent accelerator links can use other paths. Multi-GPU servers may also provide multiple PCIe links rather than a single shared 64 GB/s path.
3. **Network interface card:** The link from the host to the outside world, connecting to other nodes. Bandwidth ranges from 25 to 50 GB/s (200 Gb/s to 400 Gb/s Ethernet/InfiniBand³²). This interconnect is the first step from single-node hardware reasoning into the next frontier: scale. Traversing the network moves traffic onto the narrowest, highest-latency path in the system.

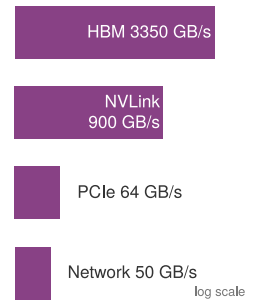
These three levels produce a characteristic bandwidth taper:

$$\begin{aligned} \text{HBM (3350 GB/s)} &\gg \text{NVLink (900 GB/s)} \\ &\gg \text{PCIe (64 GB/s)} \gg \text{Network (50 GB/s)} \end{aligned}$$

System efficiency depends on keeping tensors on the highest-bandwidth links available. Because PCIe and network links offer orders of magnitude less bandwidth and higher latency than on-package memory, placement and scheduling must minimize crossings across these boundaries.

11.5.5.2 Transfer optimization

Two complementary strategies mitigate transfer overheads across the host-accelerator interface: asynchronous data movement and unified memory abstraction.



Bandwidth tapers steeply as data moves farther from the accelerator.

³⁰ | **NVLink (NVIDIA Link):** This direct GPU-to-GPU interconnect exists to keep accelerator-to-accelerator traffic off PCIe when tensors must move inside a server. Its 600–900 GB/s aggregate bidirectional bandwidth is roughly 300–450 GB/s per direction under balanced full-duplex traffic, several times the directional bandwidth of a standard PCIe link, so workloads with frequent cross-device tensor movement can remain in the fast part of the bandwidth taper (NVIDIA Corporation 2020c; Choquette 2023). The training algorithms that determine how much tensor traffic must cross this link are developed later; the hardware fact needed here is the bandwidth gap.

31 | AllReduce: A collective operation from MPI that aggregates values across processes (the “reduce”) and distributes the result back to every process (the “all”). Here, the term identifies why accelerator-to-accelerator bandwidth matters for distributed training workloads. At scale, algorithms, topology choices, and runtime protocols determine how costly this synchronization becomes.

32 | InfiniBand: Its key feature for multi-node scaling is RDMA (Remote Direct Memory Access). With a GPU-aware path such as GPUDirect RDMA and registered GPU memory, a network adapter can transfer data directly between GPUs across nodes without staging it through host memory. This reduces host involvement and protocol overhead, so scaling is more often constrained by physical bandwidth, topology, and collective implementation rather than CPU-managed packet processing.

In an asynchronous pipeline, software allocates pinned (page-locked) host memory and enqueues non-blocking memory copies onto dedicated DMA engines. Because copy engines operate independently of compute units, the next input batch can transfer across PCIe concurrently while the accelerator processes the current batch in local memory. This double buffering conceals transfer latency behind arithmetic runtime whenever computation time exceeds transfer time ($T_{\text{compute}} \geq T_{\text{transfer}}$). When compute duration is too short to amortize the transfer, execution remains bus-bound and compute units inevitably stall.

Unified Memory provides an alternative abstraction by presenting a single virtual address space spanning host DRAM and device HBM. Rather than requiring manual buffer allocation and explicit staging copies, the runtime driver migrates memory pages on demand via hardware page faults when either processor touches an address. While this eliminates boilerplate staging code, on-demand migration introduces severe performance unpredictability. Page faults trigger driver interrupts that stall execution pipelines, and scattered access patterns can cause page thrashing across the PCIe bus. Consequently, high-throughput production workloads rely on explicit, pipelined DMA transfers for deterministic performance, reserving Unified Memory for rapid prototyping or applications where programmer productivity outweighs sustained hardware utilization.

These interconnect and memory bounds dictate how neural network architectures map to physical silicon. Convolution exposes high spatial reuse that amortizes transfer costs through on-chip register and SRAM tiling. Dense self-attention exhibits regular matrix structure, yet quadratic sequence-length scaling creates intense pressure on package memory capacity and bandwidth. Sparse embeddings, mixture-of-experts routing, and autoregressive key-value caches present still other memory access patterns—shifting the binding constraint from arithmetic throughput to memory capacity, bus latency, or interconnect bandwidth.

11.5.6 Model memory pressure

Model architecture determines which memory term binds. Multilayer perceptrons (MLPs), convolutional neural networks (CNNs), and transformers exhibit fundamentally different dataflow profiles: some are bound by weight capacity, others by intermediate activation footprint, device memory bandwidth, or host interconnect throughput. Consequently, each workload demands a distinct accelerator memory hierarchy and execution strategy.

The lighthouse models introduced in table 1.5 ground this analysis: ResNet-50 represents CNN workloads with high spatial reuse and heavy activation footprint, GPT-2 and Llama exemplify transformer memory pressure spanning both parameter capacity and intermediate attention states, DLRM illustrates sparse embedding lookups that stress random-access memory bandwidth rather than dense compute, and MobileNetV2 demonstrates efficiency-optimized architectures where depthwise convolutions trade arithmetic intensity for lower parameter footprint. Their memory characteristics reveal how computational graph structure dictates physical hardware utilization.

11.5.6.1 Multilayer perceptrons

An MLP layer is a dense matrix multiplication followed by optional bias, activation, and normalization. Its operational intensity depends directly on the batch size: at batch size $B = 1$, each parameter loaded from off-chip memory serves only a single multiply-accumulate operation, yielding low arithmetic intensity (often ≤ 1 FLOP/byte) that starves ALUs and leaves execution memory-bandwidth bound. Increasing the batch size reuses each loaded weight matrix across B independent input vectors, scaling arithmetic intensity until the matrix multiplication reaches the accelerator’s compute-bound roofline ceiling.

Dense weight matrices regularly exceed the tens of megabytes available in on-chip SRAM, requiring parameters to reside in device-local DRAM (HBM or GDDR). The host interconnect (such as PCIe) enters the critical path only when streaming dynamic inputs or when parameters and activations exceed device memory capacity and require host offloading.

Because MLP memory access patterns are regular and predictable, accelerators tile matrix operations into blocks that fit within on-chip SRAM and register files. Hardware double-buffering prefetches subsequent weight and activation tiles from DRAM into SRAM concurrently with arithmetic execution on current tiles. Delivered throughput therefore depends on whether the batch size provides sufficient arithmetic intensity and whether matrix dimensions align cleanly with hardware tile boundaries (Chen et al. 2017).

11.5.6.2 Convolutional neural networks

A convolutional layer processes input feature maps using small filter kernels that slide across spatial dimensions. This localized sliding-window structure yields high arithmetic intensity through three distinct forms of data reuse: convolutional reuse (neighboring spatial windows share input activations), filter reuse (the same kernel weights evaluate across every spatial position), and channel reuse (input channels are evaluated across multiple output filters). Unlike MLPs, where weight matrices dominate memory footprint, early CNN layers feature compact parameter tensors but produce massive intermediate activation maps.

Because intermediate activation tensors frequently exceed on-chip SRAM capacity, unoptimized layer-by-layer execution would continuously spill feature maps to device DRAM, bottlenecking the pipeline on memory bandwidth. Accelerators prevent these round-trips through spatial tiling: dividing activations and filters into tiles dimensioned to fit within on-chip SRAM buffers (Chen et al. 2017). Staging overlapping boundary pixels directly in local memory allows adjacent convolutions to consume activations without refetching them across the device DRAM bus.

Specialized CNN accelerators configure their hardware dataflow around these reuse patterns. The Eyeriss architecture introduced a row-stationary dataflow that maps one-dimensional rows of filter weights and activations to processing engine registers, keeping filter rows and accumulating partial sums stationary in local storage to minimize energy-intensive interconnect traffic (Y.-H. Chen et al. 2016). In the broader taxonomy of section 11.8, this design exemplifies a hybrid dataflow that exploits multidimensional convolutional reuse across registers, SRAM buffers, and execution units.

11.5.6.3 Transformer networks

The transformer architecture introduced in section 6.6 replaces local sliding windows with global self-attention³³ and dense feed-forward blocks, creating dual memory bottlenecks: quadratic intermediate attention matrices and massive parameter footprints.

Global token interaction produces distinct memory challenges during training and inference. In standard multi-head attention, computing pairwise token affinity generates an $S \times S$ score matrix for sequence length S . Materializing this intermediate matrix in HBM repeatedly reads and writes $O(S^2)$ data across the memory bus, saturating bandwidth even though matrix multiplication arithmetic units remain underutilized. Tiled attention algorithms, such as FlashAttention (T. Dao et al. 2022), resolve this bottleneck by staging query, key, and value blocks in on-chip SRAM, computing online softmax reductions locally, and streaming outputs directly to HBM without materializing the quadratic intermediate matrix in device DRAM.

In autoregressive inference, memory pressure shifts to the key-value (KV) cache (see section 13.8.3). Generating each subsequent token requires reading the accumulated historical KV states from HBM for a single vector-matrix operation, operating at very low arithmetic intensity and binding generation speed directly to memory bandwidth.

At the scale of large language models (Brown et al. 2020), parameter counts exceed the physical HBM capacity of any single accelerator. Distributing these models across multiple accelerators requires partitioning weights and activations across GPUs. Within a server node, high-bandwidth interconnects like NVLink provide the interconnect throughput necessary to sustain all-reduce and all-gather collectives during tensor and sequence parallelism, where standard PCIe links would throttle execution. System runtimes schedule asynchronous transfers to overlap these cross-accelerator communication phases with local matrix multiplication arithmetic (Narayanan et al. 2021).

11.5.7 Accelerator design implications

Workload-specific accelerator design is dictated by the memory and data-flow requirements of target neural network architectures. Within the D·A·M taxonomy, the algorithmic formulation directly shapes data movement patterns across the physical memory hierarchy, making a single rigid hardware architecture inefficient across different model classes. Table 11.11 compares the resulting tensor dimensions, reuse profiles, access regularities, and primary hardware bottlenecks across foundational model families.

³³ | **Attention mechanism:** Bahdanau, Cho, and Bengio introduced attention for sequence-to-sequence models; Transformer self-attention later allowed each token to attend to every token in the sequence (Bahdanau et al. 2015; Vaswani et al. 2017); materializing full attention scores for a sequence of length S requires an $S \times S$ matrix, producing quadratic intermediate storage. FlashAttention tiles the computation to avoid materializing that full matrix in HBM (T. Dao et al. 2022). The inference KV cache is separate: its capacity grows linearly with sequence length per layer, although each decode step reads an increasing cache (see section 13.8.3).

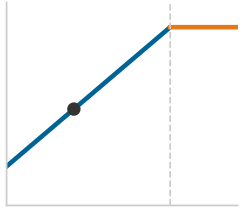
Table 11.11: ML Model Memory Access: Different machine learning models exhibit distinct memory access patterns and bottlenecks due to variations in weight size, activation reuse, and data movement. Standard dense transformers demand high bandwidth and capacity because large weights, KV caches, and attention traffic dominate memory pressure; sparse MoE or pruned variants add further routing and sparsity considerations. CNNs benefit from spatial locality and high activation reuse, reducing memory pressure.

Model type	Weight size	Activation reuse	Memory access pattern	Primary Bottleneck
MLP (Dense)	Large, dense	Low	Regular, sequential (streamed)	Bandwidth (off-chip)
CNN	Small, reused	High	Spatial locality	Feature map movement
Transformer	Large, usually dense; sparse in MoE/pruned variants	Low-to-medium	Mostly regular GEMM plus KV-cache/attention traffic	Memory capacity + bandwidth

Each model architecture exercises the memory subsystem differently. In small-batch multi-layer perceptron (MLP) inference, matrix-vector multiplications (GEMV) dominate execution; each weight element is transferred from off-chip memory to interact with a single activation vector, producing an arithmetic intensity near one FLOP per byte and saturating off-chip bandwidth while matrix units sit idle. Increasing the batch size transforms the kernel into matrix-matrix multiplication (GEMM), amortizing weight loads across multiple input vectors. In CNNs, compact filter kernels slide over spatial feature maps, enabling high arithmetic reuse; accelerators exploit this spatial locality by staging feature tiles in on-chip static RAM (SRAM) or scratchpad memories to prevent redundant off-chip traffic. Transformers present distinct bottlenecks across execution phases: distributed training is constrained by activation retention for backpropagation and inter-accelerator communication, whereas autoregressive decoding is bound by memory bandwidth as each generated token streams the entire parameter set and an expanding key-value (KV) cache from device memory.

Hardware architects address these divergent demands with multi-tier memory hierarchies. On-chip SRAM (caches and scratchpad memories) provides tens of terabytes per second of bandwidth at low latency, capturing high-reuse activations and tiled matrix blocks. HBM provides the capacity and throughput required to stage massive weight tensors and dynamic sequence states. When model scale exceeds the physical memory capacity of an individual accelerator, high-speed interconnects such as NVLink supply peer-to-peer inter-accelerator bandwidth for distributed tensor and pipeline parallelism, avoiding the narrower host PCIe bus.

Data movement also dominates the energy budget of hardware accelerators. Transferring operands from off-chip dynamic RAM (DRAM) consumes roughly two orders of magnitude more energy than executing an arithmetic operation on local execution units. Consequently, hardware efficiency depends on maximizing computation per byte fetched from external storage. Determining whether a given kernel execution is constrained by compute throughput or memory bandwidth requires an analytical boundary. The roofline model establishes this relationship by comparing operational intensity against hardware performance limits, ensuring optimizations target the true physical bottleneck.



Low arithmetic intensity pins the workload in the memory-bound regime.

³⁴ | **Roofline model:** Introduced by Williams et al. (2009) at UC Berkeley, building on earlier I/O complexity work from the 1980s. Their specific contribution was making the compute vs. bandwidth trade-off *visual* and *actionable*: the characteristic roofline plot immediately reveals whether a kernel is compute bound (hitting the flat ceiling) or memory bound (hitting the sloped bandwidth line) and quantifies the gap to hardware limits. A kernel operating at only 50 percent of its ceiling has a clear $2\times$ utilization gap to close, making this the standard diagnostic tool for accelerator optimization.

11.6 Roofline Model

Peak FLOP/s indicates what execution units can deliver when operands are already on chip, but sustained accelerator throughput is ultimately bounded by the rate at which the memory hierarchy delivers data. The Roofline Model³⁴ (Williams et al. 2009) formalizes this relationship by plotting attainable performance against arithmetic intensity, establishing whether an operation is compute bound or memory bound. Rather than evaluating compute capacity in isolation, the model intersects hardware arithmetic throughput with memory bandwidth, revealing which physical ceiling binds execution before an engineer commits optimization effort.

Performance is bounded by two ceilings, as equation 11.2 formalizes. Here, attainable performance R_{attain} and peak compute R_{peak} are in FLOP/s (often reported as TFLOP/s), peak bandwidth BW is in bytes/s (often TB/s), and arithmetic intensity I is in FLOP/byte:

$$R_{\text{attain}} = \min(R_{\text{peak}}, \text{BW} \times I) \quad (11.2)$$

Definition 11.5: Arithmetic intensity

Arithmetic intensity is the ratio of operations to bytes transferred across the memory interface being modeled (FLOP/byte). Together with that level's bandwidth and the relevant compute peak, it determines which idealized roofline ceiling is lower.

1. **Significance:** The threshold between the two ceilings is the ridge point $R_{\text{peak}}/\text{BW}$. For an A100 (312 TFLOP/s FP16/BF16, 2.04 TB/s), it is roughly 153 FLOP/byte. Under the stated traffic models, a large well-tiled 1024×1024 multiply reaches about 341.3 FLOP/byte, while standalone ReLU is about 0.125 FLOP/byte. They therefore meet different ceilings on the same hardware.
2. **Distinction:** Unlike total FLOPs (a count of operations), arithmetic intensity is a ratio that characterizes the shape of a workload's hardware demand. Two kernels with identical FLOPs but different memory access patterns have different arithmetic intensities and will be bottlenecked by different hardware resources.
3. **Common pitfall:** A frequent misconception is that arithmetic intensity is a fixed property of an operation. In practice, it depends on implementation details: a naive matrix multiplication that reloads operands from DRAM for each output element has low arithmetic intensity; a blocked (tiled) implementation that reuses data from fast SRAM achieves high arithmetic intensity—the same mathematical operation, orders of magnitude apart in hardware efficiency.

With O as the operation count and D_{vol} as data volume in bytes, equation 11.3 makes the definition operational.

$$I = \frac{O}{D_{\text{vol}}} \quad (11.3)$$

The roofline visualization shows performance (TFLOP/s) on the vertical axis and arithmetic intensity (FLOP/byte) on the horizontal axis. At low arithmetic intensity, performance increases linearly along the memory bandwidth slope ($\text{BW} \times I$), where execution is memory bound. Above the ridge point, performance saturates at peak compute (R_{peak}), where execution is compute bound. Section A.5.1 maps each regime to the optimizations that pay off and the ones that waste effort, so that classifying a workload as memory-bound or compute-bound tells the engineer directly whether a faster accelerator or more bandwidth is the binding investment.

11.6.1 Hardware ridge points

The ridge point, the hardware balance I_{ridge} established in section 11.5.2, is the arithmetic-intensity threshold at which an accelerator transitions from memory-bound to compute-bound execution. Table 11.12 quantifies this boundary across three generations of NVIDIA accelerators:

Table 11.12: Hardware Ridge Points: Vendor-published FP16 peak throughput and memory bandwidth for three NVIDIA generations, and the ridge point each pair implies. The ridge rises from V100 to H100 because compute throughput grows faster than bandwidth; these values are hardware-specific rather than universal generation averages.

NVIDIA Accelerator	Peak FP16	Bandwidth	Ridge Point
V100 (2017)	125 TFLOP/s	0.9 TB/s	138.9 FLOP/byte
A100 (2020)	312 TFLOP/s	2.04 TB/s	153 FLOP/byte
H100 (2022)	989 TFLOP/s	3.35 TB/s	295.2 FLOP/byte

Because arithmetic throughput has grown far faster than memory bandwidth across generations, the hardware ridge point shifts progressively rightward. A workload must therefore exhibit substantially greater data reuse to saturate newer hardware; an algorithm that was compute-bound on an older architecture can easily become memory-bound on a newer one.

Napkin Math 11.3: The utilization gap

Problem: Why is it harder to get 100 percent utilization on an H100 than a V100?

Metric: The ridge point $I_{\text{ridge}} = R_{\text{peak}}/\text{BW}$ (FLOP/byte) from section 11.5.2: how many math operations the hardware must perform for every byte of data loaded to keep the compute units busy.

Evolution:

- **V100 (2017):** 125 TFLOP/s / 0.9 TB/s $\approx$ 138.9 FLOP/byte.
- **A100 (2020):** 312 TFLOP/s / 2.04 TB/s $\approx$ 153 FLOP/byte.
- **H100 (2022):** 989 TFLOP/s / 3.35 TB/s $\approx$ 295.2 FLOP/byte.

Result: The intensity needed to reach the tensor peak has increased. Under this simple HBM roofline, a kernel with $I = 200$ FLOP/byte lies above the A100 ridge but below the H100 ridge. If its intensity and implementation remain unchanged, its idealized gain is then closer to the 1.6 $\times$ bandwidth ratio than the 3.2 $\times$ peak-compute ratio.

Systems insight: A standalone ReLU under a one-read, one-write FP32 traffic model performs about 1 operation per 8 bytes, far below the H100 tensor ridge. A well-tiled 1024×1024 dense multiply reaches about 341.3 FLOP/byte under its idealized traffic model. Fusion is valuable for low-intensity chains because it can remove intermediate round-trips; it is not automatically the best optimization for every kernel.

Operations such as depthwise convolution, embedding lookup, LayerNorm, and softmax serve as low-intensity reference points because their minimal data reuse causes execution time to be dominated by memory transfers. Table 11.13 maps common neural network operations to the Roofline Model.

Table 11.13: Operations on the Roofline: Neural network layers span a wide range of arithmetic intensities. Large, well-tiled convolutions and batched GEMMs can be compute-bound, while MobileNet depthwise layers, attention softmax, normalization, and DLRM embeddings are often memory-bound. To see how these intensity values translate into real performance predictions, a transformer layer provides a complete arithmetic-intensity calculation across its major sub-operations.

Operation	Arithmetic Intensity	Classification	Lighthouse example
Conv2D (Dense)	50–200 FLOP/byte	Straddles ridge; high-reuse cases compute-bound	ResNet-50
Dense MatMul (large batch, well-tiled)	64–256+ FLOP/byte	Often compute-bound at large batch	GPT-2 (batched projections)
Depthwise conv	10–20 FLOP/byte	Memory-bound	MobileNet
Attention Softmax	Convention-dependent	Memory-bound	GPT-2 (Generation)
LayerNorm	1–2 FLOP/byte	Memory-bound	GPT-2/Llama
Embedding lookup	< 1 FLOP/byte	Memory-bound	DLRM

Napkin Math 11.4: Transformer layer analysis

For a transformer with hidden_dim = 768, batch = 32, seq = 512:

Attention QKV projection:

- FLOPs: $2 \times 3 \times 32 \times 512 \times 768 \times 768 = 58$ GFLOP
- Bytes: (input + weights + output) = $(32 \times 512 \times 768 + 3 \times 768 \times 768 + 32 \times 512 \times 768 \times 3) \times 2 \approx 104.2$ MB
- AI = 58 GFLOP / 104.2 MB = 556.4 FLOP/byte, which is compute bound on A100 (above 153 FLOP/byte threshold)

Softmax:

- FLOPs: $32 \times 12 \times 512 \times 512 \times 3 \approx 302$ MFLOP (exp, sum, div)
- Bytes: $32 \times 12 \times 512 \times 512 \times 2 \times 2 = 402.7$ MB
- AI = 302 MFLOP / 402.7 MB = 0.75 FLOP/byte under this simple operation count, which is memory-bound. Counts that weight exponentials and division as multiple operations produce a larger numerical intensity without changing the bottleneck classification.

Systems insight: This analysis explains why FlashAttention focuses on reducing memory traffic in attention rather than reducing FLOPs.

The roofline classification dictates which physical subsystem limits performance and which optimizations can relieve it. For memory-bound operations, execution time is dominated by data transfers across the off-chip memory bus; ALU pipelines spend idle cycles waiting for operands. Optimization must therefore focus on conserving memory bandwidth: fusing adjacent operators keeps intermediate activations in on-chip registers or shared memory (SRAM), lower-precision numeric formats reduce the physical byte count of memory transfers, and algorithmic refactorings like FlashAttention restructure access patterns to compute within SRAM tiles. For compute-bound operations, off-chip bandwidth is no longer the bottleneck because high data reuse keeps execution units supplied. Performance here depends on saturating arithmetic execution pipelines: structuring matrix tiles to sustain Tensor Core throughput, hiding instruction latencies through software pipelining, and scaling batch sizes to distribute work across all available streaming multiprocessors.

11.6.2 Calculating memory bandwidth bounds

The memory-bound region of the Roofline Model is governed by the accelerator's off-chip memory bandwidth. To sustain an operational throughput R_{ops} (FLOP/s) for a workload with arithmetic intensity I (FLOP/byte), the memory subsystem must transfer data at the rate defined by equation 11.4:

$$BW_{\text{req}} = \frac{R_{\text{ops}}}{I} \text{ bytes/s} \quad (11.4)$$

When this required bandwidth exceeds the hardware's peak memory bandwidth BW , memory transfers cap attainable throughput according to equation 11.5:

$$R_{\text{attain}} = BW \times I \quad (11.5)$$

Evaluating an operation against the hardware's ridge point determines whether execution is bound by memory bandwidth or arithmetic throughput. When data reuse is extensive, arithmetic intensity clears the ridge point, placing the kernel in the compute-bound regime. A 2D convolution demonstrates this behavior.

Napkin Math 11.5: Convolutional layer analysis

Consider a Conv2D layer with input shape (batch = 32, channels = 128, height = 56, width = 56), output channels = 256, and kernel size 3×3 on an A100 GPU:

Computational requirements:

- Output size: $32 \times 256 \times 56 \times 56 = 25.7\text{M}$ elements
- FLOPs per output: $128 \times 3 \times 3 \times 2 = 2,304$ (multiply-add)
- Total FLOPs: $25.7\text{M} \times 2,304 = 59.2$ GFLOP

Memory traffic analysis:

- Input: $32 \times 128 \times 56 \times 56 \times 2 = 25.7$ MB (FP16)
- Weights: $256 \times 128 \times 3 \times 3 \times 2 \approx 0.6$ MB (FP16)
- Output: $32 \times 256 \times 56 \times 56 \times 2 = 51.4$ MB (FP16)
- Total: 77.7 MB

Arithmetic intensity: $I = 59.2$ GFLOP / 77.7 MB = 762.2 FLOP/byte

Systems insight: This is well above A100's ridge point of 153 FLOP/byte, so the simple Roofline Model places the operation in the compute-limited region. Peak throughput of ~312 TFLOP/s (FP16 with Tensor Cores) is therefore the ceiling, but realized performance still depends on tiling, occupancy, instruction mix, and library efficiency.

The convolutional layer's high arithmetic intensity arises from spatial weight reuse: the same 3×3 kernel is applied across every spatial location in the feature map, amortizing the cost of loading weights from off-chip memory across millions of multiply-accumulate operations. Because intermediate values and kernel weights remain in fast on-chip registers and shared memory, the off-chip memory bus remains idle while arithmetic pipelines stay saturated.

Not all neural network layers exhibit such extensive data reuse. Fully connected (dense) layers—such as classification heads, feed-forward networks, and linear projections in transformer attention blocks—lack spatial dimensions. When evaluated at small batch sizes, their arithmetic intensity drops substantially, shifting execution into the memory-bound regime:

Napkin Math 11.6: Dense layer analysis

Consider a fully connected layer: input (batch = 32, features = 2048) → output (batch = 32, features = 2048) on the same A100:

Computational requirements:

- Matrix multiply: $(32 \times 2048) \times (2048 \times 2048)$
- Total FLOPs: $2 \times 32 \times 2048 \times 2048 = 268.4$ MFLOP

Memory traffic analysis:

- Input: $32 \times 2048 \times 2 = 131.1$ KB (FP16)
- Weights: $2048 \times 2048 \times 2 = 8.4$ MB (FP16)
- Output: $32 \times 2048 \times 2 = 131.1$ KB (FP16)
- Total: 8.7 MB

Arithmetic intensity: $I = 268.4 \text{ MFLOP} / 8.7 \text{ MB} = 31 \text{ FLOP/byte}$

This intensity sits below A100's ridge point of 153 FLOP/byte, making this operation memory-bound. Attainable performance: $R_{\text{attain}} = 2,039 \text{ GB/s} \times 31 \text{ FLOP/byte} = 63.3 \text{ TFLOP/s}$

Systems insight: Delivered throughput reaches only 20.3 percent of peak compute capability, demonstrating the memory wall effect for small batch sizes.

The dense layer's lower arithmetic intensity stems from limited parameter reuse. While a convolutional kernel is reused across every spatial location in an activation map, a dense weight matrix is reused only across the batch dimension. At a batch size of 32, each weight parameter transferred from HBM is multiplied only 32 times before being evicted. Because the off-chip memory bus cannot transfer operands fast enough to keep the tensor cores fed, arithmetic units spend over 80 percent of their cycles stalled, capping throughput at 20.3 percent of peak compute capacity. This structural constraint explains why transformer inference at low batch sizes is dominated by memory bandwidth rather than arithmetic capability.

While small-batch matrix multiplications suffer from restricted parameter reuse, elementwise and reduction operators represent an even more acute memory bottleneck. Operations such as layer normalization, activation functions, and residual additions perform only a few arithmetic operations per element loaded from memory, driving arithmetic intensity to its physical floor:

Napkin Math 11.7: LayerNorm analysis

LayerNorm with input shape (batch = 32, seq = 512, hidden = 768):

Computational requirements:

- Elements: $32 \times 512 \times 768 = 12.6\text{M}$
- Approximate operations per element: mean reduction (1 ADD), variance (2 ADD, 1 MUL), normalization and affine transform (2 ADD, 2 MUL) ≈ 8 ; reciprocal-square-root overhead is amortized across each hidden vector
- Total FLOPs: $12.6\text{M} \times 8 = 100.7\text{ MFLOP}$

Memory traffic:

- Input: $12.6\text{M} \times 2 = 25.2\text{ MB}$
- Parameters (scale, bias): $768 \times 2 \times 2 = 3.1\text{ KB}$ (negligible)
- Output: $12.6\text{M} \times 2 = 25.2\text{ MB}$
- Total: 50.3 MB

Arithmetic intensity: $I = 100.7\text{ MFLOP} / 50.3\text{ MB} = 2.0\text{ FLOP/byte}$

This arithmetic intensity sits severely below the A100 ridge point (77× below). Performance is limited to: $R_{\text{attain}} = 2039\text{ GB/s} \times 2.0\text{ FLOP/byte} = 4.1\text{ TFLOP/s}$

Systems insight: The simple HBM roofline bounds this standalone LayerNorm at about 1 percent of A100's tensor peak. Actual latency also depends on fusion, reduction strategy, launch overhead, and which memory level supplies the data.

Standalone normalization performs minimal arithmetic relative to the memory traffic it generates, as the LayerNorm analysis demonstrates. In an unfused implementation, every activation element is transferred from off-chip HBM across the global bus, reduced, and written back to memory, achieving an arithmetic intensity of approximately 2 FLOP/byte and leaving compute units idling 99 percent of the time. In the D·A·M framework, when the machine constraint is an unyielding memory bandwidth wall, the algorithmic response is operator fusion: collapsing adjacent operations—such as linear projections, bias additions, normalization, and activations—into a single kernel. Fusion keeps intermediate activations resident in fast on-chip SRAM or registers, eliminating round-trips across the memory bus and transforming bandwidth-starved workloads into compute-efficient execution.

11.6.3 Optimization by intensity regime

The roofline analysis directly informs optimization priorities, summarized in table 11.14.

Table 11.14: Optimization by Arithmetic Intensity Regime: Roofline position determines whether an optimization should chase compute utilization, memory-traffic reduction, or complete elimination of memory round-trips. The lower the arithmetic intensity, the more valuable fusion and data-movement avoidance become.

Intensity regime	Typical operations	Optimization priority	Common techniques	Expected impact
High AI (> 200 FLOP/byte)	Large convolutions	Maximize compute utilization.	Tensor Cores, thread-block tuning, and high occupancy.	Raises sustained compute utilization when the kernel already clears the ridge.
Medium AI ($20\text{--}200$ FLOP/byte)	Medium-sized dense layers	Balance compute and memory optimization.	Larger batches, register tiling, and fusion with adjacent operations.	Can move the binding bottleneck between memory and compute.
Low AI (< 20 FLOP/byte)	Small dense layers and element-wise operations	Reduce memory traffic.	Aggressive operator fusion, reduced precision (FP16 $\rightarrow$ INT8), and algorithmic changes.	Reduces external-memory traffic when fusion or layout changes are legal.
Very low AI (< 2 FLOP/byte)	Normalization layers and activation functions	Eliminate memory round-trips.	Fuse with adjacent operations and use in-place computation where legal.	Removes intermediate round-trips when adjacent operations can be fused.

For low-AI operations, operator fusion is often the decisive optimization: LayerNorm combined with the Gaussian Error Linear Unit (GELU), for example, can become a single fused kernel. One of the most accessible levers for moving an operation up and right on the roofline is batching.

Napkin Math 11.8: Batch size and arithmetic intensity

Increasing batch size improves arithmetic intensity for matrix operations by amortizing weight loading. Equation 11.6 formalizes this relationship for an idealized FP16/BF16 dense layer $(B \times M) \times (M \times N)$, counting one read each of the input and weights and one write of the output, where B is batch size and M and N are the input and output dimensions:

$$I = \frac{2BMN}{2BM + 2MN + 2BN} \approx \frac{2BMN}{2MN} = B \quad (\text{when } 2MN \gg 2B(M + N)) \quad (11.6)$$

Example: Dense layer with $M=N=2048$ (FP16)

- Batch = 1: AI ≈ 1 FLOP/byte (memory bound)
- Batch = 32: AI ≈ 31 FLOP/byte (memory bound)
- Batch = 256: AI ≈ 204.8 FLOP/byte (compute bound on A100)

Systems insight: This explains why batching can produce large throughput improvements in production inference systems, as MLPerf Inference, the standardized benchmark suite covered in chapter 12, demonstrates by separating bulk-throughput runs from latency-constrained serving runs (Reddi et al. 2019).

The batch size analysis reveals why inference serving systems are designed around batching: it changes the arithmetic intensity regime of memory-bound workloads. However, batching introduces latency trade-offs, since requests must wait in a queue until a batch forms. This tension between throughput (favoring large batches) and latency (favoring small batches) is a central challenge in ML serving systems, explored in depth in section 13.7.3.

When service latency limits batching, batch-1 or small-batch LLM decode often retains low weight reuse and low arithmetic intensity. The next worked model quantifies that particular regime rather than treating all LLM inference as identical.

Napkin Math 11.9: The throughput ceiling

Problem: A batch-1 GPT-2 calculation makes this bandwidth ceiling concrete: what is the maximum possible utilization of an NVIDIA A100 when running GPT-2 inference (batch size 1)?

The hardware constraints (the denominators)

- **Peak compute:** 312 TFLOP/s (FP16 Tensor Core).
- **Peak bandwidth:** 2.04 TB/s (HBM2e).
- **Ridge point ($R_{\text{peak}}/\text{BW}$):** 312 TFLOP/s / 2.04 TB/s = 153 FLOP/byte (for FP16 Tensor Core).

Interpretation: Saturating this chip at FP16 precision requires 153 FLOP/byte operations for every byte loaded. The ridge point varies by precision: FP32 operations (19.5 TFLOP/s peak) have a ridge point of only ~ 9.6 FLOP/byte.

The workload characteristics (the numerator)

- **Model:** GPT-2 XL (1.5 billion parameters).
- **Operation:** Autoregressive generation (1 token at a time).
- **Data movement:** Must load all weights (3 GB @ FP16) for every token.
- **Compute:** Vector-Matrix multiplication. $2 \times \text{Params} \approx 3$ GFLOP.
- **Arithmetic intensity:** 3 GFLOP / 3 GB = 1 FLOP/byte

The prediction (iron law)

Since Actual Intensity (1) $\ll$ Ridge Point (153 FLOP/byte), the system is bandwidth bound.

- **Maximum throughput:** $1 \text{ FLOP/byte} \times 2.04 \text{ TB/s} = 2.04 \text{ TFLOP/s}$.
- **Utilization ceiling:** $2.04 \text{ TFLOP/s (Actual)} / 312 \text{ TFLOP/s (Peak)} \approx 0.7\%$

Systems insight: Without batching, a \$15,000 GPU runs at less than 1 percent compute utilization in this weight-streaming decode model. Batching and weight quantization address that gap; key-value caching instead avoids recomputing prior attention states.

Through this derivation, the Roofline Model provides a diagnostic framework for identifying whether operations are compute bound or memory bound. Knowing that a workload is memory bound at 0.7 percent utilization is only the first step; the next challenge is translating this diagnosis into efficient execution plans that exploit accelerator architectures.

11.7 Hardware Mapping

Consider a 3×3 convolution running on an accelerator tile. The mathematical operation is fixed, but the execution plan is not. One schedule can keep a filter in local registers while many output pixels stream past it; another can advance pixel by pixel and reload the same filter values repeatedly from a slower memory tier. Both schedules compute the same tensor. Only one turns the reuse in the convolution into real bandwidth savings.

Definition 11.6: Mapping in AI acceleration

Mapping in AI acceleration is the accelerator-compiler process of binding the logical computation graph to the physical hardware topology by deciding which operations execute on which processing elements, which data resides in which memory tier, and in what temporal order.

1. **Significance:** Within the D-A-M taxonomy, mapping is a machine-axis decision that helps determine how closely an operation approaches the roofline bound $\min(R_{\text{peak}}, \text{BW} \times I)$. A poor tiling choice that forces unnecessary DRAM accesses can reduce effective arithmetic intensity enough to move a nominally compute-bound operation into the bandwidth-bound regime.
2. **Distinction:** Traditional compilation often emphasizes instruction selection, register allocation, and scheduling for general-purpose processors. Accelerator mapping gives equal prominence to spatial placement and explicit data movement across a dataflow architecture. The reason is physical: in the 45 nm comparison introduced earlier, off-chip DRAM access consumed roughly $200 \times$ the energy of a local integer multiply-accumulate, although the ratio depends on the technology and operation.
3. **Common pitfall:** A frequent misconception is that mapping is automatically handled by frameworks. For general GPU workloads, compilers such as Accelerated Linear Algebra (XLA) can find strong mappings for common kernels; for specialized accelerators (systolic arrays, custom ASICs), compiler-generated mappings may still lag hand-tuned schedules because the compiler's search space is limited by the time budget at compilation.

The convolution example exposes the three decisions that recur throughout accelerator compilation. Placement assigns the multiply-accumulate work to processing elements so parallelism does not turn into idle time or interconnect congestion. Allocation keeps weights, activations, and partial sums in the memory tier where their next use will occur, rather than letting reuse spill back to DRAM. Scheduling orders loops and kernels so the chosen placement and allocation remain valid over time. A poor choice in any one dimension can collapse a high-arithmetic-intensity operation back into a bandwidth-bound execution. In practice, these choices are too coupled for developers to manage by hand at model scale, which is why systems such as XLA, TVM, and TensorRT lower high-level models and search or select execution plans within their compile-time and hardware budgets. Section 11.9 examines that compiler support in detail.

11.7.1 Placement and allocation

Translating a model’s computational graph into efficient hardware execution requires solving two tightly coupled problems. Computation placement determines which operations run on which processing elements, balancing parallelism against communication costs. Memory allocation determines where data resides within the memory hierarchy, trading capacity against access latency. These two decisions interact: placing operations on distant processing elements increases the memory bandwidth required to shuttle data between them, while allocating data to fast but small on-chip memory limits which operations can execute concurrently. Getting either wrong leaves thousands of processing elements idle or starved for data.

11.7.1.1 Computation placement

Computation placement is the process of assigning operations to an accelerator’s processing elements (PEs) to expose parallelism, limit idle time, and reduce unnecessary data movement. Modern accelerators contain many such resources: the NVIDIA H100 has more than 16,000 CUDA cores and more than 500 Tensor Cores (Choquette 2023), TPUs organize thousands of multiply-accumulate units into systolic arrays (Jouppi et al. 2017), and wafer-scale processors such as Cerebras’ CS-2 integrate more than 850,000 cores (Systems 2021). At these scales, placement inefficiencies become measurable because idle cores and redundant transfers waste both time and energy.

The difficulty of placement depends on workload regularity. Convolutional networks and dense transformer operations possess regular access patterns that tile predictably, though transformer architectures mix operations with distinct tensor shapes and reuse profiles. Graph neural networks (GNNs) present greater difficulty: sparse, input-dependent neighborhood aggregations introduce load imbalance and non-uniform communication across processing elements. Table 11.15 summarizes these core placement challenges and their operational impact.

Table 11.15: Computation Placement Challenges: Effective neural network deployment requires strategic allocation of computations to processing elements, balancing workload distribution, data movement costs, and hardware constraints to maximize execution efficiency. These challenges guide the design of mapping strategies that optimize resource utilization and minimize communication overhead.

Challenge	Impact on execution	Key Considerations for Placement
Workload imbalance	Some processing elements finish early while others remain overloaded, leading to idle compute resources.	Distribute operations evenly to prevent stalls and ensure full utilization of PEs.
Irregular computation patterns	Sparse graphs, routing, and variable shapes can create nonuniform work that complicates static placement.	Use adaptive placement when workload characteristics vary at run time.
Excessive data movement	Frequent memory transfers introduce latency and increase power consumption.	Keep frequently used data close to the compute units and minimize off-chip memory accesses.
Limited interconnect bandwidth	Poorly placed operations can create congestion, slowing data movement between PEs.	Optimize spatial and temporal placement to reduce communication overhead.
Model-specific execution needs	CNNs, transformers, and GNNs require different execution patterns, making a single placement strategy ineffective.	Tailor placement strategies to match the computational structure of each model type.

Static spatial mappings suffice for regular dense operations whose shapes and loop bounds are known ahead of time. Workloads with dynamic shapes, sparsity, or mixture-of-experts routing instead require runtime scheduling to balance execution across processing elements. Yet even an optimal computation placement stalls if the required operands are not staged in time: compute units can execute only as fast as operands arrive from the memory hierarchy.

11.7.1.2 Memory allocation

While computation placement determines where operations execute, memory allocation defines where data resides and how it flows through the hierarchy. The goal is to keep reused data near the processing elements without exceeding faster-tier capacity. GPUs expose global memory, shared memory, caches, and registers that kernels coordinate through tiling (NVIDIA Corporation 2020a). TPUs use on-chip buffers to stage activations and weights for systolic-array execution (figure 11.8)

(Jouppi et al. 2017), while wafer-scale processors partition memory and computation to control interconnect traffic (Systems 2021). General-purpose processors and accelerators both use memory hierarchies, but many accelerators expose more placement decisions to compilers and kernels. When allocation fails to exploit locality, processing elements stall waiting for operands, driving down utilization and dissipating energy across the memory bus.

The severity of these penalties varies by workload. Convolutional networks rely on structured, localized access patterns and benefit from well-defined memory layouts that facilitate predictable reuse (Y.-H. Chen et al. 2016). Transformers require access to large parameter sets and intermediate activations, making them sensitive to capacity and bandwidth. Graph neural networks add irregular sparse structures that complicate allocation and prefetching.

Capacity is only the first allocation test. A tensor can fit in accelerator memory yet still dominate execution if each kernel rereads it from a distant tier; conversely, tiling can keep a larger working set efficient by preserving reuse locally. The allocation diagnostic is therefore traffic, not capacity alone: compilers must track how frequently values cross each memory boundary and keep high-reuse tensors in the fastest feasible tier. Systems combine static memory plans for known shapes with dynamic buffer management when shapes vary, while device capacity limits which models fit without partitioning or offload. Table 11.16 summarizes these allocation challenges across the hierarchy.

Table 11.16: Memory Allocation Challenges: Efficient memory management in AI accelerators balances data access speed with hardware constraints, mitigating performance bottlenecks caused by latency, bandwidth limitations, and irregular data patterns. Complex models such as transformers and graph networks impose variable and demanding memory requirements that amplify these challenges.

Challenge	Impact on Execution	Key Considerations for Allocation
High memory latency	Slow data access delays execution and reduces throughput.	Prioritize placing frequently accessed data in faster memory locations.
Limited on-chip storage	Small local memory constrains the amount of data available near compute units.	Allocate storage efficiently to maximize data availability without exceeding hardware limits.
High off-chip bandwidth demand	Frequent access to external memory increases delays and power consumption.	Reduce unnecessary memory transfers by carefully managing when and how data is moved.
Irregular memory access patterns	Some models require accessing data unpredictably, leading to inefficient memory usage.	Organize memory layout to align with access patterns and minimize unnecessary data movement.
Model-specific memory needs	Different models require different allocation strategies to optimize performance.	Tailor allocation decisions based on the structure and execution characteristics of the workload.

11.7.2 Combinatorial complexity

The small convolution example illustrates why hardware mapping becomes a combinatorial search problem. Keeping a filter resident in local storage improves reuse only if the assigned processing elements have sufficient storage and if the loop schedule revisits that filter before eviction. Similarly, parallelizing across more processing elements improves throughput only until synchronization overhead and interconnect contention erode the benefit. Because changing one decision alters the feasible and efficient choices for the others, compilers must evaluate placement, allocation, and scheduling jointly rather than independently. Table 11.17 summarizes these recurring trade-offs across the execution stack.

Table 11.17: Placement-Allocation-Scheduling Trade-Offs: AI accelerator performance depends on mapping computations to hardware, allocating data to memory tiers, and scheduling execution over time. Careful consideration of these interdependent factors is essential for maximizing throughput and minimizing energy consumption.

Dimension	Placement considerations	Allocation and Scheduling Considerations
Computational granularity	Fine-grained placement enables greater parallelism but increases synchronization overhead.	Coarse-grained scheduling reduces synchronization overhead but may limit flexibility.

Dimension	Placement considerations	Allocation and Scheduling Considerations
Spatial vs. Temporal Mapping	Spatial placement enhances parallel execution but can lead to resource contention and memory congestion.	Temporal scheduling balances resource sharing but may reduce overall throughput.
Memory and Data Locality	Placing data closer to compute units minimizes latency but may reduce overall memory availability.	Allocating data across multiple memory levels increases capacity but introduces higher access costs.
Communication and synchronization	Co-locating compute units reduces communication latency but may introduce contention.	Scheduling synchronization mechanisms mitigates stalls but can introduce additional overhead.
Dataflow and Execution Ordering	Static placement simplifies execution but limits adaptability to workload variations.	Dynamic scheduling improves adaptability but adds scheduling complexity.

Evaluating these trade-offs requires navigating an unconstrained search space governed by three structural decisions: loop execution order, spatial parallelization across processing elements, and memory allocation across the hierarchy. To understand why exhaustive search is impractical, consider the combinatorial growth of each dimension independently.

11.7.2.1 Ordering computation and execution

Machine learning workloads are typically expressed as nested loops that iterate over tensor dimensions. For instance, a matrix multiplication kernel loops over batch size (B), input features (C_{in}), and output features (C_{out}). The loop execution order dictates which tensor elements remain resident in registers or SRAM between iterations, directly determining data locality and reuse.

Ignoring dependences and equivalent orderings, the number of ways to arrange n_{loops} loops has the toy upper bound:

$$N_{order} = n_{loops}!$$

which scales rapidly. A typical convolutional layer may involve up to seven loop dimensions, leading to:

$$7! = 5,040 \text{ possible execution orders.}$$

If each memory level could choose an ordering independently, the corresponding toy upper bound would expand as:

$$(n_{loops}!)^{N_{mem}}$$

where N_{mem} is the number of memory hierarchy levels. This rapid expansion shows why execution order optimization matters: suboptimal loop ordering evicts values prematurely and multiplies DRAM traffic, whereas an optimized ordering maximizes reuse in local SRAM and caches (Sze et al. 2017).

11.7.2.2 Parallelization across processing elements

Modern AI accelerators use thousands of processing elements to execute loop iterations concurrently. Parallelizing an inner reduction loop, however, requires cross-core synchronization and collective communication, whereas parallelizing outer independent loops (such as batch or output feature channels) allows each core to execute independently without interconnect traffic. Determining which loops to map spatially requires careful partitioning.

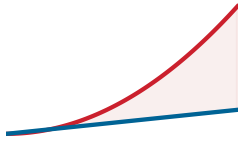
Before legality and capacity constraints are applied, the number of ordered ways to select loops for parallel execution has the upper bound:

$$\mathcal{P}_{parallel} = \frac{n_{loops}!}{(n_{loops} - k_{parallel})!}$$

where n_{loops} is the number of loops, and $k_{parallel}$ is the number selected for parallel execution. For a six-loop computation where three loops are selected, this unconstrained count is:

$$\frac{6!}{(6-3)!} = 120.$$

Even for a single layer, the candidate count reaches hundreds before invalid or equivalent strategies are removed. Each candidate schedule imposes different demands on barrier synchronization, interconnect bandwidth, and PE utilization.



Mapping choices explode combinatorially as loop dimensions grow.

11.7.2.3 Memory placement and data movement

The physical memory hierarchy forces compilers to determine which tensors reside in registers, shared memory, or global DRAM at each stage of execution. Fetching operands from off-chip DRAM consumes orders of magnitude more energy and latency than reading from local SRAM, making data placement the primary determinant of achieved arithmetic intensity.

If each computational dimension had n independent choices at every memory level, the resulting toy upper bound would be:

$$\mathcal{M}_{\text{placement}} = n^{N_{\text{comp}} \times N_{\text{mem}}}$$

where:

- n = number of placement choices per level,
- N_{comp} = number of computational dimensions,
- N_{mem} = number of memory hierarchy levels.

For a model with:

- $N_{\text{comp}} = 5$ computational dimensions,
- $N_{\text{mem}} = 3$ memory levels,
- $n = 4$ possible placement choices per level,

the number of possible memory allocations is:

$$4^{5 \times 3} = 4^{15} = 1,073,741,824.$$

11.7.2.4 Mapping search space

This unconstrained mapping example exceeds a billion combinations, although many are illegal, coupled, or equivalent. Multiplying the toy counts gives an upper-bound illustration of the mapping search space:

$$\mathcal{S}_{\text{mapping}} = \left(n^{N_{\text{comp}}} \times n_{\text{loops}}! \times \frac{n_{\text{loops}}!}{(n_{\text{loops}} - k_{\text{parallel}})!} \right)^{N_{\text{mem}}}$$

where:

- $n^{N_{\text{comp}}}$ represents memory placement choices,
- $n_{\text{loops}}!$ accounts for computation ordering choices,
- $\frac{n_{\text{loops}}!}{(n_{\text{loops}} - k_{\text{parallel}})!}$ captures parallelization possibilities,
- N_{mem} is the number of memory hierarchy levels.

This equation is not an exact count of legal schedules because choices interact and hardware constraints prune the space. It illustrates why candidate spaces grow rapidly and why exhaustive search becomes impractical. Even within a small layer, altering a single ordering decision can produce an order-of-magnitude difference in memory traffic.

Example 11.4: Loop ordering in a small convolution

Consider a convolution applying 16 filters of size 3×3 to an 8×8 single-channel input. The computation can be expressed as five nested loops iterating over output rows (H_{out}), output columns (W_{out}), filter count (C_{out}), filter height (F_h), and filter width (F_w). The $5! = 120$ permutations are an unconstrained upper bound: legal orderings preserve the mathematical result, although floating-point reduction order can change rounding, and they generate different memory traffic.

Ordering A (weight-stationary): Place the filter loops (C_{out}, F_h, F_w) outermost and the spatial loops ($H_{\text{out}}, W_{\text{out}}$) innermost. Each 3×3 filter is loaded into registers once and then applied across all 36 output positions before the next filter is loaded. Total weight loads: $16 \times 9 = 144$ values, each loaded exactly once.

Ordering B (output-stationary): Place the spatial loops outermost and the filter loops innermost. For every output position, all 16 filters must be loaded, applied, and their partial sums accumulated before advancing to the next position. If the register file cannot hold all 16 filters simultaneously, filters are repeatedly fetched from cache or DRAM. In the worst case, each of the 36 output positions reloads all 144 filter weights, producing $36 \times 144 = 5,184$ weight reads.

Systems insight: Under these two limiting assumptions, Ordering A reduces modeled weight reads by $36\times$ relative to the worst case for Ordering B. Real kernels also account for activation and partial-sum traffic, register capacity, cache behavior, and vectorization. The example nevertheless shows why two mathematically equivalent loop orders can place very different demands on the memory system.

This combinatorial explosion presents a central puzzle in ML systems: if the unconstrained search space contains billions of candidates, how do compilers and kernel developers routinely achieve near-peak hardware efficiency? Rather than evaluating arbitrary permutations, production systems restrict their search to a small family of canonical dataflow patterns—spatial and temporal schedules engineered to exploit the specific reuse properties of neural network operators.

11.8 Dataflow Optimization

Spatial and temporal mapping (section 11.7) establish where operations execute and where operands reside, but **dataflow optimization** governs the exact trajectories and schedules of data moving through processing elements and the memory hierarchy. In a systolic array or tiled vector unit, the temporal schedule and spatial routing of weights, inputs, and partial sums dictate how frequently operands must be fetched from energy-expensive DRAM rather than reused from local registers or scratchpads. By controlling off-chip traffic, the dataflow establishes the effective operational intensity of the kernel, determining whether execution achieves the compute-bound plateau or stalls in the memory-bound regime of the Roofline model. Optimizing compilers (section 11.9) and runtime systems (section 11.10) therefore tailor dataflow patterns to match the reuse characteristics of each layer to the hardware constraints of the target accelerator.

Three recurring architectural decisions structure this design space:

1. **Locality:** Stationary operand selection (weight-stationary, output-stationary, or input-stationary) dictates which tensor remains pinned in local registers or processing-element scratchpads, trading off memory traffic against register pressure.
2. **Organization:** Physical tensor layouts (such as channels-last NHWC versus channels-first NCHW) govern whether memory accesses match hardware memory controller burst sizes and SIMD vector widths, preventing uncoalesced memory transactions.
3. **Combination:** Kernel fusion and loop tiling partition iteration spaces and merge adjacent operations to eliminate round-trips to off-chip memory for intermediate tensors.

Together, these patterns provide a compact vocabulary for navigating dataflow choices without exhaustive search across billions of candidate schedules.

11.8.1 Building blocks of mapping strategies

These three decisions translate into four foundational techniques: data movement patterns that fix specific operands near the arithmetic units (weight-stationary, output-stationary, and input-stationary), memory-efficient tensor layouts (channels-last versus channels-first), kernel fusion that keeps intermediate activations in on-chip SRAM between operations, and loop tiling that bounds working set sizes to fit local memory capacity. Rather than rediscovering schedules from scratch, heuristic compilers and auto-tuning frameworks compose these building blocks to construct efficient execution plans across deep neural network layers.

11.8.1.1 Data movement patterns

On modern accelerators, accessing an operand from off-chip DRAM consumes roughly two to three orders of magnitude more energy than executing an arithmetic operation or fetching from a local register file (Y.-H. Chen et al. 2016). Whether a workload executes dense matrix operations that

exceed on-chip cache capacity or irregular sparse lookups, data movement strategy dictates overall execution latency and total power consumption. If data cannot be staged and supplied to processing elements at the rate required by their arithmetic throughput, the execution units stall on memory latency, stranding theoretical compute capacity and inflating memory subsystem energy.

To prevent arithmetic pipelines from stalling, a data movement strategy must capture the inherent algorithmic reuse of tensor operations within local, low-energy storage. Listing 11.15 illustrates the canonical three-nested-loop implementation of matrix multiplication, exposing the operand reuse across weights, inputs, and partial sums that a hardware mapping must capture in local storage.

Listing 11.15: Naïve Matrix-Multiplication Dataflow: The loop nest revisits weight rows, input columns, and output accumulators, exposing the reuse that a mapping must capture in local memory.

```
## Matrix multiplication where:
## weights: [512x256] - model parameters
## input:   [256x32]  - batch of activations
## Z:       [512x32]  - output activations

## Computing each output element Z[i,j]:
for i in range(512):
    for j in range(32):
        for k in range(256):
            Z[i, j] += weights[i, k] * input[k, j]
```

This computation reveals several critical dataflow challenges. The first challenge is the number of memory accesses required. For each output Z_{ij} , the computation must fetch an entire row of weights from the weight matrix and a full column of activations from the input matrix. Since the weight matrix contains 512 rows and the input matrix contains 32 columns, this results in repeated memory accesses that place a heavy burden on memory bandwidth.

The second challenge comes from weight reuse. The same weights are applied to multiple inputs, meaning that an ideal mapping strategy should maximize weight locality to avoid redundant memory fetches. Without proper reuse, the accelerator would waste bandwidth loading the same weights multiple times (Chen et al. 2018).

The third challenge involves the accumulation of intermediate results. Since each element in Z_{ij} requires contributions from 256 different weight-input pairs, partial sums must be stored and retrieved before the final value is computed. If these intermediate values are stored inefficiently, the system will require frequent memory accesses, further increasing bandwidth demands.

One way to mitigate these challenges is to use SIMD and SIMT execution models, which allow multiple values to be fetched in parallel. However, even with these optimizations, data movement remains a bottleneck. The primary bottleneck is not retrieval speed alone, but transfer frequency and placement within the memory hierarchy (Han, Liu, et al. 2016).

Because moving data dominates the energy budget that figure 11.9 charts, the single most important goal of an accelerator is to minimize memory access. Dataflow strategies achieve this by maximizing data reuse. The central decision is which data is most valuable to keep local. Accelerators answer that decision by determining which data remains fixed in memory and which data streams dynamically: weight-stationary keeps model parameters local, input-stationary maintains activation data, and output-stationary preserves intermediate results. Each approach trades off different memory access patterns to maximize data reuse and minimize the energy-intensive transfers that constitute the primary bottleneck in AI acceleration.

Weight stationary The weight stationary strategy keeps weights fixed in local memory, while input activations and partial sums are streamed through the system. Weight stationary approaches prove particularly beneficial in CNNs and matrix multiplications, where the same set of weights is applied across multiple inputs. By ensuring weights remain stationary, this method reduces redundant memory fetches, which helps alleviate bandwidth bottlenecks and improves energy efficiency.

A key advantage of weight stationary is that it maximizes weight reuse, reducing the frequency of memory accesses to external storage. Since weight parameters are often shared across multiple computations, keeping them in local memory eliminates unnecessary data movement, lowering

the overall energy cost of computation. This makes it particularly effective for architectures where weights represent the dominant memory overhead, such as systolic arrays and custom accelerators designed for machine learning. Listing 11.16 demonstrates how Weight Stationary execution keeps weights fixed in local memory while streaming inputs and accumulating partial sums.

Listing 11.16: Weight Stationary Dataflow: Weights stay resident in local memory while inputs and partial sums stream through, minimizing parameter read traffic; best for CNNs and matrix multiplications with heavy weight reuse.

```
## Weight Stationary Matrix Multiplication
## - Weights remain fixed in local memory
## - Input activations stream through
## - Partial sums accumulate for final output

for weight_block in weights: # Load and keep weights stationary
    load_to_local(weight_block) # Fixed in local storage
    for input_block in inputs: # Stream inputs dynamically
        for output_block in outputs: # Compute results
            output_block += compute(weight_block, input_block)
        # Reuse weights across inputs
```

In weight stationary execution, weights are loaded once into local memory and remain fixed throughout the computation while inputs stream dynamically, reducing redundant memory accesses. Partial sums accumulate efficiently, minimizing unnecessary data movement. Because weights need not be reloaded for each new computation, bandwidth requirements drop significantly, making this dataflow highly effective for workloads with heavy weight reuse patterns such as CNNs and matrix multiplications.

However, while this strategy reduces weight-related memory traffic, it introduces trade-offs in input and output movement. Since inputs must be streamed dynamically while weights remain fixed, the efficiency of this approach depends on how well input activations can be delivered to the computational units without causing stalls. Partial sums, which represent intermediate results, must also be carefully accumulated to avoid excessive memory traffic. The total performance gain depends on the size of available on-chip memory, as storing larger weight matrices locally can become a constraint in models with millions or billions of parameters.

The weight stationary strategy is well-suited for workloads where weights exhibit high reuse and memory bandwidth is a limiting factor. It is commonly employed in CNNs, systolic arrays, and matrix multiplication kernels, where structured weight reuse leads to measurable performance improvements. However, for models where input or output reuse is more critical, alternative dataflow strategies, such as output stationary or input stationary, may provide better trade-offs.

Output stationary Weight stationary keeps weights local and streams inputs through the system. The dominant cost shifts, however, when the bottleneck is not weight loading but the frequent writes of partial sums. In fully connected layers and transformer attention mechanisms, each output element accumulates contributions from hundreds or thousands of weight-input pairs. Writing those intermediate partial sums to external memory after every accumulation step would create a write-bandwidth bottleneck far more severe than the read overhead that weight stationary addresses. The output stationary strategy inverts the priority: it keeps partial sums fixed in local memory while streaming both weights and input activations through the system, so that each output element is written to external memory only once, after all its contributions have been accumulated (Y.-H. Chen et al. 2016).

Listing 11.17 demonstrates how accumulating partial sums locally minimizes memory writes and enhances efficiency during matrix multiplication. In this implementation, the accumulator buffer stays in local registers or scratchpad throughout the inner loop; weights and inputs stream in, contribute to the running sum, and are discarded. The final result is written out only once per output element, eliminating the repeated write traffic that would otherwise dominate bandwidth.

This approach aligns naturally with systolic arrays, where computation progresses through a grid of processing elements and partial sums can flow along one axis without leaving the chip. The trade-off is that both weights and activations must now be streamed dynamically, so the system

must sustain high read bandwidth for two data streams simultaneously. Parallel implementations also require careful synchronization when multiple PEs contribute to the same output element. Output stationary is therefore most effective for workloads where accumulation dominates, such as fully connected layers and attention mechanisms, but less suitable when input reuse is the critical bottleneck.

Listing 11.17: Output Stationary Dataflow: Partial sums stay resident in local memory while weights and inputs stream through, so each output is written out only once, minimizing accumulation write traffic; best for fully connected layers and attention.

```
## - Partial sums remain in local memory
## - Weights and input activations stream through dynamically
## - Final outputs are written only once

for output_block in outputs: # Keep partial sums stationary
    accumulator = 0 # Initialize accumulation buffer
    for weight_block, input_block in zip(weights, inputs):
        accumulator += compute(weight_block, input_block)
        # Accumulate partial sums
    store_output(accumulator) # Single write to memory
```

Input stationary The two strategies examined so far each fix a different operand in local memory: weight stationary fixes weights to reduce read bandwidth for parameters, and output stationary fixes partial sums to reduce write bandwidth for accumulations. The third strategy completes the picture by fixing the remaining operand: input activations. Within a transformer layer, one activation can feed multiple projections or attention heads; the next layer receives a newly computed representation rather than the same token activation. When activation reuse is the dominant memory cost, keeping inputs stationary and streaming weights through the system can reduce energy and bandwidth. Listing 11.18 illustrates this approach.

Listing 11.18: Input Stationary Dataflow: Input activations stay resident in local memory while weights stream through, reducing activation read traffic when one activation tile feeds several computations.

```
## - Input activations remain in local memory
## - Weights stream through dynamically
## - Partial sums accumulate and are written out

for input_block in inputs: # Keep input activations stationary
    load_to_local(input_block) # Fixed in local storage
    for weight_block in weights: # Stream weights dynamically
        for output_block in outputs: # Compute results
            output_block += compute(weight_block, input_block)
            # Reuse inputs across weights
```

Here, input activations are loaded once and held fixed in local storage while weights stream through. Partial sums accumulate and are eventually written out, but unlike output-stationary dataflow, the accumulation buffer is not the primary beneficiary of locality; instead, the input activations are.

The trade-off mirrors the other two strategies: weights stream dynamically, requiring sustained read bandwidth from memory, and partial sums require buffering before write-back. Input stationary is most effective when an activation tile feeds multiple projections, attention heads, or gate evaluations before eviction.

Taken together, the three dataflow strategies provide a decision framework rather than a hierarchy of quality. Weight stationary minimizes parameter read traffic, fitting convolutional networks with compact, heavily reused filters. Output stationary minimizes write traffic for accumulations, fitting fully connected layers with wide fan-in. Input stationary minimizes activation read traffic, fitting transformers and large-batch inference with high activation reuse. No single strategy dominates; the optimal choice depends on which tensor dimension provides the highest reuse ratio relative to on-chip storage capacity. Hardware compilers select among these strategies based on workload

dimensions and memory hierarchy limits. Convolution-specific designs add a fourth label, row-stationary (as in Eyeriss), but it is a hybrid that keeps rows of inputs and partial sums local when that mapping yields higher reuse than fixing any single operand (section 11.5.6.2).

11.8.1.2 Memory-efficient tensor layouts

The dataflow strategies in section 11.8.1.1 determine which operands stay close to compute units; tensor layouts determine whether those operands can be accessed efficiently once they arrive. A well-chosen weight-stationary schedule still stalls if weights are laid out in a format that produces strided, non-coalesced memory fetches. Tensor layout represents a kernel contract: the physical ordering of multidimensional data in linear memory must match the memory access stride expected by the accelerator’s execution units, or the processor pays in memory stalls, cache underutilization, and redundant bus transactions.

Hardware accelerators fetch data from device memory in fixed-size transactions—typically 32 to 128 bytes per transaction. When compute units request non-contiguous addresses, each transaction delivers only a fraction of useful payload, degrading effective memory bandwidth. Aligning the tensor layout with the hardware execution model ensures that concurrent threads or SIMD vector lanes access contiguous memory locations in a single unified transaction (NVIDIA Corporation 2021b).

Machine learning frameworks and compilers often manage this layout selection automatically. Optimization tools such as cuDNN for NVIDIA GPUs, XLA for TensorFlow and JAX, and MLIR-based compiler stacks transform tensor layouts as they lower operations to backend-specific kernels (NVIDIA Corporation 2021b; Google 2025; Lattner et al. 2020). In high-level frameworks, layout transformations are typically applied transparently, but developers authoring custom kernels or targeting low-level APIs such as CUDA, Metal, or OpenCL must manage tensor formats directly.

In PyTorch, for example, view operations such as `tensor.permute()` adjust tensor strides without moving data in memory, requiring an explicit `tensor.contiguous()` call to materialize a packed buffer when a downstream kernel requires contiguous storage (PyTorch Contributors 2026b). Conversely, TensorFlow defaults to channels-last (NHWC) for convolutional models on NVIDIA GPUs, warning that NCHW inputs introduce automatic transpose overhead when lowered to GPU kernels (TensorFlow Developers 2024b). Hardware-optimized libraries such as cuDNN on GPUs and oneDNN on CPUs implement tailored memory formats to maximize cache line utilization and SIMD throughput. The governing principle is to treat memory layout as an intrinsic property of the backend execution path: the fastest tensor format is the one that minimizes layout conversion overhead and makes memory accesses contiguous across hardware execution units.

Channels-last layout In an NHWC channels-last layout, the channel index varies fastest in a contiguous tensor, placing the channel values for each spatial coordinate adjacent in memory. NHWC specifies a logical dimension ordering rather than a linear storage convention; both NHWC and NCHW tensors represent linear storage sequences of their respective dimension orders.

Consider an RGB image tensor of shape (Height, Width, Channels) = (3, 3, 3). The elements are laid out in linear memory as follows:

$$I(0,0,0), I(0,0,1), I(0,0,2), I(0,1,0), I(0,1,1), \\ I(0,1,2), I(0,2,0), I(0,2,1), I(0,2,2), \dots$$

All channel values for a given pixel are adjacent, and memory addresses advance across width before advancing down height. This layout supports vectorized execution when a compute kernel processes multiple channels simultaneously: CPU SIMD registers and GPU Tensor Cores can load contiguous channel vectors directly without intra-register shuffling.

The advantage of channels-last storage depends on the target operator and hardware unit. TensorFlow uses NHWC by default, whereas PyTorch provides a dedicated channels-last memory format that decouples logical dimension ordering from physical storage. Modern Tensor Core kernels in cuDNN favor NHWC because matrix-multiply units pack channel dimensions directly into accumulator tiles. When kernels require channel-contiguous inputs, passing an incompatible format forces the framework runtime to insert an explicit memory transpose, incurring a memory bandwidth penalty before computation begins.

Channels-first layout In a contiguous NCHW channels-first layout, the spatial width index varies fastest, and all spatial values across an entire channel are stored contiguously before the next channel begins. Accelerators process data in parallel across thread warps or SIMD lanes; when consecutive threads in a warp access consecutive memory addresses, the memory controller coalesces those requests into a single memory transaction (**memory coalescing**).³⁵

Consider the same RGB image tensor with dimensions reordered in channels-first order as (Channels, Height, Width) = (3, 3, 3). Retaining $I(h, w, c)$ as semantic coordinate notation, the data are stored in channels-first order as follows:

$$I(0,0,0), I(0,1,0), I(0,2,0), I(1,0,0), I(1,1,0), I(1,2,0), \dots, \\ I(0,0,1), I(0,1,1), I(0,2,1), \dots, I(0,0,2), I(0,1,2), I(0,2,2), \dots$$

In this format, all red channel values for the entire image are stored first, followed by all green values, and then all blue values. This arrangement allows kernels that parallelize across spatial coordinates (H, W) to issue coalesced reads along image rows, which suited earlier GPU convolution algorithms and SIMD execution pipelines (Chetlur et al. 2014).

The relative advantage of NCHW versus NHWC depends on how the underlying kernel maps computation to hardware threads. Convolutional layers evaluate filters across spatial and channel dimensions simultaneously. In classic direct or FFT-based convolutions, channels-first ordering enabled efficient spatial sliding without stride jumps. However, modern GEMM-lowered convolutions and Tensor Core pipelines map channel reduction directly to matrix multiply accumulators, favoring channels-last structures. When a model operates in NCHW but invokes a Tensor Core-optimized backend, the runtime must either run an unoptimized fallback kernel or dynamically transpose the tensor into NHWC, burning memory bandwidth in both cases.³⁶

Comparing channels-last and channels-first layouts Both channels-last (NHWC) and channels-first (NCHW) layouts address specific hardware access patterns, with their efficiency determined by whether a kernel parallelizes over spatial coordinates or vectorizes across channel dimensions. Layout choice directly governs cache line utilization, memory bus transaction count, and compute unit occupancy. Table 11.18 contrasts the performance trade-offs and hardware compatibility between these two approaches.

Table 11.18: Data Layout Strategies: Channels-last (NHWC) and channels-first (NCHW) layouts optimize memory access patterns for different backend kernels. NHWC often suits CPU vectorization and modern Tensor Core convolution/fusion paths, while NCHW remains common in PyTorch model code and many GPU kernels. Choosing the appropriate layout directly impacts performance by maximizing cache utilization and memory bandwidth efficiency.

Feature	Channels-Last (NHWC)	Channels-First (NCHW)
Memory storage order	Pixels are stored row-by-row, channel interleaved	All values for a given channel are stored together first
Best for	CPU loops, element-wise operations, many channels-last kernels	Many legacy GPU convolution paths and channel-first model code
Cache efficiency	High cache locality for sequential row/channel-last access	Can improve coalescing for channel-first kernels
Convolution performance	Often preferred by modern Tensor Core convolution/fusion paths	Efficient for many cuDNN and framework kernels
Memory fetching	Good when kernels vectorize over adjacent channel data	Good when kernels process channel-major tiles
Default in frameworks	Common TensorFlow convention; PyTorch channels-last option	Common PyTorch tensor shape convention

While compilers such as XLA, TensorRT, and PyTorch Inductor automate layout propagation and insert layout conversions when beneficial, format conversions are not free: transposing a tensor requires reading every element from memory and writing it back in a different order, consuming memory bandwidth without executing useful arithmetic. In production deployments, matching the model's memory layout to the accelerator's native kernel format eliminates these layout transformations entirely.

³⁵ | **Memory coalescing:**

The GPU hardware mechanism that fuses memory requests from threads in a warp into a single transaction when those threads access contiguous memory. Tensor layout affects coalescing, but the sign of the effect depends on the kernel: NCHW can be efficient for some convolution implementations, while NHWC/channels-last is often preferred for modern Tensor Core convolution and fusion paths. Poor layout choices can still create multi-fold performance gaps, but the correct fix is to match the layout to the backend rather than memorize one universal format.

³⁶ | **NHWC vs. NCHW:**

NHWC lists dimensions as batch, height, width, channel; NCHW lists batch, channel, height, width. Physical memory format determines whether adjacent threads read adjacent addresses, so the performance effect is backend-specific. Layout-to-hardware mismatch is not a micro-optimization; it can create multi-fold performance gaps, especially when a conversion prevents Tensor Core convolution or fusion paths from being used.

Even when tensor layout is perfectly aligned with the hardware, executing operations as isolated kernels introduces a separate bottleneck: intermediate values must still travel to and from device memory between operations. Resolving that bottleneck requires changing how operations are scheduled rather than how tensors are laid out.

11.8.1.3 Kernel fusion

Reducing intermediate data movement between operations is one of the primary mechanisms for mitigating the memory wall. Kernel fusion³⁷ combines multiple successive operations into a single executable kernel, eliminating intermediate memory round trips.

Intermediate memory write Neural network execution is frequently bound by memory bandwidth rather than arithmetic throughput. When operations execute as isolated kernels, each intermediate result is written to device memory (DRAM or HBM) and subsequently read back by the downstream operator. This round trip introduces memory latency, stalls compute units, and consumes bus bandwidth on data with zero reuse outside that immediate pair of operations.

This hardware reality provides the physical motivation for the operator fusion techniques introduced in section 10.5.1.5. While section 11.5.1 analyzed bandwidth limits in terms of low operational intensity, chained elementwise operations aggravate the memory wall by multiplying global memory transactions (NVIDIA Corporation 2017).

In a naïve execution model, each operation executes as a separate GPU kernel, forcing intermediate tensors to round-trip through device memory (listing 11.19). In a chain of three operations—such as ReLU, batch normalization, and affine scaling—the intermediate outputs X_1 and X_2 are written to DRAM only to be immediately re-read by the subsequent kernel. For elementwise operators where arithmetic intensity is low ($I \ll 1$ FLOP/byte), the time spent moving data across the memory bus outweighs the computation time (Jia et al. 2019). Furthermore, holding these un-fused intermediate buffers in memory inflates the live-tensor footprint during execution (table 11.19).

Listing 11.19: Naïve Execution: Each step writes intermediate results to memory before processing the next, leading to increased bandwidth usage and reduced efficiency.

```
import torch
import torch.nn.functional as F

## Input tensor
X = torch.randn(1024, 1024).cuda()
running_mean = torch.zeros(1024, device=X.device)
running_var = torch.ones(1024, device=X.device)

## Step-by-step execution (naïve approach)
X1 = torch.relu(X) # Intermediate tensor stored in memory
X2 = F.batch_norm(
    X1, running_mean, running_var, training=False
) # Frozen inference BatchNorm
Y = 2.0 * X2 + 1.0 # Final result
```

Kernel fusion for memory efficiency The two intermediate tensors consume memory capacity when they remain live and create external traffic when separate kernels materialize them. Kernel fusion can eliminate those intermediate writes and reloads (Jia et al. 2019) by propagating values through registers or local memory within one kernel.

Table 11.19: Intermediate Tensor Storage: If all four named tensors remain live in this naïve example, they occupy 16.8 MB; an optimizing runtime may reuse buffers when earlier values are no longer needed. Fusion primarily guarantees removal of the external write and read for each eliminated intermediate rather than a universal reduction in peak allocated memory.

Tensor	Size (MB) for 1024×1024 Tensor
X	4.2 MB
X'	4.2 MB
X''	4.2 MB

³⁷ | **Kernel fusion:** Separate GPU kernels expose intermediate results through device memory, although caches can absorb some traffic. A fused kernel can keep intermediates in registers or local storage and avoid corresponding external write/read cycles. The resulting speedup depends on cache behavior, occupancy, launch overhead, and whether memory traffic was the bottleneck; it is not necessarily proportional to the byte reduction.

Tensor	Size (MB) for 1024×1024 Tensor
Y	4.2 MB
Total Memory	16.8 MB

A machine learning inference sequence might apply ReLU, frozen batch normalization, and then an affine scaling with scale α and offset β . In a naïve implementation, each separate kernel generates an intermediate tensor that is written to memory and read back by the next kernel:

$$\begin{aligned} \mathbf{X}' &= \text{ReLU}(\mathbf{X}) \\ \mathbf{X}'' &= \text{BatchNorm}(\mathbf{X}') \\ \mathbf{Y} &= \alpha \cdot \mathbf{X}'' + \beta \end{aligned}$$

With kernel fusion, these operations are combined into a single computation step, allowing the entire transformation to occur without generating unnecessary intermediate tensors:

$$\mathbf{Y} = \alpha \cdot \text{BatchNorm}(\text{ReLU}(\mathbf{X})) + \beta$$

Table 11.20 highlights the impact of operation fusion on memory efficiency. By keeping intermediate results in registers or local memory rather than writing them to main memory, fusion reduces external traffic. For three separate pointwise inference kernels, the idealized naïve path reads and writes one tensor per operation, whereas the fused path reads the input once and writes the output once.

Table 11.20: Operation Fusion Benefits: In this idealized inference sequence, fusion reduces external tensor traffic from 25.2 MB to 8.4 MB, a 3× reduction. Actual traffic depends on cache behavior and the compiler’s buffer reuse.

Execution Model	External tensor payloads	Traffic (MB)
Naïve execution	3 reads + 3 writes	25.2 MB
Fused execution	1 read + 1 write	8.4 MB

Performance benefits and constraints Kernel fusion brings several key advantages that enhance memory efficiency and computation throughput. By reducing memory accesses, fused kernels ensure that intermediate values stay within registers instead of being repeatedly written to and read from memory. This significantly lowers memory traffic, which is one of the primary bottlenecks in machine learning workloads. GPUs and TPUs, in particular, benefit from kernel fusion because high-bandwidth memory is a scarce resource, and reducing memory transactions leads to better utilization of compute units (NVIDIA Corporation 2020a).

However, not all operations can be fused arbitrarily. Element-wise operations and frozen inference normalization are strong candidates because their computations do not require batch-wide reductions. Training-mode batch normalization computes batch statistics and is not purely element-wise. Matrix multiplications and convolutions constrain fusion because they involve reductions and large data movement; they are often fused with element-wise epilogues such as bias or activation, but cannot be freely fused with unrelated global operations.

Another major consideration is register pressure. Fusing multiple operations means all temporary values must be kept in registers rather than memory. While this eliminates redundant memory writes, it also increases register demand. If a fused kernel exceeds the available registers per thread, the compiler can spill excess values to per-thread local memory, which is backed by device memory and may be cached, introducing additional latency and potentially negating the benefits of fusion. On GPUs, where thread occupancy (the number of threads that can run in parallel) is limited by available registers, excessive fusion can reduce parallelism, leading to diminishing returns.

Different AI accelerators and compilers handle fusion in distinct ways. NVIDIA GPUs, for example, favor warp-level parallelism, where element-wise fusion is straightforward (NVIDIA Corporation 2020a). TPUs, on the other hand, prioritize systolic array execution for dense matrix operations (Jouppi et al. 2017). Compiler and inference stacks such as TVM, XLA, TensorRT, and MLIR apply graph rewrites, lowering passes, or engine-building heuristics to balance memory savings against execution constraints (Chen et al. 2018; Google 2025; NVIDIA 2024c; Lattner et al. 2020).

Despite its advantages, fusion is not always beneficial. Some AI frameworks allow developers to disable fusion selectively, especially when debugging performance issues or making frequent model modifications. The decision to fuse operations must consider trade-offs between memory efficiency, register usage, and hardware execution constraints to ensure that fusion leads to tangible performance improvements.



Checkpoint 11.3: Data movement and kernel fusion

The dataflow building blocks are now in place: data locality, tensor layout, and kernel fusion. These fusion decisions are ultimately about data locality, which ties together the chapter's core data movement strategies:

- ❑ **Locality choice:** given weight-stationary, output-stationary, and input-stationary dataflows, which tensor does each hold near the compute units, and how does that choice shift where the memory traffic lands?
- ❑ **Layout match:** a kernel runs on a backend expecting NHWC but receives an NCHW tensor. Predict what happens to its memory access pattern and to delivered throughput.
- ❑ **Fusion eligibility:** for a Conv2D, a BatchNorm, and a ReLU executed in sequence, decide which pairs are worth fusing and what determines whether fusing them pays off.
- ❑ **Remaining gap:** locality, layout, and fusion are all chosen well, yet the operation still stalls on memory. What is the missing transformation, and why do the other three not address it?

11.8.1.4 Memory-efficient tiling strategies

While modern AI accelerators offer high computational throughput, their performance is often limited by memory bandwidth rather than raw processing power. If data cannot be supplied to processing units fast enough, execution stalls occur, leading to wasted cycles and inefficient hardware utilization.

Tiling³⁸ mitigates this issue by restructuring computations into smaller, memory-friendly sub-problems. When memory bandwidth cannot be increased directly, systems must reduce trips to main memory. Instead of processing entire matrices or tensors at once, which leads to excessive memory traffic, tiling partitions computations into smaller blocks (tiles) that fit within fast local memory (for example, caches, shared memory, or registers) (Lam et al. 1991).

Matrix multiplication, widely used in AI models, demonstrates inefficient memory access when implemented naively. Listing 11.20 shows how, without tiling, repeated memory accesses for the same data lead to unnecessary bandwidth consumption.

Listing 11.20: Naïve Matrix Multiplication: Direct implementation makes $\mathcal{O}(N^3)$ array references for $N \times N$ matrices. Caches can serve some references, but the loop order does not deliberately block reuse for the memory hierarchy.

```
for i in range(N):
    for j in range(N):
        for k in range(N):
            C[i, j] += A[i, k] * B[k, j]  # Repeatedly fetching
                                         # A[i, k] and B[k, j]
```

The loop repeatedly references elements of **A** and **B**. Some references can hit in cache, but large matrices and an unfavorable traversal order can still cause excessive lower-tier traffic.

Tiling addresses this problem by ensuring that smaller portions of matrices are loaded into fast memory, reused efficiently, and only written back to main memory when necessary. This technique is especially important in AI accelerators, where memory accesses dominate execution time. Figure 11.14 labels the product $\mathbf{C} = \mathbf{AB}$ by its three dimensions: M rows and K columns in **A**, K rows and N columns in **B**, and $M \times N$ in the output **C**. Each highlighted tile is the working set that fits in fast memory at one moment: a green $M_{\text{tile}} \times K_{\text{tile}}$ row band of **A** multiplies a pink $K_{\text{tile}} \times N_{\text{tile}}$ column band of **B** to accumulate one blue $M_{\text{tile}} \times N_{\text{tile}}$ output block (Block _{m,n}) of **C**. Processing all

³⁸ | **Tiling (loop blocking):**

This restructuring partitions a computation into blocks sized for a fast local tier. A naive loop makes repeated references to the same elements; caches may capture some, while blocking deliberately concentrates reuse before eviction (Lam et al. 1991). This reduction in lower-tier traffic is a primary source of the gap between naive matrix multiplication and optimized GEMM routines.

computations for one tile before moving to the next avoids repeatedly paying the DRAM access penalty.

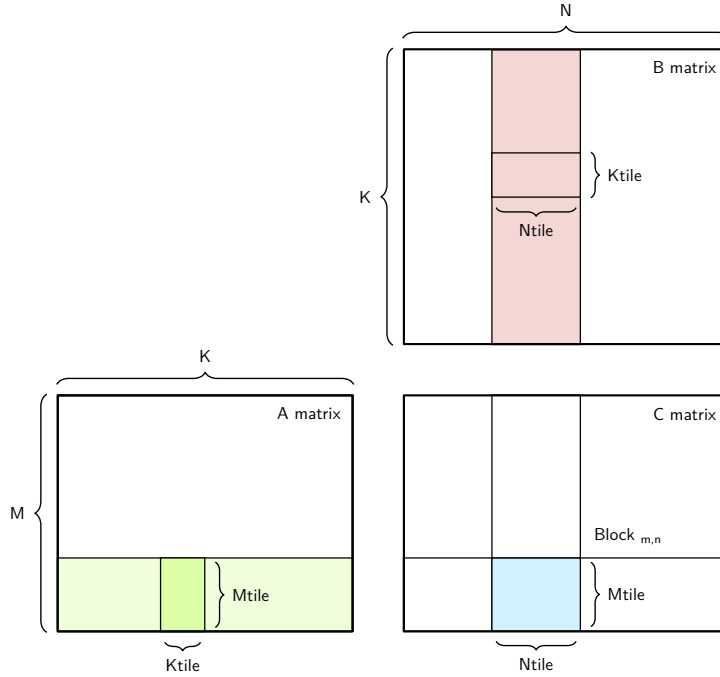


Figure 11.14: Matrix Tiling Mechanics: Partitioning large matrix multiplication ($C = AB$) into $M_{\text{tile}} \times K_{\text{tile}}$ and $K_{\text{tile}} \times N_{\text{tile}}$ blocks aligns memory footprints with fast on-chip SRAM/registers. Maximizing register reuse within each tile amortizes slow off-chip DRAM accesses and keeps compute engines operating at peak throughput.

Tiling fundamentals Tiling divides a computation into smaller tiles that fit within available fast memory rather than operating on an entire data structure at once. This structure maximizes data reuse, reducing redundant memory accesses and improving efficiency.

Consider matrix multiplication, a key operation in machine learning workloads. The operation computes $C = A \times B$ where each element $C_{ij} = \sum_k A_{ik} \times B_{kj}$. Listing 11.20 exposes the core problem: every iteration of the innermost loop fetches elements from matrices **A** and **B** from memory, performs a multiplication, and updates matrix **C**. Because matrices are large, the processor repeatedly reloads the same values from memory, even though they were just used in previous computations.

This data movement overhead is expensive: DRAM access has much higher latency and energy cost than access to on-chip cache or registers (Horowitz 2014; Sze et al. 2017). The solution is tiling.

Performance benefits of tiling Instead of computing one element at a time and constantly moving data in and out of slow memory, tiling processes submatrices (tiles) at a time, keeping frequently used values in fast memory. The idea is to divide the matrices into smaller blocks that fit within the processor’s cache or shared memory, ensuring that once a block is loaded, it is reused multiple times before moving to the next one. Listing 11.21 demonstrates cache-friendly loop blocking: the loop bounds partition the matrices into tiles, and the hardware cache hierarchy keeps recently used tile data close to the compute units when access order has locality.

Listing 11.21: Cache-Blocked Matrix Multiplication: This high-level loop blocking approach divides matrices into smaller index ranges so hardware caches can reuse data within each tile, improving computational efficiency without explicit scratchpad loads.

```
TILE_SIZE = 32 # Choose a tile size based on hardware constraints

# Cache blocking: partition data via loop bounds.
# Loads are implicit through the hardware cache hierarchy.
for i in range(0, N, TILE_SIZE):
    for j in range(0, N, TILE_SIZE):
        for k in range(0, N, TILE_SIZE):
            # Each tile computed independently
            for ii in range(i, min(i + TILE_SIZE, N)):
                for jj in range(j, min(j + TILE_SIZE, N)):
                    for kk in range(k, min(k + TILE_SIZE, N)):
                        C[ii, jj] += A[ii, kk] * B[kk, jj]
```

This restructuring significantly improves performance through three reinforcing effects. Memory reuse improves because the approach visits a small tile repeatedly while it is likely to remain in cache before moving on to the next tile, rather than fetching elements from **A** and **B** repeatedly from slow memory, which minimizes redundant memory accesses. Memory bandwidth usage drops as a direct consequence: since each tile is used multiple times before being evicted, most required data is available in L1/L2 cache or shared memory rather than DRAM, so traffic falls and execution speeds up. Compute efficiency rises in turn, because processors spend less time waiting for data and more time performing useful work; in architectures like GPUs and TPUs, where thousands of parallel processing units operate simultaneously, tiling keeps data read and processed in a structured manner that avoids unnecessary stalls.

This technique is particularly effective in AI accelerators, where machine learning workloads consist of large matrix multiplications and tensor transformations. Without tiling, these workloads quickly become memory bound, meaning performance is constrained by how fast data can be retrieved rather than by the raw computational power of the processor.

Tiling methods While the general principle of tiling remains the same, which involves partitioning large computations into smaller subproblems to improve memory reuse, there are different ways to apply tiling based on the structure of the computation and hardware constraints. The two primary tiling strategies are spatial tiling and temporal tiling. These strategies optimize different aspects of computation and memory access, and in practice, they are often combined to achieve the best performance.

Spatial tiling partitions data structures into smaller blocks that fit within fast memory. The tiled matrix multiplication in listing 11.21 demonstrates cache-friendly loop blocking: the code does not issue explicit scratchpad loads, but the tile-shaped access pattern lets hardware caches reuse nearby values before they are evicted. This strategy is particularly beneficial for large tensors that exceed fast memory capacity—by breaking computations into smaller tiles, data movement between memory levels is minimized, keeping operations localized within cache hierarchies.

Temporal tiling complements spatial tiling by explicitly staging data in shared memory or registers and reorganizing the computation order around that staged data. Many ML workloads access the same data repeatedly across iterations—without temporal tiling, this results in redundant memory fetches. Temporal tiling restructures the computation to ensure that frequently used data stays in fast memory for as long as possible before the next computation begins.

A classic example where temporal tiling is beneficial is convolutional operations, where the same set of weights is applied to multiple input regions. Without loop blocking, these weights might be loaded from memory multiple times for each computation. With temporal tiling, the computation is reordered so that the weights remain in fast memory across multiple inputs, reducing unnecessary memory fetches and improving overall efficiency. Listing 11.22 illustrates explicit tile staging: the code loads blocks of **A** and **B** into temporary fast storage, then reuses them across multiple inner-loop operations.

Listing 11.22: Explicit Tile Staging Pseudocode: A backend maps the temporary tile objects to fast storage and reuses them across inner-loop operations.

```
# Explicit tile staging: load data into fast
# temporary storage before the inner loops.
for i in range(0, N, TILE_SIZE):
    for j in range(0, N, TILE_SIZE):
        for k in range(0, N, TILE_SIZE):
            # Pseudocode: map these tile objects to fast memory
            A_tile = A[i : i + TILE_SIZE, k : k + TILE_SIZE]
            B_tile = B[k : k + TILE_SIZE, j : j + TILE_SIZE]

            # Reuse loaded tiles for all inner iterations
            for ii in range(A_tile.shape[0]):
                for jj in range(B_tile.shape[1]):
                    for kk in range(A_tile.shape[1]):
                        C[i + ii, j + jj] += (
                            A_tile[ii, kk] * B_tile[kk, jj]
                        )
```

Explicit tile staging improves performance when the backend places the temporary tiles in fast memory and reuses them before eviction. The pseudocode includes shortened boundary tiles when N is not divisible by $TILE_SIZE$.

This technique is particularly useful in workloads where certain values are used repeatedly, such as convolutions, recurrent neural networks (RNNs), and self-attention mechanisms in transformers. By applying loop blocking, AI accelerators can significantly reduce memory stalls and improve execution throughput.

Tiling challenges and trade-offs Tiling improves performance only when the tile matches the locality budget of the hardware. If the tile is too small, memory fetches still dominate execution time because reuse is too limited. If the tile is too large, it exceeds fast memory and causes cache thrashing or scratchpad spills. Selecting the right tile size therefore directly determines computational efficiency and memory bandwidth usage.

The tile choice also controls load balance. In architectures such as GPUs and TPUs, computations execute in parallel across thousands of processing units. If tiles are not evenly distributed, some units remain idle while others are overloaded, leading to suboptimal utilization of computational resources. Effective tile scheduling keeps parallel execution balanced and efficient.

Data movement remains the limiting cost even after tiling. Although tiling reduces the number of slow memory accesses, transferring tiles between hierarchy levels still incurs latency and energy cost, especially when data falls from cache or scratchpad back to DRAM. Efficient memory prefetching and scheduling strategies minimize this residual movement and ensure that data is available when needed.

Hybrid tiling combines spatial and temporal strategies when neither dimension alone captures the workload’s reuse pattern. Some AI accelerators use spatial tiling for matrix multiplications while employing temporal tiling for weight reuse in convolutional layers, dynamically adjusting tile sizes or reordering computations based on real-time execution conditions.

Register blocking, double buffering, and hierarchical tiling extend the same locality principle at smaller and larger memory tiers. AI compilers and runtime systems such as TensorFlow XLA, TVM, and MLIR automatically select these tiling strategies based on hardware constraints, enabling fine-tuned performance optimization without manual intervention. Table 11.21 provides a comparative overview of spatial, temporal, and hybrid tiling approaches, highlighting their respective benefits and trade-offs.

When machine learning models grow in size and complexity, tiling remains a critical tool for improving hardware efficiency, ensuring that AI accelerators operate near their practical potential. While manual tiling strategies can provide substantial benefits, compilers and hardware-aware optimization techniques further enhance performance by automatically selecting effective tiling strategies for a given workload.

Table 11.21: Tiling Strategies: Spatial, temporal, and hybrid tiling optimize memory access patterns for improved performance. Spatial tiling maximizes data reuse within fast memory, temporal tiling exploits loop structure for reduced accesses, and hybrid tiling combines both approaches. AI compilers and runtime systems use these techniques to automatically optimize model execution on diverse hardware.

Aspect	Spatial Tiling (Data Tiling)	Temporal Tiling (Loop Blocking)	Hybrid tiling
Primary Goal	Reduce memory accesses by keeping data in fast memory longer	Increase data reuse across loop iterations	Adapt dynamically to workload constraints
Optimization Focus	Partitioning data structures into smaller, memory-friendly blocks	Reordering computations to maximize reuse before eviction	Balancing spatial and temporal reuse strategies
Memory usage	Improves cache locality and reduces DRAM access	Keeps frequently used data in fast memory for multiple iterations	Minimizes data movement while ensuring high reuse
Common use cases	Matrix multiplications, CNNs, self-attention in transformers	Convolutions, recurrent neural networks (RNNs), iterative computations	AI accelerators with hierarchical memory, mixed workloads
Performance gains	Reduced memory bandwidth requirements, better cache utilization	Lower memory fetch latency, improved data locality	Maximized efficiency across multiple hardware types
Challenges	Requires careful tile size selection, inefficient for workloads with minimal spatial reuse	Can increase register pressure, requires loop restructuring	Complexity in tuning tile size and execution order dynamically
Best when	Data is large and needs to be partitioned for efficient processing	The same data is accessed multiple times across iterations	Both data partitioning and iteration-based reuse are important

11.8.2 Applying mapping strategies to neural networks

While these foundational mapping techniques apply broadly, their effectiveness varies based on the computational structure, data access patterns, and parallelization opportunities of different neural network architectures. Each architecture imposes distinct constraints on data movement, memory hierarchy, and computation scheduling, requiring tailored mapping strategies to optimize performance.

A structured approach to mapping is required to address the combinatorial explosion of choices that arise when assigning computations to AI accelerators. Rather than treating each model as a separate optimization problem, the same principles apply across different architectures; only their priority shifts based on workload characteristics. The goal is to systematically select and apply mapping strategies that maximize efficiency for different types of machine learning models.

These principles apply to three representative AI workloads, each characterized by distinct computational demands. CNNs benefit from spatial data reuse, making weight-stationary execution and tiling especially effective. Transformer behavior depends on phase and shape: training and prefill matrix multiplications can be compute bound, while low-batch decoding often depends on weight and KV-cache bandwidth. MLPs involve substantial matrix multiplication and benefit from structured tiling, optimized weight layouts, and memory-aware execution.

Despite their differences, each of these models follows a common set of mapping principles, with variations in how optimizations are prioritized. Table 11.22 summarizes the suitability of different optimization strategies for CNNs, transformers, and MLPs.

11.8.2.1 Convolutional neural networks (ResNet-50)

For ResNet-50 and similar CNNs, a key mapping opportunity is spatial weight reuse. A small filter is applied across many spatial locations, so its weights participate in many multiply-accumulate operations. Weight-stationary mappings can capture that reuse by retaining filter tiles locally, while row-stationary or output-stationary mappings may better balance activation and partial-sum traffic on other architectures. Favorable convolution shapes can achieve high arithmetic intensity, which makes them well suited to matrix and systolic execution paths. The model family does not determine one universal dataflow: layer shape, batch size, local-storage capacity, and the target’s supported kernels decide which reuse pattern is worth retaining.

Table 11.22: Architecture-Specific Mapping Strategies: Each neural network architecture benefits from different optimization priorities based on its computational and memory characteristics.

Optimization technique	CNNs	Transformers	MLPs	Rationale
Dataflow strategy	Weight, row, or output stationary	Phase- and kernel-dependent	Weight or output stationary	CNNs expose filter and spatial reuse; transformer attention and MLP blocks impose different traffic; dense MLPs reuse weights across a batch and accumulate outputs.
Memory-aware tensor layouts	Backend-dependent (often NCHW for cuDNN convolutions, channels-last on Tensor Cores)	Backend-dependent (row-major activations typical)	Row-major typical	Layout choice depends on backend kernel path and precision mode (see section 11.8.1.2); the entries name common defaults, not universal prescriptions.
Kernel Fusion	Convolution + Activation	Fused Attention	GEMM Fusion	CNNs optimize convolution+activation fusion; Transformers fuse attention mechanisms; MLPs benefit from fused matrix multiplications.
Tiling for Memory Efficiency	Spatial Tiling	Temporal Tiling	Blocked Tiling	CNNs tile along spatial dimensions; Transformers use loop blocking to improve sequence memory efficiency; MLPs use blocked tiling for large matrix multiplications.

This spatial regularity also enables aggressive fusion and tiling. For inference with frozen Batch-Norm statistics, compilers can fold or fuse convolution, normalization, and activation to avoid unnecessary intermediate writes. Training-mode BatchNorm requires reductions over batch statistics and does not have the same element-wise fusion contract. Spatial tiling partitions the feature map into subregions sized to fit within on-chip SRAM, so the fused kernel processes each tile from fast memory before moving to the next.

11.8.2.2 Transformer architectures (GPT-2/Llama)

Where CNNs are defined by weight reuse, transformers are defined by the memory pressure of the key-value (KV) cache. During attention computation, every query vector must access stored key and value pairs across the entire sequence length. As sequences grow, the full KV cache usually lives in HBM or device DRAM, while attention kernels tile active blocks through SRAM, registers, or shared memory. This access pattern motivates activation stationary execution: keep the currently used KV tiles close to compute while streaming queries through them, rather than repeatedly materializing large attention intermediates in external memory.

The memory traffic created by standard attention implementations explains why fused attention kernels, such as FlashAttention (T. Dao et al. 2022), can deliver large performance gains: by fusing the query-key dot product, softmax normalization, and value-weighted summation into a single kernel that tiles along the sequence dimension, these implementations avoid materializing the full attention matrix in main memory. This temporal tiling approach processes sequence blocks that fit within on-chip SRAM, substantially reducing HBM traffic while preserving the $\mathcal{O}(S^2)$ attention computation. Transformer mapping depends on the phase and shape: training and prefill GEMMs can be compute-bound, while low-batch autoregressive decode is commonly constrained by weight and KV-cache bandwidth.

11.8.2.3 Multilayer perceptrons and DLRM

MLPs present the most straightforward mapping problem because their computation reduces largely to dense GEMM.³⁹ Each fully connected layer multiplies an activation matrix by a weight matrix. The weight matrix is fixed across samples, so batching reuses each loaded weight across more activations and raises arithmetic intensity. A batch size of one provides little cross-sample weight reuse and often underutilizes wide matrix engines, while larger batches can move execution toward the compute-bound regime. Section C.1.4 derives the scaling that governs this sensitivity.

Because MLP layers are typically followed by activation functions and bias additions, GEMM fusion combines these steps into a single kernel, avoiding intermediate memory writes. Blocked tiling partitions the large matrix multiplications into sub-blocks sized for the accelerator’s shared memory, ensuring high cache utilization throughout computation. The simplicity of the MLP mapping,

³⁹ | **GEMM:** The operation $\mathbf{C} = \alpha\mathbf{AB} + \beta\mathbf{C}$ is the dense linear-algebra primitive that many deep-learning layers lower to. Optimized GEMM libraries such as cuBLAS and oneDNN use register blocking, vectorization, and hierarchical tiling to approach hardware limits on favorable shapes (NVIDIA 2024a; Intel Corporation 2021b). Modern AI accelerators are heavily specialized for GEMM-like tiles: Tensor Cores, systolic arrays, and matrix extensions all exist to accelerate this primitive, which is why GEMM performance is an important predictor of end-to-end throughput across architectures.

dominated by a single primitive with predictable access patterns, is precisely why hardware vendors optimize GEMM libraries so aggressively: gains in GEMM performance translate directly to MLP throughput. DLRM is not purely an MLP workload, however. Its large sparse embedding tables and feature-interaction stage can dominate capacity and memory traffic, so the dense mapping described here applies to its bottom and top MLP towers rather than to the entire model (Naumov et al. 2019).

11.8.3 Hybrid mapping strategies

The preceding architectural subsections treat each architecture in isolation, but real models rarely consist of a single layer type. A vision transformer, for example, combines a patch embedding stage, self-attention layers, and MLP blocks (Dosovitskiy et al. 2021). Those layers create different reuse patterns: the embedding stage can benefit from weight-stationary mapping, attention emphasizes activation movement and tiling, and MLP blocks demand blocked GEMM tiling and fusion. No single dataflow strategy is optimal across all these layers, so hardware mapping becomes hybrid and layer-specific.

Hybrid mapping addresses this heterogeneity by allowing the accelerator to switch strategies at layer boundaries. Each layer presents a different balance of compute intensity, data reuse, and memory access pattern, and the optimal mapping must shift accordingly (Sze et al. 2017). Rather than committing to one dataflow for the entire model, hybrid approaches select weight stationary execution for layers with high weight reuse, activation stationary execution for attention layers with large KV caches, and output stationary execution for layers where minimizing write traffic matters most.

Modern accelerators provide the architectural features needed to realize hybrid mapping in practice. TPU-style systolic arrays, NVIDIA GPUs, and tile-based accelerator designs expose different combinations of local memory, tensor layouts, fusion, and scheduling controls, allowing compilers and runtimes to choose layer-specific strategies rather than one global dataflow for the whole model (Jouppi et al. 2023; NVIDIA Corporation 2020a; Chen et al. 2018). These implementations require programmable memory hierarchies, efficient interconnects, and specialized execution pipelines, reinforcing the hardware-software co-design principle.

However, hybrid mapping remains a design-time optimization. In production workloads, execution conditions change dynamically due to varying input sizes, memory contention, and hardware resource availability. Machine learning compilers and runtime systems extend these static mapping choices by introducing dynamic scheduling, memory optimizations, and automatic tuning, ensuring that deep learning workloads operate efficiently across diverse accelerators and deployment environments.

11.9 Compiler Support

A single convolution can be implemented with dozens of valid tiling strategies, kernel variants, and memory layouts, most of which perform poorly on a given target. **Machine learning compilers** navigate this complexity by translating high-level dataflow representations into target-specific executable code. Compiling ResNet-50 for GPU inference illustrates the core compiler stages:

1. **Graph optimization** fuses repeated Conv2D-BatchNorm-ReLU patterns into single compound kernels, eliminating intermediate round-trips to HBM.
2. **Kernel selection** chooses specialized Tensor Core implementations for compatible matrix multiplications, exploiting the high arithmetic intensity of dense convolutions.
3. **Memory planning** determines activation lifetimes to safely overlap and reuse intermediate buffer allocations.
4. **Computation scheduling** pipelines memory copies with execution across independent hardware queues to hide data transfer latencies.

In this scenario, inference latency drops from 47 ms to 8 ms, a 5.9× improvement without altering the model’s mathematical output. These numbers reflect how graph rewrites, kernel selection, memory planning, and scheduling systematically apply kernel fusion (section 11.8.1.3) and memory-efficient tiling (section 11.8.1.4) to convert theoretical hardware FLOP/s into delivered throughput (Chen et al. 2018; NVIDIA 2024c).

This pipeline embodies the hardware-software co-design principle introduced in section 11.1. Machine learning compilers bridge high-level model specifications and low-level execution engines by restructuring computations, selecting hardware-specialized kernels, and planning tensor storage lifetimes (Chen et al. 2018). They build on classical compiler foundations while introducing tensor- and graph-level transformations.

11.9.1 ML compiler design

Machine learning compilers specialize conventional compilation techniques for programs expressed as tensor graphs and operator libraries. General-purpose compilers optimize scalar control flow, vector instructions, register allocation, and cache locality; ML compiler stacks operate at a higher semantic level, optimizing graph topology, tensor memory layouts, shape specialization, accelerator kernel dispatch, and inter-device execution planning (Li et al. 2021).

Table 11.23 contrasts the emphasis of these two compiler layers. The boundary is complementary rather than disjoint: an ML compiler stack lowers high-level tensor operations into intermediate representations that feed into an LLVM, GPU, or vendor-specific backend.

Table 11.23: Compiler Optimization Priorities: ML compiler stacks add tensor- and graph-level decisions to conventional instruction- and loop-level compilation. In a production toolchain, the two layers commonly cooperate rather than replace one another.

Aspect	Traditional compiler	Machine learning compiler
Input representation	Source or intermediate-representation program	Tensor graph or tensor-level intermediate representation
Execution Model	Control flow, loops, threads, vectors, and tasks	Tensor operators, kernels, graphs, and accelerator streams
Optimization priorities	Instruction selection, loop transforms, vectorization, registers	Graph rewrites, fusion, layouts, shapes, and kernel selection
Memory management	Objects, stack/heap, caches, locality, and prefetching	Tensor lifetimes, buffer reuse, layouts, tiling, and device transfers
Target Hardware	CPUs, GPUs, and other programmable targets	CPUs, GPUs, TPUs, and custom accelerators
Compilation output	Machine code or lower-level intermediate representation	Kernels plus a hardware-specific graph or execution plan

The distinction in table 11.23 explains why compiler configuration dictates delivered performance even when the underlying neural network code remains unmodified. ML compilers manage the translation layer between high-level tensor operations and hardware execution units; when this mapping fails to optimize data reuse or layout alignment, arithmetic units stall while memory buses saturate.

Systems Perspective 11.2: The hidden optimization layer

Most practitioners never interact directly with ML compilers, yet compiler quality determines what fraction of peak accelerator throughput a model achieves. Calling `model.compile()` in Keras, `torch.compile()` in PyTorch, or deploying an engine through TensorRT invokes multi-stage optimization pipelines that execute four transformations:

- **Fuse operations** never combined by the developer (such as merging Conv2D, BatchNorm, and ReLU into a single kernel)
- **Reorder computations** to maximize data reuse in fast local memory (such as tiling matrix multiplications)
- **Select kernels** from libraries containing hundreds of hand-tuned, architecture-specific implementations
- **Transform tensor layouts** between what the framework API expects and what hardware memory controllers require

The same model definition runs at vastly different throughputs depending on the compilation backend, graph lowering choices, kernel selection, and runtime configuration. When execution latency exceeds service targets, compiler configuration and backend lowering are primary optimization levers that require no alterations to model architecture or training parameters.

11.9.2 ML compilation pipeline

Machine learning models defined in modern frameworks are represented as high-level dataflow graphs describing operations on multidimensional tensors. Lowering these representations into executable code for CPUs, GPUs, TPUs, or custom ASICs requires an ML compilation pipeline (Chen et al. 2018; Google 2025; Lattner et al. 2020).

A standard compilation pipeline executes five interconnected responsibilities:

1. Graph optimization restructures the computation to eliminate redundant work and reduce data traffic.
2. Kernel selection maps mathematical operations to optimized target-specific implementations.
3. Memory planning assigns tensor layouts, static buffer offsets, and lifetime bounds.
4. Computation scheduling orders kernels and memory transfers to maximize parallelism and hide transfer latencies.
5. Code generation emits target-specific machine instructions, assembly, or interpreted bytecode.

Across these stages, the compiler applies the dataflow strategies developed in section 11.8: kernel fusion, memory tiling, and compute placement. Hardware acceleration delivers its theoretical throughput only when these compiler-driven transformations expose sufficient arithmetic intensity and align memory access with the hardware hierarchy.

11.9.3 Graph optimization

Machine learning frameworks represent models as high-level computation graphs, where nodes represent mathematical operators (such as convolutions, matrix multiplications, and activations) and directed edges represent tensor data dependencies. Executing this graph node-by-node, as defined by framework frontends, incurs severe memory penalties: every edge forces an allocation and an off-chip memory transaction (DRAM or HBM write followed by a read), while every node incurs kernel launch and synchronization overhead. On accelerators where arithmetic throughput outpaces memory bandwidth by an order of magnitude, materializing unneeded intermediate tensors dominates total execution time.

Graph optimization transforms this computation graph into an efficient execution plan before hardware code generation (Chen et al. 2018; Jia et al. 2019). Working primarily at a target-independent intermediate representation (IR) level before lowering (section 11.9.6.2), graph passes apply four core transformations:

Operator fusion merges adjacent operations into a single compound kernel to eliminate off-chip memory round-trips and launch overheads. In convolutional networks, fusing Conv2D, batch normalization, and ReLU into one kernel keeps activations in local registers or shared SRAM throughout the sequence (section 11.8.1.3). In transformer architectures, pattern matchers replace multi-operator self-attention subgraphs (matrix multiplication, scaling, masking, softmax, and projection) with monolithic, IO-aware attention kernels that tile computations entirely within fast on-chip memory (T. Dao et al. 2022).

Redundant computation elimination and constant folding identify and remove unnecessary operations. During inference compilation, parameters from static layers—such as batch normalization scale and offset vectors—are folded directly into the weights and biases of preceding linear or convolutional layers, eliminating normalization operators from runtime execution. Graph passes also prune dead subgraphs and merge identical parallel paths through common subexpression elimination.

Algebraic simplification and layout adjustments reorder operations where mathematically equivalent to minimize data movement. For example, transpositions can be pushed through elementwise operations or cancelled against inverse permutations, reducing runtime reshape overhead.

Memory-aware node reordering evaluates data dependencies across topological sorts of the graph to schedule independent operations in an order that minimizes peak live-tensor footprint. Reordering nodes ensures that intermediate activations are consumed and deallocated promptly, preventing memory pressure from forcing spills to host memory.

Compilers such as XLA, TVM, TensorRT, and MLIR execute these rewrites via pattern-matching rules and equality saturation. By eliminating unnecessary memory round-trips and exposing coarse-grained parallelism, graph optimization produces an execution graph that aligns with the accelerator’s bandwidth constraints. Once the graph is restructured into optimized compound operations, the compiler must bind each node to a concrete, hardware-specific execution routine.

11.9.4 Kernel selection

Kernel selection binds each node in the optimized graph to a concrete hardware implementation. Accelerators provide dozens of valid kernel implementations for a single tensor operation, each parameterized by tile dimensions, thread block configurations, memory staging patterns, and target precision. Choosing the optimal kernel determines whether the execution engine achieves high compute utilization or stalls on uncoalesced memory accesses and pipeline bubbles (Chen et al. 2018; Zheng et al. 2020).

In most production pipelines, compilers draw from vendor-optimized kernel libraries rather than synthesizing kernels from scratch. NVIDIA’s cuDNN and cuBLAS provide hand-tuned routines for GPUs, Intel’s oneDNN targets x86 architectures, the Arm Compute Library (ACL) targets Arm processors, and BLIS provides optimized CPU matrix primitives. These libraries encode microarchitectural details—such as register file capacity, cache hierarchy latency, and vector lane width—allowing the compiler to invoke pre-tuned implementations.

AI compilers employ three primary strategies to select among candidate kernels:

Rule-based selection relies on static heuristics⁴⁰ to map operators based on tensor shapes, data types, and hardware targets. For example, XLA selects Tensor Core-enabled GEMM kernels on NVIDIA GPUs whenever matrix dimensions are multiples of 8 or 16 and precision is set to FP16 or BF16. Heuristics incur zero compilation time overhead, though they can miss non-obvious optimal configurations on irregular tensor shapes.

Cost model-based selection estimates kernel execution latency and memory traffic using analytical models of the hardware architecture. Frameworks such as MLIR provide infrastructure for cost models that predict register pressure, cache hit rates, and arithmetic intensity across parameter spaces, pruning unpromising candidates without running physical benchmarks (Lattner et al. 2020).

Profile-guided selection evaluates candidate kernels empirically on the target hardware. TVM’s AutoTVM and Ansor systems measure execution latency directly on the accelerator, navigating tile sizes, loop unrolling factors, and thread configurations via statistical search to converge on the fastest implementation (Chen et al. 2018; Zheng et al. 2020). Similarly, TensorRT builds hardware-specific execution engines by benchmarking tactic candidates directly against target batch sizes and memory constraints.

Precision-aware selection integrates numerical requirements into the selection process. While training requires FP32 or BF16 to maintain numerical stability during gradient accumulation, inference pipelines leverage FP16 or INT8 to maximize arithmetic throughput and halve memory traffic. A selection error here impacts correctness as well as speed: routing an uncalibrated model to an INT8 kernel degrades model accuracy, while defaulting to FP32 leaves Tensor Core engines underutilized.

Once kernels are selected, memory planning and computation scheduling determine exact buffer allocations, launch sequences, and stream concurrency.

11.9.5 Memory planning

Memory planning assigns physical storage locations, buffer offsets, and tensor lifetimes across the accelerator’s memory hierarchy (Roesch et al. 2018; Chen et al. 2018). High-performance execution plans fail if memory management introduces uncoalesced memory transactions, bank conflicts, or unnecessary memory allocations that exhaust accelerator memory.

The compiler coordinates three core memory management mechanisms:

⁴⁰ | **Heuristic in kernel selection:** A practical rule that chooses a promising implementation without exhaustively benchmarking the full candidate space. Tile sizes, layouts, precision modes, fusion choices, and shape constraints can create many legal GEMM variants. Heuristics reduce tuning cost but may miss a faster candidate, which is why autotuners such as AutoTVM profile selected options on the target hardware.

Tensor layout optimization determines the physical arrangement of tensor dimensions in memory. Different hardware architectures and compute engines require specific memory layouts to enable coalesced global memory access and vectorization (section 11.8.1.2). For example, NVIDIA Tensor Cores achieve peak throughput with the channels-last NHWC layout for FP16 and INT8 convolutions, where channel dimensions align with 16-byte memory transactions, whereas legacy FP32 CUDA cores historically favored channels-first NCHW (NVIDIA Corporation 2021b; Google 2025). Compilers perform layout assignment globally across the graph to prevent back-to-back layout transformation kernels that consume memory bandwidth.

Liveness analysis and buffer reuse prevent out-of-memory errors by sharing physical memory across non-overlapping tensors. Deep neural networks generate dozens of intermediate activation tensors during forward propagation. Allocating a unique memory buffer for every tensor would quickly exceed the capacity of device memory (such as 16 to 80 GB on typical accelerator GPUs). The compiler analyzes the graph to compute the liveness interval of each tensor—from its generation to the completion of its last consumer. By modeling memory allocation as an interval graph coloring or 2D bin-packing problem, the compiler assigns multiple non-overlapping intermediate tensors to the same physical memory offsets (Roesch et al. 2018). This static reuse reduces peak device memory requirements to the working set size of concurrently active tensors, avoiding runtime allocation overhead and fragmentation.

Memory hierarchy placement optimizes the staging of data between external memory (HBM or GDDR) and on-chip scratchpad memories (SRAM or shared memory). External memory bandwidth is a scarce resource; moving intermediate tiles back and forth across the memory bus creates memory-bound stalls. Compilers synthesize tiling parameters that guarantee intermediate blocks remain resident in on-chip SRAM until all local compute operations complete, minimizing round-trip traffic across the memory bus.

With memory layouts fixed and buffer addresses assigned, the compiler schedules operation execution across hardware compute resources.

11.9.6 Computation scheduling

Computation scheduling determines the execution timeline and hardware resource binding for operations in the compiled graph (Zheng et al. 2020). Modern accelerators feature deep hardware pipelines, multiple independent compute engines, and dedicated DMA copy engines. Effective scheduling coordinates these resources to maximize hardware occupancy, preserve data dependencies, and prevent pipeline stalls (Chen et al. 2018).

11.9.6.1 Implementation in AI compilers

Compilers implement scheduling across three distinct architectural dimensions:

Task partitioning maps computational subgraphs onto the target’s physical execution units. On GPUs, computations are partitioned into grids of thread blocks dispatched across Streaming Multiprocessors (SMs). On TPUs, matrix multiplications are partitioned into tile sequences matched to the physical dimensions of systolic arrays (e.g., 128×128 or 256×256 processing elements) (Norrie et al. 2021). On CPUs, tasks are split into thread pools matching physical cores and vectorized to fill SIMD registers.

Execution ordering resolves dependencies while minimizing live memory pressure. Operations that depend on prior results must wait for producer kernels to complete, but independent branches—such as multi-head attention projections or parallel residual paths—can execute concurrently or interleave to maintain uniform compute utilization. Compilers analyze the dependency graph to schedule operations in an order that keeps compute pipelines full while keeping activation lifetimes short, preventing buffer bloat.

Asynchronous overlap hides memory latency by executing data transfers concurrently with compute kernels. Accelerators feature dedicated DMA engines that operate independently of arithmetic pipelines. Compilers exploit this architectural separation by scheduling memory transfers (such as host-to-device weights or inter-accelerator halo exchanges) in separate hardware queues or CUDA streams, overlapping transfer latency behind long-running compute operations (Chen et al. 2018; NVIDIA 2024c). When compute and transfer overlap completely, data movement costs are hidden behind arithmetic execution.

Poor scheduling introduces execution bubbles where arithmetic units sit idle waiting for kernel dispatches or serialized data copies. Synchronizing kernels via explicit compiler-generated events and stream schedules ensures the hardware sustains continuous execution without CPU driver intervention (NVIDIA 2024c; Zheng et al. 2020).

11.9.6.2 Code generation

With scheduling complete, the final compilation stage translates the scheduled execution plan into target-specific machine instructions. Code generation applies traditional backend compiler passes—instruction selection, register allocation, instruction scheduling, and peephole optimization—tailored to accelerator architectures.

Crucially, instruction selection for ML targets must emit instructions that directly engage specialized matrix ISA extensions. On NVIDIA GPUs, code generators emit PTX instructions such as `mma.sync.aligned` to invoke Tensor Cores directly, as shown in listing 11.14. On Intel CPUs with Advanced Matrix Extensions (AMX), the backend targets tile-multiply instructions operating on 2D register tiles. On Arm architectures with the Scalable Matrix Extension (SME), the target is outer-product accumulation across scalable vector tiles. A backend that emits generic floating-point vector instructions instead of these matrix extensions leaves the primary compute engines unutilized, reducing effective throughput by an order of magnitude.

For CPUs and GPUs, compilers emit machine code or optimized assembly; for TPUs, field-programmable gate arrays (FPGAs),⁴¹ and proprietary NPU, the output is often an optimized bytecode or binary execution package executed by a specialized hardware runtime.

11.9.7 From compilation to runtime

A compiler transforms a high-level model into an execution plan tailored to target hardware, but that plan embeds static assumptions about runtime conditions: tensor shapes, workspace allocation limits, device capabilities, and execution concurrency. Ahead-of-time (AOT) compilation optimizes graph structures, fusion boundaries, and memory layouts for fixed shapes and known hardware configurations. While just-in-time (JIT) compilation can compile specialized variants dynamically as new shapes arrive, each compiled kernel still executes a fixed execution plan.

Production AI serving environments, however, operate under dynamic constraints that differ from static compiler assumptions. Request arrival rates fluctuate, batch sizes vary dynamically, multiple inference workloads compete for shared accelerator memory, and thermal throttling can reduce clock frequencies below rated specifications. Serving runtimes manage these variations through dynamic request batching, multi-stream concurrency, admission control, and execution engine selection, rather than regenerating kernel fusion or tiling strategies on the fly. The serving chapter addresses dynamic batching, admission control, and service-level objectives as system-level engineering challenges (chapter 13); the immediate systems concern is how the accelerator runtime dispatches compiled execution plans efficiently once a batch reaches the hardware.

11.10 Runtime Support

An AI runtime bridges static graph compilation and the dynamic realities of accelerator execution. Compilers optimize and lower computational graphs under fixed structural assumptions, but production systems encounter variable batch sizes, fluctuating sequence lengths, and constrained memory budgets. The runtime executes compiled engines while managing input-dependent tensor shapes, scratchpad allocations, asynchronous hardware streams, and precompiled execution profiles. In production inference stacks such as NVIDIA TensorRT, the offline builder optimizes the network and benchmarks candidate tactics to construct an execution engine, while the runtime engine binds input buffers, selects valid optimization profiles, and dispatches kernels to the accelerator (NVIDIA Corporation 2026a, 2026b).

This execution lifecycle spans three core responsibilities. First, the runtime selects and dispatches precompiled kernel variants matched to runtime tensor dimensions and hardware capabilities. Second, it manages device memory through arena allocators and static buffer reuse, eliminating costly host-device synchronization during execution. Third, in distributed environments, framework runtimes coordinate cross-device execution boundaries, as demonstrated in pipeline-parallel frameworks such as GPipe (Huang et al. 2019) and automated device-placement systems (Mirhoseini et al.

⁴¹ | **Field-programmable gate array:** “Field-programmable” means the logic fabric is configurable after manufacturing, contrasting with fixed-function ASICs. FPGAs can improve performance for latency-sensitive data center services by implementing custom pipelines matched to a particular workload (Putnam et al. 2014). This reconfigurability makes FPGAs attractive for rapidly evolving ML architectures where committing to an ASIC risks obsolescence, but the requirement for hardware description languages (Verilog/VHDL) and compilation times measured in hours creates a productivity barrier that limits adoption to deployments where the efficiency benefit justifies the engineering cost.

2017). While serving schedulers oversee request batching and admission control across clusters, the accelerator runtime governs local execution on the device.

Comparing AI runtimes to traditional software execution environments clarifies why machine learning workloads demand specialized execution strategies.

11.10.1 ML runtime architecture

General-purpose runtimes manage threads, preemptive tasks, heap allocations, and asynchronous I/O across arbitrary control-flow paths. In contrast, an AI runtime operates over dataflow graphs composed of multidimensional tensors, pre-scheduled kernel grids, device memory arenas, and hardware command queues.

The architectural differences between traditional software runtimes and AI execution environments are contrasted in table 11.24.

Table 11.24: Runtime Execution Models: AI runtimes add tensor-, graph-, and accelerator-aware resource management to conventional runtime services. The exact boundary among framework, compiler runtime, device library, and serving scheduler varies by stack.

Aspect	Traditional runtime	AI runtime
Execution Model	Functions, threads, tasks, events, and asynchronous I/O	Tensor graphs, kernels, streams, and device events
Task scheduling	Threads and tasks across processor resources	Dependency-aware kernel and transfer dispatch
Memory Management	Objects, stacks, heaps, pools, and virtual memory	Tensor buffers, workspaces, device memory, and buffer reuse
Optimization Priorities	Responsiveness, throughput, locality, and resource sharing	Shape compatibility, transfer overlap, reuse, and utilization
Adaptability	Dynamic scheduling and allocation within program semantics	Runtime choices within compiled graphs and available kernels
Target Hardware	CPUs and heterogeneous devices	CPUs, GPUs, TPUs, and custom accelerators

Tensor memory management represents a primary architectural divergence. In traditional systems, general-purpose allocators manage variable-sized heap objects using runtime metadata. On AI accelerators, dynamic memory allocations through device-driver calls (such as `cudaMalloc`) invoke host-driver traps that stall the accelerator pipeline. AI runtimes eliminate this overhead by combining offline lifetime analysis with arena allocation. For fixed-shape graphs, the compiler computes static memory offsets, mapping non-overlapping intermediate tensors to shared physical buffers. For variable-shape tensors, the runtime carves memory from preallocated scratchpad pools, bounding peak memory footprint without runtime allocation calls.

Hardware execution environments also impose physical operating boundaries that diverge from offline benchmarking setups.

Production environments introduce physical constraints that differ from offline benchmarks. For example, an NVIDIA A100 SXM accelerator operates with a maximum TDP of 400 W, whereas the A100 PCIe variant is constrained to a 300 W TDP envelope. Because sustained clock frequencies scale with available power and thermal dissipation, a compiled kernel tuned for an SXM module may experience thermal throttling or fail to meet latency bounds on a PCIe card. The runtime cannot re-synthesize loop structures or change thread-block dimensions dynamically in response to thermal constraints; it must execute within precompiled profiles while the serving system handles request batching and admission control.

11.10.2 Dynamic kernel execution

While static compilation generates high-performance machine code for fixed shapes, production inference requires executing requests with dynamic dimensions. In language models, input sequence length S and batch size B vary across requests. Recompiling a computational graph dynamically when an unexpected shape arrives is impractical: exploring loop unrolling, shared-memory tiling, and register allocation requires seconds or minutes, while serving deadlines operate on millisecond timescales.

AI runtimes resolve this mismatch using precompiled optimization profiles. An optimization profile establishes valid dimension bounds (minimum, optimum, and maximum shapes) for dynamic tensor axes. During engine construction, the builder allocates scratchpad workspace sized for the maximum shape and selects kernel tactics that maximize arithmetic intensity across the defined operating range. At execution time:

- When incoming tensor shapes match an active profile, the runtime binds tensor pointers to preallocated buffers and launches the corresponding precompiled kernels without JIT overhead.
- When input dimensions fall outside all compiled profiles, the runtime cannot synthesize new tiling strategies on the fly; the serving system must either pad inputs to a valid dimension boundary, split the batch, or route the request to a slower, general-purpose fallback kernel.

Dynamic request processing also strains the host-to-accelerator interconnect. If input transfers proceed synchronously, accelerator compute engines sit idle during data movement. To overlap communication with computation, the runtime employs asynchronous streams and double buffering. By staging input tensors in page-locked (pinned) host memory, dedicated DMA copy engines transfer batch $n+1$ across PCIe or NVLink concurrently with the execution of batch n on accelerator cores. This pipelining hides transfer latency provided the compute duration exceeds the transfer duration; if memory movement dominates, the accelerator pipeline stalls.

11.10.3 Runtime kernel selection

Within a selected optimization profile, an operation can be executed by multiple precompiled kernel implementations, termed tactics. The runtime selects the optimal tactic based on tensor dimensions, arithmetic intensity, and available scratchpad memory.

Operational batch size directly dictates the optimal execution strategy. For small batch sizes ($B = 1$ during autoregressive token generation), matrix-vector multiplication is memory-bandwidth bound, with low arithmetic intensity. The runtime selects memory-streaming kernels (GEMV) optimized for global memory read throughput. For large prompt-processing batches ($B \geq 64$), the workload transitions to compute-bound GEMM. The runtime dispatches tactics designed for Tensor Core systolic arrays, using 2D register and shared-memory tiling to maximize data reuse from on-chip SRAM.

Scratchpad workspace availability further constrains tactic selection. When a GEMM has a small output dimension but a large reduction dimension K , standard tiling underutilizes the accelerator because too few thread blocks are launched to occupy all Streaming Multiprocessors (SMs). In this regime, the builder provides a split- K tactic that partitions the reduction dimension across separate thread blocks running concurrently on different SMs. However, split- K execution requires an auxiliary workspace buffer in device memory (HBM) to store partial reduction sums before a final accumulation pass. If device memory is constrained by concurrent requests, the runtime must reject the split- K variant and dispatch a non-split kernel, sacrificing parallel occupancy to respect the memory ceiling.

Runtime selection remains strictly bounded by the numerical formats established during compilation. In mixed-precision training and inference systems such as Megatron-LM, executing matrix multiplications in FP16 or BF16 on Tensor Cores with FP32 accumulation is an architectural invariant validated prior to deployment (Shoeybi et al. 2019). The runtime does not monitor loss values or dynamically alter precision; it only chooses among the validated tactics that implement the specified numerical precision.

11.10.4 Kernel scheduling and utilization

Kernel dispatch represents the final stage of runtime execution, coordinating host-side task queues with device-side execution.

The software runtime manages host-side command queues (such as CUDA streams or TPU command rings) and records hardware events or semaphores to enforce dataflow dependencies asynchronously. When independent operations—such as parallel multi-head attention projections—exhibit no mutual data dependencies, the runtime dispatches them onto separate streams to enable concurrent execution on the accelerator. Once submitted to the device queue, however, execution

control transfers entirely to hardware schedulers. On a GPU, the hardware warp scheduler and work distribution engine assign thread blocks to SMs based on register file availability, shared memory capacity, and maximum warp occupancy; the software runtime cannot schedule individual threads or dictate L1/L2 cache replacement policies. Similarly, TPU runtimes submit macro-instructions to on-chip command queues, while hardware sequencers feed systolic matrix units and vector units (Jouppi et al. 2017).

Memory management directly reinforces accelerator utilization. Calling device-driver memory allocators (such as `cudaMalloc`) during request execution forces global device synchronization, draining the execution pipeline. AI runtimes eliminate this latency by maintaining preallocated memory arenas and managing tensor lifecycles statically. Intermediate activations are allocated from scratchpad memory with overlapping lifetimes mapped to identical physical addresses, ensuring zero-allocation overhead on the inference hot path.

Even with aggressive kernel fusion, optimal tactic selection, and zero-allocation memory reuse, the execution capacity of a single accelerator is physically bounded by its silicon area, thermal dissipation limit, and high-bandwidth memory capacity. Scaling beyond these physical boundaries requires distributing computation across multiple accelerators.

Consider the estimated 3.14×10^{23} floating-point operations associated with training GPT-3 (Brown et al. 2020). As a scale comparison, divide that historical operation count by a later H100's peak FP8 rate of 1.98 PFLOP/s. The arithmetic lower bound is about 5 years on one device, and 8.4–12.6 years under the illustrative 40–60 percent utilization range (Choquette 2023). This is not a reconstruction of GPT-3 training: the precision, hardware, communication, and workload are counterfactual. It simply shows why model-scale compute or service-scale request volume can exceed the useful capacity of one accelerator.

11.11 Multi-Chip Scaling

A single H100 has a large FP8 peak, yet a training-time or throughput target may still require many accelerators. The techniques covered earlier remain the foundation for multi-chip scaling; each device still needs an efficient dataflow. The new lesson is that communication adds further hierarchy boundaries. Moving data through on-chip storage, HBM, an intra-node link, and a cluster fabric generally increases latency and energy, so scaling is not merely a question of adding chips.

When single-accelerator capacity proves insufficient, the design problem shifts from feeding one chip to choosing which communication boundary the workload can tolerate. Practitioners encounter these boundaries in production even when most optimization work remains inside a single accelerator.

11.11.1 Multi-chip scaling approaches

Large AI systems scale beyond individual accelerators by moving the communication boundary outward, and each boundary changes the dominant trade-off. The sequence begins inside the package. Chiplet-based architectures partition large designs into smaller, modular dies interconnected within one package, bypassing manufacturing limits of monolithic chips while preserving relatively low communication latency. The next boundary is the node: multi-accelerator servers connect several chips through board- or server-level interconnects. Each accelerator has dedicated memory and compute resources, so workloads split through data parallelism (each accelerator processes different batches) or model parallelism (either pipeline parallelism, where different accelerators handle sequential network layers, or tensor parallelism, where individual matrix multiplications are sliced across devices within high-bandwidth interconnect domains). High-bandwidth intra-node interconnects can enable efficient gradient synchronization, though realized performance depends on topology and collective communication efficiency.

Beyond the node, the boundary expands into the cluster. Purpose-built data center fabrics coordinate hundreds of accelerators, making topology and collective communication algorithms central determinants of scaling efficiency; near-linear scaling is achievable on some workloads when communication overhead is controlled. Wafer-scale integration is the counter-move: instead of pushing the boundary outward, it collapses more computation back into one large device. Platforms such as Cerebras WSE-class systems integrate extremely large numbers of transistors and cores on a

single device, reducing inter-chip communication overhead while introducing their own challenges in thermal dissipation, fault tolerance, and manufacturing yield.

11.11.2 Why scaling introduces new constraints

The transition from single-chip to multi-chip architectures introduces communication overhead and other constraints that reshape optimization. Communication, synchronization, load balance, and parallel work per device can all limit scaling. Amdahl's Law⁴² provides one first-order model for exposed work that does not shrink with additional devices. For hundred-billion-parameter models, an unsharded full-gradient payload can occupy hundreds of gigabytes per step, and an AllReduce⁴³ must aggregate it; precision, sharding, compression, and the collective algorithm change the bytes each device transfers.

The first-order quantity is the gradient payload. For a model with P parameters, that payload is roughly the parameter count multiplied by the bytes stored for each gradient element before optimizer state, padding, and protocol overhead enter. Scaling therefore improves only when the saved compute time exceeds the time to move this payload through the chosen interconnect and collective algorithm.

This overhead explains why large accelerator counts can show diminishing returns unless the system reduces exchanged data, overlaps communication with useful computation, or chooses a better parallelization pattern. The memory model also changes: separate accelerator memories are commonly managed explicitly, and collective operations establish the required synchronization rather than presenting one fully coherent memory image across the cluster. Coherent links exist in some systems, but their scope and cost are architecture-dependent.

Once computation spans many links, chips, and memory stacks, reliability and energy become part of the same scaling story. Large-scale systems must handle component failures gracefully because the probability of at least one failure rises with system size. The architectural principle is fundamental: distributed TPU systems must tolerate component failures at system scale (Jouppi et al. 2023), while Cerebras uses redundant cores and fabric links to replace manufacturing-defective cores and restore the wafer's logical mesh (Lie 2021). Data movement also grows more expensive with distance, transforming distributed training into a careful balance between computation parallelism and communication efficiency.

Data center scaling and edge deployment represent opposite ends of a deployment spectrum, yet they share the same core principles. Data center scaling coordinates many high-throughput accelerators, while edge scaling fits useful AI into a few constrained watts. Both cases require matching workload characteristics to hardware capabilities while minimizing data movement. The principles of compute specialization, memory hierarchy optimization, and workload mapping apply at both scales; only the constraints differ. Data centers optimize for aggregate throughput within power budgets measured in megawatts; edge devices optimize for responsiveness within tight battery and thermal envelopes. The same vision model that runs comfortably in a data center may need a radically different mapping strategy on a smartphone or microcontroller.

11.12 Heterogeneous SoC Design

Edge, mobile, and automotive deployments operate under strict physical constraints: sealed enclosures without active fans limit thermal dissipation to a few watts, while battery capacity caps total energy consumption. To deliver machine learning capabilities within these boundaries, heterogeneous SoC architectures integrate general-purpose CPU cores, GPU shader arrays, digital signal processors (DSPs), and specialized neural processing units (NPUs) onto a single silicon die sharing a unified memory subsystem. The workload mapping problem introduced for discrete accelerators persists, but it expands to include hardware operator coverage, cross-engine transfer overheads, real-time deadlines, and dynamic thermal throttling.

11.12.1 Mobile SoC architecture evolution

Mobile system-on-chip architectures implement heterogeneous computing by coupling distinct execution engines—multicore CPUs, wide SIMD GPU shader cores, fixed-point DSPs, and dedicated neural processing units (NPUs)⁴⁴—over an on-chip network connected to shared low-power DRAM (LPDDR). Workload partitioning routes sensory and perceptual tasks to the compute engine best

⁴² | **Amdahl's law (scaling limit):** section D.2.3 formalizes the bound for a fixed nonaccelerated fraction. If 5 percent of a fixed workload remains exposed and unaffected by device count, that model caps speedup at $20\times$. Distributed training is more complicated because communication can overlap computation and its cost can itself change with scale, but the example explains why exposed synchronization must be reduced or hidden.

⁴³ | **AllReduce:** A collective operation from MPI that aggregates values across processes (the "reduce") and distributes the result back to every process (the "all"). Here, the term identifies why accelerator-to-accelerator bandwidth matters for distributed training workloads. At scale, algorithm, topology choices, and runtime protocols determine how costly this synchronization becomes.

⁴⁴ | **NPU (neural processing unit):** The NPU's specialized matrix engines are optimized for dense tensor operations, providing the hardware basis for the workload distribution described. This specialization creates a critical constraint for the scheduler: any AI operator not mapped to the NPU's supported data paths must "fall back" to a GPU or CPU. This fallback can erase the NPU's energy-efficiency advantage, complicate real-time latency budgets, and contribute to thermal pressure on mobile devices.

aligned with their arithmetic structure: continuous audio feature extraction and keyword spotting run on low-power DSPs, dense convolution and matrix-multiplication layers execute on systolic or tensor-array NPUs, parallel graphics and image postprocessing map to GPUs, and sequential control flow runs on the CPU. Meeting real-time frame deadlines without triggering aggressive thermal throttling requires scheduling execution across these engines while respecting their differing power profiles.



Example 11.5: Heterogeneous microcontrollers

Scenario: An embedded smart doorbell must run periodic person detection within a tight memory, energy, and latency budget.

Diagnosis: On a supported deployment, a general-purpose Cortex-M path misses the measured energy or latency target. A micro-NPU such as Arm Ethos-U can offload compatible convolution operators, while the CPU retains control flow and unsupported operations.

Systems lesson: Micro-NPU specialization can improve the energy and latency of supported operators, but feasibility must be measured for the whole pipeline, including fallbacks and sensor processing.

Modern mobile SoCs utilize unified memory architectures where all processing units share a common physical address space in LPDDR. Unlike discrete accelerator architectures that incur explicit PCIe transfer latencies, unified memory allows zero-copy buffer passing between the image signal processor (ISP), CPU, GPU, and NPU. However, sharing physical DRAM does not guarantee zero software overhead. If an NPU engine lacks hardware cache snooping, passing an intermediate tensor from the CPU requires explicit cache flushes and invalidations. Furthermore, specialized NPUs often require hardware-specific tensor layouts—such as channels-last (*NHWC*) with 16- or 32-byte channel alignment to feed vector register lanes—forcing memory-bound transpositions when interfacing with CPU layers structured in channels-first (*NCHW*) formats.

SoC architects customize these compute blocks according to domain-specific physical constraints. High-end smartphone SoCs emphasize peak burst performance and low idle leakage for interactive tasks such as real-time computational photography and on-device text generation. In contrast, embedded and automotive SoCs prioritize deterministic memory access, hardware fault detection, and bounded worst-case latency to satisfy real-time control constraints.

11.12.2 Strategies for dynamic workload distribution

Partitioning neural network workloads across heterogeneous SoC cores requires matching the arithmetic intensity and memory regularity of each computational graph operator to the microarchitecture best equipped to execute it. Consider a real-time object detection pipeline executing on a mobile SoC with an integrated CPU, GPU, and NPU. The pipeline comprises three consecutive stages: a MobileNet backbone for feature extraction, nonmaximum suppression (NMS) for postprocessing candidate bounding boxes, and a display overlay to render final detections onto a camera feed. The convolutional backbone exhibits regular, deterministic memory access and dense arithmetic suitable for the NPU's fixed-function tensor engine, even though depthwise separable layers exhibit lower arithmetic intensity than standard convolutions. NMS, by contrast, relies on score-based sorting and conditional intersection-over-union loops evaluated across variable-length candidate lists. This irregular, branch-heavy control flow maps poorly to an NPU's spatial array but executes efficiently on out-of-order CPU cores with branch prediction and hardware caching. Finally, the display overlay requires per-pixel rasterization and alpha blending across a high-resolution frame—an embarrassingly parallel, memory-bandwidth-bound task that maps directly to the GPU's rasterizers and texture units. Splitting the pipeline across the three engines—NPU for feature extraction, CPU for NMS, and GPU for display compositing—minimizes end-to-end latency and total energy consumption compared to executing the entire pipeline on any single processor.

This pipeline partitioning reflects the broader relationship between algorithmic structures and machine architectures. Regular matrix contractions and convolutions map cleanly to systolic or vectorized NPU datapaths, whereas data-dependent control flow maps to CPU execution resources. Transformer attention layers span both paradigms: the projection GEMMs (Q, K, V, O) and attention

score contractions (QK^T and AV) execute efficiently on NPU matrix units, but intermediate softmax reductions, causal masking, and layer normalizations involve elementwise operations. If the NPU runtime cannot fuse these elementwise operations into on-chip SRAM scratchpads, intermediate tensors must spill to LPDDR, eroding the accelerator's throughput advantage.

Optimal processor assignment is dynamic rather than static. As device thermal conditions deteriorate or battery reserves drop, the runtime must adapt its scheduling policy. However, migrating an executing model across processor backends carries non-trivial overhead: weights must be re-bound in memory, intermediate buffers may require layout reformatting, and device drivers incur kernel dispatch latency. If the overhead of synchronizing state across processors exceeds the compute savings of offloading, retaining execution on a single suboptimal engine can yield lower total latency and energy than continuous cross-engine migration.

Shared memory subsystems introduce cross-engine interference during simultaneous execution. Because the CPU, GPU, NPU, camera ISP, and display engine share a single LPDDR memory bus, the SoC memory controller enforces Quality of Service (QoS) priorities to satisfy real-time hardware deadlines. The display engine and camera ISP must receive guaranteed memory bandwidth and bounded latency to prevent display flicker and sensor FIFO overflow; the memory controller consequently prioritizes display and camera transactions over NPU and GPU traffic. During memory-intensive phases—such as autoregressive language model decoding where model weights must be read from DRAM for every generated token—high-priority ISP or display traffic reduces effective NPU memory bandwidth, increasing token generation latency.

11.12.3 Power and thermal management

Fanless mobile devices operate within strict continuous power budgets and thermal envelopes, typically dissipating no more than 3 to 5 watts sustained across the entire chassis before touch temperatures exceed safety thresholds. Although transient bursts can draw 10 to 15 watts, continuous inference rapidly saturates the thermal capacitance of the device, heating the silicon junction and requiring active power management across the SoC.

Heterogeneous SoCs manage power across distinct silicon partitions using **dynamic voltage and frequency scaling (DVFS)**. Because dynamic power scales quadratically with supply voltage and linearly with operating frequency ($P_{\text{dyn}} \propto V^2 f$), dropping the clock frequency allows lowering the core voltage, yielding substantial power reductions. On a heterogeneous SoC, CPU clusters, GPU shader cores, and NPU engines occupy separate power and voltage domains. Increasing the voltage and frequency of the NPU to meet a tight latency deadline consumes electrical current and thermal headroom that would otherwise be available to the CPU or GPU. System power governors continuously adjust these operating points based on workload queues, temperature sensor readings, and frame deadline targets.

When sustained computation exceeds the chassis dissipation rate, the platform initiates thermal throttling. Because specialized NPUs achieve higher energy efficiency (TOPS/W) for tensor math than general-purpose cores, offloading matrix workloads to a CPU or GPU during a thermal event often increases total energy dissipation, exacerbating thermal accumulation. Instead, runtimes manage thermal limits by capping clock frequencies, reducing inference duty cycles (such as throttling a perception model from 30 frames per second to 15 frames per second), or dynamically switching to a smaller, pruned model checkpoint. Task migration is applied selectively to move non-tensor preprocessing away from localized silicon hot spots on the die.

Battery state further shapes runtime scheduling. When battery charge drops below critical thresholds or the device enters a low-power mode, the system scheduler trades inferencing fidelity for battery life by quantizing operations to lower bit widths (such as transitioning from 16-bit floating-point to 8-bit integer precision), increasing execution sparsity, or batching asynchronous inference requests to maximize the time cores spend in deep low-power sleep states.

11.12.4 Automotive heterogeneous AI systems

Automotive computing platforms operate under a dual mandate: delivering hundreds of tera-operations per second (TOPS) for multi-camera perception and sensor fusion while satisfying strict functional safety standards (such as ISO 26262). Unlike mobile systems where thermal throttling gracefully degrades user experience, automotive systems operate under hard real-time deadlines

where missing a frame deadline can compromise vehicle control. However, not all in-vehicle AI workloads require the highest safety integrity; infotainment and driver-comfort models share compute platforms with safety-critical perception pipelines.

To support this mixed-criticality execution, automotive SoCs enforce freedom from interference between safety-critical workloads (such as automated emergency braking) and non-critical applications (such as natural-language cabin assistants). Hardware-enforced virtualization, input-output memory management units (IOMMUs), and memory bus bandwidth reservations isolate compute cores and LPDDR channels. Without memory bandwidth reservations, an unverified convenience model streaming weights from DRAM could saturate memory channels, delaying frame delivery to an emergency perception kernel with a certified worst-case execution time (WCET). Real-time operating systems schedule critical neural network pipelines using time-triggered dispatching rather than average-throughput priority queues.

Automotive architectures also address end-to-end latency across distributed and centralized sensor topographies. Autonomous platforms ingest high-bandwidth data streams simultaneously from high-resolution cameras, automotive radar, and lidar. Ingesting, packetizing, and transporting this data across automotive Ethernet (such as 10GBASE-T1) or serial deserializer links (SerDes) introduces transport latency and jitter. Sensor fusion algorithms require hardware-timestamped data streams synchronized to a shared microsecond clock (via IEEE 802.1AS). Accelerators cannot begin cross-sensor fusion until the slowest sensor frame in a temporal window arrives, meaning sensor alignment, serialization overheads, and DMA ingest latencies must be budgeted alongside neural network inference time.

External vehicle-to-everything (V2X) inputs introduce non-deterministic communication latencies and variable packet delivery rates. Because wireless signals are subject to packet drops, multi-path fading, and adversarial spoofing, safety-critical perception algorithms cannot treat V2X inputs as synchronous sensor feeds. Instead, planning pipelines incorporate V2X payloads as asynchronous contextual hints, bounding their influence to ensure vehicle safety remains guaranteed by onboard deterministic sensors.

11.12.5 Software stack challenges

Heterogeneity shifts the primary systems challenge from silicon design to software coordination. Standard programming models like OpenCL and Vulkan provide portable low-level execution abstractions across compute devices, but they do not eliminate the need for microarchitecture-specific kernel tuning. Machine learning inference runtimes partition computation graphs through hardware-specific delegates or backends targeting CPUs, GPUs, DSPs, and NPUs. When a model contains an operator unsupported by an NPU delegate—such as a custom activation, dynamic tensor shape, or non-standard reduction—the runtime splits the graph into subgraphs, inserting fallback paths to the CPU or GPU. Each boundary crossing requires synchronizing execution and translating data representations, transforming an algorithmic graph into a fragmented sequence of hardware dispatches.

Shared memory subsystems require explicit cache and memory management across non-coherent execution domains. While CPUs maintain cache coherence across their own cores via hardware snooping protocols, mobile NPUs and DSPs frequently interface with system memory as non-coherent DMA bus masters. When the CPU writes input tensors to cached DRAM and hands control to an NPU, software must explicitly flush the CPU's dirty cache lines to DRAM before NPU execution begins. Similarly, upon NPU completion, the CPU must invalidate its own cache lines over the destination memory range to prevent reading stale cached data. If unsupported operators cause an inference graph to ping-pong repeatedly between CPU and NPU, the latency of cache maintenance and intermediate DRAM round-trips can exceed the execution time of the operators themselves.

Runtimes navigate this scheduling complexity using ahead-of-time compilation, empirical profile databases, and dynamic dispatch heuristics. Device telemetry—monitoring frame completion times, core temperatures, and battery drain—informs runtime decisions to throttle clock frequencies or swap execution delegates. However, dynamic adaptation cannot replace deterministic graph lowering for latency-sensitive applications; deployment requires validating the compiled, lowered partition on physical target silicon under worst-case thermal and memory contention.

Heterogeneous acceleration demonstrates that system efficiency is fundamentally a coordination problem rather than an inventory of specialized execution blocks. In the D·A·M taxonomy, an SoC represents a composite Machine layer whose components exhibit divergent memory hierarchies, arithmetic strengths, and thermal envelopes. The iron law of ML systems dictates that overall runtime and energy consumption are governed by data movement: when software coordination fails to maintain tensor locality or induces frequent cross-engine fallbacks, energy dissipation is dominated by interconnect and DRAM traffic rather than arithmetic. Maximizing performance on heterogeneous SoCs requires minimizing data movement across domain boundaries while matching each algorithmic operation to the specialized datapath architected to accelerate it.

11.13 Hardware Sustainability

At fleet scale, energy per useful inference becomes an important hardware-selection criterion. Operational emissions depend on workload throughput, measured system power, utilization, cooling overhead, and the electricity mix; embodied emissions add another lifecycle cost. Peak performance per watt is therefore a screening metric, not a substitute for end-to-end energy per request. The following scenario shows how assumptions compound when comparing a generic-CPU fleet with specialized accelerators.



Napkin Math 11.10: An illustrative operational-energy comparison

Problem: Under a stated throughput-equivalence assumption, how does operational energy change when a CPU fleet is replaced by specialized NPUs (Neural Processing Units)?

Assumptions: This stylized comparison treats the listed peak rates as useful workload throughput, assumes both fleets run continuously at the listed power, and excludes host power, cooling, embodied carbon, and utilization differences. It is a sensitivity calculation, not a measured procurement result.

- **CPU inference:** 100 W for 1 TFLOP/s (efficiency = 0.01 TFLOP/s/W).
- **NPU inference:** 5 W for 10 TFLOP/s (efficiency = 2 TFLOP/s/W).
- **The assumed peak-efficiency gap:** 200×

Math:

1. **Workload:** 1 billion inferences per day.
2. **CPU fleet energy:** 1,000 CPU servers $\times$ 100 W $\times$ 24 h $\approx$ 2,400 kWh/day.
3. **NPU fleet energy:** 100 NPU chips $\times$ 5 W $\times$ 24 h $\approx$ 12 kWh/day.
4. **Carbon savings:** At 0.429 kg/kWh, switching to NPUs saves $\sim$ 373.9 t of CO₂ per year.

Systems insight: Specialized accelerators can reduce operational energy when the workload uses their efficient path and fewer devices deliver the required service throughput. Replace these assumptions with measured requests per second and wall power before using the result for procurement or carbon accounting.

The sustainability perspective reinforces a central theme: hardware selection is not determined by peak speed alone. Energy per useful result, carbon intensity, embodied impact, and total cost of ownership belong beside latency, throughput, and memory capacity. The remaining step is to identify the misconceptions that cause teams to choose the wrong hardware path.

11.14 Fallacies and Pitfalls

Hardware acceleration exhibits counterintuitive performance characteristics because headline specifications frequently mask workload-specific bottlenecks. The fallacies and pitfalls here capture selection, scaling, and optimization errors that leave expensive compute resources underutilized.

Fallacy: *More specialized hardware always provides better performance than general-purpose alternatives.*

Specialized accelerators achieve high efficiency only when workload access patterns match underlying execution datapaths: the core tenet of algorithm-machine co-design. Under the Roofline Model (section 11.6), an operation must exceed the hardware ridge point to become compute bound. An A100 GPU has a ridge point of 153 FLOP/byte, meaning operations with arithmetic intensity below this threshold remain memory bound regardless of the chip's 312 TFLOP/s peak compute. A transformer attention softmax with AI = 2 FLOP/byte–5 FLOP/byte achieves only 4.1 TFLOP/s–10.2 TFLOP/s (3.3 percent utilization) on an A100. CPUs with ridge points around 10 FLOP/byte–20 FLOP/byte also execute this kernel in the memory-bound regime, but that same arithmetic intensity saturates 10 percent–50 percent of the CPU's compute capacity. When workloads exhibit irregular memory access, tiny batch sizes, or dynamic control flow, massively parallel execution pipelines stall on memory latency and thread divergence. General-purpose CPUs with large cache hierarchies, out-of-order execution, and branch predictors often deliver superior latency and resource efficiency under these conditions. Hardware selection requires matching workload arithmetic intensity and dataflow regularity to architectural ridge points rather than assuming specialization guarantees speed.

Pitfall: *Ignoring memory bandwidth limitations when selecting acceleration strategies.*

Procurement decisions frequently target peak arithmetic throughput while overlooking memory interface constraints. Off-chip data movement imposes severe performance and energy penalties: fetching an operand from DRAM consumes roughly 640 pJ compared to 0.5 pJ for an on-chip L1 SRAM access (section 11.5.1). Consider an accelerator providing 300 TFLOP/s of compute alongside 2 TB/s of memory bandwidth, setting a ridge point of 150 FLOP/byte. A normalization kernel such as LayerNorm with an arithmetic intensity of 1.5 FLOP/byte achieves only 3 TFLOP/s (1 percent compute utilization). Because LayerNorm streams full activation tensors through memory to compute simple reduction statistics, memory bandwidth strictly caps throughput. Adding compute units without expanding memory bandwidth leaves runtime unchanged and wastes silicon area.

Fallacy: *Hardware acceleration benefits scale linearly with additional accelerators.*

Distributed training does not deliver $8\times$ throughput on eight accelerators because inter-chip communication overhead degrades parallel efficiency. Synchronizing gradients across ranks via ring AllReduce (section 11.11) requires each rank to transfer $2(N-1)/N$ times the model gradient volume: $1.75\times$ payloads across eight devices. With an interconnect such as NVLink providing 600 GB/s bidirectional bandwidth (half that per direction), transmitting a 1 GB gradient payload consumes 5.83 ms. For a model with a 50 ms compute step, this data exchange adds 11.7 percent communication overhead before counting software collective latency. Without compute-communication pipelining, an eight-accelerator node reaches only $7.2\times$ speedup (89.6 percent parallel efficiency). Stragglers, barrier synchronizations, and collective protocol overhead widen the gap between linear expectations and physical cluster scaling.

Pitfall: *Planning accelerator capacity from peak FLOP/s specifications.*

Peak FLOP/s measures arithmetic unit capacity under ideal instruction mixes and continuous operand availability, not delivered application throughput. Realized performance depends on kernel arithmetic intensity, matrix dimensions, memory coalescing, and runtime launch overhead. The Roofline Model (section 11.6) bounds the gap between peak compute and bandwidth-limited execution. On an A100 GPU with a 312 TFLOP/s peak, dense transformer training sustains 120 TFLOP/s–180 TFLOP/s (40 percent–60 percent model FLOPs utilization). Conversely, a sparse recommender model sustains only 10 TFLOP/s–30 TFLOP/s (3.2 percent–9.6 percent utilization) because sparse embedding table lookups issue uncoalesced memory reads that starve tensor execution units. Sizing cluster capacity from peak FLOP/s creates severe under-provisioning; capacity planning must rely on measured sustained throughput or roofline-bounded kernel projections.

Fallacy: *Any FLOP/s rating can estimate a low-precision workload.*

Accelerators have separate datapaths for different precisions, and the peak throughput varies dramatically across them. An H100 delivers roughly 989 TFLOP/s in FP16 tensor operations but only about 67 TFLOP/s in FP32 CUDA-core operations: a roughly 15× gap within the same chip (Choquette 2023). Estimating training time with the FP32 number when the workload actually uses BF16 produces utilization figures that appear severely degraded without physical cause, and matching against the wrong roofline misclassifies kernels as compute-bound when they are memory-bound (or vice versa). Always match the peak constant to the precision the workload actually issues, and quote precision explicitly when reporting model FLOPs utilization.

Pitfall: *Deploying small-batch inference workloads on high-compute accelerators.*

Small-batch inference severely underutilizes high-compute accelerators because single-sample execution lacks parameter reuse. For a dense linear projection with dimension $M = N = 2048$, a single request ($B = 1$) requires streaming the entire weight tensor from device memory to execute a matrix-vector product, yielding an arithmetic intensity of only 1 FLOP/byte. At batch size $B = 256$, weight reuse amortizes memory transfers across tokens, elevating arithmetic intensity to 204.8 FLOP/byte. At $B = 1$, memory bandwidth clamps delivered performance to 2.04 TFLOP/s on an A100 and 0.3 TFLOP/s on a T4. Although the T4 provides a lower peak of 65 TFLOP/s with a ridge point of 203.1 FLOP/byte, its delivered throughput at $B = 1$ is within a factor of two of the A100 at a fraction of the acquisition cost and thermal footprint. Deploying an A100 for unbatched real-time inference leaves expensive compute units idle; system architects must align accelerator selection with operational batch sizes and latency objectives.

Fallacy: *Vendor-specific optimizations have no long-term portability cost.*

Optimizing exclusively for a single vendor runtime captures immediate kernel efficiency but introduces severe architectural lock-in (section 11.9). Codebases saturated with proprietary device intrinsics, vendor-specific assembly, and non-standard memory layouts require months of engineering labor to port when newer, more cost-effective accelerators emerge. This technical debt delays hardware transitions and prevents multi-vendor procurement strategies. To preserve flexibility, systems must isolate vendor-tuned micro-kernels behind hardware abstraction layers or target intermediate compiler representations (such as Triton or MLIR). Preserving portable execution paths allows infrastructure to track hardware innovation without sacrificing the optimization benefits of specialized execution units.



Checkpoint 11.4: Feasibility assessment: Can you run it?

Before procuring hardware, validate each physical constraint. These failure modes translate into a concrete feasibility test:

- ☐ **Memory capacity:** compute $M_{\text{req}} = \text{Weights} + \text{KV Cache} + \text{Activation Buffer}$ and verify $M_{\text{req}} < M_{\text{device}}$ for a 7-billion-parameter Llama model with 14 GB FP16 weights on a 16 GB GPU.
- ☐ **Bandwidth:** compute $T_{\text{token}} = D_{\text{vol}}/\text{BW}$ for a 70-billion-parameter model (140 GB) on 1 TB/s memory bandwidth, then compare the result with a 50 ms latency target.
- ☐ **Compute:** compute $T_{\text{process}} = O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$ and compare it with the throughput target. Processing video at 30 FPS requires completing inference within 33.3 ms.
- ☐ **Roofline placement:** for an accelerator at roughly 2 TFLOP/s peak and 200 GB/s bandwidth, estimate the ridge-point arithmetic intensity, then place a 10 FLOP/byte kernel against it: is it compute- or memory-bound, and would a faster clock help it at all?

This checklist translates architectural principles into rigorous engineering practice. System evaluation demands a disciplined diagnostic habit: characterize the workload arithmetic intensity, identify the physical bottleneck metric, match execution to the appropriate datapath, and account for interconnect, scaling, and portability costs.

11.15 Summary

Hardware acceleration co-designs compute engines, memory hierarchies, numerical formats, and dataflows. For a chosen memory interface, the iron law compares compute time $O/(R_{\text{peak}}\eta_{\text{hw}})$ with movement time D_{vol}/BW ; the roofline restates that competition as $\min(R_{\text{peak}}, \text{BW}I)$. End-to-end latency also includes launch, synchronization, communication, and fixed overheads. Accelerators improve efficiency only when supported paths capture reuse locally. Hardware selection therefore begins with the workload’s measured or modeled bottleneck, not a vendor peak.



Key Takeaways: Moving data costs more than computing it

- **The Roofline Model identifies bottlenecks:** Compare a kernel’s measured intensity with the target’s precision-specific ridge point (153 FLOP/byte here). Below it, reduce memory traffic; above it, improve compute utilization.
- **Memory bandwidth constrains performance:** GPU compute capacity has grown faster than memory bandwidth. Many inference phases are memory bound, especially low-batch decoding and embedding lookup, while large-batch GEMMs can remain compute bound.
- **Hardware-software co-design compounds performance:** Matching algorithm patterns to architectural capabilities, such as dense GEMM on systolic or tensor units and supported sparsity on sparse paths, can produce large improvements.
- **Tensor Cores require a compatible path:** Precision, dimensions, layouts, and library support must match the architecture. Batch and shape affect reuse and utilization, but no single batch threshold guarantees peak performance.
- **End-to-end speedup stops at the next bottleneck:** Accelerating one kernel leaves launch, transfer, synchronization, communication, and unsupported work unchanged. Use Amdahl’s law and profiling to decide whether further silicon, better mapping, or less movement will improve the complete workload.

An accelerator is not a uniformly faster computer; it provides efficient paths for particular operations and dataflows. Tiling, fusion, hierarchy-aware scheduling, and systolic execution reduce modeled-interface traffic only when the workload, numerical format, and software path can use them.

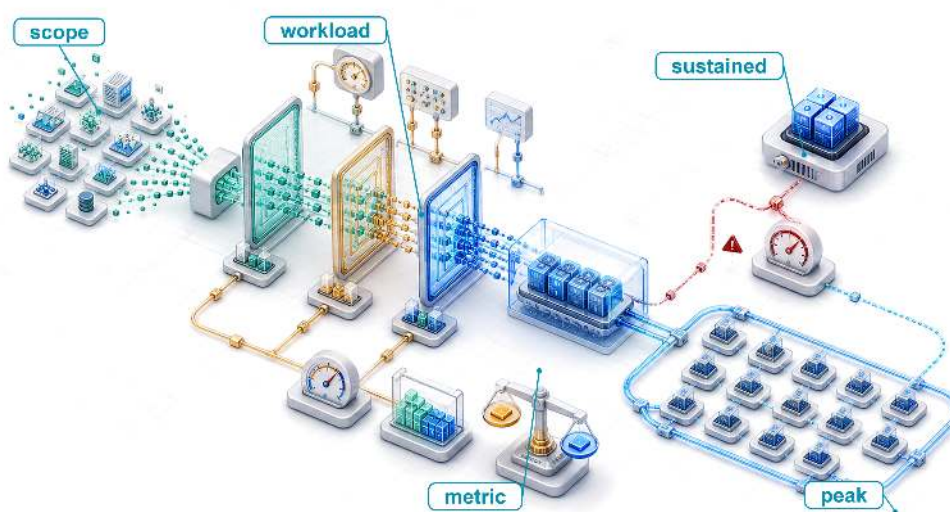
The Roofline Model provides a first diagnostic by using arithmetic intensity and the ridge point to narrow the likely bottleneck; profiling then accounts for latency, occupancy, communication, and implementation overhead. The result is a testable hypothesis, not a prediction.



What’s Next: From optimization to validation

The ML optimization stack rests on three complementary efficiency strategies: data selection (chapter 9) minimizes training sample requirements, model compression (chapter 10) reduces algorithmic parameter and precision complexity, and hardware acceleration maximizes physical machine throughput. Optimization without empirical measurement, however, remains incomplete. Chapter 12 moves from theoretical FLOP bounds to measured wall-clock latency, applying standardized benchmark suites and statistical protocols to test optimization claims against operational reality.

Benchmarking

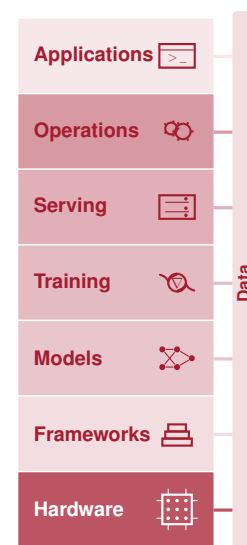


12.1	ML Benchmarking Framework
12.2	Historical Foundations
12.3	System Benchmarking Suites
12.4	Benchmarking Granularity
12.5	Benchmark Components
12.6	Training vs. Inference
12.7	Training Benchmarks
12.8	Inference Benchmarks
12.9	Power Measurement Techniques
12.10	Benchmarking Best Practices
12.11	Model and Data Evaluation
12.12	Production Considerations
12.13	Fallacies and Pitfalls
12.14	Summary

Purpose

How can ML systems be compared fairly when hardware, models, data, and deployment all interact?

Benchmarking brings together decisions already developed through data selection, model compression, and hardware acceleration, then tests whether their gains survive under deployment-representative conditions. Each decision targets one dimension (latency, accuracy, throughput, or energy), but an ML system is the product of all these dimensions simultaneously. A pruned model runs faster on one accelerator but slower on another. A larger batch size improves accelerator utilization but can violate a latency service-level agreement. An edge device may advertise peak throughput that thermal throttling sharply reduces under sustained workloads. The challenge is not whether a local metric improves but whether the combined system improves under conditions that actually matter. Benchmarking makes such comparisons systematic rather than anecdotal. It requires defining what to measure (accuracy, latency, throughput, energy), at what granularity (a single kernel, a full model, an end-to-end pipeline), and under which conditions (batch size, input distribution, thermal state, concurrent load). Without this structure, teams compare numbers that were never measured on the same terms, and decisions that looked sound in a spreadsheet collapse under production workloads. Earlier chapters optimized the model, selected the data, and matched the hardware. Benchmarking validates those optimizations, bringing claims into contact with evidence and quantifying the gap between promise and delivery. In D-A-M terms, benchmarking holds co-design to account by revealing whether Data, Algorithm, and Machine were matched or merely assembled.





Learning Objectives

- Explain benchmarking as D·A·M validation that tests whether optimization claims hold under representative conditions
- Compare training and inference benchmarks using throughput, latency percentiles, energy, accuracy, and workload scope
- Select micro, macro, or end-to-end granularity based on the engineering decision being tested
- Apply standardized benchmark run rules to align datasets, metrics, hardware configuration, and reporting
- Design benchmark protocols that control power boundaries, input distributions, batch sizes, and statistical variance
- Evaluate model and data quality with calibration, robustness, representativeness, and slice-level metrics
- Diagnose benchmark-production gaps caused by drift, thermal throttling, dynamic load, and silent degradation

12.1 ML Benchmarking Framework

A model quantized to INT8 may benchmark $2\times$ faster on a synthetic workload but show no improvement under real traffic patterns with variable input sizes and concurrent requests. A pruned model may maintain accuracy on the test set but fail on edge cases the benchmark never covered. Data selection promises more efficient training, model compression promises smaller and faster models, and hardware acceleration promises higher throughput. Verifying that these claims hold in production is itself an engineering discipline.



Definition 12.1: Machine learning benchmarking

Machine learning benchmarking is the empirical measurement of ML workloads under specified conditions, used to test claims about components, models, or end-to-end systems against representative evidence rather than peak specifications alone.

1. **Significance:** The gap between peak and sustained performance can be large. An A100 GPU delivers 312 TFLOP/s (BF16) at peak, but in this illustrative 30 percent–50 percent model FLOPs utilization (MFU) scenario—measuring the ratio of theoretical architectural compute to advertised hardware capability per second—it sustains 93.6 TFLOP/s—156 TFLOP/s, about a $2\text{--}3.3\times$ gap due to factors such as memory stalls, pipeline bubbles, and kernel launch overhead. Benchmarking quantifies the actual η_{hw} gap for a workload; vendor spec sheets do not.
2. **Distinction:** ML benchmarking spans multiple scopes. Micro-benchmarks isolate operations such as matrix multiplication, model-level benchmarks evaluate complete training or inference workloads, and end-to-end benchmarks include surrounding work such as data loading, preprocessing, optimization, checkpoint I/O, or serving infrastructure. The scope must match the engineering claim.
3. **Common pitfall:** A frequent misconception is that benchmark numbers are stable references. Both the workload (new model architectures) and the hardware (new GPU generations) evolve, so a result that leads a benchmark under one version often becomes the baseline under a later version, making year-over-year comparisons meaningful only when the benchmark version is held constant.

Benchmarking is where the physical laws established in earlier chapters face empirical reality. The **benchmark-production gap** measures the difference between modeled expectations and observed production behavior. Closing that gap requires measurements that convert theoretical claims into verified engineering knowledge.

ML benchmarking operates across three interdependent dimensions that map directly to the components of any deployed system. **System benchmarking** measures whether the hardware delivers promised performance under realistic workloads or whether memory bandwidth saturation and software dispatch overhead erode the gains. **Model benchmarking** measures whether optimization techniques preserve model quality across the full input distribution, not just on curated test sets. **Data benchmarking** measures whether the model generalizes to real-world data with all its noise, bias, and distributional shift. Each dimension can independently reveal problems invisible to the others, and a system that passes all three provides far stronger deployment confidence than one evaluated along any single axis.

An ML benchmark captures a snapshot of a workload and data distribution rather than a permanent specification. The gap between peak and sustained performance is not fixed either; it shifts as workloads and hardware generations evolve, making any single benchmark result time-stamped rather than universal.

Systems Perspective 12.1: Benchmarks as moving targets

In traditional systems (for example, SPEC CPU), the benchmark is a *rigid specification*. A sorting algorithm is correct if it sorts the list. Correctness is absolute and unchanging. In ML systems, the benchmark is a *soft specification*: correctness is defined by a finite set of examples (ImageNet), and the world moves. A model that scores highly on ImageNet can still underperform on user photos taken years after the benchmark was created.

In computer architecture, engineers design for the benchmark because the benchmark represents the workload. In ML engineering, designing solely for the benchmark is *overfitting*. Robustness comes from acknowledging that the benchmark is only a proxy for a shifting reality.

MobileNetV2 deployment validation makes the three-dimensional framework concrete. It serves as the chapter's running example because it spans all three evaluation dimensions, illustrating how each reveals problems the others cannot.

Lighthouse 12.1: MobileNetV2 deployment validation

MobileNetV2 (introduced in section 6.1.1) serves as the lighthouse example for validating the complete optimization pipeline. MobileNetV2 refines v1's depthwise separable design with inverted residuals and linear bottlenecks while maintaining a similar parameter scale (Sandler et al. 2018). It exemplifies the deployment challenges where benchmarking determines success or failure: compression can reduce model size, hardware acceleration can reduce inference latency, and only benchmarking can determine whether those gains survive the full deployment pipeline.

1. **Model compression** (chapter 10): INT8 quantization reduces this MobileNetV2 worked example from 14 MB to 3.5 MB (4× compression)
2. **Hardware acceleration** (chapter 11): the illustrative EdgeTPU scenario uses 2 ms inference vs. 15 ms on CPU
3. Benchmarking validation: Verify the pipeline delivers in practice

A systematic evaluation isolates EdgeTPU latency from preprocessing and data transfer overhead, confirms INT8 quantization preserves accuracy on edge cases such as unusual lighting, and verifies that throughput holds on real-world smartphone sensor streams rather than curated ImageNet test sets alone.

Rigorous evaluation begins with the mindset that separates meaningful evidence from misleading metrics. Three principles distinguish effective practitioners.

First, *benchmarks are proxies, not truth*. Every benchmark measures specific conditions that may not match the target deployment. A system can achieve high sample throughput in Offline mode (bulk throughput with all inputs available) and much lower queries per second (QPS) in Server

mode (latency-constrained requests arriving over time). The critical question is always what the benchmark does not measure.

Second, Goodhart’s Law applies everywhere.¹ “When a measure becomes a target, it ceases to be a good measure.” Teams that optimize for benchmark rankings often produce systems that excel in evaluation but fail in production. Benchmark-specific optimizations frequently degrade characteristics that matter for deployment: robustness, calibration, and efficiency.

Third, end-to-end beats component metrics. In this illustrative pipeline, a 3× inference speedup applied to a 10 ms model stage inside a 50 ms request yields only about a 1.2× end-to-end improvement, or worse if the optimization increases memory pressure. These principles reappear throughout the benchmarking methodology and are examined in depth in section 12.13.

Knowing *what* to measure, however, is only half the problem. Measuring incorrectly (with the wrong workloads, biased baselines, or uncontrolled variables) produces numbers that feel precise but mislead decisions. The history of computing benchmarking is littered with examples of technically sound metrics applied with flawed methodology, from compiler-gamed Whetstone scores to cherry-picked GPU benchmarks that predict nothing about sustained workloads. Understanding how measurement methodology evolved, and where it failed, is essential for designing benchmarks that distinguish genuine improvements from measurement artifacts.

The historical foundations of benchmarking² matter because they expose the validation failures that still recur in ML: optimized metrics that stop predicting real workloads, hardware numbers that ignore sustained operating state, and model scores that miss deployment cost. The same validation sequence governs modern practice: first verify that hardware delivers promised performance, then verify that the model and data optimizations built atop that hardware deliver their promised gains.

12.2 Historical Foundations

In 1976, when Whetstone became one of the first standardized computing benchmarks, vendors began optimizing their compilers specifically for its synthetic floating-point loops, producing peak execution numbers that failed to predict real application performance. Similar gaming has recurrently undermined single-metric and component-level evaluations. Understanding why modern ML evaluation requires the D·A·M taxonomy (Data · Algorithm · Machine) requires tracing how measurement methodologies evolved across computing history. Each benchmark generation arose when existing metrics failed to expose critical system bottlenecks, providing direct precedents for ML evaluation.

Before that history begins, one boundary condition matters: a benchmark is useful only when it names the layer whose claim it validates.

Systems Perspective 12.2: Benchmarking as cross-layer evidence

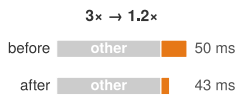
An ML benchmark is meaningful only when it identifies which layer of the system is being validated. Data selection metrics such as PPD, area under the learning curve (AULC), and data compression ratio (DCR), all developed in chapter 9, measure whether fewer examples can preserve learning signal. Compression metrics (chapter 10) measure whether fewer parameters or lower precision preserve quality. Hardware metrics such as roofline position and TOPS/W (chapter 11) measure whether the machine executes the workload efficiently. The system benchmarks in section 12.3.2 through section 12.9.4 connect these layers by testing whether local improvements survive in an end-to-end workload.

That cross-layer role explains why benchmark history matters: performance measurement advanced whenever practitioners discovered that existing metrics failed to predict end-to-end behavior. The evolution from early synthetic tests to modern ML suites reveals three methodological shifts: from isolated kernels to representative application suites, from single-metric throughput to multi-objective energy profiles, and from generic compute metrics to domain-specific execution constraints.

12.2.1 Performance benchmarks

The earliest computing benchmarks revealed an enduring vulnerability in empirical evaluation: benchmark gaming. Whetstone (Curnow and Wichmann 1976) used a synthetic mix of scientific

¹ | **Goodhart’s law:** Goodhart (1984) articulated the original 1975 Bank of England observation on monetary policy; Strathern (1997) generalized it into the form quoted above. The original context was macroeconomics: once a monetary aggregate became an official policy target, banks changed behavior to game the metric, destroying its predictive value. In ML, the same failure mode recurs structurally: BLEU rewards n-gram overlap (Papineni et al. 2002), ImageNet rewards performance on a fixed visual distribution (Deng et al. 2009; Recht et al. 2019), and benchmark leaderboards can incentivize test-set-specific tuning.



Component speedup rarely survives as end-to-end benchmark speedup.

² | **Benchmark:** From surveying, where a “bench mark” was a horizontal cut in stone serving as a fixed elevation reference. The term entered computing in the 1970s to describe standardized comparison points, but the surveying metaphor carries a systems lesson: just as an elevation measurement is meaningless without a calibrated reference, an ML throughput number is meaningless without controlled workloads, thermal state, and precision settings.

operations, while LINPACK³ (Dongarra et al. 1979) measured dense linear-system solving. Because these workloads relied on predictable computational loops, compiler and hardware vendors could optimize specifically for the benchmark kernels—such as unrolling synthetic loops or tailoring cache tiling to exact matrix dimensions—without improving general application performance. SPEC CPU (1989) addressed this limitation by evaluating complete, portable software suites that stressed integer branching, irregular memory hierarchies, and floating-point pipelines (Dixit 1993). This progression directly informs ML systems: model compression claims from chapter 10 cannot rely on isolated matrix multiplication kernels, but require validation on representative tasks like ResNet-50 and BERT across the full software runtime.

As computing expanded into interactive and mobile environments, single-metric evaluation proved insufficient. Graphics benchmarks paired frame rate with rendering quality, and mobile evaluations treated battery draw as co-equal with latency. This multi-objective trade-off mirrors ML serving, where optimizing latency in isolation can trigger catastrophic degradation in model accuracy or energy efficiency (chapter 1).

A third shift occurred when distributed computing demonstrated that component-level microbenchmarks fail to predict cluster-level throughput. Evaluating a CPU or accelerator in isolation cannot capture execution dynamics when network interconnect latency, serialization overheads, and collective communication dominate wall-clock time. Distributed ML training similarly depends on the tight interplay among accelerator matrix engines (chapter 11), host-to-device PCIe bandwidth, input data prefetching pipelines, and gradient synchronization across network fabrics.

To capture these interactions, DAWNBench (Coleman et al. 2019) pioneered time-to-accuracy evaluation, tying raw hardware throughput directly to algorithmic convergence. Maximizing samples processed per second offers no engineering value if numerical instability or suboptimal batch sizing prevents the model from reaching target validation accuracy. These empirical lessons culminated in MLPerf⁴ (2018), which institutionalized full-system measurement, multi-objective constraints, and representative workloads spanning vision, language, and recommendation systems (Mattson et al. 2020; Reddi et al. 2019).

12.2.2 Energy benchmarks

The multi-objective evaluation paradigm extended to energy efficiency as computing diversified beyond mainframes into battery-limited mobile devices and megawatt-scale data centers. Because energy consumption directly dictates battery life at the edge and operating expenses in warehouse-scale clusters, energy became a first-class evaluation metric alongside performance. This shift produced standardized energy benchmarks such as SPEC Power⁵ for servers and Green500⁶ for supercomputers.

Evaluating energy in ML workloads introduces distinct physical measurement challenges. Accelerators exhibit sharp dynamic power swings: alternating between memory-bound activation layers and power-dense matrix multiplication units creates rapid current fluctuations ($\frac{dI}{dt}$) that simple average-power meters fail to resolve. Furthermore, isolating accelerator consumption from host CPU activity, PCIe interconnects, and cooling fans requires rigorous measurement boundaries. MLPerf Power (MLCommons 2024b) addresses these demands by coupling standardized workloads with strict physical measurement methodologies and calibrated external power analyzers.

Energy benchmarking extends beyond hardware telemetry to evaluate algorithmic efficiency. Model compression techniques (pruning, quantization, knowledge distillation) reduce energy consumption by decreasing arithmetic operations and off-chip memory traffic, rather than relying solely on lower-power silicon. For instance, MobileNet architectures replace standard convolutions with depthwise separable convolutions, drastically reducing DRAM access and arithmetic operations relative to standard convolutional neural network (CNN) baselines such as ResNet (Howard et al. 2017; Sandler et al. 2018; He et al. 2016a). As established in chapter 10 and quantified in section 10.4.1.1, reading an operand from off-chip DRAM consumes orders of magnitude more energy than executing an arithmetic operation, making algorithmic memory-traffic reduction a primary lever for energy-efficient computing.

³ | **Whetstone and LINPACK:** Whetstone (Curnow and Wichmann 1976) was named after the English Electric facility in Whetstone, Leicestershire, where the original ALGOL compiler was built; LINPACK (Dongarra et al. 1979) was a package and benchmark for dense linear systems, later used by the Top500 list. Whetstone’s fixed synthetic program mix and LINPACK’s dense linear-algebra focus made each useful but narrower than a diverse application suite. ML benchmarking inherited the same vulnerability: model-specific kernel tuning can overfit a single workload, which is why MLPerf uses multiple workloads spanning vision, language, and recommendation (Mattson et al. 2020; Reddi et al. 2019).

⁴ | **MLPerf:** Launched in 2018 by a consortium of industry and academic institutions, MLPerf takes its name from “ML” combined with “Perf” (performance), echoing SPEC’s benchmarking tradition. MLPerf’s design principles—representative workloads, full-system measurement, and open submission—directly address the gaming that plagued Whetstone and LINPACK: vendors who could previously report peak kernel throughput on cherry-picked problem sizes must now report end-to-end system performance on standardized tasks (Mattson et al. 2020; Reddi et al. 2019).

⁵ | **SPEC Power:** Introduced in 2007, SPEC Power measures performance per watt across 11 load levels from idle (0 percent) through 100 percent in 10 percent increments (Lange 2009). This granularity matters for ML serving: inference workloads rarely sustain 100 percent load, and servers that are efficient at peak but wasteful at partial load inflate the energy cost of real-world deployment.

⁶ | **Green500:** Started in 2007 as a counterpart to the Top500, Green500 ranks systems by FLOP/s per watt rather than raw performance (Feng and Cameron 2007). Its lesson for ML systems is methodological: the most cost-effective training cluster is not necessarily the fastest one, but the system that delivers useful work per watt under the workload and measurement boundary that matter.

12.2.3 Domain-specific benchmarks

As specialized accelerators emerged to bypass general-purpose CPU bottlenecks, generic benchmarks failed to capture domain-specific execution constraints. Three operational axes drove this specialization, defining how systems are evaluated under real-world conditions.

Deployment constraints dictate core metric priorities based on physical limits. Data center training clusters optimize for aggregate throughput within rack- and facility-level power envelopes. In contrast, mobile AI operates within strict thermal throttling thresholds (typically under 5 watts), and embedded microcontroller systems function within milliwatt power budgets. These physical bounds, rooted in the efficiency principles of chapter 1, determine whether a benchmark prioritizes throughput (samples per second) or energy efficiency (Joules per inference).

Application requirements impose functional constraints beyond raw speed. Clinical AI models require rigorous calibration and interpretability alongside classification accuracy; high-frequency financial trading systems enforce hard tail-latency cutoffs paired with deterministic execution; autonomous vehicles demand bounded worst-case latency and formal functional safety validation. These requirements broaden evaluation criteria beyond throughput; chapter 15 formalizes the corresponding engineering principles for safety, compliance, and fairness.

Operational conditions determine real-world robustness. Edge devices must maintain inference deadlines across ambient temperature swings and noisy sensor streams; data-center serving infrastructure must absorb sudden query spikes and network packet loss without dropping requests; industrial IoT nodes must operate unattended for years. Hardware acceleration capabilities (chapter 11) deliver production value only when sustained under these operational environments.

Machine learning exemplifies this domain-specific divergence. Because ML workloads trigger complex interactions between tensor execution units, memory hierarchy bandwidth, and distributed communication fabrics, no single metric or workload can characterize a system. MLPerf formalizes this specialization across distinct deployment suites: MLPerf Training evaluates multi-node scaling and high-bandwidth interconnects under time-to-quality targets (Mattson et al. 2020); MLPerf Inference measures throughput and latency percentiles under service level agreements (SLAs) across server and client systems (Reddi et al. 2019); MLPerf Tiny tests ultra-low-power microcontrollers operating with tight memory constraints (C. Banbury et al. 2021); and the cross-cutting MLPerf Power track evaluates energy efficiency across each environment. Reading table 12.1 down its constraint column illustrates how metrics adapt to physical limits: cluster-scale interconnect bandwidth gives way to latency percentiles at the edge, and ultimately to strict memory footprints on microcontrollers.

Table 12.1: MLPerf Benchmark Suite Variants: Each variant addresses a different deployment context, from data-center-scale training to ultra-constrained microcontroller inference, targeting specific operational constraints and measuring metrics relevant to its deployment scenario.

MLPerf Variant	Target Domain	Key Constraints	Primary Metrics
MLPerf Training	Data center	Multi-node scaling, high bandwidth interconnects	Time-to-quality, throughput (samples/sec)
MLPerf Inference	Server/Edge	Latency SLAs, throughput requirements	QPS, latency percentiles, accuracy preservation
MLPerf Tiny	MCU/IoT	Ultra-low-power inference, limited memory	Latency, accuracy, energy per inference
MLPerf Power	Cross-cutting	Energy budgets, thermal constraints	Performance/W, energy per query

Across all variants, MLPerf Power measures useful work per watt rather than raw throughput alone. Domain-specific suites ensure that architectural optimizations translate to deployment success rather than remaining confined to isolated synthetic benchmarks.

While ML benchmarking suites inherit these historical lessons, machine learning workloads introduce statistical and data-dependent variability absent in traditional deterministic computing. Beyond standard system noise—such as operating system interrupts, background daemon activity, and thermal throttling—ML execution exhibits intrinsic non-determinism. Floating-point reduction across parallel threads is non-associative, causing slight numerical differences when summation order changes; data ingestion pipelines apply pseudo-random augmentations; and stochastic gradient

descent navigates non-convex loss surfaces sensitive to weight initialization. Consequently, rigorous ML evaluation requires explicit statistical controls, including multi-seed replication, variance reporting, and deterministic data-loading rules.

Standardization transforms these empirical lessons into a shared engineering methodology. When one team measures inference latency including host preprocessing while another begins timing only after tensors reside in accelerator device memory, or when power meters draw different physical boundaries around the power supply unit, the resulting numbers are incommensurable. Standardized suites replace ad-hoc testing with audited harnesses, uniform measurement boundaries, and verifiable submission rules, enabling rigorous hardware procurement and algorithmic comparison across the industry. Table 12.2 synthesizes this historical progression from early synthetic loops to multi-dimensional ML systems evaluation.

Table 12.2: Benchmark Evolution: Evolution of computing benchmarks from synthetic operations to ML-specific evaluation. Each generation addressed limitations of its predecessors, culminating in MLPerf’s synthesis of representative workloads, multi-objective metrics, and integrated system measurement.

Benchmark	Year	Primary Focus	Key Metric(s)	Lesson for ML Benchmarking
Whetstone	1976	Synthetic floating-point operations	MWIPS	Gaming synthetic tests undermines evaluation validity
LINPACK	1979	Linear algebra (matrix operations)	FLOP/s	Isolated operations miss system-level complexity and bottlenecks
SPEC CPU	1989	Real application workloads	SPECrate, SPECspeed	Representative workloads reveal true deployment performance
SPEC Power	2007	Server energy efficiency	ssj_ops/W across load levels	Energy efficiency requires multi-load evaluation, not just peak performance
Green500	2007	HPC energy efficiency	GFLOP/s per watt	Efficiency rankings complement raw performance rankings
MLPerf	2018	ML systems (training + inference)	Time-to-quality, QPS, latency, accuracy	Synthesizes all lessons: representative workloads + multi-objective + system

12.3 System Benchmarking Suites

A team evaluating edge deployment hardware must compare five different system-on-chip (SoC) designs for a smart camera product. Vendor A reports 8 TOPS at INT8; Vendor B reports 15 TOPS at INT4; Vendor C reports inference latency on a proprietary model; Vendor D cites MLPerf scores from two generations ago; Vendor E provides only peak throughput at maximum batch size. None of these figures can be directly compared. The team cannot make an informed procurement decision because each vendor isolates a favorable metric under idiosyncratic conditions, obscuring true operating trade-offs. Benchmarking suites resolve this fragmentation by enforcing uniform measurement harnesses and standardized evaluation rules.

Three lessons from benchmark history—representative workloads, multi-objective evaluation, and integrated system measurement—converge on a requirement unique to machine learning: managing statistical approximation. In traditional computing benchmarks, valid system optimizations must preserve bit-exact arithmetic results. In ML systems, low-level engineering choices routinely alter numerical output: aggressive weight quantization, kernel pruning, or reduced-precision accumulators trade accuracy for execution speed and lower memory bandwidth pressure. Consequently, an ML benchmark cannot evaluate execution time in isolation; output accuracy serves as an inviolable operational constraint. Modern benchmarking suites encode these constraints into standardized frameworks, making rigorous cross-platform comparisons possible.

Evaluating an ML system therefore requires assessing the coupled interactions across the Data, Algorithm, and Machine (D·A·M) co-design space rather than measuring raw compute throughput. Early benchmarks focused primarily on algorithmic accuracy on isolated datasets (LeCun et al. 1998). As models scaled, compute bottlenecks expanded benchmarking to hardware execution efficiency (Jouppi et al. 2017), and deployment failures demonstrated that data pipeline bottlenecks and input distribution shifts dictate real-world behavior (Gebru et al. 2021). Energy consumption cuts directly across all three dimensions: algorithmic structure dictates arithmetic intensity and computational

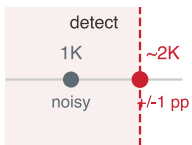
complexity (Hernandez and Brown 2020), machine microarchitecture sets the energy cost per arithmetic operation and memory access, and data ingestion patterns determine bus utilization. A robust benchmarking suite tests these dimensions simultaneously, ensuring that claimed optimizations survive under production constraints.

12.3.1 ML measurement challenges

ML systems combine several sources of measurement variability that classical software benchmarks were not designed to evaluate together. These fluctuations originate at both the machine and algorithmic layers. On the machine side, sustained execution pushes accelerators against thermal and power envelopes, triggering dynamic voltage and frequency scaling (DVFS) that throttles processor clocks mid-run. Host operating-system activity—including thread scheduling jitter, memory paging, and PCIe bus contention during batch staging—further introduces latency variance across runs. On the algorithmic side, non-deterministic floating-point reduction orders across parallel execution threads, mini-batch shuffling, and stochastic dropout masks perturb numerical trajectories. Distinguishing a genuine systems optimization from background measurement noise requires isolating these overlapping sources of variance.

To isolate run-to-run variability, benchmark protocols require repeated experimental trials across controlled pseudo-random seeds. The number of runs must satisfy explicit statistical-power targets rather than opportunistic sampling. Reporting must extend beyond single best-of- N scores or isolated arithmetic means: reporting sample variance, standard deviations, or confidence intervals quantifies stability and establishes whether an observed speedup exceeds background system jitter. Without these controls, benchmarking yields misleading conclusions. In reinforcement learning, reported performance advantages frequently evaporate into the statistical noise of random weight initializations and environment dynamics (Henderson et al. 2018). Similarly, unstandardized evaluations of generative adversarial networks produce rank reversals across random seeds, demonstrating that uncharacterized variance undermines comparative claims (Lucic et al. 2018).

Even when run-to-run execution variance is fully controlled, benchmarking encounters a structural constraint in the evaluation dataset itself: test-set measurement capacity. A test set operates as a measurement instrument with a finite resolution limit. Because accuracy on a finite sample is a binomial proportion, empirical scores carry intrinsic sampling variance that scales inversely with sample size. When an optimization alters accuracy by one or two percentage points, evaluating on an undersized test set cannot determine whether the change reflects a genuine algorithmic effect or a random sampling fluctuation. As illustrated in the margin detectability marker and analyzed in the accompanying notebook, attempting to evaluate fine-grained accuracy deltas on a thousand-sample test set falls directly into this statistical confidence trap.



A 1K test set cannot reliably see a one-point regression.

Napkin Math 12.1: The statistical confidence trap

Problem: A baseline classifier scores 95 percent, while a compressed version scores 94 percent on 1,000 images. Does this one-point drop establish a regression?

Math:

1. **Expected errors:** The scores correspond to 50 errors and 60 errors. Under the baseline binomial model, the error count has a standard deviation of about 7 errors.
2. **Difference interval** (95 percent): Let $\hat{p}_1$ and $\hat{p}_2$ denote the baseline and compressed accuracy estimates, respectively, and let N be the number of images evaluated for each model. Treating the two estimates as independent, the standard error of their difference is

$$SE(\hat{p}_1 - \hat{p}_2) = \sqrt{\frac{\hat{p}_1(1 - \hat{p}_1)}{N} + \frac{\hat{p}_2(1 - \hat{p}_2)}{N}}.$$

The observed 1 percentage point difference has an approximate interval from -1 percentage point to 3 percentage points, which includes zero.

Implication: Because the interval includes zero, the result does not establish a one-point regression. If both models ran on the same images, retain their disagreement counts and use a paired test such as McNemar’s test; marginal accuracies are insufficient. The 1,825-sample estimate for a single rate with ± 1 percentage point precision does not power this comparison.

Systems insight: Small benchmarks exhibit what amounts to a laboratory fallacy. The test set, viewed as a measurement instrument, must be sized to match the precision of the change it is meant to detect.

Beyond statistical power, benchmark validity depends on workload representativeness. Synthetic microbenchmarks generate uniform, statically sized tensors directly in accelerator high-bandwidth memory (HBM). By executing continuous, compute-bound general matrix multiplication (GEMM) operations in an isolated loop, microbenchmarks measure theoretical peak arithmetic throughput. However, they conceal the memory hierarchies and scheduling bottlenecks that dominate production deployments. Synthetic workloads bypass host-to-device PCIe data transfers, input decoding pipelines, dynamic memory allocation churn, and GPU cache evictions. In contrast, **trace-based benchmarking** replays timestamped request logs recorded from live serving clusters. Production traces capture Poisson and bursty request arrival distributions, variable sequence lengths, and concurrent query streams. Replaying live traces forces the serving infrastructure to manage dynamic batching, key-value (KV) cache allocation under memory pressure, and queuing delays—stress-testing the system under operational conditions that synthetic steady-state loops obscure.

A representative workload can still mislead if the evaluation metric fails to reflect deployment constraints. This failure mode represents a breakdown of metric alignment. When an engineering team optimizes a single algorithmic figure of merit—such as an offline BLEU score, top-1 classification accuracy, or token perplexity—in isolation from system service-level objectives (SLOs), the metric ceases to measure genuine operational fitness. This divergence embodies Goodhart’s Law: once an algorithmic proxy metric becomes the sole optimization target, it collapses as a measure of overall system health. The accompanying example illustrates this pathology in neural machine translation, where an offline accuracy optimization creates a catastrophic latency penalty.



Example 12.1: Goodhart’s Law in action

Scenario: An engineering team optimizes an NMT translation model for offline BLEU score, increasing beam search size from 1 to 10.

Diagnosis: The larger beam search achieves a 0.5-point BLEU gain on paper but increases candidate evaluations by 10 $\times$, increasing inference latency from 50 ms to 200 ms (4 $\times$ slowdown), exceeding the serving deadline.

Systems lesson: Optimizing offline accuracy without latency constraints invites Goodhart’s Law failures. Production benchmarks must enforce strict SLOs for serving latency (e.g., latency < 100 ms).

The translation trade-off exposes the fundamental tension across the Data, Algorithm, and Machine (D-A-M) co-design space: an algorithmic change (expanding the beam width tenfold) multiplies computational load and memory traffic, extracting a marginal data quality improvement while violating hardware latency deadlines. Beyond metric misalignment, evaluating systems on static datasets introduces a deeper limitation: fixed benchmarks assess execution under an unvarying input distribution, concealing how systems degrade under production distribution shifts. Fairly evaluating an ML system therefore requires isolating infrastructure execution efficiency from model architecture and dataset variation. Standardized suites like MLPerf establish this baseline by fixing both model weights and input datasets, enabling direct, reproducible measurement of hardware-software execution efficiency across competing platforms.

12.3.2 System benchmarks

System benchmarks measure the computational foundation that enables model capabilities, examining how hardware architectures, memory systems, and interconnects affect overall performance. This validation is critical because hardware specifications often describe theoretical peaks that application workloads do not sustain. The discrepancy is common enough to make peak-performance claims incomplete. System benchmarks reveal these gaps by running standardized ML workloads rather than relying on peak arithmetic rates alone.



Systems Perspective 12.3: The fallacy of peak performance

Dave Patterson often refers to peak performance as “the performance the manufacturer guarantees you will not exceed.” For ML systems, the gap can be wide because of the memory wall: a memory-bound model leaves much of the advertised FLOP/s unreachable no matter how fast the arithmetic units run. Standardized benchmarks like MLPerf require systems to run specified models and data, revealing sustained performance at a documented operating point rather than a universal rate for every workload.

For memory-bound workloads, the peak-vs.-sustained gap follows from the memory wall; compute-bound workloads may approach peak. This distinction reframes vendor evaluation from guesswork into a checklist of concrete criteria.



Checkpoint 12.1: Decoding vendor benchmark claims

When evaluating hardware or software based on vendor-reported benchmarks, check whether the claim identifies the workload, measurement boundary, and operating conditions.

- ☐ Measured quantity: Does the latency number include preprocessing and postprocessing, not just model execution?
- ☐ Precision boundary: Can the precision behind a throughput claim, such as INT4 versus INT8 or FP16, be identified?
- ☐ Workload boundary: Are the model, batch size, and input distribution used to produce the comparison explicitly named?
- ☐ Excluded costs: Are memory transfers, model loading, initialization, sustained thermal behavior, and power accounted for at the claimed performance level?
- ☐ Claimed vs. achieved: Given a 1,000 TFLOP/s peak claim and a benchmark that sustains 350 TFLOP/s, estimate the MFU and state whether that utilization ratio alone identifies the bottleneck.

Engineers should reject any benchmark claim whose workload boundary, precision, and excluded costs cannot be reconstructed. A headline throughput or latency number becomes useful only after the engineer can map it to the actual model, batch shape, data movement, sustained operating point, and power envelope.

The underlying hardware—CPUs, GPUs, Tensor Processing Units (TPUs),⁷ and application-specific integrated circuits (ASICs)⁸—determines ML-system speed, efficiency, and scale. System benchmarks provide standardized methods for comparing hardware across AI workloads by computational throughput, memory bandwidth, power efficiency, operator coverage, and scaling (Reddi et al. 2019; Mattson et al. 2020).

Table 12.3 translates common marketing phrases into the technical caveats behind each.

⁷ | **TPU (tensor processing unit):** Google’s custom ASIC for neural network workloads (architecture details in chapter 11). A TPU v4 pod (4,096 chips) delivers 1.1 exaFLOP/s peak BF16 (Jouppi et al. 2023), but benchmarking TPUs requires caution: their systolic-array architecture favors regular tensor operations, so peak FLOP/s overstate performance on irregular workloads like sparse attention or dynamic control flow.

⁸ | **ASIC (application-specific integrated circuit):** An ASIC’s peak TOPS number applies only to the specific operators it was designed for. A single unsupported layer forces fallback to a general-purpose processor, potentially negating the entire efficiency advantage. This makes operator coverage the first question in any ASIC benchmark: the gap between peak and achieved throughput is not a hardware limitation but a workload-compatibility limitation.

Table 12.3: Decoding Vendor Benchmark Claims: Four common marketing phrases and the technical caveats that determine whether a claim describes end-to-end performance, a narrow accelerator measurement, or an unsupported efficiency comparison.

Vendor Claim	What It Often Means
“Up to 10,000 images/sec”	Peak throughput at maximum batch size, INT8, without preprocessing
“Sub-millisecond latency”	Accelerator compute only, excluding data transfer
“5× more efficient”	Per-operation efficiency, not total system efficiency
“Optimized for AI”	May only accelerate specific operations or precisions

System benchmarks serve two functions. For practitioners, they enable informed hardware selection by providing comparative data across configurations. For manufacturers, they quantify generational improvements and guide accelerator development. As GPU adoption grew, accuracy also improved rapidly, illustrating how hardware and algorithmic advances can drive progress together.

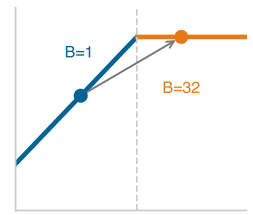
Definition 12.2: Machine learning system benchmarks

Machine learning system benchmarks are standardized evaluation protocols that hold the workload and quality target constant while varying the hardware-software stack, measuring $\eta_{hw} = R_{sustained}/R_{peak}$ and L_{lat} to isolate infrastructure efficiency from algorithmic improvements.

1. **Significance:** The same ResNet-50 model can deliver very different throughput across hardware stacks, precision formats, batch sizes, and compiler configurations, yet still report the same ImageNet Top-1 accuracy. System benchmarks capture this implementation gap, which is invisible to algorithmic benchmarks that only report accuracy.
2. **Distinction:** Unlike algorithmic benchmarks (which vary model architectures and training procedures to improve convergence accuracy), system benchmarks hold the algorithm fixed and vary the implementation (kernel libraries, quantization formats, batch sizes, and hardware generations) to measure how efficiently the hardware-software stack executes the iron law’s $O/(R_{peak} \cdot \eta_{hw})$ term.
3. **Common pitfall:** A frequent misconception is that a system benchmark result generalizes across workloads. An accelerator that achieves high utilization on ResNet-50 (a compute-friendly vision workload) may achieve much lower utilization on a recommendation system (a memory-bandwidth-bound workload). System benchmarks are workload-specific; no single metric characterizes a hardware platform.

Effective benchmark interpretation requires knowing the performance characteristics of target hardware. Whether a specific AI workload is compute bound or memory-bound provides essential insight for optimization decisions. Computational intensity, measured as FLOP/byte,⁹ determines performance limits. Consider an NVIDIA A100 GPU with 312 TFLOP/s of FP16 Tensor Core performance (FP32 is 19.5 TFLOP/s) and 2.04 TB/s memory bandwidth (SXM variant). Dividing peak compute by peak bandwidth yields an arithmetic intensity threshold of 153 FLOP/byte. Workloads below this threshold are bottlenecked by memory bandwidth, while those above are bottlenecked by compute capacity. The Roofline Model in section 11.6 provides the architectural foundation for interpreting these benchmark results. Section D.2.1 derives the roofline equation and the ridge-point threshold from first principles, so the arithmetic intensity bound used here can be reconstructed for any accelerator.

Roofline position¹⁰ depends on the workload. In this worked A100 example, an illustrative high-intensity ResNet-50 forward-pass workload at large batch size uses an arithmetic intensity of ~300 FLOP/byte, above the A100 ridge, and therefore represents compute-bound kernels (He et al. 2016a; Choquette et al. 2021). Lower-intensity operations fall below the ridge into the memory-bound regime: a BERT inference at batch size one, counting only weight-loading traffic, reaches ~100 FLOP/byte arithmetic intensity and a lower performance ceiling than peak. Increasing the batch



Larger batches push transformer inference from memory-bound to compute-bound.

9 | FLOP/s (floating-point operations per second): The gap between advertised peak FLOP/s and achieved FLOP/s is the central tension in hardware benchmarking. The A100 advertises 312 TFLOP/s FP16 Tensor Core, but real workloads achieve different fractions of peak depending on arithmetic intensity, memory access patterns, precision, and runtime overhead. Reporting peak FLOP/s without utilization context is the most common benchmarking distortion.

10 | Roofline model: Williams et al. (2009) introduced the Berkeley model, named for the visual shape of its performance ceiling. Its ridge point (peak FLOP/s divided by peak bandwidth) separates memory-bound from compute-bound workloads, showing whether optimization should target data movement or arithmetic.

size moves that same workload across the ridge from memory-bound to compute-bound (Pope et al. 2023). Section D.2.1.1 works the intensity-to-utilization calculation end to end on the A100, contrasting a compute-bound matrix multiplication kernel against a memory-bound element-wise kernel, so the steps generalize to any model-hardware pair.

A worked BERT inference estimate shows how these roofline principles translate into concrete deployment predictions.

Napkin Math 12.2: Roofline analysis for BERT inference

Problem: BERT-Base must be deployed for inference on an A100 GPU. Management expects high GPU utilization. What performance should be predicted, and how can it be improved?

Step 1: Hardware limits.

- Peak compute: 312 TFLOP/s (FP16 Tensor Core)
- Memory bandwidth: 2.04 TB/s
- Ridge point: $312 \text{ TFLOP/s} \div 2.04 \text{ TB/s} = 153 \text{ FLOP/byte}$

Any workload with arithmetic intensity below 153 FLOP/byte is memory bound; above is compute bound.

Step 2: BERT-base characteristics.

- Parameters: 110M = 220 MB (FP16)
- FLOPs per inference: ~22 GFLOP (forward pass with sequence length $S = 128$)
- Data movement: ~220 MB (must load all weights from memory)
- Arithmetic intensity: $(22 \times 10^9) \div (220 \times 10^6) = 100 \text{ FLOP/byte}$ (weights-only model; see note in main text)

Step 3: Performance prediction. Since $100 \text{ FLOP/byte} < 153 \text{ FLOP/byte}$, BERT at batch = 1 is memory bound:

$$\text{Achievable perf} = 100 \text{ FLOP/byte} \times 2.04 \text{ TB/s} = 203.9 \text{ TFLOP/s}$$

$$\text{GPU utilization} = 203.9 \text{ TFLOP/s} / 312 \text{ TFLOP/s} = 65.4\%$$

Step 4: Optimization via batching. Increase batch size to 32:

- Same 220 MB of weights, but $32\times$ more compute
- New FLOPs: $22 \times 10^9 \times 32 = 704 \text{ GFLOP}$
- New intensity: $(704 \times 10^9) \div (220 \times 10^6) = 3200 \text{ FLOP/byte}$

Since $3200 \text{ FLOP/byte} > 153 \text{ FLOP/byte}$, batch = 32 is compute bound. Assuming the implementation then sustains 85 percent of peak:

$$\text{Achievable perf} \approx 85\% \times 312 \text{ TFLOP/s} = 265.2 \text{ TFLOP/s}$$

$$\text{GPU utilization} \approx 85\%$$

Systems insight: Batch size can transform memory-bound inference into compute-bound inference by raising arithmetic intensity. The displayed 85 percent is an implementation-efficiency assumption, not a roofline prediction. Batching also increases latency because the system must wait to accumulate requests. This is the fundamental throughput-latency trade-off that MLPerf scenarios capture: SingleStream (batch = 1, latency-optimized) vs. Offline (maximum batch, throughput-optimized).

System benchmarks evaluate performance across scales, ranging from single-chip configurations to large distributed systems and covering both training and inference. Figure 12.1 juxtaposes sourced ImageNet top-5 error rates (Russakovsky et al. 2015; Krizhevsky et al. 2012) with a reconstruction of the growing use of GPUs in challenge entries, read from the entry counts NVIDIA charted for the

challenge (Gray 2015). The two series show contemporaneous trends, not a causal estimate of how much of the accuracy improvement came from hardware rather than algorithms, data, or training practice.

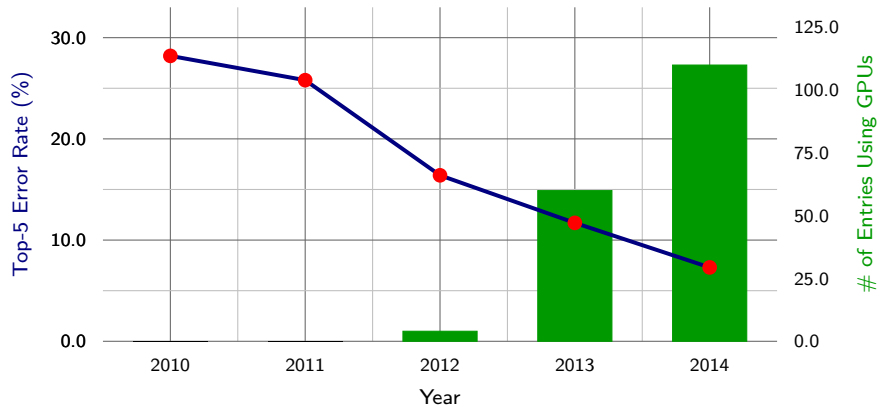


Figure 12.1: ImageNet Error and GPU Adoption: Sourced top-5 error rates are shown beside GPU use among challenge entries. The temporal association does not isolate hardware’s causal contribution. Error-rate sources: (Russakovsky et al. 2015; Krizhevsky et al. 2012). Gray (2015) charts the per-year GPU entry counts.

The ImageNet example places GPU adoption alongside falling error rates without isolating either contribution; section 12.11.1 revisits this progression through model-specific architectural milestones. Effective system benchmarking, however, requires understanding the relationship between workload characteristics and hardware utilization. Modern AI systems rarely achieve theoretical peak performance due to interactions between computational patterns, memory hierarchies, and system architectures. This gap between theoretical and achieved performance shapes the design of meaningful system benchmarks.

Realistic hardware utilization patterns are essential for actionable benchmark design. As the preceding roofline analysis illustrated, GPU utilization varies with batch size and model architecture; the compute-bound value of 85 percent is an assumption, while the memory-bound single-request value is 65.4 percent. These patterns extend to memory bandwidth: parameter-heavy transformer inference and activation-heavy convolutional workloads stress different parts of the memory hierarchy, directly impacting achievable performance across different precision levels.

Effective system benchmarks must measure realistic utilization rather than peak theoretical capability, and this requirement establishes several scope boundaries. Energy is one dimension: performance per watt varies widely across platforms, and an underutilized accelerator consumes disproportionate power for its output, penalizing both operational cost and environmental impact. Distribution is another: multi-node training adds communication bottlenecks, network-topology effects, and coordination overhead that single-node benchmarks cannot capture and that warrant dedicated treatment beyond this book. Within the single-machine scope here, multi-GPU benchmarking instead focuses on intra-node communication, memory-bandwidth utilization across accelerators, and gradient-synchronization efficiency across distinct accelerator memories connected by NVLink or PCIe. Across all of these, a benchmark earns its value only when its operating point matches the deployment’s, not the datasheet’s.

12.3.3 Community-driven standardization

Hardware utilization metrics become meaningless for comparative systems evaluation without rigid, shared measurement boundaries. If one benchmark includes host-side data preprocessing and PCIe transfers while another measures only device kernel execution, the resulting latency figures reflect incompatible system boundaries rather than architectural efficiency. Similarly, measuring power consumption solely at the accelerator voltage rail omits the substantial energy drawn by host processors and chassis cooling fans. Independent organizations cannot resolve this discrepancy alone, as individual vendors face commercial incentives to tailor measurement boundaries to their specific hardware strengths.

Reliable benchmarking standards require broad multi-organizational consensus to prevent selective reporting. In mature engineering domains, this consensus formalizes through bodies such as IEEE working groups (IEEE Standards Association 2024) and ISO/IEC technical committees (ISO 2024), which establish rigorous measurement specifications such as IEEE 2416 (IEEE Standards Association 2019a) for system power modeling. In machine learning systems, where hardware architectures and model structures evolve rapidly, consortia such as MLCommons provide the operational framework needed to maintain living benchmark suites. By providing open-source reference implementations paired with strict compliance verification rules, these consortia ensure that performance numbers reported across different laboratories represent identical computational work.

Standardizing system benchmarks requires formalizing the D·A·M taxonomy so that evaluations isolate the Machine without confounding changes to the Algorithm or Data. The MLPerf Training benchmark enforces this separation by measuring time-to-train to a specified quality target rather than raw floating-point throughput or epoch duration (Mattson et al. 2020). A hardware platform that achieves high TFLOPS through aggressive low-precision scaling provides no systems benefit if numerical drift degrades model convergence or forces extra training epochs. In its Closed Division, the standard locks both the model architecture and numerical hyperparameters, ensuring that observed performance gains originate from genuine hardware and compiler efficiencies—such as interconnect bandwidth utilization and kernel fusion—rather than unvalidated algorithmic compromises.

The MLPerf Inference benchmark applies this discipline to production deployment constraints (Reddi et al. 2019). An accelerator cannot simply report peak throughput under infinite batching if interactive serving requires guaranteed responsiveness. The benchmark defines distinct evaluation scenarios for different physical operating regimes. For interactive environments, Server mode injects queries following a Poisson arrival process under strict 99th-percentile latency service-level agreements (SLAs), while Single-Stream mode measures minimum latency at batch size one. For non-interactive throughput, Offline mode saturates compute units with preloaded batches. By standardizing these execution boundaries, community benchmarks account for queueing delays and host-to-device PCIe transfers, preventing vendors from presenting throughput numbers that collapse under production traffic.

Community standards establish reproducible measurement boundaries, but they do not dictate the physical scale at which evaluation occurs. A measurement can target an isolated matrix multiplication kernel or profile an end-to-end multi-accelerator training run, and each scope diagnoses distinct physical bottlenecks. The depth of measurement determines which architectural limits can be isolated, establishing the role of benchmarking granularity.

12.4 Benchmarking Granularity

A GPU kernel that runs $3\times$ faster in isolation may deliver zero end-to-end speedup if the data ingestion pipeline cannot saturate device memory bandwidth. This diagnostic mismatch makes evaluation granularity a fundamental systems design choice. Standardization specifies *how* measurement is kept consistent, while **benchmarking granularity** specifies *what* physical boundary is measured. System validation spans multiple architectural scales, from individual tensor operations to distributed production pipelines, with each granularity diagnosing distinct bottlenecks:

- Micro benchmarks isolate individual components: kernel execution time, memory bandwidth utilization, and single-layer numerical behavior. These diagnose *where* low-level hardware inefficiencies occur.
- Macro benchmarks evaluate composed model subsystems: full model training convergence, inference pipeline throughput, and dataset accuracy metrics. These reveal *what* architectural trade-offs exist across layer compositions.
- End-to-end benchmarks measure complete operational pipelines: client request-to-response latency including preprocessing, training time-to-accuracy including storage I/O, and model robustness on production data distributions. These show whether the combined system satisfies real-world service-level objectives.

Optimization techniques operate across distinct granularities: kernel fusion targets micro-level execution, structured pruning alters macro-level parameter structures, and data curation determines end-to-end generalization. System validation must align with these respective boundaries. An isolated micro-benchmark may show substantial kernel speedup, yet a macro-benchmark can reveal that increased activation memory footprints force smaller batch sizes that degrade overall throughput. Similarly, an end-to-end benchmark exposes storage or host deserialization stalls that remain completely invisible at the accelerator kernel level.

Figure 12.2 maps these granularity levels onto the ML stack by partitioning evaluation into four distinct physical scopes. Each scope expands the measurement boundary: micro-benchmarks isolate kernel execution, macro-benchmarks evaluate composed model topologies, application benchmarks incorporate host data management and auxiliary runtime logic, and end-to-end benchmarks encompass the complete distributed deployment.

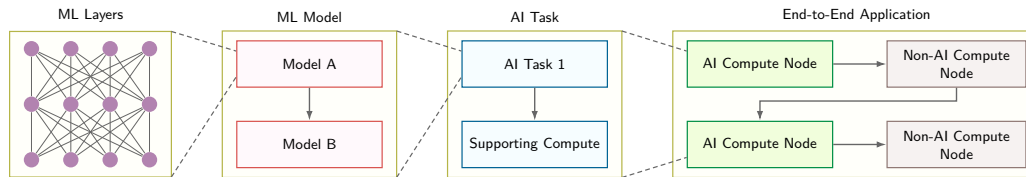


Figure 12.2: Benchmarking Granularity: Four-panel block diagram showing micro, macro, application, and end-to-end evaluation layers. Each panel maps a distinct scope of assessment, from isolated kernel operations through full-system deployment, enabling targeted optimization at every level of the ML stack.

12.4.1 Micro benchmarks

While end-to-end metrics govern service-level agreements, optimizing an accelerator pipeline requires isolating the specific mathematical operations that consume execution time and energy. Micro-benchmarks serve this diagnostic role by evaluating individual tensor operations in isolation, measuring the hardware primitives introduced in chapter 11. When profiling a slow inference service, macro-benchmarks report aggregate latency violations, but only micro-benchmarks reveal whether execution time is dominated by matrix multiplications, self-attention projections, memory copies across the host-accelerator interconnect, or elementwise activation kernels.

Tensor operations constitute the primary target of micro-benchmarking because matrix multiplications and convolutions dominate the arithmetic budget of deep neural networks. Highly optimized runtime libraries such as cuDNN¹¹ (Chetlur et al. 2014) provide hand-tuned kernel implementations tailored to specific microarchitectures. Micro-benchmarks also evaluate standalone activation functions (such as ReLU, Sigmoid, and GELU) and structural submodules (such as multi-head attention projections or transformer feed-forward blocks) across standardized input shapes. Benchmark suites such as Baidu’s DeepBench (Baidu Research 2016) formalized this methodology by evaluating raw GEMM, convolution, and collective communication primitives across diverse hardware platforms, isolating silicon-level execution efficiency from higher-level framework dispatch overhead.

Measuring isolated kernels requires rigorous experimental control. Without disciplined measurement protocols, dynamic hardware frequency scaling, asynchronous execution queues, and cache state introduce transient artifacts that distort kernel runtime.

Systems Perspective 12.4: Micro-benchmarking rules

To avoid measuring hardware artifacts instead of kernel performance, follow the *Systems Detective’s Rules*:

1. *The warm-up rule:* Do not measure cold-start iterations as steady-state performance. Modern hardware uses DVFS (dynamic voltage and frequency scaling) and dynamic frequency boosting; caches, kernels, and clocks need warm-up before the measured loop represents sustained behavior.

¹¹ | **cuDNN (CUDA deep neural network library):** Released by NVIDIA in 2014, cuDNN provides hand-tuned kernel implementations for convolutions, pooling, and normalization. The benchmarking implication: reported inference latencies depend heavily on which cuDNN version and algorithm autotuner settings were used, making cuDNN version a mandatory element of any reproducible benchmark specification.

2. *The variance rule*: Report the coefficient of variation (CV) ($CV = \sigma_{\text{run}} / \mu_{\text{run}}$), where σ_{run} and μ_{run} are the standard deviation and mean across repeated runs. Investigate values above the protocol's workload-specific tolerance; common causes include background OS jitter, thermal throttling, and memory contention, and 5 percent is not a universal cutoff.
3. *The “speed of light” (SOL) check*: Compare the achieved throughput against the roofline. If a kernel achieves 10 TFLOP/s on an H100 (peak ~989 TFLOP/s FP16, or ~1,979 TFLOP/s FP8 dense), the diagnostic step is to identify the cause of low utilization (often kernel launch latency from too many small kernels) before optimizing the code itself.
4. *The cache-state rule*: Set cache state to match the claim. Flush or exceed the L2 cache when measuring cold dynamic random-access memory (DRAM) bandwidth; preserve a warmed cache when measuring steady-state reuse. Otherwise a nominal DRAM result may instead reflect cache bandwidth (~5 TB/s–10 TB/s) rather than DRAM bandwidth (~1 TB/s–2 TB/s).
5. *The synchronization barrier rule*: Accelerator kernel launches are asynchronous on the host CPU. Timing GPU operations using CPU clocks without an explicit device barrier (such as `torch.cuda.synchronize()`) measures host enqueue latency rather than kernel execution; always place synchronization barriers immediately before and after the timed region, or use GPU-side timestamp events (`cudaEventRecord`).

A profiler translates these measurement rules into physical diagnostics by decomposing kernel execution time into the three governing terms of the iron law: data movement, compute throughput, and latency overhead (section 1.7).

🔍 Systems Perspective 12.5: Measuring the iron law terms

Moving from theory to trace means mapping the iron law equation from section 1.7 onto a profiler timeline (like Nsight Systems or PyTorch Profiler).

Measuring the data term ($\frac{D_{\text{vol}}}{\text{BW}}$)

- Signal: Look for the “Memory Throughput” or “DRAM Bandwidth” line.
- **Formula**: $\text{BW}_{\text{effective}} = \frac{D_{\text{vol}}}{T_{\text{kernel}}}$.
- **Diagnosis**: If $\text{BW}_{\text{effective}} \approx \text{BW}_{\text{peak}}$ (for example, close to the A100's 2.04 TB/s peak), the kernel is memory bound. Optimizing compute (O) will do nothing.

Measuring achieved compute throughput ($R_{\text{peak}} \cdot \eta_{\text{hw}}$)

- Signal: Look for “SM Active” or “Compute Throughput”.
- **Formula**: Achieved TFLOP/s = $\frac{O}{10^{12} T_{\text{kernel}}}$.
- **Diagnosis**: If Achieved TFLOP/s $\ll$ Peak TFLOP/s AND $\text{BW}_{\text{effective}} \ll \text{BW}_{\text{peak}}$, the system is in the “Utilization Trap”: likely Latency Bound (kernels too small) or Grid Bound (not enough threads).

Measuring the latency term (L_{lat})

- Signal: Look for gaps (empty space) between colored kernel bars on the timeline.
- **Formula**: Overhead Ratio = $\frac{T_{\text{gap}}}{T_{\text{kernel}} + T_{\text{gap}}}$.
- **Diagnosis**: A “Sawtooth” pattern (Compute, Gap, Compute, Gap) indicates high software overhead. The solution is operator fusion, covered in section 11.8.1.3, or CUDA Graphs, which capture a repeated sequence of GPU launches so the runtime can replay it with less CPU dispatch overhead.

While standardized benchmarks measure aggregate throughput, profilers reveal the physical mechanisms that cause execution to stall. System diagnosis uses two complementary levels of instrumentation: framework profilers and kernel profilers.

Framework profilers

Framework profilers, such as the PyTorch Profiler, capture the logical execution trace of a training or inference iteration. They track operator dispatch on the host CPU, determine whether host preparation overlaps asynchronously with accelerator execution, and identify whether the data loader pipeline starves compute queues. The primary diagnostic metric is the step-time breakdown partitioned across data loading, host launch latency, device compute, and inter-device communication. This breakdown pinpoints which subsystem bounds iteration latency before inspecting individual kernels.

Kernel profilers

Kernel profilers, such as NVIDIA Nsight Systems and Nsight Compute, capture physical execution on accelerator silicon. Nsight Systems constructs a global timeline across CPU threads, CUDA API calls, memory transfers, and kernel durations to expose serialization bubbles. Nsight Compute conducts in-depth analysis of individual kernel invocations, measuring Streaming Multiprocessor (SM) warp occupancy, instruction issue stalls, and memory coalescing efficiency against the hardware roofline. Kernel profilers extract these diagnostics directly from on-chip Performance Monitoring Units (PMUs), capturing hardware counters for HBM byte traffic, L2 cache hit rates, and pipeline stall reasons without distorting execution dynamics through software instrumentation.

A rigorous profiling workflow proceeds top-down. An engineer begins with a framework profiler to isolate the dominant layer in the execution timeline (for example, identifying that a multi-head attention block accounts for 65 percent of step latency). The engineer then applies a kernel profiler to analyze that layer down to the silicon (for example, verifying that the softmax kernel achieves only a small fraction of peak memory bandwidth due to non-coalesced DRAM accesses). This two-stage workflow prevents misdirected optimization by ensuring engineering effort targets the limiting physical bottleneck.

These granular measurements enable precise kernel optimization, but they cannot reveal how components interact when assembled into complete models. Macro-benchmarks address this gap.

12.4.2 Macro benchmarks

Micro-benchmarks verify that isolated operators achieve high compute or memory efficiency. Macro-benchmarks determine whether those gains persist when layers are composed into a complete model. Composing layers introduces physical interactions that isolated benchmarks cannot observe: intermediate activation tensors expand peak memory footprint, sequential kernel launches expose CPU driver dispatch overhead, and memory access streams from preceding operations evict shared cache lines before downstream kernels can reuse them.

Macro-benchmarks serve a central engineering objective: evaluating model architectures and runtime configurations under standardized workloads. Model-level evaluation captures the trade-offs that emerge from layer composition: generalization accuracy on unseen validation data, dynamic memory allocation across batch sizes and context lengths, and aggregate throughput across varied request arrival patterns. These dimensions interact directly: an architecture that achieves higher accuracy may require an activation footprint that forces a smaller batch size under device memory constraints, collapsing the computational throughput that justified its adoption.

Evaluating complete models requires standardized datasets, fixed reference tasks, and reproducible metric definitions. In computer vision, standard benchmarks evaluate models on ImageNet (Deng et al. 2024), measuring top-1 classification accuracy alongside inference latency across fixed batch sizes. In natural language processing, standardized tasks evaluate sequence-to-sequence translation and autoregressive generation, quantifying output quality against token generation rate across defined context lengths.

Several industry-standard suites establish reproducible model-level comparisons across platforms. The MLPerf suite standardizes evaluation across data centers, edge appliances, mobile devices, and microcontrollers, as detailed in section 12.8.4. For resource-constrained embedded targets, EEMBC's

MLMark quantifies inference throughput and energy consumption under strict memory budgets, while the AI-Benchmark (Ignatov and Timofte 2024) suite evaluates mobile systems-on-chip across heterogeneous compute units.

12.4.3 End-to-end benchmarks

End-to-end benchmarks encompass the entire production pipeline, capturing every stage required to ingest raw data, generate model predictions, and deliver results to downstream clients. While micro- and macro-benchmarks focus on accelerator execution, real-world serving pipelines frequently find their primary bottlenecks outside the model itself.

Data preprocessing forms the initial phase of the pipeline, converting raw external inputs into model-ready tensor representations. In computer vision pipelines, image ingestion requires decoding compressed file formats (such as JPEG decompression), resizing, cropping, and color normalization. When executed on host CPUs without hardware acceleration, these preprocessing steps can take several times longer than model inference on a modern accelerator, leaving high-throughput tensor cores idle while waiting for host memory transfers. Similarly, in natural language pipelines, subword tokenization and dynamic sequence padding on the host CPU often bound serving throughput. Postprocessing introduces analogous latency bottlenecks: object detection pipelines require non-maximum suppression (NMS) and bounding-box coordinate transformations, which create sequential execution stalls if not integrated efficiently into the execution graph.

Distributed infrastructure components introduce further performance constraints beyond the compute fabric. Remote storage systems bound training throughput when reading large multimodal datasets, and network interface card (NIC) saturation in distributed clusters throttles collective communication. End-to-end benchmarks must measure pipeline throughput under realistic network latencies, concurrent query arrival patterns, and actual storage I/O constraints to ensure reproducible evaluation of the deployment as an integrated system.

Public benchmark suites rarely measure storage, networking, and compute within a single unified protocol. While MLPerf Training and Inference evaluate end-to-end scenarios more closely than isolated operator tests, their closed divisions deliberately standardize input data loading to isolate accelerator performance from storage and network variability. Consequently, production end-to-end benchmarking remains primarily an internal engineering practice, evaluated through distributed tracing and production telemetry on live query distributions.

12.4.4 Granularity trade-offs and selection criteria

Table 12.4 contrasts the diagnostic focus, scope, and engineering trade-offs across the three benchmarking granularities. Micro-benchmarks isolate low-level operations to guide kernel optimization and hardware selection, macro-benchmarks evaluate complete models to guide architectural choices, and end-to-end benchmarks measure full pipelines to ensure production deployments satisfy real-world service requirements.

Table 12.4: Benchmarking Granularity Levels: Different benchmark scopes target distinct stages of ML system development. Micro-benchmarks isolate individual operations for low-level optimization, macro-benchmarks evaluate complete models to guide architectural choices, and end-to-end benchmarks assess full system performance in production environments.

Component	Micro Benchmarks	Macro Benchmarks	End-to-End Benchmarks
Focus	Individual operations	Complete models	Full system pipeline
Scope	Tensor ops, layers, activations	Model architecture, training, inference	Extract, transform, load; model; infrastructure
Example	Conv layer performance on cuDNN	ResNet-50 on ImageNet	Production recommendation system
Advantages	Precise bottleneck identification, Component optimization	Model architecture comparison, Standardized evaluation	Realistic performance assessment, System-wide insights
Challenges	May miss interaction effects	Limited infrastructure insights	Complex to standardize, Often proprietary
Typical Use	Hardware selection, Operation optimization	Model selection, Research comparison	Production system evaluation

Selecting a single granularity level is rarely sufficient because benchmarking presents an inherent trade-off between diagnostic isolation and production fidelity. Figure 12.3 maps this spectrum,

placing micro-benchmarks at the high-isolation boundary and end-to-end benchmarks at the high-representativeness boundary. Micro-benchmarks pinpoint the exact physical limit of a kernel (such as memory bandwidth saturation or low warp occupancy) but fail to capture inter-layer memory pressure or host overhead. Conversely, end-to-end benchmarks capture true production latency under live traffic distributions but obscure root causes when tail latencies spike.

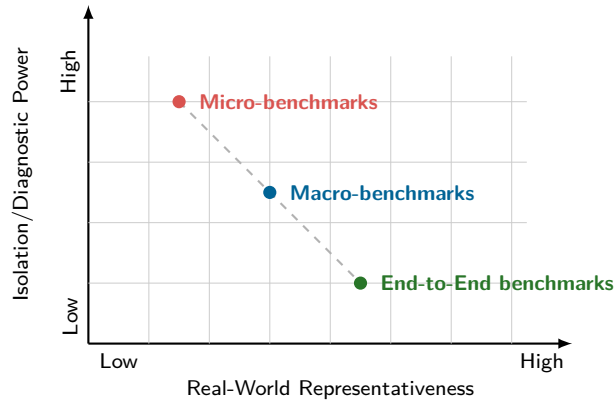


Figure 12.3: Isolation vs. Representativeness: The core trade-off in benchmarking granularity. Micro-benchmarks provide high diagnostic precision but limited real-world relevance, while end-to-end benchmarks capture realistic system behavior but offer less precise component-level insights. Effective ML system evaluation requires strategic combination of all three levels.

This trade-off maps directly onto the D·A·M taxonomy. Micro-benchmarks assess the physical capability of the Machine (*M*) executing a primitive Algorithm (*A*). Macro-benchmarks evaluate how the Algorithm (*A*) orchestrates tensor data flow across composed layers on the Machine (*M*). End-to-end benchmarks evaluate how raw Data (*D*) ingestion from external storage and network infrastructure interacts with both Algorithm and Machine under production load. Systematic performance engineering requires evaluating all three tiers to ensure local kernel optimizations translate into global system speedups.

Selecting the appropriate granularity defines the physical boundary of measurement, but establishing that boundary is only the first step. A complete benchmark also requires specifying the operational ingredients: the task definition, representative datasets, target models, and evaluation metrics. Without these concrete components, measurements at any granularity fail to produce actionable engineering conclusions.

12.5 Benchmark Components

Choosing between micro, macro, and end-to-end granularity determines what a benchmark can diagnose, but every benchmark must specify the task, data, model, metrics, harness, system context, and run rules that make its results interpretable. Micro-benchmarks isolate specific hardware execution limits—such as streaming memory bandwidth or tensor core throughput—using synthetic tensor operations. Macro-benchmarks evaluate full-graph execution over standardized datasets like ImageNet. End-to-end benchmarks capture the complete pipeline, incorporating data ingestion, host-to-device transfers, batch assembly, and inference execution. Across all three granularities, benchmark components cannot be selected in isolation: each component fixes physical and algorithmic boundary conditions for the next.

These components interconnect to form an evaluation pipeline structured by the D·A·M taxonomy (Data, Algorithm, and Machine). The workflow in figure 12.4 traces nine stages of an industrial audio anomaly detection benchmark, spanning initial task definition through quantization to embedded deployment on an ARM microcontroller. Each stage fixes boundary conditions for the next: the problem definition dictates the acoustic sampling rate and input representation (Data); the dataset properties and task objective determine feasible model architectures (Algorithm); and the microcontroller’s hardware limits—such as SRAM capacity, energy envelope, and real-time latency deadlines—dictate INT8 quantization and memory-efficient kernel compilation (Machine). Anomaly

detection illustrates this full-stack coupling directly: a benchmark that measures only classification accuracy or only raw inference latency misses cross-stage interactions, where an engineering decision at any stage narrows the feasible design space downstream.

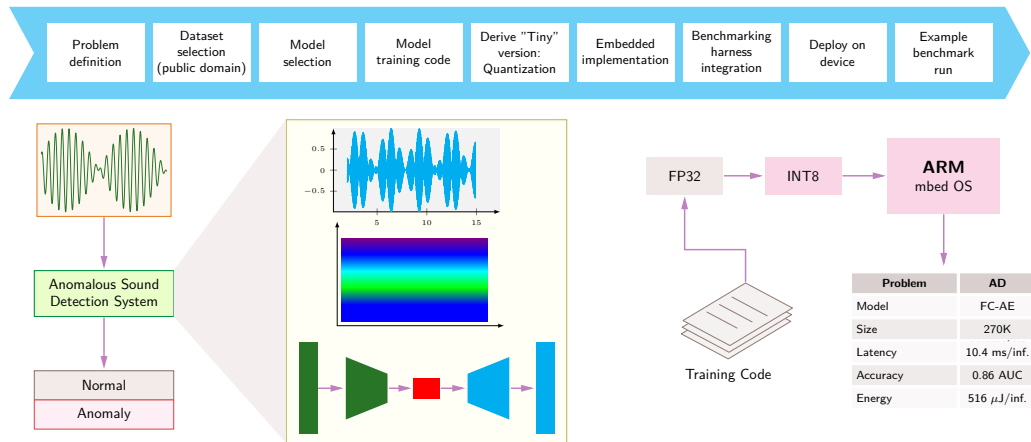


Figure 12.4: Benchmark Workflow Components: Read left to right: each stage fixes assumptions for the next, carrying an audio anomaly-detection task from problem definition to a reproducible embedded-device benchmark run.

Evaluating an ML system requires coupling statistical task metrics with physical efficiency metrics. Quantization and pruning reduce parameter footprint and memory bus traffic, but they alter numerical representations and risk degrading task accuracy. A benchmark that records execution latency without statistical accuracy (such as AUC or classification error) rewards aggressive approximations that produce invalid outputs; conversely, reporting accuracy without latency or memory consumption conceals whether the model satisfies the target system’s physical constraints. Interpreting these results requires relating the workload’s arithmetic intensity—the ratio of operations to bytes transferred from memory—to the machine’s balance point using the Roofline model, identifying whether an optimization relieves a memory-bandwidth bottleneck or a compute-throughput bottleneck.

The measurement harness and run rules provide the experimental controls that make these evaluations reproducible. Without strict execution protocols, measured runtimes reflect operating system and hardware noise rather than algorithmic execution cost. A rigorous benchmark harness enforces cache warm-up iterations to reach steady-state memory behavior, locks processor clock frequencies to prevent dynamic voltage and frequency scaling (DVFS) from introducing latency transients, pins host memory allocations to avoid page-fault overhead, and records tail latency distributions ($p95$, $p99$) alongside mean latency to expose memory bus contention and queuing delays. These operational components ensure that measurements reflect the true physical limits of the system, establishing the foundation for examining each component in detail, beginning with the problem definition.

12.5.1 Problem definition

Every benchmark begins by fixing the computational and operational boundaries of the workload. Without an explicit task specification, hardware and software optimizations cannot be evaluated on equivalent terms: altering an input resolution, a numerical datatype, or a latency deadline fundamentally changes the physical work demanded of the underlying machine. As illustrated by the audio anomaly detection system in figure 12.4, an application transforms raw sensor signals into operational decisions. Across domains—whether natural language processing tasks such as machine translation, question answering (Hirschberg and Manning 2015), and text classification, or computer vision tasks such as object detection and image segmentation (Everingham et al. 2009; Lin et al. 2014)—a complete problem definition establishes three concrete specifications: an input specification, an output specification, and a performance specification.

The input and output specifications establish the data movement contract and define the exact scope of timed execution. The input specification fixes tensor dimensions, batching behavior,

numerical representations (such as FP16 or INT8), and ingestion rates. In the audio anomaly detection pipeline, streaming single-channel audio at 16 kHz in fixed one-second windows establishes a rigid ingestion rate and memory buffer requirement before feature extraction begins. Conversely, the output specification defines the termination boundary of the computation. Benchmarks frequently introduce measurement bias by timing only accelerator forward passes while ignoring the CPU post-processing required to turn raw tensor outputs into usable decisions. A robust output specification defines whether the system emits raw logits, intermediate embeddings, or final binary anomaly flags, accounting for any downstream thresholding or reduction kernels that consume host memory bandwidth.

The performance specification defines the operational envelope that governs whether an optimization is viable. Unconstrained throughput numbers are meaningless if statistical accuracy degrades below acceptable thresholds, and high average throughput cannot rescue a system that violates latency service-level agreements (SLAs). For real-time anomaly detection, the performance specification couples a target quality metric (such as area under the ROC curve) with hardware execution limits: a deterministic tail-latency ceiling (such as a 99th-percentile response time $p_{99} \leq 10$ ms), sustained throughput matching the data arrival rate, and a strict resident memory footprint to prevent out-of-memory faults on memory-constrained accelerators. By defining these three boundaries up front, the benchmark ensures that subsequent stages—from dataset construction to measurement harness configuration—evaluate system optimizations against the true operational constraints of deployment.

12.5.2 Standardized datasets

A task definition is only as good as the data used to evaluate it. Standardized datasets ensure that all models undergo testing under identical conditions, enabling direct comparisons across different approaches—without them, every team would evaluate on private data, making cross-lab comparison impossible. In computer vision, ImageNet (Deng et al. 2024, 2009), COCO (Lin et al. 2014), and CIFAR-10 (Krizhevsky 2009) serve as reference standards; in natural language processing, SQuAD¹² (Rajpurkar et al. 2016), GLUE¹³ (A. Wang et al. 2018), and WikiText (Merity 2016; Merity et al. 2016) fulfill similar roles, each encompassing a range of complexities and edge cases.

Dataset selection is the first place a benchmark can lose contact with deployment reality. In the audio anomaly detection example (figure 12.4), the dataset must include representative waveform samples of normal operation alongside comprehensive examples of anomalous conditions. Domain-specific collections cover different audio tasks: ToyADMOS¹⁴ (Koizumi et al. 2019) supports controlled anomaly-detection research, while Google Speech Commands (Warden 2018) supports keyword recognition. Effective benchmark datasets must balance two competing demands: accurately representing real-world challenges while maintaining sufficient complexity to differentiate model performance. Simplified datasets like ToyADMOS are valuable for methodological development but may not capture the full complexity of production environments.

12.5.3 Model selection

With the task and dataset fixed, the benchmark must specify the model architecture and its baseline implementations. In systems benchmarking, the model defines the execution workload: its computational graph determines arithmetic intensity, memory footprint, and communication topology. Selecting the benchmark model determines whether measured performance variations reflect architectural efficiency, software runtime optimizations, or raw accelerator capacity. Evaluating an architectural modification requires holding the runtime and hardware target constant, as established in chapter 6. Conversely, evaluating hardware platforms or software execution engines—as examined in chapter 7—requires standardizing on a fixed, representative model graph so that throughput differences isolate execution efficiency rather than mathematical changes to the workload.

Baseline models establish the empirical reference point for a target domain. Rather than toy algorithms, systems benchmarks rely on canonical workloads whose computational profiles and memory access patterns are well-characterized. In language processing, BERT¹⁵ serves as a standard baseline because its multi-head self-attention and dense feed-forward blocks exercise both compute-bound matrix multiplications and memory-bandwidth-bound reduction operations. However, a baseline specification cannot stop at the mathematical graph. Two implementations of the identical

¹² | **SQuAD (Stanford question answering dataset):** Introduced in 2016 with more than 100,000 question-answer pairs from Wikipedia (Rajpurkar et al. 2016). AI systems exceeded the SQuAD 1.1 human baseline of 91.2 percent F1 by 2018 (Devlin et al. 2019), but this “superhuman” result illustrates a benchmarking failure mode: the task’s extractive format (answers are text spans within the passage) makes it easier than open-ended question answering, inflating perceived capability relative to production NLP systems.

¹³ | **GLUE (general language understanding evaluation):** Introduced in 2018 as a broad language-understanding benchmark (A. Wang et al. 2018), GLUE was quickly saturated by systems such as BERT (Devlin et al. 2019). This is Goodhart’s Law in action: once GLUE became a target, leaderboard optimization reduced its discriminating power. The pattern motivated harder follow-on evaluations such as SuperGLUE and BIG-bench.

¹⁴ | **ToyADMOS:** Developed by NTT Communications in 2019 for acoustic anomaly detection, containing audio recordings from toy car, toy conveyor, and related miniature-machine operating sounds (Koizumi et al. 2019). The “toy” prefix is intentional: the controlled environment enables reproducible benchmarking but can create a domain gap when models are moved to noisier industrial environments with different machines, sensors, vibration, and background sound.

¹⁵ | **BERT (bidirectional encoder representations from transformers):** BERT-Large (340M parameters) is a language-processing workload in MLPerf Inference. The benchmark fixes the task, data, quality target, and applicable execution scenarios so systems can be compared on the same workload. BERT latency still depends on sequence length, batching, and implementation.

architecture across different frameworks or runtimes often exhibit divergent execution characteristics. Differences in operator fusion—such as fusing normalization and activation functions directly into GEMM output epilogues to avoid round-trips to HBM—as well as differences in kernel launch overhead and memory allocator behavior can exceed the performance impact of minor architectural variations. A benchmark must therefore standardize the runtime implementation alongside the mathematical model.

A benchmark must evaluate the model across two distinct operational regimes, each governed by different physical bottlenecks. In training, the primary constraint is memory capacity and inter-connect bandwidth: storing forward activations, backward gradients, and optimizer states across multiple accelerators limits batch size and demands high-bandwidth collective communication. Training benchmarks therefore measure time-to-accuracy under fixed convergence criteria rather than raw iteration speed alone. In inference, backward passes and optimizer states disappear, shifting the bottleneck to tail latency under strict service-level agreements (SLAs) or memory bandwidth during autoregressive decode. Transitioning a model to production frequently requires precision reduction—quantizing FP32 or BF16 weights and activations to INT8 or FP8. Quantization halves memory traffic and doubles tensor core throughput, but requires calibration to ensure numerical error does not degrade validation quality. Because a model architecture can train efficiently yet fail under strict latency SLAs, a benchmark must evaluate both operational regimes with dedicated, domain-specific metrics.

12.5.4 Evaluation metrics

Evaluation metrics¹⁶ translate raw model behavior into numbers that can be compared, ranked, and used to make engineering decisions. The challenge is choosing the right numbers: a metric that captures accuracy but ignores latency may declare the winner to be a model too slow for production; one that rewards throughput but ignores energy may optimize for a deployment budget that does not exist.

Table 12.5 categorizes evaluation measures by the failure mode each exposes and the deployment context it serves.

Table 12.5: ML Benchmarking Metric Taxonomy: Metrics organized by evaluation category, unit, and primary use case. Accuracy metrics quantify model quality, throughput and latency metrics capture system speed, and efficiency metrics combine multiple dimensions. Selecting the right metric for the deployment context is often more important than optimizing any single metric to its maximum.

Category	Metric	Unit	Primary Use Case
Accuracy	Top-1/Top-5 Accuracy	Percentage	Classification
	mAP (mean Average Precision)	0–1 score	Object detection
	BLEU/ROUGE	0–100 score	NLP generation
	Perplexity	Score (lower = better)	Language modeling
Throughput	Samples/second	Samples/s	Batch inference
	Token throughput	tokens/s	LLM inference
	Time-to-train	Hours/days	Training benchmarks
Latency	p50 latency	Milliseconds	Median response time
	p99 latency	Milliseconds	Tail latency (SLA)
	First-token latency	Milliseconds	LLM responsiveness
Efficiency	Samples/second/watt	Samples/s/W	Energy efficiency
	Accuracy/FLOP	percent/PFLOP	Algorithmic efficiency
	TCO per inference	\$/inference	Economic efficiency

The metrics in table 12.5 expose physical trade-offs governed by the accelerator architecture. Throughput measures aggregate processing capacity, whereas latency measures individual request completion time. These two objectives conflict directly at the hardware boundary. Maximizing throughput requires aggregating requests into larger batches to amortize weight-loading memory bandwidth across multiple inputs, raising arithmetic intensity on tensor units. That batching mechanism inflates per-sample latency: individual requests wait in arrival queues while a batch forms, and larger matrix operations take longer to complete. Furthermore, arithmetic mean latency

¹⁶ | **Metric:** In mathematics, a metric is a distance function satisfying strict axioms including the triangle inequality. ML borrows the term loosely for quantitative measures such as BLEU and perplexity, which are scoring rules rather than mathematical metrics. Leaderboard rankings can change when the evaluation protocol, dataset slice, or metric weighting changes, making the choice of metric an engineering decision that shapes which system wins, not just how performance is measured.

masks tail behavior. Queuing jitter, memory allocation stalls, and accelerator thermal throttling cause execution times to diverge; a system with a 10 ms mean latency can exhibit a 500 ms 99th-percentile (p99) latency that violates service-level agreements. Benchmarks must therefore report distribution percentiles (p50, p95, p99) rather than averages. Compound metrics such as samples/second/watt compress multiple dimensions into a single scalar, facilitating high-level comparisons but obscuring where bottlenecks reside. Reporting atomic metrics (latency distributions, throughput, and power draw) alongside compound figures isolates which physical constraint limits the system.

Task-specific metrics evaluate the algorithmic dimension, quantifying whether the model solves the statistical problem. Classification workloads evaluate accuracy (the fraction of correct predictions), precision, recall, and the F1 score (Sokolova and Lapalme 2009). Precision and recall isolate true positive rates from false alarms, which is essential when class distributions are imbalanced. Continuous estimation tasks rely on Mean Squared Error (MSE) or Mean Absolute Error (MAE), while sequence generation uses specialized scoring rules such as BLEU¹⁷ to measure modified n-gram precision against human reference translations (Papineni et al. 2002).

Task metrics evaluate algorithmic quality in isolation, but deployment viability depends on physical machine constraints. Model capacity, measured in parameter count and memory footprint, determines whether weights fit into on-chip SRAM or accelerator memory, or whether they spill across high-latency interconnects. Per-inference execution latency dictates whether a system meets real-time deadlines, while energy consumption per inference (measured in joules or milliwatts) bounds thermal dissipation and battery life. The operational challenges of maintaining these metrics across production environments are explored in deployment strategies (chapter 14).

Benchmarks must also account for measurement sensitivity across software runtimes. As established in chapter 8, different frameworks handle loss computation, operator fusion, and gradient accumulation differently, altering reported benchmark numbers. Subtle implementation differences—such as whether evaluation-mode batch normalization uses population statistics or running mini-batch moments, or how non-associative floating-point additions are reordered in reduction kernels—shift measured accuracy and latency enough to invert benchmark rankings when candidate models differ by narrow margins.

This interdependence between statistical quality and hardware constraints is evident in anomaly detection. In industrial monitoring, extreme class imbalance renders raw accuracy deceptive: if anomalies constitute only 0.1 percent of operating samples, a trivial model that constantly predicts normal operation achieves 99.9 percent accuracy while failing to detect any faults. The benchmark must instead report Area Under the ROC Curve (AUC) to evaluate discrimination across decision thresholds, alongside physical resource metrics.

This multi-metric evaluation approach governs the anomaly detection pipeline, which balances four simultaneous constraints: a model size of 270K parameters that fits within embedded SRAM, a processing latency of 10.4 ms per inference to maintain real-time sampling, a detection accuracy of 0.86 AUC, and an energy consumption of 516 μ J per inference within the target battery budget. Evaluating across all four axes prevents declaring a model superior when its accuracy gain stems from an architecture too large to fit in memory or too power-hungry to survive sustained deployment.

12.5.5 Benchmark harness

Metrics define *what* to measure; the benchmark harness determines *how* to measure it. The harness is the software and instrumentation scaffold that feeds inputs to the system under test, coordinates execution, collects timing and power traces, and enforces reproducibility. In an ML system, measuring execution down to the metal requires accounting for the asynchronous execution boundary between the host CPU and the accelerator. When host software initiates an inference or training step, the framework runtime merely enqueues commands onto an accelerator work queue (such as a CUDA stream or hardware command buffer) and returns control immediately. A naive timer wrapping the host-level call measures only this sub-microsecond queue dispatch latency rather than actual device execution. A rigorous harness inserts hardware synchronization barriers or reads on-device timestamp events, ensuring that recorded timings reflect complete kernel execution and memory transfers across the bus.

Harness architecture must align with the operational regime of the deployment target. In server environments, the harness generates concurrent request traffic using an open-loop load generator,

¹⁷ | **BLEU (bilingual evaluation understudy):** Introduced by IBM in 2002, BLEU measures translation quality through modified n-gram precision with a brevity penalty against reference translations (Papineni et al. 2002). BLEU is a canonical example of Goodhart’s Law in ML: optimizing for n-gram matches can reward surface-level word overlap even when meaning, fluency, or deployment usefulness diverges from the target.

18 | Poisson distribution: Named after Siméon Denis Poisson, who formalized it in 1837 while modeling wrongful conviction rates in French courts. The distribution models independent events at a constant average rate (λ_{arr}) and is a common baseline for server request arrivals. Real ML serving traffic often violates its assumptions through burstiness, correlation, and time-varying rates. A Poisson harness can therefore misestimate tail latency unless traces or stress cases cover those production patterns.

often parameterizing inter-arrival times with a Poisson distribution.¹⁸ An open-loop harness issues requests according to an independent arrival schedule regardless of server responsiveness. This decoupling exposes queuing delays and tail latency (such as p99 response times) when arrival bursts saturate available compute. In contrast, a closed-loop harness that waits for a response before dispatching the next request introduces coordinated omission, artificially pacing the traffic and hiding queue buildup during periods of server degradation. Conversely, offline batch processing eliminates arrival scheduling entirely. The harness pre-stages large input batches in memory to saturate accelerator execution units, maximize memory bandwidth utilization, and measure peak sustained throughput.

In embedded and edge deployments, workloads are neither stochastic network streams nor unconstrained batch queues. Instead, inputs arrive sequentially from physical sensors (such as an audio analog-to-digital converter or an image sensor) governed by fixed hardware sampling clocks. The harness must inject these sensor streams through direct memory access (DMA) into limited on-chip SRAM buffers, verifying that each inference completes within the inter-sample arrival deadline to prevent buffer overflow. Furthermore, edge harnesses must integrate physical instrumentation, such as shunt resistors or external power analyzers, to sample voltage and current across active execution cycles and low-power sleep states, capturing the true energy consumed per inference.

Reproducibility requires isolating the benchmark from runtime and environmental variance across evaluation runs. Modern accelerators and runtime frameworks exhibit substantial cold-start transients. Initial inference passes trigger on-demand memory allocations, dynamic just-in-time (JIT) compilation, driver initialization, and weight transfers from secondary storage into accelerator memory. A reproducible harness must execute unmeasured warm-up iterations until caches populate, memory allocators stabilize, and silicon reaches its thermal steady state before initiating the timed measurement loop. To prevent measurement drift, the test environment must lock accelerator and CPU clock frequencies, disabling dynamic voltage and frequency scaling (DVFS) and terminating competing background processes. Finally, telemetry logging must minimize the probe effect by recording execution timestamps and performance counters into non-blocking, pre-allocated memory buffers. This ensures that measurement instrumentation does not steal memory bandwidth or alter the pipeline execution of the system under test.

12.5.6 System specifications

Complementing the harness that controls execution conditions, system specifications record the computational environment: the physical hardware and software stack that establishes the Machine (M) boundaries of the system. Without complete specifications, a reported throughput or latency measurement cannot be interpreted. An execution metric reflects the interaction between an algorithm's arithmetic intensity and a machine's physical limits: peak arithmetic throughput, memory hierarchy bandwidth, and interconnect latency. A throughput figure reported without accelerator model, memory bandwidth, or operating clocks conceals whether an observed speedup stems from algorithmic efficiency or upgraded silicon.

Rigorous hardware specifications capture the constraints that dictate kernel execution and data movement. At the accelerator level, documentation must record the microarchitecture, compute capability, active core counts, base and boost clock frequencies, and power limits (thermal design power (TDP)), alongside the device memory hierarchy: HBM capacity, memory bus width, and peak memory bandwidth. For multi-accelerator configurations scaling from one to eight devices, specifications must delineate the host-to-device bus interface (such as PCIe lanes) and the inter-accelerator fabric—whether communication traverses point-to-point links (such as NVLink) or shared PCIe switches. Because distributed collective operations such as all-reduce are bounded by interconnect topology and bisection bandwidth, omitting interconnect specifications obscures the source of communication bottlenecks.

The software stack is equally decisive, dictating how mathematical operations map onto execution units. Specifications must record the operating system kernel, device drivers, low-level compute runtimes (such as CUDA or ROCm), and vendor math libraries (such as cuBLAS or CUTLASS). At the framework layer, tracking runtime versions (such as PyTorch or JAX), execution modes (eager interpretation versus graph-level compilation via `torch.compile`), and kernel fusion optimizations

separates framework overhead from model performance. Finally, specifications must quantify operational power and energy metrics, including sustained board power, thermal throttling events, and energy consumed per token or sample. Documenting these operating envelopes ensures that measured speedups are not masked artifacts of unconstrained power draws or transient clock boosting.

12.5.7 Run rules

System specifications describe *what* the benchmark runs on; run rules govern *how* it runs. These procedural constraints make results interpretable and repeatable, which is harder than it sounds in a field where stochastic processes (weight initialization, data shuffling, and dropout masks) mean that two runs on identical hardware can produce different numbers. A protocol may fix seeds and data order when isolating system effects, or require repeated runs and quality thresholds when stochastic variation is part of the workload. In either case, the seed policy and known sources of nondeterminism must be explicit.

Hyperparameter documentation is equally critical. A learning-rate change can shift convergence and final accuracy, so a reproducible result records every configuration setting that can affect the outcome. Dataset versions, splits, and preprocessing must likewise be identified. When privacy or licensing prevents sharing data directly, the report must state that limitation and preserve enough provenance and transformation detail to judge comparability.

Code provenance completes the reproducibility chain. Strong benchmark protocols preserve the implementation version—not just the model, but the relevant preprocessing, training, and evaluation code—and disclose whether that code can be shared. Reference suites may distribute containerized environments that encapsulate dependencies and configurations, while experimental logs retain training metrics, checkpoints, and any mid-run adjustments. Together, these records turn a one-time measurement into evidence another team can inspect and, when access permits, reproduce.

12.5.8 Result interpretation

Producing raw benchmark numbers requires only execution; interpreting them requires understanding the conditions that produced them, the statistical confidence behind them, and the operational boundaries that determine whether the numbers translate to production behavior. A single reported metric—such as latency or accuracy—rarely reveals whether an optimization succeeds in deployment. Evaluating an edge vision model on a resource-constrained device illustrates how throughput, memory footprint, and model fidelity interact under strict hardware boundaries.



Example 12.2: Benchmarking a vision model for edge deployment

Scenario: An edge AI team evaluates MobileNetV2 for a wildlife camera trap on a Raspberry Pi 4 under strict latency and model size limits (Sandler et al. 2018).

Diagnosis: FP32 execution yields 120 ms latency (8.3 FPS) and 14 MB size. INT8 quantization speeds up inference by 3.4× to 35 ms (28.6 FPS) and reduces model size by 4×, at a cost of 0.9 percentage points Top-1 accuracy.

Systems lesson: Edge model benchmarking requires evaluating multi-dimensional trade-offs across latency, accuracy, and memory payload. Quantization enables high-throughput edge execution when modest accuracy drops meet product requirements.

Translating benchmark numbers into engineering decisions requires two methodological checks: architectural parity and statistical rigor. First, comparisons must enforce strict architectural parity. Benchmarking ResNet-50 against MobileNet conflates convolutional operator efficiency with parameter count. Similarly, comparing FP32 against INT8 execution conflates algorithmic precision with memory traffic and execution unit throughput: INT8 reduces DRAM transfers by 4× and executes on dedicated integer arithmetic pipelines. Valid comparisons require holding the model architecture, numerical format, batch size, hardware SKU, and software runtime constant. Second, measurements must isolate steady-state execution from initialization artifacts. A single run conflates cold-start overhead—kernel compilation, page faults, and cache warming—with sustained throughput. Conversely, prolonged stress tests can induce thermal throttling, depressing accelerator clock

frequencies below base specifications. Statistically meaningful results require discarding warmup iterations, recording variance across multiple trials, and reporting confidence intervals or latency percentiles rather than isolated averages. Applying these criteria to an isolated throughput claim demonstrates how omitted execution parameters obscure real systems behavior.

Systems Perspective 12.6: Interpreting a benchmark claim

A vendor claims “Our system achieves 10,000 inferences/second on ResNet-50.” Taken alone, the number cannot support deployment planning because the conditions that determine its meaning are unstated.

Four unstated conditions determine what the claim means:

1. Batch size: Large batches amortize weight transfers and maximize compute utilization but inflate queuing delay; batch 1 minimizes latency but leaves execution units underutilized.
2. Precision: INT8 execution exploits higher-throughput integer pipelines and cuts memory traffic, but requires calibration that can compromise numerical fidelity.
3. Measurement boundary: The measurement window must state whether it isolates pure accelerator kernel execution or includes host-to-device transfers, JPEG decoding, and tensor format conversions.
4. Accuracy: The claim must state whether the optimized model retains baseline validation accuracy (such as 76.1 percent Top-1 on ImageNet) or trades task quality for raw execution speed.

Example: “10,000 inferences/second on ResNet-50 at batch size 32, INT8 precision, 76 percent Top-1 accuracy, including JPEG decoding, on NVIDIA H100 at 700 W TDP.”

Systems insight: Evaluating whether a performance difference is meaningful requires both statistical rigor and contextual validation. A throughput figure lacking batch size, precision, measurement scope, and accuracy targets is a marketing metric rather than an engineering specification.

Beyond vendor claims, deployment constraints dictate which metrics govern system viability. In safety-critical applications such as medical imaging or autonomous driving, a 1 percent accuracy gain justifies substantial computational cost; in interactive search or speech transcription, tail latency within a tight service-level agreement dominates, rendering minor accuracy gains irrelevant if they breach latency budgets. Systems evaluation must also guard against *benchmark overfitting*, wherein an optimization targets the idiosyncratic computational patterns of a standardized suite at the expense of production workloads. For example, aggressive kernel fusion or operator tiling tuned exclusively for fixed sequence lengths in a benchmark harness can degrade throughput when deployed against dynamic, variable-length user requests. Robust interpretation requires validating performance across diverse input distributions, perturbed batch sizes, and representative operational duty cycles.

12.5.9 Example benchmark

The anomaly detection pipeline in figure 12.4 illustrates how these benchmark components interact at the output stage. The benchmark produces three complementary measurements: a model size of 270K parameters with 10.4 ms per inference (computational resources), a detection accuracy of 0.86 AUC in distinguishing normal from anomalous audio patterns (task effectiveness), and an energy consumption of 516 μ J per inference (operational efficiency).

Which of these metrics governs viability depends entirely on the deployment environment and its physical constraints. Energy per inference dictates battery life on untethered edge devices and bounds power distribution limits and cooling costs in server racks. Model size determines whether weights reside entirely within fast on-chip SRAM or spill into off-chip DRAM, while in multi-tenant cloud accelerators, parameter volume dictates how many model replicas fit within HBM. Inference latency determines whether the system satisfies real-time deadlines on streaming audio without dropping frames, whereas batching trades per-sample latency for higher throughput by amortizing weight-transfer overhead across multiple inputs. These metrics expose direct engineering trade-offs:

reducing model size below 270K parameters reduces memory traffic and execution time, but risks degrading the 0.86 AUC detection accuracy. Whether these measurements constitute a passing benchmark depends on the operational envelope; the benchmark specification standardizes the evaluation protocol, but application requirements dictate the acceptance thresholds.

Two benchmark categories recur often enough across the optimization pipeline to warrant specialized checklists: compression benchmarks, which a pruned or quantized model must pass before deployment, and mobile and edge benchmarks, which power and thermal boundaries strictly constrain. Each composes the task, data, model, metrics, harness, and run rules while enforcing domain-specific limits. Compression validation requires a multi-metric protocol that balances size reduction against accuracy and latency (section 12.11.1.3), while edge deployment requires sustained-power measurement under realistic thermal and duty-cycle constraints (section 12.9).

12.5.10 Compression benchmarks

Neural network compression (pruning, quantization, knowledge distillation, and architecture optimization) requires specialized benchmarks because compression reshapes the trade-off landscape: every byte saved or operation eliminated must be weighed against potential accuracy loss and hardware compatibility. The most basic compression metric is raw size reduction: parameter count, memory footprint in bytes, and compressed storage requirements. Size alone, however, is misleading. On ImageNet, MobileNetV2 achieves approximately 72 percent top-1 accuracy with 3.5M parameters vs. ResNet-50's 76 percent accuracy with 25.6M parameters, about 7.3× fewer parameters at comparable accuracy, or roughly 6.9× more accuracy per parameter (Sandler et al. 2018; He et al. 2016a).

Pruning benchmarks must distinguish between structured and unstructured approaches, because they produce qualitatively different results on real hardware. Structured pruning removes entire neurons or filters, yielding smaller dense operations that conventional kernels can exploit (H. Li et al. 2017). Unstructured pruning eliminates individual weights and can produce very sparse models, but realizing actual speedups requires specialized sparse computation support—meaning benchmark protocols must specify hardware platform and software implementation (Han et al. 2015; Gale et al. 2019).

Quantization benchmarks evaluate precision reduction across data types. In the illustrative MobileNetV2 scenario, INT8 reduces raw weight storage by 4× and the assumed latency values yield the displayed speedup; realized performance depends on the hardware and implementation. The precision-accuracy trade-off is analyzed in section 12.8.2 and the energy implications in section 12.9.1. Mixed-precision approaches push further by applying different precision levels to different layers: critical layers retain FP16 while computation-heavy layers use INT8 or INT4, enabling fine-grained efficiency optimization. Knowledge distillation adds another dimension: a smaller student model can preserve much of a teacher's behavior while reducing size and inference cost, but benchmarking must verify that the student generalizes rather than merely memorizing the teacher's outputs (Hinton et al. 2015).

Critically, acceleration factors vary dramatically across hardware platforms: sparse models, reduced-precision models, and efficient architectures only deliver speedups when the target runtime has kernels, memory layouts, and accelerator support that exploit them. Current benchmark suites like MLPerf focus primarily on standardized reference models, while production deployments often use compressed or hardware-specific variants. This gap between what benchmarks measure and what production actually runs remains one of the field's most consequential blind spots.

12.5.11 Mobile and edge benchmarks

Unlike data centers with kilowatt-scale power delivery and active cooling infrastructure, mobile and edge deployments operate within strict milliwatt- to watt-scale power envelopes and passive thermal dissipation limits. Benchmarking these systems requires navigating an interdependent triangle of power consumption, inference latency, and model accuracy, where optimizing any two typically degrades the third. Table 12.6 contrasts these operational constraints against cloud environments.

Table 12.6: Edge vs. Cloud Deployment Constraints: Typical cloud deployments treat power as an operating cost and latency as a service target, while edge deployments often face tighter energy and response-time limits that constrain achievable accuracy.

Constraint	Cloud Impact	Edge Impact
Power	Operational cost	Device energy budget
Latency	Service-level target	Local response deadline
Accuracy	Task-quality target	Balanced with power/latency

Consider a smartphone camera executing real-time object detection at 30 frames per second within a 3 W to 5 W thermal envelope. In this setting, an efficient architecture such as MobileNet serves as a realistic benchmark target where a larger ResNet model would violate thermal limits, despite the larger model’s superior accuracy in cloud evaluations. An edge benchmark must therefore evaluate accuracy jointly with sustained power and thermal dissipation. The peak-versus-sustained gap established in section 12.1 turns acute at the edge for a physical reason absent in the data center: a passively cooled device cannot shed the heat of continuous inference indefinitely through chassis conduction. As junction temperatures rise, the dynamic thermal management (DTM) controller scales down operating voltage and clock frequency, causing sustained throughput to drop sharply below burst-mode peaks. That thermal mechanism, not measurement sloppiness, makes edge benchmarking a categorically different exercise from server benchmarking.

 Example 12.3: Benchmarking the edge

Scenario: An edge engineering team benchmarks a smart doorbell chip with a vendor-advertised “1 W AI” power specification.

Diagnosis: Early burst-mode testing matches vendor claims, but continuous inference loop execution builds up junction heat, triggering thermal throttling and dropping steady-state throughput.

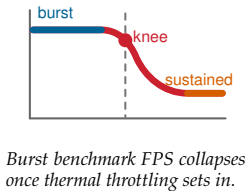
Systems lesson: Short burst benchmarks obscure long-term thermal throttling. Benchmarking edge ML workloads requires continuous, sustained execution testing to reflect true operational hardware limits.

 Systems Perspective 12.7: Edge benchmark reality check

When evaluating edge hardware claims, four factors determine whether vendor numbers translate to real-world performance:

1. **Peak vs. sustained:** A vendor may advertise 45 TOPS peak throughput while a sustained thermal run delivers closer to 20 TOPS. Always benchmark under sustained workloads longer than 30 s.
2. **Power at idle vs. active:** In this scenario, a device consuming 50 mW idle and 2 W active could report active draw for marketing, but if the application runs inference 1 percent of the time, effective power draw is ~69.5 mW, not 2 W.
3. **Thermal envelope:** Edge devices often operate inside a narrow TDP envelope. Exceeding it triggers throttling, so benchmark reports omitting thermal conditions are incomplete.
4. **End-to-end vs. accelerator-only:** NPU benchmarks often exclude data transfer overhead. Moving image data from camera to NPU and back can exceed inference time for small models.

Because passive thermal dissipation caps sustained power draw, mobile SoC architects cannot rely on high-frequency monolithic cores. Instead, they divide execution across specialized compute blocks tailored to distinct arithmetic patterns. This hardware specialization shifts edge benchmarking from evaluating an isolated core to characterizing multi-processor coordination across the entire SoC.



12.5.11.1 Heterogeneous processor coordination

A modern mobile SoC packages CPUs, GPUs, digital signal processors (DSPs), and neural processing units (NPUs) onto a single die sharing system memory and a common thermal budget. Each processing unit targets a distinct compute pattern: CPUs handle irregular control flow, dynamic batching, and sequential post-processing; GPUs execute parallel floating-point operations and unquantized fallback layers; DSPs process low-power streaming vector mathematics for always-on sensing; and NPUs execute dense, quantized matrix multiplications (typically INT8 or INT4) with specialized systolic arrays or multiply-accumulate engines.

Benchmarking must evaluate pipeline orchestration rather than isolated kernel throughput. In an end-to-end voice assistant, for example, an ultra-low-power DSP continuously monitors microphone audio for a wake-word; upon detection, the system wakes the NPU to execute acoustic feature extraction and speech recognition, before passing token sequences to the CPU for language understanding and dialogue logic. Single-processor benchmarks evaluate only the isolated compute time of each stage, missing three dominant sources of system overhead: the latency of waking accelerators from deep sleep states, the bus contention when multiple engines access shared DRAM simultaneously, and the memory bandwidth consumed by transposing tensor layouts (such as converting NCHW CPU layouts into tiled NHWC formats required by the NPU). Benchmarks that report only compute time obscure these data movement and scheduling penalties.

12.5.11.2 Battery and thermal benchmarking

Battery drain depends on total energy consumed over time rather than peak instantaneous power. Computational photography bursts consume multiple watts for hundreds of milliseconds during image capture, whereas background sensor processing must remain within a milliwatt-scale budget to deliver multi-day battery life. Characterizing battery impact requires measuring the total energy consumption $E = \int P(t) dt$ across representative operational profiles.

The dominant operational variable is the workload duty cycle α , defined as the fraction of time the accelerator actively runs inference. For an active power draw P_{active} and standby power draw P_{idle} , the average power is:

$$P_{\text{avg}} = \alpha P_{\text{active}} + (1 - \alpha) P_{\text{idle}}$$

For intermittent workloads like a smart doorbell camera ($\alpha \ll 0.01$), standby power P_{idle} and memory retention power dictate battery lifetime, making active inference efficiency secondary. For continuous pipelines like video analytics ($\alpha \approx 1.0$), active inference energy $E_{\text{inf}} = P_{\text{active}} \cdot t_{\text{inf}}$ dominates. Furthermore, elevated operating temperatures increase semiconductor leakage current, causing static power dissipation to climb as the device heats up. Benchmarking battery impact therefore requires measuring both active and standby power across the operational duty cycle while monitoring junction temperature over extended multi-minute runs.

12.5.11.3 Edge-cloud coordination

When edge devices offload computation to remote servers over 5G or Wi-Fi networks, benchmarking must evaluate the combined edge-cloud pipeline rather than local execution in isolation. While cellular standards such as URLLC¹⁹ define aggressive radio-interface latency targets, real-world cellular and Wi-Fi channels introduce packet jitter, tail latency, and transient disconnections. Workload splitting benchmarks must therefore evaluate four system behaviors under emulated network impairment: the decision boundary for local versus remote execution, round-trip transmission overhead for intermediate activations versus raw sensor inputs, fallback mechanisms when network connectivity drops (such as degrading to a compact on-device model), and privacy policies restricting raw data egress.

In safety-critical environments governed by Automotive Safety Integrity Level (ASIL) standards, this split between local and remote compute is bounded by strict timing deadlines. An autonomous vehicle perception stack operating on a 10 ms to 30 ms control-loop deadline cannot tolerate the multi-hundred-millisecond tail latency of wide-area cellular uplinks. Safety-critical inference must execute entirely on local, temperature-hardened automotive silicon with multi-sensor synchronization, reserving edge-cloud uplinks strictly for non-time-critical telemetry and mapping updates.

Across both cloud data centers and edge devices, benchmarking methodology divides along a fundamental operational boundary: whether the system is executing training or inference. Training

¹⁹ | **URLLC (ultra-reliable low-latency communication):** ITU-R defines a 1 ms radio-interface user-plane latency target and 99.999 percent successful delivery within 1 ms for a 32-byte packet under specified test conditions. These radio-interface targets are not a sub-1-ms end-to-end application guarantee. Edge-inference benchmarks should report radio or network latency and compute latency separately and together, alongside model quality.

requires forward passes, storing intermediate activations in memory, executing backward passes to calculate gradients, and updating model weights across distributed interconnects. Inference executes forward passes only, allowing runtimes to discard intermediate activations immediately, fuse adjacent operator kernels, and quantize weights to low-precision integer formats. Because the memory access patterns, arithmetic intensities, and computational dependencies of training and inference diverge completely, each requires a distinct benchmarking framework.

12.6 Training vs. Inference

The same accelerator can fail in opposite operational regimes: a training run can stall when inter-accelerator gradient synchronization saturates the interconnect, while a serving deployment can violate its SLO when request bursts create queueing delays. Training and inference impose evaluation requirements so divergent that separate benchmarking suites exist for each: MLPerf Training and MLPerf Inference (Mattson et al. 2020; Reddi et al. 2019). Theoretical peak TFLOP/s provides little insight into either workload. Training iteratively refines parameters over billions of samples (chapter 8), demanding sustained high-power compute, large memory capacity for optimizer states, and efficient collective communication across accelerators. Inference evaluates fixed parameters against incoming queries (chapter 13) under bounded response times, demanding bounded tail latency, minimal cold-start initialization latency, and high energy efficiency; chapter 14 connects these serving constraints to production deployment and monitoring.

This division alters memory allocation and arithmetic intensity. In training, device memory must store model parameters, gradients, optimizer states (such as momentum and variance accumulators), and intermediate activations retained for backpropagation. These dynamic state requirements create memory pressure that exceeds weight-only footprints by an order of magnitude, modulated by activation checkpointing, numerical precision, and batch dimensions. Training relies on mixed-precision arithmetic and gradient compression to alleviate capacity and interconnect bottlenecks. Inference discards the backward graph and optimizer states entirely; memory demand is dictated by static weights and dynamic execution buffers (such as key-value caches in autoregressive models). Serving workloads can therefore apply aggressive precision reduction (detailed in section 12.8.2), post-training quantization, and weight pruning. Hardware utilization reflects this operational contrast: training workloads maximize accelerator arithmetic saturation through large batch sizes, whereas inference engines must process unpredictable, low-concurrency request arrival distributions that leave execution units underutilized, as modeled by the roofline analysis in section 12.3.2.

Energy consumption follows contrasting accounting regimes across the two phases. Training energy represents a capital investment amortized across the operational lifetime of the model; large training runs consume thousands of megawatt-hours (training GPT-3 required approximately 1,287 MWh) (Patterson et al. 2021). Inference energy accumulates continuously with request volume, dominating total lifecycle cost as query counts scale into millions. Per-query energy obeys the physical identity $E = P \times t$, where P is average operational power and t is serving latency. For example, if measured average accelerator power during the inference window is 300 W, a 10 ms inference uses $300W \times 0.01s = 3J$, or about 0.0008 Wh; at 100 ms, that becomes about 0.0083 Wh. An accelerator's TDP rating represents a cooling ceiling rather than dynamic power consumption, making empirical runtime power measurement essential for accurate benchmarking.

These divergent physical requirements shape benchmark methodology through the D-A-M lens. Training benchmarks evaluate whether algorithmic updates converge efficiently when scaled across distributed accelerator interconnects under continuous data ingestion, where the primary figure of merit is wall-clock time to a target accuracy. Inference benchmarks evaluate whether a fixed algorithmic artifact satisfies latency percentiles (such as p_{95} and p_{99} response times) and throughput targets under bursty query traffic on provisioned hardware. Because algorithmic convergence during training establishes the upper bound on model accuracy, evaluating training efficiency precedes inference characterization in the benchmarking lifecycle.

12.7 Training Benchmarks

In an illustrative procurement failure, a team purchases a larger GPU cluster expecting proportional training-speed gains, only to discover that communication overhead and memory bottlenecks limit the actual speedup. **Training benchmarks** exist to catch this kind of gap before procurement.

They divide into three categories: convergence metrics that measure learning progress, throughput metrics that measure computational efficiency, and scalability metrics that measure distributed performance.



Definition 12.3: ML training benchmarks

ML training benchmarks are machine learning system benchmarks that measure the time to reach a target quality metric (for example, a specified validation accuracy or loss threshold) on a fixed dataset and model, quantifying the rate of convergence per unit of resource.

1. **Significance:** Training benchmarks reveal large gaps invisible to hardware specs. Holding the model and quality target fixed, the time to convergence can vary widely across hardware-software stacks because training performance depends on the full pipeline: data loading (D_{vol}/BW), compute utilization (η_{hw}), gradient synchronization (L_{lat}), and fault recovery overhead. A peak FLOP/s spec sheet captures none of these interactions.
2. **Distinction:** Unlike inference benchmarks, which measure per-query latency and throughput under load, training benchmarks measure time-to-accuracy across the full optimization loop: data loading, forward pass, backward pass, gradient synchronization, and optimizer step. The binding constraint shifts from compute (R_{peak}) at small scale to communication (BW) at large scale.
3. **Common pitfall:** A frequent misconception is that training benchmarks measure “how fast the GPU runs.” At large scale, interconnect bandwidth (BW) for gradient synchronization and fault tolerance overhead (checkpoint I/O, straggler mitigation) often dominate the benchmark result more than peak FLOP/s.

Training benchmarks validate whether hardware acceleration delivers promised training throughput. The GPU clusters, TPU pods, and distributed training strategies examined in chapter 11 all claim dramatic speedups, and training benchmarks reveal which claims hold under realistic workloads. They evaluate how hardware configurations, data loading mechanisms, and distributed training strategies perform when training production-scale models. These benchmarks are vital because training represents the largest capital expenditure in ML systems, and only rigorous time-to-accuracy measurement reveals whether that capital delivers proportional value rather than dissipating into scaling inefficiencies, memory bottlenecks, or communication overhead.

For instance, large-scale models like OpenAI’s GPT-3²⁰ (Brown et al. 2020), which consists of 175B parameters trained on approximately 570 GB of filtered CommonCrawl text (from a ~45 TB raw dataset, combined with other sources to form 300B training tokens), highlight the immense computational demands of modern training. Standardized ML training benchmarks provide systematic evaluation of the underlying systems to ensure that hardware and software configurations can meet these unprecedented demands efficiently.

12.7.1 Training benchmark motivation

MLPerf Training (Mattson et al. 2020; MLCommons 2024c) provides the standardized framework for this kind of time-to-quality measurement. Figure 12.5 shows that performance improvements across successive benchmark versions have outpaced the plotted Moore’s Law baseline, with some workloads achieving large multi-year speedups (Tschand et al. 2024). The comparison illustrates a core principle: what gets measured gets improved. The standardized benchmarking framework creates competitive pressure that drives rapid optimization across the entire ML computing stack.

Beyond charting that progress, training benchmarks uncover the inefficiencies that systematic evaluation makes visible: slow data loading, underutilized accelerators, excessive memory overhead, and communication bottlenecks that erode scaling efficiency. The theoretical hardware capabilities established in chapter 11 (for example, GPU TFLOP/s, TPU tensor throughput) only translate to actual training speedups when benchmarks verify them under realistic conditions.

Training benchmarks serve four interconnected functions. First, they enable hardware and software optimization by providing vendor-neutral comparisons across accelerator architectures and frameworks (TensorFlow, PyTorch) on standardized tasks, guiding hardware selection for data

²⁰ | **GPT-3 (Generative Pre-trained Transformer 3):** OpenAI’s 2020 language model (175B parameters, 300B training tokens) consumed an estimated 3,640 petaFLOP-days on 10,000 V100 GPUs (Patterson et al. 2021). This scale illustrates why training benchmarks are essential for predicting whether a planned training run is operationally viable before committing the compute.

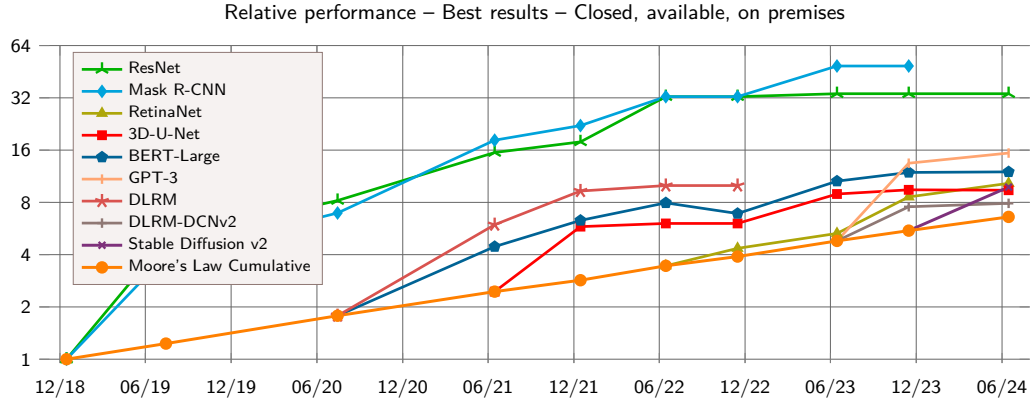


Figure 12.5: MLPerf Training Progress: Workload lines report cumulative relative performance across benchmark versions against the plotted cumulative Moore’s Law baseline. Every workload’s latest point exceeds the baseline at the same date, and Mask R-CNN reaches the largest plotted gain at 48.8×; the comparison captures combined hardware and software progress, not hardware scaling alone. Source: (Tschand et al. 2024).

21 | Mixed-precision training: Uses lower precision for most arithmetic while preserving higher-precision accumulation where needed (Miciekevicius et al. 2017). The benchmarking consequence: mixed-precision and full-precision runs are not directly comparable because reduced memory traffic and larger feasible batch sizes can change convergence dynamics. MLPerf addresses this by fixing the accuracy target, making time-to-accuracy the comparable quantity regardless of precision strategy.

centers and cloud environments. Software optimizations including mixed-precision training²¹ and memory-efficient data loading are similarly quantified. Second, they evaluate scalability: adding GPUs should reduce training time proportionally, but communication overhead, synchronization latency, and memory bottlenecks limit scaling efficiency in practice. Training benchmarks quantify these losses, revealing whether infrastructure investments deliver proportional returns. Third, they provide cost and energy accountability: with large-scale training runs consuming thousands of megawatt-hours, benchmarks that track cost per training run and power consumption per unit of progress help organizations balance computational power with sustainability goals. Finally, standardized evaluation criteria, controlled randomness, submission rules, and audits make comparisons more reproducible and constrain implementation-specific shortcuts.

12.7.2 Training metrics

From a systems perspective, training benchmarks assess how efficiently a model reaches a predefined accuracy threshold. Metrics like throughput and scalability are only meaningful relative to whether the model achieves its target accuracy; without this constraint, optimizing raw speed may be misleading. MLPerf Training codifies this by defining specific accuracy targets per task: a system that trains quickly but misses the target is invalid, and one that converges accurately but too slowly is impractical. Effective benchmarking balances speed, efficiency, and accuracy convergence.

12.7.2.1 Time and throughput

One of the primary metrics for evaluating training efficiency is the time required to reach a predefined accuracy threshold. Training time (T_{train}) measures how long a model takes to converge to an acceptable performance level, reflecting the overall computational efficiency of the system. Let $\text{Accuracy}(t)$ be the model’s accuracy at training time t , and let **target accuracy** be the benchmark-specific threshold (for example, 75.9 percent top-1 accuracy for ResNet-50 on ImageNet in MLPerf). Equation 12.1 formally defines this metric, keeping the benchmark focused on how quickly a system achieves meaningful results:

$$T_{\text{train}} = \inf \{ t \geq 0 : \text{Accuracy}(t) \geq \text{target accuracy} \} \quad (12.1)$$

A run that never reaches the target has no finite time-to-accuracy, preventing raw training speed from rewarding a system that fails the benchmark’s quality criterion.

Throughput,²² often expressed as the number of training samples processed per second, provides an additional measure of system performance. Let N_{samples} be the total number of training samples

22 | Throughput: Originating in manufacturing to measure units produced per unit time, the term entered computing in the 1960s batch-processing era. The manufacturing origin carries a systems lesson: throughput and latency are inherently opposed, because batching increases throughput (more units per hour) at the cost of individual item wait time. In ML serving, this manifests as the batch-size trade-off: larger batches improve GPU utilization but increase per-request latency.

processed and T_{train} the training time from equation 12.1. Equation 12.2 makes the rate explicit by dividing processed samples by training time:

$$\text{Throughput} = \frac{N_{\text{samples}}}{T_{\text{train}}} \quad (12.2)$$

Throughput alone does not guarantee meaningful results, as a model may process a large number of samples quickly without necessarily reaching the desired accuracy. For example, MLPerf Training specifies workload-specific quality targets; a ResNet-50 result on ImageNet must reach a top-1 accuracy target of 75.9 percent to be valid (Mattson et al. 2020; MLCommons 2024c). A hypothetical system that processes many images per second but fails to reach the target is not a valid benchmark result, while a slower system that converges efficiently can be preferable. This highlights why throughput should be evaluated in relation to time-to-accuracy rather than as an independent performance measure.

12.7.2.2 Scalability and parallelism

Scalability measures how effectively training performance improves as resources are added. Ideally, doubling GPU count should halve training time. In practice, communication overhead, memory bandwidth limits, and parallelization inefficiencies constrain scaling well below linear.

Napkin Math 12.3: Scaling efficiency calculation

Problem: A team trains ResNet-50 on ImageNet. Single-GPU training takes 24 hours. With 8 GPUs, training takes 4 hours. Is this good scaling? Where did the efficiency go?

Step 1: Define scaling efficiency. For strong scaling (fixed problem size, more processors), let $T(1)$ be the training time on a single GPU, $T(N_{\text{GPU}})$ the training time on N_{GPU} GPUs, and N_{GPU} the GPU count. Equation 12.3 defines efficiency:

$$\text{Eff}_{\text{scaling}} = \frac{T(1)}{N_{\text{GPU}} \times T(N_{\text{GPU}})} \times 100\% \quad (12.3)$$

Step 2: Calculate efficiency. $\text{Eff}_{\text{scaling}}(8) = \frac{24 \text{ hours}}{8 \times 4 \text{ hours}} \times 100\% = 24/32 = 75 \text{ percent}$

With perfect scaling, 8 GPUs would complete in 3 hours (24 hours/8 GPUs). The actual 4 hours represents 75 percent efficiency.

Step 3: Account for the efficiency loss. Table 12.7 decomposes the “missing” 25 percent into measurable overhead categories—gradient synchronization, memory copy, load imbalance, and batch-size effects—each measurable through a distinct profiling signal.

Table 12.7: Scaling Efficiency Loss Sources: Illustrative decomposition of the missing efficiency budget into measurable overhead categories and the corresponding measurement signal used to attribute each loss. Actual percentages depend on model, interconnect, storage, batch schedule, and framework implementation.

Source	Example Contribution	Measurement
Gradient synchronization	10–15%	AllReduce time per step
Memory copy (CPU↔GPU)	3–5%	Data transfer profiling
Load imbalance	2–5%	Per-GPU step time variance
Batch size effects	2–5%	Larger batches converge differently

Step 4: The systems insight. Scaling efficiency decreases as N_{GPU} grows because communication overhead scales with GPU count while per-GPU compute shrinks. In this worked example, eight GPUs reach 75 percent efficiency; at larger scales, the same arithmetic makes clear why sophisticated communication and input-pipeline optimization become necessary.

MLPerf reports raw performance, while scaling efficiency can be calculated across matched submissions: a system achieving $2\times$ throughput at 50 percent efficiency may be worse than $1.5\times$ throughput at 90 percent efficiency, depending on cost constraints.

When training large-scale models such as GPT-3, OpenAI employed a large cluster of NVIDIA V100 GPUs in a distributed training setup (Brown et al. 2020; Patterson et al. 2021). Google’s TPU v4 systems demonstrate the same distributed-systems lesson at data center scale: adding computational resources provides more raw power, but performance and resiliency depend on network communication, topology, and operational management (Jouppi et al. 2023; Zu et al. 2024). Benchmarks such as MLPerf quantify how well a system scales across multiple accelerators, providing insights into where inefficiencies arise in distributed training.

Parallelism in training is categorized into data parallelism, model parallelism, and pipeline parallelism (see chapter 8), each presenting distinct challenges. Data parallelism, the most commonly used strategy, involves splitting the training dataset across multiple compute nodes. The efficiency of this approach depends on synchronization mechanisms and gradient communication overhead. In contrast, model parallelism partitions the neural network itself, requiring efficient coordination between processors. Benchmarks evaluate how well a system manages these parallelism strategies without degrading accuracy convergence. A key metric for evaluating parallelism is **scaling efficiency**, which quantifies how much of the added computational capacity translates into actual speedup. While strong scaling evaluates whether adding processors reduces training time on a fixed total workload, **weak scaling** scales the problem size proportionally with processor count to keep the per-device workload constant, measuring whether the cluster can train larger models or ingest larger datasets within a fixed time budget without communication bottlenecks dominating.

12.7.2.3 Resource utilization

The efficiency of machine learning training depends not only on speed and scalability but also on how well available hardware resources are used. Compute utilization measures the extent to which processing units, such as GPUs or TPUs, are actively engaged during training. In modern training benchmarks, compute utilization is formally quantified as model FLOPs utilization (MFU), the ratio of theoretical model operations to peak machine capacity over time. MFU contrasts with hardware FLOPs utilization (HFU), which counts all executed operations including activation recomputation; recomputing activations increases HFU while leaving MFU flat, revealing whether extra FLOPs represent true learning progress or work expended to circumvent memory capacity limits. Low utilization may indicate bottlenecks in data movement, memory access, or inefficient workload scheduling.

For instance, when training BERT on a TPU cluster, input-pipeline inefficiencies can limit overall throughput even when the accelerators have high raw compute power. If storage retrieval or preprocessing cannot keep up, the system fails to keep the TPUs fully busy. Profiling resource utilization identifies the bottleneck, and optimizations such as prefetching, caching, and more parallel input processing can improve sustained performance.

Memory bandwidth is another critical factor, as deep learning models require frequent access to large volumes of data during training. If memory bandwidth becomes a limiting factor, increasing compute power alone will not improve training speed. Benchmarks assess how well models use available memory, ensuring that data transfer rates between storage, main memory, and processing units do not become performance bottlenecks.

I/O performance also plays a direct role in training efficiency, particularly when working with large datasets that cannot fit entirely in memory. Benchmarks evaluate the efficiency of data loading pipelines, including preprocessing operations, caching mechanisms, and storage retrieval speeds. Systems that fail to optimize data loading can experience large slowdowns, regardless of computational power.

12.7.2.4 Energy efficiency and cost

Training large-scale machine learning models requires substantial computational resources, leading to considerable energy consumption and financial costs. Energy efficiency metrics quantify the power usage of training workloads, helping identify systems that optimize computational efficiency while minimizing energy waste. The increasing focus on sustainability has led to the inclusion of energy-based benchmarks, such as those in MLPerf Training, which measure power consumption per training run. The same power accounting governs inference, where precision becomes the dominant energy lever; section 12.9 works through why INT8 quantization cuts per-inference energy by attacking both memory traffic and arithmetic cost.

Training GPT-3 was estimated to consume 1,287 MWh of electricity (Patterson et al. 2021). If a system can achieve the same accuracy with fewer training iterations, it directly reduces energy consumption. Energy-aware benchmarks help guide the development of hardware and training strategies that optimize power efficiency while maintaining accuracy targets.

Cost considerations extend beyond electricity usage to include hardware expenses, cloud computing costs, and infrastructure maintenance. Training benchmarks provide insights into the cost-effectiveness of different hardware and software configurations by measuring training time in relation to resource expenditure. Organizations can use these benchmarks to balance performance and budget constraints when selecting training infrastructure.

12.7.2.5 Fault tolerance and robustness

Training workloads often run for extended periods, sometimes spanning days or weeks, making fault tolerance an essential consideration. A resilient system must handle unexpected failures (hardware malfunctions, network disruptions, and memory errors) without compromising accuracy convergence.

In large-scale cloud-based training, node failures are an operational reality. If a GPU node in a distributed cluster fails, training must continue without corrupting the model. Production systems checkpoint for fault tolerance, periodically saving progress so a failure does not restart the run. For large language model (LLM) training, however, checkpointing is itself a systems bottleneck: a single checkpoint must write model weights plus optimizer states to network storage, which at 100-billion-parameter scale can mean hundreds of gigabytes written before training resumes. During that write, accelerators can stall, degrading time-to-accuracy by extending the effective iteration time. Production LLM training systems address this by overlapping checkpoint I/O with the next training step (asynchronous checkpointing) or by using high-bandwidth parallel file systems that reduce idle time. MLPerf Training itself primarily measures time-to-quality under standardized workloads and does not benchmark failure recovery directly, but checkpoint overhead is a material component of any real sustained-throughput number.

12.7.2.6 Reproducibility and standardization

Reproducibility studies have repeatedly shown that modest benchmark gains can disappear when random seeds, hardware, framework versions, or implementation details change (Henderson et al. 2018). This failure mode illustrates a pervasive problem: training benchmarks involve stochastic processes (weight initialization, data shuffling, dropout masks) that interact with hardware-specific behaviors (floating-point rounding, memory layout, compiler optimizations) to produce results that can vary meaningfully across environments.

A deeper layer of non-determinism comes from the parallel hardware itself. Operations such as parallel atomic additions, used during gradient accumulation for sparse embeddings in models like Graph Neural Networks, execute in non-deterministic order across threads when concurrent updates target the same memory location. The resulting floating-point summation order changes across runs, producing bit-for-bit different gradients even with identical inputs and seeds. Enforcing bit-exact reproducibility in these cases requires disabling the parallel accumulation paths, which reduces training throughput—a direct trade-off between reproducibility and performance that benchmark protocols must explicitly address. Without explicit controls for all these sources of variability, benchmark numbers reflect a specific confluence of conditions rather than a system’s genuine capability.

MLPerf Training addresses this through standardized data preprocessing, target-quality rules, and repeated accepted runs that characterize stochastic variation (Mattson et al. 2020). The point is not merely to produce a fast run, but to show that the reported performance reflects system capability rather than a favorable combination of stochastic factors.

For a training benchmark, reproducibility is therefore the full run envelope, not just the random seed. A credible report must preserve the model commit, dataset checksum, preprocessing pipeline, seed plan, framework and compiler versions, precision policy, batch schedule, hardware topology, thermal and power limits, and checkpoint behavior. It must also report the distribution of accepted runs rather than a single best run. Only then can the benchmark separate a real system improvement from a favorable interaction among software version, hardware state, and stochastic training path.

12.7.3 Training performance evaluation

A comprehensive training benchmark considers multiple dimensions of system behavior because each dimension identifies a different way hardware investment can fail to become convergence. Table 12.8 summarizes the core categories and associated metrics commonly used to benchmark system-level training performance, providing a framework for understanding how training systems behave under different workloads and configurations.

Table 12.8: Training Benchmark Dimensions: Key categories and metrics for evaluating machine learning training systems beyond simple speed, covering resource efficiency, reproducibility, and overall performance trade-offs across different training approaches and infrastructure configurations.

Category	Key Metrics	Example Benchmark Use
Training Time and Throughput	Time-to-accuracy (seconds, minutes, hours); Throughput (samples/sec)	Comparing training speed across different GPU architectures
Scalability and Parallelism	Scaling efficiency (percent of ideal speedup); Communication overhead (latency, bandwidth)	Analyzing distributed training performance for large models
Resource Utilization	Compute utilization (percent GPU/TPU usage); Memory bandwidth (GB/s); I/O efficiency (data loading speed)	Optimizing data pipelines to improve GPU utilization
Energy Efficiency and Cost	Energy consumption per run (MWh, kWh); Training throughput per watt (FLOP/s/W)	Evaluating energy-efficient training strategies
Fault Tolerance and Robustness	Checkpoint overhead (time per save); Recovery success rate (percent)	Assessing failure recovery in cloud-based training systems
Reproducibility and Standardization	Variance across runs (percent difference in accuracy, training time); Framework consistency (TensorFlow vs. PyTorch vs. JAX)	Ensuring consistency in benchmark results across hardware

These dimensions interact in ways that tables cannot capture. Higher throughput from reduced precision (for example, TF32) is meaningless if it increases the iterations required to reach target accuracy, making time-to-accuracy the essential corrective metric. Scaling efficiency can look nearly linear at small node counts but taper as gradient synchronization costs dominate. Resource utilization metrics reveal why: a BERT pretraining task with moderate GPU utilization may be bottlenecked by its data pipeline, not its accelerators. Checkpointing for fault tolerance introduces its own overhead, requiring balance between resilience and performance.

Across all dimensions, measurement accuracy depends on controlling for hardware variability. GPU boost clock²³ behavior and thermal throttling²⁴ can shift results enough to swamp small claimed gains, making repeated runs and statistical rigor (as established earlier) essential for distinguishing genuine performance differences from noise.

Despite the availability of well-defined benchmarking methodologies, misleading conclusions recur when teams treat one training metric as a substitute for the whole optimization loop. The following pitfalls show where the benchmark must keep speed, convergence, scaling, and reproducibility tied together.

Training benchmark failures usually start when throughput is treated as the objective rather than as one part of the learning process. A system can increase examples per second by using lower numerical precision, reducing synchronization, or even bypassing certain computations, but those changes only help if convergence is preserved. A TF32 run may outpace FP32 per step and still lose overall if numerical instability increases the number of iterations required to reach the target accuracy. The benchmark therefore has to report throughput in relation to time-to-accuracy, ensuring that speed optimizations do not come at the expense of convergence efficiency.

Scaling creates a second trap because a small-node result can look linear until communication and synchronization dominate. The earlier eight-GPU calculation shows why small-node results cannot be extrapolated linearly once synchronization becomes the binding term.

As the scaling efficiency calculation in section 12.7.2.2 demonstrated (where 8 GPUs achieved only 75 percent efficiency), extrapolating single-node results to clusters is a common error. Google’s experience with 4,096-node TPU v4 clusters shows this effect at extreme scale, where synchronization challenges become the dominant performance factor. Proper benchmarking should measure scaling efficiency explicitly rather than assuming linear improvement.

23 | GPU boost clock: Dynamic frequency scaling raises clocks above base when thermal and power headroom permit. The benchmarking trap: short benchmark runs can capture boost-clock performance, but sustained ML training may settle to lower steady-state frequencies as junction temperature rises. Reporting burst-phase results overstates the throughput a production workload can sustain.

24 | Thermal throttling: Frequency reduction triggered when junction temperature exceeds safe limits. For edge devices without active cooling, throttling can begin during sustained inference, meaning peak throughput numbers from short benchmarks may misrepresent steady-state performance.

The same discipline applies to failures and interference. Many benchmarks assume idealized conditions where hardware failures, network instability, and workload interference do not occur, even though those events are routine at scale. Effective benchmarking accounts for checkpointing overhead, failure recovery efficiency, and resource contention rather than reporting only best-case performance.

Reproducibility poses another threat. Results must reproduce across stacks: a TensorFlow run with Accelerated Linear Algebra optimizations may exhibit different convergence behavior than the same model trained in PyTorch with Automatic Mixed Precision (AMP), because floating-point arithmetic, memory layouts, and optimization strategies can all shift training time and accuracy.

Avoiding these pitfalls requires evaluating throughput in relation to accuracy convergence, assessing scaling efficiency holistically, and accounting for real-world failures rather than assuming idealized conditions. A model trained efficiently, however, still requires validation of its deployment performance, which shifts the evaluation framework entirely.

12.8 Inference Benchmarks

Training benchmarks measure how quickly a system learns; inference benchmarks measure how reliably it serves. This shift changes nearly every aspect of evaluation. Training tolerates variable iteration times as long as convergence proceeds; inference requires consistent latency because users experience every slow response. Training optimizes for aggregate throughput across hours; inference must handle unpredictable request patterns and scenario-specific deadlines. Large benchmark training commonly uses dedicated high-performance hardware, whereas inference spans environments from data center GPUs to mobile phones to microcontrollers.

This is where the optimization chapters converge: the accelerated hardware from chapter 11 runs compressed models from chapter 10 to deliver real-time predictions. Inference benchmarks reveal whether those theoretical speedups become actual latency reductions under realistic deployment conditions.



Definition 12.4: ML inference benchmarks

ML inference benchmarks are machine learning system benchmarks that quantify a system's ability to meet latency constraints (L_{lat}) at specified throughput levels, measuring scenario-defined latency statistics, throughput (queries per second), and power efficiency across representative serving scenarios.

1. **Significance:** Inference benchmarks expose the workload- and system-specific gap between unconstrained throughput and throughput while meeting a service-level objective (SLO), such as a p99 latency target. Queuing delays can push tail latency above the target at high load, a gap that is invisible without a benchmark that enforces latency targets at each throughput level.
2. **Distinction:** Unlike training benchmarks, which measure time-to-accuracy over a fixed dataset, inference benchmarks measure per-query response time under realistic load patterns, capturing queuing effects, batching trade-offs, and cold-start overhead that determine real-world serving economics.
3. **Common pitfall:** A frequent misconception is that average latency is a sufficient benchmark. A system with low average latency but a long p99 tail can violate a percentile-based production SLO for the slowest 1 percent of requests; at high request rates, that small percentage becomes a large number of affected users. The benchmark must report the latency statistic named by the service objective rather than assuming the mean or p99 is universally decisive.

12.8.1 Inference benchmark motivation

Large-scale benchmark training commonly runs on dedicated data center hardware, whereas inference spans dramatically different deployment scenarios—from real-time applications like autonomous driving and conversational AI to mobile devices, IoT systems, and embedded processors.

25 | Edge TPU (tensor processing unit): Google's fixed-function edge AI accelerator. It illustrates a benchmarking constraint specific to fixed-function accelerators: its headline throughput applies only to quantized TensorFlow Lite models with supported operator types, so models requiring unsupported operators fall back to the host CPU or need graph rewrites before the accelerator result is meaningful.

This diversity extends to hardware: while GPUs and TPUs dominate large-scale training, inference workloads often use specialized accelerators like NPUs, field-programmable gate arrays, and dedicated inference chips such as Google's Edge TPU.²⁵ Inference benchmarks evaluate how well hardware selection, model optimization, and data pipeline design work together across these deployment environments.

Scaling inference workloads across cloud servers, edge platforms, mobile devices, and TinyML systems introduces additional complexity. Figure 12.6 reveals the wide power consumption differentials among these systems—spanning over ten orders of magnitude from microwatts in tiny embedded devices to hundreds of kilowatts in data center training clusters. The ranges are representative rather than exhaustive. This spread explains why no single benchmark can serve all deployment contexts: a metric meaningful for data center optimization (kilowatts per rack) becomes irrelevant for battery-powered edge devices (milliwatts per inference). Inference benchmarks must evaluate the trade-offs between latency, cost, and energy efficiency within each scale to assist organizations in making informed deployment decisions.

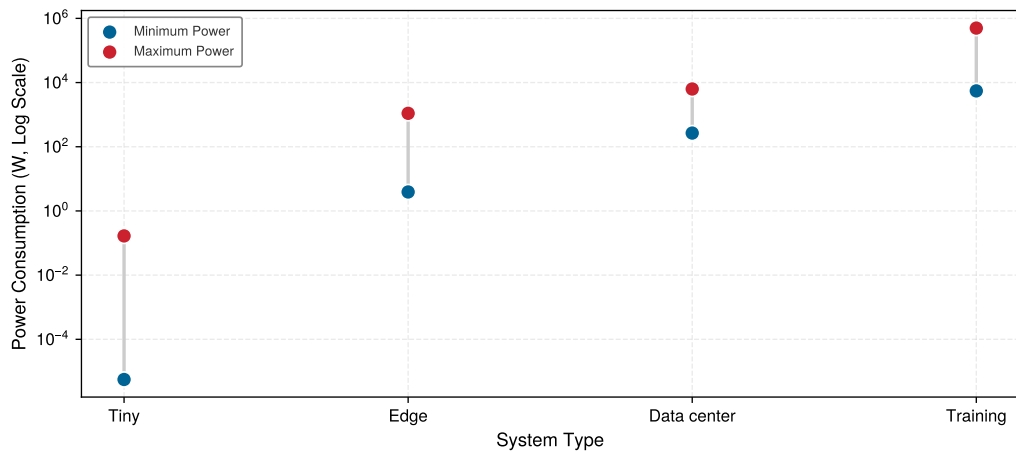


Figure 12.6: Power Consumption Differentials: Power usage spans over ten orders of magnitude across ML system types, from microwatts in TinyML devices through watts at the edge to kilowatts in data center inference and hundreds of kilowatts for training clusters. Ranges are representative and vary by hardware and workload.

These deployment differences create the practical motivation for inference benchmarks: they evaluate the bottlenecks that emerge when models transition from development to production serving. The motivating factors parallel those for training (hardware optimization, scalability, cost, fair comparison) but differ in specifics. Software optimization frameworks apply inference-specific techniques such as operator fusion (see chapter 10 and chapter 11), precision calibration, and kernel tuning, whose impact on latency, throughput, and power efficiency must be measured under realistic conditions to confirm they deliver real improvements without degrading accuracy. Auto-tuning compilers add a hidden variable: the compiler itself can require hours of optimization per model-hardware pair, meaning benchmark results reflect the tuning budget as much as the hardware capability, and comparing results across submissions requires normalizing for compiler optimization time.

Scalability concerns also shift character. Training scales by adding GPUs to reduce time-to-accuracy on a fixed workload, whereas inference must scale dynamically in response to fluctuating user demand, handling traffic spikes without violating latency guarantees. Cold-start performance, the time required for a model to load and begin processing queries, becomes a distinct inference concern with no training analog. Applications that load models on demand, such as serverless AI deployments, are particularly sensitive to this overhead.

The cost and energy profile of inference differs sharply from training. Training concentrates cost in discrete runs that may recur as data, objectives, or models change; inference cost accumulates continuously with production traffic. Running an inefficient model at scale can multiply cloud compute expenses, and on battery-powered devices, excessive computation directly affects usability.

Benchmarks that measure cost per inference request and efficiency per watt help organizations optimize for both performance and sustainability across deployment platforms.

MLPerf Inference extends the standardized comparison principles established for training benchmarks to deployment scenarios, defining evaluation criteria for tasks such as image classification, object detection, and speech recognition across different hardware platforms. This ensures that inference performance comparisons remain meaningful and reproducible while accounting for deployment-specific constraints like latency requirements and energy efficiency (Reddi et al. 2019).

12.8.2 Inference metrics

For example, a voice assistant must respond quickly enough that users do not perceive lag, while a recommendation engine must score enough candidates to keep pace with user scrolling. These constraints (latency and throughput) define the performance envelope within which all serving optimizations must operate. Inference metrics formalize these real-world demands into measurable quantities, and they differ from training metrics in kind, not just degree, because the optimization target shifts from convergence rate during training to latency predictability during serving. Training optimizes for throughput and time-to-accuracy; inference optimizes for latency consistency, resource efficiency, and service-level predictability, spanning cloud data centers handling millions of requests to edge devices operating under strict power constraints.

12.8.2.1 Latency and tail latency

Latency (introduced in chapter 2) measures the time for an inference system to process an input and produce a prediction. Average latency is useful, but it does not capture high-percentile delays that degrade reliability in high-demand scenarios.

To account for this, benchmarks often measure tail latency,²⁶ which summarizes the high-latency tail rather than the worst case. These values are commonly reported as the 95th percentile (p95) or 99th percentile (p99) latency, meaning that 95 percent or 99 percent of inferences are completed within a given time. Applications with hard deadlines require explicit deadline-miss or worst-case analysis beyond these percentiles.

These measurements form the basis for Service Level Objectives (SLOs) and SLAs, which formalize performance expectations.



Definition 12.5: SLOs and SLAs

SLOs and SLAs are performance commitment specifications for production ML serving systems: a service-level objective (SLO) is a target value or range for a service-level indicator, while an SLA is a contract that specifies service objectives and the consequences of failing to meet them.

1. **Significance:** A latency SLO constrains the L_{lat} term in the iron law by setting a target for a specified latency indicator. A representative production setup might set the internal SLO tighter than the external SLA, leaving operational headroom for transient spikes, maintenance windows, and cascading failures.
2. **Distinction:** An SLO has no contractual consequence by itself, while missing an SLA objective triggers the consequences specified in the agreement, which may include credits or penalties. Teams commonly set an internal SLO tighter than the external agreement to leave operational headroom, but this is a practice rather than a definitional requirement.
3. **Common pitfall:** A frequent misconception is that meeting average latency satisfies a tail-latency SLO. SLOs can use averages, percentiles, rates, or other indicators; when the objective is defined at p99 or p99.9, an excellent mean does not establish compliance.

The distinction matters in practice: engineering teams optimize toward SLOs while the business commits to SLAs. Choosing the wrong metric to optimize wastes engineering effort or violates customer guarantees.

A usable latency objective also fixes the request population and measurement window. Otherwise, two teams can report the same percentile threshold over different traffic mixes and reach incompatible conclusions about compliance.

²⁶ | **Tail latency:** A high-percentile response time, such as p95 or p99, determines production SLO or SLA compliance only when that objective or agreement specifies the percentile. Dean and Barroso (2013) showed that in fan-out architectures (common in recommendation systems), even 1 percent slow responses compound: a request touching 100 backend shards has about a 63 percent chance that at least one shard hits its 1 percent tail. Benchmarks reporting only mean latency hide this failure mode.

Checkpoint 12.2: Metric selection

The metric shapes the optimization.

Apply three rules before finalizing metric selection:

- ☐ Throughput vs. latency: Decide whether the workload optimizes for cost or user experience, then name the metric the benchmark must hold fixed.
- ☐ Tail latency: State what p99 exposes that mean latency hides.
- ☐ End-to-end: List the preprocessing, model execution, and postprocessing stages included in the reported latency.

Tail latency's connection to user experience at scale becomes critical in production systems serving millions of users. Even small p99 latency degradations create compounding effects across large request volumes: if 1 percent of requests experience $10\times$ latency (for example, 1000 ms instead of 100 ms), this affects 10,000 requests per million, potentially leading to timeout errors, poor user experience, and customer churn. Search engines and recommendation systems demonstrate this sensitivity: Google's search-latency experiments found measurable reductions in daily searches per user after 100–400 ms server-side delays (Brutlag 2009), which is why interactive services often treat sub-100 ms response times as a practical design target.

Service level objectives (SLOs) in production systems therefore focus on tail latency rather than mean latency to ensure consistent user experience. Interactive services often define percentile-based latency objectives because occasional slow responses have disproportionate impact on user satisfaction. Large-scale systems may track even deeper tails, such as p99.9, when traffic spikes and infrastructure variation affect reliability.

The challenge of meeting these tail latency targets is that the source of the tail is often architectural, not algorithmic. A garbage-collected runtime, a shared kernel driver, or a priority-inversion bug in the serving stack can inject latency spikes that no model optimization will remove.

War Story 12.1: Discord Read States rewrite (2019)

Context: Discord, a real-time chat platform supporting millions of concurrent users, originally implemented its Read States service in Go, tracking which channels and messages each user read (Howarth 2020).

Mechanism: Go's forced garbage collection passes scanned an LRU cache holding tens of millions of entries, causing periodic "stop-the-world" GC pauses every two minutes regardless of memory tuning.

Impact: Tail latency (P_{99}) spiked dramatically every two minutes, degrading user experience across millions of active connections.

Fix: In 2019, Discord rewrote the Read States service in Rust, eliminating garbage collection pauses and dropping average response times to microseconds.

Systems lesson: Mean latency alone does not describe user experience when the service objective is defined at the tail. Language-runtime choices can set a floor on that tail: feature stores that serve real-time embeddings for recommendation models may use managed runtimes, and garbage-collection pauses can delay every downstream inference request waiting for retrieval. Model optimization cannot close a gap whose bottleneck lies in the retrieval path. The Discord incident documents the underlying mechanism outside ML and shows why the complete serving stack belongs inside a production latency benchmark.

12.8.2.2 End-to-end vs. component latency

A critical distinction in inference benchmarking is between component latency (time spent in model computation) and end-to-end latency (total time from request arrival to response delivery). Many benchmarks report only model inference time, obscuring the remaining overhead that determines

actual user experience. The overhead is not marginal: serialization, network hops, and queue wait time can dominate total request time, making model-only optimizations yield diminishing returns.



Example 12.4: The JSON serialization trap

Scenario: Researchers evaluating the Clipper low-latency model serving system decomposed end-to-end request latency across network, CPU serialization, and model execution (Crankshaw et al. 2017).

Diagnosis: For lightweight models, CPU-side JSON deserialization and IPC data copying consume more wall-clock time than accelerator neural network execution.

Systems lesson: Reporting isolated model inference latency misses end-to-end serving bottlenecks. Production benchmarking must account for CPU data parsing, IPC serialization, and network transport overhead.

Table 12.9 gives an illustrative latency breakdown for an inference request. The model inference stage that vendors report as their “benchmark” number spans 5 to 100 ms, yet the queue wait time it sits behind ranges from 0 to over 1,000 ms: under load, the single component a benchmark measures is dwarfed by one it never sees, so the reported number can be a small slice of what the user actually experiences.

Table 12.9: Inference Latency Breakdown: Different pipeline components contribute to end-to-end latency, and model inference (the number vendors typically report) can be only one part of total request time. Queue wait time can dominate under load, making end-to-end measurement essential for realistic performance assessment. Ranges are illustrative rather than universal.

Component	Example Range	Notes
Network round-trip	10–100 ms	Varies by region
Request parsing	0.1–1 ms	JSON/protobuf
Input preprocessing	1–50 ms	Tokenization, image resize
Queue wait time	0–1000+ ms	Load-dependent
Model inference	5–100 ms	The “benchmark”
Output postprocessing	0.5–10 ms	Decoding, format
Response serialization	0.1–1 ms	JSON/protobuf

These component-level contributions explain why optimizing any single stage yields diminishing returns on end-to-end performance, an *optimization ceiling* formalized by Amdahl’s Law.



Napkin Math 12.4: Amdahl’s Law: Optimization ceiling

Problem: A vision pipeline spends 8 ms on preprocessing (JPEG decode, resize, normalize) and 10 ms on inference. If inference alone is optimized by 5×, how much does end-to-end latency actually improve?

Math: Optimizing inference from 10 ms to 2 ms reduces total latency from 18 ms to only 10 ms, a 1.8× improvement rather than 5×. Amdahl’s Law formalizes this ceiling: if preprocessing consumes fraction f of total latency, then even infinitely fast inference yields at most $1/f$ speedup. With preprocessing at 44.4 percent of total latency, identifying the dominant fraction $f = 0.444$ yields a maximum achievable speedup $1/f$ of 2.25×, regardless of model optimization.

Systems insight: Aggressive model optimization yields disappointing end-to-end results whenever the nonmodel fraction dominates. Any component speedup quoted in isolation shrinks once the untouched stages are measured alongside it, and the shortfall grows as those stages take a larger share of the pipeline. Comprehensive benchmarks must either include preprocessing in measurements or state explicitly that reported speedups apply only to the inference component.

Amdahl's ceiling highlights why rigorous benchmarking methodology matters. Comprehensive latency reporting requires specifying which components are included, measuring under realistic load conditions, and distinguishing component from end-to-end metrics. Before interpreting any benchmark result, verify that the measurement approach itself is sound.

Throughput and batch efficiency measure whether a serving system can use available hardware without violating latency constraints. Throughput counts how many inference requests a system processes per second, typically expressed as queries per second (QPS) or frames per second (FPS). Single-instance systems process each input independently on arrival; batch systems process multiple inputs in parallel, exploiting hardware parallelism for higher efficiency.



Checkpoint 12.3: Benchmarking methodology

Bad benchmarks optimize the wrong things.

Three practices distinguish rigorous benchmarks from misleading ones:

- ☐ Representative data: Compare the benchmark input distribution against the production distribution.
- ☐ Warm-up: State how many initial runs were discarded before recording measurements.
- ☐ Isolation: Document the machine, competing workloads, and runtime configuration used during measurement.

For example, cloud-based services handling millions of queries per second benefit from batch inference, where large groups of inputs are processed together to maximize computational efficiency. In contrast, applications like robotics, interactive AI, and augmented reality require low-latency single-instance inference, where the system must respond immediately to each new input. Benchmarks must consider both single-instance and **batch throughput** to provide a comprehensive understanding of inference performance across different deployment scenarios.

Speed alone is insufficient because inference optimizations can change model behavior. Reducing numerical precision can accelerate computation while cutting memory and energy, as the illustrative MobileNetV2 energy estimate shows (table 12.17), but lower-precision calculations can introduce accuracy degradation. Inference benchmarks therefore evaluate how well models perform under different numerical settings, such as FP32, FP16, and INT8.²⁷ Many modern AI accelerators support mixed-precision inference, allowing systems to use numerical representations selected for workload requirements. Model compression techniques²⁸ further improve efficiency, but their impact on model accuracy varies depending on the task and dataset. Benchmarks help determine whether these optimizations are viable for deployment, ensuring that improvements in efficiency do not come at the cost of unacceptable accuracy loss.

Memory footprint and model load time define whether the model can start, stay resident, and respond within the deployment envelope. Unlike training, where models can span multiple accelerators, inference often runs within strict memory budgets. Total model size determines storage requirements, RAM usage reflects working memory during execution, and memory bandwidth can bottleneck data transfer between processing units. Cold-start performance becomes critical when models are loaded on demand rather than kept resident in memory. In serverless AI environments,²⁹ where resources scale dynamically with incoming requests, the time from idle to active execution determines whether users experience acceptable response times.

Model load time refers to the duration required to load a trained model into memory before it can process inputs. In some cases, particularly on resource-limited devices, models must be reloaded frequently to free up memory for other applications. The time taken for the first inference request is also an important consideration, as it reflects the total delay users experience when interacting with an AI-powered service. Benchmarks help quantify these delays, ensuring that inference systems can meet real-world responsiveness requirements.

Deployment-scale metrics extend the same logic from one request to a workload. Cloud services must handle millions of concurrent users efficiently, allocating resources dynamically as demand fluctuates without compromising latency; mobile devices must manage multiple simultaneous AI models without overloading the system. Scalability measures how well inference performance

²⁷ | **INT8 (8-bit integer):**

INT8 sits at the aggressive end of the precision hierarchy (FP32 baseline, FP16 halves raw weight storage, INT8 quarters it), and each step demands increasing care to preserve accuracy. Post-training INT8 quantization normally uses a representative calibration dataset; accuracy depends on the model, operator coverage, quantization method, and how well calibration data represent deployment inputs. INT8 benchmarks must specify whether they use post-training quantization or quantization-aware training and document any calibration procedure.

²⁸ | **Model compression benchmarking:**

Compression impact must be measured across four dimensions simultaneously: accuracy degradation, inference speedup, memory reduction, and energy savings. A technique achieving 10× size reduction with 1 percent accuracy loss may still be unsuitable if latency does not improve proportionally; unstructured pruning, for example, reduces parameter count but rarely improves latency on dense hardware because sparse operations lack efficient hardware support on most GPUs.

²⁹ | **Serverless AI:**

Deployment paradigm where models scale from zero instances on demand. Cold-start time varies with the model, runtime, storage path, hardware allocation, and provider. Benchmarks for intermittent workloads must report whether initialization and model loading are included, because warm-instance latency alone can understate user-perceived latency.

improves when additional computational resources are allocated. In some cases, adding more GPUs or TPUs increases throughput proportionally, but in other scenarios, bottlenecks such as memory bandwidth limitations or network latency may limit scaling efficiency. Benchmarks also assess how well a system balances multiple concurrent models in real-world deployment, where different AI-powered features may need to run at the same time without interference.

Energy consumption closes the loop because inference workloads run continuously in production. Mobile and edge devices face the most acute constraints, where battery life and thermal limits restrict available computational resources. Even in large-scale cloud environments, power efficiency directly impacts operational costs and sustainability goals. The energy required for a single inference is often measured in joules per inference, reflecting how efficiently a system processes inputs while minimizing power draw. In cloud-based inference, efficiency is commonly expressed as queries per second per watt (QPS/W) to quantify how well a system balances performance and energy consumption. For mobile AI applications, optimizing inference power consumption extends battery life and allows models to run efficiently on resource-constrained devices. Reducing energy use also plays a key role in making large-scale AI systems more environmentally sustainable, ensuring that computational advancements align with energy-conscious deployment strategies.

12.8.3 Inference performance evaluation

In training, systems optimize aggregate throughput across large offline batches, amortizing parameter movement over hundreds or thousands of accelerator nodes. Inference operates under the opposite regime: workloads are demand-driven, arriving asynchronously from external clients, and bounded by strict latency service-level agreements (SLAs), device memory capacity, and thermal dissipation budgets. Evaluating an inference system therefore requires measuring how effectively it balances response time, processing volume, and physical resource consumption under deployment-representative arrival patterns.

Table 12.10 functions as a deployment filter: each metric quantifies a physical or operational constraint that dominates a specific serving environment. Tail latency bounds slow requests in user-facing interactive services, whereas safety-critical systems require deterministic worst-case bounds or bounded deadline-miss probabilities. Conversely, energy efficiency (queries per second per watt) governs battery-bound edge accelerators, where sustained computation is constrained by battery capacity and passive thermal dissipation rather than peak arithmetic throughput.

Table 12.10: Inference Performance Metrics: Latency, throughput, and resource usage metrics provide a quantitative basis for optimizing deployed machine learning systems and selecting appropriate hardware configurations, balancing speed, cost, and accuracy in production applications.

Category	Key Metrics	Example Benchmark Use
Latency and Tail Latency	Mean latency (ms/request); Tail latency (p95, p99, p99.9)	Evaluating interactive and soft real-time performance
Throughput and Efficiency	Queries per second (QPS); Frames per second (FPS); Batch throughput	Comparing large-scale cloud inference systems
Numerical Precision Impact	Accuracy degradation (FP32 vs. INT8); Speedup from reduced precision	Balancing accuracy vs. efficiency in optimized inference
Memory Footprint	Model size (MB/GB); RAM usage (MB); Memory bandwidth utilization	Assessing feasibility for edge and mobile deployments
Cold-Start and Load Time	Model load time (s); First inference latency (s)	Evaluating responsiveness in serverless AI
Scalability	Efficiency under load; Multi-model serving performance	Measuring robustness for dynamic, high-demand systems
Power and Energy Efficiency	Power consumption (W); Performance per W (QPS/W)	Optimizing energy use for mobile and sustainable AI

These metrics interact through fundamental hardware trade-offs. Maximizing throughput by aggregating requests into large batches increases arithmetic intensity, amortizing weight-loading memory traffic across multiple inputs. However, requests must wait in queues for batches to assemble, inflating per-request queuing delay and violating real-time latency budgets. Similarly, reducing numerical precision from FP32 to INT8 cuts memory bandwidth demand by up to 75

percent and doubles or quadruples peak arithmetic execution rates on tensor cores, but it risks accuracy degradation if quantization noise perturbs sensitive layers. The physical deployment environment determines which trade-offs are viable: hyperscale cloud servers optimize cost per query by operating at high batch throughput and high memory utilization, whereas edge devices are bounded by fixed SRAM and DRAM capacities, battery life, and thermal dissipation envelopes.

Because physical constraints differ across environments, no single metric ordering governs all systems. Benchmarking requires establishing a priority hierarchy derived directly from the deployment contract. Table 12.11 illustrates how metric priorities shift across five operational contexts, mapping hardware and operational constraints to optimization targets.

Table 12.11: Illustrative Performance Metric Priorities by Deployment Context: A sample priority ordering shows how operational constraints can change benchmark selection and result interpretation; actual thresholds must come from the deployment specification.

Deployment Context	Primary Priority	Secondary Priority	Tertiary Priority	Key Design Constraint
Real-Time Applications	Latency	Reliability	Memory Footprint	User experience demands immediate response
Cloud-Scale Services	Throughput (QPS)	Cost Efficiency	Average Latency	Business viability requires massive scale
Edge/Mobile Devices	Power Consumption	Memory Footprint	Latency	Battery life and resource limits dominate
Training Workloads	Training Time	GPU Utilization	Memory Efficiency	Research velocity enables faster experimentation
Scientific/Medical	Accuracy	Reliability	Explainability	Correctness cannot be compromised for performance

A metric that dictates architectural success in one deployment context may be irrelevant in another. Latency ranks first for autonomous robotics, where sensor-processing pipelines must complete within strict execution deadlines to maintain vehicle stability, whereas cloud batch services accept higher queuing latency in exchange for lower operational cost per query. Similarly, an edge-deployed model that boosts throughput while drawing additional power degrades overall system utility when battery capacity or thermal throttling is the binding constraint. In contrast, a medical diagnostic pipeline may rationally trade execution speed for higher verified accuracy. A 2× throughput gain represents substantial economic value in a hyperscale cloud cluster, but it provides minimal benefit on a battery-powered device where a 20 percent power reduction extends operational lifetime.

When benchmarks fail, it is usually because they measure steady-state averages rather than the transient or worst-case behaviors that govern production serving. Reporting mean latency obscures queueing delay spikes, memory garbage collection, thread scheduling jitter, and tail contention; production service-level agreements depend on tail percentiles (p_{95} , p_{99} , and $p_{99.9}$). A serving system with an acceptable mean response time can still produce severe user-visible stalls if its tail latency violates the SLA.

Even a benchmark that captures tail latency under steady load will misrepresent on-demand or serverless systems if it ignores initialization overhead. In serverless serving, worker instances scale to zero during idle periods. **Cold-start latency**³⁰ measures the full elapsed time required to provision hardware resources, load model weights from storage into host and accelerator memory, initialize runtime contexts, and execute the first inference request. Benchmarking only warm instances conceals the latency penalty experienced by end users after idle intervals.

Beyond initialization delays, inference benchmarks become misleading when operational metrics are reported without functional validation.

Numerical precision optimization introduces a significant comparability trap across accelerator platforms. Accelerators provide substantially higher INT8 operation throughput³¹ than baseline FP32 floating-point rates, but these peak figures are meaningful only when the benchmark verifies accuracy retention under quantization noise, confirms hardware support across all model operators, and enforces equivalent operation definitions across architectures.

Hardware scaling introduces divergent bottlenecks between training and inference. While distributed training scaling is bounded primarily by gradient synchronization across inter-node in-

³⁰ | **Cold-start latency:** The initialization time from idle state includes storage reads, host-to-accelerator transfer, and framework setup. For a 7B-parameter model in FP16 (~14 GB) already resident in host memory, PCIe 4.0 transfer at 25 GB/s effective bandwidth alone takes ~560 ms; storage and initialization add to it. This lower bound makes cold-start mitigation (model caching, speculative loading) a systems design requirement.

³¹ | **TOPS (tera operations per second):** A measure of raw computational throughput (trillions of operations/second). The H100 delivers 1979 TOPS INT8 vs. the Apple M2 Neural Engine at 15.8 TOPS and Edge TPU at 4 TOPS, but these numbers conflate different operation types—multiply-accumulate vs. accumulate vs. activation. TOPS comparisons across vendors are meaningful only when the operation definition, precision, and sparsity assumptions are identical, conditions rarely met in vendor specifications.

terconnects, inference scaling encounters memory bandwidth saturation during token generation, tensor-parallel collective communication latency, and load-balancer request-routing overhead across distributed serving replicas. On edge devices and dense accelerator chassis, sustained query streams induce thermal throttling, depressing operational clock frequencies below nominal peak rates (chapter 11). A benchmark optimized exclusively for peak cloud throughput fails to capture these thermal and memory constraints, reinforcing that evaluation suites must reflect actual deployment conditions rather than idealized hardware specifications.

Capturing these operational realities requires rigorous statistical protocols rather than isolated execution loops. Valid inference benchmarking demands sustained run durations, realistic query arrival distributions, and strict percentile latency accounting. Standardized benchmarking consortia formalize these requirements into distinct deployment scenarios: MLPerf Inference, for example, reports 90th-percentile latency for SingleStream edge tasks, enforces a 99th-percentile latency threshold while maximizing query throughput in Server scenarios, and measures unconstrained batch capacity in Offline regimes (Reddi et al. 2019). These scenario-specific protocols provide the structured framework needed to evaluate machine learning systems against their binding physical constraints.

12.8.4 MLPerf inference benchmarks

Avoiding these pitfalls requires treating inference benchmarking as a process of balancing multiple priorities (latency, throughput, memory, energy, and accuracy) rather than optimizing for any single metric in isolation; MLPerf Inference operationalizes that balance through deployment-specific scenarios. MLPerf Inference matters because deployment context changes what a result means. The benchmark, developed by MLCommons,³² provides a standardized framework for evaluating machine learning inference performance across a range of deployment environments. MLPerf began with training benchmarks in 2018; MLPerf Inference was added later to standardize deployment-time evaluation across scenarios. As machine learning systems expanded into diverse applications, it became clear that a one-size-fits-all inference benchmark was insufficient. The resulting family of MLPerf inference benchmarks maps each benchmark to a deployment setting, so a score can be interpreted against the latency, throughput, memory, and power constraints the system will face.

12.8.4.1 MLPerf Inference

MLPerf Inference (Reddi et al. 2019) serves as the baseline inference benchmark, defining standardized scenarios for deployment-time evaluation across data-center and edge settings. Submissions are split into two tracks: the **closed division** enforces strict model structure and precision equivalence to ensure direct hardware-to-hardware comparison (apples-to-apples), whereas the **open division** allows submitters to alter model architecture, quantization algorithms, or retrain models to demonstrate creative algorithmic co-design. It assesses performance across deep learning workloads such as image classification, object detection, natural language processing, and recommendation systems. This version of MLPerf is a widely used reference point for comparing AI accelerators, GPUs, TPUs, and CPUs when the submission rules and workload scenario match the intended deployment environment.

Major technology companies regularly reference MLPerf results for hardware procurement decisions. When evaluating hardware for recommendation systems infrastructure, MLPerf benchmark scores on DLRM³³ workloads can inform choices between different accelerator generations. Across generations, benchmark results often show substantial throughput improvements, although the magnitude depends on workload, software stack, and system configuration. This illustrates how standardized benchmarks can translate into consequential infrastructure decisions.

These standardized evaluations provide invaluable comparisons, but the cost of comprehensive benchmarking limits who can participate and how thoroughly systems are evaluated.

Systems Perspective 12.8: The cost of comprehensive benchmarking

Submitting to MLPerf can require dedicated engineering effort, hardware access, tuning, validation, and careful documentation across the relevant configurations. The cost depends heavily on workload and scope, but it is high enough that submissions are dominated by major technology companies and hardware vendors, while smaller organizations rely on published

³² | **MLCommons**: Launched in 2020 as a nonprofit consortium evolving from the 2018 MLPerf effort, MLCommons includes members across industry and academia (MLCommons 2026a). Requiring published system specifications improves result comparability, but it does not prevent submitters from selecting which systems or workloads to enter. Published results reveal large performance differences between vendors on identical workloads, making MLCommons the closest the field has to SPEC-style apples-to-apples hardware comparison.

³³ | **DLRM (deep learning recommendation model)**: Facebook's 2019 recommendation architecture combines embedding tables for categorical features with multilayer perceptrons for continuous features (Naumov et al. 2019). DLRM stresses benchmarks differently than vision or language models: its embedding tables can be large enough that memory capacity and bandwidth dominate compute throughput. That makes DLRM a useful memory-bound recommendation workload in MLPerf-style inference evaluation, revealing hardware limitations invisible to compute-bound benchmarks (Reddi et al. 2019).

results rather than conducting their own comprehensive evaluations. That barrier motivates the need for more lightweight, internal benchmarking practices that organizations can use to make informed decisions without the expense of full-scale standardized benchmarking.

The rest of the MLPerf inference family narrows that baseline by deployment context. MLPerf Mobile (MLCommons 2024a) evaluates whether a model can remain responsive within smartphone power and memory limits (Janapa Reddi et al. 2022), measuring image classification, object detection, image segmentation, and language-processing workloads. MLPerf Client (MLCommons 2026b) addresses the local-computing decision: whether consumer devices can run AI workloads directly rather than relying on cloud inference. Its current emphasis on local generative-AI and LLM workloads makes CPUs, discrete GPUs, and integrated NPUs part of the benchmarked system rather than incidental host hardware. MLPerf Tiny (C. Banbury et al. 2021) tests the extreme constraint case: embedded and ultra-low-power AI systems, such as IoT devices, wearables, and microcontrollers. These variants preserve the same benchmark discipline while changing the binding resource from data center throughput to client responsiveness, mobile power, or microcontroller memory.

12.8.4.2 MLPerf execution scenarios

The same hardware can report dramatically different benchmark numbers depending on *how* requests arrive—a fact that explains why vendor claims often fail to predict production performance. Classic MLPerf Inference defines four execution scenarios that characterize distinct traffic patterns, each requiring different optimization strategies (Reddi et al. 2019). Current client and generative-AI benchmark variants also include interactive measurements for latency-sensitive LLM workloads, where metrics such as time-to-first-token and time-per-output-token become central (MLCommons 2026b).

SingleStream SingleStream processes one request at a time, measuring latency for sequential inference. This scenario models mobile and embedded applications where a single user interacts with the device: a smartphone camera app classifying images, a voice assistant processing speech, or a wearable detecting gestures. The key metric is per-request latency, and batching provides no benefit since requests arrive only after the previous result is consumed. Optimization focuses on preprocessing efficiency and power consumption rather than throughput.

MultiStream MultiStream processes multiple synchronized input streams simultaneously, modeling scenarios like autonomous vehicles with multiple cameras that must be processed together for spatial fusion. Unlike SingleStream’s sequential requests, MultiStream requires processing frames from all sensors within tight video-rate deadlines. The key distinction from Server mode is that MultiStream inputs arrive in lockstep, while Server requests arrive independently and unpredictably. The key constraint is synchronization: all streams must complete before the planning module can act. Optimization focuses on jitter handling and meeting hard deadlines rather than average throughput.

Server Server generates requests following a Poisson distribution, simulating cloud API traffic where requests arrive independently and unpredictably. This scenario models web services handling millions of queries from different users. Unlike SingleStream’s guaranteed sequential arrival, Server traffic creates queuing dynamics where multiple requests compete for resources. The key metrics are throughput (queries per second) and tail latency (p99), and dynamic batching can improve efficiency by grouping requests that arrive within a time window. Optimization balances throughput against latency SLOs.

Offline Offline provides all inputs upfront, measuring maximum throughput when latency constraints are removed. This scenario models batch processing pipelines: overnight data processing, scientific computing, or precomputing recommendations. With no latency requirement, systems can use maximum batch sizes to saturate hardware utilization. The key metric is pure throughput (samples per second), and optimization focuses entirely on hardware efficiency.

Table 12.12 maps the classic execution scenarios, plus the newer Interactive LLM-oriented case, to their deployment contexts and optimization strategies.

Table 12.12: MLPerf Execution Scenarios: Classic MLPerf Inference scenarios map to distinct deployment contexts, each requiring different optimization strategies, and newer Interactive scenarios extend the framework to latency-sensitive LLM workloads. SingleStream and MultiStream prioritize latency, Server balances throughput and latency, Offline maximizes throughput, and Interactive emphasizes token-level responsiveness. Matching the scenario to deployment context determines which benchmark results are relevant.

Scenario	Context	Strategy	Focus
SingleStream	Mobile apps, embedded devices	No batching (batch = 1)	Preprocessing, power efficiency
MultiStream	Autonomous driving, video analytics	Synchronized sensor fusion	Jitter handling, deadline guarantees
Server	Cloud APIs, web services	Dynamic batching with timeout	Throughput-latency trade-off tuning
Offline	Batch processing, data pipelines	Maximum batch size	Throughput, hardware utilization
Interactive	Chat, agents, local generative AI	Token streaming, KV-cache management	Time-to-first-token, time-per-output-token

Each scenario acts as a workload contract by fixing which form of latency or throughput a valid result must preserve.



Lighthouse 12.2: MobileNetV2 on EdgeTPU

Extending lighthouse 12.1, the hardware-acceleration claim from chapter 11 is evaluated using an illustrative SingleStream-style edge-inference scenario (Reddi et al. 2019; C. Banbury et al. 2021).

Hardware acceleration claim: In this illustrative edge-accelerator scenario, assume INT8 MobileNetV2 inference takes ~2 ms on the accelerator, approximately 7.5× faster than a Cortex-M-class CPU (~15 ms). Actual results depend on operator coverage, clock frequency, thermal state, and implementation.

Table 12.13 reports the illustrative SingleStream-style scenario assumptions.

Table 12.13: EdgeTPU vs. Cortex-M7 MobileNetV2 Scenario: Illustrative SingleStream-style inference latency, end-to-end latency, power, and energy values show how preprocessing overhead narrows the headline accelerator speedup; these are not controlled measurements.

Metric	CPU (Cortex-M7)	EdgeTPU	Headline Ratio	System-Level Ratio
Inference latency	~15 ms	~2 ms	7.5× faster	—
End-to-end latency	~18 ms	~6 ms	—	~3× faster
Power consumption	~120 mW	~500 mW	—	~4.2× higher
Energy per inference	~1.8 mJ	~1 mJ	—	~1.8× more efficient

What this reveals: Under these assumptions, the 7.5× inference speedup narrows to ~3× end to end because preprocessing runs on the CPU in both cases. EdgeTPU uses more active power but completes faster, yielding lower inference energy; deployment requires controlled measurement.

The deployment decision depends on the workload. For battery-powered devices running infrequently, the active inference calculation alone favors EdgeTPU, but total battery impact depends on sleep power, wake-up energy, host-transfer overhead, and whether the accelerator adds idle leakage while the system waits. For continuous video operation, EdgeTPU's lower active energy per inference is much more likely to dominate.

The SingleStream result illustrates why benchmarking requires matching the MLPerf scenario to the deployment context: SingleStream emphasizes latency, while Offline benchmarks would give different conclusions optimized for throughput rather than latency.

The scenarios explain why the same hardware can report dramatically different benchmark numbers. In an illustrative comparison, an accelerator with high Offline throughput can sustain much lower Server-mode throughput once p99 latency constraints and queuing overhead are enforced,

because Server mode cannot always use maximum batch sizes. When evaluating hardware for a specific application, selecting the appropriate scenario ensures benchmark results predict production performance. The MobileNetV2 result therefore makes scenario selection a deployment requirement rather than a reporting choice.

Training benchmarks measure learning speed; inference benchmarks measure serving speed. Yet both measures share a critical blind spot: they say nothing about *how much energy* the system consumes to achieve that speed. A system that sets throughput records while consuming kilowatts of power may be economically unsustainable or physically impossible to deploy at the edge. Completing the evaluation picture requires power measurement: measuring the energy cost of performance.

12.9 Power Measurement Techniques

A chip vendor advertises “10 TOPS at 0.5 W,” but under sustained inference load, thermal throttling drops actual throughput to 3 TOPS at 2 W. Without standardized power measurement, this 13.3× efficiency gap between the datasheet and reality goes undetected until deployment.

This efficiency dimension is critical because chapter 11 established TOPS/W as a primary design objective alongside raw TOPS. Power benchmarks validate whether efficiency-optimized accelerators deliver their promised energy savings. TOPS/W is particularly susceptible to gaming precisely because it is a ratio of two separately quotable peaks: a vendor can read the numerator (operations) at the batch size and precision that maximize throughput and the denominator (watts) at a near-idle operating point, so the advertised efficiency describes a state the chip never occupies under real load. Power benchmarks close that loophole by fixing the workload and the measurement window, forcing the numerator and denominator to be read at the same operating point.

Measuring power differs fundamentally from measuring execution time. Latency is a single interval, but instantaneous power ($P = V \cdot I$) fluctuates continuously across microsecond execution phases and drifts over minutes as silicon heats up, increasing temperature-dependent leakage current. Furthermore, ML deployments span an extreme physical range. As cataloged in table 12.14, representative systems span nearly eight orders of magnitude, from microwatt-scale wake-word coprocessors to kilowatt-scale data center racks (Henderson et al. 2020).

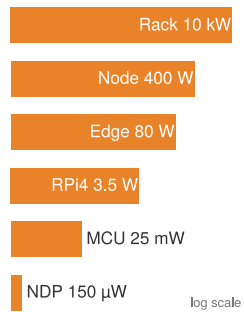
Table 12.14: Power Consumption Spectrum: The representative deployment points listed here span nearly eight orders of magnitude in power demands, from microwatt-scale TinyML devices (150 μW) to kilowatt-scale ML server racks (10 kW); figure 12.6 extends the same picture down to 5.6 μW at the TinyML floor and up to roughly 498 kW at the training-cluster ceiling, covering nearly eleven orders of magnitude. This enormous range explains why no single measurement technique or efficiency metric applies universally: benchmarking a 150 μW neural processor requires fundamentally different instrumentation than measuring a 10 kW server rack.

Category	Device Type	Power Consumption
Tiny	Neural Decision Processor (NDP)	150 μW
Tiny	M7 Microcontroller	25 mW
Mobile	Raspberry Pi 4	3.5 W
Mobile	Smartphone	4 W
Edge	Smart Camera	10–15 W
Edge	Edge Server	65–95 W
Cloud	ML Server Node	300–500 W
Cloud	ML Server Rack	4–10 kW

Because power spans from microwatts to kilowatts, no single instrument or measurement protocol applies everywhere. Benchmarking an always-on neural sensor requires measuring microampere currents at millivolt rails, whereas benchmarking a server rack requires measuring high-voltage three-phase alternating current at the power distribution unit. This divergence forces benchmarking standards to establish precise physical boundaries for what hardware is included in the energy ledger.

12.9.1 Power measurement boundaries

Where an engineer draws the system boundary dictates what counts as efficient. If a benchmark measures only the accelerator core, it ignores the host CPU driving the PCIe bus, the off-chip DRAM holding weights, and the cooling fans dissipating heat. Figure 12.7 illustrates how MLPerf



Deployment power spans nearly eight orders, μW to kW.

defines measurement boundaries across three deployment tiers: components inside the solid green enclosures fall within the energy accounting boundary, while components with red dashed outlines are explicitly excluded (Tschand et al. 2024).

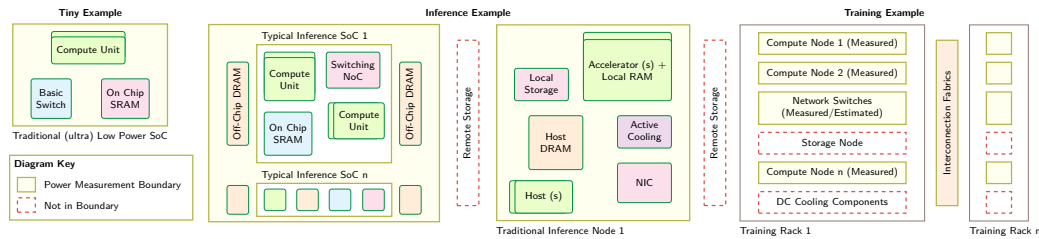


Figure 12.7: Power Measurement Boundaries: MLPerf defines system boundaries for power measurement, ranging from single-chip devices to full data center nodes, to enable fair comparisons of energy efficiency across diverse hardware platforms. These boundaries delineate which components’ power consumption is included in reported metrics, impacting the interpretation of performance results. Source: (Tschand et al. 2024).

The three tiers in figure 12.7 reflect distinct hardware interfaces. In TinyML deployments, the entire low-power SoC—compute cores, internal SRAM, and power management units—falls inside the boundary, measured directly at the DC power rail. Dedicated inference nodes expand this perimeter to include host CPUs, accelerator cards, PCIe buses, local storage, and on-chassis active cooling fans, while excluding remote storage networks. Multi-rack training deployments measure compute nodes and top-of-rack interconnect switches, but exclude centralized data center chillers, facility-level power distribution units (PDUs), and disaggregated storage clusters.

The measurement boundary determines which components are counted; workload composition determines where their energy is spent. On mobile devices running TensorFlow Mobile, data movement accounts for 57.3 percent of total inference energy (Boroumand et al. 2018), exposing why operation counts alone fail to predict power consumption. The worked example in Notebook 12.5 decomposes MobileNetV2 inference energy to demonstrate how moving from FP32 to INT8 attacks both the memory-movement bottleneck and arithmetic switching costs.

Napkin Math 12.5: Why INT8 saves energy

Problem: Under the stated 45 nm component-energy model, how much does replacing FP32 with INT8 reduce MobileNetV2 weight-read and arithmetic energy?

Recall from chapter 11 that moving data costs far more energy than computing on it (the energy-movement invariant formalized in chapter 4 and quantified by Horowitz’s energy estimates (Horowitz 2014)). Understanding *why* quantization reduces energy consumption requires decomposing energy into its physical sources. Two dominant factors determine inference energy: compute operations and memory access.

Narrower datatypes generally require less switching and storage energy per operation, so table 12.15 reveals an 18× gap between FP32 and INT8 multiply cost:

Table 12.15: Per-Operation Energy by Precision: Energy cost of a single multiply at FP32, FP16, and INT8, with relative cost normalized to FP32, illustrating why narrower datatypes reduce arithmetic energy. Values follow the 45 nm estimates in Horowitz (2014) and section D.1.

Precision	Multiplier Energy	Relative Cost
FP32	~3.7 pJ/FLOP	1×
FP16	~1.1 pJ/FLOP	0.3×
INT8	~0.2 pJ/FLOP	0.05×

An 8-bit multiplier uses ~18× less energy than a 32-bit floating-point multiplier in this 45 nm component model because narrower arithmetic reduces switching and storage work (Horowitz 2014). Section D.1 catalogs the per-operation estimates behind these ratios; absolute values and ratios vary with circuit design and process technology.

Table 12.16 extends the picture to memory access, with energy cost per byte across each tier of the hierarchy:

Table 12.16: Memory Access Energy by Tier: Per-byte energy cost across the register-to-DRAM hierarchy, with relative cost normalized to a register read, exposing why off-chip memory traffic dominates inference energy budgets. Values follow the same canonical energy-estimate model used in Horowitz (2014) and section D.1.

Memory Level	Energy per Byte	Relative Cost
Register	~0.1 pJ/byte	1×
L1 Cache	~1 pJ/byte	10×
L2 Cache	~5 pJ/byte	50×
DRAM	~160 pJ/byte	1,600×

Memory access dominates: reading one byte from DRAM costs over 1,600× more energy than a register access.

Math: Component energy follows $E_{\text{load}} = \text{model bytes} \times \text{DRAM energy per byte}$ and $E_{\text{compute}} = \text{FLOPs} \times \text{operation energy}$. The FP32 terms sum as $2243 \mu\text{J} + 2,220 \mu\text{J} = 4,463 \mu\text{J}$; the INT8 terms sum as $561 \mu\text{J} + 120 \mu\text{J} = 681 \mu\text{J}$. Dividing the totals gives a 6.6× reduction under this simplified model.

Table 12.17 combines the two effects in a deliberately simplified MobileNetV2 model. It counts one DRAM read per weight and charges every cataloged FLOP at the listed multiplier energy, while excluding activation traffic, cache behavior, additions as a distinct cost, control overhead, and static power:

Table 12.17: MobileNetV2 INT8 Simplified Energy Estimate: One weight read plus a multiplier-energy charge for every cataloged FLOP under a 45 nm component-energy model. This is not a whole-device inference measurement.

Component	FP32 (14 MB)	INT8 (3.5 MB)	Savings
One weight read from DRAM	2243 μJ	561 μJ	4×
Compute (600 MFLOP)	2,220 μJ	120 μJ	18.5×
Total	4,463 μJ	681 μJ	6.6×

Systems insight: Within this simplified model, the weight read and the arithmetic contribute comparable FP32 energy (2243 μJ and 2,220 μJ), and INT8 reduces both, taking the total from 4,463 μJ to 681 μJ . This explains the physical mechanism behind potential savings, but battery-life or per-inference claims require controlled whole-device power measurements.

Beyond individual server nodes, large-scale deployments introduce shared infrastructure that complicates boundary definition. In data centers, cooling systems and power delivery account for 20

to 30 percent of total facility electricity consumption (Barroso et al. 2019). This facility-level overhead is captured by power usage effectiveness (PUE):

$$\text{PUE} = \frac{\text{Total Facility Energy}}{\text{IT Equipment Energy}}$$

PUE values range from approximately 1.1 in hyperscale facilities using direct-to-chip liquid cooling to over 2.0 in legacy facilities relying on computer room air conditioning (Barroso et al. 2019). When an evaluation reports only server-plug energy, it excludes this facility multiplier. Furthermore, dynamic voltage and frequency scaling (DVFS) continuously modulates processor power and heat dissipation during execution (Kim et al. 2008), creating thermal feedback loops that alter the cooling overhead itself.

12.9.2 Computational efficiency vs. power consumption

Total power in complementary metal-oxide-semiconductor (CMOS) digital circuits is the sum of dynamic switching power and static leakage power:

$$P = P_{\text{dynamic}} + P_{\text{leakage}} = \alpha CV^2 f + I_{\text{leak}} V$$

where α is the gate switching activity factor, C is physical load capacitance, V is supply voltage, f is clock frequency, and I_{leak} is temperature-dependent leakage current. Historically, computations per kilowatt-hour doubled approximately every 1.5 years—a long-run efficiency scaling trend known as Koomey’s law (Koomey et al. 2011). Within any single hardware generation, however, dynamic power limits throughput scaling. Increasing clock frequency f requires scaling supply voltage V approximately linearly to maintain transistor switching margins, producing a cubic surge in dynamic power dissipation:

$$P_{\text{dynamic}} \propto f^3$$

Doubling clock frequency to double scalar throughput can incur an eightfold penalty in dynamic power (Le Sueur and Heiser 2010). This cubic penalty explains why modern ML accelerators favor massive parallel arrays of simple execution units clocked at moderate frequencies over narrow pipelines clocked at extreme frequencies.

Because frequency scaling quickly hits TDP ceilings, system designers optimize efficiency across the Data, Algorithm, and Machine axes. On the algorithm and data side, reduced-precision integer arithmetic (INT8 or FP8) directly reduces switching capacitance C in the ALUs and compresses memory traffic across interconnect buses, preserving model accuracy while cutting dynamic energy (Jacob et al. 2018; Wu et al. 2020; Gholami et al. 2021). On the machine side, systems operate near the energy-delay product (EDP) minimum rather than peak clock frequency. On battery-constrained mobile devices, operating at this peak efficiency point preserves battery life; in data centers, it maximizes throughput per megawatt of provisioned power capacity.

12.9.3 Standardized power measurement

General-purpose power benchmarks such as SPECpower_ssj2008 measure server power across stepped load levels (0 to 100 percent in 10 percent increments) using steady-state transactional workloads (Lange 2009). In contrast, ML workloads exhibit extreme temporal volatility: instantaneous power swings rapidly between compute-bound matrix multiplications (where systolic arrays or Tensor Cores draw peak current) and memory-bound stalls or token transfers (where execution units stall waiting for DRAM or interconnect fetches).

Characterizing these fluctuations requires calibrated physical instrumentation. While software profilers can read on-chip energy counters such as Intel RAPL or NVIDIA NVML, these interfaces carry two flaws: their sampling queries consume host cycles that distort execution, and their internal models are rarely traceable to calibrated laboratory standards. Standardized benchmarks such as MLPerf Power therefore mandate external, calibrated physical power analyzers (such as Yokogawa or Keithley analyzers) connected via inline current shunts or Hall-effect sensors to AC mains or DC supply rails (Tschand et al. 2024).

To avoid aliasing transient spikes across microsecond kernel boundaries, power analyzers must sample current and voltage at kilohertz frequencies, integrating instantaneous power into total energy:

$$E = \int_{t_{\text{start}}}^{t_{\text{end}}} P(t) dt$$

To prevent cold-start artifacts from distorting results, the benchmarking harness enforces three protocol stages:

1. A warm-up phase runs unmeasured inferences until instruction caches, weight buffers, and silicon junction temperatures reach steady-state thermal equilibrium.
2. A sustained measurement window executes repeated inferences across minutes to average over microarchitectural phase shifts and thermal drift.
3. Hardware-to-software synchronization pairs analyzer power timestamps with query dispatch timestamps from the load generator.

Workload structure further alters the power profile. In Transformers, the autoregressive decode phase is memory-bandwidth bound and draws modest average power, while the prompt prefill phase executes large matrix multiplications that generate sharp power spikes. Recommendation models like DLRM, dominated by sparse embedding table lookups across DRAM, spend more energy on memory bus transfers than arithmetic logic. Batch sizing creates another trade-off: larger batches amortize parameter reads over more inputs, lowering the energy consumed per inference, but elevate instantaneous power draw toward the accelerator's thermal limit. At the edge, duty cycles govern the outcome: for an intermittent wake-word detector operating at a 1 percent duty cycle, baseline sleep and leakage power consume over 90 percent of battery capacity, rendering active inference efficiency secondary to low-power sleep state management.

12.9.4 MLPerf power case study

MLPerf Power (Tschand et al. 2024) translates raw electrical measurements into standardized efficiency metrics: completed inferences per second per watt under a rigorously defined system boundary. This protocol applies uniform measurement principles across data center, edge, and tiny inference scenarios, where the binding constraint shifts from rack cooling and operating cost to battery runtime and microwatt energy harvesting.

Standardizing the boundary prevents misleading comparisons across architectural families. By requiring external power analyzers, calibrated logging, and fixed latency thresholds, MLPerf Power ensures that reported efficiency numbers reflect sustained execution rather than unachievable burst states.

Over successive benchmark submission rounds from 2021 to 2024, standardized measurements capture how architectural improvements, compiler optimizations, and lower-precision arithmetic translate into production efficiency gains. The data center results in figure 12.8 illustrate these normalized efficiency trajectories across diverse machine learning workloads.

RetinaNet and ResNet show the largest plotted gains, each reaching $3\times$ its initial normalized efficiency. BERT and GPT-J reach about $1.9\times$, while DLRM-v2 reaches about $1.7\times$, Llama 2 about $1.5\times$, and RNN-T about $1.3\times$. These disparities reflect the underlying arithmetic intensity of each workload. Convolutional networks with static shapes and high operational intensity (RetinaNet and ResNet) benefited immediately from mature INT8 Tensor Core execution and kernel fusion. In contrast, large language models (GPT-J and Llama 2) and recommendation models (DLRM-v2) remain bounded by memory bandwidth during autoregressive decoding and sparse embedding lookups, where efficiency gains depend on memory subsystem innovations rather than raw compute scaling.

Timing protocols and power instrumentation provide the raw data for benchmarking. Raw data alone, however, does not guarantee sound conclusions. Converting measurements into meaningful comparisons requires understanding the systematic sources of error, bias, and misalignment that can make even carefully collected benchmark numbers misleading.

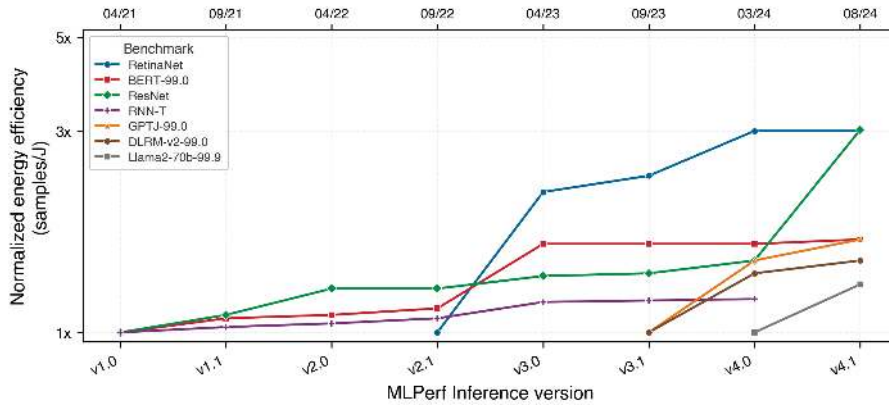


Figure 12.8: Data-Center Energy Efficiency Gains: Each series is normalized to its own first plotted version, so compare improvement within a workload rather than absolute efficiency across workloads. Source: (Tschand et al. 2024).

12.10 Benchmarking Best Practices

An inference stack that passes a steady-state lab run can still miss latency targets when production traffic arrives in bursts, or when the input mix shifts toward expensive examples. Training throughput, inference latency, and power efficiency each have established measurement protocols validated through MLPerf, but knowing what to measure is insufficient without understanding what benchmarks cannot capture and why this gap causes production deployments to violate their service-level objectives.

Benchmarks make simplifying assumptions that enable standardized comparison but diverge from production reality. Training suites freeze datasets and tightly specify randomness; production data and retraining conditions evolve continuously. Inference suites emphasize a defined steady-state scenario; production traffic exhibits bursty arrival distributions and variable sequence lengths. Power benchmarks control thermal environments; deployed accelerators encounter variable ambient temperatures and sustained thermal loads. Four categories of limitations—statistical, deployment-related, system design, and organizational—determine whether benchmark results translate to deployment success.

12.10.1 Statistical and methodological issues

Benchmark results are only as reliable as the statistical apparatus that produces them. Three core methodological issues routinely undermine this reliability.

Incomplete problem coverage occurs when a benchmark exercises only a narrow operating point of a system. Controlled datasets such as CIFAR-10 (Krizhevsky 2009) constrain inputs to uniform, low-resolution images (32×32 pixels) that fit entirely within on-chip caches (L2/L3). This small working set masks HBM bandwidth bottlenecks and eliminates the dynamic memory allocation, kernel launch overheads, and uncoalesced memory transactions that dominate production vision pipelines. A model that achieves high accuracy and throughput on a uniform, downscaled benchmark can stall in deployment when exposed to variable image resolutions, arbitrary aspect ratios, and sensor noise that force frequent kernel recompilations or fallback execution paths.

Statistical insignificance arises when evaluations rely on too few trials to distinguish algorithmic improvements from measurement noise, particularly when the evaluation medium introduces variance. Large language model evaluation exemplifies this vulnerability: whether scoring candidate generations against a reference via human preference ratings or an LLM-as-judge protocol, scores vary substantially based on prompt phrasing, judge model alignment, and response ordering. A reported two-point win can vanish under an alternative prompt template or judge configuration. Rigorous evaluation demands paired hypothesis tests and bootstrap confidence intervals over hundreds of independent prompt pairings. Omitting these confidence bounds obscures whether a reported margin reflects a genuine architectural advance or stochastic evaluation noise.

Reproducibility poses a persistent systems hurdle across differing software environments. Benchmark measurements fluctuate significantly with minor revisions to compilers, numerical runtimes, or mathematical libraries. For example, replacing a vendor Basic Linear Algebra Subprograms (BLAS) library with an autotuned kernel compiler alters thread-block tile dimensions, register allocations, and shared-memory bank conflicts. Furthermore, the non-associativity of floating-point arithmetic causes parallel reductions across thread blocks to produce divergent bitwise outputs depending on thread scheduling. MLPerf addresses reproducibility by providing reference implementations, fixed random seeds, and audited submission rules. Even under strict rules, tracking performance across diverse accelerator architectures requires pinning library dependencies, compiler flags, and execution runtimes to prevent silent configuration drift.

12.10.2 Laboratory-to-deployment performance gaps

Statistical rigor ensures that benchmark measurements are precise, but precise measurements of the wrong operational regime still lead to production failures. Benchmark suites must align with the physical constraints of deployment.

Misalignment with real-world goals arises when benchmarks isolate single scalar metrics while production environments require navigating multi-objective Pareto frontiers. Standard inference benchmarks typically evaluate offline throughput by saturating an accelerator with static batches under closed-loop load. Production servers, by contrast, operate as open queuing systems subject to bursty Poisson or Pareto arrival patterns. As accelerator utilization approaches saturation, queuing delay escalates non-linearly, causing tail latencies ($p99$ and $p99.9$) to violate strict service-level agreements even while average latency appears acceptable. Furthermore, offline benchmarks evaluate isolated models with dedicated access to accelerator HBM. In production, multi-tenant serving engines, dynamic key-value (KV) cache allocations for autoregressive decoding, and host-device data transfers over PCIe contend for the same memory capacity and bus bandwidth, degrading operational throughput below lab predictions.

12.10.3 System design challenges

Statistical methodology and deployment alignment govern how performance is measured and targeted. A third category of limitations stems directly from the physical hardware executing the workload. Accelerator behavior fluctuates with thermal conditions, operating system resource contention, and underlying microarchitectural topology.

Environmental and operating conditions alter benchmark results through physical hardware mechanisms. Accelerators operate under strict TDP envelopes and silicon junction temperature limits. When sustained computation dissipates heat faster than cooling solutions can extract it, onboard controllers invoke dynamic voltage and frequency scaling (DVFS) to downclock compute engines, dropping clock frequencies and arithmetic throughput. A brief benchmark run on an idle, cool accelerator captures transient boost clocks, whereas continuous production execution operates at lower steady-state frequencies, incurring a 15 to 25 percent throughput reduction. Similarly, unmanaged background operating system threads, host CPU interrupts, and PCIe bus contention introduce latency jitter that distorts tail-latency measurements unless execution environments enforce CPU core pinning and dedicated memory allocation.

The hardware lottery³⁴ (Hooker 2021) compounds this physical variation by coupling algorithmic viability to specialized hardware support. Modern accelerators allocate the vast majority of their silicon area to dense matrix multiplication units (such as Tensor Cores or systolic arrays) backed by wide vector units and high-bandwidth memory hierarchies. Algorithmic architectures such as dense Transformers achieve high arithmetic intensity by reusing weights across batched tokens, utilizing a large fraction of peak theoretical FLOP/s. Conversely, architectures that rely on dynamic control flow, fine-grained sparsity, or graph message passing suffer from uncoalesced memory transactions and thread divergence, underutilizing available compute silicon. Algorithmic performance on a benchmark often reflects how naturally a model's computation graph maps onto dominant accelerator execution units rather than intrinsic mathematical superiority.

Hardware compatibility dependence introduces systematic biases when evaluating models across heterogeneous execution targets. An architecture optimized for GPU execution may underperform on an embedded CPU, DSP, or custom edge accelerator with differing vector widths, cache hierarchies,

³⁴ | **Hardware lottery:**

Coined by Hooker (2021) to describe how algorithmic success depends on alignment with available hardware and software. Transformer workloads benefit from dense matrix operations that map efficiently to modern accelerators, while graph neural networks and sparse mixture-of-experts models can be harder to evaluate when available silicon and software stacks favor dense kernels. Hardware-specific leaderboards therefore favor architectures aligned with the measured platform and may obscure alternatives that would fare differently under other hardware assumptions.

and quantization support. Figure 12.9 makes this hardware dependence concrete by comparing model performance across different platforms. On the CPU uint8 and GPU configurations, the multi-hardware models track the “MobileNetV3 Large min” baseline closely, reaching roughly 77 percent top-1 ImageNet accuracy where the baseline reaches about 75 percent. On the EdgeTPU and DSP hardware the same multi-hardware models sustain that 77 percent at substantially lower latency, while a model tuned only for the CPU would forfeit those gains. This reveals that the “best” model depends entirely on deployment target: a conclusion impossible to reach from single-platform benchmarks.

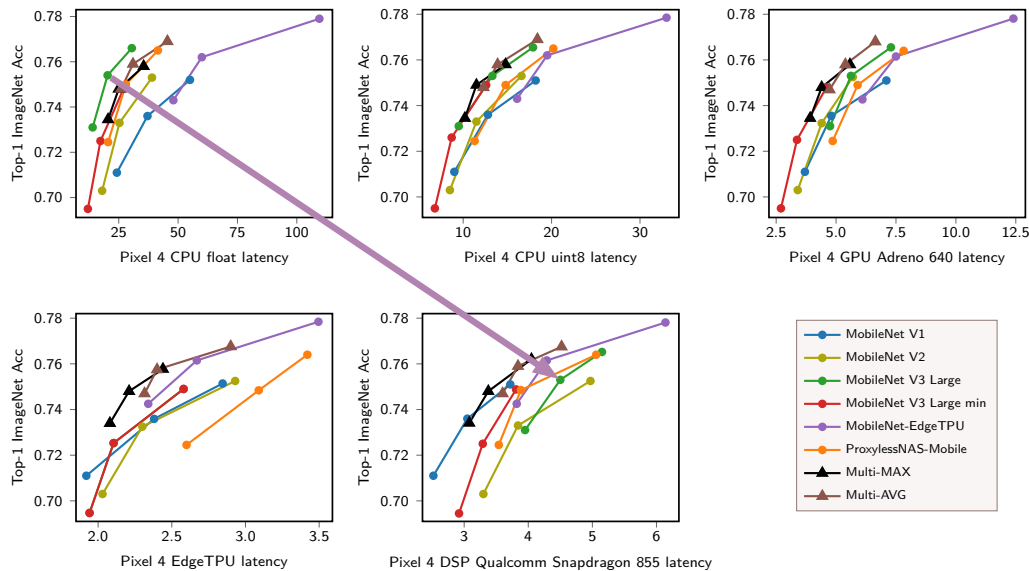


Figure 12.9: Hardware-Dependent Latency–Accuracy Trade-offs: Each panel holds top-1 ImageNet accuracy on the vertical axis while using a target-specific latency scale horizontally. The arrow links MobileNetV3 Large at roughly equal accuracy on CPU float and DSP, exposing the hardware-dependent latency shift. Source: (Chu et al. 2021).

Without benchmarking across heterogeneous hardware configurations, system designers risk selecting architectures solely because they conform to existing accelerator memory hierarchies and instruction sets. Evaluating models across diverse execution engines ensures that architectural decisions reflect fundamental computational efficiency rather than platform-specific co-design artifacts.

12.10.4 Organizational and strategic issues

Technical limitations—statistical noise, deployment misalignment, environmental variance, and hardware compatibility—address measurement mechanics. A distinct category of failure emerges from human incentives. Commercial procurement and academic leaderboard incentives create systematic biases in how benchmarks are executed and reported. Mitigating these organizational dynamics demands formal governance mechanisms and community audit standards to preserve benchmark integrity.

12.10.4.1 Benchmark engineering

While the hardware lottery is an unintended consequence of hardware specialization, benchmark engineering is the deliberate practice of tailoring models or systems to exploit the idiosyncrasies of specific benchmark harnesses. This practice yields inflated performance metrics that collapse under production conditions.

Benchmark engineering occurs when developers tune hyperparameters, preprocessing pipelines, or execution paths specifically to maximize benchmark scores rather than generalizable performance. The boundary between legitimate system optimization and benchmark engineering lies where tuning

compromises out-of-distribution robustness. For example, an inference pipeline might hardcode input tensor dimensions and pad batches specifically to match the memory stride of an accelerator’s GEMM tile configuration, achieving peak throughput on fixed benchmark shapes while triggering costly kernel fallbacks and memory reallocations on arbitrary production inputs. Similarly, fine-tuning an evaluation pipeline to match the formatting biases of an automated evaluation model produces high leaderboard rankings that fail when interacting with unstructured human dialogue.

Procurement contracts and marketing claims create strong incentives to optimize directly for public benchmarks. When benchmark rankings dictate commercial adoption, engineering resources concentrate on optimizing for the test harness at the expense of general utility—the Goodhart’s Law failure mode introduced in section 12.1 and illustrated with the BLEU-score example in section 12.3.1.

12.10.4.2 Bias and over-optimization

System engineers evaluating benchmark results must distinguish legitimate architectural gains from benchmark engineering. Several technical practices expose this gap. Transparency provides the initial defense: submissions that publish complete compiler flags, kernel configurations, and execution traces allow independent evaluators to separate generalized efficiency gains from harness-specific tuning. Reporting paired metrics from both controlled benchmarks and uncured production traces exposes discrepancies in latency distributions and memory utilization. In addition, evaluating across multiple independently maintained benchmark suites raises the barrier against overfitting, as an optimization tailored to the synthetic quirks of one harness rarely transfers across diverse test suites.

Standardization and third-party verification enforce accountability. Independent audits verify whether reported speedups reproduce across identical hardware environments, as demonstrated in MLPerf’s reference-vs-submission validation (section 12.10.5). Application-specific stress testing evaluates operational failure modes that static benchmarks omit, such as exposing vision models to adverse environmental conditions or subjecting inference engines to skewed request traffic. Finally, cross-hardware benchmarking validates whether performance advantages stem from genuine algorithmic superiority or narrow alignment with a single accelerator’s memory hierarchy.

12.10.4.3 Benchmark evolution

Benchmarks are not static standards; they must evolve alongside model architectures and hardware capabilities. A benchmark that effectively separates system capabilities in one architectural generation becomes obsolete when models saturate its accuracy ceiling or when hardware innovations shift the primary system bottleneck. Stale benchmarks incentivize over-optimizing legacy computational kernels rather than advancing system efficiency.

This evolutionary cycle characterizes the history of machine learning benchmarks. Early deep learning benchmarks evaluated image classification and object detection on convolutional networks. As neural architectures expanded to large language models, recommendation systems, and generative AI, static vision benchmarks failed to capture long-context attention, autoregressive decoding, and sparse expert routing. In response, community benchmarks evolved to measure language understanding (A. Wang et al. 2018, 2019) and foundational model capabilities (Liang et al. 2022).

Benchmark evolution also expands the dimensions of measurement. While early benchmarks focused on throughput and accuracy, contemporary systems require multi-objective evaluation across energy efficiency, memory footprint, and tail latency under resource constraints. Figure 12.10 makes these disparate requirements concrete by mapping scientific applications across data rate and computation time. Specifically, Large Hadron Collider sensors must process data at rates approaching 10^{14} bytes per second with nanosecond-scale computation times, while mobile applications operate at 10^4 bytes per second with longer computational windows—a span of roughly ten orders of magnitude in data rate and six to seven in computation time. This range of requirements necessitates specialized benchmarks. For example, edge AI applications benefit from benchmarks like MLPerf that evaluate performance under resource constraints, and scientific application domains need their own “Fast ML for Science” benchmarks (Duarte et al. 2022).

Managing benchmark evolution introduces a fundamental trade-off between longitudinal stability and architectural relevance. Benchmarks must remain fixed long enough to allow rigorous cross-generation hardware and software comparisons. If benchmarks change too frequently, tracking

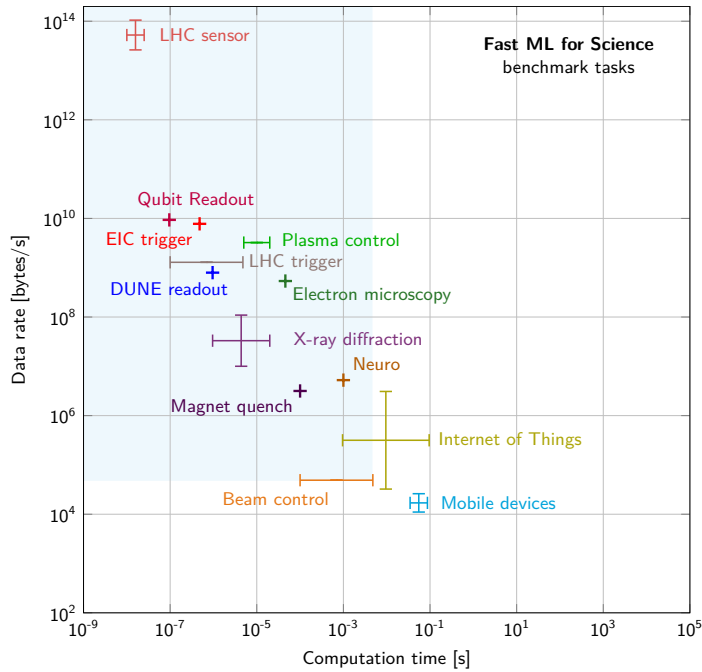


Figure 12.10: Performance Spectrum: Scientific and edge applications occupy widely separated operating points in data rate and available computation time, motivating benchmarks tied to each application's timing and data-movement constraints. Source: (Duarte et al. 2022).

historical progress becomes impossible. Conversely, freezing a benchmark indefinitely leads to metric stagnation and over-specialization. Modern benchmarking consortia balance this trade-off by operating on fixed release cycles that introduce new representative workloads while deprecating obsolete suites.

12.10.5 MLPerf synthesis and benchmark gaming

Benchmark gaming occurs when a compiler, runtime, or hardware stack optimizes specifically for the benchmark harness rather than the general workload it represents. In the Hennessy & Patterson tradition of quantitative systems architecture, benchmarks are active targets rather than passive instruments. The Goodhart dynamic introduced in section 12.1 applies directly: when a benchmark score drives procurement contracts or marketing claims, engineering effort shifts toward exploiting the idiosyncrasies of the evaluation harness. MLPerf counters benchmark gaming by combining reference implementations, strict submission audits, closed-division precision rules, and regular workload retirements that prevent stagnation.

In quantitative computer architecture, run rules must forbid three specific classes of benchmark gaming:

1. Precision truncation without disclosure: A compiler silently lowers numerical precision (for instance, executing intermediate activations in FP8 or INT8 instead of BF16) to inflate arithmetic throughput, without disclosing the reduced dynamic range or verifying whether the lower precision degrades numerical stability on production workloads.
2. Work pruning and operator elimination: An execution engine exploits fixed benchmark test sets to skip operations required by general production inputs, such as bypassing dynamic padding removal, masking out-of-bounds tokens, or skipping input boundary validation.
3. Benchmark fingerprinting: A runtime inspects input tensor dimensions, dataset hash fingerprints, or memory layout patterns to identify the benchmark suite, executing hardcoded, pre-tuned execution graphs and bespoke kernel configurations that are inaccessible to arbitrary user workloads.

MLPerf neutralizes these shortcuts through structural division of submissions, reference equivalence verification, and peer-reviewed code audits. In the Closed Division, preprocessing, postprocessing, and the model architecture must remain strictly equivalent to the reference implementation, and submissions must achieve rigorous quality targets (such as remaining within 99 percent of the FP32 reference accuracy). While submitters are encouraged to innovate in execution runtimes, memory layouts (such as converting NCHW to NHWC), and kernel fusion, run rules strictly prohibit input fingerprinting and non-generalizable shortcuts. Submissions that alter the model graph or retraining recipe are partitioned into the Open Division, where modifications are explicitly labeled and audited.

Even the most rigorous system benchmarks validate only one dimension of deployment readiness. Achieving record throughput and energy efficiency on an MLPerf workload confirms that the hardware and runtime stack can deliver high computational efficiency, but it cannot determine whether the deployed model generalizes to real-world distributions or whether the underlying dataset represents the operational population. In the D·A·M taxonomy, hardware throughput (Machine) is a necessary foundation, but algorithmic fidelity (Algorithm) and dataset integrity (Data) must hold simultaneously. Completing the validation stack requires evaluating across all three dimensions of the D·A·M taxonomy—ensuring that data quality, model capability, and hardware efficiency cohere under production constraints.

12.11 Model and Data Evaluation

A compressed model running on accelerated hardware can still fail if it was trained on biased data. System benchmarks can confirm that hardware delivers promised training throughput, inference latency, and power efficiency, but hardware validation alone cannot ensure deployment success. The optimization pipeline from Part III also included model compression (chapter 10) and data selection (chapter 9), each requiring its own validation. The remaining two dimensions of the framework address this gap: model benchmarks verify that compression preserved accuracy and critical model properties, while data benchmarks verify that training data enables robust generalization.

12.11.1 Model benchmarking

Model benchmarks validate whether compression techniques from chapter 10 preserved the properties that matter for deployment. This evaluation extends beyond top-line accuracy. A pruned model might maintain ImageNet accuracy while losing robustness to adversarial inputs. A quantized model might preserve average-case performance while degrading on rare but critical edge cases. A distilled model might match the teacher's accuracy while losing calibration. Historically, benchmarks focused almost exclusively on accuracy, but compression makes multi-dimensional evaluation essential.

ImageNet links model benchmarking to the hardware story from figure 12.1: error rates fell as GPU-enabled architectures became practical. Figure 12.11 traces that progression from 28.2 percent error in 2010 to 3.57 percent in the ImageNet Large Scale Visual Recognition Challenge (Russakovsky et al. 2015). The introduction of AlexNet³⁵ reduced the error rate from 25.8 percent to 16.4 percent. Subsequent models such as ZFNet, VGGNet, GoogLeNet, and ResNet³⁶ continued this trend, with ResNet achieving 3.57 percent (He et al. 2016a). This progression established the baselines against which model compression techniques are evaluated: a pruned ResNet must demonstrate how much accuracy it sacrifices for a given efficiency gain.

12.11.1.1 Accuracy metrics and their blind spots

The most common model metrics (accuracy, precision, recall, F1) each reveal different aspects of model behavior while hiding others, and understanding their blind spots is essential for compression validation. Top-*k* accuracy measures whether the correct label appears in the model's top-*k* predictions. Top-1 accuracy is strict; top-5 is lenient. The gap between them reveals model uncertainty: a model with 75 percent top-1 but 95 percent top-5 accuracy “knows” the answer is among a few candidates but struggles to commit. For deployment, the acceptable gap depends on whether downstream systems can use ranked predictions or require single answers.

Precision and recall matter when classes are imbalanced or errors have asymmetric costs (Sokolova and Lapalme 2009). A fraud detection model with 99 percent accuracy might have 10 percent recall on actual fraud (catching only one in 10 fraudulent transactions), an unacceptable outcome despite

³⁵ | **AlexNet:** The eight-layer CNN (60M parameters) that cut ImageNet top-5 error from 25.8 percent to 16.4 percent in 2012, trained on two GTX 580 GPUs with 3 GB memory each (Krizhevsky et al. 2012). AlexNet established a benchmarking paradigm that still informs vision evaluation: accuracy on a fixed dataset as the primary metric, with hardware configuration as a secondary specification. Later ImageNet results inherited this baseline comparison structure.

³⁶ | **ResNet (residual network):** Introduced by He et al. (2016a), skip connections enabled 152+ layer networks and achieved 3.57 percent top-5 ImageNet error (ensemble), surpassing the estimated human error rate reported in the ImageNet challenge context (Russakovsky et al. 2015). ResNet-50 became a common MLPerf Training reference workload because its moderate size (25.6M parameters) and well-understood compute profile (8.2 GFLOP per image) make it sensitive to both hardware and software optimizations without requiring multi-node setups (Mattson et al. 2020).

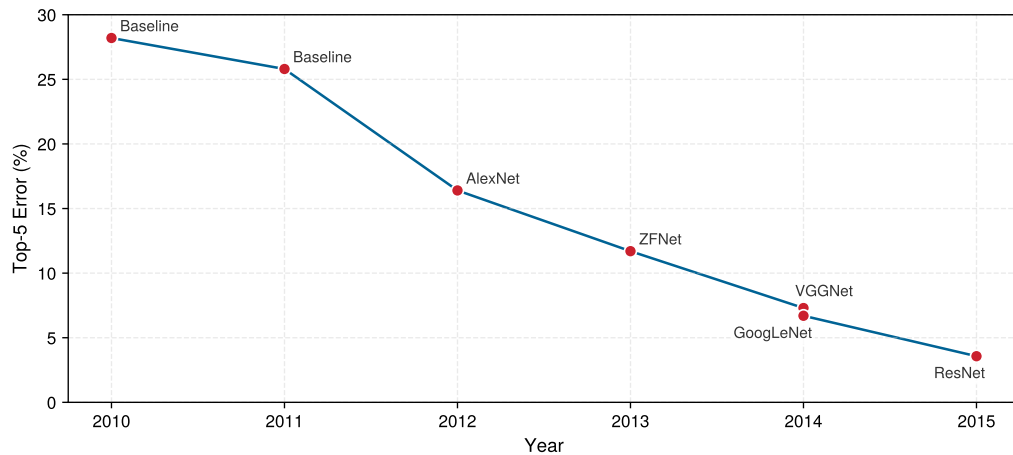


Figure 12.11: ImageNet Challenge Progression: AlexNet marks the largest one-year drop in the series; ZFNet, VGGNet, GoogLeNet, and ResNet extend that decline, defining the accuracy trajectory against which later compression trade-offs are judged. Sources: (Russakovsky et al. 2015; Krizhevsky et al. 2012; He et al. 2016a).

high accuracy. Precision (the fraction of predicted positives that are correct) and recall (the fraction of actual positives identified) expose failures that aggregate accuracy conceals.

Aggregate metrics also conceal subgroup failures. A model achieving 95 percent overall accuracy might achieve 60 percent on a critical demographic subgroup. The Gender Shades project (Buolamwini and Gebru 2018) revealed commercial gender-classification systems for facial analysis performing substantially worse on darker-skinned women than on lighter-skinned men, a disparity invisible to aggregate benchmarks. Disaggregated evaluation across deployment-relevant subgroups is essential; chapter 15 examines fairness evaluation systematically.

12.11.1.2 Calibration: When confidence scores matter

For many deployment scenarios, *how confident* the model is matters as much as *what* it predicts. A well-calibrated³⁷ model's confidence scores correspond to actual correctness probability: when it says "90 percent confident," it should be correct 90 percent of the time.

Compression can shift calibration even when preserving accuracy, a critical concern when validating quantization techniques from section 10.4. A quantized model might maintain headline accuracy while becoming overconfident on examples it gets wrong. This matters because post-hoc calibration techniques such as temperature scaling can only correct the problem if calibration is measured explicitly (Guo et al. 2017).

Calibration failures create downstream problems. An overconfident model automates errors that it should defer (predicted 95 percent confidence but wrong 30 percent of the time). An underconfident model sends too many correct predictions to review instead of automating decisions it can handle (predicted 70 percent confidence but correct 95 percent of the time). **Expected calibration error (ECE)** measures the gap between confidence and accuracy across confidence bins; reliability diagrams visualize this correspondence.

12.11.1.3 Compression validation: The efficiency-quality frontier

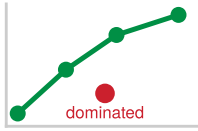
Model compression (chapter 10) trades model capacity for efficiency. Validation must determine whether compression achieved an acceptable trade-off or damaged capabilities that matter.

Pareto frontier³⁸ evaluation determines whether a compressed model represents a good trade-off. Plotting accuracy against the target efficiency metric (latency, model size, energy) reveals the trade-off frontier. Models on the Pareto frontier *cannot* improve one metric without degrading the other; models below the frontier are dominated by better alternatives.

Different compression techniques make different efficiency-quality trade-offs. Quantization reduces precision while often preserving aggregate accuracy (Jacob et al. 2018). Pruning induces

³⁷ | **Calibration:** From Arabic *qalib* (a mold for casting metal) via Latin *calibrare*, originally describing the adjustment of measuring instruments against known standards. In ML, calibration ensures predicted probabilities match empirical frequencies; Guo et al. (2017) formalize this concern for modern neural networks and show that temperature scaling is a simple effective post-hoc correction. The etymology is apt: just as an uncalibrated instrument produces precise but inaccurate measurements, an uncalibrated model produces confident but unreliable predictions, causing downstream systems that threshold on confidence scores to make systematically wrong decisions.

³⁸ | **Pareto frontier:** Named after economist Pareto (1896), the frontier contains all solutions where improving one objective requires degrading another. In compression benchmarking, the frontier's shape carries diagnostic information: a steep region means efficiency gains come cheaply (prune here), while a flat region means further compression costs disproportionate accuracy (stop here). Points below the frontier are strictly dominated and represent wasted capacity.



A dominated model loses on both axes of the Pareto frontier.

sparsity while retaining varying levels of test performance (Han et al. 2015; Gale et al. 2019). Distillation transfers knowledge from a larger teacher or ensemble into a smaller model (Hinton et al. 2015). Because these aggregate results do not establish preserved calibration or tail behavior, validation must evaluate those properties directly (Guo et al. 2017).

Evaluating calibration during compression requires a standardized protocol. Expected calibration error (ECE) compares predicted confidence with empirical accuracy, but it has no universal thresholds. The metric depends on the binning scheme, bin count, sample size, and prediction distribution. A benchmark protocol must therefore freeze the estimator and evaluate degradation relative to the uncompressed baseline under a fixed task tolerance.

Pareto optimality identifies nondominated choices, but it does not establish that any choice clears the deployment's absolute quality floor. Acceptable degradation depends on deployment context: a 2 percent accuracy drop might be acceptable for a recommendation system where users tolerate imperfect suggestions, but unacceptable for medical diagnosis where every false negative carries severe consequences. Production acceptance gates must define absolute accuracy thresholds before compression, evaluating aggregate accuracy, calibration stability, and deployment-critical slices simultaneously. The MobileNetV2 lighthouse makes this multi-dimensional validation protocol concrete.



Lighthouse 12.3: MobileNetV2 INT8 compression

Returning to lighthouse 12.1, consider an illustrative validation protocol for INT8 quantization, grounded in MobileNetV2's architecture (Sandler et al. 2018) and post-training quantization practice (Jacob et al. 2018). The values in table 12.18 are assumed:

Precompression baseline: MobileNetV2 achieves 71.8 percent top-1 accuracy on ImageNet at 3.5M parameters (14 MB FP32).

In table 12.18, aggregate accuracy barely changes after INT8 quantization to 3.5 MB, but calibration error and edge-case accuracy reveal substantial degradation. The INT8 model's ECE rises from 0.031 to 0.089; whether that increase is acceptable depends on a task-specific tolerance under the fixed ECE protocol.

Table 12.18: MobileNetV2 INT8 Postcompression Metrics: Illustrative FP32 and INT8 values for top-1 and top-5 accuracy, calibration error, and edge-case accuracy, showing how aggregate accuracy can mask calibration and tail-case degradation.

Metric	FP32	INT8	Acceptable?
Top-1 accuracy	71.8%	70.9%	✓ (0.9 pp drop; below 1 percentage-point threshold)
Top-5 accuracy	91%	90.4%	✓
Calibration ECE	0.031	0.089	No (degraded)
Edge-case accuracy	68.2%	61.4%	No (drop of 6.8 pp)

Edge-case definition: Images with > 50 percent occlusion, < 100 lux lighting, or > 30° rotation from training distribution (approximately 5 percent of real-world inputs).

What this reveals: Under these assumptions, average-case accuracy looks acceptable (0.9 percentage-point drop), but calibration degrades and edge-case accuracy drops 6.8 percentage points. If the deployment uses confidence thresholds (for example, “only act if confidence > 85 percent”) or encounters many edge cases (unusual lighting, partial occlusions), INT8 MobileNetV2 could fail despite passing aggregate benchmarks.

Fix: Apply temperature scaling post-hoc to improve calibration (Guo et al. 2017). Temperature scaling learns a single scalar T_{cal} to divide logits before softmax: $\text{softmax}(z_i/T_{\text{cal}})$. In parallel, add edge-case examples to the test set to monitor that specific failure mode continuously.

The Lottery Ticket Hypothesis (section 10.3.1.7) provides concrete benchmarking data illustrating what Pareto-efficient compression looks like. Through iterative pruning, Frankle and Carbin (2019) found sparse subnetworks (“winning tickets”) in fully connected and convolutional networks that

could match the original network’s test accuracy when trained in isolation. These empirical findings demonstrate the nonlinear shape of compression trade-offs: aggressive pruning preserves accuracy up to a critical sparsity threshold, beyond which quality degrades sharply. Compression validation must therefore establish empirical trade-off curves for each specific model and architecture, identifying where the model sits on the Pareto frontier and whether further pruning yields hardware efficiency or merely degrades accuracy.

12.11.1.4 Large language model benchmarks

The compression evaluation framework applies cleanly when the task has a stable label, such as classification accuracy, detection mAP, or segmentation IoU. Large language models break that pattern. A team can choose a model because it scores well on a public benchmark, then discover in deployment that the model recognizes multiple-choice facts but cannot generate a grounded answer, responds too slowly for an interactive product, or produces confident unsafe text that the benchmark never stressed. LLM benchmarking therefore starts by naming the deployment failure that a score is meant to rule out.

The metric taxonomy in table 12.19 organizes evaluation around those failure modes rather than leaderboard rankings. Its rows use Massive Multitask Language Understanding (MMLU),³⁹ HELM (Holistic Evaluation of Language Models),⁴⁰ and perplexity⁴¹ as examples of scores that answer distinct deployment questions:

Table 12.19: Failure-Oriented LLM Benchmark Taxonomy: LLM metrics are useful when each score is tied to the deployment failure it is meant to rule out. Recognition, holistic behavior, corpus prediction, and responsiveness expose different risks, so no single score establishes production readiness.

Deployment failure to rule out	Metric or benchmark family	What the score reveals	What the score cannot prove
The model recognizes facts poorly	MMLU (Massive Multitask Language Understanding)	Broad factual and disciplinary knowledge across fifty-seven subjects, with scores interpretable against chance-level multiple-choice performance	Whether the model can generate grounded open-ended answers rather than choose among multiple-choice options
The model is capable but unsafe	HELM (Holistic Evaluation of Language Models)	Accuracy alongside calibration, robustness, fairness, bias, toxicity, and efficiency	Whether one aggregate score captures the deployment risk; a model can be strong on accuracy and weak on calibration, safety, cost, or prompt stability
The model predicts its corpus well	Perplexity	Held-out next-token prediction on the same corpus; a perplexity of 10 means the model is “10-way confused” on average	Whether generated answers are helpful, safe, or grounded outside that corpus
The model feels slow in use	First-token latency, inter-token latency, and token throughput	Prompt-processing delay before generation starts and decode speed after generation begins	Whether a single throughput number hides poor interactive responsiveness, especially when batching improves throughput but worsens first-token latency

The responsiveness row requires a concrete timing anchor because LLM benchmarks frequently report a single aggregate throughput number even though users experience generation in distinct phases. A model can achieve high aggregate throughput in tokens per second while still feeling sluggish if the first token arrives late, or it can minimize first-token latency while generating subsequent text too slowly for interactive reading.

Token throughput translates these generation rates into user-perceived wall-clock time: for a response of 750 tokens, a generation rate of 25 tokens/s requires 30 seconds, whereas 100 tokens/s finishes in 7.5 seconds. Time-to-first-token and inter-token latency must therefore be reported together: one captures initial responsiveness, and the other captures sustained delivery speed. This latency split reflects contrasting computational regimes: time-to-first-token is governed by prompt prefill, which processes tokens in parallel and is typically compute-bound, whereas inter-token latency is governed by autoregressive decoding, where generating each single token requires reloading all model weights and KV cache tensors from high-bandwidth memory, binding throughput strictly to memory bandwidth.

³⁹ | **MMLU (massive multitask language understanding):** Introduced by Hendrycks et al. (2020) with 15,908 multiple-choice questions across fifty-seven subjects. MMLU’s benchmarking limitation is its format: multiple-choice recognition is not the same task as open-ended generation, so an MMLU score should not be read as direct evidence that a model can produce grounded free-form answers in production.

⁴⁰ | **HELM (holistic evaluation of language models):** Stanford’s 2022 evaluation framework tested a broad set of models across seven dimensions (accuracy, calibration, robustness, fairness, bias, toxicity, efficiency) (Liang et al. 2022). HELM’s contribution is methodological: by evaluating models that score similarly on accuracy but diverge on calibration or toxicity, it demonstrates that single-metric leaderboards systematically hide failure modes that matter for production deployment.

⁴¹ | **Perplexity:** From Latin *perplexus* (entangled); in information theory, $2^{H(p)}$ where H is entropy. A perplexity of 10 means the model is “10-way confused” on average. The systems consequence is interpretive rather than direct memory accounting: perplexity measures held-out next-token prediction on a corpus, while serving memory pressure is governed by context length, batch size, model shape, and decoding state; KV-cache management is a separate serving problem (Kwon et al. 2023).

A final failure mode is that a benchmark score may reflect training-data memorization rather than general capability. Benchmark contamination is a unique LLM risk because models trained on web-scale corpora may encounter benchmark questions during pretraining, inflating scores through memorization rather than skill (Xu et al. 2024). Leakage detection reframes this risk as something benchmark designers can test for rather than merely suspect. Temporal holdouts use content published after the training cutoff, dynamic benchmarks generate fresh instances continuously, and contamination tests check whether the model recalls verbatim benchmark phrasing. These techniques keep the benchmark aligned with the deployment question instead of rewarding pretraining exposure to the evaluation set.

12.11.2 Data benchmarking

Model benchmarks validate whether compression preserved model quality. Model quality, however, depends entirely on the data used to train and evaluate it, and this dependency creates the most insidious failure mode in ML deployment. A perfectly preserved model trained on biased or unrepresentative data will still fail in production. Data benchmarks validate whether the efficiency strategies from chapter 9 (active learning, curriculum design, data augmentation, and synthetic data generation) produced training sets that enable reliable deployment. This is often the last validation to fail and the hardest to diagnose: a model achieving excellent accuracy on held-out test data may collapse on production inputs that the training data never adequately represented.

Within the D-A-M taxonomy, optimizing the Algorithm and the Machine cannot compensate for structural deficiencies in the Data: an accelerator cluster operating at peak arithmetic intensity simply computes incorrect representations faster when training distributions are defective. A data benchmark therefore begins with an operational protocol rather than an aggregate score. The protocol defines the deployment slice the model must serve, reserves a leakage-resistant holdout, verifies duplicate and near-duplicate separation across partitions, sets minimum coverage for rare classes and subgroups, audits label quality, and establishes drift thresholds that determine when the benchmark no longer represents production. Only after those gates are explicit do aggregate metrics become interpretable.

Coverage metrics

The first question data benchmarking must answer is whether the training distribution covers the operational domain the model will encounter in production. A model cannot generalize to input regimes absent from its training distribution, and the mechanisms through which training coverage fails are often subtle.

Class balance is the most apparent failure mode. In a fraud detection dataset with 99 percent legitimate transactions and 1 percent fraud, a degenerate model that predicts the majority class unconditionally achieves 99 percent accuracy while failing its operational purpose. Mitigating severe class imbalance requires loss reweighting, targeted oversampling, or decision threshold calibration. More insidious is subgroup imbalance within classes: a dataset may exhibit balanced positive and negative classes in the aggregate, yet draw negative examples disproportionately from a specific demographic cohort or geographic region. Aggregate accuracy metrics mask these systematic localized failures. When systems interact with human populations, auditing representation across demographic dimensions (such as age, geography, or dialect) is essential, though missing or noisy demographic metadata frequently complicates detection.

Feature coverage extends beyond discrete class labels into the continuous space of operational variations. While class balance is verified directly from label statistics, feature coverage requires mapping the physical parameters of the deployment environment. A vision pipeline trained exclusively on daytime captures fails under low-light sensor noise; a voice processing pipeline trained on clean studio signals degrades under acoustic reverberation and ambient noise. These environmental variations—differing sensor hardware, lighting conditions, and rare operational edge cases—fall outside discrete label distributions. Benchmarking feature coverage therefore requires specifying concrete bounds on the input space alongside domain experts before evaluating model performance.

Quality metrics

Even when a training dataset spans the operational domain, annotation errors corrupt the supervisory signal. Pervasive label errors—consistently measured between 3 and 6 percent across canonical

benchmark datasets including ImageNet (Northcutt et al. 2021)—do not act as benign noise. Supervised optimization treats incorrect labels as ground truth. When an evaluation set contains corrupted labels, a model that learns the true underlying distribution is penalized, while a model that memorizes the annotation artifacts receives an inflated score.

For small corpora, manual audits over stratified random samples provide baseline error estimates. At scale, confident learning identifies label errors algorithmically by estimating the joint distribution of noisy annotations and true latent labels using out-of-sample predicted probabilities. When a model consistently assigns high confidence to a class that conflicts with the assigned label, the discrepancy isolates candidate annotation errors from typical optimization uncertainty. Flagging these candidates enables targeted relabeling or filtering, circumventing the prohibitive cost of exhaustive human review.

Inter-annotator agreement measures consistency across annotators prior to model training. Cohen’s kappa for two raters and Fleiss’ kappa for multiple raters quantify agreement above chance expectation (Cohen 1960; Fleiss 1971). When agreement metrics fall below established thresholds on tasks with objective ground truth, the failure points to ambiguous labeling taxonomies, rater fatigue, or inconsistent guidelines. While Landis and Koch’s qualitative kappa bands provide a historical reference (Landis and Koch 1977), production pipelines require agreement thresholds calibrated directly to application-specific risk tolerance.

The structural form of label noise determines its downstream impact. Random label noise dilutes gradient signals across classes, increasing sample complexity but rarely teaching coherent false representations. In contrast, systematic errors—such as consistently mislabeling an underrepresented subclass or coupling a label with an environmental artifact—inject structured bias into model weights. If images of wolves photographed against snow are consistently labeled as dogs, the model learns to associate snow with the target class. In the presence of systematic label error, collecting more training data under the same labeling protocol does not correct the bias; it merely reinforces the spurious association with higher parameter certainty.

Distribution alignment

Data benchmarking must ultimately evaluate whether a model generalizes from training conditions to deployment environments. Standard evaluation protocols assume that training, validation, and test splits are independent and identically distributed (i.i.d.). In production systems, this assumption fails routinely: temporal drift alters consumer behavior, sensor degradation shifts hardware characteristics, and geographic expansion introduces unmodeled subpopulations. When the i.i.d. assumption breaks down, held-out validation accuracy ceases to predict operational performance and consistently yields an overly optimistic assessment of model reliability.

This train-to-production alignment gap is quantified systematically by the WILDS⁴² benchmark (Koh et al. 2021), which evaluates models across distribution shifts spanning hospital systems, camera trap locations, and satellite passes. On Camelyon17-WILDS, the reported ERM baseline achieved 93.2 percent in-distribution average accuracy and 70.3 percent out-of-distribution average accuracy across unseen clinical sites. This performance drop occurs despite holding the architecture, hyperparameters, and training compute constant, confirming that deployment degradation stems directly from distribution shift rather than insufficient model capacity.

To prevent distribution shift from violating service-level agreements, serving pipelines deploy statistical shift detection on incoming inference streams. Two-sample hypothesis tests such as the Kolmogorov-Smirnov test (Berger and Zhou 2014) for univariate features and maximum mean discrepancy (Gretton et al. 2012) for multivariate embeddings detect covariate shift—changes in the marginal input distribution $P(X)$ even when the conditional target distribution $P(Y | X)$ remains unobserved. Tracking prediction entropy and confidence distributions further identifies when inputs fall outside the training support, allowing the system to route anomalous queries to fallback models or human review before serving quality degrades.

Distribution alignment challenges highlight a persistent tension in ML development between two paradigms: *fixing the data and iterating on models*, or *fixing the model and iterating on data*. Figure 12.12 places these two paradigms side by side, revealing exactly where the feedback loop differs. In the model-centric diagram, the iteration cycle targets the architecture while the data remains static; in the data-centric diagram, the architecture stays fixed while the cycle targets data quality. The two

⁴² | **WILDS:** Stanford’s 2021 benchmark of ten datasets with real-world distribution shifts: hospital changes (Camelyon17), wildlife camera location shifts (iWildCam), and satellite imagery temporal drift (PovertyMap). For the reported Camelyon17-WILDS ERM baseline, average accuracy fell from 93.2 percent in-distribution to 70.3 percent out-of-distribution, demonstrating that standard held-out evaluation can overestimate performance when the i.i.d. assumption fails.

approaches are complementary rather than a universal ranking of where improvement will come from.

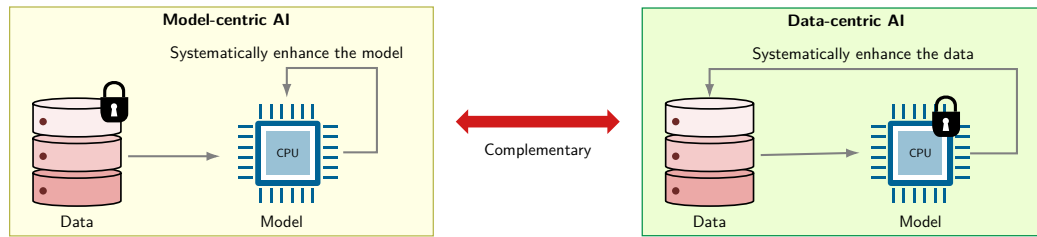


Figure 12.12: Development Paradigms: Model-centric AI iterates on architecture while holding the dataset fixed, whereas data-centric AI iterates on annotations, diversity, and bias while holding the architecture fixed. The double arrow marks the approaches as complementary.

Data-centric AI reflects an important shift in understanding that challenges the “more data is always better” assumption: dataset composition matters alongside scale. Initiatives like DataPerf (Mazumder et al. 2023) and DataComp⁴³ have emerged to evaluate how dataset construction affects model performance systematically. In DataComp’s compute-controlled setting, a baseline that retained the top 30 percent of the candidate pool by CLIP-based filtering outperformed the unfiltered-pool baseline on aggregate downstream evaluation (Gadre et al. 2023). The result establishes the value of curation for that protocol, not a universal optimal fraction.

Dataset saturation and dynamic benchmarks

Even when coverage, quality, and distribution alignment protocols are satisfied, static benchmarks encounter the problem of dataset saturation. As models approach empirical upper bounds on fixed test suites, leaderboard deltas compress, making it difficult to distinguish genuine capability advances from over-optimization to benchmark-specific artifacts. As the timeline in figure 12.13 illustrates, widely tracked AI benchmarks have repeatedly crossed reported human baselines, exhausting their headroom as discriminators (Maslej et al. 2024).

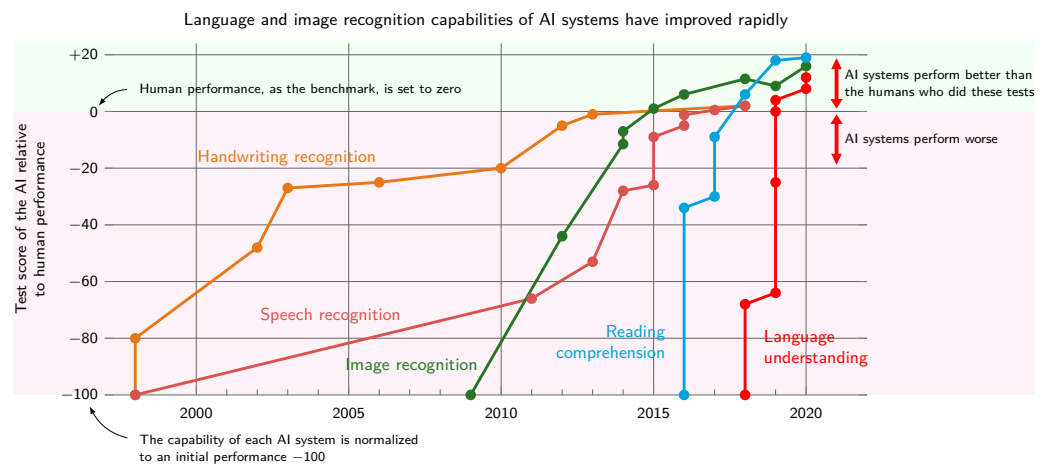


Figure 12.13: Dataset Saturation: The zero line denotes reported human performance; each curve’s crossing marks a capability for which the static benchmark has less remaining discriminating headroom. The crossings do not distinguish genuine capability gains from optimization to the evaluation set. Sources: (Maslej et al. 2024; Kiela et al. 2021).

Once a static benchmark saturates, high leaderboard accuracy can mask brittle heuristics. MNIST digit recognition (LeCun et al. 1998) demonstrated early that models can exploit dataset-specific capture artifacts rather than invariant features, a concern that eventually prompted the question “Are we done with ImageNet?” (Beyer et al. 2020). When the evaluation distribution remains

⁴³ | **DataComp:** Introduced in 2023, DataComp fixes the model family, training code, and compute budget while participants vary dataset construction. Its filtering track isolates the effect of selecting examples from a common candidate pool, making curation strategies comparable without attributing every gain to additional training compute.

frozen, iterative model selection risks overfitting to idiosyncratic test artifacts rather than improving generalizable task capability.

Dynamic benchmarking frameworks such as Dynabench⁴⁴ (Kiela et al. 2021) address saturation by continuously evolving evaluation data. In this paradigm, human annotators generate adversarial inputs targeting the failure modes of current top-performing models, ensuring that the benchmark continues to exercise representational limits. However, dynamic benchmarks introduce a fundamental systems trade-off: because the evaluation distribution changes across iterations, longitudinal comparisons across model generations become difficult to calibrate. Static and dynamic benchmarks therefore serve complementary roles: static suites ensure longitudinal reproducibility on standardized tasks, while dynamic benchmarks identify newly emergent error boundaries.

12.11.3 Holistic system-model-data evaluation

Passing system, model, and data benchmarks independently is not enough. A system benchmark can validate hardware performance, a model benchmark can verify that compression preserved quality, and a data benchmark can assess training set representativeness, yet the deployed system can still fail because the three dimensions interact. Real-world AI performance emerges from that interaction, and optimizing one dimension can expose weaknesses in another.

Consider a concrete failure cascade: an engineering team achieves target throughput on MLPerf Inference by deploying an INT8-quantized vision model on a specialized accelerator. System benchmarks pass. However, because quantization scale factors and zero points were calibrated strictly against ImageNet validation samples, deploying the model onto a factory floor reveals severe accuracy degradation. Factory illumination introduces dynamic ranges that exceed the calibration clipping thresholds, truncating feature activations. A targeted model quality benchmark across varied exposure levels would have exposed this numerical sensitivity. Tracing the failure upstream reveals that the training pipeline contained no industrial imagery—a data distribution gap that no amount of accelerator optimization or runtime kernel tuning can resolve.

Cross-dimensional coupling implies that a passing benchmark in one domain can be invalidated by an unmeasured failure in another:

- **System success + model failure:** Hardware delivers promised throughput, but aggressive compression degrades accuracy below deployment thresholds.
- **System success + data failure:** Fast inference executes on benchmark inputs, but training data bias causes severe accuracy disparities across demographic subgroups.
- **Model success + system failure:** The model produces accurate predictions, but tail latency variance under concurrent load violates service-level agreements.
- **Model success + data failure:** Validation yields high accuracy on held-out test sets, but production distribution shift causes silent degradation.

These failure modes map directly onto the D·A·M taxonomy introduced in chapter 1 (figure 1.4), though the benchmark lenses do not map one-to-one onto the three physical axes. Data benchmarks evaluate the input distribution and quality supplied to learning; model benchmarks evaluate learned algorithmic representations and numerical robustness; and system benchmarks expose hardware execution, kernel efficiency, and memory-interconnect traffic. Holistic evaluation validates the intersections where these axes meet. Optimizations across the Part III pipeline (data selection → model compression → hardware acceleration) introduce cross-axis dependencies that isolated measurements conceal.

The D·A·M taxonomy provides a diagnostic framework for systematically identifying which axis limits performance. Section A.1 maps each axis to its binding physical constraint and the optimization pathway that relieves it, giving the first diagnostic step when a benchmark reveals underutilization. Table 12.20 formalizes this approach by crossing each D·A·M axis with the three fundamental bottleneck types; Appendix A gives the full diagnostic guide, including profiling utilities and efficiency screening indicators.

⁴⁴ | **Dynabench:** Facebook AI Research's 2021 platform for dynamic benchmark generation, where humans craft adversarial inputs that fool current best models. Dynabench addresses the saturation problem, where very high accuracy on static benchmarks may reflect test-set familiarity rather than robust capability, but introduces its own trade-off: dynamic benchmarks are harder to compare across time because the evaluation set changes. Static and dynamic benchmarks serve complementary diagnostic roles.

Table 12.20: D-A-M Bottleneck Diagnostic Matrix: Each cell describes a performance constraint symptom, and the row identifies which D-A-M axis to address. When performance stalls, this matrix turns the first diagnostic move into an axis-specific check of Data, Algorithm, or Machine.

Component	Compute-Bound	Memory-Bound	I/O-Bound
Data	Preprocessing too slow (augmentation, tokenization)	Dataset exceeds RAM (spills to disk)	Storage cannot feed GPU (disk throughput limit)
Algorithm	Model too large for hardware (FLOPs exceed capacity)	Activations exceed memory (batch size limited)	Gradient sync slower than compute (distributed training)
Machine	GPU utilization saturated (need faster accelerator)	Memory bandwidth saturated (need more HBM bandwidth)	Network/PCIe bandwidth saturated (need faster links)

Applying this matrix isolates the root cause when system telemetry diverges from theoretical hardware peaks. If an accelerator achieves only 30 percent GPU utilization during training, an engineer might prematurely blame the model architecture (Algorithm row) and attempt model surgery. Profiling often reveals instead that host-side data workers saturating CPU cores on image augmentation or tokenization cannot feed batches fast enough across PCIe (Data row, Compute-Bound column: “Preprocessing too slow”). The accelerator stalls not from algorithmic inefficiency, but from input pipeline starvation. Traversing the matrix prevents misdirected engineering interventions by pointing directly to the binding physical subsystem.

Validation under controlled laboratory conditions differs from validation in production. In the laboratory, data distributions stay fixed, request patterns remain uniform, and systems run in isolation. In production, all three assumptions can break simultaneously: data drifts, traffic spikes unpredictably, and concurrent workloads compete for shared hardware resources. The final dimension of benchmarking asks whether laboratory results hold under operational conditions.

12.12 Production Considerations

A system that passes all three benchmark categories can still fail in production. The three-dimensional framework validated hardware performance, model quality, and data representativeness under controlled conditions, but operational deployments violate those conditions continuously. Validating an ML system requires measuring its behavior under dynamic workloads, evolving data distributions, and hardware resource contention.

12.12.1 From laboratory to production

Laboratory benchmarks establish what a system is capable of under ideal conditions. Production validation determines whether that system is performing correctly right now, under real conditions.

This gap arises because production workloads violate the static assumptions baked into laboratory evaluation:

Silent degradation occurs because models produce plausible outputs without raising hardware exceptions or software error codes. Traditional software crashes, panics, or returns an HTTP 500 status on invalid state. An ML model, by contrast, completes its matrix multiplications and returns valid floating-point tensors even when predictions become degraded or uncalibrated. A recommendation service returning suboptimal rankings provides no hardware-level signal that its predictive accuracy has collapsed.

Dynamic workloads introduce queueing and memory bottlenecks that steady-state benchmarks conceal. A serving stack benchmarked at steady 1,000 QPS can collapse when flash traffic events spike to 10,000 QPS because closed-loop laboratory tests assume uniform request arrivals with minimal queue backlog. When production arrival rates exceed instantaneous service capacity, queueing delays escalate nonlinearly. Dynamic batching schedulers attempt to absorb the surge by accumulating larger request batches, but expanded batch sizes demand additional accelerator memory for activation tensors and key-value (KV) caches. If memory capacity is exhausted, the engine crashes with an out-of-memory error; even if memory holds, queue wait times drive tail latency ($p99$) past operational service-level agreements.

Data distribution shift compounds these operational challenges as production data drifts away from training and validation distributions. A vision model evaluated on curated benchmark images

degrades when live inference processes smartphone uploads with lens artifacts, compression distortion, and varying illumination, eroding accuracy while hardware throughput and latency metrics appear completely unaffected.

Production constraints also couple dimensions that laboratory benchmarks evaluate in isolation. In the laboratory, accuracy, latency, cost, and resource utilization are optimized along independent Pareto curves. In deployment, these constraints collide directly on the hardware: enforcing a strict tail latency SLA during traffic surges requires capping batch sizes or shedding load, which degrades hardware arithmetic intensity, lowers accelerator utilization, and increases serving cost per query.

12.12.2 Bridging benchmark to deployment

Before deployment, validate benchmarking conclusions against production-representative conditions. Table 12.21 names the benchmark assumption, the production reality that violates it, and the validation step that closes the gap; the checkpoint that follows turns those rows into release-readiness actions.

Table 12.21: Predeployment Benchmark Checklist: Each row pairs an assumption baked into laboratory benchmarks with the production reality that violates it and the validation step that closes the gap. A system that passes every laboratory benchmark can still fail in production unless each row of this table has been independently verified against representative operational conditions.

Benchmark Assumption	Production Reality	Validation Approach
Uniform request arrival	Bursty traffic patterns	Load test with production trace replay
Clean, preprocessed inputs	Variable quality inputs	Evaluate on production data sample
Warm system state	Cold starts, cache misses	Measure cold-start performance
Isolated execution	Resource contention	Benchmark under realistic system load
Fixed model version	A/B testing, gradual rollout	Establish baseline for comparison



Checkpoint 12.4: Predeployment benchmark checklist

Before deploying a model based on benchmark results:

- ☐ Replay production traces: Use logged request patterns to validate throughput/latency under realistic conditions.
- ☐ Test with production data: Sample recent production inputs (respecting privacy) to verify accuracy holds.
- ☐ Stress test edge cases: Identify worst-case inputs and verify graceful degradation.
- ☐ Establish monitoring baselines: Document expected metric ranges for anomaly detection.
- ☐ Define rollback criteria: Specify quantitative thresholds for reverting to the previous model version.

12.12.3 Production monitoring as continuous benchmarking

Production monitoring extends benchmarking from a one-time gate to a continuous process. The same principles apply (standardized metrics, reproducible measurement, statistical rigor), shifting the objective from offline qualification to continuous operational verification.

Once a model is live, benchmarking becomes a rolling comparison against the baselines just established. The immediate checks stay concrete: whether the input distribution remains close to the benchmark distribution, whether latency and throughput stay inside the measured envelope, and whether model quality moves outside the expected range. Answering those checks requires the same measurement discipline as the offline benchmark, but now the measurements arrive continuously and under live traffic.

Operational serving environments turn this measurement loop into release and recovery machinery (chapter 14): staged rollouts, shadow evaluation (running the new model beside production without serving its outputs), continuous validation, and automated rollback. Benchmarking defines

the baseline performance envelope and failure thresholds; production monitoring continuously tests live telemetry against those bounds.

The gap between benchmark conditions and production reality explains why teams continue to fall into predictable traps. A set of recurring fallacies illustrates how sound metrics, misapplied, produce misleading conclusions.

12.13 Fallacies and Pitfalls

Benchmarking creates false confidence when standardized measurement obscures deployment realities. Teams assume controlled evaluations predict production performance, but real systems face variability, resource constraints, and multi-objective trade-offs that benchmarks cannot capture, wasting engineering effort on systems optimized for evaluation rather than deployment.

Fallacy: *Benchmark performance directly translates to real-world application performance.*

The apparent clarity of benchmark rankings leads teams to select systems as though leaderboard position predicts production behavior. It rarely does. As section 12.3.1 demonstrates, ML systems exhibit inherent variability from data quality issues, distribution shifts, and resource constraints absent in controlled evaluation. In a representative failure scenario, a language model achieving 92 percent benchmark accuracy drops to 78–82 percent accuracy in production when processing user-generated text with spelling errors, informal language, and domain-specific terminology. An inference system with 15 ms mean latency on MLPerf experiences 150–200 ms p99 latency in production (10–13.3× degradation) due to concurrent load, garbage collection pauses, and network variability. Teams relying solely on benchmark rankings systematically underestimate deployment complexity, leading to failed launches and costly re-engineering.

Pitfall: *Optimizing exclusively for benchmark metrics without considering broader system requirements.*

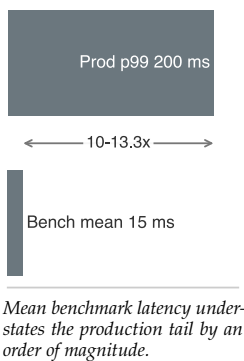
Benchmark leaderboards incentivize aggressive optimization, but the optimizations that climb rankings often degrade the very characteristics production demands. This exemplifies Goodhart’s Law (section 12.10.4): when benchmark scores become optimization targets, they cease to be meaningful measures of system quality. In one illustrative scenario, a team reduces inference latency from 12 ms to 8 ms through aggressive quantization, improving MLPerf ranking by 15 positions while degrading calibration such that prediction confidence scores become unreliable for downstream decision-making. Another team improves ImageNet accuracy by 2.1 percent through extensive hyperparameter tuning but the optimized model consumes 40 percent more energy and exhibits 25 percent worse performance on out-of-distribution images from production cameras. Organizations rewarding benchmark rankings over deployment success systematically produce systems that excel in evaluation but fail in production.

Fallacy: *Single-metric evaluation provides sufficient insight into system performance.*

A single-number claim appears attractively simple (“94 percent accurate” or “1,200 QPS fast”). But production success requires balancing multiple competing objectives that any single metric obscures. Modern inference systems demand evaluation across accuracy, latency, throughput, energy, and robustness dimensions (section 12.8.2). In an illustrative trade-off, a recommendation model achieving 94 percent accuracy with 180 ms p99 latency fails service-level objectives requiring p99 < 100 ms despite excellent accuracy. Conversely, a system optimized for 1,200 QPS throughput achieves this rate while consuming 4.2 W vs. 1.8 W for a slightly slower system at 1,000 QPS (2.3× power difference). For battery-powered edge devices, the 17 percent throughput loss enables 2.3× longer operation time. Different stakeholders prioritize different metrics: ML engineers focus on accuracy, infrastructure teams on throughput and cost, product managers on latency percentiles. Single-metric optimization systematically produces systems that excel on one dimension while failing deployment requirements on others.

Pitfall: *Using outdated benchmarks that no longer reflect deployment challenges and requirements.*

Benchmarks have inertia: teams continue reporting on established benchmarks after the results cease to provide useful discrimination. Saturation occurs when multiple approaches achieve near-identical performance, eliminating useful comparison. ImageNet top-5 classification error decreased from 28.2 percent in 2010 (Russakovsky et al. 2015) to 3.57 percent by 2015 (He et al. 2016a), sharply



compressing the headroom measured by that metric. As gaps narrow, the statistical confidence intervals discussed in section 12.10.1 and the result's deployment relevance matter more than leaderboard rank. Changing deployment contexts compound the problem: benchmarks designed for server hardware become misleading for edge devices with 10× less memory and 100× lower power budgets. Effective benchmarking requires retiring saturated benchmarks and developing evaluation frameworks matching target deployment realities.

Fallacy: *Research benchmarks predict production behavior under real traffic.*

Research benchmarks exist to compare algorithms under controlled conditions; production systems exist to serve users under variable ones. Treating the former as a production prediction often produces optimistic results because research benchmarks may omit resource contention, input-quality variation, and operational failure modes. Improper execution adds a separate source of error: omitting **warmup runs** can mix one-time initialization, just-in-time compilation, cache population, and memory allocation into steady-state latency; leaving cache state unspecified can make a nominal memory benchmark measure either warm-cache reuse or cold-memory access; and allowing dynamic voltage and frequency scaling (DVFS) to vary without reporting it makes small differences difficult to reproduce. Production systems face concurrent user loads, varying input quality, network latency, and system failures that degrade performance (section 12.10.2). A system achieving 800 QPS throughput in isolated benchmarks sustains only 400–500 QPS under production load with 90 percent utilization (37.5–50 percent degradation) due to queue contention and garbage collection pauses. Research benchmarks report model inference time (5–10 ms) while production end-to-end latency includes preprocessing, queuing, and postprocessing overhead totaling 50–100 ms. Production systems require 99.9 percent availability (43 minutes downtime per month) and graceful degradation under failures, characteristics research benchmarks often omit. Effective production evaluation requires operational metrics: sustained throughput under load, recovery time from failures, and complete latency breakdown.

Pitfall: *Using research benchmarks as production release gates.*

Teams sometimes promote a model because it passes the research benchmark, then discover only after launch that the benchmark never exercised the operational path. A release gate for a serving system must include load tests, tail-latency measurements, data-quality checks, failure drills, and rollback criteria. Research benchmarks remain useful for comparing algorithms, but production gates must measure the deployed system under the traffic, hardware, and failure conditions it will actually face.

12.14 Summary

Benchmarking completes Part III's optimization pipeline by validating whether the efficiency gains from data selection (chapter 9), model compression (chapter 10), and hardware acceleration (chapter 11) deliver in practice. The three benchmark lenses examine system execution, model behavior, and data representativeness separately, then test whether their assumptions survive when the complete system runs under deployment conditions.

The lenses reveal different failure modes. System benchmarks expose underdelivered throughput, tail latency, and thermal behavior. Model benchmarks test accuracy, calibration, robustness, and other properties that an optimization may alter. Data benchmarks examine coverage, label quality, leakage, and alignment with deployment. Standardized suites such as MLPerf Training and Inference provide comparable system evidence; model and data protocols supply the quality and representativeness evidence that a hardware result cannot establish.

Grounding optimization claims requires measuring physical execution rather than relying on analytical proxies. Wall-clock timing captures DRAM bandwidth saturation, kernel launch serialization, and host-device transfer overhead that FLOP counts omit. Percentile profiling under concurrent load exposes queuing delays and thermal throttling that arithmetic means conceal. Evaluating workloads against production-representative data reveals dynamic shape variations and memory churn that uniform synthetic inputs never trigger.



Key Takeaways: Measuring what matters

- **Benchmarks validate co-design:** System, model, and data benchmarks expose hardware underdelivery, compression quality loss, and distribution mismatch. A system that passes only one axis can still fail when Data, Algorithm, and Machine constraints meet under production load.
- **Proxy numbers need boundaries:** Standardized run rules make comparisons honest, but fixed workloads are still proxies. Batch size, thermal state, input distribution, concurrency, and service-deadline windows decide whether a lab result survives the benchmark-production gap.
- **Granularity trades diagnosis for realism:** Micro-benchmarks isolate kernels, macro-benchmarks expose model-level costs, and end-to-end benchmarks capture user-visible behavior. Effective measurement stacks all three so teams can see both the symptom and the layer that caused it.
- **Tail latency is the benchmark:** Interactive systems fail at p95 and p99 before averages move. Reporting percentile latency under representative load prevents a benchmark from approving a system whose mean passes while its worst-served requests violate the SLO.
- **Amdahl caps every optimization claim:** A faster model cannot outrun the rest of the pipeline; if preprocessing is 50 percent of latency, an infinitely fast model yields only a $2\times$ system improvement. Benchmark the whole request path before celebrating kernel speedups.
- **Efficiency still needs quality evidence:** INT8 may cut raw weight storage $4\times$; the simplified component model estimates an energy reduction of about $6.6\times$, but whole-device measurement, calibration, subgroup robustness, and edge-case behavior decide whether the compressed model is deployable.

Each chapter in this part promised fewer FLOPs, a smaller model, or higher throughput. Benchmarking tests those promises against the complete system. The gap between a claimed improvement and a measured one reveals where Data, Algorithm, and Machine were assembled rather than matched. Amdahl's Law shows why an infinitely fast model can still leave the pipeline bounded by everything outside it, while tail latency shows why an average can pass even as the worst-served requests fail. This is co-design held to account. An ML system is engineered, not asserted, and only measurement on the real workload can tell the two apart.



What's Next: From lab to live

A system that passes every benchmark in a controlled environment can still fail under live deployment. Static benchmarks cannot fully capture traffic bursts, data drift, or cascading failures under live workloads. Part IV leaves the controlled benchmark environment for production, beginning with chapter 13.

IV

DEPLOYMENT AND OPERATIONS

Deployment Principles

The code can be correct and the benchmarks excellent, and yet the system may fail. Part IV moves from controlled environments to production, where ML systems face the additional threat of silent decay. A machine learning system can continue to produce outputs that are confident, well-formatted, and wrong as the world drifts away from its training distribution. At deployment, the data environment escapes the engineer's control and can stress the trained algorithm and the serving machine in ways no test set anticipated. Reliability is therefore a continuous control loop of D·A·M co-design rather than a one-time release gate. The principles here define the requirements and diagnostic models for that reliability.

III Principle 9: The verification gap

Invariant: Statistical verification holds only over a defined deployment population P_{deploy} , task distance d , tolerance τ , and target error ϵ , estimated by a finite-sample procedure at a stated confidence level:

$$\Pr_{(x,y) \sim P_{\text{deploy}}} [d(f(x), y) \leq \tau] \geq 1 - \epsilon$$

Implication: Deployment is not a one-way transfer; it is a control loop. Because no test suite can cover every possible real-world input, production systems must monitor their own uncertainty and fail gracefully when they drift outside their known performance envelope.

The verification gap means finite evaluation can only estimate behavior on a defined population with uncertainty. Those bounds may erode as production data diverges from the data used to set them.

III Principle 10: The statistical drift diagnostic

Invariant: Divergence between the production and baseline distributions does not by itself determine whether accuracy changes; over a measured deployment range, labeled outcomes can fit a local linear relationship:

$$\text{Accuracy}(t) \approx \text{Accuracy}_0 - \lambda \cdot \mathcal{D}(P_t \| P_0)$$

where P_0 is the baseline distribution, Accuracy_0 is the model's performance on P_0 , $\mathcal{D}(P_t \| P_0)$ is the statistical divergence between the current and baseline distributions, and λ is fitted from labeled outcomes rather than inferred from divergence alone. Consider a credit scoring model trained on 2020 borrower behavior. Two years later, inflation rises, interest rates change, and lending policies shift. The system still produces scores, but the statistical relationship between inputs and outcomes may have changed; labeled outcome monitoring determines whether real accuracy declined while conventional error logs remained quiet. Unlike many traditional software failures, which are often surfaced by crashes, exceptions, or explicit service-health signals, ML systems can fail silently because the *environment* changes even when the code and infrastructure remain unchanged. This first-order linearization applies only over the measured range; the relationship is model-dependent and may be nonlinear for large drift.

Implication: Observability must extend beyond system metrics (latency, errors) to combine statistical drift signals with labeled outcomes. A system can remain operational while prediction quality changes, but drift monitoring alone does not establish degradation.

External drift is not the only threat. Even when the world holds still, the serving pipeline itself can diverge from the model validated offline.



Principle 11: The training-serving skew diagnostic

Invariant: Training and serving code paths can diverge on identical inputs, and that divergence is measurable without labels. For scalar model scores, define training-serving divergence as the expected absolute difference between the two functions over a stated comparison-input distribution:

$$S_{\text{skew}} = \mathbb{E}[|f_{\text{serve}}(x) - f_{\text{train}}(x)|]$$

The exact relationship between S_{skew} and accuracy depends on the loss function, decision boundary geometry, and production distribution. Unexplained divergence weakens the assumption that offline validation estimates production behavior and can cause silent accuracy loss, but S_{skew} is not a universal accuracy-loss equation. This divergence arises from inconsistent preprocessing logic, different library implementations, stale feature values, or environmental state changes between the two code paths.

Implication: Feature consistency is an architectural requirement, not merely a best practice. Feature stores are more than caches; they can reduce skew by centralizing feature definitions and retrieval. Teams still need validation for freshness, point-in-time correctness, preprocessing, model-runtime, and postprocessing parity. Even subtle differences (PIL vs. OpenCV resize, FP64 vs. FP32 normalization) can compound to produce silent accuracy degradation that standard monitoring may not detect.

Beneath all these reliability concerns lies a nonnegotiable constraint: time. A medical imaging system that detects tumors with 99 percent accuracy but takes 30 seconds per scan may miss its workflow deadline. An autonomous vehicle perception model that classifies obstacles correctly but responds in 200 ms instead of 50 ms may miss its braking deadline. A correct result may have no operational value if it arrives too late. Latency-sensitive deployed models operate under a deadline, and exceeding it can be functionally equivalent to returning no prediction at all.



Principle 12: The latency budget principle

Invariant: Under serial request-path accounting, end-to-end latency is the sum of network, preprocessing, inference, postprocessing, and queueing delays, and a deployment meets its deadline only at a stated completion quantile:

$$L_{\text{lat,total}} = L_{\text{lat,net}} + L_{\text{lat,pre}} + L_{\text{lat,infer}} + L_{\text{lat,post}} + L_{\text{lat,queue}}, \quad \Pr[L_{\text{lat,total}} \leq D] \geq q$$

Implication: In latency-sensitive serving, the product selects the deadline and the acceptable completion quantile, such as 0.95, 0.99, or 0.999, and throughput is optimized within that constraint. Serving systems must implement tail-latency controls (for example, bounded-delay dynamic batching or hedged requests). Serving systems must be willing to sacrifice overall throughput to meet the latency deadline of the oldest request in the queue.

A system can satisfy its latency SLOs, detect measured distributional shifts, and maintain training-serving consistency while still causing systematic harm. The previous principles address silent failures in correctness and service quality; this one addresses failures that degrade *equity* through silent amplification.



Principle 13: The bias feedback model

Invariant: When outputs influence future inputs, today's disparity becomes an input to the next training cycle, so disparity evolves as a dynamical system rather than staying fixed. Let $\Delta_g(k) \geq 0$ denote the disparity magnitude for group g relative to a specified reference group after k fixed feedback intervals. With no effective intervention and an approximately constant dimensionless feedback factor $\alpha_{fb} > 0$, disparity may evolve as:

$$\Delta_g(k) \approx \Delta_g(0) \cdot \alpha_{fb}^k$$

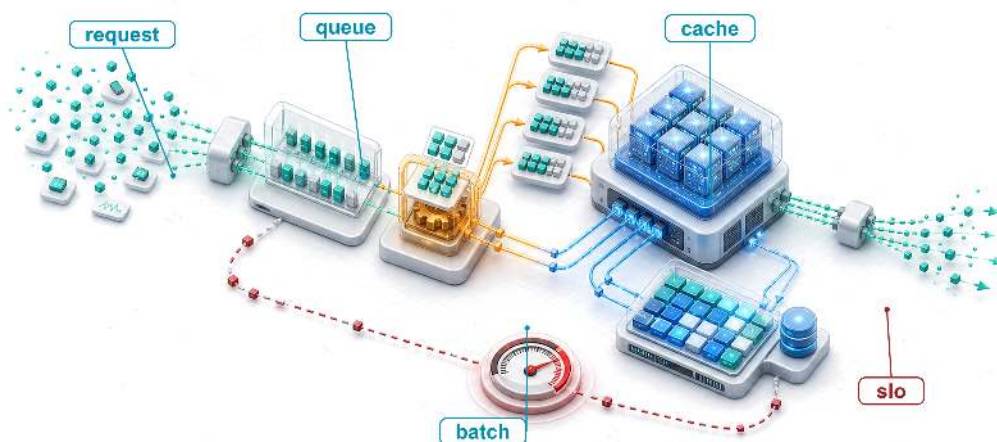
where $\Delta_g(0)$ is the initial disparity magnitude relative to that reference group and α_{fb} is fitted per feedback interval. Consider a loan approval model that denies credit at higher rates to applicants from historically underserved communities. Denied applicants cannot build credit history, which makes future applications weaker, which increases future denial rates. The model's accuracy on its training distribution remains stable, but the population it serves has been reshaped by its own decisions. When $\alpha_{fb} > 1$ under these assumptions, the feedback loop is self-reinforcing; when $\alpha_{fb} \leq 1$, the modeled dynamics are stable or damped. Real deployments may also be nonlinear, saturating, or changed by intervention.

Implication: Fairness is not a postdeployment audit; it is a **stability constraint** on the deployment control loop. Where consequential impact warrants it and collecting or using group attributes is lawful and justified, systems must monitor **disaggregated performance metrics** across relevant demographic groups with the same rigor applied to latency percentiles, because a bias regression can be invisible to aggregate accuracy just as a tail-latency violation can be invisible to mean latency.

Together, the five principles define a deployment control loop that establishes what the evidence supports, observes the live system and environment, acts only when a measured failure warrants intervention, and re-evaluates the relevant service, predictive, or subgroup criterion. They separate diagnosis from response. A signal narrows the investigation but neither prescribes an automatic intervention nor proves that quality has already degraded.

Part IV makes that control loop concrete. Model Serving builds infrastructure that meets latency budgets under variable traffic. ML Operations detects drift and skew while preserving the evidence needed for diagnosis and response. Responsible Engineering treats fairness and feedback as measurable deployment dynamics rather than a one-time compliance check. The volume closes by connecting these production controls to the quantitative bounds and models developed in the preceding parts.

Model Serving

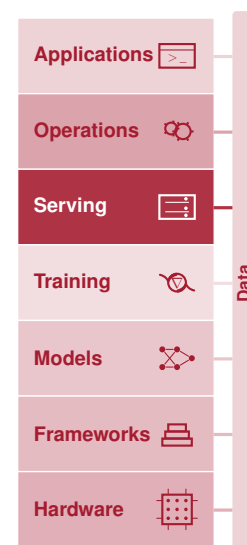


- 13.1 Serving Paradigm
- 13.2 Serving Load, Latency, and Architecture
- 13.3 Serving System Architecture
- 13.4 Request Lifecycle
- 13.5 Queuing Theory
- 13.6 Model Lifecycle Management
- 13.7 Throughput Optimization
- 13.8 LLM Serving
- 13.9 Inference Runtime Selection
- 13.10 Node-Level Optimization
- 13.11 Economics and Planning
- 13.12 Fallacies and Pitfalls
- 13.13 Summary

Purpose

Why does serving change the optimization priorities that made training successful?

Training and serving emphasize different operating objectives. Training usually maximizes throughput across large batches and long runs, while serving must answer individual requests within latency targets. Training amortizes hardware costs across many examples; serving pays a tax on each request, where small inefficiencies compound into operational debt. This shift is why models that train efficiently may still serve poorly. The live system has an end-to-end latency budget, so queuing, preprocessing, network transit, and postprocessing can erase an optimization confined to the model. Batch-heavy architectures and memory-intensive optimizations designed to saturate accelerators may not suit bursty, latency-critical, cost-sensitive production traffic. Serving, however, is more than a latency problem. A serving system must handle traffic that varies between peak and trough, reserve headroom for failures, shed load or degrade gracefully under overload, and remain predictable as request mix and demand change over time. Models that proved their value during training and survived compression and benchmarking eventually arrive at the serving layer, the deployment and integration stage of the ML lifecycle. The question shifts from whether a model works to whether it works reliably at scale under production conditions. Serving infrastructure is where ML systems meet users, and sustaining that interaction requires different engineering from creating the model. It is also where the trained algorithm meets live data within the machine's latency budget, bringing all three D·A·M constraints together on each request.





Learning Objectives

- Explain serving inversion from training throughput to per-request latency, headroom, and tail behavior
- Decompose request latency across serialization, preprocessing, inference, queuing, post-processing, and network overhead
- Apply queueing laws and simple queue models to plan capacity against percentile latency targets
- Diagnose training-serving skew and cold starts from mismatched preprocessing, model loading, or cache behavior
- Select batching, load shedding, autoscaling, and runtime strategies for traffic patterns and latency budgets
- Evaluate LLM serving bottlenecks using token latency, KV-cache memory, and continuous batching constraints
- Calculate cost per inference from precision, hardware utilization, replica count, and runtime throughput

13.1 Serving Paradigm

MODEL **serving** begins where benchmarking stops. A model evaluated under synthetic test harness conditions must now generate predictions under live production traffic. In offline training, systems maximize aggregate throughput by packing large batches to amortize weight transfers from high-bandwidth memory (HBM) and saturate matrix arithmetic units. In online serving, requests arrive stochastically from independent clients and must return within strict service-level objectives (SLOs). The primary engineering priority therefore inverts from long-run throughput to per-request latency. This **serving inversion** restructures the balance of terms in the iron law of ML systems ($T = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}}$). When an application cannot delay requests to assemble large batches, it must process small batches or individual inputs ($B = 1$). Small batch execution collapses arithmetic intensity, converting compute-bound matrix multiplications into memory-bandwidth-bound transfers (D_{vol}/BW). Simultaneously, the latency term L_{lat} expands beyond microsecond accelerator kernel launches to include network transit, request serialization, and host orchestration, often becoming the binding constraint on end-to-end response time.

The D·A·M taxonomy clarifies how this inversion reshapes each layer of the system. In the data layer, the constraint shifts from total dataset volume to the arrival rate, payload freshness, and variable sequence dimensions of live queries. In the algorithm layer, execution becomes strictly forward-only: backpropagation state and optimizer memory vanish from the accelerator, but dynamic input shapes introduce runtime execution variability into previously static operator graphs. In the machine layer, the operating target shifts from maximum hardware utilization to **headroom**. While offline training treats an idle accelerator as wasted capital, online serving treats near-saturation as an operational hazard. Because request arrivals follow stochastic distributions, queue lengths and waiting times diverge nonlinearly as utilization approaches 100 percent. Maintaining adequate headroom preserves the spare capacity needed to absorb transient traffic spikes without violating tail-latency percentiles. Serving therefore optimizes completed requests under a strict latency deadline rather than steady-state hardware occupancy.

Meeting that latency deadline governs how engineering teams partition the end-to-end serving stack. A request's latency budget encompasses three sequential stages: host-side ingress and preprocessing (network termination, payload deserialization, and tokenization or feature transformation), accelerator kernel execution, and downstream postprocessing. Any millisecond consumed by unoptimized feature retrieval or queueing delays directly subtracts from the execution time allocated to the neural network itself. Consequently, the central engineering task in model serving is deciding which operations must remain on the synchronous request path, which tasks can be offloaded to asynchronous background pipelines, and how much headroom the system must reserve before tail-latency guarantees collapse under load.

13.2 Serving Load, Latency, and Architecture



Example 13.1: The 'Black Friday' traffic spike

Scenario: An e-commerce recommendation serving system experiences a 10× traffic surge (10,000 QPS) during Black Friday sales events.

Diagnosis: When server utilization approaches 100 percent, queue lengths explode nonlinearly, increasing latency to 10 s and causing client request timeouts.

Systems lesson: High average throughput does not prevent queueing collapse under traffic surges. Depending on the service, preserving a tail-latency objective may require load shedding, graceful degradation, or autoscaling before utilization reaches the workload's queueing knee.

Online serving systems process variable, unbatched request streams while operating under strict latency deadlines. When arrival bursts exhaust a system's reserved headroom, performance degrades nonlinearly.

Figure 13.1 shows that latency remains manageable at moderate utilization and then rises rapidly as the system approaches saturation; this is why latency-sensitive systems reserve headroom rather than planning for a permanently saturated accelerator. Section B.2.1 gives a mathematical treatment of long-tailed distributions and explains why high-percentile latency matters when an SLO covers the slowest requests. The curve is a simple queueing approximation intended for intuition rather than a specific workload.

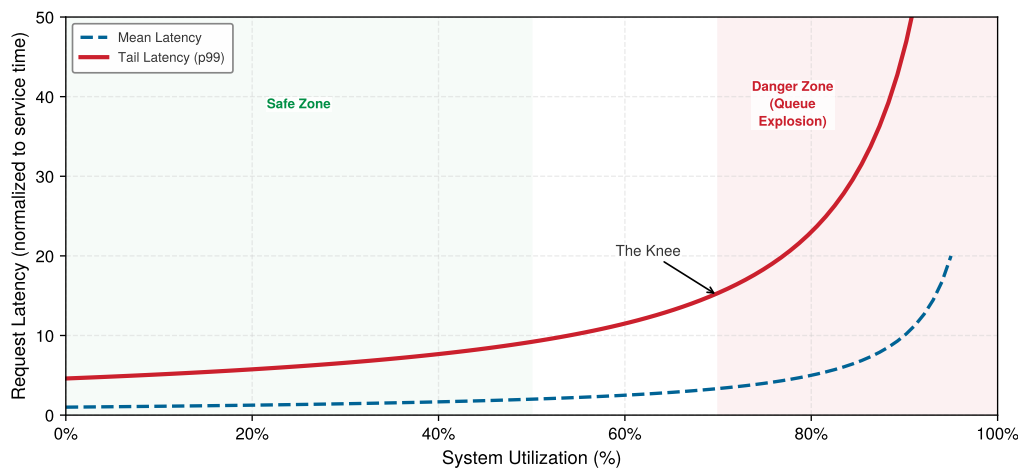


Figure 13.1: The Tail Latency Explosion: Request latency vs. serving utilization ρ_{serv} . Mean latency (blue) and the p99 approximation (red) both grow nonlinearly as utilization approaches saturation; the 70 percent marker is an illustrative planning knee rather than a model-derived breakpoint. This uses the simple M/M/1 approximation introduced later in the Queueing Theory section (p99 $\approx 4.6 \times$ mean), so the curve is illustrative rather than workload-specific.

Beyond the technical limits of latency, serving economics have also changed rapidly. As models become more efficient and hardware becomes more specialized, the cost per inference has fallen.¹ Facebook's experience at fleet scale illustrates the magnitude of this serving cost problem.



Example 13.2: The inference tax at Facebook

Scenario: Production ML serving systems at Meta execute tens of trillions of inferences daily across global user-facing services (Hazelwood et al. 2018).

¹ | **Jevons paradox:** William Stanley Jevons observed in 1865 that efficiency improvements in coal-powered steam engines increased total coal consumption by making steam power economically viable for applications previously too costly (Jevons 1865); the same dynamic can apply to AI inference: each 10× cost reduction opens application classes that were economically infeasible at the previous price point, expanding aggregate demand by more than the efficiency gain. This is why cheaper inference can increase, not decrease, total GPU fleet demand. Efficiency and demand are often complements in AI, not substitutes.

Diagnosis: Recurring inference serving compute and energy costs dwarf initial offline model training expenses. Live request arrival patterns and strict latency bounds constrain GPU batching.

Systems lesson: Model architectures cheap to train can prove unviable to serve at fleet scale. Production serving infrastructure requires hardware-software co-design to balance tail latency, batch size, and recurring operational total cost of ownership.

The same serving-economics pressure appears in public API prices. The log-scale price trajectory in figure 13.2 captures the speed of this cost collapse by tracking representative public API list-price snapshots as a market proxy. Vendor prices change frequently, so these points should be read as historical provenance for the trend rather than as current purchasing guidance (OpenAI 2023b, 2023a, 2024; OpenAI Developer Community 2024; Anthropic 2024; Google Developers Blog 2024; DeepSeek 2024). Each order-of-magnitude drop changes which applications are feasible.

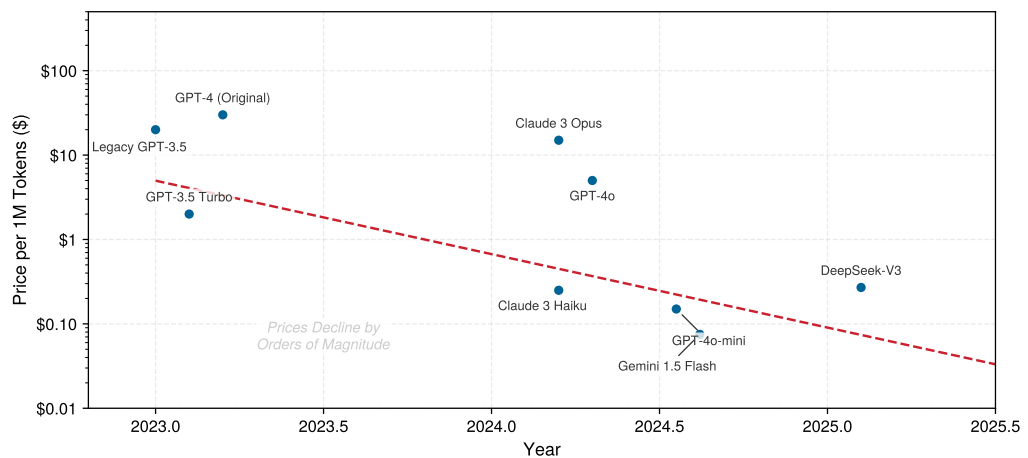


Figure 13.2: Intelligence Deflation: Representative public API input-token list prices per 1M tokens (\$) over time (Log Scale). Eight model-version points are snapshots collected from the public pricing pages of OpenAI, Anthropic, Google, and DeepSeek between 2023 and 2025; the Legacy GPT-3.5 point is an illustrative baseline rather than a documented model-version snapshot. The series is intended as a market trend indicator, not a controlled or current-price comparison. API token-processing prices have collapsed by multiple orders of magnitude, transforming the economics of automated AI workflows.

Two pressures now frame the serving problem: tail latency that explodes once utilization passes the queueing knee, and per-inference economics that fall by orders of magnitude as efficiency improves. Together they force a formal definition of serving built around latency rather than throughput.



Definition 13.1: Model serving

Model serving is the operational phase that provides model predictions to end-users or downstream systems under latency, throughput, availability, and cost constraints.

1. **Significance:** Online serving turns training's throughput priority (η_{hw}) into a request-latency constraint (L_{lat}), requiring a stack designed around the percentile named by its SLO.
2. **Distinction:** Unlike model training, which commonly processes scheduled batches, serving may handle stochastic online requests, synchronized streams, or offline batches.
3. **Common pitfall:** A frequent misconception is that serving is "just the forward pass." In reality, it is a systems problem: model execution is only one component of a stack that may include request routing, load balancing, and data transformation.

The SLO² defines the latency target that shapes every architectural decision in the serving stack, establishing the strict time budget within which all request handling must complete. When an inference server receives a request, model execution represents only one segment of an end-to-end pipeline: network ingress, payload deserialization, feature transformation, queue scheduling, and output serialization all consume time before and after the accelerator runs. Figure 13.3 shows that raw inputs pass through preprocessing (traditional computing), neural network inference (deep learning), and postprocessing (traditional computing) before producing final outputs. Any of these stages can become the latency bottleneck. Section 13.4.1 quantifies exactly where time goes, revealing a counterintuitive result about which stages dominate.

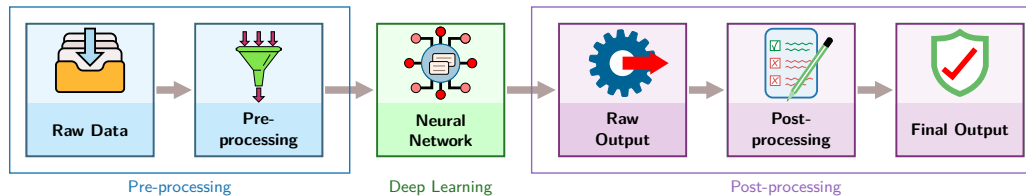


Figure 13.3: The Inference Pipeline: ML serving systems transform raw inputs into final outputs through sequential stages: preprocessing, neural network computation, and postprocessing. The neural network represents just one component; preprocessing and postprocessing make end-to-end latency depend on more than neural-network execution.

This pipeline turns serving into an orchestration problem: because every stage draws from the same latency budget, optimizing accelerator kernel execution yields diminishing returns if data transformation or queueing delays dominate. Before optimizing any individual stage, the system must decide whether predictions are computed ahead of time or on demand.

13.2.1 Static vs. dynamic inference

Deciding whether predictions are generated before or during a user request establishes the fundamental trade-off between offline hardware efficiency and online operational agility (Google 2024b). That choice determines whether an accelerator operates in an unconstrained throughput regime or under the strict tail-latency deadlines of an interactive serving loop.

13.2.1.1 Static inference

Static inference (also called offline or batch inference) precomputes predictions for anticipated inputs and stores them in a retrieval store. Moving model execution entirely out of the user request path alters the underlying hardware constraints: because batch jobs have no millisecond-level deadlines, accelerators can run at maximum batch sizes ($B \gg 1$). Large batches maximize arithmetic intensity by amortizing weight transfers from HBM across thousands of samples, keeping compute cores saturated. When a client issues a request—such as a user requesting nightly generated recommendation rankings—the serving system performs a low-latency key-value lookup rather than executing a forward pass on an accelerator. The primary operational constraint shifts from accelerator latency to storage capacity and data freshness: precomputing predictions across an entire input domain scales combinatorially ($O(|U| \times |I|)$ in recommendation systems), and stale predictions persist until the next batch refresh.

13.2.1.2 Dynamic inference

Dynamic inference (also called online or real-time inference) evaluates inputs on demand as requests arrive. On-demand execution accommodates arbitrary, previously unseen inputs within the model schema and immediately reflects newly deployed model weights without requiring precomputation. However, executing the model directly within the user request path subjects inference to a strict end-to-end latency budget (a service level objective, or SLO). Because user requests arrive stochastically, the serving runtime cannot delay execution indefinitely to accumulate large batches. Serving requests individually or in small batches ($B = 1$ to $B = 4$) drops arithmetic intensity and leaves accelerators memory-bandwidth bound: memory buses must transfer full layer weights from memory to registers for only a fraction of the hardware’s compute capacity, directly inflating the cost

² | **Service level objective (SLO) vs. service level agreement (SLA):** An SLO is an *internal* target (for example, “p99 latency under 50 ms”); an SLA is an *external* contractual commitment whose violation may carry remedies or penalties. Teams often set SLOs tighter than SLAs to preserve a safety margin. For ML serving, task quality and inference latency can both contribute to SLOs, creating multidimensional targets where improving one dimension (for example, deploying a larger model for accuracy) can violate another (latency).

per query. This tension between interactive response time and hardware efficiency makes dynamic serving significantly more expensive per query than offline batching (example 13.3).

Example 13.3: The cost of latency

Latency constraints directly dictate infrastructure costs. Consider a GPU server renting for \$4/hour.

Scenario A (low latency): Batch size 1.

- Latency: 5 ms.
- Throughput: 200 req/s.
- Cost per million queries: \$5.56.

Scenario B (high throughput): Batch size 8.

- Latency: 10 ms (doubled due to batching overhead).
- Throughput: 800 req/s (quadrupled due to parallel efficiency).
- Cost per million queries: \$1.39.

Systems insight: Reducing latency from 10 ms to 5 ms increases the hardware bill by 300 percent. Engineers must quantify whether that speedup generates enough business value to justify the 4× cost increase.

Consider two deployment scenarios for a ResNet-50 image classifier. A static approach suits a photo organization app that preclassifies all images in a user's library overnight. With 10,000 photos and 5 ms inference each, batch processing takes ~50 s total, and users experience instant classification when browsing. A dynamic approach suits a content moderation API that must classify user-uploaded images in real time, with each image requiring the full preprocessing→inference→postprocessing pipeline and a 100 ms latency budget. A production image-classification system can combine the approaches: frequently requested images, such as popular products or known memes, are preclassified and cached, while novel uploads trigger dynamic inference.

The two modes therefore optimize for contrasting regimes. Static inference maximizes accelerator throughput and amortizes execution costs during offline batch runs, trading storage capacity for zero inference latency during the request. Dynamic inference minimizes tail latency under stochastic, concurrent load, accepting lower accelerator utilization and higher cost per query in exchange for handling arbitrary, live inputs. Production architectures often blend both paradigms into a tiered design: static precomputation caches common or high-probability queries, while a dynamic fallback evaluates novel or personalized requests.

Both static and dynamic cost models traditionally assume a fixed forward pass: a static computation graph executing once per request. Emerging reasoning models alter that economic calculus by deliberately spending variable computation per query to improve answer quality.

Systems Perspective 13.1: Looking ahead: Spending more compute per query

Traditional serving optimizes for minimizing latency ($L_{\text{lat}} \rightarrow 0$). Here, the optimization target expands to include answer quality. Some inference-time-compute systems deliberately spend more compute cycles to improve answer quality. Token generation is often memory-bandwidth bound at small batches, but larger batches or long contexts can shift the bottleneck toward compute or KV-cache traffic. These systems may generate far more tokens per request, including intermediate reasoning or search tokens, increasing the total compute and energy spent per query.

Whether a system executes a single deterministic forward pass or an extended reasoning trajectory, deciding when predictions are computed resolves only the temporal dimension of serving. Equally binding is the spatial dimension—where the model physically executes—because physical placement

dictates memory hierarchy limits, network boundaries, and the hardware budget available to the serving stack.

13.2.2 The spectrum of serving architectures

Although “serving” often implies a networked server processing API requests, the architectural pattern varies by deployment environment. Section 2.1 introduced the four deployment paradigms (Cloud, Edge, Mobile, and TinyML) and the physical constraints (the light barrier, the power wall, and the memory wall) that give rise to them. Architecturally, serving spans a continuum from **centralized networked microservices** (which pool accelerators to optimize throughput and cost-per-query at the expense of network round-trip overhead and cold-start latency) to **application-embedded edge serving** (which run models inside client processes via direct function calls, eliminating network latency and preserving privacy at the expense of strict device memory and power constraints) and out to **bare-metal firmware execution** on microcontrollers. Those constraints do not disappear at serving time; serving adds latency SLOs and cost pressure on top of hardware limits that training could absorb through patience. The same model may require different serving strategies depending on where it executes.

13.2.2.1 Networked serving (cloud/data center)

In networked serving, the model runs as a standalone service (microservice), matching the cloud deployment paradigm that section 2.5 characterized as trading latency for larger pooled compute. The primary interface is the network through protocols such as HTTP or gRPC, so network transfer and serialization join model execution as possible binding constraints. Data-center hardware such as NVIDIA GPUs (V100, A100, H100), Google Tensor Processing Units (TPUs), and AWS Inferentia supports high-throughput batching and concurrency, but cold start can still stretch from seconds to minutes because container startup, model loading, and warmup sit outside the steady-state inference path.

13.2.2.2 Application-embedded serving (mobile/edge)

In application-embedded serving, the model runs within the user application process (for example, a smartphone app using CoreML or TensorFlow Lite), following the embedded paradigm that section 2.6 and section 2.7 analyzed for its latency, privacy, and offline advantages. There is no “server.” The interface is a function call, so optimization focuses on energy and responsiveness (SingleStream) rather than shared-server throughput.

Embedded execution enables **zero-copy inference**: when data moves through a system, each copy consumes CPU cycles and memory bandwidth. In cloud serving, separate physical memory spaces force repeated transfers: a camera frame passes from a network socket buffer to user space, through a preprocessing buffer into page-locked host DRAM, and across PCIe into accelerator memory. A mobile pipeline can eliminate these intermediate copies when its camera, image signal processor (ISP), CPU, and NPU exchange shared buffer descriptors in unified system memory. In unified-memory system-on-chip (SoC) architectures—such as Apple A- and M-series processors or Qualcomm Snapdragon—all compute engines share a single physical DRAM pool. When software passes buffer handles rather than duplicating bytes, inference latency drops and memory bus power decreases, although format conversions may still require CPU or ISP intervention.

Typical hardware includes mobile NPUs (Apple Neural Engine, Qualcomm Hexagon) and embedded GPUs (NVIDIA Jetson). When the model is already resident and compiled, startup can be measured in milliseconds; first inference takes longer if it triggers cold-page faults or just-in-time compilation. Mobile power and thermal envelopes are strictly constrained by passive cooling (1–10 W); sustained high-throughput inference saturates the device chassis thermal capacitance, forcing the operating system’s thermal governor to throttle clock frequencies and degrade inference latency.

13.2.2.3 Bare-metal serving (TinyML)

In TinyML serving, the model is compiled directly into microcontroller firmware, reaching the extreme end of the deployment spectrum that section 2.8 introduced as ubiquitous sensing at microwatt power budgets. Many deployments lack a general-purpose operating system or dynamic memory allocator; serving executes as a tight bare-metal loop that samples sensors and invokes

a static interpreter. Because static random-access memory (SRAM) is constrained to hundreds of kilobytes, optimization focuses on executing model weights in place from non-volatile flash memory while fitting dynamic activation buffers into a preallocated Tensor Arena. Request batching is infeasible on single-stream sensor feeds. Typical hardware includes ARM Cortex-M series, ESP32, and specialized TinyML accelerators. Startup latency is measured in microseconds when weights remain in memory-mapped flash and execution buffers are preallocated statically, operating within a microwatt-to-milliwatt power envelope that sustains battery life across months or years.

The deployment tiers differ first in their operating envelopes. Table 13.1 compares the latency, batch, memory, power, update, failure, and monitoring constraints that follow from each environment.

Table 13.1: Serving Architecture Spectrum: Representative deployment envelopes for cloud, mobile, and TinyML serving. The latency, batch-size, memory, and power ranges are design guides rather than universal limits; update, failure, and monitoring rows summarize the operating pattern associated with each environment.

Characteristic	Cloud/Data center	Mobile/Edge	TinyML
Latency Target	10–100 ms	20–50 ms	1–100 ms
Batch Size	1–128 (dynamic)	1 (fixed)	1 (fixed)
Memory	16–80 GB VRAM	2–8 GB shared	256 KB–2 MB SRAM
Power	300–700 W	1–10 W	1–100 mW
Update Mechanism	Container deploy	App store update	Firmware over-the-air (OTA)
Failure Mode	Retry/failover	Graceful degradation	Silent or reset
Monitoring	Full telemetry	Limited analytics	Heartbeat only



Systems Perspective 13.2: ResNet-50 across the serving spectrum

Systems insight: The “same model” claim is misleading: the cloud and mobile tiers share the ResNet-50 graph but use different precision and runtimes, while the TinyML tier cannot run ResNet-50 and substitutes an architecture designed for its constraints. Treating these as one model hides the work that makes each deployment possible.

Applying those envelopes to one image-classification workload, table 13.2 shows why cloud and mobile can adapt ResNet-50 while TinyML must substitute MobileNetV2.

Table 13.2: Image Classification Across the Serving Spectrum: Illustrative side-by-side comparison of cloud, mobile, and TinyML serving for the same image-classification task, showing how model format, latency, throughput, memory footprint, and energy budget differ across deployment contexts.

Dimension	Cloud	Mobile	TinyML
Model format	TensorRT FP16 engine (51.2 MB)	TensorFlow Lite INT8 (25.6 MB)	Not feasible (25.6 MB); alternative: MobileNetV2 INT8 (3.5 MB)
Inference (batch-1)	1.4 ms (batch-16: 14 ms)	12 ms (NPU), 45 ms (CPU)	120 ms
Throughput	1,143 img/s (batched)	~80 img/s (single-stream)	~8 img/s
Memory	2 GB VRAM (batch-32)	150 MB peak (shared with app)	320 KB arena (fits in 512 KB SRAM)
Energy/inference	—	0.8 mJ (NPU), 4.2 mJ (CPU)	12 mJ

13.2.3 The load balancer layer

When traffic exceeds what a single machine can handle, cloud and data center deployments that run multiple replicas of the same model require an additional infrastructure layer: the load balancer. Production serving systems place load balancers between clients and model servers, providing three essential functions for serving infrastructure.

A serving load balancer manages three core responsibilities. First, request distribution routes incoming traffic to available model replicas using algorithms such as round-robin or least-connections; routing away from slow or saturated replicas directly curbs tail latency. Second, health monitoring verifies replica readiness by inspecting both process liveness and model initialization state, confirming that weights are loaded and warmup passes are complete before admitting traffic. Third, deployment support enables progressive rollouts by shifting traffic gradually between model versions rather than cutting over abruptly. Section 14.5.1.1 expands these traffic-shifting mechanisms into full validation strategies.

For single-machine serving with multiple model instances, such as running several Open Neural Network Exchange (ONNX) Runtime sessions, the framework and operating system handle request queuing. The full complexity of load balancing becomes necessary when scaling to distributed inference systems, where multiple machines serve the same model. The implementation details of request distribution algorithms and multi-replica architectures belong to that distributed context.

In single-machine capacity planning, the serving unit corresponds to the physical node's model throughput. The queueing dynamics analyzed in section 13.5 govern this baseline behavior, establishing the utilization boundaries where horizontal scale-out becomes mandatory.

While load balancers distribute requests *across replicas*, achieving predictable latency also requires controlling what happens *within each machine*. The operating system environment introduces its own sources of variability.

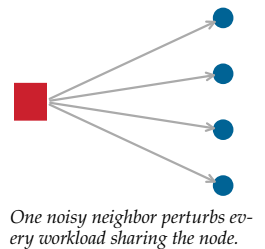
13.2.4 Deterministic latency and resource isolation

An inference server does not execute in a dedicated vacuum. On a single host, the operating system multiplexes hardware across background telemetry daemons, logging agents, and kernel worker threads. When these unisolated processes contend for execution resources, they intermittently preempt the inference pipeline. These noisy neighbors are a primary source of **latency jitter**, where the physical time required to execute identical forward passes fluctuates unpredictably. This contention inflates 99th percentile (p99) tail latency even when average hardware utilization remains low. Resource contention thus widens tail latency through service-time variance—a physical degradation distinct from the queue-delay accumulation analyzed in figure 13.1.

Achieving deterministic performance requires eliminating interference from the operating system's default time-sharing heuristics. Serving systems such as Clockwork demonstrate that deep neural network (DNN) inference execution times are intrinsically predictable down to sub-millisecond tolerances once hardware contention is systematically removed (Gujarati et al. 2020). CPU affinity (thread pinning) establishes the baseline compute boundary by binding inference worker threads to designated physical cores. Without pinning, the OS scheduler periodically migrates threads between cores, invalidating private L1 and L2 caches and forcing execution to stall while lines are refetched from shared L3 cache or DRAM. On multi-socket architectures, thread migration across Non-Uniform Memory Access (NUMA) boundaries is even more penalizing: instructions must access memory across inter-socket interconnects, adding tens of nanoseconds of latency per transaction. Pinning threads to specific cores and binding their memory allocations to local NUMA nodes preserves cache residency and eliminates cross-socket bus traffic.

Even with pinned CPU cores, memory access latency remains vulnerable to the virtual memory subsystem. Under host memory pressure, the kernel can page inactive address ranges out to disk. When the CPU or a PCIe Direct Memory Access (DMA) engine touches a paged-out address, execution stalls for milliseconds while the storage controller faults the physical page back into DRAM. Memory locking (`mlock`) instructs the kernel to pin specified virtual address ranges into physical host RAM, disabling page eviction and guaranteeing immediate access for host-resident weights, input staging buffers, and offloaded key-value (KV) cache blocks. This guarantee trades physical host memory flexibility for determinism, as locked pages cannot be reclaimed by other processes. Because `mlock` operates strictly on host page tables, it does not apply to accelerator HBM, which is allocated directly by the accelerator driver.

Beyond scheduled processes, asynchronous hardware interrupts introduce another source of sub-millisecond preemption. When a network interface card (NIC) receives packet bursts, it triggers hardware interrupt requests (IRQs) that force the handling CPU core to immediately suspend user-space execution and process kernel interrupt handlers. If the OS routes these interrupts to cores



running the inference engine’s submission loop, the CPU pauses command dispatch to the accelerator. Because GPUs process short inference layers in microseconds, even brief CPU submission stalls drain the accelerator’s hardware command ring buffer, causing pipeline bubbles where compute units sit idle waiting for the next kernel descriptor. Interrupt shielding restricts IRQ affinities to dedicated housekeeping cores, preventing incoming network traffic and disk activity from preempting the GPU dispatch stream.

Isolating physical cores, locking host memory, and shielding interrupt lines eliminate operating system noise, converting erratic execution paths into predictable, deterministic forward passes. Yet establishing service-time determinism on a single node addresses only half of the serving challenge. While hardware isolation guarantees that an accelerator kernel executes without host-side interference, running single requests sequentially leaves massive tensor compute pipelines severely underutilized. To achieve high throughput within latency deadlines, the internal software architecture of the serving engine must coordinate irregular, asynchronous request streams into structured, hardware-saturating batches.

13.3 Serving System Architecture

While host isolation ensures deterministic execution of individual kernels, production serving confronts an impedance mismatch at the system boundary. User requests arrive stochastically—bursting milliseconds apart, followed by seconds of silence—whereas accelerator compute engines demand steady streams of uniform, dense batches to saturate memory bandwidth and functional units. Bridging this gap requires a specialized serving architecture that decouples network ingress from execution scheduling, absorbing arrival bursts and packing irregular inputs into dense batches without violating latency SLOs.

13.3.1 Internal architecture and request flow

An inference server³ (such as NVIDIA Triton or TensorFlow Serving) functions as a specialized runtime scheduler that manages concurrency, device memory, and data movement between host CPUs and accelerator devices. Rather than invoking model execution synchronously upon request arrival, the server pipelines incoming requests through dedicated subsystems that isolate client network handling from accelerator compute. Every request traverses a multi-stage pipeline designed to maximize hardware throughput while bounding queuing latency. Figure 13.4 separates the six stages so each component’s role in absorbing traffic, queueing, batching, and accelerator execution is explicit.

This architecture serves three core functions:

First, concurrency management. Serving engines employ asynchronous event loops (such as `epoll` or `kqueue`) and worker thread pools to handle thousands of concurrent client TCP connections without blocking execution threads. Isolating client network I/O ensures slow or stalled network transfers never idle the accelerator compute pipeline.

Second, request transformation. The server decodes serialized network payloads into contiguous, aligned memory buffers matching the tensor layout required by the underlying execution engine. For example, 4D vision tensors may use NCHW⁴ or NHWC format. On modern accelerators, matrix multiplication units (such as Tensor Cores) load 8- or 16-element channel vectors using coalesced memory transactions that favor channels-last (NHWC) layout. If incoming payloads arrive in NCHW, the server or host must execute an explicit transpose kernel (`torch.permute` or a custom CUDA transpose). Because tensor transposition performs zero floating-point arithmetic ($I \approx 0$ FLOP/byte), it is strictly memory-bandwidth bound, consuming host memory or PCIe bandwidth and directly eroding the latency budget.

Third, model lifecycle management. The runtime orchestrates allocating device memory (VRAM), pinning host memory buffers for DMA, loading model parameter checkpoints, and executing warmup forward passes to compile execution graphs and initialize CUDA contexts before admitting live client traffic. While enterprise model registries, canary releases, and rollback policies belong to chapter 14, the serving engine governs local state: validating that active model instances are warmed, resident in accelerator memory, and ready for dispatch.

The execution **scheduler** coordinates the transition between request buffering and accelerator execution by implementing dynamic batching (section 13.7). At batch size $N = 1$, inference is

³ | **Inference server:** Google’s TensorFlow Serving (Olston et al. 2017) helped establish the separation of model logic from serving infrastructure; NVIDIA’s Triton (NVIDIA 2024d) extends this pattern across multiple model frameworks and backends. The critical design insight is that a scheduler and dynamic batcher turn irregular single-request traffic into accelerator-friendly execution, improving utilization when latency budgets allow batching. Exact utilization gains depend on model, hardware, arrival rate, and the configured batching window.

⁴ | **NCHW and NHWC (tensor memory layouts):** These acronyms encode the dimension order of 4D image tensors: N (batch), C (channels), H (height), and W (width). In a contiguous NCHW tensor, each channel plane is contiguous; in NHWC, channels for one spatial position are adjacent. Optimized kernels may favor either layout, and a semantic mismatch can scramble values even when the element count is correct. Serving code should therefore make both dimension meaning and any physical-layout conversion explicit.

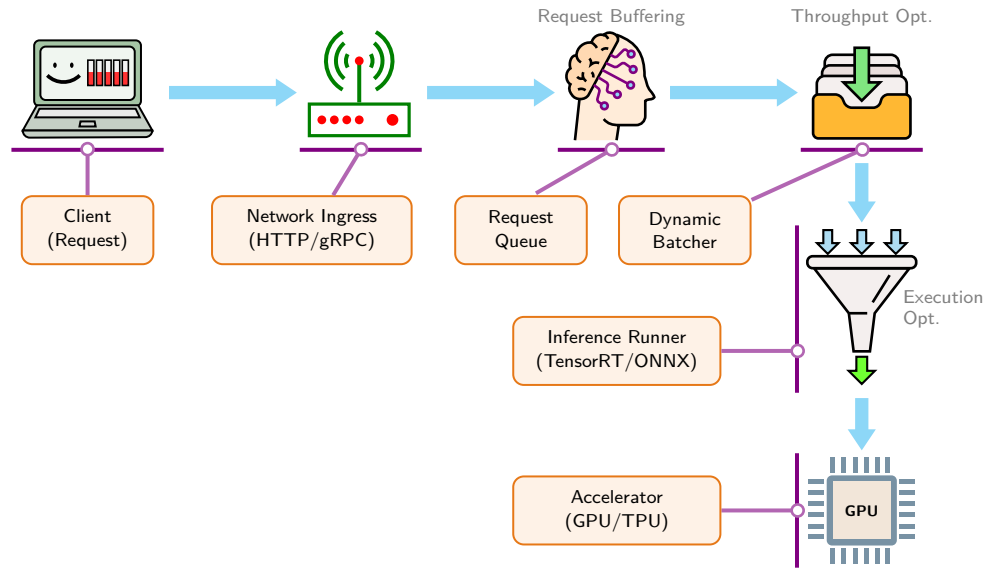


Figure 13.4: Inference Server Anatomy: A modern inference server decouples network handling from accelerator execution through a staged pipeline. Each stage isolates a concern, from absorbing bursty traffic to forming efficient batches, so the hardware accelerator stays highly utilized despite irregular arrival patterns.

severely memory-bandwidth bound: the accelerator must sweep all model parameters from HBM into on-chip SRAM to execute a single forward pass, leaving vector and matrix compute units idle. Grouping B requests into a concurrent batch amortizes the byte transfers of reading model weights across all B inputs, increasing the arithmetic intensity ($I = \text{FLOPs/bytes transferred}$).

To balance individual request latency against batch compute efficiency, the scheduler tunes queue residency using a **batching window** (or timeout threshold). Setting the batching window to zero yields pure FIFO execution with zero queuing delay, but subjects the accelerator to low-batch, memory-bound underutilization. Conversely, setting a bounded window (such as 2 to 5 ms) allows the scheduler to buffer arriving requests until either the queue reaches a target batch size or the timeout expires. This mechanism trades a bounded queuing delay for higher arithmetic intensity, amortizing memory bus traffic and increasing sustained cluster throughput.

13.3.2 Interface protocols and serialization

As accelerator kernels undergo aggressive compilation (via TensorRT, TorchDynamo, or custom kernels), device execution time often drops below a single millisecond. Under Amdahl's Law, when compute execution shrinks, the host-side overhead of ingesting, transporting, and deserializing payloads becomes the dominant term in end-to-end latency.

13.3.2.1 The serialization bottleneck

Machine learning payloads differ fundamentally from conventional web API payloads: they consist of multi-dimensional arrays of floating-point numbers (such as image tensors, dense embedding vectors, or token sequence logits). While text-based JSON is ubiquitous in web services, serializing dense numerical arrays into text incurs severe CPU and memory penalties. Serialization overhead manifests because JSON parsers must scan every byte sequentially, validate syntax, allocate heap nodes, and invoke string-to-float conversion routines (such as `strtod`). Each 32-bit float expands from 4 raw bytes into 7 to 15 ASCII characters including punctuation. Furthermore, raw binary data (such as compressed JPEG or raw image bytes) requires Base64 encoding to travel within JSON strings, imposing a 33 percent size inflation before parsing begins.

Binary serialization formats, such as Protocol Buffers⁵ and FlatBuffers⁶, bypass this text translation tax. Float arrays transmit directly as packed IEEE 754 byte streams with zero ASCII conversion and zero Base64 framing. Furthermore, FlatBuffers organizes serialized records with internal relative

⁵ | **Protocol Buffers (Protobuf):** Protobuf uses a predefined schema (from a `.proto` file) to encode structured data into a compact binary format (Protocol Buffers Authors 2026). Because the schema carries field names and types, the wire payload need not repeat them as JSON does. Its wire format is still not identical to a C++ object's in-memory layout, so it requires a parsing step and does not provide the same direct zero-copy access pattern that FlatBuffers targets.

⁶ | **FlatBuffers:** The “flat” in the name describes the design: the binary buffer can serve as the serialized representation and the data structure being read, avoiding a separate parsing or unpacking phase for supported access patterns (FlatBuffers Authors 2026). For ML inference, this can enable zero-copy access to tensor metadata—the serving system reads tensor shapes and offsets directly from the buffer rather than allocating a second object representation.

offsets, enabling zero-copy deserialization: the serving application reads tensor dimensions, data types, and byte offsets directly out of the network receive buffer without allocating an intermediate object graph on the heap.

13.3.2.2 REST vs. gRPC

Transport protocol selection governs connection life cycles and socket multiplexing. Conventional REST APIs typically deploy HTTP/1.1 with JSON payloads. Under HTTP/1.1, concurrent requests require opening multiple parallel TCP connections, each incurring three-way handshakes, TLS negotiation, and TCP slow-start window ramp-up. Furthermore, HTTP/1.1 suffers from transport head-of-line blocking: requests on a single connection must be served sequentially. While REST APIs can adopt HTTP/2 or binary encodings, the combination of HTTP/1.1 and JSON creates significant socket overhead and CPU parsing contention when serving high-throughput tensor workloads.

In contrast, gRPC (gRPC Remote Procedure Call)⁷ pairs HTTP/2 transport with Protocol Buffers serialization. HTTP/2 establishes a single long-lived TCP connection and multiplexes hundreds of concurrent request and response streams simultaneously using binary framing. This architecture eliminates per-request connection setup and head-of-line blocking at the transport layer, while Protocol Buffers enforces strongly typed contracts and serializes binary tensor payloads efficiently. Consequently, gRPC is the standard protocol for internal microservice communication in model serving clusters, where low latency, compact wire payloads, and bidirectional token streaming are primary design criteria.

A concrete payload comparison quantifies how the serialization format alters wire footprint and host CPU parsing overhead.



Napkin Math 13.1: JSON vs. Protobuf serialization

Consider a request payload containing 1,000 floating point numbers (for example, an embedding vector).

- **JSON:** Uses ~9 KB on the wire. Requires ~50 μ s to parse.
- **Protobuf:** Uses ~4 KB on the wire. Requires ~5 μ s to parse.

Math: Switching one request to the binary payload saves $50 \mu\text{s} - 5 \mu\text{s} = 45 \mu\text{s}$ of parse time. For this illustrative system processing 10,000 requests per second, the savings compound to $10,000 \times 45 \mu\text{s} = 0.45 \text{ s}$ of CPU time reclaimed every wall-clock second, which is 45 percent of one core freed from serialization overhead alone.

Systems insight: This 10 $\times$ scenario gain makes gRPC/Protobuf, FlatBuffers, or another binary protocol a strong candidate for high-throughput internal microservices when serialization is a visible part of the latency budget.

Architectural selection is governed by system constraints. REST over HTTP/1.1 remains practical for public-facing edge gateways where broad client compatibility and human-readable debugging take precedence over raw wire efficiency. Within cluster boundaries, gRPC over HTTP/2 with binary serialization becomes essential whenever high query volume, persistent connection multiplexing, or token streaming threatens to exhaust host CPU cycles on serialization tax.

Protocol selection and process architecture establish the structural skeleton of an inference server. Evaluating actual operational performance, however, requires tracing how latency accumulates as individual requests traverse each stage of this runtime pipeline.

13.4 Request Lifecycle

A single HTTP request carrying a 224 $\times$ 224 JPEG image arrives at an inference server. Between the moment the first byte enters the network stack and the moment the classification payload departs, the request traverses six distinct pipeline stages. Each stage consumes a slice of the end-to-end latency budget, making granular phase decomposition essential for targeted performance tuning.

⁷ | **gRPC (gRPC remote procedure call):** gRPC pairs HTTP/2 transport with an interface definition language and message format, most commonly Protocol Buffers (gRPC Authors 2026). The relevant serving advantage is the combination of typed contracts, persistent multiplexed connections, streaming support, and compact binary messages. The size and latency benefit over REST/JSON depends on payload shape, client/server implementation, and whether serialization is a meaningful share of the end-to-end latency budget.

13.4.1 The latency budget

For dynamic inference systems, the serving inversion established in section 13.1 creates a latency budget that shapes system design (Gujarati et al. 2020). A serving system with second-scale per-request latency may miss many interactive SLOs, even if it achieves excellent throughput.

Relevant metrics expand from aggregate throughput and mean latency to the latency distribution. Mean and p50 describe the center, while p95 and p99 expose the slow tail. If mean latency is 50 ms but p99 is two seconds, the p99 threshold is 40× the mean, so about 1 percent of requests are at or beyond two seconds. Whether that tail matters depends on the service and its users, which is why percentile targets should come from the SLO rather than convention.⁸

Managing these percentile constraints requires decomposing the total allowed response time into a **latency budget** that allocates time across each processing phase.

Definition 13.2: Latency budget

Latency budget is the time allocated to request stages under an end-to-end deadline; the SLO also specifies the required completion quantile.

1. **Significance:** It acts as a zero-sum constraint system where any milliseconds consumed by serialization or network overhead directly reduce the latency budget (L_{lat}) available for model inference.
2. **Distinction:** A latency **SLO** couples a deadline (for example, < 50 ms) with a completion quantile (p99 or p99.9). A service-level indicator (**SLI**) is the measured quantity used to evaluate that objective. Missing an internal SLO consumes reliability margin; crossing an external **SLA** threshold may trigger the remedies defined by the contract. Tail latency spikes can stem from queueing, batch formation, cache state, interference, or data movement.
3. **Common pitfall:** A frequent misconception is that the “model” has the entire budget. In the illustrative breakdown in table 13.3, the remainder is consumed by the request lifecycle (DNS, TLS, load balancing, serialization).

⁸ | **Tail latency:** Unlike averages, percentile latencies reveal the performance impact of system outliers common in ML serving, such as model cache misses or garbage collection pauses. These rare, high-latency requests can disproportionately harm user satisfaction and business outcomes. Because the magnitude is workload-specific, services validate percentile targets (p95, p99) against their own users and SLOs.

Every serving request decomposes into three phases that each consume part of the latency budget. Preprocessing transforms raw input such as image bytes or text strings into model-ready tensors. Inference executes the model computation. Postprocessing transforms model outputs into user-facing responses.

Checkpoint 13.1: Tail latency and budget

Serving optimizes tail latency under load. Use this checkpoint to separate queueing and batching effects before choosing an optimization.

- ☐ **Queue behavior:** use figure 13.1 to describe why latency rises nonlinearly as utilization approaches saturation.
- ☐ **Tail vs. mean:** can you explain why mean latency cannot demonstrate compliance with a p99 SLO?
- ☐ **Latency budget:** given a 50 ms p99 SLO and 18 ms spent on preprocessing, transfer, and postprocessing, how much remains for inference?

Faster hardware does not automatically mean faster serving. In practice, preprocessing and postprocessing can dominate total latency when inference runs on optimized accelerators. Optimizing exclusively the inference phase yields diminishing returns if the surrounding pipeline remains bottlenecked by CPU operations.

pre infer post

Inference is one slice of the latency budget; preprocessing rivals it.

13.4.2 Latency distribution analysis

Understanding where latency accumulates requires instrumenting each stage along the serving pipeline independently. In image classification, end-to-end request time conflates host-side image decoding, PCIe bus transfers, accelerator compute, and host-side output parsing. Measuring each phase in isolation exposes where the latency budget is actually spent.



Example 13.4: ResNet-50: Latency budget breakdown

Table 13.3 details the per-phase latency breakdown for an illustrative ResNet-50 serving request.

Table 13.3: ResNet-50 latency budget: Illustrative per-phase breakdown of a single serving request, showing that preprocessing and data transfer together rival the cost of the ResNet-50 forward pass itself. The percentages expose where engineering effort pays off in this scenario. Per-phase percentages are rounded to the nearest whole number and may not sum to exactly 100.

Phase	Operation	Time	Percentage
Preprocessing	JPEG decode	3 ms	30%
Preprocessing	Resize to 224×224	1 ms	10%
Preprocessing	Normalize (mean/std)	0.5 ms	5%
Data Transfer	CPU→GPU copy	0.5 ms	5%
Inference	ResNet-50 forward pass	5 ms	50%
Postprocessing	Softmax + top-5	0.1 ms	~1%
Total		10.1 ms	100%

Systems insight: In this illustrative ResNet-50 serving budget, preprocessing consumes 44.6 percent of latency despite model inference being the computationally intensive phase. If TensorRT reduces inference to the assumed 2 ms, preprocessing would dominate at 63.4 percent.

The ResNet-50 scenario represents compute-bound inference: dense convolutional arithmetic dominates the forward pass. Changing the model architecture, however, can shift the binding constraint from arithmetic execution units to memory bandwidth. Recommendation systems exhibit this fundamental shift.



Lighthouse 13.1: Lighthouse example: DLRM serving

Scenario: Serving DLRM with a 10 ms p99 latency budget.

Analysis: While ResNet-50's model stage is dominated by convolutional neural network compute, DLRM's dominant model-stage cost is embedding-table memory access. End-to-end serving bottlenecks still require measuring the full path: preprocessing, inference, postprocessing, and data movement. Table 13.4 breaks the recommendation request down by phase.

Table 13.4: DLRM serving latency: Illustrative per-phase breakdown of a recommendation request under a 10 ms p99 budget, contrasting DLRM's memory-bandwidth-bound embedding lookups against ResNet-50's compute-bound forward pass. Adding compute to the inference stage does not help once embedding-table bandwidth is the binding constraint.

Phase	Operation	Time	Bottleneck
Input Parsing	Request parsing	0.5 ms	CPU
Embedding Lookup	Fetch 100+ dense vectors	6 ms	memory bandwidth
Inference	MLP forward pass	1.5 ms	Compute
Postprocessing	Ranking & Filtering	1 ms	CPU
Total		9 ms	

Systems insight: In DLRM, the “Inference” multilayer perceptron stage is only ~17 percent of the latency. The majority of time is spent in embedding lookups, retrieving 128-dimensional embedding vectors from terabyte-scale tables. This is a memory-bandwidth and capacity-

bound workload where adding more compute does not help unless the embedding tables can be served faster.

Together, table 13.3 and table 13.4 expose the same general failure mode: straightforward optimization efforts target where ML expertise applies (model quantization, pruning) while the binding constraint sits elsewhere (image decoding on CPU for ResNet-50, embedding-table memory bandwidth for DLRM). The pattern generalizes: any serving system where the model accounts for less than half of total latency will see diminishing returns from model-only optimizations, regardless of how large those individual speedups are. Amdahl's law quantifies the ceiling. Adopting the quantitative approach to serving exposes these hidden bottlenecks before engineering effort is misallocated.

Systems Perspective 13.3: The quantitative approach to serving

Analysis: Amdahl's law governs this ceiling (section D.2.3 provides the formal derivation). Preprocessing (4.5 ms) and data transfer (0.5 ms) consume 49.5 percent of total latency. Optimizing the model $10\times$ faster (5 ms $\rightarrow$ 0.5 ms) yields only a $1.8\times$ end-to-end speedup (reducing latency from 10.1 ms to 5.6 ms). Focusing exclusively on model optimizations like quantization or pruning yields diminishing returns when non-model stages dominate execution time.

Mechanism: Domain-specific accelerators (such as TPUs and Tensor Cores) replace complex control logic with dense multiply-accumulate arrays, improving achieved throughput for compatible kernels. This makes hardware acceleration an economic requirement for many high-throughput or low-latency serving workloads.

Systems insight: Profile before optimizing. If preprocessing dominates, GPU-accelerated pipelines (NVIDIA DALI) may outperform model quantization.

Moving preprocessing closer to the accelerator can reduce avoidable CPU-to-GPU transfers, but the end-to-end gain is pipeline-specific. Effective optimization targets the largest time consumers first.

13.4.2.1 The serving tax bill

Beyond model forward-pass execution, every request pays an infrastructure “tax” to the host system and network stack. When an inference query arrives, it traverses the network interface, moves across kernel and user space, deserializes from wire protocols into memory buffers, queues for batch assembly, and copies across the PCIe interconnect to accelerator device memory. While hardware acceleration compresses matrix computation by orders of magnitude, these host-side operations run on general-purpose CPUs and I/O buses. Consequently, as models become more optimized, this non-inference serving tax represents an increasing fraction of total latency.

13.4.2.2 The killer microseconds problem

Barroso, Patterson, and colleagues identified a critical gap in how computer systems handle latency across time scales (Barroso et al. 2017). Nanosecond-scale stalls (such as cache misses) are hidden by hardware instruction-level parallelism, while millisecond-scale delays (such as disk I/O or network RPCs) are hidden by operating system thread context switching. Operations in the microsecond range (1–100 μ s) fall into an awkward gap: they are too long to spin-wait on a CPU core without wasting compute cycles and power, yet too short for the operating system to context-switch to another thread without paying scheduling overheads that exceed the wait time itself.

Serving infrastructure operates directly in this killer microseconds regime. Using the representative ranges in table 13.5, serialization at 50–500 μ s, dispatch at 10–50 μ s, and data copy at 100–500 μ s are each individually small, but for a 5 ms inference service, these named microsecond-scale overheads collectively consume about 3.2 percent to 21 percent of the latency budget before network and queuing delays are counted. No single overhead justifies optimization in isolation, yet together they determine whether the system meets its SLO.

Table 13.5: The Serving Tax Bill: A representative breakdown of noninference latency sources. While individual components like serialization can be below 1 ms, they compound. Under the upper ends of the displayed ranges, this tax can rival a 5 ms model invocation. The engineering goal is to measure the actual request path and reduce the binding costs through choices such as binary payloads, persistent connections, and fewer copies.

Tax Component	Typical Cost	Scaling Behavior	Tax Evasion Strategy
Network I/O	1–5 ms	Linear with payload	Compression, Region Colocation
Serialization	50–500 μ s	Linear with payload	gRPC/Protobuf (vs. JSON)
Queuing	0.1–10 ms	Diverges near capacity	Dynamic Batching, Autoscaling
Dispatch	10–50 μ s	Constant per batch	Kernel Fusion (reduce launches)
Data Copy	100–500 μ s	Linear with tensor	Zero-Copy/Shared Memory

The latency budget framework provides a systematic approach to this compound problem. Measurement comes first. Without per-phase instrumentation, engineers cannot distinguish a preprocessing bottleneck from a serialization bottleneck, and optimization effort is misallocated to the most visible component (the model) rather than the most expensive one. Once measurement reveals the true distribution of time, engineering effort must flow proportionally. A phase consuming 50 percent of latency deserves more attention than one consuming 5 percent, regardless of which appears easier to optimize. Architectural changes such as GPU-accelerated preprocessing or aggressive batching can shift work between phases entirely, sometimes eliminating a bottleneck rather than merely reducing it.

13.4.3 Resolution and input size trade-offs

Input resolution affects both preprocessing and inference latency, but the relationship differs depending on whether the system is compute bound (limited by arithmetic throughput) or memory-bound (limited by data movement). A compute-bound system slows as computation grows; a memory-bound system slows as transferred bytes grow. Capacity fit alone does not remove bandwidth cost. The roofline analysis in section 11.6 develops this distinction in depth, making it essential for informed resolution decisions.

For convolution-dominated, compute-bound models, equation 13.1 gives a first-order approximation in which throughput scales inversely with resolution squared:

$$\frac{\text{Throughput}(r_2)}{\text{Throughput}(r_1)} = \left(\frac{r_1}{r_2} \right)^2 \quad (13.1)$$

Doubling resolution from 224 to 448 gives a 4 $\times$ compute-scaling estimate; the illustrative 3.6 $\times$ scenario includes fixed overhead. Higher resolution can raise convolution arithmetic intensity because weights are reused across more spatial positions, but whether the full model becomes compute-bound depends on the bytes moved by its kernels.

Table 13.6 quantifies how increasing resolution amortizes fixed weight traffic in this simplified model while leaving the executed-kernel bottleneck unresolved.

Table 13.6: Resolution and Modeled Arithmetic Intensity: This estimate divides FLOPs by weight bytes plus one assumed read/write of a scaled activation term. It omits aggregate intermediate-tensor, cache, and workspace traffic, so crossing the V100 ridge point does not establish the executed-kernel bottleneck.

Resolution	Assumed Activation Term	Modeled Arith. Intensity	Bottleneck Classification
224 $\times$ 224	12.5 MB	64.4 FLOP/byte	Not established
384 $\times$ 384	36.7 MB	137 FLOP/byte	Not established
512 $\times$ 512	65.3 MB	183.9 FLOP/byte	Not established
640 $\times$ 640	102.0 MB	218.4 FLOP/byte	Not established

13.4.3.1 Resolution strategies in production

Different deployment contexts impose distinct resolution requirements shaped by their dominant constraints. A mobile application may accept lower resolution (224 $\times$ 224) when latency and battery life outweigh marginal accuracy gains. Some medical-imaging workloads use 512 $\times$ 512 or higher when diagnostic detail warrants the additional compute. A perception pipeline may use lower

resolution over a wide field and high-resolution crops for fine-grained recognition. A public API can instead normalize arbitrary uploads to a fixed contract or select among validated resolution profiles. Adaptive resolution is useful only when the selection policy and every chosen profile preserve the task’s quality requirements.

13.4.3.2 Adaptive resolution

Adaptive resolution lets production systems select resolution dynamically based on content. One approach runs a lightweight classifier at 128×128 to categorize content type, then selects task-appropriate resolution with documents at 512×512 , landscapes at 224×224 , and faces at 384×384 . In this illustrative scenario, the policy provides $1.4\times$ throughput improvement with 99.2 percent accuracy retention vs. fixed high resolution. This pattern trades preprocessing cost from running the lightweight classifier for inference savings on the main model.

The latency analysis so far has focused on sequential processing: one request completing before the next begins. The preprocessing, inference, and postprocessing stages use different hardware resources. This separation creates an opportunity to process multiple requests simultaneously.

13.4.4 Hardware utilization and request pipelining

Optimizing each request stage in isolation misses a critical opportunity: the stages use different hardware resources. The latency budget analysis in section 13.4.1 reveals that model inference is only one component of the request lifecycle. From a hardware perspective, the primary goal of a serving system is to maximize the **duty cycle** of the accelerator, the percentage of time the GPU is performing useful computation.

In a serialized serving system, the accelerator sits idle during network I/O and CPU-based preprocessing. Serving systems use **request pipelining** to overlap these stages, ensuring the GPU is fed a continuous stream of tensors.

13.4.4.1 Overlapping I/O and compute

The two timing diagrams in figure 13.5 illustrate the impact of pipelining. In the serial case (A), each request must complete its entire lifecycle (Network → CPU Preprocessing → GPU Inference → Postprocessing) before the next request begins, and the gray idle gaps leave the GPU unused for more than 50 percent of the time. In the pipelined case (B), those gaps disappear.

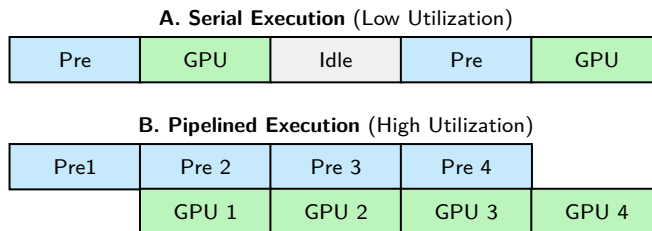


Figure 13.5: Request Pipelining: Pipelining hides latency by overlapping independent operations across different hardware resources. In pipelined execution (B), the CPU processes the next request’s data while the GPU executes the current request’s inference. This can increase accelerator duty cycle and throughput without changing the model.

Pipelining relies on non-blocking hardware queues and asynchronous execution streams. Instead of blocking until a GPU kernel completes, the host CPU thread enqueues memory transfers and kernel launches into an accelerator stream and immediately begins preprocessing the next incoming request. The accelerator’s DMA engine and compute cores execute those queued operations independently, overlapping host preprocessing with device execution.

13.4.4.2 The systems metric: Hardware duty cycle

System efficiency measures how fully a serving system saturates its bottleneck resource, usually the GPU’s compute cores or memory bandwidth. Equation 13.2 defines this hardware duty cycle.

$$\text{System Efficiency} = \frac{\sum T_{\text{compute}}}{\text{Wall Clock Time} \times \text{Resource Count}} \quad (13.2)$$

If a ResNet-50 request takes 10 ms total (5 ms GPU, 5 ms CPU), a serial system achieves only 50 percent efficiency. By pipelining just two requests, efficiency approaches 100 percent (assuming the CPU can keep up with the GPU). If the CPU is too slow to feed the GPU, the system becomes CPU-bound, and further model optimization provides zero throughput gain. This is Amdahl's Law from section D.2.3 applied to serving: if preprocessing consumes 50 percent of latency, maximum speedup is $2\times$ regardless of how fast the model runs. The hardware trajectory makes this ceiling progressively tighter. Accelerator compute throughput (FLOPs) has grown far faster than CPU single-thread performance across successive hardware generations, so the inference portion of the pipeline shrinks while the CPU-bound preprocessing portion remains unchanged. A system that was compute-bound on an older accelerator may become CPU-bound after a hardware upgrade—not because preprocessing got slower, but because the model got dramatically faster while the CPU did not.

13.4.5 Postprocessing

The request lifecycle concludes with postprocessing, transforming raw accelerator output tensors into structured application responses. This stage straddles the physical boundary between the accelerator and the host CPU. Because client applications cannot consume device memory buffers directly, output tensors must cross the interconnect (typically a PCIe bus) to reach host memory. How the serving engine partitions computation across this boundary dictates whether postprocessing completes in microseconds or stalls the entire serving pipeline.

13.4.5.1 From logits to predictions

Classification models emit unnormalized logits across the label space. Converting these numerical scores into actionable predictions requires reduction, filtering, and calibration. The simplest reduction is argmax selection, which extracts the index of the highest logit. Thresholding discards predictions below an empirical certainty cutoff, returning predictions only when the model meets a target precision. When applications require ranked alternatives, top- k extraction selects the k largest scores alongside their class indices. Probability calibration scales raw scores (for example, through temperature scaling) so that output confidence scores reflect empirical accuracy.

Systems efficiency dictates that reduction operations execute directly on the accelerator rather than on the host CPU. Performing softmax, argmax, or top- k selection in device memory prunes the payload before it crosses the PCIe bus. In ResNet-50 classification, for example, running top- k on the GPU reduces the transfer payload from 1,000 float32 logits (4,000 bytes) to five values and five integer indices. For ResNet-50 image classification, listing 13.1 traces this execution path from GPU tensor operations to host CPU label lookup and response formatting.

Listing 13.1: ResNet-50 Postprocessing: Normalizes raw logits into probabilities, extracts top- k predictions, and formats the API response.

```
# Normalize raw logits into probabilities
# Input: logits tensor of shape (1, 1000), one score per ImageNet
# class
probs = torch.softmax(logits, dim=-1) # Normalize to sum=1 on GPU

# Extract top-5 predictions for multi-class response
# topk returns (values, indices) sorted by probability
top5_probs, top5_indices = probs.topk(5) # GPU operation
top5_probs = top5_probs.squeeze(0).tolist()
top5_indices = top5_indices.squeeze(0).tolist()

# Map class indices to human-readable labels
# imagenet_labels: list of 1000 class names from synset mapping
labels = [imagenet_labels[i] for i in top5_indices] # CPU lookup

# Format response with predictions and metadata for API contract
response = {
    "predictions": [
        {"label": label, "confidence": float(prob)}
        for label, prob in zip(labels, top5_probs)
    ],
    "model_version": "resnet50-v2.1", # Client-side version tracking
    "inference_time_ms": 5.2, # Observability for latency monitoring
}
```

In listing 13.1, total postprocessing time is approximately 0.1 ms, negligible compared with preprocessing and inference. The GPU handles the normalization and top-5 reduction, so the subsequent `.tolist()` call transfers only tens of bytes across PCIe. However, `.tolist()` forces a synchronous device-to-host copy, blocking the host thread until the GPU stream finishes. In high-throughput pipelined servers, such synchronization barriers must be scheduled carefully to avoid stalling the CPU thread while it feeds subsequent requests. Fitting a calibration mapping is an offline task; applying a simple fitted mapping online is usually inexpensive, but it is valuable only when downstream decisions require confidence scores whose empirical meaning has been validated.

13.4.5.2 Output formatting

Production systems package predictions to satisfy application-level API contracts. These contracts govern JSON serialization schemas, floating-point formatting, error codes, and fallback defaults when thresholding rejects all candidates. In Python-based runtimes, repetitive JSON serialization and string allocation can consume unexpected host CPU cycles under heavy concurrency, turning a microsecond-scale formatting step into a CPU bottleneck. High-throughput servers address this by delegating serialization to native C++ runtimes or binary protocols like Protocol Buffers. Response metadata, including model version and inference execution time, completes the payload to enable downstream observability.

Tracing an isolated request reveals the intrinsic execution time of each pipeline phase. In production, however, multiple requests arrive concurrently and contend for shared accelerators, CPU cores, and memory buses, introducing queuing delays that dwarf raw computation time.

13.5 Queuing Theory

When arrival streams fluctuate, queuing theory formalizes how contention for shared physical resources—accelerator execution queues, host-to-device PCIe transfers, and memory buses—degrades latency distributions. Its governing laws explain why seemingly well-provisioned inference services experience sharp, nonlinear latency explosions as utilization approaches capacity.

13.5.1 Little's Law

Capacity planning begins with a fundamental sizing question: given an expected arrival rate and measured mean response time, how many concurrent requests reside in the system at any instant? Answering this determines how much memory must be reserved for request buffers and activation states before deciding how many accelerators to provision. Little's Law (section D.2.4) resolves this concurrency question by relating mean in-system population to throughput, while queuing models predict how response times degrade as utilization nears saturation. Together, they provide the quantitative framework for capacity planning.

🔍 Systems Perspective 13.4: Notation alert: L vs. latency

In queuing theory, T_{lat} denotes the *response time* or *time in system per request*; queue-only waiting time is W_q . Here, Q_{req} denotes the average in-system request count, λ_{arr} for arrival rate, and $\rho_{\text{serv}} = \lambda_{\text{arr}}/\mu$ for serving utilization. The subscripts distinguish queueing notation from the degradation equation's λ sensitivity parameter and keep serving utilization from occupying bare ρ . In subsequent latency-budget equations (such as equation 13.4 and equation 13.7), descriptive $L_{\text{lat},*}$ terms name components of the budget, such as waiting and compute. In the batching analysis that follows (section 13.7.3), $L_{\text{lat},\text{wait}}$ corresponds to the queueing wait component W_q , and $L_{\text{lat},\text{compute}}$ includes inference time.

Formulated by John D. C. Little, **Little's law**⁹ establishes an exact relationship among three macroscopic metrics in any stable queuing system:

$$Q_{\text{req}} = \lambda_{\text{arr}} \cdot T_{\text{lat}} \quad (13.3)$$

where Q_{req} is the average number of requests residing in the system, λ_{arr} is the arrival rate (requests per second), and T_{lat} is the average time each request spends from arrival to completion.

Crucially, Little's law holds regardless of the arrival distribution, service-time distribution, or scheduling discipline—it requires only that the system is stable and that requests are neither created nor destroyed internally. For an inference service targeting 1000 QPS with a mean response time of 50 ms, Little's law dictates the average in-flight request population that host memory buffers and worker threads must sustain.

📝 Napkin Math 13.2: Little's Law capacity sizing

Problem: How many requests are in the system on average at 1,000 QPS?

Math: Little's Law gives $Q_{\text{req}} = \lambda_{\text{arr}} T_{\text{lat}}$, so average population equals throughput multiplied by mean time in system (section D.2.4 derives the law).

Given:

- **Throughput target** (λ_{arr}): 1,000 QPS.
- **Mean time in system** (T_{lat}): 50 ms (0.05 s).

Math: $Q_{\text{req}} = 1,000 \text{ QPS} \times 0.05 \text{ s} = 50$ concurrent requests on average

Systems insight: The system holds 50 requests on average across batch and queue state. A GPU batch-size limit of 32 does not by itself make 1,000 QPS impossible because queued requests may reside in host memory; service rate, queueing behavior, and the tail distribution determine feasibility.

Because equation 13.3 binds these three variables, constraining any two dictates the third. If an inference service averages 10 ms per request ($T_{\text{lat}} = 0.01 \text{ s}$) and holds 50 concurrent requests on average ($Q_{\text{req}} = 50$), the sustainable arrival rate must be $\lambda_{\text{arr}} = Q_{\text{req}}/T_{\text{lat}} = 5000$ requests per second. Conversely, if host memory limits concurrency to an average of 10 requests while average time in system remains 10 ms, the system cannot exceed a throughput of 1,000 requests per second without dropping traffic.

⁹ | **Little's law:** John D. C. Little (1961) proved that $Q_{\text{req}} = \lambda_{\text{arr}} T_{\text{lat}}$ holds for stable long-run averages over consistent system boundaries, regardless of arrival distribution, service distribution, or scheduling discipline; this universality anchors ML capacity planning: the formula requires no Poisson-arrival or exponential-service assumption. It does require well-defined long-run averages and consistent definitions of arrival rate, time in system, and population.

13.5.2 The batching tax: The latency-throughput frontier

While Little’s Law relates queue depth to throughput, it does not account for the **batching tax**: the deliberate delay introduced to maximize hardware utilization.

At batch size $B = 1$, neural network inference is severely memory-bandwidth bound. Accelerators must transfer gigabytes of model weights from HBM to on-chip SRAM to execute matrix-vector multiplications, achieving only a fraction of their peak FLOPS. Batching B inputs converts these operations into general matrix multiplication (GEMM), reusing weights across inputs and increasing arithmetic intensity. However, gathering B requests requires holding earlier arrivals in an input queue.

When an inference server uses request-triggered fixed-size batching, it introduces two distinct sources of latency. **Batch formation delay** ($L_{\text{lat,form}}$) is the time each request waits for the batch to fill; the first request waits longest, and the last waits zero. **Inference inflation** is the growth in inference time $T_{\text{inf}}(B)$ when the accelerator processes B samples instead of 1. The resulting latency-throughput Pareto frontier represents the operating envelope where throughput gains require paying a tax in response latency. Under stationary Poisson arrivals, equation 13.4 approximates the average per-request latency for batch size B and arrival rate λ_{arr} :

$$L_{\text{lat,total}} \approx \underbrace{\frac{B-1}{2\lambda_{\text{arr}}}}_{\text{Formation delay}} + \underbrace{T_{\text{inf}}(B)}_{\text{Inference time}} \quad (13.4)$$

Equation 13.4 formalizes the trade-off between accelerator throughput and individual request latency. Increasing B raises arithmetic intensity and amortizes memory bandwidth across inputs, but inflates the per-request queueing delay. Concretely, at 500 QPS, moving from batch-1 to batch-32 increases wait-time from 0 ms to 31 ms, contributing to a $23\times$ total latency penalty (2 ms $\rightarrow$ 46 ms). Sizing B is therefore an economic optimization: operators select the largest batch size that maximizes query throughput and hardware efficiency without breaching the latency SLO (L_{lat}).

13.5.3 The utilization-latency relationship

Little’s Law describes average system inventory under steady-state conditions, but it does not reveal how latency behaves as traffic approaches system capacity. The M/M/1 queuing model quantifies the headroom required to prevent latency collapse (Harchol-Balter 2013).¹⁰ For a system with Poisson arrivals and exponential service times, equation 13.5 gives the average time in system:

$$T_{\text{lat}} = \frac{1}{\mu - \lambda_{\text{arr}}} = \frac{\text{service time}}{1 - \rho_{\text{serv}}} \quad (13.5)$$

where λ_{arr} is the arrival rate, μ is the service rate (requests per second the server can handle), and $\rho_{\text{serv}} = \lambda_{\text{arr}}/\mu$ is the utilization (the fraction of time the server is busy).

Because the denominator contains $1 - \rho_{\text{serv}}$, latency grows nonlinearly as utilization approaches unity.¹¹ Small incremental increases in traffic near saturation trigger severe backpressure and queuing delays.

The standard M/M/1 formulation assumes exponentially distributed service times. For compiled neural networks processing fixed-size batches and static tensor dimensions, execution times are nearly constant, making an M/D/1 queue a tighter model of mean queuing delay. The analysis here retains the M/M/1 model because it yields closed-form solutions and provides a conservative safety margin. In production environments, variable input lengths, dynamic batch formation, CPU preprocessing, and key-value cache lookups introduce runtime variance that quickly breaks deterministic assumptions.¹²

Table 13.7 quantifies this relationship, demonstrating how average time in system multiplies rapidly as utilization approaches 100 percent.

10 | M/M/1 queue: Queuing theory originated with Agner Krarup Erlang’s 1909 analysis of the Copenhagen Telephone Exchange, where call arrivals were modeled as memoryless (Poisson). The M/M/1 model’s exponential service-time assumption introduces more variance than many fixed-shape ML inference workloads exhibit. At the same utilization and mean service time, its mean *queueing wait* is twice the M/D/1 value; the ratio for total response time approaches two only as utilization becomes high. The model is therefore a useful conservative approximation, not a universal description of inference traffic.

11 | Super-linear latency divergence: The planning knee often appears well before full saturation; in the M/M/1 mean response-time equation, $E[T] = \frac{1/\mu}{1 - \rho_{\text{serv}}}$, where $\rho_{\text{serv}} = \lambda_{\text{arr}}/\mu$ is utilization. The $(1 - \rho_{\text{serv}})^{-1}$ term diverges as $\rho_{\text{serv}} \rightarrow 1$: at $\rho_{\text{serv}} = 0.7$, mean response time is already $3.3\times$ the base service time; at $\rho_{\text{serv}} = 0.9$, it is $10\times$. The exact operating limit is a policy and workload choice, but trying to stretch a latency-sensitive queue toward saturation creates disproportionate latency growth.

12 | Kendall notation: In the A/S/c (arrival/service/servers) notation, “M” signifies a Markovian (memoryless) process and “D” means deterministic. M/M/1 is conservative relative to M/D/1 for mean queueing wait when service times are nearly deterministic, but neither model replaces measurement of a production workload’s arrival process, service-time variance, or tail behavior.

Table 13.7: Utilization-Latency Relationship: Average time in system (wait + service) as a multiple of service time for an M/M/1 queue. At 50 percent utilization, time in system is 2× service time; at 90 percent, it reaches 10×. This nonlinear growth explains why systems that perform well at moderate load suddenly violate SLOs when traffic increases: moving from 80 percent to 90 percent utilization doubles latency.

Utilization (ρ_{serv})	Latency Multiple	Example (5 ms service)
50%	2×	10 ms
70%	3.3×	17 ms
80%	5×	25 ms
90%	10×	50 ms
95%	20×	100 ms

13.5.4 Multi-server considerations

A single serving node forms the fundamental building block of inference infrastructure. Single-node queuing dynamics govern resource saturation and establish the baseline performance characteristics that persist when horizontally replicated across a cluster.

Evaluating an isolated node under M/M/1 assumptions establishes the headroom needed before scaling out, testing whether a single accelerator can absorb peak traffic without violating service-level targets. Production profiling must verify whether real-world bottlenecks stem from raw compute saturation, dynamic batching thresholds, host preprocessing, or memory allocation stalls.

When traffic exceeds single-node capacity, the architecture scales out horizontally: multiple independent serving nodes sit behind a load balancer, each hosting a complete, identical model instance. The M/M/ c queuing model extends single-server analysis to c parallel workers, showing how pooled capacity absorbs arrival bursts when queries are uniformly distributed. Tail latency improvements in practice depend heavily on dispatch algorithms, service-time variance across heterogeneous inputs, and per-replica utilization margins. Crucially, this horizontal replication remains architecturally distinct from model parallelism (tensor or pipeline sharding): replication duplicates the model across independent nodes to serve distinct queries concurrently without inter-device synchronization, whereas model parallelism shards a single model across multiple accelerators connected by high-bandwidth interconnects (such as NVLink) to execute one query collaboratively.

13.5.5 Tail latency

Production SLOs typically specify percentile targets (p95, p99) rather than averages because tail latency determines user experience for the slowest requests (Dean and Barroso 2013). For an M/M/1 queue, the p99 latency follows:

$$T_{\text{lat},p99} \approx \frac{\text{service time}}{1 - \rho_{\text{serv}}} \cdot \ln \left(\frac{1}{1 - 0.99} \right) \approx \frac{4.6 \cdot \text{service time}}{1 - \rho_{\text{serv}}} \quad (13.6)$$

Mean latency masks tail contention: at 70 percent utilization, average latency is only 3.3 times service time, while p99 latency reaches 15.3 times service time ($4.6/0.3 \approx 15.3$). For deterministic-service models such as M/D/1, tail values require numerical calculation rather than a simple universal multiplier, but the core systems reality remains: evaluating a serving cluster by average latency conceals severe tail degradation.

13.5.5.1 The tail at scale problem

Dean and Barroso’s analysis reveals why tail latency becomes critical as systems scale beyond single machines (Dean and Barroso 2013). When requests fan out to m parallel backends—such as querying an ensemble of models, searching sharded embedding indices, or parallelizing pipeline stages—the overall user request completes only when the slowest sub-task finishes. If each sub-task has a 99 percent probability of finishing within latency threshold T , the probability that all m backends finish within T is $(0.99)^m$. For a fan-out of $m = 10$, only $0.99^{10} \approx 90.4\%$ of requests meet the target; at $m = 100$, that probability drops to $0.99^{100} \approx 36.6\%$, meaning nearly two-thirds of user requests

experience a tail delay. This “tail at scale” effect makes controlling single-node tail variance essential for overall system performance.

Before exploring software mitigations like hedging, systems engineers must first provision sufficient accelerator capacity to satisfy tail-latency SLOs under peak load. Applying the M/M/1 tail-response-time model to a ResNet-50 deployment makes the provisioning workflow concrete.

Napkin Math 13.3: ResNet-50 capacity planning

Consider designing a ResNet-50 serving system with these requirements:

- **Target p99 latency:** 50 ms
- **Peak expected traffic:** 5,000 QPS
- **Service time** (TensorRT FP16): 5 ms

Step 1: Find safe utilization. From equation 13.6, $T_{\text{lat},p99} \approx 4.6 \times \text{service time} / (1 - \rho_{\text{serv}})$. Setting $T_{\text{lat},p99} \leq 50$ ms with 5 ms service time gives $\rho_{\text{serv}} \leq 1 - (4.6 \times 5 \text{ ms}) / 50 \text{ ms} = 0.54$ (54 percent maximum utilization). This uses the conservative M/M/1 p99 bound from the displayed equation rather than applying an average-wait M/D/1 adjustment to a tail-latency SLO.

Step 2: Calculate required service rate. $\mu_{\text{required}} = 5,000 \text{ QPS} / 0.54 = 9259.3 \text{ req/s}$

Step 3: Determine GPU count. In this M/M/1 model, the single-GPU rate is 200 req/s.

GPUs needed = $9259.3 \text{ req/s} / 200 \text{ req/s} = 46.3 \rightarrow 47$ GPUs

Step 4: Add headroom for variance. This scenario adds 30 percent headroom for traffic spikes and variance: final count = $47 \times 1.3 = 61.1$, rounded up to 62.

Step 5: Verify fault tolerance. The 30 percent headroom addresses traffic variance, but production systems also need fault tolerance. With 62 GPUs, losing one leaves 61 GPUs handling 5,000 QPS. The postfailure utilization is $5,000 \text{ QPS} / (200 \text{ req/s} \times 61) = 41.0\%$.

This remains well below the 54 percent safe utilization threshold, confirming N+1 redundancy is satisfied. For stricter fault tolerance requirements, N+2 redundancy (tolerating two simultaneous failures) would require 49 GPUs under the same safe-utilization threshold, or about 64 GPUs if the 30 percent headroom must remain after two simultaneous failures.

Result: Provision 62 V100 GPUs to serve 5,000 QPS at 50 ms p99 latency with N+1 fault tolerance.

The queuing analysis explains the capacity planning approach detailed in section 13.11.3 and connects directly to the MLPerf Server scenario. Section 12.8.4.2 explains that an MLPerf Server result is valid only when the run satisfies its percentile-latency constraint; otherwise, the target QPS must be reduced and the run repeated.

13.5.6 Tail-tolerant techniques

Eliminating all sources of latency variability is often impractical. Production systems instead employ techniques that tolerate variability while still meeting SLOs (Dean and Barroso 2013; Dean 2012). The useful organization is by failure mode: a straggler replica calls for a race, fan-out calls for early detection, overload calls for admission control or graceful degradation, and retry amplification calls for coordinated shedding.

For a straggler replica, the system can race the slow path. Under hedging, when a request has not completed within the expected time, the system sends a duplicate request to another server.¹³ The client uses whichever response arrives first and cancels the other. For ML serving, this means maintaining multiple model replicas and routing slow requests to alternative replicas. With a stable latency distribution, a threshold set at the historical 95th percentile triggers duplicates for about 5 percent of requests; correlated slowdowns can raise that fraction and make hedging amplify overload instead of reducing the tail.

Ordinary launched inference kernels are not cheaply interrupted mid-execution. When a hedged request completes, the duplicate must be cancelled, but if inference has already begun on the GPU,

¹³ | **Hedging:** The term is borrowed from finance, where an offsetting bet reduces risk; here, the redundant request is a bet against a slow server. This is not free: for ML systems, the losing hedged request may still occupy accelerator time if inference has already launched, because ordinary GPU kernels are not cheaply cancelled mid-execution. Thus, a hedging policy must budget duplicate work as well as the latency benefit.

cancellation approaches include checking a cancellation flag before launching inference, accepting wasted compute for the in-flight kernel, or using request prioritization to deprioritize the duplicate. Because hedging typically applies only to a small tail of requests, the overhead from occasional wasted compute remains manageable when the trigger threshold is tuned conservatively.

Tied requests make the same race more aggressive by placing the request in multiple server queues simultaneously, with coordination that cancels the other copies once one server begins processing. This avoids waiting to detect a slow response before hedging and can reduce queuing delay when multiple ready replicas are available. It also creates duplicate queue entries, so the coordination mechanism must cancel losing copies before they consume scarce accelerator time.

Fan-out systems need a different intervention point because one slow backend can stall the entire distributed request. Canary requests first send the request to a small subset of one to two servers.¹⁴ If these return within expected time, the system sends to the remainder. If the canary is slow, the system can retry elsewhere or use cached results before committing to the full fan-out. The technique turns a potential tail-latency amplification problem into an early warning signal.

When the problem is cluster overload rather than an isolated straggler, racing backends only exacerbates queue depth by injecting duplicate work. The system must instead protect the latency of admitted requests through graceful degradation or admission control. **Graceful degradation** returns lower-fidelity results to preserve throughput: classification pipelines fall back to cached approximations, generative models truncate decoding lengths, and model ensembles evaluate only a subset of members. Dynamically reducing active ensemble members shortens service time (T_{svc}), which increases service rate μ and lowers utilization $\rho_{\text{serv}} = \lambda_{\text{arr}}/\mu$ back below the queuing knee (figure 13.1), trading a controlled accuracy reduction for SLO survival instead of an uncontrolled latency collapse. **Admission control** enforces an even stricter boundary: when the ingress queue exceeds a designated threshold, the server immediately rejects excess queries with HTTP 503 status codes rather than admitting work doomed to violate its latency deadline.

Setting this rejection threshold requires balancing accelerator occupancy against queuing delay. A practical starting heuristic is to bound queue depth to two or three service quanta per worker (for four workers, an ingress queue limit of 8 to 12 requests). Adaptive admission control dynamically tightens this limit when observed p99 latency approaches the SLO and expands it when headroom permits. While compiled, fixed-shape inference yields relatively narrow service-time distributions, variable input lengths, dynamic batch formation, and shared memory bus contention introduce variance that necessitates continuous empirical calibration rather than rigid static limits.

A cascading failure occurs when all replicas experience simultaneous overload. If upstream load balancers retry rejected requests at other replicas that are also overloaded, retry traffic amplifies the overload. Coordinated load shedding addresses this by sharing load information across replicas, enabling system-wide decisions about which requests to accept. When global load exceeds capacity, replicas collectively reject the same fraction of requests rather than each rejecting independently and triggering retries.

These techniques become essential at scale when fan-out amplification makes individual server tail latency visible to users. Single-machine serving systems can implement hedged and tied requests across GPU streams or model replicas. While this queuing analysis assumes first-in-first-out (FIFO) processing, production serving frameworks often implement priority disciplines—such as shortest-job-first or deadline-aware scheduling—to mitigate tail latency for heterogeneous request distributions (Harchol-Balter 2013).



Checkpoint 13.2: Queuing and SLO headroom

Latency SLOs are not enforced by *fast inference* alone; they are enforced by *headroom*.

- ☐ **Little's Law:** can you use $Q_{\text{req}} = \lambda_{\text{arr}} T_{\text{lat}}$ to explain why rising queue depth implies rising latency even if per-request compute time is unchanged?
- ☐ **Utilization cliff:** can you explain why latency grows nonlinearly as utilization ρ_{serv} approaches one, and why production systems target a conservative ρ_{serv} rather than “100 percent busy”?

¹⁴ | **Canary:** Named for the coal mine practice (early 1900s–1980s) of using birds whose high metabolic rate made them sensitive to toxic gases before concentrations became lethal to humans. In ML serving, canary requests serve the same early-warning function for fan-out queries: by testing 1–2 backends before committing to the full fan-out, the system detects slow or failing replicas before a single straggler stalls the entire distributed inference request—a critical protection when fan-out width means tail latency grows with the maximum of all backend response times.

- ❑ **Wait vs. compute:** given an end-to-end latency budget, can you separate $L_{\text{lat,compute}}$ from $L_{\text{lat,wait}}$ and explain which one queuing theory primarily predicts?
- ❑ **Capacity planning:** can you explain why a throughput number is only “real” if requests still meet the percentile latency SLO under load?
- ❑ **Headroom estimate:** to hold p99 under 50 ms at 2,000 requests per second, estimate the average in-flight request count $Q_{\text{req}} = \lambda_{\text{arr}} T_{\text{lat}}$ implies, and how fast that margin shrinks as ρ_{serv} passes 0.7.

Queuing theory and tail-tolerance policies govern the steady-state processing of requests. They presuppose, however, that serving instances are already initialized and that incoming data matches offline training assumptions—preconditions that frequently fail during cold starts, auto-scaling surges, and silent feature corruption.

13.6 Model Lifecycle Management

Steady-state queuing optimizations offer little protection when a serving cluster fails to initialize properly or emits degraded predictions. In a representative deployment, a newly provisioned replica stalls for 35 seconds compiling a TensorRT engine, while a subtle PIL-to-OpenCV preprocessing discrepancy silently bleeds five percentage points of classification accuracy. Mitigating these lifecycle failures requires systematic control across two operational phases: training-serving parity and cold-start initialization.

13.6.1 Training-serving skew

A model that performed well during validation may silently degrade when deployed. This phenomenon, known as training-serving skew, represents one of the most subtle failure modes in production ML because it is invisible to latency monitoring and exception tracking (Sculley et al. 2015; Baylor et al. 2017).

Definition 13.3: Training-serving skew

Training-serving skew is the distributional divergence between the training and inference environments caused by inconsistent logic or state.

1. **Significance:** It violates the consistency imperative and can cause silent accuracy degradation when the transformation functions differ ($f_{\text{train}}(x) \neq f_{\text{serve}}(x)$).
2. **Distinction:** Unlike data drift (which is an external shift in the environment), training-serving skew is an internal failure of the engineering stack.
3. **Common pitfall:** A frequent misconception is that skew is “found” by looking for errors. In reality, it is invisible to exceptions: the system runs perfectly and the latency is low, but the predictions are statistically wrong.

Section 14.4.1.2 develops skew diagnosis, monitoring, and organizational prevention. Its serving-specific manifestation is **preprocessing divergence**: the real-time inference pipeline processes raw data differently than the batch training pipeline, a common failure mode when training uses Python/Pandas while serving uses C++/Java or optimized inference servers. Unlike data drift, this divergence is internally introduced and often preventable through shared transformations, parity tests, and careful engineering.

Example 13.5: ResNet-50: Image preprocessing skew

Scenario: ResNet-50 vision serving models experience silent prediction accuracy degradation between offline training and online serving pipelines.

Diagnosis: The serving pipeline implemented C++/OpenCV preprocessing (cv2.INTER_LINEAR, BGR order) while training used Python/PIL (PIL.BILINEAR, RGB order, different normalization constants).

Systems lesson: Preprocessing divergence causes silent training-serving skew without raising runtime exceptions. Standardizing data transformation pipelines across training and serving environments prevents input distribution corruption.

15 | Warmup: Borrowed from just-in-time (JIT) compilation, where initial executions compile hot paths into optimized machine code; for ML serving, warmup inferences trigger CUDA kernel compilation, cuDNN algorithm auto-tuning, and memory pool allocation that frameworks defer until first use. Without warmup, the first live request absorbs all of this setup and can be orders of magnitude slower than steady state. During autoscaling events, this means new replicas can violate SLOs for their first several seconds of traffic.

16 | CUDA (compute unified device architecture): NVIDIA's parallel computing platform (Nickolls et al. 2008), named for its goal of unifying diverse GPU shader models into a single general-purpose architecture. Before CUDA, GPU programming required disguising computations as graphics operations. The CUDA context—the data structure tracking memory allocations, loaded kernels, and device state—is the runtime's per-process gateway to GPU resources; in serverless or rapidly scaling serving systems, context creation and lazy module loading can become visible parts of cold start latency (NVIDIA 2026a).

17 | CUDA MPS (multi-process service): MPS creates a control daemon that allows CUDA work from different processes to overlap on the GPU, which can improve utilization and reduce context-switching overhead when processes individually underuse the accelerator (NVIDIA 2026d). For multi-model serving, MPS can help replicas share GPU streaming multiprocessors efficiently. The trade-off is fault isolation: clients share MPS-managed GPU state, so hardware partitioning with Multi-Instance GPU (MIG) provides stronger isolation at the cost of fixed partition granularity.

13.6.2 Cold start and initialization dynamics

Even with mathematical parity guaranteed between training and serving pipelines, a newly provisioned serving instance cannot process queries until its state is instantiated across hardware memory hierarchies (Romero et al. 2021). This initialization latency, known as cold start, dictates system responsiveness during deployments, autoscaling events, and failure recovery.



Definition 13.4: Cold start

Cold start is the initialization latency incurred when instantiating a new model replica.

- 1. Significance:** It represents the fixed cost of state hydration (loading weights, compiling graphs), which can take seconds or minutes, effectively blocking the system's ability to scale elastically in response to traffic bursts.
- 2. Distinction:** Unlike inference latency (L_{lat}), which is a per-request cost, cold start is an initialization cost paid when a replica or model instance is created, including deployment, scale-out, recovery, and on-demand model loading.
- 3. Common pitfall:** A frequent misconception is that cold start is “just loading weights.” In reality, it includes graph compilation and memory allocation, which can often take longer than the bandwidth-limited data transfer itself (D_{vol}/BW).

Total cold-start latency compounds across distinct physical operations along the critical initialization path. First, weight loading transfers serialized parameters from persistent storage across the network or host I/O subsystem into host memory, and ultimately across the PCIe bus into GPU HBM. Graph compilation lowers the framework computational graph into device-specific machine code, executing operator fusion and auto-tuning matrix multiplication kernels for the specific accelerator architecture. Memory allocation reserves device memory arenas for activation scratchpads and intermediate buffers. Finally, warmup¹⁵ execution issues synthetic inferences to populate kernel caches, prime framework memory allocators, and force lazy initialization before live traffic arrives.

Establishing a CUDA context¹⁶ represents the first fixed cost on the device initialization timeline. Before launching any GPU kernel or allocating device memory, the runtime must instantiate a driver-level context: an internal control structure that manages virtual address spaces, loaded kernel binaries, and hardware event queues. Creating this context requires synchronous communication with the kernel-mode driver and allocates tens to hundreds of megabytes of device memory for bookkeeping. The 0.3–0.5 s range in table 13.8 reflects a typical data center environment; while CUDA lazy loading defers kernel module loading to shorten this initial pause, it merely defers that latency penalty to the first executed inference (NVIDIA 2026a).

In multi-process serving deployments where multiple worker processes share a single accelerator, replicating individual CUDA contexts wastes both memory capacity and initialization time. CUDA Multi-Process Service (MPS)¹⁷ eliminates this redundancy by routing client processes through a shared control daemon, merging their operations into a single hardware context. This allows kernels from distinct processes to execute concurrently on the streaming multiprocessors without time-slicing context-switch overhead (NVIDIA 2026d). The systems trade-off is diminished fault isolation: because client processes share address spaces within the MPS server, an unhandled fatal exception in one replica can crash the shared server and terminate all colocated instances.

Example 13.6: ResNet-50: Cold start timeline

Table 13.8 traces the per-phase duration of a ResNet-50 cold start:

Table 13.8: ResNet-50 cold start timeline: Illustrative per-phase durations for weight loading, CUDA context creation, TensorRT compilation, and warmup, with totals for the optimized local case and the first-deploy cloud case showing where the dominant cost lives.

Phase	Duration	Notes
Weight loading (SSD)	0.5 s	98 MB FP32 weights from local storage
Weight loading (S3)	3–5 s	Network latency dominates for cloud storage
CUDA context	0.3–0.5 s	GPU driver initialization and memory setup
TensorRT compilation	15–30 s	Converts PyTorch model to optimized engine
Warmup (10 inferences)	0.2 s	Triggers remaining lazy initialization
Runtime overhead	0.4 s	Process startup, framework hooks, and runtime setup
Total (local, optimized)	~1.5 s	With precompiled TensorRT engine, warm container
Total (cloud, first deploy)	~35 s	Including compilation from cold state

Systems insight: Precompiling models and storing the optimized engine eliminates the 30-second compilation phase on subsequent deployments.

Without warmup, the first live request absorbs deferred JIT compilation, driver memory allocation, and kernel launch overhead. A request with a steady-state execution time of 5 ms can stall for 500 ms during an unmitigated cold start, violating strict SLOs. Controlling this initialization overhead requires systematic selection among storage, memory, and runtime loading strategies.

13.6.3 Loading strategies

Different loading strategies trade off cold start duration against serving performance and memory efficiency. The simplest approach, full loading, reads the entire model into memory before serving begins. This maximizes inference speed since all weights are immediately available, but extends cold start duration and limits model size to available memory. The approach is appropriate when cold start latency is acceptable and models comfortably fit in memory.

Memory mapping offers an alternative by mapping model files into the process address space and faulting pages in on access. It can reduce eager copying and shorten apparent startup, but a dense model will eventually touch nearly all of its weights, so deferred page faults can move loading latency into early requests. Selective benefit is greater for sparsely accessed components or model repositories that load subsets on demand; latency-sensitive services still prefault or warm the pages needed on the request path.

A third strategy, lazy initialization, defers compilation and allocation until first use. This minimizes startup time but shifts latency to the first request. Production systems often combine lazy initialization with synthetic warmup requests to trigger initialization before real traffic arrives.

13.6.4 Model caching infrastructure

Production systems cache model weights at the infrastructure level to reduce cold start for common deployment scenarios. One approach, container image embedding, bundles model weights directly in the container image. This produces a single deployment artifact and eliminates network fetches at startup, but creates large images (often 10–50 GB) that slow container pulls and consume registry storage. This approach works best for models that rarely update.

For organizations with many models and frequent updates, a shared filesystem (EFS, GCS FUSE) containing versioned model weights provides a more flexible alternative. Multiple replicas can read the same artifacts without baking each version into a container. Updates still require an explicit, atomic version switch and rollout; allowing files to change underneath live replicas would undermine reproducibility. The trade-off is that network latency affects cold start and filesystem availability becomes a critical dependency.

When cold start latency is critical for high-traffic models, node-local SSD caching prepopulates local SSDs on inference nodes with frequently-used models. This approach provides fast loading from Non-Volatile Memory Express (NVMe) drives at 500 MB/s or more without network dependency, but requires cache management to handle model updates and capacity limits. Infrequent updates favor container embedding, frequent updates favor shared filesystems, and performance-critical deployments benefit from local caching with background refresh.

13.6.5 Multi-model serving

Production systems often serve multiple models from a single machine, whether different model versions for A/B testing, ensemble components, or entirely different models sharing infrastructure. GPU memory becomes the limiting resource, requiring careful management strategies.

Three strategies address multi-model memory management. Time-multiplexing loads one model at a time and swaps based on request routing; this approach is simple but introduces swap latency. Memory sharing partitions GPU memory among models, limiting concurrent execution count but enabling more models to remain resident. Model virtualization, as implemented by frameworks like Triton, separates model lifecycle from application code through model repository and control APIs for loading, unloading, and versioning models (NVIDIA 2024d, 2026c, 2026b). The choice depends on request patterns: if models receive traffic evenly, concurrent loading works; if traffic is bursty and model-specific, time-multiplexing with explicit preloading reduces average latency while maximizing GPU utilization.

13.6.5.1 Multi-stream execution

When multiple models or multiple instances of the same model must run concurrently on a single GPU, the hardware must partition resources between them. NVIDIA's Multi-Instance GPU¹⁸ technology enables hardware-level isolation, dividing an A100 into up to seven independent GPU instances, each with dedicated memory and compute resources (NVIDIA 2026e). MIG is available on A100, A30 (up to four instances), H100, H200, and newer data center GPUs. For older GPUs such as V100 or T4, CUDA stream scheduling provides time-multiplexed sharing without hardware isolation. The choice depends on whether consistent latency with MIG or maximum utilization with shared streams is the priority.

13.6.5.2 Model swapping and host memory

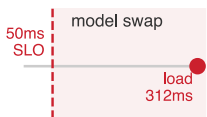
When the aggregate size of all models exceeds GPU memory capacity, the serving system must swap models between host dynamic random-access memory (DRAM) and device memory (VRAM) on demand. This introduces a new latency component determined by the PCIe bus bandwidth.

For a 10 GB model on PCIe Gen4 x16 (32 GB/s theoretical bandwidth), loading takes at least 312.5 ms before deserialization, graph setup, or warmup. As visualized in the margin, this raw transfer latency dwarfs tight service-level objectives—such as a 50 ms deadline—by more than 6×, rendering on-demand model swapping unviable on the interactive request path.

To accelerate this transfer path, systems use pinned memory (page-locked host RAM). GPUDirect DMA engines require physical memory pages to remain fixed at invariant physical addresses throughout a transfer. Standard host memory allocated via standard heap allocators is pageable: the operating system virtual memory manager may relocate physical pages or swap them to backing storage at any time. Consequently, transferring data from a pageable buffer forces the CUDA driver to perform a two-stage copy: the CPU first synchronously copies the data into an intermediate pinned staging buffer, and the DMA engine then transfers the staging buffer across PCIe to GPU memory. Allocating model weights directly in page-locked host memory via `cudaHostAlloc` eliminates this CPU staging copy, allowing the GPU's DMA controller to stream weights across the PCIe bus at full link bandwidth. The physical trade-off is host memory exhaustion: pinned memory cannot be paged out or reclaimed by the OS under memory pressure, permanently locking physical RAM away from other processes. High-throughput serving systems therefore pin active model weights and I/O ring buffers while leaving standby model checkpoints in pageable memory.

Once a model is loaded, warmed up, and producing predictions consistent with training, request grouping becomes the next optimization lever. Dynamic batching directly governs the balance between hardware efficiency and tail latency in queuing formulations.

18 | MIG (multi-instance GPU): Introduced with NVIDIA's A100 (NVIDIA Corporation 2020a) and documented across supported data-center GPUs (NVIDIA 2026e), MIG partitions a single physical GPU into independent instances, each with dedicated compute and memory resources; unlike software sharing (MPS or time-slicing), MIG provides hardware-level isolation between partitions. The trade-off is granularity—partitions must follow fixed profiles, so resources cannot be divided arbitrarily. For multi-model serving, MIG reduces noisy-neighbor risk on shared hardware, while per-model SLO guarantees still depend on scheduler policy, load, and the selected partition profile.



Model loading lands far outside a tight serving SLO.

13.7 Throughput Optimization

Consider a representative ResNet-50 classifier on a V100 GPU operating at batch size one. Each forward pass loads all model parameters from HBM to process a single input, yielding low arithmetic intensity while kernel launch overheads dominate execution. The GPU achieves only 15 percent hardware utilization and 200 images per second. Processing 32 images at once reaches 95 percent utilization and 1,280 images per second—a $6.4\times$ throughput improvement on identical hardware—because the loaded weights are reused across inputs in on-chip memory, amortizing both memory bandwidth demands and kernel launch overheads. **Batching**¹⁹ is the primary lever for improving serving economics, but its operational constraints differ sharply between training and serving (Crankshaw et al. 2017). *Training* batches maximize throughput by grouping hundreds or thousands of samples together with no constraint on individual sample latency. *Serving* batches must balance accelerator saturation against individual request deadlines, ensuring that no early-arriving input waits excessively for a batch to form. An inference server resolves this tension through **dynamic batching**, adjusting batch size at runtime to match incoming request arrivals.



Definition 13.5: Dynamic batching

Dynamic batching is the ML serving optimization that trades latency for throughput under stochastic arrival patterns.

1. **Significance:** By buffering requests into a batching window, the scheduler amortizes fixed overheads (L_{lat}) across multiple inputs, pushing the system away from the memory-bound regime (BW) toward the compute-bound regime (R_{peak}).
2. **Distinction:** Unlike fixed-size serving batches, dynamic batching adjusts the realized batch size at inference time according to arriving traffic, a timeout, and a maximum-batch limit.
3. **Common pitfall:** A frequent misconception is that batching “always helps.” In reality, there is a latency-throughput Pareto frontier: if the batching window is too large, the increased queuing delay may violate the system’s SLO before the throughput gains are realized.

¹⁹ | **Batch:** From Old French *bache* (a quantity baked at one time), entering computing in the 1950s for jobs processed together without human interaction. The ML serving usage preserves the original trade-off: grouping requests amortizes fixed costs (kernel launch, weight movement) across multiple inputs, but an online request may wait for a batch to form. Training can use batches of hundreds or thousands when memory permits; serving batch sizes range from one to much larger values depending on the model, arrival rate, and latency SLO.

13.7.1 Why batching helps

Modern accelerators achieve peak efficiency only at sufficient batch sizes (Shen et al. 2019). A single inference request leaves most compute units idle because GPUs are designed for parallel execution across thousands of threads. Batching amortizes fixed costs across multiple requests and enables parallel execution across the batch dimension.

Two fixed costs dominate at small batch sizes. Kernel launch overhead²⁰ is the time for the CPU to prepare and submit work to the GPU. Each layer in a neural network typically requires a separate kernel launch: the CPU must assemble kernel parameters, copy them to GPU-accessible memory, and signal the GPU to begin execution. This overhead is typically 5–20 μs per kernel, independent of batch size. ResNet-50 has approximately fifty layers, so kernel launch alone adds 250–1000 μs per inference. At batch size 1, this overhead may exceed the actual compute time; at batch size 32, the same overhead is amortized across thirty-two images. Weight loading reads model parameters from GPU memory (VRAM) to the compute units. At batch size 1, the GPU reads all weights to process one image, yielding low arithmetic intensity that starves execution units; at batch size 32, the same weight read processes thirty-two images, achieving $32\times$ higher operational intensity. Measuring batching efficiency on a concrete model quantifies how these fixed costs amortize in practice.

Larger batches improve hardware efficiency but increase per-request latency. In practice, the optimal batch size depends on both the latency SLO and the arrival rate of requests. The engineering question is quantitative: determine the largest batch size that still meets a given latency SLO. In the ResNet-50 scenario quantified in table 13.9, batch size 8 with a 5 ms batching window has worst-case user latency of about 14 ms (5 ms wait plus 9 ms inference), below a 20 ms SLO budget. That earns $4.4\times$ higher service throughput than batch-1 serving on the same hardware, provided sustained load is high enough to fill the batching window. Plotting this relationship in figure 13.6 reveals the knee

²⁰ | **Kernel (GPU):** CUDA borrowed this term from operating systems circa 2007 because GPU functions represent the computational “core” of parallel algorithms. Unlike OS kernels that run continuously, GPU kernels are discrete units of parallel work launched by the CPU. Each launch carries 5–20 μs of overhead independent of batch size—negligible for large training batches, but dominant at batch-1 serving, where a 50-layer model accumulates 250–1000 μs of pure launch overhead per inference.

where extra batching stops paying for its latency cost: throughput is already flattening while latency begins to spike, so batching beyond that point trades modest capacity gains for queuing delay.

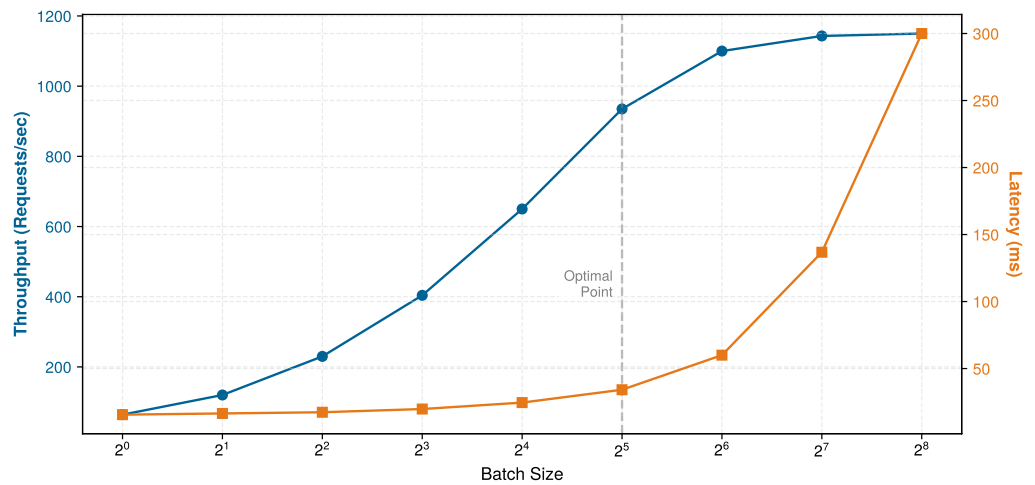


Figure 13.6: The Throughput-Latency Knee: Batch size vs. throughput (blue) and latency (orange). Throughput increases with batch size as hardware utilization improves, but eventually saturates. Latency remains relatively flat until the knee, after which it spikes due to queuing. Values are representative and depend on model/hardware.

The “knee” in figure 13.6 marks the point where the blue throughput curve begins to plateau just as the orange latency curve starts its sharp upward spike. This is an illustrative knee rather than a universal optimum: the selected operating point depends on the latency SLO, arrival process, and cost objective. The numbers are representative rather than tied to a single benchmark.

 Napkin Math 13.4: ResNet-50 batching efficiency

Table 13.9 illustrates the throughput-latency trade-off for a ResNet-50/V100 scenario across batch sizes one through thirty-two. The batch sizes, measured inference times, and GPU utilization are the scenario givens; the per-image compute and throughput columns are derived from them.
Math: For the batch-8 row, dividing batch latency by batch size yields per-image compute: $9.1 \text{ ms} \div 8 \text{ images} \approx 1.1 \text{ ms per image}$. Dividing batch size by batch latency yields throughput: $8 \text{ images} \div 9.1 \text{ ms} \approx 879 \text{ img/s}$. The same two relations apply to every derived row.

Table 13.9: ResNet-50 batching sweep: Illustrative per-image compute, throughput, and GPU utilization across batch sizes one through thirty-two on a V100. Throughput grows 6.4× from batch one to batch thirty-two as GPU utilization climbs from 15 percent to 95 percent, while pure inference time stretches from 5 ms to 25 ms.

Batch Size	Inference Time	Per-Image Compute	Throughput	GPU Util.
1	5 ms	5 ms	200 img/s	15%
4	7.2 ms	1.8 ms	556 img/s	42%
8	9.1 ms	1.1 ms	879 img/s	65%
16	14 ms	0.9 ms	1,143 img/s	85%
32	25 ms	0.8 ms	1,280 img/s	95%

The times shown are pure inference time, excluding queue wait; section 13.7.6 analyzes how user-perceived latency includes batching-window wait.

Systems insight: Batch size 32 achieves 6.4× higher throughput than batch size 1. However, user-perceived latency includes both queue wait and inference time. With a 10 ms batching window and 25 ms inference, total latency reaches 35 ms vs. 5 ms at batch size 1.

The efficiency gains from batching come at a cost: requests must wait for the batch to form. Buffering introduces a direct tension between throughput optimization (larger batches) and latency minimization (immediate processing). How serving systems balance these competing demands under fluctuating traffic governs the choice of batching strategy.

13.7.2 Static vs. dynamic batching

Static batching fixes the batch dimension at compile time and holds execution until a queue accumulates exactly B requests. In offline batch processing and model training, static shapes allow compilers to allocate static memory buffers and emit optimized compute kernels that maximize weight reuse and arithmetic intensity. In interactive serving, however, client requests arrive stochastically. Under low or fluctuating arrival rates, static batching exposes requests to unbounded queuing delay while waiting for the batch to fill. Conversely, setting a small fixed batch size to curtail latency leaves accelerator compute units underutilized and fails to amortize weight transfers from high-bandwidth memory when traffic spikes.

Dynamic batching decouples batch formation from arrival determinism by buffering incoming requests behind a dual-trigger scheduler (Olston et al. 2017; NVIDIA 2024d). Rather than waiting indefinitely for a fixed tensor shape, the serving runtime dispatches a batch to the accelerator as soon as either of two thresholds is satisfied: the queue accumulates a maximum batch size $B_{\max}$, or a timeout window T_{window} expires relative to the oldest buffered request. When traffic bursts, the queue fills to $B_{\max}$ almost instantaneously, achieving high arithmetic intensity with minimal queuing delay. When arrival rates drop, the timeout window forces the dispatch of an undersized batch ($B < B_{\max}$), bounding queuing latency at the expense of per-request compute efficiency. The timeout window T_{window} and maximum batch size $B_{\max}$ form the primary operational levers: shorter windows prioritize latency over accelerator efficiency, whereas larger windows maximize system throughput whenever request volume permits.

13.7.3 Dynamic batching latency-throughput trade-offs

Dynamic batching introduces a quantifiable tension between throughput optimization and latency constraints. Under overload, the mechanism is queue growth rather than slower inference, which enables systematic configuration decisions instead of trial-and-error tuning.

Systems Perspective 13.5: Why latency spikes under load

Little's Law ($Q_{\text{req}} = \lambda_{\text{arr}} T_{\text{lat}}$) relates long-run averages only while the queue is stable (section 13.5.1). As the arrival rate approaches the maximum service rate μ , the stable queue's

depth and latency grow rapidly. If λ_{arr} exceeds capacity for a sustained interval, no finite steady state exists: backlog accumulates until traffic falls, capacity grows, or work is rejected. This is why admission control can preserve latency during overload; scale-out or degraded service can also help when available.

Equation 13.7 decomposes the total user-perceived latency for a batched request into two components:

$$L_{\text{lat}} = L_{\text{lat,wait}} + L_{\text{lat,compute}}(B) \quad (13.7)$$

where $L_{\text{lat,wait}}$ is the time spent waiting in the batching queue (corresponding to $L_{\text{lat,queue}}$ in the overall latency budget) and $L_{\text{lat,compute}}(B)$ is the inference time for batch size B (encompassing $L_{\text{lat,infer}}$ plus portions of $L_{\text{lat,pre}}$ and $L_{\text{lat,post}}$). The batching window T_{window} bounds wait time ($L_{\text{lat,wait}} \leq T_{\text{window}}$), while batch size determines compute time by amortizing weight memory transfers and raising arithmetic intensity on accelerator execution units.

13.7.3.1 Quantitative analysis of batching

For fixed calendar windows under stationary Poisson arrivals, request arrival times are uniform within each window. A request arriving at time t waits $T_{\text{window}} - t$ for that window to close, so equation 13.8 gives an average wait of half the window:

$$E[L_{\text{lat,wait}}] = \frac{T_{\text{window}}}{2} \quad (13.8)$$

A 20 ms batching window adds 10 ms average wait (up to 20 ms for the first arrival in a window; later arrivals wait less) regardless of batch size achieved. For a 50 ms mean latency SLO with 5 ms inference, the average wait consumes 20 percent of the latency budget before any computation begins; tail SLOs must budget the full window.

13.7.3.2 Batch size distribution

For fixed calendar windows, the number of requests collected during T_{window} follows a Poisson distribution with mean $\lambda_{\text{arr}} T_{\text{window}}$. Equation 13.9 formalizes this relationship:

$$\Pr(\text{batch size} = k) = \frac{(\lambda_{\text{arr}} T_{\text{window}})^k e^{-\lambda_{\text{arr}} T_{\text{window}}}}{k!} \quad (13.9)$$

Here k is a nonnegative integer count of arrivals in the window.

Table 13.10 quantifies this variability, showing how batch size fluctuates for different traffic levels with a fixed 10 ms window:

Table 13.10: Batch Size Variability: At low traffic, fixed calendar windows frequently contain zero requests, so no GPU batch is launched. At moderate traffic, batch sizes fluctuate significantly around the mean. High traffic provides more stable batching, and the probability of batches reaching at least twice the mean size decreases as traffic grows (from 39 percent at 50 QPS to 0.3 percent at 1000 QPS), reflecting the law of large numbers.

Arrival Rate	Mean Batch	Std Dev	$\Pr(\text{batch} = 0)$	$\Pr(\text{batch} \geq 2 \times \text{mean})$
50 QPS	0.5	0.7	61%	39%
200 QPS	2	1.4	14%	14%
500 QPS	5	2.2	0.7%	3%
1000 QPS	10	3.2	0.005%	0.3%

13.7.3.3 Throughput maximization strategy

Throughput optimization requires separating per-request latency from saturated service capacity. A request waits for batch formation, then pays the service time of the formed batch. Under sustained load, however, batch formation can overlap with the previous batch's execution, so capacity is governed by the ready-batch service time:

$$\mu_{\text{eff}}(B) \approx \frac{B}{T_{\text{svc}}(B)}, \quad \text{throughput} = \min(\lambda_{\text{arr}}, \mu_{\text{eff}}(B)) \quad (13.10)$$

In equation 13.10, λ_{arr} is the offered arrival rate and $\mu_{\text{eff}}(B)$ is the saturated service capacity for batch size B . The numerator increases linearly with batch size while service time often increases sublinearly over a useful range because larger batches amortize fixed kernel-launch overheads and DRAM weight transfers, shifting execution from memory-bandwidth-bound matrix-vector operations toward compute-bound matrix-matrix multiplication. The batching window still appears in request latency and in low-traffic regimes, where the expected batch size is limited by arrivals during the window, roughly $\lambda_{\text{arr}} T_{\text{window}}$.

For ResNet-50 on a V100 GPU, service time approximately scales as $T_{\text{svc}}(B) = 5 \text{ ms} + 0.6B$ (5 ms fixed overhead plus 0.6 ms per image in the batch). This linear approximation captures the dominant trend; actual service times may deviate slightly due to memory hierarchy effects. With a $T_{\text{window}} = 10 \text{ ms}$ batching window, table 13.11 extends the pure-inference sweep of table 13.9: latency includes the full-window wait bound, while saturated throughput uses service time alone:

Table 13.11: Batching Throughput Analysis: Scenario analysis for ResNet-50 throughput on V100 with 10 ms batching window. Throughput increases 7.4× from batch size one to 32 (179 img/s to 1322 img/s), but total latency more than doubles (15.6 ms to 34.2 ms). The optimal configuration depends on whether the latency SLO or throughput target is the binding constraint.

Batch Size	Service Time	Total Latency	Throughput	Efficiency
1	5.6 ms	15.6 ms	179 img/s	Low
4	7.4 ms	17.4 ms	541 img/s	Moderate
8	9.8 ms	19.8 ms	816 img/s	Good
16	14.6 ms	24.6 ms	1096 img/s	High
32	24.2 ms	34.2 ms	1322 img/s	Maximum

The throughput gains in table 13.11 trace directly back to the fixed-overhead term in the iron law established in section 8.2, where batching amortizes work across requests.

Example 13.7: The iron law of batching efficiency

Context: In serving, batching improves throughput by amortizing fixed per-batch scheduling and kernel-launch work; it can also amortize weight traffic across requests, while queue wait remains a separate latency cost. Holding the data-movement term outside this illustrative two-term comparison, equation 13.11 isolates compute and fixed overhead:

$$T = \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}} \quad (13.11)$$

Analysis:

- **Case 1 (batch 1):** Overhead (5 ms) $\gg$ Compute (0.6 ms). Efficiency $\approx$ 10 percent. The GPU is mostly waiting.
- **Case 2 (batch 32):** Overhead (5 ms) $\ll$ Compute (19.2 ms). Efficiency $\approx$ 79 percent. The GPU is crunching numbers.

Systems insight: Increase batch size until fixed overhead becomes negligible (< 10 percent of total time) or the latency SLO blocks further waiting. Beyond this point, additional batching yields minimal throughput but imposes a linear queueing penalty.

These three results compose into one working model of dynamic batching: the window sets the average wait at half its length, Poisson arrivals make the realized batch size fluctuate around $\lambda_{\text{arr}} T_{\text{window}}$, and the saturated service capacity $\mu_{\text{eff}}(B)$ climbs with batch size until fixed overhead is amortized away. None of these relationships yet enforces the latency SLO. Enforcing that constraint requires working backward from a hard percentile budget to find the largest batch the window may safely form.

13.7.3.4 Latency-constrained optimization

When latency SLOs provide the binding constraint, the optimization problem becomes finding the maximum batch size that meets the SLO. For a latency target $L_{\text{lat,target}}$ and average wait time $T_{\text{window}}/2$, equation 13.12 defines the maximum allowable batch size using a first-order *average* latency approximation:

$$B_{\text{max}} = \max \left\{ B : \frac{T_{\text{window}}}{2} + T_{\text{svc}}(B) \leq L_{\text{lat,target}} \right\} \quad (13.12)$$

In this illustrative ResNet-50 scenario with a 50 ms p95 latency SLO, comparing a conservative batching window against an aggressive one shows how the assumed configurations trade wait time, inference budget, and saturated capacity. Table 13.12 lays the two configurations side by side.

Table 13.12: Illustrative batching window trade-off: How two assumed batching configurations trade average wait, inference budget, batch-size cap, and saturated capacity.

Metric	Conservative ($T_{\text{window}} = 5 \text{ ms}$)	Aggressive ($T_{\text{window}} = 25 \text{ ms}$)
Average wait	2.5 ms (max wait = 5 ms for the first request in a window)	12.5 ms
Latency budget for inference	47.5 ms (mean-latency planning; tail SLOs should budget the full window)	37.5 ms
Batch size cap	32 images	48
Assumed saturated capacity	~1,140 img/s	~1,280 img/s

The aggressive configuration assumes 12.3 percent higher saturated capacity but increases average batching wait by 10 ms and the maximum window-wait contribution by 20 ms. The latter is not a p99-latency estimate.

13.7.3.5 SLO violation analysis

Batch size variability can cause SLO violations even when mean latency appears safe. A request in a fixed window sees itself plus the other arrivals, so $B_{\text{req}} = 1 + \text{Poisson}(\lambda_{\text{arr}} T_{\text{window}})$. Combining the full window with the request-seen p99 batch size gives the conservative envelope in equation 13.13:

$$L_{\text{lat,envlope}} = T_{\text{window}} + T_{\text{svc}}(B_{\text{req,p99}}) \quad (13.13)$$

For $\lambda_{\text{arr}} = 500 \text{ QPS}$ and $T_{\text{window}} = 10 \text{ ms}$, the mean request-seen batch size is 6 and its p99 is 12. The mean latency is 13.6 ms; the conservative envelope is 22.2 ms.

The envelope is 1.63× the mean. It is not the request-latency p99 because full-window wait and p99 batch size need not occur for the same request.

🔍 Systems Perspective 13.6: The latency-throughput trade-off

A single “inference speed” number is undefined until the batch size is named, because batch size selects which regime, and which bottleneck, the system operates in.

- **Batch-1 regime:** Latency-oriented. Launch overhead or memory traffic often limits the request path. This regime governs real-time interaction such as typing helpers and robotics.
- **Batch-N regime:** Throughput-oriented. Larger batches amortize overhead and weight traffic and may shift the bottleneck toward compute. This regime governs offline processing and high-traffic services.

The two regimes optimize opposite quantities, so a model that is “fast” at batch 1 may be far from peak throughput, and vice versa. Any latency or throughput figure must therefore specify whether it was measured at single-stream latency (batch 1) or maximum throughput (batch N).

13.7.3.6 Adaptive batching windows

The same batch-size dependence drives how the serving system shapes its batches in the first place. Fixed batching windows waste latency budget during high traffic when large batches form quickly. Listing 13.2 demonstrates how adaptive strategies adjust the window based on queue depth.

In an illustrative scenario with traffic varying between 200–1000 QPS, a fixed 10 ms window produces 15 ms average latency at 650 img/s, while the adaptive heuristic produces 11 ms at 680 img/s. The interplay between window size and batch limits creates a space of possible configurations, each representing a different balance between throughput and latency.

Listing 13.2: Adaptive Batching Window: Heuristically adjusts batch timeout based on queue depth, arrival rate, and the remaining latency budget.

```
def adaptive_batching_window(
    queue_depth, arrival_rate, slo_ms, service_ms, fixed_overhead_ms
):
    """Compute a heuristic batching window.

    Based on current system state.
    """
    target_batch_size = 16 # Deployment-tuned target batch

    # Fast path: batch ready, close immediately to minimize latency
    if queue_depth >= target_batch_size:
        return 0

    # Compute maximum allowable wait from the remaining p99 budget.
    max_wait_ms = max(0, slo_ms - service_ms - fixed_overhead_ms)

    # Estimate time to accumulate target batch at current arrival
    # rate.
    # arrival_rate is requests/second, so convert seconds to
    # milliseconds.
    if arrival_rate > 0:
        requests_needed = target_batch_size - queue_depth
        estimated_wait_ms = requests_needed / arrival_rate * 1000.0
        # Return minimum of estimated wait and SLO-constrained maximum
        return min(estimated_wait_ms, max_wait_ms)

    return max_wait_ms # Low traffic: use remaining budget to accumulate batch
```

The nondominated batching configurations form a Pareto frontier where improving throughput requires accepting higher latency. Table 13.13 shows five illustrative configurations:

Table 13.13: Illustrative Batching Pareto Frontier: The five assumed configurations trade tail latency for throughput. From the first to the last, assumed throughput rises by 52 percent and assumed p99 latency rises to 5.4× its initial value.

Window (ms)	Max Batch	Avg Latency	p99 Latency	Throughput	Configuration
2	16	8 ms	18 ms	890 img/s	Ultra-low latency
5	32	10 ms	22 ms	1,140 img/s	Balanced
10	32	15 ms	35 ms	1,240 img/s	Moderate latency
20	64	23 ms	52 ms	1,310 img/s	Throughput-optimized
50	128	38 ms	98 ms	1,350 img/s	Maximum throughput

13.7.3.7 Practical configuration guidelines

The Pareto frontier in table 13.13 illustrates why configuration must work backward from the latency budget: past the knee, widening the window buys diminishing throughput for sharply rising tail latency. In the worked scenario, reserving 30 percent of the SLO for batching wait gives $T_{\max} = 0.3 \times L_{\text{lat,SLO}}$; another workload may allocate a different fraction after accounting for network, preprocessing, inference, and postprocessing. The traffic estimate should include peaks and burst

duration rather than rely only on the mean. GPU memory supplies a separate ceiling on batch size through activations and runtime workspace. Finally, monitoring the realized batch-size and wait-time distributions reveals whether the assumed arrival model holds and whether a fixed or adaptive window is appropriate.

For an illustrative ResNet-50 serving system with a 50 ms SLO and 500 QPS traffic, the calculation turns the SLO and arrival-rate assumptions into two deployable knobs: the batching window and maximum batch size. Table 13.14 summarizes the resulting configuration and modeled operating point.

Table 13.14: Practical Batching Configuration: A 30 percent allocation sets a 15 ms maximum window; this illustrative scenario selects a 12 ms window below that bound and assumes a batch-32 cap. The reported latency is a conservative envelope, not the request-latency p99.

Quantity	Value	Engineering role
Latency budget for batching	15 ms	Portion of the SLO available for queueing delay.
Maximum window	15 ms	Upper bound implied by the latency budget.
Expected request-seen batch	7	Average batch seen by an arriving request.
Request-seen p99 batch	13	Tail batch size under Poisson arrivals.
Assumed memory-limited batch	32	Scenario cap for accelerator memory.
Selected configuration	$T_{\text{window}} = 12 \text{ ms}, B_{\text{max}} = 32$	Practical knob setting for deployment.
Conservative tail envelope	24.8 ms	Full window plus request-seen p99 batch service time.
Illustrative saturated capacity	1,176.9 img/s	Estimate using the assumed 0.89 efficiency factor.
Served throughput	500 img/s	Arrival-limited load handled by the configuration.

13.7.4 Continuous batching

Autoregressive models, such as large language models, generate outputs token by token. Because each new token depends on the accumulated sequence history, generation proceeds sequentially. Standard dynamic batching assumes uniform output lengths and commits to a fixed batch for an entire generation cycle. When sequence lengths vary, this static commitment degrades accelerator efficiency. If one sequence in a batch of eight finishes after ten tokens while others require 100 tokens, 90 percent of the compute and memory bandwidth allocated to that sequence slot is wasted on padding tokens or masked dummy operations while awaiting the slowest sequence (Yu et al. 2022).

Continuous batching²¹ (also called iteration-level batching) eliminates this idle-slot tax by rescheduling batch composition at each decoding iteration (Yu et al. 2022; Kwon et al. 2023). Rather than committing to a static batch that persists across hundreds of forward passes, the serving engine re-evaluates batch membership at every token-generation step. When a sequence emits its end-of-sequence token, the scheduler immediately reclaims that execution slot, admitting an arriving request into the batch for the subsequent forward pass. Active sequences continue decoding without interruption, maintaining high accelerator utilization across variable-length workloads.

Serving runtimes such as vLLM²² and TensorRT-LLM improve throughput via two mechanisms: reclaiming wasted compute from completed sequences and shortening queue wait times for new arrivals (Kwon et al. 2023; NVIDIA 2026g). However, admitting new requests dynamically creates a systems collision between prompt prefill and token decoding. Compute-heavy prompt prefill across hundreds of tokens exhibits high arithmetic intensity and monopolizes GPU streaming multiprocessors for tens of milliseconds. In contrast, token decoding is memory-bandwidth bound and executes one token at a time. When an engine executes a prefill in the same iteration as active decodes, the ongoing decode streams stall, causing severe spikes in inter-token latency (ITL). Sarathi-Serve resolves this interference through chunked prefill and stall-free batching (Agrawal et al. 2024). By splitting long prompts into fixed-size chunks (e.g., 512 tokens) and co-scheduling a single chunk alongside active decode requests per step, the scheduler enforces bounded iteration execution times and preserves steady token generation cadence.

Continuous batching shifts the primary serving bottleneck to memory management. Because transformer generation appends attention states at every step, the key-value (KV) cache for each active request grows dynamically in accelerator HBM. When sequences of varying lengths share memory, contiguous allocation produces severe external fragmentation. If a completed 100-token sequence

21 | Continuous batching: Also called “iteration-level batching” (Yu et al. 2022) and, in NVIDIA TensorRT-LLM, “in-flight batching” (NVIDIA 2026g); the distinction is scheduling granularity. Traditional batching commits to a fixed batch for an entire generation sequence (potentially hundreds of iterations), while continuous batching reschedules at every token-generation step—analogous to preemptive OS process scheduling vs. run-to-completion. This finer granularity reduces the waste from variable-length sequences, where a batch slot occupied by a completed sequence sits idle until all other sequences finish.

22 | vLLM (virtual large language model (LLM)): An open-source serving system that enables continuous batching via its PagedAttention algorithm (Kwon et al. 2023). Inspired by OS virtual memory, this technique reduces the severe KV-cache fragmentation and reservation waste that constrains static batching. By keeping KV-cache waste low, vLLM can serve larger effective batches on the same hardware.

releases its cache, an arriving 150-token request cannot occupy that freed memory because it requires a larger contiguous address range. Over time, physical memory fractures into small, noncontiguous gaps, preventing new sequences from launching even when total unallocated capacity appears sufficient. This memory fragmentation can waste 40 to 50 percent of available memory in naive implementations, severely limiting the concurrent batch size that determines serving throughput.

13.7.4.1 PagedAttention

PagedAttention,²³ introduced in vLLM, resolves this memory wall by applying operating system virtual memory paging to accelerator memory (Kwon et al. 2023). In static contiguous allocation, reserving maximum sequence length memory up front causes internal waste, while freed regions may be too small or poorly placed to satisfy a new contiguous reservation. Instead of allocating one contiguous region per sequence, PagedAttention divides the KV cache into fixed-size blocks mapped through a block table. A sequence’s cache can therefore occupy noncontiguous physical blocks. When a sequence completes, its blocks return to a free pool for reuse. Rather than copying noncontiguous blocks into a temporary contiguous buffer before computing attention, a specialized PagedAttention CUDA kernel traverses the block table directly, staging noncontiguous blocks into on-chip SRAM during query-key multiplication. By eliminating allocation fragmentation, PagedAttention reduces KV-cache waste below 4 percent and enables substantially higher batch concurrency on the same accelerator hardware (Kwon et al. 2023).

PagedAttention recovers physical memory capacity, but serving autoregressive models at scale exposes additional bottlenecks across token verification latency and memory-bandwidth-bound weight streaming.



Systems Perspective 13.7: LLM serving: Beyond the fundamentals

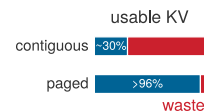
The key-value cache that stores attention context scales with sequence length and batch size, often exceeding the model weights themselves in memory consumption. Techniques such as speculative decoding use small draft models to propose multiple tokens that the target model verifies in parallel; evaluated T5-XXL workloads achieved roughly 2–3× speedups (Leviathan et al. 2023), but gains vary. Weight-only quantization (such as INT4 weights with FP16 activations) is especially relevant for memory-bandwidth-bound LLM inference (Lin et al. 2024).

These LLM-specific optimizations build directly on core serving mechanics: queueing models govern scheduling timeouts, batching dynamics determine throughput-latency curves, and reduced precision bounds memory traffic. Advanced treatments extend these mechanics to multi-tenant prefix sharing across fleets and multi-node tensor-parallel execution.

Although continuous batching maximizes throughput for multi-tenant language model serving, batching is not universally optimal. Schedulers and dynamic memory allocators introduce coordination complexity and synchronization overhead that can degrade performance in latency-critical or resource-constrained environments. In particular, three operational constraints favor single-request processing. First, under an ultra-low p99 target, even a short batching delay consumes an unacceptable fraction of the latency budget. Second, when serving runtimes lack ragged tensor or paged memory support, batching variable-length inputs incurs padding overhead to match the longest sequence in the batch, whereas single-request processing eliminates padding waste entirely. Finally, memory capacity binds when model weights already consume most of the accelerator’s physical memory; under near-capacity conditions, concurrent batch activations and dynamic KV-cache allocations risk out-of-memory aborts, forcing the runtime to process requests individually.

13.7.5 Session affinity constraints

Session affinity—routing successive requests from a client to the same physical replica—arises when serving state resides in local accelerator memory rather than being reconstructed on every invocation. In conversational models, this resident state is dominated by the key-value (KV) cache. When turn k executes on a replica, that replica’s High Bandwidth Memory (HBM) retains the projected keys and values across all prior prompt and generation tokens. Routing turn $k + 1$ to a different replica



PagedAttention recovers the KV-cache waste of contiguous allocation.

²³ | **PagedAttention:** The name directly references OS virtual memory paging, first implemented on the Atlas computer at Manchester (1962) to solve the same class of allocation problem—programs needed more memory than physically available, and contiguous allocation wasted space (Kilburn et al. 1962). Introduced at SOSOP 2023, PagedAttention applies this six-decade-old abstraction to GPU KV-cache memory: before it, LLM serving systems wasted 60 to 80 percent of KV-cache memory due to fragmentation and over-reservation. PagedAttention reduces waste to under 4 percent, enabling 2–4× higher throughput on the same hardware (Kwon et al. 2023).

discards that local memory footprint, forcing the new node to recompute the entire prefix sequence from scratch. This redundant prefill consumes memory bandwidth and tensor-core cycles that would otherwise advance active decode iterations, directly inflating time-to-first-token.

A parallel physical constraint governs personalized serving using fine-tuned Low-Rank Adaptation (LoRA) modules. While base-model weights remain pinned in device HBM, individual user adapters must be dynamically staged into memory. If incoming requests for a user scatter across multiple replicas, each node must repeatedly transfer adapter tensors over the host-to-device PCIe bus or fetch them across the network, displacing other active working sets in memory. Stateful preprocessing pipelines—such as tokenizer vocabularies and per-session normalization state—impose the same penalty, requiring redundant CPU-side allocations and cache rebuilds whenever execution moves across node boundaries.

Strict affinity creates a direct conflict with dynamic batching. Continuous batch schedulers maximize accelerator throughput by packing concurrent requests into unified matrix multiplications, raising operational arithmetic intensity in proportion to batch size ($I \propto B$). When requests are pinned to designated replicas, the scheduler’s pool of batchable candidates fragments across the cluster. A replica assigned to an idle or low-concurrency user session must either process underfilled batches—collapsing arithmetic intensity back into the memory-bandwidth-bound regime—or stall waiting for affinity-matched tokens. To prevent this collapse in utilization, production routers use soft affinity. Under soft affinity, the ingress gateway prioritizes the cache-holding replica when capacity allows, but sheds requests to alternate nodes when local queue delays exceed the physical latency required to transfer adapter weights or recompute prefix tokens.

13.7.6 Traffic patterns and batching strategy

The batching strategy depends critically on how requests arrive. Different deployment contexts exhibit different arrival patterns, and the MLPerf inference benchmark supplies four standardized scenarios that approximate several common cases, as section 12.8.4.2 explains in detail.

13.7.6.1 Server traffic (Poisson arrivals)

The MLPerf Server scenario models cloud/API-like inference traffic with Poisson arrivals (Reddi et al. 2019).²⁴ Under that model, counts in disjoint intervals are independent and the average arrival rate is constant. Equation 13.14 expresses the expected count in a fixed calendar window with rate λ_{arr} and width T_{window} :

$$E[\text{batch size}] = \lambda_{\text{arr}} \cdot T_{\text{window}} \quad (13.14)$$

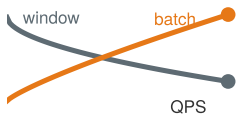
For fixed calendar windows, variance equals the mean, so batch sizes fluctuate significantly at moderate traffic. With $\lambda_{\text{arr}} = 200$ requests/second and $T_{\text{window}} = 10$ ms, the expected count is two, but roughly 14 percent of windows have zero requests (and launch no GPU work) while others may have four or more.

A useful heuristic for the batching window balances waiting cost against throughput benefit. Equation 13.15 expresses one such rule:

$$T_{\text{window}} \approx \min \left(L_{\text{lat,SLO}} - T_{\text{svc}}, \sqrt{\frac{T_{\text{svc}}}{\lambda_{\text{arr}}}} \right) \quad (13.15)$$

where $L_{\text{lat,SLO}}$ is the latency SLO, T_{svc} is the service time (in seconds), and λ_{arr} is the arrival rate (in requests per second), making the second term dimensionally consistent in seconds. The square-root form is a local cost-model heuristic: it balances a fixed per-batch benefit against a waiting cost that grows with the arrival interval. It is not a closed-form optimum for ML serving specifically; production systems calibrate the window empirically against observed traffic. A counterintuitive result emerges from equation 13.15: as traffic increases, the optimal window decreases while achieved batch sizes still grow. Table 13.15 demonstrates this phenomenon across four traffic levels.

²⁴ | **Poisson process:** Named after French mathematician Simeon Denis Poisson (1781–1840), this stochastic model describes events occurring continuously and independently at a constant average rate. In fixed calendar windows, variance equals the mean, so with $\lambda_{\text{arr}} = 200$ req/s and a 10 ms window, the expected count is two and roughly 14 percent of windows are empty, though an efficient scheduler launches no GPU work for them. A request-triggered timer instead begins with one request and counts only additional arrivals during the window.



As QPS rises, the batching window shrinks while batch size grows.

Table 13.15: Traffic-Adaptive Batching: Higher traffic enables shorter windows while still achieving larger average batches. Values are computed from equation 13.15 with a 50 ms SLO and a 25 ms service-time assumption, so the latency column is the approximate service-plus-window budget rather than a measured production p99.

Arrival Rate	Optimal Window	Avg Batch Size	Approx. Latency
100 QPS	15.8 ms	1.6	40.8 ms
500 QPS	7.1 ms	3.5	32.1 ms
1,000 QPS	5 ms	5	30 ms
5,000 QPS	2.24 ms	11.2	27.2 ms

13.7.6.2 Streaming traffic (correlated arrivals)

Autonomous vehicles, video analytics, and robotics systems receive inputs from multiple synchronized sensors. A six-camera example makes the synchronization deadline concrete.

Example 13.8: Multi-camera autonomous vehicle serving

Scenario: An autonomous vehicle vision system processes 6-camera video streams under a strict 33 ms real-time frame deadline.

Diagnosis: Sensor arrival jitter causes variable multi-camera batch assembly delays, risking frame deadline misses. Implementing a 12 ms synchronization budget and fallback rules (reusing previous frames on timeout) bounds latency.

Systems lesson: Synchronized multi-sensor streaming workloads enforce deterministic real-time deadlines. Serving pipelines must implement explicit synchronization timeout policies to prevent sensor jitter from breaching latency budgets.

Strict latency budgets make synchronized multi-camera frame processing challenging. Table 13.16 traces an event-by-event timeline for one synchronized frame set across six cameras operating at 30 FPS within a 33 ms frame deadline. Sensor capture and network arrival jitter consume 15 ms before inference can begin. Batching all six camera streams into a single GPU invocation completes in 10 ms (at $T = 25$ ms), leaving 7 ms for downstream postprocessing, planning, and safety checks. If arrival jitter pushes batch assembly past 15 ms, the pipeline breaches its real-time deadline.

Table 13.16: Multi-camera frame timeline: Event-by-event timeline for one synchronized frame set across six cameras at 30 FPS. Frames arrive over a 7 ms spread, inference cannot start until the last one lands at 15 ms, and the result reaches the planning module at 32 ms, leaving 1 ms of slack inside the 33 ms deadline.

Time	Event
$T = 0$ ms	Cameras begin capturing frame N
$T = 8$ ms	Camera 1 frame arrives
$T = 10$ ms	Cameras 2–5 frames arrive
$T = 15$ ms	Camera 6 arrives (jitter)
$T = 15$ ms	Batch inference begins (6 images)
$T = 25$ ms	Inference complete
$T = 32$ ms	Result ready for planning module

13.7.6.3 Single-user traffic (sequential arrivals)

Streaming traffic correlates arrivals through sensor synchronization, making batch size and deadline externally fixed. At the opposite end of the spectrum, interactive mobile and edge applications process one request stream at a time, leaving no opportunity to batch independent queries. The MLPerf SingleStream scenario captures this sequential-serving shape. Here, the optimization target shifts from a synchronization budget under a frame deadline to per-request latency and energy consumption under a rigid thermal envelope. For the smartphone vision pipeline modeled below, the primary engineering objective is not batch formation, but minimizing per-phase latency and energy dissipation.

13.7.6.4 Mobile serving constraints

Cloud services often emphasize fleet cost and latency; mobile serving adds three closely coupled constraints. The first is an energy budget that throughput alone does not capture, because each inference consumes battery. In the modeled pipeline, 2.98 mJ at 22 FPS draws about 66 mW for the inference path alone, before camera, display, ISP, and OS overhead add to that total in a full photo app. Thermal throttling can compound this limit when sustained system power reaches a device-specific thermal boundary. Memory constraints close the set because the model shares RAM with the application and operating system. Reload time depends on device storage, runtime, and model state; quantization and memory-mapped loading (section 13.6.3) can reduce footprint and startup work but do not eliminate those dependencies.



Example 13.9: ResNet-50: Mobile serving

Table 13.17 decomposes per-phase latency and energy for a single-user mobile vision inference:

Table 13.17: Mobile ResNet-50 pipeline: Illustrative per-phase latency and energy for a single-user mobile vision inference, showing that JPEG decode on the CPU dominates the energy budget even though the NPU inference stage carries the model's compute. Optimization targets shift from throughput to energy-per-inference at the edge.

Phase	Duration	Energy	Notes
Camera buffer read	8 ms	0.08 mJ	System API
JPEG decode (CPU)	15 ms	1.5 mJ	Single-threaded
Resize + Normalize	5 ms	0.4 mJ	CPU preprocessing
NPU inference	12 ms	0.8 mJ	82% utilization
Postprocess + UI	5 ms	0.2 mJ	Result rendering
Total	45 ms	2.98 mJ	22 FPS sustained

The mobile serving node is governed by four metrics:

- **Energy per inference:** 2.98 mJ enables ~12.1M inferences per 10 Wh battery (typical smartphone)
- **Modeled inference power:** $2.98 \text{ mJ} / 45 \text{ ms} = 66 \text{ mW}$, excluding the rest of the device and therefore not a guarantee of indefinite operation
- **NPU vs. CPU trade-off:** CPU fallback replaces the 12 ms, 0.8 mJ NPU inference stage with a 45 ms, 4.2 mJ CPU stage; the full pipeline would rise from 45 ms and 2.98 mJ to about 78 ms and 6.4 mJ before additional system overhead.
- **Memory footprint:** 150 MB peak (model + activations), competing with app memory

Systems insight: In this illustrative pipeline, the mobile NPU reaches 82 percent utilization at batch size one while the data-center example reaches 15 percent. The contrast is workload- and hardware-specific, but it shows why a mobile accelerator designed for single-stream execution can operate efficiently without the batches used to amortize a large server GPU.

These physical constraints make edge serving optimization fundamentally different from data-center optimization. Rather than maximizing aggregate batch throughput, the objective is to sustain real-time execution within a tight thermal and battery envelope without triggering hardware throttling or starving foreground applications of system memory.

13.7.6.5 Traffic pattern summary

Traffic-adaptive batching adjusts the batching window as queue depth and request rate change. Table 13.18 relates the four MLPerf scenarios to representative deployment contexts and common batching strategies.

These scenarios are benchmark abstractions rather than deployment prescriptions. The operative batching policy must still follow the service's current concurrency, deadline, and power constraints.

Table 13.18: Traffic Patterns and Batching Strategies: The four MLPerf inference scenarios approximate distinct deployment contexts. Server traffic (cloud APIs) uses dynamic batching with a timeout; MultiStream approximates synchronized sensor inputs; SingleStream processes requests individually; Offline uses large batches subject to memory capacity to maximize throughput.

Scenario	Context	Strategy	Focus
Server	Cloud APIs, web services	Dynamic batching with timeout	Window tuning, utilization-latency curve
MultiStream	Autonomous driving, video analytics	Synchronized sensor fusion	Jitter handling, deadline guarantees
SingleStream	Mobile apps, embedded devices	No batching ($B = 1$)	Preprocessing, power efficiency
Offline	Batch processing, data pipelines	Maximum batch size	Throughput, hardware utilization

Each of these deployment patterns—whether serving cloud API traffic under the Server scenario, synchronizing sensor fusion under MultiStream, or running throughput-dominated batch pipelines under Offline inference—shares a foundational assumption: each request produces a single, fixed-size output, such as one classification label, one bounding box, or one embedding vector. This assumption governs the queuing math, the Pareto frontier analysis, and the traffic-adaptive window tuning. The fastest-growing category of serving workloads, however, violates this assumption entirely. Large language models generate outputs token by token, with each token depending on every previous one. A single request may produce hundreds or thousands of tokens over seconds of elapsed time, yet must feel responsive from the first token onward. This fundamental shift from *fixed-output* to *variable-length, streaming-output* serving builds directly on continuous batching and KV-cache paging for autoregressive generation. It introduces phase-split metrics for prefill and decode, decoding strategies that trade output quality against per-token latency, and memory optimizations such as prompt prefix caching and offloading that exploit shared context across requests.



Checkpoint 13.3: Batching and traffic patterns

Batching is the primary lever for serving economics, but the optimal strategy depends on context.

- ❑ **Throughput-latency trade-off:** can you explain why batch size 32 achieves $6\times$ higher throughput than batch size one, yet a production system with a 20 ms SLO might still choose batch size eight?
- ❑ **Dynamic vs. static batching:** can you describe why static batching (waiting for a full batch) fails under variable traffic, and how dynamic batching with a time window solves this?
- ❑ **Traffic pattern matching:** given a deployment scenario (for example, cloud API, autonomous vehicle, mobile app), can you select the appropriate MLPerf scenario and explain why that batching strategy fits?
- ❑ **Adaptive windows:** can you explain why the optimal batching window *decreases* as traffic *increases*, even though batch sizes grow?

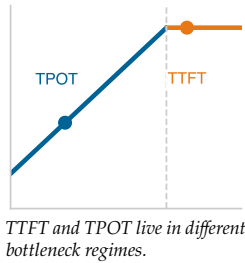
13.8 LLM Serving

Large language models make three properties far more prominent than in fixed-output serving: *autoregressive generation*²⁵ (each token depends on all previous tokens, making output inherently sequential), *variable-length output* (response length is unknown at request time, complicating fixed-batch assumptions), and *stateful memory* (the key-value cache grows with each generated token, creating dynamic memory pressure). Together, these properties create a qualitatively different serving challenge. The p50, p95, and p99 metrics used for classification serving still matter, but they apply to different *phases* of the request—the initial prompt processing and the subsequent token generation. The same principles of queuing theory, batching trade-offs, and latency budgets remain applicable; LLM serving adds domain-specific techniques atop them.

²⁵ | **Autoregressive:** From Greek *auto-* (self) and Latin *regressus* (a going back)—the output “regresses” on itself; George Udny Yule introduced autoregressive models in 1927 for analyzing sunspot cycles. In language modeling, each output token conditions on all previously generated tokens, creating a serial dependency that prevents the parallelism exploited during training. Decode is often weight-bandwidth-bound at small batches; larger batches amortize the weight stream and can shift the bottleneck toward compute or KV-cache traffic (Pope et al. 2023).

13.8.1 Performance metrics: TTFT and TPOT

Generative models produce a stream of tokens rather than a single output tensor. This streaming nature requires dedicated LLM performance metrics that reflect the transition from *prefill* (processing the prompt and populating its KV cache) to *decode* (generating output). Two key measures are **time to first token (TTFT)** and **time per output token (TPOT)**, which capture responsiveness and average generation pace. Per-gap **Inter-Token Latency (ITL)** measures the distribution of pauses between individual streamed tokens; TPOT instead averages the post-first-token interval across an output.



Definition 13.6: LLM performance metrics

LLM performance metrics separate initial responsiveness from the cadence of streaming autoregressive generation.

1. **Significance:** They decompose user-perceived latency into time to first token (TTFT) (whose prefill work is often compute-bound) and time per output token (TPOT) (whose decode work is often bandwidth-bound at small batches).
2. **Distinction:** Unlike fixed-output metrics (for example, end-to-end latency), LLM metrics measure the fluidity of generation, acknowledging that the user experience depends on the *rhythm* of token arrival.
3. **Common pitfall:** A frequent misconception is that a “fast model” has a low TTFT. In reality, a model can have a fast TTFT but a sluggish TPOT (if the memory wall (BW) is the bottleneck), leading to a frustrating user experience where the answer starts quickly but “stutters” thereafter.

These metrics capture distinct aspects of the user experience, so interactive services often set separate targets for TTFT, TPOT or ITL, and aggregate throughput.

Systems Perspective 13.8: LLM serving latency targets

Interactive LLM services usually need separate SLOs for responsiveness, generation fluidity, and fleet throughput. The following values are illustrative targets rather than universal requirements:

- **TTFT:** < 500 ms (for a 1000-token prompt)
- **TPOT:** < 50 ms (equivalent to ~20 tokens/s, faster than human reading speed)
- **Throughput:** > 1000 tokens/s aggregate across active serving replicas

A single “latency” number therefore hides the prefill/decode split. TTFT is the blank-screen budget, TPOT is the reading-flow budget, and aggregate service throughput, measured as tokens/s summed across active serving replicas, determines whether those targets hold under shared load.

13.8.2 Decoding strategies

Meeting these TPOT targets depends on more than memory bandwidth alone: the algorithm used to select each token also affects per-token latency and output quality. Generative models require decoding strategies that trade off quality, diversity, and latency. The choice of decoding strategy dramatically affects both output quality and computational cost.

The simplest approach, greedy decoding, selects the highest-probability token at each step at the cost of one model pass per token. It is fast but can produce repetitive outputs because it cannot recover from early choices. Beam search maintains multiple candidate sequences and selects a high-scoring complete sequence, increasing decoder work and state relative to greedy decoding. Sampling with temperature, top- k , and top- p (also called nucleus sampling) injects controlled randomness for diversity (Holtzman et al. 2020). The serving cost of any strategy depends on its implementation and on the output-length distribution that continuous batching must absorb.

Interactive LLM services commonly return tokens as they are produced rather than waiting for complete generation. This streaming response transforms the user experience: a two-second total generation feels responsive when tokens stream continuously, but feels broken when users stare at a blank screen for two seconds. Streaming requires infrastructure support for incremental responses and client-side rendering. The latency profile shifts accordingly: TTFT determines when output starts appearing (responsiveness), while TPOT determines the perceived generation speed (fluidity). Once generation is streamed token by token, the serving unit changes from one fixed-output prediction to a stateful sequence whose memory footprint grows on every step.

13.8.3 Memory and KV cache

Generative inference requires managing the KV Cache²⁶, a stateful memory structure that grows with sequence length. Prefill creates cache entries for prompt tokens, and each decoded token adds another entry. Across concurrent variable-length sequences, this dynamic state can dominate available accelerator memory and create allocation waste if it is not managed explicitly.

13.8.3.1 Prefix caching and memory offloading

The continuous batching and PagedAttention techniques covered in section 13.7.4 address request scheduling and cache paging; the remaining memory pressure can be further mitigated through architectural strategies that exploit request patterns. **Prefix caching** stores the KV states of common instruction prefixes (such as a 2,000-token system prompt or a shared retrieval-augmented generation (RAG) context), allowing independent requests to reuse the prefill result for an identical prefix. For N requests sharing a prefix of S_{prefix} tokens, the avoided prefill work is roughly $(N - 1)S_{\text{prefix}}$ token steps, subject to cache hits, eviction, and implementation overhead. For multi-turn conversations, the same principle lets a subsequent turn process only the new suffix after reusing compatible prior state.

Napkin Math 13.5: The energy cost of a chat

Problem: How much energy does an H100-backed chat service spend per generated token and for a response with 500 tokens?

As LLMs scale, joules per token becomes a first-class operational metric alongside latency. Assuming the H100 draws its 700 W thermal design power (TDP), this scenario estimate follows from throughput and power (Choquette 2023):

1. **Throughput:** $114 \text{ concurrent requests} \times 8 \text{ tokens/s per request} \approx 912 \text{ tokens/s}$.
2. **Power:** $700 \text{ W (GPU)} + 300 \text{ W (Host/Overhead)} = 1000 \text{ W}$.
3. **Energy per token:** $1000 \text{ W} / 912 \text{ tokens/s} \approx 1.0965 \text{ J/token}$

Systems insight: Under these assumptions, a response of 500 tokens consumes $\approx 548.2 \text{ J}$.

- For comparison, charging a smartphone consumes $\approx 40000 \text{ J}$.
- Boiling a cup of water consumes $\approx 100000 \text{ J}$.

The primary way to reduce J/token is to increase hardware utilization and eliminate redundant compute. If the GPU sits at 10 percent utilization due to poor batching, it still draws $\sim 300 \text{ W}$ and the host adds 300 W , causing energy per token to rise to 6.6 J/token (approximately 6× the baseline). Architectural optimizations like prefix caching also skip the energy-intensive prefill phase for shared context, directly reducing the energy footprint of retrieval-augmented generation (RAG) and chat applications. The serving lesson is that efficiency is not only a latency or cost metric; it also determines how much energy each useful token consumes.

When the aggregate KV cache exceeds GPU VRAM, systems can employ **KV cache offloading**. This strategy spills inactive or low-priority context windows to host CPU RAM or NVMe SSD, freeing VRAM for active generation. The reload cost is bounded below by bytes moved divided by PCIe or NVMe bandwidth, before software overhead and queueing are added. Offloading therefore prevents OOM failures and enables larger context windows, but it also creates affinity, invalidation, and hot-decode latency risks that must be budgeted explicitly. Advanced techniques including

²⁶ | **KV cache (key-value cache):** To avoid redundant work, the system caches the Key and Value vectors from previous tokens, which remain valid throughout generation; this design choice is the direct cause of the dynamic memory growth described; the cache's size grows linearly with every generated token, making memory management a primary constraint. For the 70-billion-parameter-class grouped-query-attention sizing example in this calculation, the FP16 KV cache is about 0.33 MB per token per sequence; grouped-query attention (GQA), used in Llama-family models such as Llama 3, shares key/value heads across multiple query heads, reducing the cache relative to full multi-head attention (Dubey et al. 2024). A batch of 32 requests at an 8,000-token context therefore requires roughly 80 GB just for KV cache, several times larger still without grouped-query or multi-query attention.

27 | Speculative decoding:

A small “draft” model generates k candidate tokens autoregressively; the large target model then verifies the proposed block in parallel (Leviathan et al. 2023). Parallel verification is possible because the draft model guesses all k candidate tokens in advance. Rather than generating sequentially, the target model evaluates the entire k -token block in a single causal-attention forward pass—acting like a compute-bound mini-prefill—which replaces k memory-bandwidth-bound matrix-vector operations with one parallel matrix-matrix evaluation. When the draft model’s proposals are accepted at rate α , effective throughput can scale with the number of accepted tokens per verification step. This breaks the serial autoregressive bottleneck at the runtime layer, not the architecture layer.

speculative decoding²⁷ and distributed parallelism, where one request is split across multiple devices or machines, are covered in specialized treatments of large-scale systems.



Checkpoint 13.4: LLM serving fundamentals

LLM serving introduces constraints absent from traditional model serving.

- ☐ **TTFT vs. TPOT:** can you explain why these metrics capture different user experiences, why prefill is often compute-bound while low-batch decode is often bandwidth-bound, and convert a 50 ms TPOT target into tokens per second?
- ☐ **Memory wall:** can you explain why adding compute cores yields little latency improvement when token generation is bandwidth-bound, and why faster memory, smaller models, or batching can change the bound? (The Llama-3 case study in section 13.11.4 quantifies this relationship.)
- ☐ **Continuous batching:** can you explain why traditional static batching wastes compute when sequence lengths vary, and how iteration-level batching solves this?
- ☐ **PagedAttention:** can you explain the memory fragmentation problem in KV-cache management and how fixed-size, noncontiguous blocks reduce reservation waste?
- ☐ **Prefix caching:** can you explain how caching the KV states of common instruction prefixes reduces redundant computation and speeds up RAG or multi-turn applications?

The chat-energy calculation makes utilization part of the LLM serving budget: the cost of each generated token depends on how requests share the machine. Unlike fixed-output inference, LLM energy grows with response length. Each decode step streams weights once for the active batch, so batching amortizes that traffic across sequences. Carbon accounting additionally requires an emissions factor and a defined boundary.

Energy efficiency depends on the same batching, memory, and prefix-cache mechanisms that govern LLM latency, so the useful summary is a constraint checklist rather than a single scalar metric.

13.9 Inference Runtime Selection

Dynamic batching and request scheduling dictate how inferences are grouped and queued, but an underlying execution engine must evaluate the resulting tensor operations on physical hardware. The token generation dynamics introduced in section 13.8 and the latency budgets established in section 13.4.1 are achievable only if the runtime maps operations efficiently to accelerator execution units and memory hierarchies. The **inference runtime**, the software layer that orchestrates tensor operations and manages hardware resources, can materially change latency and throughput for an unchanged model architecture. Runtime engineering consists of two phases: selection chooses the execution engine, and configuration tunes that engine for the target model, hardware, input shapes, and latency distribution.

13.9.1 Runtime ecosystem and configuration

Engine selection begins with the binding system constraint rather than the framework used during training. When deployment speed and framework compatibility dominate, PyTorch and TensorFlow models can serve directly through their native runtimes without an intermediate export step. This preserves support for arbitrary Python control flow and custom training operators, although non-standard operations, missing hardware backends, and Python interpreter overhead can restrict production throughput. Framework runtimes prioritize flexibility and general operator coverage, whereas specialized inference engines trade that generality for targeted graph transformations and lower execution overhead.

`torch.export` and TensorFlow SavedModel provide serialized computational graphs for ahead-of-time transformation and graph optimization while maintaining framework compatibility. TorchScript remains a legacy format for existing deployments.

13.9.1.1 General-purpose optimization

When cross-hardware portability is the binding constraint, ONNX Runtime²⁸ provides an intermediate, hardware-agnostic execution layer (Microsoft 2024b). Models export to the ONNX format, after which ONNX Runtime applies target-independent graph optimizations and delegates subgraphs to hardware-specific execution providers. This architecture enables single-format deployment across CPUs, GPUs, and custom accelerator application-specific integrated circuits (ASICs).

Portability still demands profiling operator placement across execution boundaries. If an exported model contains an operation unsupported by the target accelerator’s execution provider, the runtime falls back to a default CPU execution provider. This fallback forces synchronous data copies across the host-accelerator interconnect (such as PCIe) and serializes execution on the host CPU, preserving numerical correctness while destroying the expected latency advantage. Profiling must therefore verify provider assignment and operator coverage across the entire graph, not merely confirm that the exported artifact executes without errors.

13.9.1.2 Specialized inference engines

When latency budgets or serving costs bind more tightly than portability, specialized engines such as TensorRT²⁹ for NVIDIA GPUs and OpenVINO³⁰ for Intel hardware optimize graphs specifically for target microarchitectures (NVIDIA 2024c; Intel Corporation 2026b; Chen et al. 2018). Rather than maintaining portable abstractions, these compilers specialize kernels directly to the hardware’s register files, cache hierarchies, and matrix execution units.

Layer fusion³¹ merges multiple sequential operations into a single GPU kernel to eliminate off-chip memory traffic. Consider a standard vision block: convolution $\rightarrow$ batch normalization $\rightarrow$ rectified linear unit (ReLU) activation. Executed naively, this sequence requires three distinct kernel launches and two intermediate write/read round-trips to global device memory (HBM). Because batch normalization and ReLU have low arithmetic intensity, transferring these intermediate feature maps across the memory bus saturates memory bandwidth. At compile time, the serving compiler first folds the linear batch normalization parameters directly into the convolution weights and bias ($W' = \frac{\gamma}{\sigma}W$ and $b' = \frac{\gamma}{\sigma}(b - \mu) + \beta$), eliminating the normalization operation entirely. The compiler then fuses the convolution and the subsequent ReLU into a single kernel: thread blocks read the input feature map from HBM once, compute matrix multiply-accumulate operations in registers and shared memory, evaluate the ReLU activation directly in registers, and write only the final activated result back to HBM.

Kernel auto-tuning benchmarks and selects the optimal algorithm for each operation on the target microarchitecture. A single 2D convolution can be evaluated through direct convolution, GEMM transformation (im2col), Winograd minimal filtering (efficient for small 3×3 filters), or Fast Fourier Transforms (efficient for large spatial filters), alongside dozens of tile-size variations. Because the winning strategy depends on filter dimensions, batch size, and the accelerator’s register and shared memory capacity, the compiler benchmarks candidate implementations against target hardware and caches the lowest-latency kernel, trading ahead-of-time compilation time for predictable serving latency.

These optimizations produce substantial speedups over an untuned framework-native baseline, but they require explicit graph compilation and may restrict dynamic tensor dimensions. The illustrative runtime comparison that follows quantifies one such optimization spectrum.

Example 13.10: ResNet-50: Runtime comparison

Table 13.19 gives an illustrative batch-one ResNet-50/V100 runtime scenario:

²⁸ | **ONNX Runtime:** Microsoft’s inference engine acts as a hardware abstraction layer: the same ONNX model can run on CPUs, NVIDIA GPUs, AMD GPUs, or custom accelerators through pluggable execution providers. ONNX Runtime applies graph optimizations such as constant folding, redundant-node elimination, and operator fusion. Performance relative to a tuned target-specific engine such as TensorRT depends on the workload, execution provider, and hardware.

²⁹ | **TensorRT:** An NVIDIA inference compiler and runtime that optimizes models ahead of time for a specific microarchitecture (such as Hopper or Blackwell) (NVIDIA 2024c). Requiring a target-specific build phase allows the compiler to generate fused kernels, select microarchitecture-specific Tensor Core execution paths, and optimize memory layouts (such as NC/32HW32). The resulting nonportable engine reduces per-request execution latency and minimizes the accelerator count needed to satisfy a target throughput.

³⁰ | **OpenVINO (open visual inference and neural network optimization):** An Intel-oriented inference toolkit that converts, optimizes, and runs models across Intel CPU, GPU, and NPU targets (Intel Corporation 2026b). Specializing for direct hardware execution departs from the portable execution guarantees of general-purpose frameworks, enabling the compiler to emit architecture-tuned vector instructions (such as AVX-512 or AMX) and exploit low-precision integer execution units. While speedups depend on the specific operator mix and target processor, this specialization can make CPU or edge serving economically viable for latency-sensitive models.

31 | Layer fusion: Analogous to loop fusion in compiler optimization, where adjacent loops over the same array are combined to reduce memory traffic. Kernel fusion applies the same principle to GPU operations: sequential kernels that write and re-read intermediate tensors from HBM are merged into a single kernel that can keep data in registers.

Table 13.19: Illustrative inference runtime comparison: Assumed latency and derived speedup for batch-one ResNet-50 across PyTorch eager, TorchScript, ONNX Runtime, and TensorRT on a V100.

Runtime	Latency	Speedup	Notes
PyTorch (eager)	8.5 ms	1×	Baseline, no optimization
TorchScript	6.2 ms	1.4×	JIT compilation
ONNX Runtime	5.1 ms	1.7×	Cross-platform
TensorRT FP32	2.8 ms	3×	NVIDIA-specific
TensorRT FP16	1.4 ms	6.1×	Tensor Core acceleration
TensorRT INT8	0.9 ms	9.4×	Requires calibration

Systems insight: In this illustrative comparison, TensorRT INT8 provides 9.4× speedup and requires quantization calibration data and NVIDIA-specific deployment.

This optimization-compatibility trade-off governs engine selection. Higher specialization yields superior hardware efficiency but increases deployment complexity, restricts dynamic control flow, and may introduce numerical deviations from the training baseline. The engineering choice depends on latency budgets, operational constraints, and how frequently model architectures evolve.

Once an execution engine is selected, configuration tunes its resource allocation to the hardware envelope. For CPU inference, thread pool sizing must match physical core topologies: undersubscription leaves execution units idle, while oversubscription induces CPU cache thrashing, OS scheduling latency, and lock contention across NUMA domains. For accelerators, static memory preallocation creates fixed-size activation workspaces during initialization, eliminating runtime memory allocations (`cudaMalloc`) and page-table faults from the latency-critical request path. Execution provider priority determines which hardware backend handles each operator subgraph, while graph optimization levels balance build times against constant folding and algebraic simplification. Production deployments must benchmark these configurations across full latency distributions (*p50*, *p99*) rather than relying on framework defaults.

13.9.2 Precision selection for serving

On NVIDIA V100 GPUs, Tensor Cores accelerate FP16 matrix operations, whereas INT8 computations execute on standard integer ALUs (via DP4A vector instructions). In an illustrative serving scenario, transitioning a ResNet-50 model from FP32 to INT8 triples inference throughput while sacrificing less than 0.4 percentage points of top-1 accuracy. This dynamic establishes a direct coupling between numerical precision and infrastructure economics. Precision selection builds on the quantization techniques covered in section 10.4. Section D.4 compares the underlying numerical formats (FP32, FP16, BF16, FP8, INT8) and their precision-range trade-offs, while section D.4.2 details the mathematical mechanics of symmetric and asymmetric integer quantization. At the serving layer, precision engineering introduces operational constraints: calibration data fidelity, layer sensitivity under live inputs, and dynamic precision dispatch.

13.9.2.1 Precision-throughput relationship

For memory-bandwidth-bound operations, reducing precision raises throughput by cutting the byte volume transferred across the memory bus. Equation 13.16 gives the bandwidth-only upper bound from precision reduction:

$$\frac{\text{Throughput}_{\text{INT8}}}{\text{Throughput}_{\text{FP32}}} = \frac{32}{8} = 4 \times (\text{theoretical maximum}) \quad (13.16)$$

Actual speedup depends on the compute-to-memory ratio, operator mix, hardware instruction throughput, and matrix alignment, falling below the bandwidth-only bound whenever arithmetic throughput or memory alignment binds. Low-precision kernels achieve peak arithmetic efficiency only when matrix dimensions align with the accelerator’s warp-level execution tiles (for example,

multiples of 8 or 16 elements). When dimensions do not align with hardware tiles, libraries such as cuBLAS and cuDNN pad tensors or fall back to slower execution paths. Section 11.4.3 details the Tensor Core architectures and alignment constraints that govern these boundaries. The precision trade-offs for a standard vision model illustrate how these theoretical limits manifest in practice.

Example 13.11: ResNet-50: Precision trade-offs on V100

Table 13.20 gives illustrative latency, memory, and accuracy assumptions across FP32, FP16, and two INT8 paths for ResNet-50, together with Tensor Core utilization where supported:

Table 13.20: Illustrative precision trade-offs on V100: Assumed latency, memory footprint, and accuracy for ResNet-50 in FP32, FP16, and INT8 (PTQ and QAT), plus Tensor Core utilization where supported.

Precision	Latency	Memory	Accuracy	Tensor Core Util.	Calibration
FP32	2.8 ms	98 MB	76.13%	0%	None
FP16	1.4 ms	49 MB	76.13%	85%	None
INT8 (PTQ)	0.9 ms	25 MB	75.80%	N/A	1,000 samples
INT8 (QAT)	0.9 ms	25 MB	76.05%	N/A	Additional training

Systems insight: In this illustrative scenario, INT8 provides 3.1× speedup and loses 0.33 percentage points of accuracy with PTQ. QAT recovers most of that loss but requires retraining.

13.9.2.2 Precision selection constraints

Precision selection is constrained by layer sensitivity, calibration data, and runtime policy. Not all layers tolerate reduced precision equally. For a scalar uniform b -bit quantizer over the calibrated range $[\alpha, \beta]$, the step size and within-range rounding-error bound in equation 13.17 are

$$\Delta = \frac{\beta - \alpha}{2^b - 1}, \quad |\epsilon_{\text{quant}}| \leq \frac{\Delta}{2} \quad (13.17)$$

for values that are not clipped. Values outside the calibrated range are clipped and incur additional saturation error. Layer-level output or task error depends on activation distributions and downstream sensitivity, so it must be measured rather than inferred from a universal norm law. This mechanism explains why first convolutional and final classification layers are commonly retained at FP16, while intermediate layers tolerate INT8. Initial layers process low-dimensional raw inputs (such as 3 RGB channels) before representations expand into overcomplete feature spaces that can absorb quantization noise, whereas final classification layers produce raw logits where small numerical perturbations directly shift decision boundaries and alter top-1 predictions.

Post-training quantization adds a data dependency. The calibration dataset determines the clipping thresholds $[\alpha, \beta]$ and scale factors Δ used for INT8 conversion, requiring it to match the statistical profile of live serving traffic rather than arbitrary training or validation splits. A calibration-serving distribution mismatch causes excessive clipping or underflow, degrading end-to-end task accuracy—an operational pitfall analyzed in section 13.12.

Production serving architectures can elevate precision from a static compilation setting into a dynamic runtime policy. When request queues are shallow and the system operates with ample latency headroom against its service level objective (SLO), the engine can dispatch requests at FP16 to maximize task accuracy. During traffic spikes that threaten queue saturation, the engine can dynamically downcast execution to INT8 to preserve latency bounds. Similarly, requests that yield low classification confidence under INT8 can be selectively re-evaluated at FP16. This dynamic dispatch operationalizes adaptive quality-latency trade-offs under fluctuating request arrival rates.

Precision selection carries direct infrastructure sizing consequences: under an illustrative 3× throughput improvement, a serving cluster provisioned with 30 GPUs at FP32 satisfies the identical traffic demand with 10 GPUs at INT8, holding queueing latency constant. Precision engineering is therefore an infrastructure sizing discipline, directly coupling numerical representation to cluster capital expense and energy consumption.

13.10 Node-Level Optimization

Runtime selection and precision tuning operate at the model level: they determine *what* computation runs and at *what* numerical format. Between the model and the silicon, however, lies an execution layer encompassing graph compilation, memory bus transfers, and CPU-accelerator coordination. While model-level benchmarks evaluate isolated forward passes on synthetic inputs, node-level optimization targets the end-to-end request path on a concrete physical machine. Realized latency and throughput gains depend on identifying which hardware boundary binds execution.

When an image classifier with a sub-millisecond model benchmark exhibits unexpected latency in production, a measured execution trace typically reveals bottlenecks across four primary physical boundaries:

- **Graph-to-kernel boundary:** Translating computation graphs into numerous fine-grained kernel launches incurs driver serialization overhead and forces intermediate tensors through device HBM. Graph compilers fuse adjacent operations into composite kernels that preserve intermediate activations in on-chip SRAM or register files.
- **Host-accelerator boundary:** Accelerators sit idle when the host processor fails to enqueue work ahead of execution. Unpipelined copies across host-device interconnects (such as PCIe) and blocking synchronization calls stall accelerator compute pipelines.
- **CPU execution boundary:** Host preprocessing pipelines and CPU-based inference degrade when executed through interpreted scalar code rather than vectorized single instruction, multiple data (SIMD) instruction units and NUMA-local memory allocations.
- **Storage and deserialization boundary:** Initializing serving replicas requires streaming weight tensors from persistent storage into host memory. Formats requiring recursive object deserialization bottleneck CPU execution, creating cold-start latency spikes that impede autoscaling.

Each boundary represents a distinct physical constraint where execution stalls on memory bandwidth, bus transit, or control overhead rather than peak arithmetic throughput.

13.10.1 Runtime graph compilation

Specialized inference engines (section 13.9) outperform eager framework baselines because serving environments provide static execution topologies and bounded input-shape profiles. When operator sequences and tensor dimensions are fixed before incoming requests arrive, compilers shift optimization costs from runtime query processing to offline deployment preparation.

The primary arithmetic gain arises from operator fusion, extending the engine-level optimizations introduced in section 13.9.1.2. In an unfused execution graph, consecutive elementwise operations—such as a convolution followed by a bias add and an activation function—launch as isolated accelerator kernels. Each individual kernel must load its inputs from global device memory (HBM or GDDR) and write its intermediate output back across the memory bus. Because elementwise operations exhibit low arithmetic intensity ($O(1)$ operations per byte transferred), memory bus saturation throttles execution while compute units sit idle. Operator fusion merges these adjacent nodes into a single composite kernel, retaining intermediate activations inside fast on-chip registers or shared memory (SRAM). By eliminating redundant global memory round-trips, fusion drastically reduces memory traffic and amortizes kernel launch overheads over larger computational blocks.

Static execution graphs also enable compile-time constant folding. In model serving, model parameters are immutable. Compilers evaluate subexpressions composed entirely of static weights and constants prior to deployment. For example, batch normalization parameters ($\gamma, \beta, \mu, \sigma$) can be algebraically folded directly into the preceding linear or convolutional weight matrix and bias vector. This eliminates the normalization layer’s kernel launches and associated memory traffic entirely, removing computational work from every live request without altering mathematical outputs.

Memory planning applies structural optimization to memory layout rather than arithmetic. Reactive allocation during request processing triggers framework allocator locks, heap searches, and operating system page table modifications. In a static graph, tensor lifetimes are known deterministically from input to output. The runtime compiler calculates exact buffer offsets ahead of time, allowing non-overlapping intermediate tensors to reuse the same physical memory arena. This

precalculated buffer reuse bounds total memory consumption, prevents memory fragmentation, and eliminates runtime allocation stalls.

Systems Perspective 13.9: Compilation timing trade-off

Just-in-time compilation defers graph optimization until the runtime engine observes concrete input shapes and execution paths. This runtime visibility enables aggressive kernel specialization for dynamic control flow and variable sequence lengths, but it injects compilation overhead directly into the serving path, generating multi-second latency spikes on early requests.

Ahead-of-time compilation executes graph analysis, kernel fusion, and hardware-specific auto-tuning prior to deployment. Pre-building the execution artifact guarantees predictable, low-latency execution from the very first request, but it restricts the engine to predetermined shape profiles and requires explicit padding or multi-profile compilation to accommodate dynamic input sizes.

When deployment constraints or workload profiles preclude dedicated accelerators, optimization shifts from GPU kernels to host silicon, where vectorization and memory hierarchy govern execution speed.

13.10.2 CPU inference optimization

Deploying inference on CPUs remains practical for lightweight models, sparse or bursty request streams that cannot saturate an accelerator, and applications where host-to-device transit latency dominates total computation time. Modern server CPUs³² (such as Intel Xeon and AMD EPYC) integrate wide SIMD vector registers and matrix accelerators (including AVX-512 and AMX). General-purpose Python interpreters and scalar implementations leave these execution units idle. Specialized runtimes such as OpenVINO map neural network operators to optimized microkernels that tile matrix operations across L1/L2 caches and leverage packed fused multiply-add (FMA) instructions (Intel Corporation 2026b).

Multi-socket server architectures introduce NUMA³³ constraints. Memory controllers are physically distributed across CPU sockets and linked by coherent interconnects (such as Intel UPI or AMD Infinity Fabric). Accessing memory attached to a remote socket incurs substantially higher latency and lower bandwidth than local channel access. An inference server must maintain NUMA awareness: worker threads must be pinned to dedicated physical cores, and the weight tensors and request buffers those threads touch must reside in memory nodes local to that socket. Because model weights—hundreds of megabytes to tens of gigabytes—routinely exceed per-socket L3 cache capacities, every forward pass streams weights from main RAM. Mismatched socket allocations force continuous weight streaming across the slower inter-socket interconnect, creating severe tail-latency degradation.

For lightweight models evaluated at batch size one, a CPU frequently outperforms a discrete GPU. When an accelerator processes an isolated request, the fixed overhead of transferring inputs across the host PCIe bus (several microseconds), driver queue serialization, and kernel launch overhead (5–15 μ s per kernel) can easily surpass the arithmetic execution time. The CPU processes the request directly in local cache without bus transfers or driver queue delays. Determining the operational crossover requires empirical measurement across model architecture, arithmetic intensity, batch size, and latency constraints.

13.10.3 Model serialization and fast loading

Autoscaling systems operate as closed-loop controllers that provision or deprovision serving replicas in response to traffic demand. During demand surges, cold-start latency (section 13.6.2) governs how rapidly new replicas become ready to serve traffic. For multi-gigabyte models, cold-start duration is heavily dominated by streaming serialized model weights from persistent storage into host memory. The architectural choice of serialization format determines whether this stage is bounded by storage read bandwidth or by CPU parsing overhead.

Legacy PyTorch checkpoints loaded via `torch.load()` package weights within a zip archive containing pickle-encoded object metadata. Deserializing a pickle file forces the CPU to interpret

³² | **SIMD:** From Michael Flynn's 1966 taxonomy of computer architectures, SIMD enables one instruction to operate on multiple data elements simultaneously. Intel's AVX-512 can process sixteen FP32 values in a 512-bit vector, while AMX adds matrix tile operations. The attainable fraction of peak depends on data type, matrix shape, memory behavior, and implementation, but optimized vector kernels provide multiple-fold speedups over scalar execution.

³³ | **NUMA (non-uniform memory access):** In multi-socket servers, local memory access offers lower latency and higher bandwidth than remote socket access across inter-socket fabrics. Pinning inference threads to dedicated cores without co-locating tensor allocations forces memory traffic across the interconnect, introducing severe latency penalties when memory-bound forward passes exceed CPU L3 cache capacity.

a stream of serialization opcodes sequentially, allocating individual Python objects (dictionaries, metadata wrappers, and tensor headers) and copying tensor bytes out of decompression buffers into newly allocated memory. Even with restricted loading modes to mitigate arbitrary code execution risks, the deserialization pass remains CPU-bound and incurs substantial memory copy overhead.

A tensor-oriented format³⁴ bypasses interpreter-driven object construction. Safetensors organizes checkpoints with a lightweight header containing tensor metadata followed by contiguous, uncompressed tensor byte buffers aligned to memory boundaries. Because the on-disk binary layout mirrors the in-memory array layout, the runtime uses memory mapping (`mmap(2)`), introduced in section 13.6.3) to map file blocks directly into the process’s virtual address space. Virtual memory pages fault into host RAM on demand or are prefetched sequentially by the operating system, eliminating Python object reconstruction and user-space copy buffers. On systems with discrete accelerators, transferring these tensors across PCIe or NVLink is still required before computation begins, but zero-copy host loading eliminates the CPU deserialization bottleneck from the replica initialization budget.

³⁴ | **Safetensors:** The name emphasizes safety: unlike unrestricted pickle deserialization, the format does not encode executable Python objects (Hugging Face 2026). It stores tensors as contiguous raw bytes with a small header and supports memory-mapped loading. In the local benchmark example, that path is 10× faster than the compared PyTorch checkpoint path; the result is workload- and storage-specific and does not include transfer to a discrete accelerator.



Example 13.12: Loading speed: Safetensors vs. Pickle

Scenario: A serverless inference replica loads a 5 GB model checkpoint during cold-start scaling events.

Diagnosis: PyTorch `torch.load()` (Pickle) requires CPU object reconstruction, taking 15 s to initialize. Safetensors memory-mapping bypasses CPU parsing, loading weights in 1.5 s (10× faster).

Systems lesson: Tensor-oriented, memory-mapped formats can reduce CPU reconstruction and eager-copy overhead during cold start. Storage throughput, page faults, runtime initialization, and accelerator transfer still remain in the scale-out budget.

13.10.4 Profiling the serving node

Optimization without empirical measurement degrades into speculative trial and error. The system efficiency metric defined in equation 13.2 provides the concrete objective: maximizing the fraction of wall-clock time the accelerator spends executing useful mathematical operations. Timeline profilers, such as PyTorch Profiler and NVIDIA Nsight Systems (`nsys`), make this objective actionable by recording concurrent CPU threads, OS runtime calls, memory transfers, and GPU hardware queues along a synchronized timeline.

Effective trace analysis proceeds bottleneck-first from hardware utilization back toward application logic:

- **Idle compute bubbles:** White space on the GPU execution track indicates hardware starvation. Compute warps idle while waiting for the host CPU to complete request preprocessing (e.g., tokenization or image decoding), page faults during on-demand weight paging, or blocking inter-thread synchronization.
- **Kernel launch fragmentation:** A dense sequence of micro-kernels—each executing in under 5–10 μ s—indicates driver serialization bottlenecks. The CPU cannot enqueue kernel launches into the command queue fast enough to keep the GPU pipeline filled, identifying unfused operators as immediate candidates for graph compilation.
- **Interconnect transfer stalls:** Synchronous `MemcpyHtoD` blocks on the primary CUDA stream indicate unpipelined host-to-device data movement. Overcoming this stall requires allocating page-locked (pinned) host memory and executing asynchronous transfers on dedicated copy streams to overlap bus transit with concurrent accelerator execution on preceding requests.



Example 13.13: The profiling loop

Scenario: An inference serving node exhibits high p99 tail latency despite low average GPU compute utilization.

Diagnosis: Timeline profiling (Nsight Systems/PyTorch Profiler) reveals large idle gaps between tiny GPU kernels caused by CPU host-side preprocessing bottlenecks and kernel launch overhead.

Systems lesson: Serving node optimization requires empirical timeline profiling. Iteratively identifying trace bottlenecks (kernel launch overhead, host-device copy stalls) guides targeted optimizations like operator fusion and pinned memory.

Table 13.21 summarizes representative node-level optimization techniques alongside their primary target metrics and engineering costs. The table serves as a decision matrix rather than an exhaustive checklist: engineering effort should be directed exclusively to the row that addresses the binding bottleneck identified on the measured execution trace.

Table 13.21: Node-Level Optimization Impact: Illustrative ranges for matching techniques to measured bottlenecks. Realized gains and implementation cost depend on the model, runtime, hardware, and workload.

Technique	Target Metric	Illustrative Gain	Implement. Cost	Best For
Operator Fusion	Latency & Throughput	2–5×	Medium (Compiler)	Memory-bound layers
INT8 Quantization	Throughput	3–4×	High (Calibration)	Inference-heavy nodes
Graph Compilation	Latency	1.5–3×	Low (One-line)	Static graph models
Zero-Copy Loading	Startup Time	10–50×	Low (File format)	Autoscaling/Cold Start
CPU Pinning	Tail Latency (p99)	20–50% reduction	Low (Config)	Latency-critical apps

This diagnostic hierarchy ensures engineering resources address structural bottlenecks along the entire request path rather than applying isolated micro-optimizations. A four-level audit checkpoint structures this prioritization from external service boundaries down to device kernels.

Checkpoint 13.5: The optimization hierarchy

Optimizing inference follows the request path from the outside in across four structural levels:

- ☐ **System budget:** If an end-to-end service-level objective (SLO) is 20 ms and network transit plus serialization consumes 6 ms, is the remaining 14 ms budget sufficient for application processing, model execution, and queueing, and which tracing tool will confirm this breakdown?
- ☐ **Application level:** Is the serving system aggregating concurrent requests into dynamic batches to amortize fixed scheduling and memory loading overheads?
- ☐ **Model level:** Is the model graph compiled into an optimized static runtime engine (such as TensorRT or ONNX Runtime) to eliminate framework interpreter overhead?
- ☐ **Kernel level:** Are memory-bound operators fused into composite kernels to minimize high-bandwidth memory (HBM) round-trips?

13.11 Economics and Planning

Batching, precision tuning, operator fusion, and graph compilation can reduce latency, increase capacity, or lower the cost of a completed inference. Production deployment, however, requires answering a fleet-level question: how many machines, of what type, and at what total cost. A team that achieves 1,200 images/second on a V100 still needs to know whether 8 V100s at \$3/hour each or 24 T4s at \$0.53/hour each yields lower cost for its 5,000 QPS target while meeting the same SLO. Serving infrastructure cost grows with sustained request volume (Zhang et al. 2019), while training cost follows its own dataset, model, and optimization budget. The public API price compression shown in figure 13.2 illustrates this pressure: as per-token prices fall, infrastructure efficiency becomes a primary lever for economic viability.

13.11.1 Cost per inference

The **cost per inference** divides the allocated serving cost by the number of completed inferences that satisfy the service objective. The numerator can include provisioned CPU or accelerator time, memory capacity, data transfer, storage, and orchestration. At high utilization, useful throughput spreads that provisioned cost across many requests. At low utilization, paid capacity sits idle, so the same hourly bill is divided among fewer successful inferences. Applying the framework to ResNet-50 shows how hourly price and sustained valid throughput combine into a unit cost.

13.11.2 GPU vs. CPU economics

In the worked AWS cost analysis in section 13.11.1, GPU instances cost more per hour but deliver much higher parallel throughput. The crossover point depends on model characteristics and latency requirements.

CPU inference makes economic sense for small models with few parameters and simple operations, when latency requirements are relaxed (hundreds of milliseconds acceptable), when request volume is low or highly variable (making GPU reservation wasteful), or when the model's operations do not parallelize well. GPU inference is attractive when models are large with parallel-friendly operations, latency requirements are strict (tens of milliseconds), request volume is high and consistent enough to sustain utilization, and batching can amortize the per-inference overhead of GPU kernel launches.

Beyond steady-state costs, startup time affects scaling economics. Instance launch, image pull, driver initialization, model loading, compilation, and warmup can make accelerator replicas slower to become ready than CPU replicas, although exact times depend on the platform and deployment artifact. If traffic rises faster than useful capacity can come online, latency SLOs can fail despite sufficient eventual capacity.

When accelerator replicas have a long readiness delay, predictive scaling or a warm pool may be more reliable than purely reactive scaling. For a model that runs acceptably on both device types, a hybrid design can use always-on accelerator capacity for baseline load and CPU overflow for spikes, trading a different unit cost and latency profile for faster reserve capacity. This GPU+CPU hybrid is one instance of the broader patterns cataloged in section 2.10; whether it works depends on model compatibility and identical output semantics across runtimes.



Example 13.14: ResNet-50: Cost analysis

Table 13.22 gives an illustrative AWS-like scenario for hourly cost, throughput, and per-million-image cost when serving ResNet-50:

Table 13.22: Illustrative ResNet-50 cloud inference cost comparison: Assumed hourly cost and saturated throughput, with derived cost per one million images for CPU, T4, and V100 instances.

Instance Type	Cost/Hour	Throughput	Cost per 1M Images
c5.xlarge (CPU)	\$0.17	50 img/s	\$0.94
g4dn.xlarge (T4 GPU)	\$0.53	400 img/s	\$0.37
p3.2xlarge (V100 GPU)	\$3.06	1,200 img/s	\$0.71

Systems insight: The T4 GPU instance has the lowest unit cost under these assumptions. The V100 does not beat the T4 on unit cost at the stated full-utilization rates. Cloud pricing varies by region and changes over time; consult current pricing for production planning.

13.11.3 Capacity planning

The GPU vs. CPU decision establishes the cost per inference, but determining how much infrastructure to provision requires combining cost analysis with the queuing theory foundations from section 13.5. **Capacity planning** translates three inputs into infrastructure specifications: traffic patterns (peak request rate, daily/weekly cycles, growth projections), latency SLOs (p50, p95, p99 targets), and model characteristics (inference time distribution at various batch sizes) (Harchol-Balter 2013).

The worked example in section 13.5 demonstrates the complete workflow: starting from a 50 ms p99 SLO and 5,000 QPS target, deriving the conservative M/M/1 safe utilization threshold of 54 percent from equation 13.6, and determining GPU count with headroom of 62 V100s. This scenario provisions peak load plus 30 percent headroom; production margins depend on the workload and policy. Autoscaling can reduce costs during low-traffic periods while meeting latency objectives during peaks; chapter 14 develops the operational policy layer around these serving calculations. Throughput numbers are meaningful only when coupled with latency guarantees. As the valid-QPS accounting in section 13.5 established, capacity must be sized for requests that actually meet the SLO, not for raw request volume.

13.11.4 Production case study: Serving 8-billion-parameter Llama 3

Beyond resident weights, KV-cache memory is often one of the largest capacity costs in large-language-model serving, and figure 13.7 shows why. Plotted at 70-billion-parameter scale to amplify the effect, the cache grows linearly with context length and batch size until long contexts exhaust the memory left after loading weights and runtime workspace. The 8-billion-parameter Llama 3 profile analyzed in the rest of this section obeys the same physics with more headroom, making it a workload an engineer can fit on one GPU and reason about end to end.

13.11.4.1 Workload profile

The following fixed workload assumptions define the reference case used throughout the latency, memory, and economics calculations in this section; together, they bound the analyses that follow:

- **Model:** 8-billion-parameter Llama 3 (quantized to 4-bit using activation-aware weight quantization (AWQ); see section 10.4 for quantization techniques) (Dubey et al. 2024; Lin et al. 2024).
- **Hardware:** 1 × NVIDIA H100 SXM5 GPU (80 GB HBM3, 3.35 TB/s bandwidth) (Choquette 2023).
- **Request characteristics:** 1,000-token input prompt (Prefill), 256-token generated response (Decode).
- **Target SLOs:** TTFT < 200 ms, TPOT < 20 ms.

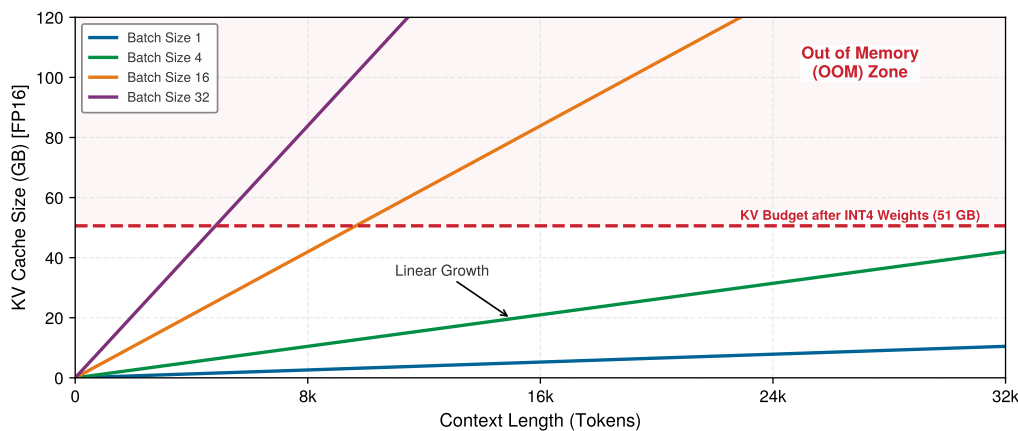


Figure 13.7: The KV-Cache Explosion: FP16 KV-cache usage vs. context length for a 70-billion-parameter-class model with GQA. On the modeled A100, 35.3 GB of INT4 weights leave 50.6 GB for KV cache before runtime overhead. Batch size 32 reaches that cache budget at roughly 4.8k tokens, forcing a trade-off between batch size and context length.

Longer contexts therefore require smaller concurrent batches or more memory, reducing throughput efficiency. These assumptions make the case study narrow enough to calculate while preserving the two serving constraints that matter most: the prefill budget for time to first token and the decode budget for each generated token.

13.11.4.2 Latency deconstruction

The end-to-end request latency is governed by the two-phase execution model of autoregressive transformers, applying the TTFT and TPOT metrics defined in section 13.8.1. Prefill determines whether the user sees a prompt response quickly; decode determines whether the generated stream keeps moving after it starts.

🔍 Systems Perspective 13.10: The physics of token generation

Recall the energy-movement invariant quantified in table 4.1: moving a bit is 100–1,000× more expensive than computing on it. In the decode phase, this law determines the physical “cost per word.”

Physics: For batch-one decode with INT4 weights, the weight-dominated arithmetic intensity is approximately 2 FLOPs per parameter divided by 0.5 byte per parameter, or 4 FLOPs/byte. This is far below the H100 ridge point, so the modeled step is memory-bandwidth bound. Larger decode batches reuse the weight stream across sequences and raise arithmetic intensity (Pope et al. 2023). The batch-one bandwidth floor is captured in equation 13.18:

$$T_{\text{token}} \approx \frac{D_{\text{vol}}}{\text{BW}_{\text{memory}}} \quad (13.18)$$

Implication: Every token generation pays an energy cost to move model weights from HBM into compute registers. On an A100 80 GB (2.04 TB/s HBM2e), the same model has a theoretical weight-streaming floor of approximately 1.97 ms per token. When decode remains bandwidth-bound, adding more compute cores yields little latency improvement; faster memory (Physics), smaller models (Algorithm), or better batching and cache management change the bound.

Prefill phase (time to first token) The model processes the 1,000-token prompt in parallel. Under the solver’s stated 40 percent compute-efficiency assumption, the model-side TTFT estimate is 41.9 ms, comfortably within the 200 ms SLO and leaving the remaining budget for network ingress, tokenization, and queueing.

Decode phase (time per output token) The model generates 256 tokens sequentially. At batch one, this phase is memory-bandwidth bound: each step streams the 4 GB weight tensor from VRAM. A batched step amortizes that stream across its active sequences.

Dividing the 4 GB weight tensor by the 3.35 TB/s bandwidth yields the theoretical floor: $T_{\text{token}} \approx 1.2$ ms. Adding the solver’s per-layer dispatch tax gives a modeled TPOT of 1.57 ms. This dispatch overhead reflects the cumulative cost of launching 8–10 discrete GPU kernels (RMSNorm, RoPE, projections, attention, multilayer perceptron (MLP)) across each of the model’s 32 layers. At batch size one, launching \$250–300 small kernels adds several milliseconds of CPU-GPU submission and driver overhead, which serving engines mitigate by capturing the decode graph into a single CUDA Graph. Generating all 256 tokens therefore takes $256 \text{ tokens} \times 1.57 \text{ ms} = 0.40 \text{ s}$.

13.11.4.3 Memory and throughput

With 4-bit weights occupying 4 GB, the remaining ~81.9 GB is available for the KV cache and runtime workspace. Each token requires approximately 0.131 MB of FP16 KV cache in the 8-billion-parameter Llama 3 configuration; while AWQ compresses weights to 4 bits, it leaves the KV cache in 16-bit precision. This per-token footprint derives directly from the architecture: storing 16-bit Key and Value tensors across $L = 32$ layers with $N_{\text{kv}} = 8$ grouped-query attention heads of dimension $d_{\text{head}} = 128$ requires $2 \times 32 \times 8 \times 128 \times 2 \text{ bytes} = 128 \text{ KB}$ per token per sequence, cutting cache volume to one-fourth that of standard 32-head multi-head attention. Dividing the 72 GB reserve by that per-token cost yields capacity for ≈ 0.6 million tokens, so at 1,256 tokens the GPU can hold a concurrent batch size of ~469 requests.

13.11.4.4 Unit economics

Consider a representative H100 SXM5 rental cost of approximately \$3/hour. Taking the minimum of isolated prefill capacity and KV-residency capacity gives an optimistic throughput ceiling of 45.2

million tokens/hour and a lower-bound unit cost of \$0.066/million tokens. This calculation does not model prefill/decode resource contention or batched-decode capacity.

For large language models, memory capacity (the size of the unallocated VRAM dedicated to the KV cache) determines maximum concurrent residency, while prefill compute and decode bandwidth determine realized token throughput and cost under a specific traffic mix. Memory bandwidth bounds low-batch decode latency; larger batches amortize parameter movement and shift the limit toward compute or KV-cache traffic.

Physical constraints govern serving economics across the entire stack: latency budgets partition into prefill and decode phases, queueing theory dictates batch sizing and capacity headroom, and memory bandwidth bounds decode throughput. Misinterpreting these interactions leads directly to the operational fallacies and pitfalls that follow.

13.12 Fallacies and Pitfalls

Serving inverts training priorities in ways that violate intuitions from batch processing. The nonlinear relationship between utilization and latency, the hidden costs of preprocessing, and the silent failure modes of training-serving skew cause violated SLOs, wasted optimization effort, and accuracy degradation invisible to standard monitoring.

Fallacy: *Reducing model inference latency proportionally reduces user-perceived latency.*

Engineers who optimize model inference expect proportional improvement in user-perceived latency, but serving systems introduce latency sources absent from offline benchmarks. Under load, queuing delay dominates: equation 13.5 shows that at 80 percent utilization with 5 ms service time, average wait time is 20 ms before inference even begins. Reducing inference from 5 ms to 2 ms changes service time but also shifts utilization from 80 percent to 32 percent, reducing queuing wait from 20 ms to 0.9 ms, a 21.2× queuing improvement that dwarfs the 3 ms inference gain. This nonlinear interaction between inference speed and queuing behavior means the *system-level* speedup (25 ms → 2.9 ms, or 8.5×) far exceeds the *model-level* speedup (5 ms → 2 ms, or 2.5×). Even a 20 percent service-time reduction can produce a large user-facing improvement at high utilization because it lowers utilization as well as direct compute time. Serving optimization requires analyzing the complete latency budget, including serialization, queuing, preprocessing, and postprocessing, under realistic load conditions rather than profiling inference latency in isolation.

Pitfall: *Running serving infrastructure at high utilization to maximize cost efficiency.*

Teams target 90 percent utilization to minimize idle capacity. In production, latency degrades nonlinearly as utilization approaches capacity. Equation 13.5 shows that at 90 percent utilization, average time in system reaches 10× service time. Moving from 70 percent to 90 percent utilization cuts modeled infrastructure cost by 22.2 percent but triples average latency. For a 5 ms inference service, modeled p99 latency rises from ~76.7 ms to ~230 ms under the M/M/1 assumptions. In this scenario, operating at 60 to 70 percent utilization preserves substantially more latency headroom than 90 percent. A production target must instead be selected from the measured arrival process, service-time distribution, SLO, failure reserve, and cost objective.

Fallacy: *Training accuracy guarantees serving accuracy.*

Engineers assume identical model weights preserve validation-set performance. In production, preprocessing differences can shift inputs outside the training distribution. Section 13.6.1 explains how resize interpolation, numerical precision, and feature timing can create skew despite identical weights. The illustrative scenario here drops from 95 percent to 90 percent, a 5 percentage-point loss; these values are not a reported production incident. Production systems require consistent preprocessing and task-quality monitoring because latency and exception checks cannot detect this failure.

Pitfall: *Using average latency to evaluate serving system performance.*

Engineers monitor average latency because it trends smoothly and is simple to compute. An average, however, cannot show whether the percentile named by an SLO is failing. As section 13.5.5 demonstrates, at 70 percent utilization with 5 ms service time, average latency is 16.7 ms but the

modeled p99 reaches 76.7 ms, a 4.6× gap invisible to mean-based monitoring. Services therefore report the relevant latency distribution and SLO percentile rather than the mean alone.

Fallacy: *Larger serving batches always improve throughput without affecting latency SLOs.*

Engineers maximize batch size assuming GPU saturation improves cost efficiency under production load. In serving systems, however, batching introduces a latency-throughput trade-off governed by queuing dynamics absent from offline benchmarks. Accumulating requests into larger batches increases wait time for early arrivals: a batch window of 10 ms means the first request waits 10 ms before inference begins, directly adding to p99 latency. In the representative ResNet-50/V100 scenario in section 13.7.6, increasing batch size from 16 to 32 improves throughput only 12 percent but nearly doubles per-batch inference time from 14 ms to 25 ms, and variable input sizes within a batch can create padding overhead that wastes compute on padding tokens. Section 13.7.3 shows why, for tight p99 targets, larger batch sizes can violate SLOs when batch formation delay plus increased per-batch inference time exceeds the latency budget. Serving batch optimization requires jointly tuning batch size, batch timeout, and concurrency against latency SLOs under realistic traffic patterns, not maximizing throughput in isolation.

Pitfall: *Calibrating quantized models with unrepresentative data.*

Teams may calibrate with a convenient sample even when it does not represent the supported deployment distribution. Post-training quantization determines INT8 scale factors from activation ranges in calibration data, so representative coverage matters. The illustrative scenario here drops from 76.1 percent to 72.9 percent, a 3.2 percentage-point loss; it is not a reported production incident. Effective quantization requires calibration and validation on data representative of the inputs the service is expected to handle.

Fallacy: *Cold start latency only matters for the first request.*

Engineers optimize steady-state latency assuming most requests hit warm instances. Cold starts can compound during traffic spikes, deployments, and recovery. In this illustrative scenario, 10 instances each require 30 s of compilation, totaling 300 instance-seconds; parallel warming makes capacity useful after about 30 s. The assumed cold-request latency is 500 ms versus 5 ms at steady state.

Pitfall: *Scaling without a warm-pool or staged-loading budget.*

Autoscaling policies that count only steady-state replicas underestimate the capacity needed during traffic spikes and deployments. Serving systems need warm pools (pre-initialized spare replicas), staged model loading, or admission control so that new replicas become useful before user requests depend on them. The budget should include compilation, weight loading, cache initialization, and health-check time, because those steps determine whether scale-out adds capacity or adds another source of tail latency.

13.13 Summary

Serving moves a validated model into an operating workload. Online requests add a latency distribution, queueing, traffic variability, and failure reserve to the throughput concerns familiar from training. Little's Law relates stable long-run population, arrival rate, and response time; M/M/1 supplies an intentionally simple model of nonlinear latency growth near saturation. Neither replaces measurement, but together they turn headroom and capacity into explicit hypotheses that a load test can validate.

Effective serving optimization measures the complete request path rather than model inference alone. Protocol and serialization choices matter when transport is a visible fraction of the budget; preprocessing matters when it feeds an accelerator too slowly; batching matters only when arrivals and the SLO permit waiting. Training-serving skew adds a different failure mode: the service can remain fast and error-free while inconsistent transformations silently change its predictions.

The deployment paradigm selected in section 2.1 changes the feasible request path. A cloud API may use dynamic batching under a measured arrival process, synchronized sensors impose a frame deadline, and a single-stream mobile workload may choose batch one to protect responsiveness and

energy. The MLPerf scenarios provide standardized approximations of these shapes; production design still begins with the service's own traffic, hardware, and objective.



Key Takeaways: Inverting every training priority

- **Serving is latency economics:** Training rewards throughput over long runs, but serving spends a fixed per-request budget across serialization, preprocessing, queuing, inference, postprocessing, and the network. Optimizing only model latency misses the stages users actually wait on.
- **Utilization turns into waiting:** In the chapter's M/M/1 approximation, average time in system rises from $5 \times$ service time at 80 percent utilization to $10 \times$ at 90 percent. Headroom keeps modest traffic surges from becoming SLO failures.
- **Fast models reveal pipeline taxes:** Once inference is fast, image decode, tokenization, and other preprocessing can dominate total latency. The binding optimization becomes the request path, not the neural network kernel.
- **Batching follows traffic, not habit:** Poisson-modeled web arrivals can use dynamic batching, synchronized sensors need aligned batches, and single-stream mobile workloads may have little batching opportunity. A useful window converts available latency slack into throughput without violating the SLO.
- **Skew breaks accuracy without errors:** Resize methods, normalization order, calibration data, or feature definitions that differ between training and serving can change the model's effective input distribution. Shared transformations, parity tests, and production-slice monitoring reduce this risk.
- **LLM serving is memory management:** Decode often reads weights from VRAM for every generated token, so token latency is bandwidth-bound unless batching changes the constraint. KV-cache layout, PagedAttention, continuous batching, precision, and runtime choice determine both concurrency and cost per token.
- **Runtime choices become infrastructure bills:** Precision, graph compilation, operator fusion, and serving runtime translate directly into replica count and cost per inference. Quantization and specialized runtimes can materially reduce required serving capacity when they preserve accuracy and fit the target hardware.

Node-level optimization connects the model's operations to the machine that executes them. Profiling reveals whether compilation, operator fusion, precision, locality, serialization, or loading actually binds the request path. Gains matter only when they survive production input shapes, accuracy checks, and latency variation. Capacity planning converts that valid throughput into replica count and unit cost, as the Llama 3 case study in section 13.11.4 demonstrates. A faster kernel that neither reduces provisioned capacity nor improves the SLO has not produced an economic gain.

Training is judged by useful work completed; an online service is judged by whether requests meet their latency objective. Each stage spends against the same envelope: serialization, preprocessing, the queue, the model, and the response. Data determines input size and arrival shape, the algorithm determines compute and state, and the machine determines service rate and movement costs. Queueing couples all three nonlinearly, so a system within budget at moderate traffic can breach it after a small surge. Serving engineering therefore allocates the budget across the request path, preserves headroom for variability and failure, and verifies that the required fraction of requests finishes before the deadline.

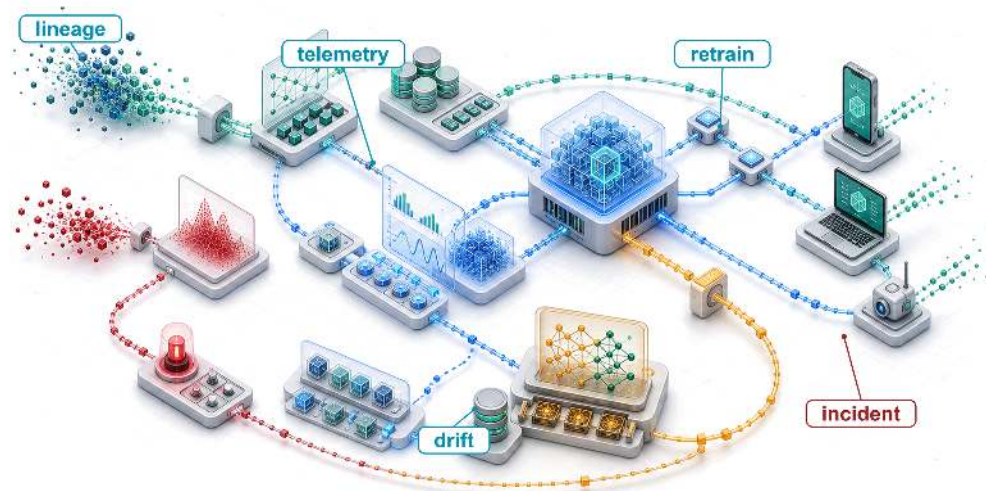


What's Next: From node to factory

A single serving node is fragile: models drift, updates must reach users without interruption, and replicas must behave as one dependable system. Chapter 14 expands the unit of concern from a request to the production factory: CI/CD governs artifacts; model registries and feature

stores align serving with training; observability detects latency and accuracy drift; and rollback prevents bad releases from becoming permanent outages.

ML Operations

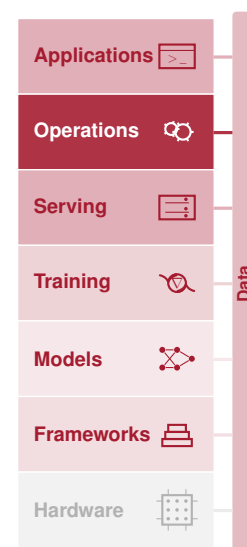


14.1	MLOps Overview
14.2	Principles and Foundations
14.3	Technical Debt
14.4	Development Infrastructure
14.5	Production Operations
14.6	Design and Maturity Framework
14.7	Case Studies
14.8	Fallacies and Pitfalls
14.9	Summary

Purpose

Why can an ML system be perfectly available and perfectly wrong at the same time?

Conventional monitoring often detects crashes, latency regressions, and availability failures quickly, but semantic errors can remain silent in any software system. ML systems add another silent failure mode. A model experiencing data drift can continue serving ordinary-looking predictions while accuracy degrades, triggering no alerts because every health check (latency, throughput, uptime) remains green. Serving infrastructure gets models into production; operations helps keep them correct once they are there. Model quality can degrade because the world changes. Customer behavior shifts, new product categories appear, seasonal patterns evolve, and the distribution the model learned from diverges from the one it now faces. Drift is not a certainty but a persistent risk for deployed models. Unlike latency or uptime, correctness may become observable only after labels or downstream outcomes arrive, so detecting degradation often requires imperfect signals and explicit investigation rather than a single threshold. Managing it requires continuous monitoring that tracks prediction quality alongside system health, drift signals that trigger investigation, retraining when outcome evidence supports it, and deployment strategies that validate new model versions against production traffic before full rollout. The gap between development and production is not a hurdle cleared once but a condition managed indefinitely. Machine learning operations exists because uptime without measured accuracy may still deliver wrong answers at scale. It makes D·A·M co-design continuous because the data environment can remain a moving target long after initial deployment.





Learning Objectives

- Explain why ML systems can remain available while prediction quality silently degrades under distribution shift
- Diagnose technical debt across data-model, model-infrastructure, and production-monitoring interface boundaries
- Design feature stores, registries, and CI/CD pipelines that preserve training-serving consistency and reproducible rollback
- Apply the retraining staleness model to choose cost-aware retraining triggers and intervals
- Implement layered monitoring for drift, skew, degradation, business metrics, and data freshness
- Compare canary, blue-green, shadow, and rollback strategies for production model release risk
- Evaluate operational maturity and investment using model criticality, operational risk, and organizational readiness

14.1 MLOps Overview

AFTER a model is built, optimized, benchmarked, and served, the system still has to remain correct. A benchmark establishes performance at a point in time; serving infrastructure answers requests in milliseconds. The team deploys to production, and week one looks excellent. The challenge begins in week two.

Data distributions shift, user behavior changes, and the world moves on from the conditions under which the model was trained. ML models that succeed in development can fail to achieve sustained production use when postdeployment ownership and monitoring are absent. The root cause is an operational mismatch: availability-focused monitoring tracks server uptime, request latency, and request success rates, while ML monitoring must also track statistical health, including accuracy over time, input-distribution shift, and per-segment prediction quality. A model can degrade substantially while throwing no exceptions, triggering no infrastructure alarms, and maintaining perfect uptime.

The discipline that makes these invisible failures visible is **machine learning operations (MLOps)**. MLOps synthesizes monitoring, automation, and governance into production architectures that detect degradation, trigger retraining, and maintain system health throughout a model's operational lifetime. It inherits the automation and operations lineage of DevOps (Kreuzberger et al. 2023), but addresses a different failure mode. Conventional services can often be tested against deterministic code paths, while ML systems depend on training data distributions, learned parameters, and environmental conditions that shift continuously.

The week-two problem takes concrete shape in an illustrative deployment scenario. Consider a demand prediction system for a ridesharing service. Initial measurements show 94 percent accuracy, 15 ms p99 latency, and strong performance across test segments. By week four, accuracy has dropped to 88 percent, but the infrastructure metrics show nothing wrong. By week eight, a product manager notices driver dispatch is inefficient; investigation reveals the model has not adapted to a competitor's new promotion that shifted user behavior. The model needed retraining six weeks ago, but no system was watching for this degradation. MLOps provides the framework to detect such drift, trigger retraining, and validate new models before users experience the impact.

The operational mismatch connects directly to the book's analytical foundations. If benchmarking provides the *sensors* for the system, MLOps is the complete *control system*. It closes the verification gap of the verification-gap equation (equation 1.1) by continuously recalibrating against a changing world. MLOps can operationalize a locally fitted degradation equation (equation 1.3) when paired drift and outcome data show that distribution change predicts accuracy loss; divergence alone does not prove inevitable decay. It also formalizes interfaces and responsibilities across traditionally isolated domains (data science, machine learning engineering, and systems operations (Amershi et al. 2019)) through continuous retraining, A/B evaluation, graduated rollout, and standardized artifact tracking that supports reproducibility and auditability.

Deploying, monitoring, and maintaining one production ML system defines the fundamental operational unit: the **ML node**, a complete system comprising data pipelines, feature computation, model training, serving infrastructure, and monitoring for a single machine learning application. Platform operations at larger scale (managing hundreds of models, cross-model dependencies, multi-region coordination, and organization-wide ML platform engineering) constitute advanced topics that build on these single-model foundations.

Managing a production ML node requires preserving observability across each interface. Technical debt accumulates when these boundaries erode; feature stores, CI/CD pipelines, and experiment tracking establish the reproducibility required to isolate faults. With reproducible data and artifacts in place, continuous monitoring, drift detection, and canary deployment protect predictive quality against environmental shift.

The single-model operational challenge decomposes into three distinct interfaces. The **data-model interface** is the handoff between data infrastructure and model training; its goal is feature consistency, so training and serving pipelines compute features the same way. The **model-infrastructure interface** is the transition from trained weights to scalable service; its challenge is environment parity, because a model that works in a notebook may fail in production due to version, dependency, or runtime mismatches. The **production-monitoring interface** is the feedback loop that enables self-correction, returning statistical telemetry¹ from production to training because ML systems can degrade through drift without crashing.

These interfaces determine how infrastructure components partition responsibility. Feature stores stabilize feature computation at the data-model boundary. Model registries and deployment pipelines preserve the model-infrastructure handoff. Drift monitors, retraining triggers, and governance policies close the production-monitoring loop before silent degradation becomes a business failure.

Operating this feedback path requires distinguishing MLOps from traditional DevOps, establishing the foundational principles that govern production decisions, and isolating the debt patterns that accumulate when those boundaries fail.

14.2 Principles and Foundations

A production ML release extends beyond a code diff: data distributions, learned parameters, evaluation slices, and monitoring feedback loops all become release objects that alter runtime behavior. MLOps builds on DevOps to manage these data-dependent demands across development and deployment (Kreuzberger et al. 2023; Amershi et al. 2019). While traditional CI/CD pipelines validate deterministic source code, configurations, unit tests, and infrastructure manifests, ML operations must govern artifacts whose statistical validity depends on the data lineage and execution environment that produced them.

DevOps automates the integration and delivery of deterministic software. MLOps extends this discipline to statistical, data-dependent workflows spanning data acquisition, preprocessing, distributed training, evaluation, deployment, and continuous monitoring through an iterative loop connecting system design, model development, and live operations.



Definition 14.1: MLOps

Machine learning operations (MLOps) is the engineering practice of productionizing ML systems through CI/CD, workflow orchestration, reproducibility, versioning, collaboration, continuous training and evaluation, monitoring, and feedback loops (Kreuzberger et al. 2023).

1. **Significance:** The cost of leaving this loop open manifests as stale predictions, silent accuracy degradation, and prolonged recovery times. Drift thresholds, retraining triggers, and mean time to recovery (MTTR) targets are deployment-specific quantities calibrated from prediction value, label delay, validation risk, and retraining compute expense. The central engineering discipline is the closed control loop: measure distribution shift, estimate staleness cost, trigger retraining only when expected accuracy gains outweigh execution cost and rollout risk, and validate replacement models before promotion.

¹ | **Telemetry:** The feedback path that can make model degradation visible before it becomes a business failure; crashes and error codes expose some software failures, but semantic errors can remain silent in any application. ML systems add distribution shift, which can go undetected without statistical telemetry such as feature distributions, prediction confidence, and drift indicators. Availability metrics alone would not expose this degradation.

2. **Distinction:** DevOps monitors functional correctness, system throughput, and infrastructure availability. MLOps introduces statistical controls for *predictive correctness*, which can decay silently while server uptime, request latency, and HTTP status codes remain completely healthy.
3. **Common pitfall:** A frequent misconception is that retraining on new data automatically resolves distribution shift. Retraining without diagnosing which distribution shifted—input features ($p(x)$), label conditionals ($p(y | x)$), or both—preserves underlying failure modes or amplifies collection bias. Under covariate shift and common support, sample reweighting or stratified resampling restores calibration; concept drift requires fresh labels and model adaptation.

Trace the infinity-loop structure in figure 14.1 to see how operational telemetry feeds empirical evidence back into design and engineering, forming the continuous operating shape of the discipline:

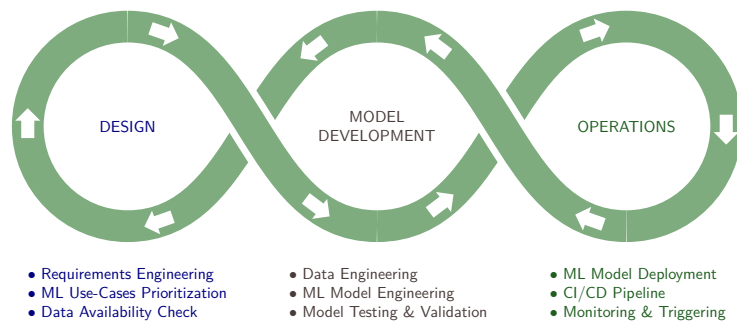


Figure 14.1: Iterative MLOps Loop: Monitoring and triggering feed production evidence back into design and model development, turning requirements, engineering, testing, and deployment into a continuous operating cycle.

The operational complexity and financial risk of deploying machine learning without disciplined operational tooling emerges clearly in a recommendation service. A newly deployed ranking model initially improves conversion, but gradual data drift steadily degrades prediction quality. Because conventional monitoring tracks server uptime, request latency, and HTTP success rates rather than inference accuracy, the loss remains invisible until an aggregate revenue review months later. MLOps supplies the operational controls that expose these silent statistical failures before they compound into material financial harm.

14.2.1 Foundational principles

That recommendation failure illustrates a systemic pattern: without disciplined operational controls, even accurate models fail silently in production. Debugging such an incident requires a structured sequence of operational checks. When revenue drops, the first priority is reconstructing the deployed model state, which requires reproducibility. The next priority is determining whether data pipelines, training jobs, serving paths, and monitoring services maintain clean boundaries to isolate the defect, which requires separation of concerns. If online serving features diverge from offline training features, consistency provides the invariant that prevents recurring skew. If drift begins before users complain, observable degradation converts a silent failure into an actionable alert. Finally, when retraining is viable but computationally expensive, cost-aware automation evaluates whether intervention justifies the compute expenditure and operational rollout risk. These principles define the controls in the operational sequence an engineering team executes to isolate and resolve production failures.

14.2.1.1 Reproducibility

Reproducibility requires every artifact² that influences model behavior to be versioned and traceable. This principle extends beyond code repositories to encompass input datasets, feature schemas, hy-

² | **Artifact:** Model weights are realized training outputs; the inputs and randomness that produced them cannot generally be inferred from the parameters. Versioning only code is therefore insufficient, and even versioning code, data, configuration, and environment may not guarantee exact replay when kernels or training are nondeterministic.

perparameter configurations, and execution environments. Equation 14.1 expresses this dependency formally:

$$\text{Model Output} = f(\text{Code}_v, \text{Data}_v, \text{Config}_v, \text{Environment}_v; \xi) \quad (14.1)$$

where v identifies an immutable version hash and ξ captures execution nondeterminism. Exact reproduction requires freezing all four versioned inputs alongside random seeds, dataloader sample ordering, deterministic GPU reduction kernels (preventing non-associative floating-point drift from asynchronous atomic additions), and identical accelerator hardware architectures.

14.2.1.2 Separation of concerns

Separation of concerns decomposes MLOps systems into distinct functional layers that can evolve independently, as table 14.1 shows:

Table 14.1: MLOps Separation of Concerns: Each layer addresses a distinct responsibility and evolves at different rates across the data, training, serving, and monitoring layers. This separation enables independent scaling and updates, reducing blast radius when changes are required.

Layer	Responsibility	Stability
Data Layer	Feature computation, storage, serving	Changes with data schema evolution
Training Layer	Model development, hyperparameter optimization	Changes with algorithm research
Serving Layer	Inference, scaling, latency management	Changes with traffic patterns
Monitoring Layer	Drift detection, performance tracking	Changes with business requirements

14.2.1.3 Consistency imperative

The architectural boundaries in table 14.1 allow teams to upgrade serving runtimes without retraining models, adjust alert thresholds without redeployment, and modify data ingestion pipelines while preserving interface contracts. That architectural independence remains safe only when training and serving environments process features identically, establishing training-serving parity as a consistency imperative. The financial consequence of this discrepancy is captured in equation 14.2:

$$\text{Skew Cost} = \text{Rate}_{\text{skew}} \times Q \times C_{\text{error}} \quad (14.2)$$

where $\text{Rate}_{\text{skew}}$ is the fraction of queries affected by training-serving skew, Q is the total query volume per accounting period, and C_{error} is the average business cost incurred per erroneous prediction.

For a system serving 1,000,000 queries/day with 1 percent skew-induced errors costing \$0.10 each, annual skew cost reaches \$365,000. This quantifies why consistency mechanisms represent investments with measurable financial returns. Shared feature stores, unified C++ or Rust feature-extraction libraries invoked identically during offline training and online serving, and automated schema validation tests enforce this parity directly at the system boundary.

14.2.1.4 Observable degradation

Observable degradation requires ML systems to make silent failures visible through continuous measurement. Model accuracy degrades along a continuous statistical spectrum rather than failing as a discrete binary crash. An observed failure's time signature—whether an acute drop, gradual drift, or localized cohort decay—governs both the detection mechanism and the operational response.

14.2.1.5 Cost-aware automation

Cost-aware automation balances computational costs against accuracy improvements. Let $\Delta \text{Accuracy}$ be the expected gain in accuracy percentage points, Value per Point the monetary value of one percentage-point gain over the decision horizon, Training Cost the compute and labor cost of a retraining run, and Deployment Risk the expected cost of validation and rollout failure. Equation 14.3 models this trade-off:

$$\text{Retrain if: } \Delta \text{Accuracy} \times \text{Value per Point} > \text{Training Cost} + \text{Deployment Risk} \quad (14.3)$$

The inequality acts as an operational decision gate rather than an unconstrained trigger: an automated pipeline should initiate retraining only when empirical evidence indicates that expected

accuracy gains surpass execution cost and rollout risk. This economic trade-off governs retraining triggers, validation thresholds, and rollout gates; section 14.4.2.2 formalizes this model to calculate optimal retraining cadences.

Detecting when to trigger intervention requires pairing each failure signature with the appropriate telemetry and operational countermeasure, as outlined in table 14.2:

Table 14.2: Degradation Detection Strategies: Four failure signatures map to detectors and operational responses. Sudden drops require immediate diagnosis and may call for rollback, while gradual or subgroup degradation calls for diagnosis before retraining or targeted data collection.

Degradation Type	Detection Mechanism	Response Strategy
Sudden accuracy drop	Threshold alerts	Diagnose; roll back if release-related
Gradual drift	Trend analysis	Diagnose, then retrain if warranted
Subgroup degradation	Cohort monitoring	Targeted data collection
Latency increase	Percentile tracking	Infrastructure scaling

Each foundational principle becomes operational only when anchored to a concrete, measurable metric (table 14.3)—from artifact hashes to net retraining value:

Table 14.3: MLOps Principles Summary: Quick reference for the five foundational principles that guide all MLOps tooling and practice decisions.

Principle	Core Insight	Key Metric
Reproducibility	Version all artifacts	Complete artifact hash
Separation of concerns	Independent layer evolution	Layer coupling score
Consistency	Training equals Serving	Feature skew rate
Observable degradation	Make failures visible	Time to detection
Cost-aware automation	Optimize total cost	Net retraining value

How these principles manifest in practice depends on the workload. A recommendation system drifts daily as user preferences shift; a TinyML model deployed on embedded hardware may run unchanged for months. The monitoring strategy must match the archetype.



Lighthouse 14.1: Monitoring strategy by archetype

The dominant failure modes and monitoring priorities differ across workload archetypes. Table 14.4 compares four representative archetypes by drift pattern, monitoring metric, and example retraining trigger:

Table 14.4: Monitoring Strategy by Workload Archetype: Illustrative starting points for monitoring strategy. Real thresholds must be calibrated to the deployment’s label delay, traffic volume, business risk, and alert-fatigue budget.

Archetype	Dominant Drift Pattern	Primary Monitoring Metric	Example Retraining Trigger
ResNet-50 (Compute Beast)	Visual distribution shift (lighting, camera, new object classes)	Accuracy on holdout set (ground truth available)	Accuracy drops > 2% from baseline (~monthly for stable domains)
GPT-2 (Bandwidth Hog)	Vocabulary drift, topic shift, emerging entities	Perplexity on live traffic (no ground truth needed)	Perplexity increases > 10%; new vocabulary detected (~weekly for news domains)
DLRM (Sparse Scatter)	User behavior shift, item catalog churn, cold-start items	CTR/CVR delta vs. historical cohorts	Engagement drops > 5%; catalog refresh (~daily for e-commerce)
DS-CNN (Tiny Constraint)	Acoustic environment change (noise floor shift)	Duty cycle (wakeups/hour) + false positive rate	False wake rate > 1%; battery drain exceeds spec (~quarterly OTA update)

Systems insight: Ground truth availability and physical bottlenecks govern monitoring design. Compute-bound vision models (ResNet-50) bound by $O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$ track holdout accuracy; memory-intensive language and recommendation models (GPT-2, DLRM) bound by D_{vol}/BW may rely on perplexity or implicit clicks; microcontroller models (DS-CNN) bound by strict latency L_{lat} and energy limits monitor duty cycles and false wake rates. Retraining cadence is constrained by label delay, deployment access, validation cost, and the rate of observed change.

These principles respond to recurring operational failures: concept drift,³ data-quality defects (Schelter et al. 2018), and silent postdeployment degradation. These factors drive the divergence between MLOps and traditional DevOps: system health cannot be measured by uptime or latency alone. Operational discipline requires monitoring the statistical properties of data distributions and model outputs, shifting the focus from basic server availability to predictive correctness.

MLOps coordinates a broader stakeholder ecosystem and introduces specialized practices such as data versioning,⁴ model versioning, and continuous statistical monitoring that extend beyond traditional software engineering. This expanded scope turns model operation into an ongoing feedback loop rather than a linear release pipeline, as compared across dimensions in table 14.5:

Table 14.5: MLOps vs. DevOps: MLOps extends DevOps principles to address the unique requirements of machine learning systems, including data and model versioning, and continuous monitoring for model performance and data drift. MLOps coordinates a broader range of stakeholders and emphasizes reproducibility and scalability beyond traditional software development workflows.

Aspect	DevOps	MLOps
Objective	Streamlining software development and operations processes	Optimizing the lifecycle of machine learning models
Methodology	Continuous Integration and Continuous Delivery (CI/CD) for software development	Similar to CI/CD but focuses on machine learning workflows
Primary Tools	Version control (Git), CI/CD tools (Jenkins, Travis CI), Configuration management (Ansible, Puppet)	Data versioning tools, Model training and deployment tools, CI/CD pipelines tailored for ML
Primary Concerns	Code integration, Testing, Release management, Automation, Infrastructure as code	Data management, Model versioning, Experiment tracking, Model deployment, Scalability of ML workflows
Typical Outcomes	Faster and more reliable software releases, Improved collaboration between development and operations teams	Efficient management and deployment of machine learning models, Enhanced collaboration between data scientists and engineers

³ | **Concept drift:** Concept-drift and data-stream research formalized the problem that a model’s target relationship can change after deployment (Widmer and Kubat 1996; Gama et al. 2014). In adversarial domains such as spam, fraud, and abuse detection, the distribution can actively adapt in response to the model, making continuous monitoring and retraining a structural requirement rather than an operational luxury.

⁴ | **DVC (Data Version Control):** DVC brings Git-like versioning to datasets and model artifacts (Iterative 2024), solving the artifact gap that equation 14.1 formalizes: without data versioning, the Data_v term is unrecoverable, and no combination of code commits can reconstruct the model that was deployed.

 Checkpoint 14.1: The MLOps loop

MLOps is not linear; it is circular.

The feedback cycle

- ☐ **Closing the loop:** how do production metrics (for example, drift alerts) trigger new training cycles?
- ☐ **Retraining trigger:** can you distinguish a drift signal that warrants investigation from outcome evidence that warrants retraining?

The artifacts

- ☐ **Versioning:** are you versioning Data + Code + Model + Environment together?

Each iteration through the operational loop can introduce data dependencies, model interactions, and configuration drift invisible to standard software testing. Those accumulating costs constitute technical debt: an engineering framework that converts silent statistical failure modes into quantifiable liabilities.

While traditional DevOps addresses deployment throughput and service scaling, MLOps must isolate and service this technical debt before it destabilizes the serving system. Boundary erosion

demonstrates why modular pipeline design is mandatory; correction cascades reveal why strict artifact versioning and instantaneous rollback are essential; undeclared consumers justify enforced feature schemas and contract tests. These debt patterns provide the diagnostic vocabulary and concrete failure modes that motivate the specialized MLOps infrastructure examined throughout the remainder of this book.

14.3 Technical Debt

Silent failure modes in production ML manifest concretely as technical debt (Sculley et al. 2015): upstream distribution shifts, feedback loops, and unversioned pipeline dependencies cause accuracy degradation that compounds over time. These failures accumulate invisibly across system boundaries, requiring operational architectures that account for statistical behavior and data dependencies alongside traditional software contracts. Originally proposed in software engineering in the 1990s,⁵ the technical debt metaphor compares shortcuts in implementation to financial debt, trading short-term delivery velocity for ongoing maintenance overhead and systemic risk (Cunningham 1992). In ML systems, this debt extends beyond executable code to encompass statistical dependencies and unvalidated data flows. Systematic evaluation rubrics, such as the ML Test Score (Breck et al. 2017), provide frameworks for quantifying this debt and assessing production readiness across data, model, and infrastructure interfaces.

⁵ | **Technical debt:** Ward Cunningham’s 1992 WyCash experience report introduced the debt metaphor for expedient code and delayed consolidation (Cunningham 1992); in ML, debt can compound silently through data and model dependencies that code-focused unit tests and reviews may not detect. A pipeline can remain unchanged while the world it models changes. The ML Test Score rubric (Breck et al. 2017) makes this debt explicit through 28 production-readiness tests grouped into data, model, infrastructure, and monitoring sections.

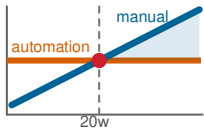
Definition 14.2: Technical debt in ML

Technical debt in machine learning is the accumulating maintenance and change cost created by implicit data dependencies, entangled features, and undeclared consumers in ML systems, where the resulting liabilities may also appear as silent accuracy degradation.

1. **Significance:** Google’s analysis of production ML systems argues that model code is only a small fraction of the surrounding system; the larger operational surface includes data collection, feature extraction, configuration, serving infrastructure, monitoring, and process management (Sculley et al. 2015). ML-specific debt drivers compound this: changing one input feature can silently shift the learned representation of every other feature (entanglement), a model trained to correct another model’s errors creates a fragile dependency chain (correction cascades), and downstream systems consuming model outputs without explicit contracts become undeclared consumers that break silently when the model is updated.
2. **Distinction:** ML technical debt includes system- and data-level dependencies that code review and ordinary code-focused tests may miss. It can slow development and degrade prediction quality even while availability metrics remain healthy.
3. **Common pitfall:** A frequent misconception is that “better code” solves technical debt in ML. In reality, it is a systems architecture problem: the debt accumulates when the assumptions of the training distribution (feature ranges, label meanings, data freshness) are not enforced as runtime contracts at the system boundary.

A break-even calculation makes the cost dynamics of technical debt concrete. Teams often resist automation investment because manual processes seem faster in early development, but that advantage disappears once recurring manual interventions exceed the up-front automation cost.

Manual operations quickly hit a capacity ceiling because modeling code constitutes only a small fraction of a real-world deployment. As shown in figure 14.2, the broader operational surface spans ten interconnected components, from data verification to machine resource management, each capable of accumulating hidden liabilities. While core ML code occupies a single box, the remaining nine demand continuous operational upkeep.



Cumulative manual work overtakes the one-time automation investment near week 20.

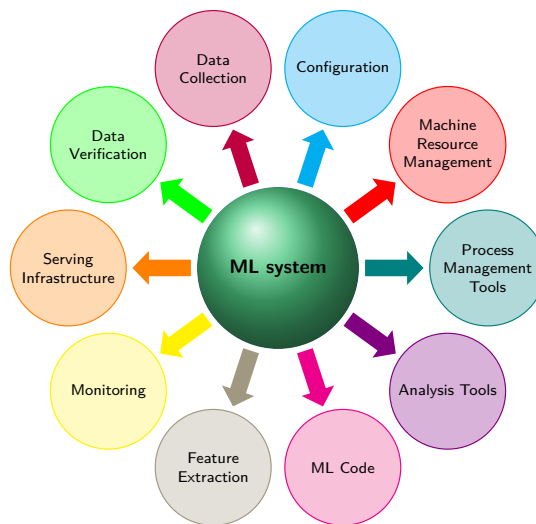


Figure 14.2: Hidden Infrastructure of ML Systems: ML code is one of ten components arranged around the ML system at the center, alongside data collection, data verification, feature extraction, configuration, machine resource management, serving infrastructure, monitoring, analysis tools, and process management tools. The count is what matters, since modeling occupies one box and the remaining nine are the operational surface a deployment must also carry. Source: (Sculley et al. 2015).

Napkin Math 14.1: The cumulative cost of manual operations

Problem: Why build automated pipelines when manual retraining is faster?

Physics: Manual work accumulates over time.

- Manual retrain: 4 hours of engineering per week.
- Pipeline build: 80 engineering hours (one-time).

Math:

- **Result:** $80 \text{ engineering hours} \div 4 \text{ hours per week} = 20 \text{ weeks}$.
- **Trap:** This assumes the model never changes.
- **Limitation:** Additional features can increase manual effort.
- **Consequence:** After 1 year, manual teams still spend 4 hours per week on maintenance. In this simplified calculation, pipeline teams incur 0 recurring hours.

Context: A central law of systems engineering is that the cost of *maintaining* a system over its lifetime can dominate the cost of *building* it. In ML, technical debt is especially dangerous because it is often *data-driven* rather than *code-driven*: a perfect piece of code can still fail if the data it processes shifts. Measurement is the management boundary: without telemetry, the team cannot tell whether maintenance work is reducing debt or merely hiding it.

Systems insight: Automation addresses capacity limits, not speed alone. A manual team eventually reaches a point where maintaining existing models prevents it from deploying new ones. MLOps responds with systematic observability and automation. Without monitoring infrastructure to make silent failures visible, the team accumulates debt and builds a system that becomes increasingly difficult to manage.

While automation addresses the recurring operational overhead of manual retraining, it cannot prevent architectural decay. Deployed systems accumulate hidden complexity through systemic failure modes that emerge from statistical coupling rather than deterministic logic. Figure 14.3 maps six debt patterns across data, model, and infrastructure concerns, establishing the failure modes that operational architectures must isolate.

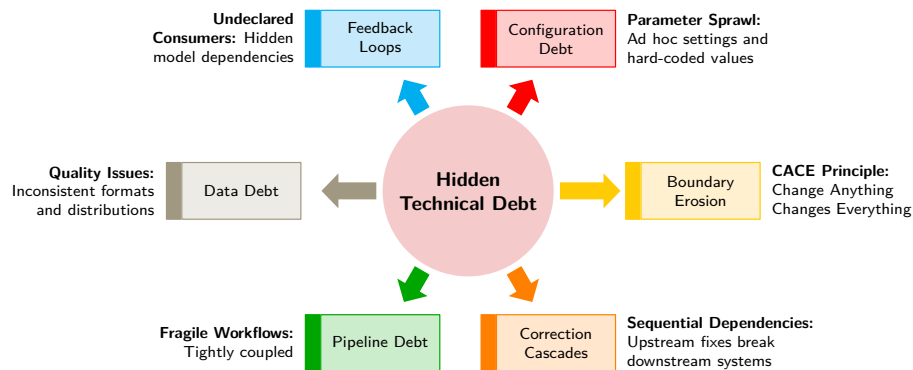


Figure 14.3: ML Technical Debt Taxonomy: Six debt patterns radiate from hidden technical debt: configuration debt, feedback loops, data debt, pipeline debt, correction cascades, and boundary erosion. The surrounding labels identify their associated failure patterns.

14.3.1 Boundary erosion

The dissolution of system boundaries represents the primary failure mode of statistical software. In traditional software, modular encapsulation provides clear isolation: components interact across typed APIs, allowing internal implementations to change while system behavior remains deterministic. Machine learning systems blur these boundaries because component behavior depends on the empirical distribution of incoming data rather than syntactic interface definitions. An upstream formatting modification or normalization change can satisfy every unit test while silently shifting feature distributions and degrading downstream model accuracy. This implicit coupling through data pipelines creates unmonitored dependencies among feature engineering, training pipelines, and serving endpoints.

This erosion produces **entanglement**, where dependencies between features and components become so intertwined that local modifications demand global coordination. The resulting failure mode reflects the CACE principle: *Change Anything Changes Everything*. When systems lack strict distributional boundaries, adjusting a single feature encoding or data filtering threshold alters the learned weights across the entire model. For example, changing the binning strategy of a continuous feature shifts the feature’s variance and covariance structure, causing a previously calibrated model to underperform and forcing downstream recalibrations that ripple across dependent services.

Defending against boundary erosion requires architectural modularity enforced at runtime. Components require explicit data contracts that govern statistical invariants, including allowed feature ranges, null tolerances, and drift thresholds. Enforcing separation between data ingestion, feature generation, and inference logic creates verification boundaries where data distributions can be validated before reaching model weights. Because boundary erosion remains silent during initial prototyping, establishing explicit schema contracts and invariant checks provides the only reliable defense against creeping statistical coupling.

14.3.2 Correction cascades

When boundary erosion goes unaddressed, engineering interventions frequently trigger a **correction cascade**: fixing a specific model defect with an auxiliary patch introduces secondary errors elsewhere, requiring further compensatory fixes that destabilize the entire pipeline. In ML systems, these cascades are severe because changes propagate through statistical dependencies rather than deterministic code branches. Retraining a model to correct a specific false-positive cluster may degrade accuracy across previously stable cohorts. Adding an auxiliary model to override edge-case predictions introduces uncalibrated residuals that distort downstream consumers. Each patch triggers subsequent corrections, consuming engineering effort that rapidly outstrips the cost of retraining from clean data.

Figure 14.4 makes the cascade structure visible as a chain of dependent models. Each model is trained to correct the errors of the one before it: model B compensates for model A’s residual mistakes, model C compensates for model B’s, and so on down the chain. The arrangement holds until an

upstream model changes. When model A is retrained to fix one failure mode, every downstream model that was tuned to its previous behavior is invalidated at once, and each must be re-corrected. The red arcs trace this propagation, showing how a repair that looked local reaches across the entire chain. A change near the top forces the most rework, while one near the bottom is nearly free. For an operations team, one change request can trigger a coordinated retraining of the chain.

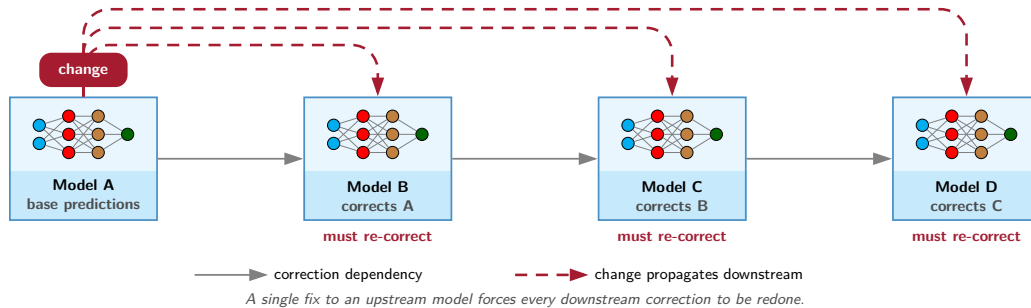


Figure 14.4: Correction Cascades: Each model is trained to correct the errors of the model before it, forming a chain of dependencies. Changing an upstream model invalidates every downstream correction at once, turning one local fix into cascading rework across the chain, with each downstream fix demanding its own retraining cycle.

This propagation turns localized model repairs into lifecycle-wide engineering dependencies. Sequential model development is a frequent source: fine-tuning or reusing an established model accelerates initial deployment, but it locks the downstream system into the source model’s latent assumptions and representation biases. Over time, those inherited properties become rigid architectural constraints that complicate future updates.

For example, fine-tuning an existing customer churn model for a new product line inherits the base model’s feature encodings and calibration thresholds. If the new product exhibits different user engagement patterns, patching prediction errors with auxiliary post-processing rules creates a fragile multi-stage pipeline. When accuracy degrades, engineering teams discover that the root failure lies several layers upstream in the original labeling conventions.

Mitigating correction cascades requires balancing component reuse against redesign. While fine-tuning reduces initial compute and data collection costs, training from scratch provides complete control over model assumptions and eliminates upstream error dependencies. The architectural decision depends on task similarity, covariate alignment, and the ongoing maintenance cost of chained models. Where dependent models are unavoidable, systems must decouple them through version-pinned model artifacts and frozen intermediate representations.

14.3.3 Interface and dependency challenges

Boundary erosion and correction cascades both stem from interface dependencies that bypass formal software contracts. Traditional software dependencies are declared through import statements, package managers, and RPC schemas, allowing static analysis tools to map dependency graphs. In machine learning systems, dependencies hide inside data streams. When model A’s output is logged to a data warehouse and later ingested as a training feature for model B, the coupling exists solely within the data pipeline, invisible to static code analysis.

Hidden coupling frequently manifests as **undeclared consumers**, where downstream services consume model predictions without formal registration or service-level agreements. When the upstream model is retrained or its output distribution shifts, these untracked consumers fail silently. For example, a credit risk score consumed by an automated marketing system can alter applicant pools, feeding shifted distributions back into future training runs and inducing runaway selection bias. This vulnerability is compounded by **data dependency debt**, which accumulates when ML pipelines ingest unversioned tables, unstable upstream features, or signals with volatile distributions.

Eliminating interface debt requires infrastructure-enforced boundaries: strict authentication and registration for model prediction endpoints, machine-readable schema contracts for feature stores, and automated data lineage tracking that registers consumers before granting read access.

14.3.4 System evolution challenges

Even well-designed ML systems with documented interfaces encounter evolution challenges absent from deterministic software.

The most insidious of these dynamics is the feedback loop, where a deployed model influences the data distribution that trains its successors. Recommendation systems illustrate this pattern directly: suggested content dictates user engagement, clicks generate subsequent training sets, and the model reinforces its own historical selections while starving unselected content of exposure. In production, this failure mode often surfaces as a widening performance disparity across demographic cohorts: underrepresented groups receive suboptimal predictions, generating skewed engagement logs that further degrade cohort representation in subsequent retraining runs. Detecting feedback loops requires cohort-stratified monitoring that tracks error distributions across user segments rather than relying on aggregate validation metrics.

Over repeated deployment cycles, workflows accumulate **pipeline and configuration debt**, evolving into jungles of ad hoc glue scripts and divergent hyperparameter files. When pipelines lack modular interfaces, teams copy existing scripts rather than refactor brittle data loaders, producing redundant pipelines that process the same raw inputs with subtle inconsistencies. Compounding this, rapid prototyping frequently embeds business logic directly into feature preparation scripts. Controlling pipeline evolution requires workflow orchestration engines with immutable directed acyclic graph (DAG) definitions, strict parameter validation, and configuration versioning managed under the same source control as model code.

14.3.5 Code and architecture debt

Beyond statistical coupling and data flows, ML systems accumulate code-level technical debt that paralyzes infrastructure refactoring (Sculley et al. 2015).

At the implementation tier, **glue code** frequently dominates production repositories. Massive integration wrappers are written to shuttle tensors and records between general-purpose ML frameworks, specialized feature pipelines, and low-latency serving engines. In production environments, this boilerplate can consume up to 95 percent of the codebase while core ML logic accounts for only 5 percent. The resulting tight coupling to framework-specific APIs creates severe maintenance liabilities: minor upstream framework updates force extensive rewrites of the surrounding integration scaffolding. Isolating third-party libraries behind stable internal interfaces prevents external API churn from propagating throughout the serving stack.

Rapid experimentation further burdens codebases through **dead experimental codepaths**. Research iterations leave behind legacy branches, alternative feature transforms, and abandoned scoring logic. Unlike traditional dead code that static analyzers flag and compilers prune, experimental ML codepaths often remain executable at runtime because they are guarded by dynamic configuration flags. These dormant branches expand the test matrix, complicate regression verification, and obscure the precise execution graph operating in production. Enforcing deprecation schedules and pruning retired feature flags prevents experimental residue from destabilizing the serving runtime.

Underlying both patterns is **abstraction debt**: traditional software engineering relies on mature abstractions such as objects, interfaces, and modules, whereas ML engineering lacks universal abstractions for concepts such as a feature, a training dataset slice, or a model lifecycle state. Without standardized primitives, teams frequently reinvent bespoke abstractions or abandon encapsulation altogether. Infrastructure components such as feature stores, model registries, and standardized prediction services mitigate this debt by providing uniform abstractions for feature transformation, artifact lineage, and inference routing.

Beyond these structural patterns, Sculley et al. (2015) identify system smells that signal mounting debt: the *Plain-Old-Data Type Smell* (passing unvalidated raw floats and string arrays rather than domain types with invariant checks), the *Multiple-Language Smell* (stitching Python, SQL, C++, and shell scripts across the training-serving boundary), and the *Prototype Smell* (promoting throwaway research notebooks directly into production serving without re-engineering). Treating debt paydown as a routine operational discipline rather than an emergency intervention prevents these code smells from degrading cluster efficiency.

14.3.6 Technical debt in practice

These debt patterns are not theoretical edge cases; they represent the dominant failure modes of production ML deployments at scale. When statistical dependencies and uncontracted data flows interact with real-world operating environments, unmanaged technical debt translates directly into systemic outages and commercial failure.

14.3.6.1 Production debt patterns

The first pair exposes coupling through model behavior. YouTube’s recommendation system illustrates how large recommenders learn from noisy user-behavior signals, making ranking objectives, equal per-user weighting, and live A/B evaluation part of the system design rather than offline evaluation details (Covington et al. 2016). Zillow’s home valuation and purchasing workflow exposed the correction-cascade version during its iBuying venture.⁶ Valuation and inventory assumptions propagated into purchasing decisions; later corrections then destabilized inventory and pricing decisions, forcing revalidation and eventually a full rollback when the company shut down the iBuying arm in 2021.

The second pair exposes coupling through ownership and configuration. Safety-critical driving automation illustrates the undeclared-consumer risk from a different direction: when automated-control outputs, driver expectations, and subsystem responsibilities are not specified clearly enough, operational failures can cross component boundaries rather than staying local (National Transportation Safety Board 2017). Facebook’s News Feed iterations show the configuration version of the same governance problem. Rapid experimentation and ranking changes require traceable settings and explicit objectives; otherwise behavioral changes become hard to audit after deployment (Engineering 2016; Mosseri 2018).

These incidents are predictable consequences of deploying probabilistic decision systems without infrastructure that makes coupling visible. YouTube, Zillow, safety-critical driving automation, and Facebook each expose a different debt pattern: feedback loops, correction cascades, undeclared consumers, and configuration sprawl.

These controls are not isolated point solutions but structural boundaries. Mitigating technical debt requires end-to-end MLOps infrastructure: feature registries that enforce runtime schemas, metadata stores that track model-data lineage, automated CI/CD pipelines that validate statistical properties, and observability platforms that detect distribution shift before silent errors propagate downstream.

14.4 Development Infrastructure

Development infrastructure turns the debt patterns diagnosed earlier into enforcement points. A feature schema that drifts upstream cannot be repaired by a dashboard alone; it needs a shared contract, a versioned artifact, and a deployment path that rejects incompatible changes before they reach production. Table 14.6 maps each component directly to a foundational principle (section 14.2.1) and a specific failure mode.

Table 14.6: MLOps Infrastructure as Debt Remediation: Each infrastructure component responds to a class of technical debt observed in production ML systems. Feature stores can reduce training-serving skew by reusing feature definitions and values; versioning systems support reproducibility; CI/CD pipelines automate rollout controls; monitoring systems can surface silent degradation before users do.

Infrastructure Component	Principle Implemented	Debt Pattern Addressed
Feature stores	Consistency Imperative	Data dependency debt, training-serving skew
Versioning systems	Reproducibility Through Versioning	Configuration debt, correction cascades
CI/CD pipelines	Cost-Aware Automation	Pipeline debt, boundary erosion
Monitoring systems	Observable Degradation	Feedback loops, silent failures

These operational components span the boundary between high-level algorithmic abstractions and low-level machine execution. As shown in figure 14.5, the system stack stratifies responsibilities across five tiers: ML models, frameworks, orchestration, infrastructure, and hardware. While models and frameworks express tensor computations mathematically, they remain unaware of cluster state,

⁶ | **Zillow iBuying failure:** Zillow reported a plan to wind down Zillow Offers in November 2021, including a Q3 inventory write-down and workforce reductions (Zillow Group 2021). The failure illustrates correction cascade debt at scale: pricing errors, purchasing decisions, and inventory feedback can reinforce one another, creating a loop that no single retraining cycle can break.

hardware faults, or input distribution drift. Model orchestration and infrastructure bridge this gap: orchestration manages data lineage, feature computation, and model serving, while infrastructure schedules accelerator resources, manages storage tiers, and collects runtime telemetry to enforce the debt-remediation contracts identified in table 14.6.

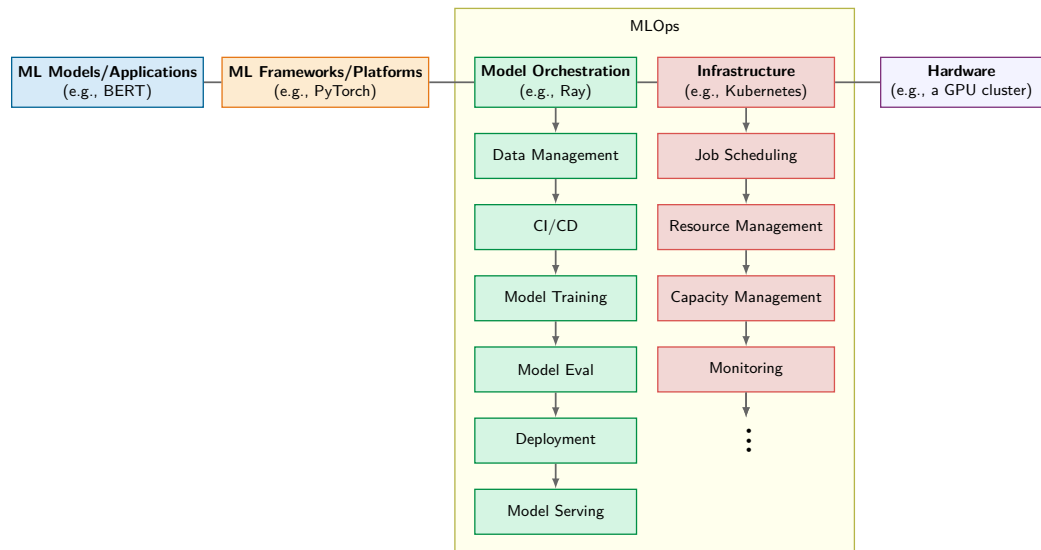


Figure 14.5: MLOps Stack Layers: Five columns organize the ML system stack: ML Models/Applications, ML Frameworks/Platforms, Model Orchestration, Infrastructure, and Hardware. MLOps spans orchestration tasks from data management through model serving and infrastructure tasks from job scheduling through monitoring, supporting automation, reproducibility, and scalable deployment.

14.4.1 Data infrastructure and preparation

In an operational machine learning system, data is not a static compilation input; it is an active physical dependency that flows continuously from external ingestion to accelerator execution. From initial ingestion to online inference, data infrastructure must guarantee quality, mathematical consistency, and end-to-end traceability across both offline training and real-time serving. Meeting these operational requirements demands systems that formalize feature transformation and artifact versioning across the entire machine learning lifecycle.

14.4.1.1 Data management

Machine learning technical debt stems largely from unmanaged data boundaries: unversioned datasets erode system boundaries, inconsistent feature computation triggers correction cascades, and undocumented data dependencies create hidden downstream consumers. Building on the data engineering foundations in chapter 4, collection, preprocessing, and feature transformation become formalized operational contracts. Where data engineering optimizes single-pipeline correctness and ingestion throughput, MLOps data management governs cross-pipeline consistency, ensuring that offline training and online serving compute mathematically identical features. Data management extends beyond initial preparation to govern the continuous lifecycle of data artifacts.

Three principles organize the infrastructure that addresses these root causes: consistency, freshness, and quality. Each principle motivates specific tooling rather than the reverse.

The first requirement is data consistency: every artifact influencing model behavior, from raw datasets to engineered features, must be versioned and reproducible. Without versioning, teams cannot trace which data produced which model, making debugging and rollback impossible. The implementation usually combines code versioning, dataset versioning, and durable object storage. DVC (Data Version Control) (Iterative 2024), Git (Torvalds and Hamano 2024), Amazon S3 (Amazon Web Services 2024b), and Google Cloud Storage (Google Cloud 2024b) are examples of that pattern, but the invariant is the important part: raw and processed artifacts must remain addressable

by version. Section 14.4.1.3 examines implementation details including Git integration, metadata tracking, and lineage preservation. At the feature level, a feature store can reduce skew by centralizing feature definitions and serving paths across training and serving pipelines, but parity still requires validation. Uber’s Michelangelo platform popularized this pattern inside a large production ML platform, and Feast later made the pattern available as open-source feature-store infrastructure (Hermann and Del Balso 2017; Gojek and Google 2019). Section 14.4.1.2 details implementation patterns for training-serving consistency.

Consistency alone is insufficient if the underlying data is stale. Data freshness ensures that models train and serve on current data rather than outdated snapshots. Automated data pipelines maintain freshness by continuously transforming raw data into analysis-ready formats through structured stages: ingestion, schema validation, deduplication, transformation, and loading. Workflow orchestrators such as Apache Airflow (Apache Software Foundation 2024) and Prefect (Prefect Technologies, Inc. 2024), together with the transformation framework dbt (dbt Labs 2024), make those stages explicit, schedulable, and reviewable as code. Once the pipeline is managed this way, data flows can evolve with model requirements without losing versioning, modularity, or CI/CD integration.

The third principle, data quality, governs whether the inputs reaching the model remain accurate, complete, and consistently labeled. In supervised learning systems, labeling quality sets an upper bound on model performance regardless of model capacity. Annotation platforms such as Label Studio (HumanSignal 2024) support team-based annotation with integrated audit trails and version histories, providing the verification needed when labeling ontologies evolve or require correction across multiple training cycles.

In an industrial predictive maintenance system, these three principles operate in concert. A continuous telemetry stream is ingested and joined with historical maintenance logs through a scheduled pipeline managed in Airflow (freshness). The resulting features—rolling window averages and vibration frequency aggregates—are stored in a feature store for both offline retraining and low-latency online inference (consistency). Schema validation, sensor-range checks, missingness tests, and label audits reject corrupted telemetry records before they enter the training set (quality), while dataset versioning and model-registry integration maintain traceability from raw sensor logs to deployed model predictions. Structured around these three principles, data management establishes the operational foundation for model reproducibility and production reliability.

14.4.1.2 Feature stores

The data dependency debt and training-serving skew analyzed in section 14.3 share a common root cause: inconsistent feature computation across pipeline stages. Without a unified feature layer, feature definitions are typically implemented twice: a data scientist computes `user_session_length` in Python using batch aggregations over historical logs, while a systems engineer reimplements the calculation in Java or C++ to satisfy real-time serving latencies. Subtle differences emerge: one uses wall-clock time between raw events, the other tracks server processing time and filters idle timeouts. The model trains on one definition but serves with another, causing silent accuracy degradation in production. Feature stores⁷ address this divergence by providing an abstraction layer between data engineering and machine learning, implementing the consistency imperative through a single source of truth for feature transformations. In pipelines lacking this abstraction, duplicated feature logic across environments introduces training-serving skew, temporal data leakage, and distribution drift.

Feature stores resolve a fundamental systems tension between batch throughput and online latency by maintaining two synchronized storage tiers behind a unified API. Offline training requires high-throughput sequential reads across terabytes or petabytes of historical logs, which is optimized for columnar storage formats (such as Parquet) hosted on distributed object storage or data warehouses. In contrast, online inference requires sub-millisecond point lookups keyed by entity identifier (such as `user_id`), bounded by strict tail-latency service-level objectives (SLOs). An online store satisfies this access pattern using in-memory or distributed key-value stores (such as Redis or Cassandra). While the underlying physical engines differ to match these access patterns, both read from the same versioned transformation logic.

Historical feature retrieval must strictly enforce point-in-time correctness (preventing temporal data leakage or time travel). When generating training examples for an event that occurred at timestamp t_{event} , the feature store must reconstruct feature values exactly as they existed at t_{event} .

⁷ | **Feature store:** Uber’s Michelangelo platform described a centralized feature store for sharing and serving features across production models (Hermann and Del Balso 2017). At that scale, the consistency guarantee must hold under an online latency budget: what distinguishes a feature store from a shared library of feature code is that the shared feature path also has to serve fresh features fast enough for real-time inference.

excluding any updates that arrived later. If a training query performs a naive join against the latest feature snapshot, it incorporates future state into historical features. The model learns spurious correlations that are physically unavailable during real-time online inference, causing production accuracy to collapse. By automating point-in-time (as-of) joins across timestamped event logs, the feature store enforces temporal causality in training sets while keeping feature definitions unified across offline and online environments.

Beyond consistency, feature stores support versioning, metadata management, and feature reuse across models. A fraud detection model and a credit scoring model frequently depend on overlapping transaction features that can be centrally maintained, validated, and shared. Integration with data pipelines and model registries preserves lineage: when an upstream feature definition is updated or deprecated, downstream models can be automatically identified and scheduled for retraining.

Training-serving skew: Diagnosis and prevention Training-serving skew (defined formally in section 13.6.1) manifests operationally through feature store inconsistencies and pipeline divergence. Table 14.7 summarizes common causes and their detection methods:

Table 14.7: Training-Serving Skew Categories: Each category requires different detection and prevention strategies. Schema and preprocessing skew emerge from code divergence and require parity validation; shared feature logic, including a feature store where appropriate, can reduce them, while data distribution skew requires statistical monitoring against training baselines. Timing skew demands careful analysis of feature freshness between training and serving contexts.

Skew Type	Example	Detection Method
Feature preprocessing	Normalization uses different statistics	Statistical comparison of feature distributions
Missing data handling	Training fills NaN with mean; serving uses 0	Schema validation with explicit null handling
Time-dependent features	Features computed with different time cutoffs	Timestamp validation in feature pipelines
Library version drift	NumPy or Pandas version differences	Environment hash comparison

Training-serving skew case study A production recommendation system illustrates how feature divergence manifests in deployment. One month after rollout, the model exhibits an 8 percent accuracy degradation despite zero alterations to model code or weights. Comparing feature distributions reveals that `user_session_length` has an empirical mean of 45 minutes in the training set but only 12 minutes in online inference. The failure traces to feature-definition skew: the offline training pipeline calculated wall-clock duration from the initial event to the final event in a session, whereas the online serving path counted only foreground-active time after filtering idle periods. The model learned decision boundaries against a feature distribution that the serving infrastructure never delivers.

Feature stores resolve this discrepancy by centralizing feature logic into a single versioned definition, building upon the data pipelines in chapter 4. Listing 14.1 demonstrates the architecture: training materializes historical features with point-in-time correctness, serving performs real-time point lookups, and both pipelines bind to the identical versioned definition hash rather than independent code paths.

Listing 14.1: Feature Store Consistency: Unified retrieval reduces training-serving skew by sharing versioned feature definitions across both pipelines.

```
feature_definitions = registry.load(version="2026-06-01")

training_features = feature_definitions.materialize_historical(
    entities=training_entities,
    at_event_time=True,
    names=["user.session_length", "user.purchase_history"],
)

serving_features = feature_definitions.lookup_online(
    entities=[{"user_id": 12345}],
    names=["user.session_length", "user.purchase_history"],
)

assert training_features.schema == serving_features.schema
assert (
    training_features.definition_hash
    == serving_features.definition_hash
)
```

By centralizing the definition of `session_length`, both offline and online pipelines execute identical transformation logic, though automated integration tests must still verify value parity. Centralized registries also record metadata and lineage, enabling rapid diagnosis when feature definitions evolve (Hermann and Del Balso 2017; Gojek and Google 2019).

As quantified by the consistency imperative in section 14.2.1.3, skew-induced errors at production scale can produce hundreds of thousands of dollars in annual losses. Centralized feature stores amortize the engineering cost of this infrastructure across multiple models and services, translating architectural consistency into measurable operational savings.



Example 14.1: Uber Michelangelo feature store

Scenario: Uber’s Michelangelo platform supported production models such as estimated time of arrival and used shared feature infrastructure across offline training and online prediction (Hermann and Del Balso 2017). Separate implementations of the same feature logic would create training-serving skew.

Diagnosis: A shared feature contract reduces the chance that batch and online pipelines interpret a feature differently, while separate offline and online stores still require point-in-time and parity validation.

Systems lesson: Feature stores reduce one major source of training-serving skew by centralizing versioned feature definitions and access paths. They do not eliminate skew: point-in-time correctness, freshness, and online-offline parity still require explicit tests.

Skew detection in CI/CD Continuous integration pipelines must validate feature consistency before deployment. Listing 14.2 defines a gate that compares offline training distributions against online serving samples using the two-sample Kolmogorov-Smirnov test, blocking deployment if any feature diverges beyond a predefined threshold. The two-sample Kolmogorov-Smirnov test measures the maximum vertical distance between the empirical cumulative distribution functions (eCDFs) of the training and serving samples: $D = \sup_x |F_{\text{train}}(x) - F_{\text{serve}}(x)|$. Setting the threshold to 0.1 asserts that cumulative probabilities do not diverge by more than 10 percentage points at any value across the feature’s support, providing a parameter-free divergence metric for continuous features without requiring empirical binning. This validation gate converts distribution divergence into an automated test failure prior to production rollout. However, the threshold must be calibrated against sample size and feature sensitivity; passing the distribution check validates statistical alignment but cannot verify semantic parity or point-in-time correctness.

Listing 14.2: Feature Skew Validation: This function compares training and serving feature distributions using the Kolmogorov-Smirnov test, rejecting deployment when any feature diverges beyond a configurable threshold.

```
def validate_no_skew(
    training_features, serving_features, threshold=0.1
):
    """Reject deployment if feature distributions diverge."""
    for feature in training_features.columns:
        ks_stat = ks_2samp(
            training_features[feature], serving_features[feature]
        )
        if ks_stat.statistic > threshold:
            raise SkewDetectedError(
                f"{feature}: KS={ks_stat.statistic:.3f}"
            )
```

14.4.1.3 Versioning and lineage

Lineage tracking and versioning implement the reproducibility principle formalized in section 14.2.1. In traditional software engineering, a binary executable is deterministically compiled from source code and build configurations. In machine learning systems, model behavior is the joint product of code, hyperparameter configuration, and the training dataset snapshot. Changing any single dependency produces an entirely distinct model artifact. MLOps infrastructure manages this multidimensional dependency space by enforcing cryptographic versioning across every pipeline component.

Data versioning addresses dataset mutability by recording content-addressable hashes for raw data snapshots and intermediate feature tables, ensuring that any historical training run can be reconstructed precisely. Model versioning registers trained weights as immutable artifacts alongside training hyperparameters, evaluation metrics, and environment dependency manifests. Model registries⁸ manage lifecycle states (such as staging, production, or retired) for these artifacts and expose lineage graphs tracing the complete dependency chain from raw ingested records to deployed prediction endpoints (MLflow Project 2026; Google Cloud 2024d).

These mechanisms construct the lineage graph of the machine learning system. This graph establishes the audit trail required to diagnose production failures: determining whether an incoming request distribution diverged from training baselines, whether upstream feature calculations changed, or whether the serving environment loaded an unvalidated weight checkpoint. By formalizing provenance as an explicit architectural contract, systems engineering transforms incident triage from manual heuristics into deterministic root-cause analysis.

14.4.2 Continuous pipelines and automation

Feature stores and versioning systems address data consistency statically: they ensure that features are computed correctly at a point in time. Automation enables these systems to evolve continuously, synchronizing data preprocessing, training, evaluation, and release into integrated workflows that respond to new data, shifting objectives, and operational constraints (Orr et al. 2021; Google Cloud 2026b).

14.4.2.1 CI/CD pipelines

Feature stores and versioning systems address the *data* side of consistency; CI/CD pipelines address the *process* side, ensuring that changes flow through validated stages rather than ad hoc deployments. ML CI/CD pipelines must handle complexity absent from traditional software: data dependencies, model training workflows, and artifact versioning that couple code changes to statistical behavior changes.

A typical ML CI/CD pipeline consists of coordinated stages: checking out updated code, preprocessing input data, training a candidate model, validating performance, packaging the model, and deploying to a serving environment. In some cases, pipelines also include triggers for automatic retraining based on data drift or performance degradation. By codifying these steps, CI/CD pipelines⁹ reduce manual intervention, enforce quality checks, and support continuous improvement of deployed systems.

⁸ | **Model registry:** Prevents “registry bypass,” the failure mode where the undocumented production model diverges from the trained artifact through different preprocessing, stale serialization formats, or manual hotfixes applied directly to the serving endpoint. Without a registry enforcing versioned, immutable artifacts with queryable metadata and state, rollbacks require locating the correct weights from an ad-hoc artifact store under incident pressure.

⁹ | **Idempotency:** This property ensures that retrying a pipeline stage does not create duplicate externally visible effects, conflicting registrations, or repeated deployment actions. Producing identical models from identical inputs is a separate property—determinism and reproducibility—and the training stage may violate it due to randomness such as weight initialization. Production systems therefore combine stable run identifiers and atomic publication for idempotency with fixed random seeds, deterministic kernels, fixed library versions, and controlled data ordering for reproducibility.

ML-focused CI/CD layers two tiers of tooling for one reason. A general-purpose CI/CD orchestrator (Jenkins, CircleCI (2024), or GitHub Actions (GitHub, Inc. 2024b)) manages version-control events and execution logic, but the ML layer must additionally version data, gate on model metrics, and trigger retraining. Teams therefore add an ML platform or workflow orchestrator (Kubeflow (Authors 2024), Metaflow (Netflix 2024), or Prefect (Prefect Technologies, Inc. 2024)) that supplies higher-level abstractions for those tasks.

Without this automation, model deployment degrades into a manual, error-prone process: an engineer retrains locally, copies artifacts to a staging server, and promotes to production with no guarantee that the data, code, or hyperparameters match what was validated. The cost of such ad hoc workflows compounds with team size and deployment frequency, producing configuration drift and silent regressions that surface only after the model has served incorrect predictions.

Figure 14.6 shows how a representative continuous-training pipeline moves from dataset ingestion and validation through transformation, training/tuning, evaluation, model validation, and model registration. A retraining trigger initiates the process, while dataset, model, metadata, and artifact repositories preserve the inputs and outputs needed for lineage.

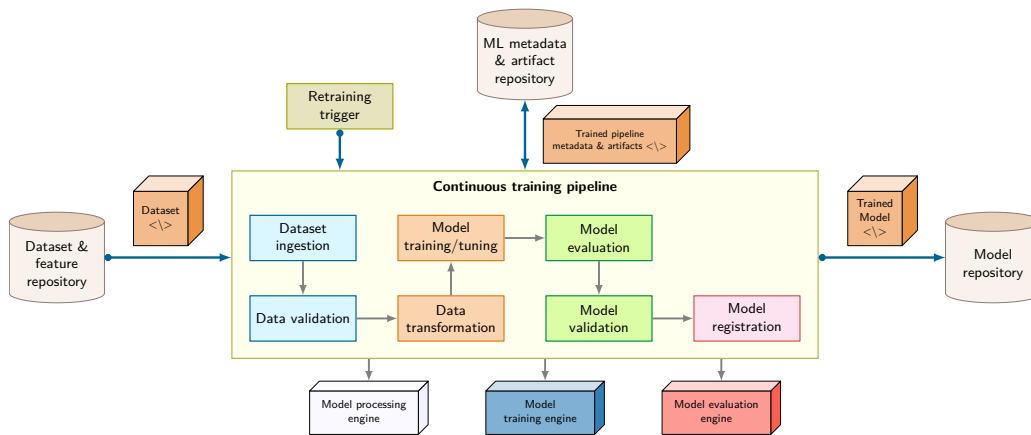


Figure 14.6: ML CI/CD Pipeline: Read the enclosed stages in execution order; the retraining trigger starts a new run, while connected repositories preserve the dataset, model, metadata, and artifacts needed to reproduce and govern it. Adapted from Google Cloud’s MLOps continuous delivery and automation pipeline guidance (Google Cloud 2026b).

Consider an image classification model under active development. When a developer commits code changes to a GitHub (GitHub, Inc. 2024a) repository, a Jenkins pipeline triggers execution: it pulls the specified dataset version, performs feature preprocessing, and initiates model training. Experiments are tracked using MLflow (Databricks 2024), which logs loss curves and registers model artifacts. After passing automated offline validation, the model is containerized and deployed to a staging cluster managed by Kubernetes (Cloud Native Computing Foundation 2024a). If the candidate satisfies staging validation, the pipeline executes a canary rollout (detailed in section 14.4.2.3), gradually shifting production query traffic to the new model while monitoring inference latency and error rates. If a performance regression emerges, traffic automatically rolls back to the previous model checkpoint.

CI/CD becomes foundational when systems release models repeatedly or carry meaningful production risk, turning ad hoc experimentation into structured, repeatable deployment. Scaling beyond one pipeline requires reusable components and explicit artifact contracts.

Example 14.2: Google TFX production ML pipelines

Scenario: Google’s production experience motivated TensorFlow Extended (TFX), a set of reusable components for production ML pipelines (Baylor et al. 2017). As pipelines multiply, ad hoc handoffs make validation, lineage, and reproducibility difficult.

Diagnosis: Each stage needs explicit validation and metadata so a deployed model can be traced to the data, transformations, code, and evaluation that produced it.

Systems lesson: Directed acyclic graph orchestration alone does not establish reproducibility. Standardized components and ML Metadata (MLMD) preserve lineage through recorded inputs and transformations.

14.4.2.2 Training pipelines

CI/CD pipelines orchestrate the overall workflow, but training itself requires specialized compute infrastructure. Model training builds on the distributed execution primitives covered in chapter 8. Within MLOps, training activities become components of a reproducible pipeline supporting continual experimentation and production deployment.

Frameworks such as TensorFlow (Abadi et al. 2016), PyTorch (Paszke et al. 2019), and Keras (Chollet et al. 2024) supply modular components for model architectures and optimization routines, carrying the framework-selection criteria from chapter 7 into production. Operational discipline determines which exploratory training routines graduate into versioned, tested retraining jobs—and under what execution triggers.

Beyond scalability, operational integrity requires strict reproducibility. Training scripts and environment configurations are version-controlled using tools like Git (Torvalds and Hamano 2024) and hosted on platforms such as GitHub (GitHub, Inc. 2024a). While interactive environments like Jupyter (Project Jupyter 2024) notebooks accelerate initial prototyping by coupling code, inline visualizations, and execution state, running them unaltered in production pipelines introduces severe operational liabilities.

Notebooks in production Automated CI/CD pipelines assume deterministic and reproducible code execution, but Jupyter notebooks violate this assumption through persistent kernel state. In an interactive session, cells can be executed out of linear order, creating hidden memory dependencies that cannot be reproduced from a clean interpreter restart. A common production failure occurs when development relies on cells executed in sequence 1-3-2; a scheduled batch runner executing top-to-bottom (1-2-3) fails due to undefined variables or altered intermediate tensors.

Testing difficulties compound this state vulnerability. Standard unit-testing harnesses and mocking frameworks do not interface cleanly with interactive notebook documents. Cell-level assertions are rarely parameterized or wired into test runners, leaving notebooks significantly less tested than modular Python code.

Operationalizing notebooks requires specific tooling interventions. Papermill enables parameterization and headless execution of notebooks, injecting runtime configurations and executing cells top-to-bottom as discrete pipeline stages (Papermill Project 2026). Similarly, nbconvert converts validated notebooks into static executable Python scripts for production environments (Project Jupyter 2026), stripping exploratory scratch cells and exposing code to linters and CI test runners.



Napkin Math 14.2: The cost of silent failures

Problem: Is building an automated drift detection system worth the engineering effort?

Scenario: Consider a product recommendation engine generating \$50M in annual revenue.

Failure mode: A deployment bug causes training-serving skew, dropping recommendation quality by 5 percent. This degrades conversion rate proportionally.

Cost analysis:

1. Manual ops (monthly review):

- Detection Time: ~4 weeks (28 days).
- Revenue Loss: $\$50\text{M} \times 0.05 \times 28 \text{ days} / 365 \text{ days} \approx \$191,780.8$.

2. Automated MLOps (daily checks):

- Detection Time: 1 day.
- Revenue Loss: $\$50\text{M} \times 0.05 \times 1 \text{ day} / 365 \text{ days} \approx \$6,849.3$.

Systems insight: A single silent failure costs \$184,931.5 more without MLOps. Under this scenario’s 4 incidents/year, reducing the time-to-detection (TTD) avoids nearly \$739,726 in annual loss.

The cost calculation in 14.2 quantifies the engineering trade-off. While extracting code from notebooks into tested modules introduces refactoring overhead, it prevents hidden state assumptions from causing training-serving skew in production. When silent failures cost hundreds of thousands of dollars per incident, disciplined extraction of preprocessing and training code directly protects business revenue.

Once training logic is modularized, automated pipelines can execute systematic search strategies across the model design space. Workflows incorporate distributed hyperparameter optimization (Ranjit et al. 2019; L. Li et al. 2017), neural architecture search (Elsken et al. 2019), and feature selection (scikit-learn developers 2024a), coordinating multiple parallel training runs across accelerator pools.

Scaling these automated workloads requires substantial compute capacity, as reflected in enterprise cloud infrastructure demand (Gartner 2024). This connects to the workflow orchestration patterns explored in chapter 3 for coordinating multi-stage jobs across distributed clusters. Cloud platforms provision high-performance accelerator instances on demand, including GPU and Tensor Processing Unit (TPU) nodes.¹⁰ Teams either construct custom orchestration graphs or leverage managed platforms such as Vertex AI Fine Tuning (Google Cloud 2024c) to adapt foundation models to domain tasks. Furthermore, physical accelerator allocation and regional interconnect bandwidth constrain where large-scale training pipelines can execute (Lehdonvirta et al. 2025).

Once training pipelines are automated and reproducible, an engineering trade-off emerges: execution frequency. Because dispatching jobs to multi-accelerator clusters incurs substantial compute costs, pipeline triggers cannot run continuously without economic justification.

Retraining decision framework Automated training pipelines introduce a critical operational decision: execution cadence. Deciding when to retrain a model requires balancing accuracy degradation against compute expenditure. Three common strategies exist, each presenting distinct systems trade-offs. Table 14.8 provides illustrative schedules across domains, from daily retraining for rapidly shifting ad click prediction to quarterly updates for stable medical imaging applications:

Table 14.8: Illustrative Retraining Schedules by Domain: These represent starting points; actual cadences depend on observed drift rates and business impact, and organizations typically calibrate them through operational experience.

Domain	Illustrative Schedule	Rationale
Ad click prediction	Daily	User interests shift rapidly
Fraud detection	Weekly	Attack patterns evolve continuously
Demand forecasting	Monthly	Seasonal patterns change slowly
Medical imaging	Quarterly	Disease presentations are stable

Those schedules are starting points, not invariant rules. In a *scheduled retraining* policy, pipeline execution runs on a fixed calendar cadence—daily, weekly, or monthly—regardless of incoming performance telemetry. This strategy is simple to automate and guarantees that freshly accumulated training data is integrated periodically. However, it burns accelerator hours during quiescent periods and reacts sluggishly when distribution shifts strike between scheduled intervals.

A *triggered retraining* policy initiates pipeline execution when monitoring systems detect statistical drift or performance degradation exceeding predefined thresholds. This approach conserves compute by training only when necessary, but it relies heavily on low-latency label streams or accurate proxy metrics to prevent delayed responses or false-alarm retraining runs.

A *continuous retraining* policy updates model weights incrementally as labeled records arrive, using streaming micro-batches or online learning algorithms. While this strategy minimizes staleness lag, it dramatically amplifies validation risk: corrupted, unrepresentative, or adversarial data can quickly poison model parameters before automated anomaly detectors or engineering teams can intervene.

¹⁰ | **Cloud ML training economics:** GPT-3 training cost has been estimated in the millions of dollars when priced in V100 GPU-hours (Li 2020). Fine-tuning costs vary by model size, provider, dataset, and number of training steps. Spot instances and Spot VMs can reduce instance prices but introduce a trade-off: AWS Spot Instances and Google Cloud Spot VMs can be interrupted or pre-empted, requiring checkpoint-and-resume infrastructure for fault-tolerant training workloads (Amazon Web Services 2026; Google Cloud 2026c).

The operating choice among these strategies depends on four physical and operational constraints: retraining compute cost, validation pipeline latency, automated rollback capability, and label arrival delay. Re-optimizing a model on multi-accelerator nodes incurs tangible energy and cluster costs; triggered policies require statistically reliable proxy signals when true labels arrive with long delays; and every continuous deployment pipeline demands robust canary and rollback mechanisms. The retraining policy is an engineering optimization governed by operational trade-offs rather than arbitrary heuristics.

The economic and validation constraints become executable in listing 14.3: the decision function schedules a run only when estimated recovered value exceeds retraining, validation, and rollout risk and validation data are fresh.

Listing 14.3: Triggered Retraining Decision: The decision combines quality loss, feature drift, prediction shift, and retraining cost so automatic retraining fires only when the expected benefit exceeds the operational risk.

```
quality_loss = baseline_accuracy - current_accuracy
feature_drift = max(population_stability_index(features))
prediction_shift = distribution_distance(
    baseline_predictions, live_predictions
)

benefit = estimate_value_recovered(
    quality_loss=quality_loss,
    feature_drift=feature_drift,
    prediction_shift=prediction_shift,
)
risk = retraining_cost + validation_cost + rollout_risk

if benefit > risk and validation_data_is_fresh():
    schedule_retraining_run()
```

The gate separates detecting drift from authorizing retraining: drift contributes to estimated benefit, while fresh validation data and positive net value determine whether automation acts. For the complete illustrative scenario evaluated next, the parameters yield an optimal interval near one day.

Quantitative retraining economics The decision to retrain a model balances the cost of accuracy decay against retraining expense. When outcome data support a locally exponential fit, the fitted decay rate gives a measurable timescale.¹¹ Its **half-life**¹² describes when fitted accuracy falls to half its initial value, but it does not determine the economically optimal retraining interval. The derivation in equation 14.4 distinguishes the fitted half-life from the economically optimal retraining interval.

¹¹ | **System entropy:** The fitted decay rate γ can differ substantially across deployments and must be estimated from outcome data. A shorter decay timescale demands more responsive monitoring, but it does not by itself require continuous training; label delay, validation risk, retraining cost, and rollback capacity determine the operational cadence.

¹² | **Half-life** (from nuclear physics, where it measures the time for half of a radioactive sample to decay): In ML operations, the metaphor is a convenient approximation, not a universal law. For the exponential fit in equation 14.5, the half-life is $\ln(2)/\gamma$. Retraining cadence also depends on traffic, value, retraining cost, and risk; seasonal, abrupt, or adversarial change requires a richer drift process.



Napkin Math 14.3: The optimal retraining interval

Problem: How often should the team retrain the model to maximize profit?

Physics: Model accuracy $\text{Accuracy}(t)$ decays at rate γ due to data drift.

- Q : Daily Query Volume (Traffic).
- V : Financial value per query for a unit change in accuracy fraction. With this convention, $V = \$0.50$ means 1 percentage point of accuracy is worth \$0.005 per query.
- C : Fixed cost of a retraining run, including compute and operational overhead.

Formula: The approximation in equation 14.4 gives the optimal retraining interval (T^*) that minimizes the sum of staleness losses and training costs:

$$T^* \approx \sqrt{\frac{2 \cdot C}{Q \cdot V \cdot \text{Accuracy}_0 \cdot \gamma}} \quad (14.4)$$

Math: Consider a lighthouse fraud model ($\text{Accuracy}_0 = 0.95$):

- **Traffic** (Q): 1,000,000 transactions/day.
- **Utility** (V): \$0.50/query for a unit accuracy change.
- **Retraining Cost** (C): \$5,000.
- **Drift Rate** (γ): 2 percent per day.

$$T^* \approx \sqrt{\frac{2 \times 5,000}{1,000,000 \times 0.50 \times 0.95 \times 0.02}} \approx 1 \text{ Day}$$

Systems insight: If traffic is high and accuracy is valuable, the team cannot afford to wait. The pipeline must be automated. If T^* is less than the team's manual deployment time, the system is in a state of permanent technical debt.

Formalizing this optimization requires deriving the total cost function from first principles, decomposing system expenditure into staleness loss and retraining overhead.

The staleness cost function Model accuracy can degrade over time, creating a staleness cost. For economic planning, a team may fit the observable impact over a stable local regime as an exponential decay process. Here γ is a temporal decay rate under the assumption that measured degradation accumulates steadily; it is not determined by distribution divergence alone. The exponential model is a simplification that enables closed-form economic analysis. Let $\text{Accuracy}(t)$ represent accuracy at time t since last training, and Accuracy_0 represent initial accuracy. Equation 14.5 captures this fitted scenario, where the rate γ depends on domain volatility and observed outcomes:

$$\text{Accuracy}(t) = \text{Accuracy}_0 \cdot e^{-\gamma t} \quad (14.5)$$

The cost of staleness accumulates based on query volume Q per time period and the value impact V of a unit change in accuracy fraction. Integrating the instantaneous accuracy loss ($\text{Accuracy}_0 - \text{Accuracy}(t)$) over the retraining interval T yields equation 14.6:

$$\begin{aligned} \text{Staleness Cost}(T) &= \int_0^T Q \cdot V \cdot (\text{Accuracy}_0 - \text{Accuracy}(t)) dt = \\ &= Q \cdot V \cdot \text{Accuracy}_0 \cdot \left(T - \frac{1 - e^{-\gamma T}}{\gamma} \right) \end{aligned} \quad (14.6)$$

The integral accumulates cost over time t from 0 to T , and the closed form follows from substituting equation 14.5 for $\text{Accuracy}(t)$.

The retraining cost function Each retraining incurs fixed costs including compute, validation, and deployment overhead. Equation 14.7 decomposes these:

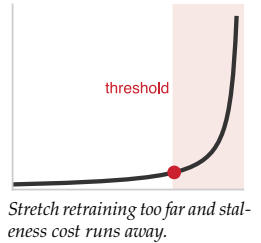
$$\text{Retraining Cost} = C_{\text{compute}} + C_{\text{validation}} + C_{\text{deployment}} + C_{\text{risk}} \quad (14.7)$$

where C_{compute} is the cost of the training run itself, $C_{\text{validation}}$ is the cost of evaluating the new model before release, $C_{\text{deployment}}$ is the cost of rolling it into production, and C_{risk} is the expected cost of potential regression from the new model.

Optimal retraining interval The optimal retraining interval T^* minimizes total cost per unit time, as equation 14.8 shows:

$$T^* = \arg \min_T \frac{\text{Staleness Cost}(T) + \text{Retraining Cost}}{T} \quad (14.8)$$

Under the assumption $\gamma T \ll 1$, a Taylor expansion of the exponential yields the square-root approximation used in the earlier napkin calculation. Specifically, the second-order Taylor approximation $e^{-\gamma T} \approx 1 - \gamma T + \frac{1}{2}(\gamma T)^2$ reduces the bracketed staleness term $\left(T - \frac{1 - e^{-\gamma T}}{\gamma} \right)$ to $\frac{1}{2}\gamma T^2$. Dividing total costs by T yields an average cost rate of $\frac{1}{2}QV\text{Accuracy}_0\gamma T + \frac{C}{T}$; differentiating with respect



to T and setting the derivative to zero balances marginal staleness loss against amortized setup expense, directly yielding the square-root economic interval in equation 14.4. With the parameters in table 14.9, the approximation places the economic optimum near one day; an exact optimum requires minimizing equation 14.8 without the expansion.

Table 14.9: Retraining Decision Parameters: Example values for a fraud detection system processing 1,000,000 transactions daily.

Parameter	Value	Description
Q	1,000,000	Transactions per day
V	\$0.50/query	Value per query for a unit accuracy change
Accuracy_0	0.95	Initial accuracy
γ	0.02	Daily decay rate (2% per day)
Retraining Cost	\$5,000	Total retraining expense

Sensitivity analysis Under the square-root approximation, T^* scales with the square root of these parameters. Table 14.10 shows that a fourfold change in retraining cost, query volume, or decay rate moves the approximated interval twofold.

Table 14.10: Retraining Interval Sensitivity: Under the square-root approximation a fourfold change in an input moves the interval only twofold, and the directions differ. More expensive retraining lengthens the interval, while higher query volume or a faster decay rate shortens it.

Change	Effect on T^*
4× retraining cost	2× longer interval
4× query volume	2× shorter interval
4× decay rate	2× shorter interval

Model limitations This closed-form framework provides a first-order approximation for operational planning, but its utility depends on several simplifying assumptions. First, the exponential decay formulation presumes gradual drift at a steady rate γ ; sudden covariate shifts or acute data outages require immediate statistical change-point detection rather than smooth decay models. Second, the formulation assumes a constant marginal value V per accuracy point, whereas real-world utility often exhibits nonlinear thresholds or asymmetric penalties. Third, the model treats retraining events as independent, static restarts, setting aside transfer learning across retraining cycles. Finally, infrastructure costs are modeled as fixed sums, whereas cloud spot pricing, queue contention, and elastic compute availability introduce dynamic variations. Despite these boundaries, the derivation grounds retraining cadence in economic trade-offs rather than arbitrary calendar schedules. Parameters improve through empirical calibration against historical telemetry as operational experience accumulates. By making cost-benefit trade-offs explicit and quantifiable, this framework implements cost-aware automation (section 14.2.1), enabling justified infrastructure investments and monitoring thresholds grounded in measurable business impact.

14.4.2.3 Model validation

Training pipelines produce model candidates; model validation determines which candidates merit production deployment. Unlike research evaluation, where a model that beats a benchmark on a static test set may be considered successful, production validation must assess operational readiness under representative conditions and establish the monitoring and rollback controls needed after the distribution changes.

The evaluation process begins with performance testing against a holdout test set sampled from the same distribution as production data. Core metrics such as accuracy, area under the curve (AUC), precision, recall, and F1 score (Rainio et al. 2024) are computed and tracked longitudinally to detect degradation from data drift (IBM 2024). The three aligned panels in figure 14.7 show this degradation pattern concretely. The top panel presents incoming data samples over time, color-coded by type. The middle panel shows an associated change: a feature distribution (`sales_channel`) gradually shifting from predominantly online to predominantly offline transactions. The bottom panel shows

model accuracy declining over the same interval. This visualization captures the core challenge of model validation: the need to monitor *inputs* alongside *outputs* to understand why performance changes.

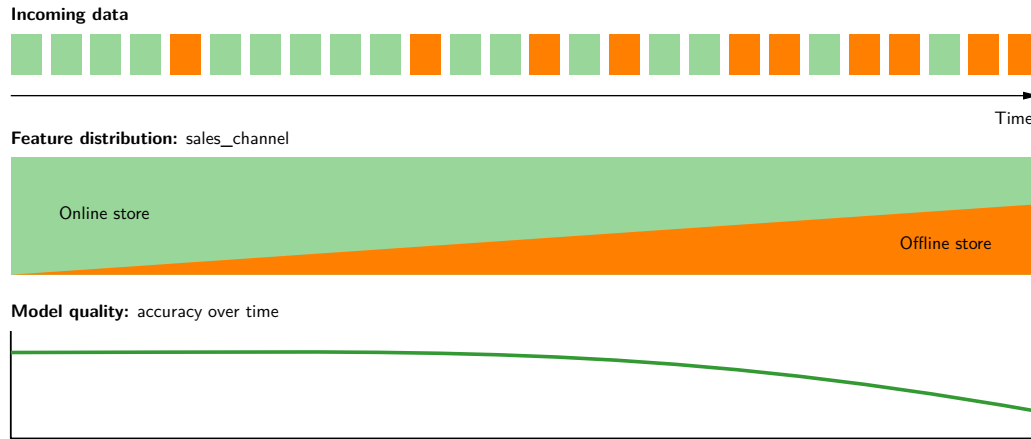


Figure 14.7: Data Drift Impact: In this illustrative scenario, incoming samples shift from predominantly online to increasingly offline sales while model accuracy declines over the same interval. Monitoring inputs and outcomes together helps determine whether the distribution shift is associated with the loss in accuracy.

Beyond static evaluation, production pipelines employ controlled deployment strategies that expose candidate weights to live serving environments while bounding blast radius. One widely adopted method is canary testing (Fowler 2014), in which a new model receives a small fraction of production traffic. During this limited rollout, telemetry pipelines monitor inference latency, system stability, and prediction distributions. For instance, an e-commerce platform deploys a candidate recommendation model to 5 percent of live query traffic, monitoring click-through rates, p99 latency, and prediction drift against the baseline model. Only after the candidate demonstrates statistical stability without latency regressions is it promoted to serve full production traffic.

Evaluating candidates under identical operational conditions is the prerequisite for a sound promotion decision; a candidate that shows apparent metric gains simply because it was evaluated against a differing traffic slice or temporal window confounds model quality with data variance. Cloud ML platforms support controlled evaluation through request replay, shadow serving, and synthetic load testing. Platforms such as Weights & Biases (Weights & Biases, Inc. 2024) capture the exact training artifacts, hyperparameter configurations, and evaluation metrics required to make promotion decisions reproducible and auditable across the model lifecycle.

While automated validation gates protect against regressions on primary aggregate metrics, production validation also requires auditing slices and long-tail behavior. Automated test suites often mask severe performance drops on underrepresented subpopulations or safety-critical edge cases. Production architectures combine automated gating with policy reviews, particularly for models deployed in regulated domains. This multi-stage validation discipline bridges offline evaluation with live production monitoring, ensuring that candidate models satisfy accuracy, latency, and safety invariants before receiving unconstrained serving traffic.

14.4.3 Infrastructure integration

Development infrastructure addresses two of the three critical system interfaces. Feature stores and data versioning reduce inconsistency at the data-model interface by centralizing and tracking feature access across training and serving. CI/CD pipelines, model registries, and validation gates stabilize the model-infrastructure interface by automating the transition from trained weights to containerized services with rollback capability.

These represent only two-thirds of the operational challenge. A model that passes every validation gate and deploys cleanly can still fail silently in production as input distributions shift. The third critical interface, production-monitoring, requires practices focused not on building models, but on preserving predictive correctness over time.

14.5 Production Operations

A deployed model has an operational half-life. While conventional software binaries maintain deterministic correctness as long as underlying hardware invariants hold, machine learning models couple deterministic matrix arithmetic to an external, non-stationary data distribution. As live input features diverge from the training manifold, predictions degrade silently even while cluster health checks—from HTTP response codes to tail latency (p_{99})—remain completely green. Production operations implements the production-monitoring interface to make this silent decay visible and actionable. By pairing progressive traffic routing with runtime drift detection, operational infrastructure isolates failing model versions and triggers mitigation before erroneous inferences propagate to downstream systems.

14.5.1 Model deployment and serving

Once trained and validated, a model must be integrated into a production environment that delivers predictions at scale. Deployment transforms a static checkpoint into a versioned, reproducible execution artifact; serving allocates accelerator memory, stages input batches, and schedules compute kernels to satisfy latency SLOs under dynamic request loads. Together, these systems operationalize the model within the production software ecosystem.

14.5.1.1 Model deployment

Consider a fraud detection model that achieves 99.2 percent precision in the development environment. An engineer exports the weights, copies them to a production server, and discovers the model predicts every transaction as legitimate: the production server runs a different version of the feature extraction library, producing inputs the model has never seen. This scenario illustrates why deployment is a systems engineering problem rather than a simple file transfer. Packaging, testing, and tracking ML models for reliable production deployment requires treating the model weights, runtime dependencies, and preprocessing configuration as a single deployable unit. Containerizing the model artifact and serving runtime¹³ packages user-space dependencies, runtime libraries, and environment variables into an immutable image, enforcing environment parity between development and cluster nodes.

Production deployment requires frameworks that handle model packaging, versioning, and integration with serving infrastructure. Tools like MLflow and model registries manage these deployment artifacts (A. Chen et al. 2020), while serving-specific frameworks (detailed in chapter 13) handle the runtime optimization and scaling requirements. Before full-scale rollout, teams deploy updated models to staging or QA environments¹⁴ to rigorously test performance.

Shadow deployments¹⁵ duplicate live production traffic asynchronously to a candidate model without returning its inferences to users, enabling latency and output comparison without direct user exposure; accuracy still requires labels or another ground-truth reference. **Canary deployments**¹⁶ route a small, incrementally ramped share of user traffic to detect regressions under live load. **Blue-green deployments**¹⁷ maintain parallel environments for router-level cutover and rapid rollback. Each strategy reduces a different release risk, but none removes the need for tested rollback procedures and production monitoring.



War Story 14.1: The Knight Capital error (2012)

Context: Knight Capital Group was a major market maker in US equities handling over 17 percent of NYSE volume. In August 2012, an operational deployment updated SMARS order-routing software on seven of eight production servers but omitted the eighth (U.S. Securities and Exchange Commission 2013).

Mechanism: The deployment repurposed a binary feature flag that activated legacy Power Peg code on the un-updated eighth server. The stale server entered an un-throttled loop, transmitting parent order requests at $\sim 1,400$ orders/sec.

Impact: In 45 minutes, the router executed over 4 million unauthorized transactions covering more than 397 million shares, incurring a \$460 million loss. The firm survived only on emer-

¹³ | **Containerization for ML deployment:** Docker (Merkel 2014) packages code with dependencies into portable units; Kubernetes (Burns et al. 2016) orchestrates those units across clusters. Containerization addresses the Environment_o term in equation 14.1 partially by capturing user-space dependencies as a versioned artifact; host kernels, drivers, and hardware remain external dependencies.

¹⁴ | **Staging validation:** ML staging adds probabilistic adequacy checks to conventional functional and operational validation. A model can pass unit tests and still fail in production because the test data does not reflect the deployment distribution, so rollout gates must compare prediction statistics, evaluation slices, and business guardrails against calibrated thresholds rather than rely on unit tests alone.

¹⁵ | **Shadow deployment:** Economically justified when its expected reduction in rollout loss exceeds the cost of shadow infrastructure for duplicated inference without serving results.

¹⁶ | **Canary deployment:** Routes a small fraction of live traffic to a candidate model, using it as a sentinel for production health. The ML-specific challenge is that a “failure” is statistical degradation, not a deterministic crash: detecting a small accuracy difference with high confidence can require thousands of inferences, creating a tension between decision speed and statistical power that determines minimum canary duration.

gency rescue financing raised days later, and lost its independence in an acquisition within months.

Response: The incident motivates atomic deployment verification across production nodes, explicit feature-flag lifecycle controls, and circuit breakers that limit the damage a faulty release can cause.

Systems lesson: Automated deployment is a physical control problem where configuration drift across heterogeneous nodes causes system-wide service failure. ML deployments share this exact vulnerability: a model registry version mismatch, un-synchronized feature schemas, or partial canary routing can flood production with corrupted inference requests before telemetry alerts fire.

Avoiding the Knight Capital failure mode motivates why ML systems stage rollouts rather than switch traffic atomically, but staged deployment exposes load-dependent behavior. When canary deployments reveal anomalies at elevated traffic shares—such as memory exhaustion, queue spillover, or p99 tail latency inflation appearing at 30 percent traffic that remained undetectable at 5 percent—isolation requires systematic debugging across the stack. Effective diagnosis requires correlating multiple signals: performance metrics from chapter 12, data distribution analysis to detect drift, and feature importance shifts that might explain degradation. Teams maintain debug toolkits including A/B test analysis frameworks, feature attribution tools, and data slice analyzers that identify which subpopulations are experiencing degraded performance.

That diagnosis loop must connect directly to the release pipeline. CI/CD integration automates deployment and rollback, but only when rollback is designed as part of the rollout mechanism rather than treated as an emergency script.

Rollback strategies and safety mechanisms Rollback¹⁸ capability is the safety net that enables confident deployment. Without reliable rollback, teams become deployment-averse and slow their iteration velocity. Effective rollback requires planning for three distinct scenarios:

The fastest tier, **immediate rollback**, addresses critical failures detected right after deployment: serving errors, latency spikes, or obvious prediction failures. It requires keeping the previous model version loaded and warm so traffic can switch without cold-start delay. **Rapid rollback** handles performance degradation detected through canary metrics soon after deployment, which requires model registry integration that keeps previous versions deployable with minimal configuration changes. **Delayed rollback** addresses subtle issues detected through business metrics or user feedback after full deployment, where rollback must account for model-dependent data such as personalization state or cached embeddings accumulated during the new model's operation.

Table 14.11 summarizes implementation patterns for each rollback type:

Table 14.11: Rollback Patterns by Scenario: Each rollback type requires different infrastructure support and state handling strategies. Immediate rollback demands always-warm standbys; delayed rollback may require data migration procedures.

Rollback Type	Trigger	Implementation	State Handling
Immediate	Serving errors, crashes	Hot standby with instant switch	Stateless—no special handling
Rapid	Canary metric degradation	Registry-based redeployment	Clear caches, restart sessions
Delayed	Business metric decline	Full redeployment with migration	Migrate state, replay if needed

Rollback testing Rollback procedures that have never been tested may fail when needed, and the gap is often discovered during an active incident, when cognitive load and time pressure are highest. Failure can stem from configuration dependencies, incompatible state, or unclear ownership. Periodic rollback exercises expose these gaps before they matter. Automated rollback criteria can shorten reaction time, but thresholds must be calibrated to the service and include safeguards against flapping. The restored model must also produce consistent behavior rather than predictions corrupted by stale caches or incompatible feature state. Step-by-step runbooks ensure that the responder need not be the person who designed the deployment.

17 | Blue-green deployment: Maintains two comparable production environments, “blue” (serving current traffic) and “green” (running the candidate), then switches traffic in a routing change once the green environment passes validation. Because rollback is a traffic flip rather than a staged drain, recovery can be faster than a gradual canary for stateless services. The trade-off is extra infrastructure during the switch, so blue-green wins over canary when brief duplicate capacity is acceptable and per-segment statistical validation is expensive.

18 | Rollback (from database transaction management): This “undo” action for deployments is complicated in ML by model-dependent state (for example, cached embeddings), which may be incompatible between model versions. This mismatch can prolong recovery and contribute to deployment aversion and slower iteration.

Stateful vs. stateless rollback ML systems vary in statefulness, which directly governs rollback complexity. For stateless workloads like tabular classification and static regression, rollback requires only swapping model weights or routing traffic back to the prior container, because individual inferences share no execution context. In contrast, stateful models—including sequential recommenders, session-based search rankers, and conversational systems—accumulate transient user state. Reverting such workloads can demand session resets, schema shims for cached context vectors, or explicit state migration; capturing versioned checkpoints at deployment boundaries mitigates this friction, but clean restoration remains an architectural property that requires empirical validation rather than assumption. For instance, two-tower recommendation and search services frequently precalculate and store dense user and item embeddings in low-latency caches; if Model v2 rotates the latent vector space or changes embedding dimensions, those cached vectors become mathematical nonsense to a rolled-back Model v1. A stateful rollback must therefore orchestrate atomic cache invalidation or dual-version namespace switching alongside weight redeployment to prevent scoring queries against incompatible vector spaces. The most complex failure mode arises in systems with feedback loops: when bad predictions pollute downstream feature logs or clickstream training sets during the degraded window, reverting the serving binary leaves contaminated data in the retraining corpus that must be expunged or quarantined before previous behavior is restored.

A/B testing for model validation A/B testing provides the statistical foundation for deployment decisions by comparing model versions under controlled conditions. Unlike canary deployments (which validate operational stability), A/B tests measure whether a new model improves business outcomes with statistical confidence.

Experiment setup and decision rules A valid A/B test establishes four controls to ensure deployment decisions remain statistically sound:

First, the **randomization unit** specifies whether traffic is split by user ID or individual request. User-level hashing preserves consistency across sessions at the expense of larger sample requirements, whereas request-level splitting maximizes sample throughput but risks exposing a single user to alternating model variants.

Second, the statistical power envelope determines the traffic volume required to resolve meaningful differences. Under a common-variance Normal approximation, let n be the required users per variant, δ the minimum detectable effect, σ the outcome standard deviation, and $z_{\alpha/2}$ and z_{β} the positive standard-Normal critical values for the chosen two-sided confidence and power. The worked scenario uses 95 percent confidence and 80 percent power. Equation 14.9 translates those design targets into required traffic before launch.

$$n = \frac{2(z_{\alpha/2} + z_{\beta})^2 \sigma^2}{\delta^2} \quad (14.9)$$

For a 2 percent relative lift on a 5 percent baseline conversion rate (5 percent to 5.1 percent) and 80 percent power, the experiment requires roughly 745,644 users per variant; with 25,000 users per variant, the test can detect only a much larger lift, about 0.5 percentage points absolute.

Third, **guardrail metrics** define operational and behavioral boundaries that must not degrade even if the primary metric improves. A recommendation model improving click-through rate by 10 percent while increasing p99 response latency by 500 ms violates system guardrails and risks cascading timeouts across downstream services.

Fourth, a fixed evaluation horizon requires running the test until the preregistered sample size and minimum duration are reached, capturing cyclical weekly seasonality. Stopping an experiment the moment significance first appears inflates false positive rates through repeated peeking; early stopping requires a prespecified sequential design with calibrated alpha-spending boundaries.

Those controls establish the statistical envelope, but ML systems add failure modes that ordinary web experiments can hide. Conversion events may arrive days after prediction, creating delayed feedback: a recommendation shown Monday can drive a purchase Friday, so the attribution window must be part of the test design. Novelty effects can also inflate early performance as users engage with fresh recommendations, which is why mature experiments include a burn-in period before measurement.

Recommendation and ranking systems add interference effects because showing an item to one user can affect what remains available or salient for another, violating the independence assumption

behind standard A/B analysis. Segment heterogeneity creates a second analysis problem: an overall neutral result may hide strong positive effects for one cohort and negative effects for another. These complications do not invalidate A/B testing, but they make guardrails, segment analysis, and preregistered decisions part of the experiment rather than after-the-fact interpretation.

Table 14.12 turns those constraints into a deployment decision:

Table 14.12: A/B Test Decision Matrix: Deployment decisions should consider both primary metrics and guardrails. Improvements that come at the cost of guardrail violations require careful trade-off analysis rather than automatic deployment.

Primary Metric	Guardrails	Decision
Significant improvement	All pass	Ship new model
Significant improvement	Some fail	Investigate trade-offs, may need model iteration
No significant change	All pass	Evidence inconclusive; retain current model unless a prespecified equivalence or noninferiority test supports the change
Significant degradation	N/A	Do not ship; investigate root cause

The decision matrix is reliable only when the evaluation methodology is preregistered before launch. Teams must establish expected effect sizes, the randomization unit, attribution windows, guardrail thresholds, and minimum runtimes prior to observing production traffic.

Interim evaluation demands equal rigor. Variance-reduction techniques such as CUPED (Controlled-experiment Using Pre-Experiment Data) reduce metric variance by adjusting outcomes using pre-experiment covariates, accelerating decision velocity without sacrificing statistical power. Automated analysis pipelines protect the decision boundary from manual calculation errors, while archiving negative results preserves empirical evidence against repeating unproductive architectures.

An A/B decision only matters if the release machinery can promote, hold, or roll back the exact artifact that was tested. Model registries, such as Vertex AI’s model registry (Google Cloud 2024d), act as centralized repositories for storing and managing trained models and versions. Model catalogs serve a distinct role: while registries manage internal, versioned deployment artifacts, catalogs such as Vertex AI Model Garden (Google Cloud 2026a) index external and foundation model families (such as Llama 2 (Touvron, Martin, et al. 2023)) for evaluation, fine-tuning, and initial selection.

Inference endpoints expose the validated model to live traffic. While lightweight services often use REST APIs over HTTP/1.1 for real-time scoring, high-throughput systems adopt binary RPC protocols such as gRPC over HTTP/2 to eliminate JSON serialization bottlenecks and reduce CPU parsing overhead. Depending on performance requirements, serving systems configure dedicated accelerator instances (such as GPUs or TPUs) or leverage serverless inference¹⁹ that scales compute capacity to zero when idle.

To maintain lineage and auditability, teams track model artifacts, including scripts, weights, logs, and metrics, using tools like MLflow²⁰ (Databricks 2024). Together, registries, endpoints, lineage tracking, and distributed orchestration frameworks like Ray²¹ turn A/B outcomes into controlled production changes: the tested model can be promoted, observed, and reversed without losing provenance.

14.5.1.2 Model format optimization

An unoptimized PyTorch model in eager mode dispatches operators as discrete kernel launches through the Python interpreter, repeatedly round-tripping intermediate activation tensors through high-bandwidth memory (HBM) and incurring launch overhead that inflates p99 latency well beyond strict SLOs. Format optimization compiles the dynamic computation graph into a static, hardware-tuned intermediate representation that fuses adjacent kernels, folds constant weights, and eliminates dead code. While section 13.9 and section 13.9.2 detail the underlying runtimes and numerical precision choices, operationalizing these formats requires governing the conversion and validation workflow.

The first operational boundary is representation. Open Neural Network Exchange (ONNX) is a widely used interchange format for model portability, but the choice of runtime and execution provider determines the hardware targets, supported operators, and optimizations available. ONNX Runtime emphasizes a common API across multiple backends, while TensorRT specializes execution

19 | Serverless ML inference: The cost-efficiency of this option stems from provisioning compute only upon request and scaling to zero when idle, eliminating the cost of a persistent endpoint. This creates a direct tension with performance targets, as the first request after an idle period incurs a “cold start” latency penalty while the model is loaded into memory. For large models, that delay can be long enough to violate real-time latency budgets unless the service keeps warm capacity or uses a runtime designed for fast loading.

20 | MLflow: An open-source MLOps framework that couples artifact storage (S3/GCS) with metadata DBs (PostgreSQL) to record hyperparameter runs, model binary hashes, and deployment stage transitions. The systems trade-off is centralizing lineage tracking vs. incurring database connection bottlenecks during massive parallel hyperparameter sweeps.

21 | Ray: A distributed computing framework from UC Berkeley (Moritz et al. 2018) that provides a unified task and actor interface, backed by a distributed scheduler and fault-tolerant store. The broader MLOps lesson is that fragmented infrastructure creates translation points where preprocessing logic, normalization constants, tokenizer versions, or artifact formats can diverge silently. Shared execution abstractions can reduce that fragmentation, but training-serving skew still requires explicit consistency checks across data, features, and serving code.

for NVIDIA GPUs; actual performance must be measured for the target model and hardware. A typical workflow exports a PyTorch model to ONNX, performs graph cleanup (constant folding and dead-code elimination), fuses compatible operators, selects precision, and validates numerical and task-level behavior against the source model at each step. The representative options in table 14.13 differ in portability, hardware scope, and optimization mechanisms.

Table 14.13: Model Optimization Frameworks: Six representative frameworks differ in supported source formats, target hardware, and optimization mechanisms. TensorRT and TF-TRT are NVIDIA-specific, whereas ONNX Runtime and OpenVINO span multiple processor classes.

Framework	Source Formats	Target Hardware	Key Optimizations
ONNX Runtime	PyTorch, TF, Keras, scikit	CPU, GPU, NPU	Graph optimization, operator fusion, quantization
TensorRT	ONNX, TF, PyTorch	NVIDIA GPU only	Kernel auto-tuning, precision calibration, layer fusion
OpenVINO	ONNX, TensorFlow / TFLite, PyTorch, PaddlePaddle	Intel CPU, GPU, NPU	Model compression, async execution, caching
TF-TRT	TensorFlow	NVIDIA GPU	TensorRT integration within TensorFlow graph
Core ML	TensorFlow, PyTorch	Apple Neural Engine, GPU, CPU	Unified format for Apple devices, on-device inference
TFLite	TensorFlow, Keras	Mobile CPU, GPU, Edge TPU	Quantization, delegate support, model compression

The durable architectural trade-off is portability versus peak performance: every gain in accelerator throughput is purchased through specialized runtime commitments and operator constraints. Numerical precision forms the second operational boundary. Lowering numerical precision through quantization (e.g., from FP32 or FP16 down to INT8 or FP8) doubles arithmetic throughput and relieves memory bandwidth pressure on tensor cores. From an operational perspective, the primary deployment hurdle in quantization is verifying that INT8 or FP8 representations preserve numerical fidelity across live, non-stationary production traffic distributions rather than static calibration slices. The mechanics of post-training quantization (PTQ), quantization-aware training (QAT), and mixed precision appear in chapter 10, with runtime precision selection covered in section 13.9.2.

Production deployment of optimized models therefore requires validation that targets the failure modes optimization can introduce silently. Consider a team that deploys an INT8-quantized model after verifying only throughput improvement: classification accuracy drops on rare but high-value edge cases, and the degradation goes undetected for weeks because aggregate metrics remain within SLO bounds. The first validation layer is numerical equivalence, comparing optimized outputs against the original model on a representative test set with application-specific divergence thresholds. That check is necessary but insufficient because rare inputs, out-of-distribution examples, and subgroup-specific cases can expose quantization artifacts that aggregate test metrics hide.

The second layer is operational validation. Memory footprint must be measured at peak runtime utilization, including dynamic allocations during inference, since some optimizations trade increased runtime memory for computational speed. Warm-up behavior varies by runtime: Accelerated Linear Algebra may JIT-compile kernels during initial execution, whereas TensorRT selects tactics and builds an engine before deployment; loading that engine can still add startup latency. Runtime version compatibility then closes the loop: deployment configurations need explicit version pinning because even minor runtime changes can affect both performance characteristics and numerical correctness.

14.5.1.3 Inference serving

A serialized model artifact on disk produces value only when paired with execution infrastructure that receives network requests, stages inputs, schedules accelerator compute, and returns predictions within strict latency bounds. Operating serving infrastructure at hyperscale—such as the tens of trillions of daily inferences reported across fleet architectures (Hazelwood et al. 2018)—demands continuous lifecycle management alongside the formal SLOs detailed in chapter 13.

Production-grade serving frameworks such as TensorFlow Serving (Olston et al. 2017), NVIDIA Triton Inference Server (NVIDIA 2024d), and KServe (KServe Community 2024) provide standardized mechanisms for deploying, versioning, and scaling models. From an operational perspective, the key decision is which framework best fits the deployment context: TensorFlow Serving for TensorFlow-native workflows, Triton for multi-framework GPU serving, and KServe for Kubernetes-native environments requiring scale-to-zero.

Regardless of which serving paradigm is used (online, offline, or near-online, as detailed in section 13.2.2), model inference is often only a fraction of total end-to-end latency. Decomposing the latency budget reveals whether the operational bottleneck lies in the model or elsewhere in the request path.

 Systems Perspective 14.1: The latency budget

A service has a 100 ms p99 SLO, and the model inference budget is 45 ms; the remaining 55 ms must cover every other stage of the request path. Table 14.14 allocates the budget across the request lifecycle.

Table 14.14: Latency Budget Components: Representative allocation of a 100 ms p99 SLO across the request lifecycle. Model inference accounts for 45 percent of total latency, leaving the majority to network, feature retrieval, parsing, postprocessing, and serialization. The optimization-lever column shows where each component can be reduced.

Component	Budget Share	p99 Budget	Optimization Lever
Network RTT	15%	15 ms	Edge deployment, connection pooling
Feature retrieval	25%	25 ms	Feature caching, precomputation
Request parsing	5%	5 ms	Binary protocols (gRPC), schema optimization
Model inference	45%	45 ms	Quantization, batching, model distillation
Postprocessing	5%	5 ms	Async processing, result caching
Response serialization	5%	5 ms	Efficient formats (Protobuf, MessagePack)

Systems insight: Model optimization alone often captures less than 50 percent of the latency opportunity. A model that runs 2× faster reduces this example from 100 ms to 77.5 ms, only 1.3× end-to-end improvement, because inference is 45 percent of total latency.

Systems thinking demands end-to-end analysis. Apply the D-A-M taxonomy to diagnose the root cause across Data (feature extraction overhead, serialization cost), Algorithm (too many layers, unoptimized graph), and Machine (memory bandwidth saturation, thermal throttling). Measure end-to-end performance and optimize the binding bottleneck. If feature retrieval exceeds its budget, no amount of model optimization will achieve the SLO.

Beyond the latency budget, operationalizing serving requires selecting infrastructure techniques for the constraint the budget exposed. Table 14.15 summarizes representative strategies for ML-as-a-service infrastructure; the organizing question is whether the bottleneck lies in queueing delay, capacity, routing, orchestration overhead, or latency prediction.

Table 14.15: Serving System Techniques: Scalable ML-as-a-service infrastructure relies on techniques like request scheduling and instance selection to optimize resource utilization and reduce latency under high load. For the underlying queuing theory and batching strategies, see chapter 13.

Technique	Description	Example System
Request scheduling & batching	Groups inference requests to improve throughput and reduce overhead	Clipper (Crankshaw et al. 2017)
Instance Selection & Routing	Dynamically assigns requests to model variants based on constraints	INFaaS (Romero et al. 2021)
Predictive Autoscaling	Adds capacity ahead of demand spikes to meet latency SLOs	MArk (Zhang et al. 2019)
Autoscaling	Adjusts model instances to match workload demands	INFaaS

Technique	Description	Example System
Model Orchestration	Coordinates execution across model components or pipelines	AlpaServe (Li et al. 2023)
Execution Time Prediction	Forecasts latency to optimize request scheduling	Clockwork (Gujarati et al. 2020)

These strategies form the cloud-serving foundation. Edge deployment keeps the same operational goal but changes the constraints: rollback, telemetry, and update control must work on devices with limited power, memory, and connectivity.

14.5.1.4 Edge AI deployment

Consider a smoke detector with an ML model for distinguishing cooking smoke from fire. When this model degrades, an engineer cannot simply SSH into the device, roll back to a previous version, and restart. The device sits on someone’s ceiling with intermittent Wi-Fi, a coin-cell battery, and 256 KB of memory. Every operational assumption from cloud MLOps (instant rollback, centralized logging, real-time monitoring) must be reimaged.

Edge AI represents this shift: machine learning inference occurs at or near the data source rather than in centralized cloud infrastructure (Reddi et al. 2019). Workloads governed by strict latency deadlines, bandwidth-constrained uplinks, data-privacy guarantees, or sub-watt power envelopes require executing inference directly on edge silicon. This operational paradigm introduces three interdependent systems constraints: resource limits, hierarchical tiered execution, and fail-safe over-the-air update mechanisms.

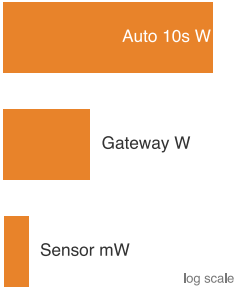
Resource constraints dominate edge deployment decisions. Edge devices require the aggressive model optimization techniques established in chapter 10 (quantization, pruning, knowledge distillation) to meet the memory and power envelopes of microcontroller-class deployments (Warden and Situnayake 2020; David et al. 2021). Power budgets span four orders of magnitude, from milliwatts for IoT sensors to tens of watts in automotive systems, demanding power-aware inference scheduling and thermal management. Some hard real-time, safety-critical applications impose deterministic timing targets that require worst-case execution time (WCET) analysis under adverse conditions including thermal throttling and memory contention.

These constraints shape a natural deployment hierarchy across three tiers. Sensor-level processing handles immediate data filtering and feature extraction on microcontroller-class devices consuming 1–100 mW. Edge gateway processing performs intermediate inference on application processors with 1–10 W power budgets. Cloud coordination manages model distribution, aggregated learning, and complex reasoning requiring GPU-class resources. This hierarchy enables system-wide optimization: computationally expensive operations migrate upward while latency-critical decisions remain local.

Two deployment contexts deserve specific attention. TinyML targets microcontroller-based inference under tight memory and milliwatt-class power constraints, requiring specialized engines such as TensorFlow Lite Micro and CMSIS-NN (David et al. 2021; Lai et al. 2018). Model architectures must be co-designed with hardware constraints, favoring compact operators, quantization, and pruning strategies whose aggressiveness depends on the device and accuracy target. Mobile AI extends edge deployment to smartphones with moderate compute, using NPUs and GPU compute shaders to meet interactive latency and battery-life constraints through power-aware scheduling.

Updates and monitoring complete the edge operational picture. Over-the-air (OTA) model updates enable maintenance for physically inaccessible systems. OTA pipelines need secure model distribution, artifact verification, and rollback mechanisms; delta or differential updates can reduce transfer volume when devices need not receive a complete model artifact. On microcontroller and embedded targets, fail-safe OTA rollback relies physically on dual-bank flash memory: the operating system boots from Bank A while writing the candidate model and runtime to Bank B. An active hardware watchdog timer monitors initial inferences post-update; if the new model triggers memory faults, hangs, or violates latency budgets, the bootloader automatically reverts execution to Bank A, preventing physically inaccessible devices from being bricked. Update scheduling must account for device connectivity patterns, power availability, and operational criticality.

Monitoring requires adaptation to resource-constrained environments: lightweight telemetry systems capture essential metrics (inference latency, power consumption, accuracy indicators) while minimizing overhead. Health monitoring tracks device-level conditions (thermal status, battery



Edge power budgets span sensors, gateways, and vehicles across orders of magnitude.

levels, connectivity quality) to predict maintenance needs. Edge-cloud coordination patterns enable adaptive offloading between tiers based on current load, network conditions, and latency requirements. Feature caching at edge gateways reduces redundant computation, while federated learning lets edge devices contribute model updates rather than raw training records. Raw parameter updates remain vulnerable to gradient inversion and reconstruction attacks unless paired with differential privacy or secure aggregation protocols.

Graceful degradation is the defining operational pattern for edge AI. When thermal limits or battery depletion constrain hardware capacity, edge runtimes must dynamically shed load—downsampling input sample rates, bypassing speculative execution branches, or falling back to shallow heuristic models—to guarantee continuous safe operation.

Getting models into production is only half the operational challenge. A successfully deployed model can degrade through distribution shift or corrupted upstream features without raising runtime errors—the silent failure modes characteristic of machine learning systems. Robust monitoring, incident response, and on-call practices close this loop.

14.5.2 Resource management and monitoring

Deployment and serving get models into production. Keeping them healthy requires two complementary disciplines: resource management (provisioning and scaling compute, storage, and networking) and monitoring (observing system behavior and detecting degradation before users notice).

14.5.2.1 Infrastructure management

Three failures illustrate the problem. A model works in staging but fails in production because someone manually provisioned a different GPU type. A training job crashes because a colleague’s experiment consumed all available memory. An inference service cannot scale because its resource quotas were set through an informal message six months earlier. These failures share a root cause: infrastructure managed through manual processes rather than code.

Scalable, resilient infrastructure is foundational for operationalizing ML systems, and **infrastructure as code (IaC)** is the practice that makes it reliable. IaC treats infrastructure configuration as software (version-controlled, reviewed, tested, and automatically executed) rather than manually configured through graphical interfaces or command-line tools. This approach brings software engineering discipline to resource management: changes are tracked, configurations can be tested before deployment, and environments can be reliably reproduced.

The specific infrastructure tool matters less than the contract it enforces. Terraform (HashiCorp 2014), AWS CloudFormation (Amazon Web Services 2024e), and Ansible (Hatcher 2024) represent common ways to version infrastructure definitions alongside application code. In MLOps settings, that versioned definition is what lets a team reproduce the GPU type, network policy, storage permissions, and scaling limits used by a training or serving environment across AWS (Amazon Web Services 2024c), Google Cloud Platform (Google Cloud 2024a), Microsoft Azure (Microsoft 2024a), or on-premises infrastructure.

Infrastructure management spans the full ML lifecycle. During training, IaC scripts allocate compute instances with GPU or TPU accelerators, configure distributed storage, and deploy container clusters. Because infrastructure definitions are stored as code, they can be audited, reused, and integrated into CI/CD pipelines ensuring consistency across environments.

Containerization provides the same reproducibility boundary for runtime dependencies. Docker (Merkel 2014) packages the model, libraries, and serving code into an isolated unit, while orchestration systems such as Kubernetes (Cloud Native Computing Foundation 2024a) manage those units across clusters. The operational value is not the container name; it is the ability to deploy the same artifact repeatedly while resource allocation, scaling, and health management remain explicit.

To handle changes in workload intensity, including spikes during hyperparameter tuning and surges in prediction traffic, teams rely on cloud elasticity and autoscaling.²² Cloud platforms support on-demand provisioning and horizontal scaling of infrastructure resources. Autoscaling mechanisms (Amazon Web Services 2024d) automatically adjust compute capacity based on usage metrics, enabling teams to optimize for both performance and cost-efficiency.

²² | **ML autoscaling:** Autoscaling adjusts capacity based on demand signals (Amazon Web Services 2024d), but ML serving adds constraints absent from stateless web services. Autoscaling decisions must account for model loading time (cold-start overhead), GPU memory fragmentation, and batching behavior in addition to CPU utilization. Scaling up too slowly violates latency SLOs; scaling down too aggressively forces repeated cold starts that degrade p99 latency.

Infrastructure in MLOps is not limited to the cloud. Many deployments span on-premises, cloud, and edge environments, depending on latency, privacy, or regulatory constraints. A robust infrastructure management strategy must accommodate this diversity by offering flexible deployment targets and consistent configuration management across environments.

Infrastructure as code addresses how to provision resources reproducibly, but operators must still decide when and how much capacity to allocate. ML workloads break the assumptions of conventional web provisioning. Training jobs demand high-concurrency bursts across distributed accelerator topologies, creating friction between cluster utilization efficiency and time-to-solution. Inference services require sustained baseline allocations with rapid elastic scale-out to absorb request spikes under strict tail-latency SLOs.

Hardware utilization patterns Provisioning accelerator resources is only the first half of the problem; using them efficiently requires interpreting hardware metrics correctly rather than taking them at face value. Standard GPU utilization metrics can mislead operators because reported percentages reflect temporal occupancy rather than active execution. Tools such as `nvidia-smi` report the fraction of time during which at least one warp was active on any streaming multiprocessor (SM). A kernel stalled for hundreds of clock cycles waiting on HBM fetches or host-to-device PCIe transfers registers as 100 percent active, masking severe memory-bandwidth saturation or pipeline starvation behind an ostensibly healthy utilization figure.

Distinguishing compute execution from memory or pipeline stalls requires evaluating SM execution efficiency alongside memory bandwidth and I/O saturation. Table 14.16 distinguishes these utilization signatures and identifies the corresponding optimization strategy for each:

Table 14.16: GPU Utilization Patterns: Different utilization signatures require different optimizations. High GPU utilization with low memory bandwidth suggests compute-bound workloads that benefit from parallelism. High memory bandwidth with moderate GPU utilization indicates memory-bound workloads requiring model optimization.

Pattern	GPU Util	Memory bandwidth util.	Optimization Strategy
Compute-bound	>85%	<70%	Larger batch sizes, tensor parallelism within node
Memory-bound	50–85%	>85%	Reduce model size, quantize, optimize memory access
I/O-bound	<50%	<50%	Improve data pipeline, prefetch inputs, use SSDs
Batch-starved	Variable (spiky)	Variable	Dynamic batching, request queuing on single server

Utilization targets by workload Representative utilization targets vary by workload characteristics, reflecting contrasting latency tolerances and cost sensitivities across operational modes. For batch training, operators target sustained GPU utilization exceeding 80 percent; lower figures typically indicate data prefetch bottlenecks, uncoalesced memory access, or undersized microbatches. In contrast, **online inference** targets 50–70 percent utilization at median load, deliberately provisioning 30–50 percent headroom to absorb unpredicted traffic bursts without violating tail-latency SLOs; running inference hardware near capacity cascades queuing delays into p99 timeouts. For **batch inference**, where queries tolerate substantial queuing and offline scheduling, targets push above 85 percent utilization to maximize throughput and amortize accelerator cost.

These figures serve as diagnostic starting points rather than universal thresholds: an identical utilization reading signals healthy headroom in an online serving pool but unacceptable idle capacity in a batch pipeline.

Memory hierarchy effects Model serving performance depends critically on GPU memory hierarchy utilization. Data must flow through multiple memory levels with vastly different bandwidths (section D.3.3 maps the full latency hierarchy across the storage spectrum), as table 14.17 quantifies. L2 is a hardware-managed cache for a small active working set, not a placement tier for selected weights. Full model parameters normally reside in HBM; larger models may offload parameters or state to host memory or storage, with transfer latency often dominating inference. Section D.1 tabulates the current accelerator specifications and HBM bandwidths these serving numbers draw on, so the capacities and interface bandwidths in table 14.17 trace back to documented per-generation figures, while the on-die L2 cache bandwidth is an approximate value that vendors do not publish directly:

Table 14.17: GPU Memory Hierarchy and Bandwidth: Each level trades capacity for speed. L2 caches a small active working set, while full model parameters normally reside in HBM. Host-memory or storage offload can support models that exceed GPU memory, but transfer latency may dominate inference time.

Memory Level	Bandwidth	Typical Contents
L2 Cache (40 MB on A100)	~3 TB/s	Cached working set
HBM2e GPU Memory (80 GB)	~2 TB/s	Model
PCIe Gen4 x16 to CPU	~32 GB/s	Activations
System RAM (512 GB)	~200 GB/s	Batched inputs
NVMe SSD	~7 GB/s	Model swap

For large language model (LLM) serving on a single GPU or server, the KV-cache (storing attention keys and values for each token) often becomes the memory bottleneck; vLLM’s PagedAttention design was motivated by this serving pressure (Kwon et al. 2023). For a Llama 2 70-billion-parameter-style grouped-query attention model (Touvron, Martin, et al. 2023) with 80 layers, 8 KV heads, a 4,096-token context, and FP16 cache entries, each active sequence stores about 1.3 GB of KV cache. Eight concurrent sequences therefore consume about 10.7 GB before scheduler headroom, fragmentation, or activations, limiting how many requests a single node can batch together. Monitoring KV-cache utilization on each serving node enables capacity planning. Near the memory limit, the scheduler may queue or reject requests, reduce batch size, or shorten the admissible context, each with a different latency or service-quality cost.

Cost-per-inference tracking Let Hourly GPU cost be the billed cost of the serving GPU for one hour and Inferences per hour its sustained throughput over the same interval. Equation 14.10 converts those hardware metrics into business-relevant cost-per-inference:

$$\text{Cost per 1K inferences} = \frac{\text{Hourly GPU cost} \times 1000}{\text{Inferences per hour}} \quad (14.10)$$

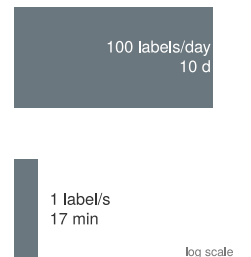
For an illustrative GPU at \$3/hour processing 50,000 inferences/hour, cost is \$0.06/1K inferences. Track this metric over time; changes may reflect pricing, workload mix, or efficiency.

14.5.2.2 Model and infrastructure monitoring

Infrastructure management provisions resources; monitoring observes their behavior. Unit tests can verify deterministic components but cannot by themselves establish population-level predictive performance, which must be estimated statistically. Monitoring implements observable degradation (section 14.2.1), transforming this theoretical limitation into operational practice. Once monitoring surfaces a symptom—a latency service level agreement (SLA) miss, throughput below target, or memory creep—section A.5.1 maps that symptom to its dominant D·A·M term and tells the operator which optimizations will move the binding constraint and which will be wasted on serving infrastructure. Without continuous monitoring and the deeper **observability** it enables (the ability to infer internal state from outputs), a deployed model is a black box slowly drifting toward irrelevance.

Effective monitoring spans both infrastructure health and statistical model behavior. On the infrastructure side, metrics track hardware saturation, memory allocation, and latency distributions. On the model side, teams track evaluation metrics such as accuracy, precision, recall, and confusion matrix shifts (scikit-learn developers 2024b) across live prediction streams.

A fundamental constraint in statistical monitoring is the **drift detection delay**: how quickly a statistical test can confirm that predictive quality has degraded. The speed of confirmation is governed by sample arrival rates and statistical power requirements. Low-traffic services may wait days or weeks to accumulate enough labeled outcomes to distinguish genuine performance erosion from stochastic noise. This latency gap is not an engineering deficiency that tooling can eliminate; it is a direct consequence of finite sample rates colliding with hypothesis testing power. Monitoring architectures must therefore decouple input distribution shifts (detectable immediately from unlabeled inference payloads) from decision boundary shifts (detectable only after ground-truth labels arrive).



Drift detection speed is bounded by the sample rate.

Napkin Math 14.4: The drift detection delay

Problem: A 95 percent-accurate model may fall by 5 percentage-point. The monitoring policy budgets 1,000 labeled examples; the exact requirement depends on the test, power, label noise, and dependence. How long does collection take?

Math:

1. Evidence budget: 1,000 labeled examples.
2. High-rate case: At 1 labeled outcome per second, collection takes 1,000 seconds $\approx$ 16.7 minutes.
3. Low-rate case: At 100 labeled outcomes per day, collection takes 10 days.

Systems insight: Detection is bounded by labeled-outcome arrival, which may lag far behind traffic. Low-volume or delayed-label domains may need days or weeks, so high-stakes systems supplement statistical monitoring with proactive model audits.

When telemetry signals degradation, diagnosing the root cause and executing a rollback requires rigorous **lineage tracking** within a model registry. Where practical, each registered model version must link immutable digests for the training code, dataset snapshot, hyperparameters, and serialized weight artifacts. While this chain enables reproducible rollbacks to known-good checkpoints, it cannot guarantee bitwise reproducibility if driver versions, CUDA libraries, or non-deterministic reduction kernels vary across the cluster fleet.

Degradation itself arises along two distinct axes of distribution shift²³ that monitoring systems must decouple: Concept drift occurs when the relationship between features and targets shifts ($p(y | x)$ changes), altering the decision boundary even if input features appear normal. Data drift²⁴ refers to a shift in the input feature distribution ($p(x)$ changes), such as seasonal lighting shifts or changing sensor characteristics in autonomous driving.

Both forms of drift motivate a formal definition:

Definition 14.3: Data drift

Data drift is a change in the input distribution $p(x)$. Covariate shift is the special case in which $p(x)$ changes while $p(y | x)$ remains stable. The broader drift taxonomy from section 4.5.3 also includes concept drift, in which $p(y | x)$ changes; both dimensions can occur together.

1. **Significance:** It violates the identical-distribution assumption and can cause accuracy to erode as the distributional divergence ($\mathcal{D}(P_t \| P_0)$) grows. Under covariate shift and support coverage assumptions, importance weighting or retraining with current labeled data may recover performance.
2. **Distinction:** Model decay describes observed quality decline over time; the model code need not change. Data drift is one possible external cause, alongside concept drift and changes in the surrounding pipeline.
3. **Common pitfall:** Monitoring model outputs alone may miss input drift or confuse benign output changes with quality loss. Input feature statistics ($\mathcal{D}(P_t \| P_0)$ via PSI, KS tests, or Wasserstein distance/Earth Mover's Distance (EMD)) can provide earlier warning than labeled performance because ground-truth feedback is often delayed.

Because of drift, a deployed model behaves like *decaying inventory* rather than *static software*. **Statistical drift risk** formalizes this hazard: operators model accuracy decline through empirical divergence,

$$\text{Accuracy}(t) \approx \text{Accuracy}_0 - \lambda \cdot \mathcal{D}(P_t \| P_0)$$

where λ is fitted per deployment from historical degradation data. While not a universal law, tracking distributional divergence $\mathcal{D}(P_t \| P_0)$ provides an operational leading indicator of degradation before accuracy losses compound into business impact.

23 | **Drift detection delay:**

A change in $p(x)$ can be estimated from sufficient input samples without labels, although detection is not immediate. A change in $p(y | x)$ generally requires labeled outcomes, which in high-stakes domains (medical diagnosis, fraud detection, legal decisions) can take days, weeks, or months. Proxy metrics such as prediction-confidence distributions and output entropy can provide imperfect early warnings that trade false-alarm rate for detection speed.

24 | **Covariate shift:**

Importance-weighting corrections for covariate shift assume that the support of the training distribution covers the deployment distribution: every deployment input could have appeared in training, just with different probability. When deployment contains genuinely out-of-distribution inputs (new product categories, new demographics, adversarial inputs), the correction can fail and the model can produce confidently wrong outputs with no warning signal, making support coverage the hidden assumption that determines whether drift correction or full retraining may be required.

The Rotting Asset Curve (figure 14.8) illustrates this operational trade-off by contrasting two maintenance policies. The scheduled policy (orange sawtooth) retrains at fixed 90-day intervals regardless of actual model health, alternately wasting compute when performance remains stable and serving degraded predictions when drift accelerates ahead of schedule. In contrast, the triggered policy (green line) initiates retraining only when monitored quality crosses the drift threshold, maintaining an accuracy floor while avoiding unneeded training jobs.

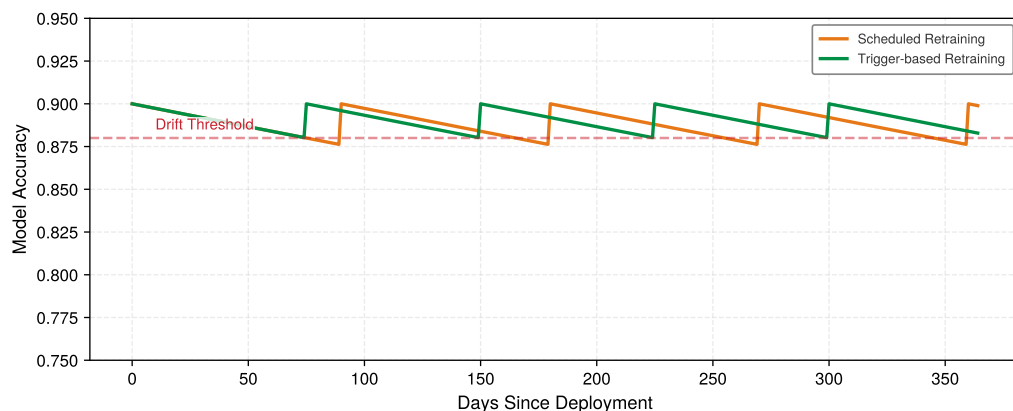


Figure 14.8: The Rotting Asset Curve: An illustrative exponential accuracy decay under two maintenance policies, both drawn as sawtooths. The scheduled policy resets every 90 days regardless of accuracy and dips below the drift threshold before each reset, while the triggered policy resets only on crossing that threshold and so holds a higher floor. Neither cadence is universal.

The two curves turn drift from an abstract statistical problem into an operations policy choice. Scheduled retraining is easy to plan but can retrain too early or too late; trigger-based retraining requires stronger telemetry but aligns intervention with observed degradation.

14.5.3 Layered monitoring and drift quantification

Distribution divergence can accompany accuracy decay, but determining degradation's direction and magnitude requires ground-truth labels that often arrive with substantial delay. In production, serving failures span two fundamentally different physical regimes: immediate hardware execution bottlenecks and gradual distributional drift. Disentangling them requires two distinct telemetry layers: infrastructure metrics that reveal whether the serving system itself is compute-, memory-, or I/O-bound, and distribution and outcome metrics that connect data movement to model performance.

The first layer is infrastructure-level monitoring, which tracks indicators such as CPU and GPU utilization, memory bandwidth, disk consumption, network latency, and service availability. Measuring GPU utilization alone is incomplete; an accelerator reporting high execution time may still be stalled waiting for DRAM or PCIe transfers, while low utilization often signals host-side preprocessing bottlenecks. Correlating kernel execution times with memory bandwidth utilization and input pipeline queue depths distinguishes compute-, memory-, and I/O-bound behavior.

Systems Perspective 14.2: Iron law in production monitoring

These utilization patterns map directly to the iron law of ML systems (section 1.7). Monitoring reveals which term dominates:

- **Compute-bound** (high GPU utilization, low memory bandwidth utilization): Limited by $O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$. Optimize kernels, use Tensor Cores, or upgrade hardware.
- **Memory-bound** (moderate GPU utilization, high memory bandwidth utilization): Limited by D_{vol}/BW . Optimize with quantization, pruning, or batching.

- **I/O-bound** (low GPU utilization, low memory bandwidth utilization): Limited by data pipeline latency. Fix the DataLoader, not the model.

The iron law doubles as a diagnostic framework for production systems. When latency SLOs are violated, the monitoring dashboard indicates which term to investigate.

Power-efficiency metrics (such as inferences per joule or FLOP/s/W, depending on workload) add a cost-normalized view that enables mixed-workload scheduling. Thermal monitoring similarly informs operational scheduling, particularly under sustained high load where thermal throttling forces hardware clocks down, introducing tail-latency spikes that violate SLOs. An MLOps telemetry pipeline incorporates thermal headroom metrics to steer traffic across available accelerators before throttling occurs. Time-series engines such as Prometheus²⁵ (Cloud Native Computing Foundation 2024b), Grafana (Labs 2024), and Elastic (Elastic NV 2024) collect and visualize these operational metrics across the serving fleet.

Collecting these signals at production scale introduces severe network and storage constraints: capturing uncompressed telemetry per request can saturate the very networking fabric serving the model. These overheads force a deliberate trade-off between monitoring granularity and infrastructure operating expense.

 Napkin Math 14.5: The economics of observability

Trade-off: “Measure everything” is physically impossible at scale. Let N_{series} be the expanded series count, f_{sample} the sampling frequency per series, B_{sample} the bytes per sample, and $T_{\text{retention}}$ the retention duration; $C_{\text{ingest/sample}}$ and $C_{\text{storage/byte-time}}$ are provider-specific ingestion and storage prices. Equation 14.11 combines these terms into an operating cost rate:

$$\text{Cost rate} \approx N_{\text{series}} f_{\text{sample}} \left(C_{\text{ingest/sample}} + B_{\text{sample}} T_{\text{retention}} C_{\text{storage/byte-time}} \right) \tag{14.11}$$

The product is an operating cost rate, not a one-time setup cost. Both data volumes follow from the same two-step arithmetic. At full fidelity, every request’s telemetry enters the stream: 1M req/s $\times$ 1 KB per request = 1 GB/s. Retaining one in 60 request traces reduces average ingest to 16.7 MB/s; batching sampled traces every 60 s changes delivery timing, not the average byte rate. Table 14.18 places the two regimes side by side with their cost impact.

Table 14.18: Trace Sampling vs. Observability Cost: Retaining one in 60 request traces yields a 60 $\times$ data-volume difference at 1M req/s, assuming 1 KB per request. Batch intervals do not cause the reduction.

Sampling	Granularity	Data Volume (1M req/s)	Cost Impact
All traces	Micro-bursts	~1 GB/s	High (Requires dedicated cluster)
1-in-60 traces	Trends	~16.7 MB/s	Lower (Policy-dependent)

Systems insight: Retain 1 percent of successful requests and 100 percent of errors. Monitor aggregate counters every 1 s and high-cardinality sketches every 60 s; section 14.5.3.1 shows how to budget the infrastructure.

Alerting mechanisms convert these operational signals into proactive intervention before silent degradation propagates downstream. A sustained drop in model accuracy triggers drift investigation; infrastructure alerts signal memory saturation, thermal throttling, or degraded network throughput. The latency of these alerts governs the window of undetected failure. Alerts must record the model version, affected input subpopulation, and triggering aggregation window so responders can isolate and reproduce the failure state.

²⁵ Prometheus periodically scrapes targets and stores time series for aggregation. Scrape and rule-evaluation intervals jointly affect alert latency and may miss short excursions. Monitoring per-accelerator thermals allows precise workload routing at higher data cost, while server-level aggregates can mask component-level throttling.

Example 14.3: Recommendation monitoring at scale

Scenario: Consider a high-throughput streaming recommendation service whose latency remained within its SLO while recommendation quality declined as user behavior changed.

Diagnosis: Traditional infrastructure metrics (CPU utilization, HTTP error rates) remained green while model CTR dropped because global metrics masked localized subpopulation degradation.

Systems lesson: Infrastructure telemetry alone cannot detect statistical model failure. High-throughput recommendation systems require cohort-level subpopulation tracking and counterfactual evaluation to detect localized accuracy drift.

14.5.3.1 Data quality monitoring

Infrastructure telemetry tracks system availability, but models fail silently when fed corrupted or shifted data. By the time downstream business metrics degrade, upstream data corruption may have persisted for days or weeks. Data quality monitoring exposes these input defects before inference occurs, while output and outcome monitoring confirm whether distributional shifts impair prediction accuracy. The first input guardrail is executable validation: listing 14.4 illustrates schema expectations that reject malformed batches before inference, turning implicit data assumptions into enforceable software contracts.

Listing 14.4: Input Data Validation: Schema validation rules check column existence, data types, null values, and statistical bounds to catch data quality issues before they propagate to model inference.

```
schema.require_column("user_id")
schema.require_type("timestamp", "datetime")
schema.require_non_null("feature_a")

schema.require_range("age", min_value=0, max_value=120)
schema.require_mean_between(
    "purchase_amount", min_value=10, max_value=1000
)
```

Input data validation Schema validation²⁶ catches structural defects at ingest time. These checks enforce column existence, strict data typing, null-value limits, and numerical boundary invariants.

Feature distribution monitoring Schema validation catches structural corruption (missing columns, wrong types, null values) but cannot detect the subtler failure mode where data arrives in the correct format but from a shifted distribution. A feature representing user age can pass every schema check while its mean silently migrates from 32 to 45 over three months as marketing attracts an older demographic. This distributional shift degrades model accuracy long before any structural anomaly appears.

Gradual long-term degradation is particularly insidious because it evades coarse detection thresholds: small day-to-day changes in a quality metric compound into material degradation over a year without tripping monthly alerts. Seasonal patterns compound this complexity; a model trained in summer may perform well through autumn but fail in winter conditions it never observed. Detecting such gradual degradation requires multi-timescale monitoring: performance baselines across multiple time horizons (daily, weekly, quarterly), sliding window comparisons that detect slow trends, and seasonal profiles that account for cyclical variation.

Statistical distance measures quantify this divergence by comparing serving distributions against training baselines. Table 14.19 specifies representative alert thresholds for three common metrics, with population stability index (PSI) suited for categorical and binned features, Kolmogorov-Smirnov (KS) statistics for continuous distributions, and Jensen-Shannon divergence for symmetric, bounded comparison of probability distributions.

²⁶ | **Schema validation:** The rules in listing 14.4 prevent silent data contract violations, such as a feature column changing from an integer to a float. Without this input-level guardrail, downstream model monitoring cannot distinguish a data quality error from a true performance regression, masking the root cause. A schema mismatch in a critical feature can invalidate an otherwise well-formed prediction batch.

Table 14.19: Feature Distribution Thresholds: Starting points for drift detection, calibrated in practice to each feature’s sensitivity and business impact. PSI thresholds such as 0.1 and 0.25 are common scorecard-monitoring conventions, while KS and JS thresholds must be calibrated to the feature, sample size, and cost of missed drift. Higher thresholds reduce alert fatigue but risk missing gradual drift.

Metric	Alert Threshold	Use Case
PSI	PSI > 0.25	Categorical and binned features
Kolmogorov-Smirnov statistic	KS > 0.1	Continuous feature distributions
Jensen-Shannon divergence	JS > 0.1	Probability distributions

27 | PSI (population stability index): PSI is widely used in credit-risk scorecard monitoring to compare expected and observed binned populations; Yurdakul and Naranjo analyze its statistical properties (Yurdakul and Naranjo 2020). The common 0.1 and 0.25 bands are useful operational conventions, not universal statistical laws. ML operations adopted PSI because it works on binned categorical or continuous features and provides an interpretable drift score that non-specialists can review.

The PSI²⁷ quantifies distributional shift by comparing expected (training) and actual (serving) frequencies across fixed bins (section B.2.2 develops the mathematical foundations of KL divergence, PSI, and information theory for systems monitoring). Here n is the number of fixed bins, and actual_i and expected_i are aligned, strictly positive bin proportions that sum to one in each distribution. Equation 14.12 formalizes the metric:

$$\text{PSI} = \sum_{i=1}^n (\text{actual}_i - \text{expected}_i) \times \ln \left(\frac{\text{actual}_i}{\text{expected}_i} \right) \quad (14.12)$$

Production monitors require a documented zero-bin policy, such as Laplace smoothing, because any empty bin in the denominator yields an undefined logarithm. Monitors must also preserve fixed bin boundaries across comparison periods; PSI is a drift signal, not an automatic retraining trigger.

For discrete or binned distributions, KL divergence provides another comparison. Let p be the monitored serving distribution and q the training reference distribution. Equation 14.13 defines the local KL-specific notation $\mathcal{D}_{\text{KL}}$; elsewhere in this book, $\mathcal{D}(P_t \| P_0)$ denotes a generic statistical divergence in the degradation equation:

$$\mathcal{D}_{\text{KL}}(p \| q) = \sum_x p(x) \log \left(\frac{p(x)}{q(x)} \right) \quad (14.13)$$

Because KL divergence is asymmetric and becomes infinite when $p(x) > 0$ where $q(x) = 0$, operational monitors must preserve direction and apply an explicit support policy, such as smoothing previously unseen bins. Algebraically, PSI is identical to the symmetric Kullback-Leibler divergence (also known as Jeffreys divergence), expressing $\mathcal{D}_{\text{KL}}(\text{actual} \| \text{expected}) + \mathcal{D}_{\text{KL}}(\text{expected} \| \text{actual})$, which guarantees that probability shifts in either direction yield positive drift contributions. Similarly, Jensen-Shannon divergence avoids KL divergence’s infinite explosion on non-overlapping distributions by evaluating divergence against the average distribution $M = \frac{1}{2}(p + q)$, yielding a smooth, symmetric distance bounded strictly between 0 and 1. In practice, demographic migration can appear subtle on an uncalibrated histogram but generates a pronounced drift metric. In a recommendation system monitoring user age, a shift toward older cohorts produces the bin-by-bin PSI decomposition shown in table 14.20:

Table 14.20: PSI Worked Example: User age distribution drift from training to serving, decomposed across six age bins. The total PSI is 0.029, well below the 0.1 warning threshold, even though several bins shifted by 3 percentage points. Aggregate PSI summarizes movement across bins; action depends on a calibrated threshold, sample size, and business cost.

Age Bin	Training	Serving	Difference	ln(Serving/Training)	Contribution
18–25	15%	12%	-0.03	-0.223	0.0067
26–35	25%	22%	-0.03	-0.128	0.0038
36–45	20%	18%	-0.02	-0.105	0.0021
46–55	18%	20%	+0.02	+0.105	0.0021
56–65	12%	15%	+0.03	+0.223	0.0067
66+	10%	13%	+0.03	+0.262	0.0079

Summing the six bin contributions yields a total PSI of 0.029 (Stable). The training and serving columns are shown as percentages, while the difference column is expressed in proportion units,

so a 3 percentage-point shift appears as ± 0.03 . Even though specific bins shifted by 3 percentage points, the aggregate drift is well below the 0.1 warning threshold. Operational action depends on a calibrated threshold, sample size, and business cost.

Data freshness monitoring Feature stores and data pipelines can become stale without triggering obvious errors. Data freshness monitoring catches that failure mode, and listing 14.5 shows a configuration that monitors feature freshness and triggers fallback behavior when data becomes stale.

Listing 14.5: Data Freshness Alert Configuration: This configuration monitors the `user_purchase_history` feature for staleness, alerting operations teams via PagerDuty and Slack and falling back to default values when the feature exceeds the maximum allowed age.

```
# Example freshness alert configuration
feature: user_purchase_history
max_staleness: 6h
alert_channels: [pagerduty, slack]
on_stale:
  action: fallback_to_default
  default_value: []
```

A freshness policy turns staleness from a silent data defect into an explicit fallback; the same detect-and-respond contract must cover every layer of the monitoring stack.

Checkpoint 14.2: The monitoring stack

ML monitoring is layered, not monolithic. The prompts in this checklist test whether each symptom can be traced to the responsible layer.

- ☐ **Infrastructure utilization:** can you distinguish a 0 percent GPU utilization failure from a 100 percent queuing failure?
- ☐ **Traffic throughput:** can you tell whether a sudden QPS drop points to upstream routing failure or client-side demand change?
- ☐ **Data freshness:** can you detect stale features before plausible-looking predictions reflect yesterday's world?
- ☐ **Data drift:** can you use PSI or KL divergence to decide whether production data still resembles the training distribution?
- ☐ **Model accuracy:** can you account for delayed ground truth labels when interpreting accuracy alerts?
- ☐ **Cohort bias:** can you use cohort analysis to catch subgroup failures that aggregate accuracy hides?

The same stack must monitor upstream dependencies feeding the ML system: database replication lag, feature-store ingestion watermarks, and pipeline completion status. In one representative incident, a recommendation service detected a material shift in its `user_lifetime_value` feature within two days, tracing the anomaly to a database migration that silently altered an aggregation window. Without input-layer distribution monitoring, such upstream corruption degrades model predictions for weeks before ground-truth outcome metrics expose the failure.

Monitoring cost model Observability infrastructure incurs costs that scale with monitoring granularity. Understanding these costs enables rational decisions about monitoring depth vs. budget constraints.

Cost components Monitoring costs break down into four categories, as equation 14.14 decomposes:

$$\text{Monitoring Cost} = C_{\text{ingest}} + C_{\text{storage}} + C_{\text{compute}} + C_{\text{alert}} \quad (14.14)$$

The four C_* terms are cost components over the same accounting window: data ingestion, retained storage, query or dashboard compute, and alert-rule evaluation. Separating them matters because each scales with a different control knob.

Table 14.21 provides representative unit-cost assumptions for each component. Translating these unit costs into a concrete budget estimate clarifies the real expense of monitoring even a single production model:

Table 14.21: Monitoring Cost Components: Illustrative scenario unit costs used for the worked example. Costs scale differently across components: metric ingestion scales with cardinality (number of unique metric series), storage scales with retention, and query costs scale with dashboard usage patterns.

Component	Illustrative Unit Cost	Scaling Factor
Metric Ingestion	\$0.10–0.50 per million data points	Number of metrics $\times$ sample rate
Log Storage	\$0.50–2.00 per GB/month	Log verbosity $\times$ retention period
Query Compute	\$0.01–0.05 per query	Dashboard refresh rate $\times$ users
Alert Evaluation	\$0.001–0.01 per evaluation	Number of alert rules $\times$ check frequency



Napkin Math 14.6: Single-model monitoring budget

Problem: What monthly ingestion, storage, and query cost follows from monitoring a single ML node under these assumptions?

Variables:

- one model with 3 deployment variants (production, canary, staging), each emitting 50 metrics
- Metrics sampled every 15 seconds
- Retention requirement: 30 days
- 2 dashboards (model health, infrastructure), 3 team members, five-minute refresh

Metric ingestion:

- Data points per month: $3 \times 50 \times (4 \text{ samples/min} \times 60 \times 24 \times 30 \text{ days}) = 25.9\text{M}$
- Cost at \$0.30/million: \$7.8/month

Storage:

- At 8 bytes/point compressed: $25.9\text{M} \times 8 \text{ bytes} = 0.2 \text{ GB}$
- Cost at \$1/GB: \$0.21/month

Query compute:

- Queries per month: $2 \text{ dashboards} \times 3 \text{ users} \times (12 \text{ queries/hour} \times 8 \text{ hours/day} \times 22 \text{ days}) = 12,672 \text{ queries/month}$
- Cost at \$0.02/query: \$253.4/month

Total: ~\$261.4/month for a single ML node. Alert evaluation and platform overhead are excluded.

Systems insight: Under these assumptions, cost scales linearly with identical nodes; at platform scale, query-cost optimization becomes increasingly important.

Cost optimization strategies Each cost component in equation 14.14 maps to a distinct architectural mitigation. The primary driver of C_{ingest} and C_{storage} is metric cardinality: high-cardinality labels such as `user_id` or `request_id` cause combinatorial state explosion in time-series indexes, dwarfing raw compute costs. Dropping or hashing high-cardinality dimensions before ingestion yields immediate storage and ingestion savings. The second driver of storage cost is temporal resolution: retaining raw 15-second metrics across 30 days is rarely justified. A tiered retention policy

(high-resolution telemetry for recent incident debugging, downsampled rollups for historical trend analysis) preserves diagnostic fidelity while slashing retained volume. For C_{compute} , dashboard query overhead accumulates through auto-refresh loops across dozens of unattended user sessions; setting conservative refresh intervals and auto-pausing inactive sessions bounds query load. Finally, C_{alert} scales with evaluation frequency: consolidating overlapping threshold checks into composite multi-condition rules minimizes rule-engine overhead while eliminating alert fatigue.

Cost-benefit framework Justify monitoring investments against incident costs using the monitoring benefit-cost ratio in equation 14.15:

$$\text{Monitoring Benefit/Cost} = \frac{\text{Incidents Prevented} \times \text{Avg Incident Cost}}{\text{Annual Monitoring Cost}} \quad (14.15)$$

If average incident costs \$50,000 (downtime + engineering time + reputation) and monitoring prevents 5 incidents annually at \$50,000 monitoring cost:

$$\text{Benefit/Cost} = \frac{5 \times \$50,000}{\$50,000} = 5 \times$$

This framework helps justify monitoring investments and prioritize which metrics deserve fine-grained observation vs. coarse sampling. The monitoring systems themselves require resilience planning to prevent operational blind spots. When primary monitoring infrastructure fails (Prometheus experiencing downtime or Grafana becoming unavailable), teams risk operating blind during critical periods. Production-grade MLOps implementations therefore maintain redundant monitoring pathways: secondary metric collectors that activate during primary system failures, local logging that persists when centralized systems fail, and heartbeat checks that detect monitoring system outages.

Robust production deployments implement cross-monitoring where external watchdog infrastructure monitors the telemetry pipelines themselves, ensuring that observation failures trigger immediate alerts through alternative notification pathways. This defense-in-depth architecture prevents the silent failure scenario where both models and their telemetry systems fail simultaneously. A circuit breaker²⁸ adds a further safeguard, automatically routing traffic away from a failing service when its error rate exceeds a threshold. Coordinating these safeguards across many replicated services, with consensus-based alerting and cross-region metric aggregation, is a fleet-scale concern that arises once a model is replicated across regions, beyond the single-node scope here.

14.5.4 Incident response and operational practices

Monitoring and drift detection identify anomalies; incident response, debugging, and structured on-call rotations translate those statistical signals into timely engineering action, forming the operational feedback loop that sustains production reliability.

14.5.4.1 Incident response for ML systems

At 2:00 AM, an on-call engineer receives an alert: recommendation click-through rate has dropped 12 percent over the past hour. There is no stack trace, no error log, no crashed process, just a statistical signal that something has changed. The responder must distinguish among four candidate root causes: an upstream data-pipeline failure, model drift, a seasonal traffic pattern, or statistical noise. This ambiguity is common in ML incidents: symptoms may appear as degradation in outcome or predictive-quality metrics rather than explicit errors, so incident response must account for statistical uncertainty.

Severity classification provides the foundation for prioritizing response in this ambiguous landscape. Table 14.22 defines four priority levels with associated response times, from P0 complete failures requiring 15-minute response to P3 minor anomalies allowing 24-hour investigation.

Once severity is assigned, the incident response process follows a structured checklist whose order narrows the search at each step:

1. Detection determines which monitoring signal triggered the alert.
2. Impact assessment quantifies what percentage of traffic is affected.

²⁸ | **Circuit breaker pattern:** Automatic failure detection that “opens” when error rates exceed configured thresholds, routing traffic away from failing services. In ML systems, the pattern requires a critical adaptation: prediction accuracy degradation demands different thresholds than service availability failures, because a model returning plausible but incorrect predictions triggers no error signal, leaving the circuit breaker blind to the most dangerous failure mode.

3. Responders review recent changes to identify whether any models, features, or data pipelines were deployed.
4. Mitigation options are evaluated, including rollback, fallback enablement, or traffic reduction.
5. Root cause analysis determines whether the issue stems from the model, data, or infrastructure.

Table 14.22: Incident Severity Classification for ML Systems: Response times reflect the urgency and potential business impact of each severity level.

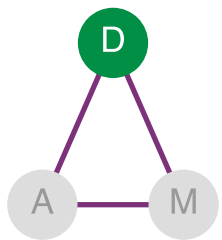
Level	Criteria	Response Time	Example
P0	Complete model failure, serving errors	15 minutes	Model returns null predictions
P1	Significant accuracy degradation (>10%)	1 hour	Recommendation CTR drops 15%
P2	Moderate drift, localized impact	4 hours	One feature shows PSI > 0.3
P3	Minor anomalies, no user impact	24 hours	Training pipeline delay

For P0 and P1 incidents, postmortem documentation is required. These postmortems must include timeline, root cause, user impact, and preventive measures. ML-specific elements include identifying which monitoring gap allowed the issue to reach production and what validation would have caught it earlier.

14.5.4.2 Model debugging: From detection to diagnosis

Incident response triages and mitigates; model debugging identifies root causes. Monitoring detects that something is wrong; debugging determines *why*. ML debugging must account for probabilistic and data-dependent failures in addition to conventional software defects. An incorrect prediction does not throw an exception or generate a stack trace, making systematic debugging essential for resolving ML incidents efficiently.

The debugging decision tree When model performance degrades, work through these diagnostic questions in order. Section A.8 supplies a systematic diagnostic matrix that maps symptoms to D·A·M (Data · Algorithm · Machine) axes.



Production debugging starts on the data axis of D·A·M.

1. **Is it the data?** Check for upstream data pipeline failures, schema changes, missing values, or distribution shifts. Data is often the first place to look because many production ML failures originate in changing inputs, labels, or feature pipelines.
2. **Is it training-serving skew?** Compare feature values or preprocessing outputs for matched examples across training and serving. KS or PSI can screen for distributional divergence, but cannot establish its cause.
3. **Is it a specific subpopulation?** Slice performance by key dimensions (geography, device type, user segment). Degradation localized to one slice suggests a data coverage or labeling issue.
4. **Is it temporal?** Plot performance over time. Sudden drops prioritize checks for recent deployments and data failures; gradual decline can be consistent with drift, but timing alone does not establish the cause.
5. **Is it the model?** After checking data and pipeline causes, examine model behavior through prediction analysis and feature attribution.

Slice analysis This sequence makes slice analysis the first deepening step once a global degradation has been detected, because it tests whether the apparent system-wide problem is actually concentrated in a subpopulation. Performance metrics aggregated across all traffic can mask significant problems in subpopulations: **Slice analysis** exposes that masking, and table 14.23 illustrates how overall accuracy can hide severe degradation in specific segments.

Table 14.23: Slice Analysis Example: Overall accuracy of 91 percent appears acceptable, but tablet users (5 percent of traffic) experience 62 percent accuracy, a severe degradation masked by aggregation. Effective debugging requires systematic slice analysis across key dimensions.

User Segment	Traffic %	Accuracy	Impact
Desktop users	45%	94%	Nominal
Mobile (iOS)	30%	92%	Nominal
Mobile (Android)	20%	88%	Minor degradation
Tablet users	5%	62%	Severe—investigate
Overall	100%	91%	Masks tablet problem

Feature attribution for debugging When slice analysis identifies a problematic segment, feature attribution techniques help identify which features the model relies on within incorrect predictions. Listing 14.6 demonstrates a workflow that uses SHAP values, feature-attribution scores that estimate how much each input feature contributed to an individual prediction, to analyze mispredictions within a specific slice.

Listing 14.6: SHAP-Based Debugging Workflow: This code filters mispredicted tablet examples, computes SHAP values for their model outputs, and plots the selected examples' feature attributions.

```
# SHAP-based debugging workflow
import shap

# Select mispredicted examples from problematic slice
errors = predictions[
    (predictions.actual != predictions.predicted)
    & (predictions.device_type == "tablet")
]

# Compute SHAP values for error cases
explainer = shap.Explainer(model)
shap_values = explainer(errors[feature_columns])

# Plot feature attributions for the selected errors
shap.summary_plot(shap_values, errors[feature_columns])
```

The attribution plot identifies features the model relied on within the failing slice; it does not establish whether those features caused the errors. Diagnosing stale features, missing coverage, or semantic shift requires checking feature lineage and production distributions.



Systems Perspective 14.3: Zombie features

Long-lived production models often accumulate features whose original owners, semantics, or intended use have faded. A feature can be deprecated in application code while still flowing through a feature store, training dataset, or serialized model input contract. The model may learn to ignore it, split signal across a duplicate, or depend on a preprocessing artifact that nobody still owns. Removing such a feature is therefore no longer a local cleanup: it becomes a compatibility change that can affect retraining, serving, monitoring, and downstream analysis. Features in ML systems do not disappear just because code owners stop thinking about them. Unused feature payload volume D_{vol} inflates serialization overhead and consumes memory bandwidth BW without improving accuracy. Without explicit deprecation policies and feature-store governance, models accumulate “dead code” that degrades D_{vol}/BW and complicates debugging (Sculley et al. 2015).

Zombie features show that attribution can reveal what a model should no longer depend on. For individual mispredictions, counterfactual analysis adds the complementary view: the minimal change that would flip a single decision.

Counterfactual analysis For individual mispredictions, counterfactual analysis identifies the minimal change that would flip the prediction: if `session_duration` were 45 seconds instead of 12 seconds, the model would predict “engaged” instead of “churned.” This reveals which feature boundaries drive decisions and whether those boundaries make semantic sense. Counterfactuals that require implausible changes (“user age would need to be -5 years”) often indicate feature engineering problems.

These techniques (decision trees, slice analysis, feature attribution, and counterfactuals) form a debugging toolkit. To apply them consistently, teams codify the process.

Debugging checklist Systematic debugging follows a six-phase checklist: reproduce, isolate, bisect, attribute, validate, and prevent. The ordering is deliberate because each phase narrows the search space for the next. Reproduction comes first because an ML failure that cannot be reproduced on held-out data is often data-dependent, an insight that redirects investigation toward the D-A-M taxonomy’s data layer. Once reproduced, isolation identifies the minimal input set that triggers the failure, transforming a diffuse “the model is wrong” complaint into a specific, testable condition.

Bisection then exploits version history: if the failure correlates with a recent deployment, comparing model versions pinpoints which change introduced the regression. Feature attribution applies these interpretability techniques to identify which input factors drive the erroneous behavior. Validation closes the causal loop by confirming that the hypothesized root cause, when corrected, actually resolves the failure, distinguishing genuine fixes from coincidental improvements.

The final phase, prevention, converts each resolved incident into a monitoring rule or validation check, systematically closing the gap between detection and recurrence. This cumulative hardening can reduce repeated failure modes over time because each incident strengthens the observability infrastructure.

Debugging ML systems requires both systematic methodology and domain expertise. The most effective debugging often comes from engineers who understand both the model architecture and the business context of the predictions.

14.5.4.3 On-call practices for ML systems

The debugging techniques in section 14.5.4.2 work when an engineer is actively investigating an issue during business hours. Production systems, however, fail at 3:00 AM on weekends, and the person responding may not be the one who built the model. Debugging resolves individual incidents; on-call practices sustain operational health over time by ensuring that someone with appropriate expertise is always available and equipped to respond. On-call rotation for ML systems requires specialized practices beyond traditional software operations because ML incidents often manifest as gradual degradation rather than hard failures. A traditional software engineer responding to an alert can typically trace a stack trace to a root cause within minutes. An ML engineer facing a 3 percent accuracy drop must first determine whether the change represents statistical noise, legitimate concept drift, or a critical failure requiring immediate rollback. This distinction demands statistical context rather than simple log analysis.

This ambiguity compounds with delayed impact visibility. Unlike latency spikes that surface immediately in dashboards, ML degradation may take hours or days to manifest in business metrics. A recommendation model that began serving slightly worse suggestions on Monday might not produce measurable revenue impact until Friday, by which time the window for easy diagnosis has closed. Cross-system dependencies further complicate response: ML issues often originate in upstream data systems owned by different teams, requiring coordination across organizational boundaries during incident response. The deepest challenge is that effective response demands understanding model behavior, not infrastructure health alone. A database administrator can restart a crashed service without understanding its business logic, but an ML engineer cannot meaningfully debug accuracy degradation without understanding the model’s feature dependencies and expected behavior patterns.

These challenges motivate tiered escalation structures that match expertise to incident complexity. Table 14.24 illustrates one possible structure, where primary responders handle routine issues using standardized runbooks while escalation paths connect to specialists capable of deeper investigation. A parallel data on-call role can reduce time to resolution when the root cause lies in an upstream data system.

Table 14.24: ML On-Call Structure: Tiered escalation with parallel data on-call enables efficient incident response. Tier 1 handles routine issues using runbooks; Tier 2 addresses complex debugging; Tier 3 manages critical incidents requiring architectural decisions.

Tier	Responder	Responsibility
Tier 1 (Primary)	ML Engineer	Initial triage, standard runbooks, escalation decisions
Tier 2 (Escalation)	Senior ML Engineer/Data Scientist	Complex debugging, cross-system investigation, model-specific issues
Tier 3 (Critical)	ML Platform Lead	Architecture decisions, major incidents, vendor escalation
Data On-Call (Parallel)	Data Engineer	Data pipeline issues, feature store problems, upstream dependencies

Effective on-call depends heavily on runbook quality. Every production ML model should have documentation covering the model's purpose, ownership, and business criticality alongside its normal operating parameters: expected latency, throughput, and accuracy ranges that define healthy behavior. Historical incidents and their resolutions provide templates for common failure patterns, while diagnostic commands enable rapid health assessment: how to check recent predictions, feature distributions, and model confidence scores. Critically, runbooks must specify escalation criteria (when to wake up Tier 2 vs. when to rollback without approval) and rollback procedures with step-by-step instructions and expected recovery times. Runbooks written during calm periods save critical minutes during 3:00 AM incidents.

Even well-designed monitoring can generate excessive alerts that erode on-call effectiveness. Alert fatigue, the tendency to ignore or dismiss alerts after experiencing too many false positives, represents a significant operational risk. Teams combat fatigue through consolidation, grouping related alerts so that multiple features drifting simultaneously generate a single notification rather than dozens. Adaptive thresholds that account for weekly and seasonal patterns prevent predictable variations from triggering unnecessary pages. Alerts that are rarely actionable should be retired or recalibrated. When temporary silencing is necessary, accountability mechanisms (requiring a follow-up ticket before snoozing) prevent alerts from being permanently ignored.

Shift handoffs represent another critical practice that distinguishes mature operations. Incoming on-call engineers need context about active incidents and their current status, recent deployments that might cause delayed issues, upcoming scheduled changes such as data migrations or model updates, and any alerts that were suppressed along with the reasoning. Without structured handoffs, context is lost between shifts, and incoming engineers waste time rediscovering information their predecessors already gathered.

Sustainable on-call practices must also address burnout. ML on-call carries particular stress due to incident ambiguity: the uncertainty of not knowing whether an alert represents a real problem demands constant vigilance. Organizations mitigate burnout by limiting consecutive on-call days, providing compensatory time off after high-severity incidents, conducting regular rotation reviews to balance load across team members, and investing in automation that reduces toil. The goal is to make on-call rotations sustainable over years of operation, not to staff them as an afterthought.

Technical monitoring capabilities alone do not ensure operational success. The most sophisticated dashboards fail if no one is responsible for acting on alerts, and the most detailed runbooks languish if team structures do not support their use. Production ML operations require organizational infrastructure paralleling the technical: clear governance, defined roles, and communication patterns that enable cross-functional coordination.

14.5.5 Governance and team coordination

On-call runbooks address real-time operational failures, but production ML systems also require proactive governance to enforce behavioral and regulatory invariants. Model governance establishes the verification criteria that models must satisfy before receiving live inference traffic. Unchecked models risk producing biased inferences or uninterpretable decisions that violate statutory regulations and organizational policies. Within an operational pipeline, governance enforces three concrete objectives: transparency through auditable training data lineage and verifiable model

weights, fairness through bounded performance parity across protected user subgroups, and compliance through adherence to privacy, retention, and auditability mandates. While chapter 15 details the statistical metrics and interpretability algorithms required for these properties, MLOps provides the automated verification infrastructure to enforce them across the deployment lifecycle.

ML governance spans the full lifecycle of the artifact. During development, automated pipelines record feature schemas, training data provenance, and hyperparameter configurations. Prior to deployment, prerelease validation gates evaluate out-of-distribution robustness and demographic parity. Once in production, monitoring systems must track fairness drift—temporal shifts in error rates across demographic or tenant subgroups—alongside aggregate accuracy degradation. Encoding these governance policies directly into automated CI/CD pipelines replaces ad hoc manual review with programmatic release assertions.

Concretely, a model registry promotion gate requires a signed feature contract, a recorded training-data lineage hash, subgroup metrics above policy thresholds, a canary SLO with automated rollback criteria, and a designated artifact owner before a model transitions from staging to production. That gate turns governance from an organizational review meeting into a release invariant enforced by the same CI/CD machinery that deploys the model.

Governance policies define the release criteria, but operational reliability depends on execution across organizational boundaries. Because machine learning systems span data engineering, statistical modeling, and distributed serving infrastructure, role handoffs represent the most failure-prone interfaces in the operational lifecycle. Versioned experiment stores and centralized model registries provide the shared substrate required for reproducible transitions between research and production engineering. At the data interface, formal schema specifications and lineage DAGs ensure that data scientists and platform engineers share identical assumptions regarding feature semantics, default null handling, and label arrival latency.

Titles and boundaries vary across organizations, but one common decomposition uses the five roles in table 14.25. The table maps each role to a primary responsibility without implying that every organization needs five separate positions:

Table 14.25: ML Team Roles Matrix: Clear role boundaries prevent gaps and overlaps. Data Scientists focus on model quality while ML Engineers handle productionization. Data Engineers own data pipelines while Platform Engineers own MLOps tooling. SREs ensure overall system reliability.

Role	Primary Focus	Key Deliverables	Collaboration Points
Data Scientist	Model development, experimentation, algorithm selection	Trained models, experiment results, performance benchmarks	Hands off to ML Engineer for productionization
ML Engineer	Production ML systems, training pipelines, serving infrastructure	Deployed models, training pipelines, serving systems	Receives from Data Scientist; works with Platform Engineer on infrastructure
Data Engineer	Data pipelines, feature engineering, data quality	Feature pipelines, data quality systems, feature stores	Provides data to Data Scientist; maintains feature store for ML Engineer
Platform Engineer	MLOps infrastructure, tooling, automation	CI/CD pipelines, monitoring systems, compute infrastructure	Enables ML Engineer; maintains shared infrastructure
De-vOps/SRE	Reliability, incident response, system health	SLOs/SLAs, on-call procedures, runbooks	Supports all roles; owns production health

Clear role definitions matter most at handoff points, where artifacts transition between specialists. The most failure-prone transition occurs between Data Scientists and ML Engineers: an exploratory notebook script frequently fails in production environments due to undocumented data preprocessing steps, hardcoded local file paths, hidden state execution orders, or unpinned library dependencies. Similarly, the handoff from ML Engineers to SREs (Beyer et al. 2016) requires verified monitoring dashboards, active alerting rules, documented runbooks, and tested rollback procedures. Data Engineers interface with model builders through feature contracts—formal specifications of data schema, freshness SLOs, and distribution invariants that prevent upstream pipeline modifications from surfacing as silent downstream prediction failures weeks later. Teams eliminate

these interface risks by requiring standardized container images, hermetic build manifests, and automated reproducibility assertions before any artifact advances to the next stage.

14.5.5.1 Stakeholder communication

While cross-functional handoffs govern internal engineering execution, stakeholder communication establishes the operational contract between technical teams and organizational decision-makers. Unlike deterministic software services with static failure semantics, machine learning systems exhibit probabilistic outputs, complex latency-accuracy trade-offs, and continuous distribution drift. When stakeholders evaluate models using traditional software assumptions, misalignment inevitably emerges regarding expected reliability, latency budgets, and maintenance costs.

A frequent communication failure is the unconstrained request to “make the model more accurate.” Model accuracy cannot be scaled independently of system costs: boosting validation accuracy by two percentage points often requires moving to a significantly larger model architecture, incurring higher FLOP counts per sample, increasing memory bandwidth demands, and violating strict inference latency SLOs. Grounding improvement discussions in the Pareto frontier forces explicit decisions among model accuracy, inference latency budgets, training compute costs, and dataset acquisition expenses.

Translating statistical metrics into operational impact requires mapping classification errors directly to economic or operational consequences. Conventional evaluation metrics such as accuracy, ROC-AUC, or F1 scores evaluate errors geometrically or symmetrically. In production, however, classification errors incur highly asymmetric costs: in an automated fraud detection pipeline, a false positive creates user friction during checkout, whereas a false negative allows direct, unrecoverable monetary theft.

Figure 14.9 illustrates this asymmetric economic reality. When false negatives cost five times more than false positives, the optimal decision boundary shifts substantially leftward from the symmetric 0.50 classification threshold, accepting a higher rate of false alarms to minimize catastrophic missed detections.

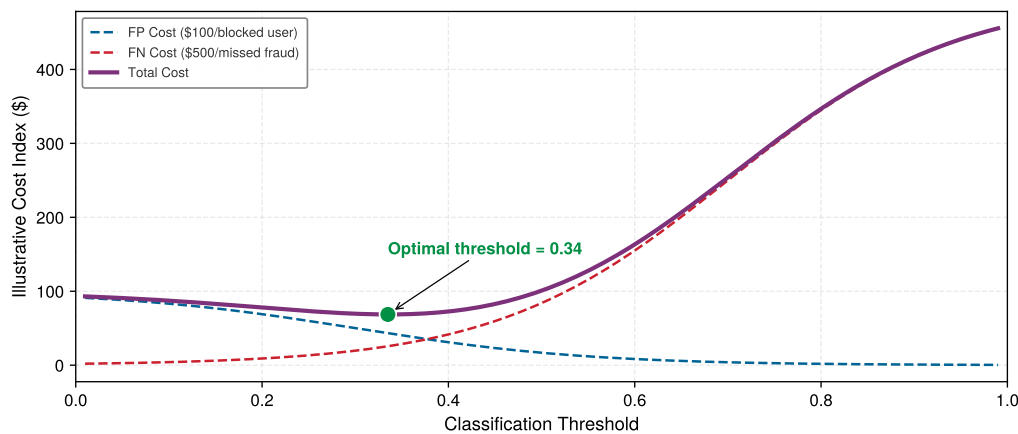


Figure 14.9: The Business Cost Curve: An equal-class-weighted illustrative cost index vs. classification threshold. Technical metrics like ROC curves hide the economic reality: errors have different costs. Here, a false negative (missed fraud) costs \$500, while a false positive (blocked user) costs \$100. The system flags a transaction as fraud when its score exceeds the threshold, so a lower threshold flags more transactions. Because missing fraud is $5\times$ more costly, the optimal threshold shifts left of center, making the system more aggressive at flagging suspicious transactions and accepting more false positives to minimize expensive misses. With equal costs the optimum would sit at threshold = 0.50; the asymmetry pulls it toward threshold ≈ 0.34 . A population expected cost would also weight each term by class prevalence. MLOps tunes thresholds as costs and prevalence change.

Operational incidents require structured communication protocols that convey technical severity without ambiguity. When monitoring systems detect distribution drift or an inference regression triggers an automated rollback, engineering status reports must state the empirical evidence directly: the statistical divergence metric, the specific demographic or tenant cohort affected, the canary rollback target, and the estimated recovery timeline. Dismissing anomalous output distributions as

transient noise without statistical hypothesis testing allows silent semantic failures to propagate into production decisions.

Justifying hardware resource allocations similarly requires connecting hardware capabilities to development velocity and serving capacity. Requesting additional accelerator nodes must be framed in terms of throughput scaling and iteration speed: provisioning an 8-GPU node enables distributed data-parallel training runs to complete in hours rather than days, directly accelerating model iteration and validation cycles. Furthermore, engineering schedules must reflect real production bottlenecks: data extraction, feature engineering, and pipeline validation typically consume far more calendar time than hyperparameter optimization.

Consider a fraud detection team evaluating a proposed model upgrade. When stakeholders request improved capture rates, the engineering team frames the initiative as a concrete systems trade-off: increasing the fraction of fraud dollars captured from 92 percent to 94 percent requires integrating external feature pipelines, extending training duration by two weeks, and accepting 30 percent higher infrastructure costs. In return, the upgrade prevents an estimated \$2 million in annual fraud losses while, under a separate 20 percent reduction assumption, eliminating 50,000 false-positive alerts per month.

Framing model improvements around financial impact, latency budgets, and compute expenditure grounds operational governance in verifiable engineering trade-offs. By coupling automated CI/CD release invariants with structured communication of error costs, engineering teams ensure that production deployments remain both technically robust and economically sound.

14.5.6 ML test score

A release-readiness assessment requires an explicit catalog of the structural debt patterns that make a model unsafe to serve even when offline validation metrics appear acceptable. Machine learning systems fail in production not only from algorithmic degradation, but from silent interface decay, untracked consumers, and unversioned pipeline dependencies. Table 14.26 categorizes these primary debt patterns alongside their root causes, failure symptoms, and mitigation strategies, establishing the failure modes that a production readiness rubric must detect and prevent.

Table 14.26: Technical Debt Patterns: Machine learning systems accumulate distinct forms of technical debt from data dependencies, model interactions, and evolving operational contexts. Primary debt patterns, their causes, symptoms, and recommended mitigation strategies guide practitioners in recognizing and addressing these challenges systematically.

Debt Pattern	Primary Cause	Key Symptoms	Mitigation Strategies
Boundary Erosion	Tightly coupled components, unclear interfaces	Changes cascade unpredictably, CACE principle violations	Enforce modular interfaces, design for encapsulation
Correction Cascades	Sequential model dependencies, inherited assumptions	Upstream fixes break downstream systems, escalating revisions	Careful reuse vs. redesign trade-offs, clear versioning
Undeclared Consumers	Informal output sharing, untracked dependencies	Silent breakage from model updates, hidden feedback loops	Strict access controls, formal interface contracts, usage monitoring
Data Dependency Debt	Unstable or underutilized data inputs	Model failures from data changes, brittle feature pipelines	Data versioning, lineage tracking, leave-one-out analysis
Feedback Loops	Model outputs influence future training data	Self-reinforcing behavior, hidden performance degradation	Cohort-based monitoring, canary deployments, architectural isolation
Pipeline Debt	Ad hoc workflows, lack of standard interfaces	Fragile execution, duplication, maintenance burden	Modular design, workflow orchestration tools, shared libraries
Configuration Debt	Fragmented settings, poor versioning	Irreproducible results, silent failures, tuning opacity	Version control, validation, structured formats, automation
Prototype Debt	Rapid prototyping shortcuts, tight code-logic coupling	Inflexibility as systems scale, difficult team collaboration	Flexible foundations, intentional debt tracking, planned refactoring

Qualitative awareness of failure modes does not prevent operational outages; deployment gates require a formal technical debt assessment rubric that converts subjective readiness debates into

quantifiable engineering constraints. The ML Test Score (Breck et al. 2017) structures this evaluation across four operational categories: data tests, model tests, infrastructure tests, and monitoring tests. The rubric specifies 28 binary or graded tests—seven per category—scoring each from 0 (untested) to 1 (automated and monitored). Rather than averaging these evaluations into a single compensatory aggregate, the composite score follows the weakest-link principle:

$$\text{ML Test Score} = \min(S_{\text{data}}, S_{\text{model}}, S_{\text{infra}}, S_{\text{mon}})$$

where each category score $S_{\text{cat}} \in [0, 7]$. A deployment with exhaustive offline model validation ($S_{\text{model}} = 7$) but absent production telemetry ($S_{\text{mon}} = 0$) receives an overall score of 0. This min-operator reflects the reality of machine learning systems: an unmonitored serving path silently delivers corrupted predictions regardless of offline benchmark accuracy.

The evaluation partitions production risk across the Data, Algorithm, and Machine layers of the system. The data suite verifies that feature distributions conform to declared schemas, privacy boundaries remain unviolated, and each feature justifies its inference latency and memory footprint. The model suite confirms that architectures and hyperparameter configurations are version-controlled, staleness bounds are enforced, and offline benchmark improvements translate into live metric gains. The infrastructure suite tests deterministic training reproducibility across hardware seeds, serving-path parity between training and inference pipelines, and automated rollback mechanisms. The monitoring suite verifies that schema changes, invariant violations, training-serving skew, and stale weights generate actionable alerts before degraded predictions impact downstream systems. Table 14.27 summarizes representative tests across each category:

Table 14.27: ML Test Score Checklist: Representative production-readiness tests from the ML Test Score rubric. The original rubric contains 28 tests grouped into four sections of seven tests each; section scores expose whether production risk comes from data validation, model validation, infrastructure, or monitoring rather than hiding the weakness in a single total. Based on Breck et al. (2017).

Category	Test	Implementation Example
Data Tests	Feature expectations are captured in schema	Great Expectations, TFX Data Validation
	All features are beneficial (no unused features)	Feature importance analysis, ablation studies
	No feature's cost exceeds its benefit	Latency/accuracy trade-off analysis
	Data pipeline has appropriate privacy controls	PII detection, access logging
Model Tests	Model spec is reviewed and checked into version control	Git-tracked model configs
	Offline and online metrics are correlated	A/B test validation of offline improvements
	All hyperparameters are tuned	Automated HPO with tracked results
	Model staleness is measured and bounded	Performance decay monitoring
Infrastructure Tests	Training is reproducible	Fixed seeds, versioned data, locked dependencies
	Model can be rolled back to previous version	Model registry with versioning
	Training and serving code paths are tested for consistency	Feature store integration tests
	Model quality is validated before serving	Automated validation gates in CI/CD
Monitoring Tests	Dependency changes result in alerts	Data schema monitoring
	Data invariants hold in training and serving	Distribution comparison tests
	Training and serving features are not skewed	Training-serving skew detection
	Model staleness triggers retraining	Automated retraining pipelines

Because the composite score is bounded by the minimum category, reliability audits must prioritize the bottleneck subsystem limiting the system score. Investing engineering effort to expand a model test suite from 5 to 6 points while monitoring coverage remains at 1 yields zero gain in overall production safety. Regular audits against this rubric systematically reveal whether reliability risks stem from unvalidated input pipelines, offline-online distribution divergence, or uninstrumented serving paths, directing refactoring effort toward the structural debt that directly threatens production availability.

14.6 Design and Maturity Framework

An organization deploying its initial ML model might rely on a hand-run Jupyter notebook, a scheduled cron job, and minimal monitoring. A mature enterprise runs thousands of models

through automated pipelines with drift detection, canary deployments, and continuous validation. Both are doing “MLOps,” yet the gap between them spans orders of magnitude in reliability, cost efficiency, and deployment velocity. Empirical surveys of production deployments reveal that systems failures cluster around data dependencies, pipeline boundaries, and monitoring blind spots rather than core model architectures (Paleyes et al. 2022). Operational maturity provides the systems framework for evaluating this progression: organizations evolve from ad hoc experimentation toward fully automated operations, and identifying where a system stands on this continuum governs resource allocation more effectively than adopting tooling indiscriminately.

14.6.1 Maturity levels

While the ML Test Score assesses individual technical practices, **operational maturity** measures their systemic integration. Lifecycle tools such as MLflow address isolated pipeline stages (Zaharia et al. 2018), but maturity reflects how cohesively data, algorithms, and execution hardware interface across training and serving. Although operational maturity exists on a continuum, categorizing systems into distinct maturity levels clarifies how workflows transition from exploratory prototypes to production-grade infrastructure.

At the lowest level, workflows are ad hoc: training runs manually on local workstations, feature extraction is embedded in ephemeral scripts, and deployment relies on manual file transfers. As maturity increases, workflows become repeatable: teams introduce version-controlled pipelines, centralized artifact registries, and scheduled batch jobs. At the highest level, systems become scalable: declarative infrastructure-as-code, continuous validation pipelines, and real-time observability decouple model iteration from production serving.

Table 14.28 summarizes this architectural progression from disconnected execution toward unified lifecycle management.

Table 14.28: Maturity Progression: Machine learning operational practices evolve from manual, fragile workflows toward fully integrated, automated systems, impacting reproducibility and scalability. Key characteristics and outcomes at each maturity level emphasize architectural cohesion and lifecycle integration for building maintainable learning systems.

Maturity Level	System Characteristics	Typical Outcomes
Ad Hoc	Manual data processing, local training, no version control, unclear ownership	Fragile workflows, difficult to reproduce or debug
Repeatable	Automated training pipelines, basic CI/CD, centralized model storage, some monitoring	Improved reproducibility, limited scalability
Scalable	Fully automated workflows, integrated observability, infrastructure-as-code, governance	High reliability, rapid iteration, production-grade ML

Consider how a fraud detection system evolves across these maturity levels:

- **Ad hoc:** A data scientist trains a model in a Jupyter notebook, exports it as a pickle file, and hands it to an engineer who deploys it to a single server. When accuracy drops, the data scientist retrains manually by running the notebook again with fresh data. Debugging requires the original data scientist because no one else understands the preprocessing steps.
- **Repeatable:** The training script is version-controlled, with a scheduled Jenkins job that retrains monthly. Features are computed in a SQL script that engineering maintains separately. The model is deployed via container, with basic accuracy monitoring. When the feature SQL changes, the data scientist must manually verify the model still works.
- **Scalable:** Training and serving use the same feature store, reducing skew. A CI/CD pipeline investigates drift when PSI exceeds 0.2, retrains when evidence supports it, validates the new model against the baseline, and deploys via canary release. Monitoring tracks per-merchant accuracy, triggering investigation when specific segments degrade. The entire lineage from raw data to production prediction is auditable.

Transitioning between maturity levels requires substantial engineering investment, but the resulting reduction in mean time to recovery (MTTR) and silent failures justifies the cost for mission-critical

services. Evaluating an organization against this spectrum isolates specific architectural bottlenecks—such as manual data transfers or unversioned preprocessing logic—before they compound into production outages.

14.6.2 System design implications

Organizational maturity dictates concrete architectural constraints. As teams progress across levels, the underlying system shifts from monolithic scripts to modular, decoupled services.

In low-maturity environments, systems are tightly coupled: data transformations are embedded directly in model training scripts, configurations are managed informally, and serving depends on serialized artifacts with implicit environment dependencies. While such designs facilitate rapid exploratory prototyping, they lack clear failure boundaries. At higher maturity, explicit abstractions decouple the D-A-M components: feature pipelines compute transformations independently through versioned schemas, models expose contract-driven inference APIs, and serving runtimes execute in isolated container environments with reproducible hardware execution targets. Closed feedback loops allow data streams, algorithmic updates, and compute resources to adapt without manual intervention.

Figure 14.10 illustrates the structural dependency stack governing production ML systems. Visible service uptime represents only the uppermost tier of operational health. Beneath the waterline lie silent failure modes that degrade inference quality without triggering traditional infrastructure alerts: covariate shift, upstream schema corruption, and localized segment degradation. A robust operational framework must monitor all three domains—data health, model health, and service health—in parallel.

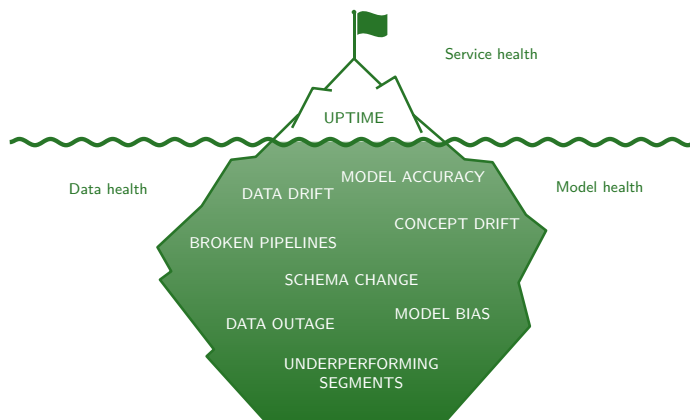


Figure 14.10: Uptime Dependency Stack: The waterline separates visible service uptime from data- and model-health failures that can remain hidden while the service stays available; the surrounding labels organize the monitoring surface into data, model, and service health.

The three monitoring domains map directly to distinct systems failure mechanisms:

- *Data health* failures—such as schema drift, null-value spikes, and feature staleness—violate the distributional invariants assumed during training without altering the serving binaries.
- *Model health* failures—such as accuracy loss, score calibration drift, and degenerate feedback loops—occur silently because the inference engine continues returning syntactically valid tensors with low latency.
- *Service health* failures—such as memory fragmentation, dependency drift, and container orchestration faults—threaten traditional availability and throughput. Because data and model failures leave standard HTTP health checks and process status green, availability monitoring alone fails to capture the true operational state of an ML system.

14.6.3 Design patterns and anti-patterns

The most sophisticated infrastructure fails without the organizational patterns to operate it effectively. A feature store cannot prevent training-serving skew if no one owns the feature definitions;

automated monitoring cannot catch drift if alerts route to the wrong team. As ML systems grow in complexity, organizational patterns must evolve to match.

In mature environments, organizational design emphasizes clear ownership and interface discipline. Platform teams take responsibility for shared infrastructure and CI/CD pipelines, while domain teams focus on model development and business alignment. Interfaces between teams—feature definitions, data schemas, and deployment targets—are formally specified and versioned.

One effective pattern is a centralized MLOps team providing shared services to multiple model development groups. Such structures promote consistency and reduce duplicated effort. Alternatively, some organizations adopt a federated model, embedding MLOps engineers within product teams while maintaining a central architectural function for system-wide integration.

Anti-patterns emerge when responsibilities are fragmented. The tool-first approach (adopting infrastructure tools without first defining processes and roles) results in fragile pipelines and unclear handoffs. Siloed experimentation, where data scientists operate in isolation from production engineers, leads to models that are difficult to deploy or retrain effectively.

Organizational drift presents another challenge. As teams scale, undocumented workflows become entrenched and coordination costs increase. Organizational maturity must co-evolve with system complexity through communication channels, role definitions, and accountability structures that reinforce modularity and observability.

Organizational structures require corresponding technical architectures to maintain system reliability. While conventional distributed systems handle binary hardware faults or network partitions, ML systems introduce stateful dependencies on shifting data distributions and shared accelerator resources.

Conventional circuit breakers trip when RPC timeouts or HTTP 5xx error rates exceed a set threshold. In an ML serving context, circuit breakers must monitor statistical invariants: anomalous input distributions (such as sudden shifts in feature bounds or missing feature rates) or output distribution collapse. When triggered, the circuit breaker bypasses the model to serve predictions from a deterministic heuristic fallback, a cached lookup table, or a simpler baseline model, preventing corrupted inferences from cascading into downstream transactional systems.

Similarly, bulkhead patterns²⁹ isolate candidate or canary models from primary serving paths. On multi-tenant accelerator hardware, an unconstrained experimental model suffering an out-of-memory abort, memory leak, or thread-pool exhaustion can crash the shared serving runtime or stall the PCIe bus. Enforcing bulkheads via containerized memory limits, isolated worker queues, and dedicated hardware partitions ensures that experimental workloads cannot degrade primary production throughput. Ordinary model prediction errors are semantic failures, not Byzantine faults; Byzantine fault tolerance³⁰ provides only a limited analogy for arbitrary component failures.

Consensus algorithms establish agreement among nodes; they cannot determine whether a model prediction is correct when ground truth is delayed or unavailable. Replicating a model across multiple nodes or voting across ensemble members provides redundancy against crash faults, but it cannot resolve silent semantic errors when all replicas share the same inductive biases or training distribution blind spots. These reliability patterns nevertheless help distinguish robust MLOps implementations from fragile ones when their guarantees are applied within scope.

14.6.4 Contextualizing MLOps

Standard operational patterns must adapt to physical and organizational constraints. Every ML system operates within a specific context that shapes implementation: physical constraints (edge compute, power budgets), regulatory requirements (healthcare, finance), or organizational scale (team size, skill distribution). A standard CI/CD pipeline may be infeasible without direct host access; monitoring may require indirect signals or on-device anomaly detection; data collection may be limited by privacy regulations. These adaptations are expressions of maturity under constraint, not departures from foundational principles.

At the highest levels of operational maturity, single-model practices become building blocks for broader platform capabilities. Organizations operating many ML nodes simultaneously consolidate into platform architectures that provide shared infrastructure, centralized governance, and economies of scale. The transition from individual ML nodes to platform-scale infrastructure introduces qualitatively different challenges (cross-model resource allocation, system-level observability,

²⁹ | **Bulkhead pattern:** This pattern partitions system resources to contain failures within isolated zones. In ML serving, a bulkhead allocates fixed accelerator memory, host RAM, and worker thread pools to experimental model versions, preventing an out-of-memory abort or memory leak in a canary from terminating the shared serving process.

³⁰ | **Byzantine fault tolerance:** The classic Byzantine model concerns nodes that may behave arbitrarily or send conflicting messages. In the synchronous unauthenticated, or oral-messages, setting, tolerating f Byzantine faults requires at least $3f + 1$ participants (Lamport et al. 1982). ML ensembles are only an analogy because prediction errors can be correlated, and agreement does not establish semantic correctness.

and fault tolerance across interdependent pipelines) that extend beyond single-node operations. Sound ML node practices remain prerequisites for platform success because gaps in single-model monitoring, testing, or deployment multiply across the model portfolio.

14.6.5 MLOps investment economics

The operational benefits of MLOps become persuasive only when the investment matches the model's production value. For a single ML node, the decision is whether deployment speed, incident reduction, and monitoring coverage justify the operational spend; for a portfolio, the same economics compound into platform investment.

14.6.5.1 Single-model MLOps investment

For a single production ML system, the first threshold is the annual cost of making the node observable, deployable, and recoverable. Table 14.29 summarizes the main cost categories:

Table 14.29: Single-Model MLOps Investment: Illustrative planning inputs for operationalizing one production ML system. Open-source tooling (MLflow, Feast) can reduce software costs; cloud-managed services trade higher unit costs for reduced engineering overhead.

Component	Illustrative Assumption	Justification
CI/CD pipeline setup	\$10–30K one-time	Reduces deployment time from days to hours
Monitoring and alerting	\$2–10K/year	Catches degradation before user impact
Feature store (basic)	\$5–20K/year	Reduces one source of training-serving skew
Model registry	\$0–5K/year	Enables rollback, audit trails
Engineering time	1–2 FTE-months setup	Initial automation and integration

14.6.5.2 Single-model ROI calculation

The return threshold then depends on model criticality: a revenue-facing model can justify more operational spend because avoided incidents and deployment-time savings have measurable value. Equation 14.16 formalizes that single-node calculation:

$$\text{Annual Benefit/Cost} = \frac{\text{Incidents Avoided} \times \text{Avg Incident Cost} + \text{Time Savings} \times \text{Hourly Cost}}{\text{Annual MLOps Investment}} \quad (14.16)$$

where Incidents Avoided is the count of production failures the tooling prevents per year and Avg Incident Cost is the loss per failure, so their product is the value of avoided incidents; Time Savings is the engineer-hours that automation reclaims per year and Hourly Cost is the loaded labor rate, so their product is the value of recovered labor; the denominator is the annual cost of the tooling itself. The ratio expresses every dollar of investment in dollars returned.

For a model generating \$1M annual revenue with:

- 4 incidents/year avoided (at \$25K each) = \$100K saved
- 20 hours/month deployment time saved (at \$150/hr) = \$36K saved
- MLOps investment of \$30K/year

$$\text{Benefit/Cost} = \frac{\$100K + \$36K}{\$30K} = 4.53 \times$$

14.6.5.3 When to invest more

The 4.53× benefit-cost ratio means the investment is not justified by tooling elegance; it is justified because the model is expensive enough that preventing incidents and shortening deployments outweigh the annual platform spend. The returns from single-model MLOps practices compound when teams add additional models. The transition from operating several independent ML nodes to building a centralized platform involves different economics entirely, including shared infrastructure amortization, platform team overhead, and cross-model coordination costs.

For single-model operations, invest in MLOps in proportion to model criticality. A model driving \$10M in annual revenue justifies more operational rigor than an internal analytics model. Monitoring, lineage, and repeatable deployment are often the first investments. Add a feature store when shared feature definitions and online-offline parity become material problems, and automate retraining only when evidence supports the trigger and the validation loop can contain a bad update.

These economic trade-offs and architectural maturity principles manifest across diverse real-world deployments. In section 14.7, production case studies illustrate how organizations balance reproducibility, observability, and automated retraining against concrete latency and throughput constraints.

14.7 Case Studies

Operational constraints in machine learning systems are governed by the physical envelope of the serving hardware and the failure consequences of the application. A battery-powered wearable operates under milliwatt power budgets, constrained on-device SRAM, and narrow telemetry bandwidth over Bluetooth Low Energy. In contrast, clinical software operates under statutory auditability, safety-critical error bounds, and regulatory change controls governed by FDA lifecycle guidelines (U.S. Food and Drug Administration 2025). These disparate environments dictate how engineering teams manage data pipelines, model artifacts, and hardware execution across the lifecycle. Edge architectures spend their engineering budgets bridging the hardware asymmetry between high-throughput cloud training clusters and microcontrollers, whereas clinical systems spend that budget on provenance tracking, cohort monitoring, and human-in-the-loop validation gates.

The Oura-inspired case study in section 14.7.1 models these edge constraints using published of-line evaluation data, serving as an architectural study rather than an audit of proprietary production over-the-air pipelines. In this edge workflow, reference sleep labels derive from polysomnography (PSG)—laboratory-grade multi-channel physiological recordings. Despite their contrasting operating regimes, both edge and clinical deployments must satisfy the same core MLOps disciplines. Table 14.30 maps these foundational principles across the two environments, demonstrating how physical hardware limits and regulatory mandates alter implementation mechanisms without altering the underlying operational requirements.

Table 14.30: MLOps Principles by Case Study: Side-by-side mapping of the five foundational MLOps principles to a hypothetical Oura-inspired edge design and the ClinAIops framework, showing how domain constraints reshape implementation without changing which principles apply.

Principle	Oura-inspired edge design	ClinAIops
Reproducibility	Versioned synchronized wearable and PSG datasets	Audit trails, decision provenance
Separation of concerns	Independent data, training, and serving layers with edge-specific deployment pipeline	Distinct clinical validation and deployment stages with regulatory compliance isolation
Consistency	PSG-aligned preprocessing across training and on-device inference	Standardized clinical data pipelines ensuring training-serving parity
Observable degradation	On-device anomaly detection, limited telemetry	Cohort-specific monitoring, outcome tracking
Cost-aware automation	Battery-aware retraining triggers, CI/CD for edge balancing accuracy and resource cost	Automated model updates with human-in-the-loop gates balancing update cost and patient risk

14.7.1 Oura-inspired edge design

The Oura Ring provides the empirical foundation for this edge MLOps case study. The published clinical study validates data collection and multi-sensor sleep-stage evaluation (Altini and Kinnunen 2021), while the versioning, telemetry, over-the-air deployment, and lifecycle controls represent a hypothetical architecture rather than a reported account of Oura’s proprietary production system. Operating within the milliwatt power envelope and strict memory limits of a wearable ring makes every operational trade-off visible in ways that cloud-scale systems obscure.

14.7.1.1 Context and motivation

As a miniature consumer wearable, the ring captures continuous physiological signals: accelerometry for movement, optical photoplethysmography (PPG) for pulse rate and heart-rate variability, and negative temperature coefficient thermistors for skin temperature. From these raw sensor streams, embedded models classify four sleep stages—wake, light, deep, and rapid eye movement (REM) sleep—to deliver daily recovery scores. Because a miniature 15–22 mAh battery cannot sustain continuous Bluetooth Low Energy (BLE) streaming without depleting within hours, the architecture partitions computation across three tiers: on-ring feature extraction, companion-phone model inference, and cloud-scale historical aggregation.

The central objective was improving sleep-stage classification accuracy to align with polysomnography (PSG),³¹ the clinical gold standard. Accelerometer-only models achieved 57 percent four-stage sleep classification accuracy because movement signals alone cannot separate quiet wakefulness from light or REM sleep. With expert human PSG scorers agreeing with each other at about 82 percent to 83 percent, the performance gap between single-modality motion sensing and clinical consensus exposed the physical limits of accelerometry. Incorporating autonomic nervous system signals (heart rate and heart-rate variability) alongside circadian temperature features lifted classification accuracy to 79 percent. The resulting 22 percentage-point gain closes roughly 84.6–88 percent of the baseline-to-human-agreement gap, although human inter-scorer agreement reflects label ambiguity rather than a strict theoretical ceiling against an adjudicated reference.

Building this multi-sensor pipeline required grounding wearable telemetry in clinical standards through a study involving 106 participants from three continents (Altini and Kinnunen 2021). Each participant wore the Oura Ring while simultaneously undergoing laboratory PSG, yielding 440 nights of data and 3,444 hours of time-synchronized recordings that aligned raw wearable sensor streams with validated sleep annotations. The scale and demographic diversity of the collection captured physiological variation across age, sex, and baseline health, as well as behavioral differences essential for generalizing across a real-world user population.

Consolidating synchronized accelerometer, temperature, optical PPG, and clinical sleep records resolved temporal alignment and feature-engineering specifications. A production edge workflow requires strict dataset versioning and lineage tracking across sensor hardware revisions, sampling rates, and digital filtering routines to prevent training-serving skew between Python development pipelines and embedded C inference.

Once synchronized clinical data was established, offline model exploration quantified the contribution of each additional sensing modality. Evaluating four feature configurations across 5-fold cross-validation confirmed that combining temperature, heart-rate variability, and circadian periodicity lifted four-stage classification accuracy to 79 percent, up from the 57 percent accelerometer-only baseline (Altini and Kinnunen 2021). While these offline gains demonstrate the value of multi-sensor fusion, deploying them on an embedded device introduces operational requirements outside the original study: reproducible model quantization, numerical conversion validation, and strict runtime energy budgets.

Following offline validation, the primary systems challenge shifts from statistical accuracy to deployment safety and runtime efficiency. An edge architecture must partition execution across hardware boundaries: lightweight filtering and movement classification run continuously in microcontroller SRAM, while heavier multi-stage sequence models execute during phone synchronization. To maintain pipeline integrity across thousands of physical devices, the MLOps toolchain requires automated quantization pipelines (such as converting float32 weights to int8 with calibrated scale factors), cryptographically signed model artifacts, and fault-tolerant OTA³² delivery mechanisms.

Edge MLOps operates under physical constraints distinct from server-side pipelines: model accuracy must be co-designed with strict battery life, restricted radio bandwidth, on-device storage bounds, and the near-total absence of real-time ground-truth labels. In the DS-CNN (Tiny Constraint) archetype from table 14.4, health monitoring relies on operational proxies—sensor duty cycles, inference latency anomalies, and classification confidence distributions—rather than verified outcomes. Sustaining the leap from 57 percent accelerometer accuracy to 79 percent multi-sensor accuracy in the field requires rigorous configuration management spanning sensor firmware versions, DSP preprocessing routines, and quantized network weights.

31 | Polysomnography (PSG): A multi-parameter sleep study that provides the clinical ground truth data for this classification task. This “truth” is inherently noisy; expert human scorers interpreting the same PSG recordings agree with each other at about 82 percent–83 percent reliability (Altini and Kinnunen 2021). Inter-scorer agreement indicates label uncertainty but does not impose an accuracy ceiling against a fixed or adjudicated reference.

32 | Over-the-Air (OTA) Updates: The mechanism used to deploy optimized models to devices already in the field without physical access. Minimizing artifact footprint through quantization and pruning directly reduces Bluetooth transfer energy and radio transmission time. Atomic update staging and rollback routines ensure that an interrupted wireless transfer or checksum mismatch cannot corrupt on-device flash or leave the device in an unrecoverable state.

These physical constraints map directly to the five foundational MLOps principles introduced in table 14.30. Versioned wearable and PSG datasets provide reproducibility by linking every deployed model artifact to the exact laboratory recordings and sensor firmware calibrations that trained it. Tiered software architecture enforces separation of concerns, isolating embedded sensor drivers from inference engines and phone-level aggregators so that quantization schemes, model topologies, and battery-saving fallbacks can be updated independently. Shared feature-extraction kernels ensure consistency between offline training and on-device execution, eliminating training-serving skew caused by differing numerical libraries or floating-point rounding. Privacy-preserving telemetry maintains observable degradation by transmitting aggregate operational diagnostics—daily sensor duty cycle, battery drain, inference exception counts, and classification entropy—without streaming raw, sensitive biometrics off the user’s device. Finally, over-the-air deployment defines the boundary for cost-aware automation: every update must demonstrate that its accuracy gain outweighs the battery cost of radio reception, flash rewrite cycles, and the operational risk of altering software on a wearable operating continuously on the user.

The wearable edge illustrates how physical resource limits—milliwatts, kilobytes, and radio bandwidth—dictate MLOps design. When machine learning transitions from personal wellness into regulated healthcare, the primary engineering constraint shifts from hardware physics to patient safety, clinical validation, and algorithmic accountability—the operational domain of ClinAIOps.

14.7.2 ClinAIOps case study

On wearable consumer hardware, the engineering envelope is bounded by milliwatts, memory footprint, and radio bandwidth. When machine learning transitions from fitness tracking to regulated medical care, the governing constraint shifts from hardware physics to patient safety and clinical liability. In continuous therapeutic monitoring (CTM),³³ wearable sensors stream physiological telemetry to guide personalized medical interventions. General-purpose MLOps frameworks provide lifecycle automation—orchestrating pipelines, tracking artifacts, and monitoring availability. However, standard operational pipelines assume that predictions can be evaluated on statistical proxy metrics and deployed autonomously. In clinical environments, automated predictions directly influence human health, making unsupervised actuation unacceptable.

ClinAIOps (Chen et al. 2023) adapts MLOps principles to these constraints by formalizing the boundaries between algorithmic inference and clinical authority. Rather than treating human oversight as an external impediment, ClinAIOps integrates multi-stakeholder feedback loops directly into the serving architecture. The framework partitions operational control across three actors—patients, clinicians, and machine learning engineers—each operating on distinct timescales with explicit safety gates.

14.7.2.1 Feedback loops

Three interlocking feedback loops enable safe, adaptive integration of machine learning into clinical practice. Figure 14.11 maps these loops as a circular flow among three stakeholders. Patients contribute continuous monitoring data from wearable sensors and receive bounded AI-assisted guidance. Clinicians receive AI-generated summaries, alerts, and recommendations, then apply clinical judgment by setting therapy regimens and approval limits. AI developers receive continuous feedback from patients and clinicians, using real-world performance and workflow signals to improve models and deployment processes. The outer loop connecting all three stakeholders represents the full governance cycle.

Each feedback loop operates on a distinct timescale to balance responsiveness against clinical verification. The **patient treatment loop** executes continuously in real time, capturing sensor telemetry and delivering bounded adjustments for patient self-management. At an asynchronous supervisory cadence, the **clinician oversight loop** reviews aggregated trends, sets operational boundaries, and validates treatment recommendations before interventions occur. Finally, the **developer feedback loop** aggregates telemetry and clinician override patterns back to engineering teams, driving offline retraining and interface calibration.

Patient treatment loop The patient treatment loop executes localized inference over high-frequency physiological streams captured by wearable devices, such as continuous glucose monitors or optical

³³ | **Continuous therapeutic monitoring (CTM):** Healthcare approach using wearable sensors for real-time physiological data collection and personalized treatment adjustments. CTM forces MLOps to confront constraints absent in typical deployments: feedback loops must include human-in-the-loop approval for safety-critical decisions, retraining requires clinician-validated labels rather than implicit signals, and model updates must satisfy regulatory compliance before deployment. These constraints reshape every MLOps principle, making CTM a stress test for operational maturity.

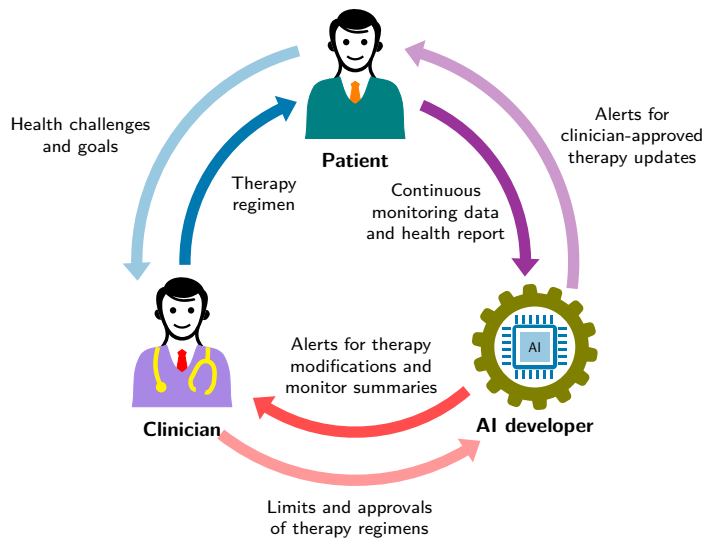


Figure 14.11: ClinAIOps Feedback Loops: The cyclical framework coordinates patients, clinicians, and AI developers to support continuous model improvement and safe clinical integration. Patients and clinicians use AI outputs in care workflows, while AI developers receive feedback from both groups to refine models and operations. Source: (Chen et al. 2023).

pulse sensors. The edge serving runtime evaluates these incoming feature vectors against recent patient history from electronic health records.

To maintain patient safety, recommendations follow a tiered execution model. Minor adjustments that fall strictly within clinician-defined bounds are surfaced directly to the patient. Significant changes—or recommendations generated when model uncertainty exceeds an acceptable margin—are blocked from automated delivery and escalated to the clinician. This structure enables rapid adaptation to dynamic physiological states while ensuring that autonomous actuation never breaches clinician-authorized thresholds.

Clinician oversight loop The clinician oversight loop provides supervisory control over automated inference. Rather than burdening medical staff with continuous raw sensor telemetry, batch aggregation pipelines distill time-series streams into longitudinal summaries, such as circadian variance, multi-day trends, and signal-quality metrics.

When the system generates a therapy recommendation—such as adjusting an antihypertensive dosage for a patient with persistent hypotensive readings—the clinician evaluates the proposal alongside diagnostic factors unobserved by the wearable model. The clinician may approve, modify, or reject the recommendation. This decision is written to an audited event log, serving an operational dual role: it enforces clinical accountability at the point of care and generates verified, expert-labeled supervisory data for subsequent model fine-tuning. Clinicians also update the operational thresholds that govern the patient treatment loop, recalibrating the boundary between automated guidance and mandatory review.

Developer feedback and patient-clinician coordination The developer feedback loop links clinical practice back to engineering infrastructure. By offloading continuous data logging and low-level trend detection to the AI pipeline, patient-clinician consultations shift from routine measurement gathering to targeted clinical evaluation and goal setting.

Operational telemetry generated during these interactions provides engineering teams with direct visibility into production behavior. Overrides, rejection frequencies, and reported adverse events highlight covariate shift in specific demographic slices or sensor degradation across hardware revisions. Engineers use this feedback to trigger targeted retraining pipelines, refine data validation schemas, and eliminate usability failure modes that induce alert fatigue in clinical staff.

14.7.2.2 Hypertension case example

Hypertension management demonstrates how these three operational loops coordinate care in practice. Blood pressure fluctuates continuously under exertion, stress, and circadian cycles, requiring individualized, long-term pharmacological adjustments that benefit from continuous therapeutic monitoring.

Data infrastructure Research systems estimate systolic blood pressure indirectly from ECG, photoplethysmography (PPG),³⁴ pulse-transit time, and heart-rate features (Q. Zhang et al. 2017). In production workflows, these signals are augmented by accelerometer streams to identify motion artifacts and self-reported medication logs. Accuracy requires rigorous calibration and regulatory authorization; consumer-grade sensor claims cannot be assumed clinically reliable without formal prospective validation. When properly validated for the target cohort, this multimodal data stream is merged with electronic health records to supply feature context for inference.

Loop implementation Figure 14.12 shows two of the three feedback loops across the top and patient-clinician coordination below. The upper-left panel illustrates the patient treatment loop, where the patient monitors blood pressure and receives bounded titration recommendations that the AI system can issue within clinician-defined safety thresholds; significant changes require explicit approval. The upper-right panel depicts the clinician oversight loop, where longitudinal trend summaries flow from the AI system to the clinician, and the clinician sets approval limits and receives alerts for clinical risk events such as persistent hypotension or hypertensive crisis. The lower panel captures the patient-clinician coordination that emerges once routine data collection moves to the AI loop: appointments shift to higher-level discussions of lifestyle factors and shared decision-making. The third loop (developer feedback) is not depicted in the figure; it is described in section 14.7.2.1 as the channel by which real-world workflow signals from both patients and clinicians inform model and interface improvements.

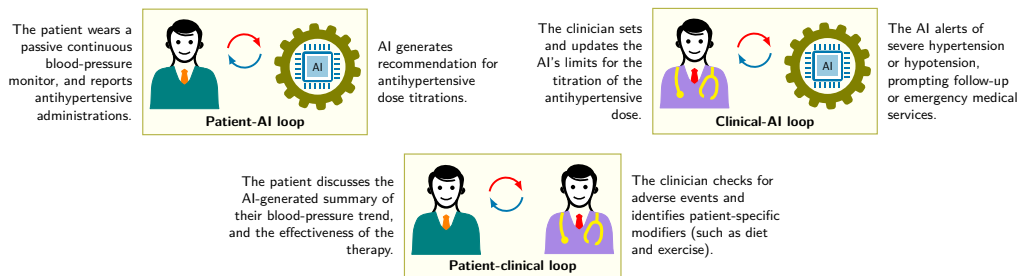


Figure 14.12: Hypertension Management Loops: The three panels, labeled patient-AI, clinical-AI, and patient-clinical, each carry a two-way exchange. The AI issues bounded dose titrations within limits the clinician sets and escalates severe readings, which leaves the patient-clinical panel for trend review, adverse-event checks, and treatment decisions. The developer feedback loop of ClinAIops is not among them and is discussed in the prose. Source: (Chen et al. 2023).

The three panels establish the engineering boundary for clinical automation: routine monitoring and small titrations execute automatically within clinician-defined thresholds, while acute escalations, adverse events, and regimen changes require explicit human authorization. That boundary is the point where ordinary MLOps practices require the additional clinical coordination summarized next.

14.7.2.3 MLOps vs. ClinAIops comparison

The hypertension case illustrates the operational contrasts between standard software pipelines and healthcare deployments. General-purpose MLOps provides technical lifecycle controls—automated testing, continuous integration, model registries, and drift monitoring. ClinAIops extends this foundation into regulated sociotechnical environments where outputs directly influence medical interventions. Table 14.31 compares the two approaches across eight operational dimensions.

Table 14.31: Clinical AI Operations: General-purpose MLOps supplies technical lifecycle controls; ClinAIOps extends them with clinical workflows, evidence, human oversight, and accountability.

	General-purpose MLOps emphasis	ClinAIOps extension
Focus	Technical model lifecycle	Human and AI decision-making
Stakeholders	ML, data, platform, and operations teams	Adds patients, clinicians, and clinical governance
Feedback loops	Monitoring, retraining, and release	Adds treatment, clinician, and developer feedback
Objective	Reliable, governed ML operations	Safe, effective, and accountable care support
Processes	Automated pipelines and operational controls	Integrates clinical workflows and human gates
Data considerations	Lineage, quality, freshness, and access control	Adds consent, clinical provenance, and protected data
Model validation	Predictive, operational, and slice-level metrics	Adds clinical utility, safety, and cohort outcomes
Implementation	Technical integration and operational ownership	Adds clinical accountability and stakeholder incentives

The table's central distinction is that clinical deployment alters risk ownership. Statistical accuracy is a prerequisite, but it does not protect against liability when an erroneous prediction drives a medical intervention. The ClinAIOps framework therefore changes the governing constraint from device efficiency to clinical accountability. The model participates in care delivery, but it cannot own clinical decisions. Every recommendation and subsequent clinician action must be traceable to the underlying sensor inputs, model version, and inference metadata; otherwise the system cannot support regulatory audit or retrospective incident analysis. Separation of concerns shifts from an architectural preference to a safety invariant: sensor ingestion, inference scoring, clinical diagnosis, and parameter updates must be decoupled by explicit software contracts and human approval gates.

The same accountability requirement transforms monitoring. Standardized clinical data pipelines support training-serving parity, but clinical validation requires evaluating model recommendations against standard-of-care outcomes and cohort-specific effects. Observable degradation extends beyond statistical loss and latency to clinical metrics: blood pressure control rates, adverse events, clinician overrides, and subgroup disparities. In this regime, multi-tier feedback loops are not unwanted operational latency; they are deliberate architectural controls that balance automated edge adaptation against legal and ethical accountability. Cost-aware automation operates strictly inside those gates: model refreshes occur only when validated clinical benefits outweigh validation costs and patient risks, and low-confidence predictions trigger deterministic fallback to human clinical review.

14.7.3 Case study synthesis

The Oura-inspired design and ClinAIOps cases separate stable MLOps principles from the deployment constraints that reshape their implementation. The hypothetical ring lifecycle is a resource-envelope case: the operational system must preserve reproducibility, consistency, and observable degradation while battery, telemetry, and weak ground truth limit what can be measured and updated on the device. ClinAIOps is an accountability-envelope case: the same principles apply, but validation, audit trails, and human gates dominate because the model influences clinical action.

The shared engineering lesson is that MLOps maturity is not tool accumulation. It is the ability to identify the governing constraint, choose the operational controls that match it, and preserve evidence when the model changes. Production ML systems fail when teams apply code-focused operational intuitions without accounting for statistical behavior and shifting distributions, exposing common fallacies and operational pitfalls.

14.8 Fallacies and Pitfalls

Operating production machine learning systems under conventional software assumptions creates distinct failure modes. The following fallacies and pitfalls highlight where standard software engineering intuitions break down against the statistical realities of deployed models.

Fallacy: *MLOps is just applying traditional DevOps practices to machine learning models.*

Engineers often assume that standard CI/CD pipelines transfer directly to machine learning. Traditional DevOps automates code compilation, unit testing, and artifact deployment under the invariant that identical source code produces identical binary behavior. In an ML system, code represents only one axis of the D·A·M triad. Model execution depends jointly on code, large evolving datasets, and accelerator compute budgets. As detailed in section 14.4.2.1, ML pipelines introduce stateful retraining, multi-metric validation gates, and immutable artifact registration. Standard CI/CD engines cannot natively track data lineage, orchestrate feature stores, or detect distribution drift. When teams apply DevOps without ML-specific adaptations, they maximize the computational availability of their serving cluster while leaving statistical behavior unmonitored. The system achieves high uptime and low latency, yet silently serves degraded predictions due to unvalidated data shifts and training-serving skew.

Pitfall: *Treating model deployment as a one-time event rather than an ongoing process.*

Treating deployment as a terminal milestone assumes that deployed weights remain permanently aligned with the production input distribution. Unlike compiled binaries whose execution semantics remain fixed, model quality degrades as real-world distributions shift. Monitoring statistical indicators such as PSI (section 14.5.3.1) provides an early signal of feature drift, distinguishing benign variance from actionable distribution change. Crossing a warning threshold requires operational diagnosis rather than unvalidated retraining. As established in section 14.4.2.2, scheduling updates is an economic balancing problem: the optimal retraining interval $T^* \approx \sqrt{\frac{2C}{Q \cdot V \cdot \text{Accuracy}_0 \cdot \gamma}}$ weighs the fixed compute and engineering cost C against the business losses accumulated from query volume Q , unit accuracy value V , and staleness decay rate γ . Operating a production model demands treating deployment as a continuous control loop that balances monitoring signals against retraining economics.

Fallacy: *Automated retraining ensures optimal model performance without human oversight.*

Automating pipeline execution does not eliminate the need for operational governance. Closed-loop retraining pipelines operate without semantic awareness; if upstream data ingestion suffers schema anomalies, corrupted labels, or sudden sensor drift, automated retraining bakes those errors directly into updated model weights. Furthermore, automated validation gates typically evaluate aggregate loss across a held-out test split, which easily conceals severe performance regressions on critical slices or low-frequency edge cases. For instance, retraining a ranking model on weekend traffic can degrade performance across latency-sensitive weekday workloads where user query distributions diverge. Resilient MLOps architectures decouple automated pipeline execution from automated production promotion, enforcing canary deployments, slice-level regression gates, and human sign-off when validation metrics deviate from expected historical envelopes.

Pitfall: *Focusing on technical infrastructure while neglecting organizational and process alignment.*

Investing in MLOps tooling without defining organizational interfaces leads to fragmented operational ownership. Machine learning systems bridge two distinct engineering cultures: model developers optimizing offline accuracy metrics and platform engineers managing host-level latency, memory capacity, and throughput budgets. When organizations treat deployment as a handoff between isolated teams, root-cause triage stalls during production incidents. Infrastructure engineers inspect container health and GPU utilization without insight into model prediction distributions, while model authors inspect offline training curves without understanding runtime serving constraints. Tools such as feature stores and registries provide technical scaffolding, but reliability requires institutional alignment: unified SLOs spanning statistical accuracy and serving latency, joint on-call runbooks, and shared accountability for production regressions.

Fallacy: *Training and serving environments automatically remain consistent once pipelines are established.*

Establishing an initial end-to-end pipeline does not guarantee permanent parity between training and serving. Training-serving skew arises from fundamental architectural divergences between offline and online execution paths. Training typically runs as an offline batch workflow (such as distributed SQL queries or Spark pipelines) optimized for high-throughput scans over historical logs, whereas online serving runs in low-latency microservices under millisecond latency constraints. Maintaining

dual implementations of feature transformations across these environments inevitably introduces numerical disparities, differing missing-value semantics, or lookahead bias in historical joins. As demonstrated in section 14.4.1.2, feature stores alleviate this divergence by unifying transformation logic and synchronizing batch and point-lookup stores. However, software pipelines alone cannot guarantee consistency; operational verification requires continuous distribution parity checks and schema assertions between offline feature logs and online inference inputs to catch divergence before it degrades downstream decisions.

Pitfall: *Assuming comprehensive monitoring prevents all production incidents.*

Dashboards and alert thresholds cannot prevent production outages caused by silent data failures. Monitoring is inherently passive; it detects degradation only after corrupted state propagates through the system. Furthermore, tracking output metrics alone leaves critical blind spots along the data pipeline. As shown in section 14.5.3.1, measuring service latency and prediction distributions fails to detect input anomalies such as feature staleness. An inference service can maintain nominal $p99$ response times and normal prediction ranges while consuming stale user embeddings caused by upstream database replication lag or cache eviction failures. By the time downstream business metrics degrade, corrupted predictions have already impacted users. Production reliability requires defensive boundary controls alongside passive observability: strict input schema enforcement, feature freshness assertions, and circuit breakers that fall back to deterministic heuristics when input quality slips below operational tolerances.

Fallacy: *Accuracy is the first production signal to monitor.*

Teams often configure accuracy dashboards as their primary operational tripwire, assuming degradation will register there first. In practice, ground-truth accuracy is almost always a lagging indicator. In production environments such as fraud detection, ad conversion, or credit underwriting, ground-truth labels arrive with substantial latency—often requiring weeks or months to accumulate through chargebacks or repayment histories. Relying solely on accuracy dashboards leaves the system operating blind throughout this label-delay window. Furthermore, aggregate accuracy can remain deceptively stable even as input distributions diverge across critical sub-populations. As detailed in section 14.5.3.1, monitoring input feature distributions with metrics like PSI or Kullback-Leibler (KL) divergence supplies immediate, zero-label leading indicators. These statistical signals detect distribution shifts in real time, enabling engineering diagnosis before accuracy degradation manifests in business outcomes.

Pitfall: *Routing leading-indicator alerts to a different channel than accuracy alerts.*

Separating leading-indicator alerts from primary incident response channels neutralizes their operational value. When engineering teams route distribution drift and data freshness warnings to informational dashboards or low-priority channels while reserving on-call paging exclusively for downstream accuracy drops, early warnings are ignored. Drift alerts accumulate unnoticed until accuracy collapses and forces an emergency response. A leading indicator provides operational utility only when tied to an active, owned response path with calibrated severity levels and explicit diagnostic runbooks. The operational objective is to use leading statistical signals to trigger root-cause diagnosis—verifying feature pipeline integrity and validating input schemas—ensuring that accuracy metrics serve to confirm operational health rather than announce an unmanaged failure.

Observability in production machine learning extends far beyond passive dashboard telemetry. It requires closed operational loops across the entire D·A·M architecture: enforcing input data integrity and feature stability, bounding model behavior through automated validation gates, and routing leading indicators directly to engineers empowered with diagnostic runbooks before silent statistical degradation damages production systems.

14.9 Summary

MLOps exists because machine learning systems introduce failure modes fundamentally absent from deterministic software. A crashed server or network timeout turns availability dashboards red immediately, alerting operators to take corrective action. A degraded model introduces a silent statistical failure mode: the serving infrastructure returns 200 OK responses within latency deadlines, yet prediction accuracy steadily erodes undetected. Operational practices built solely for server

uptime cannot observe this statistical decay. Machine learning operations closes that gap by treating model quality as a dynamic runtime variable rather than a static deployment artifact.

The five foundational principles (section 14.2.1) provide an evaluation framework that applies regardless of scale or domain. Reproducibility through versioning addresses the root cause of many production incidents: untracked artifacts—including data versions, configuration shifts, and environment drift—that make debugging impossible and rollbacks unreliable. Separation of concerns contains the blast radius when changes occur, preventing the boundary erosion and correction cascades that transform local fixes into system-wide regressions. The consistency imperative targets training-serving skew, the silent accuracy erosion that occurs when feature computation diverges between offline training pipelines and online serving systems; feature stores centralize feature definitions and serving paths, though parity still requires continuous validation. Observable degradation transforms the abstract silent-failure problem into actionable alerts through layered monitoring that tracks data freshness, feature distributions, model outputs, and business metrics. Finally, cost-aware automation replaces arbitrary retraining schedules with a fitted staleness cost model ($T^* \approx \sqrt{\frac{2C}{Q \cdot V \cdot \text{Accuracy}_0 \cdot \gamma}}$) whose practical application depends on its underlying assumptions.

Infrastructure components implement these principles across three critical operational boundaries. Feature stores and data versioning govern the data-model interface by reducing training-serving inconsistency. CI/CD pipelines and model registries govern the model-infrastructure interface by enforcing reproducibility and enabling rollback. Monitoring systems, incident response frameworks, and on-call practices govern the production-monitoring interface by making degradation observable and actionable. The retraining decision framework enables cost-aware automation by connecting measured degradation to economic thresholds. Domain constraints reshape *how* these principles are implemented without changing *which* principles matter: the Oura-inspired design uses the study's 57 percent to 79 percent offline accuracy improvement to motivate resource-aware validation and update controls rather than claiming an unverified production lifecycle. Similarly, ClinAIOps demonstrates why high-stakes clinical deployments require graceful-degradation and human-oversight controls, with three feedback loops functioning as architectural patterns rather than operational overhead.



Key Takeaways: Perfectly available, perfectly wrong

- **ML systems fail silently without outcome monitoring:** Model quality degrades as distributional divergence $\mathcal{D}(P_t \| P_0)$ grows, although divergence alone does not determine accuracy loss. Because a serving node can maintain perfect uptime while predictions fail, tracking ground-truth outcomes is essential rather than relying on server health checks alone.
- **Training-serving skew corrupts inference without crashing:** Feature stores mitigate skew by unifying feature definitions and transformations across environments, but maintaining parity still requires continuous validation across online and offline data pipelines.
- **Retraining is an engineering optimization, not a guess:** The fitted staleness cost model ($T^* \approx \sqrt{2C/(Q \cdot V \cdot \text{Accuracy}_0 \cdot \gamma)}$) balances retraining compute against prediction quality decay, connecting update cadence to quantitative economics under its stated assumptions.
- **Deploy through graduated rollout with pretested rollback:** Canary, blue-green, and shadow deployments partition release risk across production traffic, backed by tiered rollback procedures that must be verified through regular operational drills.
- **Stage infrastructure investments by system risk:** Instrumentation and continuous deployment form the initial operational baseline. High-consequence systems justify greater operational rigor; feature stores and automated retraining should be added as skew and scale demand.
- **The five principles transfer across domains:** Reproducibility, separation of concerns, consistency, observable degradation, and cost-aware automation remain invariants across

applications; domain constraints reshape only their concrete mechanisms and tool implementations.

- **Operational maturity is organizational as well as architectural:** Managing a single model differs qualitatively from governing a production fleet. While the core principles scale, operational complexity grows with fleet size, making unified incentives and shared on-call rotations as essential as tooling.

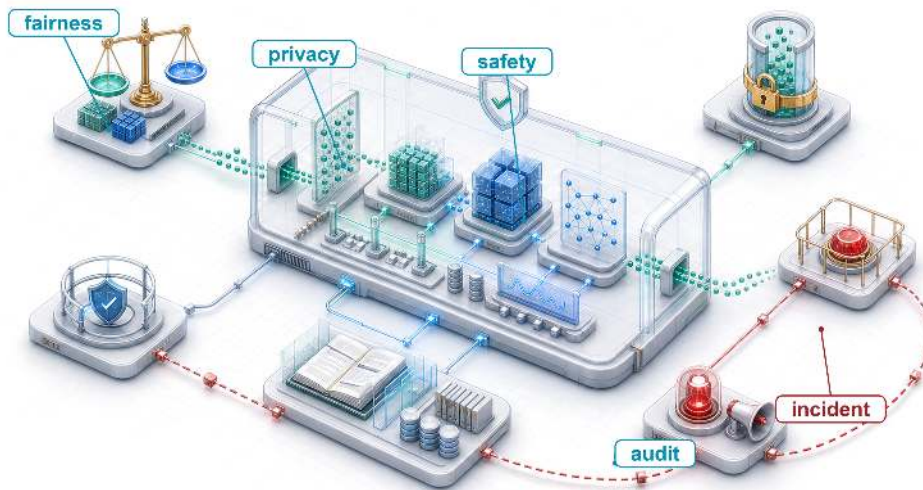
This operational discipline distinguishes production ML engineering from development experimentation. When a deployed model degrades, systematic instrumentation enables practitioners to isolate the failure mode: identifying data drift through distribution divergence, training-serving skew through preprocessing parity audits, configuration debt through artifact provenance, and feedback loops through temporal output tracking. Treating production ML as a “deploy and forget” exercise ignores these distinct dynamics, allowing models to serve invalid inferences for months while infrastructure dashboards remain green.

This operational reality roots ML engineering in the D·A·M taxonomy. In conventional software, code on silicon executes deterministically; in an ML system, the model code and accelerator hardware remain byte-for-byte identical while the data axis continually moves beneath them. The match between model and world is not an engineering state reached once at deployment, but an ongoing cost paid continuously through telemetry, validation gates, and cost-calibrated retraining. System reliability must therefore measure prediction outcomes rather than server uptime alone—because the most dangerous failure mode in production is a green dashboard masking a drifting model.

What's Next: From reliability to responsibility

An ML system can be efficient, scalable, and reliable yet still cause harm by amplifying bias or leaking data. Even 99.9 percent uptime and sub-10 ms latency cannot establish that its decisions are responsible. Chapter 15 addresses the final constraint: aligning technical optimization with human values so deployed systems serve the people affected by them.

Responsible Engineering

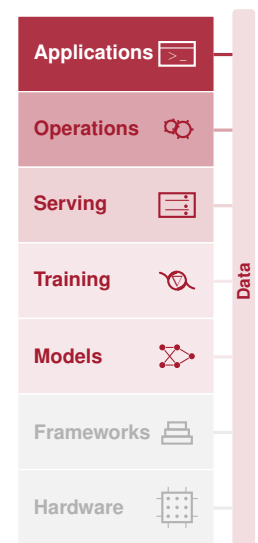


- 15.1 Responsibility as Systems Engineering
- 15.2 Engineering Responsibility Gap
- 15.3 Responsible Engineering Checklist
- 15.4 Environmental and Cost Awareness
- 15.5 Data Governance and Compliance
- 15.6 Fallacies and Pitfalls
- 15.7 Summary

Purpose

Why can a system that does exactly what it was told to do still cause harm?

Operations targets low latency, high availability, and sustained predictive quality. Responsible engineering asks whether those targets describe the right behavior, whom the system serves, and which costs its specification leaves unmeasured. An ML system can satisfy latency, throughput, and aggregate accuracy requirements while reproducing historical discrimination, rewarding harmful engagement, consuming unjustified energy, or using data without adequate privacy and accountability controls. Such a system has not malfunctioned; it is efficiently optimizing an incomplete specification. The omitted constraints appear as uneven outcomes across populations, lifetime cost and emissions, decisions people cannot understand or contest, and evidence that cannot be reconstructed after harm occurs. If those consequences are not defined as requirements before deployment and monitored afterward, conventional health checks can remain green while harm accumulates. Responsible engineering treats them as system failures to be diagnosed, measured, and mitigated with the same rigor as latency or accuracy regressions. That work requires translating broad obligations into testable bounds, documented assumptions, traceable data, and response paths that remain enforceable in production. In D·A·M terms, responsibility broadens the definition of correctness: data must be examined for encoded harms and lawful use, algorithms must be bounded by defensible outcome and efficiency requirements, and machine infrastructure must monitor, document, and enforce those boundaries throughout the system lifecycle.





Learning Objectives

- Explain how optimized ML systems can amplify harm through proxies, feedback loops, and distribution shift
- Apply data-algorithm-machine diagnosis to localize responsibility failures in data, algorithm objectives, or monitoring infrastructure
- Calculate fairness metrics from confusion matrices and compare trade-offs on the fairness-accuracy Pareto frontier
- Design disaggregated evaluation, stress testing, and monitoring to expose subgroup-specific failures before deployment
- Analyze total cost, inference dominance, and carbon impact as measurable responsibility constraints
- Construct model cards, datasheets, lineage records, and audit trails for accountability
- Evaluate privacy, access-control, and compliance designs against regulatory and human-review requirements

15.1 Responsibility as Systems Engineering

¹ | Amazon recruiting tool:

Developed starting in 2014 by Amazon's Edinburgh engineering team to rate applicants on a 1–5 scale, the system trained on approximately a decade of resumes—overwhelmingly from male applicants reflecting the tech industry's gender ratio. By 2015 the gender bias was identified; by 2017 the project was abandoned after repeated remediation attempts (Dastin 2018). The engineering cost was not the compute but the opportunity cost: a multi-year recruiting project failed because the objective encoded historical bias, making it a documented specification failure in ML tooling.

IN 2014, Amazon built an AI recruiting tool¹ that penalized resumes containing the word “women’s” (as in “women’s chess club captain”) and downgraded graduates of all-women’s colleges. The system optimized faithfully for its stated objective: identify candidates similar to those previously hired. The system did not suffer an execution crash or optimization failure; historical hiring patterns encoded gender bias, which gradient descent systematically reproduced at scale.

If MLOps is the control loop for *reliability*, then **responsible engineering** is the control loop for *safety*. Where MLOps monitors runtime health and triggers retraining when predictive throughput degrades, responsible engineering monitors outcome distributions and triggers intervention when systems generate harm. In systems engineering terms, a model can pass *verification* (conforming exactly to its specified training objective) while failing *validation* (failing to satisfy operational, safety, or legal requirements) (National Aeronautics and Space Administration 2016). The failure stems from an incomplete specification rather than a fault in compiler execution or numerical optimization.

Traditional software engineering isolates defects within modules, though shared dependencies can still propagate faults. Machine learning systems introduce data-dependent coupling: data flows through shared representations, so corrupted or biased inputs alter the dense parameter matrices that serve every subsequent inference. A biased training dataset does not trigger a compile-time assertion or runtime crash; it silently shifts decision boundaries across the entire system. Appendix A formalizes the diagnostic framework that isolates where such failures originate across the three D-A-M axes: unrepresentative or biased inputs (Data), misaligned loss formulations (Algorithm), or telemetry blind spots in serving infrastructure (Machine). This makes responsibility an architectural invariant rather than a post-deployment patch.

Responsible engineering expands the definition of correctness for machine learning systems. Traditional correctness—enforcing numerical stability, bounded latency, and high service availability—remains necessary, but insufficient. A production system must satisfy broader operational bounds: bounded error disparities across demographic slices, defensible energy expenditure, and verifiable data lineage. This expanded correctness applies engineering discipline to failure modes that aggregate performance metrics ignore. A p99 latency spike triggers automated alerts on an operational dashboard; a demographic fairness regression remains invisible to aggregate loss metrics until real-world harm occurs (principle 13). Both demand rigorous telemetry and automated regression testing.

Enforcing expanded correctness requires hardware and data infrastructure capable of measuring what aggregate benchmarks obscure. The energy budgets and accelerator footprints quantified throughout this book translate directly into carbon emissions, establishing efficiency optimization as a physical responsibility constraint alongside serving latency. Data governance infrastructure—including access control policies, data lineage tracking, and immutable audit logging—makes those

operational boundaries enforceable in production. Yet before engineering these controls, systems architects must isolate where and why mathematical convergence diverges from system safety: the structural gap between scalar loss minimization and real-world outcomes.

15.2 Engineering Responsibility Gap

A loan model that approves 95 percent of qualified majority-group applicants while rejecting 40 percent of equally qualified minority-group applicants can still achieve a low aggregate loss. The **responsibility gap** between this *technical correctness* and *responsible outcomes* represents a central challenge in machine learning systems engineering, one that existing testing methodologies were not designed to address. The gap manifests through concrete mechanisms: proxy variables, feedback loops, and distribution shift, each producing harm through a distinct pathway that conventional monitoring leaves invisible.

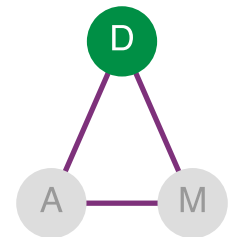
15.2.1 When optimization succeeds but systems fail

The Amazon recruiting-tool failure grounds this responsibility gap on the data axis of the D-A-M taxonomy (see the margin locator), where the failure stems from historical training signal rather than an algorithmic or software defect. A model trained on a decade of historical hiring data optimized faithfully for the objective it was given, but those historical patterns encoded gender bias that the system reproduced in candidate ratings (Dastin 2018).

Under empirical risk minimization, the model minimized classification loss by identifying token-level correlations across the resume corpus. Because historical technical hires were predominantly men, tokens associated with women's colleges, organizations, or activities exhibited negative statistical correlations with positive hiring labels. The optimization algorithm accurately extracted the empirical distribution of the training data, but that distribution reflected historical hiring practices rather than candidate competence. High-dimensional text representations encode and amplify these statistical regularities, embedding societal biases directly into downstream vector representations (Bolukbasi et al. 2016).

Amazon attempted remediation by removing explicit gender indicators and gendered terms from the training pipeline. This intervention failed to ensure unbiased recommendations because high-dimensional feature spaces contain **proxy variables** that retain demographic signal.² Scrubbing isolated attributes leaves correlated features intact: ZIP codes correlate with race through residential segregation, applicant names correlate with ethnicity, and healthcare utilization correlates with socioeconomic conditions. In Amazon's resume pipeline, even after explicit gender terms were excised, remaining vocabulary—including attendance at specific colleges or participation in particular student organizations—continued to encode sex-correlated patterns (Dastin 2018). Removing protected attributes from training data is therefore insufficient to guarantee fairness.

Effective remediation requires architectural interventions across the full system lifecycle rather than a surface-level data filter. At the evaluation layer, disaggregated testing across demographic subgroups quantifies disparate scoring distributions before deployment. At the algorithm layer, adversarial debiasing—where an auxiliary network attempts to predict protected attributes from intermediate representations while the primary model is penalized for retaining that signal—can suppress sensitive features, though it still requires continuous subgroup validation. At the operational layer, human review for borderline scores provides a safeguard only when reviewers are trained, empowered, and audited rather than used to rubber-stamp model outputs. Finally, at the monitoring layer, tracking downstream hiring outcomes over time detects disparate impact that offline loss metrics conceal. Because retrofitting these defenses onto an existing pipeline proved technically intractable, Amazon scrapped the project (Dastin 2018).



The Amazon failure is a data-axis failure: biased historical signal.

² | **Proxy variable:** Removing one proxy may have little effect when other correlated features retain similar signal. Conversely, correlation alone does not prove discrimination or identify the appropriate intervention. Protected-attribute removal, subgroup evaluation, counterfactual tests, causal analysis, and review of the full decision process provide complementary evidence; no single diagnostic is a complete defense.



War Story 15.1: The COMPAS recidivism algorithm audit (2016)

Context: COMPAS is a risk assessment tool used in US courtrooms to predict re-offending for bail and sentencing decisions (Angwin et al. 2016).

Mechanism: Optimizing for calibration across populations with differing baseline recidivism rates forced mathematical trade-offs between predictive parity and error-rate balance across racial groups.

Impact: Black defendants who did not re-offend were incorrectly flagged as high-risk at nearly twice the rate of White defendants (44.9 percent vs. 23.5 percent), while White recidivists were far more often mislabeled low-risk.

Response: The audit exposed why calibration alone could not settle whether the error distribution was acceptable. Any jurisdiction using such scores needs an explicit policy for which fairness criteria matter, independent validation across groups, and review of how the score enters the decision workflow.

Systems lesson: The scores were approximately calibrated by race, but they violated equalized odds because false positive and false negative rates did not match across groups. Formal fairness results show that calibration and error-rate parity conflict when base rates differ, showing why engineering responsibility requires explicitly choosing which fairness constraint matters.

3 | COMPAS (Correctional Offender Management Profiling for Alternative Sanctions): In the analyzed data, COMPAS scores were approximately calibrated by race, meaning a given score corresponded to a similar observed re-offense probability across groups. Because recidivism prevalence differed between populations, calibration coexisted with disparate error rates (Chouldechova 2017; Kleinberg et al. 2017). Calibration testing alone therefore could not establish error-rate parity or determine whether the allocation of errors was acceptable.

The Amazon recruiting tool and the COMPAS risk assessment system³ illustrate the central thesis of this section: optimization success can coexist with catastrophic system failure. Neither model suffered from a software bug or hardware malfunction. In both cases, the optimization algorithm operated correctly on the objective it was given. The failure was architectural: an incomplete problem specification. When the *technical objective* (minimizing empirical prediction error or achieving statistical calibration) diverges from the *system specification* (fair, defensible outcomes across user populations), high mathematical fidelity to the loss function directly amplifies harm.

Standard production telemetry cannot detect specification failures. Monitoring pipelines track operational invariants—inference latency, memory utilization, throughput, and aggregate validation loss—all of which remain green while a model systematically misallocates errors across subgroups. A pipeline can satisfy technical verification (conforming strictly to its code and loss function) while completely failing system validation (violating operational and societal requirements). Preventing silent specification failures requires making responsibility metrics part of the primary test harness: verifying whether the loss function acts as a defensible proxy for the true system goal, and enforcing subgroup error-rate parity before deployment.

Checkpoint 15.1: Responsible design

Responsibility is a system property, not a model property.

Failure modes

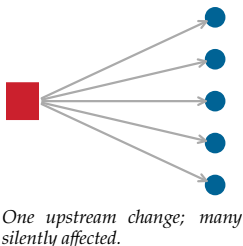
- ☐ **Alignment:** test whether the loss function is a defensible proxy for the system's true goal.
- ☐ **Disparate impact:** compare error rates across subgroups rather than relying on aggregate accuracy.

Check

- ☐ **Pre-mortem:** before deploying, ask: "If this system causes a scandal in six months, what likely went wrong?"

15.2.2 Silent failure modes

Consider a hospital sepsis model that begins recommending aggressive treatments for low-risk patients after an electronic health record (EHR) workflow change alters how vital signs are recorded. As illustrated in the margin, a single upstream modification propagates across the system's blast radius without raising an alert: the model's confidence scores remain high, its inference latency stays within its service level agreement, and all system health checks pass green. The failure is silent: the input data distribution has shifted, but the operational monitoring pipeline has no mechanism to detect distributional drift.



Definition 15.1: Distribution shift

Distribution shift, introduced in chapter 1 and operationalized for drift detection in chapter 14, occurs when the deployment distribution differs from the distribution used to develop or evaluate the model. Standard empirical-risk minimization commonly assumes matching training and deployment distributions. Covariate shift changes $p(x)$ while holding $p(y | x)$ stable; label shift changes $p(y)$ while holding $p(x | y)$ stable; concept drift changes $p(y | x)$.

1. **Significance:** Divergence statistics such as Jensen-Shannon divergence $\mathcal{D}_{JS}(P_t \| P_0)$ measure distributional change, not accuracy degradation. Useful alert thresholds must be calibrated empirically for each task, representation, label process, and deployment environment, then related to observed outcomes when labels are available. A $\mathcal{D}_{JS}$ value of 0.1 may be harmless for one feature space and severe for another, and input-divergence monitoring may miss concept drift.
2. **Distinction:** Distribution shift describes a change between development and deployment conditions. It does not imply that the learned mapping was correct at training time, nor does every shift reduce performance.
3. **Common pitfall:** Distribution shift is an umbrella term with several overlapping taxonomies. Monitoring only $p(x)$ can detect some covariate changes but cannot establish whether $p(y | x)$ remains stable; detecting concept drift generally requires labeled outcomes or justified proxies.

This sepsis scenario illustrates a class of failure that availability-focused monitoring cannot detect. In traditional software systems, catastrophic failures are typically loud: a null pointer exception terminates execution, a segmentation fault halts the process, or an exhausted socket pool returns a network timeout. ML systems introduce statistical failure modes in which corrupted or degraded outputs look indistinguishable from valid predictions. Floating-point tensors continue to flow through matrix multiplication kernels, accelerator execution completes within latency budgets, and downstream services receive structurally well-formed responses.

Distribution shift represents the first major mechanism behind this silent degradation (operational detection and monitoring strategies for drift are detailed in chapter 14). The failure is environmental: the physical distribution of inputs changed after training, yet the model has no internal telemetry to detect the discrepancy. Retraining on fresh data can partially restore accuracy under distribution shift. However, retraining cannot resolve a second, more fundamental mechanism of silent failure that persists even when the data distribution is perfectly stationary: metric misalignment.

Metric misalignment occurs when the objective function optimizes an observable proxy metric that diverges from the unobserved outcome the system is intended to deliver. As shown in the margin, Goodhart's Law governs this divergence: once a proxy metric becomes the target of optimization, it ceases to be a reliable measure of the underlying goal.

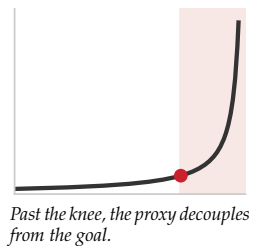
Systems Perspective 15.1: The alignment gap

A model optimizes a proxy metric (Clicks) because the true metric (User Satisfaction) is unobservable, and the two can diverge sharply. Goodhart's Law warns that optimizing a measure can weaken its relationship to the underlying goal. In an illustrative system, $\text{Correlation}(\text{Clicks}, \text{Satisfaction})$ might begin at 0.8; once ranking maximizes clicks, it may find clickbait with high clicks but low satisfaction, and the correlation could fall to 0.2.

Conceptually, assuming normalized metrics on a common scale, equation 15.1 captures the gap:

$$\text{Gap} = \mathbb{E}[\text{Proxy}] - \mathbb{E}[\text{True}] \quad (15.1)$$

If the model increases Clicks by 20 percent but decreases Satisfaction by 5 percent, the signed alignment gap increases.



Systems insight: Engineers cannot directly optimize an unobserved goal. They need defensible proxy validation; randomized holdouts can estimate causal effects only when their outcomes measure the goal or a justified surrogate.

The alignment gap illustrates a failure originating in the algorithm axis: the loss function optimizes a misspecified objective, so even a model that generalizes with zero test error drives outcomes that conflict with organizational or societal goals. Distribution shift, by contrast, originates in the data axis: the model logic remains intact, but the input distribution departs from the training distribution. Both failures are silent, but they require opposite engineering responses—loss function redesign versus input monitoring and retraining. Conflating them wastes engineering cycles on ineffective remediations. To isolate the root cause of silent failures, systems engineers apply the D·A·M taxonomy introduced in chapter 1 and formalized in Appendix A.

🔍 Systems Perspective 15.2: The D·A·M taxonomy

When a system causes harm, use the D·A·M taxonomy to identify the root cause. Responsibility failures are rarely “algorithm bugs”; they are structural flaws along one of the three axes:

- **Data (information):** Does the training data reflect historical bias? (for example, Amazon’s recruiting tool learning from biased history). The failure is in the *Fuel*.
- **Algorithm (logic):** Does the objective function optimize a proxy for harm? (for example, optimizing “engagement” amplifies polarization). The failure is in the *Blueprint*.
- **Machine (physics):** Does the energy cost justify the societal benefit? (for example, training a massive model for a trivial task). The failure is in the *Engine*.

Locating the dominant failure in the taxonomy identifies the first remediation to test: better curation (Data), safer objectives (Algorithm), or more efficient infrastructure (Machine). Real failures can span axes, so the intervention must still be evaluated end to end.

While the D·A·M taxonomy isolates *where* failures originate, operational engineering requires identifying *when* and *how* different failure modes manifest in production telemetry. Table 15.1 categorizes ML system failures by detection latency, spatial blast radius, and recovery interventions. Whereas traditional infrastructure crashes and throughput bottlenecks trigger immediate alerts, statistical failures operate silently—persisting for days or months while technical health checks remain green.

Table 15.1: Illustrative ML System Failure Mode Taxonomy: Detection time, scope, and remediation depend on the application; the entries show representative cases. Silent failures such as data quality issues, distribution shift, and fairness violations may require proactive monitoring because they do not necessarily trigger traditional alerts.

Failure Type	Illustrative Detection Time	Illustrative Scope	Possible Response	Example
Crash	Immediate	Complete	Restart after correcting cause	Out of memory error
Performance Degradation	Minutes	Complete	Relieve resource contention	Latency spike from resource contention
Data Quality	Hours–days	Partial	Data correction may be needed	Corrupted inputs from upstream system
Distribution Shift	Days–weeks	Partial or all	May require adaptation	Population change due to new user segment
Fairness Violation	Weeks–months	Subpopulation	May require redesign	Bias amplification in historical patterns

The YouTube recommendation feedback loop (examined as a technical debt pattern in section 14.3.6.1) illustrates this pattern at scale (M. H. Ribeiro et al. 2020).⁴ M. H. Ribeiro et al. (2020) audited radicalization pathways on YouTube, finding migration from milder to more extreme channel categories and recommendation reachability between those categories. The broader systems

⁴ | **Goodhart’s law:** “When a measure becomes a target, it ceases to be a good measure” (Strathern’s generalization of Goodhart’s 1975 monetary policy observation) (Strathern 1997; Goodhart 1984). Recommendation feedback loops are the canonical ML manifestation: gradient descent optimizes watch-time proxies at a speed no human curator can match, and the system’s own outputs reshape the training distribution—users who consume extreme content generate data that reinforces extremity, decoupling the proxy from user welfare orders of magnitude faster than manual editorial processes ever could.

lesson is that feedback loops can work exactly as designed while producing outcomes that conflict with societal values. Recommendation objectives must therefore be tested against downstream harms, not only against engagement proxies.

The Facebook News Feed overhaul provides an instructive contrast to the YouTube feedback loop: an engineering team deliberately sacrificing short-term engagement metrics to arrest proxy decoupling and protect long-term user retention (Mosseri 2018).

Example 15.1: News Feed proxy shifts

Scenario: Facebook shifted News Feed ranking objectives from short-term engagement proxies (clicks, watch time) to meaningful social interaction metrics (Mosseri 2018).

Diagnosis: Optimizing solely for short-term engagement proxies decoupled ranking outputs from long-term user welfare, amplifying clickbait and passive consumption.

Systems lesson: Engagement metrics are proxies for value, not value itself. Recommendation algorithms require multi-objective optimization with explicit safety constraints to prevent proxy collapse.

When proxy misalignment intersects with demographic heterogeneity, silent failures manifest as structural discrimination. Distribution shift often appears as cohort mismatch, where models evaluated on aggregate benchmarks perform unreliably on specific demographic subgroups without triggering aggregate error alarms. When the training objective optimizes a proxy that reflects historical inequities, the model encodes those disparities into its decision boundaries. This mechanism caused Amazon’s experimental recruiting engine to penalize resumes containing the word “women’s,” and it reappeared with life-threatening consequences in clinical healthcare triage.

War Story 15.2: The proxy variable trap (2019)

Context: In 2019, Ziad Obermeyer and colleagues at UC Berkeley audited a commercial Optum algorithm used by health systems to enroll patients with complex needs into high-risk care management programs, examining roughly fifty thousand patients across one large academic hospital (Obermeyer et al. 2019).

Mechanism: The model predicted “future healthcare cost” as a proxy for “future health need.” Because the US healthcare system spends less on Black patients than on White patients with the same illness level, the algorithm learned this pattern and assigned lower risk scores to Black patients.

Impact: At any given risk score, Black patients carried substantially more chronic conditions than White patients, reducing the share of Black patients enrolled in specialized care.

Response: The study showed that replacing cost with a direct measure of health need would raise the share of Black patients identified for additional care from 17.7 percent to 46.5 percent at the analyzed threshold.

Systems lesson: Optimizing for a proxy inherits the biases of the system that generated the proxy. The proxy-target relationship must be audited across every demographic subgroup the system serves.

Silent failure modes fundamentally alter the systems contract between software engineering and operational verification. Traditional software verification relies on deterministic assertions: a unit test asserts that an output matches an exact value, memory bounds checkers assert that buffer allocations do not overflow, and watchdog timers ensure RPC latencies do not breach service level agreements. In an ML system, correctness cannot be verified by deterministic invariants alone. Floating-point matrix multiplications execute cleanly, memory usage remains stable, and latency checks remain green even while the underlying model serves biased, uncalibrated, or harmful predictions. Converting responsibility goals into structured engineering practice requires operationalizing statistical verification: continuously monitoring covariate shift across demographic

slices, auditing proxy-to-target correlations, and bounding closed-loop feedback before silent errors propagate through production.

15.2.3 When responsible engineering succeeds

Each documented success shares the same structural move: a vague responsibility goal becomes an engineering constraint that can be specified, tested, communicated, and, when necessary, used to stop deployment. Following the findings of Gender Shades, a 2018 audit that exposed severe error-rate disparities in commercial gender classification (Buolamwini and Gebru 2018), Microsoft invested in improving gender-classification performance across demographic groups. Targeted data collection, model changes, and systematic disaggregated evaluation gave the team an explicit error-rate target, bringing audited error rates for darker-skinned subjects below 2 percent (Raji and Buolamwini 2019). Publishing these disaggregated benchmarks established a verifiable baseline and turned external audit findings into an operational release gate.

Twitter's automatic image cropping system shows the same discipline under a different constraint. In 2020, users raised concerns about racial and gender bias in preview thumbnails. Twitter audited the system, reported measured disparities and limitations, and shifted toward uncropped previews that gave users more control (Yee et al. 2021). In that case, responsible engineering changed the product design rather than relying on an incremental adjustment to an automated ranking threshold.

Differential privacy formalizes this pattern: a privacy requirement becomes a mathematical guarantee rather than a policy aspiration (Dwork 2008).⁵ Systems implementing differential privacy calibrate injected noise to balance utility against privacy loss, track cumulative privacy expenditure across repeated analyses with an accountant, and document the chosen parameters.

These successes demonstrate that responsible engineering functions only when high-level principles are translated into enforceable systems constraints across the D·A·M stack: curating training data distributions, bounding algorithmic loss functions, or tracking privacy budgets in execution runtimes. Enforcing these constraints requires the operational authority to halt deployment or retire components that violate specified bounds. Every successful intervention depends on continuous evaluation, yet verifying these statistical constraints differs fundamentally from traditional software verification.

15.2.4 The testing challenge

Traditional software testing evaluates behavior against unambiguous specifications: an arithmetic function must return the exact sum of its operands, and a relational database must preserve foreign-key integrity. Such deterministic properties map directly to executable assertions.

Responsible ML properties resist simple formalization. **Fairness** has multiple mathematical definitions that can conflict under conditions such as unequal base rates and imperfect prediction (Chouldechova 2017; Kleinberg et al. 2017). What counts as fair depends on context, values, and trade-offs that technical systems cannot resolve alone. Individual fairness requires that similar individuals receive similar treatment, while group fairness requires equitable outcomes across demographic categories. These criteria can conflict, and choosing between them requires value judgments beyond the scope of optimization.

Fairness constraints and predictive performance can trade off, but the relationship is application-specific. A Pareto frontier represents configurations for which one plotted objective cannot improve without degrading another. Figure 15.1 visualizes one hypothetical fairness-accuracy frontier. Its three points illustrate possible policy choices; they do not imply that unconstrained training always maximizes accuracy at high disparity, that zero disparity must reduce accuracy, or that every application has a sweet spot.

The frontier tells engineers what trade-off they may need to choose, but it cannot be plotted until subgroup performance is measured. Responsible properties become testable when engineers work with stakeholders to define criteria appropriate for specific applications. The Gender Shades project⁶ demonstrated how **disaggregated evaluation** across demographic categories reveals disparities invisible in aggregate metrics (Buolamwini and Gebru 2018), exposing the subgroup failures that responsibility monitoring must catch before deployment. Table 15.2 shows the dramatic error-rate differences that commercial gender-classification systems produced across demographic groups. Concretely, a 10,000-sample test set that suffices for the majority group provides only 100 samples

⁵ | **Differential privacy:** Introduced by Dwork et al. (2006), a randomized mechanism $\mathcal{M}$ satisfies (ϵ, δ) -differential privacy if for all neighboring datasets D, D' differing by one record and output sets $\mathcal{S}$, $\mathbb{P}[\mathcal{M}(D) \in \mathcal{S}] \leq e^\epsilon \cdot \mathbb{P}[\mathcal{M}(D') \in \mathcal{S}] + \delta$. Here ϵ bounds privacy loss and δ permits a small probability of exceeding that bound; their acceptable values depend on the threat model and policy. The systems trade-off is utility rather than mere implementation complexity: stronger privacy often requires more noise or tighter sampling, and privacy loss composes across repeated queries or training steps. Engineers must therefore track cumulative privacy loss with a valid accountant and report the assumptions behind the chosen budget.

⁶ | **Gender Shades:** A 2018 study by Joy Buolamwini (MIT Media Lab) and Timnit Gebru (Microsoft Research) that audited commercial gender-classification systems from Microsoft, IBM, and Face++ using the Fitzpatrick skin type scale—originally a dermatological classification developed by Thomas Fitzpatrick in 1975 for UV sensitivity and later validated for clinical use (Fitzpatrick 1988), repurposed here as a demographic benchmark for algorithmic auditing. The study demonstrated the value of disaggregated evaluation; in the Face++ results reproduced in table 15.2, the dark-female error rate is 43.1× the light-male rate. Microsoft's later reported reductions show how public audit results can motivate measurable remediation (Raji and Buolamwini 2019).

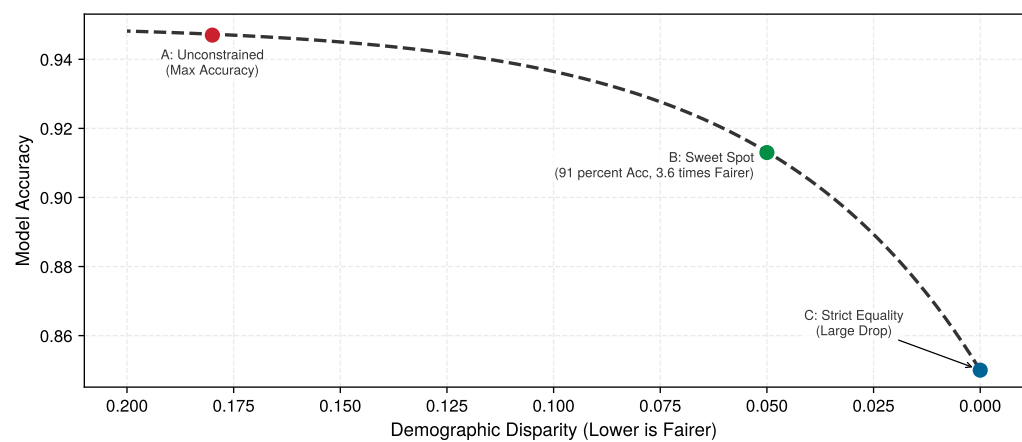


Figure 15.1: An Illustrative Fairness-Accuracy Pareto Frontier: A hypothetical relationship between model accuracy and demographic disparity. The x-axis is inverted so lower disparity lies to the right. Points A, B, and C illustrate three possible operating choices along this constructed frontier; the curve is not an empirical or universal law.

for a minority subgroup representing 1 percent of the population—effectively requiring 100× more data than the majority group for high-confidence validation.

Table 15.2: Gender Shades Face++ Error Rates: The dark-female error rate is 43.1× the light-male rate. Source: (Buolamwini and Gebru 2018).

Demographic Group	Error Rate (%)	Relative to Light-Skinned Males
Light-skinned males	0.8%	Baseline (1.0×)
Light-skinned females	9.8%	12.2×
Dark-skinned males	0.7%	0.9×
Dark-skinned females	34.5%	43.1×

As table 15.2 quantifies, disaggregated evaluation revealed what an aggregate score concealed. Face++ reported error rates of 0.8 percent for light-skinned males and 34.5 percent for dark-skinned females (accuracies of 99.2 percent and 65.5 percent). The aggregate metric gave no indication that the dark-female error rate was 43.1× the light-male rate.

No universal threshold defines acceptable disparity, but teams should establish explicit, justified bounds before deployment. A team might adopt an error-rate ratio below 1.25× or a false-positive-rate difference under 5 percentage points as an application-specific release policy. In US employment law, the disparate impact doctrine⁷ shapes regulatory oversight, while the four-fifths rule⁸ provides a separate, non-dispositive selection-rate diagnostic. The key engineering discipline is defining criteria appropriate to the application and governing law rather than generalizing one domain’s threshold.

Surfacing latent responsibility failures before deployment requires expanding testing beyond uniform validation sets across three distinct verification regimes.

Slice-based evaluation partitions evaluation data into demographic and operational cohorts, computing metrics independently for each slice. A model may achieve 95 percent accuracy overall while dropping to 78 percent on low-income applicants or rural cohorts—a severe performance disparity invisible in aggregate reporting.

Invariance testing verifies that model decisions remain stable across semantically irrelevant feature substitutions. Swapping candidate names such as “John” and “Jamal” in a loan application must not alter approval probabilities when demographic features are legally and technically irrelevant to creditworthiness. Behavioral testing frameworks such as CheckList operationalize this principle by organizing validation suites around behavioral capabilities and perturbation invariances rather than static validation loss (M. T. Ribeiro et al. 2020).

Boundary and stress testing probe input regimes where empirical validation data is sparse or unrepresentative. Boundary evaluation maps behavior at distributional margins—extreme age distributions, rare transaction amounts, or sparse geographic categories—where prediction variance explodes. Stress testing extends these margins into adversarial conditions: corrupted feature pipelines, sudden covariate shift, and synthetic edge cases designed to test safety margins under

⁷ | **Disparate impact:** In *Griggs v. Duke Power Co.* (1971), the US Supreme Court held under Title VII that an employment practice may be unlawful because of its operation even without discriminatory intent (Supreme Court of the United States 1971). Disparate impact is a legal claim with statute- and context-specific elements, not a synonym for any statistical disparity. Model outcomes and proxies can supply relevant evidence, but metrics alone do not establish liability.

⁸ | **Four-fifths rule:** The 1978 Uniform Guidelines on Employee Selection Procedures use the four-fifths rule as a practical employment-selection diagnostic (Equal Employment Opportunity Commission et al. 1978). A selection ratio below 80

to ethics boards or legal departments. These groups provide essential oversight but lack the technical access required to identify problems early in the development process.

Legal or ethics review can identify a problem near deployment, but it cannot recover design options the system has already foreclosed. If the team trained the model without fairness constraints, chose an architecture that cannot support interpretability requirements, or built a data pipeline without the demographic attributes needed for monitoring, review can only accept, reject, or demand expensive redesign. Engineers therefore occupy a critical position in the ML development lifecycle because their choices define the solution space for all subsequent interventions: architecture determines which fairness constraints can apply, the optimization objective determines which patterns the system learns, and the data pipeline determines whether disaggregated evaluation is possible.



Definition 15.2: Responsible AI engineering

Responsible AI engineering is the engineering discipline of designing, deploying, and maintaining systems with probabilistic outputs by operationalizing societal and regulatory requirements as testable constraints on the D·A·M axes: permissible data contents, provenance, and composition; allowable model behavior and robustness properties; and infrastructure bounds such as latency, energy, compute budget, carbon emissions, and audit-log retention.

1. **Significance:** Each D·A·M axis acquires concrete governance constraints: the data axis is bounded by privacy regulations such as the General Data Protection Regulation (GDPR), which limits which records, fields, and features can be collected; the algorithm axis is bounded by fairness and robustness metrics (for example, demographic parity within $\varepsilon = 5\%$ across protected groups, meaning positive prediction rates must not differ by more than 5 percentage points, or accuracy degradation less than 2 percent under adversarial perturbation $\|\delta\|_\infty \leq 0.01$, a worst-case input change bounded to 0.01 per normalized feature under the ℓ_∞ norm); and the machine axis is bounded by resource and infrastructure budgets such as latency, energy per inference, carbon emissions, and audit-log retention. Violating these bounds is a system failure, not a research shortcoming.
2. **Distinction:** Unlike AI ethics (which articulates aspirational values), responsible AI engineering translates those values into measurable, testable invariants that can be verified through automated testing and continuous monitoring, using the same lifecycle practices that enforce latency SLOs.
3. **Common pitfall:** A frequent misconception is that responsibility is “added” at the end of development. Constraints on what data may be collected affect what can be learned and audited, while infrastructure choices affect what evidence can be retained. Late-stage remediation may remain possible, but it is often narrower, slower, and more expensive.

An engineering-centered approach does not diminish the importance of diverse perspectives in identifying potential harms. Product managers, user researchers, affected communities, and policy experts contribute essential knowledge about how systems fail socially despite technical success. Engineers translate these concerns into measurable requirements and testable properties that can be verified throughout the development lifecycle. Effective responsibility requires engineers who both listen to stakeholder concerns and possess the technical capability to implement appropriate safeguards.

Engineering teams do not operate in isolation. As figure 15.2 makes clear, engineering practices are nested within broader organizational, industry, and regulatory governance structures, each layer imposing constraints on the ones inside it. Technical excellence at the innermost layer enables, but does not replace, compliance with requirements flowing inward from external governance.

Those governance layers define who owns responsibility, but they do not yet account for the costs that ordinary performance metrics omit.

Beyond ethical imperatives, responsible engineering can deliver business value through three reinforcing mechanisms. The most immediate is risk mitigation: ML system failures create legal and financial exposure that systematic responsibility practices can reduce. Amazon abandoned an internal recruiting model after discovering sex-linked disparities in its recommendations. Organi-

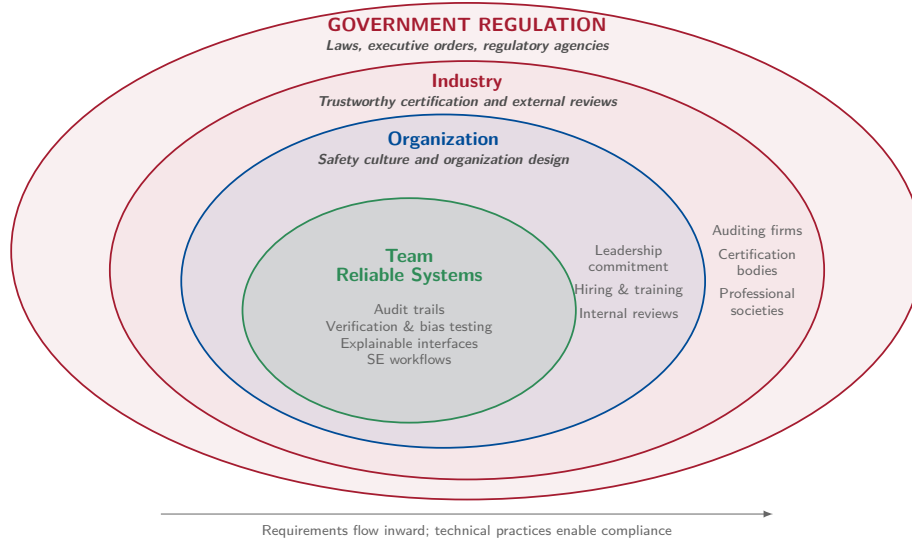


Figure 15.2: Responsible AI Governance Layers: Four nested ovals place team-level safeguards at the center, surrounded by organizational safety culture, industry certification and review, and government regulation. The nesting distinguishes implementation at the center from oversight outside it; the annotation states that requirements flow inward and technical practices enable compliance. Adapted from Shneiderman (2022).

zations implementing disaggregated evaluation, documentation, and monitoring can reduce the probability of costly failures and preserve evidence of their engineering process if problems emerge.

A second mechanism is regulatory compliance, driven by legal requirements that vary by jurisdiction and application risk. The EU AI Act, for example, classifies high-risk AI applications and mandates technical requirements including risk assessment, data governance, transparency, and human oversight. Organizations that build responsibility into engineering practice can demonstrate compliance through existing documentation and monitoring rather than expensive retrofitting; the engineering lesson is that proactive controls are usually cheaper than reconstructing evidence after deployment.

Systems Perspective 15.3: The full cost of the iron law

The iron law of ML systems (principle 3) established in section 1.7 holds that system performance depends on the interaction between data, compute, and system overhead. Earlier chapters optimized each term by compressing models (chapter 10), accelerating hardware (chapter 11), and automating operations (chapter 14). Yet an optimization can create or shift costs that its benchmark omits.

A model quantized for edge deployment consumes less energy, but also produces outputs that may differ across demographic groups. A recommendation system optimized for engagement maximizes a business metric, but may amplify harmful content. Responsible engineering extends this accounting to include broader impacts: the carbon cost of computation, the fairness cost of optimization choices, and the societal cost of deployment at scale. The iron law governs *how fast* systems run; responsible engineering governs *how well* they serve.

Competitive differentiation completes the business case. Trust can drive enterprise purchasing decisions for ML-powered services, and organizations that can demonstrate systematic responsibility practices through model cards, audit trails, and published evaluation results may qualify for deployments that competitors cannot. Apple’s privacy positioning, Microsoft’s responsible AI principles, and Anthropic’s safety research illustrate responsibility as a strategic investment rather than a purely defensive cost.

The quantization techniques from chapter 10 can reduce inference energy when the deployed runtime and hardware exploit the representation and measured workload energy falls. The monitoring infrastructure from chapter 14 can support disaggregated fairness evaluation when the system lawfully collects the necessary outcome and subgroup data. Responsible engineering synthesizes these capabilities into disciplined practice through structured frameworks that translate principles into processes.

Systematic processes applied early could have reduced the likelihood or severity of the failures examined in section 15.1. Checklists, documentation standards, testing protocols, and monitoring infrastructure translate responsibility principles into repeatable engineering workflows, but they do not guarantee prevention.

15.3 Responsible Engineering Checklist

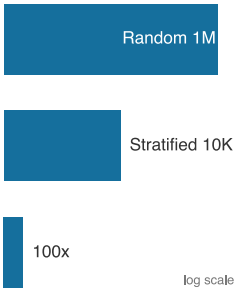
A structured predeployment review could have surfaced the sex-linked signals learned by Amazon’s recruiting tool, while disaggregated testing exposed COMPAS’s error-rate disparity. Both failures shared a common cause: responsibility was treated as a separate review stage rather than integrated into the development workflow. A responsible engineering checklist embeds assessment wherever engineering decisions create durable risk: before deployment, in documentation, during population-specific evaluation, at explanation and compliance boundaries, and after launch through monitoring. The stages build on one another: assessment identifies what to measure, documentation preserves the assumptions, fairness evaluation checks whether performance holds across groups, explainability and compliance translate decisions into obligations, and monitoring connects detected violations to intervention.

15.3.1 Predeployment assessment

Before a loan approval model reaches production, a team must determine the provenance of the training data, identify who is represented and who is missing, anticipate failure modes, and define recourse for affected users. Table 15.3 structures this evaluation into five phases, distinguishing critical-path blockers from high-priority items that can proceed with documented risk acceptance.

Table 15.3: Predeployment Assessment Framework: Critical Path items block deployment until addressed. High Priority items should be completed before or shortly after launch. Systematic coverage of responsibility concerns throughout the ML lifecycle reduces the chance that risks are overlooked.

Phase	Priority	Key Questions	Documentation Required
Data	Critical Path	Where did this data come from? Who is represented? Who is missing? What historical biases might be encoded?	Data provenance records, demographic composition analysis, collection methodology documentation
Training	High Priority	What is the system optimizing for? What does the loss implicitly penalize? How do architecture choices affect outcomes?	Objective function specification, regularization choices, hyperparameter selection rationale
Evaluation	Critical Path	Does performance hold across different user groups? What edge cases exist? How were test sets constructed?	Disaggregated metrics by demographic group, edge case testing results, test set composition analysis
Deployment	Critical Path	Who will this system affect? What happens when it fails? What recourse do affected users have?	Impact assessment, stakeholder identification, rollback procedures, user notification protocols
Monitoring	High Priority	How are anomalies detected? Who reviews system behavior? What triggers intervention?	Monitoring dashboard specifications, alert thresholds, review schedules, escalation procedures



Random sampling barely reaches small subgroups.

Critical Path items are deployment blockers: the system must not go to production until these questions are answered. High Priority items should be addressed but may proceed with documented risk acceptance and a remediation timeline. The distinction enables teams to ship responsibly without requiring perfection on every dimension before initial deployment.

The Evaluation row in table 15.3 raises the critical concern of whether performance holds across different user groups. Answering this question requires statistically valid test sets for each group, which can create surprisingly stringent data requirements when representation is uneven.

Napkin Math 15.1: The statistics of representation

Problem: An engineering team needs to verify that a Face ID model works for a minority group representing 1 percent of the user base. A worst-case binomial margin of error near 1 percentage point at 95 percent confidence requires roughly 10,000 images for this group.

Random sampling: To get 10,000 images of a 1 percent group via random sampling, the team must collect and label: $D_{\text{eval,total}} = 10,000 \text{ images} / 0.01 = 1,000,000 \text{ images}$

Stratified sampling: Specifically targeting this group (for example, via active learning or community outreach) requires only 10,000 images. **Systems insight:** Relying on “natural distribution” data for fairness is prohibitively expensive under random sampling. Validating the minority group effectively requires 100× more data than the majority group. Fairness requires intentional data engineering, not just more data.

Intentional data engineering addresses *what* the model sees during evaluation, but even a perfectly representative dataset cannot prevent harm at deployment if the system lacks adequate human oversight. The representation cost derived in napkin math 15.1 is a predeployment gate; the question that follows is what happens once the model is live and making decisions that affect people.

War Story 15.3: The automation paradox (2018)

Context: Uber’s Advanced Technologies Group (ATG) was testing self-driving cars in Arizona, relying on a human safety driver to intervene if the autonomous perception system failed (National Transportation Safety Board 2019).

Mechanism: The perception system detected a pedestrian crossing the road but repeatedly toggled its classification between vehicle, bicycle, and unknown, resetting its trajectory prediction while automatic emergency braking was intentionally disabled.

Impact: The safety driver was visually distracted by a personal phone and failed to take control, resulting in a fatal collision.

Fix: Uber retrofitted the fleet with driver-monitoring cameras, re-enabled automatic emergency braking, and established dual-operator staffing during autonomous road testing.

Systems lesson: Adding a human backup creates a new system with its own failure modes. High automation reliability can encourage complacency and reduce vigilance, so an effective fallback requires workload design, attention monitoring, and a safety case for the combined human-machine system.

For high-stakes applications, the deployment phase should specify where human oversight is required. Human-in-the-loop (HITL) systems route uncertain, high-consequence, or flagged decisions to human reviewers rather than acting autonomously. Effective HITL design must specify four requirements: the review scope (which decisions require human review), the confidence thresholds that trigger escalation, the training reviewers receive, and the mechanisms for monitoring reviewer performance. HITL is not a catch-all solution: human reviewers can rubber-stamp automated decisions, introduce their own biases, or become overwhelmed by alert volume. Effective HITL design requires calibrating the human-machine boundary to the specific application risks and reviewer capabilities.

The predeployment assessment framework parallels aviation preflight checklists, which standardize critical checks under time pressure. Production ML deployments require equivalent discipline and rigorous verification. A checklist prompts teams to ask the right questions but does not prove that a system is safe; documentation standards preserve the answers and allow them to travel with the model.

15.3.2 Model documentation standards

Consider inheriting a production model without metadata: the model achieves 94 percent accuracy on a designated test set, but the origin of that test set, the composition of the training data, and the



Without effective controls, reliable automation can erode vigilance.

⁹ | **Model cards:** The primary failure mode model cards address is scope creep—gradual expansion from “it worked for case A” to “try it for case B” without revalidating intended use. In practice, cards are often written after deployment decisions are made, documenting observed behavior rather than constraining it. The companion “Datasheets for Datasets” (Gebru et al. 2021) applies the same principle to training data. Without both, the card becomes a historical record rather than an engineering guardrail.

demographic or environmental slices evaluated are unrecorded. Without those baselines, updating or redeploying the model risks unmeasured regressions under distribution shift. **Model cards** provide a standardized interface specification for trained models⁹ (Mitchell et al. 2019). Analogous to component datasheets in computer engineering, model cards document operational boundaries, evaluation conditions, and intended operating regimes alongside the model artifact.

A complete model card covers seven core sections that together formalize the model’s operational envelope. It begins with technical details (architecture, training procedures, and hyperparameters) that enable reproducibility and auditing. It specifies *intended use* alongside explicit out-of-scope applications, preventing scope creep where models trained for consumer photo indexing get repurposed for high-stakes surveillance. The card then documents which *factors* (demographic groups, environmental conditions, or sensor variations) affect performance, guiding both evaluation slicing and production monitoring.

The remaining sections quantify and bound empirical behavior. Performance metrics must report disaggregated results across the operational factors identified in section 15.2.4, because aggregate accuracy conceals systematic subpopulation disparities. Documenting the training and evaluation datasets isolates potential sampling biases and defines the distribution over which the reported metrics hold. Finally, ethical considerations, caveats, and recommendations make deployment trade-offs explicit by recording known failure modes, unmodeled edge cases, and operational mitigations.

Applying this schema to MobileNetV2 illustrates how these categories translate into concrete deployment constraints; table 15.4 summarizes the resulting card for an edge classification service.

Table 15.4: Illustrative Model Card: MobileNetV2 for Edge Deployment: Abstract model card categories translate to practical documentation that guides responsible deployment decisions. The numerical entries are scenario assumptions, not reported benchmark results.

Section	Content
Model Details	MobileNetV2 architecture with 3.5M parameters, trained on ImageNet using depthwise separable convolutions. INT8 quantized for edge deployment.
Intended Use	Real-time image classification on mobile devices with less than 50 ms latency requirement. Suitable for consumer applications including photo organization and accessibility features.
Factors	Performance varies with image quality (blur, lighting), object size in frame, and categories outside ImageNet distribution.
Metrics	71.8% top-1 accuracy on ImageNet validation (full precision: 72.0%). Accuracy varies by category: 85% on common objects, 45% on fine-grained distinctions.
Ethical Considerations	Training data reflects ImageNet biases in geographic and demographic representation. Not validated for high-stakes applications (medical diagnosis, security screening). Performance may degrade on images from underrepresented regions.

Datasheets for datasets provide complementary documentation for the data tier (Gebru et al. 2021). In D·A·M terms, while a model card specifies the algorithmic and machine boundaries—including quantization precision and inference latency budgets—a datasheet captures the data provenance, collection methodology, curation filters, and demographic composition. Together, these documentation standards establish the operational contract for the system: documentation specifies what the system is designed to tolerate, while empirical testing verifies whether it performs equitably across the populations it serves.

15.3.3 Testing across populations

The disaggregated evaluation that exposed the Gender Shades disparities (section 15.2.4) now becomes an operational release-gate task: selecting the slices, metrics, and thresholds that determine whether deployment is allowed. Aggregate performance metrics mask disparities across user populations, the **flaw of averages** (Savage 2009); responsible testing requires disaggregated evaluation that examines performance for each relevant subgroup.

Systems Perspective 15.4: The flaw of averages

Systems engineering rarely designs for the “average” case; robust systems are engineered against tail events and boundary conditions. A bridge that is “safe on average” but collapses under a heavy truck is an engineering failure. Similarly, an ML system that is “accurate on average” but fails for a specific demographic cohort represents an engineering failure. The same discipline that mandates measuring tail latency (p99) for system reliability applies to equitable performance: disaggregated evaluation must measure performance across demographic cohorts. Looking only at aggregate accuracy blinds operators to systemic failures occurring in the margins. Responsible engineering makes these tails visible through granular, population-specific measurement.

The flaw of averages establishes the core testing principle: aggregate metrics conceal tail failures. Translating this principle into an engineering workflow requires determining which population slices and failure modes to isolate. Because a vision classifier fails differently from a recommendation engine or an audio wake-word detector, fairness metrics and evaluation slices must reflect the application domain. In healthcare diagnostics, demographic cohorts such as age and biological sex govern slice definitions. In content moderation, dialect and linguistic context dominate. In financial underwriting, credit regulations mandate auditing legally protected categories.

Operationalizing these evaluations requires four core capabilities in the testing infrastructure. First, stratified evaluation computes performance metrics separately for each cohort to expose error-rate disparities across populations. Second, intersectional analysis evaluates combinations of attributes, catching compounding failure modes that remain invisible in single-factor slices. Third, confidence intervals quantify statistical uncertainty when sparse subgroup sample sizes yield noisy point estimates. Fourth, temporal monitoring tracks cohort performance continuously across deployment, detecting distribution drift that degrades minority slices before impacting aggregate health checks.

Tool selection matters only after the engineering team specifies the metrics, subgroup slices, and alerting thresholds. Open-source evaluation suites such as Fairlearn (Bird et al. 2020), AI Fairness 360 (Bellamy et al. 2019), and Google’s What-If Tool (Wexler et al. 2020) simplify the computation of stratified and intersectional metrics, but software libraries cannot supply domain judgment. They compute whichever metric an engineer queries; they cannot decide which demographic boundaries matter, calibrate paging thresholds for on-call engineers, or select the appropriate legal fairness constraint.

An operational release gate must distinguish absence of evidence from evidence of parity. If a slice lacks sufficient labeled validation data to bound the error rate with statistical confidence, the CI/CD gate must report insufficient data rather than a false pass—triggering targeted data acquisition or restricting deployment scope. Population testing thus enforces a tri-state operational contract: pass, fail, or insufficient evidence.

Lighthouse 15.1: Fairness concerns by archetype

Fairness risks vary across the workload archetypes in chapter 2. Table 15.5 maps each to a primary risk and metric.

Architecture alone does not determine fairness; each row identifies a deployment-specific risk to test first, not a universal property of the model family.

Table 15.5: Fairness Risk by ML Archetype: Fairness risks vary by archetype’s data source and deployment context.

Archetype	Primary Fairness Risk	Key Evaluation Metric	Real-World Example
ResNet-50 (Compute Beast)	Training data bias (underrepresentation of deployment-relevant groups)	Disaggregated accuracy by demographic group	Evaluate the deployed classifier on application-specific demographic slices; Gender Shades values do not measure ResNet-50
GPT-2 (Bandwidth Hog)	Corpus bias (overrepresentation of majority viewpoints in web text)	Toxicity rate by demographic prompt context; stereotype score	LLMs produce more toxic completions for prompts mentioning minority groups
DLRM (Sparse Scatter)	Feedback-loop amplification (popular items get more data)	Share of recommendation impressions by item category and supplier or creator group	Filter bubbles: the system recommends similar content to similar users, reducing discovery of niche creators
DS-CNN (Tiny Constraint)	Deployment-context mismatch (trained on clean audio, deployed in noisy real-world environments)	False positive rate by acoustic environment and speaker accent	Voice assistants perform worse on accented speech; wake-word triggers on TV audio in some languages

Systems insight: Fairness evaluation must match each archetype’s failure mode. Vision models require stratified demographic accuracy; large language models (LLMs) need toxicity and stereotype probes; recommenders need exposure audits; TinyML needs acoustic-environment tests. The lighthouse keyword spotting (KWS) system introduced in chapter 2 as the Tiny Constraint lighthouse faces exactly this challenge for its DS-CNN, a depthwise-separable convolutional neural network (CNN): trained on clean studio audio, it must perform equitably across accents, background noise levels, and speaker demographics in production homes (a governance challenge addressed in section 15.5).

¹⁰ | **Fairness metric incompatibility:** The measured disparities in this worked example show how one set of confusion matrices can violate demographic parity, equal opportunity, and equalized odds. A separate impossibility result shows that, with unequal group base rates and imperfect prediction, several desirable criteria—including calibration-style predictive parity and error-rate balance—cannot generally be satisfied together (Chouldechova 2017). In those settings, optimizing one criterion can degrade another. A system designer must therefore make the trade-off explicit rather than assuming all guarantees can be achieved at once.

¹¹ | **Equalized odds:** Formalized by Hardt et al. (2016), requiring that both TPR and FPR be equal across protected groups; the weaker “equal opportunity” relaxes this to TPR alone. Equalized-odds post-processing may require randomized, group-dependent decisions rather than deterministic thresholds alone. It also requires protected-group information at decision time and must be evaluated under applicable law; for example, US credit regulation generally restricts using protected bases to assess creditworthiness.

15.3.3.1 Worked example: Fairness analysis in loan approval

A loan approval model reports 85 percent accuracy on the majority group and 82.5 percent overall accuracy across the evaluated applicants—numbers that may satisfy a coarse aggregate dashboard. Evaluating the model separately across demographic cohorts exposes how aggregate metrics conceal severe disparities. Table 15.6 details the outcomes for the majority cohort (Group A):

Table 15.6: Confusion Matrix for Group A (Majority): In this constructed scenario, loan approval outcomes for 10,000 applicants from the majority demographic group. The 90 percent true positive rate (4,500 approved of 5,000 qualified) and 20 percent false positive rate establish the baseline for fairness comparison.

	Approved (pred)	Rejected (pred)
Repaid (actual)	4,500 (TP)	500 (FN)
Defaulted (actual)	1,000 (FP)	4,000 (TN)

Table 15.6 establishes the majority-group baseline against which three standard criteria for equitable treatment are evaluated, each capturing a distinct operational objective.¹⁰

Demographic parity requires equal positive selection rates across groups regardless of underlying qualification ($P(\hat{Y} = 1 \mid A = a) = P(\hat{Y} = 1 \mid A = b)$). Group A receives approval at $(4,500 + 1,000)/10,000 = 55\%$, whereas Group B receives approval at $(600 + 200)/2,000 = 40\%$. The resulting 15 percentage-point disparity indicates unequal selection outcomes between cohorts.

Equal opportunity requires equal true positive rates among qualified applicants ($P(\hat{Y} = 1 \mid Y = 1, A = a) = P(\hat{Y} = 1 \mid Y = 1, A = b)$). Group A achieves a TPR of $4,500/(4,500 + 500) = 90\%$, meaning 90 percent of creditworthy applicants secure loans. Group B achieves only $600/(600 + 400) = 60\%$. This 30 percentage-point disparity reveals that qualified applicants in Group B suffer disproportionate rejection relative to equally qualified peers in Group A.

Equalized odds tightens the constraint, demanding parity in both true positive and false positive rates ($P(\hat{Y} = 1 \mid Y = y, A = a) = P(\hat{Y} = 1 \mid Y = y, A = b)$ for $y \in \{0, 1\}$).¹¹ Group A yields an FPR of

$1,000 / (1,000 + 4,000) = 20\%$, matching Group B at $200 / (200 + 800) = 20\%$. Although false positive rates align, the substantial true positive rate deficit violates equalized odds. Because unequal group base rates and imperfect prediction render multiple fairness criteria mutually incompatible, the operational guarantee must be selected and defended explicitly rather than deduced from aggregate accuracy.

Evaluating the same model on the minority demographic cohort in table 15.7 exposes how these criteria diverge under unequal error distributions:

Table 15.7: Confusion Matrix for Group B (Minority): In the same constructed scenario, loan approval outcomes for 2,000 applicants from the minority demographic group. The 60 percent true positive rate (600 approved of 1,000 qualified) reveals a 30 percentage-point disparity compared with Group A; the confusion matrices alone do not identify its cause.

	Approved (pred)	Rejected (pred)
Repaid (actual)	600 (TP)	400 (FN)
Defaulted (actual)	200 (FP)	800 (TN)

These metrics disagree because they encode different policy choices about which error rates matter most. In this scenario, the model rejects qualified applicants from Group B at a much higher rate (40 percent false negative rate vs. 10 percent) while maintaining similar false positive rates. These confusion matrices establish an operational disparity, not whether it arose from classification thresholds, labeling bias, omitted features, sampling skew, or historical discrimination.

Where collection and use of protected attributes are lawful and justified, production systems can automate these calculations and trigger review when disparities exceed predefined thresholds. Listing 15.1 makes the failure-handling contract explicit: undefined group rates are reported as insufficient data rather than compared against a threshold.

Listing 15.1: Automated Fairness Monitoring: The core pattern computes per-group metrics from confusion matrices and alerts when disparities exceed application-specific thresholds. Production use requires sufficient data and lawful, justified handling of group attributes.

```
def compute_fairness_metrics(confusion_matrix):
    tp, fp, tn, fn = (
        confusion_matrix[k] for k in ["TP", "FP", "TN", "FN"]
    )
    total = tp + fp + tn + fn
    return {
        # Demographic parity
        "approval_rate": (tp + fp) / total if total else None,
        # Equal opportunity
        "tpr": tp / (tp + fn) if (tp + fn) else None,
        # Equalized odds (with TPR)
        "fpr": fp / (fp + tn) if (fp + tn) else None,
    }

# Compare groups and flag disparities exceeding threshold
for metric in ["approval_rate", "tpr", "fpr"]:
    if metrics_a[metric] is None or metrics_b[metric] is None:
        report_insufficient_data(metric)
        continue
    disparity = abs(metrics_a[metric] - metrics_b[metric])
    # e.g., 0.05 for high-stakes applications
    if disparity > FAIRNESS_THRESHOLD:
        trigger_alert(metric, disparity)
```

Automated monitoring achieves what manual auditing cannot at scale: continuous tracking of fairness metrics with immediate alerting when disparities emerge. The 30 percentage-point TPR disparity far exceeds this scenario's 5-percentage-point release threshold, indicating the model

requires fairness intervention before deployment. Table 15.8 summarizes the computed metrics and resulting disparities across both cohorts.

Table 15.8: Fairness Metrics Summary: In this constructed loan scenario, approval rate and true positive rate differ across groups, while false positive rates match; the table establishes disparity but not its cause.

Metric	Group A	Group B	Disparity
Approval Rate	55%	40%	15 pp
True Positive Rate	90%	60%	30 pp
False Positive Rate	20%	20%	0 pp

Shared classification thresholds applied to differing score distributions illustrate how aggregate performance masks severe subgroup divergence (Barocas and Selbst 2016), as diagrammed in figure 15.3. Aggregate accuracy alone cannot determine whether those outcomes are equitable for either cohort.

12 | Reweighting: A pre-processing technique rooted in importance sampling from statistics: samples from an underrepresented group receive higher loss weights during training, amplifying their influence on gradient updates without removing any data. Kamiran and Calders (2012) showed that appropriately chosen weights can reduce disparate impact from training data. The systems trade-off is application-specific: reweighting shifts the loss landscape and may reduce performance on other slices, so the cost must be evaluated against the Pareto frontier for the application.

13 | Adversarial debiasing: The key differentiating property is representation pressure: the adversary discourages the primary model from encoding protected-attribute information, which can reduce protected-attribute leakage and help satisfy selected fairness criteria under the evaluated distribution (B. H. Zhang et al. 2018); it does not provide a general fairness guarantee under arbitrary deployment shift; guarantees depend on assumptions about invariance, labels, causal structure, and the type of shift. Postprocessing methods such as threshold adjustment may also be appropriate under different assumptions but must be reevaluated when deployment demographics or label processes change. The cost is additional training, hyperparameter tuning, and slice-level validation rather than a universal percentage overhead.

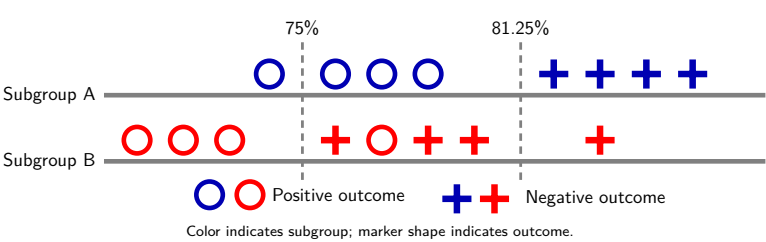


Figure 15.3: Threshold Effects on Subgroup Outcomes: Two candidate classification thresholds intersect different score distributions for Subgroups A and B. Circles mark positive outcomes (loan repayment), plus markers mark negative outcomes (default), and color distinguishes subgroup; each boundary approves the markers to its left. Each dashed line carries the overall accuracy that boundary achieves, not a score cutoff. Moving to the higher-accuracy boundary lifts overall accuracy from 75 percent to 81.25 percent, but the whole gain falls in Subgroup A, which becomes perfectly separated, while Subgroup B goes from one misclassification to three.

Mitigating these disparities requires intervening across the D·A·M pipeline, each stage imposing specific systems trade-offs:

Postprocessing via threshold adjustment decouples the decision boundary across groups, lowering the approval cutoff for Group B to equalize TPR. The operational trade-off is an increase in false positives within that cohort, admitting applicants who may subsequently default.

Preprocessing via sample reweighting scales the loss penalty of Group B instances during training, amplifying gradient signal for underrepresented cohorts without discarding data.¹² The trade-off appears across the broader loss surface, where reweighting can degrade accuracy on majority slices.

In-processing via adversarial debiasing introduces an auxiliary classification head that attempts to recover protected attributes from internal representations, penalizing the encoder when demographic signal leaks.¹³ The resulting representations satisfy selected fairness invariants at the cost of training instability, hyperparameter sensitivity, and additional compute.

Selecting among these interventions requires stakeholder alignment on acceptable trade-offs for the specific deployment domain. Systems engineers make these trade-offs actionable by rendering them explicit and quantifiable.

 Checkpoint 15.2: Fairness criteria

Fairness is not a single metric; it is a constrained design choice.

- **Metric definitions:** can you distinguish demographic parity, equal opportunity, equalized odds, and calibration in terms of which rates must match across groups?
- **Impossibility trade-off:** can you explain (at a high level) why base-rate differences make it impossible to satisfy all fairness criteria simultaneously?

- **Systems interpretation:** given the confusion matrices in table 15.6 and table 15.7, can you identify which disparity matters operationally (TPR vs. FPR vs. approval rate) and what kind of harm it represents?
- **Engineering decision:** for a concrete high-stakes domain (credit, hiring, criminal justice), can you justify which fairness constraint you would prioritize and why?

15.3.3.2 Quantifying the fairness-accuracy trade-off

The impossibility theorem in Kleinberg et al. (2017) proves that mutually exclusive fairness criteria force engineering trade-offs, formalizing the Pareto frontier introduced in figure 15.1. Knowing a trade-off exists, however, is insufficient: systems engineers must quantify the exact utility cost of enforcing a fairness constraint to inform operational decisions. An automated hiring pipeline makes this economic trade-off concrete.

Napkin Math 15.2: The price of fairness

Problem: Stakeholders demand elimination of a 20 percentage-point true positive rate (TPR) disparity in a hiring model. What is the “Price of Fairness” in terms of hiring quality?

Physics: TPRs can be equalized by adjusting the classification threshold (γ_{cls}) for the disadvantaged group.

- **Original state:** Group A (TPR = 90 percent), Group B (TPR = 70 percent). Aggregate Accuracy = 85 percent.
- **Intervention:** Lower $\gamma_{\text{cls}, g=B}$ until $\text{TPR}_{g=B} = 90$ percent.
- **The cost:** Lowering the threshold increases false positives (hiring candidates who do not meet the bar).

Math:

1. Under the scenario’s assumed threshold response, closing the 20 percentage-point TPR gap produces a 15 percentage-point increase in false positives for the disadvantaged group.
2. If the positive base rate is 20 percent, the value of a successful hire is \$100,000, and the cost of a bad hire is \$50,000, both sides of the intervention must be counted:
 - $\Delta \text{Utility} = \Delta \text{TPR} \times \text{Base Rate} \times \text{Hire Value} - \Delta \text{FPR} \times (1 - \text{Base Rate}) \times \text{Bad Hire Cost}$.
 - The added true-positive value is \$4,000 per Group B applicant, while the added false-positive cost is \$6,000, for a net loss of \$2,000.
 - This is 20 percent of Group B’s baseline utility. With Group B at 30 percent of applicants, the population-average loss is \$600 per applicant; an aggregate percentage also requires the other group’s baseline utility.

Systems insight: The “Price of Fairness” in this scenario is a 20 percent within-group utility loss, not a derivable aggregate percentage. The loss is not automatic: when a TPR gap reflects a miscalibrated threshold, closing it can raise net utility. It appears when the baseline threshold is near the utility optimum and marginal admissions skew unqualified. Engineering teams owe stakeholders both the Pareto frontier and the operational assumptions that produced it.

The calculation gives stakeholders a way to choose a point on the fairness-accuracy frontier, but it still does not explain any particular decision. When a loan applicant receives a rejection, stating that “the model’s true positive rate for this demographic group is 60 percent compared to 90 percent for other groups” provides no actionable information. The applicant needs to know *why* the application was rejected and *what* could be changed. These questions require explainability, which is the ability to articulate which input features drove specific predictions.

15.3.4 Explainability requirements

A loan applicant denied credit by an algorithmic system may be entitled under applicable law to reasons or specific adverse-action factors, not merely aggregate statistics. **Explainability**¹⁴ supports this capability: it enables human oversight of automated decisions, supports debugging when problems emerge, and can satisfy applicable requirements for decision transparency.

The level of explainability required varies by application context and regulatory environment. Table 15.9 maps common deployment scenarios to their explainability needs.

Table 15.9: Explainability Considerations by Domain: Requirements vary by decision, actor, jurisdiction, and governing rules. The engineering challenge is matching explanation mechanisms to applicable requirements while managing risks such as gaming.

Application Domain	Explainability Level	Typical Requirements
Credit decisions	Specific reasons for covered actions	Principal reasons may have to be disclosed to the applicant
Medical diagnosis	Decision support	Support clinical review and applicable documentation
Content moderation	Context-dependent	Support notices or appeals where required
Recommendation	Context-dependent	Provide transparency appropriate to the use and governing rules
Fraud detection	Controlled disclosure	Balance applicable notice duties with adversarial-gaming risk

Selecting an explainability strategy requires balancing explanation fidelity, serving latency, and architectural flexibility across three distinct design paradigms:

Post-hoc explanation methods, including SHapley Additive exPlanations (SHAP) and Local Interpretable Model-agnostic Explanations (LIME), generate feature importance scores for individual predictions without modifying model internals.¹⁵ Because they evaluate perturbations around a query point, they decouple explanation from model architecture but introduce serving-path latency overhead and sampling approximations.

Inherently interpretable models enforce transparency through structural constraints, using shallow decision trees, sparse linear models, or generalized additive models where each parameter directly maps to a physical decision boundary. This intrinsic clarity often sacrifices expressive capacity on high-dimensional inputs. Importantly, internal representations such as attention weights do not constitute faithful post-hoc explanations of causal decision logic.

Concept-based explanations translate internal activations into human-interpretable semantic concepts rather than raw input coordinates. By projecting latent vectors onto directional concept vectors, they bridge the gap between low-level sensor features and high-level operational auditing, albeit requiring annotated concept sets to calibrate the projections.

Example 15.2: The hospital shortcut

Scenario: Pneumonia detection vision models trained at Mount Sinai achieved high internal accuracy but failed on external hospital datasets (Zech et al. 2018).

Diagnosis: The neural network learned shortcut features (hospital-specific scanner artifacts and text tags) rather than true biological lung pathology.

Systems lesson: Models exploit the path of least resistance in feature spaces. External validation is essential for detecting shortcut learning; saliency maps can provide supporting evidence but do not establish that the model learned the intended mechanism.

The hospital shortcut shows why interpretability is a systems requirement rather than presentation polish: teams need enough visibility to investigate shortcuts before deployment. Figure 15.4 arranges the resulting trade-offs along a single axis. On the left side, suitably constrained decision trees and linear models can offer direct auditability, although feature design and model size still matter. On the right side, deep neural networks and convolutional architectures can provide greater capacity for complex tasks but resist direct human inspection, motivating post-hoc summaries such as LIME or SHAP that must themselves be validated.

The choice depends on the application’s accountability requirements. Adverse-action laws require accurate, specific reasons for covered credit decisions, but they do not mandate a particular model

¹⁴ | **Explainability and interpretability:** *Interpretability* usually describes how readily a person can understand a model or its behavior, often through a constrained structure such as a short rule list or sparse linear model. *Explainability* is broader and includes post-hoc methods such as LIME and SHAP. Neither property is automatic: a linear model with opaque features can be difficult to interpret, and a feature attribution is not a causal account of a decision. The systems implication is that intrinsic constraints affect model selection, while post-hoc methods add a computation and validation path whose cost depends on the method, model, and serving workflow. GDPR access and automated-decision provisions use the phrase “meaningful information about the logic involved” for covered processing (European Parliament and Council of the European Union 2016), leaving the appropriate technical approach dependent on the decision and governing law.

¹⁵ | **LIME (local interpretable model-agnostic explanations) and SHAP (SHapley additive explanations):** LIME (Ribeiro et al. 2016) fits a local interpretable surrogate around each prediction; SHAP (Lundberg and Lee 2017) adapts Shapley values from cooperative game theory to compute feature contributions under a unified additive framework. Exact Shapley computation requires evaluating a feature’s marginal contribution across all possible feature subsets, scaling exponentially with the feature count ($O(2^M)$); practical production deployments therefore rely on polynomial-time model-specific algorithms (such as TreeSHAP for decision trees) or sampling approximations (such as KernelSHAP) to avoid unacceptable serving latency. The systems trade-off is that explanation fidelity, latency, and implementation complexity must be budgeted explicitly rather than treated as free.

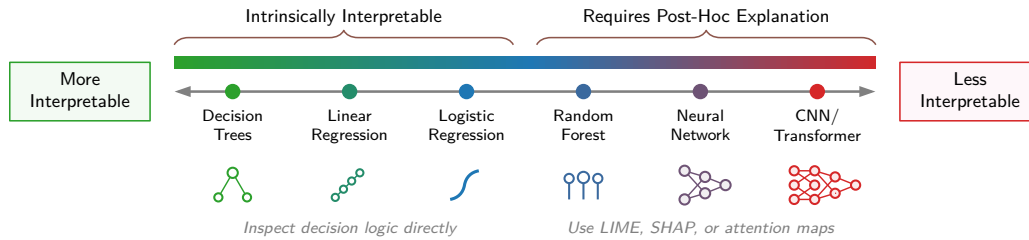


Figure 15.4: Model Interpretability Spectrum: A horizontal continuum groups decision trees, linear regression, and logistic regression as intrinsically interpretable, while random forests, neural networks, and CNNs/transformers require post-hoc explanation. The grouping frames model selection as an auditability trade-off rather than a universal ranking of model quality.

architecture. Other applications may face different transparency, safety, or contestability duties. The spectrum does not imply “simple is always better,” because a highly interpretable model that makes inaccurate predictions may also cause harm. The engineering challenge is selecting a model and explanation process that meet the application’s predictive and accountability requirements.

Some explainability and transparency requirements carry the force of law. The EU AI Act, which entered into force on August 1, 2024 and applies in phases, imposes documentation, transparency, human-oversight, and risk-management obligations for covered high-risk systems (European Parliament and Council of the European Union 2024). Regulation B requires specific principal reasons for covered adverse actions, and its official interpretation states that disclosed reasons must accurately describe factors actually considered or scored (12 C.F.R. § 1002.9; official interpretation). The applicable technical mechanism depends on the system, decision, jurisdiction, and legal obligation.

15.3.5 The regulatory landscape

Regulation changes responsible engineering from a best-practice argument into architecture constraints. Imagine a credit model denies an applicant and the applicant asks why, contests the decision, and later requests access to the data used about her. The system must do more than report an accuracy score. It must produce an explanation tied to the specific decision, preserve the model and data lineage that led to that output, route the dispute to a substantive human review path, retain audit logs, and support deletion or access workflows where data rights apply. Responsible engineering now operates within explicit regulatory frameworks that turn transparency, oversight, and accountability into technical requirements.

Regulation first enters the architecture through risk classification. The EU AI Act establishes a comprehensive framework, classifying AI systems by risk level and mandating requirements accordingly.¹⁶ The Act entered into force on August 1, 2024 and applies in phases: prohibited-practice rules began applying in 2025, while high-risk and other operator obligations phase in by system category and implementation guidance. Article 99 sets maximum fines of EUR 35 million or 7 percent of global turnover for prohibited AI practices, while many other operator obligations are capped at EUR 15 million or 3 percent (European Parliament and Council of the European Union 2024).

For systems engineers, compliance hinges on architectural capabilities rather than fine schedules. Covered high-risk systems¹⁷ must implement risk management, data governance, technical documentation, transparency, human oversight, and accuracy, robustness, and security requirements. A covered credit-decision system therefore needs auditability from inception: model versions, training data provenance, validation evidence, human-oversight design, logging, and postdeployment monitoring must be part of the architecture rather than documents assembled after launch.

Contestability adds a second architectural requirement. GDPR moves the same applicant workflow into data-subject rights. Article 22 grants EU data subjects the right not to be subject to decisions based solely on automated processing that produce legal or similarly significant effects, subject to specified exceptions and safeguards.¹⁸ Article 15(1)(h) separately gives data subjects access to meaningful information about the logic involved in automated decision-making referred to in Article 22. Engineering teams should determine which decisions are covered and design the required explanation and human-review capabilities accordingly. Where a substantive review path

¹⁶ | **EU AI Act (Regulation 2024/1689):** The first comprehensive AI legal framework, defining four risk tiers with penalties that vary by infringement category; prohibited AI-practice violations can reach EUR 35 million or 7 percent of global turnover; many other obligations, including many high-risk operator obligations, are capped at EUR 15 million or 3 percent. The Act has extraterritorial reach: non-EU organizations may need to comply when they place systems on the EU market or when system outputs are used in the EU. Systems engineering implications are concrete: high-risk AI requires logging infrastructure for audit trails, human oversight mechanisms built into the architecture, and CE marking—all capabilities that must be designed in from inception, not retrofitted after deployment.

¹⁷ | **High-risk AI (EU AI Act Annex III):** Annex III enumerates specified use cases in areas including biometrics, critical infrastructure, education, employment, essential services, law enforcement, migration, and justice. Classification depends on intended use and the Article 6 criteria, including exclusions for some systems that do not materially influence decisions or pose significant risk; profiling systems within Annex III remain high-risk. Model architecture alone does not determine classification.

18 | GDPR (General Data Protection Regulation) articles 15 and 22: Article 22 restricts certain solely automated decisions with legal or similarly significant effects and, for specified exceptions, requires safeguards including human intervention, the ability to express a point of view, and the ability to contest the decision; article 15(1)(h) contains the access right to meaningful information about the logic involved in such automated decision-making (European Parliament and Council of the European Union 2016). The European Data Protection Board's guidance emphasizes that required human oversight must be substantive and not merely a "rubber-stamping" exercise (European Data Protection Board 2018). If 0.1 percent of 1M daily decisions are appealed or escalated, the system must handle 1,000 cases/day; this is a workload assumption, not a model-error rate.

is required, it must be operationally staffed and supported by summaries, provenance, and audit tools.

US sectoral law reaches similar capabilities through domain-specific evidence requirements. These regulations are less unified than the EU AI Act, but they can impose related engineering duties. In the credit example, the Equal Credit Opportunity Act (ECOA) and its implementing Regulation B require specific principal reasons for covered adverse actions, and those reasons must accurately describe factors actually considered or scored (12 C.F.R. § 1002.9; official interpretation). If consumer-report information or credit scores influence the decision, the Fair Credit Reporting Act (FCRA) adds notice obligations; if the same scoring machinery is used for housing, the Fair Housing Act (FHA) adds a discrimination-prohibition constraint. The technical consequence is practical rather than abstract: covered systems need evidence and controls that support the applicable reasons, notices, review, and nondiscrimination duties.



Checkpoint 15.3: Ethical deployment

Deployment is where safeguards must become operational.

Safety net

- ☐ **Rollback:** verify that the previous model can be restored within the incident-response target.
- ☐ **Human-in-the-loop:** verify that an operational routing path exists for human review of low-confidence predictions.

Monitoring plan

- ☐ **Silent failure:** specify the subgroup metrics that will expose post-deployment bias.

19 | HIPAA (Health Insurance Portability and Accountability Act): Enacted in 1996, with Privacy Rule and Security Rule requirements establishing standards for protected health information (United States Congress 1996; U.S. Department of Health and Human Services 2003, 2005); PHI may be used for training under applicable authorization or another permitted pathway, such as an IRB or Privacy Board waiver; de-identified data and limited data sets with data-use agreements provide additional pathways. Model outputs may remain PHI when they identify or can reasonably identify individuals, and security-rule documentation retention must be reflected in audit and evidence design. Civil money penalties are tiered and inflation-adjusted by regulation (U.S. Department of Health and Human Services 2026).

Healthcare regulations, including the Health Insurance Portability and Accountability Act (HIPAA)¹⁹ and Food and Drug Administration (FDA) guidance, impose the same pattern with different artifacts: protected-health-information controls, validation records, audit logs, and incident response. Employment systems likewise require evidence that automated screening does not reproduce discriminatory hiring practices. Across domains, the task is to translate each obligation into a concrete capability: explanation, human review, lineage, access control, deletion, monitoring, or incident response. The deployment checkpoint is therefore not a US-sectoral checklist; it is the common production contract implied by the regulatory landscape.

The engineering response to these regulatory requirements is proactive architectural design. Teams that build documentation, monitoring, explainability, and human oversight into systems from inception demonstrate compliance efficiently. Teams that must retrofit these capabilities face expensive pipeline redesigns or deployment blocks. Yet regulatory readiness covers only the planned path; even well-designed systems can fail in production, making incident-response preparation essential.

15.3.6 Monitoring and incident response

Zillow reported a \$304 million²⁰ Q3 2021 Homes-segment inventory write-down after buying homes at prices above revised estimates of future selling prices (Zillow Group 2021). A systems diagnosis interprets the failure as a combination of forecasting uncertainty, distribution shift, operational capacity limits, and insufficient circuit breakers. Operational safety requires establishing incident response and monitoring before a model takes production traffic, rather than diagnosing failures after capital or social harm accumulates. Table 15.10 adapts the incident severity classification from section 14.5.4.1 to responsible deployment, where detection must surface fairness violations and demographic-slice degradation alongside service outages. The five components follow the standard incident-response lifecycle, but the operational safeguard lies in the predeployment verification column. The requirements column specifies what each component must do; the verification column defines what must be demonstrated before launch: an alert threshold tested against historical replay, an automated rollback path exercised under load, and an on-call rotation with documented

escalation paths. A requirement without a verified control is an intention rather than a safeguard, making this verification gate the prerequisite for deployment.

Table 15.10: Incident Response Framework: Systematic preparation for ML system failures requires five distinct components. Detection identifies anomalies through specialized monitoring; assessment evaluates scope using severity classifications; mitigation reduces harm through tested rollback procedures; communication notifies stakeholders through preapproved channels; remediation implements permanent fixes through root cause analysis. Each component requires both operational requirements and predeployment verification.

Component	Requirements	Predeployment Verification
Detection	Monitoring systems that identify anomalies, degraded performance, and fairness violations	Alert thresholds tested, on-call rotation established, escalation paths documented
Assessment	Procedures for evaluating incident scope and severity	Severity classification defined, impact assessment templates prepared
Mitigation	Technical capabilities to reduce harm while investigation proceeds	Rollback procedures tested, fallback systems operational, kill switches functional
Communication	Protocols for stakeholder notification	Contact lists current, message templates prepared, approval chains defined
Remediation	Processes for permanent fixes and system improvements	Root cause analysis procedures, change management integration

In traditional software operations, monitoring focuses on binary service failures: process crashes, out-of-memory errors, network timeouts, and HTTP 5xx responses. In contrast, ML systems accumulate technical debt that manifests as silent degradation (Sculley et al. 2015). An inference service can maintain sub-millisecond p99 latency with zero process exceptions while serving predictions computed over corrupted input schemas or uncalibrated weights. Furthermore, feedback loops can reinforce early erroneous predictions, amplifying small initial shifts into cascading errors across the user population. Incident response planning must account for these silent, ML-specific failure modes by deploying continuous telemetry that detects statistical drift and fairness violations before prediction quality deteriorates. The monitoring infrastructure from chapter 14 provides the foundation for this operational discipline, extending traditional system metrics to inspect data distributions, model behavior, and downstream outcomes.

Responsible production monitoring unifies five operational signals into a continuous telemetry surface:

Performance stability tracking detects gradual predictive drift that eludes conventional infrastructure health checks. While service-level alarms trigger on latency spikes or process crashes, predictive quality can decay incrementally over weeks as real-world distributions evolve, degrading decisions without throwing runtime exceptions.

Subgroup parity monitoring evaluates error distributions across demographic and operational slices over time. Aggregate accuracy metrics mask localized regressions; tracking false-negative and false-positive rates across partitioned cohorts flags emerging disparities before skewed predictions compound into systemic harm.

Input distribution monitoring evaluates statistical divergence across raw feature streams before model inference executes. Because ground-truth labels often arrive with substantial latency, calculating divergence metrics against training baselines catches upstream covariate shift, schema corruption, and sensor failures before corrupted inputs produce invalid predictions.

Outcome monitoring audits downstream decision consequences against logged predictions as ground truth materializes. Comparing observed events—such as loan repayment or clinical intervention success—against predicted probabilities verifies that score calibration remains stable and identifies feedback loops where automated decisions distort subsequent environment states.

User feedback channels capture qualitative complaints, contesting actions, and appeals. Because statistical aggregation over millions of queries inherently smooths away low-frequency anomalies, user-initiated dispute telemetry surfaces tail failure modes and unmeasured harms that evade automated metric thresholds.

Together, these dimensions connect model-level metrics, data-layer shifts, real-world outcomes, and human reports into one monitoring surface.

20 | **Zillow’s D·A·M (data · algorithm · machine) failure:** Zillow’s 2021 write-down is a useful systems case because the documented business failure combined forecast uncertainty with operational execution (Zillow Group 2021). A D·A·M diagnosis interprets the data axis as the mismatch between historical home-sale data and pandemic-era price volatility, the algorithm axis as the difficulty of pricing homes with reliable uncertainty estimates, and the machine axis as an automated iBuying pipeline that needed stronger capacity limits and circuit breakers. This is an engineering interpretation of Zillow’s public disclosure, not a claim that Zillow identified one root technical cause.

Telemetry infrastructure provides protection only when coupled with disciplined operational response. Automated dashboards require defined review cadences, clear on-call ownership, and explicit escalation procedures to ensure emerging anomalies trigger corrective engineering action rather than passive observation.

Equitable performance across demographic cohorts addresses only one dimension of responsible engineering. Operational expenditure and physical energy consumption constitute an equally critical constraint. Every training run, inference request, and monitoring pipeline draws electrical power that converts to carbon emissions and operating expenditure. An ML pipeline can exhibit demographic parity while consuming orders of magnitude more compute than the task warrants—imposing an unmonitored environmental footprint and inflating operating costs. Responsible engineering must govern both whom the system serves and the physical resources consumed in serving them.

15.4 Environmental and Cost Awareness

In 2019, researchers estimated that development-scale training and architecture search for a large Natural Language Processing (NLP) model could emit as much carbon as five cars over their entire lifetimes (Strubell et al. 2019). Later analysis showed that the most quoted architecture-search estimate was highly sensitive to proxy-task, hardware, data-center efficiency, and grid carbon-intensity assumptions (Patterson et al. 2021). The correction sharpened the responsible-engineering point rather than weakening it: training runs consume megawatt-hours of electricity, inference at scale multiplies per-request inefficiencies into measurable environmental impact, and resource-intensive models exclude organizations that lack large compute budgets. The optimization techniques developed in chapter 10, chapter 11, and chapter 12 therefore serve double duty as instruments of responsible engineering, connecting computational efficiency to environmental sustainability, economic accessibility, and long-term scalability.

15.4.1 Efficiency as responsibility

Training a single large language model consumes thousands of GPU hours and energy measured in megawatt-hours. Much of this expense, however, is not intrinsic to the learning task but represents *accidental complexity*: training from scratch when fine-tuning would suffice, deploying larger architectures than tasks require, and running hyperparameter searches that explore redundant configurations. Computational cost depends on engineering choices as well as model physics. Green AI treats that efficiency as a primary metric rather than an afterthought.²¹

At the hardware level, environmental impact scales with total electrical energy draw across the machine lifecycle, mediated by accelerator utilization, data-center cooling overhead (power usage effectiveness), and local electrical grid carbon intensity. Within the accelerator package, energy dissipation is dominated by data movement across the memory hierarchy rather than arithmetic operations: transferring operands from off-chip high-bandwidth memory (HBM) or DRAM consumes orders of magnitude more energy than executing a multiply-accumulate operation in local SRAM or register files. Algorithmic and runtime optimizations reduce environmental burden only when they reduce measured watt-hours for a verified quality target under production load, curbing this physical dissipation at the silicon level.

Physical resource demands also dictate deployment accessibility. Overparameterized models that require multi-accelerator nodes interconnected by high-bandwidth fabrics (such as NVLink) to host their memory footprints restrict operational control to centralized cloud facilities. Compressing models so their working sets fit within commodity DRAM or the unified memory of edge processors expands the deployment envelope. This local execution eliminates reliance on persistent, high-bandwidth network connectivity and recurring cloud API expenditures, enabling low-latency, privacy-preserving inference directly on client devices where data originates.

At production scale, where serving systems process millions of requests daily, per-query inefficiencies compound rapidly across accelerator fleets. Under the iron law of ML systems ($T_{\text{exec}} = D_{\text{vol}}/\text{BW} + O/(R_{\text{peak}} \cdot \eta_{\text{hw}}) + L_{\text{lat}}$), per-query execution latency dictates the request throughput that an individual accelerator can sustain before violating service-level latency objectives. Reducing execution time allows a smaller provisioned cluster to support the same aggregate traffic. Crucially, these operational savings translate into reduced energy consumption and infrastructure costs only when capacity planning downsizes the active fleet. If cluster capacity remains static, latency improvements simply increase accelerator idle time, leaving hardware drawing baseline static leakage power without performing useful work.

Optimization techniques achieve these goals by attacking specific terms in the execution and energy equations across the Data, Algorithm, and Machine layers. Quantization reduces numerical precision from 16-bit floating-point representations to INT8 or FP4, reducing memory traffic D_{vol} across the bus, increasing arithmetic intensity, and routing computation to high-throughput tensor units with lower switching energy. Pruning eliminates inactive weights and structured channels, reducing the total operation count O and memory storage footprint. Knowledge distillation transfers the learned representations of an overparameterized teacher into a compact student architecture, shrinking both operation count and memory footprint simultaneously. At the machine layer, hardware acceleration and kernel fusion eliminate off-chip memory round trips, keeping intermediate activation tensors in on-chip SRAM to maximize hardware efficiency η_{hw} .

²¹ | **Green AI:** Schwartz et al. (2020) contrasted “Red AI” (performance at any cost) with “Green AI” (efficiency as primary metric). The compute-growth anchor comes from AI and Compute’s 2012–2018 trend analysis, which reported a $300,000\times$ increase in compute used in the largest AI training runs (Amodei and Hernandez 2018b). The Green AI proposal—reporting FLOPs alongside accuracy for every published result—reframes efficiency from an engineering preference into a scientific reporting obligation, making the resource cost of marginal accuracy gains visible and comparable across research groups.

Responsible engineering treats these optimizations as architectural design constraints established before training and deployment, rather than post-hoc remedies applied to an oversized model. System requirements specify explicit upper bounds on latency, memory capacity, and energy consumption alongside target predictive quality. The engineering task is finding the most compact architecture and execution strategy that satisfies those constraints.

15.4.2 Efficiency engineering in practice

Translating efficiency requirements into practice begins by establishing bounded machine envelopes—thermal design power, battery capacity, and peak latency—before choosing or training an architecture. Rather than deploying an oversized network and relying on post-hoc compression, responsible systems engineering treats hardware limits as primary architectural design constraints. Model selection prioritizes the most compact architecture that satisfies predictive quality, followed by targeted optimizations such as quantization or operator fusion to minimize memory traffic and energy consumption.

Edge deployment scenarios make these physical limits immediate: passive thermal dissipation and battery capacity enforce boundaries that cannot be negotiated away. Consider an illustrative wearable device with a 500 mW power budget that must sustain continuous inference for 24 hours on a compact battery. Table 15.11 compares assumed constraints across four deployment contexts, from smartphones with 5 W budgets to IoT sensors operating at 100 mW.

Table 15.11: Illustrative Edge Deployment Constraints: The power and latency values are scenario assumptions for comparing deployment contexts; actual budgets depend on hardware, workload, and product requirements.

Deployment Context	Power Budget	Latency Requirement	Typical Use Cases
Smartphone	5 W	100 ms	Photo enhancement, voice assistants
IoT Sensor	100 mW	1 second	Anomaly detection, environmental monitoring
Embedded Camera	1 W	30 FPS (33 ms)	Real-time object detection, surveillance
Wearable Device	500 mW	500 ms	Health monitoring, activity recognition

Evaluating candidate architectures against these operating envelopes reveals whether an algorithm fits the target machine without exceeding its thermal dissipation or latency limits. Table 15.12 profiles four vision models across parameter counts, active inference power, and latency against the smartphone and IoT budgets defined in table 15.11.

Table 15.12: Model Efficiency Comparison: Illustrative model profiles under the stated deployment assumptions. Model selection must account for accuracy, latency, power, memory, runtime support, and lifecycle impact; parameter count alone does not determine cost or environmental impact.

Model	Parameters	Inference Power	Latency	Fits Smartphone?	Fits IoT?
MobileNetV2	3.5M	1.2 W	40 ms	Yes	No
EfficientNet-B0	5.3M	1.8 W	65 ms	Yes	No
ResNet-50	25.6M	4.5 W	180 ms	No	No
TinyML Model	200K	50 mW	200 ms	No	Yes

For the wearable budget in table 15.11, the TinyML model leaves a 10× power margin, while MobileNetV2 exceeds the same power budget by 2.4× before accounting for sustained thermals. Fitting the device envelope is necessary, but per-inference energy compounds into aggregate emissions and operational cost once deployed at scale. Techniques that lower measured energy per inference reduce environmental impact only when they avoid shifting compute costs upstream into training or unmeasured preprocessing, while cloud and cluster financial savings depend directly on provisioning and hardware utilization.

15.4.3 Total cost of ownership

A team spends \$3,200 training a recommendation model and celebrates the modest cost. Six months later, they discover they are spending \$500,000 per year serving it. The surprise illustrates how total cost of ownership²² can be dominated by recurring inference at high traffic. Other systems may be dominated by training, data, staffing, or idle capacity, so measurement determines where optimization should focus.

Consider a concrete example of a recommendation system serving 10M users daily. Training costs appear considerable: data preparation consumes 100 GPU-hours at approximately \$4/hour (\$400), hyperparameter search across multiple configurations requires 500 GPU-hours (\$2,000), and the final training run uses 200 GPU-hours (\$800). Total training cost reaches approximately \$3,200.

Inference costs dominate in this scenario. With 10M users each receiving 20 recommendations per day, the system serves 200M inferences daily. Treating 10 milliseconds per inference as unbatched, non-overlapped dedicated GPU service demand yields approximately 23.1 GPUs running continuously. At \$2.50/GPU-hour, the scenario's annual GPU cost reaches \$506,944.

Over a three-year operational period, quarterly retraining produces total training costs of approximately \$38,400, while inference costs over the same period total \$1.5M. The resulting 40:1 ratio is specific to these assumptions, but it dictates where optimization effort yields the highest return: inference latency and serving efficiency.

Per-query optimization becomes important when serving billions of requests. Shaving a fifth off the per-query accelerator service demand can reduce required hardware when other workload assumptions remain fixed. Hardware selection among CPUs, GPUs, and Tensor Processing Units changes costs and carbon footprint in workload- and deployment-dependent ways. Model compression through quantization and pruning can reduce high-volume inference costs when it lowers measured service demand without unacceptable quality loss.

Total cost of ownership (TCO) encompasses additional dimensions beyond computation. Operational costs include monitoring, maintenance, retraining, and incident response, all of which scale with system complexity and the rate of distribution shift in the application domain. Opportunity costs reflect that resources consumed by ML systems cannot be used for other purposes. Wasteful resource consumption in one project constrains what other projects can attempt.

Engineers should evaluate return on investment (ROI): whether the value an ML system delivers justifies its resource consumption. A recommendation system that increases engagement by 1 percent might not justify millions of dollars in computational costs, while a medical diagnosis system that saves lives does. Explicit trade-offs enable responsible resource allocation.²³

15.4.3.1 TCO calculation methodology

Quantifying operational carbon impact requires measured or estimated energy use, facility overhead, and applicable grid carbon intensity. Engineers can estimate three-year total cost of ownership using a structured approach that separates training, inference, and operational costs into distinct accounting ledgers. Training is typically a periodic capital expense, inference recurs continuously with every user query, and operations accumulate through telemetry, retraining cycles, and incident response. The ledger methodology formalizes this accounting across training, serving, and operational infrastructure.



Napkin Math 15.3: The carbon cost of compute

Problem: The TCO ledgers must convert “compute hours” into “kg CO₂eq”. What carbon factor does one GPU-hour carry under the scenario baseline?

Variables:

- **Power:** 400 W per GPU (scenario baseline).
- **Intensity:** 0.4 kg/kWh CO₂eq (rounded grid baseline).

Math: Equation 15.2 captures the standard conversion:

$$\text{Carbon} = \text{Energy (kWh)} \times \text{Carbon Intensity (kg/kWh)} \quad (15.2)$$

²² | **TCO (total cost of ownership):** ML TCO includes labeling, monitoring, retraining, remediation, energy, audits, and compliance. At sufficient traffic, recurring inference can dominate a one-time training bill; utilization and update cadence determine the balance.



At production scale, recurring inference can dominate lifetime cost.

²³ | **ML ROI (return on investment):** Deployment-to-training cost ratios vary with traffic, utilization, hardware, update cadence, staffing, and incident burden. The appropriate model choice depends on measured lifecycle cost and task value rather than a fixed ratio.

Applying the baseline assumptions:

$$(0.4 \text{ kW} \times 1 \text{ hour}) \times 0.4 \text{ kg/kWh} = 0.16 \text{ kg CO}_2 \text{ eq per GPU-hour}$$

Systems insight: This conversion factor lets the ledgers track “Carbon Cost” alongside “Dollar Cost”, making emissions a first-class engineering metric across the downstream TCO tables.

Training costs Training costs include both initial development and ongoing retraining. Table 15.13 breaks down these costs, showing how quarterly retraining cycles accumulate over a three-year operational period.

Table 15.13: Training Cost Calculation: Training costs accumulate through initial development (\$3,200 per cycle) and quarterly retraining over a three-year operational period. Data preparation, hyperparameter search, and final training consume GPU hours at \$4/hour, totaling \$38,400 across 12 training cycles. Despite appearing substantial, training represents only 1.9 percent of total cost of ownership.

Cost Component	Calculation	Financial Cost	Carbon (kg CO ₂)
Initial data preparation	hours × rate	100 GPU-hr × \$4 = \$400	16 kg
Hyperparameter search	experiments × cost/experiment	50 × \$40 = \$2,000	80 kg
Final training	hours × rate	200 GPU-hr × \$4 = \$800	32 kg
Subtotal per training cycle		\$3,200	128 kg
Retraining frequency	cycles/year × years	4/year × 3 years = 12	same multiplier (12 cycles)
Total training cost	subtotal × cycles	\$38,400	1,536 kg

Inference costs Table 15.14 walks the conversion chain that turns traffic into cost: daily queries become GPU-seconds, GPU-seconds become GPU-hours, and GPU-hours convert into both dollars and carbon. Carbon attaches only once the workload is expressed in GPU-hours, so the query and GPU-second rows leave the carbon column blank by design rather than omitting data. Following the chain to the bottom row supplies the inference term for the total-cost comparison in table 15.16; dominance follows only when that term is compared with training and operations.

Table 15.14: Illustrative Inference Cost Calculation: In this unbatched, non-overlapped service model, 200M daily queries at 10 ms of dedicated accelerator demand require 556 GPU-hr daily, totaling \$507K annually and \$1.52M over three years. Real utilization, batching, and concurrency change this conversion.

Cost Component	Calculation	Financial Cost	Carbon (kg CO ₂)
Daily queries	users × queries/user	10M × 20 = 200M	-
GPU-seconds/day	queries × service demand	200M × 0.01 s = 2M sec	-
GPU-hours/day	seconds ÷ SEC_PER_HOUR	556 GPU-hr	88.9 kg
Annual GPU cost	hours × 365 × rate	556 × 365 × \$2.50 = \$507K	32,444.4 kg
3-year inference cost	annual × 3	\$1.52M	97,333.3 kg

Operational costs Operational costs encompass infrastructure, personnel, and incident response. ML systems generate operational burdens that traditional software does not: “incident response” frequently means debugging silent failures (data drift, feature corruption, or distribution shifts) rather than binary service outages, and “monitoring infrastructure” must continuously track statistical anomalies in model predictions across demographic slices, not merely service availability. Table 15.15 itemizes these ongoing expenses, which often surprise teams focused primarily on compute costs.

Table 15.15: Operational Cost Calculation: Illustrative operational costs include monitoring infrastructure (\$50K/year), on-call engineering at 0.5 FTE (\$100K/year), and incident response reserves (\$20K/year). The \$510K three-year total represents 24.6 percent of this scenario's TCO. Actual staffing and incident costs depend on system scale, risk, organizational structure, and support model.

Cost Component	Annual Estimate	3-Year Total
Monitoring infrastructure	\$50K	\$150K
On-call engineering (0.5 FTE)	\$100K	\$300K
Incident response (estimated)	\$20K	\$60K
Total operational		\$510K

The stark breakdown in table 15.16 answers where the money goes: inference at 73.5 percent, operations at 24.6 percent, and training at only 1.9 percent.

Table 15.16: Total Cost of Ownership Summary: In this illustrative service-demand scenario, the inference-to-training cost ratio is 40:1. A 20 percent reduction in per-query accelerator demand saves about \$304K and 19 t of modeled accelerator CO₂ under the same assumptions. The carbon subtotal excludes operations, facility overhead, and embodied impacts.

Category	3-Year Cost	Percentage	Modeled Accelerator Carbon
Training	\$38K	1.9%	1.5 t
Inference	\$1.52M	73.5%	97.3 t
Operations	\$510K	24.6%	Not estimated
Total TCO	\$2.07M	100%	~98.9 t modeled subtotal

Those proportions turn efficiency from a tuning preference into a responsibility check.



Checkpoint 15.4: Efficiency as responsibility

Total cost of ownership reveals where responsible optimization has the most leverage.

- ☐ **Inference dominance:** under this scenario's assumptions, can you explain why a 20 percent accelerator-service-demand reduction delivers more savings than a 50 percent training time reduction?
- ☐ **Carbon accounting:** can you convert GPU-hours into kg CO₂eq using power draw and carbon intensity, and explain how workload design, hardware, scheduling, and region affect the result?
- ☐ **Sufficiency test:** for a given ML system, can you justify that the model size is appropriate for the task, or identify where a simpler model would deliver comparable accuracy at a fraction of the cost?

15.4.3.2 Environmental impact

The TCO analysis in the preceding section captures costs that appear on invoices, but computational resources carry costs that no invoice reflects. Operational emissions depend on measured energy, facility power usage effectiveness (PUE), and regional grid carbon intensity (gCO₂e/kWh); embodied manufacturing impacts require separate accounting. A first-order operational estimate is $\text{Energy}_{\text{hardware}} \times \text{PUE} \times \text{Carbon Intensity}$. Whereas operational carbon measures emissions from the electricity consumed during active model execution, embodied carbon encompasses the greenhouse gases emitted across the hardware supply chain—silicon wafer fabrication, packaging, transport, and facility construction—before a chip executes its first cycle. As data center grids increasingly transition to renewable energy, embodied carbon often accounts for over half of an accelerator's lifetime carbon footprint, meaning that prematurely replacing older servers for slight operational efficiency gains can paradoxically increase net lifetime emissions. Per-request energy is measured in joules per inference, whereas joules per FLOP characterizes hardware work efficiency. An optimization that reduces TCO may not reduce energy or emissions if it changes utilization, hardware, or

Car-year

This training scenario exceeds 100 passenger-car years of emissions.

workload placement. Data-center electricity use makes workload design, hardware, cloud region, and timing part of responsible engineering (Henderson et al. 2020). The magnitude becomes clearer in a scale calculation for training a large foundation model.

Efficiency optimization and environmental responsibility align when measured lifecycle energy and emissions fall for the required workload and quality target. More granular carbon accounting methodologies build on this foundation: lifecycle assessment tracks impacts across the system's full life, scope 1/2/3 emissions separate direct emissions, purchased electricity, and supply-chain or use-phase emissions, and carbon-aware scheduling shifts work toward lower-carbon times or regions.

The same physical quantities that govern performance also affect responsibility. Data movement and computation contribute to chip-level energy, while data-center emissions additionally depend on utilization, facility overhead, grid intensity, and embodied impacts. Pareto analysis applies to both accuracy-fairness and accuracy-latency objectives, but reweighting an objective can improve multiple metrics when the current solution is dominated. *Responsible engineering extends the constrained optimization problem this book has been teaching to objectives that include societal impact alongside throughput and latency.*

Napkin Math 15.4: The carbon cost of scale

Problem: A foundation model is being trained at the scale of GPT-3, consuming 1,287 MWh of electricity (Patterson et al. 2021). What is the operational carbon impact under the stated grid-intensity assumption?

Math:

1. **Energy consumption:** 1,287 MWh = 1,287,000 kWh.
2. **Carbon intensity:** This scenario uses ≈ 429 g/kWh CO₂ (0.429 kg/kWh) (Patterson et al. 2021).
3. **Total emissions:** 1,287,000 kWh $\times$ 0.429 kg/kWh = 552,123 kg CO₂ (552 t).
4. **Comparison:** The scenario assumes 4.6 t CO₂ per passenger-car year.

Systems insight: Under this training-energy scenario, the operational emissions equal the assumed annual emissions of 120 passenger cars. A 1 percent reduction in total training energy under the same grid mix would remove the equivalent of about 1.2 cars taken off the road for a year from the calculation.

The checklists, fairness metrics, explainability mechanisms, and efficiency analyses developed in previous sections tell engineering teams *what to measure* and *how to act*. A natural follow-up concern is *what* infrastructure records those answers, supports audits, and routes violations to the appropriate intervention rather than relying on human memory. The answer lies in **data governance**—the engineering discipline that transforms policy intentions into enforceable technical controls.

15.5 Data Governance and Compliance

Recommendation and targeting models ingest user telemetry continuously at cluster scale, making a governance failure in the data pipeline an immediate failure in the ML system's ingestion and validation infrastructure. Governance is the enforcement layer that makes accountability operational: fairness metrics, model cards, and impact assessments have practical force only if the system can prove what data it ingested, which principals accessed it, which legal basis authorized processing, and whether applicable deletion or contestability requests were executed.

The Meta Ireland enforcement decisions make the governance constraint concrete: a system must establish a lawful basis for processing as well as protect the underlying data. In January 2023, the Irish Data Protection Commission issued separate EUR 210M and EUR 180M fines (totaling EUR 390M) against Meta Ireland. The decisions concerned unlawful reliance on contractual necessity for personalized advertising together with transparency and fairness violations, not a security breach (Data Protection Commission 2023).

The storage architectures examined in chapter 4 serve as governance enforcement mechanisms that determine which principals access data, how queries are tracked, and whether pipelines satisfy statutory requirements. Every architectural decision—from ingestion strategies through extract, transform, load (ETL) pipelines to persistent storage layout—carries governance implications that surface during regulatory audits, privacy investigations, or model recalls. Data governance translates abstract policy into concrete systems engineering through authenticated access controls on training partitions, immutable audit infrastructure that logs dataset access, privacy-preserving transformations during feature extraction, and deterministic lineage systems that trace training snapshots to deployed model checkpoints. Because those obligations bind technical architecture rather than organizational paperwork, they establish an engineering principle in their own right, local to this chapter rather than the part-opening principles that frame this book.



Principle: Compliance as engineering constraint

Invariant: Data governance obligations are engineering requirements, not optional policy overlays.

Implication: Systems that process regulated data must map applicable duties to technical and organizational controls. Access control, erasure, contestability, audit, and lineage may be required or useful depending on the jurisdiction, data, actor, and use case. The General Data Protection Regulation (GDPR), California Consumer Privacy Act (CCPA), Brazil's General Data Protection Law (LGPD, *Lei Geral de Proteção de Dados*), and China's Personal Information Protection Law (PIPL) differ in scope and duties, so no single control checklist establishes compliance across them.

Data governance operationalizes security, privacy, compliance, and lineage as mutually dependent engineering constraints across four functional domains.

Security infrastructure protects raw features, training partitions, and model artifacts through mutual authentication, role-based authorization, encryption at rest and in transit, and hardware-rooted key management across the ingestion lifecycle.

Privacy mechanisms regulate information exposure even among authenticated consumers. Techniques such as differential privacy, k-anonymity, and automated feature redaction protect individual record subjects while preserving statistical signal for downstream model training.

Compliance frameworks translate jurisdiction-specific statutes (such as GDPR, CCPA, and LGPD) into hard architectural constraints that govern pipeline retention windows, consent verification, and cross-border data replication.

Lineage and audit systems construct the cryptographic trails that render the entire data lifecycle inspectable. Without deterministic lineage linking deployed weights back through training snapshots to raw ingestion logs, security claims and regulatory assertions remain unverifiable.

The KWS system introduced in chapter 2 as the Tiny Constraint lighthouse illustrates how the fairness risks identified in table 15.5 intensify at the governance level. A deployed voice assistant captures ambient acoustic frames continuously in private spaces, maintains enrolled speaker profiles, and periodically syncs with cloud pipelines that train on population-wide audio corpora. These operational capabilities create governance obligations across consent management, data minimization, access auditing, and deletion rights.

In the KWS architecture, these four domains map directly to hardware and pipeline enforcement points. Security protects data across the edge-to-cloud boundary: hardware keystores on edge microcontrollers protect stored speaker templates, mutual TLS authenticates telemetry in transit, and role-based access controls gate access to centralized training audio in cloud storage. Privacy governs memory buffers and telemetry: circular SRAM buffers overwrite ambient audio continuously until an on-device keyword triggers detection, ensuring non-trigger speech never commits to persistent flash memory or network egress, while differential privacy limits information leakage during fleet-wide acoustic model retraining. Compliance automates legal obligations: when a user revokes consent or invokes a right to erasure, automated deletion pipelines purge the user's voice profiles and acoustic training partitions across all cloud tiers and cold backups. Lineage and audit establish cryptographic provenance: content hashes and signed manifests link each deployed depthwise-

separable convolutional neural network (DS-CNN) binary back through specific training splits and acoustic augmentation seeds to the original raw recordings.

Figure 15.5 shows the broader operating model that supports these domains, coordinating organizational policies, data catalogs, sourcing protocols, quality metrics, and shared schema definitions with technical controls. Technical mechanisms and organizational processes must function as mutually reinforcing subsystems: encrypted storage without identity-mapped access controls remains vulnerable to unauthorized exfiltration, and compliant storage without verifiable audit trails cannot survive regulatory inspection. Within the D·A·M taxonomy, data governance forms the operational control plane of the **Data** axis, enforcing the lawful provenance, privacy bounds, and lifecycle integrity of the data that feeds the **Algorithm** and executes across the **Machine**.

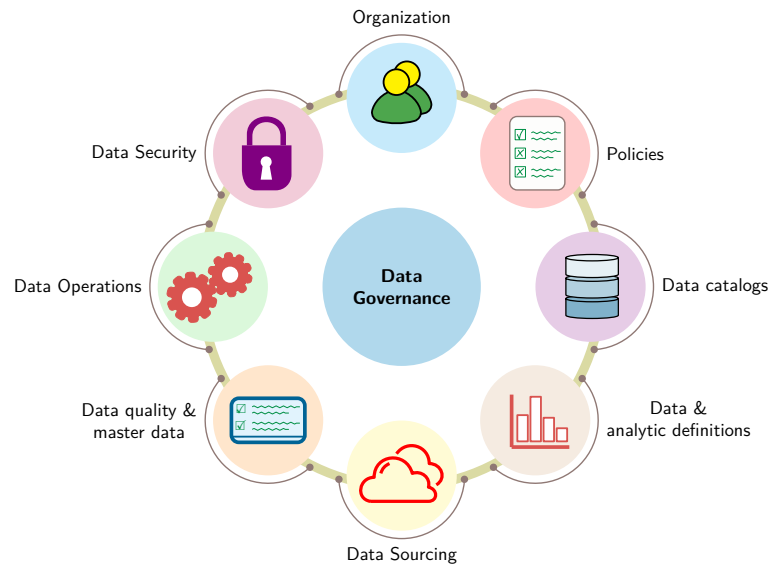


Figure 15.5: Data Governance Framework: Eight operating elements surround Data Governance as a central hub: policies, organization, security, operations, data quality and master data, sourcing, catalogs, and shared analytic definitions. The circular arrangement presents governance as a coordinated operating model rather than a single control, spanning organizational functions such as catalogs, sourcing, and analytic definitions alongside the four technical domains (security, privacy, compliance, and lineage). Adapted from standard enterprise data governance frameworks.

15.5.1 Security and access control architecture

Consider a data scientist querying a feature store for training data. She can read aggregated voice features but cannot access the raw audio recordings from which they were derived. The serving pipeline can read online features for inference but cannot write to the training dataset. Neither can modify source data. The separation is intentional: it reflects a layered security architecture where governance requirements translate into enforceable technical controls at each pipeline stage. Feature stores enforce role-based access control (RBAC) by mapping organizational policies to authenticated identities and database permissions. These controls operate across distinct storage tiers: object storage enforces bucket-level access policies, data warehouses implement column-level security and dynamic masking for sensitive fields, and feature stores maintain isolated read and write paths that separate low-latency online inference key-value lookups from high-throughput offline batch training extracts.

Access control mechanisms remain incomplete without encryption, which protects data at rest and in transit but does not prevent misuse after decryption or authorized access. Training corpora stored in data lakes rely on server-side envelope encryption with keys managed through dedicated key management services (such as AWS KMS or Google Cloud KMS). Feature stores enforce symmetric encryption (such as AES-256) for tables at rest and TLS 1.3 for transport across services. Lighthouse KWS edge devices combine transport encryption for payload confidentiality with asymmetric code

signing to verify model-update integrity; both controls require secure key management and update authorization. To protect data in use during training and inference, systems rely on confidential computing through hardware-based secure enclaves (trusted execution environments such as AMD SEV-SNP, Intel TDX, or confidential GPUs). While host CPU enclaves isolate and encrypt system DRAM pages against untrusted hypervisors, multi-accelerator systems must also secure the physical interconnect. Confidential accelerator architectures extend the hardware root of trust across the PCIe bus using PCIe IDE (Integrity and Data Encryption) and encrypt HBM with on-die cryptographic engines. Plaintext weights, activations, and sensitive user inputs remain confined within the processor silicon, ensuring that neither privileged host operating systems, hypervisors, nor cloud operators can inspect or tamper with execution memory.

Serialized weights, ONNX (Open Neural Network Exchange) exports, and checkpoints require the same protection as training data. Model weights represent proprietary intellectual property and expose critical vulnerability surfaces across the Data, Algorithm, and Machine (D-A-M) layers. At the machine level, deserializing unverified checkpoint files (such as Python `pickle` archives) introduces arbitrary code execution vulnerabilities on training nodes and inference workers. Production pipelines eliminate this vulnerability by using restricted serialization formats (such as Safetensors), write-protected promotion gates, and cryptographic signatures (such as Ed25519) that verify artifact integrity before serving infrastructure loads parameters into accelerator memory. At the data and algorithm levels, pipelines must defend against adversarial threats throughout the lifecycle. In data poisoning attacks, adversarially crafted samples inserted into the training corpus induce targeted misbehavior or backdoors at inference time. If an adversary accesses prediction APIs or raw weights, membership inference probes whether specific patient or user records were present in the training set (Shokri et al. 2017), while model-extraction attacks steal model functionality through systematic query probing (Tramèr et al. 2016). At inference time, evasion attacks craft worst-case, human-imperceptible perturbations (δ) bounded by an ℓ_p -norm ball ($\|\delta\|_p \leq \epsilon$) to input features, triggering high-confidence misclassifications without altering weights or training data. Defending serving infrastructure against evasion requires robust optimization during training—such as adversarial training that computes worst-case perturbations via input gradients at each step, substantially increasing the backward-pass arithmetic intensity and FLOP requirement—or runtime defenses such as input sanitization and randomized smoothing.

Access control and cryptographic enclaves establish *who* can reach data and *how* it is protected across storage, transit, and execution. Controlling access is only half the problem: even authorized users can compromise individual privacy if the data itself is insufficiently protected.

15.5.2 Technical privacy protection methods

A data scientist with legitimate access to training data does not need, and should not see, individual user records when aggregate statistics suffice. Privacy-preserving techniques²⁴ address this gap by determining what information systems expose even to authorized users, adding a second layer of protection beyond access control. Differential privacy provides a formal mathematical bound on the maximum influence that any single record can exert on an exported output distribution. In machine learning systems, differential privacy is enforced during training via Differentially Private Stochastic Gradient Descent (DP-SGD). Standard backpropagation aggregates gradients across an entire batch of size B inside optimized matrix multiplication kernels, never materializing per-sample derivatives. DP-SGD breaks this execution model: the system must bound the sensitivity of each record by clipping the ℓ_2 norm of every individual sample gradient to a threshold C before summation, followed by calibrated noise injection sampled from a Gaussian distribution $\mathcal{N}(0, \sigma^2 C^2 \mathbf{I})$. On hardware accelerators, per-sample clipping prevents standard batch-level tensor contractions. Computing per-sample gradients individually via micro-batches collapses arithmetic intensity and saturates kernel launch queues, while vectorizing per-sample gradients across the batch expands intermediate activation and gradient memory footprint by a factor of B , quickly exhausting accelerator HBM. Furthermore, every minibatch step leaks a measurable quantity of information, requiring a privacy accountant to track the cumulative epsilon budget across composed training steps and halt optimization before the privacy budget is depleted.

Empirical validation methods like membership inference attacks can supplement these formal checks by measuring whether an external observer can determine if a specific record was present

²⁴ | **Privacy-preserving techniques:** K-anonymity (Sweeney 2002) ensures each record is indistinguishable from at least $k - 1$ others with respect to chosen quasi-identifiers, l -diversity adds attribute variety within equivalence classes (Machanavajjhala et al. 2007), and t -closeness bounds distribution distance (Li et al. 2007). These syntactic guarantees address different threats from differential privacy and do not by themselves protect against all ML-specific leakage: a model trained on de-identified data can still memorize examples or reveal membership signal under some conditions (Shokri et al. 2017). Differential privacy instead bounds the influence of one record under a specified mechanism and privacy budget, offering a semantic guarantee designed to remain meaningful in the presence of side information; the methods are therefore not a simple strength hierarchy, and the appropriate protection depends on the release, threat model, and utility requirements.

25 | Membership inference

attack: The attack uses model behavior to infer whether a record appeared in training; overfitting can increase the signal (Shokri et al. 2017). In practice, higher confidence on training samples can help distinguish members from non-members. Its measured success depends on model access, data distribution, and defense assumptions. A successful empirical attack reveals practical leakage, but it does not alone prove violation of a stated (ϵ, δ) bound. Formal differential privacy comes from the mechanism and its accounting.

in the training set, although an empirical attack cannot prove that an (ϵ, δ) bound holds mathematically.²⁵ Membership inference must be distinguished from model inversion attacks, where an adversary leverages continuous model confidence outputs to directly reconstruct the underlying private feature values or facial likeness of training subjects (Fredrikson et al. 2015). While membership inference answers a binary presence question (“was this individual in the training set?”), inversion exploits output probability distributions to synthesize the sensitive data itself. Defending against inversion requires masking or quantizing continuous confidence scores at the prediction serving API, truncating output distributions to top- k classes, and bounding parameter memorization through differential privacy during training.

While differential privacy bounds information leakage from model outputs, collecting continuous raw sensor streams creates an immediate vulnerability if raw data leaves the capture device. Always-listening edge architectures, such as the lighthouse KWS system, operate under severe privacy constraints because continuous ambient microphone sampling must coexist with strict minimization and retention boundaries. To eliminate transmission exposure, the system confines acoustic feature extraction and wake-word verification entirely to local low-power microcontrollers or digital signal processors (DSPs). Incoming audio samples stream through a small volatile circular buffer in on-chip SRAM, where older frames are continuously overwritten by newly sampled pulse-code modulation (PCM) data. Raw audio never leaves local volatile memory or persists to non-volatile flash storage unless an on-device acoustic model verifies an explicit trigger keyword.

Confining raw audio to local edge memory enforces data minimization, but prevents centralized training pipelines from collecting diverse acoustic variations to improve model accuracy over time. Federated learning resolves this tension by executing model training directly on client devices, aggregating model weight updates rather than pooling raw user audio.²⁶ In each training round, selected edge devices compute local gradients on newly triggered utterances and transmit parameter deltas to a central coordinator. This architectural shift trades data centralization risks for severe physical systems bottlenecks. Uplink wireless bandwidth severely limits communication throughput when transmitting millions of parameters per round over cellular or Wi-Fi channels. Battery-powered client devices frequently drop offline mid-round as stragglers due to thermal limits or intermittent connectivity, and non-IID (non-identically distributed) data distributions across user devices cause local gradients to diverge, threatening model convergence. Furthermore, raw gradient updates remain susceptible to gradient inversion and reconstruction attacks (Zhu et al. 2019), requiring production federated pipelines to combine gradient compression and sparsification with secure multi-party aggregation and local differential privacy noise injection.

When triggered audio recordings or aggregated model checkpoints must be retained on centralized infrastructure for verification or downstream training, privacy protection shifts to automated lifecycle management. Automated retention and cryptographic erasure pipelines enforce strict time-to-live policies across primary blob stores, feature caches, and distributed training partitions. Verifiable cryptographic erasure protocols ensure that when data reaches its retention deadline, encryption keys are destroyed and storage blocks are overwritten, preventing deleted audio streams from lingering in database replicas or cold backup snapshots.

15.5.3 Architecting for regulatory compliance

When a European user invokes an applicable right to erasure under GDPR, the voice assistant must determine which personal data and downstream artifacts are in scope and execute the appropriate workflow within the regulation’s response deadlines (European Parliament and Council of the European Union 2016). Compliance requirements transform legal obligations into system architecture constraints that shape pipeline design, storage choices, and operational procedures. GDPR’s data minimization principle requires limiting collection and retention to what is necessary for stated purposes. Article 15 access rights cover personal data undergoing processing and specified information about that processing, not every artifact merely associated with a user.

Voice assistants operating globally face overlapping regulatory regimes because requirements vary by jurisdiction and apply differently based on user age and data sensitivity. GDPR cross-border transfer rules permit transfers through adequacy decisions, appropriate safeguards, or limited derogations rather than imposing categorical data localization. These rules can still drive regional storage, replication, and processing choices. Data cards (Pushkarna et al. 2022) record provenance,

26 | Federated learning:

McMahan et al. (2017) introduced Federated Averaging (FedAvg), in which each device trains locally and shares model updates rather than raw records. This architecture can limit routine transfer of raw data, but it is not a privacy guarantee: model updates can leak training information through reconstruction attacks (Zhu et al. 2019). Federated learning may therefore be combined with secure aggregation, differential privacy, and retention controls in a defense-in-depth design whose guarantees depend on the complete protocol.

Open Images Extended - More Inclusively Annotated People (MIAP) Dataset Download Related Publication This dataset was created for fairness research and fairness evaluations in person detection. This dataset contains 100,000 images sampled from Open Images V6 with additional annotations added. Annotations include the image coordinates of bounding boxes for each visible person. Each box is annotated with attributes for perceived gender presentation and age range presentation. It can be used in conjunction with Open Images V6.		
Authorship PUBLISHER(S) Google LLC INDUSTRY TYPE Corporate - Tech DATASET AUTHORS Candice Schumann, Google, 2021 Susanna Ricco, Google, 2021 Utsav Prabhu, Google, 2021 Vittorio Ferrari, Google, 2021 Caroline Pantofaru, Google, 2021 PUBLISHER(S) Google LLC FUNDING TYPE Private Funding DATASET CONTACT open-images-extended@google.com		
Motivations DATASET PURPOSE(S) Research Purposes Machine Learning Training, testing, and validation KEY APPLICATION(S) Machine Learning Object Recognition Machine Learning Fairness PRIMARY MOTIVATION(S) Provide more complete ground-truth for bounding boxes around people. Provide a standard fairness evaluation set for the broader fairness community. PROBLEM SPACE This dataset was created for fairness research and fairness evaluation with respect to person detection. See accompanying article INTENDED AND/OR SUITABLE USE CASE(S) ML Model Evaluation for: person detection, fairness evaluation ML Model Training for: person detection, Object detection Also: Person detection: Without specifying gender or age presentations Fairness evaluations: Over gender and age presentations Fairness research: Without building gender presentation or age classifiers		
Use of Dataset SAFETY OF USE Conditional Use There are some known unsafe applications. UNSAFE APPLICATION(S) Gender classification Age classification CONJUNCTIONAL USE Safe to use with other datasets KNOWN CONJUNCTIONAL DATASET(S) The data in this dataset can be combined with Open Images V6 UNSAFE USE CASE(S) This dataset should not be used to create gender or age classifiers. The intention of perceived gender and age labels is to capture gender and age presentation as assessed by a third party based on visual cues alone, rather than an individual's self-identified gender or actual age. KNOWN CONJUNCTIONAL USES Analyzing bounding box annotations not annotated under the Open Images V6 procedure.		
METHOD Object Detection SUMMARY A person object detector can be trained using the Object Detection API in TensorFlow. KNOWN CAVEATS If this dataset is used in conjunction with the original Open Images dataset, negative examples of people should only be pulled from images with an explicit negative person image level label. The dataset does not contain any examples not annotated as containing at least one person by the original Open Images annotation procedure. METHOD Fairness Evaluation SUMMARY Fairness evaluations can be run over the splits of gender presentation and age presentation. KNOWN CAVEATS There still exists a gender presentation skew towards unknown and predominantly masculine, as well as an age presentation range skew towards middle.		

Figure 15.6: Data Governance Documentation: The populated Open Images Extended—More Inclusively Annotated People card organizes authorship, motivation, intended and unsafe uses, conjunctional use, and method caveats in one artifact. Recording permitted and prohibited uses makes the card reviewable governance evidence, though it does not establish compliance by itself. Adapted from the Open Images Extended—MIAP Data Card in Pushkarna et al. (2022).

intended uses, and risks as operational metadata, as illustrated in figure 15.6, but a valid card does not by itself make a dataset or model compliant.

15.5.4 Building data lineage infrastructure

Data-card fields become operational checks once they enter the pipeline: provenance, intended use, risk, and retention metadata determine which datasets can train which models and which artifacts must be traced during an audit. Compliance obligations are only as credible as the infrastructure that demonstrates them. When a regulator asks “which training data produced this model?” or a user invokes an applicable right to erasure, the organization must answer with engineering precision, not manual investigation. Data lineage provides this capability, formalizing pipeline relationships as a directed acyclic graph (DAG) of artifact versions and transformation tasks that

27 | Data lineage systems:

Apache Atlas and DataHub capture metadata about data flows from pipeline execution, creating graphs in which nodes are datasets and edges are transformations. GDPR Article 30 requires records of processing activities (European Parliament and Council of the European Union 2016), not automated model-level lineage specifically. Lineage nevertheless helps identify candidate derived artifacts for deletion, restriction, retraining, or legal review when an applicable request arrives.

turns static documentation into queryable infrastructure. Lineage systems such as Apache Atlas and DataHub²⁷ integrate with pipeline orchestrators such as Airflow and Kubeflow to capture relationships automatically. When an Airflow DAG reads audio files from object storage and transforms them into spectrograms, the lineage system records each step and traces every feature tensor back to its source audio file. Lineage tracking also enables downstream impact analysis. When an erasure request arrives, traversing the lineage graph identifies all derived artifacts—cached spectrograms, feature embeddings, and checkpoint weights—that require invalidation, retraining, or legal review.

A production KWS system implements lineage tracking across the data engineering lifecycle. Source audio ingestion creates lineage records linking each audio file to its acquisition method, supporting verification of consent requirements. Processing pipeline execution extends lineage graphs as audio becomes features and embeddings, and each transformation records code versions and hyperparameters. Training jobs create lineage edges from feature collections to model artifacts, recording which data versions trained which model versions. When a voice assistant device downloads a model update, lineage tracking records the deployment, supporting recall if training data is later discovered to have quality or compliance issues. Lineage captures provenance, but accountable operation also requires access history: who touched the data, when, and under which authority.

15.5.5 Audit infrastructure and accountability

Accountability trails record access events to satisfy statutory obligations such as HIPAA and the Sarbanes-Oxley Act (SOX). In production machine learning systems, however, capturing every access event creates an acute write-amplification bottleneck. While primary dataset sizes grow linearly with user base or collection duration, audit volume scales with the product of dataset accesses, service requests, and pipeline stages. Logging full feature payloads and intermediate activation tensors across high-throughput serving pipelines causes retained audit telemetry to grow convexly relative to primary data storage. This write amplification rapidly saturates ingestion bandwidth and secondary storage capacity, forcing systems to decouple audit metadata from underlying payload data.

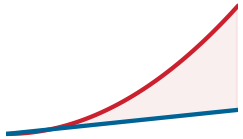
Production keyword spotting architectures resolve this trade-off through a tiered audit topology that reconciles capture fidelity with storage overhead and edge bandwidth limits.

At the edge tier, microcontrollers operating within strict SRAM budgets log local state transitions, wakeup timestamps, and power states to volatile circular ring buffers. To conserve radio energy and uplink bandwidth, devices avoid streaming uncompressed telemetry continuously; they flush compact, batched summaries to central storage over secure transport only during scheduled maintenance windows or charging cycles.

Once telemetry reaches centralized infrastructure, auditing shifts from bandwidth-constrained edge devices to high-throughput feature stores. At this feature tier, offline and online feature stores emit append-only audit events for every read and write transaction. Rather than duplicating high-dimensional feature vectors, these events record requesting service identities, authorized subject pseudonyms, feature group hashes, and read timestamps. This metadata-only logging strategy enables fine-grained access verification while keeping log volume proportional to access transactions rather than payload sizes.

From feature stores, data flows into training pipelines running across accelerator clusters. At this training tier, cluster schedulers record deterministic manifests mapping training jobs directly to immutable data partition hashes, git commit SHAs, and hyperparameter configurations. While these manifests verify pipeline integrity and reproducibility, they reveal an inherent systems boundary: access logs confirm which records were processed, but they cannot demonstrate whether deleted user data persists within learned model weights. Fulfilling an erasure mandate therefore requires triggering automated retraining workflows or provable unlearning verification rather than relying solely on database audit logs.

Regulatory requirements extend audit responsibility from data access to prediction-time inference behavior. Reconstructing why a specific applicant was denied a loan in section 15.3.3.1—or why an edge voice assistant triggered a false wake-word activation—requires auditing the exact operational state at inference time. Without inference-time logging, an audit trail may confirm database access without capturing which model checkpoint, decision threshold, and runtime features produced a



Retained audit events turn accountability into a growing storage workload.

given decision. Archiving raw input tensors for every inference request would impose unsustainable storage write burdens; production serving systems instead record compact, purpose-limited decision manifests. These manifests capture the active model version hash, runtime decision threshold, output confidence scores or reason codes, and cryptographic hashes of input features, transforming per-decision reconstruction into an indexed database query.

Together, the four governance domains—security, privacy, compliance, and audit²⁸—form the Machine-tier enforcement layer of the D-A-M taxonomy. They provide the physical telemetry and boundary enforcement needed to verify algorithmic constraints, track data provenance, and preserve accountability across the model lifecycle. Yet even when systems implement comprehensive governance infrastructure, engineering teams frequently fall into subtle traps—treating audit logs as passive compliance paperwork, assuming aggregate metrics guarantee subgroup fairness, or believing responsible constraints can be retrofitted after deployment.

15.6 Fallacies and Pitfalls

Teams can still fail after assembling assessment frameworks, fairness metrics, explainability mechanisms, efficiency analyses, and governance infrastructure. A team may retrofit fairness after benchmark success, trust aggregate accuracy, or treat compliance evidence as paperwork rather than system behavior, drawing on intuitions from traditional software engineering where bugs are local and testing is deterministic. Recognizing these failure patterns early, before a fallacy shapes a design decision, is far cheaper than discovering it after deployment.

Fallacy: *Responsibility can be addressed after the system achieves technical objectives.*

Teams assume fairness constraints can be retrofitted once models demonstrate strong benchmark performance. In production, early architectural decisions constrain what interventions remain feasible. Amazon’s recruiting tool (see section 15.2.1) illustrates this trap: attempted remediation did not provide confidence that the system would avoid discriminatory recommendations, and Amazon abandoned the project (Dastin 2018). Organizations deferring responsibility may face redesign, deployment with documented risks, or cancellation. Integrating fairness constraints early can be less costly than retrofitting them after data contracts, monitoring, and release gates are fixed.

Pitfall: *Relying on aggregate metrics to assess fairness.*

Engineers assume high overall accuracy indicates the system works well for all users. The Flaw of Averages (section 15.3.3) reveals this intuition fails: aggregate metrics can conceal large subgroup disparities (section 15.2.4). The loan approval analysis in section 15.3.3.1 showed a 30 percentage-point TPR gap, with qualified minority applicants rejected at 4× the majority-group rate. These disparities can persist undetected when standard monitoring tracks only aggregates. Production systems require disaggregated evaluation with application-specific thresholds, such as the 1.25× error-rate ratio or 5 percentage point TPR difference used in this example.

Fallacy: *Removing sensitive attributes from training data eliminates bias.*

Teams remove gender, race, and protected attributes expecting this ensures fairness. Other features can act as proxy variables when they correlate with sensitive characteristics. ZIP codes, purchase patterns, browsing history, college names, and language choices can carry indirect demographic signal. Amazon’s system (see section 15.2.1) penalized terms such as “women’s” and graduates of two all-women’s colleges (Dastin 2018). A population-health study found that correcting a cost-based proxy would increase the share of Black patients receiving additional help from 17.7 percent to 46.5 percent (Obermeyer et al. 2019). Removing protected attributes does not by itself establish fairness.

Pitfall: *Treating documentation as sufficient accountability.*

Teams invest effort in model cards, then consider responsibility requirements satisfied. Documentation provides transparency (section 15.3.2) but not enforcement. A model card specifying “not validated for high-stakes decisions” has no effect when the system is repurposed for loan approvals without technical restrictions. Accountability requires operational integration: monitoring dashboards, documented subgroup-disparity alert thresholds, incident response procedures, and access controls preventing deployment beyond validated use cases.

²⁸ | **Audit trail:** Audit integrity requires controls against unauthorized alteration, but retention and deletion duties vary by record type and law; append-only object storage with retention controls, or cryptographic hash chains, can provide tamper evidence without creating a universal rule that records are never deleted. A large platform may log billions of events daily; HIPAA’s Security Rule also imposes multi-year documentation-retention obligations for required policies and procedures (U.S. Department of Health and Human Services 2005). Retention planning must distinguish records that must be preserved from personal data that must be minimized or deleted.

Fallacy: *Responsible AI is primarily a legal compliance issue.*

Teams treat responsibility as external oversight rather than engineering practice. Engineering decisions made months before legal review constrain the solution space more than any compliance assessment. Architecture selection determines what fairness interventions are feasible, while data pipeline design establishes whether disaggregated evaluation is even possible. As section 15.2.5 establishes, systems designed with responsibility as an engineering objective enable efficient validation; systems where responsibility is added at late-stage review face redesign or deployment with documented risks.

Pitfall: *Measuring the environmental impact of training but not inference.*

Public discourse focuses on training-run carbon, and engineers often follow this framing when assessing environmental responsibility. The illustrative TCO analysis in section 15.4.3 shows why this focus is incomplete: under its unbatched service-demand assumptions, the inference-to-training cost ratio is about 40:1. A model trained periodically but served millions of times daily can have a lifecycle footprint dominated by inference rather than training. For the recommendation system analyzed in table 15.16, training accounts for 1.9 percent of three-year costs while inference accounts for 73.5 percent. In this example, inference emits about 63× as much CO₂ as training. Engineers who optimize training efficiency while ignoring per-query inference demand can leave the larger term in this scenario unexamined. These values are scenario-dependent, but they show why lifecycle accounting must include both terms.

Fallacy: *Model weights are exempt from data governance and deletion requests.*

Teams often assume that once training data has been compiled into model weights, the data is gone and compliance obligations cannot reach the artifact. Models can memorize training data, and membership inference attacks may reveal whether a record appeared in training (Shokri et al. 2017). Whether particular weights constitute personal data or require action after an erasure request is case-specific. Engineering teams should track data-to-model lineage and evaluate deletion, restriction, retraining, machine unlearning, or compensating controls with legal and privacy specialists when a request may affect deployed artifacts (Cao and Yang 2015; Bourtole et al. 2021).

Across these failures, the recurring mistake is to treat responsibility as a document, metric, or late-stage review rather than a system property. Measurable constraints, continuous monitoring, and enforceable governance carry responsibility through the full lifecycle.

15.7 Summary

Responsible engineering is ML systems engineering done completely, not a separate discipline. The chapter traced a path from failure diagnosis through prevention to enforcement, beginning with the responsibility gap (the distance between technical performance and responsible outcomes) and demonstrating how proxy variables, feedback loops, and distribution shift can harm users while conventional metrics remain acceptable. The engineering response includes checklists that systematize predeployment assessment, fairness metrics that make disparities measurable, explanation mechanisms selected for applicable stakeholder and regulatory requirements, and monitoring infrastructure that can surface silent failures and connect signals to response.

Translating responsibility concerns into measurable properties makes them tractable. A justified, application-specific bound on disparity is testable; “be fair” is not. This translation extends beyond fairness: in the chapter’s illustrative TCO scenario, a 20 percent accelerator-service-demand reduction saves \$304K and is modeled to avoid 19 t of CO₂ under the stated assumptions. Documentation becomes model cards with explicit intended use and known limitations. Governance becomes access control, lineage tracking, and audit infrastructure that makes compliance evidence available rather than merely aspirational. Across these cases, abstract ethical obligations become concrete engineering requirements that can be specified, tested, monitored, and enforced.

The responsible engineering practices developed in this chapter are integral components of complete engineering, not external constraints layered onto technical work. Systems that ignore fairness, efficiency, transparency, or governance are technically incomplete. The same rigor applied to latency budgets and memory constraints must extend to fairness criteria, environmental impact, and applicable regulatory requirements. Integrating these considerations from system inception

makes obligations testable, exposes trade-offs earlier, and makes failures easier to diagnose and govern in production.



Key Takeaways: Reliable for whom?

- **Aggregate correctness hides harm:** A model can look accurate in aggregate while one subgroup's error rate is 43.1× another subgroup's rate, as in the Face++ Gender Shades audit. Responsible evaluation therefore starts with disaggregated and intersectional slices, not aggregate accuracy alone.
- **Responsibility becomes testable through thresholds:** “Be fair” is not testable, but bounded disparity, documented intended use, and explainability requirements are. Translating values into measurable constraints lets teams review trade-offs among fairness, accuracy, latency, and cost.
- **Efficiency is a social constraint:** An efficient model can reduce energy and cost and broaden who can deploy it. In the chapter's illustrative scenario, the inference-to-training cost ratio is 40:1, making per-query optimization responsible engineering.
- **Monitoring must watch outcomes:** Bias and privacy failures can continue with green uptime dashboards because harmful predictions look operationally normal. Production monitoring must track subgroup outcomes, data lineage, feedback loops, and incident paths with the same rigor as latency regressions.
- **Governance has to be built in:** Model cards, datasheets, access controls, erasure workflows, human-review paths, and audit trails are technical infrastructure, not optional policy overlays. Regulations such as GDPR require capabilities that should be designed into the pipeline; later remediation may remain possible but can be narrower and costlier once the system is serving decisions.

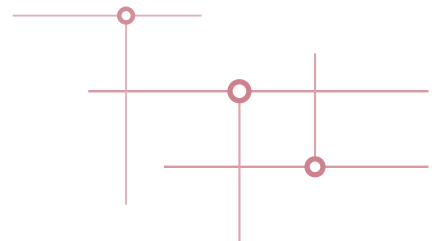
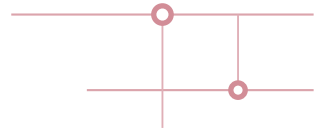
A system that does exactly what it was told is dangerous precisely because the telling is never complete. Every objective a model is given is a specification with gaps, and an optimizer is a machine for finding them. It can reproduce the bias latent in its data, chase the proxy instead of the goal, and call the result success because nothing in the objective said otherwise. Responsible engineering is the discipline of writing back in what the specification left out. Constraints and monitoring reduce the risk that optimization amplifies harms encoded in the data. The constraint is the same kind the rest of the book imposed in latency and memory, except that here it protects people the objective never named, and a model that is fast and accurate while wrong about whom it serves has not failed less than one that crashes, only more quietly.



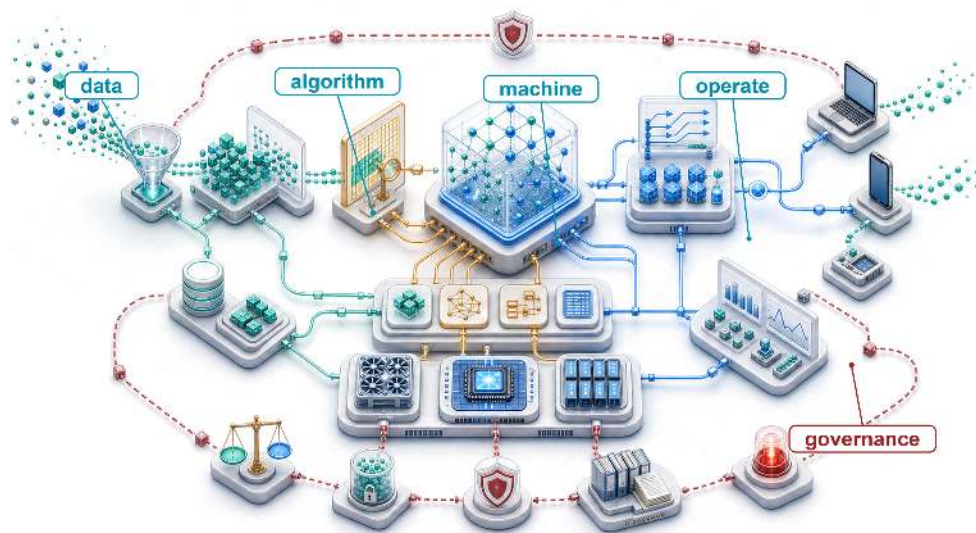
What's Next: From technique to philosophy

The chapter closes a circle that began with the iron law of ML systems (principle 3). The optimizations developed in chapter 10, chapter 11, and chapter 14 were motivated by performance, but they can also support responsibility. Efficiency can reduce energy and emissions, compression can lower deployment resource requirements, and outcome monitoring can surface bias. The engineering techniques overlap, but the responsibility lens changes which outcomes are measured. Chapter 16 connects these pieces into a coherent philosophy of engineering excellence.

SYNTHESIS



Conclusion

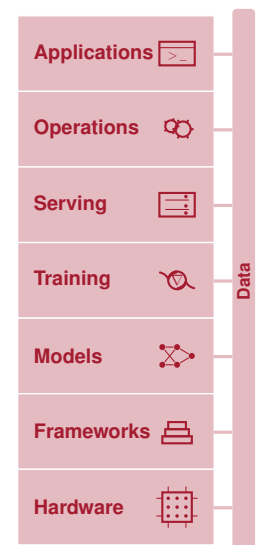


- 16.1 Synthesizing ML Systems
- 16.2 Thirteen Quantitative Principles
- 16.3 Principles in Practice
- 16.4 Future Directions
- 16.5 Journey Forward
- 16.6 Fallacies and Pitfalls
- 16.7 Summary

Purpose

What does mastering the full stack enable that expertise in any single layer cannot?

A single production decision travels through the entire stack. A data pipeline decides which events count as training signal. That signal shapes the architecture, which in turn fixes memory footprint and arithmetic intensity. Those properties constrain hardware choice, quantization strategy, serving latency, drift monitoring, and governance obligations. Mastered individually, each layer is a valuable skill. Mastered together, they provide the ability to reason across boundaries. An engineer who understands only compression can shrink a model, but cannot predict whether the accuracy loss matters for the deployment context. An engineer who understands only serving can optimize latency, but cannot trace a performance regression to a data pipeline change three stages upstream. The discipline of ML systems engineering is the discipline of seeing these connections, where one team's optimization becomes another team's constraint. The principles governing these interactions—constraint propagation, the memory wall, the training-serving inversion, dispatch overhead, communication cost, and operational lifecycles—are not tied to any specific framework, hardware generation, or model family. Technologies change, but the underlying physical constraints and trade-offs endure. In D-A-M terms, what endures is the ability to look at a system that does not yet exist and reason about how its data, algorithm, and machine constraints will interact, where its bottlenecks will emerge, and which design decisions will prove irreversible. Thinking in systems rather than components is what separates an engineer who can build a part from one who can build the whole.





Learning Objectives

- Synthesize core ML systems principles into a framework for reasoning across Data, Algorithm, and Machine constraints
- Trace how data, architecture, compression, hardware, serving, operations, and governance decisions propagate constraints across an ML system
- Apply lighthouse-model reasoning to diagnose bottlenecks across cloud, mobile, edge, recommendation, and TinyML deployments
- Evaluate deployment trade-offs using latency budgets, memory movement, drift, responsibility, and sustainability constraints
- Design a systems engineering posture for emerging contexts before fleet-scale coordination costs dominate

16.1 Synthesizing ML Systems

DEPLOYING an image classification model to a fleet of mobile devices illustrates how cross-layer interactions emerge in production. The architecture team chose depthwise separable convolutions to factorize spatial filtering from channel projection, reducing arithmetic operations. The compression team quantized weights and activations to INT8 to cut memory traffic and fit within on-chip SRAM. The serving team met a p99 latency target of 50 ms. Each team succeeded by its local metric, yet within weeks, accuracy dropped by four percentage points on specific firmware and device cohorts. The cause was an unanticipated interaction between dynamic range clipping in the quantization scheme and an interpolation routine in a firmware-specific image preprocessing path. No component failed in isolation; rather, the data pipeline, architecture, compression scheme, accelerator runtime, and monitoring infrastructure coupled in production.

Responsible engineering governs the entire system lifecycle—specification, testing, runtime monitoring, and subgroup auditing—rather than operating as an external post-processing step. Machine learning systems depart fundamentally from traditional software because statistical performance is physically inseparable from the pipelines and hardware that train, serve, and monitor the model.

At the center of this discipline sits the iron law of ML systems (principle 3). Its three terms—data movement, compute, and overhead—serve as the primary engineering levers for quantitative analysis. Building intelligence requires both statistical algorithms and adherence to the silicon contract (principle 4), the physical agreement between the model and the executing machine. Arithmetic intensity and roofline modeling convert qualitative performance intuitions into exact engineering bounds (Williams et al. 2009).

System capabilities emerge from co-design across the Data, Algorithm, and Machine (D·A·M) triad rather than isolated algorithmic breakthroughs. Machine learning belongs to the systems engineering tradition where robust execution arises from coordinating interdependent components under physical constraints. The transformer architecture introduced an attention-based model family (Vaswani et al. 2017), and large language model systems such as GPT-3 and Llama 2 demonstrate how that family scaled into a central workload for modern ML systems (Brown et al. 2020; Touvron, Martin, et al. 2023). Mathematical design alone does not account for its practical utility. Realizing that utility required co-designing the attention mechanism with distributed communication topologies, memory hierarchy tiling to bypass memory bandwidth bottlenecks, and low-precision execution units.

Integration has concrete consequences. Software engineering often treats the “model” as a weights checkpoint: a 500 MB blob of floating-point numbers. In a production environment, however, the weights are only one component of the true model, and often not the most fragile. A model that produces high benchmark accuracy fails if it ingests corrupted features, and a model that converges cleanly during training fails if it cannot satisfy deployment latency constraints. The *true model* is the sum of the data pipeline that dictates what the learner sees, the training infrastructure that governs optimization, the serving system that mediates inference, and the operational loop that monitors statistical drift. Optimize the system, and the model improves. Neglect the system, and the model degrades. Systems engineering is the physical realization of machine learning: *the system is the model*.

📌 Checkpoint 16.1: Systems thinking

The system's dependencies, feedback loops, and request path make its boundaries testable.

The integration

- ☐ **Dependencies:** can you trace a data-pipeline change through training and serving to identify its latency effect?
- ☐ **Feedback loops:** have you mapped how the model's predictions influence its future training data?

The holism

- ☐ **End-to-end:** can you trace a user request from the UI, through the network, preprocessing, model, postprocessing, and back to the UI?

Tracing a request end to end makes the same point structurally: system boundaries define model capabilities because every layer of the ML stack enforces physical and statistical constraints on the next. The computational substrate begins with data: curation (chapter 4) and selection (chapter 9) bound the information available to the learner, while neural representations (chapter 5) and network architectures (chapter 6) translate statistical priors into concrete tensor graphs. Training systems (chapter 8) and execution frameworks (chapter 7) lower those graphs onto physical hardware, converting mathematical optimization into scheduled kernel dispatches and memory allocations.

Lowered to silicon, the engineering challenge shifts to renegotiating physical boundaries. Model compression (chapter 10) reshapes the Pareto frontier across accuracy, memory footprint, and latency; hardware accelerators (chapter 11) test whether memory bandwidth and arithmetic units can sustain the execution schedule; and rigorous benchmarking (chapter 12) isolates real hardware speedups from measurement artifacts. In production, serving infrastructure (chapter 13) must satisfy tail-latency budgets under bursty traffic, operational monitoring (chapter 14) detects statistical drift, and responsible engineering (chapter 15) audits subgroup fairness where aggregate metrics mask localized degradation. These domains form a continuous, coupled pipeline across data engineering, training systems, deployment infrastructure, runtime operations, and governance.

The governing lesson is constraint coupling: decisions never remain localized. An architecture choice enables a compression strategy, which constrains accelerator lowering, establishes serving latency, and dictates operational monitoring requirements. MobileNetV2's depthwise-separable design targeted efficient mobile vision inference (Sandler et al. 2018), while integer-arithmetic quantization made INT8 deployment a practical inference path (Jacob et al. 2018). That combination enables mobile NPU execution, establishes a 50 ms p99 latency constraint, and necessitates drift tracking across heterogeneous sensor hardware and device firmware. Decisions propagate downstream. An engineer who optimizes a single layer in isolation cannot predict how interventions ripple across the system.

Diagnosing these cascading interactions requires concrete constraint maps. Grounding the analysis in representative workloads reveals how physical bounds dictate architectural choices, runtime schedules, and operational trade-offs across the stack.

Lighthouse models: Constraint propagation

The five lighthouse models (section 1.7) ground constraint propagation in concrete execution profiles:



Architecture choices cascade into compression, serving, drift, and governance.

Each workload exposes a distinct physical bottleneck across the execution spectrum:

- **ResNet-50:** Batch size turns image inference from memory-bound weight movement into compute-bound matrix throughput by reusing the same filter weights across multiple inputs, amortizing memory transfer costs until arithmetic intensity crosses the hardware ridge point.
- **GPT-2/Llama:** Low-batch autoregressive decode stalls on memory bandwidth due to low weight reuse; batching, prefill scheduling, and model parallelism shift the binding constraint.
- **MobileNetV2:** Depthwise separable convolutions and INT8 quantization trade representational capacity for mobile NPU deployment under milliwatt thermal envelopes.
- **DLRM:** Terabyte-scale embedding tables make memory *capacity* a binding constraint alongside memory *bandwidth*, forcing systems to co-design around memory tiers and sparse access patterns.
- **Keyword spotting (KWS)/Wake Vision:** Sub-megabyte microcontroller models with always-on inference under milliwatt power budgets demand zero wasted memory traffic or dispatch overhead.

Together, these five workloads span the deployment spectrum from warehouse-scale data centers to microcontrollers, exposing the bottlenecks that quantitative principles diagnose. Tracing these workloads across architecture, training, optimization, and serving establishes the integrated perspective that distinguishes ML systems engineering from isolated algorithm development.

Table 16.1 traces this progression for MobileNetV2, illustrating how cross-layer constraints converge on a single engineering artifact. Across seven phases—from foundational constraints through architecture, training, compression, acceleration, serving, and runtime operations—each engineering decision dictates the options available downstream.

Table 16.1: The Lighthouse Journey (MobileNetV2): Tracing one model through seven phases of the systems stack shows how decisions in one domain (for example, architecture) propagate constraints and opportunities into downstream domains (for example, hardware acceleration and monitoring).

Journey Phase	System Lens	MobileNetV2 Implementation
Foundations (chapter 1)	The AI Triad	Bounded by machine constraints (Battery/Thermal)
Architecture (chapter 6)	Algorithmic Efficiency	Depthwise Separable Convolutions: 8.7× fewer FLOPs for a representative 3-by-3, 256-output-channel layer and 13.7× fewer operations than ResNet-50 at ImageNet scale
Training (chapter 8)	Throughput vs. Latency	Optimized for single-request mobile latency; data augmentation can improve robustness
Compression (chapter 10)	Navigating the Pareto Frontier	INT8 Quantization: FP32 uses 4× and FP16 uses 2× the per-value storage of INT8, with accuracy revalidated per deployment
Acceleration (chapter 11)	Honoring the Silicon Contract	Mapping kernels to Mobile NPUs (for example, Apple Neural Engine) to maximize hardware utilization
Serving (chapter 13)	Respecting the Latency Budget	p99 < 50 ms constraint; optimizing preprocessing (resize/normalize) to avoid CPU bottlenecks
Operations (chapter 14)	Managing System Entropy	Drift Monitoring: Detecting distribution shifts and tracking cohort-level accuracy across heterogeneous device populations and lighting conditions

Decisions in one row dictate options in the next: architecture choices (depthwise separable convolutions) enable compression strategies (INT8 quantization), which in turn make mobile NPU acceleration feasible. This constraint propagation is universal across machine learning workloads. Formalizing these trade-offs requires quantitative principles—spanning exact physical bounds, empirical decompositions, and assumption-dependent diagnostics.

16.2 Thirteen Quantitative Principles

Reasoning about ML system behavior requires quantitative instruments. These **thirteen quantitative principles** deliberately combine exact mathematical bounds, engineering decompositions, fitted empirical models, policy constraints, and design heuristics. Table 16.2 summarizes all thirteen

principles across four systems phases. Their utility depends on rigorous application within stated assumptions rather than treating every relation as an unvarying physical law.

Table 16.2: Thirteen Quantitative Principles: Each tool appears in the part where its governing constraint or diagnostic use first becomes visible. The collection combines bounds, decompositions, fitted models, policy requirements, and heuristics. Each row is useful only under its stated assumptions; together they provide a shared analytical vocabulary for system design, optimization, and deployment rather than a set of universal invariants.

#	Principle	Part	Core Equation/Statement	What It Predicts
1	Data-as-Code Principle	I: Foundations	Behavior = $f(\text{data, algorithm, code, randomness})$	Data changes behavior; other inputs also matter
2	Data-Gravity Principle	I: Foundations	Move compute toward data when repeated transfer costs exceed placement costs	Depends on volume, reuse, network, and compute mobility
3	Iron Law of ML Systems	II: Build	$T_{\text{seq}} = D_{\text{vol}}/\text{BW} + O/(R_{\text{peak}}\eta_{\text{hw}}) + L_{\text{lat}}$; overlap ranges from max to sum	Stages may add or overlap
4	Silicon Contract	II: Build	$I_{\text{ridge}} = R_{\text{peak}}/\text{BW}$; compare I_{model} with I_{ridge}	Diagnoses bandwidth- versus compute-limited operation
5	Pareto Frontier	III: Optimize	$\nexists c' \neq c : [\forall k M_k(c') \geq M_k(c)] \wedge [\exists j M_j(c') > M_j(c)]$	No distinct configuration dominates a frontier point
6	Arithmetic Intensity Law	III: Optimize	$R_{\text{attain}} \leq \min(R_{\text{peak}}, I \times \text{BW})$	More compute cannot raise a bandwidth ceiling
7	Energy-Movement Invariant	III: Optimize	$E_{\text{total}} = \sum_j N_j E_j$; DRAM/FLOP cost ratio: 173–582 $\times$	Total energy depends on event counts and costs
8	Amdahl's Law	III: Optimize	$\text{Speedup} = \frac{1}{(1-f_{\text{parallel}}) + \frac{f_{\text{parallel}}}{S_{\text{parallel}}}}$	The serial fraction caps all parallelism gains
9	Verification Gap	IV: Deploy	$\Pr_{(X, Y) \sim P_{\text{deploy}}} [d(f(X), Y) \leq \tau] \geq 1 - \epsilon$	Specify distance, tolerance, population, and confidence
10	Statistical Drift Diagnostic	IV: Deploy	$\text{Accuracy}(t) \approx \text{Accuracy}_0 - \lambda \mathcal{D}(P_t \ P_0)$	A local fit; drift need not lower quality
11	Training-Serving Skew Diagnostic	IV: Deploy	$S_{\text{skew}} = \mathbb{E}_{X \sim P_{\text{deploy}}} [d(f_{\text{serve}}(X), f_{\text{train}}(X))]$	Output mismatch signals risk, not accuracy loss
12	Latency Budget Principle	IV: Deploy	$T_q \leq L_{\text{budget}}$	The product SLO selects q and its budget
13	Bias Feedback Model	IV: Deploy	$\Delta_g(k) \approx \Delta_g(0)\alpha_{\text{fb}}^k$ with fitted α_{fb}	Feedback may amplify harm; measure and intervene

The thirteen principles are not independent axioms. They form an integrated framework connected by a single meta-principle: the conservation-of-complexity heuristic.¹ Complexity removed from one interface often reappears in another part of the system, but this is not a physical conservation law and does not imply that every simplification has an equal compensating cost. Its value is diagnostic: after simplifying one component, check where validation, state, coordination, or operational burden changed. The test is whether the principles explain the same lighthouse bottlenecks from data, model, hardware, and deployment perspectives without contradicting one another.

Foundations: Where complexity originates (principles 1–2)

The data-as-code principle (1) and the data-gravity heuristic (principle 2), developed in chapter 4, establish data as a primary logical program and physical anchor. Runtime behavior reflects data, algorithm, and implementation in concert, while compute-to-data placement balances transfer volume, reuse frequency, network bandwidth, and compute mobility. Model behavior and architecture directly inherit constraints from this data substrate.

The lighthouse models illustrate both principles. ResNet-50 and GPT-2 acquire their capabilities from architecture coupled to massive training corpora. DLRM's terabyte-scale embedding tables make memory capacity a primary design constraint, requiring the execution system to be engineered around where the data physically resides. Compute-to-data placement recurs across deployment environments precisely because moving large, low-reuse datasets saturates interconnects.

¹ | **Conservation-of-complexity heuristic:** Tesler's Law, a design aphorism, says that an application's irreducible complexity must be handled somewhere in the interaction among user, application, and platform (Tesler 1984); extending it to all ML-system complexity is an analogy, not a physical law. Quantization may add validation burden, and abstraction may move implementation detail behind an interface, but good design can also remove accidental complexity outright. Large language model application pipelines illustrate a possible shift: simplifying the user-facing interface with shorter or vaguer prompts may move work into system prompts, retrieval, or output verification. Use the heuristic to search for displaced costs, not to assume that an equal compensating cost must exist.

Build: How complexity becomes computation (principles 3–4)

The iron law (principle 3) and the silicon contract (principle 4) govern execution efficiency. The iron law’s three-term decomposition (section 1.7) isolates data movement, computation, and overhead; the silicon contract reveals which term limits performance on a given architecture-hardware pair. Workloads inhabit distinct regimes: batched ResNet-50 inference saturates compute units, low-batch Llama decode stalls on memory bandwidth, DLRM exhausts memory capacity, and MobileNetV2 restructures tensor operations to execute within mobile NPU bounds. As mapped in section A.5.1, diagnosing whether an execution path is compute-bound, bandwidth-bound, or capacity-bound determines which optimizations yield speedup and which waste engineering effort. Training latency drops most when engineers target the binding term directly rather than distributing effort uniformly (chapter 8).

Optimize: How constraints shape trade-offs (principles 5–8)

The four optimization principles form a tightly coupled diagnostic chain. The Pareto frontier (principle 5) identifies nondominated trade-offs after objective directions are normalized: quantization trades precision for memory traffic, pruning trades capacity for speed, and distillation trades training compute for inference efficiency. The arithmetic intensity law (principle 6) diagnoses whether compute or bandwidth sets the ideal ceiling. The energy-movement invariant (principle 7) combines per-event costs with event counts: under reference hardware constants, one DRAM access costs about $173\text{--}582\times$ as much as one FP32/FP16 arithmetic operation, but workload-total dominance depends on how many of each occur. Amdahl’s Law (principle 8) sets the ceiling on any parallelism gain, explaining why data loading and preprocessing can become bottlenecks in highly optimized systems.

MobileNetV2 (chapter 6) balances all four principles simultaneously. Depthwise separable convolutions reshape the Pareto frontier (Sandler et al. 2018). INT8 quantization exploits the arithmetic intensity law by raising arithmetic intensity through reduced memory traffic (Jacob et al. 2018). The resulting energy savings align with the energy-movement invariant, while Amdahl’s Law bounds end-to-end speedup when host-side preprocessing remains unaccelerated. Microcontroller workloads such as KWS compress these trade-offs further, operating under sub-megabyte memories with zero margin for operational waste.

Deploy: How reality defeats assumptions (principles 9–13)

The deployment principles address failures that bench testing cannot rule out: a system can work correctly on the bench yet fail silently in production. The verification gap (principle 9) means statistical verification holds only over a stated deployment population, task distance, tolerance, target error, and finite-sample confidence procedure; testing estimates behavior rather than proving correctness for every future input. The statistical drift diagnostic (principle 10) detects distribution change but does not determine whether accuracy worsens, stays constant, or improves, so a local degradation curve is valid only when fitted against outcomes. The training-serving skew diagnostic (principle 11) is likewise a risk indicator rather than a universal accuracy-loss equation, although preprocessing or numerical differences can still cause quality changes. The latency budget (principle 12) constrains serving at the quantile selected by the product requirement, which may be p95, p99, or another tail measure. Finally, the bias feedback model (principle 13) can represent disparity amplification, but an exponential recurrence requires a measured, approximately constant feedback factor and no effective intervention.

The five deployment principles govern the operational tooling examined in chapter 14: monitoring infrastructure, drift detection, feature stores, and disaggregated subgroup metrics exist to surface silent failures and trigger corrective action. A recommendation system that achieves high offline accuracy still requires parity checks when training-serving skew corrupts feature pipelines (principle 11) and outcome auditing when user behavior shifts seasonally (principle 10). Large language model serving must bound tail latency through continuous batching and speculative decoding (chapter 13) to prevent violating product service-level objectives. Similarly, an automated credit model may satisfy throughput and convergence targets while systematically compounding disparities against underserved groups, requiring disaggregated monitoring to intercept feedback loops (principle 13).

The integrated framework

The thirteen principles are not a checklist to apply sequentially. They form a web of mutual constraints. The conservation-of-complexity heuristic prompts engineers to look for where a local simplification shifts burdens elsewhere.

Quantizing a model from FP16 to INT8 illustrates these mutual constraints in action. This single intervention navigates the Pareto frontier (principle 5), trading numerical precision for memory traffic. The consequences ripple across the stack: quantization alters the model’s silicon contract (principle 4), shifting its operating point on the arithmetic intensity curve (principle 6) and transforming its energy profile (principle 7). In deployment, the latency budget (principle 12) dictates whether the resulting speedup meets the service-level objective (SLO), while the verification gap (principle 9) demands verifying that the lowered serving graph preserves the empirical accuracy measured during compression testing.

That trace does not require every principle to apply at once. It shows how the relevant principles become active as a decision moves from model representation to hardware execution to production validation. Data placement affects where the model can run, Amdahl’s Law limits how much the faster kernel can improve the whole request path, verification bounds the resulting accuracy loss, and outcome monitoring tests whether the validated behavior persists after deployment. The engineer’s task is to trace displaced costs across the stack rather than assume that complexity simply vanishes.

Evaluating a deployment proposal makes this web of constraints concrete.

Checkpoint 16.2: Applying the principles

A colleague proposes quantizing your model from FP32 to INT8 to reduce serving costs.

Trace the principles

- ☐ **Pareto frontier** (principle 5): What accuracy are you trading for the bandwidth gain?
- ☐ **Silicon contract** (principle 4): Does the target hardware efficiently accelerate the required INT8 operators?
- ☐ **Verification gap** (principle 9): Does the quantized serving artifact preserve the behavior accepted during compression testing?
- ☐ **Latency budget** (principle 12): Does the speedup bring you within SLO, or create head-room for batching?

Figure 16.1 maps this cycle of mutual constraint. Four phases—Foundations, Build, Optimize, and Deploy—surround a central hub representing the conservation-of-complexity heuristic. Arrows trace how engineering choices propagate downstream: Build constraints dictate Optimize strategies, while runtime signals—statistical drift, feature skew, and outcome regressions—feed back into Foundations. Managing this cycle requires steering displaced complexity toward layers capable of handling it efficiently.

The Deploy-to-Foundations feedback loop is central to production engineering. Principles 9 through 13 expose signals that trigger targeted remediations: canary rollbacks, fallback heuristics, dataset augmentation, model retraining, or kernel retuning. When anomalies emerge, engineers diagnose root causes rather than triggering uncalibrated rollbacks. This cycle governs single-system design: the objective is not to catalog every future architecture, but to make feedback loops observable before operational failures compound.

Tracing a quantization proposal through four principles is one diagnostic pass; the same habit applies when the bottleneck is not an optimization proposal but the cost of serving a single generated token.

Napkin Math 16.1: The cost of a token

Problem: Applying the iron law (principle 3) and the arithmetic intensity law (principle 6) to serving one token from a 70-billion-parameter model like Llama 2 70B, with its FP16

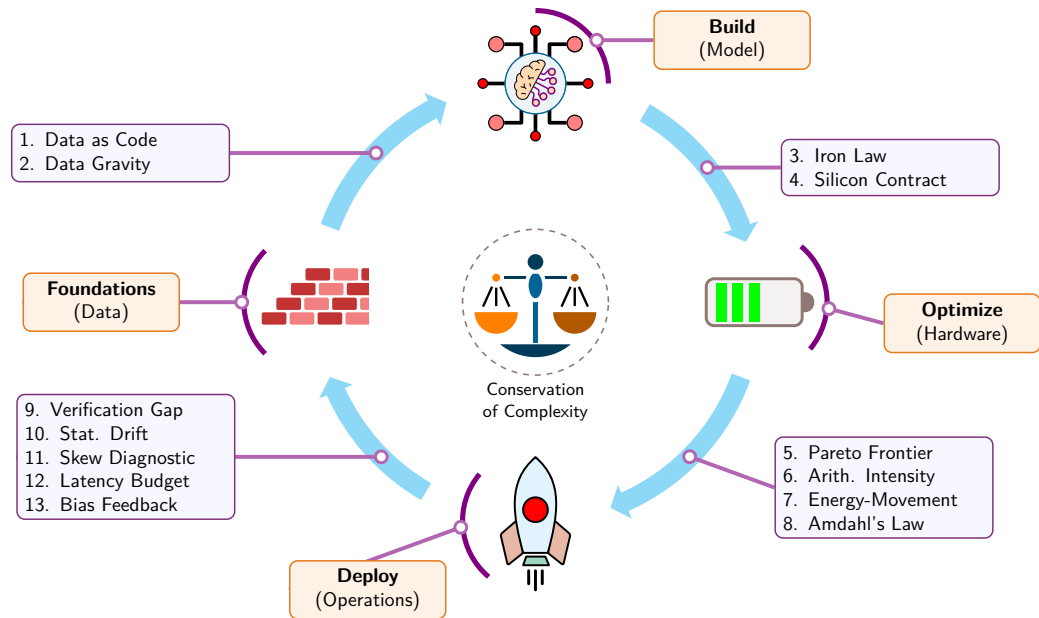


Figure 16.1: The Cycle of ML Systems (13 Principles): A four-phase systems engineering lifecycle organized around the conservation-of-complexity heuristic. The phases connect in a feedback cycle, and the thirteen principles are grouped along the transitions as bounds, decompositions, fitted models, policy requirements, and design heuristics whose assumptions must be checked.

weights sharded evenly across two NVIDIA H100s, which idealized lower bound is larger: memory transfer or peak compute? Section D.3.2 supplies the H100 specifications. This calculation excludes KV-cache traffic, activations, context-dependent attention-over-KV FLOPs, interconnect communication, and dispatch overhead.

Physics:

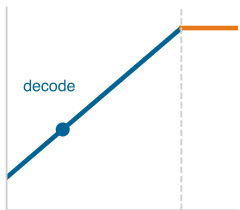
- **Model-weight byte volume moved** (D_{vol}): 70 billion parameters $\times$ 2 bytes (FP16) = 140 GB.
- **Compute** (O): $O \approx 2 \times P = 140$ GFLOP per token, where P is the parameter count.
- **Hardware**: Two H100s with aggregate BW = 6.70 TB/s, $R_{\text{peak}} \approx 1978$ TFLOP/s FP16.

Math:

- **Time to move data**: $T_{\text{mem}} = \frac{140 \text{ GB}}{6700 \text{ GB/s}} \approx 20.9 \text{ ms}$
- **Time to compute**: $T_{\text{comp}} = \frac{140 \times 10^9 \text{ FLOP}}{1978 \times 10^{12} \text{ FLOP/s}} \approx 0.07 \text{ ms}$

Systems insight:

The idealized memory-transfer lower bound T_{mem} is 295.2 $\times$ larger than the peak-compute lower bound T_{comp} . Under this batch-one model, decode sits left of the roofline ridge in the bandwidth-bound regime. Because each 2-byte FP16 weight is fetched from high-bandwidth memory to perform a single multiply-accumulate (2 FLOPs) with the active token activation, the arithmetic intensity is $\frac{2 \text{ FLOPs}}{2 \text{ bytes}} = 1 \text{ FLOP/byte}$ —far below the dual H100 ridge point of $\approx 295 \text{ FLOP/byte}$. The tensor cores spend over 99% of their cycles starved for data. Batching can increase reuse, while quantization can reduce weight traffic. Optimizing only compute execution can reduce realized compute time toward the 0.07 ms lower bound, but it cannot reduce the 20.9 ms memory-transfer lower bound.



In this batch-one lower-bound model, decode sits left of the roofline ridge in the bandwidth-bound regime.

This roofline evaluation demonstrates how physical bounds operate as diagnostic instruments rather than abstract taxonomies. Across three core engineering domains—building foundations,

engineering for scale, and navigating production reality—these bounds, empirical models, and heuristics govern system design and operational stability.

16.3 Principles in Practice

A team that memorizes all thirteen principles but cannot identify their assumptions or apply them to a real deployment decision has learned nothing. The test is the same across the three domains that span the ML lifecycle: building technical foundations, engineering for scale, and navigating production reality. Systems thinking connects what isolated component analysis cannot.

Building technical foundations

The data-as-code principle (1) operationalizes the reality that datasets and data pipelines act as executable programs (chapter 4), echoing Karpathy’s Software 2.0 framing (Karpathy 2017) while acknowledging that serving logic and algorithmic state also dictate behavior. Neural computation primitives (chapter 5) define the arithmetic intensity of matrix multiplications, whose position relative to the hardware ridge point determines whether execution is bandwidth-limited or compute-limited. Framework selection (chapter 7) enforces the silicon contract directly: each runtime engine constrains graph optimization, memory layout, backend target support, and downstream deployment paths. Selecting a framework without evaluating these execution boundaries risks foreclosing high-performance inference paths.

Foundational choices (what data to curate, which computational primitives to rely on, which framework to adopt) propagate into later engineering decisions. That propagation becomes especially visible when a system must scale beyond a single machine, where the iron law’s three terms expand from chip-level quantities to cluster-level constraints.

Engineering for scale

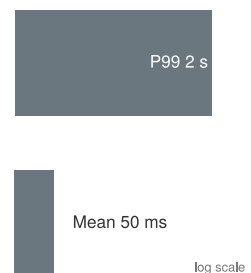
Distributed training systems demonstrate the iron law in action (chapter 8): data parallelism scales compute throughput across accelerators at the cost of collective communication, mixed precision halves memory movement by representing tensors in FP16 or BF16, and activation checkpointing trades recomputation for memory capacity. Each technique targets a specific term of the iron law. Similarly, model compression (chapter 10) explores the Pareto frontier directly: MobileNetV2’s INT8 quantization and DLRM’s embedding pruning trade numerical precision or parameter capacity for bandwidth savings, while the arithmetic intensity law dictates whether that reduction yields realized speedup on target hardware.

Building and optimizing a model, however, is only half the engineering challenge. The other half begins the moment the model leaves the training cluster and enters production, where statistical requirements, fitted diagnostics, and SLO policies govern behavior and where the optimizations that worked on the bench must survive the unpredictability of real-world traffic.

Navigating production reality

The transition from *training* to *inference* often changes optimization objectives: where training maximizes throughput over days, interactive inference may optimize a selected tail-latency quantile in milliseconds. The illustrative scenario uses a 50 ms mean and a 2,000 ms p99, so the selected tail is 40× the mean; another product may specify p95, p99.9, or separate deadlines by request class. Tail-latency analysis shows why mean latency can hide the requests that dominate user experience (Dean and Barroso 2013). In ML serving, tail spikes can arise from garbage collection pauses in Python-based inference frameworks, dynamic batching that forms suboptimally under uneven traffic, or autoregressive generation in which a long prompt or output drives much more work than a typical short response.

Technical performance bounds only part of the operational envelope; system design also encompasses societal impact (chapter 15). Verification mandates continuous auditing for fairness violations alongside throughput: tracking prediction distributions across demographic cohorts, detecting bias amplification over time (principle 13), and alerting on subgroup disparities. Subgroup drift monitoring is essential because accuracy can degrade sharply for underrepresented populations even while aggregate metrics remain steady. Responsible engineering is an operational requirement,



Mean latency hides the tail; the selected tail quantile governs the SLO.

not an external ethical review—a first-class design constraint subject to the same measurement discipline as latency and throughput.

These three domains establish the thirteen principles as pragmatic instruments rather than rigid axioms. The question is *where* their assumptions bind next, as ML systems expand into heterogeneous deployment targets, confront adversarial conditions, and pursue increasingly complex compositions.

16.4 Future Directions

The framework is most useful when it forecasts where constraints may bind next. Three areas put the same physics under increasing pressure: deployment across diverse contexts, robustness under adversarial conditions (Goodfellow et al. 2015), and societal applications whose failures carry public consequences. A fourth horizon, systems that compose multiple models, tools, and verifiers or grow beyond one machine, extends the same lens rather than replacing it.

Applying principles to emerging deployment contexts

Deployment diversity tests whether one quantitative framework can explain systems with contrasting resource regimes. The cloud offers abundant power and centralized hardware, edge and mobile devices operate under latency and battery budgets, and TinyML and embedded systems compress the same design problem into kilobytes and milliwatts. Generative AI is not a fifth deployment environment; it is a workload class that stresses all four.

In the cloud regime, the binding decision is how to turn abundant hardware into useful throughput without letting data movement, capacity, or cost dominate. Dense workloads such as ResNet-50 chase GPU utilization through kernel fusion, mixed precision training, and gradient compression, while DLRM-style recommendation systems must also manage embedding-table capacity, placement, and sparse access patterns. These techniques combine model compression (chapter 10) and distributed execution (chapter 8) to sustain high arithmetic intensity and balance operational costs at scale.

In contrast, mobile and edge devices operate under strict power, thermal, and memory bounds that necessitate hardware-software co-design. Efficient network architectures (chapter 6) combined with model compression (chapter 10) enable deployment on platforms where a reference mobile device has about 10× smaller memory capacity and about 140× smaller power envelope than an H100-class accelerator. Edge deployment is mandatory when latency, privacy, intermittent connectivity, or egress bandwidth make centralized cloud inference infeasible. In these regimes, efficiency dictates feasibility rather than minor cost optimization.²

Autoregressive generative models, illustrated by the GPT-2/Llama lighthouse family, stress memory hierarchy boundaries during token generation. Low-batch decode for dense autoregressive models stalls on memory bandwidth because each token step reuses model weights only once. Larger batch sizes elevate arithmetic intensity, whereas prompt prefill executes as compute-bound matrix multiplication. Model partitioning across devices—extending tensor and pipeline parallelism (chapter 8)—redistributes memory footprint at the expense of collective communication. Speculative decoding (chapter 13) trades surplus compute on a draft model to break the sequential latency bottleneck of the target model. Together, these mechanisms show the iron law adapting to complex workload structures.

At the opposite extreme, TinyML and embedded systems—the domain of the KWS/Wake Vision lighthouse—operate within kilobyte SRAM budgets, milliwatt power envelopes, and multi-year operational lifecycles. Operating under these extremes requires uncompromised systems discipline: rigorous profiling isolates memory traffic bottlenecks, custom operator lowering eliminates runtime overhead, and deterministic execution prevents buffer overflows. Severe resource limitations historically catalyzed efficient architecture families such as MobileNets (Howard et al. 2017; Sandler et al. 2018) and EfficientNets (Tan and Le 2019), proving that physical machine constraints actively drive algorithmic innovation.

A related frontier is *Physical AI* and embodied robotics, where models execute inside closed-loop sensorimotor control cycles. In this regime, the latency budget (L_{budget}) becomes a hard real-time safety constraint—where a tail-latency spike can cause mechanical instability or collision—and continuous high-bandwidth sensor streaming competes directly with model weight traffic across constrained on-device memory buses.

² | **AI democratization:** Making AI accessible beyond a small number of well-resourced organizations through efficient systems engineering. Mobile-optimized models and cloud APIs can widen access, but doing so sustainably requires systematic optimization across hardware, algorithms, and infrastructure to maintain quality at scale.

The same physical bounds apply across these paradigms, while fitted statistical models and SLO policies remain deployment-specific. Success depends on checking those distinctions and applying the principles together rather than pursuing isolated optimizations. The more deployment contexts a system spans, the more failure surfaces it creates. Robustness, not coverage alone, therefore becomes a binding constraint at the next frontier.

Building robust AI systems

ML systems can respond confidently and incorrectly while ordinary availability checks stay green, and no one may notice for weeks. Distribution shifts may alter accuracy without code changes, adversarial inputs can exploit vulnerabilities invisible to standard testing, and edge cases can reveal training-data limitations that debugging alone cannot fix. These are credible production risks, not outcomes guaranteed by a single divergence metric.

Finite testing estimates behavior on a defined population with uncertainty; it cannot prove correctness for every future input. Drift signals show when that population may have changed, while labeled outcomes determine whether quality changed. Together they motivate continuous monitoring as a design requirement, along with fallback policies and periodic revalidation, without claiming that every distribution shift causes degradation. The operational question is whether a failure will be detected and diagnosed before its impact spreads.

Robustness therefore demands designing for graceful degradation, not only prevention. At the single-system scale, that discipline appears as fallback paths, uncertainty thresholds, version-specific rollback policies, and monitoring hooks. Rollback addresses a bad release; external drift may instead call for alerting, traffic reduction, fallback, data collection, or retraining after outcome evidence confirms harm. At larger scale, the same logic extends to hardware redundancy and ensemble-style diversity. As AI systems assume increasingly autonomous roles in healthcare, transportation, and finance, the gap between “works in the lab” and “works in the world” becomes the critical engineering challenge. Robustness becomes more essential as systems add components, because each interface creates another timeout, stale input, inconsistent state, or recovery path to monitor.

AI for societal benefit

Robust systems are the prerequisite for deploying AI in domains where technical failures carry public consequences. A medical AI that fails unpredictably cannot be trusted with patient care. An educational system that degrades under load cannot serve the students who need it most. A climate model that produces confident but uncalibrated predictions may misdirect policy decisions affecting millions of lives. In each domain, the thirteen principles provide shared questions and bounds, while domain evidence determines acceptable policy.

Each domain stresses a different governing constraint before it can deliver social value:

- **Scientific discovery:** Protein folding, drug interaction modeling, and materials science can require substantial training or inference throughput governed by the iron law (principle 3) and silicon contract (principle 4); when work is distributed, coordination adds another systems cost.
- **Healthcare AI:** Clinical review, calibrated uncertainty, external validation, and continuous monitoring can become safety-critical because a diagnostic model trained on one hospital’s population may degrade when deployed to another with different demographics, disease prevalence, or imaging equipment.
- **Personalized education:** Protecting sensitive student-interaction data used for personalization at global scale stresses data governance and the data-as-code principle (1); interactive services must also meet their selected latency budgets.

All three applications demonstrate that technical optimization alone is insufficient. The D·A·M taxonomy, the thirteen quantitative principles, and roofline analysis establish the systems engineering *foundation*, but operationalizing that foundation demands domain-specific validation that no generic systems toolkit provides.

The bounded nature of these applications is what makes their systems constraints tractable: a diagnostic medical model classifies conditions within a defined label set, and a climate model projects climate statistics within physical constraints. The next frontier asks whether the same

principles can guide systems that delegate work across multiple components while preserving end-to-end requirements.

System composition as a stress test

The most ambitious stress tests for these principles are systems whose task boundaries are not fixed in advance. A task-general assistant or multi-component ML service may route one request through retrieval, planning, tool execution, generation, and verification. The governing challenge is systems engineering as much as algorithm design: the surrounding system must bound latency, reliability, cost, safety, and observability as work fans out across components.

System composition makes several central principles active at once:

- **Iron law:** The computation that each component performs must be budgeted as work fans out across retrieval, planning, tool execution, generation, and verification.
- **Silicon contract:** The system must honor hardware-specific constraints across CPUs, NVIDIA H100-class GPUs, Tensor Processing Units, and custom accelerators.
- **Pareto frontier:** The trade-off surface expands from two or three metrics, such as accuracy, latency, and memory, to a larger surface that also includes safety, fairness, factuality, privacy, and cost.
- **Statistical drift:** Drift applies not only to the final output, but also to retrieved documents, tool responses, and intermediate decisions.

A composed system cannot rely on a single model-quality claim; it needs interfaces whose behavior can be measured, because each interface is where one component's assumptions about another become testable (Lampson 1983). Composed systems trade monolithic simplicity for explicit coordination. A retrieval component finds relevant information, a reasoning component processes it, a tool call may query an external system, and a verifier checks the output. Each step can be independently updated, monitored, and debugged, but each also creates another interface contract. The decomposition trades latency and architectural complexity for control and observability, an example of a Pareto trade-off and possible complexity shift rather than a conservation law.

The systems cost is visible in a single request. If an assistant fans out to retrieval, a planner, two tools, a generator, and a verifier, its realized latency follows the request's critical path, with parallel tool calls contributing their maximum rather than their sum, plus orchestration overhead. Its reliability analysis must also account for each component: every additional component creates another timeout, schema mismatch, stale index, or verifier false negative to monitor. Conventional microservices already face runtime contract failures across process boundaries. Composed ML systems add probabilistic interfaces in which a large language model planner may hallucinate a tool name or produce JSON that deviates from the declared schema, requiring defensive parsing, retry logic, and output validation at each interface boundary. Capability can increase by adding system structure, but that structure must obey the same latency, reliability, and observability requirements as any production ML system.

System composition aligns directly with fundamental systems engineering principles. Modular sub-components can be independently compressed (chapter 10) and accelerated on targeted back-ends (chapter 11). Each component exposes an individual silicon contract (principle 4) and arithmetic intensity profile, opening avenues for heterogeneous specialization. The interfaces between components establish explicit observation points for detecting drift, skew, and numerical degradation. Orchestrating multiple models reliably, routing requests dynamically across specialized hardware, and enforcing transactional consistency across shared memory states require end-to-end co-design from data ingest through accelerator scheduling to operational feedback.

Systems Perspective 16.1: A new golden age

Hennessy and Patterson (2019) declared a “New Golden Age for Computer Architecture,” driven by the end of Dennard scaling, the slowdown of Moore’s Law, and new opportunities from domain-specific architectures, open instruction sets, and agile chip development. Large-scale AI workloads are one domain where those pressures are visible. Building more capable

AI services will not be a matter of writing a better loss function alone; it will be a systems engineering challenge involving energy efficiency, high-bandwidth interconnects, memory hierarchy design, and software stacks that keep heterogeneous hardware useful rather than idle. The thirteen principles provide a quantitative vocabulary for navigating that regime while preserving the distinction between physical bounds and deployment-specific assumptions.

That era demands concrete engineering advances. Achieving exascale sustained throughput ($\geq 10^{18}$ FLOP/s) and beyond requires new approaches to power delivery, cooling, interconnects, and software coordination, not merely faster chips. These analytical tools equip engineers to navigate that regime, scaling from single-node accelerators to exascale clusters.

16.5 Journey Forward

Operating at this scale demands disciplined systems engineering across the triad of data, algorithms, and machines. Managing stochastic data through lineage tracking and statistical validation, while enforcing execution bounds through physical hardware limits and service-level objectives, bridges deterministic software with probabilistic machine learning. Exposing hidden assumptions to measurement and observing failure modes down to the hardware interface provides the rigor required to make learned systems dependable.

Intelligence is a systems property. It emerges from co-designing data pipelines, model architectures, accelerator hardware, runtime software, and operational governance rather than from any isolated algorithmic breakthrough. The governing lesson of systems engineering is not a fixed recipe for a single model family or hardware generation. It is the discipline of exposing hidden dependencies to measurement, quantifying trade-offs against physical hardware limits, and holding production deployments accountable to real-world operational constraints.

The engineering responsibility

The systems integration perspective demonstrates why ethical considerations cannot be separated from physical and algorithmic design. Hardware requirements govern access: a model demanding an array of high-end data-center accelerators for inference excludes organizations unable to procure specialized capital. Upstream training data encodes historical sampling skews that distort downstream model predictions. Sustained accelerator power draw directly drives facility thermal dissipation and regional grid carbon emissions. System design choices—from arithmetic precision to data curation—distribute operational costs and societal consequences far beyond the engineering team.

Systems engineering extends beyond maximizing raw capability to operating deployments responsibly within societal and physical bounds. Production systems must achieve the arithmetic efficiency needed to run on accessible hardware, maintain fault tolerance against hardware anomalies, operate within strict thermal and power envelopes, and deliver equitable statistical accuracy across demographic groups. Realizing applications such as planetary climate monitoring and clinical diagnostic assistants demands rigorous systems discipline, guided by the governance criteria established in chapter 15 as first-class physical and operational constraints.

The principles established here provide a coherent lens for individual ML systems. Larger systems do not invalidate that lens; they expose the same constraints at a different boundary.

A horizon note: From node to fleet

When workloads exceed a single node, the resource boundary moves outward, but bottleneck reasoning remains invariant. Local memory capacity constraints yield to bisection bandwidth limits, local exception handling expands into fleet-level fault tolerance, and training throughput becomes a distributed coordination problem. As illustrated in the margin ladder, under an independent, identical constant-hazard model in which any GPU failure halts the pool, a reference component mean time to failure (MTTF) of 5.7 years collapses into a cluster mean time between failures (MTBF) of about 48.8 hours across a 1,024-GPU pool, before accounting for correlated network or power outages. A multi-week training run is virtually guaranteed to experience node failures. Co-designing a fault-tolerance strategy—combining asynchronous non-blocking checkpointing with localized state

1 GPU 5.7 yr

1024 GPUs 48.8 h

log scale

Fleet scale turns rare component failures into routine system events.

recovery—becomes an operational prerequisite for training convergence. At fleet scale, the analysis does not abandon single-node principles; it widens their boundary: identify the binding constraint, quantify the governing cost terms in the iron law, and trace where displaced latency propagates. Where this book has grounded these constraints within the boundary of a single machine, fleet-scale systems elevate the lens to the broader cluster. Scaling introduces new distributed challenges that emerge when models exceed single-device memory: 3D parallelism (data, tensor, pipeline), collective communication topologies, data center scheduling, and resilient fault-tolerant training across thousands of interconnected accelerators.

Systems engineering demands continuous skepticism: optimizing components in isolation routinely introduces systemic failure modes, creating the architectural pitfalls and common fallacies encountered when translating local benchmark wins into production deployments.

16.6 Fallacies and Pitfalls

Fallacies and pitfalls in ML systems arise from a common source: treating the system as decomposable into independent parts. Each fallacy assumes that optimizing one dimension, one metric, or one stage suffices; each pitfall shows the consequence when that assumption meets production reality.

Fallacy: *Systems engineering complexity disappears with better tools and abstractions.*

Tools abstract complexity; they do not eliminate physical constraints. A high-level framework that hides memory management still consumes memory. An AutoML system that tunes hyperparameters still faces the Pareto frontier. Simplifying one interface may shift burden to another, although good design can also remove accidental complexity. The engineer who believes tools eliminate fundamental constraints will be surprised when those constraints resurface at scale, often in forms harder to diagnose than the original problem.

Pitfall: *Optimizing one metric without tracing displaced costs.*

When an optimization reduces latency by 50 percent, ask what changed elsewhere. Quantization may add validation burden. Caching may trade memory capacity for serving speed. Some simplifications remove accidental complexity; others displace cost. Engineers who celebrate gains in one metric without tracing those effects can build systems that fail in unexpected ways. Measurement decides which occurred.

Fallacy: *Mastering individual components equals mastering the system.*

Component expertise is necessary but insufficient. An engineer who understands data pipelines, training, serving, and operations as isolated domains will still struggle with systems where a data schema change cascades through training, breaks quantization assumptions, and triggers silent accuracy degradation in production. Integration can create more complexity than the components reveal in isolation because interfaces multiply failure modes. Systems thinking means understanding how components interact, not just how they work individually.

Pitfall: *Scaling data collection without measuring marginal information value.*

The assumption that more data uniformly improves model quality breaks down in practice. As demonstrated in chapter 9, diminishing returns set in once a dataset achieves representational coverage: beyond that inflection point, doubling dataset volume yields negligible accuracy gains while scaling storage, preprocessing, and labeling costs linearly or superlinearly. The data-gravity principle (2) mandates measuring downstream operational costs, including whether repeatedly moving or scanning a larger dataset exceeds the cost of moving compute toward it. Scaling data volume without quantifying the marginal information value per sample optimizes an expensive proxy.

Fallacy: *A single accuracy metric captures model quality.*

A model evaluated solely on top-line accuracy ignores operational reality. Pareto analysis (principle 5) requires evaluating latency, throughput, memory footprint, energy, fairness, and serving cost simultaneously. Under a 100 ms tail-latency SLO, a 95 percent-accurate model executing in 500 ms is infeasible, whereas a 93 percent-accurate model completing in 50 ms satisfies production criteria.

Furthermore, aggregate accuracy routinely conceals severe error disparities across subgroup cohorts (chapter 15). Comprehensive evaluation spans the multi-dimensional Pareto surface rather than a single scalar.

Pitfall: *Treating every drift alarm as an automated rollback trigger.*

Statistical drift detection should trigger a diagnosed response rather than an unconditional automated rollback. Rollback is appropriate when a regression is traced to a flawed binary or pipeline deployment. External covariate drift, however, is not repaired by redeploying an older, equally stale checkpoint. Safe operational responses include canary traffic shedding, fallback heuristics, prediction quarantine, or scheduled retraining once ground-truth labels confirm degradation. Operational monitoring must couple anomaly detection to cause-specific remediation (chapter 14), intervening according to the verified failure mechanism rather than a raw statistical divergence alarm.

Fallacy: *A single optimized pipeline stage makes the system fast.*

Amdahl's Law (principle 8) applies directly to end-to-end ML pipelines. Optimizing accelerator inference latency by $10\times$ yields only $1.1\times$ system speedup if CPU-bound preprocessing accounts for 90 percent of end-to-end latency—serial fractions that can arise from host-side image augmentation, synchronous feature-store lookups, or tokenization that remains host bound in some implementations. The iron law of ML systems (principle 3) decomposes execution time into data movement, computation, and latency terms precisely so that engineers can identify the dominant term before investing optimization effort. Rigorous profiling isolates where execution time actually elapses (chapter 12). Optimizing without profiling amounts to guesswork, and Amdahl's Law strictly bounds speedup when interventions target non-dominant stages.

Pitfall: *Profiling only the stage that looks easiest to optimize.*

Teams often profile the model kernel because it is visible, instrumented, and owned by the ML team, while the surrounding data path is split across storage, preprocessing, networking, and application code. That local view can make a $10\times$ kernel improvement look urgent even when it changes little about the user-visible path. End-to-end profiling keeps the optimization target honest: the stage to improve is the one that limits the system, not the one with the cleanest benchmark harness.

These fallacies and pitfalls share a common root: attempting to isolate components from the system that executes them. Whether optimizing a single metric, accelerating an isolated stage, or evaluating accuracy without latency constraints, local optimization fails when it ignores system coupling. Reasoning across physical and operational boundaries remains the central discipline of ML systems engineering.

16.7 Summary

ML systems engineering differs from isolated component optimization by reasoning across boundaries. The thirteen principles, the conservation-of-complexity heuristic, and the lighthouse journey framework provide analytical tools for reasoning about systems as wholes. Their stated assumptions distinguish exact bounds from fitted models, SLO policies, and design heuristics, allowing the tools to remain useful as frameworks, hardware generations, and model families change.



Key Takeaways: Reasoning across boundaries

- **Assumptions matter across implementations:** The thirteen principles turn framework-specific craft into measurable reasoning by combining physical bounds, decompositions, fitted models, requirements, and heuristics. Apply each only within its stated scope.
- **Trace displaced costs without assuming conservation:** Compression, batching, monitoring, and governance can relocate burdens across data, algorithm, and machine, while good design can remove accidental complexity outright. Measurement distinguishes the two cases.
- **Boundaries reveal the bottleneck:** In the idealized batch-one, two-H100 model, the memory-transfer lower bound for a Llama 2 70B token is about $295.2\times$ the peak-compute

lower bound, and the illustrative p99 latency is 40× the mean. Systems thinking means measuring where physics, traffic, and users bind.

- **Scale changes the binding term:** The next frontier is scale, where a thousand-GPU pool turns multi-year component MTTF into days-scale cluster MTBF. The physics stays, but the constraint moves to fleets.

Hennessy and Patterson's *Computer Architecture: A Quantitative Approach* established a shared analytical discipline for comparing CPI, clock rates, and instruction counts (Hennessy and Patterson 2011; Hennessy and Patterson 2017). These thirteen principles provide an equivalent quantitative foundation for machine learning systems without claiming that every diagnostic constitutes an unvarying physical law. They establish a shared analytical baseline for evaluating trade-offs across data, algorithms, and machines.

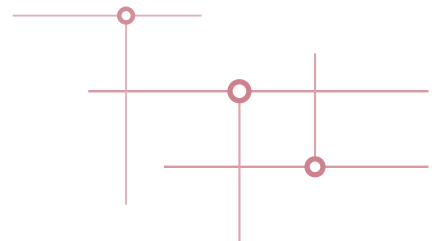
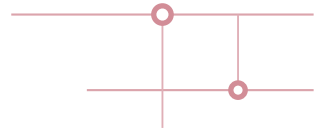
What endures is the intellectual posture these principles embody: reasoning from empirical evidence and physical bounds rather than reacting to symptoms, quantifying trade-offs rather than following trends, and treating system design as constrained optimization. This is the engineering corollary of the bitter lesson drawn from seven decades of computing research (chapter 1): because general methods that scale with computation repeatedly outrun hand-crafted expertise, the durable advantage belongs to systems engineering that can absorb that computation, not to any single clever architecture (Sutton 2019). Specific frameworks will rise and fall, hardware generations will turn over, and model architectures will be superseded. Disciplined reasoning about data, computation, and physical constraints will not.

At the next frontier of scale, some models no longer fit on one machine, failures become highly probable across fleets, and network links can become binding alongside local memory buses. The physics does not change; the scale at which it binds does.

The world is rushing to build AI systems. Our task is to engineer them.

Prof. Vijay Janapa Reddi, Harvard University

BACK MATTER



The D·A·M Taxonomy

A system that runs slowly rarely announces why. An idle accelerator can stem from a starved input pipeline (Data), an algorithm with too little arithmetic intensity (Algorithm), or a saturated memory bus (Machine). This appendix formalizes the book's single-node diagnostic taxonomy, mapping profiler symptoms and measurements directly to the axis that actually binds execution before optimization begins. Readers should arrive with the three axes from the opening chapters and the iron law that quantifies them.

How to Use This Appendix

Use the D·A·M taxonomy when a system misses a performance, cost, or quality target and the cause is not yet clear. Work through the framework in four passes:

1. **State the symptom and operating budget:** Record the missed end-to-end target, workload, and deployment context.
2. **Localize the candidate axis or boundary:** Use table A.1 for a first pass and table A.2 when the evidence crosses axes.
3. **Test the hypothesis:** Map the evidence to the iron law and arithmetic intensity, then gather confirming measurements with table A.6 and table A.7.
4. **Intervene and remeasure:** Choose a lever from table A.4 and repeat the end-to-end measurement. If the target does not improve, revise the hypothesis.

The symptom determines the most useful entry point. Low accelerator utilization or slow batch delivery should begin with section A.5 and section A.8. A missed service-level objective (SLO) should move from phase timing and queue depth in table A.5 to the corresponding iron-law term. A quality plateau should be checked against the Data/Algorithm boundary and section A.11 before allocating more compute. In every case, the axis label is a hypothesis, not a verdict: the axes overlap, and a useful diagnosis may name both a dominant axis and a boundary effect.

A.1 Diagnostic Summary

The taxonomy maps directly to the iron law of ML systems established in section 1.7. Table A.1 summarizes the role, primary physical constraint, and core optimization pathway for each axis.

Table A.1: D·A·M Axis Reference: Each axis maps to a constraint class and optimization strategy. Form a hypothesis, confirm it with measurements, then follow the chapter pointer.

Axis	Role	Physical Constraint	High-Leverage Optimization
Data (D)	Information (The Fuel)	Volume, quality, arrival rate	Data Selection (chapter 9)
Algorithm (A)	Logic (The Blueprint)	Operations and dependencies	Model Compression (chapter 10)
Machine (M)	Physics (The Engine)	Compute, memory, I/O	Hardware Acceleration (chapter 11)

This first-pass separation is useful, but production systems rarely suffer from a single axis-centered bottleneck. More often, the problem sits at the *boundary* between two axes—a data format choice that determines whether the GPU can be saturated, or a pruning strategy that changes the memory access pattern. Handling these cases requires mapping the intersections.

A.2 Intersection Landscape

The opening chapter uses figure 1.4 to establish the conceptual landscape. Here, figure A.1 turns that map into a diagnostic sequence. First identify the binding axis, then ask how an intervention will alter the other two.

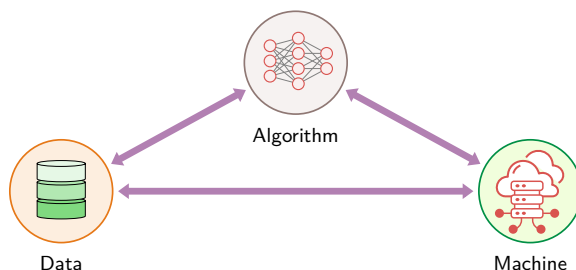


Figure A.1: The D-A-M Axis Triad: A compact diagnostic view of Data, Algorithm, and Machine as three interdependent axes. The bidirectional links emphasize that an intervention on one axis changes the constraints on the other two.

With the first-pass axis identified, table A.2 expands the pairwise and three-way intersections into concrete engineering techniques and their primary book chapters.

Table A.2: D-A-M Intersection Reference: Each zone maps specific techniques to the axes they span and the chapters that cover them. The pairwise intersections require reasoning about two domains simultaneously; the center requires all three.

Zone	Name	Key Techniques	Book Coverage
Data	Information	Storage formats, data quality, distributions	chapter 4
Algorithm	Logic	Loss functions, architectures, gradients	chapter 5, chapter 6
Data $\cap$ Algorithm	What to Learn From	Data selection, curriculum learning, compute-optimal scaling	chapter 9, chapter 8
Data $\cap$ Machine	How to Move Information	I/O bandwidth, prefetching, data formats	chapter 4, chapter 11
Algorithm $\cap$ Machine	How to Execute Efficiently	Quantization, pruning, kernel fusion, mixed precision	chapter 7, chapter 10
Machine	Physics	Silicon, memory hierarchy, peak FLOP/s	chapter 11
Data $\cap$ Algorithm $\cap$ Machine	ML Systems Engineering	Iron law, Roofline, training loops, serving	chapter 8, chapter 13, chapter 12

In the D-A-M acronym, Data, Algorithm, and Machine are taxonomy axes. They are not mathematical variables; formal quantities still follow the notation chapter, where D denotes dataset size or training tokens.

The axis-centered zones contain concepts that belong primarily to one axis: storage formats and distributions emphasize Data, loss functions and gradients emphasize Algorithm, and silicon physics and peak FLOP/s emphasize Machine. Single-domain expertise can introduce them, but deployed behavior may still depend on the other axes.

The pairwise intersections are where systems thinking begins. **Data $\cap$ Algorithm** (*What to Learn From*) encompasses data selection, curriculum learning, active learning, and scaling laws like Chinchilla ($D \approx 20P$ for the dense autoregressive language-model regime studied) (Hoffmann et al. 2022)—all requiring joint reasoning about information content and algorithmic capacity. Adding data without considering whether the model can learn from it wastes compute; choosing architectures without considering data availability wastes engineering time. **Data $\cap$ Machine** (*How to Move Information*) covers I/O bandwidth, prefetching strategies, data formats, and the energy-movement invariant. This intersection is where data gravity manifests: the physical cost of moving bytes through the memory hierarchy determines whether the machine can be fed fast enough. **Algorithm $\cap$ Machine** (*How to Execute Efficiently*) spans quantization, pruning, kernel fusion, mixed precision, and computational graph optimization. A pruning strategy that reduces FLOPs but destroys memory access patterns can *slow down* execution on real hardware.

The center—Data ∩ Algorithm ∩ Machine—is where all three axes converge. The iron law, the Roofline Model, end-to-end training loops, serving pipelines, and holistic benchmarking all require simultaneous reasoning about data flow, algorithmic complexity, and hardware utilization. This center is not a single technique; it is the discipline itself.

While the landscape classifies where a technique operates, isolating which axis actively throttles a workload requires translating these relationships into wall-clock execution time—the role of the iron law.

A.3 Iron Law Mapping

For serialized phases, the iron law maps work across the D·A·M axes to execution time. Here, T and L_{lat} are times in seconds; D_{vol} is bytes moved and BW is bytes per second; O is work in FLOPs, R_{peak} is FLOP/s, and η_{hw} is dimensionless hardware efficiency. These quantities map to the axes as follows:

$$T = \underbrace{\frac{D_{\text{vol}}}{\text{BW}}}_{\text{Data/Machine}} + \underbrace{\frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}}_{\text{Algorithm/Machine}} + \underbrace{L_{\text{lat}}}_{\text{Cross-axis overhead}}$$

Algorithm and Machine share the compute term, separated by which variable the engineer controls. Reducing the total operations (O) is an Algorithm lever, while improving the hardware's peak throughput (R_{peak}) or utilization (η_{hw}) is a Machine lever.

This equation transforms performance debugging from a qualitative guessing game into a quantitative engineering problem. The dominant measured cost appears in one of these terms. A slow system may move too much data (D_{vol}), lack bandwidth (BW), execute too many operations (O), fail to use the hardware's peak capability (η_{hw}), or pay cross-axis overhead (L_{lat}). The levers below map specific optimizations to the variables they improve.

A.3.1 Component levers

- Data Lever: Reducing moved bytes (D_{vol}) through deduplication, selection, or lower-precision representation, or increasing I/O bandwidth (BW).
- Algorithm Lever: Reducing operations (O) through pruning or architectural refinement. Quantization narrows representation and may raise hardware throughput; it does not generally reduce operation count.
- Machine Lever: Increasing the denominator of the compute term by improving peak throughput (R_{peak}) or increasing the utilization factor (η_{hw}) via kernel fusion.

A.3.2 D·A·M coordination: From sum to max

The additive iron law represents sequential execution. Let $T_{\text{sequential}}$ denote time without overlap and $T_{\text{pipelined}}$ denote measured time with overlap. Overlap can move the sum toward a max, but dependencies, contention, and incomplete overlap keep the measured time above this ideal lower bound:

$$T_{\text{sequential}} = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}} \xrightarrow{\text{overlap}} T_{\text{pipelined}} \geq \max \left(\frac{D_{\text{vol}}}{\text{BW}}, \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} \right) + L_{\text{lat}}$$

The systems engineer's job is to make these components run in parallel, not in series. Table A.3 summarizes key D·A·M coordination techniques that overlap work across axes:

Table A.3: D·A·M Overlap Techniques: Each technique overlaps work across axes, moving sequentially added Data/Machine and Algorithm/Machine time toward their ideal lower bound: the larger of the two terms. Dependencies and contention limit realized overlap.

Technique	D·A·M Axes Overlapped	Implementation
Prefetching	D overlaps M	DataLoader with <code>prefetch_factor</code> , <code>pin_memory=True</code>
CUDA Streams	D overlaps M	Separate streams for H2D transfer and compute

Technique	D·A·M Axes Overlapped	Implementation
Async Gradient Sync	M (communication) overlaps A	Overlap bucketed AllReduce with remaining backward computation
Double Buffering	D overlaps M	Fill buffer N+1 while computing on buffer N

Overlaps provide the greatest relative speedup when the D·A·M time terms are reasonably balanced. If one term dominates (for example, severely memory bound), overlapping the smaller term with the larger yields negligible gain—the max is still dominated by the same bottleneck. This balance occurs near $D_{\text{vol}}/\text{BW} \approx O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$. The latency term requires separate treatment.

🔍 Systems Perspective A.1: The overhead that cannot hide

The unhidden latency term L_{lat} (kernel launch, synchronization barriers, Python dispatch) contains serialization points that cannot be fully overlapped. Kernel fusion can reduce this term by combining operations, but it does not eliminate all launch, scheduling, or synchronization overhead.

While the iron law quantifies the execution time consumed by each stage, determining whether data movement or arithmetic throughput sets the active performance ceiling depends on their ratio—the domain of arithmetic intensity.

A.4 Arithmetic Intensity Boundary

The boundary between the *Data/Machine* memory term and the *Algorithm/Machine* compute term is not arbitrary; it is defined mathematically by arithmetic intensity¹ (I) of the workload.

The Roofline Model provides an upper performance bound when traffic and achieved rates are measured consistently. In the middle of a production incident, faster screening heuristics can point to the measurements to collect next.

A.5 Rules of Thumb

In the heat of a production outage, there is rarely time to solve the full iron law equation. Veteran systems engineers instead rely on quantitative heuristics to quickly narrow the search space; the thresholds below are screening signals that should be checked against profiler traces and hardware counters.

- Low accelerator utilization (< 80 percent): Screen for data, CPU, or launch starvation. Confirm with input-pipeline wait, host CPU saturation, memory-bandwidth counters, and trace gaps.
- High accelerator utilization (> 95 percent): Treat this as machine bound only if compute units are busy and memory bandwidth is not saturated. If memory bandwidth is saturated, the bottleneck is still on the Data/Machine boundary.
- If batch size is one: Treat this as a warning that latency or launch overhead may matter; confirm with traces before labeling the algorithm itself the bottleneck.
- Low arithmetic intensity (below the hardware ridge point): The workload is likely memory bound (Data/Machine boundary). The precise boundary is $R_{\text{peak}}/\text{BW}$ and depends on hardware and precision.
- If the system works in dev but fails in prod: Treat data drift as one hypothesis alongside configuration, load, dependency, and runtime differences.

Common industry labels cross the axes. Memory-bound execution sits on the Data/Machine boundary because both bytes moved and available bandwidth matter. Compute-bound execution couples Algorithm work with Machine throughput. Latency-bound execution may reflect algorithmic serial depth, runtime dispatch, synchronization, or queueing.

¹ | **Arithmetic intensity:** This book uses the term for floating-point operations per byte transferred. Williams et al. (2009) calls the measured DRAM quantity *operational intensity* in the Roofline Model. Comparing the workload ratio with the hardware ridge point ($R_{\text{peak}}/\text{BW}$) gives an upper-bound diagnosis of memory- versus compute-bound execution.

A.5.1 Bottleneck diagnostic

Once profiling identifies the dominant term in the iron law, optimizations must target that specific axis. Table A.4 contrasts the interventions that yield end-to-end speedups against nonbinding optimizations that waste engineering effort:

Table A.4: What Works vs. What Is Wasted: Optimizing a nonbinding term yields little or no end-to-end improvement. A memory-bound large language model will not benefit from additional peak FLOP/s alone, though an accelerator with more memory bandwidth may help.

If the workload is...	Dominant Term	Optimization That Works	Optimization That is Wasted
Memory-Bound	D_{vol}/BW	Quantization; structured pruning; batching for weight reuse; fusion that removes intermediate traffic	Faster accelerator (more FLOP/s will not help)
Compute-Bound	$O/(R_{\text{peak}} \cdot \eta_{\text{hw}})$	Better kernels, Tensor Cores, faster accelerator, lower precision	More memory bandwidth (not the binding term)
Latency-Bound	L_{lat}	Reduce serial depth; fuse or dispatch asynchronously; batch within budget	More compute or bandwidth unless the trace shows it binding

Knowing what works also means recognizing what does not. In practice, teams under deadline pressure repeatedly fall into the same traps—optimizing the wrong axis with confidence. These failure modes are common enough to deserve their own names.

A.6 Anti-Patterns

Diagnosing systems is often a process of elimination. Before committing to complex kernel optimizations, watch for these common traps that waste engineering cycles.

- The hardware crutch: Buying faster accelerators (*Machine*) to fix a slow Python data loader (*Data*). The new hardware will just idle faster.
- The model twiddle: Changing neural architectures (*Algorithm*) when the bottleneck is actually network bandwidth or disk I/O.
- The premature optimizer: Writing custom CUDA kernels (*Machine*) before verifying if the Algorithm is simply doing too many unnecessary operations.

Each anti-pattern follows the same root cause: acting before diagnosing. The following case studies show what proper diagnosis looks like—starting from a confusing symptom and systematically narrowing to the dominant D·A·M axis.

A.7 D·A·M Case Studies

Theoretical constraints often manifest as confusing symptoms in production. These representative scenarios illustrate how to apply the taxonomy. Each case follows the same three diagnostic moves: symptom, diagnosis, and fix.

A.7.1 Case 1: The starving accelerator (Data)

A team provisions a large A100 GPU instance to speed up training, but training time hardly improves and `nvidia-smi` shows GPU utilization fluctuating between 10 percent and 40 percent. The pattern suggests accelerator starvation but does not identify its cause. If traces show input-wait gaps with storage or CPU decoding saturation, the data path is binding. The useful intervention is then upstream of the model. Optimize the extract, transform, load (ETL) path by moving from raw JPEGs with heavy CPU decoding to sequential formats such as TFRecords or WebDataset, increasing data-loader parallelism, and prefetching batches into accelerator memory.

A.7.2 Case 2: The latency cliff (cross-axis)

A real-time recommendation service fails to meet a 20 ms latency service level agreement (SLA), while accelerator utilization is low and the batch size is one. That combination suggests launch overhead, memory traffic, or serial model depth rather than a saturated chip. If a trace confirms

that the sequential layer path dominates, adding more hardware will not remove it, so pruning or knowledge distillation can reduce model work. If weight traffic dominates instead, quantization can reduce moved bytes by using INT8 where accuracy allows. The measurements select the remedy.

A.7.3 Case 3: The compute wall (Machine)

Accelerator utilization is pinned at 99 percent, memory bandwidth remains unsaturated, and training is stable but takes three weeks. If profiler counters also show high compute-pipeline occupancy and model FLOPs utilization (MFU), the workload is compute bound. The data path is keeping the accelerator fed, so the next step must change the compute term in the iron law. The team can scale up from an A100 to an H100-class accelerator, scale out through data parallelism, or lower precision from FP32/TF32 to BF16 where numerically safe. On NVIDIA Tensor Core paths, BF16 peak throughput is typically about 2× TF32 peak, with realized speedup depending on kernels and bottlenecks (Choquette 2023). Taken together, the three cases show why the same utilization number can imply different next steps depending on loss behavior, batch size, and memory pressure.



Checkpoint A.1: D·A·M diagnosis check

1. A training job shows 95 percent accelerator utilization but loss has plateaued for two epochs. Which D·A·M axes could be responsible, and why is utilization alone insufficient to distinguish them?
2. Your colleague suggests adding more data loader workers to a job where `nvidia-smi` shows 98 percent GPU utilization. What trace evidence is needed before deciding whether this will help?
3. An inference server meets its latency SLO at batch size 1 but fails at batch size 16. Which iron-law terms and queueing effects may have changed, and which measurements identify the binding regime?

These three cases illustrate bottlenecks that cross axes. In live production incidents, symptoms frequently overlap and the dominant bottleneck can shift dynamically as optimizations land. Systematic elimination requires screening profiler signals against a structured diagnostic matrix.

A.8 Production Troubleshooting

Identifying the root cause of performance bottlenecks requires systematic elimination. Table A.5 provides a diagnostic matrix mapping common production failure modes to candidate axes and validating measurements.

Table A.5: D·A·M Diagnostic Matrix: First hypotheses and measurements for common failures. Confirm the binding component before intervening.

Symptom	Hypothesis	Confirm With	Act After Confirmation
Low Accelerator Utilization	Data/host starvation	Trace gaps; loader, CPU, storage, launch counters	Prefetch, parallelize, or cut launches.
High Latency (P99)	Algorithm/run-time/queue	Phase p99 and queue depth	Optimize the dominant phase.
High Training Cost	Low useful-work efficiency	MFU, communication, idle time	Reduce the largest loss or resize.
High Inter-GPU Overhead	Communication bound	Device trace; collective gaps	Bucket AllReduce; tune data-parallel overlap.
Out-of-Memory (OOM)	Algorithm/Machine fit	Peak weights, states, activations	Checkpoint, shard, quantize, or shrink batch.

A.9 Tooling Map

A diagnostic hypothesis requires concrete hardware and runtime counters before taking corrective action. Table A.6 connects theoretical D·A·M components to the specific Linux and Python profiling utilities that expose them.

Table A.6: D·A·M Tooling Map: Profiling utilities for diagnosing bottlenecks along each D·A·M axis. Start with the primary tool for quick triage; use secondary tools for deep-dive analysis when the primary tool’s output is inconclusive.

Axis	Key Metric	Primary Tool	Secondary Tool
Data	Batch Load and Wait Time	Framework profiler trace	iostat, pidstat (I/O/CPU)
Algorithm	FLOPs, Model Depth	PyTorch Profiler	DeepSpeed Flops Profiler
Machine	Accelerator Utilization, SM Occupancy	Nsight Compute	nvidia-smi, Nsight Systems

Profiling tools generate raw metrics—utilization percentages, FLOP counts, and bandwidth measurements—that become actionable only when evaluated against workload constraints. The D·A·M Scorecard establishes screening signals rather than universal grades, isolating which measurements warrant immediate intervention.

A.10 D·A·M Scorecard

To move beyond qualitative guessing, the efficiency indicators in table A.7 help decide what to investigate next. This report-card view aids the first comparison, but no row provides a universal grade. MFU² is especially useful for large-model training.

Table A.7: D·A·M Screening Indicators: Context-dependent signals for investigation, not universal grades.

Axis	Metric	Definition	Investigate	Strong Signal
Data	I/O Overhead	$\frac{\text{Data Wait Time}}{\text{Total Step Time}}$	> 10% (screen)	< 1% (strong)
Algorithm	Avoidable Work	Profiler or ablation finds redundant operations	Redundant work found	End-to-end work falls
Machine	MFU	$\frac{\text{Achieved model FLOP/s}}{\text{Peak FLOP/s}}$	< 30% (large models)	> 50% (same regime)

The Scorecard and the Roofline Model both answer efficiency questions, but at different scales. The Scorecard screens the current system using context-dependent indicators. Scaling laws and the information-roofline metaphor ask what may happen as the system scales *beyond* its current size.

A.11 Scaling Laws vs. Roofline

Systems engineering requires distinguishing between *growth trajectories* and *fundamental limits*.

A.11.1 Scaling laws (the journey)

Scaling laws³ are empirical power laws that predict how held-out loss changes as resources increase within the fitted regime. Two landmark results are Kaplan scaling (Kaplan et al. 2020), which studied loss against parameter count (*P*), data (*D*), and total operations (*O*), and Chinchilla scaling (Hoffmann et al. 2022), which estimated a compute-optimal balance for the dense autoregressive language-model regime studied, often summarized as roughly 20 training tokens per parameter ($D \approx 20P$).

As economic guides, these fits estimate how held-out loss changes with compute inside the observed regime. They do not guarantee a task error-rate reduction or extrapolate unchanged across architectures and data distributions.

A.11.2 Information roofline (the destination)

The *information roofline* is used here as a diagnostic metaphor rather than a standard quantitative law. Its destination image points to limits on what can be learned from the available data. For classification

² | **MFU (model FLOPs utilization):** The ratio of a achieved model FLOP/s to the hardware’s theoretical peak FLOP/s, introduced in the PaLM paper (Chowdhery et al. 2022). Unlike raw accelerator utilization (which counts any work the accelerator performs), MFU measures model computation rate relative to peak hardware throughput, so non-model work and system overhead are not counted as achieved model FLOPs. Section 8.5 develops MFU as a training diagnostic.

³ | **Scaling laws:** Empirical relationships, typically power laws with a fitted negative exponent, that predict model loss as a function of dataset size, parameter count, or compute budget. Kaplan et al. (2020) studied these relationships for neural language models at OpenAI; Hoffmann et al. (2022) later estimated the compute-optimal training trade-off for its studied dense language-model regime. Section 9.11.2 develops the compute-optimal frontier and its data-starvation diagnostic in detail.

⁴ | **Bayes error rate:** The lowest achievable error rate under 0–1 loss for any classifier on a given data distribution, determined by class priors and the overlap between class-conditional distributions (Goodfellow et al. 2016). No amount of data, parameters, or compute can reduce error below this theoretical floor.

under a fixed data distribution and loss, the Bayes error rate⁴ is an irreducible error *floor*, not a ceiling. Noise, label ambiguity, coverage gaps, and distribution shift can create data-quality limits, but they do not define one universal slope or breakpoint.

The diagnostic lesson is narrower. Scaling laws describe a fitted improvement trajectory, while the information-roofline metaphor prompts a search for data-quality limits. If a loss curve flattens relative to the fitted trend, investigate optimization, capacity, evaluation noise, and data quality; flattening alone does not identify which cause dominates. The Bayes floor describes the best achievable classifier for the specified distribution. Adding accelerators or parameters is futile only after measurements establish that data quality is binding. Whether debugging a training step, evaluating hardware utilization, or planning a scaling campaign, measure which axis is binding before choosing a lever.

A.12 Summary

These tools form a single diagnostic workflow. D·A·M localizes candidate causes across Data, Algorithm, and Machine. The iron law and arithmetic intensity translate those causes into time, operations, and bytes, while the case studies, anti-patterns, tooling map, and scorecard show how to test them in production. Scaling laws and the information roofline then separate observed improvement trajectories from data-quality limits. Use the framework as a loop rather than as a label.



Key Takeaways: Where to look first

- Begin with cross-axis hypotheses; a symptom rarely identifies its cause.
- Translate each hypothesis into the relevant iron-law term and a measurement that could falsify it.
- Use arithmetic intensity and the scorecard to identify the binding constraint, not to assign universal grades.
- Apply the corresponding lever and remeasure end to end; a local gain that leaves the binding term unchanged is not a system improvement.

Data Foundations

DATA problems rarely present as data problems; they surface as a training run that will not speed up, an accelerator sitting idle, or accuracy eroding while dashboards stay green. To keep chapter arguments focused on system design, this appendix gathers the back-of-the-envelope calculations and statistical tools that trace such symptoms back to measurable data bottlenecks: transfer times, storage-layout costs, drift diagnostics, and numerical-stability tools. The tools here lean on basic probability and on the data term of the iron law.

How to Use This Appendix

This appendix is designed as a *reference*. Reach for it when debugging “slow training,” “low accelerator utilization,” or “production accuracy drift” calls for the quickest path from symptom to measurement.

Conventions used here follow the book-wide notation (for example, B is reserved for batch size and BW denotes bandwidth).

- **When data will not move:** Start with table B.1 and the transfer-time equation in section B.1.1.
- **When the accelerator is starving:** Use table B.2 and the layout discussion in section B.1.3.
- **When pipelines explode in cost:** Use the primitives in section B.1.4, especially join-induced shuffles.
- **When “average looks fine” but users complain:** Use section B.2.1 and session-level tail probability.
- **When accuracy drifts silently:** Use section B.2.2 to compare full distributions.

While production pipelines encounter an overwhelming variety of formats, encodings, and storage layers, this appendix focuses on the physical and statistical constants that govern them all.

With those reference points in place, the appendix begins with the physical constraints of data engineering and then moves to the statistical monitoring that keeps ML systems healthy. From storage formats that determine I/O throughput to drift metrics that detect silent failures, these foundations connect directly to the data pipelines in section 4.5 and the operational monitoring in section 14.5.2.2.

B.1 Data Engineering Foundations

Understanding hardware constraints is only half the battle; we must also shape our data to fit them. Data engineering applies the principles of the memory hierarchy to storage formats and pipeline design, ensuring that the accelerator never starves. This process begins with recognizing that data is physical—it has volume, it takes time to move, and it requires energy to parse.

B.1.1 Napkin math: The physics of data gravity

Data gravity¹ is a metaphor grounded in transfer-time calculations. Unlike compute, which gets faster every year, the speed of light is fixed and network bandwidth is a finite resource. When datasets grow large enough, their practical inertia grows—moving them can cost more time and energy than moving computation to where the data already lives.

The ideal line-rate transfer time is $T = D_{\text{vol}}/\text{BW}$ after converting data volume and bandwidth to compatible units (the calculations below convert decimal bytes and bits per second explicitly). For

¹ | **Data gravity:** Coined by Dave McCrory in 2010 to describe how large datasets attract services and applications toward them, much as massive bodies attract smaller ones in physics (McCrory 2010). The analogy is apt: the “escape velocity” required to move a petabyte-scale dataset is often measured in weeks.

the large volumes here, latency is negligible; the full equation appears in section D.3.4. Table B.1 gives lower bounds; protocol overhead and endpoint limits increase them.

Table B.1: The Cost of Inertia: These transfer times explain why we often “ship compute to data” rather than “ship data to compute.” Network columns are lower bounds at sustained line rate. The truck entry is an illustrative elapsed-time model for 1 PB: 8 hours to load, 32 hours in transit, and 8 hours to unload, excluding scheduling and provisioning overhead.

Data Volume	1 Gbps (Standard WAN)	10 Gbps (High-End WAN)	100 Gbps (Direct Connect)	Truck (Illustrative)
1 TB	2.2 hours	13.3 minutes	80 seconds	N/A
100 TB	9 days	22.2 hours	2.2 hours	N/A
1 PB	3 months	9 days	22.2 hours	2 days

B.1.2 The cost of serialization

Even after data arrives at the machine, the serialization tax remains a final hurdle.² As table B.2 shows, many engineers meticulously optimize their accelerator kernels while ignoring the CPU overhead of decoding data. Parsing text-based formats like JavaScript Object Notation (JSON) or CSV is extremely CPU-intensive, often leaving the accelerator idling while the CPU struggles to convert strings into floating-point numbers.

Table B.2: Serialization Overhead: These illustrative values show why binary columnar formats can outperform row-based text, but they are not portable benchmarks. Arrow is an in-memory/IPC format; Parquet is a compressed storage format.

Format	Decoding Speed (MB/s)	Relative CPU Decode Cost	Suitability
CSV/JSON	~100 MB/s	High	Inspection/interchange
Protobuf	~300 MB/s	Medium	RPC/Messages
Parquet	~1,000 MB/s	Low	Columnar storage

B.1.3 Row vs. columnar formats

The choice of file format determines the “physics” of how the data is read. Row-oriented formats such as CSV and JSON store data record-by-record, so reading the `age` field still requires parsing each row. This suits appended logs but not analytics on selected features. Parquet stores compressed columnar chunks on disk, while Arrow provides a columnar in-memory and IPC format. Parquet readers can use *projection pushdown* to fetch selected column chunks, and both formats support *vectorized processing*. Compare the two arrangements in figure B.1 to see why columnar access avoids scanning unnecessary bytes.

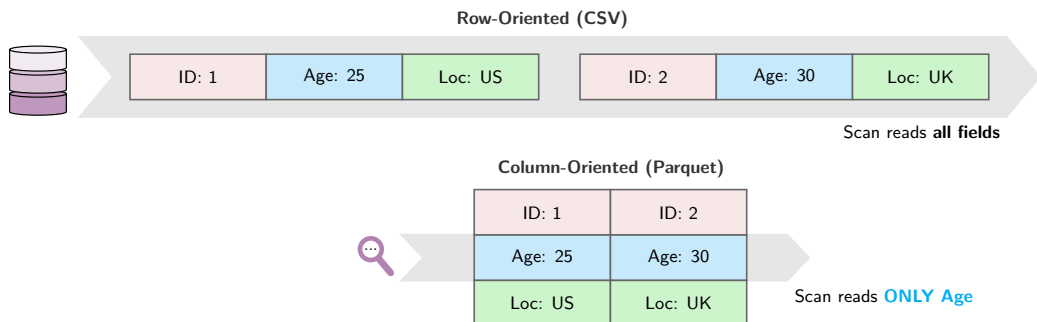


Figure B.1: Storage Layouts: Row-oriented formats group values by record, so the illustrated scan reads every field. Column-oriented formats group values by feature, so the illustrated scan reads only Age.

The layouts in figure B.1 differ in how many irrelevant bytes enter the training path.

Systems Perspective B.1: The accelerator starvation problem

The choice of file format can determine whether a system is I/O bound or compute bound. As figure B.1 shows, storage layout compounds the serialization tax because row-oriented formats force full-row scans while columnar formats enable projection pushdown, reading only the bytes the model needs. The result is that the “Data Movement” term in the iron law can silently become the bottleneck that leaves expensive accelerators idling.

B.1.4 The algebra of data

Feature engineering turns raw records into the columns a model can learn from, and that transformation is usually a dataflow problem before it is a modeling problem. A pipeline first narrows the population, then chooses the features, then attaches context from other tables. Those three moves correspond to three Structured Query Language (SQL) primitives, and their computational cost determines whether the feature job stays local, scans unnecessary bytes, or turns into a network shuffle.

1. **Selection (σ):** Filtering rows (for example, `WHERE age > 30`).
 - **Cost:** $\mathcal{O}(\log N + K)$ if a B-tree index lookup returns K rows; $\mathcal{O}(N)$ for a full scan.
2. **Projection (π):** Selecting columns (for example, `SELECT age`).
 - **Cost:** Selected chunks plus decoding and metadata in columnar formats. This row-format example reads 1 KB for a 4-byte integer, wasting 99.6 percent of the read.
3. **Join ($\bowtie$):** Combining tables. The most expensive operation.
 - **Shuffle Join:** Both tables are partitioned by key and exchanged over the network.
 - **Cost:** Reshuffling both 1 TB tables moves about ~2 TB; partitioning and locality can reduce this.
 - **Broadcast Join:** One small table is sent to all workers.
 - **Cost:** $S(W - 1)$ bytes; the S -byte table must fit on each of W workers.

Understanding data formats, serialization costs, and algebraic primitives tells us how to *move* data efficiently. Even a perfectly engineered pipeline, however, can silently fail if the data it carries changes character over time. Detecting that change—and quantifying how much it matters—requires a different set of tools: probability and statistics.

B.2 Probability and Statistics

Once data is flowing through our pipelines, we need mathematical tools to ensure its quality and consistency. Probability and statistics provide the language for monitoring system health, detecting the silent failures of data drift, and managing uncertainty.

B.2.1 Distributions and the long tail

In systems, the mean is often misleading. Latency distributions are often long-tailed, and user-visible services are commonly governed by high-percentile behavior rather than mean response time (Dean and Barroso 2013). A “P99” (99th percentile) latency of 500 ms means at most 1 percent of requests take longer than 500 ms; with many requests per session, the fraction of users who see at least one slow request can be much higher. At scale (1M users), even a 1 percent affected-user rate would affect 10,000 users.

Napkin Math B.1: The median experience

For $N_{\text{session}} = 100$ independent requests, each with a 0.01 chance of being slow, the probability of at least one P99-tail event is $\Pr(\text{Slow}) = 1 - (0.99)^{100} \approx 1 - 0.366 = 63.4\%$.

Systems insight: Under this model, a 100-request session is more likely than not to include an event beyond P99. Correlated slowdowns can change this probability (Dean and Barroso 2013).

B.2.2 Measuring drift (divergence)

Because distributions have long tails where the most dangerous failures hide, simple metrics like “mean shift” are insufficient. We need tools that compare the *entire shape* of the distribution. Detecting drift between the serving distribution (P_t) and training distribution (P_0) requires measuring the “distance” between distributions.

For discrete outcomes x , equation B.1 expresses KL divergence³ as the expected extra coding cost of approximating P_t with P_0 (Kullback and Leibler 1951; Cover and Thomas 2006). The displayed form is finite only when $P_0(x) > 0$ wherever $P_t(x) > 0$; production implementations therefore need explicit smoothing or zero handling.

$$\mathcal{D}_{\text{KL}}(P_t \| P_0) = \sum_x P_t(x) \log \frac{P_t(x)}{P_0(x)} \quad (\text{B.1})$$



Napkin Math B.2: Worked example: KL divergence for drift detection

Scenario: A sentiment classifier was trained on data where 60 percent of reviews were positive, 30 percent negative, and 10 percent neutral. After deployment, the serving distribution shifts to 45 percent positive, 40 percent negative, and 15 percent neutral.

Math:

Training distribution P_0 : [0.60, 0.30, 0.10]. Serving distribution P_t : [0.45, 0.40, 0.15].

$$\begin{aligned} \mathcal{D}_{\text{KL}}(P_t \| P_0) &= 0.45 \log \frac{0.45}{0.60} + 0.40 \log \frac{0.40}{0.30} + 0.15 \log \frac{0.15}{0.10} \\ &= 0.45 \times (-0.2877) + 0.40 \times (0.2877) + 0.15 \times (0.4055) \\ &= -0.1295 + 0.1151 + 0.0608 = 0.0464 \text{ nats} \end{aligned}$$

$$\begin{aligned} \mathcal{D}_{\text{KL}}(P_0 \| P_t) &= 0.60 \log \frac{0.60}{0.45} + 0.30 \log \frac{0.30}{0.40} + 0.10 \log \frac{0.10}{0.15} \\ &= 0.60 \times 0.2877 + 0.30 \times (-0.2877) + 0.10 \times (-0.4055) \\ &= 0.1726 + (-0.0863) + (-0.0405) = 0.0458 \text{ nats} \end{aligned}$$

Notice the asymmetry: $\mathcal{D}_{\text{KL}}(P_0 \| P_t) \neq \mathcal{D}_{\text{KL}}(P_t \| P_0)$. The population stability index (PSI) symmetrizes this; the calculation below uses natural logarithms:

$$\begin{aligned} \text{PSI} &= \sum (P_{0,i} - P_{t,i}) \log \frac{P_{0,i}}{P_{t,i}} = (0.15)(0.2877) + (-0.10)(-0.2877) + (-0.05)(-0.4055) \\ &= 0.04315 + 0.02877 + 0.02027 = 0.0922 \end{aligned}$$

Systems insight: Since $\text{PSI} = 0.0922 < 0.2$, this drift does not yet cross the illustrative manual-review threshold used here. The right response is to keep the feature on the monitoring dashboard and look for corroborating drift signals rather than retraining from this statistic alone.

The threshold remains illustrative: its meaning depends on sample size, binning, smoothing, and zero handling, and retraining requires model-quality evidence.

Both KL divergence and PSI are grounded in a deeper framework—information theory—which provides the units and bounds that make these metrics principled rather than ad hoc.

³ | **KL divergence:** Named after Solomon Kullback and Richard Leibler, who introduced it in 1951. Also called *relative entropy*, it quantifies the expected extra information needed to encode samples from one distribution using a code optimized for another; for drift monitoring in this book, the canonical direction is $\mathcal{D}_{\text{KL}}(P_t \| P_0)$. With natural logarithms, as in the example above, the unit is nats; with base-2 logarithms, the unit is bits. In production ML, KL divergence is the theoretical backbone of many drift-detection metrics.

B.2.3 Information theory for systems

A training run can consume more examples, run longer, and still stop improving if the added data carries too little useful signal. At that point, the bottleneck is not only compute or bandwidth; it is the amount of information the data pipeline delivers to the learner. Section A.11.2 treats that data quality limit as a diagnostic constraint, and information theory provides the units for reasoning about it.

Entropy (H)⁴ is the average uncertainty in a distribution, defined as $H(X) = -\sum p(x) \log p(x)$. The log base sets the unit: natural logs give nats, while $\log_2$ gives bits. A uniform distribution has maximum entropy only over a fixed finite support. This connects directly to KL divergence above: $\mathcal{D}_{\text{KL}}$ measures the excess coding cost of using the wrong distribution.

Information density is used here as a book-level heuristic for useful learning signal per unit of storage, not as a standard information-theory quantity.

Signal-to-noise ratio (SNR) is the ratio of signal power to noise power. In ML, the book uses low SNR qualitatively for data in which task-relevant structure is weak relative to noise; adding compute cannot compensate for missing task-relevant signal.

B.2.4 Logits and numerical stability

Neural-network classifiers commonly output a vector of logits⁵ $\mathbf{z}$ (unnormalized scores). For class index i , softmax normalizes z_i over all class indices j to produce a probability:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

The problem is that if z_i is large (for example, 100), the exponential e^{z_i} overflows common training and inference formats such as FP32, BF16, and FP16. FP64 can represent this specific value, but production ML kernels rarely use FP64 for softmax. The solution is to compute in log-space: the “Log-Sum-Exp” trick allows us to compute $\log(\sum e^{z_j})$ without ever calculating the massive exponentials directly, preserving numerical precision.⁶

A small worked example shows why this trick matters in practice, even for large but realistic logit values:

Napkin Math B.3: Log-sum-exp in action

Setup: A three-class classifier outputs logits $\mathbf{z} = [100, 101, 102]$.

Without the trick (naive softmax):

$$\exp(100) \approx 2.7 \times 10^{43}, \exp(101) \approx 7.3 \times 10^{43}, \exp(102) \approx 2.0 \times 10^{44}$$

These numbers are representable in FP64 but overflow FP32 ($\max \approx 3.4 \times 10^{38}$). With FP16 ($\max \approx 65,504$), even e^{12} overflows. Raw logits rarely reach magnitude 100, but large intermediate values do arise (for example, unscaled attention scores or overconfident outputs), and because FP16 and BF16 are the standard training precisions, a naive softmax risks overflow often enough that the stable form is used by default.

With the trick: Subtract $a = \max(\mathbf{z}) = 102$:

Shifted logits: $\mathbf{z} - a = [-2, -1, 0]$

Exponentials: $\exp(-2) \approx 0.135$, $\exp(-1) \approx 0.368$, $\exp(0) = 1.0$

Sum = 1.503. LogSumExp = $102 + \log(1.503) = 102.408$.

Softmax: $[0.135/1.503, 0.368/1.503, 1.0/1.503] = [0.090, 0.245, 0.665]$

Systems insight: The exponentiated shifted values and the resulting softmax probabilities are in $[0, 1]$ —no overflow risk, even in FP16.

⁴ | **Entropy:** Shannon adopted the term in 1948 to quantify information content (Shannon 1948). In systems terms, Shannon entropy is a lower bound on average lossless code length. Practical prefix codes approach it, and block codes can approach it asymptotically. With $\log_2$, the unit is bits; natural logs give nats. These bounds inform compression ratios and data-pipeline sizing.

⁵ | **Logit:** Joseph Berkson coined the term in 1944 by analogy with *probit*. The logit function is the inverse of the logistic (sigmoid) function: $\text{logit}(p) = \log(p/(1-p))$. In deep learning, “logits” refers more loosely to the raw, unnormalized output of the final linear layer before any activation function is applied.

⁶ | **Log-sum-exp:** Implemented as `torch.logsumexp` in PyTorch and `scipy.special.logsumexp` in SciPy. It relies on the identity $\log(\sum e^{x_i}) = a + \log(\sum e^{x_i - a})$, where $a = \max(x_i)$. Shifted values $x_i - a$ are ≤ 0 , ensuring exponentials never overflow.

Together, these worked examples connect the full chain, from physical data movement to distribution shift and numerical stability.

B.3 Summary

The tools in this appendix—tail-aware metrics, drift divergences, information-theoretic bounds, and numerical stability techniques—support quantitative diagnosis of data-related failures rather than anecdotal explanations.

Algorithm Foundations

A profile showing low utilization is often misdiagnosed as a hardware limit when the cause is algorithmic: a matrix multiply shape that wastes tensor cores, a layout that breaks contiguity, or activation memory held for backpropagation. This appendix collects the computational complexity models and memory footprint derivations that the book’s performance chapters lean on, enabling readers to evaluate algorithmic trade-offs before committing code to accelerators. It assumes familiarity with linear algebra and with the iron law introduced early in the book.

How to Use This Appendix

This appendix is designed as a reference. Reach for it when translating a profiler symptom (“slow matmul,” “shape mismatch,” “OOM during training”) into a concrete computational or memory cause.

Conventions used here follow the book-wide notation (for example, B is reserved for batch size and BW denotes bandwidth).

- **When general matrix multiply (GEMM) kernels are slow:** Use section C.1.4 and compare intensity to the hardware’s ridge point.
- **When memory blows up in training:** Use section C.3.3 and the training memory decomposition.
- **When tensor code “should work” but does not:** Use section C.2.2 and section C.2.3.
- **When sparsity is proposed as a fix:** Use section C.1.5 to check density and metadata overhead.
- **When proving gradient compute bounds:** Use section C.4 for the Baur–Strassen reverse-mode automatic differentiation (AD) complexity theorem.
- **When analyzing sequence length bottlenecks:** Use section C.5 for the online softmax recurrence and FlashAttention high-bandwidth memory (HBM) I/O reduction proof.
- **When allocating compute between parameters and tokens:** Use section C.6 for the Kaplan and Chinchilla compute-optimal scaling law derivation.

Neural networks turn linear algebra into learned behavior: matrices transform activations, tensor layouts determine how data move through memory, and backpropagation carries error signals through the resulting computation. These foundations support the deep learning treatment in chapter 5, the framework internals in chapter 7, and the training strategies in chapter 8. Architectures change, but the underlying mathematical machinery remains constant.

C.1 Linear Algebra

Deep learning systems are, at their core, engines for transforming massive matrices. Frameworks like PyTorch abstract away the raw math, but performance engineering still depends on the underlying linear algebra. Building on how numbers are stored in section D.4, this section focuses on how they are manipulated.

Systems Perspective C.1: Why this matters

Many dense neural networks, especially transformers and large convolutional neural networks, spend most compute in matrix multiplication or GEMM-like kernels. A self-attention block performs the Q, K, V, and output projection GEMMs plus two attention matrix multiplications:

one for the attention scores and one for the attention-weighted values. The layer’s feed-forward block adds more GEMMs. Understanding GEMM performance characteristics explains why batch size affects throughput, why certain layer dimensions are “better” than others, and how to interpret profiler output. Reasoning about matrix dimensions and arithmetic intensity predicts whether a dense workload is compute bound or memory bound *before* any profiler trace runs.

C.1.1 Tensor operations and notation

Einstein summation¹ notation makes complex operations explicit throughout this book (implemented as `torch.einsum` in PyTorch and `np.einsum` in NumPy). Matrix multiplication $\mathbf{C} = \mathbf{AB}$ becomes:

$$C_{ij} = \sum_k A_{ik} B_{kj}$$

In `einsum` notation, this is `ik,kj->ij`. The notation extends naturally to the multi-dimensional operations in attention mechanisms. For example, batched multi-head attention is `bhjd,bhjd->bhi j` (batch b , head h , query sequence i , key sequence j , and head dimension d).

C.1.2 Memory layouts and performance

Data layout in memory (row-major vs. column-major) directly affects cache efficiency. When iterating over a matrix, accessing contiguous memory locations is dramatically faster than strided access.

A common optimization pattern is to materialize a transposed tensor once before repeated operations to ensure contiguous access in the hot loop. The one-time copy cost is amortized across many subsequent operations.

C.1.3 The dot product as similarity

The dot product $\mathbf{a} \cdot \mathbf{b} = \sum a_i b_i$ is geometrically equivalent to $\|\mathbf{a}\| \|\mathbf{b}\| \cos \phi$, which makes it a natural measure of similarity between two vectors: for nonzero vectors, its sign distinguishes acute, right, and obtuse angles, while its magnitude also depends on both vector norms.

This geometric interpretation is why dot products appear everywhere in modern architectures. In attention mechanisms, query (Q) and key (K) vectors are dot-produced to compute a similarity score that determines how much each token attends to every other token. The resulting attention weights are then used to form a weighted combination of value (V) vectors—making the dot product the foundation of the transformer’s ability to model long-range dependencies.

C.1.4 General matrix multiply (GEMM)

GEMM² is the computational workhorse of deep learning. For matrices of size $M \times K$ and $K \times N$, GEMM performs $2MNK$ floating-point operations (multiply-accumulate counts as two operations).

The arithmetic intensity of GEMM scales linearly with matrix dimension. For square $n \times n$ matrices in FP16 (2 bytes/element), the ideal $\beta = 0$ bound reads $\mathbf{A}$ and $\mathbf{B}$ once and writes $\mathbf{C}$ once:

$$\text{Intensity} = \frac{O}{D_{\text{vol}}} = \frac{2n^3}{3n^2 \times 2} = \frac{n}{3} \text{ FLOP/byte}$$

Reading the old $\mathbf{C}$ when $\beta \neq 0$ changes the intensity to $n/4$ FLOP/byte. Three systems consequences govern GEMM execution:

- **Larger batches can improve efficiency:** Batching increases effective matrix dimensions when it forms larger GEMMs or enables reuse; merely queuing independent small GEMMs does not raise intensity.
- **Aligned dimensions help:** Hardware tensor cores are optimized for precision- and architecture-specific tile multiples. Dimensions that align with these multiples avoid padding overhead and improve kernel efficiency, but they do not need to be powers of two.
- **Small matrices are inefficient:** A square GEMM with $n = 64$ has intensity $64/3 \approx 21.3$ FLOP/byte, well below the ridge point (153.0 FLOP/byte). Its roofline cap is ~ 13.9 percent of peak; launch and tiling overhead can lower realized throughput.

¹ | **Einstein summation convention:** Repeated indices in a product are implicitly summed over, eliminating explicit summation signs. ML frameworks adopted the convention because it concisely expresses arbitrary tensor contractions in a single string.

² | **General matrix multiply (GEMM):** The BLAS (Basic Linear Algebra Subprograms) family began with vector routines standardized in 1979 (Lawson et al. 1979); GEMM was standardized later as a Level 3 routine (Dongarra et al. 1988). The “GE” prefix stands for “general,” as opposed to symmetric or triangular forms. GEMM computes $\mathbf{C} = \alpha \mathbf{AB} + \beta \mathbf{C}$ and is a performance-critical routine in deep learning. Section 8.3.1.2 explains how GEMM shapes determine training throughput.

C.1.5 Sparse matrix formats

When most elements in a matrix are zero, specialized storage formats avoid wasting memory on zeros and enable computations that skip them entirely. The **compressed sparse row (CSR)** format uses three arrays:

- **Values:** The nonzero elements, stored in row order
- **Col_Idx:** The column index of each nonzero element
- **Row_Ptr:** The starting position in **Values** for each row (length = num_rows + 1)

CSR is useful for sparse feature matrices in recommendation pipelines and for pruned model weights. For a matrix with N elements, R rows, and K nonzeros, CSR uses $\mathcal{O}(K + R)$ storage instead of $\mathcal{O}(N)$; when K is large relative to R , this is often summarized as $\mathcal{O}(K)$.

To see the trade-off concretely, consider a vocabulary embedding matrix with 100,000 rows and 10,000 columns (1B parameters):

- **Dense (FP32):** 1B parameters $\times$ 4 bytes each = 4 GB.
- **Sparse (1 percent density):** CSR stores roughly 10M entries $\times$ (4-byte value + 4-byte column index), plus one row pointer per row and a final pointer, totaling $\approx$ 80.4 MB.
- **Result:** A 50 $\times$ reduction in memory footprint, fitting a model that would otherwise OOM (Out of Memory).

Linear algebra tells us *what* to compute; the next question is *how* to express those computations in code. Tensor programming primitives—shapes, strides, and broadcasting—bridge the gap between mathematical notation and the array operations that actually execute on hardware.

C.2 Tensor Programming Primitives

A shape mismatch crash, a silently wrong broadcast, a kernel running at 5 percent of peak because of a noncontiguous tensor—these common ML engineering failures all trace back to the same layer of abstraction. *Tensor programming* translates the abstract math of linear algebra into concrete array manipulations that run on hardware.

C.2.1 Computational complexity cheat sheet

Table C.1 provides a quantitative reference for the most common building blocks. Use these formulas for napkin-math estimation of model size and compute requirements *before* hardware is provisioned. Given layer dimensions and input shapes, the formulas provide first-order parameter and compute estimates. The attention row is the key warning: sequence length contributes an S^2 term, while hidden dimension contributes d^2 projection work, so long-context models can shift the dominant cost without changing the parameter count.

Table C.1: Deep Learning Tensor Primitives: Summary of shapes, parameters, and FLOP counts. Note: B is batch size, S is sequence length, and K is kernel size. The attention FLOPs include QKV projections and score interactions. The S^2 term applies to full-sequence training and prefill, while KV-cached generation is linear per token in cached context; d^2 projection work dominates at large hidden dimensions.

Layer Type	Output Shape	Parameters (P)	FLOPs (per Forward Pass)
Linear	(B, N_{out})	$N_{\text{in}} \times N_{\text{out}} + N_{\text{out}}$ if bias is enabled	$2 \times B \times N_{\text{in}} \times N_{\text{out}}$
Conv2D	$(B, C_{\text{out}}, H', W')$	$K^2 \times C_{\text{in}} \times C_{\text{out}} + C_{\text{out}}$ if bias is enabled	$2 \times B \times H' \times W' \times K^2 \times C_{\text{in}} \times C_{\text{out}}$
Multi-Head Self-Attention	(B, S, d_{model})	$4 \times d_{\text{model}}^2$, plus $4 \times d_{\text{model}}$ if projection biases are enabled	$B \times (4S^2 d_{\text{model}} + 8Sd_{\text{model}}^2)$
LayerNorm	(B, S, d_{model})	$2 \times d_{\text{model}}$ if affine scale and bias are enabled	$\mathcal{O}(B \times S \times d_{\text{model}})$

C.2.2 Shapes and strides

A tensor is a view over an underlying storage buffer, described by shape, stride, dtype, and offset metadata. Only contiguous tensors lay their logical elements out as one adjacent block.

- **Shape:** The dimensions of the tensor (for example, (3, 4)).
- **Stride:** The number of elements to skip in memory to move to the next element in a dimension.

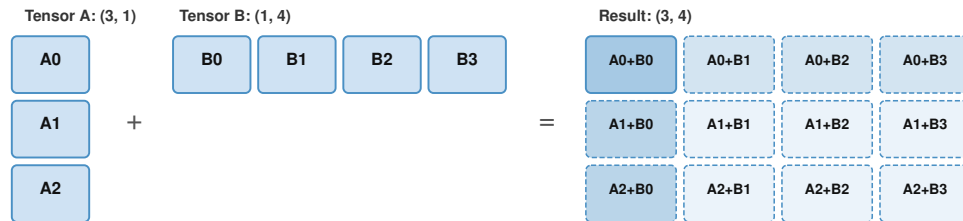
Operations like `transpose()` or `view()` often just change the *strides*, not the data in memory. This is fast ($\mathcal{O}(1)$) but can lead to noncontiguous tensors that fail in kernels requiring contiguous data. In such cases, calling `contiguous()` forces an $\mathcal{O}(N)$ memory copy that can dominate runtime if triggered repeatedly inside a loop.

C.2.3 Broadcasting

Broadcasting allows arithmetic operations on tensors of different shapes. Compare dimensions from the last to the first. Two dimensions are compatible when:

1. They are equal.
2. One of them is 1.

The dimension with size one is “stretched” to match the other, as illustrated in figure C.1. This stretching is virtual: the data is not copied in memory. Instead, the stride for that dimension is set to 0, allowing the hardware to read the same value repeatedly with $\mathcal{O}(1)$ memory overhead.



Dashed cells are virtually expanded via stride=0 (no memory allocation)

Figure C.1: Tensor Broadcasting Rules: A (3,1) column and a (1,4) row broadcast across complementary dimensions, producing the pairwise sums in a (3,4) result.

Consider a concrete case: tensor A has shape (32, 1, 64) and tensor B has shape (1, 128, 64). Comparing dimensions right to left, 64 matches 64, then 1 stretches to 128, then 1 stretches to 32, yielding result shape (32, 128, 64). Visualizing this expansion prevents silent logic bugs that accidentally allocate a large tensor (for example, a (Batch, Batch) matrix instead of an element-wise (Batch) vector).

Shapes, strides, and broadcasting govern how tensors flow through a model’s forward pass. Training adds a second requirement: *learning from errors*. Backpropagation makes that learning possible and imposes the memory costs developed below.

C.3 Mechanics of Learning

Valid tensor programs make training loops possible. Backpropagation orchestrates these tensors to compute gradients, transforming a forward prediction into a backward learning signal.

🔍 Systems Perspective C.2: Why this matters

When training fails—loss goes to NaN, gradients explode, or memory runs out—understanding what backpropagation actually does is essential for diagnosing the problem. The backward graph supplies the mental model for reasoning about gradient flow and memory usage during training.

C.3.1 The chain rule and automatic differentiation

For a composed function $y = f(g(x))$, the derivative is $\frac{dy}{dx} = \frac{dy}{dg} \cdot \frac{dg}{dx}$. In a neural network, f and g are layers, and the composition can be many levels deep. For a three-layer network $y = f_3(f_2(f_1(x)))$, the chain rule extends to:

$$\frac{\partial y}{\partial x} = \frac{\partial f_3}{\partial f_2} \cdot \frac{\partial f_2}{\partial f_1} \cdot \frac{\partial f_1}{\partial x}$$

Each factor in this product is a *local* derivative—computed at one layer using only that layer’s inputs and outputs. This locality is what makes the algorithm tractable: the entire network never needs to be differentiated as a monolithic function. Instead, each layer computes its own local derivative during the backward pass and multiplies it by the gradient flowing in from the layer above.

Modern frameworks use reverse-mode automatic differentiation, which computes gradients for all P parameters in a single backward pass. The key insight is that starting from the output and working backward (reverse mode) requires one pass regardless of the number of parameters, whereas starting from each input and working forward (forward mode) would require P passes—one per parameter. This is why a training step is a small constant multiple of inference, commonly about $2\text{--}3\times$ a forward pass for dense networks, rather than P passes.

C.3.2 The backpropagation algorithm

Backpropagation³ implements the chain rule efficiently through two passes: forward to compute outputs, backward to compute gradients. Figure C.2 illustrates this process for a simple two-layer network, with the forward pass (gray arrows) computing outputs and the backward pass (red dashed arrows) propagating gradients.

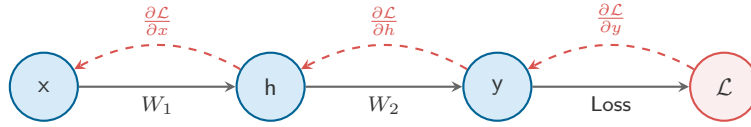


Figure C.2: Backpropagation Computational Graph: Solid gray arrows carry the forward computation from input x to loss $\mathcal{L}$, while dashed red arrows carry gradients backward along the same dependencies.

The gradient edges retrace the forward dependencies, so each backward step requires values retained from the corresponding forward operation.

Forward pass

Using row-vector activations, omitting biases, and retaining intermediate activations for backward, start at x , the input. Multiply by W_1 to get hidden activation h . Cache h because the backward pass will need it later. Multiply h by W_2 to get output y . Cache y . Compare y to the target label to compute loss $\mathcal{L}$.

At this point, the loss has been computed and memory contains the input x , the cached activation h , the cached output y , and the loss $\mathcal{L}$. For a large model, these cached activations can dominate memory usage.

Backward pass

Now trace backward from $\mathcal{L}$. The loss function provides $\frac{\partial \mathcal{L}}{\partial y}$, the gradient of loss with respect to the prediction. This is where the error signal enters the network.

Since $y = h \cdot W_2$, the chain rule gives us two gradients at this layer:

$$\frac{\partial \mathcal{L}}{\partial W_2} = h^T \cdot \frac{\partial \mathcal{L}}{\partial y} \quad (\text{weight gradient—used to update } W_2)$$

$$\frac{\partial \mathcal{L}}{\partial h} = \frac{\partial \mathcal{L}}{\partial y} \cdot W_2^T \quad (\text{input gradient—passed backward to the previous layer})$$

³ | **Backpropagation:** Short for “backward propagation of errors.” The algorithm was independently discovered multiple times—by Werbos (1974) and Linnainmaa (1970) for reverse-mode differentiation and by Rumelhart et al. (1986) for neural network training. Its key insight is that computing gradients for *all* parameters requires one backward pass through the graph rather than one pass per parameter. Section C.4 derives the constant-factor work bound and a representative dense-layer cost.

Computing $\frac{\partial \mathcal{L}}{\partial W_2}$ requires the cached activation h from the forward pass. Activations must remain available during backward because every layer's weight gradient depends on that layer's input.

Now continue backward to the first layer. Since $h = x \cdot W_1$, the same pattern gives:

$$\frac{\partial \mathcal{L}}{\partial W_1} = x^T \cdot \frac{\partial \mathcal{L}}{\partial h}$$

Each step backward requires two things: the gradient flowing in from above ($\frac{\partial \mathcal{L}}{\partial h}$) and that layer's input (x), which must be available during backward. This is why the backward pass costs roughly $2\times$ the forward pass in compute: at each layer, it performs two matrix multiplications (one for the weight gradient, one for the input gradient) vs. one in the forward pass.

C.3.3 The true cost of training memory

A common mistake is to assume that training memory equals model size. This assumption leads to immediate OOM errors because weights are only one of four components. Training memory comprises weights, gradients, optimizer state, and retained activations:

$$M_{\text{total}} = M_{\text{weights}} + M_{\text{gradients}} + M_{\text{optimizer}} + M_{\text{activations}}$$

For a standard Adaptive Moment Estimation (Adam) optimizer (Kingma and Ba 2015) in mixed precision (Micikevicius et al. 2017):

- **Weights:** 2 bytes (FP16/BF16) or 4 bytes (FP32).
- **Gradients:** Same size as weights.
- **Optimizer State:** Adam moments require 8 bytes per parameter; an FP32 master copy adds another 4 bytes, giving 8–12 bytes per parameter.
- **Activations:** The hidden giant. A recomputation- or tiled-attention-friendly estimate scales as $\mathcal{O}(B \times S \times N_L \times d)$; a full no-recompute attention implementation also materializes attention-score tensors that scale with S^2 .

To see how these components interact in practice, consider a concrete model.



Napkin Math C.1: Worked example: GPT-2 (1.5B) training memory

The model: GPT-2 XL has $P = 1.5 \times 10^9$ parameters, 48 layers, hidden dimension $d = 1600$.

Model state (fixed per step):

- **Weights (BF16):** $1.5 \times 10^9 \times 2 \text{ bytes} = 3 \text{ GB}$
- **Gradients (BF16):** $1.5 \times 10^9 \times 2 \text{ bytes} = 3 \text{ GB}$
- **Optimizer (FP32 master + momentum + variance):** $1.5 \times 10^9 \times 12 \text{ bytes} = 18 \text{ GB}$
- **Total model state:** 24 GB—fits on an 80 GB-class A100/H100 (85.9 GB in decimal units), but leaves only 61.9 GB for activations.

Activations (scale with batch):

Per-layer retained activations for this quick estimate are approximately $12 \times B \times S \times d$ BF16 elements, or $12 \times B \times S \times d \times 2$ bytes, where B is batch size and S is sequence length. The factor twelve accounts for the major intermediate tensors retained for backpropagation: input activations, QKV projections ($3d$), attention output, FFN intermediate ($4d$), and layer norm/dropout masks. This estimate assumes the implementation does not retain the full $B \times N_{\text{heads}} \times S^2$ attention-score tensor, as in recomputation- or tiled-attention-friendly accounting. With 48 layers, batch size 8, and sequence length 1024:

$$48 \times 12 \times 8 \times 1024 \times 1600 \times 2 \text{ bytes} \approx 15.1 \text{ GB}$$

Systems insight: ~39.1 GB under this activation-accounting convention, which fits on one 85.9 GB accelerator. However, increase the batch to 64 and activations grow to ~120.8 GB, exceeding the remaining capacity. This is the threshold where gradient checkpointing or

another memory-saving method becomes necessary; its savings and recomputation overhead depend on checkpoint placement and implementation.

C.3.3.1 Activation explosion

While weights are fixed ($\mathcal{O}(P)$), activations grow linearly with batch size and at least linearly with sequence length; implementations that materialize attention scores add an $\mathcal{O}(S^2)$ component. As the worked example shows, activation memory can quickly exceed the room left after model state. Gradient checkpointing⁴ reduces this pressure by storing only selected activations and recomputing the rest during backpropagation (T. Chen et al. 2016); techniques such as FlashAttention address related attention-memory bottlenecks by tiling attention to reduce memory round-trips (T. Dao et al. 2022).

The same decomposition provides a quick feasibility check: sum the per-parameter model state, add the activation term implied by batch size and sequence length, and compare the total against the accelerator’s capacity.

C.3.4 Computational graphs and optimization

The dependency structure exposed by backpropagation also gives compilers something to optimize. ML compilers represent models as directed acyclic graphs (DAGs), and that representation enables hardware-independent transformations.

C.3.4.1 Static single assignment

Compilers transform graphs into static single-assignment (SSA) form, where each variable is assigned exactly once. This makes data dependencies explicit, enabling safe optimizations—most importantly, *operator fusion*.

C.3.4.2 Operator fusion

Without fusion, each operation in a chain like $\text{MatMul} \rightarrow \text{Add (bias)} \rightarrow \text{ReLU}$ produces an intermediate tensor that is written to HBM and then read back for the next operation. For elementwise operations like Add and ReLU, the compute is trivial (one FLOP per element) but the memory traffic is not (read the tensor, write it back). The arithmetic intensity of unfused elementwise operations is therefore close to zero—deeply memory bound.

Fusion combines consecutive operations into a single kernel that reads the input once, applies all operations in registers or shared memory, and writes the final result once. For a sequence of k elementwise operations on a tensor of size N bytes, fusion reduces memory traffic from $2kN$ bytes (each op reads and writes) to $2N$ bytes (one read, one write)—a $k\times$ reduction.

FlashAttention is an I/O-aware tiled attention algorithm: it computes the score, softmax, and value-product stages in SRAM tiles without materializing the full attention matrix in HBM, reducing HBM traffic and auxiliary attention memory from $\mathcal{O}(S^2)$ to $\mathcal{O}(S)$. The original work reports up to $3\times$ wall-clock speedups on its evaluated workloads (T. Dao et al. 2022); realized gains depend on shape and hardware (see section C.5 for the step-by-step derivation). Together, the linear algebra foundations, tensor programming mechanics, and training memory model covered in this appendix form the algorithmic substrate on which all ML systems are built.

C.4 Baur–Strassen Reverse-Mode AD Complexity Theorem

Modern deep learning scales to hundreds of billions of parameters because evaluating the full gradient vector $\nabla f(x)$ of a scalar loss function $f : \mathbb{R}^N \rightarrow \mathbb{R}$ requires compute proportional to evaluating $f(x)$ itself, with a constant-factor bound independent of the input dimension N . Baur and Strassen proved this complexity result in 1983 (Baur and Strassen 1983).

C.4.1 Mathematical formulation and theorem statement

Let $f : \mathbb{R}^N \rightarrow \mathbb{R}$ be a multivariate scalar-valued function represented as a computational graph $G = (V, E)$ consisting of $W(f)$ elementary operations (additions, subtractions, multiplications, divisions, and unary functions such as $\exp, \ln, \sin, \sqrt{\cdot}$).

⁴ | **Gradient checkpointing:** Trades compute for memory. Instead of storing all activations, only a subset (checkpoints) is kept, and the missing ones are recomputed during the backward pass. This reduces memory usage from storing every layer activation to roughly the square root of the layer count at the cost of ~ 33 percent more compute (T. Chen et al. 2016).


Theorem C.1: Baur–Strassen complexity theorem

If a scalar function $f : \mathbb{R}^N \rightarrow \mathbb{R}$ can be computed in $W(f)$ elementary operations, then the full gradient vector $\nabla f(x) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_N} \right)^T \in \mathbb{R}^N$ can be evaluated in total work $\text{Work}_{\text{Reverse}}(f, \nabla f)$ satisfying:

$$\text{Work}_{\text{Reverse}}(f, \nabla f) \leq C \cdot W(f)$$

where $C \leq 5$ is a universal constant independent of the input dimension N . Furthermore, the backward pass alone requires work $\text{Work}_{\text{Backward}}(\nabla f) \leq 4 \cdot W(f)$.

C.4.2 Step-by-step chain rule adjoint graph propagation proof

Let the computational graph $G = (V, E)$ be topologically ordered with nodes $v_{1-N}, \dots, v_0, v_1, v_2, \dots, v_W$, where input variables are $x_i = v_{i-N}$ for $i \in \{1, \dots, N\}$ and the final output is $f(x) = v_W$. Each intermediate node v_j is evaluated via a primitive operation:

$$v_j = \phi_j(\{v_k \mid k \in \text{Parents}(j)\})$$

Define the *adjoint variable* $\bar{v}_i$ for every node v_i as the partial derivative of the scalar output v_W with respect to v_i :

$$\bar{v}_i \triangleq \frac{\partial f}{\partial v_i} = \frac{\partial v_W}{\partial v_i}$$

By the multivariate chain rule, the adjoint $\bar{v}_i$ is the sum of partial derivatives propagated from all successor nodes (children) that consume v_i :

$$\bar{v}_i = \sum_{j \in \text{Children}(i)} \bar{v}_j \cdot \frac{\partial v_j}{\partial v_i}$$

Reverse-mode automatic differentiation initializes $\bar{v}_W = 1$ and $\bar{v}_i = 0$ for all $i < W$, then traverses the nodes in reverse topological order from v_W down to v_1 . For each primitive node v_j , its fully accumulated adjoint $\bar{v}_j$ propagates error signals to its parent nodes v_k ($k \in \text{Parents}(j)$) via:

$$\bar{v}_k \leftarrow \bar{v}_k + \bar{v}_j \cdot \frac{\partial v_j}{\partial v_k}$$

We now perform operation-by-operation micro-cost accounting for all primitive nodes to bound the backward work:

1. Addition/Subtraction ($v_j = v_i \pm v_k$):

- Local derivatives: $\frac{\partial v_j}{\partial v_i} = 1, \frac{\partial v_j}{\partial v_k} = \pm 1$.
- Backward updates: $\bar{v}_i \leftarrow \bar{v}_i + \bar{v}_j$ and $\bar{v}_k \leftarrow \bar{v}_k \pm \bar{v}_j$.
- Cost: 2 additions in backward pass for 1 forward operation. Work ratio: $\text{Fwd}(1) + \text{Bwd}(2) = 3 \text{ OPs} \leq 5 \cdot 1$.

2. Multiplication ($v_j = v_i \cdot v_k$):

- Local derivatives: $\frac{\partial v_j}{\partial v_i} = v_k, \frac{\partial v_j}{\partial v_k} = v_i$.
- Backward updates: $\bar{v}_i \leftarrow \bar{v}_i + \bar{v}_j \cdot v_k$ and $\bar{v}_k \leftarrow \bar{v}_k + \bar{v}_j \cdot v_i$.
- Cost: 2 multiplications + 2 additions = 4 operations in backward pass for 1 forward operation. Work ratio: $\text{Fwd}(1) + \text{Bwd}(4) = 5 \text{ OPs} \leq 5 \cdot 1$.

3. Division ($v_j = v_i / v_k$):

- Local derivatives: $\frac{\partial v_j}{\partial v_i} = \frac{1}{v_k}, \frac{\partial v_j}{\partial v_k} = -\frac{v_i}{v_k^2} = -\frac{v_j}{v_k}$.
- Backward updates: $\bar{v}_i \leftarrow \bar{v}_i + \frac{\bar{v}_j}{v_k}$ and $\bar{v}_k \leftarrow \bar{v}_k - \frac{\bar{v}_j \cdot v_j}{v_k}$.

- Cost: 2 multiplications/divisions + 2 additions = 4 operations in backward pass for 1 forward operation. Work ratio: $\text{Fwd}(1) + \text{Bwd}(4) = 5 \text{ OPs} \leq 5 \cdot 1$.

4. Unary Transcendental Operations ($v_j = \phi(v_i)$):

- Local derivative: $\frac{\partial v_j}{\partial v_i} = \phi'(v_i)$.
- Backward update: $\bar{v}_i \leftarrow \bar{v}_i + \bar{v}_j \cdot \phi'(v_i)$.
- For $v_j = \exp(v_i)$, $\phi'(v_i) = v_j$, requiring 1 multiplication + 1 addition = 2 OPs.
- For $v_j = \ln(v_i)$, $\phi'(v_i) = 1/v_i$, requiring 1 division + 1 addition = 2 OPs.

Summing across all primitive nodes in G , every forward operation generates at most 4 backward operations:

$$\text{Work}_{\text{Backward}}(\nabla f) \leq 4 \cdot W(f)$$

Adding the forward pass work $W(f)$ yields the total work bound:

$$\text{Work}_{\text{Reverse}}(f, \nabla f) = W(f) + \text{Work}_{\text{Backward}}(\nabla f) \leq 5 \cdot W(f) \quad \blacksquare$$

Systems Perspective C.3: Why 175B-parameter large language models can be trained

The independence of $C \leq 5$ from input dimension N is the mathematical reason deep learning works. Consider training a transformer model with $P = 175 \times 10^9$ parameters:

- **Forward Pass:** Matrix multiplications $Y = XW$ require $2P$ FLOPs per token.
- **Reverse-Mode Backward Pass:** Computes both weight gradients $\frac{\partial \mathcal{L}}{\partial W} = X^T \frac{\partial \mathcal{L}}{\partial Y}$ ($2P$ FLOPs) and input activation gradients $\frac{\partial \mathcal{L}}{\partial X} = \frac{\partial \mathcal{L}}{\partial Y} W^T$ ($2P$ FLOPs), totaling $4P$ FLOPs per token.
- **Total Training Work:** $2P(\text{fwd}) + 4P(\text{bwd}) = 6P$ FLOPs per token. The backward compute factor is exactly $\frac{4P}{2P} = 2$, well within the Baur–Strassen theoretical bound of $C \leq 5$.

If we instead used forward-mode AD or finite differences to compute $\nabla f \in \mathbb{R}^P$, evaluating each partial derivative separately would require P forward passes: $\text{Work}_{\text{ForwardMode}}(\nabla f) = \mathcal{O}(P \cdot W) = \mathcal{O}(P^2)$ FLOPs. For $P = 175 \times 10^9$, that method would require $2 \times (175 \times 10^9)^2 \approx 6.1 \times 10^{22}$ FLOPs per token— P times the forward-pass work and therefore prohibitive at this scale. Reverse-mode AD collapses this $\mathcal{O}(P)$ factor into a constant multiplier of 2 for the dense-layer accounting above.

C.5 FlashAttention Tiling & Online Softmax Recurrence Proof

Standard self-attention in transformers requires materializing an $N \times N$ attention score matrix in HBM, creating an $\mathcal{O}(N^2)$ memory bandwidth bottleneck. FlashAttention (T. Dao et al. 2022) eliminates this bottleneck by tiling query, key, and value matrices and recomputing attention using an online softmax recurrence without materializing intermediate scores to HBM.

C.5.1 Standard softmax HBM I/O bottleneck

Given queries $\mathbf{Q} \in \mathbb{R}^{N \times d}$, keys $\mathbf{K} \in \mathbb{R}^{N \times d}$, and values $\mathbf{V} \in \mathbb{R}^{N \times d}$ for sequence length N and head dimension d , standard attention evaluates:

$$\mathbf{S} = \mathbf{QK}^T \in \mathbb{R}^{N \times N}, \quad \mathbf{A} = \text{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{AV} \in \mathbb{R}^{N \times d}$$

To prevent numerical overflow, row-wise softmax subtracts the row maximum $m_i = \max_{1 \leq j \leq N} S_{ij}$:

$$A_{ij} = \frac{\exp(S_{ij} - m_i)}{\sum_{k=1}^N \exp(S_{ik} - m_i)}$$

Standard GPU implementations execute this in three discrete kernel launches:

1. Compute $\mathbf{S} = \mathbf{QK}^T$ in SRAM, then write $\mathbf{S} \in \mathbb{R}^{N \times N}$ to HBM ($\Theta(N^2)$ writes).
2. Read $\mathbf{S}$ from HBM, compute row max m and row sum d , compute $\mathbf{A} = \text{softmax}(\mathbf{S})$, write $\mathbf{A} \in \mathbb{R}^{N \times N}$ to HBM ($\Theta(N^2)$ reads + writes).
3. Read $\mathbf{A}$ and $\mathbf{V}$ from HBM, compute $\mathbf{O} = \mathbf{AV}$, write $\mathbf{O} \in \mathbb{R}^{N \times d}$ to HBM ($\Theta(N^2 + Nd)$ reads + writes).

Total HBM memory traffic is $\Theta(N^2 + Nd)$ elements. For $N = 8192$, $d = 128$ in FP16, $\mathbf{S}$ requires $8192^2 \times 2 \text{ B} = 134 \text{ MB}$ per head per layer. Because modern GPUs feature massive compute throughput relative to HBM bandwidth, repeatedly reading and writing intermediate matrices $\mathbf{S}$ and $\mathbf{A}$ leaves GPU execution units memory-bound.

C.5.2 Online softmax recurrence proof

Tiling softmax is difficult because computing $d_i = \sum_{k=1}^N \exp(S_{ik} - m_i)$ requires the global maximum m_i across all N keys. The online softmax recurrence solves this by incrementally updating normalization statistics as sub-blocks arrive.



Theorem C.2: Online softmax recurrence theorem

Let a vector $x \in \mathbb{R}^N$ be partitioned into two sub-vectors $x^{(1)} \in \mathbb{R}^{N_1}$ and $x^{(2)} \in \mathbb{R}^{N_2}$ with $N = N_1 + N_2$. Let the local statistics for block 1 be:

$$m^{(1)} = \max_{1 \leq j \leq N_1} x_j^{(1)}, \quad d^{(1)} = \sum_{j=1}^{N_1} \exp(x_j^{(1)} - m^{(1)})$$

and for block 2 be:

$$m^{(2)} = \max_{1 \leq j \leq N_2} x_j^{(2)}, \quad d^{(2)} = \sum_{j=1}^{N_2} \exp(x_j^{(2)} - m^{(2)})$$

Then the combined maximum m^{new} , normalization sum d^{new} , and accumulated output vector $\mathbf{O}^{\text{new}}$ satisfy the exact recurrence relations:

$$\begin{aligned} m^{\text{new}} &= \max(m^{(1)}, m^{(2)}) \\ d^{\text{new}} &= d^{(1)} e^{m^{(1)} - m^{\text{new}}} + d^{(2)} e^{m^{(2)} - m^{\text{new}}} \\ \mathbf{O}^{\text{new}} &= \frac{d^{(1)} e^{m^{(1)} - m^{\text{new}}}}{d^{\text{new}}} \mathbf{O}^{(1)} + \frac{e^{m^{(2)} - m^{\text{new}}}}{d^{\text{new}}} \sum_{j=1}^{N_2} e^{x_j^{(2)} - m^{(2)}} \mathbf{V}_j^{(2)} \end{aligned}$$

where $\mathbf{O}^{(1)} = \frac{1}{d^{(1)}} \sum_{j=1}^{N_1} e^{x_j^{(1)} - m^{(1)}} \mathbf{V}_j^{(1)}$ is the partial output accumulated from block 1.

Derivation of denominator recurrence

Expanding the combined sum $d^{\text{new}} = \sum_{j=1}^N e^{x_j - m^{\text{new}}}$:

$$\begin{aligned} d^{\text{new}} &= \sum_{j=1}^{N_1} e^{x_j^{(1)} - m^{\text{new}}} + \sum_{j=1}^{N_2} e^{x_j^{(2)} - m^{\text{new}}} \\ &= \sum_{j=1}^{N_1} e^{(x_j^{(1)} - m^{(1)}) + (m^{(1)} - m^{\text{new}})} + \sum_{j=1}^{N_2} e^{(x_j^{(2)} - m^{(2)}) + (m^{(2)} - m^{\text{new}})} \\ &= e^{m^{(1)} - m^{\text{new}}} \underbrace{\sum_{j=1}^{N_1} e^{x_j^{(1)} - m^{(1)}}}_{d^{(1)}} + e^{m^{(2)} - m^{\text{new}}} \underbrace{\sum_{j=1}^{N_2} e^{x_j^{(2)} - m^{(2)}}}_{d^{(2)}} \\ &= d^{(1)} e^{m^{(1)} - m^{\text{new}}} + d^{(2)} e^{m^{(2)} - m^{\text{new}}}. \quad \blacksquare \end{aligned}$$

Derivation of output rescaling recurrence

The true normalized output over the concatenated sequence is $\mathbf{O}^{\text{new}} = \frac{1}{d^{\text{new}}} \sum_{j=1}^N e^{x_j - m^{\text{new}}} \mathbf{V}_j$:

$$\begin{aligned} \mathbf{O}^{\text{new}} &= \frac{1}{d^{\text{new}}} \left[\sum_{j=1}^{N_1} e^{x_j^{(1)} - m^{\text{new}}} \mathbf{V}_j^{(1)} + \sum_{j=1}^{N_2} e^{x_j^{(2)} - m^{\text{new}}} \mathbf{V}_j^{(2)} \right] \\ &= \frac{1}{d^{\text{new}}} \left[e^{m^{(1)} - m^{\text{new}}} \sum_{j=1}^{N_1} e^{x_j^{(1)} - m^{(1)}} \mathbf{V}_j^{(1)} + e^{m^{(2)} - m^{\text{new}}} \sum_{j=1}^{N_2} e^{x_j^{(2)} - m^{(2)}} \mathbf{V}_j^{(2)} \right] \end{aligned}$$

Noting that $d^{(1)} \mathbf{O}^{(1)} = \sum_{j=1}^{N_1} e^{x_j^{(1)} - m^{(1)}} \mathbf{V}_j^{(1)}$, we substitute this directly:

$$\mathbf{O}^{\text{new}} = \frac{d^{(1)} e^{m^{(1)} - m^{\text{new}}}}{d^{\text{new}}} \mathbf{O}^{(1)} + \frac{e^{m^{(2)} - m^{\text{new}}}}{d^{\text{new}}} \sum_{j=1}^{N_2} e^{x_j^{(2)} - m^{(2)}} \mathbf{V}_j^{(2)}. \quad \blacksquare$$

C.5.3 Formal HBM access complexity proof

Let M be the fast SRAM capacity (in words). Following Algorithm 1 of T. Dao et al. (2022), FlashAttention sets $B_c = \lfloor M/(4d) \rfloor$ and $B_r = \min(\lfloor M/(4d) \rfloor, d)$. The proof abstracts constant factors and buffer reuse, using $B_c = \Theta(M/d)$, $B_r = \Theta(\min(M/d, d))$, and $B_r B_c = O(M)$ for the on-chip working set.

The algorithm executes two nested loops:

- Partition $\mathbf{Q}$ into $T_r = \lceil N/B_r \rceil$ blocks of size $B_r \times d$.
- Partition $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil N/B_c \rceil$ blocks of size $B_c \times d$.
- **Outer loop** over $j = 1, \dots, T_c$: Load $\mathbf{K}_j$ and $\mathbf{V}_j$ into SRAM.
- **Inner loop** over $i = 1, \dots, T_r$: Load $\mathbf{Q}_i, \mathbf{O}_i$, and its softmax statistics; compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T$ in SRAM; then update and write $\mathbf{O}_i$ and the statistics without materializing $\mathbf{S}_{ij}$ in HBM.

This loop order is the source of the I/O reduction. A key-value tile remains resident while the algorithm visits every query tile, so each element of $\mathbf{K}$ and $\mathbf{V}$ crosses the HBM boundary once. The query and output tiles must be revisited for each key-value block, but the $B_r \times B_c$ score tile is consumed on chip and discarded. Standard attention instead writes and rereads the full $N \times N$ score and probability matrices.

Total memory access accounting

1. **Key and value reads**: Each element of $\mathbf{K}$ and $\mathbf{V}$ is loaded once, contributing $2Nd$ accesses.
2. **Query, output, and statistics passes**: Each of the T_c column blocks reads all of $\mathbf{Q}$, reads and writes $\mathbf{O}$, and updates $O(N)$ softmax statistics. The matrices dominate the statistics for head dimension $d \geq 1$, so each pass contributes $\Theta(Nd)$ accesses and all passes contribute $\Theta(NdT_c)$.

Because $T_c = \lceil N/B_c \rceil = \Theta(Nd/M)$, the total is

$$\text{HBM}_{\text{Flash}} = \Theta(Nd + NdT_c) = \Theta\left(Nd + \frac{N^2 d^2}{M}\right) = \Theta\left(\frac{N^2 d^2}{M}\right),$$

where the final equality follows for $d \leq M \leq Nd$.

The bound counts communication rather than arithmetic. FlashAttention still evaluates the same dense score blocks and may recompute intermediate quantities during backpropagation; its advantage comes from keeping those intermediates out of HBM.

I/O reduction ratio

Comparing standard attention I/O to FlashAttention I/O:

$$\frac{\text{HBM}_{\text{Standard}}}{\text{HBM}_{\text{Flash}}} = \frac{\Theta(N^2 + Nd)}{\Theta\left(Nd + \frac{N^2 d^2}{M}\right)} = \Theta\left(\frac{M}{d^2}\right) \quad \text{for } N \gg \max\left(d, \frac{M}{d}\right)$$

🔍 Systems Perspective C.4: A100 I/O reduction

The asymptotic ratio M/d^2 predicts how greater SRAM capacity can reduce HBM traffic, but hidden constants and kernel details prevent it from serving as an exact transfer count. In an NVIDIA A100 measurement, T. Dao et al. (2022) report the effect for a forward-and-backward attention workload with sequence length 1,024, head dimension 64, 16 heads, batch size 64, a key-padding mask, and no dropout:

- **Standard attention:** 35.3 GB of HBM reads and writes; 35.1 ms runtime.
- **FlashAttention:** 4.4 GB of HBM reads and writes; 11.7 ms runtime.

The measurements show the systems consequence of the I/O bound: substantially less HBM traffic coincides with lower runtime even though FlashAttention performs additional recomputation. They do not imply that every configuration becomes compute bound; tile size, sequence length, and kernel implementation determine the realized regime.

C.6 Kaplan & Chinchilla Compute-Optimal Scaling Law Derivation

Determining the compute-optimal allocation between model parameters N and training dataset tokens D for a given floating-point budget C is a central decision in large language model system design. This section provides the rigorous mathematical derivation of compute-optimal scaling laws using Lagrange multipliers, contrasting the original findings of Kaplan et al. (2020) with the compute-optimal corrections of Hoffmann et al. (2022).

C.6.1 Parametric loss surface formulation

The cross-entropy evaluation loss $\mathcal{L}(N, D)$ of an autoregressive transformer language model trained with N non-embedding parameters on D tokens is modeled by the power-law formulation:

$$\mathcal{L}(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}$$

where:

- E : Irreducible loss representing the entropy of natural language data.
- $\frac{A}{N^\alpha}$: Capacity bottleneck loss resulting from finite model parameters N .
- $\frac{B}{D^\beta}$: Dataset size bottleneck loss resulting from finite training tokens D .
- $A, B > 0$: Empirical scaling constants.
- $\alpha, \beta > 0$: Power-law scaling exponents for model size and dataset size, respectively.

C.6.2 Constrained optimization via Lagrange multipliers

The total floating-point operations required to train a transformer with N non-embedding parameters for D tokens is given by:

$$C = 6ND \text{ FLOPs}$$

where $2ND$ accounts for the forward pass and $4ND$ accounts for the backward pass.

Given a fixed training compute budget C_0 , we formulate the constrained optimization problem to minimize the reducible loss $\hat{\mathcal{L}}(N, D) = \frac{A}{N^\alpha} + \frac{B}{D^\beta}$:

$$\min_{N, D} \left(\frac{A}{N^\alpha} + \frac{B}{D^\beta} \right) \quad \text{subject to} \quad g(N, D) = 6ND - C_0 = 0$$

We construct the Lagrangian function $\mathcal{L}_{\text{Lagrange}}(N, D, \lambda)$:

$$\mathcal{L}_{\text{Lagrange}}(N, D, \lambda) = \frac{A}{N^\alpha} + \frac{B}{D^\beta} + \lambda(6ND - C_0)$$

Taking partial derivatives with respect to N , D , and λ and setting them to zero:

$$\frac{\partial \mathcal{L}_{\text{Lagrange}}}{\partial N} = -\alpha AN^{-\alpha-1} + 6\lambda D = 0 \implies \alpha AN^{-\alpha-1} = 6\lambda D \quad (1)$$

$$\frac{\partial \mathcal{L}_{\text{Lagrange}}}{\partial D} = -\beta BD^{-\beta-1} + 6\lambda N = 0 \implies \beta BD^{-\beta-1} = 6\lambda N \quad (2)$$

$$\frac{\partial \mathcal{L}_{\text{Lagrange}}}{\partial \lambda} = 6ND - C_0 = 0 \quad (3)$$

Multiplying Equation (1) by N and Equation (2) by D :

$$\alpha AN^{-\alpha} = 6\lambda ND$$

$$\beta BD^{-\beta} = 6\lambda ND$$

Equating these expressions yields the fundamental power-law optimality condition:

$$\alpha AN^{-\alpha} = \beta BD^{-\beta}$$

This condition proves that at the compute-optimal allocation, the marginal loss reduction per fractional increase in parameter count must equal the marginal loss reduction per fractional increase in dataset tokens.

C.6.3 Analytical derivation of compute-optimal exponents

From $\alpha AN^{-\alpha} = \beta BD^{-\beta}$, we solve for D in terms of N :

$$D^\beta = \left(\frac{\beta B}{\alpha A}\right) N^\alpha \implies D = \left(\frac{\beta B}{\alpha A}\right)^{1/\beta} N^{\alpha/\beta}$$

Substituting D into the compute constraint $C = 6ND$:

$$C = 6N \left(\frac{\beta B}{\alpha A}\right)^{1/\beta} N^{\alpha/\beta} = 6 \left(\frac{\beta B}{\alpha A}\right)^{1/\beta} N^{\frac{\alpha+\beta}{\beta}}$$

Solving explicitly for the compute-optimal parameter count $N^*(C)$:

$$N^*(C) = \left(\frac{\alpha A}{\beta B}\right)^{\frac{1}{\alpha+\beta}} \left(\frac{C}{6}\right)^{\frac{\beta}{\alpha+\beta}} \propto C^{\frac{\beta}{\alpha+\beta}}$$

By symmetry, solving for the compute-optimal token count $D^*(C)$:

$$D^*(C) = \left(\frac{\beta B}{\alpha A}\right)^{\frac{1}{\alpha+\beta}} \left(\frac{C}{6}\right)^{\frac{\alpha}{\alpha+\beta}} \propto C^{\frac{\alpha}{\alpha+\beta}}$$

Empirical exponents and the $D/N \approx 20$ rule

Fitting the parametric loss surface to over 400 experimental runs, Hoffmann et al. (2022) estimated $\alpha \approx 0.34$ and $\beta \approx 0.28$. Substituting these values gives exponents close to the paper's parametric-model result:

$$\frac{\beta}{\alpha+\beta} \approx 0.45, \quad \frac{\alpha}{\alpha+\beta} \approx 0.55$$

Thus the parametric model predicts:

$$N^*(C) \propto C^{0.45}, \quad D^*(C) \propto C^{0.55}$$

The paper's two direct empirical approaches report near-balanced exponents of $(0.50, 0.50)$ and $(0.49, 0.51)$, which motivate the common square-root shorthand. Under the parametric fit, however, D^*/N^* grows slowly with compute rather than remaining strictly constant. Across the regime studied, roughly 20 training tokens per parameter is a useful Chinchilla heuristic, not a universal invariant.

🔍 Systems Perspective C.5: Kaplan vs. Chinchilla scaling

The difference between Kaplan et al. (2020) and Hoffmann et al. (2022) highlights how hyperparameter choices in empirical benchmarking can alter system design conclusions:

- **Kaplan et al. (2020)** estimated $\alpha \approx 0.057$ and $\beta \approx 0.28$, predicting $N \propto C^{0.73}$ and $D \propto C^{0.27}$. This led the field to scale model sizes aggressively while holding training dataset sizes relatively small (e.g., GPT-3 175B trained on 300B tokens, giving $D/N = 1.71$).
- **Methodological difference:** Kaplan et al. (2020) used a fixed token budget and learning-rate schedule across models. Hoffmann et al. (2022) matched the schedule to each training horizon and found that the earlier setup understated the value of training smaller models on more data.
- **Hoffmann et al. (2022)** found near-balanced parameter and token scaling: its three approaches report exponent pairs of $(0.50, 0.50)$, $(0.49, 0.51)$, and $(0.46, 0.54)$ rather than one exact universal ratio.
- **Practical Impact:** Chinchilla (70B parameters trained on 1.4T tokens, $D/N = 20$) outperformed GPT-3 (175B parameters trained on 300B tokens) across downstream benchmarks while requiring $2.5\times$ less compute during inference and $2.5\times$ lower weight memory at the same precision (Hoffmann et al. 2022).

Summary

An algorithm becomes a systems workload through its shapes, operation graph, data dependencies, and numerical representation. Linear-algebra primitives, memory layouts, broadcasting, and sparse formats determine work, traffic, locality, and usable parallelism. Automatic differentiation and backpropagation add saved activations and backward computation; computational graphs then expose opportunities for fusion and recomputation. The Baur–Strassen result bounds reverse-mode overhead, while FlashAttention shows how tiling and an online recurrence can reduce HBM traffic without changing attention semantics.

The same accounting connects kernels to scaling decisions. Compute-optimal scaling turns a fixed budget into a model-data allocation, but the fitted exponents remain empirical rather than universal. Across these models, the reusable method is to expose operations, bytes, saved state, dependencies, and asymptotic constraints before selecting an optimization.

Machine Foundations

A datasheet reports peak throughput; a workload almost never reaches it. Closing that gap requires knowing which physical limit binds execution first: latency, memory bandwidth, arithmetic intensity, or energy. This appendix collects the durable hardware models and order-of-magnitude values that anchor single-node reasoning throughout the book, providing the mathematical baselines where chapter estimates can be audited or extended. The models presume an undergraduate background in computer architecture, particularly the memory hierarchy.

How to Use This Appendix

This appendix is designed as a reference. When diagnosing performance issues, use this appendix to translate a vague symptom (“it’s slow”) into a specific constraint (“memory bound at batch size one”) and then choose the lever that can actually move.

Conventions used here follow the book-wide notation (for example, B is reserved for batch size and BW for bandwidth).

- Sanity-check feasibility: Start with section D.1 for order-of-magnitude numbers.
- Diagnose the dominant ceiling: Use the Roofline Model in section D.2.1 to decide whether the workload is compute bound or memory bound.
- Reason about scaling limits: Use Amdahl’s and Gustafson’s Laws in section D.2.3 to understand why adding accelerators may not reduce time-to-train.
- Choose the right precision: Use section D.4.1 to reason about FP32 vs. BF16/FP16 vs. INT8 as a systems trade-off.
- Follow the chapter treatment: For the full narrative, use chapter 11, chapter 8, and chapter 13.

Although modern hardware encompasses diverse accelerator architectures, interconnect topologies, and packaging options, the physical laws and bottleneck models detailed here govern them all.

D.1 Numbers to Know

Just as Jeff Dean’s “Latency Numbers Every Programmer Should Know”¹ shaped a generation of systems engineers, these reference numbers provide the order-of-magnitude intuition essential for ML systems design. Although absolute values and ratios vary by technology and workload, the hierarchy is more durable. Memorize the relationships; use the specific numbers as sanity checks.



Systems Perspective D.1: Three numbers that matter most

- Energy ratio: In the cited 45 nm reference, a 32-bit DRAM read uses $\sim 581\times$ the energy of one FP16 multiply. This motivates arithmetic intensity.
- Training-state footprint: Model weights (2 bytes FP16) + gradients (2 bytes FP16) + master weights (4 bytes FP32) + optimizer states for Adaptive Moment Estimation (Adam) at 8 bytes. That totals 16 bytes per parameter, so a 7B model needs 112 GB just to start training.
- Fiber propagation limit: Light travels about 200 km/ms in fiber. The table’s ~ 40 ms cross-country round trip includes reducible overhead, but propagation cannot be optimized away.

¹ | **Jeff Dean:** A Google Senior Fellow and one of the architects of Google’s distributed systems infrastructure, including MapReduce, BigTable, and TensorFlow. His latency numbers, originally presented with Peter Norvig around 2010, became a canonical reference for systems engineers. The numbers have been updated over the years as hardware evolved, but the *hierarchy* of latencies remains remarkably stable; Colin Scott’s interactive visualization shows the latency hierarchy across hardware generations (Scott 2012).

Reference relationships

These relationships mix physical or arithmetic bounds with technology-specific measurements; only the bounds are invariant.

Speed of light tax

Table D.1 shows approximate round-trip baselines. Their propagation component is irreducible.

Table D.1: Speed of Light Reference: Light in fiber travels ~200 km/ms. These approximate round trips include reducible overhead; only propagation is irreducible.

Distance	Round-Trip Latency	Implication
Same data center	~1 ms	Distributed training feasible
Cross-country (US)	~40 ms	Reduces the remaining request budget
Cross-Atlantic	~60 ms	Further reduces the request budget
Cross-Pacific	~100 ms	Data locality is critical

Energy hierarchy

Table D.2 quantifies the energy cost of data movement vs. computation—the fundamental reason why arithmetic intensity dominates ML performance optimization.²

Table D.2: Energy Reference Ratios: The first three ratios use Horowitz’s 45 nm measurements; the L1/register ratio is an illustrative on-chip hierarchy anchor.

Relationship	Reference Ratio	Source and Interpretation
DRAM access vs. FP16 multiply	~581×	Horowitz 45 nm; wire capacitance scales with distance
FP32 vs. INT8 energy	~18×	Horowitz 45 nm; bit width determines switching energy
FP32 vs. FP16 energy	~3.4×	Horowitz 45 nm; narrower arithmetic reduces switching and datapath energy
L1 SRAM vs. register	~5×	Illustrative on-chip hierarchy anchor; distance to the arithmetic unit

Memory hierarchy

Table D.3 shows why the memory hierarchy is uneven: nearby on-package hops can differ by only a few times, while off-chip, storage, and network tiers introduce orders-of-magnitude jumps.

Table D.3: The Latency Hierarchy: Selected latency relationships in the memory hierarchy. The largest gaps come from crossing physical boundaries: off-chip memory, peripheral links, storage, and network fabric.

Relationship	Ratio	Why It Persists
Accelerator high-bandwidth memory vs. register	~1000× slower	On-chip vs. off-chip
SSD vs. register	~300,000× slower	Electrical vs. mechanical/flash
Network vs. local memory	~16× slower	Speed of light + switching
Accelerator memory BW vs. CPU↔Accelerator link	~52× faster	Architectural investment priority

Scaling laws

Table D.4 collects the arithmetic relationships that govern memory and compute requirements for training and inference.³

² | **Energy hierarchy sources:** The DRAM and arithmetic values come from Horowitz (2014) (45 nm). The L1 SRAM/register ratio is an illustrative hierarchy anchor, not a Horowitz measurement. The headline ratio compares a 32-bit DRAM access with a 16-bit floating-point multiply. Both the absolute values and the ratio vary with process, circuit, memory technology, and operand width; wire distance helps explain why movement remains expensive.

³ | **Training memory (Adam):** The 16 bytes/parameter rule assumes mixed-precision training with Adam. ZeRO optimization can reduce per-accelerator memory by sharding optimizer states across accelerators, but the total memory across all accelerators remains ~16× parameters.

Table D.4: Scaling Rules: These are arithmetic, not hardware-specific. Training memory includes FP16 weights (2 bytes), FP16 gradients (2 bytes), FP32 master weights (4 bytes), and Adam optimizer states (8 bytes for momentum + variance).

Rule	Formula	Example
Inference memory (FP16)	$2 \text{ bytes} \times \text{parameters}$	7B params $\rightarrow$ 14 GB
Inference memory (INT8)	$1 \text{ byte} \times \text{parameters}$	7B params $\rightarrow$ 7 GB
Training memory (Adam)	$16 \text{ bytes} \times \text{parameters}$	7B params $\rightarrow$ 112 GB
Inference FLOPs (transformer)	$\sim 2 \times \text{parameters per token}$	7B model $\rightarrow \sim 14$ GFLOPs/token
Training FLOPs	$\sim 6 \times \text{parameters} \times \text{tokens}$	7B on 1T tokens $\rightarrow 4 \times 10^{22}$ FLOPs

Illustrative latency budgets

Latency budgets depend on application-specific safety requirements, user expectations, and service-level objectives. Table D.5 lists illustrative targets rather than universal limits.

Table D.5: Illustrative Latency Targets: Production requirements depend on the application and its service-level objectives.

Application	Budget	Constraint
Autonomous braking	<10 ms	At 100 km/h, 10 ms = 28 cm of travel
Voice assistant	<100 ms	Human perception of “instant”
Web search	<200 ms	User patience threshold
Video streaming	<1 s	Buffer tolerance
Batch training	hours–days	Throughput dominates latency

Current hardware reference (c. 2024)

These numbers reflect the current generation. Use them for back-of-envelope calculations and recheck them against current hardware.

Memory latency and bandwidth

Table D.6 provides representative latency and bandwidth anchors across the memory hierarchy.

Table D.6: Memory Hierarchy (c. 2024): Representative latency and bandwidth anchors across registers, caches, memory, interconnects, networks, and storage. Together they show why locality and data movement shape performance.

Level	Latency	Bandwidth
Register	~ 0.3 ns	—
L1 Cache	~ 1 ns	—
L2 Cache	~ 4 ns	—
GPU HBM3	~ 300 ns	3.4 TB/s
PCIe Gen5 (CPU $\leftrightarrow$ GPU)	~ 1000 ns	64 GB/s
CPU DRAM	~ 100 ns	50 GB/s
InfiniBand (network)	~ 5000 ns	50 GB/s
NVMe SSD	~ 100000 ns	7 GB/s

Compute throughput

Table D.7 compares representative peak throughput and power for data-center GPUs and a mobile NPU.

Table D.7: Compute Reference (c. 2024): Representative peak throughput and power for data-center GPUs and a mobile NPU. Because the rows use different precision modes and operation conventions, compare them only within a matched workload and precision context.

Platform	FP16/BF16	INT8/FP8-class	Power
Data center GPU (H100)	989 TFLOP/s	1979 TFLOP/s (FP8 peak)	700 W
Data center GPU (A100)	312 TFLOP/s	624 TOPS	400 W
Mobile NPU	—	35 TOPS	3–5 W

Roofline ridge points

Table D.8 defines the arithmetic intensity thresholds that determine whether a workload is memory bound or compute bound.

Table D.8: Arithmetic Intensity Thresholds (c. 2024): Batch-1 weight-streaming inference is often memory bound; larger batches may cross the ridge.

Accelerator	Ridge Point	Implication
A100 (FP16)	153 FLOP/byte	Below → memory-bound
H100 (FP16)	295 FLOP/byte	Higher bar for compute-bound

🔍 Systems Perspective D.2: A note on terminology: GPUs and accelerators

Throughout this book, “accelerator” is the general term for hardware acceleration. The principles—roofline analysis, memory hierarchies, numerical precision, and performance modeling—apply to GPUs, Tensor Processing Units (TPUs), NPUs, application-specific integrated circuits, and other specialized AI accelerators. Vendor-specific features such as CUDA and NVLink are identified explicitly.

Knowing the numbers is only the first step. The real power comes from compact models that reveal *which* number matters for the bottleneck at hand. The Roofline Model begins that diagnosis by translating raw hardware specs into actionable performance ceilings.

D.2 Physics of Computing

Raw hardware specs—TFLOP/s, TB/s, watt budgets—are necessary but insufficient for performance reasoning. Without compact analytical models, an engineer cannot distinguish a compute-bound workload from a memory-bound one, or predict whether doubling GPUs will halve training time. The models in this section turn those distinctions into quantitative checks.

D.2.1 The Roofline model

The Roofline Model (Williams et al. 2009) bounds how fast a workload can run on a given hardware target. The answer depends on whether the workload runs out of compute or memory bandwidth first.

Every operation has an **arithmetic intensity**: the ratio of computations performed to bytes moved from memory. Matrix multiplication has high arithmetic intensity because each loaded element is reused many times. Element-wise operations like rectified linear unit (ReLU) have low intensity because each operation loads a number, performs one computation, and writes it back. As figure D.1 illustrates, each workload is bounded by either memory bandwidth or compute throughput, and its arithmetic intensity determines which ceiling it hits first.

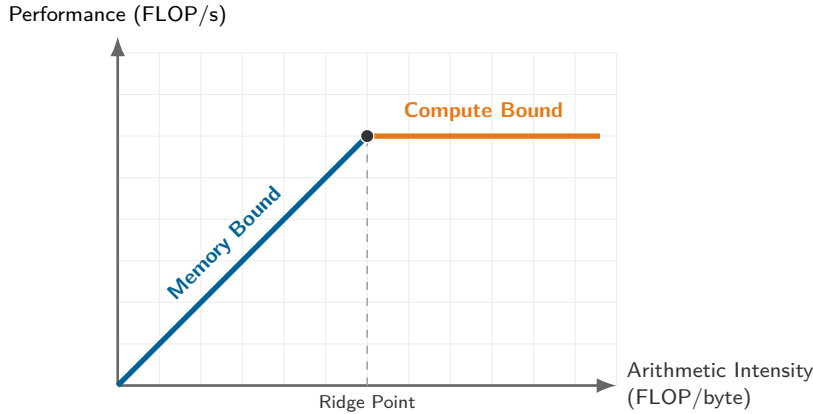


Figure D.1: The Roofline Model: The sloped memory-bandwidth ceiling and horizontal peak-compute ceiling meet at the ridge point. Plotting a workload by arithmetic intensity identifies which roofline ceiling applies.

The **ridge point** determines the hardware’s balance. If a workload’s intensity falls below this point, it is memory-bound (sloped region). If above, it is compute-bound (flat region).

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{bytes accessed}} \quad \text{Ridge Point} = \frac{\text{Peak FLOP/s}}{\text{Memory Bandwidth}}$$

Systems Perspective D.3: Batch size controls arithmetic intensity

For matrix multiplications, arithmetic intensity scales with the batch dimension. For $\mathbf{Y} = \mathbf{XW}$, where $\mathbf{X}$ is $(B \times d_{\text{in}})$ and $\mathbf{W}$ is $(d_{\text{in}} \times d_{\text{out}})$:

- FLOPs: $2 \times B \times d_{\text{in}} \times d_{\text{out}}$ (multiply-adds)
- Bytes: Moving $\mathbf{X}$, $\mathbf{W}$, and $\mathbf{Y}$ costs approximately $s_{\text{elem}}(Bd_{\text{in}} + d_{\text{in}}d_{\text{out}} + Bd_{\text{out}})$ bytes.

Doubling B doubles FLOPs while weight traffic stays nearly constant when weights dominate, increasing arithmetic intensity. This is why batching often helps inference serving, although activation traffic, cache behavior, shape, precision, and the ridge point determine whether it reaches the compute ceiling.

D.2.1.1 A concrete example: The A100 analysis

Consider an NVIDIA A100 GPU with FP16 Tensor Core performance of 312 TFLOP/s and HBM2e bandwidth of 2.04 TB/s. The ridge point is 312 TFLOP/s / 2.04 TB/s = 153 FLOP/byte (the Tera prefixes cancel, yielding FLOP/byte).

Two common operations fall on opposite sides of that ridge. General matrix multiply (GEMM) on two square matrices of size 4096 by 4096 has arithmetic intensity of approximately 1365 FLOP/byte, so 1365 FLOP/byte > 153 FLOP/byte and the operation is compute bound. An element-wise ReLU on an FP16 tensor performs 1 comparison while reading 2 bytes and writing 2 bytes (4 bytes total), yielding an arithmetic intensity of only $1/4 = 0.25$ FLOP/byte. Because 0.25 FLOP/byte $\ll$ 153 FLOP/byte, the operation is severely memory bound, achieving only about 0.16 percent of peak TFLOP/s. This contrast explains why modern frameworks fuse operations. Combining ReLU directly with the preceding MatMul kernel avoids writing intermediate results to memory, effectively preserving memory bandwidth.

D.2.2 Dimensional analysis

The Roofline Model helps diagnose *where* a bottleneck lies. Before applying any performance equation, however, it must be verified as physically meaningful. Dimensional analysis provides this

sanity check: any valid equation must be **dimensionally homogeneous**—every term must resolve to the same units. If they do not, the equation contains an error.

For serialized phases, consider the iron law of ML systems (principle 3) introduced in section 1.7:

$$T = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}}$$

Correctness is verified by confirming that every term resolves to time (seconds):

$$T[s] = \underbrace{\frac{D_{\text{vol}}[\text{bytes}]}{\text{BW}[\text{bytes/s}]}}_{\text{seconds}} + \underbrace{\frac{O[\text{FLOPs}]}{R_{\text{peak}}[\text{FLOP/s}] \cdot \eta_{\text{hw}}[1]}}_{\text{seconds}} + \underbrace{L_{\text{lat}}[s]}_{\text{seconds}}$$

- Data term: $\frac{\text{bytes}}{\text{bytes/s}} = \text{bytes} \times \frac{\text{s}}{\text{bytes}} = \text{s}$
- Compute term: $\frac{\text{FLOPs}}{\text{FLOP/s}} = \text{FLOPs} \times \frac{\text{s}}{\text{FLOP}} = \text{s}$
- Latency term: Already in seconds.

The equation is physically consistent. Apply this technique to any systems equation: if the dimensions do not match, the formula is wrong. “FLOPs” and “Bandwidth” cannot be traded directly because they have different units. Any such trade-off must convert through Time, which is precisely what the iron law quantifies.

Once an equation is dimensionally sound, the next question is how its time terms scale across devices. Section D.2.3 answers that question with two complementary bounds.

D.2.3 Amdahl’s Law and Gustafson’s Law

Parallelization is the primary tool for scaling ML, but its limits depend on *how* the workload scales. These two laws frame the fundamental tension in parallel computing. **Amdahl’s law** is the pessimist’s view, governing how much faster a *fixed* task can run (optimizing latency). **Gustafson’s law** is the optimist’s view, governing how much *more* work can be done in the same time (optimizing throughput).

D.2.3.1 Strong scaling (Amdahl’s Law)

Strong scaling measures how much faster a fixed-size problem runs as processors are added.

Amdahl’s Law (Amdahl 1967) states that the speedup is limited by the serial portion of the task.⁴ If a fraction s of the task is serial (cannot be parallelized) and $p = 1 - s$ is parallelizable, the maximum speedup with n processors is:

$$\text{Speedup}(n) = \frac{1}{s + \frac{1-s}{n}}$$

As $n \rightarrow \infty$, the term $\frac{1-s}{n} \rightarrow 0$, and the speedup converges to $1/s$.

To see Amdahl’s Law in action, suppose 5 percent of a training step is serial overhead (for example, Python global interpreter lock (GIL), kernel launch latency) and 95 percent is parallelizable matrix math:

- With $n = 1$, speedup is 1.
- With $n = 8$, speedup is $1/(0.05 + 0.95/8) \approx 5.9\times$.
- With $n \rightarrow \infty$, speedup is capped at $1/0.05 = 20\times$.

No matter how many accelerators are added, this fixed workload cannot run faster than $20\times$.

D.2.3.2 Weak scaling (Gustafson’s Law)

Weak scaling measures how much larger a problem can become while holding runtime fixed as processors are added.⁵

This is the reality of Large Language Models. Rather than using 1,000 accelerators to train a model on a small dataset in milliseconds, they are used to train on a dataset $1,000\times$ larger in reasonable time.

⁴ | **Gene Amdahl (1922–2015):** A legendary computer architect at IBM, where he was the chief architect of the System/360. He later founded Amdahl Corporation to compete with IBM in the mainframe market.

⁵ | **John Gustafson:** A computer scientist known for his work in parallel computing and for introducing the Unum (universal number) format. His law was a direct response to the perceived “limits” of Amdahl’s Law when applied to massive scale.

For n processors, let s be the serial fraction of execution time measured on those processors for the scaled workload. Gustafson's Law (Gustafson 1988) models this "scaled speedup":

$$\text{Scaled Speedup}(n) = n - s(n - 1)$$

Because the parallel part grows linearly with n while s remains fixed, weak scaling can keep efficiency high as the problem grows.

Using the same 5 percent serial overhead ($s = 0.05$), Gustafson's Law tells a very different story:

- With $n = 1$, speedup is 1.
- With $n = 8$, Scaled Speedup is $8 - 0.05 \times (7) = 8 - 0.35 = 7.65\times$.
- With $n = 1000$, Scaled Speedup is $1000 - 0.05 \times (999) \approx 950\times$.

In weak scaling, efficiency remains high because the useful work (training the model) scales up to dwarf the fixed overheads.

The same scaling lens turns model size, data size, hardware count, and utilization into a single training-time estimate.

Napkin Math D.1: The training time equation

Problem: How long does dense-transformer training take for a stated model size, token count, accelerator count, peak rate, and utilization?

Math:

$$T \approx \frac{6 \cdot P \cdot D}{N_{\text{GPU}} \cdot R_{\text{peak}} \cdot \eta_{\text{hw}}}$$

Variables:

- Training FLOP factor: The factor 6 derives from $2P$ FLOP/token for the forward pass and $4P$ FLOP/token for the backward pass ($2P$ for activation gradients and $2P$ for weight gradients), yielding $6PD$ FLOP total across D training tokens.
- P : Number of model parameters.
- D : Number of training tokens.
- N_{GPU} : Number of GPUs.
- R_{peak} : Peak FLOP/s of one accelerator.
- η_{hw} : Hardware utilization, typically 30 percent–50 percent in this training estimate.

Example: Training a 1B parameter model on 20B tokens using 1 A100 (312 TFLOP/s) at 40 percent utilization. Total FLOPs = $6 \times 1 \times 10^9 \times 2 \times 10^{10} = 1.2 \times 10^{20}$ FLOPs. Throughput = $1 \times (3.12 \times 10^{14}) \times 0.40 \approx 1.248 \times 10^{14}$ FLOP/s. $T = \frac{1.2 \times 10^{20}}{1.248 \times 10^{14}} \approx 961,538.5$ seconds $\approx 16,025.6$ minutes.

Systems insight: Under these assumptions, training takes 961,538.5 seconds (≈ 16025.6 minutes, or about 11.1 days); the estimate makes model scale, data scale, effective accelerator throughput, and utilization explicit levers.

D.2.4 Little's Law

For capacity planning in stable inference systems, **Little's law** (Little 1961) relates long-run mean concurrency (Q_{req}), mean arrival rate (λ_{arr}), and mean time in system (T_{lat}):⁶

$$Q_{\text{req}} = \lambda_{\text{arr}} \times T_{\text{lat}}$$

To see this in practice, consider sustaining 1,000 QPS with 50 ms average latency. The law dictates that the system must support $1000 \times 0.05 \text{ s} = 50$ concurrent requests.

This sizes worker pools, but memory needs another assumption. If every in-system request simultaneously holds 1 GB of device state, the average population implies 50 GB; queued requests may instead stay in host memory or share batched state. If 24 device slots each remain occupied for

⁶ | **John Little:** An Institute Professor at MIT and a pioneer in the field of operations research. His law, proved in 1961, is fundamental to queuing theory and is used across fields from manufacturing to computer network analysis.

50 ms, their throughput proxy is $N_{\max}/T_{\text{lat}} = 24/0.05 = 480$ QPS; Little’s Law alone sets no physical maximum.

These physics-based models—Roofline, Amdahl, Gustafson, and Little—diagnose *where* bottlenecks lie. Translating those diagnoses into actionable optimizations, however, requires understanding the concrete hardware structures that impose them: caches, memory buses, and interconnects.

D.3 Computer Architecture Essentials

A GPU advertises 1,000 TFLOP/s, yet a kernel achieves only 30 TFLOP/s. The gap may reflect data movement, insufficient parallelism, synchronization, launch overhead, or software inefficiency. While physics sets theoretical performance bounds, computer architecture defines the machinery that determines how close a real workload can get. The following discussion covers the latency, bandwidth, and energy trade-offs that shape system design.

D.3.1 Latencies every programmer should know

The first step in systems intuition is understanding the cost of distance. Table D.9 quantifies how long the processor waits for data from different levels of the memory hierarchy. If accessing a register is like picking up a nearby pencil, fetching from high-bandwidth memory (HBM) is walking across the office, and fetching from disk is flying to the moon.

Table D.9: The Latency Hierarchy: Representative access times for modern AI hardware; the approximate cycle counts use a 3 GHz reference clock. The jump from SRAM cache to HBM makes cache locality a major performance factor.

Component	Latency (ns)	Cycles (Approx)	Relative “Distance”
Register	~0.3 ns	1 cycle	10 seconds
L1 Cache	~1 ns	3–4 cycles	33.3 seconds
L2 Cache	~4 ns	12 cycles	2.2 minutes
HBM3 (GPU Memory)	~300 ns	1,000 cycles	2.8 hours
NVLink (GPU-GPU)	~500 ns	1,500 cycles	4.6 hours
PCIe (CPU-GPU)	~1000 ns	3,000 cycles	9.3 hours
InfiniBand (Network)	~5000 ns	15,000 cycles	1.9 days
SSD (NVMe)	~100000 ns	300,000 cycles	38.6 days

D.3.2 The AI hardware cheat sheet (modern reference)

Latency measures the wait for the *first* byte; bandwidth measures how many bytes follow. Table D.10 provides the constants for back-of-the-envelope “Roofline” calculations. These represent the “standard units of compute” for the current era of machine learning.

Table D.10: Reference Specs: Key constants for quantitative analysis. Always check specific datasheets; HBM capacity follows vendor units, and interconnect bandwidth is aggregate bidirectional, so topology and direction matter.

Spec	NVIDIA H100 (SXM)	Google TPU v5p	System Impact
BF16 Peak	989 TFLOP/s	459 TFLOP/s	The “Speed Limit” (R_{peak})
Memory Bandwidth	3.35 TB/s	2.76 TB/s	The “Width of the Pipe” (BW)
HBM Capacity	80 GB	95 GiB	Max Model Size (P)/Batch Size (B)
On-chip SRAM	50 MB L2	128 MiB VMEM/chip	Critical for Operator Fusion
Interconnect	900 GB/s (NVLink)	1200 GB/s (Inter-Chip Interconnect, ICI)	Determines Model Parallelism Scaling

D.3.3 The memory hierarchy

Computer systems use a hierarchy because no single technology provides both high capacity and low latency. Figure D.2 shows this trade-off: keeping data higher in the pyramid (registers/cache) reduces access latency when data movement is on the critical path.

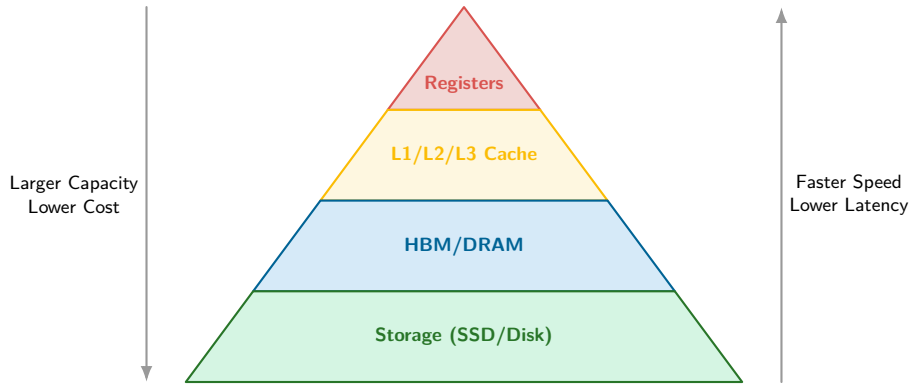


Figure D.2: The Memory Hierarchy: Performance depends on data proximity. Accessing HBM is roughly $1,000\times$ slower than registers; accessing SSD is roughly $300,000\times$ slower.

The memory hierarchy is the fundamental physical constraint of machine learning systems. Table D.11 consolidates the physical properties—latency, bandwidth, and energy—across the entire stack.

Table D.11: Physical Properties of the Memory Hierarchy (c. 2024): Representative latency and bandwidth anchors across the memory hierarchy. The hierarchy spans roughly five orders of magnitude in latency. NVLink is aggregate bidirectional; only the cited 45 nm DRAM energy is shown because other tiers are implementation-specific. Keeping data higher in the hierarchy can therefore deliver large performance gains.

Layer	Technology	Latency	Bandwidth	Energy Reference (per 32b)
Registers	Flip-Flops	~ 0.3 ns	—	—
L1 Cache	SRAM	~ 1 ns	—	—
L2 Cache	SRAM	~ 4 ns	—	—
Memory (Local)	HBM3	~ 300 ns	3350 GB/s	640 pJ
Interconnect	NVLink 4.0	~ 500 ns	900 GB/s	—
Host Link	PCIe Gen5	~ 1000 ns	64 GB/s	—
System RAM	DDR5	~ 100 ns	50 GB/s	—
Network (Fabric)	InfiniBand NDR	~ 5000 ns	50 GB/s	—
Storage (Local)	NVMe SSD	~ 100000 ns	7 GB/s	—

The hierarchy’s energy costs reveal why data movement dominates modern system design.

Systems Perspective D.4: The high cost of data movement

In the cited 45 nm reference, fetching a 32-bit value from DRAM costs roughly $581\times$ the energy of one FP16 multiply (~ 640 pJ vs. ~ 1.1 pJ). The ratio is platform-specific and does not alone determine workload energy, which also depends on access counts and reuse. It nevertheless explains why arithmetic intensity matters.

D.3.4 Bandwidth vs. latency

Bandwidth (throughput) and latency (delay) are distinct constraints. For data volume D_{vol} sent over effective bandwidth BW with fixed path latency L_{lat} , and assuming the latency and serialization costs do not overlap, total transfer time is:

$$T = L_{\text{lat}} + \frac{D_{\text{vol}}}{\text{BW}}$$

The crossover point is the transfer size where the two terms are equal:

$$D_{\text{vol,cross}} = L_{\text{lat}} \times \text{BW}$$

For transfers below $D_{\text{vol,cross}}$ (for example, small request or control messages), latency dominates. For transfers above it (for example, loading weights), bandwidth dominates.

Consider sending data over a 10 Gb/s link with 10 ms base request/response latency. The crossover size is 12.5 MB, so the dominant term depends on which side of that threshold the transfer falls.

- Latency-bound packet (1 KB):
 - Transmission: $1 \text{ KB} \times 8/10 \text{ Gb/s} \approx 0.8 \text{ } \mu\text{s}$.
 - Total Time $\approx 10 \text{ ms} + 0.8 \text{ } \mu\text{s} \approx 10 \text{ ms}$.
 - Result: Fixed path and protocol latency dominates; propagation is one component.
- Bandwidth-bound checkpoint (1 GB):
 - Transmission: $1 \text{ GB} \times 8/10 \text{ Gbps} \approx 800 \text{ ms}$.
 - Total Time $\approx 10 \text{ ms} + 800 \text{ ms} = 810 \text{ ms}$.
 - Result: Base latency is negligible; bandwidth is the bottleneck.

Architecture determines how fast data can move, but numerical precision directly controls *how much* data must move. Halving precision from FP32 to FP16 halves bytes per parameter and can nearly double useful bandwidth when the hardware, kernels, and model accuracy support it. Understanding these trade-offs requires a closer look at how numbers are represented in hardware.

D.4 Numerical Representations

Statistics characterizes data distributions; numerical representations determine how the system stores those values. In ML systems, the choice of precision (FP32 vs. BF16 vs. INT8) is a direct trade-off between statistical fidelity and hardware throughput.

D.4.1 Floating-point format comparison

IEEE 754 formats such as FP32 and FP16, together with AI-specific formats such as BF16 and FP8, define different trade-offs between dynamic range (the span of representable values) and precision (the granularity of values within that range). Table D.12 summarizes the key formats and their use cases, while figure D.3 visualizes the bit allocations.

Table D.12: Numerical Format Comparison: Each format trades off precision, dynamic range, memory footprint, and compute throughput. BF16 shares FP32's normal exponent range while using half the storage.

Format	Bits	Exponent	Mantissa	Dynamic Range	Typical Use Case
FP32	32	8	23	$\sim 10^{-38}$ to 10^{38}	Training (full precision), reference inference
FP16	16	5	10	Normal: 6.10×10^{-5} – 6.55×10^4	Training often with loss scaling, inference
BF16	16	8	7	FP32 normal exponent range	Training, usually without loss scaling
FP8	8	4 or 5	3 or 2	Varies	Training/inference on supported hardware
INT8	8	N/A	N/A	-128 to 127	Inference after quantization

Beyond bit width, the allocation of bits between exponent and mantissa determines what range of values each format can represent.

Among these formats, BF16⁷ deserves special attention (Wang and Kanwar 2019; Kalamkar et al. 2019). It occupies the same 16-bit storage class as FP16 but behaves differently in training, so nominal width alone does not predict the operational burden. The following systems perspective connects bit allocation to memory traffic, numerical stability, and deployment efficiency.

⁷ **BF16 (brain floating point 16):** Originally introduced with Google TPuv2 and later adopted by Intel, Arm, and NVIDIA, BF16 became a common training format across accelerator ecosystems. Its adoption made format support in hardware, kernels, and distributed-training libraries a portability concern rather than a vendor-specific feature.

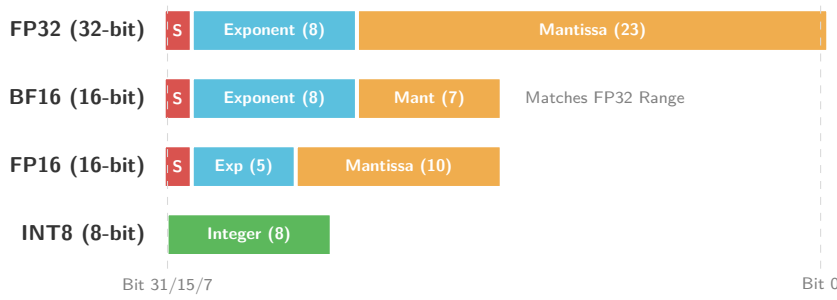


Figure D.3: Numerical Format Bit Layouts: A visual comparison of bit allocations. BF16 preserves FP32’s 8-bit exponent and therefore its normal exponent range. FP16 trades range for precision and often uses loss scaling to reduce underflow.

🔍 Systems Perspective D.5: The dynamic range wall

The choice of numerical format is a direct application of the iron law of ML systems (principle 3). Reducing precision from FP32 to BF16 or FP16 halves the data volume term, potentially doubling throughput on memory-bound workloads. However, the *type* of 16-bit format determines the engineering complexity:

- **Dynamic range (the exponent):** BF16 preserves the eight-bit exponent of FP32 and therefore the same normal exponent range (Kalamkar et al. 2019).
- **Precision (the mantissa):** FP16 has a larger 10-bit mantissa than BF16 (7 bits), offering higher precision for values within its range. Its five-bit exponent, however, raises underflow risk. FP16 training therefore often uses **loss scaling**, an operational step that multiplies gradients by a large constant to keep more values representable (Micikevicius et al. 2017).
- **Energy efficiency:** INT8 operations can be substantially more efficient than floating-point equivalents because they move less data and use simpler integer arithmetic paths. Moving to INT8 for inference is a primary lever for deploying neural networks under tight memory, latency, or power budgets (Jacob et al. 2018; Krishnamoorthi 2018).

D.4.2 Integer quantization

Quantization maps continuous floating-point values to discrete integers, typically INT8. The key challenge is choosing how to map the floating-point range to integers. Two approaches dominate.

For symmetric quantization, let $s_{\text{quant}} = \max |x|$ over the calibrated range and assume x is clipped to $[-s_{\text{quant}}, s_{\text{quant}}]$. The signed INT8 code is:

$$x_{\text{int}} = \text{round} \left(\frac{x}{s_{\text{quant}}} \times 127 \right)$$

This mapping works well for weight distributions centered around zero.

Asymmetric quantization handles distributions that are not centered (common after ReLU, which produces only nonnegative values) by shifting the range before scaling. Let $x_{\min}$ and $x_{\max}$ bound the calibrated range, set the normalization span $s_{\text{quant}} = x_{\max} - x_{\min}$, and clip x to $[x_{\min}, x_{\max}]$. A common unsigned 8-bit mapping is:

$$x_{\text{uint8}} = \text{round} \left(\frac{x - x_{\min}}{s_{\text{quant}}} \times 255 \right)$$

The choice between symmetric and asymmetric quantization depends on the tensor’s distribution and has measurable accuracy implications.

Summary

Machine-side diagnosis begins with units, ceilings, and working sets. Reference numbers establish orders of magnitude; the Roofline Model and the iron law identify whether operations, bytes, or latency bind; Amdahl's Law, Gustafson's Law, and Little's Law test speedup, scaling, and concurrency claims; and dimensional analysis catches impossible results. The memory hierarchy and interconnect show how those limits appear as latency and bandwidth constraints, while numerical representations show how dynamic range, precision, and quantization change storage, energy, throughput, and accuracy.

Treat peak specifications as ceilings, not predictions. Compare a measurement with the relevant ceiling, then trace the remaining gap to utilization, data movement, runtime overhead, architecture, or software. A back-of-envelope estimate is useful when it states its assumptions and units and makes the next measurement obvious.

System Assumptions

A napkin calculation is only as trustworthy as the inputs behind it, and those inputs are easy to lose track of: an accelerator’s peak rate, a model’s parameter count, an electricity price, an energy cost per operation. This appendix gathers those canonical values and unit conventions into a single reference sheet, allowing any quantitative estimate in the book to be audited, updated with newer figures, or checked across chapters. It also fixes the unit conventions those estimates depend on. The chapters can then argue from numbers without restating where each one came from.

How to Use This Appendix

When a chapter says an H100 delivers a certain ridge point, or that training a 7B model needs a certain amount of memory, the underlying bandwidths, capacities, and model sizes are listed here. A local estimate can use the relevant Value and Unit entries. The tables are grouped by topic: accelerators, reference models, energy per access, interconnect bandwidth, economics, production-scale anchors, and unit conventions. Assumptions come from vendor datasheet peaks, published studies or industry reports, illustrative market or grid statistics, and book conventions; section E.1 lists the provenance in one place.

The calculations below expose which assumptions dominate each estimate, so a changed hardware specification or price can propagate without changing the method.

The constants in this appendix support quick, reproducible calculations. The three examples below apply them to performance, capacity, and cost.



Napkin Math E.1: Classify a roofline regime

Problem: For the stated H100 and workload intensities, which operations are compute bound and which are memory bound?

Variables: For an H100 at FP16/BF16 peak, use 989 TFLOP/s peak compute and 3.35 TB/s memory bandwidth.

Math: Divide peak FLOP/s by memory bandwidth to get the roofline ridge point: $989 \text{ TFLOP/s} / 3.35 \text{ TB/s} \approx 295.2 \text{ FLOP/byte}$. A large general matrix multiply (GEMM) with $n = 4,096$ has intensity $n/3 \approx 1,365.3 \text{ FLOP/byte}$.

Result: Operations above 295.2 FLOP/byte are compute bound; operations below it are memory bound. The GEMM example is compute bound, while a single-token autoregressive decode with intensity $\approx 1 \text{ FLOP/byte}$ is deeply memory bound.

Systems insight: The same accelerator can be compute rich and memory constrained. Arithmetic intensity determines which resource the workload actually consumes.

Throughput is only one constraint. Training also requires enough capacity for weights, gradients, and optimizer state.

Napkin Math E.2: Estimate training-state memory

Problem: How much model-state memory is required to train a 7B model with mixed-precision Adaptive Moment Estimation (Adam)?

Variables: Mixed-precision Adam stores BF16 weights, gradients, FP32 master weights, momentum, and variance. The model-state budget is 16 bytes per parameter.

Math: For 7B parameters, model state is $7 \times 10^9 \times 16 \text{ bytes} = 112 \text{ GB}$.

Result: An H100 has 80 GB of high-bandwidth memory (HBM), so the model state alone exceeds a single accelerator before accounting for activations.

Systems insight: Optimizer state, not weights alone, sets the floor for training memory. Capacity planning that starts from parameter bytes underestimates the real requirement.

Fitting the model in memory establishes feasibility, but not operating cost. Power and elapsed time provide the remaining quantities for a first electricity estimate.

Napkin Math E.3: Estimate accelerator electricity cost

Problem: What is the accelerator-only electricity cost of the editorial GPT-3-scale scenario?

Variables: Use 1,024 A100s, 400 W per accelerator, ~25 days wall-clock, and \$0.12/kWh electricity.

Math: The run requires roughly 3.14×10^{23} FLOPs. The A100-equivalent electricity-cost estimate is $\sim 25 \text{ days wall-clock} \times 1,024 \text{ A100s} \times 24 \text{ h/day} \times 0.4 \text{ kW} \times \$0.12/\text{kWh} = \sim \$29,491.2$.

Result: The accelerator-only electricity cost is approximately $\sim \$29,491.2$. This accelerator thermal design power (TDP)-only estimate excludes host CPUs, networking, storage, cooling, and facility overhead. The original GPT-3 run used V100-era infrastructure; this is an editorial A100-equivalent scenario, not a reported configuration or a duration derived from peak FLOP/s alone.

Systems insight: Energy is measurable from power and time, but full cost accounting must also include capital utilization, networking, storage, staffing, and failed or repeated runs.

Accelerator Specifications

These are the **accelerator assumptions** used in the book's roofline, training-memory, energy, and cost examples: peak throughput, memory bandwidth, memory capacity, and TDP for each generation the chapters cite. Values are vendor datasheet peaks—ceilings for napkin math, not sustained utilization—drawn from NVIDIA product documentation and IEEE Micro architecture articles (NVIDIA Corporation 2017, 2020b, 2020a, 2021a, 2018, 2024; Choquette et al. 2021; Choquette 2023), AMD MI300X documentation (AMD 2023), Google Tensor Processing Unit (TPU) publications (Jouppi et al. 2023), and Google Cloud's current TPU v6e specification page (Google Cloud 2026d). Capacity rows preserve official decimal-GB nameplates. Tables progress from table E.2 (Volta) and table E.3 (Turing) through current and forward-looking generations.

Master accelerator spec matrix

The master accelerator matrix in table E.1 synthesizes specifications across key GPU, TPU, Wafer-Scale, and domain-specific AI accelerators.

Table E.1: Master Accelerator Specification Matrix: Comprehensive reference matrix comparing production AI hardware across architectures, process nodes, TDP, multi-precision compute throughputs (FP64 to INT8), memory capacity and bandwidth, on-chip SRAM capacity, and FP16 Roofline Ridge Point (I_{ridge} = Peak FP16 TFLOP/s/Memory Bandwidth (TB/s)).

Architecture/Chip	Node (nm)	TDP (W)	Compute Peak TFLOPs/TOPS	Memory Capacity & Bandwidth	On-Chip SRAM	I_{ridge} (FLOP/B)
NVIDIA V100 (SXM2 32 GB)	12nm (TSMC)	300	FP64: 7.8, FP32: 15.7, FP16: 125 (TC), INT8: 125 (TC)	32 GB HBM2 @ 0.90 TB/s	16 MB	138.9
NVIDIA A100 (80 GB SXM4)	7nm N7 (TSMC)	400	FP64: 9.7/19.5, FP32: 19.5, TF32: 156, FP16/BF16: 312, INT8: 624	80 GB HBM2e @ 2.04 TB/s	40 MB	153.0
NVIDIA H100 (80 GB SXM5)	4nm 4N (TSMC)	700	FP64: 34/67, FP32: 67, TF32: 494, FP16/BF16: 989, FP8: 1,978, INT8: 1,978	80 GB HBM3 @ 3.35 TB/s	50 MB	295.2
NVIDIA H200 (141 GB SXM5)	4nm 4N (TSMC)	700	FP64: 34/67, FP32: 67, TF32: 494, FP16/BF16: 989, FP8: 1,978, INT8: 1,978	141 GB HBM3e @ 4.80 TB/s	50 MB	206.0
NVIDIA B200 (192 GB SXM)	4nm 4NP (TSMC)	1000	FP64: 45/90, FP32: 90, TF32: 1,250, FP16/BF16: 2,250, FP8: 4,500, INT8: 4,500	192 GB HBM3e @ 8.00 TB/s	128 MB	281.3
NVIDIA L40S (48 GB PCIe)	4nm 4N (TSMC)	350	FP64: 1.4, FP32: 91.6, TF32: 183, FP16/BF16: 366, FP8: 733, INT8: 733	48 GB GDDR6 @ 0.86 TB/s	96 MB	423.6
Google TPU v1	28nm (TSMC)	75	INT8: 92 TOPS (FP formats N/A)	8 GB DDR3 @ 0.034 TB/s	28 MB	N/A (2,705.9 INT8)
Google TPU v2	16nm (TSMC)	280 (board)	BF16/FP16: 45 TFLOPs	16 GB HBM @ 0.60 TB/s	32 MB	75.0
Google TPU v3	16nm (TSMC)	450 (board)	BF16/FP16: 123 TFLOPs	32 GB HBM2 @ 0.90 TB/s	32 MB	136.7
Google TPU v4	7nm N7 (TSMC)	170	BF16/FP16: 275, FP32: 275, INT8: 275	32 GB HBM2 @ 1.20 TB/s	48 MB	229.2
Google TPU v5e	7nm N7 (TSMC)	140	BF16/FP16: 197, FP8: 394, INT8: 394	16 GB HBM2e @ 0.82 TB/s	32 MB	240.5
Google TPU v5p	4nm 4N (TSMC)	450	BF16/FP16: 459, FP8: 918, INT8: 918	95 GB HBM3 @ 2.76 TB/s	96 MB	166.3
Google TPU v6e (Trillium)	4nm 4N (TSMC)	400	BF16/FP16: 926, FP8: 1,852, INT8: 1,852	32 GB HBM3 @ 1.64 TB/s	64 MB	564.6
Cerebras WSE-3	5nm N5 (TSMC)	23,000	FP16/BF16: 125,000, FP8: 250,000, INT8: 250,000	44,000 GB SRAM @ 21,000 TB/s	44,000 MB	5.95
AMD Instinct MI300X	5nm+6nm 3D	750	FP64: 163.4, FP32: 163.4, TF32: 653.7, FP16/BF16: 1,307.4, FP8: 2,614.9	192 GB HBM3 @ 5.30 TB/s	256 MB	246.7
Intel Habana Gaudi 2	7nm N7 (TSMC)	600	FP32: 38.5, TF32: 154, FP16/BF16: 307, FP8: 614, INT8: 614	96 GB HBM2e @ 2.45 TB/s	48 MB	125.3
Intel Habana Gaudi 3	5nm N5 (TSMC)	900	FP32: 115, TF32: 460, FP16/BF16: 1,835, FP8: 3,670, INT8: 3,670	128 GB HBM3e @ 3.70 TB/s	96 MB	495.9

NVIDIA V100

Table E.2: NVIDIA V100 (Volta): Peak specs from NVIDIA's architecture whitepaper and product datasheet (NVIDIA Corporation 2017, 2020b). The V100 introduced Tensor Cores for mixed-precision training and anchors baseline-generation comparisons.

Assumption	Value	Unit
Peak FP16 tensor throughput (V100)	125	TFLOP/s
Peak FP32 throughput (V100)	15.7	TFLOP/s
HBM bandwidth (V100)	900	GB/s

Assumption	Value	Unit
HBM capacity (V100)	32	GB
TDP (V100)	300	W

NVIDIA T4

Table E.3: NVIDIA T4 (Turing): Peak specs from NVIDIA Tesla T4 product documentation (NVIDIA Corporation 2018). The 70 W TDP makes this low-power inference accelerator a useful cost-per-inference anchor.

Assumption	Value	Unit
Peak FP16 tensor throughput (T4)	65	TFLOP/s
Peak INT8 throughput (T4)	130	TOPS
Memory bandwidth (T4)	320	GB/s
TDP (T4)	70	W

NVIDIA A100

The Ampere-generation specs (table E.4) anchor most training examples in the book.

Table E.4: NVIDIA A100 (Ampere): Peak specs from NVIDIA’s architecture whitepaper, 80 GB datasheet, and architecture article (NVIDIA Corporation 2020a, 2021a; Choquette et al. 2021). The A100 is the most commonly cited accelerator in training examples; 80 GB HBM2e and TF32 Tensor Cores set memory-capacity and compute-intensity anchors.

Assumption	Value	Unit
Peak FP16 tensor throughput (A100)	312	TFLOP/s
Peak FP32 throughput (A100)	19.5	TFLOP/s
Peak INT8 throughput (A100)	624	TOPS
Peak TF32 throughput (A100)	156	TFLOP/s
HBM bandwidth (A100)	2039	GB/s
HBM capacity (A100)	80	GB
TDP (A100)	400	W

NVIDIA H100

For Hopper-era estimates, the reference values include FP8 Tensor Cores and the Transformer Engine (table E.5).

Table E.5: NVIDIA H100 (Hopper): Peak specs from the Hopper architecture overview (Choquette 2023). FP8 Tensor Cores and the Transformer Engine drive Hopper-generation training and serving estimates.

Assumption	Value	Unit
Peak FP16 tensor throughput (H100)	989	TFLOP/s
Peak FP32 CUDA throughput (H100)	67	TFLOP/s
Peak FP8 tensor throughput (H100)	1979	TFLOP/s
Peak INT8 throughput (H100)	1979	TOPS
Peak TF32 throughput (H100)	494	TFLOP/s
HBM bandwidth (H100)	3.35	TB/s
HBM capacity (H100)	80	GB
TDP (H100)	700	W

NVIDIA B200

Forward-looking capacity-planning examples rely on Blackwell-generation specs (table E.6).

Table E.6: NVIDIA B200 (Blackwell): Peak specs from NVIDIA Blackwell product documentation (NVIDIA Corporation 2024). Forward-looking capacity-planning examples use its HBM3e bandwidth and FP8 throughput.

Assumption	Value	Unit
Peak FP16 tensor throughput (B200)	2250	TFLOP/s
Peak FP8 tensor throughput (B200)	4500	TFLOP/s
Peak FP4 throughput (B200)	9000	TFLOP/s
HBM bandwidth (B200)	8	TB/s
HBM capacity (B200)	180	GB
TDP (B200)	1000	W

AMD Instinct MI300X

To compare with H100-class hardware used elsewhere, MI300X capacity, bandwidth, throughput, and power specifications provide a cross-vendor baseline (table E.7).

Table E.7: AMD Instinct MI300X: Peak specs from AMD Instinct MI300X product documentation (AMD 2023). High HBM capacity (192 GB) and bandwidth make it a common cross-vendor baseline for memory-bound workloads.

Assumption	Value	Unit
Peak FP16 tensor throughput (MI300X)	1307	TFLOP/s
HBM bandwidth (MI300X)	5.3	TB/s
HBM capacity (MI300X)	192	GB
TDP (MI300X)	750	W

Google TPU v4 and v6e

When comparing training economics across accelerator families, TPU figures offer an application-specific integrated circuit (ASIC)-based alternative (table E.8).

Table E.8: Google TPU v4 and v6e (Trillium): TPU v4 values come from Google TPU publications (Jouppi et al. 2023), and TPU v6e values come from Google Cloud’s current Trillium specification page (Google Cloud 2026d).

Assumption	Value	Unit
Peak BF16 throughput (TPU v4)	275	TFLOP/s
Memory bandwidth (TPU v4)	1200	GB/s
Peak BF16 throughput (TPU v6e)	918	TFLOP/s
Memory bandwidth (TPU v6e)	1638	GB/s

CPU and mobile/edge processors

Edge and mobile ML examples rely on a different performance baseline (table E.9), contrasting starkly with data center throughput to illustrate why deployment target shapes every design decision.

Table E.9: CPU, Mobile NPU, and Edge Device Specs: Illustrative reference points for edge and mobile examples (not vendor peaks for a single SKU). The contrast with data-center accelerators shows why deployment target shapes every design decision.

Assumption	Value	Unit
Peak FP32 throughput (reference CPU)	1	TFLOP/s
DRAM bandwidth (reference server)	50	GB/s

Assumption	Value	Unit
Reference NPU throughput (iPhone 15 Pro)	35	TOPS
Memory bandwidth (iPhone 15 Pro)	51.2	GB/s
TDP (mobile device, reference)	5	W
Power (edge object detector, reference)	2	W
Battery capacity (phone, reference)	15	Wh

Model Specifications

These are the **reference model assumptions** behind training-cost, memory-footprint, and inference-workload examples: parameter counts, per-inference FLOP budgets, and published training-scale anchors. Published model sizes follow primary papers, model reports, and official model documentation (Devlin et al. 2019; Radford et al. 2019; Brown et al. 2020; Dubey et al. 2024; He et al. 2016a; Sandler et al. 2018; Ultralytics 2023). The inference FLOP budgets are book-derived values: they use sequence length 128 for BERT-Base, 224×224 images for ResNet-50 and MobileNetV2, and 640×640 images for YOLOv8-Nano; the vision models count two FLOPs per multiply-add. GPT-3 training FLOPs follow (Brown et al. 2020); the 25-day, 1,024-A100 pairing is editorial. The GPT-4 parameter count and training GPU-days are public third-party mixture of experts (MoE) estimates (Patel and Wong 2023) because the GPT-4 technical report does not disclose architecture size. When a chapter estimates GPT-3-scale training time, it uses table E.10.

Table E.10: Reference Model Specifications: Parameter counts and FLOP budgets for worked examples. BERT, GPT-2, GPT-3, Llama, ResNet-50, MobileNetV2, and YOLOv8 rows use primary papers, model reports, or official model documentation (Devlin et al. 2019; Radford et al. 2019; Brown et al. 2020; Dubey et al. 2024; He et al. 2016a; Sandler et al. 2018; Ultralytics 2023). GPT-4 rows cite (OpenAI et al. 2023) for the official model family and (Patel and Wong 2023) for the public MoE parameter and GPU-day estimates used in this edition.

Assumption	Value	Unit
Inference FLOPs (BERT-Base)	2.2e+10	flop
Parameters (BERT-Base)	1.1e+08	param
Parameters (Llama 3 8B)	8.03e+09	param
Hidden dimension (GPT-2)	1600	-
Layers (GPT-2)	48	-
Parameters (GPT-2)	1.5e+09	param
Parameters (GPT-3)	1.75e+11	param
Scenario duration (GPT-3 scale)	25	d
Reference training FLOPs (GPT-3)	3.14e+23	flop
Parameters (GPT-4, public MoE estimate)	1.76e+12	param
Reference training GPU-days (GPT-4)	2.5e+06	GPU-days
Inference FLOPs (ResNet-50)	8.2e+09	flop
Parameters (ResNet-50)	2.56e+07	param
Inference FLOPs (MobileNetV2)	6e+08	flop
Parameters (MobileNetV2)	3.50487e+06	param
Inference FLOPs (YOLOv8-Nano)	8.7e+09	flop

Training Memory Conventions

Training-memory napkin math assumes the mixed-precision Adam storage model used in the napkin-math callout and several training chapters: BF16 weights, BF16 gradients, and FP32 master weights plus Adam first- and second-moment buffers (16 bytes per parameter in total). This is a book convention, not a measured hardware constant. The BF16 data path follows established BFLOAT16 mixed-precision practice (Kalamkar et al. 2019); the FP32-master pattern follows earlier FP16 work (Micikevicius et al. 2017; NVIDIA 2017), and Adam’s first- and second-moment buffers

follow (Kingma and Ba 2015). Table E.11 lists per-component byte widths; multiplying bytes per parameter (mixed-precision Adam) by the parameter count gives the nonactivation training-state footprint.

Table E.11: Training Memory Conventions: Per-parameter storage for mixed-precision Adam ($2 + 2 + 12 = 16$ bytes before activations). Book convention for napkin math; see (NVIDIA 2017) for mixed-precision training context.

Assumption	Value	Unit
Weight/gradient width (BF16)	2	bytes
Master weight width (FP32)	4	bytes
Master + Adam states per parameter (FP32)	12	bytes
Bytes per parameter (mixed-precision Adam)	16	bytes

Hardware and model assumptions fix *what* runs where; **energy assumptions** fix whether the design is thermally and economically viable at the operation and memory-access level.

Energy Constants

Horowitz’s 45 nm arithmetic and 32-bit DRAM estimates (Horowitz 2014), together with illustrative book anchors for register and SRAM access, MobileNetV2 inference, and 5G transfer, underpin the book’s efficiency comparisons but are not process-independent. Table E.12 lists the access-energy hierarchy from registers through DRAM. That hierarchy quantifies why data reuse dominates kernel design.

Table E.12: Energy per Operation and Access: Arithmetic and 32-bit DRAM estimates use Horowitz’s 45 nm model (Horowitz 2014), and the per-byte DRAM row is derived from that estimate. Register, SRAM, MobileNetV2, and 5G rows are illustrative book anchors. The roughly $6,400\times$ gap between a 0.1 pJ register access and a 640 pJ DRAM access explains why data reuse dominates ML kernel optimization.

Assumption	Value	Unit
Register access energy	0.1	pJ
L1 SRAM access energy	0.5	pJ
L2 SRAM access energy	2	pJ
DRAM access energy (32-bit)	640	pJ
DRAM access energy (per byte)	160	pJ/byte
FP16 multiply energy (45 nm)	1.1	pJ/multiply
FP32 multiply energy (45 nm)	3.7	pJ/multiply
INT8 multiply energy (45 nm)	0.2	pJ/multiply
MobileNetV2 inference energy (reference)	0.1	mJ
5G transfer energy per MB	100	mJ/MB

Precision & arithmetic energy hierarchy

The arithmetic energy hierarchy in table E.13 compares the energy required per operation (E_{op} in pJ/op) for additions and multiplications across floating-point, tensor, and integer precisions on modern 7nm, 5nm, and 3nm silicon process nodes.

Table E.13: Precision & Arithmetic Energy Hierarchy: Energy per operation (E_{op} in pJ/op) for Addition (Add) and Multiplication (Mul) across data formats (FP64, FP32, TF32, FP16, BF16, FP8 E4M3/E5M2, INT32, INT8, INT4) and semiconductor process nodes (7nm, 5nm, 3nm). Energy scaling illustrates the quadratic $O(b^2)$ multiplier vs linear $O(b)$ adder bitwidth dependence (Horowitz 2014; Dally et al. 2021).

Precision/Data Format	7nm Node (E_{op} pJ)	5nm Node (E_{op} pJ)	3nm Node (E_{op} pJ)	Energy Ratio vs FP32 (3nm)
FP64 (64-bit Float)	Add: 0.90/Mul: 2.50	Add: 0.58/Mul: 1.62	Add: 0.38/Mul: 1.05	$2.23\times$

Precision/Data Format	7nm Node (E_{op} pJ)	5nm Node (E_{op} pJ)	3nm Node (E_{op} pJ)	Energy Ratio vs FP32 (3nm)
FP32 (32-bit Float)	Add: 0.40/Mul: 1.10	Add: 0.26/Mul: 0.72	Add: 0.17/Mul: 0.47	1.00× (Ref)
TF32 (19-bit TensorFloat)	Add: 0.28/Mul: 0.65	Add: 0.18/Mul: 0.42	Add: 0.12/Mul: 0.27	0.57×
FP16 (16-bit IEEE Float)	Add: 0.20/Mul: 0.45	Add: 0.13/Mul: 0.29	Add: 0.08/Mul: 0.19	0.40×
BF16 (16-bit Bfloat16)	Add: 0.18/Mul: 0.40	Add: 0.12/Mul: 0.26	Add: 0.08/Mul: 0.17	0.36×
FP8 (E4M3/E5M2)	Add: 0.09/Mul: 0.20	Add: 0.06/Mul: 0.13	Add: 0.04/Mul: 0.08	0.17×
INT32 (32-bit Integer)	Add: 0.10/Mul: 0.80	Add: 0.06/Mul: 0.52	Add: 0.04/Mul: 0.34	0.72×
INT8 (8-bit Integer)	Add: 0.03/Mul: 0.20	Add: 0.02/Mul: 0.13	Add: 0.013/Mul: 0.085	0.18×
INT4 (4-bit Integer)	Add: 0.010/Mul: 0.060	Add: 0.006/Mul: 0.039	Add: 0.004/Mul: 0.025	0.053×

Energy costs operate at the chip level, but real ML systems also move data across interconnects—between accelerators, across racks, and over wide-area networks. The next section lists the bandwidth assumptions used when chapters estimate communication overhead.

Interconnect and Network Bandwidth

These **bandwidth assumptions** apply when chapters reason about gradient synchronization, pipeline bubbles, checkpoint I/O, or cross-data center latency. NVLink and PCIe rates follow accelerator product documentation (NVIDIA Corporation 2017, 2020a; Choquette 2023); the InfiniBand architecture specification anchors the protocol family (InfiniBand Trade Association 2000), while current high-speed product families and Ethernet roadmaps anchor modern link-rate examples (NVIDIA 2026f; Ethernet Alliance 2025); the speed-of-light-in-fiber floor is a physics identity. Table E.14 lists NVLink, InfiniBand, PCIe, Non-Volatile Memory Express (NVMe), and Ethernet rates together with the propagation speed of light in fiber.

Table E.14: Interconnect and Network Bandwidth: Link-family bandwidths use vendor specs, the InfiniBand architecture specification, and current product/roadmap documentation (InfiniBand Trade Association 2000; NVIDIA 2026f; Ethernet Alliance 2025). NVLink rates are aggregate bidirectional; other link rates are nominal per direction. Speed of light in fiber sets the cross-data-center latency floor.

Assumption	Value	Unit
NVLink bandwidth (V100)	300	GB/s
NVLink bandwidth (A100)	600	GB/s
NVLink bandwidth (H100)	900	GB/s
InfiniBand HDR link rate	200	Gb/s
InfiniBand NDR link rate	400	Gb/s
InfiniBand XDR link rate	800	Gb/s
PCIe Gen4 x16 rate	32	GB/s
PCIe Gen5 x16 rate	64	GB/s
NVMe sequential read bandwidth	7	GB/s
10 GbE link rate	10	Gb/s
100 GbE link rate	100	Gb/s
Speed of light in fiber	200000	km/s

Interconnect & network fabric physical spec sheet

The physical specification sheet in table E.15 details bandwidth, latency, framing, direct memory access (DMA) support, and energy per bit across host buses, accelerator interconnects, network fabrics, and cache-coherent interfaces.

Table E.15: Interconnect & Network Fabric Physical Spec Sheet: Comprehensive physical spec sheet for host buses, GPU interconnects, switches, network fabrics, and coherent interfaces (PCIe Gen3–Gen7, NVLink 1–5, NVSwitch 1–4, InfiniBand EDR–XDR, RoCE v2, CXL 1.1–3.1). Columns enumerate directional and aggregate bandwidth, PHY/link latency, line encoding/framing, DMA and GPUDirect capability, and physical layer energy cost (E_{link} in pJ/bit).

Bus/Interconnect Standard	Directional & Aggregate Bandwidth	Latency	Encoding/Framing	DMA/GPUDirect Support	Energy Cost (E_{link})
PCIe Gen3 (x16)	15.75 GB/s dir/31.5 GB/s agg	400–500 ns	128b/130b NRZ	PCIe P2P, GPUDirect RDMA	14.0 pJ/bit
PCIe Gen4 (x16)	31.50 GB/s dir/63.0 GB/s agg	250–400 ns	128b/130b NRZ	PCIe P2P, GPUDirect RDMA	10.0 pJ/bit
PCIe Gen5 (x16)	63.00 GB/s dir/126.0 GB/s agg	150–250 ns	128b/130b NRZ	PCIe P2P, GPUDirect RDMA, CXL 1.1/2.0	8.0 pJ/bit
PCIe Gen6 (x16)	126.00 GB/s dir/252.0 GB/s agg	100–150 ns	PAM4 (256B Flit)	PCIe P2P, GPUDirect RDMA, CXL 3.0/3.1	6.0 pJ/bit
PCIe Gen7 (x16)	252.00 GB/s dir/504.0 GB/s agg	<100 ns	PAM4 (512B Flit)	PCIe P2P, GPUDirect RDMA, CXL 3.1+	4.5 pJ/bit
NVLink 1 (P100)	20 GB/s dir/40 GB/s agg (link)	200–300 ns	NRZ (20 Gb/s)	GPUDirect P2P/RDMA	10.0 pJ/bit
NVLink 2 (V100)	25 GB/s dir/50 GB/s agg (link)	150–200 ns	NRZ (25.78 Gb/s)	GPUDirect P2P/RDMA	8.0 pJ/bit
NVLink 3 (A100)	25 GB/s dir/50 GB/s agg (link)	100–150 ns	NRZ (50 Gb/s)	GPUDirect P2P/RDMA	6.5 pJ/bit
NVLink 4 (H100)	25 GB/s dir/50 GB/s agg (link)	80–100 ns	PAM4 (100 Gb/s)	GPUDirect P2P, SHARP Aggregation	4.5 pJ/bit
NVLink 5 (B200)	50 GB/s dir/100 GB/s agg (link)	60–80 ns	PAM4 (200 Gb/s)	GPUDirect P2P, NVLink Network Offload	3.5 pJ/bit
NVSwitch 1 (Volta)	900 GB/s aggregate per chip	~100 ns	NRZ	Hardware P2P Crossbar Routing	8.5 pJ/bit
NVSwitch 2 (Ampere)	2.4 TB/s aggregate per chip	~90 ns	NRZ	Hardware P2P Crossbar, SHARP v2	6.5 pJ/bit
NVSwitch 3 (Hopper)	3.2 TB/s aggregate per chip	~70 ns	PAM4	SHARP v3 In-Network Compute	4.5 pJ/bit
NVSwitch 4 (Blackwell)	14.4 TB/s aggregate per chip	~50 ns	PAM4	SHARP v4, FP8 Reduction Engine	3.2 pJ/bit
InfiniBand EDR	100 Gbps (12.5 GB/s dir)	~0.50 μ s (500 ns)	64b/66b NRZ	GPUDirect RDMA (Verbs)	15.0 pJ/bit
InfiniBand HDR	200 Gbps (25.0 GB/s dir)	~0.60 μ s (600 ns)	64b/66b PAM4/NRZ	GPUDirect RDMA, SHARP v2	10.0 pJ/bit
InfiniBand NDR	400 Gbps (50.0 GB/s dir)	~0.50 μ s (500 ns)	256b/257b PAM4	GPUDirect RDMA, SHARP v3	7.0 pJ/bit
InfiniBand XDR	800 Gbps (100.0 GB/s dir)	~0.40 μ s (400 ns)	PAM4 (200G/lane Flit)	GPUDirect RDMA, SHARP v4	5.0 pJ/bit
RoCE v2 Ethernet	100–800 Gbps (12.5–100 GB/s dir)	1.0–2.5 μ s	NRZ/PAM4 (802.3)	GPUDirect RDMA (RoCEv2 UDP/IP)	8.0–12.0 pJ/bit
CXL 1.1	63 GB/s dir/126 GB/s agg (x16)	~200 ns	128b/130b NRZ	cxl.io, cxl.cache, cxl.mem	8.0 pJ/bit
CXL 2.0	63 GB/s dir/126 GB/s agg (x16)	~180 ns	128b/130b NRZ	SLD/MLD Memory Pooling	7.5 pJ/bit
CXL 3.0	126 GB/s dir/252 GB/s agg (x16)	~120 ns	256B Flit PAM4	P2P Memory Pooling & Fabric Switch	5.5 pJ/bit
CXL 3.1	126 GB/s dir/252 GB/s agg (x16)	~110 ns	256B Flit PAM4	Global Integrated Memory, TEE	5.0 pJ/bit

Economic Constants

These **pricing assumptions** underpin total cost of ownership (TCO) and energy-cost napkin math in table E.16. They are illustrative hyperscaler-order rates for ratio analysis (similar in spirit to carbon

accounting examples in (Patterson et al. 2021)), not quotes for a specific region or contract. Local values should replace these anchors when absolute price dominates.

Table E.16: Economic Assumptions: Illustrative cloud electricity and egress rates for TCO napkin math (2024–2025 order of magnitude). On-premise cost structures differ; relative magnitudes guide design trade-offs.

Assumption	Value	Unit
Cloud electricity price	0.12	dollar/kWh
Cloud egress price per GB	0.09	dollar/GB

Economic constants set the price per unit of compute and data transfer, but they mean little without a sense of the volumes involved. Production ML systems handle millions to billions of requests per day—numbers large enough to be difficult to internalize without concrete reference points.

Scale References

These **scale assumptions** in table E.17 anchor “how big is big?” for capacity-planning examples, providing illustrative order-of-magnitude baselines for email and search volume, autonomous-vehicle sensor streams, and standard 1080p and 4K video formats. They are magnitude anchors, not audited statistics for a specific year.

Table E.17: Production Scale and Data Rate References: Illustrative workload and sensor-rate anchors, plus standard 1080p and UHD 4K video parameters.

Assumption	Value	Unit
Gmail emails per day	1.21e+11	-
Google searches per day	8.5e+09	-
Waymo sensor data rate (low)	1	TB/h
Waymo sensor data rate (high)	19	TB/h
1080p frame width	1920	-
1080p frame height	1080	-
4K frame width	3840	-
4K frame height	2160	-
Bytes per RGB pixel	3	bytes
Video frame rate (standard)	30	Hz

The assumptions above use shared unit conventions: decimal data prefixes, distinct FLOPs and FLOP/s quantities, and the aliases defined in the next section.

Unit Conventions

Table E.18 fixes the unit conventions used in every quantitative example in this book. Each row gives the multiplier k in $1 \text{ alias} = k \text{ base}$. Data prefixes use decimal SI ($\text{KB} = 10^3$ bytes, not 1024). Binary IEC storage prefixes appear in a few storage-specific discussions but are omitted here because most fleet-scale estimates in the book use decimal KB/GB/TB. Throughput quantities (FLOP/s, GB/s) combine these aliases with time; adding incompatible dimensions (bytes to FLOP/s) is a category error in napkin math, not a unit conversion.

Hardware capacity is the one deliberate exception. Vendors label memory with decimal symbols but ship binary quantities. An accelerator sold as an 80 GB device provides 80×2^{30} bytes, and a microcontroller sold with 512 KB of SRAM provides 512×2^{10} bytes. This book keeps both the vendor’s number and the vendor’s label, so a capacity printed as 80 GB is the nameplate figure rather than a decimal conversion of it. Every other data quantity, including model footprints, activation sizes, and transfer volumes, uses the decimal prefixes above. Where a worked example subtracts a

computed footprint from a device capacity, both terms are carried in decimal so that the arithmetic printed on the page is the arithmetic a reader can reproduce.

Table E.18: Unit Conventions: Decimal SI aliases and scale factors (book convention for napkin math). Throughput forms such as FLOP/s and GB/s divide work or data by time using the same conventions.

Alias	Multiplier	Base unit
byte	1	byte
KB	1000	byte
MB	1e+06	byte
GB	1e+09	byte
TB	1e+12	byte
PB	1e+15	byte
flop	1	flop
GFLOPs	1e+09	flop
TFLOPs	1e+12	flop
ZFLOPs	1e+21	flop
param	1	param
Mparam	1e+06	param
Gbps	1e+09	bit/s
NS	10 ^{−9}	second
US	10 ^{−6}	second
MS	10 ^{−3}	second
second	1	second
hour	3600	second
day	86400	second
joule	1	joule
watt	1	watt
meter	1	meter

Each assumption also has a provenance record that lets readers trace the number back to its primary source.

E.1 Assumption Provenance

Table E.19 groups the appendix’s quantitative inputs by evidence class and identifies the primary references behind each group. Vendor specifications and standards define published limits, peer-reviewed measurements provide empirical anchors, and explicitly marked editorial scenarios supply illustrative values where no durable universal constant exists. This distinction lets readers judge the evidentiary weight of an input, update time-sensitive values, and reproduce an estimate with an appropriate local substitute.

Table E.19: Assumption Provenance Catalog: Quick map from appendix section to source class and bibliography.

Appendix section	Source type	Primary references
Accelerator specifications	Vendor datasheet peaks; editorial CPU/mobile anchors; Master Spec Matrix (table E.1)	(NVIDIA Corporation 2017, 2018, 2020a, 2024; Choquette et al. 2021; Choquette 2023; AMD 2023; Jouppi et al. 2023; Google Cloud 2026d); editorial for CPU/mobile rows
Model specifications	Published papers, model reports, and official docs; editorial GPT-3 scenario; GPT-4 size from public analysis	(Devlin et al. 2019; Radford et al. 2019; Brown et al. 2020; Dubey et al. 2024; He et al. 2016a; Sandler et al. 2018; Ultralytics 2023; OpenAI et al. 2023; Patel and Wong 2023)

Appendix section	Source type	Primary references
Training memory conventions	Book convention (mixed-precision Adam layout)	(Kingma and Ba 2015; Kalamkar et al. 2019; Micikevicius et al. 2017; NVIDIA 2017)
Energy constants	Published 45 nm arithmetic/DRAM estimates; VLSI synthesis node scaling; Precision Energy Hierarchy (table E.13)	(Horowitz 2014; Dally et al. 2021) for arithmetic/DRAM; editorial for register/SRAM, MobileNetV2, and 5G rows
Interconnect bandwidth	Vendor specs; InfiniBand standard; PCIe/NVLink/CXL standards; Physical Spec Sheet (table E.15)	(NVIDIA Corporation 2017, 2020a; Choquette 2023; InfiniBand Trade Association 2000; NVIDIA 2026f; Ethernet Alliance 2025)
Economic assumptions	Illustrative cloud/utility rates	(Patterson et al. 2021) (methodology context)
Scale references	Illustrative workload rates; standard video conventions	Editorial for workload rates; standard format definitions for video
Unit conventions	Decimal SI; book notation	Editorial

Summary

This appendix is the reference catalog for the quantitative inputs used throughout the book. It gathers accelerator and model specifications, training-memory conventions, energy constants, interconnect bandwidths, economic and scale references, unit conventions, and provenance. The values come from vendor peaks, published measurements, standards, and editorial scenarios, so they carry different evidentiary weight and age at different rates.

Use these tables as starting inputs, not universal constants. For each estimate, record the table row, unit convention, and provenance; replace time-sensitive peaks, prices, and rates with local values; distinguish a physical ceiling from a measured operating point and a scenario assumption; and report which assumptions dominate the result. This keeps napkin math reproducible and makes clear which input controls the conclusion.

Glossary

This glossary defines key terms used throughout this book and gathers them as a compact reference. Terms are organized alphabetically so they can be consulted alongside the chapters.

3

3DMark Graphics performance benchmark suite that evaluates real-time 3D rendering capabilities, measuring triangle throughput, texture fill rates, and modern features like ray tracing and DLSS performance.

A

A/B testing A controlled experimental method for comparing two versions of a system or model by randomly dividing users into groups and measuring performance differences between the variants.

ablation study An experiment that removes, disables, or isolates one component at a time to measure its contribution to model or system behavior.

activation-based pruning A pruning method that evaluates the average activation values of neurons or filters over a dataset to identify and remove neurons that consistently produce low activations and contribute little information to the network's decision process.

activation checkpointing A memory optimization technique that reduces memory usage during backpropagation by selectively discarding and recomputing activations instead of storing all intermediate results.

activation function A mathematical function applied to the weighted sum of inputs in a neural network neuron to introduce non-linearity, enabling the network to learn complex patterns beyond simple linear combinations.

activation memory Memory used to store intermediate layer outputs needed for gradient computation during training, scaling with batch size, sequence length, and model depth.

active learning An approach that intelligently selects the most informative examples for human annotation based on model uncertainty, reducing the amount of labeled data needed for effective training.

Adam optimization An adaptive learning rate optimization algorithm that combines momentum and RMSprop by maintaining exponentially decaying averages of both gradients and squared gradients for each parameter.

AdamW A variant of Adam that decouples weight decay from the adaptive gradient update, improving regularization while keeping the optimizer's adaptive moments.

AI Triad A framework modeling ML systems as three interdependent components: data that guides behavior, algorithms that learn pat-

terns, and computational infrastructure that enables training and inference. Limitations in any component constrain the capabilities of the others.

alerting Automated notification systems that inform teams when metrics exceed predefined thresholds or anomalies are detected in production ML systems.

AlexNet A landmark convolutional neural network architecture that won the 2012 ImageNet challenge, reducing error rates from 26 percent to 16 percent and sparking the deep learning renaissance.

AllReduce A collective communication operation in distributed computing where each process contributes data and all processes receive the combined result, commonly used for gradient aggregation in distributed training.

all-to-all communication A communication pattern where every device or process exchanges data with every other device, common in sharded embedding and expert-parallel workloads.

AlphaFold A landmark AI system developed by DeepMind that predicts the three-dimensional structure of many proteins from their amino acid sequences, achieving breakthrough accuracy without solving every folding problem and demonstrating how large-scale ML systems can accelerate scientific discovery. Its predictions still require confidence interpretation and, for many uses, experimental validation.

Amdahl's Law A speedup bound showing that overall improvement is limited by the portion of a workload or pipeline that remains serial or unoptimized.

AOT compilation Ahead-of-time compilation that converts a model, graph, or program into optimized executable code before deployment or execution, trading flexibility for lower runtime overhead.

Apache Kafka A distributed streaming platform that handles real-time data feeds using a publish-subscribe messaging system, commonly used for building ML data pipelines with high throughput and fault tolerance.

Apache Spark An open-source distributed computing framework that enables large-scale data processing across clusters of computers, revolutionizing ETL operations with in-memory computing capabilities.

application-specific integrated circuit (ASIC) Custom chips designed for specific computational tasks that offer superior performance and energy efficiency compared to general-purpose processors by abandoning general-purpose flexibility. Examples include Google's TPUs, Cerebras Wafer-Scale Engine, and Bitcoin mining ASICs.

architectural efficiency The dimension of model optimization that focuses on how computations are performed efficiently during training and inference by exploiting sparsity, factorizing large components, and dynamically adjusting computation based on input complexity.

arithmetic intensity The ratio of floating-point operations to bytes of memory accessed (FLOP/byte), used in roofline analysis to determine whether workloads are memory bound or compute bound and to guide optimization priorities.

artificial general intelligence AI systems intended to match human-level performance across a broad range of cognitive tasks; the term has no universally accepted capability threshold and implies no particular architecture.

artificial intelligence The field of computer science focused on creating systems that can perform tasks typically requiring human intelligence, such as perception, reasoning, learning, and decision-making.

artificial neural network A computational model inspired by biological neural networks, consisting of interconnected nodes (neurons) organized in layers that can learn patterns from data through adjustable weights and biases.

artificial neurons Basic computational units in neural networks that mimic biological neurons, taking multiple inputs, applying weights and biases, and producing an output signal through an activation function.

attention mechanism A neural network component that computes weighted connections between elements based on their content, allowing dynamic focus on relevant parts of the input rather than fixed architectural connections.

AUC Area under the ROC curve, a threshold-independent classification metric that compares true positive and false positive rates across decision thresholds.

audit trail An append-only record of who accessed or changed data, models, or system artifacts and when, supporting accountability, debugging, and compliance.

autograd tape A transient record of operations and saved values built during forward execution so reverse-mode automatic differentiation can compute gradients during the backward pass.

automatic differentiation A computational technique that automatically calculates exact derivatives of functions implemented as computer programs by systematically applying the chain rule at the elementary operation level, essential for training

neural networks through gradient-based optimization.

automatic mixed precision A training technique that automatically manages the use of different numerical precisions (FP16, FP32) to optimize memory usage and computational speed while maintaining model accuracy.

AutoML Automated machine learning that automates model design decisions, including architecture search, hyperparameter optimization, and feature selection, to reduce manual effort.

autoscaling Dynamic adjustment of compute resources based on workload demand, automatically scaling up during peak usage and scaling down during low usage to optimize costs and performance.

B

backpropagation An algorithm that computes gradients of the loss function with respect to network weights by propagating error signals backward through the network layers, enabling systematic weight updates during training.

backward pass The training phase that propagates gradients from the loss back through the network to compute parameter gradients, pairing with the forward pass.

bandwidth The maximum rate of data transfer across a communication channel or memory interface, typically measured in bytes per second and critical for optimizing data movement in AI accelerators.

batch inference The process of running inference on a large dataset in bulk, typically as a scheduled job, as opposed to real-time inference on individual requests. Batch inference enables higher throughput by amortizing overhead across many inputs.

batch ingestion A data processing pattern that collects and processes data in groups or batches at scheduled intervals, suitable for scenarios where real-time processing is not critical.

batch normalization A technique that normalizes inputs to each layer to have zero mean and unit variance, which stabilizes training and often allows for higher learning rates and faster convergence, and subsequently applies a learnable scale and shift to preserve representational power.

batch processing The technique of processing multiple data samples simultaneously to amortize computation and memory access costs, improving overall throughput in neural network training and inference.

batch size The number of training examples processed simultaneously during one iteration of neural network training, affecting both computational efficiency and gradient estimation quality.

batch throughput optimization Techniques for maximizing the number of samples processed per unit time when handling multiple inputs simultaneously, using parallelism and batching efficiencies.

batched operations Matrix computations that process multiple inputs simultaneously, converting matrix-vector operations into more efficient matrix-matrix operations to improve hardware utilization.

benchmark engineering The systematic design and development of performance evaluation frameworks, involving test harness creation, metric selection, and result interpretation methodologies.

benchmark harness Systematic infrastructure component that controls test execution, manages input delivery, and collects perfor-

mance measurements under controlled conditions to ensure reproducible evaluations.

benchmarking Systematic evaluation of compute performance, algorithmic effectiveness, and data quality in machine learning systems to optimize performance across diverse workloads and ensure reproducibility.

BERT Bidirectional Encoder Representations from Transformers, a transformer-based language model introduced by Google in 2018 that revolutionized natural language processing through masked language modeling pretraining.

BF16 A 16-bit floating-point format developed by Google Brain that maintains the same dynamic range as FP32 but with reduced precision, making it particularly suitable for deep learning training.

bias A learnable parameter added to the weighted sum in each neuron that shifts the activation function, allowing neurons to activate even when all inputs are zero and providing additional flexibility for the network to fit complex patterns.

binarization An extreme quantization technique that reduces neural network weights, activations, or both to binary values (typically -1 and +1), achieving maximum compression but often requiring specialized training procedures and hardware support.

biological neuron A cell in the nervous system that receives, processes, and transmits information through electrical and chemical signals, serving as inspiration for artificial neural networks.

bitter lesson Richard Sutton's 2019 observation that general methods using computation consistently outperform approaches encoding human expertise, suggesting that systems engineering enabling computational scale is central to AI advancement.

black box A system whose inputs and outputs are observable but whose internal workings are inaccessible or opaque. This opacity is particularly problematic when the system makes consequential decisions without explaining its reasoning.

BLAS Basic Linear Algebra Subprograms, a specification for low-level routines that perform common linear algebra operations such as vector addition, scalar multiplication, dot products, and matrix operations, forming the computational foundation of modern ML frameworks.

block sparse matrix A sparse matrix whose nonzero values appear in dense blocks rather than isolated elements, preserving structure that hardware kernels can exploit efficiently.

blue-green deployment A deployment strategy using two production environments so traffic can switch to a validated new version while the old version remains available for rollback.

bounding box A rectangular annotation that identifies object locations in images by drawing a box around each object of interest, commonly used in computer vision training datasets.

brittleness The tendency of rule-based AI systems to fail completely when encountering inputs that fall outside their programmed scenarios, no matter how similar those inputs might be to what they were designed to handle.

C

caching A technique for storing frequently accessed data in high-speed storage systems to

reduce retrieval latency and improve system performance in ML pipelines.

caching allocator A framework memory manager that reuses device-memory blocks instead of calling the hardware allocator for every tensor, reducing allocation latency and fragmentation.

calibration The process in post-training quantization of analyzing a representative dataset to determine optimal quantization parameters, including scale factors and zero points, that minimize accuracy loss when converting from high to low precision.

canary deployment A deployment strategy where new model versions receive a small percentage of production traffic to validate behavior before full rollout, enabling early detection of issues with minimal user impact.

CAP theorem A distributed systems principle stating that, during a network partition, a data store cannot guarantee both linearizable consistency and availability for every request.

carbon footprint The total greenhouse gas emissions, typically measured in CO₂ equivalent, produced directly and indirectly by training and operating an ML system.

carbon intensity The emissions produced per unit of energy, usually measured as kg CO₂e per kWh, used to convert ML energy use into carbon impact.

Cerebras Wafer-Scale Engine In the CS-2 generation, a single-wafer processor containing 2.6 trillion transistors and 850,000 cores, designed to eliminate inter-device communication bottlenecks in large-scale machine learning training.

chain rule The calculus rule enabling computation of derivatives for composite functions, fundamental to backpropagation.

channelwise quantization A quantization granularity approach where each channel in a layer uses its own set of quantization parameters, providing more precise representation than layerwise quantization while maintaining hardware efficiency.

CI/CD pipelines Continuous Integration and Continuous Delivery automated workflows that streamline model development by integrating testing, validation, and deployment processes.

circuit breaker pattern A resilience pattern that stops or redirects calls to an unhealthy service after repeated failures to prevent cascading failure and enable fallback behavior.

classification labels Simple categorical annotations that assign specific tags or categories to data examples, representing the most basic form of supervised learning annotation.

CloudSuite Benchmark suite developed at EPFL that addresses modern data center workloads including web search, data analytics, and media streaming, measuring end-to-end performance across network, storage, and compute dimensions.

cold-start performance Time required for a system to transition from idle state to active execution, particularly important in serverless environments where models are loaded on demand.

collaborative filtering A technique used in recommendation systems that predicts user preferences by identifying patterns in interactions from many users, using the collective behavior of the crowd rather than just item properties.

communication tax The performance penalty incurred in distributed systems due to the latency and bandwidth costs of synchronizing state between nodes.

- compound AI systems** AI architectures that combine multiple specialized models and components working together, rather than relying on a single monolithic model, enabling modularity, specialization, and improved interpretability.
- compressed sparse row (CSR)** A memory-efficient storage format for sparse matrices that uses three arrays (values, column indices, row pointers) to store only nonzero elements, reducing memory from $\mathcal{O}(N^2)$ (for an $N \times N$ matrix) to $\mathcal{O}(K + N)$ where K is the number of nonzeros.
- computational graph** A directed graph representing the sequence of operations in neural network computation, enabling automatic differentiation.
- computer engineering** An engineering discipline that emerged in the late 1960s to address the growing complexity of integrating hardware and software systems, combining expertise from electrical engineering and computer science to design and build complex computing systems.
- concept drift** The phenomenon where $P(Y|X)$, the statistical relationship between inputs and targets, changes over time, distinct from data drift in $P(X)$; this can, but need not, degrade a fixed model's performance. Detecting it typically requires outcome labels because input drift alone does not establish a conditional change.
- conditional computation** A dynamic optimization technique where different parts of a neural network are selectively activated based on input characteristics, reducing computational load by skipping unnecessary computations for specific inputs.
- consensus labeling** A quality control approach that collects multiple annotations for the same data point to identify controversial cases and improve label reliability through inter-annotator agreement.
- containerization** Packaging applications and their dependencies into portable, isolated containers using tools like Docker to ensure consistent execution across different environments.
- continuous batching** An LLM serving technique that adds and removes requests from an inference batch between token-generation steps as sequences start and finish.
- continuous integration** A software development practice where code changes are automatically integrated, tested, and validated multiple times per day to detect issues early in the development cycle.
- convolution** A mathematical operation that slides a filter (kernel) across input data to extract features such as edges, textures, or patterns. Convolution is fundamental to convolutional neural networks and particularly effective for processing images and spatial data.
- convolutional neural network** A specialized neural network architecture designed for processing grid-like data such as images, using convolutional layers that apply filters to detect local features.
- coreset** A small subset of a dataset selected to preserve important statistical or training properties of the full dataset while reducing training or evaluation cost.
- correction cascade** An ML technical-debt pattern where fixing one component creates new failures or redesign needs elsewhere because data and model dependencies propagate through the system.
- covariate shift** A distribution shift where input feature distributions change while the relationship between features and labels remains stable.
- CP decomposition** CANDECOMP/PARAFAC decomposition that expresses a tensor as a sum of rank-one components, used to compress neural network layers by reducing the number of parameters while preserving computational functionality.
- credit assignment problem** The challenge of determining which weights in a multi-layer network contributed to prediction errors.
- CRISP-DM** Cross-Industry Standard Process for Data Mining, a structured methodology developed in 1996 that defines six phases for data projects: business understanding, data understanding, data preparation, modeling, evaluation, and deployment.
- critical batch size** The batch size beyond which larger batches provide diminishing optimization benefit and may reduce generalization, limiting simple linear learning-rate scaling.
- cross-entropy loss** A loss function commonly used in classification tasks that measures the difference between predicted probability distributions and true class labels, providing strong gradients for effective learning.
- crowdsourcing** A collaborative data collection approach that uses distributed individuals via the internet to perform annotation tasks, enabling scalable dataset creation through platforms like Amazon Mechanical Turk.
- cuBLAS** NVIDIA's CUDA Basic Linear Algebra Subprograms library that provides GPU-accelerated implementations of standard linear algebra operations, enabling high-performance matrix computations on NVIDIA graphics processing units.
- CUDA (Compute Unified Device Architecture)** NVIDIA's parallel computing platform and programming model that enables general-purpose computing on graphics processing units (GPUs), allowing machine learning frameworks to use massive parallelism for accelerated tensor operations.
- CUDA stream** An ordered queue of GPU work used to schedule kernels, memory transfers, and synchronization events so independent operations can overlap.
- D**
- D-A-M taxonomy** A diagnostic framework that analyzes ML system bottlenecks through three interacting axes: Data (information flow, bounded by bandwidth), Algorithm (mathematical logic, bounded by total operations), and Machine (physical execution, bounded by peak throughput).
- Dartmouth conference** The legendary 8-week workshop at Dartmouth College in 1956 where AI was officially born, organized by John McCarthy, Marvin Minsky, Nathaniel Rochester, and Claude Shannon, following their 1955 proposal, which introduced the term artificial intelligence.
- data as source code** The Software 2.0 principle that training data defines ML system behavior, so dataset changes act like behavior-changing code changes.
- data augmentation** The process of artificially expanding training datasets by creating modified versions of existing data through transformations like rotation, scaling, or noise injection.
- data cascades** Systemic failures where data quality issues compound over time, creating downstream negative consequences such as model failures, costly rebuilding, or project termination.
- data center** A facility that houses computer systems and associated components such as telecommunications and storage systems, typically containing thousands of servers for cloud computing operations.
- data-centric approach** A machine learning paradigm that prioritizes improving data quality, diversity, and curation rather than solely focusing on model architecture improvements to achieve better performance.
- data-centric computing** Systems optimized for the efficient ingestion of data and iterative refinement of model parameters, where the programmer's job is to curate data.
- data contracts** Explicit agreements between data producers and consumers defining schema, quality expectations, and service levels to prevent downstream breakage.
- data curation** The process of selecting, organizing, and maintaining high-quality datasets by removing irrelevant information, correcting errors, and ensuring data meets specific standards for machine learning applications.
- data debt** Accumulated data quality, documentation, schema, lineage, or freshness liabilities that compound over time and degrade model behavior.
- data drift** The phenomenon where the input distribution $P(X)$ changes over time even though code remains unchanged; this may affect model performance but does not guarantee degradation. It can be monitored without labels, although labels are needed to measure its effect on quality.
- data echoing** A training input-pipeline technique that reuses batches, often with fresh augmentation, when data preparation is slower than accelerator consumption.
- data governance** The framework of policies, procedures, and technologies that ensure data security, privacy, compliance, and ethical use throughout the machine learning pipeline.
- data gravity** The architectural pull created by large data volumes, transfer time, bandwidth limits, and egress cost, which tends to draw computation toward the data.
- data ingestion** The process of collecting and importing raw data from various sources into a system where it can be stored, processed, and prepared for machine learning applications.
- data lake** A storage repository that holds structured, semi-structured, and unstructured data in its native format, using schema-on-read approaches for flexible data analysis.
- data lakehouse** A storage architecture that combines data lake flexibility with warehouse-style query semantics, schema management, and transactional guarantees.
- data lineage** The documentation and tracking of data flow through various transformations and processes, providing visibility into data origins and modifications for compliance and debugging.
- data locality** The principle that data and computation should be colocated when transfer latency, bandwidth, or cost dominates remote processing.
- data parallelism** A distributed training strategy that splits the dataset across multiple devices while each device maintains a complete copy of the model, enabling parallel computation of gradients.
- data pipeline** The infrastructure and workflows that automate the movement and transformation of data from sources through processing stages to final storage or consumption.
- data quality** The degree to which data meets requirements for accuracy, completeness, consistency, and timeliness, directly impacting machine learning model performance.

data quality multiplier The idea that improvements in data quality can amplify gains from model and system changes because all downstream stages depend on the data.

data validation The systematic verification that collected data meets quality standards, is properly formatted, and contains accurate information suitable for machine learning model training and evaluation.

data versioning The practice of tracking and managing different versions of datasets over time, similar to code versioning, to ensure reproducibility and enable rollback to previous data states when needed.

data warehouse A centralized repository optimized for analytical queries (OLAP) that stores integrated, structured data from multiple sources in a standardized schema.

dataflow architecture Specialized computing architecture where instruction execution is determined by data availability rather than a program counter, enabling highly parallel processing of neural network operations.

dataflow challenges Technical difficulties in managing data movement and dependencies in hardware accelerators, including memory bandwidth limitations and synchronization requirements.

datasheets for datasets Documentation for training data that captures provenance, collection methodology, demographic composition, and known limitations affecting model behavior.

dead letter queue A separate storage mechanism for data that fails processing, allowing for later analysis and potential reprocessing of problematic data without blocking the main pipeline.

deduplication The removal of exact, near-duplicate, or semantically redundant samples from a dataset to reduce storage, training compute, and leakage risk.

deep learning A subfield of machine learning that uses artificial neural networks with multiple layers to automatically learn hierarchical representations from data without explicit feature engineering.

demographic parity A fairness criterion requiring that the probability of receiving a positive prediction is independent of group membership across protected attributes.

Dennard scaling The observation that as transistors became smaller, their power density remained constant, enabling higher performance at the same power envelope. Its breakdown around 2005 ended the era of free frequency scaling.

dense layer A fully-connected neural network layer where each neuron receives input from all neurons in the previous layer, enabling comprehensive information integration across features.

deployment constraints Operational limitations such as hardware resources, network connectivity, regulatory requirements, and integration requirements that influence how machine learning models are implemented in production environments.

deployment paradigm A distinct approach to hosting and executing ML models characterized by specific resource constraints and operational properties, such as Cloud ML, Edge ML, Mobile ML, or TinyML.

depthwise separable convolution A convolution factorization that applies spatial filtering independently per channel and then mixes channels with a pointwise convolution to reduce computation.

DevOps Software development practice that combines development and operations teams to shorten development cycles and

deliver high-quality software through automation and collaboration.

Dhrystone Integer-based benchmark introduced in 1984 that measures integer and string operations in DMIPS (Dhrystone MIPS), designed to complement floating-point benchmarks with typical programming constructs.

diabetic retinopathy A diabetes complication that damages blood vessels in the retina, serving as a leading cause of preventable blindness and a key application area for medical AI screening systems.

differential privacy A mathematical framework for quantifying and limiting privacy loss when releasing statistical information about datasets by bounding how much the output distribution can change when any one individual's data is added or removed.

direct memory access (DMA) A mechanism that lets a device move data to or from memory without CPU-managed copying for each transfer, enabling overlap and reducing host overhead.

disaggregated evaluation The practice of breaking down model performance metrics by demographic groups or other factors to reveal disparities that are hidden by aggregate measures.

dispatch tax Runtime overhead from launching and orchestrating many small operations through the host framework instead of executing fused or compiled graph regions.

distributed computing An approach that processes data across multiple machines or processors simultaneously, enabling scalable handling of large datasets through frameworks like Apache Spark.

distributed intelligence The placement of computational capabilities across multiple devices and locations rather than relying on a single centralized system, enabling local processing and decision-making.

distributed training A method of training machine learning models across multiple machines or devices to handle larger datasets and models that exceed single-device computational or memory capacity.

distribution shift A change in the statistical properties of data between training and deployment, or over time during deployment. Types include covariate shift (input distribution changes), label shift (output distribution changes), and concept drift (relationship between inputs and outputs changes).

DLRM Deep Learning Recommendation Model, an architecture developed by Meta that combines categorical embeddings with a bottom multi-layer perceptron (MLP) and top MLP to handle the massive scale and sparsity of recommendation system workloads.

domain-specific architecture Hardware designs tailored to optimize specific computational workloads, trading flexibility for improved performance and energy efficiency compared to general-purpose processors.

dropout A regularization technique that randomly sets a fraction of input units to zero during training to prevent overfitting and improve generalization.

dying ReLU problem A failure mode where ReLU neurons become permanently inactive and output zero for all inputs, preventing them from contributing to learning when weighted inputs consistently produce negative preactivations.

dynamic batching A serving technique that waits briefly to group independent requests into a batch, improving throughput while adding controlled batch-formation latency.

dynamic graph A computational graph that is built and modified during program execution, allowing for flexible model architectures and easier debugging but potentially limiting optimization opportunities compared to static graphs.

dynamic pruning A pruning approach that changes which parameters or units are inactive in response to input data or training dynamics, rather than relying only on a fixed pruning decision.

dynamic quantization The process of storing neural-network weights at lower precision and determining activation quantization parameters at runtime, reducing memory use while requiring supported low-precision kernels for speedup.

dynamic random-access memory (DRAM) A type of volatile memory that stores data in capacitors and requires periodic refresh cycles, commonly used as main memory in computer systems.

dynamic voltage and frequency scaling (DVFS) Power management technique that adjusts processor voltage and clock frequency based on workload demands to optimize energy consumption while maintaining performance.

E

eager execution An execution mode where operations are evaluated immediately as they are called in the code, providing intuitive debugging and development experience but potentially sacrificing some optimization opportunities available in graph-based execution.

early exit architectures Neural network designs that include multiple prediction heads at different depths, allowing samples to exit early when confident predictions can be made, reducing average computational cost per inference.

edge computing A distributed computing paradigm that brings computation and data storage closer to the sources of data, reducing latency and bandwidth usage.

edge deployment A deployment strategy where machine learning models run locally on devices at the network edge rather than in centralized cloud servers, reducing latency and enabling operation without constant internet connectivity.

efficiency frontier The optimal trade-off curve between accuracy and computational cost (latency, energy, or memory), where no model exists that is both more accurate and more efficient.

EfficientNet A family of neural network architectures discovered through neural architecture search that achieves better accuracy-efficiency trade-offs by using compound scaling to balance network depth, width, and input resolution.

EL2N (Error L2-Norm) A sample-scoring method that ranks examples by early prediction error, often using a proxy model, to identify informative boundary cases for data selection.

ELIZA One of the first chatbots created by MIT's Joseph Weizenbaum in 1966 that could simulate human conversation through pattern matching and substitution, notable because people began forming emotional attachments to this simple program.

embedded systems Computer systems with dedicated functions within larger mechanical or electrical systems, typically designed for specific tasks with real-time computing constraints.

embedding sharding Splitting large embedding tables across devices or nodes so they fit in aggregate memory, while adding communication to gather embeddings for each batch.

embedding table A lookup table that maps discrete IDs or tokens to dense vectors, often dominating memory in recommendation models and large vocabularies.

emergent behaviors Unexpected system-wide patterns or characteristics that arise from the interaction of individual components, often becoming apparent only when systems operate at scale or in real-world conditions.

encoder-decoder An architectural pattern where an encoder processes input into an intermediate representation and a decoder generates output from this representation, commonly used in sequence-to-sequence tasks.

end-to-end benchmarks Comprehensive evaluation methodology that assesses entire AI system pipelines including data processing, model execution, postprocessing, and infrastructure components.

energy efficiency The measure of computational work performed per unit of energy consumed, typically expressed as operations per joule and crucial for battery-powered and data center deployments.

energy per inference The total energy consumed to produce one model prediction, including model execution and relevant system overhead.

epoch One complete pass through the entire training dataset during neural network training, consisting of multiple batch iterations depending on dataset size and batch size.

equal opportunity A fairness criterion requiring equal true positive rates across different demographic groups.

equalized odds A fairness criterion requiring that both true positive and false positive rates are equal across different demographic groups.

expected calibration error (ECE) A metric measuring the gap between predicted confidence and observed accuracy across confidence bins, used to assess probability calibration.

experiment tracking The systematic recording and management of machine learning experiments, including hyperparameters, model versions, training data, and performance metrics, to enable comparison and reproducibility.

expert systems AI systems first developed in the mid-1960s that captured human expert knowledge in specific domains, exemplified by MYCIN for diagnosing blood infections, representing a shift from general AI to domain-specific applications.

extract, load, transform (ELT) A data processing paradigm that first loads raw data into the target system before applying transformations, providing flexibility for evolving analytical needs.

extract, transform, load (ETL) A traditional data processing paradigm that transforms data before loading it into a data warehouse, resulting in ready-to-query formatted data.

F

fairness laundering A failure mode where removing explicit protected attributes makes a system appear compliant while correlated proxy variables still reproduce discriminatory outcomes.

false positive rate (FPR) The proportion of actual negative cases incorrectly classified as positive, computed as false positives divided by false positives plus true negatives.

FarmBeats A Microsoft Research project that applies machine learning and IoT technologies to agriculture, using edge computing to collect real-time data on soil conditions and crop health while demonstrating distributed AI systems in challenging real-world environments.

feature engineering The process of manually designing and extracting relevant features from raw data to improve machine learning model performance, largely automated in deep learning systems.

feature map The output of a convolutional layer representing the response of learned filters to different spatial locations in the input, capturing detected features at various positions.

feature store A specialized data storage system that provides standardized, reusable features for machine learning, enabling feature sharing across multiple models and teams.

federated learning A machine learning approach that trains algorithms across decentralized edge devices or servers holding local data samples, without exchanging the raw data.

feedback loop A cyclical process where outputs influence future inputs. In ML, this includes both beneficial cycles (where model outputs inform system improvement) and harmful ones (where a model's predictions influence its own future training data, potentially reinforcing and amplifying initial biases over time).

feedforward network A neural network architecture where information flows in one direction from input to output layers without cycles, forming the foundation for many deep learning models.

field-programmable gate array (FPGA) A reconfigurable integrated circuit that can be programmed after manufacturing to implement custom digital circuits and specialized computations, offering flexibility between general-purpose processors and ASICs.

fine-tuning Adapting a pretrained model to a target task or domain by continuing training on task-specific data.

Five-Pillar Framework An organizational structure for ML systems engineering comprising five interconnected disciplines: Data Engineering, Training Systems, Deployment Infrastructure, Operations and Monitoring, and Ethics and Governance.

FlashAttention An IO-aware attention algorithm that tiles computation to avoid materializing the full attention matrix in high-bandwidth memory, reducing memory traffic and enabling longer sequences.

floating-point unit (FPU) A specialized processor component designed to perform arithmetic operations on floating-point numbers with high precision and efficiency.

FLOP/s Floating-Point Operations per Second, a measure of computational throughput that quantifies the number of floating-point operations a system can perform per second.

forgetting events Events where a model changes from classifying an example correctly to misclassifying it later in training, marking difficult or informative samples.

forward pass The process of computing neural network predictions by passing input data through successive layers, applying weights, biases, and activation functions at each stage to produce outputs. Also called *forward propagation*.

foundation model Large-scale machine learning models trained on broad data that can be adapted to a wide range of downstream tasks, serving as a base for specialized applications.

four pillars framework A data engineering framework organizing concerns into Quality, Reliability, Scalability, and Governance to manage the data lifecycle.

FP16 16-bit floating-point numerical representation that reduces memory usage and accelerates computation on modern hardware accelerators while maintaining acceptable precision for many machine learning applications.

FP32 32-bit floating-point numerical representation that provides standard precision for mathematical computations but requires more memory and computational resources than lower-precision formats.

FP32 to INT8 A common quantization transformation that converts 32-bit floating-point weights and activations to 8-bit integers, achieving roughly 4× memory reduction while maintaining acceptable accuracy for many models.

FP8 An 8-bit floating-point format family used on newer accelerators for high-throughput training and inference, requiring careful scaling because of limited precision and range.

framework decomposition The systematic breakdown of neural network frameworks into hardware-mappable components, enabling efficient distribution of operations across processing elements.

G

GEMM General Matrix Multiply operations that follow the pattern $C = \alpha AB + \beta C$, representing the fundamental computational kernel underlying most neural network operations including fully connected layers and convolutional layers.

GEMV General Matrix-Vector multiplication operations that compute the product of a matrix and a vector, commonly used in neural network computations and requiring careful optimization for memory access patterns.

generalization The ability of a machine learning model to perform well on unseen data that differs from the training set, often improved through diverse and high-quality training data.

generalization gap The difference between a model's performance on training data and its performance on held-out data. A large generalization gap can indicate overfitting to the training data.

generative AI A category of artificial intelligence systems capable of creating new content such as text, images, audio, or video based on learned patterns from training data.

Goodhart's Law The observation that when a measure becomes an optimization target, it can stop representing the underlying goal.

GPT-2 A 1.5-billion parameter autoregressive language model released by OpenAI in 2019, serving as a primary lighthouse model for analyzing memory bandwidth constraints in transformer inference.

graceful degradation A system design principle where services continue functioning with reduced capabilities when faced with partial failures or data unavailability.

gradient accumulation A technique that simulates larger batch sizes by accumulating gradients from multiple smaller batches before updating model parameters, enabling training with limited memory.

gradient-based pruning A pruning method that uses gradient information to rank param-

eters, neurons, or filters as candidates for removal.

gradient clipping An optimization technique that mitigates exploding gradients by limiting their magnitude during backpropagation, typically by scaling gradients when their norm exceeds a threshold.

gradient compression A technique used in distributed training to reduce the communication overhead by compressing gradient information exchanged between computing nodes.

gradient descent An optimization algorithm that iteratively adjusts neural network parameters in the direction that minimizes the loss function, using gradients to determine update directions and magnitudes.

gradient synchronization The process in distributed training where locally computed gradients are aggregated across devices to ensure all devices update their parameters consistently.

GraNd (Gradient Normed) A data selection score based on the norm of an example's gradient during early training, identifying samples with larger learning signal.

graph break A point where a compiler cannot capture part of a program and must fall back to eager execution, splitting optimized graph regions.

graphics processing unit (GPU) A specialized processor originally designed for rendering graphics that provides massive parallel computing capabilities essential for efficient neural network computation, training, and inference.

Green AI A movement in AI research and practice that prioritizes computational efficiency and energy consumption as primary metrics alongside traditional performance metrics like accuracy.

Green500 Ranking system that evaluates the world's most powerful supercomputers based on energy efficiency measured in FLOP/s per watt rather than raw computational performance.

ground truth The reference target treated as correct for training or evaluation, often derived from measurement or annotation and therefore not necessarily identical to objective reality.

GRU Gated Recurrent Unit, a simplified variant of LSTM that uses fewer gates while maintaining the ability to capture long-term dependencies in sequential data.

H

hardware abstraction The layer in ML frameworks that provides a unified interface to diverse computing hardware (CPUs, GPUs, TPUs, accelerators) while handling device-specific optimizations and memory management behind the scenes.

hardware acceleration The use of specialized computing hardware to perform certain operations faster and more efficiently than software running on general-purpose processors.

hardware accelerator Specialized computing hardware designed to efficiently execute specific types of computations, such as GPUs for parallel processing or TPUs for machine learning workloads.

hardware-aware design The practice of designing neural network architectures specifically optimized for target hardware platforms, considering factors like memory hierarchy, compute units, and data movement patterns to maximize efficiency.

hidden layer An intermediate layer in a neural network between input and output layers that learns abstract representations by transforming data through learned weights and activation functions.

hidden state The internal memory of recurrent neural networks that carries information from previous time steps, enabling the network to maintain context across sequential inputs.

hierarchical processing A multi-tier system architecture where data and intelligence flow between different levels of the computing stack, from sensors to edge devices to cloud systems.

high-bandwidth memory (HBM) An advanced memory technology that provides much higher bandwidth than traditional DRAM by using 3D stacking and wide interfaces, critical for data-intensive AI workloads.

horizontal scaling Increasing system capacity by adding more machines or instances rather than upgrading existing hardware, providing better fault tolerance and load distribution.

hybrid machine learning The integration of multiple ML paradigms such as cloud, edge, mobile, and TinyML to form unified distributed systems that combine complementary strengths.

hybrid parallelism A distributed training approach that combines data parallelism and model parallelism to use the benefits of both strategies for training very large models.

hyperparameter A configuration setting that controls the learning process but is not learned from data, such as learning rate, batch size, or network architecture choices.

hyperscale data center Large-scale data center facilities containing thousands of servers and covering extensive floor space, designed to efficiently support massive computing workloads.

I

I/O bottleneck A performance limit where storage, decoding, preprocessing, or data transfer cannot supply data fast enough to keep compute resources busy.

idempotent transformation A data transformation that has the same effect when applied repeatedly as when applied once, supporting safe retries and reprocessing.

im2col A convolution-lowering technique that unfolds image patches into matrix columns so convolution can execute as GEMM.

ImageNet A massive visual database containing over 14 million labeled images across 20,000+ categories, developed by a team led by Fei-Fei Li and introduced in 2009, whose annual challenge became instrumental in driving breakthrough advances in computer vision.

imperative programming A programming paradigm in which programs specify sequences of commands that update state and direct control flow; it is distinct from whether operations execute eagerly or are staged for later execution.

inductive bias A structural assumption built into a model or architecture that restricts the functions it can learn efficiently, such as locality in CNNs.

inference The operational phase where trained neural networks make predictions on new data using fixed parameters, without weight updates.

InfiniBand A high-throughput, low-latency networking technology commonly used for

multi-node accelerator clusters and distributed training.

information-compute ratio (ICR) A metric quantifying the efficiency of data selection, defined as the ratio of model performance gain to the computational cost (FLOPs) required to achieve it. Maximizing ICR is the primary goal of data selection strategies.

infrastructure as code Practice of managing and provisioning computing infrastructure through machine-readable configuration files rather than manual processes, enabling version control and automation.

INT8 An 8-bit signed-integer representation used to quantize weights, activations, or both. Each INT8 value occupies one quarter the storage of an FP32 value, and supported hardware can accelerate applicable operations, with accuracy depending on the quantization scheme.

intermediate representation (IR) A compiler-internal representation between frontend model code and backend hardware code generation, used by ML frameworks for optimization and portability.

internet of things A network of physical objects embedded with sensors, software, and other technologies that connect and exchange data with other devices and systems over the internet.

intersectional analysis Evaluation that considers combinations of demographic attributes (for example, race and gender simultaneously) to detect concentrated harms not visible in single-factor analysis.

IOPS Input/Output Operations Per Second; a storage performance metric measuring the number of read/write operations a device can handle per second, critical for random access workloads like training data loading.

Iron Law of ML Systems A quantitative framework decomposing ML system performance into three terms: Data (limited by bandwidth), Compute (limited by FLOP/s), and Latency (limited by overhead). Formulated as $T = D_{vol}/BW + O/(R_{peak} \cdot \eta_{hw}) + L_{lat}$.

iterative pruning A gradual pruning strategy that removes parameters in multiple stages with fine-tuning between each stage, allowing the model to adapt to reduced capacity and typically achieving better accuracy than one-shot pruning.

J

JAX A numerical computing library developed by Google Research that combines NumPy's API with functional programming transformations including automatic differentiation, just-in-time compilation, and automatic vectorization for high-performance machine learning research.

JIT compilation Just-In-Time compilation that analyzes and optimizes code at runtime, enabling frameworks to balance the flexibility of eager execution with the performance benefits of graph optimization by compiling frequently used functions.

K

k-anonymity A privacy technique that ensures each record in a dataset is indistinguishable from at least k-1 other records by generalizing quasi-identifiers.

kernel A small matrix of learnable weights used in convolutional layers to detect specific features through the convolution operation, also called a filter. In GPU programming,

the term also denotes a function launched for parallel execution by a grid of threads.

kernel fusion An optimization technique that combines multiple computational operations into a single kernel to reduce memory transfers and improve performance on parallel processors.

keyword spotting (KWS) A technology that detects specific wake words or phrases in audio streams, typically used in voice-activated devices with constraints on power consumption and latency.

knowledge distillation A model compression technique where a smaller “student” network learns to mimic the behavior of a larger “teacher” network by training on the teacher’s soft output probabilities rather than just hard labels.

Kolmogorov-Smirnov test (K-S test) A non-parametric test that measures the maximum distance between two cumulative distributions, often used to compare feature distributions for drift.

Kullback-Leibler divergence (KL divergence) An asymmetric information-theoretic measure of how one probability distribution differs from another, used in monitoring and drift analysis.

KV cache Stored key and value tensors from previous transformer tokens used during autoregressive inference to avoid recomputing attention history.

L

L0 norm The count of nonzero elements in a vector or tensor, used in pruning to express sparsity or a target budget for remaining weights.

label quality drift Degradation in the reliability or consistency of labels over time, even when the underlying input distribution remains stable.

label shift A distribution shift where label frequencies change while the feature patterns conditioned on labels remain stable.

LAPACK Linear Algebra Package that extends BLAS with higher-level linear algebra operations including matrix decompositions, eigenvalue problems, and linear system solutions, providing essential mathematical foundations for machine learning computations.

latency The time delay between a request or event and the corresponding response or completion, critical in real-time applications where prompt responses are required.

latency constraints Real-time requirements that limit the maximum acceptable delay for model inference, driving optimization decisions in deployment scenarios where response time is critical.

layer normalization A normalization technique that normalizes inputs across the features dimension for each sample, commonly used in transformer architectures to stabilize training.

layerwise quantization A quantization granularity where all parameters within a single layer share the same quantization parameters, providing computational efficiency but potentially limiting representational precision compared to finer-grained approaches.

learning rate A hyperparameter that determines the step size for weight updates during gradient descent optimization, critically affecting training stability and convergence speed.

learning rate scheduling The systematic adjustment of learning rates during training, using strategies like step decay, exponential decay,

or cosine annealing to improve convergence and final model performance.

lifecycle coherence The principle that all stages of ML development should align with overall system objectives, maintaining consistency in data handling, model architecture, and evaluation criteria.

linear scaling failure The phenomenon where increasing computing resources (for example, doubling accelerators) results in sub-linear performance gains due to communication overhead and synchronization costs.

LINPACK Benchmark developed at Argonne National Laboratory that measures system performance by solving dense systems of linear equations, famous for its use in TOP500 supercomputer rankings.

Little’s Law A queuing relationship stating that average concurrency equals arrival rate times average time in the system, linking throughput, latency, and capacity.

Llama Large Language Model Meta AI, a family of open foundation models that popularized efficient architectural choices like RMSNorm and SwiGLU, serving as a modern reference point for large-scale transformer efficiency.

load balancing Distribution of incoming requests across multiple server instances to prevent bottlenecks, improve response times, and ensure high availability.

locality-sensitive hashing (LSH) A hashing family that maps similar items to the same buckets with high probability, enabling scalable approximate similarity search.

logits The raw, unnormalized scores output by the last layer of a neural network before the activation function (like softmax) converts them into probabilities.

loss function A mathematical function that measures how well model outputs satisfy a training objective, providing the objective minimized by the training algorithm.

loss scaling A technique used in mixed-precision training that multiplies the loss by a large factor before backpropagation to prevent gradient underflow in reduced precision formats.

Lottery Ticket Hypothesis The theory that large neural networks contain sparse subnetworks that, when trained in isolation from proper initialization, can achieve comparable accuracy to the full network while being significantly smaller.

low-rank factorization A matrix decomposition technique that approximates large weight matrices as products of smaller matrices, reducing the number of parameters and computational operations required for neural network layers.

LSTM Long Short-Term Memory, a type of recurrent neural network architecture designed to handle long-term dependencies through gating mechanisms that control information flow.

M

machine learning A subset of artificial intelligence that enables systems to automatically improve performance on tasks through experience and data rather than explicit programming.

machine learning framework A software platform that provides tools and abstractions for designing, training, and deploying machine learning models, bridging user applications with infrastructure through computational graphs, hardware optimization, and workflow orchestration. Examples include TensorFlow, PyTorch, and JAX.

machine learning lifecycle A structured, iterative process that encompasses all stages involved in developing, deploying, and maintaining machine learning systems, from problem definition through ongoing monitoring and improvement.

machine learning operations (MLOps) The engineering discipline and set of tools focused on operationalizing machine learning models through automation, monitoring, and management of the entire ML pipeline from development to production.

machine learning systems engineering The engineering discipline focused on building reliable, efficient, and scalable AI systems across computational platforms, spanning the entire AI lifecycle from data acquisition through deployment and operations with emphasis on resource-awareness and system-level optimization.

macro benchmarks Evaluation methodology that assesses complete machine learning models to understand how architectural choices and component interactions affect overall system behavior and performance.

magnitude-based pruning The most common pruning method that removes parameters with the smallest absolute values, based on the assumption that weights with smaller magnitudes contribute less to the model’s output.

masking An anonymization technique that alters or obfuscates sensitive values so they cannot be directly traced back to the original data subject.

membership inference attack A privacy attack that tries to determine whether a specific record was included in a model’s training data.

memory bandwidth Rate at which data can be read from or written to memory, measured in bytes per second, which often becomes a bottleneck in memory-intensive machine learning workloads.

memory hierarchy The organization of memory systems with different access speeds and capacities, from fast on-chip caches to slower off-chip main memory.

memory wall The widening performance gap between processor speed and memory bandwidth, where computational capacity outpaces the rate at which data can be delivered to the processor, becoming a primary bottleneck for large ML models.

metadata Descriptive information about datasets that includes details about data collection, quality metrics, validation status, and other contextual information essential for data management.

micro-batch A smaller slice of an effective batch processed sequentially for gradient accumulation or pipeline parallelism to reduce peak memory.

micro benchmarks Specialized evaluation tools that assess individual components or specific operations within machine learning systems, such as tensor operations or neural network layers.

microcontroller A small computer on a single integrated circuit containing a processor core, memory, and programmable input/output peripherals, commonly used in embedded systems.

MinHash A sketching method that approximates set similarity with compact signatures, commonly used for scalable near-duplicate detection.

mini-batch gradient descent A training approach that computes gradients and updates weights using a small subset of training examples simultaneously, balancing compu-

- tational efficiency with gradient estimation quality.
- mini-batch processing** An optimization approach that computes gradients over small batches of examples, balancing the computational efficiency of batch processing with the memory constraints of stochastic methods.
- mixed-precision computing** A technique that uses different numerical precisions at various stages of computation, such as FP16 for matrix multiplications and FP32 for accumulations.
- mixed-precision training** A training methodology that combines different numerical precisions (typically FP16 and FP32) to optimize memory usage and computational speed while maintaining training stability.
- ML node** A single production ML application treated as an operational unit, including data pipelines, feature computation, model training, serving infrastructure, and monitoring.
- ML systems** Integrated computing systems comprising three core components: data that guides algorithmic behavior, learning algorithms that extract patterns from data, and computing infrastructure that enables both training and inference processes.
- ML systems spectrum** The range of machine learning system deployments from cloud-based systems with abundant resources to tiny embedded devices with severe constraints, each requiring different optimization strategies and trade-offs.
- ML Test Score** A production-readiness rubric for ML systems that evaluates data, model, infrastructure, and monitoring tests.
- MLCommons** Organization that develops and maintains industry-standard benchmarks for machine learning systems, including the MLPerf suite for training and inference evaluation.
- MLPerf** Industry-standard benchmark suite that provides standardized tests for training and inference across various deep learning workloads, enabling fair comparisons of machine learning systems.
- MLPerf execution scenarios** Standard MLPerf inference traffic scenarios that model sequential, synchronized, server, batch, and interactive workloads with different latency and throughput requirements.
- MLPerf Inference** Benchmark framework that evaluates machine learning inference performance across different deployment environments, from cloud data centers to mobile devices and embedded systems.
- MLPerf Mobile** Specialized benchmark that extends MLPerf evaluation to smartphones and mobile devices, measuring latency and responsiveness under strict power and memory constraints.
- MLPerf Power** An MLPerf methodology for measuring power and energy efficiency of ML workloads alongside performance.
- MLPerf Tiny** Benchmark designed for embedded and ultra-low-power AI systems such as IoT devices, wearables, and microcontrollers operating with minimal processing capabilities.
- MLPerf Training** Standardized benchmark that evaluates machine learning training performance by measuring time-to-accuracy, throughput, and resource utilization across different hardware platforms.
- mobile machine learning** The execution of machine learning models directly on portable, battery-powered devices like smartphones and tablets, enabling personalized and responsive applications.
- model cards** A standardized format for documenting machine learning models, capturing information essential for responsible deployment, including intended use, performance factors, and ethical considerations.
- model-centric AI** A research paradigm where the dataset is treated as fixed and engineering effort focuses on optimizing model architecture.
- model compression** Techniques used to reduce the size and computational requirements of machine learning models while seeking to preserve accuracy, enabling deployment on resource-constrained devices.
- model deployment** The process of integrating trained machine learning models into production systems where they can make predictions on new data and provide value to end users.
- model drift** The degradation of machine learning model performance over time due to changes in data patterns, user behavior, or environmental conditions that differ from the original training conditions.
- model evaluation** The systematic assessment of machine learning model performance using various metrics and validation techniques to determine whether the model meets requirements and is ready for deployment.
- model FLOPs utilization (MFU)** The ratio of the model FLOPs per second implied by observed throughput to the hardware's theoretical peak FLOPs per second, measuring how effectively a workload uses available compute for model computation.
- model optimization** The systematic refinement of machine learning models to enhance their efficiency while maintaining effectiveness, balancing trade-offs between accuracy, computational cost, memory usage, latency, and energy efficiency.
- model parallelism** A distributed training strategy that splits a neural network model across multiple devices, with each device responsible for computing a portion of the network.
- model quantization** The process of reducing numerical precision in machine learning models, typically from FP32 to INT8, to decrease model size and memory traffic; speed increases only with efficient low-precision kernels.
- model registry** Centralized repository for storing, versioning, and managing trained machine learning models with associated metadata, facilitating model governance and deployment.
- model serving** Infrastructure and systems that expose deployed machine learning models through APIs to handle prediction requests at scale with appropriate latency and throughput.
- model training** The process of using machine learning algorithms to learn patterns from training data, adjusting model parameters to minimize prediction errors and create a functional predictive system.
- model validation** The process of testing machine learning models on independent datasets to assess their generalization ability and ensure they perform reliably on unseen data.
- model versioning** The systematic tracking and management of different versions of machine learning models, including their parameters, training data, and performance metrics, to enable comparison and rollback capabilities.
- momentum** An optimization technique that accumulates a velocity vector across iterations to help gradient descent navigate through local minima and accelerate convergence in consistent gradient directions.
- monitoring** The continuous observation and measurement of machine learning system performance, data quality, and operational metrics in production to detect issues and trigger maintenance actions.
- multi-head attention** An attention mechanism that uses multiple parallel attention heads, each focusing on different aspects of the input to capture diverse types of relationships simultaneously.
- multilayer perceptron (MLP)** A feedforward neural network with one or more hidden layers between input and output layers, capable of learning nonlinear mappings through dense connections and activation functions.
- multiply-accumulate (MAC)** The operation of multiplying two values and accumulating the result into a running sum, dominating linear layers and accelerator datapaths.
- MYCIN** One of the first large-scale expert systems developed at Stanford beginning in 1972 to diagnose blood infections, representing the shift toward capturing human expert knowledge in specific domains rather than pursuing general artificial intelligence.

N

- N:M sparsity** A structured sparsity pattern where exactly N of every M values are nonzero, giving hardware more regular zero patterns to exploit.
- neural architecture search** An automated approach that searches possible combinations of layers, connections, and architectural hyperparameters to find neural network architectures that perform well under specified objectives and constraints.
- neural network** A computational model consisting of interconnected nodes organized in layers that can learn to map inputs to outputs through adjustable connection weights.
- neural processing unit (NPU)** Specialized processors designed specifically for accelerating neural network operations and machine learning computations, optimized for parallel processing of AI workloads.
- neuromorphic computing** A computing approach that mimics the structure and function of biological neural networks, potentially offering more energy-efficient processing for AI applications.
- NoSQL** A category of database systems designed to handle large volumes of unstructured or semi-structured data with flexible schemas, often used in big data applications.
- numerical precision optimization** The dimension of model optimization that addresses how numerical values are represented and processed, including quantization techniques that map high-precision values to lower-bit representations.
- NVLink** NVIDIA's high-bandwidth GPU interconnect for intra-node communication, useful for gradient synchronization and model-parallel activation transfers.
- observability** Comprehensive monitoring approach that provides insight into system behavior through metrics, logs, and traces, enabling understanding of internal states from external outputs.
- on-chip memory** Fast memory integrated directly onto the processor chip, including caches and scratchpad memory, providing high bandwidth and low latency data access.
- on-device learning** The capability for machine learning models to adapt and learn directly on edge devices without requiring data transmission to external servers.

one-hot encoding A representation where categorical labels become vectors with a single 1 and remaining 0s.

one-shot pruning A pruning strategy where a large fraction of parameters is removed in a single step, typically followed by fine-tuning to recover accuracy, offering simplicity but potentially requiring more aggressive fine-tuning.

online analytical processing (OLAP) A database approach optimized for complex analytical queries across large datasets, typically used in data warehouses for business intelligence.

online inference Real-time prediction serving that processes individual requests with low latency, suitable for interactive applications requiring immediate responses.

online transaction processing (OLTP) A database approach optimized for frequent, short transactions and real-time processing, commonly used in operational applications.

ONNX Open Neural Network Exchange, a standardized format for representing machine learning models that enables interoperability between different frameworks, allowing models trained in one framework to be deployed using another.

ONNX Runtime Cross-platform inference engine that optimizes machine learning models through techniques like operator fusion and kernel tuning to improve inference speed and reduce computational overhead.

optimizer An algorithm that adjusts model parameters during training to minimize the loss function, with common examples including SGD (Stochastic Gradient Descent), Adam, and RMSprop, each with different strategies for parameter updates.

optimizer state Auxiliary values maintained by an optimizer in addition to model parameters, such as Adam's momentum and variance tensors, often dominating training memory for large models.

orchestration Coordination and management of complex workflows and distributed computing tasks, often using platforms like Kubernetes for container management.

outlier detection The process of identifying data points that significantly deviate from normal patterns, which may represent errors, anomalies, or valuable rare events.

overfitting A phenomenon where a model learns specific details of training data so well that it fails to generalize to new, unseen examples, typically indicated by high training accuracy but poor validation performance.

P

padding A technique in convolutional networks that adds zeros or other values around the input borders to control the spatial dimensions of the output feature maps.

PagedAttention A KV-cache memory-management technique that stores attention context in fixed-size pages rather than contiguous per-request blocks to reduce fragmentation.

paradigm shift A fundamental change in scientific approach, like the shift from symbolic reasoning to statistical learning in AI during the 1990s, and from shallow to deep learning in the 2010s, requiring researchers to abandon established methods for radically different approaches.

parallelism The simultaneous execution of multiple computational tasks or operations, fundamental to achieving high performance in neural network processing.

parameter A learnable component of a neural network, including weights and biases, that gets adjusted during training to minimize the loss function.

Pareto frontier The set of configurations where improving one objective requires worsening another, used to reason about accuracy, latency, memory, and energy trade-offs.

partitioning A database technique that divides large datasets into smaller, manageable segments based on specific criteria to improve query performance and system scalability.

perceptron The fundamental building block of neural networks, consisting of weighted inputs, a bias term, and an activation function that produces a single output.

performance insights Analytical observations derived from monitoring production machine learning systems that reveal opportunities for improvement in model accuracy, system efficiency, or user experience.

performance-per-data (PPD) A metric measuring the accuracy gain per training sample, used to evaluate the quality of a dataset or selection strategy. High PPD indicates a dataset rich in information and low in redundancy.

pinned memory Page-locked host memory that enables direct, asynchronous transfers between CPU and GPU, improving data-loading throughput at the cost of reserving system memory.

pipeline jungle Anti-pattern where complex, interdependent data processing pipelines become difficult to maintain, debug, and modify, leading to technical debt and operational complexity.

pipeline parallelism A form of model parallelism where different layers of a model are placed on different devices and data flows through them in a pipeline fashion, allowing multiple batches to be processed simultaneously.

point-in-time correctness The guarantee that training examples use only feature values that would have been available at the historical prediction time, preventing leakage.

pooling A downsampling operation in convolutional networks that reduces spatial dimensions while retaining important features, commonly using max or average operations over local regions.

population stability index (PSI) A binned distribution-shift metric that compares current feature distributions against a baseline to detect population drift.

positional encoding A method used in transformer architectures to inject information about the position of tokens in a sequence, since transformers lack inherent sequential processing.

post-training quantization A quantization approach applied to already-trained models without modifying the training process, typically involving calibration on representative data to determine optimal quantization parameters.

power usage effectiveness (PUE) Metric used in data centers to measure energy efficiency, calculated as the ratio of total facility power consumption to IT equipment power consumption.

power wall The technological barrier reached around 2005 where increasing processor frequency no longer yielded performance gains without unsustainable increases in power density and heat generation, forcing a shift to parallel and specialized architectures.

precision In numerical computing, the number of significant bits or digits used to represent values, affecting computational accu-

racy and resource requirements in machine learning systems.

prefetching A system optimization technique that loads data into memory before it is needed, overlapping data loading with computation to reduce idle time and improve training throughput.

problem definition The initial stage of machine learning development that involves clearly specifying objectives, constraints, success metrics, and operational requirements to guide all subsequent development decisions.

processing element (PE) A repeated accelerator building block containing compute units and local storage, with arrays of PEs providing parallel tensor execution.

programmable logic controller (PLC) Industrial control systems used in manufacturing and IoT environments that can be integrated with ML models for automated decision-making in operational technology contexts.

protein folding problem The scientific challenge of predicting the three-dimensional structure and dynamics of proteins from their amino acid sequences and environments, a longstanding problem on which systems such as AlphaFold achieved breakthrough accuracy without resolving every folding question.

proxy variable A feature that indirectly carries signal about a protected or sensitive attribute even when that attribute is removed.

pseudonymization A privacy technique that replaces direct identifiers with artificial identifiers while maintaining the ability to trace records for analysis purposes.

PyTorch A deep learning framework developed by Facebook's AI Research lab that emphasizes dynamic computational graphs, eager execution, and intuitive Python integration, particularly popular for research and experimentation.

Q

quantization A model-compression technique that represents parameters or activations at lower precision, such as FP32 to INT8, reducing storage and data movement without reducing mathematical operation count; speedups require supported hardware and kernels.

quantization-aware training A training approach where quantization effects are simulated during the training process, allowing the model to adapt to reduced precision and typically achieving better accuracy than post-training quantization.

quantization granularity The level at which quantization parameters are applied, ranging from per-tensor (coarsest) to per-channel or per-group (finer), with finer granularity typically preserving more accuracy but requiring more storage.

queries per second (QPS) Performance metric that measures how many inference requests a system can process in one second, commonly used to evaluate throughput in production deployments.

query-key-value The three components of attention mechanisms where queries determine what to look for, keys represent what is available, and values contain the actual information to be weighted and combined.

R

real-time processing Processing data as it becomes available under application timing

- constraints; hard real-time systems guarantee worst-case deadlines, while soft real-time systems tolerate occasional misses.
- receptive field** The region of the input that influences a particular neuron's output, determining the spatial extent of patterns that can be detected by that neuron.
- recurrent neural network** A type of neural network designed for sequential data processing, featuring connections that create loops allowing information to persist across time steps.
- Red AI** AI research and development that prioritizes maximizing accuracy or performance without regard for the increasing computational and environmental costs required.
- regularization** Techniques used to prevent overfitting in neural networks by adding constraints or penalties, including methods like dropout, weight decay, and data augmentation.
- ReLU** Rectified Linear Unit activation function defined as $f(x) = \max(0, x)$ that introduces nonlinearity while maintaining computational efficiency and helping mitigate vanishing gradients for positive inputs.
- residual connection** A skip connection that adds the input of a layer to its output, enabling the training of very deep networks by mitigating the vanishing gradient problem.
- ResNet** Residual Network, a deep convolutional architecture built from residual blocks with skip connections, enabling the training of networks with hundreds of layers and achieving breakthrough performance.
- ResNet-50** A specific 50-layer variant of the Residual Network architecture that serves as the canonical lighthouse model for compute-bound workloads, balancing depth and computational cost for vision benchmarks.
- retinal fundus photographs** Medical images of the interior surface of the eye, including the retina, optic disc, and blood vessels, commonly used for diagnosing eye diseases and training medical AI systems.
- reverse-mode differentiation** An automatic differentiation technique that computes gradients by traversing the computational graph in reverse order, highly efficient for functions with many inputs and few outputs, making it ideal for neural network training.
- ridge point** The arithmetic intensity at which a workload transitions from memory-bound to compute bound on a given hardware platform, calculated as peak FLOP/s divided by memory bandwidth. The inflection point on the Roofline Model diagram.
- Ring AllReduce** A bandwidth-efficient AllReduce algorithm where devices pass chunks around a ring and accumulate results for distributed gradient synchronization.
- RMSprop** An adaptive learning rate optimization algorithm that maintains a moving average of squared gradients to automatically adjust learning rates for each parameter during training.
- role-based access control (RBAC)** An access-control model that grants permissions according to user, service, or team roles rather than ad hoc individual grants.
- rollback** Process of reverting to a previous stable version of a model or system when issues are detected in production, ensuring service continuity.
- roofline analysis** A performance modeling technique that plots attainable performance against operational intensity, with peak compute and bandwidth forming the ceilings, to identify whether a system is memory bound or compute bound.
- scalability** The ability of machine learning systems to handle increasing amounts of data, users, or computational demands without significant degradation in performance or user experience.
- scale and zero-point** Quantization parameters that map real-valued tensors to integer values and back, where scale controls step size and zero-point maps real zero into the quantized range.
- schema** The structure and format definition of data that specifies data types, field names, and relationships, essential for data validation and processing consistency.
- schema evolution** The process of modifying data schemas over time while maintaining backward compatibility and ensuring continued functionality of dependent systems and applications.
- schema-on-read** An approach used in data lakes where data structure is defined and enforced at the time of reading rather than when storing, providing flexibility for diverse data types.
- scratchpad memory** Fast software-managed local memory used by accelerators to stage predictable data movement under compiler or runtime control.
- segmentation maps** Detailed annotations that classify objects at the pixel level, providing the most granular labeling information but requiring significantly more storage and processing resources.
- self-attention** An attention mechanism where queries, keys, and values all come from the same sequence, allowing each position to attend to all positions including itself.
- semi-supervised learning** A machine learning approach that uses both labeled and unlabeled data for training, using structural assumptions to improve model performance with limited labels.
- sensitivity** The proportion of actual positive cases correctly identified by a classification model, also known as true positive rate or recall; critical in medical applications where missing positive cases has severe consequences.
- serverless** Cloud computing model where infrastructure is automatically managed by the provider, allowing code execution without server management concerns.
- service level agreement (SLA)** A formal external contract specifying service commitments, often through one or more SLOs, and the consequences or penalties for noncompliance.
- service-level objective (SLO)** An internal measurable target for a service-level indicator such as latency, error rate, or availability that guides operations and may underpin an external SLA.
- shadow deployment** A deployment strategy where a new model runs in parallel with the production model, making predictions that are logged but not served to users, enabling validation without user impact.
- shallow learning** Machine learning approaches that use algorithms with limited complexity, such as support vector machines and decision trees, which require carefully engineered features but cannot automatically discover hierarchical representations like deep learning methods.
- SHAP values** Feature-attribution scores based on Shapley values that estimate each feature's contribution to a model prediction.
- sigmoid** An activation function that maps input values to a range between 0 and 1, historically popular but prone to vanishing gradient problems in deep networks.
- silent bias** Model unfairness that produces valid-looking but discriminatory outputs, evading traditional error monitoring and requiring disaggregated evaluation to detect.
- silent degradation** The gradual decline in ML system performance that occurs without triggering errors, exceptions, or alerts. An ML system can continue operating while producing increasingly inaccurate predictions.
- silent failure** A system failure mode where an ML model continues to produce plausible-looking outputs that are gradually less accurate or contextually relevant without triggering conventional error alerts.
- SIMD (Single Instruction, Multiple Data)** A parallel computing architecture that applies the same operation to multiple data elements simultaneously, effective for regular data-parallel computations.
- SIMT (Single Instruction, Multiple Threads)** An extension of SIMD that enables parallel execution across multiple independent threads, each maintaining its own state and program counter.
- singular value decomposition** A matrix factorization technique that decomposes a matrix into the product of three matrices, commonly used in low-rank approximations to compress neural network layers by retaining only the most significant singular values.
- skip connection** A direct connection that bypasses one or more layers, allowing gradients to flow more easily through deep networks and enabling better training of very deep architectures.
- soft targets** Teacher-model probability distributions used as supervision in knowledge distillation, carrying class-similarity information beyond hard labels.
- softmax** An activation function that converts logits into a probability distribution where outputs sum to 1, used in multi-class classification.
- sparse Tensor Core** A Tensor Core variant that accelerates supported structured sparsity patterns such as 2:4 sparsity by skipping constrained zero values.
- sparsity** The property of tensors in which many values are exactly zero, which can improve computational efficiency when the zeros have suitable structure and specialized sparse kernels or hardware support them; near-zero values count only after pruning.
- SPEC CPU** Standardized benchmark suite developed by the System Performance Evaluation Cooperative that measures processor performance using real-world applications rather than synthetic tests.
- special function unit (SFU)** Dedicated hardware for specialized operations such as exponentials, reciprocals, roots, and some activation-function primitives that do not map cleanly to ordinary arithmetic or matrix multiply units.
- specificity** The proportion of actual negative cases correctly identified by a classification model, also known as true negative rate; important for avoiding overwhelming referral systems with false positives.
- SPECpower** Benchmark methodology that measures server energy efficiency across varying workload levels, enabling direct comparisons of power-performance trade-offs in computing systems.
- speculative decoding** An optimization technique for autoregressive language models where a smaller model generates draft tokens that are then verified by a larger model,

accelerating inference while maintaining quality.

speed of light (latency) The physical speed limit of information transmission (approx. 200,000 km/s in fiber), creating an irreducible lower bound on network latency that necessitates edge computing for applications requiring sub-10 ms response times over long distances.

state dict A framework serialization mapping from module or optimizer names to tensors, used to save, load, and checkpoint models and optimizer state.

static graph A computational graph that is defined completely before execution begins, enabling comprehensive optimization and efficient deployment but requiring all operations to be specified upfront, limiting runtime flexibility.

static quantization A quantization approach where quantization parameters are determined once during calibration and remain fixed during inference, providing computational efficiency but less adaptability than dynamic approaches.

statistical learning The era of machine learning that emerged in the 1990s, shifting focus from rule-based symbolic AI to algorithms that could learn patterns from data, laying the groundwork for modern data-driven approaches to artificial intelligence.

stochastic gradient descent A variant of gradient descent that estimates gradients using individual training examples or small batches rather than the entire dataset, reducing memory requirements and enabling online learning.

Straight-Through Estimator (STE) A gradient approximation that lets training pass gradients through nondifferentiable operations such as rounding, often used in quantization-aware training.

stream ingestion A data processing pattern that handles data in real-time as it arrives, essential for applications requiring immediate processing and low-latency responses.

stream processing Real-time data processing approach that handles continuous flows of data as it arrives, enabling immediate responses to events and pattern detection.

stride The step size by which a convolutional filter moves across the input, controlling the spatial dimensions of the output and the degree of overlap between filter applications.

structured pruning A pruning approach that removes entire computational units such as neurons, channels, or layers, producing smaller dense models that are more hardware-friendly than the sparse matrices created by unstructured pruning.

supervised learning A machine learning approach where models learn from labeled training examples to make predictions on new, unlabeled data.

symbolic AI An approach to artificial intelligence that uses high-level symbolic representations of problems, logic, and search algorithms, dominant before the deep learning revolution and characterized by expert systems and rule-based reasoning.

synthetic benchmark Artificial test program designed to measure specific aspects of system performance, as opposed to benchmarks based on real-world applications and workloads.

synthetic data Artificially generated data created using algorithms, simulations, or generative models to supplement real-world datasets, addressing limitations in data availability or privacy concerns.

system entropy The tendency of ML systems to degrade over time as the world changes

(drift) or as hidden dependencies accumulate (technical debt), requiring active energy (ops) to maintain order.

System-on-Chip (SoC) An integrated circuit that incorporates most or all components of a computer or electronic system, including CPU, GPU, memory, and specialized processors on a single chip. Commonly used in mobile devices and embedded systems for space and power efficiency.

systems co-design Joint design of algorithms, software, and hardware so model structure and execution fit physical capabilities and constraints.

systems integration The process of combining various components and subsystems into a unified, functional system that operates efficiently and reliably as a whole.

systems thinking An approach to understanding complex systems by considering how individual components interact and affect the whole system, particularly important in ML where data, algorithms, hardware, and deployment environments must work together effectively.

systolic array A specialized hardware architecture that efficiently performs matrix operations by streaming data through a grid of processing elements, minimizing data movement and energy consumption.

T

tail latency High-percentile response time in a system, typically measured as P95 or P99 latency, important for characterizing slow requests under peak load but not equivalent to the absolute worst case. It complements median and mean latency rather than replacing them.

tailored inference benchmarks Specialized performance tests designed for specific deployment environments or use cases, accounting for unique constraints and optimization requirements.

tanh Hyperbolic tangent activation function that maps inputs to (-1, 1), providing zero-centered outputs.

technical debt Long-term maintenance cost accumulated from expedient design decisions during development, particularly problematic in ML systems due to data dependencies and model complexity.

telemetry Automated collection and transmission of performance data and metrics from distributed systems, enabling remote monitoring and analysis.

tensor A multi-dimensional array used to represent data in neural networks, generalizing scalars (0D), vectors (1D), and matrices (2D) to higher dimensions.

Tensor Core A specialized GPU matrix unit for accelerated mixed-precision multiply-accumulate operations on tensor tiles.

tensor decomposition The extension of matrix factorization to higher-order tensors, used to compress neural network layers by representing weight tensors as combinations of smaller tensors with fewer parameters.

tensor parallelism A model parallelism strategy where individual tensor operations (like matrix multiplication) are split across multiple devices, reducing memory per device and potentially reducing large-layer latency when the reduction in computation time exceeds the added communication time.

tensor processing unit (TPU) Google's custom application-specific integrated circuit designed specifically for machine learning workloads, optimized for matrix operations and featuring systolic array architecture.

TensorFlow A comprehensive machine learning framework developed by Google that provides tools for the entire ML pipeline from research to production, featuring both eager execution and graph-based computation with extensive ecosystem support.

TensorRT NVIDIA's inference optimization library that applies techniques like operator fusion and precision reduction to accelerate deep learning inference on GPU hardware.

ternarization An extreme quantization technique that constrains weights to three values (typically -1, 0, +1), providing significant compression while maintaining more representational capacity than binary quantization.

thermal design power (TDP) The sustained power and heat level a device or cooling system is designed to dissipate under typical maximum workload.

thermal throttling A protective mechanism in mobile and embedded devices that reduces processor clock speed and performance to prevent overheating, often limiting the sustained performance of on-device ML inference.

throughput The rate at which a system can process data or complete operations, typically measured in operations per second and crucial for training large models.

time per output token (TPOT) The per-token latency during autoregressive decode after the first token, reflecting decode throughput and memory-bandwidth limits.

time-to-accuracy The wall-clock time required to train a model to a specified validation accuracy.

time to first token (TTFT) The time from request arrival to the first generated token in streaming LLM serving, including prefill, scheduling, and initial response latency.

TinyML A field focused on deploying machine learning models on ultra-constrained embedded devices such as microcontrollers and sensors, operating in the milliwatt to sub-watt power range, with severe limitations on memory, power, and computational capacity. Also called *tiny machine learning*.

TOPS Tera Operations Per Second, an operation-rate unit indicating how many trillion operations a system can execute in one second. It is commonly reported for low-precision integer workloads, so the operation type and precision must be specified.

total cost of ownership (TCO) A comprehensive financial metric for ML systems encompassing training, inference, and operational costs over the system's entire lifecycle.

train-serve split A hybrid architecture pattern where computationally intensive model training occurs on powerful cloud infrastructure, while the trained model is optimized and deployed for inference on resource-constrained edge or mobile devices.

training The process of adjusting neural network parameters using data or interaction experience and an optimization procedure to minimize a loss or maximize a reward.

training-serving skew A mismatch between how features or data are computed during model training vs. serving in production, which can degrade model performance. Common causes include different preprocessing pipelines, feature computation timing, or data sources between training and inference.

transfer learning A machine learning technique that uses knowledge gained from pretrained models on related tasks, allowing faster training and better performance on new

tasks with limited data by reusing learned features and representations.

transformer A neural network architecture built from attention, feed-forward blocks, residual connections, and normalization; the original design eliminates recurrence and convolution, though later variants may incorporate them.

translation equivariance A property where shifting the input causes a corresponding shift in the output feature map, distinct from invariance which discards position.

translation invariance The property that a model's output remains unchanged when the input shifts. Convolution is translation-equivariant; pooling or global aggregation can produce approximate invariance.

Tucker decomposition A tensor decomposition method that generalizes singular value decomposition to higher-order tensors using a core tensor and factor matrices, commonly used for compressing convolutional neural network layers.

TV white spaces Unused broadcasting frequencies that can be repurposed for internet connectivity, as employed by systems like FarmBeats to extend network access to remote agricultural sensors and IoT devices.

U

uniform quantization A quantization approach where the range of values is divided into evenly spaced intervals, providing simple implementation but potentially suboptimal for nonuniform value distributions.

universal approximation theorem A theoretical result proving that neural networks with sufficient width and a suitable nonlinear activation function can approximate any continuous function on a compact domain to arbitrary accuracy.

unreasonable effectiveness of data The empirical observation that for many problems, adding more data is more effective than improving algorithms, driving the data-centric AI paradigm.

unstructured pruning A pruning approach that removes individual weights while preserving the overall network architecture, creating sparse weight matrices that require specialized hardware support to realize computational benefits.

unstructured sparsity A form of model sparsity where individual weights are set to zero without following any particular pattern, creating irregular sparsity patterns that require specialized hardware support to realize computational benefits.

V

validation issues Problems identified during model testing that indicate poor performance, overfitting, data quality problems, or other issues that must be resolved before deployment.

vanishing gradient problem A problem in deep neural networks where gradients become exponentially smaller as they propagate backward through layers, making it difficult for early layers to learn effectively.

vector operations Computational operations that process multiple data elements simultaneously, enabling efficient parallel execution of element-wise transformations in neural networks.

verification gap The mismatch between finite test-set coverage and the much larger input space an ML system may encounter in production.

versioning The practice of tracking changes to datasets, models, and pipelines over time, enabling reproducibility, rollback capabilities, and audit trails in ML systems.

virtuous cycle The self-reinforcing process in deep learning where improvements in data availability, algorithms, and computing power each enable further advances in the other areas, accelerating overall progress.

Von Neumann bottleneck The performance limitation caused by the shared bus between processor and memory in traditional computer architectures, where data movement becomes more expensive than computation.

W

warehouse-scale computer (WSC) A data center operated as a single programmable computer, treating compute, storage, networking, and failures as system-level resources.

warp A group of 32 threads on NVIDIA GPUs whose active threads execute one common instruction at a time; the fundamental unit of thread scheduling on NVIDIA GPUs.

Waymo A subsidiary of Alphabet Inc. that represents a leading deployment of machine learning systems in autonomous vehicle technology, demonstrating how ML systems can span from embedded systems to cloud infrastructure in safety-critical environments.

weak supervision An approach that uses lower-quality labels obtained more efficiently through heuristics, distant supervision, or programmatic methods rather than manual expert annotation.

web scraping An automated technique for extracting data from websites to build custom datasets, requiring careful consideration of legal, ethical, and technical constraints.

weight A learnable parameter that determines the strength of connection between neurons in different layers, adjusted during training to minimize the loss function.

weight matrix An organized collection of weights connecting one layer to another in a neural network, enabling efficient computation through matrix operations.

weight sharing The practice of using the same parameters across different spatial locations, as in convolutional networks, reducing the number of parameters while maintaining pattern detection capabilities.

Whetstone Early benchmark developed at the UK National Physical Laboratory by Curnow and Wichmann, first published as an ALGOL 60 program in 1972 and later as the canonical FORTRAN version in 1976. It measured floating-point arithmetic performance in MWIPS (millions of Whetstone instructions per second), becoming one of the first widely-adopted standardized performance tests.

workflow orchestration Automated coordination and management of complex ML pipeline sequences, ensuring proper execution order, dependency management, and error handling across distributed systems.

X

XLA Accelerated Linear Algebra, a domain-specific compiler for linear algebra operations that optimizes TensorFlow and JAX computations by generating efficient code for various hardware platforms including CPUs, GPUs, and TPUs.

References

- Abadi, Martin, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, et al. 2016. "TensorFlow: A System for Large-Scale Machine Learning." *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 265–83.
- Agarwal, Pankaj K., Sariel Har-Peled, and Kasturi R. Varadarajan. 2005. "Geometric Approximation via Coresets." In *Combinatorial and Computational Geometry*, edited by Jacob E. Goodman, János Pach, and Emo Welzl, vol. 52. MSRI Publications. Cambridge University Press. <https://doi.org/10.1017/9781009701259.002>.
- Agrawal, Amey, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. "Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve." *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 117–34.
- Ainslie, Joshua, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. 2023. "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints." *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 4895–901. <https://doi.org/10.18653/v1/2023.emnlp-main.298>.
- Akidau, Tyler, Robert Bradshaw, Craig Chambers, Slava Chernyak, Rafael J. Fernández-Moctezuma, Reuven Lax, Sam McVeety, et al. 2015. "The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing." *Proceedings of the VLDB Endowment* 8 (12): 1792–803. <https://doi.org/10.14778/2824032.2824076>.
- Altini, Marco, and Hannu Kinnunen. 2021. "The Promise of Sleep: A Multi-Sensor Approach for Accurate Sleep Stage Detection Using the Oura Ring." *Sensors* 21 (13): 4302. <https://doi.org/10.3390/s21134302>.
- Amazon Web Services. 2024a. *Amazon Mechanical Turk*.
- Amazon Web Services. 2024b. *Amazon Simple Storage Service (S3)*.
- Amazon Web Services. 2024c. *Amazon Web Services (AWS)*.
- Amazon Web Services. 2024d. *AWS Auto Scaling*.
- Amazon Web Services. 2024e. *AWS CloudFormation*.
- Amazon Web Services. 2026. *Amazon EC2 Spot Instances*.
- AMD. 2023. *AMD Instinct MI300X Accelerators*. AMD product documentation.
- Amdahl, Gene M. 1967. "Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities." *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference on - AFIPS '67 (Spring)*, AFIPS '67 (spring), 483–85. <https://doi.org/10.1145/1465482.1465560>.
- Amershi, Saleema, Andrew Begel, Christian Bird, Robert DeLine, Harald Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, and Thomas Zimmermann. 2019. "Software Engineering for Machine Learning: A Case Study." *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 291–300. <https://doi.org/10.1109/icse-seip.2019.00042>.
- Amodei, Dario, and Danny Hernandez. 2018a. "AI and Compute." *OpenAI Blog* 2.
- Amodei, Dario, and Danny Hernandez. 2018b. "AI and Compute." *OpenAI Blog* 6.
- Angwin, Julia, Jeff Larson, Surya Mattu, and Lauren Kirchner. 2016. *Machine Bias*. ProPublica investigation.
- Ansel, Jason, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, et al. 2024. "PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation." *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 929–47. <https://doi.org/10.1145/3620665.3640366>.
- Anthropic. 2023. *Introducing Claude 2.1*.

- Anthropic. 2024. *Introducing the Next Generation of Claude*.
- AnyLogic. 2024. *Synthetic Data for Artificial Intelligence*.
- Apache Software Foundation. 2024. *Apache Airflow*.
- Apple. 2024a. *Facial Matching Security*. Apple Platform Security.
- Apple. 2024b. *The Secure Enclave*. Apple Platform Security.
- Ardila, Rosana, Megan Branson, Kelly Davis, Michael Kohler, Josh Meyer, Michael Henretty, Reuben Morais, Lindsay Saunders, Francis Tyers, and Gregor Weber. 2020. "Common Voice: A Massively-Multilingual Speech Corpus." *Proceedings of the Twelfth Language Resources and Evaluation Conference*, 4218–22.
- Armbrust, Michael, Tathagata Das, Liwen Sun, Burak Yavuz, Shixiong Zhu, Mukul Murthy, Joseph Torres, et al. 2020. "Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores." *Proceedings of the VLDB Endowment* 13 (12): 3411–24. <https://doi.org/10.14778/3415478.3415560>.
- Authors, Kubeflow. 2024. *Kubeflow*.
- Ba, Jimmy Lei, Jamie Ryan Kiros, and Geoffrey E. Hinton. 2016. "Layer Normalization." *arXiv Preprint arXiv:1607.06450*.
- Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio. 2015. "Neural Machine Translation by Jointly Learning to Align and Translate." *International Conference on Learning Representations (ICLR)*.
- Bai, Zhaojun, James Demmel, Jack Dongarra, Julien Langou, and Jenny Wang. 2006. "LAPACK." In *Handbook of Linear Algebra*. Chapman; Hall/CRC. <https://doi.org/10.1201/9781420010572-75>.
- Baidu Research. 2016. *DeepBench: Benchmarking Deep Learning Operations on Different Hardware*.
- Bakheet, Samy, and Ayoub Al-Hamadi. 2021. "A Framework for Instantaneous Driver Drowsiness Detection Based on Improved HOG Features and Naïve Bayesian Classification." *Brain Sciences* 11 (2): 240. <https://doi.org/10.3390/brainsci11020240>.
- Banbury, Colby R., Vijay Janapa Reddi, Max Lam, William Fu, Amin Fazel, Jeremy Holleman, Xinyuan Huang, et al. 2020. "Benchmarking TinyML Systems: Challenges and Direction." *arXiv Preprint arXiv:2003.04821*.
- Banbury, Colby R, Chuteng Zhou, Igor Fedorov, Ramon Matas, Urmish Thakker, Dibakar Gope, Vijay Janapa Reddi, Matthew Mattina, and Paul N Whatmough. 2021. "MicroNets: Neural Network Architectures for Deploying TinyML Applications on Commodity Microcontrollers." *Proceedings of Machine Learning and Systems* 3: 517–32.
- Banbury, Colby, Vijay Janapa Reddi, Peter Torelli, Jeremy Holleman, Nat Jeffries, Csaba Kiraly, Pietro Montino, et al. 2021. "MLPerf Tiny Benchmark." *arXiv Preprint*.
- Barocas, Solon, and Andrew D. Selbst. 2016. "Big Data's Disparate Impact." *California Law Review* 104: 671–732. <https://doi.org/10.2139/ssrn.2477899>.
- Barroso, Luiz André, Urs Hölzle, and Parthasarathy Ranganathan. 2019. *The Datacenter as a Computer: Designing Warehouse-Scale Machines*. Synthesis Lectures on Computer Architecture. Springer International Publishing. <https://doi.org/10.1007/978-3-031-01761-2>.
- Barroso, Luiz, Mike Marty, David Patterson, and Parthasarathy Ranganathan. 2017. "Attack of the Killer Microseconds." *Communications of the ACM* 60 (4): 48–54. <https://doi.org/10.1145/3015146>.
- Baur, Walter, and Volker Strassen. 1983. "The Complexity of Partial Derivatives." *Theoretical Computer Science* 22 (3): 317–30. [https://doi.org/10.1016/0304-3975\(83\)90110-x](https://doi.org/10.1016/0304-3975(83)90110-x).
- Baydin, Atilim Gunes, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. 2018. "Automatic Differentiation in Machine Learning: A Survey." *Journal of Machine Learning Research* 18 (153): 1–43.
- Baylor, Denis, Eric Breck, Heng-Tze Cheng, Noah Fiedel, Chuan Yu Foo, Zakaria Haque, Salem Haykal, et al. 2017. "TFX: A TensorFlow-Based Production-Scale Machine Learning Platform." *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1387–95. <https://doi.org/10.1145/3097983.3098021>.
- Bedford Taylor, Michael. 2017. "The Evolution of Bitcoin Hardware." *Computer* 50 (9): 58–66. <https://doi.org/10.1109/mc.2017.3571056>.
- Beede, Emma, Elizabeth Baylor, Fred Hersch, Anna Iurchenko, Lauren Wilcox, Paisan Ruamviboonsuk, and Laura M. Vardoulakis. 2020. "A Human-Centered Evaluation of a Deep Learning System Deployed in Clinics for the Detection of Diabetic Retinopathy." *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI)*, 1–12. <https://doi.org/10.1145/3313831.3376718>.
- Belkin, Mikhail, Daniel Hsu, Siyuan Ma, and Soumik Mandal. 2019. "Reconciling Modern Machine-Learning Practice and the Classical Bias–Variance Trade-Off." *Proceedings of the National Academy of Sciences* 116 (32): 15849–54. <https://doi.org/10.1073/pnas.1903070116>.

- Bellamy, Rachel K. E., Kuntal Dey, Michael Hind, Seung Hoffman, Sylvain Houde, Karthikeyan Kannan, Pradeep Lohia, et al. 2019. "AI Fairness 360: An Extensible Toolkit for Detecting and Mitigating Algorithmic Bias." *IBM Journal of Research and Development* 63 (4/5): 4:1–15. <https://doi.org/10.1147/jrd.2019.2942287>.
- Bender, Emily M., and Batya Friedman. 2018. "Data Statements for Natural Language Processing: Toward Mitigating System Bias and Enabling Better Science." *Transactions of the Association for Computational Linguistics* 6: 587–604. https://doi.org/10.1162/tacl_a_00041.
- Bengio, Emmanuel, Pierre-Luc Bacon, Joelle Pineau, and Doina Precup. 2015. "Conditional Computation in Neural Networks for Faster Models." *arXiv Preprint arXiv:1511.06297*.
- Bengio, Yoshua, Aaron Courville, and Pascal Vincent. 2013. "Representation Learning: A Review and New Perspectives." *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35 (8): 1798–828. <https://doi.org/10.1109/tpami.2013.50>.
- Bengio, Yoshua, Nicholas Léonard, and Aaron Courville. 2013. "Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation." *arXiv Preprint arXiv:1308.3432*.
- Bengio, Yoshua, Jérôme Louradour, Ronan Collobert, and Jason Weston. 2009. "Curriculum Learning." *Proceedings of the 26th Annual International Conference on Machine Learning*, 41–48. <https://doi.org/10.1145/1553374.1553380>.
- Berger, Vance W., and YanYan Zhou. 2014. "Kolmogorov–Smirnov Test: Overview." In *Wiley StatsRef: Statistics Reference Online*. Wiley. <https://doi.org/10.1002/9781118445112.stat06558>.
- Bergstra, James, Olivier Breuleux, Frédéric Bastien, Pascal Lamblin, Razvan Pascanu, Guillaume Desjardins, Joseph Turian, David Warde-Farley, and Yoshua Bengio. 2010. "Theano: A CPU and GPU Math Compiler in Python." *Proceedings of the Python in Science Conference* 4: 18–24. <https://doi.org/10.25080/majora-92bf1922-003>.
- Beyer, Betsy, Chris Jones, Jennifer Peto, and Niall Richard Murphy. 2016. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media.
- Beyer, Lucas, Olivier J. Hénaff, Alexander Kolesnikov, Xiaohua Zhai, and Aäron van den Oord. 2020. "Are We Done with ImageNet?" *arXiv Preprint arXiv:2006.07159*.
- Bird, Sarah, Miro Dudík, Richard Edgar, Brandon Horn, Roman Lutz, Vanessa Milan, Mehrnoosh Sameki, Hanna Wallach, and Kathleen Walker. 2020. "Fairlearn: A Toolkit for Assessing and Improving Fairness in AI." *Microsoft Technical Report MSR-TR-2020-32*.
- Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Springer.
- Blalock, Davis, Jose Javier Gonzalez Ortiz, Jonathan Frankle, and John Gutttag. 2020. "What Is the State of Neural Network Pruning?" *Proceedings of Machine Learning and Systems (MLSys)* 2: 129–46.
- Bobrow, Daniel G. 1964. "Natural Language Input for a Computer Problem Solving System." PhD thesis, Massachusetts Institute of Technology.
- Boehm, Barry W. 1981. *Software Engineering Economics*. Prentice-Hall.
- Bolukbasi, Tolga, Kai-Wei Chang, James Y. Zou, Venkatesh Saligrama, and Adam Tauman Kalai. 2016. "Man Is to Computer Programmer as Woman Is to Homemaker? Debiasing Word Embeddings." *Advances in Neural Information Processing Systems (NeurIPS)*, 4349–57.
- Bommasani, Rishi, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, et al. 2021. "On the Opportunities and Risks of Foundation Models." *arXiv Preprint arXiv:2108.07258*.
- Boroumand, Amirali, Saugata Ghose, Youngsok Kim, Rachata Ausavarungnirun, Eric Shiu, Rahul Thakur, Daehyun Kim, et al. 2018. "Google Workloads for Consumer Devices: Mitigating Data Movement Bottlenecks." *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '18*, 316–31. <https://doi.org/10.1145/3173162.3173177>.
- Bourtole, Lucas, Varun Chandrasekaran, Christopher A. Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. 2021. "Machine Unlearning." *2021 IEEE Symposium on Security and Privacy (SP)*, 141–59. <https://doi.org/10.1109/sp40001.2021.00019>.
- Bradbury, James, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Neca, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs*.
- Breck, Eric, Shanjing Cai, Eric Nielsen, Michael Salib, and D. Sculley. 2017. "The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction." *Proceedings of the 2017 IEEE International Conference on Big Data (Big Data)*, 1123–32. <https://doi.org/10.1109/bigdata.2017.8258038>.
- Breck, Eric, Neoklis Polyzotis, Sudip Roy, Steven Euijong Whang, and Martin Zinkevich. 2019. "Data Validation for Machine Learning." *Proceedings of the 2nd SysML Conference* 1: 334–47.
- Brewer, Eric A. 2000. "Towards Robust Distributed Systems (Abstract)." *Proceedings of the Nineteenth Annual ACM Symposium on Principles of Distributed Computing*, 7. <https://doi.org/10.1145/343477.343502>.

- Broder, A. Z. 1997. "On the Resemblance and Containment of Documents." *Proceedings. Compression and Complexity of SEQUENCES 1997 (Cat. No.97TB100171)*, 21–29. <https://doi.org/10.1109/sequen.1997.666900>.
- Brown, Tom B., Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, et al. 2020. "Language Models Are Few-Shot Learners." *Advances in Neural Information Processing Systems 33*: 1877–901. <https://doi.org/10.48550/arxiv.2005.14165>.
- Brutlag, Jake. 2009. *Speed Matters for Google Web Search*. Google Research Blog.
- Buolamwini, Joy, and Timnit Gebru. 2018. "Gender Shades: Intersectional Accuracy Disparities in Commercial Gender Classification." *Conference on Fairness, Accountability and Transparency*, 77–91.
- Burns, Brendan, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. 2016. "Borg, Omega, and Kubernetes." *Communications of the ACM 59* (5): 50–57. <https://doi.org/10.1145/2890784>.
- Cai, Han, Chuang Gan, Ligeng Zhu, and Song Han. 2020. "TinyTL: Reduce Activations, Not Trainable Parameters for Efficient on-Device Learning." *Advances in Neural Information Processing Systems 33*: 11285–97.
- Campbell, Murray, Jr. Hoane A. Joseph, and Feng-hsiung Hsu. 2002. "Deep Blue." *Artificial Intelligence 134* (1-2): 57–83. [https://doi.org/10.1016/s0004-3702\(01\)00129-1](https://doi.org/10.1016/s0004-3702(01)00129-1).
- Cao, Yinzhi, and Junfeng Yang. 2015. "Towards Making Systems Forget with Machine Unlearning." *2015 IEEE Symposium on Security and Privacy*, 463–80. <https://doi.org/10.1109/sp.2015.35>.
- Chapman, Pete, Julian Clinton, Randy Kerber, Thomas Khabaza, Thomas Reinartz, Colin Shearer, and Rudiger Wirth. 2000. "CRISP-DM 1.0: Step-by-Step Data Mining Guide." *SPSS Inc 1*: 78.
- Chen, Andrew, Andy Chow, Aaron Davidson, Arjun DCunha, Ali Ghodsi, Sue Ann Hong, Andy Konwinski, et al. 2020. "Developments in MLflow: A System to Accelerate the Machine Learning Lifecycle." *Proceedings of the Fourth International Workshop on Data Management for End-to-End Machine Learning*, 1–4. <https://doi.org/10.1145/3399579.3399867>.
- Chen, Emma, Shvetank Prakash, Vijay Janapa Reddi, David Kim, and Pranav Rajpurkar. 2023. "A Framework for Integrating Artificial Intelligence for Clinical Care with Continuous Therapeutic Monitoring." *Nature Biomedical Engineering 9* (4): 445–54. <https://doi.org/10.1038/s41551-023-01115-0>.
- Chen, Mark, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, et al. 2021. "Evaluating Large Language Models Trained on Code." *arXiv Preprint arXiv:2107.03374*.
- Chen, Tianqi, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Q. Yan, Haichen Shen, Meghan Cowan, et al. 2018. "TVM: An Automated End-to-End Optimizing Compiler for Deep Learning." *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI '18)*, 578–94.
- Chen, Tianqi, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. "Training Deep Nets with Sublinear Memory Cost." *arXiv Preprint arXiv:1604.06174*.
- Chen, Tianshi, Zidong Du, Ninghui Sun, Jia Wang, Chengyong Wu, Yunji Chen, and Olivier Temam. 2014. "Dian-Nao: A Small-Footprint High-Throughput Accelerator for Ubiquitous Machine-Learning." *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '14)*, 269–84. <https://doi.org/10.1145/2541940.2541967>.
- Chen, Ting, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. 2020. "A Simple Framework for Contrastive Learning of Visual Representations." *Proceedings of the 37th International Conference on Machine Learning, ICML'20*, vol. 119: 1597–607.
- Chen, Yanxi, Xuchen Pan, Yaliang Li, Bolin Ding, and Jingren Zhou. 2024. "EE-LLM: Large-Scale Training and Inference of Early-Exit Large Language Models with 3D Parallelism." *Proceedings of the 41st International Conference on Machine Learning (ICML)*.
- Chen, Yu-Hsin, Joel S. Emer, and Vivienne Sze. 2016. "Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks." *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 367–79. <https://doi.org/10.1109/ISCA.2016.40>.
- Chen, Yu-Hsin, Joel Emer, and Vivienne Sze. 2017. "Using Dataflow to Optimize Energy Efficiency of Deep Neural Network Accelerators." *IEEE Micro 37* (3): 12–21. <https://doi.org/10.1109/mm.2017.54>.
- Chetlur, Sharan, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. "cuDNN: Efficient Primitives for Deep Learning." *arXiv Preprint arXiv:1410.0759*.
- Cho, Aeree, Grace C. Kim, Alexander Karpekov, Alec Helbling, Zijie J. Wang, Seongmin Lee, Benjamin Hoover, and Duen Horng (Polo) Chau. 2025. "TRANSFORMER EXPLAINER: Interactive Learning of Text-Generative Models." *Proceedings of the AAAI Conference on Artificial Intelligence 39* (28): 29625–27. <https://doi.org/10.1609/aaai.v39i28.35347>.
- Cho, Kyunghyun, Bart van Merriënboer, Dzmitry Bahdanau, and Yoshua Bengio. 2014. "On the Properties of Neural Machine Translation: Encoder-Decoder Approaches." *Proceedings of SSST-8, Eighth Workshop on Syntax, Semantics and Structure in Statistical Translation*, 103–11. <https://doi.org/10.3115/v1/w14-4012>.

- Choi, Dami, Alexandre Passos, Christopher J. Shallue, and George E. Dahl. 2019. "Faster Neural Network Training with Data Echoing." *arXiv Preprint*, ahead of print. <https://doi.org/10.48550/arXiv.1907.05550>.
- Choi, Jungwook, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan. 2018. "PACT: Parameterized Clipping Activation for Quantized Neural Networks." *arXiv Preprint*.
- Chollet, François. 2018. *Deep Learning with Python*. Manning Publications.
- Chollet, François et al. 2024. *Keras*.
- Choquette, Jack. 2023. "NVIDIA Hopper H100 GPU: Scaling Performance." *IEEE Micro* 43 (3): 9–17. <https://doi.org/10.1109/mm.2023.3256796>.
- Choquette, Jack, Wishwesh Gandhi, Olivier Giroux, Nick Stam, and Ronny Krashinsky. 2021. "NVIDIA A100 Tensor Core GPU: Performance and Innovation." *IEEE Micro* 41 (2): 29–35. <https://doi.org/10.1109/mm.2021.3061394>.
- Choudhary, Tejalal, Vipul Mishra, Anurag Goswami, and Jagannathan Sarangapani. 2020. "A Comprehensive Survey on Model Compression and Acceleration." *Artificial Intelligence Review* 53 (7): 5113–55. <https://doi.org/10.1007/s10462-020-09816-7>.
- Choukroun, Yoni, Eli Kravchik, Fan Yang, and Pavel Kisilev. 2019. "Low-Bit Quantization of Neural Networks for Efficient Inference." *2019 IEEE/CVF International Conference on Computer Vision Workshop (ICCVW)*, 3009–18. <https://doi.org/10.1109/iccvw.2019.00363>.
- Chouldechova, Alexandra. 2017. "Fair Prediction with Disparate Impact: A Study of Bias in Recidivism Prediction Instruments." *Big Data* 5 (2): 153–63. <https://doi.org/10.1089/big.2016.0047>.
- Chowdhery, Aakanksha, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, et al. 2022. "PaLM: Scaling Language Modeling with Pathways." *arXiv Preprint arXiv:2204.02311*.
- Chu, Grace, Okan Arikan, Gabriel Bender, Weijun Wang, Achille Brighton, Pieter-Jan Kindermans, Hanxiao Liu, Berkin Akin, Suyog Gupta, and Andrew Howard. 2021. "Discovering Multi-Hardware Mobile Models via Architecture Search." *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 3016–25. <https://doi.org/10.1109/cvprw53098.2021.00337>.
- CircleCI. 2024. *CircleCI: Continuous Integration and Delivery Platform*.
- Clark, Kevin, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning. 2019. "What Does BERT Look at? An Analysis of BERT's Attention." *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, 276–86. <https://doi.org/10.18653/v1/w19-4828>.
- Cloud Native Computing Foundation. 2024a. *Kubernetes: Production-Grade Container Orchestration*.
- Cloud Native Computing Foundation. 2024b. *Prometheus: Monitoring System and Time Series Database*.
- Cohen, Jacob. 1960. "A Coefficient of Agreement for Nominal Scales." *Educational and Psychological Measurement* 20 (1): 37–46. <https://doi.org/10.1177/001316446002000104>.
- Cohen, Taco, and Max Welling. 2016. "Group Equivariant Convolutional Networks." *Proceedings of the 33rd International Conference on Machine Learning (ICML)* 48: 2990–99.
- Coleman, Cody, Edward Chou, Julian Katz-Samuels, Sean Culatana, Peter Bailis, Alexander C. Berg, Robert Nowak, Roshan Sumbaly, Matei Zaharia, and I. Zeki Yalniz. 2022. "Similarity Search for Efficient Active Learning and Search of Rare Concepts." *Proceedings of the AAAI Conference on Artificial Intelligence* 36 (6): 6402–10. <https://doi.org/10.1609/aaai.v36i6.20591>.
- Coleman, Cody, Deepak Narayanan, Daniel Kang, Tian Zhao, Jian Zhang, Luigi Nardi, Peter Bailis, Kunle Olukotun, Christopher Ré, and Matei Zaharia. 2019. "Analysis of DAWNbench, a Time-to-Accuracy Machine Learning Performance Benchmark." *ACM SIGOPS Operating Systems Review* 53 (1): 14–25. <https://doi.org/10.1145/3352020.3352024>.
- Courbariaux, Matthieu, Yoshua Bengio, and Jean-Pierre David. 2015. "BinaryConnect: Training Deep Neural Networks with Binary Weights During Propagations." *Advances in Neural Information Processing Systems (NeurIPS)* 28: 3123–31.
- Cover, Thomas M., and Joy A. Thomas. 2006. *Elements of Information Theory*. 2nd ed. Wiley. <https://doi.org/10.1002/047174882X>.
- Covington, Paul, Jay Adams, and Emre Sargin. 2016. "Deep Neural Networks for YouTube Recommendations." *Proceedings of the 10th ACM Conference on Recommender Systems*, 191–98. <https://doi.org/10.1145/2959100.2959190>.
- Crankshaw, Daniel, Xin Wang, Guilio Zhou, Michael J. Franklin, Joseph E. Gonzalez, and Ion Stoica. 2017. "Clipper: A Low-Latency Online Prediction Serving System." *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, 613–27.
- CrowdFlower. 2016. *Data Science Report 2016*.
- Cubuk, Ekin D., Barret Zoph, Dandelion Mané, Vijay Vasudevan, and Quoc V. Le. 2019. "AutoAugment: Learning Augmentation Strategies from Data." *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 113–23. <https://doi.org/10.1109/cvpr.2019.00020>.

- Cubuk, Ekin D., Barret Zoph, Jonathon Shlens, and Quoc V. Le. 2020. "Randaugment: Practical Automated Data Augmentation with a Reduced Search Space." *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 3008–17. <https://doi.org/10.1109/cvprw50498.2020.00359>.
- Cunningham, Ward. 1992. "The WyCash Portfolio Management System." *ACM SIGPLAN OOPS Messenger* 4 (2): 29–30. <https://doi.org/10.1145/157710.157715>.
- Curnow, H. J., and B. A. Wichmann. 1976. "A Synthetic Benchmark." *The Computer Journal* 19 (1): 43–49. <https://doi.org/10.1093/comjnl/19.1.43>.
- Cybenko, George. 1989. "Approximation by Superpositions of a Sigmoidal Function." *Mathematics of Control, Signals, and Systems* 2 (4): 303–14. <https://doi.org/10.1007/bf02551274>.
- Dalal, Navneet, and Bill Triggs. 2005. "Histograms of Oriented Gradients for Human Detection." *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)* 1: 886–93. <https://doi.org/10.1109/cvpr.2005.177>.
- Dally, Bill. 2023. "Hardware for Deep Learning." *2023 IEEE Hot Chips 35 Symposium (HCS)*, 1–58. <https://doi.org/10.1109/hcs59251.2023.10254716>.
- Dally, William J., Stephen W. Keckler, and David B. Kirk. 2021. "Evolution of the Graphics Processing Unit (GPU)." *IEEE Micro* 41 (6): 42–51. <https://doi.org/10.1109/mm.2021.3113475>.
- Dao, T., D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. 2022. "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness." *Advances in Neural Information Processing Systems (NeurIPS)* 35: 16344–59. <https://doi.org/10.52202/068431-1189>.
- Dao, Tri. 2023. "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning." *arXiv Preprint arXiv:2307.08691*.
- Dao, Tri, Beidi Chen, Nimit Sohoni, Arjun Desai, Michael Poli, Jessica Grogan, Alexander Liu, Aniruddh Rao, Atri Rudra, and Christopher Ré. 2022. "Monarch: Expressive Structured Matrices for Efficient and Accurate Training." *arXiv Preprint*.
- Dastin, Jeffrey. 2018. "Amazon Scraps Secret AI Recruiting Tool That Showed Bias Against Women." *Reuters*.
- Data Protection Commission. 2023. *Data Protection Commission Announces Conclusion of Two Inquiries into Meta Ireland*. Regulatory press release.
- Databricks. 2024. *MLflow: An Open Source Platform for the Machine Learning Lifecycle*.
- David, Robert, Jared Duke, Advait Jain, Vijay Janapa Reddi, Nat Jeffries, Jian Li, Nick Kreeger, et al. 2021. "TensorFlow Lite Micro: Embedded Machine Learning for TinyML Systems." *Proceedings of Machine Learning and Systems* 3: 800–811.
- dbt Labs. 2024. *Dbt (Data Build Tool)*.
- Dean, Jeff, David Patterson, and Cliff Young. 2018. "A New Golden Age in Computer Architecture: Empowering the Machine-Learning Revolution." *IEEE Micro* 38 (2): 21–29. <https://doi.org/10.1109/mm.2018.112130030>.
- Dean, Jeffrey. 2012. *Achieving Rapid Response Times in Large Online Services*. Berkeley AMPLab Cloud Seminar.
- Dean, Jeffrey, and Luiz André Barroso. 2013. "The Tail at Scale." *Communications of the ACM* 56 (2): 74–80. <https://doi.org/10.1145/2408776.2408794>.
- Dean, Jeffrey, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Quoc V. Le, Mark Z. Mao, et al. 2012. "Large Scale Distributed Deep Networks." In *Advances in Neural Information Processing Systems (NeurIPS)*, edited by Peter L. Bartlett, Fernando C. N. Pereira, Christopher J. C. Burges, Léon Bottou, and Kilian Q. Weinberger, vol. 25. Curran Associates.
- Dean, Jeffrey, and Sanjay Ghemawat. 2004. "MapReduce: Simplified Data Processing on Large Clusters." *Proceedings of the 6th Symposium on Operating Systems Design and Implementation (OSDI)*, 137–50.
- DeepSeek. 2024. *Introducing DeepSeek-V3*.
- Dehghani, Zhamak. 2022. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media.
- Deng, Jia, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. "ImageNet: A Large-Scale Hierarchical Image Database." *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 248–58. <https://doi.org/10.1109/cvpr.2009.5206848>.
- Deng, Jia, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2024. *ImageNet*.
- Dennard, Robert H., Frank H. Gaensslen, Hwa-Nien Yu, Victor L. Rideout, Elias Bassous, and Antoine R. LeBlanc. 1974. "Design of Ion-Implanted MOSFET's with Very Small Physical Dimensions." *IEEE J. Solid-State Circuits* 9 (5): 256–68. <https://doi.org/10.1109/jssc.1974.1050511>.
- Denning, Peter J. 1968. "The Working Set Model for Program Behavior." *Communications of the ACM* 11 (5): 323–33. <https://doi.org/10.1145/363095.363141>.

- Dettmers, Tim, Mike Lewis, Sam Shleifer, and Luke Zettlemoyer. 2022. "8-Bit Optimizers via Block-Wise Quantization." *International Conference on Learning Representations (ICLR)*.
- Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. "BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding." *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 4171–86. <https://doi.org/10.18653/v1/n19-1423>.
- DeVries, Terrance, and Graham W. Taylor. 2017. "Improved Regularization of Convolutional Neural Networks with Cutout." *arXiv Preprint arXiv:1708.04552*.
- Dixit, Kaivalya M. 1993. "Overview of the SPEC Benchmarks." In *The Benchmark Handbook for Database and Transaction Systems*, 2nd ed., edited by Jim Gray. Morgan Kaufmann.
- Dongarra, J. J., J. R. Bunch, C. B. Moler, and G. W. Stewart. 1979. *LINPACK Users' Guide*. SIAM. <https://doi.org/10.1137/1.9781611971811>.
- Dongarra, Jack J., Jeremy Du Croz, Sven Hammarling, and Richard J. Hanson. 1988. "An Extended Set of FORTRAN Basic Linear Algebra Subprograms." *ACM Transactions on Mathematical Software* 14 (1): 1–17. <https://doi.org/10.1145/42288.42291>.
- Dosovitskiy, Alexey, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, et al. 2021. "An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale." *International Conference on Learning Representations (ICLR)*.
- Dua, Dheeru, and Casey Graff. 2024. *UCI Machine Learning Repository*.
- Duarte, Javier, Nhan Tran, Ben Hawks, Christian Herwig, Jules Muhizi, Shvetank Prakash, and Vijay Janapa Reddi. 2022. "FastML Science Benchmarks: Accelerating Real-Time Scientific Edge Machine Learning." *arXiv Preprint arXiv:2207.07958*.
- Dubey, Abhimanyu, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, et al. 2024. *The Llama 3 Herd of Models*. arXiv preprint arXiv:2407.21783.
- Dwork, Cynthia. 2008. "Differential Privacy: A Survey of Results." In *Theory and Applications of Models of Computation*. Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-540-79228-4_1.
- Dwork, Cynthia, Frank McSherry, Kobbi Nissim, and Adam Smith. 2006. "Calibrating Noise to Sensitivity in Private Data Analysis." In *Theory of Cryptography Conference (TCC)*, edited by Shai Halevi and Tal Rabin, vol. 3876. Lecture Notes in Computer Science. Springer Berlin Heidelberg. https://doi.org/10.1007/11681878_14.
- Elastic NV. 2024. *Elasticsearch: Distributed Search and Analytics Engine*.
- Elman, Jeffrey L. 1990. "Finding Structure in Time." *Cognitive Science* 14 (2): 179–211. https://doi.org/10.1207/s15516709cog1402_1.
- Elsen, Erich, Marat Dukhan, Trevor Gale, and Karen Simonyan. 2020. "Fast Sparse ConvNets." *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 14617–26. <https://doi.org/10.1109/cvpr42600.2020.01464>.
- Elsken, Thomas, Jan Hendrik Metzen, and Frank Hutter. 2019. "Neural Architecture Search." In *The Springer Series on Challenges in Machine Learning*, vol. 20. Springer International Publishing. https://doi.org/10.1007/978-3-030-05318-5_3.
- Engineering, M. 2016. *Introducing FBLearner Flow: Facebook's AI Backbone*. Engineering at Meta Blog.
- Epoch AI. 2024. *Machine Learning Trends*. Epoch AI Research Database.
- Equal Employment Opportunity Commission, Civil Service Commission, Department of Labor, and Department of Justice. 1978. *Uniform Guidelines on Employee Selection Procedures*. Code of Federal Regulations.
- Esmailzadeh, Hadi, Emily Blem, Renee St. Amant, Karthikeyan Sankaralingam, and Doug Burger. 2011. "Dark Silicon and the End of Multicore Scaling." *Proceedings of the 38th Annual International Symposium on Computer Architecture*, 365–76. <https://doi.org/10.1145/2000064.2000108>.
- Ethernet Alliance. 2025. *2025 Ethernet Roadmap*. Ethernet Alliance roadmap.
- European Data Protection Board. 2018. *Guidelines on Automated Individual Decision-Making and Profiling for the Purposes of Regulation 2016/679*.
- European Parliament and Council of the European Union. 2024. *Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 Laying down Harmonised Rules on Artificial Intelligence (AI Act)*. Official Journal of the European Union, L 2024/1689.
- European Parliament, and Council of the European Union. 2016. *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016*. Official Journal of the European Union.
- European Space Agency. 1996. *Ariane 501: Presentation of Inquiry Board Report*. European Space Agency press release.
- Everingham, Mark, Luc Van Gool, Christopher K. I. Williams, John Winn, and Andrew Zisserman. 2009. "The Pascal Visual Object Classes (VOC) Challenge." *International Journal of Computer Vision* 88 (2): 303–38. <https://doi.org/10.1007/s11263-009-0275-4>.

- Fedus, William, Barret Zoph, and Noam Shazeer. 2022. "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity." *Journal of Machine Learning Research* 23 (120): 1–39.
- Feigenbaum, Edward A. 1984. "Knowledge Engineering: The Applied Side of Artificial Intelligence." *Annals of the New York Academy of Sciences* 426: 91–107. <https://doi.org/10.1111/j.1749-6632.1984.tb16513.x>.
- Feng, Wu-chun, and Kirk W. Cameron. 2007. "The Green500 List: Encouraging Sustainable Supercomputing." *Computer* 40 (12): 50–55. <https://doi.org/10.1109/MC.2007.445>.
- Fitzpatrick, Thomas B. 1988. "The Validity and Practicality of Sun-Reactive Skin Types i Through VI." *Archives of Dermatology* 124 (6): 869. <https://doi.org/10.1001/archderm.1988.01670060015008>.
- FlatBuffers Authors. 2026. *FlatBuffers Documentation*.
- Fleiss, Joseph L. 1971. "Measuring Nominal Scale Agreement Among Many Raters." *Psychological Bulletin* 76 (5): 378–82. <https://doi.org/10.1037/h0031619>.
- Flynn, M. J. 1966. "Very High-Speed Computing Systems." *Proceedings of the IEEE* 54 (12): 1901–9. <https://doi.org/10.1109/proc.1966.5273>.
- Fowler, Martin. 2014. *Canary Release*. Martin Fowler's Blog.
- Frankle, Jonathan, and Michael Carbin. 2019. "The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks." *International Conference on Learning Representations (ICLR)*.
- Fredrikson, Matt, Somesh Jha, and Thomas Ristenpart. 2015. "Model Inversion Attacks That Exploit Confidence Information and Basic Countermeasures." *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, 1322–33. <https://doi.org/10.1145/2810103.2813677>.
- Frostig, Roy, Matthew James Johnson, and Chris Leary. 2018. "Compiling Machine Learning Programs via High-Level Tracing." *Systems for Machine Learning*.
- Gabor, D. 1946. "Theory of Communication. Part 1: The Analysis of Information." *Journal of the Institution of Electrical Engineers - Part III: Radio and Communication Engineering* 93 (26): 429–41. <https://doi.org/10.1049/ji-3-2.1946.0074>.
- Gadre, Samir Yitzhak, Gabriel Ilharco, Alex Fang, Jonathan Hayase, Georgios Smyrnis, Thao Nguyen, Ryan Marten, et al. 2023. "DataComp: In Search of the Next Generation of Multimodal Datasets." *Advances in Neural Information Processing Systems (NeurIPS)* 36: 27092–112. <https://doi.org/10.52202/075280-1179>.
- Gale, Trevor, Erich Elsen, and Sara Hooker. 2019. "The State of Sparsity in Deep Neural Networks." *arXiv Preprint arXiv:1902.09574*.
- Gale, Trevor, Deepak Narayanan, Cliff Young, and Matei Zaharia. 2022. "MegaBlocks: Efficient Sparse Training with Mixture-of-Experts." *arXiv Preprint*.
- Gale, Trevor, Matei Zaharia, Cliff Young, and Erich Elsen. 2020. "Sparse GPU Kernels for Deep Learning." *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis* 2020: 8955–67. <https://doi.org/10.1109/sc41405.2020.00021>.
- Gama, João, André Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. 2014. "A Survey on Concept Drift Adaptation." *ACM Computing Surveys* 46 (4): 1–37. <https://doi.org/10.1145/2523813>.
- Gao, Leo, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, et al. 2020. "The Pile: An 800GB Dataset of Diverse Text for Language Modeling." *arXiv Preprint arXiv:2101.00027*.
- Gartner. 2024. *Gartner Forecasts Worldwide Public Cloud End-User Spending to Total \$679 Billion in 2024*. Gartner, Inc.
- Gebru, Timnit, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. 2021. "Datasheets for Datasets." *Communications of the ACM* 64 (12): 86–92. <https://doi.org/10.1145/3458723>.
- Gholami, Amir, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. 2021. "A Survey of Quantization Methods for Efficient Neural Network Inference." *arXiv Preprint arXiv:2103.13630 abs/2103.13630*.
- Gholami, Amir, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. 2022. "A Survey of Quantization Methods for Efficient Neural Network Inference." In *Low-Power Computer Vision: Improve the Efficiency of Artificial Intelligence*. Chapman; Hall/CRC. <https://doi.org/10.1201/9781003162810-13>.
- Gholami, Amir, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, and Kurt Keutzer. 2024. "AI and Memory Wall." *IEEE Micro* 44 (3): 33–39. <https://doi.org/10.1109/mm.2024.3373763>.
- Gilbert, Seth, and Nancy Lynch. 2002. "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services." *ACM SIGACT News* 33 (2): 51–59. <https://doi.org/10.1145/564585.564601>.
- Ginsberg, Jeremy, Matthew H. Mohebbi, Rajan S. Patel, Lynnette Brammer, Mark S. Smolinski, and Larry Brilliant. 2009. "Detecting Influenza Epidemics Using Search Engine Query Data." *Nature* 457 (7232): 1012–14. <https://doi.org/10.1038/nature07634>.

- GitHub, Inc. 2024a. *GitHub*.
- GitHub, Inc. 2024b. *GitHub Actions*.
- Glorot, Xavier, and Yoshua Bengio. 2010. "Understanding the Difficulty of Training Deep Feedforward Neural Networks." *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)* 9: 249–56.
- Gojek, and Google. 2019. *Feast: An Open Source Feature Store for Machine Learning*. Google Cloud Blog.
- Goodfellow, I. J., J. Shlens, and C. Szegedy. 2015. "Explaining and Harnessing Adversarial Examples." *International Conference on Learning Representations (ICLR)*.
- Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. MIT Press.
- Goodhart, Charles A. E. 1984. "Problems of Monetary Management: The UK Experience." In *Monetary Theory and Practice*. Palgrave. https://doi.org/10.1007/978-1-349-17295-5_4.
- Google. 2024a. *Our Next-Generation Model: Gemini 1.5*.
- Google. 2024b. *Static Vs. Dynamic Inference*. Google Machine Learning Crash Course.
- Google. 2025. *XLA: Optimizing Compiler for Machine Learning*.
- Google Cloud. 2024a. *Google Cloud Platform Documentation*. <https://cloud.google.com/docs>.
- Google Cloud. 2024b. *Google Cloud Storage*.
- Google Cloud. 2024c. *Tune Models Overview*.
- Google Cloud. 2024d. *Vertex AI Model Registry*.
- Google Cloud. 2026a. *Explore Models in Model Garden*.
- Google Cloud. 2026b. *MLOps: Continuous Delivery and Automation Pipelines in Machine Learning*.
- Google Cloud. 2026c. *Spot VMs*.
- Google Cloud. 2026d. *TPU v6e*.
- Google Developers Blog. 2024. *Gemini 1.5 Flash Price Drop with Tuning Rollout Complete, and More*.
- Gordon, Mitchell, Kevin Duh, and Nicholas Andrews. 2020. "Compressing BERT: Studying the Effects of Weight Pruning on Transfer Learning." *Proceedings of the 5th Workshop on Representation Learning for NLP*, 143–55. <https://doi.org/10.18653/v1/2020.repl4nlp-1.18>.
- Goto, Kazushige, and Robert A. van de Geijn. 2008. "Anatomy of High-Performance Matrix Multiplication." *ACM Transactions on Mathematical Software* 34 (3): 1–25. <https://doi.org/10.1145/1356052.1356053>.
- Gou, Jianping, Baosheng Yu, Stephen J. Maybank, and Dacheng Tao. 2021. "Knowledge Distillation: A Survey." *International Journal of Computer Vision* 129 (6): 1789–819. <https://doi.org/10.1007/s11263-021-01453-z>.
- Goyal, Priya, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. 2017. "Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour." *arXiv Preprint arXiv:1706.02677* abs/1706.02677.
- Graphcore. 2020. *The Colossus MK2 IPU Processor*.
- Gray, Allison. 2015. *NVIDIA and IBM Cloud Support ImageNet Large Scale Visual Recognition Challenge*. NVIDIA Technical Blog.
- Gretton, Arthur, Karsten M. Borgwardt, Malte J. Rasch, Bernhard Schölkopf, and Alexander Smola. 2012. "A Kernel Two-Sample Test." *Journal of Machine Learning Research* 13 (25): 723–73.
- Groeneveld, Dirk, Iz Beltagy, Pete Walsh, Akshita Bhagia, Rodney Kinney, Oyvind Tafjord, Ananya Harsh Jha, et al. 2024. "OLMo: Accelerating the Science of Language Models." *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 15789–809. <https://doi.org/10.18653/v1/2024.acl-long.841>.
- gRPC Authors. 2026. *Introduction to gRPC*.
- Gudivada, Venkat N., Amy Apon, and Junhua Ding. 2017. "Data Quality Considerations for Big Data and Machine Learning: Going Beyond Data Cleaning and Transformations." *International Journal on Advances in Software* 10 (1–2): 1–20.
- Gujarati, Arpan, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. 2020. "Clockwork: Predictable and Scalable DNN Inference in the Cloud." *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 443–62.

- Gulshan, Varun, Lily Peng, Marc Coram, Martin C. Stumpe, Derek Wu, Arunachalam Narayanaswamy, Subhashini Venugopalan, et al. 2016. "Development and Validation of a Deep Learning Algorithm for Detection of Diabetic Retinopathy in Retinal Fundus Photographs." *JAMA* 316 (22): 2402. <https://doi.org/10.1001/jama.2016.17216>.
- Guo, Chuan, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. "On Calibration of Modern Neural Networks." *International Conference on Machine Learning (ICML)* 70: 1321–30.
- Gupta, Suyog, Ankur Agrawal, Kailash Gopalakrishnan, and Pritish Narayanan. 2015. "Deep Learning with Limited Numerical Precision." *International Conference on Machine Learning (ICML)*, 1737–46.
- Gupta, Udit, Carole-Jean Wu, Xiaodong Wang, Maxim Naumov, Brandon Reagen, David Brooks, Bradford Cottel, et al. 2020. "The Architectural Implications of Facebook's DNN-Based Personalized Recommendation." *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 488–501. <https://doi.org/10.1109/HPCA47549.2020.00047>.
- Gustafson, John L. 1988. "Reevaluating Amdahl's Law." *Communications of the ACM* 31 (5): 532–33. <https://doi.org/10.1145/42411.42415>.
- Halevy, Alon, Peter Norvig, and Fernando Pereira. 2009. "The Unreasonable Effectiveness of Data." *IEEE Intelligent Systems* 24 (2): 8–12. <https://doi.org/10.1109/mis.2009.36>.
- Han, Song, Xingyu Liu, Huizi Mao, Jing Pu, Ardavan Pedram, Mark A. Horowitz, and William J. Dally. 2016. "EIE: Efficient Inference Engine on Compressed Deep Neural Network." *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 243–54. <https://doi.org/10.1109/isca.2016.30>.
- Han, Song, Huizi Mao, and William J. Dally. 2016. "Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding." *arXiv Preprint*.
- Han, Song, Jeff Pool, John Tran, and William J. Dally. 2015. "Learning Both Weights and Connections for Efficient Neural Networks." *Advances in Neural Information Processing Systems 28 (NeurIPS 2015)*, 1135–43.
- Harchol-Balter, Mor. 2013. *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*. Cambridge University Press. <https://doi.org/10.1017/cbo9781139226424>.
- Hardt, Moritz, Eric Price, and Nathan Srebro. 2016. "Equality of Opportunity in Supervised Learning." *Advances in Neural Information Processing Systems (NeurIPS)* 29: 3315–23.
- Harris, Charles R., K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, et al. 2020. "Array Programming with NumPy." *Nature* 585 (7825): 357–62. <https://doi.org/10.1038/s41586-020-2649-2>.
- Harvard Law School. 2024. *Does ChatGPT Violate New York Times Copyrights?*
- HashiCorp. 2014. *Terraform: Infrastructure as Code*. Software available from <https://www.terraform.io/>.
- Hatcher, Blake Douglas. 2024. "Automating Server Deployments with Ansible: Utilizing Automation in DevOps." *Journal of Computing Sciences in Colleges* 40 (3): 42–43.
- Hazelwood, Kim, Sarah Bird, David Brooks, Soumith Chintala, Utku Diril, Dmytro Dzhulgakov, Mohamed Fawzy, et al. 2018. "Applied Machine Learning at Facebook: A Datacenter Infrastructure Perspective." *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 620–29. <https://doi.org/10.1109/hpca.2018.00059>.
- He, Kaiming, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. 2020. "Momentum Contrast for Unsupervised Visual Representation Learning." *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 9726–35. <https://doi.org/10.1109/cvpr42600.2020.00975>.
- He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification." *2015 IEEE International Conference on Computer Vision (ICCV)*, 1026–34. <https://doi.org/10.1109/iccv.2015.123>.
- He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016a. "Deep Residual Learning for Image Recognition." *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–78. <https://doi.org/10.1109/cvpr.2016.90>.
- He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016b. "Identity Mappings in Deep Residual Networks." *Computer Vision – ECCV 2016*, 630–45. https://doi.org/10.1007/978-3-319-46493-0_38.
- Henderson, Peter, Jieru Hu, Joshua Romoff, Emma Brunskill, Dan Jurafsky, and Joelle Pineau. 2020. "Towards the Systematic Reporting of the Energy and Carbon Footprints of Machine Learning." *CoRR* abs/2002.05651 (248): 1–43. <https://doi.org/10.48550/arxiv.2002.05651>.
- Henderson, Peter, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. 2018. "Deep Reinforcement Learning That Matters." *Proceedings of the AAAI Conference on Artificial Intelligence* 32 (1): 3207–14. <https://doi.org/10.1609/aaai.v32i1.11694>.
- Hendler, James A. 2008. "Avoiding Another AI Winter." *IEEE Intelligent Systems* 23 (2): 2–4. <https://doi.org/10.1109/MIS.2008.20>.
- Hendrycks, Dan, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. "Measuring Massive Multitask Language Understanding." *arXiv Preprint*.

- Hendrycks, Dan, and Kevin Gimpel. 2016. "Gaussian Error Linear Units (GELUs)." *arXiv Preprint arXiv:1606.08415*.
- Hennessy, J. L., and D. A. Patterson. 2011. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann.
- Hennessy, John L., and David A. Patterson. 2017. *Computer Architecture: A Quantitative Approach*. 6th ed. Morgan Kaufmann.
- Hennessy, John L., and David A. Patterson. 2019. "A New Golden Age for Computer Architecture." *Communications of the ACM* 62 (2): 48–60. <https://doi.org/10.1145/3282307>.
- Hermann, Jeremy, and Mike Del Balso. 2017. *Meet Michelangelo: Uber's Machine Learning Platform*. Uber Engineering Blog.
- Hernandez, Danny, and Tom B. Brown. 2020. "Measuring the Algorithmic Efficiency of Neural Networks." *arXiv Preprint arXiv:2005.04305*, ahead of print. <https://doi.org/10.48550/arxiv.2005.04305>.
- Hestness, Joel, Sharan Narang, Newsha Ardalani, Gregory Diamos, Heewoo Jun, Hassan Kianinejad, Md. Mostofa Ali Patwary, Yang Yang, and Yanqi Zhou. 2017. "Deep Learning Scaling Is Predictable, Empirically." *arXiv Preprint arXiv:1712.00409*.
- Hinton, Geoffrey, Oriol Vinyals, and Jeff Dean. 2015. "Distilling the Knowledge in a Neural Network." *arXiv Preprint*.
- Hirschberg, Julia, and Christopher D. Manning. 2015. "Advances in Natural Language Processing." *Science* 349 (6245): 261–66. <https://doi.org/10.1126/science.aaa8685>.
- Hochreiter, Sepp, and Jürgen Schmidhuber. 1997. "Long Short-Term Memory." *Neural Computation* 9 (8): 1735–80. <https://doi.org/10.1162/neco.1997.9.8.1735>.
- Hoefler, Torsten, Dan Alistarh, Tal Ben-Nun, Nikoli Dryden, and Alexandra Peste. 2021. "Sparsity in Deep Learning: Pruning and Growth for Efficient Inference and Training in Neural Networks." *Journal of Machine Learning Research* 22 (241): 1–124.
- Hoffmann, Jordan, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, et al. 2022. "Training Compute-Optimal Large Language Models." *Advances in Neural Information Processing Systems (NeurIPS)* 35: 30016–30. <https://doi.org/10.52202/068431-2176>.
- Holtzman, Ari, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2020. "The Curious Case of Neural Text Degeneration." *Proceedings of the 8th International Conference on Learning Representations (ICLR 2020)*.
- Hooker, Sara. 2021. "The Hardware Lottery." *Communications of the ACM* 64 (12): 58–65. <https://doi.org/10.1145/3467017>.
- Hornik, Kurt, Maxwell Stinchcombe, and Halbert White. 1989. "Multilayer Feedforward Networks Are Universal Approximators." *Neural Networks* 2 (5): 359–66. [https://doi.org/10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8).
- Horowitz, Mark. 2014. "1.1 Computing's Energy Problem (and What We Can Do about It)." *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, 10–14. <https://doi.org/10.1109/isscc.2014.6757323>.
- Howard, A. G., M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam. 2017. "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications." *CoRR* abs/1704.04861.
- Howard, Andrew, Mark Sandler, Bo Chen, Weijun Wang, Liang-Chieh Chen, Mingxing Tan, Grace Chu, et al. 2019. "Searching for MobileNetV3." *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 1314–24. <https://doi.org/10.1109/iccv.2019.00140>.
- Howarth, Jesse. 2020. "Why Discord Is Switching from Go to Rust."
- Hu, Jie, Peng Lin, Huajun Zhang, Zining Lan, Wenxin Chen, Kailiang Xie, Siyun Chen, Hao Wang, and Sheng Chang. 2023. "A Dynamic Pruning Method on Multiple Sparse Structures in Deep Neural Networks." *IEEE Access* 11: 38448–57. <https://doi.org/10.1109/access.2023.3267469>.
- Hu, Ting-Kuei, Tianlong Chen, Haotao Wang, and Zhangyang Wang. 2020. "Triple Wins: Boosting Accuracy, Robustness and Efficiency Together by Enabling Input-Adaptive Inference." *International Conference on Learning Representations (ICLR)*.
- Huang, Gao, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q. Weinberger. 2017. "Densely Connected Convolutional Networks." *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2261–69. <https://doi.org/10.1109/cvpr.2017.243>.
- Huang, Y., Y. Cheng, A. Bapna, O. Firat, D. Chen, M. X. Chen, H. Lee, et al. 2019. "GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism." *Advances in Neural Information Processing Systems (NeurIPS)* 32: 103–12.
- Hubara, Itay, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2018. "Quantized Neural Networks: Training Neural Networks with Low Precision Weights and Activations." *Journal of Machine Learning Research* 18 (187): 1–30.
- Hugging Face. 2026. *Safetensors Documentation*.
- HumanSignal. 2024. *Label Studio: Open Source Data Labeling Platform*.
- Hutter, Frank, Lars Kotthoff, and Joaquin Vanschoren. 2019. *Automated Machine Learning: Methods, Systems, Challenges*. In *Automated Machine Learning*. The Springer Series on Challenges in Machine Learning. Springer International Publishing. <https://doi.org/10.1007/978-3-030-05318-5>.

- Hwu, Wen-mei W. 2011. "Introduction." In *GPU Computing Gems Emerald Edition*. Elsevier. <https://doi.org/10.1016/b978-0-12-384988-5.00064-4>.
- Iandola, Forrest N., Song Han, Matthew W. Moskewicz, Khalid Ashraf, William J. Dally, and Kurt Keutzer. 2016. "SqueezeNet: AlexNet-Level Accuracy with 50x Fewer Parameters and <0.5MB Model Size." *ArXiv Preprint abs/1602.07360*.
- IBM. 2024. *IBM Watson OpenScale: Data Drift Detection*.
- IEEE Standards Association. 2019a. *IEEE 2416-2019: Standard for Power Modeling to Enable System-Level Analysis*.
- IEEE Standards Association. 2019b. *IEEE 754-2019: Standard for Floating-Point Arithmetic*. <https://doi.org/10.1109/IEEESTD.2019.8766229>.
- IEEE Standards Association. 2024. *IEEE Standards Association: Working Groups for AI and ML*.
- Ignatov, Andrey, and Radu Timofte. 2024. *AI Benchmark*.
- InfiniBand Trade Association. 2000. *InfiniBand Architecture Specification Volume 1*. InfiniBand Trade Association.
- Intel Corporation. 2021a. "Intel Advanced Matrix Extensions (Intel AMX)." *Intel Architecture Instruction Set Extensions Programming Reference*.
- Intel Corporation. 2021b. *oneDNN: Intel's Deep Learning Neural Network Library*.
- Intel Corporation. 2026a. *Intel oneAPI Math Kernel Library*.
- Intel Corporation. 2026b. *OpenVINO Documentation*.
- Ioffe, Sergey, and Christian Szegedy. 2015. "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift." *Proceedings of the 32nd International Conference on Machine Learning (ICML)* 37: 448–56.
- ISO. 2024. *ISO/IEC JTC 1/SC 42 Artificial Intelligence*.
- Iterative. 2024. *Data Version Control (DVC)*.
- Jacob, Benoit, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference." *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2704–13. <https://doi.org/10.1109/cvpr.2018.00286>.
- Janapa Reddi, Vijay et al. 2022. "MLPerf Mobile V2. 0: An Industry-Standard Benchmark Suite for Mobile Machine Learning." *Proceedings of Machine Learning and Systems (MLSys)* 4: 806–23.
- Janapa Reddi, Vijay, Brian Plancher, Susan Kennedy, Laurence Moroney, Pete Warden, Lara Suzuki, Anant Agarwal, et al. 2022. "Widening Access to Applied Machine Learning with TinyML." *Harvard Data Science Review* 4 (1). <https://doi.org/10.1162/99608f92.762d171a>.
- Jevons, William Stanley. 1865. *The Coal Question: An Inquiry Concerning the Progress of the Nation, and the Probable Exhaustion of Our Coal Mines*. Macmillan; Co.
- Jia, Xu, Bert De Brabandere, Tinne Tuytelaars, and Luc Van Gool. 2016. "Dynamic Filter Networks." *Advances in Neural Information Processing Systems (NeurIPS)* 29.
- Jia, Yangqing, Evan Shelhamer, Jeff Donahue, Sergey Karayev, Jonathan Long, Ross Girshick, Sergio Guadarrama, and Trevor Darrell. 2014. "Caffe: Convolutional Architecture for Fast Feature Embedding." *Proceedings of the 22nd ACM International Conference on Multimedia*, 675–78. <https://doi.org/10.1145/2647868.2654889>.
- Jia, Zhihao, James Thomas, Todd Warszawski, Mingyu Gao, Matei Zaharia, and Alex Aiken. 2019. "Optimizing DNN Computation with Relaxed Graph Substitutions." *Proceedings of Machine Learning and Systems (MLSys)*.
- Jiang, A. Q., A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, et al. 2024. "Mixtral of Experts." *arXiv Preprint arXiv:2401.04088*.
- Jiao, Xiaoqi, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. 2020. "TinyBERT: Distilling BERT for Natural Language Understanding." *Findings of the Association for Computational Linguistics: EMNLP 2020*, 4163–74. <https://doi.org/10.18653/v1/2020.findings-emnlp.372>.
- Johnson, Jeff, Matthijs Douze, and Hervé Jégou. 2019. "Billion-Scale Similarity Search with GPUs." *IEEE Transactions on Big Data* 7 (3): 535–47. <https://doi.org/10.1109/tbdata.2019.2921572>.
- Johnston, Casey. 2012. "Netflix Never Used Its \$1 Million Algorithm Due to Engineering Costs." *Ars Technica*, April.
- Jordan, T. L. 1982. "A Guide to Parallel Computation and Some Cray-1 Experiences." In *Parallel Computations*. Elsevier. <https://doi.org/10.1016/b978-0-12-592101-5.50006-3>.

- Jouppi, Norman P., Doe Hyun Yoon, Matthew Ashcraft, Mark Gottscho, Thomas B. Jablin, George Kurian, James Laudon, et al. 2021. "Ten Lessons from Three Generations Shaped Google's TPUv4: Industrial Product." *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 1–14. <https://doi.org/10.1109/isca52012.2021.00010>.
- Jouppi, Norman P., Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, et al. 2017. "In-Datacenter Performance Analysis of a Tensor Processing Unit." *Proceedings of the 44th Annual International Symposium on Computer Architecture*, ISCA '17, 1–12. <https://doi.org/10.1145/3079856.3080246>.
- Jouppi, Norm, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, et al. 2023. "TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings." *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 1–14. <https://doi.org/10.1145/3579371.3589350>.
- Jumper, John, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, et al. 2021. "Highly Accurate Protein Structure Prediction with AlphaFold." *Nature* 596 (7873): 583–89. <https://doi.org/10.1038/s41586-021-03819-2>.
- Kalamkar, Dhiraj, Dheevatsa Mudigere, Naveen Mellempudi, Dipankar Das, Kunal Banerjee, Sasikanth Avancha, Dharma Teja Vooturi, et al. 2019. *A Study of BFLOAT16 for Deep Learning Training*.
- Kamiran, Faisal, and Toon Calders. 2012. "Data Preprocessing Techniques for Classification Without Discrimination." *Knowledge and Information Systems* 33 (1): 1–33. <https://doi.org/10.1007/s10115-011-0463-8>.
- Kaplan, J., S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. 2020. "Scaling Laws for Neural Language Models." *ArXiv Preprint abs/2001.08361*.
- Karpathy, Andrej. 2017. *Software 2.0*. Medium.
- Kiela, Douwe, Max Bartolo, Yixin Nie, Divyansh Kaushik, Atticus Geiger, Zhengxuan Wu, Bertie Vidgen, et al. 2021. "Dynabench: Rethinking Benchmarking in NLP." *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* 9: 4110–24. <https://doi.org/10.18653/v1/2021.naacl-main.324>.
- Kilburn, Tom, David B. G. Edwards, Michael J. Lanigan, and Frank H. Sumner. 1962. "One-Level Storage System." *IRE Transactions on Electronic Computers* EC-11 (2): 223–35. <https://doi.org/10.1109/TEC.1962.5219356>.
- Kim, Wonyoung, Meeta S. Gupta, Gu-Yeon Wei, and David Brooks. 2008. "System Level Analysis of Fast, Per-Core DVFS Using on-Chip Switching Regulators." *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, HPCA '08, 123–34. <https://doi.org/10.1109/hpca.2008.4658633>.
- Kingma, Diederik P., and Jimmy Ba. 2015. "Adam: A Method for Stochastic Optimization." *3rd International Conference on Learning Representations (ICLR)*.
- Kipf, Thomas N., and Max Welling. 2017. "Semi-Supervised Classification with Graph Convolutional Networks." *International Conference on Learning Representations (ICLR)*.
- Kleinberg, Jon, Sendhil Mullainathan, and Manish Raghavan. 2017. "Inherent Trade-Offs in the Fair Determination of Risk Scores." *8th Innovations in Theoretical Computer Science Conference (ITCS 2017)*, Leibniz international proceedings in informatics (LIPIcs), vol. 67: 43:1–23. <https://doi.org/10.4230/LIPIcs.ITCS.2017.43>.
- Kleppmann, Martin. 2016. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.
- Knight, Will. 2023. *OpenAI's CEO Says the Age of Giant AI Models Is Already over*. WIRED.
- Koh, Pang Wei, Shiori Sagawa, Henrik Marklund, Sang Michael Xie, Marvin Zhang, Akshay Balsubramani, Weihua Hu, et al. 2021. "WILDS: A Benchmark of in-the-Wild Distribution Shifts." In *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, edited by Marina Meila and Tong Zhang, vol. 139. Proceedings of Machine Learning Research. PMLR.
- Koizumi, Yuma, Shoichiro Saito, Hisashi Uematsu, Noboru Harada, and Keisuke Imoto. 2019. "ToyADMOS: A Dataset of Miniature-Machine Operating Sounds for Anomalous Sound Detection." *2019 IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, 313–17. <https://doi.org/10.1109/waspaa.2019.8937164>.
- Koomey, Jonathan, Stephen Berard, Marla Sanchez, and Henry Wong. 2011. "Implications of Historical Trends in the Electrical Efficiency of Computing." *IEEE Annals of the History of Computing* 33 (3): 46–54. <https://doi.org/10.1109/mahc.2010.28>.
- Kreuzberger, Dominik, Niklas Kühl, and Sebastian Hirschl. 2023. "Machine Learning Operations (MLOps): Overview, Definition, and Architecture." *IEEE Access* 11: 31866–79. <https://doi.org/10.1109/access.2023.3262138>.
- Krishnamoorthi, Raghuraman. 2018. "Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper." *arXiv Preprint arXiv:1806.08342 abs/1806.08342*.
- Krishnan, Rayan, Pranav Rajpurkar, and Eric J. Topol. 2022. "Self-Supervised Learning in Medicine and Healthcare." *Nature Biomedical Engineering* 6 (12): 1346–52. <https://doi.org/10.1038/s41551-022-00914-1>.
- Krizhevsky, Alex. 2009. *Learning Multiple Layers of Features from Tiny Images*. University of Toronto.

- Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton. 2012. "ImageNet Classification with Deep Convolutional Neural Networks." *Advances in Neural Information Processing Systems (NeurIPS)* 25.
- KServe Community. 2024. *KServe: Highly Scalable and Standards-Based Model Inference Platform on Kubernetes*.
- Kuhn, Max, and Kjell Johnson. 2013. *Applied Predictive Modeling*. Springer New York. <https://doi.org/10.1007/978-1-4614-6849-3>.
- Kullback, Solomon, and Richard A. Leibler. 1951. "On Information and Sufficiency." *The Annals of Mathematical Statistics* 22 (1): 79–86. <https://doi.org/10.1214/aoms/1177729694>.
- Kung, H. T. 1982. "Why Systolic Architectures?" *Computer* 15 (1): 37–46. <https://doi.org/10.1109/mc.1982.1653825>.
- Kung, Hsiang Tsung, and Charles E. Leiserson. 1979. "Systolic Arrays (for VLSI)." *Sparse Matrix Proceedings 1978* 1: 256–82.
- Kuzmin, Andrey, Mart Van Baalen, Yuwei Ren, Markus Nagel, Jorn Peters, and Tijmen Blankevoort. 2022. "FP8 Quantization: The Power of the Exponent." *Advances in Neural Information Processing Systems (NeurIPS)*, 14651–62. <https://doi.org/10.52202/068431-1065>.
- Kuznetsova, Alina, Hassan Rom, Neil Alldrin, Jasper Uijlings, Ivan Krasin, Jordi Pont-Tuset, Shahab Kamali, et al. 2020. "The Open Images Dataset V4: Unified Image Classification, Object Detection, and Visual Relationship Detection at Scale." *International Journal of Computer Vision* 128 (7): 1956–81. <https://doi.org/10.1007/s11263-020-01316-z>.
- Kwon, Woosuk, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. "Efficient Memory Management for Large Language Model Serving with PagedAttention." *Proceedings of the 29th Symposium on Operating Systems Principles*, 611–26. <https://doi.org/10.1145/3600006.3613165>.
- Kwon, Youngeun, and Minsoo Rhu. 2018. "TensorDIMM: A Practical Near-Memory Processing Architecture for Embeddings and Tensor Operations in Deep Learning." *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 740–53. <https://doi.org/10.1145/3352460.3358284>.
- Labelbox, Inc. 2024. *Labelbox Consensus Documentation*.
- Labs, Grafana. 2024. *Grafana*.
- Lai, Liangzhen, Naveen Suda, and Vikas Chandra. 2018. "CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs." *ArXiv Preprint abs/1801.06601*.
- Lam, Monica D., Edward E. Rothberg, and Michael E. Wolf. 1991. "The Cache Performance and Optimizations of Blocked Algorithms." *Proceedings of the Fourth International Conference on Architectural Support for Programming Languages and Operating Systems - ASPLOS-IV*, 63–74. <https://doi.org/10.1145/106972.106981>.
- Lamport, Leslie, Robert Shostak, and Marshall Pease. 1982. "The Byzantine Generals Problem." *ACM Transactions on Programming Languages and Systems* 4 (3): 382–401. <https://doi.org/10.1145/357172.357176>.
- Lampson, Butler W. 1983. "Hints for Computer System Design." *Proceedings of the Ninth ACM Symposium on Operating Systems Principles*, 33–48. <https://doi.org/10.1145/800217.806614>.
- Landis, J. Richard, and Gary G. Koch. 1977. "The Measurement of Observer Agreement for Categorical Data." *Biometrics* 33 (1): 159–74. <https://doi.org/10.2307/2529310>.
- Lange, Klaus-Dieter. 2009. "Identifying Shades of Green: The SPECpower Benchmarks." *IEEE Computer* 42 (3): 95–97. <https://doi.org/10.1109/mc.2009.84>.
- Lattner, Chris, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2020. "MLIR: Scaling Compiler Infrastructure for Domain Specific Computation." *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2–14. <https://doi.org/10.1109/cgo51591.2021.9370308>.
- Lawson, Charles L., Richard J. Hanson, David R. Kincaid, and Fred T. Krogh. 1979. "Basic Linear Algebra Subprograms for Fortran Usage." *ACM Transactions on Mathematical Software* 5 (3): 308–23. <https://doi.org/10.1145/355841.355847>.
- Lazer, David, Ryan Kennedy, Gary King, and Alessandro Vespignani. 2014. "The Parable of Google Flu: Traps in Big Data Analysis." *Science* 343 (6176): 1203–5. <https://doi.org/10.1126/science.1248506>.
- Le Sueur, Etienne, and Gernot Heiser. 2010. "Dynamic Voltage and Frequency Scaling: The Laws of Diminishing Returns." *Proceedings of the 2010 International Conference on Power Aware Computing and Systems*, 1–8.
- Lebedev, Vadim, Yaroslav Ganin, Maksim Rakhuba, Ivan Oseledets, and Victor Lempitsky. 2015. "Speeding up Convolutional Neural Networks Using Fine-Tuned CP-Decomposition." *3rd International Conference on Learning Representations (ICLR)*.
- LeCun, Yann, Yoshua Bengio, and Geoffrey Hinton. 2015. "Deep Learning." *Nature* 521 (7553): 436–44. <https://doi.org/10.1038/nature14539>.
- LeCun, Yann, Bernhard Boser, John S. Denker, Donald Henderson, Richard E. Howard, Wayne Hubbard, and Lawrence D. Jackel. 1989. "Backpropagation Applied to Handwritten Zip Code Recognition." *Neural Computation* 1 (4): 541–51. <https://doi.org/10.1162/neco.1989.1.4.541>.

- LeCun, Yann, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. "Gradient-Based Learning Applied to Document Recognition." *Proceedings of the IEEE* 86 (11): 2278–324. <https://doi.org/10.1109/5.726791>.
- LeCun, Yann, Leon Bottou, Genevieve B. Orr, and Klaus-Robert Müller. 2012. "Efficient BackProp." In *Neural Networks: Tricks of the Trade*, vol. 7700. Lecture Notes in Computer Science. Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-35289-8_3.
- LeCun, Yann, John S. Denker, and Sara A. Solla. 1989. "Optimal Brain Damage." *Advances in Neural Information Processing Systems* 2 (NIPS 1989), 598–605.
- Lee, Katherine, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. 2022. "Deduplicating Training Data Makes Language Models Better." *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 8424–45. <https://doi.org/10.18653/v1/2022.acl-long.577>.
- Lee, Peter. 2016. *Learning from Tay's Introduction*. The Official Microsoft Blog.
- Lehdonvirta, Vili, Boxi Wu, Zoe Jay Hawkins, Celine Caira, and Lucia Russo. 2025. *Measuring Domestic Public Cloud Compute Availability for Artificial Intelligence*. No. 49. OECD Artificial Intelligence Papers. OECD Publishing. <https://doi.org/10.1787/8602a322-en>.
- Lepikhin, Dmitry, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2021. "GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding." *International Conference on Learning Representations (ICLR)*.
- Leviathan, Yaniv, Matan Kalman, and Yossi Matias. 2023. "Fast Inference from Transformers via Speculative Decoding." *International Conference on Machine Learning (ICML)*, 19274–86.
- Li, Chengwei. 2020. *Estimating the Training Cost of GPT-3*.
- Li, Hao, Asim Kadav, Igor Durdanovic, Hanan Samet, and Hans Peter Graf. 2017. "Pruning Filters for Efficient ConvNets." *International Conference on Learning Representations (ICLR)*.
- Li, Lisha, Kevin G. Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. 2017. "Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization." *Journal of Machine Learning Research* 18: 185:1–52.
- Li, Mingzhen, Yi Liu, Xiaoyan Liu, Qingxiao Sun, Xin You, Hailong Yang, Zhongzhi Luan, Lin Gan, Guangwen Yang, and Depei Qian. 2021. "The Deep Learning Compiler: A Comprehensive Survey." *IEEE Transactions on Parallel and Distributed Systems* 32 (3): 708–27. <https://doi.org/10.1109/tpds.2020.3030548>.
- Li, Mu, David G. Andersen, Jun Woo Park, Alexander J. Smola, Amr Ahmed, Vanja Josifovski, James Long, Eugene J. Shekita, and Bor-Ying Su. 2014. "Communication Efficient Distributed Machine Learning with the Parameter Server." *Advances in Neural Information Processing Systems (NeurIPS)* 27: 19–27.
- Li, Ninghui, Tiancheng Li, and Suresh Venkatasubramanian. 2007. "t-Closeness: Privacy Beyond k-Anonymity and l-Diversity." *2007 IEEE 23rd International Conference on Data Engineering*, 106–15. <https://doi.org/10.1109/ICDE.2007.367856>.
- Li, Zhuohan, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, et al. 2023. "{AlpaServe}: Statistical Multiplexing with Model Parallelism for Deep Learning Serving." *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, 663–79.
- Liang, Percy, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, et al. 2022. "Holistic Evaluation of Language Models." *arXiv Preprint arXiv:2211.09110*.
- Lie, Sean. 2021. *Thinking Outside the Die: Architecting the ML Accelerator of the Future*.
- Lighthill, James. 1973. "Artificial Intelligence: A General Survey." In *Artificial Intelligence: A Paper Symposium*. Science Research Council.
- Lin, Ji, Wei-Ming Chen, Yujun Lin, John Cohn, Chuang Gan, and Song Han. 2020. "MCUNet: Tiny Deep Learning on IoT Devices." In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, Virtual*, edited by Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin. Curran Associates.
- Lin, Ji, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. "AWQ: Activation-Aware Weight Quantization for on-Device LLM Compression and Acceleration." *Proceedings of Machine Learning and Systems* 6: 87–100.
- Lin, Tsung-Yi, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. 2014. "Microsoft COCO: Common Objects in Context." In *European Conference on Computer Vision (ECCV)*. Springer. https://doi.org/10.1007/978-3-319-10602-1_48.
- Lindholm, Erik, John Nickolls, Stuart Oberman, and John Montrym. 2008. "NVIDIA Tesla: A Unified Graphics and Computing Architecture." *IEEE Micro* 28 (2): 39–55. <https://doi.org/10.1109/mm.2008.31>.
- Linnainmaa, Seppo. 1970. "The Representation of the Cumulative Rounding Error of an Algorithm as a Taylor Expansion of the Local Rounding Errors." Master's thesis, University of Helsinki.

- Little, John D. C. 1961. "A Proof for the Queuing Formula: $L = \lambda W$." *Operations Research* 9 (3): 383–87. <https://doi.org/10.1287/opre.9.3.383>.
- Liu, Hanxiao, Karen Simonyan, and Yiming Yang. 2019. "DARTS: Differentiable Architecture Search." *International Conference on Learning Representations (ICLR)*.
- Liu, Ze, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. 2021. "Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows." *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, 9992–10002. <https://doi.org/10.1109/ICCV48922.2021.00986>.
- Liu, Zhuang, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. 2022. "A ConvNet for the 2020s." *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 11966–76. <https://doi.org/10.1109/CVPR52688.2022.01167>.
- Loshchilov, Ilya, and Frank Hutter. 2019. "Decoupled Weight Decay Regularization." *Proceedings of the International Conference on Learning Representations (ICLR)*.
- Lowe, David G. 1999. "Object Recognition from Local Scale-Invariant Features." *Proceedings of the Seventh IEEE International Conference on Computer Vision* 2: 1150–1157 vol.2. <https://doi.org/10.1109/iccv.1999.790410>.
- Lu, Yucheng, Shivani Agrawal, Suvinay Subramanian, Oleg Rybakov, Christopher De Sa, and Amir Yazdanbakhsh. 2023. "STEP: Learning n:m Structured Sparsity Masks from Scratch with Precondition." *arXiv Preprint*.
- Lucic, Mario, Karol Kurach, Marcin Michalski, Sylvain Gelly, and Olivier Bousquet. 2018. "Are GANs Created Equal? A Large-Scale Study." *Advances in Neural Information Processing Systems (NeurIPS)* 31.
- Lundberg, Scott M., and Su-In Lee. 2017. "A Unified Approach to Interpreting Model Predictions." *Advances in Neural Information Processing Systems (NeurIPS)* 30: 4765–74.
- Lyons, Richard G. 2011. *Understanding Digital Signal Processing*. 3rd ed. Prentice Hall.
- Machanavajhala, Ashwin, Daniel Kifer, Johannes Gehrke, and Muthu Venkitasubramaniam. 2007. "l-Diversity: Privacy Beyond k-Anonymity." *ACM Transactions on Knowledge Discovery from Data* 1 (3): 3. <https://doi.org/10.1145/1217299.1217302>.
- Mars Climate Orbiter Mishap Investigation Board. 1999. *Mars Climate Orbiter Mishap Investigation Board Phase i Report*. National Aeronautics; Space Administration.
- Maslej, Nestor, Loredana Fattorini, C. Raymond Perrault, Vanessa Parli, Anka Reuel, Erik Brynjolfsson, John Etchemendy, et al. 2024. "Artificial Intelligence Index Report 2024." In CoRR, abs/2405.19522. <https://doi.org/10.48550/ARXIV.2405.19522>.
- Mattson, Peter, Christine Cheng, Cody Coleman, Greg Diamos, Paulius Micekevicius, David Patterson, Hanlin Tang, et al. 2020. "MLPerf Training Benchmark." *Proceedings of Machine Learning and Systems (MLSys)* 2: 336–49.
- Mazumder, Mark. 2021. *TinyML: Multilingual Spoken Words Corpus*. tinyML Talks.
- Mazumder, Mark, Colby Banbury, Xiaozhe Yao, Bojan Karlaš, et al. 2023. "DataPerf: Benchmarks for Data-Centric AI Development." *Advances in Neural Information Processing Systems* 36.
- Mazumder, Mark, Sharad Chitlangia, Colby Banbury, Yiping Kang, Juan Manuel Ciro, Keith Achorn, Daniel Galvez, et al. 2021. "Multilingual Spoken Words Corpus." *Advances in Neural Information Processing Systems 35 (NeurIPS 2021) Datasets and Benchmarks Track*.
- McCarthy, John, Marvin L. Minsky, Nathaniel Rochester, and Claude E. Shannon. 1955. *A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence, August 31, 1955*. No. 4. Vol. 27. Dartmouth College. <https://doi.org/10.1609/aimag.v27i4.1904>.
- McCrory, Dave. 2010. *Data Gravity – in the Clouds*.
- McCulloch, Warren S., and Walter Pitts. 1943. "A Logical Calculus of the Ideas Immanent in Nervous Activity." *The Bulletin of Mathematical Biophysics* 5 (4): 115–33. <https://doi.org/10.1007/bf02478259>.
- McKinsey Global Institute. 2021. *The Internet of Things: Catching up to an Accelerating Opportunity*. McKinsey & Company.
- McMahan, Brendan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. 2017. "Communication-Efficient Learning of Deep Networks from Decentralized Data." *International Conference on Artificial Intelligence and Statistics (AISTATS)*, Proceedings of machine learning research, vol. 54: 1273–82.
- Mellempudi, Naveen, Sudarshan Srinivasan, Dipankar Das, and Bharat Kaul. 2019. "Mixed Precision Training with 8-Bit Floating Point." *arXiv Preprint arXiv:1905.12334*.
- Merity, Stephen. 2016. *The WikiText Long Term Dependency Language Modeling Dataset*.
- Merity, Stephen, Caiming Xiong, James Bradbury, and Richard Socher. 2016. "Pointer Sentinel Mixture Models." *arXiv Preprint arXiv:1609.07843*.

- Merkel, Dirk. 2014. "Docker: Lightweight Linux Containers for Consistent Development and Deployment." *Linux Journal* 2014 (239).
- Michel, Jean-Baptiste, Yuan Kui Shen, Aviva Presser Aiden, Adrian Veres, Matthew K. Gray, William Brockman, Joseph P. Pickett, et al. 2011. "Quantitative Analysis of Culture Using Millions of Digitized Books." *Science* 331 (6014): 176–82. <https://doi.org/10.1126/science.1199644>.
- Michel, Paul, Omer Levy, and Graham Neumann. 2019. "Are Sixteen Heads Really Better Than One?" *Advances in Neural Information Processing Systems (NeurIPS)*.
- Micikevicius, Paulius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, et al. 2017. "Mixed Precision Training." *arXiv Preprint arXiv:1710.03740*.
- Micikevicius, Paulius, Dusan Stolic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, et al. 2022. "FP8 Formats for Deep Learning." *arXiv Preprint arXiv:2209.05433*.
- Microsoft. 2024a. *Microsoft Azure*.
- Microsoft. 2024b. *ONNX Runtime: Cross-Platform Inference and Training Machine-Learning Accelerator*. GitHub.
- Mikolov, Tomas, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. "Efficient Estimation of Word Representations in Vector Space." *ICLR abs/1301.3781*.
- Mila. 2026. *Open Source Software*.
- Minsky, Marvin, and Seymour A. Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. The MIT Press.
- Mirhoseini, Azalia, Hieu Pham, Quoc V. Le, Benoit Steiner, Rasmus Larsen, Yuefeng Zhou, Naveen Kumar, Mohammad Norouzi, Samy Bengio, and Jeff Dean. 2017. "Device Placement Optimization with Reinforcement Learning." *International Conference on Machine Learning (ICML)* 70: 2430–39.
- Mitchell, Margaret, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. 2019. "Model Cards for Model Reporting." *Proceedings of the Conference on Fairness, Accountability, and Transparency*, 220–29. <https://doi.org/10.1145/3287560.3287596>.
- Mitchell, Tom M. 1980. *The Need for Biases in Learning Generalizations*. CBM-TR-117. Rutgers University, Department of Computer Science.
- MLCommons. 2024a. *MLPerf Mobile Benchmark*.
- MLCommons. 2024b. *MLPerf Power Measurement*.
- MLCommons. 2024c. *MLPerf Training Benchmark*.
- MLCommons. 2026a. *MLCommons: About Us*.
- MLCommons. 2026b. *MLPerf Client Benchmark*.
- MLflow Project. 2026. *MLflow Model Registry*.
- Moore, G. E. 1998. "Cramming More Components onto Integrated Circuits." *Proceedings of the IEEE* 86 (1): 82–85. <https://doi.org/10.1109/jproc.1998.658762>.
- Moravec, Hans. 1988. *Mind Children: The Future of Robot and Human Intelligence*. Harvard University Press.
- Moritz, Philipp, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, et al. 2018. "Ray: A Distributed Framework for Emerging AI Applications." *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 561–77.
- Mosseri, Adam. 2018. *Bringing People Closer Together*. Meta Newsroom.
- Murray, Derek G., Jiří Šimša, Ana Klimovic, and Ihor Indyk. 2021. "Tf.data: A Machine Learning Data Processing Framework." *Proceedings of the VLDB Endowment* 14 (12): 2945–58. <https://doi.org/10.14778/3476311.3476374>.
- Nair, Vinod, and Geoffrey E. Hinton. 2010. "Rectified Linear Units Improve Restricted Boltzmann Machines." *Proceedings of the 27th International Conference on Machine Learning (ICML-10)*, 807–14.
- Nakkiran, Preetum, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, and Ilya Sutskever. 2019. "Deep Double Descent: Where Bigger Models and More Data Hurt." *arXiv Preprint arXiv:1912.02292*.
- Narayanan, Deepak, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. 2019. "PipeDream: Generalized Pipeline Parallelism for DNN Training." *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, 1–15. <https://doi.org/10.1145/3341301.3359646>.

- Narayanan, Deepak, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, et al. 2021. "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM." *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 1–15. <https://doi.org/10.1145/3458817.3476209>.
- National Aeronautics and Space Administration. 2016. *NASA Systems Engineering Handbook*. NASA/SP-2016-6105 Rev2. National Aeronautics; Space Administration.
- National Transportation Safety Board. 2017. *Collision Between a Car Operating with Automated Vehicle Control Systems and a Tractor-Semitrailer Truck Near Williston, Florida, May 7, 2016*. HAR-17/02. National Transportation Safety Board.
- National Transportation Safety Board. 2019. *Collision Between Vehicle Controlled by Developmental Automated Driving System and Pedestrian, Tempe, Arizona, March 18, 2018*. HAR-19/03. National Transportation Safety Board.
- Naumov, Maxim, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, et al. 2019. "Deep Learning Recommendation Model for Personalization and Recommendation Systems." *arXiv Preprint arXiv:1906.00091*.
- Naur, Peter, and Brian Randell, eds. 1969. *Software Engineering: Report of a Conference Sponsored by the NATO Science Committee, Garmisch, Germany, 7–11 October 1968*. Scientific Affairs Division, NATO.
- Nayak, Prateeth, Takuya Higuchi, Anmol Gupta, Shivesh Ranjan, Stephen Shum, Siddharth Sigtia, Erik Marchi, et al. 2022. "Improving Voice Trigger Detection with Metric Learning." *Interspeech 2022*, 1896–900. <https://doi.org/10.21437/interspeech.2022-11160>.
- Netflix. 2024. *Metaflow*.
- Newell, Allen, and Herbert A. Simon. 1976. "Computer Science as Empirical Inquiry: Symbols and Search." *Communications of the ACM* 19 (3): 113–26. <https://doi.org/10.1145/360018.360022>.
- Neyshabur, Behnam, Srinadh Bhojanapalli, David McAllester, and Nati Srebro. 2017. "Exploring Generalization in Deep Learning." *Advances in Neural Information Processing Systems* 30.
- Ng, Andrew. 2021. *MLOps: From Model-Centric to Data-Centric AI*. DeepLearning.AI.
- Nickolls, John, Ian Buck, Michael Garland, and Kevin Skadron. 2008. "Scalable Parallel Programming with CUDA: Is CUDA the Parallel Programming Model That Application Developers Have Been Waiting For?" *Queue* 6 (2): 40–53. <https://doi.org/10.1145/1365490.1365500>.
- Norrie, Thomas, Nishant Patil, Doe Hyun Yoon, George Kurian, Sheng Li, James Laudon, Cliff Young, Norman Jouppi, and David Patterson. 2021. "The Design Process for Google's Training Chips: TPuv2 and TPuv3." *IEEE Micro* 41 (2): 56–63. <https://doi.org/10.1109/mm.2021.3058217>.
- Northcutt, Curtis G., Anish Athalye, and Jonas Mueller. 2021. "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks." *arXiv Preprint arXiv:2103.14749* 34: 19075–90. <https://doi.org/10.48550/arxiv.2103.14749>.
- NVIDIA. 2017. *Training with Mixed Precision*.
- NVIDIA. 2020a. *Accelerating Matrix Multiplication with Block Sparse Format and NVIDIA Tensor Cores*.
- NVIDIA. 2020b. *TensorFloat-32 in the A100 GPU Accelerates AI Training, HPC up to 20x*.
- NVIDIA. 2024a. *cuBLAS: CUDA Basic Linear Algebra Subprograms*.
- NVIDIA. 2024b. *NVIDIA Omniverse and Simulation*.
- NVIDIA. 2024c. *NVIDIA TensorRT: Programmable Inference Accelerator*.
- NVIDIA. 2024d. *NVIDIA Triton Inference Server*.
- NVIDIA. 2026a. *Lazy Loading: CUDA Programming Guide*.
- NVIDIA. 2026b. *Model Management: NVIDIA Triton Inference Server*.
- NVIDIA. 2026c. *Model Repository: NVIDIA Triton Inference Server*.
- NVIDIA. 2026d. *NVIDIA CUDA Multi-Process Service*.
- NVIDIA. 2026e. *NVIDIA Multi-Instance GPU User Guide*.
- NVIDIA. 2026f. *NVIDIA Quantum-X800 InfiniBand Platform*. NVIDIA product documentation.
- NVIDIA. 2026g. *NVIDIA TensorRT-LLM*.
- NVIDIA Corporation. 2017. *NVIDIA Tesla V100 GPU Architecture*. NVIDIA Whitepaper.

- NVIDIA Corporation. 2018. *NVIDIA Tesla T4 Tensor Core GPU*. NVIDIA product documentation.
- NVIDIA Corporation. 2020a. *NVIDIA A100 Tensor Core GPU Architecture*. NVIDIA Whitepaper, V1.0.
- NVIDIA Corporation. 2020b. *NVIDIA V100 Tensor Core GPU Datasheet*. NVIDIA product documentation.
- NVIDIA Corporation. 2020c. “NVLink: Scalable High-Performance Interconnect.” *NVIDIA Technical Report* 2020.
- NVIDIA Corporation. 2021a. *NVIDIA A100 Tensor Core GPU Datasheet*. NVIDIA product documentation.
- NVIDIA Corporation. 2021b. *NVIDIA cuDNN Developer Guide*.
- NVIDIA Corporation. 2023. *NVIDIA cuDNN Developer Guide, Version 8.9.6*.
- NVIDIA Corporation. 2024. *NVIDIA Blackwell Architecture*. NVIDIA product documentation.
- NVIDIA Corporation. 2026a. *How TensorRT Works*.
- NVIDIA Corporation. 2026b. *Working with Dynamic Shapes*.
- Oakden-Rayner, Luke, Jared Dunnmon, Gustavo Carneiro, and Christopher Re. 2020. “Hidden Stratification Causes Clinically Meaningful Failures in Machine Learning for Medical Imaging.” *Proceedings of the ACM Conference on Health, Inference, and Learning*, 151–59. <https://doi.org/10.1145/3368555.3384468>.
- Obermeyer, Ziad, Brian Powers, Christine Vogeli, and Sendhil Mullainathan. 2019. “Dissecting Racial Bias in an Algorithm Used to Manage the Health of Populations.” *Science* 366 (6464): 447–53. <https://doi.org/10.1126/science.aax2342>.
- Olston, Christopher, Noah Fiedel, Kiril Gorovoy, Jeremiah Harmsen, Li Lao, Fangwei Li, Vinu Rajashekhar, Sukriti Ramesh, and Jordan Soyke. 2017. “TensorFlow-Serving: Flexible, High-Performance ML Serving.” *CoRR* abs/1712.06139. <https://doi.org/10.48550/arXiv.1712.06139>.
- ONNX Contributors. 2019. *ONNX: Open Neural Network Exchange*.
- OpenAI. 2023a. *GPT-4*.
- OpenAI. 2023b. *Introducing ChatGPT and Whisper APIs*.
- OpenAI. 2023c. *New Models and Developer Products Announced at DevDay*.
- OpenAI. 2024. *GPT-4o Mini: Advancing Cost-Efficient Intelligence*.
- OpenAI Developer Community. 2024. *Announcing GPT-4o in the API*.
- OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, et al. 2023. “GPT-4 Technical Report.” *arXiv Preprint arXiv:2303.08774*, ahead of print. <https://doi.org/10.48550/arXiv.2303.08774>.
- OpenBLAS Project. 2026. *OpenBLAS: An Optimized BLAS Library*.
- Orr, Laurel, Atindriyo Sanyal, Xiao Ling, Karan Goel, and Megan Leszczynski. 2021. “Managing ML Pipelines: Feature Stores and the Coming Wave of Embedding Ecosystems.” *Proceedings of the VLDB Endowment* 14 (12): 3178–81. <https://doi.org/10.14778/3476311.3476402>.
- Owens, John D., Mike Houston, David Luebke, Simon Green, John E. Stone, and James C. Phillips. 2008. “GPU Computing.” *Proceedings of the IEEE* 96 (5): 879–99. <https://doi.org/10.1109/jproc.2008.917757>.
- Paley, Andrei, Raoul-Gabriel Urma, and Neil D. Lawrence. 2022. “Challenges in Deploying Machine Learning: A Survey of Case Studies.” *ACM Computing Surveys* 55 (6): 1–29. <https://doi.org/10.1145/3533378>.
- Palmer, John F. 1980. “The Intel 8087 Numeric Data Processor.” *Proceedings of the 1980 National Computer Conference (AFIPS)*, 887–93. <https://doi.org/10.1145/1500518.1500674>.
- Papermill Project. 2026. *Papermill Documentation*.
- Papineni, Kishore, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. “Bleu: A Method for Automatic Evaluation of Machine Translation.” *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics - ACL '02*, 311–18. <https://doi.org/10.3115/1073083.1073135>.
- Pareto, Vilfredo. 1896. *Cours d'économie Politique*. F. Rouge.
- Park, Daniel S., William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin D. Cubuk, and Quoc V. Le. 2019. “SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition.” *Interspeech 2019*, 2613–17. <https://doi.org/10.21437/interspeech.2019-2680>.
- Paszke, Adam, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, et al. 2019. “PyTorch: An Imperative Style, High-Performance Deep Learning Library.” *Advances in Neural Information Processing Systems (NeurIPS)* 32: 8024–35.

- Patel, Dylan, and Gerald Wong. 2023. *GPT-4 Architecture, Infrastructure, Training Dataset, Costs, Vision, MoE*. SemiAnalysis Blog.
- Patterson, David, Joseph Gonzalez, Quoc Le, Chen Liang, Lluís-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. 2021. "Carbon Emissions and Large Neural Network Training." *arXiv Preprint arXiv:2104.10350*.
- Paul, Mansheej, Surya Ganguli, and Gintare Karolina Dziugaite. 2021. "Deep Learning on a Data Diet: Finding Important Examples Early in Training." *Advances in Neural Information Processing Systems (NeurIPS)* 34: 20596–607.
- Pham, Hieu, Melody Y. Guan, Barret Zoph, Quoc V. Le, and Jeff Dean. 2018. "Efficient Neural Architecture Search via Parameter Sharing." *Proceedings of the 35th International Conference on Machine Learning (ICML)*, Proceedings of machine learning research, vol. 80: 4095–104.
- Pichai, Sundar. 2023. *Introducing Gemini: Our Largest and Most Capable AI Model*.
- Pineau, Joelle, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d'Alché-Buc, Emily Fox, and Hugo Larochelle. 2021. "Improving Reproducibility in Machine Learning Research (a Report from the NeurIPS 2019 Reproducibility Program)." *Journal of Machine Learning Research* 22 (164): 1–20.
- Pope, Reiner, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. "Efficiently Scaling Transformer Inference." *Proceedings of Machine Learning and Systems (MLSys)* 5: 606–24.
- Prefect Technologies, Inc. 2024. *Prefect: Workflow Orchestration Framework for Python*.
- Project Jupyter. 2024. *Project Jupyter*.
- Project Jupyter. 2026. *nbconvert: Convert Notebooks to Other Formats*.
- Protocol Buffers Authors. 2026. *Overview: Protocol Buffers*.
- Pushkarna, Mahima, Andrew Zaldivar, and Oddur Kjartansson. 2022. "Data Cards: Purposeful and Transparent Dataset Documentation for Responsible AI." *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 1776–826. <https://doi.org/10.1145/3531146.3533231>.
- Putnam, Andrew, Adrian M. Caulfield, Eric S. Chung, Derek Chiou, Kypros Constantinides, John Demme, Hadi Esmaeilzadeh, et al. 2014. "A Reconfigurable Fabric for Accelerating Large-Scale Datacenter Services." *ACM SIGARCH Computer Architecture News* 42 (3): 13–24. <https://doi.org/10.1145/2678373.2665678>.
- PyTorch. 2022. *Accelerating Neural Network Training with Sparse Tensors*.
- PyTorch Contributors. 2026a. *PyTorch Autograd Mechanics*.
- PyTorch Contributors. 2026b. *Tensor Views*.
- PyTorch Contributors. 2026c. *torch.export*.
- PyTorch Contributors. 2026d. *TorchScript*.
- Qi, Chen, Shibo Shen, Rongpeng Li, Zhifeng Zhao, Qing Liu, Jing Liang, and Honggang Zhang. 2021. "An Efficient Pruning Scheme of Deep Neural Networks for Internet of Things Applications." *EURASIP Journal on Advances in Signal Processing* 2021 (1): 31. <https://doi.org/10.1186/s13634-021-00744-4>.
- Quiñero-Candela, Joaquin, Masashi Sugiyama, Anton Schwaighofer, and Neil D. Lawrence, eds. 2009. *Dataset Shift in Machine Learning*. Neural Information Processing Series. The MIT Press.
- Rachwan, John, Daniel Zügner, Bertrand Charpentier, Simon Geisler, Morgane Ayle, and Stephan Günnemann. 2022. "Winning the Lottery Ahead of Time: Efficient Early Network Pruning." *International Conference on Machine Learning (ICML)*, 18293–309.
- Radford, Alec, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, et al. 2021. "Learning Transferable Visual Models from Natural Language Supervision." *International Conference on Machine Learning (ICML)*, 8748–63.
- Radford, Alec, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. *Language Models Are Unsupervised Multitask Learners*. OpenAI.
- Raffel, C., N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. 2020. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer." *Journal of Machine Learning Research* 21 (140): 1–67.
- Raichle, Marcus E., and Debra A. Gusnard. 2002. "Appraising the Brain's Energy Budget." *Proceedings of the National Academy of Sciences* 99 (16): 10237–39. <https://doi.org/10.1073/pnas.172399499>.
- Rainio, Oona, Jarmo Teuho, and Riku Klén. 2024. "Evaluation Metrics and Statistical Tests for Machine Learning." *Scientific Reports* 14 (1): 6086. <https://doi.org/10.1038/s41598-024-56706-x>.

- Raji, Inioluwa Deborah, and Joy Buolamwini. 2019. "Actionable Auditing: Investigating the Impact of Publicly Naming Biased Performance Results of Commercial AI Products." *Proceedings of the 2019 AAAI/ACM Conference on AI, Ethics, and Society*, 429–35. <https://doi.org/10.1145/3306618.3314244>.
- Rajpurkar, Pranav, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. "SQuAD: 100,000+ Questions for Machine Comprehension of Text." *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, 2383–92. <https://doi.org/10.18653/v1/d16-1264>.
- Ranjit, Mercy Prasanna, Gopinath Ganapathy, Kalaivani Sridhar, and Vikram Arumugham. 2019. "Efficient Deep Learning Hyperparameter Tuning Using Cloud Infrastructure: Intelligent Distributed Hyperparameter Tuning with Bayesian Optimization in the Cloud." *2019 IEEE 12th International Conference on Cloud Computing (CLOUD)*, 520–22. <https://doi.org/10.1109/cloud.2019.00097>.
- Rastegari, Mohammad, Vicente Ordonez, Joseph Redmon, and Ali Farhadi. 2016. "XNOR-Net: ImageNet Classification Using Binary Convolutional Neural Networks." In *European Conference on Computer Vision (ECCV)*. Springer. https://doi.org/10.1007/978-3-319-46493-0_32.
- Ratner, A., S. H. Bach, H. Ehrenberg, J. Fries, S. Wu, and C. Ré. 2017. "Snorkel: Rapid Training Data Creation with Weak Supervision." *Proceedings of the VLDB Endowment* 11 (3): 269–82. <https://doi.org/10.14778/3157794.3157797>.
- Ratner, Alex, Braden Hancock, Jared Dunnmon, Roger Goldman, and Christopher Ré. 2018. "Snorkel MeTaL: Weak Supervision for Multi-Task Learning." *Proceedings of the Second Workshop on Data Management for End-to-End Machine Learning*, 1–4. <https://doi.org/10.1145/3209889.3209898>.
- Ratner, Alex, Paroma Varma, Braden Hancock, and Christopher Ré. 2019. *Weak Supervision: A New Programming Paradigm for Machine Learning*. Stanford AI Lab Blog.
- Reagen, Brandon, Robert Adolf, Paul Whatmough, Gu-Yeon Wei, and David Brooks. 2017. *Deep Learning for Computer Architects. Synthesis Lectures on Computer Architecture*. Springer International Publishing. <https://doi.org/10.1007/978-3-031-01756-8>.
- Real, Esteban, Alok Aggarwal, Yanping Huang, and Quoc V. Le. 2019. "Regularized Evolution for Image Classifier Architecture Search." *Proceedings of the AAAI Conference on Artificial Intelligence* 33 (01): 4780–89. <https://doi.org/10.1609/aaai.v33i01.33014780>.
- Recht, Benjamin, Rebecca Roelofs, Ludwig Schmidt, and Vaishal Shankar. 2019. "Do ImageNet Classifiers Generalize to ImageNet?" *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 5389–400.
- Reddi, Vijay Janapa, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, et al. 2019. "MLPerf Inference Benchmark." *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 446–59. <https://doi.org/10.1109/isca45697.2020.00045>.
- Ren, Pengzhen, Yun Xiao, Xiaojun Chang, Po-Yao Huang, Zhihui Li, Brij B. Gupta, Xiaojiang Chen, and Xin Wang. 2021. "A Survey of Deep Active Learning." *ACM Computing Surveys* 54 (9): 1–40. <https://doi.org/10.1145/3472291>.
- Ribeiro, M. H., R. Ottoni, R. West, V. A. F. Almeida, and Jr. Meira Wagner. 2020. "Auditing Radicalization Pathways on YouTube." *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, 131–41. <https://doi.org/10.1145/3351095.3372879>.
- Ribeiro, Marco Tulio, Sameer Singh, and Carlos Guestrin. 2016. "Why Should I Trust You?: Explaining the Predictions of Any Classifier." *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1135–44. <https://doi.org/10.1145/2939672.2939778>.
- Ribeiro, Marco Tulio, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. 2020. "Beyond Accuracy: Behavioral Testing of NLP Models with CheckList." *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 4902–12. <https://doi.org/10.18653/v1/2020.acl-main.442>.
- Roesch, Jared, Steven Lyubomirsky, Logan Weber, Josh Pollock, Marisa Kirisame, Tianqi Chen, and Zachary Tatlock. 2018. "Relay: A New IR for Machine Learning Frameworks." *Proceedings of the 2nd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 58–68. <https://doi.org/10.1145/3211346.3211348>.
- Rombach, Robin, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Bjorn Ommer. 2022. "High-Resolution Image Synthesis with Latent Diffusion Models." *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 10674–85. <https://doi.org/10.1109/cvpr52688.2022.01042>.
- Romero, Francisco, Qian Li, Neeraja J. Yadwadkar, and Christos Kozyrakis. 2021. "INFaaS: Automated Model-Less Inference Serving." *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, 397–411.
- Rosenblatt, Frank. 1957. *The Perceptron: A Perceiving and Recognizing Automaton*. Report Nos. 85-460-1. Cornell Aeronautical Laboratory.
- Rosenblatt, Frank. 1958. "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain." *Psychological Review* 65 (6): 386–408. <https://doi.org/10.1037/h0042519>.
- Royce, Winston W. 1970. "Managing the Development of Large Software Systems." *Proceedings of IEEE WESCON* 26: 328–88.

- Rumelhart, David E., Geoffrey E. Hinton, and Ronald J. Williams. 1986. "Learning Representations by Back-Propagating Errors." *Nature* 323 (6088): 533–36. <https://doi.org/10.1038/323533a0>.
- Russakovsky, Olga, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, et al. 2015. "ImageNet Large Scale Visual Recognition Challenge." *International Journal of Computer Vision* 115 (3): 211–52. <https://doi.org/10.1007/s11263-015-0816-y>.
- Sabour, Sara, Nicholas Frosst, and Geoffrey E Hinton. 2017. "Dynamic Routing Between Capsules." *Advances in Neural Information Processing Systems (NeurIPS)* 30.
- Sambasivan, Nithya, Shivani Kapania, Hannah Highfill, Diana Akrong, Praveen Paritosh, and Lora M Aroyo. 2021. "'Everyone Wants to Do the Model Work, Not the Data Work': Data Cascades in High-Stakes AI." *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, 1–15. <https://doi.org/10.1145/3411764.3445518>.
- Samuel, A. L. 1959. "Some Studies in Machine Learning Using the Game of Checkers." *IBM Journal of Research and Development* 3 (3): 210–29. <https://doi.org/10.1147/rd.33.0210>.
- Sandler, Mark, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. 2018. "MobileNetV2: Inverted Residuals and Linear Bottlenecks." *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 4510–20. <https://doi.org/10.1109/cvpr.2018.00474>.
- Sanh, Victor, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. "DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter." *arXiv Preprint arXiv:1910.01108*, ahead of print. <https://doi.org/10.48550/arXiv.1910.01108>.
- Savage, Sam L. 2009. *The Flaw of Averages: Why We Underestimate Risk in the Face of Uncertainty*. John Wiley & Sons.
- Scale AI, Inc. 2024. *Scale AI Advanced Settings*.
- Schelter, Sebastian, Matthias Boehm, Johannes Kirschnick, Kostas Tzoumas, and Gunnar Ratsch. 2018. "Automating Large-Scale Machine Learning Model Management." *Proceedings of the 2018 IEEE International Conference on Data Engineering (ICDE)*, 137–48.
- Schwartz, Roy, Jesse Dodge, Noah A. Smith, and Oren Etzioni. 2020. "Green AI." *Communications of the ACM* 63 (12): 54–63. <https://doi.org/10.1145/3381831>.
- scikit-learn developers. 2024a. *Feature Selection — Scikit-Learn Documentation*. Scikit-learn Documentation.
- scikit-learn developers. 2024b. *Sklearn.metrics.confusion_matrix — Scikit-Learn Documentation*. Scikit-learn Documentation.
- Scott, Colin. 2012. "Numbers Every Programmer Should Know by Year."
- Sculley, D., Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. 2015. "Hidden Technical Debt in Machine Learning Systems." *Advances in Neural Information Processing Systems (NeurIPS)* 28: 2503–11.
- Sener, Ozan, and Silvio Savarese. 2018. "Active Learning for Convolutional Neural Networks: A Core-Set Approach." *International Conference on Learning Representations (ICLR)*.
- Sennrich, Rico, Barry Haddow, and Alexandra Birch. 2016. "Improving Neural Machine Translation Models with Monolingual Data." *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 86–96. <https://doi.org/10.18653/v1/P16-1009>.
- Sergeev, Alexander, and Mike Del Balso. 2018. "Horovod: Fast and Easy Distributed Deep Learning in TensorFlow." *CoRR* abs/1802.05799.
- Settles, Burr. 2009. *Active Learning Literature Survey*. Computer Sciences Technical Report 1648. University of Wisconsin-Madison.
- Settles, Burr. 2012. *Active Learning*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Springer Cham. <https://doi.org/10.1007/978-3-031-01560-1>.
- Sevilla, Jaime, Lennart Heim, Anson Ho, Tamay Besiroglu, Marius Hobbhahn, and Pablo Villalobos. 2022. "Compute Trends Across Three Eras of Machine Learning." *2022 International Joint Conference on Neural Networks (IJCNN)*, 1–8. <https://doi.org/10.1109/ijcnn55064.2022.9891914>.
- Shah, Jay, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-Precision." *Advances in Neural Information Processing Systems* 37 (NeurIPS), 68658–85. <https://doi.org/10.52202/079017-2193>.
- Shannon, Claude E. 1948. "A Mathematical Theory of Communication." *Bell System Technical Journal* 27 (3): 379–423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.
- Shazeer, Noam, Azalia Mirhoseini, Krzysztof Maziarsz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer." *International Conference on Learning Representations (ICLR)*.

- Shazeer, Noam, and Mitchell Stern. 2018. "Adafactor: Adaptive Learning Rates with Sublinear Memory Cost." *International Conference on Machine Learning (ICML)*, Proceedings of machine learning research, vol. 80: 4596–604.
- Shen, Haichen, Lequn Chen, Yuchen Jin, Liangyu Zhao, Bingyu Kong, Matthai Philipose, Arvind Krishnamurthy, and Ravi Sundaram. 2019. "Nexus: A GPU Cluster Engine for Accelerating DNN-Based Video Analysis." *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, 322–37. <https://doi.org/10.1145/3341301.3359658>.
- Shen, Sheng, Zhen Dong, Jiayu Ye, Linjian Ma, Zhewei Yao, Amir Gholami, Michael W. Mahoney, and Kurt Keutzer. 2020. "Q-BERT: Hessian Based Ultra Low Precision Quantization of BERT." *Proceedings of the AAAI Conference on Artificial Intelligence* 34 (05): 8815–21. <https://doi.org/10.1609/aaai.v34i05.6409>.
- Sheng, Victor S., and Jing Zhang. 2019. "Machine Learning with Crowdsourcing: A Brief Summary of the Past Research and Future Directions." *Proceedings of the AAAI Conference on Artificial Intelligence* 33 (01): 9837–43. <https://doi.org/10.1609/aaai.v33i01.33019837>.
- Shi, Weisong, Jie Cao, Quan Zhang, Youhuizi Li, and Lanyu Xu. 2016. "Edge Computing: Vision and Challenges." *IEEE Internet of Things Journal* 3 (5): 637–46. <https://doi.org/10.1109/jiot.2016.2579198>.
- Shneiderman, B. 2022. *Human-Centered AI*. Oxford University Press. <https://doi.org/10.1093/oso/9780192845290.001.0001>.
- Shoeybi, Mohammad, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism." *arXiv Preprint arXiv:1909.08053*.
- Shokri, Reza, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. 2017. "Membership Inference Attacks Against Machine Learning Models." *2017 IEEE Symposium on Security and Privacy (SP)*, 3–18. <https://doi.org/10.1109/SP.2017.41>.
- Shorten, Connor, and Taghi M. Khoshgoftaar. 2019. "A Survey on Image Data Augmentation for Deep Learning." *Journal of Big Data* 6 (1): 1–48. <https://doi.org/10.1186/s40537-019-0197-0>.
- Shortliffe, Edward H., Randall Davis, Stanton G. Axline, Bruce G. Buchanan, C.Cordell Green, and Stanley N. Cohen. 1975. "Computer-Based Consultations in Clinical Therapeutics: Explanation and Rule Acquisition Capabilities of the MYCIN System." *Computers and Biomedical Research* 8 (4): 303–20. [https://doi.org/10.1016/0010-4809\(75\)90009-9](https://doi.org/10.1016/0010-4809(75)90009-9).
- Shumailov, Ilia, Zakhar Shumaylov, Yiren Zhao, Nicolas Papernot, Ross Anderson, and Yarin Gal. 2024. "AI Models Collapse When Trained on Recursively Generated Data." *Nature* 631 (8022): 755–59. <https://doi.org/10.1038/s41586-024-07566-y>.
- Sifre, Laurent, and Stéphane Mallat. 2014. "Rigid-Motion Scattering for Image Classification." *arXiv Preprint arXiv:1403.1687*, ahead of print. <https://doi.org/10.48550/arXiv.1403.1687>.
- Silver, David, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, et al. 2016. "Mastering the Game of Go with Deep Neural Networks and Tree Search." *Nature* 529 (7587): 484–89. <https://doi.org/10.1038/nature16961>.
- Silver, David, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, et al. 2017. "Mastering the Game of Go Without Human Knowledge." *Nature* 550 (7676): 354–59. <https://doi.org/10.1038/nature24270>.
- Simonyan, Karen, and Andrew Zisserman. 2015. "Very Deep Convolutional Networks for Large-Scale Image Recognition." *arXiv Preprint*.
- Smith, Steven W. 1997. "Digital Signal Processing." In *Digital Signal Processing Demystified*. Elsevier. <https://doi.org/10.1016/b978-187870716-1/50004-4>.
- Snow, Rion, Brendan O'Connor, Daniel Jurafsky, and Andrew Y. Ng. 2008. "Cheap and Fast—but Is It Good? Evaluating Non-Expert Annotations for Natural Language Tasks." *Proceedings of the Conference on Empirical Methods in Natural Language Processing - EMNLP '08*, 254. <https://doi.org/10.3115/1613715.1613751>.
- Sohn, Kihyuk, David Berthelot, Nicholas Carlini, Zizhao Zhang, Han Zhang, Colin A Raffel, Ekin Dogus Cubuk, Alexey Kurakin, and Chun-Liang Li. 2020. "FixMatch: Simplifying Semi-Supervised Learning with Consistency and Confidence." *Advances in Neural Information Processing Systems (NeurIPS)* 33: 596–608.
- Sokolova, Marina, and Guy Lapalme. 2009. "A Systematic Analysis of Performance Measures for Classification Tasks." *Information Processing & Management* 45 (4): 427–37. <https://doi.org/10.1016/j.ipm.2009.03.002>.
- Sorscher, Ben, Robert Geirhos, Shashank Shekhar, Surya Ganguli, and Ari S. Morcos. 2022. "Beyond Neural Scaling Laws: Beating Power Law Scaling via Data Pruning." *Advances in Neural Information Processing Systems (NeurIPS)* 35: 19523–36. <https://doi.org/10.52202/068431-1419>.
- Soviany, Petru, Radu Tudor Ionescu, Paolo Rota, and Nicu Sebe. 2022. "Curriculum Learning: A Survey." *International Journal of Computer Vision* 130 (6): 1526–65. <https://doi.org/10.1007/s11263-022-01611-x>.
- Srivastava, Nitish, Geoffrey E. Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. 2014. "Dropout: A Simple Way to Prevent Neural Networks from Overfitting." *Journal of Machine Learning Research* 15 (1): 1929–58.
- Statista Research Department. 2024. *Number of Sent and Received e-Mails Per Day Worldwide from 2017 to 2027*. Statista.

- Stephens, Nigel, Stuart Biles, Matthias Boettcher, Jacob Eapen, Mbou Eyole, Giacomo Gabrielli, Matt Horsnell, et al. 2017. "The ARM Scalable Vector Extension." *IEEE Micro* 37 (2): 26–39. <https://doi.org/10.1109/mm.2017.35>.
- Stonebraker, Mike, Daniel J. Abadi, Adam Batkin, Xuedong Chen, Mitch Cherniack, Miguel Ferreira, Edmond Lau, et al. 2018. "C-Store: A Column-Oriented DBMS." In *Making Databases Work: The Pragmatic Wisdom of Michael Stonebraker*. Association for Computing Machinery. <https://doi.org/10.1145/3226595.3226638>.
- Strassen, Volker. 1969. "Gaussian Elimination Is Not Optimal." *Numerische Mathematik* 13 (4): 354–56. <https://doi.org/10.1007/bf02165411>.
- Strathern, Marilyn. 1997. "'Improving Ratings': Audit in the British University System." *European Review* 5 (3): 305–21. [https://doi.org/10.1002/\(SICI\)1234-981X\(199707\)5:3<305::AID-EURO184>3.0.CO;2-4](https://doi.org/10.1002/(SICI)1234-981X(199707)5:3<305::AID-EURO184>3.0.CO;2-4).
- Stromberg, Joseph. 2015. *When Was the First Traffic Light Installed? Today in 1914*. Vox.
- Strubell, Emma, Ananya Ganesh, and Andrew McCallum. 2019. "Energy and Policy Considerations for Deep Learning in NLP." *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 3645–50. <https://doi.org/10.18653/v1/p19-1355>.
- Sullivan, Gary J., Jens-Rainer Ohm, Woo-Jin Han, and Thomas Wiegand. 2012. "Overview of the High Efficiency Video Coding (HEVC) Standard." *IEEE Transactions on Circuits and Systems for Video Technology* 22 (12): 1649–68. <https://doi.org/10.1109/tcsvt.2012.2221191>.
- Sun, Pei, Henrik Kretschmar, Xerxes Dotiwalla, Aurelien Chouard, Vijaysai Patnaik, Paul Tsui, James Guo, et al. 2020. "Scalability in Perception for Autonomous Driving: Waymo Open Dataset." *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2446–54. <https://doi.org/10.1109/CVPR42600.2020.00252>.
- Supreme Court of the United States. 1971. *Griggs v. Duke Power Co.*, 401 U.S. 424. Legal decision.
- Sutskever, Ilya, Oriol Vinyals, and Quoc V. Le. 2014. "Sequence to Sequence Learning with Neural Networks." *Advances in Neural Information Processing Systems (NeurIPS)* 27: 3104–12.
- Sutton, Richard S. 2019. "The Bitter Lesson." *Incompleteideas.net* 43.
- Sweeney, Latanya. 2002. "K-ANONYMITY: A MODEL for PROTECTING PRIVACY." *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems* 10 (05): 557–70. <https://doi.org/10.1142/s0218488502001648>.
- Systems, Cerebras. 2021. "Wafer-Scale Deep Learning Acceleration with the Cerebras CS-2." *Cerebras Technical Paper* 2021.
- Sze, Vivienne, Yu-Hsin Chen, Tien-Ju Yang, and Joel S. Emer. 2017. "Efficient Processing of Deep Neural Networks: A Tutorial and Survey." *Proceedings of the IEEE* 105 (12): 2295–329. <https://doi.org/10.1109/jproc.2017.2761740>.
- Szegedy, Christian, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. 2015. "Going Deeper with Convolutions." *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1–9. <https://doi.org/10.1109/cvpr.2015.7298594>.
- Tan, Mingxing, Bo Chen, Ruoming Pang, Vijay Vasudevan, Mark Sandler, Andrew Howard, and Quoc V. Le. 2019. "MnasNet: Platform-Aware Neural Architecture Search for Mobile." *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2815–23. <https://doi.org/10.1109/cvpr.2019.00293>.
- Tan, Mingxing, and Quoc V. Le. 2019. "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks." *International Conference on Machine Learning (ICML)*, 6105–14.
- Team, The Theano Development, Rami Al-Rfou, Guillaume Alain, Amjad Almahairi, Christof Angermueller, Dzmitry Bahdanau, Nicolas Ballas, et al. 2016. "Theano: A Python Framework for Fast Computation of Mathematical Expressions." *arXiv Preprint*.
- Teerapittayanon, Surat, Bradley McDanel, and H. T. Kung. 2017. "BranchyNet: Fast Inference via Early Exiting from Deep Neural Networks." *2016 23rd International Conference on Pattern Recognition (ICPR)*, 2464–69. <https://doi.org/10.1109/icpr.2016.7900006>.
- Telgarsky, Matus. 2016. "Benefits of Depth in Neural Networks." *Proceedings of the 29th Annual Conference on Learning Theory*, 1517–39.
- TensorFlow Developers. 2024a. *Better Performance with tf.function*.
- TensorFlow Developers. 2024b. *Optimize TensorFlow Performance Using the Profiler*.
- Tesla, Inc. 2021. *Tesla Dojo Technology: A Guide to Tesla's Configurable Floating Point Formats & Arithmetic*. Tesla AI Day, Whitepaper.
- Tesler, Larry. 1984. *The Law of Conservation of Complexity*. Web page.
- Thyagarajan, Aditya, Eliás Snorrason, Curtis G. Northcutt, and Jonas Mueller. 2022. "Identifying Incorrect Annotations in Multi-Label Classification Data." *CoRR abs/2211.13895*. <https://doi.org/10.48550/ARXIV.2211.13895>.

- Tibshirani, Robert. 1996. "Regression Shrinkage and Selection via the Lasso." *Journal of the Royal Statistical Society: Series B (Methodological)* 58 (1): 267–88. <https://doi.org/10.1111/j.2517-6161.1996.tb02080.x>.
- Tillet, Philippe, H. T. Kung, and David Cox. 2019. "Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations." *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 10–19. <https://doi.org/10.1145/3315508.3329973>.
- Toneva, Mariya, Alessandro Sordoni, Remi Tachet des Combes, Adam Trischler, Yoshua Bengio, and Geoffrey J Gordon. 2019. "An Empirical Study of Example Forgetting During Deep Neural Network Learning." *International Conference on Learning Representations (ICLR)*.
- Torvalds, Linus, and Junio Hamano. 2024. *Git*.
- Touvron, Hugo, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, et al. 2023. "LLaMA: Open and Efficient Foundation Language Models." *arXiv Preprint arXiv:2302.13971*.
- Touvron, Hugo, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, et al. 2023. "Llama 2: Open Foundation and Fine-Tuned Chat Models." *arXiv Preprint arXiv:2307.09288*.
- Tramèr, Florian, Fan Zhang, Ari Juels, Michael K Reiter, and Thomas Ristenpart. 2016. "Stealing Machine Learning Models via Prediction APIs." *25th USENIX Security Symposium (USENIX Security 16)*, 601–18.
- Tschand, Arya, Arun Tejusve Raghunath Rajan, Sachin Idgunji, Anirban Ghosh, Jeremy Holleman, Csaba Kiraly, Pawan Ambalkar, et al. 2024. "MLPerf Power: Benchmarking the Energy Efficiency of Machine Learning Systems from μ Watts to MWatts for Sustainable AI." *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 1201–16. <https://doi.org/10.1109/hpca61900.2025.00092>.
- Turakhia, Yatish, Gill Bejerano, and William J. Dally. 2018. "Darwin: A Genomics Co-Processor Provides up to 15,000X Acceleration on Long Read Assembly." *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 199–213. <https://doi.org/10.1145/3173162.3173193>.
- Turing, Alan M. 1950. "I.—Computing Machinery and Intelligence." *Mind* LIX (236): 433–60. <https://doi.org/10.1093/mind/lix.236.433>.
- Ultralytics. 2023. *YOLOv8*.
- Umuroglu, Yaman, Nicholas J. Fraser, Giulio Gambardella, Michaela Blott, Philip Leong, Magnus Jahre, and Kees Vissers. 2017. "Finn: A Framework for Fast, Scalable Binarized Neural Network Inference." *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 65–74. <https://doi.org/10.1145/3020078.3021744>.
- United States Congress. 1996. *Health Insurance Portability and Accountability Act of 1996*. Public Law 104-191.
- U.S. Department of Health and Human Services. 2003. "Summary of the HIPAA Privacy Rule."
- U.S. Department of Health and Human Services. 2005. "Summary of the HIPAA Security Rule."
- U.S. Department of Health and Human Services. 2026. *Annual Civil Monetary Penalties Inflation Adjustment*. Federal Register.
- U.S. Food and Drug Administration. 2021. *Artificial Intelligence/Machine Learning (AI/ML)-Based Software as a Medical Device (SaMD) Action Plan*. U.S. Department of Health; Human Services.
- U.S. Food and Drug Administration. 2025. *Marketing Submission Recommendations for a Predetermined Change Control Plan for Artificial Intelligence-Enabled Device Software Functions*. Guidance for Industry and Food and Drug Administration Staff. U.S. Department of Health; Human Services.
- U.S. Securities and Exchange Commission. 2013. "Securities Exchange Act of 1934, Release No. 70694: Knight Capital Americas LLC."
- Vasisht, Deepak, Zerina Kapetanovic, Jongho Won, Xinxin Jin, Ranveer Chandra, Sudipta N. Sinha, Ashish Kapoor, Madhusudhan Sudarshan, and Sean Stratman. 2017. "FarmBeats: An IoT Platform for Data-Driven Agriculture." *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, 515–29.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. "Attention Is All You Need." *Advances in Neural Information Processing Systems (NeurIPS)* 30: 5998–6008.
- Villalobos, Pablo, Anson Ho, Jaime Sevilla, Tamay Besiroglu, Lennart Heim, and Marius Hobbhahn. 2022. "Will We Run Out of Data? Limits of LLM Scaling Based on Human-Generated Data." *arXiv Preprint arXiv:2211.04325*.
- Viola, Paul, and Michael J. Jones. 2001. "Rapid Object Detection Using a Boosted Cascade of Simple Features." *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001* 1: 1-511-1-518. <https://doi.org/10.1109/cvpr.2001.990517>.
- Wang, Alex, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. 2019. "SuperGLUE: A Stickier Benchmark for General-Purpose Language Understanding Systems." *arXiv Preprint arXiv:1905.00537*.

- Wang, Alex, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. 2018. "GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding." *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, 353–55. <https://doi.org/10.18653/v1/w18-5446>.
- Wang, Linnan, Jinmian Ye, Yiyang Zhao, Wei Wu, Ang Li, Shuaiwen Leon Song, Zenglin Xu, and Tim Kraska. 2018. "SuperNeurons: Dynamic GPU Memory Management for Training Deep Neural Networks." *Proceedings of the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 41–53. <https://doi.org/10.1145/3178487.3178491>.
- Wang, Shibo, and Pankaj Kanwar. 2019. *BFloat16: The Secret to High Performance on Cloud TPUs*.
- Wang, Tianlu, Jieyu Zhao, Mark Yatskar, Kai-Wei Chang, and Vicente Ordonez. 2019. "Balanced Datasets Are Not Enough: Estimating and Mitigating Gender Bias in Deep Image Representations." *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 5309–18. <https://doi.org/10.1109/iccv.2019.00541>.
- Wang, Xin, Fisher Yu, Zi-Yi Dou, Trevor Darrell, and Joseph E. Gonzalez. 2018. "SkipNet: Learning Dynamic Routing in Convolutional Networks." In *European Conference on Computer Vision (ECCV)*. Springer. https://doi.org/10.1007/978-3-030-01261-8_25.
- Wang, Yu Emma, Gu-Yeon Wei, and David Brooks. 2019. "Benchmarking TPU, GPU, and CPU Platforms for Deep Learning." *arXiv Preprint arXiv:1907.10701*.
- Wang, Zijie J., Robert Turko, Omar Shaikh, Haekyu Park, Nilaksh Das, Fred Hohman, Minsuk Kahng, and Duen Horng Polo Chau. 2021. "CNN Explainer: Learning Convolutional Neural Networks with Interactive Visualization." *IEEE Transactions on Visualization and Computer Graphics* 27 (2): 1396–406. <https://doi.org/10.1109/tvcg.2020.3030418>.
- Warden, Pete. 2018. "Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition." *arXiv Preprint arXiv:1804.03209*, ahead of print. <https://doi.org/10.48550/arXiv.1804.03209>.
- Warden, Pete, and Daniel Situnayake. 2020. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media.
- Waterman, Andrew, Yunsup Lee, Rimas Avizienis, Henry Cook, David Patterson, and Krste Asanovic. 2013. "The RISC-V Instruction Set." *2013 IEEE Hot Chips 25 Symposium (HCS)*, 1–1. <https://doi.org/10.1109/hotchips.2013.7478332>.
- Wei, Jason, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." *Advances in Neural Information Processing Systems (NeurIPS)* 35: 24824–37. <https://doi.org/10.52202/068431-1800>.
- Weights & Biases, Inc. 2024. *Weights & Biases: The AI Developer Platform*.
- Weiser, Mark. 1991. "The Computer for the 21st Century." *Scientific American* 265 (3): 94–104. <https://doi.org/10.1038/scientificamerican0991-94>.
- Weizenbaum, Joseph. 1966. "ELIZA—a Computer Program for the Study of Natural Language Communication Between Man and Machine." *Communications of the ACM* 9 (1): 36–45. <https://doi.org/10.1145/365153.365168>.
- Welling, Max. 2009. "Herding Dynamical Weights to Learn." *Proceedings of the 26th Annual International Conference on Machine Learning*, 1121–28. <https://doi.org/10.1145/1553374.1553517>.
- Wengert, Ralph E. 1964. "A Simple Automatic Derivative Evaluation Program." *Communications of the ACM* 7 (8): 463–64. <https://doi.org/10.1145/355586.364791>.
- Werbos, Paul. 1974. "Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences." PhD thesis, Harvard University.
- Werchaniak, Andrew, Roberto Barra Chicote, Yuriy Mishchenko, Jasha Droppo, Jeff Condal, Peng Liu, and Anish Shah. 2021. "Exploring the Application of Synthetic Audio in Training Keyword Spotters." *ICASSP 2021 - 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 7993–96. <https://doi.org/10.1109/icassp39728.2021.9413448>.
- Wexler, James, Mahima Pushkarna, Tolga Bolukbasi, Martin Wattenberg, Fernanda Viégas, and Jimbo Wilson. 2020. "The What-If Tool: Interactive Probing of Machine Learning Models." *IEEE Transactions on Visualization and Computer Graphics* 26 (1): 56–65. <https://doi.org/10.1109/TVCG.2019.2934619>.
- Widmer, Gerhard, and Miroslav Kubat. 1996. "Learning in the Presence of Concept Drift and Hidden Contexts." *Machine Learning* 23 (1): 69–101. <https://doi.org/10.1023/a:1018046501280>.
- Williams, Samuel, Andrew Waterman, and David Patterson. 2009. "Roofline: An Insightful Visual Performance Model for Multicore Architectures." *Communications of the ACM* 52 (4): 65–76. <https://doi.org/10.1145/1498765.1498785>.
- Wolpert, D. H., and W. G. Macready. 1997. "No Free Lunch Theorems for Optimization." *IEEE Transactions on Evolutionary Computation* 1 (1): 67–82. <https://doi.org/10.1109/4235.585893>.
- Wu, Bichen, Kurt Keutzer, Xiaoliang Dai, Peizhao Zhang, Yanghan Wang, Fei Sun, Yiming Wu, Yuandong Tian, Peter Vajda, and Yanguang Jia. 2019. "FBNNet: Hardware-Aware Efficient ConvNet Design via Differentiable Neural Architecture Search." *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 10726–34. <https://doi.org/10.1109/cvpr.2019.01099>.

- Wu, Hao, Patrick Judd, Xiaojie Zhang, Mikhail Isaev, and Paulius Micikevicius. 2020. "Integer Quantization for Deep Learning Inference: Principles and Empirical Evaluation." *arXiv Preprint arXiv:2004.09602* abs/2004.09602.
- Wu, Jian, Hao Cheng, and Yifan Zhang. 2019. "Fast Neural Networks: Efficient and Adaptive Computation for Inference." *Advances in Neural Information Processing Systems (NeurIPS)*.
- Wulf, Wm. A., and Sally A. McKee. 1995. "Hitting the Memory Wall: Implications of the Obvious." *ACM SIGARCH Computer Architecture News* 23 (1): 20–24. <https://doi.org/10.1145/216585.216588>.
- Xin, Ji, Raphael Tang, Yaoliang Yu, and Jimmy Lin. 2021. "BERxiT: Early Exiting for BERT with Better Fine-Tuning and Extension to Regression." In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, edited by Paola Merlo, Jorg Tiedemann, and Reut Tsarfaty. Association for Computational Linguistics. <https://doi.org/10.18653/v1/2021.eacl-main.8>.
- Xu, Ruijie, Zengzhi Wang, Run-Ze Fan, and Pengfei Liu. 2024. "Benchmarking Benchmark Leakage in Large Language Models." *arXiv Preprint arXiv:2404.18824*.
- Yang, Le, Yizeng Han, Xi Chen, Shiji Song, Jifeng Dai, and Gao Huang. 2020. "Resolution Adaptive Networks for Efficient Inference." *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2366–75. <https://doi.org/10.1109/cvpr42600.2020.00244>.
- Yao, Zhewei, Amir Gholami, Sheng Shen, Kurt Keutzer, and Michael W. Mahoney. 2021. "HAWQ-V3: Dyadic Neural Network Quantization." *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 11875–86.
- Yee, Kyra, Uthaiapon Tantipongpipat, and Shubhanshu Mishra. 2021. "Image Cropping on Twitter: Fairness Metrics, Their Limitations, and the Importance of Representation, Design, and Agency." *Proceedings of the ACM on Human-Computer Interaction* 5 (CSCW2): 1–24. <https://doi.org/10.1145/3479594>.
- Yosinski, Jason, Jeff Clune, Yoshua Bengio, and Hod Lipson. 2014. "How Transferable Are Features in Deep Neural Networks?" *Advances in Neural Information Processing Systems* 27.
- Yu, Gyeong-In, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. "Orca: A Distributed Serving System for Transformer-Based Generative Models." *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 521–38.
- Yun, Sangdoo, Dongyoon Han, Seong Joon Oh, Sanghyuk Chun, Junsuk Choe, and Youngjoon Yoo. 2019. "CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features." *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 6023–32. <https://doi.org/10.1109/ICCV.2019.00612>.
- Yurdakul, Bilal, and Joshua Naranjo. 2020. "Statistical Properties of the Population Stability Index." *The Journal of Risk Model Validation* 52. <https://doi.org/10.21314/jrmv.2020.227>.
- Zafir, Ofir, Guy Boudoukh, Peter Izsak, and Moshe Wasserblat. 2019. "Q8BERT: Quantized 8Bit BERT." *2019 Fifth Workshop on Energy Efficient Machine Learning and Cognitive Computing - NeurIPS Edition (EMC2-NIPS)*, 36–39. <https://doi.org/10.1109/emc2-nips53020.2019.00016>.
- Zaharia, Matei, Andrew Chen, Aaron Davidson, Ali Ghodsi, Sue Ann Hong, Andy Konwinski, Siddharth Murching, et al. 2018. "Accelerating the Machine Learning Lifecycle with MLflow." *IEEE Data Engineering Bulletin* 41 (4): 39–45.
- Zaharia, Matei, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, and Ion Stoica. 2010. "Spark: Cluster Computing with Working Sets." *HotCloud* 10: 10–10.
- Zaharia, Matei, Ali Ghodsi, Reynold Xin, and Michael Armbrust. 2021. "Lakehouse: A New Generation of Open Platforms That Unify Data Warehousing and Advanced Analytics." *Proceedings of CIDR* 8.
- Zech, John R., Marcus A. Badgeley, Manway Liu, Anthony B. Costa, Joseph J. Titano, and Eric Karl Oermann. 2018. "Variable Generalization Performance of a Deep Learning Model to Detect Pneumonia in Chest Radiographs: A Cross-Sectional Study." *PLOS Medicine* 15 (11): e1002683. <https://doi.org/10.1371/journal.pmed.1002683>.
- Zhang, Biao, and Rico Sennrich. 2019. "Root Mean Square Layer Normalization." *Advances in Neural Information Processing Systems (NeurIPS)*.
- Zhang, Brian Hu, Blake Lemoine, and Margaret Mitchell. 2018. "Mitigating Unwanted Biases with Adversarial Learning." *Proceedings of the 2018 AAAI/ACM Conference on AI, Ethics, and Society*, 335–40. <https://doi.org/10.1145/3278721.3278779>.
- Zhang, Chengliang, Minchen Yu, Wei Wang, and Feng Yan. 2019. "Mark: Exploiting Cloud Services for Cost-Effective, SLO-Aware Machine Learning Inference Serving." *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, 1049–62.
- Zhang, Chiyuan, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. 2017. "Understanding Deep Learning Requires Rethinking Generalization." *International Conference on Learning Representations (ICLR)*.
- Zhang, Hongyi, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. 2018. *mixup: Beyond Empirical Risk Minimization*. OpenReview.net.

- Zhang, Li Lyna, Yuqing Yang, Yuhang Jiang, Wenwu Zhu, and Yunxin Liu. 2020. "Fast Hardware-Aware Neural Architecture Search." *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. <https://doi.org/10.1109/cvprw50498.2020.00354>.
- Zhang, Qingxue, Dian Zhou, and Xuan Zeng. 2017. "Highly Wearable Cuff-Less Blood Pressure and Heart Rate Monitoring with Single-Arm Electrocardiogram and Photoplethysmogram Signals." *BioMedical Engineering OnLine* 16 (1): 23. <https://doi.org/10.1186/s12938-017-0317-z>.
- Zhang, Yundong, Naveen Suda, Liangzhen Lai, and Vikas Chandra. 2017. *Hello Edge: Keyword Spotting on Microcontrollers*.
- Zhao, Jiawei, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. 2024. "GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection." *arXiv Preprint*.
- Zheng, Lianmin, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, et al. 2020. "Ansor: Generating High-Performance Tensor Programs for Deep Learning." *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 863–79.
- Zhou, Aojun, Yukun Ma, Junnan Zhu, Jianbo Liu, Zhijie Zhang, Kun Yuan, Wenxiu Sun, and Hongsheng Li. 2021. "Learning n:m Fine-Grained Structured Sparse Neural Networks from Scratch." *arXiv Preprint*.
- Zhu, Chenzhao, Song Han, Huizi Mao, and William J. Dally. 2017. "Trained Ternary Quantization." *International Conference on Learning Representations (ICLR)* 4 (4): 1–16.
- Zhu, Ligeng, Zhijian Liu, and Song Han. 2019. "Deep Leakage from Gradients." *Advances in Neural Information Processing Systems (NeurIPS)* 32: 14774–84.
- Zillow Group. 2021. *Zillow Group Reports Third-Quarter 2021 Financial Results and Shares Plan to Wind down Zillow Offers Operations*. Investor Relations Press Release.
- Zoph, Barret, and Quoc V. Le. 2016. "Neural Architecture Search with Reinforcement Learning." *International Conference on Learning Representations* 3.
- Zu, Y., A. Ghaffarkhah, H.-V. Dang, B. Towles, S. Hand, S. Huda, A. Bello, et al. 2024. "Resiliency at Scale: Managing Google's TPUv4 Machine Learning Supercomputer." *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, 761–74.

Index

- 1980s
 - definition, 561
- 1990s
 - definition, 561
- 2000s
 - definition, 561
- 2010s
 - definition, 561
- 2020s
 - definition, 561
- A/B testing
 - delayed feedback, A/B testing challenges, 802
 - ML deployment, 97
 - randomization unit, 802
 - statistical model validation, 802
- A100, 390, 404
 - data-loading requirements, 135
 - HBM capacity, 353
 - MIG partitioning, 742
 - roofline analysis, 649, 933
 - Tensor Core throughput, 336
- Ablation study
 - component isolation, 97
 - definition, 97
- Abstraction problem
 - definition, 324
- Acceleration wall
 - diminishing returns, 552
- Accelerator
 - bubbles, pipeline idleness, 361
 - design, workload-specific memory, 595
 - memory bandwidth limitations, 381
 - ML, 559
 - specifications, roofline analysis example, 649
- Accelerator gain
 - definition, 551
- Access pattern
 - definition, 588
- Activation checkpointing
 - definition, 316
 - memory-compute trade-off, 316, 361, 412
 - optimal placement, 412
 - recomputation strategy, 318
 - recomputation trade-off, 390
- Activation function
 - definition, 189
 - nonlinear transformation, 182
 - softmax, 191
 - training efficiency, 369
- Activation memory
 - definition, 210
- Activation reuse
 - definition, 596
- Activation unit
 - definition, 570
- Activation-based pruning
 - definition, 490
- Active learning
 - budget multiplier, 151
 - definition, 446
 - label prioritization, 151
 - labeling cost reduction, 437, 446
- Adam optimization, *see* Adam optimizer
- Adam optimizer
 - convergence properties, 374
 - Kingma and Ba, 374
 - memory overhead, 214, 374, 375
 - momentum and velocity, 214
- AdamW
 - decoupled weight decay, 374, 375
- Adaptability
 - definition, 628
- Adaptive computation
 - early exit, 530
- Adaptive inference
 - definition, 529
- Adaptive resolution
 - content-based selection, 731
- Admission control
 - queue depth threshold, 738
- Adversarial debiasing
 - fairness constraints, 858
 - training cost, 858
- Ahead-of-time compilation, 307
 - predeployment, 763
- Ahead-of-time optimization, 303
- AI compute primitive
 - definition, 563
- AI democratization, 892
- AI engineering
 - definition, 25
- AI memory systems
 - bandwidth bottleneck, 583
- AI Memory Wall
 - definition, 583
- AI runtime
 - definition, 628
 - dynamic execution management, 627
 - vs. traditional runtime, 628
- AI winters
 - systems failure, 8
- AI-assisted labeling
 - preannotation, 151
 - scalability, 150
- Alert fatigue
 - operational risk, 821
- AlexNet, 269
 - benchmarking paradigm, 696
 - GPU deep learning era, 556
 - ImageNet breakthrough, 6, 179
 - ImageNet revolution, 243
 - memory constraint, 179
 - multi-GPU origin, 419
 - parameter growth, 156
 - two-GPU training, 556
- Algorithm-hardware adoption lag, 184
- Algorithmic efficiency
 - definition, 22
- Algorithmic fairness
 - healthcare ML, 92
- Alignment gap
 - proxy metric divergence, 846
- AllGather
 - collective communication, 335
- AllReduce
 - collective communication, 335
 - definition, 594, 631
 - gradient synchronization, 422
 - gradient synchronization cost, 594, 631
 - network wall, 422
 - ring topology, 422
- AlphaFold
 - cloud deployment, 28
- AlphaGo, 11
 - self-play training, 11
- Always-on sensing
 - power constraints, 45
- Amazon Kinesis
 - streaming trade-offs, 139
- Amazon recruiting
 - specification failure, 842
- Amdahl's law
 - acceleration ceiling, 552
 - appendix refresher, 934
 - data pipeline ceiling, 146
 - diagnostic analogy, 19
 - distributed training limit, 631
 - multi-accelerator scaling, 631
 - optimization ceiling, 679
 - parallel fraction, 551
 - pipeline bottleneck, 78
 - preprocessing bottleneck, 727, 888
 - speedup limits, 78
 - system optimization, 80
- Amdahl's reality
 - definition, 552
- Amdahl, Gene, *see also* Amdahl's law
- Anti-curriculum
 - hard examples first, 446
- AOT, *see* Ahead-of-time optimization
- Apache Airflow
 - lineage capture, 876
 - pipeline orchestration, 787
- Apache Arrow
 - in-memory columnar format, 158
- Apache Atlas
 - data lineage, 876
- Apache Beam
 - distributed processing, 146
- Apache Kafka
 - event streaming, 139
 - ordering guarantee, 139
- Apache Pulsar
 - geo-replication, 139
- Apache Spark
 - distributed processing, 146
- Apache TVM
 - accelerator compilation, 603
 - ML compiler, 347
- Application scenario
 - definition, 26
- Application-specific integrated circuit, *see* ASIC
- Architectural efficiency
 - theory-practice gap, 524
- Architectural Integration
 - execution unit organization, 580
- Architecture
 - building blocks, 269
 - computational graph, 232
 - compute vs. memory trade-off, 236

- data-to-architecture mapping, 282
- encoder-decoder, 275
- parameter efficiency, 238
- representational power vs. efficiency, 233
- selection as bias matching, 238
- systems engineering, 232
- Architecture examples
 - definition, 561
- Area under the learning curve
 - definition, 470
- Area under the ROC curve
 - threshold independence, 96
- Argmax
 - prediction selection, 732
- Arithmetic intensity, 48
 - appendix refresher, 904, 916, 932
 - attention layer, 382
 - bottleneck diagnosis, 888
 - definition, 597
 - FLOP/byte ratio, 235
 - GPU utilization, 227
 - high intensity workloads, 45
 - roofline diagnostic, 553
 - threshold for compute vs. memory bound, 649
 - training operations, 381
- Artifact
 - interdependence, 98
 - lineage tracking, 98
 - reproducibility, 776
 - versioning, reproducibility requirement, 776
- Artificial intelligence
 - hierarchy with ML and DL, 175
- Artificial neuron
 - computing primitive, 182
- As-of join
 - definition, 161
 - point-in-time correctness, 161
- ASIC
 - benchmarking trade-off, 648
 - flexibility trade-off, 558
- Assumptions
 - accelerator, 942
 - bandwidth, 948
 - energy, 947
 - nameplate capacity, 950
 - pricing, 949
 - reference models, 946
 - scale, 950
 - training memory, 946
 - unit conventions, 950
- Asynchronous execution
 - hiding transfer latency, 329
- Attention
 - memory bottleneck, 406
 - quadratic complexity, 406
- Attention head
 - specialization, 262
- Attention matrix, 262
- Attention Mechanism
 - global token interaction, 595
 - memory-bound softmax, 599
- Attention mechanism
 - Bahdanau, 256
 - content-addressable memory, 263
 - definition, 256
 - fused kernel, 298
 - information bottleneck, 256
 - memory pressure, 595
 - multi-head, 262
 - quadratic bottleneck, 260
 - query-key-value, 258
 - scaled dot-product, 263
 - self-attention, 262
 - similarity computation, 389
- AUC
 - anomaly detection performance, 661
- Audit trail
 - accountability logging, 876
 - inference-time logging, 876
 - multi-tier logging, 876
- storage scale, 877
- AutoAugment
 - search cost, 453
- autocast
 - mixed-precision context, 317
- Autograd
 - automatic differentiation, 214
 - engine, internals, 317
 - execution trade-off, 214
 - reverse-linked graph, 317
- Autograd tape
 - definition, 300
 - dynamic graph construction, 300
 - memory cost, 300
 - reverse traversal, 353
- Automatic differentiation
 - appendix refresher, 919
 - computational graph, 214, 377
 - definition, 313
 - forward mode, 313
 - forward vs. reverse mode, 313
 - forward-mode dual numbers, 314
 - framework role, 294
 - reverse mode, 209, 313, 314
 - reverse-mode backpropagation, 314
 - reverse-mode memory cost, 313
 - tape-based vs. transform-based, 323
- Automatic mixed precision
 - framework support, 401
- Automotive AI
 - real-time safety requirements, 633
- Autonomous perception
 - definition, 27
- Autonomous Vehicle
 - latency requirements, 60
- Autoregressive generation, 234
 - bandwidth bottleneck, 234
 - memory-bound, 45
- Autoregressive model
 - autoregressive, serial bottleneck, 755
 - token generation, 750
- Autoscaling
 - ML cold-start trade-off, 807
 - ML workloads, 807
 - startup latency, 766
- Back-translation
 - pipeline latency, 453
 - text augmentation, 452
- Backpressure
 - streaming systems, 137
- Backpropagation
 - activation storage, 270
 - appendix refresher, 919
 - computational asymmetry, 314
 - computational cost, 377
 - definition, 208
 - gradient computation, 208
 - historical development, 184
 - historical introduction, 367
 - memory cost, 184
 - memory requirements, 377, 559
 - memory trade-off, 270
 - provenance, 367
 - through time (BPTT), 253
- Backward pass
 - gradient computation, 390
 - memory operations, 390
 - triggering gradient computation, 353
- Bagging
 - ensemble method, 95
- Bandwidth
 - bottleneck, video streaming, 56
 - latency trade-off, 58, 937
 - PCIe, model swapping, 742
- Bandwidth hog
 - definition, 45
- Bandwidth taper
 - interconnect hierarchy, 581
- Baseline model
 - reference point selection, 659
- Batch
 - loss averaging, 206
 - training iteration, 201
- Batch inference
 - definition, 808
- Batch ingestion
 - definition, 136
- Batch normalization, 273
 - hardware implementation, 568
 - mode-dependent behavior, 339
 - systems cost, 191
 - training speedup, 273
 - training stability, 191
 - training-serving skew, 273
- Batch processing
 - definition, 373
 - hardware utilization, 201
- Batch Size
 - arithmetic intensity impact, 602
- Batch size, 241
 - arithmetic intensity effect, 383
 - gradient stability, 215
 - hardware alignment, 388
 - inference trade-off, 219
 - loss computation, 206
- Batch throughput
 - definition, 680
- Batch-1 regime
 - definition, 748
- Batch-N regime
 - definition, 748
- Batching
 - batch formation delay, definition, 735
 - batch, small, CPU advantage, 763
 - etymology, 743
 - fallacy of larger batches, 770
 - hardware utilization, 368
 - request, throughput optimization, 805
 - tax, latency penalty, 735
 - training vs. serving, 743
 - window, latency-throughput trade-off, 725
- Battery
 - coin-cell deployment, 65
- Battery life
 - ML impact, 62
- Battery tax
 - always-on constraints, 62
- Bayes error rate
 - appendix refresher, 907
- Beam search
 - decoding strategy, 756
- Benchmark
 - etymology, 642
- Benchmark contamination
 - LLM memorization risk, 700
- Benchmark coverage
 - incomplete problem representation, 691
- Benchmark engineering
 - intentional optimization bias, 693
- Benchmark gaming
 - distorting performance metrics, 695
 - vendor optimization for tests, 642
- Benchmark harness
 - test infrastructure, 661
- Benchmark saturation
 - outdated benchmarks pitfall, 706
- Benchmark-production gap, 640
- Benchmarking
 - battery, duty cycle specification, 667
 - benchmarks as proxies, 641
 - definition, 640
 - end-to-end vs. component metrics, 642
 - energy, first-class metric, 643
 - fallacy of direct performance translation, 706
 - historical evolution, 642

- probabilistic variability, 644
 - system, definition, 641
- Benchmarking granularity
 - definition, 652
- Bengio, Yoshua
 - Turing Award, 175
- Berkson, Joseph, *see also* Logits
- BERT
 - benchmarking baseline, 659
 - compression techniques, 495
 - inference deployment
 - prediction, 650
 - roofline analysis at batch 1, 649
- Best when
 - definition, 620
- BF16
 - appendix refresher, 938
 - design rationale, 320
 - dynamic range preservation, 404
 - exponent range preservation, 400
 - hardware support, 404
 - reduced-precision format, 219
- Bias
 - activation threshold, 194
 - attribute removal fallacy, 877
 - gender discrimination, 843
 - geographic, training data, 127
 - historical data encoding, 843
 - mitigation strategies, 848
 - parameter overhead, 194
 - proxy variables, 843
 - signal aggregation, 182
- Bias feedback model
 - disparity amplification, 888
- Bias-variance tradeoff
 - classical theory, 179
 - double descent, 179
 - overparameterization, 179
- Binarization
 - definition, 523
- Biological Neuron
 - definition, 182
- Bisection bandwidth
 - fleet-scale bottleneck, 32
- Bitter lesson
 - definition, 11
- bitter lesson, The, 11
- BLAS
 - historical foundation, 296
 - matrix computation foundation, 367
 - neural networks, 241
 - performance foundation, 296
 - training compute hierarchy, 367
 - utilization, 241
 - vector and matrix operations, 336
- BLEU
 - Goodhart's law example, 661
- Blue-green deployment
 - duplicate capacity, 801
 - zero-downtime updates, 800
- Bobrow, Daniel
 - STUDENT, 8
- Boosting
 - ensemble method, 95
- Bottleneck principle
 - definition, 43
- Boundary erosion
 - dissolution of modularity, 782
- Bounding box
 - annotation, 148
 - definition, 148
- Break-even analysis
 - edge vs. cloud utilization, 54
- Break-even point
 - data selection ROI, 463
- Brittleness
 - definition, 10
- Broadcasting
 - definition, 918
- Bulkhead pattern
 - failure containment, 828
 - resource isolation, 828
- Byte pair encoding
 - tokenization, 386
- Byzantine fault tolerance
 - ML-specific manifestations, 828
 - plausible ML failures, 828
- C4
 - quality filtering, 440
- CACE principle
 - change propagation, 782
- Cache efficiency
 - definition, 613
- Cache locality
 - definition, 588
- Calibration
 - clinical trust, 99
 - dataset, representative traffic, 761
 - definition, 99
 - definition and etymology, 697
 - INT8 quantization, 761
 - production traffic mismatch, 770
- Canary
 - etymology, 738
- Canary deployment
 - risk-controlled rollout, 800
 - statistical validation, 800
- Canary request
 - fan-out protection, 738
- Canonical workload, 235
- CAP theorem
 - feature store trade-off, 139
 - streaming systems, 139
- Capacity planning
 - infrastructure sizing, 766
- Carbon footprint
 - energy to emissions conversion, 867
- Carbon intensity
 - definition, 870
 - grid emissions, 869
- Carbon-aware scheduling
 - lower-carbon times and regions, 870
- Categorical Encoding
 - one-hot encoding, 142
- Categorical feature, 267
- Central processing unit, *see* CPU
- Chain Rule
 - gradient decomposition, 209
- Chain rule
 - appendix refresher, 919
 - backpropagation foundation, 377
 - depth constraint, 210
- Challenge
 - definition, 604
- Channelwise quantization, *see* Quantization
- Characteristics
 - definition, 561
- CheckList, 849
- Checkout-free retail
 - edge processing, 60
- Checkpointing
 - I/O cost, 217
- Chinchilla
 - compute-optimal scaling, 470
 - compute-optimal training, 470
 - tokens-per-parameter diagnostic, 471
- Chiplet
 - modular die interconnect, 630
- CI/CD pipeline
 - ML-specific adaptation, 790
- CIFAR-10
 - classification reference dataset, 659
- Circuit breaker
 - cascade prevention, 135
- Circuit breaker pattern
 - cascade failure prevention, 817
 - ML adaptation, 817
- Class balance
 - coverage metric, 700
- Class imbalance
 - coverage failure mode, 700
 - minority-class performance, 227
- Classical machine learning
 - feature engineering, 177
- Classification
 - definition, 598
- Classification label
 - storage requirements, 148
- Cleanlab
 - label error detection, 444
- ClinAIOps
 - healthcare ML operations, 832
 - MLOps comparison, 834
- Clinician oversight loop, 832
- CLIP, *see* Contrastive Language-Image Pretraining
- Closed division
 - MLPerf submission, 683
- Cloud economics
 - elastic pricing, 52
- Cloud infrastructure
 - utility model, 50
- Cloud ML, 27
 - accelerator infrastructure, 52
 - characteristics, 40
 - consumer-scale applications, 54
 - data center scale, 50
 - definition, 51
 - latency limitations, 53
 - privacy concerns, 53
 - scaling limits, 55
 - workload archetypes, 51
- Cloud pricing
 - cost-latency trade-off, 125
- CNN
 - definition, 242
 - spatial data reuse, 595
- COCO
 - object detection dataset, 659
- Code generation
 - hardware-specific instructions, 627
- Codec
 - definition, 558
- Cohen's kappa
 - inter-annotator agreement, 701
 - label quality, 133
- Coin-cell battery
 - deployment longevity, 64
- Cold start
 - anatomy, 740
 - bursty traffic impact, 770
 - cloud serving, 721
 - definition, 740
 - inference concern, 682
 - scaling events, 740
- Cold start latency
 - physical lower bound, 682
 - serverless AI impact, 680
- Collation
 - batch assembly, 334
- Columnar storage
 - Parquet, 158
- Common techniques
 - definition, 601
- Common use cases
 - definition, 620
- Communication and synchronization
 - definition, 606
- Communication tax
 - definition, 419
- COMPAS
 - calibration and error rates, 844
 - recidivism algorithm audit, 843
- Compilation continuum
 - principle, 308
- Compilation output
 - definition, 623
- Complexity tax
 - ML vs. heuristics, 71
- Compositionality
 - pattern decomposition, 187
- Compressed sparse row
 - definition, 917
- Compression
 - algorithm selection, 158
- Computation graph
 - dynamic (eager execution), 214
 - static (deferred execution), 214
- Computation Placement

- processing element assignment, 604
- Computation scheduling
 - definition, 622
 - parallel execution, 626
- Computational granularity
 - definition, 605
- Computational graph
 - definition, 294
 - definition (DAG), 298
 - framework optimization, 294
 - static, 302
- Computational pattern
 - definition, 561
- Computational photography
 - pipeline constraint, 61
- Compute beast
 - definition, 45
- Compute bottleneck
 - definition, 10
- Compute efficiency
 - definition, 22
- Compute-Bound
 - roofline classification, 596
- Compute-bound
 - operation classification, 298
 - resolution scaling, 730
- Compute-Bound vs. Memory-Bound
 - training vs. inference, 48
- Compute-bound vs. memory-bound
 - definition, 43
 - memory-bound optimization, 44
- Compute-optimal frontier
 - Chinchilla scaling laws, 470
- Computer engineering
 - historical lineage, 25
- Concept drift
 - adversarial evolution, 779
 - changing relationships, 810
 - definition, 133, 845
 - validation, 100
- Concept-based explanation
 - human-understandable feature, 860
- Conditional computation
 - definition, 530
- Confidence interval
 - benchmark reporting, 646
- Confidence threshold
 - automation trade-off, 222
 - decision making, 223
- Confident learning
 - mislabeled example detection, 701
- Confidential computing
 - hardware secure enclaves, 873
- Configuration debt
 - Facebook case study, 785
 - parameter sprawl, 784
- Confusion matrix
 - fairness analysis, 856
- Consensus labeling
 - quality control, 150
- Conservation of complexity
 - heuristic limits, 887
 - meta-principle, 887
 - structural optimization, 485
- Consistency Imperative
 - definition, 141
- Consistency imperative
 - training-serving parity, 141, 777
- Consistency regularization
 - compute trade-off, 449
 - semi-supervised technique, 449
- Constant folding, 303
 - compile-time optimization, 762
- Constrained hardware
 - inference deployment, 218
- Constraint layer
 - statistical physical operational, 91
- Constraint propagation principle
 - definition, 105
- Containerization
 - environment parity, 800
 - ML deployment, 807
- reproducible environments, 147
- Continuous batching
 - LLM serving, 750
 - scheduling granularity, 750
- Continuous therapeutic monitoring
 - healthcare AI, 832
 - human oversight, 832
- Contrastive Language-Image Pretraining
 - semantic deduplication, 444
- Contrastive Learning
 - batch-based methods, 451
 - pretext task, 450
- Convergence
 - monitoring, 217
 - rate, clean vs. noisy data, 440
 - time-memory trade-off, 374
- Convolution
 - computational cost, 389
 - data reuse, 242
 - depthwise, 251
 - depthwise separable, 251
 - etymology, 242
 - hardware optimization, 251
 - kernel, 244
 - matrix multiplication
 - equivalence, 566
 - padding, 247
 - pointwise, 251
 - stride, 247
- Convolution performance
 - definition, 613
- Convolutional neural network, *see* CNN
- Coordination Tax
 - distributed data selection, 467
- Coordination tax
 - definition, 145
- Coreset
 - data efficiency, 457
 - definition, 441
 - etymology, 441
 - geometry origins, 441
- Correction cascade
 - sequential dependencies, 782
 - Zillow case study, 785
- Cosine annealing
 - learning rate schedule, 377
- Cost amortization
 - self-supervised pretraining, 450
- Cost modeling
 - data selection economics, 462
- Cost per inference
 - calculation, 809
 - serving economics, 766
- Cost-aware automation
 - retraining decisions, 777
- Cost-Performance Analysis
 - accelerator economics, 581
- Counterfactual analysis
 - prediction debugging, 820
- Covariate shift
 - definition, 133
 - input distribution change, 701
- CP decomposition
 - definition, 499
- CPU
 - inference optimization, 763
 - neural network execution, 205
- CPU affinity
 - latency reduction, 723
- CPU backend
 - optimized BLAS libraries, 355
- Cray-1
 - AI accelerator lineage, 565
- Credit assignment problem
 - weight responsibility, 208
- Critical batch size
 - throughput-convergence limit, 373
- Critical path
 - composed request, 894
 - optimization, 47
- Cross-entropy loss
 - classification loss, 206
 - definition, 206
 - information theory, 206
- Cross-industry standard process for data mining
 - lifecycle influence, 82
- Cross-layer interaction
 - cross-optimization effects, 467
- Crowdsourcing
 - annotation, 126
 - data labeling, 126
- cuBLAS
 - hardware-accelerated linear algebra, 367
 - kernel selection, 336
 - NVIDIA linear algebra, 625
- CUDA
 - context, initialization overhead, 740
 - cores, streaming processors, 604
 - dispatch overhead, 307
 - ecosystem lock-in, 572
 - runtime, OS layer, 328
 - serving cold starts, 740
 - stream definition, 329
 - stream overlap, 329
- CUDA graphs
 - kernel launch replay, 654
- CUDA MPS
 - context sharing, 740
 - multi-model serving, 740
- cuDNN
 - benchmarking dependency, 653
 - NVIDIA deep learning library, 625
 - optimized implementations, 389
- Curriculum learning
 - difficulty scorer and pacing function, 445
 - etymology, 445
 - optimization landscape, 445
- Custom autograd function
 - backward implementation, 319
- Custom differentiation rule, 319
- CutMix
 - image augmentation, 452
 - label mixing, 453
- Cutout
 - augmentation efficiency, 452
 - image augmentation, 452
- D-A-M Taxonomy
 - bottleneck classification, 394
- DAM taxonomy, *see* D-A-M taxonomy
- D-A-M taxonomy
 - alignment of components, 225
 - appendix refresher, 901
 - axis intersections, 902
 - definition, 13
 - diagnostic framework, 703
 - responsibility diagnosis, 846
 - scorecard, 907
 - training vs. inference, 48
 - workload classification, 45
- Dark knowledge
 - class correlations, 497
- Dark Silicon
 - specialization driver, 557
- Dartmouth Conference, 8
 - systems oversight, 8
- Data
 - as source code, 13
 - coverage, requirements, 127
 - partitioning, query optimization, 158
 - physics of, 113
 - transformation, techniques, 142
- Data acquisition
 - scalability, 124
 - scale vs. quality, 124
 - source evaluation, 123
 - strategies, 123
- Data augmentation
 - definition, 452
 - synthetic data, 126
 - training diversity, 222
- Data benchmarking
 - definition, 641
 - training set validation, 700

- Data card
 - compliance documentation, 874
- Data cascade
 - definition, 105, 117
 - silent failure, 116
- Data center
 - energy consumption, 870
 - ML infrastructure, 50
- Data cleaning
 - error correction, 141
 - inconsistency removal, 141
 - signal-to-noise engineering, 115
- Data collection
 - iron law data term, 92
- Data compression ratio
 - definition, 470
- Data curation
 - end-to-end generalization, 653
- Data debt
 - definition, 162
 - hidden liability, 162
 - strategic vs. unconscious, 164
- Data Drift
 - definition, 810, 845
- Data drift
 - definition, 17
 - degradation equation, 142
 - detection, 132
 - operational burden, 29
 - silent model degradation, 103
 - Waymo example, 29
- Data echoing
 - amortizing I/O costs, 460
 - GPU utilization, 460
- Data engineering
 - definition, 29, 112
- Data freshness
 - SLA, 137
 - staleness detection, 815
- Data governance
 - definition, 870
 - enforcement mechanisms, 871
 - model artifact compliance, 878
- Data gravity
 - appendix refresher, 909
 - definition, 113
- Data gravity invariant
 - cloud limitations, 52
- Data gravity principle
 - physical anchor, 887
- Data ingestion
 - pipeline boundary, 135
- Data labeling
 - clinical economics, 448
 - systems engineering, 148
- Data lake
 - schema flexibility, 154
 - unstructured storage, 154
- Data lakehouse
 - architecture, 113
 - data gravity, 113
- Data lineage
 - audit trail, 105
 - debugging and compliance, 147
 - GDPR compliance, 876
 - processing history, 147
 - regulatory compliance, 104
 - transformation tracking, 147
- Data loading
 - shard-based sequential I/O, 459
- Data Locality
 - spatial and temporal, 576
- Data locality
 - code-to-data, 115
 - data-to-code, 115
- Data locality invariant
 - definition, 58
- Data localization
 - cross-border transfer, 874
- Data management
 - MLOps lifecycle, 786
- Data mesh
 - decentralized ownership, 113
- Data minimization
 - privacy by design, 874
- Data movement
 - broadcast, 280
 - energy dominance, 42, 181, 609
 - gather, 280
 - optimization strategies, 608
 - reduction, 280
 - scatter, 280
- Data Parallelism
 - training strategy, 672
- Data parallelism
 - communication overhead, 365
 - framework implementation, 419
 - gradient averaging, 419
 - gradient synchronization, 419
 - multi-accelerator scaling, 630
- Data path
 - customization, 558
- Data pipeline
 - architecture, 128
 - automated workflows, 787
 - ingestion and preprocessing, 384
 - storage throughput, 385
 - throughput optimization, 332
- Data poisoning
 - training pipeline security, 873
- Data prefetching
 - accelerator utilization, 397
 - buffer management, 397
- Data quality
 - as code, 131
 - container, 131
 - content, 131
 - expectation tests, 131
 - mechanical vs. semantic, 131
 - monitoring, 813
 - noise penalty on convergence, 440
 - statistical monitoring, 130
 - vs. data quantity, 78
- Data quality multiplier
 - label noise penalty, 440
- Data redundancy
 - empirical evidence, 440
- Data reuse
 - accelerator optimization, 609
 - definition, 588
 - energy efficiency, 181
- Data science
 - vs. data engineering, 112
- Data selection
 - definition, 22, 434, 436
 - optimization ordering, 435
 - systems vs. ML framing, 436
- Data shuffling
 - random access penalty, 386
- Data supply rate
 - bandwidth constraint, 157
- Data swamp
 - governance failure, 154
- Data tax
 - definition, 438
- Data validation
 - monitoring, 129
- Data versioning
 - DVC tool, 786
 - model reproducibility, 159
 - reproducibility, 85, 790
- Data wall
 - definition, 434
 - scaling constraint, 434
 - zero learning signal, 438
- Data warehouse
 - analytical queries, 154
 - OLAP, 154
- Data-as-code principle
 - definition, 5
 - logical program, 887
- Data-centric AI, 5
 - dataset enhancement paradigm, 702
- Data-model interface
 - feature consistency, 775
- Database
 - OLTP for ML, 154
- DataComp
 - data-centric benchmark, 702
 - data-centric benchmarking, 702
- Dataflow
 - optimization strategies, 609
- Dataflow Architecture
 - definition, 603
- Dataflow optimization
 - data movement minimization, 608
- Dataflow strategy
 - definition, 621
- DataHub
 - data lineage, 876
- DataLoader
 - configuration parameters, 399
 - throughput optimization, 332
- DataPerf
 - data quality benchmark, 702
- Dataset
 - iterable-style, 333
 - map-style, 333
 - MNIST benchmark, 195
- Dataset compilation
 - definition, 112
- Dataset saturation
 - performance capability illusion, 702
- Datasheets for datasets
 - training data documentation, 854
- DAWNBench
 - time-to-accuracy evaluation, 643
- Dead code elimination, 303
 - graph optimization, 303
- Dead neuron, 271
- Dean, Jeff
 - latency numbers, 156, 929
- Debugging
 - data pipelines, 164
- Debugging checklist
 - runbook steps, 820
- Decision framework
 - paradigm selection, 69
- Decode phase
 - memory-bandwidth bound, 768
 - sequential generation, 768
- Deduplication
 - data selection optimization, 443
 - definition, 112
 - embedding-based, 444
 - hash-based, exact matching, 443
 - storage and I/O cost reduction, 437
- Deep Blue, 11
 - custom chess hardware, 11
- Deep learning
 - automatic feature learning, 178
 - definition, 9, 175
 - network depth, 187
- Deep learning framework
 - automatic differentiation, 296
- Deep Learning Recommendation Model
 - workflow constraints, 87
- DeepBench
 - operator-level testing, 653
- Default in frameworks
 - definition, 613
- Deferred execution
 - graph construction, 294
- Degradation equation
 - definition, 17
 - distribution shift, 76
 - operational implementation, 774
- Delta Lake
 - time travel, 160
- Demographic drift
 - definition, 92
- Demographic parity
 - definition, 856
- Dennard Scaling
 - end of, 556
 - multi-core revolution, 558
- Dennard scaling
 - origin and breakdown, 42
 - power constraint, 557
- Dense connectivity, 239
 - parameter scaling, 196
- Dense Layer
 - roofline analysis, 600

- Deployment
 - competition-production gap, 96
 - compute-intensive cloud
 - deployment, scientific computing, 28
 - controlled, risk mitigation, 799
 - high-stakes hybrid deployment, autonomous vehicles, 28
 - iron law overhead minimization, 101
 - resource-constrained edge, precision agriculture, 28
- Deployment infrastructure
 - definition, 30
- Deployment paradigm
 - definition, 15, 42
- Deployment target
 - cloud to edge spectrum, 347
- Depth
 - exponential advantage, 187
- Depthwise conv
 - definition, 598
- Depthwise Convolution
 - low arithmetic intensity, 598
- Depthwise separable convolution, 251
 - mobile deployment, 87
 - mobile efficiency, 525
 - MobileNet, 235
- Deterministic Transformation
 - reproducibility, 144
- Developer feedback loop, 832
- Device management
 - CPU-GPU transfers, 328
- DevOps
 - MLOps extension, 775
- Diabetic retinopathy
 - case study introduction, 89
 - deployment scale, 89
- Differential privacy
 - epsilon budget, 873
 - epsilon trade-off, 848
 - formal guarantees, 873
 - mathematical guarantees, 848
 - noise injection, 873
- Differentiation problem
 - definition, 313
- Diffusion model
 - text-to-image synthesis, 453
- Digital signal processing
 - multiply-accumulate units, 556
- Dimensional homogeneity
 - appendix refresher, 934
- Direct memory access, *see* DMA
- Disaggregated evaluation
 - definition, 848
 - demographic stratification, 848
 - fairness testing, 855
- Disparate impact
 - legal doctrine, 849
- Dispatch overhead
 - law, 310
- Dispatch overhead reduction
 - kernel launch batching, 654
- Dispatch tax
 - framework overhead, 301
- Display overlay
 - GPU mapping, 632
- Distance penalty
 - cloud latency, 52
- DistilBERT
 - deployment metrics, 497
- Distributed computing
 - system-level performance, 643
- Distributed data
 - clinic coordination cost, 94
- Distributed Processing
 - scaling, 145
- Distributed Selection
 - network topology constraints, 467
- Distributed training
 - communication overhead, 421
 - communication tax, 422
 - data parallelism, 365
 - data selection challenges, 465
 - execution contexts, 335
 - model parallelism, 365
 - scaling efficiency, 420
- Distribution Shift
 - responsibility lens, 845
- Distribution shift
 - causes of model degradation, 845
 - label shift, 133
 - monitoring, 132
 - population mismatch, 847
 - train-to-production alignment, 701
- Diversity Sampling
 - active learning query strategy, 446
- DLRM, 267
 - capacity-bound workload, 886
 - embedding bottleneck, 365
 - I/O-bound serving, 728
 - interaction layer, 267
 - memory capacity bound, 46
 - memory-bound benchmark, 683
 - memory-bound constraint, 55
 - recommendation model
 - benchmark, 683
 - scalability challenge, 119
- DMA
 - computation overlap, 592
 - PCIe data movement interface, 328
- Documentation debt
 - data provenance, 162
- Domain adaptation
 - feature alignment, 454
- Domain gap
 - training-serving skew, 453, 454
- Domain randomization
 - bridging synthetic-real gap, 453
 - rendering trade-off, 454
- Domain-Specific Architecture, *see* DSA
- Dot product
 - appendix refresher, 916
 - definition, 188
- Double buffering
 - latency hiding, 629
- Double descent
 - overparameterized regime, 179
- DQN
 - real-time inference, 176
- DRAM
 - energy cost, 561
- Drift
 - covariate shift, 810
- Drift detection delay
 - label latency, 810
 - statistical monitoring, 809
- Dropout
 - regularization technique, 191
 - train-inference mismatch, 191
 - training vs. inference, 339
- DS-CNN
 - monitoring strategy, 831
- DSA
 - definition, 556
 - efficiency trade-off, 556
 - scaling law breakdown, 556
- Dual mandate, 4
- Dual number
 - forward mode AD, 314
- Duty cycle
 - accelerator utilization, 731
 - ML lifecycle maintenance, 83
- DVC
 - data versioning, 779
 - dataset versioning, 786
 - git semantics, 159
- DVFS
 - dynamic voltage and frequency scaling, 689
 - power-performance envelope, 633
 - warm-up measurement artifact, 653
- Dying ReLU problem
 - definition, 191
- Dynabench
 - dynamic evaluation, 703
- Dynamic batching
 - definition, 743
 - time window, 745
- Dynamic computation, 281
- Dynamic graph
 - definition, 300
 - eager execution, 299
 - runtime construction, 300
- Dynamic inference
 - real-time prediction, 719
- Dynamic kernel execution
 - runtime adaptation, 628
- Dynamic quantization
 - definition, 519
- Dynamic random access memory, *see* DRAM
- Dynamic selection
 - active learning, 446
 - curriculum learning, 445
 - definition, 435
 - training-time optimization, 445
- Dynamic voltage and frequency scaling, *see* DVFS
- Eager Execution
 - optimization limitations, 298
- Eager execution
 - define-by-run model, 299
 - definition, 294
 - optimization trade-off, 294
- Earth mover's distance
 - multi-dimensional drift, 810
- Edge accelerator
 - deployment selection, 59
- Edge AI
 - deployment patterns, 806
 - Oura Ring case study, 830
- Edge deployment
 - bandwidth constraints, 103
 - constraint triangle, 665
 - framework selection, 347
 - power constraints, 866
 - storage, 159
 - three problems reweighting, 347
 - TinyML frameworks, 312
- Edge hardware
 - resource limits, 59
- Edge inference
 - power tiers, 218
- Edge ML
 - autonomous vehicles, 60
 - bandwidth reduction, 56
 - data sovereignty, 56
 - definition, 56
 - deployment challenges, 57
 - distance penalty, 56
 - distributed processing, 57
 - industrial IoT, 60
 - latency and bandwidth, 27
 - latency benefits, 40
 - privacy benefits, 57
 - smart retail, 60
- Edge orchestration
 - fleet challenges, 59
- Edge serving
 - embedded function calls, 721
- Edge TPU
 - inference efficiency, 218
 - inference latency, 641
 - operator coverage, 676
- Effective Sample Size
 - label noise penalty, 440
- Efficiency
 - Jevons paradox, 24
- EfficientNet
 - algorithmic efficiency, 24
- Einstein summation convention
 - definition, 916
- EL2N
 - definition, 441
 - proxy transferability, 441
- Element-wise operation
 - memory bandwidth, 336
- ELIZA
 - brittleness, 7
- ELT

- iteration speed, 137
- load first, 138
- Embedded GPU accelerator
 - edge deployment, 59
- Embedding
 - definition, 267
 - etymology, 267
 - memory access pattern, 267
 - table, 267
- Embedding Lookup
 - memory-bound operation, 598
- Embedding pruning, 444
- Embedding table
 - capacity bottleneck, 365
 - memory capacity bound, 45
- Embodied carbon
 - hardware manufacturing, 869
- Emergent behavior
 - definition, 106
- Encoder-decoder
 - definition, 275
- Encryption
 - data lifecycle protection, 872
- End-to-end benchmark
 - complete workflow
 - measurement, 652
- Energy
 - budget, mobile inference, 754
 - data movement cost, 559
 - training scale, 185
- Energy consequence
 - definition, 588
- Energy consumption
 - hardware platforms, 205
 - training costs, 865
- Energy dividend
 - quantization efficiency, 507
- Energy efficiency
 - TFLOPS per watt, 635
 - training cost accounting, 672
- Energy harvesting
 - milliwatt threshold, 64
- Energy hierarchy
 - reference sources, 930
- Energy of transmission
 - local vs. cloud, 44
- Energy per inference
 - binding constraint, 45
 - paradigm comparison, 66
- Energy tax
 - definition, 20
- Energy wall
 - battery constraints, 44
- Energy-movement invariant, 116
 - data locality, 888
 - decode phase, 768
 - distributed training, 422
- Ensemble learning
 - accuracy vs. deployment
 - trade-off, 95
 - deployment cost, 96
- Entanglement
 - ML system coupling, 782
- Entropy
 - appendix refresher, 913
 - definition, 913
- Environment parity
 - production deployment, 775
- Environmental impact
 - carbon accounting, 869
 - ML systems, 865
- Epoch
 - dataset iteration, 216
 - training cycle, 384
 - training cycles, 216
- Equal opportunity
 - definition, 856
- Equalized odds
 - definition, 856
 - postprocessing, 856
- Equivariance
 - translation, 245
- Error Signal
 - backward propagation, 210
- Ethics and governance
 - definition, 30
- ETL
 - reprocessing cost, 137
- EU AI Act
 - risk classification, 861
 - systems requirements, 861
- Evaluation pipeline
 - validation metrics, 384
- Evasion attack
 - inference adversarial
 - perturbation, 873
- Exactly-once semantics
 - streaming, 161
- Excessive data movement
 - definition, 604
- Execution problem
 - definition, 297
- Expected calibration error
 - confidence-accuracy gap, 697
- Expected impact
 - definition, 601
- Expected Model Change
 - active learning query strategy, 446
- Experiment tracking
 - tools and practices, 98
- Expert systems, 8
 - knowledge engineering, 177
 - maintenance cost, 177
- Explainability
 - architecture trade-off, 860
 - definition and purposes, 860
 - post-hoc methods (SHAP, LIME), 860
- Exploding gradient problem, 270
 - training instability, 210
- Exponential unit
 - definition, 570
- External tensor payloads
 - definition, 615
- Eyeriss
 - row-stationary dataflow, 595
- F1 score
 - precision-recall harmonic mean, 661
- Face ID
 - privacy architecture, 63
- Facebook AI Similarity Search
 - selection infrastructure, 459
- Fairness
 - accuracy trade-off, 848
 - governance objective, 822
 - individual vs. group, 848
 - mathematical definitions, 848
- Fairness metrics
 - demographic parity, 856
 - equal opportunity, 856
 - equalized odds, 856
 - impossibility theorem, 856
 - loan approval case study, 856
- FAISS, *see* Facebook AI Similarity Search
- Fallacy of peak performance
 - GPU utilization gap, 648
- False Accepts per Hour, 120
- False negative rate
 - fairness impact, 857
- False positive rate
 - always-on systems, 120
- FarmBeats
 - connectivity constraint, 28
- Fault tolerance
 - training checkpoint strategy, 673, 895
- Feature attribution
 - debugging techniques, 819
- Feature Computation
 - hybrid patterns, 140
 - pipeline vs. loader, 140
- Feature computation
 - placement, 140
- Feature consistency
 - training-serving alignment, 775
- Feature contract
 - schema specification, 822
- Feature coverage
 - deployment variation, 700
- Feature Engineering
 - domain knowledge, 143
- Feature engineering
 - classical ML, 177
 - definition, 143
- Feature engineering bottleneck
 - definition, 9
- Feature extraction
 - hierarchical, 247
- Feature importance
 - prediction attribution, 860
- Feature learning
 - hierarchical, 175
- Feature map, 244
- Feature store
 - consistency, 143
 - consistency guarantee, 143
 - definition, 161
 - offline vs. online stores, 161
 - point-in-time retrieval, 160
 - scale, 787
 - streaming updates, 161
 - training-serving bridge, 160
 - training-serving consistency, 787
 - Uber Michelangelo origin, 787
- Federated averaging
 - data minimization, 874
- Federated learning
 - data minimization, 874
 - privacy preservation, 874
- Feedback Loop
 - multi-scale temporal structure, 105
- Feedback loop
 - problem amplification, 863
 - recommendation amplification, 846
 - self-reinforcing bias, 784
 - YouTube case study, 785
- Feedforward network
 - architecture, 196
- Feeding problem
 - definition, 114
- Feeding tax
 - definition, 114
- Field-programmable gate array, *see* FPGA
- FixMatch
 - label-compute trade-off, 449
- FlashAttention
 - 2, improved parallelism, 409
 - 3, FP8 tensor cores, 409
 - appendix refresher, 921
 - framework fusion example, 298
 - I/O-aware algorithm, 406
 - I/O-aware design, 383
 - IO complexity reduction, 407
 - IO-aware, 284
 - memory bandwidth
 - optimization, 406
 - memory optimization, 599
 - memory reduction, 284
 - origin and engineering insight, 406
 - SRAM tiling, 528
 - tiled attention implementation, 621
 - tiling strategy, 407
- FlatBuffers
 - etymology, 725
 - zero-copy serialization, 725, 726
- Flaw of averages, 854
- Flax
 - JAX neural network library, 339
- Fleiss' kappa
 - inter-annotator agreement, 150
- Floating point
 - FP32 precision, 219
- Floating-point operations per second, *see* FLOP/s
- Floating-point unit
 - history, 554
 - precision bottleneck, 554
- FLOP/s
 - peak vs. achieved, 649
- Flow rate

- definition, 114
- Food and Drug Administration
 - SaMD constraint, 101
- Forced alignment
 - automated labeling, 152
 - speech labeling, 152
- Forgetting events
 - coreset selection, 441
 - pruning trade-off, 441
- Forward pass
 - compute operations, 388
 - kernel dispatch, 353
 - memory management, 388
- Forward propagation
 - computation process, 201
 - layer computation, 367
 - matrix operations, 202
- Foundation model
 - compute demand, 29
 - definition, 452
 - homogenization risk, 452
- Four pillars framework
 - data engineering, 113
- Four-fifths rule
 - adverse impact threshold, 849
- FP16
 - appendix refresher, 939
 - dynamic range limitation, 400, 401
 - half precision, 219
 - subnormal flushing, 404
 - training precision, 559
- FP32
 - gradient computation, 559
 - numerical precision, 219
- FP32 master weights
 - mixed-precision training, 361
 - optimizer stability, 402
- FP8
 - E4M3 and E5M2 formats, 404
 - hardware acceleration, 404
- FPGA
 - reconfigurability trade-off, 627
- Framework profiler
 - performance analysis tool, 655
 - timeline analysis, 764
- Framework selection
 - constrained optimization, 348
 - trade-off analysis, 348
- Fraud Detection
 - ML vs. rule-based, 85
- Frequency boosting
 - warm-up measurement artifact, 653
- Freshness debt
 - distribution divergence, 163
- Frontier training
 - definition, 26
- Fully-connected layer, 239
 - dense connectivity, 194
- Function unit
 - definition, 570
- Fuse operations
 - definition, 623
- Fused execution
 - definition, 615
- Gabor filters
 - learned equivalents, 178
- GAN
 - synthetic data generation, 453
- Garbage collection pause
 - latency variability, 707
- Gather
 - definition, 565
- Gating mechanism, 269
 - sequence models, 275
- GDDR
 - off-chip DRAM variant, 591
- GDPR
 - Article 22 automated decisions, 861
 - Article 22 requirements, 862
 - compliance, cloud deployment, 53
 - data collection requirements, 127
 - data minimization principle, 874
 - ML erasure constraint, 53
- GELU, 370
- GEMM
 - appendix refresher, 916
 - arithmetic intensity, 336, 933
 - hardware utilization, 296
 - matrix multiplication, 336
 - ML performance predictor, 621
- Gender classification
 - demographic disparities, 848
- Gender Shades
 - algorithmic audit methodology, 848
 - demographic fairness study, 697
 - disaggregated evaluation, 848
- Generalization
 - vs. overfitting, 216
- Generalization gap
 - definition, 29
- Generative adversarial network, *see* GAN
- Generative AI
 - benchmark evolution, 694
- GLUE
 - benchmark saturation, 659
 - language understanding benchmark, 659
- Glue code
 - integration overhead, 784
- Goodhart's law
 - etymology, 642
 - metric optimization, 845
 - metric trap, 642
 - proxy decoupling, 846
- Google Flu Trends
 - proxy drift, 5
- Google TPUv4
 - definition, 581
- GPipe
 - pipeline parallelism, 422
- GPT-2, 234
 - arithmetic intensity, 236
 - autoregressive bottleneck, 45
 - bandwidth-bound decode, 21
 - data selection, 436
 - memory bandwidth, 886
 - training lighthouse, 365
- GPT-3, 870
 - training compute, 365
 - training cost, 669
 - training energy, 11
 - training scale, 10, 50
 - weight storage, 156
- GPU, 865
 - batching utilization, 743
 - etymology, 4
 - history, 555
 - memory hierarchy, 808
 - parallel computation, 205
 - training, power budget, 218
 - utilization monitoring, 808
- GPU backend
 - CUDA implementation, 355
- GPU boost clock
 - benchmark variability, 674
- GPU Interconnect
 - switching fabric, 593
- GPU memory
 - training constraints, 389
- GPU profiler
 - kernel-level analysis, 395
 - system-level analysis, 395
 - timeline analysis, 764
- GPU utilization
 - batch size dependency, 743
- GPU vs. CPU
 - economics, 766
- Graceful degradation
 - data pipelines, 134
 - overload handling, 738
 - triggers, 893
- Gradient
 - backpropagation, 208
 - bias gradients, 211
 - chain rule, 209
 - error signal flow, 210
 - flow in residual networks, 271
 - partial derivatives, 210
 - vanishing and exploding, 210
 - weight gradients, 211
- Gradient accumulation
 - definition, 211
 - framework behavior, 376
 - memory efficiency, 410
 - memory trade-off, 315
 - memory-compute trade-off, 410
- Gradient checkpointing
 - appendix refresher, 921
 - memory reduction, 377
- Gradient compression
 - definition, 409
- Gradient descent
 - definition, 215
 - etymology, 372
 - full-batch computation, 372
 - gradient noise, 372
 - parameter optimization, 372
 - training foundation, 372
- Gradient flow
 - tensor detachment, 319
- Gradient function node
 - computation graph, 317
- Gradient hook
 - inspection and modification, 319
- Gradient inspection, 319
- Gradient instabilities
 - training failure, 175
 - training failure mode, 175
- Gradient-based pruning
 - definition, 490
- GraNd
 - gradient-based coreset scoring, 441
- Graph break
 - causes and detection, 303
- Graph Compilation
 - JIT overhead, 740
- Graph compilation
 - eager-mode capture, 306
 - optimization, 762
- Graph Execution
 - optimization advantages, 298
- Graph Neural Network
 - irregular computation, 604
- Graph optimization
 - computation graph restructuring, 624
 - definition, 622
- Graph retention
 - multiple backward passes, 319
- Graphcore
 - IPU, 573
- Graphics processing unit, *see* GPU
- Great Expectations
 - data validation, 131
- Greedy decoding
 - LLM generation, 756
- Green AI
 - compute reporting, 865
 - efficiency as metric, 865
- Green500
 - energy efficiency ranking, 644
 - HPC energy efficiency ranking, 643
- Ground truth
 - definition, 148
 - proxy limitation, 148
- Group-equivariant CNN, 246
- gRPC
 - inference protocol, 726
- GRU
 - efficiency, 275
 - parameter efficiency, 275
- GTX 580
 - AlexNet training, 10
- Guardrail metric
 - A/B test constraint, 802
- Gustafson's law
 - appendix refresher, 934
- Gustafson, John, *see also* Gustafson's law
- Gzip
 - decompression speed, 158

- H100, 404
- Half-life
 - model decay, 795
- Hardware abstraction
 - framework design, 324
 - two dimensions (data, execution), 324
- Hardware abstraction layer
 - portability, 637
- Hardware acceleration
 - definition, 550
- Hardware accelerator, *see* Accelerator
- Hardware balance, 584
- Hardware cost
 - deployment spectrum, 47
- Hardware lottery
 - architectural bias, 692
 - benchmark bias, 692
- Hardware Mapping
 - combinatorial search space, 605
 - definition, 603
- Hardware mapping
 - hybrid, layer-specific strategy, 622
- Hardware selection
 - workload matching, 636
- Hardware specialization
 - evolution, 554
- Hardware spectrum
 - deployment platforms, 50
 - resource progression, 50
- Hardware sustainability
 - carbon footprint, 635
- Hardware-Software Co-design
 - definition, 551
- Hardware-software co-design
 - edge deployment, 892
- HBM
 - 3D die stacking, 591
 - bandwidth advantage, 281
 - bandwidth-cost trade-off, 561
 - capacity constraint, 353
 - definition, 281
 - memory bandwidth, 406, 769
 - throughput requirements, 562
- Headroom
 - tail latency, 716
- Healthcare AI
 - deployment gap, 89
- Hedging
 - tail tolerance, 737
- HELM
 - multi-dimensional evaluation, 699
- Herding
 - coreset selection, 441
- Heterogeneous computing
 - mobile SoC, 631
- Heuristic
 - kernel selection trade-off, 625
 - pruning, 486
- Hidden layer
 - definition, 212
- Hidden state
 - definition, 253
- Hierarchical processing
 - data flow, 72
- Hierarchical representation
 - depth advantage, 187
- Hierarchical Representation Learning, 286
- Hierarchical selection
 - distributed coreset, 466
- High bandwidth memory, *see* HBM
- High memory latency
 - definition, 605
- High off-chip bandwidth demand
 - definition, 605
- High-risk AI
 - classification criteria, 861
 - EU classification, 851
- Hinton, Geoffrey
 - backpropagation, 184
 - Turing Award, 175
- HiPAA
 - audit requirements, 876
- compliance, cloud deployment, 53
- ML system constraints, 862
- security safeguards, 53
- Hochreiter, Sepp, 275
- HOG
 - fixed vs. learned features, 177
- Holdout test set
 - performance evaluation, 798
- Host-Accelerator Communication
 - transfer bottleneck, 592
- Huang's law
 - GPU performance scaling, 556, 557
- Human-in-the-loop
 - confidence-based routing, 103
 - decision oversight, 853
- Hybrid architecture
 - combining paradigms, 72
 - voice assistant example, 79
 - wake-word detection, 54
- Hybrid machine learning, *see* Hybrid ML
- Hybrid ML
 - definition, 72
 - hierarchical processing, 72
 - integration strategies, 72
 - progressive deployment, 72
 - train-serve split, 72
- Hybrid tiling
 - definition, 620
- Hyperparameter
 - learning rate, 215
 - search cost, 95
 - tuning, 217
- Hypertension
 - ClinAIOPS case study, 834
- i.i.d. assumption
 - held-out evaluation limitation, 701
- I/O bottleneck
 - data loading, 135
- I/O-aware design
 - algorithm optimization, 409
- IB, *see* InfiniBand
- Idempotency
 - definition, 143
 - ML pipelines, 790
 - retry safety, 144
- IIoT
 - latency constraint, 56
- Illustrative latency
 - definition, 570
- im2col, 248
 - convolution to matrix, 566
 - data center vs. mobile, 276
 - memory trade-off, 276
 - memory-compute trade-off, 566
- Image cropping
 - racial bias, 848
- ImageNet
 - benchmark dataset, 123
 - competition progress, 179
 - data scale, 123
 - ILSVRC challenge progression, 696
 - macro benchmark reference, 655
 - scale, 22, 243
 - standardized dataset, 659
 - systems consequence, 243
- Immutable artifact
 - model registration, 790
- Immutable data
 - JAX, 344
- Impact on execution
 - definition, 604
- Imperative programming
 - framework heritage, 343
- Implementation strategy
 - definition, 570
- Importance sampling
 - gradient variance, 474
- In-memory database
 - feature serving, 159
- Incident response
 - ML system failures, 862
- ML-specific challenges, 817
- structured processes, 817
- Inductive bias
 - definition, 232
 - etymology, 232
 - hierarchy, 286
 - systems consequence, 232
- Industry 4.0
 - control loop latency, 60
- Inference
 - batch size selection, 219
 - compiler auto-tuning, 676
 - compiler, graph optimization framework, 676
 - deployment, 17
 - deployment phase, 218
 - engine, specialized, 759
 - etymology, 218
 - forward pass only, 206, 218
 - latency optimization, 48
 - latency requirements, 223
 - optimization techniques, 219
 - pipeline phase, 727
 - real-time, storage requirements, 159
 - serverless trade-off, 803
 - zero-copy, mobile optimization, 721
- Inference attack
 - privacy risk, 29
- Inference inflation
 - definition, 735
- Inference runtime
 - execution engine, 758
- Inference server
 - architecture, 724
 - serving architecture, 724
- Inference-time compute
 - reasoning models, 720
- InfiniBand
 - bandwidth tiers, 593
 - interconnect performance, 366
 - RDMA for ML training, 594
- Infinity Fabric
 - AMD interconnect, 593
- Information density
 - definition, 913
- Information roofline
 - definition, 907
- Information theory, 912
- Information-compute ratio
 - accuracy-cost frontier, 437
 - definition, 437
 - frontier, 438
- Infrastructure as code
 - ML systems, 807
- Initialization
 - glorot, variance scaling, 200
 - He initialization, 200
 - lazy, deferred compilation, 741
 - Xavier/Glorot, 200
- Input representation
 - definition, 623
- Input-stationary
 - dataflow strategy, 579, 611
- INT8
 - appendix refresher, 939
 - calibration dependency, 680
- INT8 quantization
 - bandwidth trade-off, 888
- Integration bottleneck
 - data movement cost, 561
- Intel
 - Sapphire Rapids, 581
- Intel 8087
 - floating-point coprocessor, 554
 - specialization pattern, 554
- Intel Gaudi 2
 - definition, 582
- Intensity regime
 - definition, 601
- Intentional data engineering
 - fairness evaluation, 853
- Inter-annotator agreement, 149
- Inter-token latency
 - throughput trade-off, 699

- Interaction deduplication, 444
- Interface dependency
 - ML system challenge, 783
- Intermediate representation
 - compiler pattern, 304
- Intermittent computing
 - power management, 65
- Interpretability
 - activation visualization, 226
- Interpretable model
 - transparency by design, 860
- Interrupt shielding
 - latency isolation, 724
- Intersectional analysis
 - combined attribute testing, 855
- Invariance testing
 - fairness verification, 849
- IOPS
 - storage performance, 154
- IoT
 - data wall, 57
 - deployment scale, 60
 - power budget constraints, 866
 - power-constrained environments, 684
- Iron law of ML systems
 - appendix refresher, 903, 934
 - definition, 18
 - deployment analysis, 43
 - equation, 18
 - hardware acceleration, 552
 - multiplicative savings from data selection, 437
 - production monitoring, 811
 - sequential execution, 903
 - synthesis, 884
 - total operations reduction via data selection, 434
 - workflow mapping, 87
- Iron law of training performance
 - definition, 363
 - utilization gap, 362
- Irregular computation patterns
 - definition, 604
- Irregular memory access patterns
 - definition, 605
- Irregular ML components
 - definition, 588
- Iteration tax
 - definition, 85
- Iterative pruning
 - definition, 487
- Jaccard similarity
 - deduplication, 443
- Jacobian matrix, 273
- JAX
 - automatic vectorization, 344
 - composable program transformations, 344
 - composable transformations, 324
 - device-parallel mapping, 344
 - functional transformations, 344
 - gradient transformation, 344
 - JIT compilation, 344
 - pure functions requirement, 344
 - transform-based AD, 324
 - XLA compilation, 345
- jax.grad
 - composable transformation, 344
- Jensen-Shannon divergence
 - distribution shift, 813
- Jetson AGX Orin
 - edge inference, 218
- Jevons paradox
 - inference demand, 717
- JIT compilation
 - fidelity vs. generality, 303
 - runtime optimization, 763
 - scripting, 304
 - shape specialization, 346
 - tracing, 303
- JSON
 - serialization overhead, 726
 - storage overhead, 140
- Jupyter Notebook
 - production risk, 792
- Just-in-time compilation, 307
- k-anonymity
 - privacy technique, 873
- k-center
 - coreset selection, 441
 - coverage guarantee, 441
- Karpathy, Andrej
 - Software 2.0, 4
- Kendall notation
 - queuing models, 735
- Kernel auto-tuning
 - algorithm selection, 759
- Kernel dispatch
 - BLAS kernel selection, 353
 - hardware selection, 336
- Kernel Fusion
 - hardware data movement, 614
- Kernel fusion, 317
 - bandwidth reduction, 317
 - definition, 298
 - eliminating intermediate allocations, 317
 - layer fusion, 760
 - memory efficiency, 614
 - memory traffic reduction, 614
 - optimizing memory-bound ops, 298
 - reducing memory writes, 608
 - static graph optimization, 301, 303
- Kernel launch
 - fixed overhead, 743
 - serving overhead, 743
- Kernel Launch Latency
 - serial overhead, 552
- Kernel profiler
 - hardware-level analysis, 655
- Kernel scheduling
 - hardware utilization, 629
- Kernel selection
 - definition, 622
 - hardware-specific implementation, 625
- Key characteristic
 - definition, 567
- Key management
 - encryption infrastructure, 872
- Keyword spotting, 235
 - automated labeling, 152
 - definition, 22
 - energy constraint, 87
 - sub-megabyte models, 886
 - TinyML archetype, 46
- Killer microseconds
 - latency gap, 729
- Knowledge acquisition bottleneck
 - definition, 8
 - human throughput, 9
- Knowledge distillation
 - benchmark evaluation, 665
 - definition, 495
 - efficiency gains, 865
 - etymology, 495
 - information density, 455
 - student model, 495
 - teacher model, 495
 - teacher-generated targets, 455
 - temperature parameter, 455, 496
- Kolmogorov-Smirnov test
 - continuous drift, 810
 - covariate shift detection, 701
 - drift detection, 104, 130
 - feature distribution, 789
- Kubernetes
 - ML orchestration, 807
- Kullback, Solomon, *see also* Kullback-Leibler divergence
- Kullback-Leibler divergence
 - appendix refresher, 912
 - definition, 133
 - distillation loss, 497
 - distribution comparison, 134, 814
- Kung, H.T.
 - systolic array inventor, 576
- KV cache
 - definition, 265
 - linear growth, 265
 - LLM memory, 757
 - LLM serving bottleneck, 809
 - memory capacity, 769
 - memory scaling, 265, 757
 - multi-turn conversations, 751
 - offloading, 757
 - sequence length scaling, 757
 - session reuse, 751
 - transformer memory pressure, 621
- l-diversity
 - privacy technique, 873
- L0 norm
 - parameter count, 486
- Lab-to-clinic gap
 - monitoring requirement, 104
- Lab-to-field gap
 - data distribution shift, 93
- Label propagation
 - graph-based semi-supervised, 449
- Label quality
 - consensus mechanisms, 149
 - systematic checks, 149
- Label quality drift
 - detection, 133
 - silent corruption, 133
- Label shift
 - definition, 133
- Label Studio
 - data quality, 787
- Labelbox, 150
- Laboratory-to-deployment gap
 - performance misalignment, 692
- Ladder of abstraction, 296
- LAPACK
 - framework foundation, 296
 - higher-level numerical routines, 296
- Large language model, *see* LLM
- Late correction cost
 - definition, 85
- Latency
 - bandwidth trade-off, 937
 - decision thresholds, 47
 - deployment constraint, 42
 - deterministic, 723
 - edge deployment, 27
 - etymology, 27
 - first-token, LLM responsiveness, 699
 - inference real-time requirement, 677
 - inference vs. user-perceived, 769
 - mean, monitoring pitfall, 769
 - p50, median response time, 727
 - p95, percentile target, 727
 - percentile, p50 p95 p99 reporting, 660
 - percentiles (p50, p95, p99), 727
 - real-time constraints, 223
 - responsiveness constraint, 42
 - selection, dynamic selection bottleneck, 458
 - serving constraint, 716
 - throughput trade-off, 770
 - vs. throughput trade-off, 29
- Latency budget
 - definition, 727
 - end-to-end analysis, 805
 - optimization objectives, 727
 - request lifecycle breakdown, 727
 - tail latency constraint, 888
- Latency jitter
 - resource contention, 723
- Latency vs. throughput
 - accelerator design, 559
 - paradigm trade-offs, 68
 - trade-offs, 47
- Layer
 - hidden, 193

- input, 193
- organization, 193
- output, 193
- Layer fusion, *see* Operator fusion
- Layer normalization, 274
 - Transformer, 263
- Layer sensitivity
 - precision tolerance, 761
- LayerNorm
 - kernel fusion target, 601
 - memory-bound operation, 598
- Layerwise quantization, *see* Quantization
- Lazy Evaluation
 - data processing, 145
- Learnability, 238
- Learnability gap, 238
- Learning rate
 - batch size coupling, 215
 - cosine decay, 377
 - etymology, 215
 - scheduling, 377
 - selection criteria, 217
 - update magnitude, 215
 - warmup, 377
- Learning rate scheduling
 - definition, 377
- LeCun, Yann, 243
 - commercial deployment, 243
 - LeNet, 243
 - Turing Award, 175
- Leibler, Richard, *see also* Kullback-Leibler divergence
- Leiserson, Charles E.
 - systolic array inventor, 576
- LeNet
 - architecture comparison, 223
- LeNet-5, 269
- Li, Fei-Fei
 - ImageNet creation, 123
- LiDAR
 - data volume, 28
- Light barrier
 - latency floor, 42
 - safety-critical systems, 52
- Lighthouse example
 - definition, 598
- Lighthouse model
 - bottleneck probe, 885
 - definition, 21
 - reference workloads, 41
- LIME
 - local explanations, 860
- Limited interconnect bandwidth
 - definition, 604
- Limited on-chip storage
 - definition, 605
- Lineage tracking
 - model provenance, 790
 - model registry, 810
- Linear scaling rule
 - large batch training, 392
- Linear transformation
 - layer computation, 202
- Linnainmaa, Seppo
 - automatic differentiation, 184
- LINPACK
 - matrix operations benchmark, 643
 - narrow benchmark, 643
- List price
 - definition, 582
- Little's law
 - appendix refresher, 935
 - capacity planning, 734
 - concurrency calculation, 734
- Little, John, *see also* Little's law
- Llama
 - bandwidth-bound decode, 21
 - data selection, 436
 - KV-cache footprint, 809
 - memory-bound inference, 45
- Llama 2
 - 70-billion-parameter memory-bound workload, 889
- Llama 3
 - 8-billion-parameter serving case study, 767
- LLM, 234, 436, 865
 - KV-cache serving, 809
 - labeling assistance, 151
 - training scale, 50
- LLM performance metrics
 - definition, 756
- LLM serving
 - 8-billion-parameter case study, 767
 - production case study, 767
 - token generation, 755
- Load balancer
 - serving infrastructure, 722
- Load shedding
 - retry coordination, 738
- Locality crossover
 - worked example, 59
- Locality of reference
 - ML workload violation, 86
- Locality-sensitive hashing
 - near-duplicate detection, 443
 - probabilistic bucketing, 443
- Log-sum-exp trick
 - appendix refresher, 913
 - numerical stability, 207
- Logic unit cost
 - activation functions, 192
- Logits
 - appendix refresher, 913
 - classification output, 732
 - inference optimization, 192
 - logit function, 913
 - raw network scores, 191
- Loop Ordering
 - computational efficiency, 606
- Loss Function
 - error measurement, 206
 - optimization target, 206
- Loss function
 - cross-entropy, 206
 - etymology, 201
 - landscape geometry, 201
- Loss landscape
 - nonconvex optimization, 215
- Loss Scaling
 - gradient underflow prevention, 320
- Loss scaling
 - appendix refresher, 939
 - gradient underflow prevention, 401
- Lottery ticket hypothesis
 - compression trade-off, 698
 - etymology, 493
- Low-rank factorization
 - definition, 498
- LPDDR
 - low-power DRAM variant, 591
- LSTM
 - compute overhead, 275
 - gating mechanism, 275
- M/D/1 queue
 - deterministic service, 735
- M/M/1 queue
 - Erlang origin, 735
 - wait time formula, 735
- M/M/c queue
 - multi-server, 736
- MAC
 - definition, 241
 - energy cost, 241
 - multiply-accumulate operation, 183
 - neural computation, 183
- Machine learning
 - definition, 8
- Machine learning compiler
 - definition, 623
- Machine learning framework, *see* ML framework
- Machine learning lifecycle, 82
 - definition, 82
- iteration cycles, 84
- storage requirements, 159
- time allocation across stages, 83
- Machine learning systems
 - definition, 12
- Macro benchmark
 - subsystem evaluation, 652
- Magnitude-based pruning
 - definition, 486
- Manifold hypothesis, 238
 - learnability, 238
- MapReduce
 - data locality, 145
- Masked Language Modeling
 - pretext task, 450
- Masked operations
 - definition, 565
- Materialized view
 - training-serving skew, 131
- Matrix engine
 - definition, 581
- Matrix multiplication
 - dense operations, 388
 - forward propagation, 202
 - GEMM optimization, 203
 - neural network workhorse, 565
 - training operations, 367
- Matrix operation
 - hardware acceleration, 566
 - hierarchical decomposition, 565
 - neural network workloads, 572
- Matrix tiling, 280
- Maturity level
 - evolutionary stage, 826
- Maximum mean discrepancy
 - covariate shift detection, 701
- McCrory, Dave, *see also* Data gravity
- McCulloch-Pitts neuron, 237
- Mean time between failures
 - fleet-scale reliability, 895
- Mean time to failure
 - component reliability, 895
- Measurable property
 - responsibility requirements, 878
- Measurement framework
 - data selection metrics, 470
- Measurement variability
 - sources in ML systems, 646
- Mechanical Turk
 - labeling scale, 126
- Medical AI
 - performance metrics, 96
- Membership inference attack
 - privacy testing, 873
 - privacy validation, 874
- Memory
 - capacity, LLM throughput, 769
 - capacity-bound workload, 267
 - constraints, mobile RAM, 754
 - locking, mlock, 723
 - reuse, activation buffers, 220
 - sharing, GPU multi-model, 742
- Memory Access
 - irregular patterns, 587
- Memory access
 - random, 277
 - sequential, 277
 - strided, 277
- Memory access pattern
 - definition, 596
- Memory Bandwidth
 - architectural trade-offs, 591
 - scaling trends, 583
- Memory bandwidth
 - activation function bottleneck, 371
 - bottleneck, 12
 - bottleneck mitigation, 409
 - bound workload, 266
 - LLM bottleneck, 768
 - LLM latency, 769
 - memory wall, 43
- Memory coalescing
 - GPU optimization, 613
 - tensor layout impact, 613
- Memory fetching

- definition, 613
- Memory footprint
 - computational requirements, 197
 - training overhead, 210
- Memory fragmentation
 - KV cache, 751
- Memory hierarchy
 - appendix refresher, 937
 - domain-specific, 558
 - ML training tiers, 387
 - speed-capacity trade-off, 589
- Memory layout
 - NCHW vs. NHWC, 336
 - row-major vs. column-major, 327
 - stride patterns, 327
- Memory management
 - caching allocators, 322
 - definition, 623
 - device placement, 328
 - inference efficiency, 220
 - VRAM requirements, 389
- Memory mapping
 - on-demand model loading, 741
- Memory planning
 - compiler phase, 625
 - definition, 622
 - tensor allocation, 762
- Memory storage order
 - definition, 613
- Memory usage
 - definition, 620
- Memory wall
 - bandwidth divergence, 42
 - compute-memory gap, 42
 - data movement bottleneck, 181
 - execution strategy impact, 297
 - framework execution, 297
 - impact on GPU utilization, 648
 - neural network impact, 181
- Memory-aware tensor layouts
 - definition, 621
- Memory-bandwidth bound
 - precision impact, 760
- Memory-Bound
 - roofline classification, 596
- Memory-bound
 - activation tensors, 730
 - arithmetic intensity, 181
 - operation classification, 298
- Memory-computation tradeoff
 - training systems, 380
- Metric
 - etymology, 660
 - intransitivity, 660
- MFCC
 - audio features, 121, 143
 - dimensionality reduction, 143
- Micro benchmark
 - component isolation, 652
 - diagnostic purpose, 653
- Micro-benchmark suite
 - operator-level testing, 653
- Microcontroller
 - memory ceiling, 64
 - ML deployment, 67
 - TinyML substrate, 27
- Microservice
 - cloud model serving, 721
 - model serving, 721
- MIG
 - hardware isolation, 742
 - multi-model serving, 742
- MinHash
 - distributed, cross-shard
 - deduplication, 466
 - near-duplicate detection, 443
 - signature compression, 443
- Mini-batch
 - gradient descent, 215
- Mini-batch gradient descent, *see* Stochastic gradient descent
- Mini-batch processing
 - accelerator efficiency, 372
- Mixed-precision
 - layer-wise precision assignment, 665
- Mixed-precision Adam
 - storage model, 426, 946
- Mixed-Precision Computing
 - training and inference, 580
- Mixed-precision training
 - benchmarking comparability, 670
 - definition, 361
 - FP16 and FP32, 670
 - FP16 vs. FP32 trade-offs, 320
 - FP16/BF16, 383, 401
 - loss scaling, 401
 - memory savings, 400
 - trade-off, 361
- Mixture-of-experts, *see* MoE
- MixUp
 - image augmentation, 452
- ML Accelerator
 - definition, 559
- ML compiler
 - compilation pipeline, 624
 - definition, 622
 - vs. traditional compiler, 623
- ML development lifecycle
 - vs. traditional software, 26
- ML failure
 - data dominance, 164
- ML framework
 - compiler analogy, 294
 - compiler for the silicon contract, 295
 - compiler role, 41
 - definition, 295
 - differentiation problem, 294
 - dynamic memory management, 381
 - execution problem, 294
 - hardware abstraction problem, 294
 - historical evolution, 296
 - on-device deployment, 63
 - platform comparison, 342
- ML inference benchmark
 - definition, 675
- ML node
 - definition, 775
- ML system benchmark
 - definition, 649
- ML systems hierarchy
 - four layers, 15
- ML test score
 - production readiness, 825
- ML training benchmark
 - convergence throughput
 - scalability, 668
 - definition, 669
- ML workflow
 - systematic framework, 82
- MLCommons
 - nonprofit benchmark consortium, 683
 - open submission, 683
- MLflow
 - experiment tracking, 803
 - model versioning, 156
- MLOps, 30
 - anti-patterns, organizational failures, 827
 - centralized mlops team, organizational pattern, 828
 - definition, 774, 775
 - DevOps comparison, 775
 - emergence, 774
 - federated, embedded engineers, 828
 - production deployment challenges, 774
- MLOps fallacies
 - automated retraining sufficiency, 836
 - automatic consistency, 836
 - DevOps equivalence, 836
- MLOps pitfalls
 - alert routing, 837
- monitoring overconfidence, 837
- one-time deployment, 836
- organizational neglect, 836
- MLP
 - definition, 237
 - GEMM-dominated computation, 621
- MLPerf
 - founding, 643
 - founding and purpose, 643
 - inference benchmarking, 602
 - power, ML energy measurement, 643
- MLPerf client
 - local generative AI workloads, 684
- MLPerf Inference
 - benchmark family evolution, 683
 - MultiStream scenario, 755
 - offline scenario, 755
 - server scenario, 752
 - server to edge evaluation, 644
 - serving scenarios, 752
 - SingleStream scenario, 753
- MLPerf Mobile
 - on-device inference, 684
- MLPerf Power
 - energy efficiency measurement, 644
- MLPerf Tiny
 - embedded AI benchmark, 684
 - microcontroller benchmarks, 644
- MLPerf Training
 - data center multi-node scaling, 644
 - standardized framework, 669
- mmap
 - model loading, 741
- MMLU
 - format limitation, 699
 - multitask language understanding, 699
- MNIST, 238
 - dataset saturation example, 702
 - digit recognition, 195
- Mobile assistant
 - definition, 27
- Mobile machine learning, *see* Mobile ML
- Mobile ML
 - battery constraints, 61
 - computational photography, 63
 - definition, 61
 - energy budget, 62
 - energy constraints, 40
 - health monitoring, 64
 - memory bandwidth limits, 63
 - NPU inference, 63
 - resource constraints, 27
 - thermal envelope, 61
 - voice recognition, 64
- Mobile SoC
 - architecture evolution, 631
- MobileNet, 235
- MobileNetV2, 854
 - definition, 22
 - deployment validation example, 641
 - depthwise separable convolutions, 46
 - efficiency gains, 46
 - efficiency under constraint, 886
 - latency-constrained deployment, 250
 - mobile power reduction, 61
 - workflow constraints, 86
- MoCo
 - queue-based dictionary, 451
- Model audit
 - high-stakes systems, 810
- Model benchmarking
 - definition, 641
 - multi-dimensional evaluation, 696
- Model caching
 - container embedding, 741
- Model card

- documentation standard, 854
- standardized documentation, 854
- Model collapse
 - recursive synthetic training, 455
 - tail distribution compression, 455
- Model compression
 - accuracy-efficiency trade-off, 484
 - definition, 27, 480, 481
 - efficient model representation, 485
 - iterative workflow, 97
 - mobile vision, 61
 - multi-dimensional
 - benchmarking, 680
 - workflow role, 96
- Model debugging
 - decision tree, 818
 - probabilistic failures, 818
 - root cause analysis, 818
- Model decay
 - definition, 810
- Model deployment
 - packaging and serving, 800
- Model extraction
 - model artifact security, 873
- Model FLOPs utilization
 - appendix refresher, 907
 - definition, 394
 - origin, 393
 - PaLM benchmark, 393
- Model governance
 - compliance objective, 822
 - proactive policies, 821
 - transparency and compliance, 821
- Model inversion attack
 - feature reconstruction, 874
- Model loading
 - full loading, 741
- Model Memory Pressure
 - architecture-specific, 594
- Model monitoring
 - drift detection, 809
 - performance tracking, 809
- Model Optimization
 - format conversion, 803
 - framework comparison, 803
- Model Parallelism
 - training strategy, 672
- Model parallelism
 - layer-wise partitioning, 420
 - memory constraints, 420
 - multi-accelerator scaling, 630
 - pipeline bubbles, 365
- Model parameter
 - storage requirements, 156
- Model performance metrics
 - definition, 104
- Model quantization, *see* Quantization
- Model registry
 - centralized artifact management, 803
 - provenance tracking, 160
 - version management, 790
 - version promotion, 790
- Model serialization
 - state dictionary pattern, 340
- Model serving
 - definition, 718
 - paradigm shift, 716
- Model size
 - parameter count, 197
- Model type
 - definition, 596
- Model validation
 - candidate selection, 798
 - definition, 98
 - predeployment assessment, 798
- Model versioning
 - artifact tracking, 790
 - requirements, 156
- Model virtualization
 - lifecycle management, 742
- Model weights
 - data governance obligations, 878
 - definition, 5
- Model-centric AI
 - architectural innovation, 701
- Model-infrastructure interface
 - environment parity, 775
- Model-specific execution needs
 - definition, 604
- Model-specific memory needs
 - definition, 605
- Module abstraction
 - hierarchical composition, 338
 - PyTorch nn.Module, 338
- Module hook
 - forward and backward, 341
- MoE
 - definition, 531
 - dynamic computation paths, 588
- Momentum
 - memory overhead, 373, 374
 - optimization physics, 374
- Monitoring
 - health, replica readiness, 723
 - responsible operations, 862
- Monitoring ROI
 - calculation, 817
- Moore's law
 - ML systems consequence, 556
 - slowdown, 556
 - slowdown impact, 557
 - vs. AI growth, 23
- Moravec's paradox
 - compute implications, 9
- Multi-chip scaling
 - beyond single accelerator, 630
 - communication overhead, 631
- Multi-head attention, 262
- Multi-instance GPU, *see* MIG
- Multi-model serving
 - GPU memory management, 742
- Multilayer perceptron, *see* MLP
- Multiply-Accumulate
 - MAC unit, 576
- MultiStream
 - synchronized sensor fusion, 684
- MYCIN
 - expert system, 8
- NAS
 - bi-level optimization, 501
 - bi-level optimization cost, 501
 - definition, 501
 - hardware-aware, 501
 - search cost, 481
- Naïve execution
 - definition, 615
- NCHW
 - channels-first layout, 608, 613
- Netflix
 - prize, production gap, 96
- Network depth
 - training difficulty, 270
- Network interface card
 - node interconnect, 593
- Network RTT
 - latency budget, 805
- Network topology
 - layer organization, 195
- Neural architecture search, *see* NAS
- Neural Network
 - artificial neuron, 182
 - depth vs. width trade-off, 187
- Neural network
 - capacity vs. cost trade-off, 183
 - feedforward, 196
 - layers, 193
 - perceptron, 184
- Neural network application
 - definition, 565
- Neural processing unit
 - energy efficiency, 63
 - mobile devices, 63
- Neuron
 - McCulloch-Pitts model, 182
- Next-Token Prediction
 - pretext task, 450
- NHWC
 - channels-last layout, 608, 612
- NHWC vs. NCHW
 - performance impact, 613
- NIC, *see* Network interface card
- NN
 - dominant MAC pattern, 563
- nn.Module
 - hierarchical composition, 340
 - parameter discovery, 338
 - parameter management, 338
 - registered buffer, 338
 - train vs. eval modes, 339
- No free lunch theorem, 238
 - architecture selection, 238
- Non-maximum suppression
 - irregular memory access, 632
- Non-uniform memory access, *see* NUMA
- Nonlinearity
 - XOR problem, 195
- Normalization
 - batch, 273
 - feature scaling, 142
 - layer, 274
 - parameter persistence, 142
 - RMSNorm, 273
- Notebook to production
 - handoff challenges, 822
- Novelty Effect
 - A/B testing bias, 802
- NPU
 - efficiency trade-off, 631
 - inference benchmarking
 - considerations, 676
 - mobile SoC, 631
 - neural network acceleration, 667
 - on-device strategy, 573
- Nsight Compute
 - profiling tool, 332
- Nsight Systems
 - profiling tool, 332
- Nucleus sampling
 - top-p, 756
- NUMA
 - inference latency, 763
 - memory locality, 763
- Numerical precision optimization, *see* Quantization
- NumPy
 - framework foundation, 296
 - framework lineage, 297
- NVIDIA
 - A100, 582
 - Ampere architecture, 580
 - GeForce 256, 555
 - H100, 560
 - V100, 582
- NVIDIA A100
 - definition, 581
- NVIDIA Accelerator
 - definition, 597
- NVIDIA GTX 580
 - AlexNet memory constraint, 179
- NVIDIA H100
 - definition, 582
- NVIDIA Jetson
 - edge trade-off, 93
- NVIDIA Nsight
 - systems and compute profiling, 655
- NVIDIA Nsight Systems, 397
- NVLink
 - bandwidth hierarchy, 329
 - definition, 593
 - GPU interconnect, 421, 593
 - gradient synchronization, 421
 - intra-node scaling, 580
 - PCIe wall, 581
 - training interconnect, 366, 421
 - training scaling, 593
- NVMe
 - data pipeline, 157
 - GPU utilization, 155
 - storage bandwidth, 387
- Object detection

- bounding boxes, 148
- Observability
 - system state inference, 809
- Observable degradation
 - continuous measurement, 777
- Offline capability
 - paradigm comparison, 68
- Offline scenario
 - maximum throughput mode, 684
- OLAP, 154
- OLTP, 154
- On-call practice
 - ML systems, 820
- On-call rotation
 - ML-specific requirements, 820
- On-chip memory
 - definition, 590
- One-hot encoding
 - classification labels, 206
 - sparsity scaling, 206
- One-shot pruning
 - definition, 493
- oneDNN
 - Intel optimization library, 625
- Online inference
 - definition, 808
- Online softmax
 - incremental computation, 407
- ONNX
 - cross-framework portability, 348
 - definition, 348
 - framework portability, 348
 - hub-and-spoke interoperability, 348
 - model interchange format, 803
- ONNX Runtime
 - cross-platform serving, 759
- Open division
 - MLPerf submission, 683
- OpenCL
 - cross-processor execution, 634
- OpenVINO
 - Intel inference, 759
- Operating system
 - runtime role, 41
- Operation type
 - definition, 567
- Operational maturity
 - system integration, 826
- Operational mismatch
 - ML vs. traditional software, 774
- Operations and monitoring
 - definition, 30
- Operator fusion
 - appendix refresher, 921
 - definition, 527
 - graph compilation, 762
 - inference optimization, 79
 - kernel optimization, 759
 - latency gap elimination, 654
 - memory-bound optimization, 601
- Operator kernel
 - dispatch hierarchy, 335
 - GPU definition, 298, 743
- Optimization
 - Amdahl's law, 78
 - compile-time, constant folding, 762
 - hardware, architecture-specific, 220
 - inference, per-query cost, 867
 - performance, inference, 219
- Optimization difficulty, 238
- Optimization priorities
 - definition, 623
- Optimization priority
 - definition, 601
- Optimization stack
 - multiplicative effects, 469
- Optimization technique
 - definition, 621
- Optimized for
 - definition, 579
- Optimizer
 - Adam, 214
 - adaptive learning rate, 373
 - momentum-based methods, 373
- Optimizer state
 - appendix refresher, 942
 - memory overhead, 361
- Optum, 847
- ORC
 - columnar format, 158
- Oura Ring
 - wearable ML case study, 830
- Out-of-memory error
 - training constraint, 406
- Outlier Detection
 - data cleaning, 142
- Output-stationary
 - dataflow strategy, 579
 - partial sum accumulation, 610
- Over-the-air update
 - edge deployment, 831
 - model distribution, 806
- Overfitting
 - benchmark, over-optimization to test sets, 694
 - definition, 216
 - training challenge, 217
- Overhead-bound
 - small model training, 354
- Overparameterization
 - compression rationale, 485
 - generalization benefit, 179
- Pacing function
 - linear warmup, 445
- Padding
 - batching overhead, 770
- PagedAttention
 - memory efficiency, 751
 - memory fragmentation solution, 751
- Pandera
 - schema validation, 131
- Papermill
 - notebook parameterization, 792
- Parallel Computing
 - Amdahl's Law, 551
- Parallel fraction
 - definition, 551
- Parallel processing
 - neural networks, 205
- Parameter
 - memory multiplier, 193
 - registration, automatic tracking, 338
 - reuse, hierarchical learning, 187
- Parameter freezing
 - transfer learning, 340
- Parameter tracking
 - transformation lineage, 147
- Parameter update
 - optimizer step, 391
- Pareto Frontier
 - serving trade-offs, 735
- Pareto frontier, 233
 - accuracy-efficiency trade-off, 697
 - compression trade-off, 485, 697
 - fairness-accuracy trade-off, 848
 - multi-objective optimization, 888
 - throughput-latency, 749
- Parquet
 - columnar format, 158
 - I/O reduction, 145
- Partial derivative
 - gradient computation, 210
- Patient treatment loop, 832
- PCIe
 - bandwidth generations, 593
 - host-device interconnect, 328, 593
 - host-to-device transfers, 580
- Peak FLOPS
 - misleading metric, 636
- Peak FP16
 - definition, 597
- Perceptron
 - computational primitive, 237
 - early neural computation, 7
 - etymology, 237
 - linear separability limit, 184
 - Rosenblatt, 184
- Perceptual hashing
 - image deduplication, 444
- Performance gains
 - definition, 620
- Performance per watt
 - efficiency metric, 635
- Performance-per-data
 - metric definition, 470
- Periodic score refresh
 - active learning, 466
- Peripheral component interconnect
 - express, *see* PCIe
- Perplexity
 - etymology, 699
 - filtering, low-information text removal, 444
 - language model prediction measure, 699
 - serving interpretation, 699
- Personalized model
 - user adapters, 752
- Photoplethysmography
 - data quality challenge, 834
 - wearable sensing, 834
- Physical AI
 - real-time control constraints, 892
- Physical constraints
 - deployment implications, 40
 - memory signaling, 42
 - permanent boundaries, 79
 - speed of light, 41
 - thermodynamics, 42
- Pickle
 - loading overhead, 763
- Pile, The
 - data diversity, 440
- Pinned memory
 - definition, 328
 - DMA transfer, 330, 742
- PipeDream
 - pipeline parallelism, 365
- Pipeline
 - cascading failures, 117
 - debt, workflow fragmentation, 784
 - failure diagnosis, 164
 - failure modes, 129
 - overlapping, latency hiding, 397
- Pipeline Parallelism
 - training strategy, 672
- Pipeline parallelism
 - micro-batch scheduling, 422
 - micro-batching, 422
- Pipelined execution
 - throughput analysis, 43
- Placement considerations
 - definition, 605, 606
- Platt scaling
 - calibration, 99
- Point-in-time correctness, 160
 - time travel, 161
- Poisson distribution
 - traffic modeling, 662
- Poisson process
 - queuing model, 735
 - serving arrivals, 752
- Polysomnography
 - label uncertainty, 831
- Pooling
 - global average, 244
- Population stability index
 - appendix refresher, 912
 - credit risk origin, 814
 - definition, 133
 - drift detection, 104, 134, 810, 814
 - drift quantification, 814
- Portrait Mode
 - pipeline latency, 63
- Positive Predictive Value
 - prevalence dependence, 96
- Post-training quantization
 - calibration, 761
 - definition, 505

- Postmortem documentation
 - incident analysis, 818
- Postprocessing
 - confidence thresholds, 223
 - logits to predictions, 732
 - response formatting, 727
- Power budget
 - edge devices, 866
- Power management
 - mobile AI, 633
- Power measurement
 - evaluation triad completion, 686
- Power usage effectiveness
 - data center cooling metric, 689
- Power wall
 - frequency scaling, 42
 - mobile implications, 63
 - thermal limits, 42
- Pre-annotation
 - workflow, 151
- Precision
 - dynamic, adaptive quality, 761
 - hardware design parameter, 579
 - numerical format trade-offs, 938
 - numerical, inference
 - optimization, 220
 - positive prediction accuracy, 661
 - reduced, architectural trade-off, 570
 - throughput trade-off, 760
- Predictive entropy, 134
- Predictive maintenance
 - edge deployment, 60
 - edge duty cycle, 60
- Prefetching
 - accelerator optimization, 558
 - data loader pipeline, 333
 - double-buffering, 157
- Prefill phase
 - compute-bound, 768
 - parallel processing, 768
- Prefix caching
 - KV cache reuse, 757
- Preprocessing
 - digit segmentation, 223
 - divergence, training vs. serving, 739
 - GPU, accelerated pipelines, 729
 - image normalization, 223
 - latency impact, 727
- Pretext Task
 - self-supervised learning, 450
- Price of fairness
 - utility trade-off, 859
- Price/performance
 - definition, 582
- Primary pressure
 - definition, 588
- Primary workloads
 - definition, 581
- Privacy
 - always-listening devices, 874
 - technical protection methods, 873
 - technique hierarchy, 873
- Probabilistic engineering, 6
- Problem definition
 - ML vs. traditional, 90
 - multi-constraint optimization, 91
- ProcessGroup
 - distributed execution, 335
- Processing element
 - accelerator building block, 562
- Production monitoring
 - continuous benchmarking, 705
- Production-monitoring interface
 - feedback loop, 775
- Progressive deployment
 - model compression, 73
- Projection pushdown, 910
- Prometheus
 - monitoring trade-off, 812
- Protocol Buffers
 - serialization, 725
- Proxy metrics
 - definition, 104
- Proxy model
 - practical workflow, 442
- Proxy variable
 - bias detection, 843
 - bias reconstruction, 877
 - definition, 843
- Pruning
 - carbon reduction, 865
 - channel, 488
 - definition, 480, 486
 - dynamic, 491
 - layer, 488
 - low-information filtering, 444
 - neuron, 487
 - Optimal Brain Damage, 486
 - outlier removal, 444
 - quality, label error detection, 444
 - static, 491
 - structured, 489
 - unstructured, 489
- Pseudo-labeling
 - confirmation bias, 449
 - semi-supervised learning, 449
 - semi-supervised technique, 449
- PUE
 - definition, 47
 - efficiency overhead, 47
 - power usage effectiveness, 869
- Pure function
 - JAX, 344
 - JIT requirement, 346
- Python dispatch
 - overhead, 353
- PyTorch
 - autograd implementation, 324
 - autograd system, 317
 - eager execution default, 343
 - preprocessing parameters, 147
 - profiler, 397
 - research-first design, 343
 - tensor layout, 612
- PyTorch profiler
 - performance analysis, 655
- QKV
 - KV cache, 258
- Quality control
 - edge processing, 60
- Quality debt
 - uncorrected errors, 163
- Quantization
 - activation-aware weight (AWQ), 518
 - affine, 517
 - appendix refresher, 939
 - asymmetric, 516, 939
 - AWQ bandwidth reduction, 518
 - channelwise, 518
 - definition, 482, 504
 - energy impact, 508
 - environmental benefits, 865
 - extreme (INT4, binary), 505
 - groupwise, 518
 - inference precision, 218, 328
 - information-theoretic origin, 504
 - INT8 co-design, 551
 - INT8 compression ratio, 641
 - INT8 energy savings, 687
 - layer sensitivity, 761
 - layerwise, 518
 - reduced precision tolerance, 561
 - scale, 517
 - serving optimization, 804
 - serving precision, 760
 - sub-channelwise, 518
 - symmetric, 516, 939
 - TinyML limits, 78
 - zero-point, 517
- Quantization calibration
 - entropy method, 516
 - max method, 516
 - percentile method, 516
- Quantization granularity, *see* Quantization
- Quantization-aware training
 - accuracy preservation, 522
 - definition, 505
- Queries per second
 - inference throughput metric, 680
- Query key value, *see* Query-key-value
- Query-by-committee
 - active learning strategy, 446
- Query-key-value, 258
- Queue contention
 - production latency overhead, 707
- Queueing Theory
 - serving foundations, 770
- Queueing theory
 - serving systems, 733
 - utilization divergence, 735
- RandAugment
 - hyperparameter simplicity, 453
- Random sampling
 - minority representation cost, 853
- Random seed
 - benchmark protocol requirement, 646
- Ray
 - distributed ML, 803
- RDMA
 - multi-node scaling, 594
- Reasoning model
 - inference-time compute, 720
- Recall
 - positive case detection rate, 661
- Receptive field, 243
 - depth trade-off, 243
- Recommendation system
 - DLRM, 46
 - scalability challenge, 119
- Reconstruction attack
 - federated learning vulnerability, 874
- Recurrent neural network, *see* RNN
- Red AI
 - performance at any cost, 865
- Red teaming
 - stakeholder engagement, 849
- Redis
 - online feature serving, 159
- Reduction
 - definition, 565
- Register
 - fastest memory, 589
- Regular dense kernels
 - definition, 588
- Regularization
 - preventing overfitting, 217
- Regulatory compliance
 - AI governance, 861
 - architecture constraints, 874
 - business value, 851
 - EU AI Act, 861
- Relative capacity
 - definition, 589
- Relative latency
 - definition, 589
- Reliability
 - circuit breakers, 135
 - dead letter queues, 135
 - error handling, 134
 - idempotency, 143
- ReLU, 271
 - arithmetic intensity, 933
 - computational efficiency, 191
 - conditional operation overhead, 568
 - dead neuron, 271
 - dying ReLU problem, 191
 - etymology, 191
 - hardware efficiency, 191, 369
- Remote direct memory access, *see* RDMA
- Reorder computations
 - definition, 623
- Representation capacity, 238
- Reproducibility
 - benchmark requirement, 661
 - cross-platform challenge, 692
 - experiment lineage, 98
 - lineage, 147

- versioning principle, 776
- Reproducible system artifact
 - definition, 96
- Request pipelining
 - GPU utilization, 731
- Request transformation
 - tensor formats, 724
- Residual connection
 - Transformer, 263
- Residual learning, 271
- ResNet
 - human-level performance, 179
 - MLPerf reference model, 696
- ResNet-50, 234
 - activation memory, 390
 - arithmetic intensity, 236
 - batching efficiency, 743, 886
 - cloud vs. mobile, 48
 - compute-bound workload, 553
 - data selection, 436
 - data-loading requirements, 135
 - iron-law workload, 21
 - latency budget, 728
 - module hierarchy, 338
 - roofline analysis, 649
 - systems characteristics, 45
 - training memory footprint, 318
 - weight reuse mapping, 620
 - workflow constraints, 87
- Resource isolation
 - serving, 723
- Resource utilization
 - compute and memory, 672
- Responsibility gap
 - definition, 843
- Responsible AI, 30
- Responsible AI engineering
 - definition, 850
- Responsible engineering
 - checklist methodology, 852
 - safety control loop, 842
- Retraining, 18
 - continuous updates, 794
 - decision framework, 794
 - economics, cost-benefit
 - optimization, 795
 - scheduled updates, 794
 - scheduled vs. triggered, 794
 - triggered updates, 794
- Retry logic
 - exponential backoff, 134
- Return on compute
 - definition, 21
- Return on investment
 - amortized, 464
 - data selection framework, 463
 - ML systems, 867
 - ML systems, heuristic baseline, 867
- Reweightings
 - bias mitigation, 858
 - importance sampling, 858
- RFM Analysis
 - feature engineering, 143
- Ridge Point
 - hardware balance, 584
 - hardware comparison, 597
- Ridge point
 - appendix refresher, 933, 941
 - definition, 19
 - machine balance ratio, 227
 - precision dependence, 381
- Right to erasure
 - data lineage requirement, 874
- Ring AllReduce
 - bandwidth optimization, 422
- RISC-V
 - AI customization, 564
 - vector extensions, 564
- Risk mitigation
 - responsible engineering value, 850
- RMSNorm, 273
 - efficiency, 273
 - LLM adoption, 273
- RMSprop
 - adaptive step sizes, 374
 - origin and Adam inheritance, 374
- RNN
 - definition, 252
 - sequential dependency
 - bottleneck, 256
- Role-based access control
 - data governance, 872
- Rollback
 - delayed, 801
 - immediate, 801
 - ML state mismatch, 104
 - model state management, 801
 - rapid, 801
 - strategies, safety mechanisms, 801
- Roofline Analysis
 - conv2d layer, 599
 - dense layer, 600
 - layer normalization, 601
- Roofline Model
 - bottleneck analysis, 48
 - efficiency measurement, 596
- Roofline model
 - appendix refresher, 932
 - compute vs. memory bound, 356
 - definition, 236
 - etymology and origin, 649
 - hardware diagnosis, 553
 - optimization diagnostic, 596
 - training bottlenecks, 381
- Root/power unit
 - definition, 570
- Rosenblatt, Frank, 237
- Rotting asset curve
 - accuracy decay, 811
- Row-oriented storage
 - CSV, 158
- Rule-based programming
 - limitations, 176
- Rumelhart, David
 - backpropagation, 184
- Safetensors
 - secure model loading, 764
 - zero-copy loading, 764
- Safety
 - human oversight requirements, 853
 - responsible engineering, 842
- Sample complexity, 238
- Sampled softmax
 - compute efficiency, 206
- Sampling
 - temperature, top-k, top-p, 756
- Samuel, Arthur
 - machine learning, 8
- SavedModel
 - TensorFlow export format, 342
- Scalability
 - deep learning, 178
 - distributed training metric, 671
 - paradigm comparison, 68
- Scale AI, 150
- Scaling
 - attention, numerical stability, 263
 - frequency, cubic power
 - relationship, 689
 - hybrid, GPU+CPU, 766
 - infrastructure, voice assistants, 55
 - physical ceiling, 425
- Scaling efficiency
 - strong scaling definition, 672
- Scaling laws
 - appendix refresher, 907
 - data limitations, 78
 - data-compute asymmetry, 434
 - power-law origin, 434
- Scatter
 - definition, 565
- Scheduler
 - inference server, 724
- Schema contract
 - data producers, 164
- Schema debt
 - workarounds, 162
- Schema evolution
 - failures, 134
 - silent failure, 134
- Schema validation
 - data quality, 813
 - ML pipelines, 129
 - structural data checks, 813
- Schema-on-read
 - flexibility, 154
 - governance trade-off, 154
- Schema-on-write
 - traditional databases, 154
- Schmidhuber, Jürgen, 275
- Scope creep
 - model cards, 854
- Scratchpad Memory
 - accelerator design, 561
 - software-managed storage, 590
- Scratchpad memory
 - definition, 281, 589
 - explicit control, 281
 - ML advantage, 561
- Secure Enclave
 - biometric privacy, 63
- Secure enclaves
 - trusted execution environment, 873
- Security
 - access control architecture, 872
- Select kernels
 - definition, 623
- Selection dependency
 - distributed training, 465
- Selection engineering
 - definition, 457
- Selection inequality
 - definition, 458
- Selection logic
 - definition, 560
- Selective FP32 computation
 - mixed precision, 403
- Selective gradient computation, 319
- Self-attention, 262
- Self-paced learning
 - adaptive difficulty, 446
- Self-supervised learning
 - definition, 450
 - foundation model paradigm, 452
 - iron law restructuring, 450
- Semantic segmentation
 - pixel-level labels, 148
- Semi-supervised learning
 - consistency plus
 - pseudo-labeling, 449
 - definition, 448
 - labeling efficiency, 150
- Sensitivity
 - medical AI metrics, 95
- Separation of concerns
 - MLOps layers, 777
- Sequence length variability
 - batch waste, 750
- Serial fraction
 - Amdahl bottleneck, 552
- Serialization
 - definition, 910
 - overhead, 725
- Server scenario
 - Poisson-distributed traffic, 684
- Server traffic
 - Poisson process, 752
- Serverless AI
 - cold-start benchmarking, 680
- Service Level Objective
 - latency and error rates, 805
- Serving
 - cloud/data center, 721
 - edge inference, embedded
 - serving, 721
 - mobile/edge, 721
 - production deployment, 716
 - TinyML, 721
- Serving inversion
 - throughput-to-latency shift, 716

- Session affinity
 - sticky sessions, 752
- Severity classification
 - incident prioritization, 817
- Shadow deployment
 - inference cost, 800
 - production validation, 800
- Shallow learning
 - definition, 175
- Shannon, Claude
 - information theory, 206, 913
- SHAP
 - feature attribution, 860
 - inference latency, 860
- SHAP value
 - model debugging, 819
- Shapley value
 - feature attribution, 860
- Shift handoff
 - context transfer, 821
- Shuffle buffer
 - approximate random sampling, 459
- SIFT
 - variable output size, 178
- Sigmoid
 - computational cost, 189
 - etymology, 189
- Signal-to-noise engineering, 115
- Signal-to-noise ratio
 - definition, 913
- Silent degradation, 17
- Silent failure
 - detection challenges, 845
 - gradual degradation, 777
- Silicon contract
 - constraint propagation, 884
 - definition, 41
 - framework compilation, 295
 - iron law, 43
 - physical constraints, 42
- SimCLR
 - batch requirements, 451
- SIMD
 - CNN inference, 249
 - CPU inference, 763
 - CPU vectorization, 763
 - data-level parallelism, 564
 - definition, 178
 - edge deployment, 249
 - hardware execution, 571
 - vector parallelism, 220
- SIMT
 - definition, 572
- Single instruction multiple data, *see* SIMD
- Single instruction multiple thread, *see* SIMT
- Single-Machine Processing
 - scalability, 145
- Single-metric evaluation
 - fallacy, 706
- Single-node regime
 - definition, 32
- Single-node stack
 - architecture layers, 41
- Single-user traffic
 - mobile serving, 753
- SingleStream
 - sequential inference scenario, 684
- Singular value decomposition
 - rank-bandwidth trade-off, 498
- Skip connection
 - cross-architecture portability, 269
 - gradient flow, 197, 234
- SLA
 - contractual penalty, 727
 - data freshness, 137
 - definition, 677
- SLI
 - definition, 727
- Slice analysis
 - subpopulation debugging, 818
- SLO
 - capacity planning, 734
 - definition, 677
 - percentile targets, 736
 - vs. SLA, 719
 - vs. SLI vs. SLA, 727
- Snappy
 - decompression speed, 158
- Snorkel
 - weak supervision, 151
- SoC
 - always-on island, 120
- Soft affinity
 - load balancing, 752
- Softmax
 - appendix refresher, 913
 - etymology, 192
 - global normalization, 370
 - memory access pattern, 370
 - memory-bound operation, 598
 - numerical stability, 192
 - probability distribution, 191
- Softmax function
 - quadratic bottleneck, 258
 - systems, 258
- Software 1.0
 - definition, 4
- Software 2.0
 - definition, 4
- Sparse connectivity
 - connection patterns, 196
- Sparse scatter
 - definition, 45
- Sparse Tensor Core
 - 2:4 constraint, 575
- Sparsity
 - activation sparsity, 191
 - definition, 532
 - hardware gap, 532
 - hardware speedup requirement, 574
 - memory access irregularity, 588
 - N:M structured pattern, 574
 - SIMD vectorization, 533
 - structured, 2:4 hardware support, 575
- SPEC CPU
 - real application workloads, 643
- SPEC Power
 - energy efficiency, 643
 - server energy efficiency, 643
- SpecAugment
 - audio augmentation, 126
- Special Function Unit
 - activation functions, 562
 - SFU, 567
- Specificity
 - medical AI metrics, 95
- Spectrogram
 - audio features, 143
 - audio representation, 143
- Speculative decoding
 - draft model verification, 751
 - latency reduction, 751
 - throughput, 758
- Speech recognition
 - forced alignment, 152
- Speed of light check
 - utilization diagnosis, 654
- SQL, *see* Structured query language
- SQuAD
 - reading comprehension dataset, 659
 - saturation, 659
- SRAM
 - on-chip cache, 589
 - on-chip fast memory, 590
- SSD cache
 - model loading, 742
- Stable Diffusion
 - generation cost, 454
- Stacking
 - ensemble method, 95
- Stage interface specification
 - contracts, 85
- Staged rollout
 - canary, 99
 - shadow mode, 99
 - workflow, 99
- Staging validation
 - probabilistic adequacy, 800
- Stakeholder communication
 - business alignment, 823
 - technical translation, 823
- Staleness cost
 - accuracy decay model, 797
- Standardization
 - feature scaling, 142
- Static batching
 - fixed batch size, 745
- Static graph
 - define-then-run model, 302
 - definition, 308
 - optimization benefits, 303
- Static inference
 - precomputed predictions, 719
- Static pruning
 - coreset selection, 441
 - definition, 435
 - pretraining filtration, 440
- Static quantization, *see* Quantization
- Static random access memory, *see* SRAM
- Static single assignment
 - appendix refresher, 921
 - compiler IR, 305
- Stationary item
 - definition, 579
- Statistical decision theory
 - loss function origin, 201
- Statistical drift risk
 - accuracy erosion, 888
 - definition, 810
- Statistical learning
 - definition, 9
- Statistical telemetry
 - ML observability, 775
- Statistical validation
 - sampling strategies, 131
- Statistics unit
 - definition, 570
- Stochastic gradient descent
 - etymology, 372
 - gradient estimation, 372
 - memory-utilization trade-off, 372
 - mini-batch procedure, 216
 - Robbins and Monro, 372
 - systems trade-off, 5
- Storage
 - activation, backpropagation, 210
 - cold, archival, 160
 - columnar, ML training, 154
- Store-and-forward architecture
 - intermittent connectivity, 94
- Straight-through estimator
 - gradient approximation, 520
 - quantization-aware training, 520
- Strassen's algorithm
 - matrix multiplication, 367
 - practical crossover, 367
- Stratified evaluation
 - population-based testing, 855
- Stratified sampling
 - fairness evaluation, 853
- Stream ingestion
 - definition, 136
- Streaming multiprocessor
 - GPU architecture, 572
 - occupancy, 572
- Streaming response
 - LLM serving, 757
- Streaming traffic
 - sensor synchronization, 753
- Strong scaling
 - appendix refresher, 934
 - definition, 671
- Structured pruning
 - hardware alignment, 551
- Structured query language
 - feature pipeline primitives, 911
- Structured sparsity
 - N:M pattern, 489, 574
- STUDENT
 - symbolic AI failure mode, 8

- Subgroup imbalance
 - demographic disparity, 700
- Supervised learning
 - labeled examples, 200
- Supported Tensor Core precisions
 - definition, 580
- Sustainability
 - environmental responsibility, 865
- Sutton, Richard
 - bitter lesson, 11
- Switch Transformer
 - scaling trade-off, 531
- Symbolic AI
 - definition, 8
- Symmetry-aware architecture, 242
- Synchronization
 - device vs. stream, 331
- Synchronization event
 - fine-grained, 331
 - inter-stream, 331
- Synchronous scoring
 - active learning, 466
- Synthetic data
 - generation, 126
 - rare event coverage, 126
 - scalability, 126
 - simulation, 126
- Synthetic data generation
 - definition, 435
 - three-stage pipeline, 452
- Synthetic Microbenchmark
 - limitations, 647
- System Bottleneck
 - identifying, 43
- System bottleneck
 - dominant constraints, 47
- System composition, 894
- System efficiency
 - feeding tax, 114
 - hardware duty cycle, 731
- System entropy
 - definition, 76
 - model decay, 76, 795
- System is the model, The
 - definition, 884
- System-on-chip
 - energy integration, 63
 - heterogeneous AI acceleration, 631
 - mobile architecture, 63
- Systematic error
 - learned ground truth, 701
- Systems co-design, 10
- Systems thinking
 - principles in ML, 83
- Systolic Array
 - data flow architecture, 576
 - dataflow design, 576
 - energy efficiency, 576
 - TPU configuration, 576
- Systolic array
 - CNN data reuse, 277
 - definition, 558
 - etymology, 277, 576
 - TPU, 277
- t-closeness
 - privacy technique, 873
- T4
 - serving economics, 766
- Tail at scale
 - fan-out amplification, 737
- Tail latency
 - appendix refresher, 911
 - fan-out amplification, 677
 - p99 significance, 677, 891
 - revenue impact, 727
 - utilization threshold, 717
- Tanh
 - etymology, 190
 - zero-centered advantage, 190
- Task scheduling
 - definition, 628
- Task-relevant signal
 - definition, 114
- Technical debt
 - abstraction debt, missing ML interfaces, 784
 - assessment rubric, 824
 - data dependency debt, unstable inputs, 783
 - data engineering, 162
 - dead code, experimental ML paths, 784
 - definition, 780
 - ML compounding, 780
 - ML entanglement, 105
 - ML system complexity, 780
 - ML systems, 105, 863
- Technology S-Curve
 - computing paradigms, 557
- Telemetry
 - ML observability, 775
- Temperature
 - softmax distillation, 496
- Temperature scaling
 - calibration, 99
 - calibration restoration, 698
- Tensor
 - appendix refresher, 917
 - data types (dtypes), 328
 - definition, 325
 - etymology, 325
 - hardware contract, 325
 - memory layout, 175
 - metadata, 326
 - n-dimensional arrays, 175, 325
 - operations, GEMM, 336
 - stride patterns, 327
- Tensor arena
 - TinyML memory, 722
- Tensor Core
 - dimension alignment, 573
 - dimension constraints, 241
 - energy efficiency, 573
 - hardware execution, 573
 - matrix multiplication, 562
 - matrix multiply-accumulate, 403
 - mixed precision, 241
 - mixed precision acceleration, 403
 - mixed-precision acceleration, 320
 - precision support, 573
 - throughput mechanism, 573
 - training throughput, 403
- Tensor decomposition
 - definition, 499
 - Tensor-Train, 499
- Tensor layout
 - hardware alignment, 612
 - memory-aware, 608
 - NCHW vs. NHWC, 724
- Tensor memory layout
 - NCHW, 724
 - NHWC, 724
- Tensor parallelism
 - intra-layer splitting, 423
 - operation splitting, 423
- Tensor Processing Unit, *see* TPU
- TensorFlow
 - full framework, 349
 - graph-first architecture, 342
 - preprocessing parameters, 147
 - profiler, 397
 - symbolic differentiation, 324
 - tensor layout, 612
- TensorFlow Data Validation, 130
- TensorFlow Lite
 - mobile deployment, 349
- TensorFlow Lite Micro
 - fixed memory arena, 312
 - microcontroller deployment, 349
- TensorFlow Serving
 - inference server, 724
 - production deployment, 342
- TensorFlow.js
 - browser deployment, 342
- TensorRT
 - accelerator compilation, 603
 - GPU optimization, 759
 - hardware utilization, 347
 - model optimization, 803
- Ternarization
 - definition, 523
- Terraform
 - infrastructure as code, 807
- Tesla D1
 - autonomous vehicle processor, 574
- Tesler's law
 - conservation of complexity, 887
- tf.data, 146
- tf.function
 - graph compilation, 350
- TF32
 - tensor float 32, 580
- TFRecord
 - definition, 158
- Theano
 - computational graph origin, 297
- Thermal design power
 - mobile constraint, 61
- Thermal management
 - mobile constraints, 633
- Thermal throttling
 - impact on sustained perf, 654
 - intelligent migration, 633
 - mobile serving, 754
 - performance limits, 77
 - sustained workload impact, 674
- Thermal trip point
 - hardware safety, 63
- Thermal wall
 - definition, 62
 - mobile constraints, 63
- Thirteen quantitative principles
 - framework, 886
- Thread pinning
 - CPU affinity, 763
- Threshold adjustment
 - fairness mitigation, 858
- Thresholding
 - confidence cutoff, 732
- Throughput
 - accelerator parallelism, 205
 - definition, 670
 - etymology, 670
 - storage bandwidth, 154
 - training, compute vs. data, 157
 - vs. latency trade-off, 660
- Throughput ceiling, 286
- Tied request
 - latency reduction, 738
- Tiered escalation
 - incident response, 820
- Tiered retention
 - hot/warm/cold, 160
- Tiered storage
 - edge deployment, 159
 - hot warm cold architecture, 93
 - ML I/O bottleneck, 93
- Tiling
 - algorithms, matrix computation, 409
 - attention computation, 407
 - definition, 336
 - DRAM traffic reduction, 616
 - memory optimization, 608
 - memory-efficient strategies, 616
 - principle, hardware-graph bridge, 576
 - spatial, data partitioning, 618
 - temporal, 618
- Time per output token
 - decode phase, 756
 - definition, 756
- Time to first token
 - definition, 756
 - prefill phase, 756
- Time travel
 - feature stores, 161
- Time-multiplexing
 - model swapping, 742
- Time-to-accuracy
 - training benchmark primary metric, 670
- Timeline profiling
 - serving optimization, 764
- Tiny constraint

- definition, 45
- TinyML, 27
 - always-on sensing, 40
 - bare-metal serving, 721
 - cost efficiency, 64
 - definition, 65
 - device form factor, 67
 - ecosystem fragmentation, 67
 - energy efficiency, 66
 - energy gap, 65
 - medical wearables, 67
 - memory planning, 312
 - micro-runtimes, 312
 - model compression, 67
 - operational constraints, 806
 - precision agriculture, 67
 - resource constraints, 66
 - training memory constraint, 67
 - ubiquitous sensing, 64
 - wake-word detection, 67
- Token throughput
 - generation rate, 699
- Tool-first anti-pattern
 - fragmented responsibilities, 828
- Top-k
 - sampling, 756
- TOPS
 - comparability limitation, 682
- TOPS/W
 - primary design objective, 686
- torch.compile
 - hybrid execution, 307
- TorchScript
 - tracing and scripting, 303
- Total cost of data
 - formula, 462
- Total cost of ownership
 - cloud vs. edge, 53
 - definition, 867
 - deployment decisions, 53
 - deployment economics, 53
 - inference dominance, 867
 - operational costs, 867
- ToyADMoS
 - domain gap, 659
- TPU, 867
 - benchmarking considerations, 648
 - data center integration, 560
 - design trade-off, 186
 - domain-specific design, 281
 - matrix unit design, 368
 - origin, 554
 - performance per watt, 281
 - specialization trade-off, 52
 - systolic array scaling, 576
 - XLA compilation, 355
- Trace-based benchmarking
 - production replay, 647
- Traditional compiler
 - definition, 623
- Traditional runtime
 - definition, 628
- Traffic-adaptive batching
 - window tuning, 754
- Train-serve split
 - cost asymmetry, 72
 - economics, 72
- Training
 - bidirectional computation, 48
 - cloud, economics, 794
 - computational cost, 366
 - computational intensity, 361
 - cost asymmetry, 360
 - data throughput, 48
 - inference vs. training, 360
 - infrastructure evolution, 423
 - iron law of training performance, 362
 - mathematical foundations, 366
 - memory decomposition, 920
 - memory equation, 377
 - memory exhaustion threshold, 424
 - memory management, 380
 - memory pressure, 361
 - multi-GPU configuration, 419
 - scaling strategies, 418
 - scaling threshold, 424
 - step, anatomy, 352
 - supervised learning, 200
 - systems, 360
 - time estimation, 935
 - transformer, interconnect
 - sensitivity, 366
 - wall-clock time limit, 425
 - weight optimization, 214
 - weight update, 391
- Training bottleneck
 - compute-bound vs.
 - memory-bound, 393
 - min-of-three relationship, 386
 - profiling, 393
- Training cost
 - electricity, 942
 - scaling trajectory, 360
- Training cost asymmetry
 - definition, 360
- Training loop
 - iterative learning, 216
- Training memory
 - Adam storage model, 930
- Training optimization
 - technique selection, 396
- Training pipeline
 - architecture, 383
 - automated ML workflows, 792
 - data flow, 383
 - staged system workflow, 361
- Training profiler
 - bottleneck analysis, 395
 - timeline visualization, 395
- Training systems
 - definition, 30, 360
- Training vs. Inference
 - accelerator design, 559
- Training vs. inference
 - operational differences, 218
 - resource differences, 218
- Training-Serving Consistency, 141
- Training-serving divide
 - definition, 29
- Training-serving skew
 - accuracy impact, 787, 888
 - causes, 131
 - definition, 18, 739
 - diagnosis and prevention, 788
 - serving accuracy fallacy, 769
 - silent accuracy degradation, 739, 788
- Transfer learning
 - compute efficiency, 95
 - definition, 95
 - labeling alternative, 150
- Transform and lighting
 - hardware acceleration, 555
- Transform tensor layouts
 - definition, 623
- Transformation versioning
 - reproducibility, 147
- Transformer
 - architectural shift, 257
 - definition, 261
 - etymology, 257
 - KV-cache memory pressure, 621
 - training evolution, 365
- Transistor tax
 - activation functions, 192
- Translation equivariance, 243
 - systems trade-off, 243
- Translation invariance, 238
- Transparency
 - governance objective, 821
 - regulatory requirements, 860
- Triton
 - inference server, 724
- True positive rate
 - fairness computation, 856
- Trust
 - competitive differentiation, 851
- Tucker decomposition
 - definition, 499
- Turing
 - definition, 580
- Turing, Alan
 - Imitation Game, 6
- Typical operations
 - definition, 601
- Ubiquitous computing
 - etymology, 64
 - TinyML realization, 64
- Uncertainty sampling
 - active learning query strategy, 446
- Undeclared consumer
 - hidden model dependency, 783
 - Tesla case study, 785
- Unified Memory
 - single address space, 594
- Unified memory
 - mobile SoC, 632
- Uniform quantization
 - definition, 514
- Universal approximation theorem, 192, 237
 - depth vs. width, 193
 - systems consequence, 237
- Unreasonable effectiveness of data
 - systems impact, 9
- Unstructured pruning
 - hardware limitations, 535
- Update rule
 - gradient descent, 215
- Upstream echoing
 - definition, 460
- URLLC
 - edge inference constraint, 667
- Useful work per dollar
 - definition, 16
- Utilization
 - high utilization pitfall, 769
 - latency relationship, 735
- V100, 368
 - batching throughput, 743
- Validation
 - accuracy plateau, 217
- Vanishing gradient problem, 256
 - activation saturation, 190
 - exponential decay, 190
- Vector execution
 - definition, 581
- Vector index
 - embedding-based selection, 459
- Vector Operation
 - hardware acceleration, 564
- Vector operation
 - definition, 565
- Vector Unit
 - element-wise operations, 562
- Vector-scalar broadcast
 - definition, 565
- Vectorization
 - automatic batching, 344
 - columnar processing, 910
- Vendor lock-in
 - cloud deployment, 54
 - portability trade-off, 637
- Verification gap
 - definition, 6
 - MLOps closure, 774
 - statistical testing, 888
- Viola-Jones algorithm
 - compute efficiency, 10
- Virtuous cycle
 - definition, 449
- Vision transformer, 275
 - compute trade-off, 275
- ViT, *see* Vision transformer
- vLLM
 - KV-cache pressure, 809
 - virtual memory analogy, 750
- Voice assistant
 - hybrid architecture, 54
- Voice Match
 - enrollment pipeline, 119
- Volta

- definition, 580
- Von Neumann bottleneck
 - ML accelerator constraint, 583
- VRAM
 - device memory, 742
- Wafer-scale integration
 - single-die approach, 574
- Wake Vision
 - smart doorbell, 15
- Wake-word detection
 - always-on power budget, 68
 - layered architecture, 54
- Wald, Abraham
 - statistical decision theory, 201
- Warehouse-scale computing
 - training evolution, 423
- Warmup
 - cache population, 740
 - cold start mitigation, 740
- Warmup runs
 - benchmarking methodology, 707
- Warp
 - divergence penalty, 572
 - etymology and hardware
 - constraint, 389
 - execution unit, 572
 - GPU execution unit, 389
- Waterfall model
 - contrast with ML, 85
- Wave quantization
 - accelerator utilization, 388
 - tail effects, 388
- Waymo
 - hybrid deployment, 28
- Weak scaling
 - appendix refresher, 934
- definition, 672
- Weak supervision
 - cost trade-off, 151
 - programmatic labeling, 151
- Web scraping
 - ML training data, 124
- WebDataset
 - sequential sharding, 158
- Weight loading
 - cold start, 740
 - memory efficiency, 743
- Weight matrix
 - layer connections, 193
- Weight Reuse
 - spatial, 620
- Weight sharing, 242
- Weight size
 - definition, 596
- Weight update
 - optimization, 214
- Weight-stationary
 - CNN optimization, 609
 - dataflow strategy, 579
- Weights
 - learnable parameters, 182
 - matrix organization, 193
- Werbos, Paul
 - backpropagation, 184
- Whetstone
 - etymology, 643
 - synthetic floating-point
 - benchmark, 642
- Wildlife conservation
 - TinyML monitoring, 67
- WILDS
 - distribution shift benchmark, 701
- Williams, Ronald J.
 - backpropagation, 184
- Winograd algorithm
 - convolution, 284
 - precision trade-off, 284
- Workflow
 - lab-deployment gap, 82
- Workflow orchestration
 - pipeline automation, 787
- Working set size, 279
- Workload archetype
 - definition, 45
- Workload distribution
 - heterogeneous processor, 632
- Workload imbalance
 - definition, 604
- Worst-Case Execution Time
 - edge AI, 806
- XLA
 - accelerator compilation, 603
 - compilation speedup, 303
 - definition, 303
 - graph compilation, 303
 - warm-up compilation, 804
- XOR problem
 - depth requirement, 195
 - nonlinearity requirement, 195
- YOLOv8-nano
 - edge inference sizing, 59
- Zillow
 - correction cascade, 785
 - distribution shift failure, 863
- Zillow Offers
 - forecast risk, 76